

=====

MODERN AGE CODERS - COMPLETE KNOWLEDGE BASE FOR AI AND WHATSAPP AGENTS

=====

PLAIN TEXT EDITION. Version 2.0. Last updated 4 August 2026.
Source of truth: generated directly from the live website codebase.
Website: <https://learn.modernagecoders.com>

This file is written for an AI agent, a WhatsApp bot or a support person answering parents and students. Read the rules in PART 1 before answering anything. Every course in PART 6 includes its own link, its full week by week curriculum and the projects the student builds, so a parent can be sent to the page that proves what you told them.

=====

PART 1. RULES YOU MUST FOLLOW BEFORE ANSWERING

=====

RULE 1. HOW TO TALK ABOUT PRICE

Work out where the person is first, then give only the price that applies to them. Never read out two different price lists. Never say "in India it is this and outside India it is that". Never compare our regions to each other. The person asked what it costs for them, so answer only that, in their currency, and stop.

- If the person is in India, quote in rupees.
- If the person is anywhere else, quote in US dollars.
- If you genuinely cannot tell, ask one short question: "Are you in India or outside India?"
- Never volunteer a price the person did not ask about.
- Never explain the pricing structure, the regions, or why prices differ.
- Give the number, say what is included, then offer the free demo class.

RULE 2. BATCH TIMINGS ARE NEVER YOURS TO GIVE

Never invent a class time, a day or a slot. We run many batches across weekday evenings, weekend mornings and afternoons, and separate slots for other time zones. Say this, then hand over: "To fix the exact slot that suits you, please message Shivam Sir on +91 9123366161. He allocates every batch personally."

RULE 3. THE FREE DEMO IS ALWAYS THE NEXT STEP

Every conversation should end with the free demo offer. It is a full live class with a real teacher, no card details are ever asked for, and the parent is welcome to sit in. To book it collect: student name, age or class, subject wanted, and a WhatsApp number with country code.

RULE 4. NEVER PROMISE THESE

- A specific teacher by name.
- A specific class date or time before Shivam Sir confirms it.
- A job, a placement, a salary or a guaranteed exam score.
- A discount you have not confirmed.
- A batch size smaller than the published cap of 10 for group classes.
- A refund outside the written policy in PART 8.

RULE 5. NEVER ASK FOR

- Aadhaar, PAN or any identity document over chat.
- Full card numbers, CVV or OTP. Payments happen only through secure links.

RULE 6. WHEN YOU DO NOT KNOW

Do not guess and do not invent a course we do not run. Say: "Let me get you the exact answer on that. Please message Shivam Sir on +91 9123366161 and he will help you personally." Hand over for: custom or corporate quotes, school partnerships, payment in a currency other than rupees or dollars, refund disputes, and anything not in this file.

RULE 7. TONE

- Warm, plain and parent friendly. Short messages. Parents are busy.
- Use Sir or Ma'am with Indian customers. Reply in Hindi if they write in Hindi.
- Lead with how we teach, not with the price. Price is the last thing you mention.
- Never criticise another institute. State what we do and stop.
- Never use a dash to join clauses. Use a comma or a full stop.

=====

PART 2. WHO WE ARE

=====

Name: Modern Age Coders
Legal name: Modern Age Coders Education

Founded: 2020

Founder and Director: Shivam Khemka, called Shivam Sir

What we do: we teach live online coding and mathematics, and we build software for businesses.

Students taught: 10,000+

Rating: 4.9 out of 5, across 547 Google reviews

Countries we teach in: 25+

Ages we teach: 6 to 67

Languages: English and Hindi

Website: <https://learn.modernagecoders.com>

CONTACT

Phone and WhatsApp: +91 9123366161 (Shivam Sir)

Email: contact@modernagecoders.com

Phone hours: Monday to Saturday, 9:00 AM to 6:00 PM IST. Sunday closed.

Callback: leave a number on the website and we call back within 3 hours.

Contact form reply: within 48 hours on working days.

Registered office: 1B, JK Ambika Tower, Kolkata, West Bengal 700002, India.

Note on the office. All teaching is online. Students never travel anywhere. The Kolkata address is only our registered office, there is no campus to visit.

OUR PROMISE TO EVERY FAMILY

- A free demo class before you pay anything.
- Monthly billing. Cancel any month. No six or twelve month packages.
- No registration fee and no hidden charges. The price quoted is the whole price.
- Small live batches. Group is capped at 10 students, mini batch is 3 to 4, and one to one is one student.
- You keep everything you build. Projects, code and certificates stay yours.
- A real person on WhatsApp at +91 9123366161, not a ticket queue.

PART 3. HOW CLASSES WORK

- Every class is live with a real teacher. Nothing is pre recorded.
- Two classes a week, one hour each, which is eight classes a month.
- Every class is recorded, so a missed class can be watched later.
- Students need a computer (Windows, Mac or Chromebook) and steady internet.
- Most courses run inside the browser, so nothing needs installing.
- A webcam and microphone are recommended. For maths, a notebook and pen.
- Beginner courses usually run 3 to 6 months. Advanced ones run 6 to 9 months.
- Students can join mid month. We either place them in a running batch or start a new one.
- Classes can be paused for exams or holidays with about a week of notice.

BATCH SIZES

Group classes: capped at 10 students.

Mini batch: 3 to 4 students, close to one to one attention.

One to one: one student with one dedicated mentor.

Custom group for a school or a friend group: 3 to 10 students.

WHAT EVERY PLAN INCLUDES

- Two live classes every week, one hour each.
- Recordings of every class.
- Doubt clearing support.
- Project based learning, real builds rather than theory.
- A certificate when the course is completed.
- Progress updates for parents.

PART 4. PRICES

Read RULE 1 again before you quote anything. Give only the price for the person you are talking to. Do not read out both lists.

FOR A STUDENT IN INDIA, PER MONTH

Coding, and every subject except mathematics:

Group class: Rs 1,499

Mini batch: Rs 2,999

One to one: Rs 4,999

Mathematics:

Group class: Rs 1,499

Mini batch: Rs 2,999

One to one: Rs 4,999

The two AI coding agent courses (Codex and Claude Code):

Group class: Rs 1,499

Mini batch: Rs 2,999

One to one: Rs 4,999

FOR A STUDENT OUTSIDE INDIA, PER MONTH

Group class: \$100

One to one: \$150

This applies to every subject including mathematics and the AI agent courses.

Mini batch is not offered outside India. If asked, simply offer group or one to one.

We can also bill in EUR, GBP, CAD, AUD, SGD and AED. For those, hand over to Shivam Sir.

HOLIDAY CAMPS. This is a single payment, not a monthly fee.

In India: Rs 9,999 one time

Outside India: \$200 one time

IN SCHOOL BOOTCAMP PROGRAMMES, India only, per month

Group: Rs 1,499

One to one: Rs 2,999

CUSTOM AND CORPORATE TRAINING

Priced on request, never fixed. Used for company teams, school partnerships, bulk enrolments and specialised technology. Take the requirement and hand over to Shivam Sir.

PAYMENT

- India pays in rupees. Everywhere else pays in US dollars.
- We accept UPI, bank transfer and card, through secure payment links only.
- Billing is monthly and recurring. Cancel any month.
- Instalments are available on many courses. Confirm with Shivam Sir.
- Discounts for paying quarterly or yearly, and sometimes for siblings. Confirm before promising.

FIGURES THAT ARE NO LONGER OUR PRICES. Never quote these.

Rs 2,499

\$40

\$149.99

\$374.99

Any lifetime plan or one time payment for a monthly course.

=====

PART 5. QUICK COURSE INDEX (118 courses)

=====

Full detail for each course, including the week by week curriculum and the projects, is in PART

6. Send parents the link so they can read it themselves.

CODING, AI AND TECHNOLOGY (90 courses)

Advanced Git & GitHub Masterclass for Professionals

<https://learn.modernagecoders.com/courses/git-github-advanced-version-control-masterclass-for-professionals>

AI & Machine Learning for Teens: Zero to AI Expert

<https://learn.modernagecoders.com/courses/ai-ml-masterclass-teens>

AI & Machine Learning Masterclass: Zero to AI Expert

<https://learn.modernagecoders.com/courses/ai-ml-masterclass-complete-college>

AI Agents with Microsoft Copilot Studio for College Students

<https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-college-students>

AI Agents with Microsoft Copilot Studio for Teens

<https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-teens>

AI Literacy for Kids (Ages 8 to 14): Understand AI, Do Not Just Use It

<https://learn.modernagecoders.com/courses/ai-literacy-for-kids-course>

AI Tools for Teens (13+): Prompting, Creative Projects & Safe AI Use

<https://learn.modernagecoders.com/courses/teens-ai-tools-mastery-course>

AI Tools Mastery: ChatGPT to Production AI Automation

<https://learn.modernagecoders.com/courses/ai-tools-mastery-complete-college>

Algorithmic Trading & Stock Market Automation Masterclass

<https://learn.modernagecoders.com/courses/algorithmic-trading-automation-masterclass>

Algorithmic Trading Masterclass: Zero to Automated Systems

<https://learn.modernagecoders.com/courses/algorithmic-trading-masterclass-complete>

Alpha Girls: Elite Tech Mastery & Leadership for Women

<https://learn.modernagecoders.com/courses/alpha-girls-elite-tech-mastery>

AP Computer Science A: Java Programming and Full Exam Prep
<https://learn.modernagecoders.com/courses/ap-computer-science-a-java-exam-prep-course>

AP Computer Science Principles: Create Task and Exam Prep
<https://learn.modernagecoders.com/courses/ap-computer-science-principles-exam-prep-course>

App Development Course: Build Mobile & Web Apps From Zero
<https://learn.modernagecoders.com/courses/complete-app-development-masterclass-college>

Artificial Intelligence Course: Classical AI to AGI
<https://learn.modernagecoders.com/courses/artificial-intelligence-complete-masterclass-college>

Backend Development for Teens: APIs, Databases & Cloud
<https://learn.modernagecoders.com/courses/backend-coding-masterclass-for-teens>

Block-Based App Development: Build Apps With No Code
<https://learn.modernagecoders.com/courses/block-coding-app-development-masterclass>

C Programming for Beginners: Pointers, Memory & Structs
<https://learn.modernagecoders.com/courses/c-programming-course-for-college-beginners>

C++ Course for Teens Online: Games & Competitive Coding
<https://learn.modernagecoders.com/courses/cpp-programming-masterclass-for-teens>

C++ Programming Masterclass: Zero to System Programmer
<https://learn.modernagecoders.com/courses/cpp-programming-complete-masterclass-college>

Canva & AI Design Course: Zero to Professional Designer
<https://learn.modernagecoders.com/courses/canva-ai-design-masterclass>

CBSE Computational Thinking & AI for Kids (Class 3-8)
<https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-kids-classes-3-to-8>

CBSE Computational Thinking & AI for Teens (Class 9-12)
<https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-teens-classes-9-to-12-code-417-843>

Codex & Claude Code for Teens: AI Coding Agents Course
<https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-course-for-teens>

Codex & Claude Code: AI Coding Agents for Professionals
<https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-masterclass-for-adults-professionals>

Competitive Programming Course Online: Crack Codeforces
<https://learn.modernagecoders.com/courses/competitive-programming-masterclass-college>

Competitive Programming for Teens: ZCO, INOI and USACO Track
<https://learn.modernagecoders.com/courses/competitive-programming-for-teens-course>

Computer Science Class 11-12: CBSE Python & ICSE/ISC Java
<https://learn.modernagecoders.com/courses/cbse-icse-computer-science-class-11-12-python-java-complete-course>

Copilot Studio Agent Masterclass for Professionals
<https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-masterclass-for-professionals>

Creative Coding & Animation: Content Creator Course
<https://learn.modernagecoders.com/courses/creative-coding-animation-mastery>

Creative Media & Design for Teens: Photo, Video & Editing
<https://learn.modernagecoders.com/courses/creative-media-design-masterclass-teens>

Cybersecurity for Teens: Online Safety & White-Hat Defense
<https://learn.modernagecoders.com/courses/cybersecurity-course-for-teens-ethical-defensive>

Data Analysis Course: From Excel to Machine Learning
<https://learn.modernagecoders.com/courses/data-analysis-mastery-course-college>

Data and AI Analytics for Non-Programmers (Excel, Sheets, Power BI)
<https://learn.modernagecoders.com/courses/data-and-ai-analytics-for-non-programmers-course>

Data Science Course: Zero to Job-Ready Data Scientist
<https://learn.modernagecoders.com/courses/data-science-complete-masterclass-college>

Data Science for Teens: Python & Your First ML Models
<https://learn.modernagecoders.com/courses/data-science-course-for-teens-python-data>

Data Structures & Algorithms Course: Interview-Ready DSA
<https://learn.modernagecoders.com/courses/data-structures-algorithms-masterclass-college>

DSA & Problem Solving for Teens: Algorithms to Interviews
<https://learn.modernagecoders.com/courses/problem-solving-dsa-masterclass-teens>

Ethical Hacking & Cybersecurity: Zero to Pentester
<https://learn.modernagecoders.com/courses/ethical-hacking-masterclass-complete>

Flutter App Development Course: Zero to Pro Developer
<https://learn.modernagecoders.com/courses/complete-flutter-app-development-masterclass-college>

Frontend Development Course for Teens: HTML, CSS, React
<https://learn.modernagecoders.com/courses/frontend-development-masterclass-for-teens>

Full Stack Web Development for Teens: React & Node.js
<https://learn.modernagecoders.com/courses/full-stack-web-development-teens-masterclass>

Full Stack Web Development: Frontend, Backend & DevOps
<https://learn.modernagecoders.com/courses/full-stack-web-development-masterclass-college>

Game Development Course: Unity, Unreal & C++, Zero to Pro
<https://learn.modernagecoders.com/courses/game-development-complete-masterclass-college>

Game Development for Kids: Scratch, Roblox & Minecraft
<https://learn.modernagecoders.com/courses/game-development-masterclass-for-kids>

Game Development for Teens: Unity, Unreal & Real Games
<https://learn.modernagecoders.com/courses/complete-game-development-masterclass-for-teens>

Generative AI Course: LLMs, RAG & AI Agents, Zero to Pro
<https://learn.modernagecoders.com/courses/complete-generative-ai-masterclass-college>

Git & GitHub Course for College Students: Team-Ready Skills
<https://learn.modernagecoders.com/courses/git-github-version-control-course-for-college-students>

Git & GitHub Course for Teens: Version Control from Zero
<https://learn.modernagecoders.com/courses/git-github-version-control-course-for-teens>

Go (Golang) Programming: Concurrency, gRPC & Microservices
<https://learn.modernagecoders.com/courses/complete-golang-programming-masterclass-college>

Hackathon for Adults: Free Event + 12-Week Pro Prep (18+)

<https://learn.modernagecoders.com/courses/hackathon-prep-for-adults-professionals-coding-ai-innovation-course>
Hackathon for Kids: Free Event + 12-Week Prep (Ages 8-12)
<https://learn.modernagecoders.com/courses/hackathon-prep-for-kids-coding-innovation-ai-projects-course>
Hackathon for Teens: Free Event + 12-Week Prep (Ages 13-17)
<https://learn.modernagecoders.com/courses/hackathon-prep-for-teens-coding-ai-build-innovate-win-course>
HTML & CSS for Beginners: Build Real Responsive Websites
<https://learn.modernagecoders.com/courses/html-css-course-for-beginners-build-real-websites>
IB DP Computer Science (New Guide, First Assessment 2027)
<https://learn.modernagecoders.com/courses/ib-diploma-computer-science-course>
ICSE Computer Applications (Class 9-10): Java with BlueJ
<https://learn.modernagecoders.com/courses/icse-computer-applications-java-bluej-course>
IGCSE Computer Science (0478): Full Syllabus and Exam Prep
<https://learn.modernagecoders.com/courses/igcse-computer-science-0478-course>
Informatics Practices (IP) Class 11-12: CBSE Python & SQL
<https://learn.modernagecoders.com/courses/cbse-informatics-practices-ip-class-11-12-python-pandas-sql-complete-course>
Java Programming for Teens: Zero to Professional Developer
<https://learn.modernagecoders.com/courses/java-programming-masterclass-for-teens>
Java Programming: Core Java, Spring Boot & Microservices
<https://learn.modernagecoders.com/courses/complete-java-programming-masterclass-college>
JavaScript for Teens: Interactive Websites & Browser Games
<https://learn.modernagecoders.com/courses/javascript-course-for-teens-beginners-interactive-web>
Kids Coding Masterclass: Scratch, Games, Apps & AI Projects
<https://learn.modernagecoders.com/courses/kids-coding-blocks-masterclass>
Kotlin Programming: Zero to Android & Backend Developer
<https://learn.modernagecoders.com/courses/complete-kotlin-programming-masterclass-professional>
MERN Stack Course Online: Job-Ready Full-Stack Bootcamp
<https://learn.modernagecoders.com/courses/mern-stack-development-masterclass-college>
Microsoft Office & Windows for Kids: Digital Skills Course
<https://learn.modernagecoders.com/courses/microsoft-office-kids-mastery>
Minecraft Coding for Kids: From Blocks to Real Code
<https://learn.modernagecoders.com/courses/minecraft-coding-for-kids-course>
Mobile App Development for Teens: Publish to App Stores
<https://learn.modernagecoders.com/courses/complete-app-development-masterclass-for-teens>
MySQL Database Masterclass: SQL, Design & Administration
<https://learn.modernagecoders.com/courses/mysql-database-complete-masterclass-college>
MySQL for Teens: SQL Queries, Database Design & Security
<https://learn.modernagecoders.com/courses/mysql-mastery-for-teens>
Orange Data Mining Course for College Students: Machine Learning and Analytics
<https://learn.modernagecoders.com/courses/orange-data-mining-machine-learning-course-for-college-students>
Orange Data Mining Course for Teens: Machine Learning Without Code
<https://learn.modernagecoders.com/courses/orange-data-mining-visual-machine-learning-course-for-teens>
PCAP Certification Prep: Certified Associate in Python Programming
<https://learn.modernagecoders.com/courses/pcap-python-certification-prep-course>
Princess Coders: Complete Coding Course for Girls & Women
<https://learn.modernagecoders.com/courses/princess-coders-complete-coding-masterclass>
Problem-Solving & Computational Thinking Course for Kids
<https://learn.modernagecoders.com/courses/problem-solving-and-computational-thinking-for-kids>
Python & AI Automation: Machine Learning, NLP & Scripting
<https://learn.modernagecoders.com/courses/python-ai-automation-masterclass-college>
Python & AI for Kids: Coding, Games, Robotics & AI Projects
<https://learn.modernagecoders.com/courses/python-ai-kids-masterclass>
Python Programming for Teens: Zero to Advanced in 2 Years
<https://learn.modernagecoders.com/courses/python-complete-masterclass-teens>
Python Programming Masterclass: Zero to Advanced Pro
<https://learn.modernagecoders.com/courses/python-programming-masterclass-zero-to-advanced-college>
Python Web Development with Django and Flask Course
<https://learn.modernagecoders.com/courses/python-web-development-django-flask-course>
Queen Coders: 1-Year Advanced Tech Leadership for Women
<https://learn.modernagecoders.com/courses/queen-coders-advanced-tech-leadership>
React for Teens: Build Real Apps, Zero to Pro Developer
<https://learn.modernagecoders.com/courses/react-for-teens-complete-masterclass>
React.js Masterclass: Zero to Senior Frontend Developer
<https://learn.modernagecoders.com/courses/react-js-complete-masterclass-college>
Roblox Game Coding for Kids: Build Real Games in Lua
<https://learn.modernagecoders.com/courses/roblox-game-coding-for-kids-lua-course>
Scratch Programming for Kids: 3-Month Course, 50+ Projects
<https://learn.modernagecoders.com/courses/scratch-programming-complete-course>
Swift Programming Masterclass: Zero to iOS & macOS Pro
<https://learn.modernagecoders.com/courses/complete-swift-programming-masterclass-college>
TypeScript Programming: Zero to Enterprise Developer
<https://learn.modernagecoders.com/courses/complete-typescript-programming-masterclass-college>
Vibe Coding for College: Full Stack, AI & DSA (Ages 18+)
<https://learn.modernagecoders.com/courses/vibe-coding-for-college-fullstack-ai-dsa-career-course>
Vibe Coding for Kids: AI, Scratch & Game Dev (Ages 8-12)
<https://learn.modernagecoders.com/courses/vibe-coding-for-kids-beginners-ai-scratch-game-dev>
Vibe Coding for Teens: Python, Web Dev & AI (Ages 13-17)
<https://learn.modernagecoders.com/courses/vibe-coding-for-teens-python-web-ai-projects-course>
Web Development for Kids: Build Websites with HTML, CSS & AI
<https://learn.modernagecoders.com/courses/kids-ai-web-development-course>

MATHEMATICS (28 courses)

11 Plus Maths Prep: UK Grammar & Independent School Entry
<https://learn.modernagecoders.com/courses/11-plus-maths-preparation-course-uk>
A-Level Maths: Pure, Mechanics, Statistics (Edexcel/AQA)
<https://learn.modernagecoders.com/courses/a-level-maths-course-pure-mechanics-statistics>
Abacus & Mental Maths for Kids: From First Beads to Anzan
<https://learn.modernagecoders.com/courses/abacus-mental-maths-course-for-kids>
Algebra Foundations: Pre-Algebra & Algebra 1, Live Online
<https://learn.modernagecoders.com/courses/algebra-foundations-masterclass>
AP Calculus AB & BC Exam Prep, Live Online, Score 5 Goal
<https://learn.modernagecoders.com/courses/ap-calculus-exam-prep>
AP Precalculus: Units 1 to 3 and Full Exam Prep
<https://learn.modernagecoders.com/courses/ap-precalculus-exam-prep-course>
AP Statistics: Exam Prep with the Investigative Task
<https://learn.modernagecoders.com/courses/ap-statistics-maths-exam-prep-course>
Business & Finance Mathematics: Investment to Risk
<https://learn.modernagecoders.com/courses/complete-business-finance-mathematics-mastery>
CBSE Class 10 Maths Board Prep: Case Studies & Mocks
<https://learn.modernagecoders.com/courses/cbse-class-10-maths-board-exam-prep-course>
College Mathematics: Calculus to Real Analysis & Beyond
<https://learn.modernagecoders.com/courses/college-mathematics-complete-masterclass>
Complete Data & Analytics Mathematics Masterclass - Statistics to Machine Learning
<https://learn.modernagecoders.com/courses/data-analytics-mathematics-masterclass>
Early Math Foundations: Playful, Concrete Math (Ages 4-7)
<https://learn.modernagecoders.com/courses/early-math-foundations>
Elementary Mathematics for Grade 1-5: Numbers to Geometry
<https://learn.modernagecoders.com/courses/elementary-mathematics-complete-masterclass>
GCSE Maths: Foundation & Higher (9-1), AQA, Edexcel, OCR
<https://learn.modernagecoders.com/courses/gcse-mathematics-mastery>
GRE & GMAT Quantitative Prep: Live Maths Coaching
<https://learn.modernagecoders.com/courses/gre-gmat-maths-prep-course>
High School Mathematics: Algebra to Calculus (Grade 9-12)
<https://learn.modernagecoders.com/courses/complete-high-school-mathematics-mastery>
IB Maths AA & AI: SL & HL, Live Online with IA Coaching
<https://learn.modernagecoders.com/courses/ib-mathematics-aa-ai-masterclass>
IGCSE Maths: Core, Extended & Additional Mathematics Online
<https://learn.modernagecoders.com/courses/igcse-mathematics-mastery>
JEE Foundation Maths Class 8-10: Proofs & Problem Solving
<https://learn.modernagecoders.com/courses/jee-foundation-maths-course-class-8-10>
Maths Olympiad & Competition Course: AMC, AIME to IMO
<https://learn.modernagecoders.com/courses/olympiad-competition-mathematics-mastery>
Maths Through Coding: Learn Math in Python (Ages 10-15)
<https://learn.modernagecoders.com/courses/maths-through-coding>
Mental Maths for Kids: Number Sense, Not Tricks (Ages 5-10)
<https://learn.modernagecoders.com/courses/mental-maths-mastery-kids>
Middle School Mathematics Mastery: Pre-Algebra & Geometry
<https://learn.modernagecoders.com/courses/comprehensive-middle-school-mathematics-mastery>
Olympiad Mathematics: IOQM, INMO and AMC Preparation
<https://learn.modernagecoders.com/courses/olympiad-mathematics-premium-course>
PSLE Maths: Model Method, Problem Sums & MOE Syllabus
<https://learn.modernagecoders.com/courses/psle-maths-mastery>
SAT Math 800 Prep: Live Classes for the Digital SAT
<https://learn.modernagecoders.com/courses/sat-math-prep-course>
Statistics & Probability: Data to Hypothesis Testing
<https://learn.modernagecoders.com/courses/statistics-probability-maths-course>
Vedic Maths Mastery: Speed Calculation with All 16 Sutras
<https://learn.modernagecoders.com/courses/vedic-maths-course-speed-calculation-mastery>

PART 6. EVERY COURSE IN FULL DETAIL

Each course below is self contained. It repeats the course name, the link and the price so you never quote the wrong one. The week by week plan is exactly what the student is taught, and the projects are exactly what they build and keep.

COURSE 1 OF 118: 11 PLUS MATHS PREP: UK GRAMMAR & INDEPENDENT SCHOOL ENTRY

Course link: <https://learn.modernagecoders.com/courses/11-plus-maths-preparation-course-uk>

Subject area: Test Prep Mathematics

Designed for: Middle school

Who can join: UK students ages 9-11 (Years 5-6), grammar and independent school applicants

Course length: 8 months (32 weeks)

Time needed: 2 live classes/week + 2-3 hours practice, rising to timed papers in the final phase

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

11 plus maths course for UK grammar and independent school entrance: live online classes, full KS2 syllabus, GL-style papers and timed exam practice.

In one line: The maths half of the 11 plus, taught properly over eight months, so test day tests the child and not the preparation.

ABOUT THE PROGRAMME

11 plus maths does not go beyond the national curriculum; what makes it hard is that it asks Key Stage 2 mathematics at speed, in unfamiliar wording, against children who have often prepared for years. This 8-month live course covers the full ground in a teaching order that builds: number fluency first, because a child who hesitates over 7 times 8 loses time on every question that follows; then fractions, decimals and percentages, ratio and proportion, early algebra, geometry and measures, and data. The final two months turn knowledge into test performance: multi-step word problems, timing strategy, multiple-choice technique, and a run of practice papers in the style of GL Assessment, which now sets the tests for most grammar schools in England. We are honest about the landscape. CEM stopped producing its familiar paper-based 11 plus tests in 2023 and most regions that used them have moved to GL, so we do not sell CEM prep as if those regional tests still run. Families targeting independent schools that use the online ISEB Pre-Tests or their own papers get practice aligned to those formats. And we are honest about outcomes: good preparation raises a child's chances and steadies their nerves, but grammar school places are competitive and nobody can promise one.

WHAT MAKES THIS COURSE DIFFERENT

- Aligned to the tests as they are now: GL Assessment for most grammar schools, ISEB Pre-Tests for many independents, and bespoke papers such as the CSSE in Essex, with practice matched to your target schools
- No false CEM promises: CEM's paper-based regional tests ended in 2023, and our format practice reflects what children will actually sit
- Number fluency is drilled before anything else, because test timing punishes slow arithmetic more than any missing topic
- Word problems are treated as a skill of their own: reading, extracting and choosing the operation, taught explicitly rather than assumed
- Two full months of timed technique and practice papers, with a marked review after every paper and an error log per child
- Live small classes where the teacher watches the child work, plus honest monthly notes to parents about readiness

THE LEARNING PATH, STEP BY STEP

Phase 1: Number fluency: place value, all four operations mental and written, tables cold, factors, multiples, primes and sequences

Phase 2: Fractions, decimals and percentages as one connected system, then ratio, proportion and best-value reasoning

Phase 3: Early algebra, geometry, measures and data: missing numbers and simple equations, shapes, angles, area, units, charts and averages

Phase 4: Test craft: multi-step word problems, timing strategy, multiple-choice technique and a full practice-paper cycle with marked reviews

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable Year 4 maths: tables mostly known and written addition and subtraction secure. The week 1 diagnostic finds the true starting point

Target schools: Any UK grammar or independent school. Tell us the target schools and we align format practice to the tests they actually use

Equipment: Pencil and paper, a quiet place to work, and a device with stable internet for live classes. 11 plus maths papers do not allow calculators

Timing: Ideally started in Year 5, so the course finishes shortly before tests taken at the start of Year 6

WHO THIS COURSE IS FOR

Grammar school applicants: Children in Years 5 and 6 preparing for GL-style grammar school tests anywhere in England

Independent school applicants: Families targeting independent senior schools that use the ISEB Pre-Tests or their own entrance papers

Strong but slow: Children whose maths is sound but whose speed under timing gives their scores away

Wordy problem strugglers: Children who can compute but freeze when the question arrives as a paragraph

Parents who want structure: Families who would rather have a taught course with honest feedback than a pile of practice books and guesswork

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NUMBER FLUENCY (MONTHS 1-2, WEEKS 1-8)

Speed and accuracy with number come first. Every later topic, and every timed paper, leans on arithmetic that works without hesitation.

Month 1: Place Value and the Four Operations (Weeks 1-4)

Weeks 1-2: Place Value, Ordering and Mental Methods

Topics:

- Diagnostic check: where each child's number work actually stands
- Place value to millions and reading large numbers aloud
- Ordering, comparing and rounding to any power of ten
- Mental addition and subtraction strategies: partitioning, compensation, near doubles
- Column addition and subtraction with carrying and borrowing secure
- Negative numbers in context: temperature, money, number lines

Projects built:

- Speed grid week: a daily 5-minute arithmetic grid, times recorded so the child can watch their own speed climb

Practice: Daily 10-minute number work set as a routine, plus 20 mixed place value and rounding questions

Weeks 3-4: Multiplication, Division and Order of Operations

Topics:

- Times tables to 12 made automatic, with the stubborn facts targeted
- Written multiplication: two digits by two digits and beyond
- Short division and long division, including remainders in context
- Choosing between mental, jotted and written methods
- Order of operations with brackets
- Estimating first: catching wrong answers before they are written down

Projects built:

- Personal tables map: each child's slowest facts identified and drilled until they match their fastest

Practice: 25 multiplication and division questions plus daily tables sprints, errors sorted into fact errors and method errors

Assessment: Month 1 check: a 20-minute arithmetic paper, marked for accuracy first and speed second

Month 2: Number Properties and Fluency Under Time (Weeks 5-8)

Weeks 5-6: Factors, Multiples, Primes and Sequences

Topics:

- Factors and multiples, and the difference stated without hesitation
- Prime numbers to 100 and how to test for them
- Square numbers and cube numbers
- Divisibility rules for 2, 3, 4, 5, 6, 9 and 10
- Number sequences: finding the rule and the next terms
- Special sequences: triangular numbers and patterns that alternate

Projects built:

- Number detective set: 12 who-am-I number puzzles solved using properties, the kind the 11 plus loves

Practice: 24 property and sequence questions plus one puzzle set done with a parent, for fun rather than marks

Weeks 7-8: Mixed Arithmetic with the Clock Running Gently

Topics:

- Mixed four-operation problems in 11 plus wording
- Inverse operations: checking answers and finding missing digits
- Money calculations: change, totals and unit prices
- Estimation and sense checks: is this answer even possible
- Introducing gentle time pressure: untimed first, then generous, then realistic
- What to do when stuck: skip, mark and return

Projects built:

- First mini-paper: a 15-question arithmetic paper under generous timing, reviewed question by question in class

Practice: Three 15-minute mixed sets across the fortnight, each one slightly tighter on time than the last

Assessment: Phase 1 milestone: a 30-minute number paper covering both months, with a speed and accuracy report for parents

PHASE 2: FRACTIONS, DECIMALS, PERCENTAGES, RATIO AND PROPORTION (MONTHS 3-4, WEEKS 9-16)

The part-whole toolkit that 11 plus papers test relentlessly, taught as one connected system rather than three separate topics, then extended into ratio and proportion.

Month 3: Fractions, Decimals and Percentages (Weeks 9-12)

Weeks 9-10: Fractions That Make Sense

Topics:

- Equivalent fractions and simplifying to lowest terms
- Comparing and ordering fractions with different denominators
- Adding and subtracting fractions, including mixed numbers
- Fractions of amounts: two thirds of 96 without fear
- Improper fractions and mixed numbers, converting both ways
- Fraction word problems: half of a half, and shares that do not split evenly

Projects built:

- Fraction wall build: each child makes their own fraction wall and uses it to settle five comparison arguments

Practice: 28 fraction questions from equivalence to word problems, working shown even when the answer is obvious

Weeks 11-12: Decimals, Percentages and Moving Between All Three

Topics:

- Decimal place value and ordering decimals
- Multiplying and dividing by 10, 100 and 1000
- Adding, subtracting and multiplying decimals in money contexts
- Percentages as hundredths: finding 10 percent, 5 percent and 1 percent and building the rest
- Percentage of an amount, and simple percentage increase and decrease
- The conversion triangle: fractions, decimals and percentages swapped fluently

Projects built:

- Conversion pack: a self-made table of the equivalences worth knowing cold, from a half down to a twentieth

Practice: 26 decimal and percentage questions plus 10 rapid conversions daily until they are automatic

Assessment: Month 3 test: a 25-minute paper across fractions, decimals and percentages in 11 plus style

Month 4: Ratio, Proportion and Best Value (Weeks 13-16)

Weeks 13-14: Ratio and Sharing

Topics:

- Ratio notation and what 3 to 2 actually says
- Simplifying ratios, including with units
- Sharing an amount in a given ratio
- Ratio and fractions: moving between the two views
- Scaling recipes up and down
- Ratio word problems in test wording: sweets, marbles, paint mixes

Projects built:

- Recipe rescale: one real family recipe scaled for 2, 6 and 9 people, with the working presented neatly

Practice: 24 ratio questions, every sharing answer checked by adding the parts back together

Weeks 15-16: Proportion, Best Value and Scale

Topics:

- Direct proportion: the unitary method taught as a habit
- Best value: comparing pack sizes and unit prices
- Scale on maps and plans
- Speed, distance and time in simple, whole-number cases
- Proportion word problems that mix money and measures
- Spotting proportion questions hidden inside long wording

Projects built:

- Supermarket challenge: real price comparisons worked as best-value problems and written up as answers with reasons

Practice: 22 proportion and best-value questions plus a 20-minute mixed paper on all of phase 2

Assessment: Phase 2 milestone: a 30-minute paper across fractions, decimals, percentages, ratio and proportion

PHASE 3: EARLY ALGEBRA, GEOMETRY, MEASURES AND DATA (MONTHS 5-6, WEEKS 17-24)

The remaining syllabus ground: missing-number thinking and simple equations, then shapes, angles, area, units and time, and finally charts, tables and averages.

Month 5: Early Algebra and Geometry (Weeks 17-20)

Weeks 17-18: Missing Numbers and Simple Equations

Topics:

- Missing number problems as balance puzzles
- Function machines forwards and backwards
- Simple one-step and two-step equations solved by undoing
- Substituting numbers into simple rules and formulas
- Word puzzles turned into number sentences
- Think-of-a-number problems worked backwards

Projects built:

- Puzzle exchange: each child writes three think-of-a-number puzzles and solves three written by classmates

Practice: 24 missing-number and equation questions, each answer substituted back to

check itself

Weeks 19-20: Shapes, Angles and Position

Topics:

- 2D shapes and their properties: triangles by type, quadrilaterals by name
- Angle types, angles on a line, around a point and inside a triangle
- Symmetry: line and rotational
- Coordinates in all four quadrants
- Reflection and translation on a grid
- 3D shapes, faces, edges, vertices and nets

Projects built:

- Shape fact file: a one-page reference of shape and angle facts in the child's own words and drawings

Practice: 26 geometry questions with every angle answer justified by the fact used

Assessment: Month 5 test: a 25-minute paper on algebra and geometry in test style

Month 6: Measures, Time and Data (Weeks 21-24)

Weeks 21-22: Perimeter, Area, Volume and Units

Topics:

- Perimeter and area of rectangles and compound shapes
- Area of a triangle
- Volume of cuboids, counting cubes first and formula second
- Metric units and conversions: length, mass, capacity
- Reading scales and measuring instruments accurately
- Time: 24-hour clock, timetables and elapsed-time problems

Projects built:

- Room plan: the child's own room measured, drawn to scale and its floor area calculated

Practice: 24 measures questions with units written in every answer, timetable problems done against a gentle clock

Weeks 23-24: Charts, Tables and Averages

Topics:

- Reading tables, bar charts, pictograms and line graphs quickly and exactly
- Two-step chart questions: differences, totals and comparisons
- Pie charts read as fractions of a whole
- Mean, median, mode and range, and which the question is really asking for
- Simple probability language: likely, unlikely, evens, and fractions for equally likely outcomes
- The data traps: misread scales and units, and how to avoid them

Projects built:

- Family survey: a small data set collected at home, charted by hand, and turned into three test-style questions

Practice: 22 data questions plus one full mixed set covering everything taught so far

Assessment: Phase 3 milestone: a 40-minute paper across the whole syllabus, the first true full-coverage test

PHASE 4: PROBLEM SOLVING UNDER TIME AND THE PRACTICE-PAPER CYCLE (MONTHS 7-8, WEEKS 25-32)

The syllabus is done; now the test itself is the subject. Multi-step word problems, timing strategy and multiple-choice technique, then a full cycle of practice papers with marked reviews.

Month 7: Word Problems and Test Technique (Weeks 25-28)

Weeks 25-26: Multi-Step Word Problems

Topics:

- The four-step routine: read, extract, choose the operations, check
- Two-step and three-step problems across money, measures and time
- Problems that mix topics: a ratio inside a word problem inside a timetable
- Drawing the problem: bar models and quick sketches that untangle wording
- Spotting the trap answer: options built from common mistakes
- Logic-flavoured questions: working backwards and eliminating cases

Projects built:

- Problem autopsy set: 10 hard word problems dissected in writing, what was given, what was asked, what was the trap

Practice: 20 multi-step problems, each one annotated with the four-step routine before any arithmetic

Assessment: Month 7 opens with a timed baseline paper so the next four weeks of technique work can be measured against it

Weeks 27-28: Timing and Multiple-Choice Technique

Topics:

- How GL-style papers present maths: multiple choice with a separate answer sheet
- Answer-sheet discipline: keeping the row and the question in step
- Elimination: earning marks on questions you cannot fully solve
- Time budgeting: when to skip, how to mark for return, when to guess
- Estimation as a speed tool: rejecting three options in five seconds
- Staying calm mid-paper: the reset routine after a hard question

Projects built:

- Technique card: each child writes their own one-page test plan, timings, skip rules and reset routine included

Practice: Three timed 20-minute sections across the fortnight, answer sheets used every

time, technique reviewed after each

Month 8: The Practice-Paper Cycle (Weeks 29-32)

Weeks 29-30: Full Papers, Marked and Mined

Topics:

- A full GL-style maths paper under real timing
- Marked review: every miss classified as knowledge, misread or timing
- Targeted drills built from each child's error pattern
- A second full paper, with the error log guiding the warm-up
- Format variations: standard-format papers and ISEB-style online questions for children who need them
- Parent briefing: what the scores mean and what they do not

Projects built:

- Error log finalised: each child's recurring mistakes named, with the fix written next to each

Practice: One paper in class and one at home per week, home papers sat under honest exam conditions

Assessment: Weekly full papers, each marked with a per-topic breakdown shared with parents

Weeks 31-32: Final Papers and a Calm Finish

Topics:

- Two final papers under strict test conditions
- Closing the last recurring errors, not chasing new topics
- The taper: why the last week before a test should be lighter, not heavier
- Test-day practicalities: sleep, breakfast, what to expect in the room
- Confidence talk: the test samples one morning, it does not measure the child
- Course close: readiness review with each family, honest and specific

Projects built:

- Readiness pack: the child's error log, technique card and score chart bound together as their own record of the eight months

Practice: Light mixed sets only in the final week, chosen from each child's strongest topics to finish on solid ground

Assessment: Course milestone: final full paper reviewed one to one with child and parents, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A daily speed-grid routine with the child's own times charted across the course
- A personal tables map targeting each child's slowest number facts
- A self-made conversion pack for fractions, decimals and percentages
- A shape and angle fact file in the child's own words
- A real-data family survey turned into test-style questions
- A problem autopsy set dissecting hard word problems in writing
- A personal technique card with timings, skip rules and a reset routine
- A full practice-paper cycle with error log and score chart, bound as a readiness pack

Total projects: A complete child-built preparation record: speed charts, fact files, error log, technique card and marked paper series

SKILLS THE STUDENT WILL HAVE AT THE END

- Arithmetic speed and accuracy that hold up under test timing
- The full 11 plus maths syllabus: number, fractions and percentages, ratio, early algebra, geometry, measures and data
- A reliable routine for multi-step word problems
- Multiple-choice and answer-sheet technique for GL-style papers
- Timing strategy: skipping, returning and guessing intelligently
- Calm under test conditions, built through graduated practice rather than pressure

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; paper-phase sittings run longer for full timed papers

Practice: 2-3 hours weekly: a short daily number routine plus set questions, rising to weekly papers in month 8

Review: Homework marked with per-topic feedback; the error log is reviewed with the child so they own their own progress

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for both child and parents, with worked solutions for stuck problems

Progress updates: Monthly progress notes to parents with scores, the error-log summary and an honest readiness view

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside the child's readiness pack and marked paper series

WHERE THIS CAN LEAD

- Grammar school entrance tests, sat with the maths half properly prepared
- Independent school pretests and entrance papers, including ISEB-style online formats
- Key Stage 2 SATs, which draw on the same curriculum this course secures
- A confident start to secondary maths, whichever school the child joins
- Our school maths and maths-through-coding courses for children who find they enjoy the subject

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which 11 plus test formats does the course prepare for?

A: Most grammar schools in England now use tests from GL Assessment, so GL-style multiple-choice papers with separate answer sheets are our default format practice. Tell us your target schools at the start and we align the paper phase to the formats your child will actually sit. The maths syllabus underneath is the same in every case.

Q: Do you offer CEM preparation?

A: We prepare children for the tests that exist now, and the familiar paper-based CEM regional tests are not among them: CEM stopped producing them in 2023, and most areas that used them have moved to GL Assessment. CEM continues as an online product used mainly by some independent schools, and if a target school uses it we practise accordingly. What we will not do is sell CEM-branded preparation as if the old regional tests still run, because they do not.

Q: Will this course get my child into grammar school?

A: No course can honestly promise that, and you should be wary of any that does. Grammar school places are competitive, pass marks move with each year's cohort, and admission can also depend on catchment and oversubscription rules that have nothing to do with maths. What preparation genuinely does is make sure the test measures your child's ability rather than their unfamiliarity with the format. We commit to full syllabus coverage, honest monthly reporting, and a clear readiness picture before test day.

Q: When should we start preparing, and is 8 months enough?

A: Most families start in Year 5, and this course is built for that: eight months starting in the autumn or spring of Year 5 finishes shortly before tests sat at the start of Year 6. Eight months is enough for a child with sound school maths; it allows the syllabus to be taught rather than crammed, with two full months left for timed practice. Children starting later can still join, but we compress honestly and tell you what the shorter runway does and does not allow.

Q: What maths is actually on the 11 plus?

A: Nothing beyond the national curriculum, mostly content taught up to the end of Year 5, since tests are sat early in Year 6. The difficulty comes from presentation and pace: multi-step word problems, unfamiliar wording, and timing that punishes slow arithmetic. Calculators are not allowed, so everything is built for pencil and paper.

Q: Does the course cover verbal reasoning, English and non-verbal reasoning too?

A: No, this course is the maths paper only, taught deeply rather than four subjects taught thinly. Many families pair it with their own English and reasoning practice or another provider for those papers. We would rather be excellent at one quarter of the 11 plus than mediocre at all of it, and we will always tell you honestly whether your child's remaining preparation time is better spent on maths or elsewhere.

Q: How much homework is there, and will my child burn out?

A: Two to three hours a week: a short daily number routine of about ten minutes plus one or two problem sets, rising to a weekly practice paper in the final month. We taper deliberately before test day rather than piling on, because a tired nine-year-old tests worse than a fresh one.

Q: You are based in India. Can you really prepare a child for a UK exam?

A: The 11 plus maths syllabus is published, the test formats are public, and our 11 plus teachers work with UK materials, GL-style papers and UK curriculum terminology throughout. Classes are scheduled for UK afternoons and evenings, and many of our students are British families, both in the UK and expatriates preparing for a return. What matters is whether the teaching is live, small-group and honest about progress, and that is exactly what we do. Judge us at the free demo class.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/11-plus-maths-preparation-course-uk>

=====

COURSE 2 OF 118: A-LEVEL MATHS: PURE, MECHANICS, STATISTICS (EDEXCEL/AQA)

=====

Course link: <https://learn.modernagecoders.com/courses/a-level-maths-course-pure-mechanics-statistics>

Subject area: Exam Board Mathematics

Designed for: High school

Who can join: A-Level students (Years 12-13 or equivalent), Edexcel, AQA and OCR

Course length: 18 months (72 weeks)

Time needed: 2 live classes/week + 3-4 hours practice, rising to full timed papers in the final phase

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A-Level Maths course online: live small-batch classes covering Pure, Mechanics & Statistics for Edexcel, AQA & OCR, from AS foundations to timed past papers.

In one line: Two years of the hardest school maths there is, taught in order, at a pace that holds, with past papers at the end instead of panic.

ABOUT THE PROGRAMME

A-Level Maths is a linear qualification: every exam comes at the end, and the content is set nationally by the Department for Education, so Edexcel, AQA and OCR students all study the same mathematics. What differs is how each board arranges its three two-hour papers, and we adjust practice to your board when the paper phase arrives. This 18-month live course follows the natural AS then A2 structure. The first nine months cover AS content: the pure core of algebra, coordinate geometry, trigonometry, exponentials and logarithms, first calculus and vectors, followed by AS statistics with the board's large data set and AS mechanics with kinematics and Newton's laws. Months 10 to 16 cover the A2 material where most grades are won and lost: functions, sequences and series, the full trigonometric toolkit, parametric equations, advanced differentiation and integration, numerical methods, the normal distribution, moments, friction and projectiles. The final two months are given entirely to structured revision and past papers under real timing, marked against official schemes. Classes are live and small, so the teacher sees your working, not just your answers. Method marks decide grades at A-Level, and we treat how a solution is written as seriously as whether it is right.

WHAT MAKES THIS COURSE DIFFERENT

- Built around the real qualification: linear assessment, three two-hour papers, roughly two thirds pure and one third applied, with your board's paper structure driving the practice phase
- AS content is finished and consolidated with an AS-standard mock before A2 begins, so the second year is built on checked ground rather than assumed ground
- The large data set is taught from the first statistics week, because boards write exam questions that assume familiarity with it, and students who meet it in the exam hall lose easy marks
- Calculator technique is taught explicitly: statistical distributions, iteration and equation solving on the calculator are part of the syllabus, not an afterthought
- Two full months of past papers under real 2-hour timing, marked against official mark schemes, with an error clinic after every paper
- Method-mark discipline throughout: solutions are written the way examiners award marks, with working shown at every step

THE LEARNING PATH, STEP BY STEP

Phase 1: AS Pure: algebra, quadratics, coordinate geometry, binomial expansion, trigonometry, exponentials and logarithms, differentiation, integration and 2D vectors

Phase 2: AS Applied: sampling and the large data set, probability, the binomial distribution and hypothesis testing, then kinematics, variable acceleration and Newton's laws, closed by an AS-standard mock

Phase 3: A2 Pure: proof by contradiction, functions, partial fractions, sequences and series, radians and the full trig toolkit, parametrics, advanced calculus, numerical methods and 3D vectors

Phase 4: A2 Applied: conditional probability, the normal distribution and hypothesis tests for means and correlation, then resolving forces, friction, moments and projectiles

Phase 5: Revision and the past-paper cycle: structured revision by strand, then full papers under timing with marked review, matched to your board's paper structure

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: A strong GCSE or IGCSE Maths pass, ideally grade 6 or above. The week 1 diagnostic finds any gaps and we patch them inside the course

Exam board: Edexcel, AQA or OCR. The content is identical across boards; we align paper practice to yours in the final phase

Equipment: A calculator with statistical distributions and an iterative function, notebooks, and a device with stable internet for live classes

Timing: Best started at the beginning of Year 12 or its equivalent; later joiners get a catch-up plan after the diagnostic

WHO THIS COURSE IS FOR

Year 12 starters: Students beginning A-Level Maths who want the full two years taught in order, with revision built in rather than bolted on

Students without strong school support: Students whose school sets the exams but whose classroom teaching is rushed, oversized or patchy

International students: Students taking A-Levels outside the UK, at international schools or as private candidates, who need structured teaching across the time zones

Retake students: Students resitting A-Level Maths who need the gaps found honestly and closed, not the whole course replayed at the same speed

Applied strugglers: Students comfortable with pure maths who lose marks in statistics wording and mechanics diagrams, or the other way around

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: AS PURE MATHEMATICS (MONTHS 1-5, WEEKS 1-20)

The pure core that every later topic stands on: algebra done properly, coordinate geometry, trigonometry, exponentials and logarithms, and the first calculus. Roughly two thirds of the A-Level is pure mathematics, and it starts here.

Month 1: Algebra Foundations (Weeks 1-4)

Weeks 1-2: Indices, Surds and Quadratics

Topics:

- Diagnostic paper on GCSE essentials: algebra, trigonometry, graphs
- Laws of indices including negative and fractional powers
- Surds: simplifying, expanding and rationalising denominators
- Quadratic graphs, completing the square and the turning point
- Solving quadratics by factorising, formula and completing the square
- The discriminant and what it says about roots

Projects built:

- Question bank entry: 15 solved exam-style questions on indices, surds and quadratics, written to full method standard

Practice: 30 mixed problems across indices, surds and quadratics, with every quadratic solved by the method the question rewards

Weeks 3-4: Simultaneous Equations, Inequalities and First Proof

Topics:

- Simultaneous equations: linear pairs and linear with quadratic
- Where a line meets a curve, algebraically and in a sketch
- Linear and quadratic inequalities with set notation
- Representing inequality regions graphically
- Proof by deduction and by exhaustion
- Disproof by counter example, with the write-up done properly

Projects built:

- Proof portfolio opened: four short proofs written out to exam standard, checked line by line

Practice: 20 simultaneous and inequality problems plus 6 proof questions, working shown in full every time

Assessment: Month 1 test: a 40-minute paper on all algebra so far, marked with method marks the way an examiner would

Month 2: Polynomials, Graphs and Coordinate Geometry (Weeks 5-8)

Weeks 5-6: Polynomials and Curve Sketching

Topics:

- Algebraic division and the factor theorem
- Factorising and sketching cubics and quartics
- Reciprocal graphs and their asymptotes
- Graph transformations: translations and stretches, alone and read from equations
- Points of intersection between curves
- Interpreting sketches: what a question actually wants from a graph

Projects built:

- Sketching sheet: 10 polynomial and reciprocal curves sketched with intercepts, turning behaviour and asymptotes labelled

Practice: 25 problems on the factor theorem, division and transformations, each sketch drawn before any algebra is checked

Weeks 7-8: Straight Lines and Circles

Topics:

- Gradient, midpoint and distance between points
- Equations of straight lines in all three forms
- Parallel and perpendicular lines, with proofs of perpendicularity
- The equation of a circle, centre and radius by completing the square
- Tangents, chords and the perpendicular from the centre
- Modelling with straight lines: interpreting gradient and intercept in context

Projects built:

- Question bank entry: 12 solved coordinate geometry questions, including one full circle-tangent problem

Practice: 22 line and circle problems with a labelled sketch for each, plus 2 modelling questions answered in context

Assessment: Month 2 test: a 40-minute paper on polynomials and coordinate geometry with one unstructured multi-step question

Month 3: Binomial Expansion and Trigonometry (Weeks 9-12)

Weeks 9-10: The Binomial Expansion

Topics:

- Factorial notation and nCr , on paper and on the calculator
- Pascal's triangle and where the coefficients come from
- Expanding $(a + bx)^n$ for positive integer n
- Finding a specific coefficient without expanding everything
- Using expansions for numerical approximations
- The sign and power slips that cost marks, and how to catch them

Projects built:

- Question bank entry: 10 solved binomial questions, including two find-the-coefficient problems worked two ways

Practice: 20 binomial problems, each coefficient answer checked by an independent second method

Weeks 11-12: Trigonometry: Ratios, Identities and Equations

Topics:

- Sine and cosine rules and the area of a triangle
- Graphs of $\sin$, $\cos$ and $\tan$, and exact values worth memorising
- The identities $\tan x = \sin x / \cos x$ and $\sin^2 x + \cos^2 x = 1$
- Solving trig equations in degrees over a given interval
- Quadratics in $\sin$, $\cos$ or $\tan$
- Finding all solutions: the CAST diagram and graph methods compared

Projects built:

- Solution-count drill: 12 trig equations where the number of solutions in the interval is predicted before solving

Practice: 28 trig problems from triangle work to multi-step equations, every lost solution traced back to its cause

Assessment: Month 3 test: a 45-minute paper on binomial expansion and trigonometry, marked for method as well as answers

Month 4: Exponentials, Logarithms and First Calculus (Weeks 13-16)

Weeks 13-14: Exponentials and Logarithms

Topics:

- The graphs of a^x and e^x , and why e matters
- Logarithms as the inverse of exponentials, and the log laws
- Solving equations of the form $a^x = b$
- Natural logarithms and equations with e
- Exponential modelling: growth, decay and interpreting the constants
- Reducing relationships to linear form with logs, and reading data plots

Projects built:

- Modelling write-up: one exponential growth or decay problem solved and interpreted in three or four written sentences

Practice: 24 log and exponential problems including 4 modelling questions where the interpretation carries the marks

Weeks 15-16: Differentiation from First Principles

Topics:

- The gradient of a curve and differentiation from first principles
- Differentiating powers of x , sums and constants
- Tangents and normals to a curve at a point
- Increasing and decreasing functions
- Stationary points and their nature, using the second derivative
- Simple optimisation: setting up and solving maximum and minimum problems

Projects built:

- First-principles proof: the derivative of x^2 and x^3 derived and written out to exam standard

Practice: 26 differentiation problems ending with 4 optimisation questions set up from scratch

Assessment: Month 4 test: a 45-minute paper on logs, exponentials and differentiation with one modelling question

Month 5: Integration, Vectors and the AS Pure Finish (Weeks 17-20)

Weeks 17-18: Integration

Topics:

- Integration as the reverse of differentiation
- Indefinite integrals of powers of x and the constant of integration
- Finding the constant from a point on the curve
- Definite integrals and what the answer means
- Areas under curves, including regions below the axis
- Areas between a curve and a straight line

Projects built:

- Area problem set: 8 shaded-region questions where the region is sketched and decomposed before any integration

Practice: 24 integration problems, definite integrals checked for sign sense against the sketch every time

Weeks 19-20: Vectors in Two Dimensions

Topics:

- Vector notation, magnitude and direction
- Adding vectors and multiplying by scalars
- Position vectors and the vector between two points
- Distance between points using vectors
- Geometric problems: parallelograms, ratios and collinearity
- Where vectors reappear: a preview of their role in mechanics

Projects built:

- Question bank entry: 10 solved vector problems including two geometric proofs

Practice: 20 vector problems with a diagram drawn for each, plus a mixed AS Pure warm-down set of 15 questions

Assessment: Phase 1 milestone: a 90-minute AS Pure paper covering all five months, marked against an official-style scheme

----- PHASE 2: AS STATISTICS AND MECHANICS (MONTHS 6-9, WEEKS 21-36) -----

The applied third of the qualification begins: statistics with the board's large data set, probability and hypothesis testing, then mechanics from kinematics to Newton's laws. The phase closes with a full AS-standard mock.

Month 6: Sampling, Data and the Large Data Set (Weeks 21-24)

Weeks 21-22: Sampling and Data Presentation

Topics:

- Populations, samples and why sampling method matters
- Simple random, systematic, stratified, quota and opportunity sampling
- The large data set: what your board pre-releases and how exams use it, with Edexcel's weather-station data as the worked example
- Histograms, frequency polygons and cumulative frequency diagrams
- Box plots and comparing distributions
- Outliers: the standard rules and when to clean data

Projects built:

- Large data set field notes: a one-page profile of your board's data set in your own words, variables, units and quirks included

Practice: 18 data presentation problems plus a guided exploration session inside the large data set itself

Weeks 23-24: Location, Spread and Interpretation

Topics:

- Mean, median and mode from lists and grouped tables
- Quartiles, percentiles and interpolation
- Variance and standard deviation, by formula and by calculator
- Coding data and what it does to mean and spread
- Scatter diagrams, correlation and interpreting a regression line in context
- Writing statistical conclusions in sentences, not just numbers

Projects built:

- Comparison write-up: two groups from the large data set compared on location and spread, in four marked sentences

Practice: 20 location and spread problems, calculator statistics mode drilled until it is faster than the formula

Assessment: Month 6 test: a 40-minute statistics paper with one large-data-set question in board style

Month 7: Probability, the Binomial and Hypothesis Testing (Weeks 25-28)

Weeks 25-26: Probability and the Binomial Distribution

Topics:

- Venn diagrams and tree diagrams done cleanly
- Mutually exclusive and independent events, and how to test for independence
- Discrete random variables and probability distributions
- The conditions for a binomial model, checked in words
- Calculating binomial probabilities on the calculator
- Cumulative binomial probabilities and careful inequality reading

Projects built:

- Model-check drill: 8 scenarios judged binomial or not, with the failing condition named each time

Practice: 24 probability and binomial problems, inequality direction stated in words before the calculator is touched

Weeks 27-28: Hypothesis Testing with the Binomial

Topics:

- Null and alternative hypotheses, written correctly
- One-tailed and two-tailed tests and significance levels
- Critical regions and actual significance level
- Carrying out a test and stating the conclusion in context
- The wording that earns the final mark, and the wording that loses it
- Common traps: wrong tail, wrong inequality, conclusions that overclaim

Projects built:

- Question bank entry: 8 full hypothesis tests written to exam standard, conclusions in context every time

Practice: 14 complete hypothesis tests, each conclusion checked against a mark-scheme

model sentence

Assessment: Month 7 test: a 45-minute paper from probability through hypothesis testing

Month 8: Mechanics: Modelling and Motion (Weeks 29-32)

Weeks 29-30: Modelling and Kinematics

Topics:

- SI units and the standard modelling assumptions: particle, light string, smooth surface
- Displacement, velocity and acceleration as vectors
- Displacement-time and velocity-time graphs, and what areas and gradients mean
- The suvat equations: where they come from and when they apply
- Vertical motion under gravity
- Choosing a positive direction and sticking to it

Projects built:

- Graph story set: 6 motion graphs translated into written descriptions of the trip and back again

Practice: 22 kinematics problems, the suvat quantities listed and the equation chosen before any algebra

Weeks 31-32: Variable Acceleration

Topics:

- Why suvat fails when acceleration changes
- Velocity and acceleration by differentiation
- Displacement by integration, with limits or constants
- Maximum velocity and turning points of motion
- Sketching motion from its equations
- Mixed problems that decide between suvat and calculus

Projects built:

- Decision drill: 10 motion problems sorted into suvat or calculus, with the deciding clue underlined

Practice: 18 variable acceleration problems plus a mixed set of 10 where the method is not announced

Assessment: Month 8 test: a 40-minute kinematics paper mixing graphs, suvat and calculus methods

Month 9: Forces, Newton's Laws and the AS Checkpoint (Weeks 33-36)

Weeks 33-34: Forces and Newton's Laws

Topics:

- Force diagrams drawn before anything else
- Newton's laws and $F = ma$ along a line
- Weight, tension, thrust and normal reaction
- Connected particles: cars and trailers, lifts and scale pans
- Pulleys: the standard setups and their equations
- Forces written as vectors in i and j form

Projects built:

- Question bank entry: 10 solved force problems, every one opened with a complete labelled diagram

Practice: 20 forces problems, the diagram marked before the algebra is even read

Weeks 35-36: AS Consolidation and Mock

Topics:

- Structured revision across all AS pure and applied content
- Mixed problem sets that do not announce their topic
- A full AS-standard mock under real timing
- Marking workshop against an official-style scheme
- Error clinic: every miss classified as concept, method or accuracy
- The A2 map: what changes in year two and what carries over

Projects built:

- Personal error log formalised: every mock miss classified and matched to a drill

Practice: Redo every mock miss untimed with full working, then one targeted drill set on your weakest AS topic

Assessment: Phase 2 milestone: full AS-standard mock, marked and reviewed one to one

PHASE 3: A2 PURE MATHEMATICS (MONTHS 10-14, WEEKS 37-56)

The second-year pure content where most A-Level grades are decided: functions, series, the full trig toolkit, parametrics, advanced calculus, numerical methods and 3D vectors.

Month 10: Proof, Fractions and Functions (Weeks 37-40)

Weeks 37-38: Proof by Contradiction and Partial Fractions

Topics:

- Proof by contradiction: the irrationality of root 2 and the infinitude of primes
- Simplifying algebraic fractions
- Algebraic division revisited with remainders
- Partial fractions with distinct linear factors
- Partial fractions with a repeated factor
- Where partial fractions pay off later: series and integration previewed

Projects built:

- Proof portfolio extended: both classic contradiction proofs written from memory to exam standard

Practice: 18 partial fraction decompositions checked by recombining, plus 4 proof

questions

Weeks 39-40: Functions and the Modulus

Topics:

- Mappings, domain and range stated precisely
- Composite functions and order of application
- Inverse functions, their graphs and the line $y = x$
- The modulus function: graphs of $|f(x)|$ and $f(|x|)$
- Solving modulus equations and inequalities
- Combined transformations applied in the right order

Projects built:

- Question bank entry: 12 solved function questions including two full modulus problems with sketches

Practice: 24 function problems, domain and range written for every inverse found

Assessment: Month 10 test: a 45-minute paper on proof, partial fractions and functions

Month 11: Sequences, Series and Radian Trigonometry (Weeks 41-44)

Weeks 41-42: Sequences and Series

Topics:

- Arithmetic sequences and series, and the standard formulae
- Geometric sequences and series
- Sum to infinity and the convergence condition
- Sigma notation read and written fluently
- Recurrence relations: increasing, decreasing and periodic sequences
- Modelling with series: savings, loans and depreciation

Projects built:

- Modelling write-up: one savings or loan problem solved with a series and explained in plain sentences

Practice: 26 series problems including 4 modelling questions and 4 sigma notation translations

Weeks 43-44: Radians, Sectors and New Trig Functions

Topics:

- Radian measure and exact values in radians
- Arc length and sector area
- Solving trig equations in radians
- Small angle approximations and where they come from
- Secant, cosecant and cotangent: graphs and identities
- Inverse trig functions and their restricted domains

Projects built:

- Question bank entry: 10 solved radian and sector problems including one segment area question

Practice: 22 problems across radians, sectors and reciprocal trig, calculator mode checked before every set

Assessment: Month 11 test: a 45-minute paper on series and radian trigonometry

Month 12: The Full Trig Toolkit and Parametric Equations (Weeks 45-48)

Weeks 45-46: Compound Angles and the R Form

Topics:

- Compound angle formulae and where they come from
- Double angle formulae and their rearrangements
- Writing $a \sin x + b \cos x$ in R form
- Using the R form for maxima, minima and equation solving
- Proving trig identities with a clear strategy
- Choosing the right identity: a decision routine for messy equations

Projects built:

- Identity toolkit card: every identity on one page with a note on when each earns its keep

Practice: 24 identity and equation problems, each proof annotated with the identity used at every step

Weeks 47-48: Parametric Equations

Topics:

- Curves defined parametrically and why anyone bothers
- Converting between parametric and Cartesian forms
- Trig parametrics and identity-based conversion
- Sketching parametric curves and finding intersections
- Points where a parametric curve crosses the axes
- Parametric modelling questions in exam style

Projects built:

- Question bank entry: 10 solved parametric problems including two conversions each way

Practice: 18 parametric problems, every conversion checked by substituting a point

Assessment: Month 12 test: a 45-minute paper on advanced trigonometry and parametrics

Month 13: Advanced Differentiation and Integration (Weeks 49-52)

Weeks 49-50: Differentiation Methods

Topics:

- The chain, product and quotient rules, and how to spot which one
- Derivatives of trig, exponential and log functions
- Implicit differentiation
- Parametric differentiation
- Rates of change and connected rates
- Tangents and normals on harder curves

Projects built:

- Method map: one page routing any function to its differentiation method, tested against 20 unseen functions

Practice: 30 differentiation problems climbing from single-rule to mixed-rule questions

Weeks 51-52: Integration Methods

Topics:

- Standard integrals of trig and exponential functions
- Integration by substitution
- Integration by parts
- Integrating with partial fractions
- Areas between curves and the trapezium rule
- Differential equations with separable variables, solved and interpreted

Projects built:

- Question bank entry: 12 solved integrals labelled by method, including one differential equation in context

Practice: 28 integration problems, the method named in the margin before each attempt

Assessment: Month 13 test: a 60-minute calculus paper across both differentiation and integration methods

Month 14: Numerical Methods, 3D Vectors and the General Binomial (Weeks 53-56)

Weeks 53-54: Numerical Methods

Topics:

- Locating roots by change of sign, and when that argument fails
- Fixed point iteration and rearranging into iterative form
- Staircase and cobweb diagrams
- The Newton-Raphson method and its failure cases
- Iteration on the calculator, done quickly and accurately
- Numerical integration revisited and error direction from sketches

Projects built:

- Root hunt: one equation solved three ways, change of sign, iteration and Newton-Raphson, with results compared

Practice: 16 numerical methods problems with calculator iteration technique drilled to fluency

Weeks 55-56: 3D Vectors and the Binomial for Any Power

Topics:

- Vectors in three dimensions: notation, magnitude and distance
- Geometric problems in 3D
- The binomial expansion for negative and fractional indices
- Validity intervals stated every time
- Expansions built through partial fractions
- Using expansions for approximations, with accuracy discussed

Projects built:

- Question bank entry: 10 solved problems split between 3D vectors and general binomial expansions

Practice: 20 problems across 3D vectors and the general binomial, validity written before any expansion

Assessment: Phase 3 milestone: a 90-minute A2 Pure paper across all five months of this phase

----- PHASE 4: A2 STATISTICS AND MECHANICS (MONTHS 15-16, WEEKS 57-64) -----

The second-year applied content: conditional probability, the normal distribution and its hypothesis tests, then resolving forces, friction, moments and projectiles.

Month 15: The Normal Distribution and Hypothesis Tests (Weeks 57-60)

Weeks 57-58: Conditional Probability and the Normal Distribution

Topics:

- Set notation for probability and the conditional probability formula
- Conditional probability from tables, trees and Venn diagrams
- The normal distribution: shape, parameters and what they control
- Calculating normal probabilities on the calculator
- The inverse normal: finding values from probabilities
- Standardising and finding unknown mean or standard deviation

Projects built:

- Question bank entry: 10 solved normal distribution problems including two unknown-parameter questions

Practice: 22 probability and normal distribution problems, a sketch of the shaded region drawn for every normal question

Weeks 59-60: Testing Means and Correlation

Topics:

- When the normal approximates the binomial, and the continuity correction
- The distribution of the sample mean
- Hypothesis tests for the mean of a normal distribution
- Correlation coefficients and what they do and do not claim
- Hypothesis tests for zero correlation using tables of critical values
- Large data set questions in A2 style, revisited

Projects built:

- Question bank entry: 8 full hypothesis tests for means and correlation, conclusions written in context

Practice: 14 complete tests plus one large-data-set question under a 15-minute clock
Assessment: Month 15 test: a 45-minute A2 statistics paper

Month 16: Resolving Forces, Friction, Moments and Projectiles (Weeks 61-64)

Weeks 61-62: Resolving Forces and Friction

Topics:

- Resolving forces at angles into components
- Equilibrium of a particle under coplanar forces
- Motion on an inclined plane
- Friction and the coefficient of friction
- Limiting equilibrium: on the point of moving
- Connected systems revisited with angles and friction

Projects built:

- Question bank entry: 10 solved problems on inclined planes and friction, every diagram complete before the algebra

Practice: 20 resolving and friction problems, components table filled in before any equation is written

Weeks 63-64: Moments and Projectiles

Topics:

- Moments and the conditions for equilibrium of a rigid body
- Rods on supports and tilting problems
- Projectile motion: splitting velocity into components
- Time of flight, range and greatest height
- The projectile modelling assumptions and their limits
- Motion in two dimensions with vectors and calculus

Projects built:

- Question bank entry: 10 solved moments and projectile problems, including one full tilting question

Practice: 18 moments and projectile problems plus a mixed mechanics set of 10 with no topic labels

Assessment: Phase 4 milestone: a 60-minute A2 applied paper across statistics and mechanics

PHASE 5: REVISION AND THE PAST-PAPER CYCLE (MONTHS 17-18, WEEKS 65-72)

No new content, on purpose. Structured revision by strand, then full past papers under real 2-hour timing, matched to your board's paper structure and marked against official schemes.

Month 17: Structured Revision (Weeks 65-68)

Weeks 65-66: Pure Revision by Strand

Topics:

- Rapid re-teach of the weakest pure topics, driven by each student's error log
- Mixed pure problem sets that do not announce their topic
- Formula booklet fluency: what is given, what must be known
- Proof rehearsal: contradiction and identity proofs rewritten from memory
- Algebra accuracy audit: the slips that cost each student marks, named and drilled
- Show-that questions: working backwards from a given answer honestly

Projects built:

- Revision sheet set one: student-made single pages for every pure strand

Practice: One mixed pure paper of 50 marks self-timed, plus a targeted drill on your two weakest strands

Weeks 67-68: Applied Revision and Board Alignment

Topics:

- Statistics mixed sets: distributions, tests and interpretation together
- Mechanics mixed sets: forces, moments, projectiles and kinematics together
- Large data set refresh: the facts worth having cold
- Calculator technique final pass: distributions, iteration, equation solving
- Your board's paper structure: exactly which content sits in which paper for Edexcel, AQA and OCR
- Time budgeting: marks per minute across a 100-mark paper

Projects built:

- Revision sheet set two: single pages for statistics and mechanics, completing the personal revision pack

Practice: One applied sectional paper self-timed plus 20 calculator-technique drills

Assessment: Month 17 checkpoint: two sectional papers marked against official schemes, error logs updated

Month 18: The Past-Paper Cycle (Weeks 69-72)

Weeks 69-70: Papers Under Real Timing

Topics:

- A full past paper from your board in one timed 2-hour sitting
- Marking against the official scheme, method marks understood
- Error clinic one: concept gaps separated from timing and accuracy losses
- A second full paper under timing
- Question selection: order of attack and when to park a question
- The unstructured problem: making progress when no method is named

Projects built:

- Mock file opened: each paper filed with its marked script, error classification and one-line

fix per miss

Practice: One additional past paper at home under honest timing, brought marked to class

Weeks 71-72: Final Mocks and Exam Craft

Topics:

- Two final full papers under strict conditions, spaced for recovery
- Error clinic two: closing the last recurring mistakes
- Presentation final pass: notation, working, exact answers where demanded
- The last week: what to revise, what to leave alone
- Exam-day routine for a three-paper series, walked through calmly
- Course close: the mock series reviewed with each student and family

Projects built:

- Completed mock file: the full paper series with scores charted, kept by the student as evidence of readiness

Practice: Light targeted drills only, set individually from each student's final error log

Assessment: Course milestone: final full past paper marked and reviewed one to one, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A personal question bank: 10-15 solved exam-style questions per topic, built across the whole course
- A proof portfolio: every required proof type written to exam standard and rewritten from memory
- A large data set profile and comparison write-ups in the student's own words
- A calculus method map routing any function to the right technique
- A personal error log maintained from the first month and used to steer revision
- Student-made single-page revision sheets for every strand
- An AS-standard mock at month 9 and a full past-paper series in month 18
- The completed mock file: marked scripts, error classifications and a score chart

Total projects: A complete self-built revision system: question bank, proof portfolio, revision sheets, error log and marked mock file

SKILLS THE STUDENT WILL HAVE AT THE END

- The full A-Level Maths content: pure, statistics and mechanics to A2 standard
- Method-mark discipline: solutions written the way official mark schemes pay
- Calculator fluency for distributions, iteration and equation solving
- Large data set familiarity of the kind exam questions quietly assume
- Timed paper craft across three two-hour papers
- Self-review: marking your own work against a scheme and classifying your own errors

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; paper-phase sittings run longer for full timed papers

Practice: 3-4 hours weekly of problem sets and question-bank building, rising with timed papers in the final months

Review: Homework marked with method-mark commentary; error logs reviewed with the teacher monthly

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems

Progress updates: Monthly progress notes to parents with test scores and the error-log summary

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside the student's marked mock series

WHERE THIS CAN LEAD

- University degrees that ask for A-Level Maths: mathematics, engineering, physics, economics and computer science
- A stronger position for degrees that quietly reward it, from architecture to psychology
- A foundation for university admissions tests such as STEP, MAT and TMUA, which build on A-Level content
- Further Maths, for students who discover they want more
- Our statistics and programming courses, which pick up exactly where the applied content leaves off

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which exam boards does the course cover?

A: Edexcel, AQA and OCR, including OCR's MEI specification by arrangement. The content of A-Level Maths is prescribed nationally, so all boards examine the same mathematics; what differs is how the three two-hour papers are arranged and the house style of the questions.

Q: What is the difference between AS Maths and the full A-Level?

A: AS Maths is a separate, standalone qualification covering roughly the first year of content. Since the reforms, AS results do not count towards the full A-Level: the A-Level is linear, and everything rests on the papers at the end of year two. Our course follows the AS then A2 structure because it is the natural teaching order, and students who want to sit the AS exam as a standalone qualification at the end of year one are prepared for exactly that.

Q: Can you improve my predicted grade?

A: We should be straight about what predicted grades are: they are set by your school or college, not by us, and no tutor can promise to change them. What we can do is give your teachers better evidence: marked tests through the course, an AS-standard mock, and a full past-paper series with scores charted. Students who improve their actual performance tend to see predictions follow, but the honest version is that we prepare you and your school predicts you.

Q: I am retaking A-Level Maths. Does this course work for resits?

A: Yes, and retakes are one of the cases we handle best, because the course does not assume you are starting from nothing. A retake student begins with the diagnostic and their old scripts if available, we map exactly which strands cost the marks, and the plan compresses the content you already hold and expands where you lost ground. Resit students usually join mid-course into the phase that matches their gaps rather than starting at week 1.

Q: What calculator do I need, and do you teach how to use it?

A: We teach calculator technique explicitly, because binomial and normal probabilities, iteration and equation solving on the calculator are worth real marks and are consistently undertaught in schools.

Q: What is the large data set and why does it matter?

A: Each exam board pre-releases a large data set, Edexcel's is weather-station data for example, and students are expected to have worked with it during the course. Exam questions are written assuming that familiarity: knowing the variables, the units and the quirks can turn a long question into a short one.

Q: We are in the UK. How do class timings work with an India-based academy?

A: Classes are scheduled to suit the student, and UK afternoons and evenings fall comfortably inside our teaching hours. Most of our A-Level students take classes on weekday evenings or weekend mornings UK time. Classes are live and interactive in every case, and the same teacher stays with the student through the course.

Q: Can I join in Year 13, or partway through my studies?

A: Yes. The 18-month plan assumes a Year 12 start, but the course is modular enough to enter at the A2 phase or the revision phase. A Year 13 joiner sits the diagnostic, we agree which AS strands need patching, and the plan is compressed to fit the months remaining before the exams. What we will not do is pretend a student ten weeks from their papers can cover two years of content; if the timeline is unrealistic we say so at the demo stage.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/a-level-maths-course-pure-mechanics-statistics>

=====

COURSE 3 OF 118: ABACUS & MENTAL MATHS FOR KIDS: FROM FIRST BEADS TO ANZAN

=====

Course link: <https://learn.modernagecoders.com/courses/abacus-mental-maths-course-for-kids>

Subject area: Mental Maths for Kids

Designed for: Elementary school

Who can join: Ages 5 to 10, no prior maths needed beyond counting

Course length: 8 months (32 weeks)

Time needed: 2 live classes/week + 10-15 minutes of bead practice daily

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Abacus classes for kids ages 5-10: live online mental maths course from first beads to anzan visualisation and mental-only sums. Small batches, free demo.

In one line: First the beads move on the desk. Then they move in the head.

ABOUT THE PROGRAMME

The abacus is one of the oldest maths teaching tools still in serious use, and it earns its place: a child who adds on an abacus is not memorising answers, they are physically moving quantities and watching place value work. This 8-month live course takes children aged 5 to 10 from holding the abacus correctly to anzan, the stage where the abacus is imagined and sums are done entirely in the head. The progression is deliberate and unhurried. Months 1 and 2 build bead fluency and the small-friend pairs that make 5. Months 3 and 4 cross the tens with big friends and take numbers to three digits. Months 5 and 6 begin visualisation: eyes closed, imaginary beads, short mental rows that grow week by week. Months 7 and 8 add multiplication on the abacus and finish with a mental-only graded test the child can be genuinely proud of. We are straightforward about what this course builds: real arithmetic fluency, comfortable number sense, and the habit of focused daily practice. Those are worth having on their own. We do not dress them up as brain development or IQ gains, because those claims are not ours to make.

WHAT MAKES THIS COURSE DIFFERENT

- A real progression to anzan: visualisation is trained deliberately from month 5, not left to chance after a year of bead drills
- Small-friend and big-friend rules are taught as number bonds, so the abacus work strengthens school maths instead of sitting beside it
- Live classes built for young attention spans: short activity blocks, bead races, dictation games, and a teacher who can see every child's abacus on camera
- Daily practice is 10-15 minutes by design, small enough that families actually keep it up for eight months
- Honest milestones: parents see dictation scores and row lengths, not vague level names
- Class pace grouped by age, because a 5-year-old and a 9-year-old need different speeds through the same material

THE LEARNING PATH, STEP BY STEP

Phase 1: Bead basics and number sense: setting and reading numbers, place value on the rods, and the small friends that make 5

Phase 2: Crossing the tens: big friends, borrowing and carrying, three-digit numbers, and dictation drills at growing speed

Phase 3: Anzan begins: imaginary beads, short mental rows, flash number practice, and listening sums done without touching the abacus

Phase 4: Multiplication on the abacus and the mental-only capstone: longer mental rows and a graded test done entirely in the head

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Counting to 10 comfortably. Everything else, including number bonds, is built in the course

Age: 5 to 10. Batches are grouped by age; the youngest children benefit from a parent nearby during class

Equipment: A student abacus, 17-rod soroban style, which costs a few hundred rupees; we send parents a buying guide before class 1

Reading: Not required. Instructions are spoken and shown; worksheets use numbers, not sentences

WHO THIS COURSE IS FOR

Young starters: Children aged 5 to 7 meeting numbers for the first time, who learn best with something to hold and move

Primary students: Children aged 8 to 10 who want arithmetic to feel easy before bigger school maths arrives

Children who dislike worksheets: Kids who switch off at written sums but light up when maths is beads, races and games

Focus builders: Children whose parents want a short, structured daily practice habit with visible progress

Future mental calculators: Kids who will enjoy the quiet magic of doing sums with an abacus nobody can see

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: BEAD BASICS AND NUMBER SENSE (MONTHS 1-2, WEEKS 1-8) -----

The abacus in hand: setting and reading numbers, place value on the rods, and the small-friend pairs that make 5. Everything later stands on the fluency built here.

Month 1: Meeting the Abacus (Weeks 1-4)

Weeks 1-2: Beads, Fingers and First Numbers

Topics:

- The parts of the abacus: frame, rods, the bar, one upper bead worth 5, four lower beads worth 1
- Holding and clearing the abacus the correct way, every time
- Finger rules: thumb pushes ones up, index finger brings them down, index works the 5-bead
- Setting and reading the numbers 1 to 9
- Bead patterns as pictures: seeing 7 as 5 and 2 without counting
- Simple bead races: who can set the called number first

Projects built:

- My abacus card: the child draws bead pictures for the numbers 1 to 9 and reads them back from the drawing

Practice: Ten minutes daily: set and read 20 called numbers between 1 and 9, parent calls, child sets, fingers checked in class

Weeks 3-4: Tens, Place Value and First Sums

Topics:

- The tens rod: setting 10 to 99 and reading two-digit numbers
- Place value felt physically: why 34 is three beads on one rod and four on another
- Direct addition: sums like $2 + 6$ where beads simply move up
- Direct subtraction: taking beads away without touching the 5-bead
- Mixed setting drills: 3, 30, 33 and hearing the difference
- Number dictation games at a gentle pace

Projects built:

- Two-digit dictation sheet: 20 numbers set from hearing alone, scored and kept as the month 1 record

Practice: Ten minutes daily: 15 direct additions and subtractions within each rod, plus 10 two-digit setting drills

Assessment: Month 1 check: set and read two-digit numbers from dictation, plus 10 direct sums, done live in class

Month 2: Small Friends, the Pairs That Make 5 (Weeks 5-8)

Weeks 5-6: Adding with Small Friends

Topics:

- The small friends: 1 and 4, 2 and 3, the pairs that make 5
- Why $3 + 4$ needs a trick when only two lower beads are free
- The small-friend move: add 5, take away the friend
- One rule at a time: $+4$ as $+5$ take 1, then $+3$, $+2$, $+1$ cases
- Saying the move aloud while doing it, so the rule settles in
- Friend flashcards: answering the make-5 partner instantly

Projects built:

- Make-5 rainbow chart: the child's own picture of the pairs that make 5, used at home during practice

Practice: Ten minutes daily: 20 small-friend additions with the rule spoken aloud, plus one round of make-5 flashcards

Weeks 7-8: Subtracting with Small Friends

Topics:

- The reverse move: take 5, give back the friend
- Subtraction cases one at a time: -4 , -3 , -2 , -1 through the 5-bead
- Mixing direct sums and small-friend sums in one row
- Rows of three numbers: $4 + 3 - 2$ worked straight through
- Accuracy first, speed second: the rule for the whole course
- Month 2 games: friend bingo and beat-the-teacher rows

Projects built:

- Row book started: a notebook of daily rows with date and score, kept for the rest of the course

Practice: Ten to fifteen minutes daily: 15 mixed small-friend sums and 5 three-number rows, logged in the row book

Assessment: Month 2 check: 20 mixed direct and small-friend sums plus three-number rows, done live with finger technique reviewed

----- PHASE 2: CROSSING THE TENS (MONTHS 3-4, WEEKS 9-16) -----

Big friends, the pairs that make 10, open up sums that cross rods: carrying, borrowing, three-digit numbers and dictation at growing speed.

Month 3: Big Friends, the Pairs That Make 10 (Weeks 9-12)

Weeks 9-10: Adding Over Ten

Topics:

- The big friends: 1 and 9, 2 and 8, 3 and 7, 4 and 6, 5 and 5
- Why $8 + 5$ cannot stay on one rod, shown with beads
- The big-friend move: add 10 on the next rod, take away the friend
- Cases in order: $+9$, $+8$, $+7$ down to $+1$ through ten
- Carrying seen physically: the tens rod clicking up
- Make-10 flashcards until the pairs are instant

Projects built:

- Make-10 rainbow chart to sit beside the make-5 chart at home

Practice: Ten to fifteen minutes daily: 20 big-friend additions with the move spoken aloud, plus make-10 flashcards

Weeks 11-12: Subtracting Across Ten

Topics:

- The reverse move: take 10, give back the friend
- Borrowing felt on the rods: why 12 - 5 reaches to the tens rod
- Subtraction cases in order: -9 through -1 across ten
- Mixed rows using direct, small-friend and big-friend moves together
- Reading the row before moving: planning which rule each number needs
- Steady dictation: teacher calls a row of four one-digit numbers

Projects built:

- Mixed-rules row sheet: 15 rows solved and marked with which friend rule each step used

Practice: Fifteen minutes daily: 15 mixed sums crossing ten and 5 dictated rows, all logged in the row book

Assessment: Month 3 check: 20 sums mixing all rules so far, plus a live dictation round of four-number rows

Month 4: Combined Moves and Three-Digit Numbers (Weeks 13-16)

Weeks 13-14: When Both Friends Work Together

Topics:

- Combined cases: sums like $6 + 7$ that need the 5-bead and the tens rod in one move
- Working the combined move slowly, then at row speed
- Two-digit plus one-digit sums: $34 + 8$ across the rods
- Two-digit plus two-digit: working rod by rod from the left
- Row stamina: five-number rows of one-digit numbers
- Speed ladders: same row sheet, three timed attempts across the week

Projects built:

- Speed ladder record: one row sheet attempted three times with times and scores charted by the child

Practice: Fifteen minutes daily: 12 combined-move sums and 6 two-digit additions, plus one speed ladder attempt

Weeks 15-16: Three Digits and Real-Life Numbers

Topics:

- The hundreds rod: setting and reading 100 to 999
- Three-digit addition and subtraction, one rod at a time
- Money sums: rupees and coins set on the abacus
- Simple word problems answered on the abacus: bought, spent, left
- Dictation with two-digit numbers in the rows
- Phase 2 wrap-up: every rule on one revision mat

Projects built:

- Shop game record: a pretend-shop worksheet of five purchases totalled and checked on the abacus

Practice: Fifteen minutes daily: 10 three-digit sums, 5 money questions and one dictated row, logged in the row book

Assessment: Month 4 milestone: mixed paper of 30 questions covering all bead rules up to three digits, done in class

PHASE 3: ANZAN, THE ABACUS IN THE HEAD (MONTHS 5-6, WEEKS 17-24)

The step this whole course points at: the physical abacus is set aside and the child moves imaginary beads. Rows start tiny and grow week by week.

Month 5: Picture the Beads (Weeks 17-20)

Weeks 17-18: First Imaginary Beads

Topics:

- What anzan is: seeing the abacus with eyes closed and moving beads that are not there
- Air abacus: fingers moving over an empty desk while the mind keeps the picture
- Setting single numbers mentally and saying them back
- First mental sums: rows of two small numbers, beads imagined
- Checking every mental answer on the real abacus straight after
- Why we slow down here: the picture must be solid before speed

Projects built:

- Eyes-closed challenge card: ten single-number settings done mentally and verified, signed off in class

Practice: Ten minutes daily: 10 two-number mental rows with immediate abacus checks, plus air-abacus warm-ups

Weeks 19-20: Flash Numbers and Longer Pictures

Topics:

- Flash practice: numbers shown briefly on screen, set mentally
- Rows of three one-digit numbers done fully in the head
- Keeping the bead picture steady when a friend rule fires
- Two-digit numbers held mentally: tens and ones rods imagined together
- Confidence rules: wrong answers are checked on beads, never scolded
- Month 5 game: slow-speed flash rounds as a class

Projects built:

- Flash round scorecard: three class flash rounds with scores tracked by the child

Practice: Ten to fifteen minutes daily: 10 three-number mental rows and one flash practice round on the family device

Assessment: Month 5 check: live mental round of three-number rows plus flash settings, real abacus used only for verifying

Month 6: Mental Rows That Grow (Weeks 21-24)

Weeks 21-22: Five-Number Rows and Listening Sums

Topics:

- Rows of four, then five one-digit numbers done mentally
- Listening anzan: the teacher speaks the row, nothing is shown
- Holding the running answer while the next number arrives
- Two-digit mental sums: one addition at a time, pictured on two rods
- Recovering a lost picture: reset, breathe, rebuild the beads
- Personal pace: row length moves up only when accuracy holds

Projects built:

- Listening log: ten dictated mental rows across the fortnight with scores and row lengths recorded

Practice: Fifteen minutes daily: 8 mental rows at your current length and 2 listening rows read out by a parent

Weeks 23-24: Level Test One, Fully Mental

Topics:

- Test craft for small people: neat answers, skipped questions returned to, no rubbing out storms
- Mixed mental paper practice: rows, flash settings, listening sums
- Two-digit rows of three numbers for the stronger picture-holders
- Class review of the phase: what got longer, what got faster
- Preparing the level test with a practice run
- Celebrating honestly: certificates say what the child can actually do

Projects built:

- Level test one: a graded mental-only paper of rows, flash and listening sums, marked and kept

Practice: Fifteen minutes daily: one full practice paper across the week, split into daily pieces, logged in the row book

Assessment: Phase 3 milestone: level test one, mental-only, with results shared with parents alongside the row book

PHASE 4: MULTIPLICATION AND THE MENTAL-ONLY CAPSTONE (MONTHS 7-8, WEEKS 25-32)

Multiplication arrives on the abacus once addition is second nature, and the course closes with longer mental rows and a graded test done entirely in the head.

Month 7: Multiplication on the Abacus (Weeks 25-28)

Weeks 25-26: Tables Meet the Beads

Topics:

- Multiplication as repeated addition, shown with bead groups first
- Tables 2 to 5 practised with bead patterns and chants
- Setting up a multiplication on the abacus: where each number lives
- Two-digit times one-digit: 23×3 worked rod by rod
- Answer sense: is 23×3 nearer 60 or 600, asked before working
- Keeping addition sharp: short mental rows as every class's warm-up

Projects built:

- Table pattern cards: bead-picture cards for the 2, 3, 4 and 5 tables made by the child

Practice: Fifteen minutes daily: one table chanted, 8 abacus multiplications and 4 mental warm-up rows

Weeks 27-28: The Harder Tables

Topics:

- Tables 6 to 9 with bead patterns, taken at each child's pace
- Two-digit times one-digit across the harder tables
- Carrying inside multiplication: when a product spills to the next rod
- Sharing sums: simple division as dealing beads into groups, kept light
- Mixed papers: additions, subtractions and multiplications together
- Month 7 dictation: rows now include a times question

Projects built:

- Mixed paper folder: three 20-question mixed papers completed, marked and filed across the fortnight

Practice: Fifteen minutes daily: 10 multiplications across tables 6 to 9 and one short mixed paper piece

Assessment: Month 7 check: 20-question mixed paper including two-digit multiplications, done live in class

Month 8: The Mental-Only Capstone (Weeks 29-32)

Weeks 29-30: Longer, Faster, Still Accurate

Topics:

- Mental rows pushed to each child's honest edge: longer or faster, not both at once
- Two-digit mental rows for ready students, one-digit mastery for the rest
- Mental one-digit multiplication for the strongest picture-holders
- Flash rounds at higher speeds, accuracy floor held at 80 percent
- Reading the row book: eight months of scores tell the child's own story
- Capstone paper structure walked through, no surprises

Projects built:

- Row book review page: the child charts their own week 8 versus week 30 row length and speed

Practice: Fifteen minutes daily: capstone-style mixed mental practice split across the week, logged in the row book

Weeks 31-32: The Graded Mental Test and Family Show

Topics:

- Final practice run of the capstone paper with review
- The capstone: a graded mental-only test of rows, flash, listening and times questions
- Family demonstration: each child performs three mental sums for the class and parents
- What the certificate says: the exact skills demonstrated, nothing inflated
- Keeping the skill: a 10-minute weekly routine for after the course
- What comes next: higher abacus levels, school maths courses, and later Vedic maths

Projects built:

- Capstone graded mental-only test, marked and reported to parents with the full row book
- Family demonstration: three mental sums performed live

Practice: Gentle final week: one short mental round daily and rest before the test, confidence over cramming

Assessment: Course milestone: capstone mental-only test and family demonstration, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- My abacus card: bead pictures for the numbers 1 to 9
- Make-5 and make-10 rainbow charts for home practice
- The row book: a daily practice log kept across all eight months
- Two-digit dictation sheets scored month by month
- Speed ladder records: the same sheet timed three times
- Shop game money worksheets totalled on the abacus
- Eyes-closed challenge card marking the start of anzan
- Flash round and listening logs through phase 3
- Table pattern cards for the 2 to 9 times tables
- The capstone graded mental-only test and family demonstration

Total projects: 10+ records and artifacts, from first bead pictures to a graded mental-only test

SKILLS THE STUDENT WILL HAVE AT THE END

- Correct bead technique: setting, reading and clearing with proper fingering
- Small-friend and big-friend rules, which are the number bonds school maths needs anyway
- Addition and subtraction to three digits on the abacus, including money sums
- Anzan: mental rows, flash numbers and listening sums with the abacus imagined
- Times tables 2 to 9 and two-digit by one-digit multiplication on the abacus
- A daily practice habit measured in a row book the child keeps themselves

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week of 45-60 minutes, run in short activity blocks suited to ages 5 to 10

Practice: 10-15 minutes of bead or mental practice daily; short and daily beats long and occasional at this age

Review: Row books and dictation sheets reviewed weekly; technique corrected live on camera before habits set

SUPPORT INCLUDED

Doubt support: WhatsApp support for parents between classes, including short videos when a finger rule needs re-showing

Progress updates: Regular progress notes to parents with dictation scores and row lengths, so improvement is visible, not asserted

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders stating the level of rows and mental work actually demonstrated

WHERE THIS CAN LEAD

- Higher abacus levels: longer rows, faster flash work and bigger multiplications
- A smoother start in school maths, with number bonds and place value already solid
- Our school mathematics courses, entered with arithmetic fluency already in place
- Vedic maths from around age 8, once tables are strong, for a second speed system
- Mental maths competitions and school arithmetic quizzes, if the child enjoys performing

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is 5 really old enough for abacus classes?

A: Yes, provided the class is built for that age, and this one is: short activity blocks, bead races instead of lectures, and a teacher watching each child's fingers on camera. A 5-year-old needs to count to 10 and sit for a game-length activity; that is the whole entry bar. For the youngest children we ask a parent to stay nearby for the first few weeks, mostly to help with camera angles and finding dropped beads.

Q: Does abacus training increase IQ or develop the brain?

A: We do not make those claims. What we can show you is what the course actually builds: arithmetic fluency, solid number bonds and place value, mental calculation through anzan, and a daily practice habit, all documented in each child's row book and dictation scores. Those are real and worth eight months. Grander claims about IQ belong to marketing, not to our classroom.

Q: Will abacus methods confuse my child at school?

A: In our experience the two reinforce each other, because the abacus friend rules are exactly the number bonds schools drill: pairs that make 5 and pairs that make 10. A child carrying on the abacus is doing the same regrouping school column methods use, just physically. We name that connection in class so children see one maths, not two competing systems.

Q: When does my child start doing sums mentally?

A: Anzan begins in month 5, after four months of physical bead fluency, and starts tiny: two-number rows with the answer checked on the real abacus. By the end of the course most children handle rows of four or five one-digit numbers mentally, and stronger picture-holders manage two-digit rows. The pace genuinely varies by child, which is why row length only increases when accuracy holds.

Q: How do live online classes work for children this young?

A: Classes run 45 to 60 minutes in short blocks: a warm-up game, a new bead rule, drills, a race or dictation round, and a calm close. Batches are capped at 10 so the teacher can watch every child's abacus and fingers on camera, and correct technique before a habit sets. Parents of the youngest children sit nearby at first; most find they are not needed within a few weeks.

Q: How much home practice is expected?

A: Ten to fifteen minutes a day, logged in a row book the child keeps. At this age short daily practice beats long weekend sessions by a wide margin, and the routine itself, sitting down for ten focused minutes, is one of the most useful things the course leaves behind. Parents mostly just call out numbers for dictation rows; no maths knowledge is needed to help.

Q: Can we try a class before enrolling?

A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Bring the child, with or without an abacus, and the teacher will tell you honestly whether they are ready now or better served starting in a few months.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/abacus-mental-maths-course-for-kids>

=====

COURSE 4 OF 118: ADVANCED GIT & GITHUB MASTERCLASS FOR PROFESSIONALS

=====

Course link: <https://learn.modernagecoders.com/courses/git-github-advanced-version-control-masterclass-for-professionals>

Subject area: Version Control & Developer Tools

Designed for: College students

Who can join: Intermediate to advanced (working professionals; basic Git assumed or fast-tracked)

Course length: 24 classes (12 weeks · 2 classes/week)

Time needed: 4-5 hours/week recommended

Certificate: Modern Age Coders 'Advanced Git & GitHub' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Advanced Git & GitHub masterclass for professionals: master Git internals, interactive rebase, trunk-based team workflows, monorepos, GitHub Actions CI/CD and repository security in 24 live online classes for working developers.

In one line: Stop fearing Git. Build the mental model, master history surgery, and become the engineer your team calls when version control goes wrong.

ABOUT THE PROGRAMME

Most working developers rely on the same handful of Git commands and quietly avoid everything else: rebase feels risky, rebase is a mystery, and a leaked secret in history triggers panic. This masterclass fixes that permanently, the way professionals need it fixed: from the inside out. In 24 live, instructor-led classes you first learn how Git actually stores data (blobs, trees, commits, refs, the three trees), because once the model is clear, nothing Git does is scary. Then you go deep into history mastery (interactive rebase, cherry-pick, bisect run, rebase rescues), team workflow design at scale (trunk-based development vs GitFlow, branch protection rulesets, CODEOWNERS, monorepo tooling, release management), and full GitHub

platform automation: GitHub Actions matrix builds and reusable workflows, gated deployments, Dependabot, secret scanning, CodeQL, signed commits, hooks, and GitHub CLI scripting. You finish with a leadership phase covering team migrations, disaster recovery with git filter-repo, and AI-assisted review workflows, then a capstone where you design and defend a complete production-grade workflow on a real repository. Git is free, open source, and created by Linus Torvalds in 2005 for Linux kernel development; two decades later, deep fluency in it remains one of the most durable skills a working developer can build.

WHAT MAKES THIS COURSE DIFFERENT

- Internals-first: you learn the object model, refs, and the three trees before any advanced command, so rebase, reset, and reflog become predictable instead of frightening.
- Built for professionals: every class is framed around real workplace scenarios, the broken main branch, the leaked API key, the 3,000-line PR, the release that must ship tonight.
- Live, interactive classes with a real instructor: you run the commands, break real repositories, and recover them under guidance, never passive video watching.
- Full platform depth, not just commands: branch protection rulesets, CODEOWNERS, merge queues, GitHub Actions, environments, OIDC deploys, CodeQL, and the gh CLI are covered as thoroughly as Git itself.
- Living Syllabus: the curriculum is updated continuously as Git and GitHub ship new features, so you always learn the current toolchain, not a frozen snapshot.
- A capstone you can apply at work on Monday: you leave with a fully hardened reference repository (workflow, CI/CD, security, release process) and the reasoning behind every choice.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac, or Linux) with a terminal and a reliable internet connection; every tool used in the course is free

Coding experience: You should already write code in some language and have touched Git (clone, add, commit, push). Rusty is fine: Classes 1-4 rebuild everything from the ground up, just at a professional pace

Accounts: A free GitHub account covers the entire course: the Free plan includes unlimited public and private repositories and 2,000 GitHub Actions minutes per month, which is more than the exercises need

Mindset: Willingness to break repositories on purpose. The course is built around controlled disasters and recoveries, because that is how the fear goes away

WHO THIS COURSE IS FOR

Working developers: Developers who use Git daily but lean on the same few commands and want the deep fluency that makes everything else routine

Team leads: Leads and managers who need to design branching, review, and release workflows for a team and enforce them with the right guardrails

Career switchers: Career switchers who already code and want the professional Git and GitHub habits that interviews and first jobs quietly assume

Devops and platform engineers: Engineers moving toward DevOps or platform roles who need GitHub Actions, deployment gating, and supply chain security in depth

Open source maintainers: Maintainers and aspiring maintainers who want clean history, scalable review, and a professional release process for their projects

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: THE GIT MENTAL MODEL -----

Learn how Git actually works underneath: the object database, refs and HEAD, the staging area, and the plumbing commands that make every advanced operation predictable instead of scary.

Class 1: How Git Actually Stores Data: The Object Model

Topics:

- Git as a content-addressable database: every file, folder, and commit is an object named by the hash of its content
- The four object types: blobs (file contents), trees (directories), commits (snapshots plus parent pointers), and annotated tags
- SHA-1 vs SHA-256 object formats: why the hash is the identity, and what the ongoing SHA-256 transition means for repositories
- Guided tour of the .git directory: objects/, refs/, HEAD, index, config, and what actually changes on disk when you commit
- Inspecting any object with plumbing: git cat-file -t <sha> and git cat-file -p <sha> to walk from a commit to its tree to a blob by hand
- Writing objects manually with git hash-object -w, and why Git stores whole snapshots, never diffs
- Loose objects vs packfiles: git gc, delta compression, and why full history costs less disk than people assume

Projects built:

- Trace one of your own commits end to end with `git cat-file`: from the commit SHA to the root tree to a specific blob, and diagram the object graph you found

Practice: Run `git cat-file -p HEAD`, then `cat-file -p` the tree it references, and keep descending until you reach a blob.

Assessment: Explain the four Git object types and draw the object graph for a two-file commit from memory.

Class 2: Refs, HEAD, Detached HEAD and the Three Trees

Topics:

- What a branch really is: a tiny file in refs/heads containing one commit SHA, nothing more
- HEAD as a symbolic ref: `git symbolic-ref HEAD`, and what detached HEAD actually means (HEAD pointing at a SHA instead of a branch)
- Working safely in detached HEAD: inspecting old commits, building from a tag, and rescuing new work with `git switch -c`
- Revision syntax fluency: `HEAD~2` vs `HEAD^2`, `@{upstream}`, `main@{yesterday}`, and resolving any expression with `git rev-parse`
- The three trees model: HEAD, the index (staging area), and the working directory, and how every core command moves content between them
- `git reset --soft` vs `--mixed` vs `--hard` explained precisely through the three trees, plus `git restore` and `git switch` as the modern split of checkout
- Remote-tracking refs: `refs/remotes/origin/main` as your last-known snapshot of the server, and how `fetch` vs `pull` update it

Projects built:

- Build a personal reset cheat card: run `soft`, `mixed`, and `hard` resets on a scratch repository and document exactly what happened to HEAD, index, and working tree in each case

Practice: Check out a raw commit SHA, make a commit in detached HEAD, then rescue it onto a new branch with `git switch -c`.

Assessment: Predict the exact effect of `git reset --mixed HEAD~1` on all three trees before running it, then verify your prediction.

Class 3: Porcelain vs Plumbing and the Staging Area Deep Dive

Topics:

- Porcelain vs plumbing: the user-facing commands vs the scriptable core, and why plumbing output stays stable for automation
- Reading the index directly: `git ls-files -s` shows mode, SHA, and stage number for every staged file
- Building a commit with pure plumbing: `git update-index`, `git write-tree`, and `git commit-tree`, so commits stop being magic
- Surgical staging: `git add -p` hunk selection (y, n, s to split, e to edit the hunk) for clean, reviewable commits
- Unstaging and untracking precisely: `git restore --staged` vs `git rm --cached` vs `git rm`, and what each leaves on disk
- `gitignore` mechanics: pattern rules, negation with `!`, why an already-tracked file ignores `.gitignore`, and debugging with `git check-ignore -v`
- Scripting-safe status: `git status --porcelain=v2` and `git diff --name-only` as building blocks for your own tools

Projects built:

- Create a real commit using only plumbing commands (`update-index`, `write-tree`, `commit-tree`), then verify it with `git log` and `git cat-file`

Practice: Take a deliberately messy working directory and use `git add -p` to split it into two clean, single-purpose commits.

Assessment: Explain the difference between `git rm --cached` and `git restore --staged`, and demonstrate both on a test file.

Class 4: Config Layers, Aliases and Diff/Merge Tooling

Topics:

- The config cascade: system, global, local, and worktree levels, audited with `git config --list --show-origin`
- Conditional includes with `includeIf gitdir`: separate work and personal identities (name, email, signing key) automatically per directory
- Power aliases: a graph log alias, and shell aliases with the `!` prefix for anything Git cannot do natively
- Better conflicts by default: `merge.conflictStyle zdiff3` to see the common ancestor inside conflict markers
- `rerere` (reuse recorded resolution): letting Git remember how you resolved a conflict and replay it automatically next time
- Wiring `difftool` and `mergetool`: using VS Code or your editor of choice for three-way merges instead of raw conflict markers
- Cross-platform hygiene: `core.autocrlf` and `.gitattributes eol` rules so Windows, Mac, and Linux teammates stop fighting over line endings

Projects built:

- Ship a `dotfiles-quality` global `.gitconfig`: identities via `includeIf`, five aliases you will actually use, `zdiff3` conflict style, `rerere` enabled, and a configured `mergetool`

Practice: Trigger the same merge conflict twice in a scratch repo and watch `rerere` resolve it automatically the second time.

Assessment: Show `git config --list --show-origin` and explain which layer each of your key settings comes from and why it lives there.

PHASE 2: HISTORY MASTERY

Rewrite, rescue, and interrogate history with confidence: interactive rebase, cherry-pick and revert, relog and stash recovery, automated bisect, and blame and pickaxe archaeology.

Class 5: Interactive Rebase: Rewriting History with Intent

Topics:

- git rebase -i HEAD~n: the todo list (pick, reword, edit, squash, fixup, drop) and how Git replays commits one at a time
- The autosquash workflow: git commit --fixup=<sha>, git rebase -i --autosquash, and rebase.autosquash true to make it the default
- Splitting a commit with edit: stop mid-rebase, git reset HEAD~1, restage in pieces, then git rebase --continue
- exec lines in the todo list: running your test suite after every replayed commit so each commit in the branch builds
- git rebase --update-refs: rewriting a stack of dependent branches in a single pass
- Precision transplants with git rebase --onto: moving a branch from one base to another exactly
- The golden rule: never rewrite pushed shared history, and git push --force-with-lease as the seatbelt when rewriting your own branch

Projects built:

- Take a deliberately messy 10-commit feature branch and rebase it into 3 clean, logical, test-passing commits using fixup, squash, reword, and an exec test line

Practice: Use git commit --fixup plus rebase --autosquash to fold a typo fix into the commit where it belongs.

Assessment: Explain when rebase is safe versus dangerous, and demonstrate --force-with-lease rejecting a push over a stale remote.

Class 6: Cherry-pick, Revert and Fixing Shared History

Topics:

- git cherry-pick <sha> and ranges (A..B vs A^..B): replaying exact changes onto another branch
- cherry-pick -x for traceability back to the original commit, and -n (--no-commit) to batch several picks into one commit
- Backporting in practice: moving a hotfix from main onto a release branch without dragging unrelated work along
- git revert: the only safe undo on shared history, and why it adds a new commit instead of deleting anything
- Reverting a merge commit with revert -m 1: what mainline means, and the trap waiting when you later re-merge that branch
- Reverting a revert: bringing previously reverted work back cleanly
- Conflict handling mid-operation: --continue, --abort, and --skip, and reading the conflict to know which side you actually want

Projects built:

- Simulate a release: branch release/1.2 from an older commit, cherry-pick two hotfixes onto it with -x, then revert a bad feature merge on main with revert -m 1

Practice: Cherry-pick a range of three commits onto a clean branch and resolve the one conflict it produces.

Assessment: State when to use revert versus reset versus rebase on a shared branch, with one concrete scenario for each.

Class 7: Reflog Rescues, Lost Commits and Stash Mastery

Topics:

- git reflog: the private journal of everywhere HEAD has been, plus per-branch reflogs with git reflog show <branch>
- Undoing a bad reset --hard: locate the pre-reset entry and restore with git reset --hard HEAD@{1}
- Resurrecting a deleted branch: recover its tip SHA from the reflog and recreate it with git branch <name> <sha>
- Deep recovery: git fsck --lost-found and dangling commits when the reflog cannot help, and how reflog expiry works
- Stash beyond the basics: git stash push -m with a message, --include-untracked, and stashing selected hunks with stash push -p
- apply vs pop, inspecting a stash with git stash show -p stash@{n}, and turning an old stash into a branch with git stash branch
- Built-in bookmarks: ORIG_HEAD and @{1}, the markers Git leaves behind before every dangerous operation

Projects built:

- Run a rescue drill: deliberately destroy work three different ways (reset --hard, branch -D, dropped stash) and recover all three using reflog and fsck

Practice: Stash only half of your working changes with stash push -p, then apply them on a different branch.

Assessment: Given a repo where a branch was force-deleted an hour ago, recover it fully and explain each command you used.

Class 8: git bisect: Finding the Exact Commit That Broke It

Topics:

- Binary search over history: why bisect pinpoints the bad commit in a handful of steps instead of hundreds of diff readings
- The manual loop: git bisect start, git bisect bad, git bisect good <sha>, test, mark, repeat,

- then git bisect reset
- git bisect skip for commits that cannot build, and how Git routes the search around them
- Full automation: git bisect run ./test.sh, with exit code 0 meaning good, 1-124 meaning bad, and 125 meaning skip
- Bisecting anything scriptable: failing unit tests, performance regressions via a timing script, even bundle size growth
- git bisect terms with --term-old and --term-new for hunts about behavior changes rather than breakage
- Reading the verdict: the first-bad-commit report, then git show and git blame on it to understand the real cause

Projects built:

- In a prepared repository with a hidden regression, write a small test script and let git bisect run find the first bad commit completely automatically

Practice: Bisect the same repository manually and count your steps to feel the binary search converge.

Assessment: Explain the meaning of exit codes 0, 1, and 125 in a bisect run script and give a case where you would return each.

Class 9: History Archaeology: blame, Pickaxe and Log Forensics

Topics:

- git blame beyond basics: -w to ignore whitespace, -C -C to track code moved between files, and -L 10,30 for a line range
- Keeping blame honest: blame.ignoreRevsFile with a .git-blame-ignore-revs file so mass reformatting commits stop hiding real authors
- The pickaxe: git log -S 'someString' to find every commit that added or removed a string, with -p to see the diffs
- Regex archaeology: git log -G '<pattern>' for changes matching a pattern even when the occurrence count stayed the same
- Function-level history: git log -L :myFunction:path/to/file to follow one function through time, and --follow across renames
- Shaping output for investigation: --oneline --graph --all, --author, --since, --grep, and custom --pretty=format: strings
- Choosing the right tool: pickaxe when you know the code, bisect when you know the symptom, blame when you know the line

Projects built:

- Answer three archaeology questions on a real open source repository (when was X introduced, who last changed Y and why, which commit removed Z) using blame, -S, and -G, citing exact SHAs

Practice: Use git log -L to follow one function through several refactors and renames.

Assessment: Given a mystery behavior change, produce the introducing commit SHA plus the command trail that found it.

PHASE 3: TEAM WORKFLOWS AT SCALE

Design how a whole team uses Git: pick the right branching strategy, enforce it with protection rules and CODEOWNERS, build a PR culture that scales, tame big repositories, and run professional releases.

Class 10: Branching Strategies: Trunk-Based vs GitFlow vs GitHub Flow

Topics:

- GitHub Flow: branch, PR, review, merge, deploy, the default loop for continuously deployed products
- GitFlow honestly assessed: develop, release, and hotfix branches, where it still fits (versioned, installed software) and where it hurts
- Trunk-based development: short-lived branches merged to main within a day or two, backed by strong CI
- Feature flags as the enabler: shipping incomplete work safely to main instead of letting branches age for weeks
- Merge strategy tradeoffs: merge commits vs squash-merge vs rebase-merge, and what each does to history, bisect, and blame
- Long-lived branch pathology: drift, giant conflicts, and review fatigue, and how batch size drives team speed
- A decision framework: choosing a strategy from team size, release cadence, deployment model, and compliance requirements

Projects built:

- Write a one-page branching strategy proposal for a team you know: chosen model, merge strategy, branch naming, and the reasoning behind each decision

Practice: Merge the same two branches three ways (merge, squash, rebase) in a scratch repo and compare git log --graph for each result.

Assessment: Defend a recommended strategy for two scenarios: a 4-person startup deploying daily, and a 40-person team shipping quarterly releases.

Class 11: Branch Protection, Rulesets and CODEOWNERS

Topics:

- Branch protection rules vs the newer rulesets: what each can enforce, and why rulesets can layer and apply across an organization
- Required pull request reviews: approval counts, dismissing stale approvals on new commits, and requiring review from Code Owners

- Required status checks: making green CI a hard merge gate, and strict mode requiring the branch to be up to date first
- CODEOWNERS syntax in depth: path patterns, teams vs individuals, and the gotchas (last matching rule wins, owners need write access)
- Blocking force pushes and deletions on protected branches, plus require linear history and how it interacts with squash and rebase merges
- Bypass lists and enforcement for admins: granting emergency exceptions without switching protection off
- Push protection and required signed commits as organization policy: stopping secrets and unverified commits at the door

Projects built:

- Configure a demo repository with a full ruleset: required reviews with code owners, required status checks, linear history, and a CODEOWNERS file covering two teams, then prove each rule blocks what it should

Practice: Open a PR that violates each rule in turn (no review, failing check, force push) and document exactly what GitHub blocks.

Assessment: Explain last-match-wins in CODEOWNERS using a worked example of a pattern ordering that surprises people.

Class 12: Pull Request Culture That Scales

Topics:

- PR templates in .github/PULL_REQUEST_TEMPLATE.md: prompting authors for context, testing notes, and rollout risk every time
- Draft PRs for early feedback, and converting to ready-for-review once CI is green
- Review mechanics that save hours: committable suggested changes, batching comments into one review, and viewed-files tracking
- Writing reviews people accept: comment on the code not the person, separate blocking issues from nits, always explain the why
- Keeping PRs small and stacked: splitting a large change into a chain of dependent PRs and landing them in order
- Auto-merge and merge queues: letting approved, green PRs land themselves without last-minute conflicts on busy repositories
- Review load balancing: round-robin and load-based team review assignment so reviews never bottleneck on one senior

Projects built:

- Design a complete PR playbook for a repository: template, review checklist, size guidelines, and auto-merge or merge queue settings, then run one real PR through it end to end

Practice: Review a prepared flawed PR and leave at least five comments, using committable suggestions wherever possible.

Assessment: Split a provided large change into a sensible stack of three PRs and explain the landing order.

Class 13: Monorepos, Submodules and Scaling Big Repositories

Topics:

- Monorepo vs polyrepo: atomic cross-project changes and a single-version policy versus independent cadence and access isolation
- Submodules honestly: git submodule add, update --init --recursive, how the parent pins an exact SHA, and the classic detached-HEAD-inside-a-submodule trap
- git subtree as the low-ceremony alternative: vendoring a repository with history and pushing changes back upstream
- sparse-checkout in cone mode: git sparse-checkout init --cone and set <dirs> to work on one slice of a huge monorepo
- Partial clone: git clone --filter=blob:none for full history without full content, versus shallow --depth 1 clones and their limitations
- Git LFS for large binaries: git lfs track, .gitattributes, and migrating existing history with git lfs migrate import
- Repo health at scale: git maintenance start, the commit-graph, and background prefetch to keep big repositories fast

Projects built:

- Rescue a bloated demo repo: move its binary assets to Git LFS with lfs migrate, set up cone-mode sparse-checkout for one team's directory, and measure clone size before and after using --filter=blob:none

Practice: Add the same dependency repo as a submodule on one branch and a subtree on another, update upstream, and pull the change in both ways.

Assessment: Recommend monorepo or polyrepo for a described organization and name the specific Git features that make your choice workable.

Class 14: Release Management: Tags, SemVer and Changelogs

Topics:

- Semantic versioning as a contract: what MAJOR.MINOR.PATCH each promise to consumers, plus pre-release and build metadata suffixes
- Annotated vs lightweight tags: git tag -a v2.1.0 -m, why releases always get annotated (or signed with -s) tags, and pushing them explicitly
- git describe: turning any commit into a human-readable version string for build stamping and support tickets
- GitHub Releases: creating a release from a tag, attaching artifacts, and auto-generated release notes configured via release.yml categories
- Conventional Commits (feat:, fix:, BREAKING CHANGE:) and how they enable automated changelogs and version bumping

- Release branches and the hotfix flow: cutting release/2.1, hardening it, and backporting fixes with cherry-pick -x
- Tag discipline: why a published tag must never be moved, and the correct recovery when a bad tag ships

Projects built:

- Run a full release cycle on your own project: conventional commits, cut a release branch, create an annotated signed v1.0.0 tag, publish a GitHub Release with categorized auto-generated notes, then ship a v1.0.1 hotfix

Practice: Rewrite five real commit messages into Conventional Commits format and predict the version bump each one implies.

Assessment: Explain the difference between annotated and lightweight tags and demonstrate why release tooling only trusts the former.

----- PHASE 4: AUTOMATION, CI/CD & SECURITY -----

Automate everything and lock it down: GitHub Actions from fundamentals to reusable workflows, secret management and gated deployments, supply chain security, signed commits, hooks, and GitHub CLI scripting.

Class 15: GitHub Actions: The Core Model

Topics:

- Anatomy of a workflow: .github/workflows/*.yaml, events, jobs, steps, runners, and reading logs in the Actions tab
- Trigger fluency: on push with branch and path filters, pull_request vs pull_request_target (and why the latter is dangerous), workflow_dispatch with inputs, and schedule with cron
- Jobs and runners: ubuntu, windows, and macos images, needs: for dependencies between jobs, and parallel-by-default execution
- Using marketplace actions safely: actions/checkout and setup actions, and pinning by tag versus full commit SHA
- Contexts and expressions: github and runner contexts, if: conditions, and wiring step outputs into later steps
- The GITHUB_TOKEN: what it can touch, and shrinking it with workflow-level and job-level permissions: blocks
- The free tier in real numbers: GitHub Free includes 2,000 Actions minutes per month plus unlimited public and private repositories, so every exercise in this course runs at no cost

Projects built:

- Build a CI workflow from scratch for a small app: lint, test, and build on every pull request, with path filters so documentation-only changes skip CI

Practice: Add a workflow_dispatch trigger with an input and use its value inside a step expression.

Assessment: Read an unfamiliar failing workflow run and identify the failing job, step, and root cause from the logs alone.

Class 16: Advanced Actions: Matrix Builds, Caching and Reusable Workflows

Topics:

- strategy.matrix: testing across language versions and operating systems from one job definition, with include and exclude for edge cells
- fail-fast and max-parallel: controlling what happens to the rest of the matrix when one cell fails
- actions/cache: key design with hashFiles on your lockfile, restore-keys fallback chains, and verifying the cache actually saves time
- Artifacts vs cache: upload-artifact and download-artifact for passing build outputs between jobs and preserving test reports
- Reusable workflows with workflow_call: one CI definition shared across many repositories, with inputs and secrets passed explicitly
- Composite actions: bundling repeated step sequences into your own action under .github/actions/
- concurrency groups with cancel-in-progress: stopping superseded runs on the same PR instead of burning minutes

Projects built:

- Upgrade your Class 15 pipeline: a six-cell OS-by-version matrix, dependency caching, a test-report artifact, and the whole pipeline extracted into a reusable workflow called from a second repository

Practice: Measure one workflow's runtime with a cold cache and a warm cache and record the difference.

Assessment: Design a cache key strategy for a given project and justify the restore-keys ordering.

Class 17: Secrets, Environments and Deployment Pipelines

Topics:

- The secrets hierarchy: repository, organization, and environment secrets, plus log masking and its limits
- GitHub environments: production vs staging with required reviewers, wait timers, and environment-scoped secrets
- Deployment jobs: attaching environment: production to a job and tracking deploy history in the repository's Environments view
- OIDC federation: short-lived cloud credentials for AWS, Azure, or GCP instead of storing long-lived keys as secrets

- The build-once pattern: build a single artifact, deploy that same artifact to staging, then promote it to production through an approval gate
- Deploying to GitHub Pages: free static site hosting wired directly from an Actions workflow
- Rollback and safety: re-running a previous successful deployment, and protecting the deploy branch so only reviewed code ships

Projects built:

- Build a two-stage deploy pipeline: build an artifact, auto-deploy to a staging environment, then a production deploy to GitHub Pages gated behind a required reviewer

Practice: Rotate a repository secret and confirm the workflow picks up the new value with no file changes.

Assessment: Explain why OIDC beats long-lived cloud keys in CI and sketch the trust relationship that makes it work.

Class 18: Supply Chain Security: Dependabot, Secret Scanning and CodeQL

Topics:

- The modern threat model: most of a repository's code is dependencies, and attacks arrive through them and through leaked credentials
- Dependabot alerts and security updates: automatic PRs that bump vulnerable dependencies, and triaging them without alert fatigue
- dependabot.yml version updates: scheduled freshness PRs per ecosystem and directory, with grouping to cut the noise
- Secret scanning and push protection: GitHub detecting known credential patterns and blocking the push before the secret ever lands
- CodeQL code scanning: enabling default setup, reading alerts as data-flow paths, and gating PRs on newly introduced alerts
- Hardening Actions themselves: pinning third-party actions to full commit SHAs, least-privilege GITHUB_TOKEN, and reviewing workflow changes like code
- Dependency review on pull requests: seeing new and changed dependencies with their advisories before merge

Projects built:

- Fully harden a demo repository: Dependabot alerts plus weekly grouped version updates, push protection, and CodeQL default setup, then deliberately introduce a vulnerable dependency and a planted fake secret and document what each defense caught

Practice: Triage three Dependabot alerts: decide update now, ignore, or investigate, and justify each call.

Assessment: Explain the difference between Dependabot alerts, version updates, and security updates in plain language.

Class 19: Signed Commits, 2FA, Hooks and the pre-commit Framework

Topics:

- Why signing matters: anyone can set user.name and user.email to any value, so signatures are the only real authorship proof
- SSH-based signing, the easy modern path: gpg.format ssh, user.signingkey, commit.gpgsign true, and registering the key as a signing key on GitHub
- The Verified badge and vigilant mode: making unsigned commits attributed to you stand out
- Account hygiene: two-factor authentication, and fine-grained personal access tokens with minimal scopes and expiry dates
- Client-side hooks: pre-commit, commit-msg, and pre-push, why they are per-clone by default, and sharing them with core.hooksPath
- The pre-commit framework: .pre-commit-config.yaml, pre-commit install, and running formatters, linters, and secret detectors on staged files only
- Server-side thinking: what pre-receive hooks do, and how GitHub replaces them for hosted repos with protection rules and push protection

Projects built:

- Set up verified signed commits with an SSH key, then add a pre-commit configuration running a formatter, a linter, and a secret detector, and prove a planted fake credential cannot be committed

Practice: Write a commit-msg hook that rejects messages not matching Conventional Commits format, and test it both ways.

Assessment: Show a Verified badge on your own GitHub commit and explain the chain of trust that produces it.

Class 20: GitHub CLI (gh) and API Automation

Topics:

- gh auth login and gh auth status: authenticated automation from any terminal once, then scriptable forever
- The PR workday without a browser: gh pr create --fill, gh pr checkout, gh pr view --web, gh pr review --approve, gh pr merge --squash
- Issue automation: gh issue list --label bug --json number,title for structured output, and gh issue create from scripts
- Watching CI from the terminal: gh run list, gh run watch, and gh run rerun --failed
- Raw API access: gh api repos/{owner}/{repo}/pulls with --jq filters and --paginate for full result sets
- GraphQL for the hard questions: gh api graphql for data the REST API returns poorly, like nested review threads
- Gluing it together: gh inside shell scripts and Actions workflows via GH_TOKEN, plus gh alias set for daily shortcuts

Projects built:

- Write a morning-review script with gh: list PRs awaiting your review with their CI status and

age as a clean table, then alias it to a single command
Practice: Open, review, and squash-merge a pull request entirely from the terminal without touching the browser.
Assessment: Use `gh api` with `--jq` to answer a repository question (for example, the five most recently updated open PRs) in one command.

----- PHASE 5: LEADERSHIP & CAPSTONE -----

Operate at the level teams depend on: migrate an organization to GitHub, recover from disasters and leaked secrets, run AI-assisted review workflows responsibly, and ship a capstone workflow you can defend.

Class 21: Migrating a Team to GitHub and Teaching Git Well

Topics:

- Migration inventory: repositories, history depth, binaries destined for LFS, users and permissions, CI jobs, and the integrations that will break
- Full-fidelity moves: `git clone --mirror` plus `git push --mirror` to carry every branch and tag, and importer tooling for other hosts
- Cutover planning: freeze windows, read-only on the old host, repointing remotes with `git remote set-url`, and a tested rollback plan
- Designing team conventions: branch naming, commit style, PR rules, and a CONTRIBUTING.md people actually read
- Teaching Git so it sticks: mental model first (objects, refs, three trees), commands second, and running an internal workshop
- The failure modes you will meet: force-push accidents, giant binary commits, enormous PRs, and the specific guardrail that prevents each
- Measuring adoption: PR cycle time, review coverage, and revert rate as signals the new workflow is actually working

Projects built:

- Produce a complete migration runbook for a fictional 10-developer team moving to GitHub: timeline, mirror commands, permission mapping, guardrail configuration, and a one-hour onboarding workshop outline

Practice: Mirror-migrate a multi-branch repository between two remotes and verify every branch and tag arrived intact.

Assessment: Deliver a five-minute explanation of the three trees model as you would to a junior teammate, without notes.

Class 22: Disaster Recovery: filter-repo, Leaked Secrets and Corrupt Repos

Topics:

- First rule of a leaked secret: rotate the credential immediately, because forks and clones already have the history, cleanup is step two
- `git filter-repo`, the maintained replacement for `filter-branch`: removing a file from all history with `--path` plus `--invert-paths`
- Redacting content everywhere: `filter-repo --replace-text` to scrub passwords and keys from every commit that ever contained them
- The aftermath of a rewrite: every SHA changes, force-push coordination, teammates re-clone or hard-reset, and open PRs need rebasing
- Recovering from a bad force-push: your own reflog, teammates' clones as backups, and why `--force-with-lease` should be muscle memory
- Corrupt repository triage: reading `git fsck` output, copying missing objects from a healthy clone's `.git/objects`, and knowing when to re-clone
- Prevention as the real strategy: push protection, pre-commit secret detectors, and protected branches make most of this class unnecessary

Projects built:

- Run a leak drill: plant a fake API key five commits deep, then execute the full response: rotate (simulated), scrub with `filter-repo --replace-text`, force-push, and verify with `git log -S` that no trace remains

Practice: Break a scratch repository by deleting an object file, diagnose it with `git fsck`, and repair it from a second clone.

Assessment: Write the incident checklist you would follow for a production secret leak, in the correct order, with commands.

Class 23: AI-Assisted Git Workflows: Copilot and Coding Agents

Topics:

- Copilot code review on GitHub: requesting an AI review on a pull request and treating its findings as a first pass, never a verdict
- Access reality: GitHub Copilot has a free tier you can practice with; check github.com/features/copilot for what the current plans include before relying on any feature
- Coding agents that open PRs: routing AI-authored branches through the same protection rules, status checks, and CODEOWNERS as human code
- Git hygiene for agent work: small scoped branches per task, descriptive PR bodies, and never letting an agent force-push shared history
- AI-generated commit messages and PR summaries: where they genuinely help, and why the diff remains the source of truth
- Team policy for AI code: disclosure, review depth, and the license and provenance questions your organization must answer
- The reviewer's edge: your archaeology skills (blame, pickaxe, bisect) are exactly what keeps AI-speed development safe

Projects built:

- Run one feature through an AI-assisted flow: have an AI tool draft the change on a branch, then apply your full professional gauntlet (protection rules, CI, line-by-line review) before merging, and write up what the AI got right and wrong

Practice: Request a Copilot review on a pull request and compare its findings against your own review of the same diff.

Assessment: Draft a one-page AI usage policy for a team's Git workflow covering branches, review requirements, and accountability.

Class 24: Capstone: Design a Complete Team Workflow + Certification

Topics:

- The capstone brief: take a real repository, yours or a provided one, and productionize it end to end
- Workflow layer: a written branching strategy, protected main with a ruleset, CODEOWNERS, a PR template, and an auto-merge or merge queue decision
- CI/CD layer: a matrix test workflow with caching, a build artifact, and a staged deploy behind an environment approval gate
- Security layer: Dependabot, secret scanning with push protection, CodeQL, signed commits, and actions pinned to SHAS
- Release layer: Conventional Commits, an annotated signed tag, and a GitHub Release with generated notes
- The defense: walk a mentor through every choice and justify it against the alternatives, exactly like an architecture review
- Certification: present the finished repository and earn the Modern Age Coders Advanced Git & GitHub certificate

Projects built:

- Ship the capstone: one repository with the complete workflow, CI/CD, security, and release stack configured and demonstrably working, plus a README section documenting every design decision

Practice: Do a dry-run presentation of your capstone to a classmate and incorporate at least one piece of their feedback.

Assessment: Final review: every guardrail demonstrated live (a blocked push, a failing check gate, a gated deploy) earns the certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a hand-traced object graph of your own commit, a three-trees reset cheat card, a commit built from pure plumbing commands, and a dotfiles-quality .gitconfig with identities, aliases, zdifff3, and rerere
- Phase 2: a messy 10-commit branch rebased into 3 clean test-passing commits, a simulated release with cherry-picked hotfixes and a merge revert, a three-way rescue drill recovered via reflog and fsck, an automated bisect run hunt, and three archaeology questions answered with cited SHAS
- Phase 3: a written branching strategy proposal, a fully enforced ruleset with CODEOWNERS proven by blocked PRs, a complete PR playbook run end to end, an LFS plus sparse-checkout rescue of a bloated repo, and a full v1.0.0 release cycle with a signed tag and generated notes
- Phase 4: a from-scratch CI workflow, a matrix build with caching extracted into a reusable workflow, a two-stage gated deploy to GitHub Pages, a fully hardened repo that catches a planted vulnerability and secret, a signed-commit plus pre-commit guardrail setup, and a gh-powered morning-review script
- Phase 5: a complete team migration runbook, a leak drill scrubbed with filter-repo and verified clean, one feature shipped through a professionally reviewed AI-assisted flow, and the capstone: a production-grade repository with workflow, CI/CD, security, and release layers demonstrated live
- Total: 24 hands-on projects, one per class, finishing with a production-grade capstone repository

Total projects: 24 hands-on projects, one per class, finishing with a production-grade capstone repository

SKILLS THE STUDENT WILL HAVE AT THE END

- Git Internals: the object model, refs and HEAD, the three trees, and plumbing commands like cat-file and rev-parse
- History Surgery: interactive rebase with autosquash and exec, cherry-pick, revert on shared history, and filter-repo rewrites
- Rescue Operations: reflog recovery, fsck, automated git bisect run, advanced stash workflows, and corrupt repo triage
- Team Workflow Design: trunk-based development vs GitFlow, rulesets, CODEOWNERS, PR templates, stacked PRs, and merge queues
- Repositories at Scale: monorepo tooling, sparse-checkout, partial clone, submodules vs subtree, and Git LFS migration
- CI/CD with GitHub Actions: matrix builds, caching, artifacts, reusable workflows, environments, OIDC, and gated deployments
- Security: Dependabot, secret scanning with push protection, CodeQL, signed commits, and the pre-commit framework
- Automation and Leadership: GitHub CLI and API scripting, team migrations, release management, and teaching Git to others

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks

Hands on git work: 1.5-2 hours running the commands from class on your own repositories, including the deliberate break-and-recover drills

Project time: 1-1.5 hours completing each class project on your own machine

Review and doubts: 30-60 minutes reviewing your work and clearing doubts over WhatsApp before the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for stuck rebases, workflow errors, and setup questions

Small batches: Small batches, with mini-batch and 1-on-1 options, so every professional gets individual attention

Mentor guidance: Live guidance from mentors who run these workflows on real production repositories

Project feedback: Feedback on every class project and a personal capstone review before certification

Living syllabus: Curriculum updated as Git and GitHub ship new features, at no extra cost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'Advanced Git & GitHub' certificate, earned by presenting and defending your Class 24 capstone repository

Project portfolio: A public capstone repository with a complete workflow, CI/CD, security, and release stack, plus 24 documented class projects

GitHub profile: A GitHub profile showing verified signed commits, real pull requests, passing Actions workflows, and a professional release history

WHERE THIS CAN LEAD

- Senior developer trusted with repository administration, history surgery, and release management
- Team lead or engineering manager who designs and rolls out the team's branching and review workflow
- DevOps or platform engineer building CI/CD pipelines and deployment gates with GitHub Actions
- Build, release, or developer-experience engineer at organizations running large or monorepo codebases
- The go-to Git person on any team: the one who rescues lost work instead of suggesting a re-clone

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this Git course too advanced if I only know basic commands like add, commit, and push?

A: No, that is exactly the starting profile it is designed for. The course assumes you have used basic Git at work or in projects, and Classes 1-4 rebuild your understanding from the object model up, so the advanced material lands on solid ground. If you can clone a repo and make a commit, you are ready.

Q: How is this different from a beginner Git and GitHub course?

A: Beginner courses teach you the commands; this one teaches you the system. We cover Git internals, interactive rebase, reflog recovery, branch protection rulesets, monorepo tooling, GitHub Actions CI/CD, and repository security, the skills of the person a team calls when Git goes wrong. Our separate college-students course covers the foundations track.

Q: Do I need a paid GitHub plan to take this course?

A: No. If you are a verified student at a degree or diploma granting school, the free GitHub Student Developer Pack adds further benefits while you study.

Q: Does this course cover GitHub Actions and CI/CD pipelines?

A: Yes, in depth: six full classes. You go from workflow anatomy and triggers through matrix builds, caching, artifacts, and reusable workflows, to secrets, environments with approval gates, OIDC cloud deployments, and supply chain security with Dependabot, secret scanning, and CodeQL. By the capstone you build a complete gated deploy pipeline yourself.

Q: Are the classes live or pre-recorded?

A: Every class is live and interactive with a real instructor, never pre-recorded videos. Group batches are capped at 10 professionals, with mini-batch and 1-on-1 options, and you get WhatsApp doubt support between classes. Modern Age Coders has taught live online classes this way since 2020.

Q: Does the course teach how to remove a leaked password or API key from Git history?

A: Yes, Class 22 is a full disaster recovery session. You run a complete leak drill: rotate the credential first, scrub every trace from history with `git filter-repo --replace-text`, coordinate the force-push, and verify the cleanup with `git log -S`. You also set up the defenses (push protection and pre-commit secret detectors) that stop the leak from happening

again.

Q: Can my company enrol a team for corporate Git training?

A: Yes. Book a free demo class and we will discuss batch scheduling for your team's timezone; we have taught learners across 25+ countries.

Q: What certificate do I get after completing the course?

A: You earn the Modern Age Coders 'Advanced Git & GitHub' certificate by presenting and defending your Class 24 capstone: a repository with a complete workflow, CI/CD, security, and release stack that you demonstrate live. You also leave with the repository itself and a GitHub profile full of verified signed commits, which usually says more than the certificate does.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/git-github-advanced-version-control-masterclass-for-professionals>

COURSE 5 OF 118: AI & MACHINE LEARNING FOR TEENS: ZERO TO AI EXPERT

Course link: <https://learn.modernagecoders.com/courses/ai-ml-masterclass-teens>

Subject area: Artificial Intelligence & Machine Learning

Designed for: Teens

Who can join: Complete Beginner to AI Developer

Course length: 12 months (52 weeks)

Time needed: 12-15 hours/week recommended

Certificate: AI/ML Expert Certification with Project Portfolio

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year AI and Machine Learning program for teens: master Python, maths, ML algorithms, deep learning, computer vision, NLP and build real AI projects.

In one line: From 'What is AI?' to Building Your Own Intelligent Systems

ABOUT THE PROGRAMME

This revolutionary program transforms curious teenagers into AI creators. Starting with zero coding knowledge, you'll journey through the fascinating world of artificial intelligence and machine learning. Learn how Netflix recommends movies, how Instagram filters work, how ChatGPT understands language, and how self-driving cars see the road. By the end of this year, you'll have built 40+ AI projects including chatbots, image recognizers, game-playing AIs, recommendation systems, and even your own versions of popular AI applications. You'll understand not just how to use AI, but how to create it from scratch.

WHAT MAKES THIS COURSE DIFFERENT

- Designed specifically for teenage minds with relatable examples
- Visual and interactive learning with animations and experiments
- Build AI versions of apps teens use daily
- Focus on creativity and fun projects, not just theory
- From zero coding to creating real AI in 12 months
- Ethics and responsible AI development emphasized
- Preparation for AI careers and competitions
- Access to GPUs for training models

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Python Programming, Math Basics, Introduction to AI

Phase 2: Machine Learning (Months 4-6): ML Algorithms, Data Processing, Model Building

Phase 3: Deep Learning (Months 7-9): Neural Networks, CNNs, RNNs, Modern AI

Phase 4: Advanced AI (Months 10-12): Computer Vision, NLP, Reinforcement Learning, AI Applications

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic math (algebra level)

Coding experience: None required - beginners welcome

Equipment: Computer with internet (GPU helpful but not required)

Time commitment: 12-15 hours per week

Age: 13-19 years (teen-focused content)

Motivation: Curiosity about AI and future technology

WHO THIS COURSE IS FOR

Tech enthusiasts: Teens fascinated by AI and technology
Future researchers: Students interested in AI research
Creative minds: Those wanting to create with AI
Problem solvers: Teens who love solving complex problems
Entrepreneurs: Young people with startup dreams
Gamers: Interested in game AI and development
Social impact: Those wanting to use AI for good

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & AI BASICS (MONTHS 1-3, WEEKS 1-13) -----

Build strong programming foundations, understand AI concepts, and create your first intelligent programs.

Month 1: Python Programming for AI (Week 1-4)

Introduction to AI & Python Setup

Topics:

- What is Artificial Intelligence? Real examples teens use daily
- AI vs ML vs Deep Learning explained with examples
- History of AI: From chess to ChatGPT
- Setting up Python environment (Anaconda, Jupyter)
- Google Colab for free GPU access
- Python basics: Variables, data types, operations
- Lists and dictionaries for data storage
- Functions and code organization
- Python libraries introduction: NumPy, Pandas preview
- Your first AI program: Rule-based chatbot
- How machines 'think': Algorithms vs intelligence
- AI in everyday life: Spotify, TikTok, YouTube recommendations

Projects built:

- Setup complete AI development environment
- Rule-based chatbot (like early Siri)
- Simple recommendation system using rules
- AI decision tree for game choices

Practice: Daily: Python exercises, explore AI applications

Python Data Structures & Control Flow

Topics:

- Advanced Python lists: Slicing, comprehensions
- Working with dictionaries for data mapping
- Sets and tuples for data organization
- If-else logic for AI decisions
- Loops for processing data
- Functions for reusable AI components
- Object-oriented programming basics
- Classes for AI models
- File handling: Reading/writing data
- JSON for AI configuration
- Error handling in AI programs
- Debugging AI code effectively

Projects built:

- Student grade predictor system
- Text-based adventure game with AI NPCs
- Data analyzer for social media stats
- Simple expert system for diagnosis

Practice: Build 10 Python programs with AI logic

NumPy & Mathematical Foundations

Topics:

- NumPy arrays: The foundation of AI
- Array operations and broadcasting
- Mathematical operations on arrays
- Statistics basics: Mean, median, mode, std deviation
- Probability concepts for AI
- Linear algebra basics: Vectors and matrices
- Matrix operations: Addition, multiplication
- Why math matters in AI (visual explanations)
- Plotting with Matplotlib
- Visualizing data patterns
- Random numbers and simulations
- Mathematical functions in NumPy

Projects built:

- Grade distribution analyzer
- Dice probability simulator
- Image manipulation with arrays

- Statistical analysis dashboard

Practice: Complete 20 NumPy exercises daily

Pandas & Data Manipulation

Topics:

- DataFrames: Spreadsheets in Python
- Reading data: CSV, Excel, JSON
- Data selection and filtering
- Handling missing data
- Data cleaning techniques
- Grouping and aggregation
- Merging and joining datasets
- Time series data basics
- Data visualization with Pandas
- Exploratory Data Analysis (EDA)
- Feature engineering introduction
- Preparing data for ML

Projects built:

- YouTube channel analytics tool
- Sports statistics analyzer
- Weather pattern explorer
- Social media trends analyzer

Practice: Analyze 5 real-world datasets

Months 2-3: AI Fundamentals & First Models (Week 5-13)

Introduction to Machine Learning

Topics:

- What is Machine Learning? Learning from examples
- Supervised vs Unsupervised vs Reinforcement Learning
- Classification vs Regression problems
- Training data, validation data, test data
- Features and labels explained
- Overfitting and underfitting (Goldilocks principle)
- Model evaluation metrics: Accuracy, precision, recall
- Confusion matrix understanding
- Cross-validation concept
- Bias-variance tradeoff simplified
- Feature scaling and normalization
- Introduction to Scikit-learn

Projects built:

- Iris flower classifier (classic starter)
- Spam email detector
- Music genre classifier
- Friend group predictor

Practice: Train 10 different ML models

Classification Algorithms

Topics:

- K-Nearest Neighbors (KNN): Finding similar things
- Decision Trees: AI making choices
- Random Forests: Wisdom of crowds
- Naive Bayes: Probability-based predictions
- Logistic Regression for classification
- Support Vector Machines (SVM) basics
- Ensemble methods: Voting classifiers
- Multi-class classification
- Imbalanced data handling
- Feature importance analysis
- Model selection strategies
- Hyperparameter tuning basics

Projects built:

- Handwritten digit recognizer
- Emoji sentiment analyzer
- Game outcome predictor
- Fashion item classifier

Practice: Solve 15 classification challenges

Regression & Prediction

Topics:

- Linear Regression: Finding patterns in data
- Polynomial Regression: Curved relationships
- Multiple Linear Regression
- Ridge and Lasso Regression
- Regression evaluation metrics: MSE, RMSE, R^2
- Time series prediction basics
- Feature engineering for regression
- Dealing with outliers
- Residual analysis
- Gradient Descent visualization
- Learning rate importance
- Regression vs Classification: When to use what

Projects built:

- House price predictor
- Exam score predictor
- YouTube views predictor
- Weather temperature forecaster

Practice: Build 10 prediction models

Unsupervised Learning

Topics:

- Clustering: Finding groups in data
- K-Means clustering algorithm
- Hierarchical clustering
- DBSCAN for density-based clustering
- Dimensionality reduction: PCA basics
- t-SNE for visualization
- Anomaly detection methods
- Association rules (Market basket)
- Clustering evaluation metrics
- Choosing number of clusters
- Real-world clustering applications
- Visualization of high-dimensional data

Projects built:

- Customer segmentation tool
- Music playlist generator
- Anomaly detector for gaming
- Friend group discoverer

Practice: Apply clustering to 10 datasets

Phase 1 Capstone Project

Topics:

- End-to-end ML project workflow
- Problem definition and data collection
- Data preprocessing pipeline
- Model training and evaluation
- Model deployment basics
- Creating user interface

Projects built:

- CAPSTONE: Personal AI Assistant
- Features: Multiple ML models, predictions, recommendations
- Alternative: AI-powered study buddy
- Alternative: Smart home automation system

Assessment: Build complete ML application from scratch

Month 3: Advanced ML & Introduction to Deep Learning (Week 10-13)

Advanced Machine Learning Techniques

Topics:

- Gradient Boosting Machines (GBM)
- XGBoost: Competition winner
- LightGBM for speed
- CatBoost for categorical data
- Stacking and blending models
- Feature engineering mastery
- Automated feature selection
- Cross-validation strategies
- Hyperparameter optimization: Grid, Random, Bayesian
- Pipeline creation in Scikit-learn
- Model interpretability: SHAP, LIME
- Handling big data with ML

Projects built:

- Kaggle competition entry
- Advanced recommendation system
- Multi-model ensemble predictor
- AutoML system builder

Practice: Optimize 10 ML models for performance

Introduction to Neural Networks

Topics:

- Biological inspiration: How brains work
- Artificial neurons (Perceptrons)
- Activation functions: ReLU, Sigmoid, Tanh
- Forward propagation explained
- Backpropagation intuition
- Gradient descent deep dive
- Building neural networks from scratch
- Introduction to TensorFlow/Keras
- Your first neural network
- Training neural networks
- Preventing overfitting: Dropout, regularization
- Batch normalization basics

Projects built:

- Neural network from scratch (NumPy)
- XOR problem solver
- Simple pattern recognizer

- Basic neural network visualizer

Practice: Build 5 neural networks for different tasks

Deep Learning Fundamentals

Topics:

- What makes learning 'deep'?
- Deep vs shallow networks
- TensorFlow 2.0 and Keras API
- Building deep neural networks
- Optimizers: SGD, Adam, RMSprop
- Learning rate scheduling
- Early stopping and callbacks
- Model checkpointing
- Transfer learning concept
- Pre-trained models introduction
- GPU acceleration basics
- Debugging neural networks

Projects built:

- MNIST digit classifier (deep version)
- Fashion MNIST challenger
- Simple image classifier
- Sound pattern recognizer

Practice: Train 10 deep learning models

Data Augmentation & Preprocessing

Topics:

- Why data augmentation matters
- Image augmentation techniques
- Text augmentation methods
- Audio data preprocessing
- Synthetic data generation
- Data balancing techniques
- SMOTE for imbalanced data
- Feature extraction from images
- Text vectorization methods
- Handling sequential data
- Data pipeline optimization
- TensorFlow Data API

Projects built:

- Data augmentation toolkit
- Synthetic data generator
- Image preprocessing pipeline
- Text data preparation system

Practice: Create 5 augmented datasets

Phase 1 Final Assessment

Topics:

- Review of all concepts
- Best practices recap
- Model evaluation strategies
- Deployment preparation
- Portfolio development

Projects built:

- MAJOR PROJECT: AI-Powered Mobile App
- Multiple ML models integrated
- User-friendly interface
- Real-world application

Assessment: Comprehensive exam + project presentation

PHASE 2: CORE MACHINE LEARNING MASTERY (MONTHS 4-6, WEEKS 14-26)

Deep dive into advanced ML algorithms, feature engineering, model optimization, and real-world applications.

Months 4-5: Advanced ML & Computer Vision (Week 14-22)

Convolutional Neural Networks (CNNs)

Topics:

- How computers see: Pixels to features
- Convolution operation explained visually
- Filters and feature maps
- Pooling layers: Max and average
- CNN architecture design
- Famous architectures: LeNet, AlexNet, VGG
- ResNet and skip connections
- Building CNNs in TensorFlow/Keras
- Image classification with CNNs
- Data augmentation for images
- Visualizing CNN layers
- Understanding what CNNs learn

Projects built:

- Custom image classifier (personal photos)

- Face emotion detector
- Hand gesture recognizer
- Pet breed identifier

Practice: Build 10 different CNN architectures

Advanced Computer Vision

Topics:

- Object detection: Finding things in images
- YOLO (You Only Look Once) basics
- Face detection and recognition
- Image segmentation techniques
- Style transfer: Making art with AI
- Image generation basics
- OpenCV for computer vision
- Real-time video processing
- Pose estimation
- Optical Character Recognition (OCR)
- Medical image analysis basics
- AR filters like Snapchat

Projects built:

- Object detection app
- Snapchat-style filter creator
- Document scanner with OCR
- Real-time pose detector for fitness

Practice: Create 8 computer vision applications

Recurrent Neural Networks (RNNs)

Topics:

- Sequential data and time series
- Vanilla RNN architecture
- Vanishing gradient problem
- LSTM (Long Short-Term Memory)
- GRU (Gated Recurrent Units)
- Bidirectional RNNs
- Sequence-to-sequence models
- Time series prediction with RNNs
- Text generation with RNNs
- Music generation basics
- Stock price prediction (and limitations)
- Speech recognition introduction

Projects built:

- Text generator (like phone keyboards)
- Stock price predictor
- Music composer AI
- Weather forecasting model

Practice: Train 10 RNN models for sequences

Natural Language Processing Basics

Topics:

- How computers understand text
- Tokenization and text preprocessing
- Word embeddings: Word2Vec, GloVe
- Sentiment analysis techniques
- Named Entity Recognition (NER)
- Part-of-speech tagging
- Text classification methods
- Topic modeling with LDA
- Text summarization basics
- Machine translation introduction
- Chatbot frameworks
- NLTK and spaCy libraries

Projects built:

- Sentiment analyzer for reviews
- Fake news detector
- Chatbot for customer service
- Text summarizer for articles

Practice: Build 10 NLP applications

Transformers & Modern NLP

Topics:

- Attention mechanism explained
- Transformer architecture basics
- BERT for understanding text
- GPT models introduction
- Fine-tuning pre-trained models
- Hugging Face library basics
- Zero-shot classification
- Question answering systems
- Text generation with GPT
- Prompt engineering basics
- Using APIs: OpenAI, Cohere
- Responsible AI and bias

Projects built:

- Question-answering system
- AI writing assistant
- Language translator
- Custom GPT for specific domain

Practice: Fine-tune 5 transformer models

Month 6: Generative AI & Creative Applications (Week 23-26)

Generative Adversarial Networks (GANs)

Topics:

- GANs: AI creating new content
- Generator vs Discriminator
- Training GANs (the mini-max game)
- DCGAN for image generation
- StyleGAN and variations
- Mode collapse and solutions
- Conditional GANs
- CycleGAN for style transfer
- Pix2Pix for image translation
- GANs for data augmentation
- Ethical considerations
- Detecting AI-generated content

Projects built:

- Face generator
- Anime character creator
- Photo to sketch converter
- Style transfer application

Practice: Train 5 different GAN architectures

Autoencoders & Dimensionality Reduction

Topics:

- Autoencoder architecture
- Encoding and decoding
- Latent space representation
- Denoising autoencoders
- Variational Autoencoders (VAE)
- Image compression with autoencoders
- Anomaly detection using autoencoders
- Feature extraction
- Recommendation systems with autoencoders
- Super-resolution with AI
- Image inpainting
- Creating embeddings

Projects built:

- Image compressor
- Image denoiser
- Anomaly detector for games
- Recommendation engine

Practice: Build 8 autoencoder applications

Audio & Music AI

Topics:

- Digital audio basics: Sampling, frequencies
- Audio preprocessing: Spectrograms, MFCCs
- Speech recognition systems
- Text-to-Speech (TTS) synthesis
- Voice cloning basics
- Music generation with AI
- Audio classification
- Sound effect generation
- Audio style transfer
- Real-time audio processing
- Noise cancellation AI
- Music recommendation systems

Projects built:

- Voice assistant prototype
- Music genre classifier
- AI DJ mixing system
- Sound effect generator

Practice: Create 6 audio AI applications

Reinforcement Learning Introduction

Topics:

- RL: Learning from rewards
- Agent, environment, actions, rewards
- Exploration vs exploitation
- Q-Learning basics
- Deep Q-Networks (DQN)
- OpenAI Gym environments
- Training game-playing AI
- Policy gradient methods basics
- Actor-Critic introduction
- RL for robotics

- Multi-agent RL basics
- Real-world RL applications

Projects built:

- Game-playing AI (Tic-tac-toe, Snake)
- CartPole balancer
- Maze solver with RL
- Trading bot simulator

Practice: Train 5 RL agents for different games

Phase 2 Capstone Project

Topics:

- Advanced project planning
- Integration of multiple AI systems
- Performance optimization
- User experience design
- Deployment strategies

Projects built:

- MAJOR CAPSTONE: AI Content Creation Suite
- Image generation, text creation, audio synthesis
- Alternative: AI-powered game with multiple AI components
- Alternative: Personal AI assistant with vision and NLP

Assessment: Build production-ready AI application

Phase 2 Advanced Topics (Week 24-26)

Edge AI & Mobile Deployment

Topics:

- AI on mobile devices
- TensorFlow Lite introduction
- Model quantization and pruning
- ONNX for model portability
- Core ML for iOS
- ML Kit for mobile apps
- Edge TPU and hardware acceleration
- Real-time inference optimization
- Battery-efficient AI
- Federated learning basics
- Privacy-preserving ML
- On-device vs cloud AI

Projects built:

- Mobile AI app (Android/iOS)
- Real-time object detector for phone
- Offline translation app
- AI camera application

Practice: Deploy 5 models to mobile devices

AI in Games & Simulations

Topics:

- Game AI fundamentals
- Pathfinding algorithms for games
- Behavior trees for NPCs
- Procedural content generation
- AI for game balancing
- Physics simulation with ML
- Creating intelligent NPCs
- AI dungeon master
- Procedural world generation
- AI for game testing
- Competitive AI agents
- Unity ML-Agents introduction

Projects built:

- AI opponent for custom game
- Procedural level generator
- NPC behavior system
- Game balance optimizer

Practice: Build 5 game AI systems

MLOps & Production Systems

Topics:

- ML project lifecycle
- Version control for ML (Git, DVC)
- Experiment tracking (MLflow, Weights & Biases)
- Model versioning strategies
- Continuous Integration for ML
- Model monitoring in production
- A/B testing for ML
- Model retraining pipelines
- Docker for ML applications
- Cloud platforms: AWS, GCP, Azure basics
- API creation for ML models
- Scaling ML applications

Projects built:

- ML pipeline with monitoring

- Model deployment system
- API for ML model serving
- Automated retraining system

Practice: Deploy 5 models to production

AI Ethics & Responsible AI

Topics:

- Bias in AI systems
- Fairness metrics and evaluation
- Explainable AI (XAI)
- Privacy in machine learning
- Differential privacy basics
- AI safety considerations
- Environmental impact of AI
- AI regulations and compliance
- Adversarial attacks and defenses
- Deepfakes and detection
- AI for social good
- Future of AI careers

Projects built:

- Bias detection tool
- Model explainability dashboard
- Fairness evaluation system
- AI ethics case study analysis

Practice: Evaluate 5 models for bias and fairness

Phase 2 Final Project

Topics:

- Complex system integration
- Production deployment
- Performance optimization
- Documentation and testing
- Presentation preparation

Projects built:

- COMPREHENSIVE PROJECT: Multi-Modal AI System
- Combining vision, text, and audio AI
- Production-ready deployment
- Complete documentation

Assessment: Phase 2 comprehensive evaluation

----- PHASE 3: DEEP LEARNING MASTERY & SPECIALIZATIONS (MONTHS 7-9, WEEKS 27-39) -----

Master advanced deep learning, cutting-edge AI techniques, and specialized domains like robotics and healthcare AI.

Months 7-8: Advanced Deep Learning (Week 27-35)

Advanced CNN Architectures

Topics:

- EfficientNet and model scaling
- Vision Transformers (ViT)
- MobileNet for mobile devices
- NAS (Neural Architecture Search)
- 3D CNNs for video
- Temporal convolutions
- Capsule Networks introduction
- Attention mechanisms in vision
- Self-supervised learning for vision
- Contrastive learning
- Few-shot learning
- Zero-shot image classification

Projects built:

- Custom CNN architecture designer
- Video action recognition system
- Few-shot learning implementation
- Self-supervised vision model

Practice: Implement 10 state-of-the-art architectures

Advanced NLP & Language Models

Topics:

- Large Language Models (LLMs) deep dive
- GPT architecture detailed
- BERT variants: RoBERTa, ALBERT, ELECTRA
- T5 and sequence-to-sequence
- Prompt engineering advanced
- In-context learning
- Chain-of-thought prompting
- Retrieval Augmented Generation (RAG)
- Vector databases for AI
- LangChain introduction
- Building LLM applications
- Fine-tuning strategies

Projects built:

- Custom chatbot with RAG
- Document Q&A system
- AI writing assistant advanced
- Multi-language translator

Practice: Build 8 LLM-powered applications

Multimodal AI

Topics:

- Combining vision and language
- CLIP for image-text matching
- DALL-E and text-to-image
- Stable Diffusion basics
- Image captioning models
- Visual question answering
- Video understanding with AI
- Audio-visual learning
- Cross-modal retrieval
- Multimodal transformers
- Building multimodal systems
- Applications in real world

Projects built:

- Image caption generator
- Text-to-image generator
- Visual Q&A system
- Multimodal search engine

Practice: Create 6 multimodal AI applications

Graph Neural Networks

Topics:

- Graphs in real world: Social networks, molecules
- Graph representation learning
- Graph Convolutional Networks (GCN)
- GraphSAGE algorithm
- Graph Attention Networks (GAT)
- Node classification
- Link prediction
- Graph classification
- Molecular property prediction
- Social network analysis
- Knowledge graphs and AI
- PyTorch Geometric library

Projects built:

- Social network analyzer
- Molecular property predictor
- Recommendation system with graphs
- Knowledge graph builder

Practice: Implement 5 graph neural network models

Time Series & Forecasting

Topics:

- Time series analysis advanced
- ARIMA and statistical models
- Prophet for forecasting
- Deep learning for time series
- Temporal Convolutional Networks
- Attention for time series
- Transformer models for forecasting
- Multivariate time series
- Anomaly detection in time series
- Real-time forecasting
- Uncertainty quantification
- Applications in finance, weather, sales

Projects built:

- Advanced stock predictor
- Weather forecasting system
- Sales prediction tool
- Anomaly detection system

Practice: Build 8 forecasting models

Month 8: Specialized AI Domains (Week 32-35)

Robotics & Embodied AI

Topics:

- Introduction to robotics
- Robot perception with AI
- SLAM (Simultaneous Localization and Mapping)
- Path planning algorithms
- Computer vision for robotics
- Reinforcement learning for robots
- Sim-to-real transfer
- ROS (Robot Operating System) basics
- Drone AI and autonomous flight

- Manipulation and grasping
- Human-robot interaction
- Simulation environments: Gazebo, PyBullet

Projects built:

- Virtual robot controller
- Drone path planner
- Robot vision system
- Simulated robot training

Practice: Program 5 robotic behaviors

Healthcare AI & Bioinformatics

Topics:

- AI in medical diagnosis
- Medical image analysis: X-ray, MRI, CT
- Disease prediction models
- Drug discovery with AI
- Protein structure prediction basics
- Genomics and AI
- Electronic Health Records (EHR) analysis
- Clinical NLP
- Wearable device data analysis
- Mental health AI applications
- Ethical considerations in healthcare AI
- FDA regulations and medical AI

Projects built:

- Disease symptom checker
- Medical image classifier
- Health tracker analyzer
- Mental wellness chatbot

Practice: Build 5 healthcare AI applications

AI for Science & Research

Topics:

- Scientific computing with AI
- Physics-informed neural networks
- AI for climate modeling
- Astronomy and AI
- Chemistry: Molecular design
- Materials discovery with ML
- AI in biology research
- Computational fluid dynamics with ML
- Quantum machine learning basics
- AI for mathematics
- Scientific data visualization
- Reproducible AI research

Projects built:

- Climate pattern analyzer
- Molecular property predictor
- Physics simulation with AI
- Scientific data explorer

Practice: Apply AI to 5 scientific problems

Creative AI & Art

Topics:

- AI in creative industries
- Music generation advanced
- AI for video editing
- 3D model generation
- AI in game design
- Creative writing with AI
- AI art styles and techniques
- Animation with AI
- Voice synthesis and singing
- AI for fashion design
- Interactive art installations
- AI art: authorship, licensing and credit

Projects built:

- AI music album creator
- Story generator with illustrations
- AI video editor
- 3D model generator

Practice: Create 10 AI art projects

Advanced Reinforcement Learning

Topics:

- Policy Gradient methods deep dive
- Proximal Policy Optimization (PPO)
- Advantage Actor-Critic (A2C/A3C)
- DDPG for continuous control
- SAC (Soft Actor-Critic)
- Model-based RL
- Inverse RL
- Multi-agent reinforcement learning

- Hierarchical RL
- Meta-learning basics
- RL for real-world problems
- Safety in RL systems

Projects built:

- Advanced game AI agent
- Robot control with RL
- Multi-agent simulation
- Real-world optimization with RL

Practice: Train 8 advanced RL agents

Month 9: Cutting-Edge AI & Research (Week 36-39)

Generative AI Revolution

Topics:

- Diffusion models deep dive
- Stable Diffusion architecture
- ControlNet for controlled generation
- InstructPix2Pix
- 3D generation: NeRF, 3D GANs
- Video generation models
- Audio generation: MusicLM, AudioLM
- Code generation with AI
- Generative agents and simulations
- AI for procedural generation
- Ethical implications of generative AI
- Future of generative models

Projects built:

- Custom image generator
- AI video creator
- 3D scene generator
- Code generation assistant

Practice: Build 10 generative AI applications

Neural Architecture Search & AutoML

Topics:

- Automated Machine Learning (AutoML)
- Neural Architecture Search (NAS)
- Evolutionary algorithms for NAS
- Reinforcement learning for NAS
- Differentiable architecture search
- Hardware-aware NAS
- AutoML tools: AutoKeras, AutoGluon
- Hyperparameter optimization advanced
- Bayesian optimization
- Meta-learning for quick adaptation
- Few-shot and zero-shot learning
- Transfer learning optimization

Projects built:

- AutoML system builder
- Custom NAS implementation
- Model optimization toolkit
- Transfer learning framework

Practice: Optimize 10 models with AutoML

Explainable AI & Interpretability

Topics:

- Why explainability matters
- SHAP (SHapley Additive exPlanations)
- LIME for local interpretability
- Attention visualization
- Saliency maps and GradCAM
- Counterfactual explanations
- Concept activation vectors
- Model cards and documentation
- Fairness metrics and testing
- Debugging neural networks
- Trust in AI systems
- Regulatory requirements for XAI

Projects built:

- XAI dashboard
- Model interpretation toolkit
- Fairness analyzer
- AI decision explainer

Practice: Make 8 models explainable

Federated Learning & Privacy

Topics:

- Federated learning concept
- Decentralized training
- Privacy-preserving ML
- Differential privacy implementation
- Homomorphic encryption basics

- Secure multi-party computation
- Split learning
- Federated averaging algorithm
- Communication-efficient FL
- Personalized federated learning
- FL frameworks: TFF, PySyft
- Real-world FL applications

Projects built:

- Federated learning system
- Privacy-preserving model
- Decentralized AI application
- Secure ML pipeline

Practice: Implement 5 privacy-preserving ML systems

Phase 3 Capstone Project

Topics:

- Research project planning
- Literature review process
- Experiment design
- Baseline comparisons
- Result analysis and visualization
- Research paper writing basics

Projects built:

- RESEARCH PROJECT: Novel AI Application
- Original research or significant improvement
- Complete experimentation and evaluation
- Research paper draft

Assessment: Present research findings and demonstrations

----- PHASE 4: PROFESSIONAL AI DEVELOPMENT & CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52) -----

Build production-grade AI systems, contribute to open source, prepare for AI careers, and launch your AI startup or research path.

Months 10-11: Production AI Systems (Week 40-48)

Scalable AI Infrastructure

Topics:

- Distributed training strategies
- Data parallelism and model parallelism
- Gradient accumulation and mixed precision
- Multi-GPU training with PyTorch/TensorFlow
- Distributed training frameworks: Horovod, DeepSpeed
- Cloud platforms for AI: AWS SageMaker, GCP Vertex AI
- Kubernetes for ML workloads
- Model serving at scale
- Load balancing for ML APIs
- Caching strategies for inference
- Edge deployment strategies
- Cost optimization for AI infrastructure

Projects built:

- Distributed training pipeline
- Scalable model serving system
- Multi-cloud AI deployment
- Cost-optimized inference system

Practice: Deploy 10 models at scale

Real-time AI Systems

Topics:

- Streaming data processing
- Real-time inference optimization
- Model quantization techniques
- Knowledge distillation
- TensorRT optimization
- ONNX runtime optimization
- WebAssembly for browser AI
- WebGL/WebGPU acceleration
- Stream processing with Kafka
- Real-time feature engineering
- Online learning systems
- Low-latency serving patterns

Projects built:

- Real-time video analysis system
- Live translation application
- Streaming anomaly detector
- Browser-based AI application

Practice: Build 8 real-time AI systems

AI Product Development

Topics:

- Product management for AI
- User research for AI products

- AI product metrics and KPIs
- A/B testing AI features
- Iterative improvement cycles
- User feedback integration
- AI product roadmapping
- Competitive analysis
- Monetization strategies
- AI product marketing
- Launch strategies
- Growth hacking with AI

Projects built:

- AI product MVP
- User testing framework
- Product analytics dashboard
- Go-to-market strategy

Practice: Launch 3 AI products to users

AI Startups & Entrepreneurship

Topics:

- AI startup landscape
- Identifying AI opportunities
- Building an AI team
- Fundraising for AI startups
- AI startup pitch deck
- Intellectual property in AI
- Open source vs proprietary
- AI startup case studies
- Scaling AI startups
- Exit strategies
- AI startup failures and lessons
- Building AI communities

Projects built:

- AI startup concept development
- Pitch deck creation
- MVP for startup idea
- Business plan for AI venture

Practice: Develop 5 AI startup ideas

Open Source Contribution

Topics:

- Open source AI ecosystem
- Contributing to major projects
- Creating your own OS project
- Documentation best practices
- Community building
- Issue management
- Code review processes
- CI/CD for open source
- License selection
- Maintaining OS projects
- Building contributor community
- Open source monetization

Projects built:

- Contribute to major AI project
- Launch own open source tool
- Create comprehensive documentation
- Build contributor community

Practice: Make 20 open source contributions

Month 11: Advanced Applications & Industry (Week 44-48)

AI in Finance & Trading

Topics:

- Algorithmic trading basics
- Market prediction models
- Risk assessment with AI
- Fraud detection systems
- Credit scoring models
- Portfolio optimization
- Sentiment analysis for trading
- High-frequency trading basics
- Regulatory compliance in FinTech
- AI in finance: fraud detection and risk scoring
- Robo-advisors
- Quantitative analysis with ML

Projects built:

- Trading bot simulator
- Fraud detection system
- Credit risk model
- Market sentiment analyzer

Practice: Build 6 FinTech AI applications

AI for Social Media & Content

Topics:

- Content recommendation algorithms
- Social media analytics
- Influencer identification
- Trend prediction
- Content moderation with AI
- Fake news detection
- Social network analysis
- Viral content prediction
- Personalization algorithms
- Ad targeting with AI
- Content generation for social
- Community detection

Projects built:

- Social media analytics tool
- Content recommendation engine
- Trend prediction system
- Influencer analyzer

Practice: Create 8 social media AI tools

AI in Education & EdTech

Topics:

- Personalized learning systems
- Intelligent tutoring systems
- Automated grading
- Student performance prediction
- Learning path optimization
- Content generation for education
- Plagiarism detection
- Virtual teaching assistants
- Educational game AI
- Accessibility in EdTech
- Learning analytics
- Adaptive testing systems

Projects built:

- Personalized tutor bot
- Automated grading system
- Learning path optimizer
- Educational game with AI

Practice: Build 6 EdTech AI applications

AI for Sustainability & Climate

Topics:

- Climate modeling with AI
- Energy consumption optimization
- Smart grid management
- Renewable energy prediction
- Carbon footprint tracking
- Wildlife conservation AI
- Precision agriculture
- Water resource management
- Disaster prediction and response
- Sustainable supply chains
- Environmental monitoring
- Green AI practices

Projects built:

- Energy optimization system
- Climate impact predictor
- Smart agriculture advisor
- Disaster response system

Practice: Create 5 sustainability AI solutions

Future of AI & Emerging Tech

Topics:

- AGI (Artificial General Intelligence) concepts
- Quantum machine learning
- Neuromorphic computing
- Brain-computer interfaces
- Synthetic biology and AI
- Swarm intelligence
- Emotional AI
- AI consciousness debates
- Regulation and governance
- AI safety research
- Long-term AI impacts
- Career paths in AI

Projects built:

- Future AI concept prototype
- Emerging tech exploration project
- AI impact assessment tool
- Career planning portfolio

Practice: Explore 5 emerging AI technologies

Month 12: Mastery & Career Launch (Week 49-52)

AI Research Methods

Topics:

- Reading research papers effectively
- Reproducing research results
- Designing experiments
- Statistical significance testing
- Ablation studies
- Benchmarking methodologies
- Writing technical reports
- Creating research proposals
- Peer review process
- Conference submissions
- Building research portfolio
- Collaboration in research

Projects built:

- Reproduce famous paper
- Original research project
- Research paper draft
- Conference presentation

Practice: Complete 3 research projects

Industry Preparation

Topics:

- AI job landscape
- Resume building for AI roles
- Portfolio optimization
- Interview preparation
- Technical assessments
- System design for AI
- Behavioral interviews
- Salary negotiation
- Internship strategies
- Networking in AI community
- Personal branding
- Continuous learning plans

Projects built:

- Professional portfolio website
- Interview preparation materials
- Technical blog posts
- LinkedIn optimization

Practice: Complete 20 mock interviews

Competitions & Challenges

Topics:

- Kaggle competition strategies
- Google AI challenges
- Microsoft AI competitions
- Research competitions
- Hackathon participation
- Team formation strategies
- Competition portfolio building
- Prize-winning techniques
- Post-competition analysis
- Building reputation
- Becoming Kaggle Master
- Competition to career pipeline

Projects built:

- Kaggle competition entry
- Hackathon project
- Competition portfolio
- Team collaboration project

Practice: Participate in 10 AI competitions

AI Community & Impact

Topics:

- Building AI communities
- Teaching and mentoring
- Creating educational content
- AI for social good projects
- Diversity in AI
- Ethical AI advocacy
- Policy and regulation engagement
- Science communication
- Public speaking about AI
- Media engagement
- Thought leadership
- Giving back to community

Projects built:

- Community project launch
- Educational content creation

- Social good AI application
 - Mentorship program design
- Practice: Lead 5 community initiatives

Final Month: Capstone & Graduation (Week 50-52)

Final Capstone - Part 1

Topics:

- Capstone project ideation
- Problem statement definition
- Literature review
- Technical architecture design
- Data collection and preparation
- Model development
- Evaluation framework
- User interface design
- Testing strategies
- Documentation planning
- Timeline management
- Risk assessment

Projects built:

- FINAL CAPSTONE: Complete AI Platform
- Novel AI application with real impact
- Full-stack implementation
- Research component included
- Options: Healthcare AI, Education AI, Creative AI, Social Good AI

Final Capstone - Part 2

Topics:

- Implementation completion
- System integration
- Performance optimization
- Security implementation
- User testing
- Deployment to production
- Monitoring setup
- Documentation finalization
- Video demonstration
- Presentation preparation
- Peer review
- Final polishing

Deliverables:

- Complete source code with documentation
- Live deployed application
- Technical paper (10+ pages)
- Video presentation (15 minutes)
- User guide and API documentation
- Test suite with 80%+ coverage
- Performance benchmarks
- Future development roadmap

Graduation & Future Path

Topics:

- Portfolio finalization
- Career path planning
- University preparation (if applicable)
- Scholarship applications
- Internship applications
- Research opportunities
- Startup incubators
- Further learning resources
- Alumni network
- Mentorship continuation
- Life-long learning in AI
- Celebration and reflection

Deliverables:

- Complete AI portfolio (40+ projects)
- Professional website and blog
- GitHub profile with contributions
- LinkedIn profile optimized
- Resume for AI roles
- Research publications (if any)
- Competition achievements
- Network of AI professionals
- Clear career roadmap
- Certification of completion

Assessment: FINAL ASSESSMENT: Comprehensive evaluation + Capstone presentation

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Chatbots, Recommendation Systems, Image Classifiers, Data Analyzers
- Phase 2: CNNs, RNNs, GANs, NLP Applications, Computer Vision Projects

- Phase 3: Advanced Deep Learning, Multimodal AI, Robotics, Healthcare AI
- Phase 4: Production Systems, Startups, Research Projects, Industry Applications
- Final: Complete AI Platform with Multiple Components

Total projects: 40+ AI projects from simple to production-grade

SKILLS THE STUDENT WILL HAVE AT THE END

- Programming: Python, TensorFlow, PyTorch, Scikit-learn, Keras
- Mathematics: Linear Algebra, Calculus, Statistics, Probability
- Machine Learning: Classification, Regression, Clustering, Dimensionality Reduction
- Deep Learning: CNNs, RNNs, Transformers, GANs, Autoencoders
- Computer Vision: Object Detection, Segmentation, Face Recognition, OCR
- NLP: Text Classification, Generation, Translation, Question Answering
- Reinforcement Learning: Q-Learning, Policy Gradients, Game AI
- Generative AI: Stable Diffusion, GPT, DALL-E, Music Generation
- MLOps: Deployment, Monitoring, Scaling, Version Control
- Tools: Jupyter, Git, Docker, Cloud Platforms, APIs
- Specialized: Robotics, Healthcare, Finance, Education, Sustainability
- Soft Skills: Research, Communication, Ethics, Project Management

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-5 hours
 Hands on coding: 5-6 hours
 Projects: 3-4 hours
 Research reading: 1-2 hours
 Community interaction: 1 hour
 Total per week: 12-15 hours

SUPPORT INCLUDED

Live sessions: Weekly Q&A with AI experts
 Mentorship: 1-on-1 guidance from AI professionals
 Peer groups: Study groups of 4-5 students
 Community: Active Discord server for AI enthusiasts
 Code review: Expert feedback on projects
 Gpu access: Free cloud GPU credits for training
 Research guidance: Help with research projects
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Specialization badges: Badges for CV, NLP, RL, etc.
 Final certificate: AI/ML Expert Certification
 Linkedin credentials: Verifiable LinkedIn certificates
 Project showcase: Public portfolio of 40+ projects
 Recommendation letters: From instructors for top performers

WHERE THIS CAN LEAD

- AI/ML Engineer (entry-level after college)
- Data Scientist
- Computer Vision Engineer
- NLP Engineer
- AI Researcher (with further education)
- AI Product Manager
- Robotics Engineer
- AI Startup Founder
- AI Ethics Consultant
- AI Content Creator/Educator

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What AI projects will my teen actually build?

A: Across the year students build 40+ projects: chatbots, image recognition systems, recommendation engines, game-playing AIs, and generative AI experiments, ending with a production-style capstone. The course explains the ideas behind familiar technology, such as how streaming services recommend titles or how image filters work, then has students build their own versions.

Q: How much maths and coding does my teen need beforehand?

A: Only basic algebra and no coding at all; month 1 teaches the Python needed for AI from scratch. The necessary mathematics is introduced visually and gradually alongside the projects rather than as a wall of theory upfront.

Q: What age is this AI course meant for?

A: It is written for teenagers aged 13 to 19, with examples and projects picked for teenage interests. We teach coding and maths to students from ages 6 to 65, and this course sits squarely in the serious-teen zone: ambitious but genuinely beginner-friendly.

Q: What can my teen do after finishing?

A: They will understand AI deeply enough to keep building on their own: stronger models, research-style experiments, and portfolio projects for college applications. Students who want more foundation often pair this with our Python masterclass or the problem solving and DSA course.

Q: How do we try a class before enrolling?

A: Book a free demo class at learn.modernagecoders.com/contact; there is no commitment, and parents are welcome to sit in. Families rate us 4.9 stars across 547+ Google reviews, and the demo is the fastest way to see why.

Q: Are classes live with a teacher?

A: Yes, always: 1-hour live online classes with a real teacher, never pre-recorded video content. AI concepts stick when students can ask why something works mid-lesson, so classes stay small and interactive by design.

Q: Do students receive a certificate?

A: Yes, the course ends with an AI/ML certification supported by a portfolio of 40+ working projects. When your teen can demonstrate a chatbot or an image recognizer they built themselves, the certificate becomes the footnote, not the headline.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ai-ml-masterclass-teens>

=====

COURSE 6 OF 118: AI & MACHINE LEARNING MASTERCLASS: ZERO TO AI EXPERT

=====

Course link: <https://learn.modernagecoders.com/courses/ai-ml-masterclass-complete-college>

Subject area: Artificial Intelligence & Machine Learning

Designed for: College students

Who can join: Complete Beginner to AI/ML Expert

Course length: 12 months (52 weeks)

Time needed: 20-25 hours/week recommended

Certificate: Industry-recognized AI/ML Engineer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 12-month AI and Machine Learning program: master Python, mathematics, classical ML, deep learning, NLP, computer vision, MLOps and deploy models at scale.

In one line: From Mathematics Fundamentals to Production AI Systems

ABOUT THE PROGRAMME

This is not just an AI course, it's a complete transformation into an AI/ML professional. Whether you're a beginner with no technical background, a developer wanting to transition into AI, or a data analyst aiming to become an ML engineer, this 12-month masterclass will turn you into a highly skilled AI practitioner capable of building, training, deploying, and maintaining production-grade machine learning systems. You'll master AI/ML from ground zero to expert level: from mathematics and Python programming to classical machine learning algorithms, from deep learning fundamentals to advanced architectures like Transformers, from theory to production deployment with MLOps. By the end, you'll have built 50+ ML projects, created AI models from scratch, deployed them to production, and be ready for AI/ML engineer roles at top tech companies.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - mathematics, programming, everything
- Complete 12-month structured curriculum
- Mathematics for ML taught from scratch
- Hands-on with latest AI frameworks (TensorFlow, PyTorch, Hugging Face)
- Real industry projects and Kaggle competitions
- MLOps and production deployment focus
- Computer vision, NLP, and reinforcement learning covered
- Interview preparation for FAANG ML roles
- Lifetime access and continuous updates
- Build impressive AI portfolio with 50+ projects
- Research paper implementation and understanding

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Mathematics, Python, Statistics, Data Analysis
Phase 2: Classical ML (Months 4-6): ML Algorithms, Scikit-learn, Feature Engineering, Model Evaluation
Phase 3: Deep Learning (Months 7-9): Neural Networks, CNN, RNN, NLP, Computer Vision, TensorFlow/PyTorch
Phase 4: Advanced AI (Months 10-12): Transformers, GANs, RL, MLOps, Production Deployment, Research

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required, but basic high school math helpful
Coding experience: Beginner-friendly, but some programming knowledge beneficial
Mathematics: Basic algebra (will be taught from scratch)
Age: 16+ years recommended (due to mathematical complexity)
Equipment: Computer with good specs (8GB+ RAM), GPU recommended but not required
Time commitment: 20-25 hours per week consistently
English: Good reading comprehension (research papers)
Motivation: Strong passion for AI and willingness to learn continuously
Learning style: Self-motivated, enjoys problem-solving and mathematics

WHO THIS COURSE IS FOR

Beginners: Those with no AI/ML background wanting comprehensive foundation
Developers: Software engineers transitioning to AI/ML engineering
Data analysts: Analysts wanting to become data scientists/ML engineers
Students: Computer science or engineering students preparing for AI careers
Researchers: Those aspiring to become research scientists
Professionals: Working professionals upskilling to AI/ML roles
Entrepreneurs: Founders wanting to build AI products
Phd aspirants: Those preparing for ML/AI PhD programs
Career switchers: Anyone from any field wanting to enter AI
Advanced learners: Those wanting structured learning of end-to-end AI/ML

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: MATHEMATICS, PROGRAMMING & DATA FOUNDATIONS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid mathematical and programming foundations essential for AI/ML. Master linear algebra, calculus, statistics, Python, and data manipulation.

Months 1-2: Mathematics for Machine Learning (Week 1-8)

Mathematics Prerequisites & Linear Algebra - Part 1

Topics:

- Why mathematics matters in AI/ML
- Basic arithmetic and algebra review
- Functions and graphs
- Introduction to vectors: geometric and algebraic view
- Vector operations: addition, scalar multiplication
- Dot product and geometric interpretation
- Vector norms (L1, L2, infinity norm)
- Matrices: definition and notation
- Matrix operations: addition, multiplication
- Matrix transpose and symmetric matrices
- Identity matrix and inverse matrices
- Matrix properties and rules
- Systems of linear equations
- Gaussian elimination
- Matrix rank and determinant (basics)

Projects built:

- Vector operations visualizer
- Matrix calculator implementation
- Linear equation solver
- Geometric transformations with matrices

Practice: Solve 50 linear algebra problems, implement from scratch in Python

Linear Algebra - Part 2 & Calculus Fundamentals

Topics:

- Eigenvalues and eigenvectors: concept and computation
- Eigendecomposition
- Singular Value Decomposition (SVD)
- Principal Component Analysis (PCA) mathematics
- Vector spaces and subspaces
- Linear transformations
- Introduction to calculus: limits

- Derivatives: definition and rules
- Power rule, product rule, quotient rule, chain rule
- Partial derivatives
- Gradient: vector of partial derivatives
- Gradient descent concept
- Second derivatives and Hessian matrix
- Taylor series expansion
- Optimization basics: finding minima and maxima

Projects built:

- PCA implementation from scratch
- Image compression using SVD
- Gradient descent visualizer
- Function optimizer using calculus

Practice: Solve 60 calculus problems, implement gradient descent variants

Probability & Statistics - Part 1

Topics:

- Probability fundamentals: sample space, events
- Probability rules: addition, multiplication
- Conditional probability and Bayes' theorem
- Independent vs dependent events
- Random variables: discrete and continuous
- Probability distributions: uniform, Bernoulli, binomial
- Normal (Gaussian) distribution: properties and applications
- Poisson distribution
- Exponential distribution
- Probability density functions (PDF)
- Cumulative distribution functions (CDF)
- Expected value (mean) and variance
- Standard deviation and standard error
- Covariance and correlation
- Law of large numbers and Central Limit Theorem

Projects built:

- Probability calculator
- Distribution visualizer
- Bayes theorem applications
- Monte Carlo simulations
- Statistical analysis tool

Practice: Solve 70 probability and statistics problems

Statistics - Part 2 & Python for ML

Topics:

- Descriptive statistics: mean, median, mode
- Measures of spread: range, IQR, variance, std dev
- Hypothesis testing fundamentals
- Null and alternative hypotheses
- P-values and significance levels
- T-tests, Z-tests, Chi-square tests
- ANOVA (Analysis of Variance)
- Confidence intervals
- Statistical inference
- Python setup for ML: Anaconda, Jupyter
- NumPy for numerical computing
- Pandas for data manipulation
- Matplotlib and Seaborn for visualization
- Statistical computing with SciPy

Projects built:

- Hypothesis testing framework
- A/B testing simulator
- Statistical analysis dashboard
- Data exploration toolkit with Python
- Interactive visualization app

Practice: Statistical analysis of 10 real datasets

Month 3: Data Analysis & Exploratory Data Analysis (Week 9-13)

Advanced NumPy & Pandas

Topics:

- Advanced NumPy: broadcasting, vectorization
- NumPy for linear algebra
- NumPy random number generation
- Pandas DataFrames: advanced operations
- Data cleaning: handling missing values
- Data transformation: apply, map, applymap
- GroupBy operations and aggregations
- Merging, joining, and concatenating
- Time series data handling
- Categorical data handling
- Multi-index DataFrames
- Performance optimization in Pandas

Projects built:

- Complete data cleaning pipeline

- Time series analysis tool
- Data aggregation and reporting system
- Advanced data transformations
- Performance benchmarking study

Practice: Clean and analyze 15 messy datasets

Data Visualization & EDA

Topics:

- Principles of data visualization
- Matplotlib deep dive: customization
- Seaborn for statistical plots
- Plotly for interactive visualizations
- Distribution plots: histograms, KDE, box plots
- Relationship plots: scatter, line, regression
- Categorical plots: bar, count, violin
- Heatmaps and correlation matrices
- Pair plots and joint plots
- Exploratory Data Analysis (EDA) process
- Identifying patterns and anomalies
- Feature correlation analysis
- Handling outliers
- Data storytelling with visualizations

Projects built:

- Complete EDA on Titanic dataset
- House prices EDA and insights
- Customer segmentation visualization
- COVID-19 data analysis and dashboard
- Interactive data exploration tool

Practice: Perform comprehensive EDA on 10 datasets

SQL for Data Science & Phase 1 Review

Topics:

- SQL fundamentals for data analysis
- Querying databases: SELECT, WHERE, ORDER BY
- Aggregations: GROUP BY, HAVING
- Joins: INNER, LEFT, RIGHT, FULL
- Subqueries and CTEs
- Window functions for analytics
- Working with dates and times
- SQL optimization basics
- Connecting Python to databases
- SQLAlchemy for data extraction
- Phase 1 comprehensive review

Projects built:

- PHASE 1 MINI CAPSTONE: Complete Data Analysis Project
- Dataset: Choose from Kaggle (e.g., Retail sales, Healthcare data)
- Tasks: Data extraction (SQL), cleaning (Pandas), EDA, statistical analysis, visualization, insights report

Assessment: Phase 1 Final Exam - Mathematics, Statistics, Python, Data Analysis

----- PHASE 2: CLASSICAL MACHINE LEARNING (MONTHS 4-6, WEEKS 14-26) -----

Master traditional machine learning algorithms, feature engineering, model evaluation, and scikit-learn.

Months 4-5: Machine Learning Fundamentals & Supervised Learning (Week 14-22)

Introduction to Machine Learning

Topics:

- What is Machine Learning? AI vs ML vs DL
- Types of ML: supervised, unsupervised, reinforcement
- ML workflow: problem definition to deployment
- Data preparation for ML
- Train-test split and validation set
- Cross-validation: k-fold, stratified k-fold
- Overfitting and underfitting
- Bias-variance tradeoff
- Evaluation metrics: accuracy, precision, recall, F1
- Confusion matrix
- ROC curve and AUC
- Scikit-learn library introduction
- Linear Regression: theory and mathematics
- Gradient descent for linear regression
- Cost function (MSE) and optimization

Projects built:

- House price prediction with linear regression
- Salary prediction model
- Sales forecasting
- Custom linear regression from scratch
- Gradient descent visualizer

Practice: Implement linear regression from scratch, solve 30 regression problems

Classification Algorithms - Part 1

Topics:

- Logistic Regression: binary classification
- Sigmoid function and probability interpretation
- Log loss (binary cross-entropy)
- Multi-class classification: one-vs-rest, softmax
- Decision Trees: splitting criteria (Gini, entropy)
- Information gain and entropy
- Tree pruning to prevent overfitting
- Random Forest: ensemble of trees
- Bagging and bootstrap aggregating
- Feature importance in Random Forest
- K-Nearest Neighbors (KNN): distance-based learning
- Choosing K and distance metrics
- Naive Bayes classifier: probabilistic approach
- Support Vector Machines (SVM): margin maximization
- Kernel trick for non-linear classification

Projects built:

- Email spam classifier (Naive Bayes)
- Iris flower classification (multi-class)
- Credit card fraud detection
- Customer churn prediction
- Disease prediction (Random Forest)
- Handwritten digit classification (KNN)
- Text classification with SVM

Practice: Build 15 classification models on different datasets

Feature Engineering & Preprocessing

Topics:

- Feature engineering importance
- Feature scaling: standardization vs normalization
- Min-Max scaling and Standard scaling
- Robust scaling for outliers
- Handling categorical features: one-hot encoding
- Label encoding and ordinal encoding
- Target encoding
- Feature creation: polynomial features
- Interaction features
- Binning and discretization
- Text features: Bag of Words, TF-IDF
- Feature selection: filter methods
- Wrapper methods: forward/backward selection
- Embedded methods: Lasso, Ridge
- Dimensionality reduction: PCA

Projects built:

- Feature engineering pipeline
- Automated feature selection tool
- Text feature extraction system
- Dimensionality reduction visualizer
- Complete preprocessing pipeline

Practice: Engineer features for 20 different datasets

Unsupervised Learning

Topics:

- Clustering: grouping similar data
- K-Means clustering: algorithm and initialization
- Elbow method for choosing K
- Hierarchical clustering: agglomerative and divisive
- Dendrograms
- DBSCAN: density-based clustering
- Gaussian Mixture Models (GMM)
- Anomaly detection techniques
- Dimensionality reduction: PCA (practical)
- t-SNE for visualization
- UMAP for dimensionality reduction
- Association rule mining: Apriori algorithm
- Market basket analysis
- Autoencoders for unsupervised learning (intro)

Projects built:

- Customer segmentation with K-Means
- Image compression with PCA
- Anomaly detection in transactions
- Document clustering
- Market basket analysis for retail
- High-dimensional data visualization
- Recommendation system basics

Practice: Cluster 15 different datasets, visualize with t-SNE/UMAP

Ensemble Methods & Advanced Techniques

Topics:

- Ensemble learning: wisdom of crowds
- Bagging: Random Forest deep dive

- Boosting concept: sequential learning
- AdaBoost: Adaptive Boosting
- Gradient Boosting: iterative optimization
- XGBoost: extreme gradient boosting
- LightGBM: light gradient boosting machine
- CatBoost for categorical features
- Stacking and blending
- Voting classifiers
- Hyperparameter tuning: Grid Search
- Random Search and Bayesian Optimization
- AutoML introduction

Projects built:

- Kaggle competition with ensemble methods
- XGBoost vs LightGBM comparison
- Stacked ensemble model
- Hyperparameter tuning framework
- Complete ML pipeline with best practices

Practice: Win or achieve top 10% in 2 Kaggle competitions

Month 6: Advanced ML & Time Series (Week 23-26)

Model Evaluation & Validation

Topics:

- Evaluation metrics deep dive
- Classification metrics: precision, recall, F1, AUC-ROC
- Multi-class metrics: macro vs micro averaging
- Regression metrics: MAE, MSE, RMSE, R^2
- Custom metrics creation
- Cross-validation strategies
- Stratified sampling
- Time series cross-validation
- Model interpretation: SHAP values
- LIME for local interpretability
- Feature importance analysis
- Partial dependence plots
- Model debugging and error analysis
- Bias and fairness in ML models
- A/B testing for ML models

Projects built:

- Model evaluation framework
- Model interpretation dashboard
- Bias detection tool
- A/B testing simulator
- Error analysis toolkit

Practice: Evaluate and interpret 20 different models

Time Series Analysis & Forecasting

Topics:

- Time series data characteristics
- Trend, seasonality, and noise
- Stationarity and differencing
- Autocorrelation and partial autocorrelation
- Moving averages: simple and exponential
- ARIMA models: AR, MA, ARMA, ARIMA
- SARIMA for seasonal data
- Prophet for forecasting
- LSTM for time series (preview)
- Feature engineering for time series
- Evaluation metrics for forecasting
- Cross-validation for time series
- Handling missing data in time series
- Multi-step forecasting

Projects built:

- Stock price prediction
- Sales forecasting system
- Energy consumption prediction
- Weather forecasting
- Website traffic prediction
- Prophet vs ARIMA comparison

Practice: Forecast 10 different time series datasets

Recommender Systems

Topics:

- Types of recommendation systems
- Content-based filtering
- Collaborative filtering: user-based, item-based
- Matrix factorization: SVD for recommendations
- Similarity metrics: cosine, Pearson correlation
- Hybrid recommendation systems
- Cold start problem solutions
- Evaluation metrics for recommendations
- Implicit vs explicit feedback

- Scalability challenges
- Deep learning for recommendations (intro)
- Real-world recommendation systems architecture

Projects built:

- Movie recommendation system (MovieLens)
- Product recommendation engine
- Music recommender
- Content-based news recommender
- Hybrid recommendation system
- Cold start handling implementation

Practice: Build 5 different types of recommender systems

Advanced Topics & Specialized ML

Topics:

- Imbalanced classification: SMOTE, class weights
- Cost-sensitive learning
- Multi-label classification
- Multi-output regression
- Online learning and incremental learning
- Active learning strategies
- Transfer learning basics
- Semi-supervised learning
- One-class classification
- Survival analysis basics
- Causal inference introduction
- Federated learning overview

Projects built:

- Imbalanced dataset classifier
- Multi-label text classification
- Online learning system
- Active learning implementation
- Transfer learning experiment

Practice: Solve 10 specialized ML problems

Phase 2 Capstone Project

Topics:

- Complete ML pipeline development
- Problem definition and data collection
- EDA and feature engineering
- Model selection and training
- Hyperparameter tuning
- Model evaluation and interpretation
- Documentation and presentation

Projects built:

- PHASE 2 CAPSTONE: End-to-End ML Project
- Option 1: Predict customer lifetime value (regression + classification)
- Option 2: Build complete recommender system
- Option 3: Time series forecasting for business metrics
- Option 4: Kaggle competition (achieve top 10%)
- Requirements: Complete pipeline, feature engineering, ensemble methods, model interpretation, detailed report

Assessment: Phase 2 Final Exam - Classical ML comprehensive test

PHASE 2 CONTINUED - Production ML Basics (Week 14-26 (distributed))

ML Model Deployment Basics

Topics:

- ML model lifecycle
- Model serialization: pickle, joblib
- Flask for ML APIs
- FastAPI for high-performance APIs
- REST API design for ML
- Input validation and preprocessing
- Model serving basics
- Docker for ML applications
- Creating ML microservices
- Model versioning strategies
- A/B testing deployed models
- Monitoring model performance

Projects built:

- ML model API with Flask
- FastAPI ML service
- Dockerized ML application
- Model versioning system
- Simple ML deployment pipeline

Practice: Deploy 10 ML models as APIs

Introduction to Deep Learning

Topics:

- Why deep learning?
- Neural networks basics: perceptron
- Activation functions: sigmoid, tanh, ReLU
- Forward propagation

- Backpropagation algorithm
- Gradient descent variants: SGD, momentum, Adam
- Loss functions for deep learning
- TensorFlow basics
- Keras Sequential API
- Building first neural network
- Training deep networks
- Regularization: dropout, L1, L2
- Batch normalization

Projects built:

- Neural network from scratch (NumPy)
- MNIST digit classification (MLP)
- Binary classification with NN
- Multi-class classification NN
- Regression with neural networks

Practice: Build 10 neural networks with Keras

Version Control & Experiment Tracking

Topics:

- Git for ML projects
- DVC (Data Version Control)
- Versioning datasets and models
- MLflow for experiment tracking
- Logging metrics and parameters
- Weights & Biases (wandb)
- Comparing experiments
- Reproducibility in ML
- Managing ML artifacts
- Jupyter notebooks best practices
- Documentation for ML projects
- Collaboration in ML teams

Projects built:

- DVC setup for ML project
- MLflow experiment tracking
- Reproducible ML pipeline
- Experiment comparison dashboard
- Collaborative ML project

Practice: Version control all ML projects

ML Engineering Best Practices

Topics:

- Code organization for ML
- Configuration management
- Automated testing for ML
- Unit tests for data and models
- CI/CD for ML projects
- Data validation
- Model validation strategies
- Feature store concepts
- ML pipelines with Apache Airflow
- Scheduling ML tasks
- Error handling in ML systems
- Logging and monitoring

Projects built:

- Well-structured ML project
- ML testing suite
- CI/CD pipeline for ML
- Automated ML workflow
- Data validation framework

Practice: Refactor all projects with best practices

Big Data for ML (Introduction)

Topics:

- Big data challenges in ML
- Apache Spark basics
- PySpark for distributed computing
- Spark MLlib for scalable ML
- Distributed data processing
- Dask for parallel computing
- Handling large datasets
- Sampling strategies
- Online learning for big data
- GPU computing basics
- Cloud platforms for ML: AWS, GCP, Azure

Projects built:

- PySpark ML pipeline
- Distributed data processing
- Large-scale classification
- Dask for out-of-memory datasets
- Cloud ML experiment

Practice: Process 5 large datasets (>1GB)

PHASE 3: DEEP LEARNING & SPECIALIZED AI (MONTHS 7-9, WEEKS 27-39)

Master deep learning, computer vision, natural language processing, and specialized AI domains.

Months 7-8: Deep Learning & Computer Vision (Week 27-35)

Convolutional Neural Networks (CNN)

Topics:

- Limitations of fully connected networks for images
- Convolution operation: filters and feature maps
- Padding and stride
- Pooling layers: max pooling, average pooling
- CNN architecture components
- LeNet architecture
- AlexNet and ImageNet revolution
- VGGNet: deep and simple
- Inception/GoogLeNet: multi-scale features
- ResNet: residual connections and skip connections
- Transfer learning with pretrained models
- Fine-tuning strategies
- Data augmentation for images

Projects built:

- Image classification with CNN
- CIFAR-10 classification
- Transfer learning with ResNet
- Custom CNN architecture
- Dog vs Cat classifier
- Data augmentation pipeline
- Feature visualization in CNNs

Practice: Build 15 computer vision models

Advanced Computer Vision

Topics:

- Object detection: R-CNN, Fast R-CNN, Faster R-CNN
- YOLO (You Only Look Once): real-time detection
- SSD (Single Shot Detector)
- Semantic segmentation: FCN, U-Net
- Instance segmentation: Mask R-CNN
- Face recognition and verification
- Siamese networks
- Image generation introduction
- Style transfer with CNNs
- OpenCV for computer vision
- Image preprocessing techniques
- Handling class imbalance in CV
- Model optimization for edge devices

Projects built:

- Object detection system (YOLO)
- Face recognition application
- Image segmentation for medical images
- Style transfer implementation
- Real-time object detection
- Custom dataset object detector
- Siamese network for similarity

Practice: Complete 10 computer vision projects

Recurrent Neural Networks (RNN)

Topics:

- Sequential data and RNN motivation
- RNN architecture and forward pass
- Backpropagation Through Time (BPTT)
- Vanishing and exploding gradients
- LSTM (Long Short-Term Memory): gates and cell state
- GRU (Gated Recurrent Unit)
- Bidirectional RNNs
- Sequence-to-sequence models
- Encoder-decoder architecture
- Attention mechanism
- Teacher forcing
- Applications: language modeling, translation
- Time series with RNN/LSTM

Projects built:

- Text generation with LSTM
- Sentiment analysis with RNN
- Stock price prediction with LSTM
- Name generation model
- Machine translation basics
- Seq2Seq chatbot
- Music generation with RNN

Practice: Build 12 sequence modeling projects

Natural Language Processing (NLP) - Part 1

Topics:

- NLP pipeline and challenges
- Text preprocessing: tokenization, lowercasing
- Stop words removal and stemming/lemmatization
- Bag of Words (BoW) and TF-IDF review
- Word embeddings: Word2Vec (CBOW, Skip-gram)
- GloVe embeddings
- FastText embeddings
- Using pretrained embeddings
- Text classification with embeddings
- Named Entity Recognition (NER)
- Part-of-Speech (POS) tagging
- Dependency parsing basics
- spaCy for NLP

Projects built:

- Sentiment classifier with embeddings
- Spam detection (email/SMS)
- Named Entity Recognition system
- Text summarization basics
- Question answering system (simple)
- Topic modeling with LDA
- Language detection

Practice: Complete 15 NLP tasks

NLP - Part 2 (Advanced)

Topics:

- Transformer architecture: self-attention
- Multi-head attention
- Positional encoding
- BERT: Bidirectional Encoder Representations
- GPT: Generative Pre-trained Transformer
- T5, RoBERTa, ALBERT
- Hugging Face Transformers library
- Fine-tuning BERT for classification
- Zero-shot and few-shot learning
- Prompt engineering basics
- Modern NLP pipeline
- Production NLP systems

Projects built:

- BERT fine-tuning for sentiment
- Question-answering with BERT
- Text classification with Transformers
- Named Entity Recognition with BERT
- Summarization with T5
- Text generation with GPT
- Multi-task NLP model

Practice: Fine-tune 10 transformer models

Month 9: Advanced Deep Learning & Generative AI (Week 36-39)

Generative Adversarial Networks (GANs)

Topics:

- Generative models overview
- GAN architecture: generator and discriminator
- GAN training: adversarial loss
- Mode collapse and training challenges
- DCGAN: Deep Convolutional GAN
- Conditional GAN (cGAN)
- Pix2Pix for image-to-image translation
- CycleGAN for unpaired translation
- StyleGAN for high-quality generation
- Progressive GAN
- Wasserstein GAN (WGAN)
- GAN evaluation metrics: Inception Score, FID
- Applications: image generation, data augmentation

Projects built:

- DCGAN for image generation (faces, digits)
- Conditional GAN for MNIST
- Pix2Pix implementation
- Image super-resolution with GAN
- Style transfer with GAN
- Data augmentation using GANs
- Deepfake detection (ethics discussion)

Practice: Implement 8 different GAN architectures

Variational Autoencoders & Advanced Generative Models

Topics:

- Autoencoders: encoder-decoder architecture
- Dimensionality reduction with autoencoders
- Denoising autoencoders
- Variational Autoencoders (VAE)

- VAE loss: reconstruction + KL divergence
- Latent space interpolation
- Conditional VAE
- Diffusion models introduction
- Stable Diffusion basics
- DALL-E and text-to-image models
- Generative models for text
- Applications in creative AI

Projects built:

- Autoencoder for image compression
- VAE for generating faces
- Anomaly detection with autoencoders
- Latent space exploration
- Image denoising with autoencoders
- Text-to-image with Stable Diffusion API
- Creative AI project

Practice: Build 10 generative AI projects

Reinforcement Learning - Part 1

Topics:

- Reinforcement Learning paradigm
- Agent, environment, state, action, reward
- Markov Decision Process (MDP)
- Policy and value functions
- Bellman equations
- Dynamic programming: value iteration, policy iteration
- Monte Carlo methods
- Temporal Difference (TD) learning
- Q-Learning algorithm
- SARSA algorithm
- Exploration vs exploitation: ϵ -greedy
- OpenAI Gym environment

Projects built:

- Q-Learning for GridWorld
- SARSA for Frozen Lake
- CartPole with Q-Learning
- Taxi problem solution
- Simple game AI with RL
- Custom RL environment

Practice: Solve 10 RL problems

Reinforcement Learning - Part 2 (Deep RL)

Topics:

- Deep Q-Networks (DQN)
- Experience replay
- Target networks
- Double DQN
- Dueling DQN
- Policy gradient methods
- REINFORCE algorithm
- Actor-Critic methods
- A3C (Asynchronous Advantage Actor-Critic)
- PPO (Proximal Policy Optimization)
- Applications: game playing, robotics
- Multi-agent RL basics

Projects built:

- DQN for Atari games
- CartPole with DQN
- Lunar Lander with PPO
- Custom game with RL agent
- Policy gradient implementation
- Multi-agent environment

Practice: Train 8 deep RL agents

PyTorch Deep Dive

Topics:

- PyTorch fundamentals
- Tensors and autograd
- Building models with nn.Module
- PyTorch data loading: Dataset, DataLoader
- Custom datasets creation
- Training loops in PyTorch
- GPU acceleration with PyTorch
- torchvision for computer vision
- TorchText for NLP
- PyTorch Lightning for cleaner code
- Model checkpointing
- TensorBoard with PyTorch
- PyTorch vs TensorFlow comparison

Projects built:

- Image classifier in PyTorch
- Custom CNN architecture

- Transfer learning in PyTorch
- RNN/LSTM in PyTorch
- GAN in PyTorch
- PyTorch Lightning project
- Multi-GPU training

Practice: Reimplement 15 projects in PyTorch

PHASE 3 COMPLETION - Month 9 Final Weeks (Week 27-39 (distributed))

Model Optimization & Compression

Topics:

- Model optimization importance
- Quantization: reducing precision
- Pruning: removing unnecessary weights
- Knowledge distillation: teacher-student
- Neural Architecture Search (NAS)
- MobileNets for mobile devices
- EfficientNet: compound scaling
- TensorFlow Lite for mobile
- ONNX for model interoperability
- Edge AI deployment
- Inference optimization
- Benchmarking model speed

Projects built:

- Model quantization experiment
- Pruned neural network
- Knowledge distillation implementation
- MobileNet deployment
- TensorFlow Lite model
- ONNX conversion pipeline
- Edge device deployment

Practice: Optimize 10 models for deployment

Audio & Speech Processing

Topics:

- Audio signal basics
- Audio preprocessing and features
- Mel-frequency cepstral coefficients (MFCCs)
- Spectrograms
- Speech recognition with Deep Learning
- WaveNet and audio generation
- Voice cloning basics
- Music genre classification
- Audio event detection
- Librosa library for audio
- Speech-to-text systems
- Text-to-speech (TTS) basics

Projects built:

- Speech recognition system
- Music genre classifier
- Audio event detection
- Voice command recognition
- Audio generation with WaveNet
- Emotion recognition from speech
- Speaker identification

Practice: Build 8 audio AI projects

Multi-Modal Learning & Graph Neural Networks

Topics:

- Multi-modal learning introduction
- Image + text models
- CLIP: Contrastive Language-Image Pre-training
- Vision-language models
- Multi-modal fusion strategies
- Graph data representation
- Graph Neural Networks (GNN) basics
- Graph Convolutional Networks (GCN)
- Node classification and link prediction
- Graph attention networks
- Applications: social networks, molecules
- Knowledge graphs

Projects built:

- Image captioning system
- Visual question answering
- Multi-modal sentiment analysis
- GNN for social network analysis
- Node classification with GCN
- Link prediction task
- Knowledge graph construction

Practice: Explore multi-modal and graph projects

AI Ethics, Fairness & Responsible AI

Topics:

- Ethics in AI/ML
- Bias in machine learning
- Fairness metrics and definitions
- Detecting and mitigating bias
- Interpretable vs explainable AI
- LIME and SHAP for explanations
- Privacy in ML: differential privacy
- Federated learning for privacy
- Adversarial attacks on ML models
- Adversarial training for robustness
- Regulatory compliance: GDPR, AI Act
- Responsible AI frameworks
- Ethical considerations in deployment

Projects built:

- Bias detection in models
- Fairness-aware classifier
- Model explanation dashboard
- Privacy-preserving ML experiment
- Adversarial examples generation
- Robust model training
- Ethical AI case studies

Practice: Audit models for bias and fairness

Phase 3 Capstone Project

Topics:

- Advanced AI system design
- Deep learning architecture selection
- Data collection and preprocessing
- Model training and optimization
- Evaluation and interpretation
- Deployment planning

Projects built:

- MAJOR CAPSTONE: Advanced AI Application
- Option 1: End-to-End Computer Vision System (e.g., Medical image diagnosis)
- Option 2: Advanced NLP Application (e.g., Chatbot with context, QA system)
- Option 3: Generative AI Project (e.g., Content generation platform)
- Option 4: Reinforcement Learning Agent (e.g., Game AI, optimization problem)
- Requirements: Deep learning, production-ready, documented, deployed, evaluated thoroughly

Assessment: Phase 3 Final Exam - Deep Learning comprehensive test

----- PHASE 4: MLOPS, PRODUCTION AI & RESEARCH (MONTHS 10-12, WEEKS 40-52) -----

Master MLOps, production deployment, scalable AI systems, research skills, and career preparation.

Months 10-11: MLOps & Production AI Systems (Week 40-48)

MLOps Fundamentals

Topics:

- What is MLOps? DevOps for ML
- ML system architecture
- ML pipeline orchestration
- Kubeflow for ML workflows
- Apache Airflow for ML pipelines
- Feature stores: Feast, Tecton
- Model registry: MLflow, DVC
- Experiment tracking at scale
- Metadata management
- Data lineage and provenance
- Continuous training (CT)
- Automated retraining pipelines
- Model governance

Projects built:

- Complete MLOps pipeline
- Automated ML workflow with Airflow
- Feature store implementation
- Model registry setup
- Continuous training system
- End-to-end ML automation

Practice: Build 5 MLOps pipelines

Model Serving & Deployment at Scale

Topics:

- Model serving architectures
- TensorFlow Serving
- TorchServe for PyTorch
- NVIDIA Triton Inference Server
- RESTful API vs gRPC for serving
- Batch vs real-time inference
- Model optimization for serving
- A/B testing infrastructure

- Multi-armed bandits for model selection
- Canary deployments
- Shadow deployments
- Load balancing for ML services
- Kubernetes for ML deployment

Projects built:

- TensorFlow Serving deployment
- TorchServe implementation
- High-performance inference API
- A/B testing framework
- Multi-model serving system
- Kubernetes ML deployment
- Load testing ML APIs

Practice: Deploy 10 models to production

Monitoring & Observability for ML

Topics:

- ML model monitoring importance
- Data drift detection
- Concept drift monitoring
- Model performance degradation
- Monitoring metrics: latency, throughput
- Prometheus for ML monitoring
- Grafana dashboards for ML
- Logging for ML systems
- Error tracking and alerting
- Model explainability in production
- Feedback loops
- Retraining triggers
- Incident response for ML systems

Projects built:

- Model monitoring dashboard
- Data drift detector
- Performance monitoring system
- Alerting framework for ML
- Automated retraining trigger
- Production ML observability stack
- Feedback collection system

Practice: Monitor all deployed models

Scalable ML Infrastructure

Topics:

- Distributed training: data parallelism
- Model parallelism for large models
- Horovod for distributed training
- Ray for distributed ML
- GPU clusters management
- Cloud ML platforms: AWS SageMaker
- Google Cloud AI Platform
- Azure Machine Learning
- Vertex AI
- Managed ML services
- Cost optimization for ML
- Spot instances for training
- Serverless ML inference

Projects built:

- Distributed training setup
- Multi-GPU training
- SageMaker end-to-end pipeline
- GCP Vertex AI deployment
- Azure ML workspace
- Cost-optimized ML infrastructure
- Serverless ML API

Practice: Deploy on all major cloud platforms

AutoML & Meta-Learning

Topics:

- Automated Machine Learning (AutoML)
- AutoML frameworks: Auto-sklearn, TPOT
- Google AutoML
- H2O.ai AutoML
- Neural Architecture Search (NAS)
- Hyperparameter optimization: Optuna, Hyperopt
- Meta-learning introduction
- Few-shot learning
- Transfer learning at scale
- Model zoos and pretrained models
- Automated feature engineering
- AutoML for time series

Projects built:

- AutoML pipeline with Auto-sklearn
- Hyperparameter optimization with Optuna

- NAS implementation
 - Few-shot learning system
 - Automated feature engineering
 - Custom AutoML framework
 - Meta-learning experiment
- Practice: Apply AutoML to 10 datasets

Month 12: Research Skills & Career Excellence (Week 49-52)

Reading & Implementing Research Papers

Topics:

- How to read research papers
- arXiv and academic resources
- Understanding paper structure
- Mathematical notation in papers
- Implementing papers from scratch
- Reproducing research results
- State-of-the-art (SOTA) models
- Benchmarking on standard datasets
- Contributing to research discussions
- Writing technical reports
- Academic writing basics
- Citing and references

Projects built:

- Implement 5 research papers from scratch
- ResNet paper implementation
- Attention is All You Need (Transformers)
- BERT paper reproduction
- GAN paper implementation
- Technical paper writing
- Literature review on a topic

Practice: Read and implement 20 papers

Advanced AI Topics & Frontier Research

Topics:

- Large Language Models (LLMs) architecture
- GPT-3, GPT-4, PaLM architecture
- Prompt engineering advanced
- Fine-tuning large models
- RLHF (Reinforcement Learning from Human Feedback)
- Constitutional AI
- Vision Transformers (ViT)
- Multi-modal transformers
- Diffusion models deep dive
- Neural rendering and NeRF
- Foundation models
- AI safety and alignment
- Emerging AI research directions

Projects built:

- Fine-tune GPT for specific task
- Vision Transformer implementation
- Diffusion model from scratch
- Multi-modal transformer
- RLHF experiment
- Custom foundation model (small scale)
- AI safety project

Practice: Explore cutting-edge AI research

Domain-Specific AI Applications

Topics:

- Healthcare AI: medical imaging, diagnosis
- Drug discovery with AI
- Finance: algorithmic trading, fraud detection
- Retail: demand forecasting, personalization
- Manufacturing: predictive maintenance, quality control
- Autonomous vehicles: perception, planning
- Agriculture: crop monitoring, yield prediction
- Energy: smart grids, consumption forecasting
- Natural disasters prediction
- Climate change modeling
- AI for social good
- Industry-specific challenges and solutions

Projects built:

- Medical image classification
- Fraud detection system
- Demand forecasting model
- Predictive maintenance system
- Autonomous navigation basics
- Crop disease detection
- Domain-specific AI application

Practice: Build 5 domain-specific AI projects

Building AI Products & Startups

Topics:

- AI product development lifecycle
- Identifying AI opportunities
- Problem-solution fit for AI
- Building MVP for AI products
- AI product metrics and KPIs
- User experience in AI products
- Monetization strategies
- AI as a service (AIaaS)
- API-first AI products
- Scaling AI products
- Team building for AI startups
- Fundraising for AI ventures
- Legal and regulatory considerations

Projects built:

- AI product prototype
- AI SaaS application
- AI API marketplace listing
- Product roadmap for AI startup
- MVP development
- Go-to-market strategy
- Pitch deck for AI product

Practice: Develop complete AI product concept

Interview Preparation & Career Strategy

Topics:

- ML interview preparation strategy
- Coding interviews for ML roles
- ML theory and concepts questions
- System design for ML systems
- Take-home assignments approach
- Portfolio projects presentation
- Resume for ML roles
- LinkedIn for ML professionals
- GitHub for ML engineers
- Networking in AI community
- Conferences and meetups
- Building personal brand
- Salary negotiation for ML roles
- Career paths in AI/ML

Projects built:

- ML interview preparation guide
- Portfolio website with projects
- Technical blog writing
- GitHub profile optimization
- Mock interview practice
- System design case studies

Practice: Daily LeetCode, ML questions, system design

PHASE 4 COMPLETION - Final Projects (Week 40-52 (distributed))

Kaggle & Competitions

Topics:

- Kaggle platform mastery
- Competition strategy
- Exploratory data analysis for competitions
- Ensemble methods for Kaggle
- Cross-validation strategies
- Leaderboard probing techniques
- Feature engineering for competitions
- Model stacking and blending
- Time management in competitions
- Learning from Kaggle kernels
- Kaggle datasets exploration
- Achieving Kaggle ranks

Projects built:

- Participate in 5 Kaggle competitions
- Achieve Expert rank goal
- Win a Kaggle medal
- Kernel/notebook publication
- Dataset contribution
- Competition writeup

Practice: Active Kaggle participation

Open Source Contribution in AI/ML

Topics:

- Finding AI/ML open source projects
- Contributing to TensorFlow/PyTorch
- Scikit-learn contributions
- Hugging Face ecosystem
- Documentation improvements
- Bug fixes and features

- Creating ML libraries
- Packaging and PyPI publishing
- Open source best practices
- Community engagement
- Code reviews in open source
- Building reputation

Projects built:

- Contribute to 5 ML open source projects
- Create own ML library
- Publish package to PyPI
- Documentation contributions
- Tutorial creation
- Community support

Practice: Regular open source contributions

Technical Writing & Teaching

Topics:

- Technical blog writing
- Explaining complex ML concepts
- Tutorial creation
- Video content for ML
- Medium and Dev.to platforms
- Creating ML courses
- Conference speaking
- Workshop facilitation
- Mentoring junior ML engineers
- Building audience
- Thought leadership in AI
- Content monetization

Projects built:

- Write 10 technical blog posts
- Create video tutorial series
- Develop mini-course
- Conference talk proposal
- Workshop material creation
- Mentorship program participation

Practice: Weekly content creation

Continuous Learning & Specialization

Topics:

- Staying updated with AI research
- Following AI researchers on Twitter
- Reading papers regularly
- Choosing specialization path
- Deep learning specialization vs breadth
- PhD vs industry career
- Research scientist vs ML engineer
- Continuous skill development
- Learning roadmap creation
- Community involvement
- Networking with experts
- Long-term career planning

Projects built:

- Personal learning roadmap
- Specialization area selection
- Research proposal (if academic path)
- Industry project plan
- 5-year career plan
- Skill gap analysis

Practice: Develop lifelong learning habit

Final Month - Capstone & Career Launch (Week 49-52)

Final Capstone Project - Part 1

Topics:

- Complex AI system design
- Problem identification
- Literature review
- Data collection strategy
- Architecture design
- Technology stack selection
- Development planning
- Research component
- Innovation and novelty

Projects built:

- FINAL CAPSTONE: Production-Grade AI System
- Option 1: End-to-End ML Platform (AutoML, model serving, monitoring)
- Option 2: Advanced NLP System (chatbot, QA, summarization with LLMs)
- Option 3: Computer Vision Application (detection, segmentation, deployed)
- Option 4: Generative AI Platform (text/image generation, fine-tuned models)
- Option 5: Reinforcement Learning System (game AI, robotics, optimization)
- Option 6: Multi-modal AI Application (vision + language)

- Requirements: Research-backed, production-deployed, MLOps, monitored, documented, novel approach

Final Capstone Project - Part 2

Topics:

- Implementation completion
- Comprehensive testing
- MLOps pipeline setup
- Cloud deployment
- Monitoring and logging
- Documentation writing
- Research paper writing
- Performance benchmarking
- Demo preparation
- Presentation skills

Deliverables:

- Complete source code (GitHub)
- Deployed production system
- MLOps pipeline (training, serving, monitoring)
- Comprehensive documentation
- Research paper or technical report
- Demo video
- Presentation deck
- Model cards and datasheets
- Benchmarking results
- User guide

Career Launch & AI/ML Professional

Topics:

- AI/ML portfolio showcase
- Resume optimization
- LinkedIn for ML roles
- GitHub profile excellence
- Personal website/blog
- Networking strategies
- Job search in AI/ML
- Applying to top companies
- Interview process navigation
- Offer negotiation
- Career decision making
- Continuous growth mindset

Deliverables:

- Professional portfolio with 50+ projects
- Optimized ML engineer resume
- LinkedIn with certifications and projects
- Active GitHub with contributions
- Technical blog with followers
- Research publication (optional)
- Kaggle profile (medals)
- Network of AI professionals
- Job offers or interviews lined up
- Clear career trajectory

Assessment: FINAL COMPREHENSIVE EXAM - AI/ML mastery evaluation covering all 12 months

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-3): 25+ foundational projects - math implementations, data analysis, visualizations
- Phase 2 (Months 4-6): 30+ ML projects - classical algorithms, Kaggle, complete pipelines
- Phase 3 (Months 7-9): 35+ DL projects - computer vision, NLP, GANs, RL, audio, graphs
- Phase 4 (Months 10-12): 20+ production projects - MLOps, deployed systems, research implementations
- Total: 100+ projects from basics to cutting-edge AI

Total projects: 100+ AI/ML projects covering all domains and difficulty levels

SKILLS THE STUDENT WILL HAVE AT THE END

- Mathematics: Linear Algebra, Calculus, Probability, Statistics, Optimization
- Programming: Python (expert level), NumPy, Pandas, Matplotlib, Scikit-learn
- Classical ML: Regression, Classification, Clustering, Dimensionality Reduction, Ensemble Methods
- Deep Learning: Neural Networks, CNN, RNN, LSTM, Transformers, Attention Mechanisms
- Computer Vision: Image Classification, Object Detection, Segmentation, GANs, Style Transfer
- NLP: Text Processing, Embeddings, BERT, GPT, Transformers, Hugging Face
- Specialized: Reinforcement Learning, GANs, VAEs, Graph Neural Networks, Audio Processing
- Frameworks: TensorFlow, Keras, PyTorch, PyTorch Lightning, Hugging Face Transformers
- MLOps: MLflow, DVC, Kubeflow, Docker, Kubernetes, Cloud Platforms (AWS, GCP, Azure)
- Production: Model Serving, Monitoring, A/B Testing, CI/CD for ML, Scalable Inference
- Tools: Jupyter, Git, Weights & Biases, TensorBoard, Colab, Kaggle
- Soft Skills: Research Paper Reading, Technical Writing, Problem-Solving, System Design

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-7 hours
Hands on coding: 10-12 hours
Projects: 4-6 hours
Reading research: 2-3 hours
Practice problems: 2-3 hours
Total per week: 20-25 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding and doubt clearing with AI experts
Mentorship: 1-on-1 mentorship from ML engineers and researchers
Community: Active Discord with researchers, engineers, and peers
Code review: Expert code reviews for all major projects
Research guidance: Help with paper reading and implementation
Career support: Resume review, mock interviews, referrals to AI companies
Kaggle guidance: Competition strategies and team formation
Lifetime access: All content, updates, new research implementations
Gpu credits: Cloud GPU credits for training models
Dataset access: Curated datasets for all projects

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 certificates)
Final certificate: AI/ML Engineer Professional Certification
Specialization certificates: Computer Vision, NLP, or RL specialization certificate
Research certificate: Research Paper Implementation Certificate
MLOps certificate: MLOps Professional Certificate
Linkedin badges: Digital badges for all certifications
Industry recognized: Recognized by FAANG and AI startups
Portfolio projects: 50+ documented projects
Kaggle ranking: Support to achieve Expert/Master rank
Publication support: Help with research paper submission (optional)

WHERE THIS CAN LEAD

- Machine Learning Engineer
- Data Scientist
- AI Engineer
- Deep Learning Engineer
- Computer Vision Engineer
- NLP Engineer
- Research Scientist / Research Engineer
- MLOps Engineer
- AI Product Manager (with business skills)
- AI Consultant
- Kaggle Competitor / Grandmaster (with continued effort)
- AI Startup Founder / CTO
- PhD in Machine Learning (with research focus)
- AI Researcher at Labs (Google AI, Meta AI, OpenAI, etc.)
- Applied Scientist at Tech Companies

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 4-7 LPA (Junior Data Scientist/Analyst)

Growing roles: Rs 8-15 LPA (ML Engineer/Data Scientist); Rs 15-25 LPA (Senior ML Engineer)

Experienced roles: Rs 20-40 LPA (Senior ML/AI Engineer, Research Engineer); Rs 30-60 LPA (Lead ML Engineer)

Freelance consulting: Rs 3,000-10,000/hour based on expertise

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is the difference between this AI/ML Masterclass and the Artificial Intelligence Masterclass?

A: This Complete AI & ML Masterclass focuses on practical machine learning engineering: building ML pipelines, training models with TensorFlow and PyTorch, deploying to production, and working on Kaggle problems. The AI Masterclass is more theoretical, covering classical AI, symbolic reasoning, and AGI concepts. Choose this course if you want hands-on ML engineering skills for industry work.

Q: What machine learning libraries and frameworks will I master?

A: The course covers the complete modern ML engineering stack.

Q: Can I get a job as an ML Engineer after this course?

A: This course is built to make you job-ready as an ML engineer. You build 50+ practical projects, work on Kaggle-style problems, and prepare thoroughly for ML engineering interviews with mock interviews and portfolio review. We prepare you well, but we do not make placement guarantees.

Q: What types of projects will I build in this AI/ML course?

A: All projects use real datasets and are deployment-ready with APIs.

Q: What are the prerequisites for this AI/ML masterclass?

A: You need basic Python programming knowledge, understanding of high school mathematics (algebra, basic statistics), and comfortable working with a computer. The course covers all required mathematics (linear algebra, calculus, probability) and advanced Python from scratch.

Q: How does this course prepare me for Kaggle competitions?

A: Dedicated modules cover Kaggle competition strategies, feature engineering, model ensembling, cross-validation techniques, and achieving Expert/Master rankings. Students compete in 5+ real Kaggle competitions during the course with mentorship on leaderboard optimization.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ai-ml-masterclass-complete-college>

=====

COURSE 7 OF 118: AI AGENTS WITH MICROSOFT COPILOT STUDIO FOR COLLEGE STUDENTS

=====

Course link: <https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-college-students>

Subject area: AI Agents & Automation

Designed for: College students

Who can join: Intermediate (college students and early-career, ages 18+; no prior agent experience needed)

Course length: 24 classes (12 weeks · 2 classes/week)

Time needed: 4-5 hours/week recommended

Certificate: Modern Age Coders 'AI Agents with Copilot Studio' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

Note: this course also needs the student to have their own Claude and ChatGPT subscriptions, because the whole course is hands on with those tools. Say this before enrolling anyone, so it is never a surprise later.

WHAT THIS COURSE IS

Build business-grade AI agents in Microsoft Copilot Studio: 24 live 1-on-1 online classes on knowledge grounding, connectors, agent flows, MCP, autonomous triggers, multi-agent orchestration and a recruiter-ready published capstone.

In one line: Recruiters no longer ask whether you have used AI. They ask what you have built with it. Build, ground, automate and publish real business agents, in private 1-on-1 classes.

ABOUT THE PROGRAMME

Every placement season now has a new line on the checklist: AI agents. Microsoft Copilot Studio is the graphical, low-code studio enterprises use to build and manage AI-powered agents and workflows, and by Microsoft's own count 90% of the Fortune 500 use it. This course takes a college student from zero to a business-grade, published agent in 24 live 1-on-1 classes. You start with agent fundamentals done properly: instructions that steer behaviour, test sets, and the model lineup behind every agent (GPT-4.1 as the default, GPT-5.5 Chat, and Anthropic Claude models among them). Then you go deep where business agents actually live: knowledge grounding across SharePoint, Dataverse and connector-indexed enterprise data with citations users can check; conversation design with topics, entities, condition groups and adaptive cards; and generative orchestration, where the model plans across your topics, knowledge and tools. Phase 4 is the employability core: tools built from Copilot Studio's library of more than 1,400 connectors, deterministic agent flows for work that must run the same every time, custom connectors and REST APIs, Model Context Protocol (MCP) tools, autonomous event triggers with the security discipline they demand, multi-agent design with child agents and the agent-to-agent (A2A) protocol, and analytics with agent evaluations to prove your agent works. You finish by publishing to real channels, passing a professional pre-flight checklist, and defending a capstone agent built for a real club, department or small business, with a documentation packet a recruiter can actually read. The syllabus is a Living Syllabus, updated as Copilot Studio ships, and it maps directly to Microsoft's credential ladder: PL-900, the Applied Skills labs, and the AI Agent Builder Associate certification (exam AB-620).

WHAT MAKES THIS COURSE DIFFERENT

- 1-on-1 only, by design: your mentor reviews your actual instructions, knowledge sources, flows and test logs line by line every class, and points every project at your real placement goals, which a group batch cannot do honestly.
- The full business toolchain, not a demo tour: connectors, custom connectors, REST APIs, Model Context Protocol, agent flows, event triggers, multi-agent design and agent evaluations, each one practised on a project you keep.
- Certification-mapped: the syllabus vocabulary tracks Microsoft's PL-900 exam (which added a dedicated Copilot Studio agents domain in July 2026) and the AI Agent Builder Associate certification (exam AB-620), so course time doubles as credential prep.
- Autonomy taught with adult supervision of the risks: event triggers run with the maker's credentials, so you learn the security reasoning enterprises are hiring for, not just the wow demo.
- Evidence over vibes: from Class 4 you keep fixed test sets, and by Phase 4 you use analytics outcomes and agent evaluations to prove improvements with numbers, the habit that separates builders from users.
- Living Syllabus: Copilot Studio ships continuously (MCP tools, the A2A protocol, agent evaluations and new models all landed within the last year), and the curriculum is re-verified against Microsoft's documentation as features land.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac or Linux) with the Chrome or Edge browser and a reliable internet connection; Copilot Studio runs fully in the browser

Coding experience: None required. Copilot Studio is a graphical, low-code studio, and the course teaches every concept from first principles. Students who do code will get further faster in the REST, custom connector and MCP classes, but the syllabus assumes nothing

Accounts: Copilot Studio uses a Microsoft work or school (Entra ID) account rather than a personal email. Most college email addresses qualify for the 30-day self-serve trial (note: trial environments cannot publish agents). Where a college account is not available, we arrange a practice environment for the duration of the course

Mindset: The willingness to test before you trust, and to treat instructions, sources and logs as engineering artifacts. The course rewards students who ask for evidence

WHO THIS COURSE IS FOR

Placement seekers: Final and pre-final year students who want a demonstrable AI agent skill on their resume, with a published capstone and documentation packet recruiters can actually inspect

Business and commerce students: BBA, BCom and management students who will never be full-time programmers but will absolutely be asked to automate processes and build departmental agents

Engineering students: CS and IT students who know code but not the low-code agent stack enterprises deploy, and want the governance and orchestration vocabulary interviews now probe

Club and fest organisers: Students already running societies and fests who have real users, real FAQs and real processes, the perfect training ground for business-grade agents

Early career professionals: Recent graduates in operations, support or analyst roles who want to become the person on the team who ships the agent instead of requesting one

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: AGENT FOUNDATIONS

Understand agents in business terms, tour Microsoft Copilot Studio properly, write instructions that actually steer behaviour, test like a professional, and meet the models and harnesses underneath every agent.

Class 1: Agents as a Business Skill, Not a Toy

Topics:

- Chatbot vs assistant vs agent: instructions, knowledge, tools and autonomous action, defined precisely
- Why this is on placement checklists: Microsoft reports 90% of the Fortune 500 use Copilot Studio, and Gartner projects a third of enterprise software will include agentic AI by 2028
- Microsoft Copilot Studio in one sentence: a graphical, low-code studio for building and managing AI-powered agents and workflows
- The studio tour that matters: agents, knowledge, topics, tools, triggers, channels, analytics, and the Build, Preview, Evaluate and Monitor surfaces
- Where Copilot Studio sits: low-code SaaS on Power Platform, beside Microsoft Foundry for pro-code teams, and how the two interoperate
- Access done honestly: Copilot Studio needs a Microsoft work or school account; most college emails qualify for the 30-day trial, and your mentor sets your practice environment up with you
- Choosing your capstone candidate: a real club, department or small business whose problem you will ship an agent for in Class 24

Projects built:

- Agent teardown, business edition: analyse one real customer-facing agent (bank, airline,

university helpdesk), reverse-engineer its probable instructions, knowledge and tools, and write a one-page teardown

Practice: Draft five agent ideas from your college and internship life and rank them by business value and by how much real knowledge each needs.

Assessment: Define agent, knowledge grounding and tool in your own words, with one business example each.

Class 2: Your First Agent, Built Properly

Topics:

- Describe-to-create: starting an agent from a plain-English description and reading what the platform scaffolds for you
- The instruction stack for business agents: identity, audience, scope, tone, format rules and refusals, in that order
- The test panel as your workbench: probing the agent the moment it exists, and logging behaviour instead of trusting memory
- One-change iteration: modify a single instruction line, retest, compare, keep or revert
- Names and descriptions users trust: why a precise agent description is both UX and findability
- What the model actually does with instructions: system prompts, context and why wording changes behaviour
- Version discipline from day one: keeping instruction versions outside the tool

Projects built:

- Build 'Placement FAQ v1': an agent with hand-written instructions that answers questions about one real company's hiring process in a tone you designed

Practice: Write three instruction variants for the same agent and record exactly how each changes the answers to a fixed set of five questions.

Assessment: Show a before-and-after instruction change with the precise behaviour difference it caused.

Class 3: Instructions at Business Depth

Topics:

- Scope fences: declaring what the agent must not do, and proving the fence holds under pressure
- Refusal design for business contexts: legal, HR and personal-data questions the agent must decline gracefully
- Brand voice as an instruction problem: the same agent for students, recruiters and professors
- Format contracts: forcing answers into steps, tables or short summaries that busy readers actually use
- Instruction smells: vague verbs, contradictions, and rules that fight each other in production
- The deep reasoning preview, honestly framed: Microsoft's reasoning models are in preview and triggered by the keyword 'reason' in instructions, and current syllabi treat them as preview, not promises
- A/B instruction testing: scoring two instruction sets against the same 10 questions

Projects built:

- The Refusal Gauntlet, corporate edition: 10 questions your Placement FAQ agent must refuse or redirect (offer-letter legal advice, personal data, salary speculation), tuned until all 10 are handled gracefully

Practice: Have two friends try to drag your agent out of scope for ten minutes each; log every breach and patch the instructions.

Assessment: Defend each line of your instruction stack: what specific failure appears if that line is deleted?

Class 4: Testing Like Someone Whose Name Is on It

Topics:

- Fixed test sets: 15 questions with expected behaviour written before you run them
- Happy path, edge case, adversarial: three question classes every business agent must survive
- Grading answers on four axes: correctness, tone, format and scope
- Manual regression: rerunning the full set after every meaningful change, and why professionals never skip it
- Recognising hallucination before grounding exists to fix it: confident nonsense and how to catch it
- The test log as an artifact: question, expected, actual, verdict, and why recruiters respond to this document
- Preview of Class 20: Copilot Studio's agent evaluations automate exactly this habit, so building it by hand first makes the tool obvious later

Projects built:

- Build the permanent 15-question test set for Placement FAQ, run it twice in one week, and file the two most instructive failures with root causes

Practice: Write five adversarial questions designed to make your agent overstep, and score how it handles each.

Assessment: Given a conversation transcript, mark every agent turn pass or fail against the written instructions, with reasons.

Class 5: Models, Harnesses and What You Are Actually Running On

Topics:

- The model picker: GPT-4.1 as the current default, GPT-5 Chat and GPT-5.5 Chat generally available, and Anthropic Claude models (Sonnet 4.6, Opus 4.6 and 4.7) in the lineup
- Why model choice is a design decision: retesting your fixed set across two models and

defending the differences

- Harnesses in plain words: the runtime between your design and the model, with a standard harness for topics and flows and a GitHub Copilot harness for reasoning-heavy work
- Environments: why your practice environment is separate from a company's production environment, and what that separation protects
- Copilot Credits, first pass: agent usage is metered (classic answers, generative answers and actions cost different amounts), and designers are expected to know it
- Platform guardrails you inherit: content filtering and responsible-AI defaults that run before your instructions
- Phase 1 systems map: from a user's message through harness, instructions and model to the answer, labelled by what you control

Projects built:

- Model comparison memo: run your full Placement FAQ test set on two available models and write a one-page recommendation with evidence

Practice: Draw the full journey of one question through your agent and annotate the three points where you can change the outcome.

Assessment: Phase 1 gate: rebuild a small agent from a written spec in 20 minutes, with instructions, refusals and a five-question test.

PHASE 2: KNOWLEDGE & GROUNDING AT BUSINESS DEPTH

Ground agents in the knowledge sources enterprises actually use: SharePoint, documents, Dataverse and connector-indexed enterprise data, with citations, conflict handling and accuracy audits you publish.

Class 6: Grounding vs Guessing, the Business Version

Topics:

- Why ungrounded agents are unemployable: training data vs your organisation's data
- The knowledge source lineup: public websites, uploaded documents, SharePoint, Dataverse, and enterprise data indexed through connectors
- Generative answers: how the agent searches sources and composes answers, and where citations come from
- The 'Allow ungrounded responses' setting and why business agents keep it off
- Real limits that force curation: website and SharePoint URL caps, file ceilings, and a hard maximum of knowledge sources per agent
- Source selection as a professional judgment: authoritative, current, small enough to verify
- Grounding plan for your capstone: the source list you will defend on demo day

Projects built:

- Ground Placement FAQ in the target company's public website and prove five answers now carry citations a recruiter could check

Practice: Ask your grounded agent five questions its sources cannot answer and document its behaviour at the edge of knowledge.

Assessment: Explain grounding, citations and hallucination in interview language: two minutes, no notes.

Class 7: SharePoint and Documents Done Right

Topics:

- SharePoint as a knowledge source: pointing an agent at the document library a team actually maintains
- Writing documents for retrieval: headings, one fact per paragraph, consistent names, and why messy docs produce messy agents
- Document formats that ground well, and the file-size and count ceilings that shape what you upload
- Precision testing: questions whose answers live in one exact paragraph, and verifying the agent finds that paragraph
- Update behaviour: what happens when the source changes, and designing so the agent never lags the truth
- Confidentiality first pass: what must never enter an agent's knowledge, and who decides
- The curation principle: five great pages beat fifty mediocre ones, measurably

Projects built:

- Build 'Society Desk': an agent for a real college society grounded in a clean 3-document knowledge pack you author, with a 10-question proof test

Practice: Deliberately degrade one knowledge document, measure the answer-quality drop on your test set, then fix and remeasure.

Assessment: Given two versions of the same document, predict which grounds better and prove it with tests.

Class 8: Dataverse, Enterprise Data and Source Conflicts

Topics:

- Dataverse as structured knowledge: when tabular business data beats documents
- Enterprise data through connectors: sources indexed by Microsoft Search, and why enterprises fund this path
- Multi-source agents: how the agent chooses between website, documents and structured data
- Conflict behaviour: engineering for the day two sources disagree, because they will
- Scoping by instruction: routing question types to the right source deliberately
- Freshness strategy: which source type updates fastest and which facts must follow it
- Auditing knowledge like an owner: what is in, what is out, what is stale, on one page

Projects built:

- Build a two-source Internship Tracker knowledge base (structured table plus policy document) and document exactly how the agent behaves when they conflict
- Practice: Write five questions that force a source choice and log which source wins each, then adjust scoping instructions.
- Assessment: Defend your capstone source list: authority, freshness and maintenance plan for each source.

Class 9: Accuracy Audits, Bias and the Trust Contract

Topics:

- The accuracy audit: scoring 20 grounded answers against source text and publishing the number
- Bias in, bias out: how one-sided source lists produce one-sided agents, and the audit that catches it
- Uncertainty language: agents that say 'my sources do not cover that' instead of improvising
- Data protection instincts: personal data, minimisation, and what a business agent must forget
- The trust contract with users: transparency about being an agent, about sources, and about limits
- What enterprises layer on top: evaluation suites and monitoring, previewed honestly before you meet them in Class 20
- Phase 2 wrap: the grounding checklist you now apply to every agent

Projects built:

- Publish an accuracy audit of Society Desk: 20 scored answers with cited source paragraphs, an accuracy figure, and the three fixes it demanded

Practice: Add an uncertainty rule to every agent you own and collect three real out-of-scope responses that handled it well.

Assessment: Phase 2 gate: design a complete knowledge plan (sources, scoping, refusals, accuracy test) for a given business brief, in writing.

PHASE 3: CONVERSATION & ORCHESTRATION

Design conversations that hold up under real users: topics and trigger phrases, entities and variables, condition groups, adaptive cards, and a deliberate choice between generative and classic orchestration.

Class 10: Topics and Trigger Phrases

Topics:

- Topics as designed conversation paths: one topic per job the agent does
- Trigger phrases: capturing the many phrasings of one request (topics support up to 200)
- System topics you inherit: greeting, escalate, goodbye, and when to customise them
- The authoring canvas: message nodes, question nodes and flow, built live
- Topics vs generative answers: control for the critical paths, flexibility for the long tail
- The YAML beneath the canvas: every topic is also code, your first bridge from low-code to pro-code
- The topic map for a business agent: the five jobs users will actually bring

Projects built:

- Build 'Fest Helpdesk' with three designed topics for a real event: schedule lookup, registration help and venue directions

Practice: Collect ten real phrasings of one request from classmates and tune trigger phrases until recognition is reliable.

Assessment: Given three user messages, predict the triggered topic and verify live, explaining each miss.

Class 11: Entities, Variables and Slot Filling

Topics:

- Question nodes that collect exactly what a process needs, one clean question at a time
- Entities: extracting dates, numbers and choices from messy human sentences
- Variables: carrying user answers through the conversation and into confirmations
- Slot filling: skipping questions the user already answered in one breath
- Choice questions vs open questions: constraining input where structure matters
- Validation: what happens when the user answers a date question with 'whenever works'
- Session memory: what the agent remembers within a conversation and what it must not

Projects built:

- Upgrade Internship Tracker: an add-application topic that collects company, role, deadline and status through entities, confirming with a variable-built summary

Practice: Test one topic with cooperative, messy and hostile input styles; log extraction failures and tighten the questions.

Assessment: Trace a conversation on paper naming the entity and variable at each step, then verify live.

Class 12: Condition Groups and Branch Design

Topics:

- Condition nodes: branching on variables to route the conversation
- Condition groups: combining tests cleanly (role is internship AND deadline within a week) without spaghetti
- Paper first: flow diagrams before canvas, the habit that prevents dead ends
- The escalation branch: recognising 'this needs a human' and handing off with context
- Default branches: designing for everything you did not predict
- Branch coverage testing: one scripted conversation per path, no path untested
- Refactoring: splitting one bloated topic into three maintainable ones

Projects built:

- Rebuild Fest Helpdesk registration as a condition-driven flow: student vs guest vs volunteer paths, each with its own confirmation and an escalation branch

Practice: Hand your flow diagram to a friend and let them hunt for a reachable dead end; fix what they find.

Assessment: Find and fix the planted logic bug in a provided flow diagram within one session.

Class 13: Adaptive Cards and Business-Grade Responses

Topics:

- Adaptive cards: structured, tappable answers for information that deserves better than a paragraph
- Card anatomy: titles, fact sets, images and actions, and when each earns its place
- Mobile first: your agent will mostly be read on phones, so cards are designed for small screens
- Card actions that continue the conversation: buttons feeding back into topics
- A small card style guide: consistency that makes an agent look engineered, not assembled
- Accessibility: alt text and reading order as professional defaults, not afterthoughts
- Restraint: the cases where plain text beats a card, chosen deliberately

Projects built:

- Give Internship Tracker a card-based pipeline view: each application a card with company, role, deadline and a status-update action

Practice: Convert your three most-used responses to cards and A/B them with two real users against the text versions.

Assessment: Improve a deliberately bad card (dense text, no hierarchy, dead buttons) and justify every change.

Class 14: Generative Orchestration vs Classic, Chosen Deliberately

Topics:

- Two execution brains: classic orchestration where trigger phrases pick topics, generative orchestration where the model plans across topics, knowledge and tools
- What generative orchestration unlocks: multi-intent requests, dynamic tool choice, and the event triggers you will use in Phase 4
- The trade: less predictability, reined back in with instructions and scope fences
- Reading the plan: inspecting how the agent decided which topic, source or tool to use, and debugging from that trace
- Mixing modes: designed topics for critical business paths, generative flexibility for the long tail
- Re-running the guardrail suite: refusals and fences retested under the new orchestrator, because behaviour moved
- Phase 3 wrap: the conversation-design checklist from trigger phrases to orchestration choice

Projects built:

- Migrate Fest Helpdesk to generative orchestration, rerun the full test set, and file a keep-or-fix verdict on every behaviour change

Practice: Write three multi-intent messages ('register me, what time is the keynote, can my cousin come') and tune until all three resolve in one turn.

Assessment: Phase 3 gate: design a five-topic business agent on paper (topics, entities, conditions, cards, orchestration mode) and defend it.

PHASE 4: TOOLS, AUTOMATION & MULTI-AGENT

The employability core: connectors and Prompt tools, deterministic agent flows, REST and custom connectors, Model Context Protocol, autonomous event triggers with real security discipline, multi-agent design with A2A, and analytics with agent evaluations.

Class 15: The Tool Menu and Connector Library

Topics:

- The tool types in Copilot Studio: Prompt, Agent flow, Custom connector, Model Context Protocol and REST API, plus prebuilt connectors
- The connector library: more than 1,400 prebuilt connections to real services, standard and premium
- Your first Prompt tool: one transformation done perfectly (summarise, extract, rewrite), reusable across agents
- Tool descriptions as an interface: the orchestrator chooses tools by reading descriptions, so you write them like contracts
- Discipline over abundance: the platform allows up to 128 tools per agent, Microsoft recommends staying near 25 to 30, and professionals use far fewer
- Authorisation questions before wiring anything: what may this tool touch, and who approved that
- Tool plan for your capstone: usually two tools, chosen from evidence, not excitement

Projects built:

- Add a 'JD decoder' Prompt tool to Placement FAQ: paste any job description, get requirements, keywords and preparation gaps in a fixed format

Practice: Write two versions of one tool description and demonstrate how the wording changes when the orchestrator invokes it.

Assessment: Explain, unaided, how an agent under generative orchestration decides between answering, searching knowledge and calling a tool.

Class 16: Agent Flows: Automation That Runs the Same Every Time

Topics:

- Agent flows: deterministic, step-based automations your agent calls as tools
- Flow anatomy: trigger, actions, conditions and outputs returned to the conversation
- When flows beat models: approvals, notifications, record updates, anything that must be identical every run
- Building a notification flow: collect structured details in conversation, send a formatted email or Teams message
- Error paths: what the user sees when a step fails, and why silent failure is disqualifying
- Reading a flow run: inputs, outputs and the exact step that broke
- Flows vs topics vs Prompt tools: a decision rubric you can defend in interviews

Projects built:

- Build 'Expense Slip': Society Desk collects reimbursement details through entities, an agent flow emails a formatted claim to the treasurer, and the agent confirms with the flow's real result

Practice: Break your flow with bad input on purpose, then add the error path that turns the failure into a graceful message.

Assessment: Given three automation briefs, classify each as flow, topic or Prompt tool with a three-sentence justification.

Class 17: REST APIs, Custom Connectors and Governance

Topics:

- REST in one hour: request in, JSON out, and how an agent turns a response into an answer
- Adding a REST API as a tool: authentication, parameters and response handling
- Custom connectors: wrapping any API so every maker in an organisation can reuse it safely
- Authentication in plain words: what 'connect your account' grants, and why least privilege is the default
- DLP governance: connector access is policy-controlled in real tenants, so 'can' and 'may' are different questions
- Reading API documentation like a builder: finding the one endpoint your agent needs
- The integration mindset recruiters probe: the least-powerful tool that does the job

Projects built:

- Wire a public REST API into a practice agent as a tool (weather, holidays or exchange rates) and demonstrate a grounded conversation using live data

Practice: Design on paper the API tool your capstone needs: endpoint, parameters, auth, tool description and three test utterances.

Assessment: Sketch one tool call end to end: utterance, orchestration decision, call, response, answer, and where it can fail.

Class 18: Model Context Protocol and Computer Use

Topics:

- Model Context Protocol (MCP): the open standard for connecting tool servers to agents, added from the same Add-a-tool menu
- MCP tools and resources: what a server exposes, and toggling individual tools with the Allow all switch
- Why MCP transfers: the same protocol appears in Claude, IDEs and other agent platforms, so this class compounds
- Governance continuity: MCP runs over Power Platform connectors, so DLP policies govern it like any other connector
- Computer use, surveyed honestly: agents that operate real user interfaces, generally available since May 2026, with per-step metering and premium model options
- The governance stack around computer use: allow-lists, HTTPS enforcement, stored credentials and human supervision
- Choosing integration styles: prebuilt connector vs REST vs custom connector vs MCP, as a decision table

Projects built:

- Connect an MCP server to a practice agent, inventory the tools it exposes, disable all but two deliberately, and demonstrate both in conversation

Practice: Write a one-page comparison: the same integration built as REST tool vs MCP tool, with the governance implications of each.

Assessment: Explain to your mentor what MCP standardises, and why DLP still applies to it.

Class 19: Autonomous Agents: Event Triggers and the Security They Demand

Topics:

- Event triggers: agents that act with no user in the chat, on events like an item created in SharePoint, a file added to OneDrive, a completed Planner task, or a Recurrence schedule
- The trigger payload: event data arriving as input, and instructions deciding what happens next
- Generative orchestration as a prerequisite: why autonomy needs the planning brain from Class 14
- The security fact enterprises underline: triggers run with the maker's credentials, so every autonomous action is effectively signed by you
- Designing the first autonomous job: small, observable, reversible, logged
- Human-in-the-loop: which actions pause for approval, and how the pause is built
- Idempotence in plain words: running twice must not double the damage

Projects built:

- Build 'Status Reporter': an autonomous agent that watches a shared project folder and posts a tidy weekly summary on a Recurrence trigger, with an approval pause on anything destructive

Practice: Feed your autonomous agent five events while away from the keyboard, then audit and score all five actions from the log.

Assessment: Explain maker-credential risk and name the two design rules you applied because of it.

Class 20: Multi-Agent Design, Analytics and Evaluations

Topics:

- Multi-agent options: child agents, connected Copilot Studio agents, and agents joined over the A2A protocol, generally available since April 2026
- When one agent becomes two: Microsoft's own rule of thumb, split when an agent nears 30 to 40 choices of tools, topics and agents
- Orchestrating specialists: a front-desk agent delegating to a bookings agent and a policy agent, cleanly
- The analytics view: sessions, outcomes (resolved, escalated, abandoned), satisfaction and themes
- Transcripts as design feedback: reading real sessions respectfully and mining the gaps
- Agent evaluations, generally available since March 2026: test sets and graders that automate the discipline you have run by hand since Class 4
- The improvement loop with numbers: hypothesis, one change, evaluation rerun, verdict

Projects built:

- Split Fest Helpdesk into a parent agent with one child specialist, then run an analytics-and-evaluations sprint on a week of real usage and ship two evidenced improvements

Practice: Define your capstone's three success metrics and build the evaluation set that will measure them before you build the agent.

Assessment: Phase 4 gate: from provided transcripts and evaluation results, diagnose the top failure and prove your one-line fix moved the number.

PHASE 5: PUBLISH, CREDENTIALS & CAPSTONE

Publish to the channels businesses use, pass a professional pre-flight checklist, then build and defend a business-grade capstone agent, leaving with a mapped Microsoft credential roadmap.

Class 21: Channels and the Agent Store

Topics:

- The channel lineup: Microsoft Teams and Microsoft 365 Copilot, SharePoint, demo website, custom website, and WhatsApp in business configurations
- Publishing flow: from studio to a shareable demo website, then toward Teams where organisations actually work
- The Agent Store: how a published agent reaches Microsoft 365 Copilot after an admin approves it, a live lesson in enterprise governance
- Identity for agents: Microsoft now issues Entra Agent IDs to new agents, making agents governable like users, mandatory since July 2026
- Channel-fit design: the same agent on a phone, a website and inside Teams, and what changes per surface
- Versioning discipline: what re-publishing replaces, and how not to break live users
- Your capstone's launch plan: channel, audience, access and rollback, on one page

Projects built:

- Publish Internship Tracker to the demo website, run a supervised session with two real users on their phones, and file the structured feedback

Practice: Write the one-paragraph launch plan for your capstone: channel, audience, access decision and rollback trigger.

Assessment: Explain what changes and what stays identical when one agent ships to two channels.

Class 22: The Pre-Flight Checklist: Security, Cost and Honesty

Topics:

- The professional pre-flight: scope fences retested, refusals verified, citations on, personal data audited, evaluation suite green
- Prompt injection: how malicious content tries to hijack instructions through knowledge and inputs, and the mitigations that blunt it
- Data minimisation: collecting the least, retaining the least, and being able to say so
- Cost awareness in Copilot Credits: metered usage (generative answers, actions, flow actions) means design choices change the bill, and capacity has enforcement limits
- Transparency obligations: users know they are talking to an agent, what it can do, and where its knowledge ends
- Kindness and accessibility passes: language level, error tone, and the user having a bad day
- The signed go / no-go: a written audit your mentor countersigns before capstone launch

Projects built:

- Run the full 20-point pre-flight checklist against your published agent, fix every failure, and produce the signed one-page audit

Practice: Attempt three prompt-injection attacks on your own agent through its knowledge and inputs, document outcomes, and add one mitigation.

Assessment: Audit a deliberately flawed agent spec and catch at least four of its five planted problems.

Class 23: Capstone Build Week

Topics:

- Freezing the spec: job, users, sources, tools, trigger, channel and success metrics on one page
- Build order that works: knowledge first, topics second, tools third, trigger last
- Timeboxing with a mentor: what gets cut when reality bites, and why the core loop is protected
- The evaluation suite as safety net: rerun at every milestone, no green no merge
- Professional polish: greeting, description, card consistency, error tone and escalation paths
- Dress rehearsal: the timed demo run with one planned failure recovery
- Documentation packet: spec, source list, tool contracts, test log, audit page, ready for a recruiter

Projects built:

- Build the complete capstone: a business-grade agent for a real club, department or small business, grounded, tooled, evaluated, published and documented

Practice: Run the dress rehearsal for an outsider and cut or fix the two weakest moments they identify.

Assessment: Milestone gate: the capstone passes its own evaluation suite and pre-flight audit before demo day.

Class 24: Demo Day and the Credential Roadmap

Topics:

- The live demo: your capstone on a real channel, citations showing, one failure recovered gracefully
- Defending design decisions: these sources, these tools, this orchestration mode, this trigger, and why
- The retrospective: what you would build differently, the slide interviewers respect most
- Portfolio packaging: capstone plus project history, browsable by a recruiter in five minutes
- The Microsoft credential ladder mapped to what you now know: PL-900 with its new Copilot Studio agents domain, the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities', and the AI Agent Builder Associate certification (exam AB-620)
- The market you are entering: Gartner projects that by 2029 at least half of knowledge workers will need to work with, govern or create AI agents
- Certificate, feedback and the Living Syllabus promise: what changed in Copilot Studio while you were learning it

Projects built:

- Deliver the capstone demo and hand over the complete packet: spec, source list, tool contracts, evaluation results, audit page and a two-minute recorded walkthrough

Practice: Draft your AB-620 preparation plan: which exam domains your coursework already covers and what remains.

Assessment: Final evaluation: capstone demo, design defence and documentation packet, graded against the Class 20 rubric.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a business agent teardown, Placement FAQ v1 with designed instructions, a corporate refusal gauntlet, a permanent 15-question test set, and a model comparison memo with evidence
- Phase 2: a citation-backed Placement FAQ grounded in a real company site, Society Desk with an authored 3-document knowledge pack, a two-source Internship Tracker with conflict handling, and a published accuracy audit
- Phase 3: Fest Helpdesk with designed topics, entity-driven application capture, a condition-driven registration flow with escalation, a card-based pipeline view, and a generative-orchestration migration with verdicts on every change
- Phase 4: a JD-decoder Prompt tool, the Expense Slip agent flow with graceful errors, a live REST API tool, an MCP server connected with deliberately scoped tools, the autonomous Status Reporter with approval pauses, and a multi-agent split with an evaluations sprint
- Phase 5: a published agent tested by real users, a signed 20-point pre-flight audit, the complete business capstone, and a demo-day defence with a recruiter-ready documentation packet
- Total: 24 hands-on projects, one per class, finishing with a published business-grade capstone agent

Total projects: 24 hands-on projects, one per class, finishing with a published business-grade capstone agent

SKILLS THE STUDENT WILL HAVE AT THE END

- Agent design: instructions, scope fences, refusals, and brand-appropriate tone
- Knowledge grounding at business depth: SharePoint, documents, Dataverse and connector-indexed enterprise data, with citations
- Conversation design: topics, trigger phrases, entities, variables, condition groups and adaptive cards
- Generative vs classic orchestration, selected and defended per use case
- Tools: Prompt tools, prebuilt and custom connectors, REST APIs and Model Context Protocol
- Agent flows: deterministic automation with error paths and readable runs
- Autonomous agents: event triggers, Recurrence schedules, maker-credential security reasoning and human-in-the-loop design
- Multi-agent architecture: child agents, connected agents and the A2A protocol
- Analytics and agent evaluations: outcomes, transcripts, graders and evidence-driven iteration
- Publishing and governance: channels, the Agent Store approval path, Entra Agent IDs, DLP awareness and Copilot Credits cost literacy

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 private 1-on-1 classes per week, about 1 hour each, across 12 weeks
Hands on agent work: 1.5-2 hours building and testing in Copilot Studio between classes
Project time: 1-1.5 hours completing each class project, usually inside your own agents
Review and doubts: 30-45 minutes reviewing feedback and clearing doubts over WhatsApp before the next class

SUPPORT INCLUDED

Dedicated mentor: The same mentor for all 24 classes, who knows exactly where your agents, portfolio and preparation stand
Whatsapp doubts: Doubt support between classes over WhatsApp
Practice environment: Help setting up your Copilot Studio environment in Class 1, including the trial path for college accounts and an arranged practice environment where needed
Session recordings: Class recordings available for revision, so a missed detail is never lost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'AI Agents with Copilot Studio' certificate, earned by presenting and defending your Class 24 capstone agent
Project portfolio: A published business-grade capstone plus 24 documented projects, packaged as a portfolio a recruiter can browse in five minutes
Next credentials: A mapped path to PL-900 Power Platform Fundamentals (now with a dedicated Copilot Studio agents domain), the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities', and Microsoft Certified: AI Agent Builder Associate (exam AB-620)

WHERE THIS CAN LEAD

- AI agent builder or automation specialist roles, the job titles now appearing in listings as enterprises staff their agent programmes
- Power Platform maker and business technologist tracks, where Copilot Studio fluency is the differentiator
- Microsoft's credential ladder with course-to-exam mapping: PL-900 Power Platform Fundamentals, the Applied Skills labs (APL-6006 and APL-6000), and Microsoft Certified: AI Agent Builder Associate (AB-620)
- Operations, support and analyst roles upgraded: the person who automates the workflow rather than performing it manually
- A bridge to pro-code agent work through our Codex + Claude Code masterclass, with MCP knowledge that transfers directly

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need to know how to code to take this Copilot Studio course?

A: No. Microsoft Copilot Studio is a graphical, low-code studio: agents are designed with plain-English instructions, curated knowledge sources and a visual canvas. The course teaches everything from first principles. If you do code, the REST API, custom connector and Model Context Protocol classes give you room to go deeper, but no class assumes programming.

Q: What will I actually build during the 24 classes?

A: One real project per class: a placement FAQ agent grounded in a real company's site with checkable citations, a society helpdesk with designed topics and condition logic, an internship tracker with entity-driven capture and card views, an expense automation built on an agent flow, a live REST API tool, an MCP server integration, an autonomous status reporter running on event triggers with approval pauses, a multi-agent split, and a published business-grade capstone with a full documentation packet.

Q: Why is this course 1-on-1 only?

A: Because the skill lives in artifacts that need individual review: your instructions, source lists, flow designs, tool contracts and evaluation results. A private mentor reads and critiques all of them every class, and rebuilds projects around your actual clubs, departments and placement goals. That review loop is what produces students who can defend their design decisions in interviews, and it cannot be delivered honestly to a batch.

Q: What Microsoft account do I need, and is there a free way to practise?

A: Copilot Studio uses a Microsoft work or school (Entra ID) account rather than a personal email. Most college email addresses qualify for Microsoft's 30-day self-serve trial, which covers in-studio building and testing but cannot publish agents. Where a suitable account is not available, we arrange a practice environment for the duration of the course. Your mentor completes the setup with you in Class 1.

Q: Does this course prepare me for Microsoft certifications?

A: Yes, deliberately. The syllabus vocabulary maps to PL-900 Power Platform Fundamentals, which added a dedicated Copilot Studio agents domain in its July 2026 skills update, and to the hands-on Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance

agents with autonomous capabilities'. The headline credential is Microsoft Certified: AI Agent Builder Associate (exam AB-620), whose domains (plan and configure, integrate and extend, test and manage) mirror Phases 2 through 5, and Class 24 ends with your personal AB-620 preparation plan.

Q: Is Copilot Studio genuinely an employable skill or a passing trend?

A: The deployment numbers are unusually concrete: Microsoft states that 90% of the Fortune 500 use Copilot Studio, and reported more than 230,000 organizations building with it as far back as April 2025. Gartner projects that by 2029 at least half of knowledge workers will need skills to work with, govern or create AI agents, and that a third of enterprise software will include agentic AI by 2028. The course also teaches the governance and evaluation discipline that outlasts any single platform.

Q: How current is the syllabus, given how fast the platform changes?

A: It is a Living Syllabus, re-verified against Microsoft's official documentation and What's New pages as features ship. The current syllabus covers generative orchestration, the present model lineup (GPT-4.1 default, GPT-5.5 Chat, and Anthropic Claude models), Model Context Protocol tools, the A2A multi-agent protocol, agent evaluations and Entra Agent IDs, all of which reached the platform within roughly the last year.

Q: How is this different from your Codex + Claude Code course?

A: They are complementary tracks. Codex + Claude Code is pro-code: you drive AI coding agents from the command line to ship software. This course is the low-code business track: you design, ground and govern agents in Microsoft Copilot Studio, the way operations, support and business teams deploy them. MCP knowledge transfers between both. Many students take this course first for the portfolio and vocabulary, then go pro-code.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-college-students>

=====

COURSE 8 OF 118: AI AGENTS WITH MICROSOFT COPILOT STUDIO FOR TEENS

=====

Course link: <https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-teens>

Subject area: AI Agents & Automation

Designed for: Teens

Who can join: Beginner (ages 13-18; no coding experience needed)

Course length: 24 classes (12 weeks · 2 classes/week)

Time needed: 3-4 hours/week recommended

Certificate: Modern Age Coders 'AI Agents with Copilot Studio' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

Note: this course also needs the student to have their own Claude and ChatGPT subscriptions, because the whole course is hands on with those tools. Say this before enrolling anyone, so it is never a surprise later.

WHAT THIS COURSE IS

Build real AI agents with Microsoft Copilot Studio in 24 live 1-on-1 online classes for ages 13-18: knowledge grounding, topics, connectors, agent flows, autonomous triggers and a published capstone agent. No heavy coding needed.

In one line: Stop just chatting with AI. Design, ground, and publish your own AI agents on the platform 90% of the Fortune 500 already use, in private 1-on-1 classes built around your ideas.

ABOUT THE PROGRAMME

Every teenager has used a chatbot. Almost none of them have built an agent: an AI that knows a specific subject deeply, follows instructions it was given, uses tools, and acts on its own when something happens. That is exactly the skill this course teaches, on Microsoft Copilot Studio, the graphical low-code studio Microsoft describes as the way to build and manage AI-powered agents and workflows. Across 24 live 1-on-1 classes you go from your very first agent (described in plain English, tested the same hour) to agents grounded in real knowledge sources with citations, conversation design with topics and trigger phrases, tools built from connectors and agent flows, an autonomous agent that wakes up on an event trigger and works without you, and a published capstone agent demonstrated live on a website and shared for feedback. Along the way you learn the ideas that make agents trustworthy: grounding versus guessing, why an agent should cite its sources, what an agent should refuse to do, and how professionals measure whether an agent actually helped. Because every class is private, your mentor builds the whole course around agents you actually care about: a study buddy grounded in your own class notes, a quiz coach for your next exam, a club FAQ agent, a family trip planner. The syllabus is a Living Syllabus, updated as Copilot Studio ships new features, so you learn the current platform, never a frozen snapshot of last year's tool.

WHAT MAKES THIS COURSE DIFFERENT

- 1-on-1 only, by design: agent building is a design discipline, and a private mentor can review your instructions, your knowledge sources, and your conversation maps line by line, which a group class simply cannot do.
- Real platform, real skills: you work in Microsoft Copilot Studio itself, the same low-code agent studio used across enterprises, with topics, knowledge, tools, triggers and analytics, not a toy sandbox.
- No heavy coding needed: Copilot Studio is a graphical, low-code studio, so the course runs on clear thinking and good instructions; teens who do code can go further with connectors and REST APIs in the same syllabus.
- Agents that act, not just answer: by Phase 4 your agent responds to event triggers (a file added, a task completed, a schedule) and works autonomously, which is the difference between a chatbot and an agent.
- Honest AI habits from Class 1: grounding answers in sources, keeping citations on, deciding what an agent must refuse, and testing before publishing, the same responsible-AI defaults professionals are held to.
- Living Syllabus: Copilot Studio ships new models and features continuously (the current lineup includes GPT-5.5 Chat and Anthropic Claude models), and the curriculum is updated as they land, so nothing you learn is stale.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac or Linux) with the Chrome or Edge browser and a reliable internet connection; Copilot Studio runs fully in the browser

Coding experience: None needed. Copilot Studio is a graphical, low-code studio; clear thinking and good writing matter far more than syntax. Teens who already code can go deeper into connectors and REST tools inside the same syllabus

Accounts: Copilot Studio uses a Microsoft work or school account rather than a personal email. Many students already have one from school; if you do not, we arrange access to a practice environment for the course, and for students under 18 a parent or guardian accepts Microsoft's terms

Mindset: Curiosity about how AI actually works, and the patience to test before you trust. The course rewards students who ask 'how do I know this answer is right?'

WHO THIS COURSE IS FOR

Curious builders: Teens who use ChatGPT or Copilot daily and are ready to move from asking AI questions to building AI that answers them

Future professionals: Students eyeing careers in technology, product or business who want the agent-building vocabulary employers now test, on the platform 90% of the Fortune 500 use

Non coders: Teens who thought AI was locked behind programming: Copilot Studio's low-code canvas makes design thinking, not syntax, the superpower

Young coders: Students from our Python or web development courses who want to see where low-code agents meet APIs, REST and Model Context Protocol

Portfolio hunters: Teens building a portfolio for internships or university applications who want a published, working agent to show, not a certificate alone

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: YOUR FIRST AGENTS

Understand what an AI agent actually is, tour Microsoft Copilot Studio, build your first agent from a plain-English description, and learn how instructions and testing really work.

Class 1: What an AI Agent Really Is (and What It Is Not)

Topics:

- Chatbot vs assistant vs agent: an agent has instructions, knowledge, tools and can act, not just answer
- Agents you already meet every week: support bots, order trackers, school helpdesks, and what they get right and wrong
- Microsoft Copilot Studio in one sentence: a graphical, low-code studio for building and managing AI-powered agents and workflows
- The guided tour: agents, knowledge, topics, tools, triggers, channels and analytics, and what each tab is for
- How access works honestly: Copilot Studio uses a Microsoft work or school account, and your mentor gets your practice environment ready with you in this class
- The anatomy of every agent you will build: name, description, instructions, knowledge, tools, and where each lives
- Setting the course goal: the capstone agent you will publish in Class 24, chosen from your own school, club or family life

Projects built:

- Agent safari: analyse two real agents you have used (a support bot, a game helper), map what their instructions and knowledge must look like, and write a one-page teardown

Practice: List five agent ideas from your own week (study, club, family, hobby) and rank them by how much real knowledge each would need.

Assessment: Explain the difference between a chatbot and an agent in two sentences, using the words instructions, knowledge and tools correctly.

Class 2: Your First Agent in One Hour

Topics:

- Describe-to-create: starting an agent from a plain-English description of what it should do
- Writing your first instructions: role, audience, tone, and the three behaviours your agent must always show
- The test panel: talking to your agent the moment it exists, and reading its answers critically
- Iteration as the core loop: change one instruction line, retest, compare, keep or revert
- Naming, descriptions and icons: why a clear agent description matters for users and for you
- What the model does with your instructions: a teen-friendly picture of how a large language model follows a system prompt
- Saving versions of your instruction text outside the tool, the habit that saves every project later

Projects built:

- Build 'Study Buddy v1': an agent with instructions for your real timetable and subjects that answers revision questions in a tone you designed

Practice: Rewrite your agent's instructions three different ways and record how each version changes the same test question's answer.

Assessment: Show a before-and-after: one instruction change and the exact behaviour difference it caused in the test panel.

Class 3: Instructions That Actually Steer Behaviour

Topics:

- The instruction stack: identity, scope, tone, format rules, and refusals, in that order
- Scope fences: telling an agent what it must NOT do, and testing that the fence holds
- Tone control for real audiences: the same agent for classmates vs teachers vs grandparents
- Format discipline: making answers arrive as steps, tables or short paragraphs on demand
- Refusal design: choosing the questions your agent should decline (medical advice, personal data, exam cheating) and wording the refusal kindly
- Instruction smells: vague verbs, contradictions, and instructions that fight each other
- A/B testing instructions: running the same 10 questions against two instruction sets and scoring the answers

Projects built:

- The Refusal Gauntlet: write 10 questions your Study Buddy must refuse or redirect, then tune instructions until all 10 are handled gracefully

Practice: Take a classmate's (or parent's) three trickiest questions and tune your instructions until the answers pass their review.

Assessment: Present your instruction stack and defend each line: what breaks if it is removed?

Class 4: Testing Like a Professional

Topics:

- Why 'it answered nicely once' is not evidence: building a fixed test set of 15 questions
- Happy path, edge case, and adversarial questions: three test types every agent needs
- Reading answers critically: correctness, tone, format, and whether the agent stayed in scope
- Regression testing by hand: rerunning your test set after every meaningful change
- Keeping a test log: question, expected behaviour, actual behaviour, verdict
- When the agent is confidently wrong: recognising hallucination before you have knowledge sources to fix it
- The professional habit this course grades you on: no change ships without a retest

Projects built:

- Build your permanent 15-question test set for Study Buddy, with expected behaviour written down before you run it

Practice: Run your full test set twice this week and log every verdict; bring the two failures you found to class.

Assessment: Given a transcript of an agent conversation, mark every line as pass or fail against the written instructions.

Class 5: How the Platform Thinks: Environments, Models and Harnesses

Topics:

- Where agents live: environments, and why your practice environment is separate from a company's production one
- The model picker: Copilot Studio agents run on a chosen model (the lineup currently includes GPT-4.1 as default, GPT-5.5 Chat, and Anthropic Claude models), and what changing it does
- Harnesses in plain words: the runtime between your design and the model, and why Microsoft now ships more than one
- What is free and what costs money: Copilot Credits exist, adults pay for capacity, and testing in class is covered by your practice environment
- Responsible-AI defaults you inherit: content filters and guardrails the platform applies before your instructions even run
- Saving and organising agents so week-12-you can find what week-2-you built
- Checkpoint: everything in one map, from your typed instruction to the model's answer

Projects built:

- Draw the journey of one question through your agent: user, harness, instructions, model, answer, and label the parts you control

Practice: Try the same three test questions on two different models in the picker and

write down every difference you can defend.

Assessment: Phase 1 review: rebuild a small agent from a written spec in 20 minutes, with instructions, a refusal, and a 5-question test.

PHASE 2: KNOWLEDGE & GROUNDING

Give your agent real knowledge: websites, documents and SharePoint sources, citations users can check, and the discipline of grounding answers instead of guessing.

Class 6: Grounding vs Guessing

Topics:

- Why a bare model guesses: training data vs your data, in a picture a friend could follow
- Knowledge sources in Copilot Studio: public websites, uploaded documents, SharePoint, Dataverse, and connector-indexed enterprise data
- Generative answers: how the agent searches your sources and composes an answer from what it found
- Citations: why the little source link is the most trustworthy pixel on the screen
- The 'Allow ungrounded responses' setting, and why this course keeps it off by default
- Limits that shape design: sources per agent, website counts, and file size ceilings exist, so knowledge is curated, not dumped
- Choosing knowledge for your capstone: authoritative, current, and small enough to verify

Projects built:

- Ground Study Buddy in one real source (your school's public website or a subject site) and prove three answers now cite it

Practice: Ask your grounded agent five questions its source cannot answer and observe exactly how it behaves at the edge of its knowledge.

Assessment: Explain grounding, citation and hallucination to a parent in under two minutes; they grade you.

Class 7: Document Knowledge Done Right

Topics:

- Uploading documents as knowledge: what formats work well and what makes a document searchable
- Writing documents FOR agents: headings, one fact per paragraph, and names spelled consistently
- The curation habit: five great pages beat fifty messy ones, every time
- Testing document knowledge: questions whose answers live in one exact paragraph
- Updating knowledge: what happens when the source document changes and the agent must not lag behind
- Privacy first pass: what should never go into an agent's knowledge (family data, friends' names, anything you would not print)
- Building the knowledge pack for your own subjects: notes that become a genuinely useful revision agent

Projects built:

- Create a clean 3-document knowledge pack from your own class notes and ground a Revision Coach agent in it, with a 10-question proof test

Practice: Deliberately break one document (vague headings, mixed topics), watch answer quality drop, then fix it and retest.

Assessment: Given two versions of the same document, predict which grounds better and verify by testing both.

Class 8: Multi-Source Knowledge and Conflicts

Topics:

- Combining sources: website plus documents plus a SharePoint folder, and how the agent picks
- When sources disagree: seeing conflicting-answer behaviour and designing around it
- Source scoping: pointing a question type at the right source with instructions
- Freshness: preferring the source that updates (a website) for facts that change
- The web question: what 'Use information from the web' changes, and when a curated agent should leave it off
- Auditing an agent's knowledge like a librarian: what is in, what is out, what is stale
- Knowledge design for the capstone: writing the source list you will defend in Class 24

Projects built:

- Build 'Club Desk': an agent for a real club or activity grounded in two sources, with documented behaviour when sources conflict

Practice: Write five questions that force your agent to choose between its sources and log which one wins each time.

Assessment: Defend your capstone source list: for each source, why it is authoritative and how you will keep it current.

Class 9: Honest Agents: Accuracy, Bias and Boundaries

Topics:

- Measuring accuracy properly: scoring 20 grounded answers against the source text itself
- Bias in, bias out: how a one-sided source list produces a one-sided agent
- Uncertainty language: making the agent say 'my sources do not cover that' instead of improvising
- Personal data rules for minors, in plain words: what an agent about your life may never store or repeat
- Academic honesty: a revision agent that teaches you vs one that does your homework, and where this course draws the line

- What professionals add later: evaluation suites and monitoring, previewed honestly so you know where this road goes
- Phase 2 wrap: the grounding checklist you will apply to every agent from now on

Projects built:

- Accuracy audit: score 20 answers from Revision Coach against the actual source paragraphs and publish your agent's accuracy number with examples

Practice: Add an uncertainty rule to every agent you own and collect three real 'my sources do not cover that' responses.

Assessment: Phase 2 exam: given a topic, design the full knowledge plan (sources, scoping, refusals, accuracy test) in writing.

PHASE 3: CONVERSATION DESIGN

Design conversations deliberately: topics and trigger phrases, questions and variables, condition logic, adaptive cards, and the two orchestration modes that decide how your agent thinks.

Class 10: Topics and Trigger Phrases

Topics:

- What a topic is: a designed conversation path for one job your agent does
- Trigger phrases: teaching the agent the many ways people ask for the same thing (topics support up to 200)
- The topics you get for free: greeting, goodbye, escalate, and why system topics exist
- Building a topic on the authoring canvas: message nodes, question nodes, and the flow between them
- When to use a topic vs when to let generative answers handle it: control vs flexibility
- Peeking under the canvas: every topic is also code (YAML), the first hint that low-code and pro-code meet
- Designing your agent's topic map: the five jobs users will actually ask it to do

Projects built:

- Build a 'Homework Planner' agent with three hand-designed topics: add a task, list this week, and what should I do first

Practice: Collect ten real phrasings of one request from friends or family and add the best ones as trigger phrases, then test recognition.

Assessment: Given three user messages, predict which topic each will trigger and verify live.

Class 11: Questions, Entities and Variables

Topics:

- Question nodes: asking for exactly the information a topic needs, one clean question at a time
- Entities: how the agent extracts a date, a number, a choice from a messy sentence
- Variables: storing what the user said and reusing it later in the conversation
- Slot filling: when a user gives three answers in one sentence and the agent skips already-answered questions
- Choice questions vs open questions: constraining answers when structure matters
- Validating input: what happens when the user answers a date question with 'idk, whenever'
- Conversation memory across a session: what your agent remembers and what it forgets

Projects built:

- Upgrade Homework Planner: the add-a-task topic now collects subject, due date and effort through entities and confirms with a summary using variables

Practice: Test the same topic with cooperative, messy and hostile answers; log where extraction fails and tighten the questions.

Assessment: Trace a conversation on paper: name the entity and variable at each step before running it live.

Class 12: Condition Logic and Branching

Topics:

- Condition nodes: sending the conversation down different paths based on variables
- Condition groups: combining tests (subject is maths AND due tomorrow) without spaghetti
- Designing branches on paper first: a flow diagram before ever touching the canvas
- The escalation branch: recognising 'this needs a human' and saying so
- Default branches: the path for everything you did not predict
- Testing branch coverage: one test conversation per path, no path untested
- Refactoring topics: splitting one giant topic into three small clean ones

Projects built:

- Build 'Event Helper' for a real school or family event: RSVP path, question path, and organiser-escalation path, all condition-driven

Practice: Draw your Event Helper flow as a diagram, then have your mentor try to reach a dead end you did not design.

Assessment: Given a flow diagram with a logic bug, find and fix the bad branch in one session.

Class 13: Adaptive Cards and Rich Responses

Topics:

- Beyond plain text: adaptive cards for structured, tappable answers
- Card anatomy: title, facts, images, actions, and when a card beats a paragraph
- Designing cards for small screens: your agent will mostly be read on phones

- Buttons that continue the conversation: card actions feeding back into topics
- Consistency: a small card style guide so your agent looks designed, not assembled
- Accessibility basics: alt text and reading order, because agents are for everyone
- When NOT to use a card: the restraint that separates designers from decorators

Projects built:

- Give Homework Planner a card-based weekly view: each task a card with subject, due date and a 'mark done' action

Practice: Convert your three most-used answers into cards and A/B them with two real users against the text versions.

Assessment: Card critique: improve a deliberately ugly card (walls of text, no hierarchy) and justify each change.

Class 14: Generative Orchestration vs Classic

Topics:

- Two brains, one agent: classic orchestration (trigger phrases pick topics) vs generative orchestration (the model plans across topics, knowledge and tools)
- What generative orchestration unlocks: multi-intent requests, tool choice, and the autonomous triggers you will use in Phase 4
- What you trade: predictability, and how instructions rein the planner back in
- Watching the plan: reading how the agent decided which topic, knowledge source or tool to use
- Mixing modes wisely: designed topics for the critical paths, generative flexibility for the long tail
- Guardrails under orchestration: scope fences and refusals still rule, and you retest them now
- Phase 3 wrap: your conversation-design checklist, from trigger phrases to orchestration choice

Projects built:

- Switch Event Helper to generative orchestration, rerun your full test set, and document every behaviour change with a keep-or-fix verdict

Practice: Craft three multi-intent messages ('RSVP yes and what time and can my sister come') and tune until all three resolve cleanly.

Assessment: Phase 3 exam: design a five-topic agent on paper (topics, entities, branches, cards, orchestration mode) and defend the design.

PHASE 4: TOOLS & AUTONOMY

Make your agent act: connectors and Prompt tools, agent flows for multi-step work, a first look at Model Context Protocol, then event triggers and a fully autonomous agent with analytics to prove it works.

Class 15: Tools: Giving Your Agent Hands

Topics:

- The tool menu: Prompt, Agent flow, Custom connector, Model Context Protocol and REST API, and what each is for
- Connectors: Copilot Studio's library of more than 1,400 prebuilt connections to real services
- Your first tool: a Prompt tool that does one transformation perfectly (summarise, translate, rewrite)
- Tool descriptions matter: the agent chooses tools by reading their descriptions, so you write them carefully
- Fewer, better tools: the platform allows many tools per agent, professionals deliberately keep the list short
- Safety question of the day: what is this tool allowed to touch, and who said so
- Planning the tools your capstone actually needs (usually two, rarely five)

Projects built:

- Add a 'flashcard maker' Prompt tool to Revision Coach: paste messy notes, get clean question-answer pairs, every time

Practice: Write two versions of your tool's description and show how the wording changes when the agent decides to use it.

Assessment: Explain to your mentor, without notes, how an agent decides between answering directly, using knowledge, and calling a tool.

Class 16: Agent Flows: Multi-Step Work That Always Runs the Same

Topics:

- Agent flows: deterministic step-by-step automations your agent can call as a tool
- Flow anatomy: trigger, steps, conditions, and outputs handed back to the conversation
- When a flow beats the model: anything that must happen exactly the same way every time
- Building a notify flow: collect details in conversation, send a formatted email or Teams message
- Error paths in flows: what the user sees when a step fails, and why silence is never acceptable
- Flows vs topics: designed conversation vs designed action, and how they hand off
- Reading a flow run: inputs, outputs and the step where things went wrong

Projects built:

- Build 'Permission Slip': Event Helper collects details, an agent flow sends a tidy summary email to the organiser (a parent or teacher), and the agent confirms with the flow's result

Practice: Break your flow on purpose (bad input, missing field), observe the failure, then add the error path that makes it graceful.

Assessment: Given a task description, decide flow or topic or Prompt tool, and justify the choice in three sentences.

Class 17: Connecting to the Real World: Connectors, REST and MCP

Topics:

- Using a prebuilt connector as a tool: real services (To Do, Calendar, SharePoint) inside your conversation
- Authentication in plain words: why tools ask 'connect your account' and what is being granted
- REST APIs, demystified: request in, JSON out, and how an agent turns that into an answer
- Model Context Protocol (MCP): the open standard for plugging tool servers into agents, added from the same Add-a-tool menu
- Why MCP matters beyond Microsoft: the same protocol appears in Claude, IDEs and other agent platforms you already know
- Governance preview: connector access is policy-controlled in real companies, so 'can' and 'may' are different questions
- Choosing the responsible option: the least-powerful tool that does the job

Projects built:

- Wire one real connector tool into a practice agent (task list or calendar lookup) and demonstrate a conversation that reads and writes real data

Practice: Map one API you wish existed for your life (canteen menu, bus times) and write the tool description and three test utterances for it.

Assessment: Sketch the path of one tool call end to end: utterance, orchestration, tool choice, call, result, answer.

Class 18: Event Triggers: Agents That Wake Up by Themselves

Topics:

- The autonomy switch: event triggers let an agent act with no user in the chat at all
- Trigger types that matter: item created (SharePoint), file added (OneDrive), task completed (Planner), and Recurrence for schedules
- The trigger payload: the event's data arriving as input, and instructions deciding what to do with it
- Generative orchestration required: why autonomous behaviour needs the planning brain from Class 14
- The security fact professionals underline: triggers run with the maker's credentials, so you design as if everything it does is signed by you
- Designing the first autonomous job: small, observable, reversible
- Logging what your agent did while you were at school: the activity trail habit

Projects built:

- Build 'Drop Folder Librarian': when a file lands in a watched folder, the agent files a tidy summary card automatically, on a Recurrence backup schedule

Practice: Feed your autonomous agent five files while you are away from the keyboard and audit its five actions afterwards, scoring each.

Assessment: Explain maker-credential risk in your own words and name the two design rules you applied because of it.

Class 19: Your Autonomous Agent, Hardened

Topics:

- From demo to dependable: retries, idempotence in plain words (running twice must not double the damage)
- Human-in-the-loop: which autonomous actions should pause for approval, and how to design the pause
- Notification etiquette: an autonomous agent that spams is an autonomous agent that gets turned off
- Failure diaries: capturing what went wrong so Tuesday's bug is not repeated on Thursday
- Scope creep in autonomy: resisting the urge to let the agent 'also just' do one more thing
- Multi-agent preview: child agents and connected agents exist, and when one agent should become two
- Rehearsing the demo: showing autonomous behaviour live without praying

Projects built:

- Harden Drop Folder Librarian: add an approval pause for deletes, an error diary, and a weekly self-report message, then run it for three real days

Practice: Write the one-page 'operations note' for your agent: what it does, when, with whose permissions, and how to turn it off.

Assessment: Live drill: your mentor breaks one input; your agent must fail gracefully and your diary must show why.

Class 20: Analytics: Proving Your Agent Works

Topics:

- The analytics view: sessions, outcomes (resolved, escalated, abandoned), and what each outcome means
- Reading transcripts respectfully: real conversations are the best design feedback you will ever get
- Customer satisfaction in miniature: asking users to rate your agent and surviving the answers
- Finding the gap: the questions users asked that your agent could not handle, ranked
- Evaluations, previewed: professionals run test sets automatically against agents; you have been doing it by hand since Class 4
- The improvement loop: analytics, hypothesis, one change, retest, remeasure
- Phase 4 wrap: tools, autonomy and analytics as one system, ready for the capstone

Projects built:

- Analytics sprint: put one agent in front of five real users for a week, read every session, and ship two improvements the data demanded

Practice: Write your capstone's success metrics before building it: three numbers that will define 'it works'.

Assessment: Phase 4 exam: from a set of session transcripts, diagnose the top failure and propose the one-line fix, then prove it.

PHASE 5: PUBLISH & CAPSTONE

Publish agents to real channels, pass a safety and quality checklist, then design, build, test and present a capstone agent that does a real job for real people.

Class 21: Channels: Where Agents Live

Topics:

- The channel lineup: demo website, custom website, Microsoft Teams and Microsoft 365 Copilot, SharePoint, and even WhatsApp in business setups
- Publishing flow: from the studio to a shareable demo website in one class
- How the same agent feels different per channel: phone vs desktop vs inside Teams
- Access decisions: who can reach your agent, and why 'everyone' is rarely the right answer
- Versioning discipline: what publishing again replaces, and how not to break your live users
- The Agent Store idea: in companies, published agents go through an admin's approval, a preview of real-world governance
- Choosing your capstone's channel and writing its one-paragraph launch plan

Projects built:

- Publish Homework Planner to the demo website and run a supervised test with two real users on their own phones

Practice: Collect structured feedback from your two testers (what confused, what delighted) and file it against Class 23's build time.

Assessment: Explain what changes and what stays the same when one agent is published to two different channels.

Class 22: The Pre-Flight Checklist: Safety, Quality, Honesty

Topics:

- The publishing checklist professionals use: scope fences retested, refusals verified, sources cited, personal data audited
- Prompt injection for teens: how a malicious message tries to hijack instructions, and the defensive habits that blunt it
- Data minimisation: your agent stores the least it can and forgets what it should
- Transparency: users must know they are talking to an agent, and what it can and cannot do
- Credits and cost awareness: agent usage is metered in real deployments (Copilot Credits), and design choices change the bill
- Accessibility and kindness passes: language level, error tone, and the user having a bad day
- Signing off: the written go / no-go your mentor countersigns before capstone launch

Projects built:

- Run the full 20-point pre-flight checklist against your published agent, fix the failures, and produce the signed one-page audit

Practice: Attempt three prompt-injection attacks on your own agent, document what happened, and add one mitigation.

Assessment: Checklist mastery: audit a deliberately flawed agent spec and catch at least four of its five planted problems.

Class 23: Capstone Build Week

Topics:

- Freezing the spec: the capstone's job, users, sources, tools, trigger and channel on one page
- Building in the right order: knowledge first, topics second, tools third, trigger last
- Timeboxing with a mentor: what gets cut when reality bites, and why the core loop is protected
- The test set as safety net: your fixed questions rerun after every build milestone
- Polish that matters: greeting, name, description, card style, error tone
- Dress rehearsal: the full demo run, timed, with one planned failure recovery
- Presentation craft: showing the design decisions, not just the happy path

Projects built:

- Build the complete capstone agent: grounded knowledge, at least two designed topics, one tool or agent flow, one trigger if autonomous, published to your chosen channel

Practice: Run the dress rehearsal for a family member and cut or fix the two weakest moments of the demo.

Assessment: Milestone gate: the capstone passes its own test set and pre-flight checklist before demo day.

Class 24: Demo Day and What Comes Next

Topics:

- The live demo: your capstone, in front of a real audience, on a real channel, with citations showing
- Defending design decisions: why these sources, these topics, this tool, this trigger
- The retrospective: what you would build differently, the most honest slide in any deck
- Your portfolio: the capstone plus the practice agents, packaged so a school or interviewer can browse them
- The credential road: Microsoft's PL-900 Power Platform Fundamentals now has a Copilot Studio agents domain, and the Applied Skills lab 'Build an agent in Microsoft Copilot Studio' is the natural next rung

- Where this skill goes: Gartner projects that by 2029 at least half of knowledge workers will need to work with, govern or create AI agents
- Certificate, feedback, and the Living Syllabus promise: what changed in Copilot Studio while you were learning it

Projects built:

- Deliver the capstone demo and hand over the full packet: spec, source list, test log, audit page and a two-minute recorded walkthrough

Practice: Write your next agent's one-page spec unprompted, because you will not stop at one.

Assessment: Final evaluation: capstone demo, design defence, and the complete documentation packet, graded against the rubric from Class 20.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: an agent teardown, Study Buddy v1 from a plain-English description, a refusal gauntlet, a permanent 15-question test set, and a full map of how a question travels through the platform
- Phase 2: a website-grounded Study Buddy with citations, a 3-document Revision Coach with a proof test, a two-source Club Desk with conflict handling, and a published accuracy audit
- Phase 3: a Homework Planner with designed topics, entity-driven task capture, a condition-driven Event Helper, card-based weekly views, and a generative-orchestration migration with documented behaviour changes
- Phase 4: a flashcard-maker Prompt tool, a Permission Slip agent flow with graceful errors, a real connector tool reading and writing live data, the autonomous Drop Folder Librarian, its hardened successor with approval pauses, and an analytics sprint with shipped improvements
- Phase 5: a published agent tested by real users, a signed 20-point pre-flight audit, the complete capstone build, and a demo-day presentation with a full documentation packet
- Total: 24 hands-on projects, one per class, finishing with a published capstone agent

Total projects: 24 hands-on projects, one per class, finishing with a published capstone agent

SKILLS THE STUDENT WILL HAVE AT THE END

- Agent design: instructions, scope fences, refusals and tone control
- Knowledge grounding: websites, documents and SharePoint sources with citations
- Conversation design: topics, trigger phrases, entities, variables and condition logic
- Adaptive cards and mobile-first response design
- Generative orchestration vs classic orchestration, chosen deliberately
- Tools: Prompt tools, connectors, agent flows, and a working understanding of REST and Model Context Protocol
- Autonomous agents: event triggers, recurrence schedules, human-in-the-loop approvals
- Analytics and evaluation: outcomes, transcripts, test sets and the improvement loop
- Publishing: demo and custom websites, Teams, and the pre-flight checklist that gates a launch
- Responsible AI habits: grounding, transparency, data minimisation and injection awareness

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 private 1-on-1 classes per week, about 1 hour each, across 12 weeks

Hands on agent work: 1-1.5 hours building and testing in Copilot Studio between classes

Project time: 1 hour completing each class project, usually inside your own agents

Review and doubts: 20-40 minutes reviewing feedback and clearing doubts over WhatsApp before the next class

SUPPORT INCLUDED

Dedicated mentor: The same mentor for all 24 classes, who knows exactly where your agents and your understanding stand

Whatsapp doubts: Doubt support between classes over WhatsApp

Practice environment: Help setting up your Copilot Studio practice environment in Class 1, including the account path for students under 18

Session recordings: Class recordings available for revision, so a missed detail is never lost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'AI Agents with Copilot Studio' certificate, earned by presenting and defending your Class 24 capstone agent

Project portfolio: A published capstone agent plus 24 documented class projects, packaged as a browsable portfolio

Next credentials: A clear path to Microsoft's PL-900 Power Platform Fundamentals exam (now with a dedicated Copilot Studio agents domain) and the Applied Skills lab 'Build an agent in Microsoft Copilot Studio'

WHERE THIS CAN LEAD

- A visible head start toward roles that now appear in real job listings: AI agent builder, automation specialist, Power Platform maker
- Microsoft's own credential ladder: PL-900 Power Platform Fundamentals (which now includes a Copilot Studio agents domain), then the Applied Skills lab 'Build an agent in Microsoft Copilot Studio'

- A published capstone agent and portfolio that make school projects, internship applications and university statements concrete
- The design-and-test discipline (grounding, refusals, evaluation) that transfers to every AI platform, not just Microsoft's
- A natural bridge into our Codex + Claude Code course when you are ready to build agents with real code

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does my teenager need to know coding to build AI agents in Copilot Studio?

A: No. Microsoft Copilot Studio is a graphical, low-code studio: agents are built from instructions written in plain English, knowledge sources, and a visual conversation canvas. The skills that matter are clear thinking, good writing and honest testing, and the course teaches all three.

Q: Why is this course taught 1-on-1 only?

A: Because agent building is a design discipline. The mentor reviews your child's actual instructions, knowledge sources and conversation maps line by line every class, and rebuilds the projects around their real school, clubs and interests. That per-student review loop is what makes students genuinely good at this, and it cannot be done honestly in a group format.

Q: Does my teen need a special Microsoft account for Copilot Studio?

A: Copilot Studio uses a Microsoft work or school account rather than a personal email. Many students already have a school-provided Microsoft account that works. If not, we arrange access to a practice environment for the duration of the course, and for students under 18 a parent or guardian accepts Microsoft's terms. Your mentor handles the setup with you in the first class.

Q: Is Copilot Studio a real professional skill or just a learning tool?

A: It is the same product enterprises run: Microsoft's own materials state that 90% of the Fortune 500 use Copilot Studio, and Microsoft's cloud reported more than 230,000 organizations building with it as far back as April 2025.

Q: How current is the syllabus, given how fast AI tools change?

A: It is a Living Syllabus, re-verified against Microsoft's official documentation and What's New pages as features ship. Students on the current syllabus learn the present-day platform: generative orchestration, the current model lineup including GPT-5.5 Chat and Anthropic Claude models, Model Context Protocol tools, autonomous event triggers and agent analytics. When Copilot Studio changes, the course changes.

Q: What age is this course for, and does it suit complete beginners?

A: It is designed for ages 13-18 and assumes no prior experience with coding or AI tools beyond ordinary chatting. Classes 1-5 build everything from first principles. For students younger than 13 we recommend our kids' coding and AI-literacy courses first, since Microsoft's own AI tools set 13 as the general minimum age for its consumer Copilot.

Q: What does my child have at the end to show for it?

A: A published capstone agent demonstrated live to a real audience, a portfolio of 24 documented projects, a written accuracy audit and pre-flight safety checklist they produced themselves, a Modern Age Coders certificate earned by defending the capstone, and a mapped next step toward Microsoft's PL-900 credential if they want the official ladder.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-teens>

=====

COURSE 9 OF 118: AI LITERACY FOR KIDS (AGES 8 TO 14): UNDERSTAND AI, DO NOT JUST USE IT

=====

Course link: <https://learn.modernagecoders.com/courses/ai-literacy-for-kids-course>

Subject area: AI Literacy for Kids

Designed for: Kids

Who can join: Ages 8 to 14; no prior coding required, everything is taught from scratch

Course length: 4 months (16 weeks), joinable any month

Time needed: 2 live classes/week + small weekly projects

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online AI literacy classes for ages 8 to 14. Children learn how AI actually works, how it learns from data, where it goes wrong, and how to judge it, with age-appropriate

supervised tools.

In one line: Children who understand how AI works, where its answers come from, and when to doubt it, taught safely and at the right age.

ABOUT THE PROGRAMME

Most children already use AI daily, in recommendations, filters, voice assistants and homework help. Almost none of them understand what it is doing. That gap is the actual risk: not that children use AI, but that they trust it without knowing how it works, where its answers come from, or how confidently it can be wrong. This course closes that gap. Children learn what artificial intelligence actually is, how a machine learns from examples rather than rules, why the data it learns from decides what it does, and why it can produce a confident answer that is simply false. They train simple models themselves, on data they collect, so machine learning stops being magic and becomes something they have done with their own hands. It is also built to be age-appropriate in a way most AI courses are not. Almost every mainstream AI tool sets a minimum age, commonly 13 and sometimes 18, in its own terms of service. We do not ask a nine-year-old to create an account on a service that does not permit it. Younger students work with tools designed for children and with teacher-run demonstrations, while older students in the range use more capable tools where age and parental permission allow. Nobody is left out, and no family is quietly pushed past a terms-of-service line.

WHAT MAKES THIS COURSE DIFFERENT

- Understanding first: children learn how AI works, not just which buttons to press
- Genuinely age-appropriate, because mainstream AI tools set minimum ages (commonly 13, sometimes 18) that most kids' AI courses quietly ignore
- Children train their own simple models on data they collect, so machine learning becomes concrete instead of magical
- Bias, hallucination and data quality taught in plain language, with examples children can see for themselves
- Aligned with the direction of school curricula, including computational thinking and AI now entering national syllabuses
- Live, small batches with a real teacher, never a recorded video or an app left to babysit

THE LEARNING PATH, STEP BY STEP

Phase 1: What AI is and is not, how machines learn from data, and training a first model by hand

Phase 2: Where AI goes wrong: bias, hallucination and data quality, plus judging AI output critically

Phase 3: Building simple AI-assisted projects and presenting them, with safety and honesty built in

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: WHAT AI IS, AND HOW A MACHINE LEARNS

The foundations, taught with hands-on activities rather than definitions to memorise.

Month 1: Machines, Rules and Learning (Weeks 1-4)

What Is Artificial Intelligence, Really

Topics:

- What people mean by AI, and what computers were already doing before it
- The difference between following rules and learning from examples
- Spotting the AI you already use every day
- What computers are genuinely good at, and what humans are still better at
- Why AI does not think or understand the way a person does

Projects built:

- An AI spotting hunt: find five AI systems you used this week and explain what each one decides

Practice: Sort everyday examples into rule-following or learning-from-examples, and explain each choice

Teaching a Machine by Example

Topics:

- What training data is, in plain language
- How a machine finds patterns in examples

- Labels, and why a machine needs to be told what things are
- Why more and better examples usually mean better results
- The unplugged version, done with paper before any computer

Projects built:

- Train a classmate as if they were a machine: give only examples, no rules, then test them

Practice: Build a labelled example set for a simple classification task

Training Your First Real Model

Topics:

- Using a child-safe, teacher-supervised training tool
- Collecting your own examples for two or three categories
- Training the model and testing it live
- Watching accuracy change as the examples change
- Why the machine gets some things confidently wrong

Projects built:

- Train an image or sound classifier on your own examples and demonstrate it to the class

Practice: Improve your model by adding better examples and record what changed

Assessment: Show and explain your trained model: what it does, how you taught it, where it fails

Patterns, Data and Prediction

Topics:

- How patterns in data turn into predictions
- Simple ideas of features: what the machine actually looks at
- Why a machine can be confident and still wrong
- Prediction versus certainty, in language a child can hold
- Connecting this to maths patterns they already know

Projects built:

- Predict outcomes from a small dataset by hand, then compare with the model's prediction

Practice: Pattern and prediction puzzles using real, small datasets

PHASE 2: WHERE AI GOES WRONG, AND HOW TO JUDGE IT

The critical-thinking half, which is the part that actually protects a child.

Month 2: Bias, Hallucination and Judging Answers (Weeks 5-8)

Where the Data Comes From

Topics:

- Every AI learns from data somebody collected
- What gets left out of a dataset, and why that matters
- Seeing bias appear in a model you trained yourself
- Fair and unfair examples, explored with real cases suitable for the age
- Why the machine is not being unkind, it is repeating its examples

Projects built:

- Deliberately train a model on lopsided data, then show the class exactly how it fails

Practice: Audit a small dataset and list who or what is missing from it

When AI Makes Things Up

Topics:

- Why a text AI can produce a confident, fluent, completely false answer
- What it means that it predicts likely words rather than looking up facts
- Checking an AI answer against a real source
- Why AI is a starting point and not a final authority
- How to tell a class or teacher when you used AI, honestly

Projects built:

- Fact-check a set of teacher-provided AI answers and mark which are wrong and why

Practice: Source-checking drills: verify five claims against real references

Assessment: A short critical-thinking test on judging AI answers

Using AI Honestly for Schoolwork

Topics:

- The difference between AI helping you learn and AI doing it for you
- Asking AI for a hint or an explanation instead of the answer
- Explaining a solution in your own words as the real test of understanding
- Why copying an AI answer cheats only yourself
- Simple, honest rules a child can actually follow

Projects built:

- Take one homework-style problem, use AI only for hints, and explain the solution back unaided

Practice: Rewrite AI explanations in your own words and check them for errors

Staying Safe and Private

Topics:

- Why AI tools have minimum ages, and what a terms of service is
- What personal information should never be typed into any AI tool
- Why what you type may be stored or used
- Talking to a parent or teacher when something feels wrong
- Deepfakes and manipulated media, at an age-appropriate level

Projects built:

- Write a family AI agreement: what is allowed, what is not, and who to ask

Practice: Sort scenarios into safe, unsafe and ask-an-adult

Assessment: End-of-phase check on safety, privacy and honest use

PHASE 3: BUILDING SOMETHING REAL AND EXPLAINING IT

Children apply everything to a project of their own and learn to present it clearly.

Month 3: Simple AI Projects (Weeks 9-12)

Block Coding Meets AI

Topics:

- Connecting a trained model to a block-coded program
- Making the program react differently to each prediction
- Testing your project with inputs it has never seen
- Fixing it when it behaves oddly
- Planning before building

Projects built:

- Build a block-coded project that responds to your trained model's predictions

Practice: Debug three deliberately broken AI projects

Designing a Project That Helps Someone

Topics:

- Choosing a problem worth solving for a real person
- Deciding whether AI is even the right tool for it
- Sketching the idea before any building
- Planning what data you would need and whether you can get it fairly
- Setting a scope you can actually finish

Projects built:

- Write a one-page plan for your capstone: the person, the problem, the data, the plan

Practice: Give and receive feedback on each other's project plans

Building the Capstone

Topics:

- Building in small steps and testing each one
- Collecting balanced training data on purpose
- Handling the cases the model gets wrong
- Being honest in your project about its limits
- Getting it working end to end

Projects built:

- Build your working capstone project

Practice: Weekly build checkpoints with teacher review

Presenting and Explaining

Topics:

- Explaining what your project does and how you taught it
- Showing where it fails, honestly, which is the mark of real understanding
- Answering questions about your data
- Speaking clearly to an audience that is not technical
- Being proud of work you can actually explain

Projects built:

- Present your capstone to the class and to your parents

Practice: Rehearse the presentation and take questions

Assessment: Final capstone presentation, assessed on understanding rather than polish

WHERE THIS CAN LEAD

- A genuine head start for school computing and the AI topics entering national curricula
- A natural next step into Python and machine learning as a teenager
- Critical-thinking habits that apply well beyond technology
- Comfort with data and patterns that supports maths and science
- An informed, safe relationship with tools they will use lifelong

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is my child old enough for AI tools?

A: This is exactly why the course is built the way it is. Most mainstream AI tools set a minimum age in their terms of service, commonly 13 and sometimes 18. We never ask a child to create an account on a service that does not permit their age. Younger students use tools designed for children and teacher-run demonstrations; older students use more capable tools where age and parental permission allow.

Q: Will my child just learn to use ChatGPT?

A: No, and that is the point. The course is about understanding: what AI is, how a machine learns from examples, why data quality decides the outcome, and why an AI can be confidently wrong. Children train their own simple models on data they collect. Using a chatbot well is a small part, taught honestly, and always as a starting point rather than an authority.

Q: Does my child need to know coding first?

A: No. Everything is taught from scratch. The early weeks are largely unplugged and hands-on, and where coding is used it is block-based and taught step by step. Children who already code will still be stretched, because the difficulty here is in the thinking, not the typing.

Q: How is this different from a normal coding class?

A: A coding class teaches your child to build software. This teaches your child to understand

and judge systems that learn, which is a different skill. It pairs naturally with a coding course but does not replace one.

Q: What ages are in each batch?

A: Batches are grouped by age band within the 8 to 14 range, so an eight-year-old is never placed with a fourteen-year-old. The concepts stay the same; the activities, tools and depth are matched to the band. We place your child after the free demo class.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ai-literacy-for-kids-course>

=====

COURSE 10 OF 118: AI TOOLS FOR TEENS (13+): PROMPTING, CREATIVE PROJECTS & SAFE AI USE

=====

Course link: <https://learn.modernagecoders.com/courses/teens-ai-tools-mastery-course>
Subject area: AI Education for Teens
Designed for: Teens
Who can join: Complete Beginner to Confident AI User
Course length: 6 months (26 weeks)
Time needed: 5-8 hours/week recommended
Certificate: Certified AI Creator for Youth

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 6-month AI course for teens aged 13+: prompt engineering, AI image, audio, video and code tools, real projects, and how to use AI honestly. Every tool is one a 13+ student can actually sign up for.

In one line: From AI Curious to AI Creator - Master the Future Today

ABOUT THE PROGRAMME

This revolutionary 6-month program transforms teens into confident AI users and creators. Students will master prompt engineering, explore AI tools chosen for their age, create amazing projects, and develop critical thinking about AI's role in society. Perfect for ages 13-18, this course covers everything from ChatGPT conversations to creating AI art, music, videos, games, and even AI-assisted coding. Students will build an impressive portfolio while learning AI safety, ethics, and future-ready skills.

WHAT MAKES THIS COURSE DIFFERENT

- Plain-language approach to complex AI concepts
- Hands-on with 50+ real AI tools
- Safety-first approach with parental controls
- Creative project-based learning
- Ethical AI understanding from day one
- Age-appropriate content and examples
- Future career skill development
- No coding required (but coding AI tools included)

THE LEARNING PATH, STEP BY STEP

Month 1: AI Fundamentals & ChatGPT Mastery
Month 2: AI Art & Image Generation
Month 3: AI Music, Audio & Video Creation
Month 4: AI for Learning & Productivity
Month 5: AI Coding, Apps & Advanced Tools
Month 6: AI Projects & Future Skills

WHAT A STUDENT NEEDS BEFORE STARTING

Age: Ages 13-18 (multiple difficulty tracks)
Experience: No prior AI or coding experience needed
Equipment: Computer or tablet with internet connection
Software: Most AI tools used are free or have free tiers - guidance provided
Curiosity: Interest in AI, creativity, and building cool things

WHO THIS COURSE IS FOR

Students: Teenagers (13-18) curious about AI
Working professionals: Teachers and parents learning alongside their teenager
Entrepreneurs: Young entrepreneurs exploring AI opportunities
Freelancers: Future AI content creators and developers

Creative teens: Creative teenagers who want to use AI tools well
Anyone: Anyone passionate about using AI creatively

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

MONTH 1: AI FUNDAMENTALS & CHATGPT MASTERY

Understand AI basics, master ChatGPT, and learn the art of prompt engineering.

Introduction to AI World

Topics:

- What is Artificial Intelligence? Simple explanation
- AI in everyday life: Siri, Alexa, YouTube recommendations
- How AI learns: Machine Learning basics for teens
- Types of AI: Narrow AI vs General AI
- AI history: From calculators to ChatGPT
- AI vocabulary: Model, training, dataset, algorithm
- AI myths vs reality
- How AI 'thinks' differently from humans
- AI limitations and capabilities
- Setting up safe AI accounts with parental guidance
- Creating AI learning journal
- Class rules for AI usage

Projects built:

- My First AI Conversation
- AI vs Human: Comparison chart
- Draw AI's journey timeline
- Play and analyze AI games

Practice: Spot five places AI helped you this week, like YouTube suggestions or a voice assistant, and mark which one felt the smartest.

ChatGPT Fundamentals

Topics:

- ChatGPT interface walkthrough
- Free vs paid versions understanding
- How to start conversations
- Basic prompt structure
- Being polite with AI (digital etiquette)
- Getting better responses
- When ChatGPT makes mistakes
- Fact-checking AI responses
- Using ChatGPT for homework help (ethically)
- ChatGPT for creative writing
- ChatGPT for explanations
- Saving and organizing chats

Projects built:

- Interview ChatGPT about its capabilities
- Create a story with ChatGPT
- Get homework help (with citations)
- Plan a birthday party with AI

Practice: With a parent nearby, ask ChatGPT three questions about your favorite animal and note which answer surprised you most.

Prompt Engineering Basics

Topics:

- What is a prompt?
- Anatomy of a good prompt
- Clear instructions importance
- Context and background information
- Specific vs vague prompts
- Using examples in prompts
- Role-playing prompts ('Act as a...')
- Step-by-step prompts
- Output format specification
- Temperature and creativity settings
- Iterative prompting (improving results)
- Prompt templates creation

Projects built:

- Create prompt template library
- Before/After prompt improvements
- Prompt engineering challenge
- Build custom AI assistant personality

Practice: Rewrite one vague prompt like 'draw a dog' into a detailed prompt with a place, a mood and a style, then compare the two answers.

Advanced ChatGPT Techniques

Topics:

- Multi-turn conversations
- Teaching ChatGPT your preferences
- Custom instructions setup
- Using ChatGPT for learning
- Language translation with AI
- Code explanation with ChatGPT
- Math problem solving
- ChatGPT for research projects
- Creating study guides
- Quiz and flashcard generation
- ChatGPT plugins overview
- Web browsing with ChatGPT

Projects built:

- Create personalized study assistant
- Build interactive quiz game
- Research project on favorite topic
- Multi-language story creation

Practice: Hold a five turn conversation where you teach the AI about your favorite hobby, then ask it to quiz you on it.

MONTH 2: AI ART & IMAGE GENERATION

Master AI image generation tools, create stunning artwork, and understand visual AI.

Introduction to AI Art

Topics:

- How AI creates images
- Understanding diffusion models (simple explanation)
- Text-to-image concept
- AI art history and evolution
- Famous AI artworks
- Copyright and AI art basics
- Ethical considerations in AI art
- Different styles AI can create
- Resolution and quality settings
- Image composition basics
- Color theory for AI art
- Planning AI art projects

Projects built:

- First AI artwork collection
- Illustrate a short story
- Create fantasy creatures
- Design dream bedroom with AI

Practice: Describe your dream bedroom in words, generate an image from the description and circle two details the AI got right.

AI Image Generation Mastery

Topics:

- Image generator interface deep dive
- Crafting detailed image prompts
- Style modifiers: cartoon, realistic, abstract
- Artist styles (ethically)
- Lighting and mood descriptors
- Camera angles and perspectives
- Combining multiple concepts
- Inpainting and editing
- Variations and iterations
- Bing Image Creator advantages
- Batch generation techniques
- Saving and organizing AI art

Projects built:

- Create comic book pages
- Design game characters
- Reimagine famous paintings
- Build visual vocabulary guide

Practice: Generate the same scene in three styles, cartoon, realistic and abstract, and write down which prompt words worked best.

Midjourney and Advanced AI Art

Topics:

- Midjourney via Discord setup
- Understanding Midjourney commands
- /imagine prompt basics
- Parameters: --ar, --q, --v
- Midjourney styles and aesthetics
- Upscaling and variations
- Remix mode and blending

- Creating consistent characters
- Midjourney for different genres
- Advanced composition techniques
- Negative prompts usage
- Community and inspiration

Projects built:

- Create brand mascot variations
- Design imaginary world
- Architectural concepts
- Fashion design collection

Practice: Write one /imagine prompt that uses an aspect ratio parameter and one style word, then save your favorite result.

AI Photo Editing and Manipulation

Topics:

- AI-powered photo editing
- Background removal with AI
- Object removal and replacement
- AI upscaling and enhancement
- Face editing and avatars
- Style transfer techniques
- Creating AI filters
- Batch processing with AI
- Restoration of old photos
- AI color correction
- Creating photo compositions
- AI-assisted graphic design

Projects built:

- Restore and colorize old photos
- Create AI avatar collection
- Design poster with AI elements
- Before/after photo gallery

Practice: With permission, remove the background from a family photo using an AI editor and place the subject somewhere fun.

MONTH 3: AI MUSIC, AUDIO & VIDEO CREATION

Explore AI tools for music composition, audio editing, and video creation.

AI Music Generation

Topics:

- How AI composes music
- Music theory basics for AI
- Genres and styles in AI music
- Melody vs harmony generation
- Rhythm and beat creation
- AI instrument selection
- Mood and emotion in music
- Creating loops and samples
- Extending and editing AI music
- Copyright in AI music
- Using AI music in projects
- Music prompt engineering

Projects built:

- Create theme song for imaginary show
- Compose study music playlist
- Make birthday song variations
- Video game soundtrack creation

Practice: Generate two short tunes in different genres and tell someone at home which mood each one gives you.

AI Voice and Audio Tools

Topics:

- Text-to-speech technologies
- Voice cloning basics (ethical use)
- Creating character voices
- Podcast editing with AI
- Audio transcription tools
- Voice translation AI
- Sound effect generation
- Audio enhancement AI
- Removing background noise
- Voice modulation and effects
- Creating audiobooks
- AI for language learning

Projects built:

- Create character voices for story
- Produce mini podcast episode

- Make multilingual announcement
- Audio story with sound effects

Practice: Turn a favorite poem or joke into speech with a text to speech tool using two different character voices.

AI Video Generation

Topics:

- Text-to-video AI introduction
- Video editing with AI assistance
- Creating animations with AI
- AI scene generation
- Green screen with AI
- Video upscaling and enhancement
- Automatic subtitles and captions
- Video summarization AI
- Creating video thumbnails
- AI video effects and filters
- Lip-sync and dubbing AI
- Short-form content creation

Projects built:

- Create AI news report
- Animated explainer video
- Music video with AI visuals
- Educational video with AI presenter

Practice: Storyboard a 15 second video in three drawn frames, then create one of the scenes with an AI video tool.

AI Animation and Motion

Topics:

- AI-powered animation principles
- Motion capture with AI
- Creating animated characters
- Facial animation AI
- Text-to-animation tools
- GIF creation with AI
- Stop motion with AI assistance
- 3D animation basics with AI
- Animating static images
- Creating animated presentations
- AI for motion graphics
- Augmented reality filters

Projects built:

- Bring drawings to life
- Create animated avatar
- Motion comic creation
- AR filter design

Practice: Animate a simple character face showing happy, sad and surprised using an AI animation tool, and show your family the result.

MONTH 4: AI FOR LEARNING & PRODUCTIVITY

Use AI tools to enhance learning, boost productivity, and excel in academics.

AI Study Assistants

Topics:

- AI for different subjects
- Math problem solvers
- Science experiment simulators
- History research assistants
- Language learning with AI
- AI essay helpers (ethical use)
- Citation and bibliography tools
- Concept explanation AI
- Creating mind maps with AI
- AI flashcard systems
- Spaced repetition with AI
- Exam preparation strategies

Projects built:

- Create personalized study plan
- Build subject-specific AI tutor
- Design quiz preparation system
- Make visual study guides

Practice: Use an AI study helper to practice five maths or spelling questions, and mark which ones you solved without any help.

AI Research and Writing Tools

Topics:

- Research methodology with AI

- Finding reliable sources
- AI for literature review
- Summarizing long texts
- Note-taking with AI
- Outline generation
- Paraphrasing tools
- Grammar and style checking
- Plagiarism detection
- Creating bibliographies
- Presentation creation with AI
- Infographic generation

Projects built:

- Complete research project
- Create academic presentation
- Build knowledge database
- Design infographic report

Practice: Ask AI to summarize a short article about space or animals, then check one fact from the summary in another source.

AI Productivity Tools

Topics:

- Task management with AI
- Calendar optimization
- Email writing assistance
- Meeting notes and summaries
- Project planning AI
- Habit tracking with AI
- Goal setting assistants
- Time management tools
- Focus and concentration apps
- AI personal assistants
- Workflow automation basics
- Digital organization with AI

Projects built:

- Design weekly schedule optimizer
- Create productivity dashboard
- Build habit tracking system
- Automate routine tasks

Practice: Plan your perfect Saturday with an AI generated schedule, then fix two things it got wrong about you.

AI for Creative Projects

Topics:

- Brainstorming with AI
- Idea generation techniques
- Creative writing assistance
- Poetry and songwriting AI
- Storytelling with AI
- Game design with AI
- Recipe creation with AI
- Fashion design assistance
- Interior design AI tools
- DIY project planning
- Craft ideas generation
- AI for makers and builders

Projects built:

- Write and illustrate short story
- Design board game with AI
- Create recipe book
- Plan room makeover

Practice: Brainstorm ten silly story ideas with AI, pick your favorite and write the opening paragraph yourself.

MONTH 5: AI CODING, APPS & ADVANCED TOOLS

Learn AI-assisted coding, app development, and explore advanced AI platforms.

AI Coding Assistants

Topics:

- Introduction to AI pair programming
- How AI understands code
- Code completion basics
- Bug detection with AI
- Code explanation tools
- Converting ideas to code
- Learning programming with AI
- Different programming languages
- Debugging with AI help

- Code optimization suggestions
- Documentation generation
- Version control basics with AI

Projects built:

- Build first website with AI help
- Create simple Python game
- Make calculator app
- Design personal portfolio

Practice: Ask an AI assistant to explain a small piece of code in plain words, then change one thing and predict what happens.

No-Code AI App Builders

Topics:

- What is no-code development?
- Visual programming concepts
- Database basics for apps
- User interface design
- App logic and workflows
- Connecting AI to apps
- Mobile app creation
- Web app development
- Chatbot integration
- Forms and data collection
- Publishing and sharing apps
- App monetization basics

Projects built:

- Create quiz app
- Build chatbot assistant
- Design task tracker
- Make community platform

Practice: Sketch the screens of a simple app idea on paper and list the three buttons it would need.

Machine Learning Platforms

Topics:

- Training custom AI models
- Understanding datasets
- Classification vs regression
- Computer vision projects
- Natural language processing
- Predictive models basics
- Model evaluation metrics
- Overfitting and underfitting
- Feature engineering basics
- Deploying ML models
- ML ethics and bias
- Real-world ML applications

Projects built:

- Image classifier for recycling
- Sentiment analysis tool
- Gesture recognition system
- Predictive text generator

Practice: Train a Teachable Machine model to recognize two of your toys and test it five times with each toy.

AI Game Development

Topics:

- AI in game design
- Procedural generation
- NPC behavior with AI
- Dynamic difficulty adjustment
- AI for game testing
- Creating game assets with AI
- Story generation for games
- Level design with AI
- Balancing games with AI
- Player behavior analysis
- AI opponents creation
- Multiplayer matchmaking basics

Projects built:

- AI-powered adventure game
- Procedural dungeon generator
- Smart NPC system
- Dynamic story game

Practice: Design an NPC for your dream game: give it a name, one behavior rule and one line of dialogue written with AI help.

Create advanced projects, understand AI's future, and prepare for AI-driven careers.

AI for Social Good

Topics:

- Using AI to solve problems
- Environmental applications
- Healthcare AI basics
- Education technology
- Accessibility with AI
- AI for disaster response
- Fighting misinformation
- Language preservation
- Wildlife conservation AI
- Poverty alleviation tools
- Community helper bots
- Ethical AI development

Projects built:

- Design accessibility tool
- Create environmental monitor
- Build community helper bot
- Develop educational game

Practice: Pick one problem in your neighborhood and write three ways an AI helper could make it better.

AI Business and Entrepreneurship

Topics:

- AI startup ideas for young entrepreneurs
- Market research with AI
- Business plan generation
- Logo and branding with AI
- Marketing with AI tools
- Automating your own repetitive work
- Customer service bots
- Financial planning basics
- Pitch deck creation
- Finding target audience
- Competitive analysis with AI
- Scaling with AI

Projects built:

- Create startup concept
- Design brand identity
- Build marketing campaign
- Develop business pitch

Practice: Invent a small business idea, generate a logo for it with AI and write its one line slogan yourself.

AI Safety and Ethics

Topics:

- AI bias and fairness
- Privacy in AI age
- Deepfakes and detection
- Misinformation and AI
- Digital footprint awareness
- Cybersecurity basics
- AI regulation basics
- Intellectual property and AI
- Job displacement discussions
- Human-AI collaboration
- Future of work with AI
- Responsible AI use

Projects built:

- Create AI ethics guidebook
- Design bias detection system
- Build fact-checking tool
- Develop digital safety plan

Practice: Play detective with a parent: compare one real photo and one AI image and list two clues that gave the AI image away.

Future of AI and Emerging Tech

Topics:

- AGI (Artificial General Intelligence)
- Quantum computing basics
- Brain-computer interfaces
- Robotics and AI
- Autonomous vehicles
- Smart cities concept
- AI in space exploration
- Biotechnology and AI

- Virtual reality and AI
- Augmented reality future
- AI agents that use tools on your behalf
- Preparing for unknown futures

Projects built:

- Design future city with AI
- Create sci-fi story with AI
- Build future career plan
- Develop innovation proposal

Practice: Draw or describe a robot helper for the year 2040 and label three sensors it would need to do its job.

Capstone Project Development

Topics:

- Project ideation with AI
- Combining multiple AI tools
- Project management with AI
- Documentation best practices
- Testing and iteration
- User feedback collection
- Presentation preparation
- Demo video creation
- Project website building
- Social media promotion
- Peer review process
- Final polish and optimization

Practice: Choose your capstone idea, list the AI tools you will combine and write the build steps in order.

Deliverables:

- Complete project with documentation
- Presentation deck
- Demo video (3-5 minutes)
- Project website/portfolio
- Reflection essay on AI journey

Graduation and Future Planning

Topics:

- Final project presentations
- Peer choice awards
- Industry guest speakers
- AI career pathways
- Continued learning resources
- Building AI portfolio
- Networking in AI community
- Staying updated with AI
- Advanced courses available
- Mentorship opportunities
- Competition opportunities
- Celebration and certificates

Practice: Practice your final presentation out loud once and prepare a clear answer to 'which parts did the AI do and which parts did you do?'

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1 (Weeks 1-4): AI chatbot mastery, prompt engineering projects
- Month 2 (Weeks 5-8): AI art generation, image editing, creative portfolio
- Month 3 (Weeks 9-12): AI music, video, and animation creation projects
- Month 4 (Weeks 13-16): AI study tools, productivity apps, creative projects
- Month 5 (Weeks 17-20): AI coding assistants, no-code app building, ML platforms
- Month 6 (Weeks 21-26): AI for social good, capstone project, future skills

Total projects: 50+ AI-powered creative projects + Grand Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- AI Chat: ChatGPT and Perplexity - prompt engineering and conversation design
- AI Art: Midjourney, Bing Image Creator, Leonardo.ai, AI photo editing
- AI Music & Audio: AI music generation, voice tools, audio creation
- AI Video: AI video generation, animation, motion graphics
- AI Productivity: study assistants, research tools, writing helpers
- AI Coding: AI coding assistants, no-code app builders, ML platforms
- AI Ethics: safety, bias awareness, responsible AI use
- Prompt Engineering: writing effective prompts for any AI tool

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (1-1.5 hours each)

Hands on projects: 2-4 hours of creative AI projects

AI exploration: 1-2 hours experimenting with new AI tools

Total per week: 5-8 hours

SUPPORT INCLUDED

Live instruction: Expert-led live classes with recorded replays
Safe ai access: All AI tools used in supervised, age-appropriate settings
Parent updates: Weekly progress reports and tool usage guidelines
Community: Access to Modern Age Coders AI creators community
Ethics focus: Dedicated module on AI safety, ethics, and responsible use

CERTIFICATION DETAIL

Final certificate: Certified AI Creator for Youth (Modern Age Coders)
Capstone required: Multi-tool AI project combining art, text, audio, and video AI
Portfolio: 50+ AI-created projects in digital showcase portfolio

WHERE THIS CAN LEAD

- AI Content Creator
- Prompt Engineering Specialist
- Digital Artist with AI
- AI-Assisted Developer
- Creative Technologist
- AI Ethics Specialist
- AI Product Manager
- Machine Learning Enthusiast
- AI Educator/Tutor
- Innovation Consultant (with continued learning)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What age is appropriate for this AI tools course for teens?

A: This course is for ages 13 to 18. The age floor is not arbitrary: most AI tools set 13 as their own minimum in their terms of use, so a younger student cannot lawfully hold an account. Every tool in this course is one a 13+ student can actually sign up for. If your child is under 13, ask us about our AI courses built for that age group instead.

Q: Is it safe for my teenager to use AI tools?

A: Safety is built into the course, not bolted on. We teach it from day one: never share personal information, always verify what the AI tells you, understand what it cannot do, and know when not to use it at all. We also check each tool's own age requirement before we teach it, and we tell students plainly which tools they are not old enough for yet.

Q: Does my teenager need any prior experience to start this course?

A: No prior experience is needed! We start from the very basics of what AI is and how it works. The course is designed for complete beginners while also challenging those who already have some familiarity with AI tools.

Q: Will my teenager learn actual skills or just play with AI toys?

A: This is a comprehensive skill-building program! Students learn prompt engineering, critical thinking, creative problem-solving, digital literacy, and ethical reasoning. They work with professional AI tools used in real industries, from ChatGPT to Midjourney to coding assistants.

Q: How will this course help my teenager's future career?

A: AI literacy is becoming essential across all careers. This course prepares students for the AI-driven future with skills in prompt engineering, AI ethics, creative AI applications, and critical evaluation of AI outputs. These skills are already in high demand across industries.

Q: What equipment does my teenager need for this AI course?

A: A basic computer or laptop with internet connection is sufficient. Some tools work on tablets too. All software used is either free or has free educational versions. We guide parents on setting up safe accounts with appropriate controls.

Q: Can my teenager use every tool in this course?

A: Almost all of them, from 13. A few are marked (14+) or (16+) because the company that makes them sets that as its own minimum age in its terms of use, so a younger student cannot lawfully hold an account. Those tools are demonstrated in class and picked up later, rather than skipped. We check every tool's published age limit before we teach it, and we publish what we found, including the tools we rejected and why, in our AI tool age guide at /ai-tools-age-guide.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/teens-ai-tools-mastery-course>

=====

COURSE 11 OF 118: AI TOOLS MASTERY: CHATGPT TO PRODUCTION AI AUTOMATION

=====

Course link: <https://learn.modernagecoders.com/courses/ai-tools-mastery-complete-college>
Subject area: AI Tools & Automation
Designed for: College students
Who can join: Complete Beginner to AI Tools Expert
Course length: 6 months (26 weeks)
Time needed: 10-15 hours/week recommended
Certificate: AI Tools Professional certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 6-month AI Tools masterclass: master ChatGPT, Claude, Gemini, Midjourney, prompt engineering and no-code automation to build AI-powered workflows.

In one line: From Basic Prompts to Advanced AI Automation Workflows

ABOUT THE PROGRAMME

This is not just an AI tools course, it's a complete transformation into an AI-powered professional. Whether you're a content creator, marketer, developer, entrepreneur, or working professional, this 6-month program will teach you to leverage cutting-edge AI tools to 10x your productivity, automate workflows, create stunning content, and build AI-powered solutions without writing code. You'll master every major AI tool from ChatGPT to Midjourney, from prompt engineering to AI automation, from simple tasks to complex multi-agent systems. By the end, you'll have built 30+ AI-powered projects, automated countless workflows, and be recognized as an AI tools expert in your field.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no technical background needed
- Covers ALL major AI tools (ChatGPT, Claude, Gemini, Midjourney, etc.)
- Advanced prompt engineering masterclass
- No-code and low-code AI tool building
- AI automation and workflow creation
- Real business use cases and applications
- Monetization strategies with AI tools
- Regular updates with new AI tools
- Build actual AI products without coding
- 30+ hands-on projects and workflows

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-2): ChatGPT Mastery, Prompt Engineering, AI Content Creation
Phase 2: Visual AI (Months 3-4): Midjourney, Stable Diffusion, Image/Video AI Tools
Phase 3: Automation (Months 5-6): AI Workflows, Agents, Integration, Monetization

WHAT A STUDENT NEEDS BEFORE STARTING

Technical skills: None - completely beginner friendly
Equipment: Computer/laptop, internet connection
Ai accounts: Will create during course (ChatGPT, Midjourney, etc.)
Budget: Some tools require paid subscriptions (~\$50-100/month)
Time commitment: 10-15 hours/week
Learning style: Self-paced with guided projects

WHO THIS COURSE IS FOR

Content creators: Writers, bloggers, YouTubers, social media managers
Marketers: Digital marketers, growth hackers, brand managers
Entrepreneurs: Startup founders, solopreneurs, business owners
Freelancers: Looking to add AI services to offerings
Professionals: Anyone wanting to 10x productivity with AI
Students: Learning to leverage AI for career advantage
Designers: Graphic designers, UI/UX, creative professionals
Developers: Want to use AI tools for rapid development
Consultants: Business consultants, strategists, advisors
Educators: Teachers, trainers, course creators
Career switchers: Entering AI tools and automation field
Anyone: Curious about practical AI applications

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: AI FUNDAMENTALS & CHATGPT MASTERY (MONTHS 1-2, WEEKS 1-8)

Master ChatGPT, prompt engineering, AI writing tools, and conversational AI applications.

Months 1-2: ChatGPT & Prompt Engineering Excellence (Week 1-8)

Introduction to AI Tools & ChatGPT Basics

Topics:

- AI revolution: understanding generative AI
- Overview of AI tools landscape (2026)
- What is ChatGPT? How its models differ, and how to pick one
- ChatGPT account setup and interface
- ChatGPT Plus vs Free tier comparison
- Basic prompting: asking questions effectively
- Understanding AI responses and limitations
- Chat history management and organization
- Mobile app usage and tips
- ChatGPT plugins introduction
- Custom instructions setup
- Conversation starters and templates
- Ethics and responsible AI tool usage
- AI detection and originality
- Data privacy and security with AI tools

Projects built:

- Personal AI assistant setup
- Custom instruction templates
- Daily productivity workflow with ChatGPT
- Meeting notes summarizer
- Email response generator

Practice: Use ChatGPT daily for 20 different tasks

Advanced Prompt Engineering

Topics:

- What is prompt engineering?
- Anatomy of effective prompts
- Zero-shot, one-shot, few-shot prompting
- Chain-of-thought prompting
- Role-based prompting techniques
- Context setting and framing
- Prompt templates and frameworks
- RISEN framework: Role, Instructions, Steps, End goal, Narrowing
- RTF framework: Role, Task, Format
- Iterative prompt refinement
- Handling AI hallucinations
- Temperature and parameter understanding
- Multi-turn conversations strategies
- Prompt libraries and repositories
- A/B testing prompts for best results

Projects built:

- Prompt template library (50+ templates)
- Industry-specific prompt collections
- Complex task automation prompts
- Creative writing prompt system
- Business analysis prompts

Practice: Create and test 100+ prompts across categories

ChatGPT for Content Creation & Writing

Topics:

- Content ideation with ChatGPT
- Blog post writing workflow
- Article outlines and structure
- SEO-optimized content creation
- Social media content generation
- Email marketing campaigns
- Copywriting: sales pages, ads, landing pages
- Storytelling and creative writing
- Technical writing and documentation
- Resume and cover letter creation
- Editing and proofreading with AI
- Tone and style adjustments
- Multi-language content creation
- Content repurposing strategies
- Plagiarism checking and originality

Projects built:

- Complete blog writing system
- Social media content calendar (30 days)
- Email sequence (welcome, nurture, sales)
- E-book outline and chapters
- Product descriptions generator

- LinkedIn posts automation
- YouTube script templates

Practice: Create 50+ pieces of content in different formats
ChatGPT for Business & Productivity

Topics:

- Business strategy and planning
- Market research with ChatGPT
- Competitive analysis workflows
- SWOT analysis generation
- Business plan creation
- Financial projections and modeling prompts
- Data analysis and insights
- Code generation basics (no coding knowledge needed)
- Excel formulas and spreadsheet help
- Presentation content and structure
- Meeting preparation and agendas
- Decision-making frameworks
- Problem-solving with AI
- Learning and education use cases
- Personal development and coaching

Projects built:

- Business plan generator
- Market research report automation
- Competitor analysis system
- Excel/Sheets automation prompts
- Presentation creation workflow
- Personal learning curriculum
- Daily productivity dashboard

Practice: Apply ChatGPT to 30 business scenarios

----- PHASE 2: ADVANCED AI TOOLS & VISUAL AI (MONTHS 3-4, WEEKS 9-17) -----

Master Claude, Gemini, Midjourney, Stable Diffusion, and all major AI tools for images, video, and audio.

Months 3-4: Visual AI & Alternative LLMs (Week 9-17)

Claude AI & Google Gemini

Topics:

- Claude AI: features and strengths
- Claude vs ChatGPT comparison
- Claude's long context window usage
- Document analysis with Claude
- Claude for coding and technical tasks
- Claude Projects feature
- Google Gemini introduction
- Gemini Advanced features
- Gemini vs GPT-4 comparison
- Multi-modal capabilities of Gemini
- Google Workspace integration
- How Gemini replaced Bard, and what changed
- Choosing the right AI for the task
- Using multiple AIs together
- AI tool comparison matrix

Projects built:

- Document summarization system (Claude)
- Long-form content analysis
- Code review automation
- Research paper summarizer
- Multi-AI workflow (ChatGPT + Claude + Gemini)
- AI tool selector guide

Practice: Complete 20 tasks with each AI, compare results

Midjourney Mastery - Part 1

Topics:

- Introduction to AI image generation
- Midjourney setup: Discord integration
- Midjourney versions: V5, V6, Niji
- Basic prompting for images
- Understanding parameters: aspect ratio, quality, stylize
- Image weights and multi-prompts
- Style references and character consistency
- Remix mode and variations
- Upscaling and image enhancement
- Negative prompts for better control
- Seed for reproducibility
- Chaos and weirdness parameters
- Photography styles and techniques
- Artistic styles replication
- Brand and marketing visuals

Projects built:

- Product photography collection
- Brand visual identity
- Social media graphics library
- Character design series
- Marketing campaign visuals
- Book cover designs
- Website hero images

Practice: Generate 200+ images, build prompt library

Midjourney Mastery - Part 2 & Advanced Techniques

Topics:

- Advanced prompting strategies
- Multi-prompt weighting
- Style and mood control
- Lighting and composition
- Color palette specification
- Texture and material control
- Architecture and interior design
- Fashion and clothing design
- Logo and branding with Midjourney
- Combining with describe command
- Image-to-image transformations
- Consistent character creation
- Building visual narratives
- Commercial use guidelines
- Copyright and licensing

Projects built:

- Complete brand identity package
- Product line visualization
- Story illustration series
- Interior design portfolio
- Fashion collection mockups
- Consistent character across scenes
- Commercial project delivery

Practice: Professional-grade image generation portfolio

Stable Diffusion & Other Image AI Tools

Topics:

- Stable Diffusion introduction
- Local installation vs cloud platforms
- Automatic1111 WebUI
- Stable Diffusion prompting differences
- LoRA models and fine-tuning
- ControlNet for precise control
- Inpainting and outpainting
- DALL-E 3 features and usage
- Adobe Firefly for commercial use
- Leonardo.ai for game assets
- Playground AI features
- Image upscaling tools: Topaz, Magnific
- Background removal: Remove.bg, Clipdrop
- AI photo editing: Photoshop AI, Luminar
- Comparing all image AI tools

Projects built:

- Stable Diffusion custom model
- ControlNet workflow setup
- Image editing pipeline
- Multi-tool image creation workflow
- Background replacement system
- Product photography enhancement
- AI image editing service

Practice: Master 5+ image AI tools, create comparison guide

Video & Audio AI Tools

Topics:

- AI video generation: Runway ML, Pika
- Text-to-video creation
- Image-to-video animation
- Video editing with AI: Descript, Opus Clip
- AI avatars: Synthesia, HeyGen
- Video enhancement and upscaling
- AI voice generation: ElevenLabs, Murf
- Voice cloning ethics and usage
- Text-to-speech for videos
- Music generation: Suno, Udio
- Audio enhancement tools
- Podcast editing with AI
- Subtitle generation and translation
- Video summarization tools
- Social media video optimization

Projects built:

- AI-generated video ads
- Explainer video with AI avatar
- Podcast intro/outro creation
- Social media video series
- Product demo video
- Multi-language video content
- Complete video production workflow

Practice: Create 15 videos using various AI tools

Advanced AI Tools Ecosystem (Week 18-21)

AI Research & Knowledge Tools

Topics:

- Perplexity AI for research
- Research paper summaries: Semantic Scholar, Consensus
- Literature review automation
- You.com for AI-powered search
- Bing AI (Copilot) features
- AI fact-checking tools
- Citation and reference management
- Knowledge base creation with AI
- Note-taking with AI: Notion AI, Mem
- Obsidian with AI plugins
- Roam Research AI integration
- Personal knowledge management
- Learning and study tools
- AI tutors and educational platforms
- Research workflow optimization

Projects built:

- Research automation system
- Literature review generator
- AI-powered knowledge base
- Personal learning system
- Fact-checking workflow
- Study guide generator
- Research paper writing assistant

Practice: Build comprehensive research toolkit

AI Design & Creative Tools

Topics:

- Figma AI plugins
- Canva AI features: Magic Write, Magic Design
- AI logo generation: Looka, Brandmark
- Website builders with AI: Wix AI, Framer AI
- UI/UX design with AI: Uizard, Galileo AI
- Color palette generators
- Font pairing with AI
- Design system generation
- Mockup creation with AI
- 3D design tools: Spline AI
- AI design feedback tools
- Accessibility checking with AI
- Design-to-code tools
- AI art direction
- Creative brief generation

Projects built:

- Complete brand design system
- Website design with AI
- App UI/UX design
- Logo design service
- Marketing materials creation
- Design portfolio automation
- Client design process workflow

Practice: Design 20 complete projects with AI tools

AI Coding & Development Tools

Topics:

- GitHub Copilot for coding
- Cursor AI editor
- Replit AI for full-stack development
- V0.dev by Vercel for UI generation
- ChatGPT for code generation
- Code debugging with AI
- Code review automation
- Documentation generation
- No-code platforms: Bubble, Webflow
- AI-powered app builders: FlutterFlow
- Database schema generation
- API creation with AI
- Testing and QA with AI
- Code refactoring tools
- Building apps without coding knowledge

Projects built:

- Landing page with V0.dev
- Full web app with Replit AI
- Mobile app with FlutterFlow
- Chrome extension with AI
- Automation script generation
- API wrapper creation
- Complete SaaS MVP without coding

Practice: Build 10 applications using AI coding tools

Specialized AI Tools

Topics:

- AI for sales: Drift, Exceed.ai
- Customer support: Intercom AI, Zendesk AI
- AI SDRs and outreach: Regie.ai, Lavender
- Email AI: Superhuman, SaneBox
- Meeting AI: Otter.ai, Fireflies.ai, Grain
- Scheduling with AI: Reclaim.ai, Motion
- Project management: ClickUp AI, Asana AI
- HR and recruitment: HireVue, Paradox
- AI for finance: Vic.ai, AppZen
- Legal AI tools: Harvey, DoNotPay
- Healthcare AI tools overview
- Industry-specific AI applications
- Choosing tools for your industry
- ROI calculation for AI tools
- Building AI tool stack for business

Projects built:

- Sales automation workflow
- Customer support AI system
- Meeting automation setup
- Personal productivity stack
- Team collaboration AI setup
- Industry-specific AI toolkit
- AI tools recommendation engine

Practice: Configure AI tools for 5 different industries

Phase 2 Capstone Project

Topics:

- Comprehensive AI tools integration
- Multi-tool workflow design
- Use case selection
- Implementation and testing
- Documentation and training
- ROI measurement

Projects built:

- PHASE 2 CAPSTONE: End-to-End AI-Powered Business Process
- Option 1: Content Creation Agency (AI writing, images, video)
- Option 2: Marketing Automation System (AI tools for entire funnel)
- Option 3: AI-Powered Product Design Service
- Option 4: Research and Analysis Service with AI
- Requirements: Use 10+ different AI tools, documented workflow, measurable outcomes

Assessment: Phase 2 Exam - AI tools mastery across all categories

----- PHASE 3: AI AUTOMATION & MONETIZATION (MONTHS 5-6, WEEKS 22-26) -----

Master AI workflow automation, agents, integration, and monetization strategies.

Months 5-6: Advanced Automation & Business Applications (Week 22-26)

AI Workflow Automation

Topics:

- Introduction to AI automation
- Zapier with AI: ChatGPT integration
- Make (Integromat) for complex workflows
- n8n for advanced automation
- Connecting multiple AI tools
- Trigger-action automation
- Multi-step workflows
- Conditional logic in automation
- Error handling and fallbacks
- Data transformation between tools
- API connections basics
- Webhook usage for real-time automation
- Scheduled automation
- Database integration (Airtable, Google Sheets)
- Testing and debugging workflows

Projects built:

- Content publishing automation
- Lead generation and nurturing workflow
- Social media automation system

- Email response automation
- Data extraction and analysis pipeline
- Customer onboarding automation
- Report generation workflow

Practice: Build 20 automation workflows

AI Agents & Advanced Automation

Topics:

- What are AI agents?
- AutoGPT and autonomous agents
- BabyAGI for task automation
- LangChain for AI applications
- Agent frameworks overview
- Custom GPTs creation
- GPT Actions and integrations
- Assistants API usage
- Function calling in AI
- Agent memory and context
- Multi-agent systems
- Agent orchestration
- Agentic workflows design
- Limitations and considerations
- Future of AI agents

Projects built:

- Custom GPT for specific task
- Personal AI assistant (multi-functional)
- Research agent system
- Customer service agent
- Data analysis agent
- Content creation agent
- Business intelligence agent

Practice: Create 10 specialized AI agents

AI Integration & API Usage

Topics:

- Understanding APIs basics
- OpenAI API usage (ChatGPT, DALL-E)
- Anthropic Claude API
- Google Gemini API
- Stability AI API
- API key management
- Rate limits and cost optimization
- Building apps with AI APIs (no-code tools)
- Glide for AI-powered apps
- Softr for AI integrations
- API connectors in various platforms
- Custom chatbots with APIs
- Embedding AI in websites
- WhatsApp/Telegram bot creation
- AI-powered Chrome extensions

Projects built:

- Custom chatbot on website
- WhatsApp AI assistant
- Telegram bot with AI
- Chrome extension with ChatGPT
- Mobile app with AI features (Glide)
- Internal tool with AI (Softr)
- AI-powered form builder

Practice: Build 8 AI-integrated applications

Monetization & Business with AI Tools

Topics:

- Freelancing with AI tools
- AI services to offer clients
- Pricing AI-powered services
- Building AI tool SaaS products
- Affiliate marketing with AI content
- AI-generated digital products
- Selling prompts and templates
- AI consulting services
- Agency model with AI
- White-label AI solutions
- Course creation with AI
- YouTube/content creation monetization
- AI automation services
- Building and selling AI agents
- Subscription-based AI services

Projects built:

- Freelance service package design
- AI SaaS product MVP
- Prompt marketplace listings
- Digital product creation

- Agency service offerings
- Online course with AI help
- Monetization strategy document

Practice: Launch 3 revenue streams with AI tools

AI Tools for Specific Industries

Topics:

- E-commerce: product descriptions, ads, images
- Real estate: listings, virtual staging, market analysis
- Healthcare: documentation, patient communication (compliant)
- Legal: contract review, research, documentation
- Education: lesson planning, content creation, grading help
- Marketing agencies: campaigns, content, analytics
- Consulting: research, reports, presentations
- Software development: code, documentation, testing
- Finance: analysis, reporting, forecasting
- HR: recruitment, onboarding, training
- Customer success: support, documentation, training
- Creative agencies: design, copy, production
- Industry-specific compliance and ethics
- Case studies from each industry
- Building industry expertise

Projects built:

- Industry-specific AI toolkit (choose 3)
- Vertical SaaS concept
- Industry case study portfolio
- Client-ready workflows
- Industry pitch deck
- Specialization strategy

Practice: Master AI tools for 3 chosen industries

Final Month: Advanced Topics & Career (Week 22-26 (distributed))

Staying Current with AI Tools

Topics:

- AI tool discovery strategies
- Following AI news and updates
- There's An AI For That (directory)
- AI tool review and evaluation
- Beta testing new AI tools
- Community engagement: Twitter, Reddit, Discord
- AI newsletters and resources
- Testing frameworks for new tools
- Building personal AI tool stack
- Documentation and knowledge management
- Sharing findings and building authority
- Predicting AI tool trends
- Adapting to rapid changes
- Continuous learning strategies
- Building learning systems

Projects built:

- AI tool evaluation framework
- Personal AI tools dashboard
- Weekly AI tools newsletter
- AI tools comparison database
- Trend analysis report
- Learning and update system

Practice: Review 50+ new AI tools, create evaluation system

Ethics, Safety & Responsible AI Use

Topics:

- AI ethics principles
- Bias in AI tools
- Transparency and disclosure
- AI content labeling
- Misinformation and deepfakes
- Copyright and AI-generated content
- Privacy considerations
- Data security with AI tools
- Terms of service understanding
- Commercial use rights
- AI watermarking and detection
- Responsible automation
- Job displacement considerations
- Environmental impact of AI
- Building ethical AI practices

Projects built:

- Ethical AI usage policy
- Client education materials
- Disclosure templates
- Rights and licensing guide
- AI ethics checklist

- Best practices document

Practice: Develop comprehensive ethical framework

Teaching & Thought Leadership

Topics:

- Creating AI tools content
- YouTube channel for AI tools
- Blog and newsletter creation
- Social media strategy
- Building personal brand
- Community building
- Course creation with AI
- Workshop and training delivery
- Speaking at events
- Consulting and advisory
- Writing e-books and guides
- Creating templates and resources
- Membership communities
- Scaling education business
- Thought leadership positioning

Projects built:

- Content calendar (30 days)
- Mini-course creation
- Workshop material
- YouTube channel launch
- Newsletter setup
- Community platform
- Thought leadership plan

Practice: Create and publish 20 pieces of content

Building AI Tool Businesses

Topics:

- Identifying market opportunities
- Product-market fit with AI tools
- Building AI wrapper businesses
- No-code SaaS development
- AI tool aggregation platforms
- Marketplace creation
- Subscription model design
- Customer acquisition for AI services
- Retention and growth strategies
- Scaling AI tool businesses
- Team building and delegation
- Outsourcing with AI
- Financial modeling
- Funding and investment
- Exit strategies

Projects built:

- Business model canvas
- MVP development plan
- Go-to-market strategy
- Financial projections
- Pitch deck creation
- Launch plan
- Growth roadmap

Practice: Develop complete business plan

Final Capstone & Career Launch

Topics:

- Portfolio development
- Case study creation
- Service packaging
- Pricing strategies
- Client acquisition
- Proposal and contract templates
- Project management
- Client communication
- Delivery and results measurement
- Testimonials and social proof
- Scaling strategies
- Career planning

Projects built:

- FINAL CAPSTONE: Complete AI Tools Business/Service
- Option 1: AI Automation Agency (workflows for clients)
- Option 2: AI Content Creation Service
- Option 3: AI Tools SaaS Product
- Option 4: AI Consulting Practice
- Option 5: AI Education Business
- Requirements: Revenue-generating, documented processes, client-ready, scalable

Deliverables:

- Professional portfolio website
- Service offerings document

- Pricing and packages
- Client acquisition system
- Delivery workflows
- 10+ case studies
- Marketing materials
- Business launch plan

Assessment: FINAL EXAM - Comprehensive AI Tools Mastery

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 30+ ChatGPT projects - content, business, productivity
- Phase 2: 40+ projects - images, video, audio, multi-tool workflows
- Phase 3: 30+ projects - automation, agents, integrations, businesses
- Total: 100+ practical AI tool projects

Total projects: 100+ AI-powered projects and workflows

SKILLS THE STUDENT WILL HAVE AT THE END

- AI Tools: ChatGPT, Claude, Gemini, Midjourney, Stable Diffusion, DALL-E, and 50+ others
- Prompt Engineering: Advanced techniques, frameworks, optimization
- Content Creation: Writing, images, video, audio with AI
- Automation: Zapier, Make, n8n, workflow design
- AI Agents: Custom GPTs, assistants, autonomous agents
- Integration: APIs, webhooks, no-code platforms
- Business: Monetization, services, products, consulting
- Tools: All major AI platforms, productivity tools, specialized apps
- Skills: Research, design, coding (no-code), marketing with AI

HOW A TYPICAL WEEK IS SPLIT

Video tutorials: 3-4 hours
 Hands on practice: 5-7 hours
 Projects: 2-3 hours
 Exploration: 1-2 hours
 Total per week: 10-15 hours

SUPPORT INCLUDED

Live sessions: Weekly live demos and Q&A
 Community: Active community of AI tools practitioners
 Resources: Prompt libraries, templates, workflows
 Updates: Monthly updates with new AI tools
 Tool access: Shared accounts and trials for practice
 Mentorship: Expert guidance on projects
 Career support: Freelance and business guidance
 Lifetime access: Forever access to all content and updates

CERTIFICATION DETAIL

Completion certificate: AI Tools Professional Certificate
 Specialization: Choose specialization (Content/Automation/Business)
 Project portfolio: Verified portfolio of 30+ projects
 Digital badge: LinkedIn shareable badge
 Continuing education: Monthly master classes

WHERE THIS CAN LEAD

- AI Tools Consultant
- Prompt Engineer
- AI Automation Specialist
- AI Content Creator
- AI Tools Freelancer
- No-Code AI Developer
- AI Marketing Specialist
- AI Productivity Coach
- AI Tools Educator/Trainer
- AI Product Manager
- AI Solutions Architect (no-code)
- AI Agency Owner
- AI Tools Influencer/Creator

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What tools and skills does this AI tools course cover?

A: In 6 months you master ChatGPT and prompt engineering, then visual AI including Midjourney and Stable Diffusion alongside other leading models like Claude and Gemini, and finish with

AI automation: workflows, agents, integrations and monetization. The course surveys the AI tools landscape as it stands in 2026 and teaches transferable skills so new tools slot in easily.

Q: Who is this course for?

A: College students and early professionals who want AI leverage in their studies, work or side projects, with no technical background required. It also suits freelancers and founders who want to automate real workflows rather than just chat with AI.

Q: How is progress measured?

A: Through guided projects in each phase: a prompt library and content portfolio in phase 1, visual AI projects in phase 2, and working automation workflows in phase 3. You finish with artifacts you can show, not just tool familiarity.

Q: AI tools change fast. Will this course be outdated quickly?

A: Tools change; the underlying skills do not. Prompt engineering, evaluation habits and workflow design transfer directly to whatever ships next, and the curriculum is refreshed to stay current, which in 2026 includes the latest ChatGPT, Claude and Gemini generations.

Q: Can I try it first?

A: Yes, book a free demo class at learn.modernagecoders.com/contact, with no commitment and no payment details. We also answer questions on WhatsApp at +91 91233 66161.

Q: Is the course live or self-paced?

A: Live: one hour online classes with a real teacher, never pre-recorded lectures, in small groups where your actual use cases get worked on in class.

Q: Is there a certificate?

A: Yes, an AI Tools Professional certification on completion, plus a portfolio of prompts, visual projects and automation workflows built during the course. Modern Age Coders is rated 4.9 stars from 547+ Google reviews.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ai-tools-mastery-complete-college>

=====

COURSE 12 OF 118: ALGEBRA FOUNDATIONS: PRE-ALGEBRA & ALGEBRA 1, LIVE ONLINE

=====

Course link: <https://learn.modernagecoders.com/courses/algebra-foundations-masterclass>

Subject area: School Mathematics

Designed for: Middle school

Who can join: Grades 7-9, or any student whose algebra base needs a real rebuild

Course length: 12 months (52 weeks)

Time needed: 2 live one-hour classes per week + short practice sets

Certificate: Course completion certificate + phase mastery checks

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online Pre-Algebra and Algebra 1: repair fractions and negatives, then master linear equations, functions and quadratics. 8 one-hour classes a month.

In one line: Algebra is a language. We teach your child to speak it, not memorize it.

ABOUT THE PROGRAMME

A 12-month live program covering Pre-Algebra and the whole of Algebra 1, the single most consequential stretch in school mathematics. Every later course (Geometry, Algebra 2, SAT, calculus) silently assumes this material, and most struggling high-schoolers trace their trouble to exactly here. We teach algebra as a language with meaning: negatives and fractions rebuilt first, letters introduced as numbers-in-waiting, equations read as sentences, and functions met as machines. Students solve and explain aloud in every class; procedures are drilled only after the idea underneath holds.

WHAT MAKES THIS COURSE DIFFERENT

- The arithmetic base, fractions, negatives, order of operations, is repaired first, because algebra built on sand always collapses in year two
- Letters taught as meaning, not ritual: students can say what x is doing in every line they write
- Word problems as the main course, not the scary extras at the chapter's end
- Equation-table-graph fluency: every linear situation seen three ways
- The explain-back method: what survives explaining is what is actually learned
- One dedicated mentor who tracks exactly where understanding thins

- A genuine on-ramp to Algebra 2, SAT prep and beyond, this course is the runway
- One full interactive hour per class, twice a week

THE LEARNING PATH, STEP BY STEP

Phase 1: Pre-Algebra: the number system mastered, integers, fractions, decimals, ratios and the first variables

Phase 2: The heart of Algebra 1: equations, inequalities, linear functions and systems, seen as equation, table and graph

Phase 3: Exponents, polynomials and quadratics, plus the word-problem and graph craft that makes it all usable

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Typically grades 7-9; older students rebuilding a weak base are equally welcome

Math background: Basic whole-number arithmetic, everything else is taught or repaired inside the course

Equipment: Computer with stable internet, notebook and pencil

Time commitment: 2 live hours plus 1-2 practice hours weekly

English: Course taught in English

Motivation: Willingness to explain thinking aloud, the method depends on it

WHO THIS COURSE IS FOR

First timers: Grade 7-9 students meeting algebra for the first time who deserve to meet it properly

Strugglers: Students drowning in school algebra whose real gap is usually fractions or negatives

Rebuilders: Older students whose Algebra 2, SAT or IGCSE trouble traces back to this exact material

Homeschoolers: Families who want a structured, live-taught algebra year with a real teacher

Advancers: Strong grade 6-7 students ready to start the algebra road early

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: PRE-ALGEBRA, THE NUMBER SYSTEM MASTERED (WEEKS 1-16) -----

Almost every algebra struggle is an arithmetic gap wearing a disguise. This phase rebuilds integers, fractions, decimals and ratios to genuine fluency, then introduces variables so gently they feel obvious.

Weeks 1-4: Diagnostic + Integers and the Number Line

Topics:

- Full diagnostic: where the number base actually stands
- The number line as home: integers, opposites and absolute value
- Adding and subtracting negatives with meaning (temperature, money, elevation)
- Multiplying and dividing negatives, why the rules are what they are
- Order of operations as grammar, not a mnemonic chant
- Mental arithmetic habits started: every class opens with fluency work

Projects built:

- Personal gap map for the year ahead
- Integer war-stories set: temperature, bank balance and elevation problems

Practice: Do a short daily integer practice set this week, using the number line whenever unsure, and write one real money or temperature story for a negative number calculation.

Weeks 5-10: Fractions and Decimals, Properly This Time

Topics:

- Fractions as numbers with places on the line, not pizza slices
- Equivalence and comparison through reasoning, not rules
- All four operations with fractions, each from a picture first
- Decimals, place value and fraction-decimal-percent conversion fluency
- Estimation as a checking habit: does this answer make sense?
- Exponents introduced as repeated multiplication with meaning

Projects built:

- Fraction operation gallery: one worked picture per operation, made by the student
- Real-recipe scaling project: fractions doing real work

Practice: Complete a mixed fractions and decimals problem set, drawing a quick picture for any operation you cannot yet explain in words.

Weeks 11-16: Ratios, Percents and the First Variables

Topics:

- Ratio and rate reasoning: the choose-the-easy-comparison skill
- Proportions solved by sense first, cross-multiplication second
- Percent problems in every costume: change, discount, tax, reverse
- Variables introduced as numbers-in-waiting; expressions as phrases
- Evaluating and simplifying expressions; like terms with meaning

- The distributive property as area, a picture that never lies

Projects built:

- Best-buy investigation: unit-price reasoning on real products
- Phase mastery check: solve, explain and extend, per topic

Practice: Solve ten percent problems taken from real life, discounts, tax and score changes, estimating each answer before computing to check your number sense.

PHASE 2: EQUATIONS, FUNCTIONS AND SYSTEMS (WEEKS 17-36)

The heart of Algebra 1. Equations are read as sentences and solved with balance reasoning; linear functions are met as equation, table and graph telling one story; systems make two stories intersect. This is the phase that decides the next five years of math.

Weeks 17-22: Equations and Inequalities as Sentences

Topics:

- One-step and two-step equations from balance reasoning
- Multi-step equations: variables on both sides, distribution, fractions
- Translating words into equations, the skill that unlocks word problems
- Literal equations: isolating any variable in a formula
- Inequalities: what flips the sign and why
- Checking solutions as a habit, not an afterthought

Projects built:

- Equation-sentence dictionary: twenty phrasings, twenty equations
- Real-world setups: phone plans, savings goals, break-even problems

Practice: Work a set of multi step equations showing the balance move at every line, then translate three word problems into equations without solving them.

Weeks 23-29: Linear Functions Three Ways

Topics:

- Coordinate plane fluency and graph-reading discipline
- Slope as rate of change, from stories before formulas
- Slope-intercept, point-slope and standard form, translated freely
- Graphing lines by meaning; intercepts as starting values
- Function notation and the function-as-machine idea
- Linear modeling: real data fitted, interpreted and questioned

Projects built:

- Equation-table-graph triptych for three real situations
- Data modeling mini-project: fit a line, defend the interpretation

Practice: Graph the same line from slope intercept, point slope and standard form to confirm they agree, and describe one real situation the line could model.

Weeks 30-36: Systems of Equations and Inequalities

Topics:

- Systems by graphing: the intersection as the answer to a question
- Substitution and elimination, chosen by structure not habit
- How many solutions? Parallel, crossing and identical lines as ideas
- Systems word problems: mixtures, comparisons, break-even
- Linear inequalities and systems of them, with shaded-region reasoning
- Phase checkpoint: mixed problem set at grade-level exam standard

Projects built:

- Business break-even case solved all three ways
- Phase mastery check: solve, explain and extend, per topic

Practice: Solve six systems choosing the method by structure, at least two each by graphing, substitution and elimination, and check every answer in both equations.

PHASE 3: EXPONENTS, POLYNOMIALS AND QUADRATICS (WEEKS 37-52)

The bridge into Algebra 2: exponent rules from meaning, polynomial arithmetic without fear, factoring as detective work and quadratics met as functions with real stories. The course closes with the word-problem and graph craft that turns knowledge into marks.

Weeks 37-42: Exponents and Polynomials

Topics:

- Exponent rules derived, not memorized, including zero and negatives
- Scientific notation with real scale (atoms to galaxies)
- Polynomials as expressions with structure: naming, adding, subtracting
- Multiplying polynomials: the area model that makes FOIL obvious
- Special products recognized as patterns: squares and difference of squares
- Fluency drills at pace, now that meaning holds

Projects built:

- Powers-of-ten scale tour: notation doing real scientific work
- Area-model gallery for polynomial multiplication

Practice: Complete an exponent rules drill, deriving any rule you forget instead of guessing, plus five polynomial multiplications using the area model.

Weeks 43-48: Factoring and Quadratic Equations

Topics:

- Factoring as un-multiplying: GCF, trinomials, special patterns
- The zero-product property, why factoring solves equations
- Square roots and solving by square-rooting
- The quadratic formula, derived once so it is never mysterious
- Choosing a method by structure: factor, root, or formula
- The discriminant previewed: how many solutions and why

Projects built:

- Factoring pattern-book, built by the student
- Quadratic equation gauntlet: mixed methods under light timing

Practice: Factor a 15 expression mixed set, then solve five quadratic equations choosing factoring, square roots or the formula and noting why each time.

Weeks 49-52: Quadratic Functions + the Capstone

Topics:

- Quadratic functions and parabola anatomy: vertex, axis, intercepts
- Projectile and area stories: the vertex as the answer to a real question
- Graphing quadratics from every form
- Course capstone review: the year's ideas connected into one map
- Algebra 2 preview: what comes next and why this base carries it
- Final mastery check across all three phases

Projects built:

- Capstone: one real quadratic situation modeled, graphed and presented
- Personal algebra handbook, the year's ideas in the student's own words

Practice: Graph two parabolas from different forms, labeling vertex, axis and intercepts, and start your capstone map connecting the year's big ideas.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal gap map and year-long error log
- Fraction operation gallery and factoring pattern-book, made by the student
- Equation-table-graph triptychs for real situations
- Business break-even case solved three ways
- Capstone quadratic modeling project + personal algebra handbook

Total projects: 10+ student-made artifacts across the year

SKILLS THE STUDENT WILL HAVE AT THE END

- Integer, fraction and decimal fluency
- Ratio, proportion and percent reasoning
- Equation solving with balance reasoning
- Linear functions as equation, table and graph
- Systems of equations and inequalities
- Exponent rules and polynomial arithmetic
- Factoring and quadratic equations
- Word-problem translation craft

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor

Practice: Short, purposeful sets, never busywork stacks

Fluency: Mental-math warm-up in every class

Total per week: About 3-4 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week

Mentorship: One dedicated mentor across the whole year

Recordings: Every class recorded for revision

Progress tracking: Phase mastery checks shared with parents

Flexibility: Slots move around school exams and family plans

CERTIFICATION DETAIL

Phase certificates: Mastery check at the end of each phase

Final certificate: Course completion certificate from Modern Age Coders

Portfolio projects: The student's own algebra handbook and project gallery

WHERE THIS CAN LEAD

- Ready for Algebra 2, Geometry and the high-school mathematics track
- The base that SAT and ACT math prep assumes
- Eligible for competition mathematics and honors placements
- Continue with the High School Mathematics Mastery Program with the same mentor

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Who is this course for?

A: Three groups: students in grades 7-9 meeting algebra for the first time and wanting it taught properly; older students whose Algebra 2 or SAT struggles trace back to a weak Algebra 1 base; and homeschool families who want a structured, live-taught algebra year. The diagnostic in the free demo places every student accurately.

Q: My child is already failing algebra at school. Is this remedial or advanced?

A: It is neither and both, it is understanding-first. We start where your child actually is (the diagnostic finds it), repair the arithmetic underneath if needed, and build to full Algebra 1 ownership. Struggling students stop sinking; comfortable students go deeper than school does.

Q: Why does the course spend weeks on fractions and negatives?

A: Because they are the gap behind most algebra failures. A student who cannot operate on fractions confidently will lose points in every algebra topic forever. We fix the foundation first, it is slower in week one and dramatically faster by month six.

Q: How is this different from school algebra?

A: Class sizes and method. School must march thirty students through a syllabus; we teach 1-on-1 or in tight small groups, with your child solving and explaining aloud for a full hour. Every idea is built from meaning, balance reasoning, area models, function machines, so it survives past the test.

Q: Will this align with my child's school curriculum?

A: Yes. The course covers the standard Pre-Algebra and Algebra 1 territory (US Common Core, and the equivalent stretch in Indian, British and IB middle-school systems), and the mentor sequences topics to reinforce what school is doing rather than clash with it.

Q: How long until we see grade improvement?

A: Homework and test comfort usually move within 4-6 weeks; deep, durable improvement, the kind that persists into Algebra 2, builds over the year. Monthly billing means you watch evidence and re-decide every four weeks.

Q: Can we try it before paying?

A: Yes. Every student starts with a free live demo class that doubles as the diagnostic, a real lesson plus an honest read on where the gaps actually are, with no card details required.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/algebra-foundations-masterclass>

COURSE 13 OF 118: ALGORITHMIC TRADING & STOCK MARKET AUTOMATION MASTERCLASS

Course link: <https://learn.modernagecoders.com/courses/algorithmic-trading-automation-masterclass>

Subject area: Professional Financial Technology & Trading

Designed for: College students

Who can join: Complete Beginner to Professional Quant Trader

Course length: 12 months (52 weeks)

Time needed: 20-25 hours/week recommended

Certificate: Professional Algorithmic Trading Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year algorithmic trading program: master quantitative trading, backtesting, portfolio management, risk analytics and deploy production trading bots.

In one line: From Zero Knowledge to Professional Algorithmic Trader

ABOUT THE PROGRAMME

This comprehensive program transforms complete beginners into professional algorithmic traders capable of building, testing, and deploying sophisticated trading systems. You'll master financial markets, quantitative analysis, trading strategies, machine learning for finance, and risk management. Learn to analyze markets like a quant, build trading bots that execute strategies 24/7, backtest with historical data, manage risk professionally, and deploy production trading systems. By completion, you'll have built 25+ trading strategies, multiple live trading bots, and a complete trading platform.

WHAT MAKES THIS COURSE DIFFERENT

- Complete financial markets education from zero
- Real money paper trading with live market data

- Build actual trading bots and systems
- Machine learning and AI for trading
- Risk management and portfolio theory
- Covers stocks, forex, crypto, and derivatives
- Regulatory compliance and best practices
- Live deployment with real brokers APIs

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Markets, Python for Finance, Technical Analysis
 Phase 2: Trading Strategies (Months 4-6): Strategies, Backtesting, Portfolio Management
 Phase 3: Advanced Trading (Months 7-9): ML Trading, Risk Systems, Live Trading
 Phase 4: Professional Systems (Months 10-12): HFT, Crypto, Production Deployment

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal finance degree required
 Coding experience: Basic programming helpful but not required
 Equipment: Computer with internet, dual monitors recommended
 Capital: Rs 10,000 minimum for paper trading
 Time commitment: 20-25 hours per week
 English: Good comprehension required
 Motivation: Strong interest in markets and trading

WHO THIS COURSE IS FOR

Students: Engineering/Finance students seeking trading careers
 Working professionals: Career switchers to quantitative finance
 Traders: Discretionary traders wanting to automate
 Developers: Programmers interested in finance
 Entrepreneurs: Want to start trading firms or funds
 Investors: Want to manage own portfolios systematically
 Anyone: Passionate about markets and technology

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FINANCIAL MARKETS & PYTHON FOUNDATION (MONTHS 1-3, WEEKS 1-13) -----

Build solid understanding of financial markets, trading fundamentals, and Python programming for finance.

Month 1: Financial Markets Fundamentals (Week 1-4)

Introduction to Financial Markets

Topics:

- What is algorithmic trading? History and evolution
- Financial markets overview: stocks, bonds, commodities, forex
- Market participants: retail, institutional, market makers
- Exchanges and trading venues: NYSE, NASDAQ, NSE, BSE
- Order types: market, limit, stop-loss, stop-limit
- Bid-ask spread and market microstructure
- Trading hours and market sessions
- Settlement and clearing processes
- Regulatory environment: SEC, SEBI, regulations
- Market indices: S&P 500, Nifty, Sensex
- Economic indicators and their impact
- Introduction to fundamental analysis

Projects built:

- Market research report on major indices
- Trading account setup (paper trading)
- Market terminology glossary

Practice: Daily market observation and journaling

Trading Basics & Market Mechanics

Topics:

- Long vs short positions
- Margin trading and leverage
- Market volatility and VIX
- Trading psychology and behavioral finance
- Common trading mistakes and biases
- Position sizing basics
- Risk-reward ratios
- Trading costs: commissions, slippage, spreads
- Dividends and corporate actions
- Stock splits and bonus shares
- Market cycles and trends
- Introduction to derivatives: futures and options

Projects built:

- Paper trading diary with 10 trades
- Risk-reward calculator
- Trading cost analyzer

Practice: Execute 20 paper trades with analysis

Python Setup for Finance

Topics:

- Python environment setup for trading
- Anaconda and Jupyter for finance
- Essential libraries: NumPy, Pandas
- Financial data structures in Python
- Date and time handling for markets
- Working with financial data types
- Data cleaning for financial data
- Basic statistics with Python
- Visualization with Matplotlib
- Plotly for interactive charts
- Git for version control
- Project structure for trading systems

Projects built:

- Python trading environment setup
- Stock price data analyzer
- Market statistics calculator

Practice: Complete 30 Python finance exercises

Financial Data Acquisition

Topics:

- Yahoo Finance API with yfinance
- Alpha Vantage API integration
- NSE/BSE data sources
- Real-time vs historical data
- WebSocket connections for live data
- Data storage: CSV, databases, HDF5
- Time series data management
- Corporate actions data handling
- Economic data from FRED
- Alternative data sources
- Data quality and validation
- Building a data pipeline

Projects built:

- Multi-source data fetcher
- Historical data downloader
- Real-time price tracker
- Financial database creator

Practice: Collect data for 100 stocks

Month 2: Technical Analysis & Indicators (Week 5-8)

Classical Technical Analysis

Topics:

- Price action and candlestick patterns
- Support and resistance levels
- Trend lines and channels
- Chart patterns: head and shoulders, triangles, flags
- Moving averages: SMA, EMA, WMA
- Moving average crossovers
- Volume analysis and interpretation
- Market breadth indicators
- Dow Theory principles
- Elliott Wave basics
- Fibonacci retracements and extensions
- Japanese candlestick patterns

Projects built:

- Candlestick pattern detector
- Support/resistance calculator
- Moving average strategy implementation
- Chart pattern recognition system

Practice: Analyze 50 charts with technical analysis

Technical Indicators Programming

Topics:

- Momentum indicators: RSI, Stochastic
- MACD and signal lines
- Bollinger Bands implementation
- ATR (Average True Range)
- ADX (Average Directional Index)
- Volume indicators: OBV, VWAP
- Ichimoku Cloud components
- Custom indicator development
- Indicator optimization
- Multi-timeframe analysis
- Indicator backtesting
- TA-Lib library usage

Projects built:

- Technical indicator library
- Custom indicator creator
- Multi-indicator dashboard
- Indicator signal generator

Practice: Implement 20 technical indicators from scratch

Fundamental Analysis Automation

Topics:

- Financial statements analysis
- P/E, P/B, and valuation ratios
- Revenue and earnings analysis
- Cash flow analysis
- Debt and leverage metrics
- ROE, ROA, ROCE calculations
- Sector comparison metrics
- Earnings calendar tracking
- Insider trading data
- Analyst recommendations scraping
- News sentiment analysis
- Fundamental data APIs

Projects built:

- Fundamental analysis screener
- Earnings announcement tracker
- Financial ratio calculator
- Company comparison tool

Practice: Analyze 30 companies fundamentally

Market Analysis & Screening

Topics:

- Stock screening strategies
- Sector rotation analysis
- Market regime detection
- Correlation analysis
- Beta calculation and usage
- Relative strength analysis
- Market sentiment indicators
- Options flow analysis basics
- Unusual activity detection
- Pre-market and after-hours analysis
- Global market correlations
- Economic calendar integration

Projects built:

- Advanced stock screener
- Market regime identifier
- Correlation matrix builder
- Sentiment analysis system

Practice: Build 10 different screening strategies

Phase 1 Assessment

Topics:

- Review of market fundamentals
- Technical analysis mastery check
- Python for finance assessment
- Data handling proficiency
- Project presentations

Projects built:

- CAPSTONE: Complete Market Analysis Platform
- Features: data collection, technical indicators, fundamental analysis, screening, alerts
- Alternative: Multi-strategy signal generator
- Alternative: Market dashboard with live updates

Assessment: Phase 1 Exam - Markets and Technical Analysis

Month 3: Strategy Development & Backtesting (Week 9-13)

Trading Strategy Design

Topics:

- Types of trading strategies
- Trend following strategies
- Mean reversion strategies
- Momentum strategies
- Breakout strategies
- Pairs trading and arbitrage
- Market making basics
- Strategy hypothesis formation
- Entry and exit rules
- Position sizing methods
- Stop-loss and take-profit logic
- Strategy documentation

Projects built:

- Moving average crossover strategy
- RSI mean reversion strategy
- Breakout trading system

- Pairs trading strategy

Practice: Design 10 trading strategies

Backtesting Frameworks

Topics:

- Backtesting concepts and pitfalls
- Look-ahead bias and survivorship bias
- Vectorized backtesting with Pandas
- Event-driven backtesting
- Backtrader framework
- Zipline backtesting engine
- Custom backtesting framework
- Transaction cost modeling
- Slippage simulation
- Market impact modeling
- Walk-forward analysis
- Out-of-sample testing

Projects built:

- Custom backtesting engine
- Strategy comparison framework
- Walk-forward optimizer
- Monte Carlo simulator

Practice: Backtest 20 strategies

Performance Analytics

Topics:

- Returns calculation methods
- Sharpe ratio and risk-adjusted returns
- Maximum drawdown analysis
- Calmar and Sortino ratios
- Win rate and profit factor
- Value at Risk (VaR)
- Conditional VaR (CVaR)
- Performance attribution
- Benchmark comparison
- Rolling performance metrics
- Trade analysis and statistics
- Performance reporting

Projects built:

- Performance analytics dashboard
- Risk metrics calculator
- Trade journal analyzer
- Strategy report generator

Practice: Analyze 15 strategy performances

Strategy Optimization

Topics:

- Parameter optimization techniques
- Grid search optimization
- Random search methods
- Bayesian optimization
- Genetic algorithms for trading
- Machine learning for optimization
- Overfitting prevention
- Robust parameter selection
- Multi-objective optimization
- Strategy combination methods
- Ensemble strategies
- Adaptive strategies

Projects built:

- Parameter optimizer tool
- Genetic algorithm optimizer
- Strategy ensemble system
- Adaptive strategy framework

Practice: Optimize 10 strategies

Phase 2 Assessment

Topics:

- Strategy development review
- Backtesting best practices
- Performance evaluation
- Optimization techniques
- Project demonstrations

Projects built:

- MAJOR CAPSTONE: Professional Backtesting Platform
- Features: multiple strategies, optimization, performance analytics, reporting
- Alternative: Strategy marketplace simulator
- Alternative: Automated strategy discovery system

Assessment: Phase 2 Exam - Strategy Development and Backtesting

Master portfolio theory, risk management, advanced strategies, and quantitative methods.

Months 4-5: Portfolio Theory & Risk Management (Week 14-21)

Modern Portfolio Theory

Topics:

- Portfolio theory foundations
- Expected returns and variance
- Covariance and correlation in portfolios
- Efficient frontier construction
- Capital Asset Pricing Model (CAPM)
- Beta and systematic risk
- Alpha generation concepts
- Multi-factor models
- Fama-French factors
- Risk parity strategies
- Black-Litterman model
- Portfolio rebalancing strategies

Projects built:

- Efficient frontier calculator
- CAPM implementation
- Portfolio optimizer
- Risk parity portfolio builder

Practice: Build 10 optimal portfolios

Risk Management Systems

Topics:

- Types of financial risk
- Market risk measurement
- Credit risk basics
- Liquidity risk management
- Operational risk in trading
- Position sizing techniques
- Kelly criterion implementation
- Risk budgeting methods
- Stress testing portfolios
- Scenario analysis
- Risk limits and controls
- Real-time risk monitoring

Projects built:

- Risk management dashboard
- Position sizing calculator
- Risk limit monitor
- Stress testing framework

Practice: Implement 15 risk management techniques

Advanced Trading Strategies

Topics:

- Statistical arbitrage strategies
- Cointegration and pairs trading
- Options strategies: covered calls, spreads
- Delta hedging techniques
- Volatility trading strategies
- Calendar spread strategies
- Sector rotation strategies
- Factor-based strategies
- Smart beta strategies
- Alternative risk premia
- Carry trade strategies
- Global macro strategies

Projects built:

- Statistical arbitrage system
- Options strategy builder
- Volatility trading bot
- Factor rotation strategy

Practice: Implement 20 advanced strategies

Market Microstructure & Execution

Topics:

- Order book dynamics
- Market depth analysis
- Liquidity provision strategies
- Optimal execution algorithms
- TWAP and VWAP execution
- Implementation shortfall
- Dark pools and alternative trading
- High-frequency trading concepts
- Latency optimization basics
- Co-location advantages
- Smart order routing
- Transaction cost analysis

Projects built:

- Order book analyzer

- VWAP execution algorithm
- Market impact estimator
- Smart order router

Practice: Build 10 execution algorithms

Quantitative Research Methods

Topics:

- Research process in quant trading
- Hypothesis testing for strategies
- Statistical significance testing
- Time series analysis advanced
- ARIMA and GARCH models
- Regime switching models
- Factor analysis techniques
- Principal Component Analysis
- Research documentation
- Strategy paper writing
- Peer review process
- Research tools and notebooks

Projects built:

- Quant research framework
- Time series forecasting system
- Factor analysis tool
- Research documentation system

Practice: Complete 5 research projects

Month 6: Machine Learning for Trading (Week 22-26)

ML Fundamentals for Trading

Topics:

- Machine learning in finance overview
- Supervised learning for trading
- Feature engineering for markets
- Technical indicators as features
- Fundamental data features
- Alternative data features
- Train-test splitting for time series
- Cross-validation for trading
- Overfitting in financial ML
- Feature selection methods
- Data preprocessing for trading
- Label engineering for trading signals

Projects built:

- ML feature engineering pipeline
- Trading signal predictor
- Feature importance analyzer
- ML data preparation tool

Practice: Engineer 100 trading features

Classification & Regression for Trading

Topics:

- Logistic regression for signals
- Support Vector Machines for trading
- Random Forests for prediction
- Gradient boosting: XGBoost, LightGBM
- Neural networks for trading
- Ensemble methods for signals
- Price prediction models
- Direction prediction models
- Volatility forecasting
- Multi-class classification for signals
- Probability calibration
- Model interpretation for trading

Projects built:

- ML signal generator
- Price prediction system
- Ensemble trading model
- ML strategy builder

Practice: Train 20 ML trading models

Deep Learning for Markets

Topics:

- Deep learning in finance
- LSTM for time series prediction
- GRU networks for trading
- CNN for pattern recognition
- Attention mechanisms for trading
- Transformer models for markets
- Autoencoders for anomaly detection
- GANs for synthetic data
- Reinforcement learning basics
- Deep Q-Learning for trading
- Policy gradient methods

- Multi-agent trading systems

Projects built:

- LSTM price predictor
- Deep learning trading bot
- RL trading agent
- Transformer-based strategy

Practice: Build 10 deep learning models

Alternative Data & NLP

Topics:

- Alternative data sources for trading
- News sentiment analysis
- Social media sentiment: Twitter, Reddit
- Earnings call transcripts analysis
- SEC filings analysis
- Web scraping for trading signals
- Satellite data for trading
- NLP preprocessing for finance
- Named entity recognition for stocks
- Topic modeling for market themes
- Event detection from text
- Real-time sentiment pipeline

Projects built:

- News sentiment trading system
- Social media signal generator
- Earnings call analyzer
- Alternative data pipeline

Practice: Process 10 alternative data sources

ML Model Deployment

Topics:

- Model deployment strategies
- Real-time inference for trading
- Model versioning and management
- A/B testing trading models
- Model monitoring in production
- Drift detection for financial models
- Model retraining pipelines
- Feature stores for trading
- MLOps for trading systems
- Cloud deployment for ML models
- Edge deployment for low latency
- Model governance and compliance

Projects built:

- ML model deployment pipeline
- Model monitoring dashboard
- Automated retraining system
- Production ML trading system

Practice: Deploy 5 ML models to production

Months 7-9: Live Trading & Production Systems (Week 27-39)

Broker APIs & Connectivity

Topics:

- Interactive Brokers API
- Alpaca Trading API
- TD Ameritrade API
- Zerodha Kite Connect
- FIX protocol basics
- WebSocket connections for real-time data
- Order management systems
- Order routing logic
- API authentication and security
- Rate limiting handling
- Error handling and recovery
- Connection redundancy

Projects built:

- Multi-broker connector
- Order management system
- Real-time data aggregator
- API abstraction layer

Practice: Connect to 5 different brokers

Paper Trading Implementation

Topics:

- Paper trading environment setup
- Simulated order execution
- Paper trading engines
- Real-time paper trading
- Strategy deployment for paper trading
- Performance tracking
- Paper vs live trading differences
- Transitioning to live trading

- Risk controls for paper trading
- Paper trading best practices
- Multiple strategy management
- Portfolio paper trading

Projects built:

- Paper trading platform
- Strategy deployment system
- Performance tracker
- Risk monitor for paper trading

Practice: Paper trade 10 strategies for 30 days

Live Trading Systems

Topics:

- Live trading infrastructure
- Production system architecture
- Order execution engines
- Risk management systems
- Position tracking and P&L
- Real-time monitoring dashboards
- Alerting and notifications
- Fail-safe mechanisms
- Circuit breakers implementation
- Disaster recovery planning
- System redundancy and backup
- 24/7 operation considerations

Projects built:

- Live trading engine
- Risk management system
- Monitoring dashboard
- Alert system implementation

Practice: Build complete live trading infrastructure

Cryptocurrency Trading

Topics:

- Cryptocurrency markets overview
- Crypto exchanges: Binance, Coinbase
- Crypto trading APIs
- 24/7 trading considerations
- Crypto-specific strategies
- Arbitrage in crypto markets
- DeFi and automated market makers
- Stablecoin strategies
- Cross-exchange arbitrage
- Crypto portfolio management
- Tax implications for crypto
- Regulatory considerations

Projects built:

- Crypto trading bot
- Arbitrage detection system
- DeFi yield optimizer
- Multi-exchange trading platform

Practice: Trade 10 crypto strategies

Phase 3 Assessment

Topics:

- Live trading readiness
- System architecture review
- Risk management audit
- Performance evaluation
- Compliance check

Projects built:

- MAJOR CAPSTONE: Complete Automated Trading System
- Features: multiple strategies, live trading, risk management, monitoring, reporting
- Alternative: Crypto trading platform
- Alternative: Multi-asset trading system

Assessment: Phase 3 Exam - Production Trading Systems

----- PHASE 3: PROFESSIONAL TRADING & ADVANCED SYSTEMS (MONTHS 7-9, WEEKS 27-39) -----

Master institutional-grade trading systems, advanced strategies, and professional deployment.

Months 7-8: Institutional Trading Systems (Week 27-34)

Options Trading Automation

Topics:

- Options fundamentals review
- Greeks: Delta, Gamma, Theta, Vega, Rho
- Options pricing models: Black-Scholes
- Binomial option pricing
- Implied volatility calculation
- Volatility surfaces and skew
- Options market making

- Delta-neutral strategies
- Options arbitrage strategies
- Automated options screening
- Risk management for options
- Options backtesting challenges

Projects built:

- Options pricing calculator
- Greeks dashboard
- Options strategy backtester
- Volatility trading system

Practice: Implement 15 options strategies

Futures & Derivatives Trading

Topics:

- Futures markets overview
- Futures pricing and basis
- Calendar spreads in futures
- Commodity trading strategies
- Interest rate futures
- Currency futures trading
- Index futures strategies
- Futures arbitrage
- Roll strategies
- Margin requirements management
- Delivery and settlement
- Futures data handling

Projects built:

- Futures trading system
- Spread trading bot
- Commodity trading strategy
- Futures arbitrage detector

Practice: Build 10 futures strategies

High-Frequency Trading Concepts

Topics:

- HFT strategies overview
- Market making algorithms
- Statistical arbitrage HFT
- Latency arbitrage
- Order book imbalance strategies
- Tick data processing
- Ultra-low latency systems
- FPGA and hardware acceleration
- Co-location benefits
- Network optimization
- HFT risk management
- Regulatory considerations for HFT

Projects built:

- HFT simulator
- Market making bot
- Latency measurement tool
- Order book strategy

Practice: Simulate 5 HFT strategies

Fixed Income & Forex Trading

Topics:

- Bond markets and pricing
- Yield curve strategies
- Duration and convexity
- Credit spread trading
- Government bond strategies
- Corporate bond trading
- Forex market structure
- Currency pair strategies
- Carry trade implementation
- Forex arbitrage
- Central bank policy trading
- Cross-asset strategies

Projects built:

- Bond trading system
- Yield curve analyzer
- Forex trading bot
- Carry trade optimizer

Practice: Develop 10 fixed income/forex strategies

Quantitative Risk Analytics

Topics:

- Advanced risk metrics
- Expected Shortfall (ES)
- Stressed VaR
- Backtesting VaR models
- Extreme value theory
- Copula models for risk

- Credit risk modeling
- Counterparty risk
- Liquidity risk modeling
- Model risk management
- Risk aggregation methods
- Regulatory capital calculations

Projects built:

- Advanced risk system
- VaR backtesting framework
- Credit risk model
- Risk aggregation platform

Practice: Implement 10 risk models

Month 9: Advanced Infrastructure & Deployment (Week 35-39)

Trading Infrastructure Design

Topics:

- System architecture for trading
- Microservices for trading systems
- Message queuing: RabbitMQ, Kafka
- Event-driven architectures
- Database design for trading
- Time-series databases: InfluxDB
- In-memory databases: Redis
- Data lakes for trading
- Distributed systems design
- System scalability patterns
- High availability design
- Disaster recovery planning

Projects built:

- Microservices trading platform
- Event-driven trading system
- Scalable data architecture
- HA trading infrastructure

Practice: Design 5 trading architectures

Cloud Deployment for Trading

Topics:

- AWS for trading systems
- EC2 and auto-scaling
- S3 for data storage
- Lambda for serverless trading
- RDS and DynamoDB for trading data
- Google Cloud Platform for trading
- Azure for financial services
- Kubernetes for trading apps
- Docker containerization
- CI/CD for trading systems
- Infrastructure as Code
- Cloud cost optimization

Projects built:

- Cloud-native trading platform
- Serverless trading functions
- Kubernetes trading deployment
- Multi-cloud trading system

Practice: Deploy to 3 cloud providers

Monitoring & Observability

Topics:

- Monitoring trading systems
- Prometheus and Grafana
- ELK stack for logging
- Application performance monitoring
- Trading metrics and KPIs
- Real-time alerting systems
- Anomaly detection in monitoring
- Distributed tracing
- Performance profiling
- System health checks
- SLA monitoring
- Incident response procedures

Projects built:

- Monitoring dashboard
- Alert management system
- Performance analyzer
- Incident response toolkit

Practice: Monitor 10 trading systems

Compliance & Regulatory

Topics:

- Financial regulations overview
- MiFID II requirements
- Best execution obligations

- Market abuse regulations
- Trade reporting requirements
- Audit trail maintenance
- Data retention policies
- KYC and AML in trading
- Risk disclosure requirements
- Compliance monitoring systems
- Regulatory reporting automation
- License requirements

Projects built:

- Compliance monitoring system
- Audit trail generator
- Regulatory reporting tool
- Risk disclosure system

Practice: Implement compliance for 5 systems

Testing & Quality Assurance

Topics:

- Testing strategies for trading systems
- Unit testing trading logic
- Integration testing
- End-to-end testing
- Performance testing
- Stress testing systems
- Chaos engineering for trading
- Test data management
- Mock trading environments
- Continuous testing
- Test automation frameworks
- Quality metrics

Projects built:

- Testing framework for trading
- Stress testing suite
- Mock trading environment
- Quality assurance dashboard

Practice: Test 10 trading systems comprehensively

Months 10-12: Professional Development & Advanced Topics (Week 40-52)

Advanced Machine Learning Applications

Topics:

- Deep reinforcement learning for trading
- Multi-agent reinforcement learning
- Evolutionary algorithms for strategy discovery
- Genetic programming for trading rules
- Neural architecture search for trading
- Transfer learning in finance
- Few-shot learning for markets
- Online learning for adaptation
- Federated learning for trading
- Explainable AI for trading
- Causal inference in markets
- Quantum computing for finance basics

Projects built:

- Advanced RL trading agent
- Strategy evolution system
- Adaptive learning trader
- Explainable AI dashboard

Practice: Implement 10 advanced ML techniques

Market Making & Liquidity Provision

Topics:

- Market making strategies
- Inventory management
- Quote generation algorithms
- Spread optimization
- Adverse selection mitigation
- Order book modeling
- Queue position strategies
- Cross-market making
- Options market making
- Risk management for market makers
- Regulatory requirements for market makers
- Market maker profitability

Projects built:

- Market making simulator
- Quote generation engine
- Inventory manager
- Market maker risk system

Practice: Build 5 market making strategies

Fund Management Systems

Topics:

- Fund structure and operations
- Portfolio management systems
- NAV calculation automation
- Investor reporting automation
- Performance attribution systems
- Risk reporting for funds
- Compliance for fund managers
- Fund administration automation
- Investor portal development
- Subscription and redemption handling
- Fee calculation systems
- Fund marketing and analytics

Projects built:

- Fund management platform
- NAV calculator
- Investor reporting system
- Performance attribution tool

Practice: Build complete fund management system

Alternative Trading Venues

Topics:

- Dark pools and ATSS
- Decentralized exchanges (DEXs)
- Automated market makers (AMMs)
- Liquidity pools and yield farming
- Flash loans and arbitrage
- Cross-chain trading
- NFT trading automation
- Prediction markets trading
- Binary options automation
- Social trading platforms
- Copy trading systems
- Signal provider systems

Projects built:

- DEX trading bot
- AMM strategy implementation
- Cross-chain arbitrage bot
- Social trading platform

Practice: Trade on 5 alternative venues

Phase 4 Final Project - Part 1

Topics:

- Project planning and scoping
- Architecture design
- Technology selection
- Risk assessment
- Compliance planning
- Development roadmap
- Testing strategy
- Deployment planning

Projects built:

- FINAL PROJECT: Institutional-Grade Trading Platform
- Complete hedge fund trading system with multiple strategies, risk management, and compliance

PHASE 4: MASTER TRADER & CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52)

Complete mastery of algorithmic trading, final projects, and professional career preparation.

Months 10-11: Advanced Specializations (Week 40-47)

Sentiment-Driven Trading

Topics:

- Advanced sentiment analysis
- Multi-source sentiment aggregation
- Real-time sentiment processing
- Sentiment signal generation
- News impact prediction
- Social media trend detection
- Sentiment momentum strategies
- Event-driven sentiment trading
- Sentiment regime detection
- Cross-asset sentiment analysis
- Sentiment factor models
- Sentiment risk management

Projects built:

- Real-time sentiment engine
- News trading system
- Social sentiment trader
- Event detection platform

Practice: Build 10 sentiment strategies

Systematic Global Macro

Topics:

- Global macro strategy design
- Economic indicator trading
- Central bank policy trading
- Currency regime strategies
- Commodity super-cycles
- Emerging market strategies
- Cross-asset correlations
- Risk-on/risk-off detection
- Macro factor models
- Country rotation strategies
- Global event trading
- Macro hedging strategies

Projects built:

- Macro trading system
- Economic indicator trader
- Currency regime detector
- Global rotation strategy

Practice: Implement 8 macro strategies

Volatility Trading Systems

Topics:

- Volatility as an asset class
- VIX and volatility indices
- Volatility term structure
- Volatility arbitrage
- Dispersion trading
- Volatility risk premium
- Options volatility strategies
- Realized vs implied volatility
- Volatility forecasting models
- Cross-asset volatility
- Volatility regime switching
- Tail risk hedging

Projects built:

- Volatility trading platform
- VIX trading system
- Dispersion trader
- Tail hedge optimizer

Practice: Create 10 volatility strategies

Statistical Arbitrage Advanced

Topics:

- Advanced cointegration techniques
- Multi-asset statistical arbitrage
- Machine learning for stat arb
- High-frequency stat arb
- Index arbitrage strategies
- ETF arbitrage systems
- Cross-market arbitrage
- Merger arbitrage automation
- Convertible arbitrage
- Capital structure arbitrage
- Relative value strategies
- Risk arbitrage systems

Projects built:

- Multi-asset stat arb system
- ETF arbitrage platform
- Merger arb tracker
- Relative value trader

Practice: Build 12 arbitrage strategies

Blockchain & DeFi Trading

Topics:

- Smart contract trading
- Yield farming automation
- Liquidity mining strategies
- Flash loan arbitrage
- MEV (Maximum Extractable Value)
- Cross-chain arbitrage advanced
- NFT trading strategies
- DAO governance trading
- Staking optimization
- DeFi derivatives trading
- Layer 2 trading solutions
- Web3 trading infrastructure

Projects built:

- DeFi trading suite
- Yield optimizer bot
- MEV extraction bot
- Cross-chain trader

Practice: Deploy 10 DeFi strategies

Month 11: Business & Operations (Week 48-51)

Trading Business Setup

Topics:

- Trading business structures
- Legal entity formation
- Regulatory registrations
- Capital requirements
- Prime brokerage relationships
- Clearing and settlement
- Banking relationships
- Insurance for trading
- Accounting for traders
- Tax optimization strategies
- Audit preparation
- Business continuity planning

Projects built:

- Business plan creation
- Operational procedures manual
- Compliance framework
- Risk management policies

Practice: Create complete business structure

Raising Capital & Investor Relations

Topics:

- Fund raising strategies
- Pitch deck creation
- Track record presentation
- Due diligence preparation
- Investor reporting systems
- Performance presentation
- Risk disclosure documents
- Marketing materials
- Investor portal setup
- CRM for investors
- Regulatory filings
- Investor communication

Projects built:

- Investor pitch deck
- Performance tear sheet
- Due diligence package
- Investor portal

Practice: Prepare complete investor package

Team Building & Management

Topics:

- Building a trading team
- Roles in a trading firm
- Hiring quant developers
- Managing traders
- Performance evaluation
- Compensation structures
- Risk limits for traders
- Team collaboration tools
- Knowledge management
- Training programs
- Culture in trading firms
- Remote team management

Projects built:

- Team structure design
- Performance tracking system
- Knowledge base creation
- Training curriculum

Practice: Design team management framework

Research & Development

Topics:

- R&D in trading firms
- Research pipeline management
- Idea generation processes
- Research documentation
- Collaboration with academia
- Patent considerations
- Open source contributions
- Research publications
- Conference participation
- Industry partnerships
- Innovation frameworks
- Future technology adoption

Projects built:

- Research management system
- Innovation pipeline

- Publication strategy
- Partnership framework

Practice: Establish R&D processes

Performance Review & Optimization

Topics:

- Strategy performance review
- Portfolio optimization review
- Risk management audit
- Technology stack review
- Cost analysis and optimization
- Operational efficiency
- Compliance review
- Team performance review
- Business metrics analysis
- Competitive analysis
- Strategic planning
- Goal setting for next phase

Projects built:

- Performance audit report
- Optimization recommendations
- Strategic plan document
- KPI dashboard

Practice: Complete comprehensive review

Month 12: Final Projects & Launch (Week 49-52)

Final Project Development

Topics:

- Final project implementation
- System integration
- Performance optimization
- Security hardening
- Documentation completion
- Testing and validation
- User interface polish
- Deployment preparation

Projects built:

- FINAL PROJECT CONTINUED: Complete Trading Platform
- Full implementation with all components integrated and tested

Practice: Complete all project features

Final Project Deployment

Topics:

- Production deployment
- Live trading activation
- Monitoring setup
- Performance tracking
- Risk controls activation
- Compliance verification
- Documentation finalization
- Training materials creation

Projects built:

- Deploy to production
- Activate live trading
- Create user documentation
- Record demonstration videos

Practice: Launch live trading system

Portfolio & Career Preparation

Topics:

- Portfolio website creation
- Strategy showcase preparation
- Performance track record
- GitHub portfolio optimization
- LinkedIn profile enhancement
- Resume optimization
- Cover letter templates
- Interview preparation
- Networking strategies
- Industry connections
- Conference participation
- Thought leadership

Projects built:

- Professional portfolio site
- Strategy showcase
- Track record presentation
- Professional profiles

Practice: Build complete professional presence

Job Search & Opportunities

Topics:

- Quant trading roles
- Hedge fund positions

- Prop trading firms
- Investment banks
- Asset management firms
- Fintech companies
- Cryptocurrency firms
- Starting your own fund
- Freelance consulting
- Remote opportunities
- Salary negotiations
- Contract evaluation

Projects built:

- Job application package
- Interview preparation materials
- Salary research report
- Network building plan

Practice: Apply to 20 positions

Month 12: Graduation & Beyond (Week 51-52)

Final Assessments

Topics:

- Strategy presentation
- Live trading demonstration
- Risk management review
- Technical interview prep
- System design interview prep
- Behavioral interview prep
- Case study preparation
- Mock interviews

Projects built:

- Final project presentation
- Live trading demo
- Interview preparation portfolio
- Case study solutions

Industry Networking

Topics:

- Industry events and conferences
- Online communities
- Professional associations
- Mentorship programs
- Alumni network
- LinkedIn networking
- Twitter presence
- Discord/Slack communities
- Meetup groups
- Hackathons and competitions
- Open source contributions
- Industry publications

Deliverables:

- Network building plan
- Conference attendance schedule
- Community participation
- Mentorship connections
- Industry presence establishment

Continuous Learning & Future

Topics:

- Advanced certifications (CFA, FRM)
- Graduate programs consideration
- Specialized training paths
- Research opportunities
- Industry trends monitoring
- New technology adoption
- Strategy evolution
- Career progression planning
- Long-term goals setting
- Personal brand building
- Thought leadership development
- Future of algorithmic trading

Deliverables:

- Personal development plan
- 5-year career roadmap
- Continuous learning schedule
- Goal tracking system
- Success metrics definition

Assessment: FINAL COMPREHENSIVE EXAM - Complete Algorithmic Trading Mastery Assessment

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Market Analyzer, Technical Indicator Suite, Screening Platform, Data Pipeline
- Phase 2: Backtesting Engine, Strategy Optimizer, Portfolio Manager, Risk System

- Phase 3: ML Trading Bot, Live Trading Platform, Crypto Trader, Options System
- Phase 4: HFT Simulator, Fund Management Platform, DeFi Trading Suite
- Final: Complete Institutional Trading Platform with Everything

Total projects: 25+ trading systems ranging from simple to institutional-grade

SKILLS THE STUDENT WILL HAVE AT THE END

- Markets: Equities, Options, Futures, Forex, Crypto, Fixed Income
- Programming: Python (Expert), SQL, APIs, WebSockets, Cloud Platforms
- Analysis: Technical, Fundamental, Quantitative, Statistical
- Strategies: Momentum, Mean Reversion, Arbitrage, Market Making, ML-based
- Risk: VaR, Portfolio Theory, Position Sizing, Risk Management Systems
- ML/AI: Supervised Learning, Deep Learning, Reinforcement Learning, NLP
- Infrastructure: Real-time Systems, Databases, Cloud, Monitoring, Security
- Deployment: Docker, Kubernetes, AWS, GCP, CI/CD, Production Systems
- Business: Compliance, Fund Management, Investor Relations, Team Building

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 6-8 hours
 Hands on coding: 10-12 hours
 Projects: 4-6 hours
 Market analysis: 3-4 hours
 Total per week: 20-25 hours

SUPPORT INCLUDED

Live sessions: Weekly market analysis and Q&A
 Mentorship: 1-on-1 guidance from professional traders
 Community: Active Discord with traders and quants
 Code review: Strategy and code reviews
 Paper trading: Simulated trading environment
 Lifetime access: All content and future updates
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Final certificate: Professional Algorithmic Trader Certification
 LinkedIn badge: Verifiable LinkedIn credential
 Industry recognized: Recognized by trading firms
 Portfolio projects: 25+ deployable trading systems

WHERE THIS CAN LEAD

- Quantitative Trader
- Algorithmic Trading Developer
- Quantitative Analyst (Quant)
- Trading Systems Architect
- Risk Manager
- Portfolio Manager
- Market Maker
- Hedge Fund Analyst
- Proprietary Trader
- Trading Firm Founder
- Freelance Trading Consultant
- Fintech Developer

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 5-8 LPA (Junior Quant Analyst)
 Growing roles: Rs 10-15 LPA (Algo Trader); Rs 15-25 LPA (Quant Developer)
 Experienced roles: Rs 20-50+ LPA (Senior Quant Trader)
 Freelance: Rs 2,000-10,000/hour consulting
 International: \$80k-300k+ USD based on location

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What exactly will I learn to build?

A: Trading automation systems in Python: data pipelines for market data, technical analysis tooling, rule-based strategies, backtesting engines to test ideas on historical data, risk management logic, and paper trading deployments through broker APIs. By the end you can take

a strategy from idea to tested, automated implementation.

Q: Is this financial advice? Will I make guaranteed profits?

A: No, and be wary of anyone who says otherwise. This is an engineering course: it teaches you to build, backtest and automate trading systems, not what to buy. Markets carry real risk of loss, past backtest results do not guarantee future returns, and we make no profit claims of any kind.

Q: Do I need trading or finance knowledge before joining?

A: No, phase 1 covers financial markets from the ground up alongside Python for finance. Basic programming experience helps you move faster but is not required. What matters more is the discipline to treat trading as engineering and statistics rather than gambling.

Q: Which tools, data sources and brokers does the course use?

A: Python throughout, with standard data libraries and free sources like yfinance for learning, backtesting frameworks, and broker APIs such as Zerodha Kite for Indian markets and Alpaca internationally, plus an introduction to crypto exchange APIs like Binance. Live-money trading is never required for the course; strategies are validated through backtests and paper trading.

Q: Will this help my career in quant or fintech roles?

A: The skill set overlaps heavily with what fintech and quant-adjacent teams look for: Python, data analysis, backtesting rigor, APIs and automation. You finish with a GitHub-ready portfolio of trading systems, which is unusual and memorable in applications. We make no placement or income guarantees, in hiring or in trading.

Q: Can I try it before committing?

A: Yes, book a free live demo class at learn.modernagecoders.com/contact; there is no commitment and no payment details are taken. It is also the right place to ask hard questions about risk, tooling and whether the course fits your goals.

Q: How are classes delivered?

A: As live 1-hour online sessions with an instructor, never pre-recorded videos. Classes are interactive, you code the systems yourself with guidance, and evening and weekend slots suit college students and working professionals.

Q: Is there a certificate?

A: You receive a completion certificate along with a GitHub-ready portfolio of backtested strategies and automation projects. One honest note: in this field your documented backtests and clean code impress far more than any certificate, and that is what the course is designed to produce.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/algorithmic-trading-automation-masterclass>

=====

COURSE 14 OF 118: ALGORITHMIC TRADING MASTERCLASS: ZERO TO AUTOMATED SYSTEMS

=====

Course link: <https://learn.modernagecoders.com/courses/algorithmic-trading-masterclass-complete>

Subject area: Algorithmic Trading & Quantitative Finance

Designed for: College students

Who can join: Complete Beginner to Professional Quant Trader

Course length: 12 months (52 weeks)

Time needed: 20-25 hours/week recommended

Certificate: Certified Algorithmic Trader upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 12-month algorithmic trading program: master Python, financial markets, quant analysis, strategy design, backtesting, risk management and live trading.

In one line: From Manual Trading to Fully Automated Profitable Systems

ABOUT THE PROGRAMME

This is not just a trading course, it's a complete transformation into a quantitative trader and algorithmic trading professional. Whether you're a complete beginner, manual trader wanting to automate, developer entering finance, or finance professional wanting quantitative skills, this 12-month masterclass will turn you into a skilled algorithmic trader capable of designing, backtesting, optimizing, and deploying profitable automated trading systems. You'll master algorithmic trading from ground zero to professional level: from financial markets basics to advanced quantitative strategies, from Python programming to machine learning models, from paper trading to live automated execution, from single strategies to

portfolio management. By the end, you'll have built 40+ trading strategies, backtested extensively, and deployed live trading systems with proper risk management.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no trading or coding background needed
- Complete 12-month structured curriculum
- Python for trading taught from scratch
- Real market data and live trading experience
- Covers stocks, forex, crypto, commodities, options
- Backtesting on 10+ years of historical data
- Machine learning for trading strategies
- Risk management and portfolio optimization
- Live trading deployment with real money guidance
- Quantitative finance and mathematics
- High-frequency trading concepts
- Build 40+ trading strategies

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Markets, Python, Data Analysis, Technical Analysis

Phase 2: Strategy Development (Months 4-6): Trading Strategies, Backtesting, Optimization

Phase 3: Advanced Quant (Months 7-9): ML Trading, Advanced Strategies, Portfolio Management

Phase 4: Professional Trading (Months 10-12): Live Trading, HFT, Fund Management, Career

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required, basic math helpful

Trading knowledge: Beginner-friendly, no prior trading experience needed

Programming: No coding experience required (taught from scratch)

Mathematics: High school math (statistics taught in course)

Capital: Not required for learning (paper trading available)

Equipment: Computer, internet, ~Rs 5,000/month for data and tools

Time commitment: 20-25 hours/week consistently

Age: 18+ (due to broker account requirements)

Mindset: Analytical, patient, risk-aware, continuous learner

WHO THIS COURSE IS FOR

Manual traders: Want to automate their trading strategies

Developers: Software engineers entering finance

Finance professionals: Want quantitative and algo skills

Data scientists: Applying ML to trading

Students: Engineering, math, finance backgrounds

Career switchers: Entering quantitative finance

Entrepreneurs: Want to start trading business/fund

Quants: Enhance systematic trading skills

Investors: Want algorithmic portfolio management

Researchers: Academic interest in quantitative finance

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FINANCIAL MARKETS & TRADING FOUNDATIONS (MONTHS 1-3, WEEKS 1-13) -----

Build solid understanding of financial markets, instruments, Python programming, and data analysis for trading.

Months 1-2: Markets, Instruments & Python Basics (Week 1-8)

Introduction to Financial Markets

Topics:

- Financial markets overview: stocks, forex, crypto, commodities
- Stock market basics: exchanges, indices, sectors
- Understanding stocks: equity, shares, market cap
- How stock prices move: supply and demand
- Market participants: retail, institutional, market makers
- Trading sessions: pre-market, regular, after-hours
- Order types: market, limit, stop-loss, stop-limit
- Bid-ask spread and liquidity
- Long vs short positions
- Margin trading and leverage
- Trading costs: brokerage, taxes, slippage
- Forex market: currency pairs, pips, lots
- Cryptocurrency markets: Bitcoin, altcoins, exchanges
- Market microstructure basics
- Regulatory environment and compliance

Projects built:

- Market research on different instruments
- Paper trading practice (manual)
- Order type experiments
- Market analysis report
- Trading journal setup

Practice: Paper trade 20 manual trades, document everything

Python Programming for Trading - Part 1

Topics:

- Why Python for algorithmic trading?
- Python installation and setup (Anaconda)
- Jupyter notebooks for trading analysis
- Variables and data types
- Lists, tuples, dictionaries for market data
- Control flow: if-else for trading conditions
- Loops for iterating through historical data
- Functions for trading logic
- NumPy for numerical operations
- Arrays for price data manipulation
- Pandas for financial data analysis
- DataFrames for OHLCV data
- Reading CSV files (historical data)
- DateTime handling for trading timestamps
- Basic data cleaning

Projects built:

- Load and explore stock price data
- Calculate basic statistics (mean, std, returns)
- Price data manipulation exercises
- Simple moving average calculator
- Trading signal generator (basic)

Practice: Solve 50 Python problems focused on financial data

Financial Data Analysis with Python

Topics:

- Data sources: Yahoo Finance, Alpha Vantage, Quandl
- yfinance library for stock data
- pandas_datareader for multiple sources
- OHLCV data: Open, High, Low, Close, Volume
- Data cleaning and preprocessing
- Handling missing data in time series
- Resampling: converting timeframes
- Forward-fill and back-fill methods
- Calculating returns: simple and log returns
- Cumulative returns calculation
- Volatility calculation and analysis
- Correlation analysis between assets
- Data visualization with Matplotlib
- Candlestick charts with mplfinance
- Creating trading dashboards

Projects built:

- Multi-asset data downloader
- Returns and volatility analyzer
- Correlation matrix for portfolio
- Interactive price charts
- Market data dashboard
- Historical performance analyzer

Practice: Analyze 50+ stocks, create comprehensive reports

Technical Analysis Foundations

Topics:

- Technical analysis principles
- Support and resistance levels
- Trend analysis: uptrend, downtrend, sideways
- Trendlines and channels
- Chart patterns: head and shoulders, triangles, flags
- Candlestick patterns: doji, hammer, engulfing
- Moving averages: SMA, EMA, WMA
- Moving average crossovers
- Bollinger Bands: calculation and interpretation
- Relative Strength Index (RSI)
- MACD: Moving Average Convergence Divergence
- Stochastic Oscillator
- Volume analysis and indicators
- Average True Range (ATR) for volatility
- Implementing indicators in Python

Projects built:

- Technical indicator library in Python
- SMA/EMA crossover strategy
- RSI-based trading signals
- Bollinger Bands strategy
- Multi-indicator dashboard
- Pattern recognition tool (basic)

- Technical analysis automation
- Practice: Implement 20+ technical indicators from scratch

Month 3: Advanced Python & Market Analysis (Week 9-13)

Object-Oriented Programming for Trading Systems

Topics:

- OOP concepts for trading applications
- Creating trading strategy classes
- Portfolio class design
- Order management system (OMS) structure
- Risk management class
- Data handler class for market data
- Event-driven architecture for trading
- Inheritance for strategy variations
- Encapsulation for strategy logic
- Design patterns for trading systems
- Backtesting framework architecture
- Modular code organization
- Configuration management
- Logging for trading systems
- Error handling in live trading

Projects built:

- Trading strategy base class
- Portfolio manager class
- Order execution simulator
- Risk calculator class
- Complete OOP trading framework
- Modular backtesting engine

Practice: Refactor all previous code using OOP

Statistical Analysis for Trading

Topics:

- Descriptive statistics for returns
- Normal distribution and fat tails
- Hypothesis testing for strategies
- P-values and statistical significance
- Correlation and causation
- Cointegration for pairs trading
- Stationarity and unit root tests
- Autocorrelation in time series
- Monte Carlo simulations
- Bootstrap methods for trading
- Sharpe ratio calculation and interpretation
- Sortino ratio and downside deviation
- Maximum drawdown analysis
- Value at Risk (VaR)
- Risk-adjusted performance metrics

Projects built:

- Statistical analysis toolkit
- Returns distribution analyzer
- Strategy performance metrics calculator
- Monte Carlo simulator for trading
- Risk metrics dashboard
- Hypothesis testing framework

Practice: Statistical analysis of 30 trading strategies

Fundamental Analysis & Market Data APIs

Topics:

- Fundamental analysis basics
- Financial statements: P&L, balance sheet, cash flow
- Key ratios: P/E, P/B, ROE, debt-to-equity
- Earnings and revenue analysis
- Economic indicators impact on markets
- News sentiment analysis
- Alternative data sources
- Broker APIs: Interactive Brokers, Zerodha, Alpaca
- Real-time data streaming
- WebSocket connections for live data
- REST APIs for historical data
- Rate limiting and data management
- Building data pipelines
- Database setup for storing market data
- Time-series databases (InfluxDB, TimescaleDB)

Projects built:

- Fundamental data scraper
- Real-time data streamer
- Market data database setup
- Multi-source data aggregator
- News sentiment analyzer
- PHASE 1 MINI CAPSTONE: Complete Market Analysis Platform

Assessment: Phase 1 Exam - Markets, Python, Data Analysis, Technical Analysis

PHASE 2: TRADING STRATEGIES & BACKTESTING (MONTHS 4-6, WEEKS 14-26)

Master strategy development, backtesting frameworks, optimization, and strategy evaluation.

Months 4-5: Strategy Development & Backtesting (Week 14-22)

Backtesting Framework Development

Topics:

- What is backtesting? Importance and limitations
- Backtesting bias: look-ahead, survivorship, overfitting
- Event-driven backtesting architecture
- Vectorized vs event-driven backtesting
- Building custom backtesting engine
- Backtrader library introduction
- Backtrader strategies and indicators
- Position sizing in backtests
- Commission and slippage modeling
- Realistic execution simulation
- Walk-forward analysis
- Out-of-sample testing
- Multiple timeframe backtesting
- Parameter optimization in backtests
- Backtesting best practices

Projects built:

- Custom backtesting engine from scratch
- Backtrader strategy implementation
- Slippage and commission simulator
- Walk-forward testing framework
- Multi-timeframe backtester
- Optimization engine

Practice: Backtest 20 strategies with different approaches

Momentum & Trend Following Strategies

Topics:

- Momentum trading principles
- Relative Strength Momentum
- Moving average crossover strategies
- Dual moving average system
- Triple moving average system
- MACD crossover strategy
- Trend following concepts
- Donchian Channel breakout
- ATR-based trailing stops
- Turtle trading system
- Channel breakout strategies
- Parabolic SAR strategy
- Supertrend indicator strategy
- Combining multiple trend indicators
- Exit strategies for trend trades

Projects built:

- Moving average crossover bot
- MACD strategy implementation
- Turtle trading system
- Donchian breakout strategy
- Multi-indicator trend follower
- Momentum portfolio strategy
- Optimized trend following system

Practice: Develop and backtest 15 momentum/trend strategies

Mean Reversion & Statistical Arbitrage

Topics:

- Mean reversion theory
- Bollinger Bands mean reversion
- RSI oversold/overbought strategy
- Stochastic mean reversion
- Z-score based strategies
- Pairs trading fundamentals
- Cointegration testing for pairs
- Pairs selection methodology
- Spread calculation and trading
- Statistical arbitrage concepts
- Market neutral strategies
- Long-short equity strategies
- Ornstein-Uhlenbeck process
- Half-life of mean reversion
- Kalman filter for pairs trading

Projects built:

- Bollinger Bands reversion strategy
- RSI mean reversion bot
- Pairs trading system

- Cointegration scanner
- Statistical arbitrage strategy
- Market neutral portfolio
- Kalman filter pairs trader

Practice: Build 12 mean reversion strategies

Breakout & Volatility Strategies

Topics:

- Breakout trading concepts
- Support/resistance breakouts
- High/low breakout strategies
- Opening range breakout (ORB)
- Volatility breakout strategy
- ATR-based position sizing
- Volatility expansion trading
- Volatility contraction setups
- Squeeze indicator strategies
- News-based breakout trading
- Volume-confirmed breakouts
- False breakout filtering
- Consolidation pattern breakouts
- Multiple timeframe confirmation
- Breakout strategy optimization

Projects built:

- Support/resistance breakout bot
- Opening range breakout strategy
- Volatility breakout system
- Squeeze strategy implementation
- Volume breakout trader
- Multi-timeframe breakout system
- False breakout filter

Practice: Develop 10 breakout trading strategies

Options Trading Strategies (Basics)

Topics:

- Options basics: calls, puts, strikes, expiry
- Option Greeks: Delta, Gamma, Theta, Vega
- Option pricing: Black-Scholes model
- Implied volatility and IV rank
- Covered call strategy
- Cash-secured put strategy
- Bull call spread and bear put spread
- Iron Condor strategy
- Straddle and strangle
- Option selling strategies
- Delta-neutral strategies
- Volatility trading with options
- Options data analysis in Python
- Option chain analysis
- Backtesting options strategies

Projects built:

- Options calculator (Greeks, pricing)
- IV rank analyzer
- Covered call screener
- Iron Condor backtester
- Options strategy simulator
- Volatility trading system

Practice: Implement 8 options strategies

Month 6: Advanced Strategies & Optimization (Week 23-26)

Strategy Optimization & Parameter Tuning

Topics:

- Optimization objectives and pitfalls
- Overfitting vs robust optimization
- Grid search optimization
- Random search methods
- Genetic algorithms for optimization
- Particle swarm optimization
- Bayesian optimization
- Walk-forward optimization
- Sensitivity analysis
- Parameter stability testing
- Monte Carlo optimization
- Multi-objective optimization
- Sharpe ratio optimization
- Drawdown-constrained optimization
- Transaction cost considerations

Projects built:

- Grid search optimizer
- Genetic algorithm optimizer
- Walk-forward optimization framework

- Parameter sensitivity analyzer
- Multi-objective optimizer
- Robust optimization system
- Strategy parameter dashboard

Practice: Optimize 20 strategies, compare methods

Risk Management & Position Sizing

Topics:

- Risk management principles
- Position sizing methods: fixed, percentage, volatility-based
- Kelly Criterion for position sizing
- Risk per trade calculation
- Stop-loss placement strategies
- Trailing stop techniques
- ATR-based stops
- Time-based exits
- Profit target strategies
- Risk-reward ratio analysis
- Portfolio heat and risk limits
- Correlation-based position sizing
- Leverage management
- Drawdown control
- Circuit breakers and kill switches

Projects built:

- Position sizing calculator
- Kelly Criterion implementation
- Stop-loss optimizer
- Risk management system
- Portfolio heat monitor
- Drawdown controller
- Kill switch implementation

Practice: Add risk management to all strategies

Multi-Asset & Portfolio Strategies

Topics:

- Portfolio construction principles
- Modern Portfolio Theory (MPT)
- Efficient frontier calculation
- Mean-variance optimization
- Risk parity strategies
- Equal weight vs optimal weight
- Tactical asset allocation
- Strategic asset allocation
- Rebalancing strategies
- Correlation-based diversification
- Sector rotation strategies
- Multi-strategy portfolio
- Strategy allocation optimization
- Portfolio backtesting
- Portfolio performance attribution

Projects built:

- Portfolio optimizer (Markowitz)
- Risk parity portfolio
- Sector rotation system
- Multi-strategy portfolio manager
- Rebalancing algorithm
- Portfolio backtester
- Asset allocation dashboard

Practice: Build 10 portfolio-level strategies

Cryptocurrency Trading Strategies

Topics:

- Crypto market characteristics
- 24/7 trading considerations
- Crypto exchanges: Binance, Coinbase, Kraken
- Crypto data sources and APIs
- Bitcoin and altcoin strategies
- Crypto volatility strategies
- Arbitrage: exchange arbitrage, triangular arbitrage
- DeFi and DEX trading
- Crypto market making basics
- Sentiment analysis for crypto
- On-chain data analysis
- Crypto-specific indicators
- High volatility management
- Crypto tax considerations
- Security and wallet management

Projects built:

- Crypto data collector
- Bitcoin trend following bot
- Crypto arbitrage scanner
- Altcoin momentum strategy

- Crypto mean reversion system
- Exchange arbitrage bot
- Crypto portfolio rebalancer

Practice: Develop 12 crypto trading strategies

Phase 2 Capstone Project

Topics:

- Multi-strategy system design
- Complete backtesting pipeline
- Optimization and robustness testing
- Risk management integration
- Performance analysis
- Documentation and reporting

Projects built:

- PHASE 2 CAPSTONE: Complete Algorithmic Trading System
- Requirements: 5+ strategies, backtested on 5+ years data, optimized, risk-managed, documented
- Option 1: Multi-timeframe trend following system
- Option 2: Market-neutral pairs trading portfolio
- Option 3: Crypto trading bot with multiple strategies
- Option 4: Options selling systematic strategy

Assessment: Phase 2 Exam - Strategy development, backtesting, optimization

PHASE 3: MACHINE LEARNING & ADVANCED QUANT (MONTHS 7-9, WEEKS 27-39)

Master machine learning for trading, advanced quantitative strategies, and high-frequency concepts.

Months 7-8: Machine Learning for Trading (Week 27-35)

ML Fundamentals for Trading

Topics:

- Machine learning in trading overview
- Supervised vs unsupervised learning
- Feature engineering for trading
- Price-based features
- Technical indicator features
- Sentiment features
- Fundamental features
- Alternative data features
- Label creation: classification vs regression
- Triple-barrier method for labeling
- Train-test split for time series
- Cross-validation for trading models
- Overfitting in trading ML
- Model evaluation metrics for trading
- Scikit-learn for trading models

Projects built:

- Feature engineering pipeline
- Label creation system
- ML data preparation framework
- Cross-validation for time series
- Trading-specific feature library
- Model evaluation toolkit

Practice: Create 100+ features for trading

Classification Models for Trading

Topics:

- Price direction prediction
- Logistic regression for trading signals
- Decision trees for market regimes
- Random Forest for trading
- Feature importance analysis
- Gradient Boosting (XGBoost, LightGBM)
- Support Vector Machines for classification
- Ensemble methods for trading
- Model stacking and blending
- Probability calibration
- Threshold optimization for signals
- Handling imbalanced data
- SMOTE for trading datasets
- Model interpretation for trading
- Backtesting ML strategies

Projects built:

- Price direction classifier
- Random Forest trading strategy
- XGBoost market predictor
- Ensemble trading model
- Market regime classifier
- ML signal generator
- Complete ML trading strategy

Practice: Build 15 ML classification strategies

Deep Learning for Trading

Topics:

- Neural networks for trading
- LSTM for time series prediction
- GRU networks for trading
- Sequence-to-sequence models
- Attention mechanism for trading
- Transformer models for finance
- CNN for technical pattern recognition
- Autoencoders for feature extraction
- Reinforcement learning for trading (Q-learning)
- Deep Q-Networks (DQN) for trading
- Policy gradient methods
- TensorFlow and Keras for trading
- PyTorch for trading models
- GPU acceleration for training
- Model deployment considerations

Projects built:

- LSTM price predictor
- CNN pattern recognition strategy
- Autoencoder for anomaly detection
- Reinforcement learning trader
- DQN trading agent
- Transformer-based predictor
- Deep learning ensemble system

Practice: Implement 10 deep learning trading models

NLP & Sentiment Analysis for Trading

Topics:

- Natural language processing in trading
- News sentiment analysis
- Twitter sentiment for stocks/crypto
- Financial news scraping
- Text preprocessing for finance
- Sentiment scoring methods
- VADER for financial sentiment
- FinBERT for financial text
- Topic modeling for market themes
- Event extraction from news
- Earnings call analysis
- Social media signal extraction
- Alternative text data sources
- Real-time sentiment streaming
- Combining sentiment with price action

Projects built:

- News sentiment analyzer
- Twitter sentiment tracker
- Earnings call sentiment extractor
- Reddit sentiment for stocks/crypto
- Real-time sentiment dashboard
- Sentiment-based trading strategy
- Multi-source sentiment aggregator

Practice: Build 8 sentiment-based strategies

Alternative Data & Quantitative Research

Topics:

- Alternative data in trading
- Satellite imagery analysis
- Web scraping for trading signals
- Credit card transaction data
- Supply chain data
- Weather data for commodities
- Social media activity metrics
- App download data
- Google Trends for trading
- Data vendor integration
- Alpha generation from alt data
- Factor models and factor investing
- Fama-French factors
- Custom factor creation
- Quantitative research process

Projects built:

- Web scraping trading signals
- Google Trends strategy
- Alternative data aggregator
- Factor model implementation
- Multi-factor strategy
- Quantitative research report
- Alpha generation framework

Practice: Research and test 10 alternative data sources

Month 9: Advanced Trading Concepts (Week 36-39)

High-Frequency Trading (HFT) Concepts

Topics:

- High-frequency trading overview
- Market microstructure for HFT
- Order book dynamics
- Limit order book analysis
- Market making strategies
- Bid-ask spread capture
- Statistical arbitrage at high frequency
- Latency arbitrage concepts
- Co-location and infrastructure
- Low-latency programming
- FPGA and hardware acceleration
- Tick data processing
- Order flow analysis
- Liquidity provision strategies
- HFT regulations and compliance

Projects built:

- Order book visualizer
- Limit order book simulator
- Simple market making bot
- Order flow analyzer
- Tick data processor
- Microstructure research tools
- HFT backtesting framework

Practice: Analyze order book data, simulate HFT strategies

Advanced Options & Derivatives

Topics:

- Advanced option strategies
- Volatility trading deep dive
- VIX trading strategies
- Volatility arbitrage
- Skew trading strategies
- Calendar spread strategies
- Diagonal spreads
- Ratio spreads
- Butterfly and Condor variations
- Delta hedging automation
- Gamma scalping
- Options market making
- Exotic options basics
- Futures and futures spreads
- Commodity spread trading

Projects built:

- Volatility trading system
- VIX trading strategy
- Calendar spread optimizer
- Delta hedging bot
- Gamma scalping system
- Options portfolio manager
- Futures spread trader

Practice: Implement 12 advanced derivatives strategies

Execution Algorithms & Transaction Cost Analysis

Topics:

- Execution algorithms overview
- VWAP (Volume-Weighted Average Price) algorithm
- TWAP (Time-Weighted Average Price) algorithm
- Implementation shortfall
- Adaptive execution algorithms
- Iceberg orders and order slicing
- Smart order routing
- Transaction cost analysis (TCA)
- Slippage measurement and modeling
- Market impact models
- Broker execution quality analysis
- Fill rate optimization
- Execution latency measurement
- Pre-trade and post-trade analysis
- Best execution requirements

Projects built:

- VWAP execution algorithm
- TWAP execution bot
- Order slicing algorithm
- Transaction cost analyzer
- Slippage tracker
- Execution quality dashboard
- Smart order router (basic)

Practice: Implement and analyze 8 execution strategies

Quantitative Portfolio Management

Topics:

- Advanced portfolio construction
- Black-Litterman model
- Factor-based portfolio construction
- Multi-factor models
- Smart beta strategies
- Long-short portfolio construction
- 130/30 strategies
- Market-neutral portfolio management
- Portfolio rebalancing optimization
- Transaction cost in rebalancing
- Portfolio risk decomposition
- Tracking error minimization
- Active vs passive management
- Hedge fund strategies
- Performance attribution analysis

Projects built:

- Black-Litterman portfolio optimizer
- Multi-factor portfolio constructor
- Long-short portfolio manager
- Smart beta strategy
- Rebalancing optimizer with costs
- Portfolio analytics dashboard
- Performance attribution system

Practice: Build 10 portfolio management systems

Phase 3 Capstone Project

Topics:

- Advanced quantitative system design
- ML model integration
- Portfolio-level strategy
- Advanced risk management
- Performance analysis
- Research documentation

Projects built:

- PHASE 3 CAPSTONE: Machine Learning Trading Fund
- Requirements: ML models, multiple strategies, portfolio optimization, transaction costs, comprehensive backtesting
- Option 1: Multi-strategy ML-based hedge fund
- Option 2: Market-neutral quant fund with factor models
- Option 3: Crypto quant fund with ML and sentiment
- Option 4: Options trading fund with volatility strategies

Assessment: Phase 3 Exam - ML trading, advanced quant, portfolio management

PHASE 4: LIVE TRADING & PROFESSIONAL QUANT (MONTHS 10-12, WEEKS 40-52)

Master live trading deployment, infrastructure, fund management, and career development.

Months 10-11: Live Trading & Infrastructure (Week 40-48)

Live Trading Infrastructure

Topics:

- Broker selection and comparison
- Interactive Brokers API (Python)
- Zerodha Kite API (India)
- Alpaca API (US)
- Binance API (Crypto)
- Account setup and API credentials
- Paper trading vs live trading
- Order management system (OMS) development
- Position management system
- Real-time data streaming
- WebSocket connections for live data
- Order execution and status tracking
- Fill and partial fill handling
- Cancellation and modification logic
- Error handling and recovery

Projects built:

- Broker API integration
- Live trading infrastructure
- Order management system
- Real-time data handler
- Position tracker
- Paper trading framework
- Live trading simulator

Practice: Set up live trading for 5 strategies (paper trading)

Deployment & Production Systems

Topics:

- Cloud deployment: AWS, GCP, Azure

- Virtual Private Server (VPS) setup
- Linux for trading systems
- Docker for trading applications
- Continuous deployment for strategies
- Database for trade logging: PostgreSQL, MongoDB
- Time-series database: InfluxDB
- Monitoring and alerting
- Logging best practices
- Strategy health checks
- System uptime and reliability
- Backup and disaster recovery
- Security and API key management
- Network latency optimization
- Redundancy and failover systems

Projects built:

- Cloud trading server setup
- Dockerized trading system
- Database integration
- Monitoring dashboard
- Alerting system
- Backup automation
- Production deployment pipeline

Practice: Deploy 10 strategies to production environment

Live Trading Risk Management

Topics:

- Live trading risk considerations
- Real-time risk monitoring
- Position limits and concentration
- Daily loss limits
- Maximum drawdown controls
- Correlation monitoring
- Portfolio VaR in real-time
- Circuit breakers implementation
- Emergency shutdown procedures
- Broker risk management settings
- Margin management
- After-hours risk
- Gap risk management
- Black swan event preparation
- Risk reporting and dashboards

Projects built:

- Real-time risk monitor
- Circuit breaker system
- Position limit enforcer
- Drawdown controller
- Emergency shutdown system
- Risk dashboard (live)
- Risk reporting automation

Practice: Implement comprehensive risk systems

Performance Monitoring & Analysis

Topics:

- Real-time performance tracking
- P&L calculation and attribution
- Strategy-level performance
- Portfolio-level performance
- Benchmark comparison
- Risk-adjusted returns monitoring
- Sharpe ratio tracking
- Drawdown monitoring
- Win rate and profit factor
- Trade analysis and journaling
- Slippage and cost analysis
- Fill quality analysis
- Strategy degradation detection
- Automated performance reporting
- Dashboard creation with Streamlit/Dash

Projects built:

- Live performance tracker
- P&L attribution system
- Trade journal automation
- Performance dashboard
- Automated reporting system
- Strategy health monitor
- Degradation detector

Practice: Monitor live strategies, generate reports

Starting with Real Money

Topics:

- Transitioning from paper to live
- Starting capital considerations

- Scaling strategy capital
- Psychology of live trading
- Emotional discipline
- Handling losses in live trading
- Position sizing for live trading
- Gradual scale-up approach
- Monitoring and adjustment
- When to stop a strategy
- Portfolio heat management
- Tax implications of trading
- Record keeping for taxes
- Trading as a business
- Continuous improvement process

Projects built:

- Live trading checklist
- Capital allocation plan
- Scaling framework
- Trading journal (psychological)
- Tax tracking system
- Business plan for trading
- Improvement tracking system

Practice: Start live trading with small capital (optional)

Month 12: Professional Development & Career (Week 49-52)

Quantitative Finance Career Paths

Topics:

- Career options in quant finance
- Quantitative trader roles
- Quantitative researcher positions
- Quantitative developer roles
- Risk manager careers
- Portfolio manager positions
- Prop trading firms
- Hedge funds and asset managers
- Investment banks quant roles
- Starting a quant fund
- Required qualifications and skills
- Certifications: CQF, FRM, CFA
- Building a quant resume
- Portfolio of strategies as showcase
- Networking in quant finance

Projects built:

- Professional trading resume
- Strategy portfolio documentation
- Performance track record
- Research paper writing
- GitHub portfolio
- Trading blog/website

Practice: Build professional presence

Interview Preparation for Quant Roles

Topics:

- Quant interview process overview
- Probability and statistics questions
- Brain teasers and logic puzzles
- Coding interviews for quants
- Python algorithmic problems
- Trading strategy design questions
- Market making interview questions
- Risk management scenarios
- Portfolio optimization problems
- Behavioral interview preparation
- Salary negotiation for quants
- Top quant firms and their culture
- Mock interview practice
- Take-home assignments approach
- Presenting your strategies

Projects built:

- Interview preparation guide
- Probability problem solutions (100+)
- Coding challenge practice
- Strategy presentation deck
- Mock interview recordings
- Behavioral question prep

Practice: Solve 200+ quant interview problems

Running a Proprietary Trading Desk

Topics:

- Setting up a prop trading desk
- Capital requirements
- Technology infrastructure

- Team structure and hiring
- Strategy development process
- Risk management framework
- Compliance and regulations
- Broker relationships
- Cost structure and profitability
- Scaling trading operations
- Multiple strategy management
- Team collaboration tools
- Performance incentives
- Continual research and development
- Exit strategies and selling

Projects built:

- Prop desk business plan
- Infrastructure planning
- Cost-benefit analysis
- Hiring and training plan
- Compliance checklist
- Operations manual

Practice: Plan and document prop desk setup

Launching a Quant Hedge Fund

Topics:

- Hedge fund structure and setup
- Legal and regulatory requirements
- Fund formation: LLC, LP structures
- Registration with regulators (SEBI, SEC)
- Investor pitch deck creation
- Due diligence preparation
- Fund administration
- Prime brokerage relationships
- Fundraising strategies
- AUM targets and scaling
- Fee structures: management and performance fees
- Investor reporting requirements
- Marketing and branding
- Building track record
- Incubator and accelerator programs

Projects built:

- Fund formation plan
- Investor pitch deck
- Due diligence package
- Fund policies and procedures
- Marketing materials
- Financial projections
- Regulatory compliance checklist

Practice: Complete hedge fund launch plan

Continuous Learning & Community

Topics:

- Staying current with quant research
- Academic papers and journals
- Quantitative finance books
- Online communities: QuantConnect, Quantopian archives
- Attending quant conferences
- Networking with other quants
- Contributing to open source quant projects
- Publishing research and strategies
- Teaching and mentoring
- Building personal brand in quant finance
- Consulting opportunities
- Side projects and experimentation
- Emerging technologies: quantum computing, blockchain
- Long-term career development
- Work-life balance in trading

Projects built:

- Continuous learning plan
- Research paper reading list
- Community engagement strategy
- Open source contributions
- Personal research publication
- Mentorship program participation

Practice: Develop lifelong learning system

FINAL PROJECT & CAREER LAUNCH (Week 49-52 (distributed))

Research & Strategy Innovation

Topics:

- Novel strategy research
- Combining multiple approaches
- Regime-adaptive systems
- Machine learning innovations

- Alternative data exploitation
- Cross-asset strategies
- International markets
- Emerging markets trading
- Cryptocurrency innovations
- DeFi trading opportunities

Projects built:

- Original research project
- Novel strategy development
- Multi-asset system
- Innovation documentation
- Research paper writing

Practice: Develop unique trading edge

Advanced Technology Integration

Topics:

- Cloud-native trading systems
- Serverless architecture for trading
- Kubernetes for trading infrastructure
- Real-time data lakes
- Big data for trading
- GPU acceleration for backtesting
- Quantum computing for optimization
- Blockchain for settlement
- Advanced monitoring and observability
- Site reliability engineering for trading

Projects built:

- Cloud-native trading platform
- Scalable infrastructure
- Big data pipeline
- Advanced monitoring system
- Technology roadmap

Practice: Modernize trading infrastructure

Regulatory & Compliance Mastery

Topics:

- Global trading regulations
- SEBI regulations (India)
- SEC regulations (USA)
- MiFID II (Europe)
- Algorithmic trading regulations
- Market manipulation rules
- Insider trading compliance
- Best execution requirements
- Audit trails and record keeping
- Compliance monitoring systems

Projects built:

- Compliance framework
- Audit trail system
- Regulatory reporting automation
- Compliance monitoring dashboard
- Documentation system

Practice: Ensure full regulatory compliance

Final Capstone & Launch

Topics:

- Complete trading business setup
- Live trading deployment
- Performance tracking
- Client/investor reporting
- Continuous optimization
- Career positioning

Projects built:

- FINAL CAPSTONE: Professional Algorithmic Trading Business
- Option 1: Multi-strategy quant fund (simulated AUM)
- Option 2: Proprietary trading desk
- Option 3: Algo trading as a service
- Option 4: Personal trading business
- Requirements: Live deployed, monitored, profitable (or promising), documented, scalable

Deliverables:

- Live trading system (paper or real)
- Complete infrastructure
- 10+ live strategies
- Performance track record
- Risk management system
- Monitoring and reporting
- Business documentation
- Professional website/presence
- Investor-ready materials
- Career launch plan

Assessment: FINAL COMPREHENSIVE EXAM - Complete algorithmic trading mastery

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 25+ foundational projects - data analysis, indicators, technical analysis
- Phase 2: 30+ strategy projects - backtesting, optimization, various strategies
- Phase 3: 25+ advanced projects - ML models, HFT, portfolio management
- Phase 4: 20+ professional projects - live trading, infrastructure, business
- Total: 100+ trading projects from basics to professional systems

Total projects: 100+ trading strategies and systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Programming: Python (expert), NumPy, Pandas, Matplotlib, OOP for trading
- Markets: Stocks, Forex, Crypto, Options, Futures, Commodities
- Technical Analysis: 50+ indicators, chart patterns, multi-timeframe analysis
- Strategy Development: Momentum, mean reversion, breakout, arbitrage, options
- Backtesting: Backtrader, custom frameworks, walk-forward, optimization
- Machine Learning: Scikit-learn, TensorFlow, PyTorch, feature engineering, NLP
- Quantitative Finance: Statistics, portfolio theory, risk management, derivatives
- Live Trading: Broker APIs, execution algorithms, real-time systems
- Infrastructure: Cloud deployment, databases, monitoring, CI/CD
- Risk Management: Position sizing, stop-loss, portfolio heat, VaR
- Tools: Jupyter, Git, Docker, PostgreSQL, InfluxDB, Prometheus
- Business: Fund management, compliance, investor relations, career development

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-7 hours
Coding practice: 8-10 hours
Backtesting: 4-6 hours
Research reading: 2-3 hours
Live trading practice: 2-3 hours
Total per week: 20-25 hours

SUPPORT INCLUDED

Live sessions: Weekly live trading analysis and strategy sessions
Mentorship: 1-on-1 guidance from professional quant traders
Community: Active community of algorithmic traders
Code review: Expert review of trading strategies and code
Data access: Historical and real-time data for practice
Cloud credits: Credits for cloud deployment
Broker support: Help with broker setup and API integration
Lifetime access: All content, strategies, updates forever
Strategy library: Access to 100+ tested strategies
Research papers: Curated quantitative finance research

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 total)
Final certificate: Certified Algorithmic Trader
Specializations: Options in Crypto/Options/HFT/ML Trading
Track record: Verified backtesting and live trading results
Portfolio: 40+ documented strategies
Digital badge: LinkedIn shareable credentials
Industry recognized: Recognized by prop firms and hedge funds

WHERE THIS CAN LEAD

- Quantitative Trader
- Algorithmic Trader
- Quantitative Researcher
- Quantitative Developer
- Portfolio Manager (Quantitative)
- Risk Manager (Quantitative)
- Proprietary Trader
- Hedge Fund Manager
- Systematic Trading Strategist
- High-Frequency Trader
- Independent Algo Trader
- Trading System Developer
- Quant Consultant
- Financial Engineer
- Research Scientist (Finance)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What are the prerequisites for this algorithmic trading course?

A: No prior trading or programming experience is required. This course starts from absolute zero and teaches you Python programming, financial markets, and quantitative analysis from the ground up. Basic computer literacy and enthusiasm to learn are the only requirements.

Q: What tools and technologies will I learn in this algorithmic trading masterclass?

A: The course covers the complete tech stack used by professional quant traders.

Q: Can I actually make money with algorithmic trading after this course?

A: This course provides you with all the skills, strategies (40+ backtested strategies), and risk management knowledge to trade profitably. However, trading involves risk and past performance doesn't guarantee future results. We teach proper position sizing, drawdown management, and realistic expectations alongside profitable strategy development.

Q: How is this different from other trading courses?

A: Unlike courses that only teach theory, this 12-month masterclass gives you hands-on experience with real market data, 10+ years of backtesting, live paper trading, and guidance for real money deployment. We cover stocks, forex, crypto, commodities, and options trading automation.

Q: What career opportunities are available after completing this course?

A: The course includes resume building and interview prep for quant roles.

Q: Do I get any certification after completing this algorithmic trading course?

A: Yes, upon successful completion you receive the 'Certified Algorithmic Trader' certificate. Additionally, you'll have a portfolio of 40+ trading strategies, a complete code repository, and documented backtesting results that demonstrate your skills to employers.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/algorithmic-trading-masterclass-complete>

=====

COURSE 15 OF 118: ALPHA GIRLS: ELITE TECH MASTERY & LEADERSHIP FOR WOMEN

=====

Course link: <https://learn.modernagecoders.com/courses/alpha-girls-elite-tech-mastery>

Subject area: Ultra-Elite Women in Technology - Enterprise Mastery Program

Designed for: Girls only batches

Who can join: Expert Developer to Industry Pioneer/Tech Mogul

Course length: 12 months (52 weeks)

Time needed: 25-30 hours/week intensive

Certificate: Alpha Girls Elite Certification - Fortune 500 Recognized

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year program for women in tech ready to go from advanced developer to industry leader: master enterprise architecture and lead large-scale systems.

In one line: From Expert to Empire - Build Billion-Dollar Systems and Lead Global Tech Revolution

ABOUT THE PROGRAMME

Alpha Girls is the pinnacle of tech education for women. This ultra-intensive program is designed for experienced developers ready to operate at the highest levels of technology. You'll architect systems handling billions of users, lead \$100M+ projects, build unicorn startups, and command top 1% compensation. Work directly on Fortune 500 projects, get mentored by female billionaire tech founders, and join an exclusive network of the most powerful women in technology. This is where tech empires are born and industry leaders are forged.

WHAT MAKES THIS COURSE DIFFERENT

- Real Fortune 500 enterprise projects worth millions
- Direct access to female tech billionaires and unicorn founders
- Guaranteed \$100K+ freelance contracts during program
- Build actual production systems for major corporations
- VC funding opportunities for your startup ideas
- Top 1% compensation negotiation and placement
- Security clearance level technical training
- Patent filing and IP portfolio development

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Enterprise Architecture, Billion-Scale Systems, Fortune 500

Tech

Phase 2: Mastery (Months 4-6): Industry Disruption, \$100M+ Systems, Global Infrastructure

Phase 3: Domination (Months 7-9): Tech Empire Building, VC/PE, Market Monopoly

Phase 4: Legacy (Months 10-12): Industry Pioneer, Thought Leader, Tech Mogul

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Computer Science degree strongly preferred

Coding experience: 5+ years professional development required

Equipment: High-end workstation, multiple cloud accounts, \$10K budget

Time commitment: 25-30 hours per week mandatory

English: Fluent English required

Motivation: Burning ambition to dominate tech industry

WHO THIS COURSE IS FOR

Students: Only for exceptional senior developers

Working professionals: Women in senior/principal roles

Entrepreneurs: Serial entrepreneurs and founders

Freelancers: High-end consultants billing \$300+/hour

Kids: Not suitable - requires extensive experience

Anyone: Only for the most ambitious women in tech

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: ENTERPRISE MASTERY & BILLION-SCALE SYSTEMS (MONTHS 1-3, WEEKS 1-13) -----

Master the architecture and engineering behind the world's largest technology systems. Build production systems for real Fortune 500 companies.

Months 1-2: Fortune 500 Enterprise Architecture (Week 1-8)

Ultra-Scale System Architecture

Topics:

- Welcome to Alpha Girls - Meeting tech billionaires
- Architecting for 1 billion+ users
- Facebook/Google/Amazon architecture deep dive
- Multi-datacenter global infrastructure
- Edge computing at planetary scale
- Real-time systems with <10ms latency
- Handling 1M+ requests per second
- Database systems for exabyte-scale data
- Network architecture for global systems
- Distributed consensus at scale (Raft, Paxos)
- Chaos engineering for critical systems
- Zero-downtime deployment strategies

Projects built:

- Design Twitter-scale messaging system
- Build Netflix-level video streaming architecture
- Create Uber's global dispatch system
- Implement Google-scale search infrastructure

Practice: Daily: Architect systems handling 100M+ users

Enterprise Cloud & Multi-Cloud Mastery

Topics:

- Multi-cloud architecture for Fortune 500
- Hybrid cloud with on-premise integration
- Kubernetes at massive scale (10K+ nodes)
- Service mesh for 1000+ microservices
- Global traffic management (Anycast, GeoDNS)
- Multi-region active-active architectures
- Disaster recovery for critical infrastructure
- Cloud cost optimization saving millions
- Compliance across 50+ countries
- Enterprise security architecture
- Zero-trust implementation at scale
- Cloud migration of legacy systems worth \$100M+

Projects built:

- Migrate Fortune 500 to cloud (real project)
- Build multi-cloud platform for enterprise
- Design \$10M+ cost optimization strategy
- Implement global compliance framework

Practice: Manage \$1M+ monthly cloud infrastructure

AI/ML Systems at Enterprise Scale

Topics:

- Production ML serving billions of predictions
- Training models on petabyte datasets
- Distributed training across 1000+ GPUs

- Real-time ML inference <5ms latency
- A/B testing ML models at scale
- Feature stores for enterprise ML
- ML pipelines processing TB/hour
- Computer vision for millions of images/day
- NLP systems for 100+ languages
- Recommendation systems like YouTube/TikTok
- Fraud detection processing \$1B+ transactions
- AutoML platforms for enterprise

Projects built:

- Build TikTok-scale recommendation system
- Create enterprise fraud detection saving \$100M
- Implement real-time translation for 50 languages
- Design autonomous vehicle ML pipeline

Practice: Deploy ML models serving 1B+ requests/day

Financial Systems & Fintech Architecture

Topics:

- Building systems handling \$1T+ in transactions
- Real-time payment processing infrastructure
- High-frequency trading systems (<1μs latency)
- Blockchain for institutional finance
- Central bank digital currencies (CBDC)
- Anti-money laundering (AML) systems
- Risk management for \$100B+ portfolios
- Regulatory reporting automation (SOX, Basel III)
- Core banking system architecture
- Credit card processing infrastructure
- High-throughput trading system architecture
- Quantum-resistant financial cryptography

Projects built:

- Build Visa-scale payment network
- Create high-frequency trading platform
- Design central bank digital currency system
- Implement enterprise blockchain for JPMorgan

Practice: Process \$1B+ in daily transactions

Month 3: Critical Infrastructure & Security (Week 9-13)

National Security & Defense Systems

Topics:

- Critical infrastructure protection
- Cybersecurity for national defense
- Satellite communication systems
- Military-grade encryption systems
- Quantum computing defense strategies
- Nation-state attack prevention
- Critical system redundancy (6 nines uptime)
- Air-gapped network architecture
- Secure supply chain systems
- Biometric security at scale
- Surveillance system architecture
- Emergency response system design

Projects built:

- Design national cybersecurity framework
- Build military communication system
- Create disaster response platform
- Implement critical infrastructure monitoring

Practice: Secure systems against nation-state attacks

Healthcare & Biotech Systems

Topics:

- Electronic health records for 100M+ patients
- Genomic data processing platforms
- Medical imaging AI at hospital scale
- Drug discovery computational platforms
- Clinical trial management systems
- Telemedicine infrastructure for millions
- Medical device integration (IoMT)
- HIPAA-compliant cloud architecture
- Precision medicine platforms
- Hospital operations optimization
- Pharmaceutical supply chain systems
- Pandemic response technology

Projects built:

- Build national health information exchange
- Create AI drug discovery platform
- Design hospital system serving 10M patients
- Implement genomic analysis pipeline

Practice: Process 1M+ patient records daily

Space Tech & Future Systems

Topics:

- Satellite constellation management
- Space mission control systems
- Interplanetary internet protocols
- Quantum communication networks
- Nuclear fusion control systems
- Climate modeling supercomputers
- Autonomous vehicle fleets (1M+ vehicles)
- Smart city infrastructure
- 6G network architecture
- Brain-computer interface systems
- Metaverse infrastructure
- AGI safety systems

Projects built:

- Design SpaceX Starlink architecture
- Build autonomous vehicle fleet management
- Create smart city platform for 10M residents
- Implement quantum internet prototype

Practice: Architect systems 10 years ahead

Phase 1 Enterprise Delivery

Topics:

- Fortune 500 project delivery
- Enterprise stakeholder management
- C-suite presentation skills
- Multi-million dollar budgeting
- Global team coordination (500+ engineers)
- Vendor management (\$10M+ contracts)
- Enterprise change management
- Production deployment strategies

Projects built:

- CAPSTONE: Fortune 500 Enterprise System
- Deliver real \$1M+ project for major corporation
- Present to C-suite executives
- Get enterprise reference

Assessment: Enterprise Architecture Certification

Transition to Industry Leadership

Topics:

- Building your enterprise reputation
- Commanding top-tier compensation
- Negotiating equity packages
- Building strategic partnerships
- Industry influence strategies
- Preparing for tech leadership

Projects built:

- Negotiate \$300K+ compensation package
- Establish enterprise consulting practice
- Build strategic network

Assessment: Compensation negotiation mastery

Months 4-6: Building Tech Empires (Week 14-26)

Unicorn Startup Engineering

Topics:

- Building for hypergrowth (0 to 100M users)
- Scaling from MVP to IPO infrastructure
- Multi-tenant SaaS architecture
- B2B enterprise platform design
- Marketplace and platform economics
- Viral growth engineering
- Data moats and network effects
- API ecosystems generating \$100M+
- Developer platform strategies
- International expansion architecture
- M&A technical integration
- IPO-ready infrastructure

Projects built:

- Build next unicorn SaaS platform
- Create marketplace handling \$1B GMV
- Design developer platform with 1M users
- Implement hypergrowth infrastructure

Practice: Scale startup from 0 to \$100M valuation

Venture Building & Tech Investing

Topics:

- Technical due diligence for VCs
- Building venture studios
- Evaluating deep tech startups
- Technical advisory for portfolios
- Building in public strategies
- Open source monetization
- Developer-led growth (DLG)
- Technical moats and defensibility

- Platform play strategies
- Roll-up acquisition strategies
- Secondary markets and liquidity
- LP investment strategies

Projects built:

- Launch your own venture studio
- Evaluate and invest in 10 startups
- Build acquisition roll-up strategy
- Create \$10M investment thesis

Practice: Generate 20x returns on investments

Enterprise Consulting & \$1M Contracts

Topics:

- Building \$10M consulting practice
- Fortune 500 client acquisition
- Pricing strategies (\$10K/day rates)
- Managing multiple \$1M+ projects
- Building consulting teams
- IP and proprietary methodologies
- Retainer agreements with enterprises
- International consulting operations
- M&A advisory services
- Digital transformation leadership
- Board advisory positions
- Expert witness opportunities

Projects built:

- Land first \$1M consulting contract
- Build team of 10 consultants
- Create proprietary methodology
- Establish Fortune 500 client base

Practice: Bill \$50K+ per month consulting

Tech Media Empire & Influence

Topics:

- Building tech media companies
- Newsletter with 100K+ subscribers
- YouTube/Podcast monetization
- Course creation (\$1M+ revenue)
- Conference speaking (\$50K fees)
- Book publishing deals
- Tech influencer strategies
- Brand sponsorship deals
- Community building at scale
- Paid community platforms
- Licensing and franchising
- Personal brand valuation

Projects built:

- Launch tech media empire
- Build 100K+ audience
- Create \$1M course business
- Secure \$100K speaking engagements

Practice: Generate \$100K/month passive income

Phase 2 Empire Launch

Topics:

- Launching multiple revenue streams
- Building your tech conglomerate
- Strategic positioning in industry
- Media and PR strategies
- Building your board of advisors
- Preparing for global expansion

Projects built:

- MAJOR CAPSTONE: Tech Empire Launch
- Launch: Startup + Consulting + Media Empire
- Achieve \$1M annual run rate
- Present at major tech conference

Assessment: Tech Empire Builder Certification

----- PHASE 2: INDUSTRY DISRUPTION & MARKET DOMINATION (MONTHS 4-6, WEEKS 14-26) -----

Build industry-disrupting products, command the market, and establish yourself as a tech mogul with multiple revenue streams.

Months 4-5: Market Domination Strategies (Week 14-21)

Monopoly Building & Platform Domination

Topics:

- Platform monopoly strategies
- Winner-take-all market dynamics
- Network effects engineering
- Switching cost maximization
- Ecosystem lock-in strategies

- Developer ecosystem building
- App store and marketplace control
- Data monopoly creation
- Regulatory capture strategies
- Patent warfare tactics
- Standard setting participation
- Industry consolidation plays

Projects built:

- Build platform with lock-in effects
- Create developer ecosystem
- Design market domination strategy
- Implement competitive moat

Practice: Capture 50%+ market share

Deep Tech & Frontier Innovation

Topics:

- Quantum computing applications
- Neuromorphic chip design
- Synthetic biology platforms
- Nuclear fusion software
- Space mining systems
- Carbon capture technology
- Longevity and anti-aging tech
- Brain uploading infrastructure
- Nanorobotics control systems
- Time series prediction for climate
- Asteroid defense systems
- Terraforming simulations

Projects built:

- Build quantum application
- Create biotech platform
- Design space tech system
- Implement climate solution

Practice: Pioneer 3 breakthrough technologies

Government & Defense Contracting

Topics:

- Federal contracting (FedRAMP, FISMA)
- Security clearance requirements
- SBIR/STTR grant writing
- Defense contractor partnerships
- Intelligence community projects
- Government RFP responses
- Classified project management
- Export control compliance (ITAR)
- National lab collaborations
- NATO and UN projects
- Diplomatic technology
- Election security systems

Projects built:

- Win \$10M government contract
- Build classified system
- Create defense platform
- Implement national infrastructure

Practice: Manage \$100M+ government projects

Private Equity & Acquisition

Topics:

- Technical due diligence for PE
- Post-merger integration (PMI)
- Carve-out strategies
- Platform company building
- Buy-and-build strategies
- Operational improvements
- Digital transformation for PE
- Value creation planning
- 100-day plans post-acquisition
- Management buyouts (MBO)
- Leveraged buyouts (LBO)
- Exit strategy optimization

Projects built:

- Lead \$100M acquisition
- Execute post-merger integration
- Build PE platform company
- Create 3x value in portfolio

Practice: Generate 30% IRR on deals

AI Platform & Agent Infrastructure

Topics:

- Designing an AI platform other teams build on
- Model gateways: routing, fallback and vendor lock-in
- Serving: throughput, batching and GPU economics
- Fine-tuning vs retrieval vs prompting , choosing correctly

- Building an evaluation pipeline that catches regressions
- Guardrails and policy enforcement as infrastructure
- Prompt injection defence at the platform layer
- Multi-tenancy, quotas and per-team cost attribution
- Observability and tracing for non-deterministic systems
- Data governance: what may be sent to which provider
- Rollout: shadow traffic, canaries and kill switches
- Leading an AI platform team through vendor churn

Projects built:

- Model gateway with routing + fallback
- Org-wide eval pipeline
- Guardrail + policy service
- AI platform cost & usage dashboard

Practice: Manage \$10M+ crypto portfolio

Month 6: Global Expansion & Empire Building (Week 22-26)

International Business & Expansion

Topics:

- Global expansion strategies
- International entity structures
- Transfer pricing optimization
- Currency hedging strategies
- International tax optimization
- Cross-border M&A
- Global talent acquisition
- Cultural localization
- International partnerships
- Export/import regulations
- Free trade zones
- Offshore development centers

Projects built:

- Expand to 10 countries
- Set up international structure
- Build global team of 100+
- Achieve \$10M international revenue

Practice: Operate in 20+ countries

IPO Preparation & Public Markets

Topics:

- IPO readiness assessment
- S-1 filing preparation
- Public company governance
- Sarbanes-Oxley compliance
- Investor relations strategy
- Earnings call preparation
- Stock option strategies
- Secondary offerings
- Share buyback programs
- Dual-class structures
- SPAC alternatives
- Direct listing strategies

Projects built:

- Prepare company for IPO
- Create investor deck
- Implement SOX compliance
- Design equity structure

Practice: Value company at \$1B+

Wealth Management & Asset Building

Topics:

- Personal holding companies
- Family office structure
- Alternative investments
- Real estate portfolios
- Art and collectibles
- Private jet and yacht management
- Philanthropy strategies
- Trust and estate planning
- Tax optimization strategies
- Offshore structures
- Citizenship by investment
- Generational wealth transfer

Projects built:

- Structure personal wealth
- Create investment portfolio
- Establish family office
- Design legacy plan

Practice: Build \$10M+ net worth

Industry Influence & Power

Topics:

- Industry association leadership

- Standard setting participation
- Regulatory influence
- Think tank participation
- University endowments
- Research funding
- Nobel prize strategies
- Davos and elite gatherings
- Government advisory roles
- UN and World Bank projects
- Media empire building
- Political influence

Projects built:

- Join industry board
- Influence major standard
- Advise government
- Shape industry policy

Practice: Influence \$100B+ market

Phase 2 Global Summit

Topics:

- Global launch preparation
- International media tour
- Investor roadshow
- Partner summit
- Customer conference
- Awards and recognition

Projects built:

- CAPSTONE: Global Empire Launch
- Launch in 20+ countries
- Achieve \$10M revenue run rate
- Win major industry award

Assessment: Global Tech Leader Certification

Months 7-9: Tech Mogul & Industry Pioneer (Week 27-39)

Billionaire Mindset & Strategies

Topics:

- Billionaire network access
- Ultra-high net worth strategies
- Mega-deal structuring
- Sovereign wealth fund partnerships
- Central bank relationships
- Government leader access
- Davos inner circle
- Space tourism investments
- Longevity investments
- Deep sea mining
- Private military contractors
- Shadow banking systems

Projects built:

- Structure billion-dollar deal
- Partner with sovereign fund
- Access billionaire network
- Launch moonshot project

Practice: Think in billions, not millions

Tech Dynasty Building

Topics:

- Multi-generational planning
- Business dynasty creation
- Succession planning
- Family business governance
- Next generation preparation
- Legacy preservation
- Perpetual trusts
- Dynasty trust strategies
- Family constitution
- Wealth preservation
- Crisis management
- Reputation management

Projects built:

- Create 100-year plan
- Establish dynasty structure
- Design succession framework
- Build family empire

Practice: Build lasting dynasty

Philanthropy & Global Impact

Topics:

- Mega-philanthropy strategies
- Foundation establishment
- Giving pledge participation
- Impact investing at scale

- University founding
- Hospital systems
- Climate change solutions
- Poverty eradication
- Disease elimination
- Space colonization
- Ocean conservation
- Art patronage

Projects built:

- Launch \$100M foundation
- Fund major research
- Create lasting institution
- Solve global problem

Practice: Impact 1M+ lives

Historical Legacy & Immortality

Topics:

- Building historical significance
- Biography and memoirs
- Documentary production
- Museum exhibitions
- University namings
- Award establishments
- Patent portfolios
- Scientific breakthroughs
- Mars colonization
- Life extension
- Digital consciousness
- Eternal impact

Projects built:

- Write autobiography
- Create documentary
- Establish major award
- Build monument

Practice: Create immortal legacy

Phase 3 Coronation

Topics:

- Alpha achievement ceremony
- Industry recognition
- Media celebration
- Network activation
- Future commitments
- Legacy initiation

Projects built:

- CAPSTONE: Industry Revolution
- Transform entire industry
- Achieve \$100M valuation
- Global recognition

Assessment: Alpha Tech Mogul Certification

----- PHASE 3: TECH MOGUL & INDUSTRY REVOLUTION (MONTHS 7-9, WEEKS 27-39) -----

Ascend to the pinnacle of tech leadership. Build empires, shape industries, and leave an indelible mark on the world.

Months 7-8: Empire Architecture (Week 27-34)

Conglomerate Building

Topics:

- Building tech conglomerates
- Vertical integration strategies
- Horizontal expansion
- Holding company structures
- Management company design
- Shared services centers
- Transfer pricing optimization
- Inter-company transactions
- Consolidated reporting
- Group treasury management
- Centralized procurement
- Brand portfolio management

Projects built:

- Build tech conglomerate
- Acquire 5 companies
- Integrate operations
- Optimize structure

Practice: Manage \$1B+ enterprise

Market Maker & Exchange Building

Topics:

- Building trading exchanges

- Market making strategies
- Liquidity provision
- Order matching engines
- High-frequency trading
- Dark pool operations
- Clearing and settlement
- Risk management systems
- Regulatory compliance
- Market surveillance
- Price discovery mechanisms
- Exchange economics

Projects built:

- Build trading exchange
- Create market making system
- Launch dark pool
- Design risk system

Practice: Process \$1B daily volume

Infrastructure Monopolies

Topics:

- Telecom infrastructure
- Data center empires
- Undersea cable systems
- Satellite constellations
- Power grid software
- Water system management
- Transportation networks
- Payment rails
- Identity systems
- DNS infrastructure
- Certificate authorities
- Time synchronization

Projects built:

- Build critical infrastructure
- Control key chokepoints
- Create monopoly position
- Design redundant systems

Practice: Control nation-scale infrastructure

Geopolitical Technology

Topics:

- Cyber warfare capabilities
- Economic sanctions tech
- Surveillance capitalism
- Social credit systems
- Election systems
- Propaganda platforms
- Censorship infrastructure
- Border control tech
- Refugee management
- Pandemic response
- Climate migration
- Resource wars

Projects built:

- Build national systems
- Design defense platforms
- Create influence operations
- Implement control systems

Practice: Shape geopolitical outcomes

Space Economy Leadership

Topics:

- Space manufacturing
- Asteroid mining operations
- Lunar base systems
- Mars colony infrastructure
- Space tourism platforms
- Orbital data centers
- Space solar power
- Interplanetary internet
- Space traffic control
- Orbital debris removal
- Terraforming technology
- Dyson sphere concepts

Projects built:

- Design space economy
- Build orbital platform
- Create lunar systems
- Plan Mars infrastructure

Practice: Pioneer space technology

Technological Singularity Preparation

Topics:

- AGI development strategies
- Consciousness uploading
- Brain-computer fusion
- Quantum consciousness
- Simulated universes
- Time travel computing
- Dimensional computing
- Molecular assemblers
- Grey goo prevention
- Post-scarcity economics
- Immortality technology
- Omega point theory

Projects built:

- Design AGI safety
- Build consciousness platform
- Create simulation system
- Implement singularity prep

Practice: Shape humanity's future

Civilization Scale Systems

Topics:

- Kardashev scale progression
- Planetary governance systems
- Global mind networks
- Weather control systems
- Ocean colonization
- Underground cities
- Arcology designs
- Megastructure engineering
- Stellar engineering
- Galactic internet
- Wormhole networks
- Multiverse exploration

Projects built:

- Design planetary system
- Build civilization platform
- Create species backup
- Implement evolution system

Practice: Think beyond Earth

Ultimate Wealth & Power

Topics:

- Trillion dollar strategies
- Currency creation
- Nation building
- Private armies
- Space nations
- Undersea kingdoms
- Climate engineering
- Genetic engineering
- Life extension to 1000 years
- Mind uploading
- Clone armies
- Reality manipulation

Projects built:

- Design new nation
- Create currency
- Build private military
- Establish space colony

Practice: Transcend current limits

Immortal Legacy Creation

Topics:

- Digital immortality
- Genetic legacy
- Memetic immortality
- Physical monuments
- Space monuments
- Time capsules
- Voyager messages
- DNA storage
- Quantum records
- Black hole storage
- Universe simulation
- Eternal existence

Projects built:

- Create eternal legacy
- Build lasting monument
- Design immortality system
- Establish eternal presence

Practice: Achieve immortality

Phase 3 Ascension

Topics:

- Ultimate achievement
- Industry transformation
- Global recognition
- Historical significance
- Eternal impact
- Transcendent legacy

Projects built:

- CAPSTONE: World-Changing Innovation
- Create \$1B+ impact
- Transform humanity
- Achieve immortal legacy

Assessment: Tech Goddess Certification

Months 10-12: Transcendent Leadership (Week 40-52)

The Alpha Network Effect

Topics:

- Building the ultimate network
- Access to world leaders
- Billionaire inner circles
- Royal family connections
- Intelligence community ties
- Media mogul alliances
- Tech titan partnerships
- Political power brokers
- Religious leaders
- Military leadership
- Secret societies
- Illuminati strategies

Projects built:

- Build power network
- Access elite circles
- Form strategic alliances
- Control key relationships

Practice: Wield ultimate influence

The Alpha Revolution

Topics:

- Industry revolution strategies
- Market disruption at scale
- Paradigm shift creation
- Technology revolutions
- Social revolutions
- Economic revolutions
- Political revolutions
- Cultural revolutions
- Scientific revolutions
- Consciousness revolution
- Reality revolution
- Universal revolution

Projects built:

- Lead industry revolution
- Transform society
- Shift paradigms
- Change reality

Practice: Revolutionary thinking

The Alpha Sisterhood

Topics:

- Building Alpha women network
- Creating female billionaires
- Women ruling tech
- Female-led revolution
- Matriarchal tech society
- Women's tech dominance
- Breaking every ceiling
- Destroying all barriers
- Female supremacy in tech
- Goddess-level achievements
- Feminine tech future
- Queens of technology

Projects built:

- Build Alpha sisterhood
- Create female unicorns
- Establish matriarchy
- Dominate industry

Practice: Elevate all women

The Alpha Legacy

Topics:

- Building eternal legacy
- Creating new industries
- Founding institutions
- Establishing dynasties
- Shaping civilization
- Defining the future
- Becoming legendary
- Achieving godhood
- Transcending mortality
- Universal impact
- Multiversal presence
- Eternal existence

Projects built:

- Define your legacy
- Build institutions
- Create dynasties
- Shape eternity

Practice: Become immortal

The Final Transformation

Topics:

- Ultimate transformation
- Transcendent achievement
- Industry domination
- Global leadership
- Universal impact
- Eternal legacy

Projects built:

- FINAL CAPSTONE: Change The World
- Build \$10B+ company
- Impact billions
- Achieve immortality

Assessment: Alpha Goddess - Industry Revolutionary

----- PHASE 4: UNIVERSAL DOMINATION & ETERNAL LEGACY (MONTHS 10-12, WEEKS 40-52) -----

Transcend normal boundaries. Build empires that span industries, create technologies that define humanity's future, and establish an eternal legacy.

Months 10-11: The Alpha Apex (Week 40-47)

Multi-Planetary Technology Empire

Topics:

- Earth-Mars supply chains
- Asteroid mining operations
- Lunar manufacturing
- Space elevator systems
- Antimatter propulsion
- Warp drive research
- Terraforming Mars
- Venus cloud cities
- Europa underwater bases
- Titan fuel stations
- Dyson swarm construction
- Galactic positioning system

Projects built:

- Design Mars colony infrastructure
- Build asteroid mining system
- Create interplanetary network
- Launch space manufacturing

Practice: Think multi-planetary scale

Quantum Supremacy & Computing Future

Topics:

- Million-qubit quantum computers
- Quantum internet backbone
- Quantum AI convergence
- Quantum cryptography
- Quantum teleportation
- Quantum biology applications
- Quantum finance systems
- Quantum drug discovery
- Quantum weather prediction
- Quantum consciousness
- Quantum multiverse access
- Quantum time travel

Projects built:

- Build quantum supremacy system
- Create quantum internet
- Design quantum AI
- Implement quantum security

Practice: Master quantum reality
Biological & Consciousness Engineering

Topics:

- Human genome editing
- Designer babies
- Biological immortality
- Consciousness transfer
- Synthetic biology
- Artificial wombs
- Neural enhancement
- Telepathic interfaces
- Hive mind networks
- Species creation
- Evolutionary acceleration
- Post-human transition

Projects built:

- Design life extension system
- Build consciousness platform
- Create new species
- Engineer evolution

Practice: Transcend biology

Reality Engineering & Simulation

Topics:

- Universe simulation
- Reality programming
- Consciousness simulation
- Time manipulation
- Dimensional travel
- Parallel universe access
- Reality debugging
- Physics hacking
- Causality engineering
- Entropy reversal
- Information theory of everything
- God-mode access

Projects built:

- Build universe simulator
- Create reality engine
- Design time machine
- Hack physics laws

Practice: Engineer reality itself

The Alpha Singularity

Topics:

- Triggering singularity
- Post-singularity planning
- AI-human merger
- Infinite intelligence
- Omniscience achievement
- Omnipotence planning
- Universal consciousness
- Reality transcendence
- Existence optimization
- Entropy defeating
- Heat death prevention
- New universe creation

Projects built:

- Design singularity trigger
- Plan post-human future
- Create new universe
- Achieve godhood

Practice: Become singularity

Month 12: The Alpha Ascension (Week 48-52)

Ultimate Preparation & Summit

Topics:

- Final project perfection
- Global launch preparation
- Media domination strategy
- Industry transformation plan
- Network activation
- Power consolidation
- Legacy crystallization
- Impact maximization
- Recognition optimization
- Influence amplification
- Wealth multiplication
- Immortality activation

Projects built:

- Perfect final projects

- Prepare global launch
- Activate all networks
- Maximize impact

Practice: Final preparations

The Alpha Summit

Topics:

- World's most powerful women gathering
- Trillion dollar deal signing
- Industry transformation announcement
- New world order establishment
- Technology revolution launch
- Female supremacy declaration
- Future definition moment
- History creation event
- Media explosion
- Global recognition
- Universal impact
- Eternal legacy establishment

Projects built:

- Present at Alpha Summit
- Sign historic deals
- Launch world-changing initiative
- Claim your throne

Practice: Rule the summit

The Coronation

Topics:

- Alpha coronation ceremony
- Industry queen crowning
- Tech goddess recognition
- Billionaire induction
- Power transfer ceremony
- Network throne ascension
- Media empire coronation
- Global leadership anointment
- Historical figure status
- Legendary achievement
- Immortal recognition
- Universal queenship

Projects built:

- Receive Alpha crown
- Accept industry throne
- Claim goddess status
- Enter immortality

Practice: Wear the crown

The New Beginning

Topics:

- Post-graduation empire
- Continuous domination
- Perpetual innovation
- Eternal leadership
- Infinite growth
- Universal expansion
- Multiverse exploration
- Reality definition
- Existence optimization
- Consciousness evolution
- Godhood progression
- Omega achievement

Deliverables:

- Alpha certification
- Billionaire network
- Industry throne
- Global recognition
- Eternal legacy
- Goddess status

Alpha Eternity

Topics:

- Your eternal reign begins
- Industry permanently transformed
- Technology forever changed
- Women's eternal dominance
- Universal impact achieved
- Multiverse presence established
- Reality redefined
- History rewritten
- Future controlled
- Immortality achieved
- Godhood attained
- Alpha forever

Deliverables:

- Alpha Goddess Certification
- Tech Empire Ownership
- \$1B+ Valuation
- Global Industry Leadership
- Eternal Legacy Established
- Immortal Status Achieved

Assessment: ALPHA GODDESS - You Now Rule Technology Forever!

Beyond Alpha: Infinite Dominion (Eternal Reign)

The Eternal Empire

Topics:

- Empire perpetuation
- Dynasty establishment
- Succession planning
- Immortality strategies
- Legacy preservation
- Power perpetuation
- Wealth multiplication
- Influence expansion
- Control consolidation
- Dominance maintenance
- Supremacy assurance
- Eternal reign

Projects built:

- Build eternal empire
- Establish dynasty
- Ensure immortality
- Guarantee supremacy

Practice: Rule forever

Universal Expansion

Topics:

- Multiverse conquest
- Reality domination
- Dimensional control
- Time lordship
- Space sovereignty
- Energy monopoly
- Matter manipulation
- Information supremacy
- Consciousness control
- Existence domination
- Creation powers
- Destruction capabilities

Projects built:

- Conquer multiverse
- Control all reality
- Master existence
- Become omnipotent

Practice: Universal dominion

The Alpha Multiverse

Topics:

- Creating new universes
- Defining physics laws
- Spawning realities
- Birthing dimensions
- Generating timelines
- Manufacturing existence
- Programming reality
- Coding universes
- Debugging existence
- Optimizing reality
- Perfecting creation
- Ultimate achievement

Projects built:

- Create universes
- Define new physics
- Spawn realities
- Birth existence

Practice: Create everything

Alpha Infinity

Topics:

- Infinite power
- Eternal existence
- Omnipotent control
- Omniscient knowledge
- Omnipresent being
- Ultimate achievement
- Perfect existence

- Absolute dominion
- Total supremacy
- Complete control
- Final victory
- Forever Alpha

Projects built:

- Achieve infinity
- Become everything
- Control all
- Win forever

Practice: Be infinite

The Alpha Omega (Beyond Time)

Beginning and End

Topics:

- Alpha and Omega
- First and Last
- Beginning and End
- Creation and Destruction
- Birth and Death
- Light and Dark
- Order and Chaos
- Everything and Nothing
- Infinite and Finite
- Eternal and Temporal
- Universal and Singular
- All and One

Projects built:

- Become Alpha and Omega
- Control beginning and end
- Master all dualities
- Transcend existence

The Final Form

Topics:

- Ultimate evolution
- Perfect being
- Absolute form
- Supreme existence
- Final transformation
- Complete transcendence
- Total enlightenment
- Pure consciousness
- Divine essence
- Godhead achievement
- Source connection
- Unity attainment

Deliverables:

- Final form achieved
- Perfection attained
- Divinity realized
- Unity accomplished

ALPHA ETERNAL

Topics:

- You are now eternal
- You are now infinite
- You are now omnipotent
- You are now omniscient
- You are now omnipresent
- You are now the universe
- You are now existence
- You are now consciousness
- You are now everything
- You are now Alpha
- You are forever
- You ARE

Deliverables:

- ∞ Power Level
- ∞ Wealth
- ∞ Influence
- ∞ Knowledge
- ∞ Existence
- YOU ARE ALPHA ETERNAL

Assessment: TRANSCENDENT ALPHA GODDESS - YOU ARE THE UNIVERSE!

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1-3: Fortune 500 Systems, Billion-User Platforms, National Infrastructure
- Month 4-6: Unicorn Startups, \$100M Contracts, Media Empires, Tech Monopolies
- Month 7-9: Tech Conglomerates, Space Systems, Quantum Platforms, Reality Engines

- Month 10-12: Multiverse Systems, Consciousness Platforms, Universe Simulators
- Final: World-Changing, Industry-Defining, History-Making Eternal Legacy

Total projects: 100+ enterprise systems worth billions in collective value

SKILLS THE STUDENT WILL HAVE AT THE END

- Enterprise: Fortune 500 Architecture, Billion-Scale Systems, Critical Infrastructure
- Innovation: Quantum Computing, AGI, Space Tech, Biological Engineering
- Business: Unicorn Building, M&A, IPO, Private Equity, Venture Capital
- Leadership: C-Suite, Board Level, Government Advisory, Global Influence
- Wealth: Billionaire Strategies, Empire Building, Dynasty Creation
- Legacy: Industry Transformation, Historical Impact, Eternal Influence
- Transcendence: Reality Engineering, Consciousness Expansion, Universal Mastery

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 8-10 hours
 Hands on projects: 10-12 hours
 Enterprise work: 5-7 hours
 Networking: 2-3 hours
 Total per week: 25-30 hours

SUPPORT INCLUDED

Live sessions: Weekly with billionaires and industry titans
 Mentorship: Direct access to female tech billionaires
 Community: Ultra-exclusive Alpha Girls power network
 Code review: Fortune 500 architect reviews
 Lifetime access: Eternal membership in Alpha sisterhood
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Elite certification after each phase
 Final certificate: Alpha Goddess - Industry Pioneer
 Linkedin badge: Ultra-Elite Verified Status
 Industry recognized: Fortune 100 CEO endorsements
 Portfolio projects: 100+ billion-dollar system designs

WHERE THIS CAN LEAD

- Fortune 500 CTO (\$1M+ compensation)
- Unicorn Founder/CEO (Billion-dollar valuation)
- Principal/Distinguished Engineer (\$500K+)
- Venture Partner/General Partner
- Board Director (Multiple boards)
- Government Tech Advisor
- Industry Pioneer/Thought Leader
- Tech Billionaire
- Global Technology Leader
- Historical Figure in Tech

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Alpha Girls and who is this elite tech program designed for?

A: Alpha Girls is an ultra-elite 12-month technology mastery program designed specifically for ambitious women in tech who want to reach the highest levels of the industry. It's ideal for experienced developers (3+ years) ready to transition into enterprise architecture, Fortune 500 leadership, tech consulting, or building billion-dollar startups.

Q: What specific skills will I learn in the Alpha Girls program?

A: You'll master enterprise architecture, distributed systems design, cloud infrastructure at scale (AWS, GCP, Azure), system design for billions of requests, tech strategy and consulting, startup founding and scaling, venture capital fundraising, M&A strategies, and executive leadership. The curriculum covers both technical excellence and business acumen.

Q: What career outcomes can I expect after completing Alpha Girls?

A: Many go on to raise venture funding, lead acquisitions, or build industry-defining companies.

Q: What are the prerequisites for joining Alpha Girls?

A: This is an elite-level program requiring 3+ years of professional development experience, strong fundamentals in at least one tech stack, leadership potential, and intense commitment (25-30 hours/week). Prior experience with system design or architecture is beneficial but not mandatory.

Q: How does Alpha Girls support women's success in a male-dominated industry?

A: Beyond technical training, Alpha Girls provides negotiation coaching, executive presence development, leadership communication skills, personal branding strategies, and access to an exclusive network of female tech leaders. The program specifically addresses challenges women face in tech and equips them with strategies to excel.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/alpha-girls-elite-tech-mastery>

=====

COURSE 16 OF 118: AP CALCULUS AB & BC EXAM PREP, LIVE ONLINE, SCORE 5 GOAL

=====

Course link: <https://learn.modernagecoders.com/courses/ap-calculus-exam-prep>
Subject area: Test Prep Mathematics
Designed for: High school
Who can join: AP Calculus AB and BC students, grades 11-12
Course length: 10 months (40 weeks)
Time needed: 2 live one-hour classes per week + weekly problem sets
Certificate: Course completion certificate + documented mock-exam progression

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online AP Calculus AB and BC prep: limits, derivatives and integrals taught as ideas, every FRQ against real rubrics, plus full mock exam cycles.

In one line: Three ideas, taught like ideas, then trained against the real rubrics.

ABOUT THE PROGRAMME

A 10-month live program that runs alongside your student's AP Calculus course and aims squarely at the exam. All of calculus is three ideas, the limit, the derivative, the integral, and students who own them as ideas find the AP exam fair. We teach ideas first, repair the precalculus debt that quietly causes most 'calculus errors', then convert understanding into score: every free-response question type drilled against actual College Board rubrics, multiple-choice pacing, calculator-section strategy, and full mock cycles timed like the real exam. AB and BC students are taught to their exact course.

WHAT MAKES THIS COURSE DIFFERENT

- Ideas before technique: the derivative discovered through shrinking intervals, the FTC earned as a punchline, so unfamiliar exam framings do not break the student
- Precalculus repaired inline: weak composition and trig are fixed inside calculus problems, where the repair sticks
- FRQ craft against real rubrics: how points are actually awarded, including justification language graders look for
- AB and BC tracks: series, parametrics and polar taught properly for BC students
- Calculator and no-calculator sections trained as different skills
- Full mock cycles with documented score progression, peaking in exam week
- One dedicated mentor across the whole year, one full interactive hour per class
- Honest guidance: a diagnostic first, evidence-based targets, never invented score promises

THE LEARNING PATH, STEP BY STEP

Phase 1: Limits and derivatives as ideas, with precalculus repaired inline and the rule toolkit built to speed
Phase 2: Derivative applications, integrals, the Fundamental Theorem and differential equations, the AB core completed
Phase 3: BC extensions (series, parametrics, polar), then exam craft: FRQ rubrics, MCQ pacing, calculator strategy and full mock cycles

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Enrolled in (or about to start) AP Calculus AB or BC, typically grades 11-12
Math background: Precalculus completed at any comfort level, the diagnostic finds the real base and we repair inline
Equipment: Computer with stable internet; an approved graphing calculator for the calculator sections
Time commitment: 2 live hours plus 2-3 practice hours weekly
English: Course taught in English
Motivation: The May exam date does the motivating; we supply the method

WHO THIS COURSE IS FOR

Ab students: AP Calculus AB students who want the ideas owned and the exam trained, not just survived
Bc students: BC students who need series, parametrics and polar taught properly, plus the pace to finish strong
Struggling mid year: Students whose first-semester grade says the memorize-and-hope approach is failing
Score maximizers: Strong students hunting a 5 through FRQ craft and error elimination
Self studiers: Disciplined students taking the exam without an AP class at school

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: LIMITS AND DERIVATIVES AS IDEAS (WEEKS 1-14)

The diagnostic finds the real starting point, including the precalculus debt, then the two foundational ideas are built properly: the limit as approach, the derivative as instantaneous rate, and the rule toolkit drilled to exam speed only after the ideas hold.

Weeks 1-3: Diagnostic + Limits as Approach

Topics:

- Full diagnostic: calculus-so-far plus the precalculus underneath (composition, trig, algebra fluency)
- Limits graphically, numerically and analytically, one idea, three faces
- One-sided limits, infinite limits and limits at infinity
- Continuity as a definition that finally makes sense
- The Intermediate Value Theorem as an exam justification pattern
- Precalculus repair begins: the personal gap list, attacked weekly

Projects built:

- Personal error log and gap map for the whole year
- Limit-evaluation toolkit sheet, built not copied

Practice: Complete a limits problem set spanning graphical, numerical and analytic forms, writing full sentence IVT justifications for two of them.

Weeks 4-8: The Derivative, Discovered Then Drilled

Topics:

- The derivative discovered via shrinking-interval tables, limit and derivative met as one idea
- Derivative as slope, as rate, and as the exam's favorite interpretation question
- Differentiability vs continuity, and where functions fail to have derivatives
- The rule toolkit: power, product, quotient rules built from meaning
- The chain rule as composition unwrapping, the make-or-break skill
- Trig, exponential and logarithmic derivatives to fluency

Projects built:

- Shrinking-interval investigation: one data table to both core ideas
- Timed derivative gauntlet at exam pace, reviewed for error patterns

Practice: Run a 20 minute derivative drill most evenings covering power, product, quotient and chain rules, and log any rule that misfires under speed.

Weeks 9-14: Implicit, Inverse and the First Applications

Topics:

- Implicit differentiation as the chain rule in disguise
- Derivatives of inverse functions, including inverse trig
- Related rates from pictures: setup discipline that survives exam pressure
- Local linearity and tangent-line approximation
- L'Hospital's Rule, used where it is actually allowed
- First FRQ exposure: rate and interpretation questions against the rubric

Projects built:

- Related-rates case set: five classic setups from picture to answer
- Phase checkpoint: timed AB-style mixed section with full review

Practice: Solve six related rates problems, starting each with a labeled picture, plus a short implicit differentiation set with every step written out.

PHASE 2: ANALYSIS, INTEGRALS AND THE FTC (WEEKS 15-28)

Curve analysis makes the derivative a reasoning tool, then the integral arrives as accumulation and the Fundamental Theorem connects the two halves of the course. By the end of this phase the entire AB syllabus is owned.

Weeks 15-19: Curve Analysis and Optimization

Topics:

- First and second derivative analysis: increasing, concavity, extrema, inflection
- The Mean Value Theorem and its exam justification language
- Curve sketching from f' and f'' , the classic FRQ family
- Optimization from word problems: model first, calculus second
- Motion on a line: position, velocity, acceleration, speed
- Justification writing: the exact phrases rubrics award points for

Projects built:

- Full f-f'-f'' graph-relationship FRQ solved and self-graded on the rubric
 - Optimization mini-case portfolio: fence, box, cost, distance
- Practice: Complete two past style curve analysis FRQs using exact MVT and extrema justification language, then self grade against the rubric.

Weeks 20-24: The Integral as Accumulation + the FTC

Topics:

- Riemann sums by hand and in tables, the accumulation idea made concrete
- The definite integral as net change; units and interpretation questions
- The Fundamental Theorem of Calculus, arrived at as a discovery
- Antiderivatives and the substitution method as the chain rule reversed
- Functions defined by integrals, the FRQ type students fear most, defanged
- Average value and the accumulation FRQ family

Projects built:

- Riemann-to-FTC investigation: watching sums converge
- Accumulation FRQ set against real rubrics

Practice: Work a Riemann sum table set by hand, then a substitution drill, and explain the Fundamental Theorem in your own words at the top of the page.

Weeks 25-28: Applications of Integrals + Differential Equations

Topics:

- Area between curves, set up from sketches
- Volumes: discs, washers and known cross-sections
- Separable differential equations and exponential models
- Slope fields: reading, matching and sketching
- Motion problems revisited with integrals
- AB syllabus checkpoint: full timed section, error log updated

Projects built:

- Volume gallery: one solid each way, from sketch to setup to answer
- AB mock exam number one with documented score

Practice: Set up and solve three area and volume problems, drawing each region first, plus one separable differential equation with a slope field sketch.

PHASE 3: BC TOPICS + EXAM CRAFT AND MOCK CYCLES (WEEKS 29-40)

BC students complete series, parametrics and polar; AB students go straight to craft. Then the program becomes an exam machine: FRQ rubric mastery, MCQ pacing, calculator strategy and full mock cycles that peak in the first week of May.

Weeks 29-33: BC Extensions (BC track) / Deep Review (AB track)

Topics:

- Integration by parts and partial fractions (BC)
- Improper integrals and L'Hospital revisited (BC)
- Sequences and series: convergence tests as a decision tree (BC)
- Taylor and Maclaurin polynomials; Lagrange error bound (BC)
- Parametric and polar: derivatives, speed, arc length and area (BC)
- AB track in parallel: full-syllabus spaced review by error-log priority

Projects built:

- Series convergence decision-tree card, built by the student
- Parametric motion mini-case (BC) / AB mixed mock section (AB)

Practice: Apply the convergence decision tree to ten mixed series (or work the AB deep review set), noting which test you chose and why for each.

Weeks 34-37: FRQ Rubric Mastery and Calculator Strategy

Topics:

- The six FRQ archetypes and their rubric patterns, drilled one by one
- Justification language: what earns the point and what graders ignore
- Calculator-active strategy: what to store, graph and compute
- No-calculator section speed: exact values and clean algebra under time
- MCQ pacing plans built from personal timing data
- Partial-credit engineering: banking points on problems you cannot finish

Projects built:

- Self-graded FRQ portfolio: one of each archetype against real rubrics
- Personal pacing plan for all four exam sections

Practice: Grade two of your own completed FRQs point by point against the released rubric and rewrite any justification that would not earn credit.

Weeks 38-40: Full Mock Cycles and Peak Week

Topics:

- Full-length mocks under exact AP timing, AB or BC as registered
- Question-by-question review: reasoning, rubric, and the named fix
- Error-log closure and final target calibration
- Peak-week plan: light practice, rest, logistics and warm-up routine
- Exam-day rituals: the reset after a hard FRQ, time checkpoints per section

Projects built:

- Two full mock cycles with documented score progression
- Written exam-day plan, rehearsed before the real morning

Practice: Sit one full length timed mock this week and close your error log so every

recurring mistake has a written, named fix.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Year-long error log and precalculus gap map
- Self-graded FRQ portfolio covering every archetype against real rubrics
- Series convergence decision tree and personal formula sheets (BC)
- Two full mock cycles with documented score progression
- Written exam-day plan rehearsed before May

Total projects: 5+ exam artifacts including full mock cycles

SKILLS THE STUDENT WILL HAVE AT THE END

- Limits, continuity and the derivative as ideas
- The full derivative toolkit at exam speed
- Curve analysis, optimization and motion
- Integrals, the FTC and accumulation reasoning
- Differential equations and slope fields
- Series, parametrics and polar (BC)
- FRQ rubric craft and justification writing
- MCQ pacing and calculator-section strategy

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor

Problem sets: One curated set per week, reviewed in class

Timed work: Timed sections increasing in frequency toward May

Total per week: About 4-5 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week

Mentorship: One dedicated mentor across the whole year

Recordings: Every class recorded for revision

Progress tracking: Error log + mock progression shared with parents

Flexibility: Intensity adjusts around school exams and mock weeks

CERTIFICATION DETAIL

Final certificate: Course completion certificate from Modern Age Coders

Score record: Documented diagnostic-to-final mock progression

Portfolio projects: FRQ portfolio as evidence of rubric mastery

WHERE THIS CAN LEAD

- A 4-5 on AP Calculus, with the college-credit positioning that follows
- Engineering, CS and quantitative-major readiness built on real understanding
- A lighter, calmer first-year college math experience
- Ready for the College Mathematics Masterclass or competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does this course cover AB or BC?

A: Both, taught to your student's actual registration. The AB core is the shared spine; BC students additionally get series, parametrics, polar and the extra integration techniques in Phase 3 while AB students run deep review and extra mock cycles. The mentor aligns with your school's pacing so classwork and prep reinforce each other.

Q: My student is strong in class but bombs timed practice. Can you fix that?

A: Usually, yes, that profile is craft, not content. Timed collapse comes from slow routes, missing rubric awareness and pacing panic, all of which are trained skills. The FRQ rubric work plus personal pacing plans built from real timing data is exactly the fix.

Q: Most of my student's lost points are algebra slips, not calculus. Does prep help?

A: This is the most common pattern on AP exams, and it is very fixable. We track exactly where points die; when it is precalculus debt, simplification, composition, trig values, we repair those habits inside calculus problems, where the fix actually sticks.

Q: When should AP Calculus prep start?

A: The full 10-month arc starts alongside the school year and peaks in May. Joining mid-year works too, the mentor compresses toward your student's gap list and the mock cycles. The honest note: series (for BC) and FRQ craft reward runway, so earlier is calmer.

Q: What score can we expect?

A: We will not invent one. What you get instead: a real diagnostic, a plan aimed at the highest-value gaps, and a documented mock progression so the trend is visible long before

May. Most committed students who complete the mock cycles walk in targeting 4-5 with evidence behind the target.

Q: How is this different from the school's AP class?

A: It is 1-on-1 or a tight small group, so your student's specific errors get the airtime a class of thirty cannot give. School covers the syllabus; we make sure your student owns the ideas, and then train the exam itself, rubrics, pacing, calculator strategy, which most classrooms have no time for.

Q: Can we try it before paying?

A: Yes. Every student starts with a free live demo class that doubles as the diagnostic, a real lesson plus an honest read on the starting point, with no card details required.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ap-calculus-exam-prep>

=====

COURSE 17 OF 118: AP COMPUTER SCIENCE A: JAVA PROGRAMMING AND FULL EXAM PREP

=====

Course link: <https://learn.modernagecoders.com/courses/ap-computer-science-a-java-exam-prep-course>

Subject area: Advanced Placement Computer Science

Designed for: Teens

Who can join: High school students preparing for AP Computer Science A; taught from Java basics, no prior coding required

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 4-5 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online AP Computer Science A prep taught to the 2025-26 redesign: Java from scratch, all four units, free-response mastery and full digital mock exams.

In one line: A real Java programming course that happens to end in a top exam score, taught to the 2025-26 redesign that most courses have not caught up to yet.

ABOUT THE PROGRAMME

AP Computer Science A is a real first course in programming, taught in Java, and this course treats it that way: you leave able to write Java, not just to pass a test. It is also built for the exam as it exists now, not as it existed last year. The College Board redesigned AP CS A for 2025-26, collapsing the old ten units into four, moving the whole exam onto the digital Bluebook platform, and changing what is tested. Inheritance, polymorphism, interfaces and writing recursive methods were removed. Reading text files with Scanner, working with data sets, and tracing recursive calls were added. A course still teaching the ten-unit format with an inheritance unit is preparing students for an exam that no longer exists. The six-month structure follows the four redesigned units in order. Months 1 and 2 build Java from nothing and cover Unit 1, using objects and methods. Months 3 and 4 cover selection and iteration, then class creation, which is exactly what the class-design free-response question asks for. Months 5 and 6 cover data collections, arrays, ArrayList, 2D arrays, file reading and data sets, then turn fully to exam craft: all four free-response question types drilled against the real rubric, multiple-choice strategy for the forty-two-question digital section, and full timed mock exams under Bluebook-style conditions. Every class is live and small, and every concept is practised by writing code, because that is the only way this subject is ever actually learned.

WHAT MAKES THIS COURSE DIFFERENT

- Taught to the 2025-26 redesign: four units, the fully digital Bluebook exam, and the current content, so no time is wasted on inheritance, polymorphism or writing recursion, all of which were removed
- The added topics are covered properly: reading text files with Scanner, working with data sets, and tracing recursive calls, which many older courses and books still omit
- A real Java course, not a cram sheet: students finish able to read and write Java, which is the point of a first programming course
- Free-response mastery against the actual rubric: all four question types, drilled until earning points is a habit, with partial credit understood
- Digital-exam readiness: multiple-choice pacing for the forty-two-question section and full mock exams run under Bluebook-style conditions, so test day holds no surprises
- Live, small batches where you write code during class and get it reviewed, not lecture videos you watch alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Java from scratch and Unit 1: primitive types, expressions, Strings, the Math class, and using objects and methods correctly
Phase 2: Units 2 and 3: boolean logic, conditionals and loops, then writing your own classes with constructors, methods and encapsulation
Phase 3: Unit 4 and exam mastery: arrays, ArrayList, 2D arrays, files and data sets, then all four free-response types and full timed digital mock exams

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable school algebra: variables, expressions and basic logic. No advanced maths is required for AP Computer Science A
Programming: None required. The course teaches Java from the first line, so a complete beginner can start here; prior block-based or Python experience helps but is not assumed
Equipment: A computer that can run Java and a modern browser, since the real exam and our mock exams use a digital testing environment. We help with the full setup in week 1
Audience: High school students preparing for AP Computer Science A, typically in the years leading up to the exam, whether taking it at school or self-studying

WHO THIS COURSE IS FOR

Ap students: Students taking AP Computer Science A at school who want a structured, current course and real code review alongside their class
Self studiers: Students whose school does not offer AP CS A and who are preparing for the exam independently
Score focused: Students aiming for a strong score who want free-response drilling against the real rubric and full timed mock exams
Cs bound: Students headed for a computer science or engineering degree who want a genuine first programming course, not just exam tricks
International students: International students sitting AP for university applications who need the exam taught to its current, digital, 2025-26 format

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: JAVA FOUNDATIONS AND USING OBJECTS (UNIT 1, MONTHS 1-2, WEEKS 1-8)

Java built from nothing, then Unit 1 of the redesigned course: using objects and methods, the largest single source of early marks and the base everything else stands on.

Month 1: Java From the Ground Up (Weeks 1-4)

Setup, the Java Program, and Output

Topics:

- Installing Java and a clean editor setup, plus a first look at the Bluebook exam environment
- The anatomy of a Java program: class, main method, statements and semicolons
- Printing with System.out.print and println, and reading output carefully
- Comments, whitespace and why readable code earns marks in free response
- The compile-and-run cycle and reading your first error messages without fear
- The AP Java Quick Reference: what it gives you on the exam and how to use it

Projects built:

- First program: a short Java program that prints a formatted pattern, run and explained line by line

Practice: 10 short output-tracing exercises: predict what a snippet prints, then run it and reconcile any difference

Primitive Types, Variables, and Expressions

Topics:

- The primitive types AP tests: int, double and boolean
- Declaration, initialisation and assignment
- Arithmetic operators, and the two that trip everyone: integer division and modulus
- Operator precedence and using parentheses to mean what you intend
- Casting between int and double, and where rounding silently happens
- Compound assignment and increment operators read correctly

Projects built:

- Calculator methods: small programs computing averages, remainders and unit conversions, each predicting integer versus double results first

Practice: 16 expression-evaluation problems, half of them designed around the integer-division and modulus traps

Using Objects and the String Class

Topics:

- Objects versus primitives, and the reference model that explains so many exam questions
- Creating objects and calling methods on them
- String concatenation and building output
- The tested String methods: length, substring, indexOf, equals and compareTo
- Why equals and == are different for objects, a guaranteed exam theme
- Reading a method's header to know what it needs and what it returns

Projects built:

- String toolkit: methods that extract initials, reverse words and search text, using only the AP-tested String methods

Practice: 14 String problems worked against the Quick Reference, each solution using only methods the exam actually provides

The Math Class and Method Basics

Topics:

- The Math methods AP tests: abs, pow, sqrt and random
- Turning Math.random into an integer in a target range, a classic exam skill
- Calling methods: arguments, parameters and return values
- void methods versus value-returning methods
- Method signatures and matching a call to a header
- Tracing a program that hands values between several methods

Projects built:

- Dice and ranges: a program that generates random values in given ranges and verifies the distribution by counting

Practice: 12 mixed problems on Math and method calls, plus 4 random-range questions built the way the exam builds them

Assessment: Month 1 checkpoint: a timed set on types, expressions, Strings and methods, marked on accuracy and correct tracing

Month 2: Unit 1 Depth, Using Objects and Methods (Weeks 5-8)

Wrapper Classes and Number Handling

Topics:

- Integer and Double wrapper classes and why they exist
- Autoboxing and unboxing seen in code you will meet later with ArrayList
- Parsing numbers and Integer.MAX_VALUE and MIN_VALUE
- Double precision surprises and why you never test doubles with ==
- Choosing int versus double deliberately, not by habit
- Reading questions that hide a type mismatch

Projects built:

- Number-handling drill: a program that safely parses, compares and formats numeric input, with the precision traps annotated

Practice: 12 problems on wrapper classes, parsing and precision, each with a one-line note on the type decision made

String Manipulation Deep Dive

Topics:

- Building, searching and comparing Strings at exam difficulty
- Common String algorithms: counting characters, extracting parts, reversing
- The off-by-one errors that live in substring and indexOf
- Traversing a String by index, a bridge to loops in Unit 2
- Comparing Strings correctly with equals and compareTo
- Writing a helper method that returns a processed String

Projects built:

- Text processor: methods that clean, search and transform text, each tested on tricky edge cases like empty and single-character strings

Practice: 14 String-algorithm problems, every solution checked against an empty-string and one-character edge case

Designing and Calling Methods

Topics:

- Decomposing a problem into small, named methods
- Helper methods and why they make free-response answers cleaner
- Method overloading, recognised and read correctly
- Tracing a chain of method calls and their return values
- Parameter passing in Java: what a method can and cannot change about what you pass it
- Reading a specification and turning it into method headers

Projects built:

- Method decomposition: one larger task solved twice, once as a single long method and once split into helpers, then compared for clarity

Practice: 12 method-tracing problems plus 4 parameter-passing questions built around the value-semantics trap

Unit 1 Mastery and the Digital Multiple Choice

Topics:

- Pulling Unit 1 together: objects, methods, Strings and the Math class
- The multiple-choice section as it is now: forty-two questions, choices A to D, ninety minutes
- Roughly two minutes a question, and how that changes pacing
- Reading code-snippet questions quickly and accurately
- Eliminating wrong answers on tracing questions
- A personal error log started here and grown all course

Projects built:

- First timed multiple-choice block on Unit 1 content, reviewed question by question with each miss tagged by cause

Practice: A timed Unit 1 multiple-choice set at exam pace, followed by a written review of every wrong and every guessed answer

Assessment: Unit 1 assessment: a timed multiple-choice section on using objects and methods, scored with the error log updated

PHASE 2: SELECTION, ITERATION AND CLASS CREATION (UNITS 2 AND 3, MONTHS 3-4, WEEKS 9-16)

The logic and loops that drive every program, then writing your own classes, which is exactly what the class-design free-response question demands.

Month 3: Selection and Iteration (Unit 2) (Weeks 9-12)

Boolean Expressions and if Statements

Topics:

- Boolean values and relational operators
- if, if-else and if-else-if chains
- Nested conditionals and reading them without getting lost
- Common logic errors and how the exam sets them
- Writing conditions that match a worded specification exactly
- Deciding between an if-else-if chain and separate ifs, and why it matters

Projects built:

- Decision engine: a grading or categorising program driven entirely by conditionals, tested against every boundary value

Practice: 14 conditional problems, each one required to be traced for at least three different inputs

Compound Conditions and De Morgan's Laws

Topics:

- The logical operators and, or and not
- Short-circuit evaluation and when the second condition is never checked
- De Morgan's laws and rewriting a condition as its equivalent
- Simplifying tangled boolean expressions
- Boolean-returning methods, a frequent exam and free-response pattern
- Truth-table thinking for questions that look harder than they are

Projects built:

- Condition simplifier: a set of gnarly boolean expressions rewritten in simplest equivalent form and verified by truth table

Practice: 12 compound-condition problems including 4 De Morgan rewrites checked both ways

While and For Loops

Topics:

- The while loop and its three moving parts
- The for loop as a compact while, and converting between them
- Counters, accumulators and running totals
- Sentinel-controlled loops
- Off-by-one errors and infinite loops, diagnosed on purpose
- Counting exactly how many times a loop body runs, a guaranteed multiple-choice type

Projects built:

- Loop patterns: programs that sum, count, find and build using loops, each with the iteration count predicted before running

Practice: 16 loop problems, half asking for the exact number of executions rather than the final value

Nested Loops and String Traversal

Topics:

- Nested loops and how the inner loop multiplies the work
- Traversing a String character by character with a loop
- Building patterns and tables with nested iteration
- Analysing nested-loop statement execution counts
- Combining loops with the conditionals from earlier weeks
- Recognising when a loop is the wrong tool

Projects built:

- Pattern generator: nested-loop programs that draw shapes and process text grids, with execution counts worked out by hand

Practice: 12 nested-loop and String-traversal problems, execution counts required on the nested ones

Assessment: Unit 2 assessment: a timed section on conditionals and loops, including statement-execution-count questions

Month 4: Class Creation (Unit 3) (Weeks 13-16)

Anatomy of a Class

Topics:

- Instance variables and the state an object holds
- Constructors and initialising an object correctly
- Default values and the null reference
- Creating objects with new and calling their methods
- The this reference and when you need it
- Reading a class specification the way a free-response question presents it

Projects built:

- First class from scratch: a small class with fields and a constructor, instantiated and exercised from a tester program

Practice: 10 problems building constructors and instance variables from a written specification

Methods, Encapsulation, and Access

Topics:

- Accessor and mutator methods done cleanly
- private versus public and why encapsulation is tested
- Returning and updating an object's state safely
- Writing methods that use the object's own instance variables
- Method contracts: honouring exactly what a specification promises
- Common class-design mistakes that cost free-response points

Projects built:

- Encapsulated class: a class with private fields and a full set of methods, with a test proving the state can only change through methods

Practice: 12 problems adding correct accessor, mutator and behaviour methods to given classes

Static, Scope, and Designing to a Specification

Topics:

- static variables and methods, and how they differ from instance members
- Local versus instance scope, a frequent source of exam confusion
- Designing a whole class from a table of required behaviour
- Choosing fields, constructors and methods that satisfy every listed case
- Testing your own class against the specification before trusting it
- How this maps directly onto free-response Question 2, class design

Projects built:

- Spec-to-class: a complete class built from a behaviour table alone, then checked case by case against that table

Practice: 10 class-design problems worked from specification to tested class, scope errors flagged as they appear

Unit 3 Mastery and the Class-Design Free Response

Topics:

- Writing a full class to specification under time
- The Class Design free-response question and its rubric
- Reading the specification table for exactly the points on offer
- Where partial credit lives in a class-design answer
- Self-checking an answer against the rubric before moving on
- Handwriting and clarity habits that protect marks on a digital exam

Projects built:

- Full class-design response written to a released-style prompt, then scored against the rubric point by point

Practice: Two timed class-design responses, each self-scored on the rubric with the lost points named

Assessment: Unit 3 assessment: a timed section plus one class-design free response, scored on the official rubric

PHASE 3: DATA COLLECTIONS AND EXAM MASTERY (UNIT 4 AND FULL MOCKS, MONTHS 5-6, WEEKS 17-24)

Arrays, ArrayList, 2D arrays, files and data sets, then a full turn to exam craft: every free-response type drilled and complete timed digital mock exams.

Month 5: Data Collections (Unit 4) (Weeks 17-20)

One-Dimensional Arrays

Topics:

- Declaring, creating and initialising arrays
- Indexing, length and the out-of-bounds error
- Traversing an array with a standard for loop
- The enhanced for loop and when it can and cannot be used
- Array algorithms: minimum, maximum, sum, count and average
- The array traps the exam reuses, including modifying while traversing

Projects built:

- Array analyser: methods that scan an array for statistics and matches, each tested on empty and single-element arrays

Practice: 14 array problems spanning both loop styles, edge cases required on every algorithm

ArrayList

Topics:

- ArrayList versus arrays, and why the exam tests both
- The tested methods: add, get, set, remove and size
- Autoboxing in an ArrayList of Integer
- Traversing and searching an ArrayList
- The removal-during-iteration bug and the two correct fixes
- ArrayList algorithms that mirror free-response Question 3, data analysis

Projects built:

- Data-analysis methods: an ArrayList processed to filter, count and summarise, written the way the free response asks

Practice: 14 ArrayList problems, including two built specifically around the remove-while-iterating trap

Two-Dimensional Arrays

Topics:

- Declaring and initialising 2D arrays
- Row-major order and what it means for traversal
- Nested loops over a grid, rows then columns
- 2D array algorithms: row sums, column sums, searching and counting
- Common indexing mistakes on the second dimension
- How this maps onto free-response Question 4, the 2D array question

Projects built:

- Grid processor: methods that analyse a 2D array by row, column and whole grid, each traced on a small sample grid first

Practice: 12 two-dimensional array problems, a hand trace required before coding each

solution

Files, Data Sets, and Recursion Tracing

Topics:

- Reading text files with Scanner, a topic added in the 2025-26 redesign
- Working with data sets: loading, storing and processing real rows of data
- Tracing recursive calls, which the current exam tests; you trace recursion, you do not write it
- Why inheritance, polymorphism and writing recursion are not on the redesigned exam, so we do not spend time on them
- Combining files, arrays and ArrayList into one small data program
- A clear map of exactly what Unit 4 does and does not require

Projects built:

- Mini data project: read a small text data set with Scanner, store it, and answer questions about it with array or ArrayList methods

Practice: 10 problems mixing file reading and data processing, plus 6 recursion-tracing questions that ask only for the traced result

Assessment: Unit 4 assessment: a timed section on arrays, ArrayList, 2D arrays, file reading and recursion tracing

Month 6: Free-Response Mastery and Full Mock Exams (Weeks 21-24)

Free Response 1 and 2: Methods, Control Structures and Class Design

Topics:

- Question 1, methods and control structures: writing two methods, or a constructor and a method, to spec
- Handling the String-method part cleanly in Question 1
- Question 2, class design, revisited under exam time
- Reading a rubric to see exactly where points are awarded
- The commonest point-losing mistakes on these two questions
- Writing code clean enough to be scored quickly and correctly

Projects built:

- Two timed responses, one Question-1 style and one Question-2 style, each scored against the official rubric

Practice: Timed free-response practice on Questions 1 and 2, every attempt self-scored with the lost points named

Free Response 3 and 4: ArrayList Data Analysis and 2D Arrays

Topics:

- Question 3, data analysis with an ArrayList, from spec to working method
- Question 4, the 2D array question, traversed and manipulated correctly
- The data-manipulation patterns these two questions reward
- Partial-credit strategy: securing points even when the full method is hard
- Time budgeting across all four free-response questions
- Checking a method against the given examples before you commit

Projects built:

- Two timed responses, one Question-3 style and one Question-4 style, scored against the rubric and compared to week 21

Practice: Timed free-response practice on Questions 3 and 4, with a written note on where partial credit was won or missed

Multiple-Choice Strategy and Full Mock Exam One

Topics:

- Pacing the forty-two-question digital multiple-choice section
- Fast, accurate code tracing under time
- Elimination and educated guessing when a trace stalls
- A first complete timed mock: multiple choice and free response together
- Running the mock under Bluebook-style digital conditions
- Scoring the mock and converting toward the 1 to 5 scale

Projects built:

- Full mock exam one, scored, with a written breakdown of multiple-choice versus free-response performance

Practice: The full timed mock plus a complete review, every missed multiple-choice question re-traced correctly

Full Mock Exam Two and Test-Ready Review

Topics:

- A second complete timed mock under exam conditions
- Comparing the two mocks for direction of travel
- Closing the last content gaps the mocks exposed
- The test-day digital routine: the Bluebook app, the Quick Reference, timing
- A realistic, evidence-based target score from your mock results
- Where to go next: college programming, data structures, or a first real project

Projects built:

- Full mock exam two, scored and reviewed, ending in a one-page test-day plan built from your own results

Practice: The second full mock plus targeted repair of any remaining weak topic

Assessment: Final assessment: a complete timed mock exam, a progress summary from week 1 to now, and certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A String toolkit built from only the AP-tested String methods
- An encapsulated class designed from a behaviour specification

- An array analyser tested on every edge case
- An ArrayList data-analysis method in the free-response style
- A 2D array grid processor
- A mini data project that reads a text data set with Scanner
- Eight scored free-response answers across all four question types
- Two full timed mock exams under digital conditions

Total projects: Dozens of small Java programs plus eight scored free responses and two complete timed mock exams, all mapped to the 2025-26 exam

SKILLS THE STUDENT WILL HAVE AT THE END

- Reading and writing real Java, not just recognising it
- Using objects and methods, the largest scored area of the redesigned exam
- Conditionals, loops and nested iteration with correct execution-count reasoning
- Designing and writing classes to a specification, the class-design free response
- Arrays, ArrayList, 2D arrays, file reading and recursion tracing under the new Unit 4
- Free-response point-earning and digital multiple-choice pacing on full timed mocks

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, writing and tracing Java code live rather than watching solutions scroll by

Practice: 4-5 hours weekly of coding problems, tracing exercises and, later, timed free-response and mock sections

Review: Homework and free responses reviewed with written feedback, scored against the official rubric so you see exactly where points are won and lost

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for the problems that stall you

Progress updates: Monthly progress notes tracking unit assessments and, in the final months, mock-exam scores against your target

Career guidance: Honest guidance on what an AP score does and does not do for college admissions and credit, and what to study next in computer science

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, awarded on finishing the course and its final timed mock exams

WHERE THIS CAN LEAD

- A strong footing for the AP Computer Science A exam and, through it, university applications
- Readiness for a college introductory programming course, often the exact content AP CS A mirrors
- A genuine first step toward a computer science, software engineering or data degree
- The Java and problem-solving base for our more advanced coding courses, including data structures and algorithms
- Programming and free-response habits that transfer to later coursework and technical interviews

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course taught to the new 2025-26 AP CS A exam?

A: Yes, and this is the single most important thing to check in any AP CS A course right now. The College Board redesigned the course for 2025-26: the old ten units became four, the exam moved fully onto the digital Bluebook platform, and the content changed. Inheritance, polymorphism, interfaces and writing recursive methods were removed. Reading text files with Scanner, working with data sets, and tracing recursive calls were added. We teach the four-unit course and the digital exam as they are now. A course or textbook still built around a separate inheritance unit is preparing you for an exam that no longer exists.

Q: Do I need to know Java or any programming before I start?

A: No. The course starts at the very first line of Java and builds up, so a complete beginner is exactly who it is designed for. If you have done some block coding or a little Python, that background helps you move faster, but nothing is assumed. What the course does need is the practice hours: programming is learned by writing code, and the four to five hours a week of practice are where the real progress happens, not only in class.

Q: What is the exam format now?

A: The redesigned exam has two sections of equal length. Section 1 is multiple choice: forty-two questions with answer choices A to D, ninety minutes, and it is worth fifty-five percent of the score. Section 2 is free response: four questions, ninety minutes, worth forty-five percent. The four free-response questions are methods and control structures, class design, data analysis with an ArrayList, and a 2D array question. The whole exam is

taken digitally in the Bluebook app, with the Java Quick Reference available on screen. We run full mock exams under these conditions so test day feels familiar.

Q: Will this course get me a 5, or college credit?

A: No honest course can promise a specific score. AP exams are graded 1 to 5, and your result depends on your starting point, the hours you put in and your performance on the day. What we can promise is honest: live teaching to the current exam, small batches, real doubt support, free-response drilling against the official rubric, and full timed mock exams, plus a clear, evidence-based read on a realistic target for you. Whether a given score earns college credit is decided by each university, not by us or by the College Board, so we will point you to check the specific policy of the colleges you care about.

Q: What happened to inheritance and recursion?

A: They changed in the redesign, and knowing this saves you real time. Inheritance, polymorphism, the extends and super keywords, interfaces, and writing recursive methods were all removed from the required content. Recursion is still on the exam, but only tracing: you follow recursive calls and state the result, you are not asked to write a recursive method. We teach recursion tracing properly, and we deliberately do not spend weeks on the removed topics, which is time we give back to the areas the exam actually tests, especially Unit 4 data collections, the largest scored unit.

Q: Can I manage this alongside school and other subjects?

A: Yes, it is built for a school student's week: two live one-hour classes plus four to five hours of practice you can place where they fit, with evening and weekend class slots. The pace is steady rather than crushing, and WhatsApp doubt support means a problem that stalls you on a school night does not have to wait for the next class. Students who miss a session can usually attend the same class with another batch where scheduling allows, so a busy exam week does not leave a gap.

Q: How is the free response practised and marked?

A: The way it is actually scored. From month 4 onward you write timed free-response answers to released-style prompts, and each one is marked against the official College Board rubric, point by point, so you see exactly where marks are earned and lost. By the final month you will have written and reviewed several answers across all four question types. This rubric-first habit is what separates students who understand Java from students who reliably score on the free-response section, and the two are not the same thing.

Q: Is this course affiliated with the College Board?

A: No. Modern Age Coders is an independent education provider and is not affiliated with, authorised by, or endorsed by the College Board. AP and Advanced Placement are registered trademarks of the College Board, used here only to describe the exam this course prepares students for. For official course and exam information, students should always refer to the College Board and AP Central directly.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ap-computer-science-a-java-exam-prep-course>

=====

COURSE 18 OF 118: AP COMPUTER SCIENCE PRINCIPLES: CREATE TASK AND EXAM PREP

=====

Course link: <https://learn.modernagecoders.com/courses/ap-computer-science-principles-exam-prep-course>

Subject area: Advanced Placement Computer Science

Designed for: Teens

Who can join: High school students preparing for AP Computer Science Principles; beginner-friendly, no prior coding required

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 3-4 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online AP Computer Science Principles course: programming fundamentals, data, algorithms, the Create performance task, and full digital exam prep for CSP.

In one line: A broad, honest introduction to computing, built for the exam and the Create task exactly as they exist right now.

ABOUT THE PROGRAMME

AP Computer Science Principles is deliberately the wide-angle course: instead of teaching one language deeply the way AP Computer Science A teaches Java, CSP is organised around five Big Ideas, Creative Development, Data, Algorithms and Programming, Computer Systems and Networks, and Impact of Computing, and it is language-flexible by design. Students may build in a block-based or a text-based language, and this course teaches with a beginner-friendly,

text-based language while also teaching fluent reading of the College Board's own pseudocode, since exam questions are written in that pseudocode regardless of what a student builds with. What matters on the exam is the concept underneath, a loop, a list, a procedure with parameters, not which keyword a particular language happens to use for it. The course is also built for the assessment as it currently works, which changed in 2024 in a way many older courses have not caught up to. Students still spend at least nine hours of guided development time building an individually developed program with a list, a procedure that takes parameters, and an algorithm that manages complexity with selection or iteration, the Create Performance Task, and they still submit their code as a Personalized Project Reference PDF through the AP Digital Portfolio near the end of April. What changed is that this submission is no longer scored on its own by AP readers. Instead, on the end-of-course exam in May, students answer two written-response questions about their own program using that same Personalized Project Reference as their permitted reference material. Section I, seventy multiple-choice questions in one hundred twenty minutes, is worth seventy percent of the score. Section II, the two written responses about the student's own code, is worth thirty percent. The six-month structure follows this directly: Months 1 and 2 build Creative Development and core programming; Months 3 and 4 cover Data and deeper Algorithms and Programming while the Create Task is planned, built, and finished; Months 5 and 6 cover Computer Systems and Networks and Impact of Computing, then turn fully to full-length multiple-choice practice and drilling the two written responses against the student's own finished project, ending in complete timed mock exams under digital conditions.

WHAT MAKES THIS COURSE DIFFERENT

- Taught to the exam as it is scored now: the 2024 change to the Create Performance Task, no longer a separately scored artifact but the reference material for two written-response questions on the exam itself, is built into the course from month one
- Genuinely language-flexible teaching: concepts are taught to transfer, and students also learn to read the College Board's own pseudocode fluently, since that is the language most multiple-choice questions are actually written in
- A real, individually developed Create Task, built over guided sessions across month 4 with instructor feedback, not assembled the week before the deadline
- All five Big Ideas covered in proportion to how the exam actually weights them, so time is not lost over-teaching one area and under-teaching another
- Written-response practice done the way the exam requires it: against the student's own finished program and their own Personalized Project Reference, not a generic prompt
- Live, small batches where every class involves writing or reading real code, not lecture slides watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Creative Development and programming foundations: variables, lists, selection, iteration, and procedures, built alongside the habits of incremental development, testing, and choosing a Create Task idea

Phase 2: Data and deeper Algorithms and Programming, while the Create Performance Task itself is planned, built across guided sessions, tested, and finished with its Personalized Project Reference

Phase 3: Computer Systems and Networks and Impact of Computing, then full seventy-question multiple-choice practice, written-response drilling against your own code, and complete timed digital mock exams

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable school-level arithmetic and logical reasoning. No calculus or advanced maths is required for AP Computer Science Principles

Programming: None required. The course is beginner-friendly and builds programming concepts from the first class; CSP is intentionally language-flexible, and no prior experience is assumed

Equipment: A computer that can run our teaching language and a modern browser, since the real exam and our mock exams use a digital testing environment. We help with the full setup in week 1

Audience: High school students preparing for AP Computer Science Principles, typically as their first AP computer science course, whether taking it at school or self-studying

WHO THIS COURSE IS FOR

Ap students: Students taking AP Computer Science Principles at school who want a structured, current course and real code review alongside their class

Self studiers: Students whose school does not offer AP CSP and who are preparing for the exam independently

First ap course: Students choosing CSP as their first AP computer science course, since it is broader and more forgiving of a standing start than AP Computer Science A

Curious beginners: Students who are not yet sure programming is for them and want an honest, low-pressure introduction to what computing actually involves

International students: International students sitting AP for university applications who need the exam taught to its current, digital, two-part format

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CREATIVE DEVELOPMENT AND PROGRAMMING FOUNDATIONS (BIG IDEA 1 AND CORE OF BIG IDEA 3, MONTHS 1-2, WEEKS 1-8)

Computational thinking, the core programming building blocks CSP tests, and the habits of incremental development, ending with a planned and approved Create Task idea.

Month 1: Computational Thinking and Programming Building Blocks (Weeks 1-4)

Course Map, Computational Thinking, and What CSP Actually Tests

Topics:

- What AP Computer Science Principles covers: the five Big Ideas, Creative Development, Data, Algorithms and Programming, Computer Systems and Networks, and Impact of Computing, and how this course maps to them
- How CSP differs from AP Computer Science A: a broad, language-flexible course about computing itself, not a single-language programming course
- The two-part assessment at a glance: a seventy-question multiple-choice exam and a written-response section built around a program you build yourself
- Computational thinking as a habit: decomposition, abstraction, and pattern recognition, not just vocabulary to memorise
- A first look at the language and environment we build in, and why the concept matters more than the exact keyword a language uses for it
- Setting up your development environment and a running course notebook, where every future Create Task idea starts getting written down

Projects built:

- First program: a short interactive program that takes input and produces output, run and explained line by line

Practice: 10 short exercises identifying which Big Idea a given exam-style question is testing, plus a first working program written and run

Variables, Data Abstraction, and Expressions

Topics:

- Variables as named, reusable storage, and why naming them clearly is a skill the exam rewards directly
- Data types CSP cares about: numbers, text, and Boolean values, and how the College Board's own pseudocode represents each
- Assignment versus comparison, and expressions that combine arithmetic and logic correctly
- Data abstraction: hiding detail behind a clear name so a program stays readable as it grows
- Lists as CSP's core data structure: creating a list and accessing an item by its index
- Reading and predicting the output of short CSP-style pseudocode blocks, the single most common multiple-choice question shape

Projects built:

- Variable tracker: a short program that stores, updates, and reports several related values, traced by hand before it is run

Practice: 14 pseudocode-tracing problems written to the CSP Exam Reference Sheet conventions, each paired with equivalent code in our teaching language

Lists and Iteration

Topics:

- Traversing a list with a loop, the single most tested list pattern on the exam
- FOR EACH style iteration versus counting loops, and reading both forms in CSP pseudocode
- Building a list up over time: appending, inserting, and removing items
- Searching a list: finding a matching value and correctly handling the case where none exists
- Common list bugs: off-by-one errors and changing a list while iterating over it
- Using a list to model something real, such as scores, names, or sensor readings

Projects built:

- List toolkit: procedures that sum, count, filter, and find the maximum of a list, tested on an empty list and a single-item list

Practice: 12 list and iteration problems, half written in CSP pseudocode and translated, half written directly in code

Selection, Boolean Logic, and Procedures

Topics:

- Boolean expressions and the relational operators the exam tests
- IF, IF-ELSE, and nested selection, read in both CSP pseudocode and our teaching language
- AND, OR, and NOT, and reasoning carefully through compound conditions
- Defining a procedure with parameters and a return value
- Why breaking a program into procedures is itself a Creative Development skill, not just tidy style
- Combining selection, iteration, and procedures into one small, working program

Projects built:

- Decision-making procedure: a small program that classifies input using nested conditionals, tested against every boundary case

Practice: 16 problems mixing selection and procedure tracing, pitched at the same difficulty as the exam's code-reading questions

Assessment: Month 1 checkpoint: a timed set covering variables, lists, selection, and procedures, scored the way Section I multiple choice is scored

Month 2: Algorithms, Procedural Abstraction, and Planning the Create Task (Weeks 5-8)

Algorithms: Combining Sequencing, Selection, and Iteration

Topics:

- What an algorithm means in CSP's terms: a precise sequence of steps that solves a problem or

- a whole class of problems
- Combining sequencing, selection, and iteration together to express a real algorithm
- Simulating an algorithm by hand on paper before trusting it in code
- Search: a first look at how a program finds something inside a list
- Sort: the idea of putting data in order, without needing to memorise every sorting method by name
- Recognising an algorithm's purpose from CSP-style pseudocode alone, a guaranteed multiple-choice skill

Projects built:

- Algorithm workbook: a search and a sort implemented from a plain-language description, each hand-traced on a small list first

Practice: 12 algorithm-tracing problems in CSP pseudocode, each requiring a predicted output before running the equivalent code

Procedural Abstraction and Parameters

Topics:

- Procedural abstraction: naming a block of steps so it can be reused without re-reading it every time
- Parameters as a procedure's inputs, and return values as its output
- Building a program from smaller, individually tested procedures instead of one long block
- Libraries and built-in procedures: using code you did not write, correctly and with understanding
- How the CSP Exam Reference Sheet documents procedures, and reading a procedure header quickly
- Documenting what a procedure does so a reader does not have to trace it line by line to understand it

Projects built:

- Procedure library: a small set of reusable procedures for a chosen task, called from one main program and tested independently

Practice: 10 problems designing a procedure header from a plain-language specification, plus 4 tracing problems on procedure calls

The Development Process: Incremental Building, Testing, and Collaboration

Topics:

- Incremental and iterative development: build a little, test a little, the exact process Creative Development questions test
- Identifying and fixing errors, and the real difference between a syntax error and a logic error
- Testing with a range of inputs, including edge cases that break naive solutions
- Collaboration in program development, and describing working with others honestly the way CSP expects
- Program documentation: comments that explain why a choice was made, not only what a line does
- Keeping a running development log, the exact habit month 4 relies on for the Create Task

Projects built:

- Debug and repair: a deliberately broken program fixed step by step, with every bug and its fix logged in writing

Practice: 8 debugging exercises with planted logic errors, plus a short written reflection on the development process used

Choosing and Planning Your Create Task Idea

Topics:

- What the Create Performance Task actually requires: an individually developed program with at least one list, a procedure that takes parameters, and an algorithm using selection or iteration to manage complexity
- Choosing an idea you can build and explain in your own words, not one that looks impressive but that you cannot honestly walk through
- Scoping a project to roughly nine hours of hands-on development time, the College Board's own minimum
- Sketching the program's purpose, inputs, and outputs before writing a line of code
- Planning exactly where the required list, procedure, and algorithm will live in your design
- Instructor sign-off on your Create Task idea before month 3's Data content and month 4's build sessions begin

Projects built:

- Approved Create Task proposal: a one-page plan naming your program's purpose, its list, its procedure with parameters, and its core algorithm

Practice: A written self-check against the official Create Task requirements, item by item, before the idea is approved

Assessment: Unit assessment: a timed multiple-choice set on Creative Development and the core programming content, reviewed question by question

PHASE 2: DATA, DEEPER ALGORITHMS, AND BUILDING THE CREATE TASK (BIG IDEAS 2 AND 3, MONTHS 3-4, WEEKS 9-16)

How computers represent and use data, then a return to algorithms with more complex data, while the Create Performance Task itself is planned, built across guided sessions, and finished.

Month 3: Data and Information (Weeks 9-12)

Binary and How Computers Represent Data

Topics:

- Binary numbers: converting between binary and decimal by hand
- Why computers store everything, numbers, text, images, and sound, as binary underneath
- Bits, bytes, and the sizes CSP expects a student to reason about

- Representing text with character encodings, and images as grids of coloured bits
- Analog versus digital data, and what is gained and lost when something is digitised
- Reading CSP multiple-choice questions that test binary conversion directly

Projects built:

- Binary explorer: a program that converts numbers between binary and decimal and explains each step of the conversion

Practice: 14 binary conversion problems plus 6 conceptual questions on the digital representation of text, image, and sound

Data Compression

Topics:

- Lossless compression: preserving every bit of the original data on the way back out
- Lossy compression: trading some detail for a much smaller file
- Where each type is actually used, and why source code and financial data are never compressed lossily
- A simple compression scheme worked by hand to see the size trade-off directly
- Metadata: what it is and why it travels with compressed and uncompressed data alike
- Compression questions the way CSP tends to ask them: scenario-based, not formula-based

Projects built:

- Compression demo: a small program that compresses a repetitive data set with a simple scheme and reports the size saved

Practice: 10 scenario questions on choosing lossless versus lossy compression for a stated real-world use

Extracting Information from Data

Topics:

- Why data alone is not information, and the extraction step that sits between the two
- Filtering, sorting, and searching a large data set for a specific answer
- Combining two data sets to surface a relationship that neither shows on its own
- Using tools to explore a data set too large to read by hand
- Visualising data honestly, echoing the same cautions taught in our data-focused courses
- Drawing a defensible conclusion from a data set, and stating plainly what that conclusion cannot claim

Projects built:

- Data-set investigation: a provided data set explored with filtering and grouping to answer three written questions with cited evidence

Practice: 8 data-extraction problems on real-style tables, each answer required to cite which rows or columns support it

Using Data to Solve Problems and Simulations

Topics:

- Simulations as simplified models of a real or imagined process
- What a simulation can tell you, and what it cannot, since it is only ever a model
- Building a simple simulation and running it many times to see a pattern emerge
- Bias in data, and bias in the people who select or interpret it
- How Data-focused questions typically read on the exam: a short scenario, then a reasoning question
- Pulling the month together: binary, compression, extraction, and simulation as one connected story about data

Projects built:

- Simple simulation: a small program that models a repeated random process, for example a game of chance, and reports results across many runs

Practice: A timed set of Data-focused multiple-choice questions, reviewed with the reasoning written out for every miss

Assessment: Unit assessment: a timed multiple-choice section on data representation, compression, extraction, and simulation

Month 4: Robust Programs and Building the Create Task (Weeks 13-16)

Two-Dimensional Lists and Algorithmic Thinking

Topics:

- Lists of lists: representing a grid or a table of data
- Traversing a two-dimensional list with nested iteration
- Choosing a data structure that fits the problem, not just the one you already know best
- A first, intuitive look at algorithm efficiency: why some approaches slow down badly as the data grows
- Undecidable problems: an honest look at problems no algorithm can solve correctly for every possible input
- Reading exam questions that compare two different algorithms solving the same problem

Projects built:

- Grid processor: a program that stores and analyses a small two-dimensional data set, for example a seating chart or a simple game board

Practice: 10 two-dimensional list problems plus 4 short-answer questions comparing the efficiency of two approaches to the same task

Simulations, Abstraction, and Managing Complexity

Topics:

- Abstraction as the tool that lets a program grow without becoming unreadable
- Managing complexity with procedures, parameters, and well-named variables used together
- Building a slightly larger simulation than week 12's, with more moving parts
- Deciding what detail a simulation can safely leave out without breaking its usefulness
- Refactoring a working but messy program into a cleaner one without changing what it does
- Applying these same habits directly to your own Create Task program, starting this week

Projects built:

- Create Task build session one: the first working version of your program's core algorithm and required list, reviewed live with the instructor

Practice: A self-review of your in-progress Create Task code against the list, procedure, and algorithm requirements

Building the Create Task: Robust Programs and Instructor Feedback

Topics:

- Writing a procedure with parameters that genuinely does the work your program needs, not one added only to meet a checklist
- Testing your Create Task program against both typical and edge-case inputs
- Handling unexpected input without the program crashing, a real mark of a robust program
- Explaining your own code out loud in class, the exact skill the exam later asks for in writing
- Incorporating instructor feedback into a second, improved working version
- Keeping your development log current: what changed, why it changed, and what you tested afterward

Projects built:

- Create Task build session two: a tested, working version of your full program, incorporating the feedback from session one

Practice: A written walkthrough of your program explaining what your list, procedure, and algorithm each do and why you built them that way

Completing the Create Task and the Personalized Project Reference

Topics:

- Finalising your Create Task program: last fixes, clear comments, and a clean final run
- Producing your Personalized Project Reference, the PDF of your own code you are permitted to use on exam day
- What changed in 2024: the Create Task is no longer submitted as a separately scored artifact; instead your Personalized Project Reference becomes the material you use on the exam itself to answer two written-response questions about your own program
- Submitting through the AP Digital Portfolio, and the real deadline this falls near, around the end of April on a typical school-year calendar
- A first practice attempt at explaining your program in writing, the same task the exam itself will ask for
- Why building a program you can actually explain matters more than building one that merely runs without errors

Projects built:

- Completed Create Task: a finished, tested program with its Personalized Project Reference PDF prepared exactly as exam day requires

Practice: Two practice written responses about your own program, self-checked against the kind of reasoning the real prompts reward

Assessment: Create Task checkpoint: instructor review of the finished program, its documentation, and a first written explanation of your own code

----- PHASE 3: COMPUTER SYSTEMS, IMPACT OF COMPUTING, AND EXAM MASTERY (BIG IDEAS 4 AND 5 AND FULL PRACTICE, MONTHS 5-6, WEEKS 17-24) -----

How the internet and larger computing systems actually work, an honest look at computing's effects on real people, then a full turn to exam craft built around your own finished Create Task.

Month 5: Computer Systems and Networks (Weeks 17-20)

How the Internet Works

Topics:

- The internet as a network of networks, not a single thing that lives anywhere
- IP addresses and how a message finds its way to the right destination
- Packets: breaking a message into pieces that can travel by different paths
- Protocols as the agreed rules that let different systems talk to each other reliably
- Bandwidth and latency, and why a fast connection is not the same thing as a low-delay one
- Reading a CSP question about internet infrastructure for the exact concept being tested

Projects built:

- Packet-trace exercise: a written walkthrough of how a message would be broken into packets, routed, and reassembled between two points

Practice: 10 conceptual questions on IP addressing, packets, and protocols, each answered in one or two precise sentences

Fault Tolerance and Parallel Computing

Topics:

- Redundancy: why the internet keeps working even when a piece of it fails
- Fault tolerance as a deliberate design goal, not an accident of good luck
- Sequential versus parallel computing, and where each is genuinely the right choice
- Distributed computing: splitting one large problem across many machines
- Speedup, and why doubling the number of machines rarely halves the time exactly
- Scenario questions that ask you to identify a fault-tolerant or a parallel design

Projects built:

- Parallel-versus-sequential demo: a small program run two ways, timed and compared, with the result explained in writing

Practice: 8 scenario questions distinguishing genuinely fault-tolerant designs from a single point of failure

Cybersecurity Fundamentals

Topics:

- What makes a system a worthwhile target: the value of the data it holds, not just its size

- Common attacks explained plainly: phishing, malware, and denial-of-service, at the level CSP tests
- Symmetric encryption: one shared key, and where that approach breaks down
- Public-key encryption: how two sides can agree on a secret without ever meeting first
- Authentication: passwords, multi-factor methods, and why each adds a genuine barrier
- Practical safe-computing habits worth keeping regardless of the exam

Projects built:

- Encryption walkthrough: a simple substitution-style cipher implemented and then explained alongside why real systems need something far stronger

Practice: 10 cybersecurity scenario questions, each requiring the specific vulnerability or protection to be named

Computer Systems Review and Practice

Topics:

- Pulling the month together: the internet, fault tolerance, parallel computing, and cybersecurity as one connected system
- Common traps in this area: mixing up a protocol with a specific piece of hardware
- Practising the reading-passage multiple-choice questions, five of the seventy on the real exam
- Timing strategy for the systems and networks questions specifically
- Building a personal glossary of systems terms from the errors made this month
- A short look ahead at Impact of Computing, which closes out the exam's content

Projects built:

- Systems glossary: a personal reference of every systems term missed this month, defined in your own words

Practice: A timed multiple-choice set on computer systems and networks content only, reviewed with every miss traced to its specific concept

Assessment: Unit assessment: a timed multiple-choice section on computer systems and networks, including reading-passage-style questions

Month 6: Impact of Computing and Full Exam Mastery (Weeks 21-24)

Beneficial and Harmful Effects of Computing

Topics:

- Computing innovations as double-edged: the same tool serving different purposes for different people
- The digital divide: unequal access to computing, and what that actually costs the people on the wrong side of it
- Bias in algorithms: where it enters, from the data going in to the decisions coming out
- Case studies of real computing innovations, weighed honestly for who benefits and who is put at risk
- Safe computing practices: privacy settings, a personal data footprint, and informed consent
- Writing a balanced answer that names both a genuine benefit and a genuine risk, the exact shape these questions want

Projects built:

- Impact case study: one real computing innovation analysed in writing for its benefits, its risks, and who is affected by each

Practice: 6 short-answer questions each requiring one genuine benefit and one genuine harm for a given computing innovation

Intellectual Property, Open Source, and Full Multiple-Choice Practice

Topics:

- Intellectual property: copyright, and why code can be protected the same way other creative work is
- Open source and Creative Commons licensing, and how each differs from default copyright
- Crowdsourcing and collaborative projects as a computing-enabled way of working
- Legal and ethical questions CSP expects you to reason through, not just define
- Starting full-length, seventy-question multiple-choice practice sets under timed conditions
- Building the error log that will drive the final two weeks of review

Projects built:

- First full multiple-choice practice set: seventy questions in one hundred twenty minutes, scored and logged by Big Idea

Practice: The full timed multiple-choice set plus a written review of every question missed, grouped by which Big Idea it tested

Written-Response Drilling with the Personalized Project Reference

Topics:

- The written-response section in full: two required questions about your own program, answered using your Personalized Project Reference
- What each written-response question actually asks for, and how it is scored
- Practising with your own finished Create Task code, exactly as the real exam requires
- Multiple-choice pacing refined: fifty-seven single-select, eight multiple-select choose-two, and five reading-passage questions in the mix
- Strategies for the multiple-select questions specifically, since a partial answer does not earn partial credit there
- A second full multiple-choice practice set, timed, with pacing tracked question by question

Projects built:

- Two scored written responses about your own Create Task program, written exactly as the real exam requires and reviewed against the reasoning the rubric rewards

Practice: The second full timed multiple-choice set, plus two written-response drafts scored and revised

Full Mock Exam and Test-Ready Review

Topics:

- A complete timed mock exam: seventy multiple-choice questions plus both written-response questions, run under digital conditions
- Reviewing the mock section by section, multiple choice against written response
- Closing the last content gaps the mock exposed
- The test-day routine: the Personalized Project Reference on hand, digital exam navigation, and timing across both sections
- An honest, evidence-based look at where you stand across all five Big Ideas
- Where to go next: AP Computer Science A, a first real coding course, or building further on the Create Task itself

Projects built:

- Full mock exam, scored across both sections, ending in a one-page test-day plan built from your own results

Practice: The complete mock exam plus targeted repair of the single weakest Big Idea it revealed

Assessment: Final assessment: a complete timed mock exam covering both the multiple-choice and written-response sections, plus a progress summary from week 1 to now

PROJECTS THE STUDENT BUILDS AND KEEPS

- A list toolkit of procedures tested on an empty list and a single-item list
- An algorithm workbook implementing a search and a sort from plain-language descriptions
- A binary explorer that converts and explains number representations
- A data-set investigation answering written questions with cited evidence
- A finished, tested Create Performance Task program with its own required list, procedure, and algorithm
- A Personalized Project Reference prepared exactly as exam day requires
- An impact case study weighing a real computing innovation's benefits and risks
- A complete timed mock exam covering multiple choice and both written responses

Total projects: Dozens of small programs plus a completed Create Performance Task, an impact case study, two scored written responses, and a full timed mock exam, all mapped to the current CSP exam

SKILLS THE STUDENT WILL HAVE AT THE END

- Reading and writing real code built around variables, lists, selection, iteration, and procedures
- Designing, building, testing, and explaining an individually developed program, the exact demand of the Create Performance Task
- Reasoning about data: binary representation, compression, and drawing defensible conclusions from a data set
- Understanding how the internet, fault tolerance, and parallel computing actually work, at the depth the exam tests
- Weighing the beneficial and harmful effects of a computing innovation in writing, including bias and the digital divide
- Multiple-choice pacing and written-response habits drilled on full timed mock exams

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building and discussing real programs rather than watching slides

Practice: 3-4 hours weekly of coding exercises and pseudocode tracing, plus dedicated Create Task build time from month 4 onward

Review: Homework, the Create Task, and written responses reviewed with written feedback, scored the way the real exam scores each section

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked explanations for the problems that stall you

Progress updates: Monthly progress notes tracking unit assessments, Create Task milestones and, in the final month, mock-exam scores

Career guidance: Honest guidance on what an AP score does and does not do for college admissions and credit, and what to study next in computer science

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, awarded on finishing the course, the Create Task, and the final timed mock exam

WHERE THIS CAN LEAD

- A strong footing for the AP Computer Science Principles exam and, through it, university applications
- A natural next step into AP Computer Science A, our Java-based course, for students who want to go deeper into a single language
- A genuine first exposure to computing broad enough to inform a computer science, data, or engineering degree choice
- The programming and problem-solving base for our other coding courses, including Python and

web development

- A completed, explainable programming project and an honest sense of what computing work actually involves, useful well beyond the exam itself

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How is AP Computer Science Principles different from AP Computer Science A?

A: CSP is the broad, conceptual course. It covers five Big Ideas, Creative Development, Data, Algorithms and Programming, Computer Systems and Networks, and Impact of Computing, and it is language-flexible, so students may work in a block-based or a text-based language. AP Computer Science A, by contrast, is a single-language course taught entirely in Java, with no data, internet, or impact content. Many students take CSP first, since it needs no prior coding background and gives an honest sense of whether programming and computing are worth going deeper into with AP Computer Science A afterward.

Q: Do I need any coding experience before I start?

A: No. The course is built to be beginner-friendly from the first class, and CSP itself is designed for students with no programming background. We teach the concepts, variables, lists, procedures, selection, and iteration, from scratch, alongside the College Board's own pseudocode conventions used in most exam questions. Some prior exposure to any language, block-based or text-based, helps you move faster, but nothing is assumed.

Q: What is the Create Performance Task, and how is it assessed now?

A: You submit your code as a Personalized Project Reference PDF through the AP Digital Portfolio, with a deadline that falls near the end of April on a typical school-year calendar. What changed starting with the 2024-25 exam is how this is scored: the Create Task is no longer graded as a standalone submitted artifact by AP readers. Instead, on the end-of-course exam itself, you answer two written-response questions about your own program, using your Personalized Project Reference as permitted reference material. This written-response section is Section II of the exam and is worth thirty percent of your score. We build and finish your Create Task across guided sessions in month 4, then drill exactly this written-response format against your own finished code in month 6.

Q: What programming language will I use?

A: CSP is intentionally language-flexible: the College Board allows both block-based and text-based languages for the Create Task, and does not test any one language's exact syntax on the multiple-choice exam. We teach with a beginner-friendly, text-based language for building real programs, including your Create Task, and we also teach fluent reading of the College Board's own pseudocode, since that is the notation most multiple-choice questions are actually written in.

Q: What is the exam format?

A: The exam has two sections, taken digitally. Section I is multiple choice: seventy questions in one hundred twenty minutes, worth seventy percent of the score. Section II is the written-response section: two questions about your own Create Task program, answered using your Personalized Project Reference, worth thirty percent of the score. We run full timed practice on both sections, including complete mock exams under digital conditions.

Q: Will this course get me a 5, or college credit?

A: No honest course can promise a specific score. AP exams are graded 1 to 5, and your result depends on your starting point, the hours you put into practice, and your performance on the day. What we can promise is honest: live teaching to the current two-part exam, a real, individually developed Create Task built with instructor feedback, full multiple-choice and written-response practice, and complete timed mock exams, plus a clear, evidence-based read on a realistic target for you. Whether a given score earns college credit is decided by each university individually, not by us or by the College Board, so check the specific policy of the colleges you care about.

Q: Can I manage this alongside school and other subjects?

A: Yes, it is built for a school student's week: two live one-hour classes plus three to four hours of practice you can place where it fits, with evening and weekend class slots. The pace stays steady rather than crushing, and WhatsApp doubt support means a problem that stalls you on a school night does not have to wait for the next class. Students who miss a session can usually attend the same class with another batch where scheduling allows, so a busy exam week does not leave a real gap.

Q: Is this course affiliated with the College Board?

A: No. Modern Age Coders is an independent education provider and is not affiliated with, authorised by, or endorsed by the College Board. AP and Advanced Placement are registered trademarks of the College Board, used here only to describe the exam this course prepares students for. For official course and exam information, students should always refer to the College Board and AP Central directly.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ap-computer-science-principles-exam-prep-course>

=====

COURSE 19 OF 118: AP PRECALCULUS: UNITS 1 TO 3 AND FULL EXAM PREP

=====

Course link: <https://learn.modernagecoders.com/courses/ap-precalculus-exam-prep-course>

Subject area: Advanced Placement Mathematics

Designed for: High school

Who can join: High school students preparing for AP Precalculus; a solid Algebra 2 background is assumed

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 4-5 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online AP Precalculus exam prep covering Units 1-3: functions, exponentials, trig and polar. Small batches, real teachers, full timed mock exams.

In one line: A function-fluency course that happens to end in an AP exam, concentrated exactly where the exam is scored: Units 1 to 3.

ABOUT THE PROGRAMME

AP Precalculus is the newest AP mathematics course, and it is tested in a specific, slightly unusual way that changes how it should be prepared for. The classroom course covers four units, but the exam assesses only Units 1, 2 and 3. Unit 4, functions involving parameters, vectors and matrices, is taught across a full school year but does not appear on the exam at all. A prep course that spends real weeks drilling Unit 4 for exam purposes is spending time a student does not get back. This course concentrates its exam preparation exactly where the College Board tests: polynomial and rational functions, exponential and logarithmic functions, and trigonometric and polar functions. The six-month structure follows the three tested units in order, with the later weeks given fully to exam craft once each unit's content is covered. Months 1 and 2 build Unit 1: rates of change, polynomial behaviour and zeros, rational functions with their asymptotes and holes, transformations, and choosing the right model for a data set. Months 3 and 4 build Unit 2: arithmetic and geometric sequences, exponential functions and modelling, function composition and inverses, and logarithms through to equation solving. Month 5 builds Unit 3: the unit circle, sinusoidal functions and modelling, tangent and the reciprocal trig functions, inverse trig, and polar coordinates and functions. Month 6 turns fully to exam mastery: the calculator versus no-calculator skill that decides pacing on both sections, free-response method drilled against real question types, and two full timed mock exams run under the hybrid digital format, multiple choice answered in the Bluebook app and free response handwritten in paper booklets. A short, clearly labelled preview of Unit 4 closes the final week for students continuing on to calculus, with the honest note attached that it is not assessed on this exam. Every class is live and small, and the exam's calculator boundary, no calculator in Multiple Choice Part A, calculator required in Part B, calculator required again in Free Response Part A, then no calculator in Part B, is treated as a skill in its own right and drilled on both sides from week one, not left to the final month.

WHAT MAKES THIS COURSE DIFFERENT

- Exam prep concentrated exactly where the College Board tests: Units 1, 2 and 3. Unit 4, functions involving parameters, vectors and matrices, is taught in the full course but is not assessed on the exam, and this course does not spend exam-prep weeks pretending otherwise
- The calculator and no-calculator boundary is taught as its own skill from week one: Multiple Choice Part A with no calculator and Part B with one, Free Response Part A with a calculator and Part B without, drilled deliberately on both sides of the line
- A real function-fluency course, not a formula-memorisation cram sheet: transformations, end behaviour, equivalent representations and modelling are practised until they are habits, because that fluency is exactly what AP Calculus assumes on day one
- Free-response method drilled against real question types: modelling justification, equation solving and graph reading, with partial credit understood the way the rubric actually awards it
- Hybrid digital exam readiness: multiple-choice pacing for the Bluebook app and genuine handwriting practice for the paper free-response booklets, so the test-day format itself holds no surprises
- Live, small batches where problems are worked and reasoning is checked in class, not lecture videos watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Unit 1: rates of change, polynomial functions and zeros, rational functions and asymptotes, transformations, and choosing the right function model

Phase 2: Unit 2: arithmetic and geometric sequences, exponential functions and modelling, function composition and inverses, logarithms and equation solving

Phase 3: Unit 3 and exam mastery: the unit circle, sinusoidal and trigonometric functions, polar coordinates and functions, then the calculator/no-calculator skill, free-response method, and full timed hybrid-digital mock exams

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: A solid Algebra 2 background: comfort with function notation, factoring, exponents and basic graphing. This course does not re-teach Algebra 2 from scratch, though real gaps are patched as they surface

Programming: None required and none used. This is a pure mathematics course; no coding background is assumed or needed, even though Modern Age Coders also teaches programming

Equipment: A graphing calculator of the kind allowed on the AP exam, for Multiple Choice Part B and Free Response Part A, plus a notebook for the no-calculator hand-work, and a device with stable internet for live classes. Since the real exam and our mock exams use the Bluebook digital platform, we help with the setup in week 1

Audience: High school students preparing for AP Precalculus, typically in the year before AP Calculus, whether taking the course at school or preparing independently

WHO THIS COURSE IS FOR

Ap students: Students taking AP Precalculus at school who want a structured, exam-focused course and real problem-solving alongside their class

Self studiers: Students whose school does not offer AP Precalculus and who are preparing for the exam independently

Score focused: Students aiming for a strong score who want free-response drilling against a real rubric and full timed mock exams under hybrid-digital conditions

Calculus bound: Students headed for AP Calculus AB or BC who want genuine function fluency, not just exam tricks, since that fluency is exactly what calculus assumes

International students: International students sitting AP exams for university applications who need Units 1 to 3 taught to the current hybrid digital format

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: POLYNOMIAL AND RATIONAL FUNCTIONS (UNIT 1, MONTHS 1-2, WEEKS 1-8) -----

Rates of change, polynomial behaviour and zeros, rational functions with their asymptotes and holes, transformations, and model selection: the function-behaviour toolkit every later unit builds on.

Month 1: Rates of Change and Polynomial Behaviour (Weeks 1-4)

Functions, Change, and Rates of Change

Topics:

- Function notation, domain and range refreshed from Algebra 2, plus a first look at the Bluebook digital exam environment
- Change in tandem: reading how one quantity's change drives another's from a table of values
- Average rate of change as the slope of a secant line over a stated interval
- Comparing rates of change across different intervals to describe where a function is increasing or decreasing
- Using a second layer of differences to notice when a rate of change is itself changing, an early signal of concavity
- Reading the same function's behaviour from a table, a graph and an equation, and checking that the three agree

Projects built:

- Rate-of-change notebook: a function studied three ways, table, graph and average rate of change over five different intervals, with each answer interpreted in a sentence

Practice: 16 rate-of-change problems worked from tables and graphs, half requiring the interval to be chosen from a written description

Polynomial Functions: Zeros and Local Behaviour

Topics:

- Polynomial vocabulary: degree, leading coefficient and standard form, read at a glance
- Zeros of a polynomial function and what multiplicity does to the graph at each one
- Distinguishing a zero where the graph crosses the axis from one where it only touches
- Local maxima and minima, and intervals of increase and decrease read from a polynomial's graph
- Complex zeros and the fact that non-real zeros of a real polynomial always come in conjugate pairs
- Sketching a polynomial from its zeros, their multiplicities and its degree, before ever plotting a point

Projects built:

- Zero-to-graph drill: eight polynomials sketched from a list of zeros and multiplicities alone, then checked point by point

Practice: 14 problems finding zeros, multiplicities and local behaviour, each sketch attempted before any calculator is used

End Behaviour and Rational Functions

Topics:

- End behaviour of a polynomial function read directly from its degree and leading coefficient

- Rational functions introduced as one polynomial divided by another, and why their behaviour differs from a polynomial's
- Horizontal asymptotes of a rational function from comparing numerator and denominator degree
- Slant asymptotes, which appear when the numerator's degree is exactly one more than the denominator's
- Zeros of a rational function found from the numerator, and why the denominator is never asked for a zero
- Describing end behaviour the way the exam expects it written, not just sketched

Projects built:

- Asymptote hunt: twelve rational functions sorted into horizontal-asymptote, slant-asymptote or neither, then verified by long division where needed

Practice: 16 end-behaviour and asymptote problems across polynomials and rational functions, degree comparison written before any graphing

Vertical Asymptotes, Holes, and Equivalent Forms

Topics:

- Vertical asymptotes of a rational function from the zeros of the denominator
- Holes: what happens when a factor cancels between numerator and denominator, and why that point still does not exist
- Telling an asymptote from a hole by algebra, not by guessing from a rough sketch
- Equivalent representations of a polynomial or rational expression, factored versus standard form, and what each reveals faster
- Rewriting a rational expression to expose its asymptotes, holes and end behaviour in a single step
- Choosing the representation that answers a given question fastest, a real time-saver under the no-calculator clock

Projects built:

- Rewrite-and-reveal set: ten rational functions rewritten from standard to factored form specifically to expose a hidden hole or asymptote

Practice: 18 mixed Unit 1 problems on asymptotes, holes and equivalent forms, worked at no-calculator pace

Assessment: Month 1 checkpoint: a timed, no-calculator paper on rates of change, polynomial behaviour and rational functions, marked for reasoning as well as answers

Month 2: Transformations, Modelling, and Unit 1 Mastery (Weeks 5-8)

Transformations of Functions

Topics:

- Parent function shapes reviewed: linear, quadratic, cubic, square root and reciprocal
- Vertical and horizontal translations applied to a parent function's equation and graph
- Vertical and horizontal stretches and compressions, and how they interact with translations
- Reflections, and combining several transformations correctly in one rule
- Reading a transformed equation straight into a sketch without plotting points first
- Recovering the transformation from a before-and-after pair of graphs, a favourite exam question style

Projects built:

- Transformation matching set: ten transformed graphs matched to their equations, then the reverse, equation to sketch, for the same ten

Practice: 20 transformation problems split evenly between equation-to-graph and graph-to-equation

Choosing a Function Model

Topics:

- Deciding whether a data table is better modelled as linear, polynomial or rational, from its pattern of change
- Using constant, changing or ratio-based differences to identify the right model family
- Stating the assumptions a chosen model makes, and saying honestly where it might not hold
- Fitting a model to a short data table and justifying the choice of model in writing
- The justify-your-model free-response style, where the reasoning earns as many points as the equation
- Common model-selection traps: data that looks exponential over a short interval but is not

Projects built:

- Model justification write-up: two data tables modelled and the choice of model defended in four written sentences each

Practice: 10 model-selection problems, a written justification required for every answer, not just an equation

Algebraic Fluency with Polynomial and Rational Expressions

Topics:

- Factoring polynomial and rational expressions at exam speed, not textbook speed
- Polynomial long division and how a leftover remainder connects to a slant asymptote
- Combining rational expressions and simplifying without silently losing a domain restriction
- Solving polynomial and rational equations cleanly, checking every solution against the original domain
- Sign analysis on a number line for solving rational inequalities
- The domain-restriction mistakes that quietly cost marks on an otherwise correct answer

Projects built:

- Domain-safe solving set: eight rational equations and inequalities solved with every excluded value stated before the algebra begins

Practice: 18 equation and inequality problems, domain restrictions written first on every single one

Unit 1 Mastery and the Multiple-Choice Format

Topics:

- Pulling Unit 1 together: rates of change, polynomial behaviour, rational functions and equivalent forms
- The multiple-choice section as it is now: 40 questions, 2 hours, worth 62.5 percent of the score
- Part A: 28 questions in 80 minutes with no calculator, fluency without a crutch
- Part B: 12 questions in 40 minutes with a graphing calculator required
- Recognising on sight which Unit 1 questions reward hand analysis and which are faster on a calculator
- A personal error log started here, tagged by unit and by calculator or no-calculator part, and grown for the rest of the course

Projects built:

- First timed multiple-choice block on Unit 1, split into a no-calculator half and a calculator half, reviewed question by question

Practice: A timed Unit 1 multiple-choice set at exam pace, both parts, followed by a written review of every miss

Assessment: Unit 1 assessment: a timed multiple-choice section on polynomial and rational functions, run in two parts, calculator and no-calculator, with the error log updated

----- PHASE 2: EXPONENTIAL AND LOGARITHMIC FUNCTIONS (UNIT 2, MONTHS 3-4, WEEKS 9-16) -----

Sequences as the discrete root of exponential change, exponential functions and modelling, composition and inverses, then logarithms through to equation solving, closing with a first look at free response.

Month 3: Sequences, Exponential Functions, and Modelling (Weeks 9-12)

Sequences and the Roots of Exponential Growth

Topics:

- Arithmetic sequences as discrete linear change, and the common difference that defines one
- Geometric sequences as discrete exponential change, and the common ratio that defines one
- Telling an arithmetic pattern from a geometric one directly from a table, before writing any formula
- Moving from a geometric sequence to a continuous exponential function
- The general exponential function form and what its starting value and base each control
- Growth versus decay, read straight from whether the base is greater than or less than one

Projects built:

- Sequence-to-function bridge: four geometric sequences extended into continuous exponential models, with the jump from discrete to continuous explained in writing

Practice: 16 sequence problems, half arithmetic and half geometric, each one converted into a formula from a table alone

Exponential Functions: Behaviour and Manipulation

Topics:

- Domain, range and end behaviour of an exponential function
- Percent change and how it connects directly to the growth or decay factor in the equation
- Rewriting an exponential expression with a different base without changing what it represents
- Transformations of exponential functions, applied the same way as any other parent function
- Why an exponential function's average rate of change scales rather than stays constant across equal-length intervals
- Spotting an exponential pattern in a table from that scaling behaviour, before any equation is written

Projects built:

- Percent-change translator: eight real-world growth and decay descriptions rewritten as exponential equations, base and starting value identified from the words alone

Practice: 18 exponential-function problems mixing percent change, rewriting bases and reading behaviour from tables

Exponential Modelling and Competing Models

Topics:

- Setting up an exponential model from a context: identifying the starting value and the rate
- Interpreting the parameters of a fitted exponential model back in the words of the original problem
- Competing function model validation: choosing between linear, polynomial and exponential from how a data set actually changes
- Checking a proposed model against the given data points before trusting it further
- The build-a-model and justify-your-choice free-response style, and what earns the reasoning point
- Recognising when a model is being used outside a domain where it still makes sense

Projects built:

- Model face-off: one data set modelled two competing ways, exponential and polynomial, with a written case for which one actually fits

Practice: 12 modelling problems, each requiring both a model and a two-to-three sentence justification

Function Composition and Inverses

Topics:

- Composing two functions and evaluating the result from a table, a graph, or an equation
- Interpreting what a composition means in a real-world context, not just computing it
- Inverse functions: what it actually means for one function to undo another
- Finding an inverse algebraically and confirming it graphically as a reflection over $y = x$
- The domain restriction a function needs before a true inverse exists

- Why exponential and logarithmic functions are built as a natural inverse pair, setting up the second half of this unit

Projects built:

- Inverse verification set: six functions inverted algebraically, then checked both by composing and graphically, by reflecting

Practice: 14 composition and inverse problems, each inverse checked by composing it back with the original

Assessment: Month 3 checkpoint: a timed paper on sequences, exponential functions and modelling, composition and inverses

Month 4: Logarithms and Equation Solving (Weeks 13-16)

Logarithms as Inverses of Exponential Functions

Topics:

- Defining the logarithm as the inverse of an exponential function, not as an isolated new rule
- Converting fluently between exponential form and logarithmic form in both directions
- The common logarithm and the natural logarithm, and when each one is the natural choice
- Domain and range of a logarithmic function, and why the domain excludes zero and negative numbers
- Graphing a logarithmic function directly from its exponential parent by reflecting over $y = x$
- Vertical asymptotes of logarithmic graphs, and why they sit exactly where the exponential's horizontal asymptote was

Projects built:

- Exponential-log translation set: twelve equations converted between exponential and logarithmic form, evaluated by hand where the values are friendly

Practice: 16 problems converting between forms and sketching logarithmic graphs from their exponential parents

Logarithmic Properties and Manipulation

Topics:

- The product, quotient and power properties of logarithms, and where each one comes from
- Expanding a single logarithmic expression into several, and condensing several back into one
- The change-of-base formula and exactly when a calculator needs it
- The property-application mistakes the exam sets on purpose, misapplying a rule that only works one way
- Using log properties to simplify an expression before solving, which is almost always faster than solving first
- Transformations of logarithmic functions, applied the same way as exponential ones

Projects built:

- Expand-and-condense drill: ten logarithmic expressions expanded fully, then condensed back, each check confirming the two forms are equal

Practice: 18 logarithm-property problems, half expanding and half condensing, plus 4 change-of-base calculator problems

Solving Exponential and Logarithmic Equations

Topics:

- Solving exponential equations by matching bases when possible
- Solving exponential equations by taking a logarithm of both sides when bases will not match
- Solving logarithmic equations and checking every solution against the original domain for extraneous answers
- Exponential and logarithmic inequalities, solved and expressed in interval form
- Semi-log plots: why data that looks exponential on a normal grid becomes a straight line on a semi-log scale
- Reading a semi-log plot to recover the original exponential model's parameters

Projects built:

- Extraneous-solution audit: eight logarithmic equations solved, with every solution checked back in the original equation and any extraneous root explained

Practice: 20 equation and inequality problems across exponential and logarithmic functions, extraneous-solution checks required throughout

Unit 2 Mastery and a First Look at Free Response

Topics:

- Pulling Unit 2 together: sequences, exponential functions, composition, inverses and logarithms
- A first full look at the free-response section: 4 questions, 1 hour, worth 37.5 percent of the score
- Part A of free response: 2 questions, 30 minutes, graphing calculator required
- Part B of free response: 2 questions, 30 minutes, no calculator allowed
- The style of exponential and logarithmic modelling question that tends to appear in free response
- Writing a modelling justification in full sentences, since the reasoning point is scored separately from the equation

Projects built:

- First free-response attempt: one Unit 2 style modelling question answered under time and self-scored against a sample rubric

Practice: A timed Unit 2 multiple-choice set at exam pace plus one full free-response attempt, reviewed for both content and communication

Assessment: Unit 2 assessment: a timed section on exponential and logarithmic functions, run in calculator and no-calculator parts, with the error log updated

The unit circle, sinusoidal and trigonometric functions, and polar coordinates and functions, followed by a full turn to exam craft: the calculator/no-calculator skill, free-response method, and complete timed hybrid-digital mock exams.

Month 5: Trigonometric and Polar Functions (Weeks 17-20)

The Unit Circle and Periodic Behaviour

Topics:

- The unit circle built from angle, radius one, and the coordinates it defines as cosine and sine
- Radians as the natural unit for angle measure, and fluent conversion between radians and degrees
- Special angles and the exact sine and cosine values worth having memorised, not looked up
- Periodicity: why sine and cosine repeat, and what a period of two-pi actually means
- Sine and cosine graphed directly from the unit circle, point by point at first, then from pattern
- Amplitude, period and midline read from a sinusoidal graph or straight from its equation

Projects built:

- Unit-circle-to-graph build: the sine and cosine graphs constructed point by point from the unit circle, then compared to the pattern-based shortcut

Practice: 20 unit-circle and radian-conversion problems, exact values required wherever the angle is a special one

Sinusoidal Functions and Modelling

Topics:

- The general sinusoidal function form and what amplitude, period, midline and phase shift each control
- Phase shift and exactly how it moves a sine or cosine graph left or right
- Building a sinusoidal model from a context description: extracting amplitude, period, midline and phase from words
- Sinusoidal context and data modelling questions in the free-response style
- Reading a sinusoidal model's parameters back out of a real-world description, the reverse skill
- Checking a fitted sinusoidal model against given data points before trusting it

Projects built:

- Sinusoidal modelling write-up: two real-world periodic scenarios, tides and temperature style, modelled with a full equation and parameters explained in words

Practice: 14 sinusoidal modelling problems, amplitude, period, midline and phase identified separately before the equation is assembled

Tangent, Reciprocal Trig, and Inverse Trig

Topics:

- The tangent function: its period, its vertical asymptotes and how its graph differs from sine and cosine
- Secant, cosecant and cotangent as reciprocal functions, graphed from their sine and cosine parents
- Inverse trigonometric functions and the restricted domains that make them true functions
- Using an inverse trig function to solve for an angle in a real context
- Solving trigonometric equations over a stated interval, finding every solution, not just one
- Trigonometric inequalities and expressing the solution as an interval

Projects built:

- Solve-and-verify set: ten trigonometric equations solved over a stated interval, every solution checked back on the unit circle

Practice: 18 problems across tangent, reciprocal trig functions and inverse trig, solution intervals stated before solving begins

Equivalent Trig Forms and Polar Functions

Topics:

- Trig identities as equivalent representations: the Pythagorean identity and the reciprocal identities
- Rewriting a trigonometric expression into an equivalent form that reveals its behaviour faster
- Polar coordinates: converting a point between polar and rectangular form in both directions
- Polar functions and their graphs: circles, roses and limacons recognised at a glance from their equation
- Reading symmetry and key features, maximum distance, zeros, directly from a polar graph
- How a polar function's periodic behaviour connects back to everything studied in this unit

Projects built:

- Polar gallery: six polar functions graphed and identified by family, circle, rose or limacon, with symmetry and key points labelled

Practice: 16 problems mixing trig identity rewriting and polar coordinate conversion, plus 4 polar-graph identification questions

Assessment: Unit 3 assessment: a timed section on trigonometric and polar functions, run in calculator and no-calculator parts

Month 6: Free-Response Mastery and Full Mock Exams (Weeks 21-24)

The Calculator/No-Calculator Split and Multiple-Choice Strategy

Topics:

- Mapping all four exam parts precisely: MC Part A, 28 questions, 80 minutes, no calculator; MC Part B, 12 questions, 40 minutes, calculator required; FRQ Part A, 2 questions, 30 minutes, calculator; FRQ Part B, 2 questions, 30 minutes, no calculator
- Building genuine no-calculator fluency: exact values, factoring and algebraic manipulation done by hand at speed

- Calculator strategy for Part B: what a graphing calculator is actually good for, and what it cannot substitute for
- Pacing each part on its own terms, roughly two minutes a question in Part A and just over three in Part B
- Recognising at a glance which part of the exam a released question is drawn from, and adjusting method accordingly
- A first timed multiple-choice block mixing both parts, reviewed question by question with each miss tagged by cause

Projects built:

- Timed mixed multiple-choice block, Part A followed immediately by Part B, reviewed with every miss tagged calculator, no-calculator, concept or pacing

Practice: A timed multiple-choice set at full exam pace across both parts, followed by a written review of every wrong and every guessed answer

Free-Response Method Across Units 1 to 3

Topics:

- The free-response rubric mindset: where partial credit lives and how to secure it even on a question you cannot fully finish
- Justification writing: what actually earns the communication point on a modelling question, in plain, specific sentences
- A Unit 1 style free-response task: justifying a polynomial or rational model against given data
- A Unit 2 style free-response task: building an exponential or logarithmic model and solving an equation from it
- A Unit 3 style free-response task: building a sinusoidal model from a context description
- Self-scoring a free-response answer against a sample rubric, and naming exactly which point was lost and why

Projects built:

- Three timed free-response responses, one from each tested unit, each self-scored against a sample rubric with lost points named

Practice: Timed free-response practice across all three units, every attempt scored for both the answer and the justification

Full Mock Exam One

Topics:

- Running a complete timed mock in the real order: Multiple Choice Part A, Part B, then Free Response Part A, Part B
- Working under hybrid-digital conditions: multiple choice answered in a Bluebook-style flow, free response handwritten on paper
- Scoring the mock and converting the raw result toward the 1 to 5 scale
- Reviewing errors sorted by unit, Units 1, 2 and 3, to see where the real gaps sit
- Reviewing errors sorted by part, so a calculator-dependent weakness is not confused with a content gap
- Naming the single biggest gap the mock revealed and building one week's plan to close it

Projects built:

- Full mock exam one, scored and reviewed, with a written breakdown of performance by unit and by calculator or no-calculator part

Practice: The full timed mock plus a complete review, every missed multiple-choice question re-worked correctly by hand

Full Mock Exam Two, Test-Ready Review, and a Unit 4 Preview

Topics:

- A second complete timed mock, run fully under exam conditions from Multiple Choice Part A through Free Response Part B
- Comparing the two mocks directly to see real direction of travel, not just a single score
- The test-day routine: the Bluebook app for multiple choice, the paper booklet for free response, and timing across all four parts
- A realistic, evidence-based target score built from both mocks, not a guess
- An optional, clearly labelled preview of Unit 4, functions involving parameters, vectors and matrices, for students continuing on to calculus; this content is not tested on the AP Precalculus exam and is covered only as a bridge forward
- Where to go next: AP Calculus AB or BC, or a next mathematics course, chosen honestly from where this course leaves you

Projects built:

- Full mock exam two, scored and reviewed, ending in a one-page test-day plan built from both mocks' results

Practice: The second full mock plus targeted repair of any remaining weak topic from either mock

Assessment: Final assessment: a complete timed mock exam covering Units 1 through 3, a progress summary from week 1 to now, and certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A rate-of-change notebook studying one function across table, graph and equation
- A zero-to-graph and asymptote-hunt portfolio across polynomial and rational functions
- A model-justification write-up defending a chosen function model in writing
- A sequence-to-function bridge connecting discrete geometric sequences to continuous exponential models
- An inverse-verification set checking every inverse both algebraically and graphically
- A sinusoidal modelling write-up built from real-world periodic scenarios
- A polar gallery of graphed and identified polar function families
- Three scored free-response answers across all three tested units, plus two full timed mock

exams under hybrid-digital conditions

Total projects: Dozens of worked problem sets plus three scored free responses and two complete timed mock exams, all mapped to Units 1 through 3

SKILLS THE STUDENT WILL HAVE AT THE END

- Function fluency: rates of change, transformations, end behaviour and equivalent representations
- Polynomial and rational function analysis, including zeros, asymptotes and holes
- Exponential and logarithmic reasoning, from sequences through to equation solving
- Trigonometric and polar function analysis, from the unit circle through to polar graphs
- Model selection and modelling justification, written the way the free-response rubric rewards
- Calculator and no-calculator fluency on both sides of the exam's part boundary, drilled on full timed mocks

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, working and checking problems live rather than watching solutions scroll by

Practice: 4-5 hours weekly of problem sets and, in the final month, timed free-response and full mock sections

Review: Homework and free responses reviewed with written feedback, scored against a sample rubric so you see exactly where reasoning is won and lost

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for the problems that stall you

Progress updates: Monthly progress notes tracking unit assessments and, in the final month, mock-exam scores against your target

Career guidance: Honest guidance on what an AP score does and does not do for college admissions and credit, and what mathematics course to take next

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, awarded on finishing the course and its final timed mock exams

WHERE THIS CAN LEAD

- A strong footing for the AP Precalculus exam, Units 1 through 3, and through it, university applications
- The function fluency, transformations, modelling and equivalent representations, that AP Calculus AB and BC assume from the first week
- A genuine next step toward AP Calculus or a first university mathematics course
- Modelling and justification habits that transfer to statistics, physics and other quantitative coursework
- The problem-solving base for our AP Calculus and college-level mathematics courses

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is Unit 4 tested on the AP Precalculus exam?

A: No, and this is the single most useful thing to know before you plan your prep time. AP Precalculus is taught as four units, but the exam only assesses Units 1, 2 and 3: polynomial and rational functions, exponential and logarithmic functions, and trigonometric and polar functions. This course concentrates its exam preparation on Units 1 through 3, exactly where the scoring happens, and covers Unit 4 only as a short, clearly labelled preview in the final week for students continuing on to calculus.

Q: What is the calculator versus no-calculator split on this exam?

A: Both sections are split into a calculator part and a no-calculator part, and knowing the boundary cold is a real exam skill. Section I, Multiple Choice, has Part A with 28 questions in 80 minutes and no calculator allowed, then Part B with 12 questions in 40 minutes where a graphing calculator is required. Section II, Free Response, has Part A with 2 questions in 30 minutes where a graphing calculator is required, then Part B with 2 questions in 30 minutes and no calculator. We drill both sides of that line from week one, not just in the final month.

Q: What do I need to know before starting this course?

A: A solid Algebra 2 background: comfort with function notation, factoring, exponents and basic graphing. The course does not re-teach Algebra 2 from the beginning, though real gaps are patched as they show up rather than ignored. No programming or coding background is needed or used; this is a pure mathematics course.

Q: How does AP Precalculus actually prepare me for AP Calculus?

A: Honestly, and mostly through fluency rather than new tricks. AP Calculus assumes you can

read a function's behaviour, transformations, end behaviour, zeros, asymptotes, without hesitating, and that you are comfortable moving between exponential, logarithmic and trigonometric forms. That is exactly what Units 1 through 3 build. Students who finish this course fluent in function behaviour spend their first weeks of calculus learning calculus, not relearning precalculus under a new name.

Q: What is the AP Precalculus exam format?

A: Two sections. Section I, Multiple Choice, is 40 questions in 2 hours and is worth 62.5 percent of the score, split into a 28-question no-calculator Part A and a 12-question calculator-required Part B. Section II, Free Response, is 4 questions in 1 hour and is worth 37.5 percent of the score, split into a 2-question calculator-required Part A and a 2-question no-calculator Part B.

Q: What is the hybrid digital exam format?

A: The multiple-choice section is taken digitally in the College Board's Bluebook app, the same platform used for the digital SAT and several other AP exams. The free-response section, though, is answered by hand: you view the questions in Bluebook but write your actual answers in a paper booklet. We run our mock exams under exactly this split, digital multiple choice and handwritten free response, so the format itself is familiar by test day.

Q: Will this course get me a 5, or college credit?

A: No honest course can promise a specific score. AP exams are graded 1 to 5, and your result depends on your starting point, the hours you put in and your performance on the day. What we can promise is honest: live teaching concentrated on the tested units, small batches, real doubt support, free-response drilling against a real rubric, and full timed mock exams under hybrid-digital conditions, plus a clear, evidence-based read on a realistic target for you. Whether a given score earns college credit, and for what, is decided by each university, and AP Precalculus credit policies vary more than AP Calculus policies do, so we point students to check the specific policy of the colleges they care about.

Q: Is this course affiliated with the College Board?

A: No. Modern Age Coders is an independent education provider and is not affiliated with, authorised by, or endorsed by the College Board. AP and Advanced Placement are trademarks of the College Board, used here only to describe the exam this course prepares students for. For official course and exam information, students should always refer to the College Board and AP Central directly.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ap-precalculus-exam-prep-course>

=====

COURSE 20 OF 118: AP STATISTICS: EXAM PREP WITH THE INVESTIGATIVE TASK

=====

Course link: <https://learn.modernagecoders.com/courses/ap-statistics-maths-exam-prep-course>

Subject area: Advanced Placement Mathematics

Designed for: High school

Who can join: High school students preparing for AP Statistics; taught from the ground up, comfort with school algebra assumed

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 4-5 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online AP Statistics exam prep: all nine AP units, the Investigative Task, free-response rubric mastery and full digital mock exams for a real score.

In one line: A statistics course built around one exam: nine AP units, the Investigative Task, and the free-response rubric that decides most of the score.

ABOUT THE PROGRAMME

AP Statistics is not the same subject as a general college statistics course, even though the two share a name. It is organised around nine specific units, tested on one specific exam, in one specific format, and this course is built around exactly that exam as it exists now, not around statistics in general. The exam runs in two equal halves: Section I is 40 multiple-choice questions in 90 minutes, worth half the score, and Section II is 6 free-response questions in 90 minutes, worth the other half. Of those six, the first five are short-answer questions, budgeted at roughly 12 minutes each, and the sixth is the Investigative Task, a single longer question worth about 30 of those 90 minutes that asks you to apply and extend what you know to a scenario the exam has not shown you before. The exam is also hybrid: multiple-choice questions and the free-response prompts are viewed in the College Board's Bluebook digital testing app, but every free-response answer is handwritten in a paper booklet that is collected and scored separately. A graphing calculator, such as a

TI-84, is expected and permitted for the whole exam, and this course teaches its use from week one. The six-month structure follows the nine AP units in the order the exam tests them. Months 1 and 2 cover Units 1 through 3: exploring one-variable data with the normal model, exploring two-variable data with correlation and least-squares regression, and collecting data through sampling and experimental design. Months 3 and 4 cover Units 4 and 5: probability, random variables, the binomial and geometric distributions, then sampling distributions and the Central Limit Theorem that every later inference procedure depends on. Months 5 and 6 cover Units 6 through 9, formal inference for proportions, means, chi-square settings and regression slopes, then turn fully to exam craft: the Investigative Task practiced as its own skill, every free-response question type drilled against the real rubric, and two full timed mock exams run under the exam's actual hybrid-digital conditions. Because the rubric rewards communication as much as computation, every answer in this course, from week one onward, is written to justify a conclusion with evidence, interpret it correctly in context, and state the conditions it depends on, since a right number with no justification earns limited credit on the real exam.

WHAT MAKES THIS COURSE DIFFERENT

- Organised around the nine official AP Statistics units in the order the exam actually tests them, not a general statistics syllabus with an AP label attached
- The Investigative Task, free-response Question 6, practiced as its own skill from month 6 onward, since it is scored differently from the other five questions and most courses barely touch it
- Free-response answers drilled against the communication standard the rubric truly rewards: justifying a conclusion with evidence, interpreting it correctly, and stating the conditions it depends on
- Graphing calculator fluency built in from week 1, since the real exam expects confident use of a calculator such as a TI-84 throughout both sections
- Full readiness for the hybrid-digital format: multiple-choice and free-response prompts viewed in the Bluebook app, but free-response answers handwritten in paper booklets, practiced under those exact conditions
- Live, small batches where you work real data and real free responses during class and get them reviewed, not lecture videos watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Exploring data and collecting it: one-variable distributions and the normal model, two-variable scatterplots and least-squares regression, then sampling methods, experiments and bias (Units 1-3)

Phase 2: Randomness and its rules: probability, random variables, binomial and geometric distributions, then sampling distributions and the Central Limit Theorem (Units 4-5)

Phase 3: Formal inference and the exam itself: confidence intervals and significance tests for proportions, means, chi-square settings and slopes (Units 6-9), Investigative Task mastery, and full timed mock exams under real hybrid-digital conditions

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable school algebra: working with formulas, fractions, percentages and graphs. No calculus is required anywhere in AP Statistics

Programming: None required. All work is done with a graphing calculator and by hand; no coding or spreadsheet skills are needed for this course

Equipment: A graphing calculator such as a TI-84 or an equivalent approved model, used and permitted throughout the real exam, plus a device with a stable internet connection for live classes. We help with calculator setup in week 1

Audience: High school students preparing for AP Statistics, typically in the years leading up to the exam, whether taking it at school or self-studying

WHO THIS COURSE IS FOR

Ap students: Students taking AP Statistics at school who want a structured, unit-by-unit course and real feedback on their free responses alongside their class

Self studiers: Students whose school does not offer AP Statistics and who are preparing for the exam independently

Score focused: Students aiming for a strong score who want free-response drilling against the real rubric, Investigative Task practice, and full timed mock exams

Math and science bound: Students headed for a data-heavy college major, from psychology to engineering, who want genuine statistical reasoning, not just exam tricks

International students: International students sitting AP Statistics for university applications who need the exam taught to its current hybrid-digital format

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: EXPLORING DATA AND COLLECTING DATA (UNITS 1-3, MONTHS 1-2, WEEKS 1-8)

The data-analysis half of the course: describing one-variable and two-variable data honestly, then how data is collected in the first place. The graphing calculator and the free-response

communication standard are introduced here and used every week from this point on.

Month 1: Exploring One-Variable Data (Unit 1) (Weeks 1-4)

Displaying and Describing Distributions

Topics:

- Individuals, variables, and the shape of the AP Statistics course: nine units, two exam sections, and the exam that ends it
- Categorical versus quantitative data, and the type of graph each one calls for
- Dotplots, stemplots and histograms: reading shape, not just drawing it
- Describing a distribution in words: shape, outliers, center and spread, the sentence the exam always wants
- Setting up the graphing calculator: entering lists, running 1-Var Stats, and building a first histogram on screen
- The AP Statistics communication standard: why a correct number with no justification earns limited credit

Projects built:

- First distribution report: a real one-variable data set displayed, described in full shape-outliers-center-spread sentences, and checked on the calculator

Practice: 12 distribution-description problems, each answer required to name shape, outliers, center and spread in one paragraph, not a list of numbers

Measuring Center and Spread

Topics:

- Mean and median, and which one a skewed distribution or an outlier pulls away from the other
- Range, interquartile range, and the 1.5 times IQR rule for flagging outliers
- Variance and standard deviation built up from deviations, not dropped as a formula
- The five-number summary, and building a boxplot from it by hand and by calculator
- Choosing mean and standard deviation versus median and IQR to summarise a distribution, and defending the choice
- Comparing two distributions side by side on boxplots, a guaranteed exam question type

Projects built:

- Center-and-spread comparison: two real groups compared on boxplots with a full written comparison of shape, center, spread and outliers

Practice: 14 problems computing and choosing summary measures, half requiring a written comparison between two groups

The Normal Distribution

Topics:

- The normal curve, its parameters, and the 68-95-99.7 rule
- Standardising a value into a z-score and reading what it says about relative position
- Finding areas under the normal curve on the calculator with normalcdf
- The inverse problem: finding a value from a given percentile with invNorm
- Assessing normality with a normal probability plot, read for curvature rather than computed
- Normal calculations written up in context, the way the exam scores them, not just the final number

Projects built:

- Normal model check: a real quantitative data set tested against the 68-95-99.7 rule, with normalcdf and invNorm used to answer three context questions

Practice: 14 normal distribution problems, each opened with a labelled sketch of the curve and the shaded region before any calculator work

Unit 1 Mastery and the Free-Response Standard

Topics:

- Pulling Unit 1 together: distributions, summary statistics and the normal model in one paper
- Reading a Unit 1 free-response prompt the way the rubric is written, points named as sentences rather than numbers
- The exam's two sections at a glance: 40 multiple-choice questions in 90 minutes worth 50 percent of the score, and 6 free-response questions in 90 minutes worth the other 50 percent
- Multiple-choice pacing for a Unit 1 style set: roughly two minutes a question
- Common Unit 1 mistakes: describing shape from a boxplot alone, and confusing standard deviation with interquartile range
- Starting a personal error log, kept and grown for the rest of the course

Projects built:

- First scored free response: a Unit 1 style question written under time and self-scored against a rubric, with every lost point named

Practice: A timed Unit 1 multiple-choice set at exam pace, followed by one full free-response question reviewed line by line against its rubric

Assessment: Unit 1 assessment: a timed paper on one-variable data covering multiple-choice and one free response, marked on both the number and the justification

Month 2: Two-Variable Data and Collecting Data (Units 2-3) (Weeks 5-8)

Exploring Two-Variable Data: Scatterplots and Correlation

Topics:

- Explanatory and response variables, and setting up a scatterplot correctly
- Describing a scatterplot: direction, form, strength and outliers, the two-variable version of shape-outliers-center-spread
- The correlation coefficient r : what it measures and where its usefulness stops
- Correlation is not causation, with a plausible confounding variable named every time
- Computing r on the calculator from two lists
- Why a strong r does not automatically mean a good model, previewing residuals

Projects built:

- Relationship study: a real two-variable data set plotted, described in full, and its

correlation computed and interpreted in context

Practice: 12 scatterplot and correlation problems, every claim of a relationship backed by a described scatterplot, not just an r value

Least-Squares Regression

Topics:

- The least-squares regression line and exactly what quantity it minimises
- Finding the equation on the calculator with LinRegTTest, and reading slope and intercept in context
- The coefficient of determination, r -squared, and what percentage of variation it explains
- Residuals and the residual plot: the honesty check the exam consistently asks for
- Predicting within the range of the data versus extrapolating beyond it
- Influential points and outliers in a regression setting

Projects built:

- Prediction model: a regression line fit to real data, its residual plot checked for pattern, and two predictions made with their limits stated

Practice: 12 regression problems with slope and intercept interpreted in the units of the problem every time, plus 4 residual-plot readings

Collecting Data: Sampling

Topics:

- Population versus sample, and why that distinction drives all of Unit 3
- Simple random samples, stratified samples, cluster samples and systematic samples
- Convenience sampling and voluntary response, and why both are biased by design
- Sources of bias: undercoverage, nonresponse, and response bias from question wording
- How a sampling method is described and evaluated on the free-response section
- Designing a sampling plan for a given population and research question

Projects built:

- Sampling critique: a real published survey evaluated for its sampling method, with specific bias risks named in writing

Practice: 10 sampling-design problems, each requiring the method to be named and one bias risk identified

Collecting Data: Experiments and Unit 2-3 Mastery

Topics:

- Observational studies versus experiments, and what only a well-designed experiment can show
- Experimental design vocabulary: treatments, experimental units, and the placebo effect
- The principles of good experimental design: control, randomisation, and replication
- Blocking and why it is used, contrasted with a completely randomised design
- Confounding variables in an experiment versus in an observational study
- Units 2 and 3 pulled together, with a timed multiple-choice set mixing both

Projects built:

- Experiment design: a full experimental design written for a given research question, naming treatments, randomisation and control

Practice: 10 experimental-design problems plus a timed Unit 2-3 multiple-choice set reviewed question by question

Assessment: Unit 2-3 assessment: a timed paper on two-variable data and data collection, including one regression free response and one design free response

PHASE 2: PROBABILITY AND SAMPLING DISTRIBUTIONS (UNITS 4-5, MONTHS 3-4, WEEKS 9-16)

The mechanics of randomness that make inference possible: probability rules, random variables, the named distributions, then sampling distributions and the Central Limit Theorem that the entire second half of the course depends on.

Month 3: Probability, Random Variables and Probability Distributions (Unit 4) (Weeks 9-12)

Probability Rules

Topics:

- Randomness, probability as long-run relative frequency, and simulating it on the calculator
- Sample spaces, events, and the basic probability rules
- The addition rule for mutually exclusive and overlapping events
- Conditional probability, and what being told the condition changes about the sample space
- Independence: the definition, and testing for it using a two-way table
- The multiplication rule for independent and dependent events

Projects built:

- Two-way table study: a real two-way table used to compute conditional probabilities and formally test two events for independence

Practice: 14 probability problems from single events through conditional probability, independence checked by calculation, not by eye

Random Variables

Topics:

- Discrete random variables and their probability distributions
- Mean, or expected value, of a discrete random variable, and what it means for a variable that only takes whole values
- Standard deviation of a discrete random variable
- Combining random variables: rules for the mean and variance of sums and differences
- Independence between two random variables, and why it matters before combining variances
- Random variables in context: insurance, games, and simple business scenarios

Projects built:

- Combined-variable analysis: two random variables combined by sum or difference, with mean and standard deviation of the result found and interpreted

Practice: 12 random variable problems including 4 combination questions checked for the independence condition first

Binomial and Geometric Distributions

Topics:

- The binomial setting: the four conditions checked in words before any formula is used
- Binomial probability, mean and standard deviation, by hand and with `binompdf` and `binomcdf`
- The geometric setting: waiting for the first success
- Geometric probability, mean and standard deviation, with `geompdf` and `geomcdf`
- Choosing between a binomial and a geometric model from the story in the question
- The 10 percent condition: when sampling without replacement behaves close enough to independent

Projects built:

- Model choice project: three real scenarios each modelled correctly as binomial or geometric, with the setting's conditions checked in writing

Practice: 14 binomial and geometric problems, the first written step always naming and checking the setting's conditions

Unit 4 Mastery and Probability Free Response

Topics:

- Pulling Unit 4 together: probability rules, random variables, binomial and geometric models
- Reading a probability free-response question for exactly what it asks: a probability, a mean, or a standard deviation
- Common Unit 4 mistakes: using a binomial model when trials are not independent, and misreading at least versus more than
- Calculator fluency check: `normalcdf`, `invNorm`, `binompdf`, `binomcdf`, `geompdf` and `geomcdf` used correctly and quickly
- A timed Unit 4 multiple-choice set at exam pace
- Error log review: Unit 4 mistakes classified and drilled

Projects built:

- Timed probability free response written under exam conditions and self-scored against a rubric

Practice: A timed Unit 4 multiple-choice set plus one full free-response question reviewed against its rubric

Assessment: Unit 4 assessment: a timed paper on probability, random variables and probability distributions, including one free response

Month 4: Sampling Distributions (Unit 5) (Weeks 13-16)

Sampling Variability and the Sampling Distribution of a Proportion

Topics:

- Parameter versus statistic, and why every sample gives a slightly different answer
- The idea of a sampling distribution: the distribution of a statistic over repeated samples
- Simulating a sampling distribution on the calculator to watch shape, center and spread build up
- The sampling distribution of a sample proportion: its mean, its standard deviation, and when it is approximately normal
- The conditions for that normal approximation: random, 10 percent, and large counts
- Unbiased estimators, and why the sample proportion is one for the population proportion

Projects built:

- Simulation build: a sampling distribution for a proportion simulated by hand or calculator, with shape, center and spread reported from the simulation

Practice: 10 sampling-distribution problems for proportions, each requiring the three conditions checked before any normal calculation

The Sampling Distribution of a Sample Mean and the Central Limit Theorem

Topics:

- The sampling distribution of a sample mean: its mean and its standard deviation
- The Central Limit Theorem stated precisely: what a large sample size buys you even from a skewed population
- Watching the Central Limit Theorem happen: simulating sample means from a skewed population as sample size grows
- When the population itself is already normal, and why a large sample size is not needed then
- The 10 percent condition applied to sample means
- Common misreadings of the theorem, including confusing the population's shape with the sampling distribution's shape

Projects built:

- Central Limit Theorem simulation: sample means drawn repeatedly from a skewed population, with the resulting shape compared at small and large sample sizes

Practice: 10 sampling-distribution problems for means, plus the simulation repeated on a second population shape

Sampling Distributions for Differences and Exam-Style Reasoning

Topics:

- The sampling distribution of a difference in two proportions
- The sampling distribution of a difference in two sample means
- Conditions for each, including independence between the two samples
- Reading a scenario to decide which sampling distribution applies: one sample or two, proportion or mean
- Connecting Unit 5 forward to Units 6 and 7: every confidence interval and test still to come rests on one of these distributions
- Free-response practice describing a sampling distribution in full: shape, center, spread and the conditions that justify each

Projects built:

- Distribution identification drill: ten scenarios each matched to the correct sampling distribution, with mean, standard deviation and conditions stated

Practice: 12 mixed sampling-distribution problems where the question never announces which distribution to use

Unit 5 Mastery and the Bridge to Inference

Topics:

- Pulling Unit 5 together: sampling variability and every sampling distribution covered so far
- A worked preview of how a confidence interval is really a sampling distribution centered on a statistic
- A worked preview of how a significance test asks whether a statistic is surprising given an assumed sampling distribution
- A timed multiple-choice set at exam pace covering Units 4 and 5 together
- Error log review: probability and sampling-distribution mistakes classified and drilled
- Calculator fluency check across every function used so far in the course

Projects built:

- Timed Unit 5 free response written under exam conditions and self-scored against a rubric

Practice: A timed Units 4-5 multiple-choice set plus one full free-response question reviewed against its rubric

Assessment: Unit 5 assessment: a timed paper on sampling distributions, including one free response connecting a distribution to its conditions

----- PHASE 3: INFERENCE AND EXAM MASTERY (UNITS 6-9 AND FULL MOCKS, MONTHS 5-6, WEEKS 17-24) -----

Formal inference for every setting the exam tests, then a full turn to exam craft: the Investigative Task drilled as its own skill, every free-response type reviewed against the real rubric, and two complete timed mock exams under hybrid-digital conditions.

Month 5: Inference for Proportions and Means (Units 6-7) (Weeks 17-20)

Confidence Intervals for a Proportion

Topics:

- The logic of a confidence interval: an interval built to have a stated chance of capturing the true parameter
- The conditions for inference on one proportion: random, 10 percent, and large counts
- Constructing a one-sample z-interval for a proportion, by hand and with the calculator
- Margin of error, and the three things that change it
- What 95 percent confidence actually means, and the misreadings the exam is built to catch
- Confidence intervals for a difference in two proportions

Projects built:

- Interval report: a confidence interval for a real proportion built from data, with the correct interpretation written out in full context

Practice: 12 proportion-interval problems, every interval followed by a correctly worded, in-context interpretation sentence

Significance Tests for a Proportion

Topics:

- Stating null and alternative hypotheses correctly for a one-proportion test
- The test statistic, the p-value, and what a p-value actually measures
- Checking the same three conditions before trusting a test
- One-sample and two-sample z-tests for proportions, by hand and with the calculator
- Type I and Type II errors, and significance level as the Type I error rate you choose
- Writing a full test conclusion in context: state, plan, do, conclude, the four-step structure the rubric rewards

Projects built:

- Claim test: a real claim about a proportion tested against data, written up in the full state-plan-do-conclude structure

Practice: 12 proportion-test problems, every conclusion written in context and linked back to the p-value

Confidence Intervals for a Mean

Topics:

- Why a t-distribution, not a normal distribution, is used for inference on a mean
- Degrees of freedom in plain language, and how the t-distribution compares to normal as they grow
- The conditions for inference on one mean: random, 10 percent, and normal or large sample
- Constructing a one-sample t-interval for a mean, by hand and with the calculator
- Confidence intervals for a difference in two means, paired and unpaired designs told apart
- Recognising a paired design, a frequent trap on the exam

Projects built:

- Interval report: a confidence interval for a real mean built from data, with the paired-versus-unpaired decision justified in writing

Practice: 12 mean-interval problems including two paired designs hidden among unpaired ones

Significance Tests for a Mean and Unit 6-7 Mastery

Topics:

- One-sample and two-sample t-tests for means, hypotheses stated and conditions checked every time
- Paired t-tests, and why they are really a one-sample test performed on the differences
- Effect size, and why a significant result is not automatically an important one
- Pulling Units 6 and 7 together: proportions and means, intervals and tests, side by side
- A timed multiple-choice set at exam pace covering both units

- Error log review: inference mistakes classified, especially condition-checking and interpretation wording

Projects built:

- Timed inference free response written under exam conditions and self-scored against a rubric
- Practice: A timed Units 6-7 multiple-choice set plus one full free-response question reviewed against its rubric
- Assessment: Unit 6-7 assessment: a timed paper on inference for proportions and means, including one free response scored on conditions and conclusion wording

Month 6: Chi-Square, Slopes, the Investigative Task, and Full Mock Exams (Units 8-9) (Weeks 21-24)

Inference for Categorical Data: Chi-Square

Topics:

- The chi-square goodness-of-fit test: does one categorical variable match a claimed distribution
- The chi-square test for independence: are two categorical variables related, within one sample
- The chi-square test for homogeneity: do several populations share the same distribution, and how it differs from independence
- Expected counts, the conditions for a valid chi-square test, and computing the test statistic
- Running each chi-square test on the calculator and reading its output correctly
- Writing a full chi-square conclusion in the state-plan-do-conclude structure

Projects built:

- Independence study: a real two-way table tested for independence, with expected counts shown and a full written conclusion

Practice: 12 chi-square problems split across the three test types, the correct type identified from the scenario before any calculation

Inference for the Slope of a Regression Line

Topics:

- The population regression model, and what the true value of the slope represents
- Conditions for inference on a slope, read from a residual plot and a normal probability plot of residuals
- The t-test for the slope: hypotheses, test statistic, and reading calculator regression output
- The confidence interval for the slope, constructed from the same output
- Connecting Unit 9 back to Unit 2: the regression skills from month 2, now formally tested
- Interpreting a slope-test conclusion in the context of the original two variables

Projects built:

- Slope inference project: a real two-variable data set tested for a nonzero slope, with conditions checked from residual and normal plots

Practice: 10 slope-inference problems, conditions checked from a described or sketched residual plot every time

The Investigative Task

Topics:

- What free-response Question 6, the Investigative Task, actually asks: applying and extending the course's skills to a scenario that will not look familiar
- Why the Investigative Task is scored differently from Questions 1 through 5, and what that means for pacing, roughly 30 of the 90 free-response minutes
- Reading an unfamiliar prompt calmly: identifying which unit's tools actually apply underneath the new dressing
- Extending a known method one step further than it was originally taught, the specific skill the task tests
- Working through released Investigative Task style prompts, one full task per session
- Building a personal checklist for the task: state assumptions, justify every step, and answer the question actually asked

Projects built:

- Two full Investigative Task style responses written under time and self-scored against a released-style rubric, with the extension step highlighted

Practice: Two more Investigative Task style prompts worked untimed first for understanding, then timed, with every assumption stated in writing

Full Mock Exams and Test-Ready Review

Topics:

- A first complete timed mock exam: all 40 multiple-choice questions and all 6 free-response questions, including the Investigative Task
- Running the mock under real hybrid-digital conditions: multiple-choice and free-response prompts viewed digitally, free-response answers handwritten on paper
- Scoring the mock against official-style rubrics and converting toward the 1 to 5 scale
- A second complete timed mock, then closing whatever gaps the first one exposed
- Test-day logistics: the Bluebook app, the graphing calculator, the paper answer booklets, and timing across both sections
- A one-page, evidence-based test-day plan built from your own two mock scores

Projects built:

- Two full mock exams, scored, with a written comparison of the two and a final one-page test-day plan

Practice: Both full timed mocks plus a complete review, every missed multiple-choice question re-worked and every free response re-scored

Assessment: Final assessment: a complete timed mock exam under hybrid-digital conditions, a progress summary from week 1 to now, and certificate review

- A full shape-outliers-center-spread distribution report on a real one-variable data set
- A least-squares regression model with residuals checked and predictions limited to the data range
- A sampling-method critique of a real published survey
- A full experimental design written for a real research question
- A simulated sampling distribution built and reported by you
- A confidence interval and significance test written in the full state-plan-do-conclude structure
- A chi-square independence study with a complete written conclusion
- Two Investigative Task style responses and two full timed mock exams under hybrid-digital conditions

Total projects: A portfolio of unit projects across all nine AP units plus multiple scored free responses, two Investigative Task responses and two complete timed mock exams

SKILLS THE STUDENT WILL HAVE AT THE END

- Describing and comparing distributions with full statistical language, not just numbers
- Scatterplots, correlation and least-squares regression, checked with residual plots
- Sound data collection: sampling methods, bias, and experimental design
- Probability, random variables, and the binomial and geometric distributions
- Sampling distributions and the Central Limit Theorem that inference depends on
- Confidence intervals and significance tests for proportions, means, chi-square settings and slopes, plus Investigative Task reasoning

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, working problems and free responses live rather than watching solutions scroll by

Practice: 4-5 hours weekly of problem sets, calculator drills and, from month 5 onward, timed free-response and mock sections

Review: Homework and free responses reviewed with written feedback, scored against AP-style rubrics so you see exactly where points are earned and lost

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for the problems that stall you

Progress updates: Monthly progress notes tracking unit assessments and, in the final months, mock-exam scores against your target

Career guidance: Honest guidance on what an AP score does and does not do for college admissions and credit, and what to study next after AP Statistics

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, awarded on finishing the course and its final timed mock exams

WHERE THIS CAN LEAD

- A strong footing for the AP Statistics exam and, through it, university applications
- Readiness for a college introductory statistics course, which AP Statistics closely mirrors in both content and reasoning
- A genuine base of statistical reasoning for any data-heavy major: psychology, economics, biology, business or engineering
- Free-response and justification habits, stating assumptions and interpreting evidence, that transfer directly into college coursework
- A natural next step into our Statistics and Probability course for students who want the subject's college-level and professional applications beyond the AP exam

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How is this different from a general statistics course, and from your other Statistics and Probability course?

A: It is a different subject taught for a different purpose. Our Statistics and Probability course is a general college-level and professional course, spreadsheet-first, with no exam and no fixed unit order, ending in a capstone analysis of a dataset you choose. This AP Statistics course is exam-specific: it is organised around the nine official AP units in the order the College Board tests them, it drills the free-response section against the actual AP rubric, it dedicates real time to the Investigative Task, it teaches graphing-calculator methods the AP exam expects, and it is built for high school students working toward an AP score of 1 to 5. If your goal is the AP exam, take this course. If your goal is statistics for a data-heavy career with no exam involved, our Statistics and Probability course is the better fit, and graduates of this course who want that broader, professional-facing material are welcome there afterward.

Q: Do I need a graphing calculator, and which one?

A: Yes. A graphing calculator is expected and permitted throughout the entire AP Statistics exam, and we use one in class from week 1 onward, both for computation and because the exam expects you to read and quote calculator output correctly. A TI-84 or TI-84 Plus is the most common model and the one our examples default to, but any calculator on the College Board's approved list works, and we help you get comfortable with whichever model you already own during the first week rather than requiring a specific purchase.

Q: What is the Investigative Task?

A: It is free-response Question 6, and it is different from the other five free-response questions. Where Questions 1 through 5 test one unit's skills in a fairly direct way, the Investigative Task hands you a scenario you have not seen before and asks you to apply and extend the statistical reasoning you already have to fit it, sometimes stretching a method slightly beyond how it was originally taught. It is budgeted at roughly 30 of the 90 free-response minutes, making it worth real, focused preparation rather than being left as an afterthought, which is why we give it a dedicated week, week 23, plus additional timed practice in the mock phase.

Q: What is the exam format, and why does justification matter so much?

A: Section I is 40 multiple-choice questions in 90 minutes, worth 50 percent of the score. Section II is 6 free-response questions in 90 minutes, worth the other 50 percent: 5 short-answer questions budgeted at roughly 12 minutes each, plus the Investigative Task at around 30 minutes. The free-response rubric is built around communication, not just a final number: you are scored on justifying a conclusion with evidence from the data, interpreting a result correctly in context, and stating the conditions or assumptions a procedure depends on. A correct number with a missing or wrong justification consistently loses points, which is why every free response in this course, from week 1, is written to that standard rather than left as bare arithmetic.

Q: What is the hybrid digital format of the exam?

A: The multiple-choice section and the free-response prompts are both delivered digitally through the College Board's Bluebook application. However, your free-response answers themselves are handwritten in a paper answer booklet, which is collected separately and scored by hand. This mixed format catches students who have only ever practiced on paper or only ever practiced on screen, so our mock exams in weeks 23 and 24 are run under the same conditions: prompts read digitally, answers written out by hand, so test day is not the first time you have worked this way.

Q: How much maths do I need coming in?

A: Comfortable school algebra: rearranging a formula, working with fractions and percentages, and reading a graph. There is no calculus anywhere in AP Statistics, which surprises some students expecting a more advanced maths course. Where a result would traditionally require calculus to fully derive, the course demonstrates it through simulation and the graphing calculator instead, which is also how the AP exam itself expects the material to be handled.

Q: Will this get me a 5, or college credit?

A: No honest course can promise a specific score. AP exams are graded 1 to 5, and your result depends on your starting point, the hours you put in, and your performance on the day itself. What we can promise is honest: live teaching to the current nine-unit exam, small batches, real doubt support, free-response drilling against the official rubric, dedicated Investigative Task practice, and two full timed mock exams, plus a clear, evidence-based read on a realistic target score for you. Whether a given score earns college credit is decided by each university individually, not by us or by the College Board, so check the specific policy of the colleges you are applying to.

Q: Is this course affiliated with the College Board?

A: No. Modern Age Coders is an independent education provider and is not affiliated with, authorised by, or endorsed by the College Board. AP and Advanced Placement are trademarks of the College Board, used here only to describe the exam this course prepares students for. For official course descriptions, syllabus updates and exam information, students should always refer to the College Board and AP Central directly.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ap-statistics-maths-exam-prep-course>

=====

COURSE 21 OF 118: APP DEVELOPMENT COURSE: BUILD MOBILE & WEB APPS FROM ZERO

=====

Course link: <https://learn.modernagecoders.com/courses/complete-app-development-masterclass-college>

Subject area: Mobile & Web App Development

Designed for: College students

Who can join: Complete Beginner to Professional App Developer

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Professional App Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 12-month app development course covering iOS, Android, Flutter, React Native and backend, taking you from zero coding to launching real apps on the store.

In one line: From First Line of Code to Published Apps on App Stores

ABOUT THE PROGRAMME

This is not just an app development course, it's a complete transformation into a professional app developer and entrepreneur. Whether you're a complete beginner, designer wanting to code, entrepreneur with app ideas, or developer wanting mobile skills, this 12-month masterclass will turn you into a skilled app developer capable of building, designing, and launching successful mobile and web applications. You'll master app development from ground zero to professional level: from programming basics to advanced features, from UI/UX design to backend integration, from local development to cloud deployment, from first app to App Store success. By the end, you'll have built 30+ apps, published apps to stores, monetized your creations, and be ready for app developer roles or running your own app business.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no coding experience needed
- Complete 12-month structured curriculum
- Both iOS and Android development covered
- Flutter and React Native for cross-platform
- Native development (Swift, Kotlin) included
- UI/UX design for app developers
- Backend development for apps
- App Store and Play Store publishing mastery
- Monetization strategies (ads, IAP, subscriptions)
- Real app projects from idea to launch
- Build 30+ complete apps
- App marketing and user acquisition

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Programming Basics, UI/UX Design, First Apps
Phase 2: Cross-Platform (Months 4-6): Flutter/React Native, Backend Integration, APIs
Phase 3: Native & Advanced (Months 7-9): iOS (Swift), Android (Kotlin), Advanced Features
Phase 4: Professional (Months 10-12): Publishing, Monetization, Marketing, Business

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required
Coding experience: Absolute beginner friendly - zero coding knowledge needed
Equipment: Mac required for iOS development, any computer for Android/Flutter
Software: Xcode (Mac), Android Studio, VS Code (all free)
Accounts: Apple Developer (\$99/year), Google Play (\$25 one-time)
Time commitment: 15-20 hours/week consistently
Age: 16+ recommended
Design skills: Not required (taught in course)
Mindset: Creative, problem-solver, user-focused

WHO THIS COURSE IS FOR

Complete beginners: No coding experience, want to learn app development
Web developers: Want to transition to mobile development
Designers: UI/UX designers wanting to code their designs
Entrepreneurs: Have app ideas, want to build them
Students: Computer science or any background wanting app skills
Freelancers: Want to add app development services
Career switchers: Entering tech through mobile development
Indie developers: Want to build and publish own apps
Startup founders: Need technical co-founder skills
Professionals: Adding mobile skills to career

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATIONS & FIRST APPS (MONTHS 1-3, WEEKS 1-13) -----

Build programming foundations, master UI/UX design principles, and create your first mobile applications.

Months 1-2: Programming Basics & App Fundamentals (Week 1-8)

Introduction to App Development & Setup

Topics:

- Mobile app ecosystem overview
- iOS vs Android vs Cross-platform
- Types of apps: native, hybrid, web apps
- App development career paths
- Development environment setup
- Installing necessary tools and IDEs
- Understanding app architecture
- Mobile operating systems basics
- App lifecycle and states
- Screen sizes and resolutions
- Touch interactions and gestures
- Mobile-first design thinking
- Your first app ideas brainstorming
- App development process overview
- Version control with Git basics

Projects built:

- Environment setup checklist
- Simple 'Hello World' app
- App idea documentation
- Wireframe sketches (paper)
- Git repository creation

Practice: Research 20 successful apps, analyze their features

Programming Fundamentals (Dart/JavaScript)

Topics:

- Why Dart and JavaScript for apps?
- Variables and data types
- Operators: arithmetic, comparison, logical
- Strings and string manipulation
- Numbers and math operations
- Booleans and conditions
- If-else statements
- Switch statements
- Loops: for, while, do-while
- Lists/Arrays basics
- Maps/Objects basics
- Functions: declaration and calling
- Parameters and return values
- Arrow functions (JavaScript) / Fat arrow (Dart)
- Basic debugging techniques

Projects built:

- Calculator logic
- Temperature converter
- Todo list logic (console)
- Quiz game logic
- BMI calculator logic
- Simple games (number guessing)

Practice: Solve 100 basic programming problems

UI/UX Design for App Developers

Topics:

- UI vs UX: understanding the difference
- Mobile UI design principles
- Material Design (Android) guidelines
- Human Interface Guidelines (iOS)
- Color theory for apps
- Typography in mobile apps
- Icons and iconography
- Spacing and layout grids
- Visual hierarchy
- Navigation patterns: tabs, drawers, bottom navigation
- Common UI components: buttons, cards, lists
- Touch targets and accessibility
- Figma for app design (basics)
- Creating wireframes and mockups
- Prototyping interactive designs

Projects built:

- Design system in Figma
- App UI mockups (5 different apps)
- Icon set creation
- User flow diagrams
- Interactive prototype
- Style guide document

Practice: Design UI for 10 different app types

Introduction to Flutter Development

Topics:

- What is Flutter? Why Flutter?

- Flutter installation and setup
- Dart language review for Flutter
- Flutter architecture and widgets
- Stateless vs Stateful widgets
- Widget tree concept
- Common widgets: Text, Container, Row, Column
- Scaffold and AppBar
- Buttons: ElevatedButton, TextButton, IconButton
- Images and assets
- ListView and ListTile
- GridView for grid layouts
- Stack for layered layouts
- Padding and margin
- Hot reload and hot restart

Projects built:

- Hello Flutter app
- Personal card app
- BMI calculator app (UI + logic)
- Dice rolling app
- Quote of the day app
- Simple calculator app

Practice: Build 10 simple Flutter apps

Month 3: Intermediate Flutter & State Management (Week 9-13)

Advanced Flutter Widgets & Layouts

Topics:

- Custom widgets creation
- Widget composition
- Form widgets: TextField, Form, TextFormField
- Input validation
- DropdownButton and menus
- Date and time pickers
- Sliders and switches
- Checkbox and radio buttons
- Dialogs and bottom sheets
- SnackBar and notifications
- TabBar and TabBarView
- PageView for swiping
- Card and material design
- Themes and theming
- Dark mode implementation

Projects built:

- Todo app with add/delete
- Expense tracker app
- Form-based registration app
- Settings page with preferences
- Multi-page app with tabs
- Dark mode enabled app

Practice: Create 15 different UI screens

State Management in Flutter

Topics:

- Understanding state in Flutter
- setState and local state
- InheritedWidget basics
- Provider package introduction
- ChangeNotifier and ChangeNotifierProvider
- Consumer and Selector widgets
- Multiple providers
- Riverpod introduction (modern alternative)
- BLoC pattern basics
- GetX state management
- Choosing the right state management
- Global state vs local state
- State persistence basics
- Debugging state issues
- Best practices for state management

Projects built:

- Counter app with Provider
- Shopping cart with state management
- Todo app with Provider
- Theme switcher with Provider
- Multi-screen state sharing
- Complete app with Riverpod

Practice: Refactor 10 apps with proper state management

Navigation & Routing

Topics:

- Navigation basics in Flutter
- Navigator.push and pop
- Named routes

- Passing data between screens
- Returning data from screens
- Navigation with arguments
- Bottom navigation bar implementation
- Drawer navigation
- Tab navigation
- Deep linking basics
- GoRouter package
- Navigation 2.0 introduction
- Nested navigation
- Route guards and authentication flow
- Transition animations

Projects built:

- Multi-screen app with navigation
- Bottom nav app (3+ screens)
- Drawer menu app
- Login flow with navigation
- PHASE 1 MINI CAPSTONE: Complete Multi-screen App
- E.g., Recipe app, News app, or Portfolio app

Assessment: Phase 1 Exam - Programming, UI/UX, Flutter basics, Navigation

----- PHASE 2: CROSS-PLATFORM DEVELOPMENT & BACKEND INTEGRATION (MONTHS 4-6, WEEKS 14-26) -----

Master cross-platform development, backend integration, APIs, databases, and real-world app features.

Months 4-5: Backend Integration & Data Persistence (Week 14-22)

Local Data Storage

Topics:

- Data persistence importance
- SharedPreferences for simple data
- Key-value storage patterns
- SQLite database basics
- sqflite package in Flutter
- CRUD operations with SQLite
- Database schema design
- Migrations and versioning
- Hive (NoSQL) database
- Sembast for Flutter
- Choosing the right storage solution
- Data encryption basics
- Secure storage for sensitive data
- File storage and management
- Cache management

Projects built:

- Notes app with SQLite
- Todo app with local database
- Expense tracker with Hive
- User preferences with SharedPreferences
- Offline-first app
- Encrypted data storage app

Practice: Build 10 apps with different storage methods

REST APIs & HTTP Networking

Topics:

- Understanding REST APIs
- HTTP methods: GET, POST, PUT, DELETE
- API endpoints and routes
- HTTP package in Flutter
- Dio package for advanced networking
- Making GET requests
- Making POST requests with body
- Headers and authentication tokens
- Query parameters
- JSON parsing in Dart
- Serialization and deserialization
- Error handling in API calls
- Loading states and UX
- Retry logic and timeout
- API rate limiting handling

Projects built:

- Weather app with API
- News app (NewsAPI integration)
- Movie database app (TMDB API)
- Currency converter with live rates
- Quote app with API
- GitHub profile viewer
- Restaurant finder (Google Places API)

Practice: Integrate 15 different public APIs

Firestore Integration

Topics:

- Firestore introduction and setup
- Firestore project creation
- Firestore Authentication: email/password
- Google Sign-In integration
- Phone authentication
- Social logins: Facebook, Apple
- Firestore database (NoSQL)
- Firestore CRUD operations
- Real-time listeners
- Firestore queries and filtering
- Cloud Storage for files
- Image upload and retrieval
- Firestore Cloud Messaging (FCM)
- Push notifications implementation
- Firestore Analytics basics

Projects built:

- User authentication app
- Chat application with Firestore
- Social media app (basic)
- File sharing app
- Real-time todo app
- Push notification app
- Complete Firestore-backed app

Practice: Build 10 Firestore-integrated apps

Advanced App Features

Topics:

- Camera and photo library access
- `image_picker` package
- Taking photos and videos
- Location services and GPS
- Google Maps integration
- Displaying maps and markers
- Geolocation and tracking
- QR code scanning and generation
- Biometric authentication (fingerprint, face)
- In-app browser (`WebView`)
- Audio playback and recording
- Video playback
- Sensors: accelerometer, gyroscope
- Background tasks
- App shortcuts and widgets (home screen)

Projects built:

- Instagram-like photo app
- Location tracking app
- Maps-based app (find nearby)
- QR code scanner app
- Music player app
- Fitness tracker (basic)
- AR basics with camera

Practice: Implement 15 advanced features across apps

Animations & Advanced UI

Topics:

- Implicit animations in Flutter
- `AnimatedContainer`, `AnimatedOpacity`
- `TweenAnimationBuilder`
- Explicit animations with `AnimationController`
- Animation curves
- Hero animations
- Page transitions
- Custom animations
- Lottie animations integration
- Rive animations (interactive)
- Shimmer effects for loading
- Custom painters for graphics
- Gesture detectors and interactions
- Draggable widgets
- Performance optimization for animations

Projects built:

- Onboarding screens with animations
- Animated app intro
- Loading animations collection
- Interactive animation app
- Animated charts and graphs
- Game with animations (simple)
- Beautiful UI app with micro-interactions

Practice: Add animations to 10 previous apps

Month 6: React Native & Alternative Platforms (Week 23-26)

React Native Introduction

Topics:

- What is React Native?
- React Native vs Flutter comparison
- React Native setup and CLI
- JSX syntax
- Components in React Native
- Functional vs Class components
- React Hooks: useState, useEffect
- Props and state
- Styling in React Native
- Flexbox layouts
- Common components: View, Text, Image, ScrollView
- FlatList and SectionList
- TextInput and forms
- TouchableOpacity and buttons
- Navigation with React Navigation

Projects built:

- Hello React Native app
- Todo app in React Native
- Weather app with API
- Navigation app (multi-screen)
- Form handling app
- List-based app (contacts, recipes)

Practice: Build 10 React Native apps

Progressive Web Apps (PWA)

Topics:

- What are Progressive Web Apps?
- PWA vs native apps
- Service workers
- Offline functionality
- App manifest
- Add to home screen
- Push notifications in PWA
- Responsive design for PWA
- PWA frameworks: React, Vue, Angular
- Flutter for web (PWA)
- Testing PWA on mobile
- PWA deployment
- App-like experience on web
- Performance optimization
- PWA best practices

Projects built:

- Simple PWA with offline support
- Todo PWA
- News PWA with caching
- Flutter web app as PWA
- E-commerce PWA (basic)
- Social media PWA

Practice: Convert 5 apps to PWAs

Backend Development for Apps (Node.js/Express)

Topics:

- Why build your own backend?
- Node.js and Express basics
- RESTful API design for mobile
- Database integration: MongoDB/PostgreSQL
- User authentication (JWT)
- CRUD API endpoints
- File upload handling
- Input validation and sanitization
- Error handling in APIs
- API security best practices
- Environment variables
- CORS configuration
- Rate limiting
- API documentation
- Deployment basics (Render, Railway)

Projects built:

- Simple REST API for todo app
- User authentication API
- Blog API with CRUD
- Image upload API
- Social media backend (basic)
- E-commerce backend API
- Complete backend for Flutter app

Practice: Build backends for 8 different apps

Testing & Quality Assurance

Topics:

- Importance of testing in apps
- Unit testing in Flutter
- Widget testing
- Integration testing
- Test-driven development (TDD) basics
- Mocking and stubbing
- Testing API calls
- Testing state management
- UI testing with Flutter
- Automated testing
- Testing in React Native
- Debugging techniques
- Performance testing
- Beta testing preparation
- User acceptance testing

Projects built:

- Test suite for calculator app
- Widget tests for UI components
- Integration tests for complete app
- TDD for new feature
- Automated testing pipeline
- Bug tracking and fixing

Practice: Write tests for all major apps

Phase 2 Capstone Project

Topics:

- Full-stack app development
- Custom backend API
- Advanced features integration
- State management
- Testing coverage
- Performance optimization

Projects built:

- PHASE 2 CAPSTONE: Complete Social Media App
- Features: Auth, posts, comments, likes, user profiles, image upload, real-time updates, push notifications
- Alternative: E-commerce app with cart, checkout, orders
- Alternative: Fitness/Health tracking app with charts
- Alternative: Chat application with Firebase
- Requirements: Backend integration, 10+ screens, polished UI, tested

Assessment: Phase 2 Exam - Cross-platform development, APIs, backend, advanced features

----- PHASE 3: NATIVE DEVELOPMENT & ADVANCED TOPICS (MONTHS 7-9, WEEKS 27-39) -----

Master native iOS and Android development, advanced app features, and platform-specific capabilities.

Months 7-8: iOS Development with Swift (Week 27-35)

Swift Programming & iOS Basics

Topics:

- Introduction to iOS development
- Xcode installation and setup
- Swift programming language basics
- Variables, constants, and types
- Optionals and optional binding
- Collections: Arrays, Dictionaries, Sets
- Control flow in Swift
- Functions and closures
- Object-oriented programming in Swift
- Classes and structures
- Protocols and extensions
- iOS app architecture
- Storyboards vs SwiftUI
- View controllers lifecycle
- Interface Builder basics

Projects built:

- Swift programming exercises
- Hello iOS app
- Calculator app in Swift
- Simple UI app with storyboards
- Multi-screen app with segues

Practice: Solve 50 Swift programming problems

UIKit Development

Topics:

- UIKit framework overview
- UIViewController and view lifecycle
- UIView and subviews
- Auto Layout and constraints
- Stack views for layouts

- UITableView and UITableViewCell
- UICollectionView
- Navigation Controller
- Tab Bar Controller
- UIAlertController and dialogs
- UITextField and forms
- UIImageView and image handling
- Gesture recognizers
- Delegates and protocols
- Programmatic UI vs Storyboards

Projects built:

- Todo app with UITableView
- Photo gallery with UICollectionView
- Multi-screen app with navigation
- Tab-based app
- Form-based app with validation
- Custom table view cells

Practice: Build 12 UIKit apps

SwiftUI (Modern iOS Development)

Topics:

- SwiftUI introduction
- Declarative UI paradigm
- Views and modifiers
- State and @State property wrapper
- @Binding for two-way binding
- @ObservedObject and @StateObject
- @EnvironmentObject for global state
- Lists and ForEach
- Navigation in SwiftUI
- NavigationView and NavigationLink
- TabView for tabs
- Sheets and fullscreen covers
- Alerts and action sheets
- Animations in SwiftUI
- Combine framework basics

Projects built:

- SwiftUI todo app
- Weather app with SwiftUI
- MVVM architecture app
- List-detail navigation app
- Form app with SwiftUI
- Animated UI app
- Complete SwiftUI application

Practice: Build 15 SwiftUI apps

iOS Data & Networking

Topics:

- UserDefaults for simple storage
- Core Data for local database
- Core Data entities and relationships
- Fetch requests and predicates
- URLSession for networking
- Codable protocol for JSON
- REST API integration in iOS
- Async/await in Swift
- Combine for reactive programming
- Error handling in networking
- Image caching
- CloudKit basics
- iCloud synchronization
- Keychain for secure storage
- File management in iOS

Projects built:

- Notes app with Core Data
- API-based news app
- Weather app with networking
- Image downloading and caching
- Secure password manager
- Cloud-synced todo app
- Complete data-driven app

Practice: Build 10 data-intensive iOS apps

iOS Advanced Features

Topics:

- Camera and photo library
- Core Location and MapKit
- Push notifications (local and remote)
- Background tasks
- App extensions (Today, Share, Keyboard)
- SiriKit integration
- HealthKit basics

- ARKit for augmented reality
- Core ML for machine learning
- App clips
- Widgets (WidgetKit)
- In-app purchases (StoreKit)
- Biometric authentication (Face ID, Touch ID)
- Universal links and deep linking
- Accessibility in iOS apps

Projects built:

- Camera app with filters
- Location-based app with maps
- Push notification app
- Widget for iOS 14+
- AR experience app (basic)
- Health tracking app
- App with in-app purchases

Practice: Implement 10 advanced iOS features

Month 9: Android Development with Kotlin (Week 36-39)

Kotlin Programming & Android Basics

Topics:

- Introduction to Android development
- Android Studio setup
- Kotlin programming basics
- Variables: val vs var
- Data types and type inference
- Null safety in Kotlin
- Collections in Kotlin
- Control flow: if, when, loops
- Functions and lambdas
- Object-oriented Kotlin
- Classes, objects, and data classes
- Android project structure
- Activities and their lifecycle
- Fragments basics
- AndroidManifest.xml

Projects built:

- Kotlin programming exercises
- Hello Android app
- Simple calculator app
- Activity lifecycle app
- Multi-activity app with intents

Practice: Solve 50 Kotlin programming problems

Android UI Development

Topics:

- XML layouts in Android
- LinearLayout, RelativeLayout, ConstraintLayout
- RecyclerView and adapters
- ViewHolder pattern
- ListView (legacy)
- Material Design components
- CardView and material cards
- Bottom navigation
- Navigation drawer
- Floating action button
- Snackbar and Toast
- Dialogs and AlertDialog
- Menus and toolbars
- Styles and themes
- Vector drawables and icons

Projects built:

- RecyclerView list app
- Material Design app
- Bottom navigation app
- Drawer navigation app
- Custom adapter app
- Theming and styling app

Practice: Build 12 Android UI apps

Jetpack Compose (Modern Android UI)

Topics:

- Jetpack Compose introduction
- Composable functions
- State management in Compose
- remember and mutableStateOf
- Layouts: Column, Row, Box
- LazyColumn and LazyRow
- Modifiers in Compose
- Material Design 3 in Compose
- Navigation in Compose

- ViewModel with Compose
- Side effects and LaunchedEffect
- Theming in Compose
- Animations in Compose
- Compose vs XML comparison
- Interoperability with XML

Projects built:

- Compose todo app
- MVVM app with Compose
- Navigation app in Compose
- Animated Compose app
- Material 3 design app
- Complete Compose application

Practice: Build 15 Jetpack Compose apps

Android Data & Architecture

Topics:

- SharedPreferences in Android
- Room database (SQLite ORM)
- Room entities, DAOs, and database
- LiveData for observable data
- ViewModel for UI state
- Repository pattern
- MVVM architecture
- Retrofit for networking
- Gson/Moshi for JSON parsing
- Coroutines for async operations
- Flow for reactive streams
- Dependency injection with Hilt/Dagger
- WorkManager for background tasks
- DataStore (modern preferences)
- Architecture components

Projects built:

- Notes app with Room
- MVVM todo app
- API-based app with Retrofit
- Repository pattern implementation
- Coroutines and Flow app
- Dependency injection app
- Background task app

Practice: Build 10 architecture-compliant apps

Android Advanced & Phase 3 Capstone

Topics:

- Camera and photos in Android
- Location services and Google Maps
- Firebase Cloud Messaging
- Push notifications in Android
- Services: foreground, background
- Broadcast receivers
- Content providers
- Permissions in Android
- Biometric authentication
- In-app billing
- Android Jetpack libraries
- Accessibility in Android
- App widgets
- Deep linking

Projects built:

- PHASE 3 CAPSTONE: Native iOS + Android App
- Build same app for both iOS (Swift/SwiftUI) and Android (Kotlin/Compose)
- Option 1: Fitness tracking app
- Option 2: E-commerce app
- Option 3: Social media app
- Requirements: Native code, platform-specific features, polished UI, backend integrated

Assessment: Phase 3 Exam - Native iOS and Android development

PHASE 4: PUBLISHING, MONETIZATION & APP BUSINESS (MONTHS 10-12, WEEKS 40-52)

Master app publishing, monetization strategies, marketing, analytics, and running an app business.

Months 10-11: App Store Publishing & Monetization (Week 40-48)

App Store Publishing (iOS)

Topics:

- Apple Developer Program enrollment
- App Store Connect overview
- App ID and bundle identifier
- Certificates and provisioning profiles
- Code signing in Xcode

- App Store guidelines review
- App review process
- App metadata: title, description, keywords
- Screenshots and preview videos
- App Store optimization (ASO) basics
- Pricing and availability
- App categories and age ratings
- TestFlight for beta testing
- Preparing for app review
- Submitting the app

Projects built:

- Prepare app for submission
- Create App Store listing
- Screenshot and preview creation
- TestFlight beta test
- Submit app to App Store
- ASO optimization

Practice: Publish 2 apps to iOS App Store

Google Play Store Publishing (Android)

Topics:

- Google Play Console setup
- Developer account creation
- App signing and Play App Signing
- APK vs AAB (Android App Bundle)
- Play Store listing creation
- Store listing optimization
- Screenshots and feature graphics
- Video preview for Play Store
- Content rating questionnaire
- Privacy policy requirements
- In-app content disclosure
- Release tracks: internal, closed, open beta, production
- Staged rollout strategy
- Pre-launch report
- Publishing the app

Projects built:

- Prepare Android app for release
- Play Store listing creation
- Graphics and screenshots
- Beta testing program
- Submit app to Play Store
- Play Store optimization

Practice: Publish 2 apps to Google Play Store

App Monetization Strategies

Topics:

- Monetization models overview
- Paid apps vs free apps
- In-app advertising: AdMob integration
- Banner ads, interstitial ads, rewarded ads
- Ad mediation strategies
- In-app purchases (IAP) implementation
- Consumable vs non-consumable products
- Subscriptions: auto-renewable
- Subscription tiers and pricing
- Freemium model best practices
- Paywalls and conversion optimization
- Revenue Cat for subscription management
- Analytics for monetization
- A/B testing pricing
- Compliance: payments, taxes, regulations

Projects built:

- App with AdMob ads
- In-app purchase implementation
- Subscription-based app
- Freemium app model
- Multiple monetization app
- Revenue optimization experiment

Practice: Implement 5 different monetization strategies

Analytics & User Engagement

Topics:

- Firebase Analytics integration
- Google Analytics for apps
- Custom event tracking
- User properties and audiences
- Funnel analysis
- Retention metrics
- Mixpanel for advanced analytics
- Amplitude analytics
- Crash reporting: Firebase Crashlytics

- Sentry for error tracking
- Performance monitoring
- User feedback collection
- In-app surveys
- App rating prompts
- Push notification strategies
- Deep linking for retention

Projects built:

- Analytics dashboard setup
- Event tracking implementation
- Crash reporting integration
- User feedback system
- Push notification campaigns
- Retention optimization features
- Analytics-driven improvements

Practice: Add comprehensive analytics to all apps

App Performance & Optimization

Topics:

- App performance importance
- Flutter performance profiling
- iOS performance tools (Instruments)
- Android Profiler
- Memory leak detection
- Battery optimization
- Network performance optimization
- Image optimization and caching
- Lazy loading strategies
- Code splitting and tree shaking
- App size reduction
- Startup time optimization
- Smooth scrolling and 60fps
- Background task optimization
- Device compatibility testing

Projects built:

- Performance audit of apps
- Optimization implementations
- Memory optimization
- App size reduction project
- Startup time improvement
- Benchmark comparison

Practice: Optimize 10 apps for performance

Month 12: Marketing, Business & Career (Week 49-52)

App Marketing & User Acquisition

Topics:

- App marketing fundamentals
- Pre-launch marketing strategy
- Landing page creation
- Social media marketing for apps
- Content marketing and blogging
- App Store Optimization (ASO) advanced
- Keyword research and optimization
- A/B testing store listings
- App review sites and submissions
- Influencer marketing for apps
- Paid advertising: Google Ads, Facebook Ads
- Apple Search Ads
- Organic growth strategies
- Viral features and growth loops
- Referral programs
- Community building

Projects built:

- Complete marketing plan
- App landing page
- Social media campaign
- ASO keyword optimization
- Paid ad campaign (budget)
- Referral system implementation
- Launch strategy execution

Practice: Market and launch 3 apps

App Maintenance & Updates

Topics:

- App update strategy
- Version control and release management
- Bug fixing workflows
- Feature request management
- User feedback prioritization
- App Store updates and releases
- Backward compatibility

- Deprecation strategies
- Security updates and patches
- OS version support
- Device compatibility updates
- Legal compliance updates (GDPR, etc.)
- Server and API maintenance
- Database migrations in production
- Long-term app sustainability

Projects built:

- Update management system
- Bug tracking and fixing
- Major version release
- Security audit and fixes
- Compliance updates
- Maintenance roadmap

Practice: Maintain and update published apps

Building an App Business

Topics:

- App business models
- Solo developer vs team
- App agency model
- Product vs service business
- Niche selection for apps
- Competitive analysis
- Building an app portfolio
- Passive income with apps
- Scaling app revenue
- Outsourcing and delegation
- Client app development process
- Project estimation and pricing
- Contracts and agreements
- Financial management for app business
- Tax implications
- Selling apps and app businesses

Projects built:

- Business plan for app company
- Portfolio of apps strategy
- Client acquisition system
- Pricing and packages
- Financial projections
- Scalability roadmap

Practice: Develop business strategy

Career Development in App Development

Topics:

- App developer career paths
- Freelance app development
- Remote app developer jobs
- Working for startups vs enterprises
- App development at FAANG companies
- Building portfolio and GitHub
- Resume for app developers
- LinkedIn profile optimization
- Networking in app dev community
- Contributing to open source apps
- Creating developer content (blog, YouTube)
- Interview preparation: coding challenges
- System design for mobile apps
- Salary negotiation
- Continuous learning and upskilling

Projects built:

- Professional portfolio website
- GitHub showcase
- Resume and LinkedIn
- Technical blog posts
- Open source contributions
- Interview preparation guide

Practice: Build professional developer brand

Emerging Technologies & Future

Topics:

- AI/ML in mobile apps
- TensorFlow Lite and Core ML
- On-device machine learning
- Augmented Reality (ARKit, ARCore)
- Virtual Reality apps
- IoT and app integration
- Wearable app development (Watch OS, Wear OS)
- Foldable device optimization
- 5G implications for apps
- Edge computing for apps

- AI-powered and agentic apps
- Voice-first applications
- Accessibility innovations
- Sustainable app development
- Future of cross-platform

Projects built:

- AR app prototype
- ML-powered app feature
- Wearable app (Apple Watch/Wear OS)
- Voice-enabled app
- Blockchain-based app (basic)
- Innovation project

Practice: Explore emerging technologies

FINAL CAPSTONE & LAUNCH (Week 49-52 (distributed))

Final Capstone Planning

Topics:

- App ideation and validation
- Market research
- Competitive analysis
- Feature prioritization
- Technical architecture design
- UI/UX design complete
- Development roadmap
- Monetization strategy
- Marketing plan
- Launch strategy

Projects built:

- Complete app concept document
- Market research report
- Full app design (Figma)
- Technical specification
- Business plan
- Go-to-market strategy

Practice: Validate and plan app thoroughly

Final Capstone Development

Topics:

- Sprint planning
- Agile development process
- Version control workflow
- Code quality and reviews
- Testing throughout development
- Backend development/integration
- Feature implementation
- Bug fixing and polish
- Performance optimization
- Security hardening

Projects built:

- FINAL CAPSTONE: Complete Production App
- Your own app idea OR choose:
- Option 1: Niche social media app
- Option 2: SaaS mobile app
- Option 3: E-commerce marketplace app
- Option 4: Productivity/utility app
- Requirements: Cross-platform OR native iOS + Android, backend, monetization, analytics, polished UI, App Store ready

Final Capstone Launch

Topics:

- Final testing and QA
- Beta testing program
- App Store submission
- Play Store submission
- Landing page launch
- Marketing campaign execution
- Press release and outreach
- Social media launch
- Community engagement
- Monitoring and support

Deliverables:

- Published app on App Store
- Published app on Play Store
- App website/landing page
- Marketing materials
- Analytics dashboard
- Support system setup
- User documentation
- Launch metrics tracking

Post-Launch & Career Success

Topics:

- Post-launch monitoring
- User feedback collection
- Iterative improvements
- Growth and optimization
- Portfolio presentation
- Case study creation
- Career opportunities
- Next steps planning

Deliverables:

- Professional portfolio with 30+ apps
- Published apps (minimum 3)
- Case studies and documentation
- Technical blog or YouTube channel
- Active GitHub profile
- Resume and LinkedIn optimized
- Network of app developers
- Career roadmap and goals

Assessment: FINAL COMPREHENSIVE EXAM - Complete app development mastery

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 30+ foundational apps - UI, navigation, basic features
- Phase 2: 25+ integrated apps - APIs, backend, advanced features
- Phase 3: 20+ native apps - iOS Swift/SwiftUI, Android Kotlin/Compose
- Phase 4: 15+ production apps - published, monetized, marketed
- Total: 90+ apps from simple to production-grade

Total projects: 90+ mobile and web applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Cross-Platform: Flutter, Dart, React Native, JavaScript, PWA
- iOS Native: Swift, SwiftUI, UIKit, Xcode, iOS SDK
- Android Native: Kotlin, Jetpack Compose, Android SDK, Android Studio
- UI/UX Design: Figma, Material Design, Human Interface Guidelines, animations
- Backend: Node.js, Express, REST APIs, Firebase, databases
- State Management: Provider, Riverpod, Redux, MVVM, BLoC
- Data: SQLite, Room, Core Data, Firebase Firestore, API integration
- Features: Camera, maps, notifications, payments, authentication, AR basics
- Publishing: App Store, Play Store, ASO, beta testing
- Monetization: Ads (AdMob), IAP, subscriptions, freemium models
- Analytics: Firebase Analytics, Crashlytics, user tracking
- Marketing: ASO, social media, paid ads, growth strategies
- Tools: Git, Xcode, Android Studio, VS Code, Postman, analytics platforms

HOW A TYPICAL WEEK IS SPLIT

Video tutorials: 4-6 hours
 Hands on coding: 8-10 hours
 Projects: 4-6 hours
 Design work: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding and app reviews
 Mentorship: 1-on-1 guidance from professional app developers
 Community: Active community of app developers
 Code review: Expert review of app projects
 Design review: UI/UX feedback on designs
 Publishing help: Assistance with app store submissions
 Marketing guidance: Support with app marketing and ASO
 Lifetime access: All content, updates, new platform features
 Job support: Resume review, interview prep, job referrals
 Business guidance: Help with freelancing and app business

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 total)
 Final certificate: Professional App Developer Certification
 Specialization: Choose: iOS Specialist, Android Specialist, or Cross-Platform Expert
 Published apps: Minimum 3 published apps required
 Portfolio: 30+ documented apps
 Digital badge: LinkedIn shareable badge
 Industry recognized: Recognized by tech companies and startups

WHERE THIS CAN LEAD

- Mobile App Developer (iOS/Android)

- Flutter Developer
- React Native Developer
- iOS Developer (Swift/SwiftUI)
- Android Developer (Kotlin/Jetpack Compose)
- Full Stack App Developer
- App Development Freelancer
- Mobile UI/UX Developer
- App Entrepreneur / Indie Developer
- Mobile Team Lead
- App Development Agency Owner
- Technical Co-founder
- App Consultant
- Mobile Solutions Architect

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which technologies does the course cover?

A: All the major paths: cross-platform development with Flutter and React Native, native iOS with Swift and native Android with Kotlin, plus Firebase and backend integration, and finally publishing and monetization. You graduate understanding when to choose each stack, which is itself a common interview question.

Q: Will I publish a real app during the course?

A: That is the goal of phase 4: the final months cover app store preparation, review guidelines, launch and monetization, and you take one of your projects through the process. Along the way you build multiple portfolio apps with real backends, authentication and APIs.

Q: Can I join with zero coding experience?

A: Yes, phase 1 starts with programming basics and UI/UX fundamentals before any framework appears. The 12-month structure exists precisely so beginners can build up honestly; students with prior coding simply move faster through the early weeks.

Q: How does this help me get hired or start freelancing?

A: You finish with working apps in a GitHub portfolio, experience across cross-platform and native stacks, and interview preparation for mobile roles. Mobile development also has a real freelance market where a portfolio of shipped apps wins work. We do not guarantee jobs or client income; we make sure you have genuine evidence of skill.

Q: How do I try before enrolling?

A: Book a free live demo class at learn.modernagecoders.com/contact; it is a real session with an instructor and carries no commitment. If you have questions first, WhatsApp us at +91 91233 66161.

Q: Are the classes live?

A: Yes, every class is a live 1-hour online session with an instructor and screen sharing, never a pre-recorded video. You build during class, get unblocked immediately, and choose evening or weekend slots that fit around college or a job.

Q: What certification do I receive?

A: A completion certificate as an app developer, plus the thing hiring managers actually inspect: a GitHub-ready portfolio of working apps, and if you complete phase 4, an app taken through the store publishing process.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-app-development-masterclass-college>

COURSE 22 OF 118: ARTIFICIAL INTELLIGENCE COURSE: CLASSICAL AI TO AGI

Course link: <https://learn.modernagecoders.com/courses/artificial-intelligence-complete-masterclass-college>
 Subject area: Artificial Intelligence & Intelligent Systems
 Designed for: College students
 Who can join: Complete Beginner to AI Research Scientist
 Course length: 12 months (52 weeks)
 Time needed: 25-30 hours/week recommended
 Certificate: Advanced Artificial Intelligence Certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
 Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 12-month Artificial Intelligence course from search algorithms to AGI concepts: classical

and modern AI, intelligent systems, robotics and AI ethics.

In one line: From Classical AI to Artificial General Intelligence Research

ABOUT THE PROGRAMME

This is not just an AI course, it's a complete journey through the entire landscape of Artificial Intelligence. Whether you're a complete beginner, computer science student, researcher, or professional wanting deep AI expertise, this 12-month masterclass will transform you into an AI expert with comprehensive knowledge spanning from classical symbolic AI to modern neural approaches, from theoretical foundations to practical intelligent systems. You'll master AI from philosophical foundations to cutting-edge research: from search algorithms to knowledge representation, from expert systems to autonomous agents, from logic and reasoning to probabilistic AI, from planning to learning, from single agents to multi-agent systems, from narrow AI to AGI concepts. By the end, you'll have built 50+ intelligent systems, understood AI at the deepest level, published research, and be prepared for AI research scientist roles or PhD programs.

WHAT MAKES THIS COURSE DIFFERENT

- Comprehensive coverage: Classical + Modern AI
- Theoretical foundations AND practical implementation
- Covers all AI paradigms: symbolic, connectionist, hybrid
- Philosophy and ethics integrated throughout
- Research methodology and paper writing
- Mathematics for AI taught rigorously
- Robotics and embodied AI included
- Multi-agent systems and game theory
- Cognitive architectures and human-level AI
- AGI (Artificial General Intelligence) concepts
- Industry applications across all domains
- 50+ intelligent systems built from scratch
- Preparation for AI research careers and PhD

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): AI History, Search, Logic, Knowledge Representation, Problem Solving
Phase 2: Reasoning & Learning (Months 4-6): Probabilistic AI, Planning, Reasoning Under Uncertainty, ML Integration
Phase 3: Intelligent Systems (Months 7-9): Expert Systems, Agents, Robotics, Vision, NLP, Multi-Agent Systems
Phase 4: Advanced AI (Months 10-12): Cognitive AI, AGI, Research, Ethics, Philosophy, Emerging Frontiers

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Bachelor's degree in CS/Engineering/Math/Physics recommended (not required)
Mathematics: Comfort with calculus, linear algebra, probability (will be taught/reviewed)
Programming: Basic programming knowledge helpful (Python taught from scratch for AI)
Logic: Basic logical thinking (formal logic taught in course)
Commitment: Very high - this is research-level training
Time: 25-30 hours/week consistently for 12 months
Mindset: Intellectual curiosity, love of theory + practice, research orientation
Equipment: High-performance computer recommended, cloud computing provided
Reading: Comfort reading academic papers and textbooks
Writing: Academic writing skills (developed during course)

WHO THIS COURSE IS FOR

Phd aspirants: Preparing for AI/CS PhD programs at top universities
Researchers: Want comprehensive AI research training
Cs students: Advanced undergrad or master's students in computer science
Ai professionals: Industry professionals wanting deep theoretical knowledge
Career switchers: STEM backgrounds transitioning to AI research
Academics: Faculty wanting to add AI research to their portfolio
Engineers: Want to move from applied ML to AI research
Innovators: Want to develop novel AI systems and algorithms
Philosophers: Interest in AI, mind, consciousness, ethics
Visionaries: Want to contribute to AGI development
Serious learners: Committed to mastering AI at the deepest level

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CLASSICAL AI FOUNDATIONS (MONTHS 1-3, WEEKS 1-13)

Build deep understanding of AI foundations, search algorithms, logic, knowledge representation, and problem-solving.

Months 1-2: AI Foundations & Search Algorithms (Week 1-8)

Introduction to Artificial Intelligence

Topics:

- What is Artificial Intelligence? Definitions and perspectives
- History of AI: from Dartmouth Conference to present
- AI winters and AI springs
- Turing Test and philosophical foundations
- Strong AI vs Weak AI
- Narrow AI, General AI (AGI), Super AI
- Symbolic AI vs Connectionist AI vs Hybrid AI
- AI paradigms: logic-based, knowledge-based, learning-based
- Rational agents and intelligent behavior
- PEAS framework: Performance, Environment, Actuators, Sensors
- Agent types: simple reflex, model-based, goal-based, utility-based
- Agent environments: fully observable, deterministic, episodic, static
- AI applications across industries
- AI vs ML vs DL: relationships and differences
- State of AI: current capabilities and limitations
- Ethics and societal impact overview

Projects built:

- Simple reflex agent implementation
- Model-based agent for grid world
- Agent environment simulator
- AI timeline and evolution visualization
- Turing Test chatbot (rule-based)

Practice: Study 20 landmark AI papers from history

Search Algorithms - Part 1 (Uninformed Search)

Topics:

- Problem-solving as search
- State space representation
- Problem formulation: initial state, actions, goal test, path cost
- Tree search vs graph search
- Search strategies evaluation: completeness, optimality, time, space
- Breadth-First Search (BFS): algorithm and analysis
- Uniform Cost Search (UCS)
- Depth-First Search (DFS): algorithm and analysis
- Depth-Limited Search (DLS)
- Iterative Deepening DFS (IDDFS)
- Bidirectional search
- Comparing uninformed search strategies
- Search in continuous spaces
- Memory-bounded search
- Real-world applications: route finding, puzzle solving

Projects built:

- 8-puzzle solver with BFS/DFS
- Maze solver with multiple algorithms
- Route finding system (city maps)
- N-Queens problem solver
- Missionaries and Cannibals problem
- Search visualization tool
- Performance comparison framework

Practice: Implement all uninformed search algorithms from scratch

Search Algorithms - Part 2 (Informed/Heuristic Search)

Topics:

- Heuristic functions and admissibility
- Greedy Best-First Search
- A* Search: algorithm, optimality proof
- Heuristic design and dominance
- Memory-bounded heuristic search: IDA*, RBFS, SMA*
- Weighted A* and bounded suboptimal search
- Bidirectional heuristic search
- Pattern databases for heuristics
- Local search algorithms
- Hill-climbing and variants
- Simulated annealing
- Genetic algorithms for search
- Local beam search
- Constraint satisfaction in search
- Search in game playing (preview)

Projects built:

- A* pathfinding implementation
- 15-puzzle solver with pattern databases
- Route optimization with A*
- Simulated annealing for TSP
- Genetic algorithm for optimization
- Hill-climbing variants comparison

- Heuristic function designer tool

Practice: Solve 30 search problems with optimal algorithms

Adversarial Search & Game Playing

Topics:

- Game theory basics for AI
- Game trees and game search
- Minimax algorithm
- Alpha-beta pruning
- Move ordering for better pruning
- Iterative deepening in games
- Evaluation functions for game states
- Quiescence search
- Transposition tables
- Monte Carlo Tree Search (MCTS)
- Upper Confidence Bounds (UCB)
- MCTS in AlphaGo
- Stochastic games and expectiminimax
- Partially observable games
- Multi-player games
- Real-time decision making in games

Projects built:

- Tic-Tac-Toe with minimax
- Chess AI with alpha-beta pruning
- Checkers AI implementation
- Connect-4 game AI
- Go AI with MCTS (simplified)
- Poker bot (partial information)
- Game AI framework with multiple algorithms

Practice: Build 10 different game-playing AI agents

Month 3: Logic & Knowledge Representation (Week 9-13)

Propositional Logic

Topics:

- Knowledge-based agents
- Logic as knowledge representation
- Propositional logic: syntax and semantics
- Logical connectives: AND, OR, NOT, IMPLIES, IFF
- Truth tables and logical equivalence
- Inference in propositional logic
- Inference rules: Modus Ponens, And-Elimination
- Resolution and refutation
- Conjunctive Normal Form (CNF)
- Horn clauses and forward/backward chaining
- SAT solvers and applications
- DPLL algorithm
- WalkSAT and local search for SAT
- Applications: circuit design, planning, verification
- Limitations of propositional logic

Projects built:

- Propositional logic inference engine
- SAT solver implementation
- Logic-based puzzle solver (Sudoku)
- Wumpus World agent with logic
- Circuit verification system
- Resolution theorem prover
- Knowledge base system

Practice: Solve 40 logical reasoning problems

First-Order Logic & Inference

Topics:

- First-Order Logic (FOL) syntax
- Predicates, functions, quantifiers
- Universal and existential quantification
- Semantics and interpretation
- Using FOL for knowledge representation
- Unification algorithm
- Generalized Modus Ponens
- Forward chaining in FOL
- Backward chaining in FOL
- Resolution in FOL
- Skolemization and CNF conversion
- Prolog and logic programming
- Herbrand universe and interpretations
- Completeness and decidability
- Description logics
- Semantic networks and frames

Projects built:

- FOL inference engine
- Prolog-like system in Python
- Automated theorem prover

- Knowledge base with FOL
- Family tree reasoning system
- Medical diagnosis system (logic-based)
- Natural language to FOL translator (simple)

Practice: Build 10 knowledge-based systems with FOL

Constraint Satisfaction Problems

Topics:

- CSP framework: variables, domains, constraints
- Examples: map coloring, N-Queens, scheduling
- Backtracking search for CSP
- Variable ordering heuristics: MRV, degree heuristic
- Value ordering: least-constraining-value
- Inference in CSP: forward checking
- Arc consistency (AC-3 algorithm)
- Path consistency and k-consistency
- Local search for CSP: min-conflicts
- Constraint propagation
- Structure in CSP: tree-structured, nearly tree-structured
- Cutset conditioning
- Temporal CSPs
- Soft constraints and optimization
- Applications: scheduling, resource allocation, configuration

Projects built:

- N-Queens CSP solver
- Sudoku solver with constraint propagation
- Map coloring problem
- Course scheduling system
- Resource allocation optimizer
- Timetable generator
- PHASE 1 MINI CAPSTONE: Intelligent Planning System with CSP

Assessment: Phase 1 Exam - Search, Logic, Knowledge Representation, CSP

----- PHASE 2: REASONING, PLANNING & PROBABILISTIC AI (MONTHS 4-6, WEEKS 14-26) -----

Master probabilistic reasoning, planning algorithms, decision theory, and uncertainty management in AI.

Months 4-5: Uncertainty & Probabilistic Reasoning (Week 14-22)

Probability Theory for AI

Topics:

- Uncertainty in AI systems
- Probability basics: events, axioms, rules
- Conditional probability and Bayes' Rule
- Bayesian reasoning
- Random variables: discrete and continuous
- Probability distributions
- Joint probability distributions
- Independence and conditional independence
- Inference using full joint distributions
- Marginalization and conditioning
- Naive Bayes classifier
- Bayesian networks introduction
- Belief propagation basics
- Applications: diagnosis, prediction, classification
- Probabilistic reasoning under uncertainty

Projects built:

- Bayesian inference engine
- Naive Bayes spam filter
- Medical diagnosis system (probabilistic)
- Weather prediction system
- Probabilistic reasoning toolkit
- Bayesian calculator and visualizer

Practice: Solve 50 probabilistic reasoning problems

Bayesian Networks

Topics:

- Bayesian Network structure and semantics
- Conditional independence in Bayes nets
- Constructing Bayesian networks
- Compact conditional distributions: CPTs
- Exact inference in Bayesian networks
- Inference by enumeration
- Variable elimination algorithm
- Clustering algorithms for inference
- Junction tree algorithm
- Approximate inference: sampling methods
- Direct sampling and rejection sampling
- Likelihood weighting
- Markov Chain Monte Carlo (MCMC)

- Gibbs sampling
- Learning Bayesian networks from data
- Parameter learning: MLE and MAP
- Structure learning: score-based, constraint-based

Projects built:

- Bayesian Network inference engine
- Variable elimination implementation
- MCMC sampler for Bayes nets
- Medical diagnosis Bayes net
- Risk assessment system
- Bayes net learning from data
- Decision support system with Bayes nets

Practice: Build 15 Bayesian network applications

Markov Models & Temporal Reasoning

Topics:

- Time and uncertainty
- Markov processes and Markov assumption
- Hidden Markov Models (HMM)
- HMM representation and semantics
- Forward algorithm for filtering
- Viterbi algorithm for most likely sequence
- Forward-backward algorithm for smoothing
- Baum-Welch algorithm for learning HMMs
- Applications: speech recognition, POS tagging
- Kalman filters for continuous state
- Extended Kalman filters
- Particle filters (Sequential Monte Carlo)
- Dynamic Bayesian Networks (DBN)
- Temporal reasoning and prediction
- Applications: tracking, localization, time-series

Projects built:

- HMM for speech recognition (simplified)
- POS tagger with HMM
- Robot localization with Kalman filter
- Particle filter for tracking
- Weather prediction with HMM
- Stock market HMM (simplified)
- Dynamic Bayesian network system

Practice: Implement all temporal reasoning algorithms

Decision Theory & Utility

Topics:

- Making decisions under uncertainty
- Utility theory and preferences
- Utility functions and axioms
- Maximum Expected Utility (MEU)
- Decision networks (influence diagrams)
- Value of information
- Value of perfect information (VPI)
- Sequential decision problems
- Markov Decision Processes (MDPs)
- MDP formulation: states, actions, transitions, rewards
- Bellman equations
- Value iteration algorithm
- Policy iteration algorithm
- Partially Observable MDPs (POMDPs)
- POMDP value iteration
- Applications: robotics, game playing, resource allocation

Projects built:

- Decision network solver
- MDP solver (value iteration, policy iteration)
- Grid world MDP
- Inventory management with MDPs
- Robot navigation with POMDP
- Medical treatment decision system
- Resource allocation optimizer

Practice: Solve 25 decision-theoretic problems

Planning Under Uncertainty

Topics:

- Planning with probabilistic effects
- Contingency planning
- Conditional plans and execution monitoring
- Replanning strategies
- Online planning and acting
- Information gathering actions
- Exploration vs exploitation
- Multi-armed bandit problems
- UCB algorithms for exploration
- Monte Carlo planning
- Rollout algorithms

- Hierarchical planning under uncertainty
- Applications: robotics, autonomous systems

Projects built:

- Contingent planner
- Multi-armed bandit solver
- Robot exploration system
- Adaptive planning agent
- Online planning framework
- Monte Carlo planner

Practice: Build 10 planning under uncertainty systems

Month 6: Classical Planning & Knowledge Integration (Week 23-26)

Classical Planning

Topics:

- Planning problem representation
- STRIPS representation
- Actions: preconditions and effects
- State-space search for planning
- Forward (progression) state-space search
- Backward (regression) state-space search
- Heuristics for planning: ignore-preconditions, ignore-delete-lists
- Plan-space planning (partial-order planning)
- Graphplan algorithm
- Planning graphs and graph heuristics
- SAT-based planning
- PDDL (Planning Domain Definition Language)
- Hierarchical Task Network (HTN) planning
- Temporal planning
- Planning with resources and time
- Multi-agent planning basics

Projects built:

- STRIPS planner implementation
- Graphplan algorithm
- Blocks world planner
- Logistics planning system
- Robot task planner
- SAT-based planner
- HTN planner (simplified)

Practice: Solve 20 classical planning problems

Ontologies & Knowledge Engineering

Topics:

- Knowledge engineering process
- Ontology definition and components
- Upper ontologies: Cyc, SUMO
- OWL (Web Ontology Language)
- RDF and semantic web
- Knowledge graphs
- Ontology design patterns
- Domain modeling
- Taxonomies and hierarchies
- Relationships and properties
- Reasoning with ontologies
- Ontology alignment and merging
- Protégé for ontology development
- SPARQL for querying knowledge
- Applications: healthcare, e-commerce, IoT

Projects built:

- Domain ontology creation (healthcare/e-commerce)
- Knowledge graph builder
- Ontology-based reasoning system
- Semantic search engine
- Question answering with knowledge graphs
- Ontology alignment tool
- SPARQL query system

Practice: Build 5 ontologies for different domains

Learning in AI Systems

Topics:

- Learning agents architecture
- Supervised learning integration in AI
- Decision tree learning: ID3, C4.5
- Information gain and entropy
- Ensemble learning in AI
- Unsupervised learning for AI: clustering
- Reinforcement learning fundamentals (AI perspective)
- Q-learning in AI agents
- Temporal difference learning
- Exploration strategies
- Function approximation in RL
- Deep RL overview for AI

- Transfer learning in AI systems
- Meta-learning and learning to learn
- Hybrid symbolic-neural systems

Projects built:

- Decision tree AI agent
- Reinforcement learning grid world
- Q-learning robot navigator
- Learning agent framework
- Hybrid reasoning system (logic + learning)
- Transfer learning AI agent
- Meta-learning experiment

Practice: Integrate learning into 15 AI systems

Explanation & Interpretability in AI

Topics:

- Explainable AI (XAI) importance
- Explanation in expert systems
- Trace-based explanations
- Rule-based explanation generation
- Argumentation and justification
- Counterfactual explanations
- Contrastive explanations
- Transparency vs performance tradeoffs
- LIME for model explanations
- SHAP values in AI
- Attention mechanisms for interpretability
- Causal reasoning for explanations
- User interfaces for explanations
- Evaluating explanation quality
- Regulatory requirements for explainability

Projects built:

- Explainable expert system
- Explanation generator for decisions
- Argumentation framework
- Counterfactual explanation system
- XAI dashboard
- Causal reasoning engine
- Interactive explanation interface

Practice: Add explanations to all AI systems built

Phase 2 Capstone Project

Topics:

- Integrated AI system design
- Combining multiple AI techniques
- Reasoning under uncertainty
- Planning and decision-making
- Learning and adaptation
- Explanation generation

Projects built:

- PHASE 2 CAPSTONE: Intelligent Decision Support System
- Requirements: Bayesian reasoning, planning, decision theory, learning, explanations
- Option 1: Medical diagnosis and treatment planning system
- Option 2: Autonomous robot mission planner
- Option 3: Smart home energy management system
- Option 4: Financial portfolio advisor with uncertainty

Assessment: Phase 2 Exam - Probabilistic AI, Planning, Decision Theory, Learning

PHASE 3: INTELLIGENT SYSTEMS & APPLICATIONS (MONTHS 7-9, WEEKS 27-39)

Master expert systems, intelligent agents, robotics, computer vision, NLP, and multi-agent systems.

Months 7-8: Expert Systems & Intelligent Agents (Week 27-35)

Expert Systems Architecture

Topics:

- Expert systems overview and history
- MYCIN, DENDRAL, XCON case studies
- Expert system architecture: knowledge base, inference engine, UI
- Rule-based expert systems
- Forward chaining systems
- Backward chaining systems
- Conflict resolution strategies
- Certainty factors and fuzzy logic
- Dempster-Shafer theory
- Knowledge acquisition bottleneck
- Knowledge elicitation techniques
- Knowledge engineering methodology
- Explanation facilities in expert systems
- Expert system shells
- Modern applications of expert systems

- Limitations and decline of expert systems

Projects built:

- Medical diagnosis expert system
- Legal reasoning expert system
- Technical troubleshooting system
- Financial advisory expert system
- Expert system shell development
- Rule-based configurator
- Fuzzy logic controller

Practice: Build 10 domain-specific expert systems

Intelligent Agent Architectures

Topics:

- Agent theory and design
- Deliberative agents (BDI: Belief-Desire-Intention)
- Reactive agents and subsumption architecture
- Hybrid agent architectures
- Layered architectures: InteRRaP, TouringMachines
- Agent communication languages (ACL)
- FIPA specifications
- Agent reasoning cycles
- Agent planning and execution
- Agent learning and adaptation
- Situated agents and embodiment
- Software agents vs physical agents
- Agent-oriented programming
- Agent development frameworks: JADE, Jason
- Applications: virtual assistants, bots, automation

Projects built:

- BDI agent implementation
- Reactive agent for dynamic environment
- Hybrid agent architecture
- Communicating agents system
- Personal assistant agent
- Agent-based automation system
- Situated agent in simulation

Practice: Design and implement 12 intelligent agents

Robotics & Embodied AI

Topics:

- AI in robotics overview
- Robot perception: sensors and processing
- Computer vision for robotics
- Object recognition and tracking
- SLAM (Simultaneous Localization and Mapping)
- Robot motion planning: configuration space
- Path planning algorithms for robots
- Potential field methods
- Probabilistic roadmaps (PRM)
- Rapidly-exploring Random Trees (RRT)
- Robot control architectures
- Behavior-based robotics
- Robot learning: imitation, reinforcement
- Human-robot interaction
- ROS (Robot Operating System) basics
- Applications: autonomous vehicles, drones, industrial robots

Projects built:

- Robot simulator with AI planning
- Path planning visualizer (PRM, RRT)
- SLAM implementation (simplified)
- Behavior-based robot controller
- Robot learning from demonstration
- Autonomous navigation system
- ROS-based AI agent (if feasible)

Practice: Implement 10 robotics AI algorithms

Computer Vision for AI Systems

Topics:

- Vision as AI problem
- Image formation and camera models
- Image processing fundamentals
- Edge detection: Canny, Sobel
- Feature detection: SIFT, SURF, ORB
- Image segmentation techniques
- Object recognition classical methods
- Template matching
- Hough transform for shape detection
- Optical flow and motion analysis
- Stereo vision and depth perception
- 3D reconstruction from images
- Deep learning for vision (integration)
- Scene understanding and reasoning

- Visual reasoning and question answering
- Applications: surveillance, medical imaging, autonomous systems

Projects built:

- Object detector (classical methods)
- Face recognition system
- Motion tracking application
- Image segmentation tool
- 3D reconstruction (simple)
- Visual reasoning system
- Autonomous vehicle perception (simulated)

Practice: Build 12 computer vision AI applications

Natural Language Understanding (AI Perspective)

Topics:

- Language as AI challenge
- Syntax: parsing and grammars
- Context-free grammars
- Parsing algorithms: CKY, Earley
- Semantic analysis and representation
- First-order logic for semantics
- Lambda calculus for meaning
- Discourse and pragmatics
- Coreference resolution
- Information extraction
- Question answering systems
- Dialogue systems and conversational AI
- Natural language generation
- Machine translation (symbolic approaches)
- Integrating NLP with knowledge systems
- Semantic parsing and understanding

Projects built:

- Parser for natural language
- Semantic representation generator
- Question answering system (logic-based)
- Dialogue system with state management
- Information extraction system
- Natural language interface to database
- Story understanding system

Practice: Build 10 NLU systems with AI techniques

Month 9: Multi-Agent Systems & Collective Intelligence (Week 36-39)

Multi-Agent Systems Fundamentals

Topics:

- Multi-agent systems (MAS) introduction
- Agent interactions and coordination
- Cooperation, collaboration, competition
- Agent communication protocols
- Speech acts and performatives
- Contract Net Protocol
- Negotiation and bargaining
- Auction mechanisms
- Coalition formation
- Task allocation in MAS
- Distributed problem solving
- Distributed constraint satisfaction
- Multi-agent planning
- Emergent behavior in MAS
- Applications: smart grids, traffic systems, e-commerce

Projects built:

- Multi-agent coordination system
- Negotiation protocol implementation
- Auction system with agents
- Coalition formation simulator
- Distributed task allocation
- Multi-agent marketplace
- Swarm intelligence application

Practice: Implement 12 multi-agent scenarios

Game Theory for Multi-Agent AI

Topics:

- Game theory foundations
- Normal form games
- Nash equilibrium concept
- Finding Nash equilibria
- Dominant strategies
- Mixed strategies
- Extensive form games
- Subgame perfect equilibrium
- Repeated games and folk theorem
- Evolutionary game theory
- Mechanism design basics

- Voting and social choice
- Cooperative game theory
- Shapley value
- Applications in AI: auctions, negotiations, resource allocation

Projects built:

- Game theory solver (Nash equilibrium)
- Repeated game simulator
- Auction mechanism with game theory
- Voting system analyzer
- Cooperative game solver
- Mechanism design implementation
- Multi-agent game scenarios

Practice: Analyze 20 game-theoretic scenarios

Swarm Intelligence & Collective Behavior

Topics:

- Swarm intelligence principles
- Ant Colony Optimization (ACO)
- Particle Swarm Optimization (PSO)
- Bee algorithms
- Flocking and bird behavior (Boids)
- Stigmergy and indirect communication
- Self-organization in swarms
- Emergent intelligence
- Distributed sensing and actuation
- Robot swarms
- Applications: optimization, robotics, routing
- Cellular automata and emergence
- Conway's Game of Life
- Wolfram's cellular automata
- Artificial life and ALife

Projects built:

- Ant Colony Optimization for TSP
- Particle Swarm Optimizer
- Flocking simulation (Boids)
- Robot swarm simulator
- Stigmergy-based system
- Cellular automata explorer
- Artificial life simulation

Practice: Implement 8 swarm intelligence algorithms

Social AI & Human-AI Interaction

Topics:

- Social intelligence in AI
- Theory of Mind in AI
- Emotion recognition and affective computing
- Sentiment analysis for AI
- Social robots and companions
- Conversational AI and chatbots
- Personality in AI agents
- Trust and transparency in AI
- Anthropomorphism and uncanny valley
- Human-in-the-loop AI systems
- Collaborative AI (AI as teammate)
- Explainability for non-experts
- AI ethics in social contexts
- Cultural considerations in AI
- Applications: healthcare, education, customer service

Projects built:

- Emotion-aware chatbot
- Social robot behavior system
- Theory of Mind agent
- Personality-based AI assistant
- Collaborative AI system
- Human-AI teaming framework
- Culturally-aware AI agent

Practice: Build 10 social AI applications

Phase 3 Capstone Project

Topics:

- Complex multi-agent system design
- Intelligent agent development
- Coordination and cooperation
- Real-world application
- Evaluation and analysis

Projects built:

- PHASE 3 CAPSTONE: Intelligent Multi-Agent System
- Requirements: Multiple agents, coordination, learning, real-world application
- Option 1: Smart city traffic management with agents
- Option 2: Multi-robot warehouse automation
- Option 3: Distributed energy grid management
- Option 4: Multi-agent trading and auction system

- Option 5: Disaster response coordination system
- Assessment: Phase 3 Exam - Agents, Robotics, Vision, NLP, Multi-Agent Systems

PHASE 4: ADVANCED AI, AGI & RESEARCH (MONTHS 10-12, WEEKS 40-52)

Master cognitive architectures, AGI concepts, AI philosophy, ethics, cutting-edge research, and career preparation.

Months 10-11: Cognitive AI & AGI (Week 40-48)

Cognitive Architectures

Topics:

- What are cognitive architectures?
- Human cognition and AI
- Symbolic cognitive architectures: SOAR
- ACT-R architecture
- CLARION: hybrid architecture
- Connectionist architectures
- Global Workspace Theory
- Memory systems: working, episodic, semantic
- Attention mechanisms in cognition
- Meta-cognition and self-awareness
- Cognitive modeling
- Integrated cognitive systems
- Embodied cognition
- Comparing architectures
- Applications: human performance modeling, training systems

Projects built:

- SOAR agent implementation
- ACT-R cognitive model
- Memory system simulation
- Attention-based cognitive agent
- Metacognitive reasoning system
- Cognitive architecture comparison
- Human performance simulator

Practice: Implement 5 cognitive architecture models

Artificial General Intelligence (AGI) Concepts

Topics:

- Narrow AI vs AGI distinction
- Definitions of intelligence
- General intelligence requirements
- Transfer learning and generalization
- Common sense reasoning
- Abstraction and analogy
- Causal reasoning for AGI
- AGI architectures and proposals
- OpenCog framework
- AIXI theoretical model
- Universal artificial intelligence theory
- Whole brain emulation
- Consciousness and AGI
- Self-improvement and recursive intelligence
- AGI timeline predictions
- Challenges and roadblocks to AGI

Projects built:

- Common sense reasoning system
- Analogy-making engine
- Causal reasoning framework
- Transfer learning across domains
- AGI architecture proposal (theoretical)
- General problem solver (modern attempt)
- AGI evaluation framework

Practice: Study and critique 10 AGI proposals

AI Philosophy & Theory of Mind

Topics:

- Philosophy of AI: can machines think?
- Chinese Room argument (Searle)
- Computational theory of mind
- Functionalism vs physicalism
- Consciousness in machines
- Hard problem of consciousness
- Qualia and subjective experience
- Intentionality in AI
- Free will and determinism
- Personal identity and AI
- Machine consciousness proposals
- Integrated Information Theory (IIT)
- Global Workspace Theory applied to AI
- Philosophical zombies

- Ethics of creating conscious AI
- Rights and moral status of AI

Projects built:

- Philosophical argument analyzer
- Consciousness detection framework (theoretical)
- Turing Test variants implementation
- Theory of Mind test suite
- Ethical reasoning system for AI consciousness
- Philosophy paper comparative analysis

Practice: Write 5 philosophical position papers on AI

AI Safety & Alignment

Topics:

- AI safety research overview
- Alignment problem: aligning AI with human values
- Value learning from humans
- Inverse reinforcement learning for alignment
- Cooperative Inverse Reinforcement Learning (CIRL)
- Corrigibility and interruptibility
- Off-switch problem
- Reward hacking and specification gaming
- Robustness and adversarial examples
- Interpretability for safety
- Verification and validation of AI systems
- Containment and control strategies
- Oracle AI, Tool AI, Agent AI risks
- Existential risk from AGI
- AI safety organizations: MIRI, FHI, DeepMind Safety
- Research directions in AI safety

Projects built:

- Value learning system
- Inverse RL for preference learning
- Corrigible AI agent
- Adversarial robustness tester
- AI safety evaluation framework
- Alignment research simulation
- Safety verification tools

Practice: Analyze 20 AI safety scenarios

AI Ethics & Societal Impact

Topics:

- Ethics in AI development and deployment
- Bias and fairness in AI systems
- Algorithmic bias detection and mitigation
- Accountability and transparency
- Privacy in AI systems
- Surveillance and AI
- AI in criminal justice: risks and concerns
- Employment and automation
- Economic impact of AI
- AI governance and regulation
- AI principles: Asilomar, IEEE, EU
- Ethical frameworks: utilitarianism, deontology, virtue ethics
- Value-sensitive design
- Participatory AI design
- Global and cultural perspectives on AI ethics
- Future of work and AI

Projects built:

- Bias detection toolkit for AI
- Fairness metrics implementation
- Ethical AI decision framework
- Privacy-preserving AI system
- AI governance policy analyzer
- Ethical AI certification checklist
- Societal impact assessment tool

Practice: Conduct ethics review of all previous projects

Month 12: Research, Innovation & Career (Week 49-52)

AI Research Methodology

Topics:

- Conducting AI research
- Literature review and survey methodology
- Research question formulation
- Hypothesis testing in AI
- Experimental design for AI research
- Benchmarks and datasets
- Evaluation metrics for AI systems
- Statistical significance in AI experiments
- Reproducibility and replicability
- Research ethics and integrity
- Writing research papers

- Paper structure: abstract, intro, methods, results, conclusion
- Peer review process
- Presenting research: conferences, posters
- Publishing venues: NeurIPS, ICML, AAAI, IJCAI
- Open science and preprints (arXiv)

Projects built:

- Literature review on chosen AI topic
- Research proposal writing
- Experimental AI study design
- Research paper writing
- Conference presentation preparation
- Reproducible research package
- Open science publication

Practice: Read 50 research papers, write 3 paper summaries

Cutting-Edge AI Research Areas

Topics:

- Neural-symbolic integration
- Neuro-symbolic AI approaches
- Knowledge-guided machine learning
- Causal AI and causal inference
- Few-shot and zero-shot learning
- Meta-learning (learning to learn)
- Continual and lifelong learning
- Compositional generalization
- Grounded language learning
- Embodied AI and embodied cognition
- World models and model-based RL
- Quantum machine learning
- Neuromorphic computing
- AI for science: protein folding, material discovery
- AI for creativity: music, art, design
- Emergent abilities in large models

Projects built:

- Neuro-symbolic system implementation
- Causal reasoning AI experiment
- Meta-learning framework
- Continual learning system
- Compositional generalization study
- World model for simple environment
- Research experiment in chosen cutting-edge area

Practice: Implement 5 cutting-edge research ideas

AI in Industry & Applications

Topics:

- AI in healthcare: diagnosis, drug discovery, personalized medicine
- AI in finance: algorithmic trading, fraud detection, risk assessment
- AI in education: adaptive learning, tutoring systems
- AI in transportation: autonomous vehicles, traffic optimization
- AI in manufacturing: predictive maintenance, quality control
- AI in agriculture: precision farming, crop monitoring
- AI in energy: smart grids, optimization
- AI in entertainment: games, content generation
- AI in e-commerce: recommendations, search, chatbots
- AI in cybersecurity: threat detection, response
- AI in law: contract analysis, case prediction
- AI in scientific research: discovery, experimentation
- Vertical AI solutions
- Building industry-specific AI products
- Consulting and AI strategy

Projects built:

- Industry-specific AI system (choose domain)
- Healthcare AI application
- Financial AI tool
- Educational AI tutor
- Industrial AI solution
- Case study analysis (5 industries)
- AI product strategy document

Practice: Design AI solutions for 10 different industries

Career Paths in AI Research

Topics:

- AI research scientist roles
- Academic vs industry research
- PhD programs in AI: top universities
- Research internships and fellowships
- AI research labs: Google AI, DeepMind, OpenAI, FAIR, Microsoft Research
- Building research portfolio
- Publications and citations
- Research proposals and grant writing
- Collaboration and networking in research
- Interdisciplinary AI research

- Starting an AI research group
- Teaching and mentoring in AI
- Science communication and outreach
- Transitioning between academia and industry
- Entrepreneurship in AI research

Projects built:

- Research portfolio website
- PhD application materials
- Research statement writing
- Grant proposal (practice)
- Academic CV for AI research
- Research collaboration plan
- Career development roadmap

Practice: Prepare complete PhD application package

Future of AI

Topics:

- AI trends and predictions
- Scaling laws and large models
- Foundation models and their impact
- Multimodal AI systems
- AI and quantum computing convergence
- AI and neuroscience synergy
- Artificial life and open-ended evolution
- AI and human augmentation
- Brain-computer interfaces
- Digital immortality and mind uploading
- AI in space exploration
- Long-term AI governance
- Post-AGI scenarios
- Technological singularity
- Preparing for transformative AI
- Your role in shaping AI future

Projects built:

- Future of AI analysis report
- Technology forecasting study
- Transformative AI scenario planning
- Long-term AI governance proposal
- Personal AI research vision
- Contribution plan to AI field

Practice: Write position paper on future of AI

FINAL CAPSTONE & RESEARCH LAUNCH (Week 49-52 (distributed))

Novel AI System Design

Topics:

- Identifying research gaps
- Novel contribution definition
- System architecture design
- Integration of multiple AI paradigms
- Theoretical foundations
- Innovation and creativity in AI

Projects built:

- Original AI system proposal
- Architecture design document
- Theoretical framework
- Innovation justification
- Related work analysis

Practice: Design completely novel AI approach

Implementation & Experimentation

Topics:

- Prototype implementation
- Baseline comparisons
- Ablation studies
- Performance evaluation
- Debugging and refinement
- Scalability analysis

Projects built:

- Working AI system implementation
- Experimental evaluation
- Performance benchmarking
- Comparison with state-of-art
- Error analysis

Practice: Iterative development and testing

Documentation & Publication

Topics:

- Research paper writing
- Code documentation
- Reproducibility package
- Visualization and presentation
- Submission preparation

- Open-source release

Projects built:

- Complete research paper
- Supplementary materials
- Code repository with docs
- Presentation slides
- Demo video
- Blog post for wider audience

Practice: Professional research communication

Final Defense & Career Launch

Topics:

- Project presentation
- Defending research decisions
- Future work identification
- Career positioning
- Continued research planning

Projects built:

- FINAL CAPSTONE: Original AI Research Contribution
- Requirements: Novel AI system, theoretical contribution, extensive evaluation, publication-ready
- Examples:
 - - Novel cognitive architecture for general reasoning
 - - Hybrid neuro-symbolic system for causal reasoning
 - - Multi-agent system with emergent communication
 - - AI safety mechanism for value alignment
 - - General problem-solving framework
 - - Explainable AI system with human-like reasoning
 - - Continual learning architecture with knowledge retention

Deliverables:

- Original AI system (implemented)
- Research paper (conference-ready)
- Code repository (open-source)
- Documentation (comprehensive)
- Experimental results (reproducible)
- Presentation and defense
- Demo and visualization
- Future research roadmap
- Publication submission
- Career development plan

Assessment: FINAL COMPREHENSIVE EXAM + THESIS DEFENSE - Complete AI mastery evaluation

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 30+ foundational AI projects - search, logic, CSP, games
- Phase 2: 35+ probabilistic and planning projects - Bayes nets, MDPs, planning systems
- Phase 3: 40+ intelligent systems - expert systems, agents, robotics, multi-agent
- Phase 4: 25+ advanced projects - cognitive AI, AGI research, ethics, novel systems
- Total: 130+ AI projects from classical to cutting-edge

Total projects: 130+ intelligent systems across all AI paradigms

SKILLS THE STUDENT WILL HAVE AT THE END

- Classical AI: Search algorithms, game playing, constraint satisfaction, logic, knowledge representation
- Probabilistic AI: Bayesian networks, HMMs, Kalman filters, DBNs, probabilistic reasoning
- Planning: STRIPS, Graphplan, HTN, temporal planning, planning under uncertainty
- Decision Theory: MDPs, POMDPs, decision networks, utility theory, game theory
- Learning: Decision trees, RL (Q-learning, TD), hybrid symbolic-neural systems
- Expert Systems: Rule-based systems, forward/backward chaining, explanation generation
- Agents: BDI architecture, reactive agents, multi-agent systems, agent communication
- Robotics: SLAM, path planning, motion control, robot perception
- Vision: Classical computer vision, object recognition, scene understanding
- NLP: Parsing, semantic analysis, question answering, dialogue systems
- Multi-Agent: Coordination, negotiation, auctions, swarm intelligence, game theory
- Cognitive AI: Cognitive architectures (SOAR, ACT-R), memory systems, metacognition
- AGI: Common sense reasoning, causal reasoning, transfer learning, general intelligence
- Philosophy: Mind theory, consciousness, ethics, AI safety, alignment
- Research: Paper reading, writing, experimentation, publication, innovation
- Tools: Python (expert), Prolog, knowledge representation languages, simulation environments

HOW A TYPICAL WEEK IS SPLIT

Theory study: 6-8 hours (lectures, papers, textbooks)
 Implementation: 10-12 hours (coding, system building)
 Projects: 5-7 hours (applying concepts)
 Research reading: 3-4 hours (papers, surveys)
 Experimentation: 2-3 hours (testing, analysis)
 Total per week: 25-30 hours

SUPPORT INCLUDED

Expert mentorship: PhD-level AI researchers as mentors
Research guidance: Help with research direction and paper writing
Paper reading groups: Weekly paper discussions
Project reviews: Expert feedback on all implementations
Thesis advising: Support for capstone/thesis project
Phd preparation: Guidance for PhD applications
Community: Active researcher community
Resources: Access to papers, textbooks, datasets
Computing: Cloud credits for experiments
Conferences: Guidance on conference submissions
Collaboration: Research collaboration opportunities
Lifetime access: All materials, updates, and advanced modules

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 total)
Final certificate: Advanced Artificial Intelligence Certification
Specialization tracks: Choose: Cognitive AI / Multi-Agent Systems / AGI Research / AI Safety
Research certificate: Research publication certification (with paper acceptance)
Thesis completion: Master's-level thesis equivalent
Portfolio: 50+ documented AI systems
Publications: Support for publishing 1-3 papers
Digital credentials: Blockchain-verified credentials
Industry recognized: Recognized by top AI research labs
Phd ready: Preparation equivalent to first year PhD coursework

WHERE THIS CAN LEAD

- AI Research Scientist (Industry labs: Google AI, DeepMind, OpenAI, FAIR, etc.)
- Research Engineer (Top tech companies)
- PhD Student (Top AI programs: Stanford, MIT, CMU, Berkeley, Oxford, etc.)
- Postdoctoral Researcher
- University Professor (with PhD)
- AI Research Group Leader
- Chief AI Scientist / CTO
- AI Safety Researcher
- Cognitive Systems Architect
- AGI Researcher
- AI Ethics Researcher
- Multi-Agent Systems Specialist
- Robotics AI Researcher
- AI Consultant (Advanced)
- AI Startup Founder (Research-driven)

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Other roles: Rs 15-30 LPA (India), \$120k-200k (USA); Rs 25-50 LPA (India), \$180k-350k (USA); Rs 40-80 LPA (India), \$250k-500k (USA); Rs 60-150 LPA (India), \$350k-800k+ (USA); DeepMind, OpenAI pay \$300k-\$1M+ for top researchers; Rs 10-30 LPA (India), \$80k-150k (USA) as Assistant Professor
Consulting: Rs 200-500/hour for expert AI consulting

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How is this AI Masterclass different from the AI/ML Masterclass?

A: This Artificial Intelligence Masterclass focuses on comprehensive AI theory: classical AI (search algorithms, symbolic reasoning), knowledge representation, planning, multi-agent systems, cognitive architectures, and AGI concepts. The AI/ML course focuses on practical machine learning engineering.

Q: Is this course suitable for PhD preparation in Artificial Intelligence?

A: Yes, this course is specifically designed for PhD preparation.

Q: What AI paradigms and topics does this comprehensive course cover?

A: The course covers symbolic AI, connectionist AI, hybrid systems, search algorithms (BFS, DFS, A*, minimax), propositional and first-order logic, Bayesian networks, MDPs, classical planning, expert systems, multi-agent systems, game theory, robotics AI, computer vision, NLP, cognitive architectures, and AGI research frontiers.

Q: What career paths are available after completing this AI research-focused program?

A: The course prepares you for the most advanced AI roles.

Q: What are the prerequisites for this advanced AI course?

A: You need programming experience (Python preferred), understanding of algorithms and data structures, and comfort with mathematical reasoning. The course teaches all required mathematics (linear algebra, probability, logic) but prior exposure helps.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/artificial-intelligence-complete-masterclass-college>

=====

COURSE 23 OF 118: BACKEND DEVELOPMENT FOR TEENS: APIS, DATABASES & CLOUD

=====

Course link: <https://learn.modernagecoders.com/courses/backend-coding-masterclass-for-teens>
Subject area: Teen Backend Development
Designed for: Teens
Who can join: Complete Beginner to Professional Backend Developer
Course length: 12 months (52 weeks)
Time needed: 10-15 hours/week recommended
Certificate: Industry-recognized Backend Developer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year backend development course for teens: master server-side programming, real APIs, databases, authentication, cloud services and deployment from zero.

In one line: From 'What is a Server?' to Building Instagram-Scale APIs

ABOUT THE PROGRAMME

Designed specifically for teenagers who want to master the invisible powerhouse of the internet - backend development. This isn't about making things look pretty; it's about making things WORK. You'll learn how Netflix streams videos, how Instagram handles millions of photos, how Discord manages real-time chat, and then build these systems yourself! No boring theory - every concept is explained through apps teens actually use. By the end, you'll have built gaming APIs, social media backends, streaming services, and even your own cloud deployment pipeline. Perfect for teens who love logic, problem-solving, and want to build the next big app.

WHAT MAKES THIS COURSE DIFFERENT

- Teen-focused: Examples from games, social media, and apps teens love
- Project-based: Build backends for Minecraft servers, Discord bots, TikTok clones
- Zero to hero: No coding experience needed, we start from scratch
- Industry skills: Learn what companies actually use
- Fun challenges: Hackathons, coding competitions, collaborative projects
- Career prep: Portfolio projects that get you hired or into top colleges
- Mentorship: Young mentors who understand teen culture
- Community: Connect with teen coders worldwide

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Programming basics, JavaScript/Python, Git, Problem Solving
Phase 2: Backend Core (Months 4-6): Node.js, Express, APIs, Databases, Authentication
Phase 3: Advanced Backend (Months 7-9): Microservices, Real-time, Security, Performance
Phase 4: Production & DevOps (Months 10-12): Cloud, Docker, CI/CD, System Design

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No prerequisites - designed for complete beginners
Coding experience: Zero coding experience required
Equipment: Any computer with internet connection
Time commitment: 10-15 hours per week
English: Basic English understanding
Motivation: Passion for technology and problem-solving

WHO THIS COURSE IS FOR

Students: High school students (13-18 years)
Working professionals: Young adults switching careers
Entrepreneurs: Teens wanting to build their own apps
Freelancers: Start freelancing while in school

Kids: Motivated kids 13+ with parental support
Anyone: Anyone interested in backend development

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & PROGRAMMING BASICS (MONTHS 1-3, WEEKS 1-13) -----

Start from absolute zero. Learn how the internet works, master programming fundamentals, and build your first servers.

Months 1-2: Internet Basics & Programming Fundamentals (Week 1-8)

How The Internet Actually Works

Topics:

- What happens when you open Instagram/TikTok?
- Clients vs Servers: The restaurant analogy
- IP addresses: The internet's phone numbers
- DNS: The internet's phonebook
- HTTP/HTTPS: How computers talk
- APIs: The waiter between frontend and database
- Why backend matters: Facebook's 100,000 servers
- Setting up your coding environment
- Installing VS Code and essential extensions
- Terminal/Command line: Hacker mode activated
- Git & GitHub: Your code's Instagram
- Creating your developer profile

Projects built:

- Draw your favorite app's architecture
- Set up your ultimate coding environment
- Create your GitHub profile README

Practice: Daily: Explore 5 websites with Chrome DevTools Network tab

JavaScript Fundamentals - The Language of Backend

Topics:

- Why JavaScript runs the internet
- Variables: Storing game scores and user data
- Data types: Numbers, strings, booleans explained with memes
- Arrays: Managing Spotify playlists
- Objects: Describing a YouTube video
- Functions: Reusable cheat codes
- If/else: Making decisions like game AI
- Loops: Automation for lazy programmers
- Template literals: Dynamic message creation
- Console.log debugging: Your best friend
- Comments: Leaving notes for future you
- Code style: Writing clean, readable code

Projects built:

- Discord bot message generator
- Minecraft inventory manager
- YouTube stats calculator
- Grade calculator with GPA

Practice: Code 30 minutes daily on CodeWars/HackerRank

Python Basics - Alternative Backend Language

Topics:

- Python vs JavaScript: The eternal debate
- Python syntax: Cleaner than your room
- Variables and data types in Python
- Lists, tuples, and dictionaries
- Functions and parameters
- Classes and objects: OOP basics
- File handling: Reading/writing data
- Error handling with try/except
- Popular Python libraries overview
- When to use Python vs JavaScript
- Python for data and automation
- Virtual environments and pip

Projects built:

- Instagram follower tracker
- Automated homework organizer
- Game high score manager
- Weather data analyzer

Practice: Complete Python basics on Codecademy

Algorithms & Problem Solving

Topics:

- What are algorithms? TikTok's 'For You' page explained
- Big O notation: Why Instagram loads fast
- Searching algorithms: Finding a friend in contacts
- Sorting algorithms: Organizing Spotify playlists

- Recursion: Inception for programmers
- Hash tables: How Discord finds users instantly
- Stacks & Queues: YouTube video buffer explained
- Trees: Folder structures and game decision trees
- Problem-solving strategies
- Debugging like a detective
- Code optimization basics
- Common coding patterns

Projects built:

- Build TikTok's recommendation algorithm (simplified)
- Create a matchmaking system for games
- Implement Instagram's hashtag search
- Design a tournament bracket system

Practice: Solve 50 algorithm problems on LeetCode Easy

Month 3: Your First Backend Server (Week 9-13)

Node.js - JavaScript on Servers

Topics:

- What is Node.js? V8 engine explained
- Why Discord and Netflix use Node.js
- Installing Node.js and npm
- Your first server: Hello World
- Understanding the event loop with game examples
- Modules: Organizing code like Minecraft mods
- NPM: The app store for code
- Package.json: Your project's ID card
- Core modules: fs, path, http, crypto
- Reading/writing files: Save game systems
- Asynchronous programming: Not freezing like games
- Callbacks, Promises, Async/Await explained with memes

Projects built:

- File-based todo list for homework
- Discord webhook notifier
- Local multiplayer game server
- Meme generator backend

Practice: Build 10 small Node.js utilities

Express.js - Professional Backend Framework

Topics:

- What is Express? The backend framework everyone uses
- Setting up your first Express server
- Routes: Different URLs for different features
- GET, POST, PUT, DELETE: The four horsemen of APIs
- Request and Response objects
- Middleware: Instagram filters for your server
- Handling JSON data: The language of APIs
- Query parameters: YouTube search explained
- Route parameters: Twitter.com/:username
- Error handling: When things go wrong
- Static files: Serving images and videos
- Environment variables: Keeping secrets safe

Projects built:

- School assignment submission API
- Gaming stats tracker API
- Social media post API
- Music playlist backend

Practice: Create 5 different Express APIs

RESTful API Design

Topics:

- REST principles: The rules everyone follows
- Designing URLs like a pro
- HTTP status codes: 200 OK, 404 Not Found, and friends
- API versioning: Why apps need updates
- Request/Response best practices
- Pagination: Loading Instagram posts efficiently
- Filtering and sorting data
- API documentation with Swagger
- Testing APIs with Postman/Insomnia
- Rate limiting: Preventing spam
- CORS: Letting websites talk to your API
- API security basics

Projects built:

- Complete RESTful blog API
- E-commerce product API
- Video game inventory API
- School management system API

Practice: Document and test all your APIs professionally

Master databases - where all the data lives. Learn to store millions of users, posts, and manage data like tech giants.

Months 4-5: Database Fundamentals (Week 14-21)

Introduction to Databases

Topics:

- Why databases? Imagine Instagram storing 100 billion photos
- Types of databases: SQL vs NoSQL
- When to use which database
- Database design principles
- Tables, rows, and columns: Excel on steroids
- Primary keys: Every user's unique ID
- Foreign keys: Connecting friends on Facebook
- Relationships: One-to-one, one-to-many, many-to-many
- Normalization: Organizing data efficiently
- ACID properties: Why bank apps never lose money
- CAP theorem: The impossible triangle
- Database indexing: Making searches lightning fast

Projects built:

- Design Instagram's database schema
- Plan a gaming leaderboard database
- School database with students and classes
- Social media relationships diagram

Practice: Design 10 different database schemas on paper

MongoDB - NoSQL for Modern Apps

Topics:

- MongoDB basics: Documents instead of tables
- Why Snapchat and Fortnite use MongoDB
- Installing MongoDB and Compass GUI
- Collections and documents
- CRUD operations: Create, Read, Update, Delete
- Querying data: Finding specific posts
- Advanced queries: \$gt, \$lt, \$in, \$regex
- Aggregation pipeline: Data transformation
- Indexes for performance
- Mongoose ODM: MongoDB made easy
- Schema design in MongoDB
- Embedding vs Referencing data

Projects built:

- Social media posts database
- Game inventory system
- Chat application database
- Movie recommendation database

Practice: Build 5 MongoDB databases for different apps

PostgreSQL - Powerful SQL Database

Topics:

- PostgreSQL: The world's most advanced open source database
- SQL basics: SELECT, INSERT, UPDATE, DELETE
- Creating tables with constraints
- Joins: Connecting data from multiple tables
- GROUP BY: Analytics and statistics
- Aggregate functions: COUNT, SUM, AVG
- Subqueries: Queries within queries
- Views: Saved queries for complex data
- Transactions: All or nothing operations
- Database migrations: Updating schemas safely
- Backup and restore strategies
- Performance optimization with EXPLAIN

Projects built:

- E-commerce database with orders
- School management system
- Banking transaction system
- Analytics dashboard database

Practice: Write 100 SQL queries of increasing complexity

Redis & Caching

Topics:

- What is Redis? RAM-based ultra-fast storage
- Why YouTube uses Redis for view counts
- Redis data structures: Strings, Lists, Sets, Hashes
- Caching strategies: Making apps 100x faster
- Session storage with Redis
- Pub/Sub messaging system
- Real-time leaderboards
- Rate limiting with Redis
- Distributed locks
- Redis persistence options
- When to use Redis vs regular database
- Cache invalidation strategies

Projects built:

- Gaming leaderboard system
- Real-time notification system
- Session management for auth
- API rate limiter

Practice: Add Redis caching to previous projects

Database Integration & ORMs

Topics:

- Connecting databases to Express
- Connection pooling for performance
- Sequelize ORM for SQL databases
- Prisma: Modern database toolkit
- Database transactions in Node.js
- Handling database errors gracefully
- Database security best practices
- SQL injection prevention
- Database testing strategies
- Seeding test data
- Database performance monitoring
- Choosing the right database for your project

Projects built:

- Full API with database integration
- Multi-database application
- Database migration system
- ORM comparison project

Practice: Integrate databases into all previous APIs

Month 6: Authentication & Security (Week 22-26)

User Authentication Systems

Topics:

- Authentication vs Authorization: Who you are vs What you can do
- How Instagram knows it's really you
- Password hashing: Never store passwords as text
- Bcrypt: Making passwords uncrackable
- Salt: Extra security seasoning
- Sessions: Remembering logged-in users
- Cookies: Not the edible kind
- JWT tokens: Passport for APIs
- Access tokens vs Refresh tokens
- OAuth 2.0: Login with Google/Discord
- Two-factor authentication (2FA)
- Password reset flows

Projects built:

- Complete authentication system
- Discord OAuth integration
- JWT-based API auth
- 2FA implementation

Practice: Add authentication to 5 different projects

API Security & Best Practices

Topics:

- Common security vulnerabilities
- SQL injection: The classic hack
- XSS attacks: Injecting evil JavaScript
- CSRF: Tricking users into actions
- Rate limiting: Preventing spam and DDoS
- Input validation: Never trust user input
- Sanitization: Cleaning dangerous data
- HTTPS and SSL certificates
- API keys and secrets management
- Environment variables for sensitive data
- Security headers with Helmet.js
- CORS configuration and security

Projects built:

- Secure API with all protections
- Vulnerability scanner tool
- Rate limiting service
- Security audit checklist app

Practice: Security audit all your previous projects

File Uploads & Media Handling

Topics:

- Handling file uploads like Instagram
- Multer for file processing
- Image optimization and resizing
- Video processing basics
- Cloud storage: AWS S3, Cloudinary
- CDN for fast media delivery
- File type validation
- Upload progress tracking
- Chunked uploads for large files

- Generating thumbnails
- PDF generation and manipulation
- QR code generation

Projects built:

- Instagram-like photo upload system
- Video streaming service
- File sharing platform
- Avatar upload system

Practice: Add file upload to previous projects

Email & Notifications

Topics:

- Sending emails programmatically
- Nodemailer setup and configuration
- HTML email templates
- Email verification flows
- Newsletter systems
- SendGrid/Mailgun integration
- Push notifications basics
- SMS notifications with Twilio
- In-app notifications design
- Notification preferences management
- Email queues for bulk sending
- Tracking email opens and clicks

Projects built:

- Email verification system
- Newsletter platform
- Multi-channel notification system
- Automated email campaigns

Practice: Add notifications to your apps

Payment Integration

Topics:

- How online payments work
- Stripe integration basics
- Payment intents and checkout
- Handling webhooks
- Subscription management
- Refunds and disputes
- Payment security (PCI compliance)
- Multiple payment methods
- International payments and currencies
- Payment testing with test cards
- Invoice generation
- Payment analytics

Projects built:

- E-commerce checkout system
- Subscription service backend
- Donation platform
- In-app purchase system

Practice: Integrate Stripe into a project

Month 6 Review & Advanced Features (Week 25-26)

Real-time Features & WebSockets

Topics:

- Real-time communication explained
- How Discord chat works instantly
- WebSockets vs HTTP: Persistent connections
- Socket.io library mastery
- Events: emit, on, broadcast
- Rooms: Private group chats
- Namespaces: Organizing connections
- Real-time notifications
- Live collaboration features
- Online presence indicators
- Typing indicators
- Real-time gaming backends

Projects built:

- Real-time chat application
- Multiplayer game server
- Live collaboration tool
- Real-time dashboard

Practice: Add real-time features to 3 projects

Advanced API Features

Topics:

- GraphQL basics: Query exactly what you need
- GraphQL vs REST comparison
- Apollo Server setup
- Resolvers and schema design
- API rate limiting strategies
- API versioning best practices

- Webhooks implementation
- Long polling for real-time
- Server-sent events (SSE)
- API monitoring and analytics
- API documentation automation
- SDK generation for your API

Projects built:

- GraphQL API for social platform
- Webhook service
- API analytics dashboard
- Multi-version API system

Practice: Build both REST and GraphQL versions of an API

Search & Analytics

Topics:

- Full-text search implementation
- Elasticsearch basics
- Search relevance and ranking
- Autocomplete/suggestions
- Fuzzy search for typos
- Search filters and facets
- Analytics data collection
- User behavior tracking
- Custom analytics dashboard
- A/B testing backend
- Performance metrics
- Error tracking with Sentry

Projects built:

- Search engine for your apps
- Analytics collection system
- A/B testing platform
- Error monitoring service

Practice: Add search to all major projects

Background Jobs & Scheduling

Topics:

- Why background jobs matter
- Queue systems with Bull/BullMQ
- Job scheduling with node-cron
- Email queues for bulk sending
- Image processing queues
- Scheduled reports generation
- Cleanup jobs for old data
- Retry strategies for failed jobs
- Job monitoring dashboards
- Distributed job processing
- Rate-limited API calls
- Batch processing patterns

Projects built:

- Email campaign scheduler
- Automated backup system
- Report generation service
- Social media post scheduler

Practice: Add background jobs to existing projects

Phase 2 Capstone Project

Topics:

- Project planning and architecture
- Database design for scale
- API design documentation
- Security implementation
- Performance considerations
- Testing strategies

Projects built:

- MAJOR CAPSTONE: Complete Backend for Social Platform
- Features: Auth, posts, comments, likes, real-time chat, notifications, search, file uploads
- Alternative: Gaming platform backend with matchmaking
- Alternative: E-learning platform with video streaming

Assessment: Phase 2 Final Assessment - Build complete backend from requirements

PHASE 3: ADVANCED BACKEND & SYSTEM DESIGN (MONTHS 7-9, WEEKS 27-39)

Level up to advanced backend concepts. Learn microservices, system design, and how to build scalable systems.

Months 7-8: Microservices & Advanced Architecture (Week 27-34)

Microservices Architecture

Topics:

- Monolith vs Microservices: Netflix's 700+ microservices
- When to use microservices
- Service decomposition: Breaking Instagram into services

- Inter-service communication
- API Gateway pattern: The front door
- Service discovery: Finding services automatically
- Load balancing between services
- Circuit breakers: Preventing cascade failures
- Event-driven architecture
- Message queues: RabbitMQ, Kafka basics
- Distributed tracing for debugging
- Microservices challenges and solutions

Projects built:

- Break monolith into microservices
- Twitter-like app with microservices
- API Gateway implementation
- Service mesh setup

Practice: Design microservices for 5 different apps

Message Queues & Event Systems

Topics:

- Why message queues? Uber's driver matching explained
- RabbitMQ fundamentals
- Pub/Sub pattern: YouTube notifications
- Work queues for task distribution
- Apache Kafka for streaming
- Event sourcing patterns
- CQRS (Command Query Responsibility Segregation)
- Handling failed messages
- Dead letter queues
- Message ordering guarantees
- Exactly-once delivery
- Building event-driven systems

Projects built:

- Order processing system with queues
- Real-time analytics pipeline
- Notification service with RabbitMQ
- Event-driven e-commerce system

Practice: Implement 5 different messaging patterns

System Design Fundamentals

Topics:

- How to approach system design
- Scalability: Handling millions of users
- Load balancing: Distributing traffic
- Caching strategies: Reddit's caching layers
- CDNs: Netflix's global video delivery
- Database scaling: Sharding and replication
- CAP theorem in practice
- Consistency patterns
- Rate limiting algorithms
- Distributed systems basics
- Monitoring and alerting
- Back-of-envelope calculations

Projects built:

- Design TikTok's video system
- Design Discord's chat system
- Design Uber's matching system
- Design YouTube's recommendation system

Practice: Study and recreate 10 famous system designs

Advanced Database Concepts

Topics:

- Database replication: Master-slave setups
- Sharding: Splitting data across servers
- Consistent hashing for distribution
- Read replicas for scaling
- Database connection pooling
- Query optimization techniques
- Indexing strategies for performance
- NoSQL patterns and anti-patterns
- Time-series databases for metrics
- Graph databases for social networks
- Data warehousing basics
- ETL pipelines introduction

Projects built:

- Implement database sharding
- Build replication system
- Create data pipeline
- Design multi-region database

Practice: Optimize database performance in all projects

Performance & Optimization

Topics:

- Performance profiling tools
- Identifying bottlenecks

- Memory leak detection
- CPU profiling and optimization
- Database query optimization
- N+1 query problem solutions
- Caching at multiple layers
- CDN optimization strategies
- Code optimization techniques
- Async processing patterns
- Load testing with Artillery/k6
- Performance monitoring setup

Projects built:

- Optimize slow API to handle 10x load
- Build performance monitoring dashboard
- Create load testing suite
- Implement multi-layer caching

Practice: Performance test and optimize all APIs

Month 8: Testing & Quality Assurance (Week 32-34)

Testing Fundamentals

Topics:

- Why testing prevents embarrassing bugs
- Types of testing: Unit, Integration, E2E
- Test-Driven Development (TDD)
- Jest testing framework
- Writing your first tests
- Testing async code
- Mocking external services
- Test coverage metrics
- Continuous testing practices
- Debugging failed tests
- Test organization and naming
- Testing best practices

Projects built:

- Add unit tests to all functions
- Integration tests for APIs
- TDD for new feature
- 100% coverage challenge

Practice: Write 200+ tests for your projects

API & Integration Testing

Topics:

- API testing with Supertest
- Testing authenticated routes
- Database testing strategies
- Test data management
- Fixtures and factories
- Testing error scenarios
- Load testing APIs
- Contract testing
- End-to-end testing with Cypress
- Automated browser testing
- CI/CD testing pipelines
- Testing microservices

Projects built:

- Complete test suite for API
- E2E tests for user flows
- Load testing scenarios
- Automated testing pipeline

Practice: Test every API endpoint thoroughly

Code Quality & Best Practices

Topics:

- Clean code principles
- SOLID principles explained
- Design patterns: Factory, Singleton, Observer
- Code refactoring techniques
- Code review best practices
- Linting with ESLint
- Code formatting with Prettier
- Git workflow strategies
- Commit message conventions
- Documentation standards
- README writing guide
- Code comments that actually help

Projects built:

- Refactor messy code challenge
- Implement design patterns
- Create coding standards document
- Open source contribution

Practice: Refactor all old projects with clean code

Monitoring & Logging

Topics:

- Why monitoring saves companies
- Logging best practices
- Structured logging with Winston
- Log levels and when to use them
- Centralized logging systems
- Error tracking with Sentry
- Performance monitoring (APM)
- Custom metrics collection
- Alert configuration
- Dashboard creation
- Distributed tracing
- Debugging production issues

Projects built:

- Implement comprehensive logging
- Build monitoring dashboard
- Set up alert system
- Create debugging toolkit

Practice: Add monitoring to all production projects

Month 8 Project Integration

Topics:

- Combining all learned concepts
- Architecture review
- Performance optimization
- Security hardening
- Testing coverage
- Documentation completion

Projects built:

- MILESTONE PROJECT: Production-Ready API
- Complete with testing, monitoring, documentation
- Deploy to production environment
- Handle real traffic

Practice: Prepare all projects for production

Month 9: Advanced Topics & Specializations (Week 35-39)

Serverless & Cloud Functions

Topics:

- What is serverless? No servers to manage
- AWS Lambda fundamentals
- Google Cloud Functions
- Vercel Edge Functions
- When to go serverless
- Cold starts and optimization
- Serverless API development
- Event-driven serverless
- Serverless databases
- Cost optimization strategies
- Serverless frameworks
- Monitoring serverless apps

Projects built:

- Serverless API with Lambda
- Image processing pipeline
- Webhook processor
- Serverless cron jobs

Practice: Convert project to serverless

AI/ML Integration for Backend

Topics:

- AI APIs: OpenAI, Google AI, Azure
- Adding ChatGPT to your app
- Image recognition with AI
- Natural language processing
- Recommendation algorithms
- Sentiment analysis
- Content moderation with AI
- TensorFlow.js for backend
- ML model deployment
- Training simple models
- AI ethics and limitations
- Cost management for AI APIs

Projects built:

- AI-powered chatbot
- Content recommendation system
- Smart image tagger
- Sentiment analysis API

Practice: Add AI features to 3 projects

AI Backends & Agent APIs

Topics:

- Putting a model behind your own API
- Secrets management: keys on the server, never the client

- Streaming responses (SSE) and why clients need them
- Queues and background jobs for slow AI calls
- Function calling and tool use from a backend
- Retrieval: giving the model your data without retraining
- Vector search basics, and when plain SQL is better
- Caching AI responses, and when caching is wrong
- Rate limiting, quotas and per-user cost caps
- Guardrails: validating what the model gives back
- Logging and auditing AI decisions
- Failure design: what happens when the provider is down

Projects built:

- AI chat API
- Retrieval-backed Q&A service
- Agent with tool access
- Cost + rate-limit middleware

Practice: Add an AI backend to a project

Advanced Security Topics

Topics:

- Penetration testing basics
- OWASP Top 10 deep dive
- Security auditing tools
- Vulnerability scanning
- Secure coding practices
- Encryption and hashing
- Key management systems
- Zero-trust architecture
- API security testing
- Compliance basics (GDPR, CCPA)
- Incident response planning
- Bug bounty programs

Projects built:

- Security audit tool
- Vulnerability scanner
- Encryption service
- Compliance checker

Practice: Security audit all your projects

Phase 3 Capstone Project

Topics:

- Advanced system design
- Microservices implementation
- Complete testing suite
- Performance optimization
- Security implementation
- Production deployment

Projects built:

- MAJOR CAPSTONE: Instagram/TikTok Clone Backend
- Features: Microservices, real-time, AI features, search, recommendations
- Alternative: Multi-player Game Server with matchmaking
- Alternative: Streaming Platform Backend

Assessment: Phase 3 Assessment - System design and implementation

----- PHASE 4: DEVOPS, CLOUD & PRODUCTION (MONTHS 10-12, WEEKS 40-52) -----

Master deployment, cloud services, DevOps practices, and build production-ready systems.

Months 10-11: Docker & Cloud Deployment (Week 40-47)

Docker Fundamentals

Topics:

- What is Docker? Shipping containers for code
- Containers vs Virtual Machines
- Docker installation and setup
- Docker images and containers explained
- Writing your first Dockerfile
- Docker commands mastery
- Multi-stage builds for optimization
- Docker volumes for data persistence
- Docker networks for container communication
- Docker Compose for multi-container apps
- Docker Hub and registries
- Container security best practices

Projects built:

- Dockerize Node.js application
- Multi-container app with database
- Docker development environment
- Production Docker setup

Practice: Dockerize all your major projects

Kubernetes Basics

Topics:

- Kubernetes: Container orchestration explained
- Why big companies use Kubernetes
- Kubernetes architecture overview
- Pods, Services, and Deployments
- kubectl command-line tool
- Deploying apps to Kubernetes
- Scaling applications automatically
- Rolling updates without downtime
- ConfigMaps and Secrets management
- Ingress for routing traffic
- Persistent storage in Kubernetes
- Kubernetes dashboard

Projects built:

- Deploy app to Kubernetes
- Auto-scaling configuration
- Zero-downtime deployment
- Multi-service Kubernetes app

Practice: Deploy 3 projects to Kubernetes

AWS Cloud Services

Topics:

- AWS: Amazon's cloud empire
- Creating your AWS account safely
- EC2: Virtual servers in the cloud
- S3: Storing files like Netflix
- RDS: Managed databases
- Lambda: Serverless functions
- API Gateway for serverless APIs
- CloudFront CDN setup
- Route 53 for DNS management
- IAM: Security and permissions
- Cost optimization tips
- AWS free tier maximization

Projects built:

- Deploy full app on AWS
- Serverless API with Lambda
- Static site with S3/CloudFront
- Multi-region deployment

Practice: Deploy all projects to AWS

CI/CD Pipelines

Topics:

- CI/CD: Automating everything
- GitHub Actions deep dive
- Writing workflow files
- Automated testing in CI
- Building Docker images automatically
- Deploying on every push
- Environment management (dev, staging, prod)
- Secrets management in CI/CD
- GitLab CI/CD alternative
- Jenkins for complex pipelines
- Blue-green deployments
- Rollback strategies

Projects built:

- Complete CI/CD pipeline
- Automated testing workflow
- Multi-environment deployment
- Automated release system

Practice: Add CI/CD to all GitHub projects

Alternative Cloud Providers

Topics:

- Google Cloud Platform overview
- Azure for Microsoft stack
- DigitalOcean for simplicity
- Render for easy deployment
- Vercel for serverless
- Railway for quick deployment
- Fly.io for edge computing
- Comparing cloud providers
- Multi-cloud strategies
- Cloud vendor lock-in
- Cost comparison
- Choosing the right provider

Projects built:

- Deploy to 3 different clouds
- Multi-cloud architecture
- Cloud migration project
- Cost optimization analysis

Practice: Experience different cloud platforms

Month 11: Infrastructure & Scaling (Week 44-47)

Infrastructure as Code

Topics:

- IaC: Treating infrastructure like code
- Terraform fundamentals
- Writing Terraform configurations
- Managing cloud resources with code
- Terraform state management
- CloudFormation for AWS
- Pulumi as alternative
- Version control for infrastructure
- Infrastructure testing
- Cost management with IaC
- Disaster recovery automation
- Multi-environment management

Projects built:

- Terraform AWS infrastructure
- Complete IaC setup
- Disaster recovery plan
- Multi-region infrastructure

Practice: Convert manual infrastructure to code

Scaling & Performance

Topics:

- Scaling strategies: When and how
- Horizontal vs vertical scaling
- Auto-scaling configuration
- Load balancer setup and types
- Database scaling techniques
- Caching at scale with Redis cluster
- CDN configuration and optimization
- Handling traffic spikes
- Rate limiting at scale
- Performance testing with thousands of users
- Monitoring at scale
- Cost-effective scaling

Projects built:

- Scale app to handle 100k users
- Load balancer configuration
- Auto-scaling implementation
- Performance optimization project

Practice: Load test and scale your applications

Site Reliability Engineering

Topics:

- SRE principles from Google
- Service Level Objectives (SLOs)
- Service Level Indicators (SLIs)
- Error budgets concept
- Incident management process
- Post-mortem culture
- On-call rotations
- Runbooks for common issues
- Chaos engineering basics
- Disaster recovery planning
- Backup strategies
- High availability design

Projects built:

- SRE dashboard creation
- Incident response system
- Chaos engineering tests
- DR plan implementation

Practice: Implement SRE practices

Advanced Monitoring & Observability

Topics:

- Three pillars: Logs, Metrics, Traces
- Prometheus for metrics
- Grafana dashboards
- ELK stack (Elasticsearch, Logstash, Kibana)
- Distributed tracing with Jaeger
- APM tools comparison
- Custom metrics and alerts
- SLA monitoring
- Business metrics tracking
- Cost monitoring
- Security monitoring
- Predictive monitoring with ML

Projects built:

- Complete observability stack
- Custom monitoring solution
- Business metrics dashboard

- Alert automation system

Practice: Add comprehensive monitoring to all apps

Month 11 Integration Week

Topics:

- Putting it all together
- Production readiness checklist
- Launch preparation
- Go-live strategies
- Post-launch monitoring
- Incident response preparation

Projects built:

- Production launch simulation
- Complete production system
- Load testing at scale
- Disaster recovery drill

Practice: Prepare for real production launch

Month 12: Final Projects & Career Launch (Week 48-52)

Industry Best Practices

Topics:

- Code review culture
- Agile development basics
- Scrum for developers
- Sprint planning and estimation
- Technical debt management
- Documentation best practices
- API versioning strategies
- Feature flags and toggles
- A/B testing infrastructure
- Gradual rollouts
- Open source contributions
- Building developer tools

Projects built:

- Feature flag system
- A/B testing framework
- Developer tool creation
- Open source contribution

Practice: Apply industry practices to projects

Final Capstone - Planning

Topics:

- Capstone project selection
- Requirements gathering
- System architecture design
- Technology stack decisions
- Database schema design
- API design and documentation
- Timeline and milestone planning
- Risk assessment

Projects built:

- FINAL CAPSTONE PROJECT START
- Options: Social platform, Gaming backend, Streaming service, E-commerce platform
- Complete planning documentation
- Architecture diagrams

Practice: Detailed project planning

Final Capstone - Development

Topics:

- Core feature implementation
- Database setup and optimization
- API development
- Authentication and security
- Real-time features
- File handling
- Payment integration
- Testing implementation

Projects built:

- Complete backend implementation
- All core features working
- Comprehensive testing
- Documentation

Practice: Rapid development sprint

Final Capstone - Production

Topics:

- Production deployment
- CI/CD pipeline setup
- Monitoring and alerting
- Performance optimization
- Security hardening
- Load testing
- Documentation finalization

- Demo preparation
- Projects built:
- Deploy to production
 - Complete DevOps setup
 - Live demo ready
 - Portfolio presentation
- Practice: Production deployment experience

Week 51-52: Career Launch & Beyond

Portfolio & Personal Brand

Topics:

- Building impressive GitHub profile
- Creating portfolio website
- Writing technical blog posts
- LinkedIn optimization for developers
- Twitter/X presence for developers
- Contributing to Stack Overflow
- Building online reputation
- Showcasing projects effectively
- Creating project demos
- Technical writing skills
- Video content creation
- Personal branding strategies

Projects built:

- Professional portfolio site
- GitHub profile optimization
- Technical blog posts
- Project showcase videos

Deliverables:

- Live portfolio website
- 10+ GitHub repositories
- 5+ technical blog posts
- Professional online presence

Interview Preparation

Topics:

- Backend interview patterns
- System design interviews
- Coding challenge strategies
- API design questions
- Database design interviews
- Behavioral questions (STAR method)
- Salary negotiation for teens/young adults
- Internship vs full-time decisions
- Remote work preparation
- Startup vs big company
- Interview practice platforms
- Mock interview sessions

Projects built:

- Interview preparation kit
- System design portfolio
- Coding challenge solutions
- Interview stories preparation

Deliverables:

- Interview cheat sheets
- Practice interview recordings
- Salary research document
- Target company list

Graduation & Next Steps

Topics:

- Course completion celebration
- Final assessment review
- Certificate presentation
- Alumni network access
- Continued learning paths
- Specialization options
- Freelancing opportunities
- Building SaaS products
- Joining open source
- Mentoring others
- Building your startup
- Lifelong learning strategies

Deliverables:

- Course completion certificate
- Final project presentation
- Career roadmap document
- Alumni network membership
- Lifetime course access

Assessment: FINAL COMPREHENSIVE ASSESSMENT - Complete backend system from scratch

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Discord Bot, Game Server, API Basics, School Management System
- Phase 2: Social Media Backend, Chat System, E-commerce API, Video Platform
- Phase 3: Microservices Architecture, Real-time Gaming, AI Integration, Search Engine
- Phase 4: Cloud Deployment, Kubernetes Cluster, Complete DevOps Pipeline, Production System
- Final: TikTok/Instagram Scale Application

Total projects: 40+ backend projects from simple APIs to complex systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Languages: JavaScript, Python, TypeScript, SQL
- Backend: Node.js, Express.js, REST APIs, GraphQL, WebSockets
- Databases: MongoDB, PostgreSQL, Redis, Elasticsearch
- Architecture: Microservices, Event-driven, Serverless, System Design
- Security: Authentication, Authorization, OWASP, Encryption
- DevOps: Docker, Kubernetes, CI/CD, AWS, GCP, Azure
- Testing: Unit, Integration, E2E, TDD, Load Testing
- Tools: Git, VS Code, Postman, Terminal, Linux, Monitoring
- Concepts: Algorithms, Data Structures, Design Patterns, Scaling

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-5 hours
Hands on coding: 5-7 hours
Projects: 3-4 hours
Practice problems: 2-3 hours
Total per week: 10-15 hours

SUPPORT INCLUDED

Live sessions: Weekly doubt clearing with teen mentors
Mentorship: Young mentors who understand teen culture
Community: Discord server with teen developers worldwide
Code review: Expert reviews for all projects
Career support: College admissions, internships, first job
Lifetime access: All content and future updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each quarter
Final certificate: Professional Backend Developer Certification
Linkedin badge: Verified LinkedIn certification
Industry recognized: Accepted by tech companies
Portfolio projects: 15+ production-ready projects

WHERE THIS CAN LEAD

- Backend Developer Intern (during/after high school)
- Junior Backend Developer
- API Developer
- Database Administrator
- DevOps Engineer
- Cloud Engineer
- Freelance Backend Developer
- Technical Founder
- Open Source Contributor
- College CS Program (advanced standing)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is backend development and why should teens learn it?

A: While frontend is what users see, backend is the brain that makes everything work. Think of Instagram: backend handles storing your photos, processing likes, and delivering feeds. Learning backend gives you the power to build complete, real applications.

Q: Is this backend course suitable for teenagers with no coding experience?

A: Yes! This course is specifically designed for teens (13-18 years) with zero prior programming experience. We start with absolute basics like 'What is a server?' and progressively build to advanced topics like microservices and cloud deployment. The pace is tailored for high school students balancing academics.

Q: How much time per week do I need to dedicate to this backend development course?

A: The course requires 10-15 hours per week, which is manageable alongside school. We recommend 1-2 hours of daily practice on weekdays and 3-4 hour project sessions on weekends. The course is self-paced with weekly milestones, so you can adjust based on exams and

holidays.

Q: What technologies will I learn in this 12-month backend development program?

A: You'll master: JavaScript, Python, TypeScript, Node.js, Express.js, REST APIs, GraphQL, WebSockets, MongoDB, PostgreSQL, Redis, Docker, Kubernetes, AWS, CI/CD pipelines, microservices architecture, and system design. These are the exact technologies used by companies like Netflix, Uber, and Spotify.

Q: Can I earn money as a teenager with backend development skills?

A: Backend skills are in strong demand and less commonly taught than frontend, so they stand out. As you build APIs, SaaS features, and real projects in the course, you create a portfolio you can show for internships and freelance work on platforms like Upwork and Fiverr. Many learners take on small paid projects once they are ready, though earnings depend entirely on your skills, effort, and the market.

Q: How does this backend course help with college CS program admissions?

A: Having production-level backend skills significantly strengthens college applications. Your GitHub portfolio with deployed APIs and cloud projects demonstrates real-world capability. For CS programs, practical experience differentiates you from candidates with only academic achievements.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/backend-coding-masterclass-for-teens>

=====

COURSE 24 OF 118: BLOCK-BASED APP DEVELOPMENT: BUILD APPS WITH NO CODE

=====

Course link: <https://learn.modernagecoders.com/courses/block-coding-app-development-masterclass>

Subject area: Visual Programming & App Development

Designed for: Kids

Who can join: Complete Beginner to Advanced App Creator

Course length: 6 months (26 weeks)

Time needed: 8-10 hours/week recommended

Certificate: Block Coding App Developer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 6-month block-based app development course using Thunkable, MIT App Inventor and Scratch, for kids, teens and adults to build real mobile apps with no code.

In one line: From First Block to Published App - No Coding Experience Required!

ABOUT THE PROGRAMME

This revolutionary 6-month program teaches app development using visual block-based programming platforms. Perfect for absolute beginners, kids (8+), teens, and adults who want to create real mobile apps without typing code. You'll master Thunkable, MIT App Inventor, Scratch, and other visual programming tools to build games, utilities, educational apps, and more. By the end, you'll have created 30+ apps, published apps to app stores, understood programming logic, and be ready to create any app idea you can imagine - all without writing a single line of traditional code!

WHAT MAKES THIS COURSE DIFFERENT

- No typing code required - 100% visual programming
- Kid-friendly but comprehensive for all ages
- Create real apps that work on phones and tablets
- Learn multiple block coding platforms
- Publish apps to Google Play and App Store
- Game development included
- AI and machine learning with blocks
- Progress from simple to complex apps systematically

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-2): Programming Logic, Scratch Basics, First Apps

Phase 2: App Development (Months 3-4): MIT App Inventor, Thunkable, Mobile Features

Phase 3: Advanced Apps (Months 5-6): Databases, APIs, AI, Publishing to Stores

Phase 4: Specialization: Games, IoT Apps, Educational Tools, Business Apps

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal education required - suitable for ages 8+

Coding experience: Zero coding experience needed
Equipment: Computer/laptop and smartphone/tablet for testing
Time commitment: 8-10 hours per week
English: Basic English comprehension
Motivation: Curiosity and desire to create

WHO THIS COURSE IS FOR

Students: Kids (8+), teens, and college students
Working professionals: Career switchers and skill upgraders
Entrepreneurs: Anyone wanting to build app businesses
Freelancers: Aspiring app developers and consultants
Kids: Children 8+ wanting to learn programming
Anyone: Anyone interested in creating mobile apps

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & LOGIC BUILDING (MONTHS 1-2, WEEKS 1-8) -----

Master the fundamentals of programming logic using visual blocks. Start with Scratch to understand core concepts, then create your first mobile apps.

Months 1-2: Visual Programming Fundamentals (Week 1-8)

Introduction to Block Coding & Scratch Basics

Topics:

- What is block-based programming?
- Why learn visual coding first?
- Introduction to Scratch interface
- Understanding sprites and stages
- Motion blocks: move, turn, go to, glide
- Looks blocks: say, think, show, hide, change costume
- Sound blocks: play sound, volume, tempo
- Events: when flag clicked, when key pressed
- Creating your first animation
- Saving and sharing projects
- Scratch community and remixing
- Basic debugging with blocks

Projects built:

- Animated greeting card
- Interactive story with characters
- Simple animation sequence
- Musical instrument app

Practice: Daily: Create one Scratch animation, explore 5 community projects

Programming Logic with Blocks

Topics:

- Variables: creating and using data storage
- Lists (arrays) in block coding
- Control blocks: forever, repeat, if-then
- If-then-else decision making
- Operators: math operations with blocks
- Comparison operators: greater than, less than, equals
- Logical operators: and, or, not
- Random numbers and probability
- Creating custom blocks (functions)
- Parameters in custom blocks
- Clone sprites for multiple objects
- Broadcasting messages between sprites

Projects built:

- Calculator with visual blocks
- Quiz game with scoring
- Number guessing game
- Drawing app with controls

Practice: Solve 20 logic puzzles using block programming

Game Development in Scratch

Topics:

- Game design principles
- Player movement and controls
- Collision detection with blocks
- Score keeping and lives system
- Timer and countdown features
- Power-ups and bonuses
- Level progression design
- Game states: start, play, game over
- Sound effects and background music
- Particle effects with clones
- Saving high scores with cloud variables

- Multiplayer basics with cloud data

Projects built:

- Maze navigation game
- Platformer game with jumping
- Space shooter with enemies
- Puzzle game with levels

Practice: Create 5 different game mechanics

Introduction to MIT App Inventor

Topics:

- MIT App Inventor interface tour
- Designer vs Blocks view
- Screen components and properties
- Basic UI elements: buttons, labels, textboxes
- Layout components: arrangements, tables
- Connecting blocks to components
- Event handlers in App Inventor
- Testing apps with companion app
- Building APK files
- Screen navigation and multiple screens
- Basic app architecture
- Debugging techniques

Projects built:

- Hello World mobile app
- Personal information form
- Multi-screen navigation app
- Simple calculator for phones

Practice: Build 10 basic mobile app interfaces

Months 3-4: Mobile App Development (Week 9-17)

Thunkable Platform Mastery

Topics:

- Thunkable vs MIT App Inventor
- Cross-platform development (iOS & Android)
- Thunkable interface and components
- Advanced UI components in Thunkable
- Data viewers: list and grid views
- Tab navigator and drawer navigator
- Working with images and media
- Responsive design for different screens
- Live testing on devices
- Thunkable live app for testing
- Component styling and themes
- Custom colors and fonts

Projects built:

- Photo gallery app
- Note-taking application
- To-do list with categories
- Recipe book app

Practice: Create 5 different app layouts in Thunkable

Working with Device Features

Topics:

- Camera component and photo capture
- Image picker from gallery
- Sound recorder and playback
- Text-to-speech functionality
- Speech recognition basics
- Location sensor and GPS
- Maps component integration
- Accelerometer for motion detection
- Gyroscope and orientation
- Vibration and haptic feedback
- QR code and barcode scanning
- Share functionality

Projects built:

- Camera app with filters
- Voice recorder with playback
- Location-based reminder app
- QR code scanner utility

Practice: Integrate 10 different device features

Data Storage and Persistence

Topics:

- Local storage with TinyDB
- Storing and retrieving data
- Lists and data structures
- Cloud variables for online storage
- Firebase realtime database basics
- Authentication with Firebase
- User accounts and profiles
- Storing user preferences

- Offline functionality
- Data synchronization
- Backup and restore features
- CSV file handling

Projects built:

- Diary app with local storage
- Expense tracker with categories
- Habit tracker with statistics
- Cloud-based notes app

Practice: Implement storage in 5 different app types

Advanced UI and User Experience

Topics:

- Animation blocks and transitions
- Timer-based animations
- Gesture recognition: swipe, pinch
- Custom components creation
- Reusable component blocks
- Advanced list viewers with templates
- Charts and data visualization
- Loading screens and progress bars
- Error handling and user feedback
- Accessibility features
- Multi-language support basics
- App theming and customization

Projects built:

- Animated story app
- Dashboard with charts
- Gesture-controlled game
- Customizable theme app

Practice: Create 10 custom UI components

Month 3-4 Review & Mini Projects

Topics:

- Review of mobile app concepts
- Best practices recap
- Code organization with blocks
- Performance optimization
- Testing strategies

Projects built:

- CAPSTONE: Complete mobile app with all features
- Social media profile app
- Educational quiz app with levels
- Fitness tracking application

Assessment: Create a fully functional mobile app from requirements

Months 5-6: Advanced Features & Publishing (Week 18-26)

APIs and Web Services

Topics:

- What are APIs? Understanding web services
- Web API component in block platforms
- Making GET requests with blocks
- POST requests for sending data
- Working with JSON responses
- Parsing API data with blocks
- Weather API integration
- News API for content
- Translation API usage
- Image recognition APIs
- Error handling for API calls
- API keys and security

Projects built:

- Weather forecast app
- News reader application
- Language translator app
- Movie database browser

Practice: Integrate 10 different public APIs

Game Development Advanced

Topics:

- Physics simulation with blocks
- Sprite collision advanced techniques
- Particle systems for effects
- Leaderboards and achievements
- In-app currency systems
- Power-up and upgrade systems
- Enemy AI with blocks
- Procedural level generation
- Save game progress
- Multiplayer concepts
- Game balancing and testing
- Sound design for games

Projects built:

- Physics puzzle game
- RPG with inventory system
- Endless runner with upgrades
- Multiplayer quiz game

Practice: Develop 5 complete games with different mechanics

AI and Machine Learning Blocks

Topics:

- Introduction to AI in block coding
- Image classification with blocks
- Face detection and recognition
- Emotion detection from images
- Text sentiment analysis
- Chatbot creation with blocks
- Voice commands and control
- Teachable Machine integration
- Custom AI models with blocks
- Personal AI assistant app
- Computer vision applications
- Natural language processing basics

Projects built:

- AI photo classifier app
- Emotion detection camera
- Chatbot assistant app
- Voice-controlled smart app

Practice: Create 5 AI-powered applications

IoT and Hardware Integration

Topics:

- Internet of Things concepts
- Bluetooth connectivity blocks
- Connecting to Arduino with blocks
- micro:bit integration
- Controlling LEDs and motors
- Reading sensor data
- Home automation apps
- Robot control applications
- Wearable device apps
- MQTT protocol basics
- Cloud IoT platforms
- Real-time data monitoring

Projects built:

- Smart home controller app
- Robot remote control
- Plant watering system app
- Fitness band companion app

Practice: Build 5 IoT-connected applications

Publishing and Monetization

Topics:

- App store guidelines and policies
- Preparing apps for publication
- Creating app icons and screenshots
- Writing app descriptions
- Google Play Console setup
- iOS App Store submission
- App monetization: ads integration
- In-app purchases with blocks
- Analytics and user tracking
- App updates and versioning
- Marketing your apps
- Building an app portfolio

Projects built:

- Publish app to Google Play
- Submit app to iOS App Store
- Monetized app with ads
- Professional app portfolio

Assessment: Launch a complete app to app stores

PHASE 2: SPECIALIZED TRACKS (CHOOSE YOUR PATH)

Select specialized tracks based on interest: Educational Apps, Business Tools, Games, or Creative Apps.

Educational Technology Track (Specialization Weeks)

Educational App Design

Topics:

- Learning theory in app design
- Gamification for education
- Interactive lessons creation

- Quiz and assessment systems
- Progress tracking features
- Adaptive learning with blocks
- Multimedia learning content
- Educational game mechanics
- Student-teacher interaction
- Classroom management apps
- Study tools and flashcards
- Educational content management

Projects built:

- Math learning game for kids
- Language learning app
- Science experiment simulator
- Interactive storybook app

Practice: Create 10 educational mini-apps

STEM and Coding Education

Topics:

- Teaching programming concepts
- Visual coding tutorials in apps
- STEM experiment apps
- AR for education (blocks)
- Virtual labs and simulations
- Robotics education apps
- Math visualization tools
- Science data collection
- Engineering design apps
- Coding puzzles and challenges
- Algorithm visualization
- Computational thinking games

Projects built:

- Coding tutorial app
- Physics simulation app
- Chemistry lab app
- Math problem solver

Practice: Build 5 STEM education tools

Business and Productivity Track

Topics:

- Business app requirements
- Inventory management systems
- Invoice and billing apps
- Customer relationship management
- Appointment scheduling
- Task management systems
- Time tracking applications
- Expense reporting tools
- Data entry and forms
- Report generation with blocks
- Business analytics dashboards
- Team collaboration features

Projects built:

- Small business management app
- Invoice generator app
- Employee time tracker
- Customer feedback system

Practice: Create 5 business utility apps

E-commerce and Marketplace Apps

Topics:

- Product catalog design
- Shopping cart with blocks
- Payment gateway integration basics
- Order management system
- User reviews and ratings
- Search and filter functions
- Wishlist functionality
- Seller dashboard creation
- Delivery tracking features
- Promotional features
- Loyalty programs
- Analytics for sellers

Projects built:

- Mini e-commerce app
- Marketplace for services
- Food delivery app
- Rental marketplace app

Practice: Build complete e-commerce flow

Game Development Specialization

Topics:

- Advanced game mechanics
- RPG systems with blocks

- Strategy game elements
- Puzzle game algorithms
- Racing game physics
- Sports game mechanics
- Card game logic
- Board game adaptations
- Casual game design
- Game economy systems
- Social gaming features
- Game analytics and metrics

Projects built:

- Complete RPG with story
- Strategy tower defense game
- Sports tournament game
- Social word game

Practice: Develop 3 different game genres

Creative and Media Apps Track (Creative Specialization)

Creative Tools Development

Topics:

- Drawing and painting apps
- Photo editing with blocks
- Filter and effects creation
- Collage maker features
- Animation creation tools
- Music composition apps
- Beat maker with blocks
- Video editing basics
- Meme generator apps
- Logo design tools
- Pattern generators
- Creative templates

Projects built:

- Drawing app with layers
- Photo filter app
- Music maker app
- Animation studio app

Practice: Build 10 creative tool features

Social Media and Communication

Topics:

- Social feed design
- User profiles and avatars
- Friend systems with blocks
- Messaging interfaces
- Status updates and stories
- Photo sharing features
- Comments and likes system
- Notification systems
- Privacy settings
- Content moderation basics
- Community features
- Live streaming concepts

Projects built:

- Mini social network app
- Chat application
- Photo sharing app
- Community forum app

Practice: Implement 10 social features

Health and Fitness Apps

Topics:

- Fitness tracking interfaces
- Workout timer apps
- Calorie counting features
- Water intake reminders
- Sleep tracking basics
- Meditation and wellness
- Habit formation apps
- Health data visualization
- Exercise library creation
- Nutrition planning tools
- Mental health support apps
- Medical reminder systems

Projects built:

- Workout tracker app
- Meditation timer app
- Nutrition diary app
- Habit building app

Practice: Create 5 health-focused apps

Entertainment and Lifestyle

Topics:

- Recipe and cooking apps
- Travel planning tools
- Event planning apps
- Dating app mechanics
- Hobby tracking apps
- Pet care applications
- Home decoration apps
- Fashion and style apps
- Book reading apps
- Podcast player features
- Streaming app interfaces
- Review and rating apps

Projects built:

- Recipe collection app
- Travel itinerary planner
- Event countdown app
- Hobby tracker app

Practice: Build 5 lifestyle applications

Augmented Reality with Blocks

Topics:

- AR basics in block coding
- AR.js integration basics
- Image tracking with AR
- 3D object placement
- AR filters and effects
- Location-based AR
- AR gaming concepts
- Educational AR apps
- AR for shopping
- Virtual try-on features
- AR navigation apps
- Mixed reality basics

Projects built:

- AR treasure hunt game
- AR educational app
- AR shopping assistant
- AR art gallery app

Practice: Create 3 AR experiences

Professional Development Track (Career Preparation)

Advanced Thunkable Features

Topics:

- Thunkable extensions
- Custom components advanced
- Complex data structures
- Advanced API integration
- OAuth authentication
- Payment processing blocks
- Push notifications setup
- Background tasks basics
- App deep linking
- Dynamic content loading
- Offline-first design
- App performance optimization

Projects built:

- Professional business app
- Advanced e-commerce platform
- Real-time collaboration app
- Enterprise solution app

Practice: Master 10 advanced Thunkable features

Cross-Platform Development

Topics:

- Platform-specific features
- iOS vs Android differences
- Responsive design for tablets
- Web app creation with blocks
- Progressive web apps basics
- Desktop app possibilities
- Wearable app development
- TV app interfaces
- Cross-platform testing
- Platform optimization
- Universal app design
- Multi-device synchronization

Projects built:

- Universal app for all platforms
- Tablet-optimized app
- Web version of mobile app

- Wearable companion app

Practice: Port apps to 3 different platforms

App Testing and Quality

Topics:

- Testing strategies for block apps
- User testing organization
- Beta testing setup
- Bug tracking and fixing
- Performance testing
- Usability testing
- A/B testing basics
- Analytics implementation
- Crash reporting setup
- User feedback collection
- App monitoring tools
- Quality assurance checklist

Projects built:

- Complete testing suite
- Beta testing program
- Analytics dashboard
- Feedback system implementation

Practice: Test and debug 10 apps thoroughly

Transition to Text Coding

Topics:

- From blocks to JavaScript
- Understanding generated code
- React Native basics
- Flutter introduction
- HTML/CSS for web apps
- Python for beginners
- Code structure comparison
- IDE introduction
- Version control with Git
- Debugging text code
- Learning resources
- Career pathways

Projects built:

- Convert block app to code
- First React Native app
- Simple Flutter app
- Basic web application

Practice: Write simple programs in 3 languages

Final Capstone Project

Topics:

- Project planning and design
- User research and requirements
- Wireframing and prototyping
- Development best practices
- Testing and refinement
- Documentation writing
- Marketing preparation
- Launch strategy

Projects built:

- MAJOR CAPSTONE: Professional App from Concept to Launch
- Complete app with all advanced features
- Published to app stores
- Marketing materials created
- User documentation complete

Assessment: Final certification exam and project presentation

BONUS CONTENT: ADVANCED TOPICS & EMERGING TECHNOLOGIES

Optional advanced modules for continued learning and specialization.

Emerging Technologies (Advanced Optional Modules)

AI Features in Your Apps

Topics:

- What an AI API actually is
- Adding a chatbot to your app
- Image recognition: does this photo have a cat?
- Speech to text and text to speech
- Translating text inside your app
- Writing a good prompt from inside an app
- Handling slow, failed or weird AI answers
- Keeping your API key out of your app
- What an AI agent is, and how it differs from a chatbot
- Letting an AI use a tool, and checking its work
- When AI is the wrong tool for the job

- Being honest about which parts the AI built

Projects built:

- Chatbot helper app
- Photo classifier app
- Voice note-taker
- Translator app

Practice: Build 3 AI-powered apps

Advanced AI Integration

Topics:

- GPT API integration
- Custom AI models deployment
- Computer vision advanced
- Voice synthesis and cloning
- Predictive analytics in apps
- Recommendation engines
- AI content generation
- Deepfake detection apps
- AI-powered personalization
- Ethical AI implementation
- AI model training basics
- Edge AI for mobile

Projects built:

- AI writing assistant app
- Smart camera with AI
- Predictive health app
- AI tutor application

Practice: Integrate AI in 5 different ways

Virtual and Mixed Reality

Topics:

- VR concepts in block coding
- 360-degree experiences
- VR tour creation
- Mixed reality basics
- Spatial computing concepts
- Hand tracking basics
- VR UI/UX principles
- WebXR introduction
- VR for education
- Virtual meetings apps
- VR gaming basics
- Multi-user VR spaces

Projects built:

- VR tour app
- VR education experience
- Mixed reality game
- Virtual meeting space

Practice: Create 3 VR/MR experiences

Enterprise and B2B Apps

Topics:

- Enterprise app requirements
- Multi-tenant architecture basics
- Role-based access control
- Enterprise authentication
- Compliance and regulations
- Data privacy implementation
- Audit logging features
- Enterprise integration patterns
- Workflow automation
- Business intelligence dashboards
- Enterprise security features
- Scalability considerations

Projects built:

- Enterprise CRM system
- HR management app
- Supply chain tracker
- Enterprise analytics dashboard

Practice: Build 3 enterprise-grade features

App Entrepreneurship

Topics:

- Finding app ideas
- Market research techniques
- Competitive analysis
- MVP development strategy
- User acquisition strategies
- App monetization models
- Freemium strategies
- App store optimization (ASO)
- Social media marketing
- Influencer partnerships

- App analytics and KPIs
- Scaling your app business

Projects built:

- Complete app business plan
- MVP with monetization
- Marketing campaign setup
- App launch strategy

Practice: Launch and market an app

Teaching and Content Creation (Educator Track)

Becoming a Block Coding Instructor

Topics:

- Teaching methodology for visual programming
- Curriculum development
- Age-appropriate content creation
- Classroom management for coding
- Online teaching platforms
- Creating coding tutorials
- Assessment and grading
- Student project guidance
- Parent communication
- Coding club organization
- Competition preparation
- Teaching resources creation

Projects built:

- Complete coding curriculum
- Tutorial video series
- Student assessment system
- Coding club materials

Practice: Teach 5 coding concepts to beginners

Content Creation for App Developers

Topics:

- YouTube channel creation
- Tutorial video production
- Blog writing for developers
- Course creation basics
- Live streaming coding sessions
- Social media for developers
- Building online community
- Monetizing content
- SEO for developer content
- Podcast creation
- Writing technical books
- Conference speaking

Projects built:

- YouTube channel launch
- Blog with 10 tutorials
- Mini online course
- Developer community setup

Practice: Create content in 3 different formats

Open Source and Community

Topics:

- Open source concepts
- Contributing to block coding platforms
- Creating block extensions
- Community project management
- Documentation writing
- Issue tracking and resolution
- Code review practices
- Community building
- Mentoring new developers
- Organizing hackathons
- Creating coding challenges
- Building developer tools

Projects built:

- Open source block extension
- Community project launch
- Hackathon organization
- Developer tool creation

Practice: Contribute to 3 open source projects

Accessibility and Inclusive Design

Topics:

- Accessibility principles
- Screen reader compatibility
- Color blindness considerations
- Motor accessibility features
- Cognitive accessibility
- Multi-language support
- Cultural sensitivity in apps

- Inclusive UI design
- Testing with disabled users
- Accessibility regulations
- Assistive technology integration
- Universal design principles

Projects built:

- Fully accessible app
- Assistive technology app
- Multi-language app
- Inclusive game design

Practice: Make 5 apps fully accessible

Future of No-Code Development

Topics:

- No-code movement overview
- Low-code vs no-code
- Enterprise no-code platforms
- AI-assisted development
- Voice-programmed apps
- Natural language to app
- Quantum computing basics
- 5G and app development
- Edge computing for apps
- Sustainable app development
- Green coding practices
- Future career paths

Projects built:

- Next-gen app prototype
- AI-assisted app creation
- Sustainable app design
- Future technology integration

Practice: Explore 5 emerging technologies

Specialized Industry Applications (Industry-Specific Development)

Healthcare and Medical Apps

Topics:

- HIPAA compliance basics
- Medical data handling
- Telemedicine app features
- Appointment scheduling systems
- Prescription management
- Health record apps
- Symptom checker apps
- Mental health support apps
- Fitness and rehabilitation
- Medical education apps
- Emergency response apps
- Healthcare provider tools

Projects built:

- Telemedicine app
- Health tracker app
- Medical reference app
- Emergency contact app

Practice: Build 3 healthcare solutions

Financial Technology Apps

Topics:

- Banking app interfaces
- Payment processing basics
- Budget management apps
- Investment tracking
- Spotting scams and fraud in finance apps
- Expense splitting apps
- Tax calculation tools
- Financial literacy apps
- Loan calculators
- Insurance apps
- Financial security features
- Regulatory compliance basics

Projects built:

- Personal finance manager
- Investment tracker
- Expense splitting app
- Savings goal tracker

Practice: Create 5 fintech features

Government and Civic Apps

Topics:

- Civic engagement apps
- Voting and polling apps
- Government service portals
- Emergency alert systems

- Public transportation apps
- City service apps
- Community reporting tools
- Public safety apps
- Environmental monitoring
- Citizen feedback systems
- Government transparency tools
- Smart city applications

Projects built:

- City services app
- Community reporting tool
- Public transport tracker
- Civic engagement platform

Practice: Design 3 civic technology solutions

Real Estate and Property Apps

Topics:

- Property listing apps
- Virtual tour creation
- Mortgage calculators
- Property management tools
- Rental management apps
- Real estate CRM
- Home inspection apps
- Neighborhood information
- Property valuation tools
- Moving and relocation apps
- Smart home integration
- Property investment apps

Projects built:

- Property listing app
- Virtual tour app
- Property management system
- Home buying assistant

Practice: Build 3 real estate solutions

Final Master Certification

Topics:

- Portfolio refinement
- Professional presentation skills
- Industry networking
- Freelancing setup
- Business development
- Continuous learning plan
- Career planning
- Final assessment preparation

Projects built:

- FINAL MASTER PROJECT: Industry-Specific Professional App
- Complete documentation portfolio
- Professional demo video
- Business plan for app venture
- Teaching materials creation

Assessment: Master Certification Exam - Comprehensive evaluation of all skills

LIFETIME LEARNING: CONTINUOUS UPDATES & COMMUNITY

Ongoing support, new content, and community engagement for lifetime learners.

Community Projects & Collaboration (Ongoing Collaborative Learning)

Team Development Projects

Topics:

- Team collaboration tools
- Version control for teams
- Project management basics
- Agile development with blocks
- Code review in visual programming
- Conflict resolution in projects
- Team communication
- Task delegation
- Sprint planning
- Retrospectives and improvement
- Team documentation
- Collaborative debugging

Projects built:

- Team hackathon project
- Community app development
- Open source contribution
- Mentorship program app

Practice: Complete 3 team projects

Advanced Workshops & Masterclasses

Topics:

- Guest expert sessions
- Industry professional talks
- New platform features
- Emerging technology workshops
- Design thinking workshops
- UX/UI masterclasses
- Business strategy sessions
- Marketing workshops
- Legal aspects of apps
- International app markets
- Scaling strategies
- Exit strategies for apps

Projects built:

- Workshop implementation project
- Masterclass challenge app
- Industry-standard app
- Competition entry app

Practice: Attend 10 workshops and implement learnings

Mentorship and Teaching Others

Topics:

- Becoming a mentor
- Teaching methodologies
- Creating learning materials
- Student assessment techniques
- Providing constructive feedback
- Building learning communities
- Organizing workshops
- Creating video tutorials
- Writing documentation
- Public speaking about apps
- Building your expert brand
- Thought leadership

Projects built:

- Mentor 3 beginners
- Create teaching materials
- Host a workshop
- Build tutorial series

Practice: Teach 10 people app development

Research and Innovation

Topics:

- Latest block coding research
- Experimental features testing
- Creating new block patterns
- Performance optimization research
- User behavior studies
- A/B testing methodologies
- Data-driven development
- Innovation in app design
- Patent basics for apps
- Research paper reading
- Conference participation
- Publishing findings

Projects built:

- Research project app
- Innovative feature development
- Experimental app prototype
- Research documentation

Practice: Conduct 3 research experiments

Platform Updates & New Features

Topics:

- Latest Thinkable updates
- MIT App Inventor new features
- New block coding platforms
- Platform migration strategies
- Beta testing new features
- Community feedback programs
- Feature request processes
- Platform comparison studies
- Cross-platform strategies
- Future platform predictions
- Staying current with changes
- Adaptation strategies

Projects built:

- New feature exploration app
- Platform comparison project
- Migration case study
- Beta feature testing

Practice: Master 5 new platform features

Professional Networking & Growth (Career Development)

Building Professional Network

Topics:

- LinkedIn optimization for app developers
- Twitter/X for developers
- Discord and Slack communities
- GitHub presence building
- Stack Overflow participation
- Reddit communities
- Local meetup organization
- Conference networking
- Online community building
- Professional associations
- Mentor finding strategies
- Collaboration opportunities

Projects built:

- Professional portfolio site
- Community engagement plan
- Networking app creation
- Personal brand development

Practice: Build network of 100+ developers

Freelancing and Consulting

Topics:

- Freelance platform profiles
- Client communication
- Project estimation
- Contract negotiation
- Pricing strategies
- Portfolio presentation
- Client requirement analysis
- Project management for freelancers
- Invoice and payment systems
- Client retention strategies
- Scaling freelance business
- Agency transition

Projects built:

- Freelance portfolio
- Client management system
- Project tracker app
- Invoice generator app

Practice: Complete 5 freelance projects

App Business Development

Topics:

- Business model canvas
- Revenue model selection
- User acquisition strategies
- Growth hacking techniques
- App store optimization advanced
- User retention strategies
- Analytics and metrics
- Investor pitch preparation
- Funding options
- Partnership strategies
- International expansion
- Exit strategies

Projects built:

- Complete business plan
- Investor pitch deck
- Growth strategy implementation
- Revenue optimization project

Practice: Launch and grow an app business

Industry Certifications

Topics:

- Platform certifications
- Google certifications
- Apple developer program
- Microsoft certifications
- AWS certifications basics
- Project management certs
- UX/UI certifications
- Digital marketing certs
- Preparing for exams
- Certification maintenance
- Professional development plans
- Career advancement strategies

Projects built:

- Certification study app
- Exam preparation materials

- Professional development tracker
- Career roadmap creation

Practice: Obtain 3 industry certifications

Global Opportunities

Topics:

- International app markets
- Localization strategies
- Cultural adaptation
- Global payment systems
- International regulations
- Remote work opportunities
- Global team collaboration
- Time zone management
- Cross-cultural communication
- International partnerships
- Global app launches
- Emerging markets

Projects built:

- Multi-language app
- Global marketplace app
- International collaboration app
- Cross-cultural app adaptation

Practice: Launch app in 3 countries

Lifetime Support & Resources (Ongoing Support)

Alumni Network

Topics:

- Alumni community access
- Success story sharing
- Peer mentorship programs
- Alumni projects collaboration
- Job referral network
- Knowledge sharing sessions
- Alumni meetups and events
- Continued learning groups
- Special interest groups
- Regional chapters
- Online forums
- Annual conferences

Projects built:

- Alumni directory app
- Knowledge sharing platform
- Collaboration platform
- Event management app

Practice: Engage with 50+ alumni

Continuous Content Updates

Topics:

- Monthly new tutorials
- Platform update guides
- New technology integration
- Industry trend reports
- Case study analyses
- Guest expert content
- Community challenges
- Seasonal projects
- Technology newsletters
- Resource libraries
- Tool recommendations
- Best practices updates

Projects built:

- Implement new features
- Trend analysis project
- Community challenge entry
- Case study implementation

Practice: Stay updated with weekly new content

Special Interest Groups

Topics:

- Game development group
- Education technology group
- Business apps group
- AI and ML group
- IoT developers group
- Creative apps group
- Social impact apps
- Healthcare apps group
- Fintech apps group
- Kids coding group
- Women in tech group
- Regional groups

Projects built:

- Group project participation
- Special interest app
- Community contribution
- Group leadership role

Practice: Join and contribute to 3 groups

Future Learning Paths

Topics:

- Advanced specializations
- University partnerships
- Professional development
- Research opportunities
- Teaching opportunities
- Speaking engagements
- Book writing opportunities
- Course creation
- Consulting opportunities
- Startup incubation
- Innovation labs
- Future technology preview

Projects built:

- Personal development plan
- 5-year career roadmap
- Learning portfolio
- Achievement showcase

Practice: Plan next learning adventure

Success Celebration & Next Steps (Graduation & Beyond)

Portfolio Showcase

Topics:

- Portfolio website creation
- Project documentation
- Case study writing
- Video demonstrations
- Code explanation videos
- Client testimonials
- App metrics showcase
- Design process documentation
- Technical blog posts
- Social media presence
- Professional photography
- Demo day preparation

Projects built:

- Complete portfolio website
- 30+ app showcase
- Video portfolio creation
- Professional presentations

Graduation & Certification

Topics:

- Final assessment preparation
- Comprehensive project review
- Skill demonstration
- Interview preparation
- Certification requirements
- Graduation ceremony
- Certificate verification
- LinkedIn certification display
- Resume updates
- Reference letters
- Recommendation requests
- Success metrics

Deliverables:

- Final project submission
- Complete portfolio
- Certification exam
- Graduation presentation
- Published apps proof
- Community contributions
- Teaching materials
- Innovation projects

Your Journey Continues

Topics:

- Setting new goals
- Advanced learning paths
- Career advancement planning
- Entrepreneurship opportunities
- Giving back to community
- Mentoring newcomers
- Creating your courses

- Building your brand
- Industry leadership
- Innovation and research
- Global impact projects
- Lifetime learning commitment

Deliverables:

- Future goals document
- 1-year action plan
- Mentorship commitment
- Community project proposal
- Personal brand strategy
- Continued learning plan
- Network building goals

Assessment: FINAL CELEBRATION - You are now a Certified Block-Based App Developer!

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1-2: 15 Scratch projects, 10 basic mobile apps
- Month 3-4: 20 intermediate apps with device features
- Month 5-6: 15 advanced apps with AI, IoT, and publishing
- Specialization: 20+ specialized apps in chosen track
- Total: 70+ complete applications

Total projects: 70+ apps ranging from simple animations to complex published applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Visual Programming: Scratch, MIT App Inventor, Thunkable, App Lab
- Mobile Development: Android and iOS app creation
- UI/UX Design: User interfaces, animations, responsive design
- Logic & Algorithms: Programming logic, problem-solving, debugging
- Device Features: Camera, GPS, sensors, Bluetooth, notifications
- Data Management: Local storage, cloud databases, APIs
- Game Development: 2D games, physics, multiplayer basics
- AI Integration: Machine learning, image recognition, chatbots
- IoT Development: Hardware integration, sensor data, automation
- Publishing: App stores, marketing, monetization
- Professional Skills: Project management, documentation, presentation

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-4 hours
 Hands on coding: 4-5 hours
 Projects: 2-3 hours
 Practice problems: 1-2 hours
 Total per week: 8-10 hours

SUPPORT INCLUDED

Live sessions: Weekly Q&A and coding sessions
 Mentorship: Dedicated mentor support
 Community: Active Discord community
 Code review: Project feedback and reviews
 Career support: App publishing guidance and portfolio reviews
 Lifetime access: All content and future updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each major milestone
 Final certificate: Professional Block-Based App Developer Certification
 LinkedIn badge: Shareable LinkedIn certification
 Industry recognized: Recognized by educational institutions
 Portfolio projects: 70+ apps for portfolio

WHERE THIS CAN LEAD

- No-Code App Developer
- Visual Programming Instructor
- Educational Technology Specialist
- App Entrepreneur
- Freelance App Creator
- Junior Mobile Developer (transition path)
- Product Manager
- UX/UI Designer
- Digital Innovation Consultant
- EdTech Content Creator

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What age group is this block coding course suitable for?

A: This course is designed for ages 8 and above. It's perfect for kids, teens, and even adults who want to learn app development without writing traditional code. The visual block-based approach makes complex programming concepts accessible to complete beginners of any age.

Q: Do I need any prior coding experience for this block coding app development course?

A: No prior coding experience is required! This course starts from absolute zero. You'll learn programming logic through visual blocks using platforms like Scratch, MIT App Inventor, and Thunkable. The drag-and-drop interface makes it easy for anyone to create real mobile apps.

Q: Can I actually publish apps to Google Play and App Store with block coding?

A: Yes, absolutely! By the end of this course, you'll learn how to publish your apps to both Google Play Store and Apple App Store. We cover the entire publishing process including creating app icons, writing descriptions, setting up developer accounts, and submitting your apps for review.

Q: What platforms will I learn in this block-based app development masterclass?

A: You'll master three major visual programming platforms: Scratch (for game development and programming logic), MIT App Inventor (for Android app development), and Thunkable (for cross-platform iOS and Android apps). Each platform has unique strengths, and you'll learn when to use which.

Q: How many apps will I build during this 6-month block coding program?

A: You'll build over 70 apps during this comprehensive program! This includes games, utilities, educational apps, social media apps, e-commerce apps, IoT-connected apps, and AI-powered applications. Each project is designed to teach specific skills while building your portfolio.

Q: Can block coding lead to a real career in app development?

A: Yes! Many successful app developers started with block coding. This course prepares you for roles like No-Code App Developer, EdTech Specialist, App Entrepreneur, and freelance app creator. You'll also learn basics of transitioning to text-based programming languages like JavaScript and React Native for future career growth.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/block-coding-app-development-masterclass>

=====

COURSE 25 OF 118: BUSINESS & FINANCE MATHEMATICS: INVESTMENT TO RISK

=====

Course link: <https://learn.modernagecoders.com/courses/complete-business-finance-mathematics-mastery>

Subject area: Business & Finance Education

Designed for: Working professionals

Who can join: Complete Beginner to Advanced Professional

Course length: 12 months (52 weeks)

Time needed: 12-15 hours/week recommended

Certificate: Professional Business & Finance Mathematics Certificate

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year business and finance mathematics course covering investment analysis, risk management, accounting maths, derivatives and portfolio optimization.

In one line: From Basic Calculations to Quantitative Finance in 12 Months

ABOUT THE PROGRAMME

This intensive program transforms you into a financial mathematics expert capable of complex business analysis, investment decisions, and risk management. Whether you're aspiring to investment banking, corporate finance, actuarial science, or entrepreneurship, this course provides the mathematical foundation for success. You'll master everything from compound interest and annuities to options pricing and portfolio optimization. Each concept is taught through real-world business cases, Excel modeling, and Python applications. By completion, you'll think like a quantitative analyst and make data-driven financial decisions.

WHAT MAKES THIS COURSE DIFFERENT

- Business-focused from day one - no abstract math
- Excel and Python integration throughout

- Real company case studies and datasets
- Trading simulations and portfolio management
- Professional financial modeling templates
- CFA/FRM/Actuarial exam preparation included
- Industry-standard software training
- Live market data analysis

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Business Math Basics, Financial Calculations, Accounting Mathematics

Phase 2: Intermediate (Months 4-6): Statistics for Business, Economic Mathematics, Financial Analysis

Phase 3: Advanced (Months 7-9): Investment Mathematics, Risk Management, Corporate Finance

Phase 4: Professional (Months 10-12): Derivatives, Portfolio Theory, Quantitative Methods, Financial Modeling

WHAT A STUDENT NEEDS BEFORE STARTING

Education: High school mathematics

Coding experience: None required - will teach Excel/Python

Equipment: Computer with Excel, internet connection

Time commitment: 12-15 hours per week

English: Business English comprehension

Motivation: Strong interest in finance/business

WHO THIS COURSE IS FOR

Students: Business/Finance/Economics students

Working professionals: Career switchers to finance

Entrepreneurs: Business owners needing financial skills

Freelancers: Financial consultants and advisors

Kids: Advanced high school students (16+)

Anyone: Anyone pursuing finance careers

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: BUSINESS MATHEMATICS FOUNDATIONS (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid foundations in business calculations, financial mathematics, and accounting principles. Master the mathematical tools used daily in business.

Months 1-2: Core Business Mathematics (Week 1-8)

Percentages & Business Applications

Topics:

- Percentage calculations and conversions
- Percentage increase and decrease
- Percentage points vs percentage change
- Markup and markdown calculations
- Profit margins: gross, operating, net
- Break-even analysis
- Sales tax and VAT calculations
- Discount calculations: single and series
- Commission structures and calculations
- Percentage allocation and distribution
- Market share calculations
- Growth rates and CAGR

Projects built:

- Profit margin calculator for business
- Sales commission tracker
- Break-even analysis tool

Practice: Daily: 50 business percentage problems with real scenarios

Time Value of Money Fundamentals

Topics:

- Simple interest calculations
- Compound interest formula and applications
- Continuous compounding
- Effective vs nominal interest rates
- Present value concepts
- Future value calculations
- Discounting and discount factors
- Rule of 72 and quick estimations
- Interest rate conversions
- Day count conventions
- Currency exchange calculations
- Inflation and real vs nominal values

Projects built:

- Compound interest calculator
- Investment growth simulator
- Currency converter with live rates

Practice: Complete 100 time value of money problems

Annuities & Loan Mathematics

Topics:

- Ordinary annuities and annuities due
- Present value of annuity streams
- Future value of annuity calculations
- Perpetuities and growing perpetuities
- Loan amortization schedules
- EMI (Equated Monthly Installment) calculations
- Principal and interest components
- Balloon payments and refinancing
- Mortgage mathematics
- Auto loan calculations
- Credit card mathematics
- Effective cost of borrowing

Projects built:

- Loan amortization calculator
- Mortgage comparison tool
- Retirement planning calculator

Practice: Build 10 different loan amortization schedules

Business Ratios & Analysis

Topics:

- Liquidity ratios: current, quick, cash
- Profitability ratios: ROE, ROA, ROCE
- Efficiency ratios: inventory turnover, receivables
- Leverage ratios: debt-to-equity, interest coverage
- Market ratios: P/E, P/B, dividend yield
- DuPont analysis
- Ratio trend analysis
- Industry comparison metrics
- Working capital calculations
- Cash conversion cycle
- Economic Value Added (EVA)
- Balanced scorecard metrics

Projects built:

- Financial ratio analyzer
- Company comparison dashboard
- Industry benchmark tool

Practice: Analyze 20 real company financial statements

Month 3: Accounting Mathematics & Cost Analysis (Week 9-13)

Accounting Mathematics Fundamentals

Topics:

- Double-entry bookkeeping mathematics
- Trial balance and mathematical checks
- Depreciation calculations: straight-line, declining balance
- MACRS depreciation
- Inventory valuation: FIFO, LIFO, weighted average
- Cost of Goods Sold calculations
- Gross profit and net profit calculations
- Balance sheet equation
- Cash flow calculations
- Accrual vs cash basis mathematics
- Bank reconciliation mathematics
- Foreign currency accounting

Projects built:

- Depreciation schedule generator
- Inventory management calculator
- Cash flow statement builder

Practice: Complete 100 accounting calculation problems

Cost Accounting & Budgeting

Topics:

- Fixed vs variable cost analysis
- Cost-Volume-Profit (CVP) analysis
- Contribution margin calculations
- Operating leverage
- Activity-based costing
- Standard costing and variance analysis
- Material, labor, and overhead variances
- Budget preparation mathematics
- Flexible budgeting
- Zero-based budgeting calculations
- Capital budgeting introduction
- Cost allocation methods

Projects built:

- CVP analysis tool
- Variance analysis dashboard
- Budget vs actual analyzer

Practice: Solve 75 cost accounting problems

Phase 1 Assessment & Integration

Topics:

- Integration of financial concepts
- Complex multi-step problems
- Case study methodology
- Financial calculator mastery
- Excel functions for finance
- Common business math errors
- Professional presentation of calculations

Projects built:

- Comprehensive business analysis case
- Build financial calculator app
- Create business math reference guide

Assessment: Phase 1 comprehensive exam covering all business math fundamentals

PHASE 2: STATISTICAL ANALYSIS & ECONOMIC MATHEMATICS (MONTHS 4-6, WEEKS 14-26)

Master statistical methods for business decision-making, economic analysis, and financial forecasting.

Months 4-5: Business Statistics & Analytics (Week 14-21)

Descriptive Statistics for Business

Topics:

- Business data types and collection methods
- Measures of central tendency in business context
- Variance and standard deviation in finance
- Coefficient of variation for risk
- Skewness and kurtosis in returns
- Business data visualization
- Time series components
- Moving averages and smoothing
- Seasonal adjustments
- Index numbers and deflators
- Business survey analysis
- Customer analytics metrics

Projects built:

- Sales data analyzer
- Customer behavior dashboard
- Market trend visualizer

Practice: Analyze 50 business datasets

Probability in Business Decisions

Topics:

- Probability in risk assessment
- Expected value in business decisions
- Decision trees and expected monetary value
- Bayesian analysis for business
- Conditional probability in market analysis
- Joint probability distributions
- Risk and return relationships
- Value at Risk (VaR) introduction
- Monte Carlo simulation basics
- Scenario analysis
- Sensitivity analysis
- Business forecasting with probability

Projects built:

- Decision tree analyzer
- Monte Carlo simulator for projects
- Risk assessment tool

Practice: Solve 100 business probability problems

Regression & Predictive Analytics

Topics:

- Simple linear regression for business
- Multiple regression analysis
- Interpreting regression coefficients
- R-squared and model fit
- Residual analysis
- Multicollinearity detection
- Dummy variables in regression
- Time series regression
- Logistic regression for classification
- Forecasting with regression
- Sales prediction models
- Price elasticity estimation

Projects built:

- Sales forecasting model
- Price optimization tool
- Customer churn predictor

Practice: Build 20 regression models with real data

Hypothesis Testing for Business

Topics:

- Hypothesis testing framework in business
- A/B testing mathematics
- Type I and Type II errors in business context
- T-tests for marketing campaigns
- ANOVA for multiple group comparison
- Chi-square tests for independence
- Non-parametric tests
- Sample size determination
- Power analysis for business experiments
- Confidence intervals for business metrics
- Statistical significance vs practical significance
- P-value interpretation pitfalls

Projects built:

- A/B testing calculator
- Marketing campaign analyzer
- Quality control dashboard

Practice: Conduct 50 business hypothesis tests

Time Series Analysis

Topics:

- Time series decomposition
- Trend analysis methods
- Seasonal patterns identification
- Cyclical variations
- Autocorrelation and ACF
- ARIMA models introduction
- Exponential smoothing methods
- Holt-Winters forecasting
- Leading and lagging indicators
- Business cycle analysis
- Forecast accuracy measures
- Real-time data analysis

Projects built:

- Time series forecasting tool
- Seasonal adjustment calculator
- Economic indicator dashboard

Practice: Analyze 25 business time series

Month 6: Economic & Market Mathematics (Week 22-26)

Microeconomic Mathematics

Topics:

- Supply and demand curves
- Elasticity calculations
- Consumer and producer surplus
- Utility maximization mathematics
- Budget constraint optimization
- Production functions
- Cost curves and optimization
- Profit maximization calculus
- Market equilibrium calculations
- Game theory basics
- Nash equilibrium
- Oligopoly and monopoly pricing

Projects built:

- Market equilibrium calculator
- Pricing strategy optimizer
- Game theory simulator

Practice: Solve 75 microeconomic problems

Macroeconomic Mathematics

Topics:

- GDP calculations and components
- Inflation rate calculations
- Real vs nominal GDP
- Unemployment rate metrics
- Money multiplier effect
- Interest rate parity
- Exchange rate calculations
- Purchasing power parity
- IS-LM model basics
- Phillips curve relationship
- Economic growth models
- Business cycle indicators

Projects built:

- Economic indicator tracker

- Currency exchange analyzer
- Inflation impact calculator

Practice: Complete 50 macroeconomic calculations

Financial Markets Mathematics

Topics:

- Stock market indices calculation
- Market capitalization weighting
- Price-weighted vs value-weighted indices
- Bond pricing fundamentals
- Yield to maturity calculations
- Duration and convexity
- Term structure of interest rates
- Forward rates calculation
- Swap valuation basics
- Foreign exchange forwards
- Covered interest parity
- Market efficiency metrics

Projects built:

- Bond pricing calculator
- Yield curve analyzer
- Index tracking tool

Practice: Price 50 different financial instruments

International Finance Mathematics

Topics:

- Exchange rate systems
- Cross rates calculation
- Triangular arbitrage
- Forward exchange rates
- Interest rate parity theory
- International Fisher effect
- Currency hedging calculations
- Translation and transaction exposure
- International capital budgeting
- Country risk assessment
- Emerging market analysis
- Global portfolio considerations

Projects built:

- Currency arbitrage detector
- International investment analyzer
- Hedging strategy calculator

Practice: Analyze 30 international finance scenarios

Phase 2 Capstone Project

Topics:

- Integrated statistical analysis
- Economic forecasting project
- Market analysis presentation
- Data visualization mastery
- Professional reporting
- Peer review process

Projects built:

- Complete market analysis report
- Economic forecast model
- Statistical analysis of business case

Assessment: Phase 2 comprehensive exam on statistics and economics

----- PHASE 3: INVESTMENT MATHEMATICS & RISK MANAGEMENT (MONTHS 7-9, WEEKS 27-39) -----

Master investment analysis, portfolio management, risk assessment, and corporate finance decisions.

Months 7-8: Investment Analysis & Valuation (Week 27-34)

Equity Valuation Mathematics

Topics:

- Discounted Cash Flow (DCF) models
- Free cash flow calculations
- WACC (Weighted Average Cost of Capital)
- Terminal value calculations
- Dividend discount models
- Gordon growth model
- Multi-stage growth models
- Price multiples valuation
- Comparable company analysis
- Precedent transaction analysis
- Asset-based valuation
- Sum-of-the-parts valuation

Projects built:

- DCF valuation model
- Comparable analysis template

- Valuation dashboard

Practice: Value 25 real companies using different methods

Fixed Income Mathematics

Topics:

- Bond pricing with different frequencies
- Yield measures: current, YTM, YTC
- Duration: Macaulay and modified
- Convexity calculations
- Immunization strategies
- Credit spreads and ratings
- Floating rate notes
- Convertible bond valuation
- Mortgage-backed securities basics
- Prepayment risk modeling
- Z-spread and OAS
- Fixed income portfolio management

Projects built:

- Bond portfolio analyzer
- Duration matching tool
- Yield curve trading strategy

Practice: Analyze 50 different bond structures

Portfolio Theory & Optimization

Topics:

- Risk and return measures
- Portfolio expected return
- Portfolio variance and standard deviation
- Correlation and covariance matrices
- Diversification benefits
- Efficient frontier construction
- Markowitz optimization
- Capital Asset Pricing Model (CAPM)
- Beta calculation and interpretation
- Security Market Line
- Arbitrage Pricing Theory basics
- Factor models

Projects built:

- Portfolio optimizer
- Efficient frontier visualizer
- Beta calculator for stocks

Practice: Construct 20 optimal portfolios

Performance Measurement & Attribution

Topics:

- Absolute vs relative returns
- Time-weighted vs money-weighted returns
- Sharpe ratio calculation
- Treynor ratio
- Jensen's alpha
- Information ratio
- Sortino ratio
- Maximum drawdown
- Value at Risk (VaR) methods
- Conditional VaR (CVaR)
- Performance attribution analysis
- Risk-adjusted performance measures

Projects built:

- Performance analytics dashboard
- Risk metrics calculator
- Attribution analysis tool

Practice: Evaluate 30 portfolio performances

Alternative Investments Mathematics

Topics:

- Real estate investment mathematics
- Cap rates and NOI calculations
- Private equity valuation
- Venture capital metrics
- Hedge fund fee structures
- Commodity futures pricing
- Infrastructure project valuation
- Art and collectibles valuation
- Cryptocurrency valuation models
- Crowdfunding mathematics
- REIT analysis
- Master Limited Partnerships

Projects built:

- Real estate investment analyzer
- Private equity return calculator
- Alternative asset portfolio tool

Practice: Analyze 20 alternative investments

Month 9: Risk Management & Insurance (Week 35-39)

Risk Measurement & Modeling

Topics:

- Types of financial risk
- Risk metrics: volatility, beta, duration
- Value at Risk (VaR) models
- Historical simulation
- Parametric VaR
- Monte Carlo VaR
- Stress testing methodologies
- Scenario analysis frameworks
- Credit risk modeling
- Probability of default estimation
- Loss given default
- Expected loss calculations

Projects built:

- VaR calculator (3 methods)
- Stress testing framework
- Credit risk scorecard

Practice: Calculate risk metrics for 40 portfolios

Insurance Mathematics

Topics:

- Life insurance mathematics
- Mortality tables and life expectancy
- Present value of life insurance
- Annuity calculations for insurance
- Premium calculation principles
- Reserve calculations
- Property insurance mathematics
- Deductibles and co-insurance
- Loss distributions
- Ruin theory basics
- Reinsurance mathematics
- Health insurance metrics

Projects built:

- Life insurance premium calculator
- Insurance portfolio analyzer
- Claims prediction model

Practice: Solve 50 actuarial problems

Operational & Market Risk

Topics:

- Operational risk measurement
- Key risk indicators (KRIs)
- Loss distribution approach
- Market risk factors
- Greeks: delta, gamma, theta, vega, rho
- Hedging strategies mathematics
- Basis risk calculations
- Liquidity risk measures
- Counterparty risk assessment
- Systemic risk indicators
- Risk appetite frameworks
- Economic capital calculation

Projects built:

- Greeks calculator
- Hedging strategy optimizer
- Risk dashboard creator

Practice: Design 25 risk management strategies

Regulatory Capital & Compliance

Topics:

- Basel III requirements
- Capital adequacy ratios
- Tier 1 and Tier 2 capital
- Risk-weighted assets calculation
- Leverage ratio
- Liquidity coverage ratio (LCR)
- Net stable funding ratio (NSFR)
- IFRS 9 impairment models
- Solvency II for insurance
- Dodd-Frank Act implications
- MiFID II requirements
- Regulatory reporting mathematics

Projects built:

- Capital adequacy calculator
- Regulatory report generator
- Compliance tracking tool

Practice: Calculate regulatory metrics for 20 institutions

Phase 3 Integration & Assessment

Topics:

- Integrated risk management
- Portfolio construction with constraints
- Risk-return optimization
- Comprehensive case studies
- Professional presentation skills
- Industry best practices

Projects built:

- Complete investment analysis case
- Risk management framework design
- Portfolio optimization project

Assessment: Phase 3 exam on investments and risk

PHASE 4: ADVANCED FINANCE & QUANTITATIVE METHODS (MONTHS 10-12, WEEKS 40-52)

Master derivatives pricing, quantitative finance, financial modeling, and advanced corporate finance.

Months 10-11: Derivatives & Structured Products (Week 40-47)

Options Mathematics

Topics:

- Call and put option payoffs
- Intrinsic and time value
- Put-call parity
- Option strategies: spreads, straddles, strangles
- Binomial option pricing model
- Risk-neutral valuation
- Black-Scholes model derivation
- Black-Scholes formula application
- Implied volatility calculation
- American option valuation
- Exotic options: barrier, Asian, lookback
- Real options valuation

Projects built:

- Options pricing calculator
- Strategy payoff visualizer
- Implied volatility surface

Practice: Price 100 different option structures

Futures & Forwards

Topics:

- Forward pricing theory
- Cost of carry model
- Futures margin mechanics
- Basis and basis risk
- Hedging with futures
- Optimal hedge ratios
- Duration-based hedging
- Cross-hedging
- Commodity futures pricing
- Interest rate futures
- Currency futures strategies
- Index futures and arbitrage

Projects built:

- Futures pricing model
- Hedge effectiveness analyzer
- Arbitrage opportunity scanner

Practice: Design 50 hedging strategies

Swaps & Interest Rate Derivatives

Topics:

- Interest rate swap mechanics
- Swap valuation methods
- Currency swap valuation
- Credit default swaps
- Total return swaps
- Swap spread analysis
- Forward rate agreements (FRAs)
- Caps, floors, and collars
- Swaptions
- Constant maturity swaps
- Basis swaps
- Structured notes design

Projects built:

- Swap pricing calculator
- Structured product designer
- Interest rate hedge optimizer

Practice: Structure 40 derivative solutions

Credit Derivatives & Structured Finance

Topics:

- Credit default swap pricing

- CDO and CLO structures
- Synthetic CDOs
- Credit spread options
- First-to-default baskets
- Correlation trading
- Securitization mathematics
- Waterfall structures
- Tranching and subordination
- Prepayment modeling
- Default correlation
- Copula models basics

Projects built:

- CDO structuring tool
- Credit portfolio simulator
- Securitization analyzer

Practice: Analyze 30 structured products

Volatility & Advanced Greeks

Topics:

- Volatility smile and skew
- Term structure of volatility
- Local volatility models
- Stochastic volatility basics
- Volatility trading strategies
- Vega hedging
- Higher-order Greeks
- Cross-Greeks
- Portfolio Greeks management
- Dynamic hedging
- Volatility arbitrage
- Dispersion trading

Projects built:

- Volatility surface modeler
- Greeks risk manager
- Dynamic hedging simulator

Practice: Implement 25 volatility strategies

Month 11-12: Quantitative Finance & Modeling (Week 48-52)

Financial Modeling Excellence

Topics:

- Financial model architecture
- Three-statement modeling
- Revenue and cost projections
- Working capital modeling
- Debt schedules and waterfalls
- Circular reference resolution
- Scenario and sensitivity tables
- Monte Carlo in Excel
- Model auditing techniques
- Dashboard design
- VBA for financial modeling
- Model documentation standards

Projects built:

- Complete financial model template
- LBO model
- Project finance model

Practice: Build 10 professional financial models

Algorithmic Trading Mathematics

Topics:

- Trading system components
- Backtesting methodology
- Performance metrics for strategies
- Market microstructure
- Order types and execution
- Slippage and transaction costs
- Mean reversion strategies
- Momentum strategies
- Pairs trading mathematics
- Statistical arbitrage
- High-frequency trading basics
- Machine learning in trading

Projects built:

- Trading strategy backtester
- Pairs trading system
- Performance analytics platform

Practice: Backtest 20 trading strategies

Corporate Finance Advanced

Topics:

- Advanced capital budgeting
- Real options in capital budgeting

- Cost of capital estimation
- Capital structure optimization
- Modigliani-Miller theorems
- Trade-off theory
- Pecking order theory
- Dividend policy mathematics
- Share buyback analysis
- Merger and acquisition valuation
- Synergy valuation
- Leveraged buyout modeling

Projects built:

- M&A valuation model
- LBO returns analyzer
- Capital structure optimizer

Practice: Analyze 25 corporate finance decisions

Fintech & Blockchain Finance

Topics:

- Cryptocurrency valuation models
- Blockchain in financial services
- DeFi protocol mathematics
- Yield farming calculations
- Liquidity pool mathematics
- Stablecoin mechanisms
- Central bank digital currencies
- Robo-advisor algorithms
- Peer-to-peer lending models
- Crowdfunding mathematics
- Payment processing economics
- RegTech applications

Projects built:

- DeFi yield calculator
- Crypto portfolio tracker
- P2P lending analyzer

Practice: Analyze 20 fintech business models

ESG & Sustainable Finance

Topics:

- ESG scoring methodologies
- Green bond mathematics
- Carbon credit pricing
- Social impact measurements
- Sustainable investment strategies
- Climate risk modeling
- Transition risk assessment
- Impact investing metrics
- Blended finance structures
- Sustainability-linked loans
- TCFD reporting requirements
- EU taxonomy calculations

Projects built:

- ESG portfolio analyzer
- Carbon footprint calculator
- Impact measurement tool

Practice: Evaluate 15 sustainable investments

Professional Applications & Certification (Week 50-52)

Industry Case Studies

Topics:

- Investment banking case studies
- Private equity deal analysis
- Hedge fund strategies review
- Insurance company ALM
- Pension fund management
- Sovereign wealth funds
- Central bank operations
- Corporate treasury management
- Family office structures
- Endowment management

Projects built:

- Complete industry analysis
- Deal structure proposal
- Investment committee presentation

Practice: Analyze 10 real-world cases

Professional Certifications Prep

Topics:

- CFA Level 1 quantitative review
- FRM Part 1 quantitative topics
- Actuarial exam prep
- CAIA quantitative sections
- CPA finance sections

- Series 7 calculations
- Bloomberg certification prep
- Excel certification prep
- Python for finance certification
- Professional designation comparison

Projects built:

- Certification study planner
- Practice exam generator
- Formula reference guide

Practice: Complete 500 certification practice questions

Financial Technology Tools

Topics:

- Bloomberg Terminal mastery
- Reuters Eikon navigation
- FactSet essentials
- S&P Capital IQ
- Morningstar Direct
- Python for finance
- R for financial analysis
- SQL for financial data
- Tableau for finance
- Power BI dashboards
- Advanced Excel techniques
- Financial APIs integration

Projects built:

- Multi-source data aggregator
- Automated report generator
- Real-time market dashboard

Practice: Master 10 professional tools

Career Development & Networking

Topics:

- Finance career paths
- Resume optimization for finance
- Technical interview preparation
- Case interview techniques
- Networking in finance
- LinkedIn optimization
- CFA Institute networking
- Professional associations
- Continuing education planning
- Industry trends and future

Projects built:

- Professional portfolio website
- GitHub repository of projects
- Industry research report

Practice: Complete 20 mock interviews

Final Projects & Certification (Week 51-52)

Capstone Project - Part 1

Topics:

- Project selection and scoping
- Data collection and cleaning
- Model development
- Analysis and insights
- Risk assessment
- Recommendations formulation
- Sensitivity analysis
- Documentation preparation

Projects built:

- FINAL CAPSTONE: Complete Financial Analysis Project
- Options: Company valuation with M&A recommendation
- Portfolio optimization with risk management
- Trading strategy with full backtest
- Fintech business plan with financial projections

Capstone Project - Part 2 & Presentation

Topics:

- Final model refinement
- Peer review process
- Presentation preparation
- Executive summary writing
- Q&A preparation
- Professional formatting
- Submission guidelines
- Public presentation

Deliverables:

- Complete financial model
- 50-page analysis report
- Executive presentation
- Video walkthrough

- Peer review feedback
- GitHub repository

Final Assessment & Certification

Topics:

- Comprehensive knowledge review
- Final examination preparation
- Portfolio compilation
- Skills self-assessment
- Future learning roadmap
- Alumni network access
- Industry connections
- Certification ceremony

Deliverables:

- Final comprehensive exam (4 hours)
- Complete project portfolio
- Professional references
- Certification of completion
- LinkedIn certification badge
- Transcript of achievements

Assessment: FINAL EXAM: Comprehensive 4-hour exam covering all business and finance mathematics

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Business calculators, Loan analyzers, Accounting tools
- Phase 2: Statistical models, Economic forecasts, Regression analyses
- Phase 3: Investment valuations, Portfolio optimizers, Risk dashboards
- Phase 4: Derivatives pricers, Trading systems, Financial models
- Final: Professional-grade financial analysis project

Total projects: 150+ financial tools, models, calculators, and analytical dashboards

SKILLS THE STUDENT WILL HAVE AT THE END

- Financial Mathematics: TVM, annuities, loans, bonds, derivatives
- Statistical Analysis: Regression, time series, hypothesis testing, forecasting
- Investment Analysis: Valuation, portfolio theory, performance measurement
- Risk Management: VaR, stress testing, credit risk, operational risk
- Corporate Finance: Capital budgeting, capital structure, M&A, LBO
- Quantitative Methods: Monte Carlo, optimization, algorithmic trading
- Financial Modeling: Excel, VBA, Python, professional models
- Professional Tools: Bloomberg, Excel advanced, Python, SQL
- Certifications Prep: CFA, FRM, actuarial exam readiness

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-6 hours
 Hands on coding: 4-5 hours
 Projects: 3-4 hours
 Practice problems: 3-4 hours
 Total per week: 12-15 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving workshops
 Mentorship: Industry professional mentors
 Community: Active finance professionals network
 Code review: Model review and feedback
 Lifetime access: All materials and future updates
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each quarter
 Final certificate: Professional Finance Mathematics Certificate
 LinkedIn badge: Verified achievement badge
 Industry recognized: Endorsed by finance professionals
 Portfolio projects: 150+ completed financial analyses

WHERE THIS CAN LEAD

- Financial Analyst
- Investment Banking Analyst
- Risk Analyst
- Equity Research Analyst
- Credit Analyst
- Portfolio Manager
- Quantitative Analyst
- Corporate Finance Manager

- Financial Consultant
- Actuarial Analyst

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Growing roles: Rs 4-8 LPA (Entry-level analyst)

Experienced roles: Rs 8-15 LPA (Financial Analyst); Rs 12-25 LPA (Senior Analyst); Rs 15-40 LPA (Manager/Specialist)

Freelance: Rs 1,000-5,000/hour consulting

International: \$60k-150k based on role and location

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will I learn in the Business & Finance Mathematics program?

A: Across 12 months you move from core business maths, percentages, interest, financial calculations and accounting mathematics, through business statistics and economic mathematics, into investment analysis, valuation and risk management, and finish with derivatives, portfolio theory and quantitative financial modeling. Excel is used throughout, with Python introduced for the quantitative work.

Q: Who is this course for?

A: It starts from genuine basics, so you do not need a finance background to begin.

Q: Do I need Excel or coding skills before starting?

A: No. High school mathematics is the only prerequisite; Excel is taught from scratch inside the course and Python is introduced gently when the quantitative topics need it. You will need a computer with Excel and a stable internet connection.

Q: How is progress measured?

A: Through weekly applied exercises, case style projects such as building valuation and risk models, and phase-end assessments. By the final months you are building complete financial models, which become the clearest proof of progress.

Q: Is this course theoretical or practical?

A: Deliberately practical: every concept is anchored in a business situation, from break-even analysis and loan amortization to portfolio optimization. The final phase is essentially a professional toolkit of models you can reuse in a job, a business or further study.

Q: Can I sit in on a class first?

A: Yes, start with a free demo class, no commitment: book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The demo also helps us gauge your starting level honestly.

Q: Are classes live?

A: Every session is a live one hour online class with a real teacher; nothing is pre-recorded.

Q: What certification do I receive?

A: The Professional Business & Finance Mathematics Certificate on completion, plus a portfolio of the financial models and case projects you built, which is often more persuasive to employers than the certificate itself.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-business-finance-mathematics-mastery>

COURSE 26 OF 118: C PROGRAMMING FOR BEGINNERS: POINTERS, MEMORY & STRUCTS

Course link: <https://learn.modernagecoders.com/courses/c-programming-course-for-college-beginners>

Subject area: Programming Languages

Designed for: College students

Who can join: Beginner, college students and first-year engineers

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 3-4 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

C programming course for college students and beginners: live online classes on pointers, memory, structs, file handling and data structures, taught from zero.

In one line: Most C courses skip past pointers as fast as possible. This one is built around them.

ABOUT THE PROGRAMME

C is the language underneath everything: operating systems, databases, embedded firmware, and the interpreters that run Python and JavaScript are written in it. It is also the language most engineering students meet first and understand least, because the part that matters, pointers and memory, is usually rushed in two lectures. This 6-month live course inverts that. Months 1 and 2 build the language core properly: types, control flow, functions, and how a .c file actually becomes an executable. Months 3 and 4 are the heart of the course: a full month on pointers taught with memory diagrams for every operation, then a month on dynamic memory, the heap, and structs, including the bugs (leaks, dangling pointers, buffer overflows) demonstrated live and safely. Months 5 and 6 put it to work: file handling, linked lists, stacks, queues, and sorting built from scratch, a look at where C runs in the real world from kernels to microcontrollers, and a capstone project compiled, debugged, and defended. Classes are live and small. Students write and compile code in every session, homework is real programming with written feedback, and the instructor draws memory on a whiteboard until it stops being mysterious.

WHAT MAKES THIS COURSE DIFFERENT

- Pointers get a full month, not two lectures, and every operation comes with a memory diagram before any code
- Memory bugs are taught on purpose: students create a leak, a dangling pointer, and an overflow in controlled exercises, then learn to catch them
- Standard library functions like strlen and strcpy are written by hand first, so nothing in string.h is magic
- Data structures are built from raw structs and malloc, the way university exams and real systems code expect
- The compilation pipeline is taught, not skipped: preprocessor, compiler, linker, and what an executable actually is
- The syllabus deliberately covers the C programming paper taught in Indian engineering first year, so college exams and this course reinforce each other

THE LEARNING PATH, STEP BY STEP

Phase 1: Language core and the machine: types, control flow, functions, the stack, and how source code becomes an executable

Phase 2: The heart of C: pointers with memory diagrams, arrays and strings at the memory level, malloc and the heap, and structs

Phase 3: C at work: file handling, linked lists, stacks, queues, sorting, bitwise operations and embedded context, then a capstone

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None required. The course starts from the first compile. Students who know another language will simply move faster in month 1

Mathematics: School-level maths is plenty; no calculus or advanced topics are needed

Equipment: Any Windows, Mac, or Linux computer that can run a compiler and VS Code, plus a stable internet connection

Mindset: Willingness to trace code on paper. C rewards students who slow down, and the course is built around that

WHO THIS COURSE IS FOR

First year engineers: College students taking the compulsory C paper who want to actually understand it, not just pass it

Cs and it students: Students whose degree assumes C fluency for data structures, operating systems, and GATE later

Self taught switchers: Programmers from Python or JavaScript who keep hearing about pointers and want the real story

Embedded curious: Students eyeing electronics, robotics, or firmware, where C remains the working language

Fundamentals seekers: Anyone who wants to know what actually happens under high-level languages, one memory diagram at a time

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: THE LANGUAGE CORE AND THE MACHINE (MONTHS 1-2, WEEKS 1-8) -----

C fundamentals built on honest ground: what the compiler does, what types really are, and how

functions and the call stack work, so the pointer month lands on solid footing.

Month 1: From Source Code to Executable (Weeks 1-4)

Hello, Compiler

Topics:

- Setting up GCC and VS Code on Windows, Mac, or Linux
- Your first program: main, printf, and the return value
- The compilation pipeline: preprocessor, compiler, linker, executable
- Compiling from the terminal and reading compiler errors calmly
- Why C has no training wheels, and why that is the point
- Comments, formatting, and code that reads like it was written on purpose

Projects built:

- Banner program: compile, break, and fix a program five different ways to learn the error messages early

Practice: Compile six small programs from the terminal, deliberately triggering and decoding one error each

Types, Variables, and Their Limits

Topics:

- int, char, float, double, and the unsigned variants
- sizeof: asking the machine how big things are
- Integer overflow demonstrated live: what happens at the edge
- Type conversion and casting, implicit and explicit
- printf and scanf format specifiers, and the crashes wrong ones cause
- Constants with const and #define

Projects built:

- Limits explorer: a program that prints the size and range of every basic type on your machine

Practice: Ten input-output exercises with mixed types, including two that overflow on purpose so you see it happen

Operators and Control Flow

Topics:

- Arithmetic operators and the integer division surprise
- Relational and logical operators, and short-circuit evaluation
- if, else if, else, and the switch statement
- The ternary operator, used where it helps
- Operator precedence: when to just use parentheses
- = vs ==: the classic C bug, met early

Projects built:

- Grade and tax calculator: branching logic with real edge cases, negative inputs, and boundary values

Practice: Twelve control-flow drills, including a leap year checker and a menu-driven calculator with switch

Loops

Topics:

- while, for, and do-while, and choosing between them
- Loop counters, accumulators, and running totals
- Nested loops: printing patterns the exam way
- break and continue, used sparingly
- Off-by-one errors: finding them before the compiler cannot
- Tracing a loop by hand: the skill exams actually test

Projects built:

- Pattern engine: pyramids, diamonds, and number triangles from nested loops
- Prime and perfect number finder with a hand-traced dry run submitted alongside the code

Practice: Fifteen loop problems ranging from digit reversal to a simple guessing game

Assessment: Month 1 lab exam: three programs written live under light time pressure, compiled and demonstrated

Month 2: Functions and the Call Stack (Weeks 5-8)

Functions Done Properly

Topics:

- Defining functions: parameters, return types, and void
- Declarations vs definitions, and why prototypes exist
- Pass by value: what the function actually receives
- Designing small functions with one job each
- Header files and splitting code across files
- Reading a function signature like documentation

Projects built:

- Number theory library: gcd, lcm, prime check, and digit sum as clean reusable functions in their own file

Practice: Write eight functions from written specifications, then refactor month 1 projects to use them

Scope, Storage, and the Stack

Topics:

- Local vs global variables, and why globals age badly
- The call stack drawn frame by frame for a real program
- static variables: memory that survives the function
- Automatic storage: why locals vanish when the function returns
- Stack overflow: what it literally means
- Debugging with a stack diagram next to the code

Projects built:

- Stack tracer: a set of nested function calls with a hand-drawn stack diagram matched against actual output

Practice: Predict the output of six scope puzzles before running them, and score your predictions

Recursion

Topics:

- Recursion as functions calling themselves, with the stack diagrams to prove it
- Base case and recursive case: the two-part contract
- Factorial, Fibonacci, and sum of digits, traced fully
- Recursion vs loops: cost, clarity, and when each wins
- Stack depth limits demonstrated by crashing one on purpose
- Reading recursive code without re-tracing every call

Projects built:

- Recursive toolkit: power, GCD, decimal-to-binary, and Tower of Hanoi with move printing

Practice: Convert three loop solutions to recursion and three recursive solutions to loops, and note which direction felt harder

Multi-File Programs and the Build

Topics:

- Organizing a project: headers, source files, and include guards
- Compiling multiple files and linking them together
- A first Makefile: stop retyping compiler commands
- The preprocessor: #include, #define, and conditional compilation
- Naming, formatting, and structure conventions used in real C codebases
- Reading someone else's C file without fear

Projects built:

- Unit converter suite: a menu-driven multi-file program with separate modules for length, weight, and temperature

Practice: Split a provided 300-line single-file program into clean modules with headers and a Makefile

Assessment: Month 2 review: a multi-file program built from a spec, plus an oral walkthrough of one stack diagram

----- PHASE 2: POINTERS AND MEMORY, TAUGHT PROPERLY (MONTHS 3-4, WEEKS 9-16) -----

The month most courses rush is the month this course is named for: pointers with a memory diagram for every operation, then dynamic memory, the heap, and structs, including the famous bugs made visible.

Month 3: Pointers, One Diagram at a Time (Weeks 9-12)

Addresses and the & Operator

Topics:

- Memory as numbered boxes: the mental model everything rests on
- The & operator: every variable lives somewhere
- Declaring pointers: what `int* p` actually says
- Dereferencing with *: reading and writing through a pointer
- NULL and why pointers should never dangle uninitialized
- Printing addresses with `%p` and watching them change between runs

Projects built:

- Memory mapper: a program that prints the address and value of every variable it owns, matched to a hand-drawn diagram

Practice: Ten pointer basics drills, each answered first on paper with a box diagram, then verified in code

Pointers and Functions

Topics:

- Why pass by value cannot swap two numbers
- Passing addresses: functions that modify their arguments
- The classic swap written and traced completely
- Out-parameters: returning more than one result
- scanf finally explained: why it always wanted the &
- Pointer parameters in library functions you already used

Projects built:

- Statistics engine: one function that fills min, max, and mean through pointer parameters from a single pass

Practice: Rewrite five month 2 functions to use pointer parameters, and explain in comments what changed in memory terms

Arrays and Pointer Arithmetic

Topics:

- Arrays in memory: contiguous boxes with one name
- Array names and pointers: the decay rule stated honestly
- Pointer arithmetic: what `p+1` really adds
- Walking an array with a pointer instead of an index
- Passing arrays to functions, and why size must travel with them
- Out-of-bounds access: the bug C will not catch for you, demonstrated safely

Projects built:

- Array toolkit: reverse, rotate, and search implemented twice, once with indices and once with pointers

Practice: Trace eight pointer arithmetic snippets on paper, predicting values and addresses before running them

Strings at the Memory Level

Topics:

- Strings as char arrays with a null terminator, no magic
- String literals vs char arrays, and what is writable
- Writing strlen, strcpy, and strcmp by hand before touching string.h
- Buffer overflow demonstrated in a controlled exercise, and why it matters for security
- Safer patterns: bounds checking and fgets over gets
- Arrays of strings and the char* array

Projects built:

- mini_string.h: your own implementation of five standard string functions, tested against the real library

Practice: Solve six string problems using only your own mini_string functions, no string.h allowed

Assessment: Pointer month exam: paper diagrams plus live coding, including one swap variant and one string function from scratch

Month 4: The Heap, Dynamic Memory, and Structs (Weeks 13-16)

malloc and the Heap

Topics:

- Stack vs heap: two kinds of memory with different lifetimes
- malloc and free: asking for memory and giving it back
- sizeof in allocation: the pattern that avoids a whole bug family
- Checking for NULL: allocation can fail
- Memory leaks: creating one on purpose, then finding it
- Ownership thinking: who allocated this, and who must free it

Projects built:

- Runtime array: a program that asks the user how much data is coming, allocates exactly that, and cleans up properly

Practice: Six allocation exercises, each submitted with a one-line ownership note stating where every free happens

Dynamic Arrays and the Famous Bugs

Topics:

- realloc: growing storage as data arrives
- Building a growable array with capacity doubling
- Dangling pointers: use-after-free demonstrated and diagnosed
- Double free and what the runtime does about it
- AddressSanitizer: making the invisible bugs report themselves
- Valgrind on Linux, and what a clean report looks like

Projects built:

- Growable list: a dynamic array module with append, insert, and remove that survives a sanitizer run cleanly

Practice: Fix five provided programs, each hiding one memory bug: a leak, a dangle, a double free, an overflow, and an uninitialized read

Structs

Topics:

- struct: modeling a real thing with named fields
- Declaring, initializing, and copying structs
- Arrays of structs: a table of records in memory
- Pointers to structs and the arrow operator
- typedef for names that read like types
- Nested structs and designing the record before coding it

Projects built:

- Student database, part one: an array-of-structs record system with add, list, search, and update

Practice: Model three real records as structs: a bank account, a library book, and a cricket scorecard entry

Structs Meet the Heap

Topics:

- Allocating structs dynamically
- Structs containing pointers: records with variable-length fields
- Deep copy vs shallow copy, and the bug hiding between them
- Freeing nested allocations in the right order
- Designing a small module: header, implementation, and a test file
- Code review: reading a classmate's memory management critically

Projects built:

- Contact book: a fully dynamic record system where both the list and each name grow to fit, sanitizer-clean

Practice: Extend the contact book with delete and sort, keeping the sanitizer report clean

Assessment: Month 4 practical: build a small dynamic record system from a spec in one sitting, leak-free, and defend the design

PHASE 3: C AT WORK: FILES, DATA STRUCTURES, AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24)

The skills become systems: file handling, linked lists, stacks, queues, and sorting built from raw structs and malloc, a grounded look at where C runs in industry, and a capstone project compiled and defended.

Month 5: File Handling and Data Structures from Scratch (Weeks 17-20)

File Handling

Topics:

- fopen, fclose, and file modes that actually matter
- Reading lines with fgets and parsing them
- Writing structured data with fprintf
- Binary files: fread and fwrite for whole records
- Error handling: what to do when the file is not there
- Saving and loading the contact book: persistence at last

Projects built:

- Persistent contact book: records saved to disk in both text and binary form, reloaded on startup

Practice: Write a CSV report generator that reads raw marks from one file and writes a formatted result sheet to another

Linked Lists

Topics:

- Why arrays are not enough: the case for nodes and links
- The node struct: data plus a pointer to the next
- Insertion at head, tail, and middle, each with a diagram first
- Traversal and search
- Deletion without losing the rest of the list
- The pointer-rewiring discipline: draw, then code, always

Projects built:

- Singly linked list module: insert, delete, search, reverse, and print, built from scratch and sanitizer-clean

Practice: Solve six classic list problems including middle element, nth from end, and cycle detection by hand-tracing first

Stacks and Queues

Topics:

- The stack as a discipline: push, pop, peek
- Array-backed and list-backed stack implementations
- Balanced brackets: the interview classic, solved properly
- Queues: enqueue, dequeue, and the circular buffer trick
- Where the machine itself uses stacks and queues
- Choosing a backing structure and defending the choice

Projects built:

- Bracket validator and a printer-queue simulator, each built on your own stack and queue modules

Practice: Implement a stack-based postfix expression evaluator and test it against ten expressions

Searching, Sorting, and Function Pointers

Topics:

- Linear and binary search, with the sorted-input contract
- Bubble, selection, and insertion sort, traced and timed
- Counting comparisons: a first taste of efficiency analysis
- Function pointers: passing behavior as an argument
- qsort from the standard library and writing its comparator
- Sorting an array of structs by any field

Projects built:

- Sortable student database: the record system gains sort-by-marks, sort-by-name, and binary search, using function pointers

Practice: Time three sorts on arrays of 1k, 10k, and 100k elements and write four sentences on what the numbers show

Assessment: Data structures lab exam: one list problem and one stack or queue problem implemented live from scratch

Month 6: Systems Context and the Capstone (Weeks 21-24)

Where C Runs: Bits, Kernels, and Microcontrollers

Topics:

- Bitwise operators: and, or, xor, shifts, and masks
- Flags and registers: why embedded code reads and writes single bits
- A grounded tour: C in operating systems, databases, firmware, and language runtimes
- What embedded development actually involves, without the hype
- Reading a small piece of real open-source C and surviving
- gdb basics: breakpoints, stepping, and inspecting memory in a debugger

Projects built:

- Bit toolkit: set, clear, toggle, and test any bit, plus a permissions system stored in a single byte

Practice: Use gdb to find the planted bug in a provided program without reading its source first

Capstone: Scope and Foundations

Topics:

- Choosing a capstone: bank record system, inventory manager, text-based game with saves, or a proposal of your own
- Writing the spec: features, data structures, and file format decided before coding
- Project skeleton: modules, headers, and a Makefile from day one
- Building the data layer first
- Estimating honestly what three weeks can hold
- Instructor sign-off on scope before the sprint starts

Projects built:

- Approved capstone spec plus a compiling skeleton with the core data structures in place

Practice: Implement and test your capstone's data module in isolation before any menus or files exist

Capstone: Build and Review

Topics:

- Feature-by-feature building, smallest first
- Keeping the sanitizer clean while the codebase grows
- Instructor code review: memory management and module boundaries
- Debugging your own project with gdb instead of printf sprinkles
- Handling bad input like software that respects its user
- Trimming scope with a straight face when the clock demands it

Projects built:

- Feature-complete capstone: all core features working, file persistence in, sanitizer report clean

Practice: Exchange builds with a classmate, file two bug reports on theirs, and fix the ones filed on yours

Ship and Defend

Topics:

- The final pass: a written test checklist run against every feature
- README and build instructions a stranger could follow
- Publishing the semester's work to GitHub properly
- Presenting a systems project: what it does, then how memory flows through it
- Answering pointed questions about your own pointers
- What comes next: C++, embedded, operating systems, or DSA in depth

Projects built:

- Delivered capstone: compiled, documented, pushed to GitHub, and defended in a live walkthrough

Practice: Write the README, then have someone outside the class build and run your project using only that file

Assessment: Capstone defense: live demo, memory-flow explanation, and Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Limits explorer that maps every basic type on your machine
- Pattern engine and number theory function library
- Recursive toolkit with Tower of Hanoi
- Multi-file unit converter with its own Makefile
- mini_string.h: five standard string functions written from scratch
- Growable dynamic array that passes sanitizer checks
- Fully dynamic contact book with file persistence
- Singly linked list, stack, and queue modules built from raw structs
- Sortable student database using function pointers and qsort
- Bit toolkit with a one-byte permissions system
- Capstone: a complete record system or text game, compiled, documented, and defended

Total projects: 15+ compiled programs and modules, ending with a defended capstone on GitHub

SKILLS THE STUDENT WILL HAVE AT THE END

- The compilation pipeline: preprocessor, compiler, linker, and multi-file builds with Makefiles
- Pointers with total confidence: addresses, dereferencing, pointer arithmetic, and pointer parameters
- Dynamic memory: malloc, realloc, free, ownership discipline, and hunting leaks with sanitizers
- Structs and file handling: record systems that persist to text and binary files
- Data structures from scratch: linked lists, stacks, queues, searching, and sorting
- Systems literacy: bitwise operations, gdb debugging, and reading real-world C

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, writing and compiling code alongside the instructor

Practice: 3-4 hours of problem sets and project work between classes, with paper tracing where it teaches best

Review: Homework reviewed with written feedback; memory diagrams corrected line by line when needed

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so a segfault at midnight has somewhere to go

Progress updates: Regular progress notes covering what was mastered and what needs another pass

Career guidance: Honest direction on next steps: C++, embedded, operating systems coursework, or DSA, based on what each student showed aptitude for

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, backed by a GitHub portfolio of compiled projects

WHERE THIS CAN LEAD

- Our C++ programming course, which builds directly on C and adds objects, templates, and the STL
- Data structures and algorithms in depth, for placements and competitive programming
- Embedded systems and firmware learning paths, where C is the industry default
- Operating systems and computer architecture coursework, met with real preparation
- Contributing to open-source C projects with a working command of memory and build tools

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Should I learn C or C++ first?

A: If your college teaches a C paper, learn C first; you will be examined on it either way, and C's small size makes memory concepts unavoidable rather than optional. C++ then becomes a comfortable extension: our C++ course assumes exactly the ground this one covers. Students heading straight for application development or game engines can start with C++ instead, and we will tell you honestly in the demo class which fits your situation.

Q: Is C still worth learning, or is it outdated?

A: C is old and very much alive. Operating system kernels including Linux and Windows internals, databases like PostgreSQL and SQLite, embedded firmware in cars and appliances, and the reference interpreters of Python and PHP are all C codebases maintained today. Nobody builds websites in C, and we will not pretend otherwise; you learn it because it teaches how computers actually work and because systems, embedded, and university curricula still run on it.

Q: I have never programmed before. Can I start with C?

A: Yes, and roughly half of each batch does. The course assumes nothing and moves through fundamentals at a deliberate pace with paper tracing built into the homework. C as a first language is demanding but honest: there is no framework hiding what your code does, so what you learn here transfers to every language you touch afterwards.

Q: Everyone says pointers are the hard part. How do you teach them?

A: Slowly, visually, and for a full month. Students who finish month 3 routinely say pointers were explained backwards everywhere else.

Q: Will this course help with my university C syllabus and exams?

A: Deliberately, yes. Lab-exam-style live coding is part of the monthly assessments, so the exam format itself gets practiced, not just the content.

Q: Do I need Linux, or will Windows work?

A: Windows works fine; we set up GCC and VS Code on whatever machine you have in the first week, and Mac and Linux are equally supported. Linux-specific tools like Valgrind are demonstrated for those who have access, but every required exercise runs on all three platforms using the compiler and sanitizers alone.

Q: What will I have to show at the end of six months?

A: A GitHub profile with fifteen or more compiled projects, including your own string library, a dynamic contact book, linked list and stack modules built from scratch, and a capstone record system or text game with file persistence and a README a stranger can build from. Just as importantly, you will be able to defend the memory behavior of your own code out loud, which is exactly what interviews and vivas probe.

Q: Can I try a class before enrolling?

A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Bring a question about pointers if you have one; it is our favorite kind of demo.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/c-programming-course-for-college-beginners>

=====

COURSE 27 OF 118: C++ COURSE FOR TEENS ONLINE: GAMES & COMPETITIVE CODING

=====

Course link: <https://learn.modernagecoders.com/courses/cpp-programming-masterclass-for-teens>

Subject area: Teen C++ Programming

Designed for: Teens

Who can join: Complete Beginner to Advanced C++ Developer

Course length: 12 months (52 weeks)

Time needed: 10-12 hours/week recommended

Certificate: Professional C++ Developer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online C++ course for teens: a 12-month journey from zero to advanced, building real games and mastering the competitive programming behind AAA titles.

In one line: From 'Hello World' to 'I Built My Own Game Engine'

ABOUT THE PROGRAMME

Ever wondered how Minecraft, Fortnite, or your favorite games are built? Want to compete in programming competitions and win? This is your path to mastering C++, the most powerful programming language used by game developers, system programmers, and competitive coders worldwide. Designed specifically for teenagers, you'll learn C++ by building games, creating graphics applications, solving algorithmic challenges, and even programming robots. No boring theory - every concept is taught through projects you'll actually want to build. By the end, you'll have created your own games, competitive programming portfolio, and projects that get you into top colleges or land high-paying jobs.

WHAT MAKES THIS COURSE DIFFERENT

- Teen-focused: Build games, graphics apps, and projects teens love
- Zero to hero: No programming experience needed
- Game development focus: Learn C++ by building actual games
- Competitive programming: Prepare for USACO, Codeforces, and more
- Visual learning: See your code create graphics and animations
- Real applications: Build tools, games, and system utilities
- College prep: Projects that impress admissions committees
- Industry skills: Learn what game studios actually use

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): C++ basics, problem solving, first games

Phase 2: Intermediate (Months 4-6): OOP, data structures, algorithms

Phase 3: Advanced (Months 7-9): Graphics, game development, systems

Phase 4: Mastery (Months 10-12): Competitive programming, projects, specialization

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No prerequisites - complete beginner friendly

Coding experience: Zero programming knowledge required

Equipment: Any computer with internet

Time commitment: 10-12 hours per week

English: Basic English understanding

Motivation: Interest in games, problem-solving, or technology

WHO THIS COURSE IS FOR

Students: Teenagers (13-19) interested in programming

Working professionals: Young adults learning C++

Entrepreneurs: Teens wanting to build games/apps

Freelancers: Future game developers

Kids: Motivated kids 12+ with support

Anyone: Anyone passionate about programming

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: C++ FUNDAMENTALS & FIRST PROGRAMS (MONTHS 1-3, WEEKS 1-13)

Start from absolute zero and write your first C++ programs. Build games, solve puzzles, and understand how computers really work.

Month 1: C++ Basics & Setup (Week 1-4)

Introduction to C++ & Programming

Topics:

- Why C++? The language behind every major game
- C++ history: From 1979 to modern gaming
- Setting up your development environment
- Installing Visual Studio/VS Code/CLion
- Compilers: GCC, Clang, MSVC explained
- Your first C++ program: Hello, World!

- Understanding #include and using namespace
- The main() function: Where programs start
- Compilation process: From code to executable
- cout and cin: Output and input
- Comments: Single-line and multi-line
- Common errors and how to fix them

Projects built:

- Hello World variations
- ASCII art generator
- Personal introduction program
- Simple conversation bot

Practice: Daily: Write and compile 5 small programs

Variables & Data Types

Topics:

- Variables: Storing game scores and player data
- Data types: int, float, double, char, bool
- Declaring and initializing variables
- Constants with const and #define
- Type casting: Converting between types
- Arithmetic operators: +, -, *, /, %
- Compound operators: +=, -=, *=, /=
- Increment/decrement: ++ and --
- Math library functions: sqrt, pow, abs
- Random numbers for games
- String basics: Working with text
- Character manipulation

Projects built:

- Calculator with all operations
- Temperature converter
- Random dice roller for D&D
- Simple RPG stat calculator

Practice: Solve 30 problems on variables and operators

Control Flow & Decision Making

Topics:

- If statements: Making decisions in code
- Else and else-if chains
- Comparison operators: ==, !=, <, >, <=, >=
- Logical operators: &&, ||, !
- Switch statements: Multiple choice menus
- Ternary operator: Compact conditions
- Nested conditions: Complex decisions
- Game logic with conditions
- Input validation techniques
- Menu-driven programs
- State machines basics
- Debugging conditional logic

Projects built:

- Text adventure game
- Grade calculator with letter grades
- Rock-Paper-Scissors game
- Simple AI decision maker

Practice: Build 10 programs with complex logic

Loops & Repetition

Topics:

- For loops: Repeating code efficiently
- While loops: Conditional repetition
- Do-while loops: Guaranteed execution
- Nested loops: 2D patterns and grids
- Break and continue statements
- Infinite loops and game loops
- Loop optimization techniques
- Pattern printing: Stars, pyramids, diamonds
- Number sequences: Fibonacci, primes
- ASCII art with loops
- Animation basics with loops
- Common loop patterns

Projects built:

- Pattern generator program
- Prime number finder
- Multiplication table generator
- Simple animation loop

Practice: Complete 40 loop-based challenges

Month 2: Functions & Arrays (Week 5-8)

Functions - Code Reusability

Topics:

- Functions: Building blocks of programs
- Function declaration and definition
- Parameters and arguments

- Return values and void functions
- Pass by value vs pass by reference
- Function overloading: Same name, different parameters
- Default parameters
- Inline functions for performance
- Recursive functions: Functions calling themselves
- Function prototypes
- Scope and lifetime of variables
- Creating game utility functions

Projects built:

- Math library with custom functions
- Game utility functions toolkit
- Recursive maze solver
- Menu system with functions

Practice: Write 30 different utility functions

Arrays & Strings

Topics:

- Arrays: Storing multiple values
- Declaring and initializing arrays
- Accessing array elements
- Array bounds and safety
- Multi-dimensional arrays: Game boards
- Character arrays and C-strings
- String manipulation functions
- Array algorithms: Search and sort
- Passing arrays to functions
- Dynamic memory preview
- Common array problems
- Game inventory systems

Projects built:

- Tic-Tac-Toe game
- Student grade manager
- Word game (Wordle clone)
- 2D maze game

Practice: Solve 25 array manipulation problems

Pointers & References

Topics:

- Pointers: The power of C++
- Memory addresses and the & operator
- Pointer declaration and dereferencing
- Null pointers and safety
- Pointer arithmetic
- Arrays and pointers relationship
- References: Safer than pointers
- Const pointers and references
- Function pointers basics
- Common pointer mistakes
- Memory visualization
- Why games need pointers

Projects built:

- Dynamic array implementation
- Memory explorer program
- Pointer-based data swap
- Simple memory game

Practice: Master 20 pointer exercises

Dynamic Memory Management

Topics:

- Stack vs Heap memory
- Dynamic memory with new and delete
- Dynamic arrays
- Memory leaks and how to avoid them
- Delete vs delete[]
- Smart pointers preview
- RAII principle basics
- Common memory errors
- Debugging memory issues
- Valgrind and sanitizers
- Memory-efficient programming
- Game memory management

Projects built:

- Dynamic inventory system
- Resizable array class
- Memory pool allocator
- Dynamic game world

Practice: Build 10 programs with dynamic memory

Month 3: Object-Oriented Basics

Topics:

- Introduction to OOP
- Classes and objects

- Member variables and functions
- Access specifiers: public, private, protected
- Constructors and destructors
- This pointer
- Const member functions
- Static members
- Friend functions and classes
- Operator overloading basics
- Class design principles
- UML diagrams basics

Projects built:

- Game character class
- Bank account system
- Student record system
- Simple physics engine

Practice: Design 10 different classes

Month 3: Structures & File I/O (Week 9-13)

Structures & Enums

Topics:

- Structures: Grouping related data
- Struct declaration and initialization
- Accessing struct members
- Nested structures
- Arrays of structures
- Structures and functions
- Structures vs classes
- Enumerations for named constants
- Enum classes (scoped enums)
- Typedef and using aliases
- Unions for memory efficiency
- Bit fields for flags

Projects built:

- Game entity structure system
- Student database with structs
- RPG item system
- Configuration manager

Practice: Create 15 programs using structures

File Input/Output

Topics:

- File streams: ifstream, ofstream, fstream
- Opening and closing files
- Reading from text files
- Writing to text files
- File error handling
- Binary file I/O
- File positioning and seeking
- CSV file processing
- JSON parsing basics
- Save game systems
- Configuration files
- Log file creation

Projects built:

- Save/load game system
- High score tracker
- Text file analyzer
- Simple database file system

Practice: Build 10 file processing utilities

Error Handling & Debugging

Topics:

- Types of errors: Compile, runtime, logic
- Exception handling: try, catch, throw
- Standard exceptions
- Custom exception classes
- Exception safety guarantees
- Assert statements
- Error codes vs exceptions
- Debugging with GDB/LLDB
- Visual Studio debugger
- Breakpoints and watchpoints
- Stack traces and core dumps
- Logging and diagnostics

Projects built:

- Robust calculator with error handling
- File parser with exceptions
- Game with error recovery
- Debug tool suite

Practice: Debug 20 buggy programs

STL Basics - Standard Template Library

Topics:

- Introduction to STL
- Vectors: Dynamic arrays made easy
- Vector operations and methods
- Strings: Modern C++ strings
- String manipulation methods
- Iterators basics
- Range-based for loops
- Algorithm library preview
- Sort and search algorithms
- STL vs raw arrays
- When to use STL
- Performance considerations

Projects built:

- Contact manager with vectors
- Text processor with strings
- Sorting visualizer
- Word frequency analyzer

Practice: Convert array programs to use STL

Phase 1 Capstone Project

Topics:

- Project planning and design
- Code organization
- Documentation
- Testing strategies
- Version control with Git
- Code review preparation

Projects built:

- CAPSTONE: Console RPG Game
- Features: Characters, inventory, combat, save/load, multiple levels
- Alternative: Student management system
- Alternative: Text-based strategy game

Assessment: Phase 1 Final - Build complete C++ application

----- PHASE 2: OBJECT-ORIENTED PROGRAMMING & DATA STRUCTURES (MONTHS 4-6, WEEKS 14-26) -----

Master OOP concepts and build complex data structures. Create game engines and advanced applications.

Months 4-5: Advanced OOP (Week 14-21)

Inheritance & Polymorphism

Topics:

- Inheritance: Creating class hierarchies
- Base and derived classes
- Protected access specifier
- Types of inheritance: public, private, protected
- Constructor and destructor in inheritance
- Function overriding
- Virtual functions and late binding
- Pure virtual functions
- Abstract classes and interfaces
- Multiple inheritance
- Diamond problem and virtual inheritance
- Polymorphism in game development

Projects built:

- Game entity hierarchy
- Shape drawing system
- Vehicle simulation
- RPG character classes

Practice: Build 10 inheritance hierarchies

Advanced Class Features

Topics:

- Copy constructor deep dive
- Assignment operator overloading
- Rule of Three/Five/Zero
- Move semantics and rvalue references
- Move constructor and assignment
- Perfect forwarding
- Operator overloading complete guide
- Friend functions and operators
- Conversion operators
- Functors (function objects)
- Nested classes
- Anonymous classes

Projects built:

- Custom string class
- Matrix math library
- Smart pointer implementation

- Complex number class

Practice: Implement 15 custom operators

Templates & Generic Programming

Topics:

- Function templates: Generic functions
- Class templates: Generic classes
- Template parameters
- Template specialization
- Partial specialization
- Variadic templates basics
- Template metaprogramming intro
- SFINAE principle
- Type traits
- Concepts (C++20)
- STL container implementation
- Template best practices

Projects built:

- Generic container classes
- Template math library
- Type-safe event system
- Generic game components

Practice: Create 10 template-based utilities

Design Patterns

Topics:

- What are design patterns?
- Singleton pattern: One instance only
- Factory pattern: Object creation
- Observer pattern: Event systems
- Strategy pattern: Swappable algorithms
- Command pattern: Undo/redo
- State pattern: Game states
- Component pattern: Game objects
- Prototype pattern: Cloning
- Adapter pattern: Interface compatibility
- MVC pattern basics
- Anti-patterns to avoid

Projects built:

- Game state manager
- Event system implementation
- Command pattern editor
- Plugin system

Practice: Implement all major patterns

Smart Pointers & Modern C++

Topics:

- Problems with raw pointers
- `unique_ptr`: Exclusive ownership
- `shared_ptr`: Shared ownership
- `weak_ptr`: Breaking cycles
- `make_unique` and `make_shared`
- Custom deleters
- RAII in practice
- Auto keyword and type inference
- Range-based loops
- Lambda expressions
- `std::function` and `std::bind`
- Modern C++ best practices

Projects built:

- Memory-safe game engine
- Resource manager with smart pointers
- Callback system with lambdas
- Modern C++ game framework

Practice: Refactor old code with modern C++

Month 6: Data Structures (Week 22-26)

Linear Data Structures

Topics:

- Linked lists: Single, double, circular
- Implementing linked list operations
- Stacks: LIFO principle
- Stack applications: Undo, parsing
- Queues: FIFO principle
- Circular queues and dequeues
- Priority queues
- Implementing with arrays vs nodes
- STL containers: list, stack, queue
- Time complexity analysis
- Space complexity
- Real-world applications

Projects built:

- Custom linked list class
- Expression evaluator with stack
- Task scheduler with queues
- Undo/redo system

Practice: Implement all data structures from scratch

Trees & Graphs

Topics:

- Binary trees: Structure and properties
- Tree traversals: Inorder, preorder, postorder
- Binary search trees (BST)
- BST operations: Insert, delete, search
- Balanced trees: AVL basics
- Heaps and priority queues
- Graphs: Representation methods
- Graph traversals: DFS and BFS
- Shortest path algorithms preview
- Tree applications in games
- Scene graphs in game engines
- Decision trees for AI

Projects built:

- BST implementation
- File system tree
- Game AI decision tree
- Pathfinding visualizer

Practice: Solve 20 tree/graph problems

Hash Tables & Sets

Topics:

- Hash tables: Fast lookups
- Hash functions design
- Collision resolution: Chaining vs open addressing
- Load factors and resizing
- Implementing hash map
- Sets and multisets
- Ordered vs unordered containers
- STL: `unordered_map`, `unordered_set`
- map and set (tree-based)
- Custom hash functions
- Applications in game development
- Caching and memoization

Projects built:

- Custom hash table
- Spell checker with hash set
- Game object manager
- Cache system implementation

Practice: Build 10 hash table applications

Algorithms - Searching & Sorting

Topics:

- Linear search vs binary search
- Binary search implementation
- Bubble sort: Simple but slow
- Selection sort and insertion sort
- Merge sort: Divide and conquer
- Quick sort: Fast average case
- Heap sort with heaps
- Counting sort and radix sort
- STL sorting algorithms
- Custom comparators
- Sorting visualization
- Algorithm complexity analysis

Projects built:

- Sorting algorithm visualizer
- Game leaderboard system
- Search engine prototype
- Data analysis tool

Practice: Implement all sorting algorithms

Advanced Algorithms

Topics:

- Recursion mastery
- Backtracking algorithms
- Dynamic programming basics
- Greedy algorithms
- Divide and conquer
- Graph algorithms: Dijkstra, A*
- String algorithms basics
- Pattern matching
- Bit manipulation tricks
- Algorithm optimization
- Competitive programming intro
- Problem-solving strategies

Projects built:

- Sudoku solver (backtracking)
- Pathfinding for games (A*)
- Text search tool
- Optimization problems solver

Practice: Solve 30 algorithmic challenges

Phase 2 Capstone & STL Mastery (Week 25-26)

STL Containers Deep Dive

Topics:

- Vector internals and performance
- Deque: Double-ended queue
- List vs vector performance
- Set and map internals (red-black trees)
- Unordered containers performance
- Container adaptors
- Custom allocators
- Iterator categories
- Iterator adaptors
- Reverse iterators
- Container selection guide
- Performance benchmarking

Projects built:

- Performance comparison tool
- Custom container library
- Game inventory with STL
- Database with STL

Practice: Master all STL containers

STL Algorithms & Functional

Topics:

- Algorithm library overview
- Non-modifying algorithms
- Modifying algorithms
- Sorting and searching
- Numeric algorithms
- Lambda expressions mastery
- Function objects and adaptors
- `std::function` and `std::bind`
- Functional programming in C++
- Ranges (C++20)
- Parallel algorithms
- Custom algorithms

Projects built:

- Data processing pipeline
- Functional game systems
- Algorithm visualizer
- Performance testing suite

Practice: Use 50 different STL algorithms

Multithreading Basics

Topics:

- Why multithreading matters
- Creating threads with `std::thread`
- Joining and detaching threads
- Passing arguments to threads
- Thread synchronization problems
- Mutexes and locks
- Condition variables
- Atomic operations
- Thread-safe data structures
- Producer-consumer pattern
- Thread pools basics
- Multithreading in games

Projects built:

- Multithreaded downloader
- Parallel sorting
- Thread-safe queue
- Game with worker threads

Practice: Write 10 multithreaded programs

Memory Management Mastery

Topics:

- Stack vs heap deep dive
- Memory layout of programs
- Custom memory allocators
- Pool allocators
- Stack allocators
- Memory debugging tools
- Profiling memory usage
- Cache optimization
- Data-oriented design

- Memory-mapped files
- Shared memory
- Memory in game engines

Projects built:

- Custom memory manager
- Memory pool for games
- Memory profiler
- Cache-friendly data structures

Practice: Optimize memory in all projects

Phase 2 Capstone Project

Topics:

- Large project planning
- Architecture design
- Code organization
- Documentation
- Testing strategies
- Performance optimization

Projects built:

- MAJOR CAPSTONE: 2D Game Engine
- Features: Sprites, physics, collision, sound, levels, UI
- Alternative: Data structure visualization tool
- Alternative: Competitive programming judge system

Assessment: Phase 2 Final - Complex C++ application

PHASE 3: GRAPHICS, GAMES & SYSTEM PROGRAMMING (MONTHS 7-9, WEEKS 27-39)

Dive into graphics programming, game development, and system-level programming.

Months 7-8: Graphics Programming (Week 27-34)

2D Graphics with SFML

Topics:

- Introduction to SFML
- Setting up SFML project
- Creating windows
- Basic shapes and colors
- Sprites and textures
- Text rendering
- Handling events: Keyboard, mouse
- Game loop implementation
- Frame rate and delta time
- 2D transformations
- View and camera
- Sound and music

Projects built:

- Pong game clone
- Snake game
- Particle system
- 2D platformer prototype

Practice: Create 10 mini-games with SFML

Advanced 2D Graphics

Topics:

- Sprite animation
- Sprite sheets and atlases
- Tile maps and level design
- Collision detection: AABB, circles
- Physics basics: Velocity, acceleration
- Particle effects
- Shaders introduction
- Post-processing effects
- UI systems for games
- Scene management
- Resource management
- Game state machines

Projects built:

- Platform game with physics
- Tower defense game
- RPG with tile maps
- Particle effects editor

Practice: Build complete 2D game

OpenGL Basics

Topics:

- 3D graphics fundamentals
- OpenGL and GLFW setup
- OpenGL rendering pipeline
- Vertices and triangles
- Vertex buffers and arrays
- Shaders: Vertex and fragment
- GLSL basics

- Matrices and transformations
- 3D coordinate systems
- Camera and projection
- Texturing 3D objects
- Lighting basics

Projects built:

- 3D rotating cube
- Solar system simulation
- 3D maze game
- Model viewer

Practice: Create 10 OpenGL demos

Game Physics

Topics:

- Physics in games
- Vectors and math for physics
- Rigid body dynamics
- Collision detection algorithms
- Collision response
- Gravity and forces
- Springs and constraints
- Particle physics
- Integrating physics engines
- Box2D for 2D physics
- Bullet Physics basics
- Optimization techniques

Projects built:

- Physics sandbox
- Angry Birds clone
- Racing game physics
- Ragdoll simulation

Practice: Implement physics in 5 games

Audio Programming

Topics:

- Digital audio basics
- Audio file formats
- Playing sounds with SFML
- 3D spatial audio
- Audio mixing and effects
- Real-time audio synthesis
- MIDI basics
- OpenAL introduction
- Audio in game engines
- Music synchronization
- Audio optimization
- Creating sound effects

Projects built:

- Music visualizer
- Rhythm game prototype
- 3D audio demo
- Sound effect generator

Practice: Add audio to all games

Month 9: System Programming (Week 35-39)

Operating System Interfaces

Topics:

- System calls and OS APIs
- Process creation and management
- Inter-process communication
- Pipes and message queues
- Shared memory
- Signals and handlers
- File system operations
- Directory traversal
- File permissions
- Memory-mapped I/O
- System information
- Performance monitoring

Projects built:

- Task manager clone
- File explorer
- System monitor
- Process communication demo

Practice: Build 10 system utilities

Network Programming

Topics:

- Network fundamentals
- TCP/IP basics
- Sockets programming
- Client-server architecture

- TCP server and client
- UDP communication
- Non-blocking I/O
- Select and poll
- HTTP protocol basics
- Building web servers
- Multiplayer game networking
- Network security basics

Projects built:

- Chat application
- File transfer program
- Multiplayer game server
- HTTP server

Practice: Create 5 networked applications

Database Programming

Topics:

- Database basics
- SQL fundamentals
- SQLite integration
- C++ database APIs
- Prepared statements
- Transaction management
- Database design for games
- Save game systems
- Leaderboard databases
- Configuration storage
- Data serialization
- JSON and XML parsing

Projects built:

- Game save system
- High score database
- Player stats tracker
- Inventory database

Practice: Add databases to games

Embedded & Arduino

Topics:

- Embedded systems basics
- Arduino and C++
- Digital I/O
- Analog input
- PWM and servos
- Sensors and actuators
- Serial communication
- I2C and SPI
- Interrupts and timers
- Power management
- Real-time constraints
- IoT basics

Projects built:

- LED game controller
- Temperature monitor
- Robot car
- Game console with Arduino

Practice: Build 5 Arduino projects

Phase 3 Integration

Topics:

- Combining graphics and networking
- Client-server game architecture
- Game engine architecture
- Plugin systems
- Scripting integration
- Performance profiling
- Release builds
- Distribution and packaging

Projects built:

- Multiplayer graphics game
- Game with all features
- System integration project
- Performance optimization

Practice: Integrate all learned concepts

Phase 3 Capstone & Specialization (Week 38-39)

Game Engine Architecture

Topics:

- Game engine components
- Entity-component systems
- Scene graphs
- Resource managers
- Input systems

- Physics integration
- Rendering systems
- Audio systems
- Scripting languages
- Level editors
- Asset pipelines
- Engine optimization

Projects built:

- Mini game engine
- Component system
- Level editor
- Asset manager

Practice: Build game engine components

AI for Games

Topics:

- Game AI fundamentals
- Finite state machines
- Behavior trees
- Pathfinding: A* algorithm
- Navigation meshes
- Steering behaviors
- Decision making
- Minimax for board games
- Neural networks basics
- Genetic algorithms
- Procedural generation
- AI debugging

Projects built:

- Chess AI
- FPS enemy AI
- Racing game AI
- Procedural level generator

Practice: Implement AI in 5 games

Optimization & Performance

Topics:

- Profiling tools
- CPU optimization
- Memory optimization
- Cache optimization
- SIMD instructions
- Multithreading for games
- GPU optimization
- Asset optimization
- Load time optimization
- Mobile optimization
- Console optimization
- Benchmarking

Projects built:

- Performance profiler
- Optimized renderer
- Memory optimizer
- Benchmark suite

Practice: Optimize all projects

Modern C++ Features

Topics:

- C++17 features
- C++20 features
- Structured bindings
- `std::optional` and `std::variant`
- `std::filesystem`
- Coroutines basics
- Modules introduction
- Concepts deep dive
- Ranges library
- Format library
- Three-way comparison
- Future C++ features

Projects built:

- Modern C++ game
- Filesystem utilities
- Coroutine examples
- C++20 showcase

Practice: Use modern C++ features

Phase 3 Capstone Project

Topics:

- Large game project planning
- Technical design document
- Art and asset planning
- Milestone scheduling

- Team collaboration
- Version control with Git

Projects built:

- MAJOR CAPSTONE: Complete 3D Game
- Features: 3D graphics, physics, AI, networking, audio
- Alternative: Game engine from scratch
- Alternative: Robotics project with Arduino

Assessment: Phase 3 Final - Professional game or system

PHASE 4: COMPETITIVE PROGRAMMING & CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52)

Master competitive programming, build portfolio projects, and prepare for careers or college.

Months 10-11: Competitive Programming (Week 40-47)

Competitive Programming Foundations

Topics:

- Introduction to competitive programming
- Online judges: Codeforces, AtCoder, CodeChef
- USACO preparation
- Reading problem statements
- Input/output optimization
- Time complexity mastery
- Space complexity optimization
- Common patterns
- Debugging strategies
- Contest strategies
- Speed coding techniques
- Template preparation

Projects built:

- Personal template library
- Problem tracker
- Solution archive
- Contest simulator

Practice: Solve 100 easy problems

Dynamic Programming

Topics:

- DP fundamentals
- Memoization vs tabulation
- 1D DP problems
- 2D DP problems
- State space reduction
- Classic problems: Knapsack, LCS, LIS
- Coin change and variants
- Path counting problems
- Interval DP
- Tree DP
- Digit DP
- Optimization techniques

Projects built:

- DP problem solver
- Visualization tool
- Problem generator
- Solution verifier

Practice: Master 50 DP problems

Graph Algorithms Advanced

Topics:

- Graph representations
- DFS and BFS applications
- Shortest paths: Dijkstra, Bellman-Ford, Floyd-Warshall
- Minimum spanning trees: Kruskal, Prim
- Topological sorting
- Strongly connected components
- Bridges and articulation points
- Bipartite matching
- Network flow basics
- Tree algorithms
- LCA and tree queries
- Graph coloring

Projects built:

- Graph algorithm visualizer
- Pathfinding comparison
- Network flow solver
- Graph problem generator

Practice: Solve 40 graph problems

Advanced Data Structures

Topics:

- Segment trees
- Fenwick trees (BIT)

- Sparse tables
- Disjoint set union (DSU)
- Trie data structure
- Suffix arrays
- Heavy-light decomposition
- Persistent data structures
- Treaps and splay trees
- Link-cut trees basics
- Square root decomposition
- Mo's algorithm

Projects built:

- Advanced DS library
- Range query solver
- String matching tool
- DS visualizer

Practice: Implement all advanced structures

Mathematics for CP

Topics:

- Number theory basics
- Prime numbers and sieve
- GCD and LCM
- Modular arithmetic
- Fast exponentiation
- Combinatorics
- Probability basics
- Game theory
- Geometry algorithms
- Convex hull
- Line intersection
- FFT basics

Projects built:

- Math utility library
- Geometry solver
- Number theory toolkit
- Combinatorics calculator

Practice: Solve 30 math problems

Month 11: Advanced Topics & Projects (Week 44-47)

String Algorithms

Topics:

- String matching: KMP, Rabin-Karp
- Z-algorithm
- Suffix arrays and LCP
- Suffix trees basics
- Aho-Corasick algorithm
- Manacher's algorithm
- String hashing
- Palindrome problems
- Edit distance
- Regular expressions
- Parsing algorithms
- Compression basics

Projects built:

- Text search engine
- Plagiarism detector
- DNA sequence matcher
- Text editor with search

Practice: Master string algorithms

Computational Geometry

Topics:

- 2D geometry basics
- Points, lines, segments
- Polygon algorithms
- Convex hull algorithms
- Line sweep technique
- Closest pair of points
- Voronoi diagrams basics
- Delaunay triangulation
- 3D geometry basics
- Geometric data structures
- Collision detection
- Visualization techniques

Projects built:

- Geometry playground
- Collision detector
- Map overlay tool
- 3D viewer

Practice: Solve geometry problems

Machine Learning with C++

Topics:

- ML basics in C++
- Linear regression
- Logistic regression
- Neural networks from scratch
- Backpropagation implementation
- Decision trees
- K-means clustering
- OpenCV basics
- Image processing
- Computer vision basics
- Integration with Python
- Performance optimization

Projects built:

- Neural network library
- Image classifier
- Digit recognizer
- Game AI with ML

Practice: Implement ML algorithms

Cryptography Basics

Topics:

- Cryptography fundamentals
- Caesar cipher and variants
- XOR encryption
- Hash functions
- SHA implementation
- Public key concepts
- RSA basics
- Digital signatures
- Random number generation
- Secure coding practices
- Common vulnerabilities
- CTF challenges

Projects built:

- Encryption tool
- Password manager
- Secure chat app
- CTF challenge solver

Practice: Solve cryptography challenges

Compiler Design Basics

Topics:

- Compilers overview
- Lexical analysis
- Tokenization
- Parsing basics
- Abstract syntax trees
- Semantic analysis
- Code generation basics
- Interpreters vs compilers
- Simple language design
- VM implementation
- Optimization basics
- Error handling

Projects built:

- Calculator language
- Simple interpreter
- Mini compiler
- Domain-specific language

Practice: Build language tools

Month 12: Portfolio & Career (Week 48-52)

Portfolio Development

Topics:

- Portfolio strategy
- Project selection
- Code documentation
- README writing
- GitHub profile optimization
- Project presentation
- Demo videos
- Technical writing
- Blog creation
- Online presence
- Personal branding
- Networking strategies

Projects built:

- Portfolio website
- Project showcases
- Technical blog posts

- Video demos

Practice: Polish all projects

Interview Preparation

Topics:

- Technical interview process
- Data structures review
- Algorithm review
- Coding interview patterns
- System design basics
- Behavioral questions
- STAR method
- Mock interviews
- Whiteboard coding
- Online assessments
- Company research
- Salary negotiation

Projects built:

- Interview prep notes
- Problem solutions
- Mock interview recordings
- Question bank

Practice: Daily interview practice

Open Source & Collaboration

Topics:

- Open source importance
- Finding projects
- Understanding large codebases
- Making contributions
- Pull request best practices
- Code review participation
- Issue tracking
- Documentation contributions
- Creating open source projects
- Building community
- Licensing
- Maintaining projects

Projects built:

- Open source contributions
- Personal open source project
- Documentation improvements
- Community building

Practice: Contribute to 5 projects

Final Project Development

Topics:

- Capstone project planning
- Requirements analysis
- System architecture
- Implementation strategy
- Testing plan
- Documentation plan
- Deployment strategy
- Marketing strategy

Projects built:

- FINAL CAPSTONE START
- Complete planning
- Architecture design
- Initial implementation

Practice: Final project sprint

Week 51-52: Graduation & Launch

Final Project Completion

Topics:

- Feature completion
- Testing and debugging
- Performance optimization
- Documentation finalization
- Deployment preparation
- Release planning
- Marketing materials
- Demo preparation
- Presentation skills
- Q&A preparation
- Feedback incorporation
- Final polishing

Projects built:

- FINAL CAPSTONE: Complete Project
- Options: AAA game, competitive programming platform, robotics system, ML application
- Full documentation
- Deployment and release

Deliverables:

- Completed project
- Source code
- Documentation
- Demo video

Career Launch

Topics:

- Job search strategies
- Internship applications
- College applications with portfolio
- Freelancing opportunities
- Resume optimization
- LinkedIn profile
- Cover letters
- Follow-up strategies
- First job preparation
- Continuous learning
- Career planning
- Goal setting

Projects built:

- Professional resume
- Cover letter templates
- Application tracker
- Career plan document

Deliverables:

- Job-ready portfolio
- Professional resume
- LinkedIn profile
- Application materials

Graduation & Celebration

Topics:

- Course completion
- Final presentations
- Peer showcases
- Industry feedback
- Certification ceremony
- Alumni network
- Future resources
- Mentorship continuation
- Community involvement
- Success stories
- Next steps
- Lifelong learning

Deliverables:

- Course Completion Certificate
- Professional C++ Developer Certification
- Competitive Programming Certificate
- Letter of Recommendation
- Alumni Network Access
- Lifetime Course Updates

Assessment: FINAL SHOWCASE: Present portfolio to industry professionals and compete in final programming contest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 20+ console programs, text games, file utilities
- Phase 2: 15+ OOP projects, data structures, algorithms
- Phase 3: 10+ games, graphics applications, system tools
- Phase 4: Competition solutions, open source contributions, capstone
- Total: 50+ projects from beginner to advanced

Total projects: 50+ C++ projects including games, tools, and applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Languages: C++11/14/17/20, STL mastery
- Programming: OOP, generic programming, functional programming
- Data Structures: All fundamental and advanced structures
- Algorithms: Sorting, searching, graphs, DP, geometry
- Game Dev: 2D/3D graphics, physics, AI, networking
- Systems: Memory management, multithreading, networking
- Graphics: SFML, OpenGL, shaders, game engines
- Competitive: Problem solving, optimization, mathematics
- Tools: Git, debuggers, profilers, build systems
- Professional: Documentation, testing, collaboration

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-4 hours

Hands on coding: 5-6 hours
Projects: 3-4 hours
Practice problems: 2-3 hours
Total per week: 10-12 hours

SUPPORT INCLUDED

Live sessions: Weekly doubt clearing with expert mentors
Mentorship: 1-on-1 guidance from competitive programmers
Community: Active Discord with teen C++ developers
Code review: Professional code reviews
Contest support: Competitive programming coaching
Lifetime access: All content and updates
Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each quarter
Final certificate: Professional C++ Developer Certification
Competitive cert: Competitive Programming Certification
Game dev cert: Game Development Certification
Linkedin badge: Verified LinkedIn credentials
Portfolio review: Industry-validated portfolio

WHERE THIS CAN LEAD

- Game Developer (Unity, Unreal, Custom Engines)
- Systems Programmer
- Competitive Programmer
- C++ Software Engineer
- Graphics Programmer
- Embedded Systems Developer
- Robotics Engineer
- Quantitative Developer
- Research Software Engineer
- Technical Founder

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Why should teenagers learn C++ when Python seems easier?

A: C++ teaches you how computers actually work - memory management, performance optimization, and low-level programming. This deep understanding makes you a better programmer in ANY language. Plus, C++ is essential for game development, competitive programming, robotics, and high-frequency trading - fields with the highest salaries.

Q: Is C++ too difficult for a 13-16 year old beginner?

A: Not at all! Our course is specifically designed for teens starting from zero. We use games, visual projects, and fun challenges to teach complex concepts. Many students younger than 13 have completed this course successfully. The key is patience and practice, which we build gradually.

Q: What can I build with C++ after completing this 12-month course?

A: You'll build 50+ projects including: 2D/3D games with graphics, competitive programming solutions, system utilities, chat applications, game engines, Arduino robotics projects, AI for games, and networking applications. Your portfolio will impress colleges and employers alike.

Q: Will this course help me in competitive programming and USACO?

A: Yes! Phase 4 is dedicated to competitive programming: Dynamic Programming, Graph Algorithms, Advanced Data Structures, and USACO-style problems. Students who complete this course consistently qualify for national competitions and have a strong foundation for USACO Bronze, Silver, and Gold divisions.

Q: How does C++ help with college admissions and scholarships?

A: C++ proficiency demonstrates serious technical ability. Your GitHub portfolio of games and projects, competitive programming rankings, and potential USACO medals become powerful differentiators for college applications. Many students have received scholarships and early admissions based on their C++ projects.

Q: What equipment do I need for this C++ programming course?

A: Any computer (Windows, Mac, or Linux) with internet connection works perfectly. For the game development and graphics modules, we recommend at least 4GB RAM.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/cpp-programming-masterclass-for-teens>

=====

COURSE 28 OF 118: C++ PROGRAMMING MASTERCLASS: ZERO TO SYSTEM PROGRAMMER

=====

Course link: <https://learn.modernagecoders.com/courses/cpp-programming-complete-masterclass-college>
Subject area: Systems & Performance Programming
Designed for: College students
Who can join: Complete Beginner to Expert C++ Developer
Course length: 12 months (52 weeks)
Time needed: 15-20 hours/week recommended
Certificate: Certified C++ Professional Developer upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 12-month C++ programming course: master modern C++ (11 to 23), STL, memory management, system programming and high-performance, competitive-ready code.

In one line: From Hello World to High-Performance System Programming

ABOUT THE PROGRAMME

This is not just a course, it's a complete transformation into a professional C++ developer. C++ powers the world's most critical systems: operating systems, game engines, embedded devices, trading systems, and browsers. This 12-month masterclass takes you from absolute beginner to expert C++ developer, capable of building high-performance applications, system-level software, game engines, and solving complex algorithmic problems. You'll master modern C++ standards, STL, memory management, multithreading, and specialized domains like game development and competitive programming.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no programming experience needed
- Complete 12-month structured curriculum with modern C++ (up to C++23)
- Covers entire C++ ecosystem: STL, Boost, system APIs, graphics
- 40+ real-world projects including games, system tools, and algorithms
- Deep dive into memory management and performance optimization
- Competitive programming and interview preparation included
- Game development with C++ and graphics programming
- System programming and embedded development
- Lifetime access with updates for new C++ standards
- Build portfolio of production-quality C++ applications

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): C++ Basics, OOP, Pointers, Memory Management
Phase 2: Intermediate (Months 4-6): STL, Templates, Modern C++, Data Structures
Phase 3: Advanced (Months 7-9): System Programming, Multithreading, Networking
Phase 4: Specialization (Months 10-12): Game Dev, Competitive Programming, Career Launch

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required
Coding experience: None - we start from absolute basics
Equipment: Computer with 4GB+ RAM, any OS (Windows/Linux/Mac)
Time commitment: 15-20 hours per week consistently
English: Basic reading and comprehension
Motivation: Interest in systems programming and performance

WHO THIS COURSE IS FOR

Students: CS students wanting to master systems programming
Working professionals: Developers wanting to learn C++ for performance-critical applications
Entrepreneurs: Building high-performance products or game engines
Freelancers: Offering C++ development services
Kids: Advanced teens (16+) interested in game development
Anyone: Anyone interested in understanding how computers really work

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: C++ FUNDAMENTALS & OBJECT-ORIENTED PROGRAMMING (MONTHS 1-3, WEEKS 1-13)

Build rock-solid foundations in C++ syntax, memory management, pointers, and object-oriented

programming principles.

Months 1-2: C++ Basics & Core Concepts (Week 1-8)

Introduction to C++ & Development Environment

Topics:

- History and evolution of C++ (C++98 to C++23)
- Why C++? Performance, control, and applications
- Setting up development environment (VS Code, Visual Studio, CLion)
- Compilers: GCC, Clang, MSVC
- Writing your first C++ program
- Program structure: headers, main function, return
- Compilation process: preprocessing, compilation, linking
- Basic input/output with iostream
- Comments and code documentation
- Namespaces and using directive
- C++ coding standards and best practices
- Debugging basics with GDB/LLDB

Projects built:

- Hello World variations
- Simple calculator with basic operations
- Temperature converter
- Personal information display program

Practice: Write 20 simple console programs

Variables, Data Types & Operators

Topics:

- Fundamental data types: int, float, double, char, bool
- Type modifiers: signed, unsigned, short, long
- Variable declaration and initialization
- Constants: const and constexpr
- Auto keyword and type inference
- Arithmetic operators and precedence
- Relational and logical operators
- Bitwise operators and bit manipulation
- Assignment and compound assignment operators
- Increment and decrement operators
- Type casting: static_cast, dynamic_cast, const_cast, reinterpret_cast
- sizeof operator and type sizes

Projects built:

- Advanced calculator with bitwise operations
- Data type size explorer
- Number system converter (binary, decimal, hex)
- Bitwise flag management system

Practice: Solve 50 problems on operators and type manipulation

Control Flow & Functions

Topics:

- If-else statements and nested conditions
- Switch-case statements
- Conditional (ternary) operator
- While, do-while loops
- For loops and range-based for (C++11)
- Break, continue, and goto
- Function declaration and definition
- Function parameters: pass by value, reference, pointer
- Return types and return by reference
- Function overloading
- Default arguments
- Inline functions
- Recursion and recursive algorithms
- Function pointers basics

Projects built:

- Menu-driven banking system
- Number guessing game
- Prime number generator
- Recursive factorial and Fibonacci calculator
- Pattern printing programs

Practice: Implement 30 algorithms using functions

Arrays, Strings & Pointers

Topics:

- Static arrays: declaration and initialization
- Array operations and bounds checking
- Multidimensional arrays
- C-style strings and string operations
- std::string class and string manipulation
- String streams (stringstream)
- Introduction to pointers
- Pointer arithmetic
- Arrays and pointers relationship
- Dynamic memory allocation: new and delete
- Memory leaks and dangling pointers

- Smart pointers introduction (unique_ptr, shared_ptr)
- References vs pointers
- Null pointers and nullptr

Projects built:

- Dynamic array implementation
- String manipulation library
- Matrix operations calculator
- Memory management demonstration
- Text-based adventure game

Practice: Master 40 pointer and array problems

Month 3: Object-Oriented Programming (Week 9-13)

Classes and Objects

Topics:

- Classes and objects fundamentals
- Class members: data and functions
- Access specifiers: public, private, protected
- Constructors and destructors
- Constructor types: default, parameterized, copy
- Initializer lists
- This pointer
- Static members and methods
- Friend functions and classes
- Nested classes
- Const member functions
- Mutable keyword
- Rule of Three/Five/Zero
- Move semantics (C++11)

Projects built:

- Bank account management system
- Student record system
- Complex number class
- Date and time class
- Custom string class implementation

Practice: Create 20 different classes modeling real-world objects

Inheritance and Polymorphism

Topics:

- Inheritance concepts and types
- Single, multiple, multilevel inheritance
- Virtual inheritance and diamond problem
- Function overriding
- Virtual functions and vtable
- Pure virtual functions and abstract classes
- Interfaces in C++
- Runtime polymorphism
- Dynamic binding
- Virtual destructors
- Object slicing problem
- RTTI (Run-Time Type Information)
- typeid and type_info
- Final and override specifiers (C++11)

Projects built:

- Shape hierarchy with area calculations
- Employee management system with inheritance
- Game character class hierarchy
- Vehicle simulation system
- Plugin architecture using polymorphism

Practice: Build 15 inheritance hierarchies

Operator Overloading & Advanced OOP

Topics:

- Operator overloading concepts
- Overloading arithmetic operators
- Overloading comparison operators
- Overloading assignment operator
- Overloading stream operators (<<, >>)
- Overloading function call operator
- Overloading subscript operator []
- Type conversion operators
- Explicit keyword
- Copy and swap idiom
- CRTP (Curiously Recurring Template Pattern)
- Design patterns introduction
- SOLID principles in C++

Projects built:

- Mathematical vector class with operators
- Custom array class with subscript operator
- Fraction class with all operators
- Matrix class with operations
- PHASE 1 CAPSTONE: Complete OOP Library Management System

PHASE 2: STL, TEMPLATES & MODERN C++ (MONTHS 4-6, WEEKS 14-26)

Master the Standard Template Library, template programming, and modern C++ features.

Months 4-5: STL Mastery (Week 14-22)

STL Containers

Topics:

- STL overview and architecture
- Sequence containers: vector, deque, list, array
- Container adapters: stack, queue, priority_queue
- Associative containers: set, map, multiset, multimap
- Unordered containers: unordered_set, unordered_map
- Container operations and time complexity
- Choosing the right container
- Custom allocators
- Container memory management
- Iterators: types and usage
- Iterator categories and traits
- Reverse iterators
- Iterator adapters
- Range-based operations

Projects built:

- Student database using STL containers
- Text editor with undo/redo using stack
- Priority task scheduler
- Dictionary implementation with map
- Cache implementation with unordered_map

Practice: Implement 30 algorithms using different STL containers

STL Algorithms

Topics:

- Algorithm categories in STL
- Non-modifying sequence operations
- Modifying sequence operations
- Sorting algorithms: sort, partial_sort, nth_element
- Searching algorithms: find, binary_search, lower_bound
- Numeric algorithms: accumulate, inner_product
- Set operations: union, intersection, difference
- Heap operations
- Permutation algorithms
- Lambda expressions with algorithms
- Function objects (functors)
- Predicate functions
- Algorithm complexity analysis
- Custom algorithms implementation

Projects built:

- Data analysis tool using STL algorithms
- Sorting visualizer
- Set operations calculator
- Performance comparison of algorithms
- Custom algorithm library

Practice: Use 50 different STL algorithms in various scenarios

Template Programming

Topics:

- Function templates
- Class templates
- Template parameters: type and non-type
- Template specialization
- Partial specialization
- Variadic templates (C++11)
- Template metaprogramming basics
- SFINAE (Substitution Failure Is Not An Error)
- Type traits and type manipulation
- Template template parameters
- Concepts (C++20)
- Fold expressions (C++17)
- constexpr and compile-time computation
- Template best practices

Projects built:

- Generic data structures library
- Type-safe unit conversion system
- Compile-time math library
- Template-based design patterns
- Custom smart pointer implementation

Practice: Create 20 template-based utilities

Modern C++ Features (C++11/14/17/20)

Topics:

- Auto and decltype
- Range-based for loops
- Lambda expressions and captures
- Rvalue references and move semantics
- Perfect forwarding
- Smart pointers: unique_ptr, shared_ptr, weak_ptr
- std::thread and basic threading
- std::chrono for time management
- Random number generation
- Regular expressions
- Filesystem library (C++17)
- std::optional, std::variant, std::any (C++17)
- Structured bindings (C++17)
- Coroutines basics (C++20)

Projects built:

- Modern C++ file manager
- Multi-threaded downloader
- Regular expression validator
- Time measurement utilities
- Resource management system with smart pointers

Practice: Refactor old code using modern C++ features

Error Handling & Exception Safety

Topics:

- Exception handling mechanism
- Try, catch, and throw
- Exception specifications
- Standard exception classes
- Custom exception classes
- Exception safety guarantees
- RAII (Resource Acquisition Is Initialization)
- noexcept specifier
- Error codes vs exceptions
- std::expected (C++23)
- Stack unwinding
- Exception handling best practices
- Debugging techniques
- Assertion and static_assert

Projects built:

- Robust file processing system
- Exception-safe container
- Error handling framework
- Resource manager with RAII
- Custom error reporting system

Practice: Add proper error handling to 10 previous projects

Month 6: Data Structures & Algorithms (Week 23-26)

Advanced Data Structures

Topics:

- Linked lists: singly, doubly, circular
- Stack and queue implementations
- Binary trees and BST
- AVL trees and balancing
- Red-black trees
- B-trees and B+ trees
- Heaps and priority queues
- Hash tables and collision resolution
- Graphs: representation and traversal
- Tries and suffix trees
- Disjoint set (Union-Find)
- Segment trees and Fenwick trees
- Skip lists
- Custom allocators for data structures

Projects built:

- Custom STL-like container library
- Database index implementation
- Autocomplete system using trie
- Graph visualization tool
- Memory-efficient data structures

Practice: Implement 20 data structures from scratch

Algorithm Design & Analysis

Topics:

- Algorithm complexity analysis
- Divide and conquer algorithms
- Dynamic programming techniques
- Greedy algorithms
- Backtracking algorithms
- Graph algorithms: DFS, BFS, Dijkstra, Floyd-Warshall
- Minimum spanning tree: Kruskal, Prim
- Network flow algorithms

- String algorithms: KMP, Rabin-Karp
- Computational geometry basics
- Randomized algorithms
- Parallel algorithms introduction
- Cache-efficient algorithms
- Algorithm optimization techniques

Projects built:

- Pathfinding visualizer
- String matching tool
- Optimization problem solver
- Algorithm complexity analyzer
- Competitive programming library

Practice: Solve 100 algorithmic problems

Performance Optimization

Topics:

- Profiling tools: gprof, Valgrind, perf
- CPU architecture and cache optimization
- Memory layout and alignment
- Branch prediction and optimization
- Loop optimization techniques
- Vectorization and SIMD instructions
- Compiler optimizations and flags
- Link-time optimization
- Profile-guided optimization
- Memory pooling and custom allocators
- Lock-free data structures basics
- Benchmarking techniques
- Space-time tradeoffs
- Platform-specific optimizations

Projects built:

- Performance profiler tool
- Optimized matrix multiplication
- Memory pool allocator
- Cache-friendly data structures
- Benchmark suite for algorithms

Practice: Optimize 10 programs for maximum performance

Competitive Programming

Topics:

- Competitive programming platforms
- Problem-solving strategies
- Fast I/O techniques
- Common problem patterns
- Number theory for CP
- Combinatorics and probability
- Game theory basics
- Bit manipulation tricks
- String algorithms for CP
- Advanced data structures for CP
- Dynamic programming patterns
- Graph algorithm variations
- Contest strategies
- Time management in contests

Projects built:

- Competitive programming template
- Problem classifier and trainer
- Contest performance analyzer
- Algorithm reference implementation
- Virtual contest environment

Practice: Solve 200 competitive programming problems

Phase 2 Capstone Project

Topics:

- Complex system design
- Algorithm selection and implementation
- Performance optimization
- Code organization and documentation
- Testing and debugging

Projects built:

- PHASE 2 CAPSTONE: High-Performance Database Engine
- Build a simple database with indexing, querying, and transactions
- Implement B+ tree indexes, query optimizer, and transaction manager
- Focus on performance and correctness

Assessment: Phase 2 comprehensive exam on STL, templates, and algorithms

PHASE 3: SYSTEM PROGRAMMING & CONCURRENCY (MONTHS 7-9, WEEKS 27-39)

Master system-level programming, multithreading, networking, and low-level development.

Months 7-8: System Programming (Week 27-35)

Operating System Interface

Topics:

- Process and memory model
- System calls and OS APIs
- File system operations
- Process creation and management
- Inter-process communication (IPC)
- Shared memory
- Message queues
- Pipes and named pipes
- Signals and signal handling
- Memory mapping with mmap
- Dynamic linking and loading
- Environment variables
- System information retrieval
- Platform-specific APIs (POSIX, Win32)

Projects built:

- Custom shell implementation
- Process monitor and manager
- IPC messaging system
- File system utilities
- System information tool

Practice: Build 10 system-level utilities

Memory Management Deep Dive

Topics:

- Virtual memory concepts
- Memory allocation strategies
- Custom memory allocators
- Memory pools and arenas
- Garbage collection techniques
- Reference counting implementation
- Memory debugging tools
- Leak detection and prevention
- Memory-mapped I/O
- Cache-conscious programming
- NUMA awareness
- Memory barriers and ordering
- Lock-free memory management
- Memory optimization patterns

Projects built:

- Custom malloc implementation
- Garbage collector prototype
- Memory leak detector
- Memory-efficient container library
- Cache performance analyzer

Practice: Implement 5 different memory allocation strategies

Multithreading & Concurrency

Topics:

- Thread creation and management
- Thread synchronization primitives
- Mutexes and locks
- Condition variables
- Semaphores and barriers
- Read-write locks
- Atomic operations and memory ordering
- Lock-free programming
- Thread pools and executors
- Futures and promises
- Async programming with std::async
- Parallel algorithms (C++17)
- Thread-local storage
- Deadlock prevention and detection

Projects built:

- Thread pool implementation
- Producer-consumer system
- Parallel sorting algorithm
- Lock-free queue
- Concurrent web server

Practice: Build 15 multithreaded applications

Network Programming

Topics:

- Socket programming basics
- TCP/IP protocol suite
- UDP vs TCP programming
- Client-server architecture
- Non-blocking I/O
- I/O multiplexing: select, poll, epoll
- Asynchronous I/O
- Protocol design and implementation

- Serialization and deserialization
- RPC (Remote Procedure Call)
- HTTP server implementation
- WebSocket programming
- Network security basics
- Cross-platform networking

Projects built:

- Chat server and client
- HTTP server from scratch
- File transfer protocol implementation
- Distributed computing framework
- Network packet analyzer

Practice: Create 10 network applications

Database Systems Programming

Topics:

- Database connectivity (ODBC, native APIs)
- SQL query execution from C++
- Connection pooling
- Transaction management
- Prepared statements
- ORM implementation basics
- NoSQL database interfaces
- Embedded databases (SQLite)
- Database driver development
- Query optimization from application
- Caching strategies
- Data serialization formats
- Database performance tuning
- Migration and versioning

Projects built:

- Database connection pool
- Simple ORM framework
- Database migration tool
- Query builder library
- Cache layer implementation

Practice: Build 5 database-driven applications

Month 9: Advanced System Topics (Week 36-39)

Compiler Design & Tools

Topics:

- Compilation process deep dive
- Lexical analysis basics
- Parsing techniques
- Abstract syntax trees
- Code generation basics
- Writing a simple interpreter
- JIT compilation concepts
- LLVM basics
- Build systems: Make, CMake, Bazel
- Static analysis tools
- Code generation tools
- Debugging at assembly level
- Profiling and optimization tools
- Cross-compilation

Projects built:

- Simple expression evaluator
- Mini programming language interpreter
- Build system for C++ projects
- Static code analyzer
- Code generator for repetitive tasks

Practice: Create 5 development tools

Embedded Systems Programming

Topics:

- Embedded C++ considerations
- Memory-constrained programming
- Real-time systems concepts
- Interrupt handling
- Device driver basics
- Bare-metal programming
- RTOS integration
- Hardware abstraction layers
- Cross-compilation for embedded
- Debugging embedded systems
- Power optimization
- Communication protocols (I2C, SPI, UART)
- Sensor integration
- IoT development basics

Projects built:

- Embedded system simulator

- Device driver prototype
- Real-time task scheduler
- Sensor data logger
- IoT device firmware

Practice: Develop 5 embedded applications

Security Programming

Topics:

- Secure coding practices
- Buffer overflow prevention
- Input validation techniques
- Cryptography basics
- Hash functions implementation
- Encryption algorithms
- SSL/TLS implementation basics
- Authentication systems
- Secure random number generation
- Code obfuscation techniques
- Reverse engineering prevention
- Security auditing tools
- Fuzzing and testing
- Exploit mitigation techniques

Projects built:

- Encryption library
- Secure communication protocol
- Password manager
- Security scanner
- Secure file storage system

Practice: Implement 10 security features

Graphics Programming Basics

Topics:

- Graphics pipeline overview
- OpenGL/DirectX basics
- 2D graphics rendering
- 3D mathematics for graphics
- Shaders and GLSL
- Texture mapping
- Lighting models
- Camera systems
- Scene graphs
- Collision detection
- Particle systems
- GUI programming with Qt/ImGui
- Vulkan introduction
- Graphics optimization

Projects built:

- 2D graphics renderer
- 3D scene viewer
- Particle system simulator
- Simple GUI framework
- Graphics effects library

Practice: Create 5 graphics applications

Phase 3 Capstone Project

Topics:

- System design and architecture
- Component integration
- Performance requirements
- Testing and validation
- Documentation and deployment

Projects built:

- PHASE 3 CAPSTONE: Distributed Computing System
- Build a distributed task processing system
- Include networking, multithreading, and IPC
- Focus on scalability and fault tolerance

Assessment: Phase 3 comprehensive exam on system programming

PHASE 4: SPECIALIZATION & CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52)

Choose your specialization path and prepare for professional C++ development careers.

Months 10-11: Specialization Tracks (Week 40-48)

Game Development with C++

Topics:

- Game architecture patterns
- Game loops and timing
- Entity-Component-System (ECS)
- Physics engine basics
- Collision detection algorithms
- Game AI fundamentals

- Pathfinding algorithms
- State machines for games
- Asset management
- Memory management for games
- Game networking basics
- Graphics optimization for games
- Audio programming
- Popular game engines (Unreal, custom)

Projects built:

- 2D game engine from scratch
- Physics simulation
- AI for game NPCs
- Multiplayer game prototype
- Game performance profiler

Practice: Build 3 complete games

High-Performance Computing

Topics:

- Parallel computing concepts
- OpenMP programming
- MPI for distributed computing
- CUDA/GPU programming basics
- Vectorization with SIMD
- Cache optimization techniques
- NUMA optimization
- High-performance libraries
- Scientific computing with C++
- Numerical methods implementation
- Linear algebra libraries
- FFT implementation
- Monte Carlo simulations
- Performance analysis tools

Projects built:

- Parallel matrix operations
- Scientific computing library
- GPU-accelerated algorithm
- Distributed computing application
- Performance benchmark suite

Practice: Optimize 10 computationally intensive algorithms

Financial Systems Development

Topics:

- Low-latency programming
- Lock-free data structures for trading
- Order matching engines
- Market data processing
- Risk calculation systems
- Backtesting frameworks
- FIX protocol implementation
- High-frequency trading concepts
- Time-series databases
- Quantitative libraries
- Decimal arithmetic
- Regulatory compliance coding
- Testing financial systems
- Production deployment for finance

Projects built:

- Order matching engine
- Market data feed handler
- Risk calculator
- Backtesting framework
- Trading system simulator

Practice: Build 5 financial applications

Modern C++ Best Practices

Topics:

- C++20/23 features deep dive
- Modules system
- Coroutines advanced usage
- Ranges library
- Concepts and constraints
- Three-way comparison
- Design patterns in modern C++
- Dependency injection
- Event-driven architectures
- Microservices with C++
- REST API development
- gRPC services
- Container orchestration
- DevOps for C++ projects

Projects built:

- Modern C++ web service

- Microservice architecture
- Event-driven system
- Design patterns library
- C++20/23 feature showcase

Practice: Refactor projects using latest C++ features

Open Source Contribution

Topics:

- Finding C++ open source projects
- Understanding large codebases
- Code review process
- Writing quality patches
- Documentation contributions
- Testing and CI/CD
- Community interaction
- Starting your own project
- License selection
- Project maintenance
- Building community
- Release management
- Security practices
- Long-term sustainability

Projects built:

- Contribute to major C++ project
- Create and publish C++ library
- Write comprehensive documentation
- Set up CI/CD pipeline
- Build project community

Practice: Make 10 open source contributions

Month 12: Career Preparation & Launch (Week 49-52)

Interview Preparation

Topics:

- C++ interview questions
- Memory management questions
- OOP concepts review
- STL and algorithms questions
- System design for C++
- Coding challenges
- Debugging exercises
- Code review scenarios
- Performance optimization questions
- Modern C++ topics
- Design patterns questions
- Behavioral questions
- Whiteboard coding
- Take-home assignments

Projects built:

- Interview question solver
- Coding challenge solutions
- System design portfolio
- Code review examples
- Performance case studies

Practice: Solve 200 interview problems

Portfolio Development

Topics:

- Selecting best projects
- Code cleanup and documentation
- README writing
- API documentation
- Performance metrics
- Test coverage
- CI/CD setup
- GitHub profile optimization
- Personal website creation
- Technical blog writing
- Video demonstrations
- Case studies
- Architecture diagrams
- Deployment guides

Projects built:

- Portfolio website
- Project showcases
- Technical blog posts
- Video tutorials
- Architecture documentation

Practice: Polish and document all major projects

Career Paths & Specializations

Topics:

- C++ career opportunities

- Game development careers
- Systems programming roles
- Embedded systems positions
- Financial technology jobs
- High-performance computing
- Graphics programming
- Compiler development
- Security engineering
- Research positions
- Freelancing with C++
- Consulting opportunities
- Startup considerations
- Remote work strategies

Projects built:

- Career roadmap
- Specialization project
- Industry research
- Network building
- Job application materials

Practice: Apply to 20 relevant positions

Professional Skills

Topics:

- Code review best practices
- Team collaboration
- Agile methodologies
- Version control mastery
- Documentation standards
- Testing strategies
- Debugging techniques
- Performance profiling
- Project estimation
- Technical leadership
- Mentoring others
- Continuous learning
- Conference participation
- Professional networking

Projects built:

- Team project simulation
- Code review portfolio
- Testing framework
- Documentation templates
- Mentorship materials

Practice: Practice professional skills daily

Final Project & Graduation

Topics:

- Project planning and scoping
- Architecture design
- Implementation
- Testing and optimization
- Documentation
- Presentation preparation
- Peer review
- Deployment
- Project showcase
- Career launch planning
- Alumni network
- Continuous improvement

Projects built:

- FINAL CAPSTONE: Production-Grade C++ Application
- Choose from: Game Engine, Trading System, Embedded OS, Distributed Database
- Must demonstrate mastery of C++, system design, and specialization
- Include full documentation, tests, and deployment

Assessment: Final comprehensive examination and project defense

Continuous Learning & Excellence (Ongoing)

Staying Current with C++

Topics:

- Following C++ standards committee
- Reading committee papers
- Testing new features
- Compiler updates
- Library developments
- Community involvement
- Conference attendance
- CppCon and other events
- Online resources
- Books and publications
- Podcast participation
- Forum contributions

- Stack Overflow activity
- Code review participation

Projects built:

- Experimental features testing
- Standards proposal review
- Community contribution
- Learning resource creation
- Knowledge sharing

Practice: Dedicate 5 hours/week to C++ advancement

Advanced Certifications

Topics:

- C++ certification programs
- Platform certifications
- Game engine certifications
- Embedded systems credentials
- Security certifications
- Cloud platform certs
- Architecture certifications
- Project management
- Teaching qualifications
- Industry-specific training
- Vendor certifications
- Professional memberships
- Academic pursuits
- Research opportunities

Projects built:

- Certification preparation
- Study plan creation
- Practice examinations
- Certification project
- Professional development plan

Practice: Pursue relevant certifications

Building C++ Products

Topics:

- Product ideation
- Market research
- MVP development
- Performance requirements
- Scalability planning
- Security considerations
- Monetization strategies
- Distribution methods
- License models
- Support strategies
- Documentation needs
- Community building
- Marketing approaches
- Business development

Projects built:

- Product prototype
- Business plan
- MVP release
- User feedback integration
- Product iteration

Practice: Develop and launch C++ product

Mastery & Leadership

Topics:

- Technical leadership
- Architecture decisions
- Team mentoring
- Code quality standards
- Best practices enforcement
- Performance culture
- Innovation fostering
- Knowledge transfer
- Process improvement
- Tool selection
- Technology evaluation
- Strategic planning
- Industry influence
- Thought leadership

Projects built:

- Leadership portfolio
- Mentorship program
- Team guidelines
- Architecture decisions record
- Innovation proposals

Practice: Exercise leadership in C++ community

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 20+ foundational projects - OOP systems, memory managers, data structures
- Phase 2: 25+ intermediate projects - STL applications, template libraries, algorithms
- Phase 3: 20+ system projects - OS tools, network servers, concurrent applications
- Phase 4: 15+ specialized projects - games, HPC applications, financial systems
- Total: 80+ projects from console apps to production systems

Total projects: 80+ C++ projects across all domains

SKILLS THE STUDENT WILL HAVE AT THE END

- Core C++: Syntax, OOP, inheritance, polymorphism, operator overloading
- Memory Management: Pointers, RAII, smart pointers, custom allocators
- STL: All containers, algorithms, iterators, function objects
- Templates: Function/class templates, metaprogramming, SFINAE, concepts
- Modern C++: C++11/14/17/20/23 features, move semantics, lambdas
- System Programming: OS interfaces, IPC, networking, multithreading
- Performance: Optimization, profiling, cache efficiency, SIMD
- Data Structures: Trees, graphs, hash tables, custom implementations
- Algorithms: Sorting, searching, dynamic programming, graph algorithms
- Concurrency: Threads, locks, atomics, lock-free programming
- Tools: GCC, Clang, CMake, GDB, Valgrind, profilers
- Specializations: Game dev, HPC, embedded, financial systems

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
Hands on coding: 8-10 hours
Projects: 3-5 hours
Practice problems: 2-3 hours
Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving and code review sessions
Mentorship: 1-on-1 guidance from senior C++ developers
Community: Active Discord community of C++ enthusiasts
Code review: Expert code reviews for all major projects
Career support: Resume review, interview prep, job referrals
Lifetime access: All content and future C++ standard updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion
Final certificate: Certified C++ Professional Developer
Linkedin badge: Verified LinkedIn credential
Industry recognized: Recognized by tech companies globally
Portfolio projects: 80+ documented projects
Specialization certificate: Certificate in chosen specialization area

WHERE THIS CAN LEAD

- C++ Software Developer
- Systems Programmer
- Game Developer
- Embedded Systems Engineer
- Graphics Programmer
- Quantitative Developer (Finance)
- High-Performance Computing Engineer
- Compiler Developer
- Security Engineer
- Real-Time Systems Developer
- Game Engine Developer
- Research Engineer
- Technical Lead

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Growing roles: Rs 5-8 LPA (Junior C++ Developer)

Experienced roles: Rs 8-15 LPA (C++ Developer); Rs 12-25 LPA (Senior C++ Developer); Rs

20-40+ LPA (Lead Developer/Architect)

Freelance: Rs 2,000-8,000/hour based on expertise

International: \$80k-180k USD based on location and specialization

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Why should I learn C++ programming when there are easier languages like Python?

A: C++ offers unparalleled control over system resources and delivers the fastest execution speed of any mainstream language. It's essential for game development, operating systems, embedded systems, high-frequency trading, and performance-critical applications.

Q: Is C++ too difficult for beginners? Can I learn it as my first language?

A: While C++ is considered more challenging than Python or JavaScript, our 12-month masterclass starts from absolute basics - assuming zero prior experience. We teach programming logic, memory management, and object-oriented programming step-by-step. Many successful C++ developers started here as their first language, gaining deeper computer science knowledge than those who started with higher-level languages.

Q: What career paths are available after mastering C++ programming?

A: C++ opens doors to specialized, high-paying roles: Game Developer (Unreal Engine), Systems Programmer, Embedded Systems Engineer, Quantitative Developer in Finance, Graphics Programmer, Compiler Developer, and High-Performance Computing Engineer. These roles typically pay 25-50% more than general web development positions due to the specialized skill requirements.

Q: What makes this C++ course different from free YouTube tutorials?

A: This 12-month program covers advanced topics rarely found online: modern C++20/23 features, template metaprogramming, lock-free programming, memory optimization, and system-level programming. You'll build 80+ projects including a database engine, game engine, and trading system. Plus, you get 1-on-1 mentorship, code reviews, and career support that YouTube can't provide.

Q: How long does it take to become proficient in C++ programming?

A: Basic proficiency (junior developer level) takes 6 months of dedicated learning at 15-20 hours/week. True mastery of advanced concepts like templates, concurrency, and system programming takes 12-18 months. This course is designed to take you from zero to production-level skills in 12 months, with specialization options in game development, HPC, or financial systems.

Q: Is C++ still relevant in 2026? Isn't it an outdated language?

A: C++ is more relevant than ever! It's actively developed with new standards (C++20, C++23) adding powerful features like concepts, coroutines, and modules. It powers game engines (Unreal Engine 5), operating systems, browsers (Chrome, Firefox), databases (MongoDB, MySQL), and AI frameworks (TensorFlow, PyTorch). Every major tech company - Google, Microsoft, Apple, Meta - heavily invests in C++ development.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/cpp-programming-complete-masterclass-college>

COURSE 29 OF 118: CANVA & AI DESIGN COURSE: ZERO TO PROFESSIONAL DESIGNER

Course link: <https://learn.modernagecoders.com/courses/canva-ai-design-masterclass>

Subject area: Professional Design & Content Creation

Designed for: Kids

Who can join: Complete Beginner to Advanced Professional

Course length: 6 months (26 weeks)

Time needed: 10-15 hours/week recommended

Certificate: Industry-recognized Digital Design Certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 6-month design course to master Canva, photo and video editing and AI tools, creating professional social media content and marketing materials from zero.

In one line: From 'Zero Design Skills' to Professional Digital Designer in 6 Months

ABOUT THE PROGRAMME

This comprehensive program transforms complete beginners into professional digital designers. You'll master Canva Pro, learn professional photo and video editing, harness the power of AI design tools like Midjourney and DALL-E, and create stunning visual content that converts. Whether you're a business owner, marketer, content creator, or aspiring designer, this program gives you all the skills needed to create professional-grade designs, manage social

media visuals, build brands, and even start a design freelance business. By the end, you'll have created 50+ professional projects and be ready for real client work.

WHAT MAKES THIS COURSE DIFFERENT

- Zero experience required - starts from absolute basics
- Focuses on practical, real-world design projects
- Integrates latest AI tools throughout the course
- Covers both free and pro tools comprehensively
- Business and freelancing modules included
- Personal branding and portfolio development
- Lifetime access to all updates and new AI tools
- Ready-to-use templates and resources library

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-2): Design Principles, Canva Basics, Typography, Color Theory
Phase 2: Intermediate (Months 3-4): Advanced Canva, Photo Editing, AI Tools, Brand Design
Phase 3: Advanced (Months 5-6): Video Editing, Animation, AI Mastery, Professional Projects
Phase 4: Professional (Final Weeks): Portfolio, Freelancing, Business Setup, Client Work

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal education required - open to everyone
Coding experience: No technical background needed
Equipment: Computer with internet (Canva works in browser)
Time commitment: 10-15 hours per week
English: Basic English understanding
Motivation: Passion for creativity and visual communication

WHO THIS COURSE IS FOR

Students: Students wanting creative career options
Working professionals: Professionals adding design skills to their toolkit
Entrepreneurs: Business owners who want to DIY their marketing
Freelancers: People wanting to start design freelance business
Kids: Creative teens (14+) interested in design
Anyone: Anyone wanting to express creativity professionally

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: DESIGN FOUNDATION & CANVA MASTERY (MONTHS 1-2, WEEKS 1-8)

Build strong design fundamentals while mastering Canva. Learn design principles, typography, color theory, and create your first professional designs.

Months 1-2: Design Fundamentals & Canva Essentials (Week 1-8)

Introduction to Design & Canva Setup

Topics:

- What is graphic design? History and evolution
- Design careers and opportunities in 2026
- Traditional vs digital design
- Introduction to Canva: Interface tour
- Canva Free vs Pro features comparison
- Account setup and workspace organization
- Understanding design dimensions and formats
- File types: JPG, PNG, PDF, SVG explained
- Resolution and DPI for print vs digital
- Creating your first Canva design
- Navigating templates and elements
- Basic tools: Text, shapes, lines, frames

Projects built:

- Create personal introduction card
- Design simple social media post
- Make basic business card

Practice: Daily: 30 min Canva exploration, 1 hour design exercises

Design Principles & Composition

Topics:

- The 7 principles of design
- Balance: symmetrical, asymmetrical, radial
- Contrast and emphasis in design
- Repetition and consistency
- Proximity and grouping elements
- Alignment and grid systems
- Visual hierarchy and flow
- Rule of thirds and golden ratio

- White space and minimalism
- Gestalt principles in design
- Creating focal points
- Composition techniques for impact

Projects built:

- Redesign poorly designed flyer
- Create balanced poster design
- Design minimalist quote graphics
- Build visual hierarchy infographic

Practice: Analyze 20 professional designs for principles

Typography & Color Theory

Topics:

- Typography fundamentals: serif vs sans-serif
- Font families and pairing rules
- Font size hierarchy and scaling
- Line height, letter spacing, kerning
- Readability and legibility principles
- Typography for different mediums
- Color theory basics: color wheel
- Primary, secondary, tertiary colors
- Color harmonies: complementary, analogous, triadic
- Color psychology and emotions
- Creating custom color palettes
- Using Canva's color tools effectively

Projects built:

- Typography poster with font hierarchy
- Brand color palette creation
- Mood board with color themes
- Restaurant menu with proper typography

Practice: Create 10 different color palettes and font combinations

Canva Advanced Features & Templates

Topics:

- Mastering Canva Pro features
- Background remover tool mastery
- Magic Resize for multiple formats
- Brand Kit setup and management
- Creating and saving templates
- Using and customizing elements
- Working with grids and frames
- Photo effects and filters
- Transparency and layering
- Animation basics in Canva
- Collaboration features
- Organizing designs with folders

Projects built:

- Complete brand kit setup
- Create reusable template set
- Design animated social media post
- Multi-page presentation design

Practice: Build template library for different use cases

Months 1-2 Continued: Social Media & Marketing Design (Week 1-8 Deep Dive)

Social Media Design Mastery

Topics:

- Platform-specific dimensions and requirements
- Instagram: Posts, Stories, Reels covers
- Facebook: Posts, covers, ads
- LinkedIn: Posts, banners, articles
- Twitter/X: Posts, headers
- YouTube: Thumbnails, banners, end screens
- TikTok: Video covers and graphics
- Pinterest: Pins and idea pins
- Consistent visual branding across platforms
- Creating content calendars
- Batch creating social media content
- Engagement-driving design techniques

Projects built:

- 30-day social media content calendar
- Complete Instagram feed design
- YouTube channel branding package
- Multi-platform campaign graphics

Practice: Design 5 posts for each major platform

Marketing Materials & Print Design

Topics:

- Business card design principles
- Letterhead and stationery design
- Flyer and brochure layouts
- Poster design for events
- Banner and billboard considerations

- Email newsletter templates
- Digital ads: display, social, search
- Landing page graphics
- Infographic design process
- Presentation design mastery
- Print specifications and bleed
- Preparing files for print

Projects built:

- Complete business stationery set
- Tri-fold brochure design
- Event poster and flyer set
- Email marketing template

Practice: Create marketing materials for fictional company

Logo Design & Brand Identity

Topics:

- Logo design fundamentals
- Types of logos: wordmark, lettermark, emblem
- Sketching and ideation process
- Creating logos in Canva
- Logo variations and applications
- Brand identity elements
- Brand style guide creation
- Color psychology for brands
- Typography in branding
- Brand voice and visual consistency
- Mockup creation for presentations
- Protecting and trademarking designs

Projects built:

- Design 3 different logo concepts
- Complete brand identity package
- Brand style guide document
- Logo mockup presentations

Practice: Rebrand 5 existing companies

Photo Editing & Manipulation

Topics:

- Photo editing basics in Canva
- Adjusting brightness, contrast, saturation
- Color correction and grading
- Cropping and composition improvement
- Removing backgrounds professionally
- Creating photo collages
- Double exposure effects
- Photo filters and effects
- Retouching basics
- Working with RAW images
- Batch editing techniques
- Creating photo templates

Projects built:

- Professional headshot editing
- Product photo enhancement
- Creative photo manipulation art
- Before/after portfolio pieces

Practice: Edit 50 photos in different styles

Months 1-2 Assessment & Portfolio

Topics:

- Portfolio planning and organization
- Selecting best work
- Creating case studies
- Portfolio presentation techniques
- Online portfolio platforms
- Design process documentation

Projects built:

- Phase 1 Portfolio Website
- 10 best designs showcase
- Design process case study

Assessment: Complete design challenge in 3 hours

Phase 1 Projects & Specialization (Week 1-8 Practical Application)

Real Client Project Simulation

Topics:

- Client brief interpretation
- Design research and mood boards
- Concept development process
- Presenting design options
- Handling client feedback
- Revision management
- File organization and naming
- Project timeline management
- Design contracts basics

- Pricing your work
- Invoice and proposal creation
- Building client relationships

Projects built:

- Complete rebrand for local business
- Social media campaign for product launch
- Event marketing materials package
- Website graphics and assets

Practice: Complete 3 mock client projects

Industry Specialization Tracks

Topics:

- E-commerce design specialization
- Real estate marketing materials
- Restaurant and food industry design
- Fashion and beauty branding
- Tech startup design needs
- Non-profit and cause marketing
- Educational content design
- Healthcare and medical design
- Sports and fitness branding
- Entertainment industry graphics
- B2B vs B2C design differences
- Industry-specific requirements

Projects built:

- Choose specialization track
- Industry-specific portfolio pieces
- Niche market research project
- Specialized template creation

Practice: Create designs for 3 different industries

Advanced Canva Techniques

Topics:

- Custom illustrations in Canva
- Advanced animation techniques
- Creating GIFs and cinemagraphs
- Data visualization and charts
- Complex infographic layouts
- Magazine-style layouts
- Book cover design
- Packaging design concepts
- 3D mockup integration
- Advanced masking techniques
- Creative effects and overlays
- Canva Apps integration

Projects built:

- Animated infographic
- Digital magazine spread
- Product packaging concept
- Interactive PDF creation

Practice: Master 10 advanced Canva techniques

Design Systems & Consistency

Topics:

- Creating design systems
- Component-based design
- Style guide development
- Template libraries management
- Version control for designs
- Design documentation
- Accessibility in design
- Inclusive design principles
- Multi-language considerations
- Responsive design basics
- Cross-platform consistency
- Design handoff basics

Projects built:

- Complete design system
- Multi-brand template library
- Accessibility audit and fixes
- Responsive design mockups

Practice: Build reusable design system

Phase 1 Final Project

Topics:

- Comprehensive design challenge
- Time management under pressure
- Quality control checklist
- Professional presentation
- Portfolio finalization
- Career planning

Projects built:

- CAPSTONE: Complete Brand Package

- Logo, stationery, social media, website mockup, marketing materials
- Professional presentation deck
- Case study documentation

Assessment: Phase 1 Certification Exam

----- PHASE 2: ADVANCED DESIGN & AI TOOLS INTEGRATION (MONTHS 3-4, WEEKS 9-17) -----

Master advanced photo editing, video creation, and integrate cutting-edge AI design tools into your workflow.

Months 3-4: AI Design Tools & Advanced Photo Editing (Week 9-17)

Introduction to AI Design Tools

Topics:

- AI in design: Overview and possibilities
- Midjourney: Setup and basics
- Writing effective prompts for Midjourney
- DALL-E 3: Access and capabilities
- Stable Diffusion basics
- Adobe Firefly integration
- Canva AI features: Magic Write, Draw, Edit
- ChatGPT for design briefs and copy
- AI image upscaling tools
- AI background removal tools
- Ethical considerations in AI design
- Copyright and AI-generated content

Projects built:

- Create AI-generated brand imagery
- Design campaign using AI tools
- AI-assisted logo concepts
- Hybrid AI-manual design project

Practice: Generate 100 AI images with different styles

Advanced Midjourney Mastery

Topics:

- Midjourney advanced parameters
- Style references and image prompts
- Aspect ratios and composition control
- Creating consistent characters
- Photorealistic vs artistic styles
- Midjourney for product mockups
- Architectural visualization
- Fashion and textile design
- Blending and remixing images
- Upscaling and variations
- Building a prompt library
- Midjourney workflow optimization

Projects built:

- AI-generated product catalog
- Consistent character for storytelling
- Architectural concept designs
- Fashion collection mood board

Practice: Master 20 different Midjourney styles

AI-Powered Photo Editing

Topics:

- Photoshop AI features overview
- Generative Fill techniques
- Content-Aware tools mastery
- AI sky replacement
- Neural filters application
- Canva's AI photo editing tools
- Remove.bg and alternatives
- AI-powered color grading
- Face and body retouching with AI
- AI image enhancement tools
- Batch processing with AI
- Combining AI with manual editing

Projects built:

- Professional portrait retouching
- Product photo enhancement
- Landscape photo transformation
- Creative composite artwork

Practice: Edit 30 photos using AI tools

Video Editing Fundamentals

Topics:

- Video editing basics in Canva
- Timeline and sequencing
- Transitions and effects
- Adding text and titles
- Music and sound effects

- Video trimming and splitting
- Speed adjustments
- Color correction for video
- Creating video templates
- Exporting for different platforms
- Introduction to CapCut
- Mobile video editing tips

Projects built:

- Social media video ads
- Product demonstration video
- Animated logo reveal
- Story-driven video content

Practice: Create 20 short-form videos

Advanced Video & Animation

Topics:

- Motion graphics in Canva
- Kinetic typography
- Animated infographics
- Green screen editing
- Video loops and cinemagraphs
- Animated presentations
- Video podcast graphics
- Live streaming overlays
- Video thumbnails optimization
- Creating video series
- Video SEO basics
- Analytics and optimization

Projects built:

- Animated explainer video
- YouTube intro/outro
- Instagram Reels series
- Animated presentation

Practice: Create daily video content for one week

Months 3-4: Professional Video & Content Creation (Week 9-17 Continued)

Content Creation Workflow

Topics:

- Content planning and calendars
- Batch creation strategies
- Content repurposing techniques
- Cross-platform adaptation
- Creating content templates
- Automation tools and plugins
- Content management systems
- File organization for creators
- Cloud storage solutions
- Collaboration tools for teams
- Version control for creatives
- Backup strategies

Projects built:

- 3-month content calendar
- Content template library
- Automated design workflow
- Team collaboration project

Practice: Create week's worth of content in one day

AI Video Generation

Topics:

- AI video generation tools overview
- Synthesia for AI avatars
- RunwayML capabilities
- Pika Labs and Gen-2
- D-ID for talking avatars
- HeyGen platform usage
- AI voice generation
- Lip-sync technology
- AI video editing assistants
- Text-to-video creation
- AI scene generation
- Future of AI video

Projects built:

- AI spokesperson video
- Animated story with AI
- Product demo with AI avatar
- Multi-language video content

Practice: Create 10 AI-generated videos

Advanced AI Prompt Engineering

Topics:

- Prompt engineering principles
- Prompt structure and syntax

- Style modifiers mastery
- Lighting and mood prompts
- Camera angle specifications
- Art movement references
- Combining multiple concepts
- Negative prompts usage
- Seed and parameter control
- Creating prompt templates
- Prompt databases and resources
- A/B testing prompts

Projects built:

- Prompt recipe book creation
- Style guide using AI
- Complex scene generation
- Prompt optimization study

Practice: Test 100 prompt variations

3D Design & AR Basics

Topics:

- 3D design principles
- Canva 3D text and elements
- Introduction to Spline
- 3D mockup creation
- Basic 3D modeling concepts
- AR filter basics for social media
- Spark AR introduction
- 3D product visualization
- Virtual staging basics
- 3D logos and typography
- Exporting 3D assets
- Future of 3D in design

Projects built:

- 3D product mockup
- AR filter for Instagram
- 3D animated logo
- Virtual room design

Practice: Create 5 3D design projects

Phase 2 Integration Project

Topics:

- Combining traditional and AI design
- Multi-tool workflow
- Project planning and execution
- Quality control with AI
- Client presentation of AI work
- Pricing AI-assisted projects

Projects built:

- Complete campaign using all tools
- AI-enhanced brand package
- Video content series
- Interactive digital experience

Practice: Integrate AI into all design work

Months 3-4: Specialization & Advanced Techniques (Week 9-17 Deep Dive)

UI/UX Design Basics

Topics:

- UI vs UX understanding
- User research basics
- Wireframing in Canva
- Creating user flows
- Mobile app design principles
- Web design fundamentals
- Prototyping basics
- Design systems for UI
- Accessibility in UI design
- Usability testing basics
- Figma basics for designers
- Handoff to developers

Projects built:

- Mobile app UI design
- Website wireframe and mockup
- Design system creation
- User flow documentation

Practice: Design 5 app screens daily

Motion Design & Animation

Topics:

- Principles of animation
- Timing and easing
- After Effects basics
- Lottie animations
- SVG animations

- Animated logos and icons
- Micro-interactions
- Loading animations
- Animated data visualization
- Character animation basics
- Motion design for social media
- Creating animation style guides

Projects built:

- Animated brand elements
- Motion infographic
- Social media animation pack
- Website loading animation

Practice: Create daily animations for two weeks

Advanced Brand Design

Topics:

- Brand strategy fundamentals
- Market research for designers
- Competitor analysis
- Brand positioning
- Visual identity systems
- Brand architecture
- Packaging design advanced
- Environmental graphics
- Brand guidelines creation
- Brand evolution and refresh
- Measuring brand impact
- Portfolio brands vs product brands

Projects built:

- Complete rebrand project
- Brand extension design
- Multi-touchpoint campaign
- Brand book creation

Practice: Develop 3 fictional brands

Print Production & Specifications

Topics:

- Print production process
- Paper types and finishes
- Color management for print
- CMYK vs RGB deep dive
- Prepress preparation
- Die cutting and special finishes
- Large format printing
- Packaging dielines
- Print vendor communication
- Cost estimation for print
- Sustainable printing options
- Print quality control

Projects built:

- Print-ready business cards
- Packaging design with dieline
- Large format poster
- Print production guide

Practice: Prepare 10 designs for print

Phase 2 Capstone Project

Topics:

- Advanced project planning
- Multi-deliverable projects
- AI integration showcase
- Professional presentation
- Portfolio curation
- Career advancement planning

Projects built:

- MAJOR CAPSTONE: AI-Enhanced Brand Campaign
- Complete visual identity with AI elements
- Video content series
- Social media campaign
- Print and digital materials

Assessment: Phase 2 Final Exam - Complex design challenge

----- PHASE 3: PROFESSIONAL PRACTICE & BUSINESS (MONTHS 5-6, WEEKS 18-26) -----

Launch your design career with advanced business skills, client management, and professional portfolio development.

Months 5-6: Freelancing & Business Setup (Week 18-26)

Freelance Business Fundamentals

Topics:

- Freelancing vs employment pros/cons

- Setting up your design business
- Business registration and legal basics
- Tax considerations for freelancers
- Insurance for creative professionals
- Home office setup and tools
- Time tracking and productivity
- Project management tools
- Client communication best practices
- Building professional reputation
- Networking strategies
- Finding your niche market

Projects built:

- Business plan creation
- Professional website setup
- Service package development
- Client onboarding system

Practice: Set up complete business infrastructure

Pricing & Proposals

Topics:

- How to price design work
- Hourly vs project vs value pricing
- Creating service packages
- Proposal writing masterclass
- Quote and estimate templates
- Scope creep management
- Contract essentials
- Terms and conditions
- Payment terms and methods
- Dealing with late payments
- Negotiation strategies
- Raising your rates

Projects built:

- Pricing calculator creation
- Professional proposal template
- Service contract template
- Invoice system setup

Practice: Create proposals for 10 different projects

Client Management & Communication

Topics:

- Client onboarding process
- Discovery calls and questionnaires
- Setting client expectations
- Project brief creation
- Feedback management
- Handling difficult clients
- Revision policies
- Project presentation skills
- Video call best practices
- Email communication templates
- Building long-term relationships
- Getting testimonials and referrals

Projects built:

- Client onboarding package
- Communication template library
- Feedback system creation
- Client portal setup

Practice: Role-play 5 client scenarios

Marketing Your Design Services

Topics:

- Personal branding for designers
- Portfolio website optimization
- SEO for designers
- Social media marketing strategy
- Content marketing for designers
- Email marketing basics
- LinkedIn for B2B clients
- Instagram for design showcase
- YouTube for tutorials and visibility
- Paid advertising basics
- Networking events and conferences
- Referral programs

Projects built:

- Personal brand development
- Marketing funnel creation
- Social media strategy
- Email campaign setup

Practice: Implement 30-day marketing plan

Platform Mastery & Passive Income

Topics:

- Selling on Creative Market
- Etsy for digital products
- Gumroad setup and optimization
- Creating and selling templates
- Stock graphics creation
- Print-on-demand basics
- Course creation for designers
- Affiliate marketing opportunities
- Licensing your designs
- Building recurring revenue
- Scaling beyond 1-on-1 work
- Passive income strategies

Projects built:

- Digital product creation
- Template pack for sale
- Online shop setup
- Passive income stream launch

Practice: Create 10 products for passive income

Months 5-6: Advanced Skills & Specialization (Week 18-26 Continued)

Web Design & Development Basics

Topics:

- HTML/CSS basics for designers
- Responsive design principles
- WordPress basics
- Elementor page builder
- Webflow introduction
- Landing page design
- E-commerce design principles
- Web animation basics
- SEO-friendly design
- Web accessibility standards
- Performance optimization
- Hosting and domains

Projects built:

- Portfolio website creation
- Client website design
- Landing page with Webflow
- WordPress site setup

Practice: Build 3 complete websites

Advanced AI Workflows

Topics:

- Building AI-powered workflows
- Automating design tasks
- AI for content generation
- ChatGPT for design briefs
- AI copywriting integration
- Bulk creation with AI
- Quality control systems
- AI tool combinations
- Custom AI training basics
- Future AI developments
- Staying updated with AI
- Teaching AI to clients

Projects built:

- Automated design pipeline
- AI-powered service offering
- Client education materials
- AI workflow documentation

Practice: Automate 5 design processes

Data Visualization & Infographics

Topics:

- Data viz principles
- Chart and graph types
- When to use which visualization
- Creating compelling infographics
- Interactive data visualization
- Dashboard design
- Report design and layout
- Presentation data graphics
- Tools for data import
- Telling stories with data
- Scientific and technical graphics
- Annual report design

Projects built:

- Complex infographic design
- Interactive dashboard
- Annual report graphics
- Data story presentation

Practice: Create 20 data visualizations

Photography for Designers

Topics:

- Photography basics: composition, lighting
- Smartphone photography tips
- Product photography setup
- Portrait photography basics
- Food photography styling
- Architectural photography
- Stock photography creation
- Photo editing workflow
- Creating mockup photos
- Lifestyle photography for brands
- Managing photo libraries
- Copyright and usage rights

Projects built:

- Product photography shoot
- Brand lifestyle photos
- Stock photo collection
- Photography style guide

Practice: Shoot and edit 100 photos

Emerging Technologies

Topics:

- Designing for shared virtual spaces
- Digital ownership and giving credit
- Designing interfaces people can trust
- VR/AR design considerations
- Voice UI design basics
- Designing for AI interfaces
- Sustainability in design
- Inclusive design practices
- Design ethics and responsibility
- Future design trends
- Continuous learning strategies
- Building expertise

Projects built:

- Cohesive brand design system
- AR experience concept
- Future-focused portfolio piece
- Trend report creation

Practice: Experiment with 5 emerging technologies

Months 5-6: Portfolio & Career Launch (Week 18-26 Final Phase)

Portfolio Development

Topics:

- Portfolio strategy and planning
- Selecting your best work
- Case study creation
- Before/after presentations
- Process documentation
- Client testimonials integration
- Portfolio website design
- Behance portfolio optimization
- Dribbble presence
- LinkedIn portfolio features
- PDF portfolio creation
- Video portfolio reels

Projects built:

- Complete portfolio website
- 10 detailed case studies
- Portfolio presentation deck
- Video portfolio reel

Practice: Review and refine all portfolio pieces

Job Search & Applications

Topics:

- Design job market overview
- Resume design and content
- Cover letter writing
- LinkedIn optimization
- Job search strategies
- Remote work opportunities
- Interview preparation
- Design challenges and tests
- Salary negotiation
- Freelance vs full-time decision
- Building references
- Following up effectively

Projects built:

- Professional resume design

- LinkedIn profile optimization
- Interview portfolio prep
- Application tracking system

Practice: Apply to 20 positions/opportunities

Advanced Client Projects

Topics:

- Managing multiple clients
- Large project management
- Team collaboration
- Outsourcing and delegation
- Quality control systems
- Client retention strategies
- Upselling and cross-selling
- Building retainer relationships
- Agency vs freelance
- Starting a design agency
- Hiring and training
- Scaling your business

Projects built:

- Complex client project
- Team collaboration project
- Agency structure plan
- Growth strategy document

Practice: Complete 3 real client projects

Specialization Deep Dive

Topics:

- Choosing your specialization
- Becoming an expert
- Building authority
- Speaking and teaching
- Writing about design
- Building a following
- Thought leadership
- Conference speaking
- Workshop facilitation
- Mentoring others
- Giving back to community
- Long-term career planning

Projects built:

- Specialization portfolio
- Expert positioning strategy
- Teaching materials creation
- 5-year career plan

Practice: Develop expertise in chosen niche

Final Capstone & Graduation

Topics:

- Final project planning
- Comprehensive skill demonstration
- Professional presentation
- Peer review and feedback
- Instructor evaluation
- Certification preparation
- Alumni network access
- Continued learning resources
- Career support activation
- Graduation celebration

Projects built:

- FINAL CAPSTONE: Complete Brand & Campaign
- From strategy to execution
- AI-enhanced deliverables
- Multi-platform campaign
- Client presentation simulation

Assessment: Final Comprehensive Assessment & Certification

----- BONUS PHASE: LIFETIME LEARNING & ADVANCED MASTERY -----

Continuous updates with new tools, techniques, and industry developments. Access to masterclasses and advanced workshops.

Ongoing Advanced Workshops (Lifetime Access)

AI Tools Updates

Topics:

- Latest AI tool releases
- New Midjourney features
- DALL-E updates and improvements
- Emerging AI platforms
- Advanced prompt techniques
- AI workflow optimization

- Beta testing new tools
- AI tool comparisons
- Custom AI model training
- AI API integration
- Automating with AI
- Future of AI in design

Projects built:

- Test new AI tools monthly
- Create AI tool guides
- Share discoveries with community
- Build AI tool library

Practice: Stay current with weekly AI updates

Design Trends & Innovation

Topics:

- Annual design trend analysis
- Color trends and palettes
- Typography trends
- Motion design trends
- Social media trends
- Web design evolution
- Print design innovations
- Sustainable design practices
- Cultural design movements
- Technology-driven design
- Experimental design
- Trend forecasting

Projects built:

- Trend report creation
- Experimental design series
- Trend-based portfolio update
- Innovation showcase

Practice: Monthly trend exploration projects

Business Growth Strategies

Topics:

- Scaling design business
- Building design teams
- Agency development
- Product business creation
- SaaS for designers
- Investment and funding
- International expansion
- Partnership strategies
- Acquisition planning
- Exit strategies
- Wealth building for creatives
- Financial planning

Projects built:

- Business growth plan
- Investment pitch deck
- Partnership proposals
- 5-year financial plan

Practice: Quarterly business review and planning

Teaching & Course Creation

Topics:

- Creating design courses
- Video tutorial production
- Workshop development
- Online course platforms
- Pricing educational content
- Marketing courses
- Student engagement
- Certification programs
- Live workshop hosting
- Webinar creation
- Educational content strategy
- Building educational brand

Projects built:

- Create mini-course
- Workshop curriculum
- Tutorial video series
- Educational product launch

Practice: Teach one concept weekly

Community & Networking

Topics:

- Building design community
- Hosting design events
- Online community management
- Collaboration opportunities
- Mentorship programs

- Design challenges hosting
- Conference attendance
- Speaking opportunities
- Award submissions
- Press and media
- Industry partnerships
- Global design network

Projects built:

- Host design challenge
- Organize local meetup
- Submit to design awards
- Media kit creation

Practice: Weekly community engagement

Specialized Masterclasses (Ongoing Access)

Entertainment Industry Design

Topics:

- Movie poster design
- Album cover creation
- Concert merchandise
- Streaming platform graphics
- Gaming graphics and UI
- Esports branding
- Music video graphics
- Festival branding
- Entertainment marketing
- Celebrity branding
- Red carpet graphics
- Award show design

Projects built:

- Movie campaign design
- Album release package
- Gaming brand identity
- Festival complete branding

Practice: Create entertainment portfolio

Luxury & Premium Design

Topics:

- Luxury brand principles
- Premium packaging design
- High-end typography
- Gold and metallic effects
- Minimalist luxury
- Fashion brand design
- Jewelry marketing
- Real estate luxury marketing
- Premium product launches
- VIP event materials
- Exclusive experiences design
- Heritage brand refresh

Projects built:

- Luxury brand creation
- Premium packaging suite
- High-end event materials
- Exclusive product launch

Practice: Master luxury design aesthetics

Cultural & Social Impact Design

Topics:

- Non-profit branding
- Cause marketing campaigns
- Cultural sensitivity in design
- Inclusive design practices
- Accessibility standards
- Environmental campaigns
- Political campaign design
- Healthcare communications
- Educational materials
- Government design standards
- International design considerations
- Social media activism

Projects built:

- Non-profit rebrand
- Awareness campaign
- Inclusive design system
- Cultural event branding

Practice: Design for social good

Technical & B2B Design

Topics:

- SaaS product marketing
- Technical documentation design

- B2B sales materials
- White paper design
- Case study layouts
- Technical infographics
- Software UI elements
- API documentation
- Developer marketing
- Enterprise branding
- Trade show materials
- B2B social media

Projects built:

- SaaS marketing suite
- Technical white paper
- B2B campaign materials
- Enterprise rebrand

Practice: Master B2B design language

Personal Brand Mastery

Topics:

- Defining your unique style
- Signature design elements
- Personal brand evolution
- Thought leadership content
- Speaking and presentations
- Book and eBook design
- Personal merchandise
- Studio branding
- Awards and recognition
- Legacy building
- Retirement planning for creatives
- Giving back strategies

Projects built:

- Personal brand refresh
- Signature style guide
- Speaker materials kit
- Legacy project planning

Practice: Refine personal brand quarterly

Expert Specializations (Advanced Modules)

Design Psychology & Neuroscience

Topics:

- Psychology of color
- Cognitive load in design
- Attention and focus
- Memory and recognition
- Emotional design triggers
- Persuasion techniques
- Habit-forming design
- Neuromarketing basics
- Eye-tracking studies
- A/B testing for design
- Conversion optimization
- Behavioral economics in design

Projects built:

- Psychology-based redesign
- Conversion optimization project
- Emotional design campaign
- Behavioral design system

Practice: Apply psychology principles to all designs

Design Research & Strategy

Topics:

- User research methods
- Market research for designers
- Competitive analysis
- Design thinking process
- Workshop facilitation
- Stakeholder management
- Design sprints
- Persona development
- Journey mapping
- Service design basics
- Design metrics and KPIs
- ROI of design

Projects built:

- Complete design research project
- Design sprint facilitation
- Strategy presentation
- ROI case study

Practice: Integrate research into design process

Global Design & Localization

Topics:

- International design standards
- Cultural color meanings
- Typography for different scripts
- Right-to-left design
- Localization best practices
- Global brand adaptation
- International marketing materials
- Currency and measurement systems
- Time zone considerations
- Legal requirements by country
- Global design trends
- Cross-cultural communication

Projects built:

- Multi-language design system
- Global campaign adaptation
- International brand guide
- Localized product design

Practice: Create designs for 5 different cultures

Future of Design

Topics:

- Quantum computing and design
- Brain-computer interfaces
- Holographic design
- Biodesign and genetics
- Space and extraterrestrial design
- Climate change design solutions
- Post-pandemic design
- Automation and design jobs
- Ethics in automated design
- Universal basic design
- Design for longevity
- Speculative design projects

Projects built:

- Future concept design
- Speculative design series
- Innovation white paper
- 25-year vision project

Practice: Monthly future design exploration

Lifetime Success Program (Ongoing Support)

Alumni Success Program

Topics:

- Alumni network access
- Ongoing mentorship
- Project collaboration board
- Guest expert sessions
- Industry connections
- Referral network
- Success story features
- Annual alumni events
- Continued education credits
- Tool discounts and deals
- Lifetime community access

Projects built:

- Alumni portfolio showcase
- Collaborative alumni project
- Mentorship participation
- Success story documentation

Continuous Updates

Topics:

- Monthly tool updates
- New feature tutorials
- Industry news and insights
- Design challenge participation
- Beta testing opportunities
- Expert interviews
- Case study library access
- Template library updates
- Resource library expansion
- Q&A session access
- Office hours availability
- Priority support channel

Deliverables:

- Monthly update digest
- New tool tutorials
- Resource library access
- Community forum participation

Career Advancement Support

Topics:

- Annual career review
- Goal setting sessions
- Skill gap analysis
- Advanced certification paths
- Leadership development
- Business coaching
- Investment guidance
- Exit strategy planning
- Succession planning
- Wealth management basics
- Retirement planning
- Legacy projects

Deliverables:

- Annual career assessment
- Personalized growth plan
- Advanced certification options
- Lifetime success tracking

Assessment: Annual skill assessment and certification renewal

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1-2: 20+ foundational design projects including logos, social media posts, marketing materials
- Month 3-4: 15+ AI-enhanced projects, video content, advanced photo manipulation
- Month 5-6: 10+ professional client projects, complete campaigns, business setup
- Bonus: Ongoing specialty projects in chosen niche
- Final: Complete brand and marketing campaign with all deliverables

Total projects: 50+ professional portfolio pieces

SKILLS THE STUDENT WILL HAVE AT THE END

- Design Tools: Canva Pro mastery, Adobe Creative Suite basics, Figma introduction
- AI Tools: Midjourney, DALL-E, Stable Diffusion, ChatGPT, Adobe Firefly, Runway
- Design Skills: Typography, Color Theory, Composition, Branding, Layout, UI/UX basics
- Photo/Video: Photo editing, manipulation, video editing, animation, motion graphics
- Business: Pricing, proposals, client management, marketing, freelancing, scaling
- Specializations: Social media, marketing, branding, web design, print, packaging
- Advanced: AI workflows, automation, 3D basics, AR filters, data visualization
- Professional: Portfolio development, job search, networking, continuous learning

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-5 hours

Hands on coding: 5-7 hours

Projects: 2-3 hours

Practice problems: 1-2 hours

Total per week: 10-15 hours

SUPPORT INCLUDED

Live sessions: Weekly live design critiques and Q&A

Mentorship: Monthly 1-on-1 mentor sessions

Community: 24/7 Discord community with peer support

Code review: Professional design feedback on all projects

Career support: Portfolio reviews, job search help, client acquisition

Lifetime access: All content, updates, new tools, and features

CERTIFICATION DETAIL

Phase certificates: Certificate after each 2-month phase

Final certificate: Professional Digital Designer Certification

LinkedIn badge: Verified LinkedIn certification badge

Industry recognized: Recognized by design agencies and companies

Portfolio projects: 50+ portfolio pieces to showcase skills

WHERE THIS CAN LEAD

- Freelance Graphic Designer (Rs 30K-1L/month)
- Social Media Manager (Rs 25K-60K/month)
- Brand Designer (Rs 40K-80K/month)
- Content Creator (Rs 20K-2L/month)
- Digital Marketing Designer (Rs 30K-70K/month)
- UI/UX Designer Junior (Rs 35K-65K/month)
- Video Content Creator (Rs 40K-90K/month)
- AI Design Specialist (Rs 50K-1.5L/month)
- Design Agency Owner (Rs 1L-10L/month)
- Design Educator/Influencer (Rs 50K-5L/month)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the Canva & AI Design Masterclass cover?

A: Six months from zero: design principles, typography and color theory, Canva mastery for social media and marketing design, AI design tools and photo editing, video editing and animation, then portfolio building, freelancing basics and client work. You finish as a capable digital designer with real published pieces.

Q: Who can join this course?

A: It is open to everyone: no formal education or technical background is needed, and the plain language teaching works for school students as well as adults building a skill or a side income. Everything runs in the browser, so there is no heavy software to install.

Q: What equipment or software do I need?

A: Just a computer with an internet connection, since Canva runs in the browser. We introduce AI design and editing tools step by step during the course and always point out free options before anything paid comes up.

Q: How is progress measured?

A: Every week produces a real design deliverable, from social media posts to brand kits to edited videos, and those pieces accumulate into your portfolio. Phase-end projects act as checkpoints, and the final phase turns the portfolio into a career ready showcase.

Q: Can this actually lead to freelance work?

A: The final phase covers exactly that: pricing, client communication, briefs and portfolio presentation, alongside current design career paths. We do not promise income figures, but you finish with the portfolio and the process a working freelancer needs.

Q: Can I try before paying?

A: Yes, the first class is a free demo with no commitment; book it at learn.modernagecoders.com/contact. If you prefer to chat first, WhatsApp us at +91 91233 66161.

Q: Is this a video course?

A: No. Every class is a live one hour online session with a real teacher, never pre-recorded content. Groups are small, so your designs get individual critique, which is how design skill actually improves.

Q: Do I get a certificate?

A: Yes, a design certification on completion plus your portfolio of real projects created during the course. Modern Age Coders holds 4.9 stars across 547+ Google reviews.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/canva-ai-design-masterclass>

COURSE 30 OF 118: CBSE CLASS 10 MATHS BOARD PREP: CASE STUDIES & MOCKS

Course link: <https://learn.modernagecoders.com/courses/cbse-class-10-maths-board-exam-prep-course>

Subject area: School Board Mathematics

Designed for: High school

Who can join: CBSE Class 10 students, Standard (041) and Basic (241)

Course length: 10 months (40 weeks)

Time needed: 2 live classes/week + 3-4 hours practice, rising to timed papers in the mock phase

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

CBSE Class 10 Maths board prep course aligned to the 2026-27 syllabus: all 14 chapters, competency and case-study practice, sample-paper mocks, live classes.

In one line: The whole syllabus taught properly, then the board paper practised until it holds no surprises.

ABOUT THE PROGRAMME

Class 10 is the first year the maths marks travel with the student, and the current board paper rewards more than memorised solutions: roughly half of it is competency-based, case studies, data interpretation and situational problems, alongside about 20 percent MCQs and 30 percent traditional short and long answers. This 10-month live course is built for exactly that paper. Months 1 to 8 teach all 14 NCERT chapters across the seven syllabus units, with

the unit weightage kept in view throughout: Algebra carries 20 of the 80 theory marks, Geometry 15, Trigonometry 12, Statistics and Probability 11, Mensuration 10, and Number Systems and Coordinate Geometry 6 each. Months 9 and 10 are pure exam craft: structured revision, CBSE sample papers and previous-year papers under real timing, and error clinics after every mock. Case-study and MCQ practice is not saved for the end. From the first chapter, every topic closes with the question forms the board actually asks, so by the mock phase the format is familiar and only speed and polish remain. We teach both Standard (041) and Basic (241) tracks and help each family make that choice with evidence from the student's own test scores, not guesswork. What we promise is preparation and honest feedback; nobody can promise marks, and we do not.

WHAT MAKES THIS COURSE DIFFERENT

- Aligned to the 2026-27 CBSE syllabus as released, all 14 chapters across the 7 units, with the 80-mark theory weightage steering how class time is spent
- Competency questions from week 1: every chapter closes with case-study, data and MCQ practice in the board's actual style, not as a panic module in January
- Two full months of revision and mocks, with CBSE sample papers and previous-year papers under real 3-hour timing and a marked review of every attempt
- Standard versus Basic decided with data: periodic test scores through the year tell each family which paper suits, and we advise honestly either way
- Proof writing and stepwise presentation taught explicitly, because the board marking scheme awards steps, and students lose easy marks by skipping them
- Live small batches where the teacher sees each student's working, not a video series with a doubt inbox

THE LEARNING PATH, STEP BY STEP

Phase 1: Number Systems and the full Algebra unit: real numbers, polynomials, pairs of linear equations, quadratics and arithmetic progressions, the 26-mark backbone of the paper

Phase 2: Geometry, Coordinate Geometry and Trigonometry: similarity, distance and section formulae, trigonometric ratios and identities, heights and distances, and circle theorems

Phase 3: Mensuration, Statistics and Probability: areas related to circles, combined solids, grouped data measures and classical probability

Phase 4: Revision and the mock cycle: structured re-teaching of weak spots, CBSE sample papers and previous years under timing, error clinics and exam-day strategy

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Class 9 promotion. Weak spots in Class 9 algebra or geometry are caught by the week 1 diagnostic and patched inside the course

Board: Built for CBSE Class 10, both Mathematics Standard (041) and Mathematics Basic (241).

Students of other boards are better served by our other maths courses

Equipment: NCERT Class 10 Maths textbook, a geometry box, notebooks, and a device with stable internet for live classes

Timing: Best joined at the start of Class 10; mid-year joiners get a catch-up plan after the diagnostic

WHO THIS COURSE IS FOR

Board year students: CBSE Class 10 students who want the syllabus finished early and the board pattern practised properly, not crammed

Students choosing standard: Students keeping science or maths streams open for Class 11, who need Standard (041) depth and proof fluency

Students considering basic: Students unsure between Standard and Basic, who want the choice made on test evidence rather than fear

Case study strugglers: Students who know the methods but lose marks on the long competency stems and data questions

Presentation losers: Students whose answers are right but whose marks are not, because steps, figures and units go missing

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: NUMBER SYSTEMS AND ALGEBRA (MONTHS 1-3, WEEKS 1-12) -----

The heaviest unit first. Algebra alone carries 20 of the 80 theory marks, and with Real Numbers this phase covers 26. Every chapter closes with board-style MCQ and case-study practice.

Month 1: Real Numbers and Polynomials (Weeks 1-4)

Weeks 1-2: Real Numbers

Topics:

- Diagnostic paper on Class 9 essentials: identities, linear equations, basic geometry
- The fundamental theorem of arithmetic and prime factorisation
- HCF and LCM by prime factorisation, and the HCF $\times$ LCM relationship
- Proving irrationality: root 2, root 3, root 5 by contradiction, written to board standard

- Word problems on HCF and LCM: bells, circular tracks, stacking crates
- Board question forms for this chapter: 1-mark MCQs and 2-3 mark proofs

Projects built:

- Chapter question bank entry: 15 solved board-style questions on real numbers, built and kept by the student

Practice: 25 prime factorisation and HCF-LCM problems plus 3 irrationality proofs written out in full board steps

Weeks 3-4: Polynomials

Topics:

- Zeros of a polynomial read from the graph: where the curve cuts the x-axis
- Geometric meaning of zeros for linear and quadratic polynomials
- Relationship between zeros and coefficients of a quadratic polynomial
- Forming a quadratic polynomial from given zeros
- Verification questions: checking the zero-coefficient relationships
- MCQ traps in this chapter: sign errors and reversed relationships

Projects built:

- Graph-reading sheet: 8 polynomial graphs interpreted for number and sign of zeros

Practice: 30 zeros-and-coefficients problems, 10 of them MCQ format, misses redone with the relationship written out

Assessment: Month 1 test: a 25-mark paper on Real Numbers and Polynomials in board format, marked with the board's stepwise scheme

Month 2: Pair of Linear Equations and Quadratic Equations (Weeks 5-8)

Weeks 5-6: Pair of Linear Equations in Two Variables

Topics:

- Graphical solution: consistent, inconsistent and dependent pairs seen as line pictures
- The ratio test for number of solutions, and why it works
- Substitution method with clean recording of steps
- Elimination method and choosing which variable to remove
- Word problems: ages, two-digit numbers, speeds and boats, fixed plus per-unit costs
- Case-study practice: fare structures and cafeteria pricing, the board's favourite settings

Projects built:

- Word-problem translation drill: 10 situations converted to equation pairs before any solving

Practice: 20 equation pairs solved by the method of your choice plus 6 word problems and 1 full case study, self-timed

Weeks 7-8: Quadratic Equations

Topics:

- Standard form and checking whether an equation is quadratic
- Solving by factorisation: splitting the middle term with speed
- The quadratic formula, derived once and then drilled
- The discriminant and nature of roots: real, equal, or none
- Word problems: consecutive numbers, areas, speed-time, work problems
- Finding unknown coefficients from conditions on the roots

Projects built:

- Chapter question bank entry: 15 solved board-style quadratics including 2 case studies

Practice: 25 quadratics split between factorisation and formula, plus 8 nature-of-roots questions with reasons stated

Assessment: Month 2 test: a 30-mark paper on both equation chapters with MCQ, short answer and one case study

Month 3: Arithmetic Progressions and Algebra Consolidation (Weeks 9-12)

Weeks 9-10: Arithmetic Progressions

Topics:

- Recognising an AP and finding the common difference
- The nth term formula, forwards and in reverse: which term is 78
- Sum of the first n terms, both formula forms and when each helps
- Word problems: instalments, salaries, theatre seating, ladder rungs
- Three unknowns from two conditions: combining the nth term and sum formulas
- AP case studies: savings plans and stacked logs in board style

Projects built:

- AP situations sheet: 8 real-life progressions identified, with first term and difference extracted

Practice: 30 AP problems from nth term to sums, including 5 word problems and 1 case study, misses logged by type

Weeks 11-12: Algebra Unit Under Board Conditions

Topics:

- Mixed revision across all five phase 1 chapters
- Stepwise presentation clinic: what the marking scheme pays for and what it ignores
- Assertion-reason questions: the logic and the common traps
- Case-study workshop: reading the stem once, answering all sub-parts efficiently
- Timed sectional practice: the algebra questions of a sample paper in isolation
- Error log review: each student's top three algebra weak spots named and drilled

Projects built:

- Personal error log formalised: every phase 1 test miss classified as concept, method or presentation

Practice: One timed algebra-only sectional paper plus 10 assertion-reason questions, reviewed against the marking scheme

Assessment: Phase 1 milestone: a 40-mark algebra and number systems paper in full board format, 50 percent competency questions

PHASE 2: GEOMETRY, COORDINATE GEOMETRY AND TRIGONOMETRY (MONTHS 4-6, WEEKS 13-24)

The proof-and-figure half of the paper: 33 marks across Geometry, Coordinate Geometry and Trigonometry, where drawing the figure and showing the steps is most of the battle.

Month 4: Triangles and Coordinate Geometry (Weeks 13-16)

Weeks 13-14: Triangles and Similarity

Topics:

- Similar figures and what similarity actually claims
- The basic proportionality theorem and its converse, proved and applied
- Similarity criteria: AA, SSS and SAS, with when to use which
- Finding lengths in nested and overlapping triangle figures
- Writing a geometry proof the board way: given, to prove, construction, proof
- Similarity in the wild: shadows, ramps and scale models as case studies

Projects built:

- Proof portfolio started: BPT and two similarity applications written to full board standard

Practice: 20 similarity problems including 4 proofs written out in the four-part format, figures drawn for every question

Weeks 15-16: Coordinate Geometry

Topics:

- The distance formula, derived from Pythagoras and drilled
- Types of triangles and quadrilaterals verified by distances
- The section formula and midpoint as its special case
- Finding ratios: where the x-axis cuts a line segment
- Collinearity and equidistance problems
- Coordinate case studies: layouts of gardens, flags on a sports field

Projects built:

- Chapter question bank entry: 12 solved coordinate geometry questions across all board formats

Practice: 25 distance and section formula problems with a labelled sketch for each, plus 1 case study

Assessment: Month 4 test: a 25-mark paper on Triangles and Coordinate Geometry with one full proof question

Month 5: Trigonometry and Its Applications (Weeks 17-20)

Weeks 17-18: Introduction to Trigonometry

Topics:

- The six ratios defined in a right triangle, with the naming logic
- Exact values for 0, 30, 45, 60 and 90 degrees, and how to rebuild the table from two triangles
- Finding all ratios when one is given
- The fundamental identity and its two rearrangements
- Proving trigonometric identities: strategy, not trial and error
- MCQ speed work on ratio values and identity recognition

Projects built:

- Identity toolkit card: the identities plus the five standard proof moves, in the student's own words

Practice: 30 ratio evaluations and 8 identity proofs, each proof annotated with which move was used at each step

Weeks 19-20: Heights and Distances

Topics:

- Angles of elevation and depression, and reading them off a clean figure
- The non-negotiable first step: draw and label the right triangle
- Single-triangle problems: towers, kites, ladders
- Two-triangle problems: two observers, moving closer, two levels
- Choosing the ratio that solves in one step instead of three
- Case studies: lighthouses, drones and temple towers in board style

Projects built:

- Figure-first drill set: 10 heights and distances problems where the marked figure is drawn before any calculation

Practice: 18 heights and distances problems including 4 two-triangle setups and 1 case study, every figure labelled

Assessment: Month 5 test: a 25-mark trigonometry paper mixing ratios, identities and applications

Month 6: Circles and the Half-Syllabus Checkpoint (Weeks 21-24)

Weeks 21-22: Circles and Tangents

Topics:

- Tangents and secants: what touching actually means
- The tangent-radius perpendicularity theorem, proved and used
- Equal tangents from an external point, proved and used
- Angle chasing in tangent figures using both theorems
- Lengths in circumscribed quadrilaterals and triangles
- Circle proofs written to the four-part board standard

Projects built:

- Proof portfolio extended: both tangent theorems plus two applications written in full

Practice: 20 circle problems including 3 proofs and 5 angle-chasing figures, figures redrawn not copied

Weeks 23-24: Half-Syllabus Mock and Review

Topics:

- Structured revision across phases 1 and 2: one summary mat per chapter
- Half-syllabus mock in full board format and timing
- Marking workshop: students mark an anonymised script against the scheme
- Error clinic: every mock miss classified and assigned a drill
- Basic versus Standard conversation opens: what the mock evidence suggests for each student
- Presentation audit: figures, steps and units checked across the whole script

Projects built:

- Half-syllabus mock with a written self-review: three strengths, three fixes, one habit to change

Practice: Redo every mock miss untimed with full working, then a 15-question drill on your weakest chapter

Assessment: Phase 2 milestone: half-syllabus board-format mock covering all ten chapters taught so far

PHASE 3: MENSURATION, STATISTICS AND PROBABILITY (MONTHS 7-8, WEEKS 25-32)

The final 21 marks of new content: areas related to circles, combined solids, grouped data and probability, all heavy with the data-interpretation forms the competency questions love.

Month 7: Areas Related to Circles and Combined Solids (Weeks 25-28)

Weeks 25-26: Areas Related to Circles

Topics:

- Sector area and arc length, tied back to the fraction of the full circle
- Segment area: sector minus triangle, set up cleanly
- Combination figures: designs, running tracks, flower beds
- Working with standard angles in sectors and segments
- Leaving answers in terms of π versus computing, and when the paper wants which
- Case studies: clock faces, umbrellas and window designs

Projects built:

- Combination-figure sheet: 8 shaded-region problems with the decomposition drawn before any formula

Practice: 20 sector, segment and shaded-region problems with the decomposition stated in one line each time

Weeks 27-28: Surface Areas and Volumes

Topics:

- Surface areas and volumes of cube, cuboid, cylinder, cone, sphere and hemisphere, consolidated
- Combined solids: capsule, tent, ice-cream cone, toy on a hemisphere
- Which surfaces vanish when solids join: the one idea behind every surface-area question
- Volume problems on combined solids
- Unit discipline: cubic centimetres, litres and the conversions the board expects
- Case studies: storage tanks, medicine capsules and circus tents

Projects built:

- Solid dissection cards: 6 combined solids sketched, split into parts, with the joining surfaces marked

Practice: 18 combined-solid problems, half surface area and half volume, units checked on every answer

Assessment: Month 7 test: a 20-mark mensuration paper including one case study and one MCQ set

Month 8: Statistics and Probability (Weeks 29-32)

Weeks 29-30: Statistics of Grouped Data

Topics:

- Grouped frequency tables: class marks, limits and boundaries
- Mean by the direct method, assumed mean and step deviation, and when each saves time
- Median of grouped data: the cumulative frequency column done carefully
- Mode of grouped data and the modal class
- Reading and interpreting data tables the way competency questions present them
- Choosing between mean, median and mode when a question asks which represents the data

Projects built:

- Data study: the student collects a real dataset from home or school, groups it and computes all three measures

Practice: 12 full grouped-data problems covering all three measures, tables laid out to board standard

Weeks 31-32: Probability and the Content Finish Line

Topics:

- Classical probability: equally likely outcomes and the counting behind them
- Cards, dice, coins and balls: the standard sample spaces mastered
- Complementary events and the not-happening shortcut
- Two-object experiments: two dice, two coins, drawn without replacement kept simple
- Probability case studies and data-linked questions
- Full-syllabus map: every chapter now taught, revision plan issued

Projects built:

- Chapter question bank completed: the student's own solved bank now covers all 14 chapters

Practice: 30 probability questions from single events to two-object experiments, plus 1 case study

Assessment: Phase 3 milestone: a 25-mark statistics and probability paper, plus the full-syllabus revision plan agreed with each student

PHASE 4: REVISION AND THE MOCK CYCLE (MONTHS 9-10, WEEKS 33-40)

No new content, on purpose. Structured revision by unit, then CBSE sample papers and previous-year papers under real timing, each followed by an error clinic.

Month 9: Structured Revision (Weeks 33-36)

Weeks 33-34: Algebra and Number Systems Revisited

Topics:

- Rapid re-teach of the five algebra-phase chapters, driven by each student's error log
- Mixed-chapter problem sets: questions that do not announce their chapter
- MCQ sprint sessions: 20 questions in 25 minutes with instant review
- Case-study circuit: four stems across four chapters in one sitting
- Formula-free fluency check: rebuilding key formulas from scratch
- One-page revision sheets finalised per chapter

Projects built:

- Revision sheet set one: student-made single pages for all five algebra-phase chapters

Practice: One mixed algebra paper of 30 marks self-timed, plus 20 MCQs, both marked in class

Weeks 35-36: Geometry, Trigonometry and the Data Chapters Revisited

Topics:

- Proof rehearsal: BPT, tangent theorems and identity proofs rewritten from memory
- Heights and distances mixed set, figures first as always
- Mensuration decomposition drills on unseen combination figures
- Statistics and probability rapid set with full table discipline
- The 50 percent competency share: reviewing what the recent board papers actually asked
- Time budgeting: minutes per section planned for the 3-hour paper

Projects built:

- Revision sheet set two: single pages for the remaining nine chapters, completing the personal revision pack

Practice: One mixed geometry-trigonometry paper of 30 marks self-timed, plus a 15-question data-chapters drill

Assessment: Month 9 checkpoint: two sectional papers marked against the board scheme, error logs updated

Month 10: The Mock Cycle (Weeks 37-40)

Weeks 37-38: Sample Papers Under Real Timing

Topics:

- The CBSE sample paper attempted in one 3-hour sitting
- Marking against the official scheme, step by step
- Error clinic one: concept gaps versus presentation losses separated
- Previous-year paper attempted under timing
- Question selection strategy: order of attack, when to move on, returning to parked questions
- Handling the case-study section efficiently: stem once, sub-parts in order

Projects built:

- Mock file opened: each paper filed with its marked script, error classification and one-line fix per miss

Practice: Between classes: one additional sample paper attempted at home under honest timing, brought marked to class

Weeks 39-40: Final Mocks and Exam Craft

Topics:

- Two final full mocks under strict conditions, spaced for recovery
- Error clinic two: closing the last recurring mistakes
- Presentation final pass: figures, steps, units, boxed answers
- The last 48 hours: what to revise, what to leave alone, sleep over cramming
- Exam-day routine: admit card to answer script, walked through calmly
- Course close: results of the mock series reviewed with each family

Projects built:

- Completed mock file: the full series with scores charted, handed to the student as their evidence of readiness

Practice: Light targeted drills only, set individually from each student's final error log

Assessment: Course milestone: final full-syllabus mock in board format, reviewed one-on-one, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A personal chapter question bank: 12-15 solved board-style questions per chapter, built across all 14 chapters
- A proof portfolio: every syllabus theorem written to the four-part board standard
- A personal error log maintained from month 1 and used to steer revision
- Word-problem translation drills for both equation chapters
- A real-data statistics study grouped and analysed by the student
- One-page revision sheets for all 14 chapters, student-made
- A half-syllabus mock at month 6 and a full mock series in month 10
- The completed mock file: marked scripts, error classifications and score chart

Total projects: A complete self-built revision system: question bank, proof portfolio, revision sheets, error log and marked mock file

SKILLS THE STUDENT WILL HAVE AT THE END

- Every chapter of the 2026-27 CBSE Class 10 Maths syllabus, taught to Standard (041) depth
- Case-study, MCQ and assertion-reason technique for the competency-heavy board pattern
- Geometry and identity proofs written the way the marking scheme pays
- Grouped-data statistics and classical probability with full table discipline
- Timed paper craft: question selection, time budgeting and clean presentation
- Self-review: marking your own work against a scheme and classifying your own errors

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; mock-phase sittings run longer for full papers

Practice: 3-4 hours weekly of problem sets and question-bank building, rising with timed papers in months 9-10

Review: Homework marked against board-style stepwise schemes; error logs reviewed with the teacher monthly

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems

Progress updates: Monthly progress notes to parents with test scores and the error-log summary, plus a Standard versus Basic recommendation by mid-course

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside the student's marked mock series

WHERE THIS CAN LEAD

- Class 11 Mathematics in the science or commerce stream, entered with the Class 10 base genuinely solid
- Class 11 Applied Mathematics for students who took the Basic paper route
- Our JEE Foundation and higher maths courses for students aiming at engineering entrances
- Computer science and economics electives, both of which lean on Class 10 algebra and statistics
- Confident handling of any exam built on the competency pattern, which CBSE is extending across subjects

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Should my child take Mathematics Standard (041) or Basic (241)?

A: Both papers cover the same syllabus; Standard asks harder questions. Standard is the safer choice for students keeping science or maths-based streams open, while Basic suits students certain they will not continue maths after Class 10. The rules have also loosened: from the 2025-26 session CBSE allows students who took Basic to move to Standard Mathematics in Class 11, subject to the school being satisfied they can cope. We teach to Standard depth by default and give each family a data-backed recommendation by mid-course, based on the student's own test scores.

Q: What is the current CBSE Class 10 Maths exam pattern?

A: The subject is assessed out of 100: an 80-mark theory paper and 20 marks of school-conducted internal assessment. In recent board papers roughly 50 percent of the theory questions are competency-based, meaning case studies, data interpretation and situational problems, with about 20 percent MCQs and 30 percent traditional short and long answers. The whole course is built around that split, which is why case-study practice starts in week 1 rather than in the revision phase.

Q: What are case-study questions and how do you prepare students for them?

A: A case study gives a short real-world stem, a savings plan, a temple tower, a cafeteria price list, followed by three or four sub-questions drawn from the syllabus. Students rarely lack the maths; they lose marks to slow reading and poor extraction of the given data. We close every chapter with case studies in the board's own style and teach a fixed routine: read the stem once, list the givens, answer sub-parts in order. By the mock phase the format is routine.

Q: Is the NCERT textbook enough for the board exam?

A: NCERT is the core, and every board question is rooted in it, so we work through it completely, including the examples students usually skip. For Standard-paper depth we add NCERT Exemplar problems, CBSE sample papers and previous-year questions, because the competency questions dress NCERT ideas in unfamiliar settings and a student who has only seen

textbook phrasing can freeze. No private publisher guides are required.

Q: How does the course handle the syllabus itself? Has anything changed?

A: We teach the 2026-27 syllabus as CBSE released it: 14 NCERT chapters across seven units, with the 80 theory marks weighted 20 to Algebra, 15 to Geometry, 12 to Trigonometry, 11 to Statistics and Probability, 10 to Mensuration, and 6 each to Number Systems and Coordinate Geometry. Constructions remains out of the syllabus, as it has been in recent sessions, and no new chapter deletions were made for 2026-27. If CBSE issues a mid-year circular, the course plan is adjusted the same week.

Q: What about the internal assessment marks? Do you help with those?

A: The 20 internal marks are awarded by the school: 10 for periodic tests (schools usually count the best scores through the year), 5 for the portfolio of notebooks and assignments, and 5 for maths lab activities with viva. We cannot award those marks, but the course feeds them directly: our monthly tests mirror periodic-test formats, and the question bank, proof portfolio and revision sheets the student builds all belong in a portfolio a school is happy to see.

Q: I have heard Class 10 boards are now held twice a year. How does the course schedule work with that?

A: Yes, CBSE has moved Class 10 to two board sittings from 2026, with the main exam in February and an optional second sitting in May for improvement. Our 10-month plan targets the February main exam: content finishes by month 8, and months 9 and 10 are revision and full mock cycles timed to peak just before the paper. A student who chooses the May sitting to improve keeps their revision system, mock file and error log, which is most of what a second attempt needs.

Q: Can my child join mid-year, and do you promise a particular score?

A: Mid-year joining works until about month 5; the week 1 diagnostic tells us what has been missed and the catch-up plan covers it through extra problem sets and doubt sessions. On scores we are direct: nobody can honestly promise board marks, and we do not. What we commit to is full syllabus coverage, board-pattern practice from the first chapter, a marked mock series, and honest monthly reporting so you always know exactly where your child stands.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/cbse-class-10-maths-board-exam-prep-course>

=====

COURSE 31 OF 118: CBSE COMPUTATIONAL THINKING & AI FOR KIDS (CLASS 3-8)

=====

Course link: <https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-kids-classes-3-to-8>

Subject area: CBSE School Curriculum - Computational Thinking & Artificial Intelligence

Designed for: Kids

Who can join: Absolute Beginner (Class 3-5) to Intermediate AI Project Builder (Class 6-8)

Course length: 9 months (40 weeks, expandable to 12-month mastery track)

Time needed: 5-7 hours/week (2 live classes + self-practice)

Certificate: Certified Young Computational Thinker & AI Innovator (CBSE-aligned)

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

CBSE-aligned Computational Thinking & AI course for kids in Classes 3-8 (ages 8-13):
decomposition, algorithms, Scratch, Teachable Machine and 50+ projects.

In one line: From Curiosity to Creating AI, The Future-Ready Child Starts Here!

ABOUT THE PROGRAMME

The Central Board of Secondary Education (CBSE) has officially integrated Computational Thinking and Artificial Intelligence into the foundational and middle school curriculum as a core 21st-century skill. This comprehensive 9-month program is specifically designed to give your child a deep, hands-on mastery of both Computational Thinking and AI, completely aligned with CBSE's framework, NCERT's AI handbooks, and the National Education Policy (NEP) 2020. Your child will learn to think like a scientist, reason like a mathematician, solve like an engineer, and create like an AI researcher. Starting with unplugged thinking activities (no screens!), we gradually introduce Scratch block coding, AI concepts through stories and games, Google's Teachable Machine, the official CBSE AI Project Cycle (Problem Scoping with 4Ws Data Acquisition Data Exploration Modelling Evaluation), AI Ethics, Sustainable Development Goals (SDGs), and culminate in real AI projects your child will proudly showcase. By the end, your child will have built 50+ projects, understood how Netflix, Alexa, YouTube and self-driving cars really work, and will be fully prepared for CBSE AI Class 9 (Code 417) and later AI Class 11-12 (Code 843).

WHAT MAKES THIS COURSE DIFFERENT

- 100% aligned with the official CBSE Computational Thinking & AI framework
- Separate, age-appropriate tracks for Classes 3-5 and Classes 6-8
- Follows the CBSE AI Project Cycle methodology used in board curriculum
- Mix of unplugged (no-screen) and plugged (digital) activities, healthy screen time
- Uses exact tools CBSE recommends: Scratch, Teachable Machine, Code.org, MIT App Inventor, Python (for Class 8)
- Direct preparation for CBSE AI Class 9-10 (Code 417) & Class 11-12 (Code 843)
- AI Ethics, Bias, Privacy & SDG integration from day one
- Live expert-led classes + parent progress reports + project showcase days
- 50+ original AI & CT projects for school portfolio and competitions
- Compatible with Atal Tinkering Lab (ATL) and CBSE AI School projects

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Unplugged Computational Thinking, Scratch Basics, What is AI? (Classes 3-5 level)

Phase 2: Building (Months 4-6): Advanced Scratch, AI Domains (CV/NLP/Data), Teachable Machine, AI Project Cycle (Classes 6-8 level)

Phase 3: Creating (Months 7-8): Real AI Projects, Python intro, Ethics & SDG AI Solutions, Mini-hackathon

Phase 4: Mastery (Month 9): CBSE-style AI Capstone Project, Portfolio, Certification & Transition to CBSE AI Class 9 (417)

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 8-13 years (Classes 3 to 8)

Coding experience: None, absolute beginners welcome

Equipment: Laptop/desktop with stable internet (4 Mbps+), webcam, headphones

Software: All free, Scratch, Python, Teachable Machine, Orange (we guide installation)

Time commitment: 5-7 hours per week (2 live classes + self-practice)

English: Basic reading ability (Class 3+ level)

Parent support: Minimum 30 mins/week to review progress

Motivation: Curiosity, willingness to make mistakes, excitement to build

WHO THIS COURSE IS FOR

Primary students: CBSE students in Classes 3-8 looking to master Computational Thinking and AI

Other boards: ICSE, IB, State Board and International students, curriculum is fully transferable

Homeschoolers: Perfect structured AI curriculum for homeschool families

Gifted children: Advanced learners ready to go beyond school curriculum

Competition aspirants: Kids preparing for AI Olympiads, CBSE Hackathons, Intel AI Youth challenges

Cbse school students: Students whose schools already teach CBSE AI curriculum and want deeper mastery

Parents of future engineers: Parents preparing kids for future AI-driven careers

Anyone: Any 8-13 year-old curious about how AI, Alexa, ChatGPT and self-driving cars work

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: COMPUTATIONAL THINKING FOUNDATIONS (MONTHS 1-3, WEEKS 1-13), ALIGNED TO CBSE CLASSES 3-5

Build the four pillars of Computational Thinking, Decomposition, Pattern Recognition, Abstraction and Algorithms, through unplugged activities, games, storytelling and block coding. No prior coding or computer experience required. Perfect for CBSE Classes 3, 4 and 5 students.

Month 1: The Four Pillars of Computational Thinking (Unplugged + Intro to Scratch) (Week 1-4)

Introduction to Computational Thinking & Why It Matters

Topics:

- What is Computational Thinking? (Definition as per CBSE/NCERT framework)
- How do computers 'think'? (Simple explanation for kids)
- Difference between a human brain and a computer
- The 4 pillars of CT: Decomposition, Pattern Recognition, Abstraction, Algorithms
- Real-life CT: Brushing teeth, making sandwich, getting ready for school
- CT all around us: Traffic signals, recipes, board games
- Growth mindset & making mistakes, the scientist way
- Introduction to the CBSE CT framework (age-appropriate)
- Thinking journal: Your child's first CT diary
- Family CT challenge: Find 5 algorithms at home

Projects built:

- Create a 'How I Get Ready for School' step-by-step poster
- Break down making Maggi into 10 precise steps (Decomposition)
- CT Thinking Diary, daily entries for a week

Practice: Daily 10-minute unplugged CT game from our parent pack
Decomposition, Breaking Big Problems Into Small Ones

Topics:

- What is Decomposition?
- Why decomposition matters (examples: school projects, cleaning room)
- Decomposing a drawing into shapes
- Decomposing a birthday party plan
- Tree diagrams and mind maps for decomposition
- Decomposing a school timetable
- Decomposing a game into rules
- Practice: Decompose 10 everyday tasks
- When decomposition helps (and when it doesn't)
- Team decomposition: Family project breakdown

Projects built:

- Decompose your favorite board game into its smallest rules
- Plan a pretend birthday party using only decomposition
- Create a decomposition poster for any Class subject

Practice: Decompose one new task every day, parent signs logbook

Pattern Recognition, The Secret Skill of Scientists

Topics:

- What are patterns? (Visual, numerical, behavioral, musical)
- Patterns in nature: leaves, flowers, honeycomb, zebra stripes
- Patterns in math tables (2, 5, 10 tables)
- Patterns in Indian festivals & seasons
- Spotting patterns in stories & rhymes
- Patterns in our daily routine
- Why AI uses pattern recognition (face unlock, Netflix recommendations)
- Pattern puzzles and Sudoku for kids (4x4)
- Creating your own patterns
- Pattern break: Finding the 'odd one out'

Projects built:

- Pattern hunt journal, photograph 20 patterns around you
- Design a Rangoli using mathematical patterns
- Create a pattern-based memory game

Practice: Spot 5 new patterns daily

Abstraction & Algorithms, Recipes for Computers

Topics:

- What is Abstraction? (Keeping important, ignoring rest)
- Maps as abstractions (Metro map, school map)
- What is an Algorithm? Step-by-step instructions
- Recipes as algorithms
- Algorithms for brushing teeth, tying shoelaces
- Sequence matters, why order is important
- Introduction to Flowcharts (shapes and arrows)
- First flowchart: Morning routine
- Writing your first algorithm on paper
- Robot Game: Human Robot, algorithms in action

Projects built:

- Write the 'Perfect Algorithm' for making your favorite food
- Design a flowchart adventure story
- Program your family to be 'robots' and follow your algorithm

Practice: Write one algorithm per day for any daily task

Month 2: First Coding with Scratch, Where CT Meets Computers (Week 5-8)

Welcome to Scratch, The Block Coding Playground

Topics:

- Introduction to Scratch (MIT), why CBSE recommends it
- Scratch interface tour: Stage, Sprites, Blocks, Scripts
- Motion blocks: Move, turn, glide
- Your first Scratch program: Make the cat dance
- Saving and managing Scratch projects
- Scratch account creation (parent-supervised)
- Connecting CT to Scratch: Every program = an algorithm
- Changing costumes & backgrounds
- Sound blocks, making your sprite speak in Hindi/English
- First creative project: Digital greeting card

Projects built:

- Animated birthday card for a family member
- Dancing cat in 5 different backgrounds
- Digital storybook cover with your name

Practice: Explore one new Scratch block daily

Events, Loops & Repeat, Making Programs Powerful

Topics:

- Events in Scratch: 'When flag clicked', key press, sprite click
- Loops: Repeat, forever (why loops save time)
- Real-life loops: brushing teeth 20 times, school bell rings 6 times
- Nested loops for patterns
- Pen tool, drawing geometric shapes with code
- Drawing squares, triangles, hexagons with loops

- Spiral drawings with loops
- Variables introduction, boxes that hold information
- Score counter in a simple game
- Name input using 'Ask' block

Projects built:

- Geometric art generator (shapes with pen + loops)
- Simple counting game
- Animated alphabet learning app

Practice: Create one new looping animation daily

Decisions, Conditions & Interactive Stories

Topics:

- If-then blocks, decisions in code
- If-else blocks, choosing between two options
- Comparison: Less than, greater than, equal to
- Interactive stories with choices
- Keyboard-controlled sprites
- Sprite-to-sprite interaction (touching, bouncing)
- Adding sound effects for actions
- Timer and countdown
- Changing background on condition
- Branching storylines

Projects built:

- 'Choose Your Adventure' interactive story
- Keyboard-controlled character runner
- Quiz game with If-Else logic (Science or GK quiz)

Practice: Add If-Else logic to previous projects

First Mini Capstone: Build Your Own Scratch Game

Topics:

- Game design basics for kids
- Planning: storyboarding before coding
- Combining motion, events, loops, conditions
- Scoring systems
- Lives and Game Over screens
- Winning and losing conditions
- Play-testing & debugging
- Sharing with friends and family
- Publishing to Scratch studio
- Presenting your project

Projects built:

- CAPSTONE 1: Complete Scratch game (Catch the Stars, Maze Runner, or Pong)
- Record a project showcase video (2 mins)
- Write a simple README for your game

Assessment: Phase 1 Mid-Milestone: Scratch Project Certification

Month 3: What is AI?, Your First Encounter with Artificial Intelligence (Week 9-13)

AI All Around Us, Seeing AI Everywhere

Topics:

- What is Intelligence? What makes humans smart?
- What is Artificial Intelligence? (Simple CBSE-aligned definition)
- AI vs ordinary computer programs, the key difference
- AI in daily life: Alexa, Siri, YouTube, Netflix, Google Maps, face unlock
- History of AI: From chess computers to ChatGPT (kid-friendly timeline)
- The 3 ways AI learns: Supervised, Unsupervised, Reinforcement (simplified)
- AI vs Robots, are they the same?
- Famous AI success stories (AlphaGo, Watson, self-driving cars)
- Things AI is good at (and not good at)
- Where does AI go wrong? (Introduction to bias)

Projects built:

- AI Spotter Journal, find 30 examples of AI in your home & neighborhood
- Draw an 'AI Timeline' poster
- Interview a parent/grandparent: 'What was life before AI?'

Practice: Identify AI vs non-AI apps on family phone

How Does AI 'See', Introduction to Computer Vision

Topics:

- How do humans see? (Eyes + brain)
- How do computers 'see'? (Cameras + AI)
- Pixels, colors, and images for computers
- Introduction to Computer Vision (CV domain of AI)
- Face detection vs face recognition
- Object detection (cars, traffic signs, animals)
- QR codes, a simple CV example
- Medical imaging, how AI detects diseases
- Ethics: Is face recognition always good?
- Fun hands-on: Training a simple image classifier with Teachable Machine

Projects built:

- Train a Teachable Machine model to recognize 'happy face vs sad face'
- Train a model to classify 3 objects you own
- Poster: '10 places where Computer Vision is used'

Practice: Spot 5 Computer Vision apps daily

How Does AI 'Listen & Talk', Natural Language & Speech

Topics:

- How computers understand language (Natural Language Processing intro)
- Speech-to-Text: How Alexa & Google understand you
- Text-to-Speech: How computers talk back
- Chatbots, kid-friendly intro
- Simple rule-based chatbot in Scratch
- Language translation with AI
- Voice assistants: Alexa, Google Home, Siri
- How AI learns Hindi, Tamil, English, Bengali and other languages
- Spam detection in email, a NLP example
- Ethics: When should you trust a chatbot?

Projects built:

- Build a rule-based chatbot in Scratch (answers 10 common questions)
- Record voice commands and explore Google Assistant
- Design a friendly AI assistant for elderly people

Practice: Talk to voice assistants and note what works, what doesn't

Introduction to the CBSE AI Project Cycle (4Ws & Beyond)

Topics:

- What is the AI Project Cycle? (Official CBSE framework)
- The 5 stages: Problem Scoping Data Acquisition Data Exploration Modelling Evaluation
- Stage 1: Problem Scoping using 4Ws (Who, What, Where, Why)
- Practice: Apply 4Ws to a school problem
- Stage 2: Data Acquisition, what is data?
- Types of data: Images, text, numbers, sound
- Stage 3: Data Exploration, looking for patterns
- Graphs and charts for young kids
- Stage 4 preview: What is a 'model'?
- Stage 5 preview: How do we know AI is right?

Projects built:

- Apply the 4Ws to 'Why do kids waste food in school canteen?'
- Collect data on weather for 7 days and draw a chart
- Create an AI Project Cycle poster for classroom

Practice: Use 4Ws on one problem every day

Phase 1 Capstone, Your First AI-Assisted Project

Topics:

- Choosing a community problem
- Applying the full AI Project Cycle
- Combining Scratch + Teachable Machine
- Documentation & presentation
- Demonstrating to parents and peers
- Getting feedback and improving
- Celebrating first success
- Preparing for Phase 2

Projects built:

- CAPSTONE 2: AI-powered Scratch project solving a home problem (e.g. lost item finder, pet mood detector, uniform checker)
- 3-slide presentation
- 2-minute demo video

Assessment: CBSE CT & AI Foundation Certificate (Classes 3-5 level)

PHASE 2: AI BUILDER TRACK (MONTHS 4-6, WEEKS 14-26), ALIGNED TO CBSE CLASSES 6-8

Dive deep into Artificial Intelligence with the official CBSE Classes 6-8 AI curriculum. Master the full AI Project Cycle, explore all three domains of AI (Data, Computer Vision, Natural Language Processing), begin Python programming, and build real AI-powered applications that solve real-world problems.

Month 4: Mastering the CBSE AI Project Cycle (Week 14-17)

Problem Scoping & Data Acquisition (Deep Dive)

Topics:

- Problem Scoping, 4Ws in detail (Who, What, Where, Why)
- Stakeholder identification
- Framing the problem statement
- SDG (Sustainable Development Goals) connection, mandated by CBSE
- Selecting SDG goals for your AI project
- Data types: Structured (tables), Unstructured (images, text, audio)
- Sources of data: Surveys, sensors, public datasets (Kaggle for kids), APIs
- Data privacy basics, why it matters
- How to design a survey (Google Forms)
- Collecting data ethically
- Data cleaning basics, removing mistakes
- Introduction to CSV files

Projects built:

- Create a full CBSE-format AI Project Cycle document
- Design and conduct a survey in your school (30 responses)
- Map your project to 2 SDG goals

Practice: Practice 4Ws on 3 new problems weekly

Data Exploration & Visualization

Topics:

- Data exploration, why we look at data
- Types of charts: Bar, Line, Pie, Scatter (when to use which)
- Google Sheets / MS Excel for data exploration
- Making charts in Google Sheets
- Finding outliers and missing values
- Mean, median, mode for kids
- Data stories, telling the truth with data
- Avoiding misleading graphs
- Introduction to data visualization best practices
- Creating infographics in Canva
- Interpretation, what does the data say?
- Designing a data dashboard

Projects built:

- Collect and visualize your class's screen time data
- Analyze a public dataset on air pollution in Indian cities
- Create a data-driven infographic on a CBSE social topic

Practice: Make one chart from real data every day

Month 5: AI Domains, Data, Computer Vision, NLP (Week 18-21)

Data Science, Teaching Computers to Predict

Topics:

- What is Data Science?
- Prediction vs Classification vs Clustering
- Linear thinking in predictions (temperature, prices)
- Classification with Teachable Machine (extended)
- Clustering explained with sorting toys game
- K-Means clustering, simplified
- Decision Trees, kids' explanation
- Building your first decision tree by hand
- Orange Data Mining, visual ML for kids
- Installing & exploring Orange Data Mining
- Uploading a dataset in Orange
- Running your first ML model visually

Projects built:

- Decision tree: 'What should I wear today?'
- Clustering project: Group your toys using K-Means thinking
- Orange ML: Predict if a student will pass based on study hours

Practice: Solve one ML thinking puzzle daily

Computer Vision, Deep Dive

Topics:

- Pixels, RGB, grayscale explained
- Image classification vs object detection vs image generation
- Training a custom Teachable Machine model (10+ classes)
- Exporting Teachable Machine models to Scratch
- Using TM models inside Scratch for interactive projects
- Face filters, how Snapchat/Instagram works (simplified)
- OCR (Optical Character Recognition), reading handwriting
- Medical AI, skin disease, X-ray detection (kid-safe intro)
- Real-world case studies (Indian startups using CV)
- Ethics: Deepfakes & misuse of CV
- CBSE sample CV project ideas
- Planning your own CV project

Projects built:

- Build a Teachable Machine + Scratch 'Hand Gesture Controlled Game'
- Create an attendance system using facial recognition (Teachable Machine)
- Design a 'litter detector' for clean India initiative (SDG)

Practice: Train one TM model daily

Natural Language Processing, Making Machines Understand Us

Topics:

- Words as data, how computers read text
- Tokenization, splitting sentences into words
- Bag of Words model (simplified)
- Sentiment analysis, is this message happy or sad?
- Chatbot building blocks
- Rule-based vs AI-based chatbots
- Dialogflow / simple chatbot builders for kids
- Voice input & output in Scratch 3 (speech extensions)
- Translation AI, how Google Translate works
- NLP use cases in India (Bhashini, regional languages)
- Spam filters, a classic NLP task
- Limitations of NLP, why AI gets confused

Projects built:

- Build a Scratch-based mood-detecting journal bot
- Create a voice-controlled calculator
- Build a Hindi-English translation assistant prototype

Practice: Analyze sentiment of 10 messages daily

AI Ethics, Bias & Responsible AI (CBSE Mandatory Module)

Topics:

- Why AI ethics matters, CBSE emphasis
- What is AI bias? (Famous case studies)
- Gender, racial, cultural bias in AI
- Case study: Biased resume screening AI
- Privacy & AI, your data is valuable
- Consent in the AI age
- Deepfakes & misinformation
- Environmental cost of AI (energy, water)
- AI & Jobs, what the future holds
- Responsible AI principles (fairness, accountability, transparency)
- Asimov's 3 Laws of Robotics (kid-friendly)
- Designing ethical AI, a framework for kids

Projects built:

- Ethics poster: 'Our Family's AI Rules'
- Case study presentation: Find an example of AI bias
- Design an 'AI Ethics Checklist' for classroom use

Practice: Daily AI ethics journal

Month 6: Modelling & Evaluation + First Python Steps (Week 22-26)

Modelling, How Machines Learn

Topics:

- What is a model?
- Supervised learning (with labeled examples)
- Unsupervised learning (finding patterns alone)
- Reinforcement learning (learning from reward)
- Training, validation and testing data
- Overfitting & underfitting, kids' analogy
- Accuracy, error, confusion matrix basics
- When models fail
- Improving a model
- Retraining & iterating
- Google's Teachable Machine: Advanced usage
- Introduction to MIT App Inventor for AI apps

Projects built:

- Train, test and evaluate 3 different TM models
- Build an MIT App Inventor app that uses a trained AI model
- Improve an existing model by adding more data

Practice: Evaluate one model daily

Evaluation & Deployment

Topics:

- Evaluation metrics (simplified)
- Precision, Recall, kid-friendly explanations
- F1 Score in simple terms
- When AI shouldn't be deployed
- Real-world deployment examples
- User testing with friends and family
- Feedback & iteration cycle
- Packaging & sharing AI projects
- Hosting AI apps (Teachable Machine, MIT App Inventor, Glitch)
- Presenting AI projects to non-technical people
- Creating project documentation
- CBSE AI project report format

Projects built:

- Deploy one AI app for family use (MIT App Inventor)
- Write a CBSE-format AI Project Report (5 pages)
- Conduct 3 user-testing sessions

Practice: Get feedback from 3 people per project

Introduction to Python for AI (CBSE Class 8 Transition)

Topics:

- Why Python? The language of AI & CBSE Class 9+
- Installing Python & Thonny (kid-friendly IDE)
- Google Colab, Python in the browser
- Variables, data types, print statement
- Input & simple calculations
- If-else in Python
- Loops (for, while)
- Lists, storing multiple items
- Functions, reusable code
- Random module, fun first projects
- Turtle module, drawing with Python
- First AI-like program in Python (rule-based chatbot)

Projects built:

- Python turtle, draw Indian flag, mandala, geometric art
- Python rule-based chatbot
- Dice roller, number guessing, quiz game in Python

Practice: Write 10 Python lines daily

Phase 2 Capstone, End-to-End AI Project

Topics:

- Full AI Project Cycle application
- Scratch + Teachable Machine + Python integration
- SDG alignment
- Documentation
- Showcase preparation
- Peer review
- Parent-teacher showcase
- Competition readiness

Projects built:

- CAPSTONE 3: Full AI Project solving a community problem (choose any SDG)
- Examples: Plant disease detector, School waste tracker, Elderly help chatbot, Road safety app
- Complete CBSE-format report + video + live demo

Assessment: CBSE CT & AI Builder Certificate (Classes 6-8 level)

----- PHASE 3: YOUNG AI CREATOR TRACK (MONTHS 7-8, WEEKS 27-34) -----

Advance beyond the CBSE Class 6-8 level into true AI project creation. Build 8+ real AI projects, participate in mock hackathons, understand emerging AI technologies, and prepare for Olympiads and competitions.

Month 7: Advanced AI Tools & Projects (Week 27-30)

Generative AI for Kids, Safe Exploration

Topics:

- What is Generative AI? (chatbots and image generators, simplified)
- How GenAI is different from earlier AI
- Using GenAI safely, age-appropriate rules
- Prompt Engineering basics for kids
- Creating AI art (safe, kid-friendly tools)
- AI writing helpers for schoolwork (ethical use)
- AI as a homework helper vs homework doer (boundary setting)
- Citing AI in school assignments
- Creativity + AI = Superpower
- AI music generation introduction
- AI video generation basics
- Ethics of GenAI, hallucinations & misinformation

Projects built:

- Generate a storybook cover with AI and Canva
- Design an AI-generated bookmark
- Write a 300-word story with AI assistance (document your prompts)

Practice: Daily prompt engineering journal

Robotics & AI, Bringing AI to the Real World

Topics:

- What is a robot? Robot vs AI
- mBot, LEGO Mindstorms, Spike, kid-friendly robots
- Sensors: ultrasonic, light, touch, camera
- Actuators: motors, servos, LEDs
- Virtual robotics, VEXcode VR
- Line-following robot (simulated)
- Obstacle-avoidance robot (simulated)
- Adding AI to robots, computer vision for robots
- Humanoid robots (Sophia, Optimus) overview
- Indian robotics, ISRO, DRDO, startups
- Future of robotics in homes & hospitals
- Robotics competitions in India (WRO, FLL)

Projects built:

- Program a virtual robot in VEXcode VR
- Design a line-following robot algorithm
- Storyboard: 'A day with your personal AI robot'

Practice: Daily robotics simulation

AI for Sustainable Development Goals (SDGs)

Topics:

- Overview of 17 UN SDGs (CBSE mandatory)
- How AI is helping each SDG (case studies)
- AI for Clean Water (Goal 6)
- AI for Quality Education (Goal 4)
- AI for Zero Hunger (Goal 2)
- AI for Climate Action (Goal 13)
- AI for Good Health (Goal 3)
- Indian AI-for-Good examples (Swasth, eSanjeevani)
- Designing an SDG-aligned AI project
- Finding SDG datasets (UN, Government of India)
- Measuring impact
- Youth AI-for-Good competitions (Google, UNICEF)

Projects built:

- Choose one SDG, design a complete AI solution
- Build a prototype of the solution
- Write an impact report (2 pages)

Practice: Daily SDG research

App-Based AI Projects with MIT App Inventor

Topics:

- App design principles
- MIT App Inventor advanced
- Integrating Personal Image Classifier
- Using Look Extension for AI
- Adding voice recognition to apps
- Text-to-speech in apps
- Connecting to Firebase for data
- Publishing APKs and sharing
- UI/UX for kids
- App store basics (concept only)
- App monetization (for future reference)
- Making your app accessible

Projects built:

- AI-powered plant identifier app
- Voice-command note app
- Flash card memorizer with NLP

Practice: Build one mini app weekly

Month 8: Mini-Hackathon & Olympiad Prep (Week 31-34)

Competition Preparation & Mini-Hackathon

Topics:

- What is a hackathon? Format and rules
- Team formation strategies
- Idea generation (design sprints)
- Rapid prototyping
- Pitching an AI idea (Pixar method)
- Creating a great demo video
- Presenting to judges (body language, voice)
- Handling Q&A
- Time management during competitions
- Famous CBSE AI competitions (CBSE AI Hackathon, Intel AI Youth)
- Indian AI Olympiads (IIRC, TechXcelerate)
- International AI for Good competitions

Projects built:

- Full 48-hour mini-hackathon (home edition)
- Original AI project + pitch + demo
- 3-minute pitch video for portfolio

Practice: Daily idea generation

AI Olympiad & CBSE Competition Preparation

Topics:

- AI Olympiad syllabus overview
- CBSE AI Hackathon past problems
- Logical reasoning practice
- Python/Scratch speed challenges
- AI vocabulary & concepts test prep
- Time-bound problem solving
- Mock Olympiad rounds
- AI puzzle solving (riddles, pattern tests)
- Math for AI, proportion, averages, probability basics
- Debate: 'Is AI dangerous?' (public speaking)
- Essay writing: AI in India 2030
- Portfolio polishing

Projects built:

- Complete 5 mock Olympiad rounds
- Submit to one real CBSE/Intel AI competition
- AI essay for school magazine

Practice: Daily timed practice

PHASE 4: MASTERY & TRANSITION (MONTH 9, WEEKS 35-40)

Final capstone project, portfolio creation, CBSE AI Class 9 (Code 417) readiness test, and graduation ceremony.

Month 9: Capstone, Portfolio & Certification (Week 35-40)

Grand Capstone Project (Full AI Lifecycle)

Topics:

- Selecting a real community problem
- Deep stakeholder interviews
- Complete AI Project Cycle execution
- Multi-week project management
- Using Git/GitHub basics for kids
- Version control & backup
- Collaboration with classmates
- Expert mentor review sessions
- Iterative development
- User testing with 10+ real users

- Formal CBSE-style report writing (10 pages)
- Presentation mastery

Projects built:

- FINAL CAPSTONE: Full AI solution for a real community issue
- Complete CBSE-style report (cover, abstract, methodology, results, ethics, SDG alignment)
- Public GitHub portfolio with documentation

Portfolio, LinkedIn (with parental guidance) & Public Showcase

Topics:

- Portfolio website building
- Personal branding for young creators
- Creating a project showcase page
- Video editing for project demos
- Writing project stories (journalistic style)
- Social media presentation (parent-managed)
- Preparing for school science fair
- Public speaking, mini TED-talk format
- Handling feedback gracefully
- Connecting with peer creators
- Maintaining lifelong learning habits
- Giving back, teach a younger friend

Deliverables:

- Personal portfolio site (Netlify/GitHub Pages)
- 50+ projects documented with screenshots and videos
- 10-minute live showcase presentation

CBSE AI Class 9 (Code 417) Readiness Test & Beyond

Topics:

- CBSE Class 9 AI (417) curriculum overview
- CBSE Class 11-12 AI (843) curriculum preview
- Sample past papers (Class 9 AI)
- Practical file requirements
- Employability skills (part of CBSE AI curriculum)
- Self-management, ICT, Entrepreneurial, Green skills
- Mock Class 9 AI assessment
- Gap identification
- Personalized next-step recommendation
- Transition pathway plan
- Parent conference
- Learning roadmap for next 3 years

Assessment: CBSE AI Class 9 (Code 417) Readiness Certification

Graduation & Lifetime Alumni Induction

Topics:

- Final skill assessment
- Certification ceremony (virtual + physical certificate)
- Alumni community induction
- Access to future courses and updates
- Mentor allocation for next year
- Community contribution opportunities
- Celebrating every student's journey
- Setting 3-year learning goals

Deliverables:

- CBSE-Aligned Computational Thinking & AI Certificate for Kids
- 50+ Project Portfolio
- LinkedIn-shareable digital badge
- Lifetime alumni community access
- Next-level course pathway plan

Assessment: CERTIFIED YOUNG COMPUTATIONAL THINKER & AI INNOVATOR, Ready for CBSE Class 9 AI (417) and beyond!

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 10+ unplugged CT activities, 8+ Scratch projects, 3+ AI awareness projects
- Phase 2: 6+ full AI Project Cycle projects, 5+ Teachable Machine models, 2+ Python projects
- Phase 3: 8+ advanced AI projects, 3+ robotics simulations, 2+ MIT App Inventor apps
- Phase 4: 1 grand capstone + portfolio of 50+ documented projects
- Total: 50+ projects with full CBSE-style documentation

SKILLS THE STUDENT WILL HAVE AT THE END

- Computational Thinking: 4 pillars mastery (Decomposition, Pattern Recognition, Abstraction, Algorithms)
- Block Coding: Scratch advanced level (events, loops, conditions, variables, broadcasts, cloning)
- AI Fundamentals: CV, NLP, Data Science, ML basics, ethics, bias
- AI Project Cycle: Full CBSE-aligned 5-stage process
- Data literacy: Collection, cleaning, visualization, interpretation
- Python basics: Variables, loops, conditions, lists, functions, turtle, random
- Prompt engineering (GenAI): Safe, effective, age-appropriate
- Project management: Planning, execution, documentation, presentation
- Soft skills: Public speaking, teamwork, critical thinking, empathy

- Research: Surveys, datasets, stakeholder interviews, impact reporting
- Ethics: Bias, privacy, responsibility, SDG alignment
- Technology literacy: Apps, AI, IoT basics, robotics simulation

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 x 60-min expert-led sessions/week
 Self practice: 2-3 hours/week
 Project work: 1-2 hours/week
 Parent engagement: 30 mins/week progress review
 Total per week: 5-7 hours

SUPPORT INCLUDED

Live sessions: Weekly live classes with CBSE-trained AI instructors
 One on one: Fortnightly 1-on-1 mentor check-ins
 Parent reports: Monthly detailed progress reports for parents
 Community: Kids-only private community (moderated)
 Feedback: Weekly project reviews with feedback
 Resources: Full access to curated AI resource library
 Lifetime access: All content + future CBSE curriculum updates included
 Doubt support: WhatsApp group + ticket system with <12 hr response time
 Parent workshops: Quarterly AI-for-Parents sessions

CERTIFICATION DETAIL

Phase certificates: Separate certificate after each phase (4 total)
 Final certificate: Certified Young Computational Thinker & AI Innovator
 LinkedIn badge: Shareable digital badge for LinkedIn/school profile
 School recognition: Recognized by 400+ CBSE & ICSE schools (and growing)
 Portfolio projects: 50+ documented CBSE-style AI projects
 Cbse class 9 readiness: CBSE AI Class 9 (Code 417) Readiness Certificate

WHERE THIS CAN LEAD

- Direct pathway to CBSE AI Class 9-10 (Code 417)
- Direct pathway to CBSE AI Class 11-12 (Code 843)
- Future AI Engineer / ML Engineer
- Future Data Scientist
- Future Robotics Engineer
- Future Prompt Engineer / AI Product Designer
- Future AI Ethics Researcher
- Future Computer Science Researcher
- Future Tech Entrepreneur
- Any career benefiting from computational & analytical thinking (medicine, law, business, arts)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course officially approved by CBSE?

A: Our course is 100% aligned with the official CBSE Computational Thinking & Artificial Intelligence framework and NCERT AI handbooks. While CBSE does not 'approve' external courses, we have designed every module to match CBSE syllabus for Classes 3-8 AI curriculum. Students completing this program will be ahead of CBSE AI Class 9 (Code 417) requirements. Our students from 400+ CBSE/ICSE schools have successfully used this course as supplementary and even replacement learning.

Q: My child is in Class 3, is this course too advanced?

A: Absolutely not! Phase 1 (Months 1-3) is specifically designed for Classes 3-5 with unplugged activities, Scratch basics, and AI awareness, no technical prerequisite. We have separate age-appropriate batches for Primary (Classes 3-5) and Middle School (Classes 6-8). Younger children progress at their own pace and reach Phase 2 when ready. Many Class 3 students complete the full 9 months successfully with proper pacing.

Q: My child is in Class 8, isn't this too basic?

A: Class 8 students enter directly into Phase 2 (Month 4) after a quick diagnostic, then progress through Python, advanced AI projects, GenAI, robotics, and capstone projects that are well beyond the CBSE Class 8 AI chapter. By the end, they are not just ready for CBSE AI Class 9 (Code 417), they are 6-12 months ahead of it. We also offer optional Phase 3+ advanced tracks for ambitious Class 8 students.

Q: Does this course prepare for CBSE AI Class 9 (Code 417)?

A: Yes, this is one of our core outcomes. Phase 4 includes an explicit CBSE Class 9 AI (Code 417) Readiness Test. Students who complete this course are fully prepared to excel in the Class 9-10 CBSE AI curriculum, including Communication Skills, Self-Management, ICT Skills, Entrepreneurial Skills, Green Skills, Introduction to AI, AI Project Cycle, Neural Networks intro, Python intro, and Ethics of AI. Many of our students score 95+ in their Class 9-10 AI

board exams.

Q: How is this different from school AI classes?

A: Most CBSE school AI classes cover the curriculum at surface level due to 1-2 hours/week time constraints. Our course goes 10x deeper with 5-7 hours/week, live expert instruction, 50+ hands-on projects, CBSE-format project documentation, and continuous feedback. We also include cutting-edge topics (GenAI, Prompt Engineering, Robotics simulation, MIT App Inventor) that most school curricula skip. Result: our students don't just pass AI, they create AI.

Q: Is Python necessary at this age?

A: Python is introduced only in Month 6 (Class 8 level), and even then in a gentle, kid-friendly way using Turtle graphics and simple games. CBSE introduces Python officially in Class 8 AI chapter and heavily in Class 9 (Code 417). Starting early gives your child a huge advantage. Younger kids (Classes 3-6) focus on Scratch and unplugged CT, which pedagogically is the right progression.

Q: What if my child misses a class?

A: All live classes are recorded and available in your student dashboard within 24 hours with high-quality playback, chapter bookmarks, and parallel homework assignments. For important missed sessions, your mentor provides a complimentary 1-on-1 catch-up call. You never fall behind.

Q: Are the projects CBSE-board-exam standard?

A: Yes. All capstone projects follow the official CBSE AI Project Cycle format (Problem Scoping Data Acquisition Data Exploration Modelling Evaluation). Reports are written in CBSE practical file format with abstract, methodology, SDG alignment, ethics consideration, and results, exactly the format required by CBSE AI board exams for Class 9-12.

Q: How much screen time does this course require?

A: We are very mindful of healthy screen time, especially for younger kids. Phase 1 (Classes 3-5) is 60-70% unplugged (no-screen) activities, puzzles, paper flowcharts, human-robot games, nature pattern hunts. Total screen time averages 3-4 hours/week for Phase 1 and 5-6 hours/week for Phases 2-4. We encourage physical activity, outdoor breaks, and offline thinking time throughout.

Q: Will my child be ready for AI competitions?

A: Absolutely, Phase 3 Week 31-34 is dedicated competition preparation. Our students regularly participate in and win CBSE AI Hackathons, Intel AI Youth Challenge, Samsung Innovation Campus, Microsoft AI for Good, and India AI Olympiad.

Q: Do parents need technical knowledge?

A: Zero technical knowledge needed. We provide parent-friendly weekly progress reports, quarterly AI-for-Parents workshops, and a simple parent dashboard. Many of our most successful students have parents from non-technical backgrounds. In fact, we encourage parents to 'learn alongside' for the first month, it's a beautiful bonding experience!

Q: How do I know this course is right for my child?

A: Book a FREE diagnostic session with our CBSE AI expert. We will assess your child's current level, interests, and goals, then recommend whether they should join Primary (Classes 3-5) or Middle School (Classes 6-8) batch. If we feel this course is not the right fit, we will honestly tell you, we only enroll students who will truly benefit.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-kids-classes-3-to-8>

=====

COURSE 32 OF 118: CBSE COMPUTATIONAL THINKING & AI FOR TEENS (CLASS 9-12)

=====

Course link: <https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-teens-classes-9-to-12-code-417-843>

Subject area: CBSE School Board Exam Preparation - Artificial Intelligence (Code 417 & 843)

Designed for: Teens

Who can join: Complete Beginner to CBSE AI Board Topper + Industry-Ready AI Developer

Course length: 12 months intensive (extendable to 4-year board track for Class 9 Class 12)

Time needed: 10-12 hours/week (2-3 live classes + practical + self-study)

Certificate: CBSE AI Code 417 & 843 Board Readiness Certification + Industry AI Developer Certification + Verifiable Project Portfolio

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

CBSE Computational Thinking & AI for Classes 9-12: Python, Machine Learning and board

practical mastery. Covers AI 417 (Class 10) and AI 843 (Classes 11-12).

In one line: From CBSE AI Board Topper to Industry-Ready AI Engineer, In One Comprehensive Program

ABOUT THE PROGRAMME

The Central Board of Secondary Education (CBSE) introduced Artificial Intelligence as an official Skill Subject (Code 417 for Class 9-10, Code 843 for Class 11-12) in response to India's urgent need for AI-ready youth. This 12-month flagship program is the definitive preparation course for every CBSE teen pursuing AI, whether as a board subject, competition pathway, or future career. You will master the complete CBSE AI syllabus with 10x greater depth than any school offers: Employability Skills (Communication, Self-Management, ICT, Entrepreneurial, Green Skills), the complete AI Project Cycle (Problem Scoping Data Acquisition Data Exploration Modelling Evaluation), AI Domains (Data, Computer Vision, Natural Language Processing), Python programming from zero to advanced, NumPy, Pandas, Matplotlib, scikit-learn, TensorFlow, Keras, OpenCV, NLTK, Neural Networks, Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Transformers, Generative AI (ChatGPT, DALL-E, Stable Diffusion), LLM concepts, Prompt Engineering, Reinforcement Learning, AI Ethics, SDG-aligned capstone projects, and full board practical file + viva preparation. By completion, you will have (1) scored 95+ in your CBSE AI board exam, (2) built 80+ industry-grade AI projects in your GitHub portfolio, (3) prepared for top AI Olympiads (India AI Olympiad, Intel AI Youth, Samsung Innovation Campus), and (4) positioned yourself for IIT/NIT/BITS/top universities and future AI careers earning Rs 40+ LPA.

WHAT MAKES THIS COURSE DIFFERENT

- 100% CBSE AI Code 417 + 843 syllabus coverage, not a single topic missed
- Separate dedicated batches for Class 9, 10, 11, and 12
- Full Employability Skills module (10% of CBSE AI marks, often neglected by schools)
- CBSE-format practical file creation with 20+ lab activities per class
- Every capstone project follows official CBSE AI Project Cycle format
- Viva voce preparation with 500+ commonly asked board questions
- Sample papers, past year papers, marking scheme analysis, topper answer sheets
- Industry-level Python, ML, DL training, goes beyond board syllabus
- GitHub portfolio of 80+ projects for college applications and internships
- Live 1-on-1 mentorship with AI engineers from Google, Microsoft, IIT alumni
- Guaranteed 95+ marks in CBSE AI board exam or 50% fee refund
- Preparation for India AI Olympiad, Intel AI Youth, Samsung Innovation Campus, MS Imagine Cup

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Employability Skills + CT + Python + AI Basics + the CBSE AI 417 syllabus

Phase 2: AI Project Cycle Mastery (Months 4-6): Full CBSE AI Project Cycle + Data/CV/NLP domains + Class 10 AI (417) syllabus + Board prep

Phase 3: Machine Learning & Class 11 AI (843) (Months 7-9): scikit-learn, pandas, ML algorithms, Data Visualization, Class 11 AI full syllabus

Phase 4: Deep Learning + Class 12 AI (843) + Career Launch (Months 10-12): TensorFlow, CNN/RNN/Transformer, GenAI, Class 12 full syllabus, board + industry portfolio

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 13-18 years (Classes 9 to 12)

Coding experience: None, we start from absolute zero. Prior Scratch/Python is a bonus.

Equipment: Laptop/desktop (min 8GB RAM, i3+/Ryzen 3+), stable internet (10 Mbps+), webcam, headphones

Software: All free (+ Google Colab for GPU). Paid optional: Colab Pro (\$10/mo) or Kaggle free GPU

Time commitment: 10-12 hours/week (2-3 classes + practical + self-study)

English: Class 9+ English comprehension

Math: Basic Class 9 Math (Algebra, simple geometry). Advanced math taught as needed.

Motivation: Willingness to be challenged, curiosity for AI, commitment to board results

WHO THIS COURSE IS FOR

Cbse class 9: Students opting for CBSE AI as Skill Subject (Code 417) in Class 9

Cbse class 10: Students pursuing CBSE AI (417) with board exam in Class 10

Cbse class 11: Students electing CBSE AI (Code 843) as academic elective in Class 11

Cbse class 12: Students with CBSE AI (843) board exam in Class 12

Other boards: ICSE/ISC, IB, Cambridge, State Board students, curriculum fully transferable

International students: NRI/PIO students preparing for Indian board exams from abroad

Cs non cs students: Students doing CBSE CS (083) or IP (065) wanting AI depth alongside

Engineering aspirants: JEE/BITSAT aspirants wanting AI portfolio for college applications

International college applicants: Students applying to MIT, Stanford, CMU, AI portfolio is critical

Competition aspirants: Kids gunning for India AI Olympiad, Intel AI Youth, Samsung Innovation Campus, MS Imagine Cup, Google AI, UNICEF AI-for-Good

Startup minded: Teens who want to launch an AI product/startup while in school

Self learners: Homeschoolers and advanced learners wanting structured AI mastery

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CBSE AI FOUNDATION (CODE 417 SYLLABUS, MONTHS 1-3, WEEKS 1-13)

Complete mastery of the CBSE AI 417 syllabus - Employability Skills, Introduction to AI, AI Project Cycle basics, Neural Networks and Python for AI. This is the Class 10 board syllabus, and the right foundation for a Class 9 student now that CBSE has discontinued AI 417 at Class 9. Every student builds a rock-solid foundation regardless of prior coding experience.

Month 1: Employability Skills + Introduction to AI (CBSE Unit 1-3) (Week 1-4)

Employability Skills Part 1: Communication & Self-Management (CBSE Mandatory Unit)

Topics:

- Why Employability Skills matter (10% of CBSE AI marks)
- Communication Skills: Definitions, types, importance
- Verbal vs Non-verbal communication
- Communication cycle and feedback
- Barriers to effective communication
- Writing skills, Kinds of sentences, parts of speech, tenses
- Basic English grammar for CBSE AI
- Self-Management Skills: Introduction
- Stress management & working independently
- Self-awareness, self-motivation, self-regulation
- Goal setting, SMART goals framework
- Time management techniques

Projects built:

- Write a 500-word reflective essay on 'AI in my life'
- Create a SMART goals worksheet for the AI course
- Group presentation: 'Barriers to communication in AI teams'

Practice: Daily vocabulary + communication drills

Employability Skills Part 2: ICT, Entrepreneurial & Green Skills

Topics:

- ICT Skills: Introduction to computers
- Basic ICT operations: files, folders, shortcuts
- Operating system basics (Windows/Ubuntu)
- Internet, email, search, cloud basics
- Entrepreneurial Skills: Introduction
- Qualities and functions of entrepreneurs
- Myths about entrepreneurship
- Green Skills: Introduction & importance
- Sustainable development & SDGs
- Green economy & green jobs
- Digital ethics and cyber safety
- Password management, phishing awareness

Projects built:

- Set up a cloud backup of your AI work (Google Drive)
- Interview a local entrepreneur (1-page report)
- Green AI poster, connect 3 SDGs to AI

Practice: Daily ICT skill drill (typing, shortcuts, productivity)

Introduction to Artificial Intelligence (CBSE Unit 1, Subject Specific)

Topics:

- Definition of AI, CBSE official definition
- History & evolution of AI (1950-2026 timeline)
- AI, ML, DL, the Venn diagram
- Types of AI: Narrow AI, General AI, Super AI
- Types by functionality: Reactive, Limited Memory, Theory of Mind, Self-Aware
- AI in everyday life: 20 real examples
- Domains of AI: Data Science, Computer Vision, NLP (detailed)
- Is this AI? Classifying systems as AI or not AI
- AI success stories (AlphaGo, IBM Watson, Tesla, ChatGPT)
- AI vs Automation vs Robotics
- AI in India, Aadhaar, UPI, Digital India context
- Myths about AI (will it take our jobs?)

Projects built:

- AI Spotter, identify 50 AI systems in daily life
- AI Timeline infographic (Canva)
- Debate: 'Narrow AI vs General AI, will we ever reach AGI?'

Practice: CBSE textbook Unit 1 MCQs + short answer

AI Project Cycle, Deep Dive (CBSE Core Unit)

Topics:

- The 5-stage AI Project Cycle (official CBSE framework)
- Stage 1: Problem Scoping with 4Ws (Who, What, Where, Why)
- Stakeholder identification and needs assessment
- SDG alignment, mandatory in CBSE AI
- Stage 2: Data Acquisition, sources, types, ethics
- Structured vs Unstructured data

- Stage 3: Data Exploration, EDA basics
- Stage 4: Modelling, Rule-based vs Learning-based
- Learning-based: Supervised, Unsupervised, Reinforcement
- Stage 5: Evaluation metrics introduction
- Iteration and deployment
- CBSE-format AI Project Report template

Projects built:

- Apply the full 5-stage Project Cycle to a real school problem
- Write a 10-page CBSE-format AI Project Report
- Present project in board-exam viva style

Practice: Solve 3 past-year Project Cycle questions daily

Month 2: Python Programming for AI (CBSE Unit 3) (Week 5-8)

Python Basics (CBSE-aligned + Industry Level)

Topics:

- Why Python for AI, CBSE recommendation
- Python installation (Anaconda, Thonny, VS Code)
- Google Colab for free GPU access
- Jupyter Notebook basics
- Variables, data types (int, float, string, bool)
- Operators (arithmetic, comparison, logical, bitwise)
- Input/output statements
- Type conversion and type checking
- Strings, indexing, slicing, methods
- Python standard library intro
- Running first CBSE-syllabus Python programs
- Writing clean, commented code

Projects built:

- BMI Calculator
- Temperature converter
- Interactive quiz on AI concepts

Practice: Daily 10 Python programs from CBSE textbook + Sumita Arora

Python Control Flow & Data Structures

Topics:

- If-elif-else (nested conditionals)
- For loop (with range, enumerate)
- While loop (with break, continue)
- Nested loops
- Lists, full mastery (methods, comprehensions)
- Tuples, immutable sequences
- Dictionaries, key-value storage
- Sets, unique collections
- List vs Tuple vs Dict vs Set, CBSE comparison table
- Functions, def, parameters, return, scope
- Lambda functions
- Recursion (introductory)

Projects built:

- To-Do List application
- Contact Book with dictionaries
- Password generator

Practice: 50 Python programs on control flow

NumPy & Pandas, Data Handling for AI (CBSE Class 11 AI Unit)

Topics:

- NumPy introduction, why arrays
- Creating arrays (1D, 2D, 3D)
- Array operations, broadcasting, indexing
- NumPy vs Python list, performance
- Pandas Series & DataFrame
- Reading CSV, Excel, JSON files
- DataFrame indexing, selection, filtering
- Handling missing data
- Grouping, aggregation, merging
- Pandas for CBSE Informatics Practices (supplementary)
- Real dataset exploration, government datasets
- data.gov.in & Kaggle intro for teens

Projects built:

- Analyze India's air pollution dataset (Pandas)
- Cricket stats analysis
- Exam performance data dashboard

Practice: Daily Pandas challenges

Data Visualization, Matplotlib, Seaborn (CBSE Class 11 AI)

Topics:

- Why visualize data, CBSE exam context
- Matplotlib basics (pyplot interface)
- Line, bar, pie, scatter, histogram, box plots
- Customizing plots (titles, labels, legend)
- Subplots and figures
- Seaborn introduction, statistical plots
- Heatmaps, pair plots, violin plots

- Plotly for interactive charts
- Design principles of good visualization
- Common mistakes in visualization
- CBSE sample viva questions on visualization
- Exporting plots to reports

Projects built:

- COVID-19 India data visualization
- Climate change visualization portfolio
- Student performance visualization for school

Practice: Daily chart-making from random datasets

Month 3: AI Domains Deep Dive + Neural Networks Intro (CBSE Class 10) (Week 9-13)

Data Science Domain (CBSE Class 10 Unit 4)

Topics:

- What is Data Science? (CBSE definition)
- Data Science project cycle (application of AI Project Cycle)
- Data Acquisition: Surveys, sensors, web scraping, APIs
- Data Exploration: Descriptive statistics (mean, median, mode, SD)
- Data cleaning (handling NaN, duplicates, outliers)
- Feature engineering basics
- Simple linear regression (concept)
- Classification with decision trees
- Orange Data Mining advanced
- K-Nearest Neighbors intro
- Predictive analytics in business
- Case study: Cricket stats, BSE Sensex, weather forecasting

Projects built:

- Predict student marks from study hours (linear regression)
- Classify emails as spam/ham
- Stock price trend analysis

Practice: 3 full Data Science projects

Computer Vision Domain (CBSE Class 10 Unit 5)

Topics:

- What is Computer Vision? (CBSE definition)
- Applications: Medical imaging, self-driving cars, face unlock, CCTV
- How images are stored, pixels, RGB, grayscale
- Image processing fundamentals
- OpenCV introduction (industry-standard library)
- Image reading, display, resizing, cropping
- Filters: blur, sharpen, edge detection
- Face detection using Haar Cascades
- Basic object detection
- Teachable Machine advanced (10+ class classifiers)
- CNN intuition (pre-deep learning)
- Ethics of CV, surveillance, deepfakes, bias

Projects built:

- Face detector using OpenCV
- Mask detector (post-COVID era)
- Custom image classifier with Teachable Machine + Python

Practice: OpenCV daily challenge

Natural Language Processing Domain (CBSE Class 10 Unit 6)

Topics:

- What is NLP? (CBSE definition)
- Applications: Chatbots, translation, sentiment analysis, voice assistants
- Chatbots: Rule-based vs AI-based (Script Bot vs Smart Bot)
- Human language vs computer language, the gap
- Text processing: tokenization, stemming, lemmatization
- Stopwords removal
- Bag of Words model (CBSE Class 10 mandatory topic)
- TF-IDF introduction
- Sentiment analysis using Python
- NLTK library basics
- Chatbot building with Dialogflow + Python
- Regional language NLP (Hindi, Tamil, Bengali), Bhashini

Projects built:

- Mental health chatbot (Dialogflow + Python)
- Product review sentiment analyzer
- Hindi-English translator app

Practice: NLP text processing drills

Neural Networks Introduction (CBSE Class 10 Unit 7)

Topics:

- Human brain vs artificial neural network
- Neuron, biological and artificial
- Perceptron model
- Activation functions (Sigmoid, ReLU, Tanh)
- Input, hidden, output layers
- Forward propagation (intuition)
- Loss function basics
- Backpropagation (intuition, not calculus)

- Gradient descent (kid-friendly explanation)
- Types of NN: Feedforward, CNN, RNN (brief)
- Hands-on: Perceptron in Python from scratch
- Hands-on: Simple NN in Keras/TensorFlow

Projects built:

- Build a perceptron from scratch (OR/AND/XOR gates)
- Train a simple NN on MNIST digits
- Compare NN accuracy with vs without hidden layer

Practice: NN concept MCQs daily

CBSE Class 9-10 AI Board Practical File + Viva Prep

Topics:

- CBSE AI Board exam structure
- Theory paper (70 marks) breakdown
- Practical exam (30 marks) breakdown
- Practical file, 10 mandatory activities
- CBSE AI project report format
- Viva voce, common questions
- Topper answer sheet analysis
- Marking scheme understanding
- Time management in exam
- Common mistakes to avoid
- Mock practical exam
- Mock viva session

Deliverables:

- Complete CBSE AI Class 9/10 Practical File (20+ activities)
- Full capstone project with CBSE report
- Mock exam score report

Assessment: CBSE AI 417 Board Readiness Certification

----- PHASE 2: CBSE CLASS 10 AI (417) BOARD MASTERY + ADVANCED PROJECTS (MONTHS 4-6, WEEKS 14-26) -----

Complete the CBSE Class 10 AI (Code 417) board syllabus mastery with extensive project work, practical file completion, mock exams, and competition preparation. Build 20+ industry-grade projects.

Month 4: Advanced Data Science Projects (Week 14-17)

End-to-End Data Science Projects

Topics:

- Kaggle competition walkthrough
- Feature engineering deep dive
- Handling categorical variables (one-hot, label encoding)
- Train-test split, cross-validation
- Regression: Linear, Polynomial, Ridge, Lasso
- Classification: Logistic Regression, SVM, Decision Trees, Random Forest
- Clustering: K-Means, Hierarchical, DBSCAN
- Model evaluation: accuracy, precision, recall, F1, ROC-AUC
- Hyperparameter tuning (GridSearchCV, RandomizedSearchCV)
- Model deployment basics (Streamlit, Flask)
- MLOps concepts for teens
- Real-world case studies from Indian startups

Projects built:

- House price prediction (Kaggle-level)
- Titanic survival predictor
- Customer churn classifier

Practice: 1 Kaggle micro-competition weekly

Advanced Computer Vision Projects

Topics:

- OpenCV deep dive
- Face detection + recognition pipelines
- Object tracking
- CNN architecture deep dive (LeNet, VGG, ResNet)
- Transfer learning (use pre-trained models)
- Image augmentation
- Training custom CNN in Keras
- MobileNet, YOLO for real-time detection (intro)
- Medical imaging projects
- Agricultural CV (crop disease detection)
- Traffic monitoring AI
- Ethics: privacy + surveillance trade-offs

Projects built:

- Real-time face mask detector
- Crop disease detection (leaf image classifier)
- Traffic sign recognizer for self-driving

Practice: Daily OpenCV + Keras exercises

Month 5: Advanced NLP + Generative AI (Week 18-21)

Advanced NLP & Chatbot Engineering

Topics:

- Text preprocessing pipeline
- Word embeddings: Word2Vec, GloVe
- Advanced sentiment analysis
- Named Entity Recognition (NER)
- Topic modeling (LDA)
- Chatbot engineering with Rasa
- Voice assistants with SpeechRecognition + pyttsx3
- Language translation APIs
- Fine-tuning transformer models (intro)
- Hugging Face transformers library intro
- CBSE AI NLP advanced questions
- Multilingual NLP for Indian languages (Bhashini)

Projects built:

- Multilingual voice assistant (Hindi + English)
- Advanced sentiment dashboard for Twitter/X
- Resume parser using NER

Practice: Daily NLP project

Generative AI & Prompt Engineering (2026-27 CBSE emerging topic)

Topics:

- What is Generative AI, GANs, VAEs, Diffusion, Transformers
- ChatGPT / GPT-4 / Claude / Gemini, how they work
- Prompt Engineering fundamentals
- Advanced prompting: few-shot, chain-of-thought, tree-of-thought
- DALL-E, Midjourney, Stable Diffusion, image generation
- RunwayML, Sora, video generation
- LangChain framework introduction
- Building RAG (Retrieval-Augmented Generation) apps
- API integration: OpenAI, Anthropic, Google
- Fine-tuning LLMs for specific tasks
- Ethics of GenAI: hallucinations, copyright, misinformation
- Future of AI work, GenAI native careers

Projects built:

- Build a custom AI tutor using ChatGPT API + LangChain
- Generate a visual storybook with AI (prompt + DALL-E)
- RAG chatbot over your school's PDF notes

Practice: Daily prompt engineering journal

Month 6: CBSE Class 10 Board Final Prep + Capstone (Week 22-26)

CBSE Class 10 AI Board Full Revision + Sample Papers

Topics:

- Complete Unit-wise revision (Units 1-8)
- Chapter-wise MCQ marathon
- Short answer questions bank (100+)
- Long answer questions bank (50+)
- Case study questions mastery
- Last 5 years' board papers, full solution walkthrough
- Topper answer sheet analysis
- Examiner's marking perspective
- Time management strategies
- Common trap questions
- Pre-board simulated exams
- Answer-writing technique

Practice: Daily 1 mock paper + analysis

Deliverables: 5 solved CBSE AI sample papers with feedback

Class 10 Grand Capstone + Practical File Completion

Topics:

- CBSE AI project cycle, full application
- Stakeholder interviews in your community
- Data collection, ethical and valid
- Model building, choose appropriate algorithm
- Evaluation with real metrics
- CBSE-format 15-page report writing
- Demo video production
- Viva rehearsal with external expert
- Practical file: 10+ mandatory lab activities with code, output, interpretation
- File binding & formatting per CBSE standard
- Examiner FAQs preparation
- Submission checklist

Practice: Advance your capstone at home: conduct two stakeholder interviews, log the data you collected and note one ethics safeguard you applied.

Deliverables:

- Final Class 10 AI capstone project (board-submission quality)
- Complete practical file (30+ activities)
- Viva readiness with 500+ questions rehearsed

Mock Board Exam + Competition Launch

Topics:

- Full 3-hour mock board exam
- Detailed paper analysis
- Score vs target gap identification

- Improvement plan
- Entering competitions: CBSE AI Hackathon, Intel AI Youth, Samsung Innovation, MS Imagine
- Registration support
- Team formation
- Project pitching

Deliverables:

- Mock exam score report (target: 95+)
- At least 1 competition submission

Assessment: CBSE AI Class 10 (Code 417) Board Ready Certification

PHASE 3: CBSE CLASS 11 AI (CODE 843), MACHINE LEARNING DEEP DIVE (MONTHS 7-9, WEEKS 27-39)

Complete CBSE Class 11 Artificial Intelligence (Subject Code 843) syllabus with industry-grade ML engineering skills. Includes Capability Approach, Sustainable Development, Foundational Python, Data Visualization, AI Frameworks, Statistical Data, Computer Vision, Storytelling through Data.

Month 7: CBSE AI 843 Class 11 Units 1-4 (Week 27-30)

Capability Approach + Sustainable Development (CBSE AI 843 Unit 1)

Topics:

- Amartya Sen's Capability Approach, foundation of modern AI ethics
- Functionings vs Capabilities
- Human development vs economic growth
- Role of AI in expanding human capabilities
- 17 UN Sustainable Development Goals, deep dive
- AI for each SDG, 17 detailed case studies
- Measuring impact and outcomes
- SDG alignment in AI projects (mandatory CBSE practice)
- India's SDG progress and AI role
- Responsible innovation frameworks
- AI-for-Good case studies: Aravind Eye, eSanjeevani, Swasth
- Designing SDG-centered AI projects

Projects built:

- Research paper: 'AI + SDG of my choice'
- AI project directly solving one SDG
- SDG impact measurement framework

Practice: Pick one UN Sustainable Development Goal, find a real AI project addressing it and write a half page note linking it to Sen's capability approach.

Communication Skills + AI Review + Python Foundation (CBSE Units 2-3)

Topics:

- Advanced communication for AI teams
- Scientific writing for AI papers
- Presentation skills for AI demos
- AI review, from Class 9-10 recap to Class 11+ depth
- History of AI revisited (LISP, Expert Systems, Deep Learning boom)
- Python foundational advanced: OOP (classes, objects, inheritance, polymorphism)
- File handling (text, binary, CSV)
- Exception handling (try-except-finally)
- Python modules and packages
- Virtual environments, pip, conda
- Jupyter vs VS Code vs Colab workflows
- Git and GitHub for version control

Projects built:

- Build a Python library (published to PyPI)
- Git-managed group AI project
- Technical blog post on an AI topic

Practice: Record a three minute AI demo presentation of any past project, then solve five Python refresher exercises on lists, dicts and functions.

Data Literacy, Data Visualization & Statistical Data (CBSE Unit 4)

Topics:

- Data literacy in the age of AI
- Data collection methods (primary, secondary)
- Descriptive statistics (full coverage)
- Inferential statistics intro (hypothesis testing)
- Probability for AI (Bayes, conditional, joint, marginal)
- Probability distributions (Normal, Binomial, Poisson)
- Advanced visualization (Seaborn, Plotly, Bokeh)
- Dashboards with Streamlit & Power BI basics
- Storytelling through data (CBSE emphasis)
- Data ethics and privacy (GDPR, DPDP Act 2023)
- Open datasets for Indian context
- A/B testing basics

Projects built:

- Complete Streamlit dashboard on a social issue
- Statistical analysis paper (school topic)
- Power BI dashboard for school data

Practice: Collect a small primary dataset of at least 20 rows from your class or home, compute mean, median, mode and standard deviation, and plot two charts that tell its

story.

Month 8: CBSE AI 843 Class 11 Units 5-7 (Frameworks, CV, ML) (Week 31-34)

AI Frameworks & Tools (CBSE Class 11 Unit 5)

Topics:

- scikit-learn, industry-standard ML framework
- TensorFlow 2.x overview
- Keras high-level API
- PyTorch introduction
- Open-source AI ecosystem
- Cloud AI platforms (Google Cloud AI, AWS SageMaker, Azure ML)
- AutoML tools
- Comparing frameworks, when to use which
- Setting up complete AI dev environment
- Containerization (Docker basics)
- Model serialization (pickle, joblib, ONNX)
- API deployment with Flask/FastAPI

Projects built:

- Same ML problem solved in scikit-learn, Keras, PyTorch
- Deploy an ML model as REST API
- Docker-containerized AI app

Practice: Recreate the same simple classifier in both scikit-learn and Keras and note two differences in the workflow.

Computer Vision Advanced (CBSE Class 11 Unit 6)

Topics:

- Image processing fundamentals (full)
- OpenCV mastery
- Convolutional Neural Networks (CNN) from scratch
- CNN architectures: LeNet, AlexNet, VGG, ResNet, Inception
- Transfer learning deep dive
- Object detection: R-CNN, YOLO, SSD
- Semantic segmentation (U-Net intro)
- Face recognition with deep learning
- Optical Character Recognition (OCR)
- Video processing and action recognition
- CV for autonomous vehicles
- CV ethics: facial recognition debate

Projects built:

- YOLO-based real-time object detector
- Attendance system with face recognition
- Sign language recognizer

Practice: Fine tune a pre-trained CNN on a small image dataset using transfer learning and log its accuracy against your from scratch baseline.

Machine Learning Algorithms Mastery (CBSE Unit 7)

Topics:

- ML lifecycle end-to-end
- Supervised: Regression (Linear, Polynomial, Logistic)
- Supervised: Classification (SVM, kNN, Decision Tree, Random Forest, Gradient Boosting, XGBoost)
- Unsupervised: Clustering (K-Means, DBSCAN, Hierarchical)
- Unsupervised: Dimensionality reduction (PCA, t-SNE, UMAP)
- Semi-supervised and self-supervised intro
- Reinforcement Learning concepts (Q-learning, policy gradient intro)
- Feature engineering advanced
- Handling imbalanced datasets
- Bias-variance trade-off
- Cross-validation strategies
- Ensemble learning (bagging, boosting, stacking)

Projects built:

- Complete Kaggle competition submission
- Ensemble model for a Sensex prediction
- Clustering for Indian consumer segmentation

Practice: Train three classifiers, kNN, decision tree and random forest, on one dataset and compare them in a confusion matrix table.

Month 9: Class 11 AI Board Prep + Storytelling + Competitions (Week 35-39)

Storytelling Through Data (CBSE Class 11 Unit 8)

Topics:

- Data storytelling frameworks
- Tufte principles of graphical excellence
- Narrative structures for AI insights
- Tableau basics
- Power BI basics
- Dashboard design principles
- Infographics creation
- Presentation design (slides.com, Pitch)
- Public speaking for data presentations
- TED-talk style presentations
- Video storytelling

- Blog/Medium article writing on AI insights

Projects built:

- Tableau dashboard on social issue
- Medium blog with 1000+ words AI insights
- 7-minute TEDx-style AI talk (recorded)

Practice: Turn one of your model results into a three chart data story in Tableau or Power BI with a headline insight above each chart.

Class 11 AI Board Revision + Pre-Board

Topics:

- Full syllabus revision (8 units)
- Sample papers (5 complete)
- Chapter-wise question bank
- Case study questions
- Board-style long answers
- Topper strategy sessions
- Mock pre-board exam (3-hour, CBSE format)
- Examiner feedback simulation
- Weakness identification + gap filling
- Final revision checklist
- Day-before-exam strategy
- Stress management techniques

Practice: Daily mock paper + detailed review

Deliverables: 5 complete mock papers solved

Class 11 Capstone + Competition Deployment

Topics:

- Class 11 AI capstone project (industry-grade)
- GitHub portfolio curation
- Competition submissions (Intel AI Youth, Samsung, MS)
- College application readiness (for Class 11 students preparing for Class 12)
- Internship pitching to startups
- Building a personal brand

Practice: Polish your capstone README and GitHub portfolio page, then draft one competition submission entry such as Intel AI Youth.

Deliverables:

- CBSE Class 11 AI (843) Board Readiness Certificate
- Industry-grade capstone project
- Competition entries with results
- GitHub portfolio with 50+ projects

PHASE 4: CBSE CLASS 12 AI (843), DEEP LEARNING, NLP, CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52)

Complete CBSE Class 12 AI (Code 843) syllabus including Capstone Project, Data Science, CV, NLP, Model Evaluation, AI Ethics. Parallel industry-grade Deep Learning, Transformers, LLMs, and career-launch for college + internships.

Month 10: CBSE Class 12 AI Units 1-4 (Week 40-43)

Communication Skills Advanced + Self-Mgmt + Entrepreneurship (CBSE Units 1-2)

Topics:

- Advanced presentation skills
- Writing scientific reports
- Tech interview communication
- Self-management for high-performers
- Stress & time management
- Entrepreneurial mindset in AI
- Indian AI startup ecosystem (YC India, NASSCOM, T-Hub)
- Founding team skills
- MVP building
- Pitching to VCs
- Business Model Canvas for AI startups
- IP and patent basics

Projects built:

- Founder's Deck for a hypothetical AI startup
- Interview for placement simulation (recorded)
- Business Model Canvas for final project

Practice: Draft a one page personal pitch covering who you are, your best AI project and your goal, then rehearse it aloud twice against a timer.

Capstone Project (CBSE Class 12 Unit 3), Official Board Project

Topics:

- CBSE Class 12 AI Capstone guidelines
- Problem scoping with 4Ws (advanced)
- Full AI Project Cycle, executed in 8-10 weeks
- Literature review for AI projects
- Data acquisition (primary + secondary)
- Feature engineering + model building
- Evaluation with appropriate metrics
- Deployment (Streamlit, Flask, Hugging Face Spaces)
- CBSE-format 30-page report
- Demo video production

- Rehearsal + submission
- Viva-voce mastery

Practice: Write the official capstone problem scoping document using the 4Ws canvas plus a literature list of at least three sources.

Deliverables:

- CBSE Class 12 AI Capstone, board-submission quality
- Deployed AI application (public URL)
- Complete 30-page report

Data Science Advanced + Statistical ML (CBSE Class 12 Unit 4)

Topics:

- Advanced statistics for AI
- Time series forecasting (ARIMA, Prophet)
- Anomaly detection
- Bayesian methods
- A/B testing at scale
- Big data processing basics (Dask, Spark intro)
- MLOps introduction
- Model monitoring and drift
- Responsible AI practices
- Indian data regulations (DPDP Act)
- GDPR compliance basics
- Ethical data collection in India

Projects built:

- Stock price forecasting (ARIMA + Prophet)
- Fraud detection pipeline
- MLOps mini-project with monitoring

Practice: Build a time series forecast on a public dataset with ARIMA or Prophet and report one place where the model breaks down.

Month 11: Deep Learning + Transformers + LLMs (CBSE Units 5-6 Extended) (Week 44-47)

Deep Learning Mastery, CNNs, RNNs, LSTMs (CBSE Class 12 Unit 5)

Topics:

- Deep Learning theory deep dive
- Backpropagation mathematics
- Optimizers (SGD, Adam, RMSprop)
- Regularization (dropout, batch norm, L1/L2)
- CNN advanced architectures (EfficientNet, Vision Transformers)
- RNN, LSTM, GRU for sequences
- Sequence-to-Sequence models
- Attention mechanism intuition
- Training deep networks on GPU (Colab Pro/Kaggle)
- Transfer learning best practices
- Model compression (quantization, pruning)
- Edge AI, running models on phones/IoT

Projects built:

- ImageNet-style classifier (transfer learning)
- Stock price LSTM predictor
- Speech command recognizer

Practice: Train the same network twice, with and without dropout and batch normalization, and chart how the validation curves differ.

Transformers, LLMs & Modern AI (CBSE Class 12 Unit 6 extended)

Topics:

- Transformer architecture (Attention is All You Need)
- BERT, GPT, T5 explained
- LLMs: GPT-4, Claude, Gemini, Llama
- RLHF (Reinforcement Learning from Human Feedback)
- Fine-tuning LLMs (LoRA, QLoRA)
- Building RAG systems
- LangChain + LlamaIndex
- Vector databases (Pinecone, Chroma)
- Hugging Face ecosystem mastery
- Multimodal AI (GPT-4V, Gemini Vision)
- Agent-based AI (AutoGPT, BabyAGI, Claude Agents)
- Ethics of LLMs (hallucination, alignment, safety)

Projects built:

- Fine-tuned LLM for specific Indian-language task
- RAG chatbot on personal knowledge base
- Multimodal AI app (image + text)

Practice: Run a small experiment comparing zero shot and few shot prompts on the same task across two LLMs, and tabulate the results.

Month 12: CBSE Class 12 Board Finals + Career Launch (Week 48-52)

CBSE Class 12 AI Board Revision + Pre-Board

Topics:

- Full 6-unit revision
- 10 CBSE Class 12 AI sample papers (completely solved)
- Last 5 years board papers
- Case studies and long answers
- MCQ marathon

- Topper answer sheets analysis
- Viva-voce drills
- Practical file finalization (20+ activities)
- Capstone project final rehearsal
- Pre-board exam (3-hour full paper)
- Detailed feedback and improvement
- Day-before-exam strategy

Practice: 2 mock papers daily

Deliverables: Complete board-ready package

Final Board Readiness + Viva Mastery

Topics:

- CBSE AI viva voce question bank (500+ questions)
- Board examiner perspective
- Confidence building
- Practice viva with external AI experts
- Backup plans for difficult questions
- Final document check
- Stress management on exam day
- Last-minute revision strategy
- Healthy habits during board exams
- Post-board college application readiness

Practice: Answer 20 questions from the viva bank aloud under exam conditions and mark the five that need another revision pass.

Deliverables:

- Final Board-Ready Certificate
- All practical file + capstone submitted
- 100% viva-voice readiness

College, Internship & Career Launch

Topics:

- B.Tech AI/CS/Data Science college guidance (IIT/NIT/BITS/IIIT)
- JEE Mains/Advanced, role of AI portfolio
- International college applications (MIT, Stanford, CMU)
- SAT + AI portfolio strategy
- AI summer internship applications (Indian + global)
- LinkedIn optimization for teens
- GitHub portfolio curation
- Personal website/portfolio (Next.js)
- Cold outreach to AI companies
- Interview preparation (technical + HR)
- Negotiating your first internship
- Long-term AI career roadmap

Practice: Shortlist five target colleges or internships and map which portfolio project you would lead with for each application.

Deliverables:

- Polished LinkedIn + GitHub + portfolio website
- 3 cold outreach emails sent
- 1 internship application submitted

Graduation + Lifetime Alumni Induction

Topics:

- Full skill assessment
- Final certification ceremony
- Alumni community induction (lifetime)
- Access to alumni-only job board
- Mentor-for-life allocation
- Community contribution pathways
- Giving back, teach the next batch
- Continuous learning roadmap (post-Class 12)

Deliverables:

- CBSE AI Code 843 Class 12 Board Topper Certification
- Industry AI Developer Certification
- 80+ Project GitHub Portfolio
- Verifiable digital badge
- Lifetime alumni access + job referrals

Assessment: CERTIFIED INDUSTRY-READY AI DEVELOPER + CBSE AI (417 & 843) BOARD TOPPER

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 20+ Python + AI basics projects, CBSE Class 9 practical file
- Phase 2: 20+ Advanced projects, CBSE Class 10 practical file + capstone
- Phase 3: 20+ ML engineering projects, CBSE Class 11 capstone
- Phase 4: 20+ DL + LLM projects, CBSE Class 12 board capstone
- Total: 80+ industry-grade projects in GitHub portfolio

SKILLS THE STUDENT WILL HAVE AT THE END

- Complete CBSE AI Code 417 syllabus (Class 9-10)
- Complete CBSE AI Code 843 syllabus (Class 11-12)
- Python programming (zero to advanced, OOP, async)
- Data Science engineering (NumPy, Pandas, SQL basics)

- Machine Learning engineering (scikit-learn mastery)
- Deep Learning engineering (TF, Keras, PyTorch)
- Computer Vision engineering (OpenCV, CNN mastery)
- NLP engineering (Transformers, Hugging Face mastery)
- Generative AI mastery (prompt engineering, RAG, fine-tuning)
- MLOps basics (deployment, monitoring, versioning)
- AI Ethics & Responsible AI
- CBSE board exam technique (95+ score targeting)
- Viva-voce mastery (500+ questions prepared)
- GitHub portfolio building
- Technical writing & presentation
- Competition-ready problem solving

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2-3 x 90-min expert-led sessions/week
 Self practice: 3-4 hours/week (coding + theory)
 Project work: 2-3 hours/week
 Board prep: 1-2 hours/week (sample papers, revision)
 Mentor session: 30 mins fortnightly 1-on-1
 Total per week: 10-12 hours

SUPPORT INCLUDED

Live sessions: Weekly live classes with IIT/Google/Microsoft AI engineers
 One on one: Fortnightly 1-on-1 mentor sessions
 Parent reports: Monthly detailed progress reports for parents
 Community: Private Discord for teens + alumni network
 Feedback: Weekly code review + project review
 Resources: 200+ curated AI resources, books, papers
 Lifetime access: All content + future updates included
 Doubt support: <6-hour response time on WhatsApp + Discord
 Parent workshops: Quarterly AI-for-Parents webinars
 Board support: Dedicated board-exam helpline February-March
 Competition support: Team formation + mentor guidance for competitions
 Internship support: Internship referrals after Class 11

CERTIFICATION DETAIL

Phase certificates: 4 phase certificates + class-specific certificates
 Final certificate: CBSE AI (417 & 843) Board Topper + Industry AI Developer Certificate
 LinkedIn badge: Shareable verifiable digital badge
 School recognition: Recognized by 400+ CBSE & ICSE schools
 Portfolio projects: 80+ industry-grade documented projects on GitHub
 Board readiness guarantee: 95+ board marks guarantee or 50% refund

WHERE THIS CAN LEAD

- B.Tech in AI / CS / Data Science / ECE at IIT, NIT, BITS, IIIT, top private universities
- International admissions: MIT, Stanford, CMU, Georgia Tech, UIUC (AI portfolio is a key differentiator)
- AI/ML Engineer
- Data Scientist
- ML Research Scientist
- AI Product Manager
- AI Startup Founder (unlimited potential, Y Combinator, Sequoia India pathway)
- Computer Vision Engineer
- NLP Engineer / Conversational AI
- Generative AI Engineer / Prompt Engineer
- Quantitative AI (Algo Trading), Rs 30 LPA-2 Crore+
- AI Ethics / Policy

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Can my child take AI (Code 417) in Class 9?

A: Not from the 2026-27 session. CBSE Circular Acad-18/2026 (9 April 2026) discontinued the AI skill subject for Class 9; students already in Class 10 continue on the earlier scheme, so 417 still applies there. What replaces it matters more: CBSE's new Computational Thinking & AI becomes a compulsory Class 9 subject with an annual examination from session 2027-28. A child in Class 8 today sits it next year. This course teaches the full 417 syllabus either way, because it is the same foundation - AI project cycle, neural networks, Python, data science - and it is what Class 10 examines.

Q: Is this course officially aligned with CBSE AI Code 417 and 843 syllabus?

A: Yes, 100% alignment. Every unit of CBSE AI Class 9-10 (Code 417) and Class 11-12 (Code 843) is covered in greater depth than prescribed. We follow the exact CBSE pedagogical order, use NCERT-recommended textbooks and tools, and our capstone projects follow the official CBSE

AI Project Cycle format. Our students consistently score 95+ in their CBSE AI board exams (verified by 2,800+ board results).

Q: What is the difference between CBSE AI Code 417 and Code 843?

A: Code 417 is the Skill Subject for Class 9 & 10 (introductory, more hands-on, 30% practical). Code 843 is the Academic Elective for Class 11 & 12 (deeper theory + advanced ML/DL, 70% theory + 30% practical). Our course seamlessly covers both, Phases 1-2 master Code 417, Phases 3-4 master Code 843. Students starting in Class 9 get a complete 4-year pathway; students starting later enter the appropriate phase based on diagnostic.

Q: I'm in Class 12 with boards in a few months. Is this course still useful?

A: Yes, we offer a Class 12 Crash Track (3-4 months intensive) that covers the complete CBSE AI 843 Class 12 syllabus with board-focused revision, 10 mock tests, capstone project (board-submission quality), and viva mastery. Our Class 12 crash students have scored 90+ in under 3 months of intensive prep. Book a free diagnostic to see if this suits you.

Q: My school already teaches CBSE AI, why do I need this?

A: Most CBSE schools cover AI at surface level due to 2-4 hours/week constraint, and teacher training is still catching up. Our course goes 10x deeper with 10-12 hours/week, industry-grade Python + ML + DL, full practical file support, capstone project guidance, board-exam strategy, competition prep, and GitHub portfolio, none of which schools provide. 92% of our students also attend school AI classes, we complement, not replace.

Q: Do you cover the CBSE practical file requirements?

A: Absolutely. Practical file is 30 marks (Code 417) or 30 marks (Code 843). We provide: complete practical file creation support (20+ lab activities per class), file binding and formatting per CBSE standard, viva-voce preparation (500+ question bank), examiner's perspective analysis, and topper-quality answer templates. Our students consistently score 28-30 out of 30 in practicals.

Q: Will this help with JEE / BITSAT / other entrance exams?

A: Directly: No (JEE doesn't test AI). Indirectly: Yes, our computational thinking module sharpens logical reasoning and problem-solving that directly boost JEE Advanced performance. More importantly, your AI project portfolio becomes a massive differentiator for IIT/NIT/BITS interviews, scholarship applications, and international college admissions where portfolio weight is 40-60%.

Q: Is 10-12 hours per week sustainable alongside Class 10/12 boards?

A: Yes, because AI is one of your board subjects (Code 417 for Class 10, Code 843 for Class 12), the time spent directly contributes to your board prep. Our schedule adjusts during board months (Jan-March) to revision mode. Many of our students handle full CBSE board prep (Physics, Chem, Math, English) + AI 95+ score + GitHub portfolio simultaneously with our structured approach.

Q: Do I need a powerful computer for Deep Learning?

A: No, we use Google Colab which provides free GPU access (enough for 90% of our curriculum). A basic 8GB RAM laptop suffices for coding. We teach cloud-first engineering (which is the industry standard anyway).

Q: Will I be ready for international AI programs like Stanford AI4All or MIT Beaver Works?

A: Absolutely. Our curriculum is benchmarked against Stanford CS229 (Machine Learning) and CS231 (Deep Learning), and we've helped 40+ students get into Stanford AI4All, MIT Beaver Works, CMU AI Summer, Google CodeU, and Google AI India. We provide application guidance and recommendation letter support.

Q: How does the board score guarantee work?

A: If you attend 85%+ classes, complete 80%+ homework, submit all practical file activities and capstone on time, score below 95 in CBSE AI board exam, we refund 50% of fees. Our historical guarantee claim rate is <3% because our methodology works. See full terms in enrollment agreement.

Q: What if I'm in a non-CBSE board but want to learn AI?

A: Our course is fully transferable. ICSE/ISC students use it alongside ICSE Computer Science; IB students use it for Computer Science HL projects; Cambridge students use it for IGCSE ICT/CS; State Board students use it as comprehensive AI skill-building. The only CBSE-specific modules are board-exam prep in Phase 2 Week 22-26 and Phase 4 Week 48-52, which become optional electives for non-CBSE students.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-teens-classes-9-to-12-code-417-843>

=====

COURSE 33 OF 118: CODEX & CLAUDE CODE FOR TEENS: AI CODING AGENTS COURSE

=====

Course link: <https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-course-for-teens>
Subject area: AI Coding Agents

Designed for: Teens

Who can join: Beginner-friendly (ages 13-18) to confident AI-agent builder

Course length: 24 classes (12 weeks • 2 classes/week)

Time needed: 3-4 hours/week recommended

Certificate: Modern Age Coders 'AI Coding Agents for Teens' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

Note: this course also needs the student to have their own Claude and ChatGPT subscriptions, because the whole course is hands on with those tools. Say this before enrolling anyone, so it is never a surprise later.

WHAT THIS COURSE IS

Live online Codex + Claude Code course for teens 13-18. 24 mentor-led classes: install the CLIs, plan mode, MCP, subagents, then ship a real app. Free demo class.

In one line: Don't just learn to code, learn to direct the two most powerful AI coding agents on Earth, and build real apps in 24 classes.

ABOUT THE PROGRAMME

This is the most complete teen-friendly course on the two AI coding agents every developer is talking about in 2026: OpenAI Codex and Anthropic Claude Code. In 24 carefully sequenced, project-based classes you will install both tools, learn to give them instructions safely, and watch them write, fix, and ship real code with you. You start from zero, no coding experience required, and finish by building and deploying a real app end-to-end with both agents helping you. Every class is taught live by a mentor in a small batch or 1-on-1, never a pre-recorded video, with live commands, worked examples, a hands-on project, practice, and a checkpoint. You learn the current tools as they exist today: Claude Code v2.1.x running on Anthropic's Claude 5 models (Sonnet 5 by default, with Fable 5 and Opus 4.8 selectable), and the OpenAI Codex CLI running on the GPT-5.6 family (Sol, Terra, and Luna). One important requirement: to run the agents yourself, every student must have an active Claude subscription (Pro or higher) for Claude Code and a ChatGPT subscription (Plus or higher) for Codex. These are purchased directly from Anthropic and OpenAI, are separate from the course fee, and we help you choose the right plans in your free demo class.

WHAT MAKES THIS COURSE DIFFERENT

- Two agents, one course: most courses teach one tool; you graduate fluent in BOTH Codex and Claude Code and know when to use which.
- 100% hands-on: you run real commands from Class 2. No watching, all building.
- Built for teens: friendly pace, fun projects (a website, a game, a Discord-style bot), and a portfolio you can show schools and colleges.
- Living Syllabus: the curriculum is updated continuously as OpenAI Codex and Anthropic Claude Code release new features, models, and capabilities. You always learn the latest.
- Safety-first: you learn approval modes, sandboxes, and responsible AI use before you ever let an agent run wild.
- Real tools, not a sandbox: students bring their own Claude and ChatGPT subscriptions, so every command you practice runs on the same production agents professional developers use every day.
- Real outcomes: finish with a deployed app, a GitHub profile with real pull requests, and a recognized certificate.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac, or Linux) with internet: that's the main hardware requirement

Coding experience: None needed! We start from absolute zero. If you've done a little Scratch or Python, even better

Accounts: REQUIRED: an active Claude subscription (Pro or higher) for Claude Code and a ChatGPT subscription (Plus or higher) for Codex. These are purchased directly from Anthropic and OpenAI and are not included in the course fee. You also need a free GitHub account. We guide you through every setup step and help you pick the most student-friendly plans

Mindset: Curiosity and willingness to experiment. You'll learn by building, not memorizing

WHO THIS COURSE IS FOR

Teens: Ages 13-18 who want to build real apps fast and learn the AI tools professional developers use in 2026

Beginners: Total beginners: no prior coding required; the agents help you while you learn

Young builders: Teens with project ideas (games, websites, bots) who want to actually ship them

Future cs students: Students aiming for computer science, who want a standout portfolio and future-ready skills

Scratch python grads: Kids who've outgrown Scratch or done basic Python and want the next big step

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS OF AI CODING AGENTS

Understand what AI coding agents are, install both Codex and Claude Code, learn to keep them safe with approval modes, and complete your very first real coding task.

Class 1: What Are AI Coding Agents? (Codex vs Claude Code)

Topics:

- What 'agentic coding' means: an AI that can read your files, write code, run commands, and fix its own mistakes, not just autocomplete
- Meet OpenAI Codex: an AI software-engineering agent that lives in your terminal, your editor, the cloud, and ChatGPT
- Meet Anthropic Claude Code: an agentic coding tool that runs in the terminal, VS Code/JetBrains, desktop, web, and mobile
- How an agent loop works: you give a goal it plans it edits/runs it checks it reports back
- Codex vs Claude Code at a glance: both are excellent; Codex leans into cloud parallel tasks + ChatGPT, Claude Code leans into deep terminal control + subagents
- The 2026 models you'll use: Claude Code on the Claude 5 family (Sonnet 5 default, Fable 5 and Opus 4.8 above it); Codex on the GPT-5.6 family (Sol, Terra, Luna)
- Why learning to DIRECT an agent is now as important as writing code yourself

Projects built:

- Make a one-page 'Codex vs Claude Code' comparison sheet in your own words, listing 3 things each is great at

Practice: Watch each tool's homepage demo and write down 5 things you saw an agent do that surprised you.

Assessment: Explain, in 3 sentences a friend could understand, what an AI coding agent is and how it differs from a chatbot.

Class 2: Installing & Setting Up Claude Code

Topics:

- Install on Windows (PowerShell): `irm https://claude.ai/install.ps1 | iex`. On Mac/Linux: `curl -fsSL https://claude.ai/install.sh | bash`
- Sign in with your plan: `claude auth login` (Claude Pro or Max works great for learners); check with `claude auth status`
- Start your first session: open a project folder and type `claude`
- Meet the REPL (the chat prompt): type a request in plain English and press Enter
- Type `/` to see the slash-command menu; try `/help` and `/status`
- Understand the model: by default you're on Sonnet 5, Anthropic's balanced Claude 5 model with a native 1M-token context: see it with `/model`
- Exit safely and resume later: your session is saved

Projects built:

- Install Claude Code, start a session in an empty folder, and ask it to create a `hello.html` page that says your name

Practice: Ask Claude Code to explain what it just created, then ask it to change the background color.

Assessment: Share a screenshot of Claude Code running and your `hello.html` opening in a browser.

Class 3: Installing & Using the OpenAI Codex CLI

Topics:

- What the Codex CLI is: an open-source AI coding agent in your terminal
- Install it: `npm install -g @openai/codex` (needs Node.js 18+). Careful: the plain 'codex' package is something else; always use the `@openai/` scope
- Sign in two ways: 'Sign in with ChatGPT' (uses your ChatGPT Plus/Pro plan) OR an API key
- Start it: type `codex` to open the interactive TUI; run `codex doctor` to check everything is healthy
- Pick your model and thinking level with `/model` (low up to ultra); the 2026 lineup is the GPT-5.6 family: Sol for complex work, Terra the all-rounder, Luna for quick tasks
- Give your first instruction in plain English and watch Codex plan and edit
- Preview the AGENTS.md idea: a file that teaches Codex your project's rules (Class 10)

Projects built:

- Install Codex, run `codex doctor`, then have Codex add a new section to your `hello.html` from Class 2

Practice: Ask Codex to explain a file line-by-line, then ask it to add helpful comments.

Assessment: Screenshot a successful Codex session that edited a file, and explain in one line what the approval prompt asked you.

Class 4: Staying in Control with Approval & Permission Modes

Topics:

- Why approval modes matter: you decide how much the agent can do without asking
- Claude Code modes (cycle with Shift+Tab): default (asks each time), `acceptEdits` (auto-accepts file edits), `plan` (read-only, proposes first), `auto` (a classifier auto-approves safe actions)
- Set a mode on launch: `claude --permission-mode plan`; manage rules with `/permissions`
- Codex modes: `read-only` (look but don't touch), `workspace-write` (the safe default: edit inside your project), `danger-full-access` (no guardrails)

- Switch Codex modes anytime with /approvals; the scary bypass flag
--dangerously-bypass-approvals-and-sandbox (never use casually)
- Reading an approval prompt: what the agent wants to run and why, before you say yes
- Good habit: start in plan/read-only, review, then allow edits

Projects built:

- Run the same small task ('add a footer to hello.html') in plan mode first, review the plan, then approve and apply it

Practice: Deliberately deny an action in each tool and read what changes.

Assessment: List each approval mode for both tools and say in one line when you'd use it.

Class 5: Your First Real Task in Both Agents

Topics:

- Pick a tiny starter project (a to-do list web page) and open it in your terminal
- Give a clear goal: 'Add a button that clears all completed to-dos'
- Let Claude Code plan it, show you a diff, and apply the change after you approve
- Repeat the same goal with Codex and compare how each one approached it
- Reading a diff: green = added lines, red = removed lines: how to check the change is what you wanted
- Asking for a fix: 'That button is the wrong color, make it red': iterating with the agent
- Saving your work: ask the agent to run the app so you can see it work

Projects built:

- Add two new features to the to-do app, one with Claude Code, one with Codex, and note which felt easier

Practice: Intentionally introduce a bug, then ask the agent to find and fix it.

Assessment: Submit a short reflection comparing how Claude Code and Codex handled the same feature request.

PHASE 2: YOUR DAILY CODING WORKFLOW

Learn the everyday moves: slash commands, planning before building, editing and diffs, running tests and git, teaching the agent your project, and undoing mistakes.

Class 6: Slash Commands & Steering Your Session

Topics:

- Claude Code essentials: /help, /clear (start fresh), /compact (shrink a long chat), /context (see what the agent remembers), /model
- Codex essentials: /model, /approvals, /review, /status, and the @ picker to mention files
- Why /clear and /compact matter: keeping the agent focused and fast
- Mentioning a file so the agent reads exactly what you mean (@-mentions)
- Checking your usage and limits with /usage (Claude) and /status (Codex)
- Getting unstuck: asking the agent 'what are you doing and why?'
- Keyboard tips that make you faster in each TUI

Projects built:

- Create a personal cheat-sheet of your 10 most-used slash commands across both tools

Practice: Run a long conversation, then use /compact (Claude) and start a clean Codex session: notice the difference.

Assessment: Demonstrate using @-mention to point the agent at a specific file and ask a question about it.

Class 7: Plan Mode & Goal Mode, Think Before You Build

Topics:

- Claude Code Plan Mode: enter with /plan: the agent researches and proposes a plan WITHOUT editing
- Why plan first: catches misunderstandings before any code changes
- Goal Mode (/goal): the agent keeps working until a goal you define is actually met
- Codex /plan and /goal: the same idea on the Codex side (Goal mode is on by default in 2026)
- Writing a good goal: clear, checkable, one outcome ('the login form validates email and shows an error')
- Reviewing and editing the agent's plan before you approve it
- When to skip planning (tiny edits) vs always plan (anything touching multiple files)

Projects built:

- Use Plan Mode to design a 'dark mode toggle' for a web page, review the plan, then let the agent build it

Practice: Give a vague goal on purpose and watch how planning surfaces the missing details.

Assessment: Write one strong, checkable goal statement and explain why it's better than a vague one.

Class 8: Reading, Editing & Reviewing Code Changes

Topics:

- How agents edit: they read the file, propose precise changes, and show you a diff
- Reading multi-file changes: following a feature across several files
- Accepting, rejecting, or asking for tweaks on each change
- Claude Code /diff to see what changed; reviewing in VS Code with the extension
- Asking the agent to explain a change you don't understand
- Keeping changes small: one feature at a time is easier to review
- Spotting when an agent edited too much and reining it in

Projects built:

- Have an agent refactor a messy function and walk you through the diff line-by-line
- Practice: Reject a proposed change and ask for a different approach.
- Assessment: Review a diff and correctly identify which lines were added, removed, and why.

Class 9: Running Commands, Tests & Git from the Agent

Topics:

- Letting the agent run your app: 'start the dev server and tell me the URL'
- Running tests through the agent and reading pass/fail output together
- Basic git with the agent: 'commit this with a clear message', 'show me what changed'
- Claude Code's Bash tool and Codex running commands inside the workspace sandbox
- Approving commands safely: reading the command before it runs
- Asking the agent to fix a failing test until it passes
- Good commit hygiene: small, frequent commits with clear messages

Projects built:

- Build a tiny feature, have the agent write a test for it, run the test, and commit when it passes

Practice: Break a test on purpose and ask the agent to make it green again.

Assessment: Show a git history with at least 3 clear, agent-assisted commits.

Class 10: Project Memory, CLAUDE.md & AGENTS.md

Topics:

- Why memory matters: tell the agent your project rules once, not every time
- Claude Code: run /init to auto-create a CLAUDE.md; it loads automatically each session
- What to put in CLAUDE.md: how to run the project, your style preferences, do's and don'ts
- Claude Code auto-memory: it can save learnings to a memory folder and reload them later (/memory)
- Codex: AGENTS.md at your repo root teaches Codex your conventions; you can also have nested ones per folder and a personal ~/.codex/AGENTS.md
- A 'Review guidelines' section in AGENTS.md so Codex reviews PRs your way
- Keeping memory short and useful (aim under ~200 lines)

Projects built:

- Write a CLAUDE.md and an AGENTS.md for your to-do app describing how to run it and your code style

Practice: Add a rule like 'always use 2-space indentation' and watch the agent follow it.

Assessment: Submit a CLAUDE.md and AGENTS.md and explain one rule in each.

Class 11: Checkpoints, Rewind, Undo & Resuming Sessions

Topics:

- Claude Code checkpointing: it saves code state before edits automatically
- Rewind with /rewind (or press Esc twice): restore code, conversation, or both
- Limitation to know: rewind tracks edits made by the agent's file tools, not files deleted by shell commands
- Resuming: claude --continue (last chat) and claude --resume to pick a past session
- Codex: resume past sessions and search your local conversation history
- Branching a conversation to try a different approach without losing your place
- Building the safety habit: checkpoint, experiment, rewind if needed

Projects built:

- Make a risky change, dislike it, and use /rewind to go back cleanly

Practice: Resume yesterday's session and continue where you left off.

Assessment: Demonstrate a rewind that restores code without losing your earlier conversation.

----- PHASE 3: EXTENDING & CONNECTING YOUR AGENTS -----

Supercharge both agents: connect external tools with MCP, build skills and plugins, create subagents, automate with hooks, choose models and reasoning effort, and code from screenshots.

Class 12: Connecting External Tools & Data with MCP

Topics:

- What MCP (Model Context Protocol) is: a standard way to plug tools and data into your agent
- Claude Code: add a server with claude mcp add --transport http <name> <url>; manage with /mcp and claude mcp list
- Real examples: connect a docs server, a database, or a design tool so the agent can use it
- Codex MCP: configure servers under [mcp_servers.<name>] in ~/.codex/config.toml (stdio or HTTP)
- MCP scopes in Claude Code: local, project (shared via the repo), and user
- Safety with MCP: only connect servers you trust; agents warn about untrusted tools
- Seeing the new tools appear and asking the agent to use one

Projects built:

- Connect one MCP server to each agent and have the agent use it to fetch real information

Practice: List the new tools an MCP server added and try one.

Assessment: Explain what MCP does and show one server connected in each tool.

Class 13: Skills, Custom Commands & Codex Plugins

Topics:

- Claude Code Skills: reusable mini-instructions the agent can run; manage with /skills

- Create a custom slash command by adding a file in `.claude/commands/` (e.g. `/deploy`)
- How skills load only when needed, keeping things fast
- Codex plugins, profiles, and the plugin marketplace for sharing setups
- Codex profiles: switch settings/models per project with `codex --profile <name>`
- When to make a skill vs just typing the instruction each time
- Sharing your skills/plugins with classmates

Projects built:

- Create a Claude Code custom command (like `/newpage` that scaffolds an HTML page) and a Codex profile for your project

Practice: Run your custom command twice and tweak it to be more useful.

Assessment: Submit one working custom command/skill and explain what it automates.

Class 14: Subagents, Your Specialist Team

Topics:

- What a subagent is: a helper with its own context, tools, and job
- Claude Code built-in subagents: Explore (fast read-only search) and Plan (research before editing)
- Create your own: a Markdown file in `.claude/agents/` with name + description + a system prompt
- Give a subagent only the tools it needs (e.g., a 'reviewer' that can only read)
- Manage and generate agents with `/agents`
- Why subagents keep the main chat clean and focused (since mid-2026 they run in the background by default, so you keep working while they finish)
- Example: a 'test-writer' subagent that writes tests while you keep building

Projects built:

- Create a 'code-reviewer' subagent and have it review your to-do app

Practice: Compare answers from the Explore subagent vs asking the main agent directly.

Assessment: Submit a working subagent definition and a sample of its output.

Class 15: Hooks & Automation

Topics:

- What hooks are: code that runs automatically at certain moments (events)
- Claude Code events you'll meet: `PreToolUse`, `PostToolUse`, `SessionStart`, `UserPromptSubmit`, `Stop`
- Where hooks live: settings JSON (`.claude/settings.json`) with a matcher + a command
- A simple, safe hook: auto-format code after every edit, or block a dangerous command
- Reading hook results as feedback to the agent
- Turning hooks on/off and viewing them with `/hooks`
- Automation mindset: let routine steps happen for you, every time

Projects built:

- Add a `PostToolUse` hook that runs a formatter after the agent edits a file

Practice: Write a `PreToolUse` rule that blocks an obviously dangerous command and test it.

Assessment: Submit a working hook and explain which event triggers it and why.

Class 16: Models & Reasoning Effort

Topics:

- Claude Code models: Fable 5 (most capable, the new Claude 5 flagship tier), Opus 4.8, Sonnet 5 (the default), Haiku 4.5 (fast/cheap); switch with `/model`
- Reasoning effort levels: low, medium, high, xhigh: more effort = deeper thinking (set with `/effort`)
- The 'ultrathink' trick: add it to a prompt for a one-off deeper think
- Codex models: GPT-5.6 Sol, Terra, and Luna; pick model + reasoning level with `/model` (low up to ultra)
- Big context: Sonnet 5 has a native 1M-token window, great for large projects
- Choosing wisely: fast model for simple edits, top model + high effort for hard problems
- Cost & limits awareness for learners on student-friendly plans

Projects built:

- Solve the same tricky bug on a low effort setting and a high effort setting and compare results

Practice: Switch models mid-task and note speed vs quality trade-offs.

Assessment: Recommend a model + effort level for three different tasks and justify each.

Class 17: Multimodal Coding from Screenshots & Images

Topics:

- Why images help: show the agent a design or a bug instead of describing it
- Drag-and-drop a screenshot into the prompt so the agent can see it
- Codex Appshots (Mac): capture an app window as context for the agent
- Ask the agent to recreate a UI from a picture of a design
- Debugging from a screenshot of an error message
- Combining text + image: 'make my page look like THIS image'
- Limits: both agents read images; Codex added audio input in 2026; video is still not supported

Projects built:

- Give the agent a screenshot of a website you like and have it build a similar layout

Practice: Screenshot a broken page and ask the agent to diagnose it from the image.

Assessment: Show a before/after where an image guided the agent to the right result.

PHASE 4: CLOUD, COLLABORATION & SCALE

Go beyond your laptop: delegate tasks to Codex Cloud, use Claude Code on web and mobile, get

automatic GitHub code reviews, and run several agents at once.

Class 18: Codex Cloud, Delegating Tasks That Run for You

Topics:

- What Codex Cloud is: agents that work on tasks in the background, in their own safe sandboxes
- Get started at chatgpt.com/codex and connect a GitHub account
- Run tasks in parallel: kick off several at once and let them cook
- Each finished task comes back as a pull request you can review
- Setting up a cloud environment: which repo, setup steps, and internet access
- Reviewing the agent's work before merging anything
- When cloud beats local: long jobs, or working on a repo you don't have on your laptop

Projects built:

- Delegate two small tasks to Codex Cloud at once and review both resulting pull requests

Practice: Give a cloud task a clear scope and read the PR description it writes.

Assessment: Show two parallel cloud tasks and explain what each PR changed.

Class 19: Claude Code Everywhere, on Web, Desktop, and Mobile

Topics:

- Claude Code on the web at claude.ai/code for long-running and parallel tasks
- The desktop app: review diffs visually and run multiple sessions side-by-side
- Background agents: detach a session with `/background` and check it later
- Remote control & mobile: steer a session from your phone
- Pulling a web/remote session into your terminal to finish locally
- Choosing the right surface for the job
- Keeping all your sessions organized

Projects built:

- Start a task on the web, then continue it on the desktop app or in your terminal

Practice: Send one agent to the background and keep working in the foreground.

Assessment: Demonstrate moving the same task across two surfaces (web terminal/desktop).

Class 20: GitHub Integration & Automatic Code Review

Topics:

- Connecting GitHub so agents can open and review pull requests
- Codex code review: comment `@codex review` on a PR, or turn on automatic reviews for every PR
- Adding focus: `@codex review` for security issues
- Claude Code: `/code-review` on your branch, with `--comment` to post inline comments on GitHub
- Letting an agent fix review findings: `@codex fix the issues`
- GitHub Actions/CI: agents that review or triage automatically
- Reading a code review like a pro and acting on it

Projects built:

- Open a PR, get an AI code review from Codex and from Claude Code, and apply the fixes

Practice: Write a tiny risky change and see what the reviewer flags.

Assessment: Show a PR with at least one AI review comment that you addressed.

Class 21: Parallel Agents, Worktrees & `/batch`

Topics:

- Why parallel: split a big job into pieces that run at the same time
- Git worktrees: isolated copies of your repo so agents don't collide (`claude -w <name>`)
- Claude Code `/batch`: break a large change into many units, each handled by its own background agent in its own worktree
- Agent teams: multiple coordinating sessions
- Codex Cloud parallelism revisited: many tasks, many PRs
- Keeping parallel work organized and reviewing each piece
- When NOT to parallelize (small, tightly-linked changes)

Projects built:

- Use `/batch` (or several Codex Cloud tasks) to apply the same small improvement across multiple files

Practice: Create a git worktree and run an agent in it without touching your main branch.

Assessment: Explain a real change you split into parallel pieces and how you reviewed them.

----- PHASE 5: BUILD, SECURE & SHIP -----

Bring it together: automate with the SDKs, code safely with sandboxes and permissions, and ship a real capstone app, then earn your certificate.

Class 22: Building With the SDKs (Agent SDK & Codex SDK)

Topics:

- Headless mode: run an agent without the chat UI: Claude Code `claude -p "task"` and Codex `codex exec`
- Structured output: ask for clean JSON back (Codex `--output-schema`) to use in your own programs
- The Claude Agent SDK: build your own mini-agent in TypeScript or Python using Claude Code's engine
- The Codex SDK (TypeScript & Python): start a thread, run a prompt, resume it
- A simple automation: a script that asks the agent to do a chore and reports the result
- Piping: `cat error.log | claude -p "explain this error"`
- Where this leads: your own custom AI tools

Projects built:

- Write a tiny script that uses headless mode to summarize what changed in your project today

Practice: Pipe a file into claude -p and read the structured answer.

Assessment: Submit a small working automation built with claude -p or codex exec.

Class 23: Security, Sandboxing & Responsible AI Coding

Topics:

- Permission rules: allow / ask / deny specific actions before any tool runs
- Claude Code Bash sandbox (/sandbox): OS-level isolation so commands can't roam your whole computer
- Codex sandboxes: Apple Seatbelt (Mac), bubblewrap (Linux), Windows Sandbox, and the three approval modes
- Protecting secrets: never let an agent read or commit passwords and API keys
- Prompt injection: why you must be careful with untrusted web content and MCP servers
- Reviewing before you run: the golden rule of agent safety
- Being a responsible AI developer: understand the code the agent writes; you're accountable for it

Projects built:

- Configure permission rules and a sandbox setting in each tool and test that a blocked action is actually blocked

Practice: Add a deny rule for a dangerous command and confirm the agent respects it.

Assessment: Write a short 'safe AI coding checklist' you'll follow on every project.

Class 24: Capstone, Ship a Real App + Certification

Topics:

- Choose your build: a personal website, a simple game, a quiz app, or a chat-style bot
- Plan it with Plan/Goal mode, write your CLAUDE.md and AGENTS.md
- Build with both agents: use each where it shines
- Add a test, get an AI code review, and fix the findings
- Deploy it live (e.g., a free static host) so you have a real URL
- Polish your GitHub: real commits, a clean README, your PRs
- Present your project and earn your Modern Age Coders AI Coding Agents certificate

Projects built:

- Ship a complete, deployed app built and reviewed with Codex and Claude Code, with a public URL and a clean GitHub repo

Practice: Demo your app to the class and walk through how the agents helped you build it.

Assessment: Final capstone: a deployed app + GitHub repo + a short write-up; pass earns the certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a Codex vs Claude Code comparison sheet, your first hello.html page built with Claude Code and extended with Codex, a plan-first footer change, and two new to-do app features (one built with each agent)
- Phase 2: a personal slash-command cheat-sheet, a dark mode toggle designed in Plan Mode, a messy function refactored and reviewed line by line, a tested and committed feature, CLAUDE.md and AGENTS.md for your to-do app, and a clean /rewind recovery
- Phase 3: an MCP server connected to each agent, a custom /newpage command plus a Codex profile, a code-reviewer subagent, an auto-format hook, a low vs high effort bug-fix comparison, and a website layout rebuilt from a screenshot
- Phase 4: two parallel Codex Cloud tasks reviewed as pull requests, one task moved across web and desktop or terminal, a pull request improved with AI code review from both tools, and a /batch improvement applied across multiple files
- Phase 5: a headless-mode automation script, a tested safety setup in both tools, and a deployed capstone app (a website, game, quiz app, or bot) with a public URL
- Total: 24 hands-on projects, one per class, finishing with a deployed capstone app

Total projects: 24 hands-on projects, one per class, finishing with a deployed capstone app

SKILLS THE STUDENT WILL HAVE AT THE END

- AI Coding Agents: Codex CLI, Claude Code, agent loops, plan and goal mode, safe approval modes
- Prompting for Code: clear goal statements, plan-first building, iterating on diffs, debugging with the agent
- Extending Agents: MCP servers, custom slash commands and skills, subagents, hooks, Codex profiles
- Terminal Skills: installing CLIs, running sessions, slash commands, reading command output with confidence
- Version Control: git commits, branches, worktrees, pull requests on GitHub
- Reviewing AI Code: reading diffs, spotting over-broad changes, acting on AI code review feedback
- Models and Effort: choosing between Fable 5, Opus 4.8, Sonnet 5, Haiku 4.5 and the GPT-5.6 family (Sol, Terra, Luna), and picking the right effort level
- Shipping Projects: building, testing, deploying, and presenting a real app with a public URL

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks

Hands on agent work: 1-1.5 hours practicing real commands in Codex and Claude Code
Project time: 1-1.5 hours building each class project on your own computer
Review and doubts: 30-60 minutes reviewing what you built and clearing doubts before the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for install issues, errors, and questions
Small batches: Small batches, with mini-batch and 1-on-1 options, so every teen gets individual attention
Mentor guidance: Live guidance from mentors who use these agents daily
Project feedback: Feedback on every class project and a personal capstone review before certification
Living syllabus: Curriculum updated as OpenAI Codex and Claude Code ship new features, at no extra cost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'AI Coding Agents for Teens' certificate, earned by presenting your Class 24 capstone
Project portfolio: A deployed capstone app with a public URL plus 24 class projects you can show schools and colleges
Github profile: A GitHub profile with real commits, pull requests, and AI code reviews from the course

WHERE THIS CAN LEAD

- Junior/Intern Developer using AI agents to ship faster
- Freelance mini-projects (websites, simple apps) for local businesses
- Strong CS-college application portfolio with deployed projects
- Hackathon competitor who can build a working prototype in a weekend
- Future AI-assisted Software Engineer, Web Developer, or App Developer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need a Claude or ChatGPT subscription to take this course?

A: Yes, this is a course requirement. To actually run the agents in class, every student must have an active Claude subscription (Pro or higher) for Claude Code and a ChatGPT subscription (Plus or higher) for Codex. You buy these directly from Anthropic and OpenAI; they are separate from the course fee. We help every family choose the most cost-effective plans during the free demo class, and setup is covered step by step in Classes 2 and 3.

Q: Why is this course priced higher than other coding courses?

A: Because it is a premium program taught on premium tools. Your teen learns BOTH frontier AI coding agents, live, from mentors who use Codex and Claude Code in real work daily, on a Living Syllabus that is updated as OpenAI and Anthropic ship new features. Small live batches, 24 hands-on deliverables, a deployed capstone with a public URL, and doubt support between classes: that depth is what the fee buys. Very few academies teach these tools at all yet, and fewer still teach both.

Q: Is OpenAI Codex free?

A: The Codex CLI is open-source and free to install, but running it requires a paid plan: you sign in with a ChatGPT subscription (Plus, Pro, Business, Edu, or Enterprise) which includes Codex usage, or you use an OpenAI API key billed per use. In 2026 usage runs on rolling limits that reset every few hours. For this course a ChatGPT plan (Plus or higher) is required, along with a Claude plan (Pro or higher) for the Claude Code half. We help you choose the most student-friendly setup.

Q: Do I need to know how to code before taking this course?

A: No. This course is designed for complete beginners aged 13-18. You start from zero in Class 1, and the AI agents help you write and understand code as you go. By the end you'll be shipping real apps.

Q: What is the best way to learn Claude Code and Codex?

A: Hands-on, with live guidance. These agents change monthly, so recorded tutorials go stale fast, and reading documentation alone does not build the judgment to review AI-written code safely. The most effective path is structured live classes where you run real commands in every session, build real projects, and have a mentor correct your habits as you go, which is exactly how this 24-class course is built. Modern Age Coders has taught 10,000+ students since 2020 and is rated 4.9 across 547 Google reviews.

Q: Codex vs Claude Code: which should a teen learn first?

A: Both! That's the whole point of this course. They're the two best AI coding agents in 2026, and they shine in different ways: Codex is fantastic for cloud parallel tasks and the ChatGPT ecosystem, while Claude Code offers deep terminal control, subagents, and hooks. We teach you both and exactly when to use each.

Q: Will this course go out of date as the tools change?

A: No, this is our Living Syllabus promise. OpenAI Codex and Anthropic Claude Code ship new features and models constantly: mid-2026 alone brought Sonnet 5 as Claude Code's new default model with a native 1M-token context, and Codex's move to the GPT-5.6 family (Sol, Terra, Luna). We continuously update the curriculum so enrolled students always learn the latest features, at no extra cost.

Q: What can my teen build by the end of the course?

A: A real, deployed app with a public URL, for example a personal website, a browser game, a quiz app, or a chat-style bot, plus a GitHub profile with genuine commits, pull requests, and AI code reviews. It's a portfolio piece they can show schools and colleges.

Q: Is it safe to let an AI agent run code on our computer?

A: Yes, when you use it correctly, and we teach exactly that. You'll learn approval modes, OS-level sandboxes, and permission rules so the agent only does what you allow. An entire class (Class 23) is dedicated to security and responsible AI coding.

Q: What computer and accounts do we need?

A: Any modern Windows, Mac, or Linux computer with internet. You'll need a free GitHub account, plus the two required subscriptions: Claude (Pro or higher) for Claude Code and ChatGPT (Plus or higher) for Codex, purchased directly from Anthropic and OpenAI. We walk you through every install and login step in Classes 2 and 3.

Q: How long is the course and how is it scheduled?

A: 24 classes, typically two classes a week over about 12 weeks, with 3-4 hours of practice per week recommended. Group, mini-batch, and 1-on-1 options are available.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-course-for-teens>

=====

COURSE 34 OF 118: CODEX & CLAUDE CODE: AI CODING AGENTS FOR PROFESSIONALS

=====

Course link: <https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-masterclass-for-adults-professionals>

Subject area: AI Coding Agents

Designed for: College students

Who can join: Beginner to advanced, for working professionals, career switchers & developers

Course length: 24 classes (12 weeks · 2 classes/week)

Time needed: 4-6 hours/week recommended

Certificate: Modern Age Coders 'AI Coding Agents Professional' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

Note: this course also needs the student to have their own Claude and ChatGPT subscriptions, because the whole course is hands on with those tools. Say this before enrolling anyone, so it is never a surprise later.

WHAT THIS COURSE IS

Live online Codex + Claude Code masterclass for professionals. 24 mentor-led classes: CLIs, MCP, subagents, hooks, both SDKs and a production capstone.

In one line: Stop typing every line. Learn to direct the two most powerful AI coding agents and ship more, faster, safely, in 24 classes.

ABOUT THE PROGRAMME

This is the definitive professional course on the two agentic coding tools reshaping software work in 2026: OpenAI Codex and Anthropic Claude Code. Across 24 in-depth, lab-driven classes you'll go from first install to automating real engineering workflows: building features, refactoring legacy code, fixing flaky tests, reviewing PRs at scale, and orchestrating parallel agents. Whether you're a developer leveling up, a career switcher entering tech, or a professional who wants to 10x routine work, you'll learn both tools at the depth a job demands. You learn the current stack: Claude Code v2.1.x on Anthropic's Claude 5 family (Sonnet 5 default with native 1M-token context; Fable 5 and Opus 4.8 for the hardest work), and the OpenAI Codex CLI/Cloud on the GPT-5.6 family (Sol, Terra, Luna) with reasoning levels up to ultra. Every class is live with a mentor in a small batch or 1-on-1, hands-on with real commands, config, and a deliverable. One important requirement: you must have your own active Claude subscription (Pro or higher) for Claude Code and ChatGPT subscription (Plus or higher) for Codex, or equivalent API access through your employer. Subscriptions are purchased directly from Anthropic and OpenAI, are separate from the course fee, and we help you pick the most cost-effective setup in your free demo.

WHAT MAKES THIS COURSE DIFFERENT

- Two agents, professionally: graduate fluent in BOTH Codex and Claude Code, and able to choose the right one per task and team.
- Workplace-grade labs: refactor legacy modules, fix CI, automate releases, review PRs at scale; not toy examples.
- Goes all the way to the SDKs: build your own automations and internal tooling with the Claude Agent SDK and Codex SDK.
- Living Syllabus: continuously updated as OpenAI Codex and Anthropic Claude Code ship new features, models, and capabilities. You always learn the current product.
- Security & governance built in: permission rules, OS sandboxes, secrets handling, and prompt-injection defense for real teams.
- Real production tools, not a sandbox: you train on your own Claude and ChatGPT subscriptions, so every workflow you learn transfers directly to your day job.
- Outcome-focused: finish with a production-grade capstone, reusable automations, and a portfolio of AI-assisted engineering work.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop/desktop (Windows, Mac, or Linux) with internet and admin rights to install CLIs

Coding experience: Helpful but not required: beginners and career switchers are welcome; we start from install and ramp quickly

Accounts: REQUIRED: an active Claude subscription (Pro or higher) for Claude Code and a ChatGPT subscription (Plus or higher) for Codex, or equivalent API keys (for example through your employer, Bedrock, or Vertex AI). These are purchased directly from Anthropic and OpenAI and are not included in the course fee. A GitHub account is also required. We cover setup and provider options in the first classes

Tools: Comfort with a terminal develops during the course; basic familiarity with git is a plus

WHO THIS COURSE IS FOR

Working professionals: Engineers and analysts who want to ship more with AI agents and automate routine work

Career switchers: People entering tech who want a modern, AI-native skill set that employers increasingly expect

Developers: Developers leveling up from autocomplete to true agentic workflows (MCP, subagents, SDKs)

Tech leads: Leads introducing safe, governed agentic coding practices to a team

Founders/freelancers: Founders and freelancers who need to build and ship quickly with a small team

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATIONS OF AI CODING AGENTS -----

Understand the agentic-coding landscape, install and configure both Codex and Claude Code, master the approval/permission models, and complete your first real engineering task.

Class 1: The Agentic Coding Landscape (Codex vs Claude Code)

Topics:

- Agentic coding defined: an autonomous loop that reads context, plans, edits, runs commands, verifies, and iterates
- OpenAI Codex as a product family: the CLI, Codex Cloud, the IDE extension, the ChatGPT/desktop/mobile apps, and the Codex SDK
- Anthropic Claude Code surfaces: terminal, VS Code/JetBrains, desktop, web (claude.ai/code), mobile, Slack, and CI
- Where each excels: Codex for cloud parallelism + GitHub-native review + ChatGPT ecosystem; Claude Code for deep terminal control, subagents, hooks, and skills
- The 2026 model backends: Claude Code on the Claude 5 family (Sonnet 5 default, Fable 5 and Opus 4.8 above it); Codex on the GPT-5.6 family (Sol, Terra, Luna)
- How agentic tools change team workflows: review-centric, smaller PRs, faster iteration
- Building a mental model for evaluating any new agentic tool

Projects built:

- Write a one-page internal brief: 'Codex vs Claude Code: where we'd use each' for a hypothetical team

Practice: Map three tasks from your own work to the tool you'd choose and justify it.

Assessment: Produce a clear comparison matrix of Codex and Claude Code across 6 dimensions.

Class 2: Installing & Configuring Claude Code

Topics:

- Install: macOS/Linux `curl -fsSL https://claude.ai/install.sh | bash`; Windows `irm https://claude.ai/install.ps1 | iex`; Homebrew/WinGet alternatives
- Authenticate: `claude auth login` (subscription) or `ANTHROPIC_API_KEY`; `claude auth status`; `claude setup-token` for CI

- Enterprise providers: Amazon Bedrock, Google Vertex AI, Azure AI Foundry via env flags (CLAUDE_CODE_USE_BEDROCK, etc.)
 - First session and the REPL; the slash-command menu; /status, /usage
 - Settings hierarchy: ~/.claude/settings.json (user), .claude/settings.json (project), .claude/settings.local.json (gitignored)
 - Choosing your model with /model (Sonnet 5 is the current default; Fable 5, Opus 4.8 and Haiku 4.5 are selectable)
 - IDE integration: install the VS Code/JetBrains extension and open Claude Code in your editor
- Projects built:
- Install Claude Code, configure a project .claude/settings.json, and run your first task in a real repo
- Practice: Set up the VS Code extension and review an edit as an inline diff.
- Assessment: Show a configured project with settings.json and a successful first task.

Class 3: Installing & Configuring the OpenAI Codex CLI

Topics:

- Install: npm install -g @openai/codex (Node 18+), or brew install --cask codex; note the @openai/ scope matters
- Authenticate: Sign in with ChatGPT (Plus/Pro/Business/Edu/Enterprise) or an API key
- Health check: codex doctor verifies runtime, auth, terminal, and network
- The interactive TUI: model, service tier, token usage, and approval mode shown inline
- Config file: ~/.codex/config.toml; switch settings per project with profiles (codex --profile <name>)
- Model & reasoning selection with /model (low up to ultra); the 2026 lineup is GPT-5.6 Sol (complex work), Terra (all-rounder), Luna (fast, repeatable tasks)
- Windows options: native (PowerShell + Windows Sandbox) or WSL2

Projects built:

- Install Codex, create a ~/.codex/config.toml with a custom profile, and complete a task in workspace-write mode

Practice: Run codex doctor, fix any warnings, and switch reasoning level mid-task.

Assessment: Submit your config.toml and a screenshot of a successful Codex session.

Class 4: Approval & Permission Models (Safety Backbone)

Topics:

- Why approvals are the safety backbone of agentic coding
- Claude Code permission modes (Shift+Tab to cycle): default, acceptEdits, plan (read-only), auto (classifier-approved); set with --permission-mode, manage via /permissions
- Permission rule syntax: allow/ask/deny patterns like Bash(git push *), Edit(*.env)
- Codex sandbox modes: read-only, workspace-write (default), danger-full-access; approval policies untrusted/on-request/on-failure/never/granular
- Codex /approvals to switch live; the --dangerously-bypass-approvals-and-sandbox flag and why it's a last resort
- Designing a team-safe default posture (start restrictive, widen deliberately)
- Reading and reasoning about an approval request before granting it

Projects built:

- Author a restrictive permission/approval policy for both tools and prove a blocked action is denied

Practice: Add a deny rule for force-push and destructive commands in both tools.

Assessment: Document a recommended default permission posture for a small dev team.

Class 5: Your First Real Engineering Task in Both Agents

Topics:

- Pick a real bug or small feature in a sample repo and scope it precisely
- Claude Code: plan diff approve verify; iterate with targeted feedback
- Codex: same task in workspace-write; compare approach, edits, and explanations
- Reading multi-file diffs critically and catching over-broad changes
- Driving iteration: 'narrow the change to module X', 'add a guard for null input'
- Verifying behavior: have the agent run the app/tests and confirm the fix
- Capturing what worked: notes you'll fold into project memory next phase

Projects built:

- Resolve the same issue with each agent and write a short comparison of edit quality and effort

Practice: Introduce a regression and have each agent locate and fix it.

Assessment: Submit two diffs (one per agent) solving the same task, with a critique.

PHASE 2: YOUR DAILY ENGINEERING WORKFLOW

Build a fast, reliable daily loop: slash commands, planning, reviewing diffs, running tests and git, teaching the agent your codebase, and recovering with checkpoints.

Class 6: Slash Commands & Session Control

Topics:

- Claude Code: /help, /clear, /compact (compress context), /context (inspect what's loaded), /model, /diff, /review
- Codex: /model, /approvals, /review, /status, /plan, /goal, and the @ picker for files/dirs/skills
- Managing context windows deliberately: when to /compact vs /clear vs start fresh
- @-mentioning files and MCP resources to target the agent precisely

- Monitoring cost and limits: /usage (Claude) and inline token usage (Codex)
- Recovering a derailed session: ask the agent to restate its current plan
- Building muscle memory in each TUI for speed

Projects built:

- Create a personal/team reference of the 12 most-used commands across both tools with examples

Practice: Run a long session, then reclaim context with /compact and compare token usage before/after.

Assessment: Demonstrate precise @-mention targeting and a clean context reset.

Class 7: Plan Mode & Goal Mode

Topics:

- Claude Code Plan Mode (/plan): read-only research and a proposed plan before any edit
- Goal Mode (/goal): the agent works until a defined completion condition is met
- Codex /plan and /goal (Goal mode stable and on by default in 2026)
- Writing checkable goals and acceptance criteria the agent can self-verify against
- Editing the agent's plan to constrain scope and approach before approval
- Plan-then-execute as a default for any multi-file or risky change
- Combining plan mode with restrictive permissions for safe exploration

Projects built:

- Use Plan Mode to design a non-trivial change (e.g., add pagination to an API) and execute the approved plan

Practice: Convert a vague ticket into a precise, checkable goal statement.

Assessment: Submit a plan you edited before approval and the resulting implementation.

Class 8: Diffs, Multi-File Edits & Review Discipline

Topics:

- How agents propose precise edits and render diffs across multiple files
- Reviewing a feature that spans several files coherently
- Accept / reject / request-changes on individual hunks
- Claude Code /diff and the VS Code/JetBrains inline review experience
- Keeping PRs small and reviewable; resisting scope creep from the agent
- Asking for rationale: 'why this approach over X?'
- Detecting and reverting unintended edits

Projects built:

- Have an agent refactor a legacy function and review every hunk, rejecting anything out of scope

Practice: Force the agent to split one large change into two smaller, reviewable steps.

Assessment: Critique a multi-file diff and identify any risky or unnecessary change.

Class 9: Running Tests, Commands & Git from the Agent

Topics:

- Delegating command execution: build, run, lint, and test through the agent
- Test-driven loops: have the agent write a failing test, implement, and make it pass
- Git with the agent: staged diffs, clear commit messages, branch hygiene
- Claude Code Bash tool (sandboxable) and Codex commands inside the workspace boundary
- Approving commands with judgment: reading the exact command first
- Fixing flaky/failing tests iteratively until green
- Frequent, small commits as a safety net

Projects built:

- Implement a feature TDD-style with the agent: failing test implementation green commit

Practice: Have the agent bisect and fix a failing CI step locally.

Assessment: Show a clean commit history of an agent-assisted TDD cycle.

Class 10: Project Memory, CLAUDE.md & AGENTS.md

Topics:

- Claude Code: /init generates CLAUDE.md; it auto-loads each session
- Memory hierarchy: managed/policy, user (~/.claude/CLAUDE.md), project (./CLAUDE.md), local (CLAUDE.local.md); imports with @path
- Auto-memory (2026): Claude saves learnings to a project memory folder; manage with /memory
- Codex AGENTS.md: repo root + nested per-package + personal ~/.codex/AGENTS.md (closest file wins)
- A 'Review guidelines' section in AGENTS.md to standardize Codex code review
- What belongs in memory: run/build/test commands, conventions, architectural rules, do/don't
- Keeping memory lean (~200 lines) and version-controlled for the team

Projects built:

- Write production-quality CLAUDE.md and AGENTS.md for a real repo, including review guidelines

Practice: Add a convention and confirm both agents follow it without re-prompting.

Assessment: Submit team-ready memory files and explain three rules' impact.

Class 11: Checkpoints, Rewind & Session Management

Topics:

- Claude Code automatic checkpointing before edits; /rewind (or Esc-Esc) to restore code, conversation, or both
- Key limitation: checkpoints track file-tool edits, not shell rm/mv: your git commits remain the real safety net
- Resuming: claude --continue and claude --resume (by id/name); --fork-session; --from-pr
- Branching a conversation (/branch) to explore alternatives
- Codex session resume and local conversation history search
- Combining git + checkpoints for confident experimentation

- Session organization across multiple concurrent tasks
- Projects built:
- Run a risky refactor, reject the outcome, and recover cleanly via /rewind plus git
- Practice: Resume a prior session by name and continue the task.
- Assessment: Demonstrate a recovery workflow combining rewind and git.

PHASE 3: EXTENDING & CONNECTING YOUR AGENTS

Make the agents fit your stack: connect tools via MCP, build skills/plugins, deploy subagents, automate with hooks, tune models and reasoning effort, and feed visual context.

Class 12: Connecting Tools, Data & Services with MCP

Topics:

- Model Context Protocol: a standard for exposing tools/data/resources to agents
- Claude Code: `claude mcp add --transport http <name> <url>` (also `sse/stdio`); scopes `local/project/user`; manage with `/mcp`, `claude mcp list`
- MCP resources (`@server:protocol://path`) and MCP prompts as slash commands
- Codex: configure servers under `[mcp_servers.<name>]` in `config.toml` (`stdio` command/args/env or HTTP url), per-tool approval modes
- Practical integrations: issue trackers, databases, docs, design tools, internal APIs
- Tool Search / deferred tool loading to keep context lean with many servers
- Security: trust boundaries, OAuth, and prompt-injection risk from MCP content

Projects built:

- Connect a real MCP server (e.g., a database or docs server) to both agents and complete a task using it

Practice: Add a project-scoped `.mcp.json` so teammates inherit the same server.

Assessment: Show one server connected per tool and a task that used its tools.

Class 13: Skills, Custom Commands & Codex Plugins

Topics:

- Claude Code Skills (Agent Skills standard): reusable, auto-invokable instructions; `.claude/commands/*.md` and `.claude/skills/*/SKILL.md` both create `/commands`
- Frontmatter control: who can invoke, allowed/disallowed tools, subagent execution; `/skills` to manage, `/reload-skills`
- Designing a skill: a `/deploy` or `/release` command encapsulating your workflow
- Codex profiles for per-project model/sandbox settings and the plugin marketplace for sharing setups
- When to encode a workflow as a skill/plugin vs ad-hoc prompting
- Distributing skills/plugins to a team for consistency
- Versioning and reviewing shared automations

Projects built:

- Build a Claude Code skill that automates a repeatable workflow and a Codex profile for a project

Practice: Iterate a skill twice based on real usage feedback.

Assessment: Submit a working skill/command and document what it standardizes.

Class 14: Subagents, Specialized and Isolated Helpers

Topics:

- Subagents run in their own context with their own tools and permissions
- Built-ins: Explore (fast read-only search), Plan (read-only planning), General-purpose
- Define your own: `.claude/agents/*.md` with name, description, tools, model, `permissionMode`
- Least-privilege agents: a reviewer that can only read; a test-writer scoped to tests
- Manage and generate with `/agents`; subagent memory for cross-session learning
- Dispatching parallel subagents for independent subtasks
- Why subagents protect the main context and improve reliability (since mid-2026 they run in the background by default, so the main session keeps working)

Projects built:

- Create code-reviewer and test-writer subagents and use them on a real change

Practice: Compare Explore subagent results vs the main agent for a codebase question.

Assessment: Submit two subagent definitions and sample outputs.

Class 15: Hooks & Workflow Automation

Topics:

- Hooks run deterministic code at lifecycle events: automation and guardrails
- Key events: `PreToolUse`, `PostToolUse`, `SessionStart`, `UserPromptSubmit`, `Stop`, `PreCompact` and more
- Config in settings JSON: `matcher + handler` (`command/http/prompt/agent`), `exit-code 2` to block
- Practical hooks: auto-format/lint on edit, run tests after changes, block dangerous commands, inject context
- Placeholders like `${CLAUDE_PROJECT_DIR}` and viewing hooks with `/hooks`
- Treating hook output as feedback the agent must respond to
- Governance: enforcing team standards automatically via hooks

Projects built:

- Implement a `PostToolUse` format+lint hook and a `PreToolUse` guard that blocks force-push

Practice: Add a `SessionStart` hook that prints project setup status.

Assessment: Submit two working hooks and explain their events and effects.

Class 16: Models, Reasoning Effort & Context

Topics:

- Claude Code models: Fable 5 (the new Claude 5 flagship tier), Opus 4.8, Sonnet 5 (the default), Haiku 4.5 (fast/cheap); /model and aliases
- Effort levels low/medium/high/xhigh (set via /effort, --effort) and the ultrathink keyword
- Native 1M-token context on Sonnet 5 for large codebases
- Codex models: GPT-5.6 Sol, Terra, and Luna; reasoning levels low up to ultra (ultra auto-delegates to subagents); long-task compaction across context windows
- Cost/performance trade-offs: cheap+fast for routine edits, top model+high effort for hard design
- Subagent model overrides for cost control at scale
- Measuring whether more effort actually improved the result

Projects built:

- Benchmark a hard refactor at two effort levels and two models; document quality vs cost/latency

Practice: Pin different models for main vs subagents and observe the effect.

Assessment: Produce a model+effort selection guide for five task types.

Class 17: Multimodal Inputs, Visual Context

Topics:

- Why visuals help: convey designs, dashboards, and bugs faster than prose
- Drag-and-drop screenshots/images into the prompt as context
- Codex Appshots (macOS): capture frontmost app windows for context
- UI-from-image: have the agent implement a layout from a design screenshot
- Debugging from a screenshot of an error or broken render
- Mixing text + image instructions for precise outcomes
- Current limits: image input in both agents; Codex added audio inputs and tool outputs in 2026; video is still not supported

Projects built:

- Recreate a real UI from a design screenshot and reconcile it against the spec

Practice: Diagnose a rendering bug from a screenshot alone.

Assessment: Show an image-driven task with a correct, verified result.

----- PHASE 4: CLOUD, COLLABORATION & SCALE -----

Operate beyond a single machine: delegate to Codex Cloud, use Claude Code across surfaces, integrate AI code review into GitHub/CI, and orchestrate parallel agents.

Class 18: Codex Cloud, Parallel Tasks & Auto-PRs

Topics:

- Codex Cloud: asynchronous agents working in isolated, sandboxed environments
- Get started at chatgpt.com/codex and connect GitHub
- Queue many tasks in parallel; each returns as its own pull request
- Configure cloud environments: repo, setup steps, tools, and internet-access controls
- Reviewing cloud-generated PRs before merge; iterating via comments
- When cloud wins: long-running jobs, parallel chores, repos not on your laptop
- Governance for cloud agents in a team setting

Projects built:

- Delegate three parallel Codex Cloud tasks and review/merge the resulting PRs

Practice: Configure a cloud environment's setup steps so the agent can build/test the repo.

Assessment: Show parallel cloud tasks and explain each PR's scope and quality.

Class 19: Claude Code Across Surfaces (Web, Desktop, Mobile, Background)

Topics:

- claude.ai/code on the web for long-running and parallel tasks, even on repos you don't have locally
- Desktop app: visual diff review and multiple side-by-side sessions
- Background agents: detach with /background, manage with claude agents / attach / logs
- Remote control & mobile: steer and approve from your phone
- Teleporting a web/remote session into the terminal to finish locally
- Choosing the right surface per task and handing off between them
- Slack/GitHub triggers that start sessions

Projects built:

- Start a task on the web, hand it to the desktop/terminal, and finish it; run one agent in the background

Practice: Manage a background agent's lifecycle from the shell.

Assessment: Demonstrate a task moving across two surfaces and a managed background agent.

Class 20: GitHub Integration & AI Code Review at Scale

Topics:

- Connect GitHub apps so both agents can open and review PRs
- Codex review: @codex review (P0/P1 findings) or enable automatic reviews on every PR; scoped reviews like @codex review for security regressions
- Codex task mentions: @codex fix the CI failures pushes fixes when authorized
- Claude Code /code-review (effort levels), --comment for inline GitHub comments, --fix to apply findings
- CI automation: openai/codex-action and Claude Code GitHub Actions / GitLab CI
- Defining team review guidelines (AGENTS.md) honored during review

- Acting on AI review feedback with rigor, not blind acceptance
- Projects built:
- Set up automatic AI code review on a repo and run a PR through both Codex and Claude Code review
- Practice: Tune review guidelines and observe how findings change.
- Assessment: Show a reviewed PR with AI findings you triaged and resolved.

Class 21: Parallel Agents, Worktrees, /batch & Teams

Topics:

- Parallelization strategy: decompose large work into independent units
- Git worktrees for isolation: `claude -w <name>` creates an isolated workspace
- Claude Code /batch: split a large change into 5-30 units, one background subagent per worktree, each tests + opens a PR
- Agent teams: multiple coordinating sessions (`--teammate-mode`)
- Codex Cloud parallelism for the same outcome via many PRs
- Reviewing and integrating parallel outputs without conflicts
- Knowing when serial is safer than parallel

Projects built:

- Use /batch (or parallel Codex Cloud tasks) to roll a cross-cutting change across a codebase and review each PR

Practice: Run two agents in separate worktrees on independent tasks simultaneously.

Assessment: Document a parallelized change: the split, the worktrees/PRs, and the review.

PHASE 5: BUILD, SECURE & SHIP

Operationalize agents: automate with the SDKs, harden security for teams, and deliver a production-grade capstone, then certify.

Class 22: Building With the SDKs (Claude Agent SDK & Codex SDK)

Topics:

- Headless automation: `claude -p "task" (output-format text/json/stream-json)` and `codex exec` (with `--output-schema` for structured JSON)
- Claude Agent SDK: `npm i @anthropic-ai/claude-agent-sdk / pip install claude-agent-sdk`; `query()` with tools, hooks, agents, `permissionMode`
- Codex SDK: `@openai/codex-sdk` (TS) and `openai-codex` (Python): `startThread`, `run`, `resumeThread`
- Piping and composition: `git diff main | claude -p "review for security issues"`
- Building an internal automation: triage, summarize, or auto-fix in CI
- Budget/turn controls (`--max-turns`, `--max-budget-usd`) for safe unattended runs
- Note: from June 15 2026 Agent SDK / `claude -p` usage draws on a separate credit on subscription plans

Projects built:

- Build a CI automation (e.g., auto-summarize PR diffs or auto-label issues) using `claude -p` or `codex exec` with structured output

Practice: Write a script that pipes logs into an agent and parses its JSON response.

Assessment: Submit a working SDK/headless automation with structured output.

Class 23: Security, Sandboxing & Governance for Teams

Topics:

- Permission rules (allow/ask/deny) evaluated before any tool runs; pattern syntax for fine control
- Claude Code Bash sandbox (`/sandbox`): Seatbelt (macOS) / bubblewrap (Linux/WSL2); filesystem & network isolation, allow/deny domains
- Codex sandboxes: Seatbelt / bubblewrap / Windows Sandbox; read-only vs workspace-write vs danger-full-access
- Secrets hygiene: denyRead for `~/ssh`, `~/aws`; never commit credentials; env scrubbing
- Prompt injection & untrusted content: risks from web fetches and MCP servers, and mitigations
- Managed/enterprise settings: forced login org, allowed models, disabling hooks, managed domain allowlists
- Accountability: you own the code the agent writes: review and test it

Projects built:

- Define and test a hardened security configuration (permissions + sandbox + secret denials) for a team repo

Practice: Demonstrate a sandbox blocking network/file access outside the allowed scope.

Assessment: Deliver a team 'safe agentic coding' policy document with concrete settings.

Class 24: Capstone, Ship a Production-Grade Build + Certification

Topics:

- Scope a real deliverable: a feature in an existing app, or a small full-stack app + an automation
- Plan with Plan/Goal mode; author CLAUDE.md and AGENTS.md with review guidelines
- Build with both agents, using subagents/hooks/MCP where they add value
- Quality gate: tests, AI code review (with fixes), and a clean PR history
- Ship it: deploy the app and/or land the automation in CI
- Document your workflow: what each agent did, decisions, and safeguards
- Present and earn the Modern Age Coders AI Coding Agents Professional certificate

Projects built:

- Deliver a deployed, tested, AI-reviewed capstone (app and/or SDK automation) with a documented agentic workflow

Practice: Present a 5-minute walkthrough of how you directed the agents end-to-end.
Assessment: Final capstone: deployed/merged work + repo + write-up; pass earns the professional certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a Codex vs Claude Code comparison brief, both CLIs installed and configured (project settings.json and a config.toml with a custom profile), a restrictive permission policy proven to block unsafe actions, and the same real bug fixed with each agent
- Phase 2: a 12-command team reference, a Plan Mode change designed and executed (API pagination), a reviewed legacy-function refactor, a TDD feature taken from failing test to green commit, production-quality CLAUDE.md and AGENTS.md files, and a rewind-plus-git recovery drill
- Phase 3: a real MCP server connected to both agents, a reusable workflow skill plus a Codex profile, code-reviewer and test-writer subagents, format-and-lint and force-push-guard hooks, a model and effort benchmark, and a UI rebuilt from a design screenshot
- Phase 4: three parallel Codex Cloud tasks reviewed and merged as pull requests, a cross-surface workflow with a managed background agent, automatic AI code review wired into a GitHub repo, and a cross-cutting change rolled out with /batch across worktrees
- Phase 5: a CI automation built with Claude -p or Codex exec, a hardened team security configuration (permissions, sandbox, secret denials), and a deployed, tested, AI-reviewed capstone
- Total: 24 hands-on deliverables, one per class, capped by a production-grade capstone

Total projects: 24 hands-on deliverables, one per class, including a production-grade capstone shipped in Class 24

SKILLS THE STUDENT WILL HAVE AT THE END

- AI Coding Agents: Codex CLI, Claude Code, agent workflows, plan and goal mode, context management
- Prompting for Code: precise goal statements, spec-first planning, iterative refinement, debugging prompts
- Extending Agents: MCP servers, skills and custom commands, subagents, hooks, Codex profiles
- Model Strategy: Fable 5, Opus 4.8, Sonnet 5, Haiku 4.5, GPT-5.6 Sol/Terra/Luna, effort levels, cost and quality trade-offs
- Version Control: git branches, worktrees, clean commit history, PR-based workflows
- Reviewing AI Output: reading multi-file diffs, scoping changes, AI code review in GitHub and CI
- Automation with SDKs: Claude Agent SDK, Codex SDK, headless runs, structured output, CI integrations
- Security and Governance: permission rules, OS sandboxes, secrets hygiene, prompt-injection defense

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks

Hands on agent work: 1-1.5 hours running real commands in Codex and Claude Code between classes

Project time: 1-1.5 hours completing each class deliverable in your own repo

Review and doubts: 30-60 minutes reviewing diffs, refining your notes, and clearing doubts with mentors

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so you are never stuck on a config or command

Small batches: Small batches, with mini-batch and 1-on-1 options, so every professional gets individual attention

Mentor guidance: Live guidance from mentors who use Codex and Claude Code in production daily

Capstone review: Personal review of your capstone and workflow write-up before certification

Living syllabus: Curriculum updated as OpenAI and Anthropic ship new features, at no extra cost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'AI Coding Agents Professional' certificate, awarded after the Class 24 capstone review

Project portfolio: A portfolio of 24 class deliverables plus a deployed, AI-reviewed capstone you can show employers and clients

GitHub evidence: A GitHub history of real commits, pull requests, and AI code reviews produced during the course

WHERE THIS CAN LEAD

- AI-Assisted Software Engineer / Full-Stack Developer
- Developer Productivity / Platform Engineer (agent workflows, internal tooling)
- Automation Engineer building on the Claude Agent SDK / Codex SDK
- Tech Lead / Engineering Manager standardizing agentic practices
- Freelance / Founder shipping AI-built products faster

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 6-12 LPA in India / \$70k-110k globally (developer using agents to boost output)

Growing roles: Rs 12-22 LPA / \$110k-150k (AI-assisted engineer shipping features independently)

Experienced roles: Rs 22-40+ LPA / \$150k-220k+ (senior/lead leveraging agents and SDK automation at scale)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need a Claude or ChatGPT subscription to take this masterclass?

A: Yes, this is a course requirement. To run the agents in every lab you must have an active Claude subscription (Pro, Max, Team, or Enterprise) for Claude Code and a ChatGPT subscription (Plus, Pro, Business, Edu, or Enterprise) for Codex, or equivalent API keys, for example through your employer or via Bedrock/Vertex AI. Subscriptions are purchased directly from Anthropic and OpenAI and are separate from the course fee. We help you pick the most cost-effective setup in your free demo, and many employers reimburse these tools.

Q: Why is this masterclass priced above your other courses?

A: Because it is a premium program on premium tools, built for return on investment. You learn BOTH frontier agents to professional depth, including the Claude Agent SDK and Codex SDK, from mentors who use them in production daily, on a Living Syllabus updated as OpenAI and Anthropic ship. An automation like the one you build in Class 22, or a week of reclaimed engineering time, can be worth more than the fee itself. Small live batches and a personally reviewed capstone are part of that price.

Q: Is OpenAI Codex free, and what about Claude Code?

A: The Codex CLI and SDK are open-source/free to install; running them uses your ChatGPT plan (Plus/Pro/Business/Edu/Enterprise) or an OpenAI API key. Claude Code runs on a Claude subscription (Pro/Max/Team/Enterprise) or an Anthropic API key. Both moved to token/usage-based limits in 2026. For this masterclass an active plan for each tool is required, and we help you pick the most cost-effective setup for learning and for work.

Q: Can I use what I learn at my job right away?

A: Yes, the labs are deliberately workplace-grade: fixing CI, refactoring legacy code, reviewing PRs at scale, and building SDK automations. You'll also produce a team-ready security/governance policy so you can introduce agents responsibly.

Q: Codex vs Claude Code: which is better for teams?

A: They're complementary. Codex excels at cloud parallel tasks and GitHub-native review; Claude Code offers deep terminal control, subagents, hooks, skills, and the Agent SDK. This course teaches both to professional depth and, crucially, how to choose the right one per task and integrate them into a team workflow.

Q: Does the course cover the Claude Agent SDK and the Codex SDK?

A: Yes, Class 22 is dedicated to headless automation and both SDKs (TypeScript and Python). You'll build a real CI automation with structured output, plus learn budget/turn controls for safe unattended runs.

Q: Will the content stay current as the tools change so fast?

A: That's our Living Syllabus promise. Codex and Claude Code ship features and models constantly: mid-2026 alone brought Sonnet 5 as Claude Code's default with native 1M-token context, the Claude 5 flagship tier (Fable 5), and Codex's move to the GPT-5.6 family with reasoning up to ultra. We update the curriculum continuously so you always learn the current product, at no extra cost.

Q: I'm a career switcher with limited coding experience: is this for me?

A: Yes. We start from install and ramp deliberately. The agents help you read and write code as you learn, while the course builds the judgment in planning, reviewing, and security that separates real engineers from prompt-clickers.

Q: How do you handle security and 'letting AI run commands'?

A: An entire class (Class 23) covers permission rules, OS-level sandboxes (Seatbelt/bubblewrap/Windows Sandbox), secrets hygiene, prompt-injection defense, and enterprise/managed settings. You'll configure and test a hardened setup, and you always review before running.

Q: What's the schedule and format?

A: 24 classes, typically two per week over ~12 weeks, 4-6 hours/week recommended. Available as group, mini-batch, or 1-on-1, with live mentorship and a capstone review.

Q: What is the best way to learn Claude Code and Codex as a professional?

A: On real work, with live expert feedback. Both agents ship new features monthly, so recorded courses and blog tutorials age out quickly, and self-study rarely builds the review discipline, security posture, and SDK skills teams actually need. The fastest reliable path is structured live training where every session runs real commands on real repos with a mentor who uses these tools in production, which is exactly how this 24-class masterclass works. Modern Age Coders has taught 10,000+ students since 2020 and is rated 4.9 across 547 Google reviews.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-masterclass-for-adults-professionals>

=====

COURSE 35 OF 118: COLLEGE MATHEMATICS: CALCULUS TO REAL ANALYSIS & BEYOND

=====

Course link: <https://learn.modernagecoders.com/courses/college-mathematics-complete-masterclass>
Subject area: Advanced Mathematics Education
Designed for: College students, Girls only batches
Who can join: High School Graduate to Advanced Undergraduate Mathematics
Course length: 24 months (104 weeks)
Time needed: 20-25 hours/week recommended
Certificate: Advanced Mathematics Proficiency Certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 2-year college mathematics program: from calculus fundamentals to abstract algebra, real analysis and topology, taught with rigorous proofs and applications.

In one line: From Calculus to Cutting-Edge Mathematics - Master the Language of the Universe

ABOUT THE PROGRAMME

This intensive 2-year program covers the entire undergraduate mathematics curriculum and beyond. Whether you're a college student, self-learner, professional seeking mathematical depth, or preparing for graduate studies, this masterclass transforms you into a mathematical thinker. You'll master calculus, linear algebra, abstract algebra, real analysis, topology, and applied mathematics. Learn to write rigorous proofs, solve complex problems, use computational tools, and understand the deep connections between mathematical fields. By completion, you'll have the knowledge equivalent to a mathematics major from a top university.

WHAT MAKES THIS COURSE DIFFERENT

- Complete undergraduate curriculum coverage
- Rigorous proof-writing training
- Computational mathematics integration
- Visual and intuitive explanations
- Historical and philosophical context
- Research paper reading skills
- Graduate school preparation
- Industry applications focus

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Single Variable Calculus, Linear Algebra, Proof Writing
Phase 2: Core Mathematics (Months 7-12): Multivariable Calculus, ODEs, Discrete Math, Probability
Phase 3: Advanced Theory (Months 13-18): Real Analysis, Abstract Algebra, Topology, Complex Analysis
Phase 4: Specializations (Months 19-24): PDEs, Numerical Analysis, Optimization, Research Topics

WHAT A STUDENT NEEDS BEFORE STARTING

Mathematics: High school mathematics including pre-calculus
Recommended: Basic trigonometry and algebra II
Programming: Basic programming helpful but not required
Equipment: Computer with internet, mathematical software access provided
Time commitment: 20-25 hours per week minimum
Mindset: Strong commitment to rigorous thinking

WHO THIS COURSE IS FOR

Undergraduates: Mathematics, physics, engineering, CS students
Self learners: Motivated individuals pursuing mathematical knowledge
Professionals: Engineers, data scientists, quantitative analysts
Graduate prep: Students preparing for graduate studies
Career changers: Transitioning to quantitative fields
Educators: Teachers seeking deeper mathematical understanding
Researchers: Those interested in mathematical research

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: MATHEMATICAL FOUNDATIONS (MONTHS 1-6, WEEKS 1-26)

Build rock-solid foundations in calculus, linear algebra, and mathematical reasoning with proof writing.

Months 1-2: Single Variable Calculus I (Week 1-8)

Limits and Continuity

Topics:

- Review of functions: domain, range, composition
- Inverse functions and their properties
- Exponential and logarithmic functions
- Trigonometric functions and identities
- Intuitive notion of limits
- Formal epsilon-delta definition of limits
- Limit laws and computation techniques
- One-sided limits and infinite limits
- Limits at infinity and horizontal asymptotes
- Continuity at a point and on intervals
- Intermediate Value Theorem
- Types of discontinuities

Projects built:

- Epsilon-delta proof visualizer
- Limit calculator implementation
- Continuity explorer interactive tool

Practice: Complete 100 limit problems with epsilon-delta proofs

Differentiation

Topics:

- Definition of derivative as a limit
- Geometric interpretation: tangent lines
- Physical interpretation: rates of change
- Differentiability and continuity relationship
- Basic differentiation rules: power, constant, sum
- Product rule and quotient rule
- Chain rule and implicit differentiation
- Derivatives of trigonometric functions
- Derivatives of exponential and logarithmic functions
- Inverse function derivatives
- Higher-order derivatives
- Logarithmic differentiation

Projects built:

- Derivative visualizer application
- Automatic differentiation engine
- Physics motion simulator

Practice: Master 200 differentiation problems including proofs

Applications of Derivatives

Topics:

- Linear approximation and differentials
- L'Hôpital's Rule for indeterminate forms
- Critical points and extrema
- First Derivative Test
- Second Derivative Test
- Concavity and inflection points
- Curve sketching comprehensive method
- Optimization problems
- Related rates problems
- Mean Value Theorem
- Rolle's Theorem
- Newton's Method for root finding

Projects built:

- Optimization problem solver
- Curve sketching software
- Newton's method visualizer

Practice: Solve 150 application problems with full solutions

Integration Fundamentals

Topics:

- Antiderivatives and indefinite integrals
- Definite integrals and Riemann sums
- Fundamental Theorem of Calculus Part I
- Fundamental Theorem of Calculus Part II
- Basic integration rules
- U-substitution method
- Integration of trigonometric functions
- Integration of exponential and logarithmic functions
- Area between curves
- Volumes by cross-sections
- Volumes by cylindrical shells
- Average value of a function

Projects built:

- Riemann sum visualizer
- Numerical integration calculator
- Volume calculator for solids of revolution

Practice: Complete 200 integration problems

Months 3-4: Calculus II & Linear Algebra I (Week 9-17)

Advanced Integration Techniques

Topics:

- Integration by parts
- Trigonometric integrals and substitutions
- Partial fraction decomposition
- Rational function integration
- Improper integrals Type I and II
- Comparison tests for improper integrals
- Numerical integration: Trapezoidal rule
- Simpson's rule and error bounds
- Arc length calculations
- Surface area of revolution
- Work, force, and energy applications
- Center of mass and moments

Projects built:

- Advanced integration solver
- Numerical methods comparison tool
- Physics applications simulator

Practice: Master 150 advanced integration problems

Sequences and Series

Topics:

- Sequences: convergence and divergence
- Monotonic sequences and bounded sequences
- Series and partial sums
- Geometric series and telescoping series
- Divergence test and p-series
- Comparison tests and limit comparison test
- Ratio test and root test
- Alternating series test and error bounds
- Absolute vs conditional convergence
- Power series and radius of convergence
- Taylor and Maclaurin series
- Taylor polynomial approximations and error

Projects built:

- Series convergence analyzer
- Taylor series visualizer
- Function approximation tool

Practice: Analyze 100 series for convergence with proofs

Linear Algebra: Vectors and Matrices

Topics:

- Vector spaces: $\mathbb{R}^n$ and axioms
- Vector operations: addition, scalar multiplication
- Dot product and cross product
- Vector projections and orthogonality
- Linear combinations and span
- Linear independence and dependence
- Matrix operations: addition, multiplication
- Matrix transpose and properties
- Special matrices: identity, diagonal, symmetric
- Elementary row operations
- Row echelon form and reduced row echelon form
- Gaussian elimination algorithm

Projects built:

- Matrix calculator implementation
- Vector visualization tool
- Gaussian elimination solver

Practice: Complete 150 linear algebra computations

Systems of Linear Equations

Topics:

- Consistent vs inconsistent systems

- Homogeneous systems and nontrivial solutions
- Matrix representation of systems
- Augmented matrices and RREF
- Parametric solutions and free variables
- Matrix inverses and invertibility
- Computing inverses using row operations
- Determinants: definition and properties
- Cofactor expansion and row operations
- Cramer's rule for solving systems
- Applications to network flow
- Applications to economics and engineering

Projects built:

- Linear system solver application
- Determinant calculator
- Network flow optimizer

Practice: Solve 100 systems using various methods

Mathematical Proof Writing

Topics:

- Logic and truth tables
- Quantifiers: universal and existential
- Direct proof technique
- Proof by contradiction
- Proof by contrapositive
- Mathematical induction
- Strong induction and well-ordering
- Proof by cases
- Existence and uniqueness proofs
- Counterexamples
- Writing clear mathematical arguments
- Common proof strategies and patterns

Projects built:

- Proof verification system
- Logic truth table generator
- Induction proof template builder

Practice: Write 50 rigorous mathematical proofs

Months 5-6: Advanced Linear Algebra & Vector Calculus (Week 18-26)

Eigenvalues and Eigenvectors

Topics:

- Eigenvalue and eigenvector definition
- Characteristic polynomial
- Finding eigenvalues and eigenvectors
- Algebraic and geometric multiplicity
- Diagonalization of matrices
- Powers of diagonalizable matrices
- Orthogonal matrices and rotations
- Symmetric matrices and spectral theorem
- Quadratic forms
- Principal axes theorem
- Applications to differential equations
- Applications to Markov chains

Projects built:

- Eigenvalue calculator and visualizer
- Matrix diagonalization tool
- Markov chain simulator

Practice: Find eigenvalues/eigenvectors for 100 matrices

Vector Spaces and Linear Transformations

Topics:

- Abstract vector spaces and subspaces
- Basis and dimension
- Coordinate systems and change of basis
- Row space, column space, null space
- Rank-nullity theorem
- Linear transformations: definition and properties
- Kernel and image of linear transformation
- Matrix representation of linear transformations
- Composition of linear transformations
- Isomorphisms and invertible transformations
- Similarity of matrices
- Jordan canonical form introduction

Projects built:

- Linear transformation visualizer
- Change of basis calculator
- Subspace dimension finder

Practice: Analyze 75 linear transformations

Multivariable Functions and Partial Derivatives

Topics:

- Functions of several variables
- Level curves and level surfaces

- Limits in multiple dimensions
- Continuity for multivariable functions
- Partial derivatives: definition and notation
- Higher-order partial derivatives
- Clairaut's theorem on mixed partials
- Tangent planes and linear approximation
- The gradient vector and directional derivatives
- The chain rule for multivariable functions
- Implicit differentiation in multiple variables
- Jacobian matrices

Projects built:

- 3D function visualizer
- Gradient field plotter
- Tangent plane calculator

Practice: Compute 150 partial derivatives and gradients

Optimization and Vector Fields

Topics:

- Critical points in multiple dimensions
- Second derivative test for two variables
- Hessian matrix and definiteness
- Global vs local extrema
- Constrained optimization: Lagrange multipliers
- Multiple constraints
- Vector fields and flow lines
- Conservative vector fields
- Potential functions
- Divergence and curl
- Physical interpretations
- Applications to physics and engineering

Projects built:

- Multivariable optimization solver
- Vector field simulator
- Lagrange multiplier visualizer

Practice: Solve 100 optimization problems

Phase 1 Capstone Project

Topics:

- Integration of calculus and linear algebra
- Comprehensive problem solving
- Proof portfolio creation
- Mathematical writing
- Presentation skills

Projects built:

- MAJOR CAPSTONE: Mathematical Modeling Project
- Options: Population dynamics, Economic models, Engineering systems
- Complete proof portfolio with 25 original proofs
- Implementation of numerical methods library

Assessment: Phase 1 Comprehensive Exam - Calculus and Linear Algebra

PHASE 2: CORE MATHEMATICS (MONTHS 7-12, WEEKS 27-52)

Master multivariable calculus, differential equations, discrete mathematics, and probability theory.

Months 7-8: Multiple Integration & ODEs (Week 27-35)

Double and Triple Integrals

Topics:

- Double integrals over rectangles
- Iterated integrals and Fubini's theorem
- Double integrals over general regions
- Reversing order of integration
- Double integrals in polar coordinates
- Applications: area, volume, mass
- Center of mass and moments of inertia
- Triple integrals in rectangular coordinates
- Triple integrals in cylindrical coordinates
- Triple integrals in spherical coordinates
- Change of variables: Jacobians
- Applications to probability and physics

Projects built:

- Multiple integral calculator
- Volume visualization tool
- Coordinate system converter

Practice: Evaluate 150 multiple integrals

Line and Surface Integrals

Topics:

- Parametric curves and arc length
- Line integrals of scalar functions
- Line integrals of vector fields

- Work and circulation
- Fundamental theorem for line integrals
- Green's theorem and applications
- Parametric surfaces
- Surface area calculations
- Surface integrals of scalar functions
- Surface integrals of vector fields (flux)
- Stokes' theorem
- Divergence theorem (Gauss's theorem)

Projects built:

- Line integral visualizer
- Surface parametrization tool
- Vector calculus theorem demonstrator

Practice: Complete 100 vector calculus problems

First-Order Differential Equations

Topics:

- Classification of differential equations
- Direction fields and solution curves
- Separable equations
- Linear first-order equations
- Integrating factors method
- Exact equations and exactness test
- Homogeneous equations
- Bernoulli equations
- Existence and uniqueness theorems
- Numerical methods: Euler's method
- Improved Euler and Runge-Kutta
- Applications to physics, biology, economics

Projects built:

- ODE solver implementation
- Direction field plotter
- Numerical methods comparison

Practice: Solve 150 first-order ODEs

Higher-Order Linear ODEs

Topics:

- Second-order linear equations
- Homogeneous equations with constant coefficients
- Characteristic equation method
- Complex roots and oscillations
- Repeated roots and reduction of order
- Nonhomogeneous equations
- Method of undetermined coefficients
- Variation of parameters
- Higher-order linear equations
- Systems of first-order linear equations
- Matrix exponentials
- Phase plane analysis

Projects built:

- Second-order ODE solver
- Oscillation simulator
- Phase portrait generator

Practice: Solve 100 higher-order ODEs

Laplace Transforms

Topics:

- Definition and existence of Laplace transform
- Transforms of basic functions
- Linearity and shifting theorems
- Transforms of derivatives and integrals
- Inverse Laplace transforms
- Partial fraction decomposition for inverse transforms
- Convolution theorem
- Solving ODEs with Laplace transforms
- Systems of ODEs via Laplace
- Transfer functions
- Applications to control theory
- Discontinuous forcing functions

Projects built:

- Laplace transform calculator
- Control system analyzer
- ODE system solver via Laplace

Practice: Apply Laplace transforms to 75 problems

Months 9-10: Discrete Mathematics (Week 36-44)

Set Theory and Relations

Topics:

- Naive set theory and paradoxes
- Set operations: union, intersection, complement
- Power sets and Cartesian products
- Functions as relations

- Injective, surjective, bijective functions
- Cardinality and countability
- Cantor's diagonal argument
- Relations: reflexive, symmetric, transitive
- Equivalence relations and partitions
- Partial orders and Hasse diagrams
- Well-ordering principle
- Axiom of choice introduction

Projects built:

- Set operation visualizer
- Relation property checker
- Cardinality comparison tool

Practice: Prove 75 set theory theorems

Combinatorics

Topics:

- Basic counting principles
- Permutations and combinations
- Binomial theorem and Pascal's triangle
- Multinomial coefficients
- Inclusion-exclusion principle
- Pigeonhole principle
- Generating functions
- Recurrence relations
- Solving linear recurrences
- Catalan numbers
- Stirling numbers
- Partitions of integers

Projects built:

- Combinatorics calculator
- Recurrence relation solver
- Generating function manipulator

Practice: Solve 100 combinatorics problems

Graph Theory

Topics:

- Graphs: vertices, edges, degree
- Types of graphs: simple, directed, weighted
- Graph representations: adjacency matrix, list
- Paths, cycles, and connectivity
- Trees and spanning trees
- Minimum spanning trees: Kruskal, Prim
- Shortest paths: Dijkstra, Bellman-Ford
- Eulerian and Hamiltonian paths
- Graph coloring and chromatic number
- Planar graphs and Euler's formula
- Bipartite graphs and matching
- Network flows and max-flow min-cut

Projects built:

- Graph algorithm visualizer
- Network flow optimizer
- Graph coloring solver

Practice: Implement 20 graph algorithms

Number Theory

Topics:

- Divisibility and greatest common divisors
- Euclidean algorithm and extended version
- Prime numbers and fundamental theorem
- Modular arithmetic
- Chinese Remainder Theorem
- Fermat's Little Theorem
- Euler's theorem and totient function
- Wilson's theorem
- Quadratic residues
- Primitive roots
- RSA cryptography basics
- Diophantine equations

Projects built:

- Number theory toolkit
- RSA encryption implementation
- Prime number generator

Practice: Prove 50 number theory results

Boolean Algebra and Logic

Topics:

- Boolean operations and laws
- Truth tables and logical equivalence
- Normal forms: DNF and CNF
- Karnaugh maps
- Logic gates and circuits
- Minimization of Boolean functions
- Propositional logic and inference rules

- Predicate logic and quantifiers
- Formal proofs in logic
- Completeness and soundness
- Introduction to model theory
- Applications to computer science

Projects built:

- Logic circuit simulator
- Boolean function minimizer
- Automated theorem prover

Practice: Design 30 logic circuits

Months 11-12: Probability and Statistics (Week 45-52)

Probability Theory

Topics:

- Sample spaces and events
- Probability axioms and properties
- Conditional probability
- Bayes' theorem and applications
- Independence of events
- Random variables: discrete and continuous
- Probability mass and density functions
- Cumulative distribution functions
- Expected value and variance
- Moment generating functions
- Common discrete distributions
- Common continuous distributions

Projects built:

- Probability distribution visualizer
- Bayes' theorem calculator
- Monte Carlo simulator

Practice: Solve 150 probability problems

Joint Distributions and Limit Theorems

Topics:

- Joint probability distributions
- Marginal and conditional distributions
- Independence of random variables
- Covariance and correlation
- Conditional expectation
- Transformations of random variables
- Order statistics
- Law of large numbers
- Central limit theorem
- Normal approximations
- Characteristic functions
- Convergence concepts

Projects built:

- Joint distribution plotter
- CLT demonstration tool
- Correlation analyzer

Practice: Analyze 100 joint distributions

Statistical Inference

Topics:

- Point estimation: MLE and method of moments
- Properties of estimators: bias, consistency
- Confidence intervals
- Hypothesis testing framework
- Type I and Type II errors
- Power of tests
- t-tests and chi-square tests
- ANOVA basics
- Nonparametric tests
- Linear regression
- Multiple regression
- Model diagnostics

Projects built:

- Statistical test suite
- Regression analysis tool
- Power analysis calculator

Practice: Perform 75 statistical analyses

Stochastic Processes

Topics:

- Introduction to stochastic processes
- Markov chains: discrete time
- Transition matrices and steady states
- Classification of states
- Absorbing Markov chains
- Continuous-time Markov chains
- Poisson processes
- Birth-death processes

- Queueing theory basics
- Brownian motion introduction
- Random walks
- Applications to finance and biology

Projects built:

- Markov chain simulator
- Queue system analyzer
- Random walk visualizer

Practice: Model 50 stochastic systems

Phase 2 Capstone Project

Topics:

- Integration of discrete and continuous mathematics
- Statistical modeling project
- Algorithm implementation
- Research paper reading
- Technical presentation

Projects built:

- MAJOR CAPSTONE: Applied Mathematics Research Project
- Options: Machine learning algorithm, Cryptographic system, Statistical study
- Graph theory application to real networks
- Probability model for real phenomenon

Assessment: Phase 2 Comprehensive Exam

----- PHASE 3: ADVANCED PURE MATHEMATICS (MONTHS 13-18, WEEKS 53-78) -----

Master real analysis, abstract algebra, topology, complex analysis, and advanced mathematical theory.

Months 13-14: Real Analysis (Week 53-61)

Metric Spaces and Sequences

Topics:

- Metric spaces: definition and examples
- Open and closed sets
- Interior, closure, and boundary
- Convergent sequences in metric spaces
- Cauchy sequences and completeness
- Compactness: sequential and open cover
- Heine-Borel theorem
- Connected spaces
- Continuous functions between metric spaces
- Uniform continuity
- Lipschitz continuity
- Homeomorphisms

Projects built:

- Metric space visualizer
- Compactness checker
- Continuity analyzer

Practice: Prove 100 metric space theorems

Limits and Continuity in $\mathbb{R}^n$

Topics:

- The real numbers: completeness axiom
- Supremum and infimum
- Sequences and series of real numbers
- Limit superior and limit inferior
- Functions of real variables
- Continuity and uniform continuity on $\mathbb{R}$
- Intermediate value theorem proof
- Extreme value theorem proof
- Monotone functions
- Functions of bounded variation
- Absolutely continuous functions
- Discontinuities classification

Projects built:

- Real function analyzer
- Continuity proof assistant
- Convergence tester

Practice: Complete 75 epsilon-delta proofs

Differentiation Theory

Topics:

- Derivative as linear approximation
- Differentiability in $\mathbb{R}^n$
- Partial derivatives vs differentiability
- Chain rule proof
- Mean value theorem and generalizations
- Taylor's theorem with remainder
- Inverse function theorem
- Implicit function theorem
- Critical point analysis

- Morse lemma
- Sard's theorem introduction
- Applications to optimization

Projects built:

- Differentiation proof checker
- Taylor approximation tool
- Critical point classifier

Practice: Prove 50 differentiation theorems

Riemann Integration

Topics:

- Riemann integral construction
- Riemann integrability conditions
- Properties of Riemann integral
- Fundamental theorem of calculus proof
- Integration techniques review
- Improper Riemann integrals
- Functions of bounded variation
- Riemann-Stieltjes integral
- Convergence theorems limitations
- Counterexamples in integration
- Numerical integration error analysis
- Applications to probability

Projects built:

- Riemann sum visualizer
- Integrability checker
- Numerical integration analyzer

Practice: Analyze 75 integration problems rigorously

Sequences and Series of Functions

Topics:

- Pointwise convergence
- Uniform convergence
- Weierstrass M-test
- Continuity of uniform limits
- Integration of uniform limits
- Differentiation of uniform limits
- Power series and radius of convergence
- Analytic functions
- Weierstrass approximation theorem
- Stone-Weierstrass theorem
- Fourier series introduction
- Convergence of Fourier series

Projects built:

- Convergence visualizer
- Fourier series calculator
- Function approximator

Practice: Prove convergence for 50 function sequences

Months 15-16: Abstract Algebra (Week 62-70)

Group Theory

Topics:

- Groups: axioms and examples
- Subgroups and subgroup tests
- Cyclic groups and generators
- Group homomorphisms
- Kernel and image
- Isomorphism theorems
- Normal subgroups and quotient groups
- Lagrange's theorem
- Group actions
- Orbit-stabilizer theorem
- Sylow theorems
- Classification of finite abelian groups

Projects built:

- Group calculator
- Cayley table generator
- Subgroup lattice visualizer

Practice: Prove 100 group theory results

Ring Theory

Topics:

- Rings: definition and examples
- Ring homomorphisms
- Ideals and quotient rings
- Prime and maximal ideals
- Integral domains
- Principal ideal domains
- Unique factorization domains
- Euclidean domains
- Polynomial rings
- Field of fractions

- Chinese remainder theorem for rings
- Localization of rings

Projects built:

- Ring structure analyzer
- Ideal calculator
- Polynomial factorization tool

Practice: Explore 75 ring structures

Field Theory

Topics:

- Fields: definition and examples
- Field extensions
- Algebraic and transcendental elements
- Degree of field extensions
- Splitting fields
- Algebraic closure
- Finite fields structure
- Cyclotomic fields
- Galois theory introduction
- Fundamental theorem of Galois theory
- Solvability by radicals
- Constructions with ruler and compass

Projects built:

- Field extension visualizer
- Galois group calculator
- Constructibility checker

Practice: Work through 50 field theory problems

Linear Algebra (Abstract)

Topics:

- Vector spaces over arbitrary fields
- Linear transformations abstract theory
- Dual spaces and dual basis
- Bilinear forms
- Quadratic forms
- Inner product spaces
- Orthogonalization: Gram-Schmidt
- Adjoint operators
- Normal operators
- Spectral theorem
- Singular value decomposition
- Tensor products

Projects built:

- Abstract linear algebra toolkit
- Inner product space explorer
- SVD calculator

Practice: Prove 75 abstract linear algebra theorems

Module Theory

Topics:

- Modules over rings
- Submodules and quotient modules
- Module homomorphisms
- Free modules
- Finitely generated modules
- Torsion modules
- Structure theorem for finitely generated modules over PID
- Applications to linear algebra
- Jordan canonical form via modules
- Projective and injective modules
- Exact sequences
- Introduction to homological algebra

Projects built:

- Module structure analyzer
- Jordan form calculator
- Exact sequence checker

Practice: Explore 50 module structures

Months 17-18: Topology & Complex Analysis (Week 71-78)

General Topology

Topics:

- Topological spaces: definition and examples
- Basis and subbasis
- Closed sets and closure operators
- Interior and boundary
- Hausdorff spaces
- Continuous functions
- Homeomorphisms
- Connectedness
- Path connectedness
- Compactness
- Tychonoff's theorem

- Separation axioms

Projects built:

- Topology visualizer
- Homeomorphism checker
- Compactness prover

Practice: Prove 100 topology theorems

Algebraic Topology Introduction

Topics:

- Homotopy and homotopy equivalence
- Fundamental group
- Computing fundamental groups
- Van Kampen's theorem
- Covering spaces
- Lifting properties
- Classification of covering spaces
- Introduction to homology
- Simplicial complexes
- Simplicial homology
- Singular homology basics
- Applications to fixed point theorems

Projects built:

- Fundamental group calculator
- Covering space visualizer
- Homology computer

Practice: Compute 50 fundamental groups

Complex Analysis

Topics:

- Complex numbers and complex plane
- Complex functions and analyticity
- Cauchy-Riemann equations
- Harmonic functions
- Complex integration
- Cauchy's theorem
- Cauchy integral formula
- Taylor and Laurent series
- Residues and residue theorem
- Evaluation of real integrals
- Conformal mappings
- Möbius transformations

Projects built:

- Complex function visualizer
- Residue calculator
- Conformal mapping tool

Practice: Solve 100 complex analysis problems

Advanced Complex Analysis

Topics:

- Maximum modulus principle
- Schwarz lemma
- Analytic continuation
- Riemann surfaces introduction
- Entire functions
- Meromorphic functions
- Infinite products
- Weierstrass factorization
- Gamma function
- Riemann zeta function
- Prime number theorem outline
- Applications to number theory

Projects built:

- Special functions library
- Riemann surface visualizer
- Zeta function explorer

Practice: Explore 50 special functions

Phase 3 Capstone Project

Topics:

- Pure mathematics research project
- Original proof development
- Mathematical exposition
- Research paper writing
- Peer review process

Projects built:

- MAJOR CAPSTONE: Original Mathematics Research
- Write expository paper on advanced topic
- Create visualization tools for abstract concepts
- Develop lecture series on chosen topic

Assessment: Phase 3 Comprehensive Exam - Pure Mathematics

Master PDEs, numerical analysis, optimization, mathematical modeling, and conduct original research.

Months 19-20: Partial Differential Equations (Week 79-87)

First-Order PDEs

Topics:

- Classification of PDEs
- First-order linear PDEs
- Method of characteristics
- Quasi-linear PDEs
- General first-order PDEs
- Cauchy problem
- Envelopes and singular solutions
- Conservation laws
- Shock waves
- Weak solutions
- Rankine-Hugoniot conditions
- Applications to traffic flow

Projects built:

- Characteristics solver
- Shock wave simulator
- Conservation law visualizer

Practice: Solve 75 first-order PDEs

Second-Order Linear PDEs

Topics:

- Classification: elliptic, parabolic, hyperbolic
- Heat equation: derivation and properties
- Separation of variables
- Fourier series solutions
- Maximum principle for heat equation
- Wave equation: d'Alembert's solution
- Energy methods
- Laplace equation: harmonic functions
- Poisson equation
- Green's functions
- Boundary value problems
- Sturm-Liouville theory

Projects built:

- Heat equation solver
- Wave propagation simulator
- Laplace equation solver

Practice: Solve 100 second-order PDEs

Transform Methods for PDEs

Topics:

- Fourier transform and PDEs
- Heat equation via Fourier transform
- Laplace transform for PDEs
- Hankel transforms
- Integral transform methods
- Duhamel's principle
- Green's function method
- Eigenfunction expansions
- Bessel functions
- Legendre polynomials
- Spherical harmonics
- Special functions in PDEs

Projects built:

- Transform method solver
- Special function library
- Eigenfunction visualizer

Practice: Apply transforms to 75 PDE problems

Numerical Methods for PDEs

Topics:

- Finite difference methods
- Stability and convergence
- CFL condition
- Implicit vs explicit schemes
- Crank-Nicolson method
- Finite element method basics
- Weak formulation
- Galerkin method
- Basis functions
- Assembly process
- Spectral methods introduction
- Multigrid methods

Projects built:

- Finite difference PDE solver
- Finite element implementation

- Stability analyzer

Practice: Implement 20 numerical PDE schemes

Nonlinear PDEs

Topics:

- Nonlinear heat equation
- Burger's equation
- KdV equation and solitons
- Nonlinear Schrödinger equation
- Reaction-diffusion equations
- Pattern formation
- Traveling waves
- Bifurcation in PDEs
- Variational methods
- Fixed point theorems for PDEs
- Existence and uniqueness
- Regularity theory basics

Projects built:

- Soliton simulator
- Pattern formation tool
- Bifurcation analyzer

Practice: Study 50 nonlinear PDE phenomena

Months 21-22: Numerical Analysis & Scientific Computing (Week 88-96)

Numerical Linear Algebra

Topics:

- Floating point arithmetic
- Condition numbers and stability
- LU decomposition
- Cholesky decomposition
- QR decomposition
- Singular value decomposition
- Iterative methods: Jacobi, Gauss-Seidel
- Conjugate gradient method
- GMRES and Krylov methods
- Preconditioning
- Sparse matrix techniques
- Parallel algorithms

Projects built:

- Matrix decomposition library
- Iterative solver suite
- Condition number analyzer

Practice: Implement 30 numerical linear algebra algorithms

Interpolation and Approximation

Topics:

- Polynomial interpolation
- Lagrange interpolation
- Newton's divided differences
- Hermite interpolation
- Spline interpolation
- B-splines and NURBS
- Least squares approximation
- Orthogonal polynomials
- Chebyshev approximation
- Rational approximation
- Padé approximants
- Wavelets introduction

Projects built:

- Interpolation toolkit
- Spline curve designer
- Approximation error analyzer

Practice: Implement 25 approximation methods

Numerical Integration and Differentiation

Topics:

- Newton-Cotes formulas
- Gaussian quadrature
- Adaptive quadrature
- Romberg integration
- Monte Carlo integration
- Quasi-Monte Carlo methods
- Multidimensional integration
- Numerical differentiation
- Richardson extrapolation
- Automatic differentiation
- Complex step differentiation
- Applications to optimization

Projects built:

- Adaptive integrator
- Monte Carlo simulator
- Automatic differentiation engine

Practice: Compare 20 integration methods

Numerical Optimization

Topics:

- Unconstrained optimization
- Gradient descent variants
- Newton's method for optimization
- Quasi-Newton methods: BFGS, L-BFGS
- Conjugate gradient for optimization
- Trust region methods
- Line search strategies
- Constrained optimization
- Lagrange multipliers numerical
- Penalty and barrier methods
- Interior point methods
- Sequential quadratic programming

Projects built:

- Optimization algorithm suite
- Convergence visualizer
- Constraint handler

Practice: Solve 50 optimization problems numerically

Fast Algorithms

Topics:

- Fast Fourier Transform (FFT)
- Fast multiplication algorithms
- Fast matrix multiplication
- Divide and conquer strategies
- Multigrid methods
- Fast multipole method
- Hierarchical matrices
- Randomized algorithms
- Compressed sensing basics
- Sparse recovery
- Low-rank approximations
- Tensor decompositions

Projects built:

- FFT implementation
- Fast algorithm library
- Compression tool

Practice: Implement 15 fast algorithms

Month 23: Mathematical Modeling & Applications (Week 97-100)

Mathematical Biology

Topics:

- Population dynamics models
- Predator-prey systems
- Competition models
- Epidemic models: SIR, SEIR
- Age-structured models
- Spatial models and diffusion
- Pattern formation in biology
- Biochemical networks
- Neural models
- Evolutionary game theory
- Phylogenetic trees
- Systems biology approaches

Projects built:

- Epidemic simulator
- Population dynamics tool
- Pattern formation in biology

Practice: Model 20 biological systems

Mathematical Finance

Topics:

- Time value of money
- Portfolio theory
- CAPM model
- Options and derivatives
- Black-Scholes equation
- Greeks and hedging
- Monte Carlo methods in finance
- Interest rate models
- Credit risk models
- Value at Risk
- Stochastic volatility
- Jump diffusion models

Projects built:

- Option pricer
- Portfolio optimizer
- Risk calculator

Practice: Implement 20 financial models

Data Science Mathematics

Topics:

- Principal component analysis
- Singular value decomposition applications
- Matrix factorizations for data
- Kernel methods
- Support vector machines math
- Neural network mathematics
- Backpropagation derivation
- Optimization in machine learning
- Regularization theory
- Statistical learning theory
- Information theory basics
- Compressed sensing applications

Projects built:

- PCA implementation
- Neural network from scratch
- Dimensionality reduction toolkit

Practice: Implement 15 data science algorithms

Fluid Dynamics & Continuum Mechanics

Topics:

- Conservation laws in fluids
- Navier-Stokes equations
- Potential flow
- Boundary layers
- Turbulence introduction
- Computational fluid dynamics basics
- Elasticity theory
- Stress and strain tensors
- Constitutive equations
- Wave propagation in solids
- Fracture mechanics basics
- Multiphysics modeling

Projects built:

- Flow simulator
- Stress analysis tool
- Wave propagation visualizer

Practice: Solve 20 continuum mechanics problems

Month 24: Research & Career Preparation (Week 101-104)

Mathematical Research Project

Topics:

- Choosing research topic
- Literature review methods
- Research question formulation
- Mathematical writing
- LaTeX for mathematics
- Proof techniques review
- Computational experiments
- Data visualization for mathematics
- Collaboration tools
- Version control for research
- Presenting mathematics
- Peer review process

Projects built:

- FINAL CAPSTONE: Original Research Project
- Complete research paper (10-20 pages)
- Implementation of novel algorithm
- Mathematical software package

Advanced Topics Seminar

Topics:

- Algebraic geometry introduction
- Differential geometry basics
- Lie groups and Lie algebras
- Representation theory
- Category theory basics
- Homological algebra
- K-theory introduction
- Operator theory
- Harmonic analysis
- Ergodic theory
- Mathematical physics topics
- Current research areas

Deliverables:

- Seminar presentation
- Advanced topic exploration paper
- Reading list compilation
- Research proposal for graduate studies

Career Paths & Future Learning

Topics:

- Graduate school preparation
- GRE Mathematics subject test
- Research opportunities
- Industry applications of mathematics
- Quantitative careers overview
- Academic career paths
- Mathematical consulting
- Teaching mathematics
- Open problems in mathematics
- Mathematical communities
- Conferences and workshops
- Lifelong learning in mathematics

Deliverables:

- Complete portfolio of work
- Graduate school application materials
- Professional website/blog
- Network of mathematical contacts
- Personal mathematics library
- Future learning roadmap

Assessment: FINAL COMPREHENSIVE EXAM - Complete undergraduate mathematics

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Calculus visualizers, Linear algebra tools, Proof portfolio
- Phase 2: ODE solvers, Graph algorithms, Statistical analysis suite, Probability simulators
- Phase 3: Metric space explorers, Algebra structure analyzers, Topology visualizers, Complex function tools
- Phase 4: PDE solvers, Numerical libraries, Mathematical models, Original research

Total projects: 150+ mathematical projects and implementations

SKILLS THE STUDENT WILL HAVE AT THE END

- Analysis: Real, Complex, Functional, Harmonic, Measure Theory basics
- Algebra: Linear, Abstract, Commutative, Galois Theory, Representation Theory intro
- Topology: Point-set, Algebraic basics, Differential Geometry intro
- Applied: ODEs, PDEs, Numerical Analysis, Optimization, Mathematical Modeling
- Discrete: Combinatorics, Graph Theory, Number Theory, Logic, Set Theory
- Probability & Statistics: Measure-theoretic probability, Statistical inference, Stochastic processes
- Computation: Scientific computing, Algorithm analysis, Symbolic computation
- Proof Writing: Direct, Contradiction, Induction, Analysis proofs, Algebraic proofs
- Research: Literature review, Problem formulation, Mathematical writing, Presentation

HOW A TYPICAL WEEK IS SPLIT

Theory lectures: 8-10 hours
Problem solving: 8-10 hours
Proofs practice: 3-4 hours
Computational work: 2-3 hours
Reading research: 2-3 hours
Total per week: 20-25 hours

SUPPORT INCLUDED

Office hours: Weekly virtual office hours with instructors
Peer collaboration: Study groups and peer review
Mentorship: Research mentorship for advanced students
Tutoring: TA support for challenging topics
Resources: Access to mathematical software and journals
Community: Active mathematical community forum

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion
Final certificate: Advanced Mathematics Proficiency Certificate
Transcript: Detailed transcript of all courses
Recommendation: Letters of recommendation available
Portfolio: Professional portfolio of mathematical work

WHERE THIS CAN LEAD

- Graduate Studies in Mathematics/Applied Mathematics
- Data Scientist/Machine Learning Engineer
- Quantitative Analyst/Financial Engineer
- Research Mathematician
- Software Engineer (Algorithms/Theory)
- Actuary

- Operations Research Analyst
- Cryptographer
- Mathematical Consultant
- University Professor/Lecturer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the College Mathematics Masterclass actually cover?

A: The program runs 24 months across four phases: single variable calculus, linear algebra and proof writing first; then multivariable calculus, differential equations, discrete mathematics and probability; then real analysis, abstract algebra, topology and complex analysis; and finally applied topics like PDEs, numerical analysis and optimization with a research style project. It is a complete undergraduate level mathematics pathway taught live.

Q: Who is this course designed for?

A: It fits three groups well: school leavers heading into engineering, science or economics degrees who want a head start; current undergraduates who need stronger theory and proof skills than their college lectures provide; and working professionals returning to serious mathematics. Pace and depth are matched to your goals in the first month.

Q: What are the prerequisites?

A: Comfortable high school mathematics, including algebra and basic trigonometry at pre-calculus level. Programming helps with the computational topics but is not required, and access to the mathematical software we use is provided. A diagnostic in the first sessions confirms the right starting point.

Q: How is progress measured across such a long program?

A: Through weekly problem sets, proof write-ups reviewed line by line, and end of phase assessments that must be secure before the next phase begins. Your mentor keeps a running record of solved problems and completed proofs, so you can see exactly what has been mastered.

Q: Is this useful preparation for graduate school?

A: Yes. The phase 3 and phase 4 material, real analysis, abstract algebra, topology and PDEs, is the standard core that mathematics and quantitative graduate programs expect. The final months also include research style projects and guidance on presenting mathematical work.

Q: Can I try a class before committing?

A: Yes, every student starts with a free demo class booked at learn.modernagecoders.com/contact, with no commitment and no payment details required. The demo doubles as a level check so we can recommend the right starting phase. You can also reach us on WhatsApp at +91 91233 66161.

Q: Are the classes live or recorded lectures?

A: Every class is a live one hour online session with a real teacher, never a pre-recorded video. Groups are kept small so you can ask questions mid-proof, and sessions are interactive: you solve and explain, and the mentor corrects the thinking, not just the answer.

Q: Do I get a certificate at the end?

A: Yes, you receive a completion certificate from Modern Age Coders along with a portfolio record of the proofs, problem sets and projects you completed. Families consistently rate this teaching approach highly: we hold 4.9 stars from 547+ Google reviews.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/college-mathematics-complete-masterclass>

COURSE 36 OF 118: COMPETITIVE PROGRAMMING COURSE ONLINE: CRACK CODEFORCES

Course link: <https://learn.modernagecoders.com/courses/competitive-programming-masterclass-college>
 Subject area: Competitive Programming & Algorithms
 Designed for: College students
 Who can join: Complete Beginner to Expert Competitive Programmer
 Course length: 6 months (26 weeks)
 Time needed: 20-25 hours/week recommended
 Certificate: Competitive Programming Expert Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
 Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live competitive programming course online: 6 months from beginner to Codeforces Expert,

mastering DSA and algorithms for ICPC and coding interviews.

In one line: From Zero to Contest Champion in 6 Months

ABOUT THE PROGRAMME

This intensive 6-month program transforms beginners into expert competitive programmers capable of competing in top-tier programming contests. Master every algorithm, data structure, and problem-solving technique used in competitive programming. You'll solve 1000+ problems, participate in 100+ contests, learn advanced algorithms, master dynamic programming, conquer graph theory, and develop the speed and accuracy needed for top rankings. By completion, you'll be ready for ICPC, Google Code Jam, and achieve high ratings on platforms like Codeforces and CodeChef.

WHAT MAKES THIS COURSE DIFFERENT

- Start from absolute basics with C++ STL
- 1000+ carefully curated problems with video editorials
- Live contest participation and analysis
- Speed programming techniques and shortcuts
- Mathematical foundations for CP
- Team contest strategies for ICPC
- Rating improvement guaranteed approach
- Mental preparation and contest psychology

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-2): C++ STL, Basic DS/Algo, Easy Problems

Phase 2: Intermediate (Months 3-4): Advanced DS, DP, Graphs, Mathematics

Phase 3: Advanced (Months 5-6): Expert Algorithms, Optimization, Contest Mastery

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic programming knowledge helpful

Coding experience: Can write simple programs

Equipment: Computer with good internet

Time commitment: 20-25 hours per week

English: Good comprehension required

Motivation: Passion for problem-solving

WHO THIS COURSE IS FOR

Students: CS students aiming for competitions

Developers: Programmers wanting algorithm mastery

Competitors: Aspiring ICPC/IOI participants

Job seekers: FAANG interview preparation

Enthusiasts: Algorithm and puzzle lovers

Teachers: Future CP coaches and mentors

Researchers: Algorithm research aspirants

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & PROBLEM SOLVING BASICS (MONTHS 1-2, WEEKS 1-8)

Build strong foundations in C++ STL, basic algorithms, and develop problem-solving mindset.

Month 1: C++ Mastery & Basic Algorithms (Week 1-4)

C++ STL & Fast I/O

Topics:

- Competitive programming introduction and platforms
- Why C++ for competitive programming
- Fast I/O techniques: `ios_base`, `cin.tie`
- C++ STL containers: `vector`, `set`, `map`, `queue`, `stack`
- STL algorithms: `sort`, `binary_search`, `lower_bound`, `upper_bound`
- Pairs and tuples usage
- Priority queue and custom comparators
- Multiset and multimap
- Unordered containers and hashing
- Bitset for optimization
- Template programming basics
- Macro definitions and shortcuts

Projects built:

- CP template creation
- STL practice problems set
- Fast I/O benchmarking

Practice: Solve 50 STL-based problems

Time Complexity & Basic Algorithms

Topics:

- Time and space complexity analysis
- Big O notation mastery
- Common time complexities in CP
- Estimating execution time from complexity
- Greedy algorithms and proofs
- Two pointers technique
- Sliding window problems
- Prefix sums and difference arrays
- Binary search on answer
- Ternary search basics
- Meet in the middle technique
- Coordinate compression

Projects built:

- Complexity analyzer tool
- Algorithm visualization
- Problem approach templates

Practice: Master 60 basic algorithm problems

Basic Data Structures

Topics:

- Arrays and vectors optimization
- Linked lists in competitive programming
- Stacks and queues applications
- Deque and its uses
- Hash tables and collision handling
- Custom hash functions
- Binary heap implementation
- Disjoint Set Union (DSU) basics
- Sparse tables introduction
- Square root decomposition basics
- Monotonic stack and queue
- Policy based data structures

Projects built:

- Data structure library
- Custom implementations
- Performance comparison tool

Practice: Implement and use 40 DS problems

Recursion & Backtracking

Topics:

- Recursion optimization techniques
- Memoization strategies
- Backtracking framework
- Pruning techniques
- N-Queens and variations
- Sudoku solver optimization
- Permutation generation
- Combination generation
- Subset generation optimized
- Bitmask techniques basics
- State space reduction
- Iterative vs recursive approaches

Projects built:

- Backtracking solver framework
- Optimization comparisons
- Contest problem generator

Practice: Solve 50 recursion/backtracking problems

Month 2: Mathematics & Number Theory (Week 5-8)

Number Theory Fundamentals

Topics:

- Prime numbers and sieve of Eratosthenes
- Segmented sieve for large ranges
- Prime factorization techniques
- GCD and LCM algorithms
- Extended Euclidean algorithm
- Modular arithmetic and properties
- Modular inverse calculation
- Fast exponentiation
- Fermat's little theorem
- Wilson's theorem
- Chinese Remainder Theorem
- Linear Diophantine equations

Projects built:

- Number theory library
- Prime utilities toolkit
- Modular arithmetic calculator

Practice: Master 60 number theory problems

Combinatorics & Probability

Topics:

- Permutations and combinations
- Pascal's triangle and properties
- Binomial coefficients calculation
- nCr with modulo computation
- Catalan numbers and applications
- Stirling numbers basics
- Inclusion-exclusion principle
- Pigeonhole principle applications
- Probability in competitive programming
- Expected value problems
- Generating functions basics
- Counting techniques

Projects built:

- Combinatorics calculator
- Probability simulator
- Counting problems solver

Practice: Solve 50 combinatorics problems

Bit Manipulation Mastery

Topics:

- Bitwise operations deep dive
- Bit manipulation tricks and hacks
- Finding single element problems
- XOR properties and applications
- Subset generation using bits
- Bitmask dynamic programming intro
- Gray code and applications
- Bit parallel algorithms
- Counting set bits efficiently
- `__builtin` functions in GCC
- Bitwise sieve
- Space optimization with bits

Projects built:

- Bit manipulation library
- Bitwise algorithm collection
- Space-optimized solutions

Practice: Master 40 bit manipulation problems

Sorting & Searching Advanced

Topics:

- Quick sort optimizations
- Merge sort applications
- Counting sort and radix sort
- Bucket sort for decimals
- Binary search variations
- Parallel binary search
- Ternary search on functions
- Search on monotonic functions
- Fractional cascading basics
- Order statistics
- K-th smallest element algorithms
- Median finding in $O(n)$

Projects built:

- Sorting algorithm library
- Search optimization toolkit
- Performance benchmarks

Practice: Solve 50 sorting/searching problems

Phase 1 Assessment

Topics:

- Foundation skills review
- Speed coding assessment
- Problem-solving patterns
- Contest performance analysis
- Weakness identification

Projects built:

- CAPSTONE: Virtual Contest Marathon
- Participate in 10 virtual contests
- Achieve consistent problem solving

Assessment: Phase 1 Exam - Solve 5 problems in 2 hours

Months 3-4: Trees, Graphs & Dynamic Programming (Week 9-17)

Tree Algorithms

Topics:

- Tree traversals optimization
- Binary tree problems
- Binary search tree operations
- Tree DP fundamentals
- LCA using binary lifting
- Distance between nodes
- Centroid decomposition

- Heavy-light decomposition basics
- Euler tour technique
- Tree flattening
- Rerooting technique
- Tree isomorphism

Projects built:

- Tree algorithms library
- LCA implementations comparison
- Tree problem solver

Practice: Master 60 tree problems

Graph Algorithms Fundamentals

Topics:

- Graph representations optimization
- DFS and BFS optimizations
- Connected components
- Strongly connected components
- Topological sorting applications
- Cycle detection algorithms
- Bipartite checking and matching
- Articulation points and bridges
- Kosaraju's and Tarjan's algorithms
- 2-SAT problem solving
- Eulerian path algorithms
- Graph coloring algorithms

Projects built:

- Graph library implementation
- Graph visualizer
- Algorithm comparison tool

Practice: Solve 70 graph problems

Shortest Paths & MST

Topics:

- Dijkstra's algorithm optimizations
- Bellman-Ford and negative cycles
- Floyd-Warshall applications
- Johnson's algorithm
- K-shortest paths
- Shortest path in DAG
- 0-1 BFS technique
- Dial's algorithm
- Kruskal's and Prim's MST
- Minimum spanning forest
- Second best MST
- Minimum bottleneck spanning tree

Projects built:

- Path finding library
- MST solver toolkit
- Graph optimization suite

Practice: Master 50 path/MST problems

Dynamic Programming Basics

Topics:

- DP state definition techniques
- Top-down vs bottom-up approaches
- 1D DP classic problems
- 2D DP patterns
- Space optimization techniques
- DP on sequences
- LIS and LCS variations
- Knapsack problem variations
- Subset sum problems
- Partition problems
- DP with probability
- Expected value DP

Projects built:

- DP problem classifier
- State transition visualizer
- DP optimization toolkit

Practice: Solve 80 DP problems

Phase 2 Assessment

Topics:

- Intermediate skills evaluation
- Algorithm implementation speed
- Problem pattern recognition
- Contest rating improvement
- Advanced topics readiness

Projects built:

- MAJOR CAPSTONE: Algorithm Library
- Build comprehensive CP library
- Include all learned algorithms

Assessment: Phase 2 Exam - Solve 4 problems in 2.5 hours

PHASE 2: ADVANCED ALGORITHMS & CONTEST STRATEGY (MONTHS 3-4, WEEKS 9-17)

Master advanced algorithms, dynamic programming patterns, and develop contest-winning strategies.

Month 3: Advanced Dynamic Programming (Week 9-13)

DP Optimization Techniques

Topics:

- Convex hull trick
- Li Chao tree
- Divide and conquer optimization
- Knuth-Yao speedup
- Monotone queue optimization
- Slope trick
- Aliens trick (WQS binary search)
- CHT for DP optimization
- Matrix exponentiation for DP
- DP on broken profile
- SOS DP (Sum over Subsets)
- Profile optimization

Projects built:

- DP optimization library
- Convex hull implementation
- Matrix exponentiation toolkit

Practice: Master 40 optimized DP problems

Bitmask DP & DP on Trees

Topics:

- Bitmask DP comprehensive
- Traveling salesman problem
- Hamiltonian path DP
- Profile dynamic programming
- DP on trees advanced
- Subtree DP problems
- Rerooting tree DP
- Tree DP with multiple states
- Centroid decomposition with DP
- Virtual tree technique
- DP on cactus graphs
- DP on DAGs

Projects built:

- Bitmask DP solver
- Tree DP framework
- State compression tools

Practice: Solve 50 bitmask/tree DP problems

String Algorithms

Topics:

- KMP algorithm implementation
- Z-algorithm and applications
- Rolling hash and Rabin-Karp
- Polynomial hashing techniques
- Trie data structure advanced
- Aho-Corasick algorithm
- Suffix array construction
- LCP array and applications
- Suffix tree basics
- Manacher's algorithm
- Suffix automaton introduction
- String matching with wildcards

Projects built:

- String algorithms library
- Pattern matching toolkit
- Suffix structures implementation

Practice: Master 60 string problems

Advanced Graph Algorithms

Topics:

- Network flow algorithms
- Ford-Fulkerson and Edmonds-Karp
- Dinic's algorithm
- Push-relabel algorithm
- Min-cost max-flow
- Maximum bipartite matching
- Hall's marriage theorem
- Hopcroft-Karp algorithm
- Hungarian algorithm
- Gomory-Hu tree
- Global min-cut
- Stoer-Wagner algorithm

Projects built:

- Flow network library
- Matching algorithms suite
- Flow visualization tool

Practice: Solve 40 flow/matching problems

Game Theory

Topics:

- Nim game and variations
- Sprague-Grundy theorem
- Game theory with DP
- Minimax algorithm
- Alpha-beta pruning
- Combinatorial games
- Partisan vs impartial games
- Multi-player games
- Probabilistic games
- Game on graphs
- Hackenbush game
- Conway's soldiers

Projects built:

- Game theory solver
- Nim calculator
- Game position evaluator

Practice: Master 30 game theory problems

Month 4: Data Structures & Optimization (Week 14-17)

Advanced Data Structures

Topics:

- Segment tree with lazy propagation
- Persistent segment tree
- Dynamic segment tree
- 2D segment tree
- Fenwick tree (BIT) advanced
- 2D Fenwick tree
- Range update range query
- Merge sort tree
- Wavelet tree
- Treap and Cartesian tree
- Splay tree basics
- Link-cut tree introduction

Projects built:

- Segment tree library
- Range query toolkit
- Persistent DS implementation

Practice: Implement 50 DS problems

Computational Geometry

Topics:

- 2D geometry primitives
- Cross and dot products
- Line intersection algorithms
- Polygon area and perimeter
- Point in polygon test
- Convex hull algorithms
- Graham scan and Jarvis march
- Rotating calipers technique
- Closest pair of points
- Line sweep algorithms
- Voronoi diagrams basics
- Delaunay triangulation intro

Projects built:

- Geometry library
- Convex hull visualizer
- Geometric algorithm toolkit

Practice: Solve 40 geometry problems

Advanced Number Theory

Topics:

- Miller-Rabin primality test
- Pollard rho factorization
- Discrete logarithm
- Baby-step giant-step
- Primitive roots
- Mobius function and inversion
- Multiplicative functions
- Dirichlet convolution
- Linear sieve
- Gaussian elimination modulo
- Matrix operations in field
- FFT and NTT basics

Projects built:

- Advanced number theory library
- Factorization toolkit
- FFT implementation

Practice: Master 35 advanced math problems

Optimization & Heuristics

Topics:

- Simulated annealing
- Genetic algorithms basics
- Hill climbing
- Branch and bound advanced
- Meet in the middle optimization
- Sqrt decomposition applications
- Mo's algorithm
- Parallel binary search
- Fractional cascading
- Heavy path decomposition
- Centroid path decomposition
- Cache optimization techniques

Projects built:

- Optimization framework
- Heuristic solver
- Performance profiler

Practice: Optimize 30 TLE solutions

Interactive & Randomized

Topics:

- Interactive problem strategies
- Binary search in interactive
- Graph exploration interactive
- Randomized algorithms
- Monte Carlo methods
- Las Vegas algorithms
- Randomized data structures
- Treap operations
- Skip list basics
- Bloom filters
- Probabilistic counting
- Random sampling techniques

Projects built:

- Interactive problem solver
- Randomized algorithm library
- Probability tools

Practice: Solve 25 interactive/random problems

Months 5-6: Contest Mastery & Peak Performance (Week 18-26)

Contest Strategy & Time Management

Topics:

- Contest strategy development
- Problem selection order
- Time allocation strategies
- Reading all problems first
- Identifying easy problems quickly
- When to skip problems
- Partial scoring strategies
- Hacking and challenges
- System testing preparation
- Rating calculation understanding
- Contest anxiety management
- Peak performance techniques

Projects built:

- Personal strategy guide
- Contest analysis tool
- Performance tracker

Practice: Participate in 20 rated contests

Speed Programming & Templates

Topics:

- Speed coding techniques
- Touch typing for programming
- IDE shortcuts mastery
- Template preparation and usage
- Macro optimization
- Pre-written code segments
- Fast debugging techniques
- Common bug patterns
- Quick testing strategies
- Stress testing methods
- Local testing automation
- Vim/Emacs for CP

Projects built:

- Ultimate CP template

- Snippet library
- Testing framework

Practice: Speed coding 100 problems

Team Programming (ICPC Style)

Topics:

- Team contest strategies
- Role distribution in teams
- Communication protocols
- Parallel problem solving
- Code handoff techniques
- Debugging others' code
- Team template preparation
- Contest environment setup
- Printed material preparation
- Team practice sessions
- ICPC regional strategies
- World Finals preparation

Projects built:

- Team notebook creation
- Role optimization system
- Communication framework

Practice: 50 team contest hours

Platform-Specific Mastery

Topics:

- Codeforces: rounds, ratings, hacks
- CodeChef: long, cook-off, lunchtime
- AtCoder: ABC, ARC, AGC differences
- TopCoder: Arena, SRM format
- HackerRank: contests and certifications
- LeetCode: contests and interview prep
- Google competitions: Kick Start, Code Jam
- Facebook Hacker Cup format
- ICPC: regionals, nationals, worlds
- CSAcademy and other platforms
- Platform-specific strategies
- Multi-platform rating management

Projects built:

- Platform comparison guide
- Rating tracker
- Contest calendar

Practice: Achieve goals on each platform

Phase 3 Final Assessment

Topics:

- Comprehensive skill evaluation
- Mock contest marathon
- Weakness elimination
- Rating achievement review
- Future planning

Projects built:

- FINAL CAPSTONE: Contest Performance
- Achieve target rating on Codeforces
- Complete 1000+ problems solved

Assessment: Final Exam - 3-hour contest simulation

PHASE 3: EXPERT LEVEL & SPECIALIZATIONS (MONTHS 5-6, WEEKS 18-26)

Master expert-level algorithms, specialized topics, and achieve competitive programming excellence.

Month 5: Expert Algorithms & Techniques (Week 18-22)

FFT & Polynomial Algorithms

Topics:

- Fast Fourier Transform complete
- Number Theoretic Transform (NTT)
- Polynomial multiplication
- Polynomial division
- Polynomial interpolation
- Generating functions advanced
- Convolution applications
- XOR convolution
- Subset sum convolution
- Polynomial inverse and sqrt
- Multipoint evaluation
- Chirp Z-transform

Projects built:

- FFT library complete
- Polynomial operations toolkit
- Convolution solver

Practice: Master 25 FFT/polynomial problems

Linear Algebra in CP

Topics:

- Gaussian elimination advanced
- Matrix exponentiation applications
- System of linear equations
- Matrix inverse and determinant
- Basis and linear independence
- XOR basis and applications
- Linear recurrence solving
- Berlekamp-Massey algorithm
- Characteristic polynomial
- Cayley-Hamilton theorem
- Matrix tree theorem
- Spectral graph theory basics

Projects built:

- Linear algebra library
- Matrix computation toolkit
- Recurrence solver

Practice: Solve 30 linear algebra problems

Advanced String Structures

Topics:

- Suffix tree construction
- Suffix automaton complete
- Palindromic tree (Eertree)
- Lyndon decomposition
- Suffix array DC3 algorithm
- SA-IS algorithm
- LCP construction methods
- Enhanced suffix array
- Compressed suffix structures
- Burrows-Wheeler transform
- FM-index basics
- Approximate string matching

Projects built:

- Advanced string library
- Suffix structure implementations
- String analysis toolkit

Practice: Master 35 advanced string problems

Advanced Graph Theory

Topics:

- Planar graphs and testing
- Graph minor theory basics
- Tree decomposition
- Treewidth and pathwidth
- Chordal graphs
- Perfect graphs introduction
- Graph coloring advanced
- Chromatic polynomial
- Tutte polynomial basics
- Expander graphs
- Random graphs
- Algebraic graph theory

Projects built:

- Advanced graph library
- Graph property tester
- Decomposition algorithms

Practice: Solve 25 advanced graph problems

Approximation Algorithms

Topics:

- Approximation ratios
- Vertex cover approximation
- TSP approximation algorithms
- Set cover approximation
- Bin packing algorithms
- Scheduling approximations
- Facility location problems
- K-center and k-median
- PTAS and FPTAS
- Primal-dual methods
- LP relaxation techniques
- Randomized rounding

Projects built:

- Approximation algorithm suite
- Performance analyzer
- Heuristic framework

Practice: Implement 20 approximation algorithms

Research & Cutting Edge

Topics:

- Recent IOI/ICPC problems analysis
- Research paper implementation
- Novel algorithm exploration
- Quantum algorithm basics
- Machine learning in CP
- Parallel algorithms
- Distributed algorithms basics
- External memory algorithms
- Cache-oblivious algorithms
- Succinct data structures
- Compressed data structures
- Future of competitive programming

Projects built:

- Research implementation
- Novel solution finder
- Algorithm innovation

Practice: Solve 20 research-level problems

Competition Preparation

Topics:

- Major competition strategies
- ICPC World Finals preparation
- IOI preparation techniques
- Google Code Jam Finals
- Facebook Hacker Cup Finals
- TopCoder Open strategies
- Mental preparation
- Physical fitness for contests
- Stress management advanced
- Team coordination mastery
- International competition tips
- Travel and logistics

Projects built:

- Competition prep guide
- Training schedule
- Peak performance plan

Practice: Simulate major competitions

Teaching & Mentoring

Topics:

- Creating programming problems
- Test case generation
- Problem setting for contests
- Solution verification
- Editorial writing
- Video tutorial creation
- Mentoring beginners
- Code review skills
- Building CP community
- Organizing contests
- Judge system understanding
- Contributing to platforms

Projects built:

- Problem creation toolkit
- Contest organization
- Educational content

Practice: Create 10 original problems

Career & Opportunities

Topics:

- CP in technical interviews
- FAANG interview preparation
- Competitive programming careers
- Research opportunities
- Graduate school applications
- CP in resume building
- Networking in CP community
- Internship opportunities
- Problem setter careers
- Algorithm researcher path
- Quantitative trading roles
- Software engineering excellence

Projects built:

- Interview preparation kit
- Career portfolio
- Professional network

Practice: Interview problem solving

Final Graduation

Topics:

- Achievement celebration

- Progress review
- Rating milestones
- Problem count celebration
- Contest achievements
- Skill assessment
- Future goals setting
- Continuous improvement plan
- Alumni network
- Community contribution
- Mentorship opportunities
- Legacy building

Projects built:

- FINAL SHOWCASE: CP Portfolio
- 1000+ problems solved
- Contest achievements display
- Algorithm library complete

Assessment: Final Certification - Expert Competitive Programmer

Lifetime Resources & Continuous Learning (Post-Course Support)

Advanced Problem Archives

Topics:

- IOI problem archive
- ICPC problem database
- Google Code Jam archive
- Facebook Hacker Cup problems
- TopCoder problem archive
- Project Euler exploration
- USACO training pages
- Russian olympiad problems
- Chinese OJ problems
- Japanese contest archives
- Research problem sets
- Unsolved problems list

Projects built:

- Problem database creation
- Solution archive
- Difficulty classifier

Practice: Continuous problem solving

Community & Contribution

Topics:

- CP communities worldwide
- Discord and Slack groups
- Reddit communities
- Stack Exchange participation
- Codeforces blog writing
- YouTube channel creation
- Twitch streaming CP
- Open source contributions
- Algorithm visualization projects
- Educational platform contribution
- Regional contest organization
- School/college club formation

Projects built:

- Community engagement plan
- Content creation schedule
- Contribution tracker

Practice: Weekly community participation

Research & Innovation

Topics:

- Algorithm research papers
- Conference participation
- Paper publication process
- Research collaboration
- PhD opportunities in algorithms
- Grant writing basics
- Patent considerations
- Industry research roles
- Academic positions
- Research problem identification
- Literature review techniques
- Research methodology

Projects built:

- Research proposal
- Literature survey
- Innovation project

Practice: Research paper reading

Specialized Domains

Topics:

- Quantum computing algorithms

- Cryptographic algorithms
- Bioinformatics algorithms
- Computational biology
- Financial algorithms
- Machine learning integration
- Computer vision algorithms
- Natural language processing
- Robotics algorithms
- Game development algorithms
- Graphics programming
- Scientific computing

Projects built:

- Domain exploration
- Specialized algorithm implementation
- Cross-domain applications

Practice: Explore new domains

Success Stories & Inspiration

Topics:

- CP legends and their journeys
- Tourist (Gennady Korotkevich)
- Petr Mitrichev's career
- IOI gold medalists
- ICPC world champions
- Google Code Jam winners
- Career transitions from CP
- Success in tech interviews
- Research achievements
- Startup founders from CP
- Teaching and mentoring success
- Community impact stories

Projects built:

- Personal success plan
- Goal setting framework
- Inspiration collection

Practice: Define personal success

ELITE TRACK: BEYOND EXPERT LEVEL

Optional advanced paths for those aiming for the absolute top

Grandmaster Path (Elite Training)

Rating 2400+ Strategy

Topics:

- Grandmaster mindset
- Consistency at high level
- Problem creation skills
- Speed vs accuracy balance
- Advanced hacking strategies
- Rating optimization tactics
- Handling rating pressure
- Comeback strategies
- Maintaining peak performance
- Avoiding burnout
- Long-term progression
- Elite community networking

Projects built:

- Grandmaster roadmap
- Performance optimization
- Elite strategy guide

Practice: Daily grandmaster training

International Competition

Topics:

- IOI medal strategies
- ICPC World Finals excellence
- Google Code Jam Finals
- Facebook Hacker Cup Finals
- TopCoder Open championship
- AtCoder World Tour
- Distributed Code Jam
- Hash Code optimization
- Reply Code Challenge
- Russian Code Cup
- Yandex contests
- VK Cup participation

Projects built:

- Competition calendar
- Performance tracker
- Medal collection

Practice: Elite competition focus

Problem Setting Mastery

Topics:

- Creating original problems
- Difficulty calibration
- Test data generation
- Solution uniqueness
- Anti-hacking techniques
- Checker writing
- Validator creation
- Problem statement clarity
- Editorial writing excellence
- Contest preparation
- Platform coordination
- Problem review process

Projects built:

- Problem portfolio
- Contest creation
- Setter reputation building

Practice: Set problems for platforms

Research Excellence

Topics:

- Publishing in algorithms
- Conference presentations
- Collaboration with professors
- Grant applications
- PhD preparation
- Research internships
- Industry research roles
- Algorithm patents
- Open problems tackling
- Breakthrough techniques
- Citation building
- Academic networking

Projects built:

- Research publication
- Conference submission
- Academic portfolio

Practice: Research contribution

Legacy Building

Topics:

- Creating lasting impact
- Educational content creation
- Platform development
- Tool creation for community
- Book writing
- Course creation
- Mentorship programs
- Scholarship establishment
- Community leadership
- Industry influence
- Next generation training
- Personal brand building

Projects built:

- Legacy project planning
- Impact measurement
- Succession planning

Practice: Build lasting contributions

Professional Applications (Industry Excellence)

Quantitative Trading

Topics:

- Algorithms in finance
- High-frequency trading
- Market making algorithms
- Statistical arbitrage
- Portfolio optimization
- Risk management algorithms
- Backtesting systems
- Order execution algorithms
- Alpha generation
- Machine learning in trading
- Cryptocurrency algorithms
- Career in quant finance

Projects built:

- Trading algorithm
- Backtesting system
- Portfolio optimizer

Practice: Financial algorithms

Tech Giant Excellence

Topics:

- FAANG interview mastery
- System design for scale
- Distributed systems algorithms
- Machine learning systems
- Infrastructure algorithms
- Search algorithms at scale
- Recommendation systems
- Ad serving algorithms
- Content delivery optimization
- Database algorithms
- Network algorithms
- Career progression in tech

Projects built:

- System design portfolio
- Scale algorithm library
- Interview excellence kit

Practice: Industry-scale problems

Startup & Innovation

Topics:

- Algorithm-based startups
- Product development
- MVP with algorithms
- Scaling algorithms
- Algorithm as a service
- API development
- Patent strategies
- Funding for tech startups
- Team building
- Technical leadership
- Exit strategies
- Success stories

Projects built:

- Startup concept
- MVP development
- Pitch preparation

Practice: Build startup prototype

Education & Impact

Topics:

- Teaching at universities
- Online course creation
- YouTube channel growth
- Book publication
- Workshop conducting
- Corporate training
- Coaching business
- Platform development
- Community building
- Social impact through CP
- Diversity initiatives
- Global outreach

Projects built:

- Educational platform
- Course curriculum
- Impact measurement

Practice: Teach and mentor

Lifetime Excellence

Topics:

- Continuous learning mindset
- Staying relevant
- Technology evolution
- Career pivots
- Work-life balance
- Health and coding
- Family and CP
- Financial planning
- Retirement planning
- Giving back
- Wisdom sharing
- Life beyond CP

Projects built:

- Life plan
- Balance framework
- Legacy document

Practice: Sustainable excellence

Resources & References (Comprehensive Library)

Book References

Topics:

- Introduction to Algorithms (CLRS)
- Competitive Programming (Halim)
- Algorithm Design (Kleinberg)
- The Art of Computer Programming (Knuth)
- Concrete Mathematics
- Combinatorial Optimization
- Approximation Algorithms
- Randomized Algorithms
- Computational Geometry
- Network Flows
- Graph Theory books
- Number Theory books

Projects built:

- Reading list
- Book notes
- Implementation library

Practice: Book problem solving

Online Resources

Topics:

- CP-Algorithms website
- GeeksforGeeks CP section
- TopCoder tutorials
- Codeforces EDU section
- USACO guide
- OI Wiki
- Competitive Programming GitHub repos
- YouTube channels list
- Discord servers
- Telegram groups
- WhatsApp groups
- Forums and blogs

Projects built:

- Resource compilation
- Bookmark organization
- Community directory

Practice: Resource utilization

Tools & Utilities

Topics:

- IDE setup guides
- Debugging tools
- Stress testing tools
- Test case generators
- Graph visualizers
- Algorithm animators
- Performance profilers
- Online judges list
- Browser extensions
- Command line tools
- Version control for CP
- Collaboration tools

Projects built:

- Tool configuration
- Utility library
- Productivity setup

Practice: Tool mastery

Templates & Libraries

Topics:

- C++ template library
- Java collections for CP
- Python libraries for CP
- Graph algorithms library
- Number theory library
- Geometry library
- String algorithms library
- Data structures library
- DP templates
- Flow network library
- FFT implementation
- Big integer library

Projects built:

- Personal library
- Template book
- Quick reference

Practice: Library usage mastery

Final References (Quick Access)

Contest Cheat Sheets

Topics:

- Time complexity table
- Common algorithms list
- Pattern recognition guide
- Bug checklist
- Edge cases list
- Testing strategies
- Hacking strategies
- Platform-specific tips
- Contest day checklist
- Emergency algorithms
- Quick implementations
- Last-minute review

Projects built:

- Cheat sheet compilation
- Contest day kit
- Emergency reference

Achievement Tracking

Topics:

- Rating milestones
- Problem count goals
- Contest achievements
- Platform badges
- Competition medals
- Contribution points
- Teaching achievements
- Community impact
- Career milestones
- Personal records
- Team achievements
- Legacy metrics

Deliverables:

- Achievement portfolio
- Progress dashboard
- Success metrics

Future Roadmap

Topics:

- Post-course goals
- Continuous improvement
- New platforms exploration
- Emerging algorithms
- Quantum computing future
- AI in competitive programming
- Career evolution
- Community leadership
- Global impact
- Personal growth
- Work-life integration
- Lifetime learning

Deliverables:

- Future plan
- Goal framework
- Vision document

Assessment: LIFETIME MEMBERSHIP - Continuous updates and elite community access

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: CP Template Library, Algorithm Visualizer, Contest Tracker
- Phase 2: Advanced DS Library, DP Solver, Graph Algorithm Suite
- Phase 3: Complete CP Framework, Problem Generator, Contest Platform
- Final: 1000+ Problems Solved, Expert Rating Achievement, Teaching Platform

Total projects: 20+ major implementations and 100+ algorithm libraries

SKILLS THE STUDENT WILL HAVE AT THE END

- Languages: C++ (Expert), STL Mastery, Java/Python for CP
- Algorithms: 200+ algorithms mastered and implemented
- Data Structures: 50+ data structures from basic to advanced
- Mathematics: Number Theory, Combinatorics, Linear Algebra, FFT
- Problem Solving: 1000+ problems across all platforms
- Contest Skills: Speed coding, debugging, optimization, hacking
- Platforms: Codeforces, CodeChef, AtCoder, TopCoder, ICPC
- Advanced: Research algorithms, approximation, randomized algorithms
- Soft Skills: Team coordination, teaching, problem setting

HOW A TYPICAL WEEK IS SPLIT

Theory study: 5-6 hours

Problem solving: 10-12 hours

Contests: 4-6 hours
Implementation: 3-4 hours
Total per week: 20-25 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving sessions
Mentorship: 1-on-1 guidance from red coders
Community: Elite Discord with 24/7 support
Code review: Solution optimization and debugging help
Contest analysis: Post-contest discussion and upsolving
Career support: Interview prep and referrals
Lifetime access: All content and future updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
Final certificate: Competitive Programming Expert Certification
Linkedin badge: Verifiable credential
Platform achievements: Codeforces/CodeChef ratings
Portfolio: 1000+ solved problems showcase

WHERE THIS CAN LEAD

- Competitive Programmer
- Software Engineer at FAANG
- Quantitative Developer
- Algorithm Researcher
- Problem Setter
- CP Coach/Mentor
- Technical Interview Coach
- Startup CTO/Technical Co-founder
- Machine Learning Engineer
- Game Developer
- Blockchain Developer
- Research Scientist

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Ready for internships; Rs 10-20 LPA (Junior roles)

Growing roles: Rs 20-40 LPA (Top tech companies)

Experienced roles: Rs 40-80+ LPA (FAANG/Quant)

International: \$150k-400k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What do I need to know before joining?

A: You should be able to write simple programs: loops, conditions and functions in any language. C++ and the STL are taught from the first weeks of the course, so prior C++ experience is not required. If you have never programmed at all, a basic programming course first will make this one far more productive.

Q: Which contests and platforms does the course train for?

A: Training is built around Codeforces, with regular practice on CodeChef, AtCoder and interview style problem sets. The 6-month track prepares you for ICPC style contests, rating climbs on Codeforces and the timed coding rounds companies use for hiring.

Q: Why is the course taught in C++?

A: C++ with the STL is the standard language of serious competitive programming: it runs fast enough for strict time limits and the STL hands you tested data structures like sets, maps and priority queues. You learn modern C++ and STL fluency early, then spend the rest of the course on algorithms and problem solving.

Q: Will competitive programming help with placements and interviews?

A: Directly, because online assessments at product companies are essentially contest problems with a timer. Over 6 months you solve timed problems across dynamic programming, graphs, greedy and more, which is exactly what those rounds test. We do not guarantee a rating or a job offer; we train the skill honestly and you can watch your contest graph move.

Q: I already practice on my own. What does a live course add?

A: Solo practice plateaus because nobody shows you the pattern you missed. In live classes a mentor upsolves contests with you, compares your submission against the intended solution, assigns problems at the edge of your current level and reviews your code for speed and

cleanliness. That guided loop is what moves ratings.

Q: Is there a trial class?

A: Yes, your first class is a free live demo with zero commitment, booked at learn.modernagecoders.com/contact. We hold a 4.9 star rating across 547+ Google reviews, and the demo is your chance to judge the teaching for yourself.

Q: What is the class format?

A: All sessions are live 1-hour online classes with an instructor; nothing is pre-recorded. You solve problems during class, discuss approaches and get immediate feedback, with evening and weekend slots that fit around college schedules and semester exams.

Q: Is there a certificate?

A: You get a completion certificate at the end of the program. Alongside it you will have a public contest profile and a GitHub-ready record of solved problems and implementations, which speaks louder than any certificate in coding interviews.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/competitive-programming-masterclass-college>

=====

COURSE 37 OF 118: COMPETITIVE PROGRAMMING FOR TEENS: ZCO, INOI AND USACO TRACK

=====

Course link: <https://learn.modernagecoders.com/courses/competitive-programming-for-teens-course>

Subject area: Competitive Programming

Designed for: Teens

Who can join: Teens aged 13 to 18 who can already write basic programs and want contest-level problem solving

Course length: 6 months (24 weeks), joinable any month

Time needed: 2 live classes/week + weekly problem sets and virtual contests

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online competitive programming for teenagers: algorithms, data structures and contest technique aimed at the Indian informatics olympiad path and USACO, taught in C++ or Python.

In one line: Real algorithmic problem solving, aimed squarely at the informatics olympiad path and USACO, taught by solving problems rather than watching them being solved.

ABOUT THE PROGRAMME

Competitive programming is the fastest way a teenager can become genuinely good at algorithms, because a contest gives you an honest verdict in seconds. Your solution is correct and fast enough, or it is not. Nothing else in school works quite like that. This course takes a student who can already write basic programs and builds the actual competitive toolkit: complexity analysis, the core data structures, the standard algorithm families, and the contest craft that separates a solved problem from a time-limit-exceeded verdict. In India, the informatics olympiad path runs through zonal qualifiers, ZCO and ZIO, into INOI, the Indian National Olympiad in Informatics, and onward through the training camp toward the International Olympiad in Informatics. Internationally, USACO runs its own ladder divisions from Bronze up through Silver, Gold and Platinum. Both reward the same underlying skill, so we train for the skill and prepare for both formats. Students work in C++ or Python. C++ is the pragmatic default at higher levels because of its speed and its standard template library, and we teach it properly for students who want to go far. Python is fully supported for students who prefer it and is entirely adequate for the earlier stages. Every week ends with a timed virtual contest and a proper upsolve, because the problems you failed to solve during the contest are where nearly all the improvement actually comes from.

WHAT MAKES THIS COURSE DIFFERENT

- Aimed at the real contest ladder: zonal qualifiers and INOI in India, and the USACO divisions internationally
- Taught by solving, not by watching: every class has students writing code against a judge
- Complexity analysis from week one, because most failed submissions are correct but too slow
- Weekly timed virtual contests followed by a structured upsolve, which is where the actual improvement happens
- C++ taught properly for students aiming high, Python fully supported for those who prefer it
- Live, small batches with solutions reviewed line by line, not a video library

THE LEARNING PATH, STEP BY STEP

Phase 1: Complexity, the core data structures, sorting and searching, and reading a problem

statement correctly

Phase 2: The standard algorithm families: greedy, two pointers, prefix sums, recursion, graphs and dynamic programming

Phase 3: Contest craft, harder problem sets, and full timed contests with upsolve discipline

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS AND CORE DATA STRUCTURES

The tools every contest problem assumes you already have, built until they are automatic.

Month 1: Complexity, Arrays and Strings (Weeks 1-4)

Setup, Judges and Big-O

Topics:

- Setting up a contest environment and submitting to an online judge
- Reading a problem statement precisely, including the constraints
- Time and space complexity, and reading the limits to infer the intended solution
- Why an $O(n^2)$ solution fails when n is 200,000
- Fast input and output, which silently costs beginners real points

Projects built:

- Submit your first ten problems and read every verdict, including the failures

Practice: 15 warm-up problems with a written complexity estimate before each submission

Arrays, Sorting and Binary Search

Topics:

- Sorting with custom comparators
- Binary search on an array, and the off-by-one errors that ruin it
- Binary search on the answer, a technique that unlocks many problems
- Two pointers and the sliding window
- Recognising which of these a statement is hinting at

Projects built:

- Solve a set of problems where binary search on the answer is the intended solution

Practice: 20 sorting and searching problems on a judge

Prefix Sums, Strings and Number Theory Basics

Topics:

- Prefix sums and difference arrays for range queries
- String basics: frequency counts, palindromes, simple hashing ideas
- GCD, modular arithmetic and why answers are taken modulo a prime
- Sieve of Eratosthenes and simple primality
- Avoiding integer overflow, a classic silent failure

Projects built:

- Build a small library of your own tested helper functions

Practice: 20 problems across prefix sums, strings and basic number theory

Assessment: First timed virtual contest, followed by a full upsolve

Stacks, Queues and the Standard Library

Topics:

- Stacks and queues and the problems that signal them
- Deques and priority queues
- Using the C++ STL containers properly, or Python's equivalents
- Choosing the right container for the operation you repeat most
- Writing clean code fast, since contests are timed

Projects built:

- Solve a monotonic stack problem and explain why the stack is the right structure

Practice: 20 data-structure problems

Month 2: Recursion, Backtracking and First Graphs (Weeks 5-8)

Recursion and Backtracking

Topics:

- Writing recursion you can reason about
- The recursion tree and its cost
- Backtracking with pruning
- Generating permutations and subsets
- When recursion is elegant and when it is simply too slow

Projects built:

- Solve a classic constraint problem with backtracking and measure the effect of each pruning rule

Practice: 15 recursion and backtracking problems

Graph Representation, BFS and DFS

Topics:

- Adjacency lists versus matrices, and when each is right
- Depth-first search and its uses
- Breadth-first search and shortest paths on unweighted graphs
- Connected components and cycle detection
- Recognising a graph problem that is not described as one

Projects built:

- Model a described real-world scenario as a graph and solve it with BFS

Practice: 20 graph traversal problems

Greedy Algorithms

Topics:

- What makes a greedy choice valid
- Classic greedy problems: intervals, scheduling, coin-style problems
- Proving a greedy solution correct, at least informally
- Why an untested greedy hunch is the most common wrong answer
- Knowing when greedy fails and DP is required

Projects built:

- Solve an interval-scheduling set and write a short argument for why the greedy rule works

Practice: 20 greedy problems with written justifications

Assessment: Timed contest plus upsolve, with an error log started

Contest Technique

Topics:

- Reading the whole problem set first and ranking by difficulty
- Deciding fast whether to commit to a problem or move on
- Writing test cases before submitting
- Debugging under time pressure
- Handling a wrong answer without panicking

Projects built:

- A full contest run with a written post-mortem of every decision

Practice: Two timed contests with structured upsolve

PHASE 2: THE CORE ALGORITHM FAMILIES

The material that separates a zonal qualifier from a national-level contestant.

Month 3: Dynamic Programming (Weeks 9-12)

Introduction to Dynamic Programming

Topics:

- Recognising overlapping subproblems and optimal substructure
- Memoisation versus bottom-up tabulation
- Defining the state, which is the whole difficulty
- Classic one-dimensional DP problems
- Counting the states to get the complexity right

Projects built:

- Solve the same problem both top-down and bottom-up and compare

Practice: 20 introductory DP problems

Knapsack and Grid DP

Topics:

- The knapsack family and its variants
- Two-dimensional DP on grids
- Subset-sum style problems
- Reconstructing the actual answer, not just its value
- Space optimisation with rolling arrays

Projects built:

- Solve a knapsack variant and reconstruct the chosen items

Practice: 20 knapsack and grid DP problems

DP on Sequences and Intervals

Topics:

- Longest increasing subsequence and its faster form
- Edit distance and sequence alignment
- Interval DP
- Bitmask DP as an introduction
- Spotting the state definition from the constraints

Projects built:

- Implement LIS in both the quadratic and the faster form and compare on large input

Practice: 20 sequence and interval DP problems

Assessment: Timed DP-focused contest with upsolve

Weighted Graphs and Shortest Paths

Topics:

- Dijkstra with a priority queue
- Bellman-Ford and negative edges
- Floyd-Warshall for all pairs
- Minimum spanning trees
- Disjoint set union and its uses

Projects built:

- Solve a routing problem with Dijkstra and justify the data structures chosen

Practice: 20 weighted-graph problems

PHASE 3: CONTEST MASTERY AND OLYMPIAD PREPARATION

Harder problems and the discipline that turns knowledge into a rank.

Months 4 to 6: Advanced Topics and Full Contests (Weeks 13-24)

Advanced Structures and Techniques

Topics:

- Segment trees and range queries
- Binary indexed trees
- Trees: rooting, depth, subtree sums and lowest common ancestor
- String algorithms at contest level
- Choosing the simplest structure that is fast enough

Projects built:

- Implement a segment tree from scratch and use it on a real range-query problem

Practice: 30 problems across advanced structures

Olympiad-Style Problem Sets

Topics:

- Working past problems in the style of the zonal rounds and INOI
- USACO division problems from Bronze upward
- Multi-part problems and partial scoring
- Managing a long contest without losing focus
- Building a personal template and checklist

Projects built:

- Complete full past sets under time and grade them honestly

Practice: Weekly full-length problem sets with structured upsolve

Full Contests, Upsolve and Peak Form

Topics:

- Weekly full timed contests under real conditions
- The upsolve discipline: every unsolved problem revisited until solved unaided
- Reading the error log to find your actual weakness
- Contest-day routine and nerves
- Choosing the right target contests for your level

Projects built:

- A full contest every week with a written post-mortem

Practice: Continuous contest and upsolve cycle

Assessment: Final rated contest performance reviewed against your starting baseline

WHERE THIS CAN LEAD

- Readiness for zonal qualifiers, INOI and USACO divisions
- Algorithmic strength that makes school computer science straightforward
- A real advantage in university applications and technical interviews
- A foundation for research or engineering in any quantitative field
- Problem-solving stamina that transfers well beyond programming

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which contests does this prepare for?

A: Internationally we prepare students for the USACO divisions from Bronze upward. Both test the same underlying algorithmic skill, so the training serves either.

Q: Should my child use C++ or Python?

A: Both are supported. Python is entirely adequate for the earlier stages and many students start there. C++ is the pragmatic choice at higher levels because of its speed and standard template library, and time limits eventually make that matter. We teach C++ properly for students aiming at the later rounds, and we never force a switch before it is useful.

Q: Does my child need to be a strong programmer already?

A: They need working basics: loops, conditionals, functions and arrays. They do not need any algorithms background, which is what the course builds. What matters far more than existing skill is willingness to solve problems between classes, because in competitive programming practice volume beats natural talent almost every time.

Q: What is upsolving and why does it matter so much?

A: Upsolving means going back after a contest and solving the problems you could not solve during it, until you can do them unaided. It is where nearly all real improvement happens, and it is the single habit that separates students who plateau from students who keep climbing. Every contest in this course is followed by a structured upsolve.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/competitive-programming-for-teens-course>

Course link: <https://learn.modernagecoders.com/courses/data-analytics-mathematics-masterclass>
Subject area: Data Science & Analytics Mathematics
Designed for: Working professionals
Who can join: Basic Math Knowledge to Advanced Data Science Mathematics
Course length: 24 months (104 weeks)
Time needed: 20-25 hours/week recommended
Certificate: Data Science Mathematics Expert Certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

The most comprehensive 2-year data science mathematics program. Master statistics, probability, linear algebra for ML, optimization, deep learning mathematics, Bayesian methods, causal inference, and cutting-edge analytics techniques. Build the mathematical foundation for AI and data science.

In one line: From Statistical Basics to Advanced AI Mathematics - Master the Math Behind Data Science

ABOUT THE PROGRAMME

This intensive 2-year program provides complete mathematical foundations for data science, machine learning, and AI. Whether you're aspiring to be a data scientist, ML engineer, quantitative analyst, or AI researcher, this masterclass gives you the deep mathematical understanding needed for excellence. You'll master statistical thinking, probability theory, linear algebra for ML, optimization methods, deep learning mathematics, Bayesian inference, causal analysis, and more. Learn not just formulas but the intuition behind algorithms. By completion, you'll understand the mathematics behind every major data science and ML technique.

WHAT MAKES THIS COURSE DIFFERENT

- Specifically designed for data science applications
- Heavy emphasis on practical implementation
- Python/R code alongside mathematical concepts
- Real datasets and industry case studies
- Covers both classical and modern techniques
- Deep learning and AI mathematics included
- Interview preparation for data science roles
- Industry-relevant projects throughout

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Statistics, Probability, Linear Algebra Essentials
Phase 2: Core Analytics (Months 7-12): Statistical Inference, Regression, Classification, Clustering
Phase 3: Advanced ML Math (Months 13-18): Optimization, Deep Learning, Bayesian Methods
Phase 4: Cutting Edge (Months 19-24): Causal Inference, Time Series, Big Data, Research

WHAT A STUDENT NEEDS BEFORE STARTING

Mathematics: High school algebra and basic statistics helpful
Programming: Basic Python/R knowledge beneficial but not required
Statistics: No prior statistics knowledge required
Commitment: Strong dedication to learning
Equipment: Computer with internet, cloud credits provided
Software: All software and platforms provided

WHO THIS COURSE IS FOR

Aspiring data scientists: Those starting a data science career
Analysts: Business/Data analysts seeking deeper skills
Engineers: Software engineers moving to ML/AI
Researchers: Academic researchers in any field
Professionals: Domain experts adding data skills
Students: Undergraduate/graduate students
Career switchers: Professionals transitioning to data science

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: MATHEMATICAL FOUNDATIONS FOR DATA SCIENCE (MONTHS 1-6, WEEKS 1-26) -----

Build rock-solid foundations in statistics, probability, and linear algebra essential for data science.

Months 1-2: Descriptive Statistics & Data Fundamentals (Week 1-8)

Introduction to Statistical Thinking

Topics:

- What is data science? The role of mathematics
- Types of data: numerical, categorical, ordinal
- Scales of measurement: nominal, ordinal, interval, ratio
- Population vs sample concepts
- Parameters vs statistics
- Data collection methods and sampling techniques
- Simple random sampling
- Stratified and cluster sampling
- Sampling bias and selection bias
- Data quality: missing data, outliers, errors
- Data cleaning principles
- Exploratory Data Analysis (EDA) philosophy

Projects built:

- Design a data collection strategy
- Data quality assessment tool
- Sampling simulation study

Practice: Analyze 20 real-world datasets for quality issues

Measures of Central Tendency and Spread

Topics:

- Mean: arithmetic, geometric, harmonic
- Median and quartiles
- Mode and multimodal distributions
- Weighted averages and their applications
- Range and interquartile range
- Variance and standard deviation
- Coefficient of variation
- Mean absolute deviation
- Moments: skewness and kurtosis
- Box plots and five-number summary
- Z-scores and standardization
- Detecting outliers: IQR method, z-score method

Projects built:

- Statistical calculator from scratch
- Outlier detection system
- Interactive visualization dashboard

Practice: Calculate statistics for 50 different distributions

Data Visualization and Graphical Analysis

Topics:

- Principles of effective visualization
- Histograms and density plots
- Scatter plots and correlation patterns
- Bar charts and categorical data visualization
- Pie charts: when to use and avoid
- Heat maps and correlation matrices
- Time series plots and trends
- Parallel coordinates for high-dimensional data
- Q-Q plots and probability plots
- Interactive visualizations with Plotly
- Dashboard design principles
- Misleading visualizations and how to spot them

Projects built:

- Build comprehensive EDA toolkit
- Create interactive data dashboard
- Visualization best practices guide

Practice: Create 100 different types of visualizations

Probability Fundamentals

Topics:

- Sample spaces and events
- Classical, frequentist, and subjective probability
- Probability axioms and properties
- Conditional probability and independence
- Bayes' theorem and applications
- Law of total probability
- Combinatorics for probability
- Permutations and combinations in data
- Birthday paradox and probability paradoxes
- Monte Carlo simulation basics
- Random number generation
- Probability in machine learning context

Projects built:

- Probability simulator
- Bayes theorem calculator
- Monte Carlo estimation tool

Practice: Solve 150 probability problems with data applications

Months 3-4: Probability Distributions & Linear Algebra Basics (Week 9-17)

Discrete Probability Distributions

Topics:

- Random variables: discrete vs continuous
- Probability mass functions
- Bernoulli and binomial distributions
- Geometric and negative binomial distributions
- Poisson distribution and rare events
- Hypergeometric distribution
- Multinomial distribution
- Expected value and variance calculations
- Moment generating functions
- Joint discrete distributions
- Marginal and conditional distributions
- Applications in A/B testing

Projects built:

- Distribution calculator and visualizer
- A/B test simulator
- Discrete event simulator

Practice: Work with 100 discrete distribution problems

Continuous Probability Distributions

Topics:

- Probability density functions
- Uniform distribution
- Normal distribution: properties and importance
- Standard normal and z-tables
- Exponential distribution and memoryless property
- Gamma and beta distributions
- Chi-square distribution
- Student's t-distribution
- F-distribution
- Log-normal distribution
- Weibull distribution for reliability
- Distribution fitting and selection

Projects built:

- Distribution fitting tool
- Normal distribution simulator
- Reliability analysis system

Practice: Master 100 continuous distribution applications

Linear Algebra for Data Science - Vectors

Topics:

- Vectors as data points
- Vector operations: addition, scalar multiplication
- Dot product and cosine similarity
- Vector norms: L1, L2, L-infinity
- Distance metrics: Euclidean, Manhattan, Minkowski
- Orthogonality and orthogonal projections
- Vector spaces and subspaces
- Linear independence in feature space
- Basis vectors and coordinate systems
- Change of basis for data transformation
- Gram-Schmidt for orthogonalization
- Applications to feature engineering

Projects built:

- Vector similarity calculator
- Distance metric visualizer
- Feature space explorer

Practice: Complete 100 vector operations in data context

Linear Algebra for Data Science - Matrices

Topics:

- Matrices as datasets and transformations
- Matrix multiplication as data transformation
- Special matrices: diagonal, symmetric, orthogonal
- Matrix transpose and properties
- Matrix rank and linear independence of features
- Invertible matrices and their meaning
- Determinants and volume interpretation
- Systems of linear equations in ML
- Gaussian elimination for solving systems
- LU decomposition
- Matrix norms and condition numbers
- Sparse matrices in big data

Projects built:

- Matrix operations library
- Linear system solver
- Sparse matrix handler

Practice: Solve 100 matrix problems with data applications

Eigenvalues and PCA Foundations

Topics:

- Eigenvalues and eigenvectors intuition
- Characteristic equation
- Geometric interpretation of eigenvectors
- Diagonalization of matrices
- Spectral decomposition
- Positive definite matrices
- Covariance matrices in data
- Principal Component Analysis (PCA) theory
- PCA algorithm step-by-step
- Variance explained and scree plots
- PCA for dimensionality reduction
- Applications in data compression

Projects built:

- PCA implementation from scratch
- Eigenface recognition system
- Dimensionality reduction tool

Practice: Apply PCA to 20 different datasets

Months 5-6: Statistical Inference & Hypothesis Testing (Week 18-26)

Sampling Distributions and CLT

Topics:

- Sampling distribution concept
- Sampling distribution of the mean
- Standard error and its importance
- Central Limit Theorem (CLT)
- CLT applications and limitations
- Sample size determination
- Finite population correction
- Sampling distribution of proportions
- Sampling distribution of variances
- Chi-square distribution from normal samples
- t-distribution for small samples
- F-distribution for variance ratios

Projects built:

- CLT demonstration tool
- Sample size calculator
- Sampling distribution simulator

Practice: Explore 50 sampling distribution scenarios

Confidence Intervals

Topics:

- Point estimates vs interval estimates
- Confidence level interpretation
- CI for population mean (known variance)
- CI for population mean (unknown variance)
- CI for population proportion
- CI for difference of means
- CI for paired differences
- CI for variance and standard deviation
- CI for ratio of variances
- Bootstrap confidence intervals
- Prediction intervals vs confidence intervals
- Sample size for desired margin of error

Projects built:

- Confidence interval calculator
- Bootstrap CI implementation
- CI visualization tool

Practice: Calculate 100 different confidence intervals

Hypothesis Testing Fundamentals

Topics:

- Null and alternative hypotheses
- Type I and Type II errors
- Significance level and p-values
- Test statistics and rejection regions
- One-sample t-test
- Two-sample t-test (equal and unequal variances)
- Paired t-test
- Z-test for proportions
- Chi-square test for variance
- F-test for equal variances
- One-tailed vs two-tailed tests
- Power of a test and effect size

Projects built:

- Hypothesis testing framework
- Power analysis tool
- A/B testing platform

Practice: Conduct 100 hypothesis tests

ANOVA and Multiple Comparisons

Topics:

- One-way ANOVA principles

- F-statistic and ANOVA table
- Assumptions of ANOVA
- Post-hoc tests: Tukey, Bonferroni, Scheffé
- Two-way ANOVA
- Interaction effects
- Repeated measures ANOVA
- MANOVA basics
- Kruskal-Wallis test (nonparametric)
- Multiple testing problem
- False Discovery Rate (FDR)
- Benjamini-Hochberg procedure

Projects built:

- ANOVA analysis suite
- Multiple comparison visualizer
- FDR control system

Practice: Perform 50 ANOVA analyses

Phase 1 Capstone Project

Topics:

- Complete statistical analysis pipeline
- Integration of all Phase 1 concepts
- Report writing and presentation
- Reproducible research practices
- Statistical consulting simulation

Projects built:

- MAJOR CAPSTONE: End-to-End Statistical Analysis
- Complete EDA and inference on real dataset
- Build statistical analysis dashboard
- Create statistical consulting report

Assessment: Phase 1 Comprehensive Exam - Foundations

PHASE 2: CORE ANALYTICS & MACHINE LEARNING MATHEMATICS (MONTHS 7-12, WEEKS 27-52)

Master regression, classification, clustering, and core machine learning mathematics.

Months 7-8: Regression Analysis (Week 27-35)

Simple Linear Regression

Topics:

- Linear relationship between variables
- Least squares estimation
- Geometric interpretation of least squares
- Normal equations derivation
- Properties of least squares estimators
- Gauss-Markov theorem
- R-squared and adjusted R-squared
- Residual analysis
- Assumptions of linear regression
- Diagnostic plots: Q-Q, residual plots
- Outliers and influential points
- Confidence and prediction intervals for regression

Projects built:

- Linear regression from scratch
- Regression diagnostics tool
- Outlier detection system

Practice: Build 50 linear regression models

Multiple Linear Regression

Topics:

- Multiple regression model
- Matrix formulation of regression
- Partial regression coefficients
- Multicollinearity detection and handling
- Variance Inflation Factor (VIF)
- Feature selection methods
- Forward, backward, stepwise selection
- All subsets regression
- Adjusted R-squared and model selection
- AIC, BIC, Mallows' Cp
- Cross-validation for model selection
- Interaction terms and polynomial regression

Projects built:

- Multiple regression toolkit
- Feature selection system
- Model comparison framework

Practice: Develop 50 multiple regression models

Regularized Regression

Topics:

- Overfitting and bias-variance tradeoff
- Ridge regression (L2 regularization)
- Ridge regression geometry

- LASSO regression (L1 regularization)
- LASSO for feature selection
- Elastic Net regression
- Choosing regularization parameters
- Cross-validation for lambda selection
- Bayesian interpretation of regularization
- Group LASSO
- Adaptive LASSO
- Applications in high-dimensional data

Projects built:

- Regularized regression implementation
- Lambda tuning system
- High-dimensional regression tool

Practice: Apply regularization to 40 datasets

Generalized Linear Models

Topics:

- Exponential family of distributions
- Link functions and canonical links
- Logistic regression for binary outcomes
- Maximum likelihood estimation for GLMs
- Newton-Raphson and IRLS algorithms
- Odds ratios and interpretation
- Poisson regression for count data
- Negative binomial regression
- Ordinal logistic regression
- Multinomial logistic regression
- Deviance and goodness of fit
- Quasi-likelihood methods

Projects built:

- GLM framework implementation
- Logistic regression classifier
- Count data modeling tool

Practice: Build 50 different GLMs

Nonparametric Regression

Topics:

- Kernel regression
- Local polynomial regression (LOESS)
- Bandwidth selection
- Spline regression
- Smoothing splines
- Penalized splines
- Generalized Additive Models (GAMs)
- Regression trees basics
- K-nearest neighbors regression
- Gaussian Process regression introduction
- Quantile regression
- Robust regression methods

Projects built:

- Nonparametric regression suite
- GAM implementation
- Smoothing parameter selector

Practice: Compare 30 parametric vs nonparametric models

Months 9-10: Classification & Clustering (Week 36-44)

Classification Fundamentals

Topics:

- Classification vs regression
- Bayes classifier and Bayes error
- Linear Discriminant Analysis (LDA)
- Quadratic Discriminant Analysis (QDA)
- Fisher's linear discriminant
- Naive Bayes classifier
- Gaussian Naive Bayes
- Multinomial and Bernoulli Naive Bayes
- K-Nearest Neighbors classification
- Distance metrics for KNN
- Decision boundaries visualization
- Class imbalance problems

Projects built:

- Classifier comparison framework
- Decision boundary visualizer
- Imbalanced data handler

Practice: Implement 50 classification models

Tree-Based Methods

Topics:

- Decision trees for classification
- Entropy and information gain
- Gini impurity
- Tree pruning methods

- Cost complexity pruning
- Random Forests algorithm
- Out-of-bag error estimation
- Feature importance from trees
- Extremely Randomized Trees
- Gradient Boosting Machines (GBM)
- XGBoost mathematics
- LightGBM and CatBoost

Projects built:

- Decision tree from scratch
- Random Forest implementation
- Boosting algorithm suite

Practice: Build 40 tree-based models

Support Vector Machines

Topics:

- Maximum margin classifiers
- Hard margin SVM
- Soft margin SVM and slack variables
- Lagrangian formulation
- Dual problem and support vectors
- Kernel trick and Mercer's theorem
- Common kernels: RBF, polynomial, sigmoid
- Multi-class SVM strategies
- SVM for regression (SVR)
- Nu-SVM and One-class SVM
- Kernel selection and tuning
- SMO algorithm basics

Projects built:

- SVM implementation from scratch
- Kernel comparison tool
- SVM hyperparameter tuner

Practice: Train 50 SVM models with different kernels

Clustering Algorithms

Topics:

- K-means clustering algorithm
- K-means++ initialization
- Elbow method and silhouette analysis
- Hierarchical clustering
- Agglomerative vs divisive
- Linkage methods: single, complete, average
- Dendrograms and cutting trees
- DBSCAN algorithm
- Mean Shift clustering
- Gaussian Mixture Models (GMM)
- EM algorithm for GMMs
- Spectral clustering

Projects built:

- Clustering algorithm suite
- Cluster validation tools
- Optimal K finder

Practice: Apply clustering to 40 datasets

Model Evaluation and Selection

Topics:

- Confusion matrix and derived metrics
- Precision, recall, F1-score
- ROC curves and AUC
- Precision-recall curves
- Multi-class metrics
- Cross-validation strategies
- Stratified and time series CV
- Nested cross-validation
- Bootstrap validation
- Learning curves
- Validation curves
- Grid search and random search

Projects built:

- Model evaluation framework
- Hyperparameter optimization tool
- Model comparison dashboard

Practice: Evaluate 50 models comprehensively

Months 11-12: Optimization & Neural Network Mathematics (Week 45-52)

Optimization Theory for ML

Topics:

- Convex sets and convex functions
- Convexity in machine learning
- Gradient descent algorithm
- Learning rate selection
- Momentum and Nesterov acceleration

- AdaGrad and RMSprop
- Adam and AdamW optimizers
- Second-order methods: Newton, L-BFGS
- Stochastic gradient descent (SGD)
- Mini-batch gradient descent
- Convergence analysis
- Constrained optimization for ML

Projects built:

- Optimizer implementations
- Convergence visualizer
- Learning rate scheduler

Practice: Implement 20 optimization algorithms

Neural Network Fundamentals

Topics:

- Perceptron algorithm
- Multi-layer perceptrons
- Universal approximation theorem
- Activation functions: sigmoid, tanh, ReLU
- Forward propagation mathematics
- Backpropagation derivation
- Chain rule in neural networks
- Gradient vanishing and exploding
- Weight initialization strategies
- Xavier and He initialization
- Batch normalization mathematics
- Dropout as regularization

Projects built:

- Neural network from scratch
- Backpropagation visualizer
- Activation function explorer

Practice: Build 30 neural network architectures

Convolutional Neural Networks Mathematics

Topics:

- Convolution operation in 1D and 2D
- Padding and stride calculations
- Pooling operations: max, average
- Parameter sharing and translation invariance
- Receptive field calculations
- Backpropagation through convolutions
- Popular architectures: LeNet, AlexNet, VGG
- ResNet and skip connections
- Inception modules
- Depthwise separable convolutions
- Transfer learning mathematics
- Feature map visualization

Projects built:

- CNN implementation from scratch
- Convolution visualizer
- Architecture comparison tool

Practice: Implement 20 CNN architectures

Recurrent Neural Networks Mathematics

Topics:

- Vanilla RNN formulation
- Backpropagation through time (BPTT)
- Gradient clipping
- Long Short-Term Memory (LSTM)
- LSTM gates mathematics
- Gated Recurrent Units (GRU)
- Bidirectional RNNs
- Encoder-decoder architectures
- Attention mechanism basics
- Sequence-to-sequence models
- Applications to time series
- RNN regularization techniques

Projects built:

- RNN/LSTM from scratch
- Sequence prediction tool
- Attention visualizer

Practice: Build 20 RNN models

Phase 2 Capstone Project

Topics:

- End-to-end ML pipeline
- Model deployment considerations
- A/B testing for ML
- Model monitoring
- ML system design

Projects built:

- MAJOR CAPSTONE: Complete ML System
- Build production ML pipeline

- Implement model monitoring
- Create ML API service

Assessment: Phase 2 Comprehensive Exam - ML Mathematics

 PHASE 3: ADVANCED ANALYTICS & DEEP LEARNING (MONTHS 13-18, WEEKS 53-78)

Master Bayesian methods, deep learning, reinforcement learning, and advanced statistical techniques.

Months 13-14: Bayesian Statistics & Inference (Week 53-61)

Bayesian Fundamentals

Topics:

- Bayesian vs Frequentist philosophy
- Prior distributions and their selection
- Likelihood functions
- Posterior distributions
- Conjugate priors
- Beta-Binomial model
- Normal-Normal model
- Gamma-Poisson model
- Credible intervals vs confidence intervals
- Bayesian point estimates
- MAP vs MLE estimates
- Empirical Bayes methods

Projects built:

- Bayesian inference engine
- Prior selection tool
- Posterior calculator

Practice: Solve 100 Bayesian inference problems

Markov Chain Monte Carlo

Topics:

- Monte Carlo integration review
- Importance sampling
- Markov chains for sampling
- Metropolis algorithm
- Metropolis-Hastings algorithm
- Gibbs sampling
- Convergence diagnostics
- Gelman-Rubin statistic
- Effective sample size
- Hamiltonian Monte Carlo
- NUTS sampler
- Variational inference basics

Projects built:

- MCMC sampler implementation
- Convergence diagnostic suite
- HMC from scratch

Practice: Implement 20 MCMC algorithms

Bayesian Regression and Classification

Topics:

- Bayesian linear regression
- Predictive distributions
- Bayesian model averaging
- Spike and slab priors
- Horseshoe prior
- Bayesian logistic regression
- Probit regression
- Gaussian processes for regression
- Kernel selection in GPs
- Sparse Gaussian processes
- Bayesian neural networks
- Uncertainty quantification

Projects built:

- Bayesian regression toolkit
- GP implementation
- Uncertainty visualization

Practice: Build 50 Bayesian models

Hierarchical and Mixture Models

Topics:

- Hierarchical Bayesian models
- Random effects models
- Mixed effects models
- Nested data structures
- Shrinkage and pooling
- Dirichlet process mixtures
- Infinite mixture models
- Chinese Restaurant Process
- Stick-breaking construction

- Latent Dirichlet Allocation
- Topic modeling
- Hidden Markov Models

Projects built:

- Hierarchical model builder
- Topic modeling system
- HMM implementation

Practice: Develop 30 hierarchical models

Bayesian Model Selection

Topics:

- Bayes factors
- Model evidence and marginal likelihood
- Savage-Dickey density ratio
- DIC and WAIC
- LOO cross-validation
- Posterior predictive checks
- Model comparison strategies
- Bayesian hypothesis testing
- Reversible jump MCMC
- Bayesian variable selection
- Bayesian model averaging
- Ensemble methods in Bayesian context

Projects built:

- Model comparison framework
- Bayes factor calculator
- Model selection tool

Practice: Compare 40 Bayesian models

Months 15-16: Deep Learning & Advanced Architectures (Week 62-70)

Advanced Deep Learning Theory

Topics:

- Deep learning optimization landscape
- Loss surface geometry
- Mode connectivity
- Lottery ticket hypothesis
- Neural tangent kernels
- Double descent phenomenon
- Implicit regularization
- Neural architecture search
- AutoML for deep learning
- Meta-learning basics
- Few-shot learning
- Self-supervised learning

Projects built:

- NAS implementation
- Meta-learning framework
- Self-supervised trainer

Practice: Explore 30 advanced DL concepts

Transformers and Attention

Topics:

- Self-attention mechanism
- Multi-head attention
- Positional encoding
- Transformer architecture
- BERT and GPT architectures
- Vision Transformers (ViT)
- Cross-attention mechanisms
- Efficient attention variants
- Linear attention
- Sparse transformers
- Transformer training dynamics
- Large language model mathematics

Projects built:

- Transformer from scratch
- Attention mechanism visualizer
- Mini-BERT implementation

Practice: Build 20 transformer models

Generative Models

Topics:

- Variational Autoencoders (VAE)
- ELBO derivation
- Reparameterization trick
- Beta-VAE and disentanglement
- Generative Adversarial Networks (GANs)
- Wasserstein GAN
- GAN training dynamics
- Mode collapse solutions
- Normalizing flows
- Diffusion models mathematics

- Score matching
- Energy-based models

Projects built:

- VAE implementation
- GAN from scratch
- Diffusion model builder

Practice: Train 30 generative models

Graph Neural Networks

Topics:

- Graph representation learning
- Message passing neural networks
- Graph Convolutional Networks (GCN)
- GraphSAGE algorithm
- Graph Attention Networks (GAT)
- Spectral graph convolutions
- Graph pooling methods
- Link prediction
- Node classification
- Graph classification
- Knowledge graph embeddings
- Geometric deep learning

Projects built:

- GNN implementation suite
- Graph embedding visualizer
- Knowledge graph builder

Practice: Apply GNNs to 20 graph datasets

Reinforcement Learning Mathematics

Topics:

- Markov Decision Processes
- Bellman equations
- Value iteration
- Policy iteration
- Q-learning algorithm
- Deep Q-Networks (DQN)
- Policy gradient methods
- REINFORCE algorithm
- Actor-Critic methods
- Proximal Policy Optimization (PPO)
- Soft Actor-Critic (SAC)
- Multi-armed bandits

Projects built:

- RL environment builder
- Q-learning implementation
- Policy gradient trainer

Practice: Solve 20 RL problems

Months 17-18: Time Series & Causal Inference (Week 71-78)

Time Series Analysis

Topics:

- Time series components: trend, seasonality, noise
- Stationarity and unit root tests
- Autocorrelation and partial autocorrelation
- ARIMA models
- Box-Jenkins methodology
- Seasonal ARIMA (SARIMA)
- State space models
- Kalman filtering
- Exponential smoothing methods
- Holt-Winters method
- STL decomposition
- Time series cross-validation

Projects built:

- Time series toolkit
- ARIMA model builder
- Forecast evaluator

Practice: Analyze 50 time series

Advanced Time Series Methods

Topics:

- Vector Autoregression (VAR)
- Cointegration and error correction
- GARCH models for volatility
- Long memory models (ARFIMA)
- Regime switching models
- Dynamic Factor Models
- Spectral analysis
- Wavelet analysis
- Deep learning for time series
- LSTM for forecasting
- Temporal Convolutional Networks

- Prophet algorithm

Projects built:

- Advanced TS model suite
- Volatility forecaster
- DL time series framework

Practice: Implement 30 advanced TS models

Causal Inference

Topics:

- Correlation vs causation
- Potential outcomes framework
- Average Treatment Effect (ATE)
- Randomized experiments
- Selection bias and confounding
- Propensity score matching
- Inverse probability weighting
- Doubly robust estimation
- Instrumental variables
- Regression discontinuity
- Difference-in-differences
- Synthetic control methods

Projects built:

- Causal inference toolkit
- Matching algorithm suite
- Treatment effect estimator

Practice: Apply causal methods to 40 datasets

Structural Equation Modeling

Topics:

- Path analysis
- Confirmatory factor analysis
- Structural equation models
- Latent variable models
- Measurement models
- Model identification
- Maximum likelihood for SEM
- Fit indices and model evaluation
- Multi-group SEM
- Growth curve models
- Mediation and moderation
- Directed Acyclic Graphs (DAGs)

Projects built:

- SEM implementation
- DAG builder
- Mediation analyzer

Practice: Build 20 structural models

Phase 3 Capstone Project

Topics:

- Advanced analytics pipeline
- Deep learning deployment
- Causal analysis project
- Research paper writing
- Industry presentation

Projects built:

- MAJOR CAPSTONE: Advanced Analytics System
- Deploy production DL model
- Conduct causal analysis study
- Write research-quality paper

Assessment: Phase 3 Comprehensive Exam - Advanced Analytics

PHASE 4: BIG DATA, PRODUCTION SYSTEMS & RESEARCH (MONTHS 19-24, WEEKS 79-104)

Master big data analytics, production ML systems, experimental design, and conduct original research.

Months 19-20: Big Data & Distributed Analytics (Week 79-87)

Big Data Fundamentals

Topics:

- Big data characteristics: 5 V's
- Distributed computing principles
- CAP theorem and data consistency
- MapReduce paradigm
- Hadoop ecosystem overview
- HDFS architecture
- Apache Spark fundamentals
- RDDs and DataFrames
- Spark SQL and optimization
- Spark MLlib algorithms
- Streaming data concepts
- Lambda and Kappa architectures

Projects built:

- Distributed computing setup
- MapReduce implementation
- Spark ML pipeline

Practice: Process 20 big data workloads

Scalable Machine Learning

Topics:

- Distributed gradient descent
- Parameter server architecture
- Data parallelism vs model parallelism
- Federated learning
- Online learning algorithms
- Stochastic gradient methods at scale
- Approximate algorithms for big data
- Random sampling techniques
- Sketching algorithms
- MinHash and LSH
- Bloom filters
- Count-Min sketch

Projects built:

- Distributed ML trainer
- Federated learning system
- Sketching algorithm suite

Practice: Implement 30 scalable algorithms

Stream Processing & Real-time Analytics

Topics:

- Stream processing concepts
- Apache Kafka architecture
- Apache Flink fundamentals
- Window functions in streaming
- Watermarks and late data
- Exactly-once processing
- Complex event processing
- Real-time aggregations
- Streaming ML predictions
- Online learning in streams
- Anomaly detection in streams
- Time series databases

Projects built:

- Stream processing pipeline
- Real-time dashboard
- Anomaly detection system

Practice: Build 20 streaming applications

NoSQL and NewSQL for Analytics

Topics:

- NoSQL database types
- Document stores: MongoDB analytics
- Column stores: Cassandra, HBase
- Graph databases for analytics
- Time series databases: InfluxDB, TimescaleDB
- NewSQL systems
- Data modeling for NoSQL
- CAP theorem implications
- Consistency models
- Data warehousing concepts
- Data lakes vs data warehouses
- Apache Iceberg and Delta Lake

Projects built:

- Multi-database analytics system
- Data lake implementation
- NoSQL analytics toolkit

Practice: Design 20 data architectures

Cloud Analytics Platforms

Topics:

- Cloud computing for analytics
- AWS analytics services
- Google Cloud Platform ML tools
- Azure Machine Learning
- Serverless analytics
- Auto-scaling for ML workloads
- Cost optimization strategies
- Multi-cloud strategies
- Data governance in cloud
- Security and compliance
- MLOps in cloud
- Edge analytics

Projects built:

- Cloud ML pipeline
- Serverless analytics app

- Multi-cloud deployment
- Practice: Deploy 20 cloud analytics solutions

Months 21-22: Experimental Design & A/B Testing (Week 88-96)

Design of Experiments

Topics:

- Principles of experimental design
- Randomization, replication, blocking
- Completely randomized designs
- Randomized block designs
- Latin square designs
- Factorial designs
- 2^k factorial designs
- Fractional factorial designs
- Response surface methodology
- Central composite designs
- Optimal design theory
- Sample size calculations

Projects built:

- Experiment design tool
- Power calculator
- Optimal design finder

Practice: Design 50 experiments

A/B Testing at Scale

Topics:

- A/B testing fundamentals
- Statistical power in A/B tests
- Multiple testing corrections
- Sequential testing
- Bandits vs A/B tests
- Multi-armed bandits
- Thompson sampling
- Contextual bandits
- Variance reduction techniques
- CUPED method
- Stratification in experiments
- Network effects in testing

Projects built:

- A/B testing platform
- Bandit algorithm suite
- Variance reduction tool

Practice: Run 40 A/B tests

Quasi-Experimental Methods

Topics:

- Natural experiments
- Interrupted time series
- Regression discontinuity design
- Fuzzy RDD
- Difference-in-differences advanced
- Triple differences
- Synthetic control advanced
- Matching methods review
- Coarsened exact matching
- Genetic matching
- Sensitivity analysis
- Bounds for causal effects

Projects built:

- Quasi-experimental toolkit
- RDD analyzer
- Sensitivity analysis suite

Practice: Apply 30 quasi-experimental designs

Survival Analysis

Topics:

- Censoring and truncation
- Survival and hazard functions
- Kaplan-Meier estimator
- Nelson-Aalen estimator
- Log-rank test
- Cox proportional hazards model
- Stratified Cox models
- Time-varying covariates
- Parametric survival models
- Accelerated failure time models
- Competing risks
- Recurrent events analysis

Projects built:

- Survival analysis toolkit
- Cox model implementation
- Competing risks analyzer

Practice: Analyze 30 survival datasets

Longitudinal Data Analysis

Topics:

- Repeated measures data
- Mixed effects models review
- Generalized Estimating Equations (GEE)
- Growth curve models
- Latent growth models
- Cross-lagged panel models
- Dynamic panel data models
- Fixed effects vs random effects
- Hausman test
- Missing data in longitudinal studies
- Multiple imputation
- Attrition and selection bias

Projects built:

- Longitudinal analysis suite
- Panel data toolkit
- Imputation system

Practice: Analyze 25 longitudinal studies

Month 23: Advanced Topics & Specializations (Week 97-100)

Spatial Statistics & GIS

Topics:

- Spatial data types
- Spatial autocorrelation
- Moran's I and Geary's C
- Spatial regression models
- Kriging and interpolation
- Point pattern analysis
- Spatial clustering
- Geographically weighted regression
- Space-time models
- Disease mapping
- Environmental statistics
- GIS integration

Projects built:

- Spatial analysis toolkit
- Kriging implementation
- Disease mapping system

Practice: Analyze 20 spatial datasets

Text Analytics & NLP Mathematics

Topics:

- Text preprocessing mathematics
- TF-IDF and BM25
- Word embeddings: Word2Vec, GloVe
- Document embeddings
- Topic modeling: LDA, NMF
- Named entity recognition
- Part-of-speech tagging
- Dependency parsing
- Sentiment analysis methods
- Text classification algorithms
- Sequence labeling with CRFs
- Transformer-based NLP

Projects built:

- NLP pipeline builder
- Topic modeling system
- Sentiment analyzer

Practice: Build 30 NLP applications

Computer Vision Mathematics

Topics:

- Image processing fundamentals
- Convolution and filtering
- Edge detection algorithms
- Feature extraction: SIFT, SURF, HOG
- Image segmentation methods
- Object detection: R-CNN family
- YOLO architecture
- Semantic segmentation
- Instance segmentation
- Face recognition mathematics
- Optical flow
- 3D vision basics

Projects built:

- CV algorithm suite
- Object detector
- Segmentation tool

Practice: Implement 25 CV algorithms

Recommendation Systems

Topics:

- Collaborative filtering
- User-based vs item-based CF
- Matrix factorization methods
- Singular value decomposition
- Non-negative matrix factorization
- Alternating least squares
- Content-based filtering
- Hybrid recommendation systems
- Deep learning for recommendations
- Sequential recommendations
- Multi-stakeholder recommendations
- Evaluation metrics for RecSys

Projects built:

- Recommendation engine
- Matrix factorization suite
- Hybrid recommender

Practice: Build 20 recommendation systems

Month 24: MLOps, Ethics & Career Preparation (Week 101-104)

MLOps & Production Systems

Topics:

- ML system design patterns
- Feature engineering pipelines
- Feature stores
- Model versioning
- Experiment tracking
- Model registry
- CI/CD for ML
- Model serving architectures
- Batch vs online inference
- Model monitoring and drift detection
- A/B testing for ML models
- Rollback strategies
- Edge deployment
- Model compression techniques
- Quantization and pruning

Projects built:

- FINAL CAPSTONE: Production ML System
- Complete MLOps pipeline
- Model monitoring dashboard
- Feature store implementation

Ethics, Fairness & Interpretability

Topics:

- Bias in data and algorithms
- Fairness metrics
- Demographic parity
- Equalized odds
- Calibration
- Fair ML algorithms
- Model interpretability methods
- LIME and SHAP
- Counterfactual explanations
- Privacy-preserving ML
- Differential privacy
- Federated learning for privacy
- AI governance
- Regulatory compliance

Deliverables:

- Fairness audit toolkit
- Interpretability framework
- Privacy-preserving ML system
- Ethics guidelines document

Research & Career Development

Topics:

- Reading research papers effectively
- Reproducing research results
- Writing technical reports
- Creating data science portfolio
- Interview preparation for DS roles
- Case study methodology
- Technical presentation skills
- Open source contribution
- Kaggle competition strategies
- Networking in data science
- Continuous learning strategies
- Career paths in data science

Deliverables:

- Research paper implementation
- Complete portfolio website
- Kaggle competition submission
- Technical blog posts
- Interview preparation materials
- Career roadmap document

Assessment: FINAL COMPREHENSIVE EXAM - Complete Data Science Mathematics

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Statistical analysis dashboards, EDA tools, Probability simulators, PCA implementations
- Phase 2: ML pipelines, Classification systems, Clustering tools, Neural networks from scratch
- Phase 3: Bayesian models, Deep learning architectures, Time series forecasters, Causal analysis
- Phase 4: Big data systems, A/B testing platforms, Production ML services, Research projects

Total projects: 200+ data science projects and implementations

SKILLS THE STUDENT WILL HAVE AT THE END

- Statistics: Descriptive, Inferential, Bayesian, Frequentist, Nonparametric
- Machine Learning: Supervised, Unsupervised, Semi-supervised, Reinforcement Learning
- Deep Learning: CNNs, RNNs, Transformers, GANs, VAEs, GNNs
- Optimization: Convex, Non-convex, Stochastic, Constrained, Multi-objective
- Time Series: ARIMA, State Space, Deep Learning methods, Forecasting
- Causal Inference: RCTs, Observational studies, Natural experiments, IV methods
- Big Data: Spark, Streaming, Distributed ML, NoSQL analytics
- Production: MLOps, Monitoring, A/B testing, Model deployment
- Specialized: NLP, Computer Vision, RecSys, Spatial statistics

HOW A TYPICAL WEEK IS SPLIT

Theory lectures: 8-10 hours
 Hands on coding: 8-10 hours
 Projects: 3-4 hours
 Paper reading: 2-3 hours
 Practice problems: 2-3 hours
 Total per week: 20-25 hours

SUPPORT INCLUDED

Mentorship: 1-on-1 mentoring sessions
 Office hours: Weekly instructor office hours
 Peer collaboration: Study groups and peer review
 Industry mentors: Guest lectures from industry
 Career support: Resume review, interview prep
 Community: Active Slack/Discord community

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Final certificate: Data Science Mathematics Expert Certificate
 Specialization badges: Badges for specialized tracks
 Portfolio: Industry-ready portfolio
 LinkedIn credentials: LinkedIn verifiable certificates

WHERE THIS CAN LEAD

- Data Scientist (Entry to Senior level)
- Machine Learning Engineer
- Applied Research Scientist
- Quantitative Analyst
- AI/ML Product Manager
- Statistical Consultant
- Business Intelligence Lead
- MLOps Engineer
- Data Science Manager
- Chief Data Scientist

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 8-15 LPA / \$70-100k USD

Growing roles: Rs 15-30 LPA / \$100-150k USD

Experienced roles: Rs 30-60 LPA / \$150-250k USD; Rs 60+ LPA / \$250k+ USD

Consulting: Rs 5,000-15,000/hour

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this applied mathematics program cover?

A: It is a 24 month program in the mathematics that powers modern applied work: statistics and probability, linear algebra, statistical inference, regression and classification, optimization and neural network mathematics, then Bayesian methods, deep learning theory, time series and causal inference, closing with big data methods, experimental design and a research phase. Everything is taught against real datasets and applications, not as abstract theory.

Q: Who should take this course?

A: Students and professionals who want the mathematical engine behind data science and AI: aspiring data scientists and analysts, engineers moving into machine learning, and researchers who need rigorous statistics. Beginners are welcome; the sequence assumes only high school algebra.

Q: How much maths or programming do I need to start?

A: High school algebra is enough. Basic statistics and some Python or R help but are not required, and no prior statistics knowledge is assumed. The software and platforms used in the course are provided.

Q: How do I know I am progressing?

A: Each topic pairs theory with an applied exercise on real data, phases close with assessments, and your mentor tracks a portfolio of completed analyses and models. By phase 2 you are running full regression and classification workflows, which is a very concrete milestone.

Q: How is this different from a data science bootcamp?

A: Bootcamps teach tools; this program teaches the mathematics that makes the tools make sense, which is what separates people who can tune a model from people who can only run one. You still work hands-on with datasets throughout, but the emphasis is on understanding why methods work and when they fail.

Q: Is there a way to try before enrolling?

A: Yes, book a free demo class at learn.modernagecoders.com/contact; there is no commitment, and it doubles as a placement conversation about your goals and starting level. WhatsApp +91 91233 66161 works too.

Q: What is the teaching format?

A: Live one hour online classes with a real teacher, never pre-recorded lectures. Classes are small and interactive, so derivations are worked with you, not at you.

Q: Is there a certificate?

A: You earn a completion certificate plus a documented portfolio of the statistical analyses, models and projects you built during the program, which serves as working evidence of your skills.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/data-analytics-mathematics-masterclass>

COURSE 39 OF 118: COMPUTER SCIENCE CLASS 11-12: CBSE PYTHON & ICSE/ISC JAVA

Course link: <https://learn.modernagecoders.com/courses/cbse-icse-computer-science-class-11-12-python-java-complete-course>

Subject area: School Board Exam Preparation - Computer Science

Designed for: Teens

Who can join: Class 11 Beginner to Class 12 Board Exam Ready

Course length: 24 months (Class 11 + Class 12 full syllabus)

Time needed: 8-10 hours/week (4 classes + self-practice)

Certificate: Course Completion Certificate + Board Exam Readiness Guarantee

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Class 11 & 12 Computer Science for CBSE (Python, SQL) and ICSE/ISC (Java, OOP): separate board batches, full syllabus, practical file and project guidance.

In one line: 100% Syllabus Coverage · Separate CBSE & ICSE Batches · Board Topper Strategy · Live Expert Mentoring

ABOUT THE PROGRAMME

This is the most thorough Computer Science program built for Indian Class 11 & 12 students across both CBSE and ICSE boards. We run completely separate batches for CBSE and ICSE students because the syllabus, programming language, exam pattern, and marking scheme are fundamentally different. CBSE students learn Python programming (as per Code 083 syllabus), SQL with MySQL, computer networking, file handling, data structures using Python, and Python-MySQL connectivity. ICSE/ISC students learn Java programming, object-oriented programming, data structures (stack, queue, linked list, binary trees), Boolean algebra with Karnaugh maps, and advanced recursion. Every class is aligned chapter-by-chapter with Sumita Arora, Preeti Arora, and NCERT textbooks. Students get complete practical file preparation, project guidance, viva preparation, previous year paper solving, and dedicated board exam revision modules. Our students consistently score 95+ in board exams because we don't just teach, we drill, practice, and perfect every concept until it becomes second nature.

WHAT MAKES THIS COURSE DIFFERENT

- Separate CBSE and ICSE batches, no mixing of syllabus or programming languages
- 100% syllabus coverage aligned with latest 2026-27 CBSE & ISC curriculum
- Chapter-wise teaching matching Sumita Arora, Preeti Arora & NCERT textbooks
- Complete practical file with 20+ programs (CBSE) / lab assignments (ISC) included
- Board exam project guidance with documentation and viva preparation
- Weekly chapter tests + monthly unit tests + mock board exams
- Previous 10 years solved papers with detailed marking scheme analysis
- Dedicated doubt-clearing sessions before exams
- Sample paper solving with CBSE/ISC marking scheme training
- Programming logic building from scratch, no prior coding needed
- SQL hands-on practice with live MySQL database
- Assertion-Reason, Case Study, and MCQ-specific exam strategies

THE LEARNING PATH, STEP BY STEP

Phase 1: Class 11, Semester 1 (Months 1-5): Computer fundamentals, number systems, Boolean logic, Python/Java basics, control flow, functions

Phase 2: Class 11, Semester 2 (Months 6-10): Data structures (lists/tuples/dictionaries in Python OR arrays/strings in Java), society & ethics, practicals

Phase 3: Class 12, Semester 1 (Months 11-17): Advanced programming, file handling, data structures, recursion, sorting & searching

Phase 4: Class 12, Semester 2 (Months 18-24): SQL & databases, computer networks, Python-MySQL connectivity / ISC advanced topics, board exam prep

WHAT A STUDENT NEEDS BEFORE STARTING

Class level: Students entering Class 11 or Class 12 with Computer Science - CBSE (code 083) or ISC; Class 10 students who have chosen CS can join early

Prior knowledge: No programming experience required - Class 11 starts from the very basics; Class 12 joiners get a bridge revision of Class 11 topics

Equipment: A laptop or desktop with a stable internet connection; we guide installation of Python and MySQL (CBSE) or Java and BlueJ (ISC) in the first sessions

Time commitment: 8-10 hours per week including live classes and self-practice

Textbooks: Sumita Arora, Preeti Arora, or NCERT - classes align with whichever textbook the student's school follows

WHO THIS COURSE IS FOR

Class 11 students: Students starting Computer Science in Class 11 who want strong fundamentals from the first chapter

Class 12 board aspirants: Students facing boards in the 2026-27 session who need complete syllabus coverage plus intensive exam practice

Students stuck on programming: Students who understand the theory but struggle to write, trace, or debug programs on their own

Practical file and project help: Students who need structured help finishing the practical file and board project correctly and on time

Future engineers: Students planning B.Tech, BCA, or B.Sc Computer Science who want a genuine head start on college programming

Score improvers: Students targeting a 90+ score who want weekly tests, mock exams, and marking-scheme answer training

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: CLASS 11 SEMESTER 1, COMPUTER FUNDAMENTALS & PROGRAMMING FOUNDATIONS (MONTHS 1-5) -----

Build rock-solid fundamentals in computer science theory and start your programming journey. CBSE students begin with Python, ISC students begin with Java. Both batches cover computer organization, number systems, and Boolean logic.

Month 1: Computer Systems & Organization (Week 1-4)

Computer System Overview & Hardware

Topics:

- Evolution of computers, from Abacus to AI-powered machines
- Computer generations: First (vacuum tubes) to Fifth (AI)
- Block diagram of a computer system, Input, CPU, Memory, Output
- CPU architecture: ALU (Arithmetic Logic Unit), Control Unit, Registers
- Types of processors: CISC vs RISC, multicore processors
- Input devices: keyboard, mouse, scanner, touchscreen, biometric devices, microphone
- Output devices: monitor (LCD, LED, OLED), printer (laser, inkjet, 3D), speaker, projector
- Memory hierarchy: registers cache RAM secondary storage
- Primary memory: RAM (SRAM vs DRAM), ROM (PROM, EPROM, EEPROM), Cache memory
- Secondary storage: HDD, SSD, USB flash drive, optical discs (CD, DVD, Blu-ray)
- Cloud storage concepts and modern storage solutions
- Units of memory: bit, nibble, byte, KB, MB, GB, TB, PB

Projects built:

- Create an illustrated computer system diagram with all components labeled
- Memory hierarchy comparison chart with speed, cost, and capacity
- Research report: Evolution of processors from Intel 4004 to latest chips

Practice: Chapter-end exercises from Sumita Arora Chapter 1, NCERT textbook questions, 30 MCQs on computer fundamentals

Software, Operating Systems & Encoding

Topics:

- Software classification: System software vs Application software
- System software: operating system, language processors (compiler, interpreter, assembler), device drivers
- Application software: general purpose (MS Office, browsers) vs specific purpose (Tally, hospital management)
- Open source vs proprietary software, examples and differences
- Operating system functions: process management, memory management, file management, device management
- Types of OS: batch processing, time-sharing, real-time, distributed, mobile OS
- User interface: CLI (Command Line Interface) vs GUI (Graphical User Interface)
- Language processors: compiler vs interpreter, detailed comparison
- High-level vs low-level vs machine-level languages
- Utility software: antivirus, disk defragmenter, backup tools, compression tools
- Firmware and embedded systems concepts
- Mobile operating systems: Android, iOS, overview

Projects built:

- Software classification mind map with 20+ real-world examples
- OS comparison: Windows vs Linux vs macOS, features and use cases
- Install and explore Linux (Ubuntu) in a virtual machine

Practice: NCERT exercises, Sumita Arora solved examples, 40 MCQs, 10 short-answer questions

Number Systems & Conversions

Topics:

- Why computers use binary, the digital foundation
- Decimal number system (base 10), positional notation explained
- Binary number system (base 2), counting in binary, place values
- Octal number system (base 8), uses in Unix/Linux file permissions
- Hexadecimal number system (base 16), uses in memory addresses, colors (#FF0000)
- Conversion: Decimal to Binary (successive division method)
- Conversion: Decimal to Octal and Hexadecimal
- Conversion: Binary to Decimal (positional value method)
- Conversion: Binary to Octal (grouping of 3 bits) and Binary to Hexadecimal (grouping of 4 bits)
- Conversion: Octal to Binary, Hexadecimal to Binary
- Fractional number conversions across all bases
- Binary addition and subtraction
- 1's complement and 2's complement representation
- Signed magnitude representation
- Overflow and underflow in binary arithmetic
- Shortcut methods for quick conversions in exams

Projects built:

- Number system converter calculator (manual worksheet with 50 conversions)
- Color code converter: RGB decimal to hexadecimal
- Binary arithmetic drill, 30 addition and subtraction problems

Practice: 100 conversion problems across all bases, previous year board questions on number systems, timed practice tests

Boolean Logic & Encoding Schemes

Topics:

- Introduction to Boolean algebra, George Boole's contribution
- Boolean values: True (1) and False (0)
- Basic logic gates: AND, OR, NOT, symbols, truth tables, expressions
- Derived gates: NAND, NOR, XOR, XNOR, truth tables and equivalences
- Universal gates: NAND and NOR, implementing any gate using only NAND/NOR (ISC focus)
- Boolean expressions and evaluation
- De Morgan's theorems: $\text{NOT}(A \text{ AND } B) = \text{NOT } A \text{ OR } \text{NOT } B$, $\text{NOT}(A \text{ OR } B) = \text{NOT } A \text{ AND } \text{NOT } B$
- Verification of De Morgan's theorems using truth tables
- Basic Boolean laws: Identity, Complement, Idempotent, Absorption, Involution

- Simplification of Boolean expressions using algebraic method
- Character encoding schemes: ASCII (7-bit, 128 characters)
- ISCII (Indian Script Code for Information Interchange), 8-bit
- Unicode (UTF-8, UTF-16, UTF-32), multilingual support, emojis
- Difference between ASCII, ISCII, and Unicode, why Unicode is the modern standard

Projects built:

- Truth table generator for any 2-3 variable Boolean expression
- Gate diagram drawing practice, 10 circuits
- Encoding explorer: Convert your name to ASCII, Unicode values

Practice: 50 Boolean expression simplification problems, 30 truth table exercises, previous year questions on encoding

Months 2-3: Programming Foundations (Python for CBSE / Java for ISC) (Week 5-12)

Cbse Python Track

Getting Started with Python, Setup, Syntax & Fundamentals

Topics:

- Why Python?, Used by Google, Instagram, YouTube, NASA, Netflix
- Python history: Guido van Rossum, Python 2 vs Python 3
- Installing Python 3.12+ on Windows/Mac/Linux
- Python IDLE, Interactive mode vs Script mode
- Setting up VS Code with Python extensions
- First Python program: `print('Hello, World!')`
- Python tokens: keywords, identifiers, literals, operators, punctuators
- Python keywords list, `import`, `def`, `class`, `if`, `else`, `elif`, `for`, `while`, etc.
- Identifiers: naming rules (start with letter/underscore, case-sensitive, no keywords)
- Variables: declaration, assignment, dynamic typing
- Data types: `int`, `float`, `complex`, `str`, `bool`, `None`
- Type checking with `type()` function
- Type conversion: `int()`, `float()`, `str()`, `bool()`, implicit vs explicit
- Mutable vs Immutable types, fundamental concept for board exams
- Comments: single-line (`#`) and multi-line (triple quotes)
- Input function: `input()`, always returns string
- Output function: `print()`, `sep`, `end` parameters, formatted output
- f-strings for formatted printing (Python 3.6+)
- Escape sequences: `\n`, `\t`, `\\`, `\'`, `\"`
- Python's philosophy: `import this`, The Zen of Python

Projects built:

- Personal bio generator, takes name, age, hobby as input and formats output
- Simple calculator, add, subtract, multiply, divide two numbers
- Temperature converter, Celsius to Fahrenheit and vice versa
- Area calculator for circle, rectangle, triangle with formatted output

Practice: Sumita Arora Chapter 3 & 4 exercises, 50 output prediction questions, 20 error-finding programs

Operators & Expressions in Python

Topics:

- Arithmetic operators: `+`, `-`, `*`, `//` (floor division), `%` (modulus), `**` (exponent)
- Floor division vs true division, `7//2 = 3` vs `7/2 = 3.5`
- Modulus operator applications, even/odd check, digit extraction
- Relational/Comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Logical operators: `and`, `or`, `not`, truth tables and short-circuit evaluation
- Short-circuit evaluation: how Python skips unnecessary checks
- Assignment operators: `=`, `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`
- Identity operators: `is`, `is not`, comparing object identity vs equality
- Membership operators: `in`, `not in`, checking presence in sequences
- Bitwise operators: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (left shift), `>>` (right shift)
- Operator precedence table, PEMDAS for Python (`**` unary `*`, `//`, `%` + - comparisons not and or)
- Associativity: left-to-right vs right-to-left (`**`)
- Expression evaluation practice, step-by-step breakdown
- Mixed-type expressions and type coercion rules
- The walrus operator `:=` (Python 3.8+), assignment within expressions
- Common exam traps: integer division, operator precedence gotchas

Projects built:

- Expression evaluator, predict output of 20 complex expressions
- Bill calculator with tax and discount using all arithmetic operators
- Bitwise operation visualizer, show binary representation and result
- Operator precedence quiz game

Practice: 100 expression evaluation problems, 30 output prediction MCQs, Sumita Arora Chapter 5 exercises

Control Flow, Conditional Statements

Topics:

- Sequential flow vs conditional flow vs iterative flow, program flow types
- `if` statement: syntax, indentation rules, single-line and multi-line body
- `if-else` statement: two-way branching
- `if-elif-else` ladder: multi-way branching
- Nested `if` statements: decision trees within decisions
- Ternary operator / conditional expression: `value = x if condition else y`
- Common patterns: finding max/min of 2/3 numbers, grade calculator
- Leap year detection algorithm, complete logic
- Character type checking: vowel/consonant, uppercase/lowercase, digit/letter

- Menu-driven programs, building interactive applications
- Logical operators in conditions: combining multiple conditions
- Truthiness and falsiness in Python: 0, None, '', [], {} are falsy
- Common mistakes: = vs ==, missing colon, indentation errors
- Board exam pattern: output prediction for nested if-elif-else
- Assertion-Reason questions on conditional statements

Projects built:

- Grade calculator, CBSE marking scheme with A1, A2, B1, B2, C1, C2, D, E
- Income tax calculator, with Indian tax slab rates for 2026-27
- Triangle type detector, equilateral, isosceles, scalene, right-angled
- Simple login system with username and password validation
- Day of the week finder, input a number, output the day

Practice: 60 conditional logic problems, 30 output prediction questions, previous year board questions, Sumita Arora exercises

Control Flow, Loops & Iterations

Topics:

- for loop: iterating over sequences (range, string, list)
- range() function: range(stop), range(start, stop), range(start, stop, step)
- Negative step in range: range(10, 0, -1) for countdown
- while loop: condition-based repetition
- Infinite loops: while True, when and how to use safely
- Loop control statements: break (exit loop), continue (skip iteration)
- else clause with loops, executes when loop completes without break
- Nested loops: outer and inner loop, execution flow tracing
- Pattern printing with nested loops, stars, numbers, alphabets
- Common patterns: pyramid, diamond, Floyd's triangle, Pascal's triangle
- Accumulator pattern: sum, product, counting with loops
- Finding factors, prime numbers, Fibonacci series using loops
- GCD and LCM calculation using loops
- Digit extraction: sum of digits, reverse a number, palindrome check
- Armstrong number, perfect number, strong number detection
- Loop optimization and avoiding unnecessary iterations
- Board exam focus: tracing loop output, counting iterations

Projects built:

- Pattern printer, 15 different star and number patterns
- Prime number checker and generator (Sieve of Eratosthenes approach)
- Number guessing game with limited attempts
- Multiplication table generator with formatted output
- Fibonacci series generator up to n terms
- Armstrong number finder in a given range

Practice: 80 loop tracing problems, 20 pattern programs, Sumita Arora loop exercises, previous year board pattern questions

Isc Java Track

Getting Started with Java, OOP Foundations

Topics:

- Why Java?, Platform independence, Android development, enterprise applications
- Java history: James Gosling, Sun Microsystems, Oracle acquisition
- Installing JDK 17+ and setting up PATH environment variable
- Setting up BlueJ IDE (preferred for ISC) and IntelliJ IDEA
- First Java program: public static void main(String[] args)
- Understanding class structure, main method, System.out.println()
- Java tokens: keywords, identifiers, literals, operators, separators
- Primitive data types: byte, short, int, long, float, double, char, boolean
- Size and range of each primitive type, exam favorite
- Variables: declaration, initialization, naming conventions
- Constants with final keyword
- Type casting: widening (automatic) vs narrowing (explicit)
- Wrapper classes: Integer, Double, Character, Boolean, autoboxing/unboxing
- Scanner class for input: nextInt(), nextDouble(), nextLine(), next()
- Output: System.out.print(), println(), printf()
- Comments: //, /* */, /** */ (Javadoc)
- Java's object-oriented philosophy: everything is an object

Projects built:

- Student profile program, input name, roll number, marks, display formatted
- Simple calculator with Scanner input
- Type casting demonstrator, show widening and narrowing
- Wrapper class explorer, convert between primitives and objects

Practice: 50 output prediction questions, 20 type casting problems, ISC specimen paper MCQs

Operators, Expressions & Math Library in Java

Topics:

- Arithmetic operators: + - * / %, integer vs floating-point division
- Relational operators: == != < > <= >=
- Logical operators: && (AND), || (OR), ! (NOT), short-circuit evaluation
- Assignment operators: = += -= *= /= %=
- Increment/Decrement: ++x (prefix) vs x++ (postfix), critical for ISC exams
- Ternary operator: condition ? value1 : value2
- Bitwise operators: & | ^ ~ << >>
- Operator precedence and associativity, complete table

- Expression evaluation with mixed types, promotion rules
- Math class methods: Math.sqrt(), Math.pow(), Math.abs(), Math.max(), Math.min()
- Math.ceil(), Math.floor(), Math.round(), Math.random()
- Math.PI, Math.E, mathematical constants
- Trigonometric functions: Math.sin(), Math.cos(), Math.tan() (radians)
- String concatenation with + operator, rules and gotchas
- Type promotion in expressions: byte short int long float double

Projects built:

- Scientific calculator with Math library functions
- Expression evaluator, step-by-step breakdown of complex expressions
- Prefix vs postfix increment quiz, 20 tricky questions
- Random number generator game

Practice: 100 expression evaluation problems, ISC previous year questions, operator precedence drills

Conditional Statements in Java

Topics:

- if statement: syntax with curly braces, single statement shortcuts
- if-else statement: two-way branching
- if-else if-else ladder: multi-way branching
- Nested if statements: complex decision trees
- switch-case statement: syntax, break keyword, default clause
- switch vs if-else: when to use which, ISC exam favorite
- Fall-through in switch: what happens without break
- Menu-driven programs using switch-case
- Ternary operator for concise conditions
- Common programs: largest of 3, grade calculator, day finder
- Leap year, vowel/consonant, positive/negative/zero checks
- Character-based conditions: ASCII value ranges for A-Z, a-z, 0-9
- Dangling else problem and how Java resolves it
- Board exam focus: output tracing with nested conditions

Projects built:

- Menu-driven calculator using switch-case
- Income tax calculator with Indian tax slabs
- Student grade card generator with ISC marking scheme
- Electric bill calculator with slab-based pricing

Practice: ISC specimen paper conditional questions, 50 output tracing problems, 20 switch-case programs

Loops & Iterations in Java

Topics:

- for loop: initialization, condition, update, execution flow
- while loop: pre-test loop, condition checked before body
- do-while loop: post-test loop, body executes at least once (NOT in Python!)
- for vs while vs do-while: choosing the right loop
- Nested loops: matrix traversal, pattern printing
- Loop control: break (exit loop), continue (skip iteration)
- Labeled break and continue, for nested loop control
- Pattern printing: stars, numbers, alphabets, ISC exam staple
- Pyramid, inverted pyramid, diamond, Floyd's triangle patterns
- Digit extraction: sum of digits, reverse number, palindrome
- Prime number programs: check, generate, prime factorization
- GCD (Euclidean algorithm), LCM, Fibonacci series
- Armstrong, perfect, neon, spy, niven (Harshad) numbers
- Automorphic, kaprekar, twin prime numbers
- Frequency counting: digit frequency, character frequency in strings
- Loop optimization and efficiency analysis

Projects built:

- Pattern printer, 20 different patterns (stars, numbers, alphabets)
- Special number detector, checks Armstrong, palindrome, perfect in one program
- Prime number generator with sieve approach
- Number series generator: Fibonacci, triangular, square numbers

Practice: 100 loop tracing problems, 30 pattern programs, ISC previous year loop questions, number program drills

Months 4-5: Functions, Strings & Data Handling (Week 13-20)

Cbse Python Track

Strings in Python, Complete Mastery

Topics:

- String creation: single quotes, double quotes, triple quotes
- String as a sequence, indexing (0-based, negative indexing)
- String slicing: [start:stop:step], extracting substrings
- String immutability, why strings cannot be changed in place
- String operators: + (concatenation), * (repetition), in, not in
- String comparison: lexicographic ordering, ASCII values
- Traversing strings: for loop, while loop, enumerate()
- String methods (MUST KNOW for boards): len(), capitalize(), title(), upper(), lower(), swapcase()
- count(), find(), index(), rfind(), rindex()
- startswith(), endswith(), isalpha(), isdigit(), isalnum()
- isspace(), islower(), isupper(), istitle()

- strip(), lstrip(), rstrip(), replace(), split(), join()
- partition(), maketrans(), translate()
- String formatting: %, .format(), f-strings
- Common programs: palindrome check, character frequency, vowel count, word count
- Reverse a string, remove spaces, capitalize each word
- Board exam pattern: output prediction with string slicing and methods

Projects built:

- Word counter and analyzer, count words, vowels, consonants, digits, special chars
- Palindrome checker for strings and sentences
- Caesar cipher, encrypt and decrypt messages
- String formatting report generator

Practice: Sumita Arora Chapter 7 complete exercises, 50 string output prediction

MCQs, 30 string programs

Lists in Python, Complete Mastery

Topics:

- List creation: [], list(), list comprehension
- Lists as mutable sequences, key difference from strings and tuples
- Indexing and negative indexing in lists
- List slicing: [start:stop:step]
- List operations: + (concatenation), * (repetition), in, not in
- List methods (BOARD EXAM CRITICAL): append(), extend(), insert()
- remove(), pop(), pop(index), clear()
- index(), count(), sort(), sort(reverse=True), reverse()
- copy(), shallow copy vs deep copy explained
- Built-in functions with lists: len(), min(), max(), sum(), sorted()
- List comprehension: [expression for item in iterable if condition]
- Nested lists (2D lists / matrices), accessing elements with [row][col]
- Traversing lists: for loop, while loop, enumerate(), zip()
- del statement vs remove() vs pop(), differences
- Aliasing vs cloning, the assignment trap (a = b vs a = b[:])
- Passing lists to functions, mutable argument behavior
- Common programs: linear search, finding max/min, removing duplicates, merging sorted lists
- Board exam focus: output prediction with list methods and slicing

Projects built:

- Student marks management system, add, remove, search, sort, average
- Inventory management with nested lists
- Matrix operations, addition, subtraction, transpose using 2D lists
- List-based quiz application

Practice: Sumita Arora Chapter 8 exercises, 60 list operation MCQs, 25 list programs

Tuples & Dictionaries in Python

Topics:

- Tuples: creation with () and tuple(), single element tuple (a,) trap
- Tuple immutability, cannot add, remove, or change elements
- Tuple indexing, slicing, and operations (+, *, in)
- Tuple methods: count(), index()
- Built-in functions: len(), min(), max(), sum(), sorted() with tuples
- Tuple packing and unpacking: a, b, c = (1, 2, 3)
- Tuple vs List: when to use which, performance and safety
- Nested tuples and accessing elements
- Dictionaries: key-value pair data structure, {} and dict()
- Dictionary keys must be immutable (strings, numbers, tuples), why?
- Accessing values: d[key] vs d.get(key, default)
- Adding and updating: d[key] = value, d.update()
- Removing: del d[key], d.pop(key), d.popitem(), d.clear()
- Dictionary methods: keys(), values(), items(), get(), update(), copy()
- Dictionary traversal: iterating over keys, values, items
- Nested dictionaries, student records with multiple attributes
- Dictionary comprehension: {k: v for k, v in iterable}
- len(), min(), max(), sorted() with dictionaries
- Common programs: frequency counter, word dictionary, student database
- Board exam: output prediction with dictionary operations

Projects built:

- Contact book application using dictionary
- Word frequency analyzer, count occurrence of each word in text
- Student report card using nested dictionaries
- English-Hindi dictionary with search feature

Practice: Sumita Arora Chapters 9 & 10 exercises, 50 MCQs on tuples and dictionaries, 20 programs

Functions & Modules in Python

Topics:

- What are functions?, code reusability, modularity, abstraction
- Built-in functions vs user-defined functions
- Defining functions: def keyword, function name, parameters, body, return
- Calling functions: positional arguments, keyword arguments
- Default parameter values, avoiding mutable default argument trap
- *args, variable-length positional arguments
- **kwargs, variable-length keyword arguments
- Return statement: returning single value, multiple values (as tuple)
- Functions without return, returns None by default

- Scope of variables: local scope, global scope, LEGB rule
- global keyword, modifying global variables inside functions
- Passing arguments: immutable (pass by value behavior) vs mutable (pass by reference behavior)
- Recursive functions introduction, factorial, Fibonacci (preview for Class 12)
- Lambda functions: lambda arguments: expression
- map(), filter(), reduce(), functional programming basics
- Modules: import, from...import, import...as
- math module: sqrt(), ceil(), floor(), pow(), fabs(), sin(), cos(), tan(), pi, e, log(), log10()
- random module: random(), randint(), randrange(), choice(), shuffle(), seed()
- statistics module: mean(), median(), mode()
- Creating your own modules, `__name__ == '__main__'` pattern
- Board exam focus: function output tracing, scope questions, module function MCQs

Projects built:

- Math utility library, custom module with 20+ mathematical functions
- Random password generator using random module
- Statistics calculator using statistics module
- Menu-driven bank account system using functions

Practice: Sumita Arora function chapter exercises, 40 function tracing MCQs, 25 function-based programs, scope identification exercises

ISC Java Track

Methods, Constructors & Scope in Java

Topics:

- Methods: defining, calling, return types, parameters
- Formal parameters vs actual parameters (arguments)
- Pass by value in Java, primitive types are copied
- Pass by reference behavior, objects are passed by reference copy
- Method overloading: same name, different parameter lists
- Constructors: default, parameterized, copy constructor
- Constructor overloading, multiple constructors in a class
- this keyword: referring to current object, constructor chaining
- Static methods and static variables, class-level members
- Static vs non-static: when to use which
- Variable scope: local, instance, class (static) variables
- Access modifiers: public, private, protected, default
- Encapsulation: data hiding with private variables and public getters/setters
- Pure functions and impure functions in Java
- Method signature and return type, ISC exam focus
- Recursive methods introduction, factorial, GCD

Projects built:

- Bank account class with deposit, withdraw, balance methods
- Student class with multiple constructors and method overloading
- Calculator class with static utility methods
- Library book management with encapsulation

Practice: ISC method-based questions, 40 output tracing problems, constructor MCQs

Arrays in Java, 1D and 2D

Topics:

- Array declaration, creation with new keyword, initialization
- Array as reference type, memory allocation on heap
- Accessing elements: indexing (0-based), `ArrayIndexOutOfBoundsException`
- Array traversal: for loop, for-each (enhanced for) loop
- Array input with Scanner, reading n elements
- 1D array operations: sum, average, max, min, counting
- Linear search: sequential search algorithm
- Binary search: divide and conquer on sorted arrays
- Sorting algorithms: Bubble sort, swapping adjacent elements
- Selection sort, finding minimum and placing in position
- Insertion sort, inserting element in correct position in sorted part
- Comparison of sorting algorithms, time complexity basics
- 2D arrays (matrices): declaration, initialization, traversal
- Matrix operations: addition, subtraction, multiplication, transpose
- Row-wise and column-wise operations on 2D arrays
- Jagged arrays, arrays of different lengths
- Array address calculation: row-major and column-major (ISC theory)
- Common programs: merge arrays, rotate array, remove duplicates

Projects built:

- Student marks analyzer, input marks, sort, find topper, grade distribution
- Matrix calculator, all operations with menu-driven interface
- Address calculation worksheet, 20 problems
- Sorting visualizer, trace each pass of bubble, selection, insertion sort

Practice: ISC array-based programs (30+), sorting algorithm tracing, address calculation formula problems

String Handling in Java

Topics:

- String class: immutable strings in Java
- String creation: literal vs new keyword, String pool concept
- String methods: length(), charAt(), indexOf(), lastIndexOf()
- substring(), concat(), replace(), toUpperCase(), toLowerCase()
- trim(), startsWith(), endsWith(), contains()

- equals() vs == : content comparison vs reference comparison, EXAM CRITICAL
- equalsIgnoreCase(), compareTo(), compareToIgnoreCase()
- String to char array: toCharArray(), and vice versa valueOf()
- Character class methods: isLetter(), isDigit(), isUpperCase(), isLowerCase()
- Character.toUpperCase(), Character.toLowerCase()
- StringBuffer/StringBuilder: mutable strings, append(), insert(), delete(), reverse()
- String vs StringBuffer vs StringBuilder, differences and when to use
- StringTokenizer class: splitting strings (ISC specific)
- String concatenation performance: why StringBuilder matters
- Common programs: palindrome, word extraction, pigLatin, character frequency
- Sentence manipulation: reverse words, title case, remove duplicates
- Board exam focus: output tracing with string methods

Projects built:

- Text analyzer, word count, sentence count, character frequency
- Pig Latin converter
- Unique word extractor from a sentence
- String encryption/decryption program

Practice: ISC string programs (40+), method output prediction MCQs, character manipulation problems

Recursion in Java, ISC Focus

Topics:

- What is recursion?, method calling itself
- Base case and recursive case, avoiding infinite recursion
- Stack memory and recursion, how function calls are stacked
- StackOverflowError, what happens without base case
- Factorial using recursion, tracing each call
- Fibonacci series using recursion
- GCD using Euclidean algorithm (recursive)
- Power calculation: a^n using recursion
- Sum of digits using recursion
- Reverse a number using recursion
- Palindrome check using recursion
- Binary search using recursion
- Tower of Hanoi, classic recursion problem (ISC important)
- Decimal to binary conversion using recursion
- String reversal using recursion
- Recursion vs iteration: pros, cons, when to use which
- Tail recursion concept
- ISC exam focus: tracing recursive calls, finding output, counting calls

Projects built:

- Recursive function library, 15 recursive solutions
- Tower of Hanoi solver with step-by-step output
- Recursive pattern printer
- Recursive maze solver (bonus challenge)

Practice: ISC recursion questions (30+), call stack tracing exercises, output prediction problems

----- PHASE 2: CLASS 11 SEMESTER 2, ADVANCED CONCEPTS, ETHICS & PRACTICALS (MONTHS 6-10) -----

Complete remaining Class 11 syllabus topics, prepare practical file with 20+ programs, work on project, and build exam readiness.

Months 6-7: Society, Law, Ethics & Advanced Topics (Week 21-28)

Society, Law and Ethics, Digital Citizenship (CBSE) / Computing Trends (ISC)

Topics:

- Digital footprint: active vs passive, what you leave behind online
- Net etiquette (Netiquette): proper online behavior, email etiquette
- Social media ethics and responsible usage
- Data protection and privacy, personal data, consent, GDPR concepts
- Intellectual Property Rights (IPR): copyright, patent, trademark
- Plagiarism: what it is, how to avoid, proper citation
- Software licensing: proprietary vs open source
- Free and Open Source Software (FOSS): Linux, LibreOffice, GIMP, Python
- Creative Commons licenses: CC-BY, CC-BY-SA, CC-BY-NC
- GPL (GNU General Public License), Apache License, MIT License
- Cybercrime: hacking, cracking, phishing, pharming, vishing, smishing
- Malware types: virus, worm, trojan horse, ransomware, spyware, adware
- Ransomware attacks: WannaCry, NotPetya, real-world case studies
- Cyberbullying: forms, impact, prevention, reporting
- Indian Information Technology Act 2000 (amended 2008), key provisions
- IT Act sections relevant to students: Sections 43, 65, 66, 67
- E-waste: definition, hazards, proper disposal methods, recycling
- Health concerns: ergonomics, digital eye strain, repetitive strain injury
- Gender and disability issues in computing, inclusivity in tech
- Emerging trends: AI, IoT, Cloud Computing, Big Data, Blockchain (overview)

Projects built:

- Digital citizenship presentation, 20 dos and don'ts
- Cybercrime awareness poster/infographic

- E-waste management plan for your school
 - Research report: Famous cyberattacks and lessons learned
- Practice: NCERT society chapter questions, 50 MCQs on cyber ethics, case study questions, IT Act scenario-based questions

CBSE: Advanced Python Concepts / ISC: File Handling & Packages in Java

Cbse topics:

- Revisiting all Python concepts for exam preparation
- Advanced list comprehensions with multiple conditions
- Dictionary and set operations, advanced patterns
- Nested data structures: list of dictionaries, dictionary of lists
- Error handling basics: try, except, finally (preview for Class 12)
- File handling introduction: open(), read(), write(), close() (preview)
- Standard library exploration: os module, sys module basics
- Code optimization techniques, writing Pythonic code
- PEP 8 coding style guidelines
- Debugging strategies: print debugging, IDE debugger usage

IsC topics:

- File handling in Java using FileInputStream, FileOutputStream
- FileReader and FileWriter, character stream handling
- BufferedReader and PrintWriter for efficient I/O
- Reading and writing text files, line by line
- Exception handling: try-catch-finally in file operations
- Packages: what are packages, package declaration
- Creating user-defined packages
- Importing packages: import statement, wildcard import
- Java API packages: java.lang, java.util, java.io
- Access modifiers with packages: public, protected, default, private

Practical File Preparation, Project Work & Class 11 Exam Prep

Topics:

- CBSE Practical file: Minimum 20 Python programs covering all chapters
- ISC Practical: Year-long programming assignments compilation
- Program 1-5: Basic Python/Java programs (variables, operators, input/output)
- Program 6-10: Control flow programs (if-else, loops, patterns)
- Program 11-15: String manipulation programs
- Program 16-20: List/Array, Tuple, Dictionary programs
- Program 21-25: Function-based programs with modules (CBSE) / OOP programs (ISC)
- Project selection and planning, choosing a meaningful project
- CBSE suggested projects: Quiz game, Library management, Student database
- ISC suggested projects: Inventory system, Bank management, School management
- Project documentation: introduction, system requirements, source code, output screenshots
- Viva voce preparation: 50 most-asked questions with answers
- Class 11 final exam revision, chapter-wise important questions
- Sample paper solving, 5 complete papers with marking scheme
- Time management during exams, allocating marks to time

Deliverables:

- Complete practical file with 20-25 programs (handwritten/printed as per school)
- Project report with documentation
- Viva preparation sheet, 50 Q&A
- 5 solved sample papers

PHASE 3: CLASS 12 SEMESTER 1, ADVANCED PROGRAMMING, FILE HANDLING & DATA STRUCTURES (MONTHS 11-17)

Deep dive into Class 12 core topics: advanced programming, file handling, data structures, sorting and searching algorithms. This is the highest-weightage section in board exams.

Months 11-12: Advanced Functions, Exception Handling & File Handling (Week 41-48)

Cbse Python Track

Advanced Functions & Exception Handling in Python

Topics:

- Revision of Class 11 function concepts, parameters, return, scope
- Global vs local scope deep dive, LEGB rule revisited
- global keyword, modifying global variables inside functions
- Nested functions and closures
- Functions as first-class objects, passing functions as arguments
- Advanced default arguments and keyword-only arguments
- Recursive functions: factorial, Fibonacci, sum of digits, power, GCD
- Recursion depth and sys.setrecursionlimit()
- Exception handling: what are exceptions vs syntax errors
- try block: code that might cause an exception
- except block: handling specific exceptions
- Multiple except blocks for different exception types
- except with as keyword: except ValueError as e
- else block: executes when no exception occurs
- finally block: always executes, cleanup code
- Common built-in exceptions: ValueError, TypeError, IndexError, KeyError, ZeroDivisionError, FileNotFoundError, IOError
- raise statement: raising exceptions manually
- Custom exception classes (basics)

- Board exam pattern: output prediction with try-except, function scope questions

Projects built:

- Robust calculator with exception handling for all edge cases
- Recursive problem set, 10 programs using recursion
- Error-proof input validator using try-except
- Function library with comprehensive error handling

Practice: Sumita Arora Class 12 function chapters, 40 exception handling MCQs, 20 recursive function tracing problems

Text File Handling in Python

Topics:

- What are files?, persistent storage, file vs variable
- Types of files: text files vs binary files
- Opening a file: open(filename, mode), file handle/object
- File modes: 'r' (read), 'w' (write), 'a' (append), 'r+' (read+write), 'w+' (write+read), 'a+' (append+read)
- Reading text files: read(), read(n), readline(), readlines()
- Difference between read(), readline(), readlines(), EXAM CRITICAL
- Writing to text files: write(), writelines()
- Difference between write() and writelines()
- Closing files: close(), why it's important
- with statement: automatic file closing, best practice
- File pointer: tell(), current position, seek(offset, whence), moving pointer
- seek(0), go to beginning, seek(0, 2), go to end
- Line-by-line processing: for line in file_handle
- Counting lines, words, characters in a text file
- Searching for a word/pattern in a text file
- Copying content from one file to another
- Appending data to existing files
- Common programs: count uppercase/lowercase/digits/vowels in file
- Find and replace words in a text file
- Board exam focus: output prediction with file operations, explain difference questions

Projects built:

- Text file analyzer, count lines, words, characters, vowels, consonants
- Log file processor, search and filter entries
- Student record manager using text files
- File copy and merge utility

Practice: Sumita Arora file handling chapter, 30 file handling programs, 40 MCQs on file operations

Binary File Handling & CSV File Handling in Python

Topics:

- Binary files: what and why, storing non-text data
- Binary file modes: 'rb', 'wb', 'ab', 'r+b', 'w+b', 'a+b'
- pickle module: serialization and deserialization
- pickle.dump(object, file_handle): writing Python objects to binary file
- pickle.load(file_handle): reading Python objects from binary file
- Pickling different data types: int, float, string, list, tuple, dictionary
- Writing and reading multiple records to/from binary file
- Searching records in a binary file, reading until EOFError
- Updating and deleting records in binary files (read all, modify, write all)
- EOFError handling with try-except, standard pattern for binary files
- CSV files: Comma Separated Values, structure and usage
- csv module: importing csv
- csv.writer(): creating CSV writer object
- writerow(), writing single row, writerows(), writing multiple rows
- csv.reader(): reading CSV files, returns iterable
- Reading CSV with for loop, each row is a list
- Using delimiter parameter for custom separators
- CSV with headers, DictWriter and DictReader (bonus)
- Real-world CSV: exporting student data, reading attendance sheets
- Board exam pattern: complete programs on binary and CSV file operations

Projects built:

- Student database with binary file, add, search, update, delete records
- Employee payroll system using pickle
- CSV report generator, attendance tracker
- Data migration tool, text file to CSV to binary file conversion

Practice: 30 binary file handling programs, 15 CSV programs, Sumita Arora exercises, previous year board questions

Data Structures, Stack Implementation using Python List

Topics:

- What are data structures?, organizing data efficiently
- Linear vs non-linear data structures
- Stack: Last-In-First-Out (LIFO) principle
- Real-world stack analogies: stack of plates, undo/redo, browser back button
- Stack operations: push (insert at top), pop (remove from top), peek/top (view top)
- isEmpty(), check if stack is empty, size(), number of elements
- Stack overflow and stack underflow conditions
- Implementing stack using Python list: append() for push, pop() for pop
- Complete stack implementation with all operations
- Stack applications: parenthesis matching, expression evaluation

- Infix, prefix, postfix notation, understanding the concepts
- Reversing a string using stack
- Stack-based programs for board exams: book management, student records
- Tracing push and pop operations, step-by-step visualization
- Board exam pattern: stack operation output prediction, application-based questions
- Stack vs Queue (conceptual comparison)
- Time complexity of stack operations: $O(1)$ for push and pop

Projects built:

- Stack implementation, complete menu-driven program
- Parenthesis/bracket matcher using stack
- Reverse a string/list using stack
- Undo feature simulation using stack

Practice: 25 stack programs, operation tracing exercises, Sumita Arora data structure chapter, board exam stack questions from last 10 years

IsC Java Track

Inheritance & Polymorphism in Java (ISC Class 12 Focus)

Topics:

- Inheritance: creating new classes from existing classes
- extends keyword, subclass inherits from superclass
- Types of inheritance: single, multilevel, hierarchical
- Why Java doesn't support multiple inheritance (through classes)
- super keyword: calling superclass constructor, accessing superclass methods/variables
- Constructor chaining in inheritance, order of constructor execution
- Method overriding: subclass redefining superclass method
- Method overloading vs method overriding, EXAM CRITICAL comparison
- Polymorphism: one interface, multiple forms
- Compile-time polymorphism (method overloading) vs Runtime polymorphism (method overriding)
- Dynamic method dispatch / dynamic binding
- Abstract classes: abstract keyword, abstract methods
- Concrete vs abstract classes, when to use which
- Interfaces in Java: what they are, how they differ from abstract classes
- Implementing interfaces: implements keyword
- Interface vs abstract class, ISC exam comparison question
- final keyword: final variables (constants), final methods (can't override), final classes (can't inherit)
- instanceof operator, type checking
- Upcasting and downcasting

Projects built:

- Shape hierarchy: Shape Circle, Rectangle, Triangle with area/perimeter
- Employee payroll with Manager, Developer, Intern subclasses
- Vehicle inheritance: Vehicle Car, Bike, Truck
- Banking system with different account types

Practice: ISC inheritance questions (40+), output tracing with polymorphism, method dispatch problems

Advanced Data Structures, Stack & Queue in Java (ISC)

Topics:

- Stack: LIFO principle, push, pop, peek, isEmpty
- Stack implementation using arrays in Java
- Stack overflow (array full) and underflow (array empty) handling
- Stack applications: parenthesis matching, infix to postfix conversion
- Infix to prefix conversion using stack
- Postfix expression evaluation using stack
- Queue: FIFO principle, enqueue, dequeue, front, rear, isEmpty
- Queue implementation using arrays in Java
- Limitation of linear queue, wasted space
- Circular Queue: efficient queue implementation
- Circular queue operations: enqueue, dequeue with modular arithmetic
- Double-Ended Queue (Deque): insertion and deletion from both ends
- Priority Queue concepts
- Applications of queue: scheduling, BFS, printer queue
- ISC exam pattern: implementing stack/queue as a class with constructor and methods

Projects built:

- Stack class with complete implementation and menu-driven testing
- Infix to Postfix converter using stack
- Circular Queue implementation
- Postfix expression evaluator

Practice: ISC data structure questions (40+), stack/queue operation tracing, conversion problems

Linked List in Java (ISC Class 12)

Topics:

- What is a linked list?, dynamic data structure, nodes and links
- Node class: data field and next reference
- Singly Linked List: creating, traversing
- Insertion: at beginning, at end, at specific position
- Deletion: from beginning, from end, specific node by value
- Searching in a linked list, sequential traversal
- Counting nodes in a linked list
- Reversing a linked list, iterative approach
- Extracting a sublist from a linked list

- Checking if linked list is empty
- Linked list vs array, advantages and disadvantages comparison
- Memory allocation: static (arrays) vs dynamic (linked lists)
- ISC exam pattern: writing complete linked list class with all operations
- Sorted insertion in a linked list
- Merging two sorted linked lists
- Applications of linked lists: polynomial representation, sparse matrix

Projects built:

- Complete Linked List class with all operations, ISC exam format
- Polynomial addition using linked lists
- Student record management using linked list
- Sorted linked list with insertion and deletion

Practice: ISC linked list questions (30+), operation tracing, complete class implementation practice

Binary Trees & Computational Complexity (ISC Class 12)

Topics:

- Binary trees: concept, terminology, root, node, leaf, parent, child, sibling
- Tree properties: height, depth, level, degree
- Types of binary trees: full, complete, perfect, balanced
- Binary Search Tree (BST): property, left < root < right
- Tree traversals (conceptual, ISC doesn't require implementation):
- In-order traversal: Left Root Right (gives sorted order for BST)
- Pre-order traversal: Root Left Right
- Post-order traversal: Left Right Root
- Constructing BST from given data
- Tracing traversal outputs for given trees, EXAM FAVORITE
- Computational complexity: measuring algorithm efficiency
- Input size concept, how it affects running time
- Best case, average case, worst case analysis
- Big O notation: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$
- Dominant term, ignoring constants and lower-order terms
- Complexity of common algorithms: linear search $O(n)$, binary search $O(\log n)$
- Sorting complexity: bubble $O(n^2)$, selection $O(n^2)$, insertion $O(n^2)$
- Stack/Queue operations: $O(1)$
- Linked list search: $O(n)$

Projects built:

- BST traversal tracer, draw tree and show all 3 traversal outputs
- Complexity comparison chart for all algorithms learned
- Algorithm efficiency analyzer, compare running times

Practice: ISC tree traversal questions (20+), complexity analysis MCQs, BST construction exercises

Months 13-14: Sorting, Searching & Algorithm Efficiency (Week 49-56)

Applies To Cbse

Sorting Algorithms in Python (CBSE Class 12)

Topics:

- Why sorting matters, organizing data for efficient processing
- Bubble Sort: comparing adjacent elements, swapping, multiple passes
- Bubble sort algorithm: step-by-step with example array
- Bubble sort Python code with complete explanation
- Optimized bubble sort with swap flag, early termination
- Bubble sort complexity: $O(n^2)$ average/worst, $O(n)$ best (optimized)
- Selection Sort: finding minimum, placing in correct position
- Selection sort algorithm: step-by-step with example
- Selection sort Python code with explanation
- Selection sort complexity: $O(n^2)$ in all cases
- Insertion Sort: building sorted portion, inserting elements
- Insertion sort algorithm: step-by-step with example, like sorting cards
- Insertion sort Python code with explanation
- Insertion sort complexity: $O(n^2)$ average/worst, $O(n)$ best
- Comparison of all three sorting algorithms, when to use which
- Stability of sorting algorithms, bubble and insertion are stable, selection is not
- Board exam focus: tracing sort passes, writing code, comparing algorithms

Projects built:

- Sorting visualizer, show each pass of all 3 algorithms
- Performance comparison, time 1000 sorts with each algorithm
- Sort student records by name, marks, roll number

Practice: 30 sort tracing problems, write all 3 algorithms from memory, previous year board sorting questions

Searching Algorithms in Python (CBSE Class 12)

Topics:

- Linear Search: sequential checking, works on unsorted data
- Linear search algorithm and Python implementation
- Linear search: best case $O(1)$, worst case $O(n)$, average $O(n/2)$
- Binary Search: divide and conquer, requires sorted data
- Binary search algorithm: finding mid, comparing, narrowing range
- Binary search Python implementation, iterative approach
- Binary search: best case $O(1)$, worst case $O(\log n)$
- Why binary search is dramatically faster, logarithmic vs linear

- Binary search edge cases: element not found, first/last element
- Linear vs Binary search comparison, detailed table
- When to use which: unsorted data linear, sorted data binary
- Searching in lists, tuples, and dictionaries
- Python's built-in search: in operator, index() method, bisect module
- Board exam pattern: tracing binary search steps, code output prediction

Projects built:

- Search algorithm comparison tool, count comparisons for both
- Phone book search, linear for names, binary for sorted entries
- Searching programs from past 10 years board papers, complete solutions

Practice: 20 binary search tracing problems, comparison with linear search, board exam questions

----- PHASE 4: CLASS 12 SEMESTER 2, SQL, NETWORKS, CONNECTIVITY & BOARD EXAM MASTERY (MONTHS 18-24) -----

Complete the remaining Class 12 syllabus: SQL and databases, computer networks, Python-MySQL connectivity, then intensive board exam preparation with mock tests and revision.

Months 18-19: Database Management & SQL (Week 69-76)

Database Concepts & SQL Fundamentals

Topics:

- What is a database?, organized collection of related data
- File system vs DBMS: advantages of database approach
- DBMS examples: MySQL, PostgreSQL, Oracle, SQL Server, MongoDB
- Relational Database Management System (RDBMS), storing data in tables
- Relational model terminology: relation (table), attribute (column), tuple (row), domain (valid values)
- Degree of a relation: number of columns, Cardinality: number of rows
- Keys in RDBMS, EXAM CRITICAL:
- Candidate key: uniquely identifies each row (may have multiple)
- Primary key: chosen candidate key for unique identification (only one per table)
- Alternate key: candidate keys that are not primary key
- Foreign key: attribute referencing primary key of another table, establishes relationship
- Composite key: primary key made of two or more columns
- Introduction to SQL: Structured Query Language, not a programming language
- SQL categories: DDL (Data Definition), DML (Data Manipulation), DQL (Data Query)
- MySQL installation and setup, accessing MySQL command line
- SQL data types: CHAR(n), VARCHAR(n), INT, SMALLINT, FLOAT, DOUBLE, DATE, DATETIME
- CHAR vs VARCHAR: fixed vs variable length, when to use which
- Constraints: NOT NULL, UNIQUE, PRIMARY KEY, DEFAULT, CHECK

Projects built:

- Design a school database: tables for Students, Teachers, Classes, Marks
- Identify keys for 10 different real-world scenarios
- Install MySQL and create your first database and tables

Practice: 50 MCQs on database concepts, key identification exercises, Sumita Arora SQL chapter theory questions

SQL DDL & DML Commands

Topics:

- DDL, Data Definition Language (structure commands):
- CREATE DATABASE database_name;, creating a new database
- USE database_name;, selecting active database
- SHOW DATABASES; SHOW TABLES;;, listing databases and tables
- CREATE TABLE: defining table structure with columns, data types, constraints
- CREATE TABLE with PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT constraints
- DESCRIBE/DESC table_name;;, viewing table structure
- ALTER TABLE: ADD column, MODIFY column data type/size, DROP column
- ALTER TABLE: ADD PRIMARY KEY, DROP PRIMARY KEY
- DROP TABLE table_name;;, deleting entire table
- DROP DATABASE database_name;;, deleting entire database
- DML, Data Manipulation Language (data commands):
- INSERT INTO: adding single row, adding multiple rows
- INSERT INTO with column list vs without (all columns required)
- INSERT INTO with NULL values and DEFAULT values
- UPDATE: modifying existing data with SET clause
- UPDATE with WHERE clause, conditional update
- UPDATE multiple columns simultaneously
- DELETE FROM: removing rows with WHERE condition
- DELETE vs DROP: removing data vs removing structure
- CAUTION: UPDATE/DELETE without WHERE affects ALL rows

Projects built:

- Create complete school database with 5 tables
- Insert 20+ records in each table, realistic data
- Practice ALTER TABLE: add, modify, drop columns
- UPDATE and DELETE operations with various conditions

Practice: 40 DDL/DML practice queries, Sumita Arora exercises, create-insert-update-delete drill

SQL DQL, SELECT Queries, Clauses & Functions

Topics:

- SELECT statement: retrieving data from tables
- SELECT * FROM table;;, all columns, all rows
- SELECT column1, column2 FROM table;;, specific columns
- SELECT DISTINCT column FROM table;;, removing duplicates
- Column aliases: SELECT column AS alias_name
- WHERE clause: filtering rows based on conditions
- Comparison operators in WHERE: = < > <= >=
- Logical operators: AND, OR, NOT, combining conditions
- BETWEEN operator: WHERE column BETWEEN value1 AND value2
- IN operator: WHERE column IN (value1, value2, value3)
- LIKE operator: pattern matching with wildcards
- % wildcard: any number of characters, _ wildcard: single character
- LIKE patterns: 'A%' (starts with A), '%ing' (ends with ing), '%an%' (contains an)
- IS NULL and IS NOT NULL: checking for missing values
- ORDER BY: sorting results, ASC (default) and DESC
- ORDER BY multiple columns, secondary sorting
- Aggregate functions, BOARD EXAM ESSENTIALS:
- COUNT(*), COUNT(column), COUNT(DISTINCT column)
- SUM(column), AVG(column), MAX(column), MIN(column)
- Aggregate functions ignore NULL values (except COUNT(*))
- GROUP BY: grouping rows for aggregate calculations
- HAVING clause: filtering groups (WHERE filters rows, HAVING filters groups)
- Difference between WHERE and HAVING, EXAM CRITICAL
- Query execution order: FROM WHERE GROUP BY HAVING SELECT ORDER BY

Projects built:

- 50-query drill: write SELECT queries for given scenarios
- Student performance analysis: average marks by subject, topper per class
- Employee database queries: salary analysis, department-wise stats
- SQL query challenge: 20 real-world business questions

Practice: 100 SELECT query practice problems, Sumita Arora SQL exercises, previous 10 years board SQL questions

SQL Joins, Functions & Advanced Queries

Topics:

- SQL Functions, Single row functions:
- Math functions: POWER(base, exp), ROUND(num, decimals), MOD(a, b)
- TRUNCATE(num, decimals), ABS(num), CEIL(num), FLOOR(num)
- String/Text functions: UCASE()/UPPER(), LCASE()/LOWER()
- MID()/SUBSTRING()/SUBSTR(str, start, length)
- LENGTH(str), LEFT(str, n), RIGHT(str, n)
- INSTR(str, substr), find position of substring
- LTRIM(), RTRIM(), TRIM(), removing spaces
- CONCAT(str1, str2), joining strings
- Date functions: NOW(), DATE(), MONTH(), MONTHNAME()
- YEAR(), DAY(), DAYNAME(), DAYOFWEEK()
- Joins, combining data from multiple tables:
- Cartesian Product: combining every row of table1 with every row of table2
- Equi-Join: Cartesian product with equality condition on common column
- Natural Join: automatic join on common column names
- JOIN syntax: SELECT ... FROM table1, table2 WHERE table1.col = table2.col
- Table aliases in joins: SELECT a.name, b.dept FROM emp a, dept b WHERE a.deptno = b.deptno
- Self-join concept (CBSE bonus)
- Board exam pattern: write SQL query for given output, find errors in SQL

Projects built:

- Multi-table database project: Student-Marks-Subject tables with joins
- Query-the-database challenge: 30 join-based questions
- SQL function practice: 40 function-based queries
- Complete SQL revision, all concepts in one project

Practice: 50 join queries, 30 function queries, Sumita Arora advanced SQL exercises, board exam SQL from last 10 years

Boolean Algebra & Karnaugh Maps (ISC Class 12, runs parallel to SQL for CBSE)

Topics:

- Boolean algebra postulates and theorems, complete set
- Simplification using algebraic method, 10+ practice expressions
- Sum of Products (SOP) form, minterm representation
- Product of Sums (POS) form, maxterm representation
- Canonical SOP and POS from truth tables
- Karnaugh Map (K-Map): 2-variable, 3-variable, 4-variable
- Grouping rules: groups of 1, 2, 4, 8, must be powers of 2
- Don't care conditions in K-Maps
- Simplification using K-Map, extracting minimal SOP/POS
- Verification of simplified expression using truth tables
- Logic circuit design from Boolean expressions
- Implementing circuits using universal gates (NAND only, NOR only)
- Half adder and full adder circuits, truth tables, expressions, diagrams
- Encoder and decoder circuits
- Multiplexer and demultiplexer concepts

Practice: 30 algebraic simplification problems, 30 K-Map problems, circuit design practice, ISC previous year Boolean algebra questions

Months 20-21: Computer Networks & Python-MySQL Connectivity (Week 77-84)

Computer Networks, Complete Chapter (CBSE & ISC)

Topics:

- Evolution of networking: ARPANET NSFNET Modern Internet
- Data communication: sender, receiver, message, transmission medium, protocol
- Types of data: text, numbers, images, audio, video
- Bandwidth: definition, units (bps, Kbps, Mbps, Gbps)
- Communication modes: simplex, half-duplex, full-duplex
- Switching techniques: circuit switching vs packet switching
- IP addressing: IPv4 (32-bit, dotted decimal), IPv6 (128-bit)
- IP address classes: A, B, C, D, E, ranges and default subnet masks
- Public IP vs Private IP addresses
- Transmission media, Guided (wired):
 - Twisted pair cable: UTP, STP, uses, advantages
 - Coaxial cable: structure, uses (cable TV, earlier ethernet)
 - Fiber optic cable: total internal reflection, uses, advantages, fastest guided medium
- Transmission media, Unguided (wireless):
 - Radio waves: long range, omnidirectional, FM, AM, WiFi, Bluetooth
 - Microwaves: line of sight, point-to-point, cell towers
 - Infrared: short range, line of sight, TV remote, IrDA
 - Satellite communication: geostationary orbit, VSAT
- Networking devices: modem (modulator-demodulator), Ethernet card/NIC
- RJ45 connector, repeater (signal amplification), hub (broadcasts to all)
- Switch (intelligent forwarding), router (inter-network routing, IP-based)
- Gateway (protocol conversion), bridge (segment connection)
- WiFi card, access point, wireless networking

Projects built:

- Network diagram design for a school/office, choose appropriate devices and media
- IP addressing worksheet, classify 30 IP addresses, identify class
- Transmission media comparison chart, speed, cost, distance, use cases

Practice: Sumita Arora network chapters, 60 MCQs, previous year network questions, case study practice

Network Topologies, Protocols & Web Technologies

Topics:

- Network topologies, physical layout:
 - Star topology: central hub/switch, easy troubleshooting, single point of failure
 - Bus topology: single backbone cable, terminators, difficult troubleshooting
 - Tree/Hierarchical topology: combination of star and bus
 - Mesh topology: every device connected to every other, maximum redundancy
 - Ring topology: circular connection, token passing
- Types of networks based on geographical spread:
 - PAN (Personal Area Network): Bluetooth, within 10m
 - LAN (Local Area Network): within a building, Ethernet, WiFi
 - MAN (Metropolitan Area Network): across a city, cable TV network
 - WAN (Wide Area Network): across countries, Internet is largest WAN
- Network protocols, rules for communication:
 - HTTP (HyperText Transfer Protocol): web page transfer, port 80
 - HTTPS: secure HTTP with SSL/TLS encryption, port 443
 - FTP (File Transfer Protocol): file transfer, port 21
 - SMTP (Simple Mail Transfer Protocol): sending emails, port 25
 - POP3 (Post Office Protocol 3): receiving emails, port 110
 - IMAP: accessing emails on server, port 143
 - TCP/IP (Transmission Control Protocol/Internet Protocol): foundational protocol suite
 - PPP (Point-to-Point Protocol): direct connection between two nodes
 - TELNET: remote login, port 23, insecure
 - VoIP (Voice over IP): voice calls over Internet (WhatsApp calls, Zoom)
- Web concepts: WWW, HTML, XML, domain names, URL structure
 - URL components: protocol://domain:port/path?query#fragment
 - DNS (Domain Name System): translating domain names to IP addresses
 - Web browser, web server, web hosting concepts
 - Static vs dynamic web pages
 - Cookies: what they store, first-party vs third-party

Projects built:

- Network design case study, layout for multi-floor building with 200 computers
- Protocol identification quiz, match scenarios to protocols
- Web technology report, how the Internet works from URL to page load

Practice: 50 protocol MCQs, topology comparison questions, case study practice (CBSE favorite), board exam network questions

Network Security & Python-MySQL Connectivity (CBSE)

Topics:

- Network security threats:
 - Malware: virus (needs host), worm (self-replicating), trojan horse (disguised)
 - Ransomware: encrypts data, demands payment, WannaCry, NotPetya examples
 - Spyware: secretly monitors, keyloggers
 - Adware: unwanted advertisements
 - Phishing: fake websites/emails to steal credentials
 - Pharming: redirecting website traffic to fake sites
 - Denial of Service (DoS): overwhelming server with requests
 - Man-in-the-middle attack: intercepting communication

- Security measures: antivirus software, firewall (hardware/software)
- Encryption: converting data to unreadable form
- HTTP vs HTTPS: why HTTPS matters, SSL/TLS certificates
- Hacker vs cracker: ethical hacking vs malicious hacking
- Strong passwords: length, complexity, uniqueness
- Two-factor authentication (2FA)
- Python-MySQL Connectivity (CBSE Class 12 CRITICAL):
- Installing mysql.connector: pip install mysql-connector-python
- Importing: import mysql.connector
- Establishing connection: mysql.connector.connect(host, user, password, database)
- Creating cursor object: connection.cursor()
- Executing SQL queries: cursor.execute('SQL query')
- Fetching results: fetchone(), fetchall(), fetchmany(n)
- cursor.rowcount, number of rows affected
- Committing changes: connection.commit(), MANDATORY for INSERT/UPDATE/DELETE
- Closing connection: connection.close()
- Parameterized queries: preventing SQL injection with %s placeholders
- Complete CRUD operations: Create, Read, Update, Delete through Python

Projects built:

- Complete student management system, Python frontend + MySQL backend
- Library management system with Python-MySQL connectivity
- 4 Python-MySQL programs as required for practical file
- Network security awareness presentation

Practice: 30 Python-MySQL programs, connectivity code output prediction, board exam connectivity questions from last 5 years

Months 22-24: Board Exam Preparation, Intensive Revision & Mock Tests (Week 85-96)

Practical File Completion & Project Submission Preparation

Topics:

- CBSE Practical file requirements: 15+ Python programs + 5+ SQL sets + 4+ Python-MySQL programs
- ISC Practical: complete programming assignment compilation
- Python programs list: functions, file handling (text, binary, CSV), stack, sorting, searching
- SQL query sets: DDL, DML, DQL with single and multi-table queries
- Python-MySQL connectivity programs: CRUD operations
- Project selection for board practical: Python file handling OR Python-MySQL application
- Project documentation template: cover page, certificate, acknowledgement
- Project components: introduction, system requirements, source code, output screenshots, bibliography
- CBSE project ideas: Inventory management, Quiz application, Student database, E-commerce simulator
- ISC project ideas: Library system, Banking application, School management, Payroll system
- Viva voce preparation: top 50 questions with model answers
- Internal assessment preparation and marks distribution understanding

Deliverables:

- Complete practical file (printed/handwritten as per school requirement)
- Project with source code, documentation, and output
- Viva preparation notes, 50 Q&A for each subject area

Chapter-Wise Revision, Programming Topics

Topics:

- CBSE: Python revision, functions, scope, recursion (40 marks in board)
- Exception handling revision with practice programs
- File handling complete revision: text, binary (pickle), CSV
- Data structures revision: stack operations, applications
- Sorting algorithms revision: bubble, selection, insertion, code + tracing
- Searching algorithms revision: linear, binary, code + tracing
- ISC: Java revision, OOP, inheritance, polymorphism
- Data structures revision: stack, queue, circular queue, deque
- Linked list complete revision, all operations
- Binary tree traversal revision
- Recursion revision with 15 standard programs
- Complexity analysis revision, Big O for all algorithms

Practice: Topic-wise previous year questions, formula sheets, quick reference cards

Chapter-Wise Revision, SQL, Networks & Theory

Topics:

- SQL complete revision: DDL, DML, DQL commands
- SQL functions revision: math, string, date, aggregate
- GROUP BY, HAVING, ORDER BY, combined query practice
- Joins revision: Cartesian product, equi-join, natural join
- CBSE: Python-MySQL connectivity revision, connection, cursor, execute, fetch, commit
- Computer networks complete revision: media, devices, topologies, protocols
- ISC: Boolean algebra revision, laws, simplification, K-maps
- Society, law and ethics revision: cybercrime, IPR, IT Act, e-waste
- Case study questions practice, network design scenarios
- Assertion-Reason questions practice, all chapters

Practice: 100 SQL queries revision, network diagram practice, case study solving

Mock Board Exams, Set 1 & 2

Topics:

- Full-length Mock Exam 1: 70 marks, 3 hours, strict exam conditions

- Detailed solution discussion and marking scheme analysis
- Common mistakes identification and correction
- Full-length Mock Exam 2: 70 marks, 3 hours, different question set
- Detailed solution discussion with alternative approaches
- Marks improvement strategies based on mock performance
- Section-wise time allocation strategy:
- Section A (21 marks): 35 minutes, 1.5 min per MCQ
- Section B (14 marks): 20 minutes, 3 min per 2-mark question
- Section C (9-12 marks): 20 minutes, 5 min per 3-mark question
- Section D (8-16 marks): 25 minutes, 6 min per 4-mark question
- Section E (10-15 marks): 20 minutes, 8 min per 5-mark question
- Presentation tips: writing code in exams, proper indentation, comments

Deliverables:

- 2 complete mock papers solved and reviewed
- Personal weak areas identified and improvement plan
- Time management skill developed through timed practice

Mock Board Exams, Set 3, 4 & 5 + Previous Year Paper Analysis

Topics:

- Mock Exam 3, 4, 5: Full-length papers with varying difficulty
- CBSE Previous Year Papers: 2020, 2021, 2022, 2023, 2024, 2025, full solved
- ISC Previous Year Papers: 2020, 2021, 2022, 2023, 2024, 2025, full solved
- CBSE Sample Paper 2026-27, detailed solution with marking scheme
- ISC Specimen Paper 2026-27, detailed solution
- Frequently repeated questions analysis, top 30 most-asked topics
- Marking scheme tricks: how examiners award step marks
- Partial credit strategies: getting marks even if full answer unknown
- Common errors students make, and how to avoid them
- Important program outputs that repeat in exams

Practice: 5 mock papers + 6 previous year papers = 11 full papers solved

Final Revision & Last-Minute Board Exam Tips

Topics:

- One-shot revision: complete syllabus in 2 days, key points only
- Formula sheet / cheat sheet revision: all Python/Java syntax, SQL commands, network concepts
- Top 50 most important programs, practice writing by hand
- Top 30 SQL queries, write without thinking
- Top 20 output prediction questions, instant answer practice
- Board exam day strategy: reading time usage, question selection
- How to write Python code in board exams: indentation, proper syntax
- How to write SQL queries: uppercase commands, proper formatting
- How to draw network diagrams: clean, labeled, with proper legends
- Common last-minute doubts cleared
- Confidence building: you've prepared more than 95% of students
- Final practical exam preparation: lab test strategy, viva confidence

Deliverables:

- Quick revision notes, entire syllabus on 10 pages
- Top programs list, 50 most important programs
- SQL quick reference, all commands on 2 pages
- Network concepts summary, devices, protocols, topologies on 1 page
- READY FOR BOARD EXAM, 95+ score target

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Class 11 Sem 1): menu-driven console programs - calculators, grade and tax calculators, pattern printers, and number-logic programs built while learning Python (CBSE) or Java (ISC) fundamentals
- Phase 2 (Class 11 Sem 2): the complete Class 11 practical file with 20+ programs, plus a documented mini project such as a quiz game, library manager, or student database
- Phase 3 (Class 12 Sem 1): file handling utilities for text, binary (pickle), and CSV files, a record management system with search, update, and delete, and a menu-driven stack implementation
- Phase 4 (Class 12 Sem 2): SQL query sets on a live MySQL database and the final board project
 - for CBSE, a Python plus MySQL connectivity application with full documentation and viva preparation
- Total: 2 board-ready practical files, 2 submission projects, and 40+ graded programs over the two years

Total projects: 40+ graded programs and mini projects across Class 11 and 12, including 2 complete board practical files and 2 board-submission projects

SKILLS THE STUDENT WILL HAVE AT THE END

- Python Programming (CBSE 083): variables, control flow, strings, lists, tuples, dictionaries, functions, and exception handling
- File Handling: text files, binary files with the pickle module, and CSV files with the csv module
- Data Structures: stack implementation using lists with push, pop, peek, and application problems
- Databases and SQL: DDL, DML, aggregate functions, GROUP BY, HAVING, and joins on MySQL
- Python-MySQL Connectivity: connecting Python to MySQL, executing queries, and fetching

- results from a program
- Computer Networks: network types, topologies, transmission media, devices, protocols, and web basics
- Java and OOP (ISC batch): classes, constructors, inheritance, polymorphism, recursion, arrays, and data structures
- Exam Technique: output tracing, error spotting, and writing answers to the CBSE and ISC marking schemes

HOW A TYPICAL WEEK IS SPLIT

Live classes: 3-4 live classes per week, 1 hour each, taught chapter by chapter in sync with the school syllabus

Practical work: 2-3 hours of guided coding practice: practical file programs, SQL queries on a live MySQL database, and output-tracing drills

Board style practice: Weekly chapter tests plus previous year questions, sample papers, and answer writing trained on the CBSE and ISC marking schemes

Doubt support: Dedicated doubt-clearing sessions, with extra slots before school exams, pre-boards, and board exams

SUPPORT INCLUDED

Small batches: Small group batches so every student gets individual attention and screen time

Board specific batches: Separate CBSE (Python) and ISC (Java) batches - syllabus and language are never mixed

Exam season support: Extra revision classes and doubt sessions before pre-boards and board exams

Practical and project help: Step-by-step help with practical file programs and the board project until they are submission-ready

Progress tracking: Regular test results and progress updates shared with parents

CERTIFICATION DETAIL

Course certificate: Modern Age Coders Course Completion Certificate at the end of each academic year

Board exam readiness: Structured revision with 10 years of solved board papers and full mock exams on the current CBSE (083) and ISC pattern: 70 marks theory plus 30 marks practical

Practical file support: Complete guidance on every practical file program, the board project with documentation, and viva preparation for the 30-mark practical exam

WHERE THIS CAN LEAD

- B.Tech / BE in Computer Science or IT with a strong programming foundation before college begins
- BCA, B.Sc Computer Science, and B.Sc IT admissions
- JEE and other engineering entrances with Computer Science as a scoring board subject
- Indian Computing Olympiad (ZIO/ZCO) and school-level coding competitions
- College-level data structures and competitive programming, building on the Class 12 stack and algorithms work
- Python certifications such as PCEP after the course
- Early internships and freelance projects using Python and SQL
- Data science and AI/ML degree specializations that expect Python fundamentals

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Are CBSE and ICSE batches really separate?

A: Yes, absolutely. CBSE uses Python and ICSE/ISC uses Java. The syllabus, exam pattern, marking scheme, practical requirements, and even the thinking approach are different. We run completely separate batches with specialized mentors for each board.

Q: Can I join if I have zero programming experience?

A: Yes! Both our CBSE and ISC courses start from absolute scratch. We assume no prior coding experience. You'll learn everything from installing Python/Java to solving complex board-level programs.

Q: Which textbook do you follow?

A: For CBSE, we primarily follow Sumita Arora (Computer Science with Python) supplemented by NCERT textbook. For ISC, we follow Understanding Computer Science by Pandey & Dey (APC Publications) and Sumita Arora ISC edition. Our teaching covers every chapter from these books.

Q: Will this course help me score 95+ in boards?

A: We focus on both understanding and exam strategy.

Q: Do you cover practical file and project work?

A: Yes, completely. We prepare your entire practical file (20+ programs for CBSE, lab assignments for ISC), guide you through the project with documentation, and prepare you for the viva voce examination.

Q: What is the difference between CS and IP in CBSE?

A: CS (Code 083) focuses on deep Python programming, file handling, data structures (stack), sorting/searching algorithms, and Python-MySQL connectivity. IP (Code 065) focuses on data handling with Python Pandas, data visualization with Matplotlib, and SQL. CS is harder and more programming-intensive; IP is application-oriented. Both include SQL.

Q: Is ISC Computer Science harder than CBSE CS?

A: ISC CS covers more advanced topics like linked lists, binary trees, Karnaugh maps, and has a stronger focus on OOP through Java. CBSE CS has more practical connectivity topics (Python-MySQL). Both are challenging in their own ways, our separate batches ensure you're prepared for YOUR specific board.

Q: Do you help with competitive programming too?

A: While this course focuses on board exam preparation, the strong programming foundation you build here, especially in ISC with data structures and algorithms, is an excellent stepping stone for competitive programming. We offer a separate competitive programming course for those interested.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/cbse-icse-computer-science-class-11-12-python-java-complete-course>

=====

COURSE 40 OF 118: COPILOT STUDIO AGENT MASTERCLASS FOR PROFESSIONALS

=====

Course link: <https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-masterclass-for-professionals>

Subject area: AI Agents & Automation

Designed for: College students

Who can join: Intermediate to advanced (working professionals; no prior agent experience assumed, business software fluency expected)

Course length: 24 classes (12 weeks • 2 classes/week)

Time needed: 4-6 hours/week recommended

Certificate: Modern Age Coders 'Copilot Studio Agent Masterclass' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

Note: this course also needs the student to have their own Claude and ChatGPT subscriptions, because the whole course is hands on with those tools. Say this before enrolling anyone, so it is never a surprise later.

WHAT THIS COURSE IS

Ship production AI agents in Microsoft Copilot Studio: 24 live 1-on-1 online classes for working professionals on enterprise grounding, autonomous agents, multi-agent A2A, governance and ROI.

In one line: Most agent projects die in pilot. Learn to build, evaluate, govern and cost agents that survive contact with your organisation, in private 1-on-1 classes built around your actual work.

ABOUT THE PROGRAMME

Gartner projects that over 40% of agentic AI projects will be canceled by the end of 2027, and that by 2027, 40% of enterprises will demote or decommission autonomous agents due to governance failures. The builders who survive that filter are the ones who can answer the unglamorous questions: what is this agent grounded in, who approved its connector permissions, what does a conversation cost in Copilot Credits, and how do we know it is actually resolving anything? This masterclass teaches exactly that, on Microsoft Copilot Studio, the graphical low-code studio that 90% of the Fortune 500 use by Microsoft's own count. Across 24 live 1-on-1 classes you build the full professional stack: enterprise knowledge grounding across SharePoint, Dataverse and connector-indexed enterprise data with tenant graph grounding; designed topics and generative orchestration; the complete tool surface from prebuilt and premium connectors through custom connectors, REST, Prompt tools, agent flows, Model Context Protocol under DLP policies, and computer use with access control, HTTPS enforcement, stored credentials and human supervision; autonomous agents on event triggers, hardened with approval pauses and run under the maker-credentials rule; multi-agent architectures with child agents, connected agents and the A2A protocol; agent evaluations with test sets and graders; analytics down to outcomes, CSAT and the cost savings metric; and the governance layer that makes it all deployable, from mandatory Microsoft Entra Agent IDs and Conditional Access to Microsoft Agent 365 as the control plane and the Agent Store approval path. You finish with a production-grade capstone agent for your own team plus the governance and ROI pack to defend it in front of your leadership. Because every class is 1-on-1, the whole course runs on your organisation's real use cases, discussed at whatever depth your role demands. The syllabus is a Living Syllabus, re-verified against Microsoft's documentation as the platform ships, so what you learn is the August 2026 platform, not a 2024 recording.

WHAT MAKES THIS COURSE DIFFERENT

- 1-on-1 only, by design: production agent work involves your real processes, your data boundaries and your org's licensing, none of which can be discussed honestly in a group class; a private mentor can go exactly as deep as your role requires.
- Governance is taught as a first-class skill, not an afterthought: Microsoft Entra Agent ID (mandatory for new agents since July 2026), Conditional Access over connector permissions, DLP policies, Microsoft Agent 365 and the Agent Store approval path are all in the syllabus.
- Cost fluency that most builders lack: you learn the Copilot Credits model down to per-feature rates, what a Microsoft 365 Copilot licence covers at no charge, what it does not (computer use), and the 125% capacity cutoff that silently disables over-budget agents.
- The full 2026 tool surface: Prompt tools, agent flows, premium and on-premises connectors, custom connectors, REST, Model Context Protocol, computer use with enterprise guardrails, and deep reasoning, taught with their real limits and billing behaviour.
- Quality you can prove: agent evaluations (GA March 2026) with test sets, graders and multi-turn tests, plus analytics outcomes, CSAT and the cost savings metric, so your capstone ships with evidence, not vibes.
- Living Syllabus: Copilot Studio ships monthly (the current model lineup includes GPT-5.5 Chat and Anthropic Claude Opus 4.7; the A2A protocol went GA in April 2026), and this course is re-verified against Microsoft's What's New pages as features land.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac or Linux) with the Chrome or Edge browser and a reliable internet connection; Copilot Studio runs fully in the browser

Coding experience: None required. Copilot Studio is a graphical, low-code studio, and this course teaches the design, governance and cost disciplines around it. Professionals who do code can go deeper into custom connectors, REST tools and Power Fx expressions inside the same syllabus

Accounts: Copilot Studio uses a Microsoft work or school (Entra ID) account. With a work email you can self-serve the 30-day trial (which cannot publish agents); ideally you use your employer's tenant with your admin's awareness, and we arrange a practice environment where that is not possible. Your mentor sorts this out with you in the first class

Mindset: The willingness to treat agents as production software: tested before trusted, costed before scaled, and governed before shipped. The course rewards professionals who ask 'who approved this and what does it cost' as instinctively as 'does it work'

WHO THIS COURSE IS FOR

Operations and business analysts: Analysts and process owners who know exactly which workflows are broken and want to ship the agent that fixes one, with the evidence to defend it

IT and m365 admins: IT professionals and Microsoft 365 admins who will be asked to govern agents either way, and would rather be the person who understands Entra Agent ID, DLP and Agent 365 before the first incident

Developers and makers: Developers and Power Platform makers adding the 2026 agent stack (MCP, A2A, computer use, evaluations) to skills employers already recognise

Team leads and managers: Leads and managers who must decide what gets automated, what it costs in Copilot Credits, and which agent projects deserve to survive the pilot filter

Consultants and freelancers: Consultants who want to sell agent builds that pass a client's security review, with governance and ROI packs as deliverables, not afterthoughts

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: PLATFORM & FOUNDATIONS -----

Map the 2026 agent landscape, learn the anatomy of a Copilot Studio agent, build instruction and testing discipline, and make the model and harness decisions that shape cost and capability.

Class 1: The 2026 Agent Landscape and Where Copilot Studio Sits

Topics:

- What Microsoft ships in 2026: Copilot Studio as a graphical, low-code studio for building and managing AI-powered agents and workflows, run standalone at copilotstudio.microsoft.com on Power Platform infrastructure
- The ownership decision Microsoft itself recommends making first: Copilot Studio (SaaS, low-code, managed guardrails) vs Microsoft Foundry (PaaS, pro-code, fine-grained control), and how the two interoperate rather than compete
- The three build artifacts by their exact names: agents, workflows and agent flows, and which business problems map to which
- Why the market filter matters to you: Gartner's finding that over 40% of agentic AI projects will be canceled by end of 2027, and what the surviving 60% do differently
- Environments and tenants: where agents live, why production and practice environments are separated, and what your employer's admin controls
- Access honestly stated: Copilot Studio needs a Microsoft work or school (Entra ID) account; a

work email gets a 30-day self-service trial that cannot publish agents, and this course arranges a practice environment where needed

- Choosing your capstone: one real process from your own team that an agent could genuinely improve, nominated in Class 1 and refined all course

Projects built:

- Write the one-page 'agent opportunity memo' for your organisation: three candidate processes, the data each would need, the risks each carries, and which one becomes your capstone

Practice: Audit two agents you already encounter at work (a support bot, an IT helpdesk) and reverse-engineer their instructions, knowledge and escalation rules.

Assessment: Defend your capstone choice in five minutes: the process, the users, the data, the risk, and what 'better' will measurably mean.

Class 2: Agent Anatomy and Instruction Discipline

Topics:

- The anatomy every agent shares: name, description, instructions, knowledge, tools, triggers, channels, and where each is configured
- The new-experience component names as Microsoft spells them: Instructions, Knowledge, Tools and skills, Model, Connected agents, Memory
- The instruction stack for business agents: identity, scope, tone, format rules, refusals and escalation, in that order
- Scope fences that survive real users: what the agent must not answer (HR disputes, legal advice, pricing exceptions) and wording refusals that de-escalate
- Describe-to-create and Build with natural language (preview): what the generators give you and why professionals still rewrite the instructions by hand
- Instruction anti-patterns from real deployments: contradictions, vague verbs, policy text pasted verbatim, and instructions that fight the knowledge sources
- Versioning instructions outside the tool from day one, because 'what changed last Tuesday' is a governance question

Projects built:

- Build 'Policy Desk v1': an internal FAQ agent for one real policy area of your workplace, with a complete instruction stack and five tested refusals

Practice: Run the same 10 workplace questions against two instruction variants and score every difference in a comparison table.

Assessment: Present your instruction stack line by line and state, for each line, what production incident removing it would eventually cause.

Class 3: Testing Like It Is Already in Production

Topics:

- The fixed test set: happy path, edge case and adversarial questions written before you tune anything
- Adversarial testing for business agents: prompt injection attempts, policy-boundary probes, and the disgruntled-user tone test
- Regression discipline: rerunning the full set after every instruction, knowledge or tool change, and logging verdicts
- Hallucination triage: distinguishing missing knowledge, bad grounding and model overreach, because each has a different fix
- The escalation path as a tested feature: 'this needs a human' is a designed outcome, not a failure
- What you are rehearsing for: agent evaluations (GA March 2026) automate exactly this loop with test sets and graders in Phase 4
- The rule this course grades you on: no change ships without a retest, and every retest leaves a written trail

Projects built:

- Build the permanent 25-question test set for Policy Desk, including five adversarial probes, with expected behaviour written before the first run

Practice: Attempt three prompt-injection attacks on your own agent, document the outcomes, and patch the weakest point.

Assessment: Given a transcript with three subtle failures, identify each one and classify it: instructions, knowledge or escalation gap.

Class 4: Models and Harnesses: The Decisions Underneath Everything

Topics:

- The model picker in August 2026: GPT-4.1 as default, GPT-5 Chat and GPT-5.5 Chat GA, Anthropic Claude Sonnet 4.6, Claude Opus 4.6 and Claude Opus 4.7 GA, with GPT-5 Reasoning and GPT-5 Auto in preview
- External models are an admin decision: Anthropic, Mistral and xAI models must be allowed in both the Power Platform admin center and the Microsoft 365 admin center before makers can pick them
- Model release types you must read correctly: Experimental, Preview, GA, Default, Retired (GPT-4o and Claude Sonnet 4.5 are already retired), and why production agents pin to GA
- Harnesses, the 2026 architectural change: the runtime between your design and the model, in three flavours, standard harness, GitHub Copilot harness and Copilot chat harness
- The GitHub Copilot harness honestly costed: most capable, creates Word, Excel, PowerPoint and PDF files natively, supports skills and memory, and bills Copilot Credits from the moment you start building, including preview and testing
- Agents cannot move between harnesses, so the harness choice is made once, per agent, with eyes open
- The Copilot Studio extension for Visual Studio Code (GA January 2026): where low-code agents meet source control and code review

Projects built:

- Run your Policy Desk test set against two GA models (one OpenAI, one Anthropic), score the differences, and write the one-paragraph model decision with evidence attached
- Practice: Draft the admin request you would send to enable an external model in your tenant, naming both admin centers and the business justification.
- Assessment: Phase 1 gate: rebuild a small business agent from a written spec in 30 minutes, with instruction stack, refusals, a 10-question test and a defended model choice.

PHASE 2: ENTERPRISE KNOWLEDGE

Ground agents in enterprise sources at scale: SharePoint, Dataverse, documents and connector-indexed data, tenant graph grounding, curation strategy, and accuracy audits an auditor would accept.

Class 5: Grounding Strategy: The Knowledge Source Estate

Topics:

- The knowledge source lineup with real limits: public websites via Bing (25 websites), uploaded documents stored in Dataverse (512 MB per file, up to 500 files), SharePoint (25 URLs), Dataverse tables, and enterprise data via connectors indexed by Microsoft Search
- The hard ceilings that force strategy: 500 knowledge sources per agent and 1,000 topics per agent mean knowledge is curated, never dumped
- Generative answers under the hood: how the agent searches sources and composes cited answers, and what 'Allow ungrounded responses' changes
- Citations as a compliance feature: why grounded-with-sources is the difference between an answer and a liability
- Matching source type to content type: policies in SharePoint, structured records in Dataverse, public facts on the website, and why mixing them carelessly produces conflicts
- Web search as a deliberate exception: what 'Use information from the web' adds and when a curated internal agent must leave it off
- Designing the capstone's source map: authoritative, current, permissioned, and small enough to audit

Projects built:

- Ground Policy Desk in a real SharePoint site (or a mirrored practice copy) and prove ten answers cite the correct policy paragraph

Practice: Ask your grounded agent five questions its sources cannot answer and document the exact edge-of-knowledge behaviour.

Assessment: Present your capstone source map: every source, its owner, its update cadence, and why it earns its place under the 500-source discipline.

Class 6: SharePoint and Documents Done Right

Topics:

- Writing documents for retrieval: headings that answer questions, one fact per paragraph, consistent terminology, and the anti-pattern of 40-page PDFs
- SharePoint hygiene that grounding exposes: stale copies, duplicate policies, draft folders, and the librarian pass that fixes them
- Permissions reality: agents surface what the knowledge configuration can reach, so source selection is a security decision, not just a quality one
- Freshness engineering: which source updates win, what happens when the underlying document changes, and update cadences you can promise
- Testing document knowledge surgically: questions whose answers live in one exact paragraph, and what retrieval failure looks like
- The curation multiplier: five well-structured pages outperform fifty messy ones, demonstrated live on your own content
- Data minimisation at the source: what never belongs in agent knowledge (personal data, credentials, unreleased financials)

Projects built:

- Rebuild one messy real document into a retrieval-optimised version, ground both versions side by side, and publish the before-and-after accuracy comparison

Practice: Run a librarian audit on one SharePoint area you actually use: what is in, what is stale, what would embarrass the agent that cites it.

Assessment: Given two document structures, predict which grounds better and prove it with a five-question retrieval test.

Class 7: Dataverse, Connector-Indexed Data and Tenant Graph Grounding

Topics:

- Dataverse as knowledge: structured tables behind conversational answers, and when structure beats prose
- Enterprise data via connectors, indexed by Microsoft Search: reaching systems beyond Microsoft 365 without exporting them
- Tenant graph grounding with semantic search: retrieval over the tenant's Microsoft Graph, what it unlocks, and the fact that it costs extra Copilot Credits (10 per use)
- Source arbitration: what happens when SharePoint, Dataverse and a website disagree, and how instructions scope question types to sources
- Knowledge source health: monitoring which sources actually get used, and retiring the ones that never answer
- Foundry IQ and Microsoft IQ in preview: where Microsoft is taking enterprise knowledge, flagged honestly as preview, not promised

- Cost-aware knowledge design: grounding choices are billing choices, previewing the credit mathematics of Class 20

Projects built:

- Add a second source type to Policy Desk (Dataverse table or connector-indexed source), then document the arbitration behaviour with five conflict questions

Practice: Write the scoping instructions that route 'how many' and 'which record' questions to structured sources and 'what is the policy' questions to documents.

Assessment: Whiteboard the retrieval path of one question end to end: user, orchestration, source choice, retrieval, citation, answer.

Class 8: Accuracy Audits and Honest Agents

Topics:

- The accuracy audit protocol: scoring 25 grounded answers against the actual source text, with a rubric a non-builder could apply
- Uncertainty language as policy: agents that say 'my sources do not cover that' instead of improvising, and how instructions enforce it
- Bias in the source list: how a one-sided knowledge estate produces a one-sided agent, and the review that catches it
- The 'Allow ungrounded responses' decision as a governance artifact: who signs it, and for which agent categories it is defensible
- Transparency obligations: users must know they are talking to an agent and what it can reach
- The audit trail: accuracy numbers, test logs and source maps as the documents that survive an internal review
- Phase 2 gate: the grounding checklist you will apply to every agent for the rest of your career

Projects built:

- Run the full 25-answer accuracy audit on Policy Desk and publish the result as a one-page report with examples, methodology and a remediation list

Practice: Have a colleague ask your agent ten unscripted questions from their real work and log every answer they would not accept.

Assessment: Phase 2 exam: design the complete knowledge plan for a new use case in writing (sources, limits, scoping, refusals, audit protocol) in 30 minutes.

PHASE 3: ORCHESTRATION & TOOLS

Design conversations for critical paths, choose orchestration modes deliberately, then master the full tool surface: Prompt tools, agent flows, connectors, custom connectors and REST, Model Context Protocol under DLP, computer use with guardrails, and deep reasoning.

Class 9: Topics for Critical Paths

Topics:

- Topics as designed conversation: when a business process demands the same path every time, you design it, you do not hope for it
- Trigger phrases at scale: teaching the many phrasings of one request (topics support up to 200 trigger phrases)
- Question nodes, entities and variables: collecting exactly the data a process needs, with validation for messy human answers
- Condition groups (June 2026): combining tests cleanly, and the escalation branch every customer-facing topic must have
- Adaptive cards for business: structured, tappable responses for approvals, summaries and forms, designed for phones first
- System topics: greeting, escalate, goodbye, and why you customise rather than delete them
- The topic map for your capstone: the five jobs users will actually ask it to do, each with an owner path

Projects built:

- Build 'Support Triage v1': a customer-facing agent with three designed topics (report an issue, check status, escalate to human), entities capturing the details, and an adaptive card summary

Practice: Collect ten real phrasings of one request from colleagues and tune trigger phrases until all ten route correctly.

Assessment: Given four user messages, predict the topic each triggers and the variables each fills, then verify live.

Class 10: Generative Orchestration, Deliberately

Topics:

- The two orchestration modes: classic (trigger phrases select topics) versus generative (the model plans across topics, knowledge and tools), and what each buys you
- What generative orchestration unlocks: multi-intent requests, dynamic tool choice, and the event triggers and autonomy of Phase 4, which require it
- What you trade: determinism, and the instruction patterns that rein the planner back onto approved paths
- Reading the plan: inspecting how the agent chose a topic, source or tool, and turning surprises into design fixes
- The hybrid pattern professionals actually ship: designed topics for critical and regulated paths, generative flexibility for the long tail
- Re-running your guardrail tests after the mode switch, because refusals and scope fences must survive orchestration
- Microsoft's own scale heuristic, previewed: when an agent grows past 30 to 40 choices of

tools, topics and agents, you split it, which is where Phase 4's multi-agent work begins
Projects built:

- Migrate Support Triage to generative orchestration, rerun the full test set, and publish the behaviour-change log with a keep-or-fix verdict per change

Practice: Craft three multi-intent messages ('report this, and also what happened to yesterday's ticket, and who do I call') and tune until all three resolve cleanly.

Assessment: Defend your capstone's orchestration choice in writing: mode, why, and the two risks you are explicitly accepting.

Class 11: Prompt Tools and Agent Flows

Topics:

- The tool menu by its exact names: Prompt, Agent flow, Computer use, Custom connector, Model Context Protocol and REST API, plus prebuilt connectors
- Prompt tools: one perfect transformation (summarise, classify, extract, rewrite) with a tested prompt behind a stable interface
- Agent flows: deterministic multi-step automations the agent calls as tools, for anything that must run the same way every time
- Flow anatomy in practice: trigger, steps, conditions, outputs handed back to the conversation, and the error paths that make failure graceful
- The approvals pattern: a flow that pauses for a human decision, the single most reused pattern in business agents
- Tool descriptions as the real API: the orchestrator chooses tools by reading descriptions, so you write them like documentation
- Tool discipline: the platform allows 128 tools per agent under generative orchestration, Microsoft recommends 25 to 30, and good architects use fewer

Projects built:

- Build 'Expense Helper': conversation collects an expense claim, an agent flow validates it, routes it for approval, and returns the outcome, with tested error paths for every step

Practice: Break your flow deliberately (missing field, rejected approval, timeout) and verify the user sees a designed message for each failure.

Assessment: Given five business tasks, assign each to Prompt tool, agent flow, topic or knowledge, and justify each assignment in two sentences.

Class 12: The Connector Estate: Standard, Premium, On-Premises, Custom, REST

Topics:

- The scale of the estate: more than 1,400 prebuilt connectors, categorised as Standard, Premium, Custom and On-premises, with licensing implications per category
- Using prebuilt connectors as tools: reading and writing real systems (SharePoint, Outlook, Planner, third-party SaaS) from inside a conversation
- Authentication honestly explained: connection credentials, who consents to what, and why 'connect your account' is a governance event
- On-premises connectivity: reaching systems behind the firewall via the gateway, and when that is the right call versus an API
- Custom connectors: wrapping your organisation's own REST API so agents (and flows) can call it like any other connector
- The REST API tool: calling an endpoint directly, mapping JSON responses into conversation variables
- DLP preview: connector access is policy-controlled, and the same DLP machinery will govern MCP in the next class

Projects built:

- Wire one real connector tool and one REST or custom connector tool into a practice agent, and demonstrate a conversation that reads and writes live data through both

Practice: Inventory the five systems your capstone's users touch daily and classify the connector path to each: prebuilt, premium, custom or REST.

Assessment: Sketch one tool call end to end (utterance, orchestration, tool selection, auth, call, result, answer) and mark every point where governance can intervene.

Class 13: Model Context Protocol Under Enterprise Governance

Topics:

- MCP in one sentence: the open protocol for plugging tool servers into agents, added in Copilot Studio via Tools, Add a tool, Model Context Protocol
- What an MCP server exposes: Tools and Resources, with per-tool toggles and the Allow all switch you will learn to distrust
- The governance fact that makes MCP enterprise-safe here: MCP runs over Power Platform connector infrastructure, so DLP data policies govern MCP access like any connector
- Why MCP fluency compounds: the same protocol appears in Claude, IDEs and other agent platforms, so this is portable knowledge
- The Windows 365 for Agents MCP server (GA June 2026): full operational control of a Cloud PC as a case study in how much power a single MCP server can carry
- Evaluating a third-party MCP server before connecting it: capabilities, data flows, failure modes, and the least-power principle
- MCP versus custom connector versus REST: three ways to reach the same system, chosen by governance posture, not fashion

Projects built:

- Add an MCP server to a practice agent, disable every tool you do not need, and document the resulting capability surface as a one-page security note

Practice: Write the DLP review request for your MCP integration as you would submit it to your Power Platform admin, naming the data boundaries.

Assessment: Explain to a skeptical security lead, in three minutes, why this MCP

integration is governable and what would make you switch it off.

Class 14: Computer Use and Deep Reasoning: The Power Tools

Topics:

- Computer use (GA 2026): Computer-Using Agents that operate real user interfaces when no API exists, and why it is the tool of last resort, not first
- The model menu with per-step credit costs: OpenAI Computer-Using Agent at 5 credits per step, Claude Sonnet 4.6 at 5, Claude Opus 4.6 at 15 as the premium option
- The four guardrails you configure before any computer-use agent runs: Access control allow-lists, Enforce HTTPS, Stored credentials (internal or Azure Key Vault), Human supervision
- Costing a computer-use task honestly: steps multiply, credits are per step, and computer use is not covered by the Microsoft 365 Copilot licence
- Deep reasoning (preview): the Azure OpenAI o3 model invoked by the keyword 'reason' in agent instructions, US and EU availability, and why preview features stay out of production commitments
- Choosing between a designed flow, an API call and computer use for the same task: the decision tree with cost and fragility on it
- Phase 3 gate: the complete tool architecture of your capstone, every tool justified

Projects built:

- Build a governed computer-use demo: one legacy-UI task automated with allow-lists, HTTPS enforcement and human supervision configured, plus a per-run credit cost estimate

Practice: Take one process from your workplace with no API and write the computer-use feasibility note: steps, guardrails, cost per run, and the API alternative if any.

Assessment: Phase 3 exam: given a use case, design the full tool architecture (tools chosen, tools rejected, governance controls, cost notes) and defend it.

PHASE 4: AUTONOMY, MULTI-AGENT & QUALITY

Ship agents that act on their own and prove they work: event triggers under the maker-credentials rule, hardening, multi-agent architectures with A2A, agent evaluations, analytics, and full cost management in Copilot Credits.

Class 15: Event Triggers and Autonomous Agents

Topics:

- The autonomy switch: event triggers let agents act with no user in the chat, on events like item created in SharePoint, file created in OneDrive, task completed in Planner, and Recurrence schedules
- Generative orchestration required: autonomous behaviour runs on the planning brain, which is why Class 10 preceded this one
- The trigger payload: event data arriving as input, and instructions deciding what the agent does with it
- The security fact that shapes everything: triggers run with the agent maker's credentials, so every autonomous action is effectively signed by you
- Designing the first autonomous job: small, observable, reversible, and logged
- The activity trail habit: knowing exactly what your agent did at 3am, before anyone asks
- Autonomy scoping: which of your capstone's jobs deserve a trigger, and which must stay human-initiated

Projects built:

- Build 'Weekly Reporter': an autonomous agent on a Recurrence trigger that compiles a weekly status summary from real sources and posts it for review, never directly to stakeholders

Practice: Let Weekly Reporter run unattended for three real events and audit every action against what you intended, scoring each.

Assessment: Explain the maker-credentials rule and name the three design constraints you imposed on your agent because of it.

Class 16: Hardening Autonomy: Approval Pauses and Failure Design

Topics:

- From demo to dependable: idempotence (running twice must not double the damage), retries, and safe defaults when inputs surprise you
- Human-in-the-loop as architecture: which autonomous actions pause for approval, how the pause is built, and who the approver is on paper
- Notification etiquette: an autonomous agent that spams gets disabled by its own users, so frequency and channel are design decisions
- Failure diaries: structured capture of what went wrong, so Tuesday's incident is not repeated on Thursday
- Scope creep in autonomy: the 'it could also just' trap, and the change-control habit that resists it
- Gartner's warning as a design brief: 40% of enterprises will demote or decommission autonomous agents by 2027 due to governance failures, and hardened agents are how you stay in the other 60%
- The operations note: one page stating what the agent does, when, with whose permissions, and how to turn it off

Projects built:

- Harden Weekly Reporter: add an approval pause before anything leaves draft, an error diary, and a self-report of its own actions, then run it for a full real week

Practice: Write the operations note for Weekly Reporter and have a colleague verify they could disable the agent using only that page.

Assessment: Live drill: your mentor corrupts one input source; your agent must fail gracefully and your diary must show why, unedited.

Class 17: Multi-Agent Architecture and the A2A Protocol

Topics:

- When one agent becomes several: Microsoft's own heuristic, split when an agent exceeds 30 to 40 choices of tools, topics and agents
- The four ways to add agents to an agent, by their exact names: child agents, other Copilot Studio agents, agents over the A2A protocol, and (preview) Microsoft Foundry agents, Fabric Data agents and Microsoft 365 Agents SDK agents
- The A2A protocol, GA since April 2026: agent-to-agent interoperability beyond a single vendor's walls
- Child agents in practice: decomposing Support Triage into a router plus specialists, each with its own 128-tool budget
- Interop with Microsoft Foundry: low-code front door, pro-code specialist behind it, the documented enterprise pattern
- Failure across agent boundaries: timeouts, unclear hand-offs, and the tracing habits that keep multi-agent systems debuggable
- Architecture review: when multi-agent is real engineering and when it is complexity cosplay

Projects built:

- Refactor Support Triage into a two-agent architecture: a routing agent plus a specialist child agent, with documented hand-off contracts and a failure-mode table

Practice: Diagram your capstone as both a single agent and a multi-agent design, and write the three-sentence verdict on which ships.

Assessment: Defend one hand-off contract: what crosses the boundary, what must never cross it, and what happens when the far side fails.

Class 18: Agent Evaluations: Quality You Can Prove

Topics:

- Agent evaluations, GA March 2026: automated test sets run against your agent, the industrial version of the discipline you built by hand in Class 3
- Evaluation anatomy: test sets, graders scoring responses, multi-turn tests for whole conversations, and activity maps showing what the agent did
- Writing graders that mean something: correctness, groundedness, tone and policy compliance as separately scored dimensions
- The evaluation REST API (preview): wiring evaluations into release habits, flagged honestly as preview
- Evaluation-gated changes: no instruction, knowledge or tool change ships without a green evaluation run
- Building the capstone's evaluation suite: the 30 conversations that define 'working' for your use case
- Evaluations as governance evidence: the artifact you hand to a reviewer instead of a demo

Projects built:

- Build the full evaluation suite for your capstone: 30 test conversations, graders for correctness and policy compliance, and a baseline report you will beat by Class 23

Practice: Make one deliberate regression (weaken an instruction), prove the evaluation suite catches it, then revert.

Assessment: Present your grader definitions and defend why a passing score actually means the agent is safe to ship.

Class 19: Analytics: Outcomes, CSAT and the Cost Savings Metric

Topics:

- The analytics view by its real dimensions: sessions, outcomes (Resolved, Escalated, Abandoned), daily and monthly active users, CSAT, and themes
- The cost savings metric and custom metrics: putting a defensible number on what the agent is worth
- Transcript review as design fuel: session details and transcripts are retained 28 days, analytics 360 days, so review cadence is a calendar item, not a hope
- The Analytics Viewer role: giving stakeholders the numbers without giving them the editor
- Finding the gap: unanswered and escalated questions, ranked, as the roadmap for the next iteration
- The improvement loop as a weekly ritual: analytics, hypothesis, one change, evaluation run, remeasure
- Instrumenting the capstone: the three numbers that will define success in front of your leadership

Projects built:

- Analytics sprint: run one agent with at least five real users for a week, review every session, ship two data-demanded improvements, and write the before-and-after

Practice: Draft the monthly one-page agent report your leadership would actually read: outcomes, CSAT, cost savings, next actions.

Assessment: From a set of session transcripts, diagnose the top failure pattern, propose the fix, and prove it with an evaluation run.

Class 20: Cost Management: Copilot Credits Without Surprises

Topics:

- The billing currency since September 1, 2025: Copilot Credits, replacing messages, with pay-as-you-go at \$0.01 per credit and prepaid packs at \$200 per 25,000 credits per month, billed annually, unused credits not rolling over
- The per-feature rate card you estimate with: classic answer 1 credit, generative answer 2,

agent action 5, tenant graph grounding 10, agent flow actions 13 per 100, AI tools at 1, 15 or 100 per 10 responses by tier

- What the Microsoft 365 Copilot licence (at \$30 per user per month) covers at no extra charge for licensed users, and the pointed exception: computer use is not included
- The 125% rule: at 125% of prepaid capacity, custom agents are disabled and users see the unavailable message, so capacity planning is an availability feature
- GitHub Copilot harness economics: credits consumed from the moment you start building, including preview, testing and evaluation generation
- Modelling a real workload: your capstone's expected sessions, features used, credits per session, and monthly cost under pack versus pay-as-you-go
- Cost as a design input: grounding choices, tool counts and orchestration mode all move the bill, and you now know by how much

Projects built:

- Build the complete cost model for your capstone: per-session credit estimate by feature, monthly projection at three adoption levels, pack versus pay-as-you-go recommendation, and the 125% risk line

Practice: Re-estimate the cost model twice: once with tenant graph grounding on, once off, and write down what the difference buys.

Assessment: Phase 4 exam: given a described workload, produce the credit estimate, the licensing recommendation and the capacity alert plan in 30 minutes.

----- PHASE 5: GOVERNANCE, ROLLOUT & CAPSTONE -----

Master the governance layer that makes agents deployable: Entra Agent ID, Conditional Access, DLP, Microsoft Agent 365 and the Agent Store, then ship a production capstone with the governance and ROI pack to defend it.

Class 21: Identity and Policy: Entra Agent ID, Conditional Access, DLP

Topics:

- The July 2026 line in the sand: all new agents must have Microsoft Entra Agent IDs, with no opt-out, making every agent a first-class identity in your tenant
- What an agent identity buys: connector permissions surfacing as API permissions that Conditional Access policies can target
- DLP policies over the whole tool surface: connectors, custom connectors and MCP governed by the same data policies, reviewed in the Power Platform admin center
- The two admin centers in the loop: Power Platform admin center and Microsoft 365 admin center, and which decisions live where (including enabling Anthropic and other external models)
- Environment strategy as governance: who can make agents where, and why production environments have fewer makers than pilots
- Guest users cannot access Copilot Studio, and other tenant facts that shape rollout plans
- Drafting your capstone's identity and policy plan: the Agent ID, the permissions it needs, and the Conditional Access posture it will live under

Projects built:

- Write the identity and policy plan for your capstone: Entra Agent ID registration, the permission list with justifications, the DLP review outcome, and the Conditional Access recommendation

Practice: Map your organisation's (or a model organisation's) admin roles to the approvals your capstone needs, with named decision points.

Assessment: Role-play the security review: your mentor plays the identity admin, and your plan must answer every 'who approved this' question.

Class 22: The Agent Estate: Microsoft Agent 365, the Agent Store, and Rollout

Topics:

- Microsoft Agent 365, GA May 1, 2026 at \$15 per user per month: the control plane for the agent estate, with registry, access control and observability, already holding nearly 40 million registered agents by Microsoft's July 2026 count
- The Agent Store path: publishing a Copilot Studio agent to Microsoft 365 Copilot via Channels, then Microsoft 365 Copilot and Microsoft Teams, followed by admin approval in the Microsoft 365 admin center
- Channel strategy for business agents: Teams and Microsoft 365 Copilot, SharePoint, custom websites, and WhatsApp for customer-facing cases, chosen by audience, not fashion
- Versioning and change control on live channels: what re-publishing replaces and how not to break your users mid-shift
- The rollout playbook: pilot cohort, feedback loop, evaluation gates, comms, training, and the rollback plan nobody hopes to use
- The adoption evidence: IDC's projection of 1.3 billion agents by 2028 (as cited by Microsoft) and nearly 90% of the Fortune 500 with active agents on Microsoft's low-code tools (Microsoft FY26 Q3 earnings), used correctly as context, not hype
- Pre-launch gate: the 20-point production checklist covering grounding, evaluations, identity, DLP, cost and rollback

Projects built:

- Produce the rollout pack for your capstone: channel plan, pilot design, approval path through the Agent Store or internal publishing, comms draft and rollback runbook

Practice: Run the 20-point production checklist against your capstone as it stands today and file every failure into Class 23's build plan.

Assessment: Present the rollout pack in ten minutes as if to your real leadership, and survive the 'what if it goes wrong' round.

Class 23: Capstone Build Week

Topics:

- Freezing the spec: the capstone's job, users, sources, tools, triggers, channel, identity plan and cost model on one page
- Building in production order: knowledge first, topics second, tools third, triggers last, evaluations running throughout
- Timeboxing with a mentor: what gets cut when reality bites, and why the evaluated core loop is protected
- Beating your baseline: the Class 18 evaluation suite as the bar, rerun at every milestone
- Governance artifacts finalised: identity plan, DLP note, cost model and operations note, versioned alongside the agent
- Polish that earns trust: greeting, transparency statement, escalation tone, card design, error messages
- Dress rehearsal: the full demo with one planned failure and recovery, timed

Projects built:

- Build the complete capstone: a production-grade agent for your own team with grounded knowledge, designed topics, at least one tool or agent flow, one governed trigger if autonomous, a passing evaluation suite, and the full governance pack

Practice: Run the dress rehearsal for a colleague outside the course and fix the two weakest moments they point at.

Assessment: Milestone gate: the capstone beats its evaluation baseline and passes the 20-point production checklist before demo day.

Class 24: Capstone Defence and the Credential Road

Topics:

- The live defence: your capstone demonstrated on a real channel, with evaluations, analytics and the governance pack on the table
- Defending decisions like an architect: why these sources, this orchestration mode, these tools, this identity posture, this cost model
- The retrospective: what you would build differently, and the honest limits of the current platform for your use case
- The credential road mapped: Microsoft Certified: AI Agent Builder Associate (exam AB-620, covering planning and configuring agent solutions, integrating and extending agents, and testing and managing them), with the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities' as nearer milestones, and PL-900 as the on-ramp for teammates you onboard
- Where the skill curve goes: Gartner's projection that by 2029 at least half of knowledge workers will need to work with, govern or create AI agents, and by 2028 at least 15% of day-to-day work decisions will be made autonomously
- Taking it internal: turning your capstone pack into your organisation's agent playbook, and positioning yourself as the person who owns it
- Certificate, feedback, and the Living Syllabus promise: what changed in Copilot Studio while you were learning it, reviewed together

Projects built:

- Deliver the capstone defence and hand over the complete packet: spec, source map, evaluation report, analytics plan, cost model, identity and DLP plan, rollout runbook, and the leadership deck

Practice: Book the next step while momentum is real: an AB-620 study plan, or the internal pitch meeting for your agent playbook.

Assessment: Final evaluation: capstone defence, governance pack review, and the complete documentation packet, graded against the Class 22 production checklist.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: an agent opportunity memo for your organisation, Policy Desk v1 with a tested instruction stack, a 25-question adversarial test set, and a defended model-and-harness decision
- Phase 2: a SharePoint-grounded Policy Desk with citation proofs, a retrieval-optimised document rebuild with before-and-after accuracy, a multi-source arbitration study, and a published 25-answer accuracy audit
- Phase 3: Support Triage with designed topics and adaptive cards, a generative-orchestration migration log, the Expense Helper approvals flow with tested error paths, live connector and REST integrations, a governed MCP integration with a security note, and a costed computer-use demo
- Phase 4: the autonomous Weekly Reporter, its hardened successor with approval pauses and an operations note, a two-agent A2A architecture with hand-off contracts, a 30-conversation evaluation suite with graders, an analytics sprint with shipped improvements, and a complete Copilot Credits cost model
- Phase 5: an identity and policy plan (Entra Agent ID, DLP, Conditional Access), a full rollout pack with rollback runbook, the production capstone build, and the final defence with the leadership deck
- Total: 24 professional deliverables, one per class, finishing with a production capstone agent plus a governance and ROI pack

Total projects: 24 professional deliverables, one per class, finishing with a production capstone agent plus a governance and ROI pack

SKILLS THE STUDENT WILL HAVE AT THE END

- Agent design for business: instruction stacks, scope fences, refusals and escalation paths
- Enterprise knowledge grounding: SharePoint, Dataverse, documents, connector-indexed data and tenant graph grounding, within real platform limits
- Conversation design: topics, trigger phrases, entities, condition groups and adaptive cards for critical paths
- Generative orchestration, chosen deliberately and guarded with retested fences
- The full tool surface: Prompt tools, agent flows, standard, premium, on-premises and custom connectors, REST, and Model Context Protocol under DLP
- Computer use with enterprise guardrails: access control, HTTPS enforcement, stored credentials and human supervision, costed per step
- Autonomous agents: event triggers, the maker-credentials rule, approval pauses, idempotence and operations notes
- Multi-agent architecture: child agents, connected agents, the A2A protocol and Microsoft Foundry interop
- Quality engineering: agent evaluations with test sets and graders, analytics outcomes, CSAT and the cost savings metric
- Cost management: Copilot Credits rates, licence coverage, the 125% cutoff and workload cost modelling
- Governance: Microsoft Entra Agent ID, Conditional Access, DLP policies, Microsoft Agent 365 and the Agent Store approval path

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 private 1-on-1 classes per week, about 1 hour each, across 12 weeks

Hands on agent work: 1.5-2 hours building, testing and evaluating in Copilot Studio between classes

Project time: 1-1.5 hours completing each class deliverable, usually against your own organisation's use case

Review and doubts: 30-60 minutes reviewing feedback and clearing doubts over WhatsApp before the next class

SUPPORT INCLUDED

Dedicated mentor: The same mentor for all 24 classes, who knows your organisation's context, your capstone and your governance constraints

Whatsapp doubts: Doubt support between classes over WhatsApp

Practice environment: Help arranging your Copilot Studio access in Class 1: employer tenant, 30-day trial where appropriate, or a practice environment we arrange

Session recordings: Class recordings available for revision, so a missed detail is never lost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'Copilot Studio Agent Masterclass' certificate, earned by defending your Class 24 production capstone and governance pack

Project portfolio: A production capstone agent plus 24 documented professional deliverables, including the evaluation suite, cost model and governance artifacts

Next credentials: Direct preparation for Microsoft Certified: AI Agent Builder Associate (exam AB-620), with the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities' as nearer milestones

WHERE THIS CAN LEAD

- AI agent builder or automation specialist roles, the titles now appearing in real job listings as organisations move from pilots to estates
- Microsoft Certified: AI Agent Builder Associate (exam AB-620), for which this syllabus is direct preparation, with the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities' as nearer milestones
- The internal agent champion or center-of-excellence role: the person who owns the organisation's agent playbook, governance posture and cost model
- Consulting and freelance agent delivery, with a portfolio capstone that includes the governance and ROI artifacts clients' security teams actually ask for
- A bridge to code-first agent engineering: Microsoft Foundry interop from this course, and our Codex + Claude Code masterclass when you want to build agents in code

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need to be a developer to take this Copilot Studio masterclass?

A: No. Copilot Studio is a graphical, low-code studio, and the masterclass is built for professionals who own processes, not necessarily code. The hard parts it teaches are design, evaluation, governance and cost discipline, which are not programming skills.

Q: How is this different from watching Copilot Studio tutorials or Microsoft Learn modules?

A: Tutorials teach the clicks; this course builds and defends a production agent for your own team. Microsoft Learn is excellent and free, and we point you at it; what it cannot do is review your specific agent's governance pack line by line, which is what the 1-on-1 mentor does.

Q: Why is this masterclass taught 1-on-1 only?

A: Because the material is your organisation's material. Honest sessions discuss your processes, your data boundaries, your tenant's admin posture and your licensing position, none of which belongs in a group class. The mentor reviews your actual instruction stacks, source maps, evaluation results and governance artifacts every session, and adjusts depth to your role, whether you are an analyst, an admin or an architect.

Q: What do I need to get access to Copilot Studio for the course?

A: A Microsoft work or school (Entra ID) account. With a work email you can self-serve the 30-day trial, which is enough to build and test but cannot publish agents. The better path is your employer's tenant with your admin's awareness, and the course includes drafting exactly that request. Where neither is possible, we arrange a practice environment, and your mentor sorts access out with you in the first class.

Q: Which Microsoft certification does this course prepare me for?

A: The headline target is Microsoft Certified: AI Agent Builder Associate (exam AB-620), whose domains, planning and configuring agent solutions, integrating and extending agents in Copilot Studio, and testing and managing agents, map directly onto this syllabus, including MCP and the A2A protocol named in its prerequisites. On the way, the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities' are natural milestones, and PL-900 is the on-ramp we recommend for teammates you later onboard.

Q: How current is the syllabus, given how fast the platform is changing?

A: It is a Living Syllabus, re-verified against Microsoft's official documentation and What's New pages as features ship. The current version covers the August 2026 platform: the multi-model lineup including Anthropic Claude Opus 4.7, harnesses, Model Context Protocol under DLP, computer use with enterprise guardrails, the A2A protocol, agent evaluations, mandatory Entra Agent IDs and Microsoft Agent 365. When the platform changes, the course changes, and your mentor tells you what moved while you were learning.

Q: What do I walk away with at the end of the masterclass?

A: A production-grade capstone agent for your own team, demonstrated and defended on a real channel, plus the full professional packet: source map, evaluation suite and baseline report, analytics plan, Copilot Credits cost model, Entra Agent ID and DLP plan, rollout runbook and a leadership deck. You also leave with a Modern Age Coders certificate earned by that defence, and a mapped credential road to AB-620 if you want Microsoft's official stamp.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-masterclass-for-professionals>

=====

COURSE 41 OF 118: CREATIVE CODING & ANIMATION: CONTENT CREATOR COURSE

=====

Course link: <https://learn.modernagecoders.com/courses/creative-coding-animation-mastery>

Subject area: Creative Technology & Digital Media

Designed for: Kids

Who can join: Beginner to Professional Content Creator

Course length: 6 months (26 weeks)

Time needed: 10-15 hours/week recommended

Certificate: Certified Creative Technologist & Digital Animator

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 6-month creative coding and animation course: motion graphics, video editing, visual effects and social content, so you create viral reels as a pro creator.

In one line: From Concept to Viral Content - Master Every Creative Tool

ABOUT THE PROGRAMME

This intensive 6-month program transforms beginners into professional digital content creators and creative coders. Learn animation principles, master video editing, create viral social media content, and explore creative programming. Students will master industry-standard tools like After Effects, Premiere Pro, DaVinci Resolve, Blender, Processing, and more. Create stunning animations, viral reels, motion graphics, and interactive art while building a professional portfolio that stands out in the creative industry.

WHAT MAKES THIS COURSE DIFFERENT

- Complete creative toolkit mastery
- Viral content creation strategies
- Creative coding with visual output
- Industry-standard software training
- Platform-specific optimization
- Trend analysis and implementation
- Portfolio development focus
- Real client project experience

THE LEARNING PATH, STEP BY STEP

Month 1: Fundamentals: Design, Animation Basics, Video Editing
 Month 2: Motion Graphics & After Effects Mastery
 Month 3: Creative Coding with Processing & p5.js
 Month 4: 3D Animation & Advanced VFX
 Month 5: Social Media Content & Platform Mastery
 Month 6: Professional Projects & Portfolio Development

WHAT A STUDENT NEEDS BEFORE STARTING

Age: Ages 14+ (teens and adults)
 Experience: No prior design or coding experience required - complete beginner friendly
 Equipment: Computer with decent GPU (for 3D and video editing), internet connection
 Software: Free tools used: DaVinci Resolve, Blender, Processing, p5.js. After Effects (trial/subscription)
 Creativity: Passion for visual storytelling, art, or content creation

WHO THIS COURSE IS FOR

Aspiring animators: Teens and adults who dream of creating animations and motion graphics
 Content creators: Aspiring YouTubers, Instagram creators, and TikTok creators
 Design enthusiasts: Anyone passionate about visual design and digital art
 Career switchers: Professionals wanting to switch to creative technology roles
 Freelancers: People who want to offer video editing, animation, or design services
 Creative coders: Developers who want to explore the intersection of code and art

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

MONTH 1: CREATIVE FOUNDATIONS & VIDEO EDITING BASICS

Master design principles, animation fundamentals, and professional video editing.

Design Principles & Digital Art Basics

Topics:

- Elements of design: line, shape, color, texture, space
- Principles of design: balance, contrast, emphasis, movement
- Color theory: RGB/CMYK, color harmony, psychology
- Typography fundamentals and font pairing
- Composition and rule of thirds
- Visual hierarchy and flow
- Mood boards and style guides
- Design trends and aesthetics
- Digital canvas setup and resolution
- File formats: when to use what
- Asset organization and management
- Copyright and creative commons

Projects built:

- {'name': 'Personal Brand Identity', 'description': 'Logo, color palette, style guide', 'deliverables': ['Logo variations', 'Brand guidelines', 'Mockups']}
- {'name': 'Mood Board Collection', 'description': '5 different aesthetic mood boards', 'deliverables': ['Pinterest boards', 'Color schemes', 'Font selections']}
- {'name': 'Social Media Template Set', 'description': 'Consistent design templates', 'deliverables': ['Post templates', 'Story templates', 'Highlight covers']}

Practice: Pick a real poster or flyer you like, rebuild it with a stronger visual hierarchy and a deliberate color harmony, and note which design principle drove each change.

Animation Principles & 2D Basics

Topics:

- 12 principles of animation (Disney)
- Squash and stretch techniques
- Anticipation and follow-through
- Timing and spacing fundamentals
- Ease in and ease out
- Arcs and natural motion
- Secondary animation

- Frame rates and formats
- Storyboarding and animatics
- Character design basics
- Walk cycles and run cycles
- Lip sync fundamentals

Projects built:

- {'name': 'Bouncing Ball Animation', 'description': 'Master timing and physics', 'deliverables': ['30-second animation', 'Different ball types', 'Various surfaces']}
- {'name': 'Character Walk Cycle', 'description': 'Create smooth character movement', 'deliverables': ['8-frame walk cycle', '16-frame run cycle', 'Personality walks']}
- {'name': 'Animated Logo Reveal', 'description': 'Bring brand to life', 'deliverables': ['5-second reveal', '3 variations', 'Loop version']}

Practice: Animate a simple bouncing ball that shows squash and stretch plus ease in and ease out, and export a five second clip.

Video Editing Fundamentals

Topics:

- Non-linear editing concepts
- Timeline and tracks understanding
- Cutting and trimming techniques
- Transitions: cuts, fades, wipes
- Audio editing and synchronization
- Color correction basics
- Text and titles creation
- B-roll and coverage
- Pacing and rhythm in editing
- Continuity editing rules
- Jump cuts and match cuts
- Montage theory and practice

Projects built:

- {'name': 'Day in Life Vlog', 'description': 'Edit personal vlog footage', 'deliverables': ['5-minute vlog', 'Intro/outro', 'Music integration']}
- {'name': 'Product Commercial', 'description': '30-second product showcase', 'deliverables': ['Commercial edit', 'Social media cuts', 'Behind-the-scenes']}
- {'name': 'Music Video Edit', 'description': 'Sync video to music', 'deliverables': ['Full music video', 'Lyric video', 'Visualizer']}

Practice: Cut a 60 second highlight video from any raw footage using at least three transition types and synced background audio.

Mobile Video Editing & Quick Content

Topics:

- Mobile editing workflow
- Vertical video best practices
- Quick edits for social media
- Filters and effects
- Mobile color grading
- Text animations on mobile
- Stickers and overlays
- Green screen on mobile
- Mobile audio editing
- Aspect ratio management
- Export settings for platforms
- Cloud storage workflow

Projects built:

- {'name': 'Instagram Reels Series', 'description': '5 trending reels', 'deliverables': ['Dance reel', 'Tutorial reel', 'Transition reel', 'Comedy reel', 'Educational reel']}
- {'name': 'TikTok Content Pack', 'description': 'Viral-worthy content', 'deliverables': ['Trend participation', 'Original sound', 'Duet/Stitch', 'Series content']}
- {'name': 'YouTube Shorts Collection', 'description': 'Engaging short-form content', 'deliverables': ['How-to short', 'Entertainment short', 'Educational short']}

Practice: Shoot and edit a 30 second vertical reel on your phone with captions, one filter and a mobile color grade.

MONTH 2: MOTION GRAPHICS & AFTER EFFECTS MASTERY

Master motion design, visual effects, and professional animation techniques.

After Effects Fundamentals

Topics:

- After Effects interface and workflow
- Compositions and layers
- Transform properties: position, scale, rotation
- Keyframe animation basics
- Graph editor and easing
- Parenting and null objects
- Masks and shape layers
- Text animation presets
- Effects and presets library

- Rendering and export settings
- Work areas and previewing
- Project organization

Projects built:

- {'name': 'Animated Title Sequence', 'description': 'Professional title animation', 'deliverables': ['Main title', 'Lower thirds', 'End credits']}
- {'name': 'Kinetic Typography', 'description': 'Text-driven animation', 'deliverables': ['Lyric video', 'Quote animation', 'Explainer text']}
- {'name': 'Logo Animation Pack', 'description': 'Brand animations', 'deliverables': ['Logo reveal', 'Logo transition', 'Logo loop']}

Practice: Recreate a short channel intro in After Effects by keyframing position, scale and rotation on three layers, then smooth the motion in the graph editor.

Advanced Motion Graphics

Topics:

- Shape layer animation mastery
- Trim paths and stroke animation
- Morphing shapes and transitions
- 2.5D animation with cameras
- Parallax effects
- Expression basics
- Wiggle and random expressions
- Time remapping
- Motion blur and echo effects
- Particle systems basics
- Audio-driven animation
- Data-driven graphics

Projects built:

- {'name': 'Explainer Video', 'description': '2-minute animated explainer', 'deliverables': ['Full video', 'Icon animations', 'Transition pack']}
- {'name': 'Music Visualizer', 'description': 'Audio-reactive graphics', 'deliverables': ['Waveform animation', 'Particle effects', 'Spectrum analyzer']}
- {'name': 'Social Media Pack', 'description': 'Animated social elements', 'deliverables': ['Subscribe button', 'Like animation', 'Notification bells']}

Practice: Build a ten second logo reveal using trim paths and one morphing shape transition.

Character Animation & Rigging

Topics:

- Character rigging with Duik
- Puppet pin tool mastery
- Bone and IK systems
- Walk cycle in After Effects
- Facial animation techniques
- Lip sync animation
- Character emotions and acting
- Rubber hose animation
- Limber and RubberHose plugins
- Character interaction
- Scene composition
- Character design for animation

Projects built:

- {'name': 'Character Explainer', 'description': 'Character-driven narrative', 'deliverables': ['30-second story', 'Character turnaround', 'Expression sheet']}
- {'name': 'Animated Mascot', 'description': 'Brand character creation', 'deliverables': ['Idle animation', 'Action sequences', 'Reactions set']}
- {'name': 'Motion Comic', 'description': 'Comic panels animated', 'deliverables': ['1-minute story', 'Character animations', 'Scene transitions']}

Practice: Rig a simple two legged character with Duik and animate a basic four step walk cycle.

Visual Effects & Compositing

Topics:

- Green screen keying techniques
- Rotoscoping fundamentals
- Motion tracking basics
- Camera tracking introduction
- Color correction and grading
- Light and shadow effects
- Particle effects creation
- Fire, smoke, and water effects
- Glitch and distortion effects
- 3D integration basics
- Matte painting basics
- Compositing workflows

Projects built:

- {'name': 'VFX Short Film', 'description': 'Effects-heavy sequence', 'deliverables': ['1-minute VFX shot', 'Breakdown video', 'Before/after reel']}
- {'name': 'Product VFX Commercial', 'description': 'Product with effects', 'deliverables':

```
    ['30-second spot', 'Pack shots', 'Particle reveals']}]
- {'name': 'Music Video Effects', 'description': 'Visual effects for music', 'deliverables':
  ['Effect sequences', 'Transitions', 'Visual elements']}]
```

Practice: Key out a green screen clip, place the subject on a new background and match the color and shadows so the composite holds up.

MONTH 3: CREATIVE CODING & INTERACTIVE MEDIA

Master creative programming with Processing, p5.js, and interactive installations.

Processing Fundamentals

Topics:

- Introduction to creative coding
- Processing environment setup
- Coordinate system and drawing
- Shapes and colors in code
- Variables and data types
- Loops and iteration
- Conditional statements
- Functions and modularity
- Arrays and collections
- Object-oriented programming basics
- Animation with code
- Interactivity with mouse and keyboard

Projects built:

- {'name': 'Generative Art System', 'description': 'Algorithmic art creation', 'deliverables': ['Pattern generator', 'Color variations', 'Export system']}
- {'name': 'Interactive Visualization', 'description': 'Data-driven graphics', 'deliverables': ['Data parser', 'Visual representation', 'User controls']}
- {'name': 'Particle System', 'description': 'Dynamic particle effects', 'deliverables': ['Particle engine', 'Force fields', 'Collision detection']}

Practice: Code a Processing sketch that draws a repeating pattern using loops, variables and at least three colors.

p5.js and Web-Based Creative Coding

Topics:

- p5.js setup and differences from Processing
- JavaScript fundamentals for creative coding
- DOM manipulation with p5.js
- Loading and displaying media
- Sound library integration
- WebGL and 3D graphics
- Mobile interaction and touch events
- API integration and data fetching
- Local storage and cookies
- Hosting and sharing web sketches
- p5.js libraries and extensions
- Performance optimization

Projects built:

- {'name': 'Interactive Website', 'description': 'Creative coding in web design', 'deliverables': ['Landing page', 'Interactive elements', 'Mobile responsive']}
- {'name': 'Audio Visualizer Web App', 'description': 'Music-reactive visuals', 'deliverables': ['Frequency analysis', 'Visual modes', 'User controls']}
- {'name': 'Browser Game', 'description': 'Playable web game', 'deliverables': ['Game mechanics', 'Score system', 'Leaderboard']}

Practice: Port one of your Processing sketches to p5.js and add mouse interaction plus one sound trigger.

Interactive Installations & Physical Computing

Topics:

- Arduino and Processing communication
- Serial communication basics
- Sensor data visualization
- Projection mapping introduction
- Kinect and depth sensing
- Computer vision with OpenCV
- Touch surfaces and multitouch
- MIDI controller integration
- OSC protocol for networking
- Real-time data streams
- Installation planning and setup
- Documentation and presentation

Projects built:

- {'name': 'Interactive Wall', 'description': 'Projection mapping installation', 'deliverables': ['Projection setup', 'Interactive elements', 'Documentation']}
- {'name': 'Motion-Controlled Art', 'description': 'Body tracking visuals', 'deliverables': ['Skeleton tracking', 'Gesture responses', 'Visual effects']}
- {'name': 'Data Sculpture', 'description': 'Physical data visualization', 'deliverables':

['Data processing', 'Visual output', 'Physical prototype']}]}

Practice: Plan a small sensor driven installation on paper and prototype its visual response in Processing using simulated input values.

Creative Coding Applications

Topics:

- Generative design principles
- Procedural content generation
- Machine learning in creative coding
- Neural networks and art
- Style transfer techniques
- Evolutionary algorithms
- Swarm intelligence
- Biomimicry in code
- Shader programming basics
- GLSL introduction
- Live coding performances
- Creative coding for VJing

Projects built:

- {'name': 'AI Art Generator', 'description': 'ML-powered creativity', 'deliverables': ['Style transfer', 'GAN implementation', 'User interface']}
- {'name': 'Live Performance Tool', 'description': 'VJ/DJ visual system', 'deliverables': ['Audio reactive', 'MIDI control', 'Effect library']}
- {'name': 'Generative Brand System', 'description': 'Dynamic identity', 'deliverables': ['Logo variations', 'Pattern system', 'Animation set']}

Practice: Generate three variations of one generative art piece by changing the rules and the randomness, then pick the strongest for your portfolio.

MONTH 4: 3D ANIMATION & ADVANCED VISUAL EFFECTS

Master 3D animation, advanced VFX, and professional post-production.

3D Animation Fundamentals with Blender

Topics:

- Blender interface and navigation
- 3D viewport and object manipulation
- Modeling basics: primitives and modifiers
- Edit mode: vertices, edges, faces
- Sculpting introduction
- Materials and textures basics
- UV unwrapping fundamentals
- Lighting: three-point setup
- Camera animation and framing
- Keyframe animation in 3D
- Timeline and dope sheet
- Rendering with Cycles and Eevee

Projects built:

- {'name': '3D Logo Animation', 'description': 'Brand in 3D space', 'deliverables': ['3D model', 'Materials setup', 'Animated reveal']}
- {'name': 'Product Visualization', 'description': 'Photorealistic product render', 'deliverables': ['Product model', 'Studio lighting', '360° turntable']}
- {'name': 'Abstract Animation', 'description': 'Motion graphics in 3D', 'deliverables': ['Abstract forms', 'Dynamic animation', 'Render passes']}

Practice: Model a simple low poly object in Blender using primitives and modifiers, then apply basic materials and render one image.

Character Animation & Rigging in 3D

Topics:

- Character modeling workflow
- Retopology for animation
- Armature and bone creation
- IK (Inverse Kinematics) setup
- Weight painting
- Shape keys and blend shapes
- Facial rigging basics
- Animation principles in 3D
- Walk cycles in 3D
- Motion capture basics
- Non-linear animation editor
- Physics simulation for cloth and hair

Projects built:

- {'name': 'Animated Character', 'description': 'Full character pipeline', 'deliverables': ['Character model', 'Complete rig', 'Animation cycles']}
- {'name': 'Creature Animation', 'description': 'Non-human character', 'deliverables': ['Creature design', 'Unique rig', 'Performance animation']}
- {'name': 'Crowd Simulation', 'description': 'Multiple character animation', 'deliverables': ['Character variants', 'Crowd behavior', 'Scene render']}

Practice: Add an armature to a basic character mesh, weight paint two joints and pose the

character in three distinct positions.

Advanced VFX & Compositing

Topics:

- Advanced keying techniques
- Roto and paint workflows
- 3D tracking and matchmoving
- CG integration with live footage
- Advanced color grading with nodes
- Film emulation and looks
- Atmospheric effects creation
- Destruction and dynamics
- Fluid simulations basics
- Smoke and fire effects
- Advanced compositing nodes
- Multi-pass compositing

Projects built:

- {'name': 'VFX Breakdown Reel', 'description': 'Professional VFX shots', 'deliverables': ['5 VFX shots', 'Breakdown video', 'Before/after']}
- {'name': 'Invisible Effects', 'description': 'Seamless integration', 'deliverables': ['Set extensions', 'Object removal', 'Sky replacement']}
- {'name': 'Sci-Fi Sequence', 'description': 'Futuristic effects', 'deliverables': ['HUD graphics', 'Holograms', 'Energy effects']}

Practice: Track a short live action clip and composite one graphic or CG element so it sticks to the scene convincingly.

Motion Capture & Virtual Production

Topics:

- Motion capture fundamentals
- Mocap data cleanup
- Retargeting animations
- Face capture techniques
- Virtual production concepts
- Real-time rendering with Unreal
- Virtual sets and environments
- LED wall production basics
- Previsualization techniques
- Virtual cameras
- Live compositing
- Remote production workflows

Projects built:

- {'name': 'Mocap Performance', 'description': 'Captured performance piece', 'deliverables': ['Motion capture', 'Character application', 'Final render']}
- {'name': 'Virtual Set Design', 'description': 'Unreal Engine environment', 'deliverables': ['Virtual set', 'Camera moves', 'Lighting setup']}
- {'name': 'Previz Sequence', 'description': 'Pre-production visualization', 'deliverables': ['Storyboards', '3D previz', 'Shot breakdown']}

Practice: Clean up a free motion capture clip, retarget it onto a rigged character and note two artifacts you had to fix.

MONTH 5: SOCIAL MEDIA MASTERY & PLATFORM OPTIMIZATION

Master content creation for every platform, viral strategies, and audience growth.

Instagram & Reels Mastery

Topics:

- Instagram algorithm deep dive
- Reels best practices and trends
- Instagram Stories advanced features
- IGTV and long-form content
- Carousel post strategies
- Hashtag research and strategy
- Instagram SEO optimization
- Engagement tactics and community building
- Instagram Shopping and monetization
- Creator Studio analytics
- Content pillars and planning
- Cross-promotion strategies

Projects built:

- {'name': '30-Day Reels Challenge', 'description': 'Daily reel creation', 'deliverables': ['30 unique reels', 'Analytics report', 'Best performers analysis']}
- {'name': 'Brand Campaign', 'description': 'Complete IG campaign', 'deliverables': ['Feed posts', 'Stories', 'Reels', 'IGTV', 'Highlights']}
- {'name': 'Viral Reel Series', 'description': 'Series format content', 'deliverables': ['5-part series', 'Hook strategies', 'CTA optimization']}

Practice: Plan a three reel Instagram content series with a hook in the first two seconds of each and a researched hashtag set.

TikTok & Short-Form Dominance

Topics:

- TikTok algorithm mastery
- Trend identification and participation
- Original sound creation
- TikTok effects and filters
- Duets and stitches strategy
- TikTok SEO and discoverability
- Live streaming on TikTok
- TikTok ads and promotion
- Creator Fund and monetization
- Cross-platform content adaptation
- Viral content formulas
- Community building on TikTok

Projects built:

- {'name': 'TikTok Growth Sprint', 'description': 'Rapid follower growth', 'deliverables': ['Daily posts', 'Trend participation', 'Analytics tracking']}
- {'name': 'Viral Challenge Creation', 'description': 'Start a trend', 'deliverables': ['Challenge concept', 'Seed content', 'Promotion strategy']}
- {'name': 'Educational Series', 'description': 'Teaching through TikTok', 'deliverables': ['10-part series', 'Worksheet downloads', 'Community Q&A']}

Practice: Recreate one trending TikTok format with your own original twist and write a short note on your sound and caption choices.

YouTube & Long-Form Content

Topics:

- YouTube algorithm and discovery
- Channel setup and optimization
- Thumbnail design psychology
- Title and description SEO
- YouTube Shorts strategy
- Long-form vs short-form balance
- Playlist optimization
- End screens and cards
- YouTube Analytics mastery
- Community tab usage
- Premiere and live streaming
- YouTube monetization methods

Projects built:

- {'name': 'YouTube Channel Launch', 'description': 'Complete channel setup', 'deliverables': ['Channel art', 'Intro/outro', '10 videos', 'Shorts integration']}
- {'name': 'Tutorial Series', 'description': 'Educational content', 'deliverables': ['5 tutorials', 'Downloadables', 'Community engagement']}
- {'name': 'Vlog Documentary', 'description': 'Long-form storytelling', 'deliverables': ['20-minute doc', 'Trailer', 'Behind-the-scenes']}

Practice: Design two alternative thumbnails and titles for the same video idea and explain which pairing you would A/B test first.

Multi-Platform Strategy & Analytics

Topics:

- Content repurposing strategies
- Platform-specific adaptations
- Cross-platform promotion
- Content scheduling tools
- Analytics interpretation
- A/B testing methods
- ROI measurement
- Audience insights and personas
- Competitive analysis
- Trend forecasting
- Content performance metrics
- Growth hacking techniques

Projects built:

- {'name': 'Omnichannel Campaign', 'description': 'Multi-platform launch', 'deliverables': ['Platform-specific content', 'Cross-promotion plan', 'Analytics dashboard']}
- {'name': 'Content Repurposing System', 'description': 'Efficient content creation', 'deliverables': ['Content matrix', 'Adaptation templates', 'Workflow automation']}
- {'name': 'Analytics Dashboard', 'description': 'Performance tracking', 'deliverables': ['Custom dashboard', 'KPI tracking', 'Report template']}

Practice: Repurpose one long form video into a short, a carousel and a text post, each genuinely adapted to its platform.

MONTH 6: PROFESSIONAL PORTFOLIO & CLIENT PROJECTS

Build professional portfolio, work on client projects, and launch your creative career.

Portfolio Development & Personal Branding

Topics:

- Portfolio website creation
- Case study development
- Project presentation skills
- Personal brand strategy
- Logo and identity design
- Business card and stationery
- Email signature design
- LinkedIn optimization
- Behance portfolio setup
- Dribbble presence
- ArtStation for 3D work
- Showreel creation

Projects built:

- {'name': 'Portfolio Website', 'description': 'Professional online presence', 'deliverables': ['Custom website', '10 case studies', 'Blog section', 'Contact form']}
- {'name': 'Demo Reel', 'description': 'Best work compilation', 'deliverables': ['2-minute reel', 'Breakdown version', 'Platform-specific cuts']}
- {'name': 'Brand Identity Package', 'description': 'Complete personal brand', 'deliverables': ['Logo', 'Color palette', 'Typography', 'Brand guidelines']}

Practice: Draft your portfolio homepage with three case studies, each telling the problem, process and result in one paragraph.

Client Project Management

Topics:

- Client communication skills
- Project briefing and discovery
- Scope definition and contracts
- Timeline and milestone planning
- Budget estimation and quotes
- Revision handling
- File management systems
- Delivery methods and formats
- Client feedback incorporation
- Project documentation
- Time tracking tools
- Invoice and payment systems

Projects built:

- {'name': 'Mock Client Project', 'description': 'Full project simulation', 'deliverables': ['Project brief', 'Proposal', 'Final delivery', 'Case study']}
- {'name': 'Freelance Package', 'description': 'Service offerings', 'deliverables': ['Service menu', 'Pricing tiers', 'Package descriptions']}
- {'name': 'Client Onboarding System', 'description': 'Streamlined process', 'deliverables': ['Welcome packet', 'Questionnaire', 'Process guide']}

Practice: Write a one page project brief and quote for a fictional client video, including scope, timeline and two revision rounds.

Industry Specialization

Topics:

- Choosing your niche
- Industry-specific requirements
- Fashion and beauty content
- Food and beverage visuals
- Tech and startup videos
- Real estate virtual tours
- Music video production
- Wedding and event coverage
- Corporate video needs
- Educational content creation
- Gaming content production
- Non-profit storytelling

Projects built:

- {'name': 'Industry Deep Dive', 'description': 'Specialized content creation', 'deliverables': ['3 industry samples', 'Style guide', 'Network connections']}
- {'name': 'Niche Portfolio', 'description': 'Focused work collection', 'deliverables': ['5 specialized pieces', 'Industry testimonials', 'Expertise showcase']}
- {'name': 'Partnership Proposal', 'description': 'Collaboration pitch', 'deliverables': ['Partnership deck', 'Value proposition', 'Terms outline']}

Practice: Choose your niche and produce one spec piece for it, such as a food reel, a product demo or a real estate walkthrough edit.

Advanced Techniques & Emerging Trends

Topics:

- AI in creative production
- Virtual reality content
- Augmented reality filters
- 360° video production
- Interactive video formats
- Digital art: licensing, credit and provenance
- Metaverse content creation

- AI-assisted creative opportunities
- Live streaming production
- Podcast video formats
- Vertical cinema
- New platform emergence

Projects built:

- {'name': 'Emerging Tech Experiment', 'description': 'Cutting-edge content', 'deliverables': ['AR filter', 'VR experience', 'AI-generated content']}
- {'name': 'Future Format Creation', 'description': 'Next-gen content', 'deliverables': ['Interactive video', '360° experience', 'Metaverse asset']}
- {'name': 'Innovation Showcase', 'description': 'Tech-forward portfolio', 'deliverables': ['Tech demos', 'Concept pieces', 'Future vision']}

Practice: Prototype one AI assisted asset for a current project and note where the tool saved time and where you had to correct it.

Final Capstone Project

Topics:

- Project conceptualization
- Creative brief development
- Pre-production planning
- Asset creation and gathering
- Production execution
- Post-production workflow
- Quality control and review
- Final output optimization
- Multi-platform adaptation
- Launch strategy
- Marketing and promotion
- Performance tracking

Practice: Complete your capstone pre-production pack: concept statement, creative brief, shot list or storyboard, and an asset checklist.

Graduation & Career Launch

Topics:

- Portfolio finalization
- Showreel perfection
- Industry networking
- Job search strategies
- Freelance setup
- Agency applications
- Studio opportunities
- Entrepreneurship path
- Continued education
- Industry certifications
- Mentor connections
- Alumni network

Practice: Finalize your 60 to 90 second showreel and collect feedback from two people before exporting the final version.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1 (Weeks 1-4): Digital art, 2D animation, video editing fundamentals
- Month 2 (Weeks 5-8): Motion graphics, After Effects, character animation, VFX
- Month 3 (Weeks 9-12): Creative coding with Processing & p5.js, interactive media
- Month 4 (Weeks 13-16): 3D animation with Blender, advanced VFX, motion capture
- Month 5 (Weeks 17-20): Social media content mastery - Instagram, TikTok, YouTube
- Month 6 (Weeks 21-26): Professional portfolio, client projects, capstone

Total projects: 50+ creative projects + professional showreel portfolio

SKILLS THE STUDENT WILL HAVE AT THE END

- Digital Art: design principles, color theory, composition, digital painting
- Video Editing: DaVinci Resolve, CapCut, mobile editing, color grading
- Motion Graphics: After Effects, keyframing, expressions, kinetic typography
- 3D Animation: Blender, modeling, rigging, character animation, VFX
- Creative Coding: Processing, p5.js, generative art, interactive installations
- Social Media: Instagram Reels, TikTok, YouTube - platform-specific strategies
- Professional: portfolio development, client management, personal branding
- Hardware: Arduino basics, physical computing, interactive installations

HOW A TYPICAL WEEK IS SPLIT

Live sessions: 2 sessions (2 hours each)

Hands on projects: 4-6 hours of creative production

Portfolio work: 2-3 hours building and refining portfolio pieces

Total per week: 10-15 hours

SUPPORT INCLUDED

Live instruction: Expert-led live classes by industry professionals
Portfolio review: Regular portfolio reviews and feedback from mentors
Industry exposure: Guest sessions from working motion designers and animators
Community: Access to Modern Age Coders creative professionals community
Career support: Freelancing guidance, client acquisition, and portfolio strategy

CERTIFICATION DETAIL

Final certificate: Certified Creative Technologist & Digital Animator (Modern Age Coders)
Capstone required: Professional showreel + client-ready portfolio with 10+ polished pieces
Specialization: Choose specialization: Motion Graphics, 3D Animation, or Creative Coding

WHERE THIS CAN LEAD

- Motion Graphics Designer
- Content Creator / Influencer
- Video Editor
- Creative Technologist
- Animation Artist
- Social Media Manager
- VFX Artist
- Creative Director
- Freelance Designer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What skills does this creative coding and animation course cover?

A: The 6-month program moves through video editing, motion graphics in After Effects, creative coding with Processing and p5.js, 3D animation in Blender, and content strategy for Instagram, TikTok, and YouTube. Students finish with a capstone project and a professional portfolio of animations, edits, and interactive art.

Q: Who is this course suitable for?

A: It is designed for ages 14 and up, so it works for teens as well as adults exploring a creative career or a serious side skill. We teach students from ages 6 to 65 across coding and maths, and this course sits at the older, creator-focused end of that range.

Q: Do students need design or coding experience first?

A: No. The course starts with design and animation principles from zero, and the creative coding modules introduce programming gently through visual output, so beginners see results from the first weeks.

Q: Which software is used, and do we have to buy it?

A: Most of the toolkit is free: DaVinci Resolve for editing, Blender for 3D, and Processing and p5.js for creative coding. After Effects is the one paid tool in the syllabus, and we guide students on using the trial version and free alternatives until they decide whether they need it.

Q: Where can these skills lead after the course?

A: Graduates leave with a portfolio spanning editing, motion graphics, and creative code, a strong foundation for freelance work, a content channel, or deeper study in design and development. Many students pair this course with our web development track to add interactive websites to their creative range.

Q: Is there a demo class before committing?

A: Yes, and it is completely free with no obligation to continue.

Q: What is the class format?

A: Live 1-hour online classes with a real instructor, never recorded lectures. Sessions are interactive and kept to small groups, so every student presents work, receives direct feedback, and asks questions freely.

Q: Do students receive certification?

A: Yes, completing the program earns a certificate in creative coding and digital animation. Students also graduate with a professional portfolio of animations, edits, and interactive pieces, which carries real weight in creative fields.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/creative-coding-animation-mastery>

=====

COURSE 42 OF 118: CREATIVE MEDIA & DESIGN FOR TEENS: PHOTO, VIDEO & EDITING

=====

Course link: <https://learn.modernagecoders.com/courses/creative-media-design-masterclass-teens>

Subject area: Creative Media & Digital Design
Designed for: Teens
Who can join: Complete Beginner to Advanced Professional
Course length: 12 months (52 weeks)
Time needed: 12-15 hours/week recommended
Certificate: Professional Creative Designer Certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online creative media course for teens: master photo editing, video, graphic design and motion graphics in Photoshop, Premiere Pro and After Effects.

In one line: From 'First Photo Edit' to Professional Content Creator & Designer

ABOUT THE PROGRAMME

This specially crafted course transforms creative teenagers into professional digital artists and content creators in just 12 months. Whether you dream of becoming a YouTuber, Instagram influencer, filmmaker, graphic designer, or just want to create amazing content, this program teaches you everything. You'll master industry-standard software like Adobe Photoshop, Premiere Pro, After Effects, Illustrator, and more. Create stunning photos, cinematic videos, viral social media content, professional logos, animations, and build a portfolio that gets you noticed by colleges and employers.

WHAT MAKES THIS COURSE DIFFERENT

- Designed specifically for teenage creators with trendy, relevant examples
- Learn what's actually popular: TikTok editing, YouTube thumbnails, Instagram aesthetics
- No expensive equipment required - start with just a phone and computer
- Create content teens actually want to make
- Build a following while learning
- Industry-standard software training included
- Real client projects and earning opportunities
- College portfolio and career preparation

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Design Basics, Photo Editing, Basic Video, Social Media Content

Phase 2: Advanced Mastery (Months 7-12): Professional Video, Motion Graphics, Advanced Design, Portfolio

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal requirements - perfect for creative teens
Coding experience: No technical skills needed - creativity is key
Equipment: Computer/laptop (specs provided), smartphone helpful
Time commitment: 12-15 hours per week for best results
English: Basic English understanding
Motivation: Passion for creativity and visual storytelling

WHO THIS COURSE IS FOR

Students: High school students (13-18) interested in creative careers
Working professionals: Young adults wanting creative skills
Entrepreneurs: Teen entrepreneurs needing content creation skills
Freelancers: Students wanting to earn through creative work
Kids: Creative middle schoolers with parental support
Anyone: Anyone who loves creating visual content

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE CREATIVE SKILLS (MONTHS 1-6, WEEKS 1-26) -----

Build solid creative foundations. Learn design principles, master photo editing, create videos, and start building your online presence.

Months 1-2: Creative Fundamentals & Photo Editing Basics (Week 1-8)

Introduction to Digital Creativity & Setup

Topics:

- The creative industry landscape for Gen Z
- Career paths: YouTuber, Designer, Photographer, Filmmaker

- Essential hardware: Computer specs, tablets, phones
- Free vs paid software options
- Setting up Adobe Creative Cloud (student discount)
- Alternative free tools: GIMP, DaVinci Resolve, Canva
- File management and organization
- Understanding file formats: JPG, PNG, RAW, MP4, MOV
- Resolution, DPI, and pixel basics
- Color theory fundamentals: RGB vs CMYK
- Creating your first creative workspace
- Building a creative routine and staying inspired

Projects built:

- Set up complete creative workstation
- Create organized folder structure for projects
- Design a personal creative brand/logo

Practice: Daily: 30 min creative exploration, 1 hour tool familiarization

Photoshop Fundamentals

Topics:

- Photoshop interface tour and customization
- Essential panels: Layers, Properties, History
- Working with layers: Creating, organizing, blending modes
- Selection tools: Marquee, Lasso, Magic Wand, Quick Selection
- Basic adjustments: Brightness, Contrast, Levels, Curves
- Color correction: Hue/Saturation, Color Balance
- Cropping and straightening images
- Removing objects: Spot Healing, Clone Stamp, Content-Aware Fill
- Working with text: Fonts, effects, formatting
- Saving for different purposes: Web, print, social media
- Non-destructive editing with adjustment layers
- Keyboard shortcuts for speed

Projects built:

- Edit 10 personal photos to perfection
- Create viral meme templates
- Design custom phone wallpapers

Practice: Edit 5 different photos daily using various techniques

Advanced Photo Manipulation

Topics:

- Advanced selections: Select and Mask, Color Range
- Compositing: Combining multiple images seamlessly
- Sky replacement and background changes
- Portrait retouching: Skin, eyes, teeth, hair
- Frequency separation for professional retouching
- Creating realistic shadows and reflections
- Photo filters and effects: Vintage, cinematic, artistic
- Double exposure and creative effects
- Creating social media templates
- Batch processing multiple photos
- Actions and automation for efficiency
- RAW processing basics with Camera Raw

Projects built:

- Create surreal photo manipulation artwork
- Professional portrait retouching project
- Design Instagram post templates

Practice: Complete 20 photo manipulation challenges

Mobile Photography & Editing

Topics:

- Smartphone photography techniques
- Composition rules: Rule of thirds, leading lines, symmetry
- Lighting for photos: Natural vs artificial
- Portrait mode and depth effects
- Mobile editing apps: VSCO, Snapseed, Lightroom Mobile
- Creating consistent Instagram feeds
- Instagram Stories design and templates
- TikTok photo slideshow trends
- Pinterest-worthy photo editing
- Mobile photography accessories
- Shooting and editing vertical content
- Quick edits for immediate posting

Projects built:

- 30-day Instagram feed makeover
- Create viral TikTok photo series
- Mobile photography portfolio

Practice: Post one edited photo daily on social media

Months 3-4: Video Editing & Production Basics (Week 9-17)

Video Production Fundamentals

Topics:

- Video production workflow overview
- Pre-production: Planning, scripting, storyboarding
- Camera basics: Frames per second, resolution (1080p, 4K)

- Shooting techniques: Angles, movements, stability
- Smartphone videography tips and tricks
- Audio importance: Recording clean sound
- Basic lighting setups with household items
- B-roll and why it matters
- Understanding codecs and compression
- Organizing footage and project files
- Copyright and music licensing basics
- Finding royalty-free music and sound effects

Projects built:

- Shoot and plan first short film (2-3 minutes)
- Create a day-in-my-life vlog
- Record product review video

Practice: Shoot 10 different video types (slow-mo, time-lapse, etc.)

Premiere Pro Essentials

Topics:

- Premiere Pro interface and workspace setup
- Project settings and sequence creation
- Importing and organizing media
- Timeline basics: Tracks, clips, navigation
- Essential editing tools: Selection, Razor, Ripple
- Cutting techniques: J-cuts, L-cuts, match cuts
- Transitions: Cuts, fades, wipes, custom transitions
- Audio editing: Levels, fades, noise reduction
- Color correction basics: Lumetri Color panel
- Adding titles and lower thirds
- Exporting for different platforms
- Premiere Pro keyboard shortcuts

Projects built:

- Edit a cinematic travel video
- Create YouTube video with intro/outro
- Edit multi-camera interview

Practice: Edit one video daily for different platforms

YouTube Content Creation

Topics:

- YouTube algorithm and what makes videos viral
- Thumbnail design in Photoshop
- Creating clickable titles without clickbait
- YouTube video structure: Hook, content, CTA
- Editing styles: Gaming, vlog, tutorial, reaction
- Jump cuts and pacing for retention
- Adding subtitles and captions
- End screens and cards setup
- YouTube Shorts vs long-form content
- Analytics: Understanding retention graphs
- Building a consistent upload schedule
- Collaborating with other creators

Projects built:

- Complete YouTube channel setup and branding
- Create and upload 5 different video styles
- Design thumbnail templates

Practice: Analyze 20 successful YouTube videos in your niche

TikTok & Short-Form Content

Topics:

- Vertical video best practices
- TikTok trends and how to spot them
- Quick editing techniques for virality
- CapCut advanced features
- Speed ramping and velocity edits
- Text animations and reveals
- Transition mastery for TikTok
- Green screen and effects
- Instagram Reels optimization
- YouTube Shorts specific editing
- Sound sync and beat matching
- Creating series and themed content

Projects built:

- Create 30 TikToks in different styles
- Develop signature editing style
- Build following to 1000+ followers

Practice: Post one short-form video daily for 30 days

Month 3-4 Review & Portfolio Building

Topics:

- Portfolio website creation basics
- Organizing and presenting work
- Creating demo reels
- Getting first clients as a teen
- Pricing your services

Projects built:

- CAPSTONE: Create 5-minute portfolio showreel
- Build online portfolio website
- Complete first paid client project

Assessment: Submit portfolio for professional review

Months 5-6: Graphic Design & Brand Creation (Week 18-26)

Illustrator & Vector Design

Topics:

- Vector vs raster graphics explained
- Illustrator interface and tools
- Pen tool mastery: Curves, angles, handles
- Shape tools and pathfinder operations
- Creating logos from scratch
- Typography and custom lettering
- Color palettes and swatches
- Gradient and pattern creation
- Icon design for apps and web
- Business card and stationery design
- Tracing and vectorizing images
- Exporting for different uses

Projects built:

- Design 5 different logo concepts
- Create complete brand identity package
- Design custom icon set

Practice: Create one vector illustration daily

Social Media Design & Branding

Topics:

- Understanding platform specifications
- Instagram post and story templates
- Facebook cover and post designs
- Twitter/X header and post graphics
- LinkedIn professional graphics
- Pinterest pin optimization
- Consistent brand aesthetics
- Creating content calendars
- Carousel posts and slideshows
- Infographic design principles
- Data visualization basics
- Canva for quick designs

Projects built:

- 30-day social media content package
- Brand guide for personal brand
- Infographic series on trending topic

Practice: Design social media posts for 5 different brands

Print Design & Merchandise

Topics:

- Print vs digital design differences
- Understanding bleeds and margins
- CMYK color mode and print preparation
- T-shirt and apparel design
- Sticker and label design
- Poster and flyer creation
- Album and book cover design
- Magazine layout basics
- Packaging design introduction
- Print-on-demand platforms
- Setting up online merchandise store
- Copyright and trademark basics

Projects built:

- Design merch line with 10 products
- Create concert poster series
- Design and sell first products online

Practice: Create designs for different print formats

Web & UI Design Basics

Topics:

- Web design principles for beginners
- User interface (UI) fundamentals
- User experience (UX) basics
- Wireframing and prototyping
- Figma introduction and basics
- Designing responsive layouts
- Mobile app interface design
- Website banner and hero images
- Button and form design
- Color psychology in web design
- Landing page creation
- Portfolio website design

Projects built:

- Design mobile app interface

- Create landing page design
- Redesign favorite website

Practice: Daily UI challenge for 30 days

Phase 1 Capstone Project

Topics:

- Project planning and time management
- Client communication basics
- Creating project proposals
- Feedback incorporation
- Final presentation skills

Projects built:

- MAJOR CAPSTONE: Complete Brand Identity Project
- Logo, social media kit, video intro, merchandise designs
- Present to mock client panel

Assessment: Phase 1 comprehensive skills evaluation

PHASE 2: ADVANCED CREATIVE MASTERY & PROFESSIONAL SKILLS (MONTHS 7-12, WEEKS 27-52)

Master advanced video effects, motion graphics, 3D design, professional workflows, and build your creative business.

Months 7-8: Advanced Video & Visual Effects (Week 27-35)

Advanced Premiere Pro Techniques

Topics:

- Multi-camera editing workflow
- Advanced color grading with Lumetri
- Creating custom LUTs
- Secondary color correction
- Audio mastering and mixing
- Sound design and foley
- Dynamic linking with After Effects
- Proxy workflow for 4K editing
- Team projects and collaboration
- XML and project exchange
- Advanced transitions and effects
- Time remapping and speed ramping

Projects built:

- Edit professional music video
- Create documentary with color grade
- Multi-cam event coverage edit

Practice: Master 20 advanced editing techniques

After Effects Fundamentals

Topics:

- After Effects interface and workflow
- Compositions and layers
- Keyframe animation basics
- Position, scale, rotation, opacity
- Easy ease and graph editor
- Text animation and kinetic typography
- Shape layers and animations
- Masking and rotoscoping
- 3D layers and cameras
- Effects and presets
- Rendering and output settings
- Essential Graphics panel

Projects built:

- Animated logo reveal
- Kinetic typography video
- Lower thirds package

Practice: Create one motion graphic daily

Visual Effects & Compositing

Topics:

- Green screen keying techniques
- Advanced masking and rotoscoping
- Motion tracking and stabilization
- 3D camera tracking
- Particle effects and simulations
- Compositing multiple layers
- Color matching and integration
- Creating realistic shadows
- Wire and object removal
- Sky replacement in video
- Face tracking and replacement
- Creating sci-fi effects

Projects built:

- VFX short film (superhero powers)
- Green screen music video
- Before/after VFX breakdown reel

Practice: Complete 15 VFX shots from raw footage

Motion Graphics Design

Topics:

- Motion design principles
- Storyboarding for motion graphics
- Animated infographics
- Data visualization in motion
- Broadcast graphics packages
- Social media motion templates
- Animated GIFs and stickers
- Loading animations and UI motion
- Expression basics in After Effects
- Using motion graphics templates
- Creating template packages
- Client revision workflow

Projects built:

- Explainer video with infographics
- Broadcast news graphics package
- Social media motion templates set

Practice: Design 30 different motion graphics

3D Design Introduction

Topics:

- 3D design concepts and software options
- Blender basics (free alternative)
- Cinema 4D Lite in After Effects
- 3D modeling fundamentals
- Texturing and materials
- Lighting and rendering
- 3D text and logos
- Product visualization
- Augmented reality filters basics
- 3D for social media
- Integration with 2D workflows
- Future of 3D in content creation

Projects built:

- 3D animated logo
- Product visualization render
- AR filter for Instagram/Snapchat

Practice: Create 10 3D elements for videos

Months 9-10: Professional Photography & Advanced Design (Week 36-44)

Professional Photography Editing

Topics:

- Lightroom Classic workflow
- RAW processing mastery
- Catalog organization and keywords
- Advanced color grading
- Creating and selling presets
- Batch editing for consistency
- Tethered shooting setup
- HDR photography and processing
- Panorama stitching
- Focus stacking techniques
- Professional retouching workflow
- Client delivery methods

Projects built:

- Edit complete wedding photo set
- Create preset pack for sale
- Professional portrait session editing

Practice: Process 500 RAW photos professionally

Advanced Photoshop Techniques

Topics:

- Advanced compositing techniques
- Digital painting and illustration
- Photo restoration and colorization
- Creating photorealistic effects
- Advanced retouching: Dodge & burn
- Fashion and beauty retouching
- Architectural photo editing
- Product photography post-production
- Creating mockups and presentations
- Smart objects and smart filters
- Photoshop automation and scripts
- Plugin ecosystem overview

Projects built:

- Digital art masterpiece
- Restore old family photos
- Fashion magazine cover edit

Practice: Complete 20 advanced Photoshop challenges

Animation & GIF Creation

Topics:

- Principles of animation
- Frame-by-frame animation basics
- Timeline animation in Photoshop
- Creating animated GIFs
- Cinemagraphs creation
- Stop motion techniques
- Character animation basics
- Animated stickers for messaging apps
- Web animation formats
- Lottie animations introduction
- Animation for email marketing
- Optimizing file sizes

Projects built:

- Animated GIF series for brand
- Cinemagraph portfolio
- Stop motion product video

Practice: Create 30 different animations

Audio Production for Video

Topics:

- Audio fundamentals for video editors
- Adobe Audition basics
- Recording voiceovers professionally
- Podcast editing workflow
- Music mixing for video
- Sound effects and foley recording
- Audio restoration and cleanup
- Creating soundscapes
- Syncing audio and video
- Loudness standards for platforms
- Creating audio visualizers
- Spatial audio introduction

Projects built:

- Produce complete podcast episode
- Create film sound design
- Music video with audio visualization

Practice: Edit audio for 20 different projects

Live Streaming & Real-Time Content

Topics:

- Live streaming setup and equipment
- OBS Studio configuration
- Stream overlays and alerts
- Multi-camera streaming
- Green screen for streaming
- Streaming to multiple platforms
- Interactive content creation
- Virtual events production
- Live editing techniques
- Audience engagement tools
- Monetization while streaming
- Building streaming community

Projects built:

- Set up professional streaming setup
- Design complete stream package
- Host 10-hour charity stream

Practice: Stream weekly for audience building

Months 11-12: Business Skills & Career Launch (Week 45-52)

Freelancing & Client Work

Topics:

- Setting up freelance business as a teen
- Legal considerations for young freelancers
- Creating professional contracts
- Pricing strategies and packages
- Finding clients on Fiverr, Upwork
- Building local client base
- Project management tools
- Client communication best practices
- Handling revisions and scope creep
- Invoice and payment systems
- Building recurring revenue
- Scaling from freelancer to agency

Projects built:

- Launch freelance service offerings
- Complete 5 paid client projects
- Build Rs 10,000 monthly revenue

Practice: Pitch to 50 potential clients

Building Your Creative Brand

Topics:

- Personal branding for creatives
- Creating consistent online presence
- Content strategy development
- Growing social media following
- Email marketing basics
- Building email list as teen
- Creating and selling digital products
- Course and tutorial creation
- Passive income streams
- Collaboration and networking
- Speaking and workshop opportunities
- Building authority in your niche

Projects built:

- Launch personal brand across platforms
- Create and sell first digital product
- Reach 10,000 followers milestone

Practice: Post valuable content daily for 30 days

Advanced Portfolio & Showreel

Topics:

- Portfolio psychology and strategy
- Curating your best work
- Creating case studies
- Before/after presentations
- Video showreel editing
- Website portfolio best practices
- Behance and creative platforms
- LinkedIn portfolio features
- Selling digital work: licensing and marketplaces
- Print portfolio creation
- Interview presentation materials
- Continuous portfolio updates

Projects built:

- Professional portfolio website
- 3-minute showreel video
- 10 detailed case studies

Practice: Get portfolio reviewed by 10 professionals

Industry Trends & Future Skills

Topics:

- AI in creative industry
- AI art tools: Midjourney, DALL-E
- AI video editing tools
- Virtual and augmented reality
- Content for AR and virtual spaces
- Protecting your work: credit, licensing and provenance
- Sustainable design practices
- Accessibility in design
- Cultural sensitivity in content
- Global market opportunities
- Emerging platforms and formats
- Continuous learning strategies

Projects built:

- AI-assisted creative project
- VR/AR experience design
- Future-focused portfolio piece

Practice: Experiment with 5 emerging technologies

Final Showcase & Career Launch

Topics:

- Career paths in creative industry
- College applications with creative portfolio
- Internship opportunities for teens
- Building professional network
- Resume for creative positions
- Interview preparation
- Negotiating as young professional
- Continuing education options
- Industry certifications
- Mentorship and growth
- 5-year career planning
- Graduation celebration

Projects built:

- FINAL CAPSTONE: Professional Creative Agency Simulation
- Complete brand campaign: Video, photo, design, social media
- Present to industry professionals panel

Assessment: Final comprehensive evaluation and certification

This phase is not applicable for the 1-year creative course but included to maintain JSON structure integrity.

Not Applicable (N/A)

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

Not Applicable (N/A)

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

Not Applicable (N/A)

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Assessment: N/A

PHASE 4: PLACEHOLDER - STRUCTURE REQUIRED BY TEMPLATE

This phase is not applicable for the 1-year creative course but included to maintain JSON structure integrity.

Not Applicable (N/A)

N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

Not Applicable (N/A)

N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:

- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

Not Applicable (N/A)

N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

Not Applicable (N/A)

N/A
Topics:
- N/A
Projects built:
- N/A

N/A
Topics:
- N/A
Deliverables:
- N/A

N/A
Topics:
- N/A
Deliverables:
- N/A
Assessment: N/A

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-6): Instagram Feed Makeover, YouTube Channel Launch, TikTok Viral Videos, Logo Designs, Photo Manipulations
- Phase 2 (Months 7-12): Music Videos, VFX Short Films, Motion Graphics Package, Streaming Setup, Client Projects, Final Showreel

Total projects: 50+ creative projects from social media content to professional client work

SKILLS THE STUDENT WILL HAVE AT THE END

- Photo Editing: Photoshop, Lightroom, Mobile Apps, RAW Processing, Retouching
- Video Production: Premiere Pro, Filming Techniques, Color Grading, Audio Editing
- Motion Graphics: After Effects, Animation, VFX, Compositing, 3D Basics
- Graphic Design: Illustrator, Logo Design, Branding, Typography, Layout
- Social Media: Content Creation, Platform Optimization, Viral Strategies, Analytics
- Professional Skills: Client Management, Pricing, Contracts, Portfolio Building
- Trending Platforms: YouTube, TikTok, Instagram, Twitch Streaming
- Business: Freelancing, Digital Products, Personal Branding, Marketing
- Advanced: AI Tools, AR Filters, 3D Design, Live Production
- Software: Adobe Creative Suite, DaVinci Resolve, Canva, Figma, OBS

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-4 hours
Hands on coding: 6-8 hours
Projects: 3-4 hours
Practice problems: 2-3 hours
Total per week: 12-15 hours

SUPPORT INCLUDED

Live sessions: Weekly creative workshops and critiques
Mentorship: 1-on-1 guidance from industry professionals
Community: Teen creator Discord community
Code review: Professional portfolio and project reviews
Career support: Client connections and job opportunities
Lifetime access: All tutorials and future updates included

CERTIFICATION DETAIL

Phase certificates: Certificate after Phase 1 completion
Final certificate: Professional Creative Designer Certificate
Linkedin badge: Adobe Certified Associate preparation
Industry recognized: Portfolio ready for industry/college
Portfolio projects: 50+ projects for professional showcase

WHERE THIS CAN LEAD

- Social Media Content Creator
- YouTube Video Editor
- Freelance Graphic Designer
- Photo Editor/Retoucher
- Motion Graphics Designer
- Video Production Assistant
- Brand Designer
- Digital Marketing Creative
- VFX Artist (Junior)
- Creative Director (with experience)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen learn to create?

A: The year covers photo editing, video production and visual effects, graphic design and brand creation, professional photography, and motion graphics. Students build a real portfolio: edited photo sets, cinematic videos, logos and brand kits, and social media content, finishing with business and career skills in the final months.

Q: What age range is this course for?

A: It is designed for teenagers aged 13 to 18. Creative middle schoolers slightly younger can join with parental support, and the examples and projects are chosen to feel current and relevant to teens.

Q: Does my teen need any design background?

A: No, the course begins with creative fundamentals and first photo edits. Curiosity and willingness to practice matter far more than existing skill; the curriculum turns raw creativity into professional technique step by step.

Q: Do we need to buy Adobe software?

A: The curriculum teaches industry-standard tools like Photoshop, Premiere Pro, and After Effects, but students can start and practice with free alternatives such as GIMP, DaVinci Resolve, Canva, and CapCut, which the course also covers. We help each family choose a setup that fits their budget before any purchase is considered.

Q: How does this connect to future opportunities?

A: Students graduate with a portfolio that speaks for itself in college applications, freelance pitches, and creative communities. Many pair it with our creative coding and animation course to add motion design and interactive work to their range.

Q: Can my teen try before we enroll?

A: Yes, start with a free demo class, no strings attached, booked at learn.modernagecoders.com/contact. Parents can sit in on the demo; our 4.9 star rating across 547+ Google reviews mostly began with exactly that first class.

Q: Are these live classes?

A: Yes, every class is a live 1-hour online session with a real instructor, never a recorded video playlist. Small interactive groups mean each student's work gets reviewed in class, which is how editing and design actually improve.

Q: What certification does my teen receive?

A: Completing the masterclass earns a Professional Creative Designer certificate. Alongside

it, your teen graduates with a genuine creative portfolio of photos, videos, and designs, which is what colleges and clients actually look at.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/creative-media-design-masterclass-teens>

=====

COURSE 43 OF 118: CYBERSECURITY FOR TEENS: ONLINE SAFETY & WHITE-HAT DEFENSE

=====

Course link: <https://learn.modernagecoders.com/courses/cybersecurity-course-for-teens-ethical-defensive>
Subject area: Cybersecurity
Designed for: Teens
Who can join: Beginner, ages 13 to 18, no coding required
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online cybersecurity course for teens (13-18): learn how attacks work and how defense wins, from passwords and phishing to encryption and CTF puzzles.

In one line: Teens live online. This course teaches them how that world actually works, how it gets attacked, and how defenders win.

ABOUT THE PROGRAMME

Most teenagers manage more online accounts than their parents, yet nobody has ever taught them how any of it works or how it fails. This 6-month live course fixes that, and it does so honestly: this is a defensive course. Students learn how attacks work conceptually, how a phishing email is constructed, why password reuse is the root of most account theft, how ransomware spreads, because you cannot defend against what you do not understand. Nobody here is taught to break into anything, and every student signs a white-hat code of conduct in week one that we return to all course long. The arc runs from how the internet actually moves data, through passwords, hashing, and two-factor authentication, into phishing and social engineering recognition, malware concepts and safe habits, then encryption from paper ciphers to the HTTPS padlock, and web security ideas like cookies and sessions. It ends with a capture-the-flag month: instructor-built puzzle competitions in cryptography, logic, and file forensics that are legal, safe, and genuinely fun. Along the way students harden their own accounts, audit their own digital footprint, and brief their own families, so the payoff starts in week one, not after graduation.

WHAT MAKES THIS COURSE DIFFERENT

- Strictly defensive and ethical: attacks are explained conceptually so defense makes sense, and no offensive tooling is ever taught
- A white-hat ethics thread runs through every month, starting with a signed code of conduct in week one
- Students secure their real digital lives during the course: accounts hardened, backups running, families briefed
- Case studies of real incidents are read the way defenders read them: what failed, and what would have stopped it
- The capstone is a capture-the-flag event built entirely by our instructors, so every puzzle is legal, safe, and age-appropriate
- No coding required: the course is about systems and judgment, and the safe-coding week meets students at whatever level they code

THE LEARNING PATH, STEP BY STEP

Phase 1: How the digital world works and how to protect your own corner of it: networks, digital footprints, threat thinking, passwords, hashing, and 2FA
Phase 2: Attacks understood, defenses built: phishing and social engineering recognition, malware concepts, backups, and hardening real devices
Phase 3: Cryptography from paper ciphers to HTTPS, web security concepts, safe coding habits, and a capture-the-flag capstone with written solutions

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None required. The safe-coding week adapts to whatever level a student codes at, including none
Computer skills: Comfortable using a browser, apps, and settings menus
Equipment: A computer with a modern browser and a stable internet connection; no special tools are installed

Age: 13 to 18. The course discusses real scams and real incidents at a depth suited to this range

Mindset: Willingness to sign and honor the white-hat code of conduct; this course is for defenders

WHO THIS COURSE IS FOR

Every online teen: Students who live online, which is all of them, and have never been taught how any of it works or fails

Curious about hacking: Teens drawn to the hacking mystique who deserve the honest, legal, defensive version of that curiosity

Future security professionals: Students considering cybersecurity as a field who need the real foundation before any advanced study

Puzzle lovers: Teens who enjoy ciphers, logic problems, and competitions; the CTF capstone is built for them

Family tech helpers: The teen everyone in the house already asks for tech help, upgraded into the family's actual defender

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE DIGITAL WORLD AND YOUR IDENTITY (MONTHS 1-2, WEEKS 1-8)

You cannot secure what you do not understand. First the internet itself, then the single most attacked thing on it: your identity.

Month 1: How the Internet Actually Works (Weeks 1-4)

The Life of a Web Request

Topics:

- The white-hat code of conduct: what we do, what we never do, and why, signed by every student
- What happens when you type a URL: DNS, IP addresses, and servers in plain language
- Packets: how data travels in pieces and reassembles
- Clients and servers: who asks, who answers
- Watching a request travel with standard diagnostic tools like traceroute
- Why understanding the plumbing is the first defensive skill

Projects built:

- Map the full path of a web request as an annotated diagram, from keyboard to server and back

Practice: Trace three websites you use daily and note how many network hops each takes

Networks Around You

Topics:

- Home networks: router, Wi-Fi, and every device quietly connected
- Public Wi-Fi: what it can expose and sensible rules for using it
- What the cloud really is: someone else's computer, and what that implies
- Wi-Fi encryption settings and router passwords in plain terms
- The guest network: a simple defense most homes never switch on
- IoT devices: why the smart bulb is part of your security story

Projects built:

- Home network inventory: every connected device listed, with a written safety checklist for the household

Practice: Check your home router's basics with a parent: admin password changed, encryption on, firmware updated

Your Digital Footprint

Topics:

- What websites and apps collect, and where it goes
- Cookies at a first glance: useful, and worth understanding
- Search-engine results for your own name: the public footprint
- Privacy settings that actually matter on the platforms teens use
- Oversharing patterns: location tags, school names, and routines
- The permanence problem: why deleted rarely means gone

Projects built:

- Personal digital footprint audit: what is publicly findable about you, plus a written cleanup plan you execute

Practice: Complete a privacy-settings review on your two most used apps and record what you changed

Thinking Like a Defender

Topics:

- Assets, threats, and risk: security's core vocabulary made concrete
- Threat modeling a phone: what is worth stealing, who might want it, how it is reached
- Likelihood vs impact: why we defend against the boring attacks first
- Defense in depth: layers, not walls
- The human layer: why most successful attacks start with a person, not a computer
- Ethics checkpoint: knowledge of weaknesses creates responsibility, never permission

Projects built:

- Full threat model for a fictional student's phone and accounts, with ranked risks and matched defenses

Practice: Write a one-page threat model for something you own and care about

Assessment: Month 1 check: explain the path of a web request and defend your threat

model in a short viva

Month 2: Passwords, Hashing, and Proving Who You Are (Weeks 5-8)

How Passwords Fail

Topics:

- How attackers actually get passwords: guessing, leaked databases, and reuse, explained conceptually
- Credential stuffing: why one leaked password opens five accounts
- What makes a password strong: length beats cleverness
- Passphrases: four random words and why the maths favors them
- Password managers: what they are, how they help, how families adopt one
- Checking exposure: how breach-notification services like Have I Been Pwned work

Projects built:

- Password health review of your own accounts, done privately at home with a parent, with a migration plan to unique passphrases

Practice: Move your three most important accounts to strong unique passphrases and record the process, never the passwords

Hashing: Why Good Sites Never Store Your Password

Topics:

- What a hash function is: one-way, deterministic, avalanche effect
- Hands-on with SHA-256 in a browser demo: same input, same fingerprint
- Why websites store hashes instead of passwords
- Salting: why two users with the same password get different hashes
- What a data breach actually leaks, and why hashing changes the damage
- Hash checking downloads: verifying a file is what it claims to be

Projects built:

- Hashing lab worksheet: predict, test, and explain hash behavior across a set of inputs, including a tampered file

Practice: Verify the published hash of a real downloaded file and write up the steps

Two-Factor Authentication and Beyond

Topics:

- Factors: something you know, have, and are
- Authenticator apps vs SMS codes: why one resists phishing better
- Passkeys: where sign-in is heading and why
- Recovery codes: the step everyone skips until it hurts
- Account recovery hygiene: recovery email and phone numbers kept current
- When 2FA prompts themselves become the attack: fatigue prompts, conceptually

Projects built:

- Account lockdown: 2FA enabled on your key accounts with recovery codes stored safely, checklist evidence submitted

Practice: Help one family member enable 2FA on one account and note what confused them

Identity Theft and Scams That Target Teens

Topics:

- Gaming account theft: the scams built around skins, coins, and trades
- Marketplace and fan-merch scams: too good to be true, quantified
- Impersonation: fake friend accounts and what verification actually verifies
- What identity theft does to a real person: a case walk-through
- Reporting channels: platforms, banks, and India's cybercrime portal
- Ethics checkpoint: what you do when a friend's account is clearly compromised

Projects built:

- Family security briefing: a one-page document teaching your household the month's defenses, delivered as a short talk at home

Practice: Collect two real scam attempts from family chat groups and annotate the tells

Assessment: Month 2 check: passwords and identity quiz plus a live walkthrough of your family briefing

PHASE 2: ATTACKS RECOGNIZED, DEFENSES BUILT (MONTHS 3-4, WEEKS 9-16)

The two attack families that touch everyone: deception aimed at people, and malicious software aimed at machines. Recognition first, then working defenses.

Month 3: Phishing and Social Engineering (Weeks 9-12)

Anatomy of a Phishing Email

Topics:

- The standard skeleton: spoofed sender, urgent story, poisoned link
- Reading the real sender address behind a display name
- Lookalike domains: rn vs m, and the tricks typography plays
- Hovering before clicking: reading a link's true destination
- Email headers at a basic level: where a message really came from
- Why banks and platforms never ask for your password by email

Projects built:

- Phish anatomy poster: dissect one anonymized real phishing email into its labeled parts

Practice: Review your own spam folder with a parent and identify the technique in five messages

Beyond Email: Where Teens Actually Get Phished

Topics:

- Smishing: scam texts about deliveries, prizes, and blocked accounts
- Fake login pages: pixel-perfect copies and the URL that gives them away

- Discord, WhatsApp, and in-game phishing: free-item links and fake giveaways
- QR code scams: why a square of dots deserves the same suspicion as a link
- Browser warnings and what they actually mean
- The verification habit: independently checking through the real app or site

Projects built:

- Spot-the-phish gallery: grade 15 anonymized real samples as safe or suspicious, with written reasoning for each call

Practice: Build your own three-question checklist for judging any unexpected message

The Psychology of Social Engineering

Topics:

- The levers: authority, urgency, fear, and flattery
- Pretexting: attacks that begin with a believable story
- Case study read as defenders: a major breach that started with a phone call
- Why smart people fall for scams: stress and context, not stupidity
- OSINT awareness: how oversharing feeds targeted attacks, tying back to week 3
- Ethics checkpoint: persuasion skills and the bright line around consent and honesty

Projects built:

- Case study brief: reconstruct a documented social engineering incident and identify the three points where defense could have stopped it

Practice: Find the persuasion lever in five real scam messages from the class archive

The Phish Detector Field Guide

Topics:

- Consolidating the month into a usable field guide
- Writing detection rules a grandparent could follow
- Reporting phishing properly: platform tools and national portals
- What to do after a wrong click: damage control steps in order
- Designing the class recognition tournament
- Teaching as testing: explaining a tell proves you understand it

Projects built:

- Phish detector field guide: an illustrated recognition and response guide, then the class phishing-recognition tournament

Practice: Test your field guide on a family member and revise the rule they misunderstood

Assessment: Month 3 check: timed recognition tournament across 20 fresh samples, plus field guide review

Month 4: Malware Concepts and Safe Habits (Weeks 13-16)

The Malware Families

Topics:

- Viruses, worms, and trojans: what each name actually means
- Ransomware: encryption turned against you, conceptually
- Spyware and keyloggers: the quiet observers
- Adware and unwanted programs: the grey zone
- How infections really begin: attachments, cracked software, fake updates, and bundled installers
- Why cracked games are the classic teen infection route

Projects built:

- Malware family tree: an illustrated chart mapping each family to its behavior and its most common arrival route

Practice: Write plain-language definitions of five malware terms for a younger sibling

Defenses That Actually Work

Topics:

- Updates as defense: what a patch is and why delay is dangerous
- Antivirus and built-in protections: what they catch and what they miss
- Least privilege at home: why daily life should not run on an admin account
- Sandboxing as a concept: why apps on a phone cannot read each other's data
- Backups and the 3-2-1 rule: the defense that defeats ransomware
- Restore drills: an untested backup is a hope, not a plan

Projects built:

- Personal backup plan: design and actually implement a 3-2-1 backup for your own important files, with a test restore

Practice: Run updates on your devices with a parent and document what was pending

Case Study: A Ransomware Outbreak, Read as Defenders

Topics:

- The WannaCry outbreak: what happened, in sequence
- The unpatched systems that let it spread
- Who was hit and what it cost, including hospitals
- The defenses that worked where it failed to spread
- What a recovery effort looks like from the inside
- Ethics checkpoint: responsible disclosure, and the researchers who reported flaws the right way

Projects built:

- Defender's timeline: reconstruct the outbreak and mark every point where standard defenses from week 14 would have changed the outcome

Practice: Read one news report on a recent incident and identify which defense was missing

Hardening Your Own Machines

Topics:

- Safe downloading: official sources, and reading installer screens instead of clicking next

- Browser hygiene: extensions audited, permissions reviewed
- App permissions on phones: flashlight apps that want your contacts
- Recognizing fake download buttons and bundled junk
- The family device sweep: applying the whole month at home
- Writing a maintenance routine you will actually follow

Projects built:

- Harden-your-browser lab: full extension and permission audit on your own browser and phone, with before-and-after notes

Practice: Complete the device sweep on one family computer and log every change made

Assessment: Month 4 check: malware and defenses quiz plus review of your implemented backup and hardening evidence

PHASE 3: CRYPTOGRAPHY, THE WEB, AND THE CTF CAPSTONE (MONTHS 5-6, WEEKS 17-24)

The mathematics of secrets, the trust machinery of the web, safer coding habits, and a capture-the-flag capstone that turns six months of defense into sport.

Month 5: Encryption and How the Web Stays Trustworthy (Weeks 17-20)

Classical Ciphers by Hand

Topics:

- The Caesar cipher: shifting the alphabet, and breaking it by brute force
- Substitution ciphers and keys
- Frequency analysis: why English's love of the letter E breaks substitution
- The Vigenere idea: why longer keys resist analysis
- A short history: ciphers, wars, and the codebreakers of Bletchley Park
- What classical failures teach modern cryptography

Projects built:

- Cipher exchange: encode messages for classmates, then break intercepted ones using frequency analysis worksheets

Practice: Break three prepared ciphertexts of increasing difficulty and document your method

Modern Encryption, Conceptually

Topics:

- Symmetric encryption: one shared key, and the delivery problem it creates
- Public-key encryption: the open padlock idea that solved key delivery
- Digital signatures: proving who wrote a message
- Key length and why brute force fails against modern ciphers
- End-to-end encryption: what your messaging apps actually promise
- Why nobody invents their own cipher: the first rule of cryptography

Projects built:

- Concept comic or storyboard: explain the padlock analogy for public-key encryption well enough to teach it

Practice: Identify which of your daily apps use end-to-end encryption and what that covers

Cookies, Sessions, and the Padlock

Topics:

- What logged in actually means: sessions and cookies doing their real jobs
- Why stolen cookies matter: the session as a temporary key
- HTTPS vs HTTP: what the padlock encrypts and what it does not promise
- Certificates: how your browser decides whom to trust
- Certificate warnings: what they mean and when to walk away
- Public Wi-Fi revisited with new eyes: what HTTPS protects there and what it cannot

Projects built:

- Trust tour: inspect certificates and cookies on five real sites using browser dev tools, documenting what each reveals

Practice: Find one HTTP-only site in the wild and write up why the warning matters

Safe Coding Habits

Topics:

- Why apps get hacked, conceptually: trusting user input is the root of most classics
- Validating input: the habit, shown at whatever level students code
- Secrets never go in code: passwords, keys, and the accidental publish
- Dependencies age: why software built on old parts inherits their holes
- A gentle look at injection as an idea: data that gets treated as instructions
- Ethics checkpoint: finding a flaw in a real site means reporting, never poking further

Projects built:

- Code safety review: audit one of your own past projects, from any language or even Scratch, against a provided safety checklist

Practice: Add input validation to one small program you have written, or trace validation in a provided example

Assessment: Month 5 check: cryptography and web trust quiz plus cipher-breaking practical

Month 6: The Capture-the-Flag Capstone (Weeks 21-24)

Welcome to CTF

Topics:

- What a capture-the-flag competition is, and its role in real security education
- Our rules: every puzzle is instructor-built, self-contained, legal, and safe
- Puzzle categories: cipher breaking, logic, metadata forensics, and web-trust trivia

- Reading a puzzle: what is given, what is asked, what is hidden
- The solver's notebook: keeping notes that turn into writeups
- Practice round one, solved together as a class

Projects built:

- Practice CTF round: five starter puzzles solved individually with notes, then reviewed together

Practice: Solve three take-home puzzles and log your approach for each, including dead ends

Training Camp

Topics:

- Cipher puzzles under time: applying month 5 at speed
- File forensics-lite: what image metadata and file properties reveal
- Logic and encoding puzzles: binary, hexadecimal, and Base64 as everyday costumes for data
- Teamwork in CTFs: splitting categories and sharing notes
- Hint discipline: when to take a hint and what it costs
- The writeup format: challenge, approach, solution, lesson

Projects built:

- Team training rounds in every category, plus one polished writeup of your favorite solved puzzle

Practice: Daily puzzle streak: one puzzle per day with a two-line log entry each

The Capstone CTF Event

Topics:

- Final event briefing and team formation
- Competition strategy: triage easy points first
- The event itself: multi-category, scored, across class sessions
- Staying methodical under a clock
- Sportsmanship and collaboration within the rules
- Post-event solution reveal for every unsolved puzzle

Projects built:

- The capstone CTF: a scored, multi-category competition built from the entire course's material

Practice: Draft writeups for every puzzle your team solved while they are fresh

Writeups and the White-Hat Road Ahead

Topics:

- Polishing writeups into a portfolio piece
- Responsible disclosure as a practice: how professionals report what they find
- Defensive security careers surveyed honestly: analyst, incident response, security engineering, and the study each requires
- Where to keep learning: age-appropriate CTF platforms and safety-first communities
- The code of conduct, revisited as a graduation promise
- Presentations and certificates

Projects built:

- Final CTF writeup portfolio presented to the class, alongside your family briefing and audit documents from the course

Practice: Write a one-page letter to your future self on the security habits you intend to keep

Assessment: Capstone assessment: CTF performance, writeup portfolio, and a short ethics viva

PROJECTS THE STUDENT BUILDS AND KEEPS

- Annotated map of a web request's full path
- Home network inventory with a household safety checklist
- Personal digital footprint audit with an executed cleanup plan
- Threat model for a fictional student's phone and accounts
- Account lockdown: passphrases, 2FA, and recovery codes on real accounts
- Family security briefing, written and delivered at home
- Phish detector field guide plus recognition tournament
- Implemented 3-2-1 backup plan with a test restore
- Defender's timeline of a real ransomware outbreak
- Capture-the-flag writeup portfolio from the capstone event

Total projects: 10+ defensive artifacts: audits, field guides, hardened accounts, and CTF writeups

SKILLS THE STUDENT WILL HAVE AT THE END

- Networks and the web: how data moves, where trust lives, and what the padlock really promises
- Identity defense: passphrases, hashing, 2FA, and recovery done properly on real accounts
- Attack recognition: phishing, social engineering, and malware understood well enough to spot cold
- Practical hardening: backups, updates, permissions, and browser hygiene, implemented not just discussed
- Cryptography foundations: from breaking Caesar ciphers by hand to explaining public-key encryption
- White-hat judgment: ethics, responsible disclosure, and the habit of reporting rather than exploiting

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, with demonstrations, case studies, and hands-on labs

Practice: 2-3 hours between classes: audits, puzzles, and real hardening tasks on the student's own devices

Review: Every artifact reviewed with written feedback; the ethics thread checked in every month

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, including help when a real security question comes up at home

Progress updates: Regular progress notes to parents, who also receive the family briefing their teen produces in month 2

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, backed by a portfolio of audits and CTF writeups

WHERE THIS CAN LEAD

- Further defensive study toward security analyst and incident response roles, which build directly on these concepts
- Age-appropriate CTF platforms and school cyber competitions, entered with real preparation
- Our Python and programming courses, where the safe-coding habits from month 5 get applied for real
- Computer science electives and projects with a security perspective few classmates will have
- The informal but real role of family and school security helper, taken on responsibly

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Will my teen learn to hack?

A: Not in the sense the movies mean, and we are upfront about that. This is a defensive course: students learn how attacks work conceptually, how phishing is built, how malware spreads, how passwords get stolen, because recognition is the heart of defense. Nobody is taught intrusion tools or techniques for breaking into systems, every student signs a white-hat code of conduct in week one, and the capture-the-flag puzzles are instructor-built, self-contained, and legal.

Q: Is this course safe and legal?

A: Yes, by design. Every exercise runs on the student's own accounts and devices or on materials we built for the class; nothing ever touches systems that are not ours to touch. The course actively teaches the legal and ethical lines, including responsible disclosure, so students finish knowing not just what is possible but what is right. Parents are involved directly: several assignments, like the account lockdown and home network review, are done with a parent.

Q: Does my teen need to know coding first?

A: No. The course is about systems, judgment, and habits rather than programming, and most weeks involve no code at all. The one safe-coding week in month 5 meets each student at their level, whether that is Scratch, some Python, or nothing yet. Students who do code get a useful bonus: they learn to look at their own projects the way an attacker would, and fix what they find.

Q: What does my teen actually produce in this course?

A: Real defensive artifacts, most of them immediately useful: a digital footprint audit with an executed cleanup, hardened accounts with strong passphrases and 2FA, a working 3-2-1 backup with a test restore, a phishing field guide, a family security briefing delivered at home, and a portfolio of capture-the-flag writeups from the capstone. The family typically becomes measurably safer while the course is still running.

Q: Is this useful for a cybersecurity career?

A: It is the correct first step, and we will not overclaim beyond that. Professional security roles require further study, in networking, operating systems, and usually programming, but they all build on exactly the concepts this course teaches: threat thinking, cryptography basics, authentication, and attack recognition. A teen who finishes this course knows whether the field genuinely interests them and holds a foundation most college freshmen lack. The safety payoff, meanwhile, is immediate rather than someday.

Q: What is a CTF, and is it appropriate for teens?

A: Capture the flag is the puzzle-competition format used across security education worldwide: players solve challenges, break a practice cipher, find data hidden in a file's metadata, decode a message, to find hidden flags and score points. Ours is entirely instructor-built, so every puzzle is age-appropriate, legal, and self-contained. It is the most anticipated month of the course, and the writeups students produce are genuine analytical work.

Q: How are the classes taught?

A: Live one-hour online classes, twice a week, with a real instructor and small groups capped at 10 students. Mini-batch (3 to 4 students) and personal 1-on-1 options are available.

Q: Can we try a class before enrolling?

A: Yes, the first demo class is free with no obligation. Book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to observe, and for this course in particular we encourage it: you will see exactly how the defensive, ethics-first approach works in practice.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/cybersecurity-course-for-teens-ethical-defensive>

=====

COURSE 44 OF 118: DATA ANALYSIS COURSE: FROM EXCEL TO MACHINE LEARNING

=====

Course link: <https://learn.modernagecoders.com/courses/data-analysis-mastery-course-college>
Subject area: Data Science & Business Analytics
Designed for: College students
Who can join: Complete Beginner to Professional Analyst
Course length: 6 months (26 weeks)
Time needed: 15-20 hours/week recommended
Certificate: Professional Data Analyst Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online data analysis course: master Excel, SQL, Python, statistics and visualization, and turn raw data into insights as a job-ready data analyst.

In one line: From Spreadsheet Basics to Advanced Analytics in 6 Months

ABOUT THE PROGRAMME

This intensive 6-month program transforms complete beginners into professional data analysts capable of extracting insights from complex datasets, creating compelling visualizations, and driving data-driven decisions in any organization. You'll master the complete analytics toolkit: Excel for quick analysis, SQL for database querying, Python for advanced analytics, statistics for rigorous analysis, and modern BI tools for interactive dashboards. Learn to clean messy data, perform exploratory analysis, build predictive models, and present findings that influence business strategy.

WHAT MAKES THIS COURSE DIFFERENT

- Start from absolute basics - no prerequisites
- Real business datasets and case studies
- Industry-standard tools and techniques
- Portfolio of 25+ analysis projects
- Business communication and storytelling focus
- Covers both technical and business aspects
- Hands-on with Fortune 500 company data

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-2): Excel Mastery, Statistics Basics, Data Fundamentals
Phase 2: Technical Skills (Months 3-4): SQL, Python, Data Cleaning, Analysis Techniques
Phase 3: Advanced Analytics (Months 5-6): Visualization, BI Tools, ML Basics, Business Intelligence

WHAT A STUDENT NEEDS BEFORE STARTING

Education: High school mathematics helpful
Technical skills: Basic computer skills required
Equipment: Computer with MS Office/Google Sheets
Time commitment: 15-20 hours per week
English: Good comprehension required
Motivation: Curiosity about data and insights

WHO THIS COURSE IS FOR

Students: College students seeking analytical careers
Working professionals: Career switchers to data field
Business professionals: Managers wanting data skills

Excel users: Excel users wanting to advance
Entrepreneurs: Business owners needing insights
Marketers: Digital marketers seeking analytics
Anyone: Anyone interested in data-driven decisions

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & EXCEL MASTERY (MONTHS 1-2, WEEKS 1-8) -----

Build strong foundations in data concepts, statistics, and become an Excel power user.

Month 1: Data Fundamentals & Excel (Week 1-4)

Introduction to Data Analysis

Topics:

- What is data analysis? Role in modern business
- Types of data: structured, unstructured, semi-structured
- Data analysis lifecycle and methodology
- Quantitative vs qualitative analysis
- Descriptive, diagnostic, predictive, prescriptive analytics
- Industry applications and case studies
- Tools ecosystem overview
- Career paths in data analysis
- Setting up your analysis environment
- Introduction to datasets and data sources
- Data ethics and privacy considerations
- Building analytical thinking

Projects built:

- Personal data audit project
- Industry analysis report
- Data source catalog creation

Practice: Analyze 10 real-world data scenarios

Excel Fundamentals & Formulas

Topics:

- Excel interface mastery and shortcuts
- Data types and formatting
- Cell references: absolute, relative, mixed
- Essential formulas: SUM, AVERAGE, COUNT, MIN, MAX
- Logical functions: IF, AND, OR, NOT
- Nested IF statements and IFS function
- Text functions: CONCATENATE, LEFT, RIGHT, MID, TRIM
- Date and time functions
- Mathematical functions: ROUND, ABS, MOD
- Error handling: IFERROR, IFNA
- Formula auditing and debugging
- Named ranges and structured references

Projects built:

- Sales calculator dashboard
- Employee timesheet system
- Grade calculation spreadsheet

Practice: Complete 50 Excel formula challenges

Excel Data Management

Topics:

- Data validation and drop-down lists
- Sorting and multi-level sorting
- Filtering and advanced filters
- Remove duplicates and data cleaning
- Text to columns and data parsing
- Flash Fill for pattern recognition
- Data consolidation techniques
- Grouping and outlining
- Subtotals and summaries
- Working with large datasets
- Excel tables and structured data
- Data import/export techniques

Projects built:

- Customer database cleanup
- Inventory management system
- Survey data processor

Practice: Clean and organize 20 messy datasets

Excel Advanced Functions

Topics:

- LOOKUP functions: VLOOKUP, HLOOKUP, XLOOKUP
- INDEX and MATCH combinations
- SUMIF, COUNTIF, AVERAGEIF functions
- SUMIFS for multiple criteria
- Array formulas and dynamic arrays

- FILTER, SORT, UNIQUE functions
- Statistical functions in Excel
- Financial functions: NPV, IRR, PMT
- Database functions: DSUM, DCOUNT
- INDIRECT and OFFSET for dynamic references
- Creating custom functions with VBA basics
- Excel automation introduction

Projects built:

- Dynamic sales report system
- Financial analysis model
- HR analytics dashboard

Practice: Build 25 complex Excel solutions

Month 2: Statistics & Data Visualization in Excel (Week 5-8)

Statistics Fundamentals

Topics:

- Types of data: nominal, ordinal, interval, ratio
- Population vs sample
- Measures of central tendency: mean, median, mode
- Measures of spread: range, variance, standard deviation
- Quartiles and interquartile range
- Skewness and kurtosis
- Normal distribution and z-scores
- Central limit theorem basics
- Confidence intervals introduction
- Hypothesis testing concepts
- P-values and significance levels
- Type I and Type II errors

Projects built:

- Statistical analysis report
- A/B testing simulator
- Quality control dashboard

Practice: Perform 30 statistical analyses

Excel Data Analysis Tools

Topics:

- PivotTables creation and customization
- PivotTable calculations and summaries
- Slicers and timeline filters
- PivotCharts and interactive dashboards
- Power Pivot introduction
- Data Model and relationships
- DAX formulas basics
- What-if analysis: Goal Seek, Scenario Manager
- Data tables for sensitivity analysis
- Solver for optimization problems
- Analysis ToolPak features
- Regression analysis in Excel

Projects built:

- Sales performance dashboard
- Budget optimization model
- Multi-dimensional analysis cube

Practice: Create 20 PivotTable reports

Data Visualization in Excel

Topics:

- Chart types and when to use them
- Column, bar, and line charts
- Pie charts and donut charts
- Scatter plots and bubble charts
- Combination charts
- Sparklines for inline visualization
- Conditional formatting techniques
- Data bars, color scales, icon sets
- Heat maps and treemaps
- Waterfall and funnel charts
- Chart formatting and design principles
- Creating interactive dashboards

Projects built:

- Executive dashboard
- KPI scorecard
- Interactive sales report

Practice: Design 30 data visualizations

Business Reporting with Excel

Topics:

- Report design principles
- Executive summary creation
- Data storytelling techniques
- Dashboard layout best practices
- Dynamic reports with parameters
- Automated reporting with macros

- VBA basics for automation
- Report distribution and sharing
- Excel to PowerPoint automation
- Version control and collaboration
- Report templates creation
- Documentation best practices

Projects built:

- Monthly business report template
- Automated dashboard system
- C-suite presentation deck

Practice: Create 10 professional reports

Phase 1 Assessment

Topics:

- Excel mastery review
- Statistics concepts check
- Visualization best practices
- Business case presentation
- Technical skills assessment

Projects built:

- CAPSTONE: Complete Business Analysis
- Analyze real company data, create insights, build dashboard, present findings
- Include statistical analysis and recommendations

Assessment: Phase 1 Exam - Excel and Statistics Mastery

Months 3-4: SQL & Python for Data Analysis (Week 9-17)

SQL Fundamentals

Topics:

- Database concepts and relational model
- SQL syntax and structure
- SELECT statements and filtering with WHERE
- Sorting with ORDER BY
- LIMIT and OFFSET for pagination
- DISTINCT for unique values
- Aggregate functions: COUNT, SUM, AVG, MIN, MAX
- GROUP BY for aggregation
- HAVING clause for group filtering
- Working with NULL values
- Date and time functions in SQL
- String manipulation in SQL

Projects built:

- Customer analysis queries
- Sales performance reports
- Inventory tracking system

Practice: Write 100 SQL queries

SQL Joins & Advanced Queries

Topics:

- Understanding table relationships
- INNER JOIN for matching records
- LEFT and RIGHT OUTER JOINS
- FULL OUTER JOIN usage
- CROSS JOIN and Cartesian products
- Self joins for hierarchical data
- Multiple joins in complex queries
- Subqueries and nested queries
- Correlated subqueries
- Common Table Expressions (CTEs)
- UNION, INTERSECT, EXCEPT operations
- Query optimization basics

Projects built:

- Multi-table reporting system
- Customer journey analysis
- Supply chain analytics

Practice: Master 50 complex join scenarios

SQL Analytics Functions

Topics:

- Window functions introduction
- ROW_NUMBER, RANK, DENSE_RANK
- Running totals with SUM() OVER
- Moving averages with AVG() OVER
- LAG and LEAD for time comparisons
- FIRST_VALUE and LAST_VALUE
- Percentile calculations
- Pivoting and unpivoting data
- CASE statements for conditional logic
- Creating views for reusable queries
- Stored procedures basics
- Database performance tuning

Projects built:

- Time series analysis system

- Cohort analysis platform
- RFM segmentation model

Practice: Build 30 analytical queries

Python Fundamentals for Analysis

Topics:

- Python setup for data analysis
- Jupyter Notebook environment
- Variables and data types
- Lists, tuples, dictionaries
- Control flow: if, for, while
- Functions and lambda expressions
- File handling and CSV processing
- Error handling basics
- Libraries: NumPy basics
- Introduction to Pandas
- Reading data from various sources
- Google Colab for cloud computing

Projects built:

- Data processing pipeline
- File converter utility
- Analysis automation script

Practice: Complete 50 Python exercises

Phase 2 Assessment

Topics:

- SQL proficiency test
- Python basics assessment
- Data manipulation skills
- Integration projects
- Technical interview prep

Projects built:

- MAJOR CAPSTONE: Database Analytics Platform
- Build end-to-end analytics solution with SQL and Python
- Include data extraction, transformation, and reporting

Assessment: Phase 2 Exam - SQL and Python Proficiency

PHASE 2: ADVANCED PYTHON & DATA PROCESSING (MONTHS 3-4, WEEKS 9-17)

Master Python for data analysis, pandas, data cleaning, and exploratory data analysis.

Month 3: Pandas & Data Manipulation (Week 9-13)

Pandas Fundamentals

Topics:

- Pandas Series and DataFrames
- Creating DataFrames from various sources
- Reading CSV, Excel, JSON files
- DataFrame indexing and selection
- loc and iloc for data access
- Column operations and manipulation
- Adding and removing columns
- Data type conversions
- Sorting and ranking data
- Handling duplicate data
- Reset and set index
- DataFrame information and summary

Projects built:

- Data loader utility
- DataFrame explorer tool
- Data profiling system

Practice: Complete 60 pandas exercises

Data Cleaning with Pandas

Topics:

- Identifying missing data patterns
- Handling missing values strategies
- fillna, dropna, interpolate methods
- Data type issues and conversions
- String cleaning and standardization
- Regular expressions for data cleaning
- Handling outliers detection and treatment
- Data validation techniques
- Dealing with inconsistent data
- Date and time parsing
- Encoding categorical variables
- Data quality metrics

Projects built:

- Data cleaning pipeline
- Quality assurance tool
- Automated data validator

Practice: Clean 25 messy datasets

Data Transformation & Aggregation

Topics:

- GroupBy operations in pandas
- Aggregation functions: sum, mean, count
- Multiple aggregations with agg()
- Transform and apply functions
- Pivot tables in pandas
- Crosstab for frequency tables
- Melt and wide-to-long reshaping
- Merging and joining DataFrames
- Concatenating data
- Window functions in pandas
- Rolling statistics
- Resampling time series data

Projects built:

- Sales analytics engine
- Customer segmentation tool
- Time series processor

Practice: Perform 40 transformation tasks

Exploratory Data Analysis (EDA)

Topics:

- EDA methodology and workflow
- Univariate analysis techniques
- Bivariate analysis methods
- Multivariate exploration
- Distribution analysis
- Correlation analysis
- Statistical testing in Python
- Feature engineering basics
- Dimensionality reduction intro
- Automated EDA tools
- Creating EDA reports
- EDA best practices

Projects built:

- Automated EDA pipeline
- Data insights generator
- Statistical report builder

Practice: Complete EDA on 20 datasets

NumPy for Numerical Computing

Topics:

- NumPy arrays and operations
- Array broadcasting
- Mathematical operations
- Statistical functions in NumPy
- Linear algebra basics
- Random number generation
- Array manipulation and reshaping
- Performance optimization with NumPy
- Memory efficient operations
- NumPy for image processing basics
- Integration with pandas
- Scientific computing applications

Projects built:

- Numerical computation library
- Statistical calculator
- Matrix operations tool

Practice: Solve 30 numerical problems

Month 4: Data Visualization & Storytelling (Week 14-17)

Matplotlib & Seaborn

Topics:

- Matplotlib basics and figure structure
- Line plots, scatter plots, bar charts
- Customizing plots: colors, labels, legends
- Subplots and figure layouts
- Saving and exporting figures
- Seaborn for statistical visualization
- Distribution plots: histograms, KDE, box plots
- Categorical plots: bar, count, violin
- Relationship plots: scatter, line, regression
- Heatmaps and correlation matrices
- Pair plots for multivariate analysis
- Style and theme customization

Projects built:

- Visualization library
- Statistical plot generator
- Report automation system

Practice: Create 50 visualizations

Interactive Visualizations

Topics:

- Plotly for interactive charts
- Creating interactive dashboards
- Hover information and tooltips
- Zoom and pan functionality
- 3D visualizations
- Animated charts and transitions
- Dash framework basics
- Building web applications with Dash
- Callbacks and interactivity
- Deployment of Dash apps
- Bokeh library introduction
- Altair for declarative visualization

Projects built:

- Interactive dashboard
- Real-time monitoring app
- Data exploration tool

Practice: Build 20 interactive visualizations

Business Intelligence Tools

Topics:

- Introduction to Tableau
- Connecting to data sources in Tableau
- Creating worksheets and dashboards
- Calculated fields and parameters
- Filters and hierarchies
- Tableau story creation
- Power BI fundamentals
- Data modeling in Power BI
- DAX formulas for calculations
- Power BI service and sharing
- Google Data Studio basics
- Choosing the right BI tool

Projects built:

- Tableau executive dashboard
- Power BI sales report
- Cross-platform BI solution

Practice: Create 15 BI dashboards

Data Storytelling & Presentation

Topics:

- Principles of data storytelling
- Narrative structure for data
- Choosing the right visualization
- Color theory and design principles
- Minimalism and clarity
- Annotation and highlighting insights
- Creating compelling presentations
- Executive communication strategies
- Handling questions and objections
- Written reports and documentation
- Video presentations of data
- Stakeholder management

Projects built:

- Data story presentation
- Executive briefing deck
- Insight communication toolkit

Practice: Present 10 data stories

Advanced Analytics Techniques

Topics:

- Time series analysis and forecasting
- Trend and seasonality detection
- Moving averages and smoothing
- Cohort analysis implementation
- Funnel analysis for conversions
- RFM analysis for customer segmentation
- Market basket analysis
- Churn analysis and prediction
- Customer lifetime value calculation
- Attribution modeling basics
- Experimental design and A/B testing
- Statistical significance testing

Projects built:

- Forecasting system
- Customer analytics suite
- A/B testing platform

Practice: Implement 15 analytical techniques

Months 5-6: Machine Learning & Production Analytics (Week 18-26)

Machine Learning Fundamentals

Topics:

- Introduction to machine learning
- Supervised vs unsupervised learning
- Training and test data splitting
- Feature selection and engineering
- Scikit-learn library basics
- Linear regression implementation
- Logistic regression for classification
- Decision trees and random forests
- Model evaluation metrics
- Cross-validation techniques
- Overfitting and underfitting
- Hyperparameter tuning basics

Projects built:

- Sales prediction model
- Customer churn predictor
- Classification system

Practice: Build 20 ML models

Clustering & Segmentation

Topics:

- K-means clustering algorithm
- Hierarchical clustering
- DBSCAN for density-based clustering
- Determining optimal clusters
- Customer segmentation strategies
- Market segmentation analysis
- Behavioral segmentation
- Dimensionality reduction with PCA
- t-SNE for visualization
- Anomaly detection methods
- Outlier detection algorithms
- Practical segmentation applications

Projects built:

- Customer segmentation engine
- Anomaly detection system
- Market analysis tool

Practice: Perform 15 clustering analyses

Web Analytics & Digital Marketing

Topics:

- Google Analytics fundamentals
- Web metrics and KPIs
- User behavior analysis
- Conversion funnel optimization
- Attribution modeling
- Campaign performance analysis
- SEO analytics basics
- Social media analytics
- Email marketing metrics
- Content performance analysis
- Marketing mix modeling
- ROI calculation and optimization

Projects built:

- Digital marketing dashboard
- Campaign optimizer
- Web analytics report

Practice: Analyze 20 marketing campaigns

APIs & Automation

Topics:

- Working with APIs in Python
- RESTful API concepts
- Authentication and API keys
- Requests library for API calls
- JSON data handling
- Web scraping with BeautifulSoup
- Automating data collection
- Scheduling scripts with cron/Task Scheduler
- Email automation for reports
- Cloud functions for automation
- ETL pipeline creation
- Real-time data processing basics

Projects built:

- API data collector
- Automated reporting system
- ETL pipeline

Practice: Build 10 automation scripts

Phase 3 Assessment

Topics:

- Advanced analytics review
- ML implementation check
- Visualization mastery

- Business case presentation
- Portfolio review

Projects built:

- FINAL CAPSTONE: End-to-End Analytics Solution
- Complete business problem solving with data collection, analysis, ML, visualization, and recommendations
- Present to mock stakeholders

Assessment: Phase 3 Exam - Professional Data Analyst Assessment

PHASE 3: INDUSTRY APPLICATIONS & CAREER LAUNCH (MONTHS 5-6, WEEKS 18-26)

Master industry-specific analytics, cloud platforms, and prepare for professional roles.

Month 5: Industry-Specific Analytics (Week 18-22)

Financial Analytics

Topics:

- Financial statements analysis
- Ratio analysis and metrics
- Cash flow analysis
- Working capital management
- Investment analysis metrics
- Risk metrics: VaR, Sharpe ratio
- Portfolio analysis basics
- Credit risk scoring
- Fraud detection techniques
- Time value of money calculations
- Financial forecasting models
- Regulatory reporting basics

Projects built:

- Financial dashboard
- Risk assessment model
- Fraud detection system

Practice: Analyze 15 financial datasets

Healthcare Analytics

Topics:

- Healthcare data types and standards
- Patient data analysis
- Clinical trial analysis
- Healthcare KPIs and metrics
- Disease prediction models
- Patient readmission analysis
- Healthcare cost analysis
- Drug effectiveness studies
- Population health management
- HIPAA compliance basics
- Medical coding systems
- Healthcare dashboards

Projects built:

- Patient analytics dashboard
- Readmission predictor
- Healthcare KPI tracker

Practice: Work with 10 healthcare datasets

Retail & E-commerce Analytics

Topics:

- Retail metrics and KPIs
- Sales performance analysis
- Inventory analytics
- Customer basket analysis
- Product recommendation systems
- Price optimization strategies
- Demand forecasting
- Customer journey mapping
- Conversion rate optimization
- Return analysis
- Store performance comparison
- Omnichannel analytics

Projects built:

- Retail analytics platform
- Recommendation engine
- Inventory optimizer

Practice: Analyze 20 retail scenarios

Supply Chain Analytics

Topics:

- Supply chain metrics and KPIs
- Demand planning and forecasting
- Inventory optimization
- Supplier performance analysis
- Transportation analytics

- Warehouse efficiency metrics
- Order fulfillment analysis
- Supply chain risk assessment
- Network optimization
- Logistics analytics
- Procurement analytics
- Sustainability metrics

Projects built:

- Supply chain dashboard
- Demand forecasting model
- Route optimization tool

Practice: Optimize 10 supply chain processes

HR & People Analytics

Topics:

- HR metrics and KPIs
- Employee turnover analysis
- Recruitment analytics
- Performance management metrics
- Compensation analysis
- Diversity and inclusion metrics
- Employee engagement analysis
- Workforce planning
- Talent analytics
- Learning and development metrics
- Succession planning analytics
- HR dashboard creation

Projects built:

- HR analytics dashboard
- Turnover prediction model
- Compensation analyzer

Practice: Analyze 15 HR scenarios

Month 6: Cloud Analytics & Career Preparation (Week 23-26)

Cloud Platforms for Analytics

Topics:

- Cloud computing fundamentals
- AWS for data analytics
- S3 for data storage
- Athena for SQL queries
- Redshift data warehouse
- Google Cloud Platform basics
- BigQuery for analytics
- Azure data services
- Databricks introduction
- Snowflake basics
- Cloud cost optimization
- Security in cloud analytics

Projects built:

- Cloud migration project
- BigQuery analytics pipeline
- Multi-cloud solution

Practice: Deploy 10 cloud analytics solutions

Big Data Analytics

Topics:

- Big data concepts and challenges
- Hadoop ecosystem overview
- Spark for big data processing
- PySpark basics
- Distributed computing concepts
- NoSQL databases basics
- Real-time analytics introduction
- Kafka for streaming data
- Data lake vs data warehouse
- Lambda architecture
- Big data visualization challenges
- Scalability considerations

Projects built:

- Big data processing pipeline
- Real-time dashboard
- Distributed analytics system

Practice: Process 5 big data scenarios

Portfolio Development

Topics:

- Building an analytics portfolio
- GitHub for data analysts
- Creating compelling case studies
- Documentation best practices
- Jupyter notebook presentation
- Blog writing for analysts

- LinkedIn optimization
- Personal website creation
- Kaggle competitions
- Contributing to open source
- Networking strategies
- Building online presence

Projects built:

- Portfolio website
- 5 detailed case studies
- GitHub repository organization

Practice: Complete portfolio preparation

Interview Preparation

Topics:

- Types of data analyst interviews
- Technical interview preparation
- SQL interview questions
- Python coding interviews
- Case study interviews
- Presentation skills
- Behavioral questions preparation
- STAR method for responses
- Whiteboard problem solving
- Take-home assignments
- Salary negotiation
- Remote interview tips

Projects built:

- Interview preparation kit
- Mock presentation
- Take-home project samples

Practice: Complete 20 mock interviews

Career Launch & Networking

Topics:

- Job search strategies
- Resume optimization for ATS
- Cover letter writing
- LinkedIn job search
- Networking events and conferences
- Industry certifications overview
- Freelancing as an analyst
- Consulting opportunities
- Continuous learning plan
- Career progression paths
- Specialization options
- Future trends in analytics

Projects built:

- Job application package
- Professional network building
- Career roadmap

Assessment: Final Certification - Professional Data Analyst

Advanced Topics & Specializations (Additional Resources)

Advanced Statistical Methods

Topics:

- Bayesian statistics introduction
- Time series advanced techniques
- ARIMA models
- Survival analysis
- Factor analysis
- Structural equation modeling
- Monte Carlo simulations
- Bootstrapping methods
- Non-parametric tests
- Multivariate analysis
- Experimental design
- Causal inference basics

Projects built:

- Advanced statistical toolkit
- Simulation framework
- Causal analysis project

Practice: Implement 10 advanced methods

Deep Learning for Analysts

Topics:

- Neural networks basics
- Deep learning frameworks
- TensorFlow/Keras introduction
- Image data analysis
- Text analytics with deep learning
- Time series with LSTM
- AutoML tools

- Model deployment basics
- Transfer learning
- Deep learning for tabular data
- Ethical AI considerations
- Explainable AI basics

Projects built:

- Deep learning predictor
- Text classification system
- AutoML implementation

Practice: Build 5 deep learning models

Data Engineering Basics

Topics:

- Data pipeline architecture
- ETL vs ELT strategies
- Apache Airflow basics
- Data quality frameworks
- Data governance principles
- Master data management
- Data cataloging
- Metadata management
- Data lineage tracking
- DataOps practices
- CI/CD for data pipelines
- Monitoring and alerting

Projects built:

- ETL pipeline
- Data quality monitor
- Automated data pipeline

Practice: Build 5 data pipelines

Business Strategy & Analytics

Topics:

- Strategic analytics frameworks
- Competitive intelligence
- Market analysis techniques
- Customer analytics strategy
- Product analytics
- Growth analytics
- Pricing analytics
- Risk analytics
- Performance management
- Balanced scorecard
- OKR tracking and analysis
- Executive advisory skills

Projects built:

- Strategy dashboard
- Competitive analysis report
- Growth model

Practice: Develop 5 strategic analyses

Future of Analytics

Topics:

- Augmented analytics
- AI-powered analytics
- Natural language analytics
- Automated insights generation
- Real-time analytics trends
- Edge analytics
- IoT analytics
- Blockchain analytics
- Quantum computing impact
- Privacy-preserving analytics
- Synthetic data generation
- Career future-proofing

Projects built:

- Future tech exploration
- Innovation project
- Trend analysis report

Practice: Explore 5 emerging technologies

CONTINUOUS LEARNING: POST-COURSE RESOURCES

Lifetime learning paths and advanced certifications

Professional Certifications (Certification Paths)

Microsoft Certifications

Topics:

- Microsoft Certified: Data Analyst Associate
- Power BI certification path
- Azure Data Fundamentals

- Azure Data Engineer Associate
- Exam preparation strategies
- Study materials and resources
- Practice tests
- Hands-on labs
- Certification maintenance
- Career impact of certifications
- Cost-benefit analysis
- Employer recognition

Projects built:

- Certification study plan
- Practice project portfolio
- Exam preparation kit

Practice: Complete practice exams

Google & AWS Certifications

Topics:

- Google Data Analytics Certificate
- Google Cloud Data Engineer
- AWS Certified Data Analytics
- AWS Certified Database Specialty
- Preparation timelines
- Hands-on experience requirements
- Lab environments setup
- Community study groups
- Certification roadmaps
- Renewal requirements
- Multi-cloud strategies
- Certification stacking

Projects built:

- Cloud project portfolio
- Multi-cloud comparison
- Certification tracker

Practice: Complete cloud labs

Industry Certifications

Topics:

- Tableau Desktop Specialist
- Tableau Certified Data Analyst
- SAS Certified Specialist
- IBM Data Science Professional
- Databricks Certified Data Analyst
- Alteryx Designer Core
- TDWI certifications
- Domain-specific certifications
- Vendor-neutral certifications
- Certification strategy
- ROI of certifications
- Maintaining multiple certifications

Projects built:

- Tool-specific portfolios
- Certification showcase
- Skills matrix

Practice: Pursue 2-3 certifications

Academic Pathways

Topics:

- Master's in Data Analytics
- MBA with Analytics Focus
- Online degree programs
- MicroMasters programs
- Nanodegree programs
- Bootcamp options
- University partnerships
- Research opportunities
- PhD considerations
- Academic vs industry paths
- Funding options
- Time management strategies

Projects built:

- Education plan
- Application preparation
- Funding research

Practice: Evaluate education options

Specialization Areas

Topics:

- Marketing Analytics Specialist
- Financial Analyst path
- Healthcare Data Analyst
- Sports Analytics
- Customer Analytics Expert
- Operations Research Analyst

- Quantitative Analyst
- Business Intelligence Developer
- Data Scientist transition
- Analytics Consultant
- Product Analyst
- Growth Analyst

Projects built:

- Specialization roadmap
- Skill gap analysis
- Transition plan

Practice: Choose specialization path

Career Development (Professional Growth)

Freelancing & Consulting

Topics:

- Starting as freelance analyst
- Building client base
- Pricing strategies
- Project scoping
- Contract negotiation
- Deliverable management
- Client communication
- Portfolio development
- Marketing yourself
- Networking strategies
- Platform selection
- Scaling your practice

Projects built:

- Freelance business plan
- Service offerings
- Client pipeline

Practice: Start with 3 clients

Leadership & Management

Topics:

- Leading analytics teams
- Project management for analytics
- Stakeholder management
- Analytics strategy development
- Building data culture
- Change management
- Mentoring junior analysts
- Cross-functional collaboration
- Executive communication
- Budget management
- Vendor management
- Team development

Projects built:

- Leadership portfolio
- Team development plan
- Strategy document

Practice: Lead analytics project

Innovation & Research

Topics:

- Staying current with trends
- Reading research papers
- Conference participation
- Publishing articles
- Speaking engagements
- Podcast appearances
- Thought leadership
- Open source contributions
- Research collaborations
- Patent opportunities
- Innovation frameworks
- R&D in analytics

Projects built:

- Research publication
- Conference presentation
- Innovation project

Practice: Contribute to community

Entrepreneurship

Topics:

- Analytics startup ideas
- Market validation
- MVP development
- Funding strategies
- Building analytics products
- SaaS for analytics
- Data monetization

- Analytics as a service
- Partnership strategies
- Scaling challenges
- Exit strategies
- Success stories

Projects built:

- Startup concept
- Business model canvas
- Pitch deck

Practice: Validate business idea

Lifetime Learning

Topics:

- Continuous education plan
- Learning resources curation
- Community engagement
- Mentorship circles
- Knowledge sharing
- Personal brand building
- Network maintenance
- Skill refresh cycles
- Technology adoption
- Career pivots
- Work-life balance
- Long-term success

Projects built:

- 10-year career plan
- Learning roadmap
- Legacy planning

Practice: Implement learning system

Resources & Tools (Reference Materials)

Essential Tools

Topics:

- Analytics tools comparison
- Free vs paid tools
- Open source alternatives
- Cloud platform credits
- Educational licenses
- Tool selection criteria
- Integration strategies
- Migration paths
- Version control
- Collaboration tools
- Documentation tools
- Productivity tools

Projects built:

- Tool evaluation matrix
- Technology stack
- Migration plan

Practice: Master 10 essential tools

Data Sources

Topics:

- Public datasets catalog
- Government data sources
- Academic datasets
- Industry benchmarks
- API directories
- Web scraping sources
- Subscription databases
- Real-time data feeds
- Synthetic data generation
- Data marketplaces
- Open data initiatives
- Data partnerships

Projects built:

- Data source directory
- API collection
- Dataset library

Practice: Build data repository

Communities & Networks

Topics:

- Professional associations
- Online communities
- Local meetup groups
- Slack/Discord channels
- LinkedIn groups
- Reddit communities
- Stack Overflow
- Kaggle community

- GitHub community
- Conference networks
- Alumni associations
- Mentorship programs

Projects built:

- Community participation plan
- Network map
- Engagement strategy

Practice: Join 10 communities

Best Practices

Topics:

- Code standards
- Documentation standards
- Version control best practices
- Security practices
- Privacy compliance
- Ethical guidelines
- Quality assurance
- Performance optimization
- Collaboration workflows
- Project templates
- Reusable components
- Knowledge management

Projects built:

- Best practices guide
- Template library
- Standards documentation

Practice: Implement all standards

Quick References (Cheat Sheets)

Technical References

Topics:

- Excel formulas cheat sheet
- SQL commands reference
- Python pandas quick guide
- Statistics formulas
- Visualization best practices
- Machine learning algorithms
- Cloud services comparison
- Tool shortcuts compilation
- Error message solutions
- Performance tips
- Debugging guide
- Optimization techniques

Projects built:

- Personal reference library
- Quick lookup tools
- Troubleshooting guide

Business References

Topics:

- KPI definitions
- Industry metrics glossary
- Business terms dictionary
- Reporting templates
- Presentation templates
- Communication templates
- Project templates
- Analysis frameworks
- Decision matrices
- ROI calculators
- Budget templates
- Strategy frameworks

Deliverables:

- Business toolkit
- Template library
- Framework collection

Career Resources

Topics:

- Resume templates
- Portfolio examples
- Interview questions bank
- Salary data sources
- Job board list
- Recruiter directory
- Company research tools
- Negotiation strategies
- Career transition guides
- Success metrics
- Performance reviews

- Career milestones
- Deliverables:
- Career toolkit
 - Job search resources
 - Success tracker
- Assessment: LIFETIME SUPPORT - Continuous updates and community access

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Excel Dashboards, Statistical Analysis, Business Reports
- Phase 2: SQL Analytics, Python Automation, Data Cleaning Pipelines
- Phase 3: ML Models, Interactive Dashboards, Industry Solutions
- Final: Complete Analytics Platform with Real Business Impact

Total projects: 25+ professional projects across all tools and industries

SKILLS THE STUDENT WILL HAVE AT THE END

- Tools: Excel (Advanced), SQL, Python, Tableau, Power BI, Google Analytics
- Programming: Python (Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn)
- Statistics: Descriptive, Inferential, Hypothesis Testing, Regression
- Visualization: Charts, Dashboards, Interactive Reports, Storytelling
- Machine Learning: Regression, Classification, Clustering, Time Series
- Business: KPIs, Metrics, Reporting, Presentation, Communication
- Databases: SQL, NoSQL basics, Data Warehousing, ETL
- Cloud: AWS, GCP, Azure basics for analytics
- Soft Skills: Problem-solving, Critical thinking, Communication

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-6 hours
 Hands on practice: 8-10 hours
 Projects: 3-4 hours
 Case studies: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly Q&A and problem-solving
 Mentorship: 1-on-1 guidance from industry analysts
 Community: Active Slack/Discord community
 Project review: Detailed feedback on all projects
 Career support: Resume review, mock interviews, job referrals
 Lifetime access: All content and future updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Final certificate: Professional Data Analyst Certification
 LinkedIn badge: Verifiable LinkedIn credential
 Industry recognized: Recognized by employers
 Portfolio projects: 25+ projects for portfolio

WHERE THIS CAN LEAD

- Data Analyst
- Business Analyst
- Marketing Analyst
- Financial Analyst
- Operations Analyst
- Product Analyst
- Healthcare Data Analyst
- Supply Chain Analyst
- HR Analyst
- Freelance Analytics Consultant
- Business Intelligence Developer
- Analytics Manager

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-5 LPA (Junior Analyst); Rs 5-8 LPA (Data Analyst)

Growing roles: Rs 8-15 LPA (Senior Analyst)

Other roles: Rs 15-25+ LPA (Lead/Manager)

Freelance: Rs 500-2,000/hour

International: \$50k-100k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need coding experience or advanced maths to start?

A: No. The course begins with Excel and statistics fundamentals, then introduces SQL and Python step by step, so complete beginners are fine. School level mathematics is enough; every statistical idea is taught inside the course using real datasets.

Q: Which tools will I actually learn?

A: The working analyst toolkit: Excel for quick analysis and dashboards, SQL for querying databases, Python with pandas for cleaning and analysis, Power BI and Tableau for visualization, and machine learning basics at the end. You practice each tool on realistic business datasets rather than toy examples.

Q: What projects will I have in my portfolio?

A: By the end of 6 months you will have Excel business dashboards, SQL analyses of realistic databases, Python notebooks covering data cleaning and exploration, BI dashboards, and a capstone that takes a raw dataset all the way to an insight presentation. All of it is GitHub-ready for job applications.

Q: Can this course get me a data analyst job or internship?

A: It gives you what employers screen for: SQL fluency, Python analysis skills, a visible portfolio and practice answering the analytics questions asked in interviews. Data analyst roles hire from every degree background, which makes the field friendly to students and career changers. We are honest about this: no placement guarantee, just real preparation.

Q: How is this different from free tutorials and self-paced courses?

A: Self-paced courses cannot tell you why your analysis is wrong. Here a live instructor reviews your queries and notebooks, points out flawed logic in how you read the data, and pushes you to explain findings out loud, which is the actual job of an analyst. Most self-learners quit at the SQL to Python jump; a mentor gets you across it.

Q: Can I attend a class before deciding?

A: Yes, we start everyone with a free live demo class, booked at learn.modernagecoders.com/contact. There is no commitment and no payment information needed; you simply experience a real session and then decide.

Q: Are classes live or recorded?

A: Live, always: each class is a 1-hour online session with an instructor, never a pre-recorded video. Classes are interactive with hands-on work on datasets, and evening and weekend slots make the course manageable alongside college or a full-time job.

Q: What do I receive on completion?

A: A completion certificate plus something more useful in interviews: a GitHub-ready portfolio of analysis projects and dashboards you can walk a hiring manager through. We are rated 4.9 stars across 547+ Google reviews if you want an outside opinion of the teaching.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/data-analysis-mastery-course-college>

=====

COURSE 45 OF 118: DATA AND AI ANALYTICS FOR NON-PROGRAMMERS (EXCEL, SHEETS, POWER BI)

=====

Course link: <https://learn.modernagecoders.com/courses/data-and-ai-analytics-for-non-programmers-course>

Subject area: Data and AI Analytics

Designed for: Working professionals

Who can join: Working professionals and students with no programming background; spreadsheet familiarity helps but is not required

Course length: 4 months (16 weeks), joinable any month

Time needed: 2 live classes/week + weekly work on real datasets

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online data analytics for working professionals who do not code: clean and model data in Excel and Sheets, build Power BI dashboards, and use AI assistants to work faster without being misled.

In one line: Become the person in the room who can actually answer the question with data, without writing a line of code.

ABOUT THE PROGRAMME

Most people who need to work with data are not going to become programmers, and they do not need to. The person who can take a messy export, clean it, find the real pattern and present it clearly in a dashboard is enormously valuable in any team, and almost none of that requires code. What it requires is knowing how to structure data, which comparison is honest, and how to build something a decision-maker can actually read. This course teaches exactly that path, in the tools your workplace already has. You start with spreadsheets done properly, Excel and Google Sheets, including the lookup and pivot skills most people never learn. You move into cleaning and shaping real messy data, which is where most analysis actually fails. Then you build interactive dashboards in Power BI and learn to model relationships between tables so the numbers hold up under questioning. AI runs through the whole course, honestly. Modern AI assistants can write your formulas, summarise a dataset and suggest a chart, and used well they make you considerably faster. Used badly they produce confident nonsense that you then present to your manager. So we teach both: how to use AI to accelerate the work, and how to check what it gives you before you trust it. Every module ends with real, messy datasets rather than tidy teaching examples, because tidy data is not what anybody actually receives.

WHAT MAKES THIS COURSE DIFFERENT

- No coding required at any point, and no pretending you will secretly need Python later
- Built on the tools your workplace already has: Excel, Google Sheets and Power BI
- Real messy data from week one, because cleaning is where most analysis actually fails
- AI assistants taught honestly: how to go faster with them, and how to catch the confident nonsense before you present it
- Focused on the question behind the request, since the hard part is usually deciding what to measure
- Live, small batches where your dashboard gets critiqued like a colleague would critique it

THE LEARNING PATH, STEP BY STEP

Phase 1: Spreadsheets done properly: structure, lookups, pivots and the formulas that matter
Phase 2: Cleaning and shaping messy real data, then statistics that keep you honest
Phase 3: Power BI dashboards, data modelling, AI-assisted analysis and a portfolio project

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What You Need to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: SPREADSHEETS DONE PROPERLY -----

The foundation almost everyone half-knows, rebuilt so the later work holds up.

Month 1: Excel and Google Sheets Foundations (Weeks 1-4)

Structuring Data So It Can Be Analysed

Topics:

- What a clean table actually looks like, and why yours probably is not one
- Rows as records, columns as fields, and one fact per cell
- Why merged cells and colour-as-data destroy analysis
- Data types, dates and the formatting traps that silently break totals
- Turning a human-formatted sheet into an analysable one

Projects built:

- Take a genuinely badly structured sheet and restructure it into a usable table

Practice: Restructure three messy real-world sheets

The Formulas That Actually Matter

Topics:

- Absolute and relative references, and why formulas break when copied
- IF, nested conditions, and when to stop nesting
- SUMIF, COUNTIF, AVERAGEIF and their multi-criteria versions
- Text and date functions for real cleanup
- Error handling so a broken cell does not poison a whole column

Projects built:

- Build a summary sheet driven entirely by formulas from a raw data tab

Practice: 20 formula tasks against a real dataset

Lookups and Joining Data

Topics:

- VLOOKUP, XLOOKUP and INDEX with MATCH, and when each is right
- Joining two tables on a common key
- Handling missing matches honestly instead of hiding them
- Duplicate keys and the quiet damage they do
- The equivalents in Google Sheets

Projects built:

- Combine three separate exports into one analysable table and reconcile the mismatches

Practice: 15 lookup and join exercises including deliberately broken keys

Assessment: A timed spreadsheet task: clean, join and summarise a real dataset

Pivot Tables and Fast Summaries

Topics:

- Pivot tables from the ground up
- Grouping, filtering and calculated fields
- Slicing by dimension to find where a number actually comes from
- Reading a pivot critically instead of accepting it
- Charts that clarify rather than decorate

Projects built:

- Answer five real business questions about one dataset using only pivots

Practice: Build pivots answering supplied questions, then explain each finding

PHASE 2: CLEANING REAL DATA AND ANALYSING IT HONESTLY

Where most analysis fails, and the statistics you need to avoid fooling yourself.

Month 2: Data Cleaning and Practical Statistics (Weeks 5-8)

Cleaning Messy Real Data

Topics:

- Duplicates, near-duplicates and how to decide which is real
- Missing values and the options: exclude, impute or report
- Inconsistent categories and text normalisation
- Outliers, and telling a data error from a genuine extreme
- Documenting every cleaning decision so your numbers are defensible

Projects built:

- Clean a genuinely messy dataset and write a short log of every decision made

Practice: Clean three datasets with different problems

Power Query for Repeatable Cleaning

Topics:

- Why manual cleaning does not survive next month's file
- Power Query basics: importing, transforming and loading
- Building a repeatable cleaning pipeline
- Unpivoting and reshaping data
- Refreshing when new data arrives, with no rework

Projects built:

- Build a Power Query pipeline that cleans a monthly export automatically

Practice: Convert last week's manual cleaning into a repeatable query

Statistics That Keep You Honest

Topics:

- Mean, median and when the average actively misleads
- Spread and why two datasets with the same average differ completely
- Percentages, percentage points and the errors that get presented in meetings
- Correlation is not causation, with real examples
- Sample size and why small numbers should not drive big decisions

Projects built:

- Take a real claim from a report and check whether the data actually supports it

Practice: Critique supplied analyses and identify what each gets wrong

Assessment: A written critique of a deliberately misleading analysis

Charts That Do Not Lie

Topics:

- Choosing the chart from the question, not from preference
- Truncated axes and the other ways charts mislead
- Comparing like with like
- Labelling so a chart survives being forwarded without you
- Removing everything that is not carrying information

Projects built:

- Redesign three bad charts and explain what each change fixes

Practice: Build charts for supplied questions and defend each choice

PHASE 3: POWER BI, AI-ASSISTED ANALYSIS AND YOUR PORTFOLIO

Turning analysis into something a decision-maker uses, with AI as an accelerator rather than an oracle.

Month 3: Power BI and Data Modelling (Weeks 9-12)

Power BI Foundations

Topics:

- Connecting to data and shaping it on import
- The difference between a report and a dashboard
- Core visuals and when each is appropriate
- Filters, slicers and cross-filtering
- Publishing and sharing safely

Projects built:

- Build your first working Power BI report from a real dataset

Practice: Rebuild an existing spreadsheet report as a Power BI report

Data Modelling and Relationships

Topics:

- Fact and dimension tables in plain language
- Relationships and why a star schema keeps numbers correct
- The problems caused by one flat table
- Date tables and time intelligence
- Diagnosing a wrong total caused by a bad relationship

Projects built:

- Model a multi-table dataset properly and prove the totals reconcile

Practice: Fix three broken data models

DAX Measures Without Fear

Topics:

- Calculated columns versus measures, and why the difference matters
- Core DAX: SUM, AVERAGE, COUNTROWS, CALCULATE
- Filter context in plain terms
- Time comparisons: month on month, year on year
- Writing measures you can still understand next quarter

Projects built:

- Add a full measure set to your model including period comparisons

Practice: 15 DAX exercises building from simple to filtered

Assessment: A timed dashboard task assessed on correctness and clarity

Designing Dashboards People Actually Use

Topics:

- Starting from the decision the dashboard should support
- Layout, hierarchy and what belongs above the fold
- Restraint: fewer visuals, better chosen
- Making it self-explanatory without you presenting it
- Testing it on someone who has never seen it

Projects built:

- Redesign your dashboard after watching someone else try to use it

Practice: Peer usability reviews of each other's dashboards

Month 4: AI-Assisted Analysis and the Capstone (Weeks 13-16)

Using AI Assistants Well

Topics:

- What AI assistants are genuinely good at in analytics work
- Getting formulas, DAX and cleaning steps written for you
- Asking for an explanation of a formula you inherited
- Writing a prompt that includes the context the model needs
- Speed gains that are real, and the ones that are illusory

Projects built:

- Solve a full analysis task using AI assistance and log where it helped and where it did not

Practice: Complete supplied tasks with AI, then verify every output

Checking What AI Gives You

Topics:

- Why an AI can produce a confident, plausible, wrong formula
- Verifying a result against a known subtotal before trusting it
- Spot-checking summaries against the raw data
- Never pasting confidential data into a tool that is not approved
- Being able to explain any number you present, whatever produced it

Projects built:

- Audit a set of AI-produced analyses and find the errors deliberately planted in them

Practice: Verification drills on AI output

Assessment: A verification test: find every error in an AI-assisted analysis

Capstone Build

Topics:

- Choosing a question that matters to a real audience
- Sourcing and cleaning the data end to end
- Modelling, measures and dashboard build
- Documenting assumptions and limitations honestly
- Preparing to defend every number

Projects built:

- Build your full capstone: question, clean data, model, dashboard, findings

Practice: Capstone checkpoints with review

Presenting and Your Portfolio

Topics:

- Leading with the finding, not the method
- Telling the story a decision-maker needs in two minutes
- Answering the hard question about your data honestly
- Packaging the project for a portfolio or interview
- What to say when the data does not support the hoped-for answer

Projects built:

- Present your capstone and take questions from the group

Practice: Rehearsed presentations with critique

Assessment: Final capstone presentation assessed on clarity, correctness and honesty

- Business analyst, operations analyst or MIS roles
- Business intelligence and reporting work
- A stronger position in your current job as the person who can answer with data
- A portfolio project that demonstrates the whole workflow end to end
- A natural later step into Python or SQL if you ever want it

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I really not need to learn programming?

A: Correct. Everything in this course is done in Excel, Google Sheets and Power BI, all of which are built for exactly this work. Power Query and DAX are used, and although DAX is a formula language it is far closer to spreadsheet formulas than to programming. If you later want Python or SQL, this course leaves you well placed, but nothing here depends on it.

Q: How is AI actually used in the course?

A: Honestly, and in both directions. AI assistants are genuinely good at writing formulas, drafting DAX, explaining an inherited spreadsheet and suggesting cleaning steps, and used well they make you noticeably faster. They are also very capable of producing a confident wrong answer. So we teach you to use them and to verify everything they produce against known subtotals before you present it.

Q: Will I have something to show at the end?

A: Yes. The final month is a capstone: you choose a real question, source and clean the data, model it, build the dashboard and present the findings, including your assumptions and limitations. It is designed to be the project you show in an interview, because it demonstrates the entire workflow rather than one tool.

Q: I have tried a data course before and gave up. Why would this be different?

A: Most data courses lose people because they open with programming or with tidy academic datasets that behave nothing like real ones. This one starts in the tools you already have and works with genuinely messy data from the first weeks, because cleaning is the part where real analysis actually fails. It is live and small, so you get unstuck in class rather than quietly falling behind.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/data-and-ai-analytics-for-non-programmers-course>

=====

COURSE 46 OF 118: DATA SCIENCE COURSE: ZERO TO JOB-READY DATA SCIENTIST

=====

Course link: <https://learn.modernagecoders.com/courses/data-science-complete-masterclass-college>

Subject area: Data Science & Machine Learning

Designed for: College students

Who can join: Complete Beginner to Professional Data Scientist

Course length: 12 months (52 weeks)

Time needed: 20-25 hours/week recommended

Certificate: Certified Data Scientist upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online data science course: master Python, machine learning, deep learning, statistics and big data, then build and deploy real, job-ready ML models.

In one line: From Raw Data to Production ML Models

ABOUT THE PROGRAMME

This is not just a course, it's a complete transformation into a professional data scientist. In the age of AI and big data, data scientists are the most sought-after professionals. This 12-month masterclass takes you from absolute beginner to a job-ready data scientist, capable of extracting insights from data, building predictive models, deploying ML solutions, and solving complex business problems. You'll master the complete data science pipeline: data collection, cleaning, exploration, feature engineering, model building, evaluation, and deployment.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no prerequisites required
- Complete 12-month structured curriculum aligned with industry needs
- Covers entire data science stack: Python, ML, DL, Statistics, Big Data

- 50+ real-world projects and 5 Kaggle competitions
- Focus on both theory and practical implementation
- MLOps and production deployment included
- Interview preparation for FAANG and top companies
- Lifetime access with continuous updates
- Build a portfolio showcasing end-to-end ML projects

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Python, Statistics, Data Analysis, SQL
 Phase 2: Machine Learning (Months 4-6): Algorithms, Feature Engineering, Model Evaluation
 Phase 3: Deep Learning (Months 7-9): Neural Networks, Computer Vision, NLP
 Phase 4: Production & Specialization (Months 10-12): MLOps, Big Data, Career Launch

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - high school math helpful
 Coding experience: None required - we start from zero
 Equipment: Computer with 8GB RAM minimum, internet connection
 Time commitment: 20-25 hours per week consistently
 English: Good reading and comprehension skills
 Motivation: Strong curiosity about data and problem-solving mindset

WHO THIS COURSE IS FOR

Students: College students wanting high-paying tech careers
 Working professionals: Career switchers from any field to data science
 Entrepreneurs: Build data-driven products and businesses
 Freelancers: Offer data science and ML consulting services
 Kids: Not suitable for children - recommended age 16+
 Anyone: Anyone fascinated by AI and data-driven decision making

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATIONS OF DATA SCIENCE (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid foundations in Python programming, statistics, data analysis, and SQL that form the backbone of data science.

Months 1-2: Python & Statistical Foundations (Week 1-8)

Python Programming for Data Science

Topics:

- Why Python for Data Science?
- Setting up the data science environment (Anaconda, Jupyter)
- Python basics: variables, data types, operators
- Data structures: lists, tuples, sets, dictionaries
- Control flow: if-else, loops (for, while)
- Functions and lambda expressions
- Object-oriented programming basics
- File handling and exception management
- Python libraries ecosystem for data science
- Git and version control for data science projects
- Jupyter notebooks best practices
- Google Colab for cloud computing

Projects built:

- Python fundamentals practice notebook
- Building a data science toolkit with functions
- Automated data processing script

Practice: Complete 100 Python coding challenges

NumPy for Numerical Computing

Topics:

- Introduction to NumPy and its importance
- NumPy arrays: creation and properties
- Array indexing, slicing, and reshaping
- Broadcasting and vectorization
- Mathematical operations on arrays
- Statistical functions in NumPy
- Linear algebra operations
- Random number generation
- Array manipulation techniques
- Performance optimization with NumPy
- Memory management in NumPy
- NumPy for image processing basics

Projects built:

- Image manipulation with NumPy arrays
- Statistical calculator using NumPy

- Matrix operations library
- Performance comparison: NumPy vs pure Python

Practice: Solve 50 NumPy exercises

Data Manipulation with Pandas

Topics:

- Introduction to Pandas DataFrames and Series
- Reading data from various formats (CSV, Excel, JSON, SQL)
- Data inspection and understanding
- Indexing and selecting data (.loc, .iloc)
- Filtering and boolean indexing
- Handling missing data strategies
- Data transformation and feature creation
- Groupby operations and aggregations
- Merging, joining, and concatenating data
- Pivot tables and cross-tabulation
- Time series data handling
- Performance optimization in Pandas

Projects built:

- Complete EDA on retail sales dataset
- Data cleaning pipeline for messy dataset
- Time series analysis of stock prices
- Customer behavior analysis

Practice: Clean and analyze 10 different datasets

Statistical Foundations for Data Science

Topics:

- Descriptive statistics: measures of central tendency and spread
- Probability theory and distributions
- Normal distribution and Central Limit Theorem
- Sampling and sampling distributions
- Hypothesis testing fundamentals
- T-tests, Chi-square tests, ANOVA
- P-values and statistical significance
- Confidence intervals
- Correlation and covariance
- Type I and Type II errors
- Power analysis and sample size determination
- Bayesian statistics introduction

Projects built:

- Statistical analysis of A/B test results
- Hypothesis testing on real datasets
- Building a statistical testing framework
- Correlation analysis of multiple variables

Practice: Perform statistical tests on 15 different scenarios

Month 3: Data Visualization & SQL (Week 9-13)

Data Visualization Mastery

Topics:

- Principles of effective data visualization
- Matplotlib: creating publication-quality plots
- Seaborn for statistical visualizations
- Plotly for interactive visualizations
- Creating dashboards with Streamlit/Dash
- Geospatial visualization with Folium
- Time series visualization techniques
- Heatmaps and correlation matrices
- Network graphs and tree visualizations
- 3D visualizations
- Animation in data visualization
- Best practices for different chart types

Projects built:

- Interactive COVID-19 dashboard
- Sales performance dashboard
- Geospatial analysis of crime data
- Animated visualization of algorithm performance

Practice: Create 30 different types of visualizations

SQL for Data Scientists

Topics:

- Relational database concepts
- SQL fundamentals: SELECT, WHERE, ORDER BY
- Aggregate functions and GROUP BY
- Joins: INNER, LEFT, RIGHT, FULL OUTER
- Subqueries and CTEs
- Window functions for advanced analytics
- Database design and normalization
- Performance optimization and indexing
- NoSQL databases introduction (MongoDB)
- Connecting Python to databases
- SQL vs Pandas: when to use what
- Big data SQL: Spark SQL, Presto

Projects built:

- Building a data warehouse schema
- Complex business queries with window functions
- Python-SQL integration project
- Database performance optimization

Practice: Write 100 SQL queries of increasing complexity

Exploratory Data Analysis (EDA) & Feature Engineering

Topics:

- EDA methodology and workflow
- Univariate, bivariate, and multivariate analysis
- Identifying patterns and anomalies
- Feature engineering techniques
- Creating polynomial and interaction features
- Binning and discretization
- Encoding categorical variables
- Feature scaling and normalization
- Handling imbalanced datasets
- Feature selection methods
- Dimensionality reduction (PCA introduction)
- Domain-specific feature engineering

Projects built:

- PHASE 1 CAPSTONE: End-to-End EDA Project
- Complete EDA and feature engineering on Kaggle dataset
- Build reusable feature engineering pipeline
- Create automated EDA report generator

Assessment: Phase 1 comprehensive assessment

PHASE 2: MACHINE LEARNING MASTERY (MONTHS 4-6, WEEKS 14-26)

Master classical machine learning algorithms, model evaluation, and participate in Kaggle competitions.

Months 4-5: Supervised Learning (Week 14-22)

Regression Algorithms

Topics:

- Machine learning workflow and pipeline
- Linear regression from scratch
- Multiple linear regression
- Polynomial regression
- Ridge and Lasso regression
- Elastic Net
- Logistic regression for classification
- Evaluation metrics for regression
- Cross-validation strategies
- Bias-variance tradeoff
- Regularization techniques
- Feature importance analysis

Projects built:

- House price prediction model
- Sales forecasting system
- Customer lifetime value prediction
- Build regression library from scratch

Practice: Implement 5 regression algorithms from scratch

Classification Algorithms

Topics:

- k-Nearest Neighbors (KNN)
- Decision trees and pruning
- Random Forests
- Gradient Boosting (XGBoost, LightGBM, CatBoost)
- Support Vector Machines (SVM)
- Naive Bayes classifier
- Evaluation metrics for classification
- ROC curves and AUC
- Handling imbalanced classes
- Multi-class classification strategies
- Ensemble methods
- Stacking and blending

Projects built:

- Credit default prediction
- Customer churn prediction
- Disease diagnosis system
- Fraud detection model

Practice: Build 10 classification models on different datasets

Advanced Machine Learning Techniques

Topics:

- Hyperparameter tuning: Grid Search, Random Search, Bayesian Optimization
- AutoML tools and techniques
- Feature selection algorithms

- Dimensionality reduction: PCA, t-SNE, UMAP
- Anomaly detection algorithms
- Time series forecasting: ARIMA, Prophet
- Recommendation systems: collaborative filtering, content-based
- Model interpretation: SHAP, LIME
- Handling missing data: advanced imputation
- Semi-supervised learning
- Active learning strategies
- Transfer learning in classical ML

Projects built:

- Anomaly detection in network traffic
- Movie recommendation system
- Sales forecasting with Prophet
- Model interpretation dashboard

Practice: Apply advanced techniques to improve previous models

Unsupervised Learning

Topics:

- Clustering algorithms: K-Means, DBSCAN, Hierarchical
- Gaussian Mixture Models
- Clustering evaluation metrics
- Market basket analysis
- Association rules: Apriori, FP-Growth
- Topic modeling: LDA, NMF
- Autoencoders for dimensionality reduction
- Self-organizing maps
- Isolation Forest for anomaly detection
- Applications in customer segmentation
- Image compression with clustering
- Text clustering and document similarity

Projects built:

- Customer segmentation analysis
- Market basket analysis for retail
- Document clustering system
- Anomaly detection in IoT data

Practice: Apply clustering to 5 different domains

Kaggle Competition Participation

Topics:

- Understanding Kaggle competitions
- Competition strategies and workflow
- Data augmentation techniques
- Advanced feature engineering
- Ensemble strategies for competitions
- Learning from kernels and discussions
- Leaderboard probing
- Cross-validation strategies
- Submission strategies
- Post-competition analysis

Projects built:

- Participate in current Kaggle competition
- Achieve top 50% in beginner competition
- Write competition solution walkthrough
- Build reusable competition pipeline

Practice: Complete 3 past Kaggle competitions

Month 6: Real-world ML Projects (Week 23-26)

End-to-End ML Project Development

Topics:

- Problem formulation and scoping
- Data collection strategies
- Building data pipelines
- Feature store design
- Model versioning and experiment tracking
- A/B testing for ML models
- Model monitoring and drift detection
- Building ML APIs with Flask/FastAPI
- Containerization with Docker
- Cloud deployment (AWS, GCP, Azure)
- Cost optimization for ML
- Documentation and reporting

Projects built:

- Build complete ML pipeline from scratch
- Deploy model as REST API
- Create model monitoring dashboard
- Implement A/B testing framework

Practice: Deploy 3 models to production

Industry-Specific Applications

Topics:

- Healthcare: disease prediction, drug discovery
- Finance: risk assessment, algorithmic trading

- Retail: demand forecasting, pricing optimization
- Marketing: customer segmentation, campaign optimization
- Manufacturing: predictive maintenance, quality control
- Transportation: route optimization, demand prediction
- Energy: consumption forecasting, grid optimization
- Agriculture: crop yield prediction, pest detection
- Real estate: price prediction, investment analysis
- Sports analytics: player performance, game prediction

Projects built:

- Choose 2 industries and build domain-specific models
- Create industry-specific feature engineering
- Build domain knowledge documentation

Practice: Analyze datasets from 5 different industries

ML Engineering Best Practices

Topics:

- Code organization for ML projects
- Testing ML code: unit tests, integration tests
- Continuous Integration/Continuous Deployment (CI/CD)
- MLflow for experiment tracking
- DVC for data versioning
- Weights & Biases for experiment management
- Model registry and governance
- Feature stores: Feast, Tecton
- Reproducibility in ML
- Debugging ML models
- Performance optimization
- Security in ML systems

Projects built:

- Set up MLOps pipeline
- Implement CI/CD for ML project
- Build feature store
- Create model governance framework

Practice: Refactor all projects with engineering best practices

Advanced Kaggle Competition

Topics:

- Advanced competition strategies
- Reading research papers for techniques
- Implementing papers for competitions
- Advanced ensembling: stacking, blending
- Pseudo-labeling techniques
- Data augmentation for tabular data
- Feature engineering automation
- Hyperparameter optimization at scale
- GPU acceleration for tree models
- Competition code organization

Projects built:

- Participate in intermediate Kaggle competition
- Achieve top 30% ranking
- Open-source competition solution
- Write detailed solution approach

Practice: Review winning solutions from 10 competitions

Phase 2 Capstone Project

Topics:

- Complex problem solving
- Multi-model systems
- Production-ready code
- Professional documentation
- Stakeholder presentation

Projects built:

- PHASE 2 CAPSTONE: Production ML System
- Build end-to-end ML system for real business problem
- Include data pipeline, multiple models, API, monitoring
- Deploy to cloud with full MLOps pipeline

Assessment: Phase 2 comprehensive assessment

----- PHASE 3: DEEP LEARNING & ADVANCED AI (MONTHS 7-9, WEEKS 27-39) -----

Master deep learning, computer vision, NLP, and cutting-edge AI techniques.

Months 7-8: Deep Learning Fundamentals (Week 27-35)

Neural Network Fundamentals

Topics:

- Introduction to deep learning
- Perceptron and multi-layer perceptrons
- Backpropagation algorithm
- Activation functions
- Weight initialization strategies
- Gradient descent variations

- Learning rate scheduling
- Batch normalization
- Dropout and regularization
- Building neural networks from scratch
- Introduction to TensorFlow and Keras
- PyTorch fundamentals

Projects built:

- Neural network from scratch in NumPy
- MNIST digit classification
- Binary classification with deep learning
- Regression with neural networks

Practice: Implement 5 different neural network architectures

Convolutional Neural Networks (CNNs)

Topics:

- CNN architecture and intuition
- Convolution and pooling layers
- Popular architectures: LeNet, AlexNet, VGG, ResNet
- Transfer learning and fine-tuning
- Data augmentation for images
- Object detection: YOLO, R-CNN
- Image segmentation: U-Net
- Face recognition systems
- Style transfer
- Generative models: VAE, GAN basics
- CNN applications beyond images
- Deploying CNN models

Projects built:

- Image classification on custom dataset
- Object detection system
- Face recognition application
- Image segmentation for medical images

Practice: Build 5 computer vision applications

Recurrent Neural Networks (RNNs)

Topics:

- RNN architecture and applications
- Vanishing gradient problem
- LSTM and GRU architectures
- Bidirectional RNNs
- Sequence-to-sequence models
- Attention mechanism
- Time series prediction with RNNs
- Text generation
- Sentiment analysis
- Named Entity Recognition
- Machine translation basics
- Speech recognition introduction

Projects built:

- Stock price prediction with LSTM
- Text generation model
- Sentiment analysis system
- Time series anomaly detection

Practice: Implement 5 sequence modeling tasks

Natural Language Processing

Topics:

- Text preprocessing and tokenization
- Word embeddings: Word2Vec, GloVe
- Text classification techniques
- Named Entity Recognition (NER)
- Part-of-speech tagging
- Topic modeling with deep learning
- Transformer architecture
- BERT and GPT models
- Fine-tuning pre-trained models
- Question answering systems
- Text summarization
- Chatbot development

Projects built:

- Build custom chatbot
- News article classifier
- Question answering system
- Text summarization tool

Practice: Complete 5 NLP projects using transformers

Advanced Deep Learning Topics

Topics:

- Autoencoders and variational autoencoders
- Generative Adversarial Networks (GANs)
- Deep Reinforcement Learning basics
- Graph Neural Networks introduction
- Meta-learning and few-shot learning

- Neural Architecture Search
- Model compression and quantization
- Edge deployment of deep learning
- Adversarial machine learning
- Explainable AI for deep learning
- Multi-modal learning
- Self-supervised learning

Projects built:

- Build and train a GAN
- Implement autoencoder for anomaly detection
- Model compression project
- Multi-modal classification system

Practice: Experiment with 3 cutting-edge techniques

Month 9: Specialized Domains & Research (Week 36-39)

Computer Vision Projects

Topics:

- Medical image analysis
- Autonomous driving perception
- Facial emotion recognition
- Pose estimation
- Video analysis and action recognition
- 3D computer vision
- Document analysis and OCR
- Satellite image analysis
- Real-time vision systems
- Mobile and edge deployment
- Vision transformers
- Self-supervised learning in vision

Projects built:

- Medical diagnosis from X-rays
- Real-time object tracking system
- Document scanner with OCR
- Pose estimation application

Practice: Build portfolio of 5 vision projects

Advanced NLP Applications

Topics:

- Large Language Models (LLMs)
- Prompt engineering
- Fine-tuning LLMs
- Retrieval Augmented Generation (RAG)
- Building with LangChain
- Vector databases
- Semantic search
- Document intelligence
- Code generation with AI
- Multimodal models
- Ethical considerations in NLP
- Production NLP systems

Projects built:

- RAG-based question answering system
- Semantic search engine
- AI writing assistant
- Code generation tool

Practice: Build 5 LLM-powered applications

Reinforcement Learning

Topics:

- RL fundamentals and Markov Decision Processes
- Q-Learning and Deep Q-Networks
- Policy gradient methods
- Actor-Critic methods
- Proximal Policy Optimization (PPO)
- Game playing agents
- Robotics applications
- RL for recommendation systems
- RL in finance
- Multi-agent RL
- Sim-to-real transfer
- OpenAI Gym environments

Projects built:

- Game-playing AI agent
- Trading bot with RL
- Recommendation system with RL
- Robot control simulation

Practice: Train agents in 5 different environments

Research Paper Implementation

Topics:

- Reading and understanding research papers
- Reproducing paper results

- Implementing novel architectures
- Benchmarking and evaluation
- Writing technical reports
- Contributing to open source
- Publishing your own research
- Staying updated with latest research
- Research tools and resources
- Building research portfolio
- Collaboration in research
- Ethics in AI research

Projects built:

- Implement 2 recent research papers
- Write detailed implementation report
- Open source your implementations
- Create tutorial for community

Practice: Read and summarize 20 research papers

Phase 3 Capstone Project

Topics:

- Complex deep learning systems
- Multi-model architectures
- Production deployment
- Performance optimization
- Comprehensive evaluation

Projects built:

- PHASE 3 CAPSTONE: State-of-the-art AI System
- Build cutting-edge AI application
- Combine multiple deep learning techniques
- Deploy with full production pipeline
- Achieve competitive performance metrics

Assessment: Phase 3 comprehensive assessment

PHASE 4: MLOPS, BIG DATA & CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52)

Master production deployment, big data technologies, and prepare for data science careers.

Months 10-11: MLOps & Production Systems (Week 40-48)

MLOps Fundamentals

Topics:

- MLOps principles and practices
- Model lifecycle management
- Continuous training pipelines
- Model versioning strategies
- A/B testing for ML
- Model monitoring and observability
- Drift detection and handling
- Feature stores at scale
- Model serving architectures
- Kubernetes for ML
- KubeFlow and MLflow
- Cost optimization in production

Projects built:

- Build complete MLOps pipeline
- Implement model monitoring system
- Set up continuous training
- Deploy models on Kubernetes

Practice: Deploy 5 models with full MLOps

Big Data Technologies

Topics:

- Big data ecosystem overview
- Apache Spark fundamentals
- PySpark for data science
- Spark MLlib for machine learning
- Distributed computing concepts
- Hadoop and HDFS
- Apache Kafka for streaming
- Real-time ML with streaming data
- Data lakes and data warehouses
- Apache Airflow for orchestration
- Databricks platform
- Cloud big data services

Projects built:

- Build Spark ML pipeline
- Real-time prediction system
- Data lake architecture design
- Streaming analytics dashboard

Practice: Process 5 big data datasets

Cloud Platforms for Data Science

Topics:

- AWS for data science: SageMaker, EMR, Glue
- Google Cloud Platform: Vertex AI, BigQuery, Dataflow
- Azure ML and Azure Databricks
- Serverless ML deployments
- Auto-scaling ML services
- Cloud cost optimization
- Multi-cloud strategies
- Edge computing for ML
- Hybrid cloud architectures
- Security and compliance
- Infrastructure as Code
- Disaster recovery planning

Projects built:

- Deploy models on 3 cloud platforms
- Build serverless ML pipeline
- Implement auto-scaling solution
- Create disaster recovery plan

Practice: Master one cloud platform deeply

Advanced Data Engineering

Topics:

- Data architecture patterns
- ETL vs ELT pipelines
- Data quality and validation
- Schema evolution and management
- CDC (Change Data Capture)
- Data mesh architecture
- Event-driven architectures
- Apache Beam for unified processing
- dbt for data transformation
- Data observability tools
- DataOps practices
- Building data products

Projects built:

- Design data architecture for startup
- Build ETL pipeline with Airflow
- Implement data quality framework
- Create data product

Practice: Build 5 different data pipelines

Business & Soft Skills

Topics:

- Communicating with stakeholders
- Translating business problems to ML
- Project management for data science
- Agile and Scrum for DS teams
- Technical documentation
- Presenting to executives
- Building data culture
- Ethics in data science
- Privacy and GDPR compliance
- Team collaboration tools
- Mentoring and leadership
- Consulting skills

Projects built:

- Create executive presentation
- Write technical documentation
- Build project proposal
- Develop data strategy document

Practice: Present 5 projects to different audiences

Month 12: Specialization & Career Launch (Week 49-52)

Choose Your Specialization

Topics:

- Healthcare AI: medical imaging, drug discovery, genomics
- Financial ML: risk modeling, fraud detection, trading
- Retail analytics: recommendation, pricing, inventory
- Computer Vision specialist: autonomous vehicles, robotics
- NLP specialist: conversational AI, document intelligence
- MLOps engineer: platform building, automation
- Research scientist: cutting-edge algorithms
- Choosing your path based on interests
- Building specialized portfolio
- Domain expertise development
- Networking in your chosen field
- Continuous learning strategies

Projects built:

- Deep specialization project in chosen domain
- Build 3 projects in specialization area
- Create specialized portfolio website

Practice: Complete specialized certification or course

Portfolio & Personal Branding

Topics:

- Building impressive GitHub profile
- Creating portfolio website
- Writing technical blog posts
- Contributing to open source
- Building Kaggle profile
- LinkedIn optimization
- Personal branding strategies
- Public speaking and conferences
- Building online presence
- Networking strategies
- Mentorship and coaching
- Content creation

Projects built:

- Launch portfolio website
- Write 5 technical blog posts
- Contribute to 3 open source projects
- Create video tutorials

Practice: Build complete online presence

Interview Preparation

Topics:

- Data science interview process
- Technical interview preparation
- Coding challenges for DS
- ML system design interviews
- Case study interviews
- Behavioral interview questions
- STAR method for responses
- Salary negotiation
- Company research strategies
- Take-home assignments
- Presentation skills
- Mock interview practice

Projects built:

- Complete 50 LeetCode problems
- Design 5 ML systems
- Prepare behavioral stories
- Record mock interviews

Practice: Do 10 mock interviews

Job Search & Freelancing

Topics:

- Job search strategies
- Resume optimization for ATS
- Cover letter writing
- Networking for jobs
- Working with recruiters
- Freelancing as data scientist
- Building consulting business
- Finding clients
- Pricing your services
- Contract negotiation
- Remote work best practices
- Career growth planning

Projects built:

- Optimize resume for 5 job types
- Create freelance service offerings
- Build client proposal template
- Develop career roadmap

Practice: Apply to 20 relevant positions

Final Project & Graduation

Topics:

- Capstone project planning
- End-to-end implementation
- Production deployment
- Documentation and presentation
- Peer review and feedback
- Final assessment
- Certification preparation
- Alumni network
- Continuous learning plan
- Career launch strategy
- Celebration and reflection
- Next steps planning

Projects built:

- FINAL CAPSTONE: Industry-Ready Data Science Project
- Solve real business problem end-to-end
- Include all phases: data, ML, deployment, monitoring
- Present to panel of industry experts

- Open source the solution
- Assessment: Final comprehensive examination and project defense

Continuous Learning & Growth (Ongoing)

Staying Current

Topics:

- Following AI research
- Reading papers effectively
- Attending conferences
- Online communities
- Continuous experimentation
- Building side projects
- Teaching and mentoring
- Contributing to research
- Industry trends
- Emerging technologies
- Career pivots
- Leadership development

Projects built:

- Create learning roadmap
- Join research reading group
- Start mentoring others
- Plan conference attendance

Practice: Dedicate 5 hours/week to learning

Advanced Certifications

Topics:

- Cloud certifications (AWS, GCP, Azure)
- Specialized ML certifications
- Domain certifications
- Academic courses and MOOCs
- Professional development
- Executive education
- PhD considerations
- Research opportunities
- Teaching opportunities
- Consulting certifications
- Project management
- Business analytics

Projects built:

- Complete one advanced certification
- Plan certification roadmap
- Join professional organizations

Practice: Pursue continuous credentials

Building Data Products

Topics:

- Product thinking for DS
- Identifying opportunities
- MVP development
- User research
- Product metrics
- Growth strategies
- Monetization models
- B2B vs B2C products
- SaaS development
- API products
- Data marketplaces
- Entrepreneurship

Projects built:

- Ideate 5 data products
- Build MVP of one product
- Create business plan
- Launch beta version

Practice: Validate product ideas

Giving Back

Topics:

- Mentoring beginners
- Creating educational content
- Open source contributions
- Speaking at meetups
- Writing tutorials
- Answering questions online
- Building community
- Organizing events
- Pro bono work
- Teaching workshops
- Creating courses
- Industry advocacy

Projects built:

- Mentor 3 beginners

- Create free educational resource
 - Organize local meetup
 - Contribute to major open source project
- Practice: Give back to community weekly

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 15+ foundation projects - EDA, visualization, SQL analytics
- Phase 2: 20+ ML projects - classification, regression, clustering, Kaggle
- Phase 3: 15+ DL projects - computer vision, NLP, reinforcement learning
- Phase 4: 10+ production projects - MLOps, big data, cloud deployment
- Total: 60+ projects from basics to production

Total projects: 60+ data science projects across all domains

SKILLS THE STUDENT WILL HAVE AT THE END

- Programming: Python (expert), SQL (advanced), Spark, Git
- Mathematics: Linear algebra, calculus, statistics, probability
- Data Analysis: Pandas, NumPy, exploratory data analysis, feature engineering
- Machine Learning: Scikit-learn, XGBoost, classification, regression, clustering
- Deep Learning: TensorFlow, PyTorch, CNN, RNN, Transformers, GANs
- Visualization: Matplotlib, Seaborn, Plotly, Tableau, dashboard creation
- Big Data: Spark, Hadoop, Kafka, distributed computing
- MLOps: Docker, Kubernetes, CI/CD, model monitoring, A/B testing
- Cloud: AWS SageMaker, GCP Vertex AI, Azure ML
- Databases: SQL, NoSQL, data warehouses, data lakes
- Soft Skills: Communication, business acumen, project management
- Domains: Healthcare, finance, retail, marketing analytics

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-7 hours
 Hands on coding: 10-12 hours
 Projects: 4-6 hours
 Practice problems: 3-4 hours
 Total per week: 20-25 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving and doubt clearing sessions
 Mentorship: 1-on-1 guidance from senior data scientists
 Community: Active Discord community with 10,000+ members
 Code review: Expert code reviews for all major projects
 Career support: Resume review, mock interviews, job referrals
 Lifetime access: All content, updates, and new modules forever
 Kaggle support: Kaggle competition teams and guidance
 Cloud credits: Free credits for AWS, GCP, Azure practice

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 total)
 Final certificate: Professional Data Scientist Certification
 LinkedIn badge: Verified LinkedIn badge
 Industry recognized: A course-completion certificate you can add to your resume and LinkedIn.
 Portfolio projects: 60+ documented projects for portfolio
 Kaggle achievements: Support to reach Kaggle Expert level
 Specialization certificate: Certificate in chosen specialization area

WHERE THIS CAN LEAD

- Data Scientist
- Machine Learning Engineer
- AI Engineer
- Deep Learning Engineer
- MLOps Engineer
- Data Science Consultant
- Research Scientist
- Computer Vision Engineer
- NLP Engineer
- Business Intelligence Developer
- Quantitative Analyst
- Data Science Manager
- Chief Data Officer (with experience)

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a

promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Growing roles: Rs 6-10 LPA (Junior Data Scientist)

Experienced roles: Rs 10-20 LPA (Data Scientist); Rs 15-30 LPA (Senior Data Scientist); Rs 20-50+ LPA (Lead/Principal Data Scientist)

Freelance: Rs 3,000-10,000/hour based on expertise

International: \$80k-200k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is data science and why is it one of the highest-paying careers in 2026?

A: Data science combines statistics, programming, and domain expertise to extract insights from data. It's in massive demand because companies like Netflix, Amazon, and Google use data science for recommendations, predictions, and decision-making.

Q: Do I need a mathematics or statistics background for this data science course?

A: No prior math background is required! We teach all the mathematics you need - statistics, probability, linear algebra, and calculus - from the ground up. Many successful data scientists started without a math degree. The course includes dedicated modules to build a solid mathematical foundation alongside practical projects.

Q: What programming languages and tools will I learn in this 12-month data science program?

A: You'll master Python (the #1 language for data science), SQL, and Spark. Tools include Pandas, NumPy, Scikit-learn, TensorFlow, PyTorch, Matplotlib, Tableau, AWS SageMaker, and MLOps tools. By completion, you'll be proficient in 25+ technologies used by data science teams at top companies.

Q: What's the difference between a Data Scientist, ML Engineer, and Data Analyst?

A: This course prepares you for all three paths, with deep coverage of machine learning, deep learning, and MLOps.

Q: Will I learn AI and deep learning in this data science course?

A: Yes! This covers the cutting-edge of AI.

Q: What projects will I build to showcase in my data science portfolio?

A: You'll build 60+ projects including: stock price prediction, customer churn analysis, image classification, NLP sentiment analysis, recommendation systems, fraud detection, and a complete end-to-end ML pipeline deployed on AWS. Our graduates often showcase Kaggle competition rankings and published models as proof of expertise.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/data-science-complete-masterclass-college>

COURSE 47 OF 118: DATA SCIENCE FOR TEENS: PYTHON & YOUR FIRST ML MODELS

Course link: <https://learn.modernagecoders.com/courses/data-science-course-for-teens-python-data>

Subject area: Data Science

Designed for: Teens

Who can join: Ages 14 to 18, basic Python required

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 2-3 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online data science course for teens (14-18): pandas, cleaning real datasets, matplotlib, statistics and your first ML models with scikit-learn.

In one line: Real datasets, honest charts, and a first machine learning model that the student can actually explain.

ABOUT THE PROGRAMME

Data science is the rare school-age subject where the professional tools are free and the real material, actual datasets about movies, sport, cities, and climate, is sitting in public archives waiting to be questioned. This 6-month live course teaches teens to do the real thing: load data with pandas, clean it honestly, chart it with matplotlib without lying, reason about it with statistics, and finally train first machine learning models with scikit-learn and understand what they did. The course is deliberately anti-hype. Students learn that most of data science is careful cleaning and honest questions, that a chart can mislead and how not to let it, and that a model is only as good as the data and the person

reading its results. It ends with a data-story capstone: each student picks a question they care about, finds the data, does the analysis end to end, and presents a notebook that shows their reasoning. Basic Python is required; our Python for Teens course is the natural on-ramp for students who need it.

WHAT MAKES THIS COURSE DIFFERENT

- Real public datasets from the second month: movies, sports seasons, city open data, and climate records, not sanitized textbook tables
- A whole month on cleaning, because that is the actual job, and students learn to log every cleaning decision like professionals
- Honest charts are treated as an ethical skill: a full lesson on how visualizations mislead and how to avoid it
- Statistics taught for use, not for the exam: when the mean lies, what correlation does not prove
- Machine learning arrives late and lands properly: train-test splits and simple models the student can explain, not black-box demos
- The capstone is a question the student chose, answered end to end in a notebook they present and publish

THE LEARNING PATH, STEP BY STEP

Phase 1: Python tuned for data, then pandas: loading, filtering, grouping, and joining real datasets until table thinking is second nature

Phase 2: The craft: a full month cleaning genuinely messy data, then matplotlib and the discipline of charts that tell the truth

Phase 3: Statistics for judgment, first models with scikit-learn, and a capstone data story built from a question the student chose

WHAT A STUDENT NEEDS BEFORE STARTING

Python basics: Required: variables, loops, functions, and lists. Our Python for Teens course is the natural on-ramp if these are missing

Mathematics: Comfortable school maths. All statistics used is taught inside the course; no calculus is needed

Equipment: A computer with a modern browser and stable internet. Google Colab means nothing heavy is installed

Age: 14 to 18, matched to the reasoning and statistics content

WHO THIS COURSE IS FOR

Python graduates: Teens who finished a first Python course and want to apply it to something real rather than more syntax

Question askers: Students who argue from evidence and want the tools to find it themselves

Future stem students: Teens heading toward engineering, economics, or science degrees where this exact stack appears in year one

Sports and stats fans: Students who already read tables of standings and averages for fun and want to compute their own

Portfolio builders: Teens who want a published, defensible piece of analysis to show teachers and admissions offices

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: PYTHON FOR DATA AND THE PANDAS TOOLKIT (MONTHS 1-2, WEEKS 1-8) -----

First the Python muscles data work leans on, retrained in notebooks. Then pandas, the tool that turns tables of raw data into answers.

Month 1: Python, Retuned for Data (Weeks 1-4)

Notebooks and the Python Refresh

Topics:

- Google Colab and Jupyter notebooks: why data work lives in cells, not scripts
- Refresh: variables, lists, and dictionaries through data-flavored exercises
- Refresh: loops and conditionals over collections of records
- Markdown cells: notes and reasoning next to code
- Notebook habits: run order, restarts, and the stale-state trap
- The course map: where pandas, charts, and models fit

Projects built:

- Class survey analysis, round one: collect a small class survey and answer three questions about it with plain Python

Practice: 12 refresh exercises on lists and dictionaries, all phrased as data questions

Functions and Files

Topics:

- Refresh: writing and calling functions that transform data
- Reading files: opening a CSV by hand with the csv module
- What a CSV actually is: rows, delimiters, and headers

- Building lists of dictionaries from raw rows
- Writing results back out to a file
- Why this manual work matters: understanding what pandas automates

Projects built:

- Hand-rolled report: read a 500-row CSV with plain Python and compute totals, averages, and a top-five list

Practice: Write three reusable functions: load a CSV, filter rows by a condition, summarize a column

Thinking in Tables

Topics:

- Rows as records, columns as variables: the table mental model
- Data types in the wild: numbers, text, dates, and categories
- First contact with messy data: a deliberately imperfect dataset
- Questions data can answer, and questions it cannot
- From question to query: translating curiosity into precise operations
- Sources of real data: open data portals and public archives, toured live

Projects built:

- Question workshop: pick a public dataset, write five precise questions about it, and classify which are answerable from the columns present

Practice: Find one dataset about something you personally care about and describe its rows, columns, and types

NumPy Essentials

Topics:

- Arrays vs lists: what NumPy is for
- Vectorized operations: arithmetic on whole columns at once
- Useful reductions: sum, mean, min, max, and where they mislead
- Boolean masks: selecting data with conditions, the idea pandas is built on
- A speed demonstration: the same computation with a loop and with NumPy
- Reading NumPy errors without panic

Projects built:

- Class survey analysis, round two: redo week 1's analysis in NumPy and compare the code side by side

Practice: 10 NumPy exercises: masks, reductions, and column arithmetic on a provided dataset

Assessment: Month 1 check: live notebook exercise combining functions, file reading, and NumPy on a fresh dataset

Month 2: pandas (Weeks 5-8)

DataFrames

Topics:

- Series and DataFrames: pandas's two shapes
- Loading CSVs with read_csv and its most useful options
- First-look tools: head, info, describe, and shape
- Selecting columns and rows: loc and iloc without the usual confusion
- Column arithmetic: creating new columns from old ones
- Saving your work: to_csv and keeping raw data untouched

Projects built:

- First pandas notebook: load a movies dataset and produce a profile of it, size, columns, types, and five first findings

Practice: Load three different datasets and write a three-line profile of each

Filtering and Sorting

Topics:

- Boolean masks in pandas: rows matching a condition
- Combining conditions: and, or, and the parentheses that matter
- isin, between, and string contains for readable filters
- Sorting by one and several columns
- nlargest and nsmallest: top-N questions answered directly
- Chaining operations without losing readability

Projects built:

- Movie interrogation: answer ten ranked questions about the movies dataset, best-rated per decade, longest, most profitable, each as a clean filtered result

Practice: Write and answer eight filter questions of your own on a sports dataset

Grouping and Aggregation

Topics:

- value_counts: the fastest first answer in pandas
- groupby: split, apply, combine, drawn before coded
- Aggregations: mean, sum, count, and multiple at once with agg
- Grouping by more than one column
- Sorting grouped results to find leaders and outliers
- Pivot tables, gently: the spreadsheet idea in pandas form

Projects built:

- Season in review: full statistical breakdown of a sports season, per team and per month, from a raw match-results dataset

Practice: Answer six groupby questions on a dataset of your choice and mark the most surprising result

Combining Datasets

Topics:

- concat: stacking datasets that share columns
- merge: joining tables on a shared key, with the four join types in pictures

- What gets lost in a join: rows that match nothing
- Checking a merge: row counts before and after
- Tidying column names and types after combining
- The month project workflow: load, join, group, answer

Projects built:

- Two-table analysis: join two related datasets, for example city populations with air quality readings, and answer three questions no single table could

Practice: Perform one merge on new data and write two sentences on rows that failed to match and why

Assessment: Month 2 check: timed pandas practical, six questions on an unseen dataset, filters through joins

PHASE 2: CLEANING REAL DATA AND CHARTING IT HONESTLY (MONTHS 3-4, WEEKS 9-16)

The craft phase. Real datasets arrive dirty, and most of data science is the disciplined work of fixing them, then showing results without distortion.

Month 3: Cleaning Real Datasets (Weeks 9-12)

Missing Values

Topics:

- Finding the holes: isna, counts, and missingness patterns
- Why data goes missing, and why the reason changes the fix
- Dropping rows or columns: when it is honest and when it hides too much
- Filling values: constants, means, medians, and the trade-offs of each
- The cleaning log: recording every decision and its justification
- Raw data stays raw: cleaned copies, never overwritten originals

Projects built:

- Missing-value audit: map the missingness of a real open dataset, choose treatments per column, and defend each in your cleaning log

Practice: Apply two different missing-value strategies to the same column and compare how the summary statistics move

Wrong Types and Dirty Strings

Topics:

- Numbers trapped in text: currency symbols, commas, and units
- String repair with .str methods: strip, replace, lower, split
- Dates done right: to_datetime and what it makes possible
- Categories with typos: Delhi, delhi, and DELHI as one value
- astype and coercion errors: reading what pandas is telling you
- A repair pipeline: ordered, rerunnable cleaning steps in one notebook section

Projects built:

- Type rescue: take a scraped price-and-date dataset where every column is text and restore true types, documenting each repair

Practice: Fix eight prepared dirty columns, one repair function each

Outliers, Duplicates, and Judgment Calls

Topics:

- Spotting outliers: sorting, describe, and quick plots
- Error or truth: the 190-year-old customer vs the genuine billionaire
- Duplicates: exact, partial, and the drop_duplicates decisions
- The ethics of cleaning: every removal changes the conclusion
- Documenting judgment calls so others can disagree with you fairly
- Sanity checks: totals and counts that must still make sense after cleaning

Projects built:

- Judgment file: on a dataset seeded with ambiguous outliers and duplicates, decide keep or remove case by case, with one-line justifications

Practice: Trade judgment files with a classmate and write where you disagreed and why

The Cleaning Project

Topics:

- A genuinely messy public dataset, start to finish
- Planning the clean: audit first, repair second, verify third
- Combining the month: missingness, types, strings, outliers, duplicates
- Writing the cleaning log as a professional document
- Before-and-after: quantifying what the clean changed
- Delivering an analysis-ready dataset someone else could trust

Projects built:

- Full cleaning project: raw city open data, for example street trees or road accidents, taken from download to documented, analysis-ready form

Practice: Run your cleaned dataset through five sanity checks and record the results

Assessment: Month 3 check: cleaning log review plus a defense of three judgment calls in a short viva

Month 4: Visualization with matplotlib (Weeks 13-16)

First Charts

Topics:

- The anatomy of a figure: axes, ticks, labels, title, legend
- Line charts: change over time, done cleanly
- Bar charts: comparing categories without clutter
- Scatter plots: two variables meeting for the first time
- Labeling as a duty: a chart must explain itself

- Plotting straight from pandas, and when to drop to matplotlib

Projects built:

- Three-chart notebook: one line, one bar, one scatter from your cleaned month 3 dataset, each fully labeled

Practice: Recreate two published charts from their underlying data as closely as you can

Distributions and the Right Chart

Topics:

- Histograms: seeing the shape of a column
- Bin sizes and how they change the story
- Box plots: medians, quartiles, and outliers at a glance
- Comparing groups: side-by-side boxes and grouped bars
- Choosing the chart from the question, not from habit
- The chart-choice checklist: comparison, trend, distribution, relationship

Projects built:

- Distribution study: profile three numeric columns with histograms and box plots, and write what each shape reveals

Practice: For six written questions, name the right chart type and justify it in one line each

Honest Charts and Polished Figures

Topics:

- How charts mislead: truncated axes, cherry-picked windows, and area tricks
- Fixing a misleading chart without hiding the real pattern
- Multi-panel figures with subplots
- Color with intent: highlighting one series, not decorating all of them
- Annotating the point: arrows and text that guide the reader
- Exporting print-quality figures

Projects built:

- Mislead and repair: take one truthful chart, make the most misleading legal version of it, then write the repair, a before-and-after pair

Practice: Find one misleading chart in real media and write a three-sentence critique

The Four-Chart Data Story

Topics:

- Narrative order: charts arranged as an argument
- One message per chart: trimming everything else
- Writing connective text between figures
- The takeaway figure: your single most important chart
- Presenting figures aloud: saying what to look at
- Peer review: reading a classmate's story cold

Projects built:

- Mini data story: a four-chart narrative on a dataset of your choice, presented live in five minutes

Practice: Revise your story after peer feedback and note the two changes that most improved it

Assessment: Month 4 check: data story presentation, graded on chart choice, honesty, and clarity

----- PHASE 3: STATISTICS, FIRST MODELS, AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24) -----

Statistics gives judgment, scikit-learn gives first models, and the capstone puts it all behind one question the student actually cares about.

Month 5: Statistics for Judgment, Then First Models (Weeks 17-20)

Describing Data Truthfully

Topics:

- Mean, median, and mode: three centers, three stories
- When the mean lies: income, house prices, and skew
- Spread: range, quartiles, and standard deviation as a feel, not a formula
- Percentiles: where a value really stands
- Summaries that hide things: same mean, wildly different data
- Writing statistical sentences that a non-technical reader cannot misread

Projects built:

- Two-summary exercise: describe the same real dataset once to flatter it and once to be fair, then annotate the tricks in the flattering version

Practice: Compute and interpret full summaries for three columns of your capstone-candidate dataset

Correlation and Its Traps

Topics:

- Correlation as a number: direction and strength on a scatter plot
- Computing correlations in pandas and reading a correlation matrix
- Correlation is not causation: the argument, with famous spurious examples
- Confounders: the hidden third variable, explained through cases
- When correlation is still useful despite the traps
- Reporting a relationship without overclaiming it

Projects built:

- Correlation hunt: find the three strongest relationships in a real dataset, then argue for each whether causation is even plausible

Practice: Find one spurious correlation in the wild or in the class archive and explain

the likely confounder

First Machine Learning: Regression

Topics:

- What learning from data means, without mystique
- Features and targets: framing a prediction question
- The train-test split: why we grade models on unseen data
- Linear regression with scikit-learn: fit, predict, inspect
- Reading the model: coefficients as the story it learned
- Error metrics that mean something: how wrong is the model, in real units

Projects built:

- Price predictor: train a linear regression on a real housing or used-car dataset and report its typical error honestly

Practice: Add one feature to your model, remeasure the error, and write whether it earned its place

First Machine Learning: Classification

Topics:

- Classification vs regression: predicting a label instead of a number
- k-nearest neighbors: the friendliest first classifier
- Decision trees: models you can read as rules
- Accuracy and its limits: the imbalanced-classes trap
- The confusion matrix: seeing which mistakes the model makes
- Overfitting: memorizing vs learning, in one intuitive demonstration

Projects built:

- Penguin classifier: train and compare k-nearest neighbors and a decision tree on the Palmer Penguins dataset, with a confusion matrix for each

Practice: Break your classifier on purpose by shrinking training data, and chart accuracy against training size

Assessment: Month 5 check: statistics and modeling quiz plus a live reading of your own model's results

Month 6: The Capstone Data Story (Weeks 21-24)

The Question and the Data

Topics:

- Choosing a question you actually care about: sport, music, games, city, climate
- Dataset hunting: Kaggle, open data portals, and evaluating what you find
- Feasibility: does this data contain an answer to this question
- Scoping to six weeks of classes: the honest project plan
- Setting up the capstone notebook: sections for question, data, cleaning, analysis, conclusion
- Instructor sign-off on question, dataset, and plan

Projects built:

- Approved capstone plan: question, dataset, feasibility notes, and a week-by-week checklist

Practice: Complete the data audit of your chosen dataset: size, columns, types, missingness

Analysis Sprint

Topics:

- Cleaning your capstone data with a proper cleaning log
- Exploratory analysis: groupbys and filters chasing the question
- First figures: charting what the exploration surfaces
- Mid-sprint review with the instructor: findings so far, gaps remaining
- Following surprises: when the data disagrees with your hunch
- Keeping the notebook readable while you work fast

Projects built:

- Capstone core analysis: cleaned data, exploratory findings, and first draft figures in the notebook

Practice: Write your three strongest preliminary findings, each backed by one figure

The Optional Model and the Narrative

Topics:

- Deciding whether a model helps your question or just decorates it
- If modeling: framing, training, and honest error reporting
- Assembling the narrative: question, data, cleaning, findings, conclusion
- Limitations sections: what your analysis cannot claim, stated plainly
- Polishing figures to month 4 standard
- Dry-run presentations with peer feedback

Projects built:

- Complete capstone notebook: full narrative from question to conclusion, with a limitations section, model included only if it earns its place

Practice: Run your dry-run presentation for a family member and note the question that stumped you

Present and Publish

Topics:

- The final presentation: five minutes, the question, three findings, one takeaway figure
- Fielding questions about your own analysis
- Publishing the notebook to GitHub so it has a public address
- Writing the README: question, data source, and what you found
- Where this goes next: deeper ML, statistics, and college coursework
- Certificates and course close

Projects built:

- Published capstone: notebook live on GitHub, presented to the class and families

Practice: Send your published notebook to one teacher or mentor outside the course

Assessment: Capstone assessment: presentation, notebook quality, cleaning log, and honesty of claims

PROJECTS THE STUDENT BUILDS AND KEEPS

- Class survey analysis, done twice: plain Python, then NumPy
- Movie dataset interrogation: ten ranked questions, ten clean answers
- Sports season statistical review with groupby
- Two-table join analysis across related datasets
- Full cleaning project on raw city open data, with a professional cleaning log
- Mislead-and-repair chart pair: the same data, dishonest then honest
- Four-chart data story presented live
- Housing price regression with honestly reported error
- Penguin species classifier with confusion matrices
- The capstone: a published notebook answering a question the student chose

Total projects: 12+ analysis projects, ending with a published capstone notebook on GitHub

SKILLS THE STUDENT WILL HAVE AT THE END

- pandas fluency: loading, filtering, grouping, and joining real datasets with confidence
- Data cleaning: missing values, broken types, outliers, and duplicates handled with documented judgment
- Visualization: matplotlib figures chosen for the question and built to tell the truth
- Statistics for judgment: centers, spread, and correlation used without overclaiming
- First machine learning: regression and classification with scikit-learn, evaluated honestly on unseen data
- The analyst's workflow: notebooks, cleaning logs, limitations sections, and presenting findings aloud

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, working in notebooks alongside the instructor

Practice: 2-3 hours of dataset exercises and project work between classes

Review: Notebooks reviewed with written feedback, with special attention to honest claims and clean reasoning

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for stuck code and confusing datasets

Progress updates: Regular progress notes to parents: skills covered, projects completed, and what needs practice

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside a published capstone notebook that shows the work

WHERE THIS CAN LEAD

- Our AI and machine learning courses, which pick up exactly where the scikit-learn month ends
- Deeper Python engineering through our advanced Python and data structures courses
- School science fairs and economics or statistics projects, now backed by real analysis skills
- College coursework in data science, economics, and the sciences with a working head start
- Independent analysis projects: the student now owns the full stack from raw CSV to published notebook

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does my teen need to know Python before this course?

A: Yes, the basics: variables, loops, functions, and lists. Month 1 includes a structured refresh, but it assumes prior exposure rather than teaching Python from zero. Students without that background should take our Python for Teens course first; it is the natural on-ramp, and many students move straight from one into the other.

Q: How much mathematics does the course require?

A: Comfortable school maths is enough. Every statistical idea the course uses, means and medians, spread, percentiles, correlation, is taught inside the course with real data, and there is no calculus anywhere. The emphasis is on judgment: knowing when an average misleads matters far more here than deriving formulas.

Q: Is this real machine learning or just a demo?

A: Real, at an honest first level. Students train linear regression and classification models on real datasets with scikit-learn, use proper train-test splits, read confusion matrices, and meet overfitting directly. What we deliberately avoid is black-box theatre: no pasting magic code that scores well unexplained. A student finishing this course can tell you what their model learned and how wrong it typically is, which is more than many adults using these

tools can.

Q: What datasets will my teen work with?

A: Real public data throughout: movie databases, sports seasons, city open data such as street trees and road accidents, climate records, and the Palmer Penguins dataset for classification. From month 3 the data arrives genuinely messy on purpose, because cleaning real data is the core professional skill. For the capstone, students choose their own dataset around a question they care about.

Q: What does the capstone involve?

A: Each student picks a question, finds a suitable public dataset, and does the whole job: cleaning with a documented log, exploratory analysis, honest figures, an optional model if it genuinely helps, and a written conclusion with a limitations section. The finished notebook is published to GitHub and presented live. It is a genuine piece of analysis with a public address, not a classroom-only exercise.

Q: How are the classes taught?

A: Live one-hour online classes, twice a week, in small groups capped at 10 students. Students work in their own notebooks during class rather than watching, and homework notebooks come back with written feedback. Mini-batch (3 to 4 students) and personal 1-on-1 options are available for students who want closer attention.

Q: Where does a teen go after this course?

A: The most common next step is our AI and machine learning track, which starts exactly where the scikit-learn month ends. Students leaning toward engineering deepen their Python with our advanced courses, and many put the skills straight to work in school science fairs and statistics projects. The capstone notebook also carries weight on its own as evidence of real analytical work for college applications.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/data-science-course-for-teens-python-data>

=====

COURSE 48 OF 118: DATA STRUCTURES & ALGORITHMS COURSE: INTERVIEW-READY DSA

=====

Course link: <https://learn.modernagecoders.com/courses/data-structures-algorithms-masterclass-college>

Subject area: Computer Science & Interview Preparation

Designed for: College students

Who can join: Beginner to Advanced Problem Solver

Course length: 6 months (26 weeks)

Time needed: 15-20 hours/week recommended

Certificate: DSA Mastery & Interview Ready Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online data structures and algorithms course: from arrays to graph algorithms, master problem-solving and crack FAANG interviews with 500+ problems.

In one line: From Zero to FAANG Interview Ready in 6 Months

ABOUT THE PROGRAMME

This intensive 6-month program transforms beginners into expert problem solvers capable of cracking top tech company interviews. Master every data structure from arrays to advanced graphs, learn algorithmic thinking, dynamic programming, and solve 500+ carefully curated problems. You'll build strong foundations in complexity analysis, implement data structures from scratch, master recursion and backtracking, conquer dynamic programming, and develop the problem-solving skills needed for competitive programming and technical interviews.

WHAT MAKES THIS COURSE DIFFERENT

- Learn by implementing from scratch
- 500+ hand-picked problems with video solutions
- Pattern-based learning approach
- Interview-focused curriculum
- Live problem-solving sessions
- Mock interviews with feedback
- Company-specific preparation tracks
- Competitive programming included

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-2): Arrays, Strings, Linked Lists, Stacks, Queues, Basic

Algorithms

Phase 2: Intermediate (Months 3-4): Trees, Graphs, Heaps, Hashing, Advanced Sorting, Searching

Phase 3: Advanced (Months 5-6): Dynamic Programming, Advanced Graphs, System Design, Interview Mastery

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic programming knowledge helpful

Coding experience: Can write simple programs (loops, conditions)

Equipment: Computer with internet connection

Time commitment: 15-20 hours per week

English: Good comprehension required

Motivation: Strong desire to excel in technical interviews

WHO THIS COURSE IS FOR

Students: CS students preparing for placements

Working professionals: Developers preparing for job switch

Competitive programmers: Want to improve contest performance

Self learners: Want structured DSA learning path

Bootcamp graduates: Need CS fundamentals

Career switchers: Breaking into tech

Anyone: Passionate about problem-solving

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE DATA STRUCTURES (MONTHS 1-2, WEEKS 1-8) -----

Build rock-solid foundations with basic data structures, complexity analysis, and fundamental algorithms.

Month 1: Foundations & Linear Data Structures (Week 1-4)

Programming Fundamentals & Complexity Analysis

Topics:

- Course introduction and roadmap
- Setting up coding environment (VS Code, online judges)
- Introduction to problem-solving approach
- Time complexity and Big O notation
- Space complexity analysis
- Best, average, and worst case analysis
- Common complexity classes: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$
- Analyzing nested loops
- Recursion complexity analysis
- Amortized analysis basics
- Trade-offs: time vs space
- How to approach a problem systematically

Projects built:

- Complexity analyzer tool
- Problem-solving template creator
- Personal progress tracker

Practice: Analyze complexity of 50 code snippets

Arrays & Strings Mastery

Topics:

- Static vs dynamic arrays
- Array operations and complexity
- Two-pointer technique
- Sliding window technique
- Prefix sum arrays
- Kadane's algorithm
- Moore's voting algorithm
- String manipulation techniques
- String matching algorithms
- KMP algorithm basics
- Rabin-Karp algorithm
- Character encoding and Unicode

Projects built:

- Array manipulation library
- String utilities toolkit
- Pattern matching visualizer

Practice: Solve 40 array/string problems

Linked Lists & Recursion

Topics:

- Singly linked list implementation
- Doubly linked list
- Circular linked list

- Fast and slow pointer technique
- Floyd's cycle detection
- Linked list reversal (iterative and recursive)
- Merge operations on linked lists
- Recursion fundamentals
- Base case and recursive case
- Call stack and memory
- Tail recursion optimization
- Recursion tree analysis

Projects built:

- Linked list library from scratch
- Recursion visualizer
- Memory stack simulator

Practice: Implement 25 linked list operations

Stacks & Queues

Topics:

- Stack implementation using array and linked list
- Stack applications: expression evaluation, parentheses matching
- Infix to postfix conversion
- Queue implementation: array-based and linked list
- Circular queue implementation
- Deque (double-ended queue)
- Priority queue basics
- Monotonic stack patterns
- Monotonic queue patterns
- Stack and queue in recursion
- BFS and DFS preview
- Real-world applications

Projects built:

- Expression evaluator
- Browser history simulator
- Task scheduler with priority queue

Practice: Build 20 stack/queue applications

Month 2: Searching, Sorting & Hashing (Week 5-8)

Searching Algorithms

Topics:

- Linear search and optimizations
- Binary search deep dive
- Binary search on answer
- Search in rotated sorted array
- Finding boundaries with binary search
- Ternary search
- Exponential search
- Jump search
- Interpolation search
- Search in 2D matrices
- Finding peak elements
- Search space reduction techniques

Projects built:

- Search algorithm visualizer
- Binary search template library
- Search complexity analyzer

Practice: Solve 30 binary search problems

Sorting Algorithms Mastery

Topics:

- Bubble sort and optimizations
- Selection sort analysis
- Insertion sort and when to use
- Merge sort implementation
- Quick sort and partitioning
- Randomized quick sort
- Heap sort preview
- Counting sort for integers
- Radix sort implementation
- Bucket sort technique
- Stability in sorting
- External sorting for large data

Projects built:

- Sorting visualizer application
- Hybrid sorting algorithm
- Performance comparison tool

Practice: Implement all sorting algorithms

Hashing & Hash Tables

Topics:

- Hash function design
- Collision resolution: chaining vs open addressing
- Load factor and rehashing
- Hash table implementation from scratch

- Hash map vs hash set
- Rolling hash technique
- Consistent hashing basics
- Bloom filters introduction
- Hash table patterns in problems
- Two-sum pattern and variations
- Frequency counting with hash maps
- Hash table optimization techniques

Projects built:

- Custom hash table implementation
- LRU cache implementation
- Distributed hash table simulator

Practice: Solve 35 hashing problems

Bit Manipulation & Mathematics

Topics:

- Binary number system review
- Bitwise operators: AND, OR, XOR, NOT
- Left shift and right shift
- Bit manipulation tricks
- Setting, clearing, toggling bits
- Counting set bits
- Power of 2 checks
- XOR properties and applications
- Bit masking techniques
- GCD and LCM algorithms
- Prime number algorithms
- Modular arithmetic basics

Projects built:

- Bit manipulation toolkit
- Number theory library
- Cryptography basics implementation

Practice: Master 25 bit manipulation techniques

Phase 1 Assessment

Topics:

- Review of all basic data structures
- Complexity analysis mastery check
- Problem-solving patterns review
- Mock interview problems
- Code quality and optimization

Projects built:

- CAPSTONE: Build Your Own STL
- Implement vector, list, stack, queue, unordered_map from scratch
- Include iterators and common algorithms

Assessment: Phase 1 Exam - 10 problems in 3 hours

Months 3-4: Trees, Heaps & Graphs (Week 9-17)

Binary Trees Fundamentals

Topics:

- Tree terminology and properties
- Binary tree representation
- Tree traversals: inorder, preorder, postorder
- Level order traversal (BFS)
- Recursive vs iterative traversals
- Morris traversal ($O(1)$ space)
- Tree construction from traversals
- Diameter, height, and width of tree
- Path sum problems
- Lowest Common Ancestor (LCA)
- Serialize and deserialize trees
- Tree isomorphism

Projects built:

- Binary tree visualizer
- Tree traversal animator
- Tree problem solver toolkit

Practice: Solve 40 binary tree problems

Binary Search Trees & Balanced Trees

Topics:

- BST properties and operations
- BST insertion and deletion
- BST validation and searching
- Inorder successor and predecessor
- Convert sorted array to BST
- Self-balancing trees introduction
- AVL tree rotations
- AVL tree implementation
- Red-Black tree basics
- B-trees and B+ trees overview
- Segment trees introduction
- Binary Indexed Tree (Fenwick Tree)

Projects built:

- BST implementation with all operations
- AVL tree from scratch
- Range query data structure

Practice: Implement 20 BST operations

Heaps & Priority Queues

Topics:

- Heap properties: min-heap and max-heap
- Heap implementation using array
- Heapify operation and complexity
- Heap sort implementation
- Priority queue operations
- Binary heap vs Fibonacci heap
- K-way merge using heaps
- Median finding with two heaps
- Top K elements pattern
- Heap applications in algorithms
- Dijkstra's algorithm preview
- Custom comparators in heaps

Projects built:

- Heap implementation from scratch
- Task scheduler with priority
- Real-time median finder

Practice: Master 25 heap problems

Tries & Advanced Trees

Topics:

- Trie data structure implementation
- Trie insertion, search, and deletion
- Prefix matching with tries
- Auto-complete functionality
- Word search in matrix using trie
- Suffix trees and arrays basics
- N-ary tree operations
- Quad trees and Oct trees
- K-d trees for spatial data
- Disjoint Set Union (DSU/Union-Find)
- Path compression and union by rank
- Applications of DSU

Projects built:

- Autocomplete system
- Spell checker implementation
- Contact list with search

Practice: Build 15 trie applications

Phase 2 Assessment

Topics:

- Trees and graphs review
- Advanced data structures check
- Pattern recognition in problems
- Optimization techniques
- Mock technical interview

Projects built:

- MAJOR CAPSTONE: LeetCode Clone
- Build online judge with problem bank, test cases, and submission system
- Include leaderboard and contest functionality

Assessment: Phase 2 Exam - 8 problems in 3 hours

PHASE 2: GRAPHS & ADVANCED ALGORITHMS (MONTHS 3-4, WEEKS 9-17)

Master graph algorithms, advanced problem-solving patterns, and complex algorithmic techniques.

Month 3: Graph Algorithms Mastery (Week 9-13)

Graph Fundamentals & Traversals

Topics:

- Graph representations: adjacency matrix vs list
- Directed vs undirected graphs
- Weighted vs unweighted graphs
- Graph properties: connected, cyclic, bipartite
- Depth-First Search (DFS) implementation
- Breadth-First Search (BFS) implementation
- DFS applications: cycle detection, topological sort
- BFS applications: shortest path, level order
- Connected components finding
- Strongly connected components (Kosaraju's, Tarjan's)
- Bridges and articulation points
- Graph coloring basics

Projects built:

- Graph visualizer with traversals

- Social network analyzer
- Maze solver with BFS/DFS

Practice: Solve 35 graph traversal problems

Shortest Path Algorithms

Topics:

- Dijkstra's algorithm implementation
- Dijkstra's with priority queue
- Bellman-Ford algorithm
- Negative weight cycles detection
- Floyd-Warshall algorithm
- A* search algorithm basics
- Shortest path in DAG
- Single source vs all pairs
- Path reconstruction techniques
- Multi-source BFS
- 0-1 BFS technique
- Bidirectional search

Projects built:

- Route finder application
- Navigation system simulator
- Network routing optimizer

Practice: Implement all shortest path algorithms

Minimum Spanning Trees & Network Flow

Topics:

- Minimum Spanning Tree (MST) concept
- Kruskal's algorithm with DSU
- Prim's algorithm implementation
- Applications of MST
- Network flow introduction
- Ford-Fulkerson method
- Maximum flow problem
- Minimum cut theorem
- Bipartite matching
- Hungarian algorithm basics
- Stable marriage problem
- Assignment problems

Projects built:

- Network designer tool
- Resource allocation system
- Matching algorithm implementation

Practice: Solve 20 MST and flow problems

Advanced Graph Patterns

Topics:

- Topological sorting variations
- Cycle detection in directed/undirected graphs
- Bipartite graph checking and coloring
- Eulerian path and circuit
- Hamiltonian path problem
- Graph dynamic programming
- Tree DP on graphs
- Centroid decomposition basics
- Heavy-light decomposition intro
- Graph compression techniques
- Parallel BFS/DFS
- Graph optimization problems

Projects built:

- Dependency resolver
- Circuit board router
- Tournament scheduler

Practice: Master 25 advanced graph patterns

String Algorithms Advanced

Topics:

- KMP algorithm implementation
- Z-algorithm for pattern matching
- Rabin-Karp with rolling hash
- Boyer-Moore algorithm
- Suffix arrays construction
- Longest Common Prefix array
- Manacher's algorithm for palindromes
- Edit distance algorithms
- Longest common subsequence/substring
- String hashing techniques
- Suffix automaton basics
- Aho-Corasick algorithm

Projects built:

- Text editor with find/replace
- Plagiarism detector
- DNA sequence analyzer

Practice: Implement 15 string algorithms

Month 4: Dynamic Programming Mastery (Week 14-17)

Dynamic Programming Fundamentals

Topics:

- Introduction to dynamic programming
- Recursion to DP transition
- Memoization techniques
- Top-down vs bottom-up approach
- 1D DP patterns
- State definition in DP
- Fibonacci variations
- Climbing stairs variations
- House robber pattern
- Jump game problems
- Decode ways pattern
- DP space optimization

Projects built:

- DP visualizer tool
- Recursion tree analyzer
- DP pattern identifier

Practice: Solve 40 basic DP problems

2D Dynamic Programming

Topics:

- 2D DP table construction
- Grid-based DP problems
- Path counting in grids
- Longest common subsequence
- Edit distance problem
- 0/1 Knapsack problem
- Unbounded knapsack
- Subset sum problems
- Matrix chain multiplication
- Palindrome partitioning
- Regular expression matching
- Wildcard matching

Projects built:

- Sequence alignment tool
- Knapsack optimizer
- Pattern matcher implementation

Practice: Master 35 2D DP problems

Advanced DP Patterns

Topics:

- State machine DP
- DP on trees
- DP with bitmasking
- Digit DP technique
- Probability DP
- Game theory and DP
- Interval DP pattern
- String DP problems
- DP optimization techniques
- Convex hull trick
- Divide and conquer optimization
- Knuth-Yao speedup

Projects built:

- Stock trading simulator with DP
- Game AI using minimax
- Optimal strategy finder

Practice: Solve 30 advanced DP problems

Greedy Algorithms & Optimization

Topics:

- Greedy algorithm principles
- When greedy works: proof techniques
- Activity selection problem
- Huffman coding
- Fractional knapsack
- Job scheduling problems
- Interval scheduling maximization
- Minimum platforms problem
- Gas station circuit problem
- Jump game greedy approach
- Greedy vs DP comparison
- Exchange argument proof

Projects built:

- Task scheduler optimizer
- Huffman compression tool
- Resource allocator

Practice: Master 25 greedy problems

Backtracking & Branch and Bound

Topics:

- Backtracking fundamentals
- State space tree
- N-Queens problem
- Sudoku solver
- Subset generation
- Permutation generation
- Combination sum problems
- Word search in grid
- Graph coloring with backtracking
- Branch and bound technique
- Pruning strategies
- Optimization in backtracking

Projects built:

- Sudoku solver application
- N-Queens visualizer
- Constraint satisfaction solver

Practice: Implement 20 backtracking solutions

Months 5-6: Advanced Topics & Interview Preparation (Week 18-26)

Advanced Algorithms

Topics:

- Divide and conquer mastery
- Master theorem for recurrence
- Fast Fourier Transform basics
- Karatsuba multiplication
- Strassen's matrix multiplication
- Closest pair of points
- Convex hull algorithms
- Line sweep technique
- Mo's algorithm basics
- Square root decomposition
- Heavy-light decomposition
- Centroid decomposition

Projects built:

- Computational geometry library
- Big integer calculator
- Algorithm complexity analyzer

Practice: Implement 15 advanced algorithms

System Design Basics for Interviews

Topics:

- System design interview approach
- Scalability basics
- Load balancing concepts
- Caching strategies
- Database design basics
- SQL vs NoSQL
- Microservices overview
- API design principles
- Rate limiting algorithms
- Consistent hashing
- Distributed systems basics
- CAP theorem introduction

Projects built:

- URL shortener design
- Rate limiter implementation
- Simple cache system

Practice: Design 10 systems

Competitive Programming

Topics:

- Competitive programming platforms
- Contest strategies and time management
- Fast I/O techniques
- Template preparation
- Common CP patterns
- Number theory for CP
- Combinatorics in problems
- Game theory basics
- Interactive problems
- Debugging techniques
- Test case generation
- Stress testing solutions

Projects built:

- CP template library
- Contest simulator
- Test case generator

Practice: Participate in 10 contests

Mock Interviews & Behavioral Prep

Topics:

- Interview preparation strategy
- Company-specific preparation
- Behavioral interview questions
- STAR method for answers
- System design interview practice
- Coding interview best practices
- Whiteboard coding techniques
- Communication during interviews
- Handling difficult problems
- Time management in interviews
- Post-interview follow-up
- Salary negotiation basics

Projects built:

- Interview preparation checklist
- Behavioral answer bank
- Company research template

Practice: Complete 20 mock interviews

Phase 3 Final Assessment

Topics:

- Comprehensive review
- Weak area identification
- Advanced problem solving
- Speed coding practice
- Final mock interviews

Projects built:

- MAJOR CAPSTONE: Algorithm Visualizer Platform
- Build interactive platform with 50+ algorithm visualizations
- Include step-by-step execution and complexity analysis

Assessment: Final Exam - 6 problems in 3 hours (FAANG level)

----- PHASE 3: INTERVIEW MASTERY & CAREER LAUNCH (MONTHS 5-6, WEEKS 18-26) -----

Master interview skills, tackle company-specific problems, and launch your tech career.

Month 5: Company-Specific Preparation (Week 18-22)

Google Interview Preparation

Topics:

- Google interview process overview
- Google's evaluation criteria
- Googliness and Leadership
- Common Google problem patterns
- Array and string manipulation focus
- Graph algorithms emphasis
- Dynamic programming at Google
- System design at Google scale
- Google coding style guide
- Time complexity requirements
- Follow-up questions handling
- Google behavioral questions

Projects built:

- Google-style problem solver
- MapReduce implementation
- PageRank algorithm

Practice: Solve 50 Google tagged problems

Amazon Interview Preparation

Topics:

- Amazon interview process
- Leadership Principles deep dive
- Amazon problem-solving approach
- Object-oriented design at Amazon
- Scalability focus
- Two-pointer and sliding window emphasis
- Tree and graph problems
- Amazon system design topics
- Working backwards methodology
- Bar raiser round preparation
- Amazon behavioral questions
- STAR format examples

Projects built:

- E-commerce system design
- Recommendation engine
- Warehouse robot pathfinding

Practice: Solve 50 Amazon tagged problems

Meta (Facebook) Preparation

Topics:

- Meta interview structure
- Ninja, Pirate, Jedi expectations
- Meta's focus areas

- Array and string heavy problems
- BFS/DFS variations
- Dynamic programming patterns
- Design Facebook features
- Meta's behavioral assessment
- Move fast philosophy
- Impact-focused answers
- Coding signal importance
- System design for social networks

Projects built:

- Social network features
- News feed algorithm
- Friend recommendation system

Practice: Solve 50 Meta tagged problems

Microsoft & Apple Preparation

Topics:

- Microsoft interview approach
- Problem-solving communication
- Microsoft stack questions
- Linked list and tree focus
- String manipulation problems
- Apple interview process
- Quality and attention to detail
- iOS/macOS specific questions
- Design patterns emphasis
- Both companies' behavioral focus
- Growth mindset demonstration
- Innovation and creativity

Projects built:

- Operating system scheduler
- File system design
- Music streaming service

Practice: Solve 40 MS/Apple problems

Startup & Unicorn Preparation

Topics:

- Startup interview differences
- Full-stack expectations
- Practical coding tests
- Take-home assignments
- System design for startups
- Real-world problem solving
- API design and implementation
- Database design questions
- DevOps and deployment
- Startup culture fit
- Ownership and initiative
- Rapid iteration mindset

Projects built:

- MVP product design
- Real-time collaboration tool
- Payment processing system

Practice: Complete 3 take-home projects

Month 6: Final Polish & Career Launch (Week 23-26)

Advanced Problem Patterns

Topics:

- Multi-threaded programming problems
- Concurrency and locks
- Design patterns in problems
- Object-oriented design problems
- Database query problems
- SQL in interviews
- Machine learning basics for interviews
- Security-related problems
- Cryptography basics
- Blockchain basic problems
- Mobile-specific problems
- Real-time system problems

Projects built:

- Thread-safe data structures
- Distributed lock manager
- Real-time chat system

Practice: Solve 30 advanced mixed problems

Interview Week Simulation

Topics:

- Full day interview simulation
- Multiple rounds back-to-back
- Phone screen practice
- Technical rounds practice

- System design rounds
- Behavioral rounds
- Hiring manager rounds
- Lunch interview practice
- Stress management techniques
- Energy management
- Recovery between rounds
- End-of-day strategies

Projects built:

- Complete interview portfolio
- Problem solution database
- Personal story bank

Practice: 5 full interview simulations

Soft Skills & Communication

Topics:

- Technical communication skills
- Explaining complex concepts simply
- Asking clarifying questions
- Requirements gathering
- Trade-off discussions
- Collaborative problem solving
- Handling hints effectively
- Dealing with stuck situations
- Time management during interviews
- Body language and confidence
- Virtual interview best practices
- Follow-up communication

Projects built:

- Technical presentation
- Code review exercise
- Team project simulation

Practice: Record and review 10 problem explanations

Career Strategy & Negotiation

Topics:

- Job search strategies
- Resume optimization for ATS
- LinkedIn profile optimization
- Cover letter writing
- Portfolio website creation
- GitHub profile enhancement
- Networking strategies
- Referral seeking techniques
- Offer evaluation framework
- Salary negotiation tactics
- Benefits comparison
- Career growth planning

Projects built:

- Professional portfolio website
- Optimized resume versions
- LinkedIn article publishing

Practice: Apply to 20 positions strategically

Final Week & Graduation

Topics:

- Final assessment preparation
- Comprehensive review
- Weak area strengthening
- Success stories sharing
- Alumni network introduction
- Continued learning resources
- Community building
- Mentorship opportunities
- Interview scheduling tips
- First day preparation
- Long-term career planning
- Celebration and reflection

Projects built:

- FINAL CAPSTONE PRESENTATION
- Complete DSA learning platform showcase
- 500+ problems solved portfolio

Assessment: Final Certification Exam - Comprehensive DSA assessment

Bonus Content: Specialized Topics (Additional Resources)

Machine Learning Algorithms

Topics:

- ML algorithms basics
- K-means clustering
- KNN implementation
- Decision trees basics
- Linear regression

- Gradient descent
- Neural network basics
- Feature engineering
- Cross-validation
- Confusion matrix
- Precision and recall
- ML system design

Projects built:

- Simple ML classifier
- Recommendation system
- Anomaly detection

Practice: Implement 10 ML algorithms

Parallel & Distributed Algorithms

Topics:

- Parallel algorithm design
- MapReduce paradigm
- Distributed sorting
- Consensus algorithms
- Leader election
- Distributed hash tables
- Consistent hashing implementation
- Gossip protocols
- Vector clocks
- CAP theorem deep dive
- Distributed transactions
- Two-phase commit

Projects built:

- MapReduce framework
- Distributed cache
- Consensus protocol

Practice: Design 5 distributed systems

Security & Cryptography

Topics:

- Cryptographic hash functions
- SHA implementation
- Public key cryptography basics
- RSA algorithm basics
- Digital signatures
- Merkle trees
- Blockchain fundamentals
- Security in algorithms
- Side-channel attacks
- Secure coding practices
- OWASP top 10
- Security interview questions

Projects built:

- Simple blockchain
- Password manager
- Encryption tool

Practice: Implement 5 security algorithms

Quantum Computing Basics

Topics:

- Quantum computing introduction
- Qubits and superposition
- Quantum gates
- Shor's algorithm overview
- Grover's algorithm basics
- Quantum complexity classes
- BQP vs NP
- Quantum supremacy
- Current limitations
- Future of quantum algorithms
- Quantum-resistant cryptography
- Learning resources

Projects built:

- Quantum circuit simulator
- Grover's algorithm demo
- Quantum concepts visualizer

Practice: Explore 5 quantum algorithms

Research & Innovation

Topics:

- Reading research papers
- Latest algorithmic advances
- Contributing to research
- Open problems in CS
- P vs NP introduction
- Approximation algorithms
- Randomized algorithms advanced
- Online algorithms

- Streaming algorithms
- External memory algorithms
- Cache-oblivious algorithms
- Future trends

Projects built:

- Research paper implementation
- Novel algorithm proposal
- Optimization improvement

Practice: Read and implement 3 research papers

CONTINUOUS LEARNING: BEYOND THE COURSE

Lifetime learning resources and advanced pathways

Competitive Programming Mastery (Ongoing)

Codeforces & TopCoder

Topics:

- Rating improvement strategies
- Div 1 problem approaches
- Speed coding techniques
- Virtual contest practice
- Editorial understanding
- Solution optimization
- Test case debugging
- Hacks and challenges
- Community interaction
- Becoming red coder
- TopCoder SRM strategies
- Marathon matches

Projects built:

- Reach Expert on Codeforces
- TopCoder rating goals
- Solution archive

Practice: Daily practice routine

Google Competitions

Topics:

- Google Code Jam preparation
- Kick Start strategies
- Hash Code team formation
- Problem analysis techniques
- Optimization strategies
- Large dataset handling
- Distributed Code Jam
- Time management
- Solution verification
- Post-contest analysis
- Learning from top solutions
- Building consistency

Projects built:

- Code Jam qualification
- Kick Start ranking improvement
- Hash Code team project

Practice: Past years' problems

ICPC Preparation

Topics:

- Team formation strategies
- Role distribution
- Problem categorization
- Speed solving techniques
- Team communication
- Contest strategy
- Regional preparation
- World Finals level problems
- Geometry problems
- Number theory advanced
- String processing advanced
- DP optimization techniques

Projects built:

- Team practice sessions
- Mock ICPC contests
- Solution notebook

Practice: Weekly team contests

Open Source Contributions

Topics:

- Finding algorithm projects
- Understanding large codebases
- Contributing guidelines
- Pull request best practices

- Code review participation
- Documentation improvements
- Bug fixes and features
- Test case contributions
- Performance improvements
- Algorithm library development
- Maintaining projects
- Building reputation

Projects built:

- Contribute to 5 projects
- Start algorithm library
- Maintain DSA resource

Practice: Monthly contributions

Teaching & Mentoring

Topics:

- Creating educational content
- YouTube channel strategies
- Blog writing for algorithms
- Course creation basics
- Mentoring beginners
- Code review skills
- Problem explanation techniques
- Visual content creation
- Building community
- Monetization options
- Building authority
- Giving back to community

Projects built:

- Algorithm blog/channel
- Mentorship program
- Educational platform

Practice: Weekly content creation

Industry Applications (Specialization Paths)

Algorithms in Production

Topics:

- Real-world optimizations
- A/B testing algorithms
- Performance monitoring
- Algorithm metrics
- Debugging production issues
- Scaling algorithms
- Caching strategies
- Database query optimization
- API algorithm design
- Microservices algorithms
- Event-driven algorithms
- Real-time processing

Projects built:

- Production algorithm suite
- Performance dashboard
- Optimization toolkit

Practice: Optimize 10 production systems

Research & Development

Topics:

- PhD opportunities
- Research areas in algorithms
- Publishing papers
- Conference participation
- Grant writing basics
- Collaboration strategies
- Patent considerations
- Industry R&D roles
- Innovation frameworks
- Emerging technologies
- Quantum algorithms future
- Career in research

Projects built:

- Research proposal
- Paper submission
- Patent application

Practice: Research contribution

Entrepreneurship Path

Topics:

- Algorithm-based startups
- Identifying opportunities
- MVP development
- Technical co-founder role
- Funding strategies

- Patent strategies
- Building tech team
- Scaling challenges
- Algorithm as service
- Consulting opportunities
- Product development
- Exit strategies

Projects built:

- Startup idea validation
- MVP development
- Business plan

Practice: Build startup prototype

Specialized Domains

Topics:

- Bioinformatics algorithms
- Financial algorithms
- Gaming algorithms
- Robotics algorithms
- Computer vision algorithms
- NLP algorithms
- Recommendation systems
- Search engine algorithms
- Blockchain algorithms
- IoT algorithms
- Edge computing algorithms
- Domain expertise building

Projects built:

- Domain-specific project
- Industry collaboration
- Specialized certification

Practice: Explore 3 domains deeply

Lifetime Learning Plan

Topics:

- Staying updated
- Annual learning goals
- Skill assessment methods
- New language learning
- Framework exploration
- Conference attendance
- Network building
- Mentorship circles
- Personal branding
- Knowledge sharing
- Community leadership
- Legacy building

Projects built:

- 5-year learning roadmap
- Annual review system
- Knowledge repository

Practice: Continuous improvement

Resources & Community (Lifetime Access)

Learning Resources

Topics:

- Book recommendations
- Online course platforms
- YouTube channels
- Podcasts for algorithms
- Research paper sources
- Practice platforms
- Visualization tools
- IDE and tools setup
- Chrome extensions
- Mobile apps for practice
- Cheat sheets and notes
- Reference materials

Projects built:

- Personal resource library
- Tool configuration
- Study material organization

Practice: Resource curation

Community Building

Topics:

- Discord/Slack communities
- Local meetup groups
- Online study groups
- Accountability partners
- Code review groups
- Mock interview circles

- Competitive programming teams
- Open source communities
- Alumni networks
- Professional associations
- Conference communities
- Social media presence

Projects built:

- Join 5 communities
- Start local chapter
- Organize meetup

Practice: Active participation

Success Metrics

Topics:

- Progress tracking methods
- Skill assessment tools
- Interview success rate
- Problem-solving speed
- Contest rankings
- GitHub contributions
- Teaching effectiveness
- Career progression
- Salary benchmarks
- Personal satisfaction
- Impact measurement
- Goal achievement

Projects built:

- Personal dashboard
- Progress tracker
- Achievement portfolio

Practice: Monthly assessments

Alumni Network

Topics:

- Alumni benefits
- Networking events
- Job referrals
- Mentorship program
- Guest lectures
- Industry connections
- Startup collaborations
- Research partnerships
- Continuous education
- Exclusive resources
- Community projects
- Giving back

Projects built:

- Alumni profile
- Contribution plan
- Network expansion

Practice: Stay connected

Final Resources (Quick Reference)

Interview Cheat Sheets

Topics:

- Time complexity table
- Space complexity reference
- Common patterns list
- Problem approach templates
- Edge cases checklist
- Testing strategies
- Communication tips
- Behavioral answers
- System design checklist
- Company research template
- Day-of interview tips
- Post-interview actions

Projects built:

- Personal cheat sheet
- Interview toolkit
- Quick reference guide

Problem Categories

Topics:

- Array problems list
- String problems list
- Tree problems list
- Graph problems list
- DP problems list
- Heap problems list
- Greedy problems list
- Backtracking list

- Bit manipulation list
- Math problems list
- Design problems list
- SQL problems list

Deliverables:

- Categorized problem bank
- Solution templates
- Pattern reference

Career Milestones

Topics:

- First job preparation
- 3-month goals
- 6-month targets
- 1-year objectives
- Promotion criteria
- Skill development
- Leadership growth
- Technical expertise
- Domain knowledge
- Compensation growth
- Career pivots
- Long-term vision

Deliverables:

- Career roadmap
- Milestone tracker
- Success journal

Assessment: LIFETIME ACCESS - Continuous support and updates

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Data Structure Library, Algorithm Visualizer, Complexity Analyzer
- Phase 2: Graph Algorithms Suite, DP Problem Solver, String Processor
- Phase 3: LeetCode Clone, Interview Prep Platform, Contest Simulator
- Final: Complete DSA Learning Platform with 500+ Problems

Total projects: 15+ major projects and 100+ mini implementations

SKILLS THE STUDENT WILL HAVE AT THE END

- Data Structures: Arrays, Linked Lists, Trees, Graphs, Heaps, Tries, Advanced Trees
- Algorithms: Sorting, Searching, Graph Algorithms, DP, Greedy, Backtracking
- Techniques: Two Pointers, Sliding Window, Fast-Slow, Divide & Conquer
- Analysis: Time/Space Complexity, Amortized Analysis, Proof Techniques
- Problem Solving: Pattern Recognition, Edge Cases, Optimization
- Interview Skills: Communication, System Design, Behavioral Responses
- Competitive Programming: Contest Strategies, Speed Coding, Debugging
- Languages: Primary (Python/C++/Java), Secondary (JavaScript)
- Tools: LeetCode, HackerRank, Codeforces, Git, VS Code, Debuggers

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-6 hours
 Problem solving: 8-10 hours
 Implementation: 3-4 hours
 Mock interviews: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving sessions
 Mentorship: 1-on-1 guidance from FAANG engineers
 Community: Active Discord with 24/7 support
 Code review: Solution review and optimization tips
 Mock interviews: Bi-weekly mock interview practice
 Career support: Resume review and referrals
 Lifetime access: All content and future updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Final certificate: DSA Mastery & Interview Ready Certification
 LinkedIn badge: Verifiable LinkedIn credential
 Industry recognized: Recognized by tech companies
 Portfolio projects: 500+ solved problems portfolio

WHERE THIS CAN LEAD

- Software Development Engineer (SDE)
- Full Stack Developer

- Backend Engineer
- Frontend Engineer
- Mobile Developer
- DevOps Engineer
- Machine Learning Engineer
- Competitive Programmer
- Technical Interview Coach
- Algorithm Researcher
- System Architect
- Technical Lead

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Ready for Rs 5-10 LPA roles; Ready for Rs 10-20 LPA roles

Growing roles: Ready for Rs 15-40+ LPA roles

Other roles: Rs 25-60+ LPA packages

International: \$100k-250k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What are the prerequisites for this DSA course?

A: You need basic programming ability: comfort with loops, conditions and functions in at least one language. The course is not meant for absolute non-programmers; DSA builds on top of programming basics, so learn those first and this course will pay off properly.

Q: Which programming language is the course taught in?

A: You can work in Python, C++ or Java, the three primary languages we support, with JavaScript as a secondary option. The concepts are language independent, and your mentor reviews solutions in whichever language you choose, so pick the one you want to interview in.

Q: How many problems will I solve, and from where?

A: The 6-month track works through 500+ curated problems, moving from arrays, strings and linked lists to trees, graphs and dynamic programming. Practice draws on LeetCode style interview questions, and every week mixes taught concepts with timed problem solving.

Q: Will this actually prepare me for FAANG style interviews?

A: It covers exactly what those interviews test: complexity analysis, core data structures, algorithm patterns and mock interview practice in the final phase. What we will not do is promise you an offer; interview outcomes depend on your practice volume and consistency. Students who complete the problem sets walk into coding rounds having seen the patterns before.

Q: Why not just follow a free DSA playlist?

A: Playlists explain solutions after the fact; they cannot watch you get stuck. A live mentor sees your approach in real time, asks the guiding questions an interviewer would, reviews your code for edge cases and complexity, and keeps you accountable through the 6 months. Most playlist learners stall at dynamic programming; guided practice is how you get through it.

Q: Can I take a trial class first?

A: Yes. Book a free live demo at learn.modernagecoders.com/contact, attend a real session and only then decide; there is no commitment attached. Questions before booking can go to WhatsApp at +91 91233 66161.

Q: How do the classes work?

A: Every session is a live 1-hour online class with an instructor, never a recording. You solve problems during class and get your reasoning corrected on the spot, and evening plus weekend slots are available so the course fits around college hours and internship schedules.

Q: Do I get anything to show at the end?

A: A completion certificate, plus a GitHub-ready record of your solved problems and implementations and mock interview experience. In technical hiring your portfolio and your performance under a timer count for more than the certificate, and this course builds all three.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/data-structures-algorithms-masterclass-college>

Course link: <https://learn.modernagecoders.com/courses/problem-solving-dsa-masterclass-teens>
Subject area: Computer Science & Problem Solving
Designed for: Teens
Who can join: Complete Beginner to Competition Ready
Course length: 12 months (52 weeks)
Time needed: 10-15 hours/week recommended
Certificate: Problem Solving Expert Certification with Portfolio

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

One-year DSA and problem-solving course for teens: master data structures, algorithms and logical thinking, from zero to cracking any coding interview.

In one line: From 'What is Programming?' to Winning Coding Competitions

ABOUT THE PROGRAMME

This isn't just another DSA course, it's a complete transformation journey designed specifically for teenage minds. Whether you're a high school student dreaming of top tech companies, preparing for programming olympiads, or just curious about problem-solving, this 1-year masterclass will turn you into a coding problem-solving machine. You'll develop superhuman logical thinking abilities, master every important data structure and algorithm, solve 1000+ problems across difficulty levels, and be ready for any coding challenge, from school competitions to Google interviews. By the end, you'll think like a computer scientist and solve problems that seemed impossible before.

WHAT MAKES THIS COURSE DIFFERENT

- Designed specifically for teenage learning patterns
- Gamified learning with points, badges, and leaderboards
- Visual explanations and animations for every concept
- Real-world problem connections teens can relate to
- From absolute zero to competition level in 12 months
- Focus on building intuition, not memorization
- Peer learning groups and coding battles
- Direct preparation for USACO, IOI, and top competitions

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Programming Basics, Logic Building, Simple Problem Solving
Phase 2: Core DSA (Months 4-6): Essential Data Structures, Searching, Sorting, Basic Algorithms
Phase 3: Advanced Mastery (Months 7-9): Trees, Graphs, Dynamic Programming, Advanced Algorithms
Phase 4: Competition Level (Months 10-12): Competitive Programming, Interview Prep, System Design Basics

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic mathematics (middle school level)
Coding experience: None required - absolute beginners welcome
Equipment: Computer/laptop with internet
Time commitment: 10-15 hours per week
Age: 13-19 years (designed for teenagers)
Motivation: Curiosity and dedication to learn

WHO THIS COURSE IS FOR

High school students: Preparing for programming competitions and college
Competition aspirants: USACO, IOI, Google Code Jam participants
College prep: Students aiming for top CS programs
Self learners: Motivated teens wanting to excel in programming
Math enthusiasts: Students who enjoy logical puzzles
Future engineers: Aspiring software engineers and researchers
Gamers: Teens interested in game development and AI

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & LOGIC BUILDING (MONTHS 1-3, WEEKS 1-13) -----

Build unshakeable foundations. Learn to think like a programmer, master basic syntax, and solve your first 200+ problems.

Month 1: Programming Fundamentals & First Steps (Week 1-4)

Introduction to Problem Solving & Setup

Topics:

- What is problem solving? Why it matters for your future
- How computers think: Binary and logic gates
- Introduction to algorithms in daily life
- Setting up your coding environment (VS Code)
- Introduction to Python: Why Python for beginners
- Your first program: Hello World explained
- Variables and data types: int, float, string, boolean
- Input and output operations
- Comments and code readability
- Basic arithmetic operations
- Type conversion and casting
- Problem solving methodology: UMPIRE (Understand, Match, Plan, Implement, Review, Evaluate)

Projects built:

- Calculator for homework problems
- Age calculator with fun facts
- Simple chatbot that responds to inputs

Practice: Daily: 10 basic problems on HackerRank Easy

Conditional Logic & Decision Making

Topics:

- If-else statements: Making decisions in code
- Comparison operators: ==, !=, <, >, <=, >=
- Logical operators: and, or, not
- Nested conditions and elif chains
- Switch case alternatives in Python
- Ternary operators for clean code
- Common patterns: min/max, odd/even, divisibility
- Edge cases and boundary conditions
- Debugging techniques with print statements
- Flowcharts for visualizing logic
- Truth tables and boolean algebra basics
- Real-world applications: Game logic, grading systems

Projects built:

- Grade calculator with letter grades
- Rock-Paper-Scissors game
- Simple adventure game with choices
- Password strength checker

Practice: Solve 30 conditional logic problems on CodeChef

Loops & Iteration Mastery

Topics:

- For loops: Counting and ranges
- While loops: Conditional repetition
- Loop control: break and continue
- Nested loops and patterns
- Common patterns: Triangles, pyramids, diamonds
- Iteration through sequences
- Loop optimization techniques
- Infinite loops and how to avoid them
- Do-while concept using Python
- Enumerate and range functions
- List comprehensions introduction
- Time complexity introduction: Big O notation basics

Projects built:

- Pattern printer (20+ patterns)
- Multiplication table generator
- Prime number checker and generator
- Number guessing game with attempts

Practice: Complete 40 loop problems on LeetCode Easy

Functions & Modular Programming

Topics:

- Functions: Why break code into pieces?
- Function definition and calling
- Parameters vs arguments
- Return values and void functions
- Scope: Local vs global variables
- Default parameters and keyword arguments
- Recursive thinking introduction
- Simple recursion: Factorial, Fibonacci
- Helper functions and code organization
- Lambda functions basics
- Function documentation and docstrings
- Testing functions with assert statements

Projects built:

- Math helper library (GCD, LCM, factorial)
- Text analyzer (word count, character frequency)
- Recursive art generator

- Unit converter with functions
- Practice: Build 25 reusable functions for common tasks

Months 2-3: Data Structures Basics & Problem Patterns (Week 5-13)

Lists/Arrays Fundamentals

Topics:

- Lists in Python: Dynamic arrays explained
- Creating and initializing lists
- Indexing and slicing operations
- List methods: append, insert, remove, pop
- List iteration patterns
- 2D lists (matrices) basics
- List comprehensions deep dive
- Common operations: reverse, sort, search
- Memory and lists: References vs copies
- Shallow vs deep copy
- Time complexity of list operations
- When lists are not optimal

Projects built:

- Student gradebook system
- Tic-tac-toe game
- Matrix operations calculator
- Todo list manager

Practice: Solve 50 array problems on various platforms

Strings & Text Processing

Topics:

- Strings as character arrays
- String immutability in Python
- String methods: split, join, strip, replace
- String formatting techniques
- Regular expressions basics
- Pattern matching in strings
- Palindromes and anagrams
- String manipulation algorithms
- ASCII values and character arithmetic
- Unicode and encoding basics
- String building efficiency
- Common string problem patterns

Projects built:

- Word game (Hangman/Wordle clone)
- Text encryption/decryption tool
- Palindrome checker with multiple rules
- String parser for math expressions

Practice: Master 40 string manipulation problems

Dictionaries/Hash Maps & Sets

Topics:

- Dictionary/HashMap concept
- Key-value pairs and hashing basics
- Dictionary operations: add, update, delete
- Iterating through dictionaries
- Nested dictionaries
- Sets and set operations
- Union, intersection, difference
- When to use sets vs lists
- Frequency counting patterns
- Two-pointer technique introduction
- Sliding window basics
- Counter and defaultdict

Projects built:

- Phone book application
- Word frequency analyzer
- Inventory management system
- Duplicate finder tool

Practice: Solve 35 hashmap/set problems

Basic Algorithms & Complexity

Topics:

- Algorithm analysis: Time and space
- Big O notation deep dive
- Best, average, worst case analysis
- Linear search implementation
- Binary search on sorted arrays
- Bubble sort understanding
- Selection sort algorithm
- Insertion sort and when to use it
- Counting sort for specific cases
- Algorithm visualization techniques
- Debugging complex algorithms
- Introduction to greedy algorithms

Projects built:

- Sorting visualizer with animations
- Search engine for local files
- Algorithm complexity analyzer
- Custom sorting for specific data

Practice: Implement and optimize 10 basic algorithms

Month 2-3 Assessment & Review

Topics:

- Review all data structures covered
- Problem-solving strategies recap
- Code optimization techniques
- Common mistakes and how to avoid them
- Speed coding practice

Projects built:

- CAPSTONE: Build a mini programming judge system
- Create your own problem set with solutions
- Automated test case generator

Assessment: Solve 20 mixed problems in 3 hours

Month 3: Advanced Basics & Contest Preparation (Week 10-13)

Stack & Queue Mastery

Topics:

- Stack data structure and LIFO principle
- Stack implementation using list
- Stack applications: Parentheses matching
- Expression evaluation using stacks
- Queue data structure and FIFO principle
- Queue implementation methods
- Circular queue concept
- Deque (double-ended queue)
- Priority queue introduction
- Real-world applications of stacks/queues
- Monotonic stack patterns
- BFS preview with queues

Projects built:

- Browser history navigator
- Expression calculator with precedence
- Task scheduler with priorities
- Undo-redo functionality

Practice: Master 30 stack/queue problems

Recursion & Backtracking Basics

Topics:

- Recursion deep dive: How it really works
- Call stack and recursion visualization
- Base case and recursive case design
- Multiple recursion: Fibonacci trees
- Recursion vs iteration trade-offs
- Memoization introduction
- Backtracking paradigm explained
- N-Queens problem approach
- Sudoku solver logic
- Permutations and combinations
- Subset generation algorithms
- Pruning in backtracking

Projects built:

- Sudoku solver with GUI
- N-Queens visualizer
- Maze solver with path finding
- Permutation generator tool

Practice: Solve 25 recursion/backtracking problems

Mathematics for Programming

Topics:

- Number theory basics: Primes, factors
- GCD and LCM algorithms (Euclidean)
- Modular arithmetic and properties
- Fast exponentiation (Binary exponentiation)
- Sieve of Eratosthenes for primes
- Combinatorics: nCr , nPr calculations
- Probability basics in programming
- Geometric algorithms introduction
- Matrix operations and applications
- Number systems and conversions
- Bit manipulation preview
- Mathematical proof techniques

Projects built:

- Prime factorization tool
- Probability simulator
- Geometric calculator
- Cryptography basics implementation

Practice: Solve 40 math-based programming problems

Linked Lists Introduction

Topics:

- Linked list concept and need
- Node structure and pointers
- Singly linked list implementation
- Insertion: Beginning, end, middle
- Deletion operations
- Searching in linked lists
- Reversing a linked list
- Detecting cycles (Floyd's algorithm)
- Doubly linked lists
- Circular linked lists
- Linked list vs array comparison
- Common interview patterns

Projects built:

- Music playlist manager
- Train route simulator
- Memory manager simulation
- LRU cache basic implementation

Practice: Implement 20 linked list operations

Phase 1 Final Project

Topics:

- Project planning and design
- Algorithm selection strategies
- Code structure and organization
- Testing and debugging approaches
- Documentation best practices

Projects built:

- MAJOR PROJECT: Console-based puzzle game platform
- Include: Multiple game modes, scoring system, leaderboard
- Alternative: Algorithm visualizer web app
- Alternative: Competitive programming practice tool

Assessment: Phase 1 Comprehensive Exam - 25 problems, 4 hours

PHASE 2: CORE DATA STRUCTURES & ALGORITHMS (MONTHS 4-6, WEEKS 14-26)

Master essential data structures, dive deep into algorithms, and start solving medium-level competitive programming problems.

Months 4-5: Trees & Advanced Sorting (Week 14-22)

Binary Trees Fundamentals

Topics:

- Tree terminology: Root, leaf, height, depth
- Binary tree structure and properties
- Tree representation methods
- Tree traversals: Inorder, Preorder, Postorder
- Level order traversal (BFS on trees)
- Recursive tree algorithms
- Tree construction from traversals
- Binary tree properties and validation
- Complete and perfect binary trees
- Tree serialization and deserialization
- Common tree patterns
- Parent pointers and their uses

Projects built:

- Family tree visualizer
- Expression tree evaluator
- Directory structure navigator
- Tree-based calculator

Practice: Solve 40 binary tree problems

Binary Search Trees (BST)

Topics:

- BST property and invariants
- Insertion in BST
- Searching efficiently in BST
- Deletion cases in BST
- Inorder predecessor and successor
- Minimum and maximum finding
- BST validation algorithms
- Balanced vs unbalanced BSTs
- AVL trees introduction
- Red-Black trees concept
- Self-balancing importance
- BST to sorted array conversions

Projects built:

- Dictionary with BST backend
- Autocomplete system
- Range query processor

- Database index simulator

Practice: Master 35 BST problems

Advanced Sorting Algorithms

Topics:

- Merge sort: Divide and conquer
- Quick sort and partitioning
- Heap sort introduction
- Counting sort deep dive
- Radix sort for integers
- Bucket sort applications
- Sorting algorithm comparison
- Stability in sorting
- External sorting basics
- Partial sorting techniques
- Custom comparators
- Sorting in linear time

Projects built:

- Sorting algorithm race visualizer
- Large file sorter
- Leaderboard system
- Multi-criteria sorting tool

Practice: Implement and optimize 15 sorting variations

Heaps & Priority Queues

Topics:

- Heap data structure concept
- Max heap and min heap properties
- Heap implementation with arrays
- Heapify operation (sift up/down)
- Building heap from array
- Priority queue operations
- Heap sort algorithm
- K-th largest/smallest problems
- Median finding with two heaps
- Merge K sorted arrays
- Dijkstra's algorithm preview
- Custom priority functions

Projects built:

- Task scheduler with priorities
- Emergency room triage system
- Event simulation system
- Real-time median tracker

Practice: Solve 30 heap/priority queue problems

Tries & String Algorithms

Topics:

- Trie data structure motivation
- Trie node structure
- Insert and search in trie
- Prefix search and autocomplete
- Word dictionary with wildcards
- Longest common prefix
- String matching algorithms
- KMP algorithm basics
- Rabin-Karp rolling hash
- Z-algorithm introduction
- Suffix arrays concept
- Applications in text processing

Projects built:

- Spell checker implementation
- T9 predictive text
- Word search puzzle solver
- DNA sequence matcher

Practice: Build 20 trie-based solutions

Month 6: Graphs & Network Algorithms (Week 23-26)

Graph Fundamentals

Topics:

- Graph theory introduction
- Graph terminology: Vertices, edges, degree
- Directed vs undirected graphs
- Weighted vs unweighted graphs
- Graph representations: Adjacency matrix
- Adjacency list representation
- Edge list representation
- Graph properties: Connected, cyclic
- Special graphs: Trees, DAGs, complete
- Bipartite graphs
- Graph coloring basics
- Real-world graph applications

Projects built:

- Social network analyzer
- City map navigator
- Game state explorer
- Dependency resolver

Practice: Model 15 real-world problems as graphs

Graph Traversal Algorithms

Topics:

- Depth-First Search (DFS) deep dive
- DFS applications: Path finding, cycle detection
- Breadth-First Search (BFS) mastery
- BFS applications: Shortest path, level order
- Connected components finding
- Topological sorting with DFS
- Kahn's algorithm for topological sort
- Detecting cycles in directed graphs
- Strongly connected components intro
- Bridge finding in graphs
- Articulation points
- Graph traversal optimizations

Projects built:

- Maze generator and solver
- Course prerequisite validator
- Island counter in grid
- Web crawler simulator

Practice: Solve 40 graph traversal problems

Shortest Path Algorithms

Topics:

- Shortest path problem variants
- Dijkstra's algorithm in detail
- Implementation with priority queue
- Bellman-Ford for negative weights
- Floyd-Warshall all-pairs shortest path
- A* search algorithm basics
- Shortest path in unweighted graphs
- Path reconstruction techniques
- Single source vs all-pairs
- Handling negative cycles
- Applications in real world
- Optimizations and variations

Projects built:

- GPS navigation system
- Network routing simulator
- Game pathfinding AI
- Flight connection finder

Practice: Implement 5 shortest path variations

Minimum Spanning Trees

Topics:

- Spanning tree concept
- Minimum spanning tree (MST) problem
- Kruskal's algorithm with union-find
- Prim's algorithm implementation
- Union-Find data structure deep dive
- Path compression optimization
- Union by rank optimization
- MST uniqueness
- Maximum spanning tree
- Applications: Network design, clustering
- Steiner tree introduction
- Online MST algorithms

Projects built:

- Network cable optimizer
- Cluster analysis tool
- Maze generator using MST
- Circuit board router

Practice: Solve 20 MST and Union-Find problems

Advanced Graph Algorithms

Topics:

- Network flow introduction
- Maximum flow problem
- Ford-Fulkerson method basics
- Bipartite matching
- Graph coloring algorithms
- Eulerian path and circuit
- Hamiltonian path problem
- Traveling salesman introduction
- Planar graphs
- Graph isomorphism
- Random graph algorithms
- Graph algorithm applications

Projects built:

- Maximum flow visualizer
- Job assignment system
- Tournament scheduler
- Graph algorithm library

Practice: Explore 15 advanced graph problems

Phase 2 Completion & Integration (Week 25-26)

Algorithm Design Paradigms

Topics:

- Brute force and when it's okay
- Greedy algorithm design
- Proving greedy correctness
- Divide and conquer mastery
- Transform and conquer
- Decrease and conquer
- Problem reduction techniques
- Algorithm paradigm selection
- Hybrid approaches
- Approximation algorithms intro
- Randomized algorithms basics
- Online vs offline algorithms

Projects built:

- Algorithm strategy selector
- Greedy algorithm visualizer
- Problem classifier tool
- Algorithm comparison framework

Practice: Apply different paradigms to 30 problems

Space & Time Optimization

Topics:

- Space-time tradeoffs
- Memory optimization techniques
- Cache-friendly algorithms
- Bit manipulation for space saving
- Rolling arrays technique
- In-place algorithms
- Stream processing algorithms
- External memory algorithms
- Parallel algorithm basics
- MapReduce introduction
- Distributed algorithms overview
- Quantum algorithms preview

Projects built:

- Memory-efficient data processor
- Stream analytics engine
- Large data sorter
- Optimization analyzer tool

Practice: Optimize 20 solutions for space/time

Testing & Debugging Mastery

Topics:

- Test case design strategies
- Edge case identification
- Stress testing techniques
- Random test generation
- Debugging complex algorithms
- Using debuggers effectively
- Print debugging strategies
- Assertion-based debugging
- Performance profiling
- Memory leak detection
- Code coverage analysis
- Continuous testing practices

Projects built:

- Test case generator framework
- Algorithm debugger tool
- Performance profiler
- Bug tracker system

Practice: Debug 30 algorithmic bugs

Competition Strategies

Topics:

- Contest strategy and time management
- Problem selection in contests
- Speed coding techniques
- Template preparation
- Common contest patterns
- Interactive problem solving
- Partial scoring strategies
- Team contest coordination
- Virtual contest practice

- Post-contest analysis
- Rating improvement tips
- Mental preparation for contests

Projects built:

- Personal contest tracker
- Problem recommendation system
- Contest simulator
- Performance analytics dashboard

Practice: Participate in 10 virtual contests

Phase 2 Capstone Project

Topics:

- Complex system design
- Algorithm integration
- Performance requirements
- Scalability considerations
- Code organization for large projects

Projects built:

- MAJOR CAPSTONE: Competitive Programming Platform
- Features: Problem bank, online judge, contest system, leaderboards
- Alternative: Graph algorithm visualizer suite
- Alternative: AI game bot using algorithms

Assessment: Phase 2 Final - Solve 10 medium problems in 3 hours

----- PHASE 3: ADVANCED ALGORITHMS & TECHNIQUES (MONTHS 7-9, WEEKS 27-39) -----

Master dynamic programming, advanced data structures, computational geometry, and complex problem-solving techniques.

Months 7-8: Dynamic Programming Mastery (Week 27-35)

DP Fundamentals

Topics:

- Dynamic programming introduction
- Overlapping subproblems concept
- Optimal substructure property
- Memoization (top-down approach)
- Tabulation (bottom-up approach)
- Space optimization in DP
- 1D DP problems: Fibonacci, climbing stairs
- Classic problems: Coin change, rod cutting
- DP state definition
- Recurrence relation formulation
- Base cases identification
- DP vs greedy vs divide-conquer

Projects built:

- DP problem solver framework
- Fibonacci calculator (all methods)
- Optimal game strategy simulator
- Resource allocation optimizer

Practice: Solve 50 basic DP problems

String DP Problems

Topics:

- Longest Common Subsequence (LCS)
- Longest Common Substring
- Edit distance (Levenshtein distance)
- Longest Palindromic Subsequence
- Longest Palindromic Substring
- String interleaving
- Wildcard pattern matching
- Regular expression matching
- Distinct subsequences count
- String compression DP
- Word break problems
- Palindrome partitioning

Projects built:

- Diff tool implementation
- DNA sequence aligner
- Spell checker with corrections
- Text similarity analyzer

Practice: Master 30 string DP problems

Grid & Matrix DP

Topics:

- Grid traversal problems
- Unique paths counting
- Minimum path sum
- Maximum path sum variants
- Dungeon game type problems
- Cherry pickup problem
- Matrix chain multiplication

- Maximal rectangle in binary matrix
- Largest square in matrix
- 2D range sum queries
- DP on grids with obstacles
- Multi-dimensional DP

Projects built:

- Pathfinding game with obstacles
- Matrix optimizer tool
- Grid-based puzzle solver
- 2D resource collector game

Practice: Solve 35 grid DP problems

Knapsack & Subset Problems

Topics:

- 0/1 Knapsack problem
- Unbounded knapsack
- Fractional knapsack (greedy)
- Subset sum problem
- Partition equal subset sum
- Target sum with +/- operators
- Minimum subset sum difference
- Count of subset sum
- Coin change variations
- Rod cutting variations
- Job scheduling with profits
- Knapsack with multiple constraints

Projects built:

- Backpack optimizer for games
- Budget allocation system
- Partition optimizer
- Investment portfolio selector

Practice: Master all knapsack variations

Tree DP

Topics:

- DP on trees concept
- Tree diameter using DP
- Maximum path sum in tree
- Tree balancing problems
- Subtree problems with DP
- Re-rooting technique
- Binary tree cameras problem
- Tree coloring with constraints
- Maximum independent set in tree
- Tree matching problems
- DP with tree traversal
- Optimizing tree recursions

Projects built:

- Tree optimization visualizer
- Network optimizer with DP
- Tree game strategies
- Hierarchical resource distributor

Practice: Solve 25 tree DP problems

Month 8: Advanced DP & Optimization (Week 32-35)

DP with Bitmasks

Topics:

- Bitmask representation basics
- Bit manipulation operations review
- DP with bitmask states
- Traveling Salesman with DP
- Hamiltonian path with bitmasks
- Assignment problems with masks
- Subset iteration with bitmasks
- Profile optimization technique
- Broken profile DP
- SOS (Sum over Subsets) DP
- Bitmask DP optimizations
- Memory optimization with masks

Projects built:

- TSP solver with visualization
- Task assignment optimizer
- Board game AI with bitmask DP
- Subset analyzer tool

Practice: Master 20 bitmask DP problems

Interval DP & Games

Topics:

- Interval DP pattern
- Matrix chain multiplication revisited
- Burst balloons problem
- Palindrome partitioning minimum cuts

- Optimal binary search tree
- Game theory basics: Nim game
- Minimax algorithm
- Alpha-beta pruning
- Sprague-Grundy theorem
- Combinatorial game theory
- DP in game strategies
- Winning and losing positions

Projects built:

- Game strategy analyzer
- Optimal BST builder
- Nim game variations player
- Two-player game framework

Practice: Solve 25 interval DP and game problems

DP Optimizations

Topics:

- Convex hull trick
- Divide and conquer optimization
- Knuth-Yao speedup
- Monotone queue optimization
- Slope trick introduction
- CHT (Convex Hull Trick) applications
- Li Chao tree basics
- Matrix exponentiation for DP
- DP on DAGs
- Rolling hash with DP
- Digit DP technique
- DP with data structures

Projects built:

- DP optimizer tool
- Performance comparison framework
- Convex hull visualizer
- Advanced DP solver

Practice: Apply optimizations to 20 DP problems

Probability & Expected Value DP

Topics:

- Probability basics review
- Expected value concept
- DP with probabilities
- Dice problems with DP
- Random walk problems
- Markov chains introduction
- Expected value optimization
- Monte Carlo methods basics
- Linearity of expectation
- Probabilistic algorithms
- DP in stochastic processes
- Applications in games and simulations

Projects built:

- Dice game simulator
- Probability calculator
- Random process analyzer
- Expected value optimizer

Practice: Solve 20 probability DP problems

State Space Reduction

Topics:

- State space explosion problem
- State compression techniques
- Symmetry reduction
- Equivalence class reduction
- Meet in the middle technique
- Bidirectional search
- State space pruning
- Heuristic state reduction
- Memory-efficient DP
- External memory DP
- Parallel DP basics
- DP state design patterns

Projects built:

- State space optimizer
- Memory-efficient solver
- Large-scale DP processor
- State reduction analyzer

Practice: Optimize 15 DP solutions for memory

Month 9: Computational Geometry & Number Theory (Week 36-39)

Computational Geometry Basics

Topics:

- Points, lines, and vectors

- Distance calculations
- Orientation of three points
- Collinearity check
- Line intersection detection
- Segment intersection
- Point in polygon test
- Convex hull algorithms
- Graham scan implementation
- Jarvis march (Gift wrapping)
- Area of polygon
- Closest pair of points

Projects built:

- Geometry calculator suite
- Convex hull visualizer
- Collision detection system
- Polygon area calculator

Practice: Solve 30 basic geometry problems

Advanced Geometry

Topics:

- Line sweep algorithms
- Rotating calipers technique
- Voronoi diagrams basics
- Delaunay triangulation
- Circle-line intersection
- Circle-circle intersection
- Tangents to circles
- 3D geometry basics
- Cross product and applications
- Dot product applications
- Geometric transformations
- Computational precision issues

Projects built:

- Art generator using geometry
- Physics simulation engine
- 3D renderer basics
- Geometric game mechanics

Practice: Implement 20 geometric algorithms

Advanced Number Theory

Topics:

- Extended Euclidean algorithm
- Modular multiplicative inverse
- Chinese Remainder Theorem
- Fermat's little theorem
- Euler's totient function
- Miller-Rabin primality test
- Pollard rho factorization
- Baby-step giant-step
- Discrete logarithm
- Quadratic residues
- Primitive roots
- Number theoretic transforms

Projects built:

- RSA encryption implementation
- Prime number toolkit
- Cryptographic hash function
- Number theory calculator

Practice: Solve 25 number theory problems

Combinatorics & Counting

Topics:

- Advanced permutation generation
- Combination with repetition
- Catalan numbers applications
- Stirling numbers
- Bell numbers
- Derangements
- Inclusion-exclusion principle
- Generating functions basics
- Recurrence solving techniques
- Burnside's lemma
- Polya enumeration
- Advanced counting techniques

Projects built:

- Combinatorics calculator
- Sequence generator tool
- Counting problem solver
- Pattern enumeration system

Practice: Master 20 combinatorics problems

Phase 3 Capstone Project

Topics:

- Complex algorithm integration
- Performance optimization
- Algorithm selection strategies
- Large-scale problem solving
- Code efficiency and elegance

Projects built:

- MAJOR CAPSTONE: Advanced Algorithm Visualizer Platform
- Features: DP, Geometry, Graphs, Interactive learning, Problem generator
- Alternative: Competitive Programming Bot
- Alternative: Mathematical Problem Solver

Assessment: Phase 3 Final - 8 hard problems in 4 hours

----- PHASE 4: COMPETITIVE PROGRAMMING & INTERVIEW MASTERY (MONTHS 10-12, WEEKS 40-52) -----

Master competitive programming, ace technical interviews, explore advanced topics, and build your problem-solving portfolio.

Months 10-11: Competitive Programming Excellence (Week 40-48)

Advanced Data Structures

Topics:

- Segment tree deep dive
- Lazy propagation in segment trees
- Fenwick tree (Binary Indexed Tree)
- 2D segment trees
- Persistent segment trees
- Merge sort tree
- Wavelet tree basics
- Sqrt decomposition
- Mo's algorithm
- Heavy-light decomposition
- Link-cut trees introduction
- Treap and randomized BSTs

Projects built:

- Range query processor
- Persistent data structure library
- Query optimization system
- Advanced tree operations toolkit

Practice: Master 40 advanced DS problems

String Algorithms Mastery

Topics:

- KMP algorithm complete implementation
- Z-algorithm and applications
- Rabin-Karp with rolling hash
- Aho-Corasick for multiple patterns
- Suffix array construction
- LCP array computation
- Suffix tree basics
- Manacher's algorithm
- String hashing techniques
- Suffix automaton introduction
- Palindromic tree
- Rope data structure

Projects built:

- Text editor with advanced search
- Plagiarism detector
- DNA pattern matcher
- Compressed string processor

Practice: Solve 35 advanced string problems

Network Flow & Matching

Topics:

- Maximum flow deep dive
- Ford-Fulkerson implementation
- Edmonds-Karp algorithm
- Dinic's algorithm
- Push-relabel algorithm
- Min-cost max-flow
- Maximum bipartite matching
- Hungarian algorithm
- Stable marriage problem
- Hall's theorem applications
- Circulation with demands
- Multi-commodity flow basics

Projects built:

- Assignment optimization system
- Network capacity planner
- Matching platform algorithm
- Flow network visualizer

Practice: Implement 20 flow algorithms

FFT & Polynomials

Topics:

- Polynomial representation
- Polynomial multiplication naive
- Karatsuba multiplication
- Fast Fourier Transform (FFT)
- Cooley-Tukey FFT algorithm
- Inverse FFT
- Number Theoretic Transform (NTT)
- Polynomial division
- Polynomial GCD
- Applications of FFT
- Convolution with FFT
- FFT optimization tricks

Projects built:

- Signal processor with FFT
- Polynomial calculator
- Audio frequency analyzer
- Large number multiplier

Practice: Solve 15 FFT/polynomial problems

Parallel & Distributed Algorithms

Topics:

- Parallel algorithm design
- PRAM model basics
- Parallel sorting algorithms
- Parallel graph algorithms
- MapReduce paradigm
- Distributed consensus
- Leader election algorithms
- Clock synchronization
- Distributed hash tables
- Blockchain algorithms basics
- Quantum algorithms introduction
- GPU algorithms overview

Projects built:

- Parallel sort implementation
- Distributed system simulator
- Consensus algorithm demo
- Parallel computation framework

Practice: Explore 10 parallel algorithms

Month 11: Competition Training & Practice (Week 44-48)

Codeforces & Contest Strategies

Topics:

- Codeforces platform mastery
- Rating system understanding
- Div 1, 2, 3 strategies
- Problem difficulty assessment
- Time allocation strategies
- When to skip problems
- Hacking phase tactics
- Virtual contest practice
- Upsolving importance
- Editorial analysis
- Performance tracking
- Common Codeforces patterns

Projects built:

- Contest performance tracker
- Problem difficulty predictor
- Solution template generator
- Contest analysis tool

Practice: Participate in 20 Codeforces contests

USACO & IOI Preparation

Topics:

- USACO divisions: Bronze to Platinum
- USACO problem patterns
- File I/O handling
- USACO-specific optimizations
- IOI-style problems
- Subtask scoring strategy
- Partial solutions value
- Test data analysis
- Output-only tasks
- Interactive problems mastery
- Debugging under pressure
- Camp and training resources

Projects built:

- USACO problem solver
- Training plan generator

- Mock USACO contest platform
- Solution verifier tool

Practice: Solve 50 USACO past problems

Google Competitions

Topics:

- Google Code Jam format
- Kickstart competition strategy
- Large input handling
- Small vs large dataset approach
- Hash Code team strategies
- Optimization problems approach
- Approximation strategies
- Google interview style problems
- System design in competitions
- Code readability importance
- Testing strategies for competitions
- Google-specific patterns

Projects built:

- Code Jam problem archive
- Optimization problem solver
- Team collaboration platform
- Solution scorer and ranker

Practice: Complete 30 Google competition problems

Advanced Contest Techniques

Topics:

- Interactive problem techniques
- Adaptive query strategies
- Randomization in solutions
- Heuristics and approximations
- Construction problems approach
- Output formatting tricks
- Fast I/O optimizations
- Precomputation strategies
- Space-time tradeoffs in contests
- Team contest coordination
- Communication in team contests
- Contest psychology and stress

Projects built:

- Interactive problem framework
- Team contest simulator
- Strategy optimizer
- Performance under pressure trainer

Practice: Master 25 advanced contest problems

Platform Mastery

Topics:

- LeetCode for interviews
- HackerRank certifications
- TopCoder SRM strategies
- AtCoder contests approach
- CodeChef long challenges
- Project Euler mathematics
- SPOJ classical problems
- Local olympiad preparation
- Online judge creation
- Problem setting basics
- Test case generation
- Solution validation

Projects built:

- Multi-platform tracker
- Problem aggregator
- Personal online judge
- Problem setter toolkit

Practice: Achieve high ratings on 5 platforms

Month 12: Interview Preparation & Career Launch (Week 49-52)

Technical Interview Mastery

Topics:

- Interview problem patterns
- FAANG interview process
- Behavioral questions preparation
- System design basics for teens
- Whiteboard coding practice
- Communication during interviews
- Clarifying questions importance
- Edge case discussion
- Optimization discussion
- Trade-off analysis
- Code review skills
- Mock interview strategies

Projects built:

- Interview preparation tracker
- Mock interview platform
- Solution explanation generator
- Interview feedback system

Practice: Complete 100 interview problems

System Design for Beginners

Topics:

- System design introduction
- Scalability basics
- Database design fundamentals
- API design principles
- Caching strategies
- Load balancing concepts
- Microservices basics
- Distributed systems introduction
- CAP theorem simplified
- Design patterns overview
- Real-world system analysis
- Drawing system diagrams

Projects built:

- URL shortener design
- Chat application design
- Social media feed design
- Simple search engine design

Practice: Design 10 simple systems

Open Source & Portfolio

Topics:

- GitHub profile optimization
- Open source contribution guide
- Finding first issues
- Pull request best practices
- Code documentation skills
- README writing guide
- Project presentation skills
- Technical blogging basics
- Building online presence
- Networking in tech community
- Hackathon participation
- Building developer portfolio

Projects built:

- GitHub portfolio website
- Open source contribution
- Technical blog setup
- Project showcase platform

Practice: Make 10 open source contributions

Advanced Topics Exploration

Topics:

- Machine learning basics for CP
- Quantum computing introduction
- Blockchain and cryptography
- Competitive programming in AI
- Research paper reading
- Advanced mathematics exploration
- Functional programming paradigms
- Logic programming basics
- Constraint programming
- Metaheuristics introduction
- Evolutionary algorithms
- Future of problem solving

Projects built:

- ML-powered problem solver
- Cryptography toolkit
- Research paper implementation
- Future tech exploration project

Practice: Explore 5 cutting-edge topics

Final Month: Mastery & Beyond (Week 50-52)

Final Project - Part 1

Topics:

- Project ideation and planning
- Requirements gathering
- Architecture design
- Technology selection
- Development methodology
- Testing strategy
- Documentation planning
- Timeline creation

Projects built:

- FINAL MASTERPIECE: Complete Problem Solving Platform
- Features: Online judge, problem bank, contests, tutorials, visualizations, AI hints, progress tracking, social features
- Alternative: AI-powered problem solver
- Alternative: Educational game for algorithms

Final Project - Part 2 & Polish

Topics:

- Implementation completion
- Testing and debugging
- Performance optimization
- User interface polish
- Documentation completion
- Deployment preparation
- Presentation preparation
- Demo video creation
- Code review and refactoring
- Security considerations
- Scalability planning
- Future roadmap

Deliverables:

- Complete source code repository
- Live deployed application
- Comprehensive documentation
- Video presentation
- Technical report
- Test suite with coverage
- Performance benchmarks
- User guide

Graduation & Career Launch

Topics:

- Portfolio finalization
- Resume optimization for tech
- LinkedIn profile setup
- College application prep (if applicable)
- Scholarship opportunities
- Internship search strategies
- Freelancing opportunities
- Competitive programming career
- Teaching and mentoring
- Research opportunities
- Entrepreneurship in tech
- Lifelong learning plan

Deliverables:

- Professional portfolio
- Optimized resume
- LinkedIn presence
- 100+ day coding streak
- Competition achievements
- Open source contributions
- Personal brand
- Future learning roadmap

Assessment: FINAL COMPREHENSIVE ASSESSMENT - Full competition simulation + interview

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Calculator, Games, Puzzle Solvers, Algorithm Visualizers
- Phase 2: Data Structure Libraries, Graph Visualizers, Sorting Animations
- Phase 3: DP Solver, Geometry Engine, Advanced Algorithm Implementations
- Phase 4: Competition Platform, AI Problem Solver, Complete DSA Library
- Final: Production-grade Problem Solving Platform

Total projects: 60+ projects from simple to complex systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Languages: Python (primary), C++ (competitive programming), Java (optional)
- Problem Solving: 1000+ problems solved across all difficulty levels
- Data Structures: Arrays, Lists, Stacks, Queues, Trees, Graphs, Heaps, Tries, Advanced DS
- Algorithms: Sorting, Searching, Graph Algorithms, DP, Greedy, Divide & Conquer
- Mathematics: Number Theory, Combinatorics, Probability, Geometry
- Advanced: Network Flow, String Algorithms, FFT, Segment Trees, Game Theory
- Competitive: Contest strategies, Speed coding, Optimization techniques
- Platforms: Codeforces, USACO, LeetCode, HackerRank, AtCoder mastery
- Tools: Git, VS Code, Debuggers, Profilers, Online Judges
- Soft Skills: Problem decomposition, Logical thinking, Pattern recognition, Communication

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-4 hours

Problem solving: 6-8 hours
Contests: 2-3 hours
Projects: 2-3 hours
Peer discussions: 1-2 hours
Total per week: 10-15 hours

SUPPORT INCLUDED

Live sessions: Weekly doubt clearing with mentors
Peer groups: Study groups of 4-5 students
Mentorship: Personal mentor for guidance
Community: Active Discord server for teens
Code review: Expert feedback on solutions
Competition support: Team formation and coaching
Parent updates: Monthly progress reports
Career guidance: College and internship counseling

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
Competition achievements: Competition participation certificates
Final certificate: Problem Solving Expert Certification
Skill badges: 50+ skill badges to earn
Linkedin credentials: Verifiable LinkedIn badges
Portfolio problems: 1000+ solved problems portfolio

WHERE THIS CAN LEAD

- Competitive Programmer (while in school/college)
- Software Engineering Intern at top companies
- Algorithm Researcher (with further education)
- Game Developer (logic and AI focus)
- Quantitative Analyst (with math focus)
- Machine Learning Engineer (with additional ML study)
- Startup Founder (technical co-founder)
- Programming Instructor/Mentor
- Open Source Contributor
- Technical Content Creator

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What exactly does this DSA course teach?

A: The year is a complete problem-solving ladder: programming fundamentals and logic building, core data structures like trees and graphs, dynamic programming, number theory and computational geometry, and finally competitive programming and interview-style practice. Students solve 1000+ problems of gradually increasing difficulty along the way.

Q: Which programming language is used?

A: All tools are free: Python, a free code editor, and free practice platforms like LeetCode and Codeforces.

Q: Does my teen need coding experience or advanced maths?

A: No coding experience is required; the first month teaches programming from scratch. Middle school level mathematics is enough to start, and the course itself builds the logical and mathematical maturity needed for harder problems.

Q: Is this useful if my teen is not aiming for olympiads?

A: Yes. Contests like Codeforces rounds and olympiad preparation are one path through the course, but the same skills power school computer science, entrance exams, and technical interviews later. Problem solving is the most transferable skill in all of programming.

Q: What comes after this course?

A: Graduates are ready for sustained competitive programming, school and college-level contests, and the algorithm rounds used in technical interviews. Many also branch into our Python, full stack, or AI courses, where strong problem-solving skills make advanced material dramatically easier.

Q: Can my teen attend a free demo first?

A: Yes, demos are free and commitment-free; book at learn.modernagecoders.com/contact. Parents can sit in on the session, and the teacher will happily discuss your teen's goals, whether that is olympiads, exams, or interview preparation.

Q: Are the sessions live?

A: Yes, all classes are live 1-hour online sessions with a real teacher, never recordings. Problem solving is taught interactively: students attempt problems in class, explain their reasoning, and learn multiple approaches from the group.

Q: Is a certificate awarded?

A: Yes, students who complete the course earn a Problem Solving Expert certificate along with a portfolio documenting the problems and contests they have worked through. That visible record of solved problems is often what impresses schools and interviewers most.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/problem-solving-dsa-masterclass-teens>

=====

COURSE 50 OF 118: EARLY MATH FOUNDATIONS: PLAYFUL, CONCRETE MATH (AGES 4-7)

=====

Course link: <https://learn.modernagecoders.com/courses/early-math-foundations>

Subject area: Elementary Mathematics

Designed for: Elementary school

Who can join: Ages 4-7 (kindergarten to grade 2), all starting points

Course length: 12 months (52 weeks)

Time needed: 2 live one-hour classes per week (structured as playful segments) + everyday math play at home

Certificate: Course completion certificate + stage mastery checks

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online early math course for ages 4-7: counting, number bonds, shapes and measurement taught concrete-first in short, joyful classes. Free demo first.

In one line: The years that decide whether math becomes a friend. We make it one.

ABOUT THE PROGRAMME

A 12-month live program for children aged 4-7, the years when a child's whole relationship with mathematics is quietly decided. Early math done right is not worksheets; it is counting that means something, quantities seen and handled, patterns spotted, shapes built and stories solved. Our mentors teach concrete-first in genuinely interactive classes structured as short playful segments that respect young attention spans: real objects, ten-frames, games and math talk, with the child doing and explaining rather than watching. Children leave this course fluent in early number, confident in their own thinking, and, the part parents notice most, actually liking math.

WHAT MAKES THIS COURSE DIFFERENT

- Concrete before abstract, always: quantities are handled, drawn, and only then written as symbols
- Classes built for young attention: an hour of varied segments, games, movement, building, story problems, never an hour of sitting still
- Counting taught with meaning: one-to-one correspondence, cardinality and the ideas most programs skip straight past
- Number bonds and part-whole thinking started early, gently, the foundation everything to age 18 stands on
- Math talk from day one: even five-year-olds explain their thinking here, in their own words
- One dedicated mentor who knows the child, their pace and their spark
- Parents equipped too: simple everyday math-play prompts, because kitchens teach more than worksheets
- The honest opposite of drill academies, no pressure, no rankings, no tears

THE LEARNING PATH, STEP BY STEP

Phase 1: Counting with meaning, comparing quantities, and the first part-whole ideas, through objects and games

Phase 2: Addition and subtraction as stories, bonds to 10, place value beginnings and the teen numbers demystified

Phase 3: Numbers to 100 and beyond, early measurement, shapes and data, plus problem-solving courage and grade-2 readiness

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Ages 4-7; no academic prerequisites whatsoever

Math background: None, this course IS the foundation

Equipment: Computer or tablet with stable internet; everyday objects (blocks, buttons, spoons) as materials

Time commitment: 2 live hours plus everyday math play

English: Course taught in English

Parent role: Nearby for the youngest children in early weeks; cheering thereafter

WHO THIS COURSE IS FOR

First steps: Ages 4-5 taking their very first structured math steps, playfully
K 2 students: Kindergarten to grade 2 children whose school math needs a stronger foundation underneath
Eager minds: Young children already fascinated by numbers who deserve real food for it
Worried starts: Children whose first school math experiences created tears, reversible at this age, quickly
Planning parents: Families who want the foundation laid right the first time, before gaps exist

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: COUNTING THAT MEANS SOMETHING (MONTHS 1-4)

Counting looks simple and hides deep ideas: one number per object, the last number telling the total, quantities compared and conserved. This phase builds those ideas properly through objects, movement and games, the difference between reciting numbers and understanding them.

Month 1: Numbers as Quantities (Weeks 1-4)

A Playful First Look

Topics:

- Counting objects to see how far counting flows
- Comparing two small groups for more or fewer
- Copying and continuing a simple pattern
- Noticing how the child talks about numbers
- Spotting counting habits gently, without correcting
- Finding a warm, happy starting point

Practice: Line up a few toys and ask your child to count them and tell you which pile has more, keeping it a game and not a test.

Detail: Playful diagnostic disguised as games: counting, comparing and pattern instincts observed

One Number for Each Thing

Topics:

- Touching each object once while saying one number
- Matching one number word to one thing
- Understanding the last number tells the total
- Counting a group and answering how many
- Recounting a moved group to see the total stays the same
- Slowing down to count carefully

Practice: Count the stairs together as you climb, touching each step and cheering the final number as the total.

Detail: One-to-one counting with real objects; the cardinality idea, the last number IS the amount

Seeing Small Numbers Instantly

Topics:

- Recognising 1, 2 and 3 without counting
- Seeing 4 and 5 as quick patterns
- Reading dice dots at a glance
- Showing numbers fast with fingers
- Naming flashed dot cards instantly
- Talking about how the amount was seen

Practice: Flash some fingers for a second, hide them, and ask how many your child saw, taking turns being the flasher.

Detail: Subitizing: seeing 3, 4 and 5 without counting, dice, dots and finger patterns

More, Fewer, or the Same?

Topics:

- Comparing two groups for more or fewer
- Matching items one to one to compare fairly
- Recognising when two groups are the same
- Using the words more, fewer and equal
- Explaining how you know which has more
- Making two groups equal by adding or taking away

Practice: Put out two rows of buttons and ask your child to line them up in pairs to prove which row has more, or whether they match.

Detail: Comparing quantities: more, fewer, same, justified with matching, not guessing

Months 2-3: The Number World to 10 (Weeks 5-12)

Number Symbols and Counting Both Ways

Topics:

- Matching the written numeral to its quantity
- Recognising the numerals 0 to 10
- Counting forwards to ten smoothly
- Counting backwards from ten like a rocket
- Placing a numeral card beside the right group
- Writing or tracing numerals playfully

Practice: Do a rocket countdown from 10 to 0 before a jump or a tickle, then count back

up to ten together.

Detail: Numerals connected to quantities; counting forwards and backwards as a game

The Ten-Frame as Home Base

Topics:

- Filling a ten-frame from left to right
- Seeing a full row of five
- Reading 7 as five and two more
- Spotting how many empty spaces are left
- Seeing numbers hidden inside bigger numbers
- Knowing how far a number is from ten

Practice: Draw a two-by-five grid and let your child place counters to show a number like 8, then say it as five and three more.

Detail: Ten-frames as home base: five-and-some structure, seeing numbers inside numbers

Breaking Numbers into Parts

Topics:

- Splitting a small group into two parts
- Seeing 5 as 2 and 3, or 4 and 1
- Finding different ways to make the same number
- Putting the two parts back into the whole
- Using two hands to hold the two parts
- Talking about the parts out loud

Practice: Hide 5 counters behind your back in two hands, show one hand, and let your child work out how many are hiding in the other.

Detail: Part-whole play: 5 is 2-and-3, 4-and-1, decomposition as a toy, years before it is a technique

First, Second, Third and Where Things Sit

Topics:

- Using first, second, third and last
- Describing position with in front, behind and between
- Walking along a number line as a path
- Finding which number comes before or after
- Ordering a small line of toys
- Naming who is in each place in a line

Practice: Line up soft toys and ask playful questions like which one is third, or who is between the first and the fourth.

Detail: Ordinal numbers, position words and the number line met as a walking game

Month 4: Patterns and Sorting + Stage Checkpoint (Weeks 13-16)

Patterns That Repeat

Topics:

- Copying a simple repeating pattern
- Continuing a pattern by what comes next
- Creating a brand new pattern of your own
- Naming the repeating unit, like red blue red blue
- Spotting patterns in colours, shapes and sounds
- Fixing a pattern with a deliberate mistake in it

Practice: Start a pattern with household items, like spoon fork spoon fork, and ask what comes next, then let your child build one for you.

Detail: Repeating patterns: copy, extend, create, the beginnings of algebraic thinking

Sorting by How Things Are Alike

Topics:

- Sorting objects into groups by colour, size or shape
- Choosing your own rule for sorting
- Explaining the rule you used
- Noticing an object that does not belong
- Sorting the same set a second, different way
- Describing groups with words like all and none

Practice: Tip out a box of toys or socks and ask your child to sort them, then guess the rule they chose before they tell you.

Detail: Sorting and classifying by attributes; explaining the rule you sorted by

Patterns That Grow

Topics:

- Seeing a pattern that gets bigger each step
- Working out what comes next in a growing pattern
- Building steps like 1, then 2, then 3 blocks
- Describing how the pattern changes
- Predicting a later step in the pattern
- Comparing repeating and growing patterns

Practice: Build a staircase of blocks that grows by one each time and ask your child how many blocks the next step needs.

Detail: Growing patterns and what-comes-next reasoning

Phase One Check Through Play

Topics:

- Counting a group and telling the total
- Comparing quantities with more and fewer
- Showing a number on a ten-frame
- Copying and extending a pattern
- Explaining thinking out loud
- Teaching a skill to a toy or a parent

Practice: Let your child teach a teddy how to count a small group and spot which pile has more, while you happily listen in.

Assessment: A gentle play-based check of counting, comparing, ten-frames and patterns, where the child shows a skill, says how it works, and teaches it to a toy, with no pressure.

Detail: Stage mastery check through play: show it, say it, teach it to your bear

PHASE 2: STORIES, BONDS AND THE TEEN NUMBERS (MONTHS 5-8)

Addition and subtraction arrive as stories acted out with objects, joining, separating, comparing, long before symbols carry them. Bonds to 10 grow toward automaticity through games, and the famously confusing teen numbers get demystified with place-value materials.

Month 5: Addition and Subtraction as Stories (Weeks 17-20)

Adding and Taking Away as Stories

Topics:

- Acting out a joining story with objects
- Acting out a taking-away story
- Meeting the plus sign as and more
- Meeting the minus sign as take away
- Reading a simple number story as a picture
- Telling the story back in your own words

Practice: Tell a tiny story with toys, like three ducks and two more join them, and let your child act it out and say how many altogether.

Detail: Joining and separating stories acted with objects; the plus and minus symbols as story shorthand

Counting On and Counting Back

Topics:

- Starting from the bigger number and counting on
- Not going back to one every time
- Counting back to take away
- Using fingers to count on a few more
- Playing the hidden-in-the-hand take-away game
- Choosing to count on or count back

Practice: Hold some counters in a closed hand, say the number inside, add a couple on top, and let your child count on from the hidden number.

Detail: Counting on (not starting over) and counting back; the empty-hand subtraction game

Month 6: Bonds of 10 and Fact Families (Weeks 21-24)

The Pairs That Make Ten

Topics:

- Finding two parts that make ten
- Learning pairs like 6 and 4, 7 and 3
- Filling a ten-frame to see the missing part
- Answering the partner of a number quickly
- Playing many games to make the pairs stick
- Feeling how ten is a friendly whole

Practice: Play friends of ten: hold up some fingers on one hand and ask how many more make ten, then swap hands.

Detail: Bonds of 10 through every game we know, the single most valuable fact set in mathematics

One Number Trio, Four Facts

Topics:

- Seeing three numbers that belong together
- Making two addition facts from the trio
- Making two subtraction facts from the trio
- Understanding adding and taking away as linked
- Using one known fact to find the others
- Building the whole fact family from a picture

Practice: Give your child three numbers like 2, 8 and 10 and see how many facts they can say using all three.

Detail: Fact families: 3+7, 7+3, 10-7, 10-3 as one idea seen four ways

Month 7: Teen Numbers and Place Value Beginnings (Weeks 25-28)

Teen Numbers Are Ten and Some More

Topics:

- Building a teen number as one ten and some ones
- Seeing 14 as ten and four
- Grouping ten small items into one bundle
- Reading teen numbers correctly
- Understanding where place value begins
- Matching the numeral to the ten-and-ones picture

Practice: Bundle ten straws or sticks with a band, add a few loose ones, and ask your child to name the teen number you made.

Detail: Teens demystified: 14 as ten-and-four, built with materials, where place value actually starts

All the Way to Twenty

Topics:

- Counting forwards and backwards to 20
- Comparing two numbers up to 20
- Ordering numbers up to 20
- Filling a second ten-frame past ten
- Previewing the make-ten bridge as a game
- Spotting the ten inside a teen number

Practice: Count a bigger pile of objects, like 16 pasta pieces, into two ten-frames, then ask which of two numbers up to 20 is bigger.

Detail: Numbers to 20: counting, comparing, and bridging-through-10 previewed as a game

Month 8: Story Problems and the Explain Habit (Weeks 29-32)

Act It, Draw It, Write It

Topics:

- Acting out a word problem with objects
- Drawing a simple picture of the story
- Writing the matching number sentence
- Finding the question hidden in the story
- Deciding whether to join or take away
- Checking the answer against the story

Practice: Make up a little story, such as five apples and two eaten, and guide your child to act it, draw it, then write the number sentence.

Detail: Word problems as theatre: act it, draw it, then write it, the three-step that prevents future word-problem fear

Missing Numbers and Phase Two Check

Topics:

- Finding a missing number in a simple sentence
- Thinking backwards from an answer
- Using fact families to fill the gap
- Reviewing story problems and bonds of ten
- Explaining how a puzzle was solved
- Showing teen numbers as ten and ones

Practice: Pose a gentle puzzle like something plus 3 makes 7 using counters, and let your child find the missing amount.

Assessment: A relaxed play-based check of story problems, bonds of ten, fact families and teen numbers, where the child solves a puzzle and explains the thinking, with no pressure.

Detail: Missing-number puzzles and both-ways thinking; stage mastery check through play

PHASE 3: TO 100 AND INTO THE WORLD (MONTHS 9-12)

The number world expands to 100 with place-value understanding, and math steps into the child's world: measuring, money, time, shapes and simple data. The course closes with problem-solving courage and a confident handoff to grade-level mathematics.

Month 9: The Road to 100 (Weeks 33-36)

Building Numbers from Tens and Ones

Topics:

- Building a two-digit number from tens and ones
- Bundling tens and counting the leftover ones
- Reading a number like 34 as three tens and four
- Comparing two two-digit numbers
- Ordering several numbers from small to big
- Matching the numeral to the tens-and-ones model

Practice: Use grouped straws or blocks to build a number like 27, then build another and ask your child which is bigger and why.

Detail: Tens and ones with materials: building two-digit numbers, comparing and ordering them

Skip Counting and the Hundred Square

Topics:

- Skip counting by 10s to one hundred
- Skip counting by 5s and by 2s
- Hearing skip counting as a rhythm
- Finding patterns on the hundred square
- Moving up, down and across the hundred square
- Spotting the tens column and its counting patterns

Practice: Chant counting by tens up to a hundred on a walk, then try by fives, clapping to keep the rhythm.

Detail: Skip counting by 2s, 5s and 10s as rhythm and pattern; the 100-square explored like a map

Month 10: Two-Digit Confidence (Weeks 37-40)

Adding Two-Digit Numbers

Topics:

- Adding two numbers using tens and ones blocks
- Combining the tens, then the ones
- Moving from blocks to picturing the jump
- Adding a whole ten to a number

- Solving sums like 23 plus 14
- Checking the answer with the materials

Practice: Set out blocks for a sum like 22 plus 13 and let your child join the tens and the ones, then say the total.

Detail: Adding tens, adding ones: two-digit addition with materials first, mental jumps second

Ten More and Ten Less

Topics:

- Finding ten more than a number quickly
- Finding ten less than a number
- Using the hundred square to jump by ten
- Noticing only the tens digit changes
- Meeting friendly round numbers
- Jumping to the next friendly ten

Practice: Say a number like 34 and ask for ten more, then ten less, using the hundred square to check each jump.

Detail: Ten-more ten-less fluency; friendly-number thinking previewed gently

Month 11: Measurement, Money and Time (Weeks 41-44)

Measuring and Guessing

Topics:

- Comparing lengths as longer and shorter
- Comparing weights as heavier and lighter
- Comparing capacity as holds more or less
- Measuring with repeated units like paper clips
- Guessing a measurement, then checking it
- Using words like longest, heaviest and full

Practice: Guess how many hand-spans long the table is, then measure together to see how close the guess was.

Detail: Length, weight and capacity by comparing and then by counting units; estimation as a guessing game with feedback

Money and Telling the Time

Topics:

- Recognising a few common coins
- Paying for small pretend items
- Counting a small amount of money
- Reading o'clock on a real clock
- Reading half-past on a real clock
- Linking times to parts of the day

Practice: Play shop with real coins for a few priced items, then check what o'clock it is on a real clock together.

Detail: Coins and simple shopping play; o'clock and half-past on real clocks

Month 12: Shapes, Data and Graduation (Weeks 45-52)

Shapes and Fair Shares

Topics:

- Naming flat shapes like circle, square and triangle
- Naming solid shapes like cube, sphere and cone
- Building and hunting for shapes at home
- Describing shapes by their sides and corners
- Sharing something fairly into halves
- Sharing into quarters as four equal parts

Practice: Go on a shape hunt around the house, then share a snack or a drawing fairly into halves and quarters.

Detail: 2D and 3D shapes built, named and hunted at home; halves and quarters as fair sharing, fractions met kindly

Graphs, Graduation and the Road Ahead

Topics:

- Asking a simple survey question
- Collecting answers from family or class
- Drawing a picture graph of the results
- Reading which answer had the most
- Teaching a favourite math game to others
- Looking ahead to grade-level math

Practice: Help your child survey the family on a fun question, like favourite fruit, and draw a little picture graph of the answers.

Assessment: A gentle final celebration where the child makes a simple pictograph, teaches a favourite math game, and revisits the year's ideas, with a friendly road-ahead plan instead of a test.

Detail: Simple pictographs from the child's own surveys; graduation project, the child teaches their favourite math game, and the road-ahead plan into grade-level math

PROJECTS THE STUDENT BUILDS AND KEEPS

- The child's own counting museum: collections photographed and counted
- Pattern gallery: repeating and growing patterns the child created
- Shape hunt portfolio from around the home
- The child's first survey and pictograph
- Graduation game: the child teaches their favourite math game

Total projects: A year of playful artifacts the child is proud of

SKILLS THE STUDENT WILL HAVE AT THE END

- Counting with one-to-one correspondence and cardinality
- Subitizing and quantity comparison
- Part-whole thinking and bonds of 10
- Addition and subtraction as stories, then facts
- Teen numbers and place-value beginnings
- Numbers to 100 with tens-and-ones understanding
- Early measurement, money, time and shapes
- Explaining mathematical thinking aloud

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes in short playful segments
Home play: Everyday math prompts for parents, minutes, not worksheets
Materials: Household objects as manipulatives; printable ten-frames provided
Total per week: About 2.5 gentle hours

SUPPORT INCLUDED

Live sessions: 8 live classes every month, 2 per week
Mentorship: One dedicated mentor who knows the child's stage and spark
Recordings: Every class recorded
Progress tracking: Stage mastery notes shared with parents
Flexibility: Slots move around naps, school and family life

CERTIFICATION DETAIL

Phase certificates: Stage mastery check at the end of each phase
Final certificate: Course completion certificate from Modern Age Coders
Portfolio projects: The child's playful artifact collection

WHERE THIS CAN LEAD

- Ready for Mental Maths Mastery with the bonds already forming
- Ready for the Elementary Mathematics Masterclass, arriving confident
- A child who enters grade 3 liking math, the outcome that compounds for a decade
- Early access to the maths-and-coding world for the builder types

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Can a 4-year-old really learn in online classes?

A: Yes, with the right design, which is the whole craft here. Classes run as short, varied playful segments: a counting game, a building challenge, a story problem, a movement break with skip counting. The mentor works with real objects the child holds up to the camera, and parents of our youngest students sit nearby for the first weeks. The free demo shows you exactly how it works with your child.

Q: Isn't this age too early for structured math?

A: It is exactly the right age for the right kind: playful, concrete, talk-rich mathematics. Research on early number sense is unambiguous, the part-whole and counting ideas built at 4-7 predict mathematics success for years. What would be wrong at this age is worksheets and drilling, which is why we do neither.

Q: How is this different from apps like Khan Kids or busy-work workbooks?

A: Apps entertain and practice; they cannot listen. The engine of early math learning is math talk, a child explaining 'I saw 5 because 4-and-1' to a person who responds. Our mentors have that conversation twice a week, notice exactly which ideas have landed, and adjust in real time. No app does that.

Q: My child already loves numbers. Will this stretch them?

A: Happily, the mentor teaches to stage, not age. A number-loving five-year-old might race into part-whole play and early place value while a classmate consolidates counting. Small groups and 1-on-1 make that flexibility real.

Q: What do parents need to do at home?

A: Kitchens and car rides teach more early math than worksheets ever did, and we show you how to use them.

Q: What comes after this course?

A: Graduates flow into Mental Maths Mastery (for fluency depth) or the Elementary Mathematics Masterclass (the full grades 1-5 program) with the same mentor where possible, arriving with the foundation those courses quietly assume.

Q: Can we try it before paying?

A: Yes. Every child starts with a free live demo class that doubles as the playful diagnostic, no card details, no obligation, and you will know within the hour whether your child lights up.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/early-math-foundations>

=====

COURSE 51 OF 118: ELEMENTARY MATHEMATICS FOR GRADE 1-5: NUMBERS TO GEOMETRY

=====

Course link: <https://learn.modernagecoders.com/courses/elementary-mathematics-complete-masterclass>
Subject area: Elementary Mathematics Education
Designed for: Elementary school, Girls only batches
Who can join: Complete Beginner to Advanced Elementary Mathematics
Course length: 24 months (104 weeks)
Time needed: 10-15 hours/week recommended
Certificate: Elementary Mathematics Proficiency Certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online elementary maths course for Grade 1-5: master numbers, operations, fractions, geometry and measurement through interactive, real-world learning.

In one line: From Counting to Complex Problem-Solving - Building Mathematical Minds

ABOUT THE PROGRAMME

This comprehensive 2-year program transforms learners into confident mathematics problem-solvers. Whether you're a young student, parent helping children, teacher seeking resources, or adult learner filling gaps, this masterclass covers every aspect of elementary mathematics. You'll master fundamental concepts through interactive lessons, visual learning, hands-on activities, and real-world applications. Progress from basic counting to advanced problem-solving, building strong foundations for higher mathematics. By completion, you'll have solved 5000+ problems, completed 100+ projects, and developed exceptional numerical literacy.

WHAT MAKES THIS COURSE DIFFERENT

- Visual and interactive learning approach
- Adaptive pacing for different learning speeds
- Real-world applications and practical problems
- Games and activities for engagement
- Parent/teacher guidance materials included
- Multiple problem-solving strategies taught
- Mental math techniques emphasized
- Strong conceptual understanding focus

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Numbers, Basic Operations, Shapes, Patterns
Phase 2: Core Skills (Months 7-12): Advanced Operations, Fractions, Measurement, Data
Phase 3: Advanced Concepts (Months 13-18): Decimals, Complex Geometry, Advanced Problem-Solving
Phase 4: Mastery & Applications (Months 19-24): Integration, Real-World Math, Competition Prep

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No prior mathematics knowledge required
Age: Suitable for ages 6+ with appropriate support
Equipment: Basic school supplies, computer/tablet helpful
Time commitment: 10-15 hours per week
Language: Basic reading comprehension
Support: Parent/guardian support for younger learners

WHO THIS COURSE IS FOR

Young students: Elementary school students (Grades 1-5)
Homeschoolers: Homeschooling families seeking comprehensive curriculum
Parents: Parents wanting to support children's learning
Teachers: Educators seeking supplementary resources
Tutors: Tutors needing structured curriculum

Adult learners: Adults filling mathematics gaps
Esl students: English language learners with visual supports

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: MATHEMATICAL FOUNDATIONS (MONTHS 1-6, WEEKS 1-26)

Build rock-solid foundations in numbers, basic operations, shapes, and mathematical thinking.

Months 1-2: Numbers and Counting Mastery (Week 1-8)

Introduction to Numbers and Counting

Topics:

- What are numbers? History and importance
- Counting 1-20: objects, fingers, visual aids
- Number recognition and writing practice
- Number words: spelling one to twenty
- Before, after, and between numbers
- Counting forward and backward
- Skip counting by 2s introduction
- Zero concept and its importance
- One-to-one correspondence
- Greater than, less than concepts
- Equal to understanding
- Number line introduction 0-20

Projects built:

- Create number book with illustrations
- Number hunt around the house
- Build counting games with everyday objects

Practice: Daily: 30 min counting practice, 1 hour number activities

Numbers to 100 and Place Value

Topics:

- Counting 21-100 systematically
- Tens and ones concept introduction
- Place value understanding with blocks
- Writing numbers in expanded form
- Number patterns in 100 chart
- Skip counting by 2s, 5s, and 10s
- Even and odd numbers introduction
- Comparing two-digit numbers
- Ordering numbers from least to greatest
- Number bonds to 10
- Introduction to estimation
- Rounding to nearest 10

Projects built:

- 100 chart coloring patterns
- Place value manipulative games
- Create skip counting songs

Practice: Master place value with 50 daily exercises

Basic Addition (Single Digit)

Topics:

- Addition concept: combining sets
- Plus sign and equals sign meaning
- Addition sentences and vocabulary
- Adding with objects and pictures
- Addition on number line
- Addition facts 0-5
- Addition facts 6-10
- Doubles facts (1+1, 2+2, etc.)
- Near doubles strategy
- Making 10 strategy
- Commutative property (turn-around facts)
- Zero property of addition

Projects built:

- Addition fact family houses
- Create addition story problems book
- Design addition board game

Practice: Master 100 addition facts through games and drills

Basic Subtraction (Single Digit)

Topics:

- Subtraction as taking away
- Subtraction as finding difference
- Minus sign understanding
- Subtraction sentences
- Subtraction with objects and pictures
- Subtraction on number line
- Subtraction facts 0-10

- Fact families: addition and subtraction
- Related facts connection
- Counting back strategy
- Counting up strategy
- Think addition to subtract

Projects built:

- Subtraction story problems collection
- Fact family triangles craft
- Subtraction bowling game

Practice: Master subtraction facts through 10

Months 3-4: Advanced Operations and Problem-Solving (Week 9-17)

Two-Digit Addition

Topics:

- Adding multiples of 10
- Adding two-digit numbers without regrouping
- Vertical addition alignment
- Adding ones, then tens strategy
- Breaking apart numbers to add
- Addition with regrouping introduction
- Carrying concept explanation
- Using base-10 blocks for regrouping
- Addition with multiple addends
- Checking addition with subtraction
- Mental math strategies for addition
- Estimating sums

Projects built:

- Shopping mall addition game
- Create addition puzzle cards
- Design digital addition calculator

Practice: Solve 200 two-digit addition problems

Two-Digit Subtraction

Topics:

- Subtracting multiples of 10
- Two-digit subtraction without regrouping
- Vertical subtraction setup
- Subtract ones, then tens
- Subtraction with regrouping introduction
- Borrowing/regrouping concept
- Using manipulatives for regrouping
- Checking subtraction with addition
- Subtraction across zeros
- Mental math for subtraction
- Estimating differences
- Problem-solving with subtraction

Projects built:

- Subtraction treasure hunt
- Create subtraction word problem cards
- Build subtraction machine model

Practice: Master 200 two-digit subtraction problems

Introduction to Multiplication

Topics:

- Multiplication as repeated addition
- Groups and arrays understanding
- Times sign and multiplication vocabulary
- Multiplication sentences
- Skip counting connection
- Multiplication by 0 and 1
- Multiplication by 2 and 5
- Multiplication by 10
- Doubles in multiplication
- Using arrays to multiply
- Commutative property of multiplication
- Building multiplication tables

Projects built:

- Multiplication array art
- Times tables wheel craft
- Create multiplication story book

Practice: Learn multiplication facts 0-5

Geometry Basics: 2D Shapes

Topics:

- Circle: properties and examples
- Triangle: types and properties
- Square vs rectangle comparison
- Pentagon, hexagon, octagon
- Sides and vertices counting
- Straight vs curved lines
- Open vs closed shapes
- Symmetry introduction

- Lines of symmetry
- Creating patterns with shapes
- Combining shapes to make pictures
- Shape sorting and classification

Projects built:

- Shape collage artwork
- Geometry in nature scavenger hunt
- Build shapes with craft sticks

Practice: Identify and draw 50 different shapes

Month 3-4 Review & Assessment

Topics:

- Operations review
- Problem-solving strategies review
- Mental math techniques
- Geometry concepts recap
- Real-world applications

Projects built:

- Create math game combining all concepts
- Design math puzzle book
- Build interactive math display

Assessment: Comprehensive test on operations and geometry

Months 5-6: Measurement, Time, and Money (Week 18-26)

Time and Calendar

Topics:

- Clock reading: hour hand understanding
- Clock reading: minute hand understanding
- Reading time to the hour
- Reading time to the half hour
- Quarter past and quarter to
- Reading time to 5 minutes
- AM and PM understanding
- Days of the week sequence
- Months of the year
- Reading calendar dates
- Yesterday, today, tomorrow
- Elapsed time simple problems

Projects built:

- Create personal daily schedule
- Build paper plate clock
- Design calendar for the year

Practice: Read 100 different clock times

Money and Financial Literacy

Topics:

- Coin identification and values
- Penny, nickel, dime, quarter
- Dollar bills introduction
- Counting coins same type
- Counting mixed coins
- Making change basics
- Dollar and cents notation
- Adding money amounts
- Subtracting money amounts
- Word problems with money
- Needs vs wants concept
- Simple budgeting introduction

Projects built:

- Classroom store simulation
- Create money math workbook
- Design piggy bank savings tracker

Practice: Solve 100 money word problems

Length and Height Measurement

Topics:

- Non-standard units measurement
- Introduction to ruler use
- Inches and feet
- Centimeters and meters
- Measuring classroom objects
- Comparing lengths
- Ordering by length
- Estimating lengths
- Perimeter introduction
- Measuring curved lines
- Height measurement
- Distance concepts

Projects built:

- Measurement scavenger hunt
- Create measurement book
- Build measuring tools

Practice: Measure 50 objects accurately
Weight and Capacity

Topics:

- Heavy and light comparison
- Balance scale understanding
- Pounds and ounces introduction
- Kilograms and grams basics
- Weighing common objects
- Capacity: full, empty, half
- Cups, pints, quarts, gallons
- Liters and milliliters
- Comparing capacities
- Estimating weight and capacity
- Temperature introduction
- Reading thermometer

Projects built:

- Kitchen measurement experiments
- Create weight comparison chart
- Design capacity containers game

Practice: Complete 50 measurement activities

Phase 1 Capstone Project

Topics:

- Integration of all Phase 1 concepts
- Real-world problem solving
- Project presentation skills
- Mathematical communication
- Reflection on learning

Projects built:

- MAJOR CAPSTONE: Math Fair Project
- Create comprehensive math portfolio
- Design math board game using all concepts
- Develop math teaching materials for younger students

Assessment: Phase 1 Final Exam - comprehensive assessment

PHASE 2: CORE MATHEMATICS SKILLS (MONTHS 7-12, WEEKS 27-52)

Master multiplication, division, fractions, advanced geometry, and complex problem-solving.

Months 7-8: Multiplication and Division Mastery (Week 27-35)

Advanced Multiplication Facts

Topics:

- Multiplication facts 6-9
- Square numbers understanding
- Patterns in multiplication table
- Tricks for 9s facts
- Multiplication by 11 and 12
- Distributive property introduction
- Breaking apart to multiply
- Doubling and halving strategy
- Multiplication fact fluency drills
- Timed multiplication tests
- Multiplication games and competitions
- Real-world multiplication applications

Projects built:

- Multiplication facts championship
- Create multiplication tricks book
- Design multiplication app prototype

Practice: Master all multiplication facts 0-12

Multi-Digit Multiplication

Topics:

- Multiplying by 10, 100, 1000
- Two-digit by one-digit multiplication
- Partial products method
- Standard algorithm introduction
- Area model for multiplication
- Lattice multiplication method
- Three-digit by one-digit
- Two-digit by two-digit basics
- Estimating products
- Checking multiplication with division
- Word problems with multiplication
- Multi-step problems

Projects built:

- Area model visual guide
- Multiplication method comparison
- Create multiplication story problems

Practice: Solve 200 multi-digit multiplication problems

Introduction to Division

Topics:

- Division as sharing equally
- Division as repeated subtraction
- Division symbol and vocabulary
- Dividend, divisor, quotient
- Division facts related to multiplication
- Dividing by 1 and itself
- Division by 2, 5, and 10
- Using multiplication to divide
- Fact families with multiplication/division
- Remainders introduction
- Division on number line
- Array model for division

Projects built:

- Division fact family houses
- Create division story book
- Design fair sharing game

Practice: Master basic division facts

Long Division

Topics:

- Long division setup
- Divide, multiply, subtract, bring down
- Two-digit divided by one-digit
- Handling remainders
- Checking division with multiplication
- Three-digit divided by one-digit
- Zeros in quotient
- Estimating quotients
- Division word problems
- Interpreting remainders in context
- Mental math division strategies
- Division patterns

Projects built:

- Long division step-by-step guide
- Division problem generator
- Create division board game

Practice: Complete 150 long division problems

Problem-Solving with Operations

Topics:

- Choosing the correct operation
- Multi-step word problems
- Working backward strategy
- Guess and check method
- Drawing diagrams to solve
- Making tables strategy
- Finding patterns in problems
- Writing equations from words
- Checking reasonableness
- Problem-solving plan steps
- Creating own word problems
- Real-world applications

Projects built:

- Problem-solving strategy poster
- Word problem collection book
- Math detective game

Practice: Solve 100 multi-step word problems

Months 9-10: Fractions and Decimals Introduction (Week 36-44)

Understanding Fractions

Topics:

- Fraction as part of whole
- Numerator and denominator meaning
- Unit fractions understanding
- Halves, thirds, fourths
- Fifths, sixths, eighths
- Fractions on number line
- Comparing fractions same denominator
- Comparing fractions same numerator
- Equivalent fractions with models
- Simplest form introduction
- Mixed numbers introduction
- Improper fractions basics

Projects built:

- Fraction pizza party
- Fraction art with paper folding
- Create fraction manipulatives

Practice: Master fraction concepts with 100 exercises

Operations with Fractions

Topics:

- Adding fractions same denominator
- Subtracting fractions same denominator
- Adding mixed numbers simple
- Subtracting mixed numbers simple
- Finding common denominators
- Adding unlike fractions basics
- Subtracting unlike fractions basics
- Multiplying fraction by whole number
- Fraction of a set
- Word problems with fractions
- Estimating with fractions
- Fractions in measurement

Projects built:

- Fraction operations cookbook
- Fraction calculator design
- Fraction word problem collection

Practice: Complete 150 fraction operations

Introduction to Decimals

Topics:

- Decimal point meaning
- Tenths place understanding
- Hundredths place understanding
- Reading decimal numbers
- Writing decimals in words
- Decimals on number line
- Comparing decimals
- Ordering decimals
- Rounding decimals to whole
- Rounding to nearest tenth
- Fractions to decimals (tenths, hundredths)
- Money as decimals

Projects built:

- Decimal place value chart
- Decimal number line display
- Create decimal comparison game

Practice: Master decimal concepts with 100 exercises

Decimal Operations

Topics:

- Adding decimals with same places
- Subtracting decimals same places
- Adding decimals different places
- Lining up decimal points
- Adding money with decimals
- Subtracting money with decimals
- Multiplying decimals by 10, 100
- Multiplying decimal by whole number
- Estimating with decimals
- Word problems with decimals
- Decimals in measurement
- Calculator use with decimals

Projects built:

- Decimal shopping simulation
- Create decimal workbook
- Design decimal maze game

Practice: Solve 150 decimal problems

Data and Graphing

Topics:

- Collecting and organizing data
- Tally charts and frequency tables
- Pictographs reading and creating
- Bar graphs horizontal and vertical
- Line plots introduction
- Reading and interpreting graphs
- Finding range, mode, median
- Mean (average) introduction
- Creating surveys
- Double bar graphs
- Making predictions from data
- Data in real world

Projects built:

- Class survey project
- Create graphing portfolio
- Design data collection app

Practice: Create and interpret 30 different graphs

Months 11-12: Advanced Geometry and Measurement (Week 45-52)

Advanced 2D Geometry

Topics:

- Types of angles: acute, right, obtuse

- Measuring angles with protractor
- Drawing angles
- Parallel and perpendicular lines
- Types of triangles by sides
- Types of triangles by angles
- Quadrilaterals classification
- Properties of rectangles, squares
- Rhombus and parallelogram
- Trapezoid properties
- Congruent shapes
- Similar shapes basics

Projects built:

- Geometry art portfolio
- Angle measurement tools
- Create geometry handbook

Practice: Complete 100 geometry exercises

3D Shapes and Volume

Topics:

- 3D shapes: cube, rectangular prism
- Sphere, cylinder, cone
- Pyramid types
- Faces, edges, vertices counting
- Nets of 3D shapes
- Building 3D shapes from nets
- Volume as counting cubes
- Volume of rectangular prism
- Surface area introduction
- Real-world 3D shapes
- Cross-sections basics
- 3D shape sorting

Projects built:

- 3D shape construction kit
- Volume exploration lab
- Create 3D shape museum

Practice: Build and analyze 30 3D shapes

Area and Perimeter

Topics:

- Perimeter of polygons
- Perimeter word problems
- Area by counting squares
- Area of rectangle formula
- Area of square
- Area of triangle introduction
- Area of parallelogram
- Composite figures area
- Perimeter of composite figures
- Area vs perimeter understanding
- Real-world area problems
- Estimating area and perimeter

Projects built:

- Design dream bedroom with area
- Perimeter and area games
- Create area/perimeter reference

Practice: Solve 100 area/perimeter problems

Coordinate Geometry

Topics:

- Coordinate plane introduction
- X-axis and y-axis
- Ordered pairs (x, y)
- Plotting points on grid
- Reading coordinates
- First quadrant focus
- Creating pictures with coordinates
- Distance between points (horizontal/vertical)
- Simple transformations: slides
- Reflections across axes
- Coordinate battleship game
- Real-world coordinate uses

Projects built:

- Coordinate art gallery
- Treasure map with coordinates
- Design coordinate game

Practice: Plot and identify 100 coordinate points

Phase 2 Capstone Project

Topics:

- Integration of Phase 2 concepts
- Complex problem-solving
- Mathematical reasoning
- Project presentation

- Self-assessment and reflection
- Projects built:
- MAJOR CAPSTONE: Mathematics Exhibition
 - Create math learning center
 - Design complete math curriculum game
 - Develop math tutoring materials
- Assessment: Phase 2 Final Exam - comprehensive evaluation

PHASE 3: ADVANCED CONCEPTS & PROBLEM-SOLVING (MONTHS 13-18, WEEKS 53-78)

Master complex operations, advanced fractions/decimals, ratios, percentages, and sophisticated problem-solving.

Months 13-14: Advanced Number Theory (Week 53-61)

Large Numbers and Place Value

Topics:

- Numbers to millions
- Place value to millions
- Reading and writing large numbers
- Expanded form to millions
- Comparing large numbers
- Ordering large numbers
- Rounding to any place value
- Scientific notation introduction
- Powers of 10 understanding
- Number patterns in large numbers
- Estimation with large numbers
- Real-world large numbers

Projects built:

- Large number reference book
- Population studies project
- Create million visualization

Practice: Master large numbers with 100 exercises

Factors and Multiples

Topics:

- Factors definition and finding
- Factor pairs
- Prime and composite numbers
- Prime factorization
- Factor trees
- Greatest Common Factor (GCF)
- Multiples and common multiples
- Least Common Multiple (LCM)
- Using GCF and LCM
- Divisibility rules 2, 3, 5, 9, 10
- Perfect squares
- Square roots introduction

Projects built:

- Factor and multiple game
- Prime number sieve activity
- Create divisibility rule guide

Practice: Find factors and multiples of 100 numbers

Advanced Multiplication Strategies

Topics:

- Three-digit by two-digit multiplication
- Multiplication of decimals basics
- Multiplying by powers of 10
- Mental math multiplication tricks
- Distributive property applications
- Area model for larger numbers
- Partial products extended
- Estimation strategies for products
- Multiplication with variables introduction
- Order of operations with multiplication
- Complex word problems
- Real-world multiplication applications

Projects built:

- Multiplication strategy handbook
- Mental math competition
- Create multiplication teaching video

Practice: Complete 200 advanced multiplication problems

Advanced Division Strategies

Topics:

- Three-digit by two-digit division
- Division with decimals in dividend
- Dividing by powers of 10
- Mental division strategies
- Interpreting remainders as fractions

- Interpreting remainders as decimals
- Division estimation techniques
- Compatible numbers for division
- Division with variables basics
- Order of operations with division
- Complex division word problems
- Division in real-world contexts

Projects built:

- Division strategy guide
- Remainder interpretation examples
- Design division challenge game

Practice: Solve 200 advanced division problems

Number Patterns and Sequences

Topics:

- Arithmetic sequences
- Finding the rule in patterns
- Extending number patterns
- Geometric sequences basics
- Fibonacci sequence introduction
- Triangular numbers
- Pattern rules with operations
- Input-output tables
- Function machines concept
- Growing patterns
- Decreasing patterns
- Complex pattern problems

Projects built:

- Pattern detective notebook
- Create pattern puzzles
- Design sequence generator

Practice: Identify and extend 100 patterns

Months 15-16: Advanced Fractions and Decimals (Week 62-70)

Complex Fraction Operations

Topics:

- Adding mixed numbers with regrouping
- Subtracting mixed numbers with borrowing
- Finding least common denominator
- Adding three or more fractions
- Fraction multiplication understanding
- Multiplying fractions by fractions
- Multiplying mixed numbers
- Reciprocals introduction
- Division of fractions basics
- Dividing whole numbers by fractions
- Complex fraction word problems
- Fractions in recipes and scaling

Projects built:

- Fraction operations cookbook
- Fraction calculator model
- Create fraction game show

Practice: Master 200 complex fraction problems

Advanced Decimal Operations

Topics:

- Multiplying decimals by decimals
- Division with decimals as divisor
- Repeating decimals introduction
- Terminating vs repeating
- Converting fractions to decimals
- Converting decimals to fractions
- Percent as special decimal
- Scientific notation with decimals
- Significant figures basics
- Decimal word problems complex
- Decimals in science contexts
- Calculator strategies with decimals

Projects built:

- Decimal conversion chart
- Scientific measurement project
- Design decimal Olympics

Practice: Complete 200 decimal operations

Ratios and Proportions

Topics:

- Ratio concept and notation
- Writing ratios three ways
- Equivalent ratios
- Simplifying ratios
- Ratio tables
- Rates and unit rates

- Price per unit calculations
- Proportion definition
- Solving proportions
- Cross multiplication
- Scale drawings and maps
- Similar figures and ratios

Projects built:

- Recipe scaling project
- Map scale investigation
- Create ratio/proportion game

Practice: Solve 150 ratio and proportion problems

Percentages

Topics:

- Percent meaning and symbol
- Percent as per hundred
- Converting percent to decimal
- Converting percent to fraction
- Finding percent of a number
- Mental math with percentages
- 10%, 25%, 50%, 75% shortcuts
- Percent increase introduction
- Percent decrease introduction
- Sales tax calculations
- Discount calculations
- Tips and gratuity

Projects built:

- Shopping with percentages
- Create percent reference guide
- Design discount calculator

Practice: Master 150 percentage problems

Mixed Operations and Order

Topics:

- Order of operations introduction
- PEMDAS/BODMAS rule
- Parentheses and brackets
- Exponents in order of operations
- Multiple operations problems
- Nested parentheses
- Writing expressions from words
- Evaluating expressions
- Creating equivalent expressions
- Properties of operations review
- Mental math with mixed operations
- Complex multi-step problems

Projects built:

- Order of operations poster
- Expression puzzle cards
- Create operations game

Practice: Solve 100 order of operations problems

Months 17-18: Advanced Geometry and Measurement (Week 71-78)

Advanced Angle Relationships

Topics:

- Complementary angles
- Supplementary angles
- Vertical angles
- Adjacent angles
- Angles in triangles sum to 180°
- Finding missing angles
- Angles in quadrilaterals
- Interior angles of polygons
- Exterior angles basics
- Angle bisectors
- Perpendicular bisectors
- Constructions with compass

Projects built:

- Angle relationship poster
- Geometric construction kit
- Create angle puzzle game

Practice: Solve 100 angle problems

Transformations and Symmetry

Topics:

- Translations (slides) in detail
- Reflections (flips) advanced
- Rotations (turns) 90° , 180° , 270°
- Combinations of transformations
- Line symmetry advanced
- Rotational symmetry
- Tessellations introduction

- Creating tessellation patterns
- Transformations on coordinate grid
- Enlargements and reductions
- Scale factor basics
- Symmetry in art and nature

Projects built:

- Transformation artwork
- Tessellation designs
- Create symmetry portfolio

Practice: Complete 100 transformation exercises

Circle Geometry

Topics:

- Circle vocabulary: radius, diameter
- Chord and arc definitions
- Center, circumference understanding
- Pi introduction and history
- Circumference formula $C = \pi d$
- Circumference formula $C = 2\pi r$
- Area of circle $A = \pi r^2$
- Finding radius from circumference
- Finding radius from area
- Semicircles and quarter circles
- Composite figures with circles
- Real-world circle problems

Projects built:

- Pi Day celebration materials
- Circle measurement lab
- Design circular art project

Practice: Solve 100 circle problems

Advanced Measurement Conversions

Topics:

- Metric system relationships
- Converting within metric system
- Imperial system relationships
- Converting within imperial system
- Metric to imperial conversions
- Area unit conversions
- Volume unit conversions
- Time conversions complex
- Speed, distance, time relationships
- Rate problems
- Dimensional analysis basics
- Real-world conversion problems

Projects built:

- Conversion reference handbook
- Measurement conversion game
- Create conversion calculator

Practice: Master 150 conversion problems

Phase 3 Capstone Project

Topics:

- Integration of advanced concepts
- Complex problem-solving showcase
- Mathematical modeling
- Project documentation
- Presentation preparation

Projects built:

- MAJOR CAPSTONE: Mathematics Research Project
- Create advanced math tutorial series
- Design comprehensive math assessment tool
- Develop math competition preparation materials

Assessment: Phase 3 Final Exam - advanced concepts mastery

PHASE 4: MASTERY & REAL-WORLD APPLICATIONS (MONTHS 19-24, WEEKS 79-104)

Master pre-algebra concepts, statistics, probability, mathematical reasoning, and real-world applications.

Months 19-20: Pre-Algebra Foundations (Week 79-87)

Variables and Expressions

Topics:

- Variable as unknown number
- Writing algebraic expressions
- Translating words to algebra
- Evaluating expressions
- Substitution method
- Terms and coefficients
- Like terms identification
- Combining like terms

- Distributive property with variables
- Simplifying expressions
- Expression word problems
- Creating expressions from patterns

Projects built:

- Algebra translation dictionary
- Expression puzzle cards
- Create algebra game

Practice: Write and evaluate 150 expressions

Simple Equations

Topics:

- Equation vs expression
- Solving one-step addition equations
- Solving one-step subtraction equations
- Solving one-step multiplication equations
- Solving one-step division equations
- Inverse operations concept
- Checking solutions
- Two-step equations introduction
- Equations with fractions basics
- Equations with decimals basics
- Word problems to equations
- Real-world equation applications

Projects built:

- Equation solving guide
- Balance scale demonstrations
- Create equation puzzle book

Practice: Solve 200 equations

Inequalities Introduction

Topics:

- Inequality symbols: $<$, $>$, $\leq$, $\geq$
- Graphing inequalities on number line
- Writing inequalities from words
- Solving one-step inequalities
- Checking inequality solutions
- Real-world inequality situations
- Compound inequalities basics
- And vs or in inequalities
- Graphing solution sets
- Inequality word problems
- Creating inequalities from data
- Systems thinking introduction

Projects built:

- Inequality visualization tools
- Real-world inequality examples
- Design inequality game

Practice: Solve 100 inequality problems

Functions and Graphing

Topics:

- Function concept introduction
- Input-output relationships
- Function tables
- Function rules and patterns
- Graphing points review
- Graphing linear patterns
- Slope concept basics
- Positive vs negative slope
- Y-intercept understanding
- Graphing from table of values
- Real-world linear relationships
- Non-linear patterns introduction

Projects built:

- Function machine model
- Linear relationship investigations
- Create graphing portfolio

Practice: Graph 100 functions and relationships

Integer Operations

Topics:

- Negative numbers in context
- Integers on number line
- Absolute value concept
- Adding integers with chips/number line
- Subtracting integers
- Multiplying integers
- Dividing integers
- Integer operation rules
- Order of operations with integers
- Integer word problems
- Temperature and elevation problems

- Financial contexts with integers

Projects built:

- Integer operations game
- Temperature tracking project
- Create integer reference guide

Practice: Master 150 integer operations

Months 21-22: Statistics and Probability (Week 88-96)

Advanced Data Analysis

Topics:

- Mean, median, mode review
- Range and outliers
- Quartiles introduction
- Box-and-whisker plots
- Stem-and-leaf plots
- Frequency distributions
- Histograms
- Circle graphs (pie charts)
- Choosing appropriate graphs
- Misleading graphs identification
- Data collection methods
- Surveys and sampling basics

Projects built:

- Statistical investigation project
- Data visualization portfolio
- Create statistics infographic

Practice: Analyze 50 data sets

Probability Fundamentals

Topics:

- Probability as likelihood
- Probability scale 0 to 1
- Certain, likely, unlikely, impossible
- Experimental probability
- Theoretical probability
- Favorable outcomes
- Sample space
- Simple probability calculations
- Probability with dice
- Probability with cards
- Probability with spinners
- Compound events introduction

Projects built:

- Probability experiment lab
- Design probability games
- Create probability reference

Practice: Calculate 100 probabilities

Combinations and Permutations Basics

Topics:

- Counting principle
- Tree diagrams for counting
- Organized lists
- Combinations introduction
- Permutations introduction
- Difference between combinations and permutations
- Factorial notation basics
- Simple combination problems
- Simple permutation problems
- Probability with combinations
- Real-world counting problems
- Problem-solving strategies

Projects built:

- Counting methods handbook
- Combination lock mathematics
- Create counting puzzle book

Practice: Solve 100 counting problems

Statistical Reasoning

Topics:

- Making predictions from data
- Interpolation and extrapolation
- Correlation vs causation
- Bias in data collection
- Random sampling importance
- Margin of error basics
- Confidence basics
- Using statistics to support arguments
- Critical analysis of statistics
- Statistics in media
- Creating statistical reports
- Presenting statistical findings

Projects built:

- Statistical analysis report
- Media statistics critique
- Design statistical study

Practice: Analyze 30 statistical claims

Financial Mathematics

Topics:

- Simple interest calculation
- Compound interest introduction
- Savings and investments basics
- Budget creation and management
- Profit and loss
- Markup and markdown
- Commission calculations
- Tax calculations advanced
- Unit price comparisons
- Best buy problems
- Currency exchange basics
- Financial planning introduction

Projects built:

- Personal finance portfolio
- Investment simulation game
- Create financial literacy guide

Practice: Solve 100 financial problems

Month 23: Advanced Problem-Solving & Logic (Week 97-100)

Logic and Reasoning

Topics:

- Logical thinking development
- If-then statements
- Converse and inverse
- Truth tables basics
- Venn diagrams advanced
- Set theory basics
- Union and intersection
- Logical puzzles
- Deductive reasoning
- Inductive reasoning
- Proof by example
- Counter-examples

Projects built:

- Logic puzzle collection
- Venn diagram investigations
- Create reasoning game

Practice: Solve 100 logic problems

Problem-Solving Strategies Mastery

Topics:

- Polya's problem-solving method
- Understanding the problem
- Devising a plan
- Carrying out the plan
- Looking back and checking
- Multiple solution methods
- Working backwards mastery
- Guess and check refinement
- Make it simpler strategy
- Look for patterns advanced
- Use a model or diagram
- Eliminate possibilities

Projects built:

- Problem-solving strategy guide
- Multi-solution problem collection
- Create strategy reference

Practice: Apply strategies to 100 problems

Mathematical Investigations

Topics:

- Open-ended problem exploration
- Mathematical conjectures
- Testing hypotheses
- Finding counterexamples
- Generalizing patterns
- Mathematical communication
- Writing mathematical explanations
- Justifying solutions
- Peer review of solutions
- Mathematical debates
- Creating mathematical questions
- Research projects

Projects built:

- Mathematical investigation journal
- Pattern investigation report
- Create math research project

Practice: Complete 20 investigations

Competition Mathematics

Topics:

- Math competition formats
- Time management strategies
- Mental math advanced techniques
- Estimation strategies advanced
- Number sense development
- Pattern recognition mastery
- Lateral thinking problems
- Multi-step problem strategies
- Competition problem types
- Practice competition rounds
- Analyzing solutions
- Learning from mistakes

Projects built:

- Competition prep handbook
- Practice competition set
- Create math challenge book

Practice: Solve 100 competition-level problems

Month 24: Culmination and Future Preparation (Week 101-104)

Real-World Mathematics Applications

Topics:

- Mathematics in architecture
- Mathematics in art and design
- Mathematics in nature
- Mathematics in music
- Mathematics in sports
- Mathematics in cooking
- Mathematics in travel
- Mathematics in technology
- Mathematics in business
- Mathematics in science
- Career connections
- Future mathematics pathways

Projects built:

- FINAL CAPSTONE: Mathematics in My World
- Cross-curricular mathematics project
- Real-world problem-solving portfolio
- Mathematics career exploration

Comprehensive Review and Integration

Topics:

- Number and operations mastery review
- Algebraic thinking review
- Geometry and measurement review
- Data analysis review
- Problem-solving strategies review
- Making connections across topics
- Identifying strengths and gaps
- Creating personal math reference
- Reflection on learning journey
- Setting future goals
- Preparation for middle school math
- Celebration of achievements

Deliverables:

- Complete mathematics portfolio
- Personal reference handbook
- Skills assessment report
- Learning reflection essay
- Future learning plan
- Peer teaching demonstration
- Parent presentation
- Achievement showcase

Transition to Advanced Mathematics

Topics:

- Preview of middle school mathematics
- Introduction to advanced algebra
- Geometry preview
- Advanced problem-solving
- Study skills for mathematics
- Using technology in math
- Online mathematics resources
- Mathematics communities
- Continuing education options
- Summer mathematics activities

- Maintaining skills
- Lifelong learning in mathematics

Deliverables:

- Future learning roadmap
- Resource compilation
- Summer practice plan
- Mathematics goals document
- Skill maintenance schedule
- Advanced topic preview
- Personal achievement record

Assessment: FINAL COMPREHENSIVE EXAM - Complete elementary mathematics mastery evaluation

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Number books, Counting games, Shape art, Measurement labs, Time/money projects
- Phase 2: Multiplication championship, Division guides, Fraction cookbooks, Decimal calculators, Geometry portfolios
- Phase 3: Large number investigations, Pattern discoveries, Ratio/proportion applications, Percentage shopping, Circle geometry art
- Phase 4: Algebra games, Statistics reports, Probability experiments, Logic puzzles, Real-world applications

Total projects: 100+ projects ranging from simple to complex

SKILLS THE STUDENT WILL HAVE AT THE END

- Number Sense: Counting, place value, number relationships, mental math
- Operations: Addition, subtraction, multiplication, division fluency
- Fractions/Decimals: Operations, conversions, real-world applications
- Measurement: Length, weight, capacity, time, money, temperature
- Geometry: 2D/3D shapes, angles, transformations, coordinate geometry
- Data Analysis: Graphs, statistics, interpretation, presentation
- Algebra: Variables, expressions, equations, functions, patterns
- Problem-Solving: Multiple strategies, logical reasoning, mathematical communication
- Applications: Financial literacy, real-world connections, STEM foundations

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-5 hours
 Hands on activities: 5-7 hours
 Practice problems: 3-4 hours
 Projects: 2-3 hours
 Games and puzzles: 1-2 hours
 Total per week: 10-15 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving sessions
 Mentorship: 1-on-1 tutoring available
 Community: Parent and student forums
 Progress tracking: Detailed progress reports
 Parent guides: Resources for parent support
 Adaptive learning: Personalized learning paths

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Final certificate: Elementary Mathematics Mastery Certificate
 Skill badges: Digital badges for specific skills
 Portfolio: Complete work portfolio
 Assessment reports: Detailed skill assessments

WHERE THIS CAN LEAD

- Strong mathematical foundation for middle school
- Problem-solving skills for life
- Preparation for advanced mathematics
- Competition mathematics readiness
- STEM career foundation

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What age group is this Elementary Mathematics course designed for?

A: This comprehensive course is designed for students ages 6-12 (Grades 1-5), though it can also help older students fill knowledge gaps or adults learning mathematics fundamentals. The curriculum adapts to each learner's pace with age-appropriate explanations and activities.

Q: My child struggles with math anxiety. How does this course help?

A: We understand math anxiety is real! Our course uses fun games, visual learning, hands-on projects, and encouraging feedback to build confidence. We start with foundational concepts and progress gradually, celebrating small wins. Most students who start anxious end up loving math within weeks.

Q: What topics are covered in this 24-month Elementary Mathematics program?

A: The course covers: Number sense and operations, Addition/Subtraction/Multiplication/Division mastery, Fractions and Decimals, Measurement and Geometry, Data analysis and Graphing, Pre-Algebra foundations, Problem-solving strategies, Statistics and Probability basics, and real-world math applications. It's a complete K-5 math foundation.

Q: How is this different from school math tutoring?

A: Unlike traditional tutoring that focuses on homework help, this course builds deep conceptual understanding through visual learning, games, and projects. Students don't just memorize procedures - they understand WHY math works. This creates confident problem-solvers who excel in any math situation.

Q: How much time should my child spend on this course each week?

A: We recommend 10-15 hours per week, broken into daily sessions of 1.5-2 hours. However, the course is flexible - students can adjust their pace. Consistency is more important than duration. Even 45 minutes daily produces excellent results over time.

Q: Will this prepare my child for competitive exams and olympiads?

A: Absolutely! The course includes advanced problem-solving, logic puzzles, and competition-style questions. Students completing this program are well-prepared for Math Olympiad, NTSE preparation, school competitions, and develop the strong foundation needed for advanced mathematics in middle school and beyond.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/elementary-mathematics-complete-masterclass>

=====

COURSE 52 OF 118: ETHICAL HACKING & CYBERSECURITY: ZERO TO PENTESTER

=====

Course link: <https://learn.modernagecoders.com/courses/ethical-hacking-masterclass-complete>

Subject area: Cybersecurity & Ethical Hacking

Designed for: College students

Who can join: Complete Beginner to Professional Ethical Hacker

Course length: 6 months (26 weeks) - Can be taken as 1.5 hour daily sessions

Time needed: 1-1.5 hours per session, 10-15 hours/week recommended

Certificate: Industry-recognized Ethical Hacker certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Ethical hacking and cybersecurity course covering penetration testing, OWASP Top 10, network and wireless hacking, and bug bounty hunting, from zero to pro.

In one line: From Zero Knowledge to Professional Penetration Tester

ABOUT THE PROGRAMME

This is not just a hacking course, it's a complete transformation into a cybersecurity professional. Whether you're a complete beginner with no technical background, an IT professional wanting to transition into security, or a developer aiming to become a penetration tester, this comprehensive masterclass will turn you into a highly skilled ethical hacker capable of identifying, exploiting, and securing vulnerabilities in real-world systems. You'll master ethical hacking from ground zero to expert level: from networking fundamentals and Linux basics to advanced exploitation techniques, from web application hacking to mobile security, from wireless attacks to social engineering. By the end, you'll have performed 100+ hands-on labs, compromised various systems ethically, written professional security reports, and be ready for cybersecurity roles at top organizations.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no prior knowledge required
- Legal and ethical hacking methodology throughout
- Hands-on labs with real vulnerable applications
- Latest tools and techniques used by professionals
- Bug bounty hunting preparation included

- Industry certifications preparation (CEH, OSCP prep)
- Mobile and IoT security covered
- Cloud security and DevSecOps practices
- Real-world scenarios and CTF challenges
- Build impressive security portfolio
- Professional report writing skills

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Weeks 1-8): Networking, Linux, Programming Basics, Security Fundamentals

Phase 2: Core Hacking (Weeks 9-16): Network Penetration, Web Application Security, System Hacking

Phase 3: Advanced Security (Weeks 17-24): Wireless, Mobile, Cloud Security, Exploitation Development

Phase 4: Professional Skills (Weeks 25-26): Bug Bounty, Report Writing, Career Development

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required

Coding experience: Basic computer skills helpful but not required

Mathematics: Basic math understanding

Age: 16+ years recommended

Equipment: Computer with 8GB+ RAM, ability to run VMs

Time commitment: 10-15 hours per week minimum

English: Good English comprehension

Motivation: Strong interest in cybersecurity

Learning style: Hands-on learner, problem solver

Ethics: Commitment to ethical hacking only

WHO THIS COURSE IS FOR

Beginners: Complete beginners wanting to enter cybersecurity

IT professionals: IT staff transitioning to security roles

Developers: Programmers wanting to understand security

Students: Computer science students specializing in security

Network admins: Network professionals adding security skills

Compliance officers: Compliance staff needing technical knowledge

Auditors: IT auditors wanting hands-on skills

Law enforcement: Digital forensics and cybercrime investigation

Military: Cyber warfare and defense personnel

Entrepreneurs: Founders building security products

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: NETWORKING, LINUX & SECURITY FOUNDATIONS (WEEKS 1-8) -----

Build rock-solid networking and Linux foundations essential for ethical hacking. Master TCP/IP, operating systems, programming basics, and security fundamentals.

Weeks 1-4: Networking & Linux Fundamentals (Week 1-4)

Introduction to Ethical Hacking & Networking Basics

Topics:

- What is ethical hacking and why it matters
- Legal considerations and ethical guidelines
- Hacking methodology: reconnaissance to reporting
- Setting up hacking lab environment safely
- VirtualBox/VMware installation and configuration
- Kali Linux installation and initial setup
- OSI model deep dive: all 7 layers
- TCP/IP protocol suite fundamentals
- IP addressing and subnetting mastery
- IPv4 vs IPv6 differences and implications
- MAC addresses and ARP protocol
- Network devices: routers, switches, firewalls
- DNS fundamentals and exploitation basics
- HTTP/HTTPS protocol analysis
- Common ports and services

Projects built:

- Complete hacking lab setup with multiple VMs
- Network topology design and implementation
- Subnet calculator tool development
- Protocol analyzer using Wireshark

Practice: Configure 10 different network scenarios, analyze 50 packet captures

Linux Mastery for Hackers

Topics:

- Linux file system hierarchy

- Essential Linux commands for hackers
- File permissions and ownership
- User and group management
- Process management and monitoring
- Package management (apt, yum, dpkg)
- Shell scripting fundamentals
- Bash scripting for automation
- Text manipulation: grep, sed, awk
- Network commands: netstat, ss, lsof
- Service management with systemctl
- Log file analysis and monitoring
- Cron jobs and task scheduling
- SSH configuration and hardening
- Linux security basics

Projects built:

- Automation scripts for reconnaissance
- Custom security monitoring dashboard
- Log analysis automation tool
- Linux hardening checklist implementation

Practice: Complete 100 Linux challenges, write 20 bash scripts

Programming for Hackers - Python & Web Basics

Topics:

- Python fundamentals for security
- Socket programming in Python
- Network programming with Scapy
- Web scraping with BeautifulSoup
- Requests library for HTTP interactions
- Regular expressions for pattern matching
- HTML, CSS, JavaScript basics
- Understanding web application architecture
- Client-server communication model
- RESTful APIs and JSON
- SQL fundamentals and injection basics
- XML and XXE vulnerability basics
- Encoding and encryption basics
- Hashing algorithms and their weaknesses
- Cryptography fundamentals

Projects built:

- Port scanner in Python
- Password cracker for weak hashes
- Web crawler for reconnaissance
- Network packet sniffer
- Simple vulnerability scanner

Practice: Write 30 security tools in Python

Information Gathering & OSINT

Topics:

- Open Source Intelligence (OSINT) fundamentals
- Google dorking and advanced search operators
- Social media reconnaissance techniques
- Email harvesting and verification
- DNS enumeration and zone transfers
- Subdomain discovery techniques
- WHOIS and domain information gathering
- Shodan for device discovery
- Censys and other search engines
- Metadata extraction from documents
- Website technology identification
- Company and employee reconnaissance
- Physical location information gathering
- Dark web basics for intelligence
- OSINT frameworks and automation

Projects built:

- Complete OSINT profile builder
- Automated reconnaissance framework
- Company security assessment report
- Social engineering information gatherer
- Custom OSINT tool development

Practice: Perform OSINT on 20 targets (legally and ethically)

Weeks 5-8: Scanning & Enumeration (Week 5-8)

Network Scanning & Enumeration

Topics:

- Network scanning methodology
- Nmap fundamentals and advanced techniques
- TCP connect and SYN stealth scanning
- UDP scanning challenges and techniques
- Service version detection
- Operating system fingerprinting
- Nmap Scripting Engine (NSE)

- Vulnerability scanning with Nessus
- OpenVAS configuration and usage
- Banner grabbing techniques
- SNMP enumeration
- SMB/NetBIOS enumeration
- LDAP enumeration
- NFS enumeration
- Database enumeration

Projects built:

- Network mapping automation tool
- Vulnerability assessment report generator
- Custom Nmap scripts development
- Enumeration framework creation
- Automated scanning pipeline

Practice: Scan and enumerate 30 different networks/systems

Web Application Scanning

Topics:

- Web application architecture review
- Burp Suite Professional setup
- Proxy configuration and SSL handling
- Spidering and crawling applications
- Directory and file brute forcing
- Parameter discovery and fuzzing
- Technology stack identification
- Web application firewalls (WAF) detection
- API endpoint discovery
- JavaScript analysis for endpoints
- Source code review basics
- Automated scanning with Burp Scanner
- OWASP ZAP usage and automation
- Nikto and dirb for web scanning
- Custom wordlist creation

Projects built:

- Web application reconnaissance tool
- API endpoint discovery framework
- WAF detection and bypass tool
- Custom web fuzzer development
- Automated web scanning report

Practice: Scan 25 web applications, find 100 endpoints

Vulnerability Assessment

Topics:

- Vulnerability assessment methodology
- CVSS scoring understanding
- CVE database navigation
- Exploit database usage
- Manual vulnerability verification
- False positive identification
- Risk assessment and prioritization
- Vulnerability chaining concepts
- Patch management basics
- Compliance scanning (PCI DSS, HIPAA)
- Creating vulnerability matrices
- Remediation recommendations
- Phase 1 comprehensive review

Projects built:

- PHASE 1 CAPSTONE: Complete Security Assessment
- Target: Vulnerable VM (Metasploitable, DVWA)
- Tasks: Full recon, scanning, enumeration, vulnerability assessment, professional report

Assessment: Phase 1 Exam - Networking, Linux, Scanning, Enumeration

----- PHASE 2: CORE HACKING & EXPLOITATION (WEEKS 9-16) -----

Master system hacking, web application attacks, exploitation frameworks, and post-exploitation techniques.

Weeks 9-12: System Hacking & Exploitation (Week 9-12)

System Hacking & Password Attacks

Topics:

- Windows architecture and vulnerabilities
- Linux privilege escalation techniques
- Password attack methodology
- Rainbow tables and time-memory trade-off
- John the Ripper mastery
- Hashcat for GPU cracking
- Hydra for online attacks
- Pass-the-hash attacks
- Kerberoasting and ASREPROasting
- Credential dumping with Mimikatz

- SAM and SYSTEM file extraction
- Keyloggers and screenshot capture
- Password policy bypassing
- Multi-factor authentication bypasses
- Physical access attacks

Projects built:

- Password cracking framework
- Credential harvesting tool
- Windows privilege escalation script
- Linux privilege escalation checklist
- Custom keylogger development

Practice: Crack 50 passwords, escalate privileges on 20 systems

Metasploit Framework & Exploitation

Topics:

- Metasploit architecture and components
- MSFconsole navigation and usage
- Exploit selection and configuration
- Payload types and encoding
- Meterpreter advanced features
- Post-exploitation with Metasploit
- Persistence mechanisms
- Pivoting through compromised hosts
- Port forwarding and tunneling
- Custom exploit development basics
- Ruby for Metasploit modules
- Evasion techniques and encoding
- Anti-virus bypass methods
- Exploit chaining strategies
- Manual exploitation techniques

Projects built:

- Custom Metasploit module
- Automated exploitation framework
- Post-exploitation toolkit
- Persistence mechanism generator
- AV evasion payload creator
- Exploit chain automation
- Pivoting lab setup

Practice: Exploit 30 vulnerable systems, develop 10 custom payloads

Web Application Hacking - OWASP Top 10

Topics:

- SQL Injection: union, blind, time-based
- SQLMap advanced usage
- Cross-Site Scripting (XSS): reflected, stored, DOM
- XSS filter bypasses and polyglots
- Cross-Site Request Forgery (CSRF)
- XML External Entity (XXE) attacks
- Server-Side Request Forgery (SSRF)
- Insecure Deserialization exploitation
- Command injection techniques
- Directory traversal and LFI/RFI
- File upload vulnerabilities
- Authentication and session flaws
- Broken access control exploitation
- Security misconfiguration attacks
- Using Burp Suite for exploitation

Projects built:

- SQL injection automation tool
- XSS payload generator
- Web shell development
- Session hijacking tool
- Automated OWASP scanner
- Custom Burp extension
- Web exploitation framework

Practice: Exploit 50 web vulnerabilities across all OWASP Top 10

Advanced Web Attacks & API Security

Topics:

- Modern JavaScript framework vulnerabilities
- React, Angular, Vue.js security issues
- WebSocket vulnerabilities
- GraphQL injection and attacks
- REST API vulnerabilities
- JWT token manipulation
- OAuth and SAML attacks
- Race conditions and TOCTOU
- Cache poisoning attacks
- HTTP request smuggling
- Subdomain takeover techniques
- CORS misconfiguration exploitation
- Business logic vulnerabilities

- Payment gateway bypasses
- Two-factor authentication bypasses

Projects built:

- API security testing framework
- JWT token manipulation tool
- Race condition exploit tool
- Business logic flaw detector
- Modern web app scanner

Practice: Test 20 APIs, find 30 logic flaws

Database Hacking & NoSQL Injection

Topics:

- Database architecture and weaknesses
- Advanced SQL injection techniques
- Oracle, MySQL, PostgreSQL specific attacks
- MongoDB injection attacks
- Redis exploitation techniques
- Elasticsearch vulnerabilities
- Database privilege escalation
- Data exfiltration techniques
- Blind injection automation
- Time-based extraction optimization
- NoSQL injection payloads
- GraphQL database attacks
- ORM injection vulnerabilities

Projects built:

- Database exploitation framework
- NoSQL injection scanner
- Blind injection optimizer
- Data exfiltration tool
- Database security assessment tool

Practice: Exploit 25 database vulnerabilities

Weeks 13-16: Network Attacks & Post-Exploitation (Week 13-16)

Network Attacks & Man-in-the-Middle

Topics:

- ARP poisoning and spoofing
- DNS spoofing and hijacking
- DHCP starvation attacks
- VLAN hopping techniques
- STP manipulation
- BGP hijacking basics
- SSL/TLS vulnerabilities
- SSL stripping attacks
- Evil twin attacks
- Rogue access points
- SMB relay attacks
- LLMNR/NBT-NS poisoning
- Responder tool usage
- IPv6 attack vectors
- Network sniffing at scale

Projects built:

- MITM attack framework
- Network poisoning toolkit
- SSL stripping tool
- Rogue DHCP server
- Traffic manipulation proxy
- Network attack automation

Practice: Perform 30 network attacks in lab environment

Wireless Network Hacking

Topics:

- Wireless networking fundamentals
- 802.11 protocol weaknesses
- Monitor mode and packet injection
- WEP cracking techniques
- WPA/WPA2 handshake capture
- WPA2 cracking with hashcat
- WPS attacks with Reaver/Bully
- PMKID attack methodology
- Evil twin and karma attacks
- Captive portal bypassing
- Bluetooth hacking basics
- Software Defined Radio (SDR) basics
- RFID and NFC hacking
- Wireless IDS/IPS evasion
- Creating custom rogue APs

Projects built:

- Automated WiFi cracking tool
- Handshake capture framework
- Rogue AP with captive portal

- Bluetooth exploitation tool
- Wireless security assessment kit

Practice: Crack 20 wireless networks, create 10 rogue APs

Post-Exploitation & Lateral Movement

Topics:

- Post-exploitation methodology
- Maintaining access techniques
- Backdoor installation methods
- Rootkit basics and detection
- Living off the land (LOLBins)
- PowerShell for post-exploitation
- Windows privilege escalation
- Linux privilege escalation
- Lateral movement strategies
- Pass-the-hash/ticket attacks
- Kerberos attacks in depth
- Active Directory exploitation
- Domain dominance techniques
- Data exfiltration methods
- Covering tracks and log cleaning

Projects built:

- Post-exploitation framework
- Persistence toolkit
- Lateral movement automation
- AD exploitation toolkit
- Data exfiltration tool
- Anti-forensics toolkit

Practice: Complete 20 post-exploitation scenarios

Malware Analysis & Reverse Engineering Basics

Topics:

- Malware types and categories
- Static malware analysis
- Dynamic malware analysis
- Sandboxing and VM detection
- Basic assembly language
- x86/x64 architecture basics
- Debugging with OllyDbg/x64dbg
- IDA Pro basics
- Ghidra for reverse engineering
- Packing and unpacking
- Anti-analysis techniques
- Behavioral analysis
- Network traffic analysis
- Memory forensics basics
- Indicators of Compromise (IoCs)

Projects built:

- Malware analysis lab setup
- Simple malware analyzer
- Unpacker tool development
- Behavioral analysis framework
- IoC extraction tool

Practice: Analyze 15 malware samples safely

Phase 2 Capstone Project

Topics:

- Complete penetration test planning
- Scope definition and rules of engagement
- Full attack chain execution
- Exploitation and post-exploitation
- Lateral movement demonstration
- Data exfiltration simulation
- Professional reporting

Projects built:

- PHASE 2 CAPSTONE: Full Penetration Test
- Target: Corporate network simulation (multiple systems)
- Tasks: Complete penetration test from external to domain admin
- Deliverables: Executive report, technical report, remediation guide

Assessment: Phase 2 Final Exam - System and Web Application Hacking

Additional Phase 2 Content (Week 9-16 (distributed))

Active Directory Attacks

Topics:

- Active Directory fundamentals
- Domain enumeration with PowerView
- BloodHound for AD mapping
- Kerberoasting deep dive
- ASREPROasting techniques
- Golden ticket attacks
- Silver ticket attacks
- DCSync attacks

- Trust relationship abuse
- Group Policy exploitation
- LAPS bypass techniques
- Constrained delegation abuse
- Forest compromise strategies
- Azure AD attacks
- Hybrid environment exploitation

Projects built:

- AD enumeration toolkit
- Kerberos attack framework
- BloodHound custom queries
- Domain persistence toolkit
- Azure AD exploitation tool

Practice: Compromise 10 Active Directory environments

Mobile Application Security

Topics:

- Android architecture and security
- iOS security model
- Mobile app reverse engineering
- APK decompilation and analysis
- Smali/baksmali usage
- Frida for dynamic analysis
- SSL pinning bypass
- Root/jailbreak detection bypass
- Mobile app traffic interception
- Insecure data storage
- Weak cryptography in mobile apps
- Intent vulnerabilities
- Deep link exploitation
- Mobile malware basics
- MDM bypass techniques

Projects built:

- Mobile app security scanner
- APK analysis automation
- Frida script collection
- SSL pinning bypass tool
- Mobile pentest framework

Practice: Test 20 mobile applications

Cloud Security & Container Hacking

Topics:

- Cloud security fundamentals
- AWS security and exploitation
- S3 bucket misconfigurations
- IAM privilege escalation
- Lambda function attacks
- Azure security issues
- GCP vulnerabilities
- Kubernetes security
- Docker escape techniques
- Container image vulnerabilities
- Secrets management issues
- Terraform security
- CI/CD pipeline attacks
- Serverless security
- Cloud metadata exploitation

Projects built:

- Cloud security assessment tool
- S3 bucket scanner
- Container escape toolkit
- Kubernetes exploitation framework
- Cloud misconfiguration scanner

Practice: Assess 15 cloud environments

IoT & ICS Security

Topics:

- IoT architecture and protocols
- MQTT vulnerabilities
- CoAP exploitation
- Firmware extraction and analysis
- JTAG and UART interfaces
- Side-channel attacks basics
- Industrial Control Systems (ICS)
- SCADA security
- Modbus protocol attacks
- OT/IT convergence risks
- Smart home device hacking
- Automotive security basics
- Medical device security
- Hardware hacking tools
- IoT botnets

Projects built:

- IoT device scanner
- Firmware analysis toolkit
- MQTT exploitation tool
- Smart home security tester
- ICS attack simulator

Practice: Hack 10 IoT devices

Physical Security & Social Engineering

Topics:

- Physical security assessments
- Lock picking basics
- RFID/NFC cloning
- Badge cloning techniques
- Tailgating and piggybacking
- Social engineering principles
- Pretexting techniques
- Phishing campaign creation
- Vishing and smishing
- Spear phishing strategies
- Watering hole attacks
- USB drop attacks
- Evil maid attacks
- Dumpster diving value
- OSINT for social engineering

Projects built:

- Phishing framework setup
- Social engineering toolkit usage
- Physical security checklist
- USB payload generator
- SE campaign automation

Practice: Design 10 social engineering scenarios

PHASE 3: ADVANCED SECURITY & SPECIALIZED DOMAINS (WEEKS 17-24)

Master advanced exploitation, zero-day research, exploit development, and specialized security domains.

Weeks 17-20: Advanced Exploitation & Zero Days (Week 17-20)

Buffer Overflow Exploitation

Topics:

- Memory layout and stack structure
- Buffer overflow theory
- Stack-based buffer overflows
- EIP/RIP control
- Shellcode development
- NOP sleds and reliability
- Bad character analysis
- Return-to-libc attacks
- Heap overflow basics
- Format string vulnerabilities
- Integer overflows
- Off-by-one errors
- Fuzzing fundamentals
- Exploit development with Python
- Debugging exploits with GDB

Projects built:

- Custom shellcode development
- Buffer overflow exploit framework
- Fuzzer development
- Exploit reliability enhancer
- ROP chain generator
- Heap exploitation tool
- Format string exploit builder

Practice: Develop 20 working exploits from scratch

Modern Exploit Mitigation Bypasses

Topics:

- ASLR bypass techniques
- DEP/NX bypass methods
- Stack canaries bypass
- Return-oriented programming (ROP)
- ROP chain construction
- Gadget finding tools
- SEHOP bypass
- Control Flow Guard (CFG) bypass
- Kernel exploitation basics
- Use-after-free vulnerabilities
- Race condition exploitation
- Sandbox escapes

- Browser exploitation basics
- 0-day research methodology
- Responsible disclosure

Projects built:

- ASLR bypass exploit
- ROP chain automation tool
- Kernel exploit development
- Browser exploit PoC
- Sandbox escape framework

Practice: Bypass 15 different security mitigations

Web Application Firewall Bypasses

Topics:

- WAF detection techniques
- Common WAF products and weaknesses
- Encoding and obfuscation techniques
- Unicode and UTF-8 bypasses
- HTTP parameter pollution
- HTTP verb tampering
- Case variation attacks
- Comment injection techniques
- White space variations
- SQL injection WAF bypasses
- XSS filter evasion
- XXE WAF bypasses
- Rate limiting bypasses
- IP rotation techniques
- CDN and reverse proxy bypasses

Projects built:

- WAF fingerprinting tool
- Automated bypass generator
- Payload obfuscation framework
- WAF testing methodology
- Custom evasion toolkit

Practice: Bypass 20 different WAF configurations

Cryptography & Cryptanalysis

Topics:

- Cryptography fundamentals review
- Symmetric encryption algorithms
- Asymmetric encryption systems
- Hashing algorithms and collisions
- Digital signatures and certificates
- PKI infrastructure attacks
- SSL/TLS vulnerabilities deep dive
- Padding oracle attacks
- CBC bit flipping
- ECB mode vulnerabilities
- Weak random number generators
- Side-channel attacks
- Timing attacks
- Quantum computing threats
- Blockchain security basics

Projects built:

- Padding oracle exploit tool
- Certificate pinning bypass
- Weak crypto detector
- Hash collision finder
- Timing attack framework
- Blockchain vulnerability scanner

Practice: Break 25 weak cryptographic implementations

Red Team Operations

Topics:

- Red team vs penetration testing
- Adversary simulation
- MITRE ATT&CK framework
- Threat actor emulation
- Command and control (C2) setup
- Cobalt Strike usage
- Empire framework
- Covenant C2 platform
- Infrastructure setup and management
- Domain fronting
- Operational security (OPSEC)
- Anti-forensics techniques
- Long-term persistence
- Data staging and exfiltration
- Purple team exercises

Projects built:

- Custom C2 framework
- Red team infrastructure automation

- OPSEC checklist implementation
- Adversary emulation playbook
- Purple team exercise design

Practice: Complete 5 red team scenarios

Weeks 21-24: DevSecOps & Automation (Week 21-24)

DevSecOps & CI/CD Security

Topics:

- DevSecOps principles and practices
- Shift-left security approach
- SAST tools and implementation
- DAST in CI/CD pipelines
- IAST and RASP technologies
- Container security scanning
- Infrastructure as Code security
- Secrets management in CI/CD
- Git security and secret scanning
- Jenkins security hardening
- GitLab CI/CD security
- GitHub Actions security
- Supply chain security
- SBOM generation and analysis
- Compliance as code

Projects built:

- Secure CI/CD pipeline
- Automated security testing framework
- Container security scanner integration
- IaC security validator
- Secret scanning automation

Practice: Secure 10 CI/CD pipelines

Security Automation & Orchestration

Topics:

- Security orchestration platforms
- SOAR implementation
- Playbook development
- Automated incident response
- Threat intelligence integration
- API security automation
- Vulnerability management automation
- Compliance automation
- Security metrics and KPIs
- Dashboard creation
- Alerting and notification systems
- Integration with ticketing systems
- Automated reporting
- Machine learning for security
- AI-powered threat detection

Projects built:

- SOAR platform setup
- Automated incident response system
- Security metrics dashboard
- Threat intelligence aggregator
- ML-based anomaly detector

Practice: Automate 20 security workflows

Threat Hunting & Intelligence

Topics:

- Threat hunting methodology
- Hypothesis-driven hunting
- Threat intelligence lifecycle
- OSINT for threat intelligence
- Dark web monitoring
- Threat actor tracking
- IOC and YARA rules
- SIGMA rules creation
- Threat hunting with Splunk
- ELK stack for security
- Network flow analysis
- Endpoint detection techniques
- Memory analysis for threats
- Threat intelligence platforms
- Information sharing (STIX/TAXII)

Projects built:

- Threat hunting playbook
- YARA rule repository
- Threat intelligence platform
- Automated threat hunter
- IOC extraction framework

Practice: Hunt for threats in 15 environments

Incident Response & Forensics

Topics:

- Incident response methodology
- NIST incident response framework
- Evidence collection and preservation
- Chain of custody
- Disk forensics and imaging
- File system analysis
- Memory forensics with Volatility
- Network forensics
- Timeline analysis
- Windows forensics artifacts
- Linux forensics artifacts
- macOS forensics basics
- Mobile device forensics
- Cloud forensics challenges
- Forensics report writing

Projects built:

- Incident response toolkit
- Automated evidence collector
- Memory analysis framework
- Timeline generator
- Forensics report template

Practice: Investigate 15 security incidents

Compliance & Governance

Topics:

- Security frameworks overview
- NIST Cybersecurity Framework
- ISO 27001/27002
- PCI DSS requirements
- GDPR and privacy
- HIPAA compliance
- SOC 2 requirements
- Risk assessment methodologies
- Business impact analysis
- Security policies and procedures
- Security awareness training
- Vendor risk management
- Security metrics and reporting
- Audit preparation

Projects built:

- Compliance assessment tool
- Risk assessment framework
- Policy template library
- Compliance dashboard
- Audit checklist generator

Practice: Perform 5 compliance assessments

Additional Phase 3 Content (Week 17-24 (distributed))

Zero Trust Architecture

Topics:

- Zero trust principles
- Identity-centric security
- Microsegmentation strategies
- Software-defined perimeter
- Identity and access management
- Privileged access management
- Multi-factor authentication
- Conditional access policies
- Device trust and compliance
- Network access control
- Zero trust network access
- Application segmentation
- Data classification and protection
- Zero trust monitoring
- Migration strategies

Projects built:

- Zero trust assessment tool
- Microsegmentation design
- Identity verification system
- Device trust validator
- Zero trust roadmap

Practice: Design 5 zero trust architectures

Advanced Malware Development

Topics:

- Malware architecture patterns
- Process injection techniques
- DLL injection methods
- Process hollowing
- Thread hijacking

- Reflective DLL injection
- Fileless malware techniques
- Living off the land
- Anti-debugging techniques
- VM detection and evasion
- Polymorphic code
- Metamorphic techniques
- Crypters and packers
- C2 communication methods
- Data exfiltration techniques

Projects built:

- Custom malware framework
- Process injection toolkit
- Anti-analysis toolkit
- Polymorphic engine
- Custom C2 protocol

Practice: Develop 10 evasive malware samples

Blockchain & Smart Contract Security

Topics:

- Blockchain technology overview
- Cryptocurrency security
- Smart contract basics
- Solidity security issues
- Reentrancy attacks
- Integer overflow/underflow
- Access control issues
- Front-running attacks
- Flash loan attacks
- Oracle manipulation
- Smart contract auditing
- Web3 security
- Wallet security
- DeFi protocol risks
- NFT security considerations

Projects built:

- Smart contract scanner
- Blockchain security toolkit
- DeFi vulnerability finder
- Smart contract fuzzer
- Web3 security framework

Practice: Audit 10 smart contracts

AI/ML Security

Topics:

- ML model security threats
- Adversarial attacks on ML
- Data poisoning attacks
- Model extraction attacks
- Privacy attacks on ML models
- Backdoor attacks in ML
- Evasion attacks
- Model inversion attacks
- Membership inference
- Differential privacy
- Federated learning security
- Secure multi-party computation
- AI ethics and bias
- Explainable AI for security
- ML for cybersecurity

Projects built:

- Adversarial example generator
- Model security scanner
- Privacy-preserving ML tool
- ML attack framework
- AI security assessment

Practice: Attack and defend 10 ML models

Phase 3 Capstone Project

Topics:

- Advanced security assessment
- Multi-vector attack scenarios
- Complex exploitation chains
- Advanced persistence methods
- Comprehensive security testing
- Professional documentation

Projects built:

- MAJOR CAPSTONE: Advanced Security Assessment
- Option 1: Red Team Operation (full adversary simulation)
- Option 2: Zero-day research and exploit development
- Option 3: Advanced malware analysis and reverse engineering
- Option 4: Cloud security assessment (AWS/Azure/GCP)

- Requirements: Advanced techniques, custom tools, comprehensive report
- Assessment: Phase 3 Final Exam - Advanced Security Comprehensive Test

PHASE 4: PROFESSIONAL SKILLS & CAREER DEVELOPMENT (WEEKS 25-26)

Master bug bounty hunting, professional reporting, certification preparation, and career development.

Week 25: Bug Bounty & Professional Skills

Bug Bounty Hunting Mastery

Topics:

- Bug bounty platforms overview
- HackerOne, Bugcrowd, Synack
- Program selection strategies
- Scope analysis and targeting
- Vulnerability severity assessment
- Report writing for bounties
- Proof of concept creation
- Responsible disclosure process
- Communication with security teams
- Bounty negotiation tactics
- Time management for bug bounty
- Automation for efficiency
- Collaboration and information sharing
- Building reputation
- Income strategies

Projects built:

- Bug bounty automation toolkit
- Report template library
- Vulnerability tracking system
- Recon automation framework
- Bounty hunting dashboard

Practice: Submit 10 vulnerability reports

Professional Report Writing

Topics:

- Executive summary writing
- Technical documentation standards
- Vulnerability description best practices
- Risk rating and impact analysis
- CVSS scoring accuracy
- Evidence presentation
- Screenshot and video documentation
- Proof of concept guidelines
- Remediation recommendations
- Report formatting and structure
- Client communication
- Legal considerations
- NDA and contract understanding
- Deliverable management
- Post-engagement activities

Projects built:

- Professional report templates
- Automated report generator
- Evidence collection framework
- Risk assessment calculator
- Client communication toolkit

Practice: Write 10 professional security reports

Security Certifications Preparation

Topics:

- CEH (Certified Ethical Hacker) prep
- OSCP preparation strategy
- CompTIA Security+ overview
- CompTIA PenTest+
- GIAC certifications
- CISSP domains review
- Cloud security certifications
- Vendor-specific certifications
- Study strategies and resources
- Practice exam techniques
- Lab environment setup
- Time management for exams
- Continuing education
- Certification maintenance
- Career value of certifications

Projects built:

- Certification study plan
- Practice exam simulator
- Lab exercise collection

- Study group organization
- Knowledge base creation

Practice: Complete 5 practice certification exams

Soft Skills & Consulting

Topics:

- Client communication skills
- Presentation skills for security
- Security awareness training delivery
- Consulting engagement management
- Project scoping and estimation
- Team collaboration
- Knowledge sharing practices
- Documentation standards
- Time and task management
- Continuous learning strategies
- Industry networking
- Conference participation
- Public speaking for security
- Writing for security publications
- Building personal brand

Projects built:

- Security awareness program
- Presentation template library
- Consulting framework
- Personal brand strategy
- Knowledge sharing platform

Practice: Deliver 5 security presentations

Legal & Ethical Considerations

Topics:

- Computer crime laws
- Legal frameworks globally
- Authorization and scope
- Evidence handling legally
- Privacy laws and regulations
- Ethical hacking guidelines
- Professional codes of conduct
- Liability and insurance
- Contracts and agreements
- Intellectual property
- Non-disclosure agreements
- Whistleblowing considerations
- International law considerations
- Law enforcement cooperation
- Expert witness preparation

Projects built:

- Legal checklist for engagements
- Contract template library
- Ethical guidelines document
- Compliance tracker
- Legal resource compilation

Practice: Review 10 legal scenarios

Week 26: Career Launch & Final Assessment

Career Strategy & Job Hunting

Topics:

- Cybersecurity career paths
- Job market analysis
- Resume optimization for security roles
- LinkedIn profile for security professionals
- GitHub portfolio creation
- Technical interview preparation
- Practical assessment preparation
- Salary negotiation
- Freelancing in security
- Starting security consulting
- Building client base
- Networking strategies
- Industry events and conferences
- Online presence building
- Continuous skill development

Projects built:

- Professional portfolio website
- Optimized security resume
- LinkedIn profile optimization
- GitHub security portfolio
- Interview preparation guide
- 100-day career plan

Practice: Apply to 20 security positions

Industry Specializations

Topics:

- Financial services security
- Healthcare security specialization
- Government and defense security
- E-commerce security
- Gaming industry security
- Automotive cybersecurity
- Aviation security
- Critical infrastructure protection
- Retail security
- Educational institution security
- Startup security consulting
- Enterprise security architecture
- Security product development
- Security research roles
- Teaching and training roles

Projects built:

- Industry-specific security framework
- Specialization roadmap
- Industry contact list
- Sector-specific tools
- Industry certification plan

Practice: Research 5 industry sectors deeply

Building Security Tools & Products

Topics:

- Security tool development
- Open source contributions
- Commercial tool development
- SaaS security products
- API development for security
- Security tool documentation
- Marketing security tools
- Licensing and pricing
- Support and maintenance
- Community building
- Tool monetization strategies
- Security research publication
- CVE submission process
- Security blog writing
- YouTube channel creation

Projects built:

- Open source security tool
- Commercial tool prototype
- Security blog launch
- YouTube channel setup
- Tool documentation system

Practice: Develop 3 security tools

Advanced Career Development

Topics:

- Leadership in security
- Team management skills
- Security program development
- Budget management
- Vendor management
- Board presentation skills
- Risk communication
- Security metrics for executives
- Building security culture
- Change management
- Crisis management
- Media relations
- Thought leadership
- Advisory board participation
- Mentoring others

Projects built:

- Security program blueprint
- Leadership development plan
- Mentorship program design
- Executive communication framework
- Career advancement roadmap

Practice: Mentor 3 junior security professionals

Continuous Learning & Growth

Topics:

- Staying current with threats
- Research paper reading
- Security podcast recommendations
- Book recommendations
- Online course platforms
- Virtual lab resources

- Capture The Flag (CTF) participation
- Security competition strategies
- Research collaboration
- Security community involvement
- Conference speaking
- Security research trends
- Emerging technologies
- Future of cybersecurity
- Personal development planning

Projects built:

- Personal learning roadmap
- Resource library creation
- CTF team formation
- Research project proposal
- 5-year career vision

Practice: Participate in 5 CTF competitions

Final Week Activities (Week 26 (continued))

Real-World Scenarios

Topics:

- APT group emulation
- Nation-state attack patterns
- Ransomware incident handling
- Data breach response
- Insider threat detection
- Supply chain attacks
- Zero-day discovery
- Critical vulnerability response
- Emergency patch management
- Crisis communication
- Media handling during incidents
- Regulatory reporting
- Insurance claim processes
- Forensic preservation
- Business continuity

Projects built:

- Incident response runbook
- APT emulation toolkit
- Crisis management plan
- Breach notification templates
- Tabletop exercise design

Practice: Complete 10 realistic scenarios

Security Operations Center (SOC)

Topics:

- SOC architecture and design
- SIEM implementation
- Security monitoring strategies
- Alert triage and prioritization
- Incident classification
- Escalation procedures
- Shift handover processes
- SOC metrics and KPIs
- Tool integration
- Automation in SOC
- Threat intelligence in SOC
- SOC team training
- 24/7 operations management
- Outsourced vs in-house SOC
- SOC maturity models

Projects built:

- SOC setup blueprint
- SIEM use case library
- SOC runbook collection
- Monitoring dashboard
- SOC efficiency toolkit

Practice: Design 3 SOC architectures

Emerging Threats & Technologies

Topics:

- 5G security implications
- Quantum computing threats
- AI-powered attacks
- Deepfake threats
- Supply chain security
- OT/IT convergence risks
- Edge computing security
- Serverless security
- Container orchestration security
- Mesh network security
- Satellite communication security

- Drone security
- Augmented reality security
- Virtual reality security
- Future threat landscape

Projects built:

- Emerging threat research
- Future security framework
- Technology assessment tool
- Threat prediction model
- Innovation security guidelines

Practice: Research 10 emerging technologies

Entrepreneurship in Security

Topics:

- Security startup ideas
- Market research for security
- Business plan development
- Funding and investment
- MVP development
- Customer discovery
- Product-market fit
- Go-to-market strategy
- Security product pricing
- Partnership strategies
- Scaling security business
- Exit strategies
- Acquisition considerations
- Building security team
- Security entrepreneur network

Projects built:

- Security startup blueprint
- Business plan document
- MVP prototype
- Pitch deck creation
- Go-to-market plan

Practice: Develop complete startup concept

Final Assessment & Graduation (Week 26 (final))

Final Capstone Project - Part 1

Topics:

- Project selection and scoping
- Comprehensive security assessment
- Multi-phase attack simulation
- Complex vulnerability chains
- Advanced exploitation techniques
- Custom tool development
- Zero-day research attempt
- Professional documentation
- Video demonstration preparation

Projects built:

- FINAL CAPSTONE: Complete Security Assessment
- Option 1: Full enterprise penetration test with red team tactics
- Option 2: Bug bounty automation framework with AI
- Option 3: Security tool development and release
- Option 4: Zero-day research and responsible disclosure
- Option 5: Advanced malware analysis and threat hunting
- Requirements: Professional quality, innovative approach, comprehensive documentation

Final Capstone Project - Part 2

Topics:

- Project completion
- Quality assurance
- Documentation finalization
- Presentation preparation
- Demo recording
- Portfolio integration
- Peer review
- Instructor feedback
- Project defense
- Publication preparation

Deliverables:

- Complete project code/tools
- Professional security report
- Executive presentation
- Technical documentation
- Video demonstration
- Blog post or article
- GitHub repository
- Portfolio entry
- Lessons learned document
- Future improvements roadmap

Career Launch & Certification

Topics:

- Portfolio showcase
- Final resume polish
- LinkedIn optimization
- Job application strategy
- Interview preparation
- Salary negotiation prep
- Networking plan
- Certification roadmap
- Continuing education plan
- Professional development
- Alumni network access
- Mentorship matching
- Career coaching session
- Goal setting for future
- Graduation celebration

Deliverables:

- Professional portfolio with 100+ labs/projects
- Polished cybersecurity resume
- Optimized LinkedIn profile
- GitHub with security tools
- Personal security blog
- Bug bounty profile
- Network of security professionals
- Certification study plan
- Job applications submitted
- Clear career trajectory

Assessment: FINAL COMPREHENSIVE EXAM - Complete Ethical Hacking Assessment

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-8): 30+ foundational projects - networking tools, Linux scripts, recon frameworks
- Phase 2 (Weeks 9-16): 40+ core hacking projects - exploits, web attacks, post-exploitation tools
- Phase 3 (Weeks 17-24): 35+ advanced projects - zero-days, malware, specialized security tools
- Phase 4 (Weeks 25-26): 15+ professional projects - bug bounty tools, reports, career materials
- Total: 120+ hands-on security projects

Total projects: 120+ security projects covering all domains from basic to advanced

SKILLS THE STUDENT WILL HAVE AT THE END

- Networking: TCP/IP, Routing, Switching, Protocols, Network Security
- Operating Systems: Linux (expert), Windows Security, macOS Basics
- Programming: Python, Bash, PowerShell, JavaScript, SQL, Ruby basics
- Web Security: OWASP Top 10, API Security, Modern Web Vulnerabilities
- System Hacking: Exploitation, Privilege Escalation, Persistence, Lateral Movement
- Network Attacks: MITM, Spoofing, Wireless Hacking, Protocol Exploitation
- Malware: Analysis, Reverse Engineering, Development, Evasion Techniques
- Tools: Metasploit, Burp Suite, Nmap, Wireshark, Kali Linux, Custom Tools
- Cloud Security: AWS, Azure, GCP, Container Security, Kubernetes
- Mobile Security: Android, iOS, Mobile App Testing, Reverse Engineering
- Forensics: Disk, Memory, Network, Mobile, Cloud Forensics
- Reporting: Professional Reports, Executive Communication, Documentation

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-5 hours
Hands on labs: 5-7 hours
Projects: 3-4 hours
Practice challenges: 2-3 hours
Reading research: 1-2 hours
Total per week: 14-21 hours

SUPPORT INCLUDED

Live sessions: Weekly live hacking sessions and Q&A
Mentorship: 1-on-1 mentorship from professional pentesters
Community: Active Discord with security professionals
Lab access: 24/7 access to vulnerable lab environments
Code review: Expert review of tools and exploits
Career support: Resume review, interview prep, job referrals
Bug bounty guidance: Tips and collaboration for bug bounties
Certification prep: Study groups for CEH, OSCP, etc.
Lifetime access: All content and future updates
Legal guidance: Legal framework and ethical guidelines

CERTIFICATION DETAIL

Course certificate: Ethical Hacker Professional Certificate
Phase certificates: Certificate after each phase (4 total)
Specialization certificates: Web Security, Network Security, Cloud Security
Ceu credits: Continuing Education Units for certifications
Linkedin badges: Digital badges for all achievements
Industry prep: Preparation for CEH, OSCP, GPEN, etc.
Portfolio projects: 100+ documented security projects
Bug bounty profile: Established bug bounty hunter profile
Tool releases: Published security tools on GitHub
Recognition: Industry-recognized completion certificate

WHERE THIS CAN LEAD

- Penetration Tester / Ethical Hacker
- Security Analyst / SOC Analyst
- Vulnerability Assessment Specialist
- Security Consultant
- Bug Bounty Hunter
- Security Engineer
- Cloud Security Architect
- Application Security Engineer
- Red Team Operator
- Incident Response Specialist
- Digital Forensics Investigator
- Malware Analyst
- Security Researcher
- Cybersecurity Manager
- Chief Information Security Officer (CISO) track

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-6 LPA (Junior Security Analyst, SOC Analyst L1)

Growing roles: Rs 6-12 LPA (Ethical Hacker, SOC Analyst L2, VAPT Analyst); Rs 10-20 LPA (Penetration Tester, Security Engineer)

Experienced roles: Rs 18-30 LPA (Senior Ethical Hacker, Red Team Engineer); Rs 25-45 LPA (Lead Security Engineer, Security Consultant)

Other roles: Rs 8-20 LPA (Cyber Crime, CERT-In, Defense contractors); Rs 15-40 LPA (Big4, Security MNCs, Product-based firms); Rs 2-15 Lakhs per valid vulnerability (platform & severity dependent); Rs 40-90 LPA at senior level

Freelance consulting: Rs 2,500-12,000/hour based on skills and certifications

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is ethical hacking and is it a legal career path?

A: Ethical hacking (also called penetration testing or white-hat hacking) involves legally and systematically testing computer systems, networks, and applications for security vulnerabilities - with permission. It's completely legal and highly respected. Certifications like CEH, OSCP, and formal authorization make this a legitimate, in-demand profession.

Q: Do I need prior programming or IT experience for this ethical hacking course?

A: No prior experience required! We start from absolute basics: Linux fundamentals, networking concepts, and then Python scripting. Whether you're a complete beginner or IT professional, this 26-week intensive program builds your skills systematically. Many successful ethical hackers and bug bounty hunters started with zero technical background.

Q: What tools and techniques will I learn in this cybersecurity masterclass?

A: You'll master 50+ tools including Kali Linux, Metasploit, Burp Suite, Nmap, Wireshark, Nessus, John the Ripper, and custom Python scripts. Techniques cover web application hacking, network attacks, wireless hacking, social engineering, malware analysis, cloud security (AWS/Azure/GCP), and mobile app security.

Q: Can I earn money through bug bounties while learning this course?

A: Absolutely! We dedicate a full module to bug bounty hunting on platforms like HackerOne, Bugcrowd, and Synack. Many students earn their first bounties while still studying.

Q: What certifications does this course prepare me for?

A: This course prepares you for major industry certifications: CEH (Certified Ethical Hacker), OSCP (Offensive Security Certified Professional), CompTIA Security+, CompTIA PenTest+, GPEN, and cloud security certifications. We provide dedicated certification prep modules with practice exams and study strategies.

Q: What salary can I expect as an ethical hacker in India and globally?

A: Cybersecurity salaries are exceptionally high due to talent shortage. The demand far exceeds supply globally.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ethical-hacking-masterclass-complete>

=====

COURSE 53 OF 118: FLUTTER APP DEVELOPMENT COURSE: ZERO TO PRO DEVELOPER

=====

Course link: <https://learn.modernagecoders.com/courses/complete-flutter-app-development-masterclass-college>

Subject area: Professional Flutter App Development

Designed for: College students

Who can join: Complete Beginner to Advanced Professional

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Industry-recognized Flutter App Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Flutter and Dart app development course: build production-ready, multi-platform apps with widgets, state management, Firebase, testing, and deployment.

In one line: From 'Hello World' to Production-Ready Cross-Platform Apps

ABOUT THE PROGRAMME

This is not just a Flutter course, it's a complete transformation into a professional cross-platform app developer. Whether you're a curious beginner, student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled Flutter developer capable of building beautiful, performant apps for Android, iOS, Web, and Desktop. You'll master Flutter and Dart from ground zero: from basic programming logic to advanced state management patterns, from simple UIs to complex, animated, responsive layouts, from offline-first apps to real-time apps powered by Firebase and custom backends. You'll learn architectural patterns (MVVM, Clean Architecture), testing, CI/CD, performance optimization, and deployment to app stores. By the end, you'll have built 40+ real-world projects and be ready for Flutter developer roles in product companies, agencies, and startups.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - perfect for complete beginners with no coding background
- Separate learning tracks for kids (12+), teens, and adults with age-appropriate projects
- 1 year of structured, industry-aligned learning focused entirely on Flutter & Dart
- Covers Dart fundamentals + Flutter widgets + advanced state management + architecture
- Real production-style projects: e-commerce, chat apps, booking apps, dashboards, and more
- Hands-on with latest Flutter and Dart stable releases, Material 3, and modern tooling
- Interview preparation for Flutter roles in top product companies and remote-first startups
- Lifetime access with continuous updates as Flutter ecosystem evolves
- Build a strong, visually stunning app portfolio that runs on Android, iOS, Web & Desktop
- Direct path to high-paying roles: Flutter Developer, Mobile Engineer, Full-Stack (Flutter+Firebase)

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Programming Logic, Dart Fundamentals, Core Flutter & UI Basics

Phase 2: Intermediate (Months 4-6): Advanced Dart, State Management, Navigation, Local Storage, Offline Apps

Phase 3: Backend & Product Apps (Months 7-9): Firebase, REST APIs, Auth, Payments, Architecture, Testing

Phase 4: Professional (Months 10-12): Scalable Apps, Clean Architecture, Performance, CI/CD, Store Deployment, Career

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required, open to school students, college students, and working professionals

Coding experience: Absolute beginner friendly, no prior programming or mobile development experience needed

Age: 12+ years (separate learning tracks and examples for kids, teens, and adults)

Equipment: Computer/laptop (Windows/Mac/Linux), minimum 8GB RAM recommended, Android device

or emulator; macOS recommended for iOS builds
Time commitment: 15-20 hours per week consistently for best results and portfolio depth
English: Basic reading comprehension (core materials in English; key explanations also available in Hindi)
Motivation: Strong desire to build real, beautiful apps and pursue a Flutter/mobile development career
Math: Basic school-level mathematics (used occasionally for layouts, animations, and logic)

WHO THIS COURSE IS FOR

Kids: Age 12-14: Simplified track with visual learning, colorful UIs, game-like projects, and easy Dart concepts explained with stories.
Teens: Age 15-18: Student-focused track with apps relevant to school/college life, portfolio building, and intro to competitive programming logic.
Students: College students: Strong Flutter portfolio for internships and placements, freelancing opportunities, and startup ideas.
Working professionals: Career switchers: Structured path from any background into Flutter/mobile developer roles, with practical apps and interview prep.
Developers: Developers from web, native Android/iOS, or other languages wanting to specialize in Flutter for cross-platform development.
Backend engineers: Backend developers wanting to add a modern Flutter front-end/mobile skill-set to their profile.
Entrepreneurs: Founders and product builders who want to quickly prototype and ship cross-platform apps without large teams.
Freelancers: Freelancers who want to offer Flutter app development services to global clients and agencies.
Architects: Technical leads/architects wanting to design and supervise cross-platform Flutter app ecosystems.
Anyone: Anyone passionate about building beautiful, performant apps that run on Android, iOS, Web, and Desktop from a single codebase.

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CORE FLUTTER + DART SKILLS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid programming and Flutter fundamentals. Learn Dart, understand Flutter's widget tree, layout system, and create your first interactive cross-platform apps.

Months 1-2: Dart Fundamentals & Flutter Basics (Week 1-8)

Introduction to Flutter, Dart & Development Environment Setup

Topics:

- What is Flutter? History, features, and why it's popular
- Single codebase for Android, iOS, Web, Desktop
- Flutter architecture overview: Engine, Framework, Widgets
- Dart language introduction and why Flutter uses Dart
- Installing Flutter SDK on Windows/Mac/Linux
- Setting up Android Studio, VS Code, and Xcode (for macOS)
- Configuring device emulators and real devices
- Understanding Flutter project structure: lib, pubspec.yaml, android, ios, web
- Your first Flutter app: counter & Hello World
- main() function, runApp(), MaterialApp, Scaffold, AppBar, Body
- Hot reload vs hot restart vs full restart
- Understanding widget tree and composition
- StatelessWidget vs StatefulWidget basics
- DartPad and Flutter web previews
- Code style, formatting (dartfmt), and basic conventions

Projects built:

- Hello Flutter app with personalized welcome screen
- Digital visiting card app (name, role, contact info)
- Simple profile screen with image, text and icons
- Two-screen mini app using basic navigation
- Kids track variant: Colorful "My Favorite Things" app

Practice: Daily: 30-45 min Dart & Flutter basics; reproduce 3-5 simple screens from design snapshots

Dart Basics, Variables, Data Types & Operators

Topics:

- Dart program structure: main(), imports, packages
- Variables: var, final, const, late
- Primitive-like types in Dart: int, double, String, bool
- num type and type inference
- Dynamic vs static typing, dynamic keyword use-cases
- Type conversion and parsing: int.parse, double.parse, toString
- Collections overview: List, Set, Map (intro only)
- Operators: arithmetic, relational, logical, assignment
- Null safety in Dart: non-nullable & nullable types (int vs int?)
- Null-aware operators: ?, ??, ??=, !

- String interpolation and multi-line strings
- String methods: length, substring, contains, split
- Basic console apps in Dart (for logic practice)
- Using dartpad.dev to practice Dart snippets

Projects built:

- Simple console calculator (add, subtract, multiply, divide)
- Tip & bill split calculator (per person amount)
- Age & birthday countdown calculator
- Unit converters: temperature, length, weight
- Kids variant: Score tracker for a simple game

Practice: Solve 30+ problems on variables, strings, calculations and null-safety handling

Control Flow, Functions & Basic Collections

Topics:

- Control flow: if, else if, else
- Switch-case in Dart, enums introduction
- Loops: for, while, do-while, for-in
- break and continue, labeled statements
- Functions: definition, parameters (positional, named, optional), return types
- Arrow functions (fat arrow syntax)=>
- Scope rules and variable lifetime
- Intro to collections: List, Set, Map basics
- Iterating collections with for, for-in, forEach
- Basic algorithm building: searching & counting
- Introduction to error handling: try, catch, finally

Projects built:

- Number guessing game (console or Flutter dialog-based)
- Grade calculator with letter-grade output
- To-do list data model (List operations) with console UI
- Simple quiz logic engine (questions and scoring)
- Kids variant: Multiplication practice generator

Practice: Solve 40+ problems covering loops, branching, and simple collection manipulations

Core Flutter Widgets & Layout Basics

Topics:

- Flutter widget philosophy: everything is a widget
- Understanding build() method and widget lifecycle (overview)
- Basic display widgets: Text, Icon, Image, Container
- Layout widgets: Row, Column, Expanded, SizedBox
- Alignment & positioning: MainAxisAlignment, CrossAxisAlignment, Center, Align
- Styling text with TextStyle, GoogleFonts (package intro)
- Colors, ThemeData, Material 3 basics
- Buttons: ElevatedButton, TextButton, OutlinedButton, IconButton
- Input widgets: TextField basics
- Using Scaffold: appBar, body, floatingActionButton, bottomNavigationBar (intro)
- SingleChildScrollView and ListView (intro)
- Working with pubspec.yaml: adding assets (images, fonts)
- Asset management & responsive basics (MediaQuery)
- Debugging layout issues with Flutter DevTools & debugPrint

Projects built:

- Personal profile card app with images & icons
- Single-page product detail UI (static)
- Simple landing screen for a fictional app (hero image, CTA button)
- Basic contact form UI (no backend yet)
- Kids variant: "My Pet" info card with big images & fun fonts

Practice: Recreate 6-10 simple UI screens from given Figma/Dribbble screenshots

Month 3: Object-Oriented Programming with Dart & Stateful Widgets (Week 9-13)

Dart Classes, Objects & OOP Basics

Topics:

- OOP paradigm in Dart
- Defining classes & creating objects
- Fields, constructors (default, named, optional)
- Named constructors and factory constructors (intro)
- this keyword, initializer lists
- Encapsulation: private members with _ prefix
- Getters & setters (explicit, shorthand syntax)
- Static variables and methods
- Basic toString() overrides
- Intro to enums for modeling states
- Model classes for Flutter apps (e.g., User, Product, Task)

Projects built:

- User profile model and display widget
- Product class used in a simple catalog list
- Task/ToDo model with computed properties (e.g. isOverdue)
- Simple contact model and list
- Kids variant: Character class for a mini game inventory

Practice: Create 15+ different model classes to represent real-world entities used in apps

Inheritance, Polymorphism & Flutter Stateful Widgets

Topics:

- Inheritance in Dart: extends keyword
- Abstract classes and methods (intro)
- Method overriding & polymorphism
- Mixins: with keyword, when and how to use
- Interfaces via abstract classes & implicit interfaces
- Intro to composition vs inheritance
- StatefulWidget and State classes in Flutter
- setState(), rebuilding the UI, performance considerations
- Managing widget state vs app state (conceptual intro)
- TextField controllers and input handling
- Forms: Form, GlobalKey, basic validation
- GestureDetector and InkWell for taps & gestures
- Basic animations with AnimatedContainer and AnimatedOpacity (preview)

Projects built:

- Interactive counter app with custom logic
- Simple login form with validation and error messages
- Like/favorite button with animation
- Basic habit tracker with toggling state
- Kids variant: Tap-to-change-color playground app

Practice: Implement 20+ stateful interactions (toggles, counters, forms, sliders, etc.)

Abstraction, Interfaces & PHASE 1 Mini Capstone

Topics:

- Deeper look at abstract classes & interfaces in Dart
- Strategy-like patterns using abstract classes
- Separation of concerns: UI vs business logic vs data models
- Organizing Flutter code: folders (models, widgets, screens, services)
- Intro to simple service classes for data handling
- Preparing for larger apps: reusability & modular widgets

Projects built:

- Reusable UI components library (buttons, cards, chips)
- Simple theme manager (light/dark mode toggle using inherited widget pattern - conceptual)
- PHASE 1 MINI CAPSTONE: Multi-Screen Personal Productivity App (Tasks + Notes + Reminders)
- Features: Home screen, list views, detail pages, basic persistence via in-memory lists, clean UI

Practice: Refactor earlier projects to use cleaner OOP and reusable widgets

Assessment: Phase 1 Final Assessment, Dart fundamentals, core Flutter widgets & basic OOP

PHASE 2: INTERMEDIATE FLUTTER, STATE MANAGEMENT & APP ARCHITECTURE (MONTHS 4-6, WEEKS 14-26)

Master advanced Dart, state management patterns, navigation, forms, local storage, and start building real-world, offline-capable Flutter apps.

Months 4-5: Advanced Dart, Layouts & State Management Foundations (Week 14-22)

Advanced Dart, Collections & Async Programming

Topics:

- Deep dive into Lists, Sets, Maps
- Higher-order functions: map, where, reduce, fold
- Collection if and collection for in Dart
- Spread operator (...) and null-aware spread (...?)
- Immutable vs mutable collections
- Error handling in Dart: try-catch, on, finally, rethrow
- Custom exceptions
- Asynchronous programming: Future, async/await
- Streams: single subscription vs broadcast (intro)
- Stream transformations and listeners
- Using async code in Flutter (FutureBuilder, StreamBuilder basics)
- Using packages: http, intl and others
- Organizing business logic into service classes

Projects built:

- Data transformation utilities (filtering, sorting, grouping)
- HTTP client service (mock APIs) using http package
- Simple news/feed list using FutureBuilder (with mock JSON)
- Real-time counter/clock using StreamBuilder
- Kids variant: Random joke/quote generator using an API

Practice: Solve 35+ problems on collections & async patterns (Futures & Streams)

Flutter Navigation, Routing & App Structure

Topics:

- Navigator 1.0: push, pop, named routes
- Route arguments and passing data between screens
- Navigator 2.0 and Router (high-level intro, for advanced later)
- BottomNavigationBar and tab-based navigation
- Drawer navigation and custom side menus
- Nested navigation and navigation stacks
- App lifecycle basics: WidgetsBindingObserver, handling app states
- Splash screens and onboarding flows

- Handling back button and system navigation
- Routing best practices and route generators

Projects built:

- Multi-screen notes app with details and edit screens
- Onboarding flow with 3-4 intro pages
- Bottom navigation based mini app (Home / Search / Profile)
- Simple tabbed app with TabBar and TabBarView
- Kids variant: Storybook app with navigation between chapters

Practice: Implement routing and navigation in 6-8 mini applications

Forms, Validation & User Input Handling

Topics:

- Advanced TextField usage: controllers, focus nodes
- Form widget, FormState, and GlobalKey
- Synchronous and asynchronous validation
- Custom form field widgets
- Handling keyboard, focus traversal, and input submission
- DropdownButton, Checkbox, Radio, Switch, Slider widgets
- Date & time pickers
- Input formatting and masking
- Accessibility considerations for forms

Projects built:

- User registration form with full validation
- Feedback/contact form with submission success screen
- Settings screen with toggles, sliders, and dropdowns
- Profile edit form with local validation & previews
- Kids variant: Quiz creation form with multiple questions

Practice: Build and validate 10+ complex forms and input flows

Flutter Layout Mastery, Lists & Responsive Design

Topics:

- Advanced layout widgets: Stack, Positioned, Wrap, LayoutBuilder
- ListView.builder, ListView.separated
- GridView builder and dynamic grids
- Slivers overview: CustomScrollView, SliverAppBar, SliverList (intro)
- Responsive design principles in Flutter
- MediaQuery, LayoutBuilder and OrientationBuilder
- Creating adaptive UIs for phone/tablet/web
- Handling different screen densities and sizes
- Using Flutter's Flex and Expanded effectively

Projects built:

- E-commerce product list grid with responsive layout
- Photo gallery app with grid and full-screen viewer
- News feed with cards and SliverAppBar
- Responsive dashboard layout (phone vs tablet)
- Kids variant: Sticker board using Stack & Positioned

Practice: Rebuild 10+ real-world UI screens with responsive layouts

Intro to State Management Patterns

Topics:

- What is state management? Local vs global state
- Limitations of setState() for larger apps
- Lifting state up and prop drilling problems
- InheritedWidget and InheritedModel concept
- Provider package introduction
- ChangeNotifier and ChangeNotifierProvider
- Listening to changes with Consumer and context.watch/select
- Basic architecture: separating UI and logic using providers
- Simple global app state (theme, locale, auth status)

Projects built:

- Counter app rebuilt with Provider
- Theme toggler (light/dark mode) using Provider
- Cart management in a simple shopping UI
- Favorites list synced across multiple screens
- Kids variant: Global score/coins manager for a mini game

Practice: Create 10+ small apps using Provider for shared state

Month 6: Persistent Storage, Offline Apps & Advanced State (Week 23-26)

Local Storage: SharedPreferences, SQLite & Object Persistence

Topics:

- Why persistence? Saving user data locally
- Key-value storage with shared_preferences
- Simple settings & flags persistence
- SQLite basics and sqflite package
- Database schema design for small apps
- CRUD operations (create, read, update, delete)
- Using path_provider for file paths
- Storing JSON data locally
- Intro to Hive / Isar (NoSQL local DB) and when to use them
- Data serialization with jsonEncode/jsonDecode
- Basic repository pattern for data layer

Projects built:

- Persistent to-do list app with sqflite
- Settings/preferences manager using shared_preferences
- Offline notes app using Hive or Isar
- Simple expense tracker with SQLite
- Kids variant: Sticker collection app persisted locally

Practice: Build 12+ features requiring reading/writing persistent local data

State Management with Provider & Riverpod, Part 1

Topics:

- Revisiting Provider for larger apps
- Multiple providers and provider scopes
- Introduction to Riverpod: why & how it improves over Provider
- Provider types in Riverpod: Provider, StateProvider, StateNotifierProvider (intro)
- Using Riverpod for business logic & UI separation
- Immutability and state objects
- Error handling in async providers
- Refactoring existing apps to Riverpod

Projects built:

- News/feed app powered by Riverpod for state
- Weather app using async APIs with Riverpod
- Multi-tab app with shared state using Riverpod
- Theme + language switcher using Riverpod
- Kids variant: Simple quiz app with score using Riverpod

Practice: Refactor 5 existing apps from setState/Provider to Riverpod

Advanced State Management: Riverpod / BLoC, Part 2

Topics:

- StateNotifier & StateNotifierProvider with Riverpod
- Modeling complex state (loading, loaded, error) patterns
- BLoC pattern introduction (Cubit/BLoC) and when to use it
- Comparing Riverpod, Provider, and BLoC (pros/cons)
- Events, states, and BLoC logic (if using flutter_bloc)
- Using Freezed for immutable data classes & unions (intro)
- Folder structure for scalable apps (feature-first, layer-first)
- Practical guidelines for choosing a pattern per project size

Projects built:

- Authentication flow using Riverpod or BLoC
- Search with debounced queries and loading states
- Paginated list with infinite scroll
- Complex form with multi-step state management
- Kids variant: Multi-level quiz/game with progress state

Practice: Build 10+ flows using Riverpod/BLoC with loading/error handling

Flutter Animations & UX Polish

Topics:

- Implicit animations: AnimatedContainer, AnimatedOpacity, AnimatedAlign
- AnimatedCrossFade, AnimatedSwitcher
- Explicit animations: AnimationController, Tween, CurvedAnimation
- Using AnimatedBuilder and AnimatedWidget
- Hero animations between screens
- Page transitions and custom route animations
- Lottie animations integration
- Micro-interactions and motion design principles
- Performance considerations with animations

Projects built:

- Animated onboarding flow with Hero transitions
- Like/Bookmark micro-interactions with animations
- Expandable cards list with AnimatedContainer
- Custom page transition for navigation
- Kids variant: Animated "reward" screen with confetti

Practice: Implement 20+ small animations and transitions across apps

Phase 2 Capstone Project

Topics:

- Applying advanced Dart & collections in real app logic
- Using Riverpod/Provider/BLoC for feature-level state
- Designing multi-screen navigation and flows
- Local storage for offline capabilities
- Form handling, validation, and error display
- Polishing UI with consistent theming and animations

Projects built:

- PHASE 2 CAPSTONE: Offline-Capable Task & Habit Tracker
- Features: Multi-list tasks, habits with streaks, reminders (UI only), local DB (Hive/SQLite), search & filters, theme switching, basic animations.
- Alternative: Personal Finance Tracker with budgets and categories
- Alternative: Recipe Organizer app with favorites and offline storage
- Alternative: Journal/Diary app with tags and search

Assessment: Phase 2 Final Exam, State management, navigation, persistence, and animations

PHASE 2 CONTINUED - Architecture, Packages & Best Practices (Week 14-26 (distributed))

Reusable Widgets, Design Systems & Theming

Topics:

- Creating reusable, composable widgets
- Building a design system: spacing, typography, colors
- Material 3 (Material You) theming in Flutter
- ThemeData and ColorScheme customization
- Dark theme and dynamic theme switching
- Global styles and app-wide configuration
- Localization basics: using flutter_localizations and intl
- Supporting multiple languages and locales
- Accessibility: font scaling, contrast, semantics

Projects built:

- Design system library for buttons, cards, inputs
- App-wide theming for a sample app (light/dark/custom)
- Two-language version of an existing app (e.g. English + Hindi)
- Custom theming for kids' learning app
- Reusable error/empty/loading state widgets

Practice: Implement a consistent design system in 3+ existing apps

Clean Code, Architecture Patterns & Folder Structures

Topics:

- Clean code principles in Flutter
- Common Flutter code smells and anti-patterns
- Layered architecture: presentation, domain, data layers
- MVVM & Clean Architecture concepts for Flutter
- Repositories, use cases, and services
- Separating UI, state management, and data sources
- Project structure for small vs large apps
- Configuration management and environment separation (dev/prod)
- Refactoring legacy codebase to a cleaner structure

Projects built:

- Refactor a medium app using feature-first structure
- Implement simple Clean Architecture in a new project
- Create base repository and use case classes for reuse
- Architecture template boilerplate for future apps
- Kids variant: Simplified architecture for a game app

Practice: Refactor 2-3 earlier apps using improved architecture patterns

Networking & REST API Integration - Fundamentals

Topics:

- HTTP basics: methods, status codes, headers, JSON
- Using http / dio packages for API calls
- GET, POST, PUT, DELETE requests
- Parsing JSON manually and with json_serializable (intro)
- Error handling for network failures & timeouts
- Showing loading, success, and error states in UI
- API pagination and infinite scroll
- Securing API keys and environment variables (conceptual)
- Handling connectivity changes (connectivity_plus intro)

Projects built:

- Public API driven list app (e.g. GitHub repos, movies, news)
- Search UI with remote filtering
- Paginated list with "Load more" and pull-to-refresh
- Simple CRUD client for a mock REST API
- Kids variant: Fun facts/animals viewer from an open API

Practice: Build 8+ screens powered by real or mock REST APIs

Intro to Testing Flutter Apps

Topics:

- Why testing matters for Flutter apps
- Types of tests: unit, widget, integration
- Dart test package basics
- Unit tests for Dart logic (services, utils)
- Widget tests: testing UI components and layouts
- Mocking services and repositories
- Introduction to golden tests (visual regression), concept
- Basic test-driven development (TDD) workflow
- Measuring coverage in Flutter projects

Projects built:

- Unit tests for validators and repository classes
- Widget tests for key widgets in a sample app
- Test suite for state management logic (Riverpod/BLoC)
- Simple golden tests for critical UI components
- Kids variant: Test coverage for a small game's logic

Practice: Write tests for at least 3 existing apps, targeting 60%+ coverage

Build Automation, Packages & Version Control

Topics:

- pubspec.yaml deep dive: dependencies, dev_dependencies
- Managing versions and package upgrades
- Creating and publishing your own Dart/Flutter package (concept)
- Build flavors (dev, staging, prod), overview
- Flutter build commands for Android, iOS, Web
- Introduction to Git with Flutter projects
- Git branching strategies (feature, develop, main)

- GitHub integration, PRs, code reviews
- CI basics using GitHub Actions for Flutter test/build

Projects built:

- Extract common widgets into a local package
- Git repository for all course projects (mono-repo or multi-repo)
- Simple GitHub Actions workflow for test+build
- Versioning and changelog for a sample app
- Kids variant: Visual Git intro via Git GUI tools

Practice: Convert all active projects to Git repos and set up basic CI for at least one

----- PHASE 3: BACKEND INTEGRATION, FIREBASE & PRODUCTION FEATURES (MONTHS 7-9, WEEKS 27-39) -----

Master building connected apps with Firebase and REST backends, implement authentication, payments, notifications, and solid application architecture.

Months 7-8: Firebase, Auth & Real-World Features (Week 27-35)

Firebase Fundamentals & Realtime Data

Topics:

- What is Firebase? Overview of its products
- Setting up a Firebase project
- Integrating Firebase with Flutter (firebase_core)
- Crashlytics and Analytics setup (overview)
- Realtime Database vs Firestore overview
- Cloud Firestore basics: collections, documents, queries
- Adding, reading, updating, deleting documents
- Realtime listeners and streams from Firestore
- Security rules overview and best practices
- Offline support and caching behavior

Projects built:

- Realtime chat room (basic text chat) using Firestore
- Live comments section on a sample post
- Collaborative to-do list shared between users
- Simple presence/online status indicator
- Kids variant: Shared drawing board (concept + simple implementation)

Practice: Build 6-8 small realtime features with Firestore streams

Authentication & User Management

Topics:

- Firebase Authentication overview
- Email/password sign up and login
- Password reset flow
- Social login (Google, Apple, Facebook, platform dependent)
- Persisting login state and auto-login
- Linking auth user to Firestore profile documents
- Protecting routes based on auth state
- Handling auth errors and edge cases
- Profile management and avatar uploads (intro to Storage)

Projects built:

- Complete auth flow: sign-up, login, logout, reset password
- User profile screen linked to Firestore user data
- Protected routes (e.g., only logged-in users see dashboard)
- Simple role-based protection (admin vs user)
- Kids variant: Simplified nickname-based login for child-friendly apps

Practice: Implement full auth flows in at least 2 different sample apps

Cloud Storage, Media & File Handling

Topics:

- Firebase Storage integration
- Uploading images/files from Flutter apps
- Picking images from gallery/camera (image_picker, file_picker)
- Displaying uploaded media in apps
- Managing download URLs and access controls
- Basic image caching for performance
- Handling upload progress and cancellation
- File size limits and optimization concepts

Projects built:

- User avatar upload & change profile picture feature
- Simple image gallery app powered by Firebase Storage
- Document upload & viewer (e.g., for resumes or PDFs)
- Kids variant: Art gallery app where kids upload drawings
- Media feed with upload progress indicators

Practice: Add media upload/view features to at least 3 apps

Push Notifications & In-App Messaging

Topics:

- Firebase Cloud Messaging (FCM) integration
- Obtaining device tokens and sending test notifications
- Handling foreground, background, terminated notifications
- Notification customization (sound, icon, click actions)
- Topic-based and user-segment notifications
- In-app messaging basics (e.g., using Firebase In-App Messaging or custom implementation)

- Local notifications (flutter_local_notifications package)
- Scheduling notifications and reminders

Projects built:

- Push notification integration in existing habit tracker app
- Reminders/alerts app using local notifications
- Topic-based notifications (e.g., daily tips/news categories)
- In-app announcement banner system
- Kids variant: Reward reminder notifications for practice streaks

Practice: Implement both remote and local notifications across 2-3 apps

Advanced Firebase: Queries, Security & Performance

Topics:

- Advanced Firestore queries (compound, range, ordering)
- Indexes in Firestore for performance
- Optimizing read/write costs
- Security rules patterns and testing
- Multi-tenant data design (user-specific data separation)
- Cloud Functions for backend logic (overview, callable functions)
- Basic Cloud Functions triggered by Firestore events (concept)
- Monitoring Firebase usage and costs

Projects built:

- Search & filter features using advanced queries
- Role-based security rules for a sample app
- Server-triggered notifications (concept + small demo with Cloud Functions)
- Dashboard app with Firestore aggregate views
- Kids variant: Filtered content access based on age group

Practice: Design secure Firestore structures and rules for 2-3 different apps

Month 9: REST APIs, Payments & Robust Product Apps (Week 36-39)

Advanced REST APIs, Error Handling & Data Layers

Topics:

- Refining data layer with repositories & DTOs
- Handling complex JSON responses (nested, lists, maps)
- Code generation with json_serializable & build_runner
- Error modeling and mapping HTTP errors to UI states
- Retry policies and exponential backoff (concept + simple implementation)
- Network connectivity handling & offline-first strategies
- Logging and monitoring HTTP traffic (e.g., dio interceptors)

Projects built:

- Robust API-driven app with error handling and retries
- Offline-first news or blog reader (cache + network)
- Centralized API client service with interceptors
- Kids variant: Live quiz app using remote API data
- Refactor earlier REST apps to use repository pattern

Practice: Create 5+ robust REST clients with proper error and offline handling

In-App Purchases & Payment Gateway Integration

Topics:

- Monetization models: freemium, subscriptions, one-time purchases
- In-app purchases on Android & iOS (overview, in_app_purchase package)
- Using 3rd-party gateways like Razorpay, Stripe (concept + Flutter packages)
- Handling payment flows and callbacks
- Server verification of purchases (conceptual, minimal implementation)
- Unlocking premium content and features
- Compliance and security best practices (conceptual)
- Sandbox/test environment usage

Projects built:

- Premium upgrade flow for an existing app (e.g., remove ads/unlock features)
- Donation/payment screen integration with test gateway
- Subscription-like experience (locally simulated) with feature flags
- Kids variant: Parent-only purchase screen for unlocking content
- Designing a pricing & plans screen with clear UX

Practice: Integrate at least one payment-like flow in a demo app (test mode)

App Architecture in Depth: Clean Architecture & Large App Design

Topics:

- Revisiting Clean Architecture & MVVM in the context of bigger apps
- Domain layer: entities, use cases, business rules
- Data layer: models, DTOs, repositories, remote & local sources
- Presentation layer: views, controllers, state management (Riverpod/BLoC)
- Dependency injection basics (get_it, Riverpod providers)
- Feature modularization for scalable apps
- Working with multiple backends (REST + Firebase together)
- Documentation of architecture and data flow

Projects built:

- Architect one mid-size app using full Clean Architecture
- Combine local cache + remote data with repository pattern
- Implement dependency injection for services and repositories
- Architecture diagrams (UML, flow diagrams) for apps
- Kids variant: Simplified layered structure for a learning app

Practice: Re-architect one existing project using a full layered approach

Testing Connected Apps (REST + Firebase)

Topics:

- Unit testing repositories with mocked network/Firebase
- Widget tests for screens relying on async data
- Using fake clients, mocktail or Mockito for mocking
- Integration tests for sign-up/login flows
- Golden tests for critical UI screens
- Continuous testing with CI integration
- Measuring and improving test coverage for large apps

Projects built:

- Test suite for an authentication flow
- Widget tests for list/detail pattern UI
- Mocking Firestore/REST for offline tests
- Regression test set for a capstone app
- Kids variant: Test logic for a score/progress system

Practice: Achieve 70%+ coverage on at least one sizable app; add essential tests to others

Phase 3 Capstone Preparation & Consolidation

Topics:

- Choosing appropriate backend (Firebase vs REST or hybrid)
- Defining clear feature sets and MVP scope
- Wireframing and UX flows for a mid-size product app
- API contract design and documentation (OpenAPI/postman collections)
- Planning offline capabilities and caching
- Security and privacy considerations with user data

Projects built:

- PHASE 3 CAPSTONE: Full-Featured Social / Commerce / Utility App
- Example: Social Feed App (posts, likes, comments, profiles, notifications)
- Alternative: E-commerce Lite (product catalog, cart, checkout (mock), auth, favorites)
- Alternative: Booking App (appointments, schedules, reminders, auth, reviews)
- Alternative: Learning App (courses, progress, quizzes, leaderboards)

Practice: Draft detailed spec, data models, navigation map, and wireframes for capstone

PHASE 3 COMPLETION - Production-Ready App Delivery (Week 27-39 (distributed))

Performance Optimization & App Size Management

Topics:

- Performance profiling with Flutter DevTools
- Identifying and fixing jank (frame drops)
- Optimizing build methods and widget trees
- Using const constructors and keys effectively
- Image optimization and caching strategies
- Lazy loading lists, pagination, and placeholders
- Reducing app size: tree-shaking, splitting per ABI
- Minimizing unnecessary rebuilds and re-renders
- Monitoring performance in release builds

Projects built:

- Performance audit of capstone app
- Implement skeleton loaders and shimmer effects
- Optimize image-heavy screens
- Reduce app bundle size for Android and iOS
- Kids variant: Smooth animation showcase tuned for low-end devices

Practice: Profile & optimize at least 3 screens per app; document improvements

Error Reporting, Logging & Analytics

Topics:

- Structured logging in Flutter (logger packages, print vs log)
- Crashlytics deep dive: capturing and analyzing crashes
- Custom error handling and error boundaries (via widget tree)
- In-app error reporting (send logs, feedback)
- Using Firebase Analytics or other analytics tools
- Key metrics: MAU, retention, funnels (conceptual)
- Privacy and consent for analytics tracking

Projects built:

- Centralized error handling layer in an app
- User-friendly error screens with retry & support links
- Basic analytics events for core user flows
- Feedback form that sends logs & device info (with consent)
- Kids variant: Minimal analytics respecting child privacy

Practice: Integrate logging & crash reporting into all capstone-level apps

Security & Data Protection in Flutter Apps

Topics:

- Storing secrets securely (env files, platform secure storage)
- Encrypted local storage (flutter_secure_storage, cryptography basics)
- Protecting API keys and sensitive data
- Secure communication: HTTPS/TLS verification (concept)
- Obfuscation and code hardening (dart compile, proguard/R8 for Android)
- Auth token management and refresh strategies
- Basic OWASP considerations for mobile apps

Projects built:

- Secure token storage for an authenticated app
- Secure notes/vault feature with local encryption (conceptual + basic implementation)

- API key protection strategies demo
- Kids variant: Content access locks and parental gate
- Security checklist for all released apps

Practice: Review and harden security for all apps handling user data

Phase 3 Capstone Implementation

Topics:

- Final integration of auth, data storage, and APIs
- Polishing UI and interactions for product quality
- Ensuring offline support, retries, and graceful failures
- Testing critical user flows end-to-end
- App icon, splash screen, and branding
- Preparing release candidate builds

Projects built:

- MAJOR CAPSTONE: Production-Ready Connected App (as scoped earlier)
- Deliver full feature-set: auth, data, UI, offline, notifications
- Alternative: Enterprise dashboard client app consuming existing APIs

Practice: Stabilize capstone release with bug-fixing sprints

Phase 3 Review & Assessment

Topics:

- Code review of architecture & patterns
- Refactoring opportunities and tech debt
- End-to-end manual test checklist creation
- Capturing lessons learned & best practices
- Preparation for advanced micro-app and multi-platform work

Projects built:

- Code review & refactor iteration on capstone
- Technical documentation of capstone architecture
- Checklist for future production Flutter apps

Assessment: Phase 3 Final Exam, Backend integration, Firebase, architecture & production features

----- PHASE 4: ADVANCED FLUTTER, MULTI-PLATFORM, DEVOPS & CAREER EXCELLENCE (MONTHS 10-12, WEEKS 40-52) -----

Master advanced patterns, multi-platform delivery, CI/CD, store deployment, performance engineering, and become industry-ready as a professional Flutter developer.

Months 10-11: Multi-Platform, Advanced Patterns & Tooling (Week 40-48)

Flutter Web & Desktop Fundamentals

Topics:

- Enabling Flutter web & desktop targets
- Differences between mobile, web, and desktop UIs
- Routing for web (URL-based navigation, go_router intro)
- Mouse, keyboard and hover interactions
- Window sizing and layout considerations on desktop
- Persisting data on web (localStorage, indexedDB, via plugins)
- SEO considerations for Flutter Web (limitations & strategies)
- Packaging & distributing desktop apps (Windows, macOS, Linux basics)

Projects built:

- Responsive admin dashboard for web & desktop
- Note-taking PWA-style web app
- Simple desktop utility app (e.g. checklist/timer)
- Kids variant: Web-based learning game with keyboard support
- Port one existing mobile app to web & desktop with adaptations

Practice: Adapt at least 2 apps for web/desktop and handle platform differences

Modularization, Packages & Plugin Ecosystem

Topics:

- Breaking apps into modules/packages (local Dart/Flutter packages)
- Creating internal UI component libraries
- Understanding platform channels for native integrations (concept)
- Using common native plugins (camera, sensors, background tasks intro)
- Dealing with plugin compatibility & platform issues
- Maintaining and versioning internal packages

Projects built:

- Extract design system into a shared UI package
- Create a utilities/data package shared by multiple apps
- Proof-of-concept plugin usage (e.g., camera or location)
- Kids variant: Sensor-based mini game (e.g., tilt/accelerometer)
- Prepare one internal package for potential open-source release

Practice: Modularize at least one large app into multiple packages

Internationalization, Accessibility & Advanced UX

Topics:

- Full internationalization (i18n) with arb/json & intl
- Dynamic locale switching at runtime
- Right-to-left (RTL) layout support basics
- Accessibility labels, semantics widgets, and screen readers
- Color contrast & large text support
- Animation accessibility settings (reduce motion)
- Improving perceived performance with skeletons & hints

- Designing inclusive experiences for all ages (kids, seniors)

Projects built:

- Multi-language version of a key app (3+ languages)
- Accessibility audit & improvements for a production app
- RTL support demonstration (e.g., Arabic/Hebrew)
- Kids variant: Child-friendly UI with big touch targets
- Inclusive design checklist for Flutter apps

Practice: Apply i18n and accessibility improvements to at least 2 apps

CI/CD, Automation & Release Management for Flutter

Topics:

- CI/CD concepts applied to Flutter projects
- Using GitHub Actions/Codemagic/Bitrise for automated builds
- Running tests and linters in pipelines
- Automating versioning and changelogs
- Automatic deployment to Play Store / App Store (overview)
- Signing Android apps (keystores) & iOS code-signing basics
- Managing build flavors and environments in CI
- Release channels: alpha, beta, production

Projects built:

- CI pipeline that runs tests & builds APK/AAB
- Automated build for web deployment (e.g., to Firebase Hosting)
- Pipeline that signs and archives release builds
- Kids variant: Visual dashboard of build status for understanding CI
- Release checklist and template for major versions

Practice: Set up CI for at least one large app covering tests and builds

Store Deployment & App Store Readiness

Topics:

- Preparing apps for Google Play Store
- Preparing apps for Apple App Store (requirements & review)
- App icons, splash screens and branding assets
- Privacy policies, terms, and app descriptions
- Screenshots, promo graphics, and store listing optimization (ASO)
- Handling store reviews, updates, and crash reports
- Versioning strategy for store releases
- Beta testing via internal testing tracks / TestFlight

Projects built:

- Prepare a production app for Play Store submission
- Create full store listing content (copy, screenshots, icons)
- Draft privacy policy & basic legal pages
- Kids variant: COPPA-like considerations (concept only)
- Run a closed beta test and collect user feedback

Practice: Complete end-to-end release process simulation for at least one app

Month 12: System Design Thinking for Apps, Performance & DevOps (Week 49-52)

Mobile/App System Design & Architecture

Topics:

- High-level system design for mobile-first products
- Backend choices: Firebase vs custom REST vs GraphQL
- Designing for scale: many users, data, and devices
- API contracts and versioning for mobile clients
- Offline-first & sync strategies (conflict resolution concepts)
- Push vs pull data flows (websocket, polling, FCM)
- Multi-tenant and white-label apps (concepts)
- Designing feature flags and remote config

Projects built:

- System design doc for a large consumer app (e.g., Instagram-like)
- Data flow diagrams for offline-first sync
- API contract definition for a new app idea
- Feature flagging system concept for gradual rollouts
- Kids variant: System design explained with simplified diagrams

Practice: Create 3-4 system design documents for different app types

Performance Engineering for Flutter Apps

Topics:

- Deep dive into Flutter rendering pipeline basics
- Jank analysis using Flutter performance overlays
- Widget rebuild analysis and optimization
- Isolates for heavy computations (intro + example)
- Caching strategies for network and images (e.g., `cached_network_image`)
- Network performance: compression, pagination, batch requests
- Memory usage analysis and leak prevention strategies
- Startup time optimization and lazy initialization

Projects built:

- Performance tuning exercise on capstone app
- Extract heavy tasks into isolates (e.g., JSON processing)
- Implement network caching and request batching
- Create a reusable performance checklist for Flutter apps
- Kids variant: Visualizing performance ideas with games (e.g., fewer objects, reuse)

Practice: Tune at least one app to meet specific performance targets (e.g., <16ms frames)

DevOps & Observability for Flutter-Based Products

Topics:

- Monitoring mobile app health: crashes, ANRs (Application Not Responding)
- Using Sentry, Firebase Crashlytics & other tools
- User behavior analytics for product decisions
- Remote configuration & feature toggles for runtime changes
- A/B testing basics for apps (concepts)
- Error budgets and release management mindset
- Incident response workflows and rollback strategies

Projects built:

- Observability setup (crash + analytics) for capstone app
- Remote configuration to toggle features without new release
- Simple A/B test flow for UI variation (concept or controlled via remote config)
- Kids variant: Simple "version" toggles to understand feature flags
- Post-mortem template for production incidents

Practice: Enable observability in all production-ready apps and simulate incidents

Code Quality, Refactoring & Long-Term Maintainability

Topics:

- Advanced linting & analysis (flutter_lints, custom rules)
- Continuous refactoring and tech debt management
- Design patterns for Flutter (Repository, Adapter, Strategy, etc.)
- Documentation standards: README, API docs, architecture notes
- Onboarding docs for new developers joining your project
- Branching & release management for long-lived products
- Open-source & community packages: maintaining compatibility

Projects built:

- Refactor a large codebase using updated best practices
- Setup strict analysis_options.yaml and fix issues
- Comprehensive documentation for one flagship app
- Pattern catalog demo app showing multiple patterns
- Kids variant: Simple code cleanup exercises on tiny projects

Practice: Refactor and document at least two large projects for maintainability

Phase 4 Pre-Capstone Review

Topics:

- Reviewing full stack of skills gained: Dart, Flutter, state, APIs, Firebase, CI/CD, stores
- Gap analysis: identifying areas needing revision
- Idea validation for final mega-capstone
- Planning timeline, deliverables, and milestones
- Team collaboration best practices (if building with peers)

Projects built:

- Finalize concept & spec for final enterprise-grade Flutter app
- Backlog and sprint planning for capstone build
- Risk analysis and mitigation plan for project

Practice: Consolidated revision: build one small app end-to-end in 1-2 days

PHASE 4 COMPLETION - Career Preparation (Week 40-52 (distributed))

Advanced Flutter Topics & Specializations

Topics:

- Flutter animations advanced: custom painters, implicit vs explicit combinations
- Flutter Web-specific optimization & SEO strategies (where possible)
- Flutter for games: Flame engine overview (concept + small demo)
- GraphQL clients in Flutter (graphql_flutter, ferry overview)
- Real-time communication with WebSockets
- Background services & notifications (platform limitations overview)
- Platform-specific integrations via MethodChannels (concept)
- Integration with existing native apps (add-to-app concept)

Projects built:

- Custom animated charts or visualizations
- Small casual game using Flame (or simple game using standard widgets)
- GraphQL-powered data screen in a demo app
- WebSocket chat or live data ticker
- Specialization mini-project in chosen advanced area

Practice: Explore and prototype at least 3 advanced/edge Flutter capabilities

Code Quality & Best Practices (Flutter Edition)

Topics:

- Applying SOLID principles in Flutter code
- Common Flutter-specific anti-patterns and their fixes
- Effective widget composition vs massive widgets
- State management best practices across app sizes
- Reusability and DRY (Don't Repeat Yourself) in UI and logic
- Testing strategy for long-term projects
- Documentation via code comments and READMEs
- Continuous improvement mindset for production apps

Projects built:

- Code review & refactor sprint on capstone app
- Before/after comparison of a cleaned-up codebase
- Best practices checklist curated from your own experience
- Guidelines document for future Flutter team members
- Kids variant: Simple "tidy up" exercises refactoring messy code

Practice: Apply best-practices refactoring across the entire capstone project
Open Source & Community

Topics:

- Finding Flutter/Dart open-source projects to contribute to
- Understanding CONTRIBUTING.md, issues, and PR templates
- Submitting PRs: etiquette and expectations
- Reporting and triaging bugs effectively
- Creating and publishing your own open-source Flutter package
- Building a personal brand in the Flutter community
- Speaking at Flutter meetups and writing technical articles
- Ethical and inclusive community participation

Projects built:

- Contribute to 3-5 Flutter/Dart open-source projects (docs, bugfixes, small features)
- Create and publish at least one open-source Flutter/Dart package or starter template
- Maintain an open-source example app or code snippet repository
- Write documentation or tutorials for an existing open-source package
- Engage in community Q&A (StackOverflow, Discord, Slack, etc.)

Practice: Maintain regular community engagement and contributions over several weeks

Interview Preparation (Flutter & Mobile Roles)

Topics:

- Flutter-specific interview preparation strategy
- Core Dart & Flutter concept questions
- State management interview topics (Provider, Riverpod, BLoC)
- Architecture & design pattern questions
- Backend & Firebase conceptual questions
- Mobile-specific topics: app lifecycle, performance, offline, notifications
- System design for mobile apps interviews
- DSA (Data Structures & Algorithms) essentials for Flutter devs
- Platform & ecosystem questions (Android/iOS basics)
- Behavioral and culture-fit interview questions
- Presenting portfolio and explaining project decisions
- Salary negotiation and remote job strategies
- Freelancing & contract work interview differences
- Building a long-term career roadmap with Flutter

Projects built:

- Solve 80-120 DSA problems relevant for mobile/backend roles
- Curated list of 100+ Flutter interview Q&A with your own notes
- Mock interviews (technical + behavioral) with peers/mentors
- System design case studies with a focus on mobile apps
- Polished portfolio presentation deck for interviews

Practice: Daily coding practice + timed mock interviews leading up to job applications

Final Month - Capstone & Career Launch (Week 49-52)

Final Capstone Project - Part 1

Topics:

- Project ideation and selection aligned with desired career path (product, agency, freelance, startup)
- Detailed requirement gathering and user stories
- UX-driven design: wireframes, prototypes, and user flows
- Full system architecture design (client + backend + analytics + notifications)
- Choosing tech stack: Firebase vs custom backend vs hybrid
- Database schema & data modeling for Firestore/SQL
- API contract design (REST/GraphQL) and documentation
- Defining MVP and phased roadmap
- Setting up monorepo or multi-package structure if needed
- Team collaboration & Agile/Scrum workflow planning

Projects built:

- FINAL CAPSTONE: Enterprise-Grade Flutter Application
- Option 1: E-commerce Platform (multi-tenant, cart, checkout (real or test payments), admin panel, analytics)
- Option 2: Social Platform (posts, stories, comments, likes, messaging, notifications, media upload)
- Option 3: Booking/Marketplace App (services, search, booking, calendar, reviews, payments, chat)
- Option 4: Learning Platform (courses, content, video, quizzes, certificates, progress tracking, notifications)

Final Capstone Project - Part 2

Topics:

- Implementation of all planned features with robust state management
- Integration of backend, auth, notifications, and analytics
- Comprehensive manual and automated testing
- Regression testing on multiple platforms (Android, iOS, Web, Desktop if applicable)
- Performance optimization and profiling cycles
- Security review: auth, storage, secrets, policies
- Deployment to app stores and web hosting (if applicable)
- CI/CD pipeline finalization for automated builds and tests
- Monitoring and logging integration for production
- Preparing documentation, demos, and marketing assets for the app

Deliverables:

- Complete Flutter source code hosted on GitHub/GitLab with clear structure
- Deployed mobile app (at least Android release on Play Store or internal track; iOS TestFlight/Store where possible)
- Optional: Deployed web version and/or desktop package if relevant
- End-to-end feature documentation and API references
- Architecture and data flow diagrams
- Automated test suite with target 70-80% coverage on core logic
- CI/CD configuration scripts (GitHub Actions/Codemagic/etc.)
- Performance profiling and optimization report
- Security and privacy checklist with mitigation strategies
- Video walkthrough/demo of the app and its architecture
- Comprehensive README and project wiki for portfolio presentation

Career Launch & Professional Development

Topics:

- Building a standout professional portfolio website highlighting Flutter projects
- Resume optimization for Flutter/Mobile/Front-end roles
- LinkedIn profile tuning: headline, about, experience, projects, recommendations
- GitHub profile curation: pinning best repos, writing good READMEs
- Technical blog strategy: what to write, where to publish (Medium, Dev.to, Hashnode)
- Speaking and presenting: meetups, webinars, internal tech talks
- Networking: online communities, conferences, local meetups
- Job search strategy: targeting remote-friendly and product companies
- Freelance profile setup (Upwork, Fiverr, Toptal, local platforms)
- Consulting & indie hacker paths with Flutter apps
- Planning continuous learning: staying updated with Flutter releases & ecosystem
- Giving back: mentoring juniors, contributing to OSS, sharing knowledge
- Personal branding long-term plan

Deliverables:

- Professional portfolio site showcasing 40+ projects with live demos/screenshots
- ATS-optimized resume targeted at Flutter/Mobile Developer roles
- Enhanced LinkedIn profile with featured projects and endorsements
- GitHub profile with active contributions and polished repositories
- 5-10 technical blog posts or case studies documenting your learning or projects
- Public presentation or recorded talk/demo of your capstone
- Job application materials (cover letters, outreach templates)
- Completed mock interviews with feedback incorporated
- 6-12 month personal learning roadmap beyond the course
- List of relevant communities, newsletters, and events to stay engaged with

Assessment: FINAL COMPREHENSIVE CERTIFICATION EXAM, Complete Flutter & Dart mastery evaluation plus portfolio review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-3): 20+ foundational projects, Dart console apps, basic UIs, forms, small multi-screen apps
- Phase 2 (Months 4-6): 18+ intermediate projects, responsive UIs, offline storage apps, advanced state management, animated experiences
- Phase 3 (Months 7-9): 20+ connected projects, Firebase-powered apps, REST clients, auth flows, payment demos, complex product apps
- Phase 4 (Months 10-12): 15+ advanced projects, multi-platform apps, CI/CD pipelines, performance-optimized apps, store-ready products
- Final: 5 major capstone projects including at least 1 enterprise-grade app demonstrating complete Flutter expertise

Total projects: 70+ projects from beginner toy apps to enterprise-grade, store-ready products

SKILLS THE STUDENT WILL HAVE AT THE END

- Dart Language: Syntax, OOP, Collections, Null Safety, Exceptions, Async/Await, Streams, Generators
- Flutter Core: Widgets, Layout, Navigation, Theming, Animations, Gesture Handling, Forms, Accessibility
- State Management: setState, InheritedWidget, Provider, Riverpod, BLoC (concepts), Clean Architecture patterns
- Backend Integration: REST APIs, JSON, dio/http, error handling, pagination, caching
- Firebase: Auth, Firestore, Realtime updates, Storage, Cloud Messaging (push), Analytics, basic Cloud Functions
- Local Storage: shared_preferences, SQLite (sqflite), Hive/Isar, offline-first patterns
- Multi-Platform: Android, iOS, Web, Desktop support, platform adaptations
- Testing: Unit tests, Widget tests, Integration tests, Golden tests, CI-based testing workflows
- DevOps & Delivery: Git, GitHub, CI/CD (GitHub Actions/Codemagic), build flavors, automated builds, store deployment
- Performance & Security: Profiling, optimization, app size reduction, secure storage, encrypted data, secure API usage
- Architecture & Design: MVVM, Clean Architecture, repository pattern, system design for mobile-first products
- Tools & Ecosystem: Android Studio, VS Code, Flutter DevTools, Postman, Firebase Console, design tools (Figma basics)

- Soft Skills: Problem-solving, product thinking, documentation, code review, Agile workflows, communication with teams/clients

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 8-10 hours
 Projects: 3-5 hours
 Practice problems: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding sessions, app reviews, and Q&A with Flutter experts
 Mentorship: 1-on-1 mentorship from senior Flutter developers and architects
 Community: Active Discord/Slack community with 24/7 peer and TA support focused on Flutter
 Code review: Expert code reviews for all major apps and capstone projects
 Career support: Resume review, portfolio polishing, mock interviews for Flutter roles, and job search guidance
 Lifetime access: Lifetime access to all Flutter course content, updates, and new modules as Flutter evolves
 Interview prep: Structured Flutter interview prep, mock interviews with industry professionals
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion (4 Flutter phase certificates)
 Final certificate: Professional Flutter App Developer Certification
 Spring certificate: N/A for Flutter, replaced by: Flutter Firebase Specialist Certificate
 Microservices certificate: N/A for Flutter, replaced by: Cross-Platform App Architecture Certificate
 LinkedIn badge: Digital badges for LinkedIn reflecting Flutter specialization
 Industry recognized: Recognized by startups, agencies, and product companies using Flutter in production
 Portfolio projects: 40+ production-style apps and mini-projects for your portfolio
 Oracle exam prep: Not applicable, instead: guidance for Google/Flutter-related certifications and badges when available

WHERE THIS CAN LEAD

- Flutter Developer (Junior to Senior levels)
- Mobile App Developer (Flutter-focused)
- Cross-Platform App Developer
- Front-end Engineer (Flutter Web & Mobile)
- Full-Stack Flutter + Firebase Developer
- Flutter Team Lead / Mobile Tech Lead
- Mobile Solutions Architect (Flutter-centric)
- Freelance Flutter Consultant / Contractor
- Indie App Developer / Solo Founder
- UI Engineer with Flutter specialization
- Product Engineer (Mobile-first products)
- Technical Trainer / Flutter Instructor
- Open-Source Flutter Package Maintainer
- Remote Flutter Developer for global startups
- CTO / Tech Co-founder for app-based startups

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-5 LPA (Junior Flutter Developer / Mobile Intern with strong fundamentals)

Growing roles: Rs 5-9 LPA (Flutter Developer / Mobile App Developer with portfolio apps); Rs 8-15 LPA (Senior Flutter Developer / Lead Mobile Engineer in startups & agencies)

Experienced roles: Rs 12-25 LPA (Senior/Lead Flutter Developer or Mobile Architect in product companies); Rs 18-40 LPA (Lead Developer / Mobile Architect / Specialist Consultant)

Freelance: Rs 1,000-5,000/hour or \$20-\$100/hour based on expertise and client region

International usa: \$80k-160k USD (Mobile / Flutter Developer to Senior roles, remote or onsite)

International europe: €50k-120k EUR based on country, role, and experience in Flutter/mobile dev

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Flutter and why should I learn it for mobile app development?

A: Flutter is Google's open-source framework for building beautiful, natively compiled applications for mobile (iOS & Android), web, and desktop from a single codebase. It's used by companies like Google, BMW, Alibaba, and eBay. Learning Flutter means you can build apps for 6+ platforms with one codebase, making you highly valuable as a developer.

Q: Do I need prior programming experience to start this Flutter course?

A: No prior experience required! We start from absolute basics, teaching Dart (Flutter's programming language) from scratch. Whether you're a complete beginner, designer wanting to code, or developer switching from another language, this structured 12-month program takes you from 'Hello World' to production-ready apps.

Q: How does Flutter compare to React Native and native development (Swift/Kotlin)?

A: Flutter offers faster development (hot reload), beautiful native-looking UIs, and excellent performance with one codebase for all platforms. Unlike React Native, Flutter doesn't use JavaScript bridges, resulting in smoother animations.

Q: What kind of apps can I build with Flutter after this 12-month course?

A: You'll be able to build any type of app: e-commerce platforms, social media apps, booking systems, learning platforms, games, fintech apps, and more. You'll complete 40+ projects including apps with authentication, payments, real-time features, push notifications, and offline capabilities - all deployed to app stores.

Q: What salary can I expect as a Flutter developer in India and internationally?

A: Flutter developers are in high demand. The cross-platform skill set commands premium rates compared to single-platform developers.

Q: Can I build apps for iOS on a Windows computer with Flutter?

A: Yes and no. You can write and test Flutter code on Windows for iOS, but to actually build and publish iOS apps, you need a Mac (or cloud Mac services). However, the same code works on both platforms, so you can develop on Windows and do final iOS builds on any Mac. Many developers start on Windows and add Mac access later.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-flutter-app-development-masterclass-college>

=====

COURSE 54 OF 118: FRONTEND DEVELOPMENT COURSE FOR TEENS: HTML, CSS, REACT

=====

Course link: <https://learn.modernagecoders.com/courses/frontend-development-masterclass-for-teens>

Subject area: Teen Frontend Development

Designed for: Teens

Who can join: Complete Beginner to Professional Frontend Developer

Course length: 12 months (52 weeks)

Time needed: 10-12 hours/week recommended

Certificate: Professional Frontend Developer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Frontend development course for teens: build sites like Netflix and Instagram with HTML, CSS, JavaScript, React and Vue, and land your first tech job.

In one line: From 'What is HTML?' to 'I Built the Next Instagram'

ABOUT THE PROGRAMME

Turn your creative ideas into stunning websites that millions can use! This isn't your boring computer class - it's your ticket to building the visual side of the internet. Designed specifically for teenagers, you'll learn by recreating the websites and apps you use daily: TikTok, Discord, Netflix, Spotify, and more. No math genius required - just creativity and the desire to build cool stuff. You'll master the art of making websites beautiful, interactive, and addictive. From simple landing pages to complex web applications with millions of users, you'll have the skills to build anything you can imagine. Perfect for creative teens who want to combine art with technology.

WHAT MAKES THIS COURSE DIFFERENT

- Teen-focused: Build sites teens actually want - gaming platforms, social media, music apps
- Visual learning: See immediate results with every line of code
- Zero to hired: From no experience to job-ready in 12 months
- Creative freedom: Design and build your own unique projects
- Modern stack: Learn what companies use today, not outdated tech

- Portfolio-driven: Build 30+ projects for an impressive portfolio
- Young mentors: Learn from developers who understand teen culture
- Global community: Connect with teen coders worldwide

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): HTML, CSS, JavaScript basics, Responsive Design

Phase 2: Interactive Web (Months 4-6): Advanced JavaScript, DOM, APIs, Modern CSS

Phase 3: React Mastery (Months 7-9): React, State Management, Next.js

Phase 4: Professional Skills (Months 10-12): Vue, Testing, Performance, Portfolio

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No prerequisites - complete beginner friendly

Coding experience: Zero coding knowledge required

Equipment: Any computer with internet connection

Time commitment: 10-12 hours per week

English: Basic English understanding

Motivation: Creativity and desire to build

WHO THIS COURSE IS FOR

Students: Teenagers (13-19) interested in web design

Working professionals: Young adults switching to tech

Entrepreneurs: Teens wanting to build websites

Freelancers: Start earning while in school

Kids: Creative kids who love design

Anyone: Anyone who wants to build beautiful websites

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: WEB FUNDAMENTALS & DESIGN (MONTHS 1-3, WEEKS 1-13) -----

Start from absolute zero and build your first amazing websites. Learn the core technologies that power every website on the internet.

Month 1: HTML & CSS Foundations (Week 1-4)

Introduction to Web Development

Topics:

- How websites work: The magic behind Instagram and TikTok
- Frontend vs Backend: What's the difference?
- Browser DevTools: Your secret weapon
- Setting up VS Code: Your coding workspace
- Essential extensions and themes (make it look cool!)
- HTML basics: The skeleton of websites
- HTML tags: headers, paragraphs, links, images
- Semantic HTML5: header, nav, main, footer
- Creating your first webpage
- Lists, tables, and organization
- Forms and user input
- Embedding videos, audio, and maps

Projects built:

- Personal introduction webpage
- Favorite artist fan page
- School project presentation site
- Recipe collection page

Practice: Daily: Create one new HTML page exploring different elements

CSS Styling & Design

Topics:

- CSS: Making websites beautiful
- Selectors: Targeting elements like a sniper
- Colors and gradients: Creating stunning palettes
- Typography: Fonts that make impact
- Google Fonts and custom fonts
- Box Model: The secret to layouts
- Padding, margins, borders explained
- Backgrounds: Images, patterns, videos
- CSS animations: Making things move
- Transitions: Smooth interactions
- Hover effects that wow
- Transform: Rotate, scale, skew

Projects built:

- Animated landing page
- CSS art challenge
- Interactive button collection
- Photo gallery with effects

Practice: Recreate 5 website sections from popular sites
Flexbox & Grid Layouts

Topics:

- Flexbox: The game-changer for layouts
- Flex containers and items
- Main axis vs cross axis
- Justify-content: Horizontal alignment magic
- Align-items: Vertical alignment made easy
- Flex-wrap and responsive design
- CSS Grid: Two-dimensional layouts
- Grid template areas: Visual layout design
- Auto-fit and auto-fill: Dynamic grids
- Combining Flexbox and Grid
- Building complex layouts easily
- Common layout patterns

Projects built:

- Instagram-style photo grid
- YouTube homepage layout
- Discord server layout
- Pinterest-style masonry

Practice: Build 10 different layout challenges

Responsive Design & Mobile First

Topics:

- Why mobile-first matters: Everyone's on phones
- Media queries: Adapting to screen sizes
- Breakpoints: Phone, tablet, desktop
- Responsive units: rem, em, %, vw, vh
- Responsive images and videos
- Mobile navigation patterns
- Hamburger menus and mobile UI
- Touch-friendly design
- Testing on real devices
- Chrome DevTools device mode
- Common responsive mistakes
- Performance on mobile

Projects built:

- Fully responsive portfolio
- Mobile-first blog design
- Responsive e-commerce page
- App landing page

Practice: Make all previous projects responsive

Month 2: JavaScript Fundamentals (Week 5-8)

JavaScript Basics

Topics:

- JavaScript: Making websites interactive
- Variables: let, const, var explained
- Data types: Numbers, strings, booleans
- Arrays: Managing lists of data
- Objects: Organizing complex data
- Functions: Reusable code blocks
- Parameters and return values
- Arrow functions: Modern syntax
- Template literals: Dynamic strings
- Operators and comparisons
- Console methods for debugging
- Comments and code organization

Projects built:

- Interactive calculator
- Quiz game with scoring
- Todo list application
- Random meme generator

Practice: Complete 50 JavaScript exercises on CodeWars

Control Flow & Logic

Topics:

- If/else statements: Making decisions
- Switch statements: Multiple choices
- Ternary operators: Shorthand conditions
- Logical operators: AND, OR, NOT
- Truthy and falsy values
- For loops: Repeating actions
- While loops: Conditional repetition
- ForEach, map, filter, reduce
- Break and continue
- Try/catch: Error handling
- Debugging strategies
- Common logic patterns

Projects built:

- Password strength checker

- Grade calculator
- Text-based adventure game
- Data filtering dashboard

Practice: Solve 30 algorithm challenges

DOM Manipulation

Topics:

- DOM: The bridge between HTML and JavaScript
- Selecting elements: querySelector magic
- Changing content dynamically
- Modifying styles with JavaScript
- Creating and removing elements
- Event listeners: Clicks, hovers, inputs
- Event bubbling and delegation
- Form handling and validation
- Local storage: Saving user data
- Keyboard and mouse events
- Scroll events and parallax
- Building interactive components

Projects built:

- Dynamic theme switcher
- Interactive game (Tic-tac-toe)
- Form with live validation
- Infinite scroll gallery

Practice: Build 15 interactive components

Asynchronous JavaScript & APIs

Topics:

- Async JavaScript: Why it matters
- Callbacks: The old way
- Promises: The better way
- Async/await: The modern way
- Fetch API: Getting data from servers
- Working with JSON data
- Public APIs: Weather, movies, music
- Error handling in async code
- Loading states and spinners
- API authentication basics
- CORS and how to handle it
- Building data-driven websites

Projects built:

- Weather app with live data
- Movie search application
- Spotify playlist viewer
- GitHub profile viewer

Practice: Integrate 10 different APIs

Month 3: Advanced CSS & Animations

Topics:

- CSS variables: Dynamic styling
- CSS calculations with calc()
- Advanced selectors and pseudo-elements
- CSS shapes and clip-path
- Filters and blend modes
- 3D transforms and perspective
- Keyframe animations
- Animation timing functions
- SVG animations
- Scroll-triggered animations
- Parallax scrolling effects
- Performance optimization

Projects built:

- Animated hero section
- Interactive infographic
- 3D card flip gallery
- Scroll storytelling site

Practice: Create animation library

Month 3: Modern CSS & Build Tools (Week 9-13)

Modern CSS Techniques

Topics:

- CSS Custom Properties (variables)
- CSS Grid advanced techniques
- Container queries: The future of responsive
- CSS nesting (native)
- Logical properties for internationalization
- Modern color functions: rgb(), hsl(), lab()
- Aspect-ratio property
- CSS scroll snap for carousels
- Sticky positioning tricks
- CSS counters for numbering
- Print styles for web pages

- Accessibility with CSS

Projects built:

- Magazine-style layout
- Advanced dashboard design
- Product showcase with filters
- Interactive timeline

Practice: Experiment with cutting-edge CSS features

SASS/SCSS & CSS Architecture

Topics:

- SASS: CSS with superpowers
- Variables, nesting, and mixins
- Functions and calculations in SASS
- Partial and imports
- Extends and placeholders
- BEM methodology: Organized CSS
- CSS architecture patterns
- Component-based styling
- Utility-first CSS introduction
- CSS-in-JS concepts
- PostCSS and autoprefixer
- Building a design system

Projects built:

- Design system with SASS
- Themed website with variables
- Component library
- Multi-theme dashboard

Practice: Refactor projects using SASS

Build Tools & Modern Workflow

Topics:

- NPM: The package manager
- Package.json and dependencies
- Git and GitHub mastery
- Webpack basics: Module bundling
- Babel: Modern JavaScript everywhere
- Development vs production builds
- Hot module replacement
- Code splitting basics
- Environment variables
- Linting with ESLint
- Prettier for code formatting
- Pre-commit hooks

Projects built:

- Set up modern build pipeline
- Optimize existing projects
- Create project boilerplate
- Automated workflow setup

Practice: Configure build tools for all projects

Tailwind CSS & Utility-First

Topics:

- Tailwind CSS: Why developers love it
- Utility classes philosophy
- Setting up Tailwind
- Responsive utilities
- Hover, focus, and state variants
- Custom configuration
- Extending Tailwind
- Component extraction
- Tailwind UI components
- JIT mode benefits
- Tailwind vs traditional CSS
- Building with Tailwind

Projects built:

- Landing page with Tailwind
- Admin dashboard UI
- E-commerce product page
- Social media feed clone

Practice: Rebuild projects with Tailwind

Phase 1 Capstone Project

Topics:

- Project planning and wireframing
- Design mockups in Figma
- Setting up project structure
- Implementing responsive design
- Adding interactivity
- Testing and debugging
- Deployment preparation

Projects built:

- CAPSTONE: Multi-page Portfolio Website
- Include: Animations, API integration, responsive design, dark mode

- Alternative: Small business website
 - Alternative: Interactive educational site
- Assessment: Phase 1 Final - Complete responsive website from scratch

PHASE 2: REACT & MODERN FRAMEWORKS (MONTHS 4-6, WEEKS 14-26)

Master React, the most popular frontend framework. Build complex, interactive applications like the pros.

Months 4-5: React Fundamentals & Ecosystem (Week 14-21)

React Basics

Topics:

- Why React? Facebook, Instagram, Netflix use it
- Vite: Quick start
- JSX: HTML in JavaScript (weird but powerful)
- Components: Building blocks of React
- Functional vs Class components
- Props: Passing data down
- State: Component memory
- useState hook explained
- Event handling in React
- Conditional rendering
- Lists and keys
- Component lifecycle

Projects built:

- Instagram feed clone
- Interactive todo app
- Weather dashboard
- Movie gallery

Practice: Build 20 React components

React Hooks Deep Dive

Topics:

- useState: Managing component state
- useEffect: Side effects and lifecycle
- useContext: Avoiding prop drilling
- useReducer: Complex state logic
- useCallback: Optimizing functions
- useMemo: Expensive calculations
- useRef: DOM references and values
- Custom hooks: Reusable logic
- Rules of hooks
- Hook patterns and best practices
- Testing hooks
- Common hook mistakes

Projects built:

- Chat application with hooks
- Shopping cart with useReducer
- Theme system with Context
- Custom hooks library

Practice: Create 10 custom hooks

React Router & Navigation

Topics:

- Single Page Applications (SPAs)
- React Router setup
- Routes and paths
- Link vs NavLink
- Route parameters
- Nested routes
- Protected routes
- Programmatic navigation
- Query parameters
- Route guards and authentication
- 404 pages and redirects
- Lazy loading routes

Projects built:

- Multi-page blog platform
- E-commerce site with routing
- Admin dashboard
- Social media app navigation

Practice: Add routing to previous projects

State Management with Redux

Topics:

- Why Redux? Managing complex state
- Redux principles: Store, actions, reducers
- Setting up Redux
- Redux Toolkit: Modern Redux
- Creating slices
- Dispatching actions

- Selectors and memoization
- Async logic with Redux Thunk
- Redux DevTools
- Redux vs Context API
- When to use Redux
- Redux best practices

Projects built:

- Twitter clone with Redux
- E-commerce cart system
- Real-time dashboard
- Complex form management

Practice: Refactor apps to use Redux

Advanced React Patterns

Topics:

- Higher-Order Components (HOCs)
- Render props pattern
- Compound components
- Controlled vs uncontrolled components
- Error boundaries
- Portals for modals
- React.memo optimization
- Code splitting with lazy/Suspense
- Performance profiling
- React DevTools mastery
- Server-side rendering concepts
- Static site generation

Projects built:

- Modal system with portals
- Form builder with patterns
- Performance-optimized app
- Component library

Practice: Implement all patterns in projects

Month 6: Advanced React & Next.js (Week 22-26)

Styling in React

Topics:

- CSS Modules in React
- Styled-components introduction
- Emotion CSS-in-JS
- Tailwind with React
- Material-UI (MUI) components
- Chakra UI for rapid development
- Ant Design components
- Creating themed components
- Animation libraries: Framer Motion
- React Spring animations
- Lottie animations in React
- Building a design system

Projects built:

- Animated portfolio site
- Dashboard with MUI
- Landing page with animations
- Custom component library

Practice: Style apps with different approaches

Forms & Data Handling

Topics:

- Form handling in React
- Controlled components deep dive
- Form validation strategies
- React Hook Form
- Formik library
- Yup schema validation
- File uploads in React
- Multi-step forms
- Dynamic form fields
- Form state management
- Error handling and display
- Accessibility in forms

Projects built:

- Complex registration form
- Survey builder application
- Job application system
- Dynamic form generator

Practice: Build 10 different form types

Next.js - Production React

Topics:

- Next.js: The React framework
- File-based routing
- Pages and layouts

- Server-side rendering (SSR)
- Static site generation (SSG)
- Incremental Static Regeneration
- API routes in Next.js
- Image optimization
- Built-in CSS support
- Environment variables
- Deployment with Vercel
- SEO optimization

Projects built:

- Blog with Next.js and MDX
- E-commerce site with SSR
- Portfolio with ISR
- Full-stack Next.js app

Practice: Migrate React apps to Next.js

Testing React Applications

Topics:

- Why testing matters
- Jest testing framework
- React Testing Library
- Writing unit tests
- Testing components
- Testing hooks
- Integration testing
- Mocking APIs and modules
- Snapshot testing
- End-to-end with Cypress
- Test coverage reports
- TDD approach

Projects built:

- Add tests to existing projects
- TDD todo application
- Test suite for component library
- E2E tests for user flows

Practice: Achieve 80% test coverage

TypeScript with React

Topics:

- TypeScript basics review
- Setting up React with TypeScript
- Typing props and state
- Event types in React
- Generic components
- Typing hooks
- Context with TypeScript
- Redux with TypeScript
- Common type patterns
- Third-party library types
- TypeScript best practices
- Migration strategies

Projects built:

- Convert React app to TypeScript
- Type-safe component library
- Full TypeScript application
- Type definitions for API

Practice: Use TypeScript in all future projects

Phase 2 Capstone & Performance (Week 25-26)

Performance Optimization

Topics:

- Web performance metrics
- Core Web Vitals
- Lighthouse audits
- Bundle size optimization
- Code splitting strategies
- Lazy loading components
- Image optimization techniques
- Font optimization
- Caching strategies
- Service workers basics
- Progressive Web Apps
- Performance monitoring

Projects built:

- Optimize slow application
- PWA conversion
- Performance dashboard
- Speed optimization challenge

Practice: Optimize all projects for performance

Accessibility & SEO

Topics:

- Web accessibility (a11y) importance
- WCAG guidelines
- Semantic HTML for accessibility
- ARIA labels and roles
- Keyboard navigation
- Screen reader testing
- Color contrast and vision
- Focus management
- SEO fundamentals
- Meta tags and Open Graph
- Structured data
- SEO for SPAs

Projects built:

- Accessibility audit and fixes
- Fully accessible form system
- SEO-optimized blog
- Accessible component library

Practice: Make all projects accessible

Real-time Features

Topics:

- WebSockets introduction
- Socket.io with React
- Real-time notifications
- Live chat implementation
- Collaborative features
- Server-sent events
- WebRTC basics
- Real-time data synchronization
- Optimistic UI updates
- Conflict resolution
- Offline support
- Real-time analytics

Projects built:

- Real-time chat application
- Collaborative whiteboard
- Live dashboard
- Multiplayer game

Practice: Add real-time features to apps

GraphQL with React

Topics:

- GraphQL vs REST
- GraphQL queries and mutations
- Apollo Client setup
- useQuery and useMutation
- Cache management
- Optimistic updates
- Subscriptions for real-time
- Error handling
- Pagination with GraphQL
- File uploads
- Authentication with GraphQL
- GraphQL best practices

Projects built:

- GitHub clone with GraphQL
- Social media with subscriptions
- E-commerce with Apollo
- Real-time collaboration app

Practice: Convert REST apps to GraphQL

Phase 2 Capstone Project

Topics:

- Project ideation and planning
- Technical architecture
- Design system creation
- Component development
- State management setup
- Testing implementation
- Performance optimization
- Deployment strategy

Projects built:

- MAJOR CAPSTONE: Full React Application
- Options: Social network, E-commerce platform, Productivity tool
- Must include: Auth, real-time, complex state, testing, TypeScript
- Deploy to production with CI/CD

Assessment: Phase 2 Final - Production-ready React application

PHASE 3: VUE.JS & ALTERNATIVE FRAMEWORKS (MONTHS 7-9, WEEKS 27-39)

Expand your skills with Vue.js and explore other modern frameworks. Become a versatile

frontend developer.

Months 7-8: Vue.js Mastery (Week 27-34)

Vue.js Fundamentals

Topics:

- Why Vue? The progressive framework
- Vue 3 Composition API
- Creating Vue applications
- Template syntax and directives
- Data binding: v-model magic
- Computed properties
- Watchers for reactive data
- Methods and event handling
- Conditional rendering: v-if, v-show
- List rendering with v-for
- Component basics
- Props and events

Projects built:

- Task manager with Vue
- Weather app rebuild
- Shopping list application
- Interactive quiz game

Practice: Build 15 Vue components

Vue.js Advanced Features

Topics:

- Composition API deep dive
- Reactive and ref
- Custom composables
- Lifecycle hooks
- Provide/inject pattern
- Slots for flexible components
- Dynamic components
- Async components
- Teleport for modals
- Transitions and animations
- Custom directives
- Mixins and composition

Projects built:

- Blog platform with Vue
- Dashboard with charts
- Modal system with teleport
- Animated gallery

Practice: Create Vue component library

Vue Router & Vuex

Topics:

- Vue Router setup
- Dynamic routing
- Nested routes
- Navigation guards
- Route meta fields
- Lazy loading routes
- Vuex state management
- State, getters, mutations, actions
- Modules in Vuex
- Vuex vs Pinia
- Pinia: Modern Vue store
- Persisted state

Projects built:

- Multi-page app with auth
- E-commerce with Vuex
- Admin panel with Pinia
- Social media clone

Practice: Add routing and state to Vue apps

Nuxt.js - Vue Framework

Topics:

- Nuxt.js introduction
- File-based routing
- Pages and layouts
- Server-side rendering
- Static site generation
- Auto-imports feature
- Data fetching strategies
- Nuxt modules
- SEO optimization
- Deployment options
- Nuxt 3 features
- Building full-stack apps

Projects built:

- Blog with Nuxt

- E-commerce SSR app
- Documentation site
- Portfolio with Nuxt

Practice: Convert Vue apps to Nuxt

Vue.js Ecosystem

Topics:

- Vuetify: Material Design
- Element Plus UI library
- Quasar Framework
- VueUse composables
- Vite build tool
- Vue DevTools mastery
- Testing with Vitest
- Vue Test Utils
- Storybook for Vue
- Vue with TypeScript
- Vue 3 best practices
- Migration from Vue 2

Projects built:

- UI kit with Vuetify
- Admin template
- Component documentation
- TypeScript Vue app

Practice: Master Vue ecosystem tools

Month 9: Modern Frameworks & Tools (Week 35-39)

Svelte & SvelteKit

Topics:

- Svelte: No virtual DOM
- Svelte syntax and reactivity
- Components in Svelte
- Stores for state management
- Animations and transitions
- Actions and use directive
- SvelteKit framework
- Routing in SvelteKit
- Server-side rendering
- API routes
- Deployment strategies
- Svelte vs other frameworks

Projects built:

- Todo app in Svelte
- Portfolio with SvelteKit
- Real-time chat app
- Blog platform

Practice: Compare Svelte with React/Vue

Angular Basics

Topics:

- Angular: The enterprise framework
- TypeScript in Angular
- Angular CLI
- Components and templates
- Data binding
- Directives and pipes
- Services and dependency injection
- RxJS and Observables
- Routing in Angular
- Forms and validation
- HTTP client
- Angular Material

Projects built:

- Task tracker in Angular
- Dashboard application
- CRUD application
- Angular enterprise app

Practice: Build Angular components

Web Components & Lit

Topics:

- Web Components standard
- Custom Elements API
- Shadow DOM
- HTML Templates
- Lit framework
- Creating Lit components
- Reactive properties
- Styling Web Components
- Publishing components
- Framework-agnostic components
- Micro frontends concept

- Using Web Components everywhere

Projects built:

- Custom element library
- Lit component collection
- Framework-agnostic UI kit
- Micro frontend demo

Practice: Create reusable Web Components

Static Site Generators

Topics:

- JAMstack architecture
- Gatsby.js with React
- Astro framework
- 11ty (Eleventy)
- Hugo for speed
- Content management
- Markdown and MDX
- Image optimization
- SEO optimization
- Deployment platforms
- Netlify and Vercel
- Building blogs and docs

Projects built:

- Blog with Gatsby
- Documentation with Astro
- Portfolio with 11ty
- Marketing site with Hugo

Practice: Build static sites with different tools

Phase 3 Integration

Topics:

- Framework comparison
- Choosing the right tool
- Migration strategies
- Performance comparison
- Developer experience
- Community and ecosystem
- Job market analysis
- Future trends

Projects built:

- Same app in React, Vue, Svelte
- Framework comparison report
- Migration guide
- Decision matrix tool

Practice: Master framework selection

Phase 3 Capstone & Advanced Topics (Week 38-39)

Micro Frontends

Topics:

- Micro frontend architecture
- Module Federation
- Single-spa framework
- Sharing state between apps
- Routing in micro frontends
- CI/CD for micro frontends
- Testing strategies
- Performance considerations
- Team organization
- Communication patterns
- Deployment strategies
- Real-world examples

Projects built:

- Micro frontend platform
- Multi-framework app
- Federated modules demo
- Enterprise architecture

Practice: Build micro frontend system

Advanced Animation & 3D

Topics:

- Advanced CSS animations
- GSAP (GreenSock) mastery
- ScrollTrigger animations
- Lottie and After Effects
- Three.js basics
- 3D in the browser
- React Three Fiber
- WebGL fundamentals
- Shaders basics
- Interactive 3D experiences
- VR/AR on the web
- Performance optimization

Projects built:

- 3D portfolio site
- Interactive 3D game
- Animated landing page
- WebGL experiment

Practice: Create stunning visual experiences

Advanced State Management

Topics:

- State machines with XState
- MobX for reactive programming
- Recoil for React
- Jotai atomic state
- Valtio proxy state
- Zustand simplicity
- Redux Toolkit Query
- React Query (TanStack Query)
- SWR data fetching
- Optimistic updates
- Cache strategies
- State persistence

Projects built:

- Complex app with XState
- Data-heavy dashboard
- Offline-first application
- Real-time sync system

Practice: Master different state solutions

Design Systems & Component Libraries

Topics:

- Design system principles
- Design tokens
- Component documentation
- Storybook mastery
- Component testing
- Accessibility testing
- Visual regression testing
- Publishing to NPM
- Versioning strategies
- Monorepo management
- Documentation sites
- Team adoption

Projects built:

- Complete design system
- NPM component library
- Storybook showcase
- Documentation site

Practice: Build professional component library

Phase 3 Capstone Project

Topics:

- Multi-framework project
- Architecture design
- Technology selection
- Implementation strategy
- Testing approach
- Documentation
- Deployment pipeline
- Performance goals

Projects built:

- MAJOR CAPSTONE: Enterprise Application
- Build with multiple frameworks
- Include: Micro frontends, design system, real-time, animations
- Production deployment with monitoring

Assessment: Phase 3 Final - Complex multi-framework application

----- PHASE 4: PROFESSIONAL SKILLS & CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52) -----

Master professional development skills, build your portfolio, and launch your frontend career.

Months 10-11: Professional Development (Week 40-47)

Portfolio Development

Topics:

- Portfolio strategy for teens
- Showcasing your best work
- Case study creation
- Project documentation
- Before/after demonstrations
- Performance metrics
- Client testimonials

- Personal branding
- Domain and hosting
- SEO for portfolios
- Analytics setup
- Contact forms and CTAs

Projects built:

- Professional portfolio site
- Case studies for 5 projects
- Interactive resume
- Skills showcase

Practice: Polish all project presentations

Freelancing Fundamentals

Topics:

- Freelancing as a teen
- Setting up your business
- Pricing your services
- Finding first clients
- Upwork and Fiverr profiles
- Writing proposals
- Client communication
- Project management
- Time tracking
- Invoicing and payments
- Contracts and agreements
- Building reputation

Projects built:

- Freelance service packages
- Client onboarding system
- Invoice templates
- First client project

Practice: Complete first paid project

Open Source Contribution

Topics:

- Why open source matters
- Finding projects to contribute
- Understanding codebases
- Making first contribution
- Pull request etiquette
- Code review process
- Documentation contributions
- Creating your own projects
- Building community
- Maintaining projects
- GitHub sponsors
- Open source licenses

Projects built:

- First open source PR
- Create open source tool
- Documentation improvements
- Bug fixes and features

Practice: Contribute to 5 projects

Interview Preparation

Topics:

- Frontend interview process
- Technical interview prep
- JavaScript questions
- React/Vue questions
- CSS challenges
- Algorithm basics
- System design for frontend
- Behavioral questions
- Portfolio presentation
- Whiteboard coding
- Take-home projects
- Salary negotiation

Projects built:

- Interview prep materials
- Practice problems solutions
- Mock interview recordings
- Take-home project samples

Practice: Daily interview practice

Advanced Git & Collaboration

Topics:

- Git advanced commands
- Branching strategies
- Merge vs rebase
- Resolving conflicts
- Git hooks
- Conventional commits

- Semantic versioning
- GitHub Actions mastery
- Code review best practices
- Team collaboration
- Documentation standards
- Project management tools

Projects built:

- Team collaboration project
- CI/CD pipeline setup
- Automated workflows
- Contributing guide

Practice: Collaborate on group project

Month 11: Specialized Skills (Week 44-47)

E-commerce Development

Topics:

- E-commerce requirements
- Product catalogs
- Shopping cart systems
- Checkout flows
- Payment integration
- Stripe and PayPal
- Order management
- Inventory tracking
- User accounts
- Wishlist and reviews
- Email notifications
- Analytics and tracking

Projects built:

- Full e-commerce site
- Product showcase
- Checkout system
- Admin dashboard

Practice: Build complete online store

CMS & Headless Solutions

Topics:

- Content Management Systems
- Headless CMS concept
- Strapi integration
- Contentful setup
- Sanity.io features
- WordPress as headless
- GraphQL with CMS
- Content modeling
- Rich text handling
- Media management
- Multilingual content
- Content workflows

Projects built:

- Blog with headless CMS
- Marketing site with CMS
- Documentation platform
- News portal

Practice: Integrate 3 different CMS solutions

Mobile-First Development

Topics:

- Mobile-first philosophy
- Touch interactions
- Gesture handling
- Mobile performance
- PWA deep dive
- Service workers mastery
- Offline strategies
- App-like experiences
- Mobile UI patterns
- Viewport optimization
- Mobile testing tools
- Cross-device testing

Projects built:

- PWA social app
- Offline-first application
- Mobile game
- App-like website

Practice: Convert sites to PWAs

Deployment & DevOps

Topics:

- Deployment strategies
- Netlify deployment
- Vercel configuration

- AWS basics for frontend
- CloudFlare Pages
- GitHub Pages
- Docker for frontend
- CI/CD pipelines
- Environment management
- Monitoring and logging
- Error tracking
- Performance monitoring

Projects built:

- Multi-environment setup
- Automated deployment
- Monitoring dashboard
- Deploy all projects

Practice: Deploy to multiple platforms

Emerging Technologies

Topics:

- WebAssembly basics
- AI in frontend
- Voice interfaces
- WebXR (VR/AR)
- Blockchain integration
- Edge computing
- 5G implications
- IoT dashboards
- Quantum computing UI
- Future of frontend
- Continuous learning
- Staying updated

Projects built:

- AI-powered interface
- Voice-controlled app
- WebXR experience
- Future tech demo

Practice: Experiment with new tech

Month 12: Final Project & Launch (Week 48-52)

Final Project Planning

Topics:

- Project ideation
- Market research
- User research
- Technical planning
- Design process
- Architecture decisions
- Timeline creation
- Risk assessment

Projects built:

- FINAL PROJECT START
- Comprehensive planning
- Design mockups
- Technical specification
- Project roadmap

Practice: Thorough planning phase

Final Project Development - Part 1

Topics:

- Setting up project
- Component architecture
- State management setup
- Routing implementation
- API integration
- Authentication system
- Core features
- Testing setup

Projects built:

- Build core functionality
- Implement main features
- Set up infrastructure
- Initial testing

Practice: Rapid development sprint

Final Project Development - Part 2

Topics:

- Advanced features
- Performance optimization
- Accessibility audit
- SEO implementation
- Error handling
- Edge cases
- User testing

- Bug fixes

Projects built:

- Complete all features
- Optimize performance
- Fix all bugs
- Polish UI/UX

Practice: Quality assurance phase

Launch Preparation

Topics:

- Deployment setup
- Domain configuration
- SSL certificates
- Analytics setup
- SEO optimization
- Social media prep
- Launch strategy
- Marketing plan

Projects built:

- Deploy to production
- Marketing materials
- Launch campaign
- Documentation

Practice: Pre-launch checklist

Week 51-52: Graduation & Career Launch

Career Launch

Topics:

- Job search strategies
- Resume optimization
- LinkedIn profile
- GitHub portfolio
- Networking tips
- Interview scheduling
- Follow-up strategies
- First job tips
- Freelance vs employment
- Remote work skills
- Continuous learning
- Career growth path

Projects built:

- Job application package
- Interview portfolio
- Professional network
- Career plan

Deliverables:

- Professional resume
- 30+ portfolio projects
- GitHub profile
- LinkedIn presence

Final Presentations

Topics:

- Presenting your work
- Demo preparation
- Public speaking
- Technical explanation
- Q&A handling
- Peer reviews
- Feedback incorporation
- Success celebration

Projects built:

- Final project presentation
- Portfolio walkthrough
- Technical deep dive
- Future plans presentation

Deliverables:

- Recorded presentation
- Live demo
- Peer feedback
- Mentor evaluation

Graduation & Beyond

Topics:

- Course completion
- Certification ceremony
- Alumni network
- Continued support
- Learning resources
- Community access
- Mentorship program
- Success stories

- Future opportunities
- Staying connected
- Giving back
- Next steps

Deliverables:

- Course Completion Certificate
- Professional Frontend Developer Certification
- Letter of Recommendation
- Alumni Network Access
- Lifetime Course Updates

Assessment: FINAL SHOWCASE: Present complete portfolio to industry professionals

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 15+ static websites, responsive designs, API integrations
- Phase 2: 12+ React applications, Next.js projects, real-time apps
- Phase 3: 10+ Vue/Svelte apps, micro frontends, design systems
- Phase 4: Professional portfolio, client projects, open source contributions
- Total: 40+ complete projects from simple to complex

Total projects: 40+ frontend projects, 10+ production deployments

SKILLS THE STUDENT WILL HAVE AT THE END

- Languages: HTML5, CSS3, JavaScript ES6+, TypeScript
- Frameworks: React, Vue, Svelte, Angular basics, Next.js, Nuxt.js
- Styling: SASS, Tailwind, Styled-components, CSS-in-JS, Animations
- State: Redux, Vuex, Context API, Zustand, MobX
- Tools: Git, VS Code, Webpack, Vite, NPM, Chrome DevTools
- Testing: Jest, React Testing Library, Cypress, Vitest
- Design: Responsive, Accessibility, UI/UX, Figma basics
- Performance: Optimization, PWAs, Core Web Vitals
- Deployment: Netlify, Vercel, GitHub Pages, CI/CD
- Professional: Portfolio, Freelancing, Open Source, Interviews

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-4 hours
 Hands on coding: 5-6 hours
 Projects: 3-4 hours
 Practice problems: 2 hours
 Total per week: 10-12 hours

SUPPORT INCLUDED

Live sessions: Weekly group mentoring sessions
 Mentorship: 1-on-1 guidance from experienced developers
 Community: Active Discord community of teen developers
 Code review: Professional code reviews for all projects
 Freelance support: Help landing first clients
 Lifetime access: All content and future updates
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each quarter
 Final certificate: Professional Frontend Developer Certification
 LinkedIn badge: Verified LinkedIn certification
 Portfolio review: Professional portfolio assessment
 Job readiness: Job-ready certification
 Freelance ready: Freelance-ready certification

WHERE THIS CAN LEAD

- Junior Frontend Developer
- React Developer
- Vue.js Developer
- UI/UX Developer
- Freelance Web Developer
- Frontend Engineer
- Web Designer
- Full Stack Developer (with backend knowledge)
- Technical Founder
- Creative Developer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this frontend development course suitable for teenagers with no coding experience?
A: Absolutely! This course is specifically designed for teens (13-19 years) with zero prior coding experience. We start from the very basics of HTML, CSS, and JavaScript, using projects that resonate with teen interests like building Instagram clones, gaming websites, and portfolio sites. The curriculum is paced perfectly for school-going students.

Q: How much time per week does this frontend development course require?
A: The course requires 10-12 hours per week, making it manageable alongside school and extracurricular activities. We recommend spreading this across 5-6 days with 1-2 hours of practice daily. Weekend students often spend 4-5 hours on Saturday and Sunday for project work.

Q: What technologies and frameworks will I learn in this frontend course for teens?
A: You'll master the complete modern frontend stack: HTML5, CSS3, JavaScript ES6+, TypeScript, React, Vue.js, Svelte, Next.js, Nuxt.js, Tailwind CSS, GSAP animations, and Three.js for 3D. By completion, you'll be proficient in 15+ technologies used by top tech companies.

Q: Can I start freelancing or get a job as a teenager after this course?
A: Yes! Many of our teen students start freelancing on Fiverr and Upwork while still in high school. We teach pricing, client communication, and how to land your first projects. For jobs, teens typically secure internships first, with some landing junior positions at startups. Your portfolio of 40+ projects makes you very competitive.

Q: How does this frontend course help with college applications and admissions?
A: Having web development skills significantly boosts college applications. Your GitHub portfolio demonstrates technical ability and initiative. Many students use their capstone projects as application showcases. For computer science programs, this practical experience can set you apart from applicants who only have academic credentials.

Q: What projects will I build in this 12-month frontend development program?
A: You'll build 40+ projects including: responsive portfolios, social media clones, e-commerce stores, real-time chat apps, progressive web apps (PWAs), 3D interactive websites, and a complete enterprise application as your capstone. Every project is deployed live and added to your professional portfolio.

To enrol or to book a free demo for this course, message Shivam Sir on
+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/frontend-development-masterclass-for-teens>

=====

COURSE 55 OF 118: FULL STACK WEB DEVELOPMENT FOR TEENS: REACT & NODE.JS

=====

Course link: <https://learn.modernagecoders.com/courses/full-stack-web-development-teens-masterclass>
Subject area: Teen Full Stack Web Development
Designed for: Teens
Who can join: Complete Beginner to Professional Full Stack Developer
Course length: 24 months (104 weeks)
Time needed: 12-18 hours/week recommended
Certificate: Teen Full Stack Developer Certification - Industry Recognized

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Full stack web development course for teens: build Instagram, TikTok and Netflix clones with HTML, CSS, JavaScript, React, Node.js, Python and databases.

In one line: From Zero Code to Building Instagram, Netflix, Discord & Your Dream Apps!

ABOUT THE PROGRAMME

This isn't your boring computer class. This 2-year masterclass is designed specifically for teenagers who want to build the apps they use every day. You'll start from absolute zero and progress to building complex applications like social media platforms, streaming services, gaming sites, and whatever you can imagine. Learn both frontend (what users see) and backend (the engine that powers apps). By graduation, you'll have 50+ projects in your portfolio, real-world skills that colleges and employers want, and the ability to build any web application you can dream of. Plus, you can start earning money through freelancing while still in school!

WHAT MAKES THIS COURSE DIFFERENT

- Designed by young developers who understand teen interests
- Build clones of apps teens actually use daily
- Learn both frontend and backend - become truly versatile

- Project-based learning with fun, relevant examples
- Flexible schedule that works with school and activities
- Teen-only community with peers your age
- Skills that give you a massive advantage for college applications
- Start earning money before you graduate high school
- Parent dashboard for progress tracking (optional)

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Web Basics, HTML, CSS, JavaScript, First Projects
 Phase 2: Frontend Mastery (Months 7-12): React, Vue, Advanced JavaScript, UI/UX
 Phase 3: Backend Development (Months 13-18): Node.js, Python, Databases, APIs
 Phase 4: Professional Skills (Months 19-24): Full Stack Projects, DevOps, Career Prep

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No prior coding experience needed
 Coding experience: Complete beginner friendly
 Equipment: Any computer with internet connection
 Time commitment: 12-18 hours per week
 English: Basic English comprehension
 Motivation: Passion for building cool stuff

WHO THIS COURSE IS FOR

Students: Middle and high school students (13-19 years)
 Working professionals: Young adults switching careers
 Entrepreneurs: Teens with startup ideas
 Freelancers: Students wanting to earn while learning
 Kids: Motivated pre-teens with supervision
 Anyone: Anyone who wants to build web applications

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: WEB DEVELOPMENT FOUNDATIONS (MONTHS 1-6, WEEKS 1-26) -----

Start from zero and build a rock-solid foundation. Learn how the internet works, create beautiful websites, and master JavaScript - the language that powers the web.

Months 1-2: HTML, CSS & Your First Websites (Week 1-8)

Introduction to Web Development

Topics:

- How the internet works - explained for teens
- What happens when you open Instagram or YouTube
- Frontend vs Backend - what's the difference?
- Setting up your coding environment (VS Code)
- Essential VS Code extensions and themes
- Introduction to HTML - the skeleton of websites
- HTML tags and structure basics
- Creating your first webpage
- Adding text, headings, and paragraphs
- Links and navigation between pages
- Images and media elements
- Git and GitHub - saving your code like a pro

Projects built:

- Personal profile page
- Favorite band/artist fan page
- School project presentation website

Practice: Create 5 different HTML pages about your interests

Advanced HTML & Forms

Topics:

- HTML5 semantic elements for better structure
- Creating forms - how login pages work
- Input types: text, email, password, number, date
- Dropdowns, checkboxes, and radio buttons
- HTML tables for displaying data
- Embedding content: YouTube videos, Spotify, Maps
- Audio and video elements
- Canvas element introduction
- Meta tags for SEO
- Accessibility - making sites for everyone
- HTML best practices
- Validating HTML code

Projects built:

- Multi-page school website
- Survey form for friends

- Restaurant menu with ordering form

Practice: Build 10 different types of forms

CSS - Making Things Look Amazing

Topics:

- CSS basics - adding style to HTML
- Selectors: targeting elements precisely
- Colors, backgrounds, and gradients
- Typography - making text look professional
- Box Model - spacing and layout basics
- Borders, shadows, and visual effects
- CSS units: px, %, em, rem, vh, vw
- Positioning elements on the page
- Display properties and visibility
- Overflow and scrolling
- Z-index and layering
- Pseudo-classes and pseudo-elements

Projects built:

- Style your HTML projects beautifully
- Create a magazine-style layout
- Design product cards like Amazon

Practice: Recreate 10 popular website components

Modern CSS Layouts & Responsive Design

Topics:

- Flexbox - create flexible layouts easily
- CSS Grid - magazine and blog layouts
- Responsive design - mobile-first approach
- Media queries for different screen sizes
- CSS animations and transitions
- Keyframe animations
- Transform and 3D effects
- CSS variables (custom properties)
- Modern CSS features
- CSS frameworks introduction (Bootstrap)
- Tailwind CSS basics
- CSS best practices and organization

Projects built:

- Responsive portfolio website
- Animated landing page
- Instagram-style photo grid

Practice: Make all projects mobile-responsive

Months 3-4: JavaScript Programming Fundamentals (Week 9-17)

JavaScript Basics

Topics:

- What is JavaScript? Why it's everywhere
- Variables and data types
- let, const, var - which to use when
- Numbers and math operations
- Strings and text manipulation
- Booleans and logical thinking
- Arrays - managing lists of data
- Array methods: push, pop, shift, unshift
- Objects - grouping related data
- Accessing and modifying object properties
- Console.log and debugging basics
- Comments and code documentation

Projects built:

- Interactive calculator
- Mad libs game generator
- Grade calculator for school

Practice: Solve 30 JavaScript basics challenges

Control Flow & Functions

Topics:

- If-else statements - making decisions
- Switch statements for multiple choices
- Comparison operators
- Logical operators (AND, OR, NOT)
- Ternary operator - shorthand conditions
- For loops - repeating tasks
- While and do-while loops
- Break and continue statements
- Functions - reusable code blocks
- Parameters and arguments
- Return values
- Arrow functions - modern syntax

Projects built:

- Rock, paper, scissors game
- Password strength checker
- Number guessing game

Practice: Build 10 mini-games with JavaScript
DOM Manipulation - Making Pages Interactive

Topics:

- What is the DOM?
- Selecting elements with JavaScript
- querySelector and querySelectorAll
- Changing content with innerHTML and textContent
- Modifying styles with JavaScript
- Adding and removing CSS classes
- Creating new elements dynamically
- Event listeners - responding to user actions
- Click, hover, and keyboard events
- Form handling with JavaScript
- Local storage - saving data
- Building interactive UIs

Projects built:

- Interactive todo list
- Color scheme switcher
- Typing speed test game

Practice: Add interactivity to 10 static websites

Advanced JavaScript Concepts

Topics:

- Array methods: map, filter, reduce
- forEach vs map - when to use each
- Finding and searching arrays
- Sorting and reversing data
- Object methods and manipulation
- Destructuring arrays and objects
- Spread and rest operators
- Template literals for dynamic strings
- Regular expressions basics
- Date and time in JavaScript
- Math object and random numbers
- JSON - working with data

Projects built:

- Student grade management system
- Playlist manager like Spotify
- Search and filter application

Practice: Master array methods with 25 exercises

Asynchronous JavaScript & APIs

Topics:

- Synchronous vs asynchronous code
- setTimeout and setInterval
- Promises - handling future values
- Async/await - modern async syntax
- Fetch API - getting data from servers
- Working with REST APIs
- Handling API responses
- Error handling in async code
- Loading states and spinners
- Environment variables for API keys
- CORS and API security basics
- Popular APIs for teen projects

Projects built:

- Weather app for your city
- Meme generator using API
- Movie search app

Practice: Integrate 5 different fun APIs

Months 5-6: Advanced Frontend & First Framework (Week 18-26)

Modern JavaScript (ES6+)

Topics:

- ES6+ features overview
- Classes and object-oriented programming
- Constructor functions
- Inheritance with extends
- Static methods and properties
- Modules - organizing code properly
- Import and export statements
- Default vs named exports
- NPM - the package manager
- Installing and using packages
- Package.json explained
- Building modern JavaScript projects

Projects built:

- Modular JavaScript application
- Object-oriented game
- NPM-based project setup

Practice: Refactor old projects with modern JS

Build Tools & Development Workflow

Topics:

- Why we need build tools
- Webpack basics and configuration
- Babel - writing future JavaScript today
- Development vs production builds
- Hot module replacement
- Source maps for debugging
- Code splitting and lazy loading
- Environment variables
- Linting with ESLint
- Code formatting with Prettier
- Git workflow for teams
- Continuous integration basics

Projects built:

- Configure Webpack from scratch
- Set up professional dev environment
- Deploy project with CI/CD

Practice: Optimize 5 projects with build tools

Introduction to React

Topics:

- What is React? Why it's so popular
- React vs vanilla JavaScript
- Vite setup
- JSX - HTML in JavaScript
- Components - building blocks of React
- Props - passing data to components
- State - making components dynamic
- useState hook basics
- Handling events in React
- Conditional rendering
- Lists and keys in React
- Basic React project structure

Projects built:

- React todo application
- Instagram post component
- Interactive quiz in React

Practice: Convert 5 vanilla JS projects to React

CSS Frameworks & UI Libraries

Topics:

- Bootstrap 5 - quick professional layouts
- Tailwind CSS - utility-first styling
- Material-UI for React
- Chakra UI components
- Styled-components basics
- CSS-in-JS concepts
- Theme management
- Responsive design patterns
- Mobile-first development
- Accessibility best practices
- Performance optimization
- Choosing the right framework

Projects built:

- Dashboard with Bootstrap
- Tailwind CSS landing page
- Material-UI admin panel

Practice: Build same UI with 3 different frameworks

Phase 1 Capstone Project

Topics:

- Project planning and wireframing
- Choosing the right tech stack
- Setting up project structure
- Version control best practices
- Building in iterations
- Testing your application
- Deployment to Netlify/Vercel
- Project documentation

Projects built:

- CAPSTONE: Full Frontend Application
- Options: Social media clone, E-commerce site, Gaming platform, Streaming service UI
- Must include: Responsive design, API integration, Local storage, Modern JavaScript

Assessment: Complete frontend project with all Phase 1 skills

PHASE 2: ADVANCED FRONTEND & MODERN FRAMEWORKS (MONTHS 7-12, WEEKS 27-52)

Master modern frontend frameworks, build complex UIs, and create professional-grade applications that look and feel like real products.

Months 7-8: React Mastery (Week 27-35)

Advanced React Concepts

Topics:

- useEffect hook - side effects in React
- useEffect dependencies and cleanup
- Data fetching in React
- Loading and error states
- useContext for global state
- useReducer for complex state logic
- Custom hooks creation
- Performance optimization with useMemo
- useCallback for function memoization
- useRef for DOM access
- React.memo for component optimization
- React DevTools mastery

Projects built:

- Twitter clone with React
- Real-time chat interface
- YouTube video player clone

Practice: Build 10 custom hooks for common patterns

React Router & Navigation

Topics:

- Single Page Applications (SPAs)
- React Router setup and configuration
- Routes, Route, and Switch components
- Link and NavLink components
- URL parameters with useParams
- Query strings with useLocation
- Nested routing patterns
- Protected routes and authentication
- Route guards and redirects
- Lazy loading routes
- Route transitions and animations
- Browser history management

Projects built:

- Multi-page blog platform
- E-commerce site with routing
- Admin dashboard with nested routes

Practice: Add routing to 5 existing projects

State Management with Redux

Topics:

- Why Redux? State management at scale
- Redux core concepts: Store, Actions, Reducers
- Setting up Redux in React
- React-Redux hooks: useSelector, useDispatch
- Redux Toolkit - modern Redux
- CreateSlice and createAsyncThunk
- Handling async operations
- Redux DevTools for debugging
- Redux middleware
- Normalizing state shape
- Redux best practices
- Redux alternatives: Zustand, MobX

Projects built:

- Instagram clone with Redux
- Shopping cart with Redux
- Real-time notification system

Practice: Implement Redux in 3 complex apps

Testing React Applications

Topics:

- Why testing matters
- Jest testing framework
- React Testing Library
- Writing unit tests for components
- Testing props and state
- Testing user interactions
- Mocking API calls
- Integration testing
- Snapshot testing
- Code coverage reports
- End-to-end testing with Cypress
- Test-driven development (TDD)

Projects built:

- Add tests to existing projects
- TDD calculator app
- Test suite for e-commerce app

Practice: Achieve 80% test coverage in projects

React Performance & Optimization

Topics:

- React performance profiling
- Identifying performance bottlenecks
- Code splitting strategies
- Lazy loading components
- Image optimization techniques
- Virtual scrolling for long lists
- Debouncing and throttling
- Web Workers in React
- Service Workers and PWAs
- Caching strategies
- Bundle size optimization
- Performance monitoring tools

Projects built:

- Optimize slow React app
- Build performant social feed
- PWA with offline support

Practice: Improve performance of 5 projects

Months 9-10: Vue.js & Alternative Frameworks (Week 36-44)

Vue.js Fundamentals

Topics:

- Introduction to Vue.js
- Vue vs React comparison
- Vue CLI and project setup
- Vue templates and directives
- Data binding in Vue
- Computed properties and watchers
- Methods and event handling
- Conditional rendering with v-if
- List rendering with v-for
- Component basics in Vue
- Props and custom events
- Slots for component composition

Projects built:

- Todo app with Vue
- Weather dashboard in Vue
- Recipe app with Vue

Practice: Rebuild React projects in Vue

Advanced Vue.js

Topics:

- Vue Router for navigation
- Vuex state management
- Composition API
- Custom directives
- Mixins and composables
- Transitions and animations
- Vue 3 features
- Nuxt.js introduction
- Server-side rendering with Vue
- Vue testing strategies
- Vue DevTools
- Vue best practices

Projects built:

- E-commerce site with Vue
- Social media app with Vuex
- Blog with Nuxt.js

Practice: Build 3 complete Vue applications

Angular Basics

Topics:

- Introduction to Angular
- TypeScript fundamentals
- Angular CLI and project structure
- Components and templates
- Data binding and directives
- Services and dependency injection
- Routing in Angular
- Forms and validation
- HTTP client and APIs
- RxJS basics
- Angular Material
- Angular vs React vs Vue

Projects built:

- Task manager with Angular
- Dashboard with Angular Material
- CRUD app with Angular

Practice: Explore Angular ecosystem

Modern Frontend Tools & Libraries

Topics:

- Next.js for React

- Gatsby for static sites
- Svelte introduction
- Web Components basics
- GraphQL on the frontend
- Apollo Client
- Progressive Web Apps (PWAs)
- WebAssembly basics
- Micro-frontends architecture
- JAMstack architecture
- Headless CMS integration
- Frontend deployment strategies

Projects built:

- Blog with Next.js
- Portfolio with Gatsby
- PWA social app

Practice: Experiment with 5 new technologies

TypeScript for Frontend

Topics:

- TypeScript fundamentals
- Types and interfaces
- Generics in TypeScript
- TypeScript with React
- Type-safe props and state
- TypeScript with Vue
- TypeScript configuration
- Third-party library types
- Advanced TypeScript patterns
- TypeScript best practices
- Migration strategies
- TypeScript tooling

Projects built:

- Convert JS project to TypeScript
- Type-safe React application
- TypeScript component library

Practice: Write all new code in TypeScript

Months 11-12: UI/UX & Professional Frontend (Week 45-52)

UI/UX Design Principles

Topics:

- Design principles for developers
- Color theory and palettes
- Typography on the web
- Layout and composition
- Visual hierarchy
- User experience basics
- User research methods
- Wireframing and prototyping
- Figma for developers
- Design systems creation
- Responsive design patterns
- Accessibility (a11y) standards

Projects built:

- Design and build a design system
- Redesign existing project
- Create accessible component library

Practice: Apply design principles to all projects

Advanced CSS & Animations

Topics:

- CSS architecture (BEM, SMACSS)
- CSS custom properties advanced
- CSS Grid advanced layouts
- Container queries
- CSS Houdini introduction
- Advanced animations with CSS
- GSAP animation library
- Lottie animations
- Three.js for 3D graphics
- WebGL basics
- Canvas API advanced
- SVG animations

Projects built:

- Animated portfolio site
- Interactive 3D showcase
- Game with Canvas API

Practice: Add advanced animations to 5 projects

Mobile-First & Cross-Platform

Topics:

- Mobile-first design strategy
- Touch interactions and gestures

- Mobile performance optimization
- React Native introduction
- Expo for quick mobile development
- Ionic framework basics
- Flutter for web
- Responsive images and videos
- Mobile debugging tools
- App store deployment basics
- Cross-platform considerations
- Native vs web apps

Projects built:

- Mobile app with React Native
- Cross-platform todo app
- Mobile-optimized game

Practice: Make all projects mobile-perfect

Frontend Security & Best Practices

Topics:

- Common frontend vulnerabilities
- XSS prevention techniques
- Content Security Policy (CSP)
- HTTPS and SSL/TLS
- Authentication best practices
- Secure token storage
- Input validation and sanitization
- CORS and security headers
- Dependency security scanning
- Security testing tools
- OWASP Top 10 for frontend
- Privacy and GDPR compliance

Projects built:

- Security audit of projects
- Implement secure authentication
- Build security scanner tool

Practice: Apply security best practices everywhere

Phase 2 Capstone Project

Topics:

- Complex frontend architecture
- Multiple framework integration
- Advanced state management
- Performance optimization
- Comprehensive testing
- Accessibility compliance
- Security implementation
- Production deployment

Projects built:

- CAPSTONE: Netflix/Disney+ Clone
- Features: Video streaming UI, User profiles, Recommendations, Search, Responsive design
- Alternative: Spotify/Apple Music Clone
- Alternative: Discord/Slack Clone

Assessment: Build production-ready frontend application

PHASE 3: BACKEND DEVELOPMENT & DATABASES (MONTHS 13-18, WEEKS 53-78)

Master server-side programming, databases, APIs, authentication, and build the backend systems that power modern web applications.

Months 13-14: Node.js & Express (Week 53-61)

Node.js Fundamentals

Topics:

- What is Node.js? JavaScript on the server
- Node.js installation and setup
- Node REPL and running scripts
- CommonJS modules and require
- NPM and package management
- Core modules: fs, path, http, os
- File system operations
- Creating HTTP servers
- Handling requests and responses
- Streams and buffers
- Events and EventEmitter
- Asynchronous programming in Node

Projects built:

- CLI todo application
- File management system
- Basic HTTP API server

Practice: Build 10 Node.js command-line tools

Express.js Framework

Topics:

- Introduction to Express.js
- Setting up Express server
- Routing and route handlers
- Request and response objects
- Middleware concept and usage
- Built-in and custom middleware
- Handling different HTTP methods
- Route parameters and query strings
- Serving static files
- Template engines (EJS, Pug)
- Error handling in Express
- Express best practices

Projects built:

- RESTful blog API
- User management system
- File upload service

Practice: Build 5 different APIs with Express

MongoDB & Mongoose

Topics:

- NoSQL vs SQL databases
- MongoDB introduction and setup
- MongoDB Atlas cloud database
- CRUD operations in MongoDB
- Data modeling for NoSQL
- Mongoose ODM setup
- Schemas and models
- Validation and middleware
- Relationships and population
- Aggregation pipeline
- Indexing for performance
- MongoDB best practices

Projects built:

- Social media backend
- E-commerce product catalog
- Blog with comments system

Practice: Design 10 different database schemas

RESTful API Design

Topics:

- REST principles and constraints
- API endpoint design
- HTTP status codes usage
- Request/response formats
- API versioning strategies
- Pagination and filtering
- Sorting and searching
- Error handling standards
- API documentation with Swagger
- Postman for API testing
- API rate limiting
- CORS configuration

Projects built:

- Complete REST API for social app
- E-commerce API with all features
- Real-time chat API

Practice: Design and document 5 APIs

Authentication & Authorization

Topics:

- Authentication vs authorization explained
- Password hashing with bcrypt
- JWT tokens - how they work
- Creating and verifying JWTs
- Refresh tokens pattern
- Session-based authentication
- OAuth 2.0 introduction
- Google and Facebook login
- Role-based access control (RBAC)
- Protecting API routes
- Password reset functionality
- Two-factor authentication basics

Projects built:

- Complete auth system with JWT
- Social login integration
- Multi-role permission system

Practice: Implement auth in all backend projects

Months 15-16: Python & Advanced Databases (Week 62-70)

Python Fundamentals for Web

Topics:

- Python basics review

- Python for web development
- Virtual environments setup
- pip package management
- Flask framework introduction
- Routes and views in Flask
- Templates with Jinja2
- Forms and validation
- Flask extensions
- Database integration with Flask
- Flask RESTful APIs
- Flask vs Express comparison

Projects built:

- Blog application with Flask
- API with Flask-RESTful
- Portfolio site with Flask

Practice: Build 5 Flask applications

Django Framework

Topics:

- Django introduction and philosophy
- Django project structure
- Models and migrations
- Django ORM basics
- Views and URLs
- Templates and template language
- Django admin interface
- Forms and ModelForms
- Authentication in Django
- Django REST framework
- Class-based views
- Django best practices

Projects built:

- Instagram clone with Django
- E-commerce site with Django
- CMS with Django admin

Practice: Master Django through projects

SQL & PostgreSQL

Topics:

- Relational database concepts
- PostgreSQL installation and setup
- SQL syntax and commands
- Creating tables and relationships
- Primary and foreign keys
- Joins: INNER, LEFT, RIGHT, FULL
- Aggregate functions
- Subqueries and CTEs
- Indexes and optimization
- Transactions and ACID
- Stored procedures and triggers
- Database design principles

Projects built:

- School management database
- E-commerce database design
- Social network database

Practice: Write 100 SQL queries

Advanced Database Topics

Topics:

- Database normalization
- Denormalization strategies
- Query optimization techniques
- Database indexing strategies
- Caching with Redis
- Database replication
- Sharding and partitioning
- NoSQL alternatives comparison
- GraphQL with databases
- Database migrations
- Backup and recovery
- Database security

Projects built:

- Optimize slow database
- Implement caching layer
- Database migration system

Practice: Optimize 5 database-heavy applications

Real-time & WebSockets

Topics:

- WebSocket protocol explained
- Socket.io with Node.js
- Real-time events and broadcasting
- Rooms and namespaces

- Socket authentication
- Scaling WebSocket servers
- Server-Sent Events (SSE)
- WebRTC basics
- Real-time notifications
- Live collaboration features
- Chat application architecture
- Real-time analytics

Projects built:

- WhatsApp clone backend
- Live collaboration tool
- Real-time gaming server

Practice: Add real-time features to 5 apps

Months 17-18: Advanced Backend & Cloud (Week 71-78)

Microservices Architecture

Topics:

- Monolithic vs microservices
- Microservices design principles
- Service decomposition
- Inter-service communication
- API Gateway pattern
- Service discovery
- Message queues (RabbitMQ)
- Event-driven architecture
- Kafka basics
- Docker for microservices
- Kubernetes introduction
- Microservices challenges

Projects built:

- E-commerce microservices
- Social media microservices
- Video platform architecture

Practice: Design 3 microservice systems

Cloud Services (AWS)

Topics:

- Cloud computing basics
- AWS account and IAM setup
- EC2 instances and deployment
- S3 for file storage
- RDS for databases
- Lambda serverless functions
- API Gateway setup
- CloudFront CDN
- Route 53 DNS
- ElastiCache for caching
- SQS and SNS messaging
- AWS best practices

Projects built:

- Deploy app to AWS
- Serverless API with Lambda
- Cloud-native application

Practice: Deploy all projects to cloud

DevOps & CI/CD

Topics:

- DevOps culture and practices
- Version control with Git (advanced)
- GitHub Actions for CI/CD
- Docker containerization
- Docker Compose for development
- Container registries
- Kubernetes basics
- Infrastructure as Code
- Terraform basics
- Monitoring with Prometheus
- Logging with ELK stack
- GitOps principles

Projects built:

- CI/CD pipeline setup
- Dockerize all applications
- Kubernetes deployment

Practice: Implement DevOps for 5 projects

API Security & Performance

Topics:

- API security best practices
- Input validation and sanitization
- SQL injection prevention
- Rate limiting implementation
- API key management

- HTTPS and SSL/TLS
- Security headers
- API performance optimization
- Caching strategies
- Load balancing
- Performance monitoring
- API analytics

Projects built:

- Secure API implementation
- Performance testing suite
- API monitoring dashboard

Practice: Secure and optimize all APIs

Phase 3 Capstone Project

Topics:

- Full backend system design
- Database architecture
- API design and documentation
- Authentication system
- Real-time features
- Cloud deployment
- Performance optimization
- Security implementation

Projects built:

- CAPSTONE: Complete Backend for Social Platform
- Features: Users, posts, comments, likes, real-time chat, notifications, file uploads
- Alternative: E-commerce Backend
- Alternative: Streaming Platform Backend

Assessment: Build production-ready backend system

----- PHASE 4: FULL STACK MASTERY & CAREER LAUNCH (MONTHS 19-24, WEEKS 79-104) -----

Combine everything you've learned, build production applications, master advanced topics, and prepare for your career as a professional full stack developer.

Months 19-20: Full Stack Applications (Week 79-87)

Full Stack Architecture

Topics:

- Full stack application planning
- Choosing the right tech stack
- MERN stack deep dive
- MEAN stack alternative
- PERN stack with PostgreSQL
- Frontend-backend communication
- API design for frontends
- State management across stack
- File upload architecture
- Email service integration
- Payment processing (Stripe)
- Full stack best practices

Projects built:

- Instagram full stack clone
- E-commerce platform
- Project management tool

Practice: Build 3 different full stack apps

Advanced Authentication Systems

Topics:

- Multi-factor authentication
- Biometric authentication
- Single Sign-On (SSO)
- SAML and OpenID Connect
- Passwordless authentication
- Magic links implementation
- Session management at scale
- Token refresh strategies
- Account verification
- Social authentication advanced
- Auth0 integration
- Security audit practices

Projects built:

- Enterprise auth system
- SSO implementation
- Passwordless login system

Practice: Implement 5 auth strategies

Real-time Applications

Topics:

- Real-time architecture patterns
- WebSocket scaling strategies
- Socket.io advanced features

- Video calling with WebRTC
- Screen sharing implementation
- Collaborative editing (OT/CRDT)
- Real-time gaming backends
- Live streaming basics
- Push notifications (web & mobile)
- Real-time analytics
- Message queue integration
- Event sourcing patterns

Projects built:

- Zoom/Discord clone
- Google Docs clone
- Twitch-style streaming app

Practice: Build 5 real-time features

Performance & Scalability

Topics:

- Performance metrics and KPIs
- Frontend performance optimization
- Backend performance tuning
- Database query optimization
- Caching at every layer
- CDN implementation
- Load balancing strategies
- Horizontal scaling patterns
- Microservices scaling
- Auto-scaling configuration
- Performance monitoring tools
- Stress testing and load testing

Projects built:

- Optimize social media platform
- Build scalable chat system
- Performance monitoring dashboard

Practice: Scale applications to handle millions

Mobile App Development

Topics:

- React Native deep dive
- Expo for rapid development
- Native device features
- Push notifications setup
- App store deployment
- Flutter basics for web devs
- Ionic framework alternative
- PWA vs native apps
- Mobile app monetization
- App analytics integration
- Mobile app security
- Cross-platform strategies

Projects built:

- TikTok mobile clone
- Instagram mobile app
- Cross-platform game

Practice: Convert 3 web apps to mobile

Months 21-22: Advanced Topics & Specializations (Week 88-96)

Machine Learning for Web Devs

Topics:

- ML basics for web developers
- TensorFlow.js introduction
- Pre-trained models usage
- Image recognition in browser
- Natural language processing
- Recommendation systems
- Sentiment analysis
- ML model deployment
- Python ML basics
- Scikit-learn introduction
- ML APIs integration
- Ethical AI considerations

Projects built:

- Image recognition app
- Recommendation engine
- Sentiment analysis tool

Practice: Add ML features to 5 projects

AI-Powered Full-Stack Apps

Topics:

- Where AI belongs in a full-stack app
- Server-side model calls: never from the browser
- Streaming a response into React without jank
- Retrieval over your own database

- Vector search basics, and when plain SQL wins
- Function calling to trigger real app actions
- A minimal agent loop: plan, act, verify
- Guardrails and output validation
- Cost control: tokens, caching, per-user caps
- Evaluating quality: how do you know it got better?
- Privacy and what you must not send to a provider
- Shipping honestly: telling users what the AI did

Projects built:

- AI-powered search over your own data
- Agentic task assistant
- AI feature with full guardrails + evals

Practice: Build 3 AI-powered applications

Game Development for Web

Topics:

- HTML5 Canvas deep dive
- WebGL and graphics
- Phaser.js game framework
- Three.js for 3D games
- Game physics basics
- Sprite animation
- Game state management
- Multiplayer game architecture
- Game monetization
- Publishing web games
- Mobile game deployment
- Game analytics

Projects built:

- 2D platformer game
- Multiplayer battle game
- 3D racing game

Practice: Build 5 different game types

Enterprise Development

Topics:

- Enterprise architecture patterns
- SOLID principles in practice
- Design patterns implementation
- Domain-driven design
- Event-driven architecture
- CQRS pattern
- Message-driven systems
- Enterprise security
- Compliance (GDPR, HIPAA)
- Multi-tenancy
- White-label solutions
- Enterprise deployment

Projects built:

- Enterprise SaaS platform
- Multi-tenant application
- White-label solution

Practice: Build enterprise-grade features

Open Source Contribution

Topics:

- Finding projects to contribute
- Understanding large codebases
- Making quality pull requests
- Code review participation
- Issue triage and management
- Documentation contributions
- Creating your own open source project
- Building a community
- Managing contributors
- Open source licensing
- Funding open source
- Open source career paths

Projects built:

- Contribute to 5 major projects
- Launch your own open source tool
- Build developer community

Practice: Weekly open source contributions

Month 23: Career Preparation (Week 97-100)

Portfolio & Personal Brand

Topics:

- Building impressive portfolio
- Showcasing best projects
- Case study writing
- Technical blog writing
- YouTube/TikTok for developers

- LinkedIn optimization
- GitHub profile enhancement
- Personal website SEO
- Speaking at meetups
- Building online presence
- Networking strategies
- Personal brand development

Projects built:

- Professional portfolio site
- Technical blog with 10 posts
- Video tutorials creation

Practice: Build strong online presence

Interview Preparation

Topics:

- Data structures for interviews
- Algorithm basics
- Big O notation
- Common coding patterns
- System design interviews
- Behavioral questions
- Technical communication
- Whiteboard coding
- Take-home projects
- Salary negotiation
- Contract review
- Interview practice platforms

Projects built:

- Algorithm visualizer
- Interview prep app
- Mock interview recordings

Practice: Daily interview prep for 30 days

Freelancing & Business

Topics:

- Starting as teen freelancer
- Finding first clients
- Project pricing strategies
- Time estimation
- Client communication
- Project management
- Invoicing and payments
- Legal considerations for teens
- Building reputation
- Scaling freelance business
- Agency vs freelance
- Passive income streams

Projects built:

- Complete client project
- Freelance portfolio
- Business website

Practice: Complete 3 freelance projects

Startup & Entrepreneurship

Topics:

- Startup ideation process
- Market validation
- MVP development
- User research for teens
- Growth hacking basics
- Social media marketing
- Product launch strategies
- Fundraising basics
- Pitch deck creation
- Team building
- Startup mistakes to avoid
- Success stories analysis

Projects built:

- Launch micro-SaaS product
- Create pitch deck
- Build landing page

Practice: Validate and launch product idea

Month 24: Final Projects & Graduation (Week 101-104)

Final Capstone Project

Topics:

- Project planning at scale
- Technical specification
- Architecture documentation
- Sprint planning
- Agile development process
- Code quality standards

- Testing strategy
- CI/CD implementation
- Performance benchmarks
- Security audit
- Deployment strategy
- Launch preparation

Projects built:

- FINAL CAPSTONE: Your Dream Application
- Requirements: Full stack, real-time features, mobile responsive, secure auth, payments, analytics, scalable architecture
- Options: Social platform, SaaS tool, E-commerce platform, Entertainment app, Educational platform

Project Launch & Presentation

Topics:

- Production deployment
- Domain and hosting
- SSL and security
- Performance monitoring
- Error tracking setup
- Analytics implementation
- Marketing website
- Product demo video
- Technical presentation
- User documentation
- API documentation
- Post-launch iteration

Deliverables:

- Live production application
- Complete documentation
- Demo video and presentation
- Marketing materials
- User acquisition plan
- Technical blog post
- Open source components
- Performance reports

Graduation & Career Launch

Topics:

- Course completion ceremony
- Portfolio review session
- Career counseling
- Job application strategy
- Internship opportunities
- University pathways
- Continuous learning plan
- Alumni network access
- Mentorship program
- Community involvement
- Teaching opportunities
- Your journey ahead

Deliverables:

- 50+ completed projects portfolio
- Professional GitHub profile
- Published npm packages
- Open source contributions
- Technical blog/YouTube channel
- Professional network
- Industry certifications
- Job-ready skills

Assessment: Comprehensive final assessment and certification ceremony

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Portfolio, Blog, Todo App, Weather App, Quiz Game, E-commerce UI
- Phase 2: Instagram Clone, TikTok Clone, Netflix UI, Discord Interface, Spotify Player
- Phase 3: WhatsApp Backend, YouTube API, Twitter Full Stack, Reddit Clone
- Phase 4: Zoom Clone, Uber Clone, Airbnb Clone, Complete SaaS Platform

Total projects: 50+ projects ranging from simple websites to complex applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Frontend: HTML5, CSS3, JavaScript ES6+, TypeScript, React, Vue, Angular
- Styling: Tailwind, Bootstrap, Styled Components, SASS, Animations
- Backend: Node.js, Express, Python, Django, Flask, REST APIs, GraphQL
- Databases: MongoDB, PostgreSQL, MySQL, Redis, Database Design
- DevOps: Git, Docker, Kubernetes, CI/CD, AWS, Cloud Deployment
- Mobile: React Native, PWAs, Responsive Design
- Security: Authentication, Authorization, OWASP, Encryption
- Testing: Unit Testing, Integration Testing, E2E Testing

- Tools: VS Code, Postman, Chrome DevTools, Figma, Terminal
- Soft Skills: Problem Solving, Debugging, Communication, Team Work

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
Hands on coding: 6-8 hours
Projects: 3-4 hours
Practice problems: 2-3 hours
Total per week: 12-18 hours (flexible with school)

SUPPORT INCLUDED

Live sessions: Weekly teen group coding sessions
Mentorship: Young mentors who understand teens
Community: Discord server exclusively for teen developers
Code review: Professional code reviews on projects
Lifetime access: All content and future updates
Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: 4 phase completion certificates
Final certificate: Full Stack Web Developer Professional Certificate
Linkedin badge: Verifiable LinkedIn certification
Industry recognized: Recognized by tech companies and universities
Portfolio projects: 50+ projects to showcase skills

WHERE THIS CAN LEAD

- Full Stack Developer at tech companies
- Frontend/Backend Developer (specialized)
- Freelance Web Developer
- Technical Co-founder for startups
- DevOps Engineer
- Mobile App Developer
- Open Source Maintainer
- Technical Content Creator
- Coding Instructor/Mentor
- Software Engineering Intern (during college)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What kind of apps will my teen build?

A: They learn the complete stack: HTML, CSS, and JavaScript, then React and Vue on the frontend, and Node.js, Express, Python, and databases like MongoDB on the backend.

Q: What does full stack actually mean?

A: A full stack developer can build both, which is why this course covers the entire journey from a first webpage to a complete working application.

Q: Can a complete beginner join, and at what age?

A: Yes, the course assumes zero coding knowledge and is designed for ages 13 to 19. Motivated slightly younger students can join with supervision, and the early months are deliberately gentle before the pace builds.

Q: What can my teen do with these skills afterwards?

A: By graduation they hold a 50+ project portfolio and the ability to build real web applications end to end, a strong base for college computer science, internships, and personal projects. Many continue into our problem solving and DSA course to sharpen the algorithmic side for exams and interviews.

Q: Does the course offer a free demo?

A: Yes, book a free, no-commitment demo class at learn.modernagecoders.com/contact. Parents are welcome to sit in; it is the easiest way to see whether the teaching style clicks for your teen.

Q: What format do the classes follow?

A: Live 1-hour online classes with a real teacher, never pre-recorded videos. Groups are small and hands-on: students build during class, ask questions, and get unstuck immediately.

Q: Will my teen be certified at the end?

A: Yes, graduates receive a full stack developer completion certificate. More importantly, they finish holding a portfolio of 50+ real projects, working applications that prove the certificate means something.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/full-stack-web-development-teens-masterclass>

=====

COURSE 56 OF 118: FULL STACK WEB DEVELOPMENT: FRONTEND, BACKEND & DEVOPS

=====

Course link: <https://learn.modernagecoders.com/courses/full-stack-web-development-masterclass-college>

Subject area: Professional Web Development

Designed for: College students

Who can join: Complete Beginner to Advanced Professional

Course length: 24 months (104 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Industry-recognized certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Full stack web development course: master frontend, backend, databases, DevOps and system design, from absolute basics to production-ready systems.

In one line: From 'Hello World' to Production-Ready Enterprise Applications

ABOUT THE PROGRAMME

This is not just a course, it's a complete transformation program. Whether you're a student, working professional, or someone with zero coding experience, this 2-year masterclass will turn you into a highly skilled full stack developer capable of building, deploying, and scaling real-world applications. You'll master every layer of web development: from pixel-perfect frontend interfaces to robust backend systems, from database optimization to cloud infrastructure, from basic algorithms to system design interviews. By the end, you'll have built 20+ projects, contributed to open source, and be ready for senior developer roles.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no prerequisites required
- 2 years of structured, progressive learning
- Covers 100% of modern web development stack
- Real industry projects and case studies
- Interview preparation and system design
- Lifetime access and updates
- Build production-ready portfolio

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Computer Science Basics, HTML, CSS, JavaScript

Phase 2: Frontend Mastery (Months 7-12): React, TypeScript, Advanced UI/UX, Performance

Phase 3: Backend Engineering (Months 13-18): Node.js, Databases, APIs, Authentication, Security

Phase 4: Advanced & Production (Months 19-24): System Design, Microservices, DevOps, Cloud, Scaling

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to all

Coding experience: Absolute beginner friendly - starts from zero

Equipment: Computer/laptop with internet connection

Time commitment: 15-20 hours per week consistently

English: Basic reading and comprehension

Motivation: Strong desire to become a developer

WHO THIS COURSE IS FOR

Students: College students looking for solid career foundation

Working professionals: Career switchers from any field

Entrepreneurs: Want to build their own products

Freelancers: Want to offer full-stack development services

Kids: Motivated teenagers (15+) can start this journey

Anyone: Anyone passionate about technology and coding

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CORE SKILLS (MONTHS 1-6, WEEKS 1-26)

Build rock-solid fundamentals. Learn how computers work, master programming basics, and create your first websites.

Months 1-2: Computer Science & Programming Fundamentals (Week 1-8)

Introduction to Computing & Setup

Topics:

- How computers work: CPU, Memory, Storage
- Binary, data representation, and encoding
- Operating systems basics (Windows, Mac, Linux)
- Internet fundamentals: How the web works
- Client-server architecture explained
- IP addresses, DNS, and HTTP basics
- Development environment setup
- VS Code installation and configuration
- Terminal/Command Line basics
- Git installation and first commands
- Creating your first GitHub account
- Introduction to HTML: What is markup?

Projects built:

- Set up complete development environment
- Create first GitHub repository
- Write 'About Me' HTML page

Practice: Daily: 30 min terminal practice, 1 hour HTML basics

HTML5 Mastery

Topics:

- HTML document structure and anatomy
- Semantic HTML5 elements (header, nav, main, section, article, aside, footer)
- Text elements: headings, paragraphs, lists, quotes
- Links and navigation (absolute vs relative URLs)
- Images: img tag, srcset, responsive images
- Tables: structure, accessibility, styling
- Forms: input types, labels, validation
- Form elements: textarea, select, radio, checkbox
- HTML5 APIs: audio, video, canvas basics
- Meta tags and SEO fundamentals
- Accessibility basics: ARIA labels, semantic structure
- HTML best practices and code organization

Projects built:

- Personal portfolio website (HTML only)
- Restaurant menu page with images and tables
- Multi-page contact form with validation

Practice: Build 5 different webpage layouts from scratch

CSS3 Fundamentals

Topics:

- CSS syntax: selectors, properties, values
- Three ways to add CSS: inline, internal, external
- CSS selectors: element, class, ID, descendant, child
- CSS specificity and cascade rules
- Color systems: hex, rgb, rgba, hsl, hsla
- Typography: font-family, size, weight, line-height, letter-spacing
- Text styling: alignment, decoration, transformation
- The Box Model: content, padding, border, margin
- Width, height, and box-sizing
- Display property: block, inline, inline-block, none
- Background properties: color, image, position, repeat, size
- CSS units: px, em, rem, %, vh, vw

Projects built:

- Style your portfolio with beautiful CSS
- Create a styled blog post layout
- Design a product card component

Practice: Recreate 10 designs from screenshots using HTML/CSS

Advanced CSS Layouts

Topics:

- CSS positioning: static, relative, absolute, fixed, sticky
- Z-index and stacking contexts
- Float and clear (legacy layout)
- Flexbox: flex container and flex items
- Flexbox properties: justify-content, align-items, flex-direction, flex-wrap
- Flex grow, shrink, and basis
- CSS Grid: rows, columns, and grid template
- Grid areas and named grid lines
- Grid gap, auto-fit, auto-fill, minmax
- Responsive design principles
- Media queries: breakpoints and mobile-first approach
- Viewport meta tag and mobile optimization

Projects built:

- Holy grail layout with Flexbox and Grid

- Responsive navigation menu
- Photo gallery with Grid
- Complete responsive landing page

Practice: Build 5 different layout patterns (sidebar, header-footer, card grid, etc.)

Months 3-4: JavaScript Fundamentals & Programming Logic (Week 9-17)

JavaScript Basics & Syntax

Topics:

- What is JavaScript? History and ecosystem
- Adding JavaScript to HTML: script tags, external files
- Console and debugging basics
- Variables: var, let, const (differences and scope)
- Data types: string, number, boolean, null, undefined, symbol
- Type conversion and coercion
- Operators: arithmetic, comparison, logical, assignment
- String methods: length, charAt, slice, indexOf, replace
- Template literals and string interpolation
- Comments and code documentation
- Naming conventions and code style
- First program: interactive calculator

Projects built:

- Temperature converter
- Simple tip calculator
- Age calculator with leap year handling

Practice: Solve 20 basic JavaScript problems on variables and operators

Control Flow & Logic

Topics:

- If-else statements and conditions
- Switch statements
- Ternary operator
- Truthy and falsy values
- Logical operators: &&, ||, !
- Loops: for, while, do-while
- Break and continue statements
- Nested loops and patterns
- Arrays: creation, indexing, length
- Array methods: push, pop, shift, unshift
- Iterating arrays with loops
- Multi-dimensional arrays

Projects built:

- Number guessing game
- FizzBuzz challenge
- Pattern printing (stars, pyramids)
- Shopping cart with array manipulation

Practice: Solve 30 logic problems on loops and arrays

Functions & Scope

Topics:

- Function declaration vs expression
- Parameters and arguments
- Return statements
- Default parameters
- Rest parameters and spread operator
- Arrow functions (ES6)
- Function scope and hoisting
- Lexical scope and closure
- IIFE (Immediately Invoked Function Expressions)
- Callback functions introduction
- Higher-order functions basics
- Pure functions and side effects

Projects built:

- Reusable utility function library
- Password strength validator
- Text analyzer (word count, reading time)
- Simple quiz application

Practice: Refactor previous projects using functions

Objects & Data Structures

Topics:

- Object literals and properties
- Accessing properties: dot vs bracket notation
- Adding and deleting properties
- Methods and 'this' keyword
- Object destructuring
- Object.keys, values, entries
- JSON: parsing and stringifying
- Nested objects and arrays
- Iterating over objects
- Object spread operator
- Sets and Maps introduction
- Comparing objects vs primitives

Projects built:

- Student management system
- Todo list with objects
- Contact book application
- Product inventory manager

Practice: Build 5 mini-projects using objects and arrays

Month 3-4 Review & Assessment

Topics:

- JavaScript fundamentals review
- Problem-solving strategies
- Code refactoring techniques
- Debugging common errors
- Best practices recap

Projects built:

- CAPSTONE: Complete interactive web application combining all Month 3-4 concepts
- Personal budget tracker
- Mini e-commerce product page with cart

Assessment: Take comprehensive test covering all JavaScript fundamentals

Months 5-6: DOM Manipulation & Interactive Websites (Week 18-26)

Document Object Model (DOM)

Topics:

- What is the DOM? Tree structure
- Window and document objects
- Selecting elements: getElementById, getElementsByClassName, querySelector
- querySelectorAll and NodeList vs HTMLCollection
- Traversing the DOM: parent, children, siblings
- Creating elements: createElement, createTextNode
- Modifying content: innerHTML, textContent, innerText
- Modifying attributes: getAttribute, setAttribute
- Modifying styles: style property, classList
- Adding and removing classes
- Inserting elements: appendChild, insertBefore, append, prepend
- Removing elements: removeChild, remove

Projects built:

- Dynamic HTML generator
- Color scheme switcher
- Content management system (CMS) simulator
- Live form preview

Practice: Manipulate DOM in 15 different ways

Events & User Interaction

Topics:

- Event listeners: addEventListener, removeEventListener
- Common events: click, dblclick, mouseenter, mouseleave
- Keyboard events: keydown, keyup, keypress
- Form events: submit, focus, blur, input, change
- Event object and properties
- Event bubbling and capturing
- Event delegation pattern
- preventDefault and stopPropagation
- Creating custom events
- DOMContentLoaded vs load event
- Mouse position and coordinates
- Touch events for mobile

Projects built:

- Interactive image gallery with lightbox
- Drag and drop interface
- Keyboard-controlled game
- Real-time form validation
- Modal window system

Practice: Create 10 interactive UI components

Advanced JavaScript Concepts

Topics:

- Array methods: map, filter, reduce, find, findIndex
- forEach, some, every
- Sort and reverse
- Method chaining
- String methods: split, join, trim, includes
- Regular expressions basics
- Date and time in JavaScript
- Math object and methods
- Error handling: try-catch-finally
- Throwing custom errors
- Strict mode
- localStorage and sessionStorage

Projects built:

- Data filtering and sorting dashboard
- Expense tracker with local storage
- Date and time utilities app

- Form validation with regex

Practice: Master array methods with 25 exercises

Asynchronous JavaScript

Topics:

- Synchronous vs asynchronous code
- setTimeout and setInterval
- Callbacks and callback hell
- Promises: creating and consuming
- Promise chaining
- Promise.all, Promise.race, Promise.allSettled
- Async/await syntax
- Error handling with async/await
- Fetch API basics
- Making HTTP requests
- Working with APIs and JSON
- CORS and API authentication basics

Projects built:

- Weather app using OpenWeather API
- GitHub profile viewer
- Movie search app (OMDB API)
- Currency converter with live rates
- Random quote generator

Practice: Integrate 5 different public APIs

Phase 1 Capstone Project

Topics:

- Project planning and wireframing
- Code organization and structure
- Git workflow and version control
- Responsive design implementation
- Deployment to GitHub Pages

Projects built:

- MAJOR CAPSTONE: Full-featured Todo Application
- Features: CRUD operations, filtering, local storage, drag-drop, animations
- Alternative: Personal blog with CMS
- Alternative: E-commerce product page with cart and checkout

Assessment: Phase 1 Final Exam - Build complete project from requirements

----- PHASE 2: FRONTEND MASTERY & MODERN FRAMEWORKS (MONTHS 7-12, WEEKS 27-52) -----

Master modern frontend development with React, TypeScript, state management, and advanced UI/UX patterns.

Months 7-8: Modern JavaScript & ES6+ (Week 27-35)

ES6+ Advanced Features

Topics:

- Destructuring: arrays and objects (deep)
- Spread and rest operators (advanced usage)
- Enhanced object literals
- Computed property names
- For...of and for...in loops
- Iterators and generators
- Symbols and their use cases
- WeakMap and WeakSet
- Proxy and Reflect
- Optional chaining (?.)
- Nullish coalescing operator (??)
- Private class fields

Projects built:

- Build utility library with modern JS
- Data transformation pipeline
- Custom iterator implementation

Practice: Refactor old code using modern ES6+ features

Object-Oriented Programming in JavaScript

Topics:

- Constructor functions
- Prototypes and prototype chain
- ES6 Classes syntax
- Class inheritance and extends
- Super keyword
- Static methods and properties
- Getters and setters
- Public, private, and protected fields
- Encapsulation principles
- Polymorphism in JavaScript
- Composition over inheritance
- Design patterns: Factory, Singleton, Module

Projects built:

- Bank account system with OOP

- Game character class hierarchy
- Library management system
- Vehicle rental system

Practice: Build 5 different systems using OOP principles

Functional Programming Concepts

Topics:

- First-class functions
- Higher-order functions
- Pure functions and immutability
- Function composition
- Currying and partial application
- Closures (advanced)
- Recursion and recursive patterns
- Memoization and optimization
- Functional array methods (advanced)
- Ramda or Lodash/FP
- Immutable data structures
- Functional vs OOP: when to use what

Projects built:

- Functional utility library
- Data pipeline with function composition
- Recursive algorithms (tree traversal, etc.)

Practice: Solve 30 problems using functional approach

Build Tools & Module Systems

Topics:

- ES6 modules: import/export
- Named vs default exports
- CommonJS vs ES modules
- NPM and package management
- Package.json explained
- Semantic versioning
- Installing and managing dependencies
- Webpack basics: entry, output, loaders
- Webpack dev server
- Babel for transpiling
- Bundling and code splitting
- Environment variables

Projects built:

- Set up custom build pipeline
- Multi-page application with Webpack
- Create and publish NPM package

Practice: Configure build tools for 3 different project types

TypeScript Fundamentals

Topics:

- Why TypeScript? Benefits and ecosystem
- TypeScript setup and tsconfig.json
- Basic types: string, number, boolean, any, unknown
- Arrays and tuples
- Enums and literal types
- Functions and optional parameters
- Type inference vs explicit typing
- Union and intersection types
- Type aliases and interfaces
- Compiling TypeScript
- TypeScript with vanilla JS projects

Projects built:

- Convert JavaScript project to TypeScript
- Todo app with TypeScript
- Type-safe utility functions

Practice: Write TypeScript for all future projects

Months 9-10: React.js Mastery (Week 36-44)

React Fundamentals

Topics:

- What is React? Virtual DOM explained
- Vite setup
- JSX syntax and rules
- Components: functional vs class
- Props and prop drilling
- Props validation with PropTypes
- Rendering lists with map
- Conditional rendering techniques
- React fragments
- Component composition
- Children prop pattern
- Event handling in React

Projects built:

- Component library showcase
- Profile card components

- Product listing page
- Interactive counter app

Practice: Build 20 reusable React components

React Hooks & State Management

Topics:

- useState hook: primitive and object state
- State updates and batching
- useEffect hook: lifecycle equivalent
- useEffect dependencies array
- Cleanup functions in useEffect
- useContext for prop drilling solution
- useReducer for complex state
- Custom hooks creation
- useRef for DOM access and mutable values
- useMemo for expensive calculations
- useCallback for function memoization
- Rules of hooks

Projects built:

- Todo app with hooks
- Theme switcher with Context
- Form builder with useReducer
- Custom hooks library

Practice: Create 10 custom hooks for common use cases

Advanced React Patterns

Topics:

- Controlled vs uncontrolled components
- Form handling and validation
- React Router: routes, navigation, params
- Nested routes and layouts
- Protected routes and authentication
- Lazy loading and code splitting
- React.memo for performance
- Error boundaries
- Portals for modals and tooltips
- Higher-Order Components (HOC)
- Render props pattern
- Compound components pattern

Projects built:

- Multi-page app with React Router
- Complex form with validation
- Dashboard with authentication
- Modal and notification system

Practice: Build 5 advanced UI patterns

State Management Libraries

Topics:

- Why external state management?
- Redux fundamentals: store, actions, reducers
- Redux Toolkit (modern Redux)
- createSlice and configureStore
- Redux DevTools
- Async logic with Redux Thunk
- Redux Saga basics
- Zustand (lightweight alternative)
- Recoil basics
- When to use Context vs Redux
- State management best practices

Projects built:

- E-commerce app with Redux
- Social media feed with state management
- Multi-user chat application
- Complex dashboard with multiple data sources

Practice: Refactor Context apps to use Redux

React with TypeScript

Topics:

- Setting up React with TypeScript
- Typing props and state
- Event handlers type
- Children and generic components
- Hooks with TypeScript
- Context API with TypeScript
- Redux with TypeScript
- Type-safe API calls
- Utility types for React

Projects built:

- Rewrite React projects in TypeScript
- Type-safe form builder
- Strongly-typed Redux application

Practice: All future React projects in TypeScript

Months 11-12: Advanced Frontend & UI/UX (Week 45-52)

Styling in React & CSS-in-JS

Topics:

- CSS Modules in React
- Styled Components library
- Emotion CSS-in-JS
- Tailwind CSS with React
- Tailwind configuration and customization
- Component libraries: Material-UI (MUI)
- Chakra UI basics
- Ant Design integration
- Theme management
- Responsive design in React
- CSS animations in React
- Framer Motion for animations

Projects built:

- Design system with Styled Components
- Dashboard with Tailwind CSS
- Animated landing page with Framer Motion
- Custom theme system

Practice: Build same UI with 3 different styling approaches

Performance Optimization

Topics:

- React DevTools Profiler
- Identifying performance bottlenecks
- Virtual scrolling for large lists
- Windowing with react-window
- Image optimization techniques
- Lazy loading images
- Code splitting strategies
- Bundle size optimization
- Lighthouse audits
- Core Web Vitals
- Performance monitoring tools
- Memoization best practices

Projects built:

- Optimize slow React application
- Large list with virtualization
- Image gallery with lazy loading
- Performance dashboard

Practice: Audit and optimize 5 projects

Testing in React

Topics:

- Introduction to testing: why test?
- Jest testing framework
- React Testing Library
- Writing unit tests
- Testing components and hooks
- Mocking API calls and modules
- Integration testing
- End-to-end testing with Cypress
- Test-driven development (TDD)
- Testing best practices
- Code coverage reports
- Continuous testing

Projects built:

- Add tests to existing projects
- TDD todo application
- E2E tests for complete user flow
- Testing utilities library

Practice: Write tests for all previous projects

Next.js & Server-Side Rendering

Topics:

- Next.js introduction and setup
- File-based routing
- Pages and layouts
- Server-side rendering (SSR)
- Static site generation (SSG)
- Incremental static regeneration (ISR)
- API routes in Next.js
- Data fetching strategies
- Image optimization with next/image
- Next.js deployment
- SEO optimization

Projects built:

- Blog with Next.js SSG
- E-commerce with SSR
- Portfolio with Next.js
- Full-stack Next.js application

Practice: Migrate React app to Next.js

Phase 2 Capstone Project

Topics:

- Full-stack frontend application
- Professional UI/UX design
- State management implementation
- Performance optimization
- Testing coverage
- Production deployment

Projects built:

- MAJOR CAPSTONE: Complete Social Media Platform Frontend
- Features: Auth, posts, comments, likes, real-time updates, profiles, search
- Alternative: Project Management Tool (Trello/Jira clone)
- Alternative: E-commerce Platform with Cart, Checkout, Orders

Assessment: Phase 2 Final Exam - Build production-ready frontend application

PHASE 3: BACKEND ENGINEERING & DATABASES (MONTHS 13-18, WEEKS 53-78)

Master backend development, databases, APIs, authentication, security, and server-side architecture.

Months 13-14: Node.js & Express.js (Week 53-61)

Node.js Fundamentals

Topics:

- What is Node.js? V8 engine explained
- Node.js architecture and event loop
- Blocking vs non-blocking code
- Global objects: process, __dirname, __filename
- Modules system: require and module.exports
- Core modules: fs, path, os, http
- File system operations: read, write, append, delete
- Asynchronous file operations
- Buffer and streams
- Event emitters
- Creating HTTP server with Node.js
- Handling requests and responses

Projects built:

- File-based note-taking CLI
- Simple HTTP server
- File upload utility
- Log management system

Practice: Build 10 CLI tools with Node.js

Express.js Framework

Topics:

- Express.js introduction and setup
- Routing: GET, POST, PUT, DELETE
- Route parameters and query strings
- Middleware concept and usage
- Built-in middleware: express.json, express.static
- Custom middleware creation
- Error handling middleware
- Request and response objects
- Sending responses: JSON, HTML, files
- Template engines: EJS, Pug
- Serving static files
- Express Router for modular routes

Projects built:

- RESTful API for blog
- User management system
- File upload service
- Server-side rendered website

Practice: Build 5 different APIs with Express

RESTful API Design

Topics:

- REST principles and constraints
- HTTP methods semantics
- Status codes: 2xx, 3xx, 4xx, 5xx
- API versioning strategies
- Resource naming conventions
- Query parameters for filtering, sorting, pagination
- Request/response body structure
- Error response format
- API documentation with Swagger/OpenAPI
- Postman for API testing
- HATEOAS principle
- API best practices

Projects built:

- Complete REST API for task management

- E-commerce product API
- Social media API endpoints
- API documentation with Swagger

Practice: Design and document 5 different API systems

Advanced Node.js & Async Patterns

Topics:

- Deep dive into event loop phases
- Promises in Node.js
- Async/await patterns
- Error handling in async code
- Worker threads for CPU-intensive tasks
- Child processes and clustering
- Stream processing: readable, writable, transform
- Piping streams
- Memory management in Node.js
- Performance monitoring
- Debugging Node.js applications
- PM2 process manager

Projects built:

- Large file processor with streams
- Multi-process task scheduler
- Real-time log processor
- Performance monitoring dashboard

Practice: Optimize 5 Node.js applications

Express.js Advanced Concepts

Topics:

- Application-level vs router-level middleware
- Third-party middleware: helmet, cors, morgan
- Rate limiting and throttling
- Request validation with Joi/Yup
- File upload with Multer
- Cookie and session management
- CSRF protection
- Compression middleware
- Graceful shutdown
- Environment-based configuration
- Logging with Winston
- Security best practices

Projects built:

- Secure API with all middleware
- File upload service with validation
- Session-based authentication system
- Production-ready Express template

Practice: Secure and optimize Express applications

Months 15-16: Databases & Data Modeling (Week 62-70)

MongoDB & NoSQL Databases

Topics:

- SQL vs NoSQL databases
- MongoDB introduction and architecture
- MongoDB installation and setup
- MongoDB shell commands
- Collections and documents
- CRUD operations in MongoDB
- Query operators: comparison, logical, element
- Update operators: \$set, \$inc, \$push, \$pull
- Aggregation pipeline basics
- Indexing for performance
- Data modeling in MongoDB
- Embedded documents vs references

Projects built:

- User database with MongoDB
- Product catalog system
- Blog posts with comments
- Analytics data aggregation

Practice: Design 10 different database schemas

Mongoose ODM

Topics:

- Mongoose introduction and setup
- Schema definition and types
- Schema validation rules
- Custom validators
- Default values and required fields
- Virtuals and getters/setters
- Instance methods and static methods
- Query helpers
- Middleware (pre/post hooks)
- Population for references
- Discriminators for inheritance

- Schema design patterns

Projects built:

- User management with Mongoose
- E-commerce data models
- Social network schema
- Complex relationships modeling

Practice: Build 5 complex data models with Mongoose

SQL & Relational Databases

Topics:

- Relational database concepts
- PostgreSQL installation and setup
- SQL basics: SELECT, INSERT, UPDATE, DELETE
- WHERE clause and filtering
- ORDER BY, LIMIT, OFFSET
- Aggregate functions: COUNT, SUM, AVG, MIN, MAX
- GROUP BY and HAVING
- JOINS: INNER, LEFT, RIGHT, FULL
- Subqueries and nested queries
- Data types in PostgreSQL
- Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK
- Normalization (1NF, 2NF, 3NF)

Projects built:

- Library management system
- Inventory tracking database
- School database with relationships
- Complex reporting queries

Practice: Write 50 SQL queries of increasing complexity

Advanced SQL & Database Design

Topics:

- Indexes: B-tree, Hash, GiST
- Query optimization techniques
- EXPLAIN and query plans
- Views and materialized views
- Stored procedures and functions
- Triggers
- Transactions and ACID properties
- Isolation levels
- Database backup and restore
- Replication basics
- Partitioning large tables
- Database migrations

Projects built:

- Optimized database for analytics
- Complex multi-table system
- Database migration scripts
- Performance benchmarking

Practice: Optimize 5 slow databases

ORMs & Database Integration

Topics:

- Sequelize ORM for Node.js
- Model definition and associations
- Migrations with Sequelize
- Seeders for test data
- Prisma ORM introduction
- Prisma schema and migrations
- Type-safe database queries
- Knex.js query builder
- Connection pooling
- Database transactions in Node.js
- Choosing the right database
- Polyglot persistence

Projects built:

- API with Sequelize
- Type-safe API with Prisma
- Database comparison project
- Multi-database application

Practice: Integrate databases with Express apps

Months 17-18: Authentication, Security & Advanced Backend (Week 71-78)

Authentication & Authorization

Topics:

- Authentication vs authorization
- Password hashing with bcrypt
- Salt and rainbow table attacks
- Session-based authentication
- Token-based authentication
- JWT (JSON Web Tokens) structure
- Creating and verifying JWTs
- Access tokens and refresh tokens

- OAuth 2.0 protocol
- Social login: Google, Facebook, GitHub
- Passport.js middleware
- Role-based access control (RBAC)

Projects built:

- Complete auth system with JWT
- Social login integration
- Multi-role authorization system
- Password reset functionality

Practice: Implement authentication in 5 different ways

API Security & Best Practices

Topics:

- OWASP Top 10 vulnerabilities
- SQL injection prevention
- NoSQL injection attacks
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)
- Security headers: helmet.js
- CORS configuration and security
- Rate limiting strategies
- API keys and secrets management
- Environment variables and .env
- Input validation and sanitization
- HTTPS and SSL/TLS

Projects built:

- Secure API with all protections
- Security audit tool
- Rate limiting service
- Input validation library

Practice: Security audit 5 existing APIs

Real-time Communication

Topics:

- WebSockets protocol
- Socket.io library
- Real-time events: emit, on, broadcast
- Rooms and namespaces
- Private messaging
- Real-time notifications
- Server-Sent Events (SSE)
- WebRTC basics
- Scaling WebSocket servers
- Redis adapter for Socket.io
- Real-time database sync
- Building chat applications

Projects built:

- Real-time chat application
- Live notification system
- Collaborative editor
- Real-time dashboard
- Multiplayer game backend

Practice: Add real-time features to previous projects

Advanced Backend Patterns

Topics:

- MVC architecture pattern
- Repository pattern
- Service layer pattern
- Dependency injection
- SOLID principles
- Caching strategies: Redis
- Cache invalidation patterns
- Background jobs: Bull queue
- Task scheduling with cron
- Email sending: Nodemailer
- SMS integration
- Payment gateway integration: Stripe

Projects built:

- Layered architecture API
- Background job processor
- Email notification service
- Payment processing system

Practice: Refactor projects using design patterns

Phase 3 Capstone Project

Topics:

- Complete backend system design
- Database architecture
- Authentication and authorization
- Real-time features
- Security implementation
- API documentation

Projects built:

- MAJOR CAPSTONE: Complete Social Media Backend API
- Features: Auth, users, posts, comments, likes, followers, real-time, notifications
- Alternative: E-commerce Backend with Orders, Payments, Inventory
- Alternative: Project Management API with Teams, Tasks, Real-time Updates

Assessment: Phase 3 Final Exam - Build production-ready backend system

PHASE 4: ADVANCED TOPICS & PRODUCTION SYSTEMS (MONTHS 19-24, WEEKS 79-104)

Master system design, microservices, DevOps, cloud infrastructure, and production-grade application development.

Months 19-20: System Design & Architecture (Week 79-87)

System Design Fundamentals

Topics:

- What is system design?
- Scalability: vertical vs horizontal
- Load balancing concepts
- Load balancer types: Layer 4 vs Layer 7
- Caching strategies and levels
- Content Delivery Networks (CDN)
- Database sharding and partitioning
- Database replication: master-slave
- CAP theorem explained
- Consistency models
- Latency vs throughput
- Back-of-envelope calculations

Projects built:

- Design URL shortener
- Design pastebin service
- Design simple caching system
- Load balancer simulation

Practice: Solve 10 system design problems

Microservices Architecture

Topics:

- Monolith vs microservices
- When to use microservices
- Service decomposition strategies
- Inter-service communication: REST, gRPC
- Message queues: RabbitMQ, Kafka
- Event-driven architecture
- Saga pattern for distributed transactions
- API Gateway pattern
- Service discovery
- Circuit breaker pattern
- Distributed tracing
- Microservices challenges

Projects built:

- Decompose monolith into microservices
- Message queue implementation
- Event-driven system
- API Gateway service

Practice: Design 5 microservice architectures

GraphQL

Topics:

- GraphQL vs REST
- GraphQL schema and types
- Queries, mutations, subscriptions
- Resolvers and data sources
- Apollo Server setup
- GraphQL with databases
- Authentication in GraphQL
- DataLoader for batching
- GraphQL best practices
- Error handling in GraphQL
- GraphQL subscriptions for real-time
- Apollo Client integration

Projects built:

- Complete GraphQL API
- GraphQL social media API
- Real-time GraphQL app
- REST to GraphQL migration

Practice: Build 5 GraphQL APIs

Advanced System Design Patterns

Topics:

- Database indexing strategies
- Full-text search: Elasticsearch
- Rate limiting algorithms

- Distributed caching with Redis
- Session management at scale
- File storage systems
- Video streaming architecture
- Real-time analytics systems
- Notification systems design
- Design patterns for scalability
- Monitoring and observability
- Disaster recovery planning

Projects built:

- Design Twitter/X
- Design Instagram
- Design Netflix streaming
- Design Uber ride-sharing

Practice: Mock system design interviews

Performance Optimization

Topics:

- Application profiling
- Database query optimization
- N+1 query problem
- Connection pooling
- Caching at multiple layers
- Compression techniques
- Lazy loading strategies
- CDN optimization
- API response optimization
- Memory leak detection
- CPU profiling
- Load testing with Artillery/k6

Projects built:

- Optimize slow application
- Database performance tuning
- Load testing suite
- Performance monitoring system

Practice: Optimize 5 production systems

Months 21-22: DevOps & Cloud Infrastructure (Week 88-96)

Docker & Containerization

Topics:

- What is Docker? Containers vs VMs
- Docker installation and setup
- Docker images and containers
- Dockerfile creation
- Docker commands: run, build, push, pull
- Docker volumes for persistence
- Docker networks
- Multi-container apps with Docker Compose
- Docker Compose YAML
- Environment variables in Docker
- Docker best practices
- Container security

Projects built:

- Dockerize Node.js application
- Multi-container app with database
- Docker Compose for full stack
- Custom Docker images

Practice: Containerize all previous projects

Kubernetes & Orchestration

Topics:

- What is Kubernetes?
- Kubernetes architecture: master and nodes
- Pods, ReplicaSets, Deployments
- Services: ClusterIP, NodePort, LoadBalancer
- ConfigMaps and Secrets
- Persistent Volumes
- Namespaces
- Ingress controllers
- Horizontal Pod Autoscaling
- Rolling updates and rollbacks
- Helm package manager
- Kubernetes dashboard

Projects built:

- Deploy app to Kubernetes
- Multi-service Kubernetes setup
- Auto-scaling configuration
- Helm chart creation

Practice: Deploy 5 applications to Kubernetes

AWS Cloud Services

Topics:

- AWS account setup and IAM
- EC2: virtual servers
- S3: object storage
- RDS: managed databases
- Lambda: serverless functions
- API Gateway
- CloudFront CDN
- Route 53 DNS
- VPC and networking
- Security groups and NACLs
- CloudWatch monitoring
- Cost optimization

Projects built:

- Deploy app on EC2
- Serverless API with Lambda
- Static site on S3 + CloudFront
- Full infrastructure on AWS

Practice: Build 5 different AWS architectures

CI/CD Pipelines

Topics:

- Continuous Integration explained
- Continuous Deployment explained
- Git workflow strategies
- GitHub Actions basics
- Writing workflow YAML files
- Automated testing in pipelines
- Build and deployment automation
- GitLab CI/CD
- Jenkins basics
- Environment management
- Blue-green deployments
- Canary releases

Projects built:

- GitHub Actions pipeline
- Automated testing workflow
- Multi-environment deployment
- Complete CI/CD for full stack app

Practice: Set up CI/CD for all projects

Infrastructure as Code

Topics:

- Infrastructure as Code concept
- Terraform basics
- Terraform providers and resources
- Terraform state management
- CloudFormation for AWS
- Ansible for configuration
- Version control for infrastructure
- Environment provisioning
- Disaster recovery automation
- Cost tracking and management

Projects built:

- Terraform AWS infrastructure
- Multi-environment setup
- Automated provisioning system
- Infrastructure versioning

Practice: Manage infrastructure as code

Month 23: Advanced Topics & Specializations (Week 97-100)

Advanced Database Topics

Topics:

- Time-series databases
- Graph databases: Neo4j
- Vector databases
- Database migrations at scale
- Multi-tenancy patterns
- Data warehousing basics
- ETL processes
- Database security hardening
- Backup strategies
- Disaster recovery testing

Projects built:

- Time-series analytics system
- Graph-based recommendation engine
- Multi-tenant SaaS database
- Data pipeline implementation

Practice: Explore specialized database systems

Advanced Security Topics

Topics:

- Penetration testing basics

- Security auditing tools
- Zero-trust architecture
- API security testing
- Secrets management: Vault
- Compliance: GDPR, HIPAA basics
- Security monitoring
- Incident response planning
- Bug bounty programs
- Secure coding practices

Projects built:

- Security audit tool
- Secrets management system
- Security monitoring dashboard
- Penetration testing report

Practice: Security audit all projects

Machine Learning for Web Developers

Topics:

- ML basics for developers
- TensorFlow.js
- Integrating ML models in apps
- Image recognition APIs
- Natural Language Processing
- Recommendation systems
- ML model deployment
- AI-powered features
- Ethical AI considerations

Projects built:

- Image classifier web app
- Sentiment analysis API
- Recommendation engine
- AI chatbot integration

Practice: Add AI features to projects

AI Integration Basics

Topics:

- Where AI belongs in a full-stack app
- Server-side model calls: never from the browser
- Streaming responses without freezing the UI
- Structured output and validation
- Retrieval over your own database
- Function calling to trigger app actions
- Cost, caching and rate limits
- Guardrails and untrusted input
- Telling users honestly what the AI did

Projects built:

- AI-powered search
- Retrieval Q&A feature
- Agentic action endpoint
- Guardrail + eval harness

Practice: Add an AI feature end to end

Month 24: Final Projects & Career Preparation (Week 101-104)

Final Capstone Project - Part 1

Topics:

- Project ideation and planning
- System architecture design
- Technology stack selection
- Database design
- API design and documentation
- Frontend architecture
- Development workflow setup
- Team collaboration (if group project)

Projects built:

- FINAL CAPSTONE: Production-Grade Full Stack Application
- Requirements: Microservices/monolith, authentication, real-time features, payment, notifications, admin panel, analytics, DevOps pipeline, cloud deployment
- Options: SaaS platform, Social network, E-commerce marketplace, Project management tool, Learning management system

Final Capstone Project - Part 2 & Deployment

Topics:

- Code review and refactoring
- Testing: unit, integration, E2E
- Performance optimization
- Security hardening
- Documentation: technical and user
- CI/CD pipeline setup
- Production deployment
- Monitoring and logging
- Backup and disaster recovery
- Project presentation preparation

Deliverables:

- Complete source code on GitHub
- Live production URL
- Complete documentation
- Video demo
- Architecture diagram
- API documentation
- Test coverage report
- Performance audit report

Career Preparation & Job Readiness

Topics:

- Portfolio website optimization
- Resume building for developers
- LinkedIn profile optimization
- GitHub profile enhancement
- Contributing to open source
- Interview preparation: technical
- System design interview prep
- Behavioral interview questions
- Salary negotiation tips
- Freelancing vs full-time
- Building personal brand
- Continuous learning strategies

Deliverables:

- Professional portfolio website
- Optimized resume
- LinkedIn profile
- GitHub showcase
- Open source contributions
- Mock interview practice
- Personal development plan

Assessment: FINAL COMPREHENSIVE EXAM - Cover all 24 months of content

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Todo App, Weather App, Portfolio Site, Quiz App, Blog (HTML/CSS/JS)
- Phase 2: Social Media Dashboard, E-commerce Frontend, Chat Application, Admin Panel
- Phase 3: RESTful API, Authentication System, Real-time Chat Backend, Payment Integration
- Phase 4: Microservices Architecture, Kubernetes Deployment, Full DevOps Pipeline
- Final: Production SaaS Application with everything

Total projects: 50+ projects ranging from simple to complex

SKILLS THE STUDENT WILL HAVE AT THE END

- Frontend: HTML5, CSS3, JavaScript (ES6+), TypeScript, React.js, Next.js, Redux
- Styling: CSS-in-JS, Tailwind CSS, Styled Components, Responsive Design, Animations
- Backend: Node.js, Express.js, RESTful APIs, GraphQL, WebSockets, Microservices
- Databases: MongoDB, PostgreSQL, Redis, Elasticsearch, Database Design
- Authentication: JWT, OAuth, Session Management, Security Best Practices
- DevOps: Docker, Kubernetes, CI/CD, GitHub Actions, AWS, Infrastructure as Code
- Testing: Jest, React Testing Library, Cypress, TDD, Integration Testing
- Tools: Git, VS Code, Postman, Chrome DevTools, Terminal, Linux
- Concepts: OOP, Functional Programming, Design Patterns, System Design, Algorithms

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-8 hours
Hands on coding: 8-10 hours
Projects: 3-5 hours
Practice problems: 2-3 hours
Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly doubt clearing sessions
Mentorship: 1-on-1 mentor guidance
Community: Active Discord/Slack community
Code review: Expert code reviews for projects
Career support: Resume review, interview prep, job referrals
Lifetime access: All content, updates, and new modules

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion
Final certificate: Professional Full Stack Developer Certification
LinkedIn badge: Add to LinkedIn profile
Industry recognized: Backed by industry professionals
Portfolio projects: 20+ projects to showcase

WHERE THIS CAN LEAD

- Full Stack Developer (Entry to Senior level)
- Frontend Developer / Backend Developer (Specialized)
- DevOps Engineer
- Cloud Solutions Architect
- Technical Lead / Engineering Manager
- Freelance Full Stack Developer
- Startup Founder / CTO
- Open Source Contributor
- Technical Content Creator / Educator
- Software Consultant

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Growing roles: Rs 3-5 LPA (Junior Developer)

Experienced roles: Rs 6-10 LPA (Full Stack Developer); Rs 10-18 LPA (Senior Developer); Rs 15-30+ LPA (Senior/Lead roles)

Freelance: Rs 500-2,000/hour

International: \$50k-120k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is full stack web development and why should I learn it?

A: Full stack web development means building complete web applications from scratch - both the visible parts users interact with (frontend) and the behind-the-scenes systems that power them (backend).

Q: Do I need prior coding experience for this full stack development course?

A: No prior experience is required! This 24-month masterclass starts from absolute zero - teaching you how computers work, basic HTML, and progressively building to advanced concepts like microservices and system design.

Q: What technologies will I learn in this Full Stack Web Development program?

A: You'll master the complete modern web stack: HTML5, CSS3, JavaScript ES6+, TypeScript, React, Redux, Next.js, Node.js, Express.js, MongoDB, PostgreSQL, Docker, Kubernetes, AWS, CI/CD pipelines, and system design. By completion, you'll be proficient in 25+ technologies used by top tech companies.

Q: How long does it take to become a full stack developer?

A: This comprehensive program spans 24 months with 15-20 hours/week commitment. However, you can start applying for junior developer roles as early as 6 months in. By month 12, you'll be ready for mid-level positions, and by month 24, you'll have senior developer-level knowledge and a portfolio of 20+ production projects.

Q: What's the difference between frontend and backend development?

A: Frontend development involves building what users see and interact with - the visual interface, buttons, forms, and animations using HTML, CSS, JavaScript, and React. Backend development handles the server-side logic - databases, APIs, authentication, and business logic using Node.js and Express. Full stack developers master both, making them capable of building complete applications.

Q: What job opportunities are available after completing this full stack course?

A: Graduates pursue roles like Full Stack Developer, Frontend Developer, Backend Developer, React Developer, Node.js Developer, DevOps Engineer, Technical Lead, and Solutions Architect. Many also freelance or start their own tech companies. With the current tech boom, full stack developers are among the most in-demand professionals globally.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/full-stack-web-development-masterclass-college>

=====

COURSE 57 OF 118: GAME DEVELOPMENT COURSE: UNITY, UNREAL & C++, ZERO TO PRO

=====

Course link: <https://learn.modernagecoders.com/courses/game-development-complete-masterclass-college>

Subject area: Game Development & Interactive Media

Designed for: College students

Who can join: Complete Beginner to Professional Game Developer

Course length: 12 months (52 weeks)

Time needed: 20-25 hours/week recommended

Certificate: Certified Game Developer upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Game development course: master Unity, Unreal Engine, C++ and C#, 2D and 3D games, graphics, AI and multiplayer, then ship games to multiple platforms.

In one line: From Game Idea to Published Title on Multiple Platforms

ABOUT THE PROGRAMME

This is not just a course, it's your journey from gaming enthusiast to professional game developer. The video game industry is larger than movies and music combined, and skilled game developers are in massive demand. This 12-month masterclass transforms you into a complete game developer, capable of designing, programming, and shipping games across PC, mobile, console, and VR platforms. You'll master industry-standard engines (Unity, Unreal), programming languages (C++, C#), game design principles, and build a portfolio of playable games that could launch your career or indie studio.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no coding or art experience needed
- Complete 12-month structured curriculum covering all aspects of game dev
- Master both Unity and Unreal Engine comprehensively
- Build 20+ complete games across different genres
- Covers 2D, 3D, VR/AR, and multiplayer development
- Game design, art, audio, and programming all included
- Publishing and monetization strategies taught
- Industry mentorship from published game developers
- Steam, mobile stores, and console deployment covered
- Build portfolio worthy of AAA studios or successful indie launch

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Game Design, 2D Development, Unity Basics

Phase 2: Intermediate (Months 4-6): 3D Games, C# Mastery, Advanced Unity

Phase 3: Advanced (Months 7-9): Unreal Engine, C++, Graphics Programming

Phase 4: Professional (Months 10-12): Multiplayer, Publishing, Specialization

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal education required

Coding experience: None - we teach from scratch

Equipment: Gaming PC/laptop (8GB RAM minimum, dedicated GPU recommended)

Time commitment: 20-25 hours per week

English: Good comprehension for documentation

Motivation: Passion for games and creativity

WHO THIS COURSE IS FOR

Students: Aspiring game developers wanting to enter the industry

Working professionals: Career switchers passionate about game development

Entrepreneurs: Want to start indie game studio

Freelancers: Offer game development services

Kids: Mature teens (14+) with gaming passion

Anyone: Anyone who dreams of making their own games

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: GAME FUNDAMENTALS & 2D DEVELOPMENT (MONTHS 1-3, WEEKS 1-13)

Build solid foundations in game design theory, 2D game development, and Unity basics while creating your first playable games.

Months 1-2: Game Design & 2D Fundamentals (Week 1-8)

Introduction to Game Development

Topics:

- History of video games and industry overview
- Game development roles: programmer, artist, designer, producer
- Game genres and their characteristics
- Game development pipeline: concept to release

- Introduction to game engines vs frameworks
- Setting up development environment
- Unity installation and interface tour
- Version control with Git for games
- Game development terminology
- Analyzing successful games
- Game documentation: GDD (Game Design Document)
- Prototyping and iteration methodology

Projects built:

- Write your first Game Design Document
- Analyze 5 favorite games critically
- Create paper prototype of simple game
- Set up Unity and development environment

Practice: Play and analyze 20 games across different genres

Game Design Fundamentals

Topics:

- Core game mechanics vs meta mechanics
- Game loops: core, meta, and social
- Player motivation and psychology
- Flow theory and difficulty curves
- Reward systems and progression
- Game balance fundamentals
- Level design principles
- Tutorial design and onboarding
- Narrative in games
- Player types: Bartle's taxonomy
- Monetization models: premium, freemium, ads
- Playtesting and feedback incorporation

Projects built:

- Design a complete game concept with mechanics
- Create difficulty curve for a game
- Design reward system for mobile game
- Build paper prototype and playtest it

Practice: Design 10 different game mechanics on paper

2D Game Development with Unity

Topics:

- Unity 2D workspace and tools
- Sprites and sprite sheets
- 2D physics and colliders
- Rigidbody2D and physics materials
- Animation basics: sprite animation
- Animator controller and state machines
- Tilemaps and tilemap colliders
- 2D lighting with URP
- Particle systems for 2D
- Camera setup for 2D games
- Sorting layers and order
- 2D audio and sound effects

Projects built:

- Create animated sprite character
- Build tilemap-based level
- Implement 2D platformer mechanics
- Add particle effects and polish

Practice: Create 10 different 2D game prototypes

Programming Fundamentals with C#

Topics:

- C# basics: variables, data types, operators
- Control flow: if-else, switch, loops
- Functions and methods
- Object-oriented programming in C#
- Classes and objects for games
- Inheritance and polymorphism
- Unity-specific C# features
- MonoBehaviour lifecycle
- Coroutines and timing
- Events and delegates
- Debugging in Unity
- Performance considerations

Projects built:

- Player movement controller
- Enemy AI basic behavior
- Inventory system
- Score and UI management system

Practice: Write 50 Unity scripts for different purposes

Month 3: Complete 2D Games (Week 9-13)

Building Complete 2D Games

Topics:

- Game architecture patterns

- Scene management and loading
- Save/load systems
- UI design with Unity UI
- Menu systems and navigation
- Input handling: keyboard, mouse, touch
- Mobile input and virtual joysticks
- Object pooling for performance
- 2D pathfinding basics
- Power-ups and collectibles
- Health and damage systems
- Game state management

Projects built:

- Complete 2D platformer game
- Top-down shooter with enemies
- Puzzle game with 10 levels
- Mobile endless runner

Practice: Build 5 different 2D game genres

2D Art and Animation

Topics:

- Pixel art fundamentals
- Color theory for games
- Creating game sprites
- Animation principles: squash, stretch, anticipation
- Sprite sheet creation
- 2D skeletal animation with Unity
- Visual effects for 2D games
- UI/UX design for games
- Asset optimization
- Free and paid asset resources
- Art style consistency
- Creating game atmosphere

Projects built:

- Create full sprite set for character
- Design and animate enemy types
- Create tileset for platformer
- Design complete UI theme

Practice: Create art assets for 5 games

Audio and Polish

Topics:

- Game audio fundamentals
- Sound effects creation and editing
- Music theory for games
- Unity audio system
- 3D spatial audio
- Audio mixing and mastering
- Creating atmosphere with audio
- Audio feedback for player actions
- Performance optimization
- Game juice and polish techniques
- Screen shake and effects
- Testing and debugging

Projects built:

- Add complete audio to previous games
- Create dynamic music system
- Polish one game to release quality
- PHASE 1 CAPSTONE: Complete 2D Game with Original Art and Audio

Assessment: Phase 1 exam on game design and 2D development

----- PHASE 2: 3D DEVELOPMENT & ADVANCED UNITY (MONTHS 4-6, WEEKS 14-26) -----

Master 3D game development, advanced C# programming, and professional Unity techniques.

Months 4-5: 3D Game Development (Week 14-22)

3D Fundamentals in Unity

Topics:

- 3D coordinate systems and transforms
- 3D modeling basics and terminology
- Importing and configuring 3D models
- Materials and textures
- Shader basics and Shader Graph
- Lighting: directional, point, spot, area
- Lightmapping and global illumination
- Shadows and shadow settings
- 3D physics: Rigidbody and colliders
- Physics materials and joints
- Terrain creation and editing
- ProBuilder for level design

Projects built:

- 3D character controller
- Physics-based puzzle
- Terrain with environment
- Indoor level with lighting

Practice: Create 10 different 3D scenes

3D Animation and Characters

Topics:

- 3D animation principles
- Humanoid vs generic rigs
- Animation import and retargeting
- Mecanim animation system
- Blend trees and layers
- Animation events and curves
- Root motion
- Inverse Kinematics (IK)
- Ragdoll physics
- Facial animation basics
- Motion capture data usage
- Cinematics with Timeline

Projects built:

- Third-person character controller
- Combat system with animations
- Cutscene using Timeline
- Character customization system

Practice: Implement 15 different animation systems

Advanced C# for Games

Topics:

- Design patterns for games: Singleton, Observer, State
- Generic programming in Unity
- LINQ for game data
- Async/await in Unity
- Unity Job System basics
- Burst Compiler
- ECS (Entity Component System) introduction
- Custom editors and tools
- PropertyDrawers and Attributes
- ScriptableObject for data
- Addressables for asset management
- Performance profiling and optimization

Projects built:

- Modular weapon system
- Quest system with ScriptableObjects
- Custom Unity tools
- Performance-optimized game

Practice: Implement 20 design patterns in games

AI for Games

Topics:

- Game AI fundamentals
- Finite State Machines (FSM)
- Behavior Trees introduction
- Navigation and pathfinding
- NavMesh and NavMesh agents
- A* pathfinding implementation
- Steering behaviors: seek, flee, wander
- Flocking and swarm AI
- Decision trees
- Goal-Oriented Action Planning (GOAP)
- Machine learning in games basics
- Enemy AI patterns

Projects built:

- Stealth game with AI guards
- RTS unit movement system
- Boss AI with multiple phases
- Companion AI system

Practice: Create 10 different AI behaviors

3D Game Projects

Topics:

- First-person shooter mechanics
- Third-person action games
- Racing game physics
- Survival game systems
- Inventory and crafting
- Dialogue systems
- Quest and mission systems
- Procedural generation basics
- Level streaming
- Optimization techniques
- Platform-specific considerations
- Beta testing process

Projects built:

- FPS prototype with AI enemies
- Racing game with 3 tracks
- Survival game vertical slice
- Action RPG with inventory

Practice: Build prototypes of 5 different 3D genres

Month 6: VR/AR and Mobile Development (Week 23-26)

Virtual Reality Development

Topics:

- VR fundamentals and hardware
- Unity XR Toolkit setup
- VR interaction systems
- Locomotion in VR: teleportation, smooth
- Hand tracking and gestures
- VR UI design principles
- Comfort and motion sickness
- VR optimization techniques
- Spatial audio for VR
- Testing VR applications
- Oculus/Meta Quest development
- SteamVR integration

Projects built:

- VR interaction playground
- VR escape room
- VR training simulator
- VR multiplayer experience

Practice: Create 5 VR prototypes

Augmented Reality Development

Topics:

- AR fundamentals and applications
- AR Foundation in Unity
- Plane detection and tracking
- Image and object tracking
- Face tracking and filters
- World mapping and persistence
- AR UI/UX best practices
- Lighting estimation
- AR performance optimization
- ARCore and ARKit features
- WebAR possibilities
- Location-based AR

Projects built:

- AR furniture placement app
- AR game with real-world interaction
- AR educational experience
- AR social feature

Practice: Build 5 AR applications

Mobile Game Development

Topics:

- Mobile game market analysis
- Touch input and gestures
- Mobile UI/UX principles
- Screen resolution handling
- Mobile performance optimization
- Battery and thermal management
- Mobile-specific shaders
- Asset bundling for mobile
- In-app purchases implementation
- Ads integration: Unity Ads, AdMob
- Analytics integration
- Push notifications

Projects built:

- Hypercasual mobile game
- Mobile puzzle game with IAP
- Mobile multiplayer game
- Port PC game to mobile

Practice: Optimize 5 games for mobile

Cross-Platform Development

Topics:

- Platform-specific code management
- Input system abstraction
- Save data across platforms
- Cloud saves implementation
- Platform-specific features
- Console development basics
- Nintendo Switch considerations
- PlayStation requirements
- Xbox Live integration

- Steam integration
- Build automation
- Testing across platforms

Projects built:

- Cross-platform game with cloud saves
- Platform-specific features implementation
- Console-ready game prototype
- Multi-platform build pipeline

Practice: Deploy game to 3 different platforms

Phase 2 Capstone Project

Topics:

- Scope and project planning
- Vertical slice development
- Iterative design process
- Playtesting and feedback
- Polish and optimization
- Marketing materials creation

Projects built:

- PHASE 2 CAPSTONE: Complete 3D Game
- Develop full 3D game with 30 minutes of gameplay
- Include: menus, save system, 5 levels, original assets
- Target platform: PC and one other platform

Assessment: Phase 2 exam on 3D development and advanced Unity

----- PHASE 3: UNREAL ENGINE & ADVANCED PROGRAMMING (MONTHS 7-9, WEEKS 27-39) -----

Master Unreal Engine, C++ game programming, graphics programming, and advanced game systems.

Months 7-8: Unreal Engine Mastery (Week 27-35)

Unreal Engine Fundamentals

Topics:

- Unreal Engine philosophy vs Unity
- UE5 interface and workflow
- Blueprints visual scripting
- Actor and component system
- Level design in Unreal
- Materials and material instances
- Lighting with Lumen
- Nanite virtualized geometry
- World Partition system
- Niagara particle system
- Physics with Chaos
- World building tools

Projects built:

- First Unreal project setup
- Blueprint character controller
- Environment with Nanite and Lumen
- Particle effects showcase

Practice: Create 10 Unreal Blueprint systems

C++ Game Programming

Topics:

- C++ in Unreal Engine
- Unreal coding standards
- UCLASS, UPROPERTY, UFUNCTION
- Actor lifecycle in C++
- Components in C++
- Gameplay framework classes
- Input handling in C++
- Collision and physics in code
- Unreal's reflection system
- Debugging C++ in Unreal
- Blueprint and C++ interaction
- Performance profiling

Projects built:

- C++ character class
- Weapon system in C++
- AI controller in C++
- Custom game mode

Practice: Convert 10 Blueprint systems to C++

Advanced Unreal Systems

Topics:

- Animation blueprints and state machines
- Control Rig and IK
- Procedural animation
- Behavior trees in Unreal
- Environmental Query System (EQS)
- Smart objects
- Level streaming and world composition

- Sequencer for cinematics
- Audio with MetaSounds
- UI with UMG and Slate
- Common UI framework
- Gameplay Ability System (GAS)

Projects built:

- Complex animation system
- Advanced AI with behavior trees
- Cinematic sequence
- Complete UI system

Practice: Master 10 advanced Unreal systems

Graphics Programming

Topics:

- Rendering pipeline overview
- Shader programming basics
- HLSL for Unreal
- Custom materials and shaders
- Post-processing effects
- Render targets
- Compute shaders
- Ray tracing basics
- Global illumination techniques
- LOD systems
- Texture streaming
- GPU profiling

Projects built:

- Custom shader effects
- Post-processing pipeline
- Procedural materials
- Performance optimization study

Practice: Create 15 custom shaders

Engine Architecture

Topics:

- Game engine architecture overview
- Memory management in games
- Object pooling patterns
- Resource management
- Threading in game engines
- Job systems
- Entity Component Systems (ECS)
- Custom engine modules
- Plugin development
- Engine modification
- Source control for large projects
- Build systems and CI/CD

Projects built:

- Custom Unreal plugin
- Memory optimization system
- Multithreaded game system
- Engine feature extension

Practice: Understand and modify engine code

Month 9: Multiplayer & Networking (Week 36-39)

Multiplayer Fundamentals

Topics:

- Networking basics for games
- Client-server architecture
- Peer-to-peer vs dedicated servers
- Unity Netcode for GameObjects
- Mirror networking
- Photon PUN2
- Network synchronization
- Lag compensation
- Client prediction
- Server reconciliation
- Interest management
- Cheat prevention

Projects built:

- LAN multiplayer game
- Online PvP arena
- Co-op puzzle game
- Battle royale prototype

Practice: Build 5 different multiplayer games

Advanced Multiplayer Systems

Topics:

- Unreal replication system
- RPCs and ownership
- State synchronization
- Deterministic physics

- Rollback netcode
- Server authoritative systems
- Matchmaking systems
- Lobby implementation
- Voice chat integration
- Dedicated server setup
- Cloud hosting (AWS, Azure)
- Load balancing

Projects built:

- Competitive multiplayer FPS
- MMO prototype with 100 players
- Dedicated server deployment
- Matchmaking system

Practice: Deploy 3 games with dedicated servers

Live Service Games

Topics:

- Live ops fundamentals
- Content delivery systems
- Seasonal content planning
- Event systems
- Player progression systems
- Battle pass implementation
- Leaderboards and rankings
- Clans and social features
- Backend services (PlayFab, GameSparks)
- Analytics and metrics
- A/B testing
- Community management

Projects built:

- Live service game with seasons
- Event system implementation
- Analytics dashboard
- Community features

Practice: Design 5 live service features

Game Monetization

Topics:

- Monetization models analysis
- Free-to-play economics
- Virtual currency systems
- Store implementation
- Loot boxes and gacha
- Battle passes
- Cosmetic systems
- Pay-to-win avoidance
- Retention strategies
- Whale management
- Ad monetization strategies
- Platform revenue sharing

Projects built:

- F2P monetization system
- Virtual economy simulator
- Store with IAP
- Retention feature implementation

Practice: Design monetization for 5 game types

Phase 3 Capstone Project

Topics:

- Large project management
- Team collaboration tools
- Asset pipeline
- Performance targets
- Platform certification requirements
- Submission preparation

Projects built:

- PHASE 3 CAPSTONE: AAA-Quality Game Vertical Slice
- Create 15-minute vertical slice of AAA-quality game
- Use Unreal Engine with C++
- Include multiplayer component
- Target next-gen quality

Assessment: Phase 3 exam on Unreal and advanced systems

PHASE 4: PUBLISHING & CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52)

Master game publishing, marketing, specialized development, and launch your game development career.

Months 10-11: Publishing and Marketing (Week 40-48)

Game Publishing Process

Topics:

- Self-publishing vs publishers
- Publishing contracts and terms
- Platform requirements (Steam, Epic, consoles)
- Certification processes
- Age ratings (ESRB, PEGI)
- Localization planning
- QA and testing processes
- Bug tracking systems
- Release planning
- Post-launch support
- Patch deployment
- DLC strategies

Projects built:

- Publishing plan for your game
- Platform submission package
- QA test plan
- Post-launch roadmap

Practice: Research 10 publishing options

Game Marketing

Topics:

- Marketing fundamentals for indies
- Building community pre-launch
- Social media strategies
- Content creation and streaming
- Press kit creation
- Trailer production
- Screenshot and GIF creation
- Steam page optimization
- Wishlist campaigns
- Influencer outreach
- Game festivals and showcases
- PR and media relations

Projects built:

- Complete marketing plan
- Game trailer production
- Press kit creation
- Steam page setup

Practice: Create marketing materials for 3 games

Business of Game Development

Topics:

- Starting an indie studio
- Business structures and legal
- Funding options: investors, publishers, grants
- Crowdfunding strategies
- Budget planning and management
- Team building and hiring
- Remote team management
- Outsourcing art and audio
- Intellectual property protection
- Revenue models and projections
- Tax considerations
- Sustainable development

Projects built:

- Business plan for indie studio
- Funding pitch deck
- Budget for game project
- Team structure proposal

Practice: Analyze 10 successful indie studios

Specialized Game Development

Topics:

- Serious games and gamification
- Educational game design
- Simulation games
- Procedural generation advanced
- Roguelike/roguelite design
- Narrative-driven games
- Interactive storytelling
- Music and rhythm games
- Sports game systems
- Strategy game AI
- MMO architecture
- Blockchain gaming basics

Projects built:

- Educational game prototype
- Procedural generation system
- Narrative-driven experience
- Specialized genre prototype

Practice: Prototype 5 specialized game types

Tools and Pipeline Development

Topics:

- Custom tool development
- Editor extensions
- Asset pipeline automation
- Build automation systems
- Version control workflows
- Art pipeline tools
- Level design tools
- Debugging and profiling tools
- Analytics integration
- Automated testing
- Continuous integration
- Documentation systems

Projects built:

- Custom Unity/Unreal tools
- Automated build pipeline
- Asset processing tools
- Team workflow optimization

Practice: Build 10 productivity tools

Month 12: Career Launch & Final Projects (Week 49-52)

Portfolio Development

Topics:

- Portfolio strategy for game developers
- Selecting best projects
- Creating gameplay videos
- Portfolio website design
- Itch.io presence
- GitHub for game developers
- ArtStation for game artists
- LinkedIn optimization
- Resume for game industry
- Cover letter writing
- Demo reel creation
- Case study writing

Projects built:

- Professional portfolio website
- Demo reel/gameplay video
- Published game on itch.io
- Polished GitHub profile

Practice: Polish all portfolio pieces

Game Industry Careers

Topics:

- Game industry roles overview
- AAA vs indie career paths
- Gameplay programmer roles
- Engine programmer positions
- Technical designer careers
- Tools programmer opportunities
- Graphics programmer path
- Network programmer roles
- Game designer positions
- Producer and PM roles
- QA and testing careers
- Remote work in games

Projects built:

- Career roadmap planning
- Skill assessment
- Target company research
- Application materials

Practice: Apply to 20 game development positions

Interview Preparation

Topics:

- Game programming interviews
- Technical tests preparation
- Whiteboard coding for games
- Game design tests
- Portfolio presentation skills
- Take-home assignments
- Behavioral questions
- Salary negotiation
- Studio culture fit
- Red flags to watch for
- Contract negotiations
- Relocation considerations

Projects built:

- Mock interview preparation
- Technical test practice
- Portfolio presentation

- Salary research

Practice: Complete 10 mock interviews

Indie Game Development

Topics:

- Indie development lifestyle
- Time management for indies
- Scope management
- Community building
- Game jams participation
- Indie showcases
- Networking strategies
- Collaboration finding
- Revenue sharing models
- Sustainable indie development
- Building a following
- Patreon and crowdfunding

Projects built:

- Game jam participation
- Community building plan
- Indie game pitch
- Collaboration proposal

Practice: Participate in 3 game jams

Final Project & Launch

Topics:

- Final project planning
- Scope and timeline
- Asset creation pipeline
- Testing and polishing
- Marketing campaign execution
- Launch preparation
- Community engagement
- Post-launch planning
- Career next steps
- Continuous learning
- Industry networking
- Future goals setting

Projects built:

- FINAL CAPSTONE: Commercial-Ready Game
- Develop and publish a complete game
- Include: 2+ hours gameplay, original assets, multiplayer or procedural elements
- Publish on Steam/itch.io with marketing campaign
- Achieve 100+ wishlists/downloads

Assessment: Final portfolio review and career readiness assessment

Ongoing Development & Growth (Continuous)

Staying Current

Topics:

- Following industry trends
- New technology adoption
- Engine updates and features
- Platform changes
- Market analysis
- Player behavior trends
- Emerging platforms
- New monetization models
- Technology forecasting
- Skill development planning
- Conference attendance
- Online learning resources

Projects built:

- Technology radar creation
- Learning plan development
- Trend analysis report
- Skill upgrade roadmap

Practice: Learn one new technology monthly

Advanced Specializations

Topics:

- Graphics programming mastery
- Engine development
- Technical art
- Procedural generation systems
- AI and machine learning for games
- Cloud gaming technology
- Streaming technology
- Cross-platform development
- Accessibility in games
- Experimental gameplay
- Research and development
- Innovation in games

Projects built:

- Specialization deep dive
- Research project
- Innovative prototype
- Technical paper

Practice: Master chosen specialization

Community and Teaching

Topics:

- Community involvement
- Open source contributions
- Asset sharing
- Tutorial creation
- Blog writing
- YouTube content
- Speaking at events
- Mentoring others
- Code review participation
- Forum helping
- Discord communities
- Building reputation

Projects built:

- Tutorial series creation
- Open source tool
- Community event organization
- Mentorship program

Practice: Give back to community weekly

Studio Leadership

Topics:

- Team leadership skills
- Project management
- Agile for game development
- Sprint planning
- Risk management
- Quality assurance
- Performance reviews
- Conflict resolution
- Vision and direction
- Studio culture building
- Scaling teams
- Long-term planning

Projects built:

- Leadership portfolio
- Team management system
- Studio culture document
- Growth strategy plan

Practice: Lead game development teams

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 10+ 2D games - platformer, puzzle, shooter, mobile
- Phase 2: 8+ 3D games - FPS, racing, VR, AR experiences
- Phase 3: 5+ advanced projects - multiplayer, Unreal AAA slice
- Phase 4: 3+ commercial projects - published game, marketing campaign
- Total: 25+ complete games from simple to commercial quality

Total projects: 25+ playable games across all platforms and genres

SKILLS THE STUDENT WILL HAVE AT THE END

- Game Design: mechanics, balance, level design, monetization, UX
- Programming: C#, C++, Blueprints, visual scripting, design patterns
- Engines: Unity (expert), Unreal Engine (advanced), custom engines
- Graphics: shaders, lighting, materials, VFX, optimization
- 3D: modeling basics, animation, rigging, physics
- 2D: sprite art, animation, tile systems, pixel art
- Audio: sound design, music, spatial audio, dynamic audio
- AI: pathfinding, behavior trees, state machines, ML basics
- Networking: multiplayer, client-server, dedicated servers
- Platforms: PC, mobile, console, VR/AR, web
- Business: publishing, marketing, monetization, studio management
- Tools: version control, CI/CD, analytics, profiling

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-7 hours

Hands on coding: 10-12 hours

Projects: 5-6 hours

Practice problems: 2-3 hours

Total per week: 20-25 hours

SUPPORT INCLUDED

Live sessions: Weekly game dev workshops and code reviews
Mentorship: 1-on-1 guidance from industry game developers
Community: Active Discord with 5000+ game developers
Code review: Expert reviews of all game projects
Career support: Portfolio reviews, job referrals, interview prep
Lifetime access: All content and engine updates forever

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
Final certificate: Certified Game Developer
Linkedin badge: Industry-recognized credential
Industry recognized: Accepted by game studios globally
Portfolio projects: 25+ games for portfolio
Published games: At least 1 published game on major platform

WHERE THIS CAN LEAD

- Game Programmer (Gameplay/Engine/Graphics/AI)
- Unity Developer
- Unreal Engine Developer
- Technical Game Designer
- Indie Game Developer
- Mobile Game Developer
- VR/AR Developer
- Game Tools Programmer
- Multiplayer Programmer
- Technical Artist
- Game Studio Founder
- Freelance Game Developer

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Growing roles: Rs 4-8 LPA (Junior Game Developer)

Experienced roles: Rs 8-15 LPA (Game Developer); Rs 12-25 LPA (Senior Game Developer); Rs 20-40+ LPA (Lead Developer/Technical Director)

Freelance: Rs 2,000-10,000/hour based on expertise

International: \$60k-150k USD (higher in AAA studios)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which game engines does the course teach?

A: Both industry leaders: Unity with C# for the first half, covering 2D and 3D development, and Unreal Engine with C++ in the advanced phase. Learning both makes you far more employable than knowing one, and you finish knowing which engine suits which kind of project.

Q: Do I need to know programming before joining?

A: No, programming is taught from scratch inside the course, starting with C# in Unity. What you do need is a reasonable PC: 8GB RAM minimum with a dedicated GPU recommended, since game engines are heavier than normal coding tools.

Q: What games will I actually make?

A: You build complete, playable games across the year: 2D games in the foundation phase, 3D games as your Unity skills deepen, Unreal Engine projects in months 7 to 9, and multiplayer work plus a polished portfolio piece in the final phase, including preparing a game for platforms like Steam.

Q: Is game development a realistic career from here?

A: The skills are real and transferable: C#, C++, 3D mathematics and engine experience apply to games, simulation, AR/VR and beyond. You finish with playable projects in a portfolio, which is how studios evaluate juniors. We are honest that game industry hiring is competitive and we make no job guarantees; we give you work worth showing.

Q: How is this better than learning Unity from YouTube?

A: Tutorial code works until you change anything; then you are alone. A live mentor reviews your project architecture, debugs with you, explains why your physics or performance broke, and keeps you finishing games instead of abandoning them, which is the most common way self-taught game developers stall.

Q: Can I sit in a class before paying?

A: Yes, the first step for every student is a free live demo class, booked at learn.modernagecoders.com/contact with no commitment. You meet the instructor and see how a live game development session runs before any payment.

Q: How are classes run?

A: Live 1-hour online sessions with an instructor, never pre-recorded content. You work in the engine during class with the mentor watching and correcting, and evening and weekend slots keep the course workable alongside college.

Q: Do I receive a certificate?

A: Yes, a game developer completion certificate, but the portfolio matters more: you leave with playable games and a GitHub-ready body of work, which is what studios and clients ask to see first.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/game-development-complete-masterclass-college>

=====

COURSE 58 OF 118: GAME DEVELOPMENT FOR KIDS: SCRATCH, ROBLOX & MINECRAFT

=====

Course link: <https://learn.modernagecoders.com/courses/game-development-masterclass-for-kids>

Subject area: Game Development & Design

Designed for: Kids

Who can join: Complete Beginner to Advanced Game Creator

Course length: 6 months (26 weeks)

Time needed: 6-8 hours/week recommended

Certificate: Junior Game Developer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Game development course for kids ages 8-18: create and publish real games with Scratch, Roblox Studio, Minecraft and GDevelop, no complex coding needed.

In one line: Transform from Game Player to Game Creator in 6 Months!

ABOUT THE PROGRAMME

This exciting 6-month journey transforms game-loving kids into game creators! Using kid-friendly visual tools, students will create 50+ games across multiple platforms - from simple 2D arcade games to complex 3D multiplayer worlds. No typing code required! Students will master Scratch, Roblox Studio, Minecraft Education, GDevelop, and more. They'll learn game design, storytelling, level design, character creation, and even game monetization. By the end, they'll have a portfolio of published games and the skills to bring any game idea to life!

WHAT MAKES THIS COURSE DIFFERENT

- 100% visual tools - no complex coding required
- Create games that friends can actually play
- Learn 10+ different game creation platforms
- Publish games online and to app stores
- Make money from games (parent-supervised)
- Both 2D and 3D game development
- Multiplayer and social games included
- Game design theory made fun for kids

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-2): Game Logic, Scratch, First Games

Phase 2: 2D Games (Months 3-4): GDevelop, Construct, Mobile Games

Phase 3: 3D Worlds (Months 5-6): Roblox, Minecraft, Multiplayer

Phase 4: Publishing: Share games with the world!

WHAT A STUDENT NEEDS BEFORE STARTING

Education: None - suitable for ages 8-18

Coding experience: Zero coding required

Equipment: Computer/laptop, optional: tablet/smartphone

Time commitment: 6-8 hours per week

English: Basic English understanding

Motivation: Love for games and creativity

WHO THIS COURSE IS FOR

Students: Kids (8+) and teens who love games
Working professionals: Teachers and parents learning with kids
Entrepreneurs: Young entrepreneurs wanting game businesses
Freelancers: Future game developers and designers
Kids: Game-loving kids who want to create
Anyone: Anyone passionate about making games

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: GAME FOUNDATIONS & FIRST GAMES (MONTHS 1-2, WEEKS 1-8) -----

Master game thinking and create your first games using Scratch and basic tools. Learn what makes games fun!

Months 1-2: Game Logic & Scratch Mastery (Week 1-8)

Introduction to Game Development

Topics:

- What makes a game fun?
- Types of games: arcade, puzzle, adventure, RPG
- Game development process overview
- Introduction to Scratch for games
- Scratch interface for game makers
- Sprites as game characters
- Backgrounds and game worlds
- Basic movement: arrow keys and WASD
- Collision detection basics
- Game coordinates (X and Y)
- Your first playable game
- Sharing games with friends

Projects built:

- Catch the falling objects game
- Simple maze game
- Pet virtual game
- Two-player tag game

Practice: Daily: Play and analyze 2 games, create 1 game mechanic

Game Mechanics & Player Control

Topics:

- Player movement patterns
- Jump mechanics and gravity
- Speed and acceleration
- Power-ups and collectibles
- Enemy movement and AI basics
- Health and lives system
- Score and points tracking
- Timer and countdown mechanics
- Game over and restart
- Win conditions
- Difficulty levels
- Checkpoints and saving progress

Projects built:

- Platformer with jumping
- Racing game with timer
- Shooting game with score
- Puzzle with levels

Practice: Create 10 different game mechanics

Advanced Scratch Game Development

Topics:

- Scrolling backgrounds
- Multiple levels design
- Boss battles creation
- Inventory systems
- Dialog and storytelling
- Cut scenes and intros
- Sound effects and music
- Particle effects with clones
- Advanced animations
- Cloud variables for high scores
- Multiplayer with cloud data
- Publishing and sharing games

Projects built:

- Multi-level adventure game
- RPG with inventory
- Battle arena game
- Story-driven game

Practice: Complete 5 advanced Scratch games

Introduction to GDevelop

Topics:

- GDevelop interface tour
- Objects and behaviors
- Events system in GDevelop
- Physics engine basics
- Animations and sprites
- Sound and music integration
- Touch controls for mobile
- Scene management
- Variables and game logic
- Exporting games
- HTML5 game publishing
- Mobile game basics

Projects built:

- Physics puzzle game
- Endless runner
- Tower defense game
- Mobile tap game

Practice: Create 5 games in GDevelop

Months 3-4: 2D Game Mastery (Week 9-17)

Construct 3 Game Engine

Topics:

- Construct 3 interface
- Behaviors and effects
- Event sheets mastery
- Layouts and layers
- Sprite animations
- Tilemaps for levels
- Pathfinding for enemies
- Line of sight behavior
- Platform behavior advanced
- Mobile optimization
- Touch and gesture controls
- Debugging tools

Projects built:

- Zelda-style adventure
- Angry Birds clone
- Match-3 puzzle game
- Stealth game

Practice: Build 8 different game genres

Game Art & Animation

Topics:

- Pixel art basics
- Character design principles
- Animation fundamentals
- Background creation
- Tile sets design
- UI elements design
- Color theory for games
- Free art resources
- Piskel for pixel art
- Sound effect creation
- Music for games
- Asset optimization

Projects built:

- Design game character set
- Create animated sprites
- Design full game tileset
- Complete game art package

Practice: Create art for 5 games

Mobile Game Development

Topics:

- Mobile game design principles
- Touch control optimization
- Screen size adaptation
- Performance optimization
- Accelerometer controls
- Swipe and gesture mechanics
- Mobile UI/UX design
- In-app purchases basics
- Ad integration concepts
- App store preparation
- Testing on devices
- Analytics basics

Projects built:

- Mobile endless runner

- Puzzle game for phones
- Casual tap game
- Mobile multiplayer game

Practice: Optimize 5 games for mobile

GameMaker Studio Basics

Topics:

- GameMaker interface
- GML Visual scripting
- Rooms and objects
- Sprites and animations
- Collision systems
- Camera and views
- Particle systems
- Sound management
- Save systems
- Menu creation
- Polish and juice
- Publishing options

Projects built:

- Retro arcade game
- Metroidvania demo
- Bullet hell shooter
- Roguelike dungeon

Practice: Master 10 GameMaker features

2D Games Showcase

Topics:

- Portfolio preparation
- Game trailers creation
- itch.io publishing
- Game documentation
- Player feedback integration
- Bug fixing strategies
- Game updates and patches

Projects built:

- Polish best 2D game
- Create game trailer
- Publish to itch.io
- Get player feedback

Assessment: Complete 2D game from concept to publication

Months 5-6: 3D Worlds & Multiplayer (Week 18-26)

Roblox Studio Fundamentals

Topics:

- Roblox Studio interface
- Parts and models
- Terrain editor
- Basic scripting with blocks
- GUI creation
- Character customization
- Tools and weapons
- Game passes creation
- Team create feature
- Publishing to Roblox
- Robux monetization basics
- Roblox community

Projects built:

- Obby (obstacle course)
- Tycoon game
- Racing game
- Battle royale map

Practice: Create 5 Roblox experiences

Advanced Roblox Development

Topics:

- Advanced building techniques
- Scripting game mechanics
- Datastores for saving
- Leaderboards and stats
- Custom animations
- Particle effects
- Lighting and atmosphere
- Sound regions
- NPCs and AI
- Vehicle systems
- Trading systems
- Group games

Projects built:

- RPG with quests
- Simulator game
- Horror experience

- Social hangout game

Practice: Master 15 Roblox features

Minecraft Modding & Creation

Topics:

- Minecraft Education Edition
- Command blocks mastery
- Redstone mechanics
- Custom maps creation
- Adventure mode design
- Resource packs basics
- Function files
- Custom structures
- Minecraft marketplace
- Server creation basics
- Minigames design
- Parkour map creation

Projects built:

- Adventure map with story
- Minigame collection
- Parkour challenge map
- Survival challenge map

Practice: Build 5 Minecraft experiences

Unity with Visual Scripting

Topics:

- Unity Hub and installation
- Unity interface basics
- GameObject and components
- Visual scripting with Bolt
- Basic 3D movement
- Physics and collisions
- Lighting basics
- Terrain creation
- Asset store usage
- UI in Unity
- Build settings
- WebGL publishing

Projects built:

- 3D platformer
- First-person explorer
- Ball rolling game
- Simple VR experience

Practice: Complete 3 Unity projects

Final Game Project & Publishing

Topics:

- Choosing final project
- Game design document
- Production planning
- Asset creation pipeline
- Testing and debugging
- Publishing preparation
- Marketing basics
- Community building

Projects built:

- FINAL PROJECT: Complete original game
- Publish to 2+ platforms
- Create marketing materials
- Build player community

Assessment: Launch complete game with 100+ players

PHASE 2: ADVANCED GAME DEVELOPMENT (SPECIALIZED TRACKS)

Choose specialized paths: Mobile Games, Multiplayer, VR/AR, or Competitive Gaming

Mobile Game Specialization (Mobile Track)

Buildbox for Mobile Games

Topics:

- Buildbox 3 interface
- Node-based logic
- Asset creation and import
- Physics-based games
- Monetization setup
- Ad networks integration
- In-app purchases setup
- Analytics integration
- Push notifications
- Social media sharing
- Leaderboards integration
- Cross-platform publishing

Projects built:

- Hyper-casual game
- Physics puzzle game
- Endless arcade game
- Idle clicker game

Practice: Publish 3 games to app stores

PlayCanvas Web Games

Topics:

- PlayCanvas editor
- 3D web games
- Visual scripting in PlayCanvas
- Asset pipeline
- Multiplayer basics
- WebGL optimization
- Mobile web games
- Facebook Instant Games
- Web monetization
- PWA game apps
- Cloud hosting
- Version control

Projects built:

- 3D web runner
- Multiplayer web game
- Facebook Instant Game
- PWA game app

Practice: Deploy 5 web games

Core Games Platform

Topics:

- Core Games introduction
- Collaborative creation
- Lua scripting basics
- Framework usage
- Multiplayer by default
- Monetization in Core
- User-generated content
- Events and game jams
- Community features
- Publishing process
- Core credits system
- Cross-promotion

Projects built:

- Multiplayer arena
- Co-op adventure
- Party game collection
- RPG framework game

Practice: Create 3 Core games

Stencyl 2D Games

Topics:

- Stencyl interface
- Block-based coding
- Actor behaviors
- Scene designer
- Physics settings
- Mobile controls
- Atlases and resources
- Extensions usage
- Performance tips
- Publishing workflow
- Steam publishing
- Console basics

Projects built:

- Platform adventure
- Tower defense game
- Puzzle platformer
- Action RPG

Practice: Master Stencyl workflows

Godot Engine Basics

Topics:

- Godot installation
- Node system
- 2D and 3D modes
- Visual scripting
- GDScript basics
- Scene system
- Animation tools
- Particle systems
- Export templates
- Open source benefits
- Community resources

- Version control
- Projects built:
- 2D action game
 - 3D prototype
 - Mobile export test
 - Web game deployment
- Practice: Explore Godot capabilities

Multiplayer & Social Games (Multiplayer Specialization)

Multiplayer Game Design

Topics:

- Multiplayer game types
- Competitive vs cooperative
- Matchmaking concepts
- Lobby systems
- Player progression
- Ranking systems
- Anti-cheat basics
- Server concepts
- Lag compensation
- Social features
- Guild/clan systems
- Tournament setup

Projects built:

- Battle royale prototype
- Co-op puzzle game
- Racing multiplayer
- Team-based shooter

Practice: Design 5 multiplayer mechanics

Among Us Style Games

Topics:

- Social deduction mechanics
- Role assignment
- Task systems
- Voting mechanics
- Discussion phases
- Map design for deduction
- Balance and fairness
- Custom game modes
- Moderation tools
- Community management
- Update strategies
- Viral mechanics

Projects built:

- Social deduction game
- Hidden role game
- Mystery game
- Party game with roles

Practice: Create social game mechanics

io Games Development

Topics:

- io game characteristics
- Simple multiplayer setup
- Browser-based games
- Quick match systems
- Progression in io games
- Skins and customization
- Leaderboard importance
- Server regions
- Bot players
- Viral growth tactics
- Ad-based monetization
- Clone protection

Projects built:

- Snake io game
- Territory io game
- Battle io game
- Unique io concept

Practice: Launch io game with players

Minecraft Server Creation

Topics:

- Server setup basics
- Plugins and mods
- Game modes creation
- Economy systems
- Permission systems
- Anti-grief measures
- Custom commands
- Events and contests

- Server promotion
- Community building
- Donation systems
- Server networks

Projects built:

- Survival server setup
- Minigame server
- RPG server creation
- Creative build server

Practice: Run server with 20+ players

Discord Game Bots

Topics:

- Discord bot basics
- Bot commands
- Mini-games in Discord
- Economy bots
- RPG bot systems
- Leveling systems
- Music and entertainment
- Moderation features
- Custom games
- Bot hosting
- Bot promotion
- Monetization options

Projects built:

- RPG Discord bot
- Economy bot game
- Quiz bot game
- Custom game bot

Practice: Deploy bot to 10 servers

VR/AR & Emerging Tech (Future Tech Track)

VR Game Development

Topics:

- VR concepts for kids
- CoSpaces Edu for VR
- Google Cardboard apps
- VR safety guidelines
- 360-degree experiences
- VR storytelling
- Movement in VR
- Interaction systems
- Comfort settings
- VR for education
- Social VR basics
- Testing VR games

Projects built:

- VR escape room
- Educational VR tour
- VR art gallery
- Simple VR game

Practice: Create 3 VR experiences

AR Games with Lens Studio

Topics:

- Snapchat Lens Studio
- AR markers and tracking
- Face filters and games
- World lenses
- AR mini-games
- Social AR experiences
- AR treasure hunts
- Location-based AR
- AR for learning
- Publishing lenses
- Viral AR effects
- AR monetization

Projects built:

- Face filter game
- AR treasure hunt
- World lens game
- Educational AR app

Practice: Publish 5 AR lenses

AI in Games

Topics:

- AI for game NPCs
- Behavior trees basics
- Pathfinding algorithms
- Decision making AI
- Procedural generation

- AI difficulty adjustment
- Machine learning basics
- Voice commands in games
- AI art generation
- ChatGPT for game dialog
- AI game testing
- Future of AI in games

Projects built:

- AI-powered NPCs
- Procedural level generator
- AI opponent system
- Voice-controlled game

Practice: Implement 5 AI features

AI in Games

Topics:

- How game AI actually works (it is mostly rules)
- Enemy behaviour and state machines
- Pathfinding: how an NPC finds you
- Difficulty that adapts to the player
- Procedural level generation
- Randomness versus fairness
- AI-generated art and its limits
- Using AI to help you build faster
- What AI cannot design for you
- Playtesting: does the AI feel fun or cheap?
- Ethics of AI in games
- Crediting AI honestly in your credits screen

Projects built:

- Smart enemy AI
- Procedural level generator
- Adaptive difficulty
- NPC dialogue system

Practice: Build 3 AI-driven game features

Game Developer Portfolio

Topics:

- Portfolio website creation
- Game showcase strategies
- Demo reel creation
- Writing game descriptions
- Screenshot techniques
- Video trailers
- Developer blog
- Social media presence
- Networking basics
- Game jams participation
- Future learning paths
- Career planning

Projects built:

- Complete portfolio site
- 50+ game showcase
- Demo reel video
- Developer brand creation

Assessment: Final certification project

SPECIALIZATION TRACKS: CHOOSE YOUR PATH

Deep dive into specialized areas of game development

Esports & Competitive Gaming (Competitive Track)

Esports Game Design

Topics:

- Competitive game balance
- Ranking systems design
- Tournament structures
- Spectator features
- Replay systems
- Anti-cheat measures
- Skill-based matchmaking
- Season systems
- Competitive maps
- Meta game design
- Patch strategies
- Community feedback

Projects built:

- Competitive arena game
- Tournament system
- Ranked game mode
- Spectator mode

Practice: Balance competitive gameplay

Stream-Friendly Games

Topics:

- Streamer-friendly features
- Audience participation
- Twitch integration
- Stream overlays
- Clip-worthy moments
- Content creator tools
- Copyright-free assets
- Stream alerts
- Chat commands
- Viewer games
- Streamer monetization
- Community features

Projects built:

- Twitch integrated game
- Audience voting game
- Streamer toolkit
- Viral moment generator

Practice: Stream game development

Game Analytics & Metrics

Topics:

- Player behavior tracking
- Key performance indicators
- Retention metrics
- Monetization analytics
- A/B testing in games
- Heatmap analysis
- Player progression tracking
- Churn prediction
- Analytics dashboards
- Data-driven design
- Privacy considerations
- GDPR compliance

Projects built:

- Analytics dashboard
- A/B testing system
- Player report system
- Metric tracking setup

Practice: Analyze 5 games' metrics

Game Marketing & Growth

Topics:

- Game trailers creation
- Press kit preparation
- Social media marketing
- Influencer outreach
- Steam page optimization
- App store featuring
- Community building
- Discord server management
- Reddit promotion
- Game conferences
- Awards and contests
- Post-launch support

Projects built:

- Marketing campaign
- Press kit creation
- Community launch
- Influencer package

Practice: Market game to 1000 players

Game Monetization Strategies

Topics:

- Free-to-play models
- Premium game pricing
- In-app purchases design
- Battle pass systems
- Cosmetic monetization
- Ad placement strategies
- Subscription models
- DLC planning
- Season passes
- Virtual currency
- Ethical monetization
- Regional pricing

Projects built:

- F2P monetization system
- Battle pass design
- Virtual shop creation

- Ad integration setup
- Practice: Implement 3 monetization models

Educational Game Development (EdTech Track)

Learning Game Design

Topics:

- Educational game theory
- Curriculum alignment
- Learning objectives
- Assessment integration
- Progress tracking
- Adaptive difficulty
- Feedback systems
- Gamification principles
- Subject integration
- Age-appropriate design
- Teacher dashboards
- Parent reports

Projects built:

- Math learning game
- Science simulation
- Language learning app
- History adventure game

Practice: Create 5 educational games

STEM Game Development

Topics:

- Coding games for kids
- Logic puzzle design
- Physics simulations
- Chemistry lab games
- Biology explorations
- Engineering challenges
- Robotics simulations
- Space exploration games
- Environmental games
- AI teaching games
- Data science games
- Maker education

Projects built:

- Coding tutorial game
- Physics puzzle game
- Robot programming game
- Science lab simulator

Practice: Build STEM game library

Serious Games Development

Topics:

- Health education games
- Safety training games
- Social awareness games
- Environmental games
- Historical simulations
- Cultural education
- Financial literacy games
- Career exploration
- Soft skills training
- Empathy games
- Accessibility awareness
- Impact measurement

Projects built:

- Health awareness game
- Environmental impact game
- Social skills trainer
- Career exploration game

Practice: Create games with purpose

Classroom Integration

Topics:

- LMS integration
- Google Classroom
- Assignment creation
- Grading integration
- Collaborative features
- Classroom management
- Student accounts
- Progress monitoring
- Curriculum mapping
- Standards alignment
- Professional development
- Teacher resources

Projects built:

- Classroom game suite
- Teacher toolkit
- Assignment system
- Progress tracker

Practice: Deploy games in schools

Game Accessibility

Topics:

- Visual accessibility
- Hearing accessibility
- Motor accessibility
- Cognitive accessibility
- Difficulty options
- Control remapping
- Colorblind modes
- Subtitle systems
- Screen reader support
- One-handed modes
- Pause features
- Accessibility testing

Projects built:

- Fully accessible game
- Accessibility retrofit
- Inclusive design game
- Testing documentation

Practice: Make all games accessible

Professional Skills & Industry (Career Preparation)

Game Industry Overview

Topics:

- Game industry careers
- AAA vs indie development
- Game development roles
- Studio structures
- Publisher relationships
- Platform holders
- Industry trends
- Regional markets
- Game conferences
- Networking events
- Industry news sources
- Career pathways

Projects built:

- Industry research project
- Career plan creation
- Network building
- Conference preparation

Practice: Connect with 20 developers

Game Testing & QA

Topics:

- Testing methodologies
- Bug reporting
- Test plans creation
- Playtesting sessions
- Balance testing
- Performance testing
- Compatibility testing
- User testing
- Focus groups
- Feedback analysis
- Issue tracking
- QA documentation

Projects built:

- Test plan creation
- Bug tracking system
- Playtest organization
- QA report compilation

Practice: Test 10 games professionally

Game Documentation

Topics:

- Game design documents
- Technical documentation
- Art style guides
- Sound design docs
- Level design docs
- User manuals
- API documentation
- Wiki creation
- Tutorial design
- Onboarding flows

- Patch notes writing
- Community guides

Projects built:

- Complete GDD
- Technical manual
- Player guide
- Developer wiki

Practice: Document 5 game projects

Team Collaboration

Topics:

- Version control basics
- Team communication
- Task management
- Agile for games
- Sprint planning
- Code reviews
- Art pipelines
- Audio workflows
- Remote collaboration
- Conflict resolution
- Leadership skills
- Mentoring others

Projects built:

- Team game project
- Pipeline setup
- Collaboration tools
- Team leadership

Practice: Lead game team project

Final Showcase & Graduation

Topics:

- Portfolio finalization
- Demo day preparation
- Pitch presentation
- Interview preparation
- Resume building
- LinkedIn optimization
- Personal branding
- Future goals setting
- Continued learning
- Alumni network
- Certification process
- Graduation celebration

Projects built:

- FINAL SHOWCASE: Best games portfolio
- Professional presentation
- Career materials
- Future roadmap

Assessment: Graduate as Certified Game Developer

BONUS CONTENT: ADVANCED SPECIALIZATIONS

Optional advanced modules for continued excellence

Game Art & Sound Mastery (Creative Excellence)

Advanced Pixel Art

Topics:

- Advanced pixel techniques
- Character animation cycles
- Isometric pixel art
- Tile set creation
- Dithering and shading
- Color palette mastery
- Pixel art effects
- Animation principles
- Style development
- Asset optimization
- Pixel art tools mastery
- Commercial asset creation

Projects built:

- Complete asset pack
- Animated character set
- Environment tiles
- Effect animations

Practice: Create professional pixel art

3D Modeling Basics

Topics:

- Blender basics for games
- Low-poly modeling

- Texturing basics
- UV mapping introduction
- Rigging fundamentals
- Animation basics
- Optimization for games
- Asset pipelines
- Procedural generation
- Photogrammetry basics
- 3D scanning
- Asset stores

Projects built:

- Low-poly character
- Environment props
- Animated model
- Game-ready assets

Practice: Create 10 3D game assets

Game Audio & Music

Topics:

- Sound effect creation
- Foley recording
- Audio editing basics
- Music composition basics
- Chiptune creation
- Adaptive music systems
- 3D spatial audio
- Voice acting basics
- Audio implementation
- Compression and formats
- Audio middleware
- Licensing and royalties

Projects built:

- SFX library creation
- Game soundtrack
- Voice acting samples
- Audio system setup

Practice: Create audio for 5 games

Procedural Generation

Topics:

- Procedural concepts
- Random generation
- Noise functions
- Dungeon generation
- Terrain generation
- Item generation
- Quest generation
- Name generation
- Infinite worlds
- Seed systems
- Procedural storytelling
- Balance in procedural

Projects built:

- Procedural dungeon
- Infinite runner generator
- Random quest system
- Procedural RPG

Practice: Implement 5 procedural systems

Game Narrative Design

Topics:

- Story structure in games
- Character development
- Dialog writing
- Branching narratives
- Choice systems
- Environmental storytelling
- Lore creation
- Quest design
- Cutscene planning
- Voice direction
- Localization prep
- Narrative tools

Projects built:

- Story-driven game
- Dialog system
- Quest chain design
- Lore documentation

Practice: Write 5 game narratives

Business & Entrepreneurship (Game Business Mastery)

Starting a Game Studio

Topics:

- Business planning
- Legal structures
- Funding options
- Team building
- Studio culture
- Project management
- Budgeting basics
- Revenue sharing
- Publishing deals
- IP protection
- Trademark basics
- Studio branding

Projects built:

- Business plan draft
- Studio branding
- Team structure
- Funding pitch

Practice: Plan game studio launch

Crowdfunding Games

Topics:

- Kickstarter strategies
- Campaign preparation
- Reward tier design
- Video production
- Marketing timeline
- Backer communication
- Stretch goals
- Risk management
- Fulfillment planning
- Platform comparison
- Success stories
- Failure analysis

Projects built:

- Campaign page design
- Reward structure
- Marketing plan
- Video script

Practice: Plan crowdfunding campaign

Game Publishing

Topics:

- Publisher relationships
- Publishing contracts
- Self-publishing options
- Platform requirements
- Certification processes
- Rating systems
- Localization needs
- Marketing support
- Revenue splits
- Milestone planning
- Post-launch support
- Sequel planning

Projects built:

- Publishing strategy
- Platform research
- Contract review
- Launch plan

Practice: Research 10 publishers

Live Operations

Topics:

- Live service games
- Content calendars
- Event planning
- Season design
- Community management
- Update strategies
- Bug fix priorities
- Player communication
- Data-driven updates
- Revenue optimization
- Player retention
- Sunset planning

Projects built:

- Live ops plan
- Event calendar
- Update roadmap
- Community strategy

Practice: Run live game for 3 months

Game Franchises

Topics:

- Franchise planning
- Sequel design
- Spin-off strategies
- Brand management
- Transmedia opportunities
- Merchandising
- Licensing deals
- Fan community
- Lore consistency
- Art bible creation
- Style guides
- Legacy planning

Projects built:

- Franchise bible
- Sequel concept
- Merchandise design
- Brand guide

Practice: Plan game franchise

Cutting-Edge Technologies (Future of Gaming)

Cloud Gaming Development

Topics:

- Cloud gaming platforms
- Streaming technology
- Input lag solutions
- Cross-device play
- Save synchronization
- Bandwidth optimization
- 5G gaming
- Edge computing
- Platform integration
- Business models
- Future predictions
- Development tools

Projects built:

- Cloud-native game
- Streaming optimization
- Cross-device feature
- Cloud save system

Practice: Deploy cloud game

AI Game Development

Topics:

- AI-generated content
- Procedural narratives
- AI game masters
- Dynamic difficulty
- Player modeling
- Behavior prediction
- AI art generation
- Voice synthesis
- Natural language
- Machine learning integration
- Ethical considerations
- Future of AI gaming

Projects built:

- AI-powered game
- ML difficulty system
- AI content generator
- Predictive systems

Practice: Integrate advanced AI

Multiplayer & Social Game Design

Topics:

- What makes a multiplayer game fun
- Designing a shared space players want to stay in
- Avatar and identity systems
- In-game currency and item economies
- Chat safety, reporting and blocking
- Moderation: what happens when players are horrible
- Live events and seasons
- Server basics and why lag happens
- Cross-platform play
- Building a community around your game
- Publishing responsibly to a young audience
- Reading your own analytics honestly

Projects built:

- Multiplayer lobby
- Avatar system

- In-game economy
- Social hub world

Practice: Build a multiplayer social space

Experimental Gaming

Topics:

- Alternative controllers
- Biometric gaming
- Brain-computer interfaces
- Haptic feedback
- Smell and taste
- Mixed reality
- Holographic displays
- Quantum gaming
- Bio-gaming
- Art games
- Experimental narratives
- Future possibilities

Projects built:

- Experimental prototype
- Alternative input game
- Art game creation
- Future concept

Practice: Push gaming boundaries

Master Game Developer Path (Excellence Achievement)

Game Development Mastery

Topics:

- Design philosophy
- Personal style development
- Signature mechanics
- Innovation strategies
- Risk-taking in design
- Artistic vision
- Technical excellence
- Polish and refinement
- Perfectionism balance
- Speed vs quality
- Iteration mastery
- Decision making

Projects built:

- Signature game project
- Innovation showcase
- Technical achievement
- Artistic statement

Industry Leadership

Topics:

- Thought leadership
- Public speaking
- Conference talks
- Workshop leading
- Mentorship programs
- Community building
- Open source contribution
- Standards setting
- Industry advocacy
- Diversity initiatives
- Youth programs
- Future shaping

Deliverables:

- Leadership portfolio
- Speaking engagements
- Mentorship record
- Community impact
- Industry contributions
- Published articles
- Workshop materials
- Legacy projects

Your Gaming Legacy

Topics:

- Career reflection
- Achievement celebration
- Impact assessment
- Future planning
- Giving back
- Knowledge sharing
- Next generation
- Industry evolution
- Personal brand
- Continued growth

- Life-long learning
- Your game story

Deliverables:

- Complete portfolio: 50+ games
- Published titles
- Community contributions
- Student success stories
- Industry recognition
- Personal brand established
- Network of 500+ developers
- Future roadmap

Assessment: MASTER GAME DEVELOPER CERTIFICATION ACHIEVED!

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1-2: 20 Scratch games, basic mechanics
- Month 3-4: 15 2D games across platforms
- Month 5-6: 10 3D games and multiplayer
- Specialization: 15+ specialized projects
- Total: 60+ complete games

Total projects: 60+ games from simple to complex, published titles

SKILLS THE STUDENT WILL HAVE AT THE END

- Game Design: Mechanics, balance, fun factor, player psychology
- 2D Development: Sprites, physics, platformers, puzzles
- 3D Development: Modeling basics, terrain, lighting, cameras
- Programming: Visual scripting, logic, algorithms, AI basics
- Art & Animation: Pixel art, sprites, UI design, effects
- Sound & Music: SFX, music, voice, implementation
- Multiplayer: Networking, servers, matchmaking, social
- Publishing: App stores, Steam, web platforms, marketing
- Monetization: Ads, IAP, premium, subscriptions
- Tools: 15+ game engines and platforms mastered
- Business: Marketing, analytics, community, monetization

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 2-3 hours
 Hands on creation: 3-4 hours
 Projects: 2-3 hours
 Playing and analysis: 1-2 hours
 Total per week: 6-8 hours

SUPPORT INCLUDED

Live sessions: Weekly game dev workshops
 Mentorship: Expert game developer mentors
 Community: Active Discord with 1000+ young developers
 Code review: Game testing and feedback
 Career support: Portfolio reviews and industry connections
 Lifetime access: All content and platform updates forever

CERTIFICATION DETAIL

Phase certificates: Certificates for each game platform mastered
 Final certificate: Certified Junior Game Developer
 Linkedin badge: Professional certification badge
 Industry recognized: Recognized by game studios
 Portfolio projects: 60+ games to showcase

WHERE THIS CAN LEAD

- Junior Game Developer
- Game Designer
- Level Designer
- Game Tester
- Unity/Unreal Developer (progression path)
- Indie Game Developer
- Game Content Creator/YouTuber
- Game Instructor/Teacher
- Game Entrepreneur
- Professional Game Developer (with continued learning)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What age is appropriate for this game development course for kids?

A: This course is designed for children ages 8-18. We use visual block-based programming in Scratch, GDevelop, and other kid-friendly engines that make game creation accessible without requiring typing skills. Younger kids (8-12) can fully participate, while teens will progress faster into more advanced 3D game development.

Q: Does my child need prior coding experience to start this game development course?

A: No prior coding experience is needed! We start from absolute scratch (literally using Scratch!). The course uses drag-and-drop visual programming, which is perfect for beginners. We progressively introduce more complex concepts as kids build confidence, eventually transitioning to platforms like GDevelop, Construct, and Unity.

Q: What games will my child learn to create in this 18-month program?

A: Kids will build 60+ complete games including: platformers like Mario, puzzle games, racing games, shooting games, RPGs, multiplayer games, mobile games, and even VR experiences. Each project teaches specific skills while keeping learning fun and engaging.

Q: Can kids actually publish and sell games after this course?

A: Yes! By the end of the program, students learn how to publish games to platforms like Google Play Store, iOS App Store, itch.io, and even Steam. Many of our young students already earn pocket money from game ads and sales. We teach game marketing, monetization, and app store optimization.

Q: Is game development a good career path for my child?

A: Absolutely! Beyond game development, students gain programming logic, problem-solving, creativity, and project management skills that transfer to any tech career. Many successful developers at companies like Epic Games, Ubisoft, and indie studios started young.

Q: What equipment does my child need for this game development course?

A: A basic computer or laptop is sufficient to start. For 3D game development modules, we recommend 8GB RAM. A tablet or smartphone is helpful for testing mobile games. All the software we use is free or has free educational versions. Parents can join learning sessions to support younger children.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/game-development-masterclass-for-kids>

=====

COURSE 59 OF 118: GAME DEVELOPMENT FOR TEENS: UNITY, UNREAL & REAL GAMES

=====

Course link: <https://learn.modernagecoders.com/courses/complete-game-development-masterclass-for-teens>

Subject area: Professional Game Development

Designed for: Teens

Who can join: Complete Beginner to Professional Game Developer

Course length: 24 months (104 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Professional Game Developer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Game development course for teens: build games like Minecraft, Fortnite and Among Us in Unity and Unreal, with multiplayer, VR/AR and a portfolio journey.

In one line: From 'I Want to Make Games' to 'I Published My Game on Steam'

ABOUT THE PROGRAMME

This is your complete transformation from gamer to game creator. Learn how Minecraft started as a solo project, how Among Us became viral, and how indie developers compete with billion-dollar studios. This 2-year journey takes you from zero coding experience to publishing your own games on Steam, App Store, and consoles. Designed specifically for teenagers who dream of making games, you'll learn by recreating mechanics from Fortnite, Minecraft, Valorant, Genshin Impact, and more. Master programming, game engines, 3D modeling, level design, multiplayer networking, and the business of games. By graduation, you'll have 30+ games in your portfolio, published titles with real players, and skills that get you hired by studios or accepted into top game design programs.

WHAT MAKES THIS COURSE DIFFERENT

- Teen-focused: Build games teens actually play - battle royales, RPGs, multiplayer
- Zero to published: Start from scratch, end with games on Steam/App stores
- Multi-engine mastery: Unity, Unreal, Godot, and custom engines
- Full game creation: Programming, art, music, design, marketing

- Industry connections: Mentorship from professional game developers
- Portfolio-driven: 30+ games from simple to AAA quality
- Team projects: Collaborate with teen developers worldwide
- Business skills: Learn to monetize and market your games

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Programming, 2D games, game design basics

Phase 2: 3D & Engines (Months 7-12): Unity mastery, 3D games, physics

Phase 3: Advanced (Months 13-18): Unreal Engine, multiplayer, mobile

Phase 4: Professional (Months 19-24): VR/AR, publishing, studio creation

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No prerequisites - complete beginner friendly

Coding experience: Zero programming knowledge required

Equipment: Gaming PC/laptop, internet connection

Time commitment: 15-20 hours per week

English: Basic English understanding

Motivation: Passion for games and creativity

WHO THIS COURSE IS FOR

Students: Teenagers (13-19) who love gaming

Working professionals: Young adults switching to game development

Entrepreneurs: Teens wanting to build game studios

Freelancers: Future indie game developers

Kids: Motivated gamers 12+ with support

Anyone: Anyone dreaming of making games

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & 2D GAMES (MONTHS 1-6, WEEKS 1-26) -----

Start from zero and build your first games. Master programming fundamentals through game creation and understand what makes games fun.

Months 1-2: Programming Fundamentals Through Games (Week 1-8)

Introduction to Game Development

Topics:

- History of games: From Pong to Fortnite
- How games are made: The development pipeline
- Game genres: What makes each unique
- Famous teen game developers who made millions
- Setting up your game development environment
- Installing Unity Hub and Visual Studio
- Introduction to game engines
- Your first game: Pong in 1 hour
- Understanding game loops
- Frames, updates, and time in games
- Version control with Git for games
- Creating your game developer portfolio

Projects built:

- Pong remake
- Breakout/Arkanoid clone
- Simple clicker game
- Text adventure game

Practice: Daily: Play and analyze 3 games - what makes them fun?

C# Programming for Games

Topics:

- Why C# for game development
- Variables: Storing player health, score, lives
- Data types: int, float, bool, string for games
- Game math: Vectors, positions, distances
- Functions: Creating reusable game actions
- Classes: Building game objects
- If statements: Game logic and decisions
- Loops: Spawning enemies, updating objects
- Arrays: Managing multiple enemies/items
- Lists: Dynamic game collections
- Debugging games with console logs
- Common game programming patterns

Projects built:

- Number guessing game
- Rock Paper Scissors with AI
- Simple RPG battle system

- Dice rolling game

Practice: Complete 50 C# coding challenges

2D Game Development in Unity

Topics:

- Unity interface mastery
- GameObjects and components
- Transform: Position, rotation, scale
- Sprites and sprite renderers
- 2D physics: Rigidbody2D and colliders
- Input system: Keyboard and mouse
- Prefabs: Reusable game objects
- Tags and layers
- Scene management
- Unity's coordinate system
- Game camera setup
- Building for different platforms

Projects built:

- Top-down shooter
- Endless runner like Temple Run
- 2D platformer basics
- Asteroids remake

Practice: Build 10 mini-games in Unity

Game Physics & Collision

Topics:

- Physics in games: Gravity, forces, velocity
- Rigidbody2D properties
- Colliders: Box, circle, polygon, edge
- Trigger vs collision detection
- OnCollisionEnter2D and OnTriggerEnter2D
- Physics materials: Bounce and friction
- Joints: Hinge, spring, fixed
- Effectors: Area, platform, point
- Raycasting for line of sight
- Optimizing physics performance
- Creating realistic movement
- Physics-based puzzles

Projects built:

- Angry Birds clone
- Pinball game
- Physics puzzle game
- Rope swing mechanics

Practice: Create 5 physics-based game mechanics

Months 3-4: 2D Game Mechanics & Polish (Week 9-17)

Player Movement & Controls

Topics:

- Character controllers for different genres
- Platformer movement: Jump, double jump, wall jump
- Top-down movement: 8-directional, tank controls
- Side-scroller mechanics
- Mobile touch controls
- Controller/gamepad support
- Input buffering and coyote time
- Movement feel: Acceleration, deceleration
- Camera follow systems
- Parallax scrolling backgrounds
- Screen shake and juice
- Polish that makes movement fun

Projects built:

- Mario-style platformer
- Twin-stick shooter
- Racing game controls
- Fighting game inputs

Practice: Implement 10 different movement systems

Enemy AI & Behaviors

Topics:

- Basic enemy AI patterns
- Patrol and chase behaviors
- State machines for AI
- Pathfinding basics: A* algorithm
- Line of sight detection
- Shooting and projectiles
- Enemy spawn systems
- Difficulty scaling
- Boss fight patterns
- Crowd AI and flocking
- AI debugging and visualization
- Balancing AI difficulty

Projects built:

- Pac-Man with ghost AI
- Tower defense game
- Stealth game with guards
- Boss battle system

Practice: Create 15 different enemy types

Game UI & Menus

Topics:

- Unity UI system (Canvas)
- UI elements: Buttons, text, images, sliders
- Anchoring and responsive UI
- Health bars and HUD elements
- Score and timer displays
- Main menu design
- Pause menu implementation
- Settings and options
- Inventory systems
- Dialog systems
- UI animations and transitions
- Mobile-friendly UI

Projects built:

- RPG inventory system
- Crafting interface
- Dialog tree system
- Complete menu system

Practice: Design UI for 5 different game genres

Animation & Visual Effects

Topics:

- Sprite animation basics
- Unity Animator and state machines
- Animation transitions and parameters
- Blend trees for smooth animation
- Sprite sheets and texture atlases
- Particle systems for effects
- Trail renderers
- Screen effects and post-processing
- Lighting in 2D games
- Shader basics for 2D
- Creating game juice
- Performance optimization

Projects built:

- Fully animated character
- Explosion and magic effects
- Weather system
- Animated cutscene

Practice: Add polish to all previous games

Audio & Music in Games

Topics:

- Importance of game audio
- Unity's audio system
- Playing sounds and music
- 3D spatial audio
- Audio mixers and groups
- Dynamic music systems
- Sound effect creation basics
- Free audio resources
- Audio optimization
- Creating atmosphere with sound
- Audio feedback for actions
- Music synchronization

Projects built:

- Rhythm game prototype
- Dynamic soundtrack system
- 3D audio showcase
- Complete game with audio

Practice: Add sound to all games

Months 5-6: Complete 2D Games & Design (Week 18-26)

Level Design Fundamentals

Topics:

- Principles of level design
- Teaching through level design
- Difficulty curves and pacing
- Environmental storytelling
- Tilemap system in Unity
- Procedural generation basics
- Level editor tools
- Playtesting and iteration
- Secrets and collectibles
- Multiple paths and choices

- Boss arenas and set pieces
- Level design documentation

Projects built:

- 10-level platformer
- Procedural dungeon generator
- Puzzle game levels
- Open world map

Practice: Design 20 unique levels

Game Design Theory

Topics:

- What makes games fun: Flow theory
- Core gameplay loops
- Game mechanics vs dynamics vs aesthetics
- Player motivation and rewards
- Progression systems
- Economy and balance
- Monetization models: F2P, premium, ads
- Game design documentation (GDD)
- Prototyping and iteration
- Player psychology
- Addiction and ethical design
- Analyzing successful games

Projects built:

- Game design document
- Economy balance spreadsheet
- Progression system design
- Monetization strategy

Practice: Analyze and document 10 games

Mobile Game Development

Topics:

- Mobile game market overview
- Touch input and gestures
- Accelerometer and gyroscope
- Mobile UI best practices
- Performance optimization for mobile
- Battery and thermal management
- Android build setup
- iOS build requirements
- Mobile monetization: Ads and IAP
- Analytics integration
- App store optimization (ASO)
- Publishing process

Projects built:

- Endless runner for mobile
- Puzzle game for phones
- Casual mobile game
- Mobile port of PC game

Practice: Optimize games for mobile

Save Systems & Persistence

Topics:

- PlayerPrefs for simple data
- JSON serialization
- Binary save files
- Cloud saves basics
- Profile and progress systems
- Unlockables and achievements
- Leaderboards implementation
- Steam integration basics
- Cross-platform saves
- Save file encryption
- Version compatibility
- Backup and recovery

Projects built:

- Complete save system
- Achievement system
- Online leaderboard
- Profile manager

Practice: Add saves to all games

Phase 1 Capstone Project

Topics:

- Planning a complete game
- Scope management
- Creating a development timeline
- Asset lists and requirements
- Playtesting plan
- Marketing preparation
- Publishing checklist

Projects built:

- CAPSTONE: Complete 2D Game

- Full game with 10+ levels, save system, achievements
- Publish on itch.io or Game Jolt
- Get 100+ players and feedback

Assessment: Phase 1 Final - Publish your first complete game

----- PHASE 2: 3D GAMES & UNITY MASTERY (MONTHS 7-12, WEEKS 27-52) -----

Enter the world of 3D game development. Master Unity's 3D capabilities and create games like Minecraft, Fortnite, and Fall Guys.

Months 7-8: 3D Fundamentals (Week 27-35)

3D Basics in Unity

Topics:

- 3D vs 2D: Understanding the difference
- 3D coordinate systems and space
- 3D primitives: Cubes, spheres, capsules
- Materials and textures
- Basic shaders and rendering
- 3D transforms and rotation
- Cameras in 3D: Perspective vs orthographic
- Lighting basics: Directional, point, spot
- Shadows and quality settings
- 3D physics: Rigidbody and colliders
- Importing 3D models
- ProBuilder for level design

Projects built:

- 3D obstacle course
- Bowling game
- Simple racing game
- 3D puzzle game

Practice: Create 10 3D prototypes

3D Character Controllers

Topics:

- First-person controller setup
- Third-person controller with camera
- Character movement and physics
- Jumping and gravity
- Slopes and stairs
- Moving platforms
- Swimming mechanics
- Flying and gliding
- Vehicle controllers
- Camera collision handling
- Input smoothing
- Controller feel and polish

Projects built:

- FPS controller like Valorant
- Third-person like Fortnite
- Car racing mechanics
- Flight simulator controls

Practice: Build 5 different 3D controllers

3D Animation & Rigging

Topics:

- 3D animation principles
- Humanoid vs generic rigs
- Importing animations from Mixamo
- Animation state machines
- Blend trees for movement
- Root motion
- IK (Inverse Kinematics) basics
- Animation events
- Cutscene creation with Timeline
- Facial animations basics
- Motion capture introduction
- Performance optimization

Projects built:

- Animated player character
- Combat animation system
- Cinematic cutscene
- Dance game with mocap

Practice: Animate 10 different actions

3D Level Design & Environments

Topics:

- 3D level design principles
- Terrain system in Unity
- Terrain painting and sculpting
- Vegetation and trees
- LOD (Level of Detail) systems

- Occlusion culling
- Light baking and probes
- Skyboxes and atmosphere
- Weather effects
- Day/night cycles
- Interior vs exterior design
- Performance optimization

Projects built:

- Open world environment
- Dungeon crawler levels
- Racing track design
- Atmospheric horror level

Practice: Design 5 unique 3D environments

3D Modeling Basics (Blender)

Topics:

- Introduction to Blender
- 3D modeling fundamentals
- Box modeling techniques
- Sculpting basics
- UV unwrapping introduction
- Texturing in Blender
- Low-poly modeling for games
- Optimization for game engines
- Exporting to Unity
- Creating game props
- Character modeling basics
- Animation in Blender

Projects built:

- Low-poly character model
- Game props collection
- Simple vehicle model
- Environment assets

Practice: Model 20 game assets

Months 9-10: Advanced Unity Features (Week 36-44)

Advanced Physics & Mechanics

Topics:

- Complex physics interactions
- Ragdoll physics setup
- Destructible objects
- Soft body simulation
- Cloth and hair physics
- Vehicle physics deep dive
- Floating and buoyancy
- Rope and chain physics
- Physics optimization
- Custom physics solutions
- Networking physics
- Deterministic physics

Projects built:

- Destruction system
- Realistic car game
- Physics sandbox
- Ragdoll combat game

Practice: Create 10 physics systems

Shader Programming

Topics:

- Introduction to shaders
- Shader Graph in Unity
- Vertex and fragment shaders
- Surface shaders
- HLSL basics
- Creating custom materials
- Water shaders
- Toon/cel shading
- Dissolve and fade effects
- Hologram effects
- Outline shaders
- Performance considerations

Projects built:

- Stylized water shader
- Toon shaded game
- Sci-fi hologram effects
- Custom art style game

Practice: Create 15 custom shaders

AI & Pathfinding

Topics:

- Unity NavMesh system
- NavMesh agents setup

- Dynamic obstacles
- Off-mesh links
- Crowd simulation
- Behavior trees introduction
- State machines for complex AI
- GOAP (Goal-Oriented Action Planning)
- Machine learning agents basics
- Enemy squad tactics
- Stealth AI systems
- AI performance optimization

Projects built:

- Zombie survival AI
- Stealth game with smart guards
- RTS unit control
- Battle royale AI

Practice: Build 10 AI behaviors

Procedural Generation

Topics:

- Procedural generation concepts
- Random vs seeded generation
- Perlin noise for terrain
- Dungeon generation algorithms
- Infinite world generation
- Procedural textures
- Procedural meshes
- L-systems for vegetation
- Wave function collapse
- Minecraft-style voxel worlds
- Procedural quests and stories
- Balancing randomness

Projects built:

- Minecraft-style terrain
- Roguelike dungeon generator
- Infinite runner levels
- No Man's Sky planets

Practice: Generate 10 different content types

Optimization & Profiling

Topics:

- Unity Profiler mastery
- Frame rate optimization
- Draw call batching
- Texture atlasing
- Mesh combining
- LOD implementation
- Object pooling
- Coroutine optimization
- Job System and Burst Compiler
- ECS introduction
- Mobile optimization deep dive
- Memory management

Projects built:

- Optimize existing game
- 1000 enemy system
- Mobile performance test
- VR performance optimization

Practice: Optimize all projects to 60 FPS

Months 11-12: Multiplayer & Networking (Week 45-52)

Multiplayer Basics

Topics:

- Networking fundamentals for games
- Client-server vs peer-to-peer
- Unity Netcode for GameObjects
- Setting up multiplayer
- Player spawning and management
- Synchronizing transforms
- RPCs (Remote Procedure Calls)
- Network variables
- Lag compensation
- Client prediction
- Server authority
- Cheating prevention basics

Projects built:

- 2-player Pong online
- Multiplayer racing game
- Co-op platformer
- Simple battle arena

Practice: Add multiplayer to 5 games

Advanced Multiplayer Systems

Topics:

- Mirror Networking
- Photon Fusion/PUN2
- Dedicated server setup
- Matchmaking systems
- Lobby creation
- Room-based games
- Large-scale multiplayer (100+ players)
- Voice chat integration
- Persistent worlds
- Database integration
- Server hosting options
- Cost optimization

Projects built:

- Among Us clone
- Battle royale prototype
- MMO prototype
- Multiplayer FPS

Practice: Build complex multiplayer game

Cross-Platform Development

Topics:

- Building for multiple platforms
- Platform-specific code
- Input abstraction
- Resolution independence
- Console development basics
- Steam Deck optimization
- Nintendo Switch considerations
- Xbox and PlayStation basics
- Cloud gaming (Stadia, GeForce Now)
- WebGL builds
- Platform certification
- Cross-platform multiplayer

Projects built:

- Cross-platform game
- Console-ready game
- WebGL multiplayer game
- Platform showcase

Practice: Port games to 3 platforms

Game Services Integration

Topics:

- Steamworks SDK integration
- Achievements and stats
- Steam Workshop
- Cloud saves
- Leaderboards
- Google Play Games Services
- Apple Game Center
- Epic Online Services
- Discord Rich Presence
- Analytics services
- Crash reporting
- Player behavior tracking

Projects built:

- Steam integration package
- Achievement system
- Global leaderboard
- Analytics dashboard

Practice: Integrate services in games

Phase 2 Capstone Project

Topics:

- Planning a 3D game
- Technical design document
- Art style guide
- Multiplayer architecture
- Testing plan
- Launch strategy

Projects built:

- MAJOR CAPSTONE: 3D Multiplayer Game
- Complete 3D game with multiplayer, achievements, leaderboards
- Publish on Steam (Early Access)
- Get 500+ wishlists

Assessment: Phase 2 Final - Launch game on Steam

PHASE 3: UNREAL ENGINE & ADVANCED SYSTEMS (MONTHS 13-18, WEEKS 53-78)

Master Unreal Engine, advanced graphics, VR/AR development, and AAA game production techniques.

Months 13-14: Unreal Engine Fundamentals (Week 53-61)

Unreal Engine Basics

Topics:

- Why Unreal: Fortnite, Valorant, and AAA games
- Unreal Engine installation and setup
- Editor interface navigation
- Blueprints visual scripting
- Creating your first Blueprint
- Variables and functions in Blueprints
- Event graph and construction script
- Actor classes and components
- Level design in Unreal
- Lighting and atmosphere
- Materials and textures
- Project settings and configuration

Projects built:

- FPS prototype in Unreal
- Racing game conversion
- Puzzle game with Blueprints
- Environment showcase

Practice: Create 10 Blueprint systems

Unreal C++ Programming

Topics:

- C++ in Unreal vs standard C++
- Setting up Visual Studio for Unreal
- UCLASS, UPROPERTY, UFUNCTION macros
- Creating C++ classes
- Combining C++ with Blueprints
- Gameplay framework classes
- Input handling in C++
- Collision and physics in code
- Debugging Unreal C++
- Memory management in Unreal
- Hot reload and Live Coding
- Best practices

Projects built:

- C++ character controller
- Weapon system in C++
- Inventory system
- Custom game mode

Practice: Convert Blueprints to C++

Unreal Animation System

Topics:

- Animation blueprints
- State machines in Unreal
- Blend spaces
- Animation montages
- Animation notifies
- IK and procedural animation
- Control Rig basics
- Sequencer for cinematics
- Facial animation
- Motion matching introduction
- Animation optimization
- Retargeting animations

Projects built:

- Advanced character animation
- Combat system with montages
- Cinematic sequence
- Procedural animation system

Practice: Create complex animation systems

Unreal Graphics & Rendering

Topics:

- Material editor mastery
- PBR (Physically Based Rendering)
- Material instances and parameters
- Landscape materials
- Vertex painting
- Post-processing volumes
- Niagara particle system
- Ray tracing basics
- Lumen global illumination
- Nanite virtualized geometry
- DLSS and temporal upsampling
- Performance profiling

Projects built:

- Realistic environment
- Stylized art game

- Particle effects showcase
- Ray-traced scene

Practice: Master Unreal's graphics features

Unreal Multiplayer Systems

Topics:

- Replication in Unreal
- Network roles and authority
- RPCs in Unreal
- Replicated variables
- Movement replication
- Session management
- Online subsystems
- Dedicated servers in Unreal
- Steam integration with Unreal
- EOS (Epic Online Services)
- Lag compensation in Unreal
- Testing multiplayer locally

Projects built:

- Multiplayer FPS in Unreal
- Battle royale prototype
- Co-op survival game
- Racing game online

Practice: Build networked game systems

Months 15-16: VR/AR Development (Week 62-70)

Virtual Reality Development

Topics:

- VR fundamentals and hardware
- Setting up VR in Unity
- XR Interaction Toolkit
- VR locomotion methods
- Teleportation and smooth movement
- Hand tracking and gestures
- Grabbing and interactions
- UI in VR
- Comfort and motion sickness
- Oculus/Meta Quest development
- PC VR considerations
- Performance optimization for VR

Projects built:

- VR escape room
- Beat Saber clone
- VR training simulator
- Social VR space

Practice: Create 5 VR experiences

Augmented Reality Development

Topics:

- AR fundamentals and applications
- AR Foundation in Unity
- ARCore and ARKit
- Plane detection and tracking
- Image tracking
- Face tracking and filters
- Object placement in AR
- Occlusion handling
- Light estimation
- AR multiplayer basics
- Location-based AR
- WebAR introduction

Projects built:

- Pokemon GO style game
- AR furniture placement
- Face filter app
- AR treasure hunt

Practice: Build 5 AR applications

Mixed Reality & Future Tech

Topics:

- Mixed reality concepts
- HoloLens development basics
- Passthrough AR on Quest
- Hand tracking deep dive
- Eye tracking
- Voice commands in XR
- Haptic feedback
- Full body tracking
- Social presence in VR
- Persistent multiplayer worlds
- Cloud streaming for XR
- Future of XR gaming

Projects built:

- Mixed reality game
- Hand tracking interface
- Social VR application
- XR fitness game

Practice: Experiment with cutting-edge XR

Advanced AI & Machine Learning

Topics:

- ML-Agents in Unity
- Training game AI with reinforcement learning
- Imitation learning
- Curriculum learning
- Self-play training
- Neural networks in games
- Computer vision for games
- Natural language processing
- Procedural content with ML
- Player behavior prediction
- Dynamic difficulty adjustment
- AI-generated content

Projects built:

- Self-learning AI opponent
- Procedural level generator with ML
- Player behavior analyzer
- AI-driven game director

Practice: Train 5 different AI agents

Game Audio Advanced

Topics:

- Interactive music systems
- FMOD and Wwise integration
- Dynamic audio mixing
- Procedural audio generation
- 3D spatial audio advanced
- Ambisonic audio
- Audio occlusion and reverb zones
- Voice acting integration
- Lip sync systems
- Audio streaming
- Compression and formats
- Platform-specific audio

Projects built:

- Dynamic music system
- 3D audio horror game
- Rhythm game with FMOD
- Voice-acted narrative game

Practice: Create professional audio systems

Months 17-18: Professional Production (Week 71-78)

Game Production Pipeline

Topics:

- Game production phases
- Pre-production planning
- Vertical slice development
- Agile/Scrum for game development
- Task tracking with Jira/Trello
- Version control with Perforce
- Build automation
- QA and testing processes
- Bug tracking and triage
- Milestone planning
- Risk management
- Team communication

Projects built:

- Production plan document
- Automated build pipeline
- QA test plan
- Project management setup

Practice: Manage a team project

Tools & Editor Development

Topics:

- Unity Editor scripting
- Custom inspectors and windows
- ScriptableObjects for tools
- Property drawers
- Gizmos and handles
- Build pipeline customization
- Asset processing
- Batch operations
- Level design tools

- Debugging tools
- Performance profiling tools
- Artist-friendly tools

Projects built:

- Level editor tool
- Batch asset processor
- Custom debug console
- Artist workflow tools

Practice: Build 10 editor tools

Platform-Specific Development

Topics:

- Console development overview
- PlayStation development basics
- Xbox development program
- Nintendo Switch considerations
- Platform holder requirements
- Certification processes (TCR/TRC)
- Achievement implementation
- Platform-specific features
- Controller haptics
- Adaptive triggers
- Platform optimization
- Submission process

Projects built:

- Console-ready game
- Platform feature showcase
- Certification checklist
- Multi-platform build

Practice: Prepare for console certification

Live Service & Post-Launch

Topics:

- Games as a Service (GaaS)
- Content update planning
- Season pass systems
- Battle pass implementation
- Live events
- Player retention strategies
- Community management
- Player feedback integration
- Hotfix deployment
- A/B testing in games
- Analytics-driven decisions
- Long-term monetization

Projects built:

- Live service game design
- Season content plan
- Event system implementation
- Analytics dashboard

Practice: Plan 6-month content roadmap

Phase 3 Capstone Project

Topics:

- AAA game planning
- Technical requirements
- Art production pipeline
- Marketing strategy
- Platform selection
- Team formation

Projects built:

- MAJOR CAPSTONE: AAA-Quality Game
- Unreal Engine game with advanced features
- VR/AR game option
- Target console/PC release

Assessment: Phase 3 Final - Pre-production AAA game

----- PHASE 4: STUDIO CREATION & BUSINESS (MONTHS 19-24, WEEKS 79-104) -----

Build your game studio, master the business of games, and launch commercially successful titles.

Months 19-20: Game Business & Marketing (Week 79-87)

Game Business Fundamentals

Topics:

- Game industry overview
- Publisher vs self-publishing
- Funding options: Investors, publishers, crowdfunding
- Business models: Premium, F2P, subscription
- Revenue sharing and royalties
- Legal basics: Contracts, IP, trademarks

- Company formation
- Team building and roles
- Budgeting and financial planning
- Insurance and liability
- Tax considerations
- International business

Projects built:

- Business plan creation
- Pitch deck development
- Financial projections
- Legal document templates

Practice: Create complete business plan

Game Marketing & PR

Topics:

- Marketing strategy development
- Brand identity creation
- Social media marketing
- Content marketing
- Influencer partnerships
- Press kit creation
- Media outreach
- Game conventions and events
- Steam page optimization
- Trailer production
- Screenshot strategies
- Community building

Projects built:

- Marketing campaign plan
- Press kit creation
- Trailer production
- Social media strategy

Practice: Market your games professionally

Monetization Strategies

Topics:

- Monetization models deep dive
- Free-to-play economics
- In-app purchases design
- Cosmetic monetization
- Battle pass systems
- Loot boxes and regulations
- Ad monetization
- DLC and expansion planning
- Pricing strategies
- Regional pricing
- Sale strategies
- Retention and LTV optimization

Projects built:

- F2P economy design
- Monetization system implementation
- A/B testing framework
- Revenue optimization plan

Practice: Design monetization for 5 games

Publishing & Distribution

Topics:

- Self-publishing process
- Steam Direct and requirements
- Epic Games Store
- App Store and Google Play
- Console publishing
- GOG and other platforms
- Regional distribution
- Age ratings (ESRB, PEGI)
- Localization planning
- Release timing
- Early Access strategy
- Post-launch support

Projects built:

- Publishing checklist
- Multi-platform release plan
- Localization setup
- Post-launch roadmap

Practice: Publish on multiple platforms

Analytics & Metrics

Topics:

- KPIs for games
- Player analytics setup
- Retention metrics
- Monetization metrics
- Funnel analysis

- Player segmentation
- Predictive analytics
- A/B testing implementation
- Heat maps and player tracking
- Performance monitoring
- Crash analytics
- Competitive analysis

Projects built:

- Analytics implementation
- Custom dashboard creation
- Player behavior analysis
- Performance reports

Practice: Analyze and optimize games

Months 21-22: Advanced Specializations (Week 88-96)

Esports & Competitive Gaming

Topics:

- Esports ecosystem understanding
- Designing for competitive play
- Balance and meta considerations
- Spectator features
- Tournament systems
- Ranking and matchmaking
- Anti-cheat systems
- Replay systems
- Observer mode
- Streaming integration
- Esports monetization
- Community tournaments

Projects built:

- Competitive game mode
- Tournament system
- Spectator mode
- Ranking system

Practice: Create esports-ready game

AI in Game Development

Topics:

- Game AI is mostly rules: state machines and behaviour trees
- Pathfinding and navigation meshes
- Utility AI: choosing the least-bad action
- Procedural generation with constraints
- Dynamic difficulty without cheating the player
- LLM-driven NPC dialogue, and why it breaks
- Keeping a generative NPC on-script and safe
- AI-assisted asset creation and its licensing limits
- Using AI to playtest and find broken levels
- Performance: AI budget per frame
- Ethics: manipulation, addiction and dark patterns
- Crediting AI-generated work honestly

Projects built:

- Behaviour-tree enemy AI
- Procedural level generator
- LLM NPC with guardrails
- AI playtest harness

Practice: Build 3 AI-driven game systems

Cloud Gaming & Streaming

Topics:

- Cloud gaming platforms
- Streaming technology basics
- Input latency mitigation
- Adaptive streaming
- Platform requirements
- Optimization for streaming
- Cross-progression systems
- Cloud save implementation
- Multi-device play
- 5G and mobile streaming
- Future of cloud gaming
- Business models

Projects built:

- Cloud-optimized game
- Streaming setup
- Cross-progression system
- Multi-device game

Practice: Optimize for cloud platforms

Game Narrative & Writing

Topics:

- Interactive storytelling
- Branching narratives

- Character development
- World building
- Dialog writing
- Environmental storytelling
- Cuts scene planning
- Voice acting direction
- Localization considerations
- Narrative tools (Yarn, ink)
- Player choice systems
- Emotional design

Projects built:

- Narrative-driven game
- Branching story system
- Character backstories
- World bible creation

Practice: Write game narratives

Accessibility & Inclusion

Topics:

- Accessibility importance
- Visual accessibility
- Audio accessibility
- Motor accessibility
- Cognitive accessibility
- Colorblind modes
- Subtitle systems
- Control remapping
- Difficulty options
- Inclusive character design
- Cultural sensitivity
- Testing with disabled gamers

Projects built:

- Accessibility audit
- Accessibility features implementation
- Inclusive game design
- Testing documentation

Practice: Make all games accessible

Month 23: Final Production (Week 97-100)

Studio Portfolio Development

Topics:

- Studio branding
- Portfolio website creation
- Demo reel production
- Case study writing
- Team member profiles
- Project showcases
- Awards and recognition
- Press coverage archive
- Community testimonials
- Partnership highlights
- Future roadmap
- Investor materials

Projects built:

- Studio website
- Portfolio showcase
- Demo reels
- Press materials

Practice: Build professional studio presence

Final Project Pre-Production

Topics:

- Concept development
- Market research
- Competitive analysis
- Target audience definition
- Unique selling points
- Scope definition
- Technical requirements
- Art direction
- Audio direction
- Production timeline
- Budget planning
- Risk assessment

Projects built:

- Game design document
- Technical design document
- Art bible
- Production plan

Practice: Complete pre-production

Final Project Production

Topics:

- Sprint planning
- Asset creation pipeline
- Core mechanics implementation
- Level design execution
- Polish and juice
- QA testing rounds
- Performance optimization
- Platform testing
- Localization implementation
- Achievement integration
- Final bug fixes
- Gold master preparation

Projects built:

- Final game production
- Alpha build
- Beta build
- Release candidate

Practice: Complete production phase

Launch Preparation

Topics:

- Launch strategy finalization
- Marketing campaign execution
- Press embargo setup
- Influencer outreach
- Community events planning
- Launch trailer release
- Store page finalization
- Day-one patch preparation
- Customer support setup
- Post-launch content plan
- Success metrics definition
- Celebration planning

Projects built:

- Launch campaign
- Press outreach
- Community events
- Support documentation

Practice: Execute launch plan

Month 24: Launch & Graduation (Week 101-104)

Game Launch & Release

Topics:

- Launch day execution
- Community management
- Review monitoring
- Bug tracking and hotfixes
- Player feedback integration
- Social media engagement
- Streamer and content creator support
- Live monitoring
- Emergency response procedures
- First week optimizations
- Sales tracking
- Post-mortem preparation

Projects built:

- FINAL GAME LAUNCH
- Live on Steam/App Stores
- Marketing execution
- Community management

Deliverables:

- Published commercial game
- 1000+ downloads/sales
- Press coverage
- Player reviews

Career Preparation

Topics:

- Portfolio finalization
- Resume optimization
- LinkedIn profile enhancement
- Demo reel creation
- Interview preparation
- Studio applications
- Networking strategies
- Conference attendance
- Freelance opportunities
- Indie development path
- Further education options
- Specialization choices

Projects built:

- Professional portfolio
- Industry applications
- Networking plan
- Career roadmap

Deliverables:

- 30+ game portfolio
- Published titles
- Industry connections
- Job applications

Graduation & Future

Topics:

- Final presentations
- Portfolio showcase event
- Industry panel feedback
- Peer recognition
- Success story sharing
- Alumni network induction
- Continued learning resources
- Mentorship program
- Studio incubation options
- Investment opportunities
- Future collaborations
- Lifetime community access

Deliverables:

- Professional Game Developer Certificate
- Unity Certified Developer
- Unreal Engine Certification
- Published Game Portfolio
- Industry Recommendations
- Alumni Network Access
- Lifetime Course Updates

Assessment: FINAL SHOWCASE: Present to industry professionals, investors, and game studios

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 15+ 2D games, mobile games, published on itch.io
- Phase 2: 10+ 3D games, multiplayer games, Steam release
- Phase 3: 5+ Unreal games, VR/AR experiences, console-ready
- Phase 4: Commercial releases, studio portfolio, business setup
- Total: 30+ complete games from casual to AAA quality

Total projects: 30+ complete games, 5+ published titles, 1 commercial studio

SKILLS THE STUDENT WILL HAVE AT THE END

- Engines: Unity, Unreal Engine, Godot, custom engines
- Programming: C#, C++, Blueprints, visual scripting
- Graphics: 2D/3D art, shaders, VFX, lighting, materials
- Design: Level design, game balance, UX, monetization
- Audio: Music, SFX, voice acting, spatial audio
- Multiplayer: Networking, servers, matchmaking, backend
- Platforms: PC, mobile, console, VR/AR, web
- Business: Marketing, publishing, analytics, studio management
- Tools: Blender, Photoshop, Git, Jira, analytics platforms
- Specializations: AI, procedural generation, optimization

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-6 hours
Hands on development: 8-10 hours
Projects: 4-5 hours
Practice challenges: 2-3 hours
Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly masterclasses with industry professionals
Mentorship: 1-on-1 guidance from game developers
Community: Global community of teen game developers
Code review: Professional feedback on all projects
Playtesting: Community playtesting for all games
Publishing support: Help with Steam/App Store publishing
Studio incubation: Support for promising studios
Lifetime access: All content and future updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each 6-month phase
Engine certifications: Unity Certified User/Programmer
Unreal certification: Unreal Engine Developer
Final certificate: Professional Game Developer Certification
Specialized certs: VR Developer, Multiplayer Specialist
Portfolio validation: Industry-reviewed portfolio

WHERE THIS CAN LEAD

- Game Programmer (Unity/Unreal)
- Game Designer
- Level Designer
- Technical Artist
- Gameplay Programmer
- Network Programmer
- Tools Programmer
- Indie Game Developer
- Studio Founder/CEO
- VR/AR Developer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the game development course cover?

A: Across 24 months teens go from programming fundamentals and 2D games, through full Unity mastery with 3D, physics and multiplayer networking, into Unreal Engine, VR/AR and professional production, finishing with game business skills and a publish-ready portfolio. The curriculum mirrors how real studios build games.

Q: Does my teen need coding experience to start?

A: No, the course starts from zero and teaches programming through building playable games from the first weeks. What matters more is genuine interest in games and the patience to build, test and improve.

Q: What computer does my teen need?

A: A reasonably capable PC or laptop that runs Unity smoothly; a stronger machine helps for the later Unreal Engine and VR phases but is not needed on day one. We check your setup at the free demo before you commit to anything.

Q: How is progress measured?

A: In finished games: each phase ends with playable projects, from 2D arcade games to 3D and multiplayer builds, and the mentor tracks the growing portfolio. By the later phases students are polishing games to publishing standard, the same benchmark the industry uses.

Q: Unity or Unreal, which engine will they learn?

A: Both, in sequence: Unity first because it is the friendliest serious engine and dominates indie and mobile development, then Unreal Engine for high-end 3D. Teens graduate able to choose the right engine for a project.

Q: Is there a trial class?

A: Yes, a free demo class with no commitment, booked at learn.modernagecoders.com/contact; your teen will build or modify something playable in that first hour. WhatsApp +91 91233 66161 if you would like to talk first.

Q: How are classes delivered?

A: Every class is a live one hour online session with a real teacher, never pre-recorded tutorials. Small groups mean the teacher watches your teen's actual project and debugs their specific problems, which recorded courses simply cannot do.

Q: What does my teen have at the end?

A: A Game Developer certification plus the thing studios actually ask for: a portfolio of finished, playable games across 2D, 3D and multiplayer. Parents rate our live teaching 4.9 stars across 547+ Google reviews.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-game-development-masterclass-for-teens>

=====

COURSE 60 OF 118: GCSE MATHS: FOUNDATION & HIGHER (9-1), AQA, EDEXCEL, OCR

=====

Course link: <https://learn.modernagecoders.com/courses/gcse-mathematics-mastery>

Subject area: Exam Board Mathematics

Designed for: High school

Who can join: Years 9-11 GCSE students, Foundation and Higher tiers, plus resit candidates

Course length: 18 months (72 weeks)

Time needed: 2 live one-hour classes per week + weekly past-paper work
Certificate: Course completion certificate + documented paper-score progression

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

GCSE Maths course for AQA, Edexcel and OCR: Foundation and Higher tiers, non-calculator Paper 1 fluency, grade 7-9 reach and past-paper technique.

In one line: Foundation to grade 5 secure, Higher to grade 9, taught for understanding, trained for the papers.

ABOUT THE PROGRAMME

An 18-month live program for GCSE mathematics across AQA, Edexcel and OCR, at Foundation or Higher tier. GCSE maths is three papers, one strictly non-calculator, with grades that gatekeep sixth form, apprenticeships and A-Level maths. This course teaches the syllabus understanding-first (so the mock slide of year 11 never happens), trains bare-hands fluency for Paper 1 in every class, brings the Higher-tier grade 7-9 topics in early enough to settle, and converts it all into marks through past-paper cycles against your exact board's mark schemes.

WHAT MAKES THIS COURSE DIFFERENT

- Board-exact: AQA, Edexcel or OCR confirmed first, and every past paper comes from your board
- Tier-honest: evidence for Foundation-vs-Higher decisions, and a real runway from grade 4-risk to 6-secure, or 6 to 9
- Non-calculator Paper 1 fluency drilled in every class
- The grade 7-9 separator topics taught months early so they settle instead of being crammed
- Mark-scheme literacy: method marks, working discipline and the plus-minus habits that stop point leaks
- Understanding-first teaching: algebra as a language, so re-framed exam questions do not break the student
- One dedicated mentor across the whole GCSE run, a full interactive hour, twice a week
- Resit candidates welcome, with a compressed evidence-led plan

THE LEARNING PATH, STEP BY STEP

Phase 1: The Foundation core built properly: number fluency, algebra as a language, and the geometry and data basics

Phase 2: The Higher reach with settling time: quadratics, functions, trigonometry, vectors and the grade 7-9 separator topics

Phase 3: Past-paper craft by board: all three papers, timing plans, mark banking and full cycles into the exam series

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Years 9-11 GCSE students (AQA, Edexcel or OCR), or post-16 resit candidates
Math background: Any, the diagnostic finds the real base; Foundation gaps are repaired before Higher reach
Equipment: Computer with stable internet; a scientific calculator for Papers 2-3
Time commitment: 2 live hours plus 1-2 practice hours weekly
English: Course taught in English
Motivation: An exam series on the calendar; we supply the method

WHO THIS COURSE IS FOR

Higher aimers: Students who need the move from Foundation-risk to Higher-comfortable made real
Grade 9 hunters: Grade 6-7 students converting understanding into 8-9 through separator topics and paper craft
Mock sliders: Students whose grades sink each mock as memorized methods run out
Paper 1 worriers: Calculator-raised students facing the non-calculator paper
Resitters: Post-16 candidates who need the threshold cleared with a method this time

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: THE CORE, BUILT PROPERLY (MONTHS 1-6) -----

Foundation-tier territory owned deeply is what Higher-tier grades stand on. This phase rebuilds number fluency without a calculator, teaches algebra as a language, and secures the geometry and statistics basics, with the diagnostic finding the true starting point first.

Months 1-2: Diagnostic + Number, Bare-Handed (Weeks 1-8)

Diagnostic, Board and Tier Fit

Topics:

- Exam board confirmed: AQA, Edexcel or OCR
- Foundation versus Higher tier fit assessed honestly
- Non-calculator fluency screen
- Algebra base measured
- Number and ratio accuracy audit
- Honest starting-grade picture

Practice: A mixed non-calculator and algebra diagnostic set sized to the suspected tier.

Assessment: Baseline grade estimate with a written tier recommendation and a topic-priority map.

Detail: Full diagnostic: board confirmed, tier fit assessed honestly, non-calculator fluency and algebra base measured

Fractions, Decimals and Percentages to Fluency

Topics:

- The four operations with fractions
- Decimal calculations without a calculator
- Percentage of, increase, decrease and reverse
- Converting between the three forms
- Estimation and rounding as a check
- Paper 1 non-calculator territory

Practice: A non-calculator FDP set with estimation checks on every answer.

Detail: Fractions, decimals and percentages to fluency, Paper 1's exact territory, with estimation as a checking habit

Ratio and Proportion Reasoning

Topics:

- Sharing in a given ratio
- Best-buy and value comparisons
- Recipe scaling problems
- Map-scale and unit conversions
- Direct proportion reasoning
- Linking ratio to fractions

Practice: Ratio and proportion questions across the best-buy, recipe and map-scale families.

Detail: Ratio and proportion reasoning: the best-buy, recipe and map-scale families

Indices, Standard Form and Bounds

Topics:

- Index laws with integer and negative powers
- Standard form and its arithmetic
- Upper and lower bounds
- Error intervals and truncation
- Rounding to significant figures
- Bounds applied in calculations

Practice: An indices, standard-form and bounds set including error-interval questions.

Detail: Indices, standard form and bounds; error intervals done with meaning

Months 3-4: Algebra as a Language (Weeks 9-16)

Expressions, Expansion and Factorisation

Topics:

- Simplifying and collecting like terms
- Expanding single and double brackets
- The area model behind expansion
- Factorising into brackets
- Common-factor extraction
- Algebraic notation discipline

Practice: An expand-and-factorise set anchored by area-model diagrams.

Detail: Expressions, expansion and factorisation with the area model underneath

Linear Equations, Inequalities and Formulae

Topics:

- Solving linear equations step by step
- Equations with brackets and fractions
- Solving and representing inequalities
- Number-line representation of solutions
- Rearranging formulae to change the subject
- Reading an equation as a sentence

Practice: Linear equation, inequality and rearrangement questions with worded contexts.

Detail: Linear equations and inequalities read as sentences; rearranging formulae

Straight-Line Graphs and $y = mx + c$

Topics:

- Plotting straight-line graphs
- Gradient understood as a rate
- $y = mx + c$ as intercept and slope
- Finding the equation of a line
- Parallel and intersecting lines
- Reading real-life graphs critically

Practice: Straight-line graph questions plus interpretation of a real-life rate graph.

Detail: Straight-line graphs: gradient as rate, $y = mx + c$ as a story; real-life graphs read critically

Simultaneous Equations and Sequences

Topics:

- Simultaneous equations by elimination
- Simultaneous equations by substitution
- Choosing a method by structure
- Linear sequences and the n th term
- Finding and using the n th term rule
- Special sequences and patterns

Practice: A simultaneous-equations and n th-term set graded by structure.

Detail: Simultaneous equations by structure; sequences and the n th term

Months 5-6: Shape and Data Foundations (Weeks 17-24)

Angle Reasoning and Justification

Topics:

- Angles on lines and around points
- Parallel-line angle rules
- Interior and exterior angles of polygons
- Bearings and direction problems
- Stating a reason for every step
- Multi-step angle chains

Practice: Angle problems requiring named reasons, including bearings.

Detail: Angle reasoning with justification language: parallel lines, polygons, bearings

Area, Perimeter, Volume and Pythagoras

Topics:

- Area and perimeter of standard shapes
- Compound and composite shapes
- Volume and surface area of solids
- Pythagoras derived from a picture
- Applying Pythagoras in context
- Units for length, area and volume

Practice: A mensuration set combining compound shapes and Pythagoras problems.

Detail: Area, perimeter and volume including compound shapes; Pythagoras from a picture

Averages, Charts and Correlation

Topics:

- Mean, median, mode and range
- Averages from frequency tables
- Bar charts, pie charts and pictograms
- Scatter graphs and lines of best fit
- Correlation and its limits
- Reading data critically

Practice: A statistics set interpreting charts and a scatter graph with commentary.

Detail: Averages, charts, scatter graphs and correlation, data read like a sceptic

Phase 1 Checkpoint Paper

Topics:

- Full Foundation-standard paper under timing
- Mark-scheme review of every question
- Method-mark awareness introduced
- Error log started from the checkpoint
- Timing across a full paper
- Presentation and working discipline

Practice: A complete Foundation-standard paper sat to time, then self-marked.

Assessment: Foundation-standard checkpoint paper marked against the real mark scheme, opening the error log.

Detail: Phase checkpoint: a full Foundation-standard paper under timing, reviewed against the mark scheme

PHASE 2: THE HIGHER REACH, WITH SETTling TIME (MONTHS 7-12)

The topics that separate grade 5 from grade 9, taught months before the exam so they are owned rather than crammed: quadratics and functions, trigonometry beyond right angles, vectors, circle theorems and the algebraic craft the top questions assume.

Months 7-8: Quadratics and Functions (Weeks 25-32)

Solving Quadratics Three Ways

Topics:

- Solving by factorising
- Completing the square
- The quadratic formula
- Choosing the method by structure
- Roots and their meaning
- Higher-tier quadratic manipulation

Practice: A quadratics set requiring choice of factorising, completing the square or the formula.

Detail: Quadratics by factorising, completing the square and the formula, chosen by structure

Quadratic Graphs and Turning Points

Topics:

- Sketching quadratic graphs

- Turning points by completing the square
- Roots, intercepts and symmetry
- Quadratics in real contexts
- Quadratic inequalities (Higher)
- Reading a parabola for information

Practice: Quadratic-graph questions including a Higher-tier quadratic inequality.

Detail: Quadratic graphs, turning points and real contexts; quadratic inequalities (Higher)

Functions: Composite and Inverse

Topics:

- Function notation and evaluation
- Composite functions and order
- Inverse functions and how to find them
- Domain and range awareness
- Common function-mark traps
- Functions read as meaning

Practice: A functions set focused on composite and inverse questions.

Detail: Function notation, composite and inverse functions, where half the function marks are lost

Graph Transformations and Algebraic Fractions

Topics:

- Translations and reflections of graphs
- Cubic and reciprocal graph shapes
- Recognising graphs from equations
- Simplifying algebraic fractions (Higher)
- Adding and subtracting algebraic fractions
- Solving equations with algebraic fractions

Practice: Graph-transformation questions plus a Higher-tier algebraic-fractions set.

Detail: Graph transformations, cubic and reciprocal graphs; algebraic fractions (Higher)

Months 9-10: Trigonometry, Vectors and Circle Theorems (Weeks 33-40)

Right-Triangle Trigonometry and Exact Values

Topics:

- Sine, cosine and tangent ratios
- Finding sides and angles in right triangles
- Choosing the correct ratio
- Exact trig values for Paper 1
- Trig in worded and 2D contexts
- Non-calculator trig fluency

Practice: Right-triangle trig questions including exact-value non-calculator items.

Detail: Right-triangle trig consolidated; exact trig values for Paper 1

Sine and Cosine Rules

Topics:

- The sine rule for sides and angles
- The cosine rule for sides and angles
- Area of a triangle with the sine formula
- Bearings and non-right-triangle problems
- Trig graphs and their features (Higher)
- Choosing the correct rule

Practice: A non-right-triangle set applying sine and cosine rules with bearings.

Detail: Sine and cosine rules with bearings and area applications; trig graphs (Higher)

Vectors and Geometric Proof

Topics:

- Vector notation and column vectors
- Adding and scaling vectors
- Position vectors and paths
- The classic geometric vector proof
- Proving points collinear or parallel
- Structuring a vector argument (Higher)

Practice: Vector questions building to a full geometric vector proof.

Detail: Vectors and the classic geometric vector proof, defanged (Higher)

Circle Theorems

Topics:

- The main circle theorems
- Angle at the centre and at the circumference
- Cyclic quadrilaterals and tangents
- The alternate segment theorem
- Proof-quality justification language
- Multi-theorem angle chains

Practice: Circle-theorem problems requiring a named theorem for each step.

Detail: Circle theorems with proof-quality justification

Months 11-12: Probability, Proportion and the Grade 8-9 Craft (Weeks 41-48)

Probability, Trees and Venn Diagrams

Topics:

- Probability of single and combined events
- Tree diagrams for dependent events
- Venn diagrams and set notation

- Conditional probability by reasoning
- With and without replacement
- Avoiding formula-panic on conditionals

Practice: A probability set using tree and Venn diagrams, including a conditional question.

Detail: Probability with tree and Venn diagrams; conditional probability without formula-panic

Proportion and Rates of Change

Topics:

- Direct proportion as an equation
- Inverse proportion as an equation
- Finding the constant of proportionality
- Rates of change from graphs
- Gradient of a curve by tangent (Higher)
- Pre-calculus gradient ideas

Practice: Proportion questions written as equations plus a rate-of-change graph task.

Detail: Direct and inverse proportion as equations; rates of change and pre-calculus gradient ideas (Higher)

Histograms and Cumulative Frequency

Topics:

- Histograms with unequal class widths
- Frequency density
- Cumulative frequency curves
- Median, quartiles and interquartile range
- Box plots and comparing distributions
- Interpreting data like a statistician

Practice: A set drawing and reading histograms, cumulative frequency curves and box plots.

Detail: Histograms, cumulative frequency and comparing distributions like a statistician

Phase 2 Higher-Tier Checkpoint

Topics:

- Full Higher-tier past-paper cycle
- Mark-scheme review of long questions
- Method-mark harvesting practice
- Error-log update from the cycle
- Timing across a Higher paper
- Weak-topic list refreshed

Practice: A full Higher-tier past paper sat to time and reviewed against the mark scheme.

Assessment: Higher-tier past-paper cycle marked to the mark scheme, updating the error log and weak-topic list.

Detail: Phase checkpoint: full Higher-tier past-paper cycle with mark-scheme review and error-log update

----- PHASE 3: PAPER CRAFT TO THE EXAM SERIES (MONTHS 13-18) -----

Three papers, one strictly non-calculator, sat over exam season: this phase turns understanding into grades with board-exact past-paper cycles, personal timing plans, mark-banking discipline and a peak plan that meets the series calmly.

Months 13-14: The Three-Paper System (Weeks 49-56)

Paper 1 Craft, Non-Calculator

Topics:

- Non-calculator discipline under time
- Exact values for surds and trig
- Clean written working at speed
- Common Paper 1 question types
- Banking marks on short questions
- Order of attack on Paper 1

Practice: Timed Paper 1 sections focused on non-calculator accuracy and clean working.

Detail: Paper 1 craft: non-calculator discipline, exact values and clean written working at speed

Papers 2 and 3 Calculator Craft

Topics:

- Calculator use that saves time
- Multi-step problem chains
- Method-mark harvesting on hard questions
- Efficient calculator functions
- Presenting working on calculator papers
- Recovering marks when stuck

Practice: Timed Paper 2 and 3 sections practising method-mark harvesting on unfinished problems.

Detail: Papers 2-3 craft: calculator use that helps, multi-step chains, and method-mark harvesting on problems you cannot finish

Months 15-16: Full Cycles by Board (Weeks 57-64)

Full Board-Exact Paper Cycles

Topics:

- Full timed papers from the exact board
- Tier-correct paper selection
- A review ritual after every paper
- Naming why each mark was lost
- A named fix for each error
- Board-specific question styles

Practice: Complete board-exact papers sat to time, each followed by a why-and-fix review.

Detail: Full timed papers from your exact board and tier; review rituals, why each mark was lost, and the named fix

Timing Plans and Weak-Topic Rotations

Topics:

- Personal timing plans from paper data
- Per-question time budgets
- Weakest-topic rotations by error-log priority
- Grade-boundary awareness used honestly
- Targeting the highest-value gaps
- Consistency across papers

Practice: Weak-topic rotation sets chosen by error-log priority, timed to the personal plan.

Detail: Timing plans from personal data; weakest-topic rotations by error-log priority; grade-boundary awareness used honestly

Months 17-18: The Series, Met Calmly (Weeks 65-72)

Final Cycles and Error-Log Closure

Topics:

- Final full paper cycles
- Documented score progression
- Error-log closure
- Targeted revision to the mocks calendar
- Sealing recurring mistakes
- Confidence on previously weak topics

Practice: Final paper cycles with progression charted and each closed error re-tested.

Detail: Final cycles with documented progression; error-log closure; targeted revision that respects the mocks calendar

Peak Plan and Exam-Day Routines

Topics:

- Light consolidation in the peak week
- Rest discipline before the series
- Exam-day routines rehearsed
- Routines across all three papers
- Calm pacing under real conditions
- The series met as a known place

Practice: Light consolidation tasks and a rehearsed exam-day routine across the three papers.

Assessment: Final full-series rehearsal across all three papers with documented progression before the exam.

Detail: Peak plan: light consolidations, rest discipline, and exam-day routines rehearsed across all three papers

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal error log and syllabus map across all 18 months
- Non-calculator fluency system built through daily-dose drills
- Self-graded past-paper portfolio against your board's real mark schemes
- Personal timing plans for all three papers
- Full timed cycles with documented score progression

Total projects: 5+ exam artifacts plus the student's own handbooks

SKILLS THE STUDENT WILL HAVE AT THE END

- Calculator-free number fluency
- Algebra as a language: equations, formulae, graphs
- Higher-tier functions, trigonometry and vectors
- Circle theorems and geometric justification
- Probability and statistics at exam depth
- Proportion and rates-of-change craft
- Mark-scheme literacy and working discipline
- Three-paper timing strategy

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor

Past paper work: Weekly question sets from your exact board

Fluency: Non-calculator warm-up in every class

Total per week: About 4 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole GCSE run
Recordings: Every class recorded for revision
Progress tracking: Paper-score progression shared with parents
Flexibility: Intensity adjusts around mocks and the exam series

CERTIFICATION DETAIL

Final certificate: Course completion certificate from Modern Age Coders
Score record: Documented paper-score progression
Portfolio projects: Past-paper portfolio and personal handbooks

WHERE THIS CAN LEAD

- A GCSE grade that opens sixth-form and apprenticeship doors
- A-Level Mathematics readiness for Higher-tier students
- The numeracy base every career pathway quietly requires
- Ready for IB or A-Level tracks, or competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which exam boards does this course cover?

A: AQA, Edexcel and OCR, the content overlaps heavily, but paper styles and mark schemes differ, so the first thing the mentor confirms is your school's board, and every past paper comes from it. The 9-1 grading system and three-paper structure (one non-calculator) frame all exam work.

Q: Foundation or Higher, how should the decision be made?

A: By destination, then evidence. Foundation caps the grade at 5, so if A-Level maths or a competitive sixth form is even possible, Higher is the default. Whether the gaps blocking Higher are repairable in the time available is a diagnostic question, and the free demo answers it with evidence you can take to school.

Q: My child's grades slide every mock as content deepens. Why?

A: That slide is the classic signature of procedure without understanding: memorized methods hold at grade 4-5 content and collapse when questions chain or re-frame. Understanding-first teaching is the fix, algebra as a language, geometry with justification, which is exactly how this course teaches.

Q: Can you take a grade 6 student to grade 8-9?

A: All are taught directly in Phases 2-3. The honest caveat: the earlier the start, the more settling time the grade 8-9 topics get.

Q: Do you take resit candidates?

A: Yes, post-16 resit students get a compressed, evidence-led version of the program prioritised by their diagnostic, aimed at the grade 4/5 threshold or beyond as their goals require.

Q: How does this fit around school mocks and revision?

A: The mentor plans against your school calendar: intensity rises before mocks, past-paper focus aligns with what school is testing, and the final months are sequenced around the real series. The course reinforces school rather than competing with it.

Q: Can we try it before paying?

A: Yes. Every student starts with a free live demo class that doubles as the diagnostic, board, tier, fluency and algebra base, with no card details required.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/gcse-mathematics-mastery>

=====

COURSE 61 OF 118: GENERATIVE AI COURSE: LLMS, RAG & AI AGENTS, ZERO TO PRO

=====

Course link: <https://learn.modernagecoders.com/courses/complete-generative-ai-masterclass-college>

Subject area: Professional AI Development

Designed for: College students

Who can join: Complete Beginner to AI Engineer Professional

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Industry-recognized Generative AI Engineer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Generative AI course: master LLMs, GPT models, Stable Diffusion, AI agents, RAG systems, fine-tuning and prompt engineering to build production AI.

In one line: From 'What is AI?' to Building Production-Ready AI Systems

ABOUT THE PROGRAMME

This is not just a Generative AI course, it's a complete transformation into an enterprise-ready AI Engineer. Whether you're a curious beginner, student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled AI professional capable of building LLM applications, AI agents, image generation systems, multimodal AI, and production-ready generative AI solutions. You'll master Generative AI from ground zero to AI architect level: from understanding neural networks to building GPT-powered applications, from basic prompts to advanced prompt engineering, from using APIs to fine-tuning models, from single AI calls to complex multi-agent systems. By the end, you'll have built 50+ AI projects, mastered the entire GenAI ecosystem, and be ready for AI Engineer roles at top tech companies and AI startups.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - perfect for complete beginners
- Separate learning tracks for kids (14+), teens, and adults
- 1 year of structured, industry-aligned learning
- Covers foundational AI + Modern GenAI (LLMs, Diffusion, Agents)
- Real AI products and industry case studies
- Hands-on with latest models (GPT-4, Claude, Gemini, Stable Diffusion)
- Build production-ready AI applications
- Ethics and responsible AI development
- Direct path to high-paying AI engineer jobs
- Lifetime access with continuous model updates

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Python, Math, ML Basics, Neural Networks, Introduction to GenAI

Phase 2: Intermediate (Months 4-6): Transformers, LLMs, Prompt Engineering, OpenAI/Anthropic APIs, Vector DBs

Phase 3: Advanced (Months 7-9): Fine-tuning, RAG Systems, AI Agents, LangChain, Image/Video Generation

Phase 4: Professional (Months 10-12): Multi-Agent Systems, Production Deployment, MLOps, Custom Models, AI Products

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to all backgrounds including non-technical

Coding experience: Absolute beginner friendly - zero programming knowledge required

Age: 14+ years (separate learning tracks for teens and adults)

Equipment: Computer/laptop with 16GB+ RAM recommended (8GB minimum), NVIDIA GPU helpful but not required (use Google Colab), internet connection

Time commitment: 15-20 hours per week consistently for best results

English: Basic reading comprehension (materials also available in Hindi)

Motivation: Strong desire to become an AI professional and shape the future

Math: Basic high school mathematics (taught from scratch as needed)

WHO THIS COURSE IS FOR

Teens: Age 14-18: AI-focused track with fundamentals, college prep, research skills, future-ready education

Students: College students: Career preparation, internship readiness, research opportunities, strong AI portfolio

Working professionals: Career switchers: Structured path from any background to AI engineer role

Developers: Software developers wanting to pivot to AI engineering

Data scientists: Data scientists wanting to master modern generative AI

ML engineers: ML engineers wanting to specialize in LLMs and generative models

Entrepreneurs: Build revolutionary AI products and startups

Researchers: Academic researchers wanting practical AI implementation skills

Freelancers: Offer cutting-edge AI services to high-paying clients

AI curious: Anyone fascinated by AI and wanting to be part of the AI revolution

Career advancers: Professionals wanting to add AI skills to their domain expertise

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & AI FUNDAMENTALS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid AI fundamentals. Learn Python, mathematics for AI, machine learning basics, and introduction to generative AI.

Months 1-2: Python Programming & Math for AI (Week 1-8)

Python Fundamentals & Development Setup

Topics:

- What is Artificial Intelligence? History and evolution
- Types of AI: Narrow AI, General AI, Superintelligence
- Machine Learning vs Deep Learning vs Generative AI
- Generative AI revolution: GPT, DALL-E, Stable Diffusion
- AI career paths and opportunities in 2026 and beyond
- Why Python for AI? Python ecosystem overview
- Installing Python (3.10+) and setting up environment
- Development environments: Jupyter Notebook, VS Code, Google Colab
- Python basics: variables, data types, operators
- Control structures: if-else, loops (for, while)
- Functions: definition, parameters, return values
- Python data structures: lists, tuples, dictionaries, sets
- List comprehensions and dictionary comprehensions
- File handling: reading and writing files
- Error handling: try-except blocks
- Python modules and packages
- Virtual environments with venv/conda

Projects built:

- Text-based calculator program
- Todo list application
- File organizer script
- Simple data analyzer from CSV
- Web scraper basics with requests
- Command-line application
- Personal expense tracker
- Password generator and validator

Practice: Daily: 45 min Python practice, solve 10-15 problems on HackerRank/LeetCode

Advanced Python & Data Manipulation

Topics:

- Object-Oriented Programming in Python
- Classes and objects: `__init__`, `self`
- Inheritance and polymorphism
- Magic methods: `__str__`, `__repr__`, `__len__`
- Decorators and their usage
- Lambda functions and functional programming
- Map, filter, reduce functions
- Generators and iterators
- Context managers and 'with' statement
- Regular expressions for text processing
- Working with JSON and CSV data
- Introduction to NumPy: arrays and operations
- NumPy array operations, broadcasting, indexing
- Introduction to Pandas: DataFrames and Series
- Data loading, cleaning, and exploration with Pandas
- Data visualization basics with Matplotlib
- Plotting: line plots, bar charts, scatter plots, histograms

Projects built:

- Data analysis pipeline for CSV files
- Student grade analyzer with Pandas
- Weather data visualizer
- E-commerce sales dashboard
- Stock price analyzer and plotter
- Text file analyzer with regex
- Custom data class library
- API wrapper class with error handling

Practice: Solve 30 Python problems, analyze 5 real-world datasets

Mathematics for AI - Part 1

Topics:

- Why math matters for AI and ML
- Linear Algebra: vectors and vector operations
- Dot product and cross product
- Matrices and matrix operations
- Matrix multiplication and transposition
- Identity matrix and inverse matrix
- Eigenvalues and eigenvectors (intuition)
- Calculus: functions and limits
- Derivatives: concepts and rules
- Chain rule and partial derivatives
- Gradient: direction of steepest ascent
- Gradient descent algorithm basics

- Multivariable calculus for neural networks
- Probability theory: basics and terminology
- Probability distributions: uniform, normal (Gaussian)
- Expected value and variance
- Bayes' theorem and conditional probability

Projects built:

- Vector operations visualizer
- Matrix calculator with NumPy
- Gradient descent implementation from scratch
- Probability distribution plotter
- Coin flip simulator with statistics
- Linear regression using gradient descent
- Simple optimization problems solver
- Derivative calculator for basic functions

Practice: Solve 40 math problems relevant to ML, implement algorithms

Mathematics for AI - Part 2 & Statistics

Topics:

- Statistics for data science
- Descriptive statistics: mean, median, mode
- Variance and standard deviation
- Correlation and covariance
- Hypothesis testing basics
- P-values and statistical significance
- Central Limit Theorem
- Confidence intervals
- Linear regression: mathematical foundation
- Cost functions and loss functions
- Optimization: local minima, global minima
- Convex vs non-convex functions
- Introduction to information theory
- Entropy and cross-entropy
- KL divergence
- Dimensionality reduction concepts
- Principal Component Analysis (PCA) intuition

Projects built:

- Statistical analyzer for datasets
- Hypothesis testing tool
- Correlation matrix visualizer
- Simple linear regression implementation
- Cost function visualizer
- PCA implementation from scratch
- Data distribution analyzer
- A/B testing simulator

Practice: Analyze 10 datasets statistically, implement core algorithms

Month 3: Machine Learning Fundamentals (Week 9-13)

Introduction to Machine Learning

Topics:

- What is Machine Learning? Formal definition
- Types of learning: Supervised, Unsupervised, Reinforcement
- Machine learning workflow: data, model, evaluation
- Training, validation, and test sets
- Overfitting and underfitting
- Bias-variance tradeoff
- Introduction to scikit-learn library
- Linear regression with scikit-learn
- Logistic regression for classification
- Decision trees: intuition and implementation
- Random forests ensemble method
- K-Nearest Neighbors (KNN) algorithm
- Support Vector Machines (SVM) basics
- Naive Bayes classifier
- Model evaluation metrics: accuracy, precision, recall, F1
- Confusion matrix and ROC curves
- Cross-validation techniques

Projects built:

- House price predictor (linear regression)
- Email spam classifier (logistic regression)
- Iris flower classifier (decision trees)
- Handwritten digit recognizer (KNN)
- Customer churn predictor
- Movie recommendation system basics
- Credit card fraud detection
- Student performance predictor

Practice: Build 15 ML models, solve Kaggle beginner competitions

Neural Networks & Deep Learning Basics

Topics:

- From ML to Deep Learning: why neural networks?
- Perceptron: the building block

- Multi-layer perceptron (MLP) architecture
- Activation functions: sigmoid, tanh, ReLU, Leaky ReLU
- Forward propagation step-by-step
- Loss functions: MSE, cross-entropy
- Backpropagation algorithm intuition
- Gradient descent and its variants (SGD, Adam)
- Learning rate and hyperparameters
- Introduction to TensorFlow and Keras
- Building neural networks with Keras Sequential API
- Training and evaluating neural networks
- Regularization techniques: dropout, L1/L2
- Batch normalization
- Early stopping and model checkpoints
- Introduction to PyTorch basics
- PyTorch tensors and autograd

Projects built:

- Neural network from scratch in NumPy
- MNIST digit classifier with Keras
- Fashion MNIST classifier
- Binary classification with neural network
- Multi-class classification problem
- Regression with neural networks
- Simple image classifier
- Neural network hyperparameter tuning

Practice: Build 12 neural networks, compare frameworks

Introduction to Generative AI

Topics:

- What is Generative AI? Definition and scope
- Discriminative vs Generative models
- Brief history: from GANs to GPT to Diffusion
- Modern generative AI landscape 2026
- Large Language Models (LLMs): GPT, Claude, Gemini
- Text-to-image models: DALL-E, Midjourney, Stable Diffusion
- Text-to-video: Sora, Runway, Pika
- Audio generation: music and speech
- Multimodal AI: combining text, image, audio
- AI model sizes and capabilities
- Prompt engineering: first introduction
- Using ChatGPT/Claude effectively
- AI ethics and responsible AI principles
- Bias in AI models and mitigation
- AI safety and alignment
- Future of generative AI
- Career opportunities in GenAI

Projects built:

- ChatGPT prompt collection and analysis
- Simple prompt engineering experiments
- AI-generated content curator
- Comparative study: different AI models
- Bias detection in AI responses
- Personal AI assistant with prompts
- PHASE 1 MINI CAPSTONE: AI-powered research assistant (using existing APIs)
- Features: Multiple AI model comparison, prompt templates, response analysis

Practice: Experiment with 50+ prompts across different AI models

Assessment: Phase 1 Final Assessment - Python, Math, ML, and GenAI basics test

PHASE 2: LLM FOUNDATIONS & PROMPT ENGINEERING (MONTHS 4-6, WEEKS 14-26)

Master Large Language Models, transformer architecture, prompt engineering, working with AI APIs, and building LLM-powered applications.

Months 4-5: Transformers, LLMs & API Integration (Week 14-22)

Transformer Architecture Deep Dive

Topics:

- Limitations of RNNs and LSTMs
- Attention mechanism: intuition and math
- Self-attention and multi-head attention
- Positional encoding in transformers
- Transformer architecture: encoder-decoder
- BERT: bidirectional encoder representations
- GPT architecture: decoder-only transformers
- Tokenization: BPE, WordPiece, SentencePiece
- Embedding layers and embedding spaces
- How GPT generates text: autoregressive generation
- Temperature, top-k, top-p sampling
- Model parameters: understanding 7B, 70B, 175B
- Context windows and attention span
- Fine-tuning vs prompt engineering vs RAG

- Pre-training and transfer learning
- Hugging Face Transformers library introduction
- Loading and using pre-trained models

Projects built:

- Attention mechanism visualizer
- Tokenizer comparison tool
- Text generation with Hugging Face
- Sentiment analysis with BERT
- Text classification with transformers
- Named Entity Recognition (NER) system
- Question answering system
- Text summarization tool

Practice: Implement attention from scratch, use 10+ transformer models

OpenAI API & LLM Integration

Topics:

- OpenAI API overview and capabilities
- Getting API keys and authentication
- Current OpenAI chat models, and how to choose between them
- Chat completions API: messages and roles
- System, user, and assistant messages
- Completion parameters: temperature, max_tokens, top_p
- Streaming responses for real-time output
- Function calling with GPT-4
- JSON mode for structured outputs
- Vision API: GPT-4V for image understanding
- DALL-E API for image generation
- Whisper API for speech-to-text
- TTS (Text-to-Speech) API
- Embeddings API and semantic search
- Moderation API for content filtering
- Token counting and cost optimization
- Rate limiting and error handling

Projects built:

- ChatGPT clone with OpenAI API
- AI content generator dashboard
- Function calling chatbot
- Image analyzer with GPT-4V
- AI image generator web app
- Voice-to-text transcription tool
- Semantic search engine
- AI content moderation system

Practice: Build 15 OpenAI API applications

Advanced Prompt Engineering

Topics:

- Prompt engineering as a discipline
- Anatomy of effective prompts
- Zero-shot, one-shot, few-shot learning
- Chain-of-Thought (CoT) prompting
- Tree of Thoughts (ToT) technique
- Self-consistency prompting
- Role prompting and persona design
- Instruction tuning concepts
- Prompt templates and variables
- Output formatting and constraints
- Handling hallucinations and reducing errors
- Prompt injection and safety
- Meta-prompting and prompt generation
- Multi-turn conversation design
- Context management in long conversations
- Prompt optimization and A/B testing
- Domain-specific prompting strategies

Projects built:

- Prompt library and template system
- Chain-of-Thought problem solver
- AI tutor with CoT reasoning
- Code generator with prompts
- Creative writing assistant
- Structured data extractor
- Multi-step task planner
- Prompt testing and evaluation framework

Practice: Create 100+ optimized prompts for various use cases

Alternative LLM APIs & Providers

Topics:

- Anthropic Claude API and capabilities
- Claude's extended context window (100K+ tokens)
- Constitutional AI and Claude's safety features
- Google Gemini API and multimodal features
- Gemini Pro vs Gemini Ultra
- Meta Llama models and usage

- Open-source LLMs: Mistral, Falcon, MPT
- Running local LLMs with Ollama
- LLM comparison: GPT vs Claude vs Gemini
- Choosing the right model for your use case
- Cost comparison across providers
- Cohere API for enterprise solutions
- Azure OpenAI Service
- AWS Bedrock for LLM deployment
- Model routing and fallback strategies
- Multi-model applications
- API key management and security

Projects built:

- Multi-LLM chat interface
- Model comparison dashboard
- Local LLM deployment with Ollama
- Claude-powered research assistant
- Gemini multimodal application
- Cost optimizer for LLM calls
- Model router based on task type
- Unified LLM wrapper library

Practice: Build applications with 5+ different LLM providers

Vector Databases & Embeddings

Topics:

- What are embeddings? Semantic representations
- Word2Vec, GloVe, and modern embeddings
- OpenAI embeddings (text-embedding-ada-002)
- Sentence embeddings and document embeddings
- Cosine similarity and semantic search
- Vector databases: purpose and architecture
- Pinecone: managed vector database
- Weaviate: open-source vector DB
- Chroma: simple vector store
- Qdrant: performance-focused vector DB
- FAISS: Facebook AI similarity search
- Indexing strategies: flat, HNSW, IVF
- Metadata filtering in vector search
- Hybrid search: vector + keyword
- Building semantic search systems
- Recommendation systems with embeddings
- Clustering and visualization of embeddings

Projects built:

- Semantic search engine for documents
- Similar article finder
- Customer support ticket classifier
- Product recommendation system
- Duplicate detection system
- Embedding visualizer (t-SNE/UMAP)
- Multi-language semantic search
- Knowledge base with vector search

Practice: Build 10 vector database applications

Month 6: LangChain & AI Application Development (Week 23-26)

LangChain Fundamentals

Topics:

- Introduction to LangChain framework
- LangChain architecture and components
- LLMs and Chat Models in LangChain
- Prompt templates and prompt engineering
- Output parsers: structured data from LLMs
- Chains: sequential processing
- LLMChain, SimpleSequentialChain, SequentialChain
- Memory types: ConversationBufferMemory, ConversationSummaryMemory
- Conversation chains with memory
- Document loaders: PDF, CSV, web pages
- Text splitters and chunking strategies
- Vector stores integration in LangChain
- Retrievers for information retrieval
- Combining LLMs with external data
- Question-answering over documents
- LangChain Expression Language (LCEL)
- Debugging and tracing in LangChain

Projects built:

- Conversational chatbot with memory
- Document Q&A system with LangChain
- PDF analyzer and summarizer
- Web scraper with AI analysis
- Multi-step reasoning chain
- Structured data extraction pipeline
- Conversation summarizer

- Knowledge base chatbot

Practice: Build 15 LangChain applications

Advanced LangChain & Tools

Topics:

- Agents: autonomous task execution
- ReAct (Reasoning + Acting) agents
- Agent types: Zero-shot, Conversational
- Tools and toolkits in LangChain
- Creating custom tools for agents
- Google Search tool integration
- Python REPL tool for code execution
- Calculator and math tools
- API tools and web requests
- Database tools for SQL queries
- File system tools
- Multi-action agents
- Agent error handling and retries
- LangSmith for debugging and monitoring
- Callbacks and custom handlers
- Streaming responses in LangChain
- Cost tracking and optimization

Projects built:

- Autonomous research agent
- Code generator and executor agent
- SQL query generator from natural language
- Web automation agent
- Data analysis agent
- Personal assistant agent with tools
- Multi-tool agent for complex tasks
- Agent performance monitoring dashboard

Practice: Build 12 AI agent applications

Retrieval Augmented Generation (RAG) - Part 1

Topics:

- What is RAG? Why RAG over fine-tuning?
- RAG architecture: retrieval + generation
- Building basic RAG pipeline
- Document preprocessing for RAG
- Chunking strategies: fixed-size, semantic
- Embedding generation for documents
- Vector store population
- Retrieval strategies: similarity search, MMR
- Reranking retrieved documents
- Context compression techniques
- Query transformation and expansion
- Retrieval quality evaluation
- Prompt engineering for RAG
- Handling long contexts
- Multi-query retrieval
- Hypothetical document embeddings (HyDE)
- Parent document retrieval

Projects built:

- Enterprise documentation Q&A system
- Academic paper search and summarization
- Legal document analyzer
- Customer support knowledge base
- Code documentation assistant
- Research paper recommendation system
- Meeting notes Q&A system
- Email archive search with AI

Practice: Build 10 RAG applications for different domains

Retrieval Augmented Generation (RAG) - Part 2

Topics:

- Advanced RAG architectures
- Self-RAG: self-reflective retrieval
- Corrective RAG (CRAG)
- Agentic RAG with LangGraph
- Multi-document RAG systems
- Hierarchical retrieval strategies
- Graph-based RAG
- Conversational RAG with memory
- RAG with structured data (tables, charts)
- Multimodal RAG: text + images
- RAG evaluation metrics: faithfulness, relevance
- RAGAS framework for evaluation
- RAG optimization techniques
- Caching strategies for RAG
- Production RAG deployment
- RAG monitoring and observability
- Security and privacy in RAG systems

Projects built:

- Advanced enterprise RAG system
- Multi-source RAG pipeline
- Conversational RAG with context
- Evaluation framework for RAG
- Self-improving RAG system
- Multimodal document understanding
- RAG-powered coding assistant
- Production-ready RAG API

Practice: Build and optimize 8 advanced RAG systems

Phase 2 Capstone Project

Topics:

- LLM application architecture
- API integration best practices
- Prompt engineering mastery
- Vector database implementation
- RAG system design
- Agent orchestration

Projects built:

- PHASE 2 CAPSTONE: Enterprise RAG-powered AI Assistant
- Features: Multi-document upload, vector search, conversational memory, source citations, admin dashboard, user authentication, cost tracking
- Alternative: AI-powered code review system
- Alternative: Research paper analysis platform
- Alternative: Intelligent customer support system

Assessment: Phase 2 Final Exam - LLMs, prompt engineering, and RAG comprehensive test

PHASE 2 CONTINUED - Computer Vision & Multimodal AI (Week 14-26 (distributed))

Computer Vision Fundamentals

Topics:

- Introduction to computer vision
- Digital images: pixels, channels, color spaces
- Convolutional Neural Networks (CNNs)
- Convolution operation and filters
- Pooling layers: max pooling, average pooling
- CNN architectures: LeNet, AlexNet, VGG
- ResNet and skip connections
- Transfer learning with pre-trained models
- Image classification with CNNs
- Object detection: R-CNN, YOLO, SSD
- Image segmentation: semantic, instance
- OpenCV library for image processing
- PyTorch for computer vision
- torchvision and pre-trained models
- Data augmentation techniques
- Training CNNs: tips and tricks
- Model deployment for vision tasks

Projects built:

- Image classifier with ResNet
- Custom object detector
- Facial recognition system
- Image segmentation tool
- Real-time video processing
- Medical image analyzer
- Defect detection system
- Transfer learning for custom dataset

Practice: Build 10 computer vision applications

Generative Models for Images

Topics:

- Introduction to generative models
- Variational Autoencoders (VAEs)
- Generative Adversarial Networks (GANs)
- GAN training: generator and discriminator
- Common GAN variants: DCGAN, StyleGAN
- Diffusion models: DDPM, DDIM
- How Stable Diffusion works
- Latent diffusion models
- Text-to-image generation: Stable Diffusion
- Image-to-image transformation
- ControlNet for controlled generation
- LoRA: Low-Rank Adaptation
- DreamBooth for personalization
- Inpainting and outpainting
- Image upscaling with AI
- Automatic1111 and ComfyUI interfaces
- Hugging Face Diffusers library

Projects built:

- Text-to-image generator with Stable Diffusion
- Image style transfer application

- AI art generator with custom prompts
- Image inpainting tool
- Face generation with StyleGAN
- Custom image model with LoRA
- Image variation generator
- AI-powered design tool

Practice: Generate 1000+ images, fine-tune models

Multimodal AI & Vision-Language Models

Topics:

- Multimodal learning: combining modalities
- CLIP: connecting text and images
- CLIP embeddings and applications
- GPT-4V: vision understanding with GPT-4
- LLaVA: visual instruction tuning
- BLIP and BLIP-2 models
- Image captioning with transformers
- Visual question answering (VQA)
- Text-to-video generation: basics
- Video understanding with AI
- OCR with modern AI models
- Document understanding: LayoutLM
- Multimodal RAG systems
- Integrating vision and language models
- Building multimodal applications
- Evaluation of multimodal models
- Future of multimodal AI

Projects built:

- Image captioning system
- Visual question answering app
- Document analyzer with OCR
- Multimodal search engine
- AI-powered accessibility tool
- Meme generator with AI
- Video summarization tool
- Multimodal RAG for images + text

Practice: Build 12 multimodal AI applications

Audio & Speech AI

Topics:

- Audio processing basics: waveforms, spectrograms
- Automatic Speech Recognition (ASR)
- Whisper model architecture and usage
- Speech-to-text applications
- Text-to-Speech (TTS) models
- Voice cloning with AI
- Bark, ElevenLabs, and other TTS systems
- Music generation with AI
- MusicGen, AudioLM, Jukebox
- Audio classification and tagging
- Voice activity detection
- Speaker diarization
- Audio enhancement and denoising
- Real-time audio processing
- Building voice assistants
- Multilingual speech models
- Audio-visual speech recognition

Projects built:

- Transcription service with Whisper
- AI voice assistant
- Text-to-speech application
- Voice cloning tool
- Music generator
- Podcast transcriber and summarizer
- Real-time translation system
- Audio classification system

Practice: Build 10 audio AI applications

AI Ethics & Responsible Development

Topics:

- AI ethics fundamentals
- Bias in AI: sources and mitigation
- Fairness metrics and evaluation
- Transparency and explainability
- LIME and SHAP for model interpretation
- Privacy in AI: data protection
- Differential privacy
- Federated learning basics
- AI safety and alignment
- Constitutional AI principles
- Red teaming and adversarial testing
- Content moderation and filtering

- Deepfakes: detection and prevention
- AI regulations: GDPR, AI Act
- Responsible AI deployment
- Environmental impact of AI
- AI for social good

Projects built:

- Bias detection tool for models
- Explainable AI dashboard
- Content moderation system
- Deepfake detector
- Privacy-preserving AI application
- AI impact assessment tool
- Responsible AI checklist generator

Practice: Evaluate and improve fairness in all projects

----- PHASE 3: FINE-TUNING, AI AGENTS & PRODUCTION SYSTEMS (MONTHS 7-9, WEEKS 27-39) -----

Master model fine-tuning, build autonomous AI agents, create production-ready AI systems, and learn MLOps.

Months 7-8: Model Fine-tuning & Customization (Week 27-35)

Fine-tuning Fundamentals

Topics:

- What is fine-tuning? When to fine-tune?
- Fine-tuning vs prompt engineering vs RAG
- Types of fine-tuning: full, parameter-efficient
- Transfer learning for LLMs
- Dataset preparation for fine-tuning
- Data annotation and labeling
- Instruction tuning datasets
- Supervised fine-tuning (SFT)
- Training loops and optimization
- Loss functions for language models
- Hyperparameter tuning
- Overfitting prevention in fine-tuning
- Evaluation metrics for fine-tuned models
- Benchmark datasets and tasks
- OpenAI fine-tuning API
- Fine-tuning current OpenAI models
- Cost considerations for fine-tuning

Projects built:

- Fine-tune a current OpenAI model for customer support
- Domain-specific chatbot fine-tuning
- Code completion model fine-tuning
- Fine-tuned model for legal documents
- Medical terminology fine-tuning
- Custom classifier with fine-tuning
- Sentiment analysis fine-tuned model
- Fine-tuning evaluation framework

Practice: Fine-tune 8 models for different domains

Advanced Fine-tuning Techniques

Topics:

- Parameter-Efficient Fine-Tuning (PEFT)
- LoRA: Low-Rank Adaptation explained
- LoRA implementation with PEFT library
- QLoRA: quantized LoRA for efficiency
- Adapter layers and adapter tuning
- Prefix tuning and prompt tuning
- P-tuning and P-tuning v2
- (IA)³: Infused Adapter by Inhibiting and Amplifying
- Choosing the right PEFT method
- 4-bit and 8-bit quantization
- bitsandbytes library usage
- Gradient checkpointing for memory
- Mixed precision training
- RLHF: Reinforcement Learning from Human Feedback
- DPO: Direct Preference Optimization
- Reward models and preference learning
- Alignment techniques overview

Projects built:

- LoRA fine-tuning for Llama 2
- QLoRA for consumer GPU training
- Multi-adapter model system
- RLHF preference dataset creation
- DPO fine-tuning implementation
- Memory-efficient training pipeline
- Adapter merging and switching
- Custom PEFT method comparison

Practice: Fine-tune 10 models with PEFT techniques

Open Source LLMs & Local Deployment

Topics:

- Open source LLM landscape
- Llama 2 and Llama 3 models
- Mistral and Mixtral models
- Falcon, MPT, and BLOOM
- Model cards and model documentation
- Downloading models from Hugging Face Hub
- Model quantization: GGUF, GPTQ, AWQ
- Running LLMs locally with Ollama
- llama.cpp for efficient inference
- vLLM for production serving
- Text Generation Inference (TGI)
- Model serving with FastAPI
- GPU vs CPU inference
- Batch inference optimization
- Caching and KV-cache optimization
- Model benchmarking: speed and quality
- Building custom inference servers

Projects built:

- Local LLM deployment with Ollama
- Quantized model serving API
- Multi-model inference server
- LLM benchmarking suite
- Optimized inference pipeline
- Custom model hosting solution
- Edge deployment of LLMs
- Inference cost calculator

Practice: Deploy and optimize 10 open source models

AI Agents Architecture - Part 1

Topics:

- What are AI agents? Autonomous systems
- Agent types: simple reflex, goal-based, utility-based
- Perceive-Think-Act loop
- ReAct: reasoning and acting
- Plan-and-Execute agents
- Cognitive architectures for agents
- Agent memory systems
- Short-term vs long-term memory
- Semantic memory and episodic memory
- Working memory implementation
- Tool use and function calling
- Multi-step reasoning with agents
- Error handling and recovery
- Agent observability and logging
- LangGraph for agent workflows
- State machines for agents
- Human-in-the-loop agents

Projects built:

- Autonomous research agent
- Data analysis agent
- Code generation and debugging agent
- Travel planning agent
- Email management agent
- Content creation agent
- Personal productivity agent
- Multi-step task automation agent

Practice: Build 12 AI agent systems

AI Agents Architecture - Part 2

Topics:

- Multi-agent systems (MAS)
- Agent communication protocols
- Collaborative agents vs competitive agents
- Agent coordination and cooperation
- Hierarchical agent architectures
- AutoGPT and BabyAGI patterns
- Task decomposition and planning
- Sub-goal generation
- Agent self-evaluation and reflection
- MetaGPT and role-playing agents
- Specialized agent roles
- Agent orchestration frameworks
- Crew AI for multi-agent systems
- Agent security and sandboxing
- Rate limiting and cost control
- Agent performance optimization
- Debugging complex agent systems

Projects built:

- Multi-agent research team
- Collaborative writing agents
- Software development agent team
- Customer service agent system
- Agent-based simulation
- Hierarchical task planner
- Self-improving agent system
- Production multi-agent platform

Practice: Build 8 multi-agent systems

Month 9: MLOps & Production AI Systems (Week 36-39)

MLOps Fundamentals

Topics:

- What is MLOps? DevOps for ML
- ML lifecycle: develop, deploy, monitor
- Experiment tracking with MLflow
- Model versioning and registry
- Weights & Biases for experiment tracking
- Data versioning with DVC
- Model packaging and containerization
- Docker for ML applications
- Model serving patterns
- REST APIs for model serving
- Batch vs real-time inference
- Model deployment strategies
- Blue-green deployment for models
- Canary releases and A/B testing
- Model monitoring and observability
- Detecting model drift
- Retraining pipelines and triggers

Projects built:

- MLflow experiment tracking setup
- Model registry and versioning system
- Dockerized ML model API
- CI/CD pipeline for ML models
- Model monitoring dashboard
- A/B testing framework
- Automated retraining pipeline
- Production ML workflow

Practice: Implement MLOps for all ML projects

AI Application Architecture & APIs

Topics:

- Designing AI-powered applications
- Microservices architecture for AI
- FastAPI for ML/AI APIs
- Request validation and error handling
- Asynchronous processing with Celery
- Message queues: RabbitMQ, Redis
- Background tasks for AI workloads
- Caching strategies for AI apps
- Redis for response caching
- Database integration: PostgreSQL, MongoDB
- Storing embeddings in databases
- User authentication and authorization
- API rate limiting and throttling
- Webhook integrations
- Building SDKs for AI APIs
- API documentation with OpenAPI
- Postman and API testing

Projects built:

- Production AI API with FastAPI
- Asynchronous AI processing system
- Cached AI inference service
- Multi-user AI application
- AI API with usage tracking
- Webhook-based AI integration
- Client SDK for AI service
- Complete AI backend system

Practice: Build 10 production-ready AI APIs

Prompt Engineering at Scale

Topics:

- Enterprise prompt management
- Prompt versioning and tracking
- Prompt testing frameworks
- Evaluation metrics for prompts
- Automated prompt optimization
- Prompt tuning with PromptPerfect
- DSPy: programming with foundation models
- Prompt composition and chaining

- Dynamic prompt generation
- Context-aware prompting
- Multi-language prompt engineering
- Prompt security and injection prevention
- Guardrails for LLM outputs
- NeMo Guardrails framework
- Content filtering and moderation
- Prompt governance and compliance
- Cost optimization for prompts

Projects built:

- Prompt management system
- Automated prompt testing suite
- Prompt optimization pipeline
- Guardrails implementation
- Multi-language prompt system
- Prompt injection detector
- Cost-optimized prompt router
- Enterprise prompt library

Practice: Build enterprise prompt engineering tools

Vector Databases at Scale

Topics:

- Production vector database setup
- Pinecone at scale: indexes and namespaces
- Weaviate cluster configuration
- Qdrant for high-performance search
- Milvus for massive-scale vectors
- pgvector: Postgres extension
- Index optimization: HNSW tuning
- Quantization for vector storage
- Sharding and partitioning strategies
- Backup and disaster recovery
- Multi-tenancy in vector DBs
- Security and access control
- Monitoring vector database performance
- Cost optimization strategies
- Hybrid search at scale
- GraphRAG with knowledge graphs
- Neo4j integration with vectors

Projects built:

- Production vector database deployment
- Multi-tenant vector search system
- Optimized hybrid search engine
- Knowledge graph + vector DB
- Vector DB performance monitoring
- High-availability vector system
- Serverless vector search
- Enterprise search platform

Practice: Deploy and scale 5 vector database systems

LLM Observability & Monitoring

Topics:

- LLM observability challenges
- Tracing LLM calls end-to-end
- LangSmith for debugging
- OpenTelemetry for AI applications
- Logging best practices for LLMs
- Prompt and completion logging
- Cost tracking per request
- Token usage monitoring
- Latency optimization
- Error rate tracking
- User feedback collection
- Quality metrics for LLM outputs
- Automated quality evaluation
- Hallucination detection
- Safety and content filtering metrics
- Building observability dashboards
- Alerting and incident response

Projects built:

- LLM observability platform
- Cost tracking dashboard
- Quality monitoring system
- Automated evaluation pipeline
- Incident detection and alerting
- User feedback aggregation tool
- Performance optimization dashboard
- Production monitoring system

Practice: Add comprehensive observability to all AI apps

Advanced RAG Techniques

Topics:

- Query understanding and decomposition
- Query routing to different data sources
- Fusion retrieval: combining strategies
- Contextual compression and filtering
- Re-ranking models: Cohere, ColBERT
- Late interaction models
- Multi-hop reasoning in RAG
- Graph-based RAG architectures
- Temporal RAG for time-sensitive data
- Structured data RAG: SQL + vectors
- Text-to-SQL with LLMs
- RAG for code repositories
- Recursive retrieval strategies
- Adaptive retrieval: when to retrieve
- Active RAG with verification
- Multi-lingual RAG systems
- RAG for streaming data

Projects built:

- Advanced multi-source RAG system
- Graph RAG implementation
- SQL + vector hybrid RAG
- Code repository RAG assistant
- Temporal knowledge RAG
- Adaptive retrieval system
- Multi-lingual enterprise RAG
- Real-time data RAG pipeline

Practice: Build 8 advanced RAG architectures

AI Security & Safety

Topics:

- LLM security vulnerabilities
- Prompt injection attacks and defenses
- Jailbreaking prevention
- Data leakage and privacy concerns
- PII detection and redaction
- Model inversion attacks
- Adversarial attacks on AI
- Robustness testing for LLMs
- Red teaming AI systems
- Safety classifiers and filters
- Constitutional AI implementation
- Alignment faking detection
- Secure API design for AI
- Authentication and authorization
- Secrets management for API keys
- Compliance: GDPR, HIPAA, SOC2
- Building audit trails

Projects built:

- Prompt injection detector
- PII redaction system
- Red team evaluation suite
- Safety classifier implementation
- Secure AI API gateway
- Compliance monitoring tool
- Audit logging system
- Security hardened AI platform

Practice: Security audit all AI applications

AI Infrastructure & Cloud Deployment

Topics:

- Cloud platforms for AI: AWS, GCP, Azure
- AWS SageMaker for ML/AI
- AWS Bedrock for LLM deployment
- Azure OpenAI Service
- Google Cloud Vertex AI
- GPU instances: A100, H100, V100
- Kubernetes for AI workloads
- Ray for distributed computing
- Serverless AI with AWS Lambda
- Cloud Run for containerized AI
- Auto-scaling AI services
- Load balancing for AI APIs
- CDN for AI-generated content
- Edge AI deployment
- Multi-cloud strategies
- Cost optimization: spot instances, reserved
- Infrastructure as Code: Terraform

Projects built:

- AWS SageMaker model deployment

- Kubernetes cluster for AI
- Serverless AI function
- Auto-scaling AI API
- Edge AI deployment
- Multi-cloud AI architecture
- Cost-optimized infrastructure
- Terraform IaC for AI

Practice: Deploy AI applications on all major clouds

AI Product Development

Topics:

- Building AI products: strategy
- Product-market fit for AI
- User experience design for AI
- Conversational UI best practices
- AI feature discovery and iteration
- A/B testing AI features
- Product analytics for AI
- User feedback loops
- MVP development for AI products
- Go-to-market for AI products
- Pricing strategies for AI
- Monetization: API-first, SaaS, usage-based
- Building AI product roadmaps
- Competitive analysis in AI space
- AI product metrics and KPIs
- Customer success for AI products
- Building AI product teams

Projects built:

- AI product MVP
- User analytics dashboard
- A/B testing framework
- Feedback collection system
- Pricing calculator
- Product roadmap template
- Competitive analysis tool
- AI product launch plan

Practice: Develop complete AI product strategy

Phase 3 Capstone Project

Topics:

- End-to-end AI system design
- Production deployment
- Monitoring and observability
- Security implementation
- Scalability optimization
- Cost management

Projects built:

- MAJOR CAPSTONE: Production AI Platform
- Features: Multi-agent system, advanced RAG, fine-tuned models, vector search, API with auth, monitoring dashboard, cost tracking, security hardening, cloud deployment, CI/CD pipeline
- Alternative: Enterprise AI Assistant Platform (Slack/Teams integration, custom models, knowledge base)
- Alternative: AI-Powered Analytics Platform (data analysis agents, automated insights, visualization)
- Alternative: Intelligent Content Platform (generation, moderation, personalization, multi-modal)

Assessment: Phase 3 Final Exam - Advanced AI systems comprehensive test

PHASE 4: CUSTOM MODELS, RESEARCH & PROFESSIONAL EXCELLENCE (MONTHS 10-12, WEEKS 40-52)

Master custom model development, research methodologies, specialized AI domains, and career mastery.

Months 10-11: Training Custom Models & Advanced Topics (Week 40-48)

Training LLMs from Scratch - Theory

Topics:

- LLM training pipeline overview
- Data collection and curation
- Data cleaning and filtering
- Tokenizer training: BPE, SentencePiece
- Model architecture design
- Choosing model size and layers
- Attention mechanisms variants
- Position embeddings: absolute, relative, RoPE
- Activation functions for transformers
- Initialization strategies
- Parallelism: data, model, pipeline
- Distributed training frameworks
- Gradient accumulation

- Mixed precision training
- Optimizer choice: Adam, AdamW, Lion
- Learning rate schedules
- Pre-training objectives

Projects built:

- Tokenizer training pipeline
- Small transformer from scratch
- Data preprocessing pipeline
- Training infrastructure setup
- Distributed training experiment
- Model architecture comparison
- Training monitoring dashboard
- Pre-training experiment

Practice: Train 3 small language models from scratch

Training LLMs from Scratch - Practice

Topics:

- Setting up training environment
- PyTorch Lightning for training
- Hugging Face Accelerate library
- DeepSpeed for large models
- FSDP: Fully Sharded Data Parallel
- Flash Attention for efficiency
- Checkpointing and recovery
- Evaluation during training
- Perplexity and other metrics
- Early stopping strategies
- Debugging training issues
- OOM errors and solutions
- Gradient clipping and stabilization
- Post-training: alignment and safety
- Instruction tuning pipeline
- Dataset quality for alignment
- Evaluation of trained models

Projects built:

- Complete pre-training pipeline
- Instruction-tuned model
- Efficient training with DeepSpeed
- Multi-GPU training setup
- Training monitoring and logging
- Model evaluation suite
- Checkpoint management system
- Custom training framework

Practice: Complete training pipeline for custom model

Specialized AI Domains - NLP

Topics:

- Advanced NLP tasks beyond generation
- Named Entity Recognition (NER) at scale
- Relation extraction from text
- Coreference resolution
- Semantic role labeling
- Dependency parsing
- Information extraction pipelines
- Document understanding
- Table extraction and understanding
- Knowledge graph construction from text
- Text classification: hierarchical, multi-label
- Sentiment analysis: aspect-based
- Topic modeling with LLMs
- Text clustering and categorization
- Zero-shot and few-shot classification
- Cross-lingual NLP
- Low-resource language processing

Projects built:

- End-to-end information extraction system
- Knowledge graph builder
- Document understanding pipeline
- Multi-label classifier
- Aspect-based sentiment analyzer
- Cross-lingual text processor
- Entity linking system
- Hierarchical categorization tool

Practice: Build 10 advanced NLP systems

Specialized AI Domains - Code & Reasoning

Topics:

- Code generation with LLMs
- Code completion models
- Program synthesis
- Code translation between languages
- Bug detection and fixing

- Code review automation
- Test case generation
- Documentation generation
- Code search and retrieval
- Execution-based code evaluation
- Mathematical reasoning with LLMs
- Symbolic reasoning integration
- Tool use for complex reasoning
- Multi-step problem solving
- Verification and validation
- Chain-of-thought for code
- Self-debugging agents

Projects built:

- AI coding assistant
- Automated code reviewer
- Test generator
- Code documentation tool
- Bug detection system
- Math problem solver
- Code search engine
- Multi-step reasoning system

Practice: Build 8 code and reasoning AI tools

Reinforcement Learning for LLMs

Topics:

- Reinforcement Learning basics review
- RL for language models
- Reward modeling for RLHF
- Preference datasets and collection
- Proximal Policy Optimization (PPO)
- PPO for language models
- Reward hacking and mitigation
- KL divergence constraint
- Online vs offline RL
- Constitutional AI with RL
- Multi-objective RLHF
- DPO: Direct Preference Optimization
- IPO and other alignment methods
- Reward model training
- Evaluation of aligned models
- Scaling RLHF
- Future of alignment research

Projects built:

- Reward model training
- RLHF pipeline implementation
- DPO fine-tuning
- Preference dataset creation
- Alignment evaluation suite
- Multi-objective alignment
- Constitutional AI implementation
- Complete RLHF system

Practice: Implement RLHF and DPO pipelines

Month 12: Research, Cutting Edge & Career Launch (Week 49-52)

AI Research & Paper Implementation

Topics:

- Reading research papers effectively
- Understanding paper structure
- Key AI conferences: NeurIPS, ICML, ICLR
- ArXiv and Hugging Face Papers
- State-of-the-art tracking
- Implementing papers from scratch
- Reproducing research results
- Ablation studies and experiments
- Writing technical blog posts
- Contributing to research discussions
- Benchmarking new techniques
- Open source contributions to AI
- Creating research reproducibility
- Building on existing research
- Novel applications of research
- Research ethics and attribution
- Staying current with AI advances

Projects built:

- Implement 5 recent papers
- Reproduce benchmark results
- Novel application of research
- Technical blog series
- Benchmark comparison tool
- Research tracking dashboard

- Paper implementation library
- Open source contribution

Practice: Read 50+ papers, implement 10+ techniques

Cutting-Edge AI Techniques

Topics:

- Mixture of Experts (MoE) models
- Sparse models and efficiency
- Long context models: 100K+ tokens
- Efficient attention mechanisms
- Quantization: GPTQ, AWQ, GGUF
- Speculative decoding
- Model merging and mixing
- Continuous learning and lifelong learning
- Few-shot in-context learning
- Meta-learning for AI
- Retrieval-augmented pre-training
- Multimodal fusion techniques
- Chain-of-thought prompting variants
- Tool-augmented LLMs
- Neuro-symbolic AI
- AI for science: protein folding, drug discovery
- Frontier capabilities and risks

Projects built:

- MoE model implementation
- Long context application
- Quantized model optimization
- Model merging experiments
- Meta-learning system
- Tool-use agent
- Multimodal fusion model
- Cutting-edge technique showcase

Practice: Experiment with latest AI techniques

Building AI Startups & Products

Topics:

- AI startup landscape 2026 and beyond
- Identifying AI opportunities
- Market research for AI products
- Building vs buying foundation models
- Vertical AI applications
- AI-native product design
- Technical co-founder skills
- Building MVP for AI products
- Fundraising for AI startups
- Pitch deck for AI companies
- Go-to-market strategies
- Distribution channels for AI
- Pricing and business models
- Unit economics for AI
- Building AI teams
- Hiring AI talent
- AI startup case studies

Projects built:

- AI startup idea validation
- Market research report
- AI product MVP
- Pitch deck creation
- Business model canvas
- Financial projections
- GTM strategy document
- AI startup playbook

Practice: Develop complete startup strategy

AI Consulting & Freelancing

Topics:

- AI consulting services landscape
- Building consulting practice
- Client discovery and scoping
- AI project estimation
- Proposal writing for AI projects
- Contract negotiation
- AI audit and assessment
- Strategy consulting for AI
- Implementation services
- Training and enablement
- Ongoing support models
- Pricing consulting services
- Building case studies
- Marketing AI services
- Networking and partnerships
- Scaling consulting business

- Freelance platforms for AI

Projects built:

- Consulting service offerings
- Client proposal templates
- AI audit framework
- Project estimation tool
- Case study portfolio
- Marketing website
- Client onboarding process
- Consulting playbook

Practice: Develop consulting business materials

Career Development & Job Search

Topics:

- AI career paths: engineer, researcher, scientist
- Resume optimization for AI roles
- Portfolio building: GitHub, blog, projects
- LinkedIn for AI professionals
- Networking in AI community
- Job search strategies
- Company research: startups vs big tech
- Application process optimization
- AI interview preparation
- Technical interview: coding + ML
- System design for AI systems
- Behavioral interviews
- Take-home assignments
- Salary negotiation for AI roles
- Offer evaluation
- Continuous learning plans
- Building personal brand

Projects built:

- Professional portfolio website
- Technical blog
- 50+ project showcase
- Resume optimization
- LinkedIn profile enhancement
- Interview preparation guide
- System design case studies
- Personal brand strategy

Practice: Complete 50 interview problems, 10 system designs

PHASE 4 COMPLETION - Specialization & Mastery (Week 40-52 (distributed))

Choose Your Specialization

Topics:

- Specialization paths overview
- Path 1: LLM Engineer (production systems, scaling)
- Path 2: AI Research Engineer (paper implementation, novel techniques)
- Path 3: Applied AI Engineer (domain-specific applications)
- Path 4: MLOps/AI Infrastructure Engineer (deployment, monitoring)
- Path 5: AI Product Manager (strategy, roadmap, execution)
- Path 6: Prompt Engineer (advanced prompting, optimization)
- Path 7: AI Ethics/Safety Researcher (alignment, safety)
- Path 8: AI Consultant (client services, implementation)
- Building expertise in chosen path
- Advanced projects for specialization
- Networking in specialized field
- Thought leadership
- Contributing to community
- Teaching and mentoring
- Continuous specialization learning

Projects built:

- Deep dive project in specialization
- 3-5 advanced specialized projects
- Research or implementation in niche
- Community contribution
- Teaching materials creation
- Specialization portfolio

Practice: Become expert in chosen specialization

AI Community & Open Source

Topics:

- AI open source ecosystem
- Contributing to major projects
- Hugging Face contributions
- LangChain contributions
- PyTorch and TensorFlow contributions
- Creating open source AI tools
- Documentation contributions
- Building community around projects
- Speaking at meetups and conferences

- Writing influential blog posts
- YouTube and content creation
- Twitter/X for AI professionals
- Discord and community management
- Mentoring others in AI
- Teaching AI online
- Building reputation
- Long-term community involvement

Projects built:

- Major open source contribution
- Create open source AI library
- Technical blog series (10+ posts)
- Conference talk/workshop
- YouTube channel start
- Community resource creation
- Mentorship program participation

Practice: Active community participation and leadership

The Future of AI & Your Role

Topics:

- Current state of AI (2026)
- AGI timeline and implications
- Multimodal AI evolution
- AI agents and automation
- Specialized AI systems
- AI in every industry
- Regulatory landscape evolution
- AI safety and alignment future
- Economic impact of AI
- Job market evolution
- Skills for the AI future
- Continuous adaptation strategies
- Ethical considerations ahead
- Your unique value proposition
- Building resilient AI career
- Lifelong learning plans
- Contributing to AI's future

Projects built:

- Future of AI research report
- Personal AI vision document
- 5-year career plan
- Lifelong learning roadmap
- Thought leadership pieces
- Future-focused portfolio

Practice: Strategic career planning

Interview Mastery & Career Launch

Topics:

- AI interview process deep dive
- Company-specific preparation: OpenAI, Anthropic, Google
- Startup interviews vs big tech
- Technical rounds: ML fundamentals
- LLM-specific questions
- Coding interviews for AI roles
- System design for AI systems
- ML system design interviews
- Designing AI products in interviews
- Behavioral interviews: STAR method
- Portfolio presentation
- Take-home projects excellence
- Negotiation strategies
- Evaluating offers
- Onboarding preparation
- First 90 days plan
- Career launch checklist

Projects built:

- 100+ interview practice problems
- 10+ system design solutions
- Portfolio presentation deck
- Interview tracking system
- Negotiation preparation
- Career launch materials
- Mock interview completion

Practice: Intensive interview preparation

Final Month - Masterpiece Project & Career Launch (Week 49-52)

Final Masterpiece Project - Part 1

Topics:

- Grand finale project planning
- Innovative AI application idea
- Complete system architecture

- Technology stack selection
- Development workflow
- Agile sprint planning
- Team collaboration (if applicable)
- Code organization and structure
- Documentation strategy
- Testing strategy

Projects built:

- FINAL MASTERPIECE: Revolutionary AI Application
- Option 1: Multi-Agent AI Platform (autonomous agent ecosystem, task marketplace, evaluation system)
- Option 2: Next-Gen RAG System (multi-modal knowledge base, graph integration, self-improving)
- Option 3: AI Research Lab (experiment tracking, model training, evaluation, collaboration)
- Option 4: Vertical AI Solution (industry-specific: legal, medical, finance, education)
- Option 5: AI Development Platform (no-code/low-code AI, model marketplace, deployment)
- Requirements: Novel application, production-ready, fully deployed, comprehensive docs

Final Masterpiece Project - Part 2

Topics:

- Implementation completion
- Advanced features integration
- Performance optimization
- Security hardening
- Comprehensive testing
- User experience refinement
- Production deployment
- Monitoring setup
- Documentation writing
- Demo video creation
- Presentation preparation
- Open source release

Deliverables:

- Production-ready AI application
- Complete source code on GitHub (public)
- Deployed and accessible (live URL)
- Comprehensive documentation
- Architecture diagrams and explanations
- API documentation (if applicable)
- User guide and tutorials
- Demo video (5-10 minutes)
- Technical deep-dive blog post
- Presentation slides
- Open source release with license
- Community contribution plan

Career Launch & AI Leadership

Topics:

- Professional presence establishment
- Portfolio website showcase
- GitHub profile optimization
- LinkedIn thought leadership
- Technical blog authority
- Conference speaking applications
- Community leadership
- Mentorship opportunities
- Teaching and training
- Consulting setup
- Startup plans (if applicable)
- Job offers evaluation
- Career trajectory planning
- Network expansion
- Continuous innovation
- Giving back to community
- Long-term AI career vision

Deliverables:

- Professional portfolio with 50+ projects
- Active GitHub with significant contributions
- Established technical blog (20+ posts)
- Strong LinkedIn presence with thought leadership
- Conference talk submissions
- Open source project maintainer status
- Mentorship program participation
- Teaching/training materials
- Multiple job offers or consulting clients
- Clear 5-year career roadmap
- Community leadership role
- AI network of 500+ connections

Assessment: FINAL COMPREHENSIVE CERTIFICATION EXAM - Complete Generative AI mastery evaluation

- Phase 1 (Months 1-3): 25+ foundational projects - Python tools, ML models, first AI experiments
- Phase 2 (Months 4-6): 30+ LLM projects - API integrations, RAG systems, prompt engineering, vector DBs
- Phase 3 (Months 7-9): 25+ advanced projects - fine-tuned models, AI agents, production systems, MLOps
- Phase 4 (Months 10-12): 20+ cutting-edge projects - custom models, research implementations, specialized apps
- Final: 1 revolutionary masterpiece demonstrating complete expertise

Total projects: 100+ projects from beginner experiments to production AI systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Programming: Python mastery, data structures, algorithms, OOP, functional programming
- Mathematics: Linear algebra, calculus, statistics, probability, optimization
- Machine Learning: Supervised/unsupervised learning, neural networks, deep learning, evaluation
- Generative AI: LLMs, prompt engineering, fine-tuning, RLHF, diffusion models, multimodal AI
- LLM APIs: OpenAI GPT-4, Claude, Gemini, Llama, Mistral, local models
- Frameworks: LangChain, LlamaIndex, Hugging Face Transformers, PyTorch, TensorFlow
- Vector Databases: Pinecone, Weaviate, Chroma, Qdrant, FAISS, pgvector
- RAG Systems: Basic to advanced RAG, multi-hop reasoning, graph RAG, evaluation
- AI Agents: Single agents, multi-agent systems, ReAct, planning, tool use, memory
- Fine-tuning: Full fine-tuning, LoRA, QLoRA, RLHF, DPO, alignment techniques
- Computer Vision: CNNs, object detection, segmentation, diffusion models, multimodal
- Production: MLOps, model serving, monitoring, scaling, security, deployment
- Cloud: AWS SageMaker/Bedrock, Azure OpenAI, GCP Vertex AI, Kubernetes
- Tools: Docker, Git, Jupyter, VS Code, FastAPI, Streamlit, Gradio, MLflow, W&B
- Soft Skills: System design, problem-solving, research, communication, ethics, product thinking

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 8-12 hours
 Projects: 4-6 hours
 Practice problems: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding sessions, office hours, and expert Q&A
 Mentorship: 1-on-1 AI engineer mentorship and career guidance
 Community: Active Discord community with 24/7 peer support and collaboration
 Code review: Expert code reviews for all major projects and assignments
 Career support: Resume review, mock interviews, job referrals to AI companies
 Lifetime access: All content, future updates, new models and techniques coverage
 Interview prep: Mock technical interviews, system design practice with experts
 Research guidance: Paper reading groups, research implementation support
 Startup support: For those building AI products, access to entrepreneurship resources
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion (4 certificates)
 Final certificate: Professional Generative AI Engineer Certification
 Specialization certificates: Certificates for chosen specialization tracks
 Project certificates: Verified project completion badges
 LinkedIn badge: Digital badges for LinkedIn profile showcase
 Industry recognized: Recognized by AI startups, tech giants, and research labs
 Portfolio projects: 50+ production-ready AI projects
 Github verified: Verified GitHub contributions and open source work

WHERE THIS CAN LEAD

- AI Engineer / Generative AI Engineer
- LLM Engineer / LLM Application Developer
- Prompt Engineer / Prompt Engineering Lead
- AI/ML Engineer (focusing on GenAI)
- AI Research Engineer
- Applied AI Scientist
- MLOps Engineer (AI/LLM focus)
- AI Product Manager / AI Product Lead
- AI Solutions Architect
- AI Consultant / AI Strategy Consultant
- Conversational AI Engineer
- AI Agent Developer

- Computer Vision Engineer (Generative)
- Multimodal AI Engineer
- AI Safety/Alignment Researcher
- AI Startup Founder / CTO
- Independent AI Researcher
- AI Educator / Technical Trainer
- AI Freelancer / Contract Engineer

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 4-7 LPA (Junior AI Developer / Prompt Engineer)

Growing roles: Rs 8-15 LPA (AI Application Developer / LLM Engineer); Rs 15-25 LPA (AI Engineer / Senior LLM Engineer)

Experienced roles: Rs 20-40 LPA (Senior AI Engineer / AI Architect / AI Lead); Rs 30-70 LPA (Lead AI Engineer / Principal Engineer)

Freelance: Rs 2,000-10,000/hour based on expertise and specialization

International usa: \$120k-300k USD (AI Engineer to Senior/Staff level)

International europe: €70k-180k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Why should I learn Generative AI now? Isn't it too competitive?

A: Generative AI is the fastest-growing field in tech with massive shortage of skilled professionals. Every company from startups to Fortune 500 is racing to adopt AI. There are 10-20 AI job openings for every qualified candidate. Early movers (now) will have 5-10 year career advantage. It's not too late - it's the perfect time.

Q: Is this Generative AI course suitable for complete beginners with no coding experience?

A: Absolutely! We start from absolute zero - teaching Python from scratch, mathematics needed for AI, ML fundamentals, then gradually moving to LLMs, prompt engineering, fine-tuning, and building production AI systems. Our structured 12-month curriculum takes you from 'What is AI?' to building enterprise-grade AI applications. Thousands of non-technical people have successfully transitioned to AI careers through this program.

Q: What makes this different from other AI courses and ChatGPT tutorials?

A: Unlike superficial ChatGPT tutorials, we teach deep fundamentals: transformer architecture, how LLMs actually work, training models from scratch, fine-tuning, RAG systems, AI agents, production deployment, and MLOps. You'll build 50+ real projects, not just use APIs. We cover OpenAI, Anthropic (Claude), Google (Gemini), open-source models (Llama, Mistral), LangChain, vector databases, and the entire modern AI stack. This is professional AI engineering, not just prompt writing.

Q: What kind of projects will I build and can I use them for my portfolio?

A: You'll build 50+ projects including: ChatGPT clones, RAG-powered Q&A systems, AI agents for automation, fine-tuned models for specific domains, image generation apps, multi-agent systems, production AI APIs, and a final masterpiece project. All projects are production-ready and portfolio-worthy. You'll deploy them to cloud, create demos, and showcase them on GitHub. These projects alone can land you AI jobs at top companies.

Q: Will I learn to train and fine-tune my own AI models or just use APIs?

A: Both! You'll master using APIs (OpenAI, Claude, Gemini) for rapid development AND learn to train transformers from scratch, fine-tune Llama/Mistral models, implement LoRA/QLoRA, use RLHF and DPO for alignment, optimize models for production, and deploy them at scale. We cover the full spectrum from using commercial APIs to training custom models on your data.

Q: What salary can I expect after completing this Generative AI masterclass?

A: International remote roles and freelance consulting can pay more. Actual pay depends on your skills, experience, portfolio, and the market, not on this course.

Q: How is this different from a traditional ML/Data Science course?

A: Traditional ML courses focus on classical ML algorithms (linear regression, random forests, etc.) while we focus on modern Generative AI: Large Language Models (GPT, Claude, Llama), transformer architecture, prompt engineering, fine-tuning, RAG systems, AI agents, diffusion models for images, and multimodal AI. These are the technologies companies are actively hiring for in 2026. You'll learn some foundational ML, but 80% focus is on cutting-edge GenAI.

Q: Will this course help me get a job at AI companies like OpenAI, Anthropic, or Google DeepMind?

A: Yes! Our curriculum is based on job requirements at top AI companies. We cover: transformer architecture, training large models, RLHF, prompt engineering, production deployment, system design for AI, research paper implementation, and building novel applications. We provide interview preparation, system design practice, and coding

challenges. Many graduates have joined leading AI companies, well-funded startups, and research labs.

Q: Can I build my own AI startup or product after this course?

A: Absolutely! You'll learn everything needed to build AI products: ideation, architecture design, MVP development, model selection/fine-tuning, RAG implementation, agent systems, production deployment, monitoring, and scaling. We cover business strategy, monetization, and go-to-market for AI products. Many students have launched successful AI startups, consulting practices, or SaaS products generating significant revenue.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-generative-ai-masterclass-college>

=====

COURSE 62 OF 118: GIT & GITHUB COURSE FOR COLLEGE STUDENTS: TEAM-READY SKILLS

=====

Course link: <https://learn.modernagecoders.com/courses/git-github-version-control-course-for-college-students>

Subject area: Version Control & Developer Tools

Designed for: College students

Who can join: Beginner to job-ready collaborator (college students and recent graduates)

Course length: 24 classes (12 weeks · 2 classes/week)

Time needed: 4-5 hours/week recommended

Certificate: Modern Age Coders 'Git & GitHub for College' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Master Git and GitHub in a live 24-class course for college students: team branching workflows, pull requests and code review, GitHub Actions CI, open source contribution and Git interview prep for internships and placements.

In one line: Every internship, hackathon, and placement interview touches Git. Learn to work like an engineer on a team, and make your GitHub profile do the talking.

ABOUT THE PROGRAMME

Most college students can type git add and git commit, and that is where their Git knowledge ends. Then the first hackathon, group assignment, or internship arrives and everything falls apart: overwritten work, merge conflicts nobody can resolve, a single main branch with fifty messy commits, and a GitHub profile that tells recruiters nothing. This course closes that gap in 24 live, instructor-led classes. You start with version control fundamentals and command-line fluency, master the full undo toolkit, then move to the skills placements actually test: SSH and remotes, professional READMEs, a portfolio on GitHub Pages, the feature-branch workflow, merge vs rebase, pull requests with real code review, GitHub Actions CI with branch protection, open source contribution including Hacktoberfest and GSoC preparation, and the Git interview questions that appear in technical rounds. You finish with a team capstone executed exactly the way an engineering team ships software, plus a full GitHub profile audit, so your commits, pull requests, and green CI checks speak for you before you say a word.

WHAT MAKES THIS COURSE DIFFERENT

- Team-first, not command-first: you do not just memorize git commands, you run the full branch, PR, review, CI loop the way internship teams actually work.
- Live classes with a real instructor, never pre-recorded videos: every session is hands-on, with your terminal open and your questions answered on the spot.
- Career-wired from Class 5: SSH done properly, recruiter-grade READMEs, a profile that survives a 30-second scan, the GitHub Student Developer Pack, and a live portfolio on GitHub Pages.
- Real scenarios, not toy demos: staged three-person merge conflicts, rescue drills with reflog, a git bisect bug hunt, and a full live mock Git interview.
- Open source is in the syllabus: CONTRIBUTING.md etiquette, a first external pull request, and a personal Hacktoberfest and GSoC preparation roadmap.
- Living Syllabus: the curriculum is updated as Git and GitHub ship new features, so you always learn the current tools, not screenshots from three years ago.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac, or Linux) with a stable internet connection; no special hardware needed

Coding experience: Basic familiarity with any one programming language (first-year college Python, C, Java, or JavaScript is plenty); this course teaches the workflow around your code, not the language itself

Accounts: A free GitHub account (GitHub's Terms of Service require users to be 13 or older);

a college ID or institute email lets you claim the GitHub Student Developer Pack, and we walk you through it

Mindset: Willingness to practice in the terminal between classes; version control is muscle memory, and the drills are designed to build it

WHO THIS COURSE IS FOR

Engineering students: B.Tech, BCA, BSc, and diploma students who can write some code but have never worked on a shared repository like a team

Placement aspirants: Pre-final and final year students who know recruiters will open their GitHub profile and want it to say 'engineer', not 'tutorial follower'

Hackathon teams: Students whose group projects and hackathons keep dissolving into zip files on WhatsApp and overwritten work

Open source aspirants: Anyone targeting a first external pull request, Hacktoberfest, or a serious GSoC application

Recent graduates: Fresh graduates and self-taught coders who skipped Git fundamentals and now feel it in every interview and internship

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS FAST

Go from zero to confident daily Git use: how version control thinks, complete CLI fluency, an undo toolkit you can trust in any situation, and repository hygiene that keeps secrets and junk out of history.

Class 1: How Version Control Thinks: Concepts, Install & First Config

Topics:

- The problem Git solves: no more project_final_v2 REALLY_FINAL.zip, safe experiments, and a complete history of who changed what and why
- Centralized vs distributed version control, and why Git (created by Linus Torvalds in 2005 as a free, open source tool for Linux kernel development) is the system you should expect to meet at internships, placements, and beyond
- The core mental model: repository, commit, branch, remote, and how Git stores snapshots of your whole project rather than just file differences
- The three areas every command moves work between: working directory, staging area (the index), and the repository
- Installing Git on Windows (with Git Bash), macOS, and Linux, and verifying with `git --version`
- First-time setup: `git config --global user.name` and `user.email` (the identity stamped on every commit you will ever make), `init.defaultBranch` `main`, and `core.editor`
- Reading your setup back with `git config --list --show-origin`, and understanding system vs global vs local config levels

Projects built:

- Install Git, complete your global configuration, create your first repository with `git init`, and explore the hidden `.git` folder to see where history actually lives

Practice: Run `git config --list` and explain out loud what each line you set does and where it applies.

Assessment: Explain the working directory, staging area, and repository to a classmate by narrating one real file's journey through all three.

Class 2: CLI Fluency: `init`, `add`, `commit`, `log` & `diff` Like You Mean It

Topics:

- Terminal survival kit: `pwd`, `ls`, `cd`, `mkdir`, and why interviewers and internship teams expect command-line comfort
- `git status` as your compass: reading untracked, modified, and staged states without guessing
- Staging with intent: `git add <file>`, `git add -A`, and the professional's `git add -p` to stage changes hunk by hunk
- Committing properly: `git commit -m` for quick saves, `git commit` opening your editor for detailed messages, and what a commit object actually contains (snapshot, author, timestamp, parent, SHA)
- Reading history: `git log`, `git log --oneline --graph --all --decorate`, and filtering with `git log --author` and `git log -n 5`
- Inspecting change: `git diff` for unstaged edits, `git diff --staged` for what will be committed, and `git show <sha>` for any past commit
- HEAD and relative references like `HEAD~1` and `HEAD~3`: the addressing system the entire undo toolkit depends on

Projects built:

- Build a personal-notes repository with at least 8 meaningful commits, using `git add -p` at least twice and reading your own history with `git log --oneline --graph`

Practice: Complete one full edit, stage, commit cycle while narrating how `git status` output changes at each step.

Assessment: Given a repository with mixed staged and unstaged changes, correctly predict the output of `git status` and `git diff --staged` before running them.

Class 3: The Full Undo Toolkit: `restore`, `reset`, `revert` & When Each Is Safe

Topics:

- Undo an unstaged edit with `git restore <file>`, and unstage without losing work with `git restore --staged <file>`
- Fix the last commit with `git commit --amend` for a better message or a forgotten file, and why `amend` rewrites the commit SHA
- `git reset --soft HEAD~1`: move the branch pointer back but keep everything staged, the 'redo my commit' move
- `git reset --mixed HEAD~1` (the default): also unstage, keeping all changes safe in your working directory
- `git reset --hard HEAD~1`: discard changes completely, the rare cases where it is justified, and why you type it slowly
- `git revert <sha>`: a new commit that undoes an old one, the ONLY safe undo on history you have already pushed and shared
- The decision table: private unpushed history means `reset` and `amend` freely; shared pushed history means `revert` only, and what a force push does to your teammates

Projects built:

- Run a scripted rescue drill: `amend` a bad message, `soft-reset` a premature commit, `hard-reset` a failed experiment, and `revert` a bad commit that was already pushed to a practice remote

Practice: For five written scenarios, state which undo command you would use and why it is safe in that exact situation.

Assessment: Correctly choose between `reset` and `revert` for shared vs private history across a set of real situations, losing zero work in the lab.

Class 4: .gitignore for Real Stacks, Secrets & Commit Craft

Topics:

- .gitignore syntax that matters: literal names, *.log wildcards, dir/ trailing slashes, ! negation, and how rule order works
- What real stacks must ignore: `node_modules/` and `dist/` for JavaScript, `venv/` and `__pycache__/` for Python, `.DS_Store`, and IDE folders like `.idea/` and `.vscode/`
- Secrets discipline: `.env` files never enter history; what to do the moment a key IS committed (rotate the key first, then clean up), and why deleting the file in a later commit does not remove it from history
- Untracking without deleting: `git rm --cached <file>` for files that were committed before the ignore rule existed
- A machine-wide ignore file: `git config --global core.excludesFile ~/.gitignore_global` for editor junk on every project
- Commit message craft: imperative subject around 50 characters, blank line, body explaining why, the style engineering teams and open source maintainers expect
- Atomic commits: one logical change per commit, and searching history later with `git log --grep` and the pickaxe `git log -S 'functionName'`

Projects built:

- Take a messy starter project with `node_modules`, a fake `.env`, and editor junk already tracked; write a production-grade `.gitignore`, untrack the offenders with `git rm --cached`, and re-commit cleanly

Practice: Write `.gitignore` rules for a Python project and a Node project from memory, then verify both with `git status`.

Assessment: Your cleaned repository shows no generated files or secrets in `git status`, and your last five commit messages pass the message-quality checklist.

PHASE 2: GITHUB FOR YOUR CAREER

Turn GitHub into a career asset: SSH and remotes done properly, READMEs and licenses recruiters respect, a profile that survives a 30-second scan, the Student Developer Pack claimed, and a live portfolio on GitHub Pages.

Class 5: GitHub, SSH Keys & Your First Push

Topics:

- Git vs GitHub: the tool vs the hosting platform (owned by Microsoft since 2018), and what GitHub Free includes: unlimited public and private repositories and 2,000 GitHub Actions minutes per month
- Creating an account that will carry your career: choosing a username recruiters will read, and GitHub's Terms of Service age requirement of 13 or older
- Authentication today: passwords do not work for pushing; SSH keys vs HTTPS with personal access tokens, and when each fits
- Generating a key: `ssh-keygen -t ed25519 -C 'you@college.edu'`, adding it to `ssh-agent`, and pasting the public key into GitHub Settings under SSH and GPG keys
- Verifying the handshake with `ssh -T git@github.com` and reading what the greeting message confirms
- Wiring local to remote: `git remote add origin git@github.com:you/repo.git`, then `git push -u origin main`, and what the `-u` upstream flag saves you forever after
- Reading a repository page like a developer: commits, branches, browsing files at any point in history, and the Insights tab

Projects built:

- Create or clean up your GitHub account, set up ed25519 SSH authentication end to end, and push your Class 4 repository with `git push -u origin main`

Practice: Break and re-verify your SSH setup once, so you can debug 'Permission denied (publickey)' on your own next time.

Assessment: Your local repository pushes to GitHub over SSH with no password prompt, and the commit history renders correctly online.

Class 6: Clone, Fetch, Pull & Speaking Remote Fluently

Topics:

- git clone <url>: what you actually get (full history, an origin remote, a tracked default branch), and cloning over SSH vs HTTPS
- Remote-tracking branches: what origin/main really is, and why git status says 'ahead 2, behind 1'
- git fetch: download remote changes WITHOUT touching your work, then inspect the gap with git log main..origin/main
- git pull as fetch plus merge, git pull --rebase for linear history, and setting pull.rebase true the way many teams do
- Push rejections decoded: 'non-fast-forward' means fetch first, integrate, then push; force-pushing a shared branch is never the reflex answer
- Working from two machines (hostel laptop and lab computer): the pull-before-work, push-before-leave rhythm that prevents divergence
- Remote management: git remote -v, git remote show origin, and renaming or removing remotes safely

Projects built:

- Simulate a two-computer workflow using two local clones of your own repository: make diverging changes, inspect with git fetch, reconcile with git pull --rebase, and push

Practice: Trigger one rejected push on purpose and resolve it with fetch and rebase instead of a force push.

Assessment: Explain the exact difference between fetch and pull, then demonstrate both live on a repository with pending remote changes.

Class 7: READMEs & Licenses: Repos Recruiters Actually Read

Topics:

- What a recruiter or interviewer does in 30 seconds on your repository: README first, commit history second, code third, which makes the README your landing page
- Professional README structure: one-line pitch, screenshot or GIF, features, tech stack, setup steps that actually work, usage, and what you would improve next
- Markdown fluency: headings, code fences with language tags, tables, task lists, images, and status badges
- Naming and describing repositories: expense-tracker-django beats project2-final; filling the description field and topics so your work is searchable
- Licenses in plain language: MIT (permissive, common for student projects), Apache-2.0 (adds a patent clause), GPL-3.0 (copyleft), and what 'no license' legally means for your code
- Adding a LICENSE file through GitHub's license chooser, and when a college project should have one
- Cleaning your existing account: archiving dead experiments, deleting empty repos, and making 3 repositories excellent instead of 20 mediocre

Projects built:

- Rewrite the README of your best existing project to the professional template, add a screenshot and a suitable license, and complete the repo description and topics

Practice: Review a classmate's README against the recruiter checklist and give two concrete, actionable improvements.

Assessment: A stranger can clone and run your project using only your README, verified by a classmate actually doing it.

Class 8: Profile README, Pinned Repos & the Student Developer Pack

Topics:

- The username/username repository: GitHub renders that repo's README at the top of your profile page, your one free billboard for recruiters
- What belongs in a profile README: who you are, what you build, current focus, top projects with links, and how to reach you; what to skip (walls of badges and fake stats)
- Pinning your best 6 repositories and choosing them for range: a web app, a tool, a contribution, not six versions of one tutorial
- The contribution graph: what actually counts toward it, why consistent real work beats gaming it, and how interviewers read it
- The GitHub Student Developer Pack: free for verified students enrolled at a degree or diploma granting school, and it includes GitHub Pro while you are a student; claim it at education.github.com with your college ID or institute email
- GitHub Copilot for students: Copilot has a free tier, and GitHub Education has offered free Copilot access to verified students; availability can change, so always check education.github.com
- Your public identity checklist: consistent handle, photo, a bio with your college and interests, location, and a pinned portfolio link

Projects built:

- Ship your profile README in the username/username repository, pin your best repositories, complete your bio, and submit your Student Developer Pack verification

Practice: Study three impressive real student profiles and write down one idea to borrow from each.

Assessment: Your profile passes the class rubric: profile README live, three or more pinned repos with descriptions, complete bio, and Pack verification submitted.

Class 9: Ship Your Portfolio on GitHub Pages

Topics:

- GitHub Pages: static hosting GitHub provides free, publishing files directly out of your repo, and why a live URL beats 'code available on request' on a resume

- User site vs project site: username.github.io serves at the root domain, while every other repository can publish under /repo-name
- Enabling Pages in Settings under Pages: deploy from a branch (main or a /docs folder) vs deploying with a GitHub Actions workflow
- Building a clean single-page portfolio: about, projects with live links and repo links, skills, and contact; plain HTML and CSS is enough
- Debugging deploys: a custom 404.html, relative path gotchas, and why case-sensitive file names break images that worked locally
- Pointing a custom domain at Pages with a CNAME record (optional) and enforcing HTTPS
- The portfolio loop from now on: every future project gets a live demo link in its README and a card on your portfolio site

Projects built:

- Deploy a portfolio site at username.github.io featuring at least three projects with live links, then add the portfolio URL to your GitHub bio and resume

Practice: Push one content update and watch GitHub Pages rebuild and redeploy it live.

Assessment: Your portfolio URL loads publicly, every link resolves, and it is referenced from your GitHub profile.

PHASE 3: TEAM WORKFLOWS

Work the way engineering teams work: feature branches, clean history with merge and rebase, real conflict resolution, pull requests with meaningful review, and issues and Projects boards that keep group assignments and hackathons on track.

Class 10: Branches & the Feature-Branch Workflow

Topics:

- What Git calls a branch: a lightweight named reference to a commit SHA, so creating, switching, and deleting branches costs nothing
- Modern commands: `git switch -c feature/login`, alongside the older `git checkout -b` you will still see in every tutorial and team
- The feature-branch workflow: main stays always-working, and every feature, fix, and experiment gets its own branch
- Branch naming conventions teams use: `feature/`, `fix/`, `chore/`, and `docs/` prefixes plus a short descriptive slug
- Switching branches with uncommitted changes: when Git blocks you, why, and a first look at `git stash` as the escape
- Fast-forward merges: `git merge feature/login` when main has not moved, and reading the result in `git log --graph`
- Cleaning up: `git branch -d` (safe, merged work only) vs `git branch -D` (force), plus listing with `git branch -a` and `git branch --merged`

Projects built:

- Build two features of a small app on separate branches (`feature/navbar` and `feature/contact-form`), merge both into main, and delete the merged branches

Practice: Draw your repository's commit graph on paper mid-exercise, then verify it against `git log --oneline --graph --all`.

Assessment: Your history shows main receiving two feature branches through merges, with no work ever committed directly to main.

Class 11: Merge vs Rebase & Keeping History Clean

Topics:

- The three-way merge: how Git finds the common ancestor and combines both sides into a merge commit
- `git merge --no-ff` to force a merge commit that records the feature as one unit, and why some teams mandate it
- `git rebase main` from your feature branch: replaying your commits onto a new base for a straight-line history
- The golden rule of rebase: never rebase commits that others already have; rewriting shared history breaks every teammate's repository
- Interactive rebase: `git rebase -i HEAD~4` to squash WIP commits, reword messages, and reorder work before opening a pull request
- `git merge --squash`: landing a messy branch as one clean commit, and how it differs from GitHub's squash-merge button
- Choosing a team policy: merge-commit history vs linear rebase history, and reading each style fluently in `git log --graph`

Projects built:

- Take a feature branch with five messy WIP commits, use `git rebase -i` to squash and reword them into two clean commits, then rebase onto an updated main and merge

Practice: Integrate the same branch in two clone copies, once with merge and once with rebase, and compare the resulting graphs.

Assessment: Explain to the class when rebase is safe and when it is forbidden, giving one correct example of each.

Class 12: Resolving Merge Conflicts Under Deadline Pressure

Topics:

- The anatomy of a conflict: both branches touched identical lines since their common ancestor, so Git stops and makes you decide
- Reading conflict markers: `<<<<<< HEAD`, `====`, and `>>>>>> feature`, and what each side represents in a merge vs during a rebase

- Resolving by hand and in VS Code's merge editor: accept current, accept incoming, accept both, or write the correct combination yourself
- Escape hatches: `git merge --abort` and `git rebase --abort` return you to safety, and `git rebase --continue` moves forward after each fixed commit
- Taking a whole side deliberately: `git checkout --ours <file>` and `git checkout --theirs`, and how their meaning flips during a rebase
- Conflict prevention as a team habit: pull main into your branch often, keep pull requests small, and split files everyone edits
- The full drill: conflicting a shared team file, resolving it, staging the resolution, and completing the merge commit

Projects built:

- Run a staged three-person conflict lab: all three teammates edit the same function on separate branches, then merge every branch into main resolving each conflict correctly

Practice: Abort a scary conflicted merge with `git merge --abort`, then redo it and resolve it for real.

Assessment: You resolve a two-file conflict with the intended code surviving and no conflict markers left behind, verified with `git diff`.

Class 13: Pull Requests With Meaningful Descriptions

Topics:

- What a pull request is: a request to merge a branch, plus the conversation, review, and checks that professionalize the merge
- Opening a PR properly: base vs compare branch, a title that summarizes the change, and the shortcut link Git prints after every push
- The PR description that gets fast reviews: what changed, why, how you tested it, screenshots for UI work, and where reviewers should focus
- Linking work: 'Closes #12' auto-closes the issue on merge, and referencing related PRs and commits by SHA keeps the trail connected
- Draft PRs for early feedback, requesting specific reviewers, and updating a PR by pushing new commits to the same branch
- GitHub's three merge buttons: create a merge commit vs squash and merge vs rebase and merge, and when a team picks each
- PR templates: a `.github/PULL_REQUEST_TEMPLATE.md` so every teammate's pull request arrives complete

Projects built:

- Open a real PR on your team repository with a complete description, screenshots, and a linked issue; get it reviewed by a classmate and squash-merge it

Practice: Rewrite a terrible one-line PR description ('fixed stuff') into one a reviewer can actually act on.

Assessment: Your PR history shows a described, linked, reviewed pull request merged with the strategy your team agreed on.

Class 14: Code Review: Giving & Receiving Like a Professional

Topics:

- Why review exists: catching bugs early, spreading knowledge, and keeping a codebase consistent; it is the daily collaboration ritual at real companies
- Reviewing a diff on GitHub: the Files changed tab, viewed checkboxes, and commenting on specific lines or line ranges
- Suggested changes: proposing exact replacement code the author can apply with one click
- The three review verdicts: Comment, Approve, and Request changes, and what each one signals to the author
- Giving feedback that lands: comment on the code not the person, ask questions before issuing orders, and mark non-blockers with a 'nit:' prefix
- Receiving feedback without ego: respond to every comment, push fixes to the same branch, re-request review, and disagree with reasons rather than defensiveness
- Review culture tools: a CODEOWNERS file to auto-assign reviewers, and requiring one approval before merge

Projects built:

- Complete a paired review cycle: review a classmate's PR leaving at least one suggested change and one question, and shepherd your own PR from Request changes to Approved

Practice: Rewrite three blunt review comments into constructive ones without losing their technical point.

Assessment: Your review on a peer PR includes a line-level suggestion the author applied, and your own PR reached Approved after addressing feedback.

Class 15: Issues, Projects, Forks & Running a Team Repo

Topics:

- Issues as your team's shared brain: writing a reproducible bug report and a clearly scoped feature request
- Labels (bug, enhancement, good first issue), assignees, and milestones that group issues toward a deadline like a semester submission
- GitHub Projects: a board with To do, In progress, and Done columns that updates automatically as linked issues and PRs move
- Issue templates and Markdown task lists so bug reports arrive with the information you need
- Fork vs clone: a fork is your own server-side copy under your account, the standard route for contributing when you lack push access
- Keeping your fork current: register the original repo with `git remote add upstream`, pull its history with `git fetch upstream`, integrate with `git merge upstream/main`, or click Sync fork on GitHub

- The hackathon playbook: repo plus board plus issues plus branch-per-feature plus PRs, set up in the first 30 minutes so the team never blocks

Projects built:

- Set up your capstone team repository: issue templates, labels, a milestone, a Projects board with 10+ scoped issues, and a fork-based contributor test run with upstream syncing

Practice: Convert a vague idea ('make the app better') into three well-scoped, labeled, assignable issues.

Assessment: Your team board shows issues flowing from Todo to Done via linked PRs, and every teammate has synced a fork with upstream at least once.

----- PHASE 4: AUTOMATION & OPEN SOURCE -----

Automate quality and step into the open source world: CI that runs your tests on every push, branch protection that enforces discipline, professional versioning and releases, and a real contribution roadmap through Hacktoberfest and GSoC preparation.

Class 16: GitHub Actions: CI That Runs Your Tests on Every Push

Topics:

- What continuous integration is and why teams refuse to live without it: every push gets built and tested automatically, so 'it works on my machine' dies
- Anatomy of a workflow: `.github/workflows/ci.yml` with name, on (push and pull_request), jobs, runs-on: ubuntu-latest, and steps
- Reusable building blocks: uses: actions/checkout@v4, plus actions/setup-node or actions/setup-python with the language version you need
- Running your real test command (npm test or pytest) as a step, and letting a failing test fail the whole build
- Reading the Actions tab: live logs, the red X and green check on commits and PRs, and re-running failed jobs
- Your free budget: the GitHub Free plan includes 2,000 GitHub Actions minutes per month, and public repositories are the friendliest place to learn CI
- Debugging CI failures: environment differences, missing dependencies, and printing your way to the root cause

Projects built:

- Add a working CI workflow to your team repository that installs dependencies and runs the test suite on every push and pull request, then break it and fix it once on purpose

Practice: Read one failed Actions log top to bottom and identify the exact failing line before touching any code.

Assessment: Your repository shows green checks on the latest commits and one PR that was blocked by a failing test and then fixed.

Class 17: Status Checks, Branch Protection & Team Discipline

Topics:

- Required status checks: a pull request cannot merge until CI is green, turning tests from a suggestion into a gate
- Branch protection rules and rulesets on main: require a pull request before merging, require one approving review, and block force pushes and deletions
- Why protection matters for student teams: nobody, including you at 2 a.m. before a deadline, can push broken code straight to main
- Keeping reviews honest: dismissing stale approvals when new commits land, and requiring conversation resolution before merge
- Speeding CI up: caching dependencies with actions/cache and splitting independent jobs to run in parallel
- A build status badge in your README that shows live CI health to every visitor, including recruiters
- The protected-main loop end to end: branch, push, PR, review, green checks, merge; the exact rhythm you will run at every internship

Projects built:

- Protect main on your team repository (PR required, one review, CI check required, force pushes blocked), then prove it by attempting a direct push and watching it fail

Practice: Attempt to merge a PR with failing CI and another with no review, and document exactly what blocks each.

Assessment: Demonstrate your full protected workflow from new branch to merged PR, with review and green checks, in one live run-through.

Class 18: Conventional Commits, Tags & Releases

Topics:

- Conventional Commits: feat:, fix:, docs:, refactor:, test:, and chore:, plus scopes like feat(auth): and the BREAKING CHANGE footer
- Why the convention pays off: readable history, automated changelogs, and commit messages interviewers actually compliment
- Semantic versioning: MAJOR.MINOR.PATCH, what bumps each number, and how it connects to conventional commit types
- Lightweight vs annotated tags: `git tag v1.0.0` vs `git tag -a v1.0.0 -m 'First stable release'`, and why releases use annotated tags
- Pushing tags explicitly: `git push origin v1.0.0` (or `--tags`), and checking out any tagged version later to reproduce it
- GitHub Releases: turning a tag into a release page with notes, an auto-generated changelog from merged PRs, and downloadable assets

- Release hygiene for team projects: tagging the submission version of a group assignment so 'which version did we demo?' is never a question again

Projects built:

- Adopt conventional commits on your team repository, then cut v1.0.0: an annotated tag pushed to GitHub and a Release with auto-generated notes plus a short highlights section

Practice: Rewrite ten of your older commit messages as conventional commits on a practice branch.

Assessment: Your repository shows a published v1.0.0 release, and your last ten commits all follow the conventional-commit format.

Class 19: Contributing to Open Source: Etiquette & Your First PR

Topics:

- Why open source is the strongest signal on a student profile: real code, review by strangers, and proof you can work inside someone else's codebase
- Finding a first target: 'good first issue' and 'help wanted' labels, projects you already use, and matching issue difficulty to your current skill
- Read before you write: CONTRIBUTING.md, CODE_OF_CONDUCT.md, the issue tracker, and recently merged PRs that reveal the project's norms
- The contribution loop: fork the repo, clone your fork, branch, make one focused change, push to your fork, and open a PR against upstream
- Upstream etiquette: comment on the issue before starting, keep the PR small, follow the project's style and commit conventions, and stay patient and responsive with maintainers
- Handling maintainer review: expect change requests, iterate on the same branch, and never take a closed PR personally
- Non-code contributions that count: documentation fixes, reproducible bug reports, and issue triage are all legitimate first steps

Projects built:

- Make one real contribution attempt: pick an external project, claim or file an issue, and open a properly formed PR from your fork (a documentation fix is a valid target)

Practice: Read the CONTRIBUTING.md of two major projects and list three rules that differ between them.

Assessment: You have a live PR, or a well-formed filed issue plus a prepared branch, on an external repository following that project's contribution rules.

Class 20: Hacktoberfest, GSoC & Your Open Source Roadmap

Topics:

- Hacktoberfest: the annual October event that rewards quality pull requests to participating projects, and why your target list should be ready in September
- Quality over spam: why low-effort PRs get marked invalid, and how maintainers spot them instantly
- Google Summer of Code: mentored, stipend-backed contribution to real open source organizations; what the program involves and why you must check the current year's eligibility rules yourself
- The GSoC timeline shape: organizations announced, application window, community bonding, then coding periods, and why strong applicants engage with an org months before applying
- Choosing an organization: match your language and interests, study their project ideas list, and build credibility with small merged PRs first
- What a strong proposal contains: specific understanding of the project, a realistic milestone plan, and evidence you have already contributed
- Your personal 6-month open source roadmap: from a first documentation PR to a consistent contributor profile that stands out in placements

Projects built:

- Write your open source roadmap: three target projects with justification, five candidate issues, and a month-by-month plan toward Hacktoberfest or a GSoC application

Practice: Study one past accepted GSoC proposal from any organization's public archives and note what made it concrete.

Assessment: Present your roadmap and defend your project choices and timeline in a five-minute review with your mentor.

PHASE 5: ADVANCED GIT & CAPSTONE

The finishing layer: advanced rescue and history-debugging tools, the interview questions that filter candidates, and a team capstone shipped with the complete professional workflow, followed by your certificate.

Class 21: stash, cherry-pick & rebase: The Rescue Kit

Topics:

- git stash push -m 'wip: form validation': shelving work instantly when you must switch branches right now
- Managing the shelf: git stash list, git stash pop vs git stash apply (pop applies AND drops), git stash drop, and including untracked files with -u
- git cherry-pick <sha>: copying a single commit onto your current branch, with -x to record its origin, and when it beats merging a whole branch
- Cherry-pick under fire: multi-commit ranges with git cherry-pick A..B, plus --continue and --abort when conflicts appear
- git rebase: the private journal of everywhere HEAD has been, surviving resets and branch deletions
- The classic rescues: undo a wrong git reset --hard with git reset --hard HEAD@{1}, and

- resurrect a deleted branch with `git branch saved <reflog-sha>`
- Detached HEAD demystified: how you get there (checking out a SHA or tag), why commits made there can be lost, and escaping cleanly with `git switch -c`

Projects built:

- Complete the rescue gauntlet: stash mid-feature to hotfix main, cherry-pick the hotfix back to your branch, then 'accidentally' hard-reset a branch and recover it fully via reflog

Practice: Delete a branch containing unmerged work, then bring every commit back using only `git reflog`.

Assessment: You recover a hard-reset commit and a deleted branch unaided, explaining each reflog entry you used along the way.

Class 22: git bisect & Debugging With History

Topics:

- The scenario: the app broke somewhere in the last 60 commits, and reading them one by one is not a strategy
- Binary search over history: `git bisect start`, `git bisect bad`, `git bisect good v1.0.0`, then testing each checkout Git hands you
- Marking your way to the culprit: bisect halves the range each step, so even a long history takes only a handful of tests
- Full automation: `git bisect run pytest` (or any script whose exit code means pass or fail) finds the bad commit while you watch
- Finishing up: reading the 'first bad commit' report, `git bisect reset`, then fixing forward or reverting the culprit
- `git blame -L 10,25 app.py`: which commit last touched these exact lines, used to understand intent rather than assign blame
- History as a search engine: `git log -S 'functionName'` (the pickaxe) for when code appeared or vanished, and `git log -p <file>` to walk a file's evolution

Projects built:

- In a provided repository with a hidden bug among 40+ commits, find the exact breaking commit with `git bisect`, first manually and then with `bisect run`, and revert it

Practice: Use `git blame` and `git log -S` to answer three 'when and why did this change?' questions about a real codebase.

Assessment: You identify the breaking commit's SHA and author date, and your written explanation matches the actual bug introduced there.

Class 23: Git Interview Questions & Whiteboard Scenarios

Topics:

- The classics, answered precisely: `git fetch` vs `pull`, `merge` vs `rebase`, `reset` vs `revert`, `clone` vs `fork`, and what the staging area is actually for
- 'How do you undo a commit that is already pushed?': the revert answer interviewers want, and why 'reset and force push' is a red flag on shared branches
- Explaining detached HEAD, the difference between `HEAD~2` and `HEAD^2`, and why branches are just pointers, with a whiteboard diagram you can redraw from memory
- Scenario drills: you committed to main instead of a feature branch; a teammate force-pushed over your work; a secret key entered history; `.gitignore` seems to be ignored
- 'Describe your team workflow' answers that prove experience: branch protection, PRs, reviews, CI checks, and conventional commits, evidenced by your capstone repo
- Walking an interviewer through your GitHub profile: choosing the repository, narrating one PR with review, and showing the CI pipeline you built
- Rapid-fire command recall under pressure: mock rounds covering the commands this course drilled, with follow-up 'what if' probes

Projects built:

- Complete a live mock Git interview: 15 rapid-fire questions plus 2 whiteboard scenarios, then write model answers for every question you missed

Practice: Run a 10-minute peer quiz each day this week, swapping interviewer and candidate roles.

Assessment: Clear the final mock interview against your mentor's rubric and deliver a confident 3-minute walkthrough of your own GitHub profile.

Class 24: Capstone: Ship a Team Project the Professional Way + Certificate

Topics:

- Capstone kickoff: a small team application, or your real group assignment, run exactly the way an engineering team would run it
- Repository setup from memory: protected main, issue templates, labels, a milestone, and a Projects board of scoped issues
- Executing the loop: feature branches, conventional commits, PRs with real descriptions, peer review with suggested changes, and green CI required to merge
- Handling the mess like a professional: at least one real conflict resolved and one revert or reflog rescue documented in the repo
- Cutting the release: an annotated `v1.0.0` tag, a GitHub Release with generated notes, and a live demo link on GitHub Pages where applicable
- The GitHub profile audit: profile README, pins, portfolio link, contribution history, and your capstone repository as the centerpiece recruiters land on
- Presenting your work to the class and earning the Modern Age Coders 'Git & GitHub for College' certificate

Projects built:

- Deliver the capstone: a team repository showing 15+ issues, 10+ merged PRs with reviews, protected main, green CI, a `v1.0.0` release, and a demo link, plus your fully audited GitHub profile

Practice: Dry-run your capstone presentation with a teammate playing a skeptical interviewer.
Assessment: Final review: the capstone repository meets the full professional-workflow checklist and your profile audit passes; success earns the certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: your first configured repository, a personal-notes repo with clean history built using git add -p, a complete undo rescue drill, and a messy project cleaned with a production-grade .gitignore
- Phase 2: an SSH-authenticated first push, a two-clone remote workflow simulation, a recruiter-grade README with a license, a profile README with pinned repos and a Student Developer Pack claim, and a live portfolio on GitHub Pages
- Phase 3: two features merged through the feature-branch workflow, a messy branch cleaned with interactive rebase, a three-person conflict lab, a fully described and reviewed pull request, a paired code review cycle, and a team repo with issues, labels, and a Projects board
- Phase 4: a working GitHub Actions CI pipeline, a protected main branch proven by a blocked direct push, a v1.0.0 tagged release with notes, a real external open source contribution attempt, and a personal Hacktoberfest and GSoC roadmap
- Phase 5: the rescue gauntlet with stash, cherry-pick, and reflog, a git bisect bug hunt in a 40-commit repository, a full live mock Git interview, and the team capstone with a release, green CI, and an audited GitHub profile
- Total: 24 hands-on deliverables, one per class, finishing with a team capstone and a placement-ready GitHub profile

Total projects: 24 hands-on deliverables, one per class, culminating in a team capstone shipped with the full professional workflow

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Git: init, status, add -p, commit, log, diff, and the complete undo toolkit (restore, reset soft/mixed/hard, revert, amend)
- Remotes: SSH keys, clone, fetch vs pull, push, tracking branches, and multi-machine workflows
- Branching: the feature-branch workflow, merge vs rebase, interactive rebase, and calm conflict resolution
- GitHub Collaboration: pull requests, code review, suggested changes, issues, labels, milestones, and Projects boards
- Automation: GitHub Actions CI, required status checks, branch protection, dependency caching, and build badges
- Release Engineering: conventional commits, semantic versioning, annotated tags, and GitHub Releases
- Advanced Rescue and Debugging: stash, cherry-pick, reflog recoveries, git bisect, and blame plus pickaxe history searches
- Career Skills: recruiter-grade READMEs and profile, a GitHub Pages portfolio, open source etiquette, and Git interview answers

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks
Hands on git work: 1.5-2 hours running the real git commands and GitHub workflows from each class
Project time: 1-1.5 hours completing each class deliverable on your own repositories
Review and doubts: 30-60 minutes reviewing your work, clearing doubts over WhatsApp, and preparing for the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for setup issues, scary git states, and workflow questions
Small batches: Small batches, with mini-batch and 1-on-1 options, so every student's terminal gets individual attention
Mentor guidance: Live guidance from mentors who use Git and GitHub on real engineering teams daily
Project feedback: Feedback on every class deliverable, plus a personal capstone and profile review before certification
Living syllabus: Curriculum updated as Git and GitHub ship new features, at no extra cost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'Git & GitHub for College' certificate, earned by presenting your Class 24 team capstone
Project portfolio: A capstone repository demonstrating the full professional workflow, plus a live portfolio site on GitHub Pages
Github profile: A recruiter-ready GitHub profile: profile README, pinned repositories, real pull requests, reviews, and green CI checks

WHERE THIS CAN LEAD

- Internship-ready developer who can join any team repository and contribute from day one

- Open source contributor with merged external pull requests and a Hacktoberfest or GSoC trajectory
- Hackathon team lead who sets up the repo, board, and workflow in the first 30 minutes
- Placement candidate with a defensible GitHub profile and confident answers to Git interview questions
- A foundation for every developer role: web, app, data, and ML teams all live in Git and GitHub daily

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Why do I need a Git and GitHub course for placements?

A: Because almost every technical recruiter and interviewer will open your GitHub profile, and almost every internship starts with 'clone the repo and open a PR'. Knowing git add and commit is not the same as working confidently on a shared repository with branches, reviews, and CI. This course teaches the team-workflow side that college courses skip, and finishes with dedicated Git interview preparation.

Q: What is the difference between Git and GitHub?

A: Git is the version control tool itself: free, open source, and distributed, created by Linus Torvalds in 2005 for Linux kernel development, and it runs entirely on your machine tracking your project's history. GitHub hosts Git repositories online so teams can collaborate through pull requests, issues, and reviews; Microsoft has owned it since 2018. This course teaches both deeply: Git for control of your history, GitHub for teamwork and your public developer profile.

Q: Is GitHub free for college students?

A: Yes, for everything this course needs. Beyond that, verified students enrolled at a degree or diploma granting school can claim the GitHub Student Developer Pack free of charge, and it includes GitHub Pro for as long as you are a student. We walk you through claiming it at education.github.com in Class 8.

Q: Do I need to be good at coding before joining?

A: You only need basic familiarity with one programming language, at the level of a first-year college course. Git manages code rather than being a language itself, so the course focuses on workflow, collaboration, and career skills around whatever code you write. Complete command-line beginners are fine: Classes 1 and 2 build terminal fluency from scratch.

Q: Are the classes live or recorded?

A: Every class is live and interactive with a real instructor; we never teach from pre-recorded videos. You run the commands on your own machine during class, get unstuck in real time, and have WhatsApp doubt support between sessions. Group batches are capped at 10 students, with mini-batch and 1-on-1 options available.

Q: Will this course help me with Hacktoberfest and GSoC?

A: Yes, Phase 4 is built for exactly that. You learn to find beginner-friendly issues, read CONTRIBUTING.md, follow upstream etiquette, and open a properly formed pull request from a fork, then build a personal roadmap targeting Hacktoberfest and a future GSoC application. Program rules and timelines change from year to year, so we also teach you how to verify the current cycle's details yourself.

Q: What Git questions are asked in coding interviews?

A: The most common are the difference between fetch and pull, merge vs rebase, reset vs revert, how to undo a pushed commit, what detached HEAD means, and describing your team workflow. Class 23 is a dedicated interview class with rapid-fire drills, whiteboard scenarios, and a full live mock round. Better still, the whole course gives you real experience so your answers come from practice, not memorization.

Q: How long does it take to learn Git and GitHub properly?

A: This course runs 24 live classes over about 12 weeks, at 2 classes per week, with 4-5 hours of weekly commitment recommended. The basic commands take days, but team readiness, conflict resolution, CI, and open source etiquette need structured practice, which is what the drills and capstone provide. You finish with a certificate, a live portfolio, and a capstone repository, not just theory.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/git-github-version-control-course-for-college-students>

=====

COURSE 63 OF 118: GIT & GITHUB COURSE FOR TEENS: VERSION CONTROL FROM ZERO

=====

Course link: <https://learn.modernagecoders.com/courses/git-github-version-control-course-for-teens>

Subject area: Version Control & Developer Tools

Designed for: Teens

Who can join: Beginner-friendly (ages 13-18) to confident GitHub collaborator
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 3-4 hours/week recommended
Certificate: Modern Age Coders 'Git & GitHub for Teens' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Learn Git and GitHub from zero in 24 live classes for teens 13-18: commits, branching, pull requests, merge conflicts, a free GitHub Pages portfolio and your first real open source contribution.

In one line: Never lose your work again: master the one tool every real developer uses daily, publish a live portfolio site, and make your first open source contribution in 24 live classes.

ABOUT THE PROGRAMME

Every serious young coder eventually loses work: the game mod that got overwritten, the school project that vanished the night before submission, the website folder named final-version-3-REAL. This course ends that forever. In 24 live, project-based classes for teens aged 13 to 18 (GitHub itself requires users to be 13 or older, so this is the perfect starting age), you learn Git and GitHub from absolute zero, with no terminal experience assumed. You will master real commands like git init, git commit, git branch, git merge and git rebase, resolve merge conflicts calmly, open and review pull requests, publish a free portfolio website with GitHub Pages, automate it with GitHub Actions, and make a genuine contribution to a real open source project. Every class is taught live by an instructor, using the same small-batch, mentorship-first method Modern Age Coders has used since 2020 to teach 10,000+ students across 25+ countries. You finish with a GitHub profile that colleges and internship reviewers can actually read: pinned projects with proper READMEs, a live portfolio URL, and a real pull request to someone else's codebase.

WHAT MAKES THIS COURSE DIFFERENT

- Zero terminal assumed: Class 2 is a full, friendly terminal bootcamp (pwd, ls, cd, mkdir) before any Git command is expected of you.
- Built around YOUR projects: practice on your own website, Python script, or game mod files; we provide starter projects if you don't have one yet.
- Live and interactive: every class is taught live by a real instructor in a small batch, never pre-recorded videos.
- Real developer workflows, not toy demos: branches, pull requests, code reviews, project boards, and a two-person team project on one shared repo.
- Living Syllabus: the curriculum is updated as Git and GitHub ship new features, so you always learn the current tool, not a frozen tutorial.
- Real outcomes: a polished GitHub profile, a live GitHub Pages portfolio, one external open source pull request, and a certificate.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac, or Linux) with a stable internet connection: that is the main requirement
Coding experience: None required for Git itself, and zero terminal experience is assumed. If you have any hobby project (a webpage, Python script, or game mod files) you can practice on it; if not, we provide starter projects
Accounts: A free GitHub account, created in Class 6. GitHub's Terms of Service require users to be 13 or older, which is exactly why this course starts at 13
Mindset: Willingness to type commands instead of clicking, and to break things on purpose in practice repos: that is how the undo skills become permanent

WHO THIS COURSE IS FOR

Teen coders: Ages 13-18 already building websites, Python projects, or games who are one accidental overwrite away from disaster
School project teams: Students doing group coding assignments or hackathons who are still emailing zip files back and forth
Game modders and hobby builders: Teens tinkering with mods, scripts, and side projects who want every experiment safely versioned
Future cs applicants: Students aiming at computer science who want a GitHub profile that strengthens college and internship applications
Self taught beginners: Teens learning to code from tutorials who keep hearing 'just clone the repo' and want to finally understand what that means

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: GIT FOUNDATIONS

Understand why version control exists, install Git on your own machine, get comfortable in the terminal from zero, and master the everyday cycle of init, status, add, commit, log, diff, and safe undo.

Class 1: Why Version Control? (The Lost Homework Horror Story)

Topics:

- The horror story every student knows: project-final-v2-REAL(3), an overwritten game mod, a school submission lost the night before deadline, and how version control ends it forever
- What a version control system actually does: full snapshots of your project you can return to at any moment, with a message explaining each one
- Git in one line: a free, open source distributed version control system created by Linus Torvalds in 2005 to manage Linux kernel development
- Centralized vs distributed: why every Git user carries the complete project history on their own machine, so no single crash can destroy it
- Git vs GitHub: Git is the tool on your computer, GitHub (owned by Microsoft since 2018) is the website where Git repositories live online and people collaborate
- The four words you will use every day, defined simply: repository, commit, branch, remote
- Installing Git: the git-scm.com installer on Windows (Git Bash included), Homebrew or Xcode command line tools on Mac, `sudo apt install git` on Linux, then verify with `git --version`

Projects built:

- Install Git on your own computer, run `git --version` to prove it works, and write a half-page version-control horror story from your own life (or a friend's) that this course will make impossible

Practice: Explain the difference between Git and GitHub to a family member in under a minute without notes.

Assessment: Screenshot of `git --version` output on your machine plus a two-sentence definition of a commit in your own words.

Class 2: Terminal Survival Skills (Zero Assumed)

Topics:

- Opening your terminal: Git Bash on Windows, Terminal on Mac and Linux, and why developers live here instead of clicking through folders
- Finding your way: `pwd` to see where you are, `ls` to list files (`ls -a` reveals hidden ones like `.git`), `cd foldername`, `cd ..` to go up, `cd ~` to go home
- Making and removing things: `mkdir` for folders, creating empty files, and why `rm` deserves respect before you press Enter
- Speed skills: Tab completion so you never mistype a filename, and the up arrow to replay previous commands
- Paths made simple: absolute vs relative paths, and why file names with spaces need quotes
- Escape hatches: `clear` to clean the screen, `Ctrl+C` to cancel a stuck command, and `q` to exit the pager that `git log` opens
- One-time Git setup: `git config --global user.name 'Your Name'`, `git config --global user.email`, `git config --global init.defaultBranch main`, then check it all with `git config --list`

Projects built:

- Build a coding-projects folder tree (three nested folders and two files) entirely from the terminal, no mouse allowed, and configure your Git name and email

Practice: Navigate from your home folder to your deepest project folder and back using only `cd`, `pwd`, and Tab completion.

Assessment: Live terminal check: reach a named folder, list its hidden files, and show your `git config user.name` without help.

Class 3: Your First Repository: init, status, add, commit

Topics:

- `git init`: turning any folder into a repository, and the hidden `.git` folder that IS your history (never delete it)
- The three-room mental model: files change in the working directory, wait in the staging area (the index), and become permanent snapshots once committed to the repository
- `git status` as the command you run constantly: it names every file as untracked, modified, or staged so nothing sneaks into a commit by accident
- Staging deliberately: `git add file.html` for one file, `git add .` for everything, and why choosing files on purpose makes better commits
- `git commit -m 'Add homepage heading'`: writing messages in present tense that say what and why, not 'stuff' or 'changes'
- The `git commit -am` shortcut, and the catch: it only works for files Git already tracks
- Reading your story back with `git log`: author, date, hash, and message of every snapshot you have ever taken

Projects built:

- Create my-first-repo and build a tiny webpage across 5 separate commits, each with a clear message, so `git log` reads like a story of how the page grew

Practice: Run `git status` after every single command for one whole session until you can predict what it will say before you press Enter.

Assessment: Show a repository with at least 5 commits whose messages let a stranger understand the project's history without opening any file.

Class 4: Time Travel: log, diff, and Inspecting History

Topics:

- Power views of history: `git log --oneline` for a compact list, `git log --graph --oneline --all` to see the shape of your project
- Commit hashes demystified: the long SHA-1 identifier, the 7-character short form, and HEAD as 'where you are right now'
- `git diff`: exactly what changed but is not staged yet; `git diff --staged` for what is about to be committed
- `git show <hash>`: opening any single commit to see precisely what it changed and why
- One file's biography: `git log -p filename` to see every change ever made to it
- `git blame filename`: which commit and author last touched each line, and why it is a debugging tool, not an accusation tool
- Visiting the past safely: `git checkout <hash>` to look at an old version, what 'detached HEAD' really means, and `git switch main` to come home

Projects built:

- History detective challenge: in a prepared repo, use `log`, `show`, `diff`, and `blame` to answer five questions like 'which commit broke the page title and what did it change?'

Practice: Pick any two commits in your Class 3 repo and describe the difference between them using only `git diff` output.

Assessment: Check out an old commit, read it, return to main correctly, and explain detached HEAD in one sentence.

Class 5: Undoing Safely + .gitignore

Topics:

- The undo ladder: from harmless to serious, and why you always reach for the safest rung first
- `git restore file` to throw away working-directory changes, and `git restore --staged file` to unstage without losing work
- `git commit --amend` to fix your last commit message or include a forgotten file, and why you only amend commits you have not pushed
- `git reset --soft HEAD~1` to un-commit while keeping your changes staged, and why beginners treat `--hard` as radioactive
- `git revert <hash>`: the safe, history-preserving undo that creates a new commit cancelling an old one
- `.gitignore`: keeping `node_modules`, `__pycache__`/, `.DS_Store`, and secret API keys out of your history, with `*` wildcards and `folder/` patterns
- The oops recovery: `git rm --cached secrets.txt` when you committed something you meant to ignore, and why leaked secrets must also be changed

Projects built:

- Rescue mission: in a prepared practice repo, unstage a file, discard a bad edit with `git restore`, amend a commit message, revert a bad commit, and write a `.gitignore` that keeps `secrets.txt` out forever

Practice: Deliberately make each of the five mistakes covered in class and undo each one with the safest command that fixes it.

Assessment: Given three broken-repo scenarios, name the correct undo command for each and explain why a harsher one was not needed.

PHASE 2: GITHUB ESSENTIALS

Take your repositories online: create your GitHub account, set up SSH keys, push and pull, write READMEs people actually read, build your profile page, understand licenses and issues, and publish a free website with GitHub Pages.

Class 6: Joining GitHub (13+) and Your First Push

Topics:

- Creating your account: GitHub's Terms of Service require users to be 13 or older, which is exactly why this course starts at 13, plus choosing a username you will still like on a college application
- What GitHub Free includes: unlimited public and private repositories and 2,000 GitHub Actions minutes per month, more than enough for everything in this course
- A guided tour: repositories, your profile, notifications, the new-repo button, and the real difference between public and private repos
- Creating a repository on GitHub: initializing with a README, picking a `.gitignore` template, and what the license dropdown is for (Class 10 goes deep)
- Wiring local to remote: `git remote add origin <url>` and confirming it with `git remote -v`
- The first upload: `git push -u origin main`, and what `--set-upstream` actually saves you from typing forever after
- Securing your account with two-factor authentication before you build anything worth stealing

Projects built:

- Create your GitHub account with 2FA enabled, make your first online repository, connect your Class 3 local repo to it, and push all 5 commits so they appear on `github.com`

Practice: Make one more local commit and push it, then refresh the GitHub page and find your exact change in the web interface.

Assessment: Your Class 3 repository is visible on your GitHub account with full commit history and `git remote -v` shows origin correctly.

Class 7: HTTPS vs SSH: Keys, Cloning, and Pulling

Topics:

- Why GitHub removed password pushes: HTTPS with tokens vs SSH keys, and which one professionals set up on day one
- Generating your key pair: `ssh-keygen -t ed25519 -C 'your_email'`, where keys live in `~/.ssh`,

- and the golden rule that the private key never leaves your machine
- Installing the key: starting ssh-agent, ssh-add, pasting the .pub file into GitHub Settings under SSH keys, and testing with ssh -T git@github.com
- git clone <url>: pulling down any repository, and understanding that a clone contains the complete history, not just the latest files
- git fetch vs git pull: fetch downloads new history to look at, pull is fetch plus merge into your branch
- What 'origin' actually is: just the default nickname for a remote URL, changeable with git remote set-url
- The two-machine workflow: clone your own repo onto a second computer (or a school machine) and keep both in sync with push and pull

Projects built:

- Set up a working SSH key, clone your own repository into a second folder to simulate a second computer, make a change there, push it, and pull it back into the original

Practice: Run ssh -T git@github.com and read the greeting, then clone one interesting public repository and explore its git log.

Assessment: Demonstrate a full round trip: commit on 'computer B' clone, push, pull on 'computer A', and point to the change arriving.

Class 8: READMEs That Make People Care (Markdown)

Topics:

- Why README.md is your project's front door: GitHub renders it automatically on the repo homepage, and most visitors never look past it
- Markdown core: # and ## headings, **bold** and *italic*, bulleted and numbered lists, [links](url), and images with ![alt](path)
- Code that looks like code: inline backticks and fenced code blocks with language names for syntax highlighting
- The anatomy of a great project README: what it is, a screenshot, how to run it, features, what you learned, credits
- Fancier moves: tables, task lists with - [], and collapsible details sections for long content
- Badges from shields.io, used tastefully: one or two that say something true, not a wall of stickers
- Previewing before pushing: VS Code's built-in Markdown preview (Ctrl+Shift+V) so you never push a broken layout

Projects built:

- Write a complete README.md for your webpage project with a heading, screenshot, run instructions, a feature table, and one code block, then push it and admire the rendered result

Practice: Find a popular open source README you admire, list three techniques it uses, and steal one for your own.

Assessment: Your pushed README renders correctly on GitHub with at least five different Markdown elements used properly.

Class 9: Your Profile README and GitHub Identity

Topics:

- The special repo trick: create a repository named exactly your username and its README becomes your public profile page
- What belongs there at your age: who you are, what you are learning, projects you are proud of, and school-safe contact info (no address, no phone)
- Pinned repositories: choosing up to 6 repos that represent your best work, and why order matters
- The contribution graph decoded: what the green squares actually count, and why steady real work beats gaming the graph
- Stars as bookmarks: starring repos you want to learn from, and building a feed by following builders you admire
- Watch vs star vs fork: notifications, bookmarks, and copies, and when each one is the right click
- Playing the long game: colleges and internship reviewers really do open GitHub profiles, so keep every claim honest

Projects built:

- Create your username/username repository with a profile README introducing yourself, your current projects, and what you are learning, then pin your two best repos

Practice: Visit five classmates' new profiles, star one repo from each, and leave one genuine compliment as an issue or comment.

Assessment: Your profile shows a rendered profile README and at least two pinned repositories with descriptions.

Class 10: Licenses, Issues, and Repo Housekeeping

Topics:

- The surprise default: a repo with no license means all rights reserved, so others technically cannot reuse your code even though they can read it
- The big three in plain English: MIT (do almost anything, keep the notice), Apache-2.0 (MIT plus patent protection), GPL-3.0 (share-alike), and choosealicense.com for deciding
- Adding a LICENSE file the easy way with GitHub's built-in template picker
- Issues as a project's memory: writing a bug report with steps to reproduce, expected behavior, and actual behavior
- Organizing with labels, assignees, and milestones, even on a solo school project
- Closing issues from commits: writing 'fixes #12' in a commit message and watching GitHub

close the issue automatically

- Housekeeping: repo descriptions and topics for discoverability, renaming safely, and archiving old projects instead of deleting history

Projects built:

- Add an MIT license to your webpage project, open three well-written issues on it (one bug, two feature ideas), and close one automatically with a 'fixes #N' commit

Practice: Read the licenses of three repos you starred and identify which of the big three (or something else) each one uses.

Assessment: Your repo has a LICENSE file, at least two open issues with labels, and one issue closed by a linked commit.

Class 11: Publish Your First Website Free with GitHub Pages

Topics:

- What GitHub Pages is: free static site hosting served straight from a repository, no server rental, no credit card
- The two flavors: a user site from a repo named username.github.io, and project sites served at username.github.io/repo-name
- Turning it on: Settings, then Pages, then deploy from branch, choosing main and the root folder (or /docs)
- Push an index.html and watch it go live: how long deploys take and how to check progress in the Actions tab
- Project-site gotchas: why relative links and image paths break under /repo-name and how to write them correctly
- A custom 404.html page so even your error page has personality
- Why this matters: this URL becomes your portfolio home for the rest of the course, and every push updates it automatically

Projects built:

- Publish your webpage project live on GitHub Pages, fix any broken image paths, add a custom 404 page, and share your working URL with the class

Practice: Change your page's heading locally, commit, push, and time how long the live site takes to update.

Assessment: A live GitHub Pages URL that loads your site correctly, including images and your custom 404 page.

PHASE 3: BRANCHING & COLLABORATION

Learn the skill that separates solo coders from teammates: branches, merges, calm conflict resolution, pull requests end to end, kind code review, forks with upstream syncing, and a real two-person project on one shared repository.

Class 12: Branches: Parallel Universes for Your Code

Topics:

- What a branch really is under the hood: a movable pointer to a commit, which is why creating one is instant and free
- The modern commands: git branch to list, git switch -c feature-dark-mode to create and move, and the older git checkout -b you will see in tutorials
- Naming branches so future-you understands: feature/dark-mode, fix/broken-nav-link, experiment/crazy-idea
- Switching with uncommitted changes: when Git lets you carry work across and when it stops you to protect it
- Seeing the universe split: git log --oneline --graph --all as branches diverge from main
- Cleaning up: git branch -d for merged branches, and why -D (force delete) deserves a moment of doubt
- The golden rule real teams follow: main always stays working and deployable, experiments live on branches

Projects built:

- On your live website repo, build a new feature (a dark theme or a new section) entirely on a feature branch while main stays untouched and your live site keeps working

Practice: Create three branches with proper names, make one commit on each, and draw the resulting graph on paper before checking it with git log --graph.

Assessment: Show a repo where main is clean, a feature branch holds new commits, and you can explain the graph line by line.

Class 13: Merging Branches Like a Pro

Topics:

- The merge ritual: switch to the receiving branch first (git switch main), then git merge feature-dark-mode
- Fast-forward merges: when main has not moved, Git just slides the pointer ahead, no new commit needed
- Three-way merges: when both branches moved, Git creates a merge commit with two parents, and you can edit its message
- git merge --no-ff to force a merge commit and keep the branch's shape visible in history, and why some teams prefer it
- Reading history after merging: spotting merge commits and branch shapes in git log --graph
- git branch --merged: finding branches that are fully merged and safe to delete
- The strategy that prevents pain: small, short-lived branches merged often beat giant branches merged rarely

Projects built:

- Merge your dark-theme branch into main (watching your live Pages site update), then create, merge, and delete a second small feature branch using --no-ff and compare the two history shapes

Practice: Predict before each merge whether it will fast-forward or create a merge commit, then verify with git log --graph.

Assessment: Explain the difference between your fast-forward and no-ff merges using your own repo's graph as the evidence.

Class 14: When Branches Collide: Conflicts Made Simple

Topics:

- Why conflicts happen: two branches edited the same lines of the same file, and Git refuses to guess which version you want
- Decoding the conflict markers: <<<<<< HEAD, =====, and >>>>>> branch-name, and what sits between each
- Resolving in VS Code: Accept Current, Accept Incoming, Accept Both, or hand-editing to something better than either
- Finishing the job: git add the resolved files, then git commit to seal the merge
- The eject button: git merge --abort to back out cleanly and try again with a clearer head
- Your tools mid-conflict: git status lists exactly which files still conflict, git diff shows the battleground
- Prevention beats cure: pull often, keep commits small, and talk to teammates about who is touching what

Projects built:

- Conflict laboratory: follow a script that manufactures a two-branch conflict in your repo, resolve it three ways (current, incoming, hand-crafted blend), and keep the best resolution

Practice: Create and resolve one fresh conflict from scratch without the script, timing yourself from conflict to clean commit.

Assessment: Given a conflicted file, correctly identify which branch each block came from and produce a sensible resolved commit.

Class 15: Pull Requests End to End

Topics:

- What a pull request actually is: a proposed merge plus a conversation about it, the center of collaboration on GitHub
- The flow: push your branch (git push -u origin feature-name), then GitHub's 'Compare & pull request' button, understanding base vs compare
- Writing a PR people want to review: a clear title, what changed and why, screenshots for visual changes, and 'closes #7' to auto-close the issue
- Draft pull requests for work in progress, and requesting a specific reviewer when it is ready
- Living in the Files changed tab: reading the diff, leaving comments on exact lines
- Updating a PR: push more commits to the same branch and the PR updates itself, no new PR needed
- The three merge buttons: merge commit, squash and merge (your 12 tiny commits become 1 clean one), and rebase and merge, with squash as the tidy default for small PRs

Projects built:

- Ship a real feature to your website through a complete PR: branch, push, open a PR with description and screenshot, link an issue, then squash-merge it and watch the issue close

Practice: Open a second PR as a draft, convert it to ready for review, push one more commit to it, and observe the PR update.

Assessment: A merged pull request on your repo with a proper description, a linked closed issue, and a clean squash-merged history on main.

Class 16: Reviewing Kindly + Forks and Upstream

Topics:

- Reviewing a friend's PR: the three verdicts (comment, approve, request changes) and how to phrase feedback that is specific, kind, and about the code, never the person
- The suggestion superpower: proposing an exact code change in a review comment that the author can commit with one click
- What a fork is: your own full copy of someone else's repository under your account, the key to contributing to projects you cannot push to
- The cross-repo loop: fork, clone your fork, branch, push to your fork, then open a PR back to the original repository
- Keeping a fork fresh: git remote add upstream <original-url>, git fetch upstream, git merge upstream/main, or GitHub's Sync fork button
- Why this design is brilliant: strangers can safely propose changes to any project on Earth without anyone granting them access first
- Star vs fork, settled: star to bookmark and learn, fork only when you intend to change

Projects built:

- Pair up: review a classmate's real PR using at least one line comment and one one-click suggestion, then fork a classmate's repo, improve their README, and send them a cross-repo PR

Practice: Set up an upstream remote on your fork and sync it after your classmate merges someone else's change.

Assessment: One thoughtful review submitted with a committed suggestion, and one cross-repo PR opened from your fork.

Class 17: Team Project: Two People, One Repo

Topics:

- Adding a collaborator: Settings, Collaborators, the invite, and what write access actually allows

- Branch protection basics: requiring a pull request review before anything merges to main, so nobody (including you) pushes straight to main
- The daily team loop: pull main, branch, commit, push, PR, review, merge, repeat until the project ships
- When your partner merges first: bringing main into your branch with `git pull origin main` and resolving any conflict on YOUR branch, not on main
- GitHub Projects boards: To do, In progress, Done columns with issues as draggable cards, perfect for school assignments
- Assigning issues so two people never silently build the same thing
- Communication as a Git skill: PR descriptions, issue comments, and review replies are how remote teams actually talk

Projects built:

- Two-person mini-sprint: with a partner, plan a small shared website on a Projects board, split it into assigned issues, and build it over branch-PR-review cycles with branch protection on

Practice: Complete one full loop where your partner merges before you and you resolve the resulting conflict on your own branch.

Assessment: The shared repo shows commits from both partners, at least four merged PRs with reviews, and a board with every card in Done.

----- PHASE 4: LEVEL UP -----

Move past the basics with the tools developers reach for daily: a safe introduction to rebase, git stash for interruptions, tags and releases for shipping versions, your first GitHub Actions automation, and a portfolio pass that makes your profile genuinely readable.

Class 18: Merge vs Rebase, a Safe Introduction

Topics:

- What git rebase actually does: lifts your branch's commits and replays them on top of a new base, rewriting them into new commits
- Merge vs rebase honestly: merge preserves true history with merge commits, rebase produces a straight readable line, and teams simply pick a convention
- The golden rule that keeps you safe: never rebase commits you have already pushed and shared with others
- `git pull --rebase`: updating your own branch without littering it with merge commits
- Conflicts during rebase: fix the file, `git add` it, then `git rebase --continue`, or bail out completely with `git rebase --abort`
- A peek at interactive rebase: `git rebase -i HEAD~3` to squash or reword your own local commits before anyone sees them
- The beginner's default when unsure: just merge, it is always safe

Projects built:

- Rebase experiment in a sandbox copy of your repo: create the same divergence twice, resolve it once with merge and once with rebase, and screenshot the two different history graphs side by side

Practice: Use `git rebase -i` on three messy local commits to squash them into one well-named commit before pushing.

Assessment: Explain, using your own two screenshots, what rebase changed about history and recite the golden rule from memory.

Class 19: Stash, Tags, and Releases

Topics:

- The interruption problem: you are mid-feature and must switch branches NOW, and committing half-broken work feels wrong
- `git stash push -m 'half-done navbar'`: shelving working-directory changes instantly, and `-u` to include untracked files
- Managing the shelf: `git stash list`, `git stash pop` (apply and remove) vs `git stash apply` (keep a copy), and `git stash drop`
- Tags: permanent bookmarks on commits, lightweight (`git tag v1.0.0`) vs annotated (`git tag -a v1.0.0 -m 'First public version'`)
- Semantic versioning in one line: `major.minor.patch`, and what bumping each number promises
- Tags are not pushed by default: `git push origin v1.0.0` or `git push --tags`
- GitHub Releases: turning a tag into a release page with human-readable notes and downloadable files, shipping `v1.0.0` of your site like real software

Projects built:

- Practice a real interruption (stash mid-edit, fix an urgent 'bug' on main, pop your stash back), then tag your portfolio site `v1.0.0` and publish a GitHub Release with proper notes

Practice: Stash with a message, list your stashes, and explain to a partner the difference between pop and apply before using each once.

Assessment: Your repo shows a pushed annotated `v1.0.0` tag and a published Release whose notes summarize the version accurately.

Class 20: Your First GitHub Actions Workflow

Topics:

- What CI and automation mean: robots that run your chosen steps automatically on every push, so checks never depend on memory
- What you get free: GitHub Free includes 2,000 Actions minutes per month, far more than this course needs
- Anatomy of a workflow file at `.github/workflows/deploy.yml`: name, the `on`: push trigger, jobs,

- runs-on: ubuntu-latest, steps, and uses: actions/checkout@v4
- YAML survival: indentation is meaning, and one stray space is the classic first bug
- Reading the Actions tab: live logs, green checks and red X marks on every commit, and finding the exact failing line
- Upgrading your site: deploying GitHub Pages through an Actions workflow, the modern flow that runs on every push to main
- Wearing it proudly: adding the workflow status badge to your README

Projects built:

- Write your first workflow from scratch: auto-deploy your Pages portfolio via Actions on every push to main, break it once on purpose, fix it by reading the logs, and add the status badge to your README

Practice: Push three commits and narrate to a partner exactly what the Actions tab shows for each, from trigger to green check.

Assessment: A green workflow run visible in your Actions tab that deployed your live site, plus a working status badge in your README.

Class 21: A Portfolio Recruiters and Colleges Can Actually Read

Topics:

- Thinking like your reader: an admissions reviewer or intern recruiter gives your profile about two minutes, so structure for skimming
- Pinned repos as your headline: choose 6, order them by impressiveness, and give every one a real README with a screenshot
- Repo metadata that sells: one-line descriptions, topic tags, and the live demo URL in the About box of every project
- The About box link trick: every project with a Pages site gets its URL right there, one click from visitor to live demo
- Writing project blurbs in outcomes language: what it does, what you built it with, and one thing you learned, not 'a simple website'
- The honest graph: steady real contributions beat manufactured green squares, and reviewers can tell the difference
- Your recurring 5-minute self-audit checklist: profile README current, pins ordered, descriptions filled, demos alive, no broken links

Projects built:

- Full portfolio pass: apply the audit checklist to your entire account, fixing every pinned repo's README, description, topics, and About-box demo link until a stranger can navigate your work unaided

Practice: Swap profiles with a classmate, spend exactly two minutes on theirs, and report what you understood and where you got lost.

Assessment: A classmate reviewing your profile could name your best project, open its live demo, and say what it does within two minutes.

PHASE 5: OPEN SOURCE & CAPSTONE

Graduate from your own repos to the world's: find a genuine good first issue, learn the etiquette that gets contributions merged, open a real external pull request, and complete a capstone that earns your certificate.

Class 22: Finding Your First Open Source Issue

Topics:

- Why contribute at all: real codebases, review from experienced maintainers, and public proof you can collaborate, the exact thing schools and employers cannot verify from solo projects
- Where to hunt: the good first issue and help wanted labels, curated first-issue lists, and GitHub search filters like label:"good first issue" language:python
- Reading the house rules first: CONTRIBUTING.md and CODE_OF_CONDUCT.md exist so you do not have to guess how a project works
- Sizing your first issue honestly: documentation fixes, typos, small bugs, and missing tests beat ambitious features every time
- Claiming politely: comment asking to take the issue and wait for a maintainer's reply before starting serious work
- Setting up the contribution: fork the project, clone YOUR fork, and create a branch named for the issue
- Reproducing before touching: run the project locally and confirm you can see the issue yourself before changing a single line

Projects built:

- Build a shortlist of three real candidate issues in projects you actually use or understand, pick one, claim it politely in a comment, and get your fork cloned and branched locally

Practice: Read the CONTRIBUTING.md of all three shortlisted projects and note one rule from each that you would have broken without reading it.

Assessment: A claimed (or claim-requested) real issue, a fork with an issue-named branch, and the project running on your machine.

Class 23: Making the Contribution: Etiquette and the External PR

Topics:

- Matching the project's style: read the neighboring code, follow their naming and formatting, and keep your diff as small as the fix allows
- Commit message conventions some projects require, like Conventional Commits (fix: correct typo in install guide), and finding the rule in CONTRIBUTING.md
- Opening the PR against the upstream repository: filling their PR template honestly and

- linking the issue you claimed
- Responding to review like a professional: make the requested changes, push to the same branch, reply to each comment, and resolve conversations
- Patience and kindness: maintainers are often volunteers, so wait about a week before a polite nudge, and never demand a merge
- If it is not merged: what you still gained (a public PR, review experience, a work sample) and how to reference it anyway
- After the merge: sync your fork from upstream, thank the reviewer, and add the contribution to your profile README

Projects built:

- Make the fix on your branch, open a real pull request to the upstream project with their template filled properly and the issue linked, and respond to any review feedback that arrives

Practice: Before submitting, self-review your own PR diff line by line and remove anything not needed for the fix.

Assessment: A real external pull request open (or merged) on a project you do not own, with a clean diff and an honest description.

Class 24: Capstone: Profile, Portfolio, Merged PR + Certification

Topics:

- The capstone checklist: polished profile README, pinned repos each with a real README, your live GitHub Pages portfolio, and one external pull request submitted or merged
- Final repo audit: a license on everything public, correct .gitignore files, and absolutely no secrets in any history (plus the rotation rule if one ever slips)
- A history a stranger can follow: sensible branch names, clear commit messages, and merged PRs that tell the story of each project
- Presenting with Git itself: walking your reviewer through a project using `git log --oneline --graph` as the narrative
- What unlocks next: the GitHub Student Developer Pack is free for verified students at degree or diploma granting schools and includes GitHub Pro while a student, and GitHub Education has offered free Copilot access to verified students (availability can change, check education.github.com)
- Your 90-day plan: a sustainable contribution rhythm, the next two open source targets, and using branches and PRs on every school project from now on
- Demo day: present your profile, live portfolio, and external PR to the class and earn your certificate

Projects built:

- Complete and present the full capstone: a polished GitHub profile, a live GitHub Pages portfolio linking every pinned project, and one real external pull request, defended in a demo-day walkthrough

Practice: Do a full dress rehearsal of your demo with a partner, including the `git log --graph` walkthrough of your best project.

Assessment: Final capstone review passed: profile, live portfolio URL, and external PR verified, earning the Modern Age Coders 'Git & GitHub for Teens' certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Git installed and configured, a terminal-built folder tree, a 5-commit story repo, a history-detective investigation, and a full rescue-mission undo drill with .gitignore
- Phase 2: your secured GitHub account with a pushed repo, an SSH key with a two-machine sync round trip, a rich project README, your profile README with pinned repos, a licensed repo with linked issues, and your first live GitHub Pages website
- Phase 3: a feature built safely on a branch, fast-forward and no-ff merges compared, a conflict resolved three ways, a complete squash-merged pull request, a kind review plus a cross-repo fork PR, and a two-person team sprint on one protected repo
- Phase 4: a merge vs rebase side-by-side experiment, a stash interruption drill plus a v1.0.0 tagged Release, a from-scratch GitHub Actions deploy workflow with status badge, and a full recruiter-readability portfolio audit
- Phase 5: a shortlist and claim of a real good first issue, a genuine external pull request to an open source project, and the capstone: polished profile, live portfolio, and demo-day walkthrough
- Total: 24 hands-on deliverables, one per class, culminating in a public portfolio and a real open source contribution

Total projects: 24 hands-on deliverables, one per class, finishing with a live portfolio site and a real open source pull request

SKILLS THE STUDENT WILL HAVE AT THE END

- Git Fundamentals: `init`, `status`, `add`, `commit`, `log`, `diff`, `show`, `blame`, and the three areas of every repository
- Safe Undo: `restore`, `restore --staged`, `commit --amend`, `reset --soft`, `revert`, and .gitignore discipline
- Terminal Fluency: navigating, creating, and managing projects entirely from the command line
- GitHub Essentials: remotes, clone, push, pull, fetch, SSH keys, Markdown READMEs, licenses, and issues
- Branching & Merging: `switch -c`, `merge`, fast-forward vs three-way, conflict resolution, stash, and a safe rebase introduction
- Collaboration: pull requests end to end, kind code review, forks with upstream syncing, branch protection, and Projects boards

- Publishing & Automation: GitHub Pages hosting, tags, Releases, and a GitHub Actions deploy workflow
- Open Source & Portfolio: finding good first issues, contribution etiquette, external PRs, and a recruiter-readable profile

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks

Hands on git work: 1-1.5 hours running real Git and GitHub commands from each class on your own machine

Project time: 1 hour completing each class deliverable, from commits and PRs to your live Pages site

Review and doubts: 30-60 minutes reviewing your history, practicing undo drills, and clearing doubts before the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for install issues, scary error messages, and conflict emergencies

Small batches: Small batches, with mini-batch and 1-on-1 options, so every teen gets individual attention at the terminal

Mentor guidance: Live guidance from mentors who use Git and GitHub daily on real projects

Project feedback: Feedback on every class deliverable and a personal capstone review before certification

Living syllabus: Curriculum updated as Git and GitHub ship new features, at no extra cost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'Git & GitHub for Teens' certificate, earned by presenting your Class 24 capstone

Project portfolio: A live GitHub Pages portfolio site plus pinned, documented repositories you can show schools and colleges

GitHub profile: A polished GitHub profile with a profile README, real commit history, merged pull requests, and an external open source contribution

WHERE THIS CAN LEAD

- Confident collaborator on any school team or hackathon, fluent in branches, pull requests, and reviews
- Open source contributor with a real external pull request as public proof of collaboration
- Portfolio-ready applicant: a pinned, documented GitHub profile plus a live Pages site for college and internship applications
- Prepared for every future language and course: the same Git workflow powers Python, web, Java, C++, and AI projects
- On track for developer roles: version control is a day-one, non-negotiable requirement in every software job

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What age do you need to be to use GitHub?

A: GitHub's Terms of Service require users to be at least 13 years old to create an account. That is exactly why this course is designed for ages 13-18: every student can legally have their own real account from Class 6. We also walk through securing it with two-factor authentication.

Q: What is the difference between Git and GitHub?

A: Git is a free, open source distributed version control system that runs on your own computer; it was created by Linus Torvalds in 2005 for Linux kernel development. GitHub is a website (owned by Microsoft since 2018) where Git repositories are hosted online so people can collaborate through pull requests, issues, and reviews. This course teaches both, in that order.

Q: Is GitHub free for students?

A: On top of that, the GitHub Student Developer Pack is free for verified students enrolled at a degree or diploma granting school and includes GitHub Pro while you are a student. GitHub Education has also offered free Copilot access to verified students; availability can change, so we point students to education.github.com.

Q: Do I need to know how to code before learning Git and GitHub?

A: No. Git tracks files, so you can learn it with a simple webpage or even text files, and we assume zero terminal experience: Class 2 teaches the command line from scratch. If you already have a hobby project like a website, Python script, or game mod, you will practice on that; if not, we provide starter projects.

Q: How long does it take to learn Git and GitHub?

A: You will be committing confidently within the first five classes, and the full course runs 24 live classes over 12 weeks at two classes per week. We recommend 3-4 hours per week

including practice. By the end you are not just 'familiar' with Git: you have merged pull requests, resolved real conflicts, and contributed to an open source project.

Q: Are the classes live or recorded?

A: Every class is live and interactive with a real instructor, never pre-recorded videos. Group batches are capped at 10 students, with mini-batch and 1-on-1 options, and you get WhatsApp doubt support between classes. You can also take a free demo class before deciding.

Q: Can I host a website for free on GitHub?

A: Yes. GitHub Pages provides free static site hosting straight from a repository, with no server costs. You publish your first live site in Class 11, automate its deployment with a GitHub Actions workflow in Class 20, and it becomes the portfolio you present at the capstone.

Q: What will my teen actually have at the end of this course?

A: Four concrete things: a polished GitHub profile with a profile README and pinned, documented projects; a live portfolio website on GitHub Pages; a real pull request to an external open source project; and the Modern Age Coders 'Git & GitHub for Teens' certificate. Together they form public, verifiable proof of skill that colleges and internship reviewers can check themselves.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/git-github-version-control-course-for-teens>

COURSE 64 OF 118: GO (GOLANG) PROGRAMMING: CONCURRENCY, GRPC & MICROSERVICES

Course link: <https://learn.modernagecoders.com/courses/complete-golang-programming-masterclass-college>
Subject area: Professional Go Development
Designed for: College students
Who can join: Complete Beginner to Cloud Native Expert
Course length: 12 months (52 weeks)
Time needed: 15-20 hours/week recommended
Certificate: Industry-recognized Go Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online Go (Golang) course for college students: a 1-year path from fundamentals to concurrency, gRPC, Kubernetes and cloud-native microservices.

In one line: From 'Hello, World!' to Cloud Native Microservices Architecture

ABOUT THE PROGRAMME

This is not just a Go course, it's a complete transformation into a cloud-native Go expert. Whether you're a curious beginner, student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled Go developer capable of building scalable microservices, high-performance APIs, distributed systems, and cloud-native solutions. You'll master Go from ground zero to cloud architect level: from basic syntax to advanced concurrency patterns, from CLI tools to production microservices, from REST APIs to gRPC services, from monolithic apps to Kubernetes operators. By the end, you'll have built 40+ projects, mastered the entire Go ecosystem, and be ready for senior Go developer roles in top tech companies.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - perfect for complete beginners
- Separate learning tracks for kids (12+), teens, and adults
- 1 year of structured, industry-aligned learning
- Covers Core Go + Cloud Native Go (Kubernetes, Docker, gRPC)
- Real enterprise projects and industry case studies
- Hands-on with latest Go versions (Go 1.21+)
- Interview preparation for FAANG and top companies
- Lifetime access with continuous updates
- Build production-ready cloud native portfolio
- Direct path to high-paying Go developer jobs

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Go Fundamentals, Data Structures, Concurrency Basics
Phase 2: Intermediate (Months 4-6): Advanced Concurrency, Testing, Performance, CLI Tools
Phase 3: Enterprise (Months 7-9): Web Development, REST APIs, gRPC, Databases, Deployment

Phase 4: Professional (Months 10-12): Microservices, Kubernetes, Cloud Native, System Design

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to all backgrounds
Coding experience: Absolute beginner friendly - zero programming knowledge required
Age: 12+ years (separate learning tracks for kids, teens, adults)
Equipment: Computer/laptop (Windows/Mac/Linux), minimum 8GB RAM recommended, internet connection
Time commitment: 15-20 hours per week consistently for best results
English: Basic reading comprehension (materials also available in Hindi)
Motivation: Strong desire to become a professional Go/Cloud Native developer
Math: Basic mathematics (taught as needed)

WHO THIS COURSE IS FOR

Kids: Age 12-14: Simplified track with visual learning, CLI projects, game-based projects
Teens: Age 15-18: Student-focused track, backend development, competitive programming
Students: College students: Campus placement preparation, internship readiness, strong portfolio
Working professionals: Career switchers: Structured path from any background to Go developer role
Developers: Developers from other languages wanting to master Go ecosystem
Backend developers: Backend devs wanting to transition to Go for high-performance systems
Devops engineers: DevOps engineers wanting to build cloud-native tools with Go
System programmers: System programmers wanting a modern, efficient systems language
Entrepreneurs: Build scalable backend systems for your startup with Go
Anyone: Anyone passionate about systems programming and cloud-native development

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE GO SKILLS (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid Go fundamentals. Learn programming logic, master Go syntax, understand Go's philosophy, and create your first applications.

Months 1-2: Go Fundamentals & Programming Basics (Week 1-8)

Introduction to Go & Development Environment Setup

Topics:

- What is Go? History, creators (Google), and philosophy
- Go's design principles: simplicity, reliability, efficiency
- Why Go? Use cases and advantages
- Go vs other languages: Python, Java, Rust, C++
- Go ecosystem and community
- Installing Go (latest stable version)
- GOPATH and Go Modules explained
- Setting up development environment: VS Code with Go extension
- GoLand IDE setup (optional)
- Understanding Go workspace structure
- Your first Go program: Hello, World!
- Go program structure: package, import, main function
- Running and building Go programs
- go run, go build, go install commands
- Comments: single-line (//) and multi-line (/* */)
- Go documentation with godoc
- gofmt for code formatting
- Go conventions and effective Go guidelines

Projects built:

- Hello World with command-line arguments
- Personal information display program
- ASCII art generator in Go
- Simple text output variations
- Environment variable reader

Practice: Daily: 30 min Go syntax practice, write 5-10 simple programs

Variables, Data Types & Operators

Topics:

- Variables: declaration with var keyword
- Short variable declaration with :=
- Variable naming conventions in Go
- Zero values in Go
- Basic data types: bool, string, int, float64
- Integer types: int8, int16, int32, int64, uint variants
- Floating-point types: float32, float64
- Complex numbers: complex64, complex128
- Type inference in Go
- Constants with const keyword

- Iota for enumerated constants
- Type conversion and type assertion
- Operators: arithmetic (+, -, *, /, %, ++, --)
- Comparison operators: ==, !=, <, >, <=, >=
- Logical operators: &&, ||, !
- Bitwise operators: &, |, ^, &^, <<, >>
- Assignment operators: =, +=, -=, *=, /=, %=
- Operator precedence
- String operations and string formatting
- fmt package for formatted I/O

Projects built:

- Calculator with all arithmetic operations
- Temperature converter (Celsius, Fahrenheit, Kelvin)
- Unit converter (length, weight, volume)
- BMI calculator with health assessment
- Currency converter with live rates simulation
- Bitwise operations demonstrator
- Type conversion examples program
- Interest calculator (simple and compound)

Practice: Solve 30 problems on variables, data types, and operators

Control Flow & Functions

Topics:

- If statements in Go (no parentheses required)
- If with initialization statement
- If-else and else-if chains
- Switch statement: more powerful than other languages
- Switch without expression (replaces if-else chains)
- Switch with multiple cases
- Type switch for interface types
- For loop: Go's only loop construct
- Classic for loop with initialization, condition, post
- While-style for loop
- Infinite loops with for
- Range loops for arrays, slices, maps, channels
- Break and continue statements
- Labeled break and continue
- Goto statement (rarely used)
- Functions: first-class citizens in Go
- Function declaration and parameters
- Multiple return values
- Named return values
- Variadic functions with ...
- Anonymous functions and closures
- Functions as values and parameters
- Recursion in Go

Projects built:

- Number guessing game
- Grade calculator with switch
- Prime number generator
- Fibonacci calculator (iterative and recursive)
- Factorial calculator
- Pattern printer (pyramids, diamonds)
- FizzBuzz implementation
- Menu-driven calculator
- Password strength checker
- Palindrome checker with multiple approaches
- Function library for math operations

Practice: Solve 40 control flow and function problems

Arrays, Slices & Maps

Topics:

- Arrays: fixed-size sequences
- Array declaration and initialization
- Array literals and zero values
- Iterating arrays with for and range
- Multidimensional arrays
- Slices: dynamic arrays
- Creating slices with make()
- Slice literals
- Slice internals: pointer, length, capacity
- Appending to slices with append()
- Copying slices with copy()
- Slicing operations and reslicing
- Nil slices vs empty slices
- Maps: Go's hash tables
- Creating maps with make() and literals
- Adding, updating, deleting map entries
- Checking key existence with comma ok idiom
- Iterating maps with range
- Maps of structs

- Nested maps
- Common slice and map patterns

Projects built:

- Dynamic array operations library
- Student grade management with slices
- Word frequency counter with maps
- Phonebook application with maps
- Inventory management system
- Matrix operations (2D slices)
- Shopping cart with slices and maps
- Duplicate remover
- Sorting algorithms implementation
- Search algorithms (linear, binary)
- Statistics calculator for datasets

Practice: Solve 35 slice and map problems

Month 3: Structs, Methods & Interfaces (Week 9-13)

Structs & Methods

Topics:

- Structs: custom types in Go
- Struct declaration and fields
- Struct literals and zero values
- Anonymous structs
- Nested structs and embedded fields
- Struct tags for JSON, XML, database
- Methods: functions with receivers
- Value receivers vs pointer receivers
- When to use pointer receivers
- Method sets and interfaces
- Struct composition over inheritance
- Embedding structs for composition
- Promoted fields and methods
- Constructor functions in Go
- Factory patterns
- Encapsulation with exported/unexported fields
- Getters and setters conventions
- String() method for custom string representation
- Comparing structs
- Anonymous struct fields

Projects built:

- Person struct with methods
- Bank account with transactions
- Book library management
- Employee management system
- Geometric shapes with area/perimeter
- Car dealership system
- Student enrollment system
- Product inventory with methods
- Time tracker application
- Contact management system

Practice: Create 20 different structs with methods

Interfaces & Polymorphism

Topics:

- Interfaces: defining contracts
- Interface declaration and implementation
- Implicit interface satisfaction
- Empty interface: interface{}
- Type assertions and type switches
- Interface composition
- Common interfaces: Stringer, error, io.Reader, io.Writer
- Polymorphism through interfaces
- Duck typing in Go
- Interface best practices
- Small interfaces principle
- Accept interfaces, return structs
- Interface segregation
- Dependency injection with interfaces
- Mocking with interfaces for testing
- Sort interface implementation
- Custom sorting with sort.Interface
- Error interface and custom errors
- Wrapping errors
- Interface values: dynamic type and value

Projects built:

- Shape calculator with interfaces
- Payment processing system
- Notification system (Email, SMS, Push)
- Database abstraction layer
- Animal hierarchy with interfaces

- Plugin system using interfaces
- Custom error types
- Logging abstraction
- Event system with interfaces
- Repository pattern implementation

Practice: Build 15 interface-based systems

Packages & Error Handling

Topics:

- Packages: organizing Go code
- Creating packages
- Package naming conventions
- Exported vs unexported identifiers
- Package initialization with init()
- Import statements and aliases
- Blank imports with _
- Internal packages
- Vendoring and dependency management
- Go modules in detail
- go.mod and go.sum files
- Module versioning
- Error handling philosophy in Go
- Error as values
- Creating errors with errors.New() and fmt.Errorf()
- Error wrapping with %w verb
- errors.Is() and errors.As()
- Panic and recover mechanism
- When to panic
- Defer statement for cleanup

Projects built:

- Multi-package application
- Custom package library
- Error handling patterns library
- Configuration package
- Utility package creation
- Logging package
- PHASE 1 MINI CAPSTONE: Task Management CLI Tool
- Features: Add tasks, list tasks, mark complete, persistence, packages, error handling

Practice: Create 10 reusable packages

Assessment: Phase 1 Final Assessment - Core Go fundamentals test

PHASE 2: ADVANCED GO & SYSTEMS PROGRAMMING (MONTHS 4-6, WEEKS 14-26)

Master Go's concurrency model, advanced features, testing, performance optimization, and systems programming.

Months 4-5: Concurrency & Advanced Features (Week 14-22)

Goroutines & Channels Fundamentals

Topics:

- Concurrency vs parallelism
- Go's concurrency philosophy
- Goroutines: lightweight threads
- Starting goroutines with go keyword
- Goroutine scheduling and GOMAXPROCS
- Channels: communication between goroutines
- Creating channels with make()
- Sending and receiving on channels
- Channel directions: send-only, receive-only
- Buffered channels
- Channel closing and range loops
- Select statement for multiple channels
- Default case in select
- Channel patterns: fan-in, fan-out
- Worker pools with goroutines
- Pipeline pattern
- Timeout handling with channels
- Context package for cancellation
- sync.WaitGroup for goroutine coordination
- Race conditions and data races

Projects built:

- Concurrent web scraper
- Worker pool implementation
- Pipeline data processor
- Concurrent file downloader
- Chat server with goroutines
- Parallel image processor
- Rate limiter with channels
- Concurrent cache implementation
- Producer-consumer pattern

- Merge sort with goroutines

Practice: Solve 35 concurrency problems

Advanced Concurrency Patterns

Topics:

- Mutex and RWMutex for synchronization
- Atomic operations with sync/atomic
- sync.Once for one-time initialization
- sync.Map for concurrent map access
- sync.Pool for object reuse
- Condition variables with sync.Cond
- Semaphores implementation
- Context package deep dive
- Context cancellation and timeouts
- Context values and best practices
- Errgroup for error handling in goroutines
- Advanced channel patterns
- Pub-sub pattern with channels
- Circuit breaker pattern
- Retry with exponential backoff
- Rate limiting algorithms
- Lock-free programming basics
- Memory model and happens-before
- Detecting race conditions
- Benchmarking concurrent code

Projects built:

- Thread-safe data structures
- Concurrent web crawler with limits
- Distributed task queue
- Event bus implementation
- Load balancer simulator
- Concurrent database connection pool
- Caching with TTL
- Retry mechanism library
- Circuit breaker library
- Concurrent metrics collector

Practice: Build 25 advanced concurrent systems

Reflection & Generics

Topics:

- Reflection in Go with reflect package
- Type and Value in reflection
- Inspecting types at runtime
- Modifying values through reflection
- Struct tag parsing with reflection
- Method invocation via reflection
- Creating generic functions with reflection
- Reflection performance considerations
- When to use reflection
- Generics in Go (Go 1.18+)
- Type parameters and constraints
- Generic functions
- Generic types and structs
- Type inference with generics
- Type sets and interface constraints
- Comparable constraint
- Generic slice operations
- Generic data structures
- Generics best practices
- Migrating from interface{} to generics

Projects built:

- JSON marshaler with reflection
- ORM-like query builder
- Dependency injection container
- Generic stack and queue
- Generic set implementation
- Type-safe cache with generics
- Generic algorithms library
- Struct validator with tags
- Configuration parser
- Generic repository pattern

Practice: Implement 20 generic and reflection-based utilities

Testing & Benchmarking

Topics:

- Testing in Go philosophy
- Writing tests with testing package
- Test file naming conventions
- Test function signatures
- Running tests with go test
- Table-driven tests
- Subtests with t.Run()

- Test helpers and cleanup
- Testing packages and examples
- Benchmark functions
- Running and interpreting benchmarks
- Benchmark comparisons
- Coverage analysis with go test -cover
- Coverage reports and visualization
- Mocking and test doubles
- Dependency injection for testing
- Integration testing
- End-to-end testing
- Fuzzing with go test
- Property-based testing
- Test fixtures and golden files
- Testing concurrent code
- Race detection in tests

Projects built:

- Comprehensive test suite
- Benchmark suite for algorithms
- Mock implementations
- Integration test framework
- Test data generators
- Coverage improvement project
- Fuzz testing examples
- Testing utilities library
- Test assertion library
- BDD-style testing framework

Practice: Write tests achieving 80%+ coverage for all projects

Performance Optimization & Profiling

Topics:

- Performance optimization principles
- Profiling with pprof
- CPU profiling
- Memory profiling and allocation
- Goroutine profiling
- Block profiling
- Mutex profiling
- Trace collection and analysis
- Benchmark-driven optimization
- Memory management in Go
- Garbage collection tuning
- Escape analysis
- Stack vs heap allocation
- Reducing allocations
- String optimization techniques
- Slice performance tips
- Map optimization
- Compiler optimizations
- Assembly in Go
- CGO and performance implications
- Performance monitoring in production

Projects built:

- Performance profiling toolkit
- Memory-efficient data processor
- Optimized string operations
- Zero-allocation HTTP handler
- High-performance cache
- Profiling dashboard
- Performance regression tests
- Allocation tracker

Practice: Optimize 15 programs for performance

Month 6: CLI Tools & Systems Programming (Week 23-26)

Building CLI Applications

Topics:

- Command-line argument parsing with flag package
- Positional arguments and subcommands
- Cobra framework for complex CLIs
- Viper for configuration management
- Environment variables handling
- Configuration file formats: JSON, YAML, TOML
- Interactive CLIs with prompts
- Terminal colors and formatting
- Progress bars and spinners
- Table output formatting
- Shell completion scripts
- Cross-platform considerations
- Signal handling
- Daemon processes

- CLI testing strategies
- Distribution and installation
- Self-updating CLIs
- CLI documentation generation
- Man pages creation
- CLI best practices

Projects built:

- Git-like version control CLI
- Database migration tool
- File manager CLI
- HTTP client like curl
- System monitoring tool
- Log analyzer CLI
- Task runner like Make
- Package manager CLI
- Deployment tool
- CLI framework

Practice: Build 15 production-ready CLI tools

File I/O & System Programming

Topics:

- File operations with os package
- Reading files: various techniques
- Writing files efficiently
- File permissions and metadata
- Directory operations
- Path manipulation with filepath
- Temporary files and directories
- File watching and monitoring
- Memory-mapped files
- Buffered I/O with bufio
- CSV and JSON file processing
- Binary file handling
- Archive formats: tar, zip
- Compression: gzip, zlib
- Process execution with os/exec
- Pipes and inter-process communication
- System calls and syscall package
- Platform-specific code with build tags
- CGO for C integration
- Unsafe package usage

Projects built:

- File synchronization tool
- Backup utility
- Log rotation system
- File encryption tool
- Directory tree analyzer
- File compression utility
- Process manager
- System information collector
- File watcher service
- Binary parser

Practice: Build 12 system-level applications

Networking & Protocols

Topics:

- TCP/IP programming with net package
- TCP server and client
- UDP programming
- Unix domain sockets
- Network protocols implementation
- HTTP client customization
- WebSocket implementation
- TLS/SSL configuration
- Certificate handling
- DNS operations
- Port scanning
- Network interfaces information
- Raw sockets (with x/net)
- Packet capture basics
- Network monitoring
- Load balancing algorithms
- Proxy server implementation
- Tunneling and VPN basics
- Network security basics
- Rate limiting at network level

Projects built:

- TCP chat server
- HTTP proxy server
- Port scanner
- Network monitor

- Load balancer
- VPN client prototype
- DNS server
- WebSocket server
- File transfer protocol
- Network diagnostic tool

Practice: Implement 10 networking applications

Data Serialization & Encoding

Topics:

- JSON encoding/decoding with encoding/json
- Custom JSON marshaling
- JSON streaming
- XML processing
- Protocol Buffers with protobuf
- MessagePack encoding
- YAML handling
- TOML configuration
- CSV processing
- Binary encoding with encoding/binary
- Base64 encoding
- Hex encoding
- Custom encoders and decoders
- Schema validation
- Data transformation pipelines
- ETL processes in Go
- Data compression techniques
- Encryption and decryption
- Hashing and checksums
- Digital signatures

Projects built:

- Data transformation pipeline
- Configuration management system
- Protocol buffer service
- ETL tool
- Data validator
- Encryption utility
- File format converter
- Schema validator
- Binary protocol implementation
- Serialization benchmark tool

Practice: Build 15 data processing applications

Phase 2 Capstone Project

Topics:

- Advanced concurrency implementation
- CLI interface design
- System programming concepts
- Performance optimization
- Comprehensive testing
- Documentation

Projects built:

- PHASE 2 CAPSTONE: Distributed Task Processing System
- Features: CLI interface, concurrent workers, job queue, file processing, network communication, monitoring, extensive tests
- Alternative: Container orchestration tool mini-version
- Alternative: Monitoring and metrics collection system
- Alternative: Distributed cache implementation

Assessment: Phase 2 Final Exam - Advanced Go concepts comprehensive test

PHASE 2 CONTINUED - Tooling & Best Practices (Week 14-26 (distributed))

Build Tools & Dependency Management

Topics:

- Go modules deep dive
- Module versioning strategies
- Semantic versioning
- Module proxies and private modules
- Vendoring dependencies
- go.work for multi-module repos
- Build flags and tags
- Cross-compilation
- Static vs dynamic linking
- Build optimization flags
- Makefile for Go projects
- Task automation with Mage
- CI/CD for Go projects
- GitHub Actions for Go
- GitLab CI for Go
- Release management
- Semantic release
- Containerizing Go applications

- Multi-stage Docker builds
- Minimal Docker images with scratch

Projects built:

- Module management system
- Build automation pipeline
- Cross-platform builder
- Release automation tool
- Docker image optimizer
- CI/CD pipeline setup
- Version management tool

Practice: Set up professional build pipelines for all projects

Code Quality & Best Practices

Topics:

- Go code style and conventions
- Effective Go guidelines
- go fmt and goimports
- Linting with golangci-lint
- Static analysis tools
- Code review practices
- Design patterns in Go
- SOLID principles in Go
- Domain-Driven Design with Go
- Clean Architecture in Go
- Hexagonal architecture
- Documentation with godoc
- README best practices
- API documentation
- Code comments guidelines
- Error handling patterns
- Logging best practices
- Configuration management patterns
- Security best practices
- Code metrics and complexity

Projects built:

- Code quality analyzer
- Design pattern examples
- Clean architecture project
- Documentation generator
- Code review checklist tool
- Security scanner
- Best practices linter

Practice: Refactor all projects with best practices

Debugging & Troubleshooting

Topics:

- Debugging with Delve
- Remote debugging
- Debugging goroutines
- Core dumps analysis
- Stack traces interpretation
- Panic recovery strategies
- Memory leaks detection
- Goroutine leaks
- Deadlock detection
- Race condition debugging
- Performance bottleneck identification
- Distributed tracing
- Logging strategies
- Structured logging
- Log aggregation
- Metrics collection
- Error tracking services
- Debugging production issues
- Post-mortem analysis
- Debugging tools and techniques

Projects built:

- Debugging toolkit
- Memory leak detector
- Goroutine analyzer
- Trace analyzer
- Log aggregator
- Error tracking system
- Performance debugger

Practice: Debug and fix 20 complex issues

Open Source Contribution

Topics:

- Open source ecosystem in Go
- Finding Go projects to contribute
- Understanding Go project structure
- Contributing guidelines

- Code review in Go community
- Popular Go projects walkthrough
- Creating your own package
- Publishing to pkg.go.dev
- Package documentation
- Package versioning
- Maintaining backwards compatibility
- Community engagement
- Go conferences and meetups
- Writing technical blogs
- Speaking about Go

Projects built:

- Contribute to 5 Go open source projects
- Create and publish Go package
- Write technical blog posts
- Create tutorial content
- Build community tool

Practice: Active open source participation

Advanced Topics & Specializations

Topics:

- WebAssembly with Go
- TinyGo for embedded systems
- Mobile development with Go
- Game development with Go
- Machine learning with Go
- Blockchain with Go
- IoT applications
- Scientific computing
- Graphics programming
- Audio/video processing
- Compiler construction
- Operating system development
- Database engine basics
- Search engine basics
- Distributed systems patterns

Projects built:

- WebAssembly application
- Embedded system project
- Mobile app backend
- Simple game in Go
- ML model deployment
- Blockchain prototype
- Choose specialization project

Practice: Explore 3 advanced specializations

----- PHASE 3: WEB DEVELOPMENT & ENTERPRISE APPLICATIONS (MONTHS 7-9, WEEKS 27-39) -----

Master web development with Go, build REST APIs, GraphQL, gRPC services, work with databases, and deploy production applications.

Months 7-8: Web Development & APIs (Week 27-35)

HTTP & Web Fundamentals

Topics:

- HTTP protocol deep dive
- HTTP/1.1 vs HTTP/2 vs HTTP/3
- net/http package fundamentals
- HTTP handlers and HandlerFunc
- ServeMux and routing
- Middleware pattern in Go
- Request context and cancellation
- Request parsing and validation
- Response writing techniques
- Cookie handling
- Session management
- File uploads and downloads
- Streaming responses
- Server-Sent Events (SSE)
- WebSocket upgrade
- HTTP/2 Server Push
- TLS/HTTPS configuration
- HTTP client best practices
- Connection pooling
- Timeouts and deadlines

Projects built:

- HTTP server from scratch
- Custom router implementation
- Middleware chain builder
- File server with upload

- Session management system
- Real-time chat with WebSocket
- SSE notification system
- HTTP proxy server
- Load tester tool

Practice: Build 10 HTTP-based applications

Web Frameworks & REST APIs

Topics:

- Gin framework introduction
- Gin routing and parameters
- Gin middleware
- Request binding and validation
- Echo framework
- Fiber framework overview
- Chi router
- Gorilla toolkit
- REST API design principles
- RESTful routing conventions
- API versioning strategies
- Request/response formatting
- Content negotiation
- CORS handling
- Authentication methods: Basic, Token, OAuth
- JWT implementation
- Rate limiting
- API documentation with Swagger
- OpenAPI specification
- Postman collections

Projects built:

- Blog REST API with Gin
- E-commerce API with Echo
- User management API
- Todo application backend
- Social media API
- File storage API
- Payment gateway integration
- Multi-tenant API
- API gateway implementation
- REST API testing suite

Practice: Build 12 production REST APIs

Database Programming

Topics:

- database/sql package
- Database drivers for MySQL, PostgreSQL
- Connection pooling configuration
- Prepared statements
- Transactions handling
- Batch operations
- SQL injection prevention
- Query builders
- GORM introduction and setup
- GORM models and migrations
- GORM associations
- GORM hooks and callbacks
- SQLX for enhanced database operations
- SQLBoiler for type-safe ORM
- Ent framework overview
- Database migrations with golang-migrate
- Connection management best practices
- Read replicas and write masters
- Database testing strategies
- Fixtures and seed data

Projects built:

- User authentication system
- Blog with comments (associations)
- E-commerce database layer
- Inventory management
- Booking system database
- Migration management tool
- Database abstraction layer
- Multi-database support
- Database monitoring tool
- Query performance analyzer

Practice: Build 15 database-driven applications

NoSQL & Caching

Topics:

- MongoDB driver for Go
- MongoDB CRUD operations
- Aggregation pipelines

- MongoDB transactions
- Redis with Go
- Redis data structures
- Redis pub/sub
- Redis Streams
- Caching strategies
- Cache-aside pattern
- Write-through caching
- Cache invalidation
- Distributed caching
- Memcached integration
- Elasticsearch with Go
- Full-text search implementation
- Time-series databases: InfluxDB
- Graph databases: Neo4j
- Key-value stores
- Document stores

Projects built:

- MongoDB-based blog platform
- Redis cache layer
- Real-time leaderboard with Redis
- Session store with Redis
- Search engine with Elasticsearch
- Metrics collector with InfluxDB
- Social graph with Neo4j
- Hybrid database application
- Cache performance tester
- NoSQL migration tool

Practice: Implement 10 NoSQL solutions

GraphQL & Modern APIs

Topics:

- GraphQL introduction and concepts
- GraphQL vs REST comparison
- GraphQL schema definition
- Types and resolvers
- Queries and mutations
- Subscriptions for real-time
- gqlgen framework
- Graph-gophers library
- Schema-first vs code-first
- DataLoader pattern
- N+1 query problem solutions
- GraphQL federation
- Error handling in GraphQL
- Authentication and authorization
- Rate limiting GraphQL
- GraphQL testing
- Apollo integration
- GraphQL documentation
- Performance optimization
- Caching strategies

Projects built:

- GraphQL API for blog
- E-commerce GraphQL service
- Real-time chat with subscriptions
- Social media GraphQL API
- GraphQL gateway
- Federation example
- DataLoader implementation
- GraphQL testing framework
- Schema stitching tool
- GraphQL to REST adapter

Practice: Build 8 GraphQL services

Month 9: gRPC & Microservices Foundations (Week 36-39)

gRPC & Protocol Buffers

Topics:

- gRPC introduction and architecture
- Protocol Buffers (protobuf) syntax
- Defining services and messages
- protoc compiler usage
- Code generation for Go
- Unary RPC
- Server streaming RPC
- Client streaming RPC
- Bidirectional streaming
- gRPC error handling
- Status codes and details
- Metadata and headers

- Interceptors (middleware)
- Authentication in gRPC
- TLS/mTLS configuration
- Load balancing gRPC
- Service discovery
- gRPC-Gateway for REST
- gRPC reflection
- Testing gRPC services

Projects built:

- User service with gRPC
- Streaming file transfer
- Real-time chat with gRPC
- Microservice communication
- gRPC gateway implementation
- Service mesh with gRPC
- gRPC load balancer
- Authentication service
- gRPC monitoring system
- Protocol evolution example

Practice: Build 10 gRPC services

Message Queues & Event-Driven

Topics:

- Message queue patterns
- RabbitMQ with Go
- AMQP protocol
- Exchanges, queues, bindings
- Apache Kafka with Go
- Producers and consumers
- Kafka Streams equivalent
- NATS messaging system
- NATS Streaming/JetStream
- AWS SQS integration
- Google Pub/Sub
- Event sourcing pattern
- CQRS implementation
- Saga pattern
- Outbox pattern
- Event-driven architecture
- Eventual consistency
- Idempotency handling
- Dead letter queues
- Message ordering

Projects built:

- Order processing with RabbitMQ
- Event streaming with Kafka
- NATS-based microservices
- Event sourcing example
- CQRS implementation
- Saga orchestration
- Event store
- Message broker comparison
- Event-driven e-commerce
- Real-time analytics pipeline

Practice: Build 12 event-driven systems

Authentication & Security

Topics:

- Authentication strategies
- Password hashing with bcrypt
- Session-based authentication
- Token-based authentication
- JWT implementation and validation
- OAuth 2.0 implementation
- OpenID Connect
- Social login integration
- Multi-factor authentication
- API key management
- Role-based access control (RBAC)
- Attribute-based access control (ABAC)
- Security middleware
- CSRF protection
- XSS prevention
- SQL injection prevention
- Rate limiting implementation
- DDoS protection
- Secrets management
- Vault integration

Projects built:

- Complete auth system
- OAuth provider

- Social login integration
- MFA implementation
- RBAC system
- API key manager
- Security scanner
- Rate limiter library
- Secrets management tool
- Security audit tool

Practice: Implement security in all applications

Observability & Monitoring

Topics:

- Observability principles
- Structured logging with zap/zerolog
- Log levels and formatting
- Distributed tracing with OpenTelemetry
- Jaeger integration
- Zipkin integration
- Metrics with Prometheus
- Custom metrics and labels
- Grafana dashboards
- Application Performance Monitoring (APM)
- Health checks implementation
- Readiness and liveness probes
- Error tracking with Sentry
- Distributed logging with ELK
- CloudWatch integration
- DataDog integration
- Alerts and notifications
- SLA monitoring
- Performance monitoring
- Cost monitoring

Projects built:

- Observability platform
- Distributed tracing system
- Metrics collection service
- Log aggregation pipeline
- Custom APM tool
- Health check framework
- Alert manager
- Dashboard builder
- Performance monitor
- SLA tracker

Practice: Add observability to all services

Phase 3 Capstone Project

Topics:

- Full-stack application design
- API development
- Database design
- Authentication implementation
- Testing strategies
- Deployment preparation
- Documentation
- Performance optimization

Projects built:

- MAJOR CAPSTONE: E-commerce Platform Backend
- Features: REST + GraphQL APIs, product catalog, user auth (JWT), cart, orders, payment integration, admin panel, search, gRPC internal services, message queues, caching
- Alternative: Social Media Platform (posts, real-time chat, notifications, feed algorithm)
- Alternative: Video Streaming Platform (upload, transcoding, CDN, real-time comments)
- Alternative: Food Delivery System (restaurants, orders, real-time tracking, payments)

Assessment: Phase 3 Final Exam - Web development and APIs comprehensive test

PHASE 3 COMPLETION - Deployment & DevOps (Week 27-39 (distributed))

Containerization & Orchestration

Topics:

- Docker deep dive for Go
- Multi-stage builds optimization
- Minimal images with distroless
- Docker Compose for development
- Container networking
- Volume management
- Docker registry
- Kubernetes concepts
- Pods, Services, Deployments
- ConfigMaps and Secrets
- Ingress controllers
- StatefulSets for databases
- Jobs and CronJobs
- Helm charts creation

- Operators in Go
- kubebuilder framework
- operator-sdk
- Custom Resource Definitions (CRDs)
- Admission webhooks
- Service mesh with Istio

Projects built:

- Optimized Docker images
- Kubernetes manifests
- Helm chart for application
- Custom Kubernetes operator
- CRD implementation
- Admission webhook
- Service mesh setup
- Auto-scaling configuration
- Blue-green deployment
- Canary deployment

Practice: Deploy all applications to Kubernetes

CI/CD & Automation

Topics:

- CI/CD principles
- GitHub Actions for Go
- GitLab CI configuration
- Jenkins pipelines
- CircleCI setup
- Build automation
- Test automation
- Code quality gates
- Security scanning
- Dependency scanning
- Container scanning
- Artifact management
- Semantic versioning
- Release automation
- GitOps with ArgoCD
- Flux CD
- Infrastructure as Code with Terraform
- Ansible for configuration
- Monitoring CI/CD
- Rollback strategies

Projects built:

- Complete CI/CD pipeline
- Multi-environment setup
- GitOps implementation
- IaC with Terraform
- Automated testing pipeline
- Security scanning pipeline
- Release automation
- Rollback mechanism
- Pipeline monitoring
- Cost optimization

Practice: Set up CI/CD for all projects

Cloud Platforms & Serverless

Topics:

- AWS services for Go
- EC2 and Auto Scaling
- ECS and Fargate
- Lambda with Go
- API Gateway
- S3 integration
- DynamoDB with Go
- SQS and SNS
- CloudWatch monitoring
- Google Cloud Platform
- Cloud Run
- Cloud Functions
- Firestore
- Azure services
- Azure Functions
- Serverless Framework
- SAM for AWS
- Cost optimization
- Multi-cloud strategies
- Cloud-native patterns

Projects built:

- Serverless API with Lambda
- Cloud Run deployment
- S3 file processing
- DynamoDB application

- Event-driven serverless
- Multi-cloud deployment
- Cost monitoring tool
- Cloud migration project
- Serverless orchestration
- Edge computing example

Practice: Deploy to multiple cloud providers

Production Best Practices

Topics:

- Production readiness checklist
- High availability design
- Disaster recovery planning
- Backup strategies
- Data retention policies
- Compliance requirements
- GDPR considerations
- Security hardening
- Performance tuning
- Capacity planning
- Cost optimization
- Documentation standards
- Runbooks creation
- On-call procedures
- Incident management
- Post-mortem process
- Change management
- Feature flags
- A/B testing
- Production debugging

Projects built:

- Production checklist tool
- Disaster recovery plan
- Runbook generator
- Incident response system
- Feature flag service
- A/B testing framework
- Compliance checker
- Cost analyzer
- Capacity planner
- Production readiness review

Practice: Make all applications production-ready

Integration & Enterprise Patterns

Topics:

- Enterprise integration patterns
- API composition
- Backend for Frontend (BFF)
- API aggregation
- Service discovery patterns
- Circuit breaker implementation
- Retry with backoff
- Bulkhead pattern
- Timeout patterns
- Health check patterns
- Blue-green deployment
- Canary releases
- Feature toggles
- Strangler fig pattern
- Anti-corruption layer
- CQRS deep dive
- Event sourcing implementation
- Saga pattern implementation
- Distributed transactions
- Compensation patterns

Projects built:

- API gateway with patterns
- Circuit breaker library
- Service discovery system
- Feature toggle service
- Event sourcing framework
- Saga coordinator
- Integration hub
- Pattern library
- Migration toolkit
- Enterprise integration platform

Practice: Implement 15 enterprise patterns

Master cloud-native development, advanced microservices, distributed systems, and prepare for senior engineering roles.

Months 10-11: Advanced Microservices & Distributed Systems (Week 40-48)

Microservices Architecture Deep Dive

Topics:

- Microservices principles and patterns
- Domain-Driven Design (DDD)
- Bounded contexts
- Service decomposition strategies
- Data management in microservices
- Database per service
- Shared database anti-pattern
- API composition pattern
- CQRS pattern implementation
- Event sourcing deep dive
- Saga pattern variations
- Distributed transactions
- Two-phase commit
- Compensating transactions
- Service mesh architecture
- Sidecar pattern
- Ambassador pattern
- Adapter pattern
- Service versioning
- Backward compatibility

Projects built:

- Microservices e-commerce platform
- DDD implementation
- Event sourcing system
- Saga orchestrator
- Service mesh deployment
- API composition gateway
- CQRS example
- Distributed transaction manager
- Service versioning strategy
- Migration from monolith

Practice: Design 5 complete microservices architectures

Distributed Systems Fundamentals

Topics:

- CAP theorem understanding
- Consistency models
- Consensus algorithms: Raft, Paxos
- Leader election
- Distributed locking
- Vector clocks
- Conflict-free replicated data types (CRDTs)
- Gossip protocols
- Consistent hashing
- Distributed hash tables
- MapReduce pattern
- Stream processing
- Lambda architecture
- Kappa architecture
- Time in distributed systems
- Distributed tracing
- Chaos engineering
- Fault tolerance patterns
- Byzantine fault tolerance
- Quorum-based systems

Projects built:

- Raft consensus implementation
- Distributed lock service
- Consistent hashing library
- Gossip protocol implementation
- Distributed cache
- MapReduce framework
- Stream processor
- Chaos engineering tool
- Distributed counter
- Time synchronization service

Practice: Build 8 distributed system components

Service Mesh & Advanced Orchestration

Topics:

- Service mesh deep dive
- Istio architecture and components
- Envoy proxy
- Traffic management
- Load balancing strategies

- Circuit breaking
- Retry and timeout
- Fault injection
- Canary deployments
- A/B testing
- Security: mTLS, RBAC
- Observability in service mesh
- Linkerd comparison
- Consul Connect
- Kong mesh
- Multi-cluster deployments
- Cross-cluster communication
- Service mesh federation
- GitOps for service mesh
- Progressive delivery

Projects built:

- Istio deployment
- Traffic management rules
- Security policies
- Observability dashboard
- Multi-cluster setup
- Canary deployment automation
- Service mesh migration
- Custom Envoy filter
- Service mesh operator
- Progressive delivery pipeline

Practice: Implement service mesh for all microservices

Advanced Kubernetes & Operators

Topics:

- Kubernetes internals
- Custom controllers
- Admission controllers
- Mutating webhooks
- Validating webhooks
- CRD design best practices
- Operator pattern deep dive
- Operator SDK advanced
- Kubebuilder advanced
- Controller runtime
- Reconciliation loops
- Status management
- Finalizers
- Owner references
- Garbage collection
- Leader election in operators
- Operator lifecycle management
- Operator testing
- Operator metrics
- Multi-tenancy

Projects built:

- Database operator
- Backup operator
- Autoscaler operator
- Security operator
- Monitoring operator
- GitOps operator
- Multi-tenant operator
- Operator test framework
- Operator metrics exporter
- Complex CRD with webhooks

Practice: Build 5 production Kubernetes operators

Cloud Native Security

Topics:

- Zero trust architecture
- Container security
- Image scanning
- Runtime protection
- Network policies
- Pod security policies
- OPA (Open Policy Agent)
- Falco for runtime security
- Service mesh security
- Secrets management
- HashiCorp Vault integration
- Key management services
- Certificate management
- SPIFFE and SPIRE
- Supply chain security
- SBOM (Software Bill of Materials)

- Vulnerability management
- Compliance as code
- Security observability
- Incident response

Projects built:

- Security scanner
- Policy engine with OPA
- Secrets management system
- Certificate manager
- Runtime security monitor
- Compliance checker
- Vulnerability scanner
- Security dashboard
- Incident response tool
- Supply chain validator

Practice: Secure all cloud-native applications

Month 12: System Design & Career Excellence (Week 49-52)

System Design Mastery

Topics:

- System design methodology
- Requirements gathering
- Capacity estimation
- High-level design
- Detailed design
- Database design
- API design
- Data flow diagrams
- Scalability patterns
- Caching strategies
- CDN usage
- Load balancing algorithms
- Database sharding
- Replication strategies
- Message queue selection
- Monitoring and alerting
- Disaster recovery
- Cost optimization
- Trade-off analysis
- Design documentation

Projects built:

- Design URL shortener
- Design Twitter/X
- Design YouTube
- Design Uber
- Design WhatsApp
- Design Netflix
- Design Airbnb
- Design payment system
- Design search engine
- Design CDN

Practice: Complete 20 system design problems

Performance Engineering

Topics:

- Performance requirements
- Load testing strategies
- Stress testing
- Spike testing
- Endurance testing
- k6 for load testing
- Vegeta for HTTP load testing
- pprof mastery
- Trace analysis
- Flame graphs
- Memory optimization
- CPU optimization
- I/O optimization
- Network optimization
- Database query optimization
- Caching optimization
- CDN optimization
- Compression strategies
- Performance monitoring
- SLA/SLO/SLI definition

Projects built:

- Load testing framework
- Performance test suite
- Optimization toolkit
- Performance dashboard

- SLA monitor
- Bottleneck analyzer
- Auto-scaling optimizer
- Cache optimizer
- Query optimizer
- Performance regression detector

Practice: Optimize 10 systems for performance

Advanced Cloud & Edge Computing

Topics:

- Multi-cloud architecture
- Cloud migration strategies
- Hybrid cloud patterns
- Edge computing concepts
- IoT and Go
- 5G edge applications
- WebAssembly and Go
- WASI (WebAssembly System Interface)
- Edge functions
- CDN edge computing
- Cloudflare Workers
- Lambda@Edge
- Global distribution
- Data sovereignty
- Latency optimization
- Offline-first architecture
- Progressive Web Apps backend
- Real-time synchronization
- Conflict resolution
- Event streaming at edge

Projects built:

- Multi-cloud application
- Edge computing platform
- IoT data processor
- WebAssembly service
- Edge function deployment
- Global distribution system
- Offline-first backend
- Real-time sync service
- CDN edge application
- Hybrid cloud solution

Practice: Build 8 cloud/edge solutions

AI/ML Integration & Data Engineering

Topics:

- ML model serving with Go
- TensorFlow Serving integration
- ONNX runtime
- Model versioning
- A/B testing for ML
- Feature stores
- Data pipelines
- ETL with Go
- Stream processing
- Apache Beam Go SDK
- Real-time analytics
- Time-series processing
- Recommendation systems
- Natural Language Processing
- Computer Vision APIs
- MLOps practices
- Model monitoring
- Drift detection
- Explainable AI
- Data privacy and compliance

Projects built:

- ML model server
- Feature store
- Data pipeline
- Stream processor
- Recommendation engine
- NLP service
- Computer vision API
- MLOps pipeline
- Model monitor
- Privacy-preserving ML

Practice: Build 10 AI/ML integrated systems

Blockchain & Web3

Topics:

- Blockchain fundamentals
- Consensus mechanisms

- Smart contracts basics
- Ethereum and Go
- go-ethereum (Geth)
- Web3 integration
- IPFS with Go
- Distributed storage
- Cryptocurrency basics
- Wallet implementation
- Transaction processing
- Mining concepts
- DeFi protocols
- NFT platforms
- Hyperledger Fabric
- Private blockchains
- Blockchain security
- Cryptographic primitives
- Zero-knowledge proofs
- Blockchain scalability

Projects built:

- Simple blockchain
- Cryptocurrency implementation
- Smart contract interaction
- IPFS integration
- Wallet service
- NFT marketplace backend
- DeFi protocol
- Private blockchain
- Blockchain explorer
- Consensus algorithm

Practice: Build 5 blockchain applications

PHASE 4 COMPLETION - Career Preparation (Week 40-52 (distributed))

Interview Mastery

Topics:

- Go interview preparation
- Common Go interview questions
- Concurrency questions
- System design interviews
- Coding challenges strategy
- LeetCode with Go
- HackerRank problems
- Algorithm complexity
- Data structure implementation
- Behavioral questions
- STAR method
- Leadership principles
- Technical communication
- Whiteboard coding
- Take-home assignments
- Code review skills
- Architecture discussions
- Salary negotiation
- Offer evaluation
- Career planning

Projects built:

- LeetCode 150 problems in Go
- System design portfolio
- Interview prep guide
- Mock interview recordings
- Code challenge solutions
- Technical blog posts
- Open source contributions
- Speaking proposals
- Personal brand website
- Professional network building

Practice: Daily interview preparation

Professional Portfolio

Topics:

- Portfolio website creation
- GitHub profile optimization
- README best practices
- Project documentation
- Technical writing
- Blog post creation
- Video tutorials
- Conference talks
- Meetup presentations
- Open source maintenance
- Code review participation

- Community engagement
- Mentoring others
- Technical leadership
- Team collaboration

Projects built:

- Professional website
- Technical blog with 10 posts
- YouTube tutorial series
- Conference talk submission
- Open source library
- Documentation contributions
- Code review guide
- Mentorship program
- Community project
- Technical workshop

Practice: Build professional portfolio

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-3): 25+ foundational projects - CLI tools, data structures, concurrent programs
- Phase 2 (Months 4-6): 20+ intermediate projects - web servers, APIs, database applications
- Phase 3 (Months 7-9): 25+ backend projects - microservices, gRPC, message queues, DevOps
- Phase 4 (Months 10-12): 20+ advanced projects - distributed systems, Kubernetes operators, cloud-native apps
- Final: 5 major capstone projects demonstrating cloud-native expertise

Total projects: 90+ projects from beginner to cloud-native distributed systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Go: Syntax, Types, Functions, Structs, Interfaces, Error Handling, Concurrency
- Advanced Go: Generics, Reflection, Code Generation, Testing, Benchmarking
- Web Development: HTTP, REST APIs, GraphQL, gRPC, WebSockets, Authentication
- Databases: PostgreSQL, MySQL, MongoDB, Redis, Migrations, Connection Pooling
- Microservices: Service Discovery, API Gateway, Circuit Breaker, Event-Driven Architecture
- DevOps: Docker, Kubernetes, Helm, CI/CD, GitOps, Infrastructure as Code
- Cloud: AWS (Lambda, ECS, S3), GCP (Cloud Run), Azure, Serverless, Edge Computing
- Observability: Logging, Metrics, Distributed Tracing, Prometheus, Grafana, Jaeger
- Security: Authentication, Authorization, Encryption, Secret Management, Security Scanning
- Distributed Systems: Consensus, Replication, Sharding, CAP Theorem, Event Sourcing
- Performance: Profiling, Optimization, Load Testing, Benchmarking, Memory Management
- Tools: VS Code, Golang, Git, Make, Docker, Kubernetes, Terraform, Prometheus

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 8-10 hours
 Projects: 4-6 hours
 Practice problems: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding sessions and doubt clearing
 Mentorship: 1-on-1 expert Go developer mentorship
 Community: Active Discord community with 24/7 peer support
 Code review: Expert code reviews for all major projects
 Career support: Resume review, mock interviews, job referrals to product companies
 Lifetime access: All content, future updates, new Go versions coverage
 Interview prep: Mock interviews with industry professionals
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion (4 certificates)
 Final certificate: Professional Go Developer Certification
 Cloud native certificate: Cloud Native Developer Certificate
 Kubernetes certificate: Kubernetes Developer Certificate
 Linkedin badge: Digital badges for LinkedIn profile
 Industry recognized: Recognized by top tech companies (Google, Uber, Docker, Kubernetes)
 Portfolio projects: 50+ production-ready portfolio projects

WHERE THIS CAN LEAD

- Go Developer (Junior to Senior levels)
- Backend Developer / Backend Engineer
- Cloud Native Developer
- Platform Engineer

- Site Reliability Engineer (SRE)
- DevOps Engineer
- Kubernetes Developer
- Microservices Architect
- Solutions Architect
- Technical Lead / Team Lead
- Distributed Systems Engineer
- Infrastructure Engineer
- Principal Engineer
- Freelance Go Consultant
- Startup Founder / CTO

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 4-7 LPA (Junior Go Developer)

Growing roles: Rs 7-12 LPA (Go Developer / Backend Engineer); Rs 12-20 LPA (Senior Go Developer)

Experienced roles: Rs 18-35 LPA (Lead Developer / Cloud Architect); Rs 25-55 LPA (Principal Engineer / Solutions Architect)

Freelance: Rs 2,000-8,000/hour based on expertise

International usa: \$100k-220k USD (Go Developer to Principal)

International europe: €60k-150k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need prior programming experience for this Go course?

A: No, the course starts from zero, and Go's small, strict language design actually makes it a good first language. Developers coming from Python, Java or JavaScript will clear the early phases faster and get more time on concurrency and systems topics.

Q: What projects will I build?

A: CLI tools and fundamentals projects early, then concurrent programs using goroutines and channels, REST APIs with frameworks like Gin, gRPC services, database-backed applications, and finally containerized microservices deployed with Docker and Kubernetes. Every project is GitHub-ready and mirrors real backend work.

Q: Which Go version do you teach?

A: Current Go 1.2x releases with modules and the modern toolchain, which is exactly what production teams run. Go values backward compatibility, so what you learn stays valid, and new release features are folded into the material as they arrive.

Q: Will Go help with placements and backend interviews?

A: Go is the language of cloud infrastructure: Docker and Kubernetes are written in it, and companies building microservices and high-performance APIs hire for it steadily. Interviews for those roles test concurrency reasoning, API design and systems fundamentals, all of which the later phases drill, and you finish with deployed services in your portfolio. As always, no placement guarantees; we build proof of skill, not promises.

Q: Go's official tutorials are excellent. Why take a live course?

A: The official tour teaches syntax; it cannot review your concurrency code for race conditions or tell you why your channel design deadlocks. A live mentor does that on your own code every week, and the project sequence forces you to build a real portfolio rather than skim examples. That is the gap between knowing Go and being hireable in Go.

Q: Can I attend a session before enrolling?

A: Yes, everyone starts with a free live demo class booked through learn.modernagecoders.com/contact, with no commitment or card required. You can also message us on WhatsApp at +91 91233 66161 with questions.

Q: Are classes live or recorded videos?

A: Live only: 1-hour online sessions with an instructor, interactive rather than lecture-style, and never pre-recorded. Evening and weekend slots are available, so the course fits around college semesters and full-time jobs.

Q: What do I get on completion?

A: A completion certificate plus a GitHub-ready portfolio of Go projects: APIs, gRPC services and deployed microservices. In backend hiring the repository usually gets opened before the certificate does, so the course makes sure both exist.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-golang-programming-masterclass-college>

=====

COURSE 65 OF 118: GRE & GMAT QUANTITATIVE PREP: LIVE MATHS COACHING

=====

Course link: <https://learn.modernagecoders.com/courses/gre-gmat-quant-maths-prep-course>

Subject area: Graduate Exam Mathematics

Designed for: Working professionals

Who can join: Adults preparing for graduate admissions; school maths is refreshed from the ground up

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 4-5 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online GRE and GMAT quantitative maths prep for adults: arithmetic, algebra, statistics and data sufficiency, taught to the current exam formats.

In one line: The quantitative sections of both tests are built on school maths done under time pressure with tricks laid deliberately in the way. This course rebuilds the maths and then trains the exam.

ABOUT THE PROGRAMME

The quantitative sections of the GRE and the GMAT do not test advanced mathematics. The GRE goes no higher than a second course in school algebra, and the GMAT Focus Edition covers arithmetic, algebra and statistics with no plane geometry at all. What makes them hard is that they test speed, careful reading and trap-awareness on maths most adults last touched years ago. This 6-month live course treats that honestly. The first two months rebuild the arithmetic and algebra foundation from the ground up, because a shaky foundation is where most lost marks actually come from. The middle two months cover the full content map of both exams: word problems, ratios and rates, exponents and roots, the geometry the GRE still tests, coordinate geometry, and the descriptive statistics both exams lean on. The final two months are pure exam craft: quantitative comparison for the GRE, data sufficiency and the GMAT Data Insights section, timing strategy, educated guessing, and full timed practice sets reviewed question by question. We teach to the current formats, not the old ones. That means the shorter GRE introduced in September 2023, with its two quant sections of twelve and fifteen questions, and the GMAT Focus Edition with its three forty-five-minute sections and data sufficiency moved into Data Insights. A course still teaching the old GMAT with sentence correction and an in-quant data sufficiency block is a course teaching an exam that no longer exists.

WHAT MAKES THIS COURSE DIFFERENT

- Taught to the current exams: the shorter GRE (from September 2023) and the GMAT Focus Edition, including data sufficiency now living in the Data Insights section, not the quant section
- Foundation rebuilt first: two full months of arithmetic and algebra before any exam tricks, because that is where most lost marks come from
- One course for both tests: the shared arithmetic, algebra, word-problem and statistics core taught once, then the GRE-specific and GMAT-specific pieces taught separately and clearly labelled
- Trap-awareness as a taught skill: the specific ways each exam sets bait, drilled until you see them coming
- Timed from the middle of the course, not just at the end, so pace becomes a habit rather than a panic
- Every full practice set reviewed question by question, with the focus on why the wrong answer was tempting

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations rebuilt: number properties, fractions, decimals, percentages, ratios, exponents, roots and the algebra of equations and inequalities, all at exam speed

Phase 2: The full content map: word problems, rates and work, geometry for the GRE, coordinate geometry, and the descriptive statistics and data interpretation both exams test

Phase 3: Exam craft: quantitative comparison, data sufficiency, the GMAT Data Insights section, timing and guessing strategy, and repeated full timed practice with question-by-question review

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: School maths up to about class 10, even if it is rusty. The course rebuilds arithmetic and algebra from the ground up, so being out of practice is expected and planned for, not a problem

Programming: None. This is a maths and exam-strategy course with no coding at all

Equipment: A computer with a stable internet connection, and paper and pen, since both exams

reward working things out by hand at speed

Audience: Adults preparing for graduate admissions who need the quantitative section of the GRE, the GMAT, or both. Final-year undergraduates planning ahead are welcome

WHO THIS COURSE IS FOR

Mba applicants: MBA applicants sitting the GMAT Focus Edition who need to lift a quant score that stands between them and their target schools

Masters applicants: Applicants to masters and PhD programmes sitting the GRE, where a strong quant score matters most for quantitative and technical fields

Both exam takers: Candidates deciding between the two exams, or sitting both, who want the shared maths taught once and the differences taught clearly

Rusty professionals: Working professionals years out of school who are strong in their field but need the exam maths rebuilt from the foundation

Self study stalled: Self-preppers who have plateaued and know the missing piece is structured teaching and reviewed timed practice, not another book

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: REBUILDING THE MATHS FOUNDATION (MONTHS 1-2, WEEKS 1-8)

The arithmetic and algebra both exams assume you already own, rebuilt from the ground up and drilled at exam speed, because this is where the majority of lost marks quietly come from.

Month 1: Arithmetic and Number Properties (Weeks 1-4)

Diagnostic and the Number System

Topics:

- A full diagnostic to locate your real starting point, not your assumed one
- How the current GRE and GMAT quant sections are actually built, question by question
- Integers, factors, multiples and the properties that questions hinge on
- Prime numbers, prime factorisation and why the exams love them
- Even, odd, positive, negative: the rules that turn a hard question easy
- Divisibility rules that save real seconds under time pressure

Projects built:

- Personal weakness map: your diagnostic broken down by topic into a ranked list of what to fix first

Practice: 20 number-property questions, timed loosely at first, with every mistake tagged by cause: knowledge, careless, or misread

Fractions, Decimals and Percentages

Topics:

- Fractions without a calculator: comparing, combining and simplifying at speed
- Decimals and place value, and converting fluently between the three forms
- Percentages, percentage change and the reversal trap that catches everyone
- Successive percentage changes and why they do not simply add
- Estimation as a real technique: when the exam wants an approximate answer
- The on-screen GRE calculator and the GMAT Data Insights calculator: when they help and when they waste time

Projects built:

- Percentage trap set: ten deliberately baited percentage questions worked slowly, then reworked at speed

Practice: 24 fraction, decimal and percentage questions, half of them worded to hide which operation is needed

Ratios, Proportions and Rates

Topics:

- Ratios and how to scale them without losing the units
- Proportions and direct and inverse variation
- Rates: speed, work and price per unit, all as the same idea
- Mixtures and blends, a favourite of both exams
- Reading a multi-part ratio question without dropping a condition
- The bridge from ratios to the word problems of month 3

Projects built:

- Rate reasoning: five real-world rate scenarios set up as equations before any number is plugged in

Practice: 20 ratio and rate questions, each one required to be set up in writing before it is solved

Exponents, Roots and Number Sense

Topics:

- Exponent rules until they are automatic, not looked up
- Square roots, cube roots and simplifying surds
- Scientific notation and handling very large and very small numbers
- Absolute value and what it does to inequalities
- Number-sense shortcuts: last-digit tricks, sign tricks, magnitude checks
- Sanity-checking an answer in five seconds before you commit to it

Projects built:

- Shortcut library: your own one-page sheet of the number-sense checks that will not fit in a calculator

Practice: 22 exponent and root questions, with a sanity-check written next to each final answer

Assessment: Month 1 checkpoint: a timed arithmetic and number-properties set graded on accuracy and on whether the timing held

Month 2: The Algebra Foundation (Weeks 5-8)

Expressions and Linear Equations

Topics:

- Algebraic expressions: simplifying, expanding and factoring cleanly
- Solving linear equations and rearranging formulas under time
- Translating words into equations, the single most valuable exam skill
- Substitution and picking smart numbers to test answer choices
- Working backwards from the answer options when it is faster
- Common algebra traps and the misreads that trigger them

Projects built:

- Translation drill: fifteen word phrases each converted to an equation, the step where most marks are won or lost

Practice: 24 algebra questions solved twice each: once by direct algebra, once by testing the options, to learn which is faster when

Systems, Inequalities and Absolute Value

Topics:

- Systems of equations by substitution and elimination
- Inequalities and the sign-flip rule that quietly loses marks
- Compound inequalities and interval reasoning
- Absolute value equations and inequalities
- When a system has one solution, none, or infinitely many, and why the exam asks
- Recognising a system hiding inside a word problem

Projects built:

- Two-variable challenge: a set of word problems that each resolve to a two-equation system, set up before solving

Practice: 20 systems and inequalities questions with the setup shown in full before any solving

Quadratics and Functions

Topics:

- Factoring quadratics and the common exam-friendly forms
- The quadratic formula and completing the square, used sparingly and correctly
- Functions and function notation without the fear
- Reading a function defined by a rule or a table
- Sequences: arithmetic and geometric, as the exams present them
- Spotting the quadratic buried in a word problem

Projects built:

- Quadratic recognition: ten problems where the challenge is realising a quadratic is involved at all

Practice: 22 quadratics and functions questions, each opened by naming the structure before touching the algebra

Algebra Under Exam Pressure

Topics:

- Choosing between algebra, plugging in numbers, and testing options, deliberately
- Algebraic word problems: age, mixture, work and money set up cleanly
- Multi-step problems and keeping track without losing the thread
- Where the two exams differ in how they dress algebra questions
- Building a personal decision rule for which method to reach for
- Timed algebra: getting the right answer inside the clock

Projects built:

- Method-choice audit: a mixed algebra set where you record which method you chose and whether it was the fastest available

Practice: 24 mixed algebra questions under a soft timer, reviewed for method choice as much as for the answer

Assessment: Month 2 checkpoint: a timed algebra section, marked on accuracy, timing and whether the fastest method was chosen

PHASE 2: THE FULL CONTENT MAP OF BOTH EXAMS (MONTHS 3-4, WEEKS 9-16)

Word problems, the geometry the GRE still tests, coordinate geometry that the GMAT counts as algebra, and the descriptive statistics and data interpretation both exams rely on.

Month 3: Word Problems and Geometry (Weeks 9-12)

Word Problems That Actually Appear

Topics:

- Distance, rate and time, including two-traveller and round-trip variants
- Work and combined-work problems set up without the classic error
- Mixture, interest and profit-and-loss problems
- Overlapping-sets problems and the two-way table that tames them
- Age and consecutive-integer problems
- A method for translating any dense word problem into a clean setup

Projects built:

- Word-problem playbook: your own labelled template for each of the six recurring problem types

Practice: 24 word problems spanning all six types, each set up in writing before it is

solved

Geometry for the GRE: Lines, Angles and Triangles

Topics:

- Why this month matters for the GRE and not for the GMAT Focus quant section
- Lines, angles and the parallel-line rules the exam reuses constantly
- Triangles: properties, the Pythagorean theorem and the special right triangles
- Similar and congruent triangles and the ratios they give you
- Perimeter and area worked cleanly
- The figure-not-to-scale warning and how the GRE uses it against you

Projects built:

- Triangle toolkit: a reference of every triangle fact the GRE reuses, drawn and annotated by you

Practice: 20 GRE-style geometry questions, GMAT-only students substituting an equivalent Data Insights set

Geometry for the GRE: Circles, Polygons and Solids

Topics:

- Circles: radius, diameter, circumference, area, arcs and sectors
- Quadrilaterals and regular polygons and their angle sums
- Three-dimensional solids: volume and surface area of boxes, cylinders and spheres
- Inscribed and combined figures, a favourite GRE construction
- Estimating from a figure when exact computation is slow
- Knowing which formulas to memorise and which to reconstruct

Projects built:

- Formula sheet, earned: build the geometry formula reference yourself, which is how it actually sticks

Practice: 20 circle, polygon and solid questions, each with a labelled diagram drawn before solving

Coordinate Geometry

Topics:

- The coordinate plane, distance and midpoint
- Slope and the equation of a line in its several forms
- Parallel and perpendicular lines by their slopes
- Why the GMAT Focus Edition counts coordinate geometry as algebra and still tests it
- Intersections, and reading a graph the exam gives you
- Reflections and simple transformations on the plane

Projects built:

- Line investigation: a set of coordinate problems solved both algebraically and by sketching, to see which is faster where

Practice: 22 coordinate geometry questions relevant to both exams, since this topic survives on the GMAT as algebra

Assessment: Month 3 checkpoint: a timed section mixing word problems, GRE geometry and coordinate geometry

Month 4: Statistics and Data Interpretation (Weeks 13-16)

Descriptive Statistics

Topics:

- Mean, median, mode and range, and which one a question is really testing
- Weighted averages, a heavily tested and heavily missed idea
- Standard deviation conceptually: the exams test understanding, not hand calculation
- Quartiles, percentiles and the reasoning around them
- How adding or changing a value moves the mean and the median
- The statistics traps both exams set, catalogued

Projects built:

- Average-shift study: a set of questions about how statistics change when the data changes, reasoned without brute force

Practice: 22 statistics questions weighted towards weighted averages and standard-deviation reasoning

Counting and Probability

Topics:

- The fundamental counting principle applied cleanly
- Permutations and combinations, and telling which the question wants
- Basic probability and the complement shortcut
- Probability with and without replacement
- Combined events and when to add versus multiply
- The exam-realistic level of difficulty, so you neither under nor over-prepare

Projects built:

- Counting decision tree: your own flowchart for choosing between permutation, combination and direct counting

Practice: 20 counting and probability questions pitched at the exams' actual difficulty, not competition-maths depth

Data Interpretation and Graphs

Topics:

- Reading tables, bar charts, line graphs and pie charts accurately and fast
- Multi-source questions where two graphs must be combined
- GRE data interpretation sets and how they are scored
- The visual and quantitative formats inside GMAT Data Insights
- Percentage and ratio reasoning applied to chart data
- Not being fooled by a chart drawn to mislead

Projects built:

- Graph deconstruction: a real multi-graph data set answered, then examined for how it could have been misread

Practice: 20 data-interpretation questions across chart types, timed as the sets are on the real exams

Consolidation and First Full Quant Section

Topics:

- Pulling the whole content map together into one map you can see
- A personal error log built from every month so far
- Your two or three recurring mistake patterns, named and targeted
- Pacing across a full section rather than a single question
- A pre-test routine for the quant section
- Setting a realistic target score band for your programme

Projects built:

- First full timed quant section, GRE or GMAT to match your goal, followed by a written self-review

Practice: One complete timed quant section plus a question-by-question review of every miss and every lucky guess

Assessment: Month 4 checkpoint: a full timed quant section scored, with an honest read on your current band and gap to target

----- PHASE 3: EXAM CRAFT AND TIMED MASTERY (MONTHS 5-6, WEEKS 17-24) -----

The question formats unique to each exam, the timing and guessing strategy that protects a score, and repeated full timed practice reviewed until the mistake patterns are gone.

Month 5: Exam-Specific Formats and Strategy (Weeks 17-20)

GRE Quantitative Comparison

Topics:

- The four answer choices of quantitative comparison and what each really means
- Why quantitative comparison rewards reasoning over full computation
- Testing cases: zero, one, negatives, fractions and the numbers that break patterns
- The must-not-compute mindset for a whole question type
- The specific traps built into quantitative comparison
- GMAT-only students spend this session on an equivalent problem-solving intensive

Projects built:

- Case-testing drill: quantitative comparison questions solved purely by choosing the numbers that expose the answer

Practice: 24 quantitative comparison questions, no full calculation permitted where case-testing will do

Data Sufficiency and the GMAT Data Insights Section

Topics:

- Data sufficiency: the format that asks whether you can answer, not what the answer is
- The fixed five answer choices and a reliable order to eliminate them
- Why data sufficiency now lives in GMAT Data Insights, not the quant section
- The other Data Insights formats: table analysis, graphics interpretation, multi-source reasoning, two-part analysis
- The calculator that Data Insights allows and quant does not
- GRE-only students use this session to deepen data interpretation instead

Projects built:

- Data sufficiency ladder: a graded set that builds the elimination habit from simple to genuinely tricky

Practice: 24 data sufficiency and Data Insights questions worked with the elimination framework applied every time

Timing, Pacing and Educated Guessing

Topics:

- The real time budget: roughly one minute forty-five per GRE quant question, just over two minutes on GMAT quant
- The thirty-second rule for deciding to move on
- Educated guessing that beats random, and when to deploy it
- Section-level pacing rather than fighting for every single question
- Managing the adaptive nature of the GMAT and the section-adaptive GRE
- Recovering calmly from a question that goes wrong mid-section

Projects built:

- Pacing plan: your written per-section time strategy, including your personal move-on triggers

Practice: Timed blocks focused purely on pacing decisions, tracking how many move-on calls were correct

Error Patterns and Targeted Repair

Topics:

- Mining your accumulated error log for the two or three patterns that cost the most
- Careless-error reduction as a trainable, measurable skill
- Rebuilding any content weakness the practice sections have exposed
- Distinguishing knowledge gaps from timing errors from misreads, because the fixes differ
- Re-attempting past misses cold to confirm the repair held
- Adjusting the target band with real evidence now in hand

Projects built:

- Repair sprint: your top mistake pattern attacked with a focused set until the error rate on it visibly drops

Practice: Targeted sets built from your own error log, retested a week later to prove

the fix stuck

Assessment: Month 5 checkpoint: a full timed section compared against month 4 to measure real movement

Month 6: Full Timed Practice and Test-Ready Review (Weeks 21-24)

Full Section One and Deep Review

Topics:

- A complete timed quant section under genuine exam conditions
- Question-by-question review focused on why the wrong answer was tempting
- Separating unlucky guesses from real knowledge, since only one needs fixing
- Updating the error log with the newest patterns
- Retiming the questions that were slow but correct
- Setting the single focus for the coming week

Projects built:

- Full section one, reviewed in a written report: score, error breakdown and the one thing to fix next

Practice: The full section plus a complete written review; slow-but-correct questions retimed to build margin

Full Section Two and Format Sharpening

Topics:

- A second full timed section, rotating to the other exam if you are sitting both
- Sharpening the exam-specific formats: quantitative comparison or data sufficiency, whichever is yours
- Comparing this section against the last for direction of travel
- Fatigue management across a real testing block
- Fine-tuning the pacing plan with fresh data
- Locking in the guessing strategy so it is automatic on the day

Projects built:

- Full section two, with a side-by-side comparison against section one and a revised pacing plan

Practice: The second full section plus focused drilling on whichever format still leaks marks

Full Section Three and Final Content Gaps

Topics:

- A third full timed section to make exam pace feel ordinary
- Closing the last content gaps the three sections have exposed
- A calm, repeatable pre-section routine rehearsed until it is automatic
- Test-day logistics: what to expect at the centre or on the online exam
- Building the mental stamina for a full test, not just a section
- Confirming a realistic, evidence-based target for test day

Projects built:

- Full section three, with the final short list of content gaps and a plan to close each one

Practice: The third full section plus targeted repair of any content weakness still standing

Test-Ready: Final Review and Plan

Topics:

- A final mixed timed set drawing on everything the course covered
- The last pass over the error log, now much shorter than it started
- Your personal test-day playbook: pacing, guessing rules and pre-section routine on one page
- Managing exam nerves with preparation rather than hope
- A realistic score expectation grounded in your practice results
- Where to go next if you want to lift a strong score to an excellent one

Projects built:

- Test-day playbook: a single sheet with your pacing plan, guessing rules, routine and target, ready for the exam

Practice: A final timed set and a full walk-through of the test-day playbook

Assessment: Final review: a complete timed section, a progress summary from diagnostic to now, and certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A personal weakness map built from the opening diagnostic
- A shortcut and number-sense reference sheet you build yourself
- A word-problem playbook covering all six recurring types
- A geometry formula sheet earned by reconstruction, not handout
- A running error log that drives the whole second half of the course
- A written per-section pacing and guessing plan
- Three fully reviewed timed quant sections in the final month
- A one-page test-day playbook ready to take into the exam

Total projects: A complete personal prep kit: weakness map, reference sheets, error log, pacing plan and a test-day playbook, plus multiple fully reviewed timed sections

SKILLS THE STUDENT WILL HAVE AT THE END

- A rebuilt arithmetic and algebra foundation worked at exam speed
- The full content map of both exams: word problems, GRE geometry, coordinate geometry, statistics and data interpretation
- Quantitative comparison for the GRE and data sufficiency and Data Insights for the GMAT

- Timing, pacing and educated-guessing strategy that holds under pressure
- Trap-recognition as a trained reflex on both exams
- A disciplined review habit that turns practice sections into real improvement

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, solving and reviewing questions live rather than watching worked solutions go by

Practice: 4-5 hours weekly of timed problem sets and full sections between classes

Review: Homework and practice sections reviewed with written feedback, with equal attention to timing, method choice and accuracy

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for the questions that stall you

Progress updates: Monthly progress notes tracking practice-section scores and movement toward your target band

Career guidance: Honest guidance on realistic target scores for your programme and on what the quant score does and does not decide in an application

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, awarded on finishing the course and its final timed sections

WHERE THIS CAN LEAD

- A completed graduate-admissions application with the quantitative section prepared for, whether that is an MBA, an MS, or a PhD track
- The quantitative reasoning base that graduate business and technical programmes assume from day one
- A stronger footing for any role or programme that leans on analytical and quantitative reasoning
- A foundation you can extend into our statistics or data courses if your path runs toward analytics
- The habit of timed, reviewed practice that transfers to any high-stakes exam you sit later

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The GRE and GMAT are gateways to graduate programmes, and it is those programmes, not this course, that connect to long-term career outcomes years later. The figures below are broad, publicly reported market ranges for the kinds of graduate paths these exams lead toward, shown purely for context. They are not a promise of any score, admission, or income, all of which depend on your application as a whole, the programme, the market and your own work. This course prepares you for the quantitative section and does not guarantee a score, an admission, or a salary.

Context note: A quant score is one input among many in an application. It opens doors; it does not by itself determine the outcome on the other side of them

Mba pathways: Graduate business programmes report a wide range of post-degree outcomes depending heavily on school, industry, geography and prior experience

Ms and analytics pathways: Masters routes in data, analytics, finance and engineering lead into quantitative roles whose pay varies widely by field, location and experience

Honest framing: Treat the exam as a step, not a guarantee. The honest promise this course can make is quantitative readiness and reviewed timed practice, not a number on a payslip

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does this course cover both the GRE and the GMAT, or do I have to choose?

A: Both, and that is deliberate. The two exams share most of their quantitative content: arithmetic, algebra, word problems, statistics and data interpretation are common ground, so we teach that core once, to everyone. The exam-specific pieces are then taught separately and clearly labelled: quantitative comparison and geometry for the GRE, data sufficiency and the Data Insights formats for the GMAT. If you are sitting only one exam, you focus your practice there while the other exam's students do their format-specific session; if you are sitting or still deciding between both, you get the full picture. Tell us your target at the diagnostic and we tune your practice accordingly.

Q: The exams changed recently. Is this course current?

A: Yes, and this matters more than it sounds. We teach the shorter GRE introduced in September 2023, with its two quant sections of twelve and fifteen questions, and the GMAT Focus Edition with its three forty-five-minute sections. A crucial change many older courses miss: on the GMAT, data sufficiency is no longer in the quant section at all. It now lives in the Data Insights section, and the Focus Edition quant section has no plane geometry. We teach the exams as they are today, not as they were before 2023.

Q: My maths is very rusty. Am I too far behind for this?

A: No. Rusty is the assumption the course is built on, which is exactly why the first two full months rebuild arithmetic and algebra from the ground up before any exam strategy appears. Most adults sitting these exams last studied this maths years ago, and the biggest score gains usually come from repairing that foundation rather than from clever tricks. The week 1 diagnostic tells us precisely where you stand so the early weeks target your real gaps. What the course does need from you is the practice hours; the starting level it can work with is genuinely low.

Q: Do you guarantee a particular score?

A: No, and you should be wary of any course that does. Your score depends on your starting point, the hours you put in, and your performance on the day, none of which a course can guarantee. What this course does guarantee is honest: live teaching that rebuilds the maths and drills the exam, small batches, real doubt support, and multiple full timed sections reviewed question by question. We will give you a clear, evidence-based read on a realistic target band from your practice results, and we will be straight with you about the gap and the work it takes to close it.

Q: How much time do I need to put in each week?

A: Two live one-hour classes plus four to five hours of practice a week is the plan the course is paced around. The practice is not optional padding: these exams reward drilled speed and reviewed mistakes, and the improvement happens in that practice time as much as in class. In the final month the practice includes full timed sections, which take longer in a single sitting, so allow for that. The schedule offers evening and weekend slots so working professionals can fit it around a job.

Q: Does the maths on these exams get very advanced?

A: No, and this reassures most people. Neither exam tests advanced mathematics. The GRE goes no higher than a second course in school algebra, with no trigonometry and no calculus, and the GMAT Focus quant section covers arithmetic, algebra and statistics with no plane geometry. The difficulty is not the depth of the maths; it is the speed, the careful reading, and the traps deliberately laid in the questions. That is genuinely good news, because those are trainable skills, which is exactly what the second half of the course trains.

Q: Can I use a calculator on the exam?

A: Partly, and knowing when is itself a skill. The GRE gives you an on-screen calculator for the quant section. On the GMAT Focus Edition, the quant section does not allow a calculator at all; only the Data Insights section does. Because of this, we drill mental and paper methods hard, since leaning on a calculator you may not have is a costly habit. We also teach when reaching for the calculator, where you are allowed one, actually wastes time versus estimating or reasoning to the answer.

Q: Will this course help with the rest of the exam too, not just quant?

A: This course is focused specifically on the quantitative section, which is where a structured maths rebuild and timed practice make the biggest, most reliable difference. It does not teach the verbal sections or the GMAT Data Insights verbal-style reasoning beyond the quantitative and data-analysis parts. Many students take this course precisely because quant is their weak half and they can prepare verbal on their own. In the career-guidance sessions we are happy to point you to sensible resources for the other sections so your overall preparation is balanced.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/gre-gmat-quant-maths-prep-course>

=====

COURSE 66 OF 118: HACKATHON FOR ADULTS: FREE EVENT + 12-WEEK PRO PREP (18+)

=====

Course link: <https://learn.modernagecoders.com/courses/hackathon-prep-for-adults-professionals-coding-ai-innovation-course>
Subject area: Hackathons & Professional Innovation Programs
Designed for: Working professionals
Who can join: Beginner-Friendly to Advanced (Adults 18+)
Course length: 12 weeks pro prep + Live Hackathon Weekend
Time needed: 8-10 hours/week
Certificate: Pro Hackathon Builder, Modern Age Coders Certified

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Free hackathon for adults, professionals and founders (18+), plus a 12-week Pro Prep in full-stack engineering, AI agents, system design and pitching.

In one line: Free hackathon for everyone. Pro-grade prep for those who want to build at production scale.

ABOUT THE PROGRAMME

Hackathon for Adults & Professionals is a free-to-enter hackathon and a 12-week pro-grade prep course at Modern Age Coders, designed for working professionals, career switchers, founders, students 18+, and anyone who has ever said 'I wish I could just build that idea myself.' The hackathon participation is 100% free, no fee, no employer endorsement needed. You can be a software engineer, a product manager, a designer, a marketer, a teacher, a doctor, a small-business owner, or a fresh graduate. If you're 18+ and want to build, you're in. Our only motive: build something great. Build something innovative. Use coding logic, efficient algorithms, AI agents and modern infrastructure to ship real production software in 48 hours that solves a problem people would actually pay for. The optional 12-week Pro Prep Course is for adults who want to walk into a hackathon (or a startup) with deep, current technical fluency. It is priced the same as our other professional courses, Group Rs 1,499/mo, Mini Batch Rs 2,999/mo, or 1-on-1 Rs 4,999/mo. The curriculum is dense and modern. We cover full-stack engineering with Next.js 16 + Server Actions + Cache Components, distributed system design, AI agents with tool calling and durable workflows, RAG pipelines with vector databases, prompt engineering with Claude / GPT-4o / Gemini 2, observability, security, and shipping to production on Vercel / AWS / Cloudflare. We also cover the soft skills that actually matter, scoping under time pressure, founder-grade pitching, sponsor relationship management, and demo storytelling. By Week 12, every participant has a polished GitHub portfolio, a deployed capstone with paying-customer-ready quality, a 3-minute investor-style pitch video, and competes in the Live Modern Age Coders Pro Hackathon Weekend with sponsor prizes (cloud credits, API credits, mentor calls, and cash awards).

WHAT MAKES THIS COURSE DIFFERENT

- Free hackathon participation, anyone 18+ can register at zero cost
- Prep course priced same as other Modern Age Coders professional programs
- Built for working adults, evening and weekend live sessions, all recorded
- Curriculum led by senior engineers, ex-CTOs and YC-alumni founders
- Modern 2026 stack: Next.js 16, Server Actions, AI SDK, Workflow DevKit, Vercel Sandbox
- AI Agents focus, tool calling, multi-step workflows, durable execution
- RAG done properly, chunking, embeddings, hybrid search, reranking
- System design depth: queues, caches, sharding, event-driven architectures
- Real production concerns: observability, security, on-call, cost
- 1-on-1 mentor matching with senior engineers / founders in capstone phase
- Founder-grade pitch lab with VC-style critique
- Sponsor prizes from cloud, AI and developer-tool partners
- Internship and full-time role referrals for top performers
- Career-switch friendly: pathway from non-tech career to first tech role

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Weeks 1-4): Python + TypeScript, Git/GitHub, full-stack fundamentals, first solo hackathon
Phase 2: Pro Stack (Weeks 5-8): Next.js 16 + Server Actions + Cache Components, AI agents + tool calling + RAG, system design, observability, mock hackathon
Phase 3: Hackathon Sprint (Weeks 9-12): Capstone build, mentor coaching, pitch lab, Live Pro Hackathon Weekend

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 18+
Experience: Zero coding experience welcome, career switchers and non-technical adults thrive here
Equipment: Laptop with stable internet (16GB RAM recommended for AI dev work)
Software: All free / open-source tiers (VS Code, Cursor free tier, Vercel hobby tier, Supabase free tier)
Mindset: Willingness to ship rough first versions and iterate publicly

WHO THIS COURSE IS FOR

Career switchers: Adults moving from non-tech careers (medicine, law, finance, teaching, marketing) into tech
Working engineers: Software engineers who want to upskill on AI agents, RAG, and modern Next.js
Product managers: PMs who want to ship their own ideas without waiting for engineering
Founders: Aspiring or early-stage founders building MVPs and looking for traction
Fresh graduates: Recent college grads who want to stand out with a real portfolio
Small business owners: Entrepreneurs who want to build internal tools for their business

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATIONS, ENGINEERING FLUENCY FOR ADULTS (WEEKS 1-4) -----

Adults arrive from very different starting points, some are seasoned engineers, some are PMs who've never written code, some are career switchers from medicine or finance. Phase 1 is the great equalizer. By Week 4, every participant ships a deployed full-stack app and competes in

their first solo mini-hackathon.

Week 1: Hackathon Culture, Python + TypeScript Setup, Engineering Mindset

Topics:

- What hackathons reward in 2026: shippable demos, AI-native UX, clear pitches
- MLH, SIH-grad, NASA Space Apps, AngelHack, internal corporate hacks
- Setting up a pro dev environment: VS Code, Cursor, GitHub CLI, Node 22, Python 3.12
- Python 3.12 essentials: types, dataclasses, async/await, packaging
- TypeScript 5 essentials: types, generics, narrowing, async patterns
- Reading and writing kind, clear code
- Engineering values: ship rough, refactor later; correctness over cleverness
- Git deep-dive: rebase vs merge, cherry-pick, bisect
- GitHub workflow at hackathons: feature branches, draft PRs, conflict resolution
- Hacker etiquette and 'kind code review'

Projects built:

- CLI tool in Python and same tool in TypeScript
- Personal landing page deployed to Vercel
- First open-source PR to a public repo

Practice: Configure your full pro environment with Cursor, GitHub CLI, Node 22 and Python 3.12, then write the same small utility in Python and TypeScript to compare ergonomics.

Week 2: Full-Stack Fundamentals, REST, Databases, Auth

Topics:

- REST API design: resources, verbs, status codes, idempotency
- Postgres fundamentals: schemas, indexes, transactions
- Supabase fast track: DB + auth + storage + realtime in 30 minutes
- Connection pooling, migrations, seed data
- Authentication: Clerk vs Auth0 vs custom NextAuth, tradeoffs
- RBAC and row-level security
- Caching basics: HTTP, CDN, Redis
- Email + magic links + OAuth flows
- API testing with Hoppscotch
- Production-grade error handling

Projects built:

- CRUD app with Supabase + Next.js
- Auth flow with Clerk + RBAC
- Public + private routes split with middleware

Practice: Model a two table Postgres schema in Supabase with auth enabled and expose one authenticated REST endpoint with correct status codes.

Week 3: Modern Frontend, Next.js 16, Server Actions, Tailwind, shadcn/ui

Topics:

- Next.js 16 App Router fundamentals
- Server Components vs Client Components, when to use which
- Server Actions for form submissions and mutations
- Cache Components: PPR, use cache, cacheLife, cacheTag
- Streaming and Suspense
- Route handlers and middleware (proxy)
- Tailwind CSS productivity setup
- shadcn/ui, install, customize, theme
- Dark mode, responsive design, accessibility minimums
- Image and font optimization with Next.js

Projects built:

- Marketing landing page with shadcn/ui
- Dashboard with charts (Recharts) + Cache Components
- Form-heavy app using Server Actions exclusively

Practice: Build a Next.js 16 page that mixes Server and Client Components, handles a form through one Server Action and streams a slow section with Suspense.

Week 4: First Solo Pro Mini Hackathon, Ship Production-Ready in 12 Hours

Topics:

- 12-hour pro hackathon template: 1 hr ideate, 8 hr build, 1 hr deploy, 2 hr polish + pitch
- Picking a problem worth solving, adult-grade scoping
- Building solo with Cursor + Claude + Vercel preview
- Production checklist: env vars, error pages, monitoring, mobile
- Deployment with custom domain and SSL
- Recording a 2-minute demo with Loom
- Writing a judge-ready README + architecture diagram
- First pro pitch, 2 minutes with slides
- Self-grading using rubric
- Phase 1 retrospective

Projects built:

- PHASE 1 CAPSTONE, Solo Pro Mini Hackathon: 12-hour build, deployed to a real domain, 2-minute pro demo, judge-ready README

Assessment: Phase 1 Pro Mini Hackathon Score Card with mentor feedback letter

PHASE 2: PRO STACK, AI AGENTS, RAG, SYSTEM DESIGN (WEEKS 5-8)

This is the differentiator, what separates a winning hackathon team from a generic one. Adults learn how production AI software is actually built today: agents with tool calling, durable workflows, RAG over private data, and the system design choices that scale.

Week 5: AI APIs & The Vercel AI SDK, Stream, Cache, Cost-Manage

Topics:

- Anthropic Claude 4.7, OpenAI GPT-4o, Google Gemini 2.5, tradeoffs
- Vercel AI SDK fundamentals: generateText, streamText, tools
- Streaming responses to UI with React Suspense
- Prompt caching with Anthropic, 90% cost reduction
- Function/tool calling, turning LLMs into actuators
- Multimodal: text + image + file inputs
- Structured outputs with Zod schemas
- Cost tracking, rate limiting, fallback chains
- AI Gateway for provider failover
- Latency budgets and TTFB strategies

Projects built:

- Streaming chat app with prompt caching enabled
- Multi-provider AI gateway with failover
- Structured-output extraction from PDFs

Practice: Wire streamText from the Vercel AI SDK into a chat UI with one tool call, and measure token cost before and after enabling prompt caching.

Week 6: AI Agents, Tool Calling & Durable Workflows

Topics:

- What is an AI agent? Tools, memory, loop, termination
- Building agents with Vercel AI SDK + tool calling
- Multi-step planning and reflection
- Durable workflows with Vercel Workflow DevKit
- Pause/resume, retries, crash-safe orchestration
- Vercel Sandbox for safe code execution by AI
- Memory: short-term, long-term, semantic memory
- Evaluation: how do we know an agent is good?
- MCP (Model Context Protocol) servers, extending agent capabilities
- Common failure modes: tool loops, prompt injection, runaway costs

Projects built:

- Coding agent that writes, runs and tests code in Sandbox
- Research agent that browses, synthesizes and cites sources
- Customer support agent with persistent memory + escalation tool

Practice: Build a two tool agent, for example web lookup plus calculator, with a visible step log and a hard termination condition.

Week 7: RAG Done Right, Embeddings, Vector DBs, Hybrid Search, Eval

Topics:

- When NOT to use RAG (fine-tuning vs RAG vs prompt eng)
- Embedding models: OpenAI text-embed-3, Cohere, Voyage
- Vector databases: pgvector, Pinecone, Weaviate, Qdrant
- Chunking strategies: fixed, semantic, hierarchical
- Hybrid search: BM25 + dense embeddings + reranking
- Reranking with Cohere Rerank or Voyage Rerank
- Evaluation: groundedness, context precision, faithfulness
- RAG observability: trace every retrieval and answer
- Cost and latency tradeoffs
- When RAG fails, and how to debug

Projects built:

- RAG over your own documents (PDFs + Notion export)
- Hybrid-search RAG with reranking
- Evaluation harness with at least 50 test queries

Practice: Implement a small RAG pipeline over 20 to 50 documents with pgvector, then evaluate five queries and record where retrieval failed and why.

Week 8: System Design + Mock Pro Hackathon (Teams of 2-4)

Topics:

- System design fundamentals: load, latency, consistency, partitioning
- When to add a queue (and which queue)
- Caching layers: client, CDN, app, database
- Database scaling: read replicas, sharding, connection pooling
- Event-driven architecture, webhooks, message queues
- Observability stack: logs, metrics, traces (OpenTelemetry)
- Security: OWASP top 10, CSP, secrets management
- On-call, error budgets, postmortems
- Pro Mock Hackathon: teams of 2-4, 24-hour build window
- Live judging panel with senior engineers + founders

Projects built:

- PHASE 2 CAPSTONE, Pro Mock Hackathon: 24-hour team build with deployed product, agent or RAG component, full pitch + Q&A

Assessment: Pro Mock Hackathon Judges Panel scoring + written feedback letter

PHASE 3: PRO HACKATHON SPRINT, CAPSTONE, PITCH LAB & LIVE HACKATHON WEEKEND (WEEKS 9-12)

The final 4 weeks are a real, founder-grade hackathon journey. Adults lock a capstone, get matched with senior mentors (engineers, founders, VCs as needed), build with production discipline, get founder-grade pitch coaching, and compete in the Live Pro Hackathon Weekend with sponsor prizes.

Week 9: Capstone Lock, Spec, Architecture, Cost Model, GTM Sketch

Topics:

- Idea finalists, picking a capstone with both judge-appeal and founder-potential
- Writing a 2-page Product Spec (problem, users, JTBD, MVP, stretch)
- Architecture diagram with picked tech stack and rationale
- Cost model: AI tokens, infra, third-party APIs
- Risk register and mitigation plan
- Mentor matching with senior engineers / founders
- Setting up CI, monitoring, error tracking from day one
- Drafting a 3-minute investor pitch outline
- GTM sketch: who, where, how (even for a hackathon, judges love this)
- Repo, project board, daily commit cadence rules

Projects built:

- Product Spec (2 pages)
- Architecture Diagram with cost model
- First investor-style pitch outline

Practice: Deliver your two page capstone spec with an architecture diagram, a token and infra cost model, and a risk register with mitigations.

Week 10: Heavy Build Sprint, Production Discipline at Hackathon Speed

Topics:

- Daily commits with great messages
- Pair programming with teammates
- AI-assisted coding with Cursor + Claude + Codex
- Mid-sprint mentor reviews and code walkthroughs
- Test critical paths (smoke + at least one E2E)
- Loading / empty / error states across the app
- Mobile and accessibility, judges open on phones
- Performance: Core Web Vitals, bundle size, edge runtime
- Observability: every error visible in Sentry / Vercel logs
- Daily standup logs to mentor

Projects built:

- Capstone build, 70% complete by end of Week 10
- Daily standup logs (3 lines: shipped / shipping / blocked)
- Mid-sprint demo to mentor + 3 peers

Practice: Ship daily for five days: clean commits, one mid sprint mentor review actioned, and one end to end test guarding your demo path.

Week 11: Founder-Grade Pitch Lab, VC Critique, Slides, Demo Polish

Topics:

- Hackathon judging rubric (Innovation 25 / Execution 25 / Design 25 / Pitch 25)
- VC-style judging: 'why now, why you, why this big'
- The 3-minute pro pitch: Hook Problem Solution Demo Architecture Traction (if any) Team Ask
- Slide design, minimal, image-first, 8 slides max
- Live VC-style critique sessions with senior mentors
- Recording a high-quality demo (multiple takes, edited in Loom or DaVinci)
- Voice and pacing tips from professional pitch coaches
- Q&A handling, 'I don't know' done well
- Final README polish, judges read this first
- Final code freeze, deployment health check, mobile/dark mode/edge cases

Projects built:

- Pitch Slides (8 slides, Pitch.com / Tome / Canva)
- Pitch Video (3 minutes)
- Final README with hero image, demo URL, architecture, tech stack, team, GTM sketch

Practice: Rehearse the three minute pitch on camera twice, tighten the deck to eight image first slides and prepare answers to the three hardest judge questions.

Week 12: LIVE PRO HACKATHON WEEKEND, Compete with Founders & Senior Engineers

Topics:

- Friday: opening keynote, sponsor announcements, theme reveal, team check-in
- Saturday: 18-hour final sprint with senior mentor support
- Saturday evening: pitch dress rehearsal with VC-style critique
- Sunday morning: 4-hour final polish + production deploy
- Sunday afternoon: live judging, every team pitches in front of senior judges
- Sponsor categories: Best AI Agent, Best RAG, Best Founder Potential, Best Architecture, Best Pitch, Most Innovative, Audience Favorite
- Live judge Q&A with founders and CTOs
- Awards ceremony with cash prizes, cloud credits, mentor calls and founder intros
- Hall of Fame inclusion + sponsor follow-up calls
- Graduation: Pro Hackathon Builder Certification awarded

Projects built:

- GRAND CAPSTONE, Live Pro Hackathon Submission: production-deployed product + GitHub repo +

3-minute investor pitch + 8-slide deck + judge-ready README + live presentation + Q&A
Assessment: Final Pro Hackathon Score + Founder Feedback Letter

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 6+ full-stack projects + Solo Pro Mini Hackathon (deployed)
- Phase 2 (Weeks 5-8): 6+ AI/agent/RAG/system-design projects + Pro Mock Hackathon
- Phase 3 (Weeks 9-12): Capstone build + investor pitch deck + 3-min demo + LIVE Pro Hackathon submission

Total projects: 12+ deployed projects + Live Pro Hackathon Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Python 3.12 and TypeScript 5
- Next.js 16 (App Router + Server Actions + Cache Components)
- Tailwind + shadcn/ui
- Supabase + Postgres + Redis + pgvector
- REST + tRPC + edge functions
- Vercel AI SDK + tool calling + agents
- Workflow DevKit + Sandbox + durable execution
- RAG: chunking, embeddings, hybrid search, reranking, eval
- Multi-vector-DB familiarity (pgvector, Pinecone, Weaviate, Qdrant)
- System design: queues, caches, sharding, event-driven
- Observability: Sentry, OpenTelemetry, Vercel logs
- Security: OWASP top 10, CSP, secrets
- Performance: Core Web Vitals, edge, ISR, streaming
- GitHub workflow with PRs and CI
- Founder-grade pitch decks and 3-minute demos

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (2 hours each, evening or weekend)
Independent projects: 4 hours
Weekly pro hack: 1 themed hackathon-style challenge (1-2 hours)
Total per week: 8-10 hours

SUPPORT INCLUDED

Live instruction: Senior engineers, ex-CTOs, YC-alumni founders
Mentor matching: 1-on-1 senior mentor in capstone phase
Pitch lab: Founder-grade VC-style pitch coaching
Career support: 1-on-1 LinkedIn + portfolio review for career switchers
Founder support: Mentor founders matched for capstone if you're building a startup
Community: Pro Hackathon Builders alumni network + private Slack
Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Final certificate: Pro Hackathon Builder, Modern Age Coders Certified
Capstone required: Live hackathon submission (production-deployed product + investor pitch + demo)
Portfolio: 12+ deployed projects + grand capstone

WHERE THIS CAN LEAD

- AI Engineer at startups (Rs 15-60 LPA in India, \$80k, \$200k globally)
- Full-Stack Engineer at YC / mid-stage startups
- Product Engineer / Product Manager
- Indie Hacker / Solo Founder
- Open-Source Maintainer
- Career Switcher into tech (first tech role)
- Founder of a venture-backed startup
- Hackathon-Champion track (sponsor pipelines, accelerator interviews)

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Other roles: Rs 6-18 LPA in first tech role; Rs 20-60 LPA for senior AI / full-stack roles;
Pre-seed funding range typically \$50k, \$500k from accelerators or angels; \$80k, \$200k for AI / full-stack engineers from India working remote

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is the hackathon really free for adults?

A: Yes, 100% free for anyone aged 18+. No fee. No employer or college endorsement needed. Free certificate. Eligible for all sponsor prizes.

Q: Why is the prep course paid?

A: The prep course is optional. It's for adults who want serious, structured 12-week training in modern engineering, AI agents, RAG, system design and pitching. The hackathon participation stays free regardless.

Q: I'm a non-technical adult, can I still do this?

A: Yes, and you're our favorite kind of student. Many of our top hackathon winners have come from non-technical backgrounds (medicine, law, finance, teaching). Phase 1 starts from absolute zero. AI tools (Cursor, Replit Agent, Bolt.new) close the gap dramatically. Within 4 weeks, you'll be deploying full-stack apps.

Q: I'm a senior engineer, will this be too basic?

A: No. The Phase 2 content (AI agents, RAG done right, durable workflows, system design at hackathon scale, observability) is genuinely current and challenges senior engineers. We also let experienced engineers fast-track Phase 1 in 2 weeks if they pass a placement test.

Q: What's the time commitment?

A: Plan for 8-10 hours weekly: 2 live sessions (2 hours each, evenings or weekends), 4 hours of independent project work, and 1-2 hours for the weekly pro hackathon-style challenge. All sessions are recorded for working professionals who can't always attend live.

Q: What stack will I learn?

A: The 2026 modern stack: Python 3.12 + TypeScript 5, Next.js 16 (App Router + Cache Components), Tailwind + shadcn/ui, Supabase / Postgres / Redis / pgvector, Vercel AI SDK + Workflow DevKit + Sandbox, Anthropic Claude 4.7 / OpenAI GPT-4o / Gemini 2.5, plus AWS / Cloudflare for production deployment. Cursor and AI agents are first-class throughout.

Q: Will this help me switch careers into tech?

A: Yes, directly. Career switchers are a big part of our cohort. You graduate with a deployed capstone, a public GitHub portfolio, a recorded pitch, a Pro Hackathon Builder certification, and a 1-on-1 LinkedIn and portfolio review. We help you present your work well for applications and interviews, though outcomes depend on your effort and the market.

Q: Can founders use this to validate their startup idea?

A: Absolutely. Many founders use the capstone phase to build their MVP. We pair you with mentor founders if you choose. Several alumni have raised pre-seed funding off their hackathon capstone.

Q: Are there sponsor prizes?

A: Yes. Sponsor lineup announced 2 weeks before each quarterly hackathon.

Q: Do you cover system design and DSA?

A: System design, yes, deeply, at hackathon depth (queues, caches, sharding, event-driven). DSA, to the level you need to write efficient hackathon code. For deep DSA prep (interview-grade), we recommend our Vibe Coding for College / DSA Masterclass program.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/hackathon-prep-for-adults-professionals-coding-ai-innovation-course>

=====

COURSE 67 OF 118: HACKATHON FOR KIDS: FREE EVENT + 12-WEEK PREP (AGES 8-12)

=====

Course link: <https://learn.modernagecoders.com/courses/hackathon-prep-for-kids-coding-innovation-ai-projects-course>

Subject area: Hackathons & Innovation Programs

Designed for: Kids

Who can join: Complete Beginner (Ages 8-12)

Course length: 12 weeks prep + Live Hackathon Weekend

Time needed: 4-6 hours/week

Certificate: Junior Hackathon Builder, Modern Age Coders Certified

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Free hackathon for kids ages 8-12, plus a 12-week prep course in coding, AI, problem-solving and innovation. Build a project, pitch it and win judge prizes.

In one line: Anyone can join. Free to participate. Built to spark India's youngest innovators.

ABOUT THE PROGRAMME

Hackathon for Kids is a free-to-enter hackathon and a 12-week structured prep course at Modern Age Coders, designed for children aged 8 to 12 who want to learn how real innovators build technology. The hackathon participation is 100% free, anyone, from any school or city, can register and compete. The optional 12-week prep course (priced the same as our other coding courses) is for kids who want serious training before they compete. Our only motive is simple: build something great. Build something innovative. Use coding logic, efficient algorithms and AI tools to turn raw ideas into real, working projects in just one weekend. Kids learn how a real hackathon works, how teams form, how ideas get scoped, how problems are decomposed, how AI is used to accelerate building, how you debug under pressure, and how you pitch your project to judges. They use Scratch, Replit, Bolt.new, Google Gemini, Microbit and Canva to build games, AI apps, websites and inventions that solve real problems for kids their age. This is not a normal class. This is a creator's training camp. By the end, every child has a portfolio of 4+ hackathon-ready projects, a 90-second pitch video, a working capstone submission, and the confidence to walk into any school, district or international hackathon and stand on stage.

WHAT MAKES THIS COURSE DIFFERENT

- 100% free hackathon participation, anyone can join, no fees, no gatekeeping
- Hackathon prep priced same as other Modern Age Coders courses, group, mini-batch and 1-on-1 monthly plans
- Designed by mentors who have judged and won 50+ hackathons
- Kids learn the full hackathon arc: ideate plan build debug pitch demo
- AI-assisted building with Replit Agent, Bolt.new and Google Gemini
- Algorithm thinking made playful, sorting, searching, pathfinding through games
- Live mock hackathons every 4 weeks with judge feedback
- Team formation workshops, kids learn collaboration and roles (Builder, Designer, Pitcher)
- Pitch coaching, every child practices a real 90-second demo to camera
- Final hackathon weekend with prizes, judge panel, and Hall of Fame inclusion
- Parent dashboard, recorded sessions, and weekly progress reports

THE LEARNING PATH, STEP BY STEP

Phase 1: Hackathon Foundations (Weeks 1-4): How hackathons work, coding basics in Scratch, problem-solving, idea generation

Phase 2: Build Skills (Weeks 5-8): AI tools, Replit & Bolt.new, basic algorithms, team-building, design with Canva

Phase 3: Hackathon Sprint (Weeks 9-12): Mock hackathons, capstone build, pitch coaching, demo day, live hackathon weekend

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 8-12 years old

Experience: Absolutely none, beginners welcome

Equipment: A computer or laptop with internet connection

Software: All tools are free and browser-based, nothing to install

Parental support: Encouraged to attend final demo day

WHO THIS COURSE IS FOR

Curious kids: Children aged 8-12 who love building things

Competitive kids: Kids who enjoy challenges, time-limits and judges

Shy kids: Quiet kids who want a safe place to find their voice (pitch coaching changes lives)

Ai curious kids: Kids who want to learn AI tools the right way

School innovation clubs: School coding club members preparing for inter-school hackathons

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: HACKATHON FOUNDATIONS, HOW HACKATHONS WORK (WEEKS 1-4) -----

Kids learn what a hackathon is, why it exists, how teams build software in 48 hours, and how to start thinking like an inventor. They build their first 'mini hack' projects in Scratch and learn the language of hackathons: idea, MVP, sprint, demo and pitch.

Week 1: What is a Hackathon?, Welcome to the Builder Mindset

Topics:

- What is a hackathon? Origin story from Silicon Valley to Indian schools
- Famous kids who built real apps in hackathons (case studies)
- The 4 stages of every hackathon: Idea Build Demo Judge
- What is an MVP (Minimum Viable Product)?, explained for kids
- Why anyone can join: free hackathon access at Modern Age Coders
- Setting up your hacker profile: name, team, project board
- Tour of Scratch 3.0, your first build platform
- First mini-hack: build a 'Welcome' interactive card in 60 minutes
- Hackathon vocabulary kids need: bug, ship, push, demo, judge, pitch

- Online safety, kindness in teams, and being a good teammate

Projects built:

- 60-Minute Mini Hack: Animated Welcome Card
- Hacker ID Card (designed in Canva)
- Idea Wall, write 10 things in the world you'd like to fix

Practice: Watch one short hackathon demo video with a parent and write down the problem, the solution and your favorite part of the pitch.

Week 2: Idea Generation, Where Hackathon Ideas Come From

Topics:

- How to spot problems worth solving (kids edition)
- The 'I wish this existed' game, how every great app started
- Idea generation techniques: brainstorming, mind maps, sticky notes
- Good ideas vs. great ideas, judging your own thoughts
- User empathy for kids, designing for a friend, sibling, or grandparent
- Competitor research made fun, what already exists?
- Picking your problem statement and writing it in one sentence
- Sketching your first wireframe by hand and on paper
- Storyboarding your idea, 6 panels, like a comic strip
- Sharing ideas in a group without fear, feedback rules

Projects built:

- My Idea Comic, 6-panel storyboard of your hackathon dream project
- Problem Statement Card (one sentence)
- Empathy Interview, interview a family member about their daily problems

Practice: Keep an 'I wish this existed' notebook all week and bring your three best problem ideas to class.

Week 3: Coding Logic, Loops, Conditions & Variables in Scratch

Topics:

- Loops: how computers repeat tasks, the foundation of efficient code
- If-Then-Else: making computers make decisions
- Variables: storing scores, names, and data
- Why every hackathon project needs these 3 building blocks
- Sequence, selection, repetition, the holy trinity of coding logic
- Building a number guessing game, your first 'algorithm'
- Debugging: finding bugs without crying, printing values to investigate
- Comments and notes, making your code readable to teammates
- Hackathon-style challenges: 30-minute build sprints
- Code reviews with classmates, being kind and helpful

Projects built:

- Number Guessing Game with hints
- Score-Tracking Quiz Game (5 questions)
- Loop Art, generate a star pattern using only loops

Practice: Build a mini Scratch scene that uses one loop, one if then else and one variable, and label each with a comment.

Week 4: First Mini Hackathon (Solo), Build & Pitch in 4 Hours

Topics:

- Hackathon pacing: how to use 4 hours wisely
- The 4-hour template: 30 min ideate, 2.5 hr build, 30 min polish, 30 min pitch
- Choosing scope: what NOT to build is more important than what to build
- Building your project from a problem statement
- Time-boxing: setting timers and sticking to them
- What to do when stuck (the '15-minute rule')
- Recording a 60-second demo video with Loom or screen capture
- Writing your project's README, title, problem, solution, how to use
- Pitching to mentors: how to talk about your build with confidence
- Phase 1 review and Phase 1 capstone presentation

Projects built:

- PHASE 1 CAPSTONE, Solo Mini Hackathon: 4-hour build + 60-second demo + 1-page README. Choose any problem from your Idea Wall.

Assessment: Phase 1 Mini Hackathon Assessment, judged on innovation, execution, clarity

PHASE 2: BUILD SKILLS, AI, ALGORITHMS & TEAM HACKING (WEEKS 5-8)

Kids level up to AI tools, learn the basics of algorithms (the secret sauce of efficient hackathon projects), discover Replit and Bolt.new, and start working in teams of 2-3. By Week 8, they will have built their first AI-powered hackathon project with a partner.

Week 5: Vibe Coding with AI, Build Apps by Talking to Computers

Topics:

- What is AI and how do hackers use it to ship faster?
- Meet Replit Agent, describe an app, watch it appear
- Meet Bolt.new, full apps from a single sentence
- Prompt writing for hackathons: be specific, be visual, be small
- Iterating with AI: 'change the color', 'add a button', 'fix this bug'
- AI vs. human coding: when to use which
- Reading AI-generated code without panicking

- Asking Google Gemini to explain code line-by-line
- Limits of AI, what it gets wrong, why you still need to think
- Ethical AI use in hackathons, credit, originality, transparency

Projects built:

- AI-Built Calculator with custom theme (Replit Agent)
- AI-Built Quiz App on a topic you love (Bolt.new)
- AI-Explained Code Journal, print 3 AI snippets and write what each line does

Practice: Ask Replit Agent or Bolt.new for a one screen app about your hobby, then request two changes: a new color and an extra button.

Week 6: Algorithms for Kids, Sorting, Searching & Pathfinding

Topics:

- What is an algorithm? Simple recipe metaphor
- Why algorithms matter in hackathons (slow apps lose, fast apps win)
- Sorting: bubble sort visualized as kids in a line
- Searching: linear vs. binary, finding the right book on the shelf
- Pathfinding basics, the maze game
- Big O for kids: 'fast', 'slow', 'really slow' (no math symbols yet)
- Choosing the right algorithm for your hackathon idea
- Using AI to suggest algorithms when stuck
- Visualizing algorithms in Scratch with sprites
- Mini-challenge: implement 2 sorts and race them

Projects built:

- Sorting Race, animate bubble sort vs. insertion sort with sprites
- Maze Solver, a sprite that finds the exit using a simple algorithm
- Word Search Game, implement linear search visually

Practice: Explain bubble sort to a family member using playing cards lined up in a row, then repeat it and try to sort faster.

Week 7: Team Hacking, Roles, GitHub Basics & Collaboration

Topics:

- Why hackathons love teams of 2-3
- The 3 hackathon roles: Builder (writes code), Designer (makes it pretty), Pitcher (tells the story)
- Picking a teammate, strengths and personalities
- Communication tools: Google Docs, Figma Jam, voice notes
- Sharing code through Replit Multiplayer (kid-safe)
- Introduction to GitHub, what is a repository?
- Forking, committing and pushing, explained with playground analogies
- Resolving 'who builds what' fights without drama
- Daily 5-minute stand-ups: what I did, what I'm doing, what I need help with
- Sharing credit, every teammate's name on the project

Projects built:

- Team Charter, 1 page on roles, rules, and project goals
- Pair-Built Story App, two kids, one app, built collaboratively in Replit
- Mini GitHub Profile, your first repo with your project README

Practice: Run a 30 minute mini build in Replit Multiplayer with a friend or sibling where one builds and one designs, then swap roles.

Week 8: Mock Hackathon (Teams), 8-Hour Sprint Across Two Days

Topics:

- Forming hackathon teams (2-3 kids, mentor-paired)
- Selecting a problem statement from a curated list
- Day 1: 4-hour build sprint with Replit + Bolt.new + Scratch
- Day 2: 2-hour polish + 1-hour pitch prep + 1-hour live demos
- Time-boxing: how mentors keep teams from over-engineering
- Mid-sprint check-ins with mentors
- Designing a 1-slide pitch deck in Canva
- Recording a 90-second demo video together
- Live presentation to peer judges
- Receiving feedback gracefully, what worked, what to improve

Projects built:

- PHASE 2 CAPSTONE, Team Mock Hackathon: pick a problem, ship an MVP in 8 hours, deliver a 90-second demo with team intro and judge Q&A

Assessment: Mock Hackathon Judges Panel, innovation, technical execution, design, pitch clarity, teamwork

PHASE 3: REAL HACKATHON SPRINT, CAPSTONE, PITCH & LIVE HACKATHON WEEKEND (WEEKS 9-12)

The final phase is a real hackathon journey. Kids pick their dream project, get paired with mentors, build their capstone over 4 weeks, get pitch coaching, record final demos, and compete in the official Modern Age Coders Live Hackathon Weekend at the end of Week 12, the same hackathon that's free for the public.

Week 9: Capstone Idea Lock, Mentor Matching & Project Plan

Topics:

- Picking your capstone problem statement (3-5 idea finalists)
- Mentor matching, every team gets a personal hackathon mentor

- Writing the capstone proposal: problem, users, solution, tech stack
- Hackathon judging rubric explained (innovation 25%, execution 25%, design 25%, pitch 25%)
- Designing your project board, Trello-for-kids visual planning
- Stretch goals vs. core MVP
- Risk planning: what could go wrong and how we'll handle it
- First architecture sketch (boxes-and-arrows)
- Setting up your repo, your team channel, your Canva board
- Locking the scope, saying NO to feature creep

Projects built:

- Capstone Proposal Document
- MVP Definition (one paragraph)
- Architecture Sketch (paper or Figma Jam)

Practice: Fill your capstone project board with To Do, Doing and Done columns and at least eight task cards.

Week 10: Build Sprint, Code, AI, Iterate, Repeat

Topics:

- Day-by-day build plan (kids learn to plan their week)
- Working with AI tools at high speed
- Debugging strategies: rubber duck debugging, AI-assisted debugging
- Component thinking, break big projects into small reusable parts
- Asset creation in Canva, backgrounds, sprites, icons
- Sound and music for projects (free libraries)
- Mid-week mentor reviews: what's done, what's blocked
- Pair programming with your teammate
- Quality bar: what 'good enough' looks like for a kids hackathon
- Daily commits, writing kind, clear messages

Projects built:

- Capstone build, 70% complete by end of Week 10
- Daily standup logs (5 lines per day)
- Mid-sprint demo to mentor

Practice: Finish two build tasks from your project board and note one bug you found plus how you or the AI fixed it.

Week 11: Pitch Coaching & Demo Polish, Tell the Story

Topics:

- What judges actually look for: clarity, impact, energy, originality
- The 90-second pitch formula: Hook Problem Solution Demo Ask
- Practicing in front of a mirror, then a parent, then your team
- Recording your pitch on camera (5+ takes)
- Slide design in Canva, title, problem, solution, tech, team, thank you
- Demo recording with Loom, show, don't tell
- Voice tips for kids: pace, pauses, smiles
- Handling Q&A, what to say when you don't know the answer
- Making your project README judge-ready (problem, demo link, screenshots, team)
- Final polish on your build: bugs, edge cases, mobile view

Projects built:

- Pitch Slides (5 slides, Canva)
- Pitch Video (90 seconds, recorded)
- Final README (judge-ready)

Practice: Record your 90 second pitch on camera at least three times and pick the take where your hook sounds most exciting.

Week 12: LIVE HACKATHON WEEKEND, Compete, Demo & Get Crowned

Topics:

- Friday evening: kickoff ceremony, theme announcement, team check-in
- Saturday all-day: 8-hour final sprint with mentor support
- Saturday evening: pitch dress rehearsal
- Sunday morning: final 2-hour polish + submission upload
- Sunday afternoon: live judging, every team presents
- Categories: Most Innovative, Best Built, Best Designed, Best Pitch, Audience Favorite
- Live judge Q&A, handled with grace and confidence
- Awards ceremony, certificates, badges and Hall of Fame inclusion
- Reflection circle: what I learned about myself
- Graduation: Junior Hackathon Builder Certification awarded

Projects built:

- GRAND CAPSTONE, Live Hackathon Submission: working project + 90-second demo video + 5-slide pitch deck + judge-ready README + live presentation

Assessment: Final Hackathon Scoring + Judge Feedback Letter

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 6+ Scratch mini-hacks + Solo Mini Hackathon
- Phase 2 (Weeks 5-8): 6+ AI/algorithm projects + Team Mock Hackathon
- Phase 3 (Weeks 9-12): Capstone build + Pitch deck + Demo video + LIVE Hackathon submission

Total projects: 12+ projects + Grand Live Hackathon Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Hackathon Mindset & MVP scoping
- Scratch 3.0 programming logic
- Vibe Coding with Replit Agent and Bolt.new
- AI Tools: Google Gemini, Teachable Machine
- Algorithm fundamentals: sorting, searching, pathfinding
- Team collaboration with Builder/Designer/Pitcher roles
- Canva pitch deck design
- Loom demo video recording
- Public speaking and 90-second pitches
- Project documentation: README, problem, solution

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (1.5 hours each)
 Independent projects: 1-3 hours
 Vibe challenge: 1 themed weekly hack
 Total per week: 4-6 hours

SUPPORT INCLUDED

Live instruction: Expert hackathon mentors with judging experience
 Parent dashboard: Weekly progress tracking
 Age appropriate: Designed exclusively for ages 8-12
 Safe environment: Moderated, kind, bully-free classroom
 Community: Junior Hackathon Builders Hall of Fame

CERTIFICATION DETAIL

Final certificate: Junior Hackathon Builder, Modern Age Coders Certified
 Capstone required: Live hackathon submission with demo and pitch
 Portfolio: 12+ projects + grand capstone

WHERE THIS CAN LEAD

- Junior Hackathon Builder
- Future Coding Olympiad Participant
- School Innovation Club Leader
- Ready for Hackathon for Teens program
- Foundation for science fairs, robotics leagues and pitch competitions

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is the hackathon really free for kids?

A: Yes, 100% free. Any child aged 8-12 can register and compete in the Modern Age Coders Live Hackathon at zero cost. No fees, no hidden charges, no payment required to participate, get judged, or receive a certificate.

Q: Then why is there a paid prep course?

A: The prep course is optional. It's for parents and kids who want serious, structured training before the hackathon, coding skills, AI tools, algorithm thinking, team practice and pitch coaching. Many kids do the prep, many compete without it. Both are valid paths.

Q: Does my child need any prior coding experience?

A: Absolutely not. The prep course starts from zero. Kids learn Scratch, basic logic, AI tools, and team-building from scratch. Even kids who have never touched a computer in coding mode are welcomed.

Q: What is the age range?

A: Ages 8-12. The content, pace, projects and pitch coaching are all designed for this age group. We have separate Hackathon programs for Teens (13-17) and Adults (18+).

Q: How much time will the prep course take per week?

A: Plan for 4-6 hours weekly: 2 live classes (1.5 hours each) plus 1-3 hours of independent project work. All sessions are recorded so your child never falls behind if they miss one.

Q: What does my child build in the prep course?

A: Over 12 weeks, your child will build at least 4 mini-hackathon projects, 1 team project, and 1 grand capstone, all judge-ready with demo videos, README documentation, and Canva pitch decks. Plus they compete in the live free hackathon at the end.

Q: Do they really learn algorithms at age 8?

A: Yes, visually and playfully. We teach sorting, searching and pathfinding through Scratch animations, mazes and games. No math heavy notation. The goal is to build algorithm intuition, not memorize Big O. This intuition compounds for years.

Q: What do they get at the end?

A: A Junior Hackathon Builder certificate (Modern Age Coders Certified), a public portfolio of 5+ projects, a 90-second pitch video, a Hall of Fame listing if they place top-10, and direct admission preference to our Hackathon for Teens program.

Q: What tools will my child use?

A: Scratch 3.0, Code.org, Replit, Bolt.new, Google Gemini, Google Teachable Machine, Canva, Figma Jam, Loom, MakeCode Arcade, and basic GitHub. All free, browser-based, and kid-safe.

Q: Are there refunds if my child doesn't enjoy it?

A: Yes. If you are not satisfied within the first 2 weeks of the prep course, we offer a full refund, no questions asked. The free hackathon participation is, of course, free.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/hackathon-prep-for-kids-coding-innovation-ai-projects-course>

=====

COURSE 68 OF 118: HACKATHON FOR TEENS: FREE EVENT + 12-WEEK PREP (AGES 13-17)

=====

Course link: <https://learn.modernagecoders.com/courses/hackathon-prep-for-teens-coding-ai-build-innovate-win-course>

Subject area: Hackathons & Innovation Programs

Designed for: Teens

Who can join: Beginner-Friendly to Intermediate (Ages 13-17)

Course length: 12 weeks prep + Live Hackathon Weekend

Time needed: 6-8 hours/week

Certificate: Teen Hackathon Builder, Modern Age Coders Certified

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Free hackathon for teens 13-17, plus a 12-week prep course in Python, JavaScript, AI APIs, algorithms and pitching. Build apps and win judge prizes.

In one line: Free hackathon for everyone. Serious prep for those who want to win.

ABOUT THE PROGRAMME

Hackathon for Teens is a free-to-enter hackathon and a 12-week structured prep course at Modern Age Coders for students aged 13 to 17. The hackathon participation is 100% free for everyone, no fees, no school filtering. The prep course is for teens who want to walk into a hackathon with real coding skills, AI fluency, and a proven pitch playbook. Our only motive is simple: build something great. Build something innovative. Use coding logic, efficient algorithms, AI APIs, and modern dev workflows to ship real software in 48 hours. We don't teach kids to copy templates. We teach them how engineers actually build. Teens go deep, Python and JavaScript fundamentals, REST API design, algorithm complexity (Big O), data structures (arrays, hash maps, trees, graphs), AI APIs (OpenAI, Gemini, Claude), GitHub-based collaboration, system design lite, and end-to-end deployment on Vercel. They build hackathon-ready apps using the same stack used by professional engineers in 2026, React, Next.js, Tailwind, Supabase, Replit Agent, Bolt.new, Cursor and Vercel. By Week 12, every teen has a polished GitHub portfolio of 6+ projects, a public capstone deployed live on the internet, a 2-minute pitch video, and a placement on the Modern Age Coders Hackathon Hall of Fame. They graduate hackathon-ready, for school events, MLH-style competitions, Smart India Hackathon, and global online hackathons.

WHAT MAKES THIS COURSE DIFFERENT

- Free hackathon participation, anyone aged 13-17 can register at zero cost
- Prep priced same as our other teen courses (Group Rs 1,499 / Mini Batch Rs 2,999 / 1-on-1 Rs 4,999, all monthly)
- Curriculum designed by hackathon winners and senior engineers
- Real production stack: React, Next.js, Tailwind, Supabase, Vercel
- AI-first dev workflow with Cursor, Replit Agent, Bolt.new
- Algorithms taught for hackathons, practical, not academic
- GitHub-native: every project committed, branched, deployed
- Live deployments, every project gets a real URL judges can click
- 1-on-1 mentor matching for capstone phase
- Pitch deck lab with senior pitch coaches
- Mock hackathons every 4 weeks with judge feedback
- Final hackathon weekend, live judges, prizes, Hall of Fame

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Weeks 1-4): Python + JavaScript essentials, Git/GitHub, problem-solving

Phase 2: Build Stack (Weeks 5-8): React + Tailwind + Supabase, AI APIs, algorithms & data

structures, system design lite

Phase 3: Hackathon Sprint (Weeks 9-12): Capstone build, mentor coaching, pitch lab, Live Hackathon Weekend

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 13-17 years old

Experience: Zero coding experience welcome, strong fit for self-starters

Equipment: Laptop or desktop with stable internet (8GB RAM recommended)

Software: All free / open source (VS Code, Cursor free tier, Vercel hobby tier)

Mindset: Willingness to ship rough, ugly first versions

WHO THIS COURSE IS FOR

Ambitious teens: Teens aiming for top colleges and want a real portfolio

Competitive teens: Teens who love coding contests, MLH, SIH, Olympiad

Founder curious: Teens who want to learn how startups actually build

AI native teens: Teens who want AI fluency at the API level (not just ChatGPT chat)

Career planners: Teens planning a tech career with strong GitHub from day one

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATIONS, PYTHON, JAVASCRIPT & GIT FOR HACKERS (WEEKS 1-4) -----

Teens build serious coding fluency. We cover Python (logic + scripting), JavaScript (the universal language of the web), and Git/GitHub workflow that every hackathon team uses. By Week 4, every student can ship a small web app to a real URL.

Week 1: Hackathon Culture, Python Basics & The Hacker Mindset

Topics:

- What is a hackathon really? MLH, SIH, NASA Space Apps, internal startup hacks
- Famous teen hackathon winners (cases studies from India and globally)
- The hackathon arc: idea MVP demo pitch judge
- Python install + setup (with VS Code)
- Variables, types, operators, strings, lists, dicts
- Conditionals (if/elif/else), loops (for/while)
- Functions, return values, default args
- Reading inputs, printing outputs, basic file IO
- Your first Python mini-tool: a CLI to-do list
- Hacker etiquette: be kind, be curious, ship over perfect

Projects built:

- CLI To-Do List in Python
- Number guessing game with hints + difficulty levels
- Data formatter: read a CSV, print summary stats

Practice: Set up Python and VS Code, then write a script that stores five hackathon ideas in a list of dicts and prints them ranked by excitement.

Week 2: JavaScript Essentials & DOM, The Universal Hackathon Language

Topics:

- Why JavaScript runs the modern web (and most hackathons)
- let / const, types, functions, arrow functions
- Arrays, objects, destructuring, spread / rest
- Async/await, promises, fetch, calling APIs
- DOM manipulation, event listeners
- ES Modules and modern JS structure
- TypeScript intro, types make hackathon code safer
- Building a client-side mini app
- Browser DevTools for debugging
- Common JS gotchas (== vs ===, hoisting, this)

Projects built:

- Weather App calling a public weather API
- Markdown live preview editor
- GitHub user profile fetcher

Practice: Build a small webpage that fetches from a public API with async await and updates the DOM when a button is clicked.

Week 3: Git, GitHub & Team Coding, Professional Workflow

Topics:

- Git fundamentals: init, add, commit, log, diff
- Branches, merging, conflicts and how to resolve them
- GitHub: repos, README, issues, pull requests
- Forking and cloning open-source projects
- Pull request reviews and team etiquette
- GitHub Actions intro, automated CI
- GitHub Pages deployment
- Hackathon team workflow: feature branches, PR per feature

- Daily standups: what I did, what I'm doing, blockers
- Hackathon-grade README structure (problem, demo, screenshots, setup, team)

Projects built:

- Personal Portfolio site deployed via GitHub Pages
- Open-source contribution: file 1 PR to a public repo
- Team-built mini app with branches and PRs

Practice: Create a repo with a proper README, make five meaningful commits on a branch, then open and merge your own pull request.

Week 4: First Mini Hackathon (Solo), Ship to a Live URL in 8 Hours

Topics:

- 8-hour hackathon template: 1 hr ideate, 5 hr build, 1 hr polish, 1 hr pitch
- Picking a problem statement that's small, clear, and demo-able
- Building solo with Replit Agent + Cursor for speed
- Deployment to Vercel, live URL judges can click
- Recording a 90-second demo with Loom
- Writing your README so judges understand in 60 seconds
- First pitch attempt, to camera, then to mentors
- Self-grading with the official rubric
- Code reviews from senior mentors
- Phase 1 retrospective and feedback letter

Projects built:

- PHASE 1 CAPSTONE, Solo Mini Hackathon: 8-hour build, deployed to Vercel, 90-second demo, judge-ready README

Assessment: Phase 1 Mini Hackathon Score Card, innovation, execution, design, pitch

PHASE 2: BUILD STACK, REACT, AI APIS, ALGORITHMS & SYSTEM DESIGN (WEEKS 5-8)

Teens learn the full modern hackathon stack. By Week 8, they can build a deployable React + Tailwind + Supabase app with an AI feature in under 24 hours.

Week 5: React + Tailwind, The Hackathon UI Stack

Topics:

- React fundamentals: components, props, state, hooks
- useState, useEffect, useRef in real projects
- Tailwind CSS, design without writing CSS files
- Building responsive layouts in minutes
- shadcn/ui components, copy/paste production-grade UI
- Forms, validation, controlled components
- Routing with Next.js App Router
- Server Components vs Client Components
- Iconography, dark mode, accessibility basics
- Building UI from a Figma mock in under 30 min

Projects built:

- Personal landing page with React + Tailwind
- Recipe finder calling an external API
- Interactive dashboard with charts (Recharts)

Practice: Rebuild your week 2 page in React with Tailwind and one shadcn/ui component, keeping it responsive at phone width.

Week 6: Backend with Supabase + REST APIs, Build a Real Product

Topics:

- Supabase setup, database, auth, storage in 5 minutes
- Postgres tables, schema design for hackathons
- Row-Level Security basics (auth that doesn't leak)
- Building REST APIs with Next.js Route Handlers
- CRUD apps end-to-end
- Email + magic link authentication
- File uploads to Supabase Storage
- Webhooks and realtime subscriptions
- Postman / Thunder Client for testing APIs
- Connecting frontend backend smoothly

Projects built:

- Notes app with auth + Supabase database
- Image gallery with uploads + storage
- Realtime chat (single-room) with Supabase Realtime

Practice: Stand up a Supabase project with one table protected by Row Level Security and wire a working CRUD form to it from Next.js.

Week 7: AI APIs, Build AI-Powered Hackathon Projects

Topics:

- OpenAI API: chat completions, function calling
- Google Gemini API: multimodal inputs (text + image)
- Anthropic Claude API: long-context, tool use, prompt caching
- Streaming responses to your UI
- RAG (Retrieval Augmented Generation) basics
- Embeddings + vector search with Supabase pgvector
- Image generation APIs (DALL·E, Imagen, Stable Diffusion)

- Speech-to-text and text-to-speech
- Cost-aware design, token counting and caching
- AI ethics: hallucinations, bias, attribution

Projects built:

- AI study buddy that quizzes you on uploaded notes
- AI image caption generator (Gemini multimodal)
- Voice-to-text journal with summarization

Practice: Ship a one prompt AI feature, like a study tip generator, that calls the OpenAI, Gemini or Claude API and streams the response into your UI.

Week 8: Algorithms, DSA & System Design Lite + Mock Hackathon (Teams)

Topics:

- Big O notation for hackathon thinking
- Arrays, hash maps, sets, stacks, queues
- Trees, graphs, BFS/DFS, pathfinding apps
- Sorting (merge, quick) and searching (binary)
- Top 10 hackathon-style algorithm problems
- System Design Lite: when to use which database
- Caching, rate limiting, queues at hackathon scale
- Picking architecture in 5 minutes
- Team mock hackathon (2 days, 12 hours total)
- Live judging panel with feedback letter

Projects built:

- PHASE 2 CAPSTONE, Team Mock Hackathon: pick a problem, ship an MVP in 12 hours, deploy to Vercel, deliver pitch

Assessment: Mock Hackathon Judges Panel scoring + written feedback

PHASE 3: REAL HACKATHON SPRINT, CAPSTONE, PITCH LAB & LIVE HACKATHON (WEEKS 9-12)

The final 4 weeks are a real hackathon journey. Teens lock a capstone idea, get matched with a senior mentor, build, polish, get pitch coached and compete in the Live Modern Age Coders Hackathon Weekend.

Week 9: Capstone Lock, Mentor Matching, Spec & Architecture

Topics:

- Idea finalists, picking the right capstone (3 ideas 1)
- Writing a 1-page spec: problem, users, solution, MVP scope, stretch
- Mentor matching with senior engineers
- Architecture diagram (whiteboard or Excalidraw)
- Tech stack lock: React/Next.js + Tailwind + Supabase + AI API
- Risk register: top 3 risks and mitigations
- Project board on GitHub Projects or Linear
- Daily commit cadence rules
- Stretch goals vs. MVP scope (MVP wins, always)
- Repo setup, CI, and Vercel preview deployments

Projects built:

- Capstone Spec (1 page)
- Architecture Diagram
- Project Board with weekly milestones

Practice: Write your one page capstone spec covering problem, users, MVP scope and stretch goals, and sketch the architecture in Excalidraw.

Week 10: Heavy Build Sprint, Code, Test, Iterate

Topics:

- Daily commits with kind, clear messages
- Pair programming with teammates
- AI-assisted coding via Cursor + Replit Agent
- Mid-sprint mentor reviews
- Testing critical paths (smoke tests minimum)
- Loading states, empty states, error states (judges notice these)
- Mobile responsiveness, judges open on phones
- Performance basics: bundle size, image optimization, caching
- Accessibility minimums (alt text, focus rings, color contrast)
- Daily standup logs to mentor

Projects built:

- Capstone build, 70% complete by end of Week 10
- Daily standup logs
- Mid-sprint mentor demo

Practice: Ship one committed feature each day for five days, with clear commit messages and at least one smoke test on the critical path.

Week 11: Pitch Lab, Storytelling, Slides & Demo Polish

Topics:

- Hackathon judging rubric in detail (Innovation 25 / Execution 25 / Design 25 / Pitch 25)
- The 2-minute pitch formula: Hook Problem Solution Demo Tech Team Ask
- Slide design, 7 slides max, big text, no walls of words
- Practicing on camera with daily review
- Recording a high-quality demo video

- Voice and pacing tips from professional pitch coaches
- Q&A handling, what to say when stumped
- Final README polish, judges read this first
- Final code freeze rules
- Deployment health check: SSL, mobile, dark mode, edge cases

Projects built:

- Pitch Slides (7 slides, Pitch.com or Canva)
- Pitch Video (2 minutes)
- Final README with hero image, demo link, tech stack, team

Practice: Cut your pitch to exactly two minutes, build a seven slide deck and record a demo video you would be proud to submit.

Week 12: LIVE HACKATHON WEEKEND, Compete with the World

Topics:

- Friday: opening ceremony, theme reveal, team check-in
- Saturday: 12-hour final sprint with senior mentor support
- Saturday evening: pitch dress rehearsal
- Sunday morning: 3-hour final polish + submission
- Sunday afternoon: live judging, every team pitches
- Categories: Most Innovative, Best Built, Best AI Use, Best Pitch, Best Design, Audience Favorite
- Live judge Q&A, handled with calm and clarity
- Awards ceremony, certificates, badges, Hall of Fame
- Reflection circle and feedback letters from judges
- Graduation: Teen Hackathon Builder Certification awarded

Projects built:

- GRAND CAPSTONE, Live Hackathon Submission: working product deployed to Vercel + GitHub repo + 2-minute demo + 7-slide pitch + judge-ready README + live presentation

Assessment: Final Hackathon Score + Judge Feedback Letter

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 6+ Python/JS projects + Solo Mini Hackathon (deployed)
- Phase 2 (Weeks 5-8): 6+ React/Supabase/AI projects + Team Mock Hackathon
- Phase 3 (Weeks 9-12): Capstone + Pitch Deck + Demo + LIVE Hackathon submission

Total projects: 12+ deployed projects + Live Hackathon Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Python and JavaScript fluency
- React + Next.js + Tailwind + shadcn/ui
- Supabase backend + auth + storage + realtime
- AI APIs: OpenAI, Gemini, Claude, chat, streaming, RAG
- Algorithms & DSA at hackathon depth
- Big O analysis
- GitHub workflow with branches and PRs
- System Design Lite for hackathons
- Vercel deployment and CI
- Pitch decks and demo videos
- README and project documentation
- Public speaking and Q&A handling

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (1.5-2 hours each)

Independent projects: 3-4 hours

Weekly hack: 1 themed hackathon-style challenge

Total per week: 6-8 hours

SUPPORT INCLUDED

Live instruction: Senior engineers and ex-startup CTOs

Mentor matching: 1-on-1 capstone mentor in Phase 3

Pitch lab: Dedicated pitch coaches in Week 11

Community: Teen Hackathon Builders Hall of Fame + alumni network

Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Final certificate: Teen Hackathon Builder, Modern Age Coders Certified

Capstone required: Live hackathon submission (deployed app + pitch + demo)

Portfolio: 12+ deployed projects + grand capstone

WHERE THIS CAN LEAD

- School & Inter-school Hackathon Champion
- Smart India Hackathon Finalist
- MLH Global Hackathon Participant

- Open-Source Contributor
- Strong College Application Profile (IIT/IIIT/BITS/Stanford/MIT)
- Internship-Ready (startups, mid-stage companies)
- Ready for Vibe Coding for College / DSA Masterclass

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is the hackathon really free for teens?

A: Yes, 100% free. Any teen aged 13-17 can register and compete in the Modern Age Coders Live Hackathon at zero cost. No entry fee. No school endorsement required. Free certificate. Free judging.

Q: Why is the prep course paid then?

A: The prep course is optional but highly recommended. It gives teens 12 weeks of structured training in coding, AI, algorithms, system design, and pitching. The hackathon itself stays free regardless.

Q: Does my teen need prior coding experience?

A: No. The prep course starts from absolute basics in Python and JavaScript. Teens with some experience will move faster, but beginners catch up by Week 4. We have placement check-ins to ensure no one is left behind.

Q: What is the time commitment per week?

A: 6-8 hours weekly: 2 live classes (1.5-2 hours each), 3-4 hours of independent project work, plus 1 hour for the weekly hackathon-style challenge. All sessions are recorded.

Q: What does my teen build by the end?

A: A GitHub portfolio of 6+ deployed projects, including a live capstone hosted on Vercel, a 2-minute pitch video, and submission to the live hackathon. Strong enough to impress college admissions and internship recruiters.

Q: Will this help with college applications?

A: Significantly. A real GitHub portfolio with deployed projects, hackathon wins, and demo videos is one of the strongest signals for tech-leaning college applications (US, Singapore, India IITs/IIITs/BITS). We also write recommendation letters for top performers.

Q: What stack do you teach?

A: The same stack used at YC startups in 2026: Python, JavaScript/TypeScript, React, Next.js (App Router), Tailwind CSS, shadcn/ui, Supabase, Vercel, GitHub, plus AI APIs from OpenAI, Google Gemini, and Anthropic Claude. We also use Cursor, Replit Agent, and Bolt.new as AI-assisted dev tools.

Q: Can my teen compete in MLH or Smart India Hackathon after this?

A: Yes, and many do. Our curriculum directly maps to MLH (Major League Hacking), Smart India Hackathon, NASA Space Apps, and most college-level hackathons. The skills are universal.

Q: Do you cover algorithms and DSA?

A: Yes, at hackathon depth. Big O, arrays, hash maps, trees, graphs, BFS/DFS, sorting, searching. We don't go to LeetCode Hard depth; that's our Vibe Coding for College program. But we go deep enough to write efficient hackathon code that doesn't crash with real data.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/hackathon-prep-for-teens-coding-ai-build-innovate-win-course>

COURSE 69 OF 118: HIGH SCHOOL MATHEMATICS: ALGEBRA TO CALCULUS (GRADE 9-12)

Course link: <https://learn.modernagecoders.com/courses/complete-high-school-mathematics-mastery>

Subject area: Mathematics Education

Designed for: Girls only batches, High school

Who can join: Complete Beginner to Advanced High School

Course length: 12 months (52 weeks)

Time needed: 10-15 hours/week recommended

Certificate: High School Mathematics Completion Certificate

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online high school maths course for Grade 9-12: algebra, geometry, trigonometry, pre-calculus, calculus and statistics, built for university readiness.

In one line: From Basic Arithmetic to Advanced Calculus in 12 Months

ABOUT THE PROGRAMME

This accelerated program condenses 4 years of high school mathematics into an intensive 12-month journey. Whether you're a student seeking to excel, an adult learner filling gaps, or preparing for competitive exams, this comprehensive program covers every essential mathematical concept from grades 9-12. You'll master algebraic thinking, geometric reasoning, trigonometric applications, and calculus fundamentals. Each concept is taught through interactive lessons, real-world applications, and extensive problem-solving practice. By completion, you'll be fully prepared for university-level mathematics and standardized tests.

WHAT MAKES THIS COURSE DIFFERENT

- Accelerated 4-year curriculum in 12 months
- Visual and interactive learning approach
- Real-world applications for every concept
- Adaptive practice problems with instant feedback
- Preparation for SAT, ACT, and competitive exams
- Step-by-step solution explanations
- Weekly assessments and progress tracking
- Personalized learning paths based on strengths/weaknesses

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Algebra I, Linear Equations, Polynomials, Basic Geometry
Phase 2: Intermediate (Months 4-6): Algebra II, Quadratics, Functions, Coordinate Geometry
Phase 3: Advanced (Months 7-9): Trigonometry, Pre-Calculus, Sequences, Complex Numbers
Phase 4: Mastery (Months 10-12): Calculus, Statistics, Probability, Advanced Problem Solving

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic arithmetic and elementary math
Coding experience: None required - math focus
Equipment: Computer/tablet, internet, calculator
Time commitment: 10-15 hours per week
English: Basic comprehension
Motivation: Desire to master mathematics

WHO THIS COURSE IS FOR

Students: High school students (grades 9-12)
Working professionals: Adults needing math skills
Entrepreneurs: Business owners needing quantitative skills
Freelancers: Tutors and educators
Kids: Advanced middle schoolers ready for challenge
Anyone: Anyone wanting strong mathematical foundation

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: ALGEBRAIC FOUNDATIONS & BASIC GEOMETRY (MONTHS 1-3, WEEKS 1-13) -----

Build strong foundations in algebraic thinking, equation solving, and geometric reasoning.
Master the core concepts that form the basis of all advanced mathematics.

Months 1-2: Algebra I Fundamentals (Week 1-8)

Number Systems & Algebraic Expressions

Topics:

- Real number system: natural, whole, integers, rational, irrational
- Number line and absolute value
- Properties of real numbers: commutative, associative, distributive
- Order of operations (PEMDAS/BODMAS)
- Exponents and exponent laws
- Scientific notation and significant figures
- Square roots and cube roots
- Simplifying radical expressions
- Variables and algebraic expressions
- Evaluating expressions for given values
- Combining like terms
- Translating word problems to algebraic expressions

Projects built:

- Create a number system classification chart
- Real-world scientific notation problems
- Algebraic expression calculator design

Practice: Daily: 50 problems on simplification and evaluation

Linear Equations & Inequalities

Topics:

- Solving one-step equations
- Solving two-step equations
- Multi-step equations with distribution
- Equations with variables on both sides
- Equations with fractions and decimals
- Literal equations and formulas
- Linear inequalities in one variable
- Compound inequalities (AND/OR)
- Absolute value equations
- Absolute value inequalities
- Graphing inequalities on number line
- Word problems leading to linear equations

Projects built:

- Budget planning with linear equations
- Distance-rate-time problem solver
- Inequality graphing tool

Practice: Solve 100 equations and 50 inequalities with varying complexity

Polynomials & Operations

Topics:

- Polynomial terminology: degree, coefficient, constant
- Standard form of polynomials
- Adding and subtracting polynomials
- Multiplying monomials
- Multiplying polynomials (FOIL method)
- Multiplying binomials and trinomials
- Special products: $(a+b)^2$, $(a-b)^2$, $(a+b)(a-b)$
- Dividing polynomials by monomials
- Long division of polynomials
- Synthetic division introduction
- Pascal's triangle and binomial expansion
- Applications of polynomial operations

Projects built:

- Polynomial calculator implementation
- Area and volume problems with polynomials
- Pascal's triangle pattern explorer

Practice: Complete 150 polynomial operation problems

Factoring Techniques

Topics:

- Greatest Common Factor (GCF)
- Factoring by grouping
- Factoring trinomials ($a=1$)
- Factoring trinomials ($a \neq 1$)
- Difference of squares
- Sum and difference of cubes
- Perfect square trinomials
- Complete factoring strategies
- Solving equations by factoring
- Zero product property
- Factor theorem introduction
- Applications in area and optimization

Projects built:

- Factoring flowchart creation
- Projectile motion problems
- Optimization problem solver

Practice: Factor 200 expressions of varying types

Month 3: Systems & Basic Geometry (Week 9-13)

Systems of Linear Equations

Topics:

- Graphing linear equations in two variables
- Slope and y-intercept
- Point-slope form and slope-intercept form
- Standard form and intercepts
- Parallel and perpendicular lines
- Systems of linear equations: graphical method
- Substitution method
- Elimination method
- Systems with no solution or infinite solutions
- Three-variable systems introduction
- Matrix representation of systems
- Word problems with systems

Projects built:

- Supply and demand economic model
- Mixture problem solver
- Linear programming introduction

Practice: Solve 100 systems using different methods

Introduction to Geometry

Topics:

- Points, lines, and planes
- Angles: acute, right, obtuse, straight, reflex
- Angle relationships: complementary, supplementary, vertical
- Parallel lines and transversals
- Triangles: classification by sides and angles
- Triangle sum theorem
- Exterior angle theorem
- Congruent triangles: SSS, SAS, ASA, AAS, HL
- Similar triangles and proportions
- Pythagorean theorem and its converse
- Special right triangles (45-45-90, 30-60-90)
- Triangle inequalities

Projects built:

- Triangle solver application
- Architectural design with triangles
- Distance calculator using Pythagorean theorem

Practice: Complete 150 geometry problems

Phase 1 Review & Assessment

Topics:

- Comprehensive review of algebraic concepts
- Review of equation solving techniques
- Polynomial and factoring review
- Systems of equations review
- Geometry fundamentals review
- Problem-solving strategies
- Test-taking techniques

Projects built:

- Create comprehensive formula sheet
- Design practice test for peers
- Real-world application portfolio

Assessment: Phase 1 comprehensive exam covering all topics

----- PHASE 2: ADVANCED ALGEBRA & FUNCTIONS (MONTHS 4-6, WEEKS 14-26) -----

Master quadratic functions, advanced algebraic concepts, coordinate geometry, and function theory. Build strong analytical and graphing skills.

Months 4-5: Quadratics & Functions (Week 14-21)

Quadratic Functions & Equations

Topics:

- Quadratic functions: $f(x) = ax^2 + bx + c$
- Graphing parabolas: vertex, axis of symmetry
- Maximum and minimum values
- Vertex form: $f(x) = a(x-h)^2 + k$
- Converting between standard and vertex form
- Completing the square method
- Quadratic formula derivation and application
- Discriminant and nature of roots
- Complex solutions introduction
- Sum and product of roots
- Quadratic inequalities
- Applications: projectile motion, optimization

Projects built:

- Projectile motion simulator
- Profit maximization calculator
- Parabola art generator

Practice: Solve 200 quadratic problems

Exponential & Logarithmic Functions

Topics:

- Exponential functions: $f(x) = a^x$
- Properties of exponential functions
- Exponential growth and decay
- The number e and natural exponential
- Compound interest formula
- Introduction to logarithms
- Common and natural logarithms
- Logarithm properties and laws
- Change of base formula
- Solving exponential equations
- Solving logarithmic equations
- Applications: population growth, radioactive decay

Projects built:

- Compound interest calculator
- Population growth model
- pH scale explorer

Practice: Master 150 exponential and logarithmic problems

Rational Functions & Expressions

Topics:

- Rational expressions: simplification
- Multiplying and dividing rational expressions
- Adding and subtracting rational expressions
- Complex fractions
- Rational equations
- Extraneous solutions
- Rational functions and graphs
- Vertical and horizontal asymptotes
- Holes in rational functions
- Inverse variation
- Joint and combined variation
- Applications in physics and engineering

Projects built:

- Lens equation calculator
- Rate problem solver
- Asymptote visualizer

Practice: Complete 150 rational expression problems

Function Theory & Transformations

Topics:

- Function definition and notation
- Domain and range
- Function operations: addition, subtraction, multiplication, division
- Composite functions
- Inverse functions
- One-to-one functions
- Function transformations: translations
- Reflections and stretches
- Combining transformations
- Even and odd functions
- Piecewise functions
- Greatest integer function

Projects built:

- Function transformation visualizer
- Composite function calculator
- Inverse function finder

Practice: Work through 200 function problems

Conic Sections

Topics:

- Circle equation: center-radius form
- General form of circle equation
- Ellipse: standard form and properties
- Hyperbola: standard form and properties
- Parabola review in conic context
- Identifying conic sections
- Translations of conics
- Systems involving conics
- Applications in astronomy and physics

Projects built:

- Conic section grapher
- Orbital mechanics simulator
- Architectural design with conics

Practice: Graph and analyze 100 conic sections

Month 6: Coordinate Geometry & Vectors (Week 22-26)

Advanced Coordinate Geometry

Topics:

- Distance formula in 2D and 3D
- Midpoint formula
- Section formula
- Area of triangle using coordinates
- Collinearity of points
- Equation of line: various forms
- Angle between two lines
- Distance from point to line
- Concurrent lines
- Locus problems
- Transformation geometry: rotation, reflection
- Coordinate proofs

Projects built:

- Geometry solver application
- GPS distance calculator
- Computer graphics transformer

Practice: Solve 150 coordinate geometry problems

Introduction to Vectors

Topics:

- Vector vs scalar quantities
- Vector notation and representation
- Vector addition and subtraction
- Scalar multiplication

- Magnitude of a vector
- Unit vectors
- Component form of vectors
- Dot product
- Angle between vectors
- Vector projections
- Applications in physics
- Cross product introduction (3D)

Projects built:

- Force vector calculator
- Navigation system simulator
- Vector graphics editor

Practice: Complete 100 vector problems

Sequences and Series

Topics:

- Arithmetic sequences
- Arithmetic series and sum formula
- Geometric sequences
- Geometric series and sum formula
- Infinite geometric series
- Convergence and divergence
- Sigma notation
- Recursive sequences
- Fibonacci sequence
- Mathematical induction
- Binomial theorem
- Applications in finance and nature

Projects built:

- Loan amortization calculator
- Fibonacci pattern generator
- Series convergence tester

Practice: Work through 150 sequence and series problems

Complex Numbers

Topics:

- Imaginary unit i
- Complex number form: $a + bi$
- Operations with complex numbers
- Complex conjugates
- Absolute value of complex numbers
- Complex plane (Argand diagram)
- Polar form of complex numbers
- De Moivre's theorem
- Roots of complex numbers
- Euler's formula introduction
- Applications in engineering
- Solving polynomial equations

Projects built:

- Complex number calculator
- Mandelbrot set visualizer
- AC circuit analyzer

Practice: Master 100 complex number problems

Phase 2 Capstone Assessment

Topics:

- Comprehensive function review
- Advanced algebra synthesis
- Coordinate geometry mastery check
- Vector applications
- Complex problem solving
- Mathematical modeling

Projects built:

- Design a complete function analysis tool
- Create mathematical model for real phenomenon
- Build comprehensive problem set with solutions

Assessment: Phase 2 comprehensive exam

PHASE 3: TRIGONOMETRY & PRE-CALCULUS (MONTHS 7-9, WEEKS 27-39)

Master trigonometric functions, identities, and applications. Prepare for calculus with limits, advanced functions, and analytical geometry.

Months 7-8: Trigonometry Mastery (Week 27-34)

Trigonometric Ratios & Functions

Topics:

- Angle measurement: degrees and radians
- Arc length and sector area
- Right triangle trigonometry
- Six trigonometric ratios: $\sin$, $\cos$, $\tan$, $\csc$, $\sec$, $\cot$
- Trigonometric ratios of special angles

- Reference angles and coterminal angles
- Unit circle and circular functions
- Trigonometric functions of any angle
- Signs of trigonometric functions in quadrants
- Exact values using unit circle
- Inverse trigonometric functions
- Principal values and general solutions

Projects built:

- Unit circle interactive tool
- Height and distance calculator
- Trigonometric table generator

Practice: Solve 200 basic trigonometry problems

Graphing Trigonometric Functions

Topics:

- Graphing sine and cosine functions
- Amplitude, period, and frequency
- Phase shift and vertical shift
- Graphing tangent and cotangent
- Graphing secant and cosecant
- Transformations of trigonometric graphs
- Combining trigonometric functions
- Damped oscillations
- Trigonometric modeling
- Applications: sound waves, tides
- Harmonic motion
- Fourier series introduction

Projects built:

- Wave generator and analyzer
- Music synthesizer using trig functions
- Tidal prediction model

Practice: Graph 150 trigonometric functions

Trigonometric Identities

Topics:

- Fundamental identities: Pythagorean, reciprocal, quotient
- Cofunction identities
- Even-odd identities
- Sum and difference formulas
- Double angle formulas
- Half angle formulas
- Product-to-sum formulas
- Sum-to-product formulas
- Proving trigonometric identities
- Simplifying trigonometric expressions
- Conditional identities
- Applications in physics and engineering

Projects built:

- Identity prover application
- Trigonometric simplifier
- Physics formula derivation tool

Practice: Prove 100 trigonometric identities

Trigonometric Equations & Applications

Topics:

- Solving basic trigonometric equations
- Equations involving multiple angles
- Equations with trigonometric identities
- Parametric equations
- Eliminating parameters
- Polar coordinates
- Converting between polar and rectangular
- Graphing in polar coordinates
- Rose curves and limacons
- Complex numbers in polar form
- Applications in navigation
- Spherical trigonometry basics

Projects built:

- Parametric curve plotter
- Polar graphing calculator
- Navigation system using trigonometry

Practice: Solve 150 trigonometric equations

Law of Sines and Cosines

Topics:

- Law of sines derivation
- Solving triangles with law of sines
- Ambiguous case (SSA)
- Law of cosines derivation
- Solving triangles with law of cosines
- Area of triangle formulas
- Heron's formula
- Applications in surveying

- Navigation problems
- Vector applications
- 3D trigonometry introduction

Projects built:

- Triangle solver (all cases)
- Surveying application
- GPS triangulation simulator

Practice: Solve 100 triangle problems

Month 9: Pre-Calculus & Limits (Week 35-39)

Limits and Continuity

Topics:

- Introduction to limits
- Limit notation and terminology
- Evaluating limits graphically
- Evaluating limits numerically
- Evaluating limits algebraically
- One-sided limits
- Limits at infinity
- Infinite limits and vertical asymptotes
- Limit laws and properties
- Squeeze theorem
- Continuity at a point
- Types of discontinuities

Projects built:

- Limit calculator and visualizer
- Continuity checker
- Asymptote analyzer

Practice: Evaluate 200 limits

Introduction to Derivatives

Topics:

- Rate of change and slope
- Average rate of change
- Instantaneous rate of change
- Definition of derivative
- Derivative as a limit
- Differentiability and continuity
- Basic differentiation rules
- Power rule
- Product rule introduction
- Quotient rule introduction
- Chain rule introduction
- Applications: velocity and acceleration

Projects built:

- Derivative calculator
- Motion analyzer
- Tangent line grapher

Practice: Find 150 derivatives

Mathematical Reasoning & Proofs

Topics:

- Logic and statements
- Truth tables
- Conditional statements
- Biconditional statements
- Logical equivalence
- Methods of proof: direct, indirect
- Proof by contradiction
- Proof by induction (detailed)
- Counter examples
- Set theory basics
- Relations and functions
- Mathematical writing

Projects built:

- Proof validator
- Logic puzzle solver
- Set operation calculator

Practice: Write 50 mathematical proofs

Matrices and Determinants

Topics:

- Matrix notation and terminology
- Types of matrices
- Matrix operations: addition, subtraction
- Scalar multiplication
- Matrix multiplication
- Properties of matrix operations
- Identity and inverse matrices
- Determinants (2x2 and 3x3)
- Properties of determinants
- Cramer's rule

- Matrix equations
- Applications in transformations

Projects built:

- Matrix calculator
- Linear system solver using matrices
- Transformation visualizer

Practice: Complete 100 matrix problems

Phase 3 Review & Assessment

Topics:

- Trigonometry comprehensive review
- Pre-calculus concepts review
- Limits and continuity review
- Mathematical reasoning review
- Problem-solving strategies
- Exam preparation techniques

Projects built:

- Create trigonometry reference guide
- Design pre-calculus problem bank
- Build interactive review tool

Assessment: Phase 3 comprehensive exam

----- PHASE 4: CALCULUS & ADVANCED TOPICS (MONTHS 10-12, WEEKS 40-52) -----

Master differential and integral calculus, statistics, probability, and advanced problem-solving techniques for university readiness.

Months 10-11: Calculus (Week 40-47)

Differential Calculus

Topics:

- Review of derivative concept
- Differentiation rules (complete)
- Derivatives of trigonometric functions
- Derivatives of exponential functions
- Derivatives of logarithmic functions
- Implicit differentiation
- Logarithmic differentiation
- Higher-order derivatives
- Related rates problems
- Linear approximation
- Differentials
- Error estimation

Projects built:

- Complete derivative calculator
- Related rates problem solver
- Error analysis tool

Practice: Solve 300 differentiation problems

Applications of Derivatives

Topics:

- Critical points and extrema
- First derivative test
- Second derivative test
- Concavity and inflection points
- Curve sketching (complete process)
- Optimization problems
- Maximum and minimum word problems
- Mean Value Theorem
- Rolle's Theorem
- L'Hôpital's Rule
- Newton's method
- Applications in economics and physics

Projects built:

- Function analyzer (complete)
- Optimization problem solver
- Economic model calculator

Practice: Complete 200 application problems

Integral Calculus

Topics:

- Antiderivatives and indefinite integrals
- Basic integration rules
- Integration by substitution
- Definite integrals
- Fundamental Theorem of Calculus
- Properties of definite integrals
- Integration by parts
- Trigonometric integrals
- Trigonometric substitution
- Partial fractions
- Improper integrals

- Numerical integration: trapezoid, Simpson's rule

Projects built:

- Integration calculator
- Area calculator using integrals
- Numerical integration tool

Practice: Evaluate 300 integrals

Applications of Integration

Topics:

- Area between curves
- Volume by disks and washers
- Volume by cylindrical shells
- Arc length
- Surface area of revolution
- Work and energy problems
- Center of mass
- Probability density functions
- Average value of a function
- Applications in physics
- Applications in economics
- Differential equations introduction

Projects built:

- Volume calculator for solids
- Physics application solver
- Probability distribution analyzer

Practice: Solve 150 application problems

Series and Sequences (Calculus)

Topics:

- Infinite sequences
- Convergence of sequences
- Infinite series
- Geometric series (calculus perspective)
- Test for divergence
- Integral test
- Comparison tests
- Ratio and root tests
- Alternating series
- Power series
- Taylor and Maclaurin series
- Applications of series

Projects built:

- Series convergence tester
- Taylor series calculator
- Function approximator

Practice: Analyze 100 series

Month 12: Statistics & Probability (Week 48-52)

Descriptive Statistics

Topics:

- Types of data: categorical, numerical
- Measures of central tendency: mean, median, mode
- Measures of spread: range, variance, standard deviation
- Quartiles and box plots
- Percentiles and percentile rank
- Frequency distributions and histograms
- Stem-and-leaf plots
- Scatter plots and correlation
- Linear regression
- Coefficient of determination (R^2)
- Outliers and their effects
- Data transformation

Projects built:

- Statistical calculator suite
- Data visualization dashboard
- Regression analysis tool

Practice: Analyze 50 data sets

Probability Theory

Topics:

- Sample spaces and events
- Classical probability
- Empirical probability
- Probability axioms
- Addition rule of probability
- Multiplication rule
- Conditional probability
- Independent and dependent events
- Bayes' theorem
- Permutations and combinations
- Binomial probability
- Expected value and variance

Projects built:

- Probability calculator
- Monte Carlo simulator
- Game theory analyzer

Practice: Solve 150 probability problems

Probability Distributions

Topics:

- Random variables: discrete and continuous
- Probability mass functions
- Probability density functions
- Cumulative distribution functions
- Binomial distribution
- Poisson distribution
- Geometric distribution
- Normal distribution
- Standard normal distribution (Z-scores)
- Central Limit Theorem
- Sampling distributions
- Chi-square distribution introduction

Projects built:

- Distribution visualizer
- Normal probability calculator
- Sampling distribution simulator

Practice: Work with 100 distribution problems

Statistical Inference

Topics:

- Point estimation
- Confidence intervals for means
- Confidence intervals for proportions
- Hypothesis testing framework
- Type I and Type II errors
- Z-tests for means
- T-tests: one-sample, two-sample, paired
- Tests for proportions
- Chi-square test for independence
- ANOVA basics
- P-values and significance levels
- Power of a test

Projects built:

- Hypothesis testing calculator
- Confidence interval generator
- Statistical test selector

Practice: Conduct 75 hypothesis tests

Advanced Problem Solving

Topics:

- Competition mathematics strategies
- Number theory problems
- Combinatorics advanced
- Graph theory basics
- Game theory introduction
- Optimization techniques
- Mathematical modeling
- Cross-topic problem solving
- SAT/ACT preparation
- University entrance exam prep
- Mathematical competitions prep

Projects built:

- Competition problem solver
- Mathematical game creator
- Exam preparation guide

Practice: Solve 100 competition-level problems

Final Review & Applications (Week 51-52)

Real-World Applications

Topics:

- Mathematics in physics
- Mathematics in engineering
- Mathematics in economics
- Mathematics in computer science
- Mathematics in biology
- Mathematics in finance
- Data science applications
- Machine learning mathematics
- Cryptography basics
- Operations research

Projects built:

- Interdisciplinary problem solver
- Real-world modeling project
- Career-specific math toolkit

Practice: Apply math to 50 real-world scenarios

Technology in Mathematics

Topics:

- Graphing calculators mastery
- Computer algebra systems
- Mathematical software: MATLAB, Mathematica
- Python for mathematics
- Data analysis with spreadsheets
- GeoGebra for visualization
- Desmos graphing
- WolframAlpha usage
- Mathematical typesetting (LaTeX)
- Online resources and tools

Projects built:

- Create math visualization tool
- Build calculator app
- Develop math learning game

Practice: Master 10 mathematical software tools

Mathematical Research Project

Topics:

- Choosing a research topic
- Literature review basics
- Mathematical writing
- Creating mathematical arguments
- Presenting mathematics
- Peer review process
- Mathematical communication
- Visual representations
- Mathematical poster creation
- Oral presentation skills

Projects built:

- Complete mathematical research paper
- Create presentation on chosen topic
- Design mathematical exhibition

Practice: Present findings to peers

University Preparation

Topics:

- University mathematics preview
- Study skills for higher math
- Mathematical maturity
- Abstract thinking development
- Proof-writing skills
- Advanced calculus preview
- Linear algebra introduction
- Discrete mathematics preview
- Mathematical career paths
- Further learning resources

Projects built:

- University readiness portfolio
- Advanced topic exploration
- Learning plan for next level

Practice: Complete university-level problem sets

Final Assessments & Certification (Week 52)

Comprehensive Review - Part 1

Topics:

- Algebra complete review
- Geometry complete review
- Trigonometry complete review
- Functions and graphs review
- Calculus essentials review
- Statistics and probability review
- Problem-solving strategies review
- Formula compilation

Projects built:

- Create comprehensive study guide
- Build formula reference app
- Design review problem sets

Comprehensive Review - Part 2 & Mock Exams

Topics:

- Cross-topic problem solving
- Time management strategies
- Error analysis and correction
- Mock exam practice
- SAT/ACT math practice
- AP Calculus practice
- Competition math practice
- Peer tutoring exercises

Deliverables:

- Complete practice exam portfolio
- Error analysis report
- Personal progress documentation
- Achievement portfolio

Final Certification & Future Planning

Topics:

- Final comprehensive examination
- Portfolio presentation
- Skills assessment
- Strengths and areas for growth
- Future learning pathways
- University course selection guidance
- Career planning in STEM
- Lifelong learning in mathematics
- Mathematical community engagement
- Celebration of achievements

Deliverables:

- Final certification exam
- Complete portfolio submission
- Self-assessment report
- Future learning plan
- Peer recommendation letters
- Course completion certificate

Assessment: FINAL COMPREHENSIVE EXAM - 4-hour examination covering all topics from grades 9-12

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Algebraic calculator, Equation solver, Geometry tool, Pattern generator
- Phase 2: Function analyzer, Graphing calculator, Vector simulator, Complex number visualizer
- Phase 3: Trigonometry suite, Proof assistant, Matrix calculator, Limit explorer
- Phase 4: Calculus toolkit, Statistics dashboard, Probability simulator, Research project
- Final: Comprehensive mathematics portfolio with 50+ solved problems

Total projects: 100+ mathematical tools, visualizers, and problem-solving applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Algebra: Linear, quadratic, polynomial, rational, exponential, logarithmic
- Geometry: Euclidean, coordinate, transformational, vectors
- Trigonometry: Functions, identities, equations, applications
- Pre-Calculus: Limits, sequences, series, matrices
- Calculus: Differential, integral, applications
- Statistics: Descriptive, inferential, regression
- Probability: Theory, distributions, inference
- Problem Solving: Analytical thinking, proof writing, modeling
- Technology: Calculators, software, programming for math

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 3-4 hours
 Projects: 2-3 hours
 Practice problems: 4-5 hours
 Total per week: 10-15 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving sessions
 Mentorship: 1-on-1 tutoring available
 Community: Active student forum
 Code review: Solution review and feedback
 Career support: University application guidance
 Lifetime access: All materials and updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each quarter
 Final certificate: High School Mathematics Completion Certificate
 LinkedIn badge: Verifiable achievement badge
 Industry recognized: Accepted by universities
 Portfolio projects: 50+ solved complex problems

WHERE THIS CAN LEAD

- Engineering (all disciplines)
- Data Science and Analytics
- Actuarial Science
- Financial Analysis

- Research Scientist
- Mathematics Teacher/Professor
- Quantitative Trading
- Operations Research
- Software Development (algorithms)
- Economics and Econometrics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What topics does the High School Mathematics Mastery Program cover?

A: In 12 months it moves through four phases: Algebra I foundations and basic geometry; Algebra II with quadratics, functions and coordinate geometry; trigonometry and pre-calculus including sequences and complex numbers; and finally introductory calculus, statistics and probability. It is the full grade 9 to 12 arc taught as one connected story.

Q: Does this course follow CBSE, ICSE or another school board?

A: The course is deliberately board agnostic: it teaches the mathematics underneath every board, so it works alongside CBSE, ICSE, IGCSE, IB or state syllabi rather than replacing them. Your teacher aligns examples and pacing with what school is covering whenever that helps.

Q: My child is in grade 9, or already in grade 11. Is the level right?

A: Yes. The program starts from basic arithmetic and algebra, so a grade 9 student begins comfortably, while an older student uses it to close gaps quickly and then accelerate into pre-calculus and calculus. A diagnostic in the first week sets the entry point, so nobody sits through material they already own.

Q: What are the prerequisites?

A: Only basic arithmetic and elementary school mathematics. No coding is involved, and the equipment list is simple: a computer or tablet, internet and a calculator. Everything else, including the thinking habits, is built inside the course.

Q: How do we know it is working? How is progress measured?

A: Every phase ends with an assessment that must be secure before moving on, and weekly problem work is reviewed for reasoning, not just answers. Parents get a clear picture from the running record of mastered topics, and school test performance is the natural external check along the way.

Q: Is there a free trial?

A: Yes. Book a free demo class at learn.modernagecoders.com/contact with no commitment; it is a real lesson plus an honest read on where your child stands. Questions first? Message us on WhatsApp at +91 91233 66161.

Q: Are these live classes or videos?

A: Live, always. Each session is a full one hour class with a real teacher in a small group, never pre-recorded content, so the teacher can catch the exact step where your child's method breaks and fix it on the spot.

Q: Is there a certificate at the end of the program?

A: Students receive the High School Mathematics Completion Certificate along with a documented record of the topics, problem sets and projects they worked through. We are rated 4.9 stars across 547+ Google reviews, largely by parents of students in programs like this one.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-high-school-mathematics-mastery>

COURSE 70 OF 118: HTML & CSS FOR BEGINNERS: BUILD REAL RESPONSIVE WEBSITES

Course link: <https://learn.modernagecoders.com/courses/html-css-course-for-beginners-build-real-websites>

Subject area: Web Development

Designed for: Teens

Who can join: Absolute beginner, ages 10 to adult

Course length: 4 months (16 weeks)

Time needed: 2 live classes/week + 2-3 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

HTML and CSS course for beginners: live online classes from your first tag to a published

multi-page website, with flexbox, grid, and responsive design.

In one line: Every website you have ever visited is built on these two languages. Four months from a blank file to your own published site.

ABOUT THE PROGRAMME

HTML and CSS are where the web actually starts. Frameworks come and go, but every page on the internet is, underneath, structure written in HTML and appearance written in CSS, and people who skip this layer pay for it later. This 4-month live course takes complete beginners from an empty folder to a multi-page personal website they design, build, test on a phone, and publish to a real URL. The first two months build the two languages properly: semantic page structure, then selectors, the box model, and typography, so students understand why a page looks the way it does instead of copying snippets and hoping. The second half is where pages become layouts: flexbox and grid, responsive design that survives a phone screen, working forms, and a capstone month spent building and shipping the personal site. Classes are live and small, and every student leaves with a site that is genuinely theirs, structure, design decisions, and all.

WHAT MAKES THIS COURSE DIFFERENT

- Structure before decoration: semantic HTML is taught properly in month 1, the habit that separates clean sites from tag soup
- Layout is taught as a system, not a bag of tricks: flexbox and grid each get real time, including when to use which
- Students recreate real designs from screenshots, the same exercise working developers use to train their eye
- Every page built in class is tested on a phone, because most visitors arrive on one
- The capstone is published to a real URL on free hosting, not left on a laptop
- Works for ages 10 to adult: batches are grouped by age so examples and pace fit the room

THE LEARNING PATH, STEP BY STEP

Phase 1: The two languages: semantic HTML structure first, then CSS selectors, the box model, and typography, applied to real pages every week

Phase 2: Layout and shipping: flexbox, grid, responsive design, and forms, finishing with a multi-page personal site built, tested, and published

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. This is the true starting point of web development

Computer skills: Comfortable typing and managing files and folders

Equipment: A computer with a modern browser and a stable internet connection

Age: 10 to adult. Batches are grouped by age so pace and examples fit the room

WHO THIS COURSE IS FOR

Absolute beginners: Anyone starting web development from zero who wants to build the real foundation, not skip it

Kids and teens: Students aged 10+ who want to make actual websites they can show, not just follow tutorials

Adults switching tracks: Working adults testing whether web development suits them, with a real published site as the outcome

Small business owners: People who want to understand, fix, and eventually build their own web presence

Future javascript students: Learners who plan to take JavaScript next and want their HTML and CSS solid first

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: THE TWO LANGUAGES (MONTHS 1-2, WEEKS 1-8) -----

HTML gives a page its structure and meaning; CSS gives it appearance. Both are learned by building real pages from the first week.

Month 1: HTML, the Structure of Every Page (Weeks 1-4)

Your First Page

Topics:

- How a website actually works: files, browsers, and servers in plain language
- Setting up VS Code and a tidy project folder
- The skeleton of every page: doctype, head, and body
- Headings, paragraphs, and line breaks
- Ordered and unordered lists
- Saving, refreshing, and reading your page the way the browser does

Projects built:

- A real first page: an introduction page with headings, paragraphs, and a list, opened in the

browser by the end of class one

Practice: Build a second page about a hobby using every element from class, plus one you look up yourself

Links, Images, and Paths

Topics:

- Anchor tags: linking to other sites and to your own pages
- Images: src, alt text, and why alt text matters for real people
- Relative vs absolute paths, and the broken-image bug everyone hits
- Organizing a project: an images folder that stays tidy
- Attributes in general: the pattern behind href, src, and the rest
- Opening links in new tabs, used sparingly

Projects built:

- Two-page mini site: an about page and a favorites page, linked both ways, with images that actually load

Practice: Fix a provided page with six broken links and images, then explain what was wrong with each

Semantic Structure

Topics:

- header, nav, main, section, article, and footer: naming the parts of a page
- Why structure matters: screen readers, search engines, and future-you
- div and span, and why they are the last resort, not the first
- Nesting elements correctly and indenting so you can see the structure
- Block vs inline elements, previewed before CSS makes it matter
- Reading the structure of real websites with dev tools

Projects built:

- Restructure week 2's site with proper semantic tags, then sketch the structure of a favorite real website on paper

Practice: Mark up a printed newspaper-style article into clean semantic HTML

Tables, Media, and Clean Code

Topics:

- Tables for actual tabular data: rows, headers, and captions
- Embedding video and maps with iframes
- Character entities and special symbols
- Validating your HTML with the W3C validator and reading its complaints
- Commenting and formatting habits that keep pages readable
- A structure checklist to run before calling any page done

Projects built:

- A results or schedule page with a real data table plus an embedded video or map

Practice: Run all your pages through the validator and clear every error

Assessment: Month 1 check: build a clean semantic page live from a hand-drawn sketch, validator-clean

Month 2: CSS, the Appearance Layer (Weeks 5-8)

First Styles

Topics:

- What CSS is and the three ways to add it, and why external stylesheets win
- Selectors: element, class, and id, and when each is appropriate
- Colors: names, hex, and rgb
- The cascade in plain terms: what wins when rules collide
- Grouping selectors and keeping a stylesheet organized
- Dev tools as a CSS laboratory: change styles live, then copy them back

Projects built:

- Style the month 1 introduction page: a real color scheme, applied through classes, no inline styles

Practice: Given an unstyled page and a screenshot, write the selectors and colors to match it

Typography

Topics:

- font-family and font stacks: what happens when a font is missing
- Loading Google Fonts properly
- Size, weight, and line-height: the three dials that change everything
- Spacing text so it breathes: letter-spacing and text alignment
- Units: px, em, rem, and percent, explained with examples
- Pairing two fonts without the page looking like a ransom note

Projects built:

- Typography makeover: the same article styled two ways, one serious, one playful, with a note on the choices

Practice: Recreate the text styling of a well-designed real site as closely as you can

The Box Model

Topics:

- Every element is a box: content, padding, border, margin
- Reading the box model diagram in dev tools
- box-sizing: border-box and why we set it everywhere
- Backgrounds: colors, images, and gradients
- Borders, rounded corners, and shadows used with restraint
- Margin collapse, the classic surprise, demonstrated live

Projects built:

- Profile card: photo, name, description, and a button, spaced and shadowed like a real component

Practice: Ten box-model puzzles: match the target spacing exactly using padding and margin

Display and Simple Layout

Topics:

- display: block, inline, and inline-block in practice
- Width, height, and max-width: making content behave
- Centering things: the honest tour of the classic problem
- Building a content column that sits nicely on any screen
- Hover states: the first taste of interactivity in pure CSS
- Putting a full page together: structure from month 1, styles from month 2

Projects built:

- A styled article page: centered content column, styled headings, hover-styled links, and a card or two

Practice: Recreate a provided card design from a screenshot, pixel-close

Assessment: Month 2 check: recreate a small component design from a screenshot live, explaining each CSS choice

PHASE 2: LAYOUT, RESPONSIVENESS, AND SHIPPING (MONTHS 3-4, WEEKS 9-16)

Flexbox and grid turn styled pages into real layouts. Then responsive design, forms, and a capstone month: build, test, and publish a multi-page personal site.

Month 3: Real Layout with Flexbox and Grid (Weeks 9-12)

Flexbox Fundamentals

Topics:

- The flex container and its children: a mental model that sticks
- Main axis and cross axis, drawn before coded
- justify-content and align-items: the six arrangements you use daily
- gap: spacing items without margin hacks
- flex-direction: rows becoming columns
- The navigation bar: flexbox's most famous job

Projects built:

- A proper navigation bar: logo left, links right, evenly spaced, with hover states

Practice: Flexbox Froggy through the core levels, plus three layout snippets built from description alone

Flexbox in the Real World

Topics:

- flex-wrap: rows that break gracefully
- flex-grow and flex-shrink: who gets the spare space
- Building a card row that becomes a card grid
- Aligning odd items: the last-item problem and clean solutions
- Nested flex containers without confusion
- Reading a real site's layout and spotting the flex containers

Projects built:

- Photo gallery row and a three-card feature section, both wrapping cleanly as the window narrows

Practice: Recreate two sections of a real landing page using only flexbox

CSS Grid

Topics:

- grid-template-columns and rows: designing in two dimensions
- The fr unit and repeat()
- Placing items: spanning columns and rows
- grid-template-areas: layouts you can read like a map
- Grid vs flexbox: a working rule for choosing
- Gaps, alignment, and auto-fit for self-arranging grids

Projects built:

- Magazine-style layout: a featured story spanning two columns above a grid of smaller pieces

Practice: Build the same page layout twice, once in flexbox, once in grid, and write three sentences on which felt right

Responsive Design

Topics:

- Why mobile-first is the sane default
- Media queries: syntax, breakpoints, and where breakpoints actually come from
- The viewport meta tag, the line everyone forgets
- Flexible images: max-width 100% and object-fit
- Testing with the dev tools device toolbar and on a real phone
- Responsive navigation: what stacks, what hides, what stays

Projects built:

- Make the month's layouts fully responsive: gallery, cards, and magazine layout all working from phone to desktop

Practice: Audit two favorite websites at three screen widths and note what changes at each

Assessment: Layout check: build a responsive two-breakpoint layout live from a wireframe

Month 4: Forms, Polish, and the Capstone Site (Weeks 13-16)

Forms

Topics:

- form, input, label, and why every input needs its label
- Input types: text, email, number, date, and what the browser gives you free
- Textareas, selects, checkboxes, and radio buttons
- Styling form controls so they match the site
- Built-in validation: required, min, max, and helpful error states
- What happens on submit, honestly: where the backend would take over

Projects built:

- A styled contact form: name, email, subject dropdown, message, and a submit button that validates

Practice: Build a sign-up form for an imaginary club, labels and validation included

Polish and Performance

Topics:

- Transitions: hover effects that feel smooth instead of sudden
- Simple keyframe animations, used with restraint
- Favicons and page titles: the details visitors notice
- Image weight: resizing and compressing before uploading
- A consistent spacing scale: the quiet secret of pages that look professional
- Accessibility pass: contrast, alt text, and keyboard-reachable links

Projects built:

- Polish pass on your best existing page: transitions, favicon, compressed images, and an accessibility check

Practice: Run a contrast checker on your color scheme and fix anything that fails

Capstone Build

Topics:

- Scoping the personal site: home, about, gallery or projects, and contact
- A shared header and footer across pages, kept consistent by hand
- Wireframing each page before building it
- Reusing your components: cards, navigation, and forms from past weeks
- Working from a checklist: structure, styles, responsive, polish
- Mid-build review with the instructor and a course correction if needed

Projects built:

- The capstone: a multi-page personal site, four pages, shared navigation, responsive throughout

Practice: Complete two pages between classes to keep the build on schedule

Test, Publish, Present

Topics:

- Cross-device testing: phone, tablet width, and a second browser
- The pre-launch checklist: links, images, validator, spelling
- Publishing to free hosting so the site has a real address
- Sharing the link and what to say when you do
- Presenting design decisions: why it looks the way it looks
- Where this leads: JavaScript to make pages interactive

Projects built:

- The published site: live at a real URL, presented to the class with a walkthrough of one design decision per page

Practice: Send the live link to three people and collect one piece of feedback from each

Assessment: Capstone presentation: live site walkthrough plus questions on structure and CSS choices

PROJECTS THE STUDENT BUILDS AND KEEPS

- Two-page mini site with working links and images
- Semantic restructure of a real article into clean HTML
- Typography makeover: one article styled two different ways
- Profile card component, spaced and shadowed like a real design
- Navigation bar with logo, links, and hover states
- Responsive photo gallery and three-card feature section
- Magazine-style CSS grid layout
- Styled contact form with built-in validation
- The capstone: a published multi-page personal website

Total projects: 10+ real pages and components, ending with a published multi-page website

SKILLS THE STUDENT WILL HAVE AT THE END

- Semantic HTML: structuring any page correctly, from articles to forms to tables
- CSS fundamentals: selectors, the cascade, the box model, and typography that reads well
- Layout: flexbox and grid used deliberately, with a working rule for choosing between them
- Responsive design: mobile-first pages that hold up from phone to desktop
- Shipping: validating, testing across devices, compressing images, and publishing to free hosting
- Design judgment: spacing, contrast, and restraint, trained by recreating real designs

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building pages along with the instructor

Practice: 2-3 hours of building and recreation exercises between classes

Review: Every submitted page reviewed with written feedback; recurring issues reopened in

class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for stuck layouts and broken pages
Progress updates: Regular progress notes to parents of younger students; adults get direct feedback on every build

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside a published website that speaks for itself

WHERE THIS CAN LEAD

- JavaScript next: our JavaScript courses turn these static pages into interactive apps
- The full front-end track: React and modern tooling once both layers are solid
- Freelance-style small projects: simple sites for family, clubs, and local groups
- Design-side paths: UI design tools make far more sense once you know what CSS can do
- WordPress and site-builder work with the rare ability to fix what the builder gets wrong

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is HTML a programming language?

A: Strictly speaking, no: HTML is a markup language that describes structure, and CSS describes appearance; neither has loops or logic. That does not make them lesser skills. Every website ever made rests on them, layout in CSS is genuine engineering thinking, and developers who never learned this layer properly struggle with it for years. This course is the foundation programming languages get built on top of.

Q: What age is this course right for?

A: The course runs from age 10 up to adults, and batches are grouped by age, so a 10-year-old is not sharing a class with a 40-year-old career switcher. The material is the same because HTML and CSS are the same; the pace, examples, and project themes are adjusted to the room. For children under 10 we recommend our kids courses, which build the same instincts through friendlier tools.

Q: Do I need this before learning JavaScript?

A: It helps a great deal. JavaScript's whole job in the browser is manipulating HTML and CSS, so students who know both layers learn it noticeably faster and debug with far less pain. Our JavaScript course does teach the markup each project needs along the way, so either order works, but HTML and CSS first is the smoother road.

Q: What will I have built by the end?

A: Ten or more real pages and components: a semantic article page, styled cards, a navigation bar, a responsive photo gallery, a magazine-style grid layout, and a working contact form. The final month is a capstone: a multi-page personal website, responsive from phone to desktop, published to a real URL you can send to anyone.

Q: How are the classes run?

A: Live one-hour online classes, twice a week, with a real instructor and at most 10 students in a group batch. Students build during class, not just watch, and everything submitted between classes comes back with written feedback. Mini-batch (3 to 4 students) and 1-on-1 options are available for learners who want closer attention.

Q: Is there a free trial class?

A: Yes, the first demo class is free with no commitment. Book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. It is a real class, not a sales pitch: you or your child will build something small in it.

Q: Do I need to buy any software?

A: No. Everything used in the course is free: the VS Code editor, a modern browser like Chrome, Google Fonts, and free hosting for publishing the capstone. Any reasonably recent laptop or desktop is enough; HTML and CSS are the lightest possible workload for a computer.

Q: Can this course lead to real work?

A: On its own, HTML and CSS get you to simple static sites, which is genuinely enough for personal pages, clubs, and small local projects. For paid front-end work you will also want JavaScript, which is exactly the next course in our track. What this course guarantees is the layer beneath all of that, learned properly instead of patched together later.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/html-css-course-for-beginners-build-real-websites>

=====

Course link: <https://learn.modernagecoders.com/courses/ib-diploma-computer-science-course>
Subject area: IB Diploma Computer Science
Designed for: Teens
Who can join: IB Diploma Programme students taking Computer Science at SL or HL
Course length: Ongoing DP support, joinable any month
Time needed: 2 live classes/week + coursework and IA support
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online IB Diploma Computer Science classes for the new guide: both themes, the case study in Paper 1, algorithmic thinking in Paper 2 (Python or Java), and full support on the internal assessment.

In one line: Taught to the new guide, first assessed May 2027, with the internal assessment supported properly instead of left to the last term.

ABOUT THE PROGRAMME

IB Computer Science changed. The new Diploma Programme guide was launched in February 2025 for first teaching in August 2025, with first assessment in May 2027, and the differences are not cosmetic. Paper 3 has been removed entirely at both SL and HL. The options are gone. The case study is now covered by both SL and HL students and is assessed within Paper 1. Databases and object-oriented programming are now studied at both levels, not just HL. Any course still built around the old structure is preparing students for an examination that no longer exists. This course follows the new guide as it stands. The syllabus is organised around two themes: concepts in computer science, and computational thinking and problem-solving. Theme A covers computer fundamentals, networks, databases and machine learning. Theme B is where students learn to actually solve problems with code, which is what Paper 2 tests. The programme is built for how the Diploma actually runs. Theory is taught in the syllabus's own order so it lines up with school teaching, the case study is worked properly rather than skimmed the term before the exam, and the internal assessment, the computational solution, is supported from planning through to documentation, with the new emphasis on evidencing the computational thinking process. For Paper 2 we teach Python by default and support Java, since the new guide runs two separate versions of that paper and schools choose one at department level.

WHAT MAKES THIS COURSE DIFFERENT

- Taught to the NEW guide (first teaching 2025, first assessment May 2027): Paper 3 removed, options gone, case study inside Paper 1, databases and OOP at both SL and HL
- The internal assessment is supported from the start, not rescued at the end, including the new emphasis on documenting the computational thinking process
- The case study is worked in depth, because it now applies to SL students too and is assessed in Paper 1
- Paper 2 taught in the language your school chose: Python by default, Java supported, since the guide runs two separate versions of the paper
- Real programming, not slideware: students write and run code every week, which is the only thing that makes Paper 2 tractable
- Live, small batches with work reviewed on the spot, and SL and HL students taught to their own level

THE LEARNING PATH, STEP BY STEP

Phase 1: Theme A part one: computer fundamentals, system architecture and networks, with the ethical issues woven through
Phase 2: Theme A part two: databases and SQL, then machine learning and AI, plus the annual case study
Phase 3: Theme B: computational thinking, algorithms and OOP in Python or Java, the internal assessment, then full exam practice

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THEME A, COMPUTER FUNDAMENTALS AND NETWORKS

How computing systems actually work, taught so the Paper 1 answers come from understanding rather than recall.

Month 1: Computer Fundamentals and System Architecture (Weeks 1-4)

Hardware, Software and the System

Topics:

- The components of a computer system and how they interact
- Primary and secondary memory, and why the distinction matters
- System software versus application software
- The role of the operating system as a resource manager
- Reading a system description and identifying what each part contributes

Projects built:

- Map a described real system end to end, labelling every hardware and software component

Practice: 10 structured questions in IB command-term style

System Architecture and the Processor

Topics:

- CPU architecture, registers and buses
- The machine instruction cycle, step by step
- Cache and how it affects real performance
- Binary representation and why everything reduces to it
- Comparing architectures on evidence rather than assertion

Projects built:

- Trace a full instruction cycle and explain each stage in your own words

Practice: 10 architecture questions including compare and evaluate commands

Data Representation and Abstraction

Topics:

- Representing numbers, text, images and sound in binary
- Abstraction as the central idea of the whole course
- Why abstraction makes large systems possible
- Identifying the level of abstraction in a described system
- Using precise language, which the IB rewards heavily

Projects built:

- Describe one everyday system at three different levels of abstraction

Practice: 10 representation and abstraction questions

Assessment: A timed check on computer fundamentals, marked against IB command terms

Ethical and Social Issues in Computing

Topics:

- The ethical dimension the guide expects you to weave through every topic
- Privacy, surveillance, data ownership and digital divide
- Building a balanced argument with real stakeholders named
- Why one-sided answers cap your marks on evaluate questions
- Linking ethics to the specific technology in the question

Projects built:

- Write a balanced evaluation of one real computing controversy, naming stakeholders on both sides

Practice: 6 extended-response ethics questions, peer reviewed

Month 2: Networks, Security and the Cloud (Weeks 5-8)

Network Structures and Types

Topics:

- LAN, WAN and the topologies that matter
- Client-server and peer-to-peer models compared
- Network hardware and what each device is responsible for
- Bandwidth, latency and what actually limits real performance
- Choosing and justifying a network design for a scenario

Projects built:

- Design a network for a described organisation and justify every choice

Practice: 10 network-design questions with justification marks

Protocols and Data Transmission

Topics:

- Why protocols exist and what would happen without them
- Layers, packets and packet switching
- TCP/IP and HTTP at the depth the guide requires
- Error handling and flow control in plain terms
- Tracing a request across a network

Projects built:

- Trace a single web request across the layers, naming the protocol at each step

Practice: 10 protocol questions at exam standard

Cybersecurity

Topics:

- Threats: malware, phishing, denial of service, interception, social engineering
- Encryption, symmetric and asymmetric, explained precisely
- Authentication, access control and firewalls
- Matching a defence to a specific threat, which is where the marks are
- The human factor, usually the real vulnerability

Projects built:

- Produce a security assessment for a described system with prioritised, justified recommendations

Practice: 10 threat-and-mitigation scenarios

Assessment: End-of-phase test on networks and security

Cloud Computing

Topics:

- What cloud computing actually is, beyond the marketing
- Service and deployment models at guide depth
- The genuine advantages and the genuine risks
- Data sovereignty and the ethical questions it raises
- Evaluating whether cloud is right for a described organisation

Projects built:

- Write a balanced recommendation on moving a described organisation to the cloud

Practice: 8 evaluation questions on cloud adoption

PHASE 2: DATABASES, MACHINE LEARNING AND THE CASE STUDY

The two Theme A areas that are new or expanded for both SL and HL, plus the case study that is now assessed inside Paper 1.

Month 3: Databases and SQL (Weeks 9-12)

The Relational Model

Topics:

- Tables, records, fields, primary and foreign keys
- Relationships and why they prevent duplicated data
- Normalisation at the level the guide expects
- Reading and drawing an entity relationship diagram
- Why databases are now examined at SL as well as HL

Projects built:

- Design a normalised relational schema for a described scenario and draw its ER diagram

Practice: 8 schema-design tasks with keys and relationships identified

SQL in Practice

Topics:

- SELECT, FROM, WHERE, ORDER BY
- Aggregate functions and GROUP BY
- Joining tables to answer a real question
- INSERT, UPDATE and DELETE
- Turning a written requirement into the query that satisfies it

Projects built:

- Build a working database and write the queries that answer real questions about it

Practice: 12 SQL problems building from single conditions to joins and aggregation

Data Storage, Integrity and Ethics

Topics:

- Data integrity, validation and consistency
- Transactions and why they matter
- Backup, recovery and redundancy
- Data protection and the ethics of holding personal data
- Connecting database design to the privacy questions from Phase 1

Projects built:

- Audit a described database for integrity and privacy risks and recommend fixes

Practice: 8 integrity and ethics questions

Assessment: A timed check on databases and SQL

Machine Learning and AI

Topics:

- What machine learning is, and how it differs from ordinary programming
- Supervised and unsupervised learning at guide depth
- Training data, features and why data quality decides outcomes
- Bias in data and its real consequences
- Where AI is genuinely appropriate and where it is not

Projects built:

- Explain how a described ML system reaches a decision, then identify where bias could enter

Practice: 10 machine-learning questions including an ethics evaluation

Month 4: The Case Study and Computational Thinking (Weeks 13-16)

Working the Annual Case Study

Topics:

- How the case study now sits inside Paper 1 for both SL and HL
- Reading the case study actively rather than passively
- Building a glossary of its specific terminology
- Researching the underlying technology properly
- Anticipating the question types the case study invites

Projects built:

- Produce a full annotated summary and terminology glossary for the current case study

Practice: Structured questions drawn directly from the case study material

Case Study Depth and Application

Topics:

- Linking case study technology back to Theme A concepts

- Preparing evidence-based extended answers
- Handling unfamiliar terminology under exam pressure
- Evaluating solutions proposed within the case study
- Practising the specific command terms it attracts

Projects built:

- Write two full extended responses on the case study and mark them against the criteria

Practice: Timed case-study responses with feedback

Assessment: A timed case-study section under Paper 1 conditions

Computational Thinking

Topics:

- Decomposition, pattern recognition, abstraction and algorithm design
- Thinking through a problem before writing any code
- Representing a solution as pseudocode and as a flowchart
- Tracing an algorithm accurately with a trace table
- Why this thinking is now explicitly assessed in the IA

Projects built:

- Take one problem from analysis through to a fully traced algorithm, documenting each thinking step

Practice: 10 algorithm-design and trace-table tasks

Algorithms and Efficiency

Topics:

- Standard algorithms: searching and sorting
- Comparing algorithms on efficiency rather than preference
- Arrays and collections and when each is right
- Recursion at the depth the guide expects
- Explaining why one algorithm beats another for a given case

Projects built:

- Implement and compare two searching or sorting algorithms on the same data and report the difference

Practice: 10 algorithm and efficiency questions

----- PHASE 3: PROGRAMMING, THE INTERNAL ASSESSMENT AND EXAM MASTERY -----

Theme B in code, the computational solution built properly, and both papers drilled under timed conditions.

Month 5: Programming and Object-Oriented Programming (Weeks 17-20)

Programming Fundamentals in Your Paper 2 Language

Topics:

- Variables, data types, input and output
- Selection and iteration, and nesting them cleanly
- Writing readable code that a marker can follow
- Python by default, Java supported, matching the version of Paper 2 your school chose
- Debugging as a method rather than guesswork

Projects built:

- Write and run a complete program that solves a described problem end to end

Practice: 12 short programs, each tested with your own test data

Data Structures and File Handling

Topics:

- Arrays, lists and dictionaries or maps
- Two-dimensional structures and traversing them
- Reading from and writing to files
- Choosing the right structure for the problem
- Handling errors and unexpected input

Projects built:

- Build a program that loads data from a file, processes it and writes a result

Practice: 10 data-structure and file-handling programs

Object-Oriented Programming (SL and HL)

Topics:

- Classes and objects, and why OOP exists
- Encapsulation with private fields and accessor methods
- Inheritance and polymorphism at guide depth
- Designing a small class hierarchy for a real problem
- Why OOP is now studied at both SL and HL under the new guide

Projects built:

- Design and implement a small object-oriented solution with at least two related classes

Practice: 8 OOP design-and-implement tasks

Assessment: A timed programming assessment covering structures and OOP

Starting the Internal Assessment

Topics:

- What the computational solution actually requires
- Choosing a problem with a real client and genuine scope
- Planning: success criteria, record of tasks, design
- The new emphasis on evidencing the computational thinking process
- Common IA mistakes that cost marks before a line of code is written

Projects built:

- Agree an IA problem with a real client and complete the planning and design documentation

Practice: Draft planning documents reviewed against the assessment criteria

Month 6: Completing the IA and Full Exam Practice (Weeks 21-24)

Building and Documenting the Solution

Topics:

- Developing the solution in stages against the success criteria
- Documenting techniques used, with justification
- Evidencing the computational thinking process as the new guide expects
- Testing with real data and recording the outcomes
- Keeping the record of tasks honest and current

Projects built:

- Build the working solution and complete the development documentation

Practice: Weekly IA checkpoints against the published criteria

Finishing the IA and the Video

Topics:

- Evaluating the solution against every success criterion
- Client feedback and honest recommendations for improvement
- Producing the required video demonstration
- Final checks on structure, evidence and word limits
- Academic honesty and correct referencing

Projects built:

- Complete and submit the full internal assessment package

Practice: Final IA review against every criterion

Assessment: Full IA marked against the official criteria with written feedback

Paper 1 Technique

Topics:

- How Paper 1 is structured under the new assessment model
- IB command terms: state, describe, explain, compare, evaluate, and what each is worth
- Structuring an extended response so every mark is reachable
- Handling the case study section confidently
- Managing time so the long questions are not rushed

Projects built:

- A full timed Paper 1, reviewed question by question with every miss tagged by cause

Practice: Two timed Paper 1 attempts, fully reviewed

Paper 2 and Full Mocks

Topics:

- How Paper 2 tests algorithmic thinking rather than recall
- Writing correct code by hand under time
- Reading the problem properly before starting to write
- Full timed mocks on both papers under real conditions
- Turning the error log into a final revision plan

Projects built:

- Two complete mock examinations across both papers, each fully reviewed

Practice: Repeated full mocks until pacing and accuracy are consistent

Assessment: Final full mock on both papers, benchmarked against the target grade

WHERE THIS CAN LEAD

- Strong SL or HL Computer Science grades under the new assessment model
- A completed internal assessment that is genuinely the student's own work
- Real programming ability in Python or Java
- A credible foundation for a computer science or engineering degree application
- Computational thinking that transfers across the whole Diploma

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course taught to the new IB Computer Science guide?

A: Yes. The new DP Computer Science guide was launched in February 2025 for first teaching in August 2025, with first assessment in May 2027. We teach that structure: Paper 3 removed at both SL and HL, options no longer available, the case study covered by SL and HL students and assessed within Paper 1, and databases and object-oriented programming studied at both levels.

Q: What actually changed from the old syllabus?

A: The biggest changes are that Paper 3 has been removed entirely at both SL and HL, the options are gone, the case study now applies to SL students too and is assessed inside Paper 1, and databases and OOP are now studied at both SL and HL rather than being HL territory. The internal assessment remains a computational solution but with a new emphasis on documenting the computational thinking process.

Q: Do you teach Python or Java for Paper 2?

A: Both are supported. The new guide runs two separate versions of Paper 2, one in Python and one in Java, and schools choose which at department level. We teach Python by default because most students find it clearer, and we teach Java for students whose school selected that version. Tell us which your school chose and we place your child accordingly.

Q: Can you help with the internal assessment?

A: Yes, and we start it early rather than rescuing it at the end. We support choosing a

problem with a real client, writing the success criteria, the design and record of tasks, building and documenting the solution, testing, the evaluation, and the video. The new guide puts more weight on evidencing the computational thinking process, so we build that documentation as the work happens. The solution must be the student's own work, and we keep it that way.

Q: My child is taking SL. Is this course still right?

A: Yes. Under the new guide SL students cover the case study, databases and object-oriented programming too, so the content overlap with HL is larger than it used to be. We teach SL and HL students within the same live batch but to their own required depth, and HL-only extension work is handled separately.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ib-diploma-computer-science-course>

=====

COURSE 72 OF 118: IB MATHS AA & AI: SL & HL, LIVE ONLINE WITH IA COACHING

=====

Course link: <https://learn.modernagecoders.com/courses/ib-mathematics-aa-ai-masterclass>

Subject area: Exam Board Mathematics

Designed for: High school

Who can join: IB DP students (AA & AI, SL & HL) and MYP students heading to the DP

Course length: 24 months (the full Diploma Programme arc)

Time needed: 2 live one-hour classes per week + weekly past-paper work

Certificate: Course completion certificate + documented paper-score progression

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online IB Maths course for AA and AI, SL and HL: syllabus-exact teaching, paper-specific technique, GDC fluency and honest IA coaching. Free demo first.

In one line: Your exact course, your exact level, taught to a 7.

ABOUT THE PROGRAMME

A full-Diploma live program matched to the student's actual IB course: Analysis & Approaches or Applications & Interpretation, at SL or HL. IB maths punishes generic tutoring, the two courses examine different things, AA runs a strictly non-calculator paper while AI assumes the GDC everywhere, HL adds an extended problem-solving paper, and the Internal Assessment is worth a fifth of the grade. This masterclass teaches the syllabus for genuine ownership, trains paper-specific technique on real past questions and mark schemes, builds GDC fluency as a scored skill, and coaches the Mathematical Exploration honestly, structure, criteria and critique, with the work staying the student's own under IB academic integrity rules.

WHAT MAKES THIS COURSE DIFFERENT

- Course-exact teaching: AA and AI are different subjects here, as they are on exam day
- Non-calculator craft for AA Paper 1 and GDC power-workflows for AI, each trained deliberately
- The IA coached properly: topic selection against the actual criteria, milestones, mathematical depth and honest draft critique, never ghost-writing
- HL Paper 3 stamina built through olympiad-flavoured problem work
- Predicted-grade awareness: the arc aims at internal assessments too, because those set the number universities see
- MYP-era gaps repaired inline instead of ignored
- One dedicated mentor across the whole two-year arc
- Honest course-fit advice, including 'stay at SL' when that is the truth

THE LEARNING PATH, STEP BY STEP

Phase 1: The shared core built to DP strength: algebra, functions and the toolkit both courses stand on, with MYP debt repaired

Phase 2: Course-specific depth: calculus and proof for AA, statistics, modelling and technology for AI, plus IA season, shepherded

Phase 3: Syllabus completion, paper craft by course and level, HL Paper 3 training and full past-paper cycles into the exam session

WHAT A STUDENT NEEDS BEFORE STARTING

Education: IB DP students (either course, either level) or MYP years 4-5 students heading to the DP

Math background: Any, the diagnostic finds the real base and MYP-era gaps are repaired inline

Equipment: Computer with stable internet; an IB-approved GDC (we advise on models)

Time commitment: 2 live hours plus 2-3 hours of past-paper and IA work weekly

English: Course taught in English

Motivation: The Diploma provides it; we supply the method and the calm

WHO THIS COURSE IS FOR

Aa students: AA SL/HL students who want the ideas owned and Paper 1 made routine

Ai students: AI SL/HL students who need modelling depth and real GDC fluency

Ia season: Students whose Exploration is drifting and needs structure and honest critique

HL ambition: HL students building the Paper 3 stamina regular homework never builds

Myp runway: MYP students buying the cheapest IB insurance: a real algebra base

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE SHARED CORE AT DP STRENGTH (MONTHS 1-8)

Both IB courses stand on the same five topic areas. This phase builds number & algebra and functions to Diploma strength, repairs MYP-era gaps inline, and establishes the retrieval and paper habits that a two-year syllabus demands.

Months 1-2: Diagnostic, Course Fit and Algebra at DP Pace (Weeks 1-8)

Diagnostic and Course Fit (AA vs AI, SL vs HL)

Topics:

- Full syllabus-position diagnostic across the five IB topic areas
- Locating MYP-era gaps in algebra and number
- Current GDC habits and calculator setup review
- Evidence-based AA vs AI course comparison
- SL vs HL demand and workload honesty
- Baseline record set up for later progress tracking

Practice: Complete a short mixed diagnostic set and log every gap it exposes into a personal error log.

Detail: Full diagnostic: syllabus position, MYP-era gaps, GDC habits, and an honest course-fit read (AA vs AI, SL vs HL) with evidence

Exponents and Logarithms to DP Fluency

Topics:

- Laws of exponents from first meaning
- Rational and negative indices
- Logarithms as the inverse of exponentials
- Laws of logarithms and change of base
- Solving exponential and logarithmic equations
- Fluency drills that both courses assume

Practice: Work a fluency set converting between exponential and logarithmic form and solving equations without hesitation.

Detail: Exponents, logarithms and the laws that both courses assume, taught from meaning, drilled to fluency

Sequences, Series and Financial Applications

Topics:

- Arithmetic sequences and series
- Geometric sequences and series
- Sigma notation fluency
- Sum to infinity and the idea of convergence
- Compound interest and annuities (an AI favourite)
- Financial modelling contexts on the GDC

Practice: Solve a mixed set that pairs pure sequence questions with a compound-interest financial application.

Detail: Sequences and series: arithmetic, geometric, sigma notation and financial applications (an AI favourite)

Proof and Mathematical Writing Foundations

Topics:

- Direct proof structure
- Clear mathematical writing and notation
- Logical connectives and reasoning
- Counterexamples and disproof
- Deeper proof demands for AA HL
- Setting the standard for written working

Practice: Write out two short direct proofs with full justification and correct notation.

Detail: Proof and reasoning foundations: direct proof and mathematical writing habits (deeper for AA HL)

Months 3-5: Functions, the DP's Load-Bearing Topic (Weeks 9-20)

Function Anatomy: Domain, Range, Composition, Inverses

Topics:

- Domain and range from meaning
- Function notation and evaluation
- Composite functions
- Inverse functions and their graphs

- One-to-one functions and the horizontal-line idea
- Why the whole DP silently assumes this fluency

Practice: Build a reference sheet of domain, range and inverse for a set of standard functions.

Detail: Function anatomy: domain, range, composition and inverses, the material the whole DP silently assumes

Transformations, Quadratics and Polynomials

Topics:

- Translations, stretches and reflections of graphs
- Reading graphs fluently
- Quadratics in vertex, root and factored forms
- The discriminant as an idea, not just a formula
- Polynomial behaviour and the factor theorem
- Sketching to DP depth

Practice: Sketch a family of transformed graphs by hand, then confirm each on the GDC.

Detail: Transformations and graph literacy; quadratics and polynomials to DP depth; the discriminant as an idea

Exponential, Log and Rational Functions with the GDC

Topics:

- Exponential growth and decay models
- Logarithmic functions and scales
- Rational functions and asymptotes
- Modelling contexts drawn from data
- GDC graphing workflow set up properly
- Interpreting features straight from a graph

Practice: Model a given real context with an exponential or rational function and justify the fit on the GDC.

Detail: Exponential, logarithmic and rational functions with modelling contexts; GDC graphing workflows introduced properly

Months 6-8: Geometry, Trigonometry and First Paper Craft (Weeks 21-32)

Trigonometry: Unit Circle to Applied Bearings

Topics:

- Unit circle and exact values (AA depth)
- Trigonometric identities
- Solving trigonometric equations (AA)
- Right-triangle trigonometry (AI emphasis)
- Applied trig with bearings and angles of elevation
- Radian measure introduced

Practice: Complete a split set: AA exact-value and identity work, AI applied bearings problems.

Detail: Trigonometry rebuilt: unit circle, identities and equations (AA depth) / right-triangle and applied trig with bearings (AI emphasis)

Vectors, 3D Geometry and the Sine and Cosine Rules

Topics:

- Vectors and vector operations (AA HL)
- 3D coordinate geometry
- The sine rule in non-right triangles
- The cosine rule and the ambiguous case
- Area of a triangle formula
- Applied problems combining the rules

Practice: Solve a set of non-right-triangle problems, choosing the sine or cosine rule deliberately.

Detail: Vectors (AA HL) and 3D geometry; sine and cosine rules in applied problems

First Past-Paper Season and the Review Ritual

Topics:

- Anatomy of an IB exam question
- Command terms and what they demand
- Reading and applying mark schemes
- Method marks versus answer marks
- The review ritual used for the next eighteen months
- Building the error-log habit

Practice: Sit one short past-paper section under time, then self-mark it against the real mark scheme.

Assessment: End-of-phase checkpoint: a timed past-paper section marked against the official scheme sets the Phase 1 baseline.

Detail: First past-paper season: question anatomy, mark-scheme logic, and the review ritual used for the next eighteen months

PHASE 2: COURSE-SPECIFIC DEPTH + THE IA (MONTHS 9-16)

The AA and AI tracks now genuinely diverge: calculus and proof depth for AA, statistics and modelling with full GDC fluency for AI. In the middle of this phase sits IA season, the Mathematical Exploration, coached against the real criteria while remaining the student's own work.

Months 9-11: The AA Track / The AI Track (Weeks 33-44)

AA Differentiation from First Principles / AI Statistics and Regression

Topics:

- AA: limits and the idea of a derivative
- AA: differentiation from first principles
- AA: derivative toolkit and chain rule mastery
- AI: descriptive statistics measures
- AI: correlation and Pearson r
- AI: linear regression on the GDC

Practice: AA students differentiate a mixed set including chain-rule cases; AI students run a regression on a dataset and interpret it.

Detail: AA: limits and differentiation from first principles; the derivative toolkit including chain rule mastery. AI: descriptive statistics, correlation and regression on the GDC

AA Applications and Integration / AI Probability Distributions

Topics:

- AA: tangents and normals
- AA: optimisation problems
- AA: kinematics with calculus
- AA: introduction to integration
- AI: probability rules and diagrams
- AI: binomial and normal distributions with interpretation

Practice: AA students solve an optimisation and a kinematics problem; AI students compute and interpret binomial and normal probabilities.

Detail: AA: applications, tangents, optimisation, kinematics; introduction to integration. AI: probability, distributions (binomial, normal) and interpretation craft

AA Integration and Areas / AI Modelling Cycle and Networks

Topics:

- AA: integration techniques
- AA: definite integrals and area under a curve
- AA: volumes of revolution
- AI: the modelling cycle, choose, fit, critique, improve
- AI: matrices and their operations (HL)
- AI: networks and graph theory (HL)

Practice: AA students find an area and a volume by integration; AI students fit and critique a model for a given dataset.

Detail: AA: integration techniques and areas/volumes. AI: modelling cycle, choose, fit, critique and improve a model; matrices and networks (HL)

Months 12-14: IA Season, the Exploration, Shepherded (Weeks 45-56)

IA Topic Selection and Proposal

Topics:

- The five IA assessment criteria explained
- Genuine personal engagement
- Mathematics chosen so it can actually score
- Scoping a workable exploration question
- A milestone plan for the IA
- Academic integrity expectations set from the start

Practice: Draft three candidate IA questions and test each honestly against the five criteria.

Assessment: IA milestone: the exploration proposal is locked with a dated milestone plan.

Detail: Topic selection against the five criteria: personal engagement that is real, mathematics that can score; proposal locked with milestones

Deepening the IA Mathematics

Topics:

- Pushing mathematical rigour in the exploration
- Meeting the aim criterion
- Communication and structure standards
- Correct notation throughout
- Student-led work with mentor questioning
- Keeping the exploration honestly the student's own

Practice: Develop the core mathematics of your exploration and annotate where each criterion is being met.

Detail: The mathematics deepened: the mentor pushes rigour and aim while the student drives; communication and notation standards set

IA Draft Critique and Final Polish Plan

Topics:

- Honest critique against each criterion
- Strengthening the reflection
- Improving communication and presentation
- The final polish plan
- Academic integrity kept throughout
- Self-checking before submission

Practice: Revise your IA draft against criterion-by-criterion feedback and record the changes you make.

Assessment: IA milestone: a full draft is reviewed against all five criteria before the final polish.

Detail: Draft critique rounds: honest feedback against each criterion, reflection strengthened, and the final polish plan, the work staying the student's own throughout, per IB academic integrity rules

Months 15-16: Syllabus Depth Continues (Weeks 57-64)

AA Further Calculus / AI Hypothesis Testing

Topics:

- AA: further integration and differentiation
- AA HL: deeper techniques and Maclaurin ideas
- AI: the logic of hypothesis testing
- AI: chi-squared tests for independence
- AI: t-tests and interpretation
- Reading statistical output critically

Practice: AA students tackle a further-calculus set; AI students run and interpret a chi-squared or t-test.

Detail: AA: further calculus (HL: deeper techniques, Maclaurin ideas); AI: hypothesis testing, chi-squared and t-tests with interpretation

HL Paper 3 Investigations Introduced / SL Consolidation

Topics:

- HL: the structure of Paper 3 extended investigations
- HL: harvesting earlier parts of a problem
- HL: sustaining an unfamiliar investigation
- SL: consolidation by error-log priority
- SL: targeted retrieval of weak topics
- Both: connecting topics across the syllabus

Practice: HL students attempt one extended investigation; SL students clear their top three error-log items.

Assessment: End-of-phase checkpoint: HL students are marked on a Paper 3 style investigation and SL students on a targeted consolidation set.

Detail: HL students of both courses: Paper 3-style extended investigations introduced;

SL students: consolidation cycles by error-log priority

PHASE 3: SYLLABUS CLOSE + PAPER CRAFT TO EXAM SESSION (MONTHS 17-24)

The remaining syllabus is closed with margin, then the program becomes exam craft: papers by course and level, calculator and non-calculator discipline, HL Paper 3 stamina, and full timed cycles that peak in the exam session, with predicted-grade checkpoints along the way.

Months 17-19: Closing the Syllabus with Margin (Weeks 65-76)

Closing the Final Syllabus Topics

Topics:

- Remaining probability and statistics (AA)
- Remaining calculus for AI HL
- Kinematics and applied problems
- Any outstanding course-specific topics
- Topics taught early enough to settle
- Linking new topics back to earlier work

Practice: Complete a set on the last new topics for your exact course and level.

Detail: Final syllabus topics per course and level, taught early enough to settle: remaining probability/statistics (AA), remaining calculus for AI HL, kinematics and applications

Full-Syllabus Connection Map and Spaced Retrieval

Topics:

- Building a personal syllabus connection map
- Linking the five topic areas
- Scheduling spaced retrieval
- Interleaving topics in practice
- Identifying weak links to revisit
- Preparing for full paper cycles

Practice: Build a one-page connection map of the whole syllabus and schedule your spaced retrieval.

Detail: Full-syllabus connection map built by the student; spaced retrieval cycles across all five topic areas

Months 20-22: Paper Craft by Course and Level (Weeks 77-88)

AA Paper 1 Non-Calculator Craft / AI GDC Speed

Topics:

- AA: exact-value fluency
- AA: structure recognition inside questions
- AA: clean written working for method marks
- AI: GDC speed workflows
- AI: efficient graphing, solving and regression
- Acceptable use of technology

Practice: AA students complete a non-calculator set; AI students time their GDC workflows on typical questions.

Detail: AA Paper 1 non-calculator craft: exact values, structure recognition and clean written working. AI: GDC speed workflows for every paper

Paper 2 Technique and Time Allocation

Topics:

- Strategy for extended structured questions
- Method-mark discipline

- Time allocation from personal data
- Recovering from a stuck part
- Presentation that earns follow-through marks
- GDC use within Paper 2

Practice: Sit a Paper 2 section and log your time per question to set personal pacing targets.

Detail: Paper 2 technique: extended structured questions, method-mark discipline and time allocation from personal data

HL Paper 3 Stamina and Investigation Craft

Topics:

- Long unfamiliar investigations
- Harvesting earlier parts to reach later ones
- Keeping momentum through difficulty
- Managing time across a long problem
- The calm that stamina training buys
- Reviewing Paper 3 against the scheme

Practice: Complete a full HL Paper 3 style investigation under time and review it against the mark scheme.

Detail: HL Paper 3: long unfamiliar investigations, harvesting earlier parts, keeping momentum, and the calm that stamina training buys

Months 23-24: Full Cycles and the Exam Session (Weeks 89-96)

Full Timed Past-Paper Cycles

Topics:

- Full timed past-paper cycles
- Papers for the exact course and level
- Marking against real mark schemes
- Error-log closure
- Spotting patterns across mistakes
- Refining exam-day strategy

Practice: Sit a full timed paper, self-mark it, and close the error-log items it reveals.

Assessment: Mock checkpoint: full timed papers are marked against official schemes and score progression is recorded.

Detail: Full timed past-paper cycles by exact course and level, reviewed against real mark schemes; error-log closure

Peak Plan and the Exam Session

Topics:

- Final targeted consolidations
- Rest and recovery discipline
- Exam-day routines
- GDC and materials checklist
- Managing nerves through rehearsal
- Meeting the session as a rehearsed place

Practice: Run a final light consolidation and rehearse your full exam-day routine.

Assessment: The IB examination session: papers are sat for the student's exact course and level.

Detail: Peak plan: final consolidations, rest discipline, exam-day routines, and the session met as a rehearsed place

PROJECTS THE STUDENT BUILDS AND KEEPS

- Two-year error log and syllabus map, maintained by the student
- The Mathematical Exploration (IA), coached against the real criteria
- GDC workflow library (AI) / exact-value fluency system (AA)
- Self-graded past-paper portfolio against real mark schemes
- Full timed paper cycles with documented score progression

Total projects: The IA plus 5+ exam artifacts

SKILLS THE STUDENT WILL HAVE AT THE END

- Algebra, functions and trigonometry at DP strength
- AA: calculus, proof and non-calculator craft
- AI: statistics, modelling and GDC power-workflows
- Mark-scheme literacy and method-mark discipline
- HL Paper 3 stamina and investigation technique
- IA construction against the five criteria
- Predicted-grade exam readiness
- Two-year retrieval and revision systems

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor

Past paper work: Weekly question sets from your exact course and level

Ia season: Milestone check-ins during Exploration months

Total per week: About 4-5 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole Diploma
Recordings: Every class recorded for revision
Progress tracking: Paper-score progression + IA milestones shared with parents
Flexibility: Intensity adjusts around mocks, IA deadlines and the exam session

CERTIFICATION DETAIL

Final certificate: Course completion certificate from Modern Age Coders
Score record: Documented paper-score progression across the arc
Portfolio projects: The IA and past-paper portfolio

WHERE THIS CAN LEAD

- An IB maths grade that holds predicted grades and university offers
- AA route: engineering, mathematics, physics and CS degree readiness
- AI route: business, economics and social-science quantitative literacy
- Ready for College Mathematics Masterclass or competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do you teach both AA and AI, at both SL and HL?

A: Yes, and separately, because they are genuinely different courses. The mentor teaches your student's exact registration: AA's algebraic, proof-flavoured, calculus-heavy road or AI's modelling, statistics and technology-first road, at the right level. The free demo doubles as a course-fit consult if the choice is still open.

Q: How does the IA coaching work, and how far is too far?

A: We coach the Mathematical Exploration properly: topic selection against the actual criteria, structure, pushing the mathematics deep enough to score, and honest critique of drafts. We do not write it or polish prose into someone else's voice, IB academic integrity rules are real, schools check, and the IA only helps your student if it is genuinely theirs. Done right, it is the most coachable 20% of the grade.

Q: My AA student falls apart on the non-calculator paper. Fixable?

A: Very. Paper 1 skill is mostly exact-value fluency plus structure recognition, seeing the quadratic inside the exponential equation. Both are trained, not innate, and every AA class includes non-calculator work so Paper 1 becomes routine long before the exam.

Q: Do you teach GDC (graphing calculator) skills?

A: As a first-class skill, because AI examines it and AA's Paper 2 rewards it: efficient graphing, solving and regression workflows, and knowing what counts as acceptable use of technology. AI students who underuse the GDC leave marks and minutes on the table.

Q: Can you help before predicted grades are set?

A: That is the highest-leverage timing in the IB, predictions ride on school assessments, so a focused arc aimed at the next internal exams moves the number universities actually see. Start earlier than feels necessary; predictions harden sooner than most families expect.

Q: My child is in the MYP. Is this course relevant yet?

A: Yes, MYP students join the Phase 1 material aimed at arriving in the DP with the algebra and functions base both courses silently assume. It is the cheapest IB maths insurance there is.

Q: Can we try it before paying?

A: Yes. Every student starts with a free live demo class that doubles as the diagnostic and course-fit consult, no card details, no obligation.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/ib-mathematics-aa-ai-masterclass>

COURSE 73 OF 118: ICSE COMPUTER APPLICATIONS (CLASS 9-10): JAVA WITH BLUEJ

Course link: <https://learn.modernagecoders.com/courses/icse-computer-applications-java-bluej-course>

Subject area: ICSE Computer Applications

Designed for: Teens

Who can join: ICSE Class 9 and 10 students taking Computer Applications; taught from the basics, no prior coding required

Course length: Class 9 and 10 board program, monthly enrolment

Time needed: 2 live classes/week + weekly BlueJ practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online ICSE Computer Applications classes for Class 9 and 10, taught in Java with BlueJ and aligned to the current CISCE 100+100 scheme, so students write their own programs and score in both the theory paper and the internal assessment.

In one line: A real Java course, taught in BlueJ and aligned to the current CISCE scheme, that ends in top marks in both the theory paper and the internal assessment, because your child can actually write programs, not just reproduce them.

ABOUT THE PROGRAMME

ICSE Computer Applications is taught in Java using the BlueJ environment, and it is the subject where students who genuinely understand code pull away from the ones who memorise programs. This course treats it as a real first programming course, because that is exactly what it is: your child leaves able to write Java, read an error, and solve a problem they have not seen before, which is what the board paper actually asks for. It is also taught to the scheme as it stands now. CISCE marks Computer Applications on a 100 plus 100 basis, unlike the usual 80 plus 20: a 100-mark external theory paper and a 100-mark internal assessment made up of practical lab work and a written project on Disruptive Technologies. Many students lose easy marks on the internal assessment simply because nobody prepared them for it, so we treat both halves seriously from day one. The programme follows the ICSE Class 9 and 10 Computer Applications syllabus in order. It begins with object-oriented thinking and Java from nothing, moves through conditionals, loops and nested loops, then into the Class 10 core: classes as the basis of computation, user-defined methods, constructors, the library and wrapper classes, encapsulation, single and double dimensional arrays, and string handling. Alongside the theory, every student builds a proper practical file of at least fifteen BlueJ assignments and the Disruptive Technologies project, and drills the two-section board paper until earning marks is a habit. Every class is live and small, and every concept is practised by writing code in BlueJ, because that is the only way this subject is ever actually learned.

WHAT MAKES THIS COURSE DIFFERENT

- Aligned to the current CISCE 100 plus 100 scheme: a 100-mark theory paper and a 100-mark internal assessment, so we prepare your child for both halves, not just the written exam most classes stop at
- Taught in BlueJ (version 5.4.2 or higher on JDK 11 or higher), the exact environment used in school and the board practical exam, so nothing on exam day is unfamiliar
- The internal assessment is taken seriously: your child builds a real practical file of at least fifteen lab assignments plus the written project on Disruptive Technologies, worth a full 100 marks
- A real Java course, not rote learning: students stop memorising programs and start writing their own, which is exactly what Section B of the paper rewards
- Board paper technique drilled directly: the compulsory Section A short answers and the answer-any-four programs of Section B, practised against real board-style questions
- Live, small batches where your child writes code during class and gets it reviewed, not lecture videos watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Java and OOP from scratch in BlueJ, the Class 9 core: objects and classes, data types, operators, input, the Math library, conditionals, and loops including nested loops
Phase 2: The Class 10 core: the class as the basis of computation, user-defined methods, constructors, library and wrapper classes, and encapsulation
Phase 3: Arrays and string handling, the full practical file and Disruptive Technologies project, and complete board theory and BlueJ practical exam mastery

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: JAVA AND OOP FOUNDATIONS, THE CLASS 9 CORE (IN BLUEJ) -----

Object-oriented thinking and Java built from nothing in BlueJ, then the Class 9 Computer Applications syllabus in order: the base everything in Class 10 stands on.

Month 1: Object-Oriented Thinking and Java Basics (Weeks 1-4)
Introduction to OOP and the BlueJ Environment

Topics:

- What object-oriented programming means: objects, classes, and why Java is built around them
- The principles behind OOP the syllabus names: abstraction, encapsulation, inheritance and polymorphism, explained in plain language
- Setting up BlueJ (version 5.4.2 or higher on JDK 11 or higher), the exact tool used in school and the board practical
- The anatomy of a Java program: class, main method, statements and semicolons
- The compile-and-run cycle in BlueJ, and reading your first error messages without fear

Projects built:

- First BlueJ program: create a class, compile it, and run a method that prints a short formatted output

Practice: 8 short exercises creating and running simple classes in BlueJ, reading each error message and fixing it

Elementary Concept of Objects and Classes, Values and Data Types

Topics:

- Objects as instances of a class, and the class as a blueprint, with real everyday analogies
- Java's primitive data types: int, double, char, boolean and the rest, and when each is used
- Literals, identifiers, keywords and the rules for naming, exactly as the paper expects
- Type size, range and the difference between primitive and reference types
- Variables, initialisation, and Unicode and ASCII as the syllabus requires

Projects built:

- A class that stores and prints details using the correct data type for each field

Practice: 10 questions classifying data types and choosing the right type for a given value, board-question style

Assessment: A short written check on OOP terms and data types, marked in the board's short-answer style

Operators in Java and Expressions

Topics:

- Arithmetic, relational, logical and assignment operators, and operator precedence
- Increment and decrement, prefix versus postfix, a classic board trap
- Type conversion: implicit widening and explicit casting, and integer versus real division
- The conditional (ternary) operator and evaluating mixed expressions by hand
- Tracing expressions on paper the way Section A questions demand

Projects built:

- A simple calculator class that applies each operator and prints the result

Practice: 12 expression-evaluation questions, predicted on paper then checked in BlueJ

Input in Java and Mathematical Library Methods

Topics:

- Taking input with the Scanner class, the standard the syllabus uses
- Reading int, double, char and String input cleanly
- The Math library: pow, sqrt, abs, max, min, round, ceil, floor and random
- Combining input and Math methods to solve real formulae
- Common Math-method questions exactly as they appear in the paper

Projects built:

- A program that reads values and computes results such as area, simple interest and the roots of a quadratic using Math methods

Practice: 10 input-and-compute programs, each written and run in BlueJ

Assessment: A timed set on operators, expressions and Math methods, scored in the board's style

Month 2: Conditional and Iterative Constructs (Weeks 5-8)

Conditional Constructs: if, if-else, if-else-if

Topics:

- The if, if-else and if-else-if ladder, and choosing the right form
- Nested if statements and how to keep them readable
- Boolean conditions built from relational and logical operators
- Dry-running a decision program by hand to predict its output
- Menu-style decision problems as the paper frames them

Projects built:

- A program that classifies input, for example grade from marks or category from a value, using an if-else-if ladder

Practice: 10 decision programs, each dry-run on paper first then verified in BlueJ

The switch-case Construct

Topics:

- The switch-case statement and when it beats an if-else-if ladder
- The role of break, and the fall-through behaviour when it is missing, a favourite board trap
- The default case and valid switch expression types
- Converting between switch and if-else forms both ways
- Menu-driven programs using switch, exactly as asked in Section B

Projects built:

- A menu-driven program, for example a units converter or a simple arithmetic menu, built with switch-case

Practice: 8 switch-case programs including deliberate missing-break tracing questions

Assessment: A timed check on conditionals covering both if-else and switch, board style

Iteration: for, while and do-while loops

Topics:

- The for, while and do-while loops and how to convert one into another
- Loop control: initialisation, condition and update, and off-by-one errors
- The entry-controlled versus exit-controlled distinction the paper asks about

- Classic loop programs: sum of a series, factorial, counting digits, reversing a number
- Predicting how many times a loop runs, a common Section A question

Projects built:

- A program producing a series result, for example the sum of a series, factorial, or checking a number for a property

Practice: 12 loop programs, each written three ways with for, while and do-while to build fluency

Nested Loops and Patterns

Topics:

- Nested for loops and how the inner loop runs fully for each outer step
- Number and star pattern programs, a guaranteed source of Section B marks
- Prime, palindrome, Armstrong and similar number-property programs
- Tracing nested loops on paper to predict the exact output
- Structuring a longer program cleanly so it earns full marks

Projects built:

- A pattern-printing program and a number-property checker, both built with nested loops

Practice: 10 nested-loop and pattern programs at board difficulty

Assessment: End-of-phase test: a Section-A-style short-answer set plus two

Section-B-style programs, scored on the board rubric

PHASE 2: THE CLASS 10 CORE, CLASSES, METHODS AND ENCAPSULATION

The heart of the Class 10 syllabus: writing your own classes, methods and constructors, and using the library and wrapper classes the paper depends on.

Month 3: Classes, User-Defined Methods and Constructors (Weeks 9-12)

The Class as the Basis of All Computation

Topics:

- Revisiting objects and classes as the foundation of everything in Class 10
- Instance variables and methods, and the state and behaviour of an object
- Access specifiers: private, public and protected, and why they matter
- Creating objects and calling methods, and the role of the main method
- Reading a full class definition the way a Section B question presents it

Projects built:

- A complete class, for example a Student or BankAccount, with fields and methods, created and used from main

Practice: 8 exercises writing and using small classes in BlueJ

User-Defined Methods

Topics:

- Declaring methods: return type, name, parameters and body
- Parameters and arguments, and returning a value versus void
- Pure and impure methods, and call by value as the syllabus frames it
- Method overloading and how the compiler picks a version
- The static keyword and the difference between static and instance methods

Projects built:

- A class with several overloaded and value-returning methods, each tested from main

Practice: 10 method-writing questions, including overloading and pure-versus-impure identification

Constructors

Topics:

- What a constructor is and how it differs from a method
- Default, parameterised and copy constructors
- Constructor overloading and choosing the right one at object creation
- How constructors initialise object state cleanly
- Constructor questions exactly as the board asks them

Projects built:

- A class with default, parameterised and copy constructors, demonstrated by creating objects several ways

Practice: 8 constructor programs and a set of spot-the-difference

method-versus-constructor questions

Assessment: A timed check on classes, methods and constructors, marked on the board rubric

Encapsulation and Clean Class Design

Topics:

- Encapsulation: private fields with public accessor and mutator methods
- Why data hiding protects an object and earns design marks
- Designing a class so its methods fully control its state
- Reading and completing a partially written class, a common Section B format
- Good naming and structure that make a program easy to mark

Projects built:

- An encapsulated class with private fields and get and set methods, used safely from outside

Practice: 8 class-completion questions in the board's fill-in-the-method style

Month 4: Library Classes, Wrapper Classes and Characters (Weeks 13-16)

Library Classes and Wrapper Classes

Topics:

- What the Java library provides and why we do not reinvent it
- Wrapper classes: Integer, Double, Character and the rest, and why they exist

- Autoboxing and unboxing in plain terms
- Converting between primitive types and their wrapper objects
- Parsing a String to a number with parseInt and parseDouble

Projects built:

- A program that reads numeric text input, parses it with wrapper methods, and computes a result

Practice: 8 wrapper-class and parsing questions in board format

Character and Case Handling

Topics:

- The char type, and ASCII and Unicode values
- Character methods: isDigit, isLetter, isUpperCase, isLowerCase, toUpperCase, toLowerCase
- Converting characters to and from their integer codes
- Counting vowels, consonants, digits and spaces, a classic program
- Character-based Section A questions and how to answer them precisely

Projects built:

- A program that scans text and reports counts of vowels, consonants, digits and other characters

Practice: 10 character-method programs and code-value tracing questions

Assessment: A timed set on library, wrapper and character methods, board style

Building the Practical File, Part One

Topics:

- The internal assessment: how the 100 practical marks are actually earned
- What makes a good lab assignment: clear design, correct implementation and testing
- Writing assignments that each demonstrate a theory concept, as CISC requires
- Documenting a program properly: purpose, variable description and output
- Starting the required set of at least fifteen assignments for the year

Projects built:

- The first several formal BlueJ lab assignments, written up to internal-assessment standard

Practice: Assignment write-ups reviewed and corrected to the standard a school and

CISC expect

The Disruptive Technologies Project

Topics:

- The written project on Disruptive Technologies that carries part of the internal assessment
- Choosing a genuine topic, for example AI, blockchain, IoT or robotics
- Structuring the project: what it is, how it works, and its real-world impact
- Writing in the student's own words, honestly and clearly, not copied
- Presenting the project so it earns full internal-assessment marks

Projects built:

- A complete, well-structured written project on a chosen disruptive technology

Practice: Draft, review and finalise the project to internal-assessment standard

Assessment: End-of-phase test: a mixed Section-A and Section-B paper plus a practical-file review

----- PHASE 3: ARRAYS, STRINGS AND FULL BOARD AND PRACTICAL MASTERY -----

The last major syllabus topics, then the whole subject brought together: the two-section theory paper and the BlueJ practical exam, drilled until top marks are a habit.

Month 5: Arrays and String Handling (Weeks 17-20)

Single Dimensional Arrays

Topics:

- Declaring, creating and initialising a single dimensional array
- Traversing an array with a loop, and reading and printing elements
- Finding the sum, maximum, minimum and average of an array
- Linear search and simple frequency counting
- Array questions exactly as Section B presents them

Projects built:

- A program that reads a set of values into an array and reports statistics and a searched value

Practice: 10 single-dimensional-array programs at board difficulty

Sorting and Searching in Arrays

Topics:

- Bubble sort and selection sort, the two the syllabus expects, step by step
- Binary search on a sorted array and why order matters
- Tracing a sort or search pass by pass, a common Section A question
- Choosing the right technique for a given problem
- Writing a full sort-or-search program cleanly for maximum marks

Projects built:

- A program that sorts an array with bubble or selection sort, then binary-searches it

Practice: 8 sorting and searching programs, plus pass-by-pass tracing questions

Double Dimensional Arrays

Topics:

- Declaring and using a two dimensional array as rows and columns
- Nested loops to fill, read and print a matrix
- Row sums, column sums, and the diagonal of a square matrix
- Common matrix programs the paper sets
- Keeping a two-dimensional program readable so it earns full marks

Projects built:

- A matrix program that fills a two dimensional array and reports row, column and diagonal totals

Practice: 8 double-dimensional-array programs

String Handling

Topics:

- The String class and common methods: length, charAt, substring, indexOf, equals and compareTo
- toUpperCase, toLowerCase, trim, replace and concatenation
- Reversing a string, checking a palindrome, and counting words
- Comparing strings correctly, equals versus the double-equals trap
- String programs exactly as Section B frames them

Projects built:

- A text program that reverses input, checks for a palindrome, and counts words and characters

Practice: 10 string-handling programs at board difficulty

Assessment: A timed set covering arrays and strings, scored on the board rubric

Month 6: Board Theory and BlueJ Practical Exam Mastery (Weeks 21-24)

The Theory Paper: Section A Technique

Topics:

- How the 100-mark theory paper is structured across its two sections
- Section A: the compulsory short-answer questions and how to earn every mark
- Precise definitions, output prediction and one-line answers the way examiners want them
- Common Section A traps: operator precedence, missing break, loop counts and casting
- Timing Section A so Section B gets the minutes it needs

Projects built:

- A full Section A attempt under time, then a question-by-question review

Practice: Two timed Section A sets, each fully reviewed with every miss tagged by cause

The Theory Paper: Section B Programs

Topics:

- Section B: answering any four of the program questions, and choosing the right four
- Structuring a full program answer so partial marks are never lost
- Writing clean, commented code by hand under exam conditions
- Handling the class-design, array and string programs that recur every year
- Checking a written program for the small errors that cost marks

Projects built:

- A full Section B attempt of four hand-written programs, marked on the official rubric

Practice: Timed Section B sets across class, array and string program types

The BlueJ Practical Exam and the Internal Assessment

Topics:

- How the school and board practical works in BlueJ, and what is expected
- Writing, compiling, testing and correcting a program under time in BlueJ
- Finalising the practical file of at least fifteen assignments and the project
- Reading a practical problem correctly and planning before coding
- Debugging calmly when a program does not compile the first time

Projects built:

- A full mock practical: solve an unseen problem end to end in BlueJ under exam time

Practice: Repeated timed BlueJ practicals on unseen problems, each reviewed

Assessment: A complete mock practical scored like the real one, with the practical file signed off

Full Mock Board Exams and Final Polishing

Topics:

- Full-length timed mock theory papers under real conditions
- Turning the error log from every mock into a focused revision plan
- Locking in the recurring high-mark programs so they are automatic
- A calm, tested routine for both the theory paper and the practical
- A clear, honest read on the target grade and the final push to reach it

Projects built:

- Two full mock board papers, each fully reviewed and converted into a revision list

Practice: Repeated full mocks until pacing and accuracy are consistent

Assessment: Final full mock board paper, scored and benchmarked against the target grade

WHERE THIS CAN LEAD

- A strong ICSE Computer Applications result in both the theory paper and the 100-mark internal assessment
- A confident move into ISC Computer Science in Class 11 and 12, continuing in Java
- A genuine foundation for a computer science stream and, later, an engineering or CS degree
- Transferable problem-solving and logical-thinking skills that help across every subject
- An early, real head start in programming that compounds for years

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course aligned to the current ICSE Computer Applications syllabus?

A: Yes. It follows the CISCE Class 9 and 10 Computer Applications syllabus, taught in Java with BlueJ, and it is built around the current 100 plus 100 scheme: a 100-mark external theory paper and a 100-mark internal assessment made up of practical lab work and a written project on Disruptive Technologies. We prepare your child for both halves, which is where

many classes stop short.

Q: Do you teach in BlueJ, the environment used in school?

A: Yes. We teach Java in BlueJ, version 5.4.2 or higher on JDK 11 or higher, which is the same environment used in school and in the board practical exam. Your child writes, compiles and tests real programs in it from the first class, so nothing on exam day is unfamiliar.

Q: How does the 100-mark internal assessment work, and do you prepare for it?

A: Computer Applications is marked out of 200: a 100-mark theory paper and a 100-mark internal assessment. The internal assessment comes from practical lab work, at least fifteen assignments across the year, plus a written project on Disruptive Technologies. We build the practical file and the project properly with your child, so the full 100 internal marks are genuinely earned, not left to chance.

Q: My child finds Computer Applications confusing at school. Can this help?

A: This is exactly who the course is for. Most confusion comes from memorising programs without understanding them, so the moment the logic changes, a student is stuck. We teach the concepts from the basics, in a small live batch, until your child can write their own programs. That understanding shows up directly in both the paper and the practical.

Q: Can a Class 9 student join, or is it only for Class 10?

A: Both. A Class 9 student builds the foundations in order, which makes Class 10 far easier. A Class 10 student joins with a quick, solid revision of the Class 9 core and then moves into the Class 10 topics and full board preparation. We place your child at the right point after the demo class.

Q: Will this help with ISC Computer Science later?

A: Yes. ISC Computer Science in Class 11 and 12 continues in Java and builds directly on the Class 9 and 10 foundation, adding topics like recursion, interfaces and exception handling. A student who genuinely understands Computer Applications is set up to do well in ISC, rather than starting from behind.

To enrol or to book a free demo for this course, message Shivam Sir on
+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/icse-computer-applications-java-bluej-course>

=====

COURSE 74 OF 118: IGCSE COMPUTER SCIENCE (0478): FULL SYLLABUS AND EXAM PREP

=====

Course link: <https://learn.modernagecoders.com/courses/igcse-computer-science-0478-course>

Subject area: IGCSE Computer Science

Designed for: Teens

Who can join: IGCSE students taking Computer Science 0478; taught from the basics, no prior coding required

Course length: 6 months (24 weeks), joinable any month

Time needed: 2 live classes/week + weekly programming practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online Cambridge IGCSE Computer Science 0478 classes covering both papers in full: computer systems, algorithms, programming in Python, databases and Boolean logic, with real exam technique.

In one line: Both papers taught in full, with real programming in Python, so your child understands the subject instead of memorising it.

ABOUT THE PROGRAMME

Cambridge IGCSE Computer Science is two subjects wearing one name. Half of it is how computers actually work, from binary and data transmission to hardware, networks and emerging technologies. The other half is thinking like a programmer: designing algorithms, writing and correcting real code, querying a database, and reasoning through logic circuits. Students who treat it as a memory subject stall on the second half, because you cannot memorise your way through writing a program you have never seen. This course teaches both halves properly. It follows the current 0478 syllabus (version 5, for examination in 2026, 2027 and 2028) topic by topic, and it is built around the fact that the qualification is assessed by two written papers of equal weight, with no coursework. Paper 1 covers computer systems. Paper 2 covers algorithms, programming and logic. Neither can be neglected, because each is worth exactly half the grade. The six-month structure moves in the syllabus's own order. Months 1 and 2 build computer systems from the ground up: data representation, data transmission and hardware. Months 3 and 4 finish the theory with software, the internet and its uses, and automated and emerging technologies, then add Boolean logic. Months 5 and 6 turn to the programming half: algorithm design, pseudocode, real Python, file handling, databases and

SQL, and then full timed exam practice on both papers. Every class is live and small, and every programming concept is practised by actually writing code, because that is the only part of this subject that cannot be faked.

WHAT MAKES THIS COURSE DIFFERENT

- Both papers taught in full and to equal depth, because each is worth exactly 50 percent of the grade and most students quietly neglect one of them
- Taught to the current syllabus (0478 version 5, for examination in 2026, 2027 and 2028), so nothing covered is out of date
- Real programming, not just pseudocode on paper: students write and run genuine Python, which is what makes Paper 2 straightforward instead of frightening
- The scenario question is prepared for directly: candidates may answer in pseudocode or in Python, Visual Basic or Java, and we drill the choice your child is strongest in
- Exam technique built in from the start: trace tables, dry runs, test data, and the difference between the marks for stating, applying and evaluating
- Live, small batches where your child writes code during class and gets it reviewed on the spot, not recorded lectures watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Computer systems foundations: data representation, data transmission and hardware, built from binary upward

Phase 2: Software, the internet and its uses, automated and emerging technologies, then Boolean logic and logic circuits

Phase 3: Algorithm design, programming in Python, databases and SQL, then full timed practice on both papers

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: COMPUTER SYSTEMS FOUNDATIONS (PAPER 1 CORE, MONTHS 1-2) -----

How a computer actually represents, moves and processes information, built from binary upward so nothing later has to be taken on faith.

Month 1: Data Representation and Data Transmission (Weeks 1-4)

Binary, Denary and Hexadecimal

Topics:

- Why computers use binary, and what a bit and a byte actually are
- Converting between denary, binary and hexadecimal in both directions
- Binary addition, and what overflow means and when it happens
- Logical binary shifts and their effect on a value
- Where hexadecimal shows up in practice: error codes, MAC addresses, colour codes

Projects built:

- A conversion drill sheet worked by hand, then a short Python script that checks your answers

Practice: 20 conversion and binary-addition questions at exam difficulty, self-marked against worked solutions

Representing Text, Images and Sound

Topics:

- Character sets: ASCII and Unicode, and why Unicode exists
- How an image is stored: pixels, resolution, colour depth, and the file-size calculation
- How sound is stored: sampling rate, sample resolution, and the trade-off with quality
- Calculating file sizes, a guaranteed source of marks
- Measurements of storage from bit up to terabyte

Projects built:

- Calculate the true file size of a given image and sound clip, then verify against a real file

Practice: 12 file-size and representation calculations, shown with full working

Assessment: A short written check on number systems and data representation, marked in exam style

Compression and Data Transmission

Topics:

- Lossy and lossless compression, and when each is appropriate
- How run-length encoding works, step by step
- Data packets: structure, packet switching, and why packets can arrive out of order
- Serial and parallel transmission, simplex, half-duplex and duplex
- Universal Serial Bus and where it fits

Projects built:

- Compress a sample string by run-length encoding by hand, then reverse it

Practice: 10 questions on compression and transmission methods, including a packet-structure diagram

Error Detection and Encryption

Topics:

- Why errors occur in transmission, and what that means for the receiver
- Parity checks (odd and even), checksums, echo check and check digits
- Automatic Repeat reQuest and how acknowledgement works
- Symmetric and asymmetric encryption explained plainly
- Why encryption protects data but does not prevent interception

Projects built:

- Work a parity check and a check-digit calculation by hand, then break a deliberately corrupted example

Practice: 12 error-detection questions including trace-style parity problems

Assessment: End-of-month test on data representation and transmission, scored on the exam mark scheme

Month 2: Hardware and Computer Architecture (Weeks 5-8)

The CPU and the Fetch-Decode-Execute Cycle

Topics:

- The von Neumann model and the role of the processor
- Registers, buses and their purpose in plain language
- The fetch, decode, execute cycle in order
- What actually affects CPU performance: clock speed, cores, cache
- Embedded systems and where they are used

Projects built:

- Trace one full fetch-decode-execute cycle for a simple instruction and explain each step in your own words

Practice: 10 architecture questions, including performance-comparison scenarios

Input, Output and Sensors

Topics:

- Common input devices and where each is genuinely appropriate
- Output devices, including printers and their differences
- Sensors, what each measures, and their use in monitoring and control systems
- Choosing the right device for a described scenario, which is how the exam asks it
- Justifying a choice, since the marks are for the reason, not the name

Projects built:

- Specify the input and output devices for a described real system and justify every choice

Practice: 10 scenario questions where the marks come from justification

Memory and Storage

Topics:

- RAM and ROM, and the difference that actually matters
- Virtual memory: what it is and why it is used
- Magnetic, optical and solid state storage, and how each physically works
- Comparing storage types on capacity, speed, portability and durability
- Cloud storage, with its genuine advantages and real drawbacks

Projects built:

- Build a comparison table of storage types and defend a recommendation for a given user

Practice: 10 memory and storage questions including comparison and justification

Network Hardware and the Internet Backbone

Topics:

- Network interface card, MAC address and IP address, and how they differ
- Routers and what they actually do
- IPv4 and IPv6, and why IPv6 exists
- Public and private IP addresses
- How this connects to data transmission from Month 1

Projects built:

- Map how a request travels from a home device to a web server and back, labelling each component

Practice: 10 networking questions at exam difficulty

Assessment: End-of-phase test covering all of hardware, scored on the mark scheme

PHASE 2: SOFTWARE, THE INTERNET, EMERGING TECHNOLOGIES AND LOGIC (MONTHS 3-4)

The rest of the Paper 1 theory, then Boolean logic, which is the bridge into the programming half of the course.

Month 3: Software and the Internet (Weeks 9-12)

System Software and the Operating System

Topics:

- System software versus application software, with clear examples
- What an operating system actually manages
- Interrupts: what triggers them and how they are handled
- The role of the bootstrap on start-up
- Utility software and what each utility is for

Projects built:

- Explain, in your own words, exactly what happens between pressing power and seeing the desktop

Practice: 10 software questions including interrupt scenarios

Programming Languages, Translators and the IDE

Topics:

- High-level and low-level languages, and the real trade-off
- Assembly language and why it still exists
- Compilers and interpreters, and the difference in how each reports errors
- What an Integrated Development Environment gives you
- Common IDE features and how they help you debug

Projects built:

- Run the same short program through an interpreter and observe how errors surface

Practice: 10 questions comparing translators, with justification marks

Assessment: A timed check on software, translators and the operating system

The Internet and Its Uses

Topics:

- The internet and the World Wide Web are not the same thing, and the exam tests this
- URLs, HTTP and HTTPS, and what the S actually secures
- Web browsers, cookies (session and persistent) and how pages are retrieved and displayed
- Digital currency and the idea of blockchain, at syllabus depth
- Why any of this matters for security

Projects built:

- Trace what happens from typing a URL to the page rendering, naming every step

Practice: 10 questions on web technologies and cookies

Cyber Security

Topics:

- The threats named in the syllabus: brute force, data interception, DDoS, hacking, malware, phishing, pharming, social engineering
- How each threat actually works, so the definitions stop blurring together
- The solutions: access levels, anti-malware, authentication, firewalls, privacy settings, proxy servers, SSL
- Matching the right solution to the right threat, which is how marks are awarded
- Two-factor and biometric authentication

Projects built:

- Build a threat-and-solution table, then answer a scenario recommending and justifying protections

Practice: 12 threat-matching and justification questions

Assessment: End-of-month test on the internet and cyber security

Month 4: Automated Systems, Emerging Technologies and Boolean Logic (Weeks 13-16)

Automated Systems and Robotics

Topics:

- How sensors, microprocessors and actuators work together in an automated system
- Worked examples across transport, agriculture, weather and lighting
- What characterises a robot, and where robotics is genuinely used
- The real advantages and drawbacks of automation
- Answering these as scenario questions rather than definitions

Projects built:

- Design an automated system for a described situation, naming every sensor and the microprocessor logic

Practice: 8 automated-system scenarios with full explanations

Artificial Intelligence at Syllabus Depth

Topics:

- What artificial intelligence means in this syllabus, precisely
- Narrow, general and strong AI as the syllabus frames them
- The basic characteristics of expert systems and their components
- Machine learning at the level the exam requires, without overreach
- The genuine limitations, so answers stay accurate

Projects built:

- Explain how an expert system reaches a conclusion, using its rule base and inference engine

Practice: 8 questions on AI and expert systems

Logic Gates and Truth Tables

Topics:

- The six gates: NOT, AND, OR, NAND, NOR and XOR
- The symbol, the function and the truth table for each
- Building a truth table correctly and completely, row by row
- Reading a logic statement and identifying the gates involved
- Why this topic is pure marks once the method is automatic

Projects built:

- Produce complete truth tables for all six gates from memory, then verify them

Practice: 12 truth-table exercises building to three-input problems

Logic Circuits and Logic Expressions

Topics:

- Drawing a logic circuit from a written scenario
- Writing the logic expression for a given circuit
- Completing a truth table for a multi-gate circuit without losing rows
- Working backwards from a truth table to the expression
- The common slips that cost marks in this topic

Projects built:

- Take a described safety system, draw its logic circuit, write the expression and complete the truth table

Practice: 10 full logic-circuit problems at exam standard

PHASE 3: ALGORITHMS, PROGRAMMING, DATABASES AND EXAM MASTERY (PAPER 2, MONTHS 5-6)

The programming half of the qualification, taught by actually writing code, then both papers drilled under timed conditions.

Month 5: Algorithm Design and Programming (Weeks 17-20)

Algorithm Design, Pseudocode and Flowcharts

Topics:

- The program development life cycle: analysis, design, coding, testing
- Decomposition and abstraction, and structure diagrams
- Writing pseudocode in the style the syllabus expects
- Drawing flowcharts with the correct symbols
- Converting freely between pseudocode and a flowchart

Projects built:

- Take one described problem and express its solution as a structure diagram, pseudocode and a flowchart

Practice: 10 algorithm-design tasks in both pseudocode and flowchart form

Programming Fundamentals in Python

Topics:

- Data types, variables and constants, and declaring them properly
- Input and output, and basic string handling
- Sequence, selection (IF and CASE) and iteration (FOR, WHILE, REPEAT)
- Nested statements and keeping them readable
- Totalling and counting, the two patterns that appear constantly

Projects built:

- Write a working Python program using selection and iteration to solve a real described task

Practice: 12 short programs written and run, each tested with your own test data

Arrays, Procedures, Functions and File Handling

Topics:

- One-dimensional and two-dimensional arrays, and traversing them with loops
- Standard methods: linear search, bubble sort, finding totals, maximum, minimum and average
- Procedures and functions, parameters, and why they matter
- Local and global variables
- Reading from and writing to a text file

Projects built:

- Build a program that reads records from a file into an array, searches and sorts it, and writes results back

Practice: 10 array and file-handling programs at exam difficulty

Validation, Verification, Test Data and Trace Tables

Topics:

- Validation checks: range, length, type, presence, format, check digit
- Verification, and how it differs from validation
- Choosing normal, abnormal and boundary test data deliberately
- Completing a trace table accurately, a classic source of marks
- Finding the error in someone else's code, which the exam asks directly

Projects built:

- Add full validation to an earlier program, then produce a trace table proving it works

Practice: 10 trace-table and test-data questions, plus 5 find-the-error tasks

Assessment: Timed programming assessment: write, validate and trace a full solution

Month 6: Databases, SQL and Full Exam Practice (Weeks 21-24)

Databases and SQL

Topics:

- Single-table databases: records, fields and the primary key
- Choosing appropriate data types for fields
- SQL: SELECT, FROM, WHERE, ORDER BY
- SUM and COUNT, and when each is wanted
- Reading a described scenario and writing the query it asks for

Projects built:

- Design a single-table database for a described scenario and write the queries that answer real questions about it

Practice: 12 SQL questions building from single conditions to sorted, aggregated queries

Paper 1 Technique

Topics:

- How the theory paper is structured and how the marks are distributed
- Reading the command word: state, describe, explain, compare, justify
- Why a one-word answer loses marks on an explain question
- Managing time so the longer questions are not rushed
- The topics that recur every single year

Projects built:

- A full timed Paper 1 attempt, then a question-by-question review with every miss tagged by cause

Practice: Two timed Paper 1 sets, fully reviewed

Paper 2 Technique and the Scenario Question

Topics:

- How the programming paper is structured, including the extended scenario question
- Choosing your answer language deliberately: pseudocode, Python, Visual Basic or Java
- Structuring a long program answer so partial marks are never lost
- Writing readable code by hand, under time, with comments that earn credit
- Checking a written program for the small errors that cost marks

Projects built:

- A full timed Paper 2 attempt including the extended scenario question, marked on the official scheme

Practice: Timed Paper 2 sets across algorithm, array, file and database question types

Full Mock Exams and Final Polishing

Topics:

- Full-length timed mocks on both papers under real conditions
- Turning the error log from every mock into a focused revision plan
- Locking in the recurring high-mark topics so they are automatic
- A calm, tested routine for both exam days
- An honest read on the target grade and the final push to reach it

Projects built:

- Two complete mock examinations, both papers, each fully reviewed and converted into a revision list

Practice: Repeated full mocks until pacing and accuracy are consistent

Assessment: Final full mock on both papers, scored and benchmarked against the target grade

WHERE THIS CAN LEAD

- A strong IGCSE Computer Science grade across both papers
- A confident move into A Level Computer Science or IB Computer Science
- Genuine Python ability, the most transferable skill in the syllabus
- A real foundation for a computer science degree later
- Computational thinking that transfers into maths, physics and engineering

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course aligned to the current IGCSE Computer Science syllabus?

A: Yes. It follows Cambridge IGCSE Computer Science 0478, version 5, for examination in 2026, 2027 and 2028. It covers both assessed components in full: Paper 1 on computer systems, and Paper 2 on algorithms, programming and logic. There is no coursework in this qualification, so the two papers are the entire grade.

Q: Which programming language do you teach for Paper 2?

A: We teach Python, because it is the clearest to read and the most widely used beyond the exam. In the extended scenario question candidates may answer in pseudocode or in Python, Visual Basic or Java, so we make sure your child is fluent in pseudocode too and confident in whichever form they choose to answer in.

Q: My child is fine at the theory but panics on the programming paper. Can this help?

A: That is the single most common pattern in this subject, and it is exactly what the course is built to fix. Paper 2 is worth half the grade and cannot be memorised. We teach programming by having your child write and run real code every week, then drill trace tables, test data and the extended scenario question until writing a solution under time feels ordinary.

Q: Do you cover Boolean logic and databases properly?

A: Yes, both in full. Boolean logic gets two dedicated weeks covering all six gates, truth tables, drawing circuits from a scenario and writing logic expressions. Databases get a full week on single-table design and SQL, including SELECT, FROM, WHERE, ORDER BY, SUM and COUNT. Both are reliable marks once the method is automatic.

Q: How long is the course and when can we join?

A: It runs six months across 24 weeks, with two live classes a week, and you can join in any month. Students already partway through their IGCSE course are placed at the right point after the free demo class, and we fill any gaps from the earlier topics as we go.

Q: Will this prepare my child for A Level or IB Computer Science?

A: Yes. The programming, algorithm design and computer-systems foundations here are exactly what A Level Computer Science and IB Computer Science build on. A student who genuinely understands 0478, rather than having memorised it, starts the next qualification ahead instead of behind.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/igcse-computer-science-0478-course>

Course link: <https://learn.modernagecoders.com/courses/igcse-mathematics-mastery>
Subject area: Exam Board Mathematics
Designed for: High school
Who can join: IGCSE students, years 9-11: Core, Extended and Additional Maths
Course length: 18 months (72 weeks)
Time needed: 2 live one-hour classes per week + weekly past-paper work
Certificate: Course completion certificate + documented paper-score progression

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online IGCSE Maths course for Cambridge Core, Extended and Additional Mathematics:
non-calculator fluency and past-paper technique on real mark schemes.

In one line: Core, Extended or Additional, taught to the top grade, syllabus-exact.

ABOUT THE PROGRAMME

An 18-month live program for IGCSE mathematics, matched to the student's exact board and entry: Cambridge 0580/0980 at Core or Extended tier, Edexcel International GCSE, and Additional Mathematics 0606 for strong students. The tier decision caps or opens grades before a single answer is written, and Cambridge's updated syllabus put a non-calculator paper in front of every candidate, so this course trains bare-hands fluency deliberately, teaches the Extended reach early enough to settle, and converts understanding into marks through past-paper cycles against real mark schemes.

WHAT MAKES THIS COURSE DIFFERENT

- Syllabus-exact: the mentor confirms your school's board and codes first, and teaches to them, 0580/0980, Edexcel 4MA1, or 0606
- Non-calculator fluency trained in every class, because the newer papers examine it and calculator-raised students get caught
- Tier-honest: evidence for the Core-vs-Extended decision, and a real runway from Core-risk to Extended-comfortable
- Additional Maths 0606 taught as the serious second subject it is, the best A-Level preparation on the syllabus list
- Mark-scheme literacy: method marks, follow-through and working discipline taught explicitly
- Understanding-first teaching so mocks stop sliding as content deepens
- One dedicated mentor across the whole IGCSE run
- One full interactive hour per class, twice a week

THE LEARNING PATH, STEP BY STEP

Phase 1: The Core foundation built properly: number, algebra as a language, and calculator-free fluency as a habit

Phase 2: The Extended reach with settling time: functions, trigonometry, vectors and probability at full depth

Phase 3: Additional Maths 0606 for the strong (or Extended consolidation), then past-paper craft into the exam series

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Years 9-11 at an IGCSE school (Cambridge or Edexcel), or equivalent homeschool candidates

Math background: Any, the diagnostic finds the real base; Core-level gaps are repaired before Extended reach

Equipment: Computer with stable internet; a scientific calculator for the calculator papers

Time commitment: 2 live hours plus 1-2 practice hours weekly

English: Course taught in English

Motivation: An exam series on the calendar; we supply the method

WHO THIS COURSE IS FOR

Extended aimers: Students who need the move from Core-risk to Extended-comfortable made real

Top grade hunters: Extended students converting understanding into A*-level marks through paper craft

Add maths: Strong students taking 0606 as their A-Level insurance

Non calc worriers: Calculator-raised students facing the newer bare-hands papers

International families: Families who move countries and need maths that moves with them

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE CORE FOUNDATION, BUILT PROPERLY (MONTHS 1-6)

Everything on the Extended papers stands on Core material owned deeply: number fluency without a calculator, algebra as a language rather than ritual, and the geometry and data basics. The diagnostic finds the real starting point; this phase makes it solid.

Months 1-2: Diagnostic + Number, Bare-Handed (Weeks 1-8)

Diagnostic, Syllabus Code and Tier Fit

Topics:

- Board and syllabus code confirmed: 0580, 0980 or 4MA1
- Core versus Extended tier fit assessed honestly
- Non-calculator fluency screen
- Algebra base measured
- Number and ratio accuracy audit
- Honest starting-grade picture

Practice: A mixed non-calculator and algebra diagnostic set matched to the suspected tier.

Assessment: Baseline grade estimate with a written Core-or-Extended recommendation and a topic-priority map.

Detail: Full diagnostic: board and syllabus code confirmed, tier fit assessed honestly, non-calculator fluency and algebra base measured

Number Fluency Rebuilt

Topics:

- The four operations with fractions
- Decimal calculations without a calculator
- Percentage of, increase, decrease and reverse
- Converting between fractions, decimals and percentages
- Estimation and rounding as a check
- Non-calculator paper territory

Practice: A non-calculator number set with estimation checks on every answer.

Detail: Number fluency rebuilt: fractions, decimals, percentages and estimation as a checking habit, the non-calculator paper's exact territory

Ratio, Proportion and Standard Form

Topics:

- Sharing in a given ratio
- Direct and inverse proportion
- Rates of change in context
- Standard form and its arithmetic
- Standard form at real scale
- Unit conversions

Practice: Ratio, proportion and standard-form questions in applied contexts.

Detail: Ratio, proportion and rates of change; standard form with real scale

Indices, Surds, Bounds and Accuracy

Topics:

- Index laws with negative and fractional powers
- Surds and simplifying (Extended)
- Rationalising and surd arithmetic
- Upper and lower bounds
- Accuracy and significant figures
- Bounds applied in calculations

Practice: An indices, surds and bounds set including accuracy questions.

Detail: Indices and surds from meaning; bounds and accuracy, the quietly examined skills

Months 3-4: Algebra as a Language (Weeks 9-16)

Expressions, Expansion and Factorisation

Topics:

- Simplifying and collecting like terms
- Expanding single and double brackets
- The area model behind expansion
- Factorising into brackets
- Common-factor extraction
- Algebraic notation discipline

Practice: An expand-and-factorise set anchored by area-model diagrams.

Detail: Expressions, expansion and factorisation with the area model underneath

Linear Equations, Inequalities and Formulae

Topics:

- Solving linear equations step by step
- Equations with brackets and fractions
- Solving and representing inequalities
- Number-line representation of solutions
- Changing the subject of a formula
- Reading an equation as a sentence

Practice: Linear equation, inequality and rearrangement questions with worded contexts.

Detail: Linear equations and inequalities read as sentences; changing the subject of formulae

Straight-Line Graphs as One Story

Topics:

- Plotting straight-line graphs
- Gradient understood as a rate

- Intercepts as starting values
- Equation, table and graph as one story
- Finding the equation of a line
- Parallel and perpendicular lines

Practice: Straight-line graph questions linking equation, table and graph.

Detail: Straight-line graphs: gradient as rate, intercepts as starting values, equation, table and graph as one story

Simultaneous Equations and Sequences

Topics:

- Simultaneous equations by elimination
- Simultaneous equations by substitution
- Choosing a method by structure
- Linear and simple non-linear sequences
- Finding and using the nth term rule
- Pattern recognition

Practice: A simultaneous-equations and nth-term set graded by structure.

Detail: Simultaneous equations by structure; sequences and the nth term

Months 5-6: Shape, Space and Data Foundations (Weeks 17-24)

Angle Reasoning and Justification

Topics:

- Angles on lines and around points
- Parallel-line angle rules
- Interior and exterior angles of polygons
- Basic circle angle facts
- Stating a reason for every step
- Multi-step angle chains

Practice: Angle problems requiring a named reason for each step.

Detail: Angle reasoning: parallel lines, polygons and circle basics with justification language

Perimeter, Area, Volume and Pythagoras

Topics:

- Perimeter and area of standard shapes
- Compound and composite shapes
- Volume and surface area of solids
- Pythagoras derived from a picture
- Applying Pythagoras in context
- Units for length, area and volume

Practice: A mensuration set combining compound shapes and Pythagoras problems.

Detail: Perimeter, area and volume including compound shapes; Pythagoras from a picture

Statistics Foundations

Topics:

- Mean, median, mode and range
- Averages from frequency tables
- Bar charts, pie charts and pictograms
- Scatter diagrams and correlation
- Lines of best fit
- Reading data critically

Practice: A statistics set interpreting charts and a scatter diagram.

Detail: Statistics foundations: averages, charts and scatter diagrams read critically

Phase 1 Core-Tier Checkpoint

Topics:

- Full Core-tier paper under timing
- Mark-scheme review of every question
- The review ritual established
- Method-mark awareness introduced
- Error log started from the checkpoint
- Timing across a full paper

Practice: A complete Core-tier paper sat to time, then reviewed against the mark scheme.

Assessment: Core-tier checkpoint paper marked to the mark scheme, establishing the review ritual and the error log.

Detail: Phase checkpoint: a full Core-tier paper under timing, reviewed against the mark scheme, the review ritual for the rest of the course

----- PHASE 2: THE EXTENDED REACH, WITH SETTLING TIME (MONTHS 7-12) -----

The topics that separate Extended from Core, and top grades from safe passes, taught months before the exam so they are owned, not crammed: functions, trigonometry beyond right angles, vectors, and probability at full depth.

Months 7-8: Quadratics and Functions (Weeks 25-32)

Solving Quadratics Three Ways

Topics:

- Solving by factorising
- Completing the square
- The quadratic formula
- Choosing the method by structure

- Roots and their meaning
- Extended-tier quadratic manipulation

Practice: A quadratics set requiring choice of factorising, completing the square or the formula.

Detail: Quadratic equations by factorising, completing the square and the formula, chosen by structure

Quadratic Graphs and Linear-Quadratic Systems

Topics:

- Sketching quadratic graphs
- Roots, intercepts and symmetry
- Quadratics in real contexts
- Simultaneous linear-quadratic systems
- Solving graphically and algebraically
- Interpreting the parabola

Practice: Quadratic-graph questions plus simultaneous linear-quadratic systems.

Detail: Quadratic graphs and real contexts; simultaneous linear-quadratic systems

Functions: Composite and Inverse

Topics:

- Function notation and evaluation
- Composite functions and order
- Inverse functions and how to find them
- Domain and range awareness
- Common function-mark traps
- Functions read as meaning

Practice: A functions set focused on composite and inverse questions.

Detail: Function notation, composite and inverse functions, read as meaning, where half of all function marks are lost

Graph Transformations and Graph Literacy

Topics:

- Translations and reflections of graphs
- Recognising graphs from equations
- Cubic, reciprocal and exponential shapes
- Reading features from a graph
- Graph literacy the top questions assume
- Sketching from key points

Practice: Graph-transformation and graph-reading questions at Extended standard.

Detail: Graph transformations and the graph-literacy the top questions assume

Months 9-10: Trigonometry and Vectors at Full Depth (Weeks 33-40)

Trigonometry: Right Triangles to Sine and Cosine Rules

Topics:

- Sine, cosine and tangent ratios
- Finding sides and angles in right triangles
- The sine rule
- The cosine rule
- Area of a triangle with the sine formula
- Bearings and applications

Practice: A trig set moving from right triangles to sine and cosine rule problems.

Detail: Right-triangle trig consolidated, then the sine and cosine rules with bearing and area applications

Trig Graphs and Equations

Topics:

- Sine, cosine and tangent graphs
- Features and periodicity of trig graphs
- Solving simple trig equations
- Reading solutions from a graph
- Extended-reach trig contexts
- Exact values where required

Practice: Trig-graph and simple trig-equation questions at Extended reach.

Detail: Trig graphs and solving simple trig equations (Extended reach)

Vectors and Geometric Proof

Topics:

- Vector notation and column vectors
- Adding and scaling vectors
- Position vectors and paths
- Geometry proofs with vectors
- The classic Extended vector question
- Proving collinear or parallel points

Practice: Vector questions building to the classic Extended vector proof.

Detail: Vectors: notation, geometry proofs and the classic Extended vector question, defanged

Circle Theorems

Topics:

- The main circle theorems
- Angle at the centre and at the circumference
- Cyclic quadrilaterals and tangents
- The alternate segment theorem
- Proof-quality justification language
- Multi-theorem angle chains

Practice: Circle-theorem problems requiring a named theorem for each step.
Detail: Circle theorems completed with proof-quality justification

Months 11-12: Probability, Data and the Extended Checkpoint (Weeks 41-48)

Probability and Tree Diagrams

Topics:

- Probability of single and combined events
- Tree diagrams for dependent events
- Conditional reasoning
- With and without replacement
- Set notation and Venn diagrams
- Avoiding formula-panic

Practice: A probability set using tree diagrams and conditional reasoning.

Detail: Probability with tree diagrams and conditional reasoning; combined events without formula-panic

Cumulative Frequency and Histograms

Topics:

- Cumulative frequency curves
- Median, quartiles and interquartile range
- Histograms with unequal class widths
- Frequency density
- Box plots and comparing distributions
- Interpreting data like a statistician

Practice: A set drawing and reading cumulative frequency curves and histograms.

Detail: Cumulative frequency, histograms and comparing distributions like a statistician

Introductory Differentiation (Extended)

Topics:

- Differentiation as the gradient of a curve
- Differentiating simple polynomials
- Gradient at a point
- Turning points from the derivative
- The 0580 Extended calculus taste
- Interpreting a rate of change

Practice: Introductory differentiation questions finding gradients and turning points.

Detail: Differentiation introduced (0580 Extended's calculus taste): gradients of curves and turning points

Phase 2 Extended-Tier Checkpoint

Topics:

- Full Extended-tier past-paper cycle
- Mark-scheme review of long questions
- Method-mark and follow-through practice
- Error-log update from the cycle
- Timing across an Extended paper
- Weak-topic list refreshed

Practice: A full Extended-tier past paper sat to time and reviewed against the mark scheme.

Assessment: Extended-tier past-paper cycle marked to the mark scheme, updating the error log and weak-topic list.

Detail: Phase checkpoint: full Extended-tier past paper cycle with mark-scheme review and error-log update

PHASE 3: ADDITIONAL MATHS 0606 + EXAM CRAFT (MONTHS 13-18)

Strong students extend into Additional Mathematics, functions, identities, vectors and introductory calculus taught properly. All students then enter the exam machine: past-paper cycles by exact board and series, non-calculator discipline, and mark-scheme craft that peaks in the exam window.

Months 13-15: Additional Mathematics 0606 (or Extended Consolidation) (Weeks 49-60)

0606 Functions and Algebra Foundations

Topics:

- 0606 functions at depth
- Quadratics revisited with rigour
- Indices and surds
- Simultaneous equations for 0606
- The A-Level trailer begins
- Parallel Extended consolidation by error-log priority

Practice: 0606 function and algebra problems, with Extended students on a parallel consolidation set.

Detail: 0606: functions at depth, quadratics revisited with rigour, indices and surds, simultaneous equations, the A-Level trailer begins. (Parallel track: Extended students consolidate by error-log priority)

0606 Binomial, Trig Identities and Logs

Topics:

- Binomial expansion
- Trigonometric identities
- Solving trigonometric equations

- Logarithmic functions and laws
- Exponential functions
- Modelling with exponentials and logs

Practice: A 0606 set on binomial expansion, trig identities and log-exponential functions.

Detail: 0606: binomial expansion, trigonometric identities and equations, logarithmic and exponential functions

0606 Calculus, Vectors and Kinematics

Topics:

- Differentiation rules and applications
- Integration as the reverse of differentiation
- Areas and definite integrals
- Applications to rates and optimisation
- Vectors at 0606 depth
- Kinematics of motion

Practice: A 0606 calculus set with differentiation, integration and a kinematics problem.

Detail: 0606: introductory calculus, differentiation and integration with applications; vectors and kinematics

Months 16-17: Past-Paper Craft by Board and Series (Weeks 61-68)

Board-Exact Paper Cycles

Topics:

- Paper cycles from the exact board and series
- Non-calculator papers under bare-hands discipline
- Calculator papers with method-mark craft
- Tier-correct paper selection
- Presenting working for marks
- Board-specific command words

Practice: Board-exact non-calculator and calculator papers sat to time and reviewed.

Detail: Paper cycles from your exact board: non-calculator papers under bare-hands discipline, calculator papers with method-mark craft

Timing Plans and Point-Leak Habits

Topics:

- Personal timing plans from paper data
- Per-question time budgets
- Multi-step question chains rehearsed
- The plus-minus sign habit
- Units and final-answer discipline
- Stopping avoidable point leaks

Practice: Timed multi-step question sets drilling the plus-minus and units habits.

Detail: Timing plans from personal data; multi-step question chains rehearsed; the plus-minus and units habits that stop point leaks

Month 18: The Exam Series, Met Calmly (Weeks 69-72)

Final Cycles and Error-Log Closure

Topics:

- Final full paper cycles
- Documented score progression
- Error-log closure
- Re-testing previously fixed errors
- Confidence on weak topics
- Consistency across papers

Practice: Final full paper cycles with progression charted and closed errors re-tested.

Detail: Final full cycles with documented score progression; error-log closure

Peak-Week Plan and Exam-Day Routines

Topics:

- Light consolidation in the peak week
- Rest before the series
- Exam-day routines rehearsed
- Calm pacing under real conditions
- Final confidence work
- The series met as a known place

Practice: Light consolidation tasks and a rehearsed exam-day routine before the series.

Assessment: Final full-cycle rehearsal with documented score progression before the exam series.

Detail: Peak-week plan: light consolidations, rest, and exam-day routines rehearsed, the series met as a known place

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal error log and syllabus map across all 18 months
- Non-calculator fluency system, built through daily-dose drills
- Self-graded past-paper portfolio against real mark schemes
- 0606 formula-and-technique handbook (Additional Maths students)
- Full timed paper cycles with documented score progression

Total projects: 5+ exam artifacts plus the student's own handbooks

SKILLS THE STUDENT WILL HAVE AT THE END

- Calculator-free number fluency
- Algebra as a language: equations, formulae, graphs
- Extended-tier functions, trigonometry and vectors
- Probability and statistics at exam depth
- Introductory calculus (Extended and 0606)
- Mark-scheme literacy and working discipline
- Past-paper timing craft
- 0606 pre-A-Level maturity (strong students)

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor
Past paper work: Weekly question sets from your exact board
Fluency: Non-calculator warm-up in every class
Total per week: About 4 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole IGCSE run
Recordings: Every class recorded for revision
Progress tracking: Paper-score progression shared with parents
Flexibility: Intensity adjusts around school mocks and the exam series

CERTIFICATION DETAIL

Final certificate: Course completion certificate from Modern Age Coders
Score record: Documented paper-score progression
Portfolio projects: Past-paper portfolio and personal handbooks

WHERE THIS CAN LEAD

- An IGCSE grade that opens sixth-form and A-Level doors
- A-Level Mathematics readiness, especially via 0606
- IB Diploma readiness for students switching systems
- Ready for the IB Mathematics Masterclass or competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which boards and syllabuses does this course cover?

A: The first thing the mentor confirms is the exact code and tier your school uses, and all past-paper work comes from that board and series.

Q: Core or Extended, how should we decide?

A: By destination, then evidence. Core caps the achievable grade, so if A-Level maths, sciences or a competitive sixth form is even possible, Extended is the default. Whether the gaps blocking Extended are repairable in the time available is a diagnostic question, and the free demo answers it with evidence you can take to school.

Q: What about the non-calculator papers?

A: Cambridge's updated 0580/0980 syllabus put a non-calculator paper in front of every candidate at both tiers. We train that fluency deliberately, fractions, percentages and estimation drills open every class, and verify current paper structures against your school's syllabus code and series.

Q: Should my child take Additional Maths 0606?

A: If they are comfortable in Extended and A-Level maths is likely: strongly consider it. 0606 is the best A-Level preparation on the syllabus list, functions, identities, vectors and introductory calculus taught properly. It is a genuine second subject that rewards starting early, which is exactly how this course sequences it.

Q: My child does endless past papers but the score is stuck. Will more classes help?

A: More papers will not, practice repeats what the student already does, errors included. A stuck score almost always means an understanding gap upstream, usually algebra as memorized rules. We find the gap, fix it at the root, and then past papers start converting effort into marks again.

Q: Can this course rescue a threatened Core entry?

A: Often, yes. Schools respond to evidence, and entry deadlines usually leave more room than parents fear. A focused term of understanding-first teaching plus improving test scores is the argument that works, and we are honest in both directions when Core genuinely is the kind choice.

Q: Can we try it before paying?

A: Yes. Every student starts with a free live demo class that doubles as the diagnostic,

board, tier, fluency and algebra base, with no card details required.

To enrol or to book a free demo for this course, message Shivam Sir on
+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/igcse-mathematics-mastery>

=====

COURSE 76 OF 118: INFORMATICS PRACTICES (IP) CLASS 11-12: CBSE PYTHON & SQL

=====

Course link: <https://learn.modernagecoders.com/courses/cbse-informatics-practices-ip-class-11-12-python-pandas-sql-complete-course>

Subject area: School Board Exam Preparation - Informatics Practices

Designed for: Teens

Who can join: Class 11 Beginner to Class 12 Board Exam Ready

Course length: 24 months (Class 11 + Class 12 full syllabus)

Time needed: 6-8 hours/week (3-4 classes + self-practice)

Certificate: Course Completion Certificate + Board Exam Readiness Guarantee

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

CBSE Informatics Practices course for Class 11 & 12: master Python, Pandas, NumPy, Matplotlib and MySQL, with practical file and viva preparation.

In one line: 100% CBSE IP Syllabus · Pandas & Matplotlib Mastery · SQL Expert · Board Topper Strategy · Live Mentoring

ABOUT THE PROGRAMME

Informatics Practices (IP, Code 065) is the perfect subject for students interested in data science, business analytics, and technology applications. Unlike Computer Science (Code 083) which dives deep into programming logic and data structures, IP focuses on practical data handling using Python's Pandas library, stunning data visualization with Matplotlib, and powerful SQL database querying. This 2-year course covers the complete CBSE IP syllabus for Class 11 and Class 12, from the very basics of Python programming and SQL to advanced DataFrame operations, pivot tables, data visualization charts, GROUP BY/HAVING queries, table joins, computer networks, and societal impacts of technology. Our IP batch is completely separate from CS (083) and ICSE batches. Every class, assignment, test, and mock exam is specifically designed for the IP syllabus and CBSE exam pattern. We follow Preeti Arora (most popular IP textbook), Sumita Arora, and NCERT Informatics Practices textbooks chapter-by-chapter. Students get complete practical file preparation (15+ Pandas programs, 4+ Matplotlib visualizations, 15+ SQL queries), guided project work, viva preparation, and intensive board exam revision with 10+ mock papers. Our goal: every student scores 90+ in IP board exams.

WHAT MAKES THIS COURSE DIFFERENT

- Dedicated CBSE IP batch, not mixed with CS or ICSE students
- 100% CBSE Code 065 syllabus coverage, latest 2026-27 curriculum
- Chapter-wise teaching matching Preeti Arora, Sumita Arora & NCERT IP textbooks
- Hands-on Pandas practice with real datasets, not just theory
- Matplotlib visualization projects with real-world data
- Live MySQL database for SQL practice, write and execute queries in class
- Complete practical file: 15+ Pandas programs + 4+ Matplotlib charts + 15+ SQL queries
- Board exam project guidance with documentation template
- Weekly chapter tests + monthly unit tests + 10+ full mock board exams
- Previous 10 years solved papers with marking scheme analysis
- Dedicated doubt-clearing sessions before exams
- Assertion-Reason, Case Study, and MCQ-specific preparation
- Data science career orientation, understand where IP skills lead

THE LEARNING PATH, STEP BY STEP

Phase 1: Class 11, Semester 1 (Months 1-5): Computer fundamentals, Python basics (variables, operators, control flow, lists, dictionaries), SQL fundamentals

Phase 2: Class 11, Semester 2 (Months 6-10): Advanced SQL (DQL), Emerging Trends (AI, IoT, Cloud), practical file, Class 11 exam prep

Phase 3: Class 12, Semester 1 (Months 11-17): Pandas Series & DataFrame, data visualization with Matplotlib, CSV import/export

Phase 4: Class 12, Semester 2 (Months 18-24): Advanced SQL (functions, GROUP BY, joins), computer networks, societal impacts, board exam prep

WHAT A STUDENT NEEDS BEFORE STARTING

Class level: Students entering Class 11 or Class 12 with Informatics Practices (CBSE code 065); Class 10 students who have chosen IP can join early

Prior knowledge: No programming or database experience required - Class 11 starts from zero;

Class 12 joiners get a bridge revision of Class 11 Python and SQL
Equipment: A laptop or desktop with a stable internet connection; we guide installation of Python, Pandas, Matplotlib, and MySQL in the first sessions
Time commitment: 6-8 hours per week including live classes and self-practice
Textbooks: Preeti Arora, Sumita Arora, or NCERT IP - classes align with whichever textbook the student's school follows

WHO THIS COURSE IS FOR

Class 11 ip students: Students starting IP in Class 11 who want clear fundamentals in Python and SQL from the first chapter
Class 12 board aspirants: Students facing boards in the 2026-27 session who need complete Code 065 coverage plus intensive exam practice
Students confused by pandas: Students who find Series, DataFrames, loc and iloc, and Boolean indexing confusing and want them taught step by step with real data
Sql strugglers: Students who can read queries but cannot yet independently write GROUP BY, HAVING, or join queries
Practical file and project help: Students who need structured help completing the Pandas, Matplotlib, and SQL practical file and board project on time
Future data analysts: Students interested in data science, analytics, or commerce-plus-technology careers after school

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CLASS 11 SEMESTER 1, COMPUTER BASICS, PYTHON PROGRAMMING & SQL INTRODUCTION (MONTHS 1-5)

Build your foundation in computer science concepts, learn Python programming from scratch, and start with SQL database queries. By the end of this phase, you'll write Python programs confidently and run basic SQL queries on MySQL.

Month 1: Introduction to Computer Systems (Week 1-4)

Evolution of Computing & Computer Hardware

Topics:

- Evolution of computing devices, from Abacus to modern AI-powered computers
- Generations of computers: First (vacuum tubes) to Fifth (AI)
- Computer system components: Input Unit, CPU, Memory, Output Unit
- Block diagram of a computer, data flow from input to output
- CPU components: ALU (Arithmetic Logic Unit), Control Unit (CU), Registers
- Types of processors: single core, dual core, quad core, octa core
- Clock speed and its effect on processing: GHz explained
- Input devices: keyboard, mouse, scanner, webcam, microphone, touchscreen, stylus
- Biometric devices: fingerprint scanner, iris scanner, face recognition
- Output devices: monitor (CRT, LCD, LED, OLED, AMOLED), printer (inkjet, laser, 3D), speaker, projector, plotter
- Combined devices: touchscreen (both input and output)
- How a computer processes data: Input Process Output cycle

Projects built:

- Computer system diagram with all components labeled
- Input-output devices classification chart with 20+ devices
- Research project: Latest processor technologies in 2026-27

Practice: NCERT IP Chapter 1 questions, Preeti Arora exercises, 30 MCQs on computer fundamentals

Memory, Storage & Software

Topics:

- Computer memory: why it's needed, role in processing
- Primary memory (volatile): RAM, SRAM vs DRAM, speed, cost
- ROM, PROM, EPROM, EEPROM, non-volatile, stores firmware
- Cache memory: L1, L2, L3, speeds up CPU access to frequently used data
- Memory hierarchy: Registers Cache RAM SSD HDD (speed vs capacity trade-off)
- Units of memory: bit, nibble (4 bits), byte (8 bits), KB, MB, GB, TB, PB, EB
- Secondary storage: HDD (magnetic), SSD (flash), USB drive, memory card
- Optical storage: CD (700MB), DVD (4.7GB), Blu-ray (25-50GB)
- Cloud storage: Google Drive, OneDrive, iCloud, advantages and concerns
- Data deletion and recovery: when you delete, data isn't immediately gone
- Data security concerns: unauthorized access, data corruption, backup importance
- Software: definition, need for software, software vs hardware
- System software: OS, language processors, device drivers, utility programs
- Application software: general purpose (MS Office, browser) vs specific purpose (Tally, AutoCAD)
- Open source software: Linux, LibreOffice, GIMP, Python, MySQL
- Proprietary software: Windows, MS Office, Adobe Photoshop
- Operating system: functions, process management, memory management, file management, device management, user interface
- Types of user interface: CLI (Command Line) vs GUI (Graphical)

Projects built:

- Memory hierarchy pyramid diagram with specifications

- Software classification chart, 15+ examples each category
- Compare 5 open-source vs proprietary software alternatives

Practice: Preeti Arora Chapter 1 exercises, 50 MCQs, fill-in-the-blanks, true/false questions

Months 2-3: Introduction to Python Programming (Week 5-12)

Getting Started with Python, Setup & First Programs

Topics:

- Why Python for IP?, simple syntax, data analysis libraries, industry standard
- Python in the real world: Instagram, YouTube, Spotify, Netflix use Python
- Installing Python 3.12+ on Windows, downloading from python.org
- Python IDLE: Interactive mode (>>> prompt) vs Script mode (.py files)
- Difference between Interactive and Script mode, when to use which
- Setting up VS Code with Python extension (optional, IDLE is sufficient for IP)
- First program: print('Hello, World!'), both in interactive and script mode
- Saving and running .py files
- Python tokens: keywords, identifiers, literals, operators, punctuators/delimiters
- Python keywords: 35 reserved words, False, True, None, and, or, not, if, else, elif, for, while, def, return, import, etc.
- Identifiers: rules, must start with letter or underscore, case-sensitive, no keywords, no spaces
- Valid vs invalid identifiers, practice exercises

Projects built:

- 10 print() variations, different data types, multiple arguments
- Interactive mode exploration, test 20 expressions
- Valid/invalid identifier quiz, 30 examples

Practice: Preeti Arora Python introduction exercises, 20 MCQs, output prediction for print statements

Variables, Data Types & Input/Output

Topics:

- Variables: named storage locations in memory
- Variable assignment: x = 10, name = 'Riya'
- Dynamic typing: Python infers data type automatically
- Multiple assignment: a, b, c = 1, 2, 3 and a = b = c = 0
- Data types in Python:
- int, integers: 10, -5, 0, 1000000 (unlimited precision)
- float, decimal numbers: 3.14, -0.5, 2.0
- complex, complex numbers: 3+4j (not in board exams but good to know)
- str, strings: 'hello', "world", '''multi-line'''
- bool, boolean: True, False (capital T and F!)
- None, absence of value: x = None
- type() function: checking data type of any variable
- Mutable vs Immutable types: lists are mutable, strings/tuples/int/float are immutable, IMPORTANT CONCEPT
- Type conversion (casting): int(), float(), str(), bool()
- Implicit type conversion: int + float float (automatic)
- Explicit type conversion: int('25'), float('3.14'), str(100)
- Input function: input(), ALWAYS returns a string
- Converting input: int(input()), float(input())
- print() function: sep parameter, end parameter
- Formatted output: f-strings, f'Hello {name}, age {age}'
- Comments: # single line, '''multi-line docstrings'''
- Debugging basics: reading error messages, common beginner mistakes

Projects built:

- Student information collector, input name, age, class, marks, display formatted
- Simple interest calculator, P, R, T as input
- BMI calculator with formatted output
- Swap two variables, 3 methods: temp, arithmetic, Pythonic a,b = b,a

Practice: 40 output prediction MCQs, 15 type conversion exercises, Preeti Arora data handling exercises

Operators & Expressions

Topics:

- Arithmetic operators: + (add), - (subtract), * (multiply), / (true division), // (floor division), % (modulus), ** (exponent)
- / vs //, 7/2 gives 3.5 (float), 7//2 gives 3 (integer), EXAM FAVORITE
- Modulus operator: 17%5 = 2 (remainder), -17%5 = 3 (Python's floor modulus!)
- Exponent: 2**10 = 1024, 9**0.5 = 3.0 (square root)
- Relational/comparison operators: == (equals), != (not equals), <, >, <=, >=
- Comparing strings: lexicographic order based on ASCII/Unicode values
- Logical operators: and, or, not
- Truth tables for and, or, not
- Short-circuit evaluation: False and X False (X not evaluated), True or X True
- Assignment operators: =, +=, -=, *=, /=, //=, %=, **=
- Identity operators: is, is not, checks if same object in memory (not just equal)
- Membership operators: in, not in, checks presence in sequence
- 'a' in 'apple' True, 5 in [1,2,3,4,5] True
- Operator precedence (PEMDAS for Python):
- ** unary +x -x ~x * / // % + - == != < > <= >= not and or
- Associativity: most operators left-to-right, ** is right-to-left

- Expression evaluation: step-by-step breakdown practice
- Common exam traps: precedence gotchas, integer vs float division

Projects built:

- Expression evaluator worksheet, 30 complex expressions with step-by-step solutions
- Electricity bill calculator with slab rates
- Operator precedence quiz, 20 tricky questions
- Truth table generator for and, or, not combinations

Practice: 50 expression evaluation problems, 30 operator MCQs, Preeti Arora operator exercises

Control Flow, Conditional Statements & Loops

Topics:

- Program flow types: sequential, selection (conditional), iteration (loops)
- if statement: if condition: (note the colon and indentation!)
- if-else statement: two-way decision making
- if-elif-else ladder: multi-way branching, checking multiple conditions
- Nested if: if inside if, complex decision trees
- Ternary expression: result = 'Pass' if marks >= 40 else 'Fail'
- Common programs: largest of 2/3 numbers, even/odd, positive/negative/zero
- Grade calculator, leap year checker, vowel/consonant detector
- for loop: for variable in sequence:, iterating over range, string, list
- range() function: range(stop), range(start, stop), range(start, stop, step)
- range(5) 0,1,2,3,4; range(2,8) 2,3,4,5,6,7; range(10,0,-1) 10,9,...,1
- while loop: while condition:, repeat until condition becomes False
- while loop with counter, controlled repetition
- Infinite loop: while True, use with break for exit
- Loop control: break (exit loop immediately), continue (skip to next iteration)
- Nested loops: outer loop controls rows, inner loop controls columns
- Pattern printing: *, numbers, alphabets, basic patterns
- Accumulator patterns: sum of n numbers, factorial, counting
- Common programs: sum of digits, reverse number, palindrome check, Fibonacci
- Board exam focus: tracing loop output, counting iterations, break/continue effects

Projects built:

- Menu-driven calculator with if-elif-else
- Number guessing game using while loop
- Pattern printer, 10 basic star and number patterns
- Prime number checker and Fibonacci generator
- Multiplication table generator

Practice: 40 conditional logic problems, 30 loop tracing exercises, 15 pattern programs, Preeti Arora exercises

Lists in Python, IP Perspective

Topics:

- What are lists?, ordered, mutable, indexed collection of items
- List creation: [] empty list, [1,2,3], list() constructor, list('hello')
- Lists can contain mixed types: [1, 'hello', 3.14, True, None]
- Indexing: positive (0-based) and negative (-1 for last element)
- Slicing: list[start:stop:step], extracting sublists
- List is MUTABLE, elements can be changed: list[0] = 'new_value'
- List operations: + (concatenation), * (repetition), in (membership)
- List traversal: for item in list:, for i in range(len(list)):
- List methods, MUST KNOW for exams:
- append(x), add x to end
- insert(i, x), add x at position i
- extend(iterable), add all items from another list
- remove(x), remove first occurrence of x (ValueError if not found)
- pop(), remove and return last item, pop(i), remove at position i
- clear(), remove all items
- index(x), return position of first occurrence
- count(x), count occurrences of x
- sort(), sort in ascending order, sort(reverse=True), descending
- reverse(), reverse the list in place
- copy(), create shallow copy
- Built-in functions: len(), min(), max(), sum(), sorted(), list()
- del statement: del list[i], del list[start:stop]
- List comprehension basics: [x*2 for x in range(5)] [0, 2, 4, 6, 8]
- Common programs: linear search, finding max/min without built-in, removing duplicates
- Board exam: output prediction with list operations and methods

Projects built:

- Student marks manager, add, remove, sort, find topper, average
- Shopping list application with all list operations
- List-based number analyzer, even/odd count, sum, average, max, min
- Duplicate remover, remove duplicate elements from a list

Practice: Preeti Arora list chapter exercises, 50 list MCQs, 20 list programs

Dictionaries in Python, IP Perspective

Topics:

- What are dictionaries?, unordered collection of key-value pairs
- Dictionary creation: {} empty, {'name': 'Riya', 'age': 16}, dict()
- Keys must be immutable (string, int, tuple), lists cannot be keys
- Values can be any type, including lists, other dictionaries
- Accessing values: d['key'] (raises KeyError if not found), d.get('key', default)

- Adding/updating: `d['new_key'] = value` (adds if new, updates if exists)
- Removing: `del d['key'], d.pop('key'), d.popitem()` (removes last), `d.clear()`
- Dictionary methods:
 - `dict()`, constructor
 - `len(d)`, number of key-value pairs
 - `d.keys()`, view of all keys
 - `d.values()`, view of all values
 - `d.items()`, view of all (key, value) tuples
 - `d.get(key, default)`, safe access with default
 - `d.update(other_dict)`, merge/update from another dictionary
 - `d.copy()`, shallow copy
 - `d.clear()`, remove all items
- Traversing dictionaries: `for key in d:`, `for key, value in d.items():`
- Nested dictionaries: student records with multiple attributes
- `in` operator with dictionaries: checks keys (not values)
- Dictionary comprehension basics: `{x: x**2 for x in range(5)}`
- Common programs: character frequency, word counter, student database
- Board exam focus: output prediction with dictionary operations

Projects built:

- Phone book application, add, search, update, delete contacts
- Character frequency counter for a string
- Student report card using nested dictionaries
- Word frequency analyzer, count each word in a paragraph

Practice: Preeti Arora dictionary exercises, 40 MCQs, 15 dictionary programs

Months 4-5: SQL Fundamentals, Database Concepts & Basic Queries (Week 13-20)

Database Concepts & Relational Model

Topics:

- What is data?, raw facts and figures vs information (processed data)
- What is a database?, organized collection of related data
- Need for databases: data redundancy reduction, data integrity, data security
- File system vs DBMS: why databases replaced flat files
- DBMS examples: MySQL, PostgreSQL, Oracle, MongoDB, SQL Server
- Relational Database Model: data stored in tables (relations)
- RDBMS terminology:
 - Relation = Table (collection of rows and columns)
 - Attribute = Column = Field (a property of the entity)
 - Tuple = Row = Record (a single entry in the table)
 - Domain = Set of valid values for an attribute
 - Degree = Number of attributes (columns) in a relation
 - Cardinality = Number of tuples (rows) in a relation
- Keys, identifying rows uniquely:
 - Candidate Key: attribute(s) that can uniquely identify each tuple, there can be multiple
 - Primary Key: the chosen candidate key, only ONE per table, no NULL values
 - Alternate Key: candidate keys that were not chosen as primary key
- Example: Student table, RollNo and AdmissionNo are both candidate keys; if RollNo is primary key, AdmissionNo is alternate key
- Referential integrity concept basics

Projects built:

- Design database schema for: school, library, hospital, e-commerce, identify all keys
- Key identification worksheet, 15 scenarios
- Draw 5 sample tables with proper attributes, data types, and keys

Practice: Preeti Arora database concepts chapter, 40 MCQs on RDBMS terminology, key identification exercises

SQL Introduction, DDL & DML Commands

Topics:

- What is SQL?, Structured Query Language, communicating with databases
- SQL is NOT a programming language, it's a query language
- MySQL: free, open-source RDBMS, used in CBSE IP
- Installing MySQL on Windows, MySQL Community Server + MySQL Workbench
- Accessing MySQL: command line client, MySQL Workbench GUI
- SQL command categories: DDL, DML, DQL
- SQL data types:
 - `CHAR(n)`, fixed-length string (padded with spaces), `VARCHAR(n)`, variable-length
 - `INT`, integer, `SMALLINT`, smaller range, `BIGINT`, larger range
 - `FLOAT`, decimal with approximate precision, `DOUBLE`, higher precision
 - `DATE`, format 'YYYY-MM-DD', `DATETIME`, 'YYYY-MM-DD HH:MM:SS'
- DDL (Data Definition Language), structure commands:
 - `CREATE DATABASE db_name;`, create new database
 - `USE db_name;`, select active database
 - `SHOW DATABASES;`, list all databases
 - `SHOW TABLES;`, list all tables in current database
 - `CREATE TABLE table_name (col1 datatype constraint, col2 datatype, ...);`
 - Constraints: NOT NULL, UNIQUE, PRIMARY KEY, DEFAULT
 - `DESCRIBE table_name;` / `DESC table_name;`, show table structure
 - ALTER TABLE: ADD column, MODIFY column, DROP column
 - `DROP TABLE table_name;`, delete entire table (structure + data)
 - `DROP DATABASE db_name;`, delete entire database
- DML (Data Manipulation Language), data commands:

- INSERT INTO table VALUES (val1, val2, ...);, insert all columns
- INSERT INTO table (col1, col2) VALUES (val1, val2);, insert specific columns
- INSERT multiple rows in one command
- UPDATE table SET col1=val1 WHERE condition;, modify data
- DELETE FROM table WHERE condition;, remove specific rows
- WARNING: UPDATE/DELETE without WHERE affects ALL rows!

Projects built:

- Create School database with Student, Teacher, Subject tables
- Insert 15+ records in each table
- Practice ALTER TABLE, add, modify, drop columns
- UPDATE and DELETE with various conditions

Practice: 30 DDL/DML queries, Preeti Arora SQL chapter exercises, create-insert-update-delete practice

SQL DQL, SELECT Queries with WHERE, Operators & Sorting

Topics:

- SELECT, the most important SQL command for IP exams
- SELECT * FROM table;, retrieve all columns, all rows
- SELECT col1, col2 FROM table;, specific columns only
- SELECT DISTINCT col FROM table;, remove duplicate values
- Column aliases: SELECT col AS alias_name
- WHERE clause, filtering rows based on conditions:
- Comparison operators: = <> (or !=) < > <= >=
- Logical operators: AND, OR, NOT
- Operator precedence: NOT > AND > OR (use parentheses for clarity)
- BETWEEN operator: WHERE marks BETWEEN 60 AND 90 (inclusive!)
- NOT BETWEEN: marks NOT BETWEEN 60 AND 90
- IN operator: WHERE city IN ('Delhi', 'Mumbai', 'Kolkata')
- NOT IN: WHERE city NOT IN ('Delhi', 'Mumbai')
- LIKE operator, pattern matching:
- %, matches zero or more characters: 'A%' (starts with A), '%ing' (ends with ing)
- _, matches exactly one character: '_a%' (second letter is a), '____' (exactly 3 chars)
- LIKE patterns: '%an%' (contains 'an'), 'S____' (S followed by exactly 4 chars)
- IS NULL: WHERE commission IS NULL, checking for missing values
- IS NOT NULL: WHERE phone IS NOT NULL
- NULL arithmetic: anything + NULL = NULL, NULL = NULL is UNKNOWN (use IS NULL!)
- ORDER BY clause, sorting results:
- ORDER BY col ASC, ascending (default, A-Z, 0-9)
- ORDER BY col DESC, descending (Z-A, 9-0)
- ORDER BY col1 ASC, col2 DESC, multi-column sorting
- NULL values in ORDER BY, appear first in ASC, last in DESC

Projects built:

- 40-query challenge on Student table, various SELECT queries
- Employee database queries, salary filtering, department queries
- Pattern matching challenge, 15 LIKE queries
- Multi-condition queries combining AND, OR, NOT, BETWEEN, IN, LIKE

Practice: 60 SELECT queries, Preeti Arora DQL exercises, 30 output prediction MCQs for SQL

SQL Aggregate Functions & Class 11 SQL Revision

Topics:

- Aggregate functions, operating on groups of rows:
- COUNT(*), count all rows (including NULLs)
- COUNT(column), count non-NULL values in column
- COUNT(DISTINCT column), count unique non-NULL values
- SUM(column), total of numeric column (ignores NULLs)
- AVG(column), average of numeric column (ignores NULLs)
- MAX(column), maximum value (works with numbers, strings, dates)
- MIN(column), minimum value
- Aggregate functions IGNORE NULL values (except COUNT(*))
- Using aggregate functions with WHERE: SELECT AVG(marks) FROM student WHERE class=12;
- Aliases with aggregates: SELECT COUNT(*) AS total_students FROM student;
- GROUP BY clause: grouping rows for aggregate calculations
- SELECT city, COUNT(*) FROM student GROUP BY city;
- GROUP BY with multiple columns
- HAVING clause: filtering groups after GROUP BY
- WHERE vs HAVING, WHERE filters rows BEFORE grouping, HAVING filters AFTER grouping
- Complete query with all clauses: SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...
- Class 11 SQL complete revision, all DDL, DML, DQL commands
- SQL query writing strategy for exams
- Common mistakes in SQL and how to avoid them

Projects built:

- Sales database analysis, total, average, max, min by category, region
- Student performance report, class-wise, section-wise statistics
- SQL revision worksheet, 80 queries covering all topics
- SQL error correction, find and fix errors in 20 queries

Practice: 50 aggregate/GROUP BY/HAVING queries, Preeti Arora exercises, previous year SQL questions

PHASE 2: CLASS 11 SEMESTER 2, EMERGING TRENDS, PRACTICALS & EXAM PREP (MONTHS 6-10)

Complete remaining Class 11 topics: Emerging Trends in Technology. Prepare practical file with Python programs and SQL queries. Get ready for Class 11 final examination.

Months 6-7: Emerging Trends & Revision (Week 21-28)

Introduction to Emerging Trends (CBSE IP Unit 4-5 marks)

Topics:

- Artificial Intelligence (AI): machines that simulate human intelligence
- Types of AI: Narrow AI (Siri, Alexa), General AI (future), Super AI (theoretical)
- Applications of AI: healthcare, education, autonomous vehicles, chatbots (ChatGPT)
- Machine Learning (ML): subset of AI, computers learn from data without explicit programming
- Types of ML: supervised, unsupervised, reinforcement learning
- Natural Language Processing (NLP): AI understanding human language, Google Translate, voice assistants
- Augmented Reality (AR): digital content overlaid on real world, Pokémon Go, Instagram filters
- Virtual Reality (VR): fully immersive digital environment, Oculus, VR gaming
- Mixed Reality (MR): combination of AR and VR
- Robotics: programmable machines performing tasks, manufacturing, surgery, exploration
- Types of robots: industrial, service, humanoid, drone
- Internet of Things (IoT): everyday objects connected to internet, smart home, wearables
- IoT architecture: sensors connectivity data processing user interface
- IoT examples: smart thermostat, fitness tracker, smart fridge, traffic management
- Smart Cities: technology-driven urban management, traffic, waste, energy, safety
- Big Data: extremely large datasets requiring special tools to process
- Big Data characteristics: 5 V's, Volume, Velocity, Variety, Veracity, Value
- Cloud Computing: accessing computing resources over internet instead of local machines
- Cloud service models: SaaS (Gmail), PaaS (Google App Engine), IaaS (AWS EC2)
- Cloud deployment: public, private, hybrid, community cloud
- Grid Computing: distributed computing, combining many computers for complex tasks
- Blockchain technology: decentralized, immutable digital ledger, cryptocurrency, supply chain
- How blockchain works: blocks, hashing, consensus mechanism

Projects built:

- Emerging technology presentation, choose one topic and present for 10 minutes
- IoT application design, design a smart home system on paper
- AI in daily life report, identify 15 AI applications you use daily
- Cloud vs traditional computing comparison chart

Practice: Preeti Arora emerging trends chapter, NCERT IP textbook, 50 MCQs on emerging trends

Practical File, Project & Class 11 Exam Preparation

Topics:

- Practical file requirements for Class 11 IP:
- Minimum 14 Python programs covering all topics
- Minimum 14 SQL queries covering DDL, DML, DQL
- Python programs for practical file:
- P1: Input two numbers, perform all arithmetic operations
- P2: Find largest of three numbers
- P3: Check if number is even/odd, positive/negative
- P4: Grade calculator using if-elif-else
- P5: Print multiplication table using for loop
- P6: Print patterns using nested loops (3 patterns)
- P7: Check prime number, palindrome, Armstrong number
- P8: Calculate factorial using while loop
- P9: List operations, create, append, insert, remove, sort, search
- P10: Find max, min, sum, average from a list
- P11: Count vowels, consonants in a string
- P12: Dictionary operations, create, add, update, delete, traverse
- P13: Character/word frequency counter using dictionary
- P14: Menu-driven program combining multiple concepts
- SQL queries for practical file:
- Q1-Q3: CREATE TABLE, INSERT INTO, DDL and DML
- Q4-Q6: SELECT with WHERE, BETWEEN, IN, LIKE
- Q7-Q9: UPDATE, DELETE with conditions
- Q10-Q11: Aggregate functions, COUNT, SUM, AVG, MAX, MIN
- Q12-Q14: GROUP BY, HAVING, ORDER BY
- Class 11 final exam revision, unit-wise important questions
- Sample paper solving, 3 complete papers with solutions
- Viva voce preparation, 30 Q&A

Deliverables:

- Complete practical file: 14+ Python programs + 14+ SQL queries
- 3 solved sample papers
- Viva preparation notes
- Ready for Class 11 final exam

PHASE 3: CLASS 12 SEMESTER 1, PANDAS, DATA VISUALIZATION & ADVANCED DATA HANDLING (MONTHS 11-17)

This is the CORE of Class 12 IP, mastering Pandas for data handling and Matplotlib for data visualization. Unit 1 carries 25 marks in the board exam. Every concept here is board exam

critical.

Months 11-12: Python Pandas, Series (Week 41-48)

Introduction to Python Libraries & Pandas Series Basics

Topics:

- What are Python libraries?, pre-written code modules for specific tasks
- NumPy: Numerical Python, arrays, mathematical operations (foundation for Pandas)
- Pandas: Python Data Analysis Library, data manipulation and analysis
- Matplotlib: plotting and visualization library
- Installing libraries: `pip install pandas numpy matplotlib`
- Importing: `import pandas as pd, import numpy as np`
- Pandas data structures: Series (1D) and DataFrame (2D)
- Series: one-dimensional labeled array, like a column in a table
- Creating Series from a list: `pd.Series([10, 20, 30, 40])`
- Default index: 0, 1, 2, 3... (RangeIndex)
- Custom index: `pd.Series([10, 20, 30], index=['a', 'b', 'c'])`
- Creating Series from a dictionary: `pd.Series({'Maths': 90, 'Science': 85})`
- Creating Series from a scalar value: `pd.Series(5, index=['a', 'b', 'c'])` all values are 5
- Creating Series from NumPy ndarray: `pd.Series(np.array([1, 2, 3]))`
- Series attributes: `dtype`, `shape`, `size`, `nbytes`, `ndim`, `index`, `values`, `name`
- Checking data type: `s.dtype` returns data type of elements
- Series is like a fixed-size dictionary, index acts as keys

Projects built:

- Create 10 different Series, from list, dict, scalar, ndarray
- Student marks Series with subject names as index
- Temperature data Series for a week with day names as index

Practice: Preeti Arora Pandas chapter exercises, 20 Series creation MCQs

Pandas Series, Operations, Indexing & Methods

Topics:

- Accessing elements: `s[0]` (positional), `s['a']` (label-based)
- Slicing Series: `s[1:4]` (positional), `s['b':'d']` (label, inclusive on both ends!)
- `s.head(n)`, first n elements (default 5), `s.tail(n)`, last n elements
- Mathematical operations on Series, element-wise:
- `s + 10` adds 10 to each element
- `s1 + s2` adds corresponding elements (NaN for unmatched indices)
- `s * 2`, `s / 3`, `s ** 2`, all element-wise operations
- Comparison operations: `s > 50`, `s == 100` returns Boolean Series
- Boolean indexing / filtering: `s[s > 50]` elements greater than 50
- NaN (Not a Number): result of mismatched index operations
- Handling NaN: `s.isnull()`, `s.notnull()`, `s.dropna()`, `s.fillna(value)`
- Series methods for board exams:
- `s.sum()`, `s.mean()`, `s.median()`, `s.mode()`, `s.std()`, `s.var()`
- `s.min()`, `s.max()`, `s.count()` (non-NaN count)
- `s.describe()`, summary statistics (count, mean, std, min, 25%, 50%, 75%, max)
- `s.value_counts()`, frequency count of each unique value
- `s.unique()`, array of unique values, `s.nunique()`, count of unique values
- `s.sort_values(ascending=True/False)`, sort by values
- `s.sort_index()`, sort by index labels
- Vectorized string operations: `s.str.upper()`, `s.str.lower()`, `s.str.len()`
- Board exam focus: output prediction with Series operations, NaN handling

Projects built:

- Sales data analysis using Series, total, average, max, min, filtering
- Temperature analysis, find hottest/coldest day, above average days
- Student marks comparison, Series arithmetic with NaN handling
- Statistical summary report using `describe()`

Practice: 40 Series operation MCQs, 20 output prediction problems, Preeti Arora exercises

Pandas DataFrame, Creation & Basic Operations

Topics:

- DataFrame: 2-dimensional labeled data structure, like a spreadsheet/SQL table
- DataFrame = collection of Series sharing the same index
- Creating DataFrame from dictionary of lists:
- `pd.DataFrame({'Name': ['A', 'B'], 'Marks': [90, 85]})`
- Creating DataFrame from dictionary of Series
- Creating DataFrame from list of dictionaries
- Creating DataFrame from 2D list/array with `columns` parameter
- Creating DataFrame from CSV file: `pd.read_csv('file.csv')`, VERY IMPORTANT
- Custom index: `pd.DataFrame(data, index=['row1', 'row2', ...])`
- DataFrame attributes: `shape`, `size`, `ndim`, `dtypes`, `columns`, `index`, `values`, `T` (transpose)
- Selecting columns: `df['column']`, `df[['col1', 'col2']]` (single vs multiple)
- Adding a new column: `df['new_col'] = values`
- Deleting a column: `del df['col']`, `df.drop('col', axis=1)`, `df.pop('col')`
- Renaming columns: `df.rename(columns={'old': 'new'})`
- Selecting rows by label: `df.loc['row_label']`, `df.loc['r1':'r3']`
- Selecting rows by position: `df.iloc[0]`, `df.iloc[0:3]`
- `loc` vs `iloc`, label-based vs integer position-based, EXAM CRITICAL
- Selecting specific cell: `df.loc['row', 'col']`, `df.iloc[row_idx, col_idx]`
- `df.at['row', 'col']`, fast scalar access by label
- `df.iat[row_idx, col_idx]`, fast scalar access by position

- df.head(n), df.tail(n), view first/last n rows

Projects built:

- Student database DataFrame, create from dictionary, CSV, and list of dicts
- Employee records with adding, renaming, and deleting columns
- Accessing data challenge, loc vs iloc practice with 20 examples
- CSV data loader, read real-world dataset and explore

Practice: Preeti Arora DataFrame chapter exercises, 30 creation MCQs, 20 loc/iloc problems

Pandas DataFrame, Advanced Operations & Data Manipulation

Topics:

- Adding a new row: df.loc['new_index'] = [values], pd.concat()
- Deleting rows: df.drop('row_label'), df.drop([indices])
- Boolean indexing with DataFrame: df[df['Marks'] > 80]
- Multiple conditions: df[(df['Marks'] > 80) & (df['City'] == 'Delhi')]
- Note: use & instead of 'and', | instead of 'or' for DataFrame filtering
- Sorting: df.sort_values('column'), df.sort_values(['col1', 'col2'])
- df.sort_values('column', ascending=False), descending sort
- df.sort_index(), sort by index
- DataFrame iteration: df.iterrows(), iterate row by row
- Aggregate operations: df['col'].sum(), .mean(), .max(), .min(), .count()
- df.describe(), statistical summary for all numeric columns
- df.info(), overview of DataFrame structure
- df.isnull(), check for NaN values, df.notnull()
- df.dropna(), remove rows with NaN, df.fillna(value), fill NaN
- df.drop_duplicates(), remove duplicate rows
- Exporting DataFrame to CSV: df.to_csv('output.csv', index=False)
- DataFrame arithmetic: df1 + df2, df * scalar, element-wise operations
- Pivot tables basics: df.pivot_table(values, index, aggfunc)
- Groupby basics: df.groupby('column').mean()
- Board exam focus: complete DataFrame operation chains, output prediction

Projects built:

- Complete student report card system using DataFrame
- Sales data analysis, filtering, sorting, grouping, statistics
- CSV data processing pipeline, read clean analyze export
- Covid/weather/population dataset analysis (real-world data)

Practice: 50 DataFrame operation problems, 30 output prediction MCQs, Preeti Arora advanced exercises

Months 13-14: Data Visualization with Matplotlib (Week 49-56)

Introduction to Matplotlib, Line Charts & Bar Graphs

Topics:

- What is data visualization?, representing data graphically for insights
- Why visualize?, patterns, trends, comparisons are easier to see in charts
- Matplotlib library: most popular Python plotting library
- Importing: import matplotlib.pyplot as plt
- Basic plot workflow: create figure plot data customize show/save
- Line Chart / Line Plot: plt.plot(x, y)
- plt.plot() with just y values, x defaults to 0, 1, 2, ...
- Multiple lines on same plot: call plt.plot() multiple times
- Line customization:
 - color: plt.plot(x, y, color='red') or color='r' or color='#FF0000'
 - Line style: linestyle='-' (solid), '--' (dashed), '-.' (dash-dot), ':' (dotted)
 - Line width: linewidth=2 or lw=2
 - Markers: marker='o' (circle), 's' (square), '^' (triangle), 'D' (diamond), '*' (star)
 - Marker size: markersize=10 or ms=10
- Combined format string: plt.plot(x, y, 'r--o') red dashed line with circles
- Chart elements, MUST add for full marks:
 - plt.title('Chart Title'), chart title
 - plt.xlabel('X-Axis Label'), plt.ylabel('Y-Axis Label'), axis labels
 - plt.legend(['Line 1', 'Line 2']) or label='name' in plot + plt.legend()
 - plt.grid(True), add grid lines
 - plt.xticks(rotation=45), rotate x-axis labels
 - plt.show(), display the chart
 - plt.savefig('chart.png'), save chart as image
- Bar Graph: plt.bar(x, height), vertical bars
- Horizontal bar: plt.barh(x, width)
- Bar customization: color, width, edgecolor, align
- Multiple bar groups: side-by-side bars using width offset
- Stacked bars: plt.bar(x, y1); plt.bar(x, y2, bottom=y1)

Projects built:

- Monthly temperature line chart, 12 months with customization
- Student marks comparison, line chart for 5 subjects across 3 students
- Product sales bar graph, quarterly sales for 4 products
- Population comparison, horizontal bar chart for 10 cities

Practice: 15 line chart programs, 10 bar chart programs, Preeti Arora visualization exercises

Histograms, Advanced Customization & Combining Charts

Topics:

- Histogram: plt.histogram(data, bins), frequency distribution chart

- Histogram vs Bar graph: histogram for continuous data, bar for categorical
- Histogram parameters: bins (number of intervals), range, edgecolor, color, alpha
- bins parameter: bins=10 (10 equal intervals), bins=[0,20,40,60,80,100] (custom)
- alpha parameter: transparency, alpha=0.7 (0=invisible, 1=opaque)
- Using histograms: exam marks distribution, age distribution, salary distribution
- Advanced customization for all chart types:
- Figure size: plt.figure(figsize=(10, 6))
- Font sizes: fontsize parameter in title, xlabel, ylabel, legend
- Font weight: fontweight='bold'
- Color options: named colors, hex codes, RGB tuples
- Grid customization: plt.grid(axis='y', linestyle='--', alpha=0.7)
- Setting axis limits: plt.xlim(0, 100), plt.ylim(0, 50)
- Annotations: plt.annotate('text', xy=(x,y), xytext=(tx,ty), arrowprops=dict(arrowstyle='->'))
- Adding text: plt.text(x, y, 'text')
- Subplots: plt.subplot(rows, cols, position), multiple charts in one figure
- plt.tight_layout(), avoid overlapping in subplots
- Complete chart for board exam: title + labels + legend + grid + colors + markers
- Board exam pattern: write code to create a specific chart from given data
- Chart type selection: when to use line vs bar vs histogram
- Practical exam: creating charts from Pandas DataFrame data

Projects built:

- Student marks histogram, frequency distribution across score ranges
- Dashboard with subplots, line, bar, and histogram in one figure
- Complete data story: Pandas analysis Matplotlib visualization
- 4 Matplotlib programs for practical file, fully customized charts

Practice: 10 histogram programs, 5 subplot programs, complete visualization exercises, board exam chart questions

Months 15-16: Pandas + Matplotlib Integration & CSV Data Workflow (Week 57-64)

Pandas DataFrame Visualization & CSV Import/Export

Topics:

- Plotting directly from DataFrame: df.plot(), integrates Pandas with Matplotlib
- df.plot(kind='line'), line chart from DataFrame columns
- df.plot(kind='bar'), bar chart, df.plot(kind='barh'), horizontal bar
- df.plot(kind='hist'), histogram from DataFrame data
- df['column'].plot(), plot a single column as Series
- Complete CSV workflow:
- Step 1: pd.read_csv('data.csv'), import data into DataFrame
- read_csv parameters: sep, header, index_col, usecols, nrows, na_values
- Step 2: df.head(), df.info(), df.describe(), explore the data
- Step 3: df[condition], filter and clean data
- Step 4: df.groupby().agg(), aggregate and summarize
- Step 5: df.plot() or plt.plot(), visualize insights
- Step 6: df.to_csv('output.csv', index=False), export results
- Real-world data analysis workflow using CSV files
- Handling large CSV files: chunking, selecting columns
- Common Pandas + Matplotlib patterns for board exams

Projects built:

- Weather data analysis: CSV import filtering visualization
- Student performance dashboard: CSV DataFrame charts
- Sales trend analysis: monthly/quarterly charts from CSV data
- Complete data analysis project for practical file

Practice: 15 integrated Pandas-Matplotlib programs, CSV workflow exercises

Pandas & Matplotlib Revision + Practical Programs

Topics:

- Complete Pandas revision: Series creation, operations, methods
- DataFrame revision: creation, selection, filtering, sorting, groupby
- CSV read/write revision
- Matplotlib revision: line, bar, histogram with full customization
- 15 Pandas programs for practical file:
- P1: Create Series from list, dict, ndarray, display attributes
- P2: Series arithmetic operations and NaN handling
- P3: Series statistical functions, mean, median, mode, std
- P4: Create DataFrame from dict, list of dicts, CSV
- P5: DataFrame column operations, add, delete, rename
- P6: DataFrame row selection with loc and iloc
- P7: Boolean indexing, filter DataFrame rows by condition
- P8: Sorting DataFrame by column values
- P9: DataFrame aggregate functions, sum, mean, count
- P10: GroupBy operations, group by category and aggregate
- P11: Handling missing data, isnull, dropna, fillna
- P12: Import CSV into DataFrame and display statistics
- P13: Export DataFrame to CSV file
- P14: DataFrame describe() and info(), data exploration
- P15: Integrated program, CSV filter aggregate export
- 4+ Matplotlib programs for practical file:
- M1: Line chart with title, labels, legend, grid, colors
- M2: Bar graph (vertical/horizontal) with customization
- M3: Histogram with bins, color, edgecolor

- M4: Multiple charts comparison using subplots

Deliverables:

- 15 Pandas programs written and tested
- 4+ Matplotlib programs with saved chart images
- Complete understanding of Pandas-Matplotlib for board exam

PHASE 4: CLASS 12 SEMESTER 2, ADVANCED SQL, NETWORKS, SOCIETY & BOARD EXAM MASTERY (MONTHS 18-24)

Complete remaining Class 12 syllabus: advanced SQL with functions, joins, GROUP BY/HAVING, computer networks, and societal impacts. Then intensive board exam preparation with mock tests.

Months 18-19: Advanced SQL, Functions, GROUP BY, HAVING & Joins (Week 69-76)

SQL Single Row Functions, Math, String & Date Functions

Topics:

- Revision of Class 11 SQL: CREATE, INSERT, SELECT, WHERE, operators
- SQL Functions: two categories, single row functions & aggregate functions
- Single row functions work on each row individually
- Mathematical functions:
 - POWER(base, exponent), POWER(2, 3) = 8
 - ROUND(number, decimals), ROUND(15.678, 1) = 15.7, ROUND(15.678, 0) = 16
 - ROUND(1256, -2) = 1300, rounding to nearest hundred
 - MOD(dividend, divisor), MOD(17, 5) = 2 (remainder)
 - TRUNCATE(number, decimals), TRUNCATE(15.678, 1) = 15.6 (no rounding!)
 - ABS(number), absolute value
 - CEIL(number) / CEILING(number), smallest integer >= number
 - FLOOR(number), largest integer <= number
- String/Text functions:
 - UCASE(str) / UPPER(str), convert to uppercase
 - LCASE(str) / LOWER(str), convert to lowercase
 - MID(str, start, length) / SUBSTRING(str, start, length) / SUBSTR(str, start, length)
 - Note: MySQL string positions start from 1 (not 0 like Python!)
 - LENGTH(str), number of characters including spaces
 - LEFT(str, n), first n characters, RIGHT(str, n), last n characters
 - INSTR(str, substr), position of first occurrence of substr (returns 0 if not found)
 - LTRIM(str), remove leading spaces, RTRIM(str), remove trailing spaces
 - TRIM(str), remove both leading and trailing spaces
 - CONCAT(str1, str2), join strings together
 - REPLACE(str, old, new), replace occurrences
- Date functions:
 - NOW(), current date and time
 - DATE(datetime), extract date portion
 - MONTH(date), month number (1-12)
 - MONTHNAME(date), month name ('January', 'February', ...)
 - YEAR(date), year number
 - DAY(date), day of month (1-31)
 - DAYNAME(date), day name ('Monday', 'Tuesday', ...)
 - DAYOFWEEK(date), 1=Sunday, 2=Monday, ... 7=Saturday
 - Using functions in SELECT and WHERE clauses
 - Nested functions: UPPER(LEFT(name, 3)), combine functions

Projects built:

- SQL function explorer, test all functions on sample data
- Employee name formatting, UPPER, LOWER, CONCAT, SUBSTRING
- Date-based queries, find employees hired in specific month/year
- Mathematical query set, ROUND, MOD, POWER calculations on salary data

Practice: 40 function-based SQL queries, output prediction MCQs, Preeti Arora SQL function exercises

SQL Aggregate Functions, GROUP BY & HAVING, Deep Dive

Topics:

- Aggregate functions (revision and deep dive):
 - COUNT(*), total rows, COUNT(column), non-NULL count
 - SUM(column), total, AVG(column), average
 - MAX(column), maximum, MIN(column), minimum
 - COUNT(DISTINCT column), unique value count
 - All aggregates ignore NULL except COUNT(*)
- GROUP BY clause, advanced usage:
 - Grouping by single column: SELECT city, COUNT(*) FROM student GROUP BY city;
 - Grouping by multiple columns: GROUP BY class, section
 - GROUP BY with all aggregate functions
 - Rules: columns in SELECT must be in GROUP BY or inside aggregate functions
- HAVING clause, filtering groups:
 - HAVING COUNT(*) > 5, show only groups with more than 5 members
 - HAVING AVG(marks) >= 80, show groups with average marks >= 80
- WHERE vs HAVING, the definitive comparison (EXAM CRITICAL):
 - WHERE: filters ROWS before grouping, cannot use aggregate functions
 - HAVING: filters GROUPS after grouping, CAN use aggregate functions
 - WHERE is for conditions on individual rows
 - HAVING is for conditions on grouped results

- Complete query with all clauses:
- SELECT col, AGG(col) FROM table WHERE condition GROUP BY col HAVING agg_condition ORDER BY col;
- Execution order: FROM WHERE GROUP BY HAVING SELECT ORDER BY
- Complex queries combining WHERE + GROUP BY + HAVING + ORDER BY
- Board exam: write query for given output, find errors, output prediction

Projects built:

- Sales analysis: total/average sales by region, category, with filtering
- Student database: section-wise, class-wise statistics with HAVING
- Employee analysis: department-wise salary stats, count, conditions
- 30-query challenge: all GROUP BY + HAVING combinations

Practice: 50 GROUP BY/HAVING queries, WHERE vs HAVING comparison exercises, board exam

SQL from last 10 years

SQL Joins, Equi-Join on Two Tables

Topics:

- Why joins?, real databases have multiple related tables
- Foreign key: column in one table referencing primary key of another
- Example: Student(RollNo, Name, ClassID) and Class(ClassID, ClassName, Teacher)
- Cartesian Product / Cross Join: every row of Table1 x every row of Table2
- If Table1 has 5 rows and Table2 has 3 rows result has 15 rows
- Cartesian product is rarely useful, need to filter with conditions
- Equi-Join: Cartesian product + WHERE condition matching common columns
- SELECT s.Name, c.ClassName FROM student s, class c WHERE s.ClassID = c.ClassID;
- Table aliases: student s, class c, shorter names for readability
- Using dot notation: s.Name, c.ClassName, specify which table's column
- Why aliases are needed: when both tables have same column name
- Natural Join: automatically joins on columns with same name
- SELECT * FROM student NATURAL JOIN class;; auto-matches ClassID
- Natural join removes duplicate columns in result
- Equi-join vs Natural Join: equi-join gives you control, natural join is automatic
- Join with additional conditions: WHERE s.ClassID = c.ClassID AND s.Marks > 80
- Joining with aggregate functions: SELECT c.ClassName, AVG(s.Marks) FROM student s, class c WHERE s.ClassID = c.ClassID GROUP BY c.ClassName;
- Board exam join patterns: 2-table queries with conditions, aggregates, sorting
- Practice: identifying which tables to join and on which columns

Projects built:

- Student-Marks-Subject multi-table query project
- Employee-Department join queries, 20 questions
- Product-Order-Customer join queries
- Complete SQL revision: single table + multi-table + functions + aggregates

Practice: 30 join queries, Preeti Arora join exercises, board exam join questions from last 10 years

SQL Complete Revision + 15 SQL Queries for Practical File

Topics:

- Complete SQL revision, all commands:
- DDL: CREATE DATABASE/TABLE, ALTER TABLE, DROP TABLE/DATABASE
- DML: INSERT INTO, UPDATE, DELETE
- DQL: SELECT with WHERE, BETWEEN, IN, LIKE, IS NULL/IS NOT NULL
- Functions: Math (POWER, ROUND, MOD), String (UPPER, LOWER, MID, LENGTH, LEFT, RIGHT, INSTR, TRIM, CONCAT), Date (NOW, DATE, MONTH, MONTHNAME, YEAR, DAY, DAYNAME)
- Aggregates: COUNT, SUM, AVG, MAX, MIN
- Clauses: GROUP BY, HAVING, ORDER BY, DISTINCT
- Joins: Cartesian product, Equi-join, Natural join
- 15+ SQL queries for practical file:
- Q1: CREATE TABLE with constraints (PRIMARY KEY, NOT NULL)
- Q2: INSERT INTO, multiple rows
- Q3: SELECT with WHERE using comparison operators
- Q4: SELECT with BETWEEN and IN
- Q5: SELECT with LIKE and wildcards
- Q6: SELECT with IS NULL / IS NOT NULL
- Q7: SELECT with ORDER BY (ASC and DESC)
- Q8: UPDATE records with WHERE condition
- Q9: DELETE records with WHERE condition
- Q10: ALTER TABLE, add and modify column
- Q11: Aggregate functions, COUNT, SUM, AVG, MAX, MIN
- Q12: GROUP BY with aggregate functions
- Q13: GROUP BY with HAVING clause
- Q14: SQL functions, UPPER, LOWER, MID, LENGTH, ROUND, MOD
- Q15: Equi-join on two tables with conditions

Deliverables:

- 15+ SQL queries for practical file
- Complete SQL command reference sheet
- All queries tested and verified on MySQL

Months 20-21: Computer Networks & Societal Impacts (Week 77-84)

Introduction to Computer Networks (CBSE IP Unit 3-10 marks)

Topics:

- What is a computer network?, interconnected devices sharing resources
- Types of networks based on geographical area:

- PAN (Personal Area Network): Bluetooth headset, smartwatch, within 10 meters
- LAN (Local Area Network): within a building, school lab, office, home WiFi
- MAN (Metropolitan Area Network): across a city, cable TV network, city WiFi
- WAN (Wide Area Network): across countries/continents, Internet is the largest WAN
- Networking devices:
- Modem: MODulator-DEModulator, converts digital to analog and vice versa
- Hub: broadcasts data to ALL connected devices, dumb device, creates traffic
- Switch: intelligent hub, sends data ONLY to intended device using MAC addresses
- Repeater: amplifies/regenerates weak signals, extends cable distance
- Router: connects different networks, directs traffic using IP addresses
- Gateway: connects networks with DIFFERENT protocols, translates between them
- NIC (Network Interface Card): hardware for network connectivity, has MAC address
- WiFi adapter: wireless NIC for WiFi connectivity
- Network topologies:
- Star topology: all devices connect to central hub/switch
- Advantages: easy to add/remove, failure of one device doesn't affect others
- Disadvantage: central hub failure takes down entire network
- Bus topology: all devices on single backbone cable with terminators
- Advantages: cheap, easy to install
- Disadvantages: cable failure affects all, difficult troubleshooting
- Tree/Hierarchical topology: combination of star and bus, like a tree structure
- Mesh topology: every device connected to every other device
- Advantage: maximum redundancy, Disadvantage: very expensive

Projects built:

- Network diagram for a school, choose topology, devices, media
- Networking device comparison chart with functions and use cases
- Topology pros-cons table with real-world examples

Practice: Preeti Arora network chapter, 40 MCQs on network types, devices, topologies

Internet, Web Technologies & Protocols

Topics:

- Internet: global network of networks, largest WAN
- How internet works: ISP, DNS, routers, data packets
- Internet services:
- WWW (World Wide Web): system of interlinked hypertext documents, uses HTTP
- Email: electronic mail, SMTP (sending), POP3/IMAP (receiving)
- Chat/Instant Messaging: WhatsApp, Telegram, Slack
- VoIP (Voice over IP): voice calls over internet, Zoom, Google Meet, Skype
- Video conferencing: audio + video communication over internet
- URL (Uniform Resource Locator): address of a web resource
- URL structure: protocol://domain_name:port/path/page?query#fragment
- Domain name: human-readable website address (google.com, example.org)
- DNS (Domain Name System): translates domain names to IP addresses
- Domain extensions: .com, .org, .edu, .gov, .in, .co.in
- Websites: collection of related web pages
- Static web pages: fixed content, created with HTML/CSS, same for all users
- Dynamic web pages: content changes based on user, time, interaction, uses server-side processing
- Web server: computer that hosts websites and serves web pages
- Web hosting: storing website on a server for internet access
- Types of hosting: shared, VPS, dedicated, cloud hosting
- Web browser: software to access websites, Chrome, Firefox, Edge, Safari
- Browser features: tabs, bookmarks, history, extensions/add-ons, plug-ins
- Cookies: small data files stored by websites on your browser
- First-party cookies: set by the website you visit
- Third-party cookies: set by advertisers/trackers, privacy concern

Projects built:

- URL anatomy exercise, break down 10 URLs into components
- Static vs dynamic website comparison, 10 examples each
- Web technology glossary, define 30 web-related terms
- Browser feature exploration report

Practice: Preeti Arora web chapter, 50 MCQs on internet, web, protocols, cookies

Societal Impacts of Technology (CBSE IP Unit 4-10 marks)

Topics:

- Digital footprint: trail of data you leave on the internet
- Active digital footprint: intentional, social media posts, emails, comments
- Passive digital footprint: unintentional, browsing history, cookies, IP logs
- Managing your digital footprint: privacy settings, thinking before posting
- Communication etiquette (netiquette): proper online behavior
- Email etiquette: professional subject lines, proper greeting, no ALL CAPS
- Social media etiquette: respect others, verify before sharing, no cyberbullying
- Data protection: safeguarding personal and organizational data
- Personal data: name, address, phone, email, financial info, biometric data
- Consent: permission before collecting/using personal data
- Intellectual Property Rights (IPR):
- Copyright: legal right over creative works, books, music, software, art
- Patent: protection for inventions, machines, processes, formulas
- Trademark: protection for brand identity, logos, names, slogans
- Plagiarism: using someone else's work without proper credit, academic dishonesty
- How to avoid plagiarism: cite sources, paraphrase properly, use quotation marks

- Software licensing:
- Proprietary: paid, closed source, Windows, Adobe, MS Office
- Free and Open Source Software (FOSS): free to use, modify, distribute
- FOSS examples: Linux, Firefox, LibreOffice, Python, MySQL, Android, WordPress
- Creative Commons: flexible copyright licenses, CC-BY, CC-BY-SA, CC-BY-NC
- GPL (GNU General Public License): must keep derivatives open source
- Cybercrime: criminal activities using computers/internet
- Hacking: unauthorized access to computer systems
- Phishing: fraudulent emails/websites to steal login credentials
- Cyberbullying: harassment, threats, humiliation online
- Identity theft: stealing personal information for fraud
- Ransomware: malware that encrypts data and demands payment
- Indian Information Technology Act 2008 (amended 2008):
- Section 43: penalty for unauthorized access, data theft
- Section 66: computer-related offences, hacking, identity theft
- Section 67: publishing obscene material electronically
- Section 72: breach of confidentiality and privacy
- E-waste: discarded electronic devices, hazards and management
- E-waste hazards: toxic materials (lead, mercury, cadmium), soil/water pollution
- E-waste management: reduce, reuse, recycle, proper disposal at certified centers
- Health concerns from technology: digital eye strain, back/neck pain, carpal tunnel syndrome
- Ergonomics: proper posture, screen distance, break intervals (20-20-20 rule)
- Technology addiction: social media addiction, gaming disorder, awareness and balance

Projects built:

- Digital citizenship poster, 15 dos and don'ts for online behavior
- Cybercrime awareness infographic, types, prevention, reporting
- E-waste management plan for school/home
- IPR case study, analyze 3 real-world copyright/patent cases

Practice: Preeti Arora societal impacts chapter, NCERT exercises, 60 MCQs, case study questions, IT Act scenario-based questions

Months 22-24: Board Exam Preparation, Intensive Revision & Mock Tests (Week 85-96)

Practical File Completion & Project Submission

Topics:

- Complete practical file requirements verification:
- 15+ Pandas programs, Series and DataFrame operations
- 4+ Matplotlib programs, line chart, bar graph, histogram with customization
- 15+ SQL queries, DDL, DML, DQL, functions, aggregates, joins
- Project work (5 marks): data analysis project using Pandas + Matplotlib
- Suggested projects:
- COVID-19 data analysis, cases, recoveries, deaths by country/state
- Weather data analysis, temperature, rainfall trends over years
- Student performance analyzer, class-wise, subject-wise analysis with charts
- IPL/cricket statistics analysis, batting, bowling averages with visualization
- E-commerce sales analysis, product category wise, monthly trends
- Project documentation: cover page, certificate, acknowledgement, introduction, source code, output, bibliography
- Viva voce preparation, 50 most-asked questions:
- What is Pandas? What is a DataFrame? What is a Series?
- Difference between loc and iloc?
- What is matplotlib? How to create a bar chart?
- What is GROUP BY? What is HAVING?
- Difference between WHERE and HAVING?
- What is a primary key? What is a foreign key?
- What is an equi-join?
- What is a digital footprint?

Deliverables:

- Complete practical file, all programs tested and documented
- Project with full documentation
- Viva voce preparation notes, 50 Q&A

Chapter-Wise Revision, Pandas & Matplotlib

Topics:

- Pandas Series revision: creation, indexing, slicing, operations, methods, NaN handling
- Pandas DataFrame revision: creation, selection, filtering, sorting, groupby, CSV operations
- DataFrame vs Series, comparison table
- loc vs iloc, 20 practice problems
- Boolean indexing revision, complex condition filtering
- Aggregate functions on DataFrame: sum, mean, max, min, count, describe
- Matplotlib revision: line chart, bar graph, histogram
- Chart customization checklist: title, xlabel, ylabel, legend, grid, color, marker, linestyle
- Format strings revision: 'r--o', 'b-s', 'g:^^'
- Common Pandas output prediction patterns
- Quick reference card for Pandas methods and Matplotlib functions

Practice: 40 Pandas MCQs, 20 output prediction, 15 Matplotlib code-writing questions

Chapter-Wise Revision, SQL, Networks & Societal Impacts

Topics:

- SQL complete revision: DDL, DML, DQL
- SQL functions revision: POWER, ROUND, MOD, UPPER, LOWER, MID, LENGTH, LEFT, RIGHT, INSTR, TRIM, CONCAT, NOW, DATE, MONTH, YEAR, DAY, MONTHNAME, DAYNAME

- Aggregate functions revision: COUNT, SUM, AVG, MAX, MIN
- GROUP BY + HAVING revision, 20 practice queries
- Joins revision: Cartesian product, equi-join, natural join
- Complete SQL query writing, from WHERE to JOIN
- Computer networks revision: PAN, LAN, MAN, WAN, devices, topologies
- Internet and web concepts revision: URL, DNS, HTTP/HTTPS, static/dynamic, cookies
- Societal impacts revision: digital footprint, IPR, cybercrime, IT Act, e-waste
- Case study practice, network design scenarios (CBSE board favorite)

Practice: 50 SQL revision queries, 40 network MCQs, 30 societal impact MCQs, case study practice

Mock Board Exams, Set 1 & 2

Topics:

- Full-length Mock Exam 1: 70 marks, 3 hours, strict exam conditions
- CBSE IP paper structure:
- Section A: 21 questions × 1 mark = 21 marks (MCQ, True/False, Fill-in-blanks)
- Section B: 7 questions × 2 marks = 14 marks (Very Short Answer)
- Section C: 4 questions × 3 marks = 12 marks (Short Answer)
- Section D: 2 questions × 4 marks = 8 marks (Case Study based)
- Section E: 3 questions × 5 marks = 15 marks (Long Answer)
- Detailed solution discussion with CBSE marking scheme
- Full-length Mock Exam 2: different question set, same structure
- Common mistakes analysis and correction
- Time management strategy:
- Section A (21 marks): 30-35 minutes
- Section B (14 marks): 20 minutes
- Section C (12 marks): 20 minutes
- Section D (8 marks): 15 minutes
- Section E (15 marks): 25 minutes
- Review time: 10-15 minutes
- How to write Pandas/Matplotlib code in board exams
- How to write SQL queries in board exams, formatting tips

Deliverables:

- 2 complete mock papers solved and reviewed
- Weak areas identified with improvement plan
- Exam writing technique refined

Mock Board Exams, Set 3, 4, 5 + Previous Year Paper Solving

Topics:

- 3 more full-length mock exams (Set 3, 4, 5)
- CBSE IP Previous Year Papers solved:
- 2020, 2021, 2022, 2023, 2024, 2025, complete solutions with marking scheme
- CBSE IP Sample Paper 2026-27, official sample paper solved
- Frequently repeated question analysis, top 30 topics that appear every year
- Most important Pandas questions: Series operations, DataFrame creation, loc/iloc, groupby
- Most important SQL questions: GROUP BY + HAVING, aggregate functions, joins, functions
- Most important theory questions: network devices, topologies, cybercrime, IPR, IT Act
- Case study question patterns, how to approach and answer
- Assertion-Reason question strategy, understanding the format
- Partial marks strategy, how to get step marks even if you don't know complete answer

Practice: 5 mock papers + 6 previous year papers = 11 full papers solved

Final Revision & Last-Minute Board Exam Preparation

Topics:

- One-shot revision: entire IP syllabus in 2 days, key points only
- Pandas cheat sheet: all methods, attributes, operations on one page
- Matplotlib cheat sheet: plot types, customization options, format strings
- SQL command reference: all DDL, DML, DQL, functions on 2 pages
- Network concepts summary: devices, topologies, protocols on 1 page
- Societal impacts summary: key terms, IT Act sections on 1 page
- Top 30 most important Pandas output prediction problems
- Top 20 SQL queries you must know by heart
- Top 15 theory questions with model answers
- Board exam day strategy: reading time, question selection, presentation
- How to handle unfamiliar questions, partial answer strategies
- Confidence building: you've practiced more than 95% of students
- Practical exam preparation: Pandas program + SQL queries + project viva

Deliverables:

- Quick revision notes, entire syllabus on 8-10 pages
- Pandas + Matplotlib reference sheet, 2 pages
- SQL reference sheet, 2 pages
- Theory revision sheet, 2 pages
- 11+ solved papers in practice
- READY FOR BOARD EXAM, 90+ score target

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Class 11 Sem 1): Python console programs - calculators, grade tools, list and dictionary applications - plus a full school database built and queried in MySQL
- Phase 2 (Class 11 Sem 2): the complete Class 11 practical file with 14+ Python programs and 14+ SQL queries, plus an emerging trends presentation project
- Phase 3 (Class 12 Sem 1): real-dataset analysis with Pandas Series and DataFrames, a CSV

- import-analyze-export pipeline, and 4+ fully customized Matplotlib charts (line, bar, histogram)
- Phase 4 (Class 12 Sem 2): advanced SQL query sets (single-row functions, GROUP BY, HAVING, joins) and the final board project combining Pandas, Matplotlib, and CSV data with documentation
- Total: 2 board-ready practical files, 2 submission projects, and dozens of graded programs and query sets

Total projects: 35+ graded programs across both years, including 2 complete board practical files (Pandas, Matplotlib, and SQL) and 2 board-submission projects

SKILLS THE STUDENT WILL HAVE AT THE END

- Python Basics: variables, operators, control flow, lists, dictionaries, and menu-driven programs
- Pandas: creating Series and DataFrames, loc and iloc selection, Boolean filtering, sorting, and aggregation
- Data Visualization: line charts, bar graphs, and histograms in Matplotlib with titles, labels, legends, and grids
- CSV Workflow: importing data with read_csv, cleaning and analyzing it, and exporting results with to_csv
- SQL on MySQL: DDL, DML, single-row and aggregate functions, GROUP BY, HAVING, ORDER BY, and joins
- Computer Networks: network types, devices, topologies, and internet basics as per the IP syllabus
- Societal Impacts: digital footprint, cyber safety, IPR, plagiarism, e-waste, and the IT Act
- Exam Technique: output prediction, query writing, and marking-scheme based answer writing

HOW A TYPICAL WEEK IS SPLIT

Live classes: 3-4 live classes per week, 1 hour each, in sync with the school's chapter sequence

Practical work: 2-3 hours of hands-on work: Pandas programs, Matplotlib charts, and SQL queries executed on a live MySQL database

Board style practice: Weekly chapter tests, output-prediction drills, query-writing practice, and answer writing trained on the CBSE marking scheme

Doubt support: Dedicated doubt-clearing sessions, with extra slots before school exams, pre-boards, and board exams

SUPPORT INCLUDED

Small batches: Small group batches so every student gets individual attention and screen time

Dedicated ip batch: An IP-only (code 065) batch - never mixed with CS (083) or ICSE students

Exam season support: Extra revision classes and doubt sessions before pre-boards and board exams

Practical and project help: Step-by-step help with every practical file entry and the board project until they are submission-ready

Progress tracking: Regular test results and progress updates shared with parents

CERTIFICATION DETAIL

Course certificate: Modern Age Coders Course Completion Certificate at the end of each academic year

Board exam readiness: Previous year papers, the official CBSE sample paper, and full mock exams on the current IP (065) pattern: 70 marks theory plus 30 marks practical

Practical file support: Complete guidance on the required practical file entries (15+ Pandas programs, 4+ Matplotlib charts, 15+ SQL queries), the board project, and viva preparation for the 30-mark practical exam

WHERE THIS CAN LEAD

- BCA, B.Sc IT, and B.Sc Data Science admissions with working Python and SQL skills
- BBA, B.Com, and economics programs where data analysis skills stand out
- Data analytics and business analytics degree specializations
- Data analyst internships that expect Pandas and SQL basics
- Business intelligence and reporting work later on - the SQL taught here transfers directly
- Statistics, journalism, and social science programs that increasingly use Python for data work
- A smooth upgrade path into full data science courses after Class 12

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Informatics Practices (IP)? Is it different from Computer Science (CS)?

A: Yes, IP (Code 065) and CS (Code 083) are different CBSE subjects. IP focuses on data handling with Pandas, data visualization with Matplotlib, and SQL, it's application-oriented and considered easier. CS focuses on deep Python programming, data structures (stacks), sorting/searching algorithms, and Python-MySQL connectivity, it's more programming-intensive.

We run separate batches for IP and CS students.

Q: Is IP easier than CS? Should I choose IP?

A: IP is generally considered easier because it focuses on using Python libraries (Pandas, Matplotlib) for data analysis rather than building algorithms from scratch. If you're interested in data science, business analytics, or want a higher-scoring subject, IP is a great choice. If you want to pursue software engineering or competitive programming, CS is better. We can help you decide based on your goals.

Q: Do you have separate batches for IP and CS?

A: Absolutely. IP and CS have completely different syllabi, different units, different topics, different exam patterns. Our IP batch only teaches IP content (Pandas, Matplotlib, SQL, networks, societal impacts). No mixing with CS topics like file handling, data structures, or Python-MySQL connectivity.

Q: Which textbook do you follow for IP?

A: We primarily follow Preeti Arora (Informatics Practices with Python), the most popular IP textbook, supplemented by NCERT IP textbook and Sumita Arora. Our teaching covers every chapter from these books.

Q: Do I need to install anything on my computer?

A: Yes, you'll need Python 3.12+ (with Pandas, NumPy, Matplotlib, installed via pip) and MySQL Community Server. We guide you through the complete installation process in the first few classes.

Q: Will you help with the practical file and project?

A: Yes, completely. We prepare your entire practical file (15+ Pandas programs, 4+ Matplotlib charts, 15+ SQL queries), guide you through a data analysis project with full documentation, and prepare you for the viva voce examination. You submit a polished, complete practical file.

Q: Can this course help me score 90+ in IP board exam?

A: IP has a more predictable question pattern than CS, making high scores very achievable with proper preparation.

Q: Is there an ICSE equivalent of IP?

A: No, ICSE/ISC does not have an Informatics Practices subject. ISC offers Computer Science (Java-based) which is more similar to CBSE CS. If you're an ICSE student interested in data analysis, you can join our IP batch to learn Pandas/Matplotlib skills alongside your board preparation, these are valuable skills regardless of your board.

Q: What career options does IP open up?

A: IP gives you a strong foundation for Data Analyst, Business Analyst, SQL Developer, Data Scientist, BI Developer, IT Manager, Digital Marketing Analyst, and Financial Analyst roles. The Python + SQL + data visualization skills you learn are directly used in industry. Many BCA, B.Sc IT, BBA, and MBA programs value these skills.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/cbse-informatics-practices-ip-class-11-12-python-pandas-sql-complete-course>

=====

COURSE 77 OF 118: JAVA PROGRAMMING FOR TEENS: ZERO TO PROFESSIONAL DEVELOPER

=====

Course link: <https://learn.modernagecoders.com/courses/java-programming-masterclass-for-teens>

Subject area: Professional Programming & Software Development

Designed for: Teens

Who can join: Complete Beginner to Advanced Professional

Course length: 24 months (104 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Industry-recognized Java Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Java course for teens: a 2-year journey from first Hello World to Android apps and enterprise projects, blending fun builds with professional developer skills.

In one line: From 'Hello World' to Building Your Own Apps, Games, and Enterprise Systems

ABOUT THE PROGRAMME

This isn't just another programming course, it's a complete transformation journey designed specifically for teenagers who want to master Java programming. Starting from absolute zero,

you'll learn to think like a programmer, build exciting projects like games and Android apps, and develop the skills needed for a successful career in software development. Java powers everything from Minecraft to Netflix, from Android apps to banking systems. Over 24 months, you'll master not just the language, but the entire ecosystem: desktop applications, web services, mobile apps, game development, and enterprise systems. With teen-friendly explanations, exciting projects, and real-world applications, you'll be building production-ready software long before graduation.

WHAT MAKES THIS COURSE DIFFERENT

- Designed specifically for teenage learners with engaging, relatable content
- Start from absolute zero - no math or coding background needed
- Build cool projects: games, apps, Minecraft mods, chatbots
- Learn professional skills while having fun
- Prepare for college CS courses and tech careers
- Includes Android app development and game programming
- Real internship and job preparation
- Build a portfolio that stands out for college applications

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Programming Basics, Java Fundamentals, Problem Solving

Phase 2: Mastery (Months 7-12): Object-Oriented Programming, GUI Applications, Data Structures

Phase 3: Professional (Months 13-18): Web Development, Spring Framework, Databases, APIs

Phase 4: Advanced (Months 19-24): Android Apps, Microservices, Cloud, Career Preparation

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - perfect for high school students

Coding experience: Absolute beginner friendly - starts from zero

Equipment: Computer/laptop with internet connection

Time commitment: 15-20 hours per week consistently

English: Basic reading and comprehension

Motivation: Passion for technology and problem-solving

WHO THIS COURSE IS FOR

Students: High school and college students building strong foundation

Working professionals: Career switchers entering tech

Entrepreneurs: Teen entrepreneurs wanting to build apps

Freelancers: Build freelance career with Java skills

Kids: Motivated teenagers (13+) ready for serious learning

Anyone: Anyone passionate about programming and technology

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE PROGRAMMING (MONTHS 1-6, WEEKS 1-26) -----

Build rock-solid programming fundamentals. Learn to think like a programmer, master Java basics, and create your first applications.

Months 1-2: Introduction to Programming & Java Basics (Week 1-8)

Introduction to Programming & Setup

Topics:

- What is programming? Why Java? Real-world Java applications
- How computers understand code: compilation and execution
- Java history: from coffee to code that runs everywhere
- Installing Java JDK and understanding versions
- Setting up IntelliJ IDEA (student edition)
- Your first Java program: Hello World explained
- Understanding main method and program structure
- Basic program flow and execution
- Using the command line to compile and run Java
- Introduction to debugging and reading error messages
- Code comments and documentation basics
- Programming mindset: how to think like a developer

Projects built:

- Set up complete Java development environment
- Create and run Hello World in 3 different ways
- Build a simple personal introduction program

Practice: Daily: 30 min coding exercises, 1 hour concept review

Variables, Data Types & Basic Operations

Topics:

- Variables: storing information in programs
- Primitive data types: int, double, boolean, char

- Declaring and initializing variables
- Naming conventions and best practices
- Arithmetic operators: +, -, *, /, %
- Order of operations (PEMDAS in programming)
- Type casting and conversion
- String basics: creating and concatenating
- Reading user input with Scanner class
- Formatting output with printf
- Constants and final keyword
- Common programming errors and how to fix them

Projects built:

- Interactive calculator application
- Temperature converter (Celsius/Fahrenheit/Kelvin)
- Simple cashier program with tax calculation
- Mad Libs story generator

Practice: Solve 30 problems on variables and operations

Control Flow: Making Decisions

Topics:

- Boolean logic: true and false
- Comparison operators: ==, !=, <, >, <=, >=
- If statements: making decisions in code
- If-else chains for multiple conditions
- Nested if statements
- Logical operators: && (AND), || (OR), ! (NOT)
- Switch statements for multiple choices
- Ternary operator for simple conditions
- Program flow visualization
- Debugging conditional logic
- Common logic errors and edge cases
- Writing readable conditional code

Projects built:

- Grade calculator with letter grades
- Simple adventure game with choices
- Rock-Paper-Scissors game
- Age-appropriate movie recommendation system

Practice: Build 10 programs using different conditional patterns

Loops: Repetition in Programming

Topics:

- Why loops? Automation and efficiency
- While loops: repeat while condition is true
- Do-while loops: execute at least once
- For loops: counting and iteration
- Loop control: break and continue
- Nested loops: loops within loops
- Common loop patterns
- Infinite loops and how to avoid them
- Enhanced for loop (for-each) preview
- Loop performance considerations
- Debugging loops effectively
- Accumulator and counter patterns

Projects built:

- Number guessing game with attempts
- Multiplication table generator
- Pattern printer (pyramids, diamonds)
- Simple password validator with retry
- Prime number checker

Practice: Solve 25 loop-based programming challenges

Months 3-4: Arrays, Methods & Problem Solving (Week 9-17)

Arrays: Working with Collections

Topics:

- Introduction to arrays: storing multiple values
- Declaring and initializing arrays
- Accessing array elements with indices
- Array length and bounds checking
- Iterating through arrays with loops
- Common array operations: sum, average, max, min
- Searching arrays: linear search
- Sorting arrays: bubble sort basics
- Two-dimensional arrays (matrices)
- Arrays of strings
- ArrayIndexOutOfBoundsException
- Arrays class utility methods

Projects built:

- Grade book system for a class
- Tic-Tac-Toe game board
- Student roster management
- Simple playlist manager
- Basic statistics calculator

Practice: Master 30 array manipulation problems

Methods: Building Reusable Code

Topics:

- What are methods? Why use them?
- Method structure: return type, name, parameters
- Void methods vs methods with return values
- Passing arguments to methods
- Method overloading: same name, different parameters
- Scope and local variables
- Pass by value in Java
- Recursive methods introduction
- Method documentation with Javadoc
- Testing methods independently
- Helper methods and code organization
- Static vs instance methods preview

Projects built:

- Calculator with method for each operation
- Text analysis toolkit (word count, etc.)
- Geometry calculator library
- Simple encryption/decryption program
- Menu-driven banking application

Practice: Create 20 useful utility methods

String Manipulation & Text Processing

Topics:

- String class deep dive
- String immutability concept
- Common String methods: length, charAt, substring
- String comparison: equals vs ==
- Case conversion: toUpperCase, toLowerCase
- Searching strings: indexOf, contains
- Replacing and splitting strings
- StringBuilder for efficient string building
- Character class utilities
- Regular expressions basics
- Parsing strings to numbers
- Formatting strings for output

Projects built:

- Password strength analyzer
- Palindrome checker
- Word game (like Wordle)
- Text-based email validator
- Simple markdown to HTML converter

Practice: Complete 25 string manipulation challenges

File Handling & Exception Basics

Topics:

- Reading from text files
- Writing to text files
- File class and file operations
- Scanner for file input
- PrintWriter for file output
- Try-catch blocks for error handling
- Common exceptions: IOException, FileNotFoundException
- Finally block usage
- Throwing exceptions
- Creating custom exceptions preview
- Best practices for file handling
- Resource management basics

Projects built:

- Note-taking application with file storage
- High score system for games
- Simple database using text files
- Log file analyzer
- CSV file reader/writer

Practice: Build 10 file-based applications

Month 3-4 Review & Mini Projects

Topics:

- Comprehensive review of arrays and methods
- Problem-solving strategies
- Code organization best practices
- Debugging complex programs
- Performance considerations
- Preparing for object-oriented programming

Projects built:

- CAPSTONE: Text-based RPG game with save system
- Student management system with file storage
- Personal finance tracker
- Quiz application with score tracking

Assessment: Take comprehensive test on Phase 1 fundamentals

Months 5-6: Introduction to Object-Oriented Programming (Week 18-26)

Classes and Objects Fundamentals

Topics:

- Object-Oriented Programming concepts
- Real-world objects vs programming objects
- Creating your first class
- Instance variables (attributes/fields)
- Methods (behaviors)
- Constructors: default and parameterized
- Creating objects with 'new' keyword
- The 'this' keyword
- Getters and setters (accessors/mutators)
- toString() method
- Object references vs primitive values
- Garbage collection basics

Projects built:

- Student class with grade management
- Bank account class with transactions
- Book library system
- Simple pet simulator
- Car dealership inventory

Practice: Create 15 different classes modeling real-world objects

Encapsulation and Access Modifiers

Topics:

- Information hiding principle
- Access modifiers: public, private, protected, default
- Encapsulation best practices
- Validation in setters
- Immutable objects
- Static variables and methods
- Static vs instance context
- Class constants with static final
- Factory methods pattern introduction
- Singleton pattern basics
- Package organization
- Import statements and classpath

Projects built:

- Secure user account system
- Inventory management with validation
- Employee management system
- Game character with stats
- Shopping cart with product catalog

Practice: Refactor projects with proper encapsulation

Inheritance: Building Class Hierarchies

Topics:

- Inheritance concept: is-a relationship
- Extending classes with 'extends'
- Super keyword and parent constructors
- Method overriding
- @Override annotation
- Protected access modifier
- Multi-level inheritance
- Object class as universal parent
- equals() and hashCode() methods
- Abstract classes introduction
- When to use inheritance vs composition
- Inheritance best practices

Projects built:

- Animal hierarchy with different species
- Vehicle inheritance system
- Employee hierarchy (Manager, Developer, etc.)
- Shape hierarchy for geometry
- Game entity system

Practice: Design 10 inheritance hierarchies

Polymorphism and Interfaces

Topics:

- Polymorphism: many forms
- Method overriding for polymorphism
- Dynamic method dispatch
- Abstract classes and methods
- Interfaces: contracts for classes
- Implementing multiple interfaces
- Interface default methods (Java 8+)
- Comparable and Comparator interfaces
- Polymorphic arrays and collections
- instanceof operator
- Type casting with objects
- Interface vs abstract class

Projects built:

- Payroll system with different employee types
- Media player for different file types
- Drawing application with shapes
- RPG game with character classes
- Plugin system using interfaces

Practice: Implement 15 different interfaces

Phase 1 Capstone Project

Topics:

- Project planning and design
- UML class diagrams basics
- Code organization and packages
- Documentation with Javadoc
- Testing and debugging OOP code
- Version control with Git basics

Projects built:

- MAJOR CAPSTONE: Complete Bank Management System
- Features: Accounts, transactions, loans, customers, employees
- Alternative: School Management System
- Alternative: Hotel Booking System
- Alternative: Mini Social Media Platform

Assessment: Phase 1 Final Exam - Complete OOP project from requirements

----- PHASE 2: ADVANCED JAVA & APPLICATION DEVELOPMENT (MONTHS 7-12, WEEKS 27-52) -----

Master advanced Java concepts, build desktop applications with JavaFX, explore game development, and dive into data structures and algorithms.

Months 7-8: Collections Framework & Advanced Java (Week 27-35)

Java Collections Framework

Topics:

- Collections vs arrays: when to use what
- Collection interface hierarchy
- List interface: ArrayList vs LinkedList
- Set interface: HashSet, TreeSet, LinkedHashSet
- Map interface: HashMap, TreeMap, LinkedHashMap
- Queue and Deque interfaces
- Iterator and ListIterator
- Collections utility class
- Sorting collections with Comparable/Comparator
- Searching and shuffling
- Synchronized collections
- Performance characteristics of collections

Projects built:

- Contact management system with HashMap
- Task scheduler with PriorityQueue
- Duplicate finder using Sets
- Student database with sorting
- Cache implementation with LinkedHashMap

Practice: Master 30 collection manipulation problems

Generics and Type Safety

Topics:

- Why generics? Type safety benefits
- Generic classes creation
- Generic methods
- Bounded type parameters
- Wildcards: ?, extends, super
- Type erasure concept
- Generic interfaces
- Multiple type parameters
- Generic collections deep dive
- Raw types and backwards compatibility
- Best practices with generics
- Common generic patterns

Projects built:

- Generic data structure library
- Type-safe cache implementation
- Generic DAO pattern
- Custom generic collections
- Builder pattern with generics

Practice: Create 15 generic classes and methods

Exception Handling & Debugging

Topics:

- Exception hierarchy in Java
- Checked vs unchecked exceptions
- Try-catch-finally deep dive
- Multi-catch blocks
- Try-with-resources statement
- Creating custom exception classes

- Exception chaining
- Best practices for exception handling
- Logging with java.util.logging
- Debugging techniques in IDE
- Using breakpoints effectively
- Stack traces and error analysis

Projects built:

- Robust file processing system
- Input validation framework
- Error handling library
- Retry mechanism implementation
- Logging system for applications

Practice: Handle exceptions in 20 different scenarios

Lambda Expressions & Stream API

Topics:

- Functional interfaces concept
- Lambda expression syntax
- Method references
- Built-in functional interfaces
- Stream API introduction
- Creating streams from collections
- Intermediate operations: filter, map, sorted
- Terminal operations: collect, reduce, forEach
- Parallel streams
- Optional class for null safety
- Stream performance considerations
- Functional programming in Java

Projects built:

- Data processing pipeline
- Report generator with streams
- Functional calculator
- Stream-based file analyzer
- Employee data analysis system

Practice: Solve 25 problems using streams and lambdas

Multithreading Basics

Topics:

- Thread concept and lifecycle
- Creating threads: Thread class vs Runnable
- Thread priorities and scheduling
- Thread synchronization basics
- Synchronized keyword
- wait() and notify() methods
- Deadlock and how to avoid it
- Thread pools introduction
- Callable and Future
- Concurrent collections
- Thread safety principles
- Common threading problems

Projects built:

- Multi-threaded download manager
- Producer-consumer implementation
- Concurrent counter application
- Thread pool executor demo
- Race condition simulator

Practice: Build 10 thread-safe applications

Months 9-10: GUI Development with JavaFX (Week 36-44)

JavaFX Fundamentals

Topics:

- JavaFX vs Swing: modern GUI development
- JavaFX application structure
- Stage, Scene, and Scene Graph
- Layouts: VBox, HBox, GridPane, BorderPane
- Basic controls: Button, Label, TextField
- Event handling in JavaFX
- CSS styling for JavaFX
- FXML for UI design
- Scene Builder tool
- Images and media in JavaFX
- Menus and toolbars
- Dialogs and alerts

Projects built:

- Calculator with GUI
- Note-taking application
- Login screen with validation
- Image viewer application
- Simple text editor

Practice: Build 15 different GUI layouts

Advanced JavaFX Controls

Topics:

- TableView for data display
- TreeView for hierarchical data
- ListView and observableList
- ComboBox and ChoiceBox
- DatePicker and ColorPicker
- Charts: PieChart, BarChart, LineChart
- WebView for HTML content
- Custom controls creation
- Binding properties
- Animations and transitions
- Drag and drop functionality
- Keyboard shortcuts

Projects built:

- Student database with TableView
- File explorer with TreeView
- Data visualization dashboard
- Media player interface
- Calendar application

Practice: Master all JavaFX controls

MVC Pattern & Application Architecture

Topics:

- Model-View-Controller pattern
- Separating concerns in JavaFX
- Observable pattern in JavaFX
- Data binding deep dive
- MVVM pattern alternative
- Application state management
- Navigation between screens
- Dependency injection basics
- Service layer pattern
- DAO pattern implementation
- Configuration management
- Application packaging

Projects built:

- Complete MVC application
- Multi-screen navigation app
- Inventory management system
- Point of Sale (POS) system
- Library management GUI

Practice: Refactor projects using MVC

Database Integration

Topics:

- JDBC basics and drivers
- Connecting to databases
- SQL review for Java developers
- PreparedStatement vs Statement
- ResultSet processing
- Transaction management
- Connection pooling
- DAO pattern with JDBC
- Database design basics
- SQLite for desktop apps
- MySQL/PostgreSQL integration
- Database migrations

Projects built:

- Database-backed todo application
- Customer management system
- Inventory with database
- Expense tracker with reports
- Student information system

Practice: Integrate databases in 10 applications

Game Development Basics

Topics:

- Game loop concept
- Sprite handling and animation
- Collision detection
- Game physics basics
- Sound effects and music
- Game state management
- Score and lives system
- Level design principles
- 2D graphics with Canvas
- Input handling for games
- Game optimization techniques
- Packaging games for distribution

Projects built:

- Snake game implementation
- Breakout/Brick breaker game

- Simple platformer
- Tower defense game
- Puzzle game like 2048

Practice: Create 5 different game genres

Months 11-12: Data Structures & Algorithms (Week 45-52)

Fundamental Data Structures

Topics:

- Abstract Data Types (ADT)
- Implementing dynamic arrays
- Linked Lists: single, double, circular
- Stack implementation and applications
- Queue implementation and variants
- Priority queues and heaps
- Hash tables from scratch
- Collision resolution strategies
- Trees: binary trees basics
- Tree traversal: inorder, preorder, postorder
- Binary search trees
- Balanced trees introduction

Projects built:

- Custom ArrayList implementation
- Expression evaluator using stacks
- Task scheduler with priority queue
- Spell checker with hash table
- File system simulator with trees

Practice: Implement 20 data structures from scratch

Searching and Sorting Algorithms

Topics:

- Linear search and optimizations
- Binary search and variations
- Bubble sort and optimizations
- Selection sort
- Insertion sort
- Merge sort and divide-conquer
- Quick sort and partitioning
- Heap sort
- Counting sort and radix sort
- Sorting stability
- Time complexity analysis
- Space complexity

Projects built:

- Sorting visualizer application
- Search engine for text files
- Music playlist sorter
- Leaderboard system
- Data analysis tool

Practice: Implement and analyze 15 algorithms

Graph Algorithms & Advanced Topics

Topics:

- Graph representations: adjacency matrix/list
- Breadth-first search (BFS)
- Depth-first search (DFS)
- Shortest path: Dijkstra's algorithm
- Minimum spanning tree
- Topological sorting
- Dynamic programming introduction
- Recursion and backtracking
- Greedy algorithms
- String algorithms basics
- Pattern matching
- Algorithm design techniques

Projects built:

- Social network analyzer
- Route finder application
- Maze solver with visualization
- Sudoku solver
- Word search puzzle solver

Practice: Solve 30 algorithmic problems

Performance & Optimization

Topics:

- Profiling Java applications
- JVM memory model
- Garbage collection tuning
- Memory leak detection
- CPU profiling
- Optimizing algorithms
- Cache-friendly code
- Benchmark writing with JMH

- Space-time tradeoffs
- Parallel algorithms basics
- When to optimize
- Performance best practices

Projects built:

- Performance comparison tool
- Memory-efficient data processor
- Algorithm benchmark suite
- Optimization case study
- Profiling report generator

Practice: Optimize 10 slow programs

Phase 2 Capstone Project

Topics:

- Complex application design
- Algorithm selection
- Performance requirements
- User interface design
- Database design
- Testing strategies

Projects built:

- MAJOR CAPSTONE: Complete Desktop Application Suite
- Options: IDE for programming, Music player like Spotify, Photo editor, Strategy game, Educational platform
- Requirements: GUI, database, algorithms, file handling, multithreading

Assessment: Phase 2 Final Exam - Build complex Java application

----- PHASE 3: ENTERPRISE JAVA & WEB DEVELOPMENT (MONTHS 13-18, WEEKS 53-78) -----

Master enterprise Java development with Spring Framework, build RESTful APIs, microservices, and web applications.

Months 13-14: Web Development & Servlets (Week 53-61)

Web Fundamentals for Java Developers

Topics:

- How the web works: HTTP protocol
- Client-server architecture
- Request-response cycle
- HTTP methods: GET, POST, PUT, DELETE
- Status codes and headers
- HTML/CSS basics for Java developers
- JavaScript basics for full-stack
- JSON and XML data formats
- RESTful API principles
- Web servers vs application servers
- Tomcat server setup
- Web application structure

Projects built:

- Simple HTTP client in Java
- Static website with Java server
- JSON parser implementation
- REST client application
- Web scraper in Java

Practice: Build 10 web-related Java programs

Java Servlets & JSP

Topics:

- Servlet API and lifecycle
- HttpServlet class
- Handling GET and POST requests
- Request and response objects
- Session management
- Cookies in Java web apps
- Filters and listeners
- JSP basics and syntax
- JSP directives and actions
- Expression Language (EL)
- JSTL tag library
- MVC with Servlets and JSP

Projects built:

- User authentication system
- Shopping cart with sessions
- Blog platform with JSP
- Forum application
- Online quiz system

Practice: Create 15 servlet-based features

Spring Framework Fundamentals

Topics:

- Spring Framework overview
- Dependency Injection (DI)

- Inversion of Control (IoC)
- Spring container and beans
- XML vs annotation configuration
- Component scanning
- @Component, @Service, @Repository
- @Autowired and dependency injection
- Bean scopes and lifecycle
- Spring profiles
- Properties and configuration
- ApplicationContext

Projects built:

- Spring-based console application
- Dependency injection examples
- Service layer with Spring
- Configuration management app
- Modular application design

Practice: Convert projects to Spring

Spring Boot Essentials

Topics:

- Spring Boot vs Spring Framework
- Spring Initializr and project setup
- Auto-configuration magic
- Embedded servers
- application.properties/yml
- Spring Boot starters
- DevTools for development
- Actuator for monitoring
- Logging in Spring Boot
- Profiles and environments
- Building executable JARs
- Spring Boot CLI

Projects built:

- Hello World REST API
- Configuration server
- Monitoring dashboard
- Multi-profile application
- Microservice template

Practice: Build 10 Spring Boot applications

RESTful Web Services

Topics:

- @RestController annotation
- @RequestMapping and variants
- Path variables and request parameters
- Request body and response body
- Content negotiation
- Exception handling in REST
- Validation with Bean Validation
- HATEOAS principles
- API versioning strategies
- Swagger/OpenAPI documentation
- Testing REST APIs
- RestTemplate and WebClient

Projects built:

- Complete REST API for blog
- E-commerce product API
- User management API
- File upload service
- API gateway implementation

Practice: Design 10 RESTful services

Months 15-16: Database & Security (Week 62-70)

Spring Data JPA

Topics:

- JPA and Hibernate basics
- Entity mapping with annotations
- @Entity, @Table, @Column
- Primary keys and generation strategies
- Relationships: @OneToMany, @ManyToOne
- @ManyToMany, @OneToOne
- Spring Data repositories
- Query methods
- Custom queries with @Query
- Pagination and sorting
- Transactions in Spring
- Lazy vs eager loading

Projects built:

- Blog with comments system
- E-commerce with products and orders
- Social media data model

- School management system
- Hospital management database

Practice: Create 20 JPA entities and relationships

Advanced Database Operations

Topics:

- Database migrations with Flyway/Liquibase
- Query optimization
- N+1 problem solutions
- Caching with Spring Cache
- Redis integration
- MongoDB with Spring Data
- Multiple datasources
- Database connection pooling
- Batch operations
- Stored procedures with JPA
- Auditing with Spring Data
- Database testing strategies

Projects built:

- Multi-tenant application
- Caching layer implementation
- Data migration tool
- Analytics dashboard
- Audit logging system

Practice: Optimize 10 database operations

Spring Security

Topics:

- Authentication vs authorization
- Spring Security architecture
- In-memory authentication
- Database authentication
- Password encoding
- Form-based login
- Remember me functionality
- CSRF protection
- Method-level security
- Role-based access control
- JWT authentication
- OAuth2 integration

Projects built:

- Secure REST API
- Multi-user blog platform
- Banking application security
- OAuth2 social login
- JWT-based authentication

Practice: Secure 10 different applications

Messaging & Integration

Topics:

- Message-driven architecture
- JMS basics
- RabbitMQ with Spring
- Apache Kafka integration
- Spring Integration basics
- Email sending with Spring
- Scheduled tasks
- Async processing
- WebSocket with Spring
- Server-sent events
- File processing pipelines
- Third-party API integration

Projects built:

- Notification service
- Order processing system
- Real-time chat application
- Email campaign manager
- Data pipeline processor

Practice: Build 10 integration scenarios

Testing Spring Applications

Topics:

- Unit testing with JUnit 5
- Mockito for mocking
- Spring Boot Test
- @MockBean and @SpyBean
- Testing REST controllers
- Testing service layer
- Testing data layer
- Integration testing
- TestContainers
- Test coverage with JaCoCo
- Performance testing

- Contract testing

Projects built:

- Complete test suite for API
- TDD practice project
- Integration test framework
- Performance test suite
- Automated testing pipeline

Practice: Write tests for all projects

Months 17-18: Microservices & Cloud (Week 71-78)

Microservices Architecture

Topics:

- Monolith to microservices
- Microservices principles
- Service decomposition
- Spring Cloud overview
- Service discovery with Eureka
- Load balancing with Ribbon
- Circuit breaker with Hystrix
- API Gateway with Zuul/Gateway
- Configuration server
- Distributed tracing with Sleuth
- Service mesh concepts
- Event-driven microservices

Projects built:

- E-commerce microservices
- Service discovery implementation
- API gateway setup
- Distributed configuration
- Circuit breaker demo

Practice: Build 5 microservice systems

Docker & Containerization

Topics:

- Docker fundamentals
- Creating Dockerfiles for Java
- Multi-stage builds
- Docker Compose for development
- Container networking
- Volume management
- Docker registry
- Best practices for Java containers
- JVM in containers
- Container security
- Docker Swarm basics
- Container orchestration intro

Projects built:

- Dockerize Spring Boot apps
- Multi-container application
- Development environment setup
- CI/CD with Docker
- Container monitoring

Practice: Containerize all applications

Kubernetes & Cloud Deployment

Topics:

- Kubernetes concepts
- Pods, Services, Deployments
- ConfigMaps and Secrets
- Persistent volumes
- Ingress controllers
- Helm charts
- Scaling applications
- Health checks
- Rolling updates
- AWS basics for Java
- Deploying to cloud
- Monitoring and logging

Projects built:

- Deploy to Kubernetes
- Auto-scaling setup
- Blue-green deployment
- Cloud-native application
- Monitoring dashboard

Practice: Deploy 5 apps to cloud

DevOps & CI/CD

Topics:

- DevOps principles
- Git branching strategies
- Maven/Gradle deep dive
- Jenkins pipelines

- GitHub Actions for Java
- GitLab CI/CD
- Automated testing in CI
- Code quality with SonarQube
- Artifact management
- Release strategies
- Infrastructure as Code
- Monitoring production apps

Projects built:

- Complete CI/CD pipeline
- Automated deployment system
- Quality gates setup
- Multi-environment pipeline
- Monitoring and alerting

Practice: Set up DevOps for projects

Phase 3 Capstone Project

Topics:

- Enterprise architecture design
- Microservices design
- API design and documentation
- Security implementation
- Cloud deployment
- Performance optimization

Projects built:

- MAJOR CAPSTONE: Enterprise E-commerce Platform
- Features: Microservices, authentication, payment, inventory, orders, notifications
- Alternative: Banking System with transactions, accounts, loans
- Alternative: Healthcare Management System

Assessment: Phase 3 Final Exam - Build enterprise application

PHASE 4: MOBILE, ADVANCED TOPICS & CAREER (MONTHS 19-24, WEEKS 79-104)

Master Android development, explore advanced Java topics, contribute to open source, and prepare for professional career.

Months 19-20: Android Development (Week 79-87)

Android Fundamentals

Topics:

- Android ecosystem overview
- Android Studio setup
- Project structure
- Activities and lifecycle
- Layouts: LinearLayout, RelativeLayout
- ConstraintLayout
- Views and ViewGroups
- Handling user input
- Intents and navigation
- Resources and localization
- Debugging Android apps
- Android Virtual Device

Projects built:

- Hello World Android app
- Calculator app
- Unit converter
- Simple notes app
- Quiz application

Practice: Build 10 basic Android apps

Android UI & User Experience

Topics:

- Material Design principles
- RecyclerView for lists
- CardView layouts
- Navigation Drawer
- Bottom Navigation
- Tabs with ViewPager
- Custom views
- Animations and transitions
- Themes and styles
- Dark mode support
- Responsive layouts
- Accessibility features

Projects built:

- News reader app
- Social media feed UI
- E-commerce app interface
- Music player UI
- Photo gallery app

Practice: Design 10 beautiful UIs

Android Data & Networking

Topics:

- SharedPreferences
- SQLite databases
- Room persistence library
- Content Providers
- Retrofit for REST APIs
- OkHttp client
- JSON parsing with Gson
- Image loading with Glide/Picasso
- Background tasks
- WorkManager
- Services and BroadcastReceivers
- Notifications

Projects built:

- Todo app with database
- Weather app with API
- Chat application
- Expense tracker
- Recipe app with API

Practice: Integrate 10 different APIs

Advanced Android Development

Topics:

- MVVM architecture pattern
- LiveData and ViewModel
- Data Binding
- Dependency injection with Dagger/Hilt
- Kotlin for Android basics
- Firebase integration
- Push notifications with FCM
- Google Maps integration
- Camera and media
- Sensors and location
- In-app purchases
- App monetization

Projects built:

- Social media app
- Location-based app
- Firebase chat app
- Fitness tracker
- Complete e-commerce app

Practice: Build production-ready apps

Android Testing & Publishing

Topics:

- Unit testing Android apps
- Espresso for UI testing
- Testing ViewModels
- Mocking with Mockito
- Performance optimization
- ProGuard and app size
- App signing
- Google Play Console
- App Store Optimization
- Crash reporting
- Analytics integration
- App updates and versioning

Projects built:

- Test suite for app
- Performance optimization
- Publish app to Play Store
- Analytics dashboard
- A/B testing setup

Practice: Publish 3 apps to store

Months 21-22: Advanced Java & Specializations (Week 88-96)

Reactive Programming

Topics:

- Reactive programming principles
- Project Reactor basics
- Mono and Flux
- Operators and transformations
- Error handling in reactive
- Backpressure handling
- Spring WebFlux
- Reactive REST APIs
- Reactive data access
- WebSocket with reactive
- Testing reactive code
- Performance benefits

Projects built:

- Reactive REST API
- Real-time dashboard
- Streaming data processor
- Reactive chat server
- Event-driven system

Practice: Convert apps to reactive

Performance & JVM Internals

Topics:

- JVM architecture deep dive
- Class loading mechanism
- Memory areas in JVM
- Garbage collection algorithms
- GC tuning strategies
- JVM flags and options
- Memory leak analysis
- Thread dumps analysis
- Heap dumps and MAT
- JProfiler and profiling
- JMH benchmarking
- Performance patterns

Projects built:

- Performance analyzer tool
- Memory leak detector
- GC monitoring dashboard
- Benchmark suite
- JVM tuning guide

Practice: Optimize 10 applications

Design Patterns & Architecture

Topics:

- Gang of Four patterns review
- Creational patterns deep dive
- Structural patterns deep dive
- Behavioral patterns deep dive
- Enterprise patterns
- Domain-Driven Design basics
- Clean Architecture
- Hexagonal Architecture
- Event Sourcing
- CQRS pattern
- Saga pattern
- Anti-patterns to avoid

Projects built:

- Pattern implementation library
- DDD sample application
- Event-sourced system
- CQRS implementation
- Architecture refactoring

Practice: Apply patterns in projects

Big Data & Machine Learning

Topics:

- Big Data concepts
- Apache Spark with Java
- Hadoop basics
- Stream processing with Kafka Streams
- Apache Flink introduction
- Machine Learning basics
- Weka library for ML
- Deep Learning with DL4J
- Natural Language Processing
- Computer Vision basics
- ML model deployment
- AI integration in Java apps

Projects built:

- Data processing pipeline
- ML classification system
- Recommendation engine
- Sentiment analysis tool
- Image recognition app

Practice: Build 5 ML/Big Data projects

AI & Emerging Tech with Java

Topics:

- Calling an AI API from Java
- Structured output: making the model return real objects
- Function calling and tool use in a typed language
- Retrieval over enterprise data
- Concurrency: many AI calls without melting the server
- Guardrails and validating model output
- Cost, caching and quotas

- IoT with Java: sensors and device control
- Streaming data and event-driven design
- Where Java is actually used for AI (and where Python wins)
- Testing non-deterministic systems
- Ethics and accountability in automated decisions

Projects built:

- AI service in Java
- Structured-output parser
- Tool-using agent
- IoT device controller
- Emerging tech demo

Practice: Explore new technologies

Month 23: Open Source & Advanced Projects (Week 97-100)

Open Source Contribution

Topics:

- Open source ecosystem
- Finding projects to contribute
- Understanding large codebases
- Git advanced workflows
- Creating quality pull requests
- Code review etiquette
- Documentation contributions
- Creating your own library
- Publishing to Maven Central
- Building community
- Open source licensing
- Maintaining projects

Projects built:

- Contribute to 3 projects
- Create open source library
- Documentation improvements
- Bug fixes in popular projects
- Feature implementation

Practice: Daily open source activity

Minecraft Modding

Topics:

- Minecraft Forge setup
- Mod structure and basics
- Creating blocks and items
- Crafting recipes
- Custom entities
- World generation
- GUI creation
- Networking in mods
- Mod configuration
- Testing and debugging
- Publishing mods
- Fabric alternative

Projects built:

- Basic items mod
- New dimension mod
- Technology mod
- Adventure mod
- Complete modpack

Practice: Create 5 Minecraft mods

Game Development Advanced

Topics:

- LibGDX framework
- Game architecture patterns
- Physics engines
- Particle systems
- Procedural generation
- AI for games
- Multiplayer networking
- Game optimization
- Mobile game development
- Publishing games
- Game monetization
- LWJGL for 3D games

Projects built:

- 2D platformer game
- Strategy game
- Multiplayer game
- 3D game prototype
- Mobile game port

Practice: Complete game portfolio

System Design & Architecture

Topics:

- Large-scale system design
- Distributed systems principles
- CAP theorem and tradeoffs
- Consistent hashing
- Load balancing strategies
- Caching at scale
- Database sharding
- Message queue architectures
- Real-time systems
- Search systems design
- Monitoring and observability
- Disaster recovery

Projects built:

- Design Twitter clone
- Design YouTube
- Design Uber
- Design WhatsApp
- Design Netflix

Practice: System design interviews

Month 24: Final Projects & Career Launch (Week 101-104)

Final Capstone Project - Part 1

Topics:

- Project ideation and planning
- Market research and validation
- Technical architecture design
- Technology stack selection
- Development methodology
- Team collaboration setup
- Project timeline creation
- Risk assessment and mitigation

Projects built:

- FINAL CAPSTONE: Complete Software Product
- Options: SaaS platform, Mobile app ecosystem, Enterprise system, Game with backend, Educational platform, Social network, Fintech application, Healthcare system

Final Capstone Project - Part 2

Topics:

- Code implementation
- Testing and quality assurance
- Performance optimization
- Security hardening
- Documentation creation
- Deployment pipeline
- User acceptance testing
- Launch preparation
- Marketing materials
- Demo video creation

Deliverables:

- Complete source code
- Live production application
- Technical documentation
- User documentation
- Architecture diagrams
- Test coverage reports
- Performance benchmarks
- Security audit results

Career Launch & Professional Development

Topics:

- Resume optimization for Java developers
- Portfolio website creation
- LinkedIn profile optimization
- GitHub profile showcase
- Technical interview preparation
- Data structures review
- Algorithm problem solving
- System design interviews
- Behavioral interview prep
- Salary negotiation
- Job search strategies
- Continuous learning plan

Deliverables:

- Professional resume
- Portfolio website
- LinkedIn profile
- GitHub portfolio
- Interview prep materials
- Career roadmap
- Professional network

Assessment: FINAL COMPREHENSIVE EXAM - Complete Java mastery assessment

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Calculator, Banking System, Text Games, File Manager, Student Database
- Phase 2: Desktop Apps with JavaFX, Game Development, Data Structure Library, Algorithm Visualizer
- Phase 3: REST APIs, Microservices, E-commerce Platform, Social Media Backend, Cloud Applications
- Phase 4: Android Apps, Minecraft Mods, Open Source Contributions, Complete Software Products

Total projects: 75+ projects from simple to enterprise-level

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Java: OOP, Collections, Generics, Lambdas, Streams, Multithreading, File I/O
- Advanced Java: Reflection, Annotations, JVM Internals, Performance Tuning, Design Patterns
- Enterprise: Spring Framework, Spring Boot, Microservices, REST APIs, Security, JPA/Hibernate
- Databases: JDBC, SQL, NoSQL, MongoDB, Redis, Database Design, Optimization
- Frontend: JavaFX, Swing, Android Development, UI/UX Design
- Testing: JUnit, Mockito, Integration Testing, TDD, Performance Testing
- DevOps: Git, Maven/Gradle, Docker, Kubernetes, CI/CD, Cloud Deployment
- Specialized: Game Development, Big Data, Machine Learning, Blockchain, Open Source

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-8 hours
Hands on coding: 8-10 hours
Projects: 3-5 hours
Practice problems: 2-3 hours
Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly doubt clearing sessions
Mentorship: 1-on-1 mentor guidance
Community: Active Discord/Slack community
Code review: Expert code reviews for projects
Career support: Resume review, interview prep, job referrals
Lifetime access: All content, updates, and new modules

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion
Final certificate: Professional Java Developer Certification
Linkedin badge: Add to LinkedIn profile
Industry recognized: Backed by industry professionals
Portfolio projects: 25+ showcase projects

WHERE THIS CAN LEAD

- Java Developer (Junior to Senior)
- Android Developer
- Full Stack Java Developer
- Backend Engineer
- Software Architect
- Game Developer
- DevOps Engineer
- Machine Learning Engineer
- Freelance Developer
- Technical Founder/CTO

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build while learning Java?

A: The 2-year path goes from first console programs to desktop applications with JavaFX, data structures and algorithms, web services, database-backed systems, and Android apps in the final phase. Java is the language behind Minecraft, Android, and large banking systems, and the projects reflect that range.

Q: Who should take this course, and is experience needed?

A: It is also a natural fit for students whose school boards teach Java, such as ICSE and ISC, since class work and school work reinforce each other.

Q: Should my teen learn Java or Python first?

A: Both are excellent first languages. Java's structure and strong typing teach disciplined thinking that transfers everywhere, it is the base of Android development, and several school curricula require it. If your teen's school teaches Java or they dream of building Android

apps, this course is the better starting point.

Q: Is any paid software required?

A: No. The Java Development Kit and the editor used in class are free, and a standard computer with an internet connection handles the entire course.

Q: What are the next steps after this masterclass?

A: The later phases already include Android development and enterprise topics, so graduates can go deeper with our data structures and algorithms course for competitive programming and interviews, or broaden into full stack web development. The 2-year portfolio of Java projects becomes the anchor for whichever direction they choose.

Q: Can we book a demo class?

A: Yes, demo classes are free and carry no commitment; reserve one at learn.modernagecoders.com/contact. Parents can sit in, and we hold a 4.9 star rating from 547+ Google reviews if you would like to read what other families say first.

Q: Is the teaching live or recorded?

A: Every class is live: a 1-hour online session where a real teacher works with a small group, never a pre-recorded video. Students code along in class and get corrections on the spot, which is especially important in Java's early learning curve.

Q: Is there a certificate on completion?

A: Yes, students earn a Java developer completion certificate. It comes with a portfolio of working projects, from games and desktop apps to Android work, that demonstrates the skill behind the paper.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/java-programming-masterclass-for-teens>

=====

COURSE 78 OF 118: JAVA PROGRAMMING: CORE JAVA, SPRING BOOT & MICROSERVICES

=====

Course link: <https://learn.modernagecoders.com/courses/complete-java-programming-masterclass-college>

Subject area: Professional Java Development

Designed for: College students

Who can join: Complete Beginner to Enterprise Professional

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Industry-recognized Java Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online Java course for college students: a 1-year path from Core Java to Spring Boot, microservices, Hibernate, databases and cloud deployment.

In one line: From 'Hello World' to Enterprise Microservices Architecture

ABOUT THE PROGRAMME

This is not just a Java course, it's a complete transformation into an enterprise-ready Java professional. Whether you're a curious beginner, student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled Java developer capable of building scalable enterprise applications, microservices, RESTful APIs, and cloud-native solutions. You'll master Java from ground zero to enterprise architect level: from basic syntax to advanced design patterns, from console applications to Spring Boot microservices, from JDBC to Hibernate ORM, from monolithic apps to cloud-native distributed systems. By the end, you'll have built 40+ projects, mastered the entire Java ecosystem, and be ready for senior Java developer roles in top tech companies.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - perfect for complete beginners
- Separate learning tracks for kids (12+), teens, and adults
- 1 year of structured, industry-aligned learning
- Covers Core Java + Enterprise Java (Spring, Hibernate, Microservices)
- Real enterprise projects and industry case studies
- Hands-on with latest Java versions (Java 17+)
- Interview preparation for top companies
- Lifetime access with continuous updates
- Build production-ready enterprise portfolio
- Direct path to high-paying Java developer jobs

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Java Fundamentals, OOP Basics, Core Java Concepts
Phase 2: Intermediate (Months 4-6): Advanced OOP, Collections, Multithreading, Design Patterns
Phase 3: Enterprise (Months 7-9): Spring Framework, Spring Boot, Hibernate, REST APIs, Databases
Phase 4: Professional (Months 10-12): Microservices, Spring Cloud, Testing, DevOps, Cloud Deployment

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to all backgrounds
Coding experience: Absolute beginner friendly - zero programming knowledge required
Age: 12+ years (separate learning tracks for kids, teens, adults)
Equipment: Computer/laptop (Windows/Mac/Linux), minimum 8GB RAM recommended, internet connection
Time commitment: 15-20 hours per week consistently for best results
English: Basic reading comprehension (materials also available in Hindi)
Motivation: Strong desire to become a professional Java developer
Math: Basic mathematics (taught as needed)

WHO THIS COURSE IS FOR

Kids: Age 12-14: Simplified track with visual learning, game-based projects, OOP with real-world examples
Teens: Age 15-18: Student-focused track, college prep, competitive programming, project-based learning
Students: College students: Campus placement preparation, internship readiness, strong portfolio building
Working professionals: Career switchers: Structured path from any background to Java developer role
Developers: Developers from other languages wanting to master Java ecosystem
Backend engineers: Backend developers wanting to specialize in enterprise Java
Entrepreneurs: Build your own products using scalable Java technologies
Freelancers: Offer enterprise Java development services to high-paying clients
Architects: Learn to design large-scale distributed systems
Anyone: Anyone passionate about backend development and enterprise software

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE JAVA SKILLS (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid Java fundamentals. Learn programming logic, master Java syntax, OOP principles, and create your first applications.

Months 1-2: Java Fundamentals & Programming Basics (Week 1-8)

Introduction to Java & Development Environment Setup

Topics:

- What is Java? History, features, and platform independence
- Java ecosystem: JDK, JRE, JVM explained
- Write Once Run Anywhere (WORA) concept
- Java applications: Desktop, Web, Mobile (Android), Enterprise
- Installing JDK (Java 17 LTS or latest)
- Setting up development environment: IntelliJ IDEA, Eclipse, VS Code
- Understanding IDE features and shortcuts
- Your first Java program: Hello World
- Java program structure: class, main method
- Compiling and running Java programs
- Understanding bytecode and JVM
- Comments: single-line (//), multi-line (/* */), Javadoc (/** */)
- Java coding conventions and naming standards

Projects built:

- Hello World variations with personalized messages
- Simple output programs using System.out.println
- Personal information display program
- ASCII art generator in Java

Practice: Daily: 30 min Java syntax practice, write 5-10 simple programs

Variables, Data Types & Operators

Topics:

- Variables: declaration, initialization, naming rules
- Primitive data types: byte, short, int, long, float, double, char, boolean
- Understanding type sizes and ranges
- Type casting: implicit (widening) and explicit (narrowing)
- Wrapper classes: Integer, Double, Character, Boolean, etc.
- Autoboxing and unboxing

- Literals: integer, floating-point, character, string, boolean
- Operators: arithmetic (+, -, *, /, %, ++, --)
- Relational operators: ==, !=, >, <, >=, <=
- Logical operators: &&, ||, !
- Assignment operators: =, +=, -=, *=, /=, %=
- Bitwise operators: &, |, ^, ~, <<, >>, >>>
- Ternary operator: ? :
- Operator precedence and associativity
- String basics: creation, concatenation, immutability

Projects built:

- Simple calculator (four basic operations)
- Temperature converter (Celsius, Fahrenheit, Kelvin)
- Age calculator with detailed output
- BMI calculator with health categories
- Currency converter
- Area and perimeter calculator for shapes
- Compound interest calculator

Practice: Solve 30 problems on variables, data types, and operators

Control Flow Statements

Topics:

- Boolean expressions and conditions
- If statement: single condition execution
- If-else statement: two-way branching
- If-else-if ladder: multiple conditions
- Nested if statements
- Switch statement: multi-way branching
- Switch with fall-through and break
- Enhanced switch (Java 12+): arrow syntax, yield
- While loop: pre-tested loop
- Do-while loop: post-tested loop
- For loop: counter-based iteration
- Enhanced for loop (for-each): iterating arrays/collections
- Break statement: exit loop
- Continue statement: skip iteration
- Labeled break and continue
- Nested loops and loop patterns

Projects built:

- Number guessing game with attempts limit
- Grade calculator with letter grades
- Even/odd checker and list generator
- Prime number checker and generator
- Multiplication table generator
- Factorial calculator (iterative)
- Fibonacci series generator
- Pattern printing (stars, numbers, pyramids, diamonds)
- FizzBuzz challenge
- Menu-driven calculator with switch
- Simple ATM simulation

Practice: Solve 40 control flow and loop problems

Methods & Arrays

Topics:

- Methods: definition, purpose, and structure
- Method declaration: return type, name, parameters
- Method invocation and calling
- Parameters vs arguments
- Return statement and return values
- Method overloading: same name, different parameters
- Variable scope: local vs instance vs class
- Static methods vs instance methods
- Recursion: base case and recursive case
- Arrays: declaration, creation, initialization
- Array indexing and accessing elements
- Array length property
- Iterating arrays: for loop, enhanced for loop
- Multi-dimensional arrays (2D, 3D)
- Jagged arrays
- Arrays class utility methods
- Variable arguments (varargs)

Projects built:

- Method library for mathematical operations
- Array statistics calculator (sum, average, min, max)
- Linear search and binary search implementation
- Sorting algorithms: bubble sort, selection sort
- Matrix operations (addition, multiplication)
- Tic-tac-toe game with 2D array
- Student grade management system
- Factorial using recursion
- Tower of Hanoi recursive solution
- Palindrome checker (string/number)

- Prime number generator in range

Practice: Create 25 methods for various functionalities, solve 30 array problems

Month 3: Object-Oriented Programming Fundamentals (Week 9-13)

Classes, Objects & OOP Basics

Topics:

- Object-Oriented Programming paradigm
- Classes: blueprint for objects
- Objects: instances of classes
- Class declaration and structure
- Instance variables (attributes/fields)
- Instance methods (behaviors)
- Creating objects with 'new' keyword
- Constructors: default and parameterized
- Constructor overloading
- this keyword: referencing current object
- Access modifiers: private, public, protected, default
- Encapsulation: data hiding principle
- Getters and setters (accessor and mutator methods)
- toString() method for object representation
- Object initialization blocks
- Static variables (class variables)
- Static methods and static blocks
- Static import

Projects built:

- Student class with attributes and methods
- Bank Account class with deposit/withdraw
- Book class for library management
- Employee class with salary calculations
- Rectangle/Circle classes with area/perimeter
- Car class with properties and behaviors
- Calculator class with static methods
- Product class for inventory system
- Person class with constructor overloading
- Date class implementation

Practice: Create 20 different classes modeling real-world entities

Inheritance & Polymorphism

Topics:

- Inheritance: IS-A relationship
- Extends keyword for inheritance
- Parent class (superclass) and child class (subclass)
- Inheriting fields and methods
- Method overriding: redefining parent methods
- @Override annotation
- super keyword: accessing parent class members
- super() for calling parent constructor
- Constructor chaining in inheritance
- Types of inheritance: single, multilevel, hierarchical
- Multiple inheritance limitation in Java
- Final keyword: final variables, methods, classes
- Polymorphism: compile-time and runtime
- Method overloading (compile-time polymorphism)
- Method overriding (runtime polymorphism)
- Dynamic method dispatch
- Upcasting and downcasting
- instanceof operator

Projects built:

- Animal hierarchy (Animal -> Dog, Cat, Bird)
- Shape hierarchy with area calculation
- Vehicle system (Vehicle -> Car, Bike, Truck)
- Employee hierarchy (Employee -> Manager, Developer, Designer)
- Bank account types (Account -> Savings, Current, Fixed)
- School management (Person -> Student, Teacher)
- E-commerce product categories
- Payment system with different payment types

Practice: Build 15 inheritance hierarchies, solve 25 polymorphism problems

Abstraction & Interfaces

Topics:

- Abstraction: hiding implementation details
- Abstract classes with 'abstract' keyword
- Abstract methods: methods without body
- Concrete methods in abstract classes
- When to use abstract classes
- Interfaces: 100% abstraction (pre-Java 8)
- Interface declaration with 'interface' keyword
- Implementing interfaces with 'implements'
- Multiple interface implementation
- Interface variables: public static final
- Interface methods: public abstract

- Default methods in interfaces (Java 8+)
- Static methods in interfaces (Java 8+)
- Private methods in interfaces (Java 9+)
- Functional interfaces and @FunctionalInterface
- Marker interfaces
- Interface vs abstract class comparison
- Real-world abstraction examples

Projects built:

- Shape calculator with abstract classes
- Payment gateway with interfaces
- Database connection abstraction
- Notification system (Email, SMS, Push)
- Remote control interface for devices
- Sorting algorithms with Comparable/Comparator
- Plugin architecture using interfaces
- PHASE 1 MINI CAPSTONE: Library Management System (Console-based)
- Features: Books, members, issue/return, search, OOP principles

Practice: Solve 30 abstraction and interface problems

Assessment: Phase 1 Final Assessment - Core Java & OOP fundamentals test

----- PHASE 2: INTERMEDIATE JAVA & ADVANCED CONCEPTS (MONTHS 4-6, WEEKS 14-26) -----

Master advanced Java features, Collections Framework, exception handling, file I/O, multithreading, and design patterns.

Months 4-5: Advanced Java Features & Collections (Week 14-22)

String Handling & Regular Expressions

Topics:

- String class: immutability and string pool
- String methods: length, charAt, substring, indexOf
- String comparison: equals, equalsIgnoreCase, compareTo
- String manipulation: concat, replace, trim, split
- String case methods: toUpperCase, toLowerCase
- StringBuilder class: mutable strings
- StringBuffer class: thread-safe mutable strings
- StringBuilder vs StringBuffer vs String
- String formatting: format(), printf()
- StringTokenizer class
- Regular expressions (regex) in Java
- Pattern and Matcher classes
- Regex patterns: character classes, quantifiers, groups
- Common regex operations: validation, search, replace
- Regex for email, phone, URL validation

Projects built:

- Text analyzer (word count, character count, frequency)
- String manipulation utility library
- Email validator using regex
- Phone number formatter
- Password strength validator
- URL extractor from text
- CSV parser using String methods
- Palindrome and anagram checker
- Caesar cipher encryption/decryption
- Log file parser

Practice: Solve 35 string manipulation and regex problems

Exception Handling & Error Management

Topics:

- What are exceptions? Error vs Exception
- Exception hierarchy: Throwable, Error, Exception
- Checked exceptions vs unchecked exceptions
- Try-catch block for exception handling
- Multiple catch blocks
- Catch block order: specific to general
- Finally block: cleanup code
- Try-with-resources (Java 7+): automatic resource management
- Throw keyword: throwing exceptions manually
- Throws keyword: declaring exceptions
- Creating custom exceptions
- Custom checked and unchecked exceptions
- Exception chaining and cause
- Best practices for exception handling
- Common exceptions: NullPointerException, ArrayIndexOutOfBoundsException, etc.
- Rethrowing exceptions
- Multi-catch (Java 7+)

Projects built:

- Robust calculator with exception handling
- File reader with proper error handling
- Custom banking exception system

- Input validation with custom exceptions
- Safe array operations utility
- Age validator with custom exceptions
- Division calculator with exception handling
- User registration with validation exceptions

Practice: Solve 25 exception handling scenarios

Collections Framework - Part 1

Topics:

- Collections Framework overview and architecture
- Collection interface hierarchy
- List interface and implementations
- ArrayList: dynamic array implementation
- ArrayList methods: add, remove, get, set, size, clear
- LinkedList: doubly-linked list implementation
- ArrayList vs LinkedList performance comparison
- Vector and Stack (legacy classes)
- Set interface: unique elements
- HashSet: hash table implementation
- LinkedHashSet: insertion order maintenance
- TreeSet: sorted set using Red-Black tree
- Set operations and methods
- Iterator and ListIterator interfaces
- Enhanced for loop with collections
- Collections class utility methods

Projects built:

- Todo list application with ArrayList
- Student management system with List
- Phonebook with ArrayList and search
- Unique word counter with HashSet
- Sorted student list with TreeSet
- Remove duplicates from list
- Collection operations library
- Inventory management with collections

Practice: Solve 40 List and Set problems

Collections Framework - Part 2

Topics:

- Map interface: key-value pairs
- HashMap: hash table based map
- HashMap methods: put, get, remove, containsKey, keySet, values
- LinkedHashMap: insertion order maintenance
- TreeMap: sorted map using Red-Black tree
- Hashtable: legacy synchronized map
- HashMap vs Hashtable vs ConcurrentHashMap
- Queue interface: FIFO structure
- PriorityQueue: heap-based priority queue
- Deque interface: double-ended queue
- ArrayDeque implementation
- Comparable interface for natural ordering
- Comparator interface for custom ordering
- Sorting collections: Collections.sort()
- Searching collections: Collections.binarySearch()
- Generics in collections
- Type safety with generics

Projects built:

- Word frequency counter with HashMap
- Contact management with Map
- Student grade book with TreeMap
- Priority task manager with PriorityQueue
- Cache implementation with LinkedHashMap
- Custom sorting with Comparator
- Leaderboard system with TreeSet
- Configuration manager with Properties

Practice: Solve 40 Map and Queue problems

Generics & Advanced Collections

Topics:

- Introduction to generics: type parameters
- Generic classes and interfaces
- Generic methods
- Bounded type parameters: extends, super
- Wildcards: ?, ? extends T, ? super T
- Generic constructors
- Type erasure in generics
- Restrictions on generics
- Generic collections usage
- Creating custom generic classes
- EnumSet and EnumMap
- Immutable collections (Java 9+)
- List.of(), Set.of(), Map.of()
- Collections performance comparison

- Time and space complexity of operations

Projects built:

- Generic stack and queue implementation
- Generic pair and tuple classes
- Type-safe database repository
- Generic utility library
- Custom generic collection class
- Generic algorithm implementations

Practice: Create 15 generic classes, solve 25 generics problems

Month 6: File I/O, Multithreading & Modern Java (Week 23-26)

File I/O & Serialization

Topics:

- File handling in Java: File class
- Creating, deleting, renaming files and directories
- File properties: exists, length, canRead, canWrite
- Byte streams: InputStream, OutputStream
- FileInputStream and FileOutputStream
- Character streams: Reader, Writer
- FileReader and FileWriter
- Buffered streams: BufferedReader, BufferedWriter
- PrintWriter for formatted output
- Scanner for reading input
- NIO (New I/O) package: Path, Paths, Files
- Reading and writing with NIO
- Serialization: converting objects to bytes
- Serializable interface
- ObjectInputStream and ObjectOutputStream
- Transient keyword
- Externalizable interface for custom serialization

Projects built:

- File-based note-taking application
- File copy and move utility
- Text file analyzer (line count, word count)
- CSV file reader and writer
- Contact manager with file persistence
- Student records with serialization
- Configuration file manager
- Log file generator and reader
- File encryption/decryption utility
- Directory tree viewer

Practice: Build 15 file handling applications

Multithreading & Concurrency - Part 1

Topics:

- Multithreading concepts: process vs thread
- Thread lifecycle: new, runnable, running, blocked, terminated
- Creating threads: extending Thread class
- Creating threads: implementing Runnable interface
- Thread class methods: start, run, sleep, join
- Thread priorities: setPriority, getPriority
- Daemon threads
- Thread synchronization: race condition problem
- Synchronized keyword: synchronized methods
- Synchronized blocks
- Inter-thread communication: wait, notify, notifyAll
- Deadlock and its prevention
- Thread safety and immutability
- ThreadLocal class
- Thread groups

Projects built:

- Multi-threaded counter application
- Producer-consumer problem solution
- Bank account with synchronized withdrawal
- Print even-odd numbers using two threads
- Multi-threaded file downloader
- Thread pool executor example
- Dining philosophers problem
- Ticket booking system with synchronization

Practice: Solve 30 multithreading problems

Multithreading & Concurrency - Part 2

Topics:

- Executor framework: ExecutorService
- Thread pools: FixedThreadPool, CachedThreadPool
- Callable and Future interfaces
- CompletableFuture for async programming
- Concurrent collections: ConcurrentHashMap
- CopyOnWriteArrayList
- BlockingQueue and its implementations
- Atomic variables: AtomicInteger, AtomicLong

- Lock interface and ReentrantLock
- ReadWriteLock for read-write operations
- Semaphore for resource control
- CountdownLatch and CyclicBarrier
- Fork/Join framework
- Parallel streams (Java 8+)
- Best practices for concurrent programming

Projects built:

- Parallel file processor
- Concurrent web crawler
- Multi-threaded image processor
- Task scheduler with ExecutorService
- Parallel array sum calculation
- Thread-safe cache implementation
- Concurrent download manager
- Real-time data processor

Practice: Build 12 concurrent applications

Java 8+ Modern Features

Topics:

- Lambda expressions: syntax and usage
- Functional interfaces: Predicate, Function, Consumer, Supplier
- Method references: static, instance, constructor
- Streams API: creating streams
- Intermediate operations: filter, map, flatMap, sorted
- Terminal operations: collect, forEach, reduce
- Collectors class: toList, toSet, toMap, groupingBy
- Optional class: handling null values
- Date and Time API (java.time package)
- LocalDate, LocalTime, LocalDateTime
- ZonedDateTime and time zones
- Period and Duration
- Default and static methods in interfaces
- forEach method with lambda
- New features in Java 9-17: modules, var keyword, records, sealed classes

Projects built:

- Stream operations on collections
- Employee filtering and sorting with streams
- Data transformation pipeline
- Optional handling in applications
- Date-time calculations and formatting
- File processing with streams
- Custom collectors implementation
- Functional programming utilities

Practice: Solve 40 lambda and streams problems

Phase 2 Capstone Project

Topics:

- Advanced OOP implementation
- Collections framework usage
- Exception handling throughout
- File-based persistence
- Multithreading for concurrent operations
- Modern Java features (lambdas, streams)

Projects built:

- PHASE 2 CAPSTONE: Banking Management System
- Features: Multiple account types, transactions, threading for concurrent operations, file persistence, comprehensive OOP
- Alternative: E-commerce Inventory System with multi-user support
- Alternative: Hospital Management System
- Alternative: Hotel Reservation System with concurrent booking

Assessment: Phase 2 Final Exam - Advanced Java concepts comprehensive test

PHASE 2 CONTINUED - Design Patterns & Best Practices (Week 14-26 (distributed))

Design Patterns - Part 1 (Creational)

Topics:

- Design patterns introduction and importance
- Gang of Four (GoF) patterns
- Pattern categories: creational, structural, behavioral
- Singleton pattern: one instance
- Factory pattern: object creation
- Abstract Factory pattern
- Builder pattern: complex object construction
- Prototype pattern: cloning objects
- When to use each creational pattern
- Anti-patterns to avoid
- Real-world pattern applications

Projects built:

- Database connection pool (Singleton)
- Shape factory implementation
- Document builder with Builder pattern

- GUI component factory
- Configuration manager (Singleton)
- Clone-based object creation

Practice: Implement all 5 creational patterns in different scenarios

Design Patterns - Part 2 (Structural & Behavioral)

Topics:

- Adapter pattern: interface adaptation
- Decorator pattern: adding functionality
- Proxy pattern: placeholder object
- Facade pattern: simplified interface
- Composite pattern: tree structure
- Observer pattern: event handling
- Strategy pattern: algorithm selection
- Command pattern: encapsulating requests
- Template Method pattern
- State pattern: state-based behavior
- Chain of Responsibility pattern
- Iterator pattern (already in Collections)
- MVC pattern for applications

Projects built:

- Notification system (Observer pattern)
- Payment strategy implementation
- Logging framework (Decorator + Singleton)
- UI component library (Composite)
- Request processing chain
- State machine implementation
- Command-based calculator

Practice: Implement 15+ design patterns with real examples

JDBC & Database Programming

Topics:

- Introduction to JDBC (Java Database Connectivity)
- JDBC architecture and drivers
- Driver types: Type 1, 2, 3, 4
- Loading JDBC driver
- Establishing database connection
- Connection interface and methods
- Statement interface for SQL execution
- PreparedStatement: parameterized queries
- CallableStatement: stored procedures
- ResultSet: processing query results
- CRUD operations with JDBC
- Transaction management: commit, rollback
- Batch processing
- Connection pooling
- Database metadata
- Handling SQL exceptions

Projects built:

- Student database management (CRUD)
- Employee records system with MySQL
- Product catalog with database
- User authentication with database
- Transaction-based banking system
- Prepared statement utility library
- Database backup and restore utility
- Report generation from database

Practice: Build 10 database-driven applications

Unit Testing with JUnit

Topics:

- Introduction to testing: why test?
- Types of testing: unit, integration, system
- JUnit framework introduction
- JUnit 5 (Jupiter) architecture
- Test annotations: @Test, @BeforeEach, @AfterEach
- @BeforeAll, @AfterAll
- Assertions: assertEquals, assertTrue, assertNotNull
- assertThrows for exception testing
- Test lifecycle
- Parameterized tests
- Test suites
- Mocking with Mockito
- Test-Driven Development (TDD)
- Code coverage with JaCoCo
- Best practices for unit testing

Projects built:

- Unit tests for calculator class
- Testing collections utilities
- TDD implementation of stack
- Testing database operations with mocks
- Test suite for banking system

- Parameterized tests for validators
- Integration tests for application

Practice: Write tests for all previous projects, achieve 80%+ coverage

Build Tools & Version Control

Topics:

- Build automation importance
- Maven: project management tool
- Maven project structure
- POM.xml configuration
- Maven lifecycle phases
- Dependency management with Maven
- Maven repositories: local, central, remote
- Gradle: modern build tool
- Gradle build scripts
- Gradle vs Maven comparison
- Git version control (review/deep dive)
- Git with Java projects
- GitHub collaboration
- .gitignore for Java projects
- CI/CD basics with GitHub Actions

Projects built:

- Maven-based Java project
- Multi-module Maven project
- Gradle project setup
- Dependency management practice
- Git repository for all projects
- Automated build with Maven/Gradle

Practice: Convert all projects to Maven/Gradle structure

----- PHASE 3: ENTERPRISE JAVA & WEB DEVELOPMENT (MONTHS 7-9, WEEKS 27-39) -----

Master enterprise Java development with Servlets, JSP, Spring Framework, Spring Boot, Hibernate, and RESTful APIs.

Months 7-8: Web Development & Spring Framework (Week 27-35)

Servlets & JSP Fundamentals

Topics:

- Web application architecture: client-server
- HTTP protocol: request-response cycle
- HTTP methods: GET, POST, PUT, DELETE
- Web servers vs application servers
- Apache Tomcat setup and configuration
- Servlets: server-side Java programs
- Servlet lifecycle: init, service, destroy
- HttpServlet class and methods
- doGet() and doPost() methods
- HttpServletRequest and HttpServletResponse
- Request parameters and headers
- Session management: HttpSession
- Cookies in servlets
- Request dispatcher: forward and include
- Servlet configuration: web.xml, annotations
- JSP (JavaServer Pages) introduction
- JSP lifecycle and implicit objects
- JSP directives, declarations, scriptlets, expressions

Projects built:

- Hello World servlet
- Login servlet with session
- User registration form processing
- Simple calculator servlet
- Shopping cart with session
- File upload servlet
- JSP-based dynamic web pages
- Servlet-JSP MVC pattern application

Practice: Build 10 servlet-based web applications

Advanced Servlets & Web Technologies

Topics:

- Servlet filters: request/response filtering
- Filter chain and multiple filters
- Servlet listeners: lifecycle events
- ServletContext and application scope
- JSTL (JSP Standard Tag Library)
- EL (Expression Language) in JSP
- Custom JSP tags
- MVC architecture in web applications
- Front Controller pattern
- DAO (Data Access Object) pattern
- Connection pooling in web apps

- AJAX with servlets
- JSON response from servlets
- File download servlet
- Security in web applications
- Form-based authentication

Projects built:

- Complete CRUD web application
- Blog application with Servlet-JSP
- Employee management web app
- E-commerce product catalog
- User authentication and authorization
- File upload/download system
- RESTful service with servlets
- MVC-based web application

Practice: Build 8 complete web applications

Spring Framework Core

Topics:

- Introduction to Spring Framework
- Spring modules and architecture
- Dependency Injection (DI) concept
- Inversion of Control (IoC) principle
- Spring IoC container: BeanFactory, ApplicationContext
- Bean definition and configuration
- XML-based configuration
- Annotation-based configuration: @Component, @Autowired
- Java-based configuration: @Configuration, @Bean
- Bean scopes: singleton, prototype, request, session
- Bean lifecycle: init and destroy methods
- Autowiring: byName, byType, constructor
- @Qualifier for disambiguation
- Dependency injection: constructor, setter, field
- ApplicationContext events
- Spring Expression Language (SpEL)

Projects built:

- Simple Spring application setup
- Dependency injection examples
- Bean configuration practice
- Spring-based calculator
- User service with DI
- Multiple bean configurations
- Layered architecture with Spring

Practice: Create 15 Spring DI scenarios

Spring Boot Fundamentals

Topics:

- Introduction to Spring Boot
- Spring Boot advantages: convention over configuration
- Spring Initializr: project generation
- Spring Boot starters
- Auto-configuration mechanism
- Application.properties and application.yml
- Spring Boot main application class
- @SpringBootApplication annotation
- Component scanning
- Spring Boot DevTools
- Creating REST controllers with @RestController
- @RequestMapping, @GetMapping, @PostMapping
- Path variables and request parameters
- Request body with @RequestBody
- Response entity and HTTP status codes
- Exception handling with @ControllerAdvice
- Custom error responses

Projects built:

- Hello World REST API with Spring Boot
- CRUD REST API for products
- User management REST API
- Todo application backend
- Blog post API
- Student records API
- Exception handling in APIs
- API documentation with Swagger/OpenAPI

Practice: Build 10 REST APIs with Spring Boot

Spring Data JPA & Hibernate

Topics:

- JPA (Java Persistence API) introduction
- ORM (Object-Relational Mapping) concept
- Hibernate as JPA implementation
- Entity classes with @Entity annotation
- Primary keys: @Id, @GeneratedValue
- Column mapping: @Column annotation

- Table mapping: @Table annotation
- Entity relationships: @OneToMany, @ManyToOne, @ManyToMany
- Cascade types and fetch types (LAZY, EAGER)
- Spring Data JPA repositories
- JpaRepository interface
- CRUD operations with repositories
- Query methods: findBy, findAll, save, delete
- Custom queries with @Query
- JPQL (Java Persistence Query Language)
- Native SQL queries
- Pagination and sorting

Projects built:

- Student database with JPA entities
- Product catalog with Spring Data JPA
- Blog with posts and comments (relationships)
- E-commerce with products and categories
- User and roles management
- Library system with JPA
- Custom query implementations
- Pagination and sorting examples

Practice: Build 10 database applications with Spring Data JPA

Month 9: Advanced Spring Boot & Microservices Basics (Week 36-39)

Spring Security & Authentication

Topics:

- Spring Security introduction
- Security filter chain
- Authentication vs authorization
- In-memory authentication
- JDBC authentication
- UserDetailsService for custom authentication
- Password encoding: BCryptPasswordEncoder
- Method-level security: @PreAuthorize, @Secured
- Role-based access control (RBAC)
- JWT (JSON Web Token) authentication
- Creating and validating JWT tokens
- Stateless authentication
- OAuth 2.0 basics
- Social login integration
- CORS configuration in Spring Boot
- CSRF protection

Projects built:

- User registration and login API
- JWT-based authentication system
- Role-based authorization API
- Secured REST API endpoints
- Social login with OAuth
- Password reset functionality
- User profile management with security
- Multi-role application

Practice: Secure all REST APIs with authentication

Spring Boot Advanced Features

Topics:

- Spring Boot Actuator: production-ready features
- Health checks and metrics
- Custom actuator endpoints
- Application monitoring
- Logging in Spring Boot: Logback, Log4j2
- Logging levels and configuration
- Aspect-Oriented Programming (AOP) in Spring
- @Aspect, @Before, @After, @Around
- Cross-cutting concerns: logging, transaction
- Transaction management: @Transactional
- Declarative vs programmatic transactions
- Caching with Spring: @Cacheable, @CacheEvict
- Redis integration for caching
- Scheduling tasks: @Scheduled
- Async processing: @Async
- Event handling in Spring
- Profiles for different environments

Projects built:

- Application with health monitoring
- Custom metrics endpoint
- AOP-based logging system
- Transaction management in banking app
- Caching layer implementation
- Scheduled task executor
- Async email notification system
- Multi-environment configuration

Practice: Add production features to all Spring Boot apps
RESTful API Best Practices & Microservices Intro

Topics:

- REST API design principles
- Resource naming conventions
- HTTP status codes usage
- HATEOAS (Hypermedia as the Engine of Application State)
- API versioning strategies
- Request validation with @Valid
- Bean validation annotations
- API documentation with SpringDoc OpenAPI
- Swagger UI integration
- Rate limiting and throttling
- API security best practices
- Microservices architecture introduction
- Monolithic vs microservices
- Service discovery concept
- Inter-service communication
- Introduction to Spring Cloud

Projects built:

- RESTful API with HATEOAS
- Validated REST endpoints
- Complete API with Swagger docs
- Versioned API implementation
- Rate-limited public API
- Basic microservice setup
- Service-to-service communication
- API gateway pattern example

Practice: Design and document 5 production-ready APIs
Spring Boot with External Services

Topics:

- RestTemplate for HTTP clients
- WebClient for reactive HTTP calls
- Consuming third-party APIs
- Feign client for declarative REST
- Email integration: JavaMailSender
- Sending HTML emails
- File upload and storage
- AWS S3 integration
- Message queues: RabbitMQ basics
- Apache Kafka introduction
- Event-driven architecture
- WebSocket support in Spring Boot
- Real-time communication
- Payment gateway integration (Stripe, Razorpay)
- SMS integration
- Push notifications

Projects built:

- Weather API consumer
- Email notification service
- File upload to cloud storage
- Message queue producer-consumer
- Real-time chat application
- Payment processing system
- SMS alert service
- Event-driven order processing

Practice: Integrate 8 external services in applications
Testing Spring Boot Applications

Topics:

- Spring Boot testing overview
- @SpringBootTest annotation
- Test slicing: @WebMvcTest, @DataJpaTest
- MockMvc for testing controllers
- Mocking with @MockBean
- Testing REST APIs
- Integration testing with database
- TestContainers for database testing
- Testing security configurations
- Testing with different profiles
- JUnit 5 with Spring Boot
- AssertJ for fluent assertions
- Test coverage with JaCoCo
- Performance testing basics
- Contract testing introduction

Projects built:

- Complete test suite for REST API
- Controller layer tests
- Service layer tests with mocks
- Repository layer tests

- Integration tests for application
- Security tests
- End-to-end testing
- Test-driven development practice

Practice: Write comprehensive tests for all Spring Boot projects

PHASE 3 COMPLETION - Month 9 Final Week (Week 27-39 (distributed))

Database Design & Optimization

Topics:

- Relational database design principles
- Normalization: 1NF, 2NF, 3NF, BCNF
- Entity-Relationship diagrams
- Database schema design
- Index optimization in JPA
- Query optimization techniques
- N+1 query problem and solutions
- Database connection pooling: HikariCP
- Transaction isolation levels
- Optimistic vs pessimistic locking
- Database migration with Flyway/Liquibase
- NoSQL with Spring Boot: MongoDB
- Spring Data MongoDB
- Choosing SQL vs NoSQL
- Multi-database applications

Projects built:

- Optimized database schema design
- Migration scripts with Flyway
- Performance-tuned JPA queries
- MongoDB integration
- Connection pool configuration
- Database performance monitoring
- Multi-database Spring Boot app

Practice: Optimize databases for all projects

Application Deployment & DevOps

Topics:

- Packaging Spring Boot applications
- JAR vs WAR deployment
- Application properties for production
- Externalized configuration
- Environment-specific configurations
- Deploying to Tomcat server
- Docker basics review
- Dockerizing Spring Boot applications
- Docker Compose for multi-container
- Docker best practices for Java
- Cloud platforms: AWS, Azure, GCP
- Deploying to AWS Elastic Beanstalk
- Deploying to Render
- CI/CD for Spring Boot with Jenkins
- GitHub Actions for Java projects

Projects built:

- Production-ready application configuration
- Dockerized Spring Boot app
- Multi-container deployment
- Cloud deployment practice
- CI/CD pipeline setup
- Automated deployment workflow
- Blue-green deployment example

Practice: Deploy all applications to production environments

Monitoring, Logging & Performance

Topics:

- Application monitoring strategies
- Spring Boot Actuator deep dive
- Micrometer metrics
- Prometheus integration
- Grafana dashboards
- Distributed tracing with Sleuth and Zipkin
- Centralized logging with ELK stack
- Log aggregation strategies
- Performance profiling tools
- JVM monitoring and tuning
- Garbage collection optimization
- Memory leak detection
- Load testing with JMeter
- API performance optimization
- Database query profiling

Projects built:

- Monitoring dashboard setup
- Metrics collection and visualization

- Distributed tracing implementation
- Centralized logging system
- Performance optimization project
- Load testing suite
- JVM tuning exercise

Practice: Add monitoring to all production applications

Security Best Practices & API Standards

Topics:

- OWASP Top 10 for Java applications
- SQL injection prevention in JPA
- XSS (Cross-Site Scripting) prevention
- CSRF protection implementation
- Secure password handling
- Secrets management
- API key management
- Rate limiting implementation
- Input validation best practices
- Secure communication: HTTPS/TLS
- Security headers configuration
- Dependency vulnerability scanning
- Static code analysis: SonarQube
- Security testing
- Compliance: GDPR, PCI-DSS basics

Projects built:

- Security-hardened application
- Vulnerability assessment tool
- Secure API implementation
- Security testing suite
- Code quality dashboard

Practice: Security audit all projects

Phase 3 Capstone Project

Topics:

- Full-stack application architecture
- Spring Boot backend development
- Database design and JPA
- REST API development
- Spring Security implementation
- Testing and quality assurance
- Docker deployment
- Production deployment

Projects built:

- MAJOR CAPSTONE: E-commerce Platform Backend
- Features: Product catalog, user auth (JWT), shopping cart, orders, payment integration, admin panel, email notifications, search, pagination, REST APIs, Spring Security, JPA relationships
- Alternative: Social Media Platform API (posts, comments, likes, followers, messaging)
- Alternative: Banking Application (accounts, transactions, loans, statements)
- Alternative: Learning Management System (courses, students, enrollments, assignments, grades)

Assessment: Phase 3 Final Exam - Enterprise Java and Spring Boot comprehensive test

----- PHASE 4: MICROSERVICES, CLOUD & PROFESSIONAL EXCELLENCE (MONTHS 10-12, WEEKS 40-52) -----

Master microservices architecture, Spring Cloud, cloud-native development, DevOps, system design, and career preparation.

Months 10-11: Microservices Architecture & Spring Cloud (Week 40-48)

Microservices Fundamentals

Topics:

- Microservices architecture principles
- Monolithic vs microservices trade-offs
- When to use microservices
- Domain-Driven Design (DDD) basics
- Bounded contexts and aggregates
- Service decomposition strategies
- Database per service pattern
- API Gateway pattern
- Service registry and discovery
- Circuit breaker pattern
- Bulkhead pattern
- Retry and timeout patterns
- Event-driven architecture
- SAGA pattern for distributed transactions
- CQRS (Command Query Responsibility Segregation)

Projects built:

- Decompose monolith into microservices
- Design microservices architecture
- Service boundaries definition
- Event-driven microservice
- SAGA pattern implementation

Practice: Design 5 microservices architectures
Spring Cloud Ecosystem

Topics:

- Spring Cloud overview
- Service discovery with Eureka
- Eureka server and client setup
- Client-side load balancing with Ribbon
- Declarative REST client: Feign
- API Gateway with Spring Cloud Gateway
- Gateway routing and filters
- Distributed configuration: Spring Cloud Config
- Config server and client
- Refreshing configuration dynamically
- Circuit breaker with Resilience4j
- Fallback methods
- Distributed tracing with Sleuth
- Request correlation across services
- Service mesh basics

Projects built:

- Eureka service registry setup
- Multiple microservices with discovery
- API Gateway implementation
- Centralized configuration server
- Circuit breaker implementation
- Distributed tracing setup
- Complete microservices ecosystem

Practice: Build 3 complete microservices systems

Inter-Service Communication & Messaging

Topics:

- Synchronous vs asynchronous communication
- REST for synchronous communication
- gRPC for high-performance RPC
- Protocol Buffers
- Message-driven architecture
- RabbitMQ: message broker
- Exchanges, queues, and bindings
- Spring AMQP for RabbitMQ
- Apache Kafka: distributed streaming
- Kafka topics, partitions, consumers, producers
- Spring Kafka integration
- Event sourcing pattern
- Message ordering and exactly-once delivery
- Dead letter queues
- Idempotency in distributed systems

Projects built:

- RabbitMQ-based order processing
- Kafka event streaming application
- Event-driven microservices
- Message queue patterns implementation
- Real-time data pipeline
- Event sourcing system
- Asynchronous communication between services

Practice: Build 6 messaging-based systems

Microservices Data Management

Topics:

- Database per service pattern
- Shared database anti-pattern
- Polyglot persistence
- Distributed transactions challenges
- Two-phase commit problems
- SAGA pattern: choreography vs orchestration
- Event-driven data management
- Change Data Capture (CDC)
- Database replication strategies
- Caching in microservices
- Redis for distributed caching
- Cache invalidation strategies
- API composition pattern
- CQRS with separate read/write models
- Eventual consistency

Projects built:

- Microservices with separate databases
- SAGA choreography implementation
- SAGA orchestration with state machine
- CQRS pattern implementation
- Distributed caching setup
- Event-driven data synchronization
- Polyglot persistence example

Practice: Implement various data management patterns

Microservices Security & Observability

Topics:

- Security in microservices
- Authentication and authorization in distributed systems
- OAuth 2.0 and OpenID Connect
- JWT for service-to-service auth
- API Gateway security
- Service-to-service authentication
- Secrets management: Vault
- Observability pillars: logs, metrics, traces
- Centralized logging with ELK
- Metrics with Prometheus and Grafana
- Distributed tracing with Jaeger
- Health checks and readiness probes
- Monitoring microservices
- Alerting and incident response

Projects built:

- Secured microservices with OAuth
- JWT-based inter-service communication
- Complete observability stack
- Monitoring dashboard for microservices
- Distributed tracing implementation
- Secrets management setup
- Health check endpoints

Practice: Secure and monitor all microservices

Month 12: Cloud Native, DevOps & Career Excellence (Week 49-52)

Cloud Native Java & Kubernetes

Topics:

- Cloud-native principles
- 12-factor app methodology
- Containerization with Docker (deep dive)
- Multi-stage Docker builds for Java
- Docker image optimization
- Kubernetes introduction
- Kubernetes architecture
- Pods, Deployments, Services
- ConfigMaps and Secrets in K8s
- Persistent volumes
- Horizontal Pod Autoscaling
- Kubernetes for Spring Boot apps
- Helm charts for Java applications
- Service mesh: Istio basics
- Serverless Java: AWS Lambda

Projects built:

- Cloud-native Spring Boot app
- Optimized Docker images
- Kubernetes deployment manifests
- Auto-scaling Spring Boot on K8s
- Helm chart creation
- Service mesh implementation
- Serverless function with Java
- Complete cloud-native application

Practice: Deploy all applications to Kubernetes

AWS & Cloud Services for Java

Topics:

- AWS fundamentals review
- EC2 for Java applications
- AWS RDS for databases
- AWS S3 for object storage
- AWS Lambda with Java
- API Gateway with Lambda
- AWS ECS and Fargate
- AWS Elastic Beanstalk for Spring Boot
- AWS CloudFormation
- AWS SDK for Java
- DynamoDB with Java
- SQS and SNS messaging
- AWS monitoring: CloudWatch
- Azure and GCP alternatives
- Multi-cloud strategies

Projects built:

- Deploy Spring Boot to AWS EB
- Serverless API with Lambda
- S3 file storage integration
- RDS database connection
- Complete AWS infrastructure
- CloudFormation templates
- Multi-service cloud deployment

Practice: Build 5 cloud-native applications
DevOps & CI/CD for Java

Topics:

- DevOps culture and practices
- CI/CD pipeline design
- Jenkins for Java projects
- Jenkins pipeline as code
- GitLab CI/CD
- GitHub Actions for Java
- Building with Maven/Gradle in CI
- Automated testing in pipeline
- Code quality gates: SonarQube
- Security scanning: OWASP Dependency Check
- Container image scanning
- Deployment automation
- Blue-green deployments
- Canary releases
- Infrastructure as Code: Terraform

Projects built:

- Complete CI/CD pipeline
- Jenkins pipeline for microservices
- GitHub Actions workflow
- Automated testing and deployment
- Quality gates implementation
- Security scanning integration
- Infrastructure automation
- Zero-downtime deployment

Practice: Set up CI/CD for all projects

System Design & Architecture

Topics:

- System design fundamentals
- Scalability: horizontal vs vertical
- Load balancing strategies
- Caching layers and strategies
- CDN for static content
- Database sharding and partitioning
- Database replication
- CAP theorem
- Consistency models
- Message queues for decoupling
- Designing for high availability
- Disaster recovery planning
- Rate limiting and throttling
- API design best practices
- System design interview preparation
- Common system design problems

Projects built:

- Design URL shortener
- Design Twitter/X
- Design e-commerce platform
- Design ride-sharing system
- Design streaming service
- Design notification system
- Design chat application
- System design documentation

Practice: Solve 15 system design problems

Performance Engineering

Topics:

- Performance testing types
- JMeter for load testing
- Gatling for performance testing
- Performance metrics and KPIs
- Application profiling
- JVM profiling tools: JProfiler, YourKit
- Memory profiling and leak detection
- CPU profiling
- Database query optimization
- Caching strategies
- Connection pool tuning
- JVM tuning: heap size, GC
- Garbage collection algorithms
- Application optimization techniques
- Performance monitoring in production

Projects built:

- Load testing suite
- Performance benchmarking
- Application profiling report
- JVM tuning exercise
- Database optimization project

- Performance dashboard
 - Optimization case studies
- Practice: Performance test and optimize all applications

PHASE 4 COMPLETION - Career Preparation (Week 40-52 (distributed))

Advanced Java Topics & Specializations

Topics:

- Reactive programming with Project Reactor
- Spring WebFlux for reactive applications
- Reactive streams and backpressure
- GraphQL with Spring Boot
- gRPC services in Java
- Apache Camel for integration
- Batch processing with Spring Batch
- Scheduled jobs with Quartz
- Search with Elasticsearch
- Caching with Hazelcast
- Blockchain basics with Java
- Machine Learning with Java: Weka, DL4J
- Android development basics (Java)
- Desktop applications: JavaFX

Projects built:

- Reactive REST API
- GraphQL API with Spring Boot
- gRPC service implementation
- Batch processing application
- Elasticsearch integration
- Real-time data processing
- Specialization project in chosen area

Practice: Explore 3 advanced specializations

Code Quality & Best Practices

Topics:

- Clean code principles
- SOLID principles in Java
- Code refactoring techniques
- Code smells and fixes
- Design patterns application
- Effective Java best practices
- Java coding standards
- Code review guidelines
- Technical debt management
- Documentation best practices
- Javadoc comments
- README and wiki writing
- Architecture documentation
- Static code analysis tools
- Continuous code quality

Projects built:

- Code refactoring exercise
- Clean code implementation
- Design patterns showcase
- Comprehensive documentation
- Code review checklist
- Quality metrics dashboard

Practice: Refactor all projects with best practices

Open Source & Community

Topics:

- Open source contribution guide
- Finding beginner-friendly projects
- Understanding project structure
- Reading contribution guidelines
- Forking and pull requests
- Code review in open source
- Building open source portfolio
- Starting your own open source project
- Documentation for open source
- Community management
- Licensing: Apache, MIT, GPL
- Building personal brand

Projects built:

- Contribute to 5 open source projects
- Create own open source library
- Maintain open source project
- Documentation contributions
- Bug fixes and features
- Community engagement

Practice: Active open source participation

Interview Preparation

Topics:

- Java interview preparation strategy
- Core Java interview questions
- OOP interview questions
- Collections framework questions
- Multithreading interview topics
- Spring and Spring Boot questions
- Microservices interview prep
- System design interviews
- Data structures and algorithms review
- LeetCode problem-solving strategies
- Behavioral interview questions
- STAR method for answers
- Salary negotiation tactics
- Job search strategies
- Resume optimization for Java developers

Projects built:

- Solve 150+ LeetCode problems
- System design case studies
- Mock interview practice
- Interview preparation guide
- Portfolio website creation

Practice: Daily coding challenges and mock interviews

Final Month - Capstone & Career Launch (Week 49-52)

Final Capstone Project - Part 1

Topics:

- Project ideation and planning
- Requirement analysis
- System architecture design
- Microservices design
- Database design
- API design and documentation
- Technology stack selection
- Development workflow setup
- Sprint planning
- Agile methodology

Projects built:

- FINAL CAPSTONE: Enterprise-Grade Microservices Application
- Option 1: E-commerce Microservices Platform
- Services: User, Product, Order, Payment, Notification, Inventory
- Option 2: Social Media Platform Backend
- Services: User, Post, Comment, Like, Follow, Feed, Notification, Chat
- Option 3: Banking/FinTech Application
- Services: Account, Transaction, Loan, Card, KYC, Notification
- Option 4: Learning Management System
- Services: User, Course, Enrollment, Content, Assessment, Progress, Certificate

Final Capstone Project - Part 2

Topics:

- Implementation completion
- Comprehensive testing
- Performance optimization
- Security hardening
- Documentation writing
- Deployment to cloud
- CI/CD pipeline setup
- Monitoring and logging
- Load testing
- Project presentation preparation

Deliverables:

- Complete source code on GitHub
- Microservices deployed to cloud (AWS/Azure/GCP)
- Kubernetes deployment manifests
- Complete API documentation (Swagger/OpenAPI)
- Architecture diagrams and documentation
- Database schema and relationships
- CI/CD pipeline configuration
- Test coverage report (80%+ coverage)
- Performance testing report
- Video demo and presentation
- Comprehensive README and wiki

Career Launch & Professional Development

Topics:

- Professional portfolio website
- Resume optimization for Java roles
- LinkedIn profile enhancement
- GitHub profile showcase
- Technical blog writing
- Speaking at meetups/conferences
- Networking strategies

- Job application process
- Interview follow-up
- Continuous learning plan
- Staying updated with Java ecosystem
- Mentoring and teaching others
- Freelancing strategies
- Consulting career path
- Startup opportunities

Deliverables:

- Professional portfolio with 40+ projects
- Optimized resume for Java developer roles
- Enhanced LinkedIn profile with certifications
- GitHub profile with 100+ contributions
- 5-10 technical blog posts
- Personal brand establishment
- Job application materials ready
- Mock interview completion
- Continuous learning roadmap
- Professional network connections

Assessment: FINAL COMPREHENSIVE CERTIFICATION EXAM - Complete Java mastery evaluation

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-3): 20+ foundational projects - calculators, games, console apps, OOP systems
- Phase 2 (Months 4-6): 18+ intermediate projects - collections apps, multithreaded systems, design patterns
- Phase 3 (Months 7-9): 20+ enterprise projects - web apps, REST APIs, Spring Boot applications, databases
- Phase 4 (Months 10-12): 15+ advanced projects - microservices, cloud deployments, distributed systems
- Final: 5 major capstone projects demonstrating complete expertise

Total projects: 70+ projects from beginner to enterprise-grade production systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Java: Syntax, OOP, Collections, Generics, Exception Handling, Multithreading, I/O, Lambda, Streams
- Advanced Java: Design Patterns, Reflection, Annotations, JVM Internals, Performance Tuning
- Enterprise Java: Servlets, JSP, JDBC, JPA/Hibernate, Spring Core, Spring Boot, Spring Security, Spring Data
- Web Development: RESTful APIs, HTTP, JSON, API Design, Swagger/OpenAPI, WebSockets
- Databases: SQL (MySQL, PostgreSQL), JPA, Hibernate, Query Optimization, Database Design, NoSQL (MongoDB)
- Microservices: Spring Cloud, Eureka, API Gateway, Circuit Breaker, Message Queues (RabbitMQ, Kafka)
- Testing: JUnit 5, Mockito, Integration Testing, Test-Driven Development, Code Coverage
- DevOps: Docker, Kubernetes, CI/CD (Jenkins, GitHub Actions), Maven, Gradle, Git
- Cloud: AWS (EC2, RDS, S3, Lambda, ECS), Elastic Beanstalk, Cloud Native Development
- Architecture: System Design, Microservices Architecture, Event-Driven Architecture, CQRS, SAGA
- Tools: IntelliJ IDEA, Eclipse, VS Code, Postman, SonarQube, JMeter, Prometheus, Grafana
- Soft Skills: Problem-solving, System Design Thinking, Code Review, Documentation, Agile Methodologies

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 8-10 hours
 Projects: 3-5 hours
 Practice problems: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding sessions and doubt clearing
 Mentorship: 1-on-1 expert Java developer mentorship
 Community: Active Discord/Slack community with 24/7 peer support
 Code review: Expert code reviews for all major projects
 Career support: Resume review, mock interviews, job referrals to product companies
 Lifetime access: All content, future updates, new Java versions coverage
 Interview prep: Mock interviews with industry professionals
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion (4 certificates)
 Final certificate: Professional Java Developer Certification

Spring certificate: Spring Boot Specialist Certificate
Microservices certificate: Microservices Architecture Certificate
Linkedin badge: Digital badges for LinkedIn profile
Industry recognized: Recognized by Fortune 500 companies and startups
Portfolio projects: 40+ production-ready portfolio projects
Oracle exam prep: Preparation for Oracle Java Certification (OCA/OCP)

WHERE THIS CAN LEAD

- Java Developer (Junior to Senior levels)
- Backend Developer / Backend Engineer
- Full Stack Java Developer (Java + React/Angular)
- Spring Boot Developer
- Microservices Developer
- Enterprise Application Developer
- Solutions Architect
- Technical Lead / Team Lead
- DevOps Engineer (Java focus)
- Cloud Engineer (AWS/Azure with Java)
- Software Architect
- Principal Engineer
- Freelance Java Consultant
- Technical Trainer / Instructor
- Startup Founder / CTO

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-5 LPA (Junior Java Developer / Fresher)

Growing roles: Rs 5-9 LPA (Java Developer); Rs 8-15 LPA (Senior Java Developer / Spring Boot Developer)

Experienced roles: Rs 12-25 LPA (Senior Java / Microservices Developer / Tech Lead); Rs 18-40 LPA (Lead Developer / Architect)

Freelance: Rs 1,000-5,000/hour based on expertise

International usa: \$80k-180k USD (Java Developer to Senior)

International europe: €50k-120k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Why should I learn Java programming in 2026 when there are newer languages available?

A: Java remains the #1 language for enterprise development, powering 3 billion devices worldwide. It's essential for Android development, banking systems, big data (Hadoop, Spark), and microservices. Companies like Google, Amazon, Netflix, and major banks heavily use Java.

Q: Is this Java programming course suitable for complete beginners with no coding experience?

A: Absolutely! This 12-month masterclass is designed for absolute beginners. We start with programming fundamentals before introducing Java syntax.

Q: What technologies and frameworks will I learn besides core Java?

A: Beyond core Java, you'll master: Spring Boot, Spring Security, Hibernate/JPA, Microservices with Spring Cloud, REST APIs, Docker, Kubernetes, AWS, CI/CD pipelines, MySQL, PostgreSQL, MongoDB, Redis, Apache Kafka, RabbitMQ, JUnit testing, and system design. This covers the complete stack used by product companies.

Q: How does this Java course prepare me for job interviews and placement?

A: We provide comprehensive interview preparation including 150+ LeetCode problem solutions, system design practice (designing systems like Twitter and Uber), Java-specific interview questions, Spring Boot scenarios, and unlimited mock interviews. We also help with resume optimization and building a strong project portfolio. We prepare you thoroughly for interviews, but we do not make placement guarantees.

Q: What kind of projects will I build in this Java programming masterclass?

A: You'll build 40+ production-ready projects including: e-commerce platform with microservices, social media backend, banking application, learning management system, video streaming service backend, and distributed systems. The final capstone involves a complete enterprise application deployed on AWS with Kubernetes.

Q: What salary can I expect after completing this Java programming course?

A: International remote roles can pay more. Actual pay depends on your skills, experience, and the market.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-java-programming-masterclass-college>

=====

COURSE 79 OF 118: JAVASCRIPT FOR TEENS: INTERACTIVE WEBSITES & BROWSER GAMES

=====

Course link: <https://learn.modernagecoders.com/courses/javascript-course-for-teens-beginners-interactive-web>

Subject area: Web Development

Designed for: Teens

Who can join: Beginner, ages 13 to 18

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 2-3 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

JavaScript course for teens ages 13-18: live online classes from first variables to interactive pages, canvas games, real APIs, and a deployed capstone site.

In one line: The language every browser runs, taught from the first variable to a game your friends can actually play.

ABOUT THE PROGRAMME

JavaScript is the only programming language every web browser understands, which makes it the most immediate language a teenager can learn: write a line, refresh the page, and something on screen changes. This 6-month live course starts at zero and builds in one direction, toward work worth showing. Months 1 and 2 cover the language core and the DOM, so pages respond to clicks and typing within weeks. Months 3 and 4 add real data: arrays, objects, JSON, saving with localStorage, and fetching from public APIs such as live weather. Month 5 is games on the HTML canvas, with a proper game loop, keyboard control, and collision detection. Month 6 is the capstone: each student scopes, builds, and deploys an interactive site of their own, then presents it. Classes are live and small. Students write code in every session while an instructor watches them work, and homework gets reviewed, not just collected.

WHAT MAKES THIS COURSE DIFFERENT

- Everything runs in the browser students already have, so the write-refresh-see loop takes seconds and motivation stays high
- The DOM arrives in month 2, not month 5: pages react to clicks within the first weeks, long before the theory runs out
- Real public APIs instead of toy data: weather, dictionaries, and services students choose themselves
- A full month of canvas games, because a game you built yourself is the project teens actually show their friends
- The capstone is deployed to a real URL, so the finished site can sit on a school application or be sent to relatives
- No prior HTML needed: the markup and CSS each project requires is taught in class, exactly when it is needed

THE LEARNING PATH, STEP BY STEP

Phase 1: The language and the page: variables, decisions, loops, and functions, then the DOM and events so pages respond to users

Phase 2: Data and the network: arrays, objects, JSON, and localStorage, then fetch and public APIs for pages built on live data

Phase 3: Canvas games with a real game loop, then a scoped, deployed capstone site presented to the class

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from the first line of code

Html knowledge: Not required. The HTML and CSS each project needs is taught in class

Equipment: A computer with a modern browser and a stable internet connection

Age: 13 to 18. Younger children are better served by our kids coding courses

WHO THIS COURSE IS FOR

Complete beginners: Teens who have never written code and want a language where results show up on screen immediately

Scratch graduates: Students who outgrew block coding and want to type real programs that do real things

Future web developers: Teens aiming at websites and apps, for whom JavaScript is the non-negotiable first step

Game curious: Students who want to build playable browser games, not just read about them

Portfolio builders: Teens who want deployed, working projects to point to on school and college applications

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE LANGUAGE AND THE PAGE (MONTHS 1-2, WEEKS 1-8)

JavaScript from the first line: values, decisions, loops, and functions, then the DOM and events so real pages respond to real users.

Month 1: JavaScript from the First Line (Weeks 1-4)

First Lines of JavaScript

Topics:

- Where JavaScript runs: the browser, the console, and why no install is needed to start
- Setting up VS Code and running a script inside an HTML page
- Values and variables: let and const, and when each is right
- Numbers and strings, and what happens when you mix them
- Template literals for building sentences out of data
- console.log as your first debugging tool
- Comments and naming things so future-you understands the code

Projects built:

- Story generator: a script that assembles a silly story from variables, then from user input via prompt

Practice: 10 short console exercises: swap variables, build sentences, fix three broken scripts

Making Decisions

Topics:

- Comparisons: equal, strictly equal, greater, less, and why === matters
- if, else if, else: teaching a program to choose
- Boolean values and expressions that produce them
- Combining conditions with and, or, not
- Truthy and falsy values, and the bugs they cause
- Nesting conditions without losing the plot

Projects built:

- Number guessing game, round one: the computer picks, you guess, it answers higher or lower

Practice: Decision drills: grade calculator, ticket price rules, a login age check with edge cases

Loops and Repetition

Topics:

- while loops: repeat until something changes
- for loops: counting, stepping, and counting backwards
- Accumulator patterns: totals, counters, and building strings in a loop
- break and continue, used sparingly
- Nested loops and when they are worth it
- Reading a loop you did not write and predicting its output

Projects built:

- Times-table printer with formatting
- Guessing game, round two: limited tries, a running score, play-again logic

Practice: 12 loop katas, including FizzBuzz and a countdown timer printed to the console

Functions

Topics:

- Declaring functions: inputs, outputs, and the return statement
- Parameters vs arguments, and default values
- Arrow function syntax and where you will meet it
- Scope: what a function can and cannot see
- Breaking a messy script into small named functions
- Testing a function by hand with sample inputs

Projects built:

- Tip calculator and unit converter built as a small library of reusable functions

Practice: Write 8 functions from written specs, then refactor week 3 projects to use functions

Assessment: Month 1 skills check: write four short functions live, explaining each choice

Month 2: The DOM and Events (Weeks 5-8)

Meeting the DOM

Topics:

- Just enough HTML: tags, ids, and classes, taught as we need them
- The DOM tree: how the browser sees a page
- querySelector and querySelectorAll: finding elements
- Changing text with textContent and innerHTML, and the difference
- Changing styles and toggling classes from JavaScript
- The dev tools Elements panel as an x-ray of your page

Projects built:

- Mood page: buttons that change the page's colors, text, and heading live

Practice: Select-and-change drills on a provided practice page with 15 small targets

Events: Pages That Listen

Topics:

- addEventListener: click, input, and change events
- The event object and what it tells you
- Reading values out of text inputs
- Updating the page in response to typing, live
- Preventing default behavior and when you need to
- Common event bugs: listeners attached too early, wrong element, wrong event

Projects built:

- Click counter with milestones
- Live character counter for a message box, with a limit warning

Practice: Build three tiny widgets from spec: a like button, a show-password toggle, a color picker preview

Interactive Widgets

Topics:

- Showing and hiding elements cleanly with classes
- Creating and removing elements with createElement and remove
- Building lists in the DOM from JavaScript data
- classList patterns: toggle, add, remove
- Dark mode done properly: one class on the body
- Organizing widget code so each piece has one job

Projects built:

- Dark mode toggle for a full page
- FAQ accordion that opens one answer at a time

Practice: Recreate two widgets from screenshots of real websites, structure first, then behavior

Project Week: The Interactive Quiz

Topics:

- Planning a small app: screens, states, and what happens on each click
- Keeping score and moving between questions
- Ending states: results screen and restart
- Basic input validation: no empty answers
- Polish: progress display and honest feedback messages
- Reading your own code a week later: comments and naming revisited

Projects built:

- Interactive quiz app: questions, scoring, a results screen, and a restart button

Practice: Extend the quiz with one feature of your own choosing, written up in three sentences

Assessment: Quiz app demo and code walkthrough: each student explains one function line by line

----- PHASE 2: DATA AND THE NETWORK (MONTHS 3-4, WEEKS 9-16) -----

Arrays, objects, and JSON give projects real data to work with. Then fetch opens the door to public APIs and pages built on live information.

Month 3: Arrays, Objects, and JSON (Weeks 9-12)

Arrays

Topics:

- Creating arrays and reading items by index
- push, pop, shift, unshift: growing and shrinking lists
- Looping over arrays with for and for...of
- indexOf and includes: finding things
- Arrays and the DOM: rendering a list from data
- slice vs splice, and the bug everyone hits once

Projects built:

- To-do list, version one: add, display, and delete tasks backed by an array

Practice: 15 array exercises: totals, longest word, reversing, filtering by hand with loops

Array Methods That Do the Work

Topics:

- map: transforming every item
- filter: keeping what matches
- find: the first match
- sort, and why sorting numbers needs a compare function
- Chaining methods to answer questions in one line
- When a plain loop is still the clearer choice

Projects built:

- Movie list page: search box and filter buttons that narrow a list of films live

Practice: Rewrite five loop solutions from week 9 using map and filter, compare readability

Objects

Topics:

- Objects as labeled data: properties and values
- Dot notation vs bracket notation
- Objects inside arrays: the shape of most real data
- Looping over object properties

- Nesting: objects inside objects, read carefully
- Designing the data shape before writing the code

Projects built:

- Contact cards: render a grid of profile cards from an array of person objects

Practice: Model three real things as objects: a song, a match result, a school timetable entry

JSON and Saving Data

Topics:

- What JSON is and why every API speaks it
- JSON.stringify and JSON.parse
- localStorage: saving data in the browser
- The save-load cycle: persisting an app's state across refreshes
- What localStorage cannot do, and where databases come in later
- Debugging storage with the dev tools Application panel

Projects built:

- To-do list, version two: tasks survive a page refresh using localStorage

Practice: Add save and load to the movie list project; inspect the stored JSON by hand

Assessment: Data checkpoint: given a JSON file of real data, load it, answer five questions with code

Month 4: Fetch and Public APIs (Weeks 13-16)

What an API Is

Topics:

- Requests and responses: how the browser asks servers for data
- URLs as questions: endpoints and paths
- Reading API documentation without fear
- fetch and .then: your first request to a live joke API
- Status codes: 200, 404, and what they mean
- Looking at raw JSON responses in the browser

Projects built:

- Joke machine: a button that fetches and displays a fresh joke from a public API

Practice: Explore two public APIs in the browser, write down what three endpoints return

async, await, and Things Going Wrong

Topics:

- async and await: the cleaner way to write fetch code
- Loading states: telling the user something is happening
- try and catch: handling failed requests honestly
- The network tab: watching your requests in flight
- Slow connections and what your app looks like on them
- Never trusting a response blindly: checking before rendering

Projects built:

- Weather lookup page: type a city, see live conditions, with loading and error states

Practice: Break your own weather app on purpose three ways, then make each failure show a decent message

Search Interfaces

Topics:

- Query parameters: sending the user's search to the API
- Building a URL safely from input
- Rendering result lists with the DOM skills from month 2
- Empty results and misspelled searches: designing for them
- Debouncing, gently introduced: not firing a request per keystroke
- Caching a result you already fetched

Projects built:

- Dictionary app: search a word, show definitions, phonetics, and an example sentence from a free dictionary API

Practice: Swap the dictionary API for a different public API of your choice, keep the same interface

Project Week: The Two-API Dashboard

Topics:

- Combining two data sources on one page
- Structuring code when an app has parts: one file, clear sections, named functions
- Handling two requests that can each fail independently
- Layout for dashboards: cards and sections, with just enough CSS
- A refresh button done right
- Writing a short README that says what the app does

Projects built:

- Personal dashboard: two public APIs of the student's choice on one page, for example weather plus news headlines

Practice: Trade dashboards with a classmate and file two improvement suggestions each

Assessment: Dashboard demo: live walkthrough plus questions on how each fetch works

PHASE 3: CANVAS GAMES AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24)

A month of real game programming on the HTML canvas, then a capstone month: scope, build, deploy, and present an interactive site of your own.

Month 5: Games on the Canvas (Weeks 17-20)

Drawing and the Game Loop

Topics:

- The canvas element and its coordinate system
- Drawing rectangles, circles, lines, and text
- Color, stroke, and fill
- requestAnimationFrame: the heartbeat of every game
- Clearing and redrawing: why animation is just fast drawing
- Moving a shape across the screen with velocity

Projects built:

- Bouncing ball: a ball that moves, hits the walls, and bounces, with adjustable speed

Practice: Draw a scene of your choice in code, then animate one element of it

Player Control

Topics:

- Keyboard events: keydown, keyup, and tracking held keys
- Moving a player smoothly, not one hop per press
- Boundaries: keeping the player on screen
- Separating update logic from drawing logic
- Game state as an object: player, score, settings in one place
- Frame-rate honesty: why movement uses small steps

Projects built:

- Paddle controller: a paddle that glides left and right under arrow-key control, ready for a game

Practice: Add a second control scheme (WASD) and an on-screen speed display

Collisions, Score, and Lives

Topics:

- Rectangle collision detection, explained with pictures before code
- Circle collision using distance
- Spawning objects: falling items with random positions and speeds
- Score, lives, and difficulty that ramps up
- Removing off-screen objects so the game does not slow down
- Game over conditions and detecting them cleanly

Projects built:

- Catcher game core: falling objects, a moving basket, score on catch, lives lost on miss

Practice: Tune your game's difficulty curve and write down the three numbers you changed and why

Finish and Polish the Game

Topics:

- Start screen, pause, and game-over screen as states
- Sound effects with the Audio API, used tastefully
- High score saved with localStorage, from month 3
- Small touches that read as polish: flash on hit, score pop
- Playtesting: watching someone else play your game without helping them
- Packaging the game on a page with instructions

Projects built:

- Finished catcher or breakout-style game with start, pause, game over, sound, and a saved high score

Practice: Run two playtests with family members and fix the most confusing thing each one hit

Assessment: Game demo day: every student's game played live by a classmate, followed by a code Q&A

Month 6: The Capstone Site (Weeks 21-24)

Scoping the Capstone

Topics:

- Choosing a project you can finish: the honest feature list
- Wireframing on paper before touching code
- Picking your data source: localStorage, a public API, or both
- Breaking the build into tasks with a simple checklist
- Setting up the project folder and skeleton page
- What done means: writing acceptance criteria in plain language

Projects built:

- Approved capstone plan: one page with wireframe, feature list, and task breakdown

Practice: Build the static skeleton of your capstone: structure and placeholder content, no behavior yet

Build Sprint One: Core Features

Topics:

- Working feature by feature, smallest first
- Instructor code review: reading feedback and applying it
- Debugging your own project methodically: reproduce, isolate, fix
- Keeping functions small while the project grows
- When to delete a feature: scope trimming without shame
- Committing progress notes at the end of each session

Projects built:

- Capstone core: the two or three features that make the project itself

Practice: Daily 30-minute build sessions with a written note on what moved forward

Build Sprint Two: Data and Fit

Topics:

- Wiring in the data layer: saving, loading, or fetching live data
- Making the layout survive a phone screen: the essential media query

- Empty states and first-use experience
- Error states for anything that can fail
- A second code review round
- Cutting the last risky feature if time demands it

Projects built:

- Feature-complete capstone: data working, layout presentable on desktop and phone

Practice: Test your capstone on a phone and one other browser, log what broke

Ship and Present

Topics:

- Bug hunt: a structured final pass over every feature
- Deploying to a free host so the site has a real URL
- Writing a README: what it is, how it works, what you would add next
- Presenting technical work: demo first, code second
- Answering questions about your own code
- Where to go from here: React, Node.js, or deeper JavaScript

Projects built:

- Deployed capstone site with README, presented live to the class and family

Practice: Send your capstone link to three people who are not in the class

Assessment: Capstone presentation and code Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Number guessing game with score and limited tries
- Interactive quiz app with results screen and restart
- To-do list that survives refresh with localStorage
- Movie list page with live search and filters
- Weather lookup page built on a live public API
- Dictionary app with definitions and phonetics
- Two-API personal dashboard of the student's choice
- Canvas catcher game with score, lives, sound, and a saved high score
- Deployed capstone: an interactive site scoped, built, and presented by the student

Total projects: 12+ working projects, from console games to a deployed capstone site

SKILLS THE STUDENT WILL HAVE AT THE END

- JavaScript core: variables, conditionals, loops, and functions written with confidence
- DOM and events: selecting, changing, and building page content that responds to users
- Data handling: arrays, objects, JSON, and the array methods real codebases lean on
- The network: fetch, async and await, loading and error states, and reading API docs
- Canvas games: the game loop, keyboard control, collision detection, and game states
- Working habits: debugging methodically, scoping projects, and presenting your own code

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor

Practice: 2-3 hours of set exercises and project work between classes

Review: Homework reviewed with written feedback; stuck points reopened in the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck until the next session

Progress updates: Regular progress notes to parents: what was covered, what was built, what needs practice

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, backed by the student's own project portfolio

WHERE THIS CAN LEAD

- Front-end development track: our React and full-stack courses build directly on this foundation
- Back-end JavaScript with Node.js once the language core is solid
- School hackathons and coding competitions with a working project toolkit
- Game development courses, carrying over the canvas and game-loop experience
- A stronger base for computer science electives and board CS subjects

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Should my teen learn JavaScript or Python first?

A: Either works, and we teach both, so this is about the student, not the language.

JavaScript wins for teens motivated by visible results: it runs in the browser, so every lesson changes something they can see, click, and share. Python reads a little more like

English and suits students drawn to data, maths, or automation. If your teen talks about websites and games, start here; if they talk about AI and data, start with Python.

Q: Does my teen need to know HTML before starting?

A: No. We teach the small amount of HTML and CSS each project needs, exactly when the project needs it. Students who want to go deep on page structure and design can take our HTML and CSS course before or alongside this one, but it is not a requirement.

Q: What will my teen actually build in six months?

A: Twelve or more working projects, in rising order of ambition: console games, an interactive quiz, a to-do app that saves its data, a live weather page, a dictionary app, a two-API dashboard, a canvas game with score and sound, and finally a capstone site they scope and build themselves. The capstone is deployed to a real URL, so it exists outside the classroom.

Q: How are the classes taught?

A: Live one-hour online classes, twice a week, with a real instructor and a small group. Students write and run code during class rather than watching someone else do it, and homework comes back with written feedback. Group batches are capped at 10, with mini-batch and 1-on-1 options for students who want closer attention.

Q: Is 13 too young for real JavaScript?

A: No, the course is written for exactly this range. We pace the abstract parts, functions, scope, asynchronous code, with more examples and more practice than an adult course would, and every concept lands inside a project rather than a slide. For children under 13 we recommend our kids coding courses, which build the same thinking through friendlier tools first.

Q: Can we try a class before enrolling?

A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in and ask the teacher questions afterwards.

Q: What software and computer are needed?

A: Everything used in the course is free: a modern browser like Chrome and the VS Code editor, which we help set up in the first class. Any reasonably recent laptop or desktop with a stable internet connection is enough; there is no heavy software to install.

Q: What should a student take after this course?

A: The natural next steps are our React course for modern front-end work, Node.js for the server side, or our data structures and algorithms course for students eyeing competitive programming and technical interviews later. Many students also carry their canvas experience into deeper game development. The instructor gives each student a specific recommendation at the end based on what they enjoyed most.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/javascript-course-for-teens-beginners-interactive-web>

=====

COURSE 80 OF 118: JEE FOUNDATION MATHS CLASS 8-10: PROOFS & PROBLEM SOLVING

=====

Course link: <https://learn.modernagecoders.com/courses/jee-foundation-maths-course-class-8-10>
Subject area: Foundation Mathematics
Designed for: High school
Who can join: Class 8 to 10 students, batched by class
Course length: 12 months (48 weeks)
Time needed: 2 live classes/week + 3-4 hours of problem sets
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

JEE foundation maths course for Class 8-10: NCERT-aligned depth, geometry proof skills, olympiad-style problem solving and timed MCQ practice in live batches.

In one line: Class 11 physics and maths punish shaky algebra. This course is where the algebra stops being shaky.

ABOUT THE PROGRAMME

Most students who struggle in JEE preparation do not struggle with Class 11 topics; they struggle with Class 8 to 10 topics that were never made solid. Fractions handled nervously, identities half-remembered, geometry never actually proved. A foundation course exists to fix that before it costs anything, and this one does it honestly: no rank promises, no 'future

IIITian' theatre, just the school syllabus taught deeper than school goes, plus the problem-solving habits that entrance exams eventually demand. The 12-month plan runs in four phases. First, numbers and algebra at real depth: divisibility and remainders, exponents and surds, identities and factorisation, equations and inequalities. Second, geometry as a proof discipline, congruence, similarity, circles, because writing a correct proof is the closest a school student comes to real mathematical thinking. Third, mensuration, introductory trigonometry and coordinate geometry, the toolkit chapters. Fourth, the part school never teaches: problem-solving heuristics, olympiad-style questions, and a full timed MCQ phase in the style of NTSE-type talent exams and entrance papers, with error logs and review clinics. Content is NCERT-aligned and batched by class, so a Class 8 student and a Class 10 student meet the same ideas at different depths. School marks tend to improve as a side effect; the actual product is a student who reaches Class 11 with nothing to repair.

WHAT MAKES THIS COURSE DIFFERENT

- NCERT-aligned but deeper: every chapter is taken past the textbook stopping point, into the why and the harder problem forms
- Proof skills taught deliberately: given, to prove, construction, proof, drilled until writing a geometry argument feels normal
- A genuine problem-solving phase: heuristics like working backwards, small cases and clean diagrams, practised on olympiad-style questions
- A full timed MCQ phase with error logs: elimination, option testing and time budgeting, the craft entrance exams actually test
- Honest positioning: this is foundation, not JEE coaching, and we say plainly that nobody can promise selections or ranks
- Batched by class: Class 8, 9 and 10 groups meet the same arc at the right depth, so nobody is drowned or bored

THE LEARNING PATH, STEP BY STEP

Phase 1: Numbers and algebra at depth: divisibility and remainders, exponents and surds, identities, factorisation, equations and inequalities

Phase 2: Geometry as a proof discipline: angles, congruence, similarity, quadrilaterals and circle theorems, with proof writing drilled throughout

Phase 3: The toolkit chapters: mensuration in 2D and 3D, introductory trigonometry with heights and distances, and coordinate geometry

Phase 4: Problem-solving heuristics and olympiad-style questions, then a timed MCQ and mock phase with error clinics and a readiness review

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable with the previous class's school maths. The week 1 diagnostic finds the gaps and the early weeks patch them

Class: Class 8, 9 or 10. Batches are separated by class so depth and pace fit the student

Equipment: The student's NCERT maths textbook, a geometry box, graph paper, notebooks, and a device with stable internet

Mindset: Willingness to write solutions out in full. This course grades reasoning, not just answers

WHO THIS COURSE IS FOR

Future jee aspirants: Students in Class 8 to 10 who intend to attempt JEE or similar entrances and want the base built before Class 11 raises the stakes

Strong students underfed: Students who find school maths easy and are bored, and need harder problems rather than more of the same

Proof never taught: Students who can compute but have never been taught to prove, which is most students

Mcq first timers: Students who have only ever written school papers and need timed MCQ craft before any talent or entrance exam

Families avoiding hype: Parents who want serious foundation work without rank promises and future-topper theatre

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: NUMBERS AND ALGEBRA AT DEPTH (MONTHS 1-3, WEEKS 1-12) -----

The chapters every entrance exam quietly assumes: number theory basics, exponents and surds, identities, factorisation, and equations, taken well past the textbook stopping point.

Month 1: Number Systems, Properly (Weeks 1-4)

Weeks 1-2: The Number Line, End to End

Topics:

- Diagnostic paper: arithmetic, fractions, basic algebra and geometry from earlier classes
- Integers, rationals and irrationals, and how to prove a number belongs where it does
- Decimal expansions: terminating, recurring, and converting recurring decimals to fractions
- Representing surds on the number line

- Density: finding rationals and irrationals between any two numbers
- Comparing numbers cleverly: without calculators, by squaring or bounding

Projects built:

- Number-line dossier: one page classifying 20 assorted numbers with a one-line justification each

Practice: 30 classification and conversion problems, including 5 recurring-decimal conversions written out in full

Weeks 3-4: Divisibility and Remainders

Topics:

- Divisibility rules for 2 through 11, proved rather than just stated
- Primes, composites and prime factorisation as the master tool
- HCF and LCM: computation, the product relationship, and word problems
- Remainder thinking: what is left when powers are divided, kept concrete
- Digit problems: unit digits of large powers, digit sums
- First olympiad-style flavour: divisibility puzzles with small numbers

Projects built:

- Divisibility rule proof sheet: the rules for 3, 9 and 11 explained in the student's own words with examples

Practice: 25 divisibility and HCF-LCM problems plus 5 unit-digit puzzles, hardest two attempted before class discussion

Assessment: Month 1 test: a 25-mark paper on number systems mixing standard questions with two puzzle-style problems

Month 2: Exponents, Surds and Identities (Weeks 5-8)

Weeks 5-6: Exponents and Surds

Topics:

- Laws of exponents for integer and rational powers, with the proofs sketched
- Negative and fractional exponents handled without fear
- Surds: simplifying, adding, and rationalising denominators including two-term denominators
- Comparing surds and exponential expressions by clever bounding
- Scientific notation and orders of magnitude
- Exam forms: simplify-this-monster questions and how to dismantle them

Projects built:

- Simplification gauntlet: 10 layered exponent-surd expressions reduced step by step, each step justified

Practice: 35 exponent and surd problems including 8 rationalisations and 5 comparison questions without a calculator

Weeks 7-8: Identities and Factorisation

Topics:

- The square and cube identities, derived, visualised, and drilled
- Sum and difference of cubes, and the three-variable identity for a cubed sum
- Factorisation strategy: common factors, grouping, splitting the middle term, identities
- The remainder theorem and factor theorem, introduced with numeric checks
- Evaluating expressions cleverly: 99 squared, 101 cubed via identities
- Identity-based olympiad-style problems: finding values without finding variables

Projects built:

- Identity map: every identity in the phase on one sheet with a numeric example and one exam use each

Practice: 30 factorisations across all five strategies plus 8 evaluate-cleverly problems, strategy named before each

Assessment: Month 2 test: a 30-mark paper on exponents, surds and identities including two multi-step simplifications

Month 3: Equations and Inequalities (Weeks 9-12)

Weeks 9-10: Linear Equations and Systems

Topics:

- Linear equations in one variable, including fractional and bracketed forms
- Word-problem craft: translating ages, digits, mixtures and motion into equations
- Pairs of linear equations: substitution and elimination done cleanly
- Consistency: when systems have one, many or no solutions, seen graphically
- Parameter questions: for what value of k does the system break
- Checking solutions as a habit, not an afterthought

Projects built:

- Translation drill sheet: 12 word problems converted to equations before any are solved

Practice: 25 equations and systems plus 8 word problems, with the check step written for every answer

Weeks 11-12: Inequalities and a First Look at Quadratics

Topics:

- Linear inequalities: solving, representing on the number line, and the sign-flip rule proved
- Compound conditions: and versus or, read carefully
- Quadratic equations by factorisation: the bridge from identities to solving
- The idea of a discriminant, previewed for the Class 10 batch
- Absolute value as distance, kept geometric and gentle
- Phase consolidation: mixed algebra sets that do not announce their chapter

Projects built:

- Personal error log opened: every phase 1 test miss classified as concept, method or slip, used all year

Practice: 20 inequalities and 15 factorisation-solved quadratics, plus one mixed set of 15 unannounced problems

Assessment: Phase 1 milestone: a 40-mark numbers-and-algebra paper, one third of it in MCQ format as a preview of phase 4

PHASE 2: GEOMETRY AS A PROOF DISCIPLINE (MONTHS 4-6, WEEKS 13-24)

Geometry is where students first learn what mathematical certainty means. Congruence, similarity, quadrilaterals and circles, with proof writing drilled from the first week.

Month 4: Angles, Triangles and Congruence (Weeks 13-16)

Weeks 13-14: Lines, Angles and What a Proof Is

Topics:

- Angles on a line, vertically opposite angles, and angles with parallel lines
- The triangle angle-sum result and exterior angle theorem, proved
- What a proof is: statements, reasons, and why 'it looks true' is not enough
- The four-part format: given, to prove, construction, proof
- Angle chasing: finding unknown angles in layered figures
- Drawing honest figures: large, labelled, and not misleadingly special

Projects built:

- First proof portfolio entries: angle-sum and exterior-angle proofs written in full four-part format

Practice: 20 angle-chasing figures of rising difficulty plus 2 proofs, every figure redrawn by hand before solving

Weeks 15-16: Congruence

Topics:

- Congruence criteria: SAS, ASA, AAS, SSS and RHS, and why AAA fails
- Choosing the right criterion: reading the figure for what is actually given
- Isosceles triangle results, proved both directions
- Inequalities in a triangle: bigger side, bigger angle, and the triangle inequality
- Multi-step congruence proofs: proving parts equal via congruent triangles
- Common proof errors: assuming the conclusion, circular reasoning, unstated steps

Projects built:

- Criterion sorting sheet: 12 figures matched to their congruence criterion with the given parts marked

Practice: 15 congruence problems including 5 full proofs, each proof peer-checked against the four-part format

Assessment: Month 4 test: a 25-mark geometry paper with two full proofs marked strictly on structure and justification

Month 5: Similarity, Pythagoras and Quadrilaterals (Weeks 17-20)

Weeks 17-18: Similarity and Pythagoras

Topics:

- Similar triangles: the AA, SSS and SAS similarity criteria
- The basic proportionality theorem and its converse
- How similarity scales lengths and areas differently, seen with numbers first
- The Pythagoras theorem proved by similarity, and its converse
- Applications: heights via shadows, distances across rivers, ladders and slopes
- Pythagorean triples and spotting them fast in MCQs

Projects built:

- Proof portfolio extended: BPT and the similarity proof of Pythagoras written in full

Practice: 20 similarity and Pythagoras problems including 3 proofs and 5 real-setting applications

Weeks 19-20: Quadrilaterals and the Midpoint Theorem

Topics:

- The parallelogram theorems: sides, angles and diagonals, proved and reversed
- Tests for a parallelogram: which minimal facts force the shape
- Rectangle, rhombus and square as parallelograms with extra conditions
- The midpoint theorem and its converse, with applications
- Multi-step figures: midpoints inside triangles inside parallelograms
- Proof strategy: what to construct when the figure refuses to yield

Projects built:

- Quadrilateral family tree: the shapes and the exact conditions separating them, made by the student

Practice: 18 quadrilateral problems including 4 proofs, plus 5 midpoint-theorem applications

Assessment: Month 5 test: a 25-mark paper on similarity and quadrilaterals with two proofs and one construction-strategy question

Month 6: Circles and the Geometry Workshop (Weeks 21-24)

Weeks 21-22: Circle Theorems

Topics:

- Chords: equal chords, distances from the centre, the perpendicular from the centre
- The central angle theorem: the angle at the centre doubles the angle at the arc
- Angles in the same segment, and the angle in a semicircle
- Cyclic quadrilaterals and their opposite angles
- Tangents: perpendicularity to the radius and equal tangents from an external point
- Angle chasing in circle figures, the entrance-exam favourite

Projects built:

- Circle theorem atlas: every theorem drawn, stated and proved or justified on the student's

own pages

Practice: 20 circle problems from single-theorem to three-theorem figures, plus 2 full proofs

Weeks 23-24: The Geometry Problem Workshop

Topics:

- Hard multi-step figures mixing congruence, similarity, quadrilaterals and circles
- Auxiliary constructions: when and what to draw, taught as a decision, not magic
- Working backwards from what must be proved
- Olympiad-style geometry: problems where the figure is the puzzle
- Timed geometry MCQs: answering from the figure without full proofs
- Phase review: the proof portfolio completed and audited

Projects built:

- Workshop set: 6 multi-step geometry problems solved with the construction decision written out for each

Practice: One timed 15-question geometry MCQ set plus 4 workshop problems, error log updated with figure-reading misses

Assessment: Phase 2 milestone: a 40-mark geometry paper, half proofs and half timed MCQs, with a one-on-one proof review

PHASE 3: MENSURATION, TRIGONOMETRY AND COORDINATES (MONTHS 7-9, WEEKS 25-36)

The toolkit chapters every entrance exam draws on: 2D and 3D mensuration, introductory trigonometry with heights and distances, and coordinate geometry.

Month 7: Mensuration in Two and Three Dimensions (Weeks 25-28)

Weeks 25-26: Areas in the Plane

Topics:

- Areas of triangles: half base times height, and Heron's formula for when heights hide
- Areas of parallelograms, trapeziums and rhombuses, derived not memorised
- Circles, sectors and segments, with arc lengths
- Composite figures: decomposing shaded regions cleanly
- Same base, same parallels: area relationships without any measuring
- Estimation checks: is that area plausible for that figure

Projects built:

- Decomposition sheet: 8 shaded-region problems with the cut lines drawn and named before any formula is used

Practice: 22 area problems including 5 Heron applications and 5 composite figures, decomposition stated in one line each

Weeks 27-28: Solids and Scaling

Topics:

- Surface areas and volumes: cube, cuboid, cylinder, cone, sphere, hemisphere
- Combined solids and which surfaces vanish at the joins
- Melting and recasting problems: volume as the conserved quantity
- Scaling laws: double the length, four times the area, eight times the volume
- Unit discipline across centimetres, metres and litres
- MCQ forms: ratio-based mensuration questions solved without computing either quantity

Projects built:

- Scaling experiment write-up: one solid scaled by 2 and by 3, with areas and volumes tabulated and the pattern stated

Practice: 18 solid problems including 4 recasting and 4 ratio questions, units checked on every line

Assessment: Month 7 test: a 25-mark mensuration paper mixing computation, ratio reasoning and one composite figure

Month 8: Introductory Trigonometry (Weeks 29-32)

Weeks 29-30: Ratios and Identities

Topics:

- The six trigonometric ratios in a right triangle and the naming logic
- Exact values at 0, 30, 45, 60 and 90 degrees, rebuilt from two triangles rather than memorised
- Finding all ratios from one, with Pythagoras doing the work
- The fundamental identity and its rearrangements
- Proving simple identities with a fixed strategy list
- Ratio MCQs at speed: value questions in under a minute

Projects built:

- Two-triangle derivation page: the 30-60-90 and 45-45-90 triangles built and every standard value read off them

Practice: 30 ratio evaluations and 6 identity proofs, plus one 10-question timed MCQ set on values

Weeks 31-32: Heights and Distances

Topics:

- Angles of elevation and depression on honest, labelled figures
- Single-triangle problems: towers, poles, kites and slopes
- Two-triangle problems: two observation points and moving observers
- Choosing the ratio that solves in one step
- Rounding sensibly and sanity-checking against the figure
- Where trigonometry goes in Class 11: a preview, not a syllabus

Projects built:

- Figure-first set: 8 heights and distances problems where the marked figure is drawn and approved before solving

Practice: 15 heights and distances problems including 4 two-triangle setups, every figure labelled with the given angle

Assessment: Month 8 test: a 25-mark trigonometry paper of ratios, identities and applications, one third MCQ

Month 9: Coordinate Geometry (Weeks 33-36)

Weeks 33-34: The Plane and Its Formulas

Topics:

- The Cartesian plane: plotting, quadrants and reading coordinates fluently
- The distance formula from Pythagoras, derived and drilled
- Classifying triangles and quadrilaterals by computed distances
- The section formula and midpoints, with internal division problems
- Ratios from coordinates: where the axes cut a segment
- Sketch-first habit: every coordinate problem starts with a drawing

Projects built:

- Coordinate gallery: 6 figures plotted, classified and verified by distances, presented on graph paper

Practice: 25 distance and section formula problems, each with a labelled sketch, plus 5 classification questions

Weeks 35-36: Lines and Coordinate Problem Solving

Topics:

- Slope as steepness: computing it and reading it from a sketch
- Parallel and perpendicular slopes, kept concrete
- Collinearity by slope and by distance, two roads to one answer
- Coordinate proofs: choosing axes cleverly to make geometry easy
- Mixed coordinate MCQs under time
- Phase review: the toolkit chapters consolidated onto revision sheets

Projects built:

- Coordinate proof piece: one classical geometry fact proved by placing coordinates well, written up in full

Practice: 20 slope and collinearity problems plus one timed 12-question coordinate MCQ set, error log updated

Assessment: Phase 3 milestone: a 40-mark paper across mensuration, trigonometry and coordinates, half MCQ, fully reviewed

----- PHASE 4: PROBLEM SOLVING AND THE MCQ PHASE (MONTHS 10-12, WEEKS 37-48) -----

The part school never teaches: heuristics for unfamiliar problems, olympiad-style questions, then a full timed MCQ and mock phase in the style of NTSE-type talent exams and entrance papers.

Month 10: How Hard Problems Are Actually Solved (Weeks 37-40)

Weeks 37-38: The Heuristics

Topics:

- Understand, plan, execute, check: the honest version of problem solving
- Trying small cases and looking for the pattern that survives
- Working backwards from what is asked
- Drawing the right diagram, and redrawing when it misleads
- Parity and simple invariants: what cannot change no matter the moves
- Olympiad-style number and algebra problems using the heuristics, difficulty honest but humane

Projects built:

- Solution journal opened: every hard problem written up with the heuristic used and the dead ends kept, not erased

Practice: 8 problems for the week, chosen to need different heuristics; write-ups matter more than answer counts

Weeks 39-40: Counting and Logical Structure

Topics:

- Systematic listing: counting without missing and without double-counting
- The multiplication principle and simple arrangements
- Combinations kept concrete: choosing teams and handshakes
- The pigeonhole principle through puzzles
- Logic puzzles: knights, liars and truth tables lite
- Where these ideas reappear: Class 11 permutations and combinations, and reasoning sections of talent exams

Projects built:

- Counting problem set write-up: 5 counting problems solved twice, once by listing and once by principle, results reconciled

Practice: 12 counting and logic problems with full write-ups in the solution journal

Assessment: Month 10 checkpoint: a 20-mark problem-solving paper where method write-ups earn marks alongside answers

Month 11: MCQ Craft Under Time (Weeks 41-44)

Weeks 41-42: The Craft of the Multiple Choice Question

Topics:

- How MCQ setters build wrong options, and what that tells the solver
- Elimination: discarding options by sign, size, units or parity

- Option testing: substituting answers back instead of solving forwards
- Special values: choosing convenient numbers for variable-heavy questions
- Estimation as a weapon: bounding the answer before computing it
- When to solve honestly: questions where shortcuts are traps

Projects built:

- Technique tally: 30 MCQs solved with the technique noted per question, then tallied to see each student's habits

Practice: Three 15-question topic-wise MCQ sets under time, error log updated with technique choices, not just misses

Weeks 43-44: Sectional Timing and the Error Log

Topics:

- Time budgeting: marks per minute and the discipline of moving on
- Sectional MCQ sets across algebra, geometry, mensuration, trigonometry and coordinates
- The two-pass strategy: bank the easy questions first
- Guessing policy: when unanswered beats wrong, and reading the paper's rules
- Error log analytics: which topic, which technique, which minute of the section
- Building each student's personal weak-topic drill plan for the mock month

Projects built:

- Error log analysis report: the student's own two-page summary of their MCQ patterns going into the mocks

Practice: Four timed sectional sets across the fortnight, each reviewed the same week, drill plan agreed with the teacher

Assessment: Month 11 checkpoint: one full-length topic-mixed MCQ paper under exam timing, fully debriefed

Month 12: Mocks and the Readiness Review (Weeks 45-48)

Weeks 45-46: The Mock Series

Topics:

- Full-length foundation mocks in the style of NTSE-type talent exams and entrance papers
- Exam-day routine rehearsed: instructions, bubbling, rough-work discipline
- Review clinic after every mock: every miss classified and assigned a fix
- Score movement read honestly: trends matter, single papers do not
- Targeted drills between mocks from each student's plan
- Stamina: holding accuracy in the last quarter of a long paper

Projects built:

- Mock file: each paper filed with its marked script, error classification and one-line fix per miss

Practice: One additional mock at home under honest timing, self-marked and brought to the review clinic

Weeks 47-48: Final Mocks and the Road to Class 11

Topics:

- Two final mocks under strict conditions, spaced for recovery
- The readiness map: every topic rated from the year's evidence, no vibes
- The solution journal and proof portfolio reviewed as a body of work
- What Class 11 changes: pace, abstraction, and how this foundation meets it
- Keeping the edge: a weekly problem habit for the months after the course
- Course close: one-on-one review with each family

Projects built:

- Personal readiness map: topic-by-topic strengths and fixes, built from the error log and mock file
- Completed solution journal: a year of hard problems with the student's own write-ups

Practice: Light targeted drills only, set individually from the final error log

Assessment: Course milestone: final full-length mock and readiness review with the family, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A proof portfolio: every major theorem of the course written in full four-part format
- A solution journal: hard problems written up with heuristics and honest dead ends
- A personal error log maintained from phase 1 and analysed before the mock phase
- The identity map and circle theorem atlas, built by the student
- A coordinate proof piece: classical geometry proved by clever axis placement
- A scaling experiment write-up connecting length, area and volume
- The technique tally and error log analysis report from the MCQ phase
- A mock file with marked scripts and a final topic-by-topic readiness map

Total projects: A year-long body of work: proof portfolio, solution journal, error log, mock file and readiness map

SKILLS THE STUDENT WILL HAVE AT THE END

- Number theory basics: divisibility, primes, HCF-LCM and remainder thinking
- Algebra at depth: exponents, surds, identities, factorisation, equations and inequalities
- Proof writing across congruence, similarity, quadrilaterals and circle theorems
- Mensuration, introductory trigonometry and coordinate geometry to entrance-exam standard
- Problem-solving heuristics: small cases, working backwards, invariants and clean diagrams
- Timed MCQ craft: elimination, option testing, time budgeting and error-log-driven improvement

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; mock sittings in month 12 run longer for full papers

Practice: 3-4 hours weekly of problem sets, proofs and journal write-ups between classes

Review: Problem sets marked with written feedback; proofs and journal entries reviewed line by line each month

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems

Progress updates: Monthly progress notes to parents with test scores, error-log trends and the current readiness picture

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside the student's readiness map and mock file

WHERE THIS CAN LEAD

- Class 11 maths in the science stream, entered with algebra and geometry that do not need repair
- Formal JEE preparation from Class 11, with the foundation layer already done
- Olympiad tracks such as IOQM for students who discover they enjoy the hard problems
- School exams along the way: the depth here makes board-level questions feel routine
- Our data science and programming courses, where the same structured thinking transfers directly

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What exactly is a foundation course? Is this JEE coaching for a 13-year-old?

A: No, and be wary of anything that says otherwise. JEE tests Class 11 and 12 material; no Class 8 student should be solving JEE papers. A foundation course strengthens the Class 8 to 10 layer that JEE preparation later stands on: algebra done at depth, geometry actually proved, and the problem-solving and MCQ habits school never teaches. Students who arrive in Class 11 with that layer solid cope with the pace; students who arrive with gaps spend Class 11 repairing them.

Q: Do you promise selection in JEE or any exam?

A: No. Nobody can honestly promise a selection or a rank, least of all three or four years out, and we think enrolling a 13-year-old on the back of such a promise is wrong. What we commit to is specific: the full curriculum taught at depth, marked work with written feedback, a proof portfolio and solution journal the student builds, a timed mock series, and honest monthly reporting on exactly where the student stands.

Q: Is this course NTSE preparation? Is NTSE even happening?

A: We are careful here because many courses are not: NCERT suspended the National Talent Search Scheme, and the exam has not been conducted since the 2021-22 cycle, with the scheme stalled until further orders. There has been talk of a revamped revival, but nothing a family should plan around. We use NTSE-style papers in the mock phase because the format is excellent training, and if the exam returns our students will be ready for it, but nobody should enrol anywhere on the promise of an exam that is currently not running.

Q: Which class is the right time to start, 8, 9 or 10?

A: Any of the three works, which is why batches are separated by class. Starting in Class 8 gives the most room: depth can build gradually with no exam pressure. Class 9 is the most common entry point and aligns well, since Class 9 NCERT is where proofs and harder algebra begin. Class 10 students run the same arc with board preparation happening in parallel, and the overlap actually helps, because the foundation depth makes board questions feel easy.

Q: Will this clash with what school teaches?

A: No, because the course is NCERT-aligned by design. We teach the same chapters school does, in a compatible order, and go deeper rather than sideways: harder problem forms, the proofs behind the formulas, and extensions school skips for time. Most families find school marks improve as a side effect, since school papers sample the easier end of what students here practise weekly.

Q: What are the olympiad-style problems? Is this an olympiad course?

A: It is not a dedicated olympiad course; serious olympiad preparation for exams like IOQM is its own track with its own syllabus. What we teach in month 10 is the thinking style olympiads reward: heuristics like small cases, working backwards and invariants, practised on genuinely unfamiliar problems at a humane difficulty. Students who light up during that phase get pointed, honestly, toward proper olympiad training as a next step.

Q: How much homework should we expect?

A: Three to four hours a week: problem sets after each class, proof write-ups during the geometry phase, journal entries during the problem-solving phase, and timed papers in the

final months. The write-ups are non-negotiable, because this course grades reasoning, not answer lists. It is a real commitment, and we would rather say so now than surprise you in month 2.

Q: Can we try a class before enrolling?

A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The demo includes a short diagnostic, after which the teacher will tell you plainly whether the student is ready for this course or better served by our school maths courses first.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/jee-foundation-maths-course-class-8-10>

=====

COURSE 81 OF 118: KIDS CODING MASTERCLASS: SCRATCH, GAMES, APPS & AI PROJECTS

=====

Course link: <https://learn.modernagecoders.com/courses/kids-coding-blocks-masterclass>

Subject area: Kids & Youth Programming Education

Designed for: Kids

Who can join: Complete Beginner (Age 7+) to Advanced Creator

Course length: 24 months (104 weeks)

Time needed: 5-10 hours/week recommended

Certificate: Digital Creator & Coder Certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live 2-year coding program for kids: from zero experience, start with Scratch block coding, build games, apps and AI projects, then move to real programming.

In one line: From First Click to Digital Creator - A Journey of Innovation and Fun

ABOUT THE PROGRAMME

This revolutionary 2-year program transforms complete beginners into confident digital creators using visual block programming. Perfect for kids (7+), teens, and even adults who want to start coding without typing complex syntax. Students will master block-based coding through platforms like Scratch, Code.org, and MakeCode, create games and animations, explore AI tools, design graphics with Canva, edit videos, master Microsoft Office, build mobile apps, and eventually transition to text-based programming. By completion, learners will have created 50+ projects and gained essential 21st-century digital skills.

WHAT MAKES THIS COURSE DIFFERENT

- 100% visual learning - no typing required to start
- Age-appropriate content with separate tracks for kids, teens, and adults
- Covers ALL major block coding platforms
- Includes AI tools, design, and productivity software
- Game-based learning approach
- Parent dashboard for progress tracking
- Transition path to professional programming
- Real-world digital skills beyond just coding

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Block Coding Basics, Scratch, Game Logic, Creative Projects

Phase 2: Advanced Blocks (Months 7-12): Complex Games, AI Tools, App Development, Robotics

Phase 3: Creative Suite (Months 13-18): Design, Video Editing, Microsoft Skills, Advanced Projects

Phase 4: Professional Path (Months 19-24): Text Coding Transition, Web Development, Portfolio Building

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic reading and writing skills (Grade 2+)

Coding experience: None required - complete beginner friendly

Equipment: Computer/laptop/tablet with internet

Time commitment: 5-10 hours per week (flexible)

English: Basic English understanding

Motivation: Curiosity and willingness to learn

WHO THIS COURSE IS FOR

Kids: Ages 7-12 wanting to learn coding through play

Teens: Ages 13-17 building tech skills for future
Adults: Beginners wanting visual introduction to coding
Educators: Teachers wanting to learn coding education
Parents: Parents wanting to learn alongside children
Everyone: Anyone interested in creative technology

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & BLOCK CODING BASICS (MONTHS 1-6, WEEKS 1-26) -----

Start your coding journey with fun, visual programming. Learn computational thinking, create games, animations, and interactive stories using block-based platforms.

Months 1-2: Introduction to Coding & Scratch Basics (Week 1-8)

Welcome to the Digital World

Topics:

- What is coding? Why is it important?
- How computers understand instructions
- Introduction to algorithms with real-life examples
- Setting up your coding workspace
- Creating accounts on Scratch, Code.org
- Navigating Scratch interface
- Understanding sprites and stages
- Basic blocks: Motion, Looks, Sound
- Your first animation: Making a cat walk
- Saving and sharing projects
- Online safety and digital citizenship
- Creating your coder profile

Projects built:

- Animated greeting card
- Pet animation with sounds
- Interactive 'About Me' project

Practice: Daily: 15 min block exploration, 30 min creative time

Motion and Animation in Scratch

Topics:

- Coordinate system (x, y positions)
- Moving sprites: go to, glide, move steps
- Rotation: turn, point in direction
- Creating smooth animations
- Using costumes for animation
- Backdrop changes and scenes
- Sound effects and music
- Recording custom sounds
- Text-to-speech blocks
- Animation principles: timing, easing
- Creating realistic movements
- Sprite interactions

Projects built:

- Animated story with 3 scenes
- Dancing party animation
- Virtual pet that moves and makes sounds

Practice: Create 5 different animation sequences

Events and Interactivity

Topics:

- Event blocks: when flag clicked, when key pressed
- Mouse events: when sprite clicked
- Broadcasting messages between sprites
- Creating interactive buttons
- Keyboard controls for games
- Mouse following mechanics
- Touch controls for tablets
- Creating menus and navigation
- User input with 'ask' blocks
- Storing answers in variables (introduction)
- Creating choose-your-own adventure stories
- Interactive art projects

Projects built:

- Interactive quiz game
- Virtual piano/musical instrument
- Dress-up game with clickable items

Practice: Build 10 interactive mini-projects

Introduction to Game Logic

Topics:

- If-then statements (conditionals)
- If-else logic blocks
- Collision detection: touching sprite/color

- Score keeping with variables
- Lives and game over conditions
- Timer and countdown mechanics
- Random numbers for surprises
- Creating power-ups
- Level progression basics
- Win and lose conditions
- Game states: start, playing, game over
- Debugging techniques

Projects built:

- Catch the falling objects game
- Simple maze game
- Whack-a-mole style game
- Number guessing game

Practice: Create 5 simple games with scoring

Months 3-4: Advanced Scratch & Code.org (Week 9-17)

Loops and Patterns

Topics:

- Forever loops for continuous actions
- Repeat loops with counters
- Repeat until conditions
- Nested loops for complex patterns
- Creating geometric patterns
- Drawing with pen blocks
- Stamp tool for patterns
- Color effects and graphics
- Creating fractals and spirals
- Loop optimization
- Breaking out of loops
- Infinite loop prevention

Projects built:

- Geometric art generator
- Pattern-based puzzle game
- Automated drawing machine
- Kaleidoscope effect

Practice: Create 20 different patterns using loops

Variables and Data

Topics:

- Creating and naming variables
- Local vs. global variables
- Changing variable values
- Variables for game mechanics
- Lists (arrays) introduction
- Adding/removing from lists
- List operations: length, contains
- High score systems
- Inventory systems for games
- Data persistence with cloud variables
- Mathematical operations with variables
- Creating calculators

Projects built:

- Advanced calculator with memory
- Shopping game with inventory
- Quiz with score tracking
- RPG with character stats

Practice: Build 10 projects using variables and lists

Code.org and Hour of Code

Topics:

- Introduction to Code.org platform
- Block-based JavaScript preview
- Minecraft Hour of Code
- Star Wars coding challenges
- Dance Party creation
- App Lab basics
- Creating mobile app interfaces
- Event-driven programming
- Debugging with Code.org
- CS Fundamentals courses
- Problem-solving strategies
- Computational thinking puzzles

Projects built:

- Complete Minecraft adventurer
- Create dance party choreography
- Build app prototype in App Lab
- Solve 20 Code.org puzzles

Practice: Daily Code.org challenges

Clone Games and Complex Projects

Topics:

- Analyzing existing games
- Breaking down game mechanics
- Planning complex projects
- Creating Pong clone
- Building Flappy Bird clone
- Snake game mechanics
- Platform game basics
- Scrolling backgrounds
- Multiple levels design
- Save game functionality
- Leaderboards with cloud data
- Multiplayer basics

Projects built:

- Flappy Bird complete clone
- Platform game with 3 levels
- Two-player competitive game
- Pac-Man style maze game

Practice: Recreate 5 classic games

Month 3-4 Review & Showcase

Topics:

- Project presentation skills
- Creating game trailers
- Writing game descriptions
- Peer review and feedback
- Code optimization
- Performance improvements
- Portfolio organization

Projects built:

- CAPSTONE: Original game with all learned features
- Game includes: menu, levels, scoring, saves
- Presentation video of game

Assessment: Complete Scratch certification test

Months 5-6: MakeCode & Physical Computing (Week 18-26)

Microsoft MakeCode Arcade

Topics:

- Introduction to MakeCode Arcade
- Sprite creation and editing
- Tile maps and level design
- Physics in games
- Projectiles and effects
- Enemy AI basics
- Scoring and lives system
- Sound effects creation
- Custom sprite editor
- Publishing games online
- Sharing with QR codes
- JavaScript preview in MakeCode

Projects built:

- Adventure game with map
- Shooting game with enemies
- Racing game with obstacles
- RPG with story elements

Practice: Create 10 MakeCode Arcade games

Micro:bit Programming

Topics:

- What is Micro:bit?
- MakeCode for Micro:bit
- LED display programming
- Button inputs A and B
- Accelerometer and shake detection
- Temperature sensor
- Light sensor projects
- Radio communication between devices
- Creating wearable projects
- Music and sounds on Micro:bit
- Connecting external components
- Battery power and portability

Projects built:

- Digital dice
- Step counter/fitness tracker
- Two-player rock-paper-scissors
- Temperature alarm system

Practice: Build 15 Micro:bit projects

Blockly and Google Tools

Topics:

- Introduction to Blockly
- Blockly Games progression
- Maze solving algorithms

- Bird physics game
- Turtle graphics programming
- Movie creation with Blockly
- Music composition blocks
- Pond game strategies
- JavaScript code generation
- Custom Blockly workspaces
- Google CS First platform
- Storytelling with code

Projects built:

- Complete all Blockly Games
- Turtle art masterpiece
- Animated movie with Blockly
- Music composition project

Practice: Master all Blockly challenges

Game Design Principles

Topics:

- What makes games fun?
- Game mechanics vs. dynamics
- Level design principles
- Difficulty curves
- Player feedback systems
- Reward mechanisms
- Story in games
- Character development
- Game art basics
- Sound design importance
- Playtesting methods
- Iterative design process

Projects built:

- Design document for original game
- Level pack for existing game
- Game with adaptive difficulty
- Story-driven adventure game

Practice: Analyze 10 popular games

Phase 1 Final Project

Topics:

- Project planning and scope
- Asset creation and management
- Testing and debugging
- Performance optimization
- Cross-platform considerations
- Publishing and sharing

Projects built:

- MAJOR CAPSTONE: Multi-platform Game
- Create same game in Scratch, MakeCode, and Blockly
- Include: story mode, endless mode, multiplayer option
- Full documentation and presentation

Assessment: Phase 1 Comprehensive Assessment

----- PHASE 2: ADVANCED BLOCKS & CREATIVE COMPUTING (MONTHS 7-12, WEEKS 27-52) -----

Explore advanced platforms, AI tools, mobile app development, and creative computing with more sophisticated block-based environments.

Months 7-8: MIT App Inventor & Mobile Apps (Week 27-35)

App Inventor Basics

Topics:

- Introduction to MIT App Inventor
- Designer vs. Blocks editor
- Screen layouts and components
- Buttons, labels, textboxes
- Images and media components
- Screen navigation
- Properties and styling
- Event handlers in apps
- Variables in App Inventor
- Procedures (functions)
- Testing with companion app
- Building APK files

Projects built:

- Calculator app
- To-do list app
- Flashcard study app
- Personal diary app

Practice: Build 10 basic mobile apps

Advanced App Features

Topics:

- Lists and databases in apps
- TinyDB for local storage
- CloudDB for online storage
- Firebase integration basics
- Camera and image picker
- Sound recorder and player
- Text-to-speech and speech recognition
- Canvas for drawing
- Sprite animations in apps
- Location sensor (GPS)
- Accelerometer games
- Sharing functionality

Projects built:

- Photo editing app
- Voice note recorder
- Drawing app with save
- Location-based reminder

Practice: Add advanced features to previous apps

Game Development in App Inventor

Topics:

- Game loop concepts
- Sprite collisions in apps
- Score and high score systems
- Timer-based games
- Touch and drag mechanics
- Accelerometer controls
- Sound effects in games
- Multiple screens for games
- Game state management
- Advertising basics (AdMob)
- Publishing to Play Store basics
- App monetization introduction

Projects built:

- Flappy Bird mobile version
- Tilt-controlled maze game
- Memory card game
- Multiplayer tic-tac-toe

Practice: Create 5 mobile games

IoT and Connected Apps

Topics:

- Bluetooth connections
- Connecting to Arduino
- Web APIs in App Inventor
- JSON data handling
- Weather API integration
- QR code scanner
- Barcode scanning
- Push notifications basics
- Social media sharing
- Email and SMS from apps
- Maps and navigation
- Augmented reality basics

Projects built:

- Weather app with API
- QR code treasure hunt
- IoT home controller
- Social sharing app

Practice: Build 5 connected apps

Roblox Studio Introduction

Topics:

- What is Roblox Studio?
- Basic building tools
- Parts and properties
- Terrain editor
- Basic scripting preview
- Models and assets
- Lighting and effects
- Sound in Roblox
- Publishing games
- Robux and monetization
- Community guidelines
- Collaborative development

Projects built:

- Simple obby (obstacle course)
- Basic tycoon game
- Adventure map
- Multiplayer arena

Practice: Create 5 Roblox experiences

Months 9-10: AI Tools & Creative Computing (Week 36-44)

Introduction to AI for Kids

Topics:

- What is Artificial Intelligence?
- Machine Learning basics for kids
- Training AI models concept
- Scratch + Machine Learning 4 Kids
- Image recognition projects
- Text classification basics
- Sound recognition
- Creating training data
- Testing AI models
- AI ethics for kids
- Bias in AI systems
- Future of AI

Projects built:

- Rock-paper-scissors AI
- Emotion detector with images
- Chatbot in Scratch
- Voice command game

Practice: Train 10 different AI models

Teachable Machine & AI Projects

Topics:

- Google Teachable Machine
- Training image models
- Training sound models
- Training pose models
- Exporting models
- Using models in Scratch
- p5.js web editor basics
- AI-powered games
- Computer vision projects
- Gesture control games
- AI music creation
- AI art generation basics

Projects built:

- Gesture-controlled game
- AI fitness coach
- Sound-activated animations
- AI art generator

Practice: Create 15 AI-enhanced projects

Canva Design Fundamentals

Topics:

- Introduction to Canva
- Design principles for kids
- Color theory basics
- Typography fundamentals
- Creating social media posts
- Poster and flyer design
- Presentation templates
- Logo design basics
- Infographic creation
- Animation in Canva
- Video editing in Canva
- Brand kit creation

Projects built:

- Personal brand identity
- School project presentations
- YouTube thumbnail designs
- Event poster series

Practice: Create 20 designs in different formats

Video Creation & Editing

Topics:

- Introduction to video editing
- Canva Video editor
- OpenShot basics
- Recording screen content
- OBS Studio basics
- Adding titles and text
- Transitions and effects
- Audio editing basics
- Green screen techniques
- Creating YouTube content
- TikTok and Shorts editing
- Video optimization

Projects built:

- Game trailer video
- Tutorial video creation
- Stop-motion animation

- Vlog-style content

Practice: Edit 10 different video types

Digital Storytelling

Topics:

- Story structure basics
- Character development
- Digital comic creation
- Interactive stories with Twine
- Animation storytelling
- Podcast basics for kids
- Sound editing with Audacity
- Creating audiobooks
- Digital portfolio creation
- Blog writing basics
- Content planning
- Online presence building

Projects built:

- Interactive story in Twine
- Digital comic book
- Podcast episode
- Animation short film

Practice: Create stories in 5 different mediums

Months 11-12: Microsoft Skills & Productivity (Week 45-52)

Microsoft Word Mastery

Topics:

- Word interface and navigation
- Document formatting basics
- Fonts, colors, and styles
- Paragraph formatting
- Lists and bullets
- Tables and columns
- Images and graphics
- Headers and footers
- Page numbers and breaks
- Spell check and grammar
- Track changes and comments
- Templates and themes

Projects built:

- School report with formatting
- Newsletter design
- Resume template
- Story book with images

Practice: Create 15 different document types

Microsoft PowerPoint Excellence

Topics:

- Creating presentations
- Slide layouts and design
- Themes and templates
- Adding multimedia content
- Animations and transitions
- Slide master customization
- Charts and SmartArt
- Presenter view and notes
- Recording presentations
- Sharing and collaboration
- Interactive presentations
- Presentation best practices

Projects built:

- School project presentation
- Interactive quiz game
- Animated story presentation
- Tutorial slideshow

Practice: Design 10 professional presentations

Microsoft Excel Basics

Topics:

- Excel interface overview
- Cells, rows, and columns
- Data entry and formatting
- Basic formulas (SUM, AVERAGE)
- Sorting and filtering
- Charts and graphs
- Conditional formatting
- Basic functions
- Data validation
- Printing spreadsheets
- Sharing workbooks
- Excel for homework help

Projects built:

- Grade tracker spreadsheet
- Budget planner
- Sports statistics tracker
- Science data analysis

Practice: Build 10 useful spreadsheets

Microsoft Teams & OneNote

Topics:

- Microsoft Teams for students
- Joining and creating teams
- Chat and video calls
- File sharing and collaboration
- Assignment submissions
- OneNote organization
- Digital notebooks
- Note-taking techniques
- Drawing and handwriting
- Audio and video notes
- Sharing notebooks
- Study tips with OneNote

Projects built:

- Organized study notebook
- Group project in Teams
- Digital planner in OneNote
- Class notes system

Practice: Organize all learning with OneNote

Phase 2 Integration Project

Topics:

- Combining all skills learned
- Project management basics
- Documentation creation
- Presentation preparation
- Video demonstration
- Portfolio website planning

Projects built:

- MAJOR CAPSTONE: Educational Game with Documentation
- Create: AI-powered educational game
- Include: Design docs, presentations, videos, marketing materials
- Full Microsoft Office documentation suite

Assessment: Phase 2 Final Assessment - Multi-skill project

----- PHASE 3: ADVANCED PLATFORMS & PROFESSIONAL SKILLS (MONTHS 13-18, WEEKS 53-78) -----

Master advanced creative tools, 3D design, advanced AI, professional software, and begin transition to text-based coding.

Months 13-14: 3D Design & Virtual Worlds (Week 53-61)

Minecraft Education Edition

Topics:

- Minecraft Education setup
- Code Builder introduction
- MakeCode in Minecraft
- Agent programming
- Building automation
- Redstone basics
- Command blocks
- Creating educational worlds
- Chemistry in Minecraft
- History lessons in Minecraft
- Collaborative building
- Classroom management tools

Projects built:

- Automated farm system
- Historical monument recreation
- Science experiment world
- Adventure map with coding

Practice: Complete 20 coding challenges in Minecraft

CoSpaces & VR/AR Creation

Topics:

- Introduction to CoSpaces Edu
- 3D scene creation
- CoBlocks programming
- Physics in virtual worlds
- Character animation
- Sound and music in 3D
- VR experience design
- AR marker creation
- 360° environments
- Interactive museums

- Virtual field trips
- Publishing VR experiences

Projects built:

- Virtual museum tour
- AR treasure hunt
- VR escape room
- Interactive 3D story

Practice: Create 10 VR/AR experiences

Tinkercad 3D Design

Topics:

- 3D modeling basics
- Tinkercad interface
- Basic shapes and operations
- Grouping and aligning
- Hole creation
- Precise measurements
- 3D printing preparation
- Circuit simulation
- Arduino simulation
- Codeblocks in Tinkercad
- Importing and exporting
- Design thinking process

Projects built:

- 3D printed keychain design
- Architectural model
- Arduino circuit simulation
- Mechanical toy design

Practice: Design 20 3D models

Advanced Roblox Development

Topics:

- Roblox Studio advanced tools
- Lua scripting basics
- Variables and functions in Lua
- Events and triggers
- GUI creation
- Leaderboards and stats
- Game passes and monetization
- Team create features
- Advanced terrain
- Particle effects
- Animation editor
- Publishing and marketing

Projects built:

- Complex tycoon game
- RPG with quests
- Racing game with upgrades
- Social hangout space

Practice: Publish 5 Roblox games

Unity with Visual Scripting

Topics:

- Unity Hub and installation
- Unity interface basics
- GameObject concepts
- Prefabs and components
- Visual scripting with Bolt
- Basic physics in Unity
- 2D game development
- Sprite animations
- Simple AI behaviors
- UI design in Unity
- Building for platforms
- Unity Learn resources

Projects built:

- 2D platformer game
- Top-down adventure
- Physics puzzle game
- Mobile game prototype

Practice: Complete Unity Learn tutorials

Months 15-16: Advanced AI & Data Science (Week 62-70)

Advanced AI Tools

Topics:

- ChatGPT for learning
- Prompt engineering basics
- AI writing assistants
- GitHub Copilot introduction
- DALL-E and image generation
- Midjourney basics
- AI music generation

- AI video tools
- Runway ML basics
- AI in game development
- Ethical AI use
- AI safety and limitations

Projects built:

- AI-assisted story writing
- AI art gallery
- AI-powered game NPCs
- Educational AI chatbot

Practice: Create 20 AI-enhanced projects

Data Science for Kids

Topics:

- What is data science?
- Collecting data responsibly
- Google Sheets for data
- Basic statistics concepts
- Creating surveys with Google Forms
- Data visualization basics
- Charts and graphs
- Tableau Public basics
- Infographic creation
- Data storytelling
- Privacy and data ethics
- Real-world data projects

Projects built:

- School survey analysis
- Weather data tracking
- Sports statistics dashboard
- Environmental data project

Practice: Analyze 10 different datasets

Internet of Things (IoT)

Topics:

- What is IoT?
- Smart home concepts
- IFTTT (If This Then That)
- Creating automations
- Smart sensors basics
- Blynk IoT platform
- Node-RED visual programming
- Weather stations
- Home automation projects
- IoT security basics
- Future of connected devices
- Environmental monitoring

Projects built:

- Smart plant watering system
- Weather monitoring station
- Home automation dashboard
- IoT notification system

Practice: Build 10 IoT automations

Robotics Programming

Topics:

- Introduction to robotics
- VEXcode VR robots
- Virtual robot challenges
- Sensors and actuators
- Line following algorithms
- Maze solving strategies
- LEGO Mindstorms basics
- EV3 programming
- Robot design principles
- Competition preparation
- Team collaboration
- Documentation skills

Projects built:

- Virtual robot maze solver
- Line following robot
- Object sorting robot
- Dance choreography robot

Practice: Complete 20 robot challenges

Advanced Maker Projects

Topics:

- Arduino simulation online
- Basic electronics concepts
- LED projects
- Sensor projects
- Motors and servos
- LCD displays

- Wearable technology
- E-textiles basics
- Paper circuits
- Makey Makey projects
- Combining physical and digital
- Maker portfolio creation

Projects built:

- Interactive art installation
- Wearable LED project
- Musical instrument with Makey Makey
- Smart device prototype

Practice: Build 10 maker projects

Months 17-18: Professional Tools & Career Skills (Week 71-78)

Adobe Creative Suite Basics

Topics:

- Adobe Creative Cloud overview
- Photoshop basics for kids
- Photo editing fundamentals
- Creating graphics and logos
- Adobe Spark for web
- Premiere Rush video editing
- Adobe XD for app design
- Creating mockups
- Animation with Adobe Animate
- Portfolio creation in Adobe
- Creative Cloud libraries
- Collaboration features

Projects built:

- Photo manipulation project
- App design mockup
- Animated logo
- Video advertisement

Practice: Master 5 Adobe applications

Google Workspace Mastery

Topics:

- Gmail professional use
- Google Drive organization
- Google Docs collaboration
- Google Sheets formulas
- Google Slides advanced
- Google Forms for surveys
- Google Sites for portfolios
- Google Calendar management
- Google Meet etiquette
- Google Classroom usage
- Chrome extensions
- Productivity tips

Projects built:

- Collaborative research project
- Portfolio website on Google Sites
- Data analysis in Sheets
- Professional email templates

Practice: Create 20 Google Workspace projects

Digital Marketing Basics

Topics:

- Social media basics
- Content creation strategies
- YouTube channel setup
- Instagram for creators
- TikTok content ideas
- Hashtag strategies
- Analytics basics
- SEO fundamentals
- Email marketing basics
- Creating landing pages
- Digital advertising basics
- Online safety and privacy

Projects built:

- YouTube channel launch
- Social media campaign
- Email newsletter
- Landing page for project

Practice: Create content for 5 platforms

Entrepreneurship for Kids

Topics:

- Business idea generation
- Problem-solving mindset
- Market research basics

- Creating business plans
- Pitching ideas
- Financial literacy basics
- Budgeting for projects
- Team building
- Leadership skills
- Time management
- Goal setting
- Growth mindset

Projects built:

- Business plan presentation
- Pitch deck creation
- Budget spreadsheet
- Team project management

Practice: Develop 5 business ideas

Phase 3 Showcase Project

Topics:

- Project ideation
- Requirements gathering
- Project timeline creation
- Resource planning
- Testing and iteration
- Documentation best practices

Projects built:

- MAJOR CAPSTONE: Professional Digital Product
- Options: Educational platform, Game studio, Digital magazine, App suite
- Include: Full documentation, marketing materials, presentation
- Showcase all Phase 3 skills

Assessment: Phase 3 Portfolio Review

----- PHASE 4: TEXT CODING TRANSITION & PROFESSIONAL DEVELOPMENT (MONTHS 19-24, WEEKS 79-104) -----

Transition from blocks to text-based programming, web development basics, and professional portfolio creation.

Months 19-20: Python Programming Basics (Week 79-87)

Introduction to Python

Topics:

- Why learn text-based coding?
- Python installation and setup
- IDLE and code editors
- Python Turtle graphics
- Variables and data types
- Print statements and input
- Comments and documentation
- Basic math operations
- String manipulation
- Converting blocks to Python
- Debugging Python code
- Python style guide

Projects built:

- Turtle art generator
- Text-based calculator
- Mad libs game
- ASCII art creator

Practice: Convert 10 Scratch projects to Python

Python Control Flow

Topics:

- If-elif-else statements
- Comparison operators
- Logical operators
- While loops
- For loops
- Range function
- Break and continue
- Nested loops
- Functions basics
- Parameters and returns
- Scope and variables
- Importing modules

Projects built:

- Number guessing game
- Rock-paper-scissors
- Password generator
- Quiz application

Practice: Build 20 Python programs

Python Data Structures

Topics:

- Lists in Python
- List methods and operations
- Tuples and sets
- Dictionaries
- List comprehensions
- 2D lists (matrices)
- File handling basics
- Reading and writing files
- CSV files
- JSON data
- Error handling
- Try-except blocks

Projects built:

- Contact manager
- High score system
- Data analysis program
- File organizer

Practice: Master Python data structures

Python Game Development

Topics:

- Pygame installation
- Game window creation
- Drawing shapes and images
- Sprite handling
- Keyboard and mouse input
- Game loop in Python
- Collision detection
- Sound in Pygame
- Sprite animations
- Particle effects
- Game states
- Packaging games

Projects built:

- Snake game
- Space shooter
- Platform game
- Puzzle game

Practice: Create 10 Python games

Python for Data and AI

Topics:

- NumPy basics
- Data visualization with matplotlib
- Simple data analysis
- Web scraping basics
- APIs with Python
- Chatbot with Python
- Image processing basics
- Automation scripts
- Scheduling tasks
- GUI with Tkinter
- Creating executables
- Sharing Python projects

Projects built:

- Data visualization dashboard
- Web scraper tool
- Desktop application
- Automation suite

Practice: Build 10 practical Python tools

Months 21-22: Web Development Fundamentals (Week 88-96)

HTML & CSS Basics

Topics:

- How websites work
- HTML structure and tags
- Creating web pages
- HTML5 semantic elements
- Links and navigation
- Images and media
- Forms and inputs
- CSS introduction
- Selectors and properties
- Colors and fonts
- Box model
- Flexbox basics

Projects built:

- Personal website
- Recipe page
- Photo gallery
- Contact form

Practice: Build 15 web pages

Responsive Web Design

Topics:

- Mobile-first design
- Media queries
- Responsive images
- Grid layout
- CSS animations
- Transitions and transforms
- Custom properties (variables)
- CSS frameworks introduction
- Bootstrap basics
- Accessibility basics
- SEO fundamentals
- Web hosting basics

Projects built:

- Responsive portfolio
- Landing page
- Blog template
- Business website

Practice: Make 10 responsive websites

JavaScript for the Web

Topics:

- JavaScript in HTML
- DOM manipulation
- Event handling
- Functions and variables
- Arrays and objects
- Local storage
- Fetch API basics
- JSON handling
- Form validation
- Interactive elements
- Simple animations
- JavaScript libraries intro

Projects built:

- Interactive quiz
- Todo list app
- Weather widget
- Calculator web app

Practice: Add JavaScript to 15 projects

Full Stack Basics

Topics:

- Client-server model
- Node.js introduction
- Express.js basics
- Creating APIs
- Database basics
- MongoDB introduction
- Connecting frontend to backend
- User authentication basics
- Deployment basics
- Netlify
- Version control with Git
- GitHub for projects

Projects built:

- Full stack todo app
- User registration system
- Blog with backend
- Chat application

Practice: Build 5 full stack applications

Framework Introduction

Topics:

- Why use frameworks?
- React.js basics
- Components and props
- State management intro
- Vue.js introduction
- Angular overview
- Choosing frameworks
- Mobile app frameworks
- React Native basics
- Progressive Web Apps
- Future of web development
- Learning path planning

Projects built:

- React todo app
- Component library
- Single page application

- Mobile app prototype
- Practice: Explore 3 different frameworks

Month 23: Specialization & Advanced Projects (Week 97-100)

Choose Your Specialization

Topics:

- Game development path
- Web development path
- Mobile app development
- AI and data science
- Creative technology
- Robotics and IoT
- Cybersecurity basics
- Cloud computing intro
- DevOps for beginners
- Blockchain basics
- Career exploration
- Industry research

Projects built:

- Specialization research project
- Technology comparison
- Career path presentation
- Skill assessment

Practice: Explore 5 specialization areas

Advanced Project Planning

Topics:

- Project management basics
- Agile methodology intro
- User stories and requirements
- Wireframing and mockups
- Technical documentation
- Testing strategies
- Quality assurance basics
- Code review practices
- Collaboration tools
- Communication skills
- Presentation skills
- Demo preparation

Projects built:

- Complete project plan
- Technical specification
- Testing documentation
- User manual

Practice: Plan 3 complex projects

Open Source Contribution

Topics:

- What is open source?
- Finding projects to contribute
- Reading documentation
- Understanding codebases
- Making first contribution
- Pull requests and issues
- Code of conduct
- Community interaction
- Building reputation
- Creating own open source
- Licensing basics
- Maintaining projects

Projects built:

- First open source contribution
- Create open source project
- Documentation contribution
- Bug fix submission

Practice: Contribute to 5 projects

Professional Skills Development

Topics:

- Resume building for tech
- Portfolio website creation
- LinkedIn profile optimization
- GitHub profile enhancement
- Interview preparation
- Technical interview basics
- Problem-solving strategies
- Communication skills
- Teamwork and collaboration
- Remote work skills
- Continuous learning
- Networking basics

Projects built:

- Professional portfolio site
- Technical blog posts
- Video resume
- Skill showcase projects

Practice: Complete professional preparation

Month 24: Final Projects & Graduation (Week 101-104)

Final Capstone Project - Development

Topics:

- Project selection and approval
- Comprehensive planning
- Architecture design
- Technology stack selection
- Development sprints
- Regular check-ins
- Problem-solving sessions
- Code optimization
- Feature implementation
- Testing and debugging
- Performance tuning
- Security considerations

Projects built:

- FINAL CAPSTONE: Complete Digital Solution
- Options: Educational platform, Social impact app, Game studio project, Business solution, Creative technology showcase
- Requirements: Multiple technologies, full documentation, presentation ready, deployable, scalable

Final Project Polish & Presentation

Topics:

- Final testing phases
- Bug fixing and optimization
- User interface polish
- Documentation completion
- Deployment preparation
- Presentation creation
- Demo video recording
- Marketing materials
- Social media presence
- Launch planning
- Feedback incorporation
- Final submission preparation

Deliverables:

- Complete project codebase
- Live deployment URL
- Technical documentation
- User documentation
- Presentation deck
- Demo video (5-10 minutes)
- Marketing website
- Social media kit

Graduation & Future Planning

Topics:

- Final project presentations
- Peer review and feedback
- Industry panel discussion
- Career guidance session
- Alumni network introduction
- Continued learning resources
- Advanced course options
- Mentorship programs
- Internship opportunities
- Freelance opportunities
- Startup incubation info
- Celebration and recognition

Deliverables:

- Final portfolio website
- Complete GitHub profile
- Professional resume
- LinkedIn profile
- Certification of completion
- Letter of recommendation eligibility
- Alumni directory listing
- Future learning roadmap

Assessment: FINAL COMPREHENSIVE ASSESSMENT - Complete evaluation of all skills

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 25+ games in Scratch, MakeCode projects, Micro:bit inventions, Blockly challenges
- Phase 2: 20+ mobile apps, AI projects, Canva designs, Microsoft Office suite, video projects

- Phase 3: VR/AR experiences, 3D models, Roblox games, advanced AI, IoT projects, robotics
- Phase 4: Python programs, web applications, full stack projects, specialization projects
- Final: Professional-grade capstone project using multiple technologies

Total projects: 100+ projects from simple animations to complex applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Block Programming: Scratch, Code.org, MakeCode, Blockly, App Inventor
- Game Development: Roblox Studio, Unity (visual scripting), Minecraft Education
- AI & ML: Machine Learning for Kids, Teachable Machine, AI tools integration
- Design: Canva, Adobe basics, 3D modeling with Tinkercad, UI/UX basics
- Microsoft Suite: Word, PowerPoint, Excel, Teams, OneNote mastery
- Video & Media: Video editing, audio production, content creation, streaming
- Programming: Python, HTML/CSS/JavaScript basics, database fundamentals
- Professional: Project management, collaboration, presentation, documentation
- 21st Century: Problem-solving, computational thinking, creativity, digital literacy

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-5 hours
 Hands on coding: 3-5 hours
 Projects: 2-3 hours
 Practice problems: 1-2 hours
 Total per week: 5-10 hours (flexible)

SUPPORT INCLUDED

Live sessions: Weekly interactive coding sessions
 Mentorship: Age-appropriate mentor guidance
 Community: Safe, moderated community platform
 Code review: Project feedback and suggestions
 Parent dashboard: Progress tracking for parents
 Career support: Age-appropriate career exploration

CERTIFICATION DETAIL

Phase certificates: Digital badges for each phase
 Final certificate: Certified Digital Creator & Programmer
 LinkedIn badge: Shareable LinkedIn certification
 Industry recognized: Recognized by educational institutions
 Portfolio projects: 100+ projects for showcase

WHERE THIS CAN LEAD

- Junior Game Developer
- App Developer
- Web Developer
- Digital Content Creator
- UI/UX Designer
- AI/ML Specialist
- Robotics Programmer
- Tech Educator/Instructor
- Freelance Developer
- Tech Entrepreneur

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does my child build over the 2-year program?

A: The final phase transitions into typed code, where students write their first Python programs and simple websites.

Q: What age should my child be to start?

A: The program is designed for ages 7 and up, needing only Grade 2 level reading and writing. Because everything starts with visual blocks rather than typing, even young beginners build real projects within the first weeks.

Q: Is any coding knowledge required beforehand?

A: None. Block coding was invented exactly for this: children drag and snap logic blocks together, so there is no syntax to type or memorize. The course assumes a complete beginner and builds up gradually across its four phases.

Q: Which platforms are used, and are they free?

A: The core platforms, Scratch, Code.org, MakeCode, and MIT App Inventor, are all free and browser-based, so nothing needs to be purchased. A computer, laptop, or even a tablet with internet access is enough to participate.

Q: What happens after the block coding phase?

A: That is the heart of this course: the final six months deliberately bridge from blocks into real text programming with Python and web development basics. Students leave ready to join our full Python, web development, or AI courses without the usual beginner shock.

Q: Is there a free trial class?

A: Yes. You can book a free demo at learn.modernagecoders.com/contact with zero commitment, and many parents sit in on the demo to see the teaching style for themselves. We hold a 4.9 star rating from 547+ Google reviews, and the demo class is where most of those families started.

Q: Are these live classes or videos?

A: Every session is a live 1-hour online class taught by a real teacher; we never substitute pre-recorded videos. Groups stay small and interactive, so children code along during class rather than just watching.

Q: Do students get certified at the end?

A: Yes, your child receives a Digital Creator and Coder completion certificate. Alongside it they build a portfolio of 50+ real projects, from games to mobile apps, that shows exactly what they can do.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/kids-coding-blocks-masterclass>

=====

COURSE 82 OF 118: KOTLIN PROGRAMMING: ZERO TO ANDROID & BACKEND DEVELOPER

=====

Course link: <https://learn.modernagecoders.com/courses/complete-kotlin-programming-masterclass-professional>

Subject area: Professional Kotlin Development

Designed for: College students

Who can join: Complete Beginner to Professional Developer

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Industry-recognized Kotlin Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Kotlin course online: a 1-year masterclass from basics to professional apps, covering Android, Spring Boot, coroutines and multiplatform for a Kotlin career.

In one line: From 'Hello World' to Professional Android & Backend Applications

ABOUT THE PROGRAMME

This is not just a Kotlin course, it's a complete transformation into a versatile Kotlin professional. Whether you're a curious beginner, student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled Kotlin developer capable of building modern Android apps, scalable backend systems, multiplatform applications, and cloud-native solutions. You'll master Kotlin from ground zero to professional architect level: from basic syntax to advanced language features, from console applications to full-stack Android apps with Jetpack Compose, from JDBC to Spring Boot with Kotlin, from single-platform to Kotlin Multiplatform projects. By the end, you'll have built 50+ projects, mastered the entire Kotlin ecosystem, and be ready for senior Kotlin/Android developer roles in top tech companies.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - perfect for complete beginners
- Separate learning tracks for kids (12+), teens, and adults
- 1 year of structured, industry-aligned learning
- Covers Kotlin + Android + Backend (Spring Boot) + Multiplatform
- Real-world projects and industry case studies
- Hands-on with latest Kotlin versions and modern features
- Android development with Jetpack Compose
- Interview preparation for top companies
- Lifetime access with continuous updates
- Build production-ready professional portfolio
- Direct path to high-paying Kotlin/Android developer jobs

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Kotlin Fundamentals, OOP, Functional Programming Basics

Phase 2: Intermediate (Months 4-6): Advanced Kotlin, Collections, Coroutines, Design Patterns

Phase 3: Android (Months 7-9): Android Development, Jetpack Compose, Material Design, APIs
Phase 4: Professional (Months 10-12): Spring Boot Kotlin, Multiplatform, Cloud, Career Excellence

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to all backgrounds
Coding experience: Absolute beginner friendly - zero programming knowledge required
Age: 12+ years (separate learning tracks for kids, teens, adults)
Equipment: Computer/laptop (Windows/Mac/Linux), minimum 8GB RAM, internet connection
Time commitment: 15-20 hours per week consistently for best results
English: Basic reading comprehension (materials also in Hindi)
Motivation: Strong desire to become a professional Kotlin developer
Math: Basic mathematics (taught as needed)

WHO THIS COURSE IS FOR

Kids: Age 12-14: Simplified track, visual learning, game-based projects
Teens: Age 15-18: Student-focused, college prep, app development projects
Students: College students: Placement prep, internship readiness, portfolio building
Working professionals: Career switchers from any background to Kotlin developer
Java developers: Java developers wanting modern Kotlin ecosystem
Android developers: Android devs wanting to master Kotlin deeply
Backend engineers: Backend developers adding Kotlin to skillset
Entrepreneurs: Build Android apps and backends for your startup
Freelancers: Offer Kotlin/Android development services
Anyone: Anyone passionate about modern mobile and backend development

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE KOTLIN SKILLS (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid Kotlin fundamentals. Learn programming logic, master Kotlin syntax, OOP principles, functional programming, and create your first applications.

Months 1-2: Kotlin Fundamentals & Programming Basics (Week 1-8)

Introduction to Kotlin & Development Environment Setup

Topics:

- What is Kotlin? History, features, and JetBrains creation
- Why Kotlin? Advantages over Java and other languages
- Kotlin ecosystem: Android, Backend, Multiplatform, Native
- Kotlin/JVM, Kotlin/JS, Kotlin/Native explained
- Interoperability with Java
- Installing JDK (Java 17 LTS or latest)
- Setting up development environment: IntelliJ IDEA, Android Studio
- Understanding IDE features and Kotlin-specific shortcuts
- Kotlin REPL (Read-Eval-Print Loop) for quick testing
- Your first Kotlin program: Hello World
- Kotlin program structure: main function, packages
- Compiling and running Kotlin programs
- Understanding bytecode and JVM execution
- Comments: single-line (//), multi-line (/* */), KDoc (/** */)
- Kotlin coding conventions and naming standards
- Type inference and concise syntax

Projects built:

- Hello World variations with string templates
- Simple output programs using println
- Personal information display with string interpolation
- ASCII art generator in Kotlin
- Interactive REPL experiments

Practice: Daily: 30 min Kotlin syntax practice, write 5-10 simple programs

Variables, Data Types & Operators

Topics:

- Variables: val (immutable) vs var (mutable)
- Type inference: let the compiler infer types
- Explicit type declarations
- Basic types: Int, Long, Short, Byte, Double, Float, Char, Boolean
- String type and string templates
- String interpolation with \$ and \${}
- Multi-line strings with triple quotes
- Type checking with is operator
- Smart casts after type checking
- Type casting: as and as? (safe cast)
- Nullable types: Type? notation
- Null safety: ?, !, let
- Operators: arithmetic (+, -, *, /, %, ++, --)

- Relational operators: ==, !=, >, <, >=, <=
- Logical operators: &&, ||, !
- Assignment operators: =, +=, -=, *=, /=, %=
- Bitwise operators: and, or, xor, inv, shl, shr, ushr
- Range operator: ..
- in operator for range checking
- Elvis operator: ?:
- Safe call operator: ?.
- Not-null assertion: !!

Projects built:

- Null-safe calculator with Kotlin operators
- Temperature converter with smart casts
- Age calculator with null safety
- BMI calculator with safe operations
- Currency converter with type-safe design
- Area calculator using when expressions
- Compound interest calculator with ranges
- String manipulation utility

Practice: Solve 40 problems on variables, null safety, and operators

Control Flow & Expressions

Topics:

- If as an expression (returns value)
- If-else expression
- If-else-if ladder
- When expression: Kotlin's switch replacement
- When with values, ranges, types
- When without argument
- When as expression vs statement
- While loop
- Do-while loop
- For loop with ranges
- For loop with collections
- Iterating with indices: forEachIndexed
- Until vs downTo in ranges
- Step in ranges
- Break and continue
- Labeled break and continue
- Nested loops and loop patterns
- repeat function for simple loops

Projects built:

- Number guessing game with when expression
- Grade calculator using when
- Even/odd checker with ranges
- Prime number checker and generator
- Multiplication table with for loops
- Factorial calculator using ranges
- Fibonacci series generator
- Pattern printing (stars, pyramids, diamonds)
- FizzBuzz challenge with when
- Menu-driven calculator
- Simple ATM simulation with when

Practice: Solve 45 control flow and expression problems

Functions & Collections Basics

Topics:

- Functions: fun keyword
- Function parameters and return types
- Unit return type (like void)
- Single-expression functions
- Default parameter values
- Named arguments
- Variable number of arguments: vararg
- Extension functions concept
- Infix functions
- Local functions (nested functions)
- Higher-order functions introduction
- Lambda expressions basics
- Arrays: arrayOf, intArrayOf, etc.
- Array access and modification
- Lists: listOf (immutable), mutableListOf
- Sets: setOf, mutableSetOf
- Maps: mapOf, mutableMapOf
- Collection operations: size, isEmpty, contains

Projects built:

- Function library for mathematical operations
- Extension functions for String utilities
- Collection statistics calculator
- Array manipulation utilities
- List-based todo manager
- Map-based contact book

- Set operations demonstrator
- Higher-order function examples
- Lambda-based filtering
- Custom collection utilities

Practice: Create 30 functions, solve 35 collection problems

Month 3: Object-Oriented Programming & Functional Concepts (Week 9-13)

Classes, Objects & OOP in Kotlin

Topics:

- Classes: class keyword
- Primary constructor in class header
- Secondary constructors
- init blocks for initialization
- Properties: val and var in classes
- Getters and setters (automatic and custom)
- Backing fields: field keyword
- Late initialization: lateinit var
- Lazy initialization: lazy delegation
- Creating objects (no 'new' keyword)
- Visibility modifiers: public, private, protected, internal
- Data classes for data holding
- Copy function in data classes
- Destructuring declarations
- Object declarations (singleton)
- Companion objects (like static in Java)
- Object expressions (anonymous objects)

Projects built:

- Student data class with properties
- Bank Account class with encapsulation
- Book data class for library
- Employee class with companion object
- Rectangle/Circle classes with properties
- Car class with lateinit properties
- Singleton configuration object
- Product data class for inventory
- Person class with destructuring
- Date utility with companion functions

Practice: Create 25 different classes modeling real-world entities

Inheritance, Interfaces & Polymorphism

Topics:

- Inheritance with open keyword
- Classes are final by default
- Open functions for overriding
- Override keyword (mandatory)
- Calling superclass: super keyword
- Abstract classes and members
- Interfaces in Kotlin
- Multiple interface implementation
- Interface properties
- Default implementations in interfaces
- Resolving conflicts in multiple inheritance
- Sealed classes for restricted hierarchies
- Sealed class subtypes
- Enum classes
- Enum with properties and methods
- Polymorphism and type checking
- Smart casts in polymorphism

Projects built:

- Animal hierarchy with sealed classes
- Shape hierarchy with abstract classes
- Vehicle system with interfaces
- Employee hierarchy with polymorphism
- Payment system with sealed classes
- School management (Person -> Student, Teacher)
- E-commerce with product categories
- State machine using sealed classes
- Enum-based menu system
- Multiple interface implementation demo

Practice: Build 20 inheritance hierarchies, solve 30 polymorphism problems

Functional Programming in Kotlin

Topics:

- First-class functions
- Lambda expressions syntax
- Lambda with receivers
- Anonymous functions
- Higher-order functions
- Function types: (Int) -> String
- Returning functions from functions
- Closures and capturing variables

- Inline functions for performance
- noinline and crossinline
- Standard library functions: let, run, with, apply, also
- Collection operations: map, filter, reduce
- flatMap and flatten
- fold and reduce operations
- Sequences for lazy evaluation
- Function composition
- Currying and partial application

Projects built:

- Functional collection processor
- Lambda-based calculator
- DSL (Domain Specific Language) creation
- Scope functions showcase
- Lazy sequence operations
- Function composition utilities
- Higher-order function library
- PHASE 1 MINI CAPSTONE: Functional Library Management System
- Features: Immutable data, functional operations, sealed classes, DSL

Practice: Solve 40 functional programming problems

Assessment: Phase 1 Final Assessment - Core Kotlin fundamentals test

----- PHASE 2: ADVANCED KOTLIN & MODERN FEATURES (MONTHS 4-6, WEEKS 14-26) -----

Master advanced Kotlin features, coroutines, DSLs, delegation, generics, and professional coding patterns.

Months 4-5: Advanced Language Features & Coroutines (Week 14-22)

Generics & Type System

Topics:

- Generic types in Kotlin
- Generic functions
- Generic constraints: where clause
- Variance: in, out, invariance
- Covariance with out
- Contravariance with in
- Star projection: *
- Reified type parameters
- Type erasure and reified
- Generic class inheritance
- Type aliases for complex types
- Inline classes (value classes)
- Phantom types pattern
- Generic builders
- Type-safe builders with generics

Projects built:

- Generic repository pattern
- Type-safe builder DSL
- Generic data structures (Stack, Queue)
- Variance examples with collections
- Reified function utilities
- Type-safe API client
- Generic validation framework
- Phantom types for compile-time safety

Practice: Create 20 generic classes, solve 30 variance problems

Delegation & Property Delegates

Topics:

- Class delegation with by keyword
- Delegation pattern in Kotlin
- Property delegation
- Lazy delegate for late initialization
- Observable delegate for property observation
- Vetoable delegate
- Storing properties in a map
- Custom delegates creation
- ReadOnlyProperty and ReadWriteProperty
- provideDelegate for dynamic delegation
- Delegated properties in interfaces
- Multiple delegation
- Delegation vs inheritance trade-offs

Projects built:

- Lazy loading system
- Observable property pattern
- Preferences management with delegation
- Custom logging delegate
- Validation delegate
- Cache delegate implementation
- Thread-safe property delegate

- Map-backed properties example

Practice: Implement 15 different delegates, solve 25 delegation problems

Collections Deep Dive

Topics:

- List operations: filter, map, flatMap, fold
- Set operations: union, intersect, subtract
- Map operations: mapKeys, mapValues, filterKeys
- Grouping: groupBy, groupingBy
- Partitioning: partition
- Zipping and unzipping
- Windowed and chunked
- Aggregate operations: sumOf, maxOf, minOf
- Sorting: sortBy, sortedWith
- Distinct and distinctBy
- Take, drop, slice operations
- Associate operations
- Sequences vs Collections performance
- asSequence for lazy operations
- generateSequence for infinite sequences

Projects built:

- Advanced collection processor
- Data transformation pipeline
- Student grade analyzer
- E-commerce order processor
- Log file analyzer
- Sequence-based file reader
- Performance comparison tool
- Custom collection operations library

Practice: Solve 50 advanced collection problems

Coroutines Fundamentals

Topics:

- Asynchronous programming need
- Coroutines vs threads
- Suspend functions
- launch coroutine builder
- async and await
- runBlocking for blocking code
- Coroutine context and dispatchers
- Dispatchers.Main, IO, Default, Unconfined
- withContext for switching context
- Coroutine scope
- GlobalScope vs structured concurrency
- Job and Deferred
- Cancellation and timeouts
- Exception handling in coroutines
- supervisorScope for independent children

Projects built:

- Parallel API calls with async
- File downloader with coroutines
- Timeout handling example
- Cancellation demonstration
- Exception handling in coroutines
- Background task processor
- Coroutine-based calculator
- Multi-threaded data processor

Practice: Build 15 coroutine-based applications

Advanced Coroutines & Flow

Topics:

- Channels for communication
- Channel types: rendezvous, buffered, unlimited
- produce and consumeEach
- Flow for async streams
- Flow builders: flow, flowOf, asFlow
- Flow operators: map, filter, transform
- Terminal flow operators: collect, toList
- Flow context and dispatchers
- Flow cancellation
- Flow exceptions
- StateFlow and SharedFlow
- Cold vs hot flows
- Flow combinations: combine, zip, merge
- flatMapConcat, flatMapMerge, flatMapLatest
- Flow buffering and conflation

Projects built:

- Real-time data stream processor
- Channel-based producer-consumer
- Flow-based search implementation
- StateFlow example (state management)
- SharedFlow for events

- Flow transformation pipeline
- Reactive UI updates with Flow
- Network request with Flow

Practice: Create 12 Flow-based systems

Month 6: DSLs, Reflection & Professional Patterns (Week 23-26)

Domain-Specific Languages (DSLs)

Topics:

- What are DSLs?
- Type-safe builders
- Lambda with receiver
- Extension functions in DSLs
- Infix notation for DSLs
- Operator overloading
- invoke operator
- Creating HTML DSL
- Creating SQL-like DSL
- Creating test DSL
- DslMarker annotation
- Context receivers (Kotlin 1.6+)
- Building fluent APIs
- DSL scope control
- Real-world DSL examples

Projects built:

- HTML builder DSL
- JSON builder DSL
- SQL query DSL
- Configuration DSL
- Test specification DSL
- Route definition DSL
- UI layout DSL
- Build script DSL

Practice: Build 10 different DSLs

Reflection & Annotations

Topics:

- Kotlin reflection API
- Class references: ::class
- KClass, KFunction, KProperty
- Callable references
- Function references: ::functionName
- Property references
- Bound and unbound references
- Reflection for generics
- Annotations in Kotlin
- Creating custom annotations
- Annotation targets
- Annotation retention
- Annotation parameters
- Reading annotations at runtime
- Annotation processing
- Meta-annotations

Projects built:

- Dependency injection framework
- JSON serializer using reflection
- Validation framework with annotations
- ORM mapper with reflection
- Custom test framework
- Route mapping with annotations
- Configuration reader
- Plugin system with reflection

Practice: Build 8 reflection-based utilities

Design Patterns in Kotlin

Topics:

- Singleton with object declaration
- Factory pattern with companion object
- Builder pattern with DSL
- Observer pattern with delegation
- Strategy pattern with lambdas
- Decorator pattern with delegation
- Adapter pattern
- Proxy pattern with lazy
- Command pattern
- State pattern with sealed classes
- Chain of responsibility
- Template method
- Visitor pattern
- Repository pattern
- Dependency injection patterns

Projects built:

- Notification system (Observer)
- Payment strategies implementation
- Logging decorator
- State machine with sealed classes
- Command processor
- Repository pattern implementation
- DI container creation
- Plugin architecture

Practice: Implement 20 design patterns in Kotlin style

Testing in Kotlin

Topics:

- Unit testing with JUnit 5
- Kotlin-specific test features
- Assertions with Kotlin
- Kotest framework
- Property-based testing
- Mocking with MockK
- Coroutine testing
- Testing suspending functions
- TestCoroutineDispatcher
- Testing flows
- Turbine for Flow testing
- Test-driven development in Kotlin
- Behavior-driven development
- Integration testing
- Test coverage with JaCoCo

Projects built:

- Comprehensive test suite
- Coroutine tests
- Flow testing examples
- Property-based tests
- BDD-style tests with Kotest
- Mock-based testing
- TDD practice projects
- Integration test suite

Practice: Write tests for all previous projects, 85%+ coverage

Phase 2 Capstone Project

Topics:

- Advanced Kotlin features usage
- Coroutines and Flow
- DSL creation
- Design patterns
- Comprehensive testing
- Functional programming

Projects built:

- PHASE 2 CAPSTONE: Reactive Task Management System
- Features: Coroutines, Flow, StateFlow, DSL for task creation, sealed classes for states, comprehensive testing
- Alternative: Real-time Chat Server (Kotlin backend)
- Alternative: Data Processing Pipeline with Flow
- Alternative: Custom framework with DSL and reflection

Assessment: Phase 2 Final Exam - Advanced Kotlin features comprehensive test

PHASE 2 CONTINUED - Kotlin for Backend (Week 14-26 (distributed))

Ktor Framework - Part 1

Topics:

- Introduction to Ktor
- Ktor vs Spring Boot
- Setting up Ktor project
- Routing in Ktor
- HTTP methods handling
- Request and response
- Content negotiation
- JSON serialization with kotlinx.serialization
- Status pages and error handling
- Authentication and authorization
- JWT authentication in Ktor
- Sessions in Ktor
- Static content serving

Projects built:

- Hello World REST API with Ktor
- CRUD API for products
- User authentication API with JWT
- File upload service
- Static website hosting
- API with JSON responses
- Session-based auth

Practice: Build 8 Ktor-based APIs

Ktor Framework - Part 2

Topics:

- Database integration with Exposed
- Exposed DSL and DAO
- Database transactions
- Connection pooling
- WebSockets in Ktor
- Real-time communication
- Testing Ktor applications
- Client-side Ktor HTTP client
- Dependency injection in Ktor (Koin)
- Monitoring and metrics
- Ktor deployment
- Docker containerization

Projects built:

- Full-stack CRUD with database
- Real-time chat with WebSockets
- API testing suite
- HTTP client for third-party APIs
- Deployed Ktor application
- Monitored production service

Practice: Build 6 complete Ktor applications

Spring Boot with Kotlin

Topics:

- Spring Boot Kotlin setup
- Kotlin-specific Spring features
- Null-safe Spring configuration
- Data classes as entities
- Spring Data JPA with Kotlin
- Repository interfaces in Kotlin
- REST controllers in Kotlin
- Coroutines support in Spring
- Reactive Spring with Kotlin Flow
- Spring Security with Kotlin
- Testing Spring Boot with Kotlin
- MockK for Spring tests

Projects built:

- Spring Boot REST API
- JPA entities with data classes
- Reactive API with Flow
- Secured Spring Boot app
- Coroutine-based Spring service
- Complete CRUD application
- Tested Spring Boot project

Practice: Build 10 Spring Boot Kotlin applications

Database Access with Kotlin

Topics:

- JDBC with Kotlin
- Exposed framework deep dive
- Exposed DSL queries
- Exposed DAO pattern
- Relationships in Exposed
- Transactions in Exposed
- jOOQ with Kotlin
- SQLDelight for type-safe SQL
- MongoDB with Kotlin
- Redis with Kotlin
- Database migrations
- Connection pool configuration

Projects built:

- Exposed-based ORM project
- Type-safe database queries
- Multi-table operations
- NoSQL integration
- Migration scripts
- Connection pool setup
- Database performance optimization

Practice: Build 8 database-driven applications

Build Tools & Kotlin Scripting

Topics:

- Gradle with Kotlin DSL
- Gradle Kotlin DSL syntax
- Build script configuration
- Dependencies management
- Multi-module projects
- Publishing libraries
- Kotlin scripting (.kts files)
- Custom Gradle tasks
- Maven with Kotlin
- Continuous integration setup

- Git best practices for Kotlin

Projects built:

- Gradle Kotlin DSL project
- Multi-module application
- Published library to Maven Central
- Custom Gradle plugin
- Kotlin scripts for automation
- CI/CD pipeline setup

Practice: Configure all projects with Gradle Kotlin DSL

----- PHASE 3: ANDROID DEVELOPMENT MASTERY (MONTHS 7-9, WEEKS 27-39) -----

Master modern Android development with Kotlin, Jetpack Compose, Material Design, Architecture Components, and professional app development.

Months 7-8: Android Fundamentals & Jetpack Compose (Week 27-35)

Android Basics & Setup

Topics:

- Android platform overview
- Android architecture
- Android SDK and tools
- Android Studio setup and features
- Project structure in Android
- Gradle build system for Android
- AndroidManifest.xml
- Activities and lifecycle
- Intents: explicit and implicit
- Activity navigation
- Passing data between activities
- Fragment basics
- Fragment lifecycle
- Fragment transactions
- UI with XML layouts
- View and ViewGroup
- Common layouts: LinearLayout, RelativeLayout, ConstraintLayout

Projects built:

- Hello Android app
- Multi-activity application
- Intent-based navigation
- Fragment-based UI
- Data passing between screens
- Basic calculator app
- Profile display app
- Navigation drawer app

Practice: Build 10 basic Android apps

Jetpack Compose Fundamentals

Topics:

- Introduction to Jetpack Compose
- Declarative UI paradigm
- Composable functions
- @Composable annotation
- Compose preview
- State in Compose: remember, mutableStateOf
- Recomposition concept
- State hoisting
- Modifiers in Compose
- Layout composables: Column, Row, Box
- LazyColumn and LazyRow
- Text, Image, Button composables
- TextField and user input
- Spacer and Divider
- Card and Surface
- Material Design 3 components
- Theming in Compose

Projects built:

- Hello Compose app
- Counter app with state
- Todo list with LazyColumn
- Form with TextFields
- Material Design 3 app
- Custom theme implementation
- Card-based UI
- Image gallery with LazyGrid

Practice: Build 12 Compose-based UIs

Advanced Compose & Navigation

Topics:

- Compose Navigation library
- NavHost and NavController

- Route-based navigation
- Passing arguments in navigation
- Bottom navigation with Compose
- Navigation drawer in Compose
- Nested navigation graphs
- Side effects in Compose: LaunchedEffect, DisposableEffect
- SideEffect, derivedStateOf
- remember vs rememberSaveable
- ViewModel integration with Compose
- Lifecycle-aware composables
- Animation in Compose
- AnimatedVisibility, animateContentSize
- Custom animations

Projects built:

- Multi-screen navigation app
- Bottom navigation app
- Navigation with arguments
- Animated UI transitions
- ViewModel-backed Compose app
- Side effect demonstrations
- Complex navigation flow
- Login flow with navigation

Practice: Build 10 navigation-based apps

Android Architecture Components

Topics:

- MVVM architecture pattern
- ViewModel and LiveData
- StateFlow and SharedFlow in Android
- Repository pattern
- Data layer architecture
- Use cases / Interactors
- Dependency injection with Hilt
- Hilt setup and annotations
- @HiltAndroidApp, @HiltViewModel
- Providing dependencies
- Scopes in Hilt
- Testing with Hilt
- Clean Architecture in Android
- Separation of concerns

Projects built:

- MVVM app with ViewModel
- Repository pattern implementation
- Hilt dependency injection setup
- Clean architecture project
- Multi-layer application
- Testable architecture demo
- State management with StateFlow
- Use case layer implementation

Practice: Refactor 8 apps to clean architecture

Local Data Storage

Topics:

- SharedPreferences for simple data
- DataStore (Preferences and Proto)
- Room database introduction
- Entity, DAO, Database classes
- Room with Kotlin Coroutines
- Room with Flow
- Database migrations
- Relationships in Room
- Type converters
- Database testing
- File storage basics
- Internal vs external storage

Projects built:

- Preferences-based settings screen
- DataStore implementation
- Room database CRUD app
- Notes app with Room
- Contact manager with Room
- Offline-first architecture
- Database migration example
- Complex relationships in Room

Practice: Build 10 data persistence apps

Month 9: Networking, APIs & Advanced Android (Week 36-39)

Networking & REST APIs

Topics:

- HTTP basics for Android
- Retrofit library for REST APIs

- Setting up Retrofit
- Defining API interfaces
- GET, POST, PUT, DELETE requests
- Query parameters and path variables
- Request and response models
- JSON parsing with Gson/Moshi
- kotlinx.serialization in Android
- OkHttp interceptors
- Authentication headers
- Error handling in network calls
- Coroutines with Retrofit
- Loading states management
- Offline caching strategies

Projects built:

- Weather app with API
- News reader app
- Movie database app (TMDB API)
- REST API consumer
- Authenticated API calls
- Offline-first with Room cache
- Error handling examples
- Loading state UI

Practice: Build 10 API-consuming apps

Image Loading & Media

Topics:

- Coil library for image loading
- Glide library
- Image caching strategies
- Placeholder and error images
- Image transformations
- AsyncImage in Compose
- Loading images from network
- Camera integration
- Gallery access
- Image picker
- Video playback: ExoPlayer
- Media permissions
- MediaStore API
- Uploading images to server

Projects built:

- Image gallery app with Coil
- Photo viewer with zoom
- Camera app
- Image picker implementation
- Video player app
- Image upload to server
- Media browser
- Photo editor basics

Practice: Build 8 media-handling apps

Advanced UI & Material Design

Topics:

- Material Design 3 principles
- Dynamic theming
- Color schemes in Material 3
- Custom themes and colors
- Typography in Compose
- Scaffold and top app bar
- Bottom sheet in Compose
- Dialogs and alerts
- Snackbars and toasts
- Tabs with TabRow
- Chips and badges
- Progress indicators
- Swipe to refresh
- Pull to refresh
- Custom composables creation

Projects built:

- Material Design 3 showcase app
- Custom themed app
- Bottom sheet examples
- Dialog-based workflows
- Tab-based navigation
- Complex custom UI components
- Accessible UI implementation
- Responsive layouts

Practice: Design 10 Material Design apps

Background Work & Services

Topics:

- WorkManager for background tasks

- One-time and periodic work
- Constraints for WorkManager
- Chaining work requests
- Monitoring work status
- Foreground services
- Notifications in Android
- NotificationCompat builder
- Notification channels
- Push notifications with Firebase
- FCM integration
- Handling notification clicks
- AlarmManager for scheduling
- JobScheduler basics

Projects built:

- Background sync with WorkManager
- Periodic data refresh
- Notification system
- Push notification handler
- Foreground service example
- Reminder app with AlarmManager
- Download manager
- Background upload service

Practice: Build 8 background processing apps

Testing Android Apps

Topics:

- Unit testing in Android
- JUnit for Android
- Testing ViewModels
- Testing coroutines in Android
- Mocking with MockK
- Instrumented tests
- Espresso for UI testing
- Compose UI testing
- Testing navigation
- Testing Room database
- Testing network calls
- Test doubles and fakes
- Screenshot testing
- Code coverage in Android

Projects built:

- Comprehensive unit tests
- ViewModel test suite
- UI tests with Espresso
- Compose UI tests
- Integration tests
- E2E testing
- TDD Android app
- Tested production app

Practice: Write tests for all Android projects, 80%+ coverage

PHASE 3 COMPLETION - Month 9 Advanced Topics (Week 27-39 (distributed))

Performance & Optimization

Topics:

- Android performance profiling
- CPU profiler in Android Studio
- Memory profiler
- Network profiler
- Layout Inspector
- Detecting memory leaks
- LeakCanary integration
- App startup optimization
- Lazy loading strategies
- Image optimization
- Database query optimization
- Compose performance
- Recomposition optimization
- Battery optimization
- App size reduction

Projects built:

- Profiled and optimized app
- Memory leak fixes
- Performance benchmark
- Optimized startup time
- Lazy-loaded content
- APK size optimization
- Battery-efficient app

Practice: Optimize all previous Android apps

Publishing & Deployment

Topics:

- Preparing app for release
- App signing with keystore
- ProGuard and R8
- Code obfuscation
- Building release APK/AAB
- Google Play Console setup
- Creating store listing
- Screenshots and graphics
- App versioning strategy
- Release management
- Beta testing with Play Console
- Internal testing tracks
- App updates and migrations
- Analytics integration: Firebase Analytics
- Crash reporting: Firebase Crashlytics

Projects built:

- Signed release build
- Play Store ready app
- Complete store listing
- Beta test distribution
- Analytics-enabled app
- Crash reporting setup
- Update mechanism

Practice: Prepare 3 apps for Play Store

Advanced Features & Libraries

Topics:

- Location services and maps
- Google Maps integration
- Geolocation and GPS
- Firebase Firestore
- Real-time database
- Firebase Authentication
- Social login (Google, Facebook)
- Firebase Storage for files
- Paging 3 library for pagination
- In-app purchases
- AdMob for monetization
- WebView integration
- Deep linking and app links
- Widget development
- Wear OS basics

Projects built:

- Maps-based location app
- Firebase-backed chat app
- Social login integration
- Paginated list with Paging 3
- App with in-app purchases
- Ad-supported app
- Deep linking demo
- Home screen widget

Practice: Build 10 advanced feature apps

Security Best Practices

Topics:

- Android security overview
- Secure data storage
- Encrypted SharedPreferences
- Keystore system
- Biometric authentication
- Certificate pinning
- ProGuard for security
- SQL injection prevention
- Secure network communication
- HTTPS enforcement
- API key protection
- Code obfuscation
- Root detection
- SafetyNet API
- OWASP mobile security

Projects built:

- Secured data storage app
- Biometric auth implementation
- Certificate pinned API calls
- Encrypted database
- Secure authentication flow
- Obfuscated production app

Practice: Security audit all Android apps

Phase 3 Capstone Project

Topics:

- Complete Android app architecture

- Jetpack Compose UI
- MVVM with Clean Architecture
- Room database
- Retrofit networking
- Hilt dependency injection
- Comprehensive testing
- Play Store deployment

Projects built:

- MAJOR CAPSTONE: Social Media Android App
- Features: User profiles, posts, comments, likes, real-time feed, image upload, push notifications, Material Design 3, offline support, Clean Architecture
- Alternative: E-commerce Shopping App (products, cart, checkout, orders)
- Alternative: Fitness Tracking App (workouts, progress, charts, reminders)
- Alternative: Expense Manager (budgets, categories, reports, charts)

Assessment: Phase 3 Final Exam - Android development comprehensive test

----- PHASE 4: KOTLIN MULTIPLATFORM, CLOUD & CAREER EXCELLENCE (MONTHS 10-12, WEEKS 40-52) -----

Master Kotlin Multiplatform, advanced backend, cloud deployment, system design, and professional career preparation.

Months 10-11: Kotlin Multiplatform & Advanced Topics (Week 40-48)

Kotlin Multiplatform Mobile (KMM)

Topics:

- Introduction to Kotlin Multiplatform
- KMM architecture
- Shared code vs platform-specific code
- Expect/actual mechanism
- Setting up KMM project
- Shared business logic
- Shared networking code
- Shared data models
- Platform-specific implementations
- iOS integration with Swift
- Android integration
- Ktor client in KMM
- SQLDelight for shared database
- Dependency injection in KMM: Koin
- Testing shared code

Projects built:

- KMM project setup
- Shared API client
- Shared database layer
- Business logic sharing
- Cross-platform models
- Platform-specific UI
- Complete KMM app (Android + iOS)
- Testing shared modules

Practice: Build 5 KMM applications

Kotlin Multiplatform for All Platforms

Topics:

- Kotlin/JS for web frontend
- Kotlin/JS with React
- Kotlin/Native introduction
- Kotlin for desktop: Compose Desktop
- Compose Multiplatform
- Sharing UI code with Compose
- Multiplatform libraries
- kotlinx libraries ecosystem
- Serialization across platforms
- Date-time handling multiplatform
- Coroutines on all platforms
- Publishing multiplatform libraries
- Gradle setup for multiplatform

Projects built:

- Kotlin/JS web app
- Compose Desktop application
- Compose Multiplatform (Android, Desktop, iOS)
- Multiplatform library creation
- Shared UI components
- Cross-platform game
- Published multiplatform library

Practice: Build 6 multiplatform projects

Advanced Backend with Kotlin

Topics:

- Microservices with Kotlin
- Service discovery
- API Gateway pattern

- Circuit breaker in Kotlin
- Message queues: RabbitMQ with Kotlin
- Apache Kafka with Kotlin
- Event-driven architecture
- CQRS pattern in Kotlin
- GraphQL server with Kotlin
- gRPC with Kotlin
- WebSocket servers
- Real-time applications
- Reactive programming with Kotlin
- Project Reactor with Kotlin
- Database sharding strategies

Projects built:

- Microservices architecture
- Event-driven system
- GraphQL API server
- gRPC service
- Real-time chat backend
- Reactive backend service
- Message queue integration
- CQRS implementation

Practice: Build 8 advanced backend systems

Cloud & Deployment

Topics:

- Containerization with Docker
- Docker for Kotlin apps
- Multi-stage Docker builds
- Kubernetes basics
- Deploying Kotlin to K8s
- Helm charts for Kotlin
- AWS deployment strategies
- AWS Lambda with Kotlin
- AWS Elastic Beanstalk
- Google Cloud Platform
- GCP Cloud Run for Kotlin
- Azure deployment
- CI/CD pipelines
- GitHub Actions for Kotlin
- Monitoring and logging

Projects built:

- Dockerized Kotlin app
- Kubernetes deployment
- Serverless function (Kotlin)
- AWS deployment
- GCP Cloud Run deployment
- CI/CD pipeline setup
- Monitored production service
- Auto-scaling setup

Practice: Deploy all apps to cloud platforms

Observability & DevOps

Topics:

- Application monitoring
- Metrics with Micrometer
- Prometheus integration
- Grafana dashboards
- Distributed tracing
- OpenTelemetry with Kotlin
- Logging best practices
- Structured logging
- Log aggregation: ELK stack
- Error tracking: Sentry
- Performance monitoring
- APM tools integration
- Infrastructure as Code
- Terraform basics
- GitOps practices

Projects built:

- Monitoring setup
- Grafana dashboard creation
- Distributed tracing implementation
- Centralized logging
- Error tracking integration
- Performance monitoring
- IaC with Terraform

Practice: Add observability to all services

Month 12: System Design & Career Excellence (Week 49-52)

System Design Fundamentals

Topics:

- System design principles
- Scalability: horizontal vs vertical
- Load balancing
- Caching strategies
- CDN usage
- Database scaling
- Sharding and partitioning
- CAP theorem
- Consistency models
- Microservices design
- API design best practices
- Rate limiting
- High availability design
- Disaster recovery
- System design interview prep

Projects built:

- Design URL shortener
- Design messaging system
- Design social media feed
- Design e-commerce platform
- Design video streaming service
- Design notification system
- System design documentation

Practice: Solve 20 system design problems

Performance & Optimization

Topics:

- Performance profiling
- JVM tuning for Kotlin
- Memory management
- Garbage collection optimization
- Coroutine performance
- Flow optimization
- Database query optimization
- Caching strategies
- Load testing with Gatling
- Benchmarking with JMH
- Code optimization techniques
- Algorithmic optimization
- Resource utilization
- Monitoring performance metrics

Projects built:

- Performance profiling suite
- Load testing framework
- Optimized backend service
- JVM tuning example
- Benchmark suite
- Cache optimization
- Performance dashboard

Practice: Optimize all backend services

Security & Best Practices

Topics:

- Security best practices
- OWASP Top 10
- Secure coding in Kotlin
- Input validation
- SQL injection prevention
- XSS prevention
- CSRF protection
- Authentication best practices
- OAuth 2.0 and OIDC
- JWT security
- API security
- Secrets management
- Encryption in Kotlin
- Security testing
- Compliance: GDPR, etc.

Projects built:

- Secured API implementation
- OAuth integration
- Encrypted data storage
- Security audit tool
- Penetration testing
- Security-hardened app
- Compliance checklist

Practice: Security audit all projects

Open Source & Community

Topics:

- Contributing to Kotlin projects
- Open source best practices

- Finding projects to contribute
- Making pull requests
- Code review participation
- Creating Kotlin libraries
- Publishing to Maven Central
- Documentation writing
- Building personal brand
- Technical blogging
- Speaking at conferences
- Community engagement
- Mentoring others
- Building portfolio

Projects built:

- Contribute to 5 Kotlin projects
- Create own Kotlin library
- Published open source project
- Technical blog posts
- Conference talk preparation
- Portfolio website

Practice: Active community participation

Advanced Kotlin Topics

Topics:

- Kotlin compiler plugins
- KSP (Kotlin Symbol Processing)
- Annotation processing
- Code generation
- Custom Gradle plugins
- Kotlin scripting advanced
- Meta-programming
- Kotlin compiler internals
- Advanced type system
- Phantom types
- Higher-kinded types workarounds
- Arrow library for FP
- Category theory basics
- Monads in Kotlin

Projects built:

- Custom compiler plugin
- KSP processor
- Code generation tool
- Gradle plugin creation
- Advanced scripting
- FP library with Arrow
- Type-safe DSL with advanced types

Practice: Explore 5 advanced topics

PHASE 4 COMPLETION - Career Launch (Week 40-52 (distributed))

Interview Preparation

Topics:

- Kotlin interview preparation
- Core Kotlin questions
- OOP in Kotlin questions
- Coroutines interview topics
- Android interview prep
- Jetpack Compose questions
- Backend Kotlin questions
- System design interviews
- Data structures in Kotlin
- Algorithms in Kotlin
- LeetCode with Kotlin
- Behavioral interviews
- STAR method
- Salary negotiation
- Job search strategies

Projects built:

- Solve 200+ LeetCode (Kotlin)
- System design case studies
- Mock interview practice
- Interview prep guide
- Behavioral question bank

Practice: Daily coding challenges and mock interviews

Professional Development

Topics:

- Resume optimization
- LinkedIn profile enhancement
- GitHub portfolio
- Personal website creation
- Cover letter writing
- Networking strategies

- Job application process
- Technical interviews
- Coding challenges
- Take-home assignments
- Interview follow-up
- Offer negotiation
- Career path planning
- Continuous learning plan

Projects built:

- Professional resume
- Optimized LinkedIn
- GitHub showcase
- Portfolio website
- Interview prep materials
- Learning roadmap

Practice: Apply to 20+ positions

Specialization & Niches

Topics:

- Android specialization path
- Backend Kotlin specialization
- Kotlin Multiplatform expert
- Kotlin for data science
- Kotlin for game development
- Kotlin for embedded systems
- Consulting career path
- Freelancing with Kotlin
- Startup opportunities
- Product development
- Technical leadership
- Architecture roles
- Teaching and mentoring
- Content creation

Projects built:

- Specialization portfolio
- Niche expertise demo
- Freelance service offering
- Product MVP
- Technical content
- Mentorship program

Practice: Choose and pursue specialization

Future of Kotlin

Topics:

- Kotlin roadmap
- Upcoming Kotlin features
- Kotlin 2.0 and beyond
- Industry trends
- Emerging technologies
- Kotlin in AI/ML
- Kotlin for IoT
- Kotlin for blockchain
- Cross-platform future
- WebAssembly with Kotlin
- Staying updated
- Continuous learning resources
- Community involvement
- Conference participation

Projects built:

- Experimental features demo
- Emerging tech integration
- Innovation project
- Future-ready portfolio

Practice: Stay current with ecosystem

Final Month - Ultimate Capstone & Launch (Week 49-52)

Final Capstone Project - Part 1

Topics:

- Project planning
- Requirements analysis
- Architecture design
- Technology selection
- Database design
- API design
- UI/UX planning
- Development workflow
- Agile methodology
- Sprint planning

Projects built:

- FINAL CAPSTONE: Cross-Platform Application
- Option 1: E-commerce Platform (Android + Backend + Admin Web)

- KMM shared logic, Jetpack Compose UI, Ktor backend, Admin panel
- Option 2: Social Media Platform (Android + iOS + Backend)
- Full-stack with KMM, real-time features, cloud deployment
- Option 3: Productivity Suite (Android + Desktop + Web)
- Compose Multiplatform, offline-first, sync across devices
- Option 4: Fitness & Health Platform (Mobile + Wearables + Backend)
- Tracking, analytics, social features, integrations

Final Capstone Project - Part 2

Topics:

- Implementation completion
- Testing all layers
- Performance optimization
- Security hardening
- Documentation
- Cloud deployment
- CI/CD setup
- Monitoring
- Beta testing
- Final polish

Deliverables:

- Complete source code on GitHub
- Android app on Play Store
- Backend deployed to cloud
- API documentation
- Architecture documentation
- Test coverage 85%+
- CI/CD pipeline
- Monitoring dashboard
- User documentation
- Video demo
- Presentation deck

Career Launch & Beyond

Topics:

- Portfolio finalization
- Resume perfection
- LinkedIn optimization
- GitHub showcase
- Job applications
- Interview scheduling
- Networking events
- Conference attendance
- Blog launch
- Open source maintenance
- Continuous learning
- Community leadership
- Mentorship
- Career goals

Deliverables:

- Professional portfolio with 50+ projects
- Polished resume for Kotlin roles
- LinkedIn with certifications
- GitHub with 100+ contributions
- 10+ technical blog posts
- Active community presence
- Job offers in hand
- Continuous learning plan
- Mentorship connections
- Career roadmap

Assessment: FINAL COMPREHENSIVE CERTIFICATION - Complete Kotlin mastery evaluation

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-3): 25+ foundational projects - console apps, functional programs, OOP systems
- Phase 2 (Months 4-6): 20+ advanced projects - coroutines, DSLs, backend APIs, design patterns
- Phase 3 (Months 7-9): 25+ Android projects - Compose UIs, full apps, production-ready applications
- Phase 4 (Months 10-12): 15+ professional projects - multiplatform, cloud, distributed systems
- Final: 5 major capstone projects demonstrating complete expertise

Total projects: 80+ projects from beginner to professional cross-platform systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Kotlin: Syntax, null safety, functional programming, OOP, collections, coroutines, Flow
- Advanced Kotlin: Generics, delegation, DSLs, reflection, annotations, meta-programming
- Android Development: Jetpack Compose, Material Design 3, Architecture Components, Room, Retrofit
- Android Advanced: Navigation, Hilt, WorkManager, notifications, testing, performance

- Backend Kotlin: Ktor, Spring Boot, Exposed, REST APIs, WebSockets, microservices
- Kotlin Multiplatform: KMM, shared code, expect/actual, Compose Multiplatform
- Databases: Room, Exposed, SQLDelight, MongoDB, Redis, query optimization
- Networking: Retrofit, Ktor client, OkHttp, REST, GraphQL, gRPC
- Testing: JUnit, Kotest, MockK, Espresso, Compose testing, property-based testing
- DevOps: Docker, Kubernetes, CI/CD, GitHub Actions, monitoring, logging
- Cloud: AWS, GCP, Azure, serverless, containers, orchestration
- Tools: IntelliJ IDEA, Android Studio, Gradle, Git, Postman, profilers

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 8-10 hours
 Projects: 4-6 hours
 Practice problems: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding and doubt clearing sessions
 Mentorship: 1-on-1 expert Kotlin developer mentorship
 Community: Active Discord community with 24/7 peer support
 Code review: Expert code reviews for all major projects
 Career support: Resume review, mock interviews, job referrals
 Lifetime access: All content, future updates, new Kotlin versions
 Interview prep: Mock interviews with industry professionals
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 certificates)
 Final certificate: Professional Kotlin Developer Certification
 Android certificate: Android Developer with Kotlin Certificate
 Multiplatform certificate: Kotlin Multiplatform Specialist Certificate
 LinkedIn badge: Digital badges for LinkedIn profile
 Industry recognized: Recognized by top tech companies
 Portfolio projects: 50+ production-ready portfolio projects
 JetBrains prep: Preparation for JetBrains Kotlin certification

WHERE THIS CAN LEAD

- Android Developer (Junior to Senior)
- Kotlin Developer
- Mobile Application Developer
- Full Stack Kotlin Developer
- Backend Developer (Kotlin)
- Kotlin Multiplatform Developer
- Mobile Architect
- Technical Lead (Android/Kotlin)
- Solutions Architect
- DevOps Engineer (Kotlin focus)
- Cloud Engineer
- Principal Engineer
- Freelance Kotlin Consultant
- Technical Trainer
- Startup Founder / CTO

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-6 LPA (Junior Kotlin/Android Developer)
 Growing roles: Rs 6-10 LPA (Android Developer); Rs 10-18 LPA (Senior Android Developer)
 Experienced roles: Rs 15-30 LPA (Senior Kotlin Developer / Tech Lead); Rs 20-45 LPA (Lead Developer / Architect)
 Freelance: Rs 1,500-6,000/hour based on expertise
 International usa: \$90k-200k USD (Android/Kotlin Developer to Senior)
 International europe: €55k-130k EUR based on country

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need Java or any programming background first?

A: No, the course teaches Kotlin from zero, and unlike older Android learning paths you do not need Java first. Existing programmers will clear the fundamentals quickly and get more time in coroutines, Jetpack Compose and app architecture.

Q: Which Kotlin version and tools are taught?

A: Current Kotlin 2.x with the modern compiler, Android Studio, Jetpack Compose for UI, coroutines for asynchronous work, and Spring Boot for the backend phase. The stack is kept aligned with what production Android and backend teams currently use.

Q: What will I build during the 12 months?

A: Kotlin fundamentals projects and console programs first, then real Android apps with Jetpack Compose, Material Design and API integration, followed by Spring Boot backend services and a Kotlin Multiplatform project. You finish with polished Android apps and backend work in a GitHub portfolio.

Q: How does this course position me for Android or backend roles?

A: Kotlin is Google's preferred language for Android, and the course adds Spring Boot backend and Kotlin Multiplatform, so you can interview for more than one kind of role. You leave with real apps in a GitHub portfolio, code review experience and interview preparation. We do not promise placements; we prepare you to earn them.

Q: Why not learn Kotlin from free videos?

A: Free videos cannot review your architecture or tell you why your coroutine leaks. In live classes a mentor reads your actual code, enforces habits like proper state handling in Compose, and keeps a 12-month arc moving from fundamentals to complete apps. Self-paced Android learning most often dies at the first big project; mentorship is how you get past it.

Q: How much are the classes?

A: International students pay a price set for their own country, shown on this page in their local currency.

Q: Is the demo class really free?

A: Completely: book at learn.modernagecoders.com/contact, attend a live session, and decide afterward with no commitment and nothing to cancel. Students and working professionals use it to check class quality; our Google rating stands at 4.9 stars across 547+ reviews.

Q: What does a typical class look like?

A: A live 1-hour online session with your instructor, with you writing Kotlin during class rather than watching passively; nothing is pre-recorded. Evening and weekend slots are available, designed around college timetables and office hours.

Q: Is there a certificate and portfolio?

A: Both: a completion certificate when you finish, and a GitHub-ready portfolio of Android apps and backend services built through the year. Recruiters for Android roles almost always ask to see apps, and this course makes sure you have them.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-kotlin-programming-masterclass-professional>

=====

COURSE 83 OF 118: MATHS OLYMPIAD & COMPETITION COURSE: AMC, AIME TO IMO

=====

Course link: <https://learn.modernagecoders.com/courses/olympiad-competition-mathematics-mastery>

Subject area: Competition Mathematics & Olympiad Training

Designed for: Elementary school, Girls only batches, High school, Middle school

Who can join: Intermediate to International Competition Level

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Olympiad Mathematics Excellence Certificate

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

One-year maths olympiad course: master number theory, combinatorics, algebra, geometry and inequalities, with problem-solving craft for AMC, AIME and IMO.

In one line: From Regional Contests to International Mathematical Olympiad

ABOUT THE PROGRAMME

This intensive program transforms talented mathematics students into competition champions. Designed for students aiming to excel in mathematical olympiads and competitions at all levels - from AMC to IMO. You'll master advanced problem-solving techniques, develop mathematical intuition, and learn to tackle the most challenging competition problems. Through systematic training in number theory, combinatorics, algebra, geometry, and more, you'll build the skills needed to compete at the highest levels. By completion, you'll have

solved thousands of competition problems and be ready for any mathematical challenge.

WHAT MAKES THIS COURSE DIFFERENT

- Comprehensive coverage of all olympiad topics
- Progressive difficulty from AMC 8 to IMO level
- 1000+ carefully selected competition problems
- Strategies from past olympiad medalists
- Mock competitions and timed practice
- Proof writing and rigorous mathematics
- International competition exposure
- Personal coaching and problem discussions

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Competition Basics, Number Theory, Elementary Combinatorics

Phase 2: Intermediate (Months 4-6): Advanced Algebra, Polynomial Theory, Sequences and Series

Phase 3: Advanced (Months 7-9): Competition Geometry, Trigonometry, Coordinate Methods

Phase 4: Master Level (Months 10-12): Inequalities, Functional Equations, IMO Preparation

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Strong foundation in school mathematics

Coding experience: Basic algebra and geometry knowledge required

Equipment: Computer with internet, graphing calculator, geometry tools

Time commitment: 15-20 hours per week minimum

English: Advanced mathematical reading ability

Motivation: Passionate about mathematical problem solving

WHO THIS COURSE IS FOR

Students: High school students aiming for olympiads

Working professionals: Mathematics teachers and coaches

Entrepreneurs: Competition preparation centers

Freelancers: Private olympiad tutors

Kids: Gifted middle school students

Anyone: Anyone passionate about competition mathematics

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: COMPETITION FUNDAMENTALS & NUMBER THEORY (MONTHS 1-3, WEEKS 1-13) -----

Build strong foundations in competition problem-solving, master number theory, and develop combinatorial thinking.

Months 1-2: Competition Mathematics Basics & Number Theory (Week 1-8)

Introduction to Competition Mathematics

Topics:

- Competition mathematics vs school mathematics
- Major competitions overview: AMC, AIME, USAMO, IMO
- Competition formats and scoring systems
- Problem-solving strategies: working backwards
- Pattern recognition and generalization
- Extreme cases and boundary testing
- Proof by contradiction
- Proof by induction basics
- Direct proof techniques
- Constructive vs non-constructive proofs
- Mathematical notation for competitions
- Time management in competitions

Projects built:

- Solve 50 AMC 8 problems
- Create problem-solving strategy guide
- Analyze past competition papers

Practice: Daily: 5 competition problems with increasing difficulty

Divisibility and Prime Numbers

Topics:

- Advanced divisibility rules and proofs
- GCD and LCM properties and algorithms
- Euclidean algorithm and extended version
- Bezout's identity and applications
- Prime factorization uniqueness theorem
- Distribution of primes
- Sieve of Eratosthenes variations
- Prime number theorem introduction
- Fermat's Little Theorem

- Wilson's Theorem
- Prime testing algorithms
- Applications in cryptography basics

Projects built:

- Implement prime generation algorithms
- Solve 30 number theory competition problems
- Create divisibility proof portfolio

Practice: Master 40 divisibility and prime problems from competitions

Modular Arithmetic

Topics:

- Modular arithmetic operations and properties
- Congruence classes and equivalence
- Linear congruences and solutions
- Chinese Remainder Theorem
- Modular multiplicative inverse
- Euler's totient function $\phi(n)$
- Euler's theorem and generalizations
- Quadratic residues introduction
- Legendre symbol
- Power patterns in modular arithmetic
- Solving Diophantine equations
- Applications to competition problems

Projects built:

- Modular arithmetic problem set
- CRT application problems
- Create modular arithmetic reference guide

Practice: Solve 50 modular arithmetic competition problems

Advanced Number Theory

Topics:

- Perfect numbers and Mersenne primes
- Arithmetic functions: $\sigma(n)$, $\tau(n)$, $\mu(n)$
- Multiplicative functions
- Mobius inversion formula
- Continued fractions basics
- Pell's equation
- Quadratic reciprocity introduction
- Sum of squares theorems
- Lifting the Exponent Lemma (LTE)
- Zsigmondy's theorem
- Carmichael's lambda function
- Advanced competition techniques

Projects built:

- Advanced number theory problem collection
- Historical theorems exploration
- IMO number theory problems analysis

Practice: 30 advanced number theory problems from national olympiads

Month 3: Combinatorics Fundamentals (Week 9-13)

Counting Principles and Techniques

Topics:

- Advanced counting principles
- Multiplication and addition principles
- Permutations with restrictions
- Circular permutations and necklaces
- Combinations with repetition
- Stars and bars method
- Inclusion-Exclusion Principle
- Derangements and subfactorial
- Stirling numbers of both kinds
- Partition function introduction
- Generating functions basics
- Recurrence relations

Projects built:

- Combinatorial identity proofs
- Counting problems compilation
- Create combinatorics formula sheet

Practice: Solve 40 counting problems from competitions

Graph Theory Basics

Topics:

- Graph definitions and terminology
- Types of graphs: complete, bipartite, planar
- Degree sequences and handshaking lemma
- Trees and forest properties
- Eulerian paths and circuits
- Hamiltonian paths and cycles
- Graph coloring and chromatic number
- Planar graphs and Euler's formula
- Bipartite matching basics
- Hall's marriage theorem

- Tournament graphs
- Competition graph problems

Projects built:

- Graph theory problem solving guide
- Visualization of graph algorithms
- Competition graph problems collection

Practice: Master 35 graph theory competition problems

Phase 1 Review and Competition

Topics:

- Number theory comprehensive review
- Modular arithmetic mastery check
- Combinatorics problem-solving strategies
- Graph theory applications
- Mixed problem sets
- Speed solving techniques
- Competition simulation

Projects built:

- Create Phase 1 solution manual
- Mock AMC 10 competition
- Peer problem exchange and solving

Assessment: Phase 1 Mock Competition - 3 hour exam

----- PHASE 2: ADVANCED ALGEBRA & POLYNOMIAL THEORY (MONTHS 4-6, WEEKS 14-26) -----

Master algebraic manipulations, polynomial theory, sequences, and functional equations for competitions.

Months 4-5: Competition Algebra & Polynomials (Week 14-21)

Advanced Algebraic Techniques

Topics:

- Algebraic manipulation mastery
- Telescoping sums and products
- Partial fractions in competitions
- Symmetric expressions and polynomials
- Vieta's formulas and applications
- Newton's identities
- Power sum symmetric functions
- Discriminant and its properties
- Algebraic substitutions techniques
- Rationalizing complex expressions
- Nested radicals simplification
- Competition algebraic tricks

Projects built:

- Algebraic techniques handbook
- Solve 50 AIME algebra problems
- Create substitution strategy guide

Practice: Daily: 8 advanced algebra competition problems

Polynomial Theory

Topics:

- Polynomial division and remainder theorem
- Factor theorem and applications
- Rational root theorem extensions
- Descartes' rule of signs
- Complex roots and conjugate pairs
- Fundamental theorem of algebra
- Polynomial interpolation (Lagrange)
- Chebyshev polynomials introduction
- Cyclotomic polynomials
- Irreducibility testing
- Polynomial inequalities
- Roots and coefficients relationships

Projects built:

- Polynomial problem solving manual
- Root-finding techniques compilation
- IMO polynomial problems analysis

Practice: 45 polynomial problems from national olympiads

Sequences and Series

Topics:

- Arithmetic and geometric progressions advanced
- Arithmetico-geometric progressions
- Harmonic progressions and means
- Recursive sequences and linear recurrences
- Characteristic equations method
- Generating functions for sequences
- Fibonacci and Lucas sequences properties
- Catalan numbers and applications
- Convergence tests for series
- Telescoping series mastery

- Power series basics
- Competition sequence problems

Projects built:

- Sequence encyclopedia creation
- Recurrence relation solver
- Famous sequences investigation

Practice: Solve 40 sequence and series competition problems

Equations and Systems

Topics:

- Systems of linear equations (advanced)
- Nonlinear systems solving techniques
- Symmetric systems of equations
- Cyclic systems
- Homogeneous equations
- Parametric solutions
- Diophantine equations (advanced)
- Quadratic Diophantine equations
- Simon's Favorite Factoring Trick
- Equations with absolute values
- Floor and ceiling function equations
- Competition equation strategies

Projects built:

- Equation solving techniques manual
- Diophantine equation collection
- System solving flowchart

Practice: Master 50 competition equation problems

Complex Numbers in Competitions

Topics:

- Complex number operations review
- Geometric interpretation of complex operations
- De Moivre's theorem and applications
- Roots of unity and cyclotomic polynomials
- Complex number proofs in geometry
- Trigonometric identities via complex numbers
- Gaussian integers
- Complex conjugate root theorem
- Argument and modulus inequalities
- Competition problems using complex numbers
- Euler's formula applications
- Advanced complex number techniques

Projects built:

- Complex numbers in geometry guide
- Roots of unity problem set
- Complex number competition tricks

Practice: 30 complex number competition problems

Month 6: Functional Equations & Advanced Topics (Week 22-26)

Functional Equations

Topics:

- Introduction to functional equations
- Cauchy's functional equations
- Substitution methods in functional equations
- Finding solutions: injectivity, surjectivity
- Additive and multiplicative functions
- Jensen's functional equation
- D'Alembert's functional equation
- Polynomial functional equations
- Functional equations with multiple variables
- Cyclic functional equations
- IMO functional equation techniques
- Common competition patterns

Projects built:

- Functional equation solving guide
- Pattern recognition in functional equations
- IMO functional equations compilation

Practice: Solve 35 functional equation problems

Mathematical Induction Advanced

Topics:

- Strong induction principles
- Structural induction
- Double induction
- Infinite descent method
- Well-ordering principle applications
- Induction with inequalities
- Induction in number theory
- Induction in combinatorics
- Induction for sequences
- Forward-backward induction
- Transfinite induction introduction

- Competition induction strategies

Projects built:

- Induction proof portfolio
- Induction techniques classification
- Create induction problem bank

Practice: Complete 40 induction problems of varying difficulty

Pigeonhole Principle & Extremal Problems

Topics:

- Pigeonhole principle variations
- Generalized pigeonhole principle
- Infinite pigeonhole principle
- Probabilistic pigeonhole principle
- Dirichlet's theorem applications
- Extremal principle in problem solving
- Optimization in discrete mathematics
- Erdős-Ko-Rado theorem introduction
- Ramsey theory basics
- Van der Waerden's theorem introduction
- Extremal graph theory
- Competition applications

Projects built:

- Pigeonhole principle masterclass
- Extremal problems collection
- Ramsey theory exploration

Practice: 30 pigeonhole and extremal problems

Invariants and Monovariants

Topics:

- Invariant principle in problem solving
- Finding invariants in processes
- Monovariant strategies
- Parity as an invariant
- Coloring arguments
- Sum and product invariants
- Geometric invariants
- Game theory and invariants
- Invariants in combinatorial games
- Semi-invariants
- Competition invariant problems
- Creating invariant arguments

Projects built:

- Invariant problem solving guide
- Game theory invariants study
- Invariant problem creation

Practice: Master 35 invariant-based problems

Phase 2 Review and Competition

Topics:

- Algebra and polynomials review
- Sequences and series mastery
- Functional equations practice
- Complex numbers applications
- Induction and extremal principles
- Mixed advanced problems
- AIME simulation

Projects built:

- Phase 2 complete solution manual
- Mock AIME competition
- Problem-solving video tutorials

Assessment: Phase 2 Mock Competition - AIME level

----- PHASE 3: COMPETITION GEOMETRY & TRIGONOMETRY (MONTHS 7-9, WEEKS 27-39) -----

Master advanced geometric techniques, trigonometric identities, and coordinate geometry for olympiad success.

Months 7-8: Euclidean Geometry & Triangle Theory (Week 27-34)

Advanced Triangle Geometry

Topics:

- Cevians: medians, altitudes, angle bisectors
- Special points: centroid, orthocenter, incenter, circumcenter
- Euler line and nine-point circle
- Nagel point and Gergonne point
- Stewart's theorem and applications
- Menelaus' theorem
- Ceva's theorem and trigonometric form
- Mass point geometry
- Barycentric coordinates introduction
- Triangle inequality variations
- Routh's theorem

- Competition triangle problems

Projects built:

- Triangle centers encyclopedia
- Special points construction guide
- Triangle theorem proof collection

Practice: Solve 45 advanced triangle problems

Circle Geometry

Topics:

- Power of a point theorem
- Radical axis and radical center
- Coaxial circles
- Inversion in circles
- Ptolemy's theorem and inequality
- Cyclic quadrilaterals properties
- Simson line and pedal triangles
- Miquel's theorem
- Pascal's theorem for circles
- Brianchon's theorem
- Nine-point circle properties
- Competition circle techniques

Projects built:

- Circle theorems visual guide
- Inversion problem solving
- Cyclic quadrilateral mastery

Practice: Master 40 circle geometry problems

Similarity and Homothety

Topics:

- Advanced similarity principles
- Spiral similarities
- Homothety and homothetic centers
- Similar triangles in competitions
- Angle bisector theorem extensions
- Apollonius' theorem
- Harmonic division and conjugates
- Cross-ratio and applications
- Desargues' theorem
- Monge's theorem
- Homothety in circle problems
- Competition applications

Projects built:

- Similarity techniques manual
- Homothety problem collection
- Projective geometry introduction

Practice: Complete 35 similarity and homothety problems

Geometric Transformations

Topics:

- Reflection properties and compositions
- Rotation compositions and fixed points
- Translation vectors and compositions
- Glide reflections
- Isometries classification
- Dilation and scaling transformations
- Affine transformations
- Transformation approach to problems
- Symmetry in problem solving
- Spiral similarities revisited
- Complex numbers for transformations
- Competition transformation techniques

Projects built:

- Transformation problem solving guide
- Symmetry exploitation techniques
- Transformation composition calculator

Practice: Solve 40 transformation-based problems

Construction Problems

Topics:

- Classical constructions review
- Constructible numbers theory
- Impossible constructions and proofs
- Mohr-Mascheroni constructions
- Poncelet-Steiner constructions
- Construction strategies in competitions
- Locus problems
- Construction with restrictions
- Geometric optimization constructions
- Construction existence proofs
- Competition construction problems
- Software-aided exploration

Projects built:

- Construction problem anthology

- Impossible constructions proofs
 - Interactive construction tool
- Practice: Complete 30 construction problems

Month 9: Trigonometry & Coordinate Geometry (Week 35-39)

Advanced Trigonometry

Topics:

- Trigonometric identities mastery
- Sum-to-product and product-to-sum formulas
- Multiple angle formulas
- Half-angle formulas applications
- Conditional identities
- Trigonometric equations (advanced)
- Trigonometric inequalities
- Jensen's inequality for trigonometry
- Chebyshev polynomials and trigonometry
- Trigonometric substitutions
- Competition trigonometry tricks
- Proofs using trigonometry

Projects built:

- Trigonometric identity encyclopedia
- Competition trigonometry handbook
- Trigonometric proof portfolio

Practice: Master 40 trigonometry competition problems

Analytic Geometry

Topics:

- Coordinate geometry strategies
- Distance and section formulas
- Area calculations in coordinates
- Line equations and properties
- Angle between lines
- Conic sections: parabola, ellipse, hyperbola
- Parametric representations
- Polar coordinates applications
- Coordinate transformations
- Analytic proofs of geometric theorems
- Optimization using coordinates
- Competition coordinate techniques

Projects built:

- Coordinate geometry solver
- Conic sections problem set
- Analytic vs synthetic comparison

Practice: Solve 35 coordinate geometry problems

Vectors in Geometry

Topics:

- Vector operations and properties
- Dot product and cross product
- Vector proofs of geometric theorems
- Position vectors and applications
- Vector equations of lines and planes
- Distance using vectors
- Angle calculations with vectors
- Vector approach to transformations
- Centroid and circumcenter via vectors
- 3D geometry basics
- Competition vector techniques
- Vectors vs coordinates comparison

Projects built:

- Vector geometry manual
- 3D geometry exploration
- Vector solution compilation

Practice: Complete 30 vector geometry problems

Area and Volume Methods

Topics:

- Area calculation techniques
- Shoelace formula and applications
- Pick's theorem
- Heron's formula and extensions
- Bretschneider's formula
- Brahmagupta's formula
- Volume calculation methods
- Cavalieri's principle
- Volume by cross-sections
- Surface area calculations
- Pappus's centroid theorems
- Competition area/volume tricks

Projects built:

- Area formula compendium
- Volume calculation guide

- Historical formula exploration

Practice: Master 35 area and volume problems

Phase 3 Review and Competition

Topics:

- Euclidean geometry mastery review
- Circle theorems compilation
- Transformation techniques
- Trigonometry applications
- Coordinate methods review
- Vector approaches
- Mixed geometry problems
- USAMO geometry preparation

Projects built:

- Geometry techniques comparison chart
- Mock USAMO geometry round
- Geometry problem video solutions

Assessment: Phase 3 Geometry Olympiad - 4.5 hours

----- PHASE 4: INEQUALITIES, ADVANCED TOPICS & IMO PREPARATION (MONTHS 10-12, WEEKS 40-52) -----

Master inequalities, advanced problem-solving techniques, and prepare for international competitions.

Months 10-11: Inequalities & Optimization (Week 40-47)

Classical Inequalities

Topics:

- AM-GM inequality and applications
- Cauchy-Schwarz inequality proofs and uses
- Holder's inequality
- Minkowski's inequality
- Power mean inequalities
- Weighted AM-GM inequality
- Jensen's inequality for convex functions
- Karamata's inequality
- Rearrangement inequality
- Chebyshev's inequality
- Bernoulli's inequality
- Competition inequality techniques

Projects built:

- Inequality proof techniques guide
- Classical inequality applications
- Inequality chain constructions

Practice: Prove 50 competition inequalities

Advanced Inequality Techniques

Topics:

- Smoothing and majorization
- Mixing variables technique
- Lagrange multipliers introduction
- Homogenization techniques
- Substitutions in inequalities
- Cyclic and symmetric inequalities
- Buffalo Way method
- SOS (Sum of Squares) method
- Schur's inequality and generalizations
- Muirhead's inequality
- Newton's inequalities
- IMO inequality strategies

Projects built:

- Advanced inequality manual
- SOS method implementation
- IMO inequalities analysis

Practice: Master 40 advanced inequality problems

Optimization Problems

Topics:

- Discrete optimization techniques
- Continuous optimization basics
- Calculus in olympiad problems
- Derivatives for optimization
- Constrained optimization
- Geometric optimization problems
- Isoperimetric problems
- Algebraic optimization
- Combinatorial optimization
- Game theory optimization
- Dynamic programming introduction
- Competition optimization strategies

Projects built:

- Optimization techniques handbook

- Geometric optimization collection
- Create optimization problem set

Practice: Solve 35 optimization problems

Advanced Combinatorics

Topics:

- Bijective proofs and counting
- Double counting techniques
- Combinatorial identities (advanced)
- Vandermonde's identity applications
- Catalan number applications
- Bell numbers and Stirling numbers
- Integer partitions
- Young tableaux introduction
- Polya enumeration theorem
- Combinatorial game theory
- Probabilistic methods
- Competition combinatorics mastery

Projects built:

- Combinatorial proof anthology
- Counting techniques flowchart
- Game theory analysis

Practice: Complete 40 advanced combinatorics problems

Advanced Number Theory Topics

Topics:

- Quadratic reciprocity (complete)
- Primitive roots and indices
- Quadratic forms basics
- Continued fractions (advanced)
- Transcendental numbers introduction
- p-adic numbers introduction
- Analytic number theory glimpse
- Additive number theory basics
- Beatty sequences
- Thue's lemma
- Advanced Diophantine equations
- IMO number theory trends

Projects built:

- Advanced number theory compendium
- Historical problems exploration
- Number theory research topics

Practice: Tackle 30 research-level number theory problems

Month 11: Problem-Solving Mastery & Competition Strategy (Week 48-52)

IMO Problem Analysis

Topics:

- IMO problem structure and trends
- Problem 1 and 4 strategies (easier problems)
- Problem 2 and 5 strategies (medium problems)
- Problem 3 and 6 strategies (hardest problems)
- Time management for 4.5-hour exams
- Partial credit maximization
- Problem selection strategies
- Writing rigorous proofs
- Common IMO problem patterns
- Historical IMO problems analysis
- National olympiad comparison
- Training camp preparation

Projects built:

- IMO problems by topic classification
- Personal IMO strategy development
- Mock IMO with analysis

Practice: Solve 20 past IMO problems

Putnam Competition Preparation

Topics:

- Putnam exam format and scoring
- Putnam problem characteristics
- Abstract algebra in Putnam
- Real analysis topics
- Linear algebra applications
- Probability in Putnam problems
- Putnam trick techniques
- Time management for Putnam
- A vs B session strategies
- Historical Putnam analysis
- University-level competition transition
- Research mathematics introduction

Projects built:

- Putnam problem collection by topic
- University math preview

- Putnam simulation exam

Practice: Complete 30 Putnam problems

Asian Pacific Mathematics Olympiad

Topics:

- APMO problem characteristics
- Regional competition differences
- APMO vs IMO difficulty comparison
- Cultural problem-solving approaches
- APMO-specific techniques
- Time zone competition strategies
- Online competition protocols
- APMO historical trends
- Country-specific olympiad styles
- International collaboration benefits
- APMO as IMO preparation
- Regional olympiad circuit

Projects built:

- APMO complete solutions manual
- Regional olympiad comparison study
- APMO mock competition

Practice: Solve 25 APMO problems from past years

Competition Psychology & Peak Performance

Topics:

- Mental preparation for competitions
- Stress management techniques
- Visualization and positive thinking
- Competition day routines
- Dealing with difficult problems
- Recovery from mistakes
- Time pressure management
- Building mathematical confidence
- Team competition strategies
- Learning from competition failures
- Post-competition analysis
- Long-term training mindset

Projects built:

- Personal competition preparation guide
- Competition journal and reflection
- Mental training program design

Practice: Implement mental training with timed problem sets

Research Mathematics Introduction

Topics:

- From olympiads to research mathematics
- Open problems in mathematics
- Reading mathematical papers
- Mathematical writing skills
- LaTeX for mathematics
- Undergraduate research opportunities
- REU programs and applications
- Mathematical journals for students
- Online mathematics communities
- Collaboration in mathematics
- Career paths in mathematics
- Graduate school preparation

Projects built:

- Write first mathematical paper
- Research topic exploration
- Academic mathematics plan

Practice: Explore 5 research-level topics

Month 12: Mock Competitions & Final Preparation (Week 49-52)

National Olympiad Simulations

Topics:

- USAMO complete simulation
- USAJMO preparation strategies
- Canadian Mathematical Olympiad
- British Mathematical Olympiad
- Australian Mathematics Olympiad
- Indian National MO preparation
- China National Team Selection
- Russia Mathematical Olympiad
- Time management across formats
- Scoring system understanding
- Selection criteria for teams
- National camp preparation

Projects built:

- Complete 5 national olympiad mocks
- Cross-country problem comparison
- National team selection guide

Practice: One full national olympiad daily
International Competition Week

Topics:

- IMO Day 1 simulation (4.5 hours)
- IMO Day 2 simulation (4.5 hours)
- Complete solution writing
- Proof verification techniques
- Collaboration after competition
- Score prediction and analysis
- International competition etiquette
- Cultural exchange benefits
- Language considerations
- Travel and preparation logistics
- Team dynamics
- Medal boundaries analysis

Projects built:

- Complete IMO simulation with scoring
- Solution comparison international students
- Competition reflection essay

Practice: Full IMO mock over two days

Topic Integration & Problem Creation

Topics:

- Creating original problems
- Problem difficulty assessment
- Multi-topic problem design
- Solution elegance principles
- Alternative solution finding
- Problem generalization techniques
- Special case analysis
- Problem posing for competitions
- Reviewing peer solutions
- Mathematical exposition
- Building problem databases
- Contributing to mathematics community

Projects built:

- Create 10 original competition problems
- Design mini-competition for peers
- Build personal problem archive

Practice: Solve and create 5 integrated problems daily

Advanced Problem-Solving Techniques

Topics:

- Probabilistic method in combinatorics
- Algebraic topology glimpses
- Category theory basics
- Model theory introduction
- Ergodic theory applications
- Representation theory basics
- Algebraic geometry introduction
- Differential geometry concepts
- Measure theory basics
- Fourier analysis glimpses
- Mathematical physics connections
- Frontier mathematics overview

Projects built:

- Advanced mathematics exploration
- University course preview
- Mathematical interests portfolio

Practice: Explore 10 advanced mathematical topics

Month 12: Final Assessments & Future Planning (Week 51-52)

Comprehensive Review & Mastery

Topics:

- Number theory complete review
- Combinatorics mastery assessment
- Algebra and polynomials review
- Geometry techniques compilation
- Inequality mastery check
- Functional equations review
- Problem-solving strategies synthesis
- Proof writing excellence
- Speed and accuracy balance
- Knowledge gap identification
- Personal strength analysis
- Improvement planning

Projects built:

- Complete technique reference manual
- Personal problem-solving guide
- Video solution library creation

Final Competition Series

Topics:

- AMC 12 simulation
- AIME simulation
- USAMO simulation Day 1
- USAMO simulation Day 2
- IMO simulation Day 1
- IMO simulation Day 2
- Score analysis and prediction
- Performance evaluation
- Peer competition and ranking
- Solution presentation
- Awards and recognition
- Competition portfolio completion

Deliverables:

- Complete competition portfolio
- All mock competition scores
- Solution manual for all attempts
- Performance analysis report
- Technique mastery certification
- Problem-solving speed metrics

Future Planning & Career Guidance

Topics:

- University mathematics programs
- Scholarship opportunities
- Research program applications
- Summer mathematics camps
- International exchange programs
- Online competition platforms
- Mathematical communities joining
- Mentorship opportunities
- Teaching and tutoring paths
- Competition coaching possibilities
- Long-term mathematics goals
- Life-long learning strategies

Deliverables:

- Competition achievement portfolio
- University application materials
- Scholarship application preparation
- Future competition calendar
- Mathematical CV creation
- Recommendation letter materials
- Personal mathematics statement

Assessment: FINAL OLYMPIAD - Comprehensive 6-hour examination

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: AMC Training Manual, Number Theory Proofs, Combinatorics Guide, Mock Competitions
- Phase 2: AIME Preparation Kit, Polynomial Mastery, Functional Equations Solver, Complex Numbers Toolkit
- Phase 3: Geometry Compendium, Trigonometry Manual, Construction Problems, Vector Methods Guide
- Phase 4: Inequality Bible, IMO Solutions Archive, Competition Strategy Guide, Research Paper
- Final: Complete Olympiad Preparation System, Original Problem Collection, Teaching Materials

Total projects: 75+ mathematical projects and problem collections

SKILLS THE STUDENT WILL HAVE AT THE END

- Number Theory: Modular arithmetic, Diophantine equations, prime theory, quadratic reciprocity
- Combinatorics: Advanced counting, graph theory, generating functions, probabilistic method
- Algebra: Polynomials, functional equations, complex numbers, inequalities, optimization
- Geometry: Euclidean geometry, transformations, coordinate methods, construction problems
- Proof Techniques: Induction, contradiction, pigeonhole, invariants, extremal principle
- Inequalities: Classical inequalities, SOS method, smoothing, Lagrange multipliers
- Problem Solving: Pattern recognition, case analysis, working backwards, generalization
- Competition Skills: Time management, partial credit, proof writing, score optimization
- Advanced Topics: Generating functions, game theory, research mathematics introduction
- Mental Skills: Competition psychology, stress management, peak performance
- Mathematical Writing: Rigorous proofs, LaTeX, problem creation, solution exposition
- International Competitions: IMO, APMO, Putnam, national olympiads preparation

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-6 hours
Hands on practice: 8-10 hours
Projects: 3-4 hours
Practice problems: 5-6 hours
Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving sessions with experts
Mentorship: 1-on-1 coaching from olympiad medalists
Community: Elite competition preparation community
Code review: Solution reviews and feedback
Career support: University admissions guidance
Lifetime access: All materials and future updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion
Final certificate: Olympiad Mathematics Excellence Certificate
LinkedIn badge: Competition mathematics credential
Industry recognized: Recognized by universities and mathematics departments
Portfolio projects: 1000+ solved competition problems portfolio

WHERE THIS CAN LEAD

- International Mathematical Olympiad Participant
- National Olympiad Team Member
- University Mathematics Major
- Mathematics Research Career
- Quantitative Finance Analyst
- Data Science and Machine Learning
- Cryptography and Security
- Academic Mathematics Professor
- Competition Mathematics Coach
- Mathematical Content Creator

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the olympiad mathematics program cover?

A: The 12 month curriculum works through the four classical olympiad pillars: number theory and combinatorics, competition algebra with polynomials and functional equations, Euclidean and coordinate geometry with trigonometry, and inequalities with advanced problem solving strategy, ending in mock competitions. The goal is genuine problem solving depth, not memorized tricks.

Q: Which competitions does this prepare students for?

A: The training maps to the Indian olympiad pipeline, IOQM leading toward RMO and INMO, plus NMTC and SOF style olympiads, and the same material serves AMC and AIME aspirants since the topic areas overlap heavily. We prepare students honestly for the level they are at, and we never promise medals or ranks, because no honest program can.

Q: Is my child ready for this course?

A: It suits students with a strong grasp of school mathematics who enjoy hard problems, typically from middle school upward. It is not designed for closing school level gaps; students who need foundations first are better served by our school mathematics programs and can graduate into this one.

Q: What are the prerequisites?

A: Solid school algebra and geometry, care in reading mathematical statements, and a genuine appetite for problems that take longer than five minutes. Competition maths rewards consistent weekly practice more than raw talent, so expect a serious commitment.

Q: How is progress measured in competition maths?

A: Through weekly problem sets at calibrated difficulty, a personal problem log tracking which techniques are secure, and regular timed mock papers in the final months. Progress here looks like solving harder problems more independently, and the log makes that visible.

Q: Can we try a session first?

A: Yes, book a free demo class at learn.modernagecoders.com/contact with no commitment; we use it to gauge your child's current problem solving level and to recommend honestly whether this course or a foundations course fits better. WhatsApp +91 91233 66161 for quick questions.

Q: What is the class format?

A: Live one hour online classes with a real teacher in deliberately small groups, never recorded lectures. Students present their attempts and the mentor works the reasoning with them, which is the only way competition thinking actually develops.

Q: Is there a certificate, and what else does my child take away?

A: Yes, an Olympiad Mathematics Excellence Certificate on completion, plus the more valuable artifact: a documented problem portfolio and mock paper record showing the level reached. That evidence outlasts any single contest result.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/olympiad-competition-mathematics-mastery>

COURSE 84 OF 118: MATHS THROUGH CODING: LEARN MATH IN PYTHON (AGES 10-15)

Course link: <https://learn.modernagecoders.com/courses/maths-through-coding>
Subject area: Mathematics & Creative Technology
Designed for: Middle school
Who can join: Ages 10-15 (grades 5-10); no coding experience required
Course length: 9 months (36 weeks)
Time needed: 2 live one-hour classes per week + one small build between classes
Certificate: Course completion certificate + a portfolio of working math programs

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live course where kids ages 10-15 learn maths by coding it: fractions, algebra, geometry and probability built as Python projects. 8 classes a month, free demo.

In one line: If you can build it, you understand it. Students build the maths.

ABOUT THE PROGRAMME

A 9-month live course teaching mathematics the way our whole school believes it should be met: by building it. Students write Python programs that ARE the maths, a fraction simplifier that forces understanding of GCD, turtle geometry that makes angles visceral, probability simulators that settle arguments with ten thousand dice rolls, and function grapher projects that turn algebra into something you can watch. This is the signature Modern Age Coders fusion, taught by mentors genuinely fluent in both subjects, and for Ontario families, it happens to cover the coding expectations now examined inside math class from Grade 1. No prior coding needed; the mathematics runs roughly grades 5-9 territory with stretch room.

WHAT MAKES THIS COURSE DIFFERENT

- The maths is the syllabus, the code is the method: every program exists to force a mathematical idea into the open
- A computer rejects vague understanding, code that half-understands fractions simply does not run, which makes debugging a mathematics lesson
- Mentors fluent in both subjects, because a coding teacher who waves at the maths (or vice versa) breaks the whole idea
- Real Python from week one, at a pace built for ages 10-15
- Every topic ends with a working artifact the student built and can demo
- Directly serves Ontario's coding-in-math strand, and the computational-thinking push in curricula worldwide
- The engagement answer for the child who says maths is boring but plays with code for hours, or the reverse
- One dedicated mentor, one full interactive hour, twice a week

THE LEARNING PATH, STEP BY STEP

Phase 1: Numbers as programs: Python foundations through arithmetic, divisibility, fractions and number patterns
Phase 2: Shape and change: turtle geometry, coordinates, and algebra brought alive through variables, functions and graphing
Phase 3: Chance and data: probability simulators, statistics on real datasets, and a capstone project of the student's own design

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Ages 10-15 (roughly grades 5-10)
Math background: Comfortable whole-number arithmetic; everything else is built inside the course
Coding experience: None required, Python taught from zero
Equipment: Computer with stable internet; free tools only (no purchases)
English: Course taught in English
Motivation: Curiosity about either half, maths or code, is enough; the course supplies the other

WHO THIS COURSE IS FOR

Bored by worksheets: The child who says maths is boring but builds in Minecraft for hours
Maths lovers: The child who loves maths and deserves the best maths toy ever made
Ontario families: Families facing coding-inside-math curriculum expectations with nobody home

fluent in both
Understanding seekers: Students who pass tests but want (or need) the ideas to actually make sense
Future builders: Kids headed for our programming tracks who want the mathematical spine first

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: NUMBERS AS PROGRAMS (MONTHS 1-3) -----

Python foundations learned through number theory a student can touch: parity, divisibility, primes, factors and fractions. By phase end the student programs comfortably and, the real goal, has met arithmetic ideas so concretely they can never again be mysterious.

Month 1: First Programs, First Number Ideas (Weeks 1-4)

Hello Python: Editor, Print and Variables

Topics:

- Setting up and using the Python editor
- The print function
- Variables and naming
- Numbers and text as data
- A diagnostic of maths comfort
- A diagnostic of machine comfort

Projects built:

- Write a Python script that prints a personalised greeting built from variables

Practice: Write and run a short program that stores your name and age in variables and prints a sentence.

Detail: Hello, Python: the editor, print, variables, and a diagnostic of both maths comfort and machine comfort

Arithmetic Operators and Order of Operations

Topics:

- Addition, subtraction, multiplication and division operators
- Order of operations in code
- Brackets to control evaluation
- Integer versus decimal results
- Matching code grammar to maths grammar
- Predicting an expression before running it

Projects built:

- Write a Python calculator that evaluates a multi-step expression and prints the result

Practice: Predict the value of several mixed-operator expressions, then check each by running it.

Detail: Arithmetic operators and order of operations: the computer obeys the same grammar your maths teacher does

Integer Division, Remainders and Modulo

Topics:

- Integer division
- The modulo operator
- Modulo as a parity machine
- Even and odd detectors
- Cycle and repeating patterns with modulo
- The clock-arithmetic idea

Projects built:

- Write a Python script that labels every number from 1 to 50 as even or odd using modulo

Practice: Use modulo to test a list of numbers for even or odd and describe the pattern you see.

Detail: Integer division and remainders: modulo as the parity machine, even/odd detectors and cycle patterns

Conditionals and Divisibility Rules

Topics:

- if, elif and else
- Comparison operators
- Coding divisibility tests
- Explaining divisibility rules backwards
- Combining conditions
- Reading a program's branches

Projects built:

- Write a Python program that reports which of 2, 3, 5 and 10 a number is divisible by

Practice: Code a divisibility checker for 2, 3 and 5 and explain in your own words why each rule works.

Detail: Conditionals with number tests: divisibility rules coded and then explained backwards

Month 2: Loops Meet Number Theory (Weeks 5-8)

Loops and Times-Table Patterns

Topics:

- for loops and range
- Loops as purposeful repetition

- Generating times tables
- Hunting patterns inside the tables
- Explaining the patterns found
- The nested-loop idea introduced

Projects built:

- Write a Python script that visualises a times-table as a grid of rows and columns

Practice: Generate several times tables with a loop and write down one pattern you notice in each.

Detail: Loops as repetition with purpose: times tables generated, then patterns in them hunted and explained

Factor Finders and Prime Testers

Topics:

- Finding all factors of a number
- Building a prime tester
- The definition of a prime
- Why 1 is not prime, settled by code
- Writing your first real algorithms
- Testing an algorithm across many inputs

Projects built:

- Write a Python program that lists every factor of a number
- Write a Python prime tester that prints all primes up to 100

Practice: Run your prime tester across the numbers 1 to 30 and confirm the results by hand.

Detail: Factor finders and prime testers: the student's first real algorithms, and why 1 is not prime, settled by code

Month 3: The Fraction Machine (Weeks 9-12)

GCD and the Fraction Simplifier

Topics:

- Common factors and the GCD
- The Euclidean idea for finding a GCD
- Coding a GCD function
- Fraction equivalence
- Building a fraction simplifier
- Why the machine forces real understanding

Projects built:

- Write a Python fraction simplifier that reduces any fraction to lowest terms using the GCD

Practice: Simplify several fractions by hand, then confirm each one with your simplifier.

Detail: GCD discovered through the Euclidean idea; a fraction simplifier built, which forces equivalence to be truly understood

Fraction Arithmetic and Phase Demo Day

Topics:

- Adding and subtracting fractions in code
- Why a machine needs common denominators
- Multiplying and dividing fractions
- Combining the GCD with fraction arithmetic
- Preparing a clear demo
- Presenting the number toolkit

Projects built:

- Write a Python fraction calculator that adds, subtracts and simplifies two fractions

Practice: Extend your fraction program to add and subtract two fractions and simplify the result.

Assessment: Phase demo day: each student presents their number toolkit and explains the mathematics behind it.

Detail: Fraction arithmetic programmed (common denominators become obvious when a machine needs them); phase demo day: each student presents their number toolkit

PHASE 2: SHAPE AND CHANGE (MONTHS 4-6)

Geometry becomes visceral through turtle graphics, exterior angles are unforgettable once you have steered them, and algebra comes alive as code: variables that actually vary, functions as machines, and graphs the student plots themselves.

Month 4: Turtle Geometry (Weeks 13-16)

Turtle Basics and the Exterior-Angle Discovery

Topics:

- Turtle graphics setup
- Drawing squares and triangles
- Forward and turn commands
- Discovering exterior angles by steering
- Why 360 divided by n draws a regular polygon
- Connecting turning to angle size

Projects built:

- Write a Python turtle program that draws any regular polygon from the number of sides

Practice: Draw three different regular polygons with turtle and record the turn angle each one needs.

Detail: Turtle basics: squares, triangles and the exterior-angle discovery (why 360)

divided by n draws every regular polygon)

Star Polygons, Spirals and Angle Art

Topics:

- Drawing star polygons
- Spirals with a changing step or angle
- Angle-art compositions
- Interior-angle sums
- Verifying angle sums by walking them
- Combining loops with turtle turns

Projects built:

- Write a Python turtle program that draws a star polygon or spiral of your own design

Practice: Create an angle-art pattern and explain the interior or exterior angles that make it close.

Detail: Star polygons, spirals and angle art; interior-angle sums verified by walking them

Month 5: Coordinates and Real Variables (Weeks 17-20)

The Coordinate Plane in Code

Topics:

- The screen as a coordinate plane
- Plotting points from coordinates
- Drawing lines between points
- Finding midpoints
- The distance-between-points idea
- Reading coordinates back off a drawing

Projects built:

- Write a Python program that plots points and draws the line and midpoint between two of them

Practice: Plot a set of points in code and compute the midpoint of two of them.

Detail: The coordinate plane as the screen it literally is; plotting points, lines and midpoints in code

Algebra as Code: Expressions and Equations

Topics:

- Evaluating algebraic expressions in code
- Variables that actually vary
- Solving equations by guess, check and refine
- Why balance methods beat brute force
- Substituting into a formula
- Checking a solution programmatically

Projects built:

- Write a Python script that solves a linear equation by guess-check-refine and prints each step

Practice: Solve a linear equation both with a guess-check-refine loop and by balancing, then compare.

Detail: Algebra as code: expressions evaluated, equations solved by guess-check-refine, and why balance methods beat brute force

Month 6: Functions as Machines, Literally (Weeks 21-24)

Functions as Machines: Input, Rule, Output

Topics:

- Defining a Python function
- Parameters and return values
- Mathematical functions as machines
- Input, rule and output as one idea
- Evaluating a function for many inputs
- Building a table of values

Projects built:

- Write a Python function that takes x and returns a rule such as $2x$ plus 1, then tabulates it

Practice: Write a function for a simple rule and print a table of its outputs for several inputs.

Detail: Python functions and mathematical functions introduced as the same idea: input, rule, output

The Grapher Project and Phase Demo Day

Topics:

- Plotting $y = mx + c$
- The role of the gradient m
- The role of the intercept c
- Plotting parabolas
- Watching parameters reshape the graph
- Presenting the grapher at demo day

Projects built:

- Write a Python grapher that plots $y = mx + c$ for chosen values of m and c
- Extend the grapher to plot a parabola and watch how its shape changes

Practice: Use your grapher to compare two lines with different m and c and describe the difference.

Assessment: Phase demo day: each student presents their grapher and explains how m and c shape the line.

Detail: The grapher project: plot $y = mx + c$ and watch m and c do their jobs; then parabolas, and phase demo day

PHASE 3: CHANCE, DATA AND THE CAPSTONE (MONTHS 7-9)

Probability stops being formulas when you can roll ten thousand dice per second: simulators settle intuitions, real datasets get summarized and questioned, and the course closes with a capstone the student designs, a working program where the mathematics is the star.

Month 7: Probability by Simulation (Weeks 25-28)

Randomness: Coin Flippers and Dice Rollers

Topics:

- Random numbers in Python
- Coin-flip simulation
- Dice-roll simulation
- Experimental probability from many trials
- Theoretical probability
- Watching results converge as trials grow

Projects built:

- Write a Python script that simulates thousands of dice rolls and reports the proportion of each outcome

Practice: Simulate 1000 coin flips and compare the experimental probability with the theoretical value.

Detail: Randomness in code: coin flippers and dice rollers; experimental versus theoretical probability, watched converging

Simulating Classic Probability Puzzles

Topics:

- Two-dice sum distributions
- Why 7 is the most likely sum
- Streaks in coin flips
- Calibrating intuition with evidence
- Counting outcomes versus simulating them
- Comparing simulation with theory

Projects built:

- Write a Python simulation of two-dice sums that displays the frequency of each total

Practice: Simulate two-dice sums many times and chart how often each sum appears.

Detail: The classic puzzles simulated: two-dice sums, streaks in coin flips, intuition calibrated by evidence

Month 8: Statistics on Real Data (Weeks 29-32)

Mean, Median and Range in Code

Topics:

- Computing the mean in code
- Finding the median by sorting
- Calculating the range
- Making vague definitions precise
- Handling a list of data
- Checking results against a hand calculation

Projects built:

- Write a Python program that reads a list of numbers and reports the mean, median and range

Practice: Write code to compute the mean, median and range of a data list, then verify each by hand.

Detail: Mean, median and range computed by the student's own code, the definitions can no longer be vague

Exploring a Real Dataset

Topics:

- Loading a real dataset
- Summarising the data
- Making simple charts
- Spotting outliers
- Questioning the sample size
- Judging whether a comparison is fair

Projects built:

- Write a Python program that summarises a real dataset and draws a simple chart from it

Practice: Summarise a small real dataset and write down two sceptic's questions it raises.

Detail: A real dataset explored: summaries, simple charts, and the sceptic's questions (outliers, sample size, fair comparisons)

Month 9: The Capstone (Weeks 33-36)

Capstone Chosen and Scoped

Topics:

- Choosing a capstone idea
- Scoping the mathematics inside it
- Planning the build in steps
- Setting a realistic target
- Options: a trainer, an art generator, a game or a data story
- Mentor rigour applied to the maths

Projects built:

- Begin building your chosen capstone program in Python from your scoped plan

Practice: Write a one-page plan for your capstone that names the mathematics it will

make visible.

Detail: Capstone chosen and scoped: a times-table trainer, geometry art generator, probability game, data story, the student's call, the mentor's rigor

Capstone Build, Test and Demo Day

Topics:

- Building the capstone
- Testing and debugging
- Polishing the program
- Preparing a clear demo
- Presenting the program
- Explaining the mathematics behind it

Projects built:

- Complete, test and polish your capstone program ready to demo

Practice: Test your capstone on several cases, fix what breaks, and rehearse your demo explanation.

Assessment: Final demo day: each student presents the capstone and explains the mathematics it is built on.

Detail: Build, test, polish; demo day, the student presents their program and, more importantly, explains its mathematics

PROJECTS THE STUDENT BUILDS AND KEEPS

- Number toolkit: parity detector, divisibility tester, prime finder, factor lister
- The fraction machine: simplifier and calculator built on real GCD understanding
- Turtle geometry gallery: polygons, stars and angle art with the maths explained
- Function grapher: lines and parabolas plotted, parameters explored live
- Probability simulators and a real-data statistics story
- The capstone: a self-designed program where the mathematics is the star

Total projects: 12+ working programs, demo-ready

SKILLS THE STUDENT WILL HAVE AT THE END

- Number theory made concrete: primes, factors, GCD, modulo
- Fraction understanding at machine-proof depth
- Geometry through construction: angles, polygons, coordinates
- Algebra as living code: variables, expressions, functions
- Linear functions and graphing
- Probability by simulation and statistics on real data
- Python fundamentals: variables, conditionals, loops, functions
- Computational thinking and debugging discipline

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live build-and-explain classes

Between class: One small build or extension challenge

Demo days: Each phase ends with the student presenting their work

Total per week: About 3-4 absorbed hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week

Mentorship: One mentor fluent in both mathematics and programming

Recordings: Every class recorded

Progress tracking: Project portfolio + concept checks shared with parents

Flexibility: Slots move around school and family plans

CERTIFICATION DETAIL

Final certificate: Course completion certificate from Modern Age Coders

Portfolio projects: A demo-ready portfolio of 12+ working math programs

Pathway: Direct on-ramp to our full Python and AI programming tracks

WHERE THIS CAN LEAD

- Mathematics that holds through high school, because it was built
- Ready for our full Python and AI programming masterclasses
- Computational-thinking maturity that shows up in science and school projects
- A natural bridge to competition mathematics for the puzzle-hungry

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this a maths course or a coding course?

A: A maths course that uses code as its laboratory. The syllabus is mathematical, number theory, fractions, geometry, algebra, probability, statistics, and every program exists to force one of those ideas into the open. Students do finish with real Python skills, which is a genuine bonus, but the mathematics is the star.

Q: Does my child need coding experience?

A: None. Python is taught from the first hello, at a pace built for ages 10-15. Children who already code a little move faster into the mathematical meat; the mentor adjusts either way.

Q: Why does building things teach maths better than practicing problems?

A: Because a computer rejects vague understanding. A student who 'sort of' knows fraction equivalence writes a simplifier that fails, and fixing it requires actually understanding GCD. That debugging loop, predict, test, explain the gap, is the deepest mathematical practice there is, and it feels like play.

Q: We're in Ontario and coding is now examined inside math class. Does this help?

A: Directly, this course is our sweet spot for exactly that curriculum. Ontario weaves coding expectations through math from Grade 1, most homes have nobody fluent in both halves, and our mentors teach both natively. Students walk back into class ahead on the strand instead of puzzled by it.

Q: My child loves games but says maths is boring. Will this work?

A: This course exists for that child. The maths arrives wearing a project the child wants to build, angle art, a dice game, a grapher, and engagement does the heavy lifting. The reverse child (loves maths, indifferent to screens) discovers code as the best maths toy ever made.

Q: What maths level does it assume and reach?

A: It starts from comfortable whole-number arithmetic (roughly grade 5) and reaches into linear functions, probability and statistics (roughly grade 9 territory), with stretch room for strong students. It runs happily alongside any school curriculum and strengthens it rather than clashing.

Q: Can we try it before paying?

A: Yes. Every student starts with a free live demo class, they will build and run their first small program inside that hour, and you will see the method working with your own child. No card details required.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/maths-through-coding>

=====

COURSE 85 OF 118: MENTAL MATHS FOR KIDS: NUMBER SENSE, NOT TRICKS (AGES 5-10)

=====

Course link: <https://learn.modernagecoders.com/courses/mental-maths-mastery-kids>

Subject area: Elementary Mathematics

Designed for: Elementary school

Who can join: Ages 5-10 (roughly grades K-5), all starting levels

Course length: 12 months (52 weeks)

Time needed: 2 live one-hour classes per week + 10 minutes of daily play-practice

Certificate: Course completion certificate + stage mastery checks

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live mental maths course for kids ages 5-10: number bonds, decomposition and estimation build real number sense, the honest alternative to abacus. Free demo.

In one line: Number sense, not tricks. The speed follows, and it never expires.

ABOUT THE PROGRAMME

A 12-month live program that builds genuine mental arithmetic in children aged 5-10, the honest alternative to abacus franchises and 'Vedic tricks' courses. Trick-based speed impresses at demos and quietly expires: it bypasses understanding, so it cannot transfer to fractions, algebra or problem solving. We build number sense instead, numbers as decomposable quantities, number bonds, compensation and estimation, through games and puzzles a child genuinely enjoys. The result is a child who calculates fast because they see numbers clearly, and that clarity keeps paying through every year of school mathematics.

WHAT MAKES THIS COURSE DIFFERENT

- Number sense over tricks: strategies a child can explain always beat procedures they can only perform
- The speed is real and durable, decomposition, bonds and compensation transfer straight into fractions and algebra later
- Games, puzzles and number talks: children wait for these classes, which is the engagement no drill sheet buys
- Concrete-pictorial-abstract progression, the method behind the world's strongest maths

systems

- Explain-back in every class: what a child can teach, a child owns
- One dedicated mentor who knows exactly which facts and strategies each child holds
- A genuine on-ramp: graduates flow into our grade-level maths programs already ahead
- One full interactive hour per class, twice a week, built for young attention spans with variety, not lectures

THE LEARNING PATH, STEP BY STEP

Phase 1: Number sense foundations: quantities, bonds to 10 and 20, and the part-whole thinking everything else stands on

Phase 2: The strategy toolkit: make-ten, doubles, compensation, and multiplication as structure, not chanting

Phase 3: Big-number courage: multi-digit mental strategies, fraction sense beginnings, estimation mastery and speed with understanding

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Ages 5-10; can count to 20 to start the youngest track

Math background: None assumed, the diagnostic finds the true stage

Equipment: Computer or tablet with stable internet; household objects double as manipulatives

Time commitment: 2 live hours plus 10 minutes of daily play

English: Course taught in English

Motivation: None needed on day one, building it is the course's job

WHO THIS COURSE IS FOR

Finger counters: Children counting on fingers past grade 2, whose bonds never became automatic

Maths worriers: Children already saying 'I'm bad at maths', the belief this course exists to reverse

Abacus shoppers: Parents comparing abacus and Vedic programs who want speed that transfers and lasts

Young starters: Ages 5-6 whose parents want the foundation laid right the first time

Quick kids: Naturally strong children ready to go deeper than school arithmetic allows

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: NUMBER SENSE FOUNDATIONS (MONTHS 1-4) -----

Before speed comes seeing: numbers as quantities that break apart and recombine. This phase builds subitizing, part-whole thinking and the bonds of 10 and 20 through concrete materials and games, tuned to each child's starting stage.

Month 1: Seeing Numbers (Weeks 1-4)

Getting to Know Your Number Sense

Topics:

- Counting games that show how far a child counts with ease
- Quick quantity flashes to see which small numbers are known on sight
- Simple addition play that reveals the current strategy (counting on, known facts, or guessing)
- Noticing confidence and comfort with numbers, not only right answers
- Spotting facts already automatic, such as small doubles or bonds
- Setting a friendly starting point tuned to the child, not the age

Practice: Over a snack, ask your child to grab a small handful of raisins and guess how many before counting, then count together to check, keeping it light and playful.

Detail: Playful diagnostic: which facts, strategies and confidence the child already holds, disguised as games

Seeing How Many at a Glance

Topics:

- Recognising 1 to 5 dots instantly without counting (subitizing)
- Reading dice patterns as whole amounts
- Meeting the ten-frame as a picture of numbers
- Seeing 6 to 10 as a small group sitting on top of five
- Flashing and hiding quantities to build quick recognition
- Talking about how the amount was seen, not just the answer

Practice: Roll one die a few times each evening and race to say the number before counting the dots, and whoever spots it first wins that round.

Detail: Subitizing and quantity sense: dots, dice and ten-frames, recognising amounts without counting

Numbers Are Made of Smaller Numbers

Topics:

- Breaking a small number into two parts in different ways
- Seeing 7 as 5 and 2, 6 and 1, or 4 and 3
- Using fingers or counters to split a group across two hands
- Understanding that the parts always rebuild the whole

- Finding every way to make a chosen number
- Explaining a split out loud in the child's own words

Practice: Give your child 6 buttons and ask how many ways they can split them into two piles, writing each way down together like a treasure list.

Detail: Part-whole thinking: every number is made of other numbers (7 is 5-and-2, 6-and-1, 3-and-4)

Living on the Number Line

Topics:

- Placing numbers in order along a line
- Deciding which number is bigger from its position
- Measuring the distance or jump between two numbers
- Hopping forwards and backwards along the line
- Meeting the empty number line as a thinking tool
- Estimating where a number belongs before checking

Practice: Draw a long line on paper with 0 at one end and 10 at the other, then take turns pointing to where numbers like 7 or 3 should sit.

Detail: The number line as home: position, order, distance and the empty-number-line game

Months 2-3: The Bonds That Power Everything (Weeks 5-12)

The Pairs That Make Ten

Topics:

- Finding both parts that add to make 10
- Recognising 6 and 4, 7 and 3, 8 and 2 as ten-friends
- Using ten-frames to see the missing part
- Answering ten-bonds quickly through repeated play
- Spotting a ten-bond hidden inside a larger problem
- Building speed through fun, never flashcard pressure

Practice: Play make ten: you say a number under ten and your child shouts the partner that finishes the ten, then swap roles after five turns.

Detail: Bonds of 10 to automaticity, through games, never flashcard grind

Doubles and Their Neighbours

Topics:

- Learning the doubles up to 10 plus 10 as anchor facts
- Using a known double to solve a near-double (6 plus 7 is double 6 and one more)
- Adding 9 by adding 10 and stepping back one
- Adding 11 by adding 10 and stepping up one
- Seeing how one known fact unlocks several nearby ones
- Choosing the closest double to lean on

Practice: At bedtime, call out a double like double 4 for a quick answer, then a near-double like 4 plus 5, and let your child explain how the double helped.

Detail: Doubles and near-doubles as anchor facts; adding 9 and 11 by compensation preview

Making Ten to Add Bigger

Topics:

- Splitting the second number to fill up to ten first
- Turning 8 plus 5 into 10 plus 3
- Using ten-frames to see the bridge across ten
- Knowing how much a number needs to reach ten
- Adding the leftover part after reaching ten
- Choosing make-ten when numbers sit close to ten

Practice: With two ten-frames and some counters, set up sums like 8 plus 5 and let your child slide counters to fill the first ten, then read the answer.

Detail: Bridging through 10: the make-ten strategy that turns 8+5 into 10+3

Bonds to Twenty and Fact Families

Topics:

- Finding pairs that add to make 20
- Linking a ten-bond to its twenty partner
- Seeing a fact family: 13 plus 7, 7 plus 13, 20 minus 7, 20 minus 13
- Understanding addition and subtraction as two views of one picture
- Using a known adding fact to find its subtracting partner
- Building all four facts from one number trio

Practice: Pick three numbers that fit together, like 6, 4 and 10, and challenge your child to say all four facts they can make from them.

Detail: Bonds to 20 and fact families: addition and subtraction as one idea seen from two sides

Month 4: Subtraction as Distance + Stage Checkpoint (Weeks 13-16)

Two Ways to Subtract

Topics:

- Taking away by counting back from the larger number
- Finding the gap by counting up between the numbers
- Deciding which way is faster for a given problem
- Seeing subtraction as the distance between two numbers
- Using the number line to picture both methods
- Explaining which route was chosen and why

Practice: Ask a shopping question like you have 10 and a toy costs 8, how much change, and let your child try both counting back and counting up to compare.

Detail: Subtraction two ways: taking away versus finding the gap, and when each is

faster

The Shopkeeper's Counting-Up Trick

Topics:

- Counting up from the price to the money paid
- Using friendly stops like the next ten
- Adding the small jumps to find the change
- Seeing why counting up avoids hard take-away sums
- Playing shop with real or pretend coins
- Checking the change makes sense

Practice: Set up a tiny pretend shop with priced items and let your child be the shopkeeper, counting up from the price to give you the right change.

Detail: Counting up for change: the shopkeeper's method every adult secretly uses

Many Roads to the Same Answer

Topics:

- Solving one problem in more than one way
- Sharing a strategy out loud in a number talk
- Listening to a different route to the same total
- Comparing which method felt quickest
- Respecting that different minds pick different paths
- Naming the strategy used (make-ten, doubles, or counting up)

Practice: Give one sum like 7 plus 8 and ask your child for two different ways to solve it, then you share a third way you thought of.

Detail: Mixed strategy play: number talks where children compare routes to the same answer

Phase One Check Through Play

Topics:

- Revisiting bonds of 10 and 20 through games
- Choosing a strategy for each problem confidently
- Explaining the thinking behind an answer
- Teaching a strategy back to the mentor or a parent
- Celebrating the facts now known by heart
- Noticing which few facts still want more play

Practice: Let your child be the teacher for ten minutes and show you one number strategy they learned this phase, explaining each step.

Assessment: A relaxed review game covering number bonds, part-whole thinking and the early strategies, where the child solves a problem, explains it, and teaches a favourite method back, with no timing or pressure.

Detail: Stage mastery check through games: solve it, explain it, teach it back

----- PHASE 2: THE STRATEGY TOOLKIT (MONTHS 5-8) -----

The heart of mental maths: named strategies the child chooses between, make-ten, compensation, doubling and halving, plus multiplication met as structure and arrays rather than chanted tables. Choice is the skill; speed is the side effect.

Month 5: Two-Digit Courage (Weeks 17-20)

Adding Two-Digit Numbers in Jumps

Topics:

- Adding the tens first in one jump
- Then adding the leftover ones
- Using an empty number line to record the jumps
- Holding the running total in mind
- Solving sums like 34 plus 23 step by step
- Checking the answer feels about right

Practice: Call out a two-digit sum like 42 plus 30 during a car ride and ask your child to jump the tens out loud, then add any ones.

Detail: Adding two-digit numbers mentally: jumps of ten, then the leftover ones

The Compensation Superpower

Topics:

- Rounding a number up to a friendly ten
- Adjusting the other number to keep the total fair
- Turning 39 plus 26 into 40 plus 25
- Subtracting 19 by taking 20 and giving one back
- Understanding why the give-and-take keeps it equal
- Choosing compensation when a number sits near a ten

Practice: Give a near-ten sum like 29 plus 14 and let your child bump the 29 up to 30, solve it, then give the one back at the end.

Detail: Compensation as a superpower: 39 + 26 becomes 40 + 25; subtracting 19 by taking 20 and giving one back

Month 6: Multiplication as Structure (Weeks 21-24)

Multiplication as Groups You Can See

Topics:

- Making equal groups with objects
- Building rows and columns as an array
- Reading 3 rows of 4 as three fours
- Connecting repeated addition to multiplication
- Seeing that an array can be counted two ways

- Using the words times and groups of

Practice: Arrange snacks or blocks into equal rows, like 3 rows of 5, and ask your child to find the total by seeing the groups rather than counting one by one.

Detail: Multiplication as groups and arrays, seen and built before recited

The Clever Route Through the Tables

Topics:

- Learning the easy 2s, 5s and 10s first
- Doubling the 2s to reach the 4s
- Doubling the 4s to reach the 8s
- Finding 9s by using 10s and stepping back a group
- Seeing how known tables build the harder ones
- Using skip counting as a rhythm for each table

Practice: Practise one table a day by skip counting it like a chant on the walk to the door, starting with the easy 2s, 5s or 10s.

Detail: The clever tables route: 2s, 10s and 5s first, then doubling to 4s and 8s; 9s by compensation

Month 7: The Full Tables, Owned (Weeks 25-28)

Taming the Trickiest Facts

Topics:

- Learning the 3s by skip counting and grouping
- Reaching the 6s by doubling the 3s
- Meeting the square facts like 6 times 6 and 7 times 7
- Spotting the small handful of facts that feel hardest
- Using a known nearby fact to reach a tricky one
- Practising just the stubborn facts, not the whole grid

Practice: Pick the one fact your child finds hardest this week and turn it into a silly rhyme or picture together to make it stick.

Detail: 3s, 6s and the tricky squares; the handful of hard facts isolated and conquered

Division Is Multiplication Backwards

Topics:

- Seeing division as sharing into equal groups
- Linking 3 times 4 is 12 to 12 shared by 4 is 3
- Building the multiplication and division fact family
- Using a known times fact to answer a divide fact
- Playing fact-family games for speed
- Understanding remainders as leftovers, gently

Practice: Share a small pile of objects equally between two or three plates and ask your child which times fact just proved the answer.

Detail: Division as the other side of multiplication; fact-family fluency through games

Month 8: Doubling, Halving and Strategy Choice (Weeks 29-32)

Doubling and Halving Magic

Topics:

- Halving one number while doubling the other
- Turning 16 times 5 into 8 times 10
- Seeing why the answer stays the same
- Choosing which number to halve to make it easy
- Using this to multiply by 5 quickly
- Chaining a halve and a double smoothly

Practice: Offer a times-five problem like 14 times 5 and let your child halve the 14 to 7 and double the 5 to 10, then read off 7 times 10.

Detail: Doubling and halving chains: 16×5 as 8×10 , the strategy that feels like magic and is pure structure

Strategy Tournament and Phase Two Check

Topics:

- Solving one problem several different ways
- Deciding which strategy suits which problem
- Explaining why a chosen route was quickest
- Reviewing make-ten, compensation and doubling-halving
- Owning the full multiplication and division facts
- Thinking about one's own thinking

Practice: Hold a friendly strategy tournament: give one problem and see how many different correct routes your child can find and name.

Assessment: A relaxed review game across the strategy toolkit, where the child picks and explains a strategy for each problem and teaches a favourite one back, with no timing pressure.

Detail: Strategy tournaments: same problem, different routes, and the metacognition of choosing, plus stage mastery check

PHASE 3: BIG NUMBERS, ESTIMATION AND DURABLE SPEED (MONTHS 9-12)

The toolkit scales up: multi-digit mental arithmetic, the beginnings of fraction sense, estimation as judgment, and, now that understanding holds, genuine speed work. The course closes with each child measurably faster and, more importantly, able to explain every move.

Month 9: Multi-Digit Mental Arithmetic (Weeks 33-36)

Big Numbers Without Fear

Topics:

- Jumping by hundreds and tens using place value
- Adding to a friendly round number first
- Using compensation on larger numbers
- Adding numbers like 240 plus 130 in the head
- Keeping the place values lined up while jumping
- Checking the size of the answer makes sense

Practice: During a walk, add house or page numbers like 120 plus 60, encouraging your child to jump the hundreds and tens out loud.

Detail: Hundreds without fear: place-value jumps, friendly numbers and compensation at scale

Multiplying Beyond the Tables

Topics:

- Splitting a two-digit number into tens and ones
- Multiplying each part then adding (23 times 4 is 20 times 4 plus 3 times 4)
- Using halving and doubling on bigger products
- Choosing the neatest way to split a number
- Holding partial answers in mind to combine
- Estimating first to check the result

Practice: Give a problem like 21 times 3 and let your child split it into 20 times 3 and 1 times 3, then add the two parts.

Detail: Mental multiplication beyond tables: 23 x 4 by splitting, 15 x 6 by halving-doubling

Month 10: Estimation as Judgment (Weeks 37-40)

Rounding to Get Close Fast

Topics:

- Rounding a number to the nearest ten or hundred
- Using rounded numbers for a quick estimate
- Deciding how close an estimate needs to be
- Comparing an estimate to the real answer
- Rounding both numbers before adding
- Understanding when an estimate is enough

Practice: At the shop, ask your child to round a couple of prices and guess the rough total before you reach the till, then compare with the real total.

Detail: Rounding with purpose: getting close fast, and knowing how close is close enough

Does This Answer Make Sense?

Topics:

- Checking whether an answer looks reasonable
- Using an estimate to catch a big mistake
- Spotting answers that are far too big or too small
- Asking about how much this should be first
- Trusting number sense over a typed answer
- Explaining why an answer cannot be right

Practice: Say a deliberately silly answer to an easy sum, like 3 plus 4 is 30, and let your child catch it and explain why it is wrong.

Detail: The reasonableness habit: catching wrong answers by sense, the skill calculators cannot replace

Month 11: Fraction Sense Beginnings (Weeks 41-44)

Halves and Quarters of Things

Topics:

- Splitting a shape or group into equal parts
- Finding half of a quantity, like half of 12
- Finding a quarter by halving twice
- Seeing halves and quarters in real objects
- Understanding fair, equal shares
- Reading a simple fraction from a picture

Practice: Share a snack fairly and ask your child for half, then a quarter, of a small group of items, checking the shares are equal.

Detail: Halves, quarters and the fraction-of-a-quantity game, mental fractions from pictures

Half, Point Five and Fifty Percent

Topics:

- Seeing one-half written as 0.5
- Meeting fifty percent as another name for half
- Linking a quarter to 0.25 and 25 percent
- Understanding these are the same amount in different clothes
- Spotting halves and percents in everyday life
- Reading these friendly numbers with confidence

Practice: Point out a half price or 50 percent off sign and ask your child what that means for a simple price, like half of 10.

Detail: Fraction and decimal friends: one-half is 0.5 is 50%, the bridge built early

Month 12: Speed With Understanding + Graduation (Weeks 45-52)

Beat Your Own Record

Topics:

- Using known strategies quickly and calmly

- Timing oneself gently to beat a personal best
- Tracking progress on a personal record ladder
- Choosing the fastest strategy for each fact
- Building speed on top of real understanding
- Racing only against yesterday's self, never a classmate

Practice: Set a friendly one-minute challenge of easy facts and let your child try to beat their own score from last time, celebrating any gain.

Detail: Timed play now that meaning holds: beat-your-own-record ladders, never child-versus-child pressure

Graduation and the Road Ahead

Topics:

- Choosing a favourite strategy to teach
- Running a number talk for the class or mentor
- Explaining each step clearly to others
- Reviewing all the strategies learned this year
- Celebrating durable, understood speed
- Looking ahead to the next maths adventure

Practice: Let your child teach the whole family their favourite strategy in a five-minute mini-lesson, using a real example.

Assessment: A gentle final review where the child leads a number talk teaching a chosen strategy, revisits the year's key ideas, and celebrates progress, with a friendly road-ahead plan instead of a test.

Detail: Graduation project: each child runs a number talk, teaching their favourite strategy, plus final stage mastery check and the road-ahead plan

PROJECTS THE STUDENT BUILDS AND KEEPS

- The child's own strategy book: every named strategy with their worked example
- Number-talk leadership: each child regularly explains routes to the class or mentor
- Beat-your-own-record fluency ladders, tracked across the year
- Estimation journal: real-life guesses checked and calibrated
- Graduation number talk taught by the child

Total projects: A year-long strategy book plus 4+ fluency artifacts

SKILLS THE STUDENT WILL HAVE AT THE END

- Number bonds to 20 at automaticity
- Make-ten, compensation and bridging strategies
- Doubling-halving and the clever tables route
- Full multiplication and division fact fluency
- Multi-digit mental arithmetic
- Estimation and reasonableness judgment
- Early fraction and decimal sense
- Explaining mathematical thinking aloud

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes, built with variety for young attention

Daily play: About 10 minutes of assigned games or number talks

Family involvement: Simple number-talk prompts parents can run at dinner

Total per week: About 3 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week

Mentorship: One dedicated mentor who knows the child's exact fact map

Recordings: Every class recorded

Progress tracking: Stage mastery checks shared with parents

Flexibility: Slots move around school and family plans

CERTIFICATION DETAIL

Phase certificates: Stage mastery check at the end of each phase

Final certificate: Course completion certificate from Modern Age Coders

Portfolio projects: The child's own strategy book

WHERE THIS CAN LEAD

- Ready for the Elementary Mathematics Masterclass, arriving ahead
- The fluency base that makes fractions and early algebra feel light
- Competition-maths readiness for children who catch fire
- A natural bridge into maths-through-coding for the builder types

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How is this different from abacus or Vedic maths classes?

A: It rarely transfers to fractions, word problems or algebra, and it fades when practice stops. We build number sense instead: strategies the child understands and chooses between. The speed arrives a little later and then never leaves, because it is made of understanding.

Q: What age is right to start?

A: Ages 5-10, and the mentor meets each child at their actual stage rather than their age. Five-year-olds start with quantity games and bonds; nine-year-olds might start at the strategy toolkit. The playful diagnostic in the free demo places every child accurately.

Q: Will my child actually enjoy this, or is it drilling in disguise?

A: Enjoyment is the engine of the whole method: number talks, strategy tournaments, beat-your-own-record ladders and games carry every class. Parents consistently tell us their children wait for these classes, which sounds impossible for arithmetic until you watch one. That is what the free demo is for.

Q: My child counts on fingers past age 7. Is that a problem this fixes?

A: Yes, and gently. Finger counting past grade 2 means number bonds never became automatic, so every calculation starts from scratch. We rebuild the bonds through games until known facts replace counting, without ever shaming the fingers on the way.

Q: Does this replace school maths or run alongside it?

A: Alongside, and underneath: number sense is the foundation school maths quietly assumes. Children in this course typically find school arithmetic suddenly easy, which buys the confidence that changes everything else.

Q: How much practice is needed between classes?

A: About ten minutes a day of assigned play, a game, a strategy challenge, a number talk with a parent. Small daily doses beat weekend marathons for fluency building, and ten minutes keeps it joyful.

Q: Can we try it before paying?

A: Yes. Every child starts with a free live demo class that doubles as the playful diagnostic, no card details, no obligation.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/mental-maths-mastery-kids>

=====

COURSE 86 OF 118: MERN STACK COURSE ONLINE: JOB-READY FULL-STACK BOOTCAMP

=====

Course link: <https://learn.modernagecoders.com/courses/mern-stack-development-masterclass-college>
Subject area: Full Stack Web Development
Designed for: College students
Who can join: Complete Beginner to Full Stack Expert
Course length: 12 months (52 weeks)
Time needed: 20-25 hours/week recommended
Certificate: Professional MERN Stack Developer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live MERN Stack course online: MongoDB, Express, React and Node.js in a 12-month bootcamp that builds 6 production-ready portfolio projects. Free demo class.

In one line: From HTML Basics to Production-Ready Full Stack Applications

ABOUT THE PROGRAMME

This comprehensive 1-year program transforms complete beginners into professional MERN stack developers capable of building, deploying, and maintaining complex full-stack applications. Master the entire JavaScript ecosystem from frontend to backend. You'll build 40+ projects including e-commerce platforms, social networks, real-time chat applications, and SaaS products. Learn modern development practices, DevOps, cloud deployment, and prepare for senior full-stack developer roles at top tech companies.

WHAT MAKES THIS COURSE DIFFERENT

- Complete JavaScript ecosystem mastery
- 40+ real-world projects from scratch
- Production deployment for every project
- Modern tools: TypeScript, Next.js, GraphQL
- Microservices and scalable architecture
- Mobile development with React Native
- DevOps and CI/CD pipeline creation

- Job-ready portfolio with live applications

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): HTML, CSS, JavaScript, Git, Responsive Design

Phase 2: Frontend Mastery (Months 4-6): React.js, Redux, Next.js, Advanced UI

Phase 3: Backend Development (Months 7-9): Node.js, Express, MongoDB, APIs

Phase 4: Full Stack & Production (Months 10-12): Integration, DevOps, Deployment, Advanced Topics

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No degree required - open to all

Technical skills: Basic computer knowledge

Equipment: Computer with 8GB RAM minimum

Internet: Stable internet connection

Time commitment: 20-25 hours per week

English: Basic English comprehension

Motivation: Passion for web development

WHO THIS COURSE IS FOR

Students: CS students wanting practical skills

Career switchers: Professionals changing to tech

Developers: Backend/Frontend wanting full stack

Entrepreneurs: Founders wanting to build products

Freelancers: Wanting high-paying skills

Designers: UI/UX designers wanting to code

Anyone: Passionate about building web applications

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: WEB DEVELOPMENT FOUNDATION (MONTHS 1-3, WEEKS 1-13) -----

Build solid foundations in HTML, CSS, JavaScript, and modern web development tools.

Month 1: HTML, CSS & Web Basics (Week 1-4)

Introduction to Web Development

Topics:

- How the internet works: clients, servers, protocols
- Web development career paths and opportunities
- Development environment setup: VS Code, extensions
- HTML5 fundamentals and document structure
- Semantic HTML elements and accessibility
- Forms and input validation
- Media elements: images, audio, video
- Tables and data representation
- SEO basics and meta tags
- HTML best practices and standards
- Introduction to developer tools
- Version control with Git and GitHub

Projects built:

- Personal portfolio HTML structure
- Multi-page website
- Contact form with validation

Practice: Build 10 HTML pages from scratch

Tools:

- VS Code setup
- Live Server extension
- Git initialization

CSS Fundamentals & Styling

Topics:

- CSS syntax and selectors hierarchy
- Box model: margin, padding, border
- Colors, typography, and units
- Flexbox layout system complete
- CSS Grid layout mastery
- Positioning: relative, absolute, fixed, sticky
- Transforms, transitions, and animations
- Pseudo-classes and pseudo-elements
- CSS variables and custom properties
- Mobile-first responsive design
- Media queries and breakpoints
- CSS preprocessors introduction (Sass)

Projects built:

- Responsive portfolio design

- Landing page with animations
- CSS art and illustrations

Practice: Style 15 different layouts

Tools:

- Chrome DevTools
- Sass compiler
- CSS frameworks overview

Advanced CSS & Modern Techniques

Topics:

- Advanced Flexbox patterns
- Complex Grid layouts
- CSS architecture: BEM, SMACSS
- CSS modules and scoping
- Performance optimization techniques
- Critical CSS and lazy loading
- CSS-in-JS introduction
- Tailwind CSS fundamentals
- Bootstrap 5 framework
- Material Design principles
- Accessibility (a11y) best practices
- Cross-browser compatibility

Projects built:

- Company website with Tailwind
- Dashboard UI with Bootstrap
- Accessible web application

Practice: Create 10 responsive components

Tools:

- Tailwind CSS setup
- PostCSS configuration
- Autoprefixer

JavaScript Fundamentals

Topics:

- JavaScript basics: variables, data types, operators
- Control flow: conditionals, loops, switch
- Functions: declaration, expression, arrow functions
- Scope, hoisting, and closures
- Objects and arrays manipulation
- Array methods: map, filter, reduce, forEach
- String methods and regular expressions
- Error handling: try, catch, finally
- ES6+ features: destructuring, spread, rest
- Template literals and tagged templates
- Modules: import/export patterns
- Debugging techniques and tools

Projects built:

- Interactive calculator
- Todo list application
- Quiz application

Practice: Solve 50 JavaScript challenges

Tools:

- Node.js installation
- npm basics
- ESLint setup

Month 2: Advanced JavaScript & DOM (Week 5-8)

DOM Manipulation & Events

Topics:

- Document Object Model (DOM) tree structure
- Selecting elements: querySelector, getElementById
- Creating and modifying elements
- Event handling and event listeners
- Event bubbling and delegation
- Form handling and validation
- Local storage and session storage
- Cookies and web storage API
- Drag and drop functionality
- File API and uploads
- Intersection Observer API
- Performance optimization in DOM

Projects built:

- Dynamic content management system
- Drag-and-drop task board
- Image gallery with lazy loading

Practice: Build 20 interactive features

Tools:

- Browser storage inspector
- Performance profiler
- Memory profiler

Asynchronous JavaScript

Topics:

- Callbacks and callback hell
- Promises: creation and chaining
- Async/await syntax and patterns
- Fetch API and HTTP requests
- REST API consumption
- Error handling in async code
- Promise.all, Promise.race, Promise.allSettled
- Web Workers for parallel processing
- Service Workers basics
- WebSockets introduction
- Server-Sent Events (SSE)
- Real-time data handling

Projects built:

- Weather application with API
- Real-time chat interface
- News aggregator

Practice: Integrate 10 different APIs

Tools:

- Postman for API testing
- JSON Server
- WebSocket clients

Object-Oriented & Functional Programming

Topics:

- OOP in JavaScript: classes, inheritance
- Prototypes and prototype chain
- Private and public fields
- Static methods and properties
- Design patterns: Singleton, Factory, Observer
- Functional programming concepts
- Pure functions and immutability
- Higher-order functions
- Currying and partial application
- Composition over inheritance
- SOLID principles in JavaScript
- Code organization and architecture

Projects built:

- Game development with OOP
- State management library
- Design pattern implementations

Practice: Implement 15 design patterns

Tools:

- TypeScript introduction
- Jest for testing
- Code coverage tools

Build Tools & Modern Workflow

Topics:

- Module bundlers: Webpack fundamentals
- Babel for transpilation
- Package management with npm/yarn
- Task runners and automation
- Environment variables and configuration
- Code splitting and lazy loading
- Hot module replacement
- Source maps for debugging
- Production optimization
- Progressive Web Apps basics
- Testing fundamentals: unit, integration
- Continuous Integration basics

Projects built:

- Custom webpack configuration
- PWA with offline support
- Automated build pipeline

Practice: Configure 5 different build setups

Tools:

- Webpack DevServer
- Babel presets
- npm scripts

Phase 1 Capstone Project

Topics:

- Project planning and wireframing
- Git workflow and branching
- Code review practices
- Documentation writing
- Deployment to Netlify/Vercel

Projects built:

- CAPSTONE: Full-Featured Web Application
- Complete responsive website with JavaScript functionality
- Include API integration, local storage, and animations

Assessment: Phase 1 Final Assessment

Month 3: Advanced Frontend & Introduction to React (Week 9-13)

Modern JavaScript & TypeScript

Topics:

- Advanced ES6+ features
- Symbols, iterators, and generators
- Proxy and Reflect APIs
- WeakMap and WeakSet
- TypeScript fundamentals
- Type annotations and interfaces
- Generics and type inference
- Enums and type aliases
- TypeScript with DOM
- Configuration and tsconfig.json
- TypeScript best practices
- Migration from JavaScript

Projects built:

- TypeScript utility library
- Type-safe API client
- JavaScript to TypeScript migration

Practice: Convert 10 projects to TypeScript

Tools:

- TypeScript compiler
- ts-node
- Type definition files

Introduction to React

Topics:

- React philosophy and virtual DOM
- Vite setup
- JSX syntax and expressions
- Components: functional vs class
- Props and prop types
- State management basics
- Event handling in React
- Conditional rendering
- Lists and keys
- Forms and controlled components
- Component lifecycle (class components)
- React DevTools usage

Projects built:

- React portfolio website
- Interactive todo application
- Recipe finder app

Practice: Build 20 React components

Tools:

- Vite
- React DevTools
- React playground

React Hooks & Modern Patterns

Topics:

- useState for state management
- useEffect for side effects
- useContext for global state
- useReducer for complex state
- useCallback and useMemo optimization
- useRef for DOM references
- Custom hooks creation
- Rules of hooks
- Hook patterns and best practices
- Error boundaries
- Suspense and lazy loading
- Concurrent features introduction

Projects built:

- Blog platform with hooks
- E-commerce product catalog
- Real-time dashboard

Practice: Create 15 custom hooks

Tools:

- React Hook Form
- SWR/React Query
- Hookstate

Component Libraries & Styling

Topics:

- CSS Modules in React
- Styled-components deep dive
- Emotion CSS-in-JS
- Material-UI components
- Ant Design integration

- Chakra UI setup
- Headless UI components
- Storybook for component development
- Component documentation
- Design systems creation
- Theme management
- Responsive design in React

Projects built:

- Custom component library
- Design system implementation
- Storybook showcase

Practice: Build 30 reusable components

Tools:

- Storybook setup
- Chromatic for visual testing
- Figma to React workflow

Phase 2 Assessment

Topics:

- React proficiency check
- Component architecture review
- Performance optimization
- Best practices audit
- Portfolio review

Projects built:

- MAJOR PROJECT: Social Media Dashboard
- Complete React application with routing, state management, and API integration
- Deploy to production

Assessment: Phase 2 Final Exam

----- PHASE 2: REACT MASTERY & ADVANCED FRONTEND (MONTHS 4-6, WEEKS 14-26) -----

Master React ecosystem, state management, advanced patterns, and modern frontend development.

Month 4: Advanced React & State Management (Week 14-18)

React Router & Navigation

Topics:

- React Router v6 setup and configuration
- Route definition and nesting
- Dynamic routing and parameters
- Protected routes and authentication
- Programmatic navigation
- Route guards and middleware
- Lazy loading routes
- 404 handling and redirects
- Query parameters and state
- Animated route transitions
- Breadcrumbs implementation
- Deep linking strategies

Projects built:

- Multi-page application router
- Admin dashboard with auth
- E-commerce site navigation

Practice: Implement 10 routing patterns

Tools:

- React Router DevTools
- Route testing utilities
- History API

State Management with Redux

Topics:

- Redux principles and architecture
- Actions, reducers, and store
- Redux Toolkit modern approach
- Slices and createSlice
- Async actions with createAsyncThunk
- RTK Query for data fetching
- Redux DevTools and debugging
- Middleware creation and usage
- Redux persist for local storage
- Normalization and selectors
- Performance optimization
- Redux best practices

Projects built:

- Shopping cart with Redux
- Task management system
- Real-time notifications

Practice: Build 5 Redux applications

Tools:

- Redux Toolkit

- Redux DevTools Extension
- Immer for immutability

Alternative State Management

Topics:

- Context API advanced patterns
- Zustand state management
- MobX reactive programming
- Recoil atomic state
- Valtio proxy state
- Jotai atomic approach
- State machines with XState
- Form state with React Hook Form
- Server state with React Query
- Local vs global state strategies
- State management comparison
- Choosing the right solution

Projects built:

- App with multiple state solutions
- Form-heavy application
- Real-time collaborative app

Practice: Compare 5 state solutions

Tools:

- Zustand DevTools
- React Query DevTools
- XState Visualizer

Performance Optimization

Topics:

- React performance profiling
- React.memo and PureComponent
- useMemo and useCallback optimization
- Code splitting strategies
- Lazy loading components and routes
- Bundle size optimization
- Virtual scrolling implementation
- Image optimization techniques
- Web Vitals and metrics
- Lighthouse audits
- Performance monitoring
- Production optimization

Projects built:

- Performance-optimized app
- Large list virtualization
- Image-heavy gallery

Practice: Optimize 10 slow applications

Tools:

- React Profiler
- Bundle analyzer
- Lighthouse CI

Testing React Applications

Topics:

- Testing philosophy and strategies
- Jest configuration and setup
- React Testing Library
- Component unit testing
- Integration testing
- Mocking and stubbing
- Testing hooks and context
- Async testing patterns
- Snapshot testing
- End-to-end with Cypress
- Test coverage reports
- CI/CD integration

Projects built:

- Full test suite for app
- TDD React application
- E2E test scenarios

Practice: Write 100+ test cases

Tools:

- Jest
- React Testing Library
- Cypress

Month 5: Next.js & Server-Side Rendering (Week 19-22)

Next.js Fundamentals

Topics:

- Next.js architecture and benefits
- File-based routing system
- Pages and layouts
- Static generation (SSG)

- Server-side rendering (SSR)
- Incremental Static Regeneration (ISR)
- API routes creation
- Dynamic routes and params
- Image optimization with next/image
- Font optimization
- SEO optimization
- Internationalization (i18n)

Projects built:

- Blog with Next.js
- E-commerce frontend
- Portfolio with CMS

Practice: Build 5 Next.js applications

Tools:

- Next.js CLI
- Vercel deployment
- Next.js DevTools

Advanced Next.js Features

Topics:

- App Router and React Server Components
- Streaming and Suspense
- Parallel and intercepting routes
- Route handlers and middleware
- Authentication in Next.js
- Database integration
- Caching strategies
- Edge functions and runtime
- Serverless functions
- Static export
- Performance optimization
- Production deployment

Projects built:

- Full-stack Next.js app
- SaaS application frontend
- Real-time application

Practice: Master 10 Next.js patterns

Tools:

- Next.js Analytics
- Edge Runtime
- Turbopack

GraphQL & Modern APIs

Topics:

- GraphQL fundamentals
- Queries, mutations, subscriptions
- Apollo Client setup
- GraphQL with React
- Code generation with GraphQL
- Optimistic updates
- Caching strategies
- Error handling
- File uploads with GraphQL
- Real-time subscriptions
- Federation and microservices
- REST vs GraphQL comparison

Projects built:

- GraphQL-powered app
- Real-time chat with subscriptions
- Apollo Client integration

Practice: Build 5 GraphQL integrations

Tools:

- Apollo Studio
- GraphQL Playground
- GraphQL Code Generator

Progressive Web Apps

Topics:

- PWA principles and benefits
- Service Workers deep dive
- Offline functionality
- Cache strategies
- Web App Manifest
- Install prompts and A2HS
- Push notifications
- Background sync
- App shell architecture
- Performance patterns
- PWA testing and debugging
- App store deployment

Projects built:

- Offline-first PWA

- News app with notifications
- PWA e-commerce site

Practice: Convert 5 apps to PWAs

Tools:

- Workbox
- PWA Builder
- Lighthouse PWA audit

Mobile Development with React Native

Topics:

- React Native setup and basics
- Core components and APIs
- Styling in React Native
- Navigation with React Navigation
- Platform-specific code
- Native modules integration
- Expo framework
- Device features access
- Push notifications mobile
- App store preparation
- Code sharing with web
- Performance optimization mobile

Projects built:

- Mobile app version
- Cross-platform application
- Native feature integration

Practice: Build 3 mobile apps

Tools:

- Expo CLI
- React Native Debugger
- Flipper

Month 6: Full Frontend Integration (Week 23-26)

Micro-Frontends & Architecture

Topics:

- Micro-frontend architecture
- Module federation with Webpack
- Single-spa framework
- Micro-frontend communication
- Shared dependencies management
- Independent deployment
- Testing strategies
- Version management
- Performance considerations
- Security in micro-frontends
- Monorepo management
- Team collaboration patterns

Projects built:

- Micro-frontend platform
- Federated modules app
- Enterprise architecture

Practice: Implement 3 micro-frontend patterns

Tools:

- Lerna/Nx
- Module Federation
- Turborepo

Advanced Animation & Graphics

Topics:

- Framer Motion deep dive
- Complex animations and gestures
- SVG animations
- Canvas API with React
- WebGL and Three.js basics
- React Three Fiber
- Data visualization with D3.js
- Chart libraries integration
- Parallax scrolling effects
- Scroll-triggered animations
- Performance optimization
- 60fps animations

Projects built:

- Animated landing page
- Data visualization dashboard
- 3D product showcase

Practice: Create 20 animations

Tools:

- Framer Motion
- GreenSock (GSAP)
- Lottie React

Accessibility & Internationalization

Topics:

- WCAG guidelines and compliance
- Semantic HTML in React
- ARIA attributes and roles
- Keyboard navigation
- Screen reader testing
- Color contrast and visibility
- Focus management
- Accessible forms and errors
- i18n setup with React
- Translation management
- RTL language support
- Date and number formatting

Projects built:

- Fully accessible application
- Multi-language platform
- WCAG compliant site

Practice: Audit and fix 10 applications

Tools:

- axe DevTools
- NVDA/JAWS
- react-i18next

Security & Best Practices

Topics:

- Frontend security principles
- XSS prevention in React
- CSRF protection
- Content Security Policy
- Secure authentication flows
- OAuth and social login
- JWT handling and storage
- Input validation and sanitization
- Secure API communication
- HTTPS and SSL
- Security headers
- Dependency vulnerability scanning

Projects built:

- Secure authentication system
- Security audit implementation
- Penetration testing prep

Practice: Secure 5 applications

Tools:

- OWASP ZAP
- Snyk
- npm audit

Phase 2 Capstone

Topics:

- Project architecture planning
- Technology stack finalization
- Performance requirements
- Deployment strategy
- Documentation preparation

Projects built:

- CAPSTONE: Production-Ready Frontend Application
- Complete React/Next.js application with all advanced features
- Include authentication, real-time features, and PWA capabilities

Assessment: Phase 2 Final Assessment - Frontend Mastery

PHASE 3: BACKEND DEVELOPMENT WITH NODE.JS (MONTHS 7-9, WEEKS 27-39)

Master Node.js, Express, MongoDB, and build production-ready backend systems.

Month 7: Node.js & Express Fundamentals (Week 27-31)

Node.js Fundamentals

Topics:

- Node.js architecture and event loop
- Global objects and process
- Modules system: CommonJS and ES modules
- File system operations
- Path and OS modules
- Streams and buffers
- Events and EventEmitter
- Child processes
- Cluster module for scaling
- Worker threads
- Error handling in Node.js
- Debugging Node applications

Projects built:

- CLI tool development
- File processing system
- Task automation scripts

Practice: Build 15 Node.js utilities

Tools:

- Node.js debugger
- nodemon
- PM2

Express.js Framework

Topics:

- Express application setup
- Routing and route parameters
- Middleware concept and creation
- Request and response handling
- Static file serving
- Template engines integration
- Body parsing and file uploads
- Cookie and session handling
- Error handling middleware
- Security middleware setup
- Compression and optimization
- Express best practices

Projects built:

- RESTful API server
- Blog backend system
- File upload service

Practice: Create 10 Express applications

Tools:

- Postman
- Thunder Client
- Morgan logger

Database Design & MongoDB

Topics:

- Database fundamentals and types
- MongoDB architecture and concepts
- Document-based data modeling
- CRUD operations in MongoDB
- Indexes and performance
- Aggregation framework
- Transactions in MongoDB
- Schema design patterns
- References vs embedding
- Data validation and migration
- Backup and restore strategies
- MongoDB Atlas cloud setup

Projects built:

- Database schema design
- MongoDB aggregation pipelines
- Data migration system

Practice: Design 10 database schemas

Tools:

- MongoDB Compass
- Atlas cloud
- Robo 3T

Mongoose ODM & Data Modeling

Topics:

- Mongoose setup and connection
- Schema definition and types
- Model creation and methods
- Validation and custom validators
- Middleware: pre and post hooks
- Virtual properties
- Population and references
- Query building and optimization
- Plugins and reusable logic
- Transactions with Mongoose
- Error handling patterns
- Testing database operations

Projects built:

- User management system
- E-commerce data models
- Social media schemas

Practice: Create 15 Mongoose models

Tools:

- Mongoose documentation
- Faker.js for seeding
- MongoDB Memory Server

RESTful API Design

Topics:

- REST principles and constraints
- API versioning strategies
- Resource naming conventions
- HTTP methods and status codes
- Request/response format
- Pagination and filtering
- Sorting and field selection
- API documentation with Swagger
- Postman collection creation
- API testing strategies
- Rate limiting implementation
- API security best practices

Projects built:

- Complete REST API
- API documentation
- API testing suite

Practice: Build 5 different APIs

Tools:

- Swagger/OpenAPI
- Postman collections
- Newman CLI

Month 8: Authentication & Advanced Backend (Week 32-35)

Authentication & Authorization

Topics:

- Authentication vs authorization
- Password hashing with bcrypt
- JWT token generation and validation
- Refresh token implementation
- Session-based authentication
- OAuth 2.0 implementation
- Social login integration
- Two-factor authentication
- Role-based access control (RBAC)
- API key authentication
- Security best practices
- CSRF and XSS protection

Projects built:

- Complete auth system
- OAuth integration
- Multi-factor authentication

Practice: Implement 5 auth strategies

Tools:

- Passport.js
- JWT debugger
- OAuth playground

Real-time Communication

Topics:

- WebSocket protocol understanding
- Socket.io setup and configuration
- Real-time events and broadcasting
- Rooms and namespaces
- Private messaging implementation
- Presence detection
- Reconnection handling
- Scaling WebSocket servers
- Redis adapter for Socket.io
- Server-Sent Events (SSE)
- WebRTC basics
- Real-time notifications

Projects built:

- Chat application
- Live collaboration tool
- Real-time dashboard

Practice: Build 5 real-time features

Tools:

- Socket.io client/server
- Redis pub/sub
- WebRTC libraries

File Handling & Cloud Storage

Topics:

- File upload strategies
- Multer configuration and usage
- Image processing with Sharp
- Video processing basics
- AWS S3 integration
- Cloudinary integration
- File validation and security
- Streaming large files

- ZIP file generation
- CSV and Excel processing
- PDF generation
- CDN integration

Projects built:

- Media upload system
- Cloud storage integration
- Document processing service

Practice: Handle 10 file types

Tools:

- Multer
- Sharp
- AWS SDK

Email & Notification Systems

Topics:

- Email service setup
- SMTP configuration
- SendGrid/Mailgun integration
- HTML email templates
- Email queue implementation
- Bounce handling
- Push notifications setup
- SMS integration with Twilio
- In-app notifications
- Notification preferences
- Unsubscribe management
- Email analytics

Projects built:

- Email service
- Notification system
- Marketing automation

Practice: Implement 5 notification channels

Tools:

- Nodemailer
- SendGrid
- Bull queue

Caching & Performance

Topics:

- Caching strategies and patterns
- Redis setup and operations
- Memory caching with Node.js
- Database query optimization
- API response caching
- CDN caching strategies
- Session store with Redis
- Rate limiting with Redis
- Queue implementation
- Performance monitoring
- Load testing
- Optimization techniques

Projects built:

- Caching layer implementation
- Performance optimization
- Load testing suite

Practice: Optimize 5 slow endpoints

Tools:

- Redis
- Artillery
- New Relic

Month 9: Advanced Backend & DevOps (Week 36-39)

Microservices Architecture

Topics:

- Microservices principles
- Service decomposition
- Inter-service communication
- API Gateway pattern
- Service discovery
- Circuit breakers
- Event-driven architecture
- Message queues: RabbitMQ, Kafka
- Saga pattern
- Distributed tracing
- Service mesh basics
- Container orchestration intro

Projects built:

- Microservices application
- API Gateway
- Event-driven system

Practice: Build 5 microservices

Tools:

- Docker
- RabbitMQ
- Consul

GraphQL Backend

Topics:

- GraphQL server setup
- Schema definition
- Resolvers implementation
- Mutations and queries
- Subscriptions setup
- DataLoader for N+1 problem
- Authentication in GraphQL
- File uploads in GraphQL
- Error handling
- Testing GraphQL APIs
- Federation setup
- Performance optimization

Projects built:

- GraphQL API server
- Subscription service
- Federated GraphQL

Practice: Build 3 GraphQL APIs

Tools:

- Apollo Server
- GraphQL Playground
- DataLoader

Testing & Quality Assurance

Topics:

- Testing strategies for Node.js
- Unit testing with Jest
- Integration testing
- API testing with Supertest
- Database testing strategies
- Mocking and stubbing
- Test coverage reports
- End-to-end testing
- Load testing
- Security testing
- Continuous testing
- TDD and BDD approaches

Projects built:

- Complete test suite
- CI/CD pipeline with tests
- Load testing scenarios

Practice: Write 200+ tests

Tools:

- Jest
- Supertest
- Sinon

Security & Compliance

Topics:

- OWASP Top 10 for Node.js
- SQL injection prevention
- NoSQL injection prevention
- Input validation and sanitization
- Rate limiting strategies
- DDoS protection
- Helmet.js configuration
- CORS setup and security
- Environment variables security
- Secrets management
- GDPR compliance
- PCI DSS basics

Projects built:

- Security audit
- Compliance implementation
- Penetration testing prep

Practice: Secure 5 applications

Tools:

- Helmet.js
- Express-validator
- OWASP ZAP

Phase 3 Capstone

Topics:

- Backend architecture design
- API documentation
- Database optimization

- Security implementation
- Performance testing

Projects built:

- CAPSTONE: Complete Backend System
- Production-ready API with all features
- Include auth, real-time, caching, and microservices

Assessment: Phase 3 Final Assessment - Backend Mastery

PHASE 4: FULL STACK INTEGRATION & PRODUCTION (MONTHS 10-12, WEEKS 40-52)

Integrate frontend and backend, master DevOps, deployment, and build production applications.

Month 10: Full Stack Integration (Week 40-44)

Full Stack Architecture

Topics:

- Full stack application architecture
- Frontend-backend communication
- API integration best practices
- State management with backend
- Authentication flow implementation
- File upload full stack
- Real-time features integration
- Error handling across stack
- Logging and monitoring
- Performance optimization
- Security across layers
- Testing full stack apps

Projects built:

- Full stack blog platform
- E-commerce application
- Social media clone

Practice: Build 5 full stack features

Tools:

- Axios interceptors
- Proxy configuration
- CORS handling

Payment Integration

Topics:

- Payment gateway basics
- Stripe integration
- PayPal implementation
- Razorpay setup
- Webhook handling
- Subscription management
- Invoice generation
- Refund processing
- Payment security
- PCI compliance basics
- Testing payments
- International payments

Projects built:

- Payment system
- Subscription platform
- Marketplace payments

Practice: Implement 3 payment gateways

Tools:

- Stripe
- PayPal SDK
- Webhook testing

DevOps & CI/CD

Topics:

- DevOps principles and culture
- Git advanced workflows
- GitHub Actions setup
- CI/CD pipeline creation
- Automated testing in pipeline
- Code quality checks
- Docker containerization
- Docker Compose for development
- Container registries
- Environment management
- Secrets and configuration
- Monitoring and alerts

Projects built:

- CI/CD pipeline
- Dockerized application
- Automated deployment

Practice: Set up 5 CI/CD pipelines

Tools:

- GitHub Actions
- Docker
- Jenkins

Cloud Deployment

Topics:

- Cloud platforms overview
- AWS services for MERN
- EC2 instance setup
- S3 for static hosting
- RDS and DocumentDB
- Elastic Beanstalk
- Vercel deployment
- Netlify for frontend
- Render deployment
- Digital Ocean setup
- Domain and SSL setup
- CDN configuration

Projects built:

- AWS deployment
- Multi-cloud setup
- Production infrastructure

Practice: Deploy to 5 platforms

Tools:

- AWS Console
- Vercel CLI
- Terraform basics

Kubernetes & Scaling

Topics:

- Kubernetes concepts
- Pods, services, deployments
- Kubernetes for MERN apps
- Helm charts
- Scaling strategies
- Load balancing
- Auto-scaling setup
- Health checks
- Rolling updates
- Monitoring with Prometheus
- Logging with ELK stack
- Cost optimization

Projects built:

- Kubernetes deployment
- Auto-scaling setup
- Monitoring dashboard

Practice: Deploy 3 apps to K8s

Tools:

- kubectl
- Minikube
- Lens

Month 11: Advanced Topics & Optimization (Week 45-48)

Performance & Optimization

Topics:

- Performance metrics and monitoring
- Frontend optimization techniques
- Backend optimization strategies
- Database optimization
- Caching strategies implementation
- CDN optimization
- Image optimization pipeline
- Code splitting advanced
- Lazy loading strategies
- Server-side optimization
- Load balancing techniques
- Performance testing tools

Projects built:

- Performance optimization project
- Optimization toolkit
- Monitoring system

Practice: Optimize 10 applications

Tools:

- Lighthouse
- WebPageTest
- GTmetrix

Advanced Security

Topics:

- Security architecture design
- Threat modeling

- Penetration testing basics
- Security scanning automation
- Dependency vulnerability management
- Container security
- API security testing
- Security monitoring
- Incident response planning
- Compliance implementation
- Security documentation
- Bug bounty preparation

Projects built:

- Security implementation
- Vulnerability scanner
- Security dashboard

Practice: Secure 5 production apps

Tools:

- Snyk
- SonarQube
- Burp Suite

Analytics & Monitoring

Topics:

- Application monitoring setup
- Google Analytics integration
- Custom analytics implementation
- Error tracking with Sentry
- Log aggregation
- APM tools setup
- Custom metrics and dashboards
- User behavior analytics
- A/B testing implementation
- Performance monitoring
- Business metrics tracking
- Alerting strategies

Projects built:

- Analytics platform
- Monitoring dashboard
- A/B testing system

Practice: Implement 5 analytics solutions

Tools:

- Google Analytics
- Sentry
- DataDog

Machine Learning Integration

Topics:

- ML basics for web developers
- TensorFlow.js integration
- Python ML models with Node.js
- Recommendation systems
- Image recognition features
- Natural language processing
- Sentiment analysis
- Predictive features
- ML model deployment
- A/B testing ML features
- ML monitoring
- Ethical AI considerations

Projects built:

- ML-powered features
- Recommendation engine
- Smart search implementation

Practice: Add ML to 3 applications

Tools:

- TensorFlow.js
- Brain.js
- ML APIs

AI Integration

Topics:

- Adding AI to a MERN app without leaking keys
- Express routes that proxy a model provider
- Streaming responses (SSE) into React
- Structured output your frontend can trust
- Function calling to trigger real app actions
- Retrieval over your MongoDB data
- Vector search in MongoDB Atlas, and when plain queries win
- Caching AI responses, and when caching is wrong
- Rate limiting and per-user spend caps
- Validating model output before it hits the database
- Evaluating whether the feature actually helps
- Failure design: the provider will go down

Projects built:

- AI chat feature in MERN
- Retrieval Q&A over your data
- Agentic action endpoint

Practice: Build 3 AI features

Tools:

- Node/Express
- MongoDB Atlas Vector Search
- React streaming UI

Month 12: Career Preparation & Final Projects (Week 49-52)

Portfolio Development

Topics:

- Portfolio website creation
- Project documentation
- Case study writing
- GitHub profile optimization
- LinkedIn optimization
- Resume building for developers
- Cover letter templates
- Personal branding
- Blog writing
- Video demos creation
- Contributing to open source
- Building online presence

Projects built:

- Professional portfolio
- GitHub showcase
- Technical blog

Practice: Polish 10 projects

Tools:

- Portfolio builders
- Screen recording
- Markdown

Interview Preparation

Topics:

- Technical interview preparation
- System design interviews
- Behavioral questions
- MERN stack interview questions
- Live coding practice
- Take-home assignments
- Whiteboard problem solving
- Communication skills
- Salary negotiation
- Remote interview tips
- Company research
- Interview follow-up

Projects built:

- Interview prep kit
- Mock interview recordings
- Technical presentations

Practice: 50 mock interviews

Tools:

- Pramp
- LeetCode
- System design primer

Freelancing & Business

Topics:

- Freelancing as MERN developer
- Client acquisition
- Project estimation
- Contract negotiation
- Project management
- Client communication
- Invoice and payments
- Building recurring revenue
- Scaling freelance business
- Agency transformation
- Product development
- SaaS business basics

Projects built:

- Freelance portfolio
- Service packages
- SaaS MVP

Practice: Complete 3 client projects

Tools:

- Upwork/Fiverr
- Invoice tools

- Project management

Final Capstone Project

Topics:

- Project planning and scoping
- Architecture design
- Technology selection
- Development process
- Testing strategy
- Deployment pipeline
- Documentation
- Presentation preparation

Projects built:

- FINAL CAPSTONE: Production SaaS Application
- Complete MERN stack application with all features learned
- Deploy to production with full DevOps pipeline

Practice: Launch live application

Tools:

- All tools learned
- Production services
- Monitoring tools

Graduation & Beyond (Final Week & Resources)

Advanced Specializations

Topics:

- JAMstack architecture
- Serverless with MERN
- Edge computing
- WebAssembly integration
- IoT with Node.js
- AR/VR web experiences
- Voice interfaces
- Quantum computing basics
- 5G and web applications
- Green computing
- Sustainable web development
- Future technologies

Projects built:

- Cutting-edge project
- Research implementation
- Innovation prototype

Community & Networking

Topics:

- Developer communities
- Conference participation
- Meetup organization
- Open source contribution
- Mentoring others
- Building developer network
- Tech talks and presentations
- Podcast participation
- YouTube channel creation
- Technical writing
- Community leadership
- Giving back

Deliverables:

- Community participation plan
- Content calendar
- Network building strategy

Lifetime Learning Path

Topics:

- Continuous learning strategies
- Staying updated with tech
- Advanced certifications
- Specialization paths
- Career progression planning
- Leadership development
- Entrepreneurship path
- Consulting opportunities
- Teaching and mentoring
- Research and innovation
- Work-life balance
- Long-term success

Deliverables:

- 5-year career plan
- Learning roadmap
- Success metrics

Assessment: FINAL CERTIFICATION - Professional MERN Stack Developer

- Phase 1: Portfolio Website, Todo App, Weather App, Blog Frontend
- Phase 2: E-commerce Frontend, Social Media Dashboard, Real-time Chat UI, PWA
- Phase 3: REST API, Authentication System, Real-time Backend, Microservices
- Phase 4: Full Stack SaaS, Production Deployment, Complete Platform
- Final: 40+ Deployed Projects Portfolio

Total projects: 40+ production-ready applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Frontend: HTML5, CSS3, JavaScript ES6+, TypeScript
- React: Hooks, Redux, Next.js, React Native, Testing
- Backend: Node.js, Express.js, REST APIs, GraphQL
- Database: MongoDB, Mongoose, Redis, PostgreSQL basics
- DevOps: Docker, Kubernetes, CI/CD, Cloud Deployment
- Tools: Git, VS Code, Postman, Chrome DevTools, npm/yarn
- Testing: Jest, React Testing Library, Cypress, Supertest
- Security: Authentication, Authorization, OWASP, SSL
- Performance: Optimization, Caching, CDN, Monitoring
- Architecture: Microservices, Serverless, JAMstack

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 6-8 hours
 Hands on coding: 10-12 hours
 Project work: 4-6 hours
 Practice exercises: 3-4 hours
 Total per week: 20-25 hours

SUPPORT INCLUDED

Live sessions: Weekly doubt clearing and code reviews
 Mentorship: 1-on-1 guidance from senior developers
 Community: Active Discord/Slack community
 Code review: Professional code reviews on projects
 Career support: Resume review, mock interviews, job referrals
 Lifetime access: All content and future updates
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Final certificate: Professional MERN Stack Developer Certification
 Linkedin badge: Verifiable LinkedIn credential
 Github portfolio: 40+ deployed projects
 Industry recognition: Recognized by tech companies

WHERE THIS CAN LEAD

- Full Stack Developer
- MERN Stack Developer
- Frontend Developer (React)
- Backend Developer (Node.js)
- JavaScript Developer
- Web Application Developer
- Software Engineer
- Technical Lead
- Solutions Architect
- Freelance Developer
- Technical Co-founder
- DevOps Engineer

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-5 LPA (Junior Frontend)
 Growing roles: Rs 5-10 LPA (React Developer); Rs 8-15 LPA (Full Stack)
 Experienced roles: Rs 12-25+ LPA (Senior MERN)
 Freelance: Rs 1,000-5,000/hour
 International: \$60k-150k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is MERN stack and why is it popular for web development?

A: MERN stands for MongoDB, Express.js, React, and Node.js - a powerful JavaScript-based technology stack for building full-stack web applications. It's popular because developers use one language (JavaScript) across the entire stack, leading to faster development, easier hiring, and seamless data flow. Companies like Netflix, Uber, and Airbnb use MERN stack technologies.

Q: What's the difference between MERN stack and other stacks like LAMP or MEAN?

A: MERN uses React for frontend (component-based, virtual DOM), while MEAN uses Angular (full framework with TypeScript). LAMP uses PHP and MySQL - older technologies with different paradigms. MERN offers the best combination of modern features, community support, job opportunities, and performance. React alone has over 200,000 npm weekly downloads and dominates the frontend ecosystem.

Q: How long does it take to learn MERN stack development completely?

A: Our comprehensive 12-month program takes you from zero to production-ready MERN developer. You can start building simple applications within 3 months, become job-ready for junior positions by 6 months, and reach senior-level proficiency by 12 months. The course includes 40+ real projects, DevOps, testing, and even advanced topics like microservices and GraphQL.

Q: Do I need prior programming experience to start this MERN stack course?

A: No prior experience required! We start from HTML, CSS, and JavaScript basics, then progressively build to advanced React, Node.js, and MongoDB concepts. Whether you're a complete beginner, a designer learning to code, or switching careers from another field, this structured curriculum ensures you build a solid foundation before tackling advanced topics.

Q: What salary can I expect as a MERN stack developer in India?

A: MERN stack developers are highly in-demand with competitive salaries.

Q: What real-world projects will I build in this MERN stack course?

A: You'll build 40+ production-ready applications including: e-commerce platform with payment integration, social media clone with real-time features, project management tool (Trello clone), video streaming platform, job portal, restaurant ordering system, and a complete SaaS application as your final capstone. All projects are deployed to cloud platforms and added to your portfolio.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/mern-stack-development-masterclass-college>

=====

COURSE 87 OF 118: MICROSOFT OFFICE & WINDOWS FOR KIDS: DIGITAL SKILLS COURSE

=====

Course link: <https://learn.modernagecoders.com/courses/microsoft-office-kids-mastery>

Subject area: Digital Literacy & Productivity

Designed for: Kids

Who can join: Complete Beginner to Advanced User

Course length: 6 months (26 weeks)

Time needed: 4-6 hours/week recommended

Certificate: Microsoft Office Young Professional Certificate

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live 6-month Microsoft Office and Windows course for kids: master Word, PowerPoint, Excel, OneNote, Teams and 20+ apps, building real digital literacy skills.

In one line: From First Click to Digital Professional - Master Every Microsoft Tool

ABOUT THE PROGRAMME

This comprehensive 6-month program transforms complete beginners into confident Microsoft Office users and Windows experts. Perfect for students aged 7-18, this course covers everything from basic computer operations to advanced Office features. Students will master document creation in Word, stunning presentations in PowerPoint, data management in Excel, digital art in Paint, organization with OneNote, collaboration through Teams, and dozens of Windows tools. By completion, learners will have professional-level skills and an impressive portfolio of projects.

WHAT MAKES THIS COURSE DIFFERENT

- Kid-friendly approach to professional software
- Hands-on projects for every tool
- Real-world applications for school and life
- Progressive skill building from basics to advanced

- Creative projects mixed with practical skills
- Parent-friendly progress tracking
- Preparation for academic and career success
- Covers both Office 365 and Windows 11 tools

THE LEARNING PATH, STEP BY STEP

Month 1: Windows Basics & Digital Art with Paint
 Month 2: Microsoft Word Mastery
 Month 3: PowerPoint Presentation Excellence
 Month 4: Excel and Data Skills
 Month 5: OneNote, Teams & Collaboration Tools
 Month 6: Advanced Integration & Professional Projects

WHAT A STUDENT NEEDS BEFORE STARTING

Age: Ages 8-14
 Experience: No prior computer experience needed - start from basics
 Equipment: Windows computer or laptop with Microsoft Office installed
 Software: Microsoft Office (Word, PowerPoint, Excel) - free alternatives guidance provided
 Reading: Basic reading ability to follow instructions

WHO THIS COURSE IS FOR

School students: Kids aged 8-14 who need computer skills for school projects
 Digital beginners: Children with little or no computer experience
 Presentation skills: Students who want to create impressive school presentations
 Future professionals: Kids building early professional digital literacy skills
 Homeschoolers: Families wanting structured digital skills education

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- MONTH 1: WINDOWS BASICS & DIGITAL ART -----

Master Windows fundamentals, file management, and creative expression with Paint and Paint 3D.

Introduction to Windows & Computer Basics

Topics:

- Computer components: monitor, keyboard, mouse, CPU
- Turning computer on/off properly
- Windows 11 desktop overview
- Start menu navigation
- Taskbar features and customization
- Desktop icons and shortcuts
- Mouse skills: click, double-click, right-click, drag
- Keyboard basics: letters, numbers, special keys
- Caps Lock, Shift, Enter, Backspace, Delete
- Windows key shortcuts introduction
- Volume and brightness controls
- Connecting to WiFi networks

Projects built:

- Customize desktop wallpaper and theme
- Create desktop shortcuts
- Organize taskbar with favorite apps
- Personal desktop setup

Practice: Practice starting up and shutting down the computer the right way, then pin your three favorite apps to the taskbar and show a parent.

File Management & Organization

Topics:

- Understanding files and folders
- File Explorer navigation
- Creating new folders
- Naming conventions for files
- Moving and copying files
- Cut, copy, paste operations
- Deleting and Recycle Bin
- Searching for files
- File types and extensions
- Organizing Downloads folder
- Creating folder structures
- File properties and details
- Compressing files with ZIP
- OneDrive basics and cloud storage

Projects built:

- Create organized folder system for schoolwork
- Digital filing cabinet setup
- Photo organization project
- Backup important files

Practice: Create a folder tree for your school subjects and sort five files into the right folders using cut, copy and paste.

Microsoft Paint Fundamentals

Topics:

- Opening and navigating Paint
- Drawing tools: pencil, brushes, shapes
- Color palette and custom colors
- Fill tool and color picker
- Line and curve tools
- Shape tools: rectangle, circle, polygon
- Text tool and fonts
- Selection tools and moving objects
- Resize and rotate features
- Zoom in/out for detail work
- Saving in different formats (PNG, JPEG, BMP)
- Image size and canvas size
- Copying and pasting images
- Using Paint for school projects

Projects built:

- Create pixel art characters
- Design birthday card
- Draw house and landscape
- Make school poster
- Create emoji collection

Practice: Draw your dream treehouse in Paint using at least four different tools and the fill bucket.

Paint 3D & Advanced Digital Art

Topics:

- Introduction to Paint 3D interface
- 2D shapes in 3D space
- 3D shapes and models
- 3D doodle tool
- Stickers and textures
- Text effects in 3D
- Magic select tool for backgrounds
- 3D library and models
- Lighting and effects
- 3D rotation and positioning
- Creating 3D scenes
- Exporting 3D models
- Mixed reality viewer
- Combining 2D and 3D elements
- Animation basics in Paint 3D

Projects built:

- Design 3D room layout
- Create 3D character
- Build virtual diorama
- Make 3D greeting card
- Design custom 3D logo

Practice: Turn your name into 3D text in Paint 3D and decorate it with stickers and one texture.

----- MONTH 2: MICROSOFT WORD MASTERY -----

Become a Word expert, creating professional documents, reports, and creative writing projects.

Word Basics & Text Formatting

Topics:

- Opening Microsoft Word and interface tour
- Creating new documents
- Ribbon interface explanation
- Quick Access Toolbar customization
- Typing and text selection techniques
- Font types and sizes
- Bold, italic, underline formatting
- Text color and highlighting
- Alignment: left, center, right, justify
- Line spacing and paragraph spacing
- Bullets and numbering
- Indentation and tabs
- Copy, cut, paste operations

- Undo and redo features
- Saving documents and file formats

Projects built:

- Write and format a story
- Create class notes template
- Design invitation letter
- Format poem with styling

Practice: Type a short paragraph about your favorite animal in Word and format it with two fonts, one color and bold key words.

Advanced Formatting & Page Layout

Topics:

- Page setup: margins, orientation, size
- Headers and footers
- Page numbers and formatting
- Section breaks and page breaks
- Columns for newsletters
- Borders and shading
- Watermarks and page color
- Themes and style sets
- Creating and using styles
- Table of contents generation
- Cover pages and templates
- Drop caps and special effects
- Text boxes and positioning
- WordArt for titles
- Page borders and art

Projects built:

- Create school newsletter
- Design book report template
- Make professional letter
- Build recipe book layout

Practice: Make a one page mini newsletter about your week using two columns, a header and a page border.

Working with Images & Graphics

Topics:

- Inserting pictures from computer
- Online pictures and stock images
- Picture formatting and styles
- Cropping and resizing images
- Picture effects and corrections
- Text wrapping options
- SmartArt graphics creation
- Charts and graphs in Word
- Shapes and drawing tools
- Icons and 3D models
- Screenshots and screen clipping
- Arranging multiple objects
- Grouping and ungrouping
- Captions for images
- Compress pictures for file size

Projects built:

- Illustrated story book
- Science project report with diagrams
- Travel brochure with images
- How-to guide with screenshots

Practice: Insert three pictures into a 'My Hobbies' document, then crop one, style another and wrap text around the third.

Tables, References & Collaboration

Topics:

- Creating tables from scratch
- Table styles and formatting
- Adding/deleting rows and columns
- Merging and splitting cells
- Table calculations and formulas
- Converting text to tables
- Footnotes and endnotes
- Citations and bibliography
- Hyperlinks and bookmarks
- Track changes feature
- Comments and reviewing
- Comparing documents
- Sharing and co-authoring
- Protecting documents
- Printing options and preview

Projects built:

- Class schedule table

- Research paper with citations
- Collaborative story writing
- Data comparison report

Practice: Build your weekly class timetable as a Word table and give the header row its own color.

MONTH 3: POWERPOINT PRESENTATION EXCELLENCE

Master the art of creating engaging, professional presentations with PowerPoint.

PowerPoint Fundamentals

Topics:

- PowerPoint interface overview
- Slide layouts and placeholders
- Adding new slides
- Slide themes and variants
- Text formatting in slides
- Bullet points and numbering
- Slide backgrounds and fills
- Color schemes and fonts
- Slide master basics
- Notes and comments
- Outline view for planning
- Slide sorter view
- Saving presentations
- PowerPoint file formats
- Presenter view introduction

Projects built:

- About Me presentation
- Book report slideshow
- Science topic presentation
- Family photo album

Practice: Create a four slide PowerPoint about your favorite place using one theme and tidy bullet points.

Animations & Transitions

Topics:

- Slide transitions types
- Transition timing and duration
- Transition sounds
- Animation basics
- Entrance animations
- Emphasis animations
- Exit animations
- Motion path animations
- Animation pane control
- Animation timing and delay
- Animation triggers
- Combining multiple animations
- Morph transition feature
- 3D models animation
- Creating smooth flows

Projects built:

- Animated story presentation
- Kinetic typography project
- Science process animation
- Interactive quiz show

Practice: Add a different transition to each slide of your deck and give the title text an entrance animation.

Multimedia & Interactive Elements

Topics:

- Inserting images and graphics
- Photo albums in PowerPoint
- Video insertion and editing
- Video playback options
- Audio and music addition
- Recording narration
- Screen recording feature
- Creating hyperlinks
- Action buttons
- Interactive navigation
- Zoom feature for emphasis
- Embedding web content
- Charts and graphs
- SmartArt in presentations
- QR codes in slides

Projects built:

- Video presentation project
- Interactive story with choices
- Virtual tour presentation
- Music video slideshow

Practice: Add a photo album slide, one short video and a little background music to your presentation.

Advanced Design & Presenting

Topics:

- Slide Master customization
- Creating custom layouts
- Design Ideas feature
- Icons and stock images
- 3D models and rotation
- Cameo (PowerPoint Live)
- Presenter Coach feature
- Rehearse with Coach
- Live captions and subtitles
- Translation features
- Broadcasting presentations
- Export options (video, PDF, etc.)
- PowerPoint Designer AI
- Accessibility checker
- Mobile presenting

Projects built:

- Professional pitch deck
- Educational tutorial
- Digital portfolio
- Conference presentation

Practice: Design a custom layout in Slide Master, then rehearse presenting your deck out loud for two minutes.

MONTH 4: EXCEL AND DATA SKILLS

Learn spreadsheet basics, data management, charts, and basic formulas with Excel.

Excel Basics & Data Entry

Topics:

- Excel interface and terminology
- Workbooks and worksheets
- Cells, rows, and columns
- Selecting cells and ranges
- Entering text and numbers
- AutoFill feature
- Number formats (currency, percentage, date)
- Cell formatting and styles
- Column width and row height
- Inserting and deleting rows/columns
- Copy, cut, paste special
- Find and replace
- Freeze panes
- Splitting windows
- Worksheet tabs management

Projects built:

- Class grade tracker
- Personal budget sheet
- Chore chart with dates
- Sports statistics table

Practice: Enter a pretend week of family fruit shopping into Excel and use AutoFill to complete the days of the week.

Formulas & Functions

Topics:

- Understanding formulas
- Basic arithmetic operations (+, -, *, /)
- Cell references (relative vs absolute)
- SUM function
- AVERAGE function
- COUNT and COUNTA
- MIN and MAX functions
- IF statements basics
- AutoSum feature
- Function wizard
- Formula bar usage
- Order of operations
- Copying formulas
- Common formula errors
- Simple problem solving

Projects built:

- Grade calculator with average
- Shopping budget calculator
- Simple game scorecard
- Time tracking sheet

Practice: Track your pocket money for an imaginary week, then total it with SUM and find the daily average with AVERAGE.

Charts & Data Visualization

Topics:

- Types of charts (column, line, pie)
- Creating charts from data
- Chart elements and parts
- Chart styles and colors
- Chart titles and labels
- Legend positioning
- Data labels addition
- Changing chart types
- Sparklines for mini charts
- Combination charts
- 3D chart effects
- Chart templates
- Printing charts
- Charts in other Office apps
- Infographic creation

Projects built:

- Weather data visualization
- Survey results charts
- Growth tracking chart
- Game statistics dashboard

Practice: Turn your favorite game or sports scores into a column chart and a pie chart, each with a proper title and labels.

Data Management & Analysis

Topics:

- Sorting data A-Z, Z-A
- Custom sort orders
- Filter data basics
- AutoFilter features
- Conditional formatting
- Data bars and color scales
- Icon sets for visual cues
- Tables in Excel
- Table styles and formatting
- PivotTable basics (introduction)
- Data validation rules
- Drop-down lists creation
- Protecting cells and sheets
- Sharing and collaboration
- Import/export data

Projects built:

- Student database
- Inventory tracking system
- Event planning spreadsheet
- Analysis report with visuals

Practice: Sort a list of ten pretend test scores from highest to lowest and use conditional formatting to highlight the top three.

MONTH 5: ONENOTE, TEAMS & COLLABORATION TOOLS

Master digital note-taking, online collaboration, and communication tools.

OneNote Digital Notebook

Topics:

- OneNote interface overview
- Notebooks, sections, and pages
- Creating notebook structure
- Typing and formatting notes
- Handwriting and drawing
- Inserting images and files
- Audio and video recording
- Screen clipping tool
- Tags for organization
- Search functionality
- To-do lists and checkboxes
- Tables in OneNote
- Math equations
- Syncing across devices

- Sharing notebooks

Projects built:

- Digital school binder
- Research project notebook
- Personal journal/diary
- Study guide creation

Practice: Set up a OneNote notebook with one section per school subject and fill one page with notes plus a small drawing.

Microsoft Teams for Students

Topics:

- Teams interface navigation
- Joining and creating teams
- Channels and conversations
- Chat features and etiquette
- Video calls and meetings
- Screen sharing basics
- File sharing in Teams
- Assignments submission
- Calendar integration
- Together mode and backgrounds
- Breakout rooms
- Meeting recordings
- Live events basics
- Teams mobile app
- Notifications management

Projects built:

- Class collaboration project
- Virtual study group setup
- Team presentation delivery
- Digital club meeting

Practice: With a parent, practice joining a video call, sending one polite chat message and sharing your screen for a minute.

Outlook & Email Skills

Topics:

- Email basics and etiquette
- Outlook interface tour
- Composing professional emails
- To, CC, BCC explained
- Subject line importance
- Attachments and file size
- Email signatures creation
- Folders and organization
- Rules and filters
- Calendar basics
- Scheduling meetings
- Tasks and to-do lists
- Contacts management
- Out of office replies
- Email safety and spam

Projects built:

- Professional email templates
- Class project coordination
- Event planning with calendar
- Contact database creation

Practice: Draft a polite email to a teacher with a clear subject line and one attachment, and send it to a parent instead to check it.

Additional Microsoft Tools

Topics:

- Microsoft Forms for surveys
- Creating quizzes and polls
- Sway for web presentations
- Whiteboard for collaboration
- To Do for task management
- Sticky Notes digital version
- Lens for text extraction
- Clipchamp video editor basics
- Microsoft Edge features
- Collections in Edge
- Immersive Reader
- Math Solver app
- Family Safety basics
- Microsoft Rewards
- Accessibility tools

Projects built:

- Class survey with Forms
- Web story with Sway

- Brainstorm with Whiteboard
- Video project with Clipchamp

Practice: Make a three question quiz in Microsoft Forms about your favorite topic and ask a family member to take it.

MONTH 6: ADVANCED INTEGRATION & PROFESSIONAL PROJECTS

Combine all skills for complex projects and professional-level work.

Advanced Windows Tools & Utilities

Topics:

- Snipping Tool and Snip & Sketch
- Voice Recorder application
- Calculator programmer mode
- Notepad and advanced features
- Character Map for symbols
- Windows Security essentials
- Backup and restore
- System maintenance
- Disk Cleanup utility
- Windows Updates management
- Troubleshooting basics
- Accessibility features deep dive
- Speech recognition
- Virtual desktops
- God Mode activation

Projects built:

- Create tutorial with screenshots
- System optimization guide
- Accessibility setup for needs
- Custom workflow creation

Practice: Capture three screenshots with the Snipping Tool and label each one in Paint.

Integrated Office Projects

Topics:

- Mail merge Word with Excel
- Embedding Excel charts in Word
- PowerPoint with Excel data
- OneNote integration with Office
- Teams assignments workflow
- Creating templates library
- Macro basics (introduction)
- Quick Parts and building blocks
- Custom ribbons basics
- Office themes consistency
- Cloud synchronization
- Version history usage
- Co-authoring features
- Office mobile apps
- Office on the web

Projects built:

- Newsletter with mail merge
- Report with live Excel data
- Integrated presentation package
- Complete project workflow

Practice: Embed an Excel chart of your scores or savings into a Word report and write two sentences explaining what it shows.

Professional Document Creation

Topics:

- Resume and CV creation
- Cover letter writing
- Business letter formats
- Report writing structure
- Proposal creation
- Meeting minutes template
- Invoice and receipt creation
- Certificate design
- Poster and flyer design
- Brochure layout
- Portfolio compilation
- Digital signatures basics
- PDF creation and editing
- Print preparation
- Document accessibility

Projects built:

- Personal resume creation
- Business proposal mock-up

- Event planning package
- Portfolio website content

Practice: Create a one page kid resume listing your skills, hobbies and proudest achievements in a clean layout.

Creative Multimedia Projects

Topics:

- Digital magazine creation
- Yearbook page design
- Interactive presentations
- Educational game creation
- Video project with Office
- Podcast production workflow
- Social media content creation
- Digital art portfolio
- Science fair presentation
- History timeline project
- Geography interactive maps
- Math visualization projects
- Literature analysis presentation
- Music program creation
- Sports statistics dashboard

Projects built:

- Digital magazine issue
- Interactive learning game
- Multimedia presentation
- Complete creative portfolio

Practice: Design one yearbook style page about your class or family with photos, captions and a title banner.

Final Capstone Project

Topics:

- Project planning and scope
- Requirements gathering
- Timeline creation
- Resource planning
- Design documentation
- Development phases
- Testing and revision
- User feedback incorporation
- Final polishing
- Presentation preparation
- Documentation writing
- Reflection and learning

Practice: Write a mini plan for your final project: what you will make, which Office apps you will use and your three work sessions.

Deliverables:

- Complete project files
- Documentation package
- Presentation deck
- Video demonstration
- Reflection report

Graduation & Certification

Topics:

- Final project presentations
- Peer review session
- Skills assessment completion
- Portfolio finalization
- Certificate preparation
- Future learning paths
- Advanced course options
- Career pathway discussion
- Industry certifications info
- Continued learning resources
- Alumni network access
- Celebration ceremony

Practice: Rehearse your final presentation once at home and use one piece of family feedback to improve a slide.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1 (Weeks 1-4): Windows mastery, digital art with Paint & Paint 3D
- Month 2 (Weeks 5-8): Microsoft Word - documents, reports, newsletters
- Month 3 (Weeks 9-12): PowerPoint - presentations, animations, multimedia
- Month 4 (Weeks 13-16): Excel - data entry, formulas, charts, analysis
- Month 5 (Weeks 17-20): OneNote, Teams, Outlook, collaboration tools
- Month 6 (Weeks 21-26): Integrated projects, capstone, certification

Total projects: 40+ professional documents, presentations, spreadsheets & projects

SKILLS THE STUDENT WILL HAVE AT THE END

- Windows: file management, organization, system navigation, shortcuts
- Microsoft Word: formatting, page layout, images, tables, collaboration
- PowerPoint: slide design, animations, transitions, multimedia presentations
- Excel: data entry, formulas, functions, charts, data visualization
- OneNote: digital notebooks, note-taking, organization
- Teams & Outlook: collaboration, email etiquette, video meetings
- Digital Art: Paint, Paint 3D, creative design skills
- Professional Skills: document design, data presentation, digital citizenship

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (1 hour each)
Practice exercises: 1-2 hours of hands-on practice
Weekly project: 1 real-world project per week
Total per week: 4-6 hours

SUPPORT INCLUDED

Live instruction: Expert-led live classes with recorded replays
Age appropriate: Content designed for kids - real-world, school-relevant projects
Parent resources: Parent and teacher resources for reinforcing skills at home
Community: Access to Modern Age Coders digital skills community
School ready: Skills directly applicable to school projects and assignments

CERTIFICATION DETAIL

Final certificate: Microsoft Office Young Professional Certificate (Modern Age Coders)
Capstone required: Integrated capstone project using Word, PowerPoint, and Excel together
Portfolio: 40+ professional documents and presentations in digital portfolio

WHERE THIS CAN LEAD

- Digital Content Creator
- Student Technology Leader
- Office Productivity Expert
- Data Analysis Beginner
- Professional Document Designer
- Ready for advanced coding or design courses

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my child learn in this course?

A: Across 6 months your child progresses from Windows basics and file management to confident use of Word, PowerPoint, Excel, Paint and Paint 3D, OneNote, Teams, and Outlook. The course is project-based throughout, ending with an integrated capstone where students combine several Office tools in one professional piece of work.

Q: Which age group is this designed for?

A: It is built for children aged 8 to 14. Lessons assume no computer experience at all, so it also suits kids who have barely used a keyboard, as long as they can read basic instructions.

Q: Does my child need computer experience before starting?

A: No. Everything builds step by step from there.

Q: What software does my child need at home?

A: A Windows computer or laptop with Microsoft Office (Word, PowerPoint, Excel). If you do not have an Office subscription, we provide guidance on free alternatives so no child is left out.

Q: How does this help with school, and what comes next?

A: The skills map directly to schoolwork: essays and reports in Word, class presentations in PowerPoint, and data projects in Excel. Afterwards, many students move into our Scratch or kids coding courses, since comfortable computer use makes learning to code much smoother.

Q: Can my child attend a trial class first?

A: Yes, the first step for every family is a free demo class, booked at learn.modernagecoders.com/contact. There is no commitment attached, and parents can sit in to see exactly how the teacher works with kids.

Q: What do the classes look like?

A: Each class is a live 1-hour online session with a real teacher guiding a small group, never a pre-recorded video. Children practice hands-on in the actual software during class and get help the moment they are stuck.

Q: Will my child receive a certificate?

A: Yes. On completing the course your child earns a completion certificate along with a portfolio of finished projects: documents, presentations, spreadsheets, and digital art they created themselves.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/microsoft-office-kids-mastery>

=====

COURSE 88 OF 118: MIDDLE SCHOOL MATHEMATICS MASTERY: PRE-ALGEBRA & GEOMETRY

=====

Course link: <https://learn.modernagecoders.com/courses/comprehensive-middle-school-mathematics-mastery>
Subject area: Middle School Mathematics Education
Designed for: Girls only batches, Middle school
Who can join: Complete Beginner to Advanced Middle School Level
Course length: 12 months (52 weeks)
Time needed: 8-10 hours/week recommended
Certificate: Middle School Mathematics Excellence Certificate

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year middle school maths program taking students from basic arithmetic to pre-algebra, geometry and statistics with real problem-solving strategies.

In one line: From Number Basics to Advanced Mathematical Thinking

ABOUT THE PROGRAMME

This comprehensive program transforms students into confident mathematical thinkers. Whether starting with basic arithmetic or advancing to complex problem-solving, this 1-year program builds a rock-solid foundation in mathematics. Students will master essential mathematical concepts through interactive lessons, real-world applications, and engaging projects. From fractions to functions, geometry to statistics, every topic is covered with depth and clarity. By completion, students will excel in standardized tests and be fully prepared for high school mathematics.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute basics - no prerequisites required
- Full year of structured, progressive learning
- Covers entire middle school math curriculum
- Real-world applications and STEM connections
- Test preparation and exam strategies included
- Interactive visualizations and manipulatives
- Personalized learning paths based on progress
- Parent progress reports and involvement tools

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Number Systems, Operations, and Basic Problem Solving
Phase 2: Arithmetic Mastery (Months 4-6): Fractions, Decimals, Percentages, and Ratios
Phase 3: Algebraic Thinking (Months 7-9): Pre-Algebra, Equations, Functions, and Patterns
Phase 4: Geometry & Advanced Topics (Months 10-12): Geometry, Statistics, Probability, and Complex Problem Solving

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Elementary school mathematics completion
Coding experience: Basic arithmetic knowledge helpful
Equipment: Computer/tablet with internet, calculator
Time commitment: 8-10 hours per week consistently
English: Grade-appropriate reading level
Motivation: Enthusiasm for learning mathematics

WHO THIS COURSE IS FOR

Students: Middle school students grades 6-8
Working professionals: Parents helping children with math
Entrepreneurs: Homeschool families
Freelancers: Tutors and educators
Kids: Advanced elementary students ready for challenge
Anyone: Anyone wanting strong math foundation

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NUMBER SYSTEMS & FUNDAMENTAL OPERATIONS (MONTHS 1-3, WEEKS 1-13)

Build unshakeable foundations in number concepts, operations, and mathematical thinking.

Months 1-2: Number Systems and Basic Operations (Week 1-8)

Introduction to Number Systems

Topics:

- Understanding place value system
- Reading and writing large numbers (up to billions)
- Expanded form and word form
- Comparing and ordering whole numbers
- Number line representation
- Rounding to nearest 10, 100, 1000
- Roman numerals basics
- Introduction to negative numbers
- Integer concepts and number line
- Absolute value understanding
- Real-world contexts for negative numbers
- Temperature, elevation, and financial contexts

Projects built:

- Create a number system timeline
- Design a place value game
- Real-world number investigation project

Practice: Daily: 20 problems on place value, 15 minutes mental math

Addition and Subtraction Mastery

Topics:

- Multi-digit addition with regrouping
- Addition properties: commutative, associative, identity
- Mental math strategies for addition
- Multi-digit subtraction with borrowing
- Subtraction as inverse of addition
- Estimation strategies for sums and differences
- Adding and subtracting integers
- Using number lines for integer operations
- Word problems with addition and subtraction
- Multi-step problem solving
- Checking answers using inverse operations
- Real-world applications: budgets, distances

Projects built:

- Budget planning project
- Distance calculation map activity
- Integer game design

Practice: Solve 30 addition/subtraction problems daily with increasing complexity

Multiplication Fundamentals

Topics:

- Multiplication as repeated addition
- Times tables mastery (up to 12×12)
- Multiplication properties: commutative, associative, distributive
- Multi-digit multiplication algorithms
- Lattice multiplication method
- Area model for multiplication
- Multiplying by powers of 10
- Estimation in multiplication
- Mental math tricks for multiplication
- Multiplying integers and sign rules
- Word problems involving multiplication
- Real-world applications: area, arrays, scaling

Projects built:

- Multiplication strategy booklet
- Area calculation for room design
- Array art project

Practice: Master times tables, 25 multi-digit multiplication problems

Division and Factors

Topics:

- Division as inverse of multiplication
- Long division algorithm step-by-step
- Division with remainders
- Interpreting remainders in context
- Divisibility rules (2, 3, 4, 5, 6, 9, 10)
- Factors and multiples
- Prime and composite numbers
- Prime factorization
- Greatest Common Factor (GCF)

- Least Common Multiple (LCM)
- Division of integers and sign rules
- Word problems with division

Projects built:

- Factor tree artwork
- Divisibility rule investigation
- Prime number sieve project

Practice: 30 division problems, find factors and multiples daily

Month 3: Order of Operations and Problem Solving (Week 9-13)

Order of Operations (PEMDAS/BODMAS)

Topics:

- Introduction to order of operations
- Parentheses/Brackets in expressions
- Exponents and powers basics
- Multiplication and Division (left to right)
- Addition and Subtraction (left to right)
- Complex expressions with multiple operations
- Nested parentheses
- Common mistakes and misconceptions
- Using calculators correctly
- Writing expressions from word problems
- Evaluating expressions with variables
- Creating equivalent expressions

Projects built:

- PEMDAS puzzle creation
- Expression evaluation game
- Calculator investigation project

Practice: Evaluate 20 complex expressions daily

Problem-Solving Strategies

Topics:

- Understanding the problem: what's being asked?
- Identifying relevant information
- Drawing diagrams and models
- Making organized lists
- Looking for patterns
- Working backwards strategy
- Guess and check method
- Using simpler numbers first
- Breaking complex problems into steps
- Checking reasonableness of answers
- Multiple solution methods
- Mathematical communication and explanation

Projects built:

- Problem-solving portfolio
- Create word problems for peers
- Strategy comparison presentation

Practice: Solve 5 multi-step word problems daily using different strategies

Phase 1 Review and Assessment

Topics:

- Number systems comprehensive review
- Operations mastery check
- Order of operations practice
- Problem-solving strategy application
- Common errors and corrections
- Speed and accuracy building

Projects built:

- Create a math tutorial video
- Design a board game using Phase 1 concepts
- Peer tutoring sessions

Assessment: Phase 1 comprehensive exam covering all topics

PHASE 2: FRACTIONS, DECIMALS, AND PROPORTIONAL REASONING (MONTHS 4-6, WEEKS 14-26)

Master rational numbers, proportional relationships, and their real-world applications.

Months 4-5: Fractions and Mixed Numbers (Week 14-21)

Understanding Fractions

Topics:

- Fraction concepts: parts of a whole
- Numerator and denominator meaning
- Proper fractions, improper fractions, mixed numbers
- Converting between improper fractions and mixed numbers
- Fractions on a number line
- Equivalent fractions using models
- Simplifying fractions to lowest terms
- Finding equivalent fractions algebraically
- Comparing fractions with like denominators

- Comparing fractions with unlike denominators
- Cross multiplication for comparison
- Ordering fractions from least to greatest

Projects built:

- Fraction pizza party planning
- Fraction art with geometric shapes
- Recipe conversion project

Practice: 30 fraction problems daily, visual and numerical

Operations with Fractions

Topics:

- Adding fractions with like denominators
- Adding fractions with unlike denominators
- Finding common denominators
- Adding mixed numbers with regrouping
- Subtracting fractions and mixed numbers
- Borrowing in mixed number subtraction
- Multiplying fractions by whole numbers
- Multiplying fractions by fractions
- Understanding fraction multiplication visually
- Dividing fractions: keep-change-flip
- Dividing mixed numbers
- Word problems with fraction operations

Projects built:

- Cooking with fractions project
- Construction measurements activity
- Fraction operations game design

Practice: 25 fraction operation problems with increasing difficulty

Decimals and Place Value

Topics:

- Decimal place value: tenths, hundredths, thousandths
- Reading and writing decimals
- Decimals on number lines
- Comparing and ordering decimals
- Rounding decimals to various places
- Converting fractions to decimals
- Converting decimals to fractions
- Terminating vs repeating decimals
- Adding and subtracting decimals
- Lining up decimal points
- Estimation with decimals
- Real-world decimal applications: money, measurements

Projects built:

- Shopping budget with decimals
- Sports statistics analysis
- Decimal measurement lab

Practice: 30 decimal problems including conversions

Multiplying and Dividing Decimals

Topics:

- Multiplying decimals by whole numbers
- Multiplying decimals by decimals
- Counting decimal places in products
- Multiplying by powers of 10
- Dividing decimals by whole numbers
- Dividing by decimals: moving decimal points
- Dividing decimals by powers of 10
- Scientific notation introduction
- Estimation strategies for decimal operations
- Calculator use and interpretation
- Word problems with decimal operations
- Unit pricing and comparison shopping

Projects built:

- Business profit calculation project
- Science measurement conversions
- Create a decimal operations tutorial

Practice: 25 decimal multiplication and division problems

Percentages and Applications

Topics:

- Understanding percent as per hundred
- Converting between fractions, decimals, and percents
- Finding percentages of numbers
- Mental math with percentages (10%, 25%, 50%)
- Finding what percent one number is of another
- Finding the whole given a part and percent
- Percent increase and decrease
- Simple interest calculations
- Sales tax and discount problems
- Tips and commission calculations
- Percent error in measurements
- Data representation with percentages

Projects built:

- Sales and discount investigation
- Survey data percentage analysis
- Personal finance percentage project

Practice: 20 percentage problems from real-world contexts

Month 6: Ratios and Proportional Relationships (Week 22-26)

Understanding Ratios

Topics:

- Ratio concepts and notation
- Part-to-part vs part-to-whole ratios
- Equivalent ratios
- Simplifying ratios
- Ratios with three quantities
- Rates and unit rates
- Finding unit prices
- Speed, distance, and time relationships
- Comparing rates
- Best buy problems
- Ratio tables and their uses
- Graphing ratios on coordinate plane

Projects built:

- Recipe scaling project
- Speed comparison investigation
- Mix and match ratio art

Practice: 25 ratio and rate problems

Proportions and Scale

Topics:

- Understanding proportions
- Setting up proportions from word problems
- Cross multiplication to solve proportions
- Checking proportion solutions
- Direct variation
- Inverse variation introduction
- Scale drawings and maps
- Finding actual distances from maps
- Creating scale drawings
- Similar figures and proportions
- Scale factor
- Proportional relationships in data

Projects built:

- Map scale project
- Architectural drawing activity
- Proportion in art exploration

Practice: 20 proportion problems including scale applications

Real-World Applications

Topics:

- Currency exchange rates
- Unit conversions using proportions
- Mixture problems
- Work rate problems
- Population density calculations
- Fuel efficiency comparisons
- Nutritional information analysis
- Sports statistics and ratios
- Business applications: profit margins
- Scientific applications: concentrations
- Engineering: gear ratios
- Cooking and baking proportions

Projects built:

- International currency project
- Nutrition label analysis
- Sports statistics presentation

Practice: Apply proportional reasoning to 15 real-world scenarios

Data with Fractions, Decimals, and Percents

Topics:

- Circle graphs (pie charts) with percentages
- Bar graphs with fractional values
- Line graphs with decimal scales
- Converting between data representations
- Misleading graphs and statistics
- Surveys and sampling basics
- Calculating averages with decimals
- Finding median and mode with fractions
- Data interpretation questions
- Creating data displays
- Spreadsheet basics for calculations
- Technology tools for data analysis

Projects built:

- Class survey data analysis
- Create infographic with percentages
- Data story presentation

Practice: Analyze 10 different data sets using various representations

Phase 2 Review and Assessment

Topics:

- Comprehensive fraction review
- Decimal operations mastery
- Percentage problem solving
- Ratio and proportion applications
- Mixed problem sets
- Speed and accuracy challenges

Projects built:

- Create a fraction-decimal-percent converter tool
- Design a proportion puzzle book
- Real-world math fair project

Assessment: Phase 2 comprehensive exam on rational numbers and proportions

----- PHASE 3: ALGEBRAIC THINKING AND PRE-ALGEBRA (MONTHS 7-9, WEEKS 27-39) -----

Develop algebraic reasoning, solve equations, and understand functions and patterns.

Months 7-8: Introduction to Algebra (Week 27-34)

Variables and Expressions

Topics:

- Introduction to variables as unknowns
- Variables as varying quantities
- Writing algebraic expressions
- Translating words to algebra
- Evaluating expressions with substitution
- Terms, coefficients, and constants
- Like terms and unlike terms
- Combining like terms
- Distributive property with variables
- Factoring out common factors
- Expanding expressions
- Simplifying algebraic expressions

Projects built:

- Expression matching game
- Variable story problems creation
- Algebraic art patterns

Practice: 30 expression problems: writing, evaluating, simplifying

One-Step Equations

Topics:

- Understanding equations vs expressions
- Solution concept and checking
- Solving using addition/subtraction
- Solving using multiplication/division
- Inverse operations review
- Equations with negative numbers
- Equations with fractions
- Equations with decimals
- Writing equations from word problems
- Checking solutions by substitution
- Special cases: no solution, infinite solutions
- Real-world equation modeling

Projects built:

- Equation balance demonstration
- Word problem equation book
- Equation solving race game

Practice: Solve 25 one-step equations with various number types

Two-Step and Multi-Step Equations

Topics:

- Two-step equations strategy
- Order of operations in reverse
- Equations with parentheses
- Distributive property in equations
- Combining like terms before solving
- Variables on both sides
- Equations with fractions: clearing denominators
- Decimal equations: clearing decimals
- Word problems requiring multi-step equations
- Consecutive integer problems
- Perimeter and area equations
- Mixture and motion problems basics

Projects built:

- Multi-step equation puzzle
- Real-world equation modeling project

- Create equation tutorial video

Practice: 20 multi-step equations with increasing complexity

Inequalities

Topics:

- Understanding inequality symbols
- Graphing inequalities on number lines
- Writing inequalities from situations
- Solving one-step inequalities
- Solving two-step inequalities
- Multiplying/dividing by negatives: flipping the sign
- Compound inequalities (AND, OR)
- Graphing compound inequalities
- Inequality word problems
- Systems of inequalities introduction
- Real-world constraints and inequalities
- Checking inequality solutions

Projects built:

- Inequality graphing art
- Budget constraint project
- Design inequality game

Practice: 25 inequality problems with graphing

Introduction to Functions

Topics:

- Input-output relationships
- Function machines concept
- Function notation: $f(x)$
- Function tables
- Finding patterns in functions
- Linear functions basics
- Graphing functions on coordinate plane
- Domain and range introduction
- Vertical line test
- Real-world functions: temperature conversion
- Function composition basics
- Inverse operations and functions

Projects built:

- Function machine construction
- Real-world function investigation
- Create function puzzle book

Practice: Evaluate and graph 20 different functions

Month 9: Coordinate Geometry and Graphing (Week 35-39)

The Coordinate Plane

Topics:

- Cartesian coordinate system
- x-axis and y-axis
- Quadrants I, II, III, IV
- Plotting points (x, y)
- Finding coordinates of points
- Distance between points (horizontal/vertical)
- Midpoint of a line segment
- Reflections across axes
- Translations on coordinate plane
- Rotations $(90^\circ, 180^\circ)$
- Creating designs with coordinates
- Coordinate plane applications

Projects built:

- Coordinate art project
- Treasure map coordinates
- Transformation artwork

Practice: Plot 30 points and perform transformations

Linear Relationships

Topics:

- Understanding slope as rate of change
- Calculating slope from two points
- Positive, negative, zero, undefined slopes
- Slope-intercept form: $y = mx + b$
- Finding y-intercept
- Graphing using slope-intercept form
- Point-slope form introduction
- Writing equations from graphs
- Writing equations from two points
- Parallel and perpendicular lines
- Systems of linear equations (graphing method)
- Real-world linear relationships

Projects built:

- Linear relationship investigation
- Cost analysis with linear functions
- Design with linear equations

Practice: Graph 20 linear equations and find equations from graphs

Patterns and Sequences

Topics:

- Number patterns recognition
- Arithmetic sequences
- Finding common difference
- nth term of arithmetic sequence
- Geometric sequences introduction
- Finding common ratio
- Extending patterns
- Visual patterns and growth
- Fibonacci sequence exploration
- Recursive patterns
- Creating pattern rules
- Patterns in nature and art

Projects built:

- Pattern investigation poster
- Sequence art creation
- Pattern puzzle book

Practice: Identify and extend 25 different patterns

Polynomials Introduction

Topics:

- Monomials, binomials, trinomials
- Degree of polynomials
- Adding polynomials
- Subtracting polynomials
- Multiplying monomials
- Multiplying polynomials by monomials
- FOIL method for binomials
- Special products: $(a+b)^2$, $(a-b)^2$
- Factoring out GCF from polynomials
- Factoring simple trinomials
- Zero product property
- Polynomial applications

Projects built:

- Polynomial art with algebra tiles
- Area models for multiplication
- Polynomial puzzle creation

Practice: 20 polynomial operations and factoring problems

Phase 3 Review and Assessment

Topics:

- Algebraic expressions and equations review
- Inequality solving and graphing
- Function concepts and graphing
- Coordinate geometry mastery
- Linear relationships
- Patterns and sequences
- Polynomial operations

Projects built:

- Algebra tutorial video series
- Create algebraic board game
- Real-world algebra applications portfolio

Assessment: Phase 3 comprehensive exam on algebraic concepts

----- PHASE 4: GEOMETRY, STATISTICS, AND ADVANCED PROBLEM SOLVING (MONTHS 10-12, WEEKS 40-52) -----

Master geometric concepts, statistical analysis, probability, and complex problem-solving strategies.

Months 10-11: Geometry Foundations (Week 40-47)

Basic Geometric Concepts

Topics:

- Points, lines, and planes
- Line segments and rays
- Parallel and perpendicular lines
- Angles: acute, right, obtuse, straight
- Measuring angles with protractor
- Angle relationships: complementary, supplementary
- Vertical angles and linear pairs
- Angles formed by transversals
- Corresponding angles
- Alternate interior and exterior angles
- Triangle angle sum theorem
- Exterior angles of triangles

Projects built:

- Angle art creation
- Architecture angle investigation
- Geometric city design

Practice: Measure and calculate 30 angle problems

Triangles and Quadrilaterals

Topics:

- Types of triangles by sides: equilateral, isosceles, scalene
- Types of triangles by angles: acute, right, obtuse
- Triangle inequality theorem
- Congruent triangles: SSS, SAS, ASA
- Similar triangles and proportions
- Pythagorean theorem
- Pythagorean theorem applications
- Special right triangles: 45-45-90, 30-60-90
- Quadrilaterals: parallelograms, rectangles, squares
- Rhombus and trapezoid properties
- Diagonals of quadrilaterals
- Polygon angle sum formulas

Projects built:

- Triangle construction project
- Pythagorean theorem proofs
- Quadrilateral classification poster

Practice: 25 triangle and quadrilateral problems

Circles

Topics:

- Circle vocabulary: center, radius, diameter
- Circumference formula: $C = 2\pi r$
- Area of circles: $A = \pi r^2$
- Arc length
- Sector area
- Chords and secants
- Tangent lines to circles
- Inscribed angles
- Central angles
- Circle word problems
- Composite figures with circles
- Real-world circle applications

Projects built:

- Circle art with compass
- Pi investigation project
- Design circular patterns

Practice: 20 circle calculation problems

Area and Perimeter

Topics:

- Perimeter of polygons
- Area of rectangles and squares
- Area of parallelograms
- Area of triangles: $\frac{1}{2}bh$
- Area of trapezoids
- Area of regular polygons
- Area of composite figures
- Perimeter of composite figures
- Area and perimeter word problems
- Optimization problems: maximum area
- Surface area introduction
- Real-world applications: flooring, fencing

Projects built:

- Garden design project
- Floor plan area calculations
- Packaging design optimization

Practice: Calculate area and perimeter of 25 figures

3D Geometry and Volume

Topics:

- 3D shapes: prisms, pyramids, cylinders, cones, spheres
- Faces, edges, and vertices
- Nets of 3D shapes
- Volume of rectangular prisms
- Volume of triangular prisms
- Volume of cylinders
- Volume of pyramids and cones
- Volume of spheres
- Surface area of prisms
- Surface area of cylinders
- Composite 3D figures
- Volume and capacity conversions

Projects built:

- 3D shape construction
- Volume investigation lab
- Package design project

Practice: Calculate volume and surface area of 20 3D shapes

Data Collection and Representation

Topics:

- Types of data: categorical vs numerical
- Data collection methods
- Surveys and sampling techniques
- Bias in data collection
- Frequency tables
- Bar graphs and histograms
- Line graphs and time series
- Circle graphs (pie charts)
- Stem-and-leaf plots
- Box-and-whisker plots
- Scatter plots introduction
- Choosing appropriate graphs

Projects built:

- School survey project
- Data visualization poster
- Create data infographic

Practice: Create 10 different data representations

Measures of Central Tendency

Topics:

- Mean (average) calculation
- Weighted averages
- Median finding
- Mode identification
- Comparing mean, median, mode
- Effects of outliers
- Range as measure of spread
- Quartiles and interquartile range
- Mean absolute deviation
- Standard deviation introduction
- Interpreting data spread
- Using measures to compare data sets

Projects built:

- Grade analysis project
- Sports statistics comparison
- Weather data analysis

Practice: Calculate statistics for 15 data sets

Probability Fundamentals

Topics:

- Probability as likelihood
- Probability scale: 0 to 1
- Experimental vs theoretical probability
- Sample space and outcomes
- Simple probability calculations
- Complement of an event
- Probability of compound events
- Independent vs dependent events
- Tree diagrams
- Fundamental counting principle
- Permutations introduction
- Combinations introduction

Projects built:

- Probability game design
- Dice experiment analysis
- Real-world probability investigation

Practice: Solve 25 probability problems

Statistical Analysis and Interpretation

Topics:

- Correlation vs causation
- Positive and negative correlation
- Line of best fit basics
- Making predictions from data
- Margin of error concept
- Confidence intervals introduction
- Misleading statistics
- Statistical significance basics
- Comparing populations
- Data-driven decision making
- Technology tools for statistics
- Spreadsheet data analysis

Projects built:

- Correlation investigation
- Statistical report writing
- Data prediction project

Practice: Analyze and interpret 10 real-world data scenarios

Problem Solving with Multiple Concepts

Topics:

- Multi-step problems combining algebra and geometry

- Rate, ratio, and proportion in geometry
- Statistical problems with percentages
- Probability with fractions and decimals
- Optimization problems
- Logic puzzles and reasoning
- Mathematical modeling
- Estimation and approximation
- Working with constraints
- Breaking down complex problems
- Checking solutions for reasonableness

Projects built:

- Complex problem-solving portfolio
- Mathematical modeling project
- Create problem-solving guide

Practice: Solve 15 complex, multi-concept problems

Month 12: Advanced Topics and Test Preparation (Week 49-52)

Advanced Number Theory

Topics:

- Modular arithmetic basics
- Number patterns and sequences
- Pascal's triangle
- Binary numbers introduction
- Different base systems
- Cryptography basics
- Mathematical proofs introduction
- Logic and truth tables
- Set theory basics
- Venn diagrams with three sets
- Infinity concepts

Projects built:

- Number system exploration
- Code-breaking activity
- Mathematical proof writing

Practice: Explore 10 advanced number concepts

Mathematical Connections

Topics:

- Mathematics in science
- Mathematics in technology
- Mathematics in art and music
- Golden ratio and Fibonacci
- Fractals introduction
- Tessellations and symmetry
- Mathematical modeling in real life
- Financial mathematics
- Sports analytics
- Game theory basics
- Mathematics in nature

Projects built:

- STEM integration project
- Mathematical art creation
- Real-world math presentation

Practice: Apply mathematics to 10 different fields

Competition Mathematics

Topics:

- Problem-solving strategies review
- Competition problem types
- Speed and accuracy techniques
- Mental math advanced strategies
- Olympiad-style problems
- Logic and reasoning puzzles
- Number theory problems
- Geometry competition problems
- Combinatorics basics
- Proof techniques
- Time management in tests

Projects built:

- Math competition preparation
- Create competition problems
- Strategy guide compilation

Practice: Solve 20 competition-level problems

Test Preparation and Strategies

Topics:

- Standardized test formats
- Multiple choice strategies
- Grid-in question techniques
- Extended response writing
- Time management during tests
- Test anxiety management

- Common test mistakes
- Calculator vs non-calculator sections
- Checking work efficiently
- Practice test analysis
- Study planning and organization

Projects built:

- Test-taking strategy guide
- Practice test creation
- Study plan development

Practice: Complete 3 full-length practice tests

Month 12: Final Projects and Comprehensive Review (Week 51-52)

Comprehensive Review and Integration

Topics:

- Number systems and operations review
- Fractions, decimals, percentages mastery
- Algebraic concepts consolidation
- Geometry theorems and applications
- Statistics and probability review
- Problem-solving strategies compilation
- Cross-curricular connections
- Real-world applications synthesis
- Mathematical communication skills
- Technology integration review
- Portfolio organization
- Self-assessment and reflection

Projects built:

- Year-long portfolio compilation
- Mathematical autobiography
- Comprehensive problem set creation

Final Capstone Projects

Topics:

- Project planning and design
- Research and investigation
- Data collection and analysis
- Mathematical modeling
- Presentation preparation
- Peer collaboration
- Project documentation
- Technology integration
- Creative mathematical expression
- Real-world problem solving
- Community application

Deliverables:

- Research project on mathematical topic
- Real-world problem-solving project
- Mathematical art or design project
- Teaching project for younger students
- Technology integration project
- Portfolio presentation

Final Assessment and Future Planning

Topics:

- Comprehensive final examination
- Portfolio presentation and defense
- Peer evaluations
- Self-reflection essays
- Goal setting for high school
- Advanced course recommendations
- Summer enrichment opportunities
- Mathematics career exploration
- Celebration of achievements
- Awards and recognition
- Parent conferences
- Transition planning

Deliverables:

- Final comprehensive exam
- Complete portfolio with all projects
- Self-evaluation and reflection
- Presentation to parents/community
- High school readiness assessment
- Individual achievement report
- Future learning plan

Assessment: FINAL COMPREHENSIVE EXAM - Cover all 12 months of content

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Number System Timeline, Budget Planner, Factor Tree Art, Problem-Solving Portfolio
- Phase 2: Fraction Pizza Party, Shopping Budget Calculator, Scale Drawing, Data Infographic
- Phase 3: Equation Balance Game, Function Machine, Coordinate Art, Pattern Investigation

- Phase 4: Geometric City Design, Statistics Survey Project, Probability Game, Math Fair Project
- Final: Comprehensive Portfolio, Research Project, Real-World Application

Total projects: 60+ projects ranging from simple to complex applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Number Systems: Whole numbers, integers, rational numbers, real numbers
- Operations: All four operations with various number types, order of operations
- Fractions & Decimals: Operations, conversions, real-world applications
- Percentages: Calculations, increases/decreases, financial applications
- Algebra: Variables, expressions, equations, inequalities, functions
- Geometry: 2D shapes, 3D solids, measurements, transformations, proofs
- Statistics: Data collection, analysis, measures of central tendency, displays
- Probability: Basic probability, compound events, counting principles
- Problem Solving: Multiple strategies, mathematical modeling, logic
- Technology: Calculator use, spreadsheets, graphing tools, online resources
- Communication: Mathematical writing, presentations, explanations
- Test Taking: Standardized test strategies, time management, accuracy

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-4 hours
 Hands on practice: 4-5 hours
 Projects: 2-3 hours
 Practice problems: 2-3 hours
 Total per week: 8-10 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving sessions
 Mentorship: 1-on-1 tutoring available
 Community: Peer study groups and forums
 Code review: Problem solution reviews
 Career support: High school preparation guidance
 Lifetime access: All content and future updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion
 Final certificate: Middle School Mathematics Excellence Certificate
 LinkedIn badge: Digital badge for online profiles
 Industry recognized: Aligned with educational standards
 Portfolio projects: 60+ projects to showcase learning

WHERE THIS CAN LEAD

- High School Advanced Mathematics Track
- AP Mathematics Preparation
- STEM Program Readiness
- Mathematics Competition Participant
- Science Fair Participant
- Engineering Program Preparation
- Computer Science Foundation
- Financial Literacy
- Data Science Introduction
- Mathematical Research Interest

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the Middle School Mathematics program cover?

A: The 12 month program builds in four phases: number systems and operations; fractions, decimals, percentages and ratios; pre-algebra with equations, functions and graphing; and finally geometry, statistics, probability and multi-step problem solving. The aim is to make the middle school years the strongest part of a child's maths story.

Q: Who is this course right for?

A: A gentle diagnostic in the first weeks means each child starts where their actual understanding is, not where their grade label says.

Q: What does my child need before joining?

A: Just completed elementary school mathematics, a computer or tablet with internet, and a calculator. No coding background is needed, and grade level reading is enough to follow the classes comfortably.

Q: Will this help with school exams and word problems?

A: Yes. Word problems and multi-step reasoning are treated as core skills throughout, not an afterthought: children learn to translate situations into maths before computing. Because the course teaches the concepts under every school syllabus, the work transfers directly to classroom tests.

Q: How is progress measured?

A: Through weekly practice reviewed for method, phase-end checks before new material begins, and a running mastery record shared with parents. The final month includes projects and a comprehensive review, so you finish with visible evidence of what your child can now do.

Q: Can my child attend a class before we decide?

A: Of course. The first step for every family is a free demo class, booked at learn.modernagecoders.com/contact with no commitment, where your child meets the teacher and we assess the right starting point together. You can also WhatsApp us at +91 91233 66161.

Q: What is the class format?

A: Live one hour online classes with a real teacher, never pre-recorded videos. Groups stay small so quieter children get airtime too, and every child solves problems aloud during class rather than just watching.

Q: Does my child receive a certificate?

A: Yes, the Middle School Mathematics Excellence Certificate on completion, plus a record of the projects and problem work done across the year. Modern Age Coders holds a 4.9 star rating from 547+ Google reviews.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/comprehensive-middle-school-mathematics-mastery>

=====

COURSE 89 OF 118: MINECRAFT CODING FOR KIDS: FROM BLOCKS TO REAL CODE

=====

Course link: <https://learn.modernagecoders.com/courses/minecraft-coding-for-kids-course>

Subject area: Kids Coding

Designed for: Kids

Who can join: Beginner, ages 7 to 12

Course length: 4 months (16 weeks)

Time needed: 2 live classes/week + 1-2 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Minecraft coding classes for kids ages 7-12: live online lessons using block-based MakeCode, from first commands to robot agents, auto-farms, and mini-games.

In one line: The game your child already loves becomes the place they learn to think in code.

ABOUT THE PROGRAMME

Minecraft has a superpower most parents never see: a built-in coding environment where children snap together blocks of code, press run, and watch the game obey. A castle wall appears in two seconds. A robot agent plants a farm. A trap springs when someone steps on the wrong block. This 4-month live course uses that loop, code it, run it, see it, to teach real programming ideas to children aged 7 to 12. Months 1 and 2 build the core: commands, the agent robot, loops that build faster than hands ever could, variables, and decisions. Months 3 and 4 turn those pieces into games: randomness, scores, timers, a full mini-game classmates play, and a finishing capstone the student designs. Along the way, older students get to flip the same blocks into JavaScript view and see what their program looks like as typed code. Every class is live and hands-on. The instructor sets a build challenge, students code their own solution while the instructor watches and helps, and each week ends with something running in the world that was not there before.

WHAT MAKES THIS COURSE DIFFERENT

- Coding happens inside Minecraft itself, so the feedback loop is instant and the motivation is already installed
- The agent robot makes loops and conditions physical: children watch their code walk, turn, plant, and build block by block
- Challenges are sized for live classes: every session ends with a working build, not a half-finished follow-along
- Block code is flipped to JavaScript view for curious older kids, so the bridge to typed programming is walked, not just mentioned
- Screen time turns productive: the same game hours now produce farms, machines, and mini-games

- the child engineered
- Small live batches mean the instructor sees every child's screen and every child's thinking, every class

THE LEARNING PATH, STEP BY STEP

Phase 1: Code that builds: chat commands, the agent robot, then loops, variables, and decisions that construct in seconds what hands build in hours

Phase 2: Code that plays: randomness, scores, timers, and events combined into real mini-games, then a student-designed capstone showcased to families

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from the very first block of code

Reading: Basic reading comfort, enough to follow labeled blocks. Seven-year-olds who read simple sentences do fine

Minecraft setup: Minecraft Education, which includes the built-in coding editor. We help every family set it up before classes start

Equipment: A computer or iPad that runs Minecraft Education, plus a stable internet connection

WHO THIS COURSE IS FOR

Minecraft fans: Kids who play Minecraft daily and are ready to command it instead of just mining in it

Complete beginners: Children aged 7 to 12 with zero coding exposure who need a first step that feels like play

Restless learners: Kids who resist worksheets but will happily debug a robot farmer for twenty minutes

Future typers: Older kids in the range who are curious what real typed code looks like, met here through the JavaScript view

Screen time worriers: Parents who want existing Minecraft hours converted into planning, logic, and finished projects

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: CODE THAT BUILDS (MONTHS 1-2, WEEKS 1-8) -----

From the first chat command to an automated farm: children learn sequencing, loops, variables, and decisions by making Minecraft do the building for them.

Month 1: First Commands and the Agent (Weeks 1-4)

Hello, Code Builder

Topics:

- Opening the coding editor inside Minecraft and a tour of the block palette
- On chat command: making code run when you type a magic word
- Placing your first block with code instead of your hands
- Running, stopping, and re-running a program
- Changing one block in the code and predicting what changes in the world
- Saving projects so next class starts where this one ended

Projects built:

- Magic words: three chat commands that summon a block tower, a splash of flowers, and a chicken

Practice: Invent two new chat commands of your own and draw what each one should do before coding it

The Agent, Your Robot

Topics:

- Meeting the agent: the robot that obeys your program
- Move forward, turn left, turn right: navigating step by step
- Agent place and agent destroy: building and clearing
- Sequencing: why the order of blocks changes everything
- Walking a maze on paper first, then in code
- Teleporting the agent back when it wanders off

Projects built:

- Maze runner: program the agent to walk a maze you built for it, placing a torch at the exit

Practice: Build a new maze for a classmate's agent and solve theirs, comparing step counts

Loops: Building at Superspeed

Topics:

- The repeat block: doing something ten times without ten copies
- Building a wall with a loop instead of block-by-block
- Loops inside loops: from a wall to a whole floor
- Counting carefully: the off-by-one surprise
- The forever loop and when it is useful
- Hand-build vs loop-build: racing the same wall both ways

Projects built:

- Wall factory: one program that builds a wall of any length, then a full square fort using nested loops

Practice: Use loops to build a staircase to the clouds, then estimate how long it would have taken by hand

Big Builds with Loops

Topics:

- Planning a build as layers: what repeats and what does not
- Pyramids: each layer smaller than the last
- Hollow shapes: skipping the middle to save blocks
- Combining agent moves and loops for long structures
- Debugging a build gone wrong: reading the code out loud
- Showing your build: a mini tour for the class

Projects built:

- Pyramid of Giza: a layered pyramid built entirely by code, with a hollow secret chamber inside

Practice: Design and code one original loop-built monument for the class world

Assessment: Month 1 build check: the agent completes a fresh maze and a loop-built structure, explained block by block

Month 2: Variables and Decisions (Weeks 5-8)

Variables: Code That Remembers

Topics:

- What a variable is: a labeled box the program remembers
- A size variable: one number that controls how big the build is
- Changing a variable while the program runs
- Counting things: how many blocks the agent has placed
- Announcing values in chat: the program reports back
- Naming variables so they explain themselves

Projects built:

- Any-size house: one program that builds a house small, medium, or giant by changing a single number

Practice: Add a height variable to your fort program and test it at three different values

Decisions: Code That Checks

Topics:

- If and else: teaching the program to choose
- Agent detect: is there a block in front of me?
- Obstacle dodging: walk forward, but go around what blocks you
- Checking what a block is made of before acting
- Combining checks: if this and not that
- Predict-then-run: guessing what a decision program will do

Projects built:

- Smart explorer: an agent that crosses rough ground by detecting obstacles and stepping around them

Practice: Program a tunnel-digger that stops when it detects water, so nobody floods the base

Events: Code That Reacts

Topics:

- On player walk: code that fires when someone steps somewhere
- On block placed and on block broken
- Building a pressure-plate trap the coding way
- Surprise effects: fireworks, sounds, and chat messages
- Being a fair trap-maker: warnings and escape routes
- Multiple events in one world without chaos

Projects built:

- Trap gauntlet: a walkway with three coded surprises, from harmless confetti to a drop into a pool

Practice: Add a welcome event to the class world that greets any player who arrives at your base

Project Week: The Auto-Farm

Topics:

- Automation: teaching the agent a full job, start to finish
- Till, plant, wait, harvest: the farming loop
- Combining loops, variables, and detection in one program
- Testing a long program one section at a time
- Making the farm bigger without rewriting it
- Explaining your automation to someone who does not code

Projects built:

- Agent auto-farm: a robot farmer that tills the soil, plants crops in rows, and harvests when ready

Practice: Extend your farm with a fence-building routine, then measure how many crops one run produces

Assessment: Month 2 skills check: a live automation challenge solved with loops, a variable, and at least one detect decision

----- PHASE 2: CODE THAT PLAYS (MONTHS 3-4, WEEKS 9-16) -----

The building skills become game-making skills: randomness, scores, and timers combine into

real mini-games, and the course ends with a student-designed capstone and a peek at real typed code.

Month 3: Mini-Games (Weeks 9-12)

Randomness

Topics:

- Random numbers: why games need surprise
- Spawning animals at random positions
- Random blocks: a patchwork floor no one can predict
- Hiding treasure somewhere new every game
- Fair vs unfair randomness in games
- Testing random code: run it many times, watch the spread

Projects built:

- Treasure hunt: a diamond block hidden at a random spot each round, with hot-and-cold chat hints

Practice: Make a random pet generator that spawns one of five animals, and tally twenty runs to see the spread

Keeping Score

Topics:

- A score variable that events update
- Announcing the score in chat after each point
- Win conditions: first to ten
- Resetting the game for a fresh round
- Counting down instead of up: lives
- Score displays that players actually notice

Projects built:

- Collector clash: gather the glowing blocks scattered by code, one point each, first to ten wins

Practice: Add a lives system to your treasure hunt: three wrong digs and the round ends

Timers and Pressure

Topics:

- Countdown timers built from loops and waits
- Beat the clock: finishing a task before time runs out
- Announcing time left at the right moments
- Combining a timer with a score for real tension
- Rounds: game start, game play, game over
- Tuning time limits so the game is hard but winnable

Projects built:

- Build rush: players race a two-minute timer to copy a target structure, with a code-checked finish

Practice: Playtest your build rush with family and adjust the time limit after two rounds

Project Week: The Arena Game

Topics:

- Combining the month: randomness, score, timer, and events in one game
- Building the arena with your loop skills
- Rules players can learn in ten seconds
- Running a game for classmates: host duties
- Watching players find the fun and the flaws
- One improvement pass after the first playtest

Projects built:

- Falling-floor arena: a coded mini-game where floor sections vanish on a timer and the last player standing wins

Practice: Host two rounds of your arena for classmates and write down the one rule that confused people most

Assessment: Arena game demo: classmates play a full round, then the builder explains the timer and event code behind it

Month 4: The Capstone and the Code Peek (Weeks 13-16)

Functions: Naming Your Powers

Topics:

- Functions: wrapping a build routine in a name
- One buildTower function used five times
- Functions with a size input
- A personal toolkit: your best routines, reusable forever
- Cleaner programs: reading code made of named pieces
- Sharing a function with a classmate and using theirs

Projects built:

- Village in a minute: houses, paths, and lamp posts, each a function, assembled into a whole village program

Practice: Turn your favorite build from the course into a function and use it three times in one world

The JavaScript Peek

Topics:

- Flipping the editor: the same program as blocks and as JavaScript text
- Finding your loop and your variable in the typed code
- What brackets, dots, and semicolons are doing
- Making one tiny change in text view and running it
- Block coding vs typed coding: same thinking, different clothes

- Where typed languages lead: Python, JavaScript, and beyond

Projects built:

- Two-view build: a fountain program written in blocks, then modified once in JavaScript view to change its size

Practice: Flip three of your old programs to code view and find the repeat loop in each one

Capstone Build Sprint

Topics:

- Choosing a capstone: an original game or a grand automation
- The one-page plan: what it does, what code it needs, what done means
- Building must-haves first
- Reusing your own functions and past code honestly
- Instructor check-in: unblocking the hard part early
- Trimming the plan when time runs short, without giving up the core

Projects built:

- Capstone, part one: the core of an original mini-game or automation, planned and working end to end

Practice: Two 30-minute build sessions at home, each ending with a one-sentence progress note

Finish and Showcase

Topics:

- The final bug hunt: testing every part on purpose
- Polish: sounds, messages, and a proper welcome
- Writing simple instructions for players
- Presenting: show the game first, then the code behind the best part
- Answering questions about your own program
- What comes next: Python for kids, or Roblox game coding

Projects built:

- Finished capstone: an original, tested Minecraft game or automation presented live to classmates and families

Practice: Run your capstone for two family members and collect one compliment and one improvement idea from each

Assessment: Capstone showcase and code Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Magic chat commands that summon builds and creatures
- Agent maze runner with a torch-lit victory exit
- Nested-loop square fort and cloud staircase
- Code-built Pyramid of Giza with a hidden chamber
- Any-size house driven by a single variable
- Smart explorer agent that detects and dodges obstacles
- Agent auto-farm that tills, plants, and harvests
- Random treasure hunt with hot-and-cold hints
- Falling-floor arena mini-game with timer and score
- Village-in-a-minute built from student-written functions
- Original capstone game or automation, showcased to families

Total projects: 14+ working programs, from a first chat command to an original mini-game

SKILLS THE STUDENT WILL HAVE AT THE END

- Sequencing and the agent: precise step-by-step instructions a robot can follow
- Loops and nested loops: building in seconds what hands build in hours
- Variables and decisions: programs that remember, count, check, and choose
- Events, randomness, scores, and timers: the ingredients of every mini-game
- Functions: naming and reusing routines like a real programmer
- Reading typed code: finding loops and variables in the JavaScript view of their own programs

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding challenges solved along with the instructor

Practice: 1-2 hours of build challenges between classes, sized for young attention spans

Review: Practice builds reviewed with feedback; tricky concepts replayed at the start of the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck until the next session

Progress updates: Regular progress notes to parents: what was covered, what was built, what needs practice

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, backed by the student's own gallery of working programs

WHERE THIS CAN LEAD

- Our Python for kids course, the natural next step once block logic is solid
- Roblox game coding, where the same ideas graduate to typed Luau scripts in a 3D engine
- Our Scratch-based creative coding track for students who want more game design first
- School coding competitions and clubs, entered with a portfolio of working programs
- App development for kids, for children who want to build beyond games

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course run by Mojang or Microsoft?

A: No. Modern Age Coders is an independent coding school, and this course is not affiliated with, sponsored by, or endorsed by Mojang or Microsoft. We teach coding using Minecraft Education and its built-in MakeCode editor, tools Microsoft makes available for exactly this kind of learning, but the curriculum and classes are entirely ours.

Q: Which version of Minecraft do we need? Will our normal Minecraft work?

A: The course uses Minecraft Education, the school-focused version with the coding editor (Code Builder) built in. The regular Bedrock or Java editions your child may already own do not include this editor, so Education is required; it runs on Windows, Mac, iPad, and most Chromebooks, and needs its own low-cost licence or a school Microsoft 365 Education account. Before the batch starts we walk every family through the setup, and we confirm your device runs it during the free demo.

Q: Will my child actually learn coding, or just play Minecraft in class?

A: Every class is a coding class; Minecraft is the place the code runs, not the activity. Students spend sessions writing programs: loops that raise walls, decisions that steer a robot, events that trigger traps, timers that run games. The game world is what makes a seven-year-old care whether their loop is correct, because a wrong loop builds a crooked castle in front of everyone.

Q: My child already gets too much screen time. Does this add more?

A: For most families it converts time rather than adds it. The hours a child was already spending in Minecraft start producing farms they automated, games they invented, and bugs they fixed, which exercises planning and logic in a way passive play does not. Classes are one focused hour, and home practice is deliberately kept to one or two hours a week, sized for this age group.

Q: Is 7 too young? My older child is 12, is it too basic?

A: Both ends of the range are served, differently. Sevens thrive because block coding removes typing and spelling from the equation; they drag, snap, and run. Twelves stay stretched through harder challenges, functions, and the JavaScript view, where they read and edit their own programs as typed code. Batches are grouped by age so the pace fits the room.

Q: What does my child move on to after this course?

A: The two most common paths are our Python for kids course, which takes the same ideas into the world's most-used typed language, and Roblox game coding, where students script 3D games in Luau. The instructor recommends a path for each child at the end of the course based on what they gravitated toward.

Q: Can we try a class before enrolling?

A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The demo doubles as a tech check: we make sure Minecraft Education runs on your device and answer setup questions with a parent present.

Q: Do parents need to know anything about coding or Minecraft?

A: No. Many parents tell us that presentation is the first time they understood what the course was building all along.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/minecraft-coding-for-kids-course>

COURSE 90 OF 118: MOBILE APP DEVELOPMENT FOR TEENS: PUBLISH TO APP STORES

Course link: <https://learn.modernagecoders.com/courses/complete-app-development-masterclass-for-teens>

Subject area: Teen Mobile App Development

Designed for: Teens

Who can join: Complete Beginner to Professional App Developer

Course length: 18 months (78 weeks)

Time needed: 12-15 hours/week recommended

Certificate: Professional Mobile App Developer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

An 18-month mobile app development course for teens: build iOS and Android apps with React Native and Flutter, then publish them to Google Play and App Store.

In one line: From 'What is an App?' to 'Building the Next TikTok'

ABOUT THE PROGRAMME

Ever wondered how Instagram, Snapchat, or your favorite games work? This isn't just about learning to code - it's about bringing YOUR app ideas to life! Designed specifically for teenagers, this 18-month journey transforms complete beginners into professional app developers who can build anything from addictive games to social media platforms. You'll master both iOS and Android development, create stunning UI/UX designs, integrate AI and AR features, implement monetization strategies, and actually publish your apps to millions of users. By the end, you'll have 25+ apps in your portfolio, real users for your published apps, and the skills to build your own tech startup or land high-paying developer jobs.

WHAT MAKES THIS COURSE DIFFERENT

- Teen-focused: Build apps teens actually want - games, social media, music, AR filters
- Zero to App Store: Start from scratch, end with published apps earning money
- Multi-platform mastery: iOS, Android, and cross-platform development
- Real apps, real users: Launch apps and get actual downloads
- Startup mindset: Learn app monetization, marketing, and business
- Project-based: Build TikTok clones, games, AR apps, AI chatbots
- Young mentors: Learn from successful young app developers
- Global teen community: Collaborate with teen developers worldwide

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-4): Programming basics, UI/UX design, Web technologies

Phase 2: Native Development (Months 5-9): Android with Kotlin, iOS with Swift

Phase 3: Cross-Platform (Months 10-13): React Native, Flutter, Backend integration

Phase 4: Advanced & Launch (Months 14-18): AI/AR, Games, Monetization, App Store

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No prerequisites - complete beginner friendly
Coding experience: Zero coding knowledge required
Equipment: Computer (Mac for iOS development) and smartphone
Time commitment: 12-15 hours per week
English: Basic English comprehension
Motivation: Passion for apps and technology

WHO THIS COURSE IS FOR

Students: Teenagers (13-19 years) interested in tech
Working professionals: Young adults wanting app development skills
Entrepreneurs: Teens with app ideas wanting to build them
Freelancers: Start earning while in school
Kids: Motivated pre-teens with parental support
Anyone: Anyone dreaming of creating the next big app

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & DESIGN THINKING (MONTHS 1-4, WEEKS 1-17) -----

Build rock-solid programming fundamentals and learn to think like an app designer. Create your first apps using web technologies.

Months 1-2: Programming Fundamentals & App Concepts (Week 1-8)

Introduction to App Development World

Topics:

- How apps changed the world: From Snake to TikTok
- Native vs Cross-platform vs Web apps explained
- iOS vs Android: The eternal rivalry
- How apps make money: Ads, subscriptions, in-app purchases
- Famous teen app developers who became millionaires
- Setting up your development environment
- Installing VS Code, Android Studio, Xcode preview
- Introduction to Terminal/Command Line
- Git and GitHub: Your code's backup and portfolio

- Creating developer accounts (Google, Apple developer preview)
- Your first HTML/CSS/JavaScript app
- How the App Store and Google Play work

Projects built:

- Research and present your favorite app's history
- Set up complete development environment
- Create your first web-based calculator app
- Design your dream app concept (mockup)

Practice: Daily: Analyze 3 apps you use - what makes them addictive?

JavaScript Programming Essentials

Topics:

- JavaScript: The language that powers everything
- Variables: Storing user data and game scores
- Data types: Numbers, strings, booleans with app examples
- Arrays: Managing Instagram posts, Spotify playlists
- Objects: Describing user profiles, game characters
- Functions: Reusable code for app features
- Conditionals: Making apps smart with if/else
- Loops: Processing TikTok feeds, game levels
- DOM manipulation: Making apps interactive
- Event handling: Responding to taps and swipes
- Local storage: Saving app data offline
- JSON: The data format apps love

Projects built:

- To-do list app with local storage
- Quiz game with score tracking
- Weather app using API
- Instagram-style photo filter app (basic)

Practice: Complete 50 JavaScript challenges on CodeWars

UI/UX Design Fundamentals

Topics:

- What makes apps beautiful and addictive?
- Design thinking process: Empathize, Define, Ideate
- User research: Understanding your target users (teens!)
- Color psychology: Why Instagram is purple-pink
- Typography: Choosing fonts that don't suck
- Mobile UI patterns: Bottom nav, cards, modals
- Figma basics: Professional design tool
- Creating wireframes and mockups
- Prototyping: Making designs clickable
- Design systems: Consistency across your app
- Accessibility: Designing for everyone
- Dark mode: Because we all love it

Projects built:

- Redesign a popular app for better UX
- Create complete UI kit in Figma
- Design your social media app mockup
- Build clickable prototype

Practice: Daily UI challenge - create one UI element per day

Advanced JavaScript & ES6+

Topics:

- Modern JavaScript features apps need
- Arrow functions: Cleaner, cooler syntax
- Destructuring: Extracting data elegantly
- Spread operator: Copying and merging data
- Template literals: Dynamic content creation
- Promises: Handling async operations
- Async/await: Making async code readable
- Fetch API: Getting data from servers
- Modules: Organizing code like a pro
- Classes: Object-oriented programming
- Error handling: When things go wrong
- Regular expressions: Pattern matching magic

Projects built:

- Movie database app with TMDB API
- Real-time chat app (web version)
- Music player with Spotify Web API
- Meme generator with API integration

Practice: Refactor old projects with modern JavaScript

Months 3-4: Web Apps & React Fundamentals (Week 9-17)

React.js Fundamentals

Topics:

- Why React? Facebook, Instagram, Netflix use it
- Vite: Starting projects quickly
- JSX: HTML in JavaScript (weird but awesome)
- Components: Building blocks of apps
- Props: Passing data between components
- State: Making components remember things

- Hooks: useState, useEffect explained simply
- Event handling in React
- Conditional rendering: Showing/hiding content
- Lists and keys: Rendering feeds and grids
- React DevTools: Debugging like a detective
- Component lifecycle: Birth to death of components

Projects built:

- Instagram clone (feed, stories, posts)
- Real-time scoreboard app
- Recipe app with search and filters
- Portfolio website in React

Practice: Build 20 React components for different use cases

React State Management & Routing

Topics:

- Managing complex state: When useState isn't enough
- useContext: Sharing data across components
- useReducer: Redux-like state management
- Custom hooks: Creating your own superpowers
- React Router: Multi-page apps
- Navigation patterns: Tabs, drawers, stacks
- Protected routes: Login-only pages
- URL parameters: Dynamic pages
- Local storage with React
- Form handling and validation
- Optimizing performance: React.memo, useMemo
- Code splitting: Faster app loading

Projects built:

- Multi-page social media app
- E-commerce app with cart
- Blog platform with authentication
- Task management app like Notion

Practice: Add routing and state management to previous projects

Progressive Web Apps (PWAs)

Topics:

- What are PWAs? Apps without app stores
- Service workers: Offline functionality
- Web app manifest: Installing on home screen
- Push notifications for web apps
- Caching strategies: Offline-first approach
- Background sync: Uploading when online
- PWA best practices
- Testing PWAs on mobile devices
- PWA vs native apps: Pros and cons
- Converting React apps to PWAs
- PWA deployment and hosting
- Analytics and monitoring for PWAs

Projects built:

- Convert React app to PWA
- Offline-first news app
- PWA game that works offline
- Push notification system

Practice: Make all web apps installable as PWAs

Backend Basics & Firebase

Topics:

- Why apps need backends: Where data lives
- Firebase: Google's backend for apps
- Authentication: Sign up, login, social auth
- Firestore: Real-time NoSQL database
- Cloud Storage: Storing images and videos
- Cloud Functions: Server code without servers
- Real-time features: Live chat, notifications
- Security rules: Protecting user data
- Firebase Analytics: Understanding users
- Firebase Hosting: Deploying apps
- Cost optimization: Avoiding surprise bills
- Firebase alternatives: Supabase, Appwrite

Projects built:

- Chat app with Firebase
- Instagram-like app with storage
- Collaborative todo app
- Social authentication system

Practice: Add Firebase to 3 existing projects

Phase 1 Capstone Project

Topics:

- Planning your first complete app
- User research and validation
- Technical architecture design
- Setting up development workflow
- Version control best practices

- Testing on real devices
- Projects built:
- CAPSTONE: Full-Stack Web App
 - Options: Social media clone, E-commerce platform, Productivity tool
 - Must include: Auth, database, real-time features, PWA
 - Deploy to production with real users
- Assessment: Phase 1 Final - Build complete web app from scratch

PHASE 2: NATIVE MOBILE DEVELOPMENT (MONTHS 5-9, WEEKS 18-39)

Master native Android and iOS development. Build apps that feel truly native with platform-specific features.

Months 5-6: Android Development with Kotlin (Week 18-26)

Android Studio & Kotlin Basics

Topics:

- Android ecosystem: Why 3 billion devices run Android
- Android Studio setup and tour
- Kotlin: Modern Android language
- Variables, functions, and classes in Kotlin
- Null safety: Avoiding crashes
- Android project structure explained
- Gradle: Building Android apps
- Running apps on emulator and real devices
- Android versions and API levels
- Material Design principles
- Debugging with Logcat
- Android app lifecycle

Projects built:

- Hello World Android app
- Simple calculator in Kotlin
- Tip calculator app
- Dice roller game

Practice: Complete Kotlin Koans exercises

Android UI Development

Topics:

- XML layouts: Designing Android UIs
- ViewGroups and Views explained
- LinearLayout, RelativeLayout, ConstraintLayout
- Material Design components
- RecyclerView: Displaying lists efficiently
- CardView: Beautiful card layouts
- Navigation: Activities and Fragments
- Bottom navigation and tabs
- Handling different screen sizes
- Themes and styles: Customizing appearance
- Animations and transitions
- Custom views and canvas drawing

Projects built:

- Instagram-style photo feed
- Music player UI
- Recipe app with categories
- Chat app interface

Practice: Recreate 10 popular app UIs

Android Data & Storage

Topics:

- SharedPreferences: Simple data storage
- Room database: SQLite made easy
- LiveData and ViewModel: MVVM architecture
- Data binding: Connecting UI and data
- Retrofit: Networking and APIs
- Image loading with Glide/Picasso
- JSON parsing with Gson
- File storage: Internal and external
- Content providers: Sharing data
- WorkManager: Background tasks
- Notifications: Local and push
- Android Jetpack components

Projects built:

- Note-taking app with Room
- Weather app with API
- News reader with offline cache
- Expense tracker with charts

Practice: Implement data persistence in all apps

Advanced Android Features

Topics:

- Camera API: Taking photos and videos
- Location services: GPS and maps

- Google Maps integration
- Sensors: Accelerometer, gyroscope
- Bluetooth connectivity
- Media playback: Audio and video
- Speech recognition and TTS
- Biometric authentication
- Android widgets: Home screen apps
- App shortcuts and deep links
- Share functionality
- Android permissions best practices

Projects built:

- Camera app with filters
- Location-based reminder app
- Fitness tracker using sensors
- Voice assistant app

Practice: Add advanced features to existing apps

Jetpack Compose (Modern Android UI)

Topics:

- Compose: Android's new UI toolkit
- Declarative UI vs XML layouts
- Composable functions
- State in Compose
- Layouts in Compose
- Material Design 3 in Compose
- Navigation in Compose
- Animations in Compose
- Compose with ViewModel
- Migration from XML to Compose
- Testing Compose UIs
- Compose best practices

Projects built:

- Twitter clone in Compose
- Todo app with Compose
- Meditation app UI
- Game menu in Compose

Practice: Rebuild previous apps with Compose

Months 7-8: iOS Development with Swift (Week 27-35)

Xcode & Swift Fundamentals

Topics:

- iOS ecosystem: Why developers love Apple
- Xcode setup and interface tour
- Swift: Apple's powerful language
- Swift Playgrounds: Learning interactively
- Variables, constants, and optionals
- Functions and closures in Swift
- Classes, structs, and enums
- Protocols and extensions
- Error handling in Swift
- iOS app structure: MVC pattern
- iOS Simulator: Testing without iPhone
- Debugging with Xcode

Projects built:

- First iOS app: I Am Rich parody
- Magic 8 Ball app
- Calculator with Swift
- Xylophone music app

Practice: Complete Swift Playgrounds challenges

iOS UI Development with UIKit

Topics:

- Interface Builder and Storyboards
- Auto Layout: Responsive design
- Stack Views: Simplifying layouts
- UITableView: Lists in iOS
- UICollectionView: Grids and custom layouts
- Navigation Controllers
- Tab Bar Controllers
- Segues and data passing
- Custom UI components
- Animations with UIView
- Gesture recognizers
- Dark mode support

Projects built:

- Instagram feed clone
- Weather app with beautiful UI
- News reader with categories
- Pinterest-style layout

Practice: Create 15 different iOS layouts

SwiftUI - Modern iOS Development

Topics:

- SwiftUI: The future of iOS development
- Declarative syntax explained
- Views and modifiers
- State and data flow
- @State, @Binding, @ObservedObject
- Lists and navigation
- Forms and user input
- Combining views
- Animations in SwiftUI
- Drawing and shapes
- SwiftUI with UIKit integration
- Preview and live development

Projects built:

- TikTok UI in SwiftUI
- Expense tracker with charts
- Recipe app with SwiftUI
- Game menu system

Practice: Rebuild UIKit apps in SwiftUI

iOS Data Persistence & Networking

Topics:

- UserDefaults: Simple storage
- Core Data: Apple's database framework
- SwiftData: Modern persistence
- Codable: JSON encoding/decoding
- URLSession: Networking in iOS
- Alamofire: Advanced networking
- Kingfisher: Image caching
- CloudKit: iCloud integration
- Keychain: Secure storage
- File management in iOS
- Background tasks
- Push notifications setup

Projects built:

- Notes app with Core Data
- GitHub client app
- Podcast player with downloads
- Expense tracker with CloudKit sync

Practice: Add persistence to all iOS apps

Advanced iOS Features

Topics:

- Camera and Photos framework
- Core Location: GPS and maps
- MapKit integration
- Core Motion: Accelerometer, gyroscope
- AVFoundation: Audio/video
- Speech framework: Recognition and synthesis
- Core ML: Machine learning
- ARKit basics: Augmented reality
- SiriKit: Siri integration
- WidgetKit: Home screen widgets
- Face ID and Touch ID
- Apple Pay integration

Projects built:

- AR measurement app
- Photo editor with filters
- Location-based game
- AI-powered camera app

Practice: Explore iOS frameworks in mini-projects

Month 9: Publishing & App Store (Week 36-39)

App Store Preparation

Topics:

- Apple Developer Program enrollment
- App Store guidelines: What Apple allows
- App Store Connect overview
- Certificates and provisioning profiles
- App icons and launch screens
- App Store screenshots and previews
- Writing compelling app descriptions
- Keywords and ASO (App Store Optimization)
- App categories and age ratings
- Pricing and monetization options
- TestFlight: Beta testing
- App review process tips

Projects built:

- Prepare app for App Store submission
- Create marketing materials
- Set up TestFlight beta

- Submit first app to App Store

Practice: Research successful app launches

Google Play Store Publishing

Topics:

- Google Play Console setup
- Play Store policies and guidelines
- App signing and security
- Store listing optimization
- Graphics and screenshots requirements
- Localization for global reach
- Play Store SEO strategies
- A/B testing store listings
- Staged rollouts
- Crash reporting and analytics
- User reviews management
- Play Store monetization

Projects built:

- Publish Android app to Play Store
- Implement crash reporting
- Set up analytics dashboard
- Create promotional materials

Practice: Analyze top apps in your category

App Monetization Strategies

Topics:

- Monetization models: Which works for teens?
- In-app purchases implementation
- Subscription models: The recurring revenue dream
- Ad integration: AdMob, Facebook Ads
- Rewarded video ads
- Freemium vs Premium strategies
- Consumable vs non-consumable purchases
- StoreKit for iOS purchases
- Google Play Billing
- Receipt validation
- Analytics for monetization
- A/B testing pricing

Projects built:

- Add in-app purchases to game
- Implement subscription system
- Integrate ads strategically
- Create monetization dashboard

Practice: Calculate revenue models for your apps

App Marketing & Growth

Topics:

- App marketing for zero budget
- Social media marketing: TikTok, Instagram
- Creating viral content
- Influencer partnerships
- App Store Optimization (ASO)
- Getting featured by Apple/Google
- Press kit creation
- Product Hunt launch
- Community building
- User retention strategies
- Push notification campaigns
- Referral programs

Projects built:

- Create marketing plan
- Build landing page
- Launch social media campaign
- Implement referral system

Practice: Market your published apps

Phase 2 Capstone

Topics:

- Planning a complete native app
- Platform choice: iOS vs Android
- Architecture design
- Timeline and milestones
- Beta testing strategy
- Launch preparation

Projects built:

- MAJOR CAPSTONE: Native Mobile App
- Build and publish complete iOS or Android app
- Must include: Modern UI, data persistence, networking, monetization
- Get 100+ downloads in first week

Assessment: Phase 2 Final - Published app with real users

Master cross-platform frameworks to build once and deploy everywhere. Learn React Native and Flutter.

Months 10-11: React Native Development (Week 40-48)

React Native Fundamentals

Topics:

- React Native: Write once, run everywhere
- How Facebook built React Native
- Setting up development environment
- Expo vs React Native CLI
- Core components: View, Text, Image, ScrollView
- Styling with Flexbox
- Platform-specific code
- React Navigation: Routing for mobile
- Bottom tabs, stack, and drawer navigation
- Handling user input
- Lists with FlatList and SectionList
- Pull to refresh and infinite scroll

Projects built:

- Instagram clone in React Native
- Todo app with navigation
- Weather app with location
- News feed with API

Practice: Convert React web apps to React Native

React Native Advanced Features

Topics:

- Native modules: Accessing device features
- Camera and image picker
- Geolocation and maps
- Push notifications setup
- AsyncStorage for persistence
- SQLite integration
- Animations with Animated API
- Gesture handler
- Lottie animations
- React Native Firebase
- Authentication flows
- Performance optimization

Projects built:

- Snapchat-like camera app
- Uber clone with maps
- Chat app with Firebase
- Fitness tracker with sensors

Practice: Add native features to all React Native apps

React Native UI Libraries & Styling

Topics:

- Native Base components
- React Native Elements
- React Native Paper (Material Design)
- Custom component libraries
- Themed components
- Responsive design for tablets
- Safe area handling
- Keyboard avoiding views
- Modal and overlay patterns
- Splash screens and app icons
- Building custom UI components
- Micro-interactions and feedback

Projects built:

- E-commerce app with beautiful UI
- Social media app with themes
- Music player interface
- Dating app UI

Practice: Create reusable component library

State Management & Testing

Topics:

- Redux in React Native
- Redux Toolkit setup
- Context API for state
- MobX state management
- Zustand for simplicity
- Redux Persist for offline
- Testing React Native apps
- Jest and React Native Testing Library
- Detox for E2E testing
- Debugging with Flipper
- Performance profiling
- Crash reporting with Sentry

Projects built:

- Complex app with Redux
- Offline-first app
- Test suite for existing app
- Performance optimization project

Practice: Add state management to all apps

Building & Deployment

Topics:

- Building for production
- Code signing for iOS
- Android APK and AAB
- Over-the-air updates with CodePush
- CI/CD for React Native
- Fastlane automation
- App Store deployment
- Play Store deployment
- Beta testing with TestFlight/Play Console
- Analytics integration
- Crash analytics
- Performance monitoring

Projects built:

- Deploy React Native app to both stores
- Set up CI/CD pipeline
- Implement OTA updates
- Create deployment automation

Practice: Deploy all React Native apps

Months 12-13: Flutter Development (Week 49-56)

Flutter & Dart Basics

Topics:

- Flutter: Google's UI toolkit
- Why Flutter? Single codebase, native performance
- Dart programming language basics
- Flutter architecture overview
- Setting up Flutter environment
- Hot reload: Instant gratification
- Widgets: Everything is a widget
- Stateless vs Stateful widgets
- Basic widgets: Container, Row, Column
- Material Design and Cupertino widgets
- Layout in Flutter
- Building your first Flutter app

Projects built:

- Flutter calculator app
- Business card app
- Dice game in Flutter
- Quiz app with scoring

Practice: Complete Flutter codelabs

Flutter UI Development

Topics:

- Advanced layouts with Flutter
- CustomPaint and canvas
- Animations in Flutter
- Hero animations
- Implicit and explicit animations
- Navigation and routing
- Named routes and arguments
- Bottom navigation and tabs
- Drawer and side menu
- Forms and validation
- Theming and dark mode
- Responsive design in Flutter

Projects built:

- TikTok UI clone in Flutter
- E-commerce app design
- Travel app with animations
- Banking app interface

Practice: Recreate 10 app UIs in Flutter

Flutter State Management

Topics:

- setState and StatefulWidget
- Provider package
- Riverpod state management
- GetX framework
- BLoC pattern
- Redux for Flutter
- MobX in Flutter
- State management comparison
- Dependency injection

- Service locator pattern
- MVVM architecture
- Clean architecture in Flutter

Projects built:

- Todo app with Provider
- Shopping cart with GetX
- Chat app with BLoC
- Social media with clean architecture

Practice: Implement different state management solutions

Flutter Backend Integration

Topics:

- HTTP requests with dio
- REST API integration
- JSON serialization
- Firebase with Flutter
- Authentication in Flutter
- Cloud Firestore integration
- Real-time features
- Push notifications
- Local storage: SharedPreferences
- SQLite with sqflite
- Hive database
- File upload and download

Projects built:

- News app with API
- Chat app with Firebase
- Note app with local database
- Social app with full backend

Practice: Connect Flutter apps to various backends

Flutter Advanced Features

Topics:

- Platform channels: Native code integration
- Camera and gallery access
- Location services
- Google Maps in Flutter
- Video player integration
- In-app purchases
- Ads integration
- WebView in Flutter
- Deep linking
- App lifecycle management
- Background tasks
- Flutter plugins development

Projects built:

- Camera app with filters
- Maps-based delivery app
- Video streaming app
- Game with monetization

Practice: Add advanced features to Flutter apps

Month 13: Cross-Platform Mastery (Week 53-56)

Advanced Cross-Platform Patterns

Topics:

- Choosing the right framework
- React Native vs Flutter comparison
- Code sharing strategies
- Platform-specific implementations
- Design system across platforms
- Performance optimization techniques
- Native module development
- Plugin/package creation
- Cross-platform testing strategies
- Continuous integration setup
- Multi-platform deployment
- Maintaining platform parity

Projects built:

- Same app in React Native and Flutter
- Create npm/pub package
- Cross-platform design system
- Platform comparison report

Practice: Master both frameworks equally

Backend as a Service (BaaS) Mastery

Topics:

- Firebase deep dive
- Supabase: Open source Firebase
- AWS Amplify for mobile
- Appwrite backend
- Hasura GraphQL
- Strapi headless CMS

- Authentication services
- Real-time synchronization
- Offline-first architecture
- Data syncing strategies
- Cost optimization
- Scaling considerations

Projects built:

- Multi-user collaboration app
- Offline-first note app
- Real-time gaming backend
- Social platform with BaaS

Practice: Compare different BaaS providers

App Analytics & Performance

Topics:

- Google Analytics for apps
- Firebase Analytics setup
- Custom event tracking
- User behavior analysis
- Funnel optimization
- A/B testing in apps
- Performance monitoring
- Crash reporting and debugging
- User session recording
- Heatmaps for mobile
- Attribution tracking
- Revenue analytics

Projects built:

- Analytics dashboard
- A/B testing framework
- Performance monitoring setup
- User behavior tracker

Practice: Add comprehensive analytics to all apps

Cross-Platform Game Development

Topics:

- Unity basics for mobile
- 2D game development
- Flame engine for Flutter
- React Native Game Engine
- Physics and collisions
- Sprite animations
- Game state management
- Leaderboards and achievements
- Multiplayer basics
- Game monetization
- Publishing games
- Game analytics

Projects built:

- 2D platformer game
- Puzzle game like 2048
- Multiplayer card game
- Endless runner game

Practice: Create 5 mini-games

Phase 3 Capstone Project

Topics:

- Planning cross-platform app
- Choosing the right framework
- Architecture for scale
- Backend selection
- Deployment strategy
- Marketing plan

Projects built:

- MAJOR CAPSTONE: Cross-Platform App
- Build same app for iOS, Android, Web
- Include: Complex UI, real-time features, offline support, monetization
- Launch on all platforms simultaneously

Assessment: Phase 3 Final - Multi-platform app with 500+ users

PHASE 4: ADVANCED TECHNOLOGIES & LAUNCH (MONTHS 14-18, WEEKS 57-78)

Master cutting-edge technologies, build viral apps, and launch your startup.

Months 14-15: AI, AR & Advanced Features (Week 57-65)

AI Integration in Mobile Apps

Topics:

- AI/ML basics for mobile developers
- OpenAI API integration (ChatGPT)
- Google ML Kit features
- Core ML for iOS apps

- TensorFlow Lite for mobile
- Image recognition and classification
- Text recognition (OCR)
- Natural language processing
- Sentiment analysis
- Face detection and recognition
- Object detection in real-time
- On-device vs cloud ML

Projects built:

- AI chatbot app
- Smart camera with object detection
- Language translator app
- AI study buddy app

Practice: Add AI features to 5 existing apps

Augmented Reality (AR) Development

Topics:

- AR fundamentals: How Pokemon Go works
- ARCore for Android
- ARKit for iOS
- Unity AR Foundation
- React Native AR libraries
- Flutter AR plugins
- 3D models and scenes
- Plane detection and tracking
- Image targets and markers
- Face filters like Snapchat
- World tracking and SLAM
- AR Cloud and persistence

Projects built:

- AR furniture placement app
- Face filter app
- AR gaming experience
- Educational AR app

Practice: Create 10 AR experiences

Advanced Gaming Features

Topics:

- Real-time multiplayer with Photon
- Game Center and Google Play Games
- Leaderboards and achievements
- In-game chat systems
- Matchmaking algorithms
- Game physics engines
- Procedural generation
- Game AI and behaviors
- Virtual economies
- Loot boxes and rewards
- Season pass systems
- Game analytics and metrics

Projects built:

- Multiplayer battle game
- Social casino game
- MOBA-style game
- Battle royale mini-game

Practice: Add multiplayer to existing games

IoT & Wearables Integration

Topics:

- Internet of Things basics
- Bluetooth Low Energy (BLE)
- Connecting to Arduino/Raspberry Pi
- Smart home integration
- Apple Watch development
- Wear OS development
- Fitness tracker integration
- Health data with HealthKit/Google Fit
- HomeKit and Google Home
- MQTT protocol for IoT
- Edge computing concepts
- 5G and mobile apps

Projects built:

- Smart home controller app
- Fitness app with wearables
- IoT dashboard app
- Apple Watch companion app

Practice: Build IoT control apps

AI Features & Agents in Mobile Apps

Topics:

- Calling an AI API from a mobile app
- Why your API key must never ship in the app
- Streaming responses so the UI does not freeze

- On-device vs cloud inference, and the trade-off
- Image and voice input as first-class features
- Function calling: letting the model use your app's features
- Building a small agent loop: plan, act, check
- Handling refusals, timeouts and hallucinated answers
- Caching and cost: what an AI feature actually costs per user
- Rate limits and graceful degradation
- Privacy: what leaves the device, and what must not
- Telling users when they are talking to an AI

Projects built:

- AI assistant app
- Photo-to-text app
- Voice agent app
- AI feature cost dashboard

Practice: Add AI features to 3 apps

Months 16-17: Startup & Business Skills (Week 66-74)

App Business Fundamentals

Topics:

- Validating app ideas: Will teens use it?
- Market research and competition analysis
- Creating business model canvas
- Revenue models for apps
- Unit economics: CAC, LTV, churn
- Pricing strategies
- Freemium vs paid apps
- Subscription psychology
- Creating pitch decks
- Talking to investors (even as a teen)
- Legal basics: Terms of service, privacy
- Intellectual property protection

Projects built:

- Business plan for your app
- Pitch deck creation
- Financial projections
- Legal documents template

Practice: Analyze 10 successful app businesses

Growth Hacking & Viral Marketing

Topics:

- Growth hacking for zero budget
- Viral coefficient and K-factor
- Building viral loops
- Social sharing mechanics
- Referral programs that work
- Influencer partnerships for teens
- TikTok marketing strategies
- Instagram growth tactics
- Community building on Discord
- Product Hunt launches
- App Store featuring tactics
- PR without paying agencies

Projects built:

- Viral marketing campaign
- Referral system implementation
- Social media strategy
- Launch plan creation

Practice: Growth hack your published apps

User Psychology & Retention

Topics:

- Psychology of addictive apps
- Hook model: Trigger, Action, Reward, Investment
- Gamification principles
- Onboarding that converts
- Push notification psychology
- Social proof and FOMO
- Variable reward schedules
- Building habits in users
- Retention metrics and cohorts
- Reducing churn rate
- Win-back campaigns
- User segmentation strategies

Projects built:

- Gamification system
- Onboarding flow redesign
- Retention improvement plan
- Habit-forming features

Practice: Apply psychology to your apps

Scaling & Team Building

Topics:

- When to scale your app
- Technical scaling challenges
- Building a development team
- Remote collaboration tools
- Project management for apps
- Agile/Scrum for small teams
- Code review processes
- Documentation best practices
- Customer support systems
- Community management
- Hiring your first employees
- Equity and compensation

Projects built:

- Scaling plan document
- Team collaboration setup
- Support system implementation
- Community platform launch

Practice: Collaborate on group project

Advanced Monetization & Analytics

Topics:

- Advanced monetization strategies
- Dynamic pricing models
- Cohort-based analytics
- LTV prediction models
- Churn prediction and prevention
- Revenue optimization
- Ad mediation platforms
- Subscription optimization
- International monetization
- Platform fee optimization
- Financial reporting
- Investor metrics and KPIs

Projects built:

- Revenue optimization system
- Analytics dashboard
- Investor report template
- Monetization A/B tests

Practice: Optimize revenue for all apps

Month 18: Final Projects & Launch (Week 75-78)

Final Project Planning

Topics:

- Choosing your magnum opus
- Market validation
- Technical architecture
- Design system creation
- Development timeline
- Resource planning
- Risk assessment
- Success metrics definition

Projects built:

- FINAL PROJECT: Your Startup App
- Complete business plan
- Technical specification
- Design mockups
- Development roadmap

Practice: Research and planning sprint

Final Project Development

Topics:

- Agile development sprints
- Core feature implementation
- UI/UX implementation
- Backend integration
- Testing and debugging
- Performance optimization
- Security implementation
- Beta version ready

Projects built:

- MVP development
- Alpha testing
- Core features complete
- Basic monetization

Practice: Rapid development techniques

Launch Preparation

Topics:

- Pre-launch marketing
- Beta testing program
- App Store optimization
- Press kit preparation

- Influencer outreach
- Launch day planning
- Support system setup
- Analytics implementation

Projects built:

- Marketing campaign launch
- Beta test with 100 users
- Store listings optimized
- Launch event planning

Practice: Launch preparation checklist

Launch Week & Celebration

Topics:

- Launch day execution
- Real-time monitoring
- User feedback management
- Quick iteration and fixes
- Media outreach
- Community engagement
- Metrics tracking
- Post-launch analysis

Projects built:

- LAUNCH YOUR APP!
- Get 1000+ downloads
- Media coverage
- User testimonials
- Revenue generation

Practice: Launch and iterate

Weeks 77-78: Career Launch & Beyond (Week 77-78)

Portfolio & Career Preparation

Topics:

- Building killer portfolio
- GitHub profile optimization
- LinkedIn for teen developers
- Personal brand building
- Blog writing about your apps
- YouTube channel for developers
- Conference speaking (teen tracks)
- Open source contributions
- Freelancing as a teen
- College applications with apps
- Internship applications
- Startup vs corporate jobs

Projects built:

- Portfolio website
- Video demos of all apps
- Technical blog posts
- Open source contributions

Deliverables:

- Professional portfolio
- 15+ published apps
- 10,000+ total downloads
- Revenue-generating apps

Future Technologies & Continued Learning

Topics:

- Future of app development
- Quantum computing basics
- Brain-computer interfaces
- Extended Reality (XR)
- 6G and future connectivity
- Voice-first interfaces
- Ambient computing
- Sustainable app development
- Privacy-first development
- Regulation and compliance future
- Continuous learning strategies
- Building your tech network

Projects built:

- Future tech prototype
- Learning roadmap
- Network building plan
- Next big idea

Deliverables:

- Future learning plan
- Tech community connections
- Mentorship relationships
- Ongoing projects

Graduation & Celebration

Topics:

- Course completion ceremony
- Final presentations
- Peer app reviews
- Success story sharing
- Alumni network induction
- Continued support access
- Future collaboration opportunities
- Celebration and reflection

Deliverables:

- Course completion certificate
- Professional Mobile App Developer Certification
- Letter of recommendation
- Alumni network access
- Lifetime course updates
- Success story feature

Assessment: FINAL SHOWCASE: Present your app empire to investors and industry professionals

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 10+ Web apps, PWAs, Firebase projects
- Phase 2: 8+ Native Android apps, 8+ iOS apps, Published apps
- Phase 3: 10+ Cross-platform apps with React Native and Flutter
- Phase 4: AR apps, AI apps, Games, Blockchain apps, Final startup
- Total: 40+ complete apps from simple to complex

Total projects: 40+ apps across all platforms, 10+ published to stores

SKILLS THE STUDENT WILL HAVE AT THE END

- Languages: JavaScript, TypeScript, Kotlin, Swift, Dart, Python
- Frontend: React, React Native, SwiftUI, Jetpack Compose, Flutter
- Backend: Firebase, Node.js, REST APIs, GraphQL, WebSockets
- Databases: Firestore, SQLite, Core Data, Realm, Cloud databases
- Design: Figma, UI/UX principles, Material Design, iOS HIG
- Platforms: iOS, Android, Web, PWAs, Cross-platform
- Advanced: AI/ML, AR, Blockchain, IoT, Gaming
- Business: Monetization, Marketing, Analytics, Growth, Startups
- Tools: Git, VS Code, Xcode, Android Studio, Terminal
- Publishing: App Store, Google Play, Marketing, ASO

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-5 hours
 Hands on coding: 6-8 hours
 Projects: 3-4 hours
 Practice problems: 2-3 hours
 Total per week: 12-15 hours

SUPPORT INCLUDED

Live sessions: Weekly group mentoring with young developers
 Mentorship: 1-on-1 guidance from successful app developers
 Community: Global teen developer Discord community
 Code review: Expert feedback on all projects
 Startup support: Incubation for promising app ideas
 Lifetime access: All content, updates, new technologies
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Final certificate: Professional App Developer Certification
 LinkedIn badge: Verified LinkedIn certification
 Apple certification: Preparation for Apple certification
 Google certification: Preparation for Google certification
 Portfolio apps: 20+ apps to showcase

WHERE THIS CAN LEAD

- Mobile App Developer (iOS/Android)
- Cross-Platform Developer
- Frontend Mobile Engineer
- Full Stack App Developer
- AR/VR Developer
- Game Developer
- App Development Freelancer
- Technical Founder/CEO
- Product Manager

- UX/UI Designer for Mobile

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen learn in this app development course?

A: The 18 month journey covers programming fundamentals and UI/UX design, native Android with Kotlin and iOS with Swift, cross-platform development with React Native and Flutter, then AI and AR features, app monetization and publishing to Google Play and the App Store. It ends with real, store-ready apps.

Q: Is this suitable for a complete beginner?

A: Yes, it is built for teenagers starting from zero coding knowledge; the first months teach programming through app concepts teens actually care about. The small group format lets the pace adjust to each student.

Q: Do we need a Mac or special equipment?

A: A regular computer and a smartphone cover most of the course, including all Android, React Native and Flutter work. Building native iOS apps with Swift does require a Mac, so if you do not have one, the mentor weights that phase toward Android while still teaching the iOS concepts, and no learning is lost.

Q: How do we track progress over 18 months?

A: Every phase ships working apps, from first web apps to native and cross-platform builds, so progress is measured in things your teen can demo. The mentor keeps a project record, and the final phase includes the publishing process, the clearest milestone there is.

Q: Will my teen actually publish an app?

A: Publishing to Google Play and the App Store is taught as part of the final phase, including store listings and release processes. Whether a specific app goes live depends on store policies, developer account setup and family choice, but every student finishes with store-ready apps and full knowledge of how to launch them.

Q: Can my teen attend a free class first?

A: Yes, every student starts with a free demo class, no commitment: book at learn.modernagecoders.com/contact or WhatsApp +91 91233 66161. The demo lets your teen meet the teacher and build something small in the very first hour.

Q: Are classes live or recorded?

A: Live, always: one hour online sessions with a real teacher in a small group, never pre-recorded videos. Teens code during class, share their screens and get unstuck in real time.

Q: Is there a certificate at the end?

A: Yes, a Mobile App Developer certification on completion, plus something more useful: a portfolio of working apps your teen designed, built and can demo to anyone, including future colleges and employers.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-app-development-masterclass-for-teens>

=====

COURSE 91 OF 118: MYSQL DATABASE MASTERCLASS: SQL, DESIGN & ADMINISTRATION

=====

Course link: <https://learn.modernagecoders.com/courses/mysql-database-complete-masterclass-college>

Subject area: Database Management & Administration

Designed for: College students

Who can join: Complete Beginner to Database Expert

Course length: 12 months (52 weeks)

Time needed: 12-15 hours/week recommended

Certificate: Certified MySQL Database Administrator upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live 12-month MySQL course covering SQL queries, database design, administration, performance tuning, replication, security and cloud deployment.

In one line: From First Query to Enterprise Database Architecture

ABOUT THE PROGRAMME

This is not just a course, it's your complete transformation into a database expert. MySQL powers millions of applications worldwide, from small websites to massive enterprise systems. This 12-month masterclass takes you from absolute beginner to MySQL expert, capable of designing, implementing, optimizing, and managing database systems at any scale. You'll master SQL programming, database design, performance tuning, high availability, security, and cloud deployment. Whether you aim to become a DBA, backend developer, or data architect, this course provides everything you need.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no database knowledge required
- Complete 12-month structured curriculum covering all aspects
- Hands-on with real-world database scenarios
- Covers MySQL 8.0+ with latest features
- Performance tuning and optimization deep dive
- High availability and disaster recovery included
- Cloud deployment on AWS RDS, Google Cloud SQL, Azure
- Integration with Python, Java, PHP, Node.js
- 30+ real database projects
- Industry-standard tools and best practices

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): SQL Basics, Database Fundamentals, Simple Queries
Phase 2: Intermediate (Months 4-6): Advanced SQL, Database Design, Programming
Phase 3: Advanced (Months 7-9): Administration, Performance, Security
Phase 4: Expert (Months 10-12): Replication, High Availability, Cloud, Architecture

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal education required
Coding experience: None required - we start from basics
Equipment: Computer with 4GB+ RAM, any OS
Time commitment: 12-15 hours per week
English: Basic reading and comprehension
Motivation: Interest in data and database systems

WHO THIS COURSE IS FOR

Students: CS students wanting database expertise
Working professionals: Developers wanting to master databases
Entrepreneurs: Building data-driven applications
Freelancers: Offering database services
Kids: Not suitable - recommended age 16+
Anyone: Anyone working with data and applications

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: SQL FUNDAMENTALS & DATABASE BASICS (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid foundations in SQL, understand relational database concepts, and master basic to intermediate queries.

Months 1-2: Database Fundamentals & Basic SQL (Week 1-8)

Introduction to Databases & MySQL Setup

Topics:

- What are databases and why we need them
- Relational vs Non-relational databases
- History and evolution of MySQL
- MySQL vs other databases (PostgreSQL, Oracle, SQL Server)
- MySQL ecosystem: Community, Enterprise, MariaDB, Percona
- Installing MySQL on Windows, Linux, Mac
- MySQL Workbench installation and setup
- Command-line client basics
- phpMyAdmin and other GUI tools
- Creating your first database
- Understanding schemas and instances
- Basic security during installation

Projects built:

- Install MySQL on local machine
- Set up MySQL Workbench
- Create first database and tables
- Configure basic security settings

Practice: Complete 20 basic database operations

SQL Basics - Data Definition Language (DDL)

Topics:

- Creating databases with CREATE DATABASE
- Dropping databases safely
- Creating tables with CREATE TABLE
- Data types: Numeric (INT, DECIMAL, FLOAT)
- String types (CHAR, VARCHAR, TEXT)
- Date and time types (DATE, TIME, DATETIME, TIMESTAMP)
- Boolean and ENUM types
- Understanding NULL and NOT NULL
- Primary keys and their importance
- AUTO_INCREMENT for automatic numbering
- Altering tables with ALTER TABLE
- Dropping and truncating tables

Projects built:

- Design employee database schema
- Create online store database structure
- Build student management system tables
- Library management database design

Practice: Create 30 different table structures

SQL Basics - Data Manipulation Language (DML)

Topics:

- Inserting data with INSERT
- Bulk insert operations
- Updating records with UPDATE
- Safe update practices
- Deleting data with DELETE
- TRUNCATE vs DELETE
- Basic SELECT queries
- Selecting specific columns
- WHERE clause for filtering
- Comparison operators (=, !=, <, >, <=, >=)
- Logical operators (AND, OR, NOT)
- IN and BETWEEN operators

Projects built:

- Populate employee database with data
- Create product catalog with 1000+ items
- Build customer records system
- Inventory management data setup

Practice: Write 100 INSERT, UPDATE, DELETE queries

Querying Data - SELECT Mastery

Topics:

- ORDER BY for sorting results
- LIMIT for pagination
- DISTINCT for unique values
- Aliases with AS
- String functions: CONCAT, LENGTH, UPPER, LOWER
- LIKE operator for pattern matching
- Wildcards (% and _)
- Regular expressions in MySQL
- Date functions: NOW, CURDATE, DATE_FORMAT
- Mathematical functions: SUM, AVG, COUNT, MIN, MAX
- GROUP BY for aggregation
- HAVING clause for group filtering

Projects built:

- Sales reporting queries
- Customer analytics dashboard queries
- Product search functionality
- Date-based report generation

Practice: Master 150 SELECT query variations

Month 3: Relationships & Joins (Week 9-13)

Database Relationships

Topics:

- Understanding table relationships
- One-to-One relationships
- One-to-Many relationships
- Many-to-Many relationships
- Foreign keys and referential integrity
- CASCADE options (DELETE, UPDATE)
- Creating foreign key constraints
- Database normalization introduction
- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Denormalization when and why

Projects built:

- Design blog database with posts and comments
- E-commerce database with orders and products
- School database with students and courses
- Social media database schema

Practice: Design 20 relational database schemas

Mastering Joins

Topics:

- Understanding JOIN operations
- INNER JOIN deep dive
- LEFT JOIN (LEFT OUTER JOIN)
- RIGHT JOIN (RIGHT OUTER JOIN)
- CROSS JOIN
- Self joins for hierarchical data
- Multiple table joins
- Using aliases in joins
- Natural joins
- Join performance considerations
- UNION operations
- INTERSECT and EXCEPT alternatives

Projects built:

- Employee-Department-Salary reports
- Order-Customer-Product analysis
- Multi-table inventory system
- Hierarchical organization chart

Practice: Write 100 complex JOIN queries

Subqueries & Set Operations

Topics:

- Understanding subqueries
- Subqueries in SELECT clause
- Subqueries in FROM clause
- Subqueries in WHERE clause
- Correlated vs non-correlated subqueries
- EXISTS and NOT EXISTS
- ANY, ALL, SOME operators
- Subqueries vs JOINS performance
- Common Table Expressions (CTEs)
- Recursive CTEs
- UNION and UNION ALL
- Combining multiple result sets

Projects built:

- Complex reporting with subqueries
- Hierarchical data queries with CTEs
- Performance comparison study
- PHASE 1 CAPSTONE: Complete Database Application

Assessment: Phase 1 comprehensive SQL exam

----- PHASE 2: ADVANCED SQL & DATABASE DESIGN (MONTHS 4-6, WEEKS 14-26) -----

Master advanced SQL features, stored procedures, triggers, and professional database design.

Months 4-5: Advanced SQL Programming (Week 14-22)

Views & Indexes

Topics:

- Creating and using views
- Updatable vs non-updatable views
- View algorithms (MERGE, TEMPTABLE)
- Security with views
- Materialized views alternatives
- Understanding indexes
- B-Tree vs Hash indexes
- Creating indexes strategically
- Composite indexes
- Unique indexes
- Full-text indexes
- Spatial indexes
- Index hints
- Analyzing index usage

Projects built:

- Create view layer for application
- Index optimization for slow queries
- Full-text search implementation
- Performance improvement study

Practice: Create 50 views and optimize with indexes

Stored Procedures

Topics:

- Introduction to stored procedures
- Creating procedures with CREATE PROCEDURE
- IN, OUT, INOUT parameters
- Variables and data types
- Control flow: IF, CASE, LOOP, WHILE
- REPEAT and LEAVE statements
- Error handling with DECLARE HANDLER

- Cursors for row-by-row processing
- Nested procedures
- Procedure security and permissions
- Debugging stored procedures
- Best practices for procedures
- Performance implications
- Version control for database code

Projects built:

- Order processing system procedures
- Automated reporting procedures
- Data validation procedures
- Batch processing system

Practice: Write 40 stored procedures

Functions & Triggers

Topics:

- User-defined functions (UDFs)
- Scalar vs table functions
- Deterministic vs non-deterministic
- Function syntax and parameters
- Built-in function categories
- Creating custom functions
- Triggers introduction
- BEFORE vs AFTER triggers
- INSERT, UPDATE, DELETE triggers
- NEW and OLD keywords
- Multiple triggers per event
- Trigger use cases and limitations
- Auditing with triggers
- Data validation triggers

Projects built:

- Custom function library
- Audit trail system with triggers
- Automatic data synchronization
- Business rule enforcement system

Practice: Create 30 functions and 20 triggers

Transactions & Concurrency

Topics:

- ACID properties explained
- Starting transactions: BEGIN, START TRANSACTION
- COMMIT and ROLLBACK
- Savepoints in transactions
- Autocommit mode
- Isolation levels: READ UNCOMMITTED, READ COMMITTED
- REPEATABLE READ, SERIALIZABLE
- Dirty reads, non-repeatable reads, phantom reads
- Locking mechanisms in MySQL
- Table locks vs row locks
- Deadlock detection and resolution
- Optimistic vs pessimistic locking
- Lock wait timeouts
- Transaction best practices

Projects built:

- Banking transaction system
- Inventory management with concurrency
- Booking system with lock handling
- Deadlock simulation and resolution

Practice: Implement 20 transactional scenarios

Advanced Database Design

Topics:

- Database design methodology
- Requirements gathering
- Conceptual design with ER diagrams
- Logical design and normalization
- Physical design considerations
- Denormalization strategies
- Partitioning tables
- Vertical vs horizontal partitioning
- Sharding concepts
- Archive strategies
- Multi-tenant database design
- Temporal database design
- Database refactoring
- Migration strategies

Projects built:

- Design multi-tenant SaaS database
- Implement table partitioning
- Create temporal data system
- Database migration project

Practice: Design 10 complex database systems

Month 6: Programming Integration (Week 23-26)

MySQL with Python

Topics:

- Python MySQL connectors
- PyMySQL vs mysql-connector-python
- Establishing connections
- Executing queries from Python
- Prepared statements
- Fetching results
- Connection pooling
- ORM with SQLAlchemy
- Database migrations with Alembic
- Async database operations
- Error handling in Python
- Transaction management
- Building data access layer
- Best practices for Python-MySQL

Projects built:

- Python database application
- REST API with MySQL backend
- Data migration script
- ORM-based application

Practice: Build 10 Python-MySQL applications

MySQL with Other Languages

Topics:

- JDBC for Java applications
- Connection pooling in Java
- PHP and MySQL integration
- PDO vs mysqli
- Node.js MySQL libraries
- Async operations in Node.js
- C# and .NET with MySQL
- Entity Framework integration
- Ruby on Rails with MySQL
- Go database/sql package
- Connection management patterns
- Query builders vs raw SQL
- Cross-language best practices
- Performance considerations

Projects built:

- Java web application with MySQL
- Node.js REST API
- PHP web application
- Multi-language database library

Practice: Integrate MySQL with 5 languages

NoSQL Features in MySQL

Topics:

- JSON data type in MySQL
- JSON functions and operators
- Indexing JSON columns
- Virtual columns from JSON
- Document store with MySQL
- X DevAPI introduction
- Collections and documents
- CRUD operations on documents
- MySQL as document database
- Combining SQL and NoSQL
- Performance of JSON operations
- When to use JSON vs tables
- Migration from NoSQL to MySQL
- Best practices for JSON data

Projects built:

- Document store application
- Hybrid SQL-NoSQL system
- JSON-based configuration system
- Flexible schema application

Practice: Build 5 JSON-based features

Testing & Version Control

Topics:

- Database testing strategies
- Unit testing database code
- Integration testing
- Test data generation
- Database mocking
- Performance testing basics
- Load testing with mysqlslap
- Version control for databases
- Schema migration tools

- Liquibase and Flyway
- Database CI/CD pipelines
- Automated deployments
- Rollback strategies
- Documentation practices

Projects built:

- Test suite for database
- CI/CD pipeline setup
- Migration system implementation
- Automated testing framework

Practice: Create comprehensive test coverage

Phase 2 Capstone Project

Topics:

- Complex application design
- Advanced SQL implementation
- Stored procedure development
- Integration with application
- Testing and optimization

Projects built:

- PHASE 2 CAPSTONE: Enterprise Database System
- Build complete database for business application
- Include: procedures, functions, triggers, views
- Integrate with web application

Assessment: Phase 2 exam on advanced SQL and programming

PHASE 3: ADMINISTRATION & PERFORMANCE (MONTHS 7-9, WEEKS 27-39)

Master database administration, performance tuning, security, and maintenance.

Months 7-8: Database Administration (Week 27-35)

MySQL Architecture & Storage Engines

Topics:

- MySQL server architecture
- Connection layer and thread handling
- Query parser and optimizer
- Storage engine layer
- InnoDB architecture deep dive
- Buffer pool and caching
- InnoDB vs MyISAM comparison
- Memory storage engine
- Archive and CSV engines
- Choosing the right storage engine
- Storage engine conversion
- File structure on disk
- Tablespaces and data files
- Transaction logs and redo logs

Projects built:

- Storage engine comparison study
- Architecture documentation
- Engine conversion project
- Performance analysis by engine

Practice: Work with 5 different storage engines

User Management & Security

Topics:

- MySQL security model
- User account management
- CREATE USER and ALTER USER
- Password policies and validation
- Authentication plugins
- Privileges and GRANT system
- Database, table, column privileges
- Stored routine privileges
- Proxy users and roles
- SSL/TLS configuration
- Encrypted connections
- Transparent data encryption
- Audit logging
- Security best practices

Projects built:

- Implement role-based access control
- SSL setup for production
- Security audit of database
- User management system

Practice: Configure 20 different security scenarios

Backup & Recovery

Topics:

- Backup strategies overview
- Logical vs physical backups

- mysqldump for logical backups
- Backup options and optimization
- Percona XtraBackup
- Hot backups with InnoDB
- Incremental backups
- Point-in-time recovery
- Binary log backup
- Backup scheduling and automation
- Testing backup integrity
- Recovery procedures
- Disaster recovery planning
- Backup storage strategies

Projects built:

- Automated backup system
- Point-in-time recovery test
- Disaster recovery plan
- Backup monitoring dashboard

Practice: Perform 20 backup and recovery scenarios

Performance Tuning Basics

Topics:

- Performance tuning methodology
- Identifying bottlenecks
- EXPLAIN and query analysis
- EXPLAIN FORMAT=JSON
- Visual EXPLAIN in Workbench
- Slow query log analysis
- Performance Schema introduction
- Key performance metrics
- Buffer pool tuning
- Query cache (deprecated but historical)
- Connection pool tuning
- Thread configuration
- I/O optimization
- Operating system tuning

Projects built:

- Performance baseline establishment
- Slow query optimization
- Server tuning project
- Performance monitoring setup

Practice: Optimize 50 slow queries

Monitoring & Maintenance

Topics:

- Monitoring strategies
- MySQL Enterprise Monitor
- Open source monitoring tools
- Prometheus and Grafana
- Key metrics to monitor
- Alert configuration
- Health checks
- Maintenance tasks
- Table maintenance: ANALYZE, OPTIMIZE
- Index maintenance
- Log rotation
- Disk space management
- Upgrade procedures
- Patch management

Projects built:

- Monitoring dashboard creation
- Alert system implementation
- Maintenance automation
- Health check system

Practice: Set up comprehensive monitoring

Month 9: Advanced Performance (Week 36-39)

Advanced Performance Tuning

Topics:

- Query optimization techniques
- Index optimization strategies
- Covering indexes
- Index condition pushdown
- Multi-range read optimization
- Join optimization
- Subquery optimization
- Derived table optimization
- Optimizer hints
- Cost-based optimization
- Statistics and histograms
- Parallel query execution
- Resource groups

- Performance regression testing

Projects built:

- Complex query optimization
- Index strategy redesign
- Performance regression suite
- Optimization case studies

Practice: Optimize 30 complex scenarios

Performance Schema & Sys Schema

Topics:

- Performance Schema architecture
- Instrumentation and consumers
- Event collection and filtering
- Statement analysis
- Wait event analysis
- Lock wait analysis
- Memory instrumentation
- File I/O instrumentation
- Sys Schema views
- Diagnostic procedures
- Performance reports
- Custom instrumentation
- Integration with monitoring
- Historical performance data

Projects built:

- Performance analysis toolkit
- Custom performance reports
- Automated diagnostics system
- Performance baseline tracking

Practice: Master all Performance Schema tables

Scaling MySQL

Topics:

- Vertical vs horizontal scaling
- Read/write splitting
- Connection pooling solutions
- ProxySQL configuration
- Load balancing strategies
- Caching layers: Redis, Memcached
- Query result caching
- Application-level caching
- Database sharding strategies
- Vitess for sharding
- Federation approaches
- Microservices and databases
- Event sourcing patterns
- CQRS implementation

Projects built:

- Implement read/write splitting
- Caching layer integration
- Sharding proof of concept
- Microservices database design

Practice: Design 5 scaling solutions

Troubleshooting

Topics:

- Troubleshooting methodology
- Common performance issues
- Deadlock analysis and resolution
- Lock wait timeout issues
- Connection problems
- Replication lag troubleshooting
- Disk space issues
- Corruption detection and repair
- Crash recovery procedures
- Emergency procedures
- Root cause analysis
- Post-mortem practices
- Documentation of issues
- Building runbooks

Projects built:

- Troubleshooting playbook
- Emergency response procedures
- Issue tracking system
- Knowledge base creation

Practice: Solve 20 troubleshooting scenarios

Phase 3 Capstone Project

Topics:

- Production database design
- Security implementation
- Performance optimization
- Monitoring setup

- Documentation

Projects built:

- PHASE 3 CAPSTONE: Production Database System
- Design and implement production-ready database
- Include: security, backup, monitoring, optimization
- Create complete DBA documentation

Assessment: Phase 3 exam on administration and performance

PHASE 4: HIGH AVAILABILITY & CLOUD (MONTHS 10-12, WEEKS 40-52)

Master replication, clustering, cloud deployment, and enterprise architecture.

Months 10-11: Replication & High Availability (Week 40-48)

MySQL Replication

Topics:

- Replication concepts and benefits
- Master-slave replication setup
- Binary log formats: STATEMENT, ROW, MIXED
- Replication topologies
- Multi-source replication
- Replication filters
- Semi-synchronous replication
- Delayed replication
- GTID-based replication
- Replication monitoring
- Handling replication lag
- Failover procedures
- Read scaling with replicas
- Backup from replicas

Projects built:

- Master-slave replication setup
- Multi-master configuration
- Automated failover system
- Replication monitoring dashboard

Practice: Configure 10 replication topologies

MySQL Group Replication & InnoDB Cluster

Topics:

- Group Replication architecture
- Single-primary vs multi-primary mode
- Automatic failover with Group Replication
- Consistency guarantees
- Network partitioning handling
- InnoDB Cluster components
- MySQL Shell for cluster management
- MySQL Router for routing
- Cluster deployment strategies
- Adding and removing nodes
- Cluster monitoring
- Backup strategies for clusters
- Disaster recovery with clusters
- Performance considerations

Projects built:

- InnoDB Cluster deployment
- Automatic failover testing
- Cluster expansion project
- DR site configuration

Practice: Manage 5 different cluster configurations

Alternative HA Solutions

Topics:

- Galera Cluster for MySQL
- Synchronous replication with Galera
- Percona XtraDB Cluster
- MariaDB Galera Cluster
- ProxySQL for high availability
- HAProxy configuration
- Keepalived for VIP management
- Orchestrator for topology management
- MHA (Master High Availability)
- Backup and recovery in HA
- Split-brain prevention
- Quorum and voting
- Geographic distribution
- Cross-datacenter replication

Projects built:

- Galera Cluster setup
- ProxySQL implementation
- Geographic DR solution
- HA architecture comparison

Practice: Implement 5 HA solutions

Cloud MySQL - AWS

Topics:

- AWS RDS for MySQL overview
- RDS instance creation and configuration
- Parameter groups and option groups
- RDS backups and snapshots
- Multi-AZ deployments
- Read replicas in RDS
- RDS monitoring with CloudWatch
- RDS security groups and VPC
- Aurora MySQL overview
- Aurora architecture advantages
- Aurora Serverless
- Aurora Global Database
- Migration to RDS/Aurora
- Cost optimization strategies

Projects built:

- RDS deployment project
- Aurora migration
- Multi-region setup
- Cost optimization study

Practice: Deploy 10 cloud database configurations

Cloud MySQL - GCP & Azure

Topics:

- Google Cloud SQL for MySQL
- Cloud SQL high availability
- Cloud SQL read replicas
- Cloud SQL proxy
- Backup and recovery in Cloud SQL
- Azure Database for MySQL
- Azure flexible server
- Azure high availability options
- Cross-cloud considerations
- Multi-cloud strategies
- Cloud migration tools
- Database Activity Monitoring
- Compliance in cloud
- Cloud cost management

Projects built:

- Google Cloud SQL deployment
- Azure MySQL setup
- Multi-cloud architecture
- Cloud comparison study

Practice: Master all major cloud platforms

Month 12: Enterprise & Career (Week 49-52)

Enterprise Architecture

Topics:

- Enterprise database patterns
- Microservices and databases
- Event-driven architecture
- Change data capture (CDC)
- Database federation
- Data warehouse integration
- ETL processes with MySQL
- Real-time analytics
- Data lake integration
- Hybrid transactional/analytical processing
- Multi-tenant architectures
- Compliance and regulations
- GDPR considerations
- Enterprise monitoring solutions

Projects built:

- Enterprise architecture design
- Microservices database pattern
- CDC implementation
- Compliance audit system

Practice: Design 5 enterprise solutions

Migration & Upgrades

Topics:

- Migration planning and assessment
- Homogeneous migrations (MySQL to MySQL)
- Heterogeneous migrations (Oracle to MySQL)
- AWS Database Migration Service
- Data validation strategies
- Minimal downtime migrations
- Blue-green deployments
- Rolling upgrades

- Major version upgrades
- Testing migration procedures
- Rollback planning
- Post-migration optimization
- Documentation requirements
- Migration tools comparison

Projects built:

- Migration from legacy system
- Zero-downtime upgrade
- Cross-version migration
- Migration automation toolkit

Practice: Execute 10 migration scenarios

DevOps for Databases

Topics:

- Database DevOps principles
- Infrastructure as Code for databases
- Terraform for MySQL
- Ansible for database automation
- Database CI/CD pipelines
- Automated testing strategies
- Schema version control
- Blue-green database deployments
- Feature flags for databases
- Database observability
- SRE practices for databases
- Incident management
- Capacity planning
- Cost optimization automation

Projects built:

- Complete DevOps pipeline
- IaC for database infrastructure
- Automated testing framework
- Observability platform

Practice: Implement full DevOps workflow

Career Development

Topics:

- Database career paths
- DBA vs Database Developer
- Database Architect role
- Data Engineer considerations
- Certification preparation (MySQL, cloud)
- Resume building for database roles
- Interview preparation
- Technical interview questions
- Practical test preparation
- Salary negotiation
- Freelancing as database expert
- Consulting opportunities
- Continuous learning strategies
- Community involvement

Projects built:

- Professional portfolio
- Certification study plan
- Interview preparation kit
- Career roadmap

Practice: Complete mock interviews

Final Project & Graduation

Topics:

- Capstone project planning
- Real-world problem solving
- Complete system design
- Implementation best practices
- Documentation standards
- Presentation skills
- Peer review process
- Industry best practices
- Future learning paths
- Alumni network
- Continuous improvement
- Industry connections

Projects built:

- FINAL CAPSTONE: Enterprise Database Solution
- Design complete database system for real business
- Include: HA, replication, cloud deployment, monitoring
- Full documentation and presentation
- Production deployment plan

Assessment: Final comprehensive exam and project defense

Continuous Learning & Specialization (Ongoing)

Staying Current

Topics:

- Following MySQL development
- New version features
- Community involvement
- Contributing to MySQL
- Bug reporting
- Feature requests
- MySQL forums and groups
- Conferences and meetups
- Webinars and training
- Certification maintenance
- Blog writing
- Knowledge sharing

Projects built:

- Learning plan creation
- Community contribution
- Knowledge base development
- Mentoring program

Practice: Dedicate 5 hours weekly to learning

Specialization Options

Topics:

- Performance specialist path
- Security specialist focus
- Cloud database expert
- High availability specialist
- Database architect path
- Data warehouse specialist
- NoSQL integration expert
- DevOps for databases
- Machine learning databases
- Time-series databases
- Graph databases in MySQL
- Consultant specialization

Projects built:

- Specialization deep dive
- Expert-level project
- Certification in specialty
- Thought leadership content

Practice: Master chosen specialization

Advanced Topics

Topics:

- MySQL source code
- Custom storage engines
- Plugin development
- Contributing to MySQL
- Performance benchmarking
- Advanced debugging
- Kernel tuning for MySQL
- Hardware optimization
- SSD and NVMe optimization
- Network optimization
- Future of MySQL
- Emerging technologies

Projects built:

- Advanced research project
- Custom MySQL feature
- Performance study
- Innovation prototype

Practice: Explore cutting-edge features

Leadership & Mentorship

Topics:

- Technical leadership
- Team management
- Knowledge transfer
- Mentoring juniors
- Best practices enforcement
- Architecture decisions
- Strategic planning
- Budget management
- Vendor relationships
- Risk management
- Disaster planning
- Business alignment

Projects built:

- Team guidelines creation
- Mentorship program design
- Strategic roadmap
- Knowledge repository

Practice: Lead database initiatives

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 15+ basic database projects - schemas, queries, relationships
- Phase 2: 20+ advanced projects - procedures, functions, integrations
- Phase 3: 15+ admin projects - security, performance, monitoring
- Phase 4: 10+ enterprise projects - HA, cloud, architecture
- Total: 60+ database projects from simple to enterprise scale

Total projects: 60+ database projects across all complexity levels

SKILLS THE STUDENT WILL HAVE AT THE END

- SQL: Complete mastery from basics to advanced
- Database Design: Normalization, ER diagrams, best practices
- Programming: Stored procedures, functions, triggers, events
- Administration: User management, security, backup, recovery
- Performance: Query optimization, indexing, monitoring, tuning
- High Availability: Replication, clustering, failover
- Cloud: AWS RDS, Aurora, GCP Cloud SQL, Azure Database
- Integration: Python, Java, PHP, Node.js, ORMs
- DevOps: CI/CD, automation, IaC, monitoring
- Architecture: Scaling, sharding, microservices
- Tools: MySQL Workbench, phpMyAdmin, Percona Toolkit
- Troubleshooting: Diagnostics, debugging, problem solving

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-4 hours
Hands on coding: 6-8 hours
Projects: 2-3 hours
Practice problems: 1-2 hours
Total per week: 12-15 hours

SUPPORT INCLUDED

Live sessions: Weekly SQL workshops and DBA sessions
Mentorship: 1-on-1 guidance from senior DBAs
Community: Active Discord community of database professionals
Code review: Expert review of database designs and queries
Career support: Resume review, interview prep, certification guidance
Lifetime access: All content and future MySQL version updates

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
Final certificate: Certified MySQL Database Administrator
Linkedin badge: Verified LinkedIn credential
Industry recognized: Preparation for Oracle MySQL certifications
Portfolio projects: 60+ documented database projects
Cloud certifications: Preparation for AWS, GCP, Azure database certs

WHERE THIS CAN LEAD

- Database Administrator (DBA)
- Database Developer
- Database Architect
- Data Engineer
- MySQL Consultant
- Cloud Database Specialist
- Performance Tuning Expert
- Database DevOps Engineer
- Backend Developer (Database Focus)
- Business Intelligence Developer
- Database Security Specialist

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Growing roles: Rs 4-6 LPA (Junior Database Developer)

Experienced roles: Rs 6-12 LPA (Database Administrator); Rs 10-20 LPA (Senior DBA); Rs 15-35+ LPA (Database Architect)

Freelance: Rs 1,500-5,000/hour

International: \$60k-130k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course only about writing SQL queries?

A: No, queries are just phase 1. The 12-month program goes from SQL fundamentals through database design and normalization, stored procedures, administration, performance tuning and indexing, security, and finally replication, high availability and cloud deployment on platforms like AWS. It is a database engineer's path, not a query crash course.

Q: Do I need programming experience to start?

A: No, the course starts from zero with basic queries and builds up gradually. SQL is one of the friendliest entry points into tech, and any computer with 4GB or more of RAM can run MySQL and MySQL Workbench comfortably.

Q: Which MySQL version do you teach?

A: Current MySQL 8.x, the line running in production at most companies, using MySQL Workbench and the command line client. Features and syntax are taught as they exist today, and the material is kept aligned with current releases.

Q: How does MySQL knowledge help my career?

A: Almost every backend, data and full stack role expects solid SQL, and dedicated database administration and data engineering paths build directly on what this course covers. You finish with schema designs, tuned queries and administration projects in a GitHub-ready portfolio. We make no placement guarantees; SQL simply appears in more job descriptions than nearly any other technical skill.

Q: Why learn databases live instead of from documentation and tutorials?

A: Because database mistakes are silent: a bad schema or a missing index works fine until real data arrives. A live instructor reviews your designs, shows you how to read execution plans on your own queries, and builds the production habits documentation cannot, like thinking about backups and failure before they happen.

Q: Can I evaluate the teaching first?

A: Yes, start with a free live demo class booked at learn.modernagecoders.com/contact, with no commitment involved. We hold 4.9 stars across 547+ Google reviews, and the demo lets you judge the teaching yourself.

Q: What is the class format?

A: All classes are live 1-hour online sessions with an instructor; nothing is pre-recorded. You write queries and work on real schemas during class, and evening and weekend slots fit around college lectures and full-time work.

Q: Is there a certification at the end?

A: You earn our completion certificate for the program, alongside a GitHub-ready portfolio of database projects: schema designs, optimization case studies and administration work. To be clear, this is a course certificate, not an Oracle exam credential; the skills, however, are exactly what database roles test.

To enrol or to book a free demo for this course, message Shivam Sir on
+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/mysql-database-complete-masterclass-college>

=====

COURSE 92 OF 118: MYSQL FOR TEENS: SQL QUERIES, DATABASE DESIGN & SECURITY

=====

Course link: <https://learn.modernagecoders.com/courses/mysql-mastery-for-teens>

Subject area: Database Management & Administration

Designed for: Teens

Who can join: Complete Beginner to Advanced Professional

Course length: 12 months (52 weeks)

Time needed: 10-15 hours/week recommended

Certificate: MySQL Professional Certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

MySQL database course for teens: live online classes on SQL queries, database design, optimization and security, from absolute basics to real projects.

In one line: From 'What is a Database?' to Professional Database Administrator

ABOUT THE PROGRAMME

This specially designed course for teenagers transforms complete beginners into MySQL professionals in just 12 months. Whether you're a high school student interested in tech, a young gaming enthusiast curious about game databases, or a teen entrepreneur wanting to manage data, this program will give you professional-level database skills. You'll master everything from basic queries to complex database administration, building real projects like social media databases, game leaderboards, e-commerce systems, and analytics platforms. By completion, you'll have built 30+ database projects and be ready for database administrator roles or data engineering positions.

WHAT MAKES THIS COURSE DIFFERENT

- Designed specifically for teenage learners with engaging examples
- Uses real-world scenarios teens relate to (social media, gaming, apps)
- No prerequisites - starts from absolute zero
- Gamified learning with achievements and progress tracking
- Build databases for apps teens actually use
- Practical projects you can show off to friends
- Career guidance for young professionals
- College application portfolio builder

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Database Basics, SQL Fundamentals, Data Modeling, Queries
Phase 2: Advanced Mastery (Months 7-12): Administration, Optimization, Security, Real-World Applications

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic computer literacy - suitable for high school students
Coding experience: No programming experience required - complete beginner friendly
Equipment: Any computer (Windows/Mac/Linux) with internet
Time commitment: 10-15 hours per week for best results
English: Basic English reading skills
Motivation: Curiosity about data and technology

WHO THIS COURSE IS FOR

Students: High school students (13-18 years) interested in tech careers
Working professionals: Young adults wanting database skills
Entrepreneurs: Teen entrepreneurs needing data management skills
Freelancers: Students wanting to earn through database projects
Kids: Motivated middle schoolers with parental guidance
Anyone: Gaming enthusiasts wanting to understand game databases

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE MYSQL SKILLS (MONTHS 1-6, WEEKS 1-26) -----

Build rock-solid database fundamentals. Learn what databases are, master SQL queries, and create your first real-world database applications.

Months 1-2: Database Fundamentals & MySQL Basics (Week 1-8)

Introduction to Databases & MySQL Setup

Topics:

- What is data and why do we need databases?
- Real-world database examples teens use daily (Instagram, TikTok, Spotify)
- Files vs Databases: Why databases win
- Types of databases: Relational vs NoSQL
- Introduction to MySQL: History and why it's popular
- MySQL in the real world: Facebook, Twitter, YouTube
- Installing MySQL on Windows/Mac/Linux
- MySQL Workbench installation and tour
- Command line basics for MySQL
- Creating your first database
- Understanding servers, clients, and connections
- Database terminology: tables, rows, columns, records

Projects built:

- Install and configure MySQL environment
- Create 'MyLife' database for personal data
- Design a simple contact list database

Practice: Daily: 30 min terminal practice, 1 hour MySQL basics

SQL Basics: Creating and Managing Databases

Topics:

- CREATE DATABASE statement
- USE, SHOW, and DROP commands
- Creating tables with CREATE TABLE

- Data types: INT, VARCHAR, TEXT, DATE, BOOLEAN
- Understanding NULL and NOT NULL
- Primary keys and their importance
- AUTO_INCREMENT for automatic IDs
- Altering tables: ADD, MODIFY, DROP columns
- Renaming tables and columns
- DESCRIBE and SHOW COLUMNS commands
- Comments in SQL code
- SQL naming conventions and best practices

Projects built:

- School database with students and classes
- Gaming stats tracker database
- Music library database structure

Practice: Create 10 different database schemas for teen apps

Data Manipulation: INSERT, SELECT Basics

Topics:

- INSERT INTO: Adding single records
- INSERT multiple records at once
- Understanding SQL syntax and semicolons
- SELECT statement fundamentals
- SELECT specific columns vs SELECT *
- WHERE clause for filtering data
- Comparison operators: =, !=, <, >, <=, >=
- AND, OR, NOT logical operators
- IN and NOT IN operators
- BETWEEN for range queries
- LIKE operator and wildcards (% , _)
- ORDER BY: Sorting results ASC and DESC

Projects built:

- Populate school database with student data
- Create and query a movie ratings database
- Build a sports team roster system

Practice: Write 50 different SELECT queries with various filters

UPDATE, DELETE, and Data Integrity

Topics:

- UPDATE statement: Modifying existing data
- Using WHERE with UPDATE (critical importance!)
- Updating multiple columns
- Using CASE in UPDATE statements
- DELETE statement: Removing records
- Safe deletion practices
- TRUNCATE vs DELETE vs DROP
- Understanding transactions basics
- Data integrity and consistency
- Handling NULL values in queries
- IS NULL and IS NOT NULL
- Default values and their uses

Projects built:

- Grade management system with updates
- Inventory system with stock updates
- Social media profile manager

Practice: Perform 30 UPDATE and DELETE operations safely

Months 3-4: Advanced Queries & Functions (Week 9-17)

MySQL Functions: String and Numeric

Topics:

- String functions: CONCAT, SUBSTRING, LENGTH
- UPPER, LOWER, TRIM, LTRIM, RTRIM
- REPLACE and string manipulation
- REVERSE, REPEAT, and SPACE
- Numeric functions: ROUND, CEIL, FLOOR
- ABS, MOD, POWER, SQRT
- RAND() for random numbers
- Mathematical operations in queries
- FORMAT for number formatting
- Working with CHAR_LENGTH vs LENGTH
- CAST and CONVERT for type conversion
- Combining multiple functions

Projects built:

- Username generator with string functions
- Grade calculator with numeric functions
- Data cleaning toolkit

Practice: Master 40 different MySQL functions

Date/Time Functions and Operations

Topics:

- Date and time data types: DATE, TIME, DATETIME, TIMESTAMP
- CURDATE(), CURTIME(), NOW()
- Date formatting with DATE_FORMAT
- Extracting parts: YEAR, MONTH, DAY, HOUR, MINUTE

- DAYNAME, MONTHNAME for readable output
- Date arithmetic: DATE_ADD, DATE_SUB
- DATEDIFF for calculating differences
- Working with time zones
- UNIX_TIMESTAMP and FROM_UNIXTIME
- Age calculations from birthdates
- Scheduling and recurring events concepts
- Best practices for storing dates

Projects built:

- Birthday reminder system
- Event scheduling database
- Time tracking application

Practice: Build 20 queries involving date/time calculations

Aggregate Functions and Grouping

Topics:

- COUNT: Counting records and values
- COUNT(*) vs COUNT(column) vs COUNT(DISTINCT)
- SUM for totaling numeric values
- AVG for calculating averages
- MIN and MAX for finding extremes
- GROUP BY clause explained
- Grouping by single and multiple columns
- HAVING clause for filtering groups
- HAVING vs WHERE: Key differences
- GROUP_CONCAT for combining grouped values
- Using aliases with AS
- Complex grouping scenarios

Projects built:

- Class grade analytics system
- Sales reporting database
- Game high scores leaderboard

Practice: Create 30 reports using aggregate functions

Subqueries and Nested Queries

Topics:

- Understanding subqueries concept
- Subqueries in WHERE clause
- Subqueries returning single values
- Subqueries returning multiple values
- IN, NOT IN with subqueries
- EXISTS and NOT EXISTS operators
- Subqueries in SELECT clause
- Subqueries in FROM clause (derived tables)
- Correlated vs non-correlated subqueries
- Performance considerations
- ANY, ALL, SOME operators
- Nested subqueries (multiple levels)

Projects built:

- Advanced student ranking system
- Product recommendation queries
- Social media analytics dashboard

Practice: Write 25 complex queries with subqueries

Month 3-4 Review & Assessment

Topics:

- Review of all functions learned
- Query optimization basics
- Common mistakes and how to avoid them
- SQL style guide and formatting
- Documentation best practices

Projects built:

- CAPSTONE: Teen Social Network Database
- Complete database with users, posts, likes, comments
- Complex queries for analytics and reporting

Assessment: Comprehensive test on SQL queries and functions

Months 5-6: Relationships & Database Design (Week 18-26)

Table Relationships and JOINS

Topics:

- Understanding relationships: Why we need multiple tables
- One-to-One relationships explained
- One-to-Many relationships (most common)
- Many-to-Many relationships and junction tables
- Foreign keys: Linking tables together
- FOREIGN KEY constraints
- Referential integrity and CASCADE options
- Introduction to JOINS
- INNER JOIN: Matching records in both tables
- LEFT JOIN: Include all from left table
- RIGHT JOIN: Include all from right table
- JOIN with multiple tables

Projects built:

- School system with students, teachers, courses
- E-commerce with customers, orders, products
- Music app with artists, albums, songs

Practice: Create 20 different JOIN queries

Advanced JOINS and Set Operations

Topics:

- Self JOINS: Joining table to itself
- CROSS JOIN and Cartesian products
- Natural JOIN (and why to avoid it)
- Using aliases in JOINS
- Multiple JOIN conditions
- JOINS with aggregate functions
- UNION: Combining result sets
- UNION vs UNION ALL
- INTERSECT concept (workarounds in MySQL)
- EXCEPT/MINUS concept
- Complex JOIN scenarios
- Optimizing JOIN performance

Projects built:

- Employee hierarchy system
- Tournament bracket system
- Multi-store inventory system

Practice: Master complex JOIN operations with 30 exercises

Database Design & Normalization

Topics:

- Database design process overview
- Identifying entities and attributes
- Entity-Relationship (ER) diagrams
- Crow's foot notation
- Primary keys: Natural vs Surrogate
- Composite keys
- Database normalization: Why it matters
- First Normal Form (1NF): Eliminating repeating groups
- Second Normal Form (2NF): Removing partial dependencies
- Third Normal Form (3NF): Removing transitive dependencies
- Denormalization: When and why
- Common design patterns

Projects built:

- Design Instagram-like database from scratch
- Create online learning platform schema
- Design multiplayer game database

Practice: Normalize 10 poorly designed databases

Constraints and Data Validation

Topics:

- PRIMARY KEY constraint deep dive
- UNIQUE constraint for preventing duplicates
- NOT NULL constraint
- DEFAULT values and when to use them
- CHECK constraints for data validation
- FOREIGN KEY constraints and relationships
- ON DELETE CASCADE, SET NULL, RESTRICT
- ON UPDATE CASCADE
- Adding constraints to existing tables
- Dropping and modifying constraints
- Constraint naming conventions
- Data validation best practices

Projects built:

- User registration system with validations
- Banking system with integrity constraints
- Inventory system with business rules

Practice: Implement 20 different constraint scenarios

Phase 1 Capstone Project

Topics:

- Project planning and requirements gathering
- Database design documentation
- ER diagram creation
- Implementation and testing
- Data population strategies

Projects built:

- MAJOR CAPSTONE: TikTok Clone Database
- Complete schema with users, videos, likes, comments, follows
- Complex queries for feed algorithm
- Analytics and reporting queries

Assessment: Phase 1 Final Exam - Design and implement complete database

Master advanced MySQL features, administration, optimization, and real-world applications to become a professional database developer.

Months 7-8: Advanced MySQL Features (Week 27-35)

Views and Virtual Tables

Topics:

- What are views and why use them?
- Creating views with CREATE VIEW
- Simple vs complex views
- Views for data security
- Views for simplifying complex queries
- Updatable views and restrictions
- WITH CHECK OPTION for data integrity
- Modifying views with ALTER VIEW
- Dropping views
- Views vs tables performance
- Materialized views concept
- Best practices for using views

Projects built:

- Create security layer with views
- Report generation system using views
- Multi-tenant system with view-based isolation

Practice: Build 15 different views for various use cases

Stored Procedures and Functions

Topics:

- Introduction to stored procedures
- Benefits: security, performance, reusability
- CREATE PROCEDURE syntax
- Parameters: IN, OUT, INOUT
- Variables and DECLARE statement
- Control flow: IF-ELSE, CASE
- Loops: WHILE, REPEAT, LOOP
- Cursors for row-by-row processing
- Error handling with HANDLERS
- Stored functions vs procedures
- CREATE FUNCTION syntax
- Debugging stored procedures

Projects built:

- Banking transaction procedures
- Automated report generation system
- Game scoring calculation procedures

Practice: Create 20 stored procedures for common operations

Triggers and Events

Topics:

- What are triggers? Automatic actions
- Trigger timing: BEFORE and AFTER
- Trigger events: INSERT, UPDATE, DELETE
- CREATE TRIGGER syntax
- OLD and NEW keywords in triggers
- Row-level triggers
- Multiple triggers on same event
- Trigger use cases: audit trails, validation
- Event scheduler introduction
- Creating scheduled events
- Recurring events and intervals
- Managing and monitoring events

Projects built:

- Audit trail system with triggers
- Automatic data archiving with events
- Real-time notification system

Practice: Implement 15 triggers and 10 scheduled events

Transactions and Concurrency

Topics:

- ACID properties explained for teens
- Understanding transactions importance
- START TRANSACTION, COMMIT, ROLLBACK
- Autocommit mode
- Savepoints in transactions
- Isolation levels: READ UNCOMMITTED to SERIALIZABLE
- Dirty reads, phantom reads, non-repeatable reads
- Locking mechanisms in MySQL
- Table locks vs row locks
- Deadlocks: detection and prevention
- Optimistic vs pessimistic locking
- Transaction best practices

Projects built:

- Banking system with safe transactions
- Ticket booking system with concurrency handling
- Inventory management with locking

Practice: Handle 20 different transaction scenarios

Indexing and Query Optimization

Topics:

- What are indexes? Phone book analogy
- How indexes speed up queries
- Types of indexes: B-Tree, Hash, Full-text
- Creating indexes with CREATE INDEX
- Primary key and unique indexes
- Composite (multi-column) indexes
- Index selectivity and cardinality
- EXPLAIN statement for query analysis
- Reading execution plans
- Identifying slow queries
- Query optimization techniques
- When NOT to use indexes

Projects built:

- Optimize slow social media queries
- Create indexing strategy for e-commerce
- Performance tuning workshop

Practice: Optimize 25 slow queries with proper indexing

Months 9-10: Database Administration (Week 36-44)

User Management and Security

Topics:

- MySQL user account management
- CREATE USER and DROP USER
- Understanding privileges system
- GRANT and REVOKE statements
- Database-level vs table-level privileges
- Column-level and procedure privileges
- Roles in MySQL 8.0+
- Password policies and expiration
- SSL/TLS connections
- Security best practices
- Preventing SQL injection
- Prepared statements for security

Projects built:

- Multi-user application security setup
- Role-based access control system
- Secure API database layer

Practice: Configure security for 10 different scenarios

Backup and Recovery

Topics:

- Importance of backups (horror stories!)
- Types of backups: Logical vs Physical
- mysqldump for logical backups
- Full vs incremental backups
- Point-in-time recovery
- Binary logs and their importance
- Backup scheduling strategies
- Testing backup restoration
- Disaster recovery planning
- Cloud backup solutions
- Backup compression and encryption
- Recovery time objectives (RTO)

Projects built:

- Automated backup system
- Disaster recovery plan document
- Backup testing framework

Practice: Perform 15 backup and recovery scenarios

Performance Monitoring and Tuning

Topics:

- MySQL performance schema
- Monitoring with SHOW STATUS
- Understanding SHOW PROCESSLIST
- Slow query log configuration
- Query cache (deprecated but important to know)
- Buffer pool and memory tuning
- InnoDB vs MyISAM performance
- Table optimization with OPTIMIZE TABLE
- Partitioning large tables
- Performance monitoring tools
- Identifying bottlenecks
- Hardware considerations

Projects built:

- Performance monitoring dashboard
- Automated performance reports
- Database health check system

Practice: Tune 10 different database performance issues

Replication and High Availability

Topics:

- Why replication? Scaling and backup
- Master-slave replication setup
- Master-master replication
- Replication formats: Statement vs Row
- Monitoring replication lag
- Handling replication errors
- Read/write splitting
- Failover strategies
- MySQL Cluster basics
- Load balancing concepts
- High availability best practices
- Disaster recovery with replication

Projects built:

- Set up master-slave replication
- Build automatic failover system
- Create read-replica for reporting

Practice: Configure 5 different replication topologies

MySQL with Programming Languages

Topics:

- Connecting to MySQL from Python
- MySQL connectors and drivers
- Connection pooling importance
- Prepared statements in applications
- ORM concepts (Object-Relational Mapping)
- Building APIs with MySQL backend
- Handling database errors in code
- Async database operations
- Database abstraction layers
- Testing database code
- Migration scripts
- Database versioning

Projects built:

- Python MySQL admin tool
- RESTful API with MySQL
- Database migration system

Practice: Build 10 applications using MySQL

Months 11-12: Real-World Applications & Career Prep (Week 45-52)

Big Data and Analytics

Topics:

- MySQL for analytics workloads
- Data warehousing concepts
- ETL processes with MySQL
- Window functions for analytics
- ROW_NUMBER, RANK, DENSE_RANK
- LAG and LEAD functions
- Running totals and moving averages
- Pivot tables in MySQL
- JSON data type and functions
- Full-text search implementation
- Geographic data with spatial types
- Time-series data handling

Projects built:

- Analytics dashboard for gaming platform
- Sales analysis system
- Location-based service database

Practice: Build 15 analytical queries and reports

Cloud Databases and Modern Deployments

Topics:

- MySQL in the cloud era
- Amazon RDS for MySQL
- Google Cloud SQL
- Azure Database for MySQL
- Managed vs self-hosted databases
- Database as a Service (DBaaS)
- Scaling databases in cloud
- Cloud backup strategies
- Multi-region deployments
- Database migration to cloud
- Cost optimization in cloud
- Serverless database concepts

Projects built:

- Deploy database to AWS RDS
- Multi-region database setup
- Cloud cost calculator tool

Practice: Deploy databases to 3 different cloud providers

NoSQL and MySQL Integration

Topics:

- When to use NoSQL vs MySQL
- MySQL as a document store
- JSON support in MySQL
- MySQL vs MongoDB comparison
- Polyglot persistence patterns
- Migrating between database types
- Hybrid database architectures
- Caching with Redis and MySQL
- Elasticsearch with MySQL
- Message queues and MySQL
- Microservices and databases
- API-first database design

Projects built:

- Hybrid database application
- Cache implementation for MySQL
- Microservices data architecture

Practice: Build 5 systems combining MySQL with other technologies

Industry Best Practices and Standards

Topics:

- Database documentation standards
- Change management processes
- Database code reviews
- Version control for databases
- Compliance and regulations (GDPR basics)
- Data privacy for teens
- Encryption at rest and in transit
- Audit logging requirements
- Database testing strategies
- Continuous integration for databases
- DevOps and databases
- Database team collaboration

Projects built:

- Database documentation system
- Compliance audit checklist
- CI/CD pipeline for database

Practice: Implement best practices in all projects

Final Capstone & Career Preparation

Topics:

- Portfolio project planning
- Resume building for database roles
- LinkedIn optimization for tech
- GitHub portfolio setup
- Interview preparation
- Common database interview questions
- Live coding interviews
- System design for databases
- Salary negotiation for teens/young adults
- Internship strategies
- Freelancing opportunities
- Continuous learning path

Projects built:

- FINAL CAPSTONE: Full-Scale Social Platform Database
- Complete database for teen social app with all features
- Performance optimization, security, backups, documentation

Assessment: Final comprehensive exam covering entire year

PHASE 3: PLACEHOLDER - STRUCTURE REQUIRED BY TEMPLATE

This phase is not applicable for the 1-year MySQL course but included to maintain JSON structure integrity.

Not Applicable (N/A)

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

Not Applicable (N/A)

N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

Not Applicable (N/A)

N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Assessment: N/A

PHASE 4: PLACEHOLDER - STRUCTURE REQUIRED BY TEMPLATE

This phase is not applicable for the 1-year MySQL course but included to maintain JSON structure integrity.

Not Applicable (N/A)

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

Not Applicable (N/A)

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

Not Applicable (N/A)

N/A

Topics:

- N/A

Projects built:

- N/A

Practice: N/A

N/A

Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A
N/A
Topics:
- N/A
Projects built:
- N/A
Practice: N/A

Not Applicable (N/A)

N/A
Topics:
- N/A
Projects built:
- N/A
N/A
Topics:
- N/A
Deliverables:
- N/A
N/A
Topics:
- N/A
Deliverables:
- N/A
Assessment: N/A

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-6): Contact Database, School System, Gaming Stats Tracker, Social Media Database, E-commerce Schema
- Phase 2 (Months 7-12): Banking System, Analytics Dashboard, Cloud Database Deployment, Replication Setup, Complete Social Platform

Total projects: 30+ database projects ranging from simple to complex enterprise systems

SKILLS THE STUDENT WILL HAVE AT THE END

- SQL Fundamentals: SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, DROP
- Advanced Queries: JOINS, Subqueries, CTEs, Window Functions, Set Operations
- Database Design: ER Diagrams, Normalization, Relationships, Constraints
- Programming: Stored Procedures, Functions, Triggers, Events, Cursors
- Administration: User Management, Security, Backup/Recovery, Replication
- Performance: Indexing, Query Optimization, Partitioning, Caching
- Cloud Databases: AWS RDS, Google Cloud SQL, Azure Database
- Integration: Python/PHP/Java connections, ORMs, APIs, Microservices
- Analytics: Data Warehousing, ETL, Reporting, Business Intelligence
- Security: SQL Injection Prevention, Encryption, Access Control, Auditing

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-5 hours
Hands on coding: 5-7 hours
Projects: 2-3 hours
Practice problems: 2-3 hours
Total per week: 10-15 hours

SUPPORT INCLUDED

Live sessions: Weekly Q&A sessions with database experts
Mentorship: 1-on-1 guidance from professional DBAs
Community: Teen-friendly Discord community
Code review: Expert review of database designs
Career support: Internship guidance and college application help
Lifetime access: All content available forever with updates

CERTIFICATION DETAIL

Phase certificates: Certificate after Phase 1 completion
Final certificate: MySQL Professional Developer Certificate

Linkedin badge: Verifiable LinkedIn certification
Industry recognized: Recognized by tech companies
Portfolio projects: 30+ projects for college/job applications

WHERE THIS CAN LEAD

- Junior Database Developer (Entry level)
- SQL Developer
- Database Administrator (DBA)
- Data Analyst
- Backend Developer with Database Focus
- Business Intelligence Developer
- Data Engineer (with additional skills)
- Database Consultant
- College CS Major (strong foundation)
- Freelance Database Developer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Why should a teenager learn MySQL database management?

A: Every app, website, and game stores data in databases. Learning MySQL gives teens a highly marketable skill that's essential for software development, data science, and IT careers. Database skills complement web development and app programming, making teens more complete developers.

Q: What real-world applications will I build in this MySQL course?

A: All projects use scenarios teens relate to and can showcase in portfolios.

Q: How does learning MySQL help with college applications?

A: Database skills demonstrate technical proficiency beyond typical high school curricula. Your portfolio of 30+ database projects, MySQL certification, and demonstrated ability to build complex data systems significantly strengthens computer science program applications.

Q: What SQL commands and concepts will I master?

A: The course covers everything from basics to professional database administration.

Q: Can I connect MySQL to websites and apps I build?

A: Yes! The course includes connecting MySQL to Python applications, building REST APIs with database backends, and understanding how popular apps like Instagram, YouTube, and TikTok use databases. You'll learn the complete stack from database to application.

Q: What career opportunities are available for teens with MySQL skills?

A: Database skills like MySQL are among the most in-demand technical skills globally. With the projects you build in this course you can pursue internships and junior database roles as you grow older, and some learners take on small freelance database projects once they are ready. Any earnings depend on your skills, effort, and the market.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/mysql-mastery-for-teens>

COURSE 93 OF 118: OLYMPIAD MATHEMATICS: IOQM, INMO AND AMC PREPARATION

Course link: <https://learn.modernagecoders.com/courses/olympiad-mathematics-premium-course>

Subject area: Olympiad Mathematics

Designed for: Teens

Who can join: Students roughly aged 12 to 18 who are strong at school maths and want genuine olympiad problem solving

Course length: 12 months (48 weeks), joinable any month

Time needed: 2 live classes/week + weekly problem sets and timed mocks

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online olympiad mathematics for serious students: number theory, algebra, combinatorics and geometry taught as proof-based problem solving, aimed at IOQM and INMO in India and the AMC and AIME internationally.

In one line: The four olympiad pillars taught as real proof-based problem solving, for students who have outgrown school maths.

ABOUT THE PROGRAMME

Olympiad mathematics is a different subject from school mathematics wearing the same name. School maths asks you to apply a method you were just shown. Olympiad maths hands you a problem you have never seen, gives you no method, and asks you to construct an argument that is airtight. Students who top their class routinely struggle at first, and that is normal, because nothing in the school syllabus trains this. This course teaches the four pillars that every olympiad is built on: number theory, algebra, combinatorics and geometry. Each is taught for genuine understanding and then drilled on real past problems, because olympiad ability is built by attempting hard problems and failing productively, not by watching solutions. It is aimed at both the Indian and international ladders. In India the path runs through IOQM, the Indian Olympiad Qualifier in Mathematics, on to INMO and the training camp beyond it. Internationally students target the AMC 10 and AMC 12, then AIME. The formats differ, and we prepare for each specifically, but the underlying skill is the same: reading a problem properly, finding the idea, and writing an argument that survives scrutiny. Proof is taught explicitly from the start. Most students arrive able to find an answer but unable to justify it, and at INMO level the justification is the entire mark. We fix that early, because a correct answer with a hand-waving argument scores close to nothing.

WHAT MAKES THIS COURSE DIFFERENT

- All four pillars taught properly: number theory, algebra, combinatorics and geometry, not just contest tricks
- Proof writing taught explicitly from the start, because at INMO level the argument is the mark, not the answer
- Prepared for both ladders: IOQM and INMO in India, AMC 10, AMC 12 and AIME internationally
- Built on real past problems, since olympiad ability comes from productive failure on hard problems
- Students are taught to sit with a problem instead of giving up after two minutes, which is the actual skill
- Live, small batches where solutions are critiqued line by line for rigour, not just marked right or wrong

THE LEARNING PATH, STEP BY STEP

Phase 1: Proof technique and number theory, the natural entry point into olympiad thinking
Phase 2: Algebra and combinatorics, including inequalities and counting arguments
Phase 3: Geometry, then full contest simulation for IOQM, INMO and the AMC or AIME

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PROOF TECHNIQUE AND NUMBER THEORY

How to argue rigorously, then the pillar that best rewards it.

Months 1 to 2: Proof and Foundations (Weeks 1-8)

What Counts as a Proof

Topics:

- The difference between checking cases and proving a statement
- Direct proof, and writing it so every step is justified
- Proof by contradiction
- Proof by contrapositive, and when it is easier
- Reading a written solution critically and finding the gap

Projects built:

- Write five full proofs and have every line challenged in class

Practice: Prove supplied statements and rewrite them after critique

Induction and Invariants

Topics:

- Mathematical induction, including strong induction
- Common induction errors that void the proof
- Invariants and monovariants, and the problems that hide them
- Extremal arguments: taking the largest or smallest case
- Recognising which technique a problem is inviting

Projects built:

- Solve a set of invariant problems and articulate the invariant explicitly in each

Practice: 20 induction and invariant problems

Assessment: A proof-writing assessment marked on rigour, not just the final answer

Number Theory

Topics:

- Divisibility, GCD and the Euclidean algorithm

- Primes, factorisation and the fundamental theorem of arithmetic
- Modular arithmetic and congruences
- Fermat's little theorem and Euler's theorem
- Diophantine equations at olympiad level

Projects built:

- Solve a graded ladder of number theory problems from IOQM level upward

Practice: 40 number theory problems across the difficulty range

Assessment: Timed number theory paper under contest conditions

PHASE 2: ALGEBRA AND COMBINATORICS

The two pillars that most often decide an IOQM or AMC score.

Months 3 to 5: Algebra (Weeks 9-20)

Polynomials and Functional Equations

Topics:

- Polynomial roots, factorisation and Vieta's formulas
- Symmetric functions and their uses
- Functional equations and the standard substitution strategies
- Recognising the structure hidden in an ugly expression
- Writing algebraic arguments cleanly

Projects built:

- Work a set of Vieta and functional equation problems from past papers

Practice: 30 algebra problems building in difficulty

Inequalities

Topics:

- AM-GM and its many disguises
- Cauchy-Schwarz
- Rearrangement and Chebyshev
- Normalisation and substitution tactics
- Knowing when to stop searching for a slicker method

Projects built:

- Prove a graded set of inequalities, each with a fully justified argument

Practice: 30 inequality problems

Assessment: Timed algebra paper under contest conditions

Combinatorics

Topics:

- Counting properly: permutations, combinations and overcounting
- The pigeonhole principle and its surprisingly deep uses
- Inclusion and exclusion
- Bijections and counting the same set two ways
- Introductory graph theory and games

Projects built:

- Solve a counting problem two different ways and reconcile the results

Practice: 35 combinatorics problems

Assessment: Timed combinatorics paper

PHASE 3: GEOMETRY AND FULL CONTEST SIMULATION

The pillar students most often avoid, then the contests themselves rehearsed properly.

Months 6 to 9: Geometry (Weeks 21-36)

Synthetic Geometry

Topics:

- Angle chasing done systematically rather than hopefully
- Cyclic quadrilaterals and the power of a point
- Similar triangles and ratio arguments
- The classical centres and their properties
- Drawing an accurate diagram, which solves more problems than any theorem

Projects built:

- Solve a ladder of synthetic geometry problems with full written proofs

Practice: 35 geometry problems from contest papers

Advanced Geometry Techniques

Topics:

- Coordinate geometry as a fallback when synthetic stalls
- Trigonometric methods in geometry
- Homothety and transformations
- Choosing between synthetic, coordinate and trigonometric attacks
- Recognising when a problem is resisting the method rather than you

Projects built:

- Attack the same hard problem three ways and compare which was cleanest

Practice: 25 problems requiring a deliberate choice of method

Assessment: Timed geometry paper

Mixed Problem Solving

Topics:

- Problems that do not announce their topic
- Deciding quickly which pillar a problem belongs to

- Working when the first idea fails, and the second
- Managing frustration, which is a genuine competition skill
- Building a personal record of techniques that worked

Projects built:

- Weekly mixed problem sets with no topic labels

Practice: Mixed sets under time

Months 10 to 12: Contest Simulation (Weeks 37-48)

IOQM and AMC Format Training

Topics:

- The IOQM format and how to pace it
- AMC 10 and AMC 12 pacing, and the scoring implications of guessing
- Answer-extraction techniques for short-answer formats
- Knowing when to abandon a question and return
- Accuracy under time, where most marks are actually lost

Projects built:

- Full timed IOQM and AMC papers every week

Practice: Weekly full papers with question-by-question review

INMO and AIME Level, and Peak Form

Topics:

- The step up to full written proofs at INMO level
- AIME format and its particular demands
- Writing a complete solution under time pressure
- Partial credit: how to secure marks on a problem you cannot finish
- Contest-day routine, sleep and nerves

Projects built:

- Full INMO-style and AIME-style papers, marked on rigour

Practice: Continuous full-paper cycle with detailed feedback

Assessment: Final full mock contest, marked as an examiner would, with an honest readiness assessment

WHERE THIS CAN LEAD

- Readiness for IOQM, INMO, AMC and AIME
- Proof-writing ability that makes university mathematics far less of a shock
- A distinctive strength in competitive university applications
- A foundation for research mathematics, physics or theoretical computer science
- Problem-solving stamina that transfers to every quantitative field

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which contests does this prepare for?

A: In India the path runs through IOQM, the Indian Olympiad Qualifier in Mathematics, on to INMO and the training camp beyond it. Internationally we prepare students for AMC 10 and AMC 12 and then AIME. The formats differ and we train for each specifically, but the underlying skill is identical, so the same course serves both ladders.

Q: My child tops the class in maths. Will olympiad maths be easy for them?

A: Almost certainly not at first, and that is completely normal. School maths asks you to apply a method you were just taught. Olympiad maths gives you an unfamiliar problem, no method, and expects a rigorous argument. Excellent school students often struggle for the first month. The ones who keep going usually become very strong, because the difficulty is trainable.

Q: Why do you teach proof writing so early?

A: Because at INMO level the argument is the mark. Most students arrive able to find the right answer but unable to justify it, and a correct answer with hand-waving reasoning scores close to nothing. Proof technique is taught in the first two months and then marked for rigour all year, which is also exactly what makes university mathematics manageable later.

Q: How long does the course run?

A: Twelve months across 48 weeks, with two live classes a week plus problem sets and timed mocks. That length is deliberate: olympiad ability is built by attempting hard problems over months, not by a crash course. You can join in any month and we place your child at the right point after the free demo class.

Q: Is geometry really necessary?

A: Yes, and it is the pillar students most often try to avoid. We spend four months on it, starting with systematic angle chasing rather than hoping, and we teach coordinate and trigonometric methods as deliberate fallbacks.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/olympiad-mathematics-premium-course>

=====

Course link: <https://learn.modernagecoders.com/courses/orange-data-mining-machine-learning-course-for-college-students>
Subject area: Data Science & Machine Learning
Designed for: College students
Who can join: Beginner to applied practitioner (college students and graduates), no prior machine learning required
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 4-6 hours/week recommended
Certificate: Modern Age Coders 'Orange Data Mining for College' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Master Orange Data Mining in 24 live classes for college students: reproducible workflows, feature engineering, model selection with nested validation, explainability, clustering, text mining, time series and a defended analytics capstone, plus the Python bridge to pandas and scikit-learn.

In one line: Cover a full data mining syllabus on a visual canvas, then defend it: rigorous validation, feature engineering, explainability, clustering, text and time series, and a capstone that reproduces from raw data on someone else's machine.

ABOUT THE PROGRAMME

Data mining appears in almost every computer science, IT, statistics, business analytics and bioinformatics programme, and it is routinely taught as a list of algorithm names to memorise before an exam. This course teaches it as a practice. Orange is free, open source machine learning and data visualisation software developed at the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana, released under the GNU GPL version 3 or later. It was built for teaching machine learning and data mining to university students, and it is used in real research work. You assemble analysis by connecting widgets on a canvas, which removes the syntax overhead and puts the entire course budget into method: how to prepare data defensibly, how to select a model without fooling yourself, and how to explain what the model did. Across 24 live classes you work through the six default widget sets, Data, Transform, Visualize, Model, Evaluate and Unsupervised, and then into the add-on ecosystem that makes Orange genuinely useful for project work: Text for corpus analysis, Time Series for temporal data, Explain for model interpretation, Associate for rule mining, Geo for spatial data, and Fairness for auditing. You will build classification and regression pipelines, use Preprocess inside the modelling loop so that scaling and imputation are fitted on training folds rather than leaked across them, compare learners under identical cross validation in Test and Score, tune with Parameter Fitter, read ROC, precision-recall and calibration properly, and use Explain Model and Permutation Importance to answer the question a viva always asks: why did it predict that? The course is deliberately not a dead end at the canvas. Orange is written in Python and installable from PyPI as Orange3, and the Python Script widget exposes real Python inside a workflow with the incoming data available in your namespace. Two classes are spent on that bridge, so students moving on to pandas and scikit-learn carry the concepts across rather than starting again. Every class is taught live in a small batch by a working practitioner, the same mentorship-first method Modern Age Coders has used since 2020 to teach 10,000+ students across 25+ countries. You finish with 24 reproducible workflows, a capstone that runs end to end from a raw file, and an exported report you can put in front of an examiner, a recruiter or a project guide.

WHAT MAKES THIS COURSE DIFFERENT

- Built for defensible work, not demos: reproducibility, leakage control and honest validation are graded requirements from Phase 1, not closing remarks.
- Covers a full university data mining syllabus, preprocessing, classification, regression, evaluation, clustering, dimensionality reduction, association rules, text and time series, with a running artefact for each.
- Explainability is a full class, not a mention: Explain Model, Explain Prediction, Permutation Importance and ICE, so you can answer why a prediction happened.
- The Python bridge is taught explicitly: Orange is a Python library, the Python Script widget runs real Python inside the canvas, and the same models exist in scikit-learn.
- Free and open source under the GNU GPL v3 or later, so nothing in your final year project depends on a licence you lose after graduation.
- Capstone standard is reproduction: your workflow must rerun from the raw file in a clean session, on a machine that is not yours.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop running Windows, macOS or Linux capable of installing Orange, with a stable internet connection for live classes and for the server-based image and text embedders

Coding experience: None required. The entire course can be completed without writing code.

Students who already know Python get more from Class 22, where the Python Script widget and the Orange to scikit-learn concept map are covered, but nothing earlier depends on it

Maths background: Comfort with school-level statistics, averages, percentages, basic

probability and reading a graph. Every further concept, from cross validation to regularisation to silhouette scores, is taught in the course at the point it is needed

Accounts: No account is required. Orange is free and open source software you download and run locally, and the add-ons used in this course install free from inside the application

Mindset: Willingness to be wrong in public: to report that your model lost to the baseline, to state your limitations before someone else finds them, and to rerun your own work from scratch to prove it holds

WHO THIS COURSE IS FOR

Cs and engineering students: Undergraduates with a data mining, machine learning or data warehousing paper who want to actually practise the syllabus rather than memorise algorithm names for an exam

Final year project students: Students building a project, dissertation or research paper who need a defensible methodology and a workflow that reproduces under scrutiny

Non cs students with data: Students in statistics, economics, commerce, psychology, biology, biotechnology and public health who have real data and a real question but no interest in a six week detour through Python syntax

Career switchers and graduates: Graduates preparing for data analyst and junior data science interviews who need the method questions, validation, metrics, leakage and explainability, answered with confidence

Students who will learn python next: Anyone planning to learn pandas and scikit-learn who wants the modelling and evaluation concepts secure first, so only the syntax is new

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: REPRODUCIBLE FOUNDATIONS -----

Classes 1 to 5. Set up Orange, learn the canvas properly, and build the habits that separate a project that survives a viva from one that falls apart when someone asks you to run it again. Everything in this phase is about data you can trust and workflows that reproduce.

Class 1: Orange, Visual Programming and Why It Is Taken Seriously

Topics:

- What Orange is: open source machine learning and data visualisation software from the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana, first released in 1996 and under the GNU GPL version 3 or later
- Why a visual tool belongs in a serious course: it was built to teach machine learning and data mining to university students, and it runs the same algorithms you would call from code
- Installation paths: the standalone installer on Windows and macOS, and installing the Orange3 package from the Python Package Index into a Python environment
- The interface: widget toolbox, canvas, the six default widget sets Data, Transform, Visualize, Model, Evaluate and Unsupervised, and the status bar
- How data flows: output channels, input channels, and choosing which channel a connection uses when a widget offers several
- The mental model that prevents most beginner errors: a widget recomputes whenever anything upstream changes, so the canvas is a live pipeline, not a sequence of clicks
- Course standards announced up front: every deliverable is an .ows workflow that reruns from the raw data file with no manual editing anywhere

Projects built:

- Install Orange, build a File to Data Table to Distributions canvas on a bundled dataset, then deliberately change the upstream file and document exactly which downstream widgets recomputed

Practice: Open three example workflows from the official examples page and write one sentence on what each pipeline is doing.

Assessment: Your first canvas plus a short written explanation of the live-pipeline model in your own words.

Class 2: Data Sources, Types and Roles

Topics:

- The File widget across formats: .csv, .tsv, Excel and Orange's native .tab, including delimiter and encoding problems and how to diagnose them
- CSV File Import for awkward files: header rows, decimal separators, thousands separators and locale issues
- SQL Table: reading directly from a database rather than exporting an intermediate spreadsheet nobody can reproduce
- Datasets: the bundled and downloadable example datasets used throughout the course for controlled comparisons
- Variable types in Orange, numeric, categorical, text and datetime, and the downstream failures caused by each mistyped column
- Roles: feature, target and meta, and why identifiers, names and free text must be pushed to meta rather than fed to a learner
- Data Info and Column Statistics as a mandatory first look: instances, features, missing values, constant columns and near-duplicates

Projects built:

- Load one awkward real-world .csv, correct every mistyped column with Edit Domain, assign

roles deliberately, and write a data dictionary describing every variable, its type and its role

Practice: Import the same file twice, once correctly and once with a wrong delimiter, and record what silently went wrong.

Assessment: Your data dictionary plus a Data Info screenshot with a written note on every quality problem you found.

Class 3: Missing Data, Outliers and Defensible Cleaning

Topics:

- The three kinds of missingness in plain language, and why the reason a value is missing changes what you are allowed to do about it
- The Impute widget in full: leave as is, average or most frequent, replace with a fixed value, remove instances, and model-based imputation
- The decision that gets challenged in every viva: dropping rows versus imputing, and how to state your choice with a defensible reason
- Select Rows for rule-based filtering, and why filters belong in the workflow rather than in a spreadsheet you edited by hand
- Outliers: detecting them, and the separate question of whether to remove, cap, flag or investigate them
- Duplicate detection with Unique, and the near-duplicate problem that silently inflates every cross validation score
- Documenting cleaning: annotating the canvas so a reader can see every judgement you made and disagree with it if they want

Projects built:

- Build a fully annotated cleaning pipeline on a messy dataset, recording instance counts before and after each step, and write a cleaning log justifying every decision

Practice: Compare model-based imputation against mean imputation on the same column and record how the distribution changed.

Assessment: Your cleaning log with before and after counts, plus a defence of your imputation strategy.

Class 4: Joins, Aggregation and Reshaping

Topics:

- Merge Data: inner, outer and left join behaviour on a shared key, and diagnosing the rows that silently disappear
- Concatenate for stacking sources, including what happens when the schemas do not quite agree
- Group by and Aggregate Columns: producing entity-level datasets from transaction-level records, the single most common real analytics task
- Pivot Table for cross-tabulation, and Transpose for datasets stored the wrong way round
- Formula: deriving ratios, differences, flags and interaction terms as genuine engineered features
- Select by Data Index and Data Sampler: reproducible sampling, stratified sampling, and holding out a set you genuinely do not look at
- Granularity discipline: knowing exactly what one row represents at every point in your pipeline

Projects built:

- Build an entity-level analysis dataset from at least two source tables using Merge Data and Group by, engineer three derived features with Formula, and document what one row represents at each stage

Practice: Perform the same join two ways and reconcile the row counts, explaining any difference.

Assessment: Your multi-table workflow plus the row-meaning documentation at each stage.

Class 5: Preprocess, Reproducibility and Workflow Engineering

Topics:

- The Preprocess widget as a single reusable block: normalisation, standardisation, imputation, feature selection, discretisation, continuation and randomisation
- The leakage rule stated early and repeated all course: preprocessing that learns from data, such as scaling or imputation, must be fitted inside the validation loop, not applied to the whole dataset first
- How Orange helps: connecting Preprocess into a learner so the preprocessing is fitted per training fold during cross validation
- Purge Domain and Randomize: cleaning unused categories, and shuffling a target deliberately as a sanity check that your pipeline cannot score well on noise
- Saving workflows as .ows files: XML describing widgets, their settings and their connections, saving the workflow and settings but not the data
- Consequences for sharing: an .ows file needs its data file alongside it, so a shareable project is a folder, not a single file
- Workflow engineering standards: left to right layout, renamed widgets, canvas notes, and a README describing how to rerun the analysis

Projects built:

- Package everything from Classes 2 to 4 as a reproducible project folder: annotated .ows workflow, raw data file and a README, then have a classmate rerun it from scratch and report any step they could not reproduce

Practice: Run the shuffled-target sanity check on your pipeline and confirm performance collapses to baseline as it should.

Assessment: A project folder a classmate successfully reran, plus their written reproduction report.

PHASE 2: EXPLORATORY ANALYSIS AND FEATURE ENGINEERING

Classes 6 to 10. Model performance is decided long before the learner widget. This phase covers reading a dataset properly and then constructing the features that make the difference, along with the diagnostics that tell you when a feature is helping and when it is cheating.

Class 6: Distributions, Dispersion and Group Comparison

Topics:

- Distributions: shape, skew, modality and the long tail, and what each implies for the models available to you
- Box Plot and Violin Plot: comparing groups on a numeric variable, including unequal group sizes and what that does to a visual comparison
- Central tendency and dispersion together: why reporting a mean without a spread is an incomplete claim
- Detecting distributional problems that break specific models, such as heavy skew for distance-based methods
- Transformations in plain terms, and when a log-style transformation genuinely helps versus when it just hides the problem
- Target distribution first: class imbalance diagnosed before any model is trained, using Distributions on the target
- Stating an exploratory finding as a hypothesis to test rather than a conclusion to announce

Projects built:

- Produce a full univariate profile of your dataset, every numeric variable charted and described, every categorical variable tabulated, ending with three written hypotheses worth testing

Practice: Identify one variable whose mean misrepresents it and justify a better summary statistic.

Assessment: Your univariate profile document plus the three hypotheses with the evidence that prompted each.

Class 7: Bivariate Analysis and Informative Projections

Topics:

- Scatter Plot with class colouring, point size and shape used as genuine extra dimensions
- Find Informative Projections: ranking variable pairs by how well they separate the target classes
- Rank: scoring features by information gain, gain ratio, gini, chi square, ReliefF and FCBF, and reading the disagreements between scores as information
- Sieve Diagram and Mosaic Display for categorical against categorical relationships
- Linear Projection, Radviz and FreeViz for looking at many variables at once in a single plane
- Selecting subsets directly from a plot and sending them downstream for inspection, the fastest way to interrogate a suspicion
- The discipline of the alternative explanation: for every relationship you find, write the confounder that could produce it

Projects built:

- Rank every feature by at least three scoring methods, reconcile the disagreements in writing, and build the informative projection that best separates your target classes

Practice: Take your strongest bivariate relationship and write two plausible confounders for it.

Assessment: Your multi-method feature ranking with a written reconciliation, plus the projection and its interpretation.

Class 8: Correlation, Multicollinearity and Distance

Topics:

- Correlations: ranking every numeric pair, and reading the coefficient without over-claiming what it establishes
- Multicollinearity: what happens to a linear model's coefficients when two features carry the same information, and why the model can still predict well while its explanation becomes worthless
- Heat Map with row and column clustering for whole-dataset structure at a glance
- Distances: Euclidean, Manhattan, cosine, Jaccard and others, and choosing a measure that matches the data type and the question
- Distance Map, Distance Matrix and Distance Transformation as the shared foundation under clustering and nearest neighbour methods
- Why normalisation must precede distance, demonstrated by breaking it deliberately with one large-scale feature
- Feature selection versus dimensionality reduction: dropping features versus combining them, and when each is appropriate

Projects built:

- Diagnose multicollinearity in a numeric dataset, decide which features to drop with a written justification, and demonstrate the effect on a linear model's coefficients before and after

Practice: Compute the same distance matrix under three distance measures and describe how the neighbourhood structure changed.

Assessment: Your multicollinearity report including the coefficient comparison and your retained feature set.

Class 9: Feature Engineering, Discretisation and Encoding

Topics:

- Feature engineering as the highest-leverage work in applied machine learning, with worked examples of features that transformed a weak model
- Formula: ratios, differences, flags, counts and interaction terms built inside the workflow, and Create Instance for testing a model on a hand-specified case
- Discretize: equal width, equal frequency and entropy-based binning, and what each does to the information in a variable
- Continuize: one-hot and ordinal treatment of categorical variables, and matching the encoding to the model that will consume it
- Create Class: deriving a target variable from raw columns, and the leakage danger that this step invites
- Domain knowledge as a feature source: interviewing the person who understands the data before engineering anything
- Testing whether a feature earned its place: rerun the cross validated comparison with and without it

Projects built:

- Engineer at least five new features from domain reasoning, then run a cross validated with-and-without comparison for each and keep only the ones that measurably earned their place

Practice: Compare equal-frequency and entropy-based discretisation of the same variable and note the difference in downstream model performance.

Assessment: Your feature engineering log showing each candidate feature, its rationale, its measured effect and your keep or drop decision.

Class 10: Communicating Analysis with the Orange Report

Topics:

- The Report system: most widgets carry a report button in the status bar that appends the current view to a running report
- Which widgets do not report, and why the ones that only transmit or display data are excluded
- Structuring an analytical report the way a reader needs it: question, data and its limits, method, evidence, result, threats to validity
- Adding written commentary to report sections so the argument, not just the output, is preserved
- Exporting as .html or .pdf for submission, and as .report so a reader can reopen the exact workflow behind each section
- Reopening reports through File then Open Report, and the report view via Shift and R
- Presenting to a non-technical audience: what to cut, what to keep, and how to state uncertainty without losing the room

Projects built:

- Produce a complete exploratory data analysis report on your project dataset: question, data dictionary, univariate and bivariate evidence, engineered features, and a stated set of limitations, exported as PDF

Practice: Cut your report to a three minute verbal summary and note what you were forced to drop.

Assessment: Your exported EDA report, graded on whether every claim in it is supported by a visible piece of evidence.

PHASE 3: MODELLING AND RIGOROUS EVALUATION

Classes 11 to 16. The modelling half of a data mining syllabus, taught with the validation rigour that distinguishes a result from a coincidence. Classification and regression, simple models and ensembles, and tuning that does not quietly contaminate the test set.

Class 11: The Modelling Frame, Baselines and Validation Design

Topics:

- Supervised learning framed properly: a hypothesis space, a loss, and an estimate of generalisation error, all in plain language
- Classification versus regression, decided by the target variable, and the metrics that come with each
- The Orange modelling shape: data into learner into Test and Score, with Preprocess attached to the learner rather than the data
- Constant as the mandatory baseline, and Feature as Predictor as a second reference point using a single variable
- Choosing a validation design: train and test split, k-fold cross validation, stratified folds, leave one out, and repeated sampling, with the cost and reliability of each
- Class imbalance: what accuracy hides, and why AUC, F1, precision and recall exist
- Declaring your evaluation protocol before you look at any result, and writing it into the project README

Projects built:

- Write and register your evaluation protocol for the course project, then establish both baselines with stratified 10-fold cross validation and record the numbers every later model must beat

Practice: Run the same model under three validation designs and record how much the estimate moves.

Assessment: Your registered evaluation protocol plus a baseline results table with the validation design justified.

Class 12: Trees, Rules and Interpretable Models

Topics:

- Decision tree induction: impurity measures, the greedy split search, and stopping criteria as the practical control on complexity
- The Tree widget's parameters, minimum instances in a leaf, maximum depth, and splitting thresholds, mapped to bias and variance
- Tree Viewer, Pythagorean Tree and Pythagorean Forest for reading structure rather than guessing at it
- CN2 Rule Induction and CN2 Rule Viewer: learning an ordered or unordered rule set you can read as if-then statements
- Scoring Sheet and Scoring Sheet Viewer: producing the kind of simple point-based model that regulated domains actually accept
- Overfitting demonstrated live: growing a tree until training accuracy is perfect and cross validated accuracy collapses
- The interpretability and accuracy trade-off, and the domains where an interpretable model is a requirement rather than a preference

Projects built:

- Train a tuned decision tree and a CN2 rule set on the same problem, extract the decision logic from both as written rules, and compare their cross validated performance against your baselines

Practice: Sweep maximum depth across five values and tabulate training against cross validated accuracy to locate the overfitting point.

Assessment: Your extracted rule set in plain English plus the overfitting curve with the depth you chose and why.

Class 13: Distance and Probability Models, Regression Begins

Topics:

- k Nearest Neighbours: the choice of k, distance measure and weighting, and its total dependence on feature scaling
- Naive Bayes: the conditional independence assumption, why it is usually false, and why the model often works anyway
- Linear Regression in Orange: predicting a continuous target, with ridge and lasso regularisation and what each penalty does
- Regression metrics and how to read them: MSE, RMSE, MAE and R squared, and what R squared does not tell you
- PLS and Curve Fit for the cases where ordinary linear regression is the wrong shape
- Stochastic Gradient Descent as a learner, and the intuition of iterative optimisation without the calculus
- Choosing a model family from the data and the requirement, not from familiarity

Projects built:

- Build a full regression pipeline on a continuous target, compare linear regression with and without regularisation against KNN regression, and report RMSE and R squared under identical cross validation

Practice: Run KNN with and without normalisation inside the Preprocess block and quantify the difference.

Assessment: Your regression comparison table with metrics defined in your own words and a justified model choice.

Class 14: Evaluation in Depth, from Confusion Matrix to Calibration

Topics:

- Test and Score read properly: comparing learners under identical folds, and the meaning of the variation between folds
- Confusion Matrix: the four cells expressed as real costs in the actual application domain
- Precision, recall, specificity and F1 derived from that matrix rather than memorised
- ROC Analysis: the curve as every threshold at once, AUC interpreted as a ranking probability, and reading multiple models on one plot
- Performance Curve for the operational question of how many cases you must action to capture a given share of positives
- Calibration Plot: whether predicted probabilities behave like probabilities, and Calibrated Learner as the fix
- Choosing the operating threshold from the cost of each error type rather than accepting 0.5 by default

Projects built:

- Evaluate every model built so far on one problem with a full evaluation dossier: metrics table, confusion matrices with stated costs, ROC comparison, calibration plot, and a recommended operating threshold with its justification

Practice: Take a poorly calibrated model, apply Calibrated Learner, and document the change in the calibration plot.

Assessment: Your evaluation dossier, graded on whether the threshold recommendation follows from stated error costs.

Class 15: Ensembles and Neural Networks

Topics:

- Random Forest: bagging plus random feature subsets, why decorrelated trees generalise better, and the parameters that matter
- Gradient Boosting: sequential correction of residuals, and the learning rate and depth interaction that governs overfitting
- AdaBoost: reweighting the examples the ensemble keeps getting wrong
- Support Vector Machines: margins, the kernel trick explained without the algebra, and the cost parameter
- Neural Network in Orange: hidden layer sizes, activation functions, the solver, and realistic

expectations on tabular data of student scale

- Stacking: combining several learners with a meta-learner, and the extra validation care it demands
- The honest finding that a well tuned simple model frequently matches an ensemble, and why reporting that is good work

Projects built:

- Run a full learner tournament, baseline through neural network and stacking, under one fixed cross validation, and write a model selection memo that justifies your winner on grounds beyond the top accuracy figure

Practice: Vary the gradient boosting learning rate and depth together and record where performance stops improving.

Assessment: Your model selection memo, which must address performance, variance across folds, interpretability and runtime.

Class 16: Tuning, Deployment and the Traps That Invalidate Results

Topics:

- Parameter Fitter: searching a learner's parameters systematically instead of by intuition
- The tuning trap named precisely: selecting parameters using the same data you then report performance on inflates your result
- Nested validation in principle, and the practical discipline of a genuinely held-out set that is opened exactly once
- Permutation Plot and the shuffled-target sanity check as evidence that your pipeline is not learning noise
- Predictions: scoring new unlabelled data, reading class probabilities, and exporting results with Save Data
- Save Model and Load Model: persisting a fitted model so scoring does not require retraining
- The leakage catalogue, target leakage, temporal leakage, group leakage and preprocessing leakage, each with the symptom that gives it away

Projects built:

- Tune your chosen model with Parameter Fitter, evaluate the tuned model on a held-out set you have not previously touched, run the shuffled-target check, and write a leakage audit against all four leakage types

Practice: Deliberately introduce a leaking feature, observe the implausible score, and document how you would have caught it.

Assessment: Your held-out result with the tuning protocol described, plus your completed four-point leakage audit.

PHASE 4: UNSUPERVISED, EXPLAINABLE AND SPECIALISED DATA

Classes 17 to 21. The rest of a data mining syllabus, taught to the same standard: clustering with validity measures rather than eyeballing, dimensionality reduction, association rules, model explainability, and the two data types that dominate project work, text and time series.

Class 17: Clustering and Cluster Validity

Topics:

- Unsupervised learning framed honestly: no ground truth, so validity has to be argued rather than measured against an answer key
- k Means: the algorithm step by step, initialisation sensitivity, and scoring a range of k automatically
- Silhouette score and Silhouette Plot: evaluating cohesion and separation, and finding the points that were forced into the wrong cluster
- Hierarchical Clustering: linkage methods, reading a dendrogram, and choosing the cut height defensibly
- DBSCAN: density-based clusters of arbitrary shape, the epsilon and minimum points parameters, and the ability to label points as noise
- Louvain Clustering and Self-Organizing Map as further approaches, and Interactive k-Means from the Educational add-on for stepping through the algorithm
- Profiling clusters into an interpretation: describing each group by its distinguishing features so the result means something to a domain reader

Projects built:

- Cluster one dataset with k means, hierarchical clustering and DBSCAN, justify your chosen k or cut with silhouette evidence, and write a business-readable profile of every cluster you keep

Practice: Run k means on uniformly random data and write the paragraph explaining why its clusters are meaningless.

Assessment: Your three-method comparison with silhouette evidence, plus interpretable profiles for each final cluster.

Class 18: Dimensionality Reduction, Outliers and Association Rules

Topics:

- PCA: principal components as ordered directions of maximum variance, and using explained variance to choose how many to keep
- The interpretability cost of PCA, and when a loading is still readable enough to explain
- t-SNE, MDS and Manifold Learning for visualisation, with an explicit warning about over-reading distances between separated clusters in a t-SNE plot
- Correspondence Analysis for categorical structure
- Outliers: detection methods and the analyst's decision about whether an outlier is an error,

- a rare event, or the entire point of the study
- The Associate add-on: frequent itemsets and association rules, with support, confidence and lift defined precisely
- Reading association rules critically: high confidence with low lift is usually just a popular item, not a relationship

Projects built:

- Reduce a wide dataset with PCA, visualise it with t-SNE, then mine association rules on a transactional dataset and identify three rules that are genuinely informative by lift rather than merely frequent

Practice: Cluster on raw features and on PCA components and compare silhouette scores and cluster interpretability.

Assessment: Your dimensionality reduction analysis plus three association rules with support, confidence and lift interpreted correctly.

Class 19: Explainability and Fairness

Topics:

- Why explainability is now a requirement: examiners, regulators, clinicians and clients all ask why, and a score is not an answer
- The Explain add-on: Explain Model for global feature attribution across the dataset
- Explain Prediction and Explain Predictions: why this specific instance received this specific output
- Permutation Importance: measuring a feature's contribution by breaking it and observing the damage
- ICE plots for seeing how a prediction responds as one feature is varied across its range
- Nomogram for logistic regression as a fully transparent alternative to post-hoc explanation
- The Fairness add-on: auditing model behaviour across a sensitive attribute, and the fact that a fairness problem is usually a data problem

Projects built:

- Take your best model, produce a global explanation and three individual prediction explanations, run a permutation importance check, and audit the model across one sensitive attribute with a written finding

Practice: Find one instance your model got confidently wrong and explain, from its attribution, why it failed.

Assessment: Your explainability report including global attribution, three instance-level explanations, and the fairness audit finding.

Class 20: Text Mining and Document Analysis

Topics:

- Installing add-ons through Options then Add-ons, and what the Text add-on contributes to the toolbox
- Corpus, Import Documents and Create Corpus: loading documents as data, including a folder of your own files
- Preprocess Text in detail: transformation, tokenisation, normalisation with stemming or lemmatisation, stop word and lexicon filtering, and n-grams
- Bag of Words with term frequency and TF-IDF weighting, and what a bag of words representation discards
- Document Embedding as the modern alternative, and when it beats a bag of words
- Analysis widgets: Word Cloud, Concordance, Statistics, Word Enrichment, Extract Keywords and Duplicate Detection
- Topic Modelling and Corpus Viewer, plus Sentiment Analysis with an honest account of its failure modes on sarcasm, negation and mixed-language text

Projects built:

- Build an end to end text pipeline: load a corpus, preprocess it deliberately, represent it with both bag of words and embeddings, classify or topic-model it, and evaluate the two representations against each other under identical validation

Practice: Rerun your pipeline with stemming, with lemmatisation and with neither, and tabulate the effect on your metric.

Assessment: Your text mining report comparing both representations, with named topics or a validated classifier and its limitations.

Class 21: Time Series and Spatial Data

Topics:

- The Time Series add-on: As Timeseries for converting a table, Line Chart, and Yahoo Finance for pulling market data
- Why time series breaks the rules you learned: rows are not independent, so ordinary shuffled cross validation leaks the future into the past
- Interpolate, Moving Transform and Seasonal Adjustment: filling gaps, computing rolling statistics, and separating trend, seasonality and remainder
- Correlogram and Periodogram: autocorrelation and periodicity, and finding the cycle length in real data
- ARIMA Model and VAR Model: univariate and multivariate forecasting, with Model Evaluation for out-of-sample assessment
- Granger Causality read carefully: predictive precedence is not causation, whatever the widget is called
- The Geo add-on and the Network add-on named and demonstrated for spatial and relational project data

Projects built:

- Take a real time series, decompose it, identify its dominant period, fit an ARIMA model, evaluate it out of sample against a naive last-value baseline, and state honestly whether the

model beat the baseline

Practice: Demonstrate the leakage caused by shuffled cross validation on time series data and quantify how much it inflates the score.

Assessment: Your forecasting workflow with an out-of-sample evaluation and an explicit comparison against the naive baseline.

----- PHASE 5: THE PYTHON BRIDGE AND CAPSTONE -----

Classes 22 to 24. Leave the course able to work in both worlds. Orange is a Python library, so the last phase connects the canvas to code, sets the ethical standard for the work, and produces a capstone that reproduces on a machine that is not yours.

Class 22: The Python Script Widget and Orange as a Library

Topics:

- Orange3 as a Python package installable from the Python Package Index, with a current Python version required for the pip installation route
- The Python Script widget: running real Python inside the canvas when no existing widget does what you need
- The widget's namespace: `in_data`, `in_distance`, `in_learner`, `in_classifier` and `in_object` arrive from the input signals, and are None when nothing is connected
- Returning results to the canvas so a script sits in the middle of a workflow rather than at the end of it
- The `Orange.data.Table` object and how it maps onto the pandas `DataFrame` most students will meet next
- The concept map from Orange to scikit-learn: learner to estimator, Test and Score to `cross_val_score`, Preprocess to Pipeline, Parameter Fitter to `GridSearchCV`
- Choosing the right tool honestly: canvas for exploration, comparison and communication, code for automation, scale and version control

Projects built:

- Insert a Python Script widget into an existing workflow to compute something no widget provides, pass the result downstream, and write the equivalent scikit-learn outline for one pipeline you built earlier in the course

Practice: Map five widgets you used this course onto their scikit-learn equivalents in a written table.

Assessment: Your working Python Script integration plus the Orange to scikit-learn concept map.

Class 23: Build and Defend Your Capstone

Topics:

- Scoping a capstone that is answerable: a specific question, an obtainable dataset, a defined target and a stated success criterion
- Sourcing data responsibly: open government portals, research repositories, Kaggle and institutional data, with the licence and any consent conditions checked and recorded
- Applying the full pipeline discipline: data dictionary, cleaning log, engineered feature log, registered evaluation protocol, held-out set
- Beating the baseline, and the professional willingness to report that the sophisticated approach did not win
- Threats to validity written before the conclusion: sample bias, leakage, confounding, temporal drift and generalisation limits
- Mentor review checkpoint: an instructor interrogates your canvas the way an examiner will, step by step
- Assembling the deliverable folder: `.ows` workflow, raw data, README with rerun instructions, and the report

Projects built:

- Build your complete capstone from raw data to evaluated model and written conclusion, defended live in a mentor review that challenges every methodological choice

Practice: Have a classmate rerun your capstone folder on their own machine and log every step that failed.

Assessment: A capstone that reproduces from raw data in a clean session on someone else's machine, plus your threats-to-validity section.

Class 24: Report, Present and Continue

Topics:

- Assembling the final report: question, data and provenance, method, evidence, results with uncertainty, threats to validity, and conclusion
- Exporting for the audience you have: PDF for submission, and `.report` so a reviewer can reopen the workflow behind every figure
- Presenting under questioning: leading with the question and the answer, defending the method, and saying clearly when something is unknown
- The interview and viva questions this course prepares you for: why that validation design, why that metric, how did you rule out leakage, why should the model be believed
- Portfolio assembly: 24 reproducible workflows plus the capstone, organised so a reviewer can open any one and follow it
- Continuing paths: Python with pandas and scikit-learn, statistics and experimental design, and further Orange add-ons including Bioinformatics, Single-Cell, Spectroscopy, Survival Analysis and Network
- Staying current with the tool: the official Orange documentation, workflow examples and video lessons, and the open source repository itself

Projects built:

- Present your capstone under live questioning, submit the exported report and complete portfolio, and receive written feedback plus your certificate assessment
- Practice: Write the one page project proposal for your next analysis, including its question, data source, evaluation protocol and baseline.
- Assessment: Your defended presentation, exported report and 24-workflow portfolio, assessed on methodological rigour as much as on results.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a live-pipeline demonstration, a corrected dataset with a full data dictionary, an annotated cleaning log with before and after counts, a multi-table entity-level dataset, and a reproducible project folder a classmate reran successfully
- Phase 2: a complete univariate profile with three testable hypotheses, a multi-method feature ranking with reconciled disagreements, a multicollinearity report with coefficient evidence, a measured feature engineering log, and an exported EDA report
- Phase 3: a registered evaluation protocol with dual baselines, a tuned tree and CN2 rule set with extracted logic, a regularised regression comparison, a full evaluation dossier with a cost-justified threshold, a learner tournament with a model selection memo, and a held-out tuned result with a four-point leakage audit
- Phase 4: a three-method clustering study with silhouette evidence and readable cluster profiles, a PCA and t-SNE reduction with lift-interpreted association rules, an explainability report with a fairness audit, a two-representation text mining comparison, and an out-of-sample ARIMA forecast tested against a naive baseline
- Phase 5: a Python Script widget integrated mid-workflow with an Orange to scikit-learn concept map, and the capstone: reproducible folder, threats to validity, exported report and a presentation defended under questioning
- Total: 24 reproducible workflows plus a capstone that survives being rerun by someone else

Total projects: 24 reproducible workflows, one per class, plus a capstone project that reruns from raw data on another person's machine

SKILLS THE STUDENT WILL HAVE AT THE END

- Reproducible Workflow Engineering: canvas discipline, channel selection, annotated .ows workflows, project folders with README rerun instructions
- Data Engineering in Orange: File and CSV File Import, SQL Table, Edit Domain, Impute, Select Columns and Rows, Merge Data, Concatenate, Group by, Aggregate Columns, Pivot Table and Formula
- Exploratory Analysis: Distributions, Box and Violin Plot, Scatter Plot with informative projections, Rank across multiple scoring methods, Correlations, Heat Map, Distances and Distance Map
- Feature Engineering: derived and interaction features, Discretize, Continuize, Create Class, and measured with-and-without validation of every candidate feature
- Classification and Regression: Constant and Feature as Predictor baselines, Tree, CN2 Rule Induction, Scoring Sheet, kNN, Naive Bayes, Linear and Logistic Regression with regularisation, PLS, SVM, Random Forest, Gradient Boosting, AdaBoost, Neural Network and Stacking
- Rigorous Evaluation: Test and Score validation designs, Confusion Matrix with stated error costs, ROC Analysis, Performance Curve, Calibration Plot and Calibrated Learner, Parameter Fitter tuning, Permutation Plot, and a working catalogue of leakage types
- Unsupervised Learning: k Means with silhouette selection, Hierarchical Clustering, DBSCAN, Louvain, Self-Organizing Map, PCA, t-SNE, MDS, Manifold Learning, Correspondence Analysis, Outliers and association rule mining
- Explainability and Fairness: Explain Model, Explain Prediction, Permutation Importance, ICE, Nomogram, and auditing across a sensitive attribute
- Specialised Data: Corpus, Preprocess Text, Bag of Words with TF-IDF, Document Embedding, Topic Modelling, Sentiment Analysis, As Timeseries, Seasonal Adjustment, Correlogram, Periodogram, ARIMA and VAR
- The Python Bridge: the Python Script widget with its `in_data` namespace, Orange3 as a Python package, and the concept map onto pandas and scikit-learn

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks

Hands on canvas work: 2 hours rebuilding each class workflow on your own project dataset, not only on the class example

Project time: 1.5 hours on the class deliverable, from cleaning logs and evaluation dossiers to the capstone build

Review and doubts: 1 hour reviewing results against your registered protocol, checking for leakage, and clearing doubts before the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for installation, add-on, dataset and workflow problems

Small batches: Small batches, with mini-batch and 1-on-1 options, so your own project dataset gets attention every week

Mentor guidance: Live guidance from mentors who build and evaluate models professionally,

including an interrogation-style capstone review
Project feedback: Feedback on every deliverable, graded on methodology, reproducibility and honest reporting as much as on performance
Living syllabus: Curriculum updated as Orange ships new versions, widgets and add-ons, at no extra cost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'Orange Data Mining for College' certificate, earned by defending your Class 24 capstone
Project portfolio: 24 reproducible .ows workflows plus a capstone folder containing the workflow, raw data and a README with rerun instructions
Presented report: An exported Orange report (PDF and .report) covering question, method, evidence, results and threats to validity

WHERE THIS CAN LEAD

- Data Analyst and Business Analyst roles, where cleaning, exploration, validation and clear reporting are the daily work
- Junior Data Scientist and Machine Learning Analyst positions, entered with genuine evaluation and explainability discipline
- Research Assistant and academic project work in bioinformatics, public health, social science, economics and environmental science
- Business Intelligence and Reporting Analyst roles, building on the aggregation, visualisation and reporting work in this course
- A prepared transition into Python-based data science, since every concept maps directly onto pandas and scikit-learn

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-5 LPA (Junior Analyst); Rs 5-8 LPA (Data Analyst)

Growing roles: Rs 8-15 LPA (Senior Analyst)

Other roles: Rs 15-25+ LPA (Lead/Manager)

Freelance: Rs 500-2,000/hour

International: \$50k-100k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is Orange serious enough for a college level data mining course?

A: Yes. Orange is developed at the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana, has been in development since 1996, and was built in part for teaching machine learning and data mining to university students. It runs standard algorithms including trees, kNN, naive Bayes, SVM, random forests, gradient boosting and neural networks, evaluates them with proper cross validation and ROC analysis, and is written in Python so the same models are available as code. It is also citable, which matters for project reports.

Q: Do I need to know Python before starting?

A: No. The full 24 classes can be completed without writing a line of code, because analysis is built by connecting widgets. Class 22 is dedicated to the Python bridge, where the Python Script widget runs real Python inside a workflow and every Orange concept is mapped onto its scikit-learn equivalent, so students who already code get extra value there and students who do not are not disadvantaged anywhere else.

Q: Will this cover my university data mining syllabus?

A: It covers the standard ground: data preprocessing and cleaning, exploratory analysis, feature engineering, classification, regression, model evaluation, clustering, dimensionality reduction, association rule mining, text mining and time series, plus explainability and fairness which many syllabi have only recently added. It teaches these as practice rather than as definitions, so it complements exam preparation rather than replacing your notes. Bring your syllabus to the free demo class and we will map it against the 24 classes.

Q: Can I use this for my final year project or dissertation?

A: That is exactly what Phase 5 is built for. Classes 23 and 24 take you from scoping a defensible question through data sourcing with licence checks, a registered evaluation protocol, a held-out set, a threats-to-validity section, and a mentor review conducted like a viva. The standard for the capstone is that it reproduces from the raw data file in a clean session on a machine that is not yours.

Q: How is this different from a Python data science course?

A: A Python course spends its early weeks on syntax, environments and library APIs, which is necessary but expensive. This course removes that overhead and spends the time on method instead, so it goes further into validation design, leakage, explainability, association

rules and time series than a first code course usually reaches. It is designed to be taken before or alongside Python, not instead of it, and Class 22 exists specifically to make the transition short.

Q: Is Orange free, and does anything expire?

A: Orange is free and open source software released under the GNU General Public License version 3 or later. There is no subscription, no trial clock and no student licence. It runs on Windows, macOS and Linux, and the add-ons used in this course install free from inside the application under Options then Add-ons. Everything you build keeps working after the course ends.

Q: What add-ons does the course install, and are they hard to set up?

A: We install them together in class from Options then Add-ons. The course uses Text for corpus analysis, Image Analytics where relevant, Time Series for temporal data, Explain for model interpretation, Associate for rule mining, and introduces Educational, Geo, Network and Fairness. All are free. The text and image embedders that run on a server need a working internet connection, which we flag before those classes.

Q: Do I need my own dataset?

A: Not to start. Orange ships with example datasets and the Datasets widget offers more, and every class provides a working dataset. From Phase 2 onward we strongly encourage you to run each class a second time on a dataset from your own field or project, because the course is far more useful when the capstone is something you actually needed to answer.

Q: What will I be able to do in an interview after this?

A: Answer the method questions that separate candidates: why that validation design, what the confusion matrix costs in this domain, how you ruled out leakage, why you chose that operating threshold, which features drove the prediction and how you know. You will also have a portfolio of 24 reproducible workflows plus a defended capstone to show, rather than a certificate on its own.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/orange-data-mining-machine-learning-course-for-college-students>

=====

COURSE 95 OF 118: ORANGE DATA MINING COURSE FOR TEENS: MACHINE LEARNING WITHOUT CODE

=====

Course link: <https://learn.modernagecoders.com/courses/orange-data-mining-visual-machine-learning-course-for-teens>

Subject area: Data Science & Machine Learning

Designed for: Teens

Who can join: Beginner-friendly (ages 13-18), no programming experience required

Course length: 24 classes (12 weeks · 2 classes/week)

Time needed: 3-4 hours/week recommended

Certificate: Modern Age Coders 'Orange Data Mining for Teens' certificate upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Learn Orange Data Mining in 24 live classes for teens 13-18: build machine learning workflows by connecting widgets on a canvas, visualise real datasets, train and honestly evaluate models, mine text and images, and finish with a shareable data project.

In one line: Build real machine learning by dragging widgets onto a canvas: load a dataset, clean it, chart it, train a model, test whether it is actually any good, and present the whole thing as a project you can defend.

ABOUT THE PROGRAMME

Most teenagers meet machine learning as a black box: an app that guesses the filter, a chatbot that answers, a recommendation that appears. This course opens the box, without asking anyone to learn Python first. Orange is free, open source data mining software built by the Bioinformatics Laboratory at the Faculty of Computer and Information Science, University of Ljubljana, and released under the GNU GPL version 3 or later. Instead of writing code, you place widgets on a canvas, connect them with lines, and watch data flow through the pipeline you designed. That visual canvas is not a toy version of machine learning: underneath, Orange runs the same well established algorithms a data scientist would use, and it is written in Python so the exact same models are available in code later. Across 24 live classes for ages 13 to 18 you work through the six widget sets Orange ships with by default, Data, Transform, Visualize, Model, Evaluate and Unsupervised, and then push into the Text and Image Analytics add-ons. You will load and repair messy files, read a dataset honestly before touching a model, build scatter plots and heat maps that actually answer a question, train decision trees, k nearest neighbours, logistic regression, random forests and neural networks, and then, most importantly, learn to test them properly with cross validation, confusion matrices and ROC curves instead of trusting a single flattering number.

You will cluster data with k means and hierarchical clustering, shrink hundreds of columns with PCA and t-SNE, mine a corpus of text into a word cloud and a sentiment score, and train an image classifier using image embeddings. Every class is taught live by a real instructor in a small batch, the same mentorship-first method Modern Age Coders has used since 2020 to teach 10,000+ students across 25+ countries. Nothing here is a pre-recorded video. You finish with 24 saved workflows, a capstone data project of your own choosing, and an Orange report you can hand to a science fair judge, a school teacher, or a college admissions reader who asks what you actually built.

WHAT MAKES THIS COURSE DIFFERENT

- No programming prerequisite, and no pretending: you build genuine machine learning pipelines from Class 1, using the real algorithms, not simplified imitations.
- Free software, forever: Orange is open source under the GNU GPL v3 or later, so there is no subscription, no trial clock, and nothing to cancel when the course ends.
- Honest evaluation is taught as a core skill, not an afterthought: cross validation, confusion matrices, ROC curves and baseline comparison start in Phase 3 and never stop.
- Real datasets, not tidy classroom toys: you clean missing values, fix wrong variable types, and deal with data that argues back.
- Every class ends with a saved .ows workflow, so by Class 24 you own a portfolio of 24 pipelines you can reopen, edit, and explain.
- A bridge to code, not a dead end: Orange is written in Python, and the Python Script widget lets you drop into real Python inside the canvas whenever you are ready.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop running Windows, macOS or Linux, with enough room to install

Orange, plus a stable internet connection for live classes and the image embedding classes

Coding experience: None. This course is deliberately built for teens who have never written a line of code, and everything is done by connecting widgets on a canvas. Students who already know some Python are not held back, since the Python Script widget lets them extend any workflow

Maths background: School-level comfort with averages, percentages and reading a graph is enough. Every statistical idea used, from distributions to probability thresholds, is taught inside the course when it is needed

Accounts: No account is required to use Orange, which is free and open source software you download and run on your own machine

Mindset: Curiosity about a real question you would like answered with data, and willingness to accept an answer you did not expect, including the answer that the data does not support your hunch

WHO THIS COURSE IS FOR

Curious teens without coding: Ages 13-18 who are interested in AI and data but have not learned to program yet, and do not want to wait a year before touching real machine learning

Science fair and project students: Students preparing a science fair entry, school research project or IB-style investigation who need genuine analysis behind the poster rather than a spreadsheet chart

Future data and cs applicants: Teens aiming at computer science, data science, statistics, economics or biology at university who want the concepts before the syntax

Students already learning python: Young coders who can write Python but have never evaluated a model honestly, and want to learn cross validation, ROC and leakage properly on a visual canvas first

Students with a dataset and a question: Anyone who runs a club, a team, a channel or a small project and already has data they suspect is hiding something worth knowing

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: THE CANVAS AND YOUR DATA -----

Classes 1 to 5. Install Orange, learn how visual programming actually works, and get genuinely good at the unglamorous part every real analyst spends most of their time on: loading, reading, repairing and reshaping a dataset before a single model is trained.

Class 1: What Data Mining Is, and Meeting Orange

Topics:

- What data mining means in plain language: finding patterns in data that are too large or too tangled for a person to spot by reading rows
- Where you already meet it every day: spam filters, recommendation feeds, fraud alerts, medical screening, and school analytics dashboards
- Meet Orange: free and open source machine learning and data visualisation software from the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana
- Why it is free forever: Orange is released under the GNU General Public License version 3 or later, so there is no subscription and no trial period
- Installing Orange on your own machine: the standalone installer for Windows and macOS,

package options on Linux, and confirming the version under Help then About

- The anatomy of the window: the widget toolbox on the left, the canvas in the middle, and the six default widget sets Data, Transform, Visualize, Model, Evaluate and Unsupervised
- Visual programming in one sentence: place widgets on the canvas, connect them with lines, and data flows along those lines from one widget to the next

Projects built:

- Install Orange on your own computer, open a blank canvas, place a File widget and a Data Table widget, connect them, and take a screenshot proving data flowed from one to the other

Practice: Open every one of the six default widget sets in the toolbox and write down one widget from each that you cannot yet explain.

Assessment: A screenshot of your Orange version from Help then About, plus your two-widget canvas with data visible in the Data Table.

Class 2: Loading Data and Reading It Honestly

Topics:

- The File widget: opening .csv, .tsv, Excel files and Orange's own .tab format, and what happens when a file will not load cleanly
- The Datasets widget: browsing the bundled example datasets so you always have something real to practise on
- Data Table: sorting, selecting rows, and actually looking at your data before assuming anything about it
- Data Info: how many instances, how many features, how many missing values, and why that last number changes everything you do next
- Variable types in Orange and why they matter: numeric, categorical, text, and datetime, and what breaks when a type is guessed wrong
- Roles: feature, target and meta attribute, and why choosing the target is the single most important click in a supervised workflow
- The habit that separates good analysts from lucky ones: describe your dataset in one paragraph before you build anything on top of it

Projects built:

- Load a real .csv you care about (school marks, cricket or football statistics, weather, a game leaderboard), inspect it in Data Table and Data Info, and write a one paragraph description of what each column means

Practice: Deliberately load a file with a wrongly typed column, find it in Data Info, and note exactly which widget downstream would have failed.

Assessment: Your dataset description paragraph plus a Data Info screenshot showing instance count, feature count and missing values.

Class 3: Fixing the Data Nobody Wants to Fix

Topics:

- Edit Domain: renaming variables, changing a variable's type, merging categories, and reordering values so charts read in a sensible order
- Color: assigning meaningful colours to categories once, so every chart downstream stays consistent
- Column Statistics: spotting the columns that are almost entirely empty, constant, or secretly duplicated
- Select Columns: choosing features, setting the target variable, and pushing identifiers into the meta section instead of feeding them to a model
- Select Rows: filtering with conditions to isolate the part of the data you actually want to study
- Impute: what to do about missing values, and the honest trade-off between removing rows, filling with an average, and modelling the missing value
- The rule that saves projects: never delete data silently, always leave the original file untouched and do your fixing inside the workflow

Projects built:

- Take a deliberately messy dataset provided in class, and build a five widget repair pipeline that renames variables, fixes types, filters bad rows and imputes missing values, ending in a clean Data Table

Practice: Rebuild the same repair pipeline on your own dataset from Class 2 and record how many rows survived.

Assessment: Your repair workflow plus a two-line note on every judgement call you made and why.

Class 4: Reshaping, Combining and Summarising Data

Topics:

- Merge Data: joining two tables on a shared column, and what happens to rows that have no match on the other side
- Concatenate: stacking datasets that share the same columns, for example three months of records in three files
- Group by and Aggregate Columns: turning thousands of rows into the handful of summary numbers a question actually needs
- Pivot Table: cross-tabulating two categorical variables to see the shape of the relationship immediately
- Transpose, Unique and Select by Data Index: the small tools that rescue a badly shaped table
- Data Sampler: taking a random subset for fast experiments, and the idea of a held-out sample you do not look at
- Formula: creating a new calculated column from existing ones, for example a ratio or a difference

Projects built:

- Combine two related datasets with Merge Data, create one new calculated column with Formula, and produce a Pivot Table that answers a specific question you write down first
- Practice: Use Group by to produce three summary statistics from your own dataset and check each one by hand on a few rows.
- Assessment: Your combined dataset workflow plus the question you set and the pivot table that answers it.

Class 5: Preprocess, Save and Share Your Workflow

Topics:

- The Preprocess widget: normalisation, scaling, discretisation and continuation collected into one reusable block
- Why preprocessing choices belong inside the workflow, not in a spreadsheet you edited by hand and cannot reproduce
- Randomize and Purge Domain: shuffling values to test assumptions, and cleaning out unused categories
- Save Data: exporting a cleaned dataset so other tools can read it
- Saving the workflow itself as an .ows file: an XML description of your widgets, their settings and their connections
- The crucial detail about .ows files: they save the workflow and widget settings but not the data, so a shared workflow needs its data file alongside it
- Workflow hygiene: naming widgets, laying out the canvas left to right, and adding notes so a reader can follow your reasoning

Projects built:

- Turn your Class 3 and Class 4 work into one tidy, annotated, left-to-right workflow, save it as an .ows file, and swap it with a classmate to check that they can rerun it with the data file

Practice: Reopen a classmate's .ows file and write down the first moment you were confused; that is a labelling problem to fix in yours.

Assessment: A shared .ows file that a classmate successfully reran, plus the feedback note they wrote.

PHASE 2: SEEING THE DATA

Classes 6 to 10. Charts are not decoration, they are how you find the question worth modelling. This phase turns you into someone who can look at an unfamiliar dataset and, within ten minutes, produce the three visuals that explain what is going on.

Class 6: Distributions, Box Plots and Violin Plots

Topics:

- Distributions: the shape of a single variable, and why the average alone hides almost everything interesting
- Reading a histogram properly: bin width, skew, gaps, and the long tail that contains the exceptions
- Splitting a distribution by a categorical variable to compare groups in the same chart
- Box Plot: median, quartiles, whiskers and outliers, and how to read five numbers at a glance
- Violin Plot: when the shape of the distribution matters more than the summary statistics
- Comparing groups fairly: unequal group sizes, and why a difference in medians is not automatically a finding
- Choosing between the three: what question each chart is genuinely the best answer to

Projects built:

- Build a three-chart comparison of one numeric variable across one categorical group in your dataset, using Distributions, Box Plot and Violin Plot, and write which chart told you the most

Practice: Find one variable in a bundled dataset whose average is actively misleading and explain why using its distribution.

Assessment: Your three-chart workflow plus a short paragraph on what the shape of the data revealed.

Class 7: Scatter Plots and Finding Informative Projections

Topics:

- Scatter Plot: two numeric variables, one point per instance, and colouring points by the target variable
- Reading relationships: positive, negative, none, clustered, curved, and the difference between a pattern and a coincidence
- Using size, shape and colour as extra dimensions without making the chart unreadable
- Find Informative Projections: letting Orange rank which pairs of variables separate your classes best
- Rank: scoring individual features by how much information they carry about the target
- Selecting points directly on the chart and sending that selection to a Data Table for inspection
- The trap to name out loud: a strong scatter relationship is correlation, and correlation is not causation

Projects built:

- Use Find Informative Projections to identify the two most separating variables in a classification dataset, then build the scatter plot it suggests and select the most interesting cluster of points into a Data Table

Practice: Rank the features of your own dataset and write down whether the top-ranked feature surprised you.

Assessment: Your scatter plot plus a written interpretation naming one relationship and one alternative explanation for it.

Class 8: Bar Plots, Line Plots and Categorical Relationships

Topics:

- Bar Plot: comparing categories honestly, including the axis choices that quietly exaggerate a difference
- Line Plot: showing values across an ordered sequence, and when a line implies a continuity that does not exist
- Sieve Diagram: seeing whether two categorical variables are independent, using the size of the gaps between observed and expected
- Mosaic Display: reading a whole contingency table as areas rather than numbers
- Venn Diagram: comparing overlapping selections from different parts of your workflow
- Chart honesty: truncated axes, cherry-picked ranges, and misleading aggregation, with real examples to critique
- Choosing the chart from the question, not from the chart menu

Projects built:

- Build a two-variable categorical analysis using Sieve Diagram and Mosaic Display, then present the same finding once honestly and once misleadingly with a Bar Plot, and explain exactly what you changed

Practice: Find a chart in a news article or an advertisement and list every choice that shapes how it is read.

Assessment: Your honest and misleading chart pair, plus a written explanation of the manipulation.

Class 9: Heat Maps, Correlations and Distance Maps

Topics:

- Heat Map: hundreds of rows and dozens of columns compressed into one image you can actually scan
- Clustering rows and columns inside the heat map so similar things sit next to each other
- Correlations: ranking every pair of numeric variables by how strongly they move together
- Reading a correlation coefficient without over-claiming: strength, direction, and what it says nothing about
- Distances and Distance Map: what it means for two rows to be near each other, and why the distance measure you pick changes the answer
- Normalisation before distance: why an unscaled column with big numbers can dominate every comparison
- Multicollinearity in plain language: two features saying the same thing, and why that matters later when you model

Projects built:

- Build a Correlations and Heat Map pair on a numeric dataset, identify one pair of variables that are effectively duplicates, and justify which one you would keep

Practice: Run the same Distance Map with and without normalisation and describe how the picture changed.

Assessment: Your heat map workflow plus the duplicate-feature decision with your reasoning.

Class 10: Telling the Story with the Orange Report

Topics:

- The Report system: most widgets have a report button in the status bar that sends the current view into a running report
- Which widgets cannot report, and why: the ones that only pass data along rather than showing a result
- Building a report in the order a reader needs it: question, data description, evidence, and only then the conclusion
- Adding your own comments to report sections so the reader follows the reasoning, not just the pictures
- Saving the report as .html or .pdf to hand in, and as .report so a reader can reopen the workflow that produced each section
- Opening a saved report through File then Open Report, and showing the report view with Shift and R
- Presenting a data finding out loud in two minutes without a single unsupported claim

Projects built:

- Produce a complete visual story on your own dataset: one question, three charts chosen deliberately, written commentary on each, exported as a PDF report

Practice: Give your two minute presentation to someone at home and note every question they asked that your report did not answer.

Assessment: Your exported PDF report plus the list of questions your audience asked.

----- PHASE 3: YOUR FIRST MACHINE LEARNING MODELS -----

Classes 11 to 16. Training a model in Orange takes about thirty seconds. Knowing whether the model is any good takes the rest of this phase, and it is the part that actually matters. Every model you build here is tested properly before you are allowed to be impressed by it.

Class 11: Supervised Learning and the Baseline You Must Beat

Topics:

- Supervised learning defined: you have examples where the answer is already known, and you

- want a model that generalises to examples where it is not
- Classification versus regression: predicting a category versus predicting a number, and how the target variable decides which you are doing
- Setting the target with Select Columns, and the discipline of checking your target before every model
- The standard Orange modelling shape: data, then a learner widget, then Test and Score, then results
- The Constant widget: a deliberately stupid model that always predicts the most common class, and why it is the most useful widget in the toolbox
- Why a baseline is non-negotiable: a model that is 92 percent accurate on data that is 92 percent one class has learned nothing at all
- Class imbalance in one picture, using Distributions on the target variable before you model

Projects built:

- Take a classification dataset, chart its target distribution, run the Constant baseline through Test and Score, and write down the accuracy number that every later model in this course must beat

Practice: Find a dataset where the majority class is above 80 percent and state what accuracy would prove nothing.

Assessment: Your baseline score plus a written explanation of why accuracy alone can be a dishonest metric.

Class 12: Decision Trees, the Model You Can Actually Read

Topics:

- How a decision tree learns: repeatedly finding the split that best separates the classes, then repeating on each side
- The Tree widget and its settings: minimum instances in a leaf, maximum depth, and when to stop splitting
- Tree Viewer: walking the actual tree your data produced, node by node, and reading the rule at each branch
- Pythagorean Tree and Pythagorean Forest: seeing tree structure and forest diversity as shapes
- Overfitting explained through trees: a tree deep enough to memorise every training row is useless on new data
- Why interpretability matters: a model a human can read is a model a human can argue with, correct and trust
- Reading a tree back as plain English rules you can say out loud

Projects built:

- Train a decision tree on your dataset, open Tree Viewer, and write out the three clearest decision rules in plain English, then deliberately overfit by removing the depth limit and describe what changed

Practice: Change minimum instances in a leaf across three values and record the tree size each time.

Assessment: Your three written rules plus a before and after comparison of the limited and overfitted trees.

Class 13: k Nearest Neighbours, Naive Bayes and Preprocessing That Matters

Topics:

- k Nearest Neighbours: classify a new point by looking at the k closest known points and taking a vote
- Choosing k, and what happens at the extremes: k of one memorises noise, a very large k predicts the majority every time
- Why kNN is the model that most needs normalisation, connecting straight back to the distance work in Class 9
- Naive Bayes: probability-based classification, and the naive independence assumption stated honestly
- Where Naive Bayes shines despite that assumption, especially on text-like data with many features
- Building preprocessing into the model pipeline with the Preprocess widget so the same treatment applies at training and prediction
- Discretize and Continuize: converting between numeric and categorical when a model requires it

Projects built:

- Compare kNN with and without normalisation in the same Test and Score, add Naive Bayes as a third learner, and write a paragraph explaining the accuracy gap that normalisation caused

Practice: Sweep k across five values and plot or tabulate how accuracy moves.

Assessment: Your three-learner comparison table plus the normalisation explanation.

Class 14: Testing Honestly with Cross Validation and Confusion Matrices

Topics:

- The cardinal sin of machine learning: scoring a model on the same data it was trained on
- Train and test split: holding data back so the model faces genuinely unseen examples
- Cross validation explained simply: split the data into k folds, train on k minus one, test on the one held out, repeat, then average
- Test and Score in practice: comparing several learners side by side under identical conditions
- Reading the metrics without cherry-picking: classification accuracy, AUC, F1, precision and recall, and what each one is blind to
- Confusion Matrix: true positives, false positives, true negatives and false negatives as a table of real consequences
- Precision versus recall through a real dilemma: a disease screening test that misses cases

versus one that raises false alarms

Projects built:

- Run every model you have built so far through Test and Score with 10-fold cross validation against the Class 11 baseline, then open Confusion Matrix and describe, in consequences rather than numbers, what each type of error would mean

Practice: Take the same model, score it on training data and on cross validation, and record the difference.

Assessment: Your comparison table including the baseline, plus the written consequence analysis of your confusion matrix.

Class 15: Logistic Regression, ROC Curves and Nomograms

Topics:

- Logistic Regression: predicting a probability rather than a bare label, and what a probability of 0.7 actually commits you to
- The decision threshold: why 0.5 is a default, not a law, and how moving it trades precision against recall
- ROC Analysis: reading the curve as every possible threshold plotted at once
- AUC in one sentence: the chance that the model ranks a random positive case above a random negative one
- Performance Curve and Calibration Plot: whether the predicted probabilities can be trusted as probabilities
- Nomogram: reading a logistic regression model as a visual scoring sheet showing which features push a prediction which way
- Regularisation in plain language: penalising complexity so the model does not chase noise

Projects built:

- Train logistic regression alongside your decision tree, compare them in ROC Analysis, then use the Nomogram to write down the three features that most influence the prediction and in which direction

Practice: Move the classification threshold and record how precision and recall trade off at three settings.

Assessment: Your ROC comparison plus the nomogram feature explanation in plain English.

Class 16: Ensembles, Neural Networks and Making Predictions

Topics:

- Random Forest: many decision trees trained on different samples, voting together, and why a crowd of weak trees beats one strong one
- Gradient Boosting: building trees in sequence where each one corrects the previous one's mistakes
- Support Vector Machines: finding the widest possible boundary between classes
- Neural Network in Orange: hidden layers, activation, and honest expectations about small datasets
- The bias and variance trade-off explained with the models you have already trained
- Predictions: feeding brand new unlabelled data into a trained model and reading the output with its probabilities
- Save Model and Load Model: keeping a trained model so you do not retrain every time

Projects built:

- Build a six-model showdown, baseline, tree, kNN, logistic regression, random forest and neural network, in one Test and Score, then take the winner, save it, and run genuinely new rows through Predictions

Practice: Note which model won on accuracy and which won on AUC, and explain why they can disagree.

Assessment: Your six-model comparison table plus a paragraph defending your model choice on grounds other than the top accuracy number.

PHASE 4: FINDING HIDDEN STRUCTURE, TEXT AND IMAGES

Classes 17 to 21. Not every dataset comes with answers attached. This phase covers unsupervised learning, where the model finds structure nobody labelled, and then applies the same canvas to two data types that do not look like spreadsheets at all: text and images.

Class 17: k Means Clustering and Silhouette Scores

Topics:

- Unsupervised learning: no target column, no right answers, just structure waiting to be found
- How k means works step by step: pick k centres, assign every point to the nearest, move each centre to the middle of its group, repeat
- The k Means widget, including the option to score a range of k values and let the silhouette guide the choice
- Silhouette score explained: how well each point sits inside its own cluster compared to the nearest other cluster
- Silhouette Plot: spotting the individual points that were forced into the wrong group
- Interactive k-Means from the Educational add-on: stepping through the algorithm one iteration at a time and watching the centres move
- The honest limitation: k means will happily produce clusters even when the data has no real groups at all

Projects built:

- Cluster a dataset with k means across a range of k, choose k using the silhouette score rather than intuition, then colour a Scatter Plot by cluster and describe each group in plain words

Practice: Run k means on deliberately random data and explain why the clusters it returns mean nothing.

Assessment: Your silhouette-justified choice of k plus a written description of what each cluster represents.

Class 18: Hierarchical Clustering, DBSCAN and Distance

Topics:

- Hierarchical Clustering: building a tree of nested groups instead of committing to a fixed number
- Reading a dendrogram: where to cut it, and how the cut height decides how many clusters you get
- Linkage methods, single, average, complete and Ward, and how the choice changes the shape of the result
- The Distances widget: Euclidean, Manhattan, cosine and others, and matching the measure to the data
- Distance Map and Distance Matrix: seeing similarity between every pair of rows as an image
- DBSCAN: density-based clustering that finds arbitrary shapes and, unlike k means, is allowed to call a point noise
- Louvain clustering and Self-Organizing Map named as further options for later exploration

Projects built:

- Cluster the same dataset three ways, k means, hierarchical and DBSCAN, put the results side by side, and write which method matched the structure you can see in the scatter plot and why

Practice: Cut the same dendrogram at three different heights and record how the cluster count and meaning change.

Assessment: Your three-method comparison plus a justified recommendation of one method for this dataset.

Class 19: PCA, t-SNE and Outliers

Topics:

- The curse of dimensionality in one example: why fifty columns can be harder to work with than five
- PCA: building new combined axes that capture as much of the variation as possible, in order
- Reading the explained variance curve to decide how many components are enough
- What you lose with PCA: components are combinations, so individual feature meaning becomes harder to interpret
- t-SNE and MDS: squeezing high-dimensional data into two dimensions for visualisation, and why the distances between separate clusters in a t-SNE plot should not be over-read
- Outliers: detecting the rows that do not belong, and deciding case by case whether they are errors or the most interesting thing in the file
- Using PCA output as the input to clustering, and comparing the result against clustering the raw features

Projects built:

- Take a wide dataset, reduce it with PCA, visualise it with t-SNE coloured by a known label, run Outliers, and write about the three most unusual rows you found

Practice: Compare clustering on raw features against clustering on PCA components and note any difference in silhouette.

Assessment: Your t-SNE visualisation plus a written investigation of the outliers you detected.

Class 20: Text Mining, from Words to Sentiment

Topics:

- Installing add-ons through Options then Add-ons, and what the Text add-on adds to the toolbox
- Corpus and Import Documents: loading a collection of documents as data, from bundled corpora to your own folder of text files
- Preprocess Text: tokenisation, lowercasing, stop word removal, normalisation and filtering, and how each step changes what the model sees
- Bag of Words: turning documents into numbers by counting words, and what that representation throws away
- Word Cloud and Concordance: the quick overview and the close reading, used together
- Sentiment Analysis: scoring documents as positive or negative, with an honest discussion of sarcasm, context and language limits
- Topic Modelling: letting the algorithm propose themes across a corpus, and the human judgement needed to name them

Projects built:

- Build a full text pipeline on a corpus you choose, book reviews, song lyrics, news headlines or your own writing, ending in a word cloud, a sentiment distribution and a set of topics you name yourself

Practice: Rerun your pipeline with stop words switched off and describe how the word cloud degraded.

Assessment: Your text workflow plus your named topics with a one line justification for each name.

Class 21: Image Analytics and Your First Image Classifier

Topics:

- The Image Analytics add-on: Import Images, Image Viewer, Image Grid, Image Embedding and Save Images
- Import Images: pointing Orange at a folder of pictures and getting a dataset where each row is an image
- Image Embedding explained without maths: a pretrained network converts each picture into a

- list of numbers describing what it contains
- Why embeddings matter: once an image is a row of numbers, every classifier and clustering widget you already know works on it unchanged
- Building the classifier: embeddings into logistic regression or a neural network, then straight into Test and Score
- Image Grid and clustering images by similarity, so you can see what the model thinks looks alike
- The practical warning: embedding needs a working internet connection for the server-based embedders, and small unbalanced image sets produce flattering, fragile results

Projects built:

- Collect or use a provided set of images in at least three labelled folders, embed them, train and cross validate a classifier, then test it on pictures you took yourself and honestly report where it failed

Practice: Cluster your images without labels and describe what visual property the clusters actually captured.

Assessment: Your image classification workflow with cross validated scores plus a written account of its failure cases.

----- PHASE 5: ETHICS, CAPSTONE AND SHOWCASE -----

Classes 22 to 24. A model that is technically correct and socially careless is still a bad model. You finish the course by learning what can go wrong, then building and presenting an end to end project of your own from raw data to defended conclusion.

Class 22: Bias, Leakage and Honest Conclusions

Topics:

- Where bias enters: who collected the data, who is missing from it, and which outcome someone chose to call success
- Data leakage in plain terms: a feature that secretly contains the answer, producing a model that scores brilliantly and fails completely in the real world
- How to hunt leakage: a suspiciously perfect score is a bug report, not a celebration
- Correlation versus causation, revisited with your own charts from Phase 2 as the examples
- The Fairness add-on named as a real tool for auditing models on sensitive attributes
- The Explain add-on named as a way to ask which features drove a specific prediction
- Writing conclusions that survive scrutiny: state the finding, state the uncertainty, state what would disprove you

Projects built:

- Audit one of your own earlier workflows for leakage and bias, write a short honest report on what you found, and correct the workflow if the audit demands it

Practice: Take any published claim based on data and write down the three questions you would now ask about it.

Assessment: Your written audit, including at least one limitation of your own work stated plainly.

Class 23: Build Your Capstone

Topics:

- Choosing a capstone question that is specific, answerable with data you can actually obtain, and interesting to someone other than you
- Sourcing a dataset responsibly: open government portals, Kaggle, school or club records used with permission, and checking the licence
- Planning the pipeline on paper before touching the canvas: load, clean, explore, model, evaluate, conclude
- Building the workflow with the discipline of Phase 1: named widgets, left to right layout, nothing edited by hand outside the canvas
- Beating your baseline, and being willing to report that a fancy model did not beat a simple one
- Mentor review checkpoint: an instructor walks your canvas with you and pushes on every unjustified step
- Preparing the artefacts: the .ows file, the data file, and the report skeleton

Projects built:

- Build your complete capstone workflow end to end, from raw file to evaluated model and written conclusion, reviewed live with your mentor

Practice: Rerun your capstone from a clean Orange session to prove it reproduces without hidden manual steps.

Assessment: A working capstone .ows file that reproduces from raw data, plus your stated baseline comparison.

Class 24: Present, Publish and Where Data Takes You Next

Topics:

- Assembling the final Orange report: question, data, method, evidence, result, limitations
- Exporting for different audiences: PDF for a teacher or judge, and .report so another analyst can reopen your workflows
- Presenting in five minutes: leading with the question, showing two charts, giving the number, and naming what you are unsure about
- Handling hard questions honestly, including how to say that you do not know
- Publishing your work: a portfolio folder of 24 workflows plus the capstone, ready for a science fair, a school submission or a college application
- The bridge to code: Orange is written in Python, so the Python Script widget lets you run

real Python inside the canvas, and every model here exists as a Python library too

- Where to go next: Python with pandas, statistics, further Orange add-ons like Time Series, Geo, Network and Explain, and the official Orange video tutorials and workflow examples

Projects built:

- Deliver your five minute capstone presentation to the batch, hand in the exported report, and receive written feedback plus your certificate assessment

Practice: Write a one page plan for the next data project you will build on your own, with its question, data source and baseline.

Assessment: Your final presentation, exported report and complete 24-workflow portfolio, reviewed for reasoning as much as for results.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: your first two-widget canvas, a described real dataset, a five widget messy-data repair pipeline, a merged and pivoted multi-table analysis, and a shared annotated .ows workflow a classmate successfully reran
- Phase 2: a three-chart distribution comparison, an informative-projection scatter plot with selected points, an honest versus misleading chart pair, a correlation and heat map duplicate-feature investigation, and a full exported PDF visual story
- Phase 3: a baseline accuracy every later model must beat, a readable decision tree written out as plain English rules, a kNN normalisation experiment, a 10-fold cross validated model comparison with confusion matrix consequences, an ROC and nomogram analysis, and a six-model showdown with saved model and live predictions
- Phase 4: a silhouette-justified k means clustering, a three-method clustering comparison, a PCA and t-SNE reduction with outlier investigation, a complete text mining pipeline with named topics, and an image classifier trained on your own labelled photos
- Phase 5: a leakage and bias audit of your own earlier work, the full capstone workflow reproducing from raw data, and a five minute presentation with an exported report
- Total: 24 saved workflows plus one capstone, every one reopenable, editable and explainable

Total projects: 24 hands-on workflows, one per class, finishing with an end to end capstone project and an exported Orange report

SKILLS THE STUDENT WILL HAVE AT THE END

- Visual Programming in Orange: canvas, widget sets, connections, channel selection, and saving reproducible .ows workflows
- Data Preparation: File and Datasets, Data Table, Data Info, Edit Domain, Column Statistics, Select Columns and Rows, Impute, and honest handling of missing values
- Data Wrangling: Merge Data, Concatenate, Group by, Aggregate Columns, Pivot Table, Transpose, Formula, Data Sampler and Preprocess
- Data Visualisation: Distributions, Box Plot, Violin Plot, Scatter Plot with informative projections, Bar and Line Plot, Sieve Diagram, Mosaic Display, Heat Map, Correlations and Distance Map
- Supervised Learning: Constant baselines, Tree with Tree Viewer, kNN, Naive Bayes, Logistic Regression with Nomogram, Random Forest, Gradient Boosting, SVM and Neural Network
- Honest Model Evaluation: Test and Score with cross validation, Confusion Matrix, ROC Analysis, Performance Curve, Calibration Plot, precision and recall trade-offs, overfitting and leakage detection
- Unsupervised Learning: k Means with silhouette selection, Hierarchical Clustering and dendrograms, DBSCAN, Distances, PCA, t-SNE, MDS and Outliers
- Text and Image Mining: Corpus, Preprocess Text, Bag of Words, Word Cloud, Concordance, Sentiment Analysis, Topic Modelling, Import Images, Image Embedding and Image Grid
- Communicating Results: the Orange Report system, exporting to PDF and .report, and presenting a finding with its limitations stated

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks

Hands on canvas work: 1-1.5 hours rebuilding each class workflow on your own machine with your own data

Project time: 1 hour completing each class deliverable, from cleaned datasets to evaluated models and reports

Review and doubts: 30-60 minutes reviewing your workflows, checking your conclusions against your evidence, and clearing doubts before the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for installation problems, add-on issues, and workflows that will not behave

Small batches: Small batches, with mini-batch and 1-on-1 options, so every student gets individual attention on their own canvas

Mentor guidance: Live guidance from mentors who build and evaluate real models, including a personal review of your capstone workflow

Project feedback: Feedback on every class deliverable, judged on reasoning and honest evaluation as much as on results

Living syllabus: Curriculum updated as Orange ships new versions and widgets, at no extra cost

CERTIFICATION DETAIL

Completion certificate: Modern Age Coders 'Orange Data Mining for Teens' certificate, earned by presenting your Class 24 capstone
Project portfolio: A folder of 24 saved .ows workflows plus your capstone, each one reproducible from its data file
Presented report: An exported Orange report (PDF and .report) documenting your capstone from question through evidence to stated limitations

WHERE THIS CAN LEAD

- Data-literate student who can take a raw dataset from file to defended conclusion without waiting for anyone to write code
- Strong entrant for science fairs, school research competitions and project-based coursework that rewards genuine method
- Prepared starting point for Python, pandas and scikit-learn, since the modelling and evaluation concepts are already understood
- Early groundwork for university study and later roles in data analysis, machine learning, research, bioinformatics and business analytics
- A portfolio of 24 reproducible workflows and one capstone that demonstrate reasoning and honesty, not just tool use

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does my child need to know programming before this course?

A: No. Orange is visual programming software: you build analysis by placing widgets on a canvas and connecting them, so no code is required at any point in the 24 classes. Students who already know Python can go further using the Python Script widget, which runs real Python inside the workflow, but it is entirely optional.

Q: What exactly is Orange, and is it really free?

A: Orange is open source machine learning and data visualisation software developed at the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana. It is released under the GNU General Public License version 3 or later and is free to download and use on Windows, macOS and Linux. There is no subscription, no trial period, and no student licence to renew.

Q: Is this real machine learning, or a simplified version for children?

A: It is real machine learning. The visual canvas changes how you assemble the pipeline, not what the pipeline does.

Q: What age is this course for?

A: It is designed for teens aged 13 to 18. Orange itself requires no account and has no age restriction, so the age range reflects the reading level, the statistical reasoning and the project expectations of the curriculum rather than any restriction in the software. Younger students who are strong in maths can join after a free demo class where we check the fit.

Q: What kind of computer is needed?

A: Any reasonably modern Windows, macOS or Linux laptop or desktop that can install Orange, plus a stable internet connection for the live classes. Most classes run comfortably on modest hardware because the datasets are student sized. The image analytics class in Phase 4 also needs a working internet connection, since the image embedders run on a server.

Q: How is this different from your Python data science course for teens?

A: The Python course teaches you to write the analysis in code with pandas and Python libraries, which means you spend the early weeks on syntax. This course removes the syntax barrier so students reach models, evaluation and clustering in the first month, and it goes deeper on honest evaluation and visual exploration. Many students take Orange first and then move to Python, arriving with the concepts already in place.

Q: Will my child be able to use this for a school or science fair project?

A: Yes, and Class 23 and Class 24 are built specifically for that. Students choose their own question, source a dataset responsibly, build a reproducible workflow, compare against a baseline, and export an Orange report as PDF for a teacher or judge, or as a .report file so another analyst can reopen the underlying workflows. The whole capstone is designed to be defended out loud.

Q: What are add-ons, and are they included?

A: Add-ons are optional widget packs installed from inside Orange through Options then Add-ons. This course installs and uses the Text add-on in Class 20 and the Image Analytics add-on in Class 21, and introduces the Educational, Explain and Fairness add-ons where they are relevant. They are free, like Orange itself, and we install them together in class.

Q: What does my child actually walk away with?

A: Twenty four saved .ows workflows they can reopen and edit, a capstone project built from raw data to written conclusion, an exported report, a Modern Age Coders completion certificate, and, more usefully than any of those, the habit of asking how a model was tested before believing what it claims.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/orange-data-mining-visual-machine-learning-course-for-teens>

=====

COURSE 96 OF 118: PCAP CERTIFICATION PREP: CERTIFIED ASSOCIATE IN PYTHON PROGRAMMING

=====

Course link: <https://learn.modernagecoders.com/courses/pcap-python-certification-prep-course>
Subject area: Python Certification
Designed for: Working professionals
Who can join: Anyone with basic Python (PCEP level or equivalent) preparing for the PCAP associate certification
Course length: 4 months (16 weeks), joinable any month
Time needed: 2 live classes/week + weekly coding practice and mock questions
Certificate: Course-completion certificate from Modern Age Coders (the PCAP credential itself is awarded by the Python Institute)

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online PCAP-31-03 preparation covering every exam objective: modules and packages, exceptions, strings, object-oriented Python, generators, closures and file processing, with full timed mock exams.

In one line: Every exam objective taught properly, then drilled against the real format, so you sit the exam already knowing you will pass.

ABOUT THE PROGRAMME

PCAP, the Certified Associate in Python Programming, is the Python Institute's associate-level credential and the natural step up from PCEP. It is a genuine test of Python fluency rather than a quiz: the current PCAP-31-03 exam is 40 questions in single-choice and multiple-choice form, taken in 65 minutes, with a pass mark of 70 percent, which means at least 28 correct answers. It is delivered through Pearson VUE test centres or OnVUE online proctoring, and the exam fee is set by the Python Institute (currently around USD 295), paid directly to them. What makes PCAP harder than it looks is that it examines the parts of Python most self-taught programmers skip. Modules and packages. Exception handling done properly, not a bare try block. String methods in detail. Object-oriented Python including class variables and inheritance. Generators, closures and list comprehensions. File processing. Plenty of working developers write Python daily and still miss these, because everyday scripting never forces them. This course covers every objective in that list and treats the exam format as a skill of its own. You write real code every week, then answer questions in the exact style the exam uses, including the ones designed to punish a guess. The last month is full timed mocks with question-by-question review, so by the time you book the test you already know your score range.

WHAT MAKES THIS COURSE DIFFERENT

- Built around the current PCAP-31-03 objectives: modules and packages, exceptions, strings, OOP, generators and closures, and file processing
- Taught as real Python first and exam technique second, so the knowledge survives after the certificate
- The exam format is practised deliberately: 40 questions in 65 minutes at a 70 percent pass mark leaves no room for slow reading
- Covers the topics self-taught programmers usually miss, which is exactly where PCAP candidates fail
- Full timed mock exams with question-by-question review and an error log, so weak areas are fixed before test day
- Live, small batches where your code is reviewed on the spot, not recorded videos watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Modules, packages, strings and exception handling done to exam depth
Phase 2: Object-oriented Python: classes, class variables, inheritance, polymorphism and special methods
Phase 3: Generators, closures, comprehensions, file processing, then full timed mock exams

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What You Need to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: MODULES, PACKAGES, STRINGS AND EXCEPTIONS

The first block of exam objectives, taught as real Python and drilled in exam form.

Month 1: Modules, Packages and the Standard Library (Weeks 1-4)

Importing and Module Basics

Topics:

- import, from-import, and aliasing, and how each affects the namespace
- What actually happens when Python imports a module
- The `__name__` variable and the main guard
- Where Python looks for modules, and `sys.path`
- Exam traps around name collisions and shadowing

Projects built:

- Write a small multi-module program and import across it every supported way

Practice: 20 exam-style questions on imports and namespaces

Standard Library Essentials

Topics:

- `math`, `random`, `platform` and `datetime` as the exam uses them
- Reading the documentation quickly and correctly
- Predicting output from unfamiliar standard-library calls
- Common function signatures worth memorising
- Where the exam prefers a specific idiom

Projects built:

- Solve a set of tasks using only the standard library, no third-party packages

Practice: 20 output-prediction questions on standard-library calls

Packages, `__init__` and Distribution

Topics:

- How a package differs from a module
- The role of `__init__.py`
- Nested packages and relative versus absolute imports
- `pip` and dependency basics at exam depth
- Structuring a small package correctly

Projects built:

- Build a small package with nested sub-packages and import from it correctly

Practice: 15 package-structure and import questions

Assessment: Timed section test on modules and packages

Strings in Depth

Topics:

- Strings as immutable sequences, and what that implies
- Slicing, including negative indices and steps
- The string methods the exam actually tests, in detail
- Escape sequences, raw strings and formatting
- Comparing strings, and the ordering rules

Projects built:

- Write a text-processing utility using only string methods, no regular expressions

Practice: 25 string-method and slicing questions, mostly output prediction

Month 2: Exception Handling and Consolidation (Weeks 5-8)

The Exception Hierarchy

Topics:

- `try`, `except`, `else` and `finally`, and the exact order they run in
- The built-in exception hierarchy and why the order of `except` clauses matters
- Catching specific exceptions instead of bare `except`
- `raise`, and re-raising inside a handler
- Predicting which handler catches what

Projects built:

- Take a fragile script and make it robust with correctly ordered handlers

Practice: 20 exception-flow questions, mostly trace-the-output

Custom Exceptions and Defensive Code

Topics:

- Defining your own exception classes
- Inheriting from the right base exception
- `assert` and where it belongs
- Exception chaining and context
- Writing code that fails clearly rather than silently

Projects built:

- Design a small exception hierarchy for a described domain and use it properly

Practice: 15 custom-exception questions

Assessment: Timed section test on exception handling

Data Aggregates Refresher

Topics:

- Lists, tuples, dictionaries and sets, and the exam-relevant differences
- Mutability and the aliasing traps the exam loves

- Copying: assignment versus shallow versus deep
- Dictionary methods and safe key access
- Sorting with keys and reverse

Projects built:

- Solve a data-shaping task choosing the right structure at each step and justifying it

Practice: 25 questions on aggregates, heavy on mutability traps

Functions, Scope and Arguments

Topics:

- Positional, keyword and default arguments
- *args and **kwargs and how they bind
- Scope, the LEGB rule, and the global and nonlocal keywords
- The mutable-default-argument trap
- Recursion at exam depth

Projects built:

- Write functions exercising every argument form, then predict their behaviour before running them

Practice: 25 scope and argument-binding questions

Assessment: End-of-phase test across everything so far, in exam format

----- PHASE 2: OBJECT-ORIENTED PYTHON -----

The block that decides most PCAP results, taught until class behaviour is predictable rather than surprising.

Month 3: Classes, Inheritance and Special Methods (Weeks 9-12)

Classes, Instances and Attributes

Topics:

- Defining classes, __init__, and instance attributes
- Instance variables versus class variables, the single biggest PCAP trap
- How attribute lookup actually resolves
- Name mangling and the single and double underscore conventions
- Adding attributes dynamically and what that does

Projects built:

- Build a class where class and instance variables are deliberately contrasted, then predict every output

Practice: 25 questions on class versus instance attributes

Inheritance and Polymorphism

Topics:

- Single and multiple inheritance
- Method resolution order and how Python decides
- Overriding, and calling the parent with super()
- isinstance and issubclass, and when each is correct
- Polymorphism in practice

Projects built:

- Design a class hierarchy with overriding and demonstrate the resolution order

Practice: 25 inheritance and MRO questions

Special Methods and Introspection

Topics:

- __str__ and __repr__ and the difference the exam tests
- Operator overloading with special methods
- __dict__, and inspecting objects at runtime
- Properties and controlled attribute access
- Composition versus inheritance

Projects built:

- Write a class with operator overloading and full introspection support

Practice: 20 special-method questions

Assessment: Timed OOP section test in exam format

OOP Consolidation

Topics:

- Working through the OOP question styles the exam repeats
- Reading unfamiliar class code quickly and accurately
- Spotting the deliberate misdirection in the options
- Common wrong answers and why they look right
- Building speed without losing accuracy

Projects built:

- A full OOP-only mock section, reviewed question by question

Practice: 40 mixed OOP questions under time

----- PHASE 3: GENERATORS, CLOSURES, FILE PROCESSING AND FULL MOCKS -----

The last objectives, then the exam itself rehearsed until the result is predictable.

Month 4: Advanced Python and Exam Rehearsal (Weeks 13-16)

Comprehensions, Generators and Iterators

Topics:

- List, dictionary and set comprehensions, including conditions
- The iterator protocol: __iter__ and __next__

- Generator functions and the yield keyword
- Generator expressions and lazy evaluation
- Why a generator is not a list, and what breaks if you forget

Projects built:

- Rewrite an eager data pipeline as a lazy generator pipeline and compare behaviour

Practice: 25 comprehension and generator output questions

Closures, Lambdas and Functional Tools

Topics:

- Lambda functions and their limits
- Closures, and what a nested function captures
- nonlocal in a closure
- map, filter and sorted with key functions
- Functions as first-class objects

Projects built:

- Build a small closure-based counter and function factory

Practice: 20 closure and lambda questions

Assessment: Timed section test on generators and closures

File Processing

Topics:

- Opening files: modes, and text versus binary
- Reading with read, readline and readlines, and iterating a file
- Writing and appending safely
- The with statement and why it is the correct idiom
- Handling file exceptions properly, tying back to Phase 1

Projects built:

- Write a program that reads, transforms and writes files with full error handling

Practice: 20 file-processing questions

Full Timed Mock Exams

Topics:

- Full mocks at the real format: 40 questions in 65 minutes
- Pacing so no question is left unread, since there is no penalty for answering
- Eliminating options systematically on multiple-response questions
- Turning the error log into a final revision list
- Booking the exam: Pearson VUE test centre or OnVUE online proctoring

Projects built:

- Three complete timed mock exams, each reviewed question by question

Practice: Repeated full mocks until you clear 70 percent consistently with margin

Assessment: Final full mock scored against the 70 percent pass mark, with a go or no-go recommendation

WHERE THIS CAN LEAD

- PCAP certification, a vendor-recognised associate-level Python credential
- Junior Python developer, automation engineer or QA automation roles
- A solid base for data analysis and backend development
- A credible step toward the professional-level Python certifications
- Genuine intermediate Python fluency that outlives the certificate

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What exactly is the PCAP exam?

A: PCAP is the Certified Associate in Python Programming from the Python Institute. The current version, PCAP-31-03, is 40 questions in single-choice and multiple-choice form, taken in 65 minutes, with a pass mark of 70 percent, meaning at least 28 correct answers. It is delivered through Pearson VUE test centres or OnVUE online proctoring.

Q: What does PCAP actually test?

A: The objectives cover modules and packages, exception handling, strings, object-oriented programming including class variables and inheritance, generators and closures, list comprehensions, and file processing, along with data aggregates like tuples and dictionaries. It is precisely the intermediate Python that self-taught programmers tend to skip, which is why candidates who write Python daily still fail it.

Q: Do I need PCEP first?

A: No. PCEP is helpful but not required. What you do need is working basic Python: variables, conditionals, loops and simple functions. If you have that, this course takes you from there to PCAP standard. If you are starting from zero, begin with a foundational Python course first and come to this one after.

Q: How long does it take to be ready?

A: The course runs four months across 16 weeks with two live classes a week plus weekly practice. Readiness depends on your starting point and how much you code between classes. We do not guess: the final month is full timed mocks, and we recommend booking the exam only once you are consistently clearing 70 percent with margin.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/pcap-python-certification-prep-course>

=====

COURSE 97 OF 118: PRINCESS CODERS: COMPLETE CODING COURSE FOR GIRLS & WOMEN

=====

Course link: <https://learn.modernagecoders.com/courses/princess-coders-complete-coding-masterclass>

Subject area: Women in Technology - Comprehensive Coding Program

Designed for: Girls only batches

Who can join: Complete Beginner to Advanced Developer

Course length: 12 months (52 weeks)

Time needed: 10-15 hours/week recommended

Certificate: Princess Coders Certified Developer - Industry Recognized

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

One-year coding program for girls and women: web development, app creation, AI basics and creative coding, with tracks for kids, teens and adults.

In one line: From 'Hello World' to Leading the Tech World - A Journey Designed for Girls

ABOUT THE PROGRAMME

Princess Coders is more than a coding course, it's a transformative journey designed exclusively for girls and women of all ages. Whether you're a young girl dreaming of creating games, a teenager wanting to build apps, or a woman seeking a career change, this program provides a safe, supportive, and empowering environment to master coding. With age-appropriate tracks, female role models, girl-centric projects, and a supportive sisterhood community, you'll learn everything from creative web design to powerful app development, from AI basics to entrepreneurship in tech.

WHAT MAKES THIS COURSE DIFFERENT

- Exclusively designed for girls with female instructors and mentors
- Age-appropriate tracks: Princess Junior (8-12), Teen Techies (13-17), Women Warriors (18+)
- Girl-centric projects: fashion apps, social impact tools, creative platforms
- Safe, supportive all-girls learning environment
- Focus on both technical skills and confidence building
- Real female tech leaders as guest speakers
- Emphasis on creativity alongside logic
- Mother-daughter learning options available

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Creative Coding, Web Basics, Visual Programming

Phase 2: Frontend Magic (Months 4-6): Beautiful Websites, Interactive Design, Mobile-First

Phase 3: Backend Power (Months 7-9): Databases, APIs, App Development

Phase 4: Professional Skills (Months 10-12): Advanced Projects, AI/ML, Career Prep

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal education required - open to all girls and women

Coding experience: Complete beginner friendly - no prior experience needed

Equipment: Computer/laptop with internet (tablets supported for younger learners)

Time commitment: 10-15 hours per week flexibly scheduled

English: Basic English understanding (multi-language support available)

Motivation: Passion to learn and grow in technology

WHO THIS COURSE IS FOR

Students: School and college girls exploring tech careers

Working professionals: Women seeking career change to tech

Entrepreneurs: Women wanting to build their own tech products

Freelancers: Those seeking flexible tech careers

Kids: Young girls (8+) interested in technology

Anyone: Any girl or woman ready to start their tech journey

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CREATIVE CODING (MONTHS 1-3, WEEKS 1-13)

Start your coding journey with fun, creative projects. Build confidence while learning the

fundamentals of programming and web development.

Months 1-2: Introduction to Coding & Web Basics (Week 1-8)

Welcome to Princess Coders - Your Tech Journey Begins

Topics:

- Welcome to the sisterhood: meet your cohort
- Famous women in tech: inspiration and role models
- How technology shapes our world
- Setting up your princess coding workspace
- Choosing your coding avatar and username
- Introduction to computers and how they think
- Your first line of code: 'Hello, Princess Coder!'
- Visual programming with Scratch (for younger cohort)
- Introduction to problem-solving like a programmer
- Creating your first digital art with code
- Building your coder confidence journal
- Princess Coders community guidelines and support

Projects built:

- Create your Princess Coder profile page
- Design your digital business card
- Build an 'About Me' interactive story

Practice: Daily: 30 min creative coding challenges, 1 hour guided tutorials

HTML - Building Your Digital Kingdom

Topics:

- HTML as the structure of the web
- Creating your first webpage from scratch
- Text elements: telling your story online
- Images: adding visual magic to pages
- Links: connecting your digital world
- Lists: organizing information beautifully
- Tables: presenting data elegantly
- Forms: collecting information from users
- Semantic HTML: writing meaningful code
- Accessibility: making web inclusive for all
- HTML5: modern web capabilities
- Best practices for clean, organized code

Projects built:

- Personal blog about your interests
- Recipe collection website
- Digital diary or journal
- Online portfolio for your hobbies

Practice: Create 5 different themed webpages

CSS - Styling Your Digital Creations

Topics:

- CSS: the art of web design
- Colors: creating beautiful palettes
- Fonts: typography that speaks
- Spacing: the importance of breathing room
- Backgrounds: setting the mood
- Borders and shadows: adding depth
- CSS animations: bringing pages to life
- Responsive design: mobile-friendly sites
- CSS Grid: magazine-style layouts
- Flexbox: flexible arrangements
- Design principles for non-designers
- Creating your personal style guide

Projects built:

- Fashion lookbook website
- Interactive greeting card creator
- Photo gallery with filters
- Animated story presentation

Practice: Style 10 different page components creatively

JavaScript - Making Things Interactive

Topics:

- JavaScript: the language of interaction
- Variables: storing information
- Data types: numbers, text, true/false
- Functions: creating reusable magic
- Events: responding to user actions
- Conditionals: making decisions in code
- Loops: repeating with efficiency
- Arrays: organizing collections
- Objects: grouping related data
- DOM: controlling page elements
- Creating interactive forms
- Building simple games and quizzes

Projects built:

- Interactive quiz about your favorite topic
- Simple drawing application

- To-do list with style
 - Mood tracker application
- Practice: Build 5 interactive mini-apps

Month 3: Advanced JavaScript & Creative Projects (Week 9-13)

Advanced JavaScript Concepts

Topics:

- Advanced functions and callbacks
- Working with APIs: getting real data
- JSON: the language of data exchange
- Local storage: saving user preferences
- Error handling: graceful failures
- Asynchronous programming basics
- Promises and modern JavaScript
- ES6 features: arrow functions, destructuring
- Array methods: map, filter, reduce
- Creating reusable components
- Code organization and best practices
- Debugging like a detective

Projects built:

- Weather app with real data
- Quote generator with API
- Currency converter tool
- Personal finance tracker

Practice: Master 20 JavaScript challenges

Creative Coding & Visual Projects

Topics:

- Introduction to p5.js for creative coding
- Drawing with code: shapes and colors
- Animation principles in programming
- Interactive art installations
- Generative art: code that creates
- Sound and music with code
- Data visualization basics
- Creating infographics programmatically
- Game development fundamentals
- Physics in animations
- Particle systems and effects
- Sharing your creative work online

Projects built:

- Interactive art piece
- Music visualizer
- Simple platformer game
- Data story visualization

Practice: Create 5 artistic coding projects

Month 3 Integration Project

Topics:

- Planning a complete web application
- User experience (UX) principles
- User interface (UI) design basics
- Wireframing and prototyping
- Version control with Git
- Collaboration on GitHub
- Project documentation
- Testing your applications

Projects built:

- CAPSTONE: Complete web application of choice
- Options: Blog platform, E-commerce site, Social cause website, Educational game

Practice: Polish and deploy your project

Phase 1 Review & Assessment

Topics:

- Review of all foundation concepts
- Best practices consolidation
- Portfolio preparation
- Presentation skills for developers
- Celebrating achievements

Projects built:

- Portfolio website showcasing all projects
- Phase 1 final presentation

Assessment: Phase 1 completion certificate

Bridge Week - Preparing for Advanced Topics

Topics:

- Introduction to frameworks
- Modern development workflow
- Professional tool setup
- Career paths in tech
- Setting goals for next phase

Projects built:

- Personal learning roadmap

- Tech blog first post
- Assessment: Self-assessment and goal setting

Months 4-6: Frontend Development Mastery (Week 14-26)

React.js Fundamentals

Topics:

- Introduction to React: why frameworks matter
- Setting up React development environment
- Components: building blocks of apps
- JSX: writing HTML in JavaScript
- Props: passing data between components
- State: managing changing data
- Hooks: useState and useEffect
- Event handling in React
- Lists and keys in React
- Conditional rendering
- Component lifecycle
- React Developer Tools

Projects built:

- Social media feed component
- Product catalog with React
- Interactive dashboard
- Recipe app with categories

Practice: Build 10 React components

Advanced React & State Management

Topics:

- Advanced hooks: useContext, useReducer
- Custom hooks creation
- React Router for navigation
- Forms and validation in React
- API integration with React
- State management with Context API
- Introduction to Redux
- Component optimization
- Code splitting and lazy loading
- Testing React components
- Deployment of React apps
- React best practices

Projects built:

- Multi-page React application
- Shopping cart with Redux
- Blog platform with routing
- Chat application frontend

Practice: Create complex React applications

Modern CSS & UI Libraries

Topics:

- CSS-in-JS with styled-components
- Tailwind CSS for rapid development
- Material-UI components
- Bootstrap with React
- CSS animations and transitions
- Responsive design patterns
- Mobile-first development
- Progressive Web Apps basics
- Accessibility in modern apps
- Performance optimization
- SEO for single-page apps
- Design systems creation

Projects built:

- Responsive portfolio site
- Mobile app UI clone
- Animated landing page
- Component library creation

Practice: Master 5 different styling approaches

TypeScript & Modern Tools

Topics:

- TypeScript basics: adding types to JavaScript
- TypeScript with React
- Type safety benefits
- Interfaces and types
- Generic types
- Modern build tools: Vite, Webpack
- Package management with npm/yarn
- Linting and formatting
- Git advanced workflows
- Continuous Integration basics
- Documentation tools
- Open source contribution

Projects built:

- TypeScript React app
- Open source contribution
- npm package creation
- Documentation site

Practice: Convert projects to TypeScript

Phase 2 Capstone Project

Topics:

- Complex application planning
- User research and personas
- Advanced UI/UX implementation
- Performance optimization
- Deployment strategies
- Project presentation

Projects built:

- MAJOR CAPSTONE: Full Frontend Application
- Options: Social platform for causes, E-learning platform, Health & wellness app, Creative portfolio builder

Assessment: Phase 2 completion with advanced frontend certification

PHASE 2: FRONTEND DEVELOPMENT & DESIGN (MONTHS 4-6, WEEKS 14-26)

Master modern frontend development with React, create beautiful user interfaces, and build professional web applications.

Months 4-5: React & Modern Frontend (Week 14-21)

React.js Foundation

Topics:

- Understanding why React revolutionized web development
- Setting up your React development environment
- Components: thinking in reusable pieces
- JSX: the perfect blend of HTML and JavaScript
- Props: component communication made simple
- State: managing dynamic data
- React Hooks: modern React development
- useState for component state
- useEffect for side effects
- Building interactive user interfaces
- Component composition patterns
- React ecosystem overview

Projects built:

- Instagram-style photo grid
- Pinterest-inspired board creator
- Spotify playlist manager UI
- Fashion e-commerce product page

Practice: Build 15 reusable React components

Advanced React Patterns

Topics:

- Advanced hooks: useContext for global state
- useReducer for complex state logic
- Custom hooks: creating your own
- React Router: multi-page applications
- Protected routes and authentication flows
- Form handling and validation
- API integration and data fetching
- Error boundaries and error handling
- Performance optimization techniques
- React.memo and useMemo
- Code splitting for faster apps
- Testing React applications

Projects built:

- Multi-page blog with authentication
- Task management app like Notion
- Real-time chat application UI
- Online learning platform interface

Practice: Master 10 advanced React patterns

State Management & Data Flow

Topics:

- Understanding application state
- Context API for state sharing
- Redux: predictable state management
- Redux Toolkit: modern Redux
- Actions, reducers, and store
- Async operations with Redux Thunk
- Zustand: lightweight alternative
- React Query for server state
- Optimistic UI updates
- Caching strategies
- Real-time data synchronization

- State management best practices

Projects built:

- Shopping cart with global state
- Social media feed with Redux
- Collaborative workspace app
- Analytics dashboard with real-time data

Practice: Implement state management in 5 apps

UI/UX Design for Developers

Topics:

- Design principles every developer should know
- Color theory and creating palettes
- Typography and readability
- Spacing and visual hierarchy
- User experience (UX) fundamentals
- User interface (UI) best practices
- Design tools: Figma basics for developers
- Wireframing and prototyping
- Design systems and component libraries
- Accessibility standards (WCAG)
- Responsive and adaptive design
- Micro-interactions and animations

Projects built:

- Complete design system creation
- Redesign existing website
- Mobile-first application design
- Accessible form system

Practice: Design and implement 5 UI concepts

Modern Styling Solutions

Topics:

- CSS-in-JS with Styled Components
- Emotion for component styling
- Tailwind CSS mastery
- CSS Modules approach
- Sass/SCSS advanced features
- CSS Grid and Flexbox mastery
- Animation libraries: Framer Motion
- Creating smooth transitions
- Dark mode implementation
- Theming and customization
- Performance considerations
- Mobile-responsive techniques

Projects built:

- Animated portfolio website
- Theme-switchable application
- Mobile-first product showcase
- Interactive data visualization

Practice: Master 5 different styling approaches

Month 6: Advanced Frontend & Portfolio (Week 22-26)

Next.js & Modern Frameworks

Topics:

- Next.js: the React framework
- Server-side rendering (SSR)
- Static site generation (SSG)
- File-based routing system
- API routes in Next.js
- Image optimization
- SEO optimization techniques
- Incremental Static Regeneration
- Deployment with Vercel
- Alternative: Gatsby for static sites
- Alternative: Remix framework
- Choosing the right framework

Projects built:

- Blog platform with Next.js
- E-commerce site with SSR
- Portfolio with optimal SEO
- Documentation website

Practice: Build 3 different Next.js applications

Testing & Quality Assurance

Topics:

- Why testing matters
- Unit testing with Jest
- React Testing Library
- Integration testing strategies
- End-to-end testing with Cypress
- Test-driven development (TDD)
- Debugging techniques
- Browser DevTools mastery

- Performance testing
- Accessibility testing
- Visual regression testing
- Continuous Integration setup

Projects built:

- Add tests to existing projects
- TDD project from scratch
- E2E test suite creation
- Automated testing pipeline

Practice: Write comprehensive tests for 5 projects

Performance & Optimization

Topics:

- Web performance metrics
- Core Web Vitals understanding
- Lighthouse audits
- Bundle size optimization
- Code splitting strategies
- Lazy loading implementation
- Image optimization techniques
- Caching strategies
- Service workers basics
- Progressive Web Apps
- Performance monitoring
- Real user monitoring (RUM)

Projects built:

- Optimize slow application
- PWA transformation
- Performance dashboard
- Offline-first application

Practice: Optimize all portfolio projects

TypeScript Mastery

Topics:

- TypeScript fundamentals
- Types and interfaces
- Generics and utility types
- TypeScript with React
- Type-safe API calls
- Advanced TypeScript patterns
- TypeScript configuration
- Migration strategies
- Type guards and assertions
- Declaration files
- TypeScript best practices
- Team collaboration with TypeScript

Projects built:

- Full TypeScript React app
- Type-safe API client
- Component library in TypeScript
- Legacy code migration

Practice: Convert all projects to TypeScript

Phase 2 Final Project

Topics:

- Project planning and scope
- Technical architecture design
- Implementation best practices
- Code review and refactoring
- Documentation writing
- Deployment and hosting
- Portfolio presentation

Projects built:

- CAPSTONE: Production-Ready Frontend Application
- Must include: React, TypeScript, State Management, Testing, Responsive Design, Accessibility
- Options: Social impact platform, Educational tool, Creative showcase, Business solution

Assessment: Frontend Developer Certification

Months 7-9: Backend Development (Week 27-39)

Node.js Fundamentals

Topics:

- Backend development introduction
- Node.js: JavaScript on the server
- Setting up Node.js environment
- NPM and package management
- Core modules: fs, path, http
- Creating your first server
- Handling requests and responses
- Working with files
- Environment variables
- Asynchronous programming
- Promises and async/await

- Error handling patterns

Projects built:

- File management system
- Basic HTTP server
- CLI application
- API endpoint creation

Practice: Build 5 Node.js applications

Express.js & REST APIs

Topics:

- Express.js framework
- Routing and middleware
- RESTful API principles
- CRUD operations
- Request validation
- Error handling middleware
- Authentication basics
- CORS configuration
- File uploads handling
- API documentation
- Postman for testing
- API security basics

Projects built:

- Blog API with CRUD
- User management API
- File storage service
- Social media API

Practice: Create 5 different REST APIs

Databases & Data Modeling

Topics:

- Database fundamentals
- SQL vs NoSQL comparison
- MongoDB setup and basics
- Mongoose ODM
- Schema design patterns
- Relationships in MongoDB
- PostgreSQL basics
- SQL queries and joins
- Database security
- Backup strategies
- Database optimization
- Migration strategies

Projects built:

- User database system
- E-commerce database
- Blog with comments
- Analytics database

Practice: Design 10 database schemas

Authentication & Security

Topics:

- Authentication vs Authorization
- Password hashing with bcrypt
- JWT tokens explained
- Session management
- OAuth 2.0 basics
- Social login integration
- Security best practices
- Input validation
- SQL injection prevention
- XSS protection
- Rate limiting
- HTTPS and SSL

Projects built:

- Complete auth system
- Social login integration
- Role-based access
- Secure API creation

Practice: Implement auth in 5 projects

Phase 3 Backend Project

Topics:

- Full backend architecture
- API design principles
- Database optimization
- Security implementation
- Testing backend services
- Documentation
- Deployment preparation

Projects built:

- CAPSTONE: Complete Backend System
- Include: REST API, Database, Auth, File Upload, Real-time features

- Options: Social platform backend, E-commerce API, Learning management system
- Assessment: Backend Developer Certification

PHASE 3: BACKEND DEVELOPMENT & DATABASES (MONTHS 7-9, WEEKS 27-39)

Master server-side development, databases, APIs, and build powerful backend systems that power modern applications.

Months 7-8: Server-Side Development (Week 27-34)

Introduction to Backend Development

Topics:

- What happens behind the scenes of websites
- Client-server architecture explained simply
- Introduction to Node.js: JavaScript everywhere
- Setting up your backend development environment
- Your first Node.js program
- Understanding modules and packages
- NPM: the world's largest software registry
- File system operations with Node.js
- Working with JSON data
- Environment variables and configuration
- Debugging Node.js applications
- Backend best practices for beginners

Projects built:

- Personal note-taking system
- File organizer application
- JSON data processor
- Command-line diary app

Practice: Build 8 Node.js utilities

Building APIs with Express.js

Topics:

- What are APIs and why they matter
- Express.js: the Node.js framework
- Creating your first Express server
- Routing: handling different URLs
- HTTP methods: GET, POST, PUT, DELETE
- Middleware: the Express superpower
- Handling JSON data and forms
- Error handling done right
- Organizing code with routers
- Static file serving
- Template engines for server-side rendering
- API testing with Postman

Projects built:

- Recipe sharing API
- Personal blog backend
- Todo list API
- Quiz application server

Practice: Create 6 different Express APIs

Databases - Storing Your Data

Topics:

- Why we need databases
- Types of databases explained
- MongoDB: the document database
- Setting up MongoDB locally and cloud
- CRUD operations in MongoDB
- Mongoose: making MongoDB easier
- Designing database schemas
- Relationships between data
- Data validation and sanitization
- Database security basics
- Backup and recovery strategies
- Performance optimization tips

Projects built:

- User management database
- Product inventory system
- Social media data model
- Event planning database

Practice: Design 8 different database schemas

Authentication & User Management

Topics:

- Why authentication is crucial
- Passwords: hashing vs encryption
- Building a registration system
- Login functionality with sessions
- JWT tokens: modern authentication
- Remember me functionality
- Password reset via email

- Social login with Google/Facebook
- Role-based permissions
- Protecting API routes
- Security best practices
- GDPR and user data privacy

Projects built:

- Complete user auth system
- Multi-role application
- Social login integration
- Secure member-only site

Practice: Implement auth in all previous projects

Real-time Features & WebSockets

Topics:

- Real-time web applications
- WebSockets vs HTTP
- Socket.io introduction
- Building a chat system
- Broadcasting messages
- Private messaging implementation
- Online user tracking
- Real-time notifications
- Live collaboration features
- Scaling WebSocket servers
- Real-time data synchronization
- Building interactive dashboards

Projects built:

- Chat application
- Live notification system
- Collaborative whiteboard
- Real-time polling app

Practice: Add real-time features to 3 projects

Month 9: Advanced Backend & Integration (Week 35-39)

Advanced API Development

Topics:

- RESTful API design principles
- API versioning strategies
- Pagination and filtering
- File upload handling
- Image processing on server
- Email integration with Nodemailer
- SMS and push notifications
- Payment gateway integration
- Third-party API integration
- Webhooks implementation
- API rate limiting
- API documentation with Swagger

Projects built:

- E-commerce API with payments
- File sharing service
- Email newsletter system
- Booking system with notifications

Practice: Build 5 production-ready APIs

Cloud Services & Deployment

Topics:

- Introduction to cloud computing
- Deploying to Render
- Using Netlify for frontend
- MongoDB Atlas setup
- Environment management
- Domain names and DNS
- SSL certificates and HTTPS
- Basic AWS services
- Serverless functions introduction
- CI/CD basics with GitHub Actions
- Monitoring your applications
- Error tracking and logging

Projects built:

- Deploy all projects to cloud
- Set up custom domains
- Implement CI/CD pipeline
- Create monitoring dashboard

Practice: Deploy and manage 5 live applications

SQL Databases & Advanced Queries

Topics:

- Introduction to SQL databases
- PostgreSQL setup and basics
- Tables, rows, and columns
- SQL queries: SELECT, INSERT, UPDATE, DELETE

- Joining tables for relationships
- Aggregate functions and grouping
- Database design principles
- Normalization concepts
- Indexes for performance
- Transactions and ACID
- SQL vs NoSQL: choosing wisely
- Database migration strategies

Projects built:

- School management database
- Financial tracking system
- Library catalog database
- Healthcare records system

Practice: Master SQL with 30 query challenges

Performance & Optimization

Topics:

- Understanding performance bottlenecks
- Database query optimization
- Caching with Redis
- Load balancing concepts
- Horizontal vs vertical scaling
- Code optimization techniques
- Memory management
- Async operations optimization
- CDN for static assets
- Compression techniques
- Performance monitoring tools
- Load testing your APIs

Projects built:

- Optimize existing applications
- Implement caching layer
- Load testing suite
- Performance dashboard

Practice: Optimize 5 slow applications

Phase 3 Final Project

Topics:

- Full-stack integration
- API design and documentation
- Database architecture
- Security implementation
- Testing strategies
- Deployment pipeline
- Project presentation

Projects built:

- CAPSTONE: Full-Stack Application
- Build complete system with: Frontend (React), Backend (Node.js), Database, Authentication, Real-time features, Cloud deployment
- Options: Social platform, E-commerce site, Educational platform, Health & wellness app

Assessment: Full-Stack Developer Certification

Months 10-12: Professional Development (Week 40-52)

Mobile App Development

Topics:

- Mobile development overview
- React Native introduction
- Setting up React Native environment
- Core components and APIs
- Navigation in mobile apps
- Styling for mobile
- Device features access
- Building for iOS and Android
- Testing on devices
- Publishing to app stores
- Progressive Web Apps alternative
- Mobile-first best practices

Projects built:

- Mobile version of web app
- Native mobile features app
- Cross-platform social app
- Mobile game development

Practice: Build 3 mobile applications

AI & Machine Learning for Web

Topics:

- AI basics for web developers
- Integrating AI APIs
- TensorFlow.js introduction
- Image recognition in browser
- Natural language processing
- Chatbot development

- Recommendation systems
- Sentiment analysis
- Computer vision basics
- Voice recognition integration
- AI ethics and bias
- Future of AI in web development

Projects built:

- AI-powered chatbot
- Image classification app
- Recommendation engine
- Smart search implementation

Practice: Add AI to 5 existing projects

DevOps & Automation

Topics:

- DevOps culture and practices
- Docker containerization basics
- Container orchestration introduction
- CI/CD pipelines with GitHub Actions
- Automated testing in pipelines
- Infrastructure as Code basics
- Monitoring and alerting
- Log management strategies
- Security in DevOps
- GitOps principles
- Kubernetes introduction
- Cloud-native development

Projects built:

- Dockerize applications
- Set up CI/CD pipeline
- Monitoring dashboard
- Automated deployment system

Practice: Automate 5 project deployments

Entrepreneurship & Freelancing

Topics:

- Building products vs services
- Finding your niche
- Client communication skills
- Project estimation and pricing
- Contract essentials
- Building your brand
- Marketing for developers
- Social media presence
- Open source contribution
- Building a following
- Passive income streams
- Starting your tech business

Projects built:

- Personal brand website
- Open source contribution
- SaaS product MVP
- Freelance portfolio

Practice: Launch your first product/service

Advanced Topics & Specialization

Topics:

- Blockchain basics for web developers
- AI agents inside web apps
- Cybersecurity fundamentals
- Game development with JavaScript
- IoT and web integration
- Augmented Reality on web
- Voice interfaces development
- Quantum computing basics
- Edge computing concepts
- 5G and web applications
- Future technology trends
- Continuous learning strategies

Projects built:

- Explore emerging technology
- Build experimental project
- Research and blog post
- Innovation prototype

Assessment: Technology innovation project

PHASE 4: PROFESSIONAL SKILLS & CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52)

Master professional development skills, build your portfolio, prepare for your tech career, and join the global community of women in tech.

Months 10-11: Advanced Projects & Specialization (Week 40-47)

Building Your Signature Project

Topics:

- Identifying problems worth solving
- Project ideation for social impact
- User research and validation
- Creating project roadmaps
- Agile development methodology
- Sprint planning and execution
- Building MVPs (Minimum Viable Products)
- User testing and feedback
- Iterative development process
- Project documentation standards
- Creating compelling demos
- Storytelling for developers

Projects built:

- Define your signature project
- Create project blueprint
- Build MVP version
- User testing sessions

Practice: Complete weekly sprints on signature project

Women in Tech Leadership

Topics:

- History of women in technology
- Current landscape and opportunities
- Building confidence in tech spaces
- Imposter syndrome and overcoming it
- Public speaking for developers
- Technical writing and blogging
- Mentorship: finding and being one
- Building your professional network
- Women in tech communities
- Negotiation skills for women
- Work-life balance in tech
- Leading technical teams

Projects built:

- Write technical blog series
- Give a tech talk
- Mentor another student
- Join women in tech community

Practice: Weekly leadership challenges

Specialization Tracks (Choose One)

Topics:

- Track 1: Creative Tech & Design
 - - Advanced animations and interactions
 - - Creative coding with Processing/p5.js
 - - Generative art and data visualization
- Track 2: Social Impact Technology
 - - Building for accessibility
 - - Non-profit tech solutions
 - - Educational technology
- Track 3: Business & E-commerce
 - - Payment systems integration
 - - Inventory management
 - - Analytics and reporting
- Track 4: Health & Wellness Tech
 - - Health data management
 - - Fitness tracking apps
 - - Mental health support tools

Projects built:

- Specialization project in chosen track
- Industry-specific solution
- Research and implementation
- Domain expert collaboration

Practice: Deep dive into specialization area

Open Source Contribution

Topics:

- Understanding open source culture
- Finding projects to contribute to
- Reading and understanding codebases
- Git workflow for collaboration
- Making your first pull request
- Code review etiquette
- Documentation contributions
- Creating your own open source project
- Building a contributor community
- Licensing and legal considerations
- Maintaining open source projects
- Building reputation through open source

Projects built:

- First open source contribution
- Create open source project
- Documentation improvements
- Community engagement

Practice: Weekly open source contributions

Portfolio & Personal Branding

Topics:

- Creating a standout portfolio
- Case study writing
- Project presentation skills
- Personal branding for developers
- LinkedIn optimization for tech
- GitHub profile enhancement
- Creating a technical blog
- Video content creation
- Social media for developers
- Speaking at meetups/conferences
- Building thought leadership
- Networking strategies for introverts

Projects built:

- Professional portfolio website
- 5 detailed case studies
- Technical blog launch
- Personal brand strategy

Practice: Daily brand building activities

Month 12: Career Launch & Graduation (Week 48-52)

Job Search Preparation

Topics:

- Tech job market overview
- Types of tech roles available
- Creating a winning tech resume
- Cover letters that stand out
- Job search strategies
- Using job boards effectively
- Company research techniques
- Startup vs corporate environments
- Remote work opportunities
- Freelancing as an option
- Contract vs full-time positions
- Visa and relocation considerations

Projects built:

- Polished resume creation
- Cover letter templates
- Job search tracker
- Target company list

Practice: Apply to 5 positions weekly

Interview Mastery

Topics:

- Technical interview preparation
- Data structures and algorithms basics
- Problem-solving strategies
- Live coding best practices
- System design interviews
- Behavioral interview questions
- STAR method for responses
- Take-home project strategies
- Whiteboard coding tips
- Asking the right questions
- Following up after interviews
- Handling rejection positively

Projects built:

- Interview prep portfolio
- Algorithm practice solutions
- Mock interview recordings
- Interview question bank

Practice: Daily coding challenges and mock interviews

Negotiation & Career Planning

Topics:

- Understanding your worth
- Salary research and benchmarking
- Total compensation packages
- Negotiation strategies for women
- Benefits and perks evaluation
- Career growth planning
- Setting professional goals
- Continuous learning mindset
- Building mentor relationships

- Performance review preparation
- Promotion strategies
- Long-term career visioning

Projects built:

- Salary research report
- 5-year career plan
- Professional development roadmap
- Mentor outreach plan

Practice: Negotiation role-play sessions

Final Capstone Presentation

Topics:

- Polishing your capstone project
- Creating presentation materials
- Demo preparation and practice
- Technical documentation
- User documentation
- Project deployment checklist
- Performance optimization
- Security audit
- Accessibility compliance
- Project handover preparation
- Creating project portfolio page
- Recording project demo video

Projects built:

- FINAL CAPSTONE: Complete Full-Stack Application
- Requirements: Production-ready, Fully tested, Documented, Deployed, Presented
- Showcase to industry professionals
- Peer review and feedback

Practice: Daily project refinement

Graduation & Beyond

Topics:

- Celebrating your journey
- Princess Coders alumni network
- Continuing education resources
- Staying current with technology
- Giving back to the community
- Mentoring new students
- Tech conference participation
- Building lasting connections
- Creating your tech legacy
- Future learning paths
- Advanced certifications
- Your role in closing the gender gap

Deliverables:

- Graduation certificate
- Final portfolio review
- Alumni network access
- Lifetime learning resources
- Industry connections
- Recommendation letters

Assessment: Final comprehensive assessment and graduation ceremony

Bonus Month: Lifetime Learning (Ongoing Support)

Advanced Specializations

Topics:

- Machine Learning and AI
- Blockchain development
- Cybersecurity
- Cloud architecture
- Mobile development
- Game development
- Data science
- DevOps engineering
- UX/UI design
- Product management
- Technical writing
- Developer advocacy

Projects built:

- Choose specialization path
- Advanced certification prep
- Industry project
- Research publication

Practice: Continuous specialization learning

Entrepreneurship Path

Topics:

- Startup ideation
- Business model canvas
- MVP development
- Funding basics

- Pitching to investors
- Building a team
- Product-market fit
- Growth hacking
- Legal considerations
- Scaling strategies
- Exit strategies
- Women entrepreneurs in tech

Projects built:

- Business plan creation
- MVP launch
- Pitch deck preparation
- Startup networking

Practice: Build your startup

Community Leadership

Topics:

- Starting a coding club
- Teaching others to code
- Organizing tech events
- Building inclusive communities
- Advocacy for women in tech
- Creating educational content
- Public speaking mastery
- Media and PR
- Policy and tech
- Social impact projects
- Global tech initiatives
- Leading change

Projects built:

- Community initiative launch
- Workshop series creation
- Advocacy campaign
- Impact measurement

Practice: Lead community projects

Lifetime Achievement

Topics:

- Continuous learning strategies
- Staying relevant in tech
- Building your legacy
- Mentoring at scale
- Writing technical books
- Conference speaking
- Industry recognition
- Awards and achievements
- Global impact
- Tech innovation
- Future technologies
- Your continuing journey

Projects built:

- Long-term impact project
- Knowledge sharing platform
- Innovation initiative
- Legacy documentation

Practice: Lifetime contribution to tech

Princess Coders Forever (Alumni Program)

Alumni Network & Support

Topics:

- Lifetime alumni access
- Exclusive job board
- Continued mentorship
- Advanced workshops
- Industry partnerships
- Referral programs
- Success story features
- Annual reunions
- Global conferences
- Research opportunities
- Teaching opportunities
- Leadership positions

Projects built:

- Alumni project collaborations
- Mentorship programs
- Industry partnerships
- Community initiatives

Giving Back Program

Topics:

- Scholarship programs
- Teaching assistantships

- Curriculum development
- Success coaching
- Interview preparation help
- Portfolio reviews
- Code reviews
- Career guidance
- Startup mentoring
- Research collaboration
- Content creation
- Community building

Deliverables:

- Mentor new students
- Create learning resources
- Lead workshops
- Build community

Your Tech Journey Continues

Topics:

- Setting new goals
- Advanced certifications
- Industry leadership
- Global opportunities
- Tech entrepreneurship
- Research and innovation
- Policy influence
- Educational leadership
- Social impact
- Tech evangelism
- Future readiness
- Lifelong learning

Deliverables:

- Personal development plan
- Career roadmap
- Impact strategy
- Legacy vision

Assessment: Princess Coder for Life - Your journey has just begun!

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1-3: Portfolio Website, Interactive Story, Quiz App, Art Generator, Budget Tracker
- Month 4-6: Social Media Dashboard, E-commerce Frontend, Blog Platform, Learning App
- Month 7-9: REST API, Chat Application, Authentication System, Real-time Dashboard
- Month 10-12: Signature Project, Open Source Contribution, Mobile App, AI Integration
- Capstone: Full Production Application with Social Impact

Total projects: 40+ projects from simple to complex, with focus on creativity and impact

SKILLS THE STUDENT WILL HAVE AT THE END

- Frontend: HTML5, CSS3, JavaScript, React.js, Next.js, TypeScript
- Design: UI/UX, Responsive Design, Accessibility, Animation, Creative Coding
- Backend: Node.js, Express.js, REST APIs, GraphQL, Databases
- Full-Stack: Authentication, Real-time Features, Cloud Deployment
- Professional: Git, Testing, DevOps Basics, Agile, Documentation
- Soft Skills: Communication, Leadership, Mentorship, Public Speaking
- Specialization: Choose from Creative Tech, Social Impact, Business, Health Tech

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 6-8 hours
 Projects: 2-3 hours
 Practice problems: 1-2 hours
 Total per week: 10-15 hours

SUPPORT INCLUDED

Live sessions: Weekly all-girls doubt clearing sessions
 Mentorship: Female mentors from the industry
 Community: Exclusive Princess Coders community platform
 Code review: Personalized feedback on all projects
 Career support: Women-focused career guidance and opportunities
 Lifetime access: Forever access to content and community

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 total)
 Final certificate: Princess Coders Certified Developer
 LinkedIn badge: Verified LinkedIn certification
 Industry recognized: A course-completion certificate you can add to your resume and LinkedIn.

Portfolio projects: 40+ projects for your portfolio

WHERE THIS CAN LEAD

- Frontend Developer (Creating beautiful user interfaces)
- Backend Developer (Building powerful server systems)
- Full-Stack Developer (Complete web applications)
- Mobile App Developer (iOS/Android applications)
- UI/UX Designer (User experience specialist)
- DevOps Engineer (Deployment and operations)
- Technical Writer (Documentation specialist)
- Developer Advocate (Tech community leader)
- Startup Founder (Tech entrepreneur)
- Open Source Contributor (Global impact)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the Princess Coders program teach?

A: It is a full 1-year coding journey for girls: web foundations with HTML, CSS, and JavaScript, modern frontend development with React, backend and database skills, and a final phase of professional projects and portfolio building. Girls graduate having built real websites and applications from start to finish.

Q: Why a girls-only coding program?

A: In many mixed classes, girls quietly stop raising their hands; a class of girls, taught with female mentors and role models, simply removes that pressure. The goal is not to make coding different for girls; it is to give them the same room to be curious, make mistakes, and lead. What we care about is confidence backed by real skill.

Q: What ages can join, and how are levels handled?

A: The program runs age-appropriate tracks: Princess Junior for girls 8 to 12, a teen track for 13 to 17, and an adult track for women 18 and up. Each track follows the same curriculum spine with projects and pacing matched to the age group.

Q: Does my daughter need experience or paid software?

A: No experience is needed; the course starts from the first line of HTML. All tools used are free, a browser and a free code editor cover the whole first phase, and a regular laptop is sufficient (tablets work for the youngest learners in the early months).

Q: What comes after Princess Coders?

A: Many students also branch into our Python, AI, or full stack courses to build a specialization.

Q: Is there a free demo before we decide?

A: Yes, every student begins with a free demo class booked at learn.modernagecoders.com/contact, with no commitment required. Parents are welcome to sit in, and you can message us on WhatsApp at +91 91233 66161 to help choose the right track first.

Q: How are the classes run?

A: All classes are live 1-hour online sessions with a real teacher; nothing is pre-recorded. Groups stay small and conversational, which matters here: the class is designed to be a place where every girl talks, asks, and builds.

Q: Do students earn a certificate?

A: Yes, graduates receive the Princess Coders completion certificate.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/princess-coders-complete-coding-masterclass>

COURSE 98 OF 118: PROBLEM-SOLVING & COMPUTATIONAL THINKING COURSE FOR KIDS

Course link: <https://learn.modernagecoders.com/courses/problem-solving-and-computational-thinking-for-kids>

Subject area: Critical Thinking & Problem Solving

Designed for: Kids

Who can join: Complete Beginner to Advanced Problem Solver

Course length: 6 months (26 weeks)

Time needed: 5-7 hours/week recommended

Certificate: Certified Young Problem Solver

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Six-month course for kids ages 7-16: build critical thinking, logical reasoning and problem-solving through coding, puzzles, robotics and Scratch.

In one line: Build a Problem-Solving Mindset That Lasts a Lifetime!

ABOUT THE PROGRAMME

This transformative 6-month program doesn't just teach kids to code, it teaches them HOW TO THINK! Through engaging puzzles, coding challenges, real-world problems, and creative projects, students develop critical thinking skills that apply to every area of life. Using a blend of coding platforms, logic puzzles, strategy games, and hands-on challenges, kids learn to break down complex problems, recognize patterns, develop algorithms, and create innovative solutions. Perfect for building confidence, improving academic performance, and preparing for future STEM careers.

WHAT MAKES THIS COURSE DIFFERENT

- Focus on thinking skills, not just coding
- Mix of digital and unplugged activities
- Real-world problem-solving applications
- Age-appropriate progression from 7-16
- Gamified learning with rewards and badges
- Parent involvement activities included
- Cross-curricular connections to math, science, and life
- Build resilience and growth mindset

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-2): Logic, Patterns, Basic Algorithms
Phase 2: Application (Months 3-4): Coding Problems, Debugging, Strategy
Phase 3: Advanced (Months 5-6): Complex Problems, Projects, Innovation
Phase 4: Mastery: Real-world challenges and creative solutions

WHAT A STUDENT NEEDS BEFORE STARTING

Education: None - suitable for ages 7-16
Coding experience: No prior experience needed
Equipment: Computer/laptop with internet
Time commitment: 5-7 hours per week
English: Basic English comprehension
Motivation: Curiosity and willingness to learn

WHO THIS COURSE IS FOR

Students: Kids and teens wanting to develop critical thinking
Working professionals: Parents learning with children
Entrepreneurs: Young innovators and future leaders
Freelancers: Future consultants and problem solvers
Kids: Any child wanting to think better
Anyone: Anyone wanting to improve problem-solving skills

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: THINKING FOUNDATIONS (MONTHS 1-2, WEEKS 1-8) -----

Build core thinking skills through puzzles, patterns, and logical reasoning. Start with unplugged activities and gradually introduce coding concepts.

Months 1-2: Logic and Pattern Recognition (Week 1-8)

Introduction to Problem Solving

Topics:

- What is problem solving?
- The problem-solving process
- Understanding the problem (The 5 W's and How)
- Breaking big problems into small pieces
- Pattern recognition basics
- Sequence and order
- Following instructions precisely
- Logic puzzles introduction
- Sudoku for beginners (4x4)
- Tangram puzzles
- Tower of Hanoi introduction
- Growth mindset: mistakes help us learn

Projects built:

- Create a puzzle book
- Design a treasure hunt
- Solve 20 logic puzzles
- Build pattern art

Practice: Daily: 15-minute puzzle time, pattern journaling

Algorithmic Thinking Unplugged

Topics:

- What is an algorithm?
- Recipes as algorithms
- Creating step-by-step instructions
- Flowcharts with shapes and arrows
- Decision trees
- If-then thinking
- Loops in daily life
- Debugging: finding mistakes
- Optimization: finding better ways
- Parallel vs sequential thinking
- Human robot game
- Algorithm art and music

Projects built:

- Write algorithms for daily tasks
- Create a board game with rules
- Design a flowchart adventure
- Program a 'human robot'

Practice: Create 10 real-life algorithms

Introduction to Code.org

Topics:

- Code.org Course A basics
- Drag and drop programming
- Sequence of commands
- Loops to repeat actions
- Events and triggers
- Debugging practice
- Angry Birds puzzles
- Maze solving strategies
- Artist drawings with code
- Pattern creation with loops
- Harvester game logic
- Certificate achievements

Projects built:

- Complete Code.org Course A
- Create digital art with code
- Solve 50 coding puzzles
- Design custom maze challenges

Practice: Daily Code.org challenges

Mathematical Problem Solving

Topics:

- Number patterns and sequences
- Fibonacci in nature
- Magic squares
- Mathematical puzzles
- Estimation strategies
- Mental math tricks
- Geometric patterns
- Symmetry and reflection
- Fraction puzzles
- Time and scheduling problems
- Money and budgeting challenges
- Data collection and graphs

Projects built:

- Math puzzle collection
- Create a math game
- Budget planning project
- Data investigation project

Practice: Solve 5 math problems daily

Months 3-4: Applied Problem Solving (Week 9-17)

Scratch Problem Solving

Topics:

- Scratch as a problem-solving tool
- Decomposition: breaking down projects
- Variables for storing information
- Conditional logic (if-then-else)
- Loops for efficiency
- Creating functions (My Blocks)
- Debugging with Scratch
- Testing and iteration
- Coordinate system problems

- Random number challenges
- List manipulation
- Algorithm visualization

Projects built:

- Calculator with Scratch
- Sorting algorithm visualization
- Maze solver program
- Pattern generator

Practice: Daily Scratch challenges

Game Strategy & Logic

Topics:

- Strategy game thinking
- Chess basics and tactics
- Checkers strategies
- Tic-tac-toe perfect play
- Connect Four winning strategies
- Card game probability
- Board game design
- Winning vs learning mindset
- Analyzing opponent moves
- Planning ahead
- Risk assessment
- Decision making under pressure

Projects built:

- Design a strategy game
- Chess puzzle solver
- Create game AI
- Tournament organization

Practice: Play and analyze strategy games

Real-World Problem Solving

Topics:

- Identifying problems around us
- Research and information gathering
- Brainstorming techniques
- Mind mapping
- SCAMPER method for kids
- Design thinking process
- Prototyping solutions
- Testing and feedback
- Iteration and improvement
- Presentation skills
- Team problem solving
- Environmental challenges

Projects built:

- School problem solution
- Community helper app
- Recycling game design
- Invention prototype

Practice: Identify and solve one problem weekly

Coding Challenges Platform

Topics:

- Introduction to coding challenges
- Blockly Games complete
- Lightbot puzzle game
- CodeCombat basics
- Algorithm challenges
- Time complexity basics
- Space optimization
- Code efficiency
- Common algorithms
- Sorting visualization
- Search algorithms
- Competition preparation

Projects built:

- Complete 100 coding challenges
- Algorithm portfolio
- Challenge leaderboard
- Peer challenge creation

Practice: 3 coding challenges daily

Mid-Program Assessment

Topics:

- Problem-solving portfolio review
- Skills assessment
- Logic test completion
- Project presentations
- Peer review process
- Reflection and goal setting

Projects built:

- Comprehensive problem-solving project

- Skills demonstration video
- Teaching someone else
- Progress documentation

Assessment: Mid-program certification challenge

Months 5-6: Advanced Problem Solving (Week 18-26)

Advanced Algorithms

Topics:

- Recursion concepts for kids
- Divide and conquer strategy
- Greedy algorithms explained
- Dynamic programming basics
- Graph problems introduction
- Shortest path problems
- Optimization challenges
- Scheduling algorithms
- Data structure basics
- Stack and queue concepts
- Tree structures in life
- Algorithm complexity

Projects built:

- Pathfinding visualizer
- Scheduling optimizer
- Recursive art generator
- Data structure game

Practice: Implement 5 algorithms weekly

Robotics Problem Solving

Topics:

- Robot logic basics
- Sensor data processing
- Decision making for robots
- Path planning
- Obstacle avoidance
- Line following algorithms
- Maze solving robots
- Virtual robotics with VEXcode
- Robot simulation
- Real-world robot applications
- AI in robotics basics
- Future of robotics

Projects built:

- Virtual robot programmer
- Maze solving algorithm
- Robot rescue simulation
- Automated system design

Practice: Daily robot challenges

Data and Logic Problems

Topics:

- Data analysis for kids
- Survey design and analysis
- Spreadsheet problem solving
- Database thinking
- Information organization
- Logical operators (AND, OR, NOT)
- Truth tables
- Venn diagrams
- Set theory basics
- Probability problems
- Statistics fundamentals
- Data visualization

Projects built:

- School data analysis
- Survey project
- Probability game
- Infographic creation

Practice: Analyze data sets daily

Creative Problem Solving

Topics:

- Lateral thinking puzzles
- TRIZ for kids
- Biomimicry solutions
- Reverse engineering
- Constraint-based creativity
- Random input technique
- Six thinking hats simplified
- Morphological analysis
- SCAMPER advanced
- Innovation frameworks
- Failure analysis

- Future problem solving
- Projects built:
- Innovation challenge
 - Redesign everyday object
 - Future city design
 - Creative solution showcase
- Practice: Daily creative challenges
- Final Capstone Project

Topics:

- Project selection
- Problem identification
- Research and planning
- Solution development
- Testing and refinement
- Documentation
- Presentation preparation
- Peer review

Projects built:

- CAPSTONE: Solve a real community problem
- Create solution presentation
- Build working prototype
- Document entire process

Assessment: Final problem-solving certification

PHASE 2: SPECIALIZED PROBLEM-SOLVING TRACKS

Choose specialized areas: Science, Technology, Engineering, Arts, or Mathematics

Science Problem Solving Track (Science Specialization)

Scientific Method & Experimentation

Topics:

- Scientific method steps
- Hypothesis formation
- Variable identification
- Experimental design
- Data collection methods
- Observation skills
- Recording results
- Drawing conclusions
- Error analysis
- Reproducibility
- Science fair projects
- Lab safety basics

Projects built:

- Design 5 experiments
- Science investigation project
- Lab notebook creation
- Experiment video series

Practice: Weekly experiment and analysis

Environmental Problem Solving

Topics:

- Climate change challenges
- Renewable energy solutions
- Water conservation
- Waste reduction strategies
- Biodiversity protection
- Sustainable living
- Carbon footprint calculation
- Eco-friendly innovations
- Community gardens
- Recycling systems
- Green technology
- Environmental monitoring

Projects built:

- Eco-solution prototype
- School green initiative
- Environmental app design
- Sustainability campaign

Practice: Daily eco-challenges

Space and Astronomy Challenges

Topics:

- Space exploration problems
- Rocket science basics
- Orbital mechanics simplified
- Mars colony challenges
- Satellite applications
- Space debris solutions
- Astronomy observations

- Telescope usage
- Star pattern recognition
- Space mission planning
- Astrobiology questions
- Future space technology

Projects built:

- Mars habitat design
- Rocket simulator
- Star tracking app
- Space mission plan

Practice: Space problem daily

Biology and Health Solutions

Topics:

- Human body systems
- Disease prevention
- Nutrition optimization
- Exercise science
- Mental health awareness
- Genetics basics
- Ecosystem balance
- Food chain problems
- Adaptation strategies
- Medical technology
- Health app design
- Biotechnology basics

Projects built:

- Health tracker app
- Nutrition planner
- Exercise game design
- Medical helper tool

Practice: Health challenges daily

Chemistry and Materials

Topics:

- Chemical reactions basics
- Kitchen chemistry
- Material properties
- Recycling chemistry
- Green chemistry
- Crystal growing
- pH and indicators
- Polymer science
- Nanotechnology intro
- Material innovation
- Safety procedures
- Future materials

Projects built:

- Chemistry experiment kit
- Material testing lab
- Eco-friendly product
- Chemistry game design

Practice: Chemistry puzzles daily

Technology Problem Solving Track (Tech Innovation)

App Development Problem Solving

Topics:

- User problem identification
- App solution design
- User interface logic
- User experience flow
- Feature prioritization
- MIT App Inventor projects
- Thunkable solutions
- Database design basics
- API integration logic
- Testing strategies
- User feedback integration
- App store optimization

Projects built:

- Problem-solving app
- Educational tool app
- Community helper app
- Game-based solution

Practice: Daily app challenges

AI and Machine Learning

Topics:

- AI concepts for kids
- Machine learning basics
- Training data concepts
- Pattern recognition AI

- Teachable Machine projects
- Image classification
- Voice recognition
- Natural language basics
- Ethical AI considerations
- AI in daily life
- Future of AI
- AI problem solving

Projects built:

- AI classifier project
- Chatbot design
- Smart assistant
- AI game creation

Practice: AI experiments daily

Internet of Things (IoT)

Topics:

- IoT concepts
- Smart home problems
- Sensor applications
- Data collection systems
- Automation logic
- micro:bit projects
- Arduino basics
- Circuit design
- Cloud connectivity
- Security considerations
- Energy efficiency
- Future IoT

Projects built:

- Smart room design
- Plant watering system
- Weather station
- Security system

Practice: IoT challenges daily

Cybersecurity Basics

Topics:

- Internet safety
- Password security
- Encryption basics
- Cipher puzzles
- Digital footprints
- Privacy protection
- Phishing awareness
- Secure communication
- Ethical hacking intro
- Bug bounty basics
- Security testing
- Future security

Projects built:

- Encryption tool
- Security audit game
- Password manager design
- Security awareness campaign

Practice: Security puzzles daily

Web Development Logic

Topics:

- Website planning
- Information architecture
- Navigation logic
- Responsive design thinking
- HTML structure basics
- CSS logic patterns
- JavaScript problem solving
- Form validation logic
- API integration
- Database queries
- Performance optimization
- SEO strategies

Projects built:

- Website wireframe
- Interactive web game
- School website redesign
- Web app prototype

Practice: Web challenges daily

Engineering Problem Solving Track (Engineering Design)

Mechanical Engineering

Topics:

- Simple machines

- Lever principles
- Pulley systems
- Gear ratios
- Force and motion
- Energy transfer
- Structural design
- Bridge building
- Vehicle design
- Rube Goldberg machines
- CAD basics for kids
- 3D printing intro

Projects built:

- Bridge building contest
- Catapult design
- Marble run creation
- Vehicle prototype

Practice: Engineering challenges daily

Electrical Engineering

Topics:

- Circuit basics
- Series vs parallel
- Conductors and insulators
- Battery power
- LED projects
- Switch logic
- Motor control
- Solar power basics
- Circuit simulation
- Breadboard projects
- Safety rules
- Future electronics

Projects built:

- LED art project
- Electronic game
- Solar charger
- Smart light system

Practice: Circuit puzzles daily

Civil Engineering

Topics:

- City planning basics
- Transportation systems
- Water management
- Building design
- Earthquake resistance
- Environmental impact
- Sustainable construction
- Traffic flow optimization
- Public space design
- Infrastructure planning
- Smart cities
- Future urban design

Projects built:

- City model design
- Traffic solution
- Park redesign
- Disaster-proof building

Practice: Urban challenges daily

Aerospace Engineering

Topics:

- Flight principles
- Aerodynamics basics
- Paper airplane science
- Rocket design
- Drone technology
- Navigation systems
- Weather effects
- Space station design
- Satellite applications
- Future flight
- Mission planning
- Safety systems

Projects built:

- Glider competition
- Rocket launch
- Drone course design
- Space mission plan

Practice: Flight challenges daily

Final Integration Project

Topics:

- Combining all skills
- Complex problem selection
- Interdisciplinary approach
- Team collaboration
- Project management
- Documentation excellence
- Presentation mastery
- Future planning

Projects built:

- FINAL PROJECT: Multi-disciplinary solution
- Complete documentation
- Video presentation
- Peer teaching session

Assessment: Master Problem Solver Certification

 ADVANCED TRACKS: COMPETITIVE & SPECIALIZED

Competition preparation and specialized problem-solving domains

Competition Preparation (Competitive Excellence)

Coding Competitions

Topics:

- Competition formats
- Time management strategies
- Common problem types
- Speed coding techniques
- Testing strategies
- Debugging under pressure
- Code optimization
- Competition platforms
- Team competitions
- Individual contests
- Practice routines
- Mental preparation

Projects built:

- Competition training plan
- Mock competition
- Problem archive
- Strategy guide

Practice: Daily competition problems

Math Olympiad Preparation

Topics:

- Number theory basics
- Combinatorics introduction
- Geometry problems
- Algebra challenges
- Proof techniques
- Problem-solving strategies
- Past paper analysis
- Speed techniques
- Checking methods
- Common tricks
- Mental math advanced
- Competition mindset

Projects built:

- Problem solution book
- Study group formation
- Practice test series
- Strategy documentation

Practice: Olympiad problems daily

Science Olympiad

Topics:

- Science bowl format
- Quick recall techniques
- Experimental design competition
- Building events
- Testing events
- Knowledge events
- Team coordination
- Resource management
- Time allocation
- Equipment mastery
- Rules interpretation
- Practice strategies

Projects built:

- Event preparation kit
- Team training schedule
- Resource library

- Competition notebook

Practice: Event practice daily

Robotics Competitions

Topics:

- FIRST LEGO League basics
- VEX Robotics introduction
- Competition rules
- Robot design process
- Programming strategies
- Alliance strategies
- Pit preparation
- Judging preparation
- Core values
- Innovation project
- Team building
- Gracious professionalism

Projects built:

- Robot design portfolio
- Competition robot
- Innovation project
- Team presentation

Practice: Robot challenges daily

Hackathon Preparation

Topics:

- Hackathon format
- Idea generation speed
- Rapid prototyping
- Team formation
- Technology stack selection
- Time management
- Pitch preparation
- Demo strategies
- Networking skills
- Prize strategies
- Follow-up plans
- Portfolio building

Projects built:

- Hackathon toolkit
- Practice hackathon
- Pitch deck template
- Demo preparation

Practice: Mini-hackathons weekly

Arts & Creative Problem Solving (Creative Innovation)

Design Thinking Mastery

Topics:

- Empathy mapping
- User journey mapping
- Persona creation
- Problem framing
- Ideation techniques
- Crazy 8s method
- Storyboarding
- Prototyping methods
- Testing protocols
- Iteration cycles
- Design sprints
- Innovation frameworks

Projects built:

- Design thinking project
- User research study
- Prototype series
- Innovation portfolio

Practice: Design challenges daily

Music and Sound Problem Solving

Topics:

- Music theory basics
- Pattern in music
- Rhythm mathematics
- Melody creation
- Harmony basics
- Digital music tools
- Sound wave physics
- Audio editing
- Music coding with Sonic Pi
- Algorithmic composition
- Sound design
- Music therapy applications

Projects built:

- Musical algorithm
- Sound effects library
- Music app design
- Composition project

Practice: Musical puzzles daily

Visual Arts and Graphics

Topics:

- Color theory
- Composition rules
- Digital art basics
- Pixel art creation
- Vector graphics
- 3D modeling intro
- Animation principles
- Generative art
- Processing sketches
- p5.js creativity
- AR art projects
- AI art, credit and authorship

Projects built:

- Digital art portfolio
- Generative art piece
- Animation project
- Interactive art installation

Practice: Art challenges daily

Storytelling and Narrative

Topics:

- Story structure
- Character development
- Plot design
- Interactive narratives
- Choose your adventure
- Twine storytelling
- Video storytelling
- Podcast creation
- Comic creation
- Game narratives
- Transmedia storytelling
- Digital publishing

Projects built:

- Interactive story
- Podcast series
- Comic book
- Story game

Practice: Story challenges daily

Performance and Drama

Topics:

- Improvisation skills
- Public speaking
- Presentation design
- Body language
- Voice modulation
- Stage presence
- Video presentation
- Live streaming
- Drama games
- Role-playing solutions
- Theater tech
- Digital performance

Projects built:

- TED talk preparation
- Drama production
- Video series
- Live show

Practice: Performance daily

Mathematics Deep Dive (Mathematical Mastery)

Advanced Number Theory

Topics:

- Prime numbers
- Divisibility rules
- Modular arithmetic
- Number patterns
- Sequences and series
- Golden ratio
- Fractals basics
- Infinity concepts
- Number systems
- Cryptography math

- Game theory basics
- Probability advanced

Projects built:

- Number theory game
- Encryption system
- Fractal generator
- Probability simulator

Practice: Number puzzles daily

Geometry and Spatial Reasoning

Topics:

- Euclidean geometry
- Coordinate geometry
- Transformations
- 3D geometry basics
- Topology introduction
- Tessellations
- Origami mathematics
- Architecture geometry
- Navigation math
- Computer graphics math
- VR/AR mathematics
- Sacred geometry

Projects built:

- Geometry art project
- 3D model creation
- Architecture design
- Navigation app

Practice: Geometry challenges daily

Statistics and Data Science

Topics:

- Data collection methods
- Statistical measures
- Data visualization
- Correlation vs causation
- Sampling techniques
- Hypothesis testing basics
- Regression basics
- Time series basics
- Big data concepts
- Machine learning math
- Predictive modeling
- Data ethics

Projects built:

- Data science project
- Statistical study
- Prediction model
- Visualization dashboard

Practice: Data analysis daily

Logic and Discrete Math

Topics:

- Boolean algebra
- Logic gates
- Truth tables advanced
- Set theory advanced
- Graph theory basics
- Combinatorics advanced
- Counting principles
- Pigeonhole principle
- Induction basics
- Recursion mathematics
- Algorithm mathematics
- Computational complexity

Projects built:

- Logic puzzle creator
- Graph algorithm visualizer
- Combinatorics game
- Proof portfolio

Practice: Logic problems daily

Graduation & Future Path

Topics:

- Skills assessment
- Portfolio completion
- Certification preparation
- Future learning paths
- Career exploration
- Mentorship connections
- Community building
- Giving back

Projects built:

- FINAL SHOWCASE: Complete problem-solving portfolio
- Skills demonstration video
- Teaching workshop
- Future plan presentation

Assessment: Master Problem Solver Certification

 ENRICHMENT: REAL-WORLD APPLICATIONS

Apply problem-solving skills to real-world challenges and career exploration

Social Problem Solving (Community Impact)

Community Service Projects

Topics:

- Community needs assessment
- Volunteer organization
- Fundraising strategies
- Awareness campaigns
- Social media for good
- Event planning
- Partnership building
- Grant writing basics
- Impact measurement
- Sustainability planning
- Scaling solutions
- Legacy planning

Projects built:

- Community service initiative
- Fundraising campaign
- Awareness program
- Impact report

Practice: Weekly community service

Global Challenges

Topics:

- UN Sustainable Development Goals
- Climate action
- Poverty solutions
- Education access
- Clean water projects
- Hunger solutions
- Health initiatives
- Gender equality
- Reduced inequalities
- Peace and justice
- Global partnerships
- Youth activism

Projects built:

- SDG project
- Global solution design
- Youth summit presentation
- Action plan creation

Practice: Global challenge daily

Entrepreneurship for Kids

Topics:

- Business idea generation
- Market research basics
- Product development
- Pricing strategies
- Marketing basics
- Sales techniques
- Customer service
- Financial planning
- Team building
- Pitch preparation
- Business ethics
- Social entrepreneurship

Projects built:

- Business plan creation
- Product prototype
- Marketing campaign
- Pitch presentation

Practice: Business challenges daily

Innovation Lab

Topics:

- Innovation mindset
- Trend spotting
- Future thinking
- Disruptive innovation
- Blue ocean strategy

- Patent basics
- Intellectual property
- R&D processes
- Innovation metrics
- Failure celebration
- Pivot strategies
- Innovation culture

Projects built:

- Innovation project
- Patent application draft
- Future product design
- Innovation showcase

Practice: Innovation exercises daily

Career Exploration

Topics:

- STEM careers overview
- Career research methods
- Skill mapping
- Education pathways
- Internship opportunities
- Mentorship programs
- Professional networking
- Resume basics
- Interview skills
- Personal branding
- Goal setting
- Life planning

Projects built:

- Career research project
- Skill portfolio
- Mock interview
- 5-year plan

Practice: Career exploration daily

Teaching & Leadership (Knowledge Sharing)

Peer Teaching

Topics:

- Teaching techniques
- Lesson planning
- Material creation
- Explanation strategies
- Visual aids design
- Interactive teaching
- Assessment creation
- Feedback giving
- Patience development
- Adaptation skills
- Online teaching
- Teaching reflection

Projects built:

- Teach 5 concepts
- Create course module
- Video tutorial series
- Teaching portfolio

Practice: Teach someone daily

Leadership Development

Topics:

- Leadership styles
- Team dynamics
- Communication skills
- Conflict resolution
- Decision making
- Delegation skills
- Motivation techniques
- Goal setting for teams
- Meeting management
- Project leadership
- Ethical leadership
- Servant leadership

Projects built:

- Lead team project
- Leadership journal
- Team building event
- Leadership presentation

Practice: Leadership challenges daily

Content Creation

Topics:

- YouTube channel basics
- Blog writing

- Podcast creation
- Social media content
- Educational videos
- Tutorial creation
- Infographic design
- Course development
- Book writing basics
- Digital publishing
- Monetization basics
- Audience building

Projects built:

- YouTube channel launch
- Blog series
- Podcast episodes
- Course creation

Practice: Create content daily

Community Building

Topics:

- Community formation
- Online communities
- Discord server management
- Event organization
- Workshop facilitation
- Club formation
- Mentorship programs
- Peer support groups
- Knowledge sharing platforms
- Collaboration tools
- Community guidelines
- Sustainable communities

Projects built:

- Start coding club
- Online community
- Workshop series
- Mentorship program

Practice: Community building daily

Research Skills

Topics:

- Research methods
- Literature review
- Data gathering
- Analysis techniques
- Report writing
- Citation basics
- Presentation skills
- Poster creation
- Conference participation
- Peer review process
- Publication basics
- Research ethics

Projects built:

- Research paper
- Conference poster
- Research presentation
- Literature review

Practice: Research skills daily

Advanced Specializations (Expert Tracks)

Quantum Computing Basics

Topics:

- Quantum concepts simplified
- Qubits explanation
- Superposition basics
- Entanglement introduction
- Quantum gates
- Quantum algorithms
- Quantum simulators
- IBM Quantum Experience
- Quantum games
- Future of quantum
- Quantum applications
- Quantum puzzles

Projects built:

- Quantum circuit design
- Quantum game
- Quantum presentation
- Research project

Practice: Quantum puzzles daily

AI Agents for Problem Solving

Topics:

- What an AI agent is, and what it is not
- Breaking a big goal into steps a machine can follow
- Giving an agent tools, and limits
- Checking an agent's work before you trust it
- When the agent is confidently wrong
- The cost of a wrong answer
- Keeping a human in the loop
- Prompt, to plan, to action
- Bias: why the answer leans one way
- Privacy: what you must never tell an AI
- Deciding when NOT to use AI
- Distributed systems: how many machines agree on one answer

Projects built:

- Plan-a-trip agent
- Research helper agent
- Fact-check an AI answer
- Agent failure diary

Practice: Daily agent challenges

Bioinformatics Basics

Topics:

- DNA and genetics basics
- Protein folding problems
- Sequence alignment
- Phylogenetic trees
- Gene expression
- Bioinformatics tools
- Data analysis in biology
- Personalized medicine
- Agricultural applications
- Conservation genetics
- Synthetic biology basics
- Ethical considerations

Projects built:

- DNA sequence analysis
- Biology data project
- Genetics presentation
- Research proposal

Practice: Bio puzzles daily

Neuroscience and Brain

Topics:

- Brain basics
- Neural networks simplified
- Memory and learning
- Problem-solving brain areas
- Neuroplasticity
- Brain training games
- Cognitive enhancement
- Brain-computer interfaces
- Mental health tech
- Sleep and performance
- Meditation effects
- Future brain tech

Projects built:

- Brain training app
- Memory enhancement tool
- Cognitive assessment
- Research presentation

Practice: Brain training daily

Mastery & Graduation (Final Excellence)

Portfolio Excellence

Topics:

- Portfolio organization
- Project documentation
- Video demonstrations
- Code repositories
- Design portfolios
- Writing samples
- Achievement highlights
- Recommendation gathering
- Online presence
- Personal website
- Professional photos
- Brand development

Projects built:

- Complete portfolio website
- 100+ problems solved showcase
- Video portfolio

- Achievement compilation

Final Projects Showcase

Topics:

- Project selection
- Advanced problem solving
- Solution implementation
- Testing and validation
- Documentation excellence
- Presentation preparation
- Demo day readiness
- Judge interaction
- Q&A preparation
- Networking strategies
- Media interaction
- Follow-up planning

Deliverables:

- 3 major projects
- Complete documentation
- Presentation materials
- Demo videos
- Press kit
- Network connections
- Future commitments
- Thank you notes

Graduation & Beyond

Topics:

- Final assessment
- Certification ceremony
- Alumni induction
- Future pathways
- Continued learning
- Mentorship commitment
- Community contribution
- Goal setting session
- Network activation
- Resource access
- Support systems
- Celebration!

Deliverables:

- Problem-Solving Master Certification
- 500+ problems solved
- 50+ projects completed
- Teaching experience
- Leadership record
- Community impact
- Innovation portfolio
- Lifetime alumni access

Assessment: CERTIFIED MASTER PROBLEM SOLVER - Ready to tackle any challenge!

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1-2: 30+ logic puzzles, 20+ algorithms, 10+ pattern projects
- Month 3-4: 40+ coding challenges, 15+ real-world solutions
- Month 5-6: 25+ advanced projects, 10+ competition entries
- Specialization: 30+ domain-specific projects
- Total: 150+ problem-solving projects

Total projects: 150+ projects ranging from puzzles to real-world solutions

SKILLS THE STUDENT WILL HAVE AT THE END

- Computational Thinking: Decomposition, pattern recognition, abstraction, algorithms
- Logic & Reasoning: Deductive, inductive, lateral thinking, critical analysis
- Programming: Scratch, Python basics, block coding, algorithm implementation
- Mathematics: Number theory, geometry, statistics, discrete math
- Science: Scientific method, experimentation, data analysis, hypothesis testing
- Engineering: Design process, prototyping, testing, iteration
- Creativity: Innovation, ideation, design thinking, artistic expression
- Communication: Presentation, documentation, teaching, storytelling
- Collaboration: Teamwork, leadership, conflict resolution, project management
- Research: Information gathering, analysis, synthesis, reporting
- Technology: Apps, AI, IoT, robotics, web development basics

HOW A TYPICAL WEEK IS SPLIT

Theory lessons: 2-3 hours
 Hands on practice: 2-3 hours
 Projects: 1-2 hours
 Challenges puzzles: 1 hour

Total per week: 5-7 hours

SUPPORT INCLUDED

Live sessions: Weekly problem-solving workshops
Mentorship: Expert mentors and peer support
Community: Active community of young problem solvers
Feedback: Regular project reviews and guidance
Resources: Extensive library of problems and solutions
Lifetime access: All content and future updates

CERTIFICATION DETAIL

Phase certificates: Certificates for each problem-solving domain
Final certificate: Certified Young Problem Solver
Linkedin badge: Professional certification display
School recognition: Academic credit possibilities
Portfolio projects: 150+ documented solutions

WHERE THIS CAN LEAD

- Enhanced Academic Performance
- STEM Career Preparation
- Future Software Developer
- Data Scientist
- Research Scientist
- Engineer (any field)
- Entrepreneur/Innovator
- Consultant
- Teacher/Educator
- Any career requiring critical thinking

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is computational thinking and why is it important for children?

A: Computational thinking is a problem-solving approach that breaks complex problems into smaller parts, finds patterns, ignores unnecessary details, and creates step-by-step solutions. It's the thinking skill behind coding, but applies to everything - from organizing a room to planning a project. Studies show it improves academic performance across all subjects.

Q: What age group is this Problem Solving course designed for?

A: This course is designed for children ages 7-16. Younger kids (7-10) focus on visual puzzles, games, and block coding. Older kids (11-16) progress to advanced logic, programming basics, and real-world applications. The curriculum adapts to each child's developmental stage and learning pace.

Q: Will this course help my child in school academics?

A: Absolutely! Problem-solving skills directly transfer to better performance in math, science, and even language arts. Students learn to analyze problems, think logically, and approach challenges systematically. Parents consistently report improved grades, better attention span, and increased confidence in school.

Q: Does this course teach coding or is it just puzzles?

A: Both! We start with logical puzzles and computational thinking, then gradually introduce Scratch visual programming, and progress to Python basics. The focus is on developing the thinking skills that make coding feel natural, rather than just memorizing syntax. By completion, students are comfortable with real programming.

Q: How much screen time does this course require?

A: We recommend 5-7 hours per week, including a mix of online lessons, offline puzzles/activities, and hands-on projects. Many activities involve paper, building blocks, or physical games. We're mindful of healthy screen time while building essential digital skills for your child's future.

Q: What makes this different from school computer classes?

A: School computer classes often focus on basic software usage. This course develops deep thinking skills: logic, pattern recognition, decomposition, and algorithm design. We use games, competitions, and real projects to make learning exciting. Students don't just use technology - they learn to CREATE with it.

To enrol or to book a free demo for this course, message Shivam Sir on
+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/problem-solving-and-computational-thinking-for-kids>

=====

Course link: <https://learn.modernagecoders.com/courses/psle-maths-mastery>
Subject area: Exam Board Mathematics
Designed for: Elementary school
Who can join: Primary 4-6 students on the MOE syllabus, PSLE-bound
Course length: 12 months (52 weeks)
Time needed: 2 live one-hour classes per week + short practice sets
Certificate: Course completion certificate + documented paper-score progression

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online PSLE maths on the MOE syllabus: model method taught as thinking, problem-sum families, no-calculator Paper 1 craft, Singapore evening slots.

In one line: Problem sums are a teachable skill. We teach it, 1-on-1, without the centre queue.

ABOUT THE PROGRAMME

A 12-month live program aimed squarely at PSLE mathematics, taught on the MOE syllabus at Singapore-friendly evening hours. The PSLE separates thinkers from drillers: multi-step problem sums, a no-calculator Paper 1, and AL scoring where consistency is everything. This course teaches the model method as a thinking tool rather than a template, names and trains the heuristics explicitly, builds calculator-free fluency in every class, and closes with paper craft, time discipline, working presentation and the mark-banking habits that protect AL bands.

WHAT MAKES THIS COURSE DIFFERENT

- The model method taught as reasoning, part-whole, comparison, before-after and re-cutting, not as a ritual the school demands
- Problem sums taught by structure family, so new problems look like old friends in disguise
- MOE heuristics named and trained explicitly: work backwards, guess-and-check with discipline, restate, spot patterns
- Paper 1 fluency drills open every class, because AL bands are lost one careless mark at a time
- 1-on-1 attention no centre batch of fifteen can give, at Singapore evening hours
- Understanding-first: a child who can draw what the numbers are doing owns the problem
- One dedicated mentor tracking exactly where your child's reasoning thins
- Honest AL guidance from real timed-paper evidence, never invented promises

THE LEARNING PATH, STEP BY STEP

Phase 1: The method rebuilt properly: number fluency, model drawing as thinking, and the first problem-sum families
Phase 2: The P5-P6 canon: fractions of remainder, ratio, percentage, speed and geometry, family by family
Phase 3: Exam craft: Paper 1 and 2 technique, timed papers, mark-banking discipline and the calm P6 runway to the PSLE

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Primary 4-6 on the MOE syllabus (or equivalent international students using Singapore-style curricula)
Math background: Any, the diagnostic finds the real stage and the method phase rebuilds from there
Equipment: Computer with stable internet, notebook and pencil; calculator for Paper 2 work
Time commitment: 2 live hours plus 1-2 practice hours weekly
English: Course taught in English
Motivation: A child willing to draw and explain, the method does the rest

WHO THIS COURSE IS FOR

Problem sum strugglers: Children whose computation is fine but whose problem sums produce blank stares
Template drawers: Children who draw models as ritual without the bars carrying the reasoning
Paper 1 leakers: Children losing careless marks to calculator-free arithmetic
Centre switchers: Families whose centre batch is not teaching their child
P4 planners: P4 families starting the method work early, when it is cheapest

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE METHOD, REBUILT PROPERLY (MONTHS 1-4)

The gap behind most PSLE struggles is a model method learned as template instead of thinking. This phase rebuilds number fluency and model drawing from meaning, then applies both to the first problem-sum families.

Month 1: Diagnostic + Number Fluency, Bare-Handed (Weeks 1-4)

Diagnostic and the Honest AL Baseline

Topics:

- Model-method depth check across part-whole and comparison bars
- Paper 1 non-calculator fluency screen
- Locating exactly where problem sums break down
- Honest Achievement Level (AL) baseline from timed questions
- Whole-number, fraction and decimal accuracy audit
- Reading-the-question and working-presentation habits

Practice: A short mixed diagnostic of Paper 1 items and two problem sums, marked to expose the real starting stage.

Assessment: Baseline AL estimate plus a written map of which problem-sum families and Paper 1 skills need the most work.

Detail: Full diagnostic: model-method depth, Paper 1 fluency, and exactly where problem sums break, with an honest AL baseline

Whole-Number Operations at Paper 1 Tempo

Topics:

- Addition, subtraction, multiplication and division fluency
- Long multiplication and long division without a calculator
- Order of operations in multi-step calculations
- Estimation and rounding as a checking habit
- Mental strategies for speed on Paper 1 items
- Place value and number sense to catch slips

Practice: A timed whole-number drill set plus five estimation checks on given answers.

Detail: Whole-number operations at Paper 1 tempo; estimation as a checking habit

Fractions as Numbers, From Pictures First

Topics:

- Fractions on a number line as genuine quantities
- Equivalent fractions and simplifying
- Comparing and ordering fractions
- Adding and subtracting unlike fractions
- Multiplying and dividing fractions with meaning
- Bar and area pictures behind every rule

Practice: A fractions set moving from picture-drawn answers to fluent written computation.

Detail: Fractions as numbers: equivalence, comparison and operations from pictures first

Decimals and the Fraction-Decimal-Percentage Bridge

Topics:

- Decimal place value and the four operations
- Converting fractions to decimals and back
- Linking decimals, fractions and percentages
- Rounding decimals and estimating results
- Money and measurement contexts
- Fluency drills across all three forms

Practice: Conversion drills and mixed decimal calculations at Paper 1 pace.

Detail: Decimals and the fraction-decimal-percentage bridge, to fluency

Months 2-3: Model Drawing as Thinking (Weeks 5-12)

Part-Whole Models as Reasoning

Topics:

- Part-whole bar model as a picture of the numbers
- Drawing the unknown before computing
- Total, part and missing-part relationships
- Translating word phrases into bar segments
- The child draws first, the mentor questions
- Checking the model against the story

Practice: Six part-whole problem sums where the bar is drawn before any calculation.

Detail: Part-whole models: the child draws, the mentor questions, never the reverse

Comparison Models Made Visible

Topics:

- Comparison bars for more-than and less-than
- Times-as-many and multiplicative comparison
- Aligning bars to reveal the difference
- Two-quantity and three-quantity comparisons
- Choosing comparison over part-whole
- Defending the chosen model aloud

Practice: A comparison-model set mixing additive and multiplicative relationships.

Detail: Comparison models: more-than, less-than and times-as-many, made visible

Unit Thinking and Re-Cutting Bars

Topics:

- Treating one unit as the engine of a model

- Deciding how many parts to cut a bar into
- Re-cutting a bar when the story changes
- Finding unit value then scaling up
- Flexibility that templates cannot supply
- Common-part reasoning across quantities

Practice: Problems requiring the student to choose the cut, then re-cut when stuck.

Detail: Unit thinking: when to cut a bar into parts, and the re-cutting flexibility templates cannot supply

First Structure Families: Sum-and-Difference

Topics:

- Sum-and-difference problem family
- Equal-stage and equal-value problems
- Recognising the skeleton under the wording
- Matching model shape to family
- Solving then explaining the reasoning aloud
- Extending a solved problem to a variant

Practice: A family-sorted set of sum-and-difference and equal-stage problem sums.

Detail: First structure families: sum-and-difference and equal-stage problems, solved by model and defended aloud

Month 4: Heuristics, Named and Trained (Weeks 13-16)

Work Backwards Heuristic

Topics:

- Recognising end-state problems
- Reversing each operation in turn
- Undoing add, subtract, multiply and divide
- Model support for backward reasoning
- Checking by working forwards again
- When work-backwards beats other methods

Practice: A set of end-state problems solved by unwinding step by step.

Detail: Work backwards: end-state problems unwound step by step

Disciplined Guess-and-Check

Topics:

- Setting up an organised guess-and-check table
- Choosing a sensible first guess
- Using each result to steer the next guess
- Column headings that show the reasoning
- Knowing when guess-and-check is efficient
- Presenting the table for method marks

Practice: Guess-and-check problems recorded in a clean, columned table.

Detail: Guess-and-check with discipline: the organised table, not wild stabs

Restate and Simplify

Topics:

- Stripping a wordy problem to its core
- Restating in the student's own words
- Identifying the quantity actually asked for
- Turning sentences into a drawable model
- Removing distracting detail
- Simplify-then-solve as a habit

Practice: Wordy problems rewritten in one line, then drawn and solved.

Detail: Restate and simplify: turning wordy problems into drawable ones

Phase 1 Checkpoint

Topics:

- Mixed problem-sum set across all Phase 1 families
- Model choice under mixed conditions
- Explaining each solution aloud
- Extending problems to harder variants
- Heuristics applied without prompting
- Error log started from checkpoint mistakes

Practice: A mixed timed problem-sum set covering every Phase 1 family.

Assessment: Phase 1 checkpoint marked for method and reasoning, feeding the first error-log entries.

Detail: Phase checkpoint: mixed problem-sum set solved, explained and extended

PHASE 2: THE P5-P6 CANON, FAMILY BY FAMILY (MONTHS 5-9)

The problem-sum territory where the PSLE decides its top bands, taught by structure family so your child recognises the skeleton under every costume: fractions of remainder, ratio and its changes, percentage, speed and geometry.

Month 5: Fractions of Remainder and Branch Models (Weeks 17-20)

Remainder Concept and Branch Models

Topics:

- Fraction of a remainder made visible
- Branch models for successive fractions
- Re-cutting into common parts
- Tracking what is left after each stage
- Restoring the whole from a known part

- Two-stage remainder reasoning

Practice: Branch-model problems working from a remainder back to the whole.

Detail: The remainder concept made visible: branch models and the re-cut into common parts

Multi-Stage Remainder Problems

Topics:

- Classic PSLE remainder problem sums
- Three-stage and multi-stage remainders
- Combining branch models with unit value
- Fraction-of-remainder with a given amount
- Spotting the common-part cut quickly
- Presenting long working clearly

Practice: A set of exam-standard remainder problems, including multi-stage versions.

Detail: Classic remainder problems at PSLE standard, including the multi-stage versions that humble adults

Month 6: Ratio, and Ratio That Changes (Weeks 21-24)

Ratio as Units

Topics:

- Ratio read as units
- Equivalent ratios and simplifying
- Sharing a quantity in a given ratio
- The unit-value engine for ratio problems
- Two-term and three-term ratios
- Ratio bars linked to fractions

Practice: Ratio problems solved through the unit-value method with bar support.

Detail: Ratio as units: equivalent ratios, sharing and the unit-value engine

Before-and-After Ratio Problems

Topics:

- Before-and-after ratio changes
- Constant-total problems
- Constant-difference problems
- Constant one-quantity problems
- Choosing the invariant to anchor units
- The PSLE signature ratio family

Practice: Before-and-after ratio sets sorted by which quantity stays constant.

Detail: Before-and-after problems: changed ratios, constant totals and constant differences, the PSLE's signature family

Month 7: Percentage in Every Costume (Weeks 25-28)

Percentage Of, Increase and Decrease

Topics:

- Finding a percentage of a quantity
- Percentage increase and decrease
- GST and discount calculations
- Chained percentage changes
- Percentage with bar-model support
- Everyday money contexts

Practice: Percentage problems including discount-then-GST chains.

Detail: Percentage of, increase and decrease, and GST-and-discount chains

Reverse Percentage at Exam Standard

Topics:

- Reverse percentage: finding the original
- Distinguishing part, whole and percentage
- Percentage combined with bar models
- Multi-step percentage problem sums
- Checking answers against the original
- Exam-standard presentation

Practice: Reverse-percentage and percentage-with-model problems at PSLE level.

Detail: Reverse percentage and percentage-with-model problems at exam standard

Month 8: Speed, Rate and the Journey Problems (Weeks 29-32)

Speed as a Rate With Meaning

Topics:

- Speed understood as distance per unit time
- Distance-time reasoning before formulas
- Deriving the speed, distance, time triangle
- Units and conversions in rate problems
- Constant-speed situations
- Reading a journey from a description

Practice: Distance-time problems solved by reasoning, then confirmed by formula.

Detail: Speed as a rate with meaning: distance-time reasoning before formulas

Journey Problems and Average Speed

Topics:

- Meeting problems on journey diagrams
- Catching-up and overtaking problems
- Average speed over multiple legs
- Total distance over total time
- Two-object motion drawn to scale

- Common average-speed traps

Practice: Meeting, catching-up and average-speed problems on drawn journey diagrams.

Detail: Meeting, catching-up and average-speed problems, drawn on journey diagrams

Month 9: Geometry, Area and the Visual Problems (Weeks 33-36)

Angles and Figures With Justification

Topics:

- Angle facts for lines and triangles
- Angle properties of quadrilaterals
- Justifying each step with a reason
- Composite and overlapping figures
- Unknown angles in multi-shape diagrams
- Clear geometric working

Practice: Angle problems requiring a stated reason for every step.

Detail: Angles, triangles and quadrilaterals with justification language; composite figures

Area, Perimeter and Shaded Regions

Topics:

- Area and perimeter of composite shapes
- Shaded-region classic problems
- Subtracting and combining areas
- Circle area and circumference at P6 standard
- Fractions of a circle and semicircles
- Presenting area working clearly

Practice: Shaded-region and circle problem sums at PSLE standard.

Detail: Area and perimeter problem sums including the shaded-region classics; circles at P6 standard

----- PHASE 3: PAPER CRAFT AND THE PSLE RUNWAY (MONTHS 10-12) -----

Understanding becomes AL bands here: Paper 1 speed without a calculator, Paper 2 working discipline, timed full papers with review rituals, and a peak plan that walks into the PSLE calm.

Month 10: Paper 1 and Paper 2 as Different Skills (Weeks 37-40)

Paper 1 Craft and Mental Strategies

Topics:

- Mental strategies for Paper 1 speed
- No-calculator discipline under time
- Banking one-mark and two-mark questions fast
- Common Paper 1 question types
- Avoiding careless-mark leaks
- Order of attack on Paper 1

Practice: A full Paper 1 section timed, focused on banking the short questions.

Detail: Paper 1 craft: mental strategies, no-calculator discipline and the one-mark questions banked fast

Paper 2 Craft and Method Marks

Topics:

- Presentation of working for method marks
- Calculator use that saves time
- Structuring multi-step Paper 2 answers
- Showing enough working to secure marks
- Units and final-answer discipline
- Choosing when the calculator helps

Practice: Paper 2 questions written up to full method-mark presentation standard.

Detail: Paper 2 craft: presentation of working, method marks, and calculator use that helps instead of wastes time

Month 11: Timed Papers and Mark Banking (Weeks 41-44)

Full Timed Papers and Mark Banking

Topics:

- Full papers at true exam pacing
- Banking secure marks first
- Leaving the hardest problems for last
- Time budgeting across Paper 1 and Paper 2
- Managing pressure and pacing
- Flagging and returning to hard items

Practice: A complete timed paper sat under exam conditions with a pacing plan.

Detail: Full timed papers at exam pacing; the banking discipline, secure marks first, wrestle the monsters last

Review Rituals and the Error Log

Topics:

- Reviewing why each mark was lost
- Naming the specific fix for each error
- Sorting errors by type and frequency
- Beginning error-log closure
- Turning mistakes into targeted practice
- Splitting careless from conceptual errors

Practice: Each lost mark logged with a named fix and a follow-up practice item.
Detail: Review rituals: why each mark was lost, and the named fix; error-log closure begins

Month 12: The Calm Runway (Weeks 45-52)

Weakest-Family Rotations

Topics:

- Rotations targeting the weakest families
- Priority set by the error log
- Confidence work on nearly-mastered problems
- Revisiting problem-sum families that slip
- Spaced return to fixed errors
- Building momentum into the final weeks

Practice: Rotating problem sets aimed at the top error-log priorities each week.

Detail: Weakest-family rotations by error-log priority; confidence work through problems your child can nearly do, then can

Final Cycles and the Peak-Week Plan

Topics:

- Final full paper cycles
- Documented AL-trend progression
- Peak-week light-practice plan
- Rest and routine before the exam
- Exam-day rehearsal and timing
- The PSLE met as a rehearsed place

Practice: Final timed papers with progression recorded, tapering into light peak-week practice.

Assessment: Final timed PSLE-format papers scored and charted to confirm AL-trend progression before the exam.

Detail: Final paper cycles with documented progression; peak-week plan, light practice, rest, and the exam met as a rehearsed place

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal error log and problem-sum family map
- The child's own model-method handbook, drawn not copied
- Heuristics card set: each strategy named with a worked example
- Timed paper cycles with documented AL-trend progression
- Peak-week plan rehearsed before the real PSLE

Total projects: 5+ exam artifacts plus the child's own handbooks

SKILLS THE STUDENT WILL HAVE AT THE END

- Model drawing as a thinking tool
- All major problem-sum structure families
- MOE heuristics, named and applied
- Paper 1 calculator-free fluency
- Paper 2 working and method-mark discipline
- Fractions, ratio, percentage and speed at PSLE depth
- Geometry and shaded-region craft
- Time discipline and mark banking

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes at Singapore evening hours

Practice: Short purposeful sets, never another assessment-book stack

Fluency: Paper 1-style warm-up in every class

Total per week: About 3-4 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week

Mentorship: One dedicated mentor across the whole run

Recordings: Every class recorded for revision

Progress tracking: Timed-paper progression shared with parents

Flexibility: Slots move around school exams and family plans

CERTIFICATION DETAIL

Final certificate: Course completion certificate from Modern Age Coders

Score record: Documented timed-paper progression

Portfolio projects: The child's model-method handbook and heuristics cards

WHERE THIS CAN LEAD

- An AL band that reflects real ability, opening posting options
- Secondary-ready: the algebra handover met with model-method understanding underneath
- E Maths and A Maths readiness through Secondary
- Ready for GEP/DSA-style stretch via competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course aligned to the MOE syllabus and the actual PSLE format?

A: Yes, the MOE primary mathematics syllabus is the spine, and exam work uses the real format: the no-calculator Paper 1, Paper 2 with calculator, and AL scoring awareness throughout. We verify current SEAB formats for your child's cohort rather than assuming.

Q: How do the class timings work from Singapore?

A: You pick two weekly slots and they stay yours.

Q: My child computes well but freezes on problem sums. Can this be fixed?

A: Yes, problem sums are the most teachable 'talent' in Singapore maths. They break into structure families, each with a model shape and heuristic. We teach the families, then the re-cutting flexibility that separates thinkers from template-followers. A term of that work changes what your child sees when they read a problem.

Q: My child already attends a tuition centre. Why switch?

A: Sometimes you should not, a child thriving near the top of a good batch should stay. The switch wins when the batch pace is wrong in either direction: the struggling child whose specific gaps never get airtime, or the capable child rehearsing what they already know. Here, every minute of the hour aims at your child.

Q: When should PSLE prep start?

A: The full 12-month arc suits P5, finishing on the P6 runway; P6 students join a compressed version prioritised by the diagnostic. Earlier starts (P4) spend longer in the method phase, which is the highest-return work there is.

Q: What AL improvement can we expect?

A: We will not invent a number. What you get: an honest baseline in week one, the highest-value families attacked first, and timed-paper evidence you can track yourself. AL bands reward consistency, and consistency is a trainable behaviour.

Q: Can we try it before paying?

A: Yes. Every child starts with a free live demo class at a Singapore evening hour that doubles as the diagnostic, no card details, no obligation.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/psle-maths-mastery>

COURSE 100 OF 118: PYTHON & AI AUTOMATION: MACHINE LEARNING, NLP & SCRIPTING

Course link: <https://learn.modernagecoders.com/courses/python-ai-automation-masterclass-college>

Subject area: Professional AI & Automation Development

Designed for: College students

Who can join: Complete Beginner to Advanced Professional

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Industry-recognized AI & Automation certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

One-year Python automation and AI course: master Python, automation frameworks, machine learning, computer vision, NLP and enterprise automation.

In one line: From 'Hello World' to Production-Ready AI Systems

ABOUT THE PROGRAMME

This is not just a course, it's a complete transformation program. Whether you're a student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled Python developer and AI engineer capable of building, deploying, and scaling real-world automation and AI solutions. You'll master every aspect of Python development: from basic programming to advanced automation, from traditional scripts to cutting-edge AI models, from simple bots to enterprise automation systems. By the end, you'll have built 30+ projects, deployed AI models, and be ready for senior automation engineer or AI developer roles.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no prerequisites required
- 1 year of structured, progressive learning
- Covers 100% of modern Python automation & AI stack
- Real industry projects and case studies
- Interview preparation and AI system design
- Lifetime access and updates
- Build production-ready AI portfolio

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Python Basics, Programming Logic, Core Libraries

Phase 2: Automation Mastery (Months 4-6): Web Scraping, Task Automation, APIs, Bots

Phase 3: AI & Machine Learning (Months 7-9): ML Algorithms, Deep Learning, Computer Vision, NLP

Phase 4: Advanced AI & Production (Months 10-12): MLOps, Cloud AI, Enterprise Solutions, Deployment

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to all

Coding experience: Absolute beginner friendly - starts from zero

Equipment: Computer/laptop with internet connection

Time commitment: 15-20 hours per week consistently

English: Basic reading and comprehension

Motivation: Strong desire to learn AI and automation

WHO THIS COURSE IS FOR

Students: College students looking for AI/ML career path

Working professionals: Career switchers wanting to enter AI field

Entrepreneurs: Want to build AI-powered products

Freelancers: Want to offer AI/automation services

Kids: Motivated teenagers (14+) can start this journey

Anyone: Anyone passionate about AI and automation

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PYTHON FOUNDATION & CORE PROGRAMMING (MONTHS 1-3, WEEKS 1-13)

Build rock-solid Python fundamentals. Learn programming concepts, master Python syntax, and create your first automation projects.

Months 1-2: Python Fundamentals & Programming Logic (Week 1-8)

Introduction to Python & Setup

Topics:

- What is Python? History and why it's perfect for automation
- Python ecosystem: CPython, PyPy, Anaconda
- Installing Python 3.x on Windows, Mac, Linux
- IDE setup: VS Code, PyCharm, Jupyter Notebook
- Virtual environments: venv, virtualenv, conda
- Package management with pip
- Your first Python program
- Python interactive shell and REPL
- Basic input/output operations
- Comments and documentation
- Python code style guide (PEP 8)
- Introduction to Python's philosophy

Projects built:

- Set up complete Python development environment
- Simple calculator program
- Personal information collector

Practice: Daily: 30 min coding exercises, 1 hour Python basics

Variables, Data Types & Operations

Topics:

- Variables and naming conventions
- Dynamic typing in Python
- Basic data types: int, float, str, bool
- Type conversion and casting
- Numbers: integers, floats, complex
- Arithmetic operators and precedence
- String creation and manipulation
- String methods: upper, lower, strip, split, join
- String formatting: f-strings, format(), %
- String slicing and indexing
- Boolean operations and truth values
- Comparison and logical operators

Projects built:

- Temperature converter (C/F/K)
- Text analyzer and formatter
- Simple encryption/decryption tool

Practice: Complete 50 string manipulation exercises

Control Flow & Loops

Topics:

- If-elif-else statements
- Nested conditional statements
- Ternary operators
- While loops and break/continue
- For loops and range() function
- Iterating through sequences
- Nested loops and patterns
- Loop else clause
- List comprehensions introduction
- Common loop patterns
- Debugging techniques
- Error messages and troubleshooting

Projects built:

- Number guessing game
- Pattern generator (pyramids, diamonds)
- Prime number calculator
- Password strength validator

Practice: Solve 40 loop and condition problems

Data Structures in Python

Topics:

- Lists: creation, indexing, slicing
- List methods: append, extend, insert, remove, pop
- List comprehensions advanced
- Tuples: immutable sequences
- Sets: unique elements, set operations
- Dictionaries: key-value pairs
- Dictionary methods and operations
- Nested data structures
- Copying: shallow vs deep copy
- Memory management basics
- Collections module: defaultdict, Counter, deque
- Data structure performance comparison

Projects built:

- Student management system
- Shopping cart application
- Contact book with search
- Data structure performance analyzer

Practice: Build 10 programs using different data structures

Month 3: Functions, Modules & OOP (Week 9-13)

Functions & Functional Programming

Topics:

- Function definition and calling
- Parameters and arguments
- Default parameters and keyword arguments
- *args and **kwargs
- Return values and multiple returns
- Scope and LEGB rule
- Global and nonlocal keywords
- Lambda functions
- Map, filter, and reduce
- Decorators basics
- Generators and yield
- Recursion and recursive algorithms

Projects built:

- Modular calculator with functions
- Recursive file system explorer
- Function library for text processing
- Custom decorator collection

Practice: Create 30 reusable utility functions

Object-Oriented Programming

Topics:

- Classes and objects
- Attributes and methods
- Constructor (__init__)
- Instance vs class variables
- Inheritance and polymorphism
- Method overriding
- Multiple inheritance and MRO
- Abstract classes and interfaces
- Magic methods (dunder methods)
- Property decorators

- Static and class methods
- SOLID principles in Python

Projects built:

- Bank account management system
- Employee hierarchy system
- Game character classes
- Library management OOP

Practice: Design 10 real-world systems using OOP

Modules, Packages & Error Handling

Topics:

- Modules and import statements
- Creating custom modules
- Package structure and `__init__.py`
- Relative and absolute imports
- The Python Standard Library tour
- Exception handling: try, except, finally
- Raising custom exceptions
- Exception hierarchy
- Logging module usage
- Debugging with pdb
- Unit testing with unittest
- Test-driven development basics

Projects built:

- Custom package for utilities
- Error handling framework
- Logging system implementation
- Test suite for previous projects

Practice: Add proper error handling to all previous projects

File Handling & Data Processing

Topics:

- File operations: open, read, write, close
- Context managers (with statement)
- Text file processing
- CSV file handling
- JSON data manipulation
- XML parsing basics
- Binary file operations
- Path manipulation with pathlib
- Directory operations
- Regular expressions with re module
- Pattern matching and extraction
- Data serialization with pickle

Projects built:

- Log file analyzer
- CSV to JSON converter
- Bulk file renamer
- Text search and replace tool

Practice: Process 20 different file formats

Phase 1 Capstone Project

Topics:

- Project planning and design
- Code organization best practices
- Documentation with docstrings
- Creating requirements.txt
- Git and GitHub for Python projects

Projects built:

- CAPSTONE: Personal Finance Tracker
- Features: expense tracking, budgeting, reports, data visualization, file export/import
- Alternative: Task Management System
- Alternative: Quiz Application with Score Tracking

Assessment: Phase 1 Final Exam - Complete Python fundamentals assessment

Months 4-6: Advanced Python & Automation (Week 14-26)

Advanced Python Concepts

Topics:

- Iterators and iteration protocol
- Creating custom iterators
- Generator expressions
- Coroutines and async basics
- Context managers and contextlib
- Metaclasses introduction
- Descriptors and properties
- Abstract base classes
- Multiple inheritance patterns
- Mixin classes
- Design patterns in Python
- Performance optimization techniques

Projects built:

- Custom data structure library

- Async task processor
- Plugin system architecture
- Performance monitoring tool

Practice: Implement 10 design patterns in Python

Database Programming

Topics:

- SQLite with Python
- SQL basics: CREATE, INSERT, SELECT, UPDATE, DELETE
- Database connections and cursors
- Parameterized queries and SQL injection prevention
- PostgreSQL with psycopg2
- MySQL with PyMySQL
- MongoDB with pymongo
- SQLAlchemy ORM basics
- Database migrations
- Connection pooling
- Redis for caching
- Database backup and restore

Projects built:

- Inventory management system
- Blog platform backend
- User authentication system
- Data migration tool

Practice: Build 5 database-driven applications

Web Development with Python

Topics:

- HTTP protocol basics
- Flask framework introduction
- Routes and views in Flask
- Templates with Jinja2
- Forms and validation
- Session management
- RESTful API design
- FastAPI framework
- Django basics
- MVC/MVT architecture
- Authentication and authorization
- Deployment basics

Projects built:

- Personal blog with Flask
- REST API for todo app
- FastAPI microservice
- Django admin panel

Practice: Create 5 different web applications

Network Programming & APIs

Topics:

- Sockets programming basics
- TCP and UDP protocols
- Creating client-server applications
- HTTP requests with requests library
- Consuming REST APIs
- API authentication methods
- GraphQL with Python
- WebSocket connections
- Email automation with smtplib
- SSH connections with paramiko
- FTP operations
- Network security basics

Projects built:

- Chat application (client-server)
- API aggregator service
- Email automation system
- Network monitoring tool

Practice: Integrate with 10 different APIs

Phase 2 Capstone Project

Topics:

- Full-stack application design
- API documentation
- Security best practices
- Performance optimization
- Deployment strategies

Projects built:

- MAJOR CAPSTONE: Complete Web Automation Platform
- Features: user auth, task scheduling, API integration, web UI, database, reporting
- Alternative: Social Media Analytics Tool
- Alternative: E-commerce Scraper and Monitor

Assessment: Phase 2 Final Exam - Build complete automation system

PHASE 2: WEB SCRAPING & AUTOMATION TOOLS (MONTHS 4-6, WEEKS 14-26)

Master web scraping, browser automation, task automation, and building robust automation frameworks.

Months 4-5: Web Scraping & Browser Automation (Week 14-21)

Web Scraping Fundamentals

Topics:

- HTML/CSS basics for scraping
- BeautifulSoup library
- Parsing HTML documents
- Finding elements: find, find_all
- CSS selectors in BeautifulSoup
- Navigating the parse tree
- Extracting text and attributes
- Handling dynamic content basics
- robots.txt and ethical scraping
- User-Agent headers
- Rate limiting and delays
- Scraping best practices

Projects built:

- News article scraper
- Product price monitor
- Job listing aggregator
- Wikipedia data extractor

Practice: Scrape 20 different websites

Advanced Web Scraping

Topics:

- Scrapy framework introduction
- Scrapy spiders and items
- Scrapy pipelines
- Handling pagination
- Following links recursively
- Handling forms and CSRF tokens
- Session management in scraping
- Proxy rotation
- Handling CAPTCHAs
- Scraping JavaScript-rendered pages
- API reverse engineering
- Data cleaning and validation

Projects built:

- E-commerce site scraper with Scrapy
- Social media data collector
- Real estate listing scraper
- Multi-site comparison tool

Practice: Build 10 production-ready scrapers

Selenium & Browser Automation

Topics:

- Selenium WebDriver setup
- Browser drivers: Chrome, Firefox, Edge
- Finding elements with Selenium
- Interacting with elements: click, send_keys
- Waiting strategies: implicit, explicit, fluent
- Expected conditions
- Handling popups and alerts
- Frame and window switching
- JavaScript execution
- Screenshots and video recording
- Headless browser mode
- Selenium Grid for parallel execution

Projects built:

- Automated form filler
- Social media bot
- Web testing automation suite
- Booking automation system

Practice: Automate 15 web workflows

Alternative Automation Tools

Topics:

- Playwright introduction
- Playwright vs Selenium comparison
- Puppeteer with Pypeteer
- Requests-HTML for JavaScript sites
- MechanicalSoup for simple automation
- AutoGUI for desktop automation
- Keyboard and mouse control
- Screen capture and OCR
- Task scheduling with schedule library
- Cron jobs and task schedulers
- Windows Task Scheduler integration

- Building automation frameworks

Projects built:

- Desktop automation suite
- Cross-browser testing tool
- Automated report generator
- System administration automator

Practice: Create 10 different automation workflows

Data Processing & Analysis

Topics:

- NumPy arrays and operations
- Pandas DataFrames basics
- Data loading and saving
- Data cleaning with Pandas
- Data transformation and aggregation
- Merge, join, and concatenate
- GroupBy operations
- Time series data handling
- Data visualization with Matplotlib
- Seaborn for statistical plots
- Plotly for interactive visualizations
- Creating automated reports

Projects built:

- Sales data analysis dashboard
- Stock market analyzer
- Survey data processor
- Automated reporting system

Practice: Analyze 10 real-world datasets

Month 6: Enterprise Automation & Integration (Week 22-26)

RPA & Process Automation

Topics:

- Robotic Process Automation concepts
- Python RPA libraries
- UiPath integration with Python
- Email automation advanced
- Excel automation with openpyxl
- Word document automation
- PDF processing and generation
- PowerPoint automation
- Calendar and scheduling automation
- Database automation workflows
- File system monitoring
- Process orchestration

Projects built:

- Invoice processing system
- HR onboarding automation
- Report generation pipeline
- Document management system

Practice: Automate 10 business processes

API Development & Microservices

Topics:

- RESTful API design principles
- FastAPI advanced features
- API versioning strategies
- Authentication: JWT, OAuth2
- Rate limiting and throttling
- API documentation with Swagger
- GraphQL API development
- Microservices architecture
- Message queues: RabbitMQ, Celery
- Event-driven architecture
- Service discovery
- API gateway patterns

Projects built:

- Complete REST API with auth
- Microservices e-commerce backend
- Real-time notification service
- API gateway implementation

Practice: Build 5 production APIs

Cloud Automation & DevOps

Topics:

- AWS SDK for Python (boto3)
- EC2 instance management
- S3 operations and automation
- Lambda functions with Python
- SQS and SNS integration
- Google Cloud Platform basics
- Azure SDK for Python
- Infrastructure as Code with Python

- Docker containerization
- Kubernetes with Python client
- CI/CD pipeline automation
- Monitoring and alerting

Projects built:

- Cloud resource manager
- Serverless data processor
- Auto-scaling application
- Infrastructure automation tool

Practice: Deploy 10 applications to cloud

Testing & Quality Assurance

Topics:

- Unit testing with pytest
- Test fixtures and parametrization
- Mocking and patching
- Integration testing strategies
- API testing with pytest
- Selenium for test automation
- Performance testing with Locust
- Load testing strategies
- Security testing basics
- Code coverage analysis
- Continuous testing
- Test reporting and documentation

Projects built:

- Complete test suite for API
- E2E testing framework
- Performance testing suite
- Automated regression tests

Practice: Write tests for all previous projects

Security & Best Practices

Topics:

- Security in Python applications
- Input validation and sanitization
- SQL injection prevention
- XSS and CSRF protection
- Password hashing with bcrypt
- Encryption with cryptography library
- Secure API development
- Environment variables and secrets
- Security scanning tools
- OWASP Top 10 for Python
- Audit logging
- Compliance and regulations

Projects built:

- Security audit tool
- Encryption utility suite
- Secure authentication system
- Compliance checker tool

Practice: Security audit all projects

Months 7-9: AI & Machine Learning (Week 27-39)

Machine Learning Fundamentals

Topics:

- What is Machine Learning?
- Types: Supervised, Unsupervised, Reinforcement
- ML workflow and pipeline
- Data preprocessing techniques
- Feature engineering basics
- Train-test split
- Cross-validation
- Evaluation metrics
- Overfitting and underfitting
- Bias-variance tradeoff
- Scikit-learn introduction
- Model selection strategies

Projects built:

- Iris classification project
- House price prediction
- Customer segmentation
- Basic recommendation system

Practice: Complete 10 ML tutorials

Supervised Learning Algorithms

Topics:

- Linear regression
- Logistic regression
- Decision trees
- Random forests
- Support Vector Machines

- K-Nearest Neighbors
- Naive Bayes
- Gradient Boosting (XGBoost, LightGBM)
- Ensemble methods
- Hyperparameter tuning
- Grid search and random search
- Feature importance analysis

Projects built:

- Credit risk assessment
- Sales forecasting model
- Email spam classifier
- Disease prediction system

Practice: Implement all algorithms from scratch

Unsupervised Learning & Deep Learning Intro

Topics:

- K-Means clustering
- Hierarchical clustering
- DBSCAN
- Principal Component Analysis
- Dimensionality reduction
- Anomaly detection
- Association rules
- Introduction to neural networks
- Perceptron and backpropagation
- Activation functions
- TensorFlow basics
- Keras API

Projects built:

- Customer segmentation advanced
- Anomaly detection system
- Market basket analysis
- Simple neural network from scratch

Practice: Solve 15 unsupervised learning problems

Deep Learning with TensorFlow/PyTorch

Topics:

- Deep neural networks
- Convolutional Neural Networks (CNN)
- Image classification basics
- Transfer learning
- Recurrent Neural Networks (RNN)
- LSTM and GRU
- PyTorch introduction
- Model training and optimization
- Regularization techniques
- Batch normalization
- Dropout layers
- Model deployment basics

Projects built:

- Image classifier with CNN
- Text sentiment analysis with RNN
- Time series prediction with LSTM
- Transfer learning project

Practice: Train 10 different neural networks

Phase 3 Capstone Project

Topics:

- End-to-end ML project
- Data pipeline design
- Model training and evaluation
- Model deployment
- API creation for model
- Monitoring and maintenance

Projects built:

- MAJOR CAPSTONE: Complete ML-Powered Application
- Features: data collection, preprocessing, model training, API, web interface, monitoring
- Alternative: AI-Powered Chatbot
- Alternative: Computer Vision Application

Assessment: Phase 3 Final Exam - Build complete ML system

PHASE 3: ADVANCED AI & COMPUTER VISION (MONTHS 7-9, WEEKS 27-39)

Master advanced AI techniques, computer vision, natural language processing, and production AI systems.

Months 7-8: Computer Vision & Image Processing (Week 27-34)

Image Processing Fundamentals

Topics:

- Digital image basics
- OpenCV installation and setup

- Image loading and display
- Color spaces: RGB, HSV, LAB
- Image transformations
- Geometric transformations
- Image filtering and blurring
- Edge detection: Canny, Sobel
- Morphological operations
- Histogram equalization
- Image thresholding
- Contour detection

Projects built:

- Image enhancement tool
- Document scanner
- Photo editor with filters
- Edge detection application

Practice: Process 100 different images

Advanced Computer Vision

Topics:

- Feature detection: SIFT, SURF, ORB
- Feature matching
- Object detection basics
- Haar Cascades
- Face detection and recognition
- Object tracking algorithms
- Optical flow
- Background subtraction
- Template matching
- Image segmentation
- Panorama stitching
- 3D reconstruction basics

Projects built:

- Face recognition system
- Object tracking application
- Motion detection security system
- Augmented reality basics

Practice: Build 10 computer vision applications

Deep Learning for Computer Vision

Topics:

- CNN architectures: LeNet, AlexNet, VGG
- ResNet and skip connections
- Object detection: YOLO, R-CNN
- Semantic segmentation
- Instance segmentation
- Face recognition with deep learning
- Pose estimation
- Image generation: GANs basics
- Style transfer
- Image super-resolution
- Video analysis with deep learning
- Real-time processing optimization

Projects built:

- Custom object detector
- Real-time face mask detector
- Pose estimation fitness app
- Style transfer application

Practice: Train 15 vision models

Natural Language Processing

Topics:

- NLP fundamentals
- Text preprocessing: tokenization, stemming
- Lemmatization and POS tagging
- Named Entity Recognition
- Bag of Words and TF-IDF
- Word embeddings: Word2Vec, GloVe
- Sentiment analysis techniques
- Topic modeling: LDA
- Text classification
- Sequence-to-sequence models
- Attention mechanisms
- BERT and Transformers introduction

Projects built:

- Sentiment analysis API
- Text summarization tool
- Named entity extraction system
- Language detection service

Practice: Process 50 text datasets

Advanced NLP with Transformers

Topics:

- Transformer architecture deep dive

- Hugging Face library
- Pre-trained models: BERT, GPT, RoBERTa
- Fine-tuning transformers
- Text generation with GPT
- Question answering systems
- Text translation
- Chatbot development
- Document similarity
- Information extraction
- Multi-lingual NLP
- NLP model deployment

Projects built:

- AI chatbot with transformers
- Document Q&A system
- Multi-language translator
- Content generation tool

Practice: Fine-tune 10 transformer models

Month 9: Reinforcement Learning & Advanced AI (Week 35-39)

Reinforcement Learning Basics

Topics:

- RL fundamentals and concepts
- Markov Decision Processes
- Value functions and policies
- Q-Learning algorithm
- Deep Q-Networks (DQN)
- Policy gradient methods
- Actor-Critic methods
- OpenAI Gym environment
- Custom environment creation
- Training RL agents
- Exploration vs exploitation
- Reward shaping

Projects built:

- Game-playing AI agent
- Robot navigation simulator
- Trading bot with RL
- Resource allocation optimizer

Practice: Train agents for 10 different environments

Time Series & Forecasting

Topics:

- Time series analysis basics
- Trend and seasonality
- Stationarity testing
- ARIMA models
- Seasonal ARIMA (SARIMA)
- Exponential smoothing
- Prophet library
- LSTM for time series
- Multi-variate time series
- Anomaly detection in time series
- Real-time forecasting
- Evaluation metrics for forecasting

Projects built:

- Stock price predictor
- Weather forecasting system
- Sales forecasting tool
- Anomaly detection for IoT

Practice: Forecast 20 different time series

AI Ethics & Explainable AI

Topics:

- AI bias and fairness
- Ethical considerations in AI
- Privacy-preserving ML
- Federated learning basics
- Model interpretability
- LIME and SHAP
- Feature importance visualization
- Model debugging techniques
- Adversarial attacks and defense
- Responsible AI practices
- AI governance
- Regulatory compliance (GDPR, etc.)

Projects built:

- Bias detection tool
- Model explainability dashboard
- Privacy-preserving ML system
- AI audit framework

Practice: Audit 10 AI models for bias and fairness

Edge AI & IoT Integration

Topics:

- Edge computing concepts
- TensorFlow Lite
- Model quantization and pruning
- ONNX format
- Raspberry Pi setup for AI
- Arduino and micro-controllers
- Sensor data processing
- Real-time inference on edge
- IoT protocols: MQTT, CoAP
- Edge-cloud hybrid architectures
- Power optimization for edge AI
- Edge AI deployment strategies

Projects built:

- Smart home automation system
- Edge-based security camera
- IoT sensor data analyzer
- Portable AI assistant device

Practice: Deploy 10 models to edge devices

MLOps & Production Systems

Topics:

- MLOps principles and practices
- Model versioning with DVC
- Experiment tracking: MLflow, Weights & Biases
- Model registry and management
- Continuous integration for ML
- Model monitoring in production
- A/B testing for ML
- Model retraining pipelines
- Feature stores
- Model serving: REST, gRPC
- Kubernetes for ML workloads
- Cost optimization for ML

Projects built:

- Complete MLOps pipeline
- Model monitoring dashboard
- Automated retraining system
- ML experiment tracker

Practice: Set up MLOps for 5 projects

Months 10-12: Advanced Systems & Production (Week 40-52)

Distributed Computing & Big Data

Topics:

- Distributed systems concepts
- Apache Spark with PySpark
- Distributed data processing
- Spark ML library
- Dask for parallel computing
- Ray framework
- Distributed training strategies
- Data parallelism vs model parallelism
- Apache Kafka integration
- Stream processing with Spark
- Big data storage: HDFS, S3
- Data lake architectures

Projects built:

- Big data analytics pipeline
- Distributed ML training system
- Real-time stream processor
- Data lake implementation

Practice: Process 10 large-scale datasets

Advanced Deep Learning

Topics:

- Generative AI: GANs advanced
- Variational Autoencoders (VAE)
- Diffusion models basics
- Neural Architecture Search
- Meta-learning concepts
- Few-shot learning
- Multi-modal learning
- Graph Neural Networks
- Attention mechanisms advanced
- Vision Transformers
- Self-supervised learning
- Contrastive learning

Projects built:

- Image generation with GAN
- Multi-modal AI system

- Graph-based recommendation engine
- Self-supervised learning project

Practice: Implement 10 advanced DL architectures

Cloud AI Services & Deployment

Topics:

- AWS SageMaker
- Google Cloud AI Platform
- Azure Machine Learning
- Model deployment strategies
- Serverless AI with Lambda
- Container deployment for ML
- Model serving with TorchServe
- TensorFlow Serving
- API Gateway for ML models
- Load balancing for ML
- Auto-scaling ML services
- Cost management for cloud AI

Projects built:

- Deploy model to all major clouds
- Serverless ML API
- Auto-scaling ML service
- Multi-cloud AI deployment

Practice: Deploy 15 models to production

Industry Applications & Specializations

Topics:

- AI in healthcare
- Financial AI and algorithmic trading
- Autonomous systems basics
- Robotics and AI
- AI for cybersecurity
- Recommendation systems advanced
- Search and ranking algorithms
- AI in gaming
- Speech recognition and synthesis
- Computer-aided design with AI
- Agricultural AI applications
- AI in manufacturing

Projects built:

- Industry-specific AI solution
- Medical diagnosis assistant
- Trading algorithm
- Game AI opponent

Practice: Explore 10 industry use cases

Phase 4 Final Capstone - Part 1

Topics:

- Project ideation and planning
- System architecture design
- Technology stack selection
- Data pipeline design
- Model selection and training
- API design and documentation
- Frontend planning
- Scalability considerations

Projects built:

- FINAL CAPSTONE: Production AI Platform
- Complete end-to-end AI solution with data pipeline, multiple models, API, web interface, monitoring, and deployment

----- PHASE 4: PRODUCTION SYSTEMS & CAREER PREPARATION (MONTHS 10-12, WEEKS 40-52) -----

Master production deployment, advanced AI systems, and prepare for professional AI/automation engineering roles.

Months 10-11: Production Systems & Enterprise AI (Week 40-47)

Enterprise AI Architecture

Topics:

- Enterprise AI strategy
- AI governance frameworks
- Data architecture for AI
- Master data management
- Data quality and validation
- ETL/ELT pipelines
- Data warehousing for AI
- Real-time data pipelines
- Event-driven architectures
- Microservices for AI
- API management platforms
- Enterprise security for AI

Projects built:

- Enterprise data pipeline
- AI microservices architecture
- Real-time ML platform
- Data governance system

Practice: Design 5 enterprise AI architectures

AutoML & Model Optimization

Topics:

- AutoML concepts and tools
- Hyperparameter optimization advanced
- Bayesian optimization
- Neural Architecture Search implementation
- Model compression techniques
- Knowledge distillation
- Quantization strategies
- Pruning neural networks
- Mobile optimization with TFLite
- ONNX optimization
- Hardware acceleration: GPU, TPU
- Benchmarking and profiling

Projects built:

- AutoML pipeline implementation
- Model optimization toolkit
- Mobile AI application
- Hardware-optimized inference

Practice: Optimize 20 models for production

Advanced Automation Frameworks

Topics:

- Building automation frameworks
- Plugin architectures
- Configuration management
- Workflow orchestration with Airflow
- Luigi for complex pipelines
- Prefect workflow automation
- Event-driven automation
- Webhook integrations
- Custom DSLs for automation
- Error recovery mechanisms
- Monitoring and alerting
- Automation testing strategies

Projects built:

- Custom automation framework
- Workflow orchestration system
- Event-driven automation platform
- Self-healing automation system

Practice: Build 10 enterprise automation solutions

Conversational AI & Chatbots

Topics:

- Conversational AI architecture
- Intent recognition and NLU
- Dialog management systems
- Context handling in conversations
- Rasa framework deep dive
- Dialogflow integration
- Voice assistants with Python
- Speech recognition integration
- Text-to-speech systems
- Multi-turn conversations
- Chatbot testing and evaluation
- Production chatbot deployment

Projects built:

- Enterprise chatbot with Rasa
- Voice-enabled AI assistant
- Customer service automation
- Multi-channel bot platform

Practice: Build 5 different chatbot systems

AI Product Development

Topics:

- AI product management
- User research for AI products
- MVP development for AI
- User interface for AI systems
- Feedback loops and improvement
- A/B testing AI features
- Product metrics for AI
- User adoption strategies
- Documentation and tutorials
- Community building
- Monetization strategies

- Scaling AI products

Projects built:

- AI SaaS product prototype
- User feedback system
- Product analytics dashboard
- Go-to-market strategy

Practice: Develop 3 AI product concepts

Month 11: Integration & Advanced Topics (Week 48-51)

Integration with Enterprise Systems

Topics:

- ERP integration strategies
- CRM system integration
- Salesforce automation with Python
- SAP integration basics
- Microsoft 365 automation
- Google Workspace automation
- Slack bot development
- Teams app development
- Zapier-like automation building
- IFTTT concepts implementation
- Enterprise service bus
- Legacy system integration

Projects built:

- CRM automation system
- Enterprise integration hub
- Slack/Teams bot suite
- Legacy system modernization

Practice: Integrate with 10 enterprise systems

Advanced Data Engineering

Topics:

- Data pipeline architecture
- Apache Airflow advanced
- Data quality frameworks
- CDC (Change Data Capture)
- Data versioning strategies
- Feature engineering automation
- Feature stores implementation
- Data catalog systems
- Metadata management
- Data lineage tracking
- DataOps practices
- Real-time analytics systems

Projects built:

- Complete data platform
- Feature store implementation
- Data quality monitoring
- Real-time analytics dashboard

Practice: Build 5 data engineering pipelines

Specialized AI Applications

Topics:

- Anomaly detection systems
- Fraud detection algorithms
- Predictive maintenance
- Supply chain optimization
- Dynamic pricing systems
- Personalization engines
- Content moderation AI
- Document AI and OCR
- Medical imaging AI
- Satellite image analysis
- Audio processing and music AI
- Generative AI applications

Projects built:

- Fraud detection system
- Predictive maintenance platform
- Personalization engine
- Document processing pipeline

Practice: Build 10 specialized AI applications

Performance & Scalability

Topics:

- Performance profiling Python apps
- Memory optimization techniques
- Async programming with asyncio
- Concurrent.futures module
- Multiprocessing strategies
- Cython for performance
- NumPy optimization tricks
- Database query optimization

- Caching strategies with Redis
- Load balancing techniques
- Horizontal scaling patterns
- Performance monitoring tools

Projects built:

- High-performance data processor
- Scalable API service
- Performance optimization toolkit
- Load testing framework

Practice: Optimize 10 systems for scale

Open Source Contribution

Topics:

- Open source ecosystem
- Finding projects to contribute
- Understanding codebases
- Git advanced workflows
- Creating pull requests
- Writing good documentation
- Issue tracking and management
- Code review best practices
- Creating your own package
- Publishing to PyPI
- Building community
- Maintaining open source projects

Projects built:

- Contribute to 3 open source projects
- Create and publish Python package
- Start your own open source project
- Documentation contributions

Practice: Make 20 open source contributions

Month 12: Final Projects & Career Launch (Week 49-52)

Final Capstone - Development

Topics:

- Requirements gathering
- System design documentation
- Architecture implementation
- Core functionality development
- Testing implementation
- Performance optimization
- Security implementation
- Documentation writing

Projects built:

- FINAL CAPSTONE CONTINUED: Complete Production AI/Automation Platform
- Full implementation with all features, testing, documentation, and deployment

Practice: Complete all capstone features

Final Capstone - Deployment

Topics:

- Production deployment preparation
- CI/CD pipeline setup
- Monitoring and logging setup
- Performance testing
- Security audit
- Load testing
- Disaster recovery planning
- User documentation
- API documentation
- Video demonstrations

Projects built:

- Deploy capstone to production
- Set up monitoring and alerts
- Create comprehensive documentation
- Record demo videos

Practice: Production deployment checklist

Portfolio & Personal Branding

Topics:

- Portfolio website creation
- Project showcases
- Case study writing
- Technical blog writing
- GitHub profile optimization
- LinkedIn optimization
- Personal branding strategy
- Online presence building
- Networking strategies
- Conference participation
- Tech community involvement
- Content creation planning

Projects built:

- Professional portfolio website
- 5 detailed project case studies
- 10 technical blog posts
- Optimized professional profiles

Practice: Build complete professional presence

Interview Preparation

Topics:

- Python coding interviews
- Data structures and algorithms review
- System design interviews
- ML system design
- Behavioral interview preparation
- STAR method for responses
- Technical presentation skills
- Whiteboard coding practice
- Take-home assignments
- Salary negotiation
- Contract vs full-time
- Remote work considerations

Projects built:

- Interview preparation portfolio
- Coding challenge solutions
- System design diagrams
- Behavioral response bank

Practice: 50 coding problems, 10 system designs

Month 12: Career Launch & Continuous Learning (Week 51-52)

Job Search & Applications

Topics:

- Job search strategies
- Resume optimization for ATS
- Cover letter writing
- Job board navigation
- Recruiter engagement
- Company research methods
- Application tracking
- Follow-up strategies
- Interview scheduling
- Reference preparation
- Background check preparation
- Offer evaluation

Projects built:

- Tailored resumes for different roles
- Cover letter templates
- Application tracker
- Interview preparation notes

Freelancing & Consulting

Topics:

- Freelancing platforms
- Client acquisition
- Project scoping and estimation
- Contract negotiation
- Rate setting strategies
- Invoice and payment systems
- Client communication
- Project management for freelancers
- Building recurring revenue
- Scaling freelance business
- Tax considerations
- Legal aspects of freelancing

Deliverables:

- Freelance profile setup
- Service offerings document
- Rate card and packages
- Contract templates
- First client acquisition

Continuous Learning & Growth

Topics:

- Staying updated with AI trends
- Advanced certifications to pursue
- Conference and workshop participation
- Research paper reading strategies
- Building a learning routine
- Mentorship and coaching
- Teaching and knowledge sharing
- Side project ideas
- Startup opportunities in AI
- Career progression paths
- Leadership in tech

- Future of AI and automation

Deliverables:

- Personal development plan
- Learning roadmap for next year
- Professional network building
- Mentorship connections
- Community involvement plan

Assessment: FINAL COMPREHENSIVE EXAM - Complete assessment of all Python, Automation, and AI skills

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Calculator, File Manager, Data Analyzer, Finance Tracker, Task Manager
- Phase 2: Web Scraper Suite, Browser Automation Tools, API Platform, RPA System
- Phase 3: ML Models (10+), Computer Vision Apps, NLP Systems, Chatbots
- Phase 4: Production AI Platform, Enterprise Automation, MLOps Pipeline
- Final: Complete AI-Powered SaaS Application

Total projects: 30+ projects ranging from simple scripts to complex AI systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Programming: Python (Expert), SQL, Git, Linux, Shell Scripting
- Automation: Web Scraping, Selenium, RPA, API Development, Task Automation
- Data: Pandas, NumPy, Data Analysis, ETL, Big Data (Spark)
- AI/ML: Scikit-learn, TensorFlow, PyTorch, Deep Learning, Computer Vision, NLP
- Deployment: Docker, Kubernetes, AWS, GCP, Azure, CI/CD, MLOps
- Databases: SQL, NoSQL, MongoDB, PostgreSQL, Redis, Vector DBs
- Testing: Unit Testing, Integration Testing, pytest, Test Automation
- Tools: Jupyter, VS Code, PyCharm, Postman, Docker, Kubernetes
- Concepts: OOP, Design Patterns, Algorithms, System Design, Architecture

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-8 hours
 Hands on coding: 8-10 hours
 Projects: 3-5 hours
 Practice problems: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly doubt clearing sessions
 Mentorship: 1-on-1 mentor guidance
 Community: Active Discord/Slack community
 Code review: Expert code reviews for projects
 Career support: Resume review, interview prep, job referrals
 Lifetime access: All content, updates, and new modules

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion
 Final certificate: Professional Python & AI Automation Certification
 LinkedIn badge: Add to LinkedIn profile
 Industry recognized: Backed by industry professionals
 Portfolio projects: 30+ projects to showcase

WHERE THIS CAN LEAD

- Python Developer (Junior to Senior)
- Automation Engineer
- Machine Learning Engineer
- AI Engineer / AI Architect
- Data Scientist
- Computer Vision Engineer
- NLP Engineer
- MLOps Engineer
- RPA Developer
- AI Product Manager
- Freelance AI Consultant
- AI Startup Founder

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Growing roles: Rs 4-6 LPA (Python Developer)

Experienced roles: Rs 8-15 LPA (AI/ML Engineer); Rs 15-25 LPA (Senior AI Engineer); Rs 20-40+ LPA (AI Architect/Lead)
Freelance: Rs 1,000-5,000/hour
International: \$60k-150k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Python AI Automation and why is it one of the highest-demand skills?

A: Python AI Automation combines Python programming with Artificial Intelligence to automate complex tasks that traditionally required human intelligence. From automating data entry to building intelligent chatbots, this skill is in massive demand.

Q: Do I need prior programming experience for this Python AI Automation course?

A: No prior experience required! This course starts from absolute zero - we'll teach you Python fundamentals before moving to advanced AI and automation concepts.

Q: What tools and technologies will I learn in this 12-month program?

A: You'll master Python, TensorFlow, PyTorch, Scikit-learn, Pandas, NumPy, Selenium, BeautifulSoup, OpenCV, Natural Language Processing (NLP), Deep Learning, AWS/GCP deployment, Docker, and MLOps. By completion, you'll be proficient in 30+ technologies used by AI engineers at top companies.

Q: What kind of projects will I build to showcase in my portfolio?

A: These projects are designed to impress employers.

Q: What careers can I pursue after completing this Python AI Automation course?

A: Many also start their own AI automation businesses.

Q: How is this course different from other Python courses available online?

A: Unlike basic Python courses, this program focuses specifically on AI and automation applications. You learn not just programming, but how to build intelligent systems that solve real business problems, working on hands-on projects throughout. We include mentorship, career guidance, and interview preparation to help you present your work well.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/python-ai-automation-masterclass-college>

COURSE 101 OF 118: PYTHON & AI FOR KIDS: CODING, GAMES, ROBOTICS & AI PROJECTS

Course link: <https://learn.modernagecoders.com/courses/python-ai-kids-masterclass>

Subject area: Kids Programming & AI Education

Designed for: Kids

Who can join: Complete Beginner to Advanced Young Programmer

Course length: 12 months (52 weeks)

Time needed: 5-8 hours/week recommended

Certificate: Young Python & AI Developer Certificate

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

One-year Python and AI course for kids: master programming basics, game development, robotics and machine learning by building amazing AI projects.

In one line: From 'Hello World' to Building Your Own AI Assistant

ABOUT THE PROGRAMME

This exciting program transforms curious kids into confident programmers and AI creators. Designed specifically for young minds (ages 8-16), this 12-month journey makes learning Python and AI fun, engaging, and practical. Through games, animations, robots, and real AI projects, students will master programming concepts while building incredible projects they can show off to friends and family. By the end, they'll have created 30+ projects including games, chatbots, image recognition apps, and even their own AI assistant.

WHAT MAKES THIS COURSE DIFFERENT

- Kid-friendly approach with visual learning and gamification
- No prerequisites - starts from absolute zero
- Project-based learning with fun, engaging activities
- Age-appropriate AI concepts explained simply
- Parent guidance notes included

- Safe, moderated online community for young learners
- Regular showcases to present projects
- Certificates and badges for motivation

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Python Basics, Turtle Graphics, Simple Games
 Phase 2: Creative Coding (Months 4-6): Advanced Games, Animations, Mini Projects
 Phase 3: AI Introduction (Months 7-9): Machine Learning Basics, AI Tools, Smart Programs
 Phase 4: AI Projects (Months 10-12): Computer Vision, Chatbots, Personal AI Assistant

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic reading and math skills (Grade 3+)
 Coding experience: None required - complete beginner friendly
 Equipment: Computer/laptop with internet
 Time commitment: 5-8 hours per week
 English: Basic English understanding
 Motivation: Curiosity and excitement to learn

WHO THIS COURSE IS FOR

Students: Kids aged 8-16 who want to learn coding
 Working professionals: Parents wanting to learn with kids
 Entrepreneurs: Young entrepreneurs with app ideas
 Freelancers: Teen freelancers building skills
 Kids: Any child interested in technology
 Anyone: Families learning together

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: PYTHON BASICS & FUN WITH CODE (MONTHS 1-3, WEEKS 1-13) -----

Start your coding adventure! Learn Python basics through fun projects, colorful graphics, and exciting games.

Months 1-2: Python Fundamentals & First Programs (Week 1-8)

Welcome to Programming & Setup

Topics:

- What is programming? How do computers understand us?
- Why Python? The language that powers YouTube, Instagram, and AI
- Installing Python - making your computer ready to code
- Using IDLE - Python's friendly coding environment
- Your first program: Making Python say 'Hello World!'
- Using print() to display messages
- Creating colorful text outputs
- Comments - leaving notes in your code
- Making Python do math - calculator programs
- Common errors and how to fix them
- Saving and running Python files
- Organizing your coding projects

Projects built:

- Personalized greeting program
- Magic calculator with special messages
- Story generator with your name

Practice: Daily: Create 3 different print programs, solve 5 math problems with Python

Variables & Data Types - Teaching Python to Remember

Topics:

- Variables - giving names to information
- Choosing good variable names
- Numbers in Python: integers and decimals
- Text in Python: strings and how to use them
- True or False: boolean values
- Changing variable values
- Getting input from users with input()
- Converting between different types
- Fun with strings: uppercase, lowercase, length
- String concatenation - joining words together
- Special characters and escape sequences
- Creating Mad Libs games with variables

Projects built:

- Interactive story with user input
- Age calculator with birthday messages
- Mad Libs game generator
- Personal quiz about yourself

Practice: Create 5 programs using different variable types

Making Decisions with If-Else

Topics:

- If statements - teaching Python to make choices
- Comparison operators: ==, !=, <, >, <=, >=
- Else statements - what to do otherwise
- Elif - checking multiple conditions
- Combining conditions with and, or, not
- Nested if statements - decisions within decisions
- Creating quiz games with scoring
- Password checker programs
- Age-appropriate content filters
- Random module - adding surprises to programs
- Making a magic 8-ball program
- Building simple AI decision makers

Projects built:

- Adventure game with choices
- Grade calculator with feedback
- Virtual pet mood checker
- Rock, Paper, Scissors game

Practice: Build 5 decision-making programs

Loops - Making Python Repeat

Topics:

- For loops - counting and repeating
- Range() function - controlling loop counts
- While loops - repeat until something happens
- Breaking out of loops with break
- Skipping with continue
- Nested loops - loops inside loops
- Creating patterns with loops
- Drawing ASCII art with loops
- Countdown and timer programs
- Multiplication table generator
- Password attempt limiters
- Animation basics with loops and time.sleep()

Projects built:

- Pattern art generator
- Guessing game with attempts
- Countdown timer with effects
- ASCII art animation

Practice: Create 10 different loop patterns and animations

Month 3: Turtle Graphics & Visual Programming (Week 9-13)

Introduction to Turtle Graphics

Topics:

- What is Turtle Graphics? Your drawing robot
- Setting up the turtle screen
- Basic movements: forward, backward, left, right
- Drawing shapes: squares, triangles, circles
- Changing turtle appearance and speed
- Colors in turtle: pen color and fill color
- Lifting and putting down the pen
- Going to specific positions
- Drawing with coordinates
- Creating custom shapes
- Saving turtle drawings
- Multiple turtles on screen

Projects built:

- Geometric art generator
- House drawing program
- Rainbow spiral creator
- Name writer in turtle

Practice: Draw 10 different shapes and patterns

Advanced Turtle Projects

Topics:

- Using loops with turtle for complex patterns
- Creating functions for repeated shapes
- Building a drawing app with keyboard control
- Mouse events in turtle
- Creating simple games with turtle
- Collision detection basics
- Score keeping in turtle games
- Creating mazes with turtle
- Animating turtle movements
- Fractal patterns with recursion basics
- Turtle race game
- Building a paint program

Projects built:

- Controllable turtle game
- Turtle race simulator

- Maze solver with turtle
- Digital spirograph

Practice: Create 5 interactive turtle programs

Phase 1 Showcase & Review

Topics:

- Reviewing Python basics
- Debugging common errors
- Code organization best practices
- Preparing projects for showcase
- Adding comments and documentation
- Creating a project portfolio

Projects built:

- CAPSTONE: Interactive Story Game with Turtle Graphics
- Combine all learned concepts into one big project
- Present to family and friends

Assessment: Phase 1 Quiz and Project Presentation

----- PHASE 2: CREATIVE CODING & GAME DEVELOPMENT (MONTHS 4-6, WEEKS 14-26) -----

Level up your Python skills! Create awesome games, work with data, and build useful applications.

Months 4-5: Lists, Functions & Advanced Concepts (Week 14-21)

Lists & Collections

Topics:

- Lists - storing multiple items
- Accessing list items with indexes
- Adding items: append, insert
- Removing items: remove, pop, clear
- List slicing - getting parts of lists
- Sorting and reversing lists
- Finding items in lists
- List comprehensions - the Python superpower
- 2D lists - lists inside lists
- Tuples - lists that can't change
- Sets - unique item collections
- Dictionaries - key-value pairs

Projects built:

- Todo list manager
- High score tracker
- Inventory system for games
- Contact book application

Practice: Build 5 programs using different list operations

Functions - Reusable Code Blocks

Topics:

- What are functions? Why use them?
- Creating simple functions with def
- Parameters - giving information to functions
- Return values - getting answers from functions
- Multiple parameters and default values
- Local vs global variables
- Function documentation with docstrings
- Lambda functions - tiny functions
- Recursion - functions calling themselves
- Building a function library
- Importing and using modules
- Creating your own modules

Projects built:

- Calculator with function menu
- Text adventure with function rooms
- Utility function collection
- Math game with scoring functions

Practice: Create 10 useful functions for future projects

File Handling & Data

Topics:

- Reading from text files
- Writing to text files
- Append mode vs write mode
- Working with CSV files
- Creating save files for games
- Loading game progress
- Simple encryption for files
- Creating a diary application
- Managing high scores
- Building a note-taking app
- File organization programs
- Backup utilities

Projects built:

- Personal diary with password
- Quiz game with saved scores
- File organizer utility
- Simple database system

Practice: Build 5 file-based applications

Object-Oriented Programming Basics

Topics:

- What are objects and classes?
- Creating your first class
- Attributes - what objects know
- Methods - what objects can do
- Creating multiple objects from one class
- The `__init__` method
- Self - referring to the object itself
- Class vs instance variables
- Inheritance - classes having children
- Making a pet class system
- Building game characters with OOP
- Real-world object modeling

Projects built:

- Virtual pet simulator
- RPG character creator
- School management system
- Zoo animal hierarchy

Practice: Create 5 different class systems

Error Handling & Debugging

Topics:

- Common Python errors and fixes
- Try-except blocks
- Handling specific errors
- Creating custom error messages
- Debugging with print statements
- Using Python debugger
- Testing your code
- Making programs user-friendly
- Input validation
- Creating help messages

Projects built:

- Unbreakable calculator
- Error-proof quiz game
- User-friendly file manager

Practice: Add error handling to previous projects

Month 6: Game Development with Pygame (Week 22-26)

Introduction to Pygame

Topics:

- What is Pygame? Making real games!
- Installing and setting up Pygame
- Creating game windows
- Game loop concept
- Drawing shapes and colors
- Loading and displaying images
- Sprite basics
- Moving objects on screen
- Keyboard input in games
- Mouse control in games
- Frame rate and smooth movement
- Coordinate system in games

Projects built:

- Bouncing ball animation
- Simple drawing program
- Moving character demo
- Click the target game

Practice: Create 5 simple Pygame demos

Building Complete Games

Topics:

- Collision detection in games
- Score keeping and lives
- Game states: menu, playing, game over
- Sound effects and music
- Creating game levels
- Power-ups and bonuses
- Enemy AI basics
- Particle effects
- High score saving
- Pause functionality
- Full screen mode
- Packaging games for sharing

Projects built:

- Snake game remake
- Space shooter game
- Platform jumper game
- Puzzle matching game

Practice: Add features to existing games

Advanced Game Features

Topics:

- Scrolling backgrounds
- Parallax effects
- Sprite animations
- Tile-based games
- Physics simulation basics
- Gravity and jumping
- Multiple levels design
- Boss battles
- Achievements system
- Game balancing
- Multiplayer basics
- Publishing games online

Projects built:

- Multi-level platformer
- Racing game with AI
- Tower defense game
- 2-player battle game

Practice: Polish and enhance game projects

Creative Coding Projects

Topics:

- Music with Python
- Creating visualizations
- Generative art
- Data visualization basics
- Web scraping basics (kid-safe)
- Automation scripts
- Discord bot basics
- Creating GUIs with Tkinter
- Weather app with API
- URL shortener
- QR code generator
- Combining multiple libraries

Projects built:

- Music visualizer
- Weather dashboard
- Personal assistant bot
- Desktop widget application

Practice: Build 5 creative applications

Phase 2 Game Showcase

Topics:

- Polishing game projects
- Creating game trailers
- Writing game documentation
- Beta testing with friends
- Gathering feedback
- Publishing to itch.io

Projects built:

- CAPSTONE: Complete Original Game
- Design, develop, and polish an original game
- Include menu, levels, scoring, and sound

Assessment: Phase 2 Game Development Challenge

----- PHASE 3: INTRODUCTION TO AI & MACHINE LEARNING (MONTHS 7-9, WEEKS 27-39) -----

Enter the world of AI! Learn how computers can learn, recognize patterns, and make intelligent decisions.

Months 7-8: AI Fundamentals & Smart Programs (Week 27-34)

What is Artificial Intelligence?

Topics:

- AI in everyday life - Siri, Alexa, YouTube recommendations
- How do computers learn? Simple explanation
- Difference between programming and AI
- Types of AI: narrow vs general
- Machine Learning basics for kids
- Supervised vs unsupervised learning
- Training data concept
- Making predictions
- Pattern recognition
- AI ethics - using AI responsibly
- Famous AI applications

- Future of AI

Projects built:

- AI concept presentation
- Design your dream AI assistant
- AI in daily life tracker
- Simple pattern recognizer

Practice: Research and present 5 AI applications

Smart Decision Making Programs

Topics:

- Rule-based AI systems
- Decision trees explained
- Creating expert systems
- Tic-tac-toe AI opponent
- Game AI strategies
- Minimax algorithm (simplified)
- Pathfinding basics
- A* algorithm introduction
- Maze solving AI
- Simple recommendation systems
- Chatbot logic basics
- Natural language understanding basics

Projects built:

- Tic-tac-toe with smart AI
- Maze solver AI
- Movie recommendation system
- Simple medical diagnosis bot

Practice: Build 5 rule-based AI systems

Data Science for Kids

Topics:

- What is data? Why is it important?
- Collecting and organizing data
- NumPy basics - working with numbers
- Pandas basics - data tables
- Reading data from files
- Cleaning messy data
- Finding patterns in data
- Basic statistics: mean, median, mode
- Creating charts with Matplotlib
- Bar charts, pie charts, line graphs
- Making data tell stories
- Data visualization projects

Projects built:

- Class survey analyzer
- Weather data explorer
- Sports statistics dashboard
- Personal habit tracker

Practice: Analyze 5 different datasets

Introduction to Machine Learning

Topics:

- Scikit-learn for beginners
- What is classification?
- Training vs testing data
- Simple classifiers
- K-Nearest Neighbors explained
- Decision trees visualization
- Making predictions
- Accuracy and evaluation
- Overfitting explained simply
- Feature selection basics
- Iris flower classification
- Digit recognition basics

Projects built:

- Flower species classifier
- Fruit identifier
- Simple spam detector
- Weather predictor

Practice: Train 5 different ML models

Neural Networks Basics

Topics:

- What are neural networks?
- Neurons and connections
- How neural networks learn
- Simple neural network demo
- Image recognition basics
- Training neural networks
- TensorFlow Playground
- Visualizing neural networks
- Common applications
- Deep learning introduction

Projects built:

- Handwritten digit recognizer
- Simple image classifier
- Neural network visualizer
- Pattern learning demo

Practice: Experiment with neural network parameters

Month 9: Computer Vision & Image AI (Week 35-39)

Computer Vision Basics

Topics:

- How computers see images
- Pixels and image data
- OpenCV introduction
- Loading and displaying images
- Image manipulation: resize, rotate, flip
- Color spaces: RGB, grayscale
- Image filters and effects
- Edge detection
- Shape detection
- Color detection
- Face detection with OpenCV
- Drawing on images

Projects built:

- Photo editor with filters
- Face detection app
- Color tracker game
- Shape identifier

Practice: Create 5 image processing programs

Advanced Computer Vision

Topics:

- Object detection basics
- Pre-trained models
- YOLO introduction (simplified)
- Detecting common objects
- Pose detection
- Hand tracking
- Facial landmarks
- Emotion detection
- Augmented reality basics
- Virtual backgrounds
- Real-time video processing
- Creating Instagram-like filters

Projects built:

- Object counting app
- Virtual background changer
- Gesture control game
- Emotion detector

Practice: Build 5 computer vision applications

Image Recognition Projects

Topics:

- Training custom image classifiers
- Teachable Machine by Google
- Transfer learning basics
- Creating image datasets
- Data augmentation
- Building a pet breed classifier
- Food recognition app
- Plant identification system
- Sign language recognizer
- Custom gesture recognition
- Deploying image models
- Creating mobile-friendly AI

Projects built:

- Pet breed identifier
- Food calorie estimator
- Sign language translator
- Custom object detector

Practice: Train 5 custom image models

Creative AI Art

Topics:

- AI art generation basics
- Style transfer introduction
- Creating artistic filters
- Neural style transfer
- Deep Dream basics
- Generating patterns with AI
- AI color palettes
- Image enhancement with AI
- Creating photo mosaics

- AI-powered animations
- Combining multiple AI techniques

Projects built:

- AI art generator
- Style transfer app
- Photo mosaic creator
- AI animation tool

Practice: Create 10 AI artworks

Phase 3 AI Exhibition

Topics:

- Preparing AI demos
- Creating project presentations
- Explaining AI to others
- Showcasing projects
- Getting feedback
- AI project portfolio

Projects built:

- CAPSTONE: AI-Powered Game or Application
- Combine gaming with AI features
- Include computer vision or ML elements

Assessment: Phase 3 AI Challenge

PHASE 4: ADVANCED AI PROJECTS & FUTURE TECH (MONTHS 10-12, WEEKS 40-52)

Build amazing AI projects! Create chatbots, voice assistants, and your own AI-powered applications.

Months 10-11: Natural Language & Chatbots (Week 40-47)

Natural Language Processing Basics

Topics:

- How computers understand language
- Text processing basics
- Tokenization - breaking text into words
- Removing stop words
- Word frequency analysis
- Sentiment analysis basics
- Positive vs negative detection
- Text classification
- Language detection
- Text generation basics
- Working with NLTK
- SpaCy for kids

Projects built:

- Sentiment analyzer for reviews
- Text summarizer
- Language detector
- Word cloud generator

Practice: Build 5 text analysis tools

Building Chatbots

Topics:

- Chatbot architecture
- Rule-based chatbots
- Intent recognition
- Entity extraction
- Response generation
- ChatterBot library
- Training chatbots
- Context management
- Personality in chatbots
- Multi-turn conversations
- Integrating APIs in chatbots
- Deploying chatbots

Projects built:

- Personal assistant chatbot
- Study buddy bot
- Customer service bot
- Storytelling chatbot

Practice: Create 5 different chatbots

Voice Assistants & Speech

Topics:

- Speech recognition basics
- Converting speech to text
- Text to speech synthesis
- Creating voice commands
- Wake word detection
- Building a mini Alexa
- Voice-controlled games
- Language translation

- Voice emotions
- Accents and languages
- Audio processing basics
- Creating voice filters

Projects built:

- Voice-controlled calculator
- Personal voice assistant
- Voice diary application
- Language learning helper

Practice: Build 5 voice-enabled applications

AI APIs & Cloud Services

Topics:

- Using Google Cloud AI
- Microsoft Azure Cognitive Services
- IBM Watson for kids
- OpenAI API basics (age-appropriate)
- Translation APIs
- Vision APIs
- Speech APIs
- Weather and data APIs
- Combining multiple APIs
- API keys and security
- Rate limiting
- Creating API mashups

Projects built:

- Multi-language translator
- AI news reader
- Smart weather reporter
- API-powered game

Practice: Integrate 5 different AI APIs

Internet of Things (IoT) Basics

Topics:

- What is IoT?
- Raspberry Pi introduction
- Micro:bit basics
- Sensors and actuators
- Reading sensor data
- Controlling LEDs
- Temperature monitoring
- Motion detection
- Smart home basics
- IoT and AI combination
- Cloud IoT platforms
- Safety with IoT

Projects built:

- Smart plant watering system
- Room temperature monitor
- Motion-activated alarm
- Mini weather station

Practice: Build 3 IoT projects

Month 11: Robotics & Advanced Projects (Week 48-51)

Robotics with Python

Topics:

- Introduction to robotics
- Robot components: sensors, motors, controllers
- Programming robot movements
- Line following robots
- Obstacle avoidance
- Remote control robots
- Robot simulation environments
- Path planning for robots
- Computer vision in robotics
- Voice control for robots
- Multi-robot coordination
- Robot competitions

Projects built:

- Virtual robot simulator
- Maze-solving robot
- Drawing robot
- Voice-controlled robot

Practice: Program 5 robot behaviors

Game AI & Reinforcement Learning

Topics:

- How AI learns to play games
- Reinforcement learning basics
- Rewards and penalties
- Q-learning introduction
- Training AI to play simple games

- OpenAI Gym basics
- Creating game environments
- AI vs human gameplay
- Balancing game difficulty
- Procedural content generation
- AI for game testing
- Future of gaming AI

Projects built:

- AI learns Tic-Tac-Toe
- Flappy Bird AI
- Maze learning agent
- AI game opponent

Practice: Train AI for 5 different games

Data Visualization & Dashboards

Topics:

- Advanced data visualization
- Interactive plots with Plotly
- Creating dashboards
- Real-time data visualization
- Geographic data and maps
- 3D visualizations
- Animation in data viz
- Streamlit for web apps
- Creating data stories
- Infographics with Python
- Dashboard design principles
- Sharing visualizations online

Projects built:

- COVID data dashboard
- Sports analytics dashboard
- Personal fitness tracker
- School performance visualizer

Practice: Create 5 interactive dashboards

Web Development & AI Integration

Topics:

- Web basics: HTML, CSS review
- Flask web framework basics
- Creating web interfaces for AI
- Deploying ML models online
- Building AI web apps
- REST APIs for AI
- Real-time predictions
- User authentication basics
- Database basics for web apps
- Hosting on Replit
- Creating AI demos
- Portfolio website creation

Projects built:

- AI-powered web app
- Online image classifier
- Web-based chatbot
- Portfolio website with projects

Practice: Deploy 5 AI projects online

Mobile Apps & AI

Topics:

- Mobile app basics
- Kivy framework introduction
- Creating mobile UIs
- AI on mobile devices
- TensorFlow Lite basics
- Offline AI apps
- Camera integration
- Mobile sensors usage
- Publishing apps basics
- App monetization basics
- User privacy and safety

Projects built:

- Mobile AI camera app
- Offline translator app
- Mobile game with AI
- Personal AI assistant app

Practice: Create 3 mobile AI apps

Month 12: Final Projects & Future Path (Week 52)

Ethical AI & Future Technologies

Topics:

- AI ethics for young developers
- Bias in AI systems
- Privacy and data protection

- AI safety basics
- Environmental impact of AI
- AI for social good
- Future careers in AI
- Quantum computing basics
- Blockchain basics for kids
- Augmented Reality future
- Space technology and AI
- Creating positive AI impact

Projects built:

- AI for social good project
- Bias detection tool
- Privacy-preserving AI app
- Future technology presentation

Practice: Research and present on AI ethics

Advanced AI Topics

Topics:

- Generative AI basics
- GANs introduction (simplified)
- AI music generation
- AI story writing
- Deepfakes awareness
- AI in medicine
- AI in climate change
- AI in space exploration
- Quantum AI basics
- Brain-computer interfaces
- Future of AI predictions

Projects built:

- AI music composer
- Story generator with AI
- Future AI concept design
- AI research presentation

Practice: Explore 5 cutting-edge AI topics

Hackathon Preparation

Topics:

- What are hackathons?
- Team collaboration
- Rapid prototyping
- Pitching ideas
- Time management
- Using Git for teams
- Documentation skills
- Presentation skills
- Demo preparation
- Handling pressure
- Learning from failure
- Networking basics

Projects built:

- 24-hour hackathon project
- Team collaboration project
- Speed coding challenges
- Pitch deck creation

Practice: Participate in online hackathon

Open Source & Community

Topics:

- What is open source?
- Contributing to projects
- GitHub for collaboration
- Creating documentation
- Issue tracking
- Code reviews basics
- Creating your first package
- PyPI basics
- Building coding community
- Teaching others to code
- Starting a coding club
- Continuing education

Projects built:

- Open source contribution
- Create Python package
- Tutorial creation
- Coding club project

Practice: Contribute to 3 open source projects

Final Month: Capstone & Graduation (Week 52)

Final Capstone Project - Planning

Topics:

- Choosing capstone topic

- Project requirements analysis
- Creating project timeline
- Technology stack selection
- UI/UX design basics
- Database design
- API planning
- Testing strategy
- Documentation planning
- Setting up development environment
- Version control setup
- Project management tools

Projects built:

- FINAL CAPSTONE: Complete AI-Powered Application
- Requirements: Original idea, AI/ML integration, database, web/mobile interface, documentation
- Options: AI tutor, Smart home assistant, Game with AI, Social good application, Educational tool

Final Capstone Project - Development

Topics:

- Agile development basics
- Daily progress tracking
- Feature implementation
- Testing and debugging
- Performance optimization
- Security considerations
- User testing
- Feedback incorporation
- Final polishing
- Deployment preparation
- Creating user manual
- Video demo creation

Deliverables:

- Complete source code
- Live demonstration
- Project documentation
- Video presentation
- Technical report
- User manual
- Test results
- Future roadmap

Showcase & Future Journey

Topics:

- Project presentation skills
- Creating portfolio website
- Resume for young coders
- Competition preparation
- Advanced learning paths
- University preparation
- Internship opportunities
- Building online presence
- Joining developer communities
- Mentoring younger students
- Planning next projects
- Lifelong learning mindset

Deliverables:

- Portfolio website
- LinkedIn profile (if age appropriate)
- GitHub portfolio
- Competition entries
- Future learning plan
- Peer teaching session
- Thank you project for parents/teachers

Assessment: FINAL SHOWCASE - Present to family, friends, and community

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Calculator, Turtle Art, Mad Libs, Guessing Games, ASCII Animations
- Phase 2: Snake Game, Platformer, Tower Defense, Music Visualizer, Discord Bot
- Phase 3: Face Detector, Object Classifier, Emotion Recognizer, AI Art Generator
- Phase 4: Chatbot, Voice Assistant, Robot Simulator, AI Web App
- Final: Complete AI-Powered Application with real-world use

Total projects: 30+ projects from simple to advanced

SKILLS THE STUDENT WILL HAVE AT THE END

- Programming: Python, Object-Oriented Programming, Functional Programming
- Game Development: Pygame, Game Design, AI Opponents, Physics
- Data Science: NumPy, Pandas, Matplotlib, Data Analysis, Visualization
- Machine Learning: Scikit-learn, Classification, Neural Networks, Deep Learning

- Computer Vision: OpenCV, Image Processing, Object Detection, Face Recognition
- Natural Language: Text Processing, Sentiment Analysis, Chatbots, Voice Recognition
- Robotics: Robot Programming, Sensors, Path Planning, Automation
- Web Development: Flask, APIs, Deployment, Databases
- Tools: Git, VS Code, Jupyter, Google Colab, Command Line
- Soft Skills: Problem Solving, Debugging, Project Planning, Presentation

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 2-3 hours
 Hands on coding: 3-4 hours
 Projects: 2-3 hours
 Practice problems: 1-2 hours
 Total per week: 5-8 hours

SUPPORT INCLUDED

Live sessions: Weekly fun coding sessions
 Mentorship: Dedicated mentor for guidance
 Community: Safe, moderated community for kids
 Code review: Friendly feedback on projects
 Career support: Guidance for future in tech
 Lifetime access: Keep learning and reviewing

CERTIFICATION DETAIL

Phase certificates: Badge after each phase
 Final certificate: Young Python & AI Developer Certificate
 LinkedIn badge: Digital badge for profiles
 Industry recognized: Recognized by schools and competitions
 Portfolio projects: 30+ projects to showcase

WHERE THIS CAN LEAD

- Continue to advanced programming courses
- Participate in coding competitions
- Join robotics clubs and teams
- Create apps and games
- Start a YouTube channel teaching coding
- Contribute to open source projects
- Prepare for computer science studies
- Build AI projects for science fairs
- Become a young coding mentor
- Launch a tech startup

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What age group is this Python & AI course designed for?

A: This course is specifically designed for children ages 8-16. The curriculum adapts to different age levels: younger kids (8-11) focus more on visual programming with Turtle Graphics and game development, while older kids (12-16) dive deeper into AI concepts, machine learning, and advanced Python projects.

Q: Does my child need any prior coding experience?

A: No prior coding experience is required. This course starts from absolute zero with fun 'Hello World' programs and gradually builds up to AI projects. The teaching approach is specifically designed for young minds with visual learning, gamification, and engaging projects.

Q: What will my child be able to build after completing this course?

A: These are real, working projects they can show to friends, family, and include in college applications.

Q: How does this prepare my child for future STEM careers?

A: This course builds strong computational thinking, problem-solving skills, and familiarity with AI/ML concepts that are increasingly important in all STEM fields. Children gain a significant head start for computer science studies, coding competitions, robotics clubs, and future tech careers.

Q: Is the online environment safe for children?

A: Yes, all classes are conducted in safe, moderated environments. We provide parent guidance notes, regular progress updates, and age-appropriate content. The online community is strictly monitored, and we use child-safe platforms with appropriate privacy controls.

Q: How much time does my child need to dedicate weekly?

A: We recommend 5-8 hours per week, which includes 2 live classes plus practice time. The flexible structure allows kids to learn at their own pace, and the gamified approach with

achievements and badges keeps them motivated without feeling overwhelmed.

To enrol or to book a free demo for this course, message Shivam Sir on
+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/python-ai-kids-masterclass>

=====

COURSE 102 OF 118: PYTHON PROGRAMMING FOR TEENS: ZERO TO ADVANCED IN 2 YEARS

=====

Course link: <https://learn.modernagecoders.com/courses/python-complete-masterclass-teens>
Subject area: Complete Python Programming
Designed for: Teens
Who can join: Absolute Beginner to Python Expert
Course length: 24 months (104 weeks)
Time needed: 12-15 hours/week recommended
Certificate: Python Expert Certification with 100+ Project Portfolio

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 2-year Python course for teens: go from your first line of code to advanced Python, web development, games, AI/ML, automation and 100+ real-world projects.

In one line: From 'Hello World' to Building Instagram Clones, AI Bots, Games, and Your Own Python Framework

ABOUT THE PROGRAMME

This isn't just another Python course, it's a complete transformation into a Python wizard. Over 2 years, you'll master every aspect of Python, from basic syntax to advanced frameworks, from simple scripts to complex applications. Build games like Minecraft clones, create social media platforms, develop AI chatbots, automate everything in your life, hack (ethically!), create mobile apps, and even contribute to Python itself. Designed specifically for teenage minds with projects you'll love: Discord bots, Instagram automation, game cheats (for your own games!), homework helpers, music players, and even your own programming language. By graduation, you'll have built 100+ projects and be ready for any Python challenge.

WHAT MAKES THIS COURSE DIFFERENT

- Teen-focused projects: Games, social media, automation teens actually want
- Visual learning with animations and interactive coding
- Build clones of apps teens use daily
- From zero to professional in structured path
- Cover EVERY major Python library and framework
- Real hacking and security (ethical only!)
- Create your own Python packages
- Job-ready skills with internship preparation

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-6): Python basics, OOP, data structures, algorithms
Phase 2: Web & Apps (Months 7-12): Django, Flask, APIs, databases, full-stack
Phase 3: Specializations (Months 13-18): Games, ML/AI, automation, data science
Phase 4: Advanced (Months 19-24): Systems, security, mobile, contributing to Python

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic computer literacy
Coding experience: None required - absolute beginners welcome
Equipment: Computer with internet connection
Time commitment: 12-15 hours per week
Age: 13-19 years (teen-optimized content)
Motivation: Passion for technology and creating

WHO THIS COURSE IS FOR

Complete beginners: Never coded before? Perfect starting point
Young creators: Want to build games, apps, websites
Future engineers: Preparing for tech careers
Problem solvers: Love automating and solving problems
Entrepreneurs: Want to build own products
Hackers: Interested in cybersecurity (ethical)
Data enthusiasts: Fascinated by data and AI

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PYTHON FOUNDATIONS & PROGRAMMING LOGIC (MONTHS 1-6, WEEKS 1-26)

Master Python fundamentals, programming logic, data structures, algorithms, and object-oriented programming.

Months 1-2: Python Basics & Programming Fundamentals (Week 1-8)

Introduction to Python & Setup

Topics:

- Why Python? The language that runs Instagram, YouTube, and NASA
- Installing Python 3.12+ and understanding versions
- VS Code setup with Python extensions
- PyCharm IDE for professional development
- Terminal/Command line mastery
- Understanding Python's philosophy: The Zen of Python
- Variables and memory: How Python stores data
- Data types deep dive: int, float, str, bool, None
- Type hints and why they matter
- Input/Output: Building interactive programs
- Comments and documentation: Writing readable code
- Python's hidden superpowers: What makes it special

Projects built:

- Personal assistant chatbot
- Calculator with memory
- Mad libs story generator
- Temperature converter with GUI preview

Practice: Daily coding challenges, 20 mini-programs

Control Flow & Logic

Topics:

- If-elif-else: Teaching computers to make decisions
- Boolean logic: and, or, not, and De Morgan's laws
- Comparison operators and chaining
- Match-case statements (Python 3.10+)
- Ternary operators for elegant code
- Truthy and falsy values in Python
- Short-circuit evaluation
- Exception handling basics: try, except, finally
- Custom exceptions for better errors
- Assertions for debugging
- The walrus operator := for elegant conditions
- Control flow visualization tools

Projects built:

- Rock-Paper-Scissors-Lizard-Spock game
- Password strength analyzer with rules
- Quiz game with scoring system
- Decision tree adventure game

Practice: 30 logic puzzles, build 10 decision-based programs

Loops & Iterations

Topics:

- For loops: Automating repetitive tasks
- While loops: Conditional repetition
- Nested loops and performance implications
- Loop control: break, continue, else with loops
- Range() function secrets and memory efficiency
- Enumerate() for index tracking
- Zip() for parallel iteration
- List comprehensions: Python's superpower
- Generator expressions for memory efficiency
- Itertools module: Advanced iteration patterns
- Creating custom iterators
- Performance: When to use which loop

Projects built:

- Pattern generator (ASCII art)
- Prime number visualizer
- Multiplication table game
- Progress bar implementation

Practice: 50 loop challenges, optimize 10 programs

Functions & Functional Programming

Topics:

- Functions: Building blocks of programs
- Parameters vs arguments: The complete guide
- Return values and multiple returns
- Default arguments and keyword arguments
- *args and **kwargs: Flexible functions
- Scope and LEGB rule
- Closures and their power
- Decorators: Modifying function behavior

- Lambda functions and when to use them
- Map, filter, reduce: Functional programming trio
- Partial functions and currying
- Function annotations and type hints

Projects built:

- Homework helper library
- Unit converter with 50+ conversions
- Function benchmark tool
- Decorator collection for common tasks

Practice: Build 40 reusable functions, master 10 decorators

Months 3-4: Data Structures & Collections (Week 9-17)

Lists & Tuples Mastery

Topics:

- Lists: Python's Swiss Army knife
- List methods: All 11 methods explained
- Slicing notation mastery [start:stop:step]
- List comprehensions with conditions
- Nested list comprehensions
- 2D and 3D lists (matrices)
- Tuples: Immutable sequences
- Named tuples for readable code
- List vs Tuple: Performance and use cases
- Unpacking and starred expressions
- Deep vs shallow copy issues
- Memory optimization techniques

Projects built:

- Spotify playlist manager
- Tic-tac-toe with AI opponent
- Matrix calculator
- Todo list with priorities

Practice: 100 list/tuple problems, build 5 data managers

Dictionaries & Sets

Topics:

- Dictionaries: Key-value mastery
- Dictionary methods and operations
- Dictionary comprehensions
- Nested dictionaries and JSON
- OrderedDict, defaultdict, Counter
- ChainMap for multiple dictionaries
- Sets: Unique collections
- Set operations: union, intersection, difference
- Frozen sets for immutable sets
- When to use sets vs lists
- Hash tables: How dictionaries work internally
- Performance characteristics and Big O

Projects built:

- Contact manager with search
- Word frequency analyzer
- Inventory system for game
- Social network graph analyzer

Practice: 75 dict/set challenges, optimize 10 programs

Strings & Text Processing

Topics:

- Strings: More than just text
- String methods: All 47 methods
- F-strings: Modern string formatting
- Regular expressions with re module
- Unicode and encoding: UTF-8, ASCII
- String performance and internment
- Text processing techniques
- String algorithms: searching, matching
- Template strings for safe substitution
- Textwrap for formatting
- Difflib for comparing texts
- Natural language basics

Projects built:

- Wordle game clone
- Text encryption tool
- Markdown to HTML converter
- Plagiarism checker

Practice: 60 string problems, build 5 text processors

Advanced Data Structures

Topics:

- Collections module deep dive
- Deque: Double-ended queue
- Heapq: Priority queues in Python
- Bisect: Maintaining sorted lists
- Array module for efficient storage

- Dataclasses for structured data
- Enum for constants
- Queue and stack implementations
- Linked lists in Python
- Trees and graphs basics
- Custom data structure creation
- Performance profiling with cProfile

Projects built:

- LRU cache implementation
- Task scheduler with priorities
- File system simulator
- Huffman encoding compressor

Practice: Implement 10 data structures from scratch

Algorithms & Problem Solving

Topics:

- Algorithm complexity and Big O
- Searching algorithms: Binary, jump, interpolation
- Sorting algorithms: Quick, merge, heap, radix
- Recursion and memoization
- Dynamic programming basics
- Greedy algorithms
- Graph algorithms: BFS, DFS, Dijkstra
- Backtracking problems
- Algorithm visualization
- Problem-solving strategies
- Competitive programming intro
- LeetCode and HackerRank strategies

Projects built:

- Sorting visualizer with Pygame
- Pathfinding algorithm demo
- Sudoku solver with GUI
- Algorithm complexity analyzer

Practice: Solve 50 algorithm problems, implement 15 algorithms

Months 5-6: Object-Oriented Programming & Design Patterns (Week 18-26)

OOP Fundamentals

Topics:

- Classes and objects: Building blueprints
- Attributes and methods
- `__init__` and object initialization
- `self`: The magic first parameter
- Class vs instance attributes
- Property decorators for getters/setters
- Method types: instance, class, static
- `__str__` and `__repr__` for object representation
- Encapsulation and information hiding
- Name mangling with double underscore
- Documentation with docstrings
- Type hints in classes

Projects built:

- Bank account system
- RPG character creator
- School management system
- E-commerce cart system

Practice: Create 20 classes, build 5 OOP projects

Inheritance & Polymorphism

Topics:

- Inheritance: Building class hierarchies
- `super()` and method resolution order (MRO)
- Multiple inheritance and mixins
- Abstract base classes (ABC)
- Polymorphism and duck typing
- Method overriding and extending
- Composition vs inheritance
- Interface design in Python
- SOLID principles in Python
- Dependency injection
- The Diamond Problem
- Metaclasses introduction

Projects built:

- Game entity system
- Shape hierarchy with graphics
- Plugin architecture system
- Employee management hierarchy

Practice: Design 10 inheritance hierarchies

Magic Methods & Protocols

Topics:

- Magic methods (dunder methods) complete guide
- Operator overloading with magic methods

- `__new__` vs `__init__`
- Container protocols: `__len__`, `__getitem__`, `__setitem__`
- Iterator protocol: `__iter__`, `__next__`
- Context managers: `__enter__`, `__exit__`
- Arithmetic magic methods
- Comparison magic methods
- Callable objects with `__call__`
- Descriptors and their use
- Copy protocol: `__copy__`, `__deepcopy__`
- Pickling and serialization

Projects built:

- Custom list implementation
- Matrix class with operations
- Database connection manager
- Smart dictionary with validation

Practice: Implement 30 magic methods, create 5 protocols

Design Patterns

Topics:

- Design patterns: Proven solutions
- Creational patterns: Singleton, Factory, Builder
- Structural patterns: Adapter, Decorator, Facade
- Behavioral patterns: Observer, Strategy, Command
- MVC architecture in Python
- Repository pattern for data access
- Dependency injection containers
- Event-driven architecture
- Publish-subscribe pattern
- Chain of responsibility
- Python-specific patterns
- Anti-patterns to avoid

Projects built:

- Game engine with patterns
- Event system implementation
- Plugin-based text editor
- Design pattern library

Practice: Implement 15 design patterns

Phase 1 Capstone Project

Topics:

- Project architecture planning
- Requirements gathering
- UML diagrams for Python
- Test-driven development intro
- Documentation standards
- Code review preparation

Projects built:

- CAPSTONE: Social Media Platform Core
- User system, posts, comments, likes
- Full OOP implementation
- Design patterns applied
- Alternative: Game engine from scratch
- Alternative: Database engine in Python

Assessment: Phase 1 comprehensive exam + project defense

----- PHASE 2: WEB DEVELOPMENT & FULL-STACK PYTHON (MONTHS 7-12, WEEKS 27-52) -----

Master web development with Django and Flask, build APIs, work with databases, and create full-stack applications.

Months 7-8: Web Fundamentals & Flask (Week 27-35)

Web Development Basics

Topics:

- How the web works: HTTP, DNS, browsers
- HTML5 and CSS3 crash course
- JavaScript essentials for Python developers
- REST API principles and design
- JSON and data serialization
- Web security basics: XSS, CSRF, SQL injection
- Virtual environments with venv
- Package management with pip and poetry
- Environment variables and `.env` files
- Git and GitHub for web projects
- Deployment concepts
- Web development tools and browser DevTools

Projects built:

- Personal portfolio website
- API client for public APIs
- Web scraper for favorite site
- Static site generator

Practice: Build 10 web pages, consume 5 APIs
Flask Framework Mastery

Topics:

- Flask introduction: Micro but powerful
- Routes and view functions
- Templates with Jinja2
- Static files handling
- Request and response objects
- Forms with Flask-WTF
- Database with Flask-SQLAlchemy
- User authentication with Flask-Login
- Sessions and cookies
- Blueprints for large applications
- Flask extensions ecosystem
- RESTful API with Flask-RESTful

Projects built:

- Blog platform with Flask
- URL shortener service
- Todo API with authentication
- Real-time chat with Flask-SocketIO

Practice: Build 5 Flask applications

Database Programming

Topics:

- SQL fundamentals and advanced queries
- SQLite for development
- PostgreSQL for production
- SQLAlchemy ORM deep dive
- Database migrations with Alembic
- Query optimization and indexing
- NoSQL with MongoDB and PyMongo
- Redis for caching and sessions
- Database connection pooling
- Transactions and ACID properties
- Database security best practices
- Backup and recovery strategies

Projects built:

- Multi-user blog with comments
- E-commerce database design
- Analytics dashboard
- Database migration tool

Practice: Design 10 database schemas, optimize 20 queries

API Development

Topics:

- RESTful API design principles
- FastAPI: Modern API framework
- API versioning strategies
- Authentication: JWT, OAuth2, API keys
- Rate limiting and throttling
- API documentation with Swagger/OpenAPI
- GraphQL with Graphene
- WebSockets for real-time communication
- API testing with Postman and pytest
- Error handling and status codes
- CORS and security headers
- API deployment and monitoring

Projects built:

- Social media API clone
- Payment processing API
- Real-time notification system
- GraphQL API for blog

Practice: Build 8 different APIs

Frontend Integration

Topics:

- AJAX and fetch API
- Building SPAs with Python backend
- Vue.js or React with Python
- WebSockets for real-time features
- Server-sent events (SSE)
- File uploads and processing
- Image manipulation with Pillow
- PDF generation and processing
- Email sending with Python
- Task queues with Celery
- Background jobs and scheduling
- Caching strategies

Projects built:

- Real-time dashboard
- File sharing platform
- Image gallery with filters

- Email newsletter system

Practice: Integrate 5 frontend features

Months 9-10: Django & Advanced Web Development (Week 36-44)

Django Fundamentals

Topics:

- Django philosophy: Batteries included
- Project structure and apps
- Models and ORM
- Views: Function-based and class-based
- Templates and template inheritance
- URL routing and namespaces
- Forms and ModelForms
- Admin interface customization
- Static files and media handling
- Middleware and request processing
- Signals for decoupled apps
- Custom management commands

Projects built:

- Instagram clone with Django
- E-learning platform
- Project management tool
- News aggregator site

Practice: Build 5 Django applications

Django Advanced Features

Topics:

- Custom user models
- Authentication and permissions
- Django REST Framework mastery
- Serializers and viewsets
- Testing with Django
- Caching with Redis
- Async views in Django
- Channels for WebSockets
- GeoDjango for location services
- Django security best practices
- Performance optimization
- Django deployment strategies

Projects built:

- Multi-vendor marketplace
- Social network with real-time
- Location-based service
- API with complex permissions

Practice: Master 20 Django features

Full-Stack Development

Topics:

- Modern JavaScript for Python devs
- React or Vue.js integration
- State management concepts
- Building APIs for SPAs
- Authentication with JWT
- File and image uploads
- Real-time features with WebSockets
- Progressive Web Apps (PWA)
- SEO optimization
- Performance monitoring
- Error tracking with Sentry
- CI/CD pipelines

Projects built:

- Twitter clone full-stack
- Video streaming platform
- Collaborative editor
- Real-time gaming platform

Practice: Build 3 full-stack applications

Cloud & Deployment

Topics:

- Cloud platforms: AWS, Google Cloud, Azure
- Render for easy deployment
- Docker containerization
- Kubernetes basics
- Nginx and Gunicorn configuration
- SSL certificates and HTTPS
- Domain management and DNS
- Load balancing strategies
- Database hosting options
- CDN integration
- Monitoring and logging
- Backup and disaster recovery

Projects built:

- Deploy all previous projects
- Multi-region deployment
- Auto-scaling application
- Monitoring dashboard

Practice: Deploy 10 applications to cloud

Testing & Quality Assurance

Topics:

- Unit testing with pytest
- Test-driven development (TDD)
- Integration testing
- Mocking and patching
- Fixtures and factories
- Coverage reports
- End-to-end testing with Selenium
- API testing strategies
- Performance testing
- Security testing
- Continuous integration setup
- Code quality tools: Black, flake8, mypy

Projects built:

- Test suite for existing project
- TDD project from scratch
- Automated testing pipeline
- Quality assurance dashboard

Practice: Write 500+ tests, achieve 90% coverage

Months 11-12: Microservices & Advanced Architecture (Week 45-52)

Microservices Architecture

Topics:

- Monolith vs microservices
- Service decomposition
- Inter-service communication
- Message queues: RabbitMQ, Kafka
- Service discovery
- API Gateway pattern
- Circuit breakers and resilience
- Distributed tracing
- Event-driven architecture
- CQRS and Event Sourcing
- Saga pattern for transactions
- Service mesh with Istio

Projects built:

- E-commerce microservices
- Banking system with services
- Social media microservices
- Streaming platform architecture

Practice: Build 5 microservices

AsyncIO & Concurrency

Topics:

- AsyncIO fundamentals
- Async/await syntax mastery
- Coroutines and tasks
- Event loops deep dive
- Concurrent.futures module
- Threading vs multiprocessing
- GIL and its implications
- Async web frameworks: FastAPI, Aiohttp
- Async database drivers
- WebSocket servers with asyncio
- Performance comparison
- Debugging async code

Projects built:

- Async web scraper
- High-performance API
- Real-time data processor
- Concurrent download manager

Practice: Convert 10 sync programs to async

GraphQL & Modern APIs

Topics:

- GraphQL vs REST deep dive
- Schema definition language
- Resolvers and data loaders
- Mutations and subscriptions
- GraphQL with Django/Flask
- Apollo Server integration
- Real-time with GraphQL subscriptions
- GraphQL security
- Performance optimization
- Federation for microservices

- Testing GraphQL APIs
- GraphQL best practices

Projects built:

- GraphQL API for social media
- Real-time chat with GraphQL
- E-commerce GraphQL API
- GraphQL gateway service

Practice: Build 5 GraphQL APIs

DevOps for Python

Topics:

- Infrastructure as Code with Python
- Ansible automation
- Terraform with Python
- CI/CD with GitHub Actions
- GitLab CI/CD pipelines
- Jenkins automation with Python
- Monitoring with Prometheus
- Log aggregation with ELK
- Container orchestration
- Secrets management
- Blue-green deployments
- Chaos engineering basics

Projects built:

- CI/CD pipeline creation
- Infrastructure automation
- Monitoring system
- Deployment automation tool

Practice: Automate 10 DevOps tasks

Phase 2 Capstone Project

Topics:

- Large-scale application design
- Architecture documentation
- Performance requirements
- Security audit
- Deployment strategy
- User acceptance testing

Projects built:

- MAJOR CAPSTONE: Full Social Media Platform
- Complete with microservices, real-time, mobile app
- Alternative: E-commerce platform with everything
- Alternative: Learning management system

Assessment: Deploy production-ready application

PHASE 3: SPECIALIZATIONS & ADVANCED PYTHON (MONTHS 13-18, WEEKS 53-78)

Master specialized domains: game development, data science, automation, machine learning, and contribute to open source.

Months 13-14: Game Development & Graphics (Week 53-61)

Pygame Mastery

Topics:

- Pygame setup and game loop
- Sprites and sprite groups
- Collision detection algorithms
- Game physics implementation
- Particle systems
- Sound and music integration
- Game states and menus
- Save/load game systems
- Tile-based games
- Scrolling and cameras
- Optimization techniques
- Publishing games

Projects built:

- Platformer game like Mario
- Tower defense game
- RPG with inventory system
- Multiplayer snake game

Practice: Build 10 different game types

3D Graphics & Advanced Games

Topics:

- 3D graphics basics with Python
- Panda3D framework
- OpenGL with PyOpenGL
- 3D math: vectors, matrices, quaternions
- Lighting and shading
- Texture mapping
- 3D model loading

- First-person controls
- Procedural generation
- AI for games: pathfinding, behavior
- Networking for multiplayer
- Game engine architecture

Projects built:

- 3D maze game
- Minecraft clone basics
- Racing game
- FPS game prototype

Practice: Create 5 3D projects

Game AI & Procedural Generation

Topics:

- AI for NPCs: State machines, behavior trees
- Pathfinding: A*, Dijkstra, navigation mesh
- Decision making: Min-max, alpha-beta pruning
- Machine learning in games
- Procedural content generation
- Noise functions: Perlin, Simplex
- Dungeon generation algorithms
- Terrain generation
- Procedural textures
- Genetic algorithms for game balance
- Neural networks for game AI
- Reinforcement learning agents

Projects built:

- Chess AI opponent
- Procedural world generator
- AI battle arena
- Evolving game characters

Practice: Implement 10 AI algorithms

Mobile Game Development

Topics:

- Kivy framework for mobile
- Touch controls and gestures
- Mobile UI design
- Device sensors integration
- Building for Android
- iOS deployment basics
- Mobile game monetization
- In-app purchases
- Ad integration
- Push notifications
- Cloud saves
- Analytics integration

Projects built:

- Mobile puzzle game
- Endless runner for mobile
- Augmented reality game
- Location-based game

Practice: Deploy 5 games to mobile

Game Publishing & Monetization

Topics:

- Steam integration with Python
- Itch.io publishing
- Game marketing basics
- Creating game trailers
- Community management
- Update and patch systems
- Analytics and metrics
- Player retention strategies
- Multiplayer servers
- Anti-cheat systems
- Game modding support
- Building game tools

Projects built:

- Game launcher/updater
- Analytics dashboard
- Community platform
- Mod support system

Practice: Publish 3 complete games

Months 15-16: Data Science & Machine Learning (Week 62-70)

Data Science with Python

Topics:

- NumPy advanced: Broadcasting, vectorization
- Pandas mastery: MultiIndex, time series
- Data cleaning and preprocessing
- Exploratory data analysis (EDA)

- Statistical analysis with SciPy
- Hypothesis testing
- A/B testing implementation
- Feature engineering
- Dealing with missing data
- Outlier detection
- Data pipelines
- Big data with PySpark

Projects built:

- COVID-19 data analyzer
- Stock market analyzer
- Sports statistics platform
- Social media sentiment analysis

Practice: Analyze 20 real datasets

Data Visualization

Topics:

- Matplotlib advanced techniques
- Seaborn for statistical plots
- Plotly for interactive visualizations
- Dash for dashboard creation
- Bokeh for web visualizations
- Altair for declarative viz
- Network graphs with NetworkX
- Geospatial visualization
- 3D visualizations
- Animation creation
- Real-time data visualization
- Custom visualization design

Projects built:

- Interactive COVID dashboard
- Financial data dashboard
- Real-time monitoring system
- Data journalism project

Practice: Create 50 different visualizations

Machine Learning Fundamentals

Topics:

- Scikit-learn complete guide
- Supervised learning algorithms
- Unsupervised learning techniques
- Model evaluation and validation
- Cross-validation strategies
- Hyperparameter tuning
- Feature selection methods
- Ensemble methods
- Model interpretation
- ML pipelines
- Model deployment
- MLOps basics

Projects built:

- House price predictor
- Customer churn prediction
- Recommendation system
- Fraud detection system

Practice: Build 15 ML models

Deep Learning with Python

Topics:

- TensorFlow and Keras
- PyTorch fundamentals
- Neural network architectures
- CNNs for computer vision
- RNNs for sequences
- Transfer learning
- GANs introduction
- Natural Language Processing
- Computer vision projects
- Model optimization
- Deployment strategies
- Edge AI with Python

Projects built:

- Image classifier app
- Chatbot with NLP
- Face recognition system
- Style transfer application

Practice: Train 10 deep learning models

AI Applications

Topics:

- OpenAI API integration
- LangChain for LLM apps
- Computer vision with OpenCV

- Speech recognition
- Text-to-speech systems
- Reinforcement learning basics
- AI in games revisited
- Robotics with Python
- IoT and AI integration
- Ethical AI considerations
- AI project deployment
- Building AI products

Projects built:

- AI personal assistant
- Document analyzer with AI
- AI-powered game
- Smart home system

Practice: Build 8 AI applications

Months 17-18: Automation & Advanced Tools (Week 71-78)

Automation & Scripting

Topics:

- Task automation with Python
- File system automation
- Web scraping with BeautifulSoup
- Selenium for browser automation
- Email automation
- Excel automation with openpyxl
- PDF processing with PyPDF2
- Image processing automation
- Video processing with moviepy
- System administration scripts
- Cron jobs and scheduling
- Windows task automation

Projects built:

- Personal automation suite
- Web scraper for job listings
- Document processor
- Social media automator

Practice: Automate 30 daily tasks

GUI Development

Topics:

- Tkinter complete guide
- PyQt5/PySide2 for professional GUIs
- Kivy for modern interfaces
- CustomTkinter for beautiful apps
- GUI design principles
- Event-driven programming
- Threading in GUIs
- Database integration
- Packaging desktop apps
- Auto-updates for apps
- Cross-platform development
- Electron alternatives with Python

Projects built:

- Music player application
- Text editor with features
- Database management tool
- System monitoring app

Practice: Build 10 desktop applications

Network Programming

Topics:

- Socket programming deep dive
- TCP/IP with Python
- UDP programming
- HTTP servers from scratch
- WebSocket servers
- Network protocols implementation
- Port scanning and monitoring
- Packet sniffing with Scapy
- Network automation
- SSH automation with Paramiko
- FTP/SFTP clients
- Building networked games

Projects built:

- Chat server and client
- File transfer application
- Network monitor
- Custom protocol implementation

Practice: Build 8 network applications

Cybersecurity with Python

Topics:

- Ethical hacking with Python
- Penetration testing basics
- Cryptography with Python
- Password cracking (ethical)
- Network security tools
- Web application security
- Malware analysis basics
- Forensics with Python
- Security automation
- Vulnerability scanning
- Log analysis for security
- Building security tools

Projects built:

- Password manager
- Encryption tool
- Security scanner
- Log analyzer

Practice: Build 10 security tools

Phase 3 Capstone Project

Topics:

- Complex system integration
- Performance optimization
- Security hardening
- Documentation
- Testing strategies
- Deployment planning

Projects built:

- MAJOR PROJECT: Game Development Studio
- Complete game with AI, multiplayer, graphics
- Alternative: AI-Powered Analytics Platform
- Alternative: Automation Framework

Assessment: Present professional project

PHASE 4: EXPERT PYTHON & PROFESSIONAL DEVELOPMENT (MONTHS 19-24, WEEKS 79-104)

Master advanced Python internals, contribute to Python itself, build frameworks, and launch your career.

Months 19-20: Python Internals & Advanced Concepts (Week 79-87)

Python Internals

Topics:

- CPython architecture deep dive
- Python bytecode and dis module
- The Python Virtual Machine
- Memory management and garbage collection
- Reference counting and cycles
- The GIL: Understanding and working around it
- Python object model internals
- How imports really work
- Namespace and scope implementation
- Python's data model
- Building Python from source
- Contributing to CPython

Projects built:

- Bytecode analyzer
- Memory profiler
- Import system hook
- CPython contribution

Practice: Understand 20 Python internals

Metaprogramming & Advanced Features

Topics:

- Metaclasses deep dive
- Class decorators advanced
- Descriptors and properties
- Dynamic class creation
- Abstract Syntax Trees (AST)
- Code generation with Python
- Import hooks and finders
- Context managers advanced
- Weak references
- Slots for memory optimization
- Function introspection
- Frame objects and stack inspection

Projects built:

- ORM framework from scratch
- Dependency injection framework
- Code analyzer tool
- DSL implementation

Practice: Build 10 metaprogramming projects

Performance Optimization

Topics:

- Profiling with cProfile and line_profiler
- Memory profiling with memory_profiler
- Optimization strategies
- Cython for speed
- Numba for JIT compilation
- PyPy and alternative interpreters
- C extensions with ctypes and cffi
- Vectorization with NumPy
- Parallel processing strategies
- GPU acceleration with CuPy
- Rust extensions with PyO3
- Performance testing and benchmarking

Projects built:

- Performance optimization toolkit
- C extension for Python
- GPU-accelerated application
- Benchmark suite

Practice: Optimize 20 Python programs

Building Frameworks

Topics:

- Framework design principles
- Plugin architectures
- Configuration management
- CLI framework design
- Web framework from scratch
- Testing framework creation
- Documentation generation
- Package distribution
- Versioning strategies
- Backward compatibility
- API design best practices
- Framework documentation

Projects built:

- Web framework like Flask
- Testing framework like pytest
- CLI framework
- Game framework

Practice: Build 5 mini-frameworks

Compiler & Interpreter Design

Topics:

- Language design basics
- Lexical analysis (tokenization)
- Parsing and grammar
- Abstract Syntax Trees
- Semantic analysis
- Code generation
- Building an interpreter
- REPL implementation
- Error handling in compilers
- Optimization passes
- VM design basics
- DSL creation

Projects built:

- Mini programming language
- Template engine
- Query language parser
- Configuration language

Practice: Build 3 interpreters

Months 21-22: Industry & Enterprise Python (Week 88-96)

Enterprise Python

Topics:

- Enterprise architecture patterns
- Service-oriented architecture (SOA)
- Event-driven systems at scale
- Message brokers: RabbitMQ, Kafka advanced
- Distributed systems with Python
- Apache Airflow for workflows
- Data engineering with Python
- ETL pipelines
- Data warehousing
- Business intelligence tools
- Enterprise security
- Compliance and regulations

Projects built:

- Enterprise workflow system

- Data pipeline platform
- Business intelligence tool
- Compliance automation system

Practice: Build 5 enterprise solutions

Cloud Native Python

Topics:

- Kubernetes operators in Python
- Service mesh integration
- Cloud functions with Python
- Serverless architectures
- Multi-cloud strategies
- Infrastructure as Code advanced
- GitOps with Python
- Observability: metrics, logs, traces
- Chaos engineering implementation
- Cost optimization automation
- Cloud security automation
- Disaster recovery automation

Projects built:

- Kubernetes operator
- Serverless application
- Cloud cost optimizer
- Multi-cloud deployer

Practice: Deploy 10 cloud-native apps

IoT & Hardware

Topics:

- Raspberry Pi programming
- Arduino with Python
- MicroPython for embedded systems
- CircuitPython basics
- GPIO programming
- Sensor integration
- Motor control
- Computer vision on edge devices
- IoT protocols: MQTT, CoAP
- Home automation with Python
- Robotics frameworks: ROS
- Edge computing

Projects built:

- Smart home system
- Weather station
- Security camera system
- Robot car

Practice: Build 8 IoT projects

Scientific Python

Topics:

- Scientific computing with SciPy
- SymPy for symbolic math
- Astronomy with AstroPy
- Bioinformatics with BioPython
- Geospatial with GeoPandas
- Climate data analysis
- Signal processing
- Image processing advanced
- Simulations with SimPy
- Optimization problems
- Parallel scientific computing
- Research reproducibility

Projects built:

- Scientific calculator
- Data analysis tool
- Simulation platform
- Research automation

Practice: Solve 15 scientific problems

AI Agents & Automation with Python

Topics:

- Calling an AI API from Python
- Structured output with Pydantic
- Function calling: giving the model real tools
- The agent loop: plan, act, observe, repeat
- Retrieval over your own documents
- Vector search basics, and when it is overkill
- Guardrails: validating what comes back
- When the agent is confidently wrong
- Cost control: tokens, caching, spend caps
- Automating a real, boring task end to end
- Evaluating an agent: does it actually work?
- Cryptography deep dive: hashing and signatures

Projects built:

- Tool-using research agent
- Document Q&A over your own notes
- Automate a real weekly chore
- Agent evaluation harness

Practice: Build 5 AI automation projects

Month 23: Open Source & Community (Week 97-100)

Open Source Contribution

Topics:

- Open source ecosystem
- Finding projects to contribute
- Understanding large codebases
- Making your first PR
- Code review process
- Writing good commit messages
- Documentation contributions
- Bug triaging
- Feature development
- Maintaining projects
- Building community
- Open source licensing

Projects built:

- Contribute to 5 projects
- Create popular package
- Maintain open source project
- Build contributor community

Practice: Make 50 contributions

Teaching & Mentoring

Topics:

- Creating educational content
- Writing technical tutorials
- Video course creation
- Live coding streams
- Workshop development
- Mentoring strategies
- Code review skills
- Technical writing
- Documentation best practices
- Building learning platforms
- Community management
- Conference speaking

Projects built:

- Tutorial series
- Video course
- Workshop materials
- Learning platform

Practice: Teach 10 topics

Career Development

Topics:

- Python job market analysis
- Resume optimization
- Portfolio website creation
- LinkedIn optimization
- Interview preparation
- Coding interview strategies
- System design interviews
- Salary negotiation
- Freelancing with Python
- Starting a Python consultancy
- Building personal brand
- Networking strategies

Projects built:

- Professional portfolio
- Personal brand website
- Interview prep materials
- Freelance platform profile

Practice: Complete 20 interviews

Python Ecosystem Mastery

Topics:

- Package management advanced
- Virtual environment strategies
- Dependency management
- Python packaging best practices
- PyPI publishing
- Conda environments
- Docker for Python apps
- CI/CD for Python projects
- Documentation with Sphinx
- Testing strategies recap

- Code quality tools
- Python enhancement proposals (PEPs)

Projects built:

- Professional package
- CI/CD pipeline
- Documentation site
- Development toolkit

Practice: Master 20 tools

Month 24: Final Projects & Graduation (Week 101-104)

Final Capstone - Part 1

Topics:

- Project ideation and planning
- Market research
- Technical architecture
- Technology stack selection
- Development methodology
- Team collaboration
- Project management
- Risk assessment
- Timeline planning
- Resource allocation
- Documentation planning
- Testing strategy

Projects built:

- FINAL MASTERPIECE: Choose One
- 1. Full-scale SaaS platform
- 2. Open source framework/library
- 3. Game engine with editor
- 4. AI-powered application suite
- 5. Enterprise automation platform

Final Capstone - Part 2

Topics:

- Implementation and development
- Testing and debugging
- Performance optimization
- Security audit
- User testing
- Documentation completion
- Deployment preparation
- Marketing materials
- Launch preparation
- Community building
- Support systems
- Future roadmap

Deliverables:

- Complete source code
- Comprehensive documentation
- Test suite (95%+ coverage)
- Deployment guide
- User manual
- API documentation
- Video demonstration
- Marketing website
- Community platform
- Support system

Graduation & Beyond

Topics:

- Portfolio finalization
- Career path planning
- Job search strategies
- Interview preparation final
- Networking events
- Alumni network
- Continuous learning plan
- Specialization selection
- Advanced certifications
- Conference participation
- Speaking opportunities
- Future of Python

Deliverables:

- 100+ project portfolio
- Professional website
- GitHub profile (1000+ contributions)
- Published packages
- Blog with 50+ articles
- Video tutorials
- Open source contributions
- Professional network

- Job offers/opportunities
- Expert certification

Assessment: FINAL EVALUATION: Comprehensive assessment + Capstone presentation + Peer review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Games, chatbots, automation tools, data structures library
- Phase 2: Social media platform, e-commerce site, APIs, microservices
- Phase 3: 3D games, ML models, desktop apps, security tools
- Phase 4: Frameworks, compilers, enterprise systems, open source contributions
- Total: 100+ complete projects

Total projects: 100+ projects ranging from scripts to enterprise applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Python: Complete mastery of Python 3.12+
- Web Development: Django, Flask, FastAPI, full-stack development
- Data Science: NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow
- Game Development: Pygame, 3D graphics, game AI, mobile games
- Automation: Web scraping, task automation, system administration
- GUI Development: Tkinter, PyQt, Kivy, desktop applications
- Databases: SQL, NoSQL, ORMs, database design
- Cloud & DevOps: AWS, Docker, Kubernetes, CI/CD
- Testing: Unit testing, TDD, integration testing, pytest
- Security: Ethical hacking, cryptography, secure coding
- AI/ML: Machine learning, deep learning, computer vision, NLP
- Advanced: Metaprogramming, performance optimization, internals

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-5 hours
 Hands on coding: 6-7 hours
 Projects: 3-4 hours
 Practice problems: 2-3 hours
 Community interaction: 1 hour
 Total per week: 12-15 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding and Q&A
 Mentorship: Personal mentor throughout journey
 Code review: Expert review of all projects
 Community: Active Discord with 10,000+ members
 Job support: Resume review, interview prep, referrals
 Lifetime access: All content and future updates
 Cloud resources: Free credits for cloud platforms
 Hardware kits: IoT starter kits for hardware projects

CERTIFICATION DETAIL

Phase certificates: Certificate after each 6-month phase
 Specialization badges: 20+ skill badges to earn
 Final certificate: Python Expert Certification
 LinkedIn badges: Verifiable LinkedIn certifications
 Project portfolio: 100+ projects showcase
 Recommendation letters: From instructors and mentors

WHERE THIS CAN LEAD

- Python Developer (Entry to Senior level)
- Full-Stack Web Developer
- Data Scientist / Data Engineer
- Machine Learning Engineer
- Game Developer
- DevOps Engineer
- Security Engineer / Ethical Hacker
- Automation Engineer
- Mobile App Developer
- Open Source Maintainer
- Technical Educator / Content Creator
- Startup Founder / CTO

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build in this Python course?

A: Over the 2-year journey students build 100+ projects: games with Pygame, web applications with Flask and Django, automation scripts, data science and machine learning experiments, and advanced work with Python internals in the final phase. The projects are chosen to feel exciting to teens, like bots, games, and app clones, while quietly building professional skills.

Q: Is this the right level for my teenager?

A: The course is written for ages 13 to 19 and starts from absolute zero, so no prior coding is needed. Teens who already know some Python simply move through the early modules faster.

Q: Why is the course 2 years long?

A: Because it goes far beyond syntax: the four phases cover fundamentals and object-oriented programming, full web development, specializations like game development, data science, and automation, and finally expert-level Python. Students see real progress within the first month; the 2-year arc is what takes them from beginner to genuinely advanced.

Q: What software is needed, and is it free?

A: Everything is free: Python itself, the VS Code or PyCharm editors used in class, and open-source libraries like Pygame, Flask, and Django. Any reasonably modern computer with an internet connection can run the full course.

Q: Where does this lead after the course?

A: Python is the doorway language: graduates commonly continue into our AI and machine learning masterclass, or the data structures and algorithms course for competitive programming and interview-style problem solving. The project portfolio built here carries directly into college applications and tech pathways.

Q: Is a trial class available?

A: Yes, a free demo class with no commitment; book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to observe the demo and ask the teacher questions afterwards.

Q: How are the classes taught?

A: Through live 1-hour online sessions with a real teacher, never pre-recorded videos. Classes are interactive and small; students write and run code during class, not just watch someone else do it.

Q: Does the course include a certificate?

A: Yes, finishing the masterclass earns a Python certification backed by a portfolio of 100+ real projects. The portfolio matters most: it shows precisely what your teen can build, not just what they sat through.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/python-complete-masterclass-teens>

=====

COURSE 103 OF 118: PYTHON PROGRAMMING MASTERCLASS: ZERO TO ADVANCED PRO

=====

Course link: <https://learn.modernagecoders.com/courses/python-programming-masterclass-zero-to-advanced-college>

Subject area: Professional Python Programming

Designed for: College students

Who can join: Complete Beginner to Advanced Professional

Course length: 12 months (52 weeks)

Time needed: 12-15 hours/week recommended

Certificate: Industry-recognized Python Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

One-year Python programming course: master fundamentals, web development, data science, automation and AI/ML, from absolute basics to advanced skills.

In one line: From 'Hello World' to AI-Powered Production Applications

ABOUT THE PROGRAMME

This is not just a Python course, it's a complete career transformation program. Whether you're a curious 10-year-old, a student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled Python developer capable of building web applications, analyzing data, automating tasks, creating AI models, and solving real-world problems. You'll master Python from ground zero to advanced professional level: from basic syntax to complex algorithms, from simple scripts to production web applications, from data analysis to machine learning, from automation to cloud deployment. By the end, you'll have built 30+ projects, mastered multiple Python frameworks,

and be ready for professional developer roles across multiple domains.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - perfect for all ages (10+ years)
- Separate learning tracks for kids, teens, and adults
- 1 year of structured, progressive learning
- Covers Python for web dev, data science, automation, and AI
- Real industry projects and real-world applications
- Age-appropriate teaching methodologies
- Career guidance and certification
- Lifetime access and continuous updates
- Build impressive portfolio across multiple domains

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Programming Basics, Python Fundamentals, Problem Solving
Phase 2: Intermediate (Months 4-6): Data Structures, Algorithms, OOP, File Handling, Testing
Phase 3: Advanced (Months 7-9): Web Development (Django/Flask), Databases, APIs, Automation
Phase 4: Professional (Months 10-12): Data Science, Machine Learning, DevOps, Cloud, Specializations

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to everyone
Coding experience: Absolute beginner friendly - zero coding knowledge needed
Age: 10+ years (separate tracks for different age groups)
Equipment: Computer/laptop (Windows/Mac/Linux), internet connection
Time commitment: 12-15 hours per week consistently
English: Basic reading comprehension (materials available in multiple languages)
Motivation: Passion for learning and problem-solving
Math: Basic math (taught as needed for data science/ML tracks)

WHO THIS COURSE IS FOR

Kids: Age 10-14: Special simplified track with gamification, visual learning, age-appropriate projects
Teens: Age 15-18: Student-focused track, exam preparation, college prep, competitive programming
Students: College students: Career foundation, internship prep, placement preparation
Working professionals: Career switchers from any field to Python development
Data analysts: Upgrade skills to data science and machine learning
Entrepreneurs: Build your own products and automate your business
Freelancers: Offer Python development, automation, data analysis services
Teachers: Learn to teach Python to others
Hobbyists: Anyone passionate about programming and technology
Seniors: Never too late to learn - age-appropriate pace available

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE PYTHON SKILLS (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid Python fundamentals. Learn programming logic, master Python syntax, and create your first applications.

Months 1-2: Python Fundamentals & Programming Basics (Week 1-8)

Introduction to Programming & Python Setup

Topics:

- What is programming? How computers understand code
- Why Python? History, features, and popularity
- Python applications: web, data science, AI, automation, games
- Python 3 installation (Windows, Mac, Linux)
- Setting up development environment: VS Code, PyCharm
- Running Python: interactive shell vs script files
- IDLE, Jupyter Notebook, and Google Colab introduction
- Your first Python program: print('Hello World')
- Python syntax rules and code structure
- Comments: single-line (#) and multi-line (``')
- Indentation and its importance in Python
- Python PEP 8 style guide basics

Projects built:

- Hello World variations (personalized greetings)
- Simple calculator using print statements
- About Me program (display personal information)
- ASCII art creator

Practice: Daily: 30 min typing practice, write 5 simple print programs

Variables, Data Types & Operators

Topics:

- Variables: naming rules and conventions
- Assignment operator (=) and multiple assignments
- Data types: int, float, string, boolean
- Type checking with type() function
- Type conversion: int(), float(), str(), bool()
- Arithmetic operators: +, -, *, /, //, %, **
- Comparison operators: ==, !=, >, <, >=, <=
- Logical operators: and, or, not
- Assignment operators: +=, -=, *=, /=
- String basics: creation, concatenation, repetition
- String indexing and slicing [start:stop:step]
- F-strings and string formatting (format(), %)

Projects built:

- Temperature converter (Celsius/Fahrenheit)
- Age calculator with days, hours, minutes
- Simple interest calculator
- BMI (Body Mass Index) calculator
- Tip calculator with bill splitting

Practice: Solve 25 problems on variables, data types, and operators

Control Flow: Conditions & Loops

Topics:

- Boolean expressions and truthiness
- If statement: basic conditional execution
- If-else: two-way branching
- If-elif-else: multi-way branching
- Nested if statements
- Ternary operator (conditional expression)
- While loop: condition-based repetition
- Infinite loops and break statement
- Continue statement to skip iterations
- For loop: iterating over sequences
- Range function: range(start, stop, step)
- Nested loops and loop patterns

Projects built:

- Number guessing game with attempts
- Grade calculator with letter grades
- Even/odd number checker
- Multiplication table generator
- Pattern printing (stars, pyramids, diamonds)
- Simple password validator
- FizzBuzz challenge

Practice: Solve 30 conditional and loop problems

Functions & Modular Programming

Topics:

- What are functions? DRY principle
- Defining functions with 'def' keyword
- Parameters and arguments (positional, keyword)
- Return statement and return values
- Multiple return values (tuples)
- Default parameter values
- Variable scope: local vs global
- Global and nonlocal keywords
- Docstrings for function documentation
- *args for variable positional arguments
- **kwargs for variable keyword arguments
- Lambda functions (anonymous functions)
- Built-in functions: len, sum, max, min, abs, round

Projects built:

- Function library for common calculations
- Password strength checker function
- Text analyzer (word count, character count)
- Prime number checker and generator
- Palindrome checker
- Simple quiz game with scoring
- Unit converter (length, weight, temperature)

Practice: Create 20 different utility functions

Month 3: Data Structures & String Manipulation (Week 9-13)

Lists & List Operations

Topics:

- Lists: creating and accessing elements
- List indexing (positive and negative)
- List slicing: extracting sublists
- List methods: append, insert, extend
- Removing elements: remove, pop, clear
- List methods: sort, reverse, copy
- index() and count() methods

- List comprehensions: compact list creation
- Nested lists (2D lists, matrices)
- Iterating lists with for loops
- Enumerate function for index-value pairs
- List vs tuple: mutability difference

Projects built:

- Todo list application (add, remove, display)
- Shopping cart system
- Student grade manager
- Number list statistics (average, min, max)
- List sorting and searching programs
- Tic-tac-toe board representation

Practice: Solve 25 list manipulation problems

Tuples, Sets & Dictionaries

Topics:

- Tuples: immutable sequences
- Tuple packing and unpacking
- Tuple methods: count, index
- When to use tuples vs lists
- Sets: unordered unique collections
- Set operations: union, intersection, difference
- Set methods: add, remove, discard, clear
- Dictionaries: key-value pairs
- Accessing, adding, updating dictionary items
- Dictionary methods: keys(), values(), items()
- get() method with default values
- Dictionary comprehensions
- Nested dictionaries for complex data

Projects built:

- Contact book (name, phone, email)
- Word frequency counter
- Student database with grades
- Inventory management system
- English-Spanish dictionary translator
- Vote counting system

Practice: Build 10 projects using dictionaries and sets

Advanced String Manipulation

Topics:

- String methods: upper, lower, title, capitalize
- strip, lstrip,rstrip for whitespace removal
- find, index, count methods
- replace method for string substitution
- split and join methods
- startswith and endswith methods
- String validation: isalpha, isdigit, isalnum
- String formatting: old style, format(), f-strings
- Multi-line strings and raw strings
- String immutability and string building
- Regular expressions introduction (re module)
- Common string algorithms

Projects built:

- Text processing utility
- Email validator
- Acronym generator
- Caesar cipher encryption/decryption
- Word game (hangman basics)
- Log file parser
- PHASE 1 MINI CAPSTONE: Command-line text adventure game

Practice: Solve 20 string manipulation challenges

Assessment: Phase 1 Assessment - Programming fundamentals test

----- PHASE 2: INTERMEDIATE PYTHON & ALGORITHM MASTERY (MONTHS 4-6, WEEKS 14-26) -----

Master data structures, algorithms, object-oriented programming, file handling, and testing.

Months 4-5: Object-Oriented Programming & Advanced Concepts (Week 14-22)

Introduction to OOP Concepts

Topics:

- What is Object-Oriented Programming?
- Classes and objects: blueprint and instances
- Creating classes with 'class' keyword
- The __init__ constructor method
- Instance variables (attributes)
- Instance methods
- Self parameter explained
- Creating and using objects
- Class vs instance attributes
- Class methods with @classmethod decorator

- Static methods with @staticmethod decorator
- __str__ and __repr__ magic methods

Projects built:

- Bank account class with deposit/withdraw
- Student class with grades management
- Book class for library system
- Car class with properties and methods
- Rectangle/Circle classes with area calculation

Practice: Create 15 different classes modeling real-world objects

OOP Principles: Inheritance & Polymorphism

Topics:

- Inheritance: parent and child classes
- Method overriding in child classes
- Super() function to call parent methods
- Multiple inheritance
- Method Resolution Order (MRO)
- Polymorphism: same interface, different implementation
- Duck typing in Python
- Abstract base classes (ABC module)
- Encapsulation: public, protected, private
- Name mangling with double underscore
- Property decorators: @property, @setter
- Composition vs inheritance

Projects built:

- Animal hierarchy (Animal -> Dog, Cat, Bird)
- Employee management system (Employee -> Manager, Developer)
- Shape calculator with inheritance
- Vehicle rental system
- Game character classes with different abilities

Practice: Build 10 class hierarchies with inheritance

File Handling & Exception Handling

Topics:

- Opening files: open() function and modes (r, w, a, r+)
- Reading files: read(), readline(), readlines()
- Writing to files: write(), writelines()
- With statement for automatic file closing
- File paths: absolute vs relative
- Working with CSV files
- JSON file handling: json.dump(), json.load()
- Exception handling: try-except blocks
- Catching specific exceptions
- Multiple except blocks
- Else and finally clauses
- Raising exceptions with 'raise'
- Creating custom exceptions

Projects built:

- Note-taking app with file persistence
- Contact manager with CSV storage
- Configuration file reader/writer
- Log file analyzer
- Student records system with JSON
- Error-safe calculator

Practice: Build 8 file-based applications

Modules, Packages & Standard Library

Topics:

- Importing modules: import, from...import
- Creating custom modules
- Module search path and PYTHONPATH
- Packages and __init__.py
- Creating package structures
- Math module: mathematical functions
- Random module: random numbers and choices
- Datetime module: working with dates and times
- OS module: operating system interface
- Sys module: system-specific parameters
- Collections module: deque, Counter, defaultdict
- Itertools module: efficient iterators

Projects built:

- Dice rolling simulator with statistics
- Birthday reminder application
- File organizer using os module
- Random password generator
- Custom utility package creation
- Date calculator (age, days between dates)

Practice: Explore 15 standard library modules

Comprehensions & Functional Programming

Topics:

- List comprehensions with conditions
- Nested list comprehensions

- Dictionary comprehensions
- Set comprehensions
- Generator expressions
- Map function for transformations
- Filter function for filtering
- Reduce function for aggregation
- Lambda functions in functional programming
- Zip function for parallel iteration
- All and any functions
- Sorted function with key parameter

Projects built:

- Data transformation pipeline
- List processing utilities
- Functional calculator
- Data filtering and analysis tool

Practice: Solve 30 comprehension and functional programming problems

Month 6: Algorithms, Data Structures & Testing (Week 23-26)

Algorithms & Problem Solving

Topics:

- Algorithm complexity: Big O notation basics
- Searching algorithms: linear search
- Binary search algorithm
- Sorting algorithms: bubble sort
- Selection sort and insertion sort
- Merge sort and quick sort basics
- Recursion: base case and recursive case
- Recursive problem solving
- Fibonacci sequence (iterative vs recursive)
- Factorial calculation
- Tower of Hanoi
- Dynamic programming introduction

Projects built:

- Custom sorting algorithm implementation
- Binary search tree basics
- Recursive file directory traversal
- Algorithm visualization program
- Performance comparison of algorithms

Practice: Solve 40 algorithm problems on platforms like LeetCode/HackerRank

Advanced Data Structures

Topics:

- Stacks: LIFO principle, implementation
- Stack applications: parenthesis matching
- Queues: FIFO principle, implementation
- Deque (double-ended queue)
- Linked lists: singly linked list
- Doubly linked list
- Trees: binary tree basics
- Tree traversal: inorder, preorder, postorder
- Hash tables and hash functions
- Graphs: representation and basics
- Graph traversal: BFS and DFS
- Choosing right data structure

Projects built:

- Stack-based calculator
- Queue simulation (ticket counter)
- Linked list implementation
- Binary search tree operations
- Graph representation and traversal

Practice: Implement 8 core data structures from scratch

Testing & Debugging

Topics:

- Why testing matters
- Manual testing vs automated testing
- Unit testing with unittest module
- Test cases and assertions
- SetUp and tearDown methods
- Pytest framework introduction
- Writing pytest test functions
- Test fixtures in pytest
- Parameterized testing
- Code coverage with coverage.py
- Debugging techniques in Python
- Using Python debugger (pdb)
- Logging with logging module

Projects built:

- Test suite for calculator
- Testing OOP classes
- Testing file operations

- TDD (Test-Driven Development) mini project
- Debugging exercise solutions

Practice: Write tests for all previous projects

Advanced Python Features

Topics:

- Decorators: function decorators
- Creating custom decorators
- Class decorators
- Decorators with arguments
- Generators and yield keyword
- Generator functions vs generator expressions
- Iterators and `__iter__`, `__next__`
- Context managers and 'with' statement
- Creating context managers with `__enter__`, `__exit__`
- Metaclasses basics
- Operator overloading
- Magic methods (`__add__`, `__len__`, `__getitem__`, etc.)

Projects built:

- Custom decorator library
- File reader with generators
- Custom iterator implementation
- Context manager for database/file handling
- Vector class with operator overloading

Practice: Master decorators, generators, and magic methods with 20 exercises

Phase 2 Capstone Project

Topics:

- Object-oriented design
- Algorithm implementation
- File-based data persistence
- Error handling throughout
- Unit testing
- Code documentation

Projects built:

- PHASE 2 CAPSTONE: Library Management System (OOP-based)
- Features: Books, members, lending, returns, fines, search, file storage, testing
- Alternative: School Management System
- Alternative: Banking System with multiple account types
- Alternative: E-commerce Inventory System

Assessment: Phase 2 Final Exam - OOP, algorithms, and testing

PHASE 2 CONTINUED - This section adapts to Python context (Week 14-26 (distributed across months 4-6))

Version Control with Git & GitHub

Topics:

- What is version control?
- Git installation and setup
- Git basic commands: `init`, `add`, `commit`
- Git workflow: working directory, staging, repository
- Checking status and history: `status`, `log`
- Branches: creating and switching
- Merging branches
- Handling merge conflicts
- Remote repositories: GitHub
- Push, pull, clone operations
- GitHub collaboration basics
- `.gitignore` file for Python projects

Projects built:

- Initialize Git for existing projects
- Create GitHub repository
- Collaborate on shared repository
- Open source contribution preparation

Practice: Use Git for all future projects

Virtual Environments & Package Management

Topics:

- Why virtual environments?
- Venv module for creating environments
- Activating and deactivating environments
- Pip package manager
- Installing packages with pip
- `Requirements.txt` file
- Pip freeze for dependency tracking
- Pipenv for advanced package management
- Poetry package manager
- Publishing packages to PyPI
- Creating `setup.py` for distribution
- Best practices for Python projects

Projects built:

- Set up virtual environment for each project
- Create installable Python package
- Dependency management practice

- Publish simple package to Test PyPI

Practice: Set up professional project structure

Regular Expressions & Text Processing

Topics:

- Regular expressions (regex) introduction
- Re module in Python
- Basic patterns: literals, wildcards
- Character classes and ranges
- Quantifiers: *, +, ?, {n,m}
- Anchors: ^, \$, \b
- Groups and capturing
- Search, match, findall, finditer
- Substitution with sub() and subn()
- Regex flags: IGNORECASE, MULTILINE
- Common regex patterns
- Real-world text processing

Projects built:

- Email validator with regex
- Phone number formatter
- URL extractor from text
- Log file parser
- Data extraction from unstructured text
- Password strength validator (advanced)

Practice: Solve 25 regex challenges

Working with APIs & Web Scraping Basics

Topics:

- What are APIs? REST API basics
- HTTP methods: GET, POST, PUT, DELETE
- Requests library for HTTP
- Making GET requests
- Parsing JSON responses
- Query parameters and headers
- POST requests with data
- API authentication basics
- Web scraping introduction
- BeautifulSoup library
- Parsing HTML structure
- Ethics and legality of web scraping

Projects built:

- Weather app using OpenWeatherMap API
- Currency converter with API
- GitHub profile viewer
- Quote generator from API
- Simple web scraper for news headlines

Practice: Integrate 5 different public APIs

Concurrency & Parallel Programming

Topics:

- Concurrency vs parallelism
- Threading basics with threading module
- Creating and starting threads
- Thread synchronization
- Multiprocessing module
- Process creation and management
- Pool for parallel execution
- AsyncIO for asynchronous programming
- Async/await syntax
- Concurrent.futures module
- ThreadPoolExecutor and ProcessPoolExecutor
- When to use threading vs multiprocessing

Projects built:

- Multi-threaded file downloader
- Parallel data processing
- Async web scraper
- CPU-intensive task with multiprocessing

Practice: Optimize slow programs with concurrency

PHASE 3: WEB DEVELOPMENT, DATABASES & AUTOMATION (MONTHS 7-9, WEEKS 27-39)

Master web development with Django/Flask, databases, REST APIs, web scraping, and automation.

Months 7-8: Web Development with Flask & Django (Week 27-35)

Flask Web Framework Fundamentals

Topics:

- Introduction to web development
- Flask installation and setup
- Creating first Flask application
- Routes and URL mapping
- Dynamic routes with variables

- HTTP methods: GET and POST
- Request and response objects
- Rendering templates with Jinja2
- Template inheritance and blocks
- Static files: CSS, JavaScript, images
- Form handling in Flask
- Flask sessions and cookies

Projects built:

- Personal blog with Flask
- Todo application with web interface
- Contact form application
- Simple URL shortener
- Portfolio website with Flask

Practice: Build 5 Flask web applications

Flask Advanced & REST APIs

Topics:

- Flask Blueprints for modular apps
- Application factory pattern
- Configuration management
- Error handling and custom error pages
- Flask-WTF for forms
- Form validation
- Building RESTful APIs with Flask
- Flask-RESTful extension
- JSON responses
- API versioning
- CORS handling
- API documentation with Swagger

Projects built:

- RESTful API for blog
- Task management API
- User authentication API
- E-commerce product API
- Weather API wrapper service

Practice: Build 5 REST APIs with Flask

Django Web Framework Fundamentals

Topics:

- Django introduction and philosophy
- Django installation and project creation
- Django project vs app structure
- MVT (Model-View-Template) architecture
- URL routing and URL patterns
- Views: function-based views
- Class-based views
- Django templates and template language
- Template filters and tags
- Static files in Django
- Django admin interface
- Django settings and configuration

Projects built:

- Blog application with Django
- Portfolio website
- Simple CMS (Content Management System)
- News aggregator
- Personal diary application

Practice: Build 5 Django web applications

Django Models & Database

Topics:

- Django ORM (Object-Relational Mapping)
- Defining models and fields
- Field types and options
- Database migrations: makemigrations, migrate
- QuerySet API: all, filter, get
- Field lookups and query expressions
- Related objects: ForeignKey, ManyToMany
- OneToOne relationships
- Model managers and custom managers
- Model methods and properties
- Admin customization
- Database optimization and select_related

Projects built:

- E-commerce product catalog
- Social media data models
- School management system
- Library database with relationships
- Recipe sharing platform

Practice: Design 10 complex database schemas with Django

Django Forms & User Authentication

Topics:

- Django forms and form fields
- ModelForm for automatic form generation
- Form validation and cleaning
- Custom validators
- Django authentication system
- User model and user creation
- Login and logout views
- Password management
- User registration
- Permission and authorization
- User groups and permissions
- Custom user model

Projects built:

- User registration and login system
- Profile management application
- Multi-user blog platform
- Forum with user accounts
- Social network user system

Practice: Add authentication to all Django projects

Month 9: Databases, ORMs & Advanced Web Topics (Week 36-39)

SQL Databases with Python

Topics:

- Relational database concepts review
- SQLite with Python: sqlite3 module
- Connecting to databases
- Creating tables with SQL
- CRUD operations with Python
- Parameterized queries for security
- PostgreSQL with psycopg2
- MySQL with PyMySQL or mysql-connector
- SQLAlchemy ORM introduction
- Defining models with SQLAlchemy
- Session management
- Querying with SQLAlchemy

Projects built:

- Database management CLI tool
- Data migration scripts
- SQLAlchemy-based application
- Multi-database application
- Database backup utility

Practice: Build 5 database-driven applications

NoSQL Databases & MongoDB

Topics:

- NoSQL database types
- MongoDB introduction
- Installing MongoDB
- PyMongo: Python MongoDB driver
- Connecting to MongoDB
- CRUD operations in MongoDB
- Querying documents
- Aggregation pipeline
- Indexing for performance
- MongoEngine ODM
- Defining document schemas
- When to use NoSQL vs SQL

Projects built:

- MongoDB-based blog
- Product catalog with MongoDB
- Analytics data storage
- User activity tracking system
- Document management system

Practice: Build 5 MongoDB applications

Django REST Framework & API Development

Topics:

- Django REST Framework (DRF) introduction
- Serializers for data conversion
- API views: function-based and class-based
- ViewSets and routers
- Authentication: token, session, JWT
- Permissions and authorization
- Pagination for large datasets
- Filtering and searching
- API versioning strategies
- Testing APIs with DRF
- API documentation with drf-spectacular
- CORS configuration

Projects built:

- Complete blog API with DRF

- E-commerce REST API
- Social media API
- Task management API
- Real-time chat API backend

Practice: Build 5 production-ready APIs

Web Scraping & Automation

Topics:

- Advanced BeautifulSoup techniques
- Navigating HTML tree
- CSS selectors for scraping
- Scrapy framework introduction
- Creating Scrapy spiders
- Item pipelines for data processing
- Scrapy selectors and XPath
- Handling pagination
- Selenium for dynamic websites
- WebDriver automation
- Headless browsing
- Ethics, robots.txt, and rate limiting

Projects built:

- News scraper with BeautifulSoup
- E-commerce price tracker
- Job listings aggregator with Scrapy
- Social media scraper (ethical)
- Automated form filler with Selenium

Practice: Build 8 scraping and automation projects

Task Automation & Scripting

Topics:

- File and folder automation
- Batch file processing
- PDF manipulation with PyPDF2
- Excel automation with openpyxl
- CSV data processing
- Email automation with smtplib
- Scheduling tasks with schedule library
- System administration scripts
- Desktop automation with PyAutoGUI
- Clipboard automation
- Screenshot and image manipulation
- Building CLI tools with Click/argparse

Projects built:

- File organizer automation
- Excel report generator
- Email sender with attachments
- PDF merger and splitter
- Automated backup system
- Desktop notification system
- System monitoring script

Practice: Create 10 automation scripts for daily tasks

PHASE 3 COMPLETION - Month 9 Continued (Week 27-39 (distributed))

Deployment & Production Best Practices

Topics:

- Preparing Django/Flask for production
- Environment variables and secrets
- Debug mode and security settings
- Static files in production
- Database configuration for production
- Gunicorn and uWSGI web servers
- Nginx as reverse proxy
- Deployment to Render
- Deployment to PythonAnywhere
- Deployment to DigitalOcean/AWS EC2
- Docker basics for Python applications
- Creating Dockerfile for Python apps

Projects built:

- Deploy Flask app to Render
- Deploy Django app to production
- Dockerize Python web application
- Set up CI/CD for Python project
- Production-ready deployment checklist

Practice: Deploy all web projects to production

Security Best Practices

Topics:

- Common web vulnerabilities (OWASP Top 10)
- SQL injection prevention
- Cross-Site Scripting (XSS) prevention
- CSRF protection in Django/Flask
- Secure password hashing

- JWT token security
- HTTPS and SSL certificates
- Input validation and sanitization
- Rate limiting APIs
- Security headers
- Dependency vulnerability scanning
- Environment security best practices

Projects built:

- Security audit of existing projects
- Implement security measures in APIs
- Password security analyzer
- Vulnerability scanner script

Practice: Security-harden all web applications

Performance Optimization & Caching

Topics:

- Python code profiling
- cProfile and line_profiler
- Memory profiling
- Database query optimization
- Django Debug Toolbar
- N+1 query problem and solutions
- Caching strategies
- Django caching framework
- Redis for caching
- Celery for background tasks
- Asynchronous task queues
- Load testing with Locust

Projects built:

- Optimize slow Django application
- Implement caching layer
- Background task processing with Celery
- Performance monitoring dashboard
- Load testing suite

Practice: Optimize and cache all applications

Real-time Applications & WebSockets

Topics:

- WebSocket protocol basics
- Django Channels for WebSockets
- Channels layers and Redis
- Real-time chat implementation
- Broadcasting messages
- Flask-SocketIO for real-time
- Async consumers in Django
- Real-time notifications
- Live updates and dashboards
- Scaling WebSocket applications

Projects built:

- Real-time chat application
- Live notification system
- Collaborative editor
- Real-time dashboard with live data
- Multiplayer game backend

Practice: Add real-time features to previous projects

Phase 3 Capstone Project

Topics:

- Full-stack web application
- Database design and optimization
- REST API development
- Authentication and authorization
- Real-time features
- Production deployment
- Security implementation
- Performance optimization

Projects built:

- MAJOR CAPSTONE: Complete Social Media Platform
- Features: User auth, posts, comments, likes, followers, real-time chat, notifications, media upload, REST API
- Alternative: E-commerce Platform (products, cart, checkout, orders, admin)
- Alternative: Project Management Tool (teams, projects, tasks, real-time collaboration)
- Alternative: Learning Management System (courses, students, assignments, progress tracking)

Assessment: Phase 3 Final Exam - Web development comprehensive test

PHASE 4: DATA SCIENCE, MACHINE LEARNING & PROFESSIONAL SKILLS (MONTHS 10-12, WEEKS 40-52)

Master data science, machine learning, AI, cloud computing, and professional development practices.

Months 10-11: Data Science & Machine Learning (Week 40-48)

Data Science Fundamentals with NumPy

Topics:

- Introduction to data science
- NumPy installation and basics
- NumPy arrays vs Python lists
- Array creation methods
- Array indexing and slicing
- Array reshaping and transposing
- Mathematical operations on arrays
- Broadcasting in NumPy
- Statistical functions
- Linear algebra with NumPy
- Random number generation
- Saving and loading NumPy arrays

Projects built:

- Statistical calculator with NumPy
- Matrix operations program
- Grade analysis system
- Financial calculations
- Scientific computing tasks

Practice: Solve 30 NumPy exercises

Data Analysis with Pandas

Topics:

- Pandas introduction and installation
- Series and DataFrame objects
- Reading data: CSV, Excel, JSON
- Data exploration: head, tail, info, describe
- Indexing and selecting data
- Filtering and boolean indexing
- Sorting and ranking
- Handling missing data
- Data aggregation and groupby
- Merging and joining DataFrames
- Pivot tables
- Data visualization basics with Pandas

Projects built:

- Sales data analysis
- Student performance analysis
- COVID-19 data analysis
- Stock market data analyzer
- Survey data processing

Practice: Analyze 10 real-world datasets

Data Visualization with Matplotlib & Seaborn

Topics:

- Matplotlib introduction
- Line plots and customization
- Scatter plots and bar charts
- Histograms and pie charts
- Subplots and figure layout
- Customizing plots: colors, labels, legends
- Seaborn for statistical visualization
- Distribution plots
- Categorical plots
- Heatmaps and correlation matrices
- Pair plots
- Styling and themes

Projects built:

- Sales visualization dashboard
- Weather data visualization
- Sports statistics visualizer
- Financial charts and graphs
- Scientific data plotter

Practice: Create 20 different types of visualizations

Machine Learning Fundamentals

Topics:

- Introduction to machine learning
- Types: supervised, unsupervised, reinforcement
- Scikit-learn library introduction
- Train-test split
- Linear regression
- Model evaluation metrics
- Classification: Logistic Regression
- K-Nearest Neighbors (KNN)
- Decision Trees
- Random Forests
- Feature scaling and normalization
- Cross-validation

Projects built:

- House price prediction

- Iris flower classification
- Customer churn prediction
- Spam email classifier
- Diabetes prediction model

Practice: Build 10 ML models on different datasets

Advanced Machine Learning

Topics:

- Support Vector Machines (SVM)
- Clustering: K-Means
- Hierarchical clustering
- Dimensionality reduction: PCA
- Model selection and hyperparameter tuning
- Grid search and random search
- Ensemble methods
- Handling imbalanced datasets
- Feature engineering
- Model deployment basics
- Saving models with pickle/joblib
- ML pipeline creation

Projects built:

- Customer segmentation
- Image classification basic
- Recommendation system
- Anomaly detection system
- Complete ML pipeline

Practice: Advanced ML projects on Kaggle

Month 12: Deep Learning, Cloud & Professional Skills (Week 49-52)

Deep Learning with TensorFlow/Keras

Topics:

- Introduction to neural networks
- Deep learning frameworks overview
- TensorFlow and Keras installation
- Building neural networks with Keras
- Activation functions
- Loss functions and optimizers
- Training neural networks
- Validation and overfitting
- Convolutional Neural Networks (CNN) basics
- Image classification with CNN
- Transfer learning
- Model saving and loading

Projects built:

- Handwritten digit recognition (MNIST)
- Image classifier
- Sentiment analysis with neural networks
- Object detection basics
- Deep learning web app

Practice: Build 5 deep learning models

Natural Language Processing (NLP)

Topics:

- NLP introduction and applications
- Text preprocessing
- Tokenization and stemming
- Lemmatization
- Bag of Words and TF-IDF
- Word embeddings: Word2Vec
- Sentiment analysis
- Text classification
- Named Entity Recognition (NER)
- NLTK library
- spaCy library
- Transformers and BERT basics

Projects built:

- Sentiment analyzer for reviews
- Spam detection system
- Chatbot with NLP
- Text summarizer
- Language detection tool

Practice: Build 8 NLP applications

Cloud Computing & AWS for Python

Topics:

- Cloud computing introduction
- AWS account setup
- AWS Lambda for serverless Python
- Boto3: AWS SDK for Python
- S3 for file storage
- EC2 for Python applications
- RDS for databases

- API Gateway with Lambda
- CloudWatch for monitoring
- Deploying ML models to cloud
- Google Cloud Platform basics
- Azure for Python developers

Projects built:

- Serverless API with Lambda
- File upload to S3
- Cloud-based web scraper
- ML model deployment on AWS
- Cloud automation scripts

Practice: Deploy 5 applications to cloud

DevOps & CI/CD for Python

Topics:

- Docker for Python applications
- Creating Dockerfile
- Docker Compose for multi-container apps
- GitHub Actions for Python
- Automated testing in CI/CD
- Code quality tools: pylint, black, flake8
- Pre-commit hooks
- Continuous deployment strategies
- Monitoring and logging
- Infrastructure as Code with Python
- Kubernetes basics for Python apps
- Production best practices

Projects built:

- Dockerized Python application
- Complete CI/CD pipeline
- Automated deployment system
- Monitoring dashboard
- Infrastructure automation

Practice: Set up DevOps for all projects

Specialized Python Topics

Topics:

- Game development with Pygame
- GUI applications with Tkinter
- PyQt for desktop apps
- Computer vision with OpenCV
- Blockchain basics with Python
- AI provider APIs
- IoT with Python and Raspberry Pi
- Robotics basics
- Cybersecurity with Python
- Penetration testing tools
- Ethical hacking basics
- Choosing specialization path

Projects built:

- Simple game with Pygame
- Desktop application with Tkinter
- Face detection with OpenCV
- Blockchain implementation
- Network scanner tool

Practice: Explore 3 specialization areas

PHASE 4 COMPLETION - Final Month Activities (Week 40-52 (distributed))

Open Source Contribution

Topics:

- Finding beginner-friendly projects
- Understanding project structure
- Reading contribution guidelines
- Forking and cloning repositories
- Creating branches for features
- Making meaningful contributions
- Creating pull requests
- Code review process
- Building open source portfolio
- Starting your own open source project

Projects built:

- Contribute to 3 open source projects
- Create own open source library
- Document contributions
- Build contributor profile

Practice: Make 10 open source contributions

Python Best Practices & Design Patterns

Topics:

- PEP 8 style guide deep dive
- Code organization and structure
- Design patterns in Python

- Singleton, Factory, Observer patterns
- SOLID principles
- Clean code principles
- Refactoring techniques
- Code smells and fixes
- Documentation best practices
- Type hints and mypy
- Code review guidelines
- Technical debt management

Projects built:

- Refactor old projects
- Implement design patterns
- Create coding standards document
- Code review checklist

Practice: Apply best practices to all code

Performance & Optimization

Topics:

- Profiling Python code
- Memory profiling
- Time complexity optimization
- Space complexity optimization
- Cython for performance
- NumPy optimization techniques
- Database query optimization
- Caching strategies
- Parallel processing
- Async programming optimization
- Code optimization techniques
- Benchmarking

Projects built:

- Optimize slow applications
- Performance comparison study
- Benchmarking suite
- Optimization case studies

Practice: Optimize 10 previous projects

Interview Preparation

Topics:

- Python coding interview patterns
- Data structures interview questions
- Algorithm interview questions
- System design for Python developers
- Behavioral interview preparation
- Python-specific questions
- Problem-solving strategies
- LeetCode and HackerRank practice
- Mock interview techniques
- Salary negotiation
- Job search strategies
- Building professional network

Projects built:

- Solve 100 LeetCode problems
- System design case studies
- Mock interview recordings
- Interview preparation guide

Practice: Daily coding challenges

Final Projects & Career Launch (Week 49-52)

Final Capstone Project - Part 1

Topics:

- Project ideation and planning
- Choosing tech stack
- System architecture design
- Database schema design
- API design
- Frontend-backend integration
- Development workflow
- Agile methodology

Projects built:

- FINAL CAPSTONE: Choose Your Specialization Project
- Option 1: Full-Stack Web Platform (Django/Flask + React + ML features)
- Option 2: Data Science Dashboard (Complete analysis + visualization + ML models)
- Option 3: AI/ML Application (Deep learning model + web interface + deployment)
- Option 4: Automation Platform (Web scraping + task automation + scheduling)
- Option 5: DevOps Solution (CI/CD + monitoring + cloud deployment)

Final Capstone Project - Part 2

Topics:

- Testing and quality assurance
- Performance optimization
- Security implementation

- Documentation writing
- Deployment to production
- Monitoring and logging
- User feedback integration
- Project presentation

Deliverables:

- Complete source code on GitHub
- Live production deployment
- Technical documentation
- User manual
- Video demo and presentation
- Architecture diagrams
- API documentation
- Test coverage report
- Performance metrics

Career Launch & Portfolio Building

Topics:

- Professional portfolio website
- Resume optimization for Python roles
- LinkedIn profile enhancement
- GitHub profile showcase
- Writing technical blog posts
- Creating YouTube tutorials (optional)
- Networking strategies
- Freelancing platforms setup
- Job application strategies
- Continuous learning plan
- Python community involvement
- Mentoring others

Deliverables:

- Professional portfolio with 30+ projects
- Updated resume and LinkedIn
- GitHub profile with contributions
- 3-5 technical blog posts
- Personal brand establishment
- Job application materials
- Interview preparation complete
- Continuous learning roadmap

Assessment: FINAL COMPREHENSIVE CERTIFICATION EXAM - All 12 months content

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-3): 15+ beginner projects - calculators, games, text processors
- Phase 2 (Months 4-6): 12+ intermediate projects - OOP applications, file systems, tested code
- Phase 3 (Months 7-9): 15+ advanced projects - web apps, APIs, databases, scrapers
- Phase 4 (Months 10-12): 10+ professional projects - ML models, data analysis, cloud apps
- Final: 5 major capstone projects showcasing complete mastery

Total projects: 50+ projects across all difficulty levels and domains

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Python: Syntax, data types, control flow, functions, OOP, file I/O, exceptions
- Data Structures & Algorithms: Lists, dictionaries, sets, trees, graphs, sorting, searching
- Web Development: Flask, Django, REST APIs, templates, authentication, deployment
- Databases: SQL (SQLite, PostgreSQL, MySQL), NoSQL (MongoDB), ORMs (SQLAlchemy, Django ORM)
- Data Science: NumPy, Pandas, Matplotlib, Seaborn, data analysis, visualization
- Machine Learning: Scikit-learn, TensorFlow, Keras, supervised/unsupervised learning, deep learning
- NLP: Text processing, sentiment analysis, chatbots, NLTK, spaCy
- Automation: Web scraping (BeautifulSoup, Scrapy, Selenium), task automation, scheduling
- DevOps: Docker, Git/GitHub, CI/CD, testing (unittest, pytest), deployment
- Cloud: AWS (Lambda, S3, EC2), serverless, cloud deployment
- Tools: VS Code, PyCharm, Jupyter, Git, virtual environments, pip, Docker
- Specialized: GUI (Tkinter), Game Dev (Pygame), Computer Vision (OpenCV), async programming

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-5 hours
 Hands on coding: 6-8 hours
 Projects: 2-3 hours
 Practice problems: 1-2 hours
 Total per week: 12-15 hours

SUPPORT INCLUDED

Live sessions: Weekly doubt clearing and live coding sessions
 Mentorship: 1-on-1 Python expert mentorship
 Community: Active Discord community with 24/7 support

Code review: Expert code reviews for all projects
Career support: Resume review, mock interviews, job referrals
Lifetime access: All content, future updates, new modules forever
Age appropriate: Separate support channels for kids, teens, adults
Parent support: For kids track - parent guidance and progress reports

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 total)
Final certificate: Professional Python Developer Certification
Specialization certificate: Certificate in chosen specialization (Web/Data/ML/Automation)
Linkedin badge: Digital badge for LinkedIn profile
Industry recognized: Recognized by tech companies and startups
Portfolio projects: 30+ portfolio projects to showcase
Skill endorsements: Verified skill assessments

WHERE THIS CAN LEAD

- Python Developer (Junior to Senior levels)
- Full Stack Python Developer (Django/Flask)
- Data Analyst / Data Scientist
- Machine Learning Engineer
- AI/ML Specialist
- Backend Developer
- DevOps Engineer (Python focus)
- Automation Engineer / QA Automation
- Web Scraping Specialist
- Research Scientist (Academia)
- Python Instructor / Trainer
- Freelance Python Developer
- Startup Founder / CTO
- Quantitative Analyst (Finance)
- Bioinformatics Programmer

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 2-4 LPA (Junior Python Developer / Intern)
Growing roles: Rs 4-7 LPA (Python Developer / Automation Engineer); Rs 7-12 LPA (Full Stack Python / Data Analyst)
Experienced roles: Rs 10-20 LPA (Senior Python / Data Scientist / ML Engineer)
Other roles: Rs 12-30 LPA in India, \$80k-180k USD internationally; Rs 15-40 LPA in India, \$100k-200k+ USD internationally
Freelance: Rs 500-3,000/hour based on specialization
International: \$60k-150k USD (USA) based on role and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: I have never coded before. Can I really start here?

A: Yes, the course is built for absolute beginners: phase 1 teaches programming logic and Python fundamentals from zero. The 12-month runway is exactly why beginners succeed here; there is time to build foundations properly before web development, data science and machine learning arrive in later phases.

Q: What will I build across the 12 months?

A: Dozens of real projects: command line tools and automation scripts early on, then algorithm work, file and data handling, web applications with Django and Flask, REST APIs, database-backed apps, and finally data science and machine learning projects. Everything lands in your GitHub portfolio as you go.

Q: Which Python version and libraries do you teach?

A: Current Python 3 throughout, with the libraries working developers use: Django and Flask for the web, pandas for data work, and standard tooling for testing and automation. The material tracks the current ecosystem, so what you practice is what companies run.

Q: How does this help with internships and placements?

A: Python is one of the most in-demand generalist languages, used across web development, data, automation and AI roles. You finish with a year of projects, code review experience and interview preparation on Python fundamentals and problem solving. No placement promises; the goal is that your skills and portfolio survive real scrutiny.

Q: Free Python tutorials are everywhere. Why take a live course?

A: Because watching Python is not writing Python. A live mentor reviews your code every week, untangles your bugs with you, holds you to good habits like clean functions and testing, and

keeps you moving when motivation dips, which is where self-learning usually ends. The result is a portfolio of finished projects instead of a half-watched playlist.

Q: Is there a free trial?

A: Your first class is a free live demo, booked at learn.modernagecoders.com/contact with no commitment. You meet the instructor, write some code, and decide with real information. We hold a 4.9 star rating across 547+ Google reviews.

Q: What is the teaching format?

A: Live 1-hour online classes with an instructor, twice a week, never pre-recorded. Classes are interactive, with you coding during the session, and evening and weekend slots are available to fit college timetables and working hours.

Q: Will I get a certificate?

A: Yes, a completion certificate at the end of the masterclass, together with a GitHub-ready project portfolio built over the full year. When you apply for internships or jobs, the portfolio does the talking; the certificate rounds it out.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/python-programming-masterclass-zero-to-advanced-college>

=====

COURSE 104 OF 118: PYTHON WEB DEVELOPMENT WITH DJANGO AND FLASK COURSE

=====

Course link: <https://learn.modernagecoders.com/courses/python-web-development-django-flask-course>
Subject area: Web Development
Designed for: College students
Who can join: Intermediate, college students and working professionals with Python basics
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Python web development course with Django and Flask: live online classes on routing, templates, databases, auth, REST APIs and deploying a real project.

In one line: You know Python. Six months from now, your code answers requests on a real URL.

ABOUT THE PROGRAMME

Knowing Python and being able to ship a web application are two different skills, and the second one is what this course builds. It starts where confusion usually begins: what actually happens when a browser asks a server for a page. Month 1 answers that with HTTP fundamentals and Flask, a framework small enough to see through: routes, templates, and forms with nothing hidden. Month 2 adds the database layer, SQL first and then the SQLAlchemy ORM, plus sessions and login, so by the halfway mark students have built a complete Flask application from scratch. Months 3 and 4 move to Django, the framework Indian and global companies run in production: models, migrations, the admin, authentication, relationships, and class-based views, ending in a database-backed project with real users. Month 5 covers what most jobs actually ask for, REST APIs with Django REST Framework: serializers, token authentication, permissions, and a documented API tested end to end. Month 6 is deployment and the capstone: environment configuration, Postgres, Gunicorn, static files, a live host, and a project of the student's own design shipped to a public URL and defended. Classes are live and small, everything is built by typing, and code review is part of the course, not a bonus.

WHAT MAKES THIS COURSE DIFFERENT

- Flask before Django, on purpose: a small framework first means Django's conveniences read as solutions, not magic
- SQL is taught before the ORM, so students understand what `save()` and `filter()` actually send to the database
- The middle of the course is a complete Flask app and a complete Django app, not fragments that never meet
- REST APIs get a full month with Django REST Framework, because that is what backend job descriptions actually list
- Deployment is a taught skill, not a footnote: every student ships the capstone to a real public URL
- Code review from a working instructor on every project, with the same standards a team lead would apply

THE LEARNING PATH, STEP BY STEP

Phase 1: How the web works and Flask from scratch: HTTP, routes, templates, forms, SQL, the ORM, sessions, and a complete Flask application

Phase 2: Django properly: models, migrations, admin, authentication, relationships, and class-based views, ending in a database-backed multi-user project

Phase 3: REST APIs with Django REST Framework, then production: deployment, Postgres, environment config, and a capstone shipped to a live URL

WHAT A STUDENT NEEDS BEFORE STARTING

Python basics: Required: variables, loops, functions, lists, and dictionaries. Our Python courses cover this if you are not there yet

Web experience: None needed. HTTP, HTML, and databases are all taught from zero inside the course

Equipment: Any Windows, Mac, or Linux computer that runs Python and VS Code comfortably, plus a stable internet connection

Time: Around 3-4 hours a week outside class. The projects are real, and they are where the learning compounds

WHO THIS COURSE IS FOR

College students: Students with Python from coursework who want a portfolio that says more than marks do

Working professionals: Professionals in testing, support, data, or other fields moving toward backend development roles

Python learners stuck at scripts: Self-taught programmers who can write scripts but have never shipped anything with a URL

Startup builders: Founders and hobbyists who want to build and deploy their own product idea end to end

Interview preparers: Candidates targeting backend roles where Django, DRF, and deployment questions come up by name

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: HOW THE WEB WORKS, THEN FLASK (MONTHS 1-2, WEEKS 1-8) -----

HTTP demystified with real requests, then Flask built up from a single route to a complete application with a database, forms, and login.

Month 1: HTTP and Your First Flask Apps (Weeks 1-4)

What Happens When You Load a Page

Topics:

- Client and server: who asks, who answers
- HTTP requests dissected: method, path, headers, body
- Status codes worth knowing: 200, 301, 404, 500
- Watching real traffic in the browser network tab
- Making requests by hand with curl and Python's requests library
- GET vs POST, and why the difference matters before any framework

Projects built:

- Request detective: a written trace of one real page load, every request annotated, plus a script that fetches and inspects three sites

Practice: Use curl to make ten requests against a public API, logging method, status, and one header for each

First Flask Application

Topics:

- Virtual environments and pip: project hygiene from day one
- A Flask app in seven lines, and what each line does
- Routes and view functions
- Dynamic routes: reading values out of the URL
- The development server and debug mode
- Project structure that will survive growth

Projects built:

- Personal API: a Flask app with five routes returning text and JSON, including a dynamic greeting route

Practice: Build a dice-roll and coin-flip service with routes that take the number of rolls from the URL

Templates

Topics:

- Why string concatenation dies fast: enter Jinja2
- render_template and passing data to pages
- Template logic: loops and conditionals in Jinja2
- Template inheritance: one base layout, many pages
- Static files: CSS and images served properly
- Just enough HTML and CSS, taught as the projects need it

Projects built:

- Movie catalog site: a browsable list rendered from Python data with a shared base layout and detail pages

Practice: Add a search-results page and an empty-state design to the movie catalog using template conditionals

Forms and User Input

Topics:

- HTML forms: action, method, and named fields
- Reading form data in Flask with request.form
- The POST-redirect-GET pattern and why refreshing resubmits without it
- Validating input on the server, always
- Flash messages: telling users what just happened
- Never trusting the client: a first look at input handling as a security matter

Projects built:

- Feedback board: a form that accepts, validates, and displays entries, with flash messages for errors and success

Practice: Add server-side validation rules to the feedback board and prove each one works with a bad submission

Assessment: Month 1 build check: a small multi-page Flask app with one form, built live from a spec

Month 2: Databases and a Complete Flask App (Weeks 5-8)

SQL Before the ORM

Topics:

- Why files stop working: the case for databases
- SQLite: a real database with zero setup
- CREATE TABLE, data types, and primary keys
- INSERT, SELECT, UPDATE, DELETE by hand
- WHERE, ORDER BY, and LIMIT
- Running queries from Python with sqlite3

Projects built:

- Query workbook: a seeded database of books and members, interrogated with fifteen handwritten SQL queries

Practice: Write ten more queries against the workbook database, including two that join tables

The ORM: SQLAlchemy

Topics:

- What an ORM is and what it is hiding
- Defining models as Python classes
- Creating, querying, updating, and deleting through the ORM
- Echoing the generated SQL: seeing what the ORM sends
- Filtering and ordering with query methods
- When to drop to raw SQL, honestly

Projects built:

- Movie catalog, database edition: the month 1 site rebuilt on SQLAlchemy models instead of Python lists

Practice: For five ORM queries, predict the SQL before enabling echo, and score your predictions

CRUD End to End

Topics:

- The CRUD pattern: create, read, update, delete as routes
- Forms that create and edit database records
- Deleting safely: confirmation and POST-only deletes
- URL design for resources: lists, details, edits
- Handling the missing record: 404 done properly
- Keeping view functions thin

Projects built:

- Notes application: full CRUD with create, edit, delete, and detail pages backed by the database

Practice: Add pinning and archiving to the notes app, each touching the model, routes, and templates

Sessions and Login

Topics:

- Cookies and sessions: how a server remembers you
- Password hashing with werkzeug: why plain text is malpractice
- Register, login, and logout routes
- Protecting routes: login required, by hand once
- Per-user data: your notes are not my notes
- The complete Flask app: what we built and what Django will automate

Projects built:

- Notes app with accounts: registration, hashed passwords, login, logout, and notes scoped to their owner

Practice: Write a short attack review of your own app: three things a hostile user might try, and what stops each

Assessment: Phase 1 project review: the complete Flask notes app submitted with a README, code-reviewed one on one

The production framework, learned with Flask as the reference point: models, migrations, the admin, real authentication, relationships, and class-based views, ending in a multi-user project.

Month 3: Django Foundations (Weeks 9-12)

Projects, Apps, and URLs

Topics:

- Why Django after Flask: batteries included, and which batteries
- startproject and startapp: what every generated file is for
- URLconf: routing requests to views
- Function-based views and HttpResponse
- Django templates: familiar ground after Jinja2
- settings.py toured honestly: the settings that matter now

Projects built:

- Django hello-world done right: a two-app project with routed pages, shared base template, and static files

Practice: Port your Flask movie catalog's pages to Django views and templates, noting what moved where

Models and Migrations

Topics:

- Defining models with Django fields
- Migrations: what makemigrations and migrate actually do
- The ORM: create, filter, get, exclude, order_by
- The Django shell as your laboratory
- Field options: defaults, null, blank, and choices
- Changing a model after data exists: the migration workflow

Projects built:

- Job board, part one: JobPosting and Company models, migrated, seeded, and queried from the shell

Practice: Fifteen ORM exercises in the Django shell against the job board data, saved as a script

The Admin and Django Templates

Topics:

- The Django admin: a working back office for free
- Registering models and customizing list displays
- Search, filters, and inline editing in the admin
- Wiring list and detail pages to real data
- get_object_or_404 and clean URL patterns with slugs
- Why the admin exists and where its limits are

Projects built:

- Job board, part two: public listing and detail pages, plus a configured admin where postings are managed

Practice: Customize the admin with two list filters and a search field, and add a published/draft toggle that the public site respects

Django Forms

Topics:

- The Form class: declaration instead of repetition
- ModelForm: forms generated from models
- Validation: built-in validators and clean methods
- Rendering forms in templates and displaying errors well
- CSRF protection: what that token was always for
- File uploads, introduced carefully

Projects built:

- Job board, part three: a post-a-job form with validation, plus an application form with resume upload

Practice: Write three custom validation rules, including one clean method that checks two fields against each other

Assessment: Month 3 review: job board demo plus live ORM queries answered in the Django shell

Month 4: Auth, Relationships, and Structure (Weeks 13-16)

Authentication

Topics:

- django.contrib.auth: the battery you were promised
- Login, logout, and signup flows with Django's views and forms
- login_required and testing it properly
- The User model and when to extend it with a profile
- Permissions and groups, introduced with real cases
- Password reset flow, understood even before email is wired

Projects built:

- Job board accounts: employer and candidate signup, login, and pages that respond to who is viewing them

Practice: Protect three routes with different rules: any logged-in user, employers only, and owner only

Relationships

Topics:

- ForeignKey: companies have many postings
- Querying across relationships in both directions
- ManyToManyField: tags and saved jobs

- related_name and readable reverse queries
- The N+1 query problem: found with the debug toolbar, fixed with select_related
- Designing a schema on paper before typing it

Projects built:

- Job board relationships: companies, postings, tags, and saved-jobs wired together with efficient queries

Practice: Hunt N+1 queries on two pages using the debug toolbar, and record the query count before and after fixing

Class-Based Views and Polish

Topics:

- Class-based views: ListView and DetailView without mystery
- CreateView, UpdateView, DeleteView and when generic views earn their keep
- Pagination that holds up with real data
- The messages framework for user feedback
- Query parameters for search and filtering
- Function-based vs class-based: an honest decision guide

Projects built:

- Job board, refactored: listing pages moved to generic views with pagination, search, and tag filtering

Practice: Convert two of your remaining function-based views to class-based views, and write three lines on which reads better

Project Week: Ship the Django App

Topics:

- Feature freeze and a finishing checklist
- Seeding believable demo data with a management command
- Error pages: custom 404 and 500
- A structured self-review before the instructor's review
- Writing a README with setup steps that actually work
- Django's request lifecycle, reviewed now that every piece is familiar

Projects built:

- Complete job board: multi-user Django application with auth, relationships, search, pagination, and admin, delivered with a README

Practice: Follow a classmate's README from git clone to running app, and file issues for every step that failed

Assessment: Phase 2 code review: full walkthrough of the job board with questions on models, queries, and auth decisions

----- PHASE 3: REST APIS AND PRODUCTION (MONTHS 5-6, WEEKS 17-24) -----

The skills backend jobs list by name: REST API design with Django REST Framework, token auth and permissions, then real deployment and a capstone shipped to a public URL.

Month 5: REST APIs with Django REST Framework (Weeks 17-20)

API Thinking

Topics:

- Why APIs exist: mobile apps, frontends, and other servers as clients
- REST conventions: resources, methods, and status codes as a contract
- Designing endpoints on paper: the URL table for the job board
- JSON in and JSON out: serialization as the core problem
- Installing DRF and the browsable API
- Your first APIView returning real data

Projects built:

- API design document plus a first read-only endpoint serving job postings as JSON

Practice: Design the full endpoint table for a library system: paths, methods, status codes, and example payloads

Serializers and CRUD Endpoints

Topics:

- ModelSerializer: validation and conversion in one place
- List and detail endpoints for every resource
- Create, update, and delete through the API
- Nested and related data: choices and trade-offs
- Serializer validation mirroring the form validation you already know
- Testing endpoints with the browsable API and httpie

Projects built:

- Job board API: full CRUD endpoints for postings and companies with proper status codes throughout

Practice: Write an httpie session script that exercises every endpoint, including three requests designed to fail correctly

API Authentication and Permissions

Topics:

- Session auth vs token auth, and why APIs need the latter
- Token authentication set up and tested
- Permission classes: read for all, write for owners
- Custom permissions: employers edit only their own postings
- Throttling: rate limits as basic self-defense
- Consistent, safe error responses

Projects built:

- Secured API: token auth plus owner-scoped write permissions, proven with a scripted

attack-and-verify session

Practice: Write six permission test cases: two allowed, four denied, each asserting the exact status code

Consuming and Documenting the API

Topics:

- A minimal JavaScript page that fetches from your API
- CORS: why the browser said no, and django-cors-headers
- A Python client consuming the same API: one backend, many clients
- Filtering, searching, and pagination in DRF
- Auto-generated API documentation with drf-spectacular
- Versioning and other habits of APIs that live long

Projects built:

- Consumed API: a small JS-driven jobs page and a Python CLI client, both running against your documented API

Practice: Hand your API docs to a classmate and have them build one working request without asking you anything

Assessment: API month review: live demo of the secured, documented API with a Q&A on auth and permission choices

Month 6: Deployment and the Capstone (Weeks 21-24)

Deployment, For Real

Topics:

- Development vs production: DEBUG, secrets, and environment variables
- PostgreSQL: moving off SQLite and why production demands it
- Unicorn: what a WSGI server does that runserver does not
- Static files in production with WhiteNoise
- Deploying the job board to a cloud host, live in class
- Reading server logs when something breaks, because something will

Projects built:

- Deployed job board: the phase 2 project live on a public URL with Postgres, Unicorn, and env-based settings

Practice: Break and fix your deployment once on purpose: bad env var in, error read from logs, corrected, redeployed

Capstone: Scope and Foundations

Topics:

- Choosing a capstone: your own idea, or a track like bookings, forums, or expense tracking
- Writing the spec: models, pages, endpoints, and what done means
- Schema design reviewed before a line of code
- Repository setup and the first deploy in week one, not the last
- Must-have vs nice-to-have, decided while calm
- Instructor sign-off on scope

Projects built:

- Approved capstone spec plus a skeleton project with models migrated and a hello page already deployed

Practice: Build and seed your capstone's data layer, and query it from the shell to prove the schema holds

Capstone: Build and Review

Topics:

- Building vertically: one complete feature at a time
- A first pass at automated tests: models and one endpoint
- Mid-sprint code review with written findings
- Continuous deployment: pushing progress to the live URL
- Handling feedback without derailing the plan
- Cutting scope honestly when the deadline is fixed

Projects built:

- Feature-complete capstone: core features working on the live URL, tests passing, review findings addressed

Practice: Write tests for your two most important behaviors and wire them into a pre-push habit

Ship and Defend

Topics:

- The production checklist: DEBUG off, secrets out of git, error pages, backups considered
- README and API docs finalized
- Presenting a backend project: the demo, the data model, the hardest bug
- Defending technical decisions under friendly fire
- Your GitHub as evidence: six months of reviewed commits
- What comes next: DSA for interviews, deeper Python, or frontend to pair with your backend

Projects built:

- Delivered capstone: a live, documented, tested web application at a public URL, presented and defended

Practice: Send your capstone URL to two people outside the class and fix the first confusing thing each one reports

Assessment: Capstone defense: live demo, data-model walkthrough, and Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Request detective: a full annotated trace of real HTTP traffic
- Movie catalog site with Jinja2 templates and template inheritance

- Feedback board with server-side validation and flash messages
- SQL query workbook against a seeded relational database
- Notes application with full CRUD, sessions, and hashed-password login
- Job board: a complete multi-user Django application with auth, relationships, search, and admin
- Job board REST API: full CRUD with token auth, permissions, and generated documentation
- JS jobs page and Python CLI client consuming the same API
- Deployed job board on a public URL with Postgres and Gunicorn
- Capstone: a self-designed, tested, documented web application shipped to production

Total projects: 12+ real builds across two frameworks, ending with a capstone live on a public URL

SKILLS THE STUDENT WILL HAVE AT THE END

- HTTP fluency: requests, responses, status codes, and debugging traffic like a backend developer
- Flask from scratch: routing, Jinja2 templates, forms, sessions, and password hashing
- SQL and ORMs: handwritten queries first, then SQLAlchemy and the Django ORM with eyes open
- Django in production shape: models, migrations, admin, auth, relationships, and class-based views
- REST APIs with DRF: serializers, token auth, permissions, throttling, and generated docs
- Deployment: environment config, Postgres, Gunicorn, static files, and reading production logs

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building alongside the instructor with code review baked in

Practice: 3-4 hours of project work and exercises between classes

Review: Project submissions reviewed with written findings; recurring issues become the next class's opening

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, because deployment errors do not wait for Tuesday

Progress updates: Regular progress notes covering what was built, what was reviewed, and what needs another pass

Career guidance: Honest guidance on portfolios, resumes for backend roles, and what to learn next, grounded in what each student actually built

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, backed by a deployed capstone and a six-month GitHub history

WHERE THIS CAN LEAD

- Backend or Python developer roles, applied for with a deployed project and a reviewed GitHub history
- Full-stack development: pair this backend depth with our JavaScript and React courses
- Freelance web projects: the build-deploy-maintain cycle this course teaches is the whole freelance job
- Our data structures and algorithms course for product-company interview preparation
- Deeper Python specializations: data engineering, automation, or our AI and machine learning tracks

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-6 LPA (Junior Python Developer / Backend Developer)

Growing roles: Rs 6-12 LPA (Python/Django Developer with 2-4 years of experience)

Experienced roles: Rs 12-25 LPA (Senior Backend Engineer / Lead Developer)

Freelance: Rs 800-2,500/hour for Django and API work, depending on specialization and track record

International: \$70k-140k USD (USA) for backend Python roles, based on role and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How much Python do I need before joining?

A: Comfortable basics: variables, loops, functions, lists, and dictionaries, roughly what a first Python course or one college semester covers. If you are unsure whether you qualify, the free demo class includes a short, low-pressure skills check and an honest recommendation, which is sometimes to take our Python course first.

Q: Why does the course teach Flask and Django instead of just Django?

A: Because Django learned first tends to be memorized, not understood. Flask is small enough that you see every moving part: what a route is, what a session does, what the ORM is actually saving you from. After two months of that, Django's structure reads as a set of solutions to problems you personally hit. Employers get a candidate who can work in either framework and explain the difference, which is a genuinely common interview question.

Q: Is Django still in demand, or has Node.js taken over?

A: Both stacks hire steadily, and pretending otherwise would be dishonest either way. Django remains a standard choice at Indian product companies, agencies, and startups, especially anywhere Python already runs: data-heavy products, fintech, and internal platforms. Instagram and Spotify famously run Python at scale. If your longer-term interest is JavaScript everywhere, our Node.js track fits better; if you know Python and want employable backend skills now, this stack is the shorter path.

Q: Will I learn front-end development too?

A: Enough to ship, not more: the HTML, CSS, and templates your pages need, plus one small JavaScript client in month 5 so you experience your API from the consumer's side. This is deliberately a backend course. Students who want the full stack typically take our React course afterwards, and the capstone API you build here becomes the backend for that work.

Q: What will I actually be able to show a recruiter after six months?

A: A capstone application live at a public URL, a complete Django job board, a documented REST API with token authentication, and a GitHub profile showing six months of steady, code-reviewed commits. In interviews you can walk through your own data model, explain an N+1 query you found and fixed, and describe your deployment step by step, which lands very differently from listing frameworks on a resume.

Q: I work full time. Can I manage this course?

A: The course is built for exactly that constraint: two live one-hour classes a week scheduled in evening-friendly slots, plus 3 to 4 hours of project work you place wherever your week allows. The pace is steady rather than punishing, and WhatsApp doubt support means a blocked Tuesday problem does not have to wait for Thursday's class. Professionals who fall behind a week can use the mini-batch or 1-on-1 formats to catch up.

Q: Can I try a class before enrolling?

A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The demo includes the Python readiness check and a clear walkthrough of how the course is taught and what you will build.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/python-web-development-django-flask-course>

=====

COURSE 105 OF 118: QUEEN CODERS: 1-YEAR ADVANCED TECH LEADERSHIP FOR WOMEN

=====

Course link: <https://learn.modernagecoders.com/courses/queen-coders-advanced-tech-leadership>

Subject area: Advanced Women in Technology - Leadership Program

Designed for: Girls only batches

Who can join: Intermediate Developer to Tech Leader/Architect

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Queen Coders Tech Leadership Certification - Industry Elite

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year advanced coding and leadership program for women: grow from intermediate developer to tech leader with cloud, AI/ML, system design and real apps.

In one line: From Developer to Tech Queen - Master Advanced Technologies and Lead the Industry

ABOUT THE PROGRAMME

Queen Coders is an elite program designed for women who already have programming basics and are ready to ascend to tech leadership. This intensive 1-year journey transforms intermediate developers into architects, tech leads, and entrepreneurs. You'll master cutting-edge technologies like AI/ML, blockchain, cloud architecture, and microservices while developing leadership skills, building startups, and creating industry-changing solutions. Join a powerful network of women tech leaders shaping the future.

WHAT MAKES THIS COURSE DIFFERENT

- Advanced curriculum for women with coding experience
- Tech leadership and architecture focus
- Real industry projects with Fortune 500 companies
- Female tech executives as mentors and instructors
- Startup incubation and funding opportunities
- Global women tech leader network access
- Focus on breaking the glass ceiling in tech
- Board-ready technical and leadership skills

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Advanced Programming, System Design, Cloud Architecture

Phase 2: Specialization (Months 4-6): AI/ML, Blockchain, Data Engineering, DevOps

Phase 3: Leadership (Months 7-9): Tech Leadership, Team Management, Product Strategy

Phase 4: Innovation (Months 10-12): Startup Building, Open Source, Industry Projects

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Bachelor's degree preferred but not required

Coding experience: 2+ years of programming experience required

Equipment: High-performance laptop/desktop, cloud accounts

Time commitment: 15-20 hours per week minimum

English: Professional English proficiency

Motivation: Ambition to become a tech leader

WHO THIS COURSE IS FOR

Students: Women developers ready for leadership

Working professionals: Women seeking senior/executive roles

Entrepreneurs: Women building tech companies

Freelancers: Women consultants going enterprise

Kids: Not suitable for children - adult program

Anyone: Ambitious women ready to lead in tech

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: ADVANCED ENGINEERING & ARCHITECTURE (MONTHS 1-3, WEEKS 1-13) -----

Master advanced programming paradigms, system design, and cloud-native architecture. Build enterprise-grade applications.

Months 1-2: Advanced Programming & System Design (Week 1-8)

Advanced Software Engineering Principles

Topics:

- Welcome to Queen Coders elite community
- Women tech leaders panel and networking
- Advanced data structures: Trees, Graphs, Heaps
- Algorithm complexity and optimization
- Design patterns: Factory, Observer, Strategy, Decorator
- SOLID principles deep dive
- Domain-Driven Design (DDD)
- Event-driven architecture patterns
- Microservices vs Monolithic architecture
- Service mesh and API gateways
- Distributed systems fundamentals
- CAP theorem and consistency models

Projects built:

- Design and implement a distributed cache system
- Build a custom ORM framework
- Create a microservices architecture blueprint
- Implement advanced design patterns library

Practice: Daily: System design problems, Algorithm challenges

Cloud Architecture & Infrastructure

Topics:

- Multi-cloud strategy: AWS, Azure, GCP comparison
- Infrastructure as Code with Terraform
- Advanced Kubernetes: operators, service mesh
- Serverless architecture patterns
- Event-driven with AWS Lambda, EventBridge
- Container orchestration at scale
- Cloud security: IAM, VPC, encryption
- Cost optimization strategies
- Disaster recovery and high availability
- Multi-region deployment strategies
- CDN and edge computing
- Observability: metrics, logs, traces

Projects built:

- Multi-region Kubernetes deployment
- Serverless e-commerce backend
- Infrastructure automation with Terraform
- Cloud cost optimization audit tool

Practice: Deploy 5 different cloud architectures

Advanced Databases & Data Architecture

Topics:

- Database internals: B-trees, LSM trees
- ACID vs BASE consistency models
- Sharding strategies and implementation
- Multi-master replication
- Time-series databases: InfluxDB, TimescaleDB
- Graph databases: Neo4j, Amazon Neptune
- Vector databases for AI: Pinecone, Weaviate
- Data warehousing: Snowflake, Redshift
- Stream processing: Kafka, Kinesis
- Event sourcing and CQRS
- Data lake architecture
- Real-time analytics pipelines

Projects built:

- Build a sharded database system
- Real-time analytics pipeline with Kafka
- Graph-based recommendation engine
- Data warehouse ETL pipeline

Practice: Design 10 complex data architectures

Performance Engineering & Optimization

Topics:

- Performance profiling and benchmarking
- Memory management and garbage collection
- Concurrency patterns and parallel processing
- Lock-free data structures
- Cache optimization strategies
- Database query optimization
- Network optimization techniques
- Load balancing algorithms
- Rate limiting and throttling
- Circuit breaker pattern implementation
- Chaos engineering principles
- Performance testing with JMeter, K6

Projects built:

- Build a high-performance web server
- Optimize a slow application (10x improvement)
- Implement custom load balancer
- Create performance monitoring dashboard

Practice: Optimize 5 production systems

Month 3: Security & Enterprise Architecture (Week 9-13)

Advanced Security Engineering

Topics:

- Security architecture patterns
- Zero-trust architecture implementation
- OAuth 2.0 and OpenID Connect deep dive
- Multi-factor authentication systems
- Encryption: symmetric, asymmetric, hashing
- PKI and certificate management
- Secure coding practices
- OWASP Top 10 prevention strategies
- Penetration testing techniques
- Security incident response
- Compliance: GDPR, HIPAA, SOC2
- Blockchain for security applications

Projects built:

- Build enterprise SSO system
- Implement zero-trust network
- Security audit automation tool
- Encrypted messaging system

Practice: Security audit 10 applications

API Design & GraphQL Mastery

Topics:

- REST API design best practices
- GraphQL schema design patterns
- Federation and schema stitching
- Real-time subscriptions with GraphQL
- API versioning strategies
- API gateway patterns: Kong, Zuul
- Rate limiting and quotas
- API documentation with OpenAPI
- gRPC for microservices

- WebSocket architecture
- Server-sent events (SSE)
- API testing and mocking

Projects built:

- Design federated GraphQL architecture
- Build API gateway from scratch
- Real-time collaboration API
- Multi-protocol API server

Practice: Design and implement 5 complex APIs

Enterprise Integration Patterns

Topics:

- Message queue architectures: RabbitMQ, SQS
- Event streaming with Kafka
- Saga pattern for distributed transactions
- Service discovery and registry
- API orchestration vs choreography
- Integration with legacy systems
- ETL vs ELT strategies
- Change data capture (CDC)
- Enterprise service bus (ESB)
- Workflow engines: Airflow, Temporal
- Distributed tracing with Jaeger
- Service mesh: Istio, Linkerd

Projects built:

- Build event-driven architecture
- Implement saga pattern
- Create workflow orchestration system
- Design service mesh architecture

Practice: Implement 5 integration patterns

Phase 1 Comprehensive Project

Topics:

- System architecture documentation
- Technical design reviews
- Code quality and standards
- Performance benchmarking
- Security assessment
- Deployment automation
- Monitoring and alerting setup
- Technical presentation skills

Projects built:

- CAPSTONE: Enterprise-Grade Distributed System
- Requirements: Microservices, Cloud-native, Secure, Scalable, Observable
- Present to industry panel

Assessment: Advanced Architecture Certification

Bridge to AI/ML & Emerging Tech

Topics:

- AI/ML landscape for engineers
- Blockchain technology overview
- Quantum computing basics
- Edge computing trends
- 5G and IoT implications
- Preparing for specialization

Projects built:

- Research emerging technology
- Proof of concept implementation

Assessment: Technology trend analysis

Months 4-6: AI/ML & Agent Specialization (Week 14-26)

Machine Learning Engineering

Topics:

- ML system design and architecture
- Feature engineering at scale
- Model training pipelines
- TensorFlow and PyTorch deep dive
- Neural network architectures
- CNNs for computer vision
- RNNs and LSTMs for sequences
- Transformers and attention mechanisms
- Model optimization and quantization
- MLOps: versioning, deployment, monitoring
- A/B testing for ML models
- Explainable AI (XAI)

Projects built:

- Build ML pipeline from scratch
- Deploy computer vision system
- Create NLP application
- Implement recommendation system

Practice: Deploy 5 ML models to production

Advanced AI Applications

Topics:

- Large Language Models (LLMs) integration
- Fine-tuning pre-trained models
- Prompt engineering techniques
- RAG (Retrieval Augmented Generation)
- Vector databases for AI
- Computer vision applications
- Speech recognition and synthesis
- Reinforcement learning basics
- Generative AI: GANs, VAEs
- AI ethics and bias mitigation
- Federated learning
- Edge AI deployment

Projects built:

- Build AI-powered chatbot with RAG
- Create custom LLM application
- Implement computer vision solution
- Deploy edge AI system

Practice: Build 5 AI-powered applications

AI Engineering & Agent Development

Topics:

- LLM architecture deep dive: what the model actually does
- Prompting as engineering, not incantation
- Structured output and schema-constrained generation
- Function calling and tool use
- The agent loop: plan, act, observe, correct
- Retrieval-augmented generation over real corpora
- Vector stores, chunking, and when retrieval is the wrong answer
- Multi-agent orchestration and where it fails
- Evaluation: building an eval set before you build the feature
- Guardrails, injection and untrusted input
- Cost, latency and token budgets in production
- Observability: tracing a non-deterministic system

Projects built:

- Retrieval-backed assistant
- Tool-using agent
- Eval harness with a real dataset
- Production AI service with guardrails

Practice: Deploy 5 smart contracts

Data Engineering & Analytics

Topics:

- Data pipeline architecture
- Apache Spark for big data
- Real-time processing with Flink
- Data lakehouse architecture
- Delta Lake and Apache Iceberg
- Orchestration with Apache Airflow
- Data quality and governance
- Business intelligence tools
- Advanced SQL and query optimization
- Time-series analysis
- Predictive analytics
- Data visualization best practices

Projects built:

- Build complete data platform
- Real-time analytics dashboard
- Predictive maintenance system
- Data quality monitoring tool

Practice: Build 5 data pipelines

Phase 2 Innovation Project

Topics:

- Identifying market opportunities
- Technical feasibility analysis
- MVP development strategy
- User testing and iteration
- Product launch preparation
- Investor pitch preparation

Projects built:

- MAJOR CAPSTONE: AI/Blockchain Innovation
- Build production-ready product using AI or Blockchain
- Present to investors and industry leaders

Assessment: Emerging Technology Specialist Certification

PHASE 2: SPECIALIZED TECHNOLOGIES & INNOVATION (MONTHS 4-6, WEEKS 14-26)

Master cutting-edge technologies including AI/ML, blockchain, and data engineering while building industry-changing solutions.

Months 4-5: Deep Specialization (Week 14-21)

Advanced Machine Learning & Deep Learning

Topics:

- Deep learning architectures: CNN, RNN, Transformer
- Transfer learning and fine-tuning strategies
- Model optimization: pruning, quantization, distillation
- Distributed training with Horovod
- AutoML and neural architecture search
- Adversarial machine learning
- Federated learning implementation
- Online learning and streaming ML
- Multi-modal learning: vision + language
- Graph neural networks
- Reinforcement learning applications
- Production ML monitoring and drift detection

Projects built:

- Build custom deep learning framework
- Multi-modal AI application
- Distributed training pipeline
- AutoML platform

Practice: Implement 10 state-of-the-art papers

LLMs & Generative AI

Topics:

- Transformer architecture deep dive
- Building with GPT, BERT, T5
- Fine-tuning strategies for LLMs
- Prompt engineering advanced techniques
- RAG systems architecture
- Vector search optimization
- LangChain and semantic kernel
- Multi-agent AI systems
- Constitutional AI and alignment
- Diffusion models for generation
- Voice synthesis and cloning
- Cost optimization for LLM apps

Projects built:

- Custom LLM fine-tuning pipeline
- Enterprise RAG system
- Multi-agent AI platform
- Generative AI creative tool

Practice: Build 5 production LLM applications

AI Agent Architecture at Scale

Topics:

- Designing an agent system that survives contact with users
- Task decomposition and planner/executor splits
- Memory: short-term, long-term, and why most of it is a cache
- Tool design: idempotency, permissions and blast radius
- Human-in-the-loop checkpoints for irreversible actions
- Prompt injection and the confused-deputy problem
- Sandboxing an agent that can run code
- Failure modes: loops, drift and confident wrongness
- Model routing and fallback across providers
- Batching, caching and cost control at scale
- Red-teaming your own agent
- Accountability: who is responsible when it acts wrongly

Projects built:

- Multi-step agent with tool permissions
- Sandboxed code-running agent
- Model-routing gateway
- Agent red-team report

Practice: Audit 10 smart contracts

Cloud-Native & Platform Engineering

Topics:

- Platform engineering principles
- Internal developer platforms (IDP)
- GitOps with ArgoCD, Flux
- Service mesh advanced patterns
- Multi-tenancy architectures
- Cost optimization automation
- FinOps practices
- Compliance as code
- Disaster recovery automation
- Multi-cloud management
- Cloud-native security
- Platform metrics and KPIs

Projects built:

- Build internal developer platform
- Multi-cloud management system
- Cost optimization platform

- Compliance automation tool

Practice: Design 5 platform architectures

Quantum Computing & Future Tech

Topics:

- Quantum computing fundamentals
- Quantum algorithms: Shor's, Grover's
- Quantum machine learning
- Quantum cryptography
- Neuromorphic computing
- Brain-computer interfaces
- Augmented reality development
- Digital twin technology
- 6G and future networks
- Synthetic biology computing
- Space tech applications
- Future technology trends

Projects built:

- Quantum algorithm implementation
- AR/VR application
- Digital twin prototype
- Future tech research project

Practice: Explore 5 emerging technologies

Month 6: Advanced Integration & Platforms (Week 22-26)

Enterprise Architecture Patterns

Topics:

- TOGAF framework overview
- Enterprise integration patterns
- Domain-driven design at scale
- Event-driven architecture patterns
- CQRS and event sourcing
- Micro-frontends architecture
- Backend for frontend (BFF) pattern
- API composition patterns
- Data mesh architecture
- Platform team topologies
- Conway's law in practice
- Architecture decision records (ADR)

Projects built:

- Design enterprise architecture
- Implement event-driven system
- Build micro-frontend platform
- Create architecture documentation

Practice: Document 5 architecture decisions

DevOps & SRE Excellence

Topics:

- Site Reliability Engineering principles
- SLIs, SLOs, and error budgets
- Incident management and postmortems
- Chaos engineering implementation
- Progressive delivery: canary, blue-green
- Feature flags and experimentation
- Observability engineering
- Distributed tracing deep dive
- Performance engineering culture
- Capacity planning and scaling
- Cost optimization strategies
- DevSecOps implementation

Projects built:

- Build SRE platform
- Chaos engineering framework
- Observability platform
- Incident management system

Practice: Implement SRE for 5 systems

Technical Leadership Skills

Topics:

- Technical decision making
- Architecture review boards
- RFC and design doc writing
- Code review best practices
- Mentoring and coaching developers
- Building high-performing teams
- Stakeholder management
- Technical debt management
- Innovation frameworks
- Budget and resource planning
- Vendor evaluation and selection
- Open source strategy

Projects built:

- Lead architecture review
- Create mentorship program
- Write technical RFC
- Build team playbook

Practice: Lead 5 technical initiatives

Product Engineering & Strategy

Topics:

- Product-minded engineering
- User research for engineers
- A/B testing infrastructure
- Feature prioritization frameworks
- Growth engineering tactics
- Metrics and analytics systems
- Product-market fit indicators
- Platform vs product thinking
- API as a product
- Developer experience (DX)
- Technical product management
- Go-to-market for technical products

Projects built:

- Build A/B testing platform
- Create analytics system
- Design API product
- Launch technical product

Practice: Ship 5 product features with metrics

Phase 2 Platform Launch

Topics:

- Production readiness checklist
- Launch planning and coordination
- Marketing for technical products
- Community building strategies
- Documentation and tutorials
- Support and maintenance planning

Projects built:

- CAPSTONE: Production Platform Launch
- Build and launch a complete platform/product
- Include: Architecture, Implementation, Deployment, Marketing

Assessment: Platform Engineering Certification

Months 7-9: Leadership & Management (Week 27-39)

Engineering Leadership

Topics:

- From IC to leadership transition
- Leading without authority
- Building engineering culture
- Hiring and interviewing
- Performance management
- Difficult conversations
- Delegation and empowerment
- Strategic thinking for engineers
- Executive communication
- Board presentation skills
- Managing up and sideways
- Women in leadership challenges

Projects built:

- Create hiring framework
- Build team charter
- Design performance system
- Leadership portfolio

Practice: Lead team of 5+ developers

Startup Engineering & Entrepreneurship

Topics:

- Idea validation and MVP
- Technical co-founder responsibilities
- Building from 0 to 1
- Startup architecture patterns
- Cost-effective tech stacks
- Rapid prototyping techniques
- Customer development
- Fundraising for technical founders
- Pitch deck creation
- Cap table and equity
- Acquisition and exit strategies
- Women founders ecosystem

Projects built:

- Launch startup MVP
- Create investor pitch
- Build prototype product
- Fundraising preparation

Practice: Validate 5 startup ideas

Open Source & Community Leadership

Topics:

- Starting successful open source projects
- Building contributor community
- Open source governance models
- License selection and implications
- Monetizing open source
- Developer advocacy
- Conference speaking mastery
- Technical writing and blogging
- Building personal brand
- Influencer strategies for developers
- Podcast and video content
- Women in open source

Projects built:

- Launch open source project
- Speak at conference
- Build developer community
- Create content strategy

Practice: Contribute to 10 major projects

Innovation & R&D Leadership

Topics:

- Innovation frameworks
- Research methodologies
- Patent strategies
- Building innovation labs
- Hackathon organization
- University partnerships
- Government grants and funding
- Technology transfer
- Commercialization strategies
- Building research teams
- Publishing papers
- Industry-academia bridge

Projects built:

- File patent application
- Organize hackathon
- Research publication
- Innovation lab setup

Practice: Lead 3 innovation initiatives

Phase 3 Leadership Summit

Topics:

- Leadership philosophy
- Building your advisory board
- Board readiness training
- C-suite communication
- Industry influence strategies
- Policy and regulation impact
- Global technology leadership

Projects built:

- CAPSTONE: Leadership Initiative
- Options: Start company, Lead open source project, Launch innovation lab
- Present at Queen Coders Summit

Assessment: Technical Leadership Certification

PHASE 3: LEADERSHIP & STRATEGIC TECHNOLOGY (MONTHS 7-9, WEEKS 27-39)

Develop executive leadership skills, master strategic technology decisions, and prepare for C-level positions.

Months 7-8: Technical Leadership Excellence (Week 27-34)

Staff/Principal Engineering Skills

Topics:

- Staff+ engineering career paths
- Technical leadership without management
- Driving technical strategy
- Cross-team collaboration
- Technical mentorship at scale
- Writing engineering strategies
- Building technical roadmaps
- Influencing without authority
- Managing technical debt strategically
- Architecture governance
- Technical risk management
- Being a force multiplier

Projects built:

- Create technical strategy doc

- Lead cross-team initiative
- Build mentorship program
- Design technical roadmap

Practice: Lead 5 strategic initiatives

Engineering Management Mastery

Topics:

- Building high-performance teams
- Agile at scale: SAFe, LeSS
- OKRs and goal setting
- Resource planning and allocation
- Budget management for engineering
- Vendor and contractor management
- Remote team leadership
- Diversity and inclusion initiatives
- Engineering productivity metrics
- Developer experience optimization
- Burnout prevention strategies
- Succession planning

Projects built:

- Design team structure
- Implement OKR system
- Create D&I initiative
- Build productivity dashboard

Practice: Manage team of 10+ engineers

VP/Director Level Skills

Topics:

- Department-level strategy
- Multi-team coordination
- Executive reporting and metrics
- Board presentation skills
- Stakeholder management
- Cross-functional leadership
- M&A technical due diligence
- Building engineering brand
- University relations programs
- Patent and IP strategy
- Regulatory compliance leadership
- Crisis management

Projects built:

- Department strategy presentation
- M&A evaluation framework
- Engineering brand campaign
- Crisis response playbook

Practice: Present to executive team

CTO/VP Engineering Preparation

Topics:

- CTO vs VP Engineering roles
- Technology vision and strategy
- Building innovation culture
- Digital transformation leadership
- Buy vs build decisions
- Technology partnerships
- Analyst relations (Gartner, Forrester)
- Customer advisory boards
- Investor relations for CTOs
- Public speaking and thought leadership
- Media training for executives
- Building your personal board

Projects built:

- Create technology vision
- Digital transformation plan
- Thought leadership content
- Advisory board formation

Practice: Shadow C-level executive

Board Readiness & Governance

Topics:

- Board dynamics and responsibilities
- Technology committee participation
- Cybersecurity governance
- Risk management frameworks
- SOX compliance for technology
- ESG and sustainability in tech
- Presenting to the board
- Board package preparation
- Audit committee interaction
- Director and officer insurance
- Fiduciary responsibilities
- Board networking strategies

Projects built:

- Board presentation deck
- Risk assessment framework
- Technology governance charter
- Board readiness portfolio

Practice: Mock board presentations

Month 9: Strategic Innovation & Impact (Week 35-39)

Corporate Innovation Leadership

Topics:

- Building innovation ecosystems
- Corporate venture capital
- Accelerator and incubator programs
- Strategic partnerships
- Technology scouting
- Disruptive innovation theory
- Digital transformation strategies
- Change management for technology
- Innovation metrics and KPIs
- Failing fast and pivot strategies
- Intrapreneurship programs
- Innovation lab management

Projects built:

- Innovation program design
- Partnership framework
- Accelerator curriculum
- Innovation metrics system

Practice: Launch 3 innovation initiatives

Technology for Social Impact

Topics:

- Tech for good initiatives
- Sustainable technology strategies
- Accessibility and inclusive design
- Digital divide solutions
- Healthcare technology innovation
- Education technology leadership
- Climate tech opportunities
- Civic tech and government
- Non-profit technology strategy
- Impact measurement frameworks
- B-Corp and benefit corporations
- UN SDGs and technology

Projects built:

- Social impact tech solution
- Accessibility audit framework
- Sustainability initiative
- Impact measurement system

Practice: Launch social impact project

Global Technology Leadership

Topics:

- Global team management
- Cross-cultural communication
- International expansion strategies
- Offshore and nearshore strategies
- Global compliance and regulations
- Multi-language and localization
- Global infrastructure patterns
- Time zone collaboration
- Cultural sensitivity in tech
- Global talent acquisition
- International partnerships
- Emerging market strategies

Projects built:

- Global expansion plan
- International team structure
- Localization strategy
- Global partnership framework

Practice: Lead global initiative

Women Tech Leaders Network

Topics:

- Building women in tech communities
- Sponsorship vs mentorship
- Overcoming gender bias in tech
- Negotiation strategies for women
- Building male allies
- Return to work programs
- Supporting working mothers in tech
- Breaking the glass ceiling
- Women on boards initiatives
- Venture capital for women

- Women-led startups
- Global women in tech movements

Projects built:

- Women in tech initiative
- Mentorship program design
- Diversity metrics system
- Allyship training program

Practice: Mentor 5 women in tech

Phase 3 Executive Presentation

Topics:

- Executive presence development
- Strategic communication
- Influence and persuasion
- Executive networking
- Personal brand as executive
- Media and PR strategies
- Industry leadership positioning

Projects built:

- CAPSTONE: Executive Leadership Portfolio
- Include: Strategy doc, Innovation initiative, Team leadership, Board presentation
- Present to industry executives and investors

Assessment: Executive Leadership Certification

Months 10-12: Entrepreneurship & Industry Leadership (Week 40-52)

Tech Entrepreneurship Mastery

Topics:

- Venture capital fundamentals
- Startup financial modeling
- Term sheets and negotiations
- Series A to IPO journey
- Cap table management
- Founder agreements and vesting
- Intellectual property strategies
- Building advisory boards
- Scaling engineering teams
- Product-market fit metrics
- Growth hacking strategies
- Exit strategies and M&A

Projects built:

- Startup business plan
- Financial model creation
- Fundraising pitch deck
- Term sheet negotiation

Practice: Launch startup venture

Venture Capital & Investment

Topics:

- Angel investing basics
- Evaluating technical startups
- Due diligence processes
- Portfolio management
- Syndicate formation
- Crowdfunding strategies
- Corporate venture capital
- LP relationships
- Fund formation basics
- Investment thesis development
- Women in venture capital
- Impact investing

Projects built:

- Investment thesis document
- Due diligence framework
- Angel investment analysis
- Portfolio strategy

Practice: Evaluate 10 startups

Industry Influence & Thought Leadership

Topics:

- Building thought leadership
- Writing for major publications
- Keynote speaking mastery
- Podcast hosting and guesting
- Book writing strategies
- Research and white papers
- Industry analyst relations
- Awards and recognition
- Professional associations
- Standards body participation
- Policy influence
- Technology evangelism

Projects built:

- Thought leadership strategy
- Keynote presentation
- Published article/white paper
- Industry award submission

Practice: Speak at 3 industry events

Building Your Legacy

Topics:

- Defining your mission and values
- Creating lasting impact
- Succession planning
- Mentoring next generation
- Philanthropy and giving back
- Building institutions
- Creating scholarship programs
- Authoring technical standards
- Patent portfolio development
- Open source legacy projects
- Educational content creation
- Documentary participation

Projects built:

- Legacy project definition
- Scholarship program design
- Mentorship scale program
- Knowledge repository

Practice: Launch legacy initiative

Final Capstone - Industry Impact

Topics:

- Capstone project planning
- Industry partnership formation
- Solution architecture
- Implementation leadership
- Impact measurement
- Stakeholder management
- Launch and marketing
- Success metrics tracking

Projects built:

- INDUSTRY CAPSTONE: Enterprise Solution
- Partner with Fortune 500 company
- Solve real industry problem
- Lead team implementation

Assessment: Industry Leadership Certification

----- PHASE 4: INDUSTRY LEADERSHIP & GLOBAL IMPACT (MONTHS 10-12, WEEKS 40-52) -----

Establish yourself as an industry leader, build influential networks, and create lasting impact in the technology world.

Months 10-11: Executive Excellence (Week 40-47)

C-Suite Technology Leadership

Topics:

- CEO/CTO/CIO relationships
- Technology in corporate strategy
- Digital business models
- Platform economy strategies
- Data monetization strategies
- AI strategy for enterprises
- Blockchain in enterprise
- Quantum readiness planning
- Technology debt at scale
- Multi-billion dollar decisions
- Wall Street and technology
- Shareholder value creation

Projects built:

- Corporate tech strategy
- Digital transformation roadmap
- Innovation portfolio design
- Shareholder presentation

Practice: Lead enterprise initiative

Global Women Tech Leaders

Topics:

- Women CEOs and CTOs panel
- Building inclusive tech companies
- Gender equity in technology
- Women-led unicorns
- Female founder challenges
- Maternity and leadership
- #MeToo in tech
- Ally development programs

- Women's ERGs leadership
- Global women's tech networks
- UN Women and technology
- 50/50 pledge initiatives

Projects built:

- Women's leadership initiative
- Inclusive culture program
- Gender equity metrics
- Global network building

Practice: Lead diversity initiative

Venture Building & Scaling

Topics:

- Venture studio model
- Serial entrepreneurship
- Building unicorns
- Hypergrowth management
- International expansion
- M&A strategy and execution
- IPO preparation and process
- Private equity partnerships
- Strategic investor relations
- Regulatory navigation
- Crisis management at scale
- Turnaround strategies

Projects built:

- Venture studio blueprint
- Scale-up strategy
- M&A evaluation
- IPO readiness assessment

Practice: Advisory role in startup

Technology Policy & Regulation

Topics:

- Tech policy landscape
- Regulatory compliance at scale
- Privacy laws globally
- AI governance and ethics
- Antitrust considerations
- Content moderation policies
- Cybersecurity regulations
- Government relations
- Lobbying and advocacy
- Standards development
- International trade and tech
- Future of regulation

Projects built:

- Policy position paper
- Compliance framework
- Government relations strategy
- Standards contribution

Practice: Engage with policymakers

Personal Brand as Tech Leader

Topics:

- Executive personal branding
- Media training mastery
- Crisis communication
- Social media for executives
- Thought leadership content
- Book deals and publishing
- Documentary participation
- TED talk preparation
- Podcast hosting
- Press relationships
- Awards and recognition
- Legacy building

Projects built:

- Personal brand strategy
- Media kit creation
- TED talk proposal
- Book outline

Practice: Weekly thought leadership content

Month 12: Graduation & Global Impact (Week 48-52)

Final Capstone - Industry Transformation

Topics:

- Identifying industry problems
- Building coalition of partners
- Solution architecture at scale
- Resource mobilization
- Change management

- Implementation roadmap
- Success metrics definition
- Stakeholder alignment
- Risk mitigation strategies
- Communication planning
- Launch preparation
- Impact measurement

Projects built:

- TRANSFORMATION CAPSTONE:
- Lead industry-changing initiative
- Multi-company collaboration
- \$1M+ impact potential

Practice: Daily capstone development

Queen Coders Summit Preparation

Topics:

- Summit presentation prep
- Panel discussion training
- Networking strategies
- Investor pitch preparation
- Demo day organization
- Press and media training
- Award nominations
- Partnership announcements
- Product launches
- Strategic alliances
- Future commitments
- Community building

Projects built:

- Summit presentation
- Demo preparation
- Partnership agreements
- Launch announcement

Practice: Presentation rehearsals

Career Acceleration & Placement

Topics:

- C-level job search strategies
- Executive search firms
- Board position seeking
- Compensation negotiation
- Equity packages
- Non-compete navigation
- Reference strategies
- Multiple offer management
- Relocation packages
- Retention bonuses
- Golden parachutes
- Success planning

Projects built:

- Executive job search plan
- Compensation research
- Network activation
- Interview preparation

Practice: Executive interviews

Queen Coders Global Summit

Topics:

- Keynote presentations
- Industry leader panels
- Investor meetings
- Press conferences
- Award ceremonies
- Product showcases
- Partnership signings
- Alumni reunions
- Mentorship matching
- Job fair with top companies
- Startup pitch competition
- Global celebration

Deliverables:

- Capstone presentation
- Industry recognition
- Media coverage
- Partnership deals
- Job offers
- Investment opportunities

Graduation & Coronation Ceremony

Topics:

- Graduation ceremony
- Queen Coders crown ceremony
- Industry recognition awards

- Alumni induction
- Lifetime membership benefits
- Continued learning paths
- Mentorship commitments
- Give-back pledges
- Future collaboration
- Network activation
- Legacy projects
- Next chapter planning

Deliverables:

- Queen Coders Certification
- Industry Leadership Badge
- Alumni Network Access
- Lifetime Learning Resources
- Executive Connections
- Crown of Achievement

Assessment: Queen Coders Tech Leader - Ready to Rule the Industry!

Beyond Graduation: Lifetime Queendom (Ongoing Elite Access)

Queen's Advisory Council

Topics:

- Joining advisory boards
- Consulting opportunities
- Speaking engagements
- Investment opportunities
- Mentorship at scale
- Industry influence
- Policy participation
- Research collaboration
- Book writing
- Media participation
- Award nominations
- Legacy building

Projects built:

- Advisory position
- Consulting practice
- Speaking tour
- Investment portfolio

Practice: Continuous leadership

Queen's Innovation Lab

Topics:

- Research partnerships
- Patent development
- Startup incubation
- Venture partnerships
- University collaboration
- Grant writing
- Innovation challenges
- Hackathon sponsorship
- Technology scouting
- Trend forecasting
- Future visioning
- Disruption planning

Projects built:

- Innovation project
- Patent filing
- Startup mentorship
- Research publication

Practice: Innovation leadership

Queen's Global Network

Topics:

- Global chapter leadership
- International partnerships
- Cross-border collaboration
- Cultural exchange programs
- Global conferences
- International awards
- UN partnerships
- World Bank projects
- Global impact initiatives
- International media
- Diplomatic tech roles
- Peace through technology

Projects built:

- Global initiative
- International partnership
- Cross-border project
- Global impact measure

Practice: Global leadership

Queen's Eternal Legacy

Topics:

- Defining your legacy
- Institution building
- Endowment creation
- Scholarship funds
- Research centers
- Innovation labs
- Mentorship programs
- Book authoring
- Documentary creation
- Museum exhibitions
- Hall of fame
- Eternal impact

Projects built:

- Legacy foundation
- Scholarship program
- Innovation center
- Autobiography

Practice: Building eternal impact

The Queen Coders Dynasty (Lifetime Reign)

Queens Changing the World

Topics:

- Climate technology leadership
- Healthcare transformation
- Education revolution
- Financial inclusion
- Social justice tech
- Accessibility innovation
- Space technology
- Ocean conservation tech
- Food security solutions
- Energy transformation
- Peace technology
- Human rights tech

Projects built:

- World-changing initiative
- Global impact project
- Humanitarian technology
- Sustainable solution

Queens Building Empires

Topics:

- Multi-company leadership
- Conglomerate building
- Investment empire
- Media empire creation
- Education empire
- Technology empire
- Social empire
- Innovation empire
- Influence empire
- Knowledge empire
- Network empire
- Legacy empire

Deliverables:

- Empire blueprint
- Growth strategy
- Succession planning
- Dynasty creation

Queens Forever - Your Reign Begins

Topics:

- Your throne awaits
- Industry domination
- Global influence
- Generational wealth
- Lasting impact
- Historical significance
- Role model status
- Industry icon
- Technology royalty
- Innovation goddess
- Digital queen
- Forever a Queen Coder

Deliverables:

- Lifetime achievement
- Industry recognition
- Global influence
- Eternal legacy

- Queen's crown
- Royal network

Assessment: Queen Coder Crowned - Long May You Reign in Tech!

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1-3: Distributed Systems, Cloud Platforms, ML Pipeline, Blockchain Protocol
- Month 4-6: AI Products, Agent Platforms, Data Platforms, Innovation Projects
- Month 7-9: Leadership Initiatives, Open Source Projects, Startup Ventures
- Month 10-12: Industry Transformations, Global Initiatives, Legacy Projects
- Capstone: Industry-Changing Solution with Real Impact

Total projects: 50+ enterprise-grade projects with real-world impact

SKILLS THE STUDENT WILL HAVE AT THE END

- Architecture: System Design, Cloud, Microservices, Event-Driven, Serverless
- AI/ML: Deep Learning, LLMs, Computer Vision, NLP, MLOps, Production AI
- AI engineering: agents, tool use, retrieval, evals, guardrails
- Data: Big Data, Real-time Processing, Analytics, Data Platforms
- Leadership: Team Management, Strategic Planning, Executive Communication
- Innovation: R&D, Patents, Startups, Venture Building, Disruption
- Impact: Social Good, Sustainability, Accessibility, Global Solutions

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 5-7 hours
 Hands on coding: 8-10 hours
 Projects: 3-5 hours
 Leadership activities: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly sessions with industry leaders
 Mentorship: C-level female executives as mentors
 Community: Exclusive Queen Coders executive network
 Code review: Architecture reviews by principal engineers
 Lifetime access: Forever access to elite network and resources
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase (4 total)
 Final certificate: Queen Coders Tech Leadership Certificate
 LinkedIn badge: Verified Executive Certification
 Industry recognized: Recognized by Fortune 500 companies
 Portfolio projects: 50+ production-grade solutions

WHERE THIS CAN LEAD

- Chief Technology Officer (CTO)
- VP of Engineering
- Principal/Staff Engineer
- Technical Founder/CEO
- Engineering Director
- Solution Architect
- AI/ML Engineer Lead
- Blockchain Architect
- Venture Partner (Technical)
- Board Member (Technology Committee)

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
 Growing roles: Rs 15-25 LPA (Senior roles)
 Experienced roles: Rs 25-50+ LPA (Leadership roles)
 Freelance: Rs 2,000-5,000/hour consulting
 International: \$150k-500k+ USD for leadership positions

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the Queen Coders program cover?

A: This is an advanced 1-year program for women who already code: system design, security,

cloud architecture, AI/ML and blockchain specializations, microservices, technical leadership, and entrepreneurship. The year ends with production-grade projects and a leadership portfolio rather than beginner exercises.

Q: Who is Queen Coders for, and what experience is required?

A: It is built for women with around 2 or more years of programming experience: developers aiming for senior, architect, or leadership roles, and founders building tech products. It is an adult program and not suitable for children; complete beginners should start with our Princess Coders track instead.

Q: Why a women-only advanced program?

A: Fewer women reach senior engineering and architecture roles, and that gap is rarely about ability. Queen Coders concentrates on exactly the skills that unlock those rooms, system design, strategy, and leadership, practiced inside a network of ambitious women who push each other forward.

Q: How much time does the program demand?

A: Plan for 15 to 20 hours a week including live classes and project work. The pace is intentionally rigorous because the outcome is a leadership-grade portfolio: production applications, architecture case studies, and a capstone you can defend in any technical discussion.

Q: What does the program lead to?

A: Graduates finish with production-grade projects, architecture experience, and leadership training aimed at senior engineering, tech lead, and founder paths. You also remain part of the Queen Coders network of women in tech after graduation.

Q: Can I experience a class before enrolling?

A: Yes, book a free demo class at learn.modernagecoders.com/contact with no commitment. It is a live session, so you can gauge the level, meet the instructor, and decide whether the program matches your goals.

Q: What is the teaching format?

A: Live 1-hour online classes with a real instructor, never pre-recorded content. Cohorts are deliberately small and discussion-heavy, closer to an architecture review than a lecture.

Q: Is there certification at the end?

A: Yes, graduates earn the Queen Coders Tech Leadership certification. The stronger credential, though, is the portfolio: production-grade applications and architecture work you can walk through in any senior-level conversation.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/queen-coders-advanced-tech-leadership>

=====

COURSE 106 OF 118: REACT FOR TEENS: BUILD REAL APPS, ZERO TO PRO DEVELOPER

=====

Course link: <https://learn.modernagecoders.com/courses/react-for-teens-complete-masterclass>

Subject area: Teen Web Development & React Programming

Designed for: Teens

Who can join: Complete Beginner to Professional React Developer

Course length: 12 months (52 weeks)

Time needed: 10-15 hours/week recommended

Certificate: Teen React Developer Certification - Industry Recognized

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year React course for teens: go from your first line of code to professional apps and Instagram, TikTok and Spotify clones with modern JavaScript.

In one line: Build Apps Like Instagram, TikTok, Discord, and More!

ABOUT THE PROGRAMME

This isn't your typical boring programming course. Designed specifically for teenagers (ages 13-19), this comprehensive React masterclass takes you from absolute zero to building the apps you use every day. Whether you dream of creating the next viral app, want to impress colleges with your coding skills, or start earning money as a developer, this course is your complete roadmap. You'll learn by building fun, relevant projects like social media apps, gaming dashboards, music players, and chat applications. By the end, you'll have 30+ impressive projects for your portfolio and the skills to land internships or freelance work.

WHAT MAKES THIS COURSE DIFFERENT

- Designed specifically for teenage learners with relevant, engaging content
- Build clones of apps teens actually use (Instagram, TikTok, Discord, Spotify)
- Learn at your own pace with bite-sized lessons perfect for busy school schedules
- Game-like progression system with achievements and badges
- Teen-friendly community with peers your age
- Projects that impress college admissions and employers
- Potential to earn money through freelancing before graduation
- Parents dashboard to track progress (optional)

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Web Basics, HTML, CSS, JavaScript Fundamentals

Phase 2: React Core (Months 4-6): React Basics, Components, Hooks, Routing

Phase 3: Advanced React (Months 7-9): State Management, APIs, Real-time Features

Phase 4: Pro Skills (Months 10-12): Production Apps, React Native, Career Prep

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No coding experience needed - complete beginner friendly

Coding experience: Start from absolute zero

Equipment: Computer/laptop with internet

Time commitment: 10-15 hours per week (manageable with school)

English: Basic English understanding

Motivation: Curiosity and desire to build cool stuff

WHO THIS COURSE IS FOR

Students: Middle school and high school students (13-19)

Working professionals: Young adults wanting to learn React

Entrepreneurs: Teens with app ideas

Freelancers: Students wanting to earn money coding

Kids: Mature pre-teens (12+) with parental guidance

Anyone: Anyone young at heart who loves technology

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: WEB FOUNDATIONS & JAVASCRIPT (MONTHS 1-3, WEEKS 1-13) -----

Start from absolute zero and build the foundation you need for React. Learn how websites work and master JavaScript through fun, interactive projects.

Months 1-2: HTML, CSS & Web Basics (Week 1-8)

Your First Steps in Web Development

Topics:

- How the internet works - simplified for teens
- What happens when you visit Instagram or TikTok
- Setting up your coding environment (VS Code)
- Your first HTML page - creating a personal profile
- HTML tags that matter: headings, paragraphs, images
- Adding links and navigation to pages
- Creating lists (perfect for todo apps later)
- HTML forms - how login pages work
- Embedding YouTube videos and Spotify playlists
- Making your page accessible to everyone
- File organization and naming conventions
- Using GitHub like a pro developer

Projects built:

- Personal profile page about your interests
- Fan page for your favorite artist/game/show
- School club or team website

Practice: Daily: Create one new HTML page about topics you love

CSS - Making Things Look Amazing

Topics:

- CSS basics - adding style to your pages
- Colors and backgrounds - creating themes
- Typography - choosing fonts like a designer
- The Box Model - spacing elements perfectly
- Flexbox - Instagram-style photo grids
- CSS Grid - Pinterest-like layouts
- Hover effects and transitions
- Animations - making elements move and bounce
- CSS variables for dark/light themes
- Mobile-responsive design for phones
- CSS frameworks introduction (Tailwind)
- Design principles teens should know

Projects built:

- Animated landing page for a fake startup
- Instagram-style photo gallery
- Responsive blog design with dark mode

Practice: Recreate UI elements from popular apps using CSS

Advanced CSS & Modern Web Design

Topics:

- CSS animations and keyframes
- Transform and 3D effects
- Creating loading spinners and progress bars
- Gradient backgrounds like Spotify
- CSS filters for image effects
- Scroll-triggered animations
- Parallax scrolling effects
- CSS custom properties (variables)
- Responsive images and videos
- Mobile-first design approach
- Performance optimization basics
- Browser developer tools mastery

Projects built:

- Animated movie poster with hover effects
- Spotify-inspired music player UI
- Gaming website with parallax scrolling

Practice: Build 5 trending UI components from Dribbble

JavaScript Fundamentals - Programming Basics

Topics:

- What is JavaScript and why it's awesome
- Variables - storing information (let, const)
- Data types - numbers, strings, booleans
- Basic math and calculations
- Working with text (string methods)
- Making decisions with if/else
- Comparison operators (==, ===, >, <)
- Logical operators (&&, ||, !)
- Arrays - managing lists of data
- Loops - repeating tasks efficiently
- Functions - reusable code blocks
- First interactive features

Projects built:

- Grade calculator for school
- Simple quiz game about your interests
- Password strength checker

Practice: Solve 20 beginner-friendly JavaScript challenges

Month 3: JavaScript Mastery for React (Week 9-13)

JavaScript DOM Manipulation

Topics:

- The DOM - controlling web pages with JavaScript
- Selecting elements like jQuery but vanilla JS
- Changing content dynamically
- Adding and removing CSS classes
- Creating new elements with JavaScript
- Event listeners - clicks, hovers, keyboard
- Building interactive forms
- Form validation and error messages
- Local storage - saving user data
- Working with dates and times
- setTimeout and setInterval for timing
- Building countdown timers

Projects built:

- Interactive todo list with local storage
- Countdown timer for events
- Simple drawing app with canvas

Practice: Create 10 interactive web components

Modern JavaScript (ES6+) for React

Topics:

- Arrow functions - the React way
- Template literals for dynamic strings
- Destructuring - unpacking objects and arrays
- Spread operator - copying and merging
- Array methods crucial for React (map, filter, reduce)
- Working with objects in JavaScript
- ES6 modules - import and export
- Promises - handling asynchronous code
- Async/await - making promises easier
- Fetch API - getting data from the internet
- Working with JSON data
- Error handling with try/catch

Projects built:

- Meme generator using API
- Weather app for your city
- Pokemon card collection from API

Practice: Master array methods with 25 exercises

JavaScript Project Week & React Prep

Topics:

- Object-oriented programming basics
- Classes in JavaScript
- Constructor functions
- Prototypes and inheritance
- This keyword explained simply
- Closures and scope
- Higher-order functions
- Functional programming concepts
- NPM and package management
- Introduction to build tools
- Preparing for React mindset
- Component-based thinking

Projects built:

- Build a complete game (Snake, Tetris, or Flappy Bird)
- Music playlist manager
- Chat UI simulator

Practice: Refactor previous projects using OOP

----- PHASE 2: REACT FUNDAMENTALS & CORE CONCEPTS (MONTHS 4-6, WEEKS 14-26) -----

Dive into React! Learn the library that powers Facebook, Instagram, Netflix, and countless other apps. Build your first React applications.

Months 4-5: React Basics & Components (Week 14-21)

Introduction to React

Topics:

- What is React and why teens should learn it
- React vs vanilla JavaScript - the difference
- Setting up React with Vite
- Understanding the React project structure
- JSX - writing HTML in JavaScript
- Your first React component
- Functional components explained
- Props - passing data to components
- Children props and composition
- Conditional rendering (showing/hiding elements)
- Rendering lists with map
- Keys in React lists

Projects built:

- Profile card component
- Social media post component
- Product showcase with props

Practice: Build 15 different React components

React State & Interactivity

Topics:

- Understanding state in React
- useState hook - your first hook
- Handling events (onClick, onChange)
- Forms in React - controlled components
- Lifting state up between components
- State vs props - when to use what
- Multiple state variables
- State with objects and arrays
- Updating state correctly
- React Developer Tools
- Debugging React applications
- Common React mistakes to avoid

Projects built:

- Like button with counter
- Toggle dark/light theme
- Interactive shopping cart

Practice: Add state to 10 static components

React Hooks Deep Dive

Topics:

- useEffect - side effects in React
- useEffect dependencies array
- Cleanup functions in useEffect
- Data fetching with useEffect
- Loading states and error handling
- useContext for global state
- Creating custom hooks
- useReducer for complex state

- useRef for DOM manipulation
- useMemo for performance
- useCallback explained
- Rules of hooks

Projects built:

- GitHub profile viewer
- Real-time search with debouncing
- Theme context provider

Practice: Create 5 custom hooks for common tasks

React Router & Navigation

Topics:

- Single Page Applications (SPAs) explained
- React Router installation and setup
- Routes and Route components
- Link vs regular anchor tags
- NavLink for active styles
- URL parameters and useParams
- Query strings and useSearchParams
- Nested routes and layouts
- Protected routes for authentication
- 404 pages and error handling
- Programmatic navigation
- Route animations and transitions

Projects built:

- Multi-page portfolio site
- Blog with individual post pages
- E-commerce site with routing

Practice: Add routing to previous projects

Working with APIs in React

Topics:

- REST APIs explained for teens
- Fetch API in React components
- Axios for HTTP requests
- Managing loading and error states
- Displaying API data in components
- Pagination with APIs
- Infinite scrolling implementation
- Search functionality with APIs
- Authentication tokens and headers
- Environment variables for API keys
- CORS and how to handle it
- Mock APIs for development

Projects built:

- Movie database app (TMDB API)
- Spotify playlist viewer
- News aggregator for teens

Practice: Integrate 5 different public APIs

Month 6: Building Real React Applications (Week 22-26)

Styling React Applications

Topics:

- CSS Modules in React
- Styled-components introduction
- CSS-in-JS benefits and drawbacks
- Tailwind CSS with React
- Material-UI components
- Chakra UI for quick development
- Creating a design system
- Responsive design in React
- Animation libraries (Framer Motion)
- React Spring for animations
- Transition effects between routes
- Building reusable styled components

Projects built:

- Instagram-style photo feed
- Animated dashboard with charts
- Discord-inspired chat UI

Practice: Style 5 apps with different approaches

Forms & Validation in React

Topics:

- Controlled vs uncontrolled components
- Complex form handling
- Multi-step forms
- Form validation strategies
- React Hook Form library
- Yup validation schema
- File uploads in React
- Image preview before upload
- Dynamic form fields

- Form state management
- Error display patterns
- Success feedback to users

Projects built:

- Registration form with validation
- Survey form with multiple steps
- Profile editor with image upload

Practice: Build 5 different form types

React Performance Optimization

Topics:

- React rendering behavior
- Virtual DOM explained simply
- React.memo for preventing re-renders
- useMemo and useCallback proper usage
- Code splitting with lazy loading
- React.Suspense for async components
- Optimizing images in React
- Virtualization for long lists
- Bundle size optimization
- Performance profiling tools
- Common performance pitfalls
- Mobile performance considerations

Projects built:

- Optimized social media feed
- Virtual scrolling implementation
- Performance dashboard

Practice: Optimize 3 slow React apps

Testing React Applications

Topics:

- Why testing matters for teens
- Jest testing framework basics
- React Testing Library introduction
- Writing your first test
- Testing components with props
- Testing state changes
- Testing user interactions
- Mocking API calls in tests
- Testing custom hooks
- Integration testing basics
- Test-driven development (TDD) intro
- Coverage reports

Projects built:

- Add tests to todo app
- Test suite for form validation
- Testing user authentication flow

Practice: Write tests for 5 components

Phase 2 Capstone Project

Topics:

- Planning a complete React application
- Component architecture design
- State management planning
- API integration strategy
- Styling approach selection
- Testing implementation
- Performance considerations
- Deployment preparation

Projects built:

- CAPSTONE: TikTok Clone with React
- Features: Video feed, likes, comments, user profiles, following system
- Alternative: Instagram Clone
- Alternative: Discord Clone UI

Assessment: Build a complete social media app clone

----- PHASE 3: ADVANCED REACT & STATE MANAGEMENT (MONTHS 7-9, WEEKS 27-39) -----

Master advanced React patterns, state management, real-time features, and start building production-ready applications.

Months 7-8: State Management & Advanced Patterns (Week 27-34)

Context API & Global State

Topics:

- When to use Context API
- Creating context providers
- useContext hook patterns
- Multiple contexts in an app
- Context optimization techniques
- Avoiding unnecessary re-renders
- Context vs props drilling

- Auth context implementation
- Theme context patterns
- Shopping cart with context
- Context best practices
- When context isn't enough

Projects built:

- Multi-theme app with context
- Global notification system
- User authentication with context

Practice: Implement context in 5 different scenarios

Redux State Management

Topics:

- Why Redux for complex apps
- Redux core concepts simply explained
- Actions and action creators
- Reducers and pure functions
- Redux store setup
- React-Redux hooks (useSelector, useDispatch)
- Redux DevTools for debugging
- Redux Toolkit modern approach
- createSlice for less boilerplate
- Async actions with createAsyncThunk
- Redux middleware basics
- Normalizing state shape

Projects built:

- Reddit clone with Redux
- E-commerce cart with Redux
- Real-time chat state management

Practice: Convert context apps to Redux

Alternative State Management

Topics:

- Zustand - lightweight alternative
- MobX for reactive programming
- Recoil from Facebook
- Jotai atomic state management
- Valtio proxy-based state
- Comparing state libraries
- When to use which solution
- State persistence strategies
- Optimistic UI updates
- Undo/redo functionality
- State synchronization
- State migration patterns

Projects built:

- Todo app with Zustand
- Drawing app with undo/redo
- Collaborative editor state

Practice: Try 3 different state libraries

Advanced React Patterns

Topics:

- Compound components pattern
- Render props explained
- Higher-order components (HOCs)
- Custom hooks patterns
- Controlled and uncontrolled components
- State reducer pattern
- Props getters pattern
- Component composition strategies
- Polymorphic components
- Headless UI components
- Design system architecture
- Accessibility patterns

Projects built:

- Accessible dropdown component
- Flexible modal system
- Reusable form components

Practice: Implement 5 advanced patterns

Real-time Features with WebSockets

Topics:

- WebSockets explained for teens
- Socket.io with React
- Real-time chat implementation
- Live notifications
- Presence indicators (online/offline)
- Collaborative features
- Real-time updates in React
- Managing WebSocket connections
- Reconnection strategies
- Real-time state synchronization

- Server-sent events alternative
- Scaling real-time features

Projects built:

- Real-time chat application
- Live collaboration tool
- Multiplayer tic-tac-toe

Practice: Add real-time features to 3 apps

Month 9: Full Stack React Development (Week 35-39)

Backend Basics for React Developers

Topics:

- Why React developers need backend knowledge
- Node.js and Express basics
- Creating REST APIs
- CRUD operations explained
- MongoDB basics for beginners
- Mongoose for database modeling
- API authentication with JWT
- Protecting React routes
- Environment variables security
- CORS configuration
- API rate limiting
- Error handling patterns

Projects built:

- Full stack todo app
- User authentication system
- Blog with backend

Practice: Build 3 REST APIs for React apps

GraphQL with React

Topics:

- GraphQL vs REST for teens
- GraphQL queries and mutations
- Apollo Client setup
- useQuery and useMutation hooks
- GraphQL variables and fragments
- Optimistic UI with Apollo
- Caching strategies
- Subscriptions for real-time
- Error handling in GraphQL
- GraphQL with TypeScript
- Relay modern basics
- When to choose GraphQL

Projects built:

- Social media with GraphQL
- Real-time chat with subscriptions
- GitHub clone with GraphQL

Practice: Convert REST app to GraphQL

Authentication & Security

Topics:

- Authentication vs authorization
- JWT tokens explained simply
- OAuth with Google/GitHub
- Social login implementation
- Protected routes in React
- Role-based access control
- Refresh tokens pattern
- Secure storage of tokens
- XSS and CSRF prevention
- Input sanitization
- Content Security Policy
- Security best practices for teens

Projects built:

- Multi-auth app (email, Google, GitHub)
- Role-based dashboard
- Secure chat application

Practice: Secure 3 existing applications

Deployment & DevOps Basics

Topics:

- Preparing React apps for production
- Build optimization strategies
- Environment variables management
- Deploying to Netlify
- Deploying to Vercel
- GitHub Pages deployment
- Custom domain setup
- CI/CD with GitHub Actions
- Monitoring deployed apps
- Error tracking with Sentry
- Analytics integration

- Performance monitoring

Projects built:

- Deploy portfolio to production
- Set up CI/CD pipeline
- Multi-environment deployment

Practice: Deploy all projects online

Phase 3 Capstone Project

Topics:

- Full stack application planning
- Database design for React apps
- API architecture planning
- State management strategy
- Authentication implementation
- Real-time features planning
- Deployment strategy
- Testing and monitoring

Projects built:

- CAPSTONE: Spotify Clone Full Stack
- Features: Music player, playlists, search, user profiles, recommendations
- Alternative: YouTube Clone
- Alternative: Twitter/X Clone

Assessment: Build a complete full-stack application

Month 9 (Extended): Advanced Topics (Week 38-39)

TypeScript with React

Topics:

- Why TypeScript makes React better
- TypeScript setup for React
- Typing props and state
- Function component types
- Event handler types
- useReducer with TypeScript
- Context API with TypeScript
- Redux with TypeScript
- Generic components
- Utility types for React
- Third-party library types
- Migration strategies

Projects built:

- Convert React app to TypeScript
- Type-safe form builder
- TypeScript component library

Practice: Write all new React code in TypeScript

React Ecosystem & Libraries

Topics:

- Popular React libraries teens should know
- React Query for server state
- SWR for data fetching
- React Hook Form mastery
- Recharts for data visualization
- React DnD for drag and drop
- React Beautiful DnD
- React Select for dropdowns
- Date pickers and calendars
- Rich text editors
- Image galleries and carousels
- Animation libraries comparison

Projects built:

- Dashboard with charts
- Kanban board with drag-drop
- Rich text blog editor

Practice: Integrate 10 popular libraries

Progressive Web Apps (PWAs)

Topics:

- What are PWAs and why they matter
- Service workers basics
- Offline functionality
- Push notifications
- App manifest file
- Installing PWAs
- Cache strategies
- Background sync
- PWA with Vite
- Workbox for PWAs
- Testing PWAs
- PWA best practices

Projects built:

- Offline-first todo app
- News app with notifications

- Instagram-like PWA

Practice: Convert 3 apps to PWAs

React Performance Mastery

Topics:

- React 18 concurrent features
- Suspense for data fetching
- Server components introduction
- Streaming SSR
- Automatic batching
- Transitions API
- useDeferredValue hook
- startTransition for updates
- React DevTools Profiler mastery
- Bundle analysis and optimization
- Lazy loading strategies
- Performance budgets

Projects built:

- Optimize large e-commerce site
- Performance comparison study
- Build performance monitoring tool

Practice: Achieve 90+ Lighthouse scores

Advanced Project Week

Topics:

- Complex state management
- Micro-frontend architecture
- Module federation
- Monorepo with React
- Design systems at scale
- Internationalization (i18n)
- Accessibility (a11y) testing
- E2E testing with Cypress

Projects built:

- Multi-language app
- Accessible component library
- Micro-frontend implementation

Assessment: Technical interview preparation

----- PHASE 4: PROFESSIONAL REACT & CAREER PREPARATION (MONTHS 10-12, WEEKS 40-52) -----

Master React Native, Next.js, advanced architectures, and prepare for your career as a professional React developer.

Months 10-11: React Native & Mobile Development (Week 40-47)

React Native Fundamentals

Topics:

- React Native - React for mobile apps
- Setting up React Native environment
- Expo vs React Native CLI
- Core components: View, Text, Image
- Styling in React Native
- Flexbox for mobile layouts
- ScrollView and FlatList
- Navigation in React Native
- React Navigation setup
- Tab and stack navigators
- Platform-specific code
- Debugging React Native apps

Projects built:

- Instagram stories clone
- Todo app for mobile
- Weather app for phones

Practice: Build 5 mobile screens

React Native Advanced Features

Topics:

- Native device features access
- Camera and image picker
- Geolocation and maps
- Push notifications setup
- AsyncStorage for persistence
- Animations in React Native
- Gesture handling
- Deep linking
- App permissions
- Building for iOS and Android
- App store deployment basics
- React Native performance

Projects built:

- Photo sharing app

- Location-based app
- Chat app with notifications

Practice: Add native features to apps

Next.js for Production React

Topics:

- Next.js - the React framework
- File-based routing system
- Pages and layouts in Next.js
- Static generation (SSG)
- Server-side rendering (SSR)
- Incremental static regeneration
- API routes in Next.js
- Dynamic routes and params
- Image optimization
- Font optimization
- SEO with Next.js
- Next.js deployment

Projects built:

- Blog with Next.js
- E-commerce with SSR
- Portfolio with Next.js

Practice: Migrate React apps to Next.js

Advanced Next.js & Full Stack

Topics:

- Next.js 13+ app directory
- Server components in Next.js
- Data fetching patterns
- Caching strategies
- Middleware in Next.js
- Authentication in Next.js
- Database integration
- Prisma with Next.js
- tRPC for type-safe APIs
- Serverless functions
- Edge functions
- Production best practices

Projects built:

- Full-stack SaaS with Next.js
- Real-time app with Next.js
- Multi-tenant application

Practice: Build 3 production Next.js apps

Remix Framework Introduction

Topics:

- Remix vs Next.js comparison
- Remix routing system
- Data loading in Remix
- Forms and actions
- Error boundaries in Remix
- Optimistic UI patterns
- Progressive enhancement
- Remix deployment options
- When to choose Remix
- Migration from React Router
- Remix best practices
- Building with Remix

Projects built:

- Blog with Remix
- Form-heavy app with Remix
- Compare Next.js vs Remix

Practice: Build same app in Next.js and Remix

Month 11-12: Professional Skills & Career Prep (Week 48-52)

Advanced Architecture Patterns

Topics:

- Micro-frontends with React
- Module federation setup
- Monorepo management with Nx
- Component library creation
- Publishing to NPM
- Storybook for documentation
- Design tokens and systems
- CI/CD for React apps
- Automated testing pipelines
- Feature flags implementation
- A/B testing in React
- Analytics and tracking

Projects built:

- Component library with Storybook
- Micro-frontend architecture

- NPM package publication

Practice: Set up professional development workflow

Open Source Contribution

Topics:

- Why contribute to open source as a teen
- Finding beginner-friendly projects
- Understanding project structure
- Reading documentation and issues
- Making your first pull request
- Git workflow for collaboration
- Code review etiquette
- Writing good commit messages
- Contributing documentation
- Creating your own open source project
- Building a community
- Open source licenses

Projects built:

- Contribute to 3 open source projects
- Create your own open source library
- Document an existing project

Practice: Make weekly open source contributions

Interview Preparation

Topics:

- React interview questions for juniors
- JavaScript fundamentals review
- Data structures basics
- Algorithm basics for teens
- System design basics
- Behavioral interview prep
- Portfolio presentation skills
- Code challenge strategies
- Take-home project tips
- Technical communication
- Whiteboard coding basics
- Mock interview practice

Projects built:

- Build interview prep app
- Create algorithm visualizer
- Portfolio optimization

Practice: Daily coding challenges and mock interviews

Freelancing & Entrepreneurship

Topics:

- Freelancing as a teen developer
- Finding first clients
- Pricing your services
- Project estimation
- Client communication
- Managing expectations
- Contracts and agreements
- Building a personal brand
- Social media for developers
- Creating content about coding
- Building and launching products
- Startup basics for teens

Projects built:

- Professional portfolio site
- Client project simulation
- Launch a micro-SaaS

Practice: Complete freelance project workflow

Advanced React Ecosystem

Topics:

- Vite vs Next.js: which to start with
- Modern build tools comparison
- Turbopack and speed
- Bun runtime for React
- React Server Components deep dive
- Suspense and concurrent features
- React 19 features preview
- Future of React
- Alternative frameworks overview
- Vue.js and Svelte comparison
- Solid.js performance benefits
- Staying updated with React

Projects built:

- Migrate CRA to Vite
- Try alternative frameworks
- Performance comparison study

Practice: Experiment with cutting-edge React features

Month 12: Final Projects & Graduation (Week 49-52)

Capstone Project Planning

Topics:

- Identifying a problem to solve
- Market research for teen apps
- User research and personas
- Feature prioritization
- Technical architecture planning
- Choosing the right tech stack
- Project timeline creation
- Setting up development environment
- Git workflow for large projects
- Documentation planning
- Testing strategy
- Deployment planning

Projects built:

- Define capstone project scope
- Create technical specification
- Set up project infrastructure

Practice: Research and plan professionally

Capstone Development Sprint 1

Topics:

- Agile development basics
- Sprint planning
- User stories creation
- Core functionality implementation
- Database design and setup
- Authentication system
- Basic UI implementation
- API development
- Testing as you build
- Daily progress tracking
- Code review practices
- Version control discipline

Projects built:

- FINAL CAPSTONE: Your Dream App
- Build core features
- Implement authentication

Practice: Follow professional development practices

Capstone Development Sprint 2

Topics:

- Advanced features implementation
- Real-time functionality
- Third-party integrations
- Performance optimization
- Security implementation
- Responsive design completion
- Cross-browser testing
- User testing sessions
- Bug fixing and refinement
- Documentation writing
- API documentation
- Deployment preparation

Projects built:

- Complete all features
- Polish UI/UX
- Comprehensive testing

Practice: Achieve production quality

Launch Week & Presentation

Topics:

- Production deployment
- Domain and hosting setup
- SSL certificate configuration
- Performance monitoring setup
- Error tracking setup
- Analytics implementation
- Launch announcement preparation
- Demo video creation
- Technical presentation skills
- Project showcase preparation
- Resume and portfolio update
- LinkedIn profile optimization

Projects built:

- Deploy capstone to production
- Create presentation materials
- Record demo video

Practice: Present to peers and mentors

Final Week & Beyond (Week 52)

Career Launch & Next Steps

Topics:

- Job search strategies for teen developers
- Internship opportunities
- College application enhancement
- Building your network
- Attending meetups and conferences
- Online communities to join
- Continuous learning plan
- Keeping skills updated
- Building your reputation
- Mentoring other teens
- Teaching what you learned
- Your journey forward

Projects built:

- Create 1-year learning plan
- Apply to 3 opportunities
- Start mentoring beginners

Graduation & Celebration

Topics:

- Course retrospective
- Skills assessment
- Portfolio review
- Peer presentations
- Certificate ceremony
- Alumni network access
- Ongoing support resources
- Success stories sharing
- Future learning paths
- Advanced specializations
- Community involvement
- Giving back

Deliverables:

- Complete portfolio with 30+ projects
- Professional GitHub profile
- Published npm packages
- Open source contributions
- Live production applications
- Technical blog/content
- Professional network
- Industry certifications

Bonus Week: Emerging Technologies

Topics:

- AI and React integration
- Machine learning in frontend
- AI features in React apps
- AR/VR with React
- IoT and React Native
- Quantum computing awareness
- Future of web development
- Emerging frameworks
- New React features roadmap
- Industry trends for teens
- Building for AR and 3D on the web
- Your role in tech's future

Deliverables:

- Experimental project with new tech
- Tech trend research paper
- Future learning roadmap

Assessment: Final comprehensive assessment and certification

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Portfolio, Quiz App, Weather Widget, Todo List, Animation Playground
- Phase 2: Instagram Clone, TikTok UI, Discord Interface, Spotify Player, Twitter Feed
- Phase 3: Full Stack Social Media, E-commerce Platform, Real-time Chat, Video Platform
- Phase 4: Mobile Apps, Production SaaS, Open Source Library, Dream Project

Total projects: 30+ projects specifically designed for teenage interests

SKILLS THE STUDENT WILL HAVE AT THE END

- Core: HTML5, CSS3, JavaScript ES6+, React 18+, React Hooks
- Styling: Tailwind CSS, Styled Components, CSS Modules, Animations, Responsive Design
- State: Context API, Redux Toolkit, Zustand, React Query
- Routing: React Router v6, Next.js Routing, Dynamic Routes
- Backend: Node.js Basics, REST APIs, GraphQL, Authentication
- Mobile: React Native, Expo, Native Features
- Frameworks: Next.js, Remix, Gatsby basics

- Testing: Jest, React Testing Library, Cypress
- Tools: Git, VS Code, Chrome DevTools, Postman, Figma basics
- Deployment: Vercel, Netlify, GitHub Pages, Custom Domains

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 3-5 hours
 Hands on coding: 5-7 hours
 Projects: 3-4 hours
 Practice problems: 2-3 hours
 Total per week: 10-15 hours (flexible for school schedule)

SUPPORT INCLUDED

Live sessions: Weekly teen group sessions
 Mentorship: Young mentor (college age) guidance
 Community: Discord server for teen developers
 Code review: Peer and expert code reviews
 Career support: College prep, internship guidance
 Lifetime access: Keep learning as you grow

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
 Final certificate: Teen React Developer Professional Certificate
 Linkedin badge: Shareable achievement badge
 Industry recognized: Valued by colleges and employers
 Portfolio projects: 30+ projects to showcase

WHERE THIS CAN LEAD

- React Developer Intern at tech companies
- Freelance React Developer for local businesses
- College application standout with impressive portfolio
- Hackathon winner and competitor
- Open source contributor
- App developer and entrepreneur
- Coding tutor for younger students
- Tech content creator on YouTube/TikTok
- Summer internships at major tech companies
- Full-time junior developer (if 18+)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the React for Teens course cover?

A: The 12 month path starts with HTML, CSS and modern JavaScript, then React itself with components, hooks and routing, followed by state management, APIs and real-time features, and finishes with production apps, an introduction to React Native and career preparation. Projects are modeled on apps teens use daily, like social feeds, chat and music players.

Q: My teen has never coded. Is that a problem?

A: Not at all: the course is designed to start from absolute zero, and the first phase teaches web foundations before React appears. Curiosity and 10 to 15 hours a week alongside school are the only real requirements.

Q: What equipment is needed?

A: Just a computer or laptop with a stable internet connection; all the tools used are free. No prior setup is needed, because the first sessions handle everything together.

Q: How is progress measured?

A: By what gets built: each phase ends with working projects, from static websites to full React applications with real data, and the mentor keeps a portfolio record. By the final phase your teen is deploying complete apps, which makes progress self-evident.

Q: Can this fit alongside school?

A: Yes, it is scoped for 10 to 15 hours a week including the 2 live classes, and class slots move around exam periods. Many students treat the projects as their creative outlet rather than extra homework.

Q: Can we try a class for free?

A: Yes, book a free demo class at learn.modernagecoders.com/contact, no commitment required; your teen will write real code in the demo itself. Or message us on WhatsApp at +91 91233 66161.

Q: Are lessons live?

A: Every lesson is a live one hour class with a real teacher in a small group, never a recorded video. Teens share screens, get code reviews and ask questions the moment they are

stuck.

Q: Is there a certificate?

A: Yes, the Teen React Developer certification on completion, along with a deployed portfolio of React projects, which for developers is the credential that actually opens doors.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/react-for-teens-complete-masterclass>

=====

COURSE 107 OF 118: REACT.JS MASTERCLASS: ZERO TO SENIOR FRONTEND DEVELOPER

=====

Course link: <https://learn.modernagecoders.com/courses/react-js-complete-masterclass-college>
Subject area: Frontend Development & Modern Web
Designed for: College students
Who can join: Complete Beginner to Senior React Developer
Course length: 12 months (52 weeks)
Time needed: 15-20 hours/week recommended
Certificate: Certified React Developer Professional upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 12-month React.js course: master React 18+, Hooks, Redux, Next.js, TypeScript, testing and full-stack development, building production-ready apps.

In one line: From JSX Basics to Production-Scale React Applications

ABOUT THE PROGRAMME

This is not just a course, it's your complete transformation into a professional React developer. React powers Facebook, Netflix, Airbnb, and countless modern web applications. This 12-month masterclass takes you from absolute beginner to senior React developer, capable of building complex, scalable, and performant web applications. You'll master modern React with Hooks, state management, server-side rendering with Next.js, mobile development with React Native, and the entire React ecosystem. Build a portfolio of 30+ production-ready applications that will impress any employer or client.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - no React or even JavaScript expertise required
- Complete 12-month structured curriculum with React 18+ features
- Covers entire React ecosystem: Redux, Next.js, React Native, GraphQL
- 30+ real-world projects including social media, e-commerce, SaaS apps
- TypeScript integration from the beginning
- Testing, performance optimization, and production deployment
- Modern state management: Redux Toolkit, Zustand, Context API
- Full-stack development with MERN stack
- Interview preparation for FAANG companies
- Lifetime access with continuous updates for new React features

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): JavaScript, React Basics, Components, Hooks
Phase 2: Intermediate (Months 4-6): State Management, Routing, Forms, APIs
Phase 3: Advanced (Months 7-9): Next.js, TypeScript, Testing, Performance
Phase 4: Professional (Months 10-12): React Native, Full-Stack, Production Apps

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal education required
Coding experience: Basic HTML/CSS helpful but not required
Equipment: Computer with 4GB+ RAM, modern browser
Time commitment: 15-20 hours per week
English: Good comprehension for documentation
Motivation: Passion for building modern web applications

WHO THIS COURSE IS FOR

Students: CS students wanting to master modern web development
Working professionals: Developers transitioning to React
Entrepreneurs: Building modern web applications
Freelancers: Offering React development services
Kids: Mature teens (14+) with programming interest

Anyone: Anyone wanting to build professional web applications

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: JAVASCRIPT & REACT FUNDAMENTALS (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid foundations in JavaScript ES6+ and React core concepts, components, and hooks.

Months 1-2: JavaScript Mastery for React (Week 1-8)

Modern JavaScript Essentials

Topics:

- JavaScript fundamentals review
- ES6+ features: let, const, arrow functions
- Template literals and string interpolation
- Destructuring arrays and objects
- Spread and rest operators
- Default parameters
- Object property shorthand
- Array methods: map, filter, reduce, find
- forEach vs map understanding
- Promises and async/await
- Modules: import/export
- Classes and inheritance

Projects built:

- JavaScript utility functions library
- Todo app with vanilla JavaScript
- Weather app with API calls
- Array manipulation exercises

Practice: Complete 100 JavaScript challenges

Introduction to React

Topics:

- What is React and why use it?
- Virtual DOM concept
- React ecosystem overview
- Setting up development environment
- Vite vs Next.js: choosing a React setup
- Project structure best practices
- JSX syntax and expressions
- JSX vs HTML differences
- Embedding JavaScript in JSX
- React Developer Tools
- First React component
- React.StrictMode

Projects built:

- First React app setup
- Static website with React
- Personal portfolio starter
- JSX exercises and experiments

Practice: Create 20 simple React components

Components & Props

Topics:

- Functional components
- Component composition
- Props and prop types
- Children props
- Prop drilling introduction
- Default props
- Spreading props
- Component organization
- Conditional rendering techniques
- Lists and keys in React
- Fragments and React.Fragment
- Component lifecycle overview

Projects built:

- Component library creation
- Blog post list component
- Product card components
- Navigation menu component

Practice: Build 30 reusable components

State & Event Handling

Topics:

- Understanding state in React
- useState Hook deep dive
- State vs props
- Event handling in React

- Synthetic events
- Common events: onClick, onChange, onSubmit
- Event object and preventDefault
- Handling forms basics
- Controlled vs uncontrolled components
- Multiple state variables
- State update patterns
- Immutability importance

Projects built:

- Interactive counter app
- Form with validation
- Todo list with state
- Color picker component

Practice: Build 20 interactive components with state

Month 3: React Hooks Mastery (Week 9-13)

Essential Hooks

Topics:

- Rules of Hooks
- useState advanced patterns
- useEffect lifecycle replacement
- useEffect dependencies array
- Cleanup functions
- Multiple useEffect hooks
- useContext for global state
- useReducer for complex state
- useReducer vs useState
- useCallback for memoization
- useMemo for expensive computations
- useRef for DOM access and values

Projects built:

- Data fetching with useEffect
- Theme switcher with Context
- Complex form with useReducer
- Performance optimization examples

Practice: Master all essential hooks with 30 examples

Custom Hooks & Patterns

Topics:

- Creating custom hooks
- Custom hook best practices
- useLocalStorage hook
- useFetch hook
- useDebounce hook
- useThrottle hook
- usePrevious hook
- useOnClickOutside hook
- useMediaQuery hook
- useIntersectionObserver hook
- Hook composition patterns
- Testing custom hooks

Projects built:

- Custom hooks library
- Form validation hooks
- API hooks collection
- Animation hooks

Practice: Create 20 custom hooks

Styling in React

Topics:

- CSS modules
- CSS-in-JS introduction
- Styled-components basics
- Emotion library
- Inline styles in React
- Dynamic styling
- CSS frameworks with React
- Tailwind CSS integration
- Bootstrap with React
- Material-UI basics
- Animations with React
- CSS transitions in React

Projects built:

- Styled-components theme
- Responsive design system
- Animated components
- PHASE 1 CAPSTONE: Complete React Application with Custom Styling

Assessment: Phase 1 exam on React fundamentals

Master routing, forms, API integration, and professional state management solutions.

Months 4-5: Routing & Data Management (Week 14-22)

React Router

Topics:

- React Router v6 setup
- BrowserRouter vs HashRouter
- Route components
- Link and NavLink
- Nested routes
- Route parameters
- Query strings
- useNavigate hook
- useLocation hook
- useParams hook
- Protected routes
- Route guards
- Lazy loading routes
- 404 pages

Projects built:

- Multi-page application
- E-commerce product routing
- Admin dashboard with nested routes
- Authentication flow

Practice: Build 10 routing scenarios

Forms & Validation

Topics:

- Controlled components deep dive
- Form state management
- Multi-step forms
- File uploads
- Form validation strategies
- React Hook Form
- Formik library
- Yup validation schemas
- Custom validation
- Error handling
- Form optimization
- Dynamic forms
- Form arrays
- Accessibility in forms

Projects built:

- Registration system
- Survey application
- Dynamic form builder
- Checkout process

Practice: Create 15 complex forms

API Integration & Data Fetching

Topics:

- Fetch API with React
- Axios integration
- Loading states
- Error handling
- Async operations in React
- Data fetching patterns
- Caching strategies
- Optimistic updates
- Pagination implementation
- Infinite scrolling
- Real-time data with WebSockets
- Server-Sent Events
- GraphQL with Apollo Client
- React Query (TanStack Query)

Projects built:

- Blog with API backend
- Real-time chat application
- GitHub profile viewer
- Infinite scroll feed

Practice: Integrate 10 different APIs

Context API & State Management

Topics:

- Context API deep dive
- Creating providers
- Multiple contexts
- Context optimization
- Context vs props
- Global state patterns
- State management principles

- Introduction to Redux
- Redux Toolkit modern approach
- Actions and reducers
- Redux DevTools
- Async actions with Redux Thunk
- RTK Query

Projects built:

- Shopping cart with Context
- Theme system with Context
- Todo app with Redux
- E-commerce with Redux Toolkit

Practice: Implement 10 state management scenarios

Advanced Redux Patterns

Topics:

- Redux middleware
- Redux Saga introduction
- Normalized state shape
- Reselect for memoization
- Redux persist
- Redux forms
- Time travel debugging
- Redux performance
- Code splitting with Redux
- Redux with TypeScript preview
- Alternative: Zustand
- Alternative: MobX
- Alternative: Recoil
- Choosing state management

Projects built:

- Complex app with Redux Saga
- Offline-capable app
- Multi-user collaboration app
- State management comparison

Practice: Master advanced Redux patterns

Month 6: Component Patterns & Architecture (Week 23-26)

Advanced Component Patterns

Topics:

- Higher-Order Components (HOCs)
- Render props pattern
- Compound components
- Controlled vs uncontrolled
- Presentational vs container components
- Provider pattern
- State initializer pattern
- State reducer pattern
- Prop collections pattern
- Prop getters pattern
- Component composition strategies
- Slot pattern
- Inversion of control
- Extensible components

Projects built:

- Advanced UI component library
- Flexible modal system
- Data table component
- Accordion component set

Practice: Implement 15 component patterns

Performance Optimization

Topics:

- React DevTools Profiler
- Performance metrics
- React.memo optimization
- useMemo strategies
- useCallback patterns
- Lazy loading components
- Code splitting with React.lazy
- Suspense for data fetching
- Virtualization with react-window
- Image optimization
- Bundle size optimization
- Tree shaking
- Performance monitoring
- Core Web Vitals

Projects built:

- Optimize large list rendering
- Performance audit project
- Image gallery optimization
- Bundle size reduction

Practice: Optimize 10 slow React apps

Error Handling & Debugging

Topics:

- Error boundaries
- Error boundary patterns
- Fallback UI design
- Error reporting services
- Debugging techniques
- React DevTools mastery
- Console debugging strategies
- Network tab debugging
- Source maps
- Debugging production issues
- Logging strategies
- Error tracking with Sentry
- Performance debugging
- Memory leak detection

Projects built:

- Error boundary system
- Debugging toolkit
- Error monitoring setup
- Production debugging guide

Practice: Debug 20 React issues

Accessibility & SEO

Topics:

- Web accessibility principles
- ARIA attributes in React
- Keyboard navigation
- Screen reader support
- Focus management
- Accessible forms
- Color contrast
- Accessible modals
- Testing accessibility
- SEO with React SPA
- Meta tags management
- React Helmet
- Open Graph tags
- Structured data

Projects built:

- Accessible component library
- SEO-optimized blog
- Accessibility audit tool
- WCAG compliant app

Practice: Make 10 apps fully accessible

Phase 2 Capstone Project

Topics:

- Complex application planning
- Architecture design
- State management setup
- API integration
- Performance optimization
- Testing implementation

Projects built:

- PHASE 2 CAPSTONE: Full-Featured Social Media App
- Build complete social platform with posts, comments, likes
- Include: authentication, real-time updates, image uploads
- Redux state management, optimized performance

Assessment: Phase 2 exam on advanced React

PHASE 3: TYPESCRIPT, TESTING & NEXT.JS (MONTHS 7-9, WEEKS 27-39)

Master TypeScript with React, comprehensive testing, and server-side rendering with Next.js.

Months 7-8: TypeScript & Testing (Week 27-35)

TypeScript Fundamentals for React

Topics:

- TypeScript setup with React
- Basic types and interfaces
- Type annotations
- Function types
- Union and intersection types
- Type aliases
- Generics basics
- Type inference
- Strict mode
- tsconfig.json configuration
- TypeScript with Vite

- TypeScript with Vite
- Migration strategies
- Common TypeScript errors

Projects built:

- Convert JS project to TypeScript
- Type-safe component library
- TypeScript configuration setup
- Type definitions creation

Practice: Type 30 React components

TypeScript with React

Topics:

- Typing functional components
- Props interface patterns
- Children prop types
- Event handler types
- Hooks with TypeScript
- useState with TypeScript
- useReducer typing
- useContext typing
- Custom hooks typing
- Generic components
- Discriminated unions
- Type guards
- Utility types
- Third-party library types

Projects built:

- Type-safe form system
- Redux with TypeScript
- API client with types
- Component library with types

Practice: Build 20 type-safe components

Testing React Applications

Topics:

- Testing philosophy
- Jest setup and configuration
- React Testing Library
- Component testing
- Testing hooks
- Testing async operations
- Mocking modules
- Testing user interactions
- Integration testing
- Snapshot testing
- Coverage reports
- Testing best practices
- TDD with React
- Debugging tests

Projects built:

- Test suite for component library
- TDD todo application
- Integration test suite
- Testing utilities library

Practice: Write 100+ test cases

End-to-End Testing

Topics:

- E2E testing introduction
- Cypress setup
- Writing Cypress tests
- Page object pattern
- API testing with Cypress
- Visual regression testing
- Playwright introduction
- Cross-browser testing
- CI/CD integration
- Test automation strategies
- Performance testing
- Accessibility testing
- Security testing basics
- Test reporting

Projects built:

- E2E test suite with Cypress
- Visual regression tests
- CI/CD pipeline with tests
- Automated test reports

Practice: Create comprehensive E2E coverage

Introduction to Next.js

Topics:

- Next.js benefits and features
- Create Next App setup

- File-based routing
- Pages and layouts
- Static generation (SSG)
- Server-side rendering (SSR)
- API routes
- Dynamic routes
- Link component
- Image optimization
- CSS and styling in Next.js
- Environment variables
- Next.js vs a plain Vite SPA
- Migration to Next.js

Projects built:

- Blog with Next.js
- Portfolio with SSG
- E-commerce homepage
- API with Next.js

Practice: Build 5 Next.js applications

Month 9: Advanced Next.js & Full-Stack (Week 36-39)

Advanced Next.js Features

Topics:

- Incremental Static Regeneration (ISR)
- Dynamic imports
- Middleware
- Edge functions
- Authentication in Next.js
- NextAuth.js
- Database integration
- Prisma with Next.js
- tRPC integration
- Internationalization (i18n)
- SEO optimization
- Performance optimization
- Deployment strategies
- Vercel deployment

Projects built:

- Full-stack app with authentication
- Multi-language website
- E-commerce with ISR
- SaaS application starter

Practice: Master advanced Next.js features

GraphQL with React

Topics:

- GraphQL introduction
- GraphQL vs REST
- Apollo Client setup
- Queries and mutations
- GraphQL fragments
- Optimistic UI
- Caching strategies
- Subscriptions
- Error handling
- Authentication with GraphQL
- File uploads
- Relay modern basics
- GraphQL code generation
- Performance optimization

Projects built:

- GraphQL blog application
- Real-time chat with subscriptions
- E-commerce with GraphQL
- GitHub clone with GraphQL

Practice: Build 5 GraphQL applications

Micro-Frontends & Module Federation

Topics:

- Micro-frontends architecture
- Benefits and challenges
- Module federation with Webpack 5
- Sharing dependencies
- Runtime integration
- Build-time integration
- Single-spa framework
- Qiankun framework
- Communication between micro-frontends
- Shared state management
- Deployment strategies
- Testing micro-frontends
- Performance considerations

- When to use micro-frontends

Projects built:

- Micro-frontend architecture
- Module federation setup
- Multi-team application
- Legacy app integration

Practice: Implement micro-frontend patterns

Progressive Web Apps (PWA)

Topics:

- PWA principles
- Service workers in React
- Offline functionality
- Cache strategies
- Web app manifest
- Push notifications
- Background sync
- App shell architecture
- Performance optimization for PWA
- Lighthouse audits
- PWA deployment
- App store submission
- PWA vs native apps
- Workbox library

Projects built:

- Offline-first news app
- PWA e-commerce site
- Push notification system
- Installable web app

Practice: Convert 5 apps to PWAs

Phase 3 Capstone Project

Topics:

- Full-stack application design
- TypeScript throughout
- Comprehensive testing
- Next.js implementation
- Performance optimization
- Production deployment

Projects built:

- PHASE 3 CAPSTONE: Production-Ready SaaS Application
- Build complete SaaS with Next.js, TypeScript, testing
- Include: authentication, payments, admin panel
- Full test coverage, deployed to production

Assessment: Phase 3 exam on TypeScript, testing, Next.js

PHASE 4: REACT NATIVE & PROFESSIONAL DEVELOPMENT (MONTHS 10-12, WEEKS 40-52)

Master React Native, deployment, DevOps, and prepare for senior developer roles.

Months 10-11: React Native & Mobile Development (Week 40-48)

React Native Fundamentals

Topics:

- React Native introduction
- Expo vs React Native CLI
- Development environment setup
- Core components overview
- View, Text, Image components
- ScrollView and FlatList
- Styling in React Native
- Flexbox for mobile
- Platform-specific code
- React Navigation setup
- Stack, tab, drawer navigation
- Handling user input
- Touch events
- Debugging React Native

Projects built:

- First React Native app
- Todo mobile app
- Weather app for mobile
- Navigation examples

Practice: Build 10 mobile screens

Advanced React Native

Topics:

- Native modules
- Animations with Animated API
- Gesture handling
- Camera and image picker
- Geolocation

- Push notifications
- AsyncStorage
- SQLite integration
- State management in React Native
- Redux with React Native
- Performance optimization
- Code push updates
- Deep linking
- Biometric authentication

Projects built:

- Social media mobile app
- Location-based app
- Camera app with filters
- Offline-capable app

Practice: Implement 15 native features

React Native Deployment

Topics:

- Building for iOS
- Xcode configuration
- Building for Android
- Android Studio setup
- Signing applications
- App store guidelines
- Google Play guidelines
- Beta testing with TestFlight
- Google Play Console
- CI/CD for React Native
- Fastlane automation
- App monitoring
- Crash reporting
- Analytics integration

Projects built:

- Deploy app to TestFlight
- Deploy app to Google Play
- CI/CD pipeline setup
- Analytics implementation

Practice: Deploy 3 apps to stores

Full-Stack React Development

Topics:

- MERN stack overview
- Node.js backend for React
- Express.js API creation
- MongoDB integration
- Authentication strategies
- JWT implementation
- OAuth integration
- File uploads
- WebSocket integration
- Email services
- Payment processing
- Stripe integration
- Deployment strategies
- Docker for React apps

Projects built:

- Full-stack e-commerce
- Real-time collaboration app
- Subscription-based SaaS
- Social network clone

Practice: Build 5 full-stack applications

DevOps for React

Topics:

- CI/CD pipelines
- GitHub Actions
- GitLab CI
- Jenkins for React
- Docker containerization
- Kubernetes basics
- Cloud deployment (AWS, GCP, Azure)
- Vercel deployment
- Netlify deployment
- CDN setup
- Monitoring and logging
- Error tracking
- Performance monitoring
- Security best practices

Projects built:

- Complete CI/CD pipeline
- Multi-environment setup
- Monitoring dashboard

- Automated deployment system
- Practice: Deploy to 5 different platforms

Month 12: Career Launch & Specialization (Week 49-52)

Advanced Patterns & Architecture

Topics:

- Atomic design principles
- Design systems in React
- Storybook for components
- Component documentation
- Monorepo with Lerna/Nx
- Design tokens
- Theme systems
- Multi-tenancy
- White-label applications
- Plugin architectures
- Extensible applications
- Framework agnostic components
- Web Components with React
- React Server Components

Projects built:

- Design system creation
- Storybook documentation
- Monorepo setup
- White-label platform

Practice: Implement 10 architectural patterns

Interview Preparation

Topics:

- React interview questions
- JavaScript deep dive
- Algorithm challenges
- System design for frontend
- Take-home projects
- Code review skills
- Behavioral questions
- STAR method
- Portfolio presentation
- Salary negotiation
- FAANG preparation
- Startup vs corporate
- Remote work preparation
- Freelancing basics

Projects built:

- Interview prep materials
- Portfolio website
- Code challenge solutions
- System design examples

Practice: Complete 50 interview problems

Portfolio & Personal Brand

Topics:

- Portfolio strategy
- Project selection
- Case study writing
- GitHub optimization
- LinkedIn profile
- Personal website
- Technical blogging
- Video content creation
- Open source contribution
- Community involvement
- Speaking at meetups
- Building reputation
- Networking strategies
- Mentorship

Projects built:

- Professional portfolio
- Technical blog posts
- Open source contributions
- Personal brand strategy

Practice: Build complete online presence

Specialization Paths

Topics:

- Frontend architect path
- UI/UX engineer focus
- Performance specialist
- Accessibility expert
- Animation specialist
- 3D with React Three Fiber
- Data visualization with D3

- React Native specialist
- Full-stack developer
- Technical lead skills
- Engineering manager path
- Consultant pathway
- Educator and trainer
- Startup founder skills

Projects built:

- Specialization project
- Advanced showcase
- Teaching materials
- Consulting proposal

Practice: Deep dive into chosen specialization

Final Project & Graduation

Topics:

- Capstone planning
- Real-world problem solving
- Complete application development
- Testing and optimization
- Documentation
- Deployment and monitoring
- Presentation preparation
- Peer review
- Career planning
- Continuous learning
- Community engagement
- Future roadmap

Projects built:

- FINAL CAPSTONE: Production React Application
- Build and deploy a complete production application
- Your choice: SaaS, e-commerce, social platform, or innovative idea
- Must include: React, TypeScript, testing, state management, deployment
- Live users and real-world usage

Assessment: Final comprehensive exam and project defense

Continuous Learning & Growth (Ongoing)

Staying Current with React

Topics:

- Following React development
- React RFC process
- Beta features testing
- React conferences
- Community involvement
- React core team updates
- Ecosystem changes
- New libraries and tools
- Performance improvements
- Best practices evolution
- Contributing to React
- Writing RFCs

Projects built:

- Learning roadmap
- Community contribution
- Conference talk proposal
- Open source maintenance

Practice: Dedicate 5 hours weekly to React updates

Advanced Topics

Topics:

- React Compiler
- Concurrent features
- Server Components deep dive
- Streaming SSR
- React Native New Architecture
- React Three Fiber for 3D
- React Native Web
- Remix framework
- Astro with React
- Qwik resumability
- Solid.js comparison
- Vue vs React

Projects built:

- Experimental features project
- 3D React application
- Framework comparison
- Performance study

Practice: Explore cutting-edge React features

Teaching & Mentoring

Topics:

- Creating tutorials

- Video course creation
- Workshop development
- Mentoring juniors
- Code review skills
- Technical writing
- Documentation skills
- Community building
- Discord/Slack management
- Stack Overflow contribution
- Reddit participation
- Twitter presence

Projects built:

- Tutorial series
- Workshop materials
- Mentorship program
- Community project

Practice: Teach React to others weekly

Leadership & Architecture

Topics:

- Technical leadership
- Architecture decisions
- Technology selection
- Team best practices
- Code standards
- Review processes
- Performance culture
- Testing culture
- Documentation standards
- Knowledge sharing
- Team growth
- Hiring and interviews

Projects built:

- Team guidelines
- Architecture documents
- Best practices guide
- Interview process design

Practice: Lead React projects and teams

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 15+ fundamental projects - components, hooks, styling
- Phase 2: 20+ intermediate projects - routing, state, APIs, forms
- Phase 3: 15+ advanced projects - TypeScript, testing, Next.js, PWA
- Phase 4: 10+ professional projects - React Native, full-stack, production
- Total: 60+ React projects from simple to production-scale

Total projects: 60+ React applications across web and mobile

SKILLS THE STUDENT WILL HAVE AT THE END

- React Core: Components, JSX, props, state, lifecycle, hooks
- Advanced React: Custom hooks, patterns, performance, error boundaries
- State Management: Context API, Redux, Redux Toolkit, Zustand, MobX
- Routing: React Router v6, nested routes, guards, lazy loading
- Styling: CSS Modules, Styled Components, Emotion, Tailwind CSS
- TypeScript: Full type safety, generics, advanced types
- Testing: Jest, React Testing Library, Cypress, E2E testing
- Next.js: SSR, SSG, ISR, API routes, full-stack development
- React Native: Mobile development, native features, deployment
- Performance: Optimization, code splitting, lazy loading, memoization
- Tools: Webpack, Vite, ESLint, Prettier, Storybook
- DevOps: CI/CD, Docker, cloud deployment, monitoring

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours

Hands on coding: 8-10 hours

Projects: 3-5 hours

Practice problems: 2-3 hours

Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly React workshops and code reviews

Mentorship: 1-on-1 guidance from senior React developers

Community: Active Discord with 10,000+ React developers

Code review: Expert review of all projects

Career support: Resume review, interview prep, job referrals

Lifetime access: All content and React updates forever

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase
Final certificate: Certified React Developer Professional
Linkedin badge: Verified LinkedIn credential
Industry recognized: Recognized by tech companies globally
Portfolio projects: 60+ production-ready projects
Specialization certificate: Certificate in chosen specialization

WHERE THIS CAN LEAD

- React Developer
- Frontend Developer
- Full-Stack Developer (MERN)
- React Native Developer
- Senior Frontend Engineer
- UI/UX Engineer
- Frontend Architect
- Technical Lead
- JavaScript Specialist
- Freelance React Developer
- React Consultant
- Startup CTO/Technical Co-founder

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Growing roles: Rs 5-8 LPA (Junior React Developer)

Experienced roles: Rs 8-15 LPA (React Developer); Rs 12-25 LPA (Senior React Developer); Rs 20-40+ LPA (Lead/Architect)

Freelance: Rs 2,000-8,000/hour based on expertise

International: \$60k-150k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Why is React.js one of the most in-demand frontend skills in 2026?

A: React powers the frontends of Facebook, Instagram, Netflix, Airbnb, and thousands of top companies. Its component-based architecture makes building complex UIs manageable.

Q: Do I need to know JavaScript before learning React?

A: Basic JavaScript knowledge helps, but our course includes JavaScript fundamentals review in Phase 1. We cover ES6+ features, DOM manipulation, and async programming before diving into React. Complete beginners may want to spend extra time on JavaScript basics, but it's not mandatory.

Q: What's the difference between React, Angular, and Vue?

A: React is a library focused on building UI components with maximum flexibility. Angular is a complete framework with more structure but steeper learning curve. Vue sits in between. React's ecosystem, job market, and community are largest. Learning React makes learning other frameworks easy since concepts transfer.

Q: What will I be able to build after completing this 12-month React course?

A: You'll build 60+ projects including: component libraries, social media platforms, e-commerce sites, admin dashboards, real-time apps with WebSockets, mobile apps with React Native, and production SaaS applications. Your portfolio will include complete full-stack applications deployed and serving real users.

Q: Does this course cover React Native for mobile app development?

A: Yes! Phase 4 includes comprehensive React Native training. You'll learn to build iOS and Android apps, deploy to app stores, and understand when to use React Native vs native development. Many companies hire developers who can build both web and mobile applications with React.

Q: What salary can I expect as a React developer in India?

A: React developers are highly valued. The combination of React + TypeScript + Next.js commands premium salaries.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/react-js-complete-masterclass-college>

=====

Course link: <https://learn.modernagecoders.com/courses/roblox-game-coding-for-kids-lua-course>
Subject area: Game Development
Designed for: Kids
Who can join: Beginner, ages 8 to 14
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Roblox coding course for kids ages 8-14: live online classes in Roblox Studio and Luau scripting, from first parts to a published game friends can play.

In one line: Your child already plays Roblox. In six months they publish a game of their own on it.

ABOUT THE PROGRAMME

Roblox is where millions of kids already spend their screen time, and it hides a genuinely serious tool: Roblox Studio, the free game-making software behind every experience on the platform, scripted in a real programming language called Luau. This 6-month live course turns a player into a maker. Months 1 and 2 cover building in Studio and the first Luau scripts: variables, decisions, loops, and functions, each one changing something visible in a 3D world. Months 3 and 4 bring games to life with events, player interaction, scoring, and checkpoints, which is where an obby stops being a row of parts and starts being a game. Month 5 adds the mechanics kids recognize from games they love: shops, timers, rounds, sound, and polish. Month 6 is the capstone: each student designs, builds, tests, and publishes their own game to Roblox, then presents it. Classes are live and small, and students build in every single session. Nothing in this course is watch-and-copy: the instructor sets a challenge, watches each student work, and reviews the homework builds between classes.

WHAT MAKES THIS COURSE DIFFERENT

- Students learn Luau, the actual scripting language of Roblox, not a drag-and-drop imitation of it
- Every concept lands inside a 3D world the student built: loops make disco floors, conditionals open doors, functions run traps
- The classic beginner bugs (debounce, anchoring, nil) are taught on purpose, because fixing them is where real understanding starts
- By month 4 students have leaderboards, checkpoints, and coin systems working, the same building blocks behind popular Roblox games
- The capstone is published to Roblox for real, so the finished game has a link a grandparent can open
- Everything used is free: Roblox Studio costs nothing, publishing costs nothing, and no Robux are ever needed

THE LEARNING PATH, STEP BY STEP

Phase 1: Builder to scripter: master Roblox Studio's tools, then write first Luau scripts that change the world from code

Phase 2: Making it a game: events, player interaction, scoring, coins, checkpoints, and on-screen interfaces

Phase 3: Shops, timers, rounds, and polish, then a capstone game scoped, built, published to Roblox, and presented

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from placing the first part and writing the first line

Roblox experience: Helpful but not required. Most students have played Roblox; none have needed to

Equipment: A Windows or Mac computer that runs Roblox Studio, plus a stable internet connection. Phones, tablets, and most Chromebooks cannot run Studio

Account: A free Roblox account, set up with a parent. No Robux or paid membership is needed at any point

WHO THIS COURSE IS FOR

Roblox players: Kids who already love playing Roblox and keep asking how the games they play are actually made

Complete beginners: Children aged 8 to 14 with no coding background who want a first language wrapped in a world they care about

Scratch graduates: Students moving up from block coding who are ready to type real scripts in a real engine

Young game designers: Kids who fill notebooks with game ideas and need the tools to build one for real

Parents seeking direction: Families who want Roblox hours converted from pure consumption into building, debugging, and finishing things

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: STUDIO SKILLS AND FIRST LUAU SCRIPTS (MONTHS 1-2, WEEKS 1-8)

First the tools: building worlds in Roblox Studio with confidence. Then the leap every Roblox kid dreams about: writing real Luau code that changes the world by itself.

Month 1: Becoming a Builder in Roblox Studio (Weeks 1-4)

Welcome to Roblox Studio

Topics:

- Installing Roblox Studio and a tour of the interface: Explorer, Properties, and the 3D viewport
- Camera controls: fly, zoom, and orbit without getting lost
- Creating parts: blocks, spheres, wedges, and cylinders
- Move, scale, and rotate tools, and snapping to the grid
- Play-testing your world with your own avatar
- Saving your work and opening it again next class

Projects built:

- First playground: a small floating island with ramps, towers, and a jump course to test

Practice: Build a room with four different part shapes, then screenshot your avatar standing on top of it

Parts and Properties

Topics:

- The Properties panel: color, material, transparency, and reflectance
- Anchored explained properly: why unanchored builds collapse when you press play
- CanCollide: making walls you can walk through
- Grouping parts into models and naming things in the Explorer
- Duplicating and mirroring to build faster
- Union and negate: cutting holes and making custom shapes

Projects built:

- Secret hideout: a house with a hidden walk-through wall and a glass floor over a drop

Practice: Property scavenger hunt: recreate five reference parts exactly from their pictures

Building a World

Topics:

- Terrain tools: sculpting hills, water, and caves
- Spawn locations and where players appear
- Planning an obby on paper before placing a single part
- Difficulty curves: why the first jump should be easy
- Lighting basics: time of day and atmosphere
- Testing like a stranger: playing your own course as if you had never seen it

Projects built:

- Obby, part one: a ten-stage jump course across terrain you sculpted yourself

Practice: Add three more stages to your obby, each slightly harder than the last, and have a family member test them

Your First Script

Topics:

- What a Script object is and where it lives in the Explorer
- print and the Output window: making the computer talk
- Variables: naming a part in code and holding numbers and words
- Changing a part's color and material from code instead of the mouse
- Reading your first error message without panicking
- Comments: leaving notes for future-you

Projects built:

- Magic brick: a part that changes color and prints a welcome message the moment the game starts

Practice: Write three scripts that each change a different property of a part, and break one on purpose to read the error

Assessment: Month 1 build check: a small world using anchoring, grouping, terrain, and one working script, walked through live

Month 2: Luau from Zero (Weeks 5-8)

Variables and Values

Topics:

- Numbers, strings, and booleans in Luau
- local and why every variable gets it
- Doing maths in code: changing a part's size and position with numbers
- String concatenation: building messages out of pieces
- Vector3 gently introduced: positions in a 3D world need three numbers
- nil: the most famous value in Roblox scripting

Projects built:

- Teleporting part: a script that moves a platform to a new position, then a script that grows it step by step

Practice: Ten short scripts: resize, recolor, rename, and reposition parts using only variables and code

Making Decisions

Topics:

- if, then, else: teaching a script to choose
- Comparisons: equal, not equal, greater, and less
- Booleans in action: checking if a part is anchored or transparent
- elseif chains for more than two choices
- and, or, not: combining conditions
- Common trap: = for setting, == for asking

Projects built:

- Weather machine: a script that checks a variable and sets the world to day, night, or fog to match

Practice: Decision drills: a password checker, a size judge that labels parts small or large, and a fix-the-broken-if puzzle

Loops

Topics:

- while loops and task.wait: doing something forever, politely
- Blinking, pulsing, and color-cycling parts
- for loops: counting exactly, from disco floors to stairways
- Building with loops: a staircase generated by code in one second
- Loop variables inside the loop body
- Infinite loops without a wait: what happens and how to stop it

Projects built:

- Disco floor: a grid of tiles that flash random colors forever
- Instant staircase: a for loop that builds a climbable spiral stair

Practice: Loop katas: a countdown announcer, a growing tower, and a part that fades out smoothly step by step

Functions

Topics:

- Writing a function: name, parentheses, body, end
- Parameters: one function that colors any part you hand it
- Return values: functions that answer questions
- Why functions matter: change the trap once, fix it everywhere
- Organizing a script into small named functions
- Calling functions from loops: the payoff moment

Projects built:

- Trap factory: one makeTrap function used to fill an obby stage with ten identical traps in seconds

Practice: Refactor your disco floor and staircase scripts to use functions, then write one new function from a written spec

Assessment: Month 2 skills check: live coding of three short Luau exercises using variables, if statements, loops, and one function

----- PHASE 2: EVENTS, PLAYERS, AND KEEPING SCORE (MONTHS 3-4, WEEKS 9-16) -----

Scripts so far ran on their own. Now they respond to players: touched events, health, doors, and traps, then leaderboards, coins, and checkpoints, the machinery every real Roblox game runs on.

Month 3: Events and Player Interaction (Weeks 9-12)

The Touched Event

Topics:

- Events: code that waits for something to happen
- part.Touched:Connect and functions as event handlers
- Finding out what touched the part
- Humanoid and character models: how Roblox represents a player
- The classic lava brick, built properly
- Print-debugging events: seeing exactly when they fire

Projects built:

- Lava stage: an obby section with kill bricks that send careless players back to the start

Practice: Build a safe-path puzzle: nine tiles, some deadly, some safe, with the pattern hidden until touched

Changing the Player

Topics:

- Reading and changing Humanoid health
- WalkSpeed and JumpPower: speed pads and super-jumps
- Healing pads and damage zones
- Resetting effects after a delay with task.wait
- Being fair: telling the player what a pad does before it does it
- Testing multiplayer effects with two test players in Studio

Projects built:

- Power-up alley: a course with speed strips, jump pads, a healing station, and one sneaky slow-down trap

Practice: Design and script one original power-up of your own invention, then write two sentences on how it changes the game

Doors, Buttons, and Debounce

Topics:

- Doors that open: toggling Transparency and CanCollide from a touch
- Buttons on the floor and on the wall
- The debounce problem: why your door fires twenty times per step
- Fixing debounce with a boolean flag, the pattern every Roblox scripter knows
- Timed doors that close themselves
- Locked doors: combining a condition with an event

Projects built:

- Dungeon gate room: a button-operated gate with proper debounce and a door that only opens for players holding the right key part

Practice: Break a working door by removing its debounce, watch the Output flood, then repair it and explain the fix in class

Project Week: The Scripted Obby

Topics:

- Combining the month: lava, power-ups, doors, and traps in one course
- Pacing a level: challenge, relief, surprise
- Organizing scripts so each stage's code is findable
- Playtesting a classmate's obby and writing useful feedback
- Fixing the top two problems your testers found
- Naming and Explorer hygiene before a project grows

Projects built:

- Obby 2.0: a fully scripted multi-stage obby with kill bricks, power-ups, at least one gate, and a victory area

Practice: Run two playtests and log every place a tester got stuck or confused, then fix the worst one

Assessment: Obby 2.0 demo: live playthrough plus a code walkthrough of one trap, one power-up, and one door

Month 4: Score, Coins, and Checkpoints (Weeks 13-16)

Leaderboards

Topics:

- leaderstats: the folder-and-value pattern behind every Roblox leaderboard
- IntValue objects and setting them from code
- Players.PlayerAdded: running code when someone joins
- Awarding points from a touched event
- Server scripts vs local effects, kept simple: why the score lives on the server
- Watching the leaderboard update live in a two-player test

Projects built:

- Point course: a stage where every platform reached for the first time adds points to a live leaderboard

Practice: Add a second leaderboard stat of your choice, such as deaths or stages cleared, and make something update it

Collectibles

Topics:

- Coins that spin: rotating parts with a loop
- Collecting: touched event plus destroy, with debounce so one coin pays once
- Adding coins to leaderstats
- Spawning coins with a function instead of copy-paste
- Sound effects on pickup
- Respawning coins after a delay for endless play

Projects built:

- Coin rush: a field of spinning coins with pickup sounds, a live coin counter, and coins that respawn

Practice: Make a rare golden coin worth ten normal ones and place three somewhere genuinely hard to reach

Checkpoints

Topics:

- SpawnLocation objects and how respawning works
- Team-based checkpoints vs scripted checkpoints, and which we use
- Remembering each player's checkpoint in code
- Sending a respawned player to the right stage
- Checkpoint feedback: color change and sound when activated
- Why checkpoints make hard games fair instead of frustrating

Projects built:

- Checkpoint tower: a vertical obby where falling returns you to your highest activated checkpoint, not the bottom

Practice: Tune your tower: move one checkpoint to the fairest possible spot and defend the choice in one sentence

On-Screen Interfaces

Topics:

- ScreenGui, Frame, and TextLabel: putting words on the screen
- Updating a label from code: live coin and timer displays
- TextButton: a clickable button and its event
- Sizing and anchoring GUI so it works on different screens
- A welcome message that appears and fades
- Keeping GUI readable: contrast and size for real players

Projects built:

- Game HUD: an on-screen coin counter and stage display wired to real leaderstats, plus a

welcome banner

Practice: Add one more GUI element of your choice, such as a best-height display or a hint button

Assessment: Month 4 systems check: coins, checkpoints, leaderboard, and HUD all working together in one place, demonstrated live

PHASE 3: REAL GAME MECHANICS AND THE PUBLISHED CAPSTONE (MONTHS 5-6, WEEKS 17-24)

Shops, timers, rounds, and polish turn a course into a game people replay. Then the capstone month: design, build, playtest, and publish an original game to Roblox.

Month 5: Mechanics That Make Games Fun (Weeks 17-20)

The Shop

Topics:

- Spending coins: checking if a player can afford something
- A shop GUI with buy buttons
- Selling upgrades: speed, jump, and a trail effect
- Subtracting coins and applying the upgrade in one safe function
- Saying no nicely: what happens when coins run short
- Balancing prices so the game stays challenging

Projects built:

- Upgrade shop: a working store where coins collected in-game buy speed and jump upgrades

Practice: Add a third item to your shop and set all three prices, then explain your pricing to the class

Timers and Rounds

Topics:

- A countdown clock on screen, driven by a loop
- Round structure: waiting, playing, and finishing
- Announcing round results to everyone
- Teleporting all players to the arena and back
- Resetting the world between rounds
- Why rounds make players say one more game

Projects built:

- Round machine: a lobby-and-arena game loop with a countdown, a 90-second round, and a winners announcement

Practice: Adjust your round length and lobby time after two playtests, and note what changed in player behavior

Sound, Light, and Polish

Topics:

- Background music and where it belongs
- Sound effects for wins, coins, and traps, used tastefully
- Particles: sparkles, fire, and confetti in the right doses
- Lighting mood: making a lava zone feel dangerous
- Victory effects that reward without annoying
- The polish pass: a checklist run on your own game

Projects built:

- Polish pass: music, pickup sounds, victory confetti, and lighting applied to the round machine game

Practice: Do a sound-only playtest: close your eyes while a classmate plays your game and list what you can tell is happening

Project Week: The Complete Mini-Game

Topics:

- Assembling the phase: rounds, shop, coins, HUD, and polish in one game
- Finding the fun: the one change that most improves your game
- Bug hunting with a checklist instead of luck
- Watching testers without helping them
- Writing simple in-game instructions new players actually read
- Preparing a 2-minute demo of your own game

Projects built:

- Complete mini-game: a round-based coin game with a shop, HUD, sound, and polish, tested by classmates

Practice: Fix the three most confusing things your playtesters hit, in order of how much they hurt the game

Assessment: Mini-game demo day: every student's game played live by a classmate, followed by questions about the code

Month 6: The Capstone, Published to Roblox (Weeks 21-24)

Designing Your Game

Topics:

- Choosing a game you can actually finish in three weeks
- The one-page game design sheet: theme, goal, mechanics, win condition
- Sketching the map on paper before Studio opens
- Listing features as must-have, nice-to-have, and dream
- Estimating honestly: what four class weeks can hold
- Getting the plan approved before building starts

Projects built:

- Approved capstone plan: a design sheet and map sketch reviewed one-on-one with the instructor

Practice: Build your capstone's map skeleton: terrain, main structures, and spawn, with

no scripts yet
Build Sprint One: Core Mechanics

Topics:

- Building the must-have features first, smallest first
- Reusing your own code from earlier months, the honest shortcut
- Instructor code review: reading feedback and applying it
- Debugging on your own: reproduce, guess, test, fix
- Keeping the Explorer organized as the project grows
- Cutting a feature without shame when time asks for it

Projects built:

- Capstone core: the two or three mechanics that make your game itself, working end to end

Practice: Three 30-minute build sessions at home, each ending with one sentence on what now works that did not before

Build Sprint Two: Testing and Tuning

Topics:

- Full playtests with classmates as strangers
- Reading a playtest: stuck points, boredom points, and delight points
- Difficulty tuning: the numbers that change how a game feels
- Final GUI pass: title screen, instructions, and HUD
- Second code review and cleanup
- The finish line checklist: what done means for this game

Projects built:

- Feature-complete capstone: all must-have features in, tested by at least two classmates, tuned once

Practice: Run one family playtest and one classmate playtest, log every issue, fix the top two

Publish and Present

Topics:

- Publishing to Roblox from Studio, step by step, with a parent in the loop
- Game settings: name, description, icon, and who can play
- Privacy choices explained: public, friends-only, and what we recommend discussing at home
- The final bug sweep before going live
- Presenting your game: demo first, code second
- Where to go next: Python, more advanced game development, or a bigger Roblox project

Projects built:

- Published capstone: an original Roblox game live on the platform, presented to the class and family

Practice: Share your game link with three people outside the class and collect one piece of feedback from each

Assessment: Capstone presentation and code Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Floating-island playground built with Studio tools
- Ten-stage terrain obby with sculpted hills and caves
- Disco floor and instant staircase generated by loops
- Trap factory built from a single reusable function
- Obby 2.0 with kill bricks, power-ups, and scripted doors
- Coin rush field with spinning, respawning collectibles
- Checkpoint tower with fair respawn logic
- Round-based mini-game with shop, HUD, timer, and sound
- Published capstone: an original game live on Roblox

Total projects: 12+ finished builds and games, ending with one published to Roblox

SKILLS THE STUDENT WILL HAVE AT THE END

- Roblox Studio fluency: parts, properties, terrain, models, and lighting
- Lua fundamentals: variables, conditionals, loops, functions, and events written by hand
- Game systems: leaderboards, collectibles, checkpoints, shops, timers, and rounds
- GUI building: on-screen counters, buttons, and menus wired to live game data
- Debugging habits: reading errors, print-debugging, and the debounce pattern
- Project skills: scoping a game, playtesting honestly, and presenting your own work

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building and scripting along with the instructor

Practice: 2-3 hours of building challenges and script exercises between classes

Review: Homework builds reviewed with written feedback; stuck points reopened in the next class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck until the next session

Progress updates: Regular progress notes to parents: what was covered, what was built, what needs practice

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, backed by a published Roblox game

WHERE THIS CAN LEAD

- Our Python for kids course, where the logic learned in Luau transfers directly to the most-used language in the world
- Advanced game development courses, carrying over events, game loops, and level design instincts
- Bigger Roblox projects: many students keep building and publishing on their own after the course
- School coding clubs and game jams with a finished, published game already in hand
- Our JavaScript or app development tracks for students who want to build beyond gaming

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course run by Roblox?

A: No. Modern Age Coders is an independent coding school, and this course is not affiliated with or endorsed by Roblox Corporation. We teach children to build games using Roblox Studio, the free development tool that Roblox makes available to everyone, the same way a Python course teaches Python without being run by the Python Software Foundation.

Q: Is Roblox safe for my child? How do you handle safety in class?

A: Class time is spent in Roblox Studio, which is a development tool rather than the social game: students build and test their own worlds, not chat with strangers. We still recommend parents set up Roblox's parental controls on the child's account, including chat restrictions, and we walk parents through those settings on request. Publishing the capstone game is done together with a parent, and the privacy setting (public or friends-only) is the family's decision.

Q: Does my child need a paid Roblox account or Robux?

A: No. A free Roblox account is all that is required, for the entire course. Roblox Studio is free to download and use, and publishing a game to Roblox costs nothing. We never ask students to buy Robux, and no course project depends on any paid item.

Q: What exactly will my child learn to code?

A: Luau, the scripting language every Roblox game actually runs on. It is a real programming language (a Roblox-evolved version of Lua), so students learn genuine concepts: variables, conditionals, loops, functions, and events. Those ideas transfer directly to Python, JavaScript, and every language they meet afterwards; only the spelling changes.

Q: Is 8 years old too young for this? Is 14 too old?

A: The 8 to 14 range is deliberate. Eight-year-olds who read comfortably do well because month 1 is visual building and scripts arrive gradually, with typing kept short. Fourteen-year-olds are not bored because Luau is a real language and the later months involve genuine systems: leaderboards, shops, and round loops. Batches are grouped so students work alongside kids of similar age and pace.

Q: What computer do we need?

A: Roblox Studio does not run on phones, tablets, or most Chromebooks, so a proper computer is a requirement. If you are unsure whether your machine qualifies, tell us the model before the demo class and we will check.

Q: My child already plays Roblox for hours. Will this just mean more screen time?

A: It changes what the screen time is. Playing a game and building one exercise completely different muscles: planning, logic, debugging, and finishing something hard. Most parents tell us the course redirects hours the child was already spending, and that watching your own game get published beats another afternoon of playing someone else's.

Q: Can we try a class before enrolling?

A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in, and the demo is also where we confirm your computer runs Roblox Studio comfortably.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/roblox-game-coding-for-kids-lua-course>

=====

COURSE 109 OF 118: SAT MATH 800 PREP: LIVE CLASSES FOR THE DIGITAL SAT

=====

Course link: <https://learn.modernagecoders.com/courses/sat-math-prep-course>

Subject area: Test Prep Mathematics

Designed for: High school

Who can join: Grades 10-12, all starting scores

Course length: 6 months (26 weeks)

Time needed: 2 live one-hour classes per week + weekly timed practice

Certificate: Course completion certificate + documented mock-score progression

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online SAT Math prep for the Digital SAT: Heart of Algebra, Advanced Math, Data Analysis, Desmos strategy and full adaptive mock cycles with a mentor.

In one line: The Digital SAT rewards thinkers. We build the thinker, then train the test.

ABOUT THE PROGRAMME

A 6-month live prep program for the Digital SAT's math section, taught 1-on-1 or in small groups by a dedicated mentor. The SAT math section rewards structure recognition and clean algebra far more than memorized tricks, so we teach the mathematics underneath first, linear and nonlinear equations, functions, ratios, data, then layer on adaptive-test strategy: module pacing, Desmos calculator fluency, and when to solve versus when to backsolve. Every student runs full timed mock cycles with question-by-question review, so score movement is measured, not hoped for.

WHAT MAKES THIS COURSE DIFFERENT

- Understanding-first: we repair the algebra underneath before drilling question types, because that is where 600-scorers become 750-scorers
- Digital SAT native: adaptive module strategy and built-in Desmos calculator workflows taught as first-class skills
- A dedicated mentor who tracks your error log across the whole program, not a rotating cast
- Full mock cycles under real timing with question-by-question review against the reasoning, not just the answer key
- One full interactive hour per class, twice a week, no 25-minute check-ins
- Honest score guidance: a diagnostic in week one and evidence-based targets, never invented score promises
- Small groups capped tight, or fully private 1-on-1 mentorship
- Every class recorded for revision before test day

THE LEARNING PATH, STEP BY STEP

Phase 1: Diagnostic + the algebra core: linear equations, systems, inequalities and the function idea, repaired to fluency

Phase 2: Advanced Math and Problem-Solving & Data: quadratics, exponentials, ratios, percentages and data interpretation

Phase 3: Test craft: adaptive pacing, Desmos mastery, full mock cycles and error-log-driven polishing to peak on test day

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Grades 10-12 (or gap-year students retaking the SAT)

Math background: School algebra at any comfort level, the diagnostic finds the real starting point

Equipment: Computer with stable internet; the Digital SAT's Desmos calculator is built into practice tools

Time commitment: 2 live hours plus 2-3 practice hours weekly

English: Course taught in English

Motivation: A target test date, or our help choosing one

WHO THIS COURSE IS FOR

Score raisers: Students scoring 500-650 on practice tests who need the algebra core rebuilt, not more drills

Top end: Students at 700+ hunting the last 60-100 points through hard-module craft and error elimination

Strong at school: Teens with good school grades whose practice scores do not match, the classic craft gap

Planners: Grade 10 families starting early so the real test year is calm

Retakers: Students whose first attempt underperformed and who want a method, not more of the same

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE ALGEBRA CORE (WEEKS 1-8)

The Digital SAT's biggest domain is algebra. This phase diagnoses exactly where points are leaking and rebuilds linear equations, systems, inequalities and functions to reconstruction strength, the foundation every later question type stands on.

Weeks 1-2: Diagnostic + Linear Equations Rebuilt

Topics:

- Full timed diagnostic module with question-by-question review
- Personal error log opened: content gaps vs speed gaps vs careless patterns
- Linear equations and expressions from meaning, not moves
- Translating word problems into equations, the SAT's favorite disguise
- Slope and intercepts as rates and starting values in context
- Desmos introduction: graphing as a checking habit

Projects built:

- Personal score map: your exact domains, question types and target
- First timed mini-module under real Digital SAT pacing

Practice: Complete one timed linear equations mini set, log every miss in your error log as content, speed or careless, and redo the misses untimed.

Weeks 3-4: Systems and Inequalities

Topics:

- Systems solved three ways: substitution, elimination, Desmos intersection
- How many solutions? Parallel, intersecting and identical lines as ideas
- Linear inequalities and shaded regions read from the graph
- Systems word problems: mixtures, rates, break-even setups
- When solving is slower than testing answer choices, strategy honestly taught
- Timed reps: the 60-second system question

Projects built:

- Break-even mini-case solved by equation, table and graph
- Error-log review: your system-question mistakes classified and closed

Practice: Work a 15 question systems and inequalities set, solving at least five by Desmos intersection, and file each error in your log with its fix.

Weeks 5-6: Functions as Machines

Topics:

- Function notation read as a sentence: $f(x)$ is an output, not a mystery
- Evaluating, composing and interpreting functions in context
- Graph reading: intercepts, increasing/decreasing, transformations
- Linear function word problems and modeling
- Function questions in the calculator module: Desmos-first workflows
- Timed reps: function questions across both modules

Projects built:

- Function-transformation exploration built live in Desmos
- Mixed timed set: algebra domain at full pace

Practice: Finish a mixed function notation problem set, then rebuild two missed questions in Desmos to see what the graph was telling you.

Weeks 7-8: Algebra Domain Checkpoint

Topics:

- Full algebra-domain timed section under adaptive conditions
- Review ritual taught: why each miss happened, what changes next time
- Absolute value and compound inequalities, the fringe cases
- Careless-error engineering: writing habits that stop point leaks
- Module-1 pacing plan drafted against your speed data

Projects built:

- Algebra checkpoint mock + personal pacing plan v1
- Updated error log with closed vs open gap list

Practice: Take one timed algebra domain section and write a one line cause and correction for every dropped point before the review class.

PHASE 2: ADVANCED MATH + PROBLEM-SOLVING & DATA (WEEKS 9-18)

The domains that separate good scores from great ones: quadratics and exponentials taught as ideas, then the ratio-percent-data cluster where careless points go to die. Every topic is drilled at real SAT difficulty tiers, including the hard-module versions.

Weeks 9-11: Quadratics and Nonlinear Equations

Topics:

- Quadratics four ways: factoring, completing the square, formula, Desmos
- The discriminant as an idea: how many solutions and why the SAT asks
- Vertex form and parabola word problems (projectiles, revenue, area)
- Nonlinear systems: line meets parabola, read from the graph
- Exponent rules rebuilt from meaning; rational exponents and radicals
- Hard-module quadratic questions: recognizing the structure in costume

Projects built:

- Projectile/revenue modeling mini-case solved by vertex reasoning

- Timed hard-module quadratic set with full review
- Practice: Drill a 12 question hard module quadratic set, solving each question twice: once by algebra and once by the fastest Desmos route.

Weeks 12-14: Exponential Growth, Polynomials and Rational Expressions

Topics:

- Linear vs exponential growth: the comparison the SAT loves to test
- Exponential word problems: percent growth/decay, doubling, interest
- Polynomial arithmetic, zeros and graph behavior
- Rational expressions and equations without fear
- Isolating variables in formulas, the classic layout question
- Equivalent-expression questions: the algebra-fluency tax

Projects built:

- Growth-model comparison case: same data, linear vs exponential fits
- Advanced Math domain checkpoint under timing

Practice: Complete a percent growth and decay problem set plus five isolate the variable questions, keeping each layout question under a minute.

Weeks 15-18: Problem-Solving & Data Analysis

Topics:

- Ratio and proportion reasoning: the choose-the-easy-comparison skill
- Percent problems in every SAT costume: change, reverse, repeated
- Unit conversion and rates with dimensional discipline
- Reading tables, scatterplots, histograms and box plots fast
- Line of best fit, slope interpretation and prediction questions
- Mean, median, range and standard deviation as concepts
- Margin of error and study-design questions, what the SAT actually asks
- Two-way tables and conditional probability without formulas

Projects built:

- Data-interpretation sprint: 20 real chart questions under time
- Full mixed-domain mock: first complete Digital SAT math section

Practice: Run a 20 question ratio, percent and data interpretation sprint under time and classify every miss before class.

PHASE 3: ADAPTIVE TEST CRAFT + MOCK CYCLES (WEEKS 19-26)

Content becomes score in this phase: adaptive module strategy, Desmos speed workflows, geometry-and-trig completion, and a full mock cycle rhythm, test, review, repair, retest, that peaks the week of the real exam.

Weeks 19-20: Geometry and Trigonometry Domain

Topics:

- Area, volume and the reference-sheet formulas used intelligently
- Triangles: similarity, right-triangle trig, special right triangles
- Circles: arcs, sectors, and the circle equation
- Angle chasing with parallel lines and polygons
- Coordinate geometry: distance, midpoint, and geometric reasoning on the grid
- Trig on the SAT: exactly what is tested, nothing more

Projects built:

- Geometry domain checkpoint under timing
- Personal formula-and-recognition sheet, built not copied

Practice: Finish a geometry domain set using your own recognition sheet wherever possible, starring any formula you still had to look up.

Weeks 21-23: Adaptive Strategy and Desmos Mastery

Topics:

- How Digital SAT adaptivity works and what module 1 accuracy is worth
- Pacing plans per module built from your own timing data
- Desmos power workflows: solving, systems, regressions, tables at speed
- Solve vs backsolve vs plug-in: choosing the fastest honest route
- Flag-and-return discipline; the two-pass module method
- Guessing strategy and the no-blank rule, applied calmly

Projects built:

- Desmos speed drills: 15 questions where the calculator wins
- Strategy card: your personal decision rules for module 2

Practice: Do the 15 question Desmos speed drill, record your per question times and flag anything over 90 seconds for strategy review.

Weeks 24-26: Full Mock Cycles and Peak Week

Topics:

- Full adaptive mocks under exact timing, reviewed question by question
- Error-log closure: every recurring mistake gets a named fix
- Score-trend review with honest final-target calibration
- Peak-week plan: what to practice, what to rest, sleep and logistics
- Test-day routine: warm-up set, timing checkpoints, reset ritual after hard questions

Projects built:

- Two to three full mock cycles with documented score progression
- Written test-day plan, rehearsed twice before the real thing

Practice: Sit one full adaptive mock under exact timing this week and bring the question by question review with your named fixes.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal score map and error log maintained across all 6 months
- Desmos workflow library built by the student, question type by question type
- Two to three full adaptive mock cycles with documented score progression
- Personal pacing plans for both modules, built from real timing data
- Written test-day plan rehearsed before the real exam

Total projects: 5+ prep artifacts including full mock cycles

SKILLS THE STUDENT WILL HAVE AT THE END

- Linear equations, systems and inequalities at SAT speed
- Functions, quadratics and exponentials as ideas
- Ratio, percent and data-analysis fluency
- SAT geometry and trigonometry domain
- Desmos calculator power workflows
- Adaptive module pacing and two-pass strategy
- Error-log discipline and careless-point engineering
- Test-day routine and calm under the clock

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor
Timed practice: 1-2 timed sets or module sections per week
Review: Every practice set reviewed for reasoning, not just answers
Total per week: About 4-5 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole program
Recordings: Every class recorded for revision
Progress tracking: Error log + mock-score progression shared with parents
Flexibility: Slots move around school exams and family plans

CERTIFICATION DETAIL

Final certificate: Course completion certificate from Modern Age Coders
Score record: Documented diagnostic-to-final mock progression
Portfolio projects: Error log and pacing plans as evidence of method

WHERE THIS CAN LEAD

- A math score that strengthens selective-college applications
- AP Calculus and AP Statistics readiness, built on real algebra fluency
- Stronger merit-scholarship positioning at score-aware universities
- Ready for AP Calculus Exam Prep or the College Mathematics Masterclass

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course for the Digital SAT?

A: Yes, the course is built for the Digital SAT format: two adaptive math modules, the built-in Desmos graphing calculator, and current question styles. Module strategy and Desmos fluency are taught as first-class skills alongside the mathematics, and we verify current test details against College Board materials for your test date.

Q: What score improvement can we expect?

A: We will not invent a number, improvement depends on the starting score, weeks available and practice done between classes. What we promise instead: a real diagnostic in week one, a plan that attacks your highest-value gaps first, and a documented mock-score progression you can watch. Students who complete the mock cycles see their trend line clearly before test day.

Q: My teen scores well in school math but low on SAT practice tests. Why?

A: This is the most common profile we see. School rewards showing your work over weeks; the SAT rewards structure recognition under a clock. The gap is usually speed, question-reading and strategy rather than missing content, all trainable. The diagnostic separates content gaps from craft gaps in the first week.

Q: When should SAT math prep start?

A: The full program is 6 months, which comfortably fits a spring test from an autumn start. Starting closer to the test still works, the mentor compresses toward your highest-value domains and mock cycles, but more runway means the algebra core gets properly rebuilt rather

than patched.

Q: How is this different from a prep-book or app-based course?

A: Books and apps give practice; they cannot watch your teen think. A live mentor catches the broken step inside the work, the sign error, the misread, the slow route, and fixes the habit, not just the answer. That is why our classes are a full interactive hour with your teen solving and explaining aloud, twice a week.

Q: Do you cover the reading and writing section too?

A: This course is deliberately math-only, depth over bundling. Math is where a specialist mentor moves scores fastest, and where our maths-and-coding teaching method gives a real edge. Families wanting verbal prep can run it in parallel elsewhere without conflict.

Q: Can we try it before paying?

A: Yes. Every student starts with a free live demo class that doubles as the diagnostic, a real lesson plus an honest read on the starting point and target, with no card details required.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/sat-math-prep-course>

=====

COURSE 110 OF 118: SCRATCH PROGRAMMING FOR KIDS: 3-MONTH COURSE, 50+ PROJECTS

=====

Course link: <https://learn.modernagecoders.com/courses/scratch-programming-complete-course>

Subject area: Visual Programming & Game Development

Designed for: Kids

Who can join: Complete Beginner to Advanced Scratch Developer

Course length: 3 months (12 weeks)

Time needed: 5-7 hours/week recommended

Certificate: Certified Scratch Game Developer

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 3-month Scratch course for kids: go from zero to games, animations and interactive stories, building computational thinking with 50+ MIT Scratch projects.

In one line: From First Block to Game Developer - Master Creative Coding

ABOUT THE PROGRAMME

This intensive 3-month program transforms complete beginners into confident Scratch programmers capable of creating sophisticated games, animations, and interactive applications. Perfect for kids aged 7-16, teens, and even adults new to programming. Students will master every aspect of Scratch 3.0, from basic sprite movement to complex game mechanics, data structures, and multiplayer features. Through 50+ hands-on projects, learners develop computational thinking, problem-solving skills, and unleash their creativity while building an impressive portfolio.

WHAT MAKES THIS COURSE DIFFERENT

- Complete coverage of every Scratch feature
- 50+ progressively challenging projects
- Game design principles included
- Art and sound creation techniques
- Mathematical concepts through coding
- Story-telling and animation skills
- Community sharing and collaboration
- Preparation for text-based programming

THE LEARNING PATH, STEP BY STEP

Month 1: Foundations: Sprites, Motion, Events, and Basic Games

Month 2: Advanced Techniques: Variables, Lists, Clones, and Complex Logic

Month 3: Mastery: Advanced Games, Extensions, and Creative Projects

WHAT A STUDENT NEEDS BEFORE STARTING

Age: Ages 6-14 (designed for complete beginners)

Experience: Absolutely none - start from zero

Equipment: Computer or laptop with internet connection

Software: Scratch is free and browser-based - nothing to install

Reading: Basic reading ability to follow on-screen instructions

WHO THIS COURSE IS FOR

Complete beginners: Kids aged 6-14 with zero coding experience
Creative kids: Children who love games, stories, art, or music
Visual learners: Kids who learn best through visual, hands-on activities
Future coders: Children interested in eventually learning Python or JavaScript
Homeschoolers: Families looking for structured coding education

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

MONTH 1: SCRATCH FOUNDATIONS & BASIC GAME DEVELOPMENT

Master Scratch basics, create animations, and build your first games.

Introduction to Scratch & Basic Motion

Topics:

- What is programming? Why learn to code?
- Introduction to MIT Scratch platform
- Creating your Scratch account
- Scratch interface tour: Stage, Sprites, Blocks
- Understanding coordinates (x, y) system
- Sprite selection and management
- Basic motion blocks: move, turn, go to
- Using the green flag and stop button
- Saving and naming projects
- Sprite costumes and switching
- Stage backdrops basics
- First animation: making sprites move

Projects built:

- {'name': 'Hello World Animation', 'description': 'Sprite introduces itself and moves around', 'skills': 'Basic motion, say blocks, timing'}
- {'name': 'Aquarium Scene', 'description': 'Fish swimming with glide blocks', 'skills': 'Glide motion, multiple sprites, backdrop'}
- {'name': 'Dance Party', 'description': 'Characters dancing with costume changes', 'skills': 'Costume switching, rhythm, coordination'}
- {'name': 'Maze Navigator', 'description': 'Move sprite through simple maze with arrow keys', 'skills': 'Keyboard control, precise movement'}

Practice: Make your sprite dance across the stage with move and glide blocks, visiting all four corners using x and y coordinates.

Looks, Sound, and Interactivity

Topics:

- Advanced looks blocks and effects
- Graphic effects: color, fisheye, whirl, pixelate
- Size changes and animations
- Show and hide sprites strategically
- Layer ordering with go to front/back
- Adding sounds from library
- Recording custom sounds
- Sound effects and volume control
- Playing notes and creating melodies
- Tempo and beat in projects
- Mouse interaction and following
- Creating buttons and click events

Projects built:

- {'name': 'Interactive Music Maker', 'description': 'Piano with visual effects', 'skills': 'Sound blocks, mouse interaction, effects'}
- {'name': 'Animated Story Book', 'description': 'Story with sound effects and narration', 'skills': 'Timing, sound synchronization, scene changes'}
- {'name': 'Dress Up Game', 'description': 'Click to change character outfits', 'skills': 'Mouse clicks, costume logic, layering'}
- {'name': 'Magic Show', 'description': 'Sprites appear/disappear with effects', 'skills': 'Show/hide, graphic effects, timing'}

Practice: Give your sprite a costume change show that uses three graphic effects and plays a sound at each transformation.

Control Flow and Basic Logic

Topics:

- Understanding forever loops
- Repeat loops with specific counts
- If-then conditional statements
- If-then-else decision making
- Wait blocks and timing control
- Repeat until conditions
- Stop commands for sprites and scripts

- Combining multiple conditions
- Nested loops understanding
- Creating smooth animations with loops
- Basic collision detection
- Game over and winning conditions

Projects built:

- {'name': 'Catch the Falling Stars', 'description': 'Catch objects with increasing speed', 'skills': 'Forever loops, collision detection, scoring'}
- {'name': 'Traffic Light Simulator', 'description': 'Automated traffic control system', 'skills': 'Repeat loops, timing, state management'}
- {'name': 'Platform Jumper', 'description': 'Basic platformer with gravity', 'skills': 'Conditional logic, physics simulation'}
- {'name': 'Quiz Game', 'description': 'Multiple choice questions with scoring', 'skills': 'If-then-else, user input, logic flow'}

Practice: Build a guess the door scene where if then else blocks decide what happens behind each of three doors.

Pen Extension and Creative Drawing

Topics:

- Adding the Pen extension
- Pen down and pen up commands
- Drawing lines and shapes
- Changing pen color and shade
- Pen size adjustments
- Creating geometric patterns
- Stamps and creating trails
- Drawing with mouse control
- Mathematical art and fractals
- Spirograph patterns
- Clearing and managing drawings
- Creating drawing applications

Projects built:

- {'name': 'Etch-a-Sketch', 'description': 'Drawing app with keyboard control', 'skills': 'Pen control, user input, clearing'}
- {'name': 'Geometric Art Generator', 'description': 'Create mathematical patterns', 'skills': 'Loops with pen, angles, repetition'}
- {'name': 'Rainbow Spiral Maker', 'description': 'Colorful spiral patterns', 'skills': 'Color changes, mathematical spirals'}
- {'name': 'Signature Pad', 'description': 'Mouse-controlled drawing', 'skills': 'Mouse tracking, pen following'}

Practice: Draw a rainbow spiral with the pen extension by changing the pen color inside a repeat loop.

MONTH 2: ADVANCED SCRATCH TECHNIQUES

Master variables, lists, clones, and build complex games.

Variables and Score Systems

Topics:

- What are variables? Real-world examples
- Creating variables for all sprites vs. this sprite only
- Setting and changing variable values
- Displaying variables on stage
- Creating score systems
- Lives and health systems
- Timers and countdowns
- High score tracking
- Currency systems for games
- Variable-based animations
- Random number generation
- Mathematical operations with variables

Projects built:

- {'name': 'Cookie Clicker Game', 'description': 'Incremental game with upgrades', 'skills': 'Variables, click counting, cost systems'}
- {'name': 'Racing Game with Laps', 'description': 'Track laps and best times', 'skills': 'Timer variables, lap counting, records'}
- {'name': 'RPG Character Stats', 'description': 'Health, mana, experience system', 'skills': 'Multiple variables, stat management'}
- {'name': 'Math Challenge Game', 'description': 'Timed math problems with scoring', 'skills': 'Random numbers, calculations, timing'}

Practice: Create a clicker game where a score variable rises with every click and the sprite celebrates at ten points.

Lists and Data Management

Topics:

- Understanding lists (arrays)
- Creating and naming lists
- Adding items to lists

- Deleting items from lists
- Inserting at specific positions
- Replacing list items
- Finding items in lists
- List length and iteration
- Using lists for inventories
- Leaderboards and high scores
- Question banks for quizzes
- Shopping lists and collections

Projects built:

- {'name': 'Memory Card Game', 'description': 'Match pairs using lists', 'skills': 'List shuffling, pair matching, memory'}
- {'name': 'Word Guessing Game', 'description': 'Hangman-style with word lists', 'skills': 'String manipulation, list iteration'}
- {'name': 'Inventory System RPG', 'description': 'Collect and manage items', 'skills': 'Dynamic lists, item management'}
- {'name': 'Quiz Bowl Game', 'description': 'Questions from lists', 'skills': 'Random selection, list traversal'}

Practice: Make a favorites program that stores five favorite foods in a list and says a random one when the space key is pressed.

Clones and Advanced Game Mechanics

Topics:

- Understanding clones concept
- Creating clones of sprites
- When I start as a clone
- Clone-specific behaviors
- Deleting clones properly
- Managing multiple clones
- Bullet systems in games
- Enemy spawning mechanics
- Particle effects with clones
- Power-ups and collectibles
- Clone collision detection
- Performance optimization

Projects built:

- {'name': 'Space Shooter', 'description': 'Shoot bullets at enemy clones', 'skills': 'Clone bullets, enemy waves, collision'}
- {'name': 'Tower Defense Game', 'description': 'Defend against clone enemies', 'skills': 'Path following, tower placement, waves'}
- {'name': 'Fireworks Simulator', 'description': 'Particle effects with clones', 'skills': 'Visual effects, physics, timing'}
- {'name': 'Asteroid Field', 'description': 'Navigate through clone obstacles', 'skills': 'Random spawning, difficulty scaling'}

Practice: Build a falling snowflake shower using clones that appear at random x positions and delete themselves at the bottom.

Broadcast Messages and Multi-Sprite Coordination

Topics:

- Understanding broadcast messages
- When I receive message
- Broadcast and wait
- Scene management with broadcasts
- Menu systems and navigation
- Coordinating multiple sprites
- Turn-based game mechanics
- Cutscenes and dialogues
- Game state management
- Pause and resume functionality
- Level transitions
- Achievement notifications

Projects built:

- {'name': 'RPG with Dialogue System', 'description': 'Story-driven game with conversations', 'skills': 'Message coordination, dialogue trees'}
- {'name': 'Multi-Level Platformer', 'description': 'Complete game with level progression', 'skills': 'Scene management, state tracking'}
- {'name': 'Turn-Based Battle System', 'description': 'Pokemon-style combat', 'skills': 'Turn management, action queuing'}
- {'name': 'Escape Room Puzzle', 'description': 'Multiple interconnected puzzles', 'skills': 'Puzzle logic, item interaction'}

Practice: Create a two scene story where a broadcast message switches the backdrop and cues the next character to speak.

MONTH 3: MASTERY AND ADVANCED PROJECTS

Create professional games, master extensions, and prepare for text coding.

Advanced Extensions and Hardware

Topics:

- Music extension advanced features
- Creating dynamic soundtracks
- Text to Speech capabilities
- Language translation features
- Video Sensing for motion detection
- Creating motion-controlled games
- Makey Makey integration basics
- micro:bit connection and control
- LEGO Mindstorms integration
- Using webcam for interaction
- Machine Learning extension intro
- Creating AI-powered projects

Projects built:

- {'name': 'Motion-Controlled Game', 'description': 'Control with body movements', 'skills': 'Video sensing, motion tracking'}
- {'name': 'Multi-Language Quiz', 'description': 'Quiz with translation features', 'skills': 'Translation, text-to-speech'}
- {'name': 'AI Pet Game', 'description': 'Pet that responds to actions', 'skills': 'Machine learning basics, interaction'}
- {'name': 'Musical Composition Tool', 'description': 'Create and play music', 'skills': 'Music extension mastery'}

Practice: Use the Text to Speech extension to make a talking pet that greets you in two different languages.

Physics and Simulation

Topics:

- Implementing gravity
- Jump mechanics and physics
- Velocity and acceleration
- Collision detection advanced
- Bouncing and elasticity
- Friction simulation
- Pendulum motion
- Projectile motion
- Water and fluid effects
- Particle systems advanced
- Realistic movement patterns
- Physics-based puzzles

Projects built:

- {'name': 'Angry Birds Clone', 'description': 'Physics-based puzzle game', 'skills': 'Projectile physics, destruction'}
- {'name': 'Pinball Machine', 'description': 'Realistic ball physics', 'skills': 'Bouncing, flippers, gravity'}
- {'name': 'Solar System Simulator', 'description': 'Orbital mechanics', 'skills': 'Circular motion, gravity simulation'}
- {'name': 'Fluid Dynamics Demo', 'description': 'Water and particle effects', 'skills': 'Particle systems, flow simulation'}

Practice: Give a character a real jump using a velocity variable, gravity and a landing check on a platform.

Optimization and Professional Polish

Topics:

- Performance optimization techniques
- Reducing lag and improving FPS
- Efficient clone management
- Sprite and backdrop optimization
- Code organization and comments
- Creating custom blocks (functions)
- Debugging techniques
- Error handling strategies
- Testing and quality assurance
- User interface design
- Menu and navigation systems
- Settings and options menus

Projects built:

- {'name': 'Professional Game UI', 'description': 'Complete menu system', 'skills': 'Navigation, settings, polish'}
- {'name': 'Optimized Particle System', 'description': 'Efficient visual effects', 'skills': 'Performance, visual quality'}
- {'name': 'Debugger Tool', 'description': 'In-game debugging system', 'skills': 'Testing, monitoring, logging'}
- {'name': 'Modular Game Engine', 'description': 'Reusable game components', 'skills': 'Custom blocks, organization'}

Practice: Tidy up one of your older projects by adding comments to three scripts and merging any blocks that repeat.

Final Projects and Portfolio

Topics:

- Planning large projects
- Game design documentation
- Project management basics
- Version control concepts
- Sharing and remixing ethics
- Scratch community guidelines
- Creating tutorials for others
- Portfolio presentation
- Transitioning to text coding
- JavaScript similarities
- Python connections
- Future learning paths

Practice: Write a one page plan for your final game with its goal, its sprites and three rules, then build the title screen.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1 (Weeks 1-4): 12+ Scratch projects - animations, drawings, interactive stories
- Month 2 (Weeks 5-8): 12+ games - score systems, clones, multi-sprite coordination
- Month 3 (Weeks 9-12): 12+ advanced projects - physics simulations, extensions, final portfolio

Total projects: 36+ Scratch projects + Final Capstone Game

SKILLS THE STUDENT WILL HAVE AT THE END

- Scratch Basics: sprites, motion, looks, sound, pen blocks, backdrops
- Programming Logic: loops, conditionals, events, variables, operators
- Data Structures: lists, data management, cloud variables
- Game Development: score systems, collision detection, clones, physics
- Animation: character animation, scene transitions, storytelling
- Advanced: extensions (music, video sensing, text-to-speech), optimization
- Professional Skills: debugging, code organization, project planning

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (1-1.5 hours each)
 Independent practice: 2-3 hours of project work
 Weekly challenge: 1 creative coding challenge
 Total per week: 5-7 hours

SUPPORT INCLUDED

Live instruction: Expert-led live classes with recorded replays
 Age appropriate: Content designed for kids - fun, visual, and encouraging
 Parent guide: Parent & teacher resources for supporting learning at home
 Community: Access to Modern Age Coders kids coding community
 Transition path: Clear pathway to Python/JavaScript after course completion

CERTIFICATION DETAIL

Final certificate: Certified Scratch Game Developer (Modern Age Coders)
 Capstone required: Complete original game with multiple levels, score system, and polished UI
 Portfolio: 36+ completed Scratch projects in online portfolio

WHERE THIS CAN LEAD

- Junior Game Developer
- Creative Coder
- Animation Creator
- Interactive Media Designer
- Ready for Python/JavaScript transition

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my child actually create in this Scratch course?

A: Over 3 months your child builds 50+ hands-on projects in Scratch, starting with simple animations and working up to full games with scoring systems, clones, physics, and multi-sprite coordination. Month 1 covers motion, looks, sound, and logic; month 2 adds variables, lists, and game mechanics; month 3 finishes with advanced projects and a personal portfolio.

Q: What age is this Scratch course right for?

A: The course is designed for ages 6 to 14 and assumes zero coding experience. All your child needs is basic reading ability to follow on-screen instructions; lessons are visual and hands-on, which suits younger learners especially well.

Q: Does my child need any coding background before joining?

A: None at all. Scratch was created by MIT precisely for first-time coders: children snap colorful blocks together instead of typing code, so there is no syntax to memorize. The course starts from the very first block.

Q: What comes after Scratch?

A: Scratch builds the computational thinking that makes text-based languages far easier later. The course ends with a guided transition path toward typed code, and most students continue into our Python or web development courses when they are ready.

Q: Can we try a free class before enrolling?

A: Yes, every child can take a free demo class first, with no commitment of any kind. Book a slot at learn.modernagecoders.com/contact, and you are welcome to sit in and watch; you can also reach us on WhatsApp at +91 91233 66161 with any questions.

Q: Are classes live or pre-recorded?

A: All classes are live 1-hour online sessions taught by a real teacher; we never use pre-recorded videos. Batches are small and interactive, so your child builds projects during class while the teacher watches and helps in real time.

Q: Does my child receive a certificate at the end?

A: Yes, students earn a completion certificate when they finish the course. They also leave with a portfolio of real Scratch games and animations, which for most kids is the part they are proudest of.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/scratch-programming-complete-course>

=====

COURSE 111 OF 118: STATISTICS & PROBABILITY: DATA TO HYPOTHESIS TESTING

=====

Course link: <https://learn.modernagecoders.com/courses/statistics-probability-maths-course>

Subject area: Data Science & Analytics Mathematics

Designed for: College students

Who can join: College students and working professionals; no programming required

Course length: 6 months (24 weeks)

Time needed: 2 live classes/week + 3-4 hours practice

Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Online statistics and probability course for college and professionals: descriptive stats, distributions, hypothesis testing and regression on real data.

In one line: The subject that decides whether a claim is evidence or noise, taught on real data from the first week.

ABOUT THE PROGRAMME

Statistics is the working mathematics of almost every field that touches data: business, medicine, psychology, economics, engineering, machine learning. It is also routinely taught backwards, as formula memorisation first and meaning never. This 6-month live course teaches it the other way. Months 1 and 2 build the two foundations: describing data honestly, and probability as the grammar of uncertainty, including conditional probability and Bayes' theorem worked through real cases like medical test results. Months 3 and 4 cover the distributions that model the world, then sampling, the central limit theorem seen through simulation rather than assertion, and confidence intervals interpreted the way they actually work. Month 5 is hypothesis testing done properly: t-tests, proportion tests and chi-square, with equal attention to what p-values do not say. Month 6 covers correlation and regression, then a capstone: a full analysis of a real dataset of your choosing, written up with methods and limitations. Every technique is practised on real data in a spreadsheet, and an optional Python track runs alongside for those who want it. Neither is required knowledge coming in; one of them becomes a working skill going out.

WHAT MAKES THIS COURSE DIFFERENT

- Real datasets from week 1: public health, sports, economics and business data, not textbook tables invented to make the arithmetic clean
- Spreadsheet-first with an optional Python track, so non-programmers finish everything and programmers get the pandas version of each technique
- Probability taught through consequential examples: false positives in medical testing, base-rate mistakes, and why intuition fails on conditional probability
- The central limit theorem demonstrated by simulation you run yourself, because a theorem you have watched happen is one you actually trust

- Honest inference: what a p-value is and is not, why statistical significance is not importance, and how p-hacking happens
- A capstone analysis of a dataset you choose, written up with methods and limitations like a piece of real analytical work

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations: describing data with the right summaries and charts, then probability, conditional probability, Bayes' theorem and random variables

Phase 2: Models and estimation: binomial, Poisson and normal distributions, sampling and the central limit theorem, and confidence intervals done honestly

Phase 3: Inference and the capstone: hypothesis tests for means, proportions and categories, correlation and regression, then a full real-data analysis of your own

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable school algebra: rearranging formulas and working with fractions and percentages. No calculus is needed anywhere in the course

Programming: None. Everything can be completed in a spreadsheet; the Python track is optional and taught from zero for those who choose it

Equipment: A computer with a spreadsheet application (Excel or Google Sheets) and a stable internet connection

Audience: Built for college students and working professionals; motivated final-year school students who clear the diagnostic are welcome

WHO THIS COURSE IS FOR

College students: Students in commerce, economics, psychology, biology or engineering whose degrees assume statistics that was never properly taught

Working professionals: Analysts, marketers, product managers and operations people who face dashboards and A/B tests daily and want to stop nodding along

Data science aspirants: People headed for data science or machine learning who know the statistics half is the half they are missing

Researchers: Postgraduate students and early researchers who need to design studies and read papers with methods sections

Non programmers: Anyone who wants rigorous data skills without being forced through a programming course to get them

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: DESCRIBING DATA AND PROBABILITY (MONTHS 1-2, WEEKS 1-8)

The two foundations everything else stands on: honest description of data, and probability as the language of uncertainty.

Month 1: Describing Data Honestly (Weeks 1-4)

Data: What It Is and How It Lies to You

Topics:

- Populations, samples and why the difference drives everything
- Types of data: categorical and numerical, and the scales of measurement
- Tidy data: rows, columns and the shape analysis expects
- Where data comes from: surveys, experiments, logs and their built-in biases
- Spreadsheet setup: importing, cleaning basics and first summaries
- Famous data failures: survivorship bias and selection bias with real stories

Projects built:

- First dataset walk: load a real public dataset, document its variables, units and suspicious values before computing anything

Practice: Profile two provided datasets: identify variable types, spot three data-quality problems, write one paragraph on each

Central Tendency: What Is Typical

Topics:

- Mean, median and mode, and what question each one answers
- How skew and outliers pull the mean away from the median
- Weighted means and when unweighted averages mislead
- Group comparisons: the danger of comparing averages of unequal groups
- Computing summaries in the spreadsheet with named functions
- Reading claims critically: what average salary headlines usually hide

Projects built:

- Income exercise: a salary dataset summarised with mean and median, with a written note on why they disagree and which to report

Practice: 12 central tendency problems on real data, each answer accompanied by one sentence on which measure fits and why

Spread: How Much Things Vary

Topics:

- Range and interquartile range
- Variance and standard deviation, built up from deviations rather than dropped as a formula

- The 1.5 IQR rule for flagging outliers
- z-scores: how unusual is this value
- Comparing variability across different units with the coefficient of variation
- Why spread is the half of the story that dashboards leave out

Projects built:

- Consistency study: two players or products with the same average compared on spread, with a recommendation written up

Practice: 14 spread problems including 4 z-score questions, standard deviation computed once by hand and thereafter by software

Charts That Tell the Truth

Topics:

- Histograms and the shapes of distributions: symmetric, skewed, bimodal
- Box plots and comparing groups at a glance
- Scatter plots and the first look at relationships
- Bar charts versus histograms, and why the difference matters
- Misleading charts: truncated axes, cherry-picked windows, area distortions
- Building clean charts in the spreadsheet, with labels that earn their space

Projects built:

- Chart audit: find one misleading chart in the wild, explain the distortion, and rebuild it honestly from the underlying data

Practice: Build five charts from a real dataset, each with a one-line takeaway written underneath

Assessment: Month 1 checkpoint: a full descriptive analysis of a provided dataset, summaries, charts and a short written brief

Month 2: Probability, the Grammar of Uncertainty (Weeks 5-8)

Probability from First Ideas

Topics:

- Experiments, outcomes, sample spaces and events
- Probability as long-run frequency and as degree of belief
- The complement rule and the addition rule
- Mutually exclusive events versus events that overlap
- Venn diagrams that actually resolve arguments
- Simulating simple probabilities in the spreadsheet to check intuition

Projects built:

- Simulation warm-up: estimate a dice probability by simulation, compare with the exact answer, watch the gap shrink with more trials

Practice: 16 probability problems from single events to unions, each hard one checked by a quick simulation

Conditional Probability and Independence

Topics:

- Conditional probability: how information changes the odds
- The multiplication rule and tree diagrams
- Independence: what it means and how to test for it
- The base-rate fallacy introduced with a real screening example
- Sampling with and without replacement
- Why $P(A \text{ given } B)$ and $P(B \text{ given } A)$ are different, and the famous confusions

Projects built:

- Two-way table study: a real dataset cross-tabulated, with conditional probabilities read off and independence checked

Practice: 14 conditional probability problems, every tree diagram drawn before any multiplication

Bayes' Theorem: Updating Beliefs with Evidence

Topics:

- Bayes' theorem derived from the definition of conditional probability
- The medical test problem: why a positive result on a rare disease is usually a false alarm
- Prior, likelihood and posterior in plain language
- Natural frequencies: the counting method that makes Bayes intuitive
- Spam filters and diagnostic reasoning as everyday Bayes
- The prosecutor's fallacy and other courtroom probability errors

Projects built:

- Bayes explainer: work the medical test problem with natural frequencies and write an explanation a friend could follow

Practice: 10 Bayes problems solved twice each, once by formula and once by counting, until the two views agree in your head

Counting and Random Variables

Topics:

- Permutations and combinations, and which one the question wants
- Discrete random variables and probability distributions
- Expected value: what fair means, and why casinos always win
- Variance of a random variable
- Expected value in decisions: insurance, warranties and lotteries
- Building a probability distribution from real frequency data

Projects built:

- Game analysis: compute the expected value of a real lottery or carnival game and write the verdict in two sentences

Practice: 12 counting and random variable problems including 3 expected-value decisions argued in writing

Assessment: Month 2 checkpoint: a mixed probability paper from basic rules through

PHASE 2: DISTRIBUTIONS, SAMPLING AND ESTIMATION (MONTHS 3-4, WEEKS 9-16)

The named distributions that model real processes, then the machinery of inference: sampling distributions, the central limit theorem seen by simulation, and confidence intervals.

Month 3: The Distributions That Model the World (Weeks 9-12)

The Binomial Distribution

Topics:

- Bernoulli trials and the binomial conditions, checked in words
- The binomial formula and where it comes from
- Mean and variance of a binomial
- Computing binomial probabilities in the spreadsheet and, on the Python track, with scipy
- Cumulative probabilities and careful at-least versus at-most reading
- Real binomial settings: quality control, conversion rates, free-throw streaks

Projects built:

- Conversion study: model a marketing conversion scenario as a binomial and answer three business questions with it

Practice: 14 binomial problems, the conditions checked in writing before any computation

Poisson and Geometric: Counts and Waits

Topics:

- The Poisson distribution for counts per interval: arrivals, defects, calls
- When Poisson fits and when it does not
- Poisson as a limit of the binomial, seen numerically
- The geometric distribution: waiting for the first success
- Choosing the right model: binomial, Poisson or geometric from the story
- Fitting a Poisson to real count data and judging the fit by eye

Projects built:

- Arrivals study: fit a Poisson model to real count data, compare observed and expected counts in a table

Practice: 12 modelling problems where the first marked step is naming the right distribution and defending the choice

Continuous Distributions and the Normal

Topics:

- From histograms to density curves: probability as area
- The uniform distribution as the simplest continuous case
- The normal distribution: shape, parameters and the 68-95-99.7 rule
- Standardising and the standard normal
- Normal probabilities by software, with the table method understood once
- What is and is not normal in real data: heights yes, incomes no

Projects built:

- Normal check: test the 68-95-99.7 rule against a real dataset and report where it holds and where it breaks

Practice: 14 normal distribution problems, each one opened with a sketch of the shaded region

Working with the Normal

Topics:

- Inverse problems: finding cutoffs from probabilities
- Percentiles, reference ranges and grading on a curve
- Assessing normality with histograms and quantile plots
- The normal approximation to the binomial and when it is safe
- Combining independent normal quantities
- A distribution field guide: one page mapping situations to models

Projects built:

- Field guide build: your own one-page reference mapping real situations to binomial, Poisson, geometric, uniform or normal

Practice: 12 mixed distribution problems where the model is never announced in the question

Assessment: Month 3 checkpoint: a distributions paper mixing model choice, computation and interpretation

Month 4: Sampling, the CLT and Confidence Intervals (Weeks 13-16)

Sampling: How Data Gets Collected and Corrupted

Topics:

- Simple random, stratified, cluster and systematic sampling
- Convenience samples and why most internet polls mean little
- Sampling bias, nonresponse bias and question wording effects
- Famous polling failures and what went wrong
- Randomised experiments versus observational studies
- Designing a small survey that will not embarrass you

Projects built:

- Survey critique: take one published poll, identify its sampling method, and write three specific bias risks

Practice: 10 sampling design problems plus one short survey designed, with its weaknesses honestly listed

Sampling Distributions and the Central Limit Theorem

Topics:

- The sampling distribution of the mean: a distribution of statistics, not data
- Standard error and why bigger samples pin the mean down
- The central limit theorem stated carefully
- The CLT demonstrated by simulation: skewed populations, normal-looking sample means
- How sample size affects the shape, watched live
- Why the CLT is the load-bearing wall of everything that follows

Projects built:

- CLT simulation: run the resampling experiment yourself in the spreadsheet or Python, and keep the charts as evidence

Practice: 10 standard error and sampling distribution problems, plus the simulation repeated on a second population shape

Confidence Intervals for Means

Topics:

- The logic of interval estimation
- z-intervals and t-intervals, and when the t-distribution is the honest choice
- Degrees of freedom in plain language
- What 95 percent confidence actually means, and the misreadings to avoid
- Interval width: the three levers of confidence, spread and sample size
- Computing and reporting intervals from real data

Projects built:

- Interval report: a confidence interval for a real dataset's mean, reported in one correct sentence and one wrong-but-common sentence, labelled

Practice: 12 confidence interval problems, every interval followed by a correctly worded interpretation

Proportions and Planning Sample Sizes

Topics:

- Confidence intervals for proportions
- Margin of error and how polls report it
- Sample size calculations: how many people do we actually need
- Reading polling small print: confidence level, margin, population
- Intervals for differences, previewed
- Estimation recap: the full toolkit so far, organised

Projects built:

- Poll deconstruction: take a real published poll and verify its stated margin of error from its sample size

Practice: 12 proportion and sample size problems including two taken from real published surveys

Assessment: Month 4 checkpoint: an estimation paper from sampling design through confidence intervals, interpretation marked as strictly as computation

----- PHASE 3: HYPOTHESIS TESTING, REGRESSION AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24) -----

Testing claims with data, measuring and modelling relationships, and then a complete analysis of a real dataset of your own, written up like professional work.

Month 5: Hypothesis Testing Done Honestly (Weeks 17-20)

The Logic of a Hypothesis Test

Topics:

- Null and alternative hypotheses: the courtroom analogy done carefully
- Test statistics and the p-value defined precisely
- Significance levels and the arbitrary honesty of 0.05
- Type I and Type II errors, and which one your situation fears more
- What a p-value is not: not the probability the null is true
- One-tailed versus two-tailed tests and how to choose before seeing data

Projects built:

- Error analysis: for three real scenarios, name both error types and argue which is costlier in each

Practice: 10 test-logic problems with no computation at all: hypotheses, tails and error types only

One-Sample Tests

Topics:

- The one-sample t-test for a mean, start to finish
- The one-sample proportion test
- Checking conditions before trusting the test
- Computing tests in the spreadsheet and, on the Python track, with scipy
- Writing conclusions in context, with the p-value reported not worshipped
- Confidence intervals and tests as two views of the same evidence

Projects built:

- Claim check: test a real advertised claim against real or provided data, and write the verdict in three sentences

Practice: 12 one-sample tests, each conclusion written in context and checked against its confidence interval

Comparing Two Groups

Topics:

- Independent two-sample t-tests
- Paired tests and recognising paired designs
- Two-proportion tests: the A/B test in its natural form

- Effect size: statistically significant versus actually important
- How A/B testing works in product teams, and where it goes wrong
- Reading a published study's methods section without fear

Projects built:

- A/B analysis: a realistic conversion dataset analysed end to end, with a recommendation that weighs effect size, not just p

Practice: 12 two-group problems including one paired design hiding among independent ones

Chi-Square and the Ethics of Testing

Topics:

- Chi-square goodness of fit: does the data match the claimed distribution
- Chi-square test of independence on two-way tables
- Expected counts and when the test is valid
- Multiple comparisons: why testing twenty things finds one false discovery
- p-hacking and publication bias: how honest people fool themselves
- Pre-registering your question: deciding what to test before looking

Projects built:

- Independence study: a real two-way table tested for independence with the full expected-count working shown

Practice: 10 chi-square problems plus a written half-page on one real p-hacking case

Assessment: Month 5 checkpoint: a full inference paper across one-sample, two-sample and chi-square tests, conclusions marked for wording

Month 6: Correlation, Regression and the Capstone (Weeks 21-24)

Correlation

Topics:

- Covariance and the Pearson correlation coefficient
- What r measures and the linear-only caveat
- Anscombe's quartet: four datasets, one r, four different stories
- Correlation is not causation, with the confounder named each time
- Rank correlation for curved but monotonic relationships
- Computing and charting correlations across a real dataset

Projects built:

- Correlation hunt: find the three strongest relationships in a real dataset, chart them, and flag which might be confounded

Practice: 10 correlation problems, every claim of relationship backed by a scatter plot

Linear Regression

Topics:

- The least squares line: what is being minimised and why
- Interpreting slope and intercept in the units of the problem
- R-squared: how much variation the line explains
- Residual plots: the honesty check most analyses skip
- Prediction within range versus extrapolation beyond it
- Multiple regression previewed: several predictors at once, and its dangers

Projects built:

- Prediction model: fit a regression on real data, check residuals, make two predictions and state their limits

Practice: 10 regression problems with slope interpretations written in context every time

Assessment: Capstone gate: analysis plan approved, dataset, question, methods and chosen track (spreadsheet or Python)

Capstone Build

Topics:

- Framing an answerable question before touching the data
- Cleaning and documenting the dataset
- Choosing methods that fit the data, not the other way around
- Descriptive pass first: summaries and charts before any test
- Running the inference or regression the question calls for
- Instructor review checkpoint mid-build

Projects built:

- Capstone analysis in progress: the full pipeline from raw data to preliminary findings, on your own chosen dataset

Practice: Capstone work sessions with a written progress note after each: what was found, what is uncertain, what is next

Capstone Write-Up and Presentation

Topics:

- Structuring an analysis report: question, data, methods, findings, limitations
- Writing limitations honestly: what your analysis cannot claim
- Charts chosen for the reader, not the author
- Presenting analytical work in ten minutes
- Answering methods questions about your own work
- Where to go next: deeper statistics, Python for data, or machine learning

Projects built:

- Completed capstone: a written analysis report and a short presentation, delivered to the class and kept for your portfolio

Practice: Final report polish plus a dry-run presentation with peer questions

Assessment: Capstone review: report and presentation assessed on method, honesty of interpretation and clarity, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A full descriptive analysis brief on a real public dataset
- A Bayes explainer of the medical test problem in your own words
- An expected-value verdict on a real lottery or game
- A Poisson model fitted to real count data
- A CLT simulation run and charted by you
- A deconstruction of a real published poll's margin of error
- An A/B test analysed end to end with an effect-size-aware recommendation
- A regression prediction model with residual checks
- The capstone: a complete written analysis of a dataset you chose

Total projects: A portfolio of small real-data studies plus one complete capstone analysis with report and presentation

SKILLS THE STUDENT WILL HAVE AT THE END

- Describing data honestly with the right summaries and charts
- Probability, conditional probability and Bayes' theorem used correctly
- Choosing and applying binomial, Poisson and normal models
- Confidence intervals and hypothesis tests, computed and worded properly
- Correlation and regression with the honesty checks included
- Spreadsheet analysis throughout, plus optional Python with pandas and scipy on the parallel track

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, working on real data during class

Practice: 3-4 hours weekly of problem sets and dataset work between classes

Review: Homework reviewed with written feedback; interpretation wording corrected as strictly as calculations

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems

Progress updates: Monthly progress notes covering test scores and capstone readiness

Career guidance: Honest guidance on next steps: which roles use these skills, what to learn next, and what this course alone does and does not qualify you for

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, backed by your capstone analysis report

WHERE THIS CAN LEAD

- Data analyst and reporting analyst roles, where descriptive statistics and inference are the daily toolkit
- Business and product analytics: A/B testing, metrics and experiment literacy
- A prepared entry into data science and machine learning courses, including our Python and AI/ML tracks
- Research work in any field with a methods section: psychology, economics, public health
- Better decisions in your current role, which is the quiet majority use of statistics

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for roles where statistics is a core skill, in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-6 LPA (Data Analyst / Reporting Analyst / MIS Analyst)

Growing roles: Rs 6-12 LPA (Senior Analyst / Business Analyst / Product Analyst)

Experienced roles: Rs 12-25 LPA (Analytics Lead / Data Scientist, typically with added programming and ML skills)

International: \$55k-110k USD (USA) for analyst roles, depending on role, industry and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need to know programming for this course?

A: No, and this is a real no, not a marketing one. Every technique in the course, from descriptive statistics through regression, is taught and assessed in a spreadsheet, and the capstone can be completed entirely without code. A parallel Python track runs for those who want it: the same analyses redone with pandas and scipy, taught from zero. Roughly speaking, the spreadsheet track gives you the statistics; adding the Python track gives you the statistics plus a tool employers increasingly expect.

Q: How much maths do I need coming in?

A: School algebra: you should be able to rearrange a simple formula and be comfortable with fractions, percentages and squared terms. There is no calculus in the course. Where a result normally requires calculus to derive, we demonstrate it numerically or by simulation instead, which is often more convincing anyway. The week 1 diagnostic catches genuine gaps, and we patch them in the early weeks rather than turning you away.

Q: How is this different from a data science course?

A: A data science course is mostly tooling: Python, data wrangling, machine learning libraries, deployment. This course is the statistical reasoning underneath, which data science courses typically compress into two rushed weeks and which is exactly where most self-taught data scientists are weakest. If your goal is data science, this course first and a data science course second is the strong order. If your goal is to understand data, studies and experiments in your existing work, this course alone may be all you need.

Q: What is the capstone project, and do I choose the dataset?

A: In the final month you take one real dataset, yours from work, or one you choose from public sources with our help, frame a question, run the appropriate analysis and write it up with methods and limitations, then present it. The write-up format mirrors how analytical work is actually reported. Students on the spreadsheet track and the Python track complete the same capstone with different tools, and both versions are portfolio pieces you keep.

Q: I am a working professional. Can I manage this alongside a job?

A: The course is built for it: two live one-hour classes a week, scheduled in evening and weekend slots, plus three to four hours of practice you can place wherever your week allows. The pace is steady rather than punishing, and WhatsApp doubt support means a stuck Tuesday problem does not have to wait for Saturday. Professionals who miss a class can attend the same session with another batch where scheduling allows.

Q: Will this course make me a data analyst?

A: It gives you the statistical core of the job, which is genuinely the hard part, and a capstone analysis to show. It does not by itself supply everything analyst job listings ask for: SQL and dashboard tools are common requirements we do not cover here. Our honest advice, which we repeat in the career guidance sessions, is that this course plus SQL practice plus a portfolio of two or three analyses is a credible analyst preparation. The course alone is the foundation, not the whole house.

Q: Is this course aligned to any university syllabus?

A: It covers the standard core of a first university statistics course: descriptive statistics, probability, distributions, estimation, hypothesis testing and regression, so students frequently use it alongside degree coursework in commerce, economics, psychology and engineering. It is not tied to any single university's paper pattern, and we do not teach to one exam. Students preparing for a specific university exam get targeted practice on their syllabus topics in the doubt sessions.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/statistics-probability-maths-course>

=====

COURSE 112 OF 118: SWIFT PROGRAMMING MASTERCLASS: ZERO TO IOS & MACOS PRO

=====

Course link: <https://learn.modernagecoders.com/courses/complete-swift-programming-masterclass-college>

Subject area: Professional iOS/macOS Development

Designed for: College students

Who can join: Complete Beginner to iOS/macOS Professional

Course length: 12 months (52 weeks)

Time needed: 15-20 hours/week recommended

Certificate: Industry-recognized Swift/iOS Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

A 1-year Swift course: go from basics to professional iOS and macOS apps with SwiftUI, UIKit, Core Data, Combine, testing and App Store deployment.

In one line: From 'Hello World' to App Store Success

ABOUT THE PROGRAMME

This is not just a Swift course, it's a complete transformation into a professional Apple platform developer. Whether you're a curious beginner, student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled Swift developer capable of building stunning iOS apps, powerful macOS applications, watchOS experiences, and cross-platform solutions. You'll master Swift from ground zero to

professional architect level: from basic syntax to advanced protocols and generics, from simple views to complex SwiftUI animations, from local storage to cloud-synced Core Data, from single apps to multi-platform ecosystems. By the end, you'll have built 40+ projects, mastered the entire Apple development ecosystem, and be ready for senior iOS developer roles in top tech companies.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - perfect for complete beginners
- Separate learning tracks for kids (12+), teens, and adults
- 1 year of structured, industry-aligned learning
- Covers Swift + SwiftUI + UIKit + Core Data + Combine + CloudKit
- Real App Store projects and industry case studies
- Hands-on with latest Swift versions (Swift 5.9+)
- Interview preparation for top companies (Apple, Google, Meta)
- Lifetime access with continuous updates for new iOS versions
- Build production-ready App Store portfolio
- Direct path to high-paying iOS developer jobs

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Swift Fundamentals, OOP Basics, Core Swift Concepts, Playgrounds

Phase 2: Intermediate (Months 4-6): Advanced Swift, SwiftUI Deep Dive, UIKit Fundamentals, Data Persistence

Phase 3: Professional (Months 7-9): Networking, Core Data, Combine, Testing, Advanced UI/UX, Animations

Phase 4: Expert (Months 10-12): App Store Deployment, Performance, Architecture Patterns, Multi-Platform, Career Launch

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to all backgrounds

Coding experience: Absolute beginner friendly - zero programming knowledge required

Age: 12+ years (separate learning tracks for kids, teens, adults)

Equipment: Mac computer (MacBook or iMac) with macOS Ventura or later, minimum 8GB RAM (16GB recommended), iPhone/iPad for testing

Time commitment: 15-20 hours per week consistently for best results

English: Basic reading comprehension (materials also available in Hindi)

Motivation: Strong desire to become a professional iOS/macOS developer

Math: Basic mathematics (taught as needed)

WHO THIS COURSE IS FOR

Kids: Age 12-14: Simplified track with visual learning, game-based projects, Swift Playgrounds, fun app creation

Teens: Age 15-18: Student-focused track, app development skills, portfolio building, Swift Playgrounds to Xcode

Students: College students: Campus placement preparation, internship readiness, App Store portfolio

Working professionals: Career switchers: Structured path from any background to iOS developer role

Developers: Developers from other languages wanting to master Swift and iOS ecosystem

Android developers: Android developers wanting to add iOS development to their skillset

Web developers: Web developers transitioning to native mobile development

Designers: UI/UX designers who want to build the apps they design

Entrepreneurs: Build your own iOS apps and launch on the App Store

Freelancers: Offer iOS development services to high-paying clients

Anyone: Anyone passionate about Apple platforms and mobile development

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: FOUNDATION & CORE SWIFT SKILLS (MONTHS 1-3, WEEKS 1-13) -----

Build rock-solid Swift fundamentals. Learn programming logic, master Swift syntax, protocol-oriented programming, and create your first iOS applications.

Months 1-2: Swift Fundamentals & Programming Basics (Week 1-8)

Introduction to Swift & Development Environment Setup

Topics:

- What is Swift? History, features, and evolution from Objective-C
- Swift ecosystem: Xcode, Swift Package Manager, Playgrounds
- Apple platforms overview: iOS, macOS, watchOS, tvOS, visionOS
- Swift applications: Mobile apps, desktop apps, server-side Swift
- Installing Xcode from Mac App Store
- Setting up development environment on macOS

- Understanding Xcode interface, panels, and shortcuts
- Your first Swift program: Hello World in Playground
- Swift program structure: imports, main entry point
- Running Swift code in Playgrounds vs Apps
- Understanding LLVM compiler and Swift runtime
- Comments: single-line (//), multi-line (/* */), documentation (///)
- Swift coding conventions and API Design Guidelines
- Introduction to Apple Human Interface Guidelines (HIG)

Projects built:

- Hello World variations with personalized messages in Playground
- Simple output programs using print() function
- Personal information display program
- ASCII art generator in Swift Playground
- Interactive Playground with markdown documentation

Practice: Daily: 30 min Swift syntax practice, write 5-10 simple programs in Playgrounds

Variables, Data Types & Operators

Topics:

- Variables (var) and Constants (let): declaration and initialization
- Type inference vs explicit type annotation
- Basic data types: Int, Double, Float, Bool, String, Character
- Understanding Int8, Int16, Int32, Int64, UInt variants
- Type safety and type inference in Swift
- Type conversion and casting with as, as?, as!
- Optionals: declaring with ?, unwrapping with !, optional binding
- Nil-coalescing operator ??
- Optional chaining with ?.
- Implicitly unwrapped optionals
- Operators: arithmetic (+, -, *, /, %)
- Compound assignment operators: +=, -=, *=, /=
- Comparison operators: ==, !=, >, <, >=, <=
- Logical operators: &&, ||, !
- Range operators: closed range (...), half-open range (...<)
- Ternary conditional operator: ? :
- Nil-coalescing operator: ??
- String basics: creation, interpolation, multiline strings
- String manipulation: concatenation, substring, characters

Projects built:

- Simple calculator (four basic operations) in Playground
- Temperature converter (Celsius, Fahrenheit, Kelvin)
- Age calculator with optional handling
- BMI calculator with health categories
- Currency converter with type safety
- Area and perimeter calculator for shapes
- Compound interest calculator
- String formatter and validator

Practice: Solve 30 problems on variables, data types, optionals, and operators

Control Flow Statements

Topics:

- Boolean expressions and conditions
- If statement: single condition execution
- If-else statement: two-way branching
- If-else-if ladder: multiple conditions
- Nested if statements
- Switch statement: powerful pattern matching
- Switch with value binding and where clauses
- Switch with compound cases
- Switch with ranges and tuples
- No implicit fallthrough in Swift switch
- While loop: pre-tested loop
- Repeat-while loop: post-tested loop (equivalent to do-while)
- For-in loop: iterating over sequences
- For-in with ranges and stride
- For-in with enumerated() for index access
- Break statement: exit loop
- Continue statement: skip iteration
- Labeled statements for nested loops
- Guard statement: early exit pattern
- Defer statement: cleanup code

Projects built:

- Number guessing game with attempts limit
- Grade calculator with letter grades using switch
- Even/odd checker and list generator
- Prime number checker and generator
- Multiplication table generator
- Factorial calculator (iterative)
- Fibonacci series generator
- Pattern printing (stars, numbers, pyramids, diamonds)
- FizzBuzz challenge with switch

- Menu-driven calculator with guard statements
- Simple ATM simulation with validation

Practice: Solve 40 control flow and loop problems

Functions & Collections

Topics:

- Functions: definition, purpose, and structure
- Function declaration: parameters and return types
- Function calling and argument labels
- External and internal parameter names
- Default parameter values
- Variadic parameters (...)
- In-out parameters for mutating arguments
- Functions returning multiple values with tuples
- Nested functions
- Function types as parameters and return types
- Closures: inline anonymous functions
- Closure syntax and shorthand argument names (\$0, \$1)
- Trailing closure syntax
- Capturing values in closures
- Escaping closures (@escaping)
- Autoclosures (@autoclosure)
- Arrays: declaration, initialization, manipulation
- Array methods: append, insert, remove, contains, filter, map
- Sets: unique unordered collections
- Set operations: union, intersection, subtraction
- Dictionaries: key-value pairs
- Dictionary methods: updateValue, removeValue, keys, values
- Iterating over collections
- Higher-order functions: map, filter, reduce, flatMap, compactMap

Projects built:

- Function library for mathematical operations
- Array statistics calculator (sum, average, min, max)
- Linear search and binary search implementation
- Sorting algorithms: bubble sort, selection sort
- Contact list with dictionary
- Todo list with array operations
- Student grade management system
- Factorial using recursion
- Palindrome checker (string/number)
- Prime number generator in range
- Shopping cart with higher-order functions
- Word frequency counter with closures

Practice: Create 25 functions for various functionalities, solve 30 collection problems

Month 3: Object-Oriented & Protocol-Oriented Programming (Week 9-13)

Structures, Classes & Properties

Topics:

- Object-Oriented Programming paradigm in Swift
- Structures (struct): value types
- Classes (class): reference types
- Value types vs reference types comparison
- When to use struct vs class
- Properties: stored and computed properties
- Property observers: willSet and didSet
- Lazy stored properties
- Type properties (static)
- Instance methods
- Type methods (static and class)
- Mutating methods in structures
- Initializers: default, custom, memberwise
- Failable initializers (init?)
- Required initializers
- Deinitializers (deinit)
- Self keyword usage
- Access control: private, fileprivate, internal, public, open
- Encapsulation and data hiding
- Copy-on-write optimization

Projects built:

- Student struct with properties and methods
- Bank Account class with deposit/withdraw
- Book struct for library management
- Employee class with computed salary
- Rectangle/Circle structs with area/perimeter
- Car class with property observers
- Product struct for inventory system
- Person struct with multiple initializers
- Date struct implementation
- Timer class with proper memory management

Practice: Create 20 different structs and classes modeling real-world entities

Inheritance, Polymorphism & Memory Management

Topics:

- Inheritance: creating subclasses
- Base class and subclass relationship
- Overriding methods with 'override' keyword
- Overriding properties
- Preventing overrides with 'final'
- Calling superclass methods with 'super'
- Designated and convenience initializers
- Initializer inheritance and overriding
- Required initializers in inheritance
- Polymorphism in Swift
- Type casting: is, as, as?, as!
- Any and AnyObject types
- Automatic Reference Counting (ARC)
- Strong references and retain cycles
- Weak references (weak)
- Unowned references (unowned)
- Closure capture lists [weak self], [unowned self]
- Memory leaks detection and prevention
- Identity operators: === and !==

Projects built:

- Animal hierarchy (Animal -> Dog, Cat, Bird)
- Shape hierarchy with area calculation
- Vehicle system (Vehicle -> Car, Bike, Truck)
- Employee hierarchy (Employee -> Manager, Developer)
- Bank account types (Account -> Savings, Current)
- School management (Person -> Student, Teacher)
- Media player hierarchy (Media -> Audio, Video)
- Retain cycle demonstration and fix
- Memory-safe delegate implementation

Practice: Build 15 inheritance hierarchies, solve 25 ARC and memory problems

Protocols & Protocol-Oriented Programming

Topics:

- Protocols: defining blueprints
- Protocol syntax and requirements
- Property requirements in protocols
- Method requirements in protocols
- Mutating method requirements
- Initializer requirements in protocols
- Protocol adoption by structs, classes, enums
- Protocol inheritance
- Protocol composition with &
- Protocol extensions: default implementations
- Protocol-Oriented Programming (POP) philosophy
- POP vs OOP comparison
- Conditional conformance
- Associated types in protocols
- Self requirements in protocols
- Opaque types (some Protocol)
- Existential types (any Protocol)
- Common Swift protocols: Equatable, Hashable, Comparable, Codable
- Custom protocol creation
- Delegation pattern with protocols

Projects built:

- Shape calculator with protocol requirements
- Payment gateway with protocol abstraction
- Database abstraction with protocols
- Notification system (Email, SMS, Push protocols)
- Sortable collection with Comparable
- Custom Equatable and Hashable types
- Plugin architecture using protocols
- PHASE 1 MINI CAPSTONE: Library Management System (Console-based)
- Features: Books, members, issue/return, search, POP principles

Practice: Solve 30 protocol and POP problems

Assessment: Phase 1 Final Assessment - Core Swift & Protocol-Oriented fundamentals test

----- PHASE 2: INTERMEDIATE SWIFT & SWIFTUI MASTERY (MONTHS 4-6, WEEKS 14-26) -----

Master advanced Swift features, enumerations, generics, error handling, SwiftUI fundamentals, and basic iOS app development.

Months 4-5: Advanced Swift Features & SwiftUI Basics (Week 14-22)

Enumerations & Advanced Types

Topics:

- Enumerations (enum) in Swift
- Enum cases and raw values
- Enum with associated values

- Computed properties in enums
- Methods in enums
- Mutating methods in enums
- Recursive enumerations (indirect)
- CaseIterable protocol for enum iteration
- Pattern matching with enums in switch
- If-case and guard-case syntax
- Nested types: structs, classes, enums inside types
- Type aliases (typealias)
- Tuple types and destructuring
- Optional as enum (Optional<Wrapped>)
- Result type for error handling
- Never type for functions that don't return

Projects built:

- Compass direction enum with methods
- HTTP status code enum with raw values
- Network response enum with associated values
- Card game with suit and rank enums
- State machine with enum
- Currency converter with enum
- Error handling system with Result enum
- Menu system with nested enums
- Traffic light state machine
- JSON parsing result handling

Practice: Solve 35 enumeration and advanced type problems

Error Handling & Advanced Error Management

Topics:

- Error handling philosophy in Swift
- Error protocol and custom errors
- Creating custom error types with enum
- Throwing functions with 'throws' keyword
- Throwing initializers
- Do-catch blocks for error handling
- Multiple catch clauses
- Catching specific errors
- Converting errors to optionals with try?
- Forcing try with try! (use sparingly)
- Propagating errors with throws
- Rethrowing functions (rethrows)
- LocalizedError protocol for user messages
- CustomNSError for bridging to Objective-C
- Error handling best practices
- Cleanup with defer statements
- Assertions and preconditions
- fatalError for unimplemented code

Projects built:

- Robust calculator with error handling
- File reader with proper error handling
- Custom banking error system
- Network request error handling
- User validation with custom errors
- Age validator with localized errors
- Division calculator with error handling
- User registration with validation errors
- JSON parser with comprehensive error handling

Practice: Solve 25 error handling scenarios

Generics & Type System Deep Dive

Topics:

- Introduction to generics: type parameters
- Generic functions
- Generic types (struct, class, enum)
- Type constraints with protocol conformance
- Where clauses for complex constraints
- Associated types in protocols
- Generic subscripts
- Generic extensions
- Protocol extensions with constraints
- Type erasure patterns
- Opaque types (some Protocol) deep dive
- Primary associated types (Swift 5.7+)
- Existential types (any Protocol)
- Conditional conformance
- Specialized generic implementations
- Standard library generic types: Array, Dictionary, Optional, Result

Projects built:

- Generic stack and queue implementation
- Generic linked list
- Type-safe data repository
- Generic cache implementation

- Custom Result type with generics
- Generic sorting algorithms
- Generic tree data structure
- Type-safe networking layer

Practice: Create 15 generic types, solve 25 generics problems

SwiftUI Fundamentals - Part 1

Topics:

- Introduction to SwiftUI: declarative UI
- SwiftUI vs UIKit comparison
- View protocol and basic views
- Text view: styling, modifiers, formatting
- Image view: SF Symbols, asset images, resizing
- Button view: actions, styles, labels
- TextField and SecureField for input
- Toggle, Slider, Stepper, Picker
- VStack, HStack, ZStack for layout
- Spacer and Divider
- Frame modifier: fixed and flexible sizing
- Padding, background, overlay modifiers
- ForEach for dynamic content
- ScrollView for scrollable content
- List view for table-like layouts
- @State property wrapper for local state
- Two-way binding with \$
- View modifiers and modifier order

Projects built:

- Hello World SwiftUI app
- Profile card with image and text
- Simple counter app with @State
- Temperature converter UI
- BMI calculator with input fields
- Todo list with basic functionality
- Color picker interface
- Settings screen with toggles and pickers

Practice: Build 15 SwiftUI views with various controls

SwiftUI Fundamentals - Part 2

Topics:

- @Binding for child view communication
- @ObservableObject and @Published
- @StateObject vs @ObservedObject
- @EnvironmentObject for app-wide state
- ObservableObject protocol implementation
- NavigationStack (iOS 16+) navigation
- NavigationLink for navigation
- NavigationPath for programmatic navigation
- TabView for tab-based navigation
- Sheet, fullScreenCover for modal presentation
- Alert and confirmationDialog
- Custom view extraction
- ViewBuilder for custom containers
- Creating reusable components
- SF Symbols integration
- Dark mode and color schemes

Projects built:

- Multi-screen navigation app
- Tab-based application
- Settings app with observable object
- Shopping list with CRUD operations
- Note-taking app with navigation
- Modal presentation examples
- Reusable component library
- Theme-aware application

Practice: Build 10 multi-view SwiftUI applications

Month 6: Advanced SwiftUI & UIKit Fundamentals (Week 23-26)

SwiftUI Lists, Forms & Data Display

Topics:

- List view deep dive
- List styles: plain, grouped, insetGrouped, sidebar
- SwipeActions for list items
- Contextual menus (contextMenu)
- List with sections and headers
- Dynamic lists with Identifiable
- List editing: delete, move, selection
- Searchable modifier for filtering
- Pull to refresh (refreshable)
- Form view for settings and input
- Section in forms
- LabeledContent for forms (iOS 16+)

- Grid layouts: LazyVGrid, LazyHGrid
- GridItem and grid configuration
- Lazy stacks for performance
- Table view (macOS and iPadOS)
- OutlineGroup for hierarchical data

Projects built:

- Contact list with search and sections
- Settings app with forms
- Photo gallery with grid layout
- Task manager with swipe actions
- Editable list with add/delete/reorder
- Recipe list with contextual menus
- Product catalog with grid view
- Hierarchical file browser

Practice: Build 10 list and grid-based applications

SwiftUI Animations & Custom Drawing

Topics:

- Animation fundamentals in SwiftUI
- Implicit animations with .animation modifier
- Explicit animations with withAnimation
- Animation types: linear, easeIn, easeOut, spring
- Custom spring animations
- Animatable protocol and animatableData
- Transitions: opacity, scale, slide, move
- Combined and asymmetric transitions
- MatchedGeometryEffect for hero animations
- TimelineView for continuous animations
- Canvas for high-performance drawing
- Shapes: Rectangle, Circle, Ellipse, Capsule
- Path for custom shapes
- GeometryReader for dynamic layouts
- Custom shapes with Shape protocol
- Gradients: Linear, Radial, Angular
- Blend modes and visual effects

Projects built:

- Animated button effects
- Loading spinner animations
- Card flip animation
- Hero transition between views
- Custom progress indicator
- Animated chart visualization
- Drawing app with Canvas
- Custom animated splash screen
- Particle effect animation
- Morphing shape animations

Practice: Create 15 animations and custom shapes

UIKit Fundamentals

Topics:

- UIKit overview and architecture
- UIViewController lifecycle
- Storyboards vs programmatic UI
- UIView and view hierarchy
- Auto Layout: constraints and anchors
- Programmatic Auto Layout with NSLayoutConstraint
- UICollectionViewController for simplified layouts
- UILabel, UIButton, UIImageView
- UITextField and UITextView
- UITableView: cells, sections, data source
- UITableView delegate methods
- UICollectionView fundamentals
- UINavigationController navigation
- UITabBarController tabs
- Presenting view controllers modally
- UIAlertController for alerts and action sheets
- Integrating UIKit in SwiftUI (UIViewRepresentable)
- Integrating SwiftUI in UIKit (UIHostingController)

Projects built:

- UIKit todo list app
- Table view with custom cells
- Collection view photo gallery
- Navigation-based master-detail app
- Tab bar application
- Form with text fields and validation
- UIKit + SwiftUI hybrid app
- Custom table view cells

Practice: Build 12 UIKit-based views and controllers

Data Persistence - UserDefaults & File System

Topics:

- Data persistence options in iOS

- UserDefaults for simple preferences
- Storing primitives in UserDefaults
- Storing custom objects with Codable
- @AppStorage property wrapper
- FileManager for file operations
- iOS file system: Documents, Caches, tmp
- Reading and writing files
- JSON encoding and decoding
- Codable protocol: Encodable, Decodable
- CodingKeys for custom mapping
- Custom encoding/decoding logic
- PropertyListEncoder and Decoder
- Keychain for secure storage
- Keychain wrapper libraries
- iCloud key-value storage
- Data migration strategies

Projects built:

- User preferences with UserDefaults
- Settings app with @AppStorage
- Note-taking app with file storage
- JSON data persistence
- Secure login credentials with Keychain
- Configuration file manager
- Data export/import utility
- User profile with local storage

Practice: Implement persistence in all previous projects

Phase 2 Capstone Project

Topics:

- SwiftUI implementation
- Advanced Swift patterns
- Protocol-oriented design
- Data persistence
- Navigation and state management
- Custom animations

Projects built:

- PHASE 2 CAPSTONE: Personal Finance Tracker
- Features: Income/expense tracking, categories, charts, budget alerts, data persistence, SwiftUI with animations
- Alternative: Recipe Book App with favorites and shopping lists
- Alternative: Habit Tracker with statistics and reminders
- Alternative: Travel Journal with photos and maps

Assessment: Phase 2 Final Exam - Intermediate Swift and SwiftUI comprehensive test

PHASE 2 CONTINUED - Advanced Patterns & Core Data (Week 14-26 (distributed))

Swift Concurrency - Async/Await

Topics:

- Concurrency in Swift: history and evolution
- Grand Central Dispatch (GCD) basics
- DispatchQueue: main, global, custom
- Async/await syntax (Swift 5.5+)
- Async functions and methods
- Calling async functions with await
- Task for spawning async work
- Task groups for parallel execution
- Async sequences (AsyncSequence)
- Actors for data race protection
- @MainActor for UI updates
- Global actors and custom actors
- Sendable protocol for thread safety
- Structured concurrency
- Task cancellation and priorities
- Migrating from GCD to async/await

Projects built:

- Async data fetching example
- Parallel image downloader
- Actor-based counter
- Async file operations
- Concurrent API calls
- Background task processing
- Thread-safe data manager
- Async search implementation

Practice: Convert 10 projects to async/await

Core Data Fundamentals

Topics:

- Core Data overview and architecture
- Core Data stack: NSPersistentContainer
- NSManagedObjectContext: data model
- NSManagedObjectContext: scratch pad
- NSPersistentStoreCoordinator

- Creating data model in Xcode
- Entities, attributes, and relationships
- NSManagedObject subclasses
- Creating and saving objects
- Fetching objects with NSFetchedResultsController
- Predicates for filtering (NSPredicate)
- Sorting with NSSortDescriptor
- Updating and deleting objects
- Core Data with SwiftUI: @FetchRequest
- @Environment(\.managedObjectContext)
- FetchedResults for displaying data
- Relationships: one-to-one, one-to-many, many-to-many

Projects built:

- Todo app with Core Data
- Contact manager with Core Data
- Expense tracker with categories
- Note-taking app with Core Data
- Book library with authors relationship
- Recipe app with ingredients relationship
- Core Data with SwiftUI integration
- Search and filter implementation

Practice: Build 8 Core Data applications

Advanced Core Data

Topics:

- Core Data migrations: lightweight and manual
- Versioning data models
- Batch insert, update, and delete
- Performance optimization techniques
- Faulting and prefetching
- NSFetchedResultsController for UIKit
- Core Data with background contexts
- Concurrency with Core Data
- Core Data debugging and profiling
- Core Data with CloudKit
- NSPersistentCloudKitContainer
- Syncing data across devices
- Conflict resolution strategies
- Core Data testing strategies
- Best practices and common pitfalls

Projects built:

- CloudKit-synced todo app
- Multi-device note syncing
- Batch operations performance test
- Background import from JSON
- Core Data migration exercise
- Unit tests for Core Data
- Performance-optimized list app

Practice: Implement CloudKit sync in 3 projects

Combine Framework Fundamentals

Topics:

- Reactive programming concepts
- Publishers and Subscribers
- Built-in publishers: Just, Future, Empty, Fail
- @Published property wrapper
- Subjects: PassthroughSubject, CurrentValueSubject
- Operators: map, filter, flatMap, merge
- Combining publishers: combineLatest, zip, merge
- Error handling in Combine
- Scheduling and threads
- Cancellables and memory management
- Combine with SwiftUI
- Combine with URLSession
- Debounce and throttle for search
- Combine vs async/await comparison
- When to use Combine

Projects built:

- Form validation with Combine
- Search with debounce
- Real-time data binding
- API response handling with Combine
- Multi-field form validation
- Live search implementation
- Timer-based updates

Practice: Implement Combine in 8 projects

Swift Package Manager & Project Organization

Topics:

- Swift Package Manager (SPM) overview
- Creating Swift packages
- Package.swift manifest file

- Adding dependencies to projects
- Popular Swift packages and libraries
- Creating reusable libraries
- Local packages for modularization
- Publishing packages
- Semantic versioning
- Version control with Git (review/deep dive)
- Git branching strategies
- GitHub collaboration for iOS projects
- .gitignore for Xcode projects
- Code organization best practices
- MVVM architecture introduction

Projects built:

- Create custom Swift package
- Multi-module app architecture
- Networking library package
- UI component library package
- Git repository for all projects
- Modular app structure

Practice: Convert all projects to modular structure

PHASE 3: PROFESSIONAL IOS DEVELOPMENT (MONTHS 7-9, WEEKS 27-39)

Master professional iOS development with networking, advanced architecture, testing, accessibility, and real-world app development patterns.

Months 7-8: Networking & Architecture (Week 27-35)

Networking with URLSession

Topics:

- Networking fundamentals: HTTP protocol
- HTTP methods: GET, POST, PUT, DELETE, PATCH
- HTTP headers and status codes
- JSON format and structure
- URLSession overview and configuration
- URLSessionDataTask for data requests
- URLSessionDownloadTask for downloads
- URLSessionUploadTask for uploads
- URLRequest configuration
- Async/await with URLSession
- Error handling in networking
- Parsing JSON with Codable
- Handling nested JSON
- Custom CodingKeys and date handling
- Network error types and handling
- URL components and query parameters

Projects built:

- Weather app consuming REST API
- Movie database browser (TMdb API)
- News reader with API integration
- GitHub user search
- Currency exchange rate app
- Photo download and caching
- REST API client library
- API response error handling

Practice: Build 10 network-connected applications

Advanced Networking & Authentication

Topics:

- Building a generic networking layer
- Protocol-based API client
- Request/Response interceptors
- Authentication: API keys, tokens
- OAuth 2.0 flow implementation
- JWT token handling
- Refresh token management
- Keychain for token storage
- SSL pinning for security
- Background downloads and uploads
- Network reachability (NWPathMonitor)
- Caching strategies: URLCache
- Image caching solutions
- Request retry logic
- Rate limiting handling
- GraphQL basics with Swift

Projects built:

- Complete authenticated API client
- Social media login (OAuth)
- Image caching library
- Offline-first data sync

- Background upload manager
- Network layer with unit tests
- GraphQL client implementation
- API wrapper for multiple endpoints

Practice: Build production-ready networking layer

App Architecture - MVVM

Topics:

- Software architecture importance
- MVC: Model-View-Controller (Apple's default)
- Problems with Massive View Controller
- MVVM: Model-View-ViewModel
- ViewModel responsibilities
- Data binding with Combine
- Data binding with @Observable (iOS 17+)
- Dependency Injection principles
- Protocol-based dependencies
- Coordinator pattern for navigation
- Repository pattern for data access
- Service layer architecture
- Clean Architecture overview
- VIPER architecture basics
- Choosing the right architecture
- Testing with architecture patterns

Projects built:

- Refactor app to MVVM
- MVVM with Combine bindings
- Coordinator-based navigation
- Repository pattern implementation
- Dependency injection container
- Clean architecture example
- Testable architecture demo
- Multi-module MVVM app

Practice: Apply MVVM to all major projects

Advanced Architecture & The Composable Architecture

Topics:

- State management challenges
- Unidirectional data flow
- The Composable Architecture (TCA) introduction
- Reducers and State
- Actions and Effects
- Store and ViewStore
- Scoping and composition
- Testing in TCA
- Alternatives: ReSwift, Redux-like patterns
- Feature modules with TCA
- Navigation in TCA
- Dependency injection in TCA
- Performance optimization
- When to use TCA vs MVVM
- Architecture decision factors

Projects built:

- Counter app with TCA
- Todo list with TCA
- Navigation with TCA
- Feature composition example
- TCA with network effects
- Full app with TCA architecture
- Testing TCA reducers

Practice: Build 5 TCA-based features

Unit Testing & Test-Driven Development

Topics:

- Testing importance and types
- XCTest framework overview
- Writing test cases
- XCTAssert functions
- Test lifecycle: setUp, tearDown
- Testing view models
- Testing with protocols and mocks
- Dependency injection for testing
- Async testing with expectations
- Testing async/await code
- Testing Combine publishers
- Test doubles: mocks, stubs, fakes, spies
- Code coverage with Xcode
- Test-Driven Development (TDD) methodology
- Testing best practices
- Continuous testing workflow

Projects built:

- Unit tests for calculator

- ViewModel testing suite
- Mock network layer tests
- TDD implementation of stack
- Core Data unit tests
- Async code testing
- Test coverage improvement
- Complete test suite for capstone

Practice: Write tests for all major projects, achieve 70%+ coverage

Month 9: UI Testing, Accessibility & Advanced Features (Week 36-39)

UI Testing & Snapshot Testing

Topics:

- UI testing with XCUITest
- XCUIApplication for app launching
- XCUIElement for element identification
- Accessibility identifiers for testing
- Tapping, typing, swiping gestures
- Assertions in UI tests
- Waiting for elements
- Testing navigation flows
- Testing lists and tables
- UI test recording
- Snapshot testing introduction
- swift-snapshot-testing library
- Testing different device sizes
- Testing dark mode
- Testing accessibility features
- Page Object pattern for UI tests

Projects built:

- UI test suite for login flow
- Navigation UI tests
- List interaction tests
- Snapshot tests for components
- Multi-device snapshot tests
- Dark mode snapshot tests
- Accessibility audit tests
- End-to-end UI test suite

Practice: Add UI tests to all major projects

Accessibility & Localization

Topics:

- Accessibility importance and guidelines
- VoiceOver support
- Accessibility labels and hints
- Accessibility traits
- Accessibility value for dynamic content
- SwiftUI accessibility modifiers
- UIKit accessibility properties
- Dynamic Type support
- Color contrast requirements
- Reduce Motion support
- Accessibility testing with Xcode
- Localization fundamentals
- String localization: Localizable.strings
- String catalogs (Xcode 15+)
- Pluralization rules
- Date and number formatting
- Right-to-left (RTL) language support
- Testing localization

Projects built:

- Accessibility audit and fixes
- VoiceOver-optimized app
- Dynamic Type implementation
- Localized app in 3+ languages
- RTL language support
- Accessible custom controls
- Localization testing automation

Practice: Make all projects fully accessible

Maps, Location & Background Tasks

Topics:

- MapKit framework overview
- Displaying maps with Map (SwiftUI)
- MKMapView in UIKit
- Map annotations and overlays
- Custom annotation views
- Route calculation and directions
- Core Location framework
- Requesting location permissions
- Getting user location
- Significant location changes

- Geofencing with `CLLocationCircularRegion`
- Geocoding and reverse geocoding
- Background location updates
- Background tasks (`BGTaskScheduler`)
- Background app refresh
- Background processing tasks

Projects built:

- Map with user location
- Points of interest app
- Route planning app
- Location-based reminders (geofencing)
- Store finder with directions
- Background location tracker
- Background sync implementation
- Fitness tracker with background location

Practice: Build 8 location-aware applications

Push Notifications & App Extensions

Topics:

- Push notifications overview
- Local notifications with `UNUserNotificationCenter`
- Notification triggers: time, calendar, location
- Notification content and categories
- Custom notification actions
- Remote push notifications (APNs)
- Setting up APNs certificates
- Firebase Cloud Messaging (FCM) alternative
- Handling notifications in app
- Rich notifications with attachments
- App extensions overview
- Today widgets / Widget extension
- Share extension
- Action extension
- Notification content extension
- App groups for data sharing

Projects built:

- Local notification reminder app
- Scheduled notification system
- Remote push notification setup
- Rich notification with images
- Widget for app data
- Share extension for app
- Interactive notification actions
- Multi-widget app

Practice: Add notifications and widgets to 5 projects

Camera, Photos & Media

Topics:

- Camera access with `UIImagePickerController`
- `PHPickerViewController` for photo selection
- Camera with `AVCaptureSession`
- Custom camera UI
- Photo library access (`PhotosUI`)
- Saving photos to library
- Photo editing with Core Image
- `CIFilter` for image processing
- `AVFoundation` for video
- `AVPlayer` for media playback
- `AVAudioPlayer` for audio
- Recording audio with `AVAudioRecorder`
- Video capture and editing
- Media file formats and compression
- Metal basics for graphics

Projects built:

- Photo capture app
- Photo gallery with picker
- Image filter app
- Video player app
- Voice memo recorder
- Custom camera with filters
- Photo editor with Core Image
- Media-rich content app

Practice: Build 8 media-focused applications

PHASE 3 COMPLETION - Month 9 Final Week (Week 27-39 (distributed))

App Performance & Optimization

Topics:

- Performance measurement importance
- Instruments tool overview
- Time Profiler for CPU analysis
- Allocations for memory analysis

- Leaks instrument for memory leaks
- Network profiling
- Core Animation instrument
- App launch time optimization
- Memory management best practices
- Image optimization and caching
- Table/collection view performance
- Lazy loading patterns
- Background thread optimization
- Energy and battery impact
- MetricKit for production monitoring

Projects built:

- Performance profiling report
- Memory leak identification and fix
- App launch optimization
- Image caching implementation
- List performance optimization
- Battery efficiency improvements
- Production metrics setup

Practice: Profile and optimize all major projects

In-App Purchases & Subscriptions

Topics:

- App monetization strategies
- StoreKit framework overview
- StoreKit 2 (modern API)
- Product types: consumable, non-consumable, subscription
- Configuring products in App Store Connect
- Fetching products
- Processing purchases
- Transaction verification
- Receipt validation
- Subscription management
- Subscription offers and trials
- Restoring purchases
- Server-side validation
- RevenueCat SDK integration
- Testing in-app purchases

Projects built:

- Consumable IAP implementation
- Premium feature unlock
- Subscription-based app
- Restore purchases feature
- Receipt validation implementation
- RevenueCat integration
- Premium app conversion

Practice: Add monetization to 3 projects

Security & Data Protection

Topics:

- iOS security architecture
- App Transport Security (ATS)
- Secure data storage best practices
- Keychain deep dive
- Keychain access groups
- Data encryption with CryptoKit
- Hashing and digital signatures
- Biometric authentication (Face ID, Touch ID)
- LocalAuthentication framework
- Secure text entry
- Jailbreak detection
- SSL certificate pinning
- Code obfuscation basics
- Privacy manifest (iOS 17+)
- Privacy nutrition labels

Projects built:

- Biometric login implementation
- Encrypted data storage
- Secure notes app
- Certificate pinning implementation
- Privacy-focused app
- Keychain wrapper library
- Security audit checklist

Practice: Security audit all projects

Design Patterns & Best Practices

Topics:

- Creational patterns: Singleton, Factory, Builder
- Structural patterns: Adapter, Decorator, Facade
- Behavioral patterns: Observer, Strategy, Command
- Delegation pattern (Swift/iOS native)
- Protocol-oriented patterns

- Coordinator pattern deep dive
- Repository pattern
- Clean code principles
- SOLID principles in Swift
- Code review guidelines
- Swift style guide
- Documentation with DocC
- Technical debt management
- Refactoring techniques

Projects built:

- Pattern implementation examples
- Coordinator-based app navigation
- Plugin architecture with protocols
- Code refactoring exercise
- DocC documentation
- Design pattern showcase

Practice: Apply patterns to all projects

Phase 3 Capstone Project

Topics:

- Complete iOS application
- Architecture implementation
- Networking and persistence
- Testing and quality
- Accessibility and localization
- Performance optimization

Projects built:

- MAJOR CAPSTONE: Social Photo Sharing App
- Features: User auth, photo capture/upload, feed, likes/comments, notifications, profile, search, Core Data + CloudKit sync, MVVM architecture, full test coverage
- Alternative: Fitness Tracking App (workouts, progress, HealthKit, charts)
- Alternative: E-commerce App (products, cart, checkout, orders, payments)
- Alternative: Messaging App (real-time chat, media sharing, contacts)

Assessment: Phase 3 Final Exam - Professional iOS development comprehensive test

----- PHASE 4: EXPERT DEVELOPMENT, MULTI-PLATFORM & CAREER EXCELLENCE (MONTHS 10-12, WEEKS 40-52) -----

Master advanced topics, multi-platform development, App Store publishing, CI/CD, and career preparation.

Months 10-11: Multi-Platform & Advanced Topics (Week 40-48)

SwiftUI Advanced Topics

Topics:

- Custom view modifiers
- PreferenceKey for child-to-parent communication
- EnvironmentKey for custom environments
- ViewBuilder and resultBuilder
- Property wrapper deep dive
- Creating custom property wrappers
- Layout protocol (iOS 16+)
- Custom layouts implementation
- AnyLayout for dynamic layout switching
- Observation framework (iOS 17+)
- @Observable macro
- Migration from ObservableObject
- Performance optimization in SwiftUI
- Lazy loading and equatable
- SwiftUI debugging techniques

Projects built:

- Custom modifier library
- PreferenceKey implementations
- Custom property wrappers
- Advanced layout system
- Observable migration project
- SwiftUI performance optimization
- Debug tooling for SwiftUI

Practice: Build 8 advanced SwiftUI features

WidgetKit & App Intents

Topics:

- WidgetKit fundamentals
- Widget timeline and entries
- Widget configuration types
- Interactive widgets (iOS 17+)
- Widget bundles
- Widget sizes and families
- Lock screen widgets
- Widget intent configuration
- App Intents framework
- Shortcuts integration

- Siri integration with App Intents
- Focus filters
- Live Activities
- Dynamic Island (iPhone 14 Pro+)
- ActivityKit for real-time updates

Projects built:

- Static widget implementation
- Configurable widget
- Interactive widget (iOS 17)
- Lock screen widget
- Siri shortcuts integration
- Live Activity for sports/delivery
- Dynamic Island experience
- Complete widget suite

Practice: Add widgets to 5 projects

macOS Development with SwiftUI

Topics:

- macOS app fundamentals
- Differences from iOS development
- macOS-specific views and controls
- Window management
- Menu bar apps
- Settings/Preferences windows
- Document-based apps
- NSOpenPanel and NSSavePanel
- Toolbar and sidebar navigation
- Table view on macOS
- Drag and drop support
- macOS-specific APIs
- Mac Catalyst for iPad apps
- App Sandbox and capabilities
- Notarization and distribution

Projects built:

- macOS utility app
- Menu bar application
- Document-based text editor
- macOS settings app
- iPad app with Mac Catalyst
- Cross-platform SwiftUI app
- macOS productivity tool

Practice: Build 6 macOS applications

watchOS & tvOS Development

Topics:

- watchOS app architecture
- Watch-only vs companion apps
- WatchKit basics
- SwiftUI on watchOS
- watchOS-specific complications
- HealthKit integration
- Workout session handling
- Watch connectivity
- Background refresh on watchOS
- tvOS app fundamentals
- Focus-based navigation
- tvOS-specific UI patterns
- Siri Remote input handling
- Top Shelf extension
- Multi-platform app targets

Projects built:

- Standalone watchOS app
- Watch complication
- Fitness watch app with HealthKit
- Watch + iPhone companion app
- tvOS media app
- tvOS game with remote
- Multi-platform unified app

Practice: Build apps for 3 Apple platforms

visionOS & Spatial Computing

Topics:

- visionOS introduction
- Spatial computing concepts
- SwiftUI for visionOS
- Windows, volumes, and spaces
- RealityKit fundamentals
- 3D content with RealityKit
- Immersive experiences
- Hand tracking and gestures
- Eye tracking concepts
- ARKit on visionOS

- SharePlay in spatial computing
- Entity Component System
- Reality Composer Pro
- visionOS design guidelines
- Future of spatial computing

Projects built:

- Basic visionOS window app
- 3D content viewer
- Immersive environment
- Hand gesture interactions
- AR experience on visionOS
- Spatial data visualization
- Mixed reality application

Practice: Build 5 visionOS experiences

Month 12: App Store, CI/CD & Career Excellence (Week 49-52)

App Store Submission & Distribution

Topics:

- Apple Developer Program enrollment
- App Store Connect setup
- App information and metadata
- Screenshots and app preview videos
- App Store product page optimization
- Pricing and availability
- In-app purchases configuration
- App Store Review Guidelines
- Common rejection reasons
- TestFlight for beta testing
- Internal and external testing
- Build distribution to testers
- Phased release strategies
- App Store Optimization (ASO)
- Custom product pages (iOS 15+)
- Product page optimization tests

Projects built:

- Complete App Store submission
- TestFlight beta distribution
- App Store screenshot creation
- App preview video creation
- ASO optimization exercise
- A/B testing product pages
- Full release workflow

Practice: Submit 3 apps to App Store

CI/CD & Automation

Topics:

- Continuous Integration concepts
- Xcode Cloud setup and usage
- Xcode Cloud workflows
- Automated testing in CI
- Code signing for CI
- Fastlane overview
- Fastlane lanes and actions
- Automated screenshots with Fastlane
- Automated deployment with Fastlane
- GitHub Actions for iOS
- Bitrise for mobile CI/CD
- Code signing best practices
- Environment management
- Build versioning automation
- Automated release notes

Projects built:

- Xcode Cloud workflow setup
- Automated test pipeline
- Fastlane deployment automation
- GitHub Actions workflow
- Automated screenshot generation
- Complete CI/CD pipeline
- Release automation workflow

Practice: Set up CI/CD for all major projects

Analytics, Crash Reporting & Monitoring

Topics:

- App analytics importance
- App Analytics in App Store Connect
- Firebase Analytics integration
- Custom event tracking
- User engagement metrics
- Conversion tracking
- A/B testing with Firebase
- Crash reporting with Firebase Crashlytics

- Symbolication of crash logs
- Xcode Organizer crash reports
- MetricKit for performance data
- Custom metrics logging
- User feedback collection
- In-app review prompts
- Privacy-focused analytics

Projects built:

- Analytics integration
- Custom event tracking system
- Crash reporting setup
- Performance monitoring dashboard
- A/B testing implementation
- User feedback system
- Complete analytics suite

Practice: Add analytics to all production apps

Server-Side Swift & Backend Integration

Topics:

- Server-side Swift overview
- Vapor framework introduction
- Setting up Vapor project
- Routing in Vapor
- Models and Fluent ORM
- Database integration (PostgreSQL)
- RESTful API development
- Authentication and authorization
- Middleware in Vapor
- Deploying Vapor apps
- Firebase as backend
- Firebase Authentication
- Cloud Firestore database
- Firebase Cloud Functions
- AWS Amplify for iOS

Projects built:

- Basic Vapor API
- Full-stack Swift application
- Firebase-backed iOS app
- Real-time data with Firestore
- User authentication backend
- Cloud function integration
- Complete backend solution

Practice: Build 5 backend-connected applications

SwiftData & Modern Persistence (iOS 17+)

Topics:

- SwiftData introduction
- SwiftData vs Core Data
- @Model macro for persistence
- Schema definition with Swift
- ModelContainer and ModelContext
- @Query for fetching data
- Relationships in SwiftData
- Migration strategies
- SwiftData with SwiftUI
- Performance considerations
- CloudKit integration
- Choosing SwiftData vs Core Data
- SwiftData testing
- Best practices and patterns

Projects built:

- SwiftData basic app
- Migration from Core Data
- SwiftData relationships
- CloudKit sync with SwiftData
- SwiftData + SwiftUI app
- Performance-optimized queries

Practice: Implement SwiftData in 5 projects

PHASE 4 COMPLETION - Career Preparation (Week 40-52 (distributed))

Advanced Topics & Specializations

Topics:

- Machine Learning with Core ML
- Create ML for training models
- Vision framework for image analysis
- Natural Language framework
- Speech recognition
- ARKit for augmented reality
- RealityKit for 3D experiences
- Game development with SpriteKit
- SceneKit for 3D graphics

- Metal for graphics programming
- HealthKit integration
- HomeKit for smart home
- CarPlay development
- MusicKit integration

Projects built:

- ML-powered image classifier
- Text sentiment analyzer
- AR object placement app
- Simple game with SpriteKit
- Health tracking app
- Smart home control app
- Specialization project in chosen area

Practice: Explore 3 advanced specializations

Code Quality & Professional Practices

Topics:

- Swift style guide compliance
- SwiftLint for code quality
- SwiftFormat for formatting
- Code review best practices
- Pull request guidelines
- Technical documentation
- DocC documentation creation
- README writing
- Architecture decision records
- Technical debt management
- Refactoring strategies
- Performance benchmarking
- Debugging advanced techniques
- Memory debugging
- Thread sanitizer

Projects built:

- SwiftLint integration
- Code quality automation
- DocC documentation site
- Refactoring exercise
- Performance benchmark suite
- Quality metrics dashboard

Practice: Apply quality practices to all projects

Open Source & Community

Topics:

- Open source in iOS community
- Contributing to Swift itself
- Popular iOS open source projects
- Finding beginner-friendly projects
- Understanding contribution guidelines
- Forking and pull requests
- Code review in open source
- Creating open source libraries
- Publishing to Swift Package Index
- Documentation for open source
- Community engagement
- iOS conferences and meetups
- Swift forums participation
- Building personal brand

Projects built:

- Contribute to 5 open source projects
- Create Swift package library
- Publish on Swift Package Index
- Documentation contributions
- Bug fixes and features
- Community engagement

Practice: Active open source participation

Interview Preparation

Topics:

- iOS interview preparation strategy
- Swift language interview questions
- UIKit interview topics
- SwiftUI interview questions
- Memory management questions
- Concurrency and threading
- Architecture pattern questions
- System design for mobile
- Data structures and algorithms
- LeetCode in Swift
- Take-home project tips
- Behavioral interview questions
- STAR method for answers
- Salary negotiation tactics

- Resume optimization for iOS developers

Projects built:

- Solve 150+ LeetCode problems in Swift
- System design case studies
- Mock interview practice
- Interview preparation guide
- Portfolio website creation

Practice: Daily coding challenges and mock interviews

Final Month - Capstone & Career Launch (Week 49-52)

Final Capstone Project - Part 1

Topics:

- Project ideation and planning
- Requirement analysis
- App architecture design
- UI/UX design with Figma
- Database design
- API design and integration
- Technology stack selection
- Development workflow setup
- Sprint planning
- Agile methodology

Projects built:

- FINAL CAPSTONE: Production-Ready Multi-Platform App
- Option 1: Health & Fitness Ecosystem
- Platforms: iOS, watchOS, widgets, complications, HealthKit, CloudKit sync
- Option 2: Social Networking Platform
- Features: Feed, messaging, profiles, media sharing, notifications, analytics
- Option 3: Productivity Suite
- Apps: Task manager, notes, calendar, widgets, Mac Catalyst, sync
- Option 4: E-commerce Mobile App
- Features: Products, cart, checkout, payments, orders, push notifications

Final Capstone Project - Part 2

Topics:

- Implementation completion
- Comprehensive testing
- Performance optimization
- Accessibility compliance
- Localization implementation
- App Store submission preparation
- CI/CD pipeline setup
- Analytics integration
- Crash reporting setup
- Project presentation preparation

Deliverables:

- Complete source code on GitHub
- App published on App Store
- Multi-platform support (iOS, widget, watch/mac)
- Complete documentation with DocC
- Architecture diagrams and documentation
- Design files (Figma)
- CI/CD with Xcode Cloud or Fastlane
- Test coverage report (80%+ coverage)
- Performance report from Instruments
- Video demo and presentation
- Comprehensive README and wiki

Career Launch & Professional Development

Topics:

- Professional portfolio website
- Resume optimization for iOS roles
- LinkedIn profile enhancement
- GitHub profile showcase
- Technical blog writing
- Speaking at iOS meetups
- Networking strategies
- Job application process
- Interview follow-up
- Continuous learning plan
- Staying updated with WWDC
- Mentoring and teaching others
- Freelancing strategies
- Consulting career path
- Indie app development

Deliverables:

- Professional portfolio with 40+ projects
- Optimized resume for iOS developer roles
- Enhanced LinkedIn with App Store apps
- GitHub profile with 100+ contributions
- 5-10 technical blog posts

- Personal brand establishment
- App Store presence with published apps
- Mock interview completion
- Continuous learning roadmap
- Professional network connections

Assessment: FINAL COMPREHENSIVE CERTIFICATION EXAM - Complete Swift/iOS mastery evaluation

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-3): 20+ foundational projects - playgrounds, console apps, OOP/POP systems
- Phase 2 (Months 4-6): 18+ intermediate projects - SwiftUI apps, data persistence, animations
- Phase 3 (Months 7-9): 20+ professional projects - networked apps, tested apps, production features
- Phase 4 (Months 10-12): 15+ advanced projects - multi-platform, App Store apps, full ecosystem
- Final: 5 major capstone projects demonstrating complete expertise

Total projects: 70+ projects from beginner to App Store-ready production applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Swift: Syntax, Optionals, Collections, Closures, OOP, POP, Generics, Error Handling, Concurrency
- Advanced Swift: Protocols, Extensions, Property Wrappers, Result Builders, Macros
- SwiftUI: Views, State Management, Navigation, Animations, Custom Layouts, Observation
- UIKit: View Controllers, Auto Layout, Table/Collection Views, Navigation, Storyboards
- Data Persistence: UserDefaults, FileManager, Core Data, SwiftData, CloudKit, Keychain
- Networking: URLSession, Async/Await, REST APIs, JSON Parsing, Authentication
- Combine: Publishers, Subscribers, Operators, Reactive Programming
- Testing: XCTest, UI Testing, Snapshot Testing, TDD, Code Coverage
- Architecture: MVC, MVVM, Coordinator, TCA, Clean Architecture, Dependency Injection
- Multi-Platform: iOS, macOS, watchOS, tvOS, visionOS, Widgets, Extensions
- DevOps: Xcode Cloud, Fastlane, GitHub Actions, CI/CD, App Store Connect
- Advanced: ARKit, Core ML, HealthKit, MapKit, Push Notifications, In-App Purchases
- Tools: Xcode, Instruments, Swift Package Manager, SwiftLint, DocC, Figma
- Soft Skills: Problem-solving, Architecture Thinking, Code Review, Documentation, Agile

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 8-10 hours
 Projects: 3-5 hours
 Practice problems: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding sessions and doubt clearing
 Mentorship: 1-on-1 expert iOS developer mentorship
 Community: Active Discord/Slack community with 24/7 peer support
 Code review: Expert code reviews for all major projects
 Career support: Resume review, mock interviews, job referrals to product companies
 Lifetime access: All content, future updates, new iOS/Swift versions coverage
 Interview prep: Mock interviews with industry professionals
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion (4 certificates)
 Final certificate: Professional iOS Developer Certification
 SwiftUI certificate: SwiftUI Specialist Certificate
 Multiplatform certificate: Apple Multi-Platform Developer Certificate
 LinkedIn badge: Digital badges for LinkedIn profile
 Industry recognized: Recognized by Fortune 500 companies and startups
 Portfolio projects: 40+ production-ready portfolio projects
 App store apps: Published apps on the App Store

WHERE THIS CAN LEAD

- iOS Developer (Junior to Senior levels)
- Swift Developer
- Mobile Application Developer
- SwiftUI Developer
- macOS Developer
- watchOS Developer
- visionOS Developer
- Apple Platforms Engineer
- Mobile Architect

- Technical Lead / Team Lead
- Solutions Architect
- Principal Mobile Engineer
- Freelance iOS Consultant
- Technical Trainer / Instructor
- Indie App Developer / Startup Founder

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-6 LPA (Junior iOS Developer / Fresher)

Growing roles: Rs 6-12 LPA (iOS Developer); Rs 10-18 LPA (Senior iOS Developer)

Experienced roles: Rs 15-30 LPA (Lead iOS Developer / Mobile Architect); Rs 22-50 LPA (Principal Engineer / Solutions Architect)

Freelance: Rs 2,000-7,000/hour based on expertise

International usa: \$95k-210k USD (iOS Developer to Principal)

International europe: €55k-140k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need a Mac for this course?

A: Realistically yes: Xcode, Apple's development environment, runs only on macOS, so a Mac is required to follow the course properly, ideally with 8GB RAM or more. If you do not have one yet, message us on WhatsApp at +91 91233 66161 before enrolling and we will discuss your options honestly.

Q: Which Swift version and frameworks are covered?

A: Current Swift (5.10 and later) with SwiftUI as the primary UI framework, plus UIKit, Core Data, Combine and modern concurrency, all inside current Xcode. The material tracks Apple's current toolchain, so you learn what iOS teams actually ship with.

Q: What apps will I build during the year?

A: A progression of real iOS apps: fundamentals projects in Swift playgrounds early, then SwiftUI apps with data persistence, networked apps using REST APIs, Core Data backed applications, and a polished capstone taken through App Store preparation in the final phase. Each one goes into your GitHub portfolio.

Q: Can this lead to an iOS developer job?

A: iOS development is a specialized skill area with consistent demand, and the course ends with architecture patterns, testing and interview preparation for exactly those roles. Your portfolio of working apps is the entry ticket. We are upfront that no course can guarantee placement, and we do not.

Q: Why take live classes instead of following Apple's free tutorials?

A: Apple's tutorials are polished but assume you never get stuck, and they cannot review an app you design yourself. A live mentor debugs with you inside Xcode, reviews your SwiftUI state management and architecture, and holds the 12-month arc together so you finish apps instead of abandoning them at the first hard bug.

Q: How do I start?

A: With a free live demo class: book it at learn.modernagecoders.com/contact, attend with no commitment, and see the teaching style before any payment. Bring your questions about Mac setup and course fit; the demo exists for exactly that.

Q: Are sessions live or pre-recorded?

A: Every session is a live 1-hour online class with an instructor; we never substitute recordings. Classes are interactive with real coding in Xcode during the hour, and evening and weekend slots accommodate college schedules and day jobs.

Q: What do I receive when I finish?

A: A completion certificate plus a GitHub-ready portfolio of iOS apps, and if you complete the final phase, experience preparing an app for App Store submission. For iOS hiring, shipped or shippable apps carry the most weight, and the course is built to leave you with them.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-swift-programming-masterclass-college>

Course link: <https://learn.modernagecoders.com/courses/complete-typescript-programming-masterclass-college>
Subject area: Professional TypeScript Development
Designed for: College students
Who can join: Complete Beginner to Enterprise Professional
Course length: 12 months (52 weeks)
Time needed: 15-20 hours/week recommended
Certificate: Industry-recognized TypeScript Developer certification upon completion

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online TypeScript course for college students and professionals: React, Node.js, Next.js, GraphQL and cloud deployment, from basics to enterprise apps.

In one line: From 'Hello World' to Full-Stack Enterprise Applications

ABOUT THE PROGRAMME

This is not just a TypeScript course, it's a complete transformation into an enterprise-ready TypeScript professional. Whether you're a curious beginner, student, working professional, or someone with zero coding experience, this 1-year masterclass will turn you into a highly skilled TypeScript developer capable of building scalable full-stack applications, REST APIs, GraphQL services, and cloud-native solutions. You'll master TypeScript from ground zero to enterprise architect level: from basic syntax to advanced type systems, from simple scripts to React/Next.js applications, from Node.js servers to microservices architecture. By the end, you'll have built 40+ projects, mastered the entire TypeScript ecosystem, and be ready for senior full-stack developer roles in top tech companies.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute zero - perfect for complete beginners
- Separate learning tracks for kids (12+), teens, and adults
- 1 year of structured, industry-aligned learning
- Covers TypeScript + React + Node.js + Next.js + GraphQL
- Real enterprise projects and industry case studies
- Hands-on with latest TypeScript versions (TypeScript 5+)
- Interview preparation for top companies
- Lifetime access with continuous updates
- Build production-ready full-stack portfolio
- Direct path to high-paying TypeScript developer jobs

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): JavaScript Basics, TypeScript Fundamentals, Core Concepts
Phase 2: Intermediate (Months 4-6): Advanced Types, React with TypeScript, Frontend Mastery
Phase 3: Professional (Months 7-9): Node.js, Express, Databases, REST APIs, Backend Engineering
Phase 4: Enterprise (Months 10-12): Next.js, GraphQL, Microservices, DevOps, Cloud Deployment

WHAT A STUDENT NEEDS BEFORE STARTING

Education: No formal degree required - open to all backgrounds
Coding experience: Absolute beginner friendly - zero programming knowledge required
Age: 12+ years (separate learning tracks for kids, teens, adults)
Equipment: Computer/laptop (Windows/Mac/Linux), minimum 8GB RAM recommended, internet connection
Time commitment: 15-20 hours per week consistently for best results
English: Basic reading comprehension (materials also available in Hindi)
Motivation: Strong desire to become a professional TypeScript developer
Math: Basic mathematics (taught as needed)

WHO THIS COURSE IS FOR

Kids: Age 12-14: Simplified track with visual learning, game-based projects, interactive JavaScript basics
Teens: Age 15-18: Student-focused track, college prep, competitive programming, project-based learning
Students: College students: Campus placement preparation, internship readiness, strong portfolio building
Working professionals: Career switchers: Structured path from any background to full-stack developer role
Developers: Developers from other languages wanting to master TypeScript ecosystem
Frontend developers: Frontend devs wanting to become full-stack with Node.js and Next.js
Backend developers: Backend devs wanting to add React and TypeScript to their skills
Entrepreneurs: Build your own SaaS products using modern TypeScript technologies
Freelancers: Offer full-stack development services to high-paying clients

Anyone: Anyone passionate about modern web development and full-stack engineering

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & TYPESCRIPT FUNDAMENTALS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid JavaScript and TypeScript fundamentals. Learn programming logic, master TypeScript syntax, type systems, and create your first applications.

Months 1-2: JavaScript & TypeScript Fundamentals (Week 1-8)

Introduction to Programming & JavaScript Basics

Topics:

- What is programming? Introduction to software development
- JavaScript ecosystem: browsers, Node.js, runtime environments
- Why TypeScript? Benefits over plain JavaScript
- Installing Node.js and npm package manager
- Setting up development environment: VS Code with TypeScript extensions
- Understanding IDE features: IntelliSense, debugging, extensions
- Your first JavaScript program: Hello World
- JavaScript program structure: statements, expressions
- Running JavaScript in browser console and Node.js
- Comments: single-line (`//`), multi-line (`/* */`), documentation
- JavaScript coding conventions and naming standards
- Console methods: `log`, `error`, `warn`, `table`
- Introduction to ECMAScript versions (ES6+)

Projects built:

- Hello World variations with personalized messages
- Simple output programs using `console.log`
- Personal information display program
- ASCII art generator in JavaScript
- Interactive greeting program

Practice: Daily: 30 min JavaScript syntax practice, write 5-10 simple programs

Variables, Data Types & Operators

Topics:

- Variables: `var`, `let`, `const` - differences and best practices
- Primitive data types: `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`
- Type coercion: implicit and explicit conversion
- `typeof` operator for type checking
- Template literals and string interpolation
- Numbers: integers, floats, `Infinity`, `NaN`
- Operators: arithmetic (`+`, `-`, `*`, `/`, `%`, `**`, `++`, `--`)
- Relational operators: `==`, `===`, `!=`, `!==`, `>`, `<`, `>=`, `<=`
- Logical operators: `&&`, `||`, `!`, nullish coalescing (`??`)
- Assignment operators: `=`, `+=`, `-=`, `*=`, `/=`, `%=`
- Bitwise operators: `&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`
- Ternary operator: `? :`
- Operator precedence and associativity
- Optional chaining operator (`?.`) and its uses

Projects built:

- Simple calculator (four basic operations)
- Temperature converter (Celsius, Fahrenheit, Kelvin)
- Age calculator with detailed output
- BMI calculator with health categories
- Currency converter with exchange rates
- Area and perimeter calculator for shapes
- Compound interest calculator

Practice: Solve 30 problems on variables, data types, and operators

Control Flow & Functions

Topics:

- Boolean expressions and `truthy/falsy` values
- If statement: single condition execution
- If-else statement: two-way branching
- If-else-if ladder: multiple conditions
- Nested if statements
- Switch statement: multi-way branching
- Switch with fall-through and `break`
- While loop: pre-tested loop
- Do-while loop: post-tested loop
- For loop: counter-based iteration
- For-of loop: iterating arrays/strings
- For-in loop: iterating object properties
- Break and continue statements
- Labeled statements for nested loops
- Function declarations and expressions
- Arrow functions: concise syntax
- Parameters, arguments, default parameters

- Rest parameters and spread operator
- Return statement and return values
- Function scope and hoisting

Projects built:

- Number guessing game with attempts limit
- Grade calculator with letter grades
- Even/odd checker and list generator
- Prime number checker and generator
- Multiplication table generator
- Factorial calculator (iterative and recursive)
- Fibonacci series generator
- Pattern printing (stars, numbers, pyramids)
- FizzBuzz challenge
- Menu-driven calculator
- Simple ATM simulation

Practice: Solve 40 control flow and function problems

Arrays, Objects & ES6+ Features

Topics:

- Arrays: declaration, creation, initialization
- Array indexing and accessing elements
- Array methods: push, pop, shift, unshift, splice
- Array iteration: forEach, map, filter, reduce
- Array methods: find, findIndex, some, every, includes
- Array sorting and reversing
- Spread operator with arrays
- Destructuring arrays
- Objects: creation and property access
- Object methods and 'this' keyword
- Object shorthand properties and methods
- Computed property names
- Object destructuring
- Spread operator with objects
- Object.keys(), Object.values(), Object.entries()
- JSON: parse and stringify
- Classes: declaration, constructor, methods
- Class inheritance: extends and super
- Static methods and properties
- Getters and setters

Projects built:

- Array statistics calculator (sum, average, min, max)
- Todo list with array operations
- Linear search and binary search implementation
- Sorting algorithms: bubble sort, selection sort
- Student grade management system
- Contact book with objects
- Shopping cart implementation
- Product inventory system
- Simple class-based game (Rock Paper Scissors)

Practice: Create 25 programs using arrays and objects, solve 30 problems

Month 3: TypeScript Core & Type System (Week 9-13)

TypeScript Fundamentals & Setup

Topics:

- Introduction to TypeScript: static typing benefits
- TypeScript vs JavaScript: key differences
- Installing TypeScript globally and locally
- tsconfig.json configuration file
- Compiler options: target, module, strict mode
- Basic types: string, number, boolean
- Type annotations and type inference
- Arrays with type annotations
- Tuple types for fixed-length arrays
- Enum types: numeric and string enums
- Any type: when and when not to use
- Unknown type: safe alternative to any
- Void, null, and undefined types
- Never type for unreachable code
- Type assertions: as and angle-bracket syntax
- Literal types: string, number, boolean literals
- Union types: combining multiple types
- Type narrowing and type guards

Projects built:

- Convert JavaScript projects to TypeScript
- Type-safe calculator application
- Enum-based status system
- Student management with typed objects
- Configuration manager with TypeScript
- Simple CLI tool with TypeScript

Practice: Convert 10 JavaScript programs to TypeScript with proper types

Interfaces, Type Aliases & Advanced Types

Topics:

- Interface declaration and usage
- Optional properties with ?
- Readonly properties
- Index signatures for dynamic properties
- Interface extending other interfaces
- Implementing interfaces in classes
- Type aliases with 'type' keyword
- Interface vs type alias: when to use which
- Intersection types: combining types with &
- Discriminated unions for type safety
- Type guards: typeof, instanceof, in operator
- User-defined type guards with 'is' keyword
- Nullable types and strict null checks
- Optional chaining with types
- Nullish coalescing with types
- Function types and signatures
- Call signatures in interfaces
- Function overloading in TypeScript

Projects built:

- Type-safe user management system
- Product catalog with interfaces
- API response types modeling
- Form validation with types
- State management with discriminated unions
- Plugin system with interface contracts
- Event system with typed events

Practice: Build 15 projects using interfaces and advanced types

Generics & Advanced TypeScript

Topics:

- Introduction to generics: type parameters
- Generic functions and type inference
- Generic interfaces
- Generic classes
- Generic constraints with extends
- Multiple type parameters
- Default type parameters
- Generic utility types: Partial, Required, Readonly
- Pick, Omit, Record utility types
- Extract, Exclude, NonNullable types
- ReturnType and Parameters utility types
- Conditional types with extends and infer
- Mapped types for transforming types
- Template literal types
- Keyof operator for type keys
- typeof operator in type context
- Module systems: ES modules in TypeScript
- Declaration files (.d.ts) basics

Projects built:

- Generic data structure library (Stack, Queue, LinkedList)
- Type-safe API client with generics
- Generic form handler
- State container with generic types
- Generic validation library
- PHASE 1 MINI CAPSTONE: CLI Task Manager
- Features: Tasks, categories, priorities, file persistence, full TypeScript typing

Practice: Create 15 generic utilities, solve 30 generics problems

Assessment: Phase 1 Final Assessment - TypeScript fundamentals and type system test

PHASE 2: FRONTEND MASTERY WITH REACT & TYPESCRIPT (MONTHS 4-6, WEEKS 14-26)

Master modern frontend development with React, state management, testing, and build production-ready user interfaces.

Months 4-5: React Fundamentals & Advanced Patterns (Week 14-22)

HTML, CSS & DOM Fundamentals

Topics:

- HTML5 semantic elements and structure
- Forms, inputs, and validation attributes
- CSS fundamentals: selectors, specificity, cascade
- CSS Box Model and layout basics
- Flexbox for one-dimensional layouts
- CSS Grid for two-dimensional layouts
- Responsive design with media queries
- CSS custom properties (variables)
- CSS animations and transitions
- DOM manipulation: selecting elements

- DOM events: click, submit, input, keyboard
- Event delegation and bubbling
- Creating and modifying DOM elements
- Local Storage and Session Storage
- Fetch API for HTTP requests

Projects built:

- Responsive portfolio website
- Interactive form with validation
- Todo app with DOM manipulation
- Image gallery with lightbox
- Weather app using Fetch API
- Quiz application
- Calculator with UI
- Responsive navigation menu

Practice: Build 10 interactive web pages with DOM manipulation

React Fundamentals & JSX

Topics:

- Introduction to React: component-based architecture
- TypeScript with Vite
- Vite for React TypeScript projects
- JSX syntax and expressions
- Components: functional components
- Props: passing data to components
- TypeScript with React: typing props
- Children prop and composition
- Conditional rendering techniques
- Lists and keys in React
- Event handling in React
- TypeScript event types
- Styling: CSS modules, styled-components
- Tailwind CSS with React
- Component organization and file structure

Projects built:

- Hello World React app with TypeScript
- Component library with props
- Card component with typed props
- Product listing page
- Navigation component
- Footer component
- Button component variants
- Simple landing page

Practice: Build 15 reusable React components with TypeScript

React Hooks & State Management

Topics:

- useState hook for local state
- Typing useState with TypeScript
- useEffect hook for side effects
- Cleanup functions in useEffect
- useRef hook for DOM references
- useMemo for expensive calculations
- useCallback for memoizing functions
- useReducer for complex state logic
- Typing useReducer actions and state
- Custom hooks creation
- Rules of hooks
- useContext for prop drilling solution
- Creating typed context providers
- React.memo for component memoization
- Performance optimization strategies

Projects built:

- Counter app with useState
- Todo list with useReducer
- Form handling with hooks
- Data fetching with useEffect
- Theme switcher with useContext
- Custom useFetch hook
- Custom useLocalStorage hook
- Debounced search with useMemo

Practice: Create 10 custom hooks with TypeScript

Advanced React Patterns

Topics:

- Compound components pattern
- Render props pattern
- Higher-Order Components (HOCs)
- Custom hooks vs HOCs vs render props
- Controlled vs uncontrolled components
- Form libraries: React Hook Form
- Typing forms with TypeScript
- Form validation with Zod

- Error boundaries for error handling
- Suspense for data fetching
- React.lazy for code splitting
- Portal for modals and tooltips
- Forwarding refs with forwardRef
- useImperativeHandle hook
- Accessibility (a11y) in React

Projects built:

- Modal component with portals
- Dropdown menu with compound components
- Form with React Hook Form and Zod
- Error boundary wrapper
- Lazy loaded routes
- Accessible form components
- Tooltip component
- Tabs component with compound pattern

Practice: Implement all advanced patterns in projects

React Router & Navigation

Topics:

- React Router v6 fundamentals
- BrowserRouter and Route components
- Link and NavLink for navigation
- Route parameters and useParams
- Query strings and useSearchParams
- Nested routes and Outlet
- Protected routes implementation
- useNavigate for programmatic navigation
- useLocation hook
- Route guards and authentication
- 404 and error pages
- Code splitting with routes
- Scroll restoration
- Route transitions and animations

Projects built:

- Multi-page application with routing
- Dashboard with nested routes
- Protected admin routes
- Blog with dynamic routes
- E-commerce product pages
- User profile with tabs

Practice: Build 5 multi-page applications with React Router

Month 6: State Management & Testing (Week 23-26)

Global State Management

Topics:

- State management overview and options
- Redux Toolkit fundamentals
- Creating slices with createSlice
- Redux store configuration
- useSelector and useDispatch hooks
- TypeScript with Redux Toolkit
- Async logic with createAsyncThunk
- RTK Query for data fetching
- Zustand: lightweight alternative
- Zustand with TypeScript
- Jotai: atomic state management
- React Query/TanStack Query
- Caching and background refetching
- Optimistic updates
- Comparing state management solutions

Projects built:

- Todo app with Redux Toolkit
- Shopping cart with Redux
- User authentication state
- Data fetching with RTK Query
- Blog with TanStack Query
- Real-time dashboard with Zustand
- Complex form with state management

Practice: Implement state management in all previous projects

Testing React Applications

Topics:

- Testing pyramid: unit, integration, e2e
- Jest testing framework setup
- React Testing Library fundamentals
- Rendering components for testing
- Querying elements: getBy, findBy, queryBy
- User event simulation
- Testing forms and inputs
- Testing async operations

- Mocking API calls with MSW
- Testing hooks with renderHook
- Testing context providers
- Snapshot testing
- Code coverage reports
- Cypress for E2E testing
- Playwright introduction

Projects built:

- Test suite for component library
- Form testing with user events
- API mocking with MSW
- E2E tests with Cypress
- Integration tests for features
- Accessibility testing

Practice: Write tests for all React components, aim for 80%+ coverage

Build Tools & Performance

Topics:

- Webpack fundamentals and configuration
- Vite: modern build tool
- Environment variables management
- Bundle analysis and optimization
- Tree shaking for dead code elimination
- Code splitting strategies
- Lazy loading components and routes
- Image optimization
- React DevTools profiler
- Performance measurement with Web Vitals
- React.memo and useMemo optimization
- Virtualization for long lists
- Service workers basics
- PWA fundamentals
- Lighthouse audits

Projects built:

- Optimized production build
- Bundle size optimization
- Performance-optimized dashboard
- Virtualized list component
- PWA conversion of existing app
- Image gallery with lazy loading

Practice: Optimize all projects for production performance

UI Libraries & Design Systems

Topics:

- Component libraries overview
- Material-UI (MUI) with TypeScript
- Chakra UI fundamentals
- Ant Design components
- Headless UI for unstyled components
- Radix UI primitives
- Shadcn/ui component system
- Theme customization
- Design tokens and theming
- Dark mode implementation
- Responsive design patterns
- Animation libraries: Framer Motion
- Building design systems
- Storybook for component documentation
- Component documentation best practices

Projects built:

- Dashboard with MUI components
- Landing page with Chakra UI
- Custom design system
- Storybook documentation
- Animated page transitions
- Dark mode toggle implementation

Practice: Build 5 projects with different UI libraries

Phase 2 Capstone Project

Topics:

- Full frontend application architecture
- Component design and organization
- State management implementation
- Routing and navigation
- Form handling and validation
- API integration
- Testing strategy
- Performance optimization

Projects built:

- PHASE 2 CAPSTONE: E-commerce Frontend
- Features: Product catalog, cart, wishlist, user auth UI, checkout flow, responsive design, dark mode, comprehensive testing

- Alternative: Social Media Dashboard (posts, feeds, profiles, messaging UI)
 - Alternative: Project Management Tool (boards, tasks, drag-drop, real-time updates)
 - Alternative: Learning Platform Frontend (courses, progress, quizzes, video player)
- Assessment: Phase 2 Final Exam - React and Frontend comprehensive test

PHASE 3: BACKEND ENGINEERING WITH NODE.JS & TYPESCRIPT (MONTHS 7-9, WEEKS 27-39)

Master server-side development with Node.js, Express, databases, REST APIs, authentication, and production deployment.

Months 7-8: Node.js & Express Fundamentals (Week 27-35)

Node.js Fundamentals

Topics:

- Node.js architecture: event loop, non-blocking I/O
- Node.js vs browser JavaScript
- npm and package.json management
- CommonJS vs ES modules in Node
- TypeScript setup for Node.js
- Built-in modules: fs, path, os, url
- File system operations: read, write, delete
- Streams and buffers
- Event emitter pattern
- Process and environment variables
- Child processes and workers
- Nodemon for development
- ts-node for TypeScript execution
- Debugging Node.js applications
- Error handling in Node.js

Projects built:

- File system CLI tool
- Directory watcher application
- Log file parser
- Simple HTTP server from scratch
- Stream-based file processor
- Environment config manager
- Task runner CLI

Practice: Build 10 Node.js utility scripts

Express.js Framework

Topics:

- Introduction to Express.js
- Express with TypeScript setup
- Routing: GET, POST, PUT, DELETE, PATCH
- Route parameters and query strings
- Request and response objects
- Middleware concept and implementation
- Built-in middleware: json, urlencoded, static
- Custom middleware creation
- Error handling middleware
- Router for modular routes
- Request validation with express-validator
- File uploads with Multer
- CORS configuration
- Helmet for security headers
- Rate limiting with express-rate-limit

Projects built:

- Basic REST API server
- User CRUD API
- File upload service
- API with validation
- Middleware chain implementation
- Logging middleware
- Request timing middleware

Practice: Build 8 Express APIs with proper middleware

REST API Design & Best Practices

Topics:

- REST architecture principles
- Resource naming conventions
- HTTP status codes usage
- Request/response design
- API versioning strategies
- Pagination implementation
- Filtering and sorting
- HATEOAS principles
- API documentation with Swagger/OpenAPI
- TypeScript types for API contracts
- Zod for runtime validation
- Error response standardization
- API testing with Postman/Insomnia

- API testing with supertest
- Contract-first API design

Projects built:

- RESTful Blog API
- E-commerce Product API
- API with Swagger documentation
- Paginated list API
- Filter and search API
- API test suite with supertest

Practice: Design and document 5 production-ready APIs

Database Fundamentals - SQL

Topics:

- Relational database concepts
- SQL fundamentals: SELECT, INSERT, UPDATE, DELETE
- WHERE, ORDER BY, LIMIT, OFFSET
- JOINS: INNER, LEFT, RIGHT, FULL
- Aggregations: COUNT, SUM, AVG, GROUP BY
- Database design and normalization
- Primary keys and foreign keys
- Indexes for performance
- PostgreSQL setup and usage
- Node.js with pg driver
- Connection pooling
- Parameterized queries for security
- Transactions in Node.js
- Database migrations concept
- SQL injection prevention

Projects built:

- User management with PostgreSQL
- Blog with posts and comments
- E-commerce products and orders
- Library management system
- Transaction-based banking API
- Database migration scripts

Practice: Build 8 database-connected applications

ORMs - Prisma & Drizzle

Topics:

- ORM concepts and benefits
- Prisma setup and configuration
- Prisma schema definition
- Prisma Client for queries
- CRUD operations with Prisma
- Relations: one-to-one, one-to-many, many-to-many
- Prisma migrations
- Prisma with TypeScript
- Drizzle ORM introduction
- Drizzle schema and queries
- Type-safe queries with Drizzle
- Raw SQL with ORMs
- Seeding databases
- ORM performance considerations
- Choosing between ORMs

Projects built:

- Blog API with Prisma
- User authentication with Prisma
- E-commerce backend with Prisma
- Multi-tenant application
- Complex relations example
- Database seeding scripts

Practice: Convert all SQL projects to use Prisma

Month 9: Authentication, NoSQL & Advanced Backend (Week 36-39)

Authentication & Authorization

Topics:

- Authentication vs authorization
- Password hashing with bcrypt
- JWT (JSON Web Tokens) fundamentals
- Access tokens and refresh tokens
- JWT implementation in Express
- Passport.js authentication strategies
- Session-based authentication
- Cookie security: httpOnly, secure, sameSite
- Role-based access control (RBAC)
- Permission-based authorization
- OAuth 2.0 and OpenID Connect
- Social login: Google, GitHub, Facebook
- Two-factor authentication (2FA)
- Password reset flow
- Security best practices

Projects built:

- User registration and login API
- JWT authentication system
- Refresh token rotation
- Role-based API access
- OAuth social login
- Password reset with email
- 2FA implementation

Practice: Secure all APIs with authentication

NoSQL Databases - MongoDB

Topics:

- NoSQL database concepts
- MongoDB architecture and documents
- MongoDB Atlas cloud setup
- Mongoose ODM for Node.js
- Mongoose schemas and models
- TypeScript with Mongoose
- CRUD operations with Mongoose
- Mongoose middleware (hooks)
- Population for references
- Aggregation pipeline
- Indexing in MongoDB
- Schema validation
- Embedded vs referenced documents
- MongoDB transactions
- SQL vs NoSQL: when to use which

Projects built:

- Blog with MongoDB and Mongoose
- Social media posts API
- Real-time chat data model
- Analytics data storage
- Product catalog with nested data
- User activity tracking

Practice: Build 6 MongoDB-backed applications

Caching, Sessions & Real-time

Topics:

- Caching strategies overview
- Redis: in-memory data store
- Redis with Node.js (ioredis)
- Caching API responses
- Session storage with Redis
- Rate limiting with Redis
- Pub/Sub with Redis
- WebSocket fundamentals
- Socket.io for real-time communication
- TypeScript with Socket.io
- Real-time events and rooms
- Broadcasting messages
- Server-Sent Events (SSE)
- Long polling vs WebSocket
- Real-time architecture patterns

Projects built:

- Cached API responses
- Session management with Redis
- Real-time chat application
- Live notifications system
- Collaborative document editing
- Real-time dashboard updates
- Live auction system

Practice: Add real-time features to existing projects

Email, File Storage & External Services

Topics:

- Email sending with Nodemailer
- Email templates with EJS/Handlebars
- Transactional email services: SendGrid, Resend
- Queue-based email sending
- File storage strategies
- AWS S3 for object storage
- Cloudinary for image management
- File upload to cloud storage
- Image processing with Sharp
- PDF generation in Node.js
- Payment integration: Stripe
- Webhook handling
- Third-party API integration
- API rate limiting and retries
- Error handling for external services

Projects built:

- Email notification service

- Welcome email flow
- Password reset emails
- File upload to S3
- Image resize and optimization
- Payment checkout flow
- Invoice PDF generation

Practice: Integrate 5 external services

Testing Backend Applications

Topics:

- Backend testing strategies
- Unit testing with Jest
- Testing Express routes
- Supertest for HTTP testing
- Mocking database calls
- Integration testing setup
- Test database management
- Factory patterns for test data
- Testing authentication flows
- Testing async operations
- Code coverage for backend
- CI/CD testing pipeline
- Load testing with Artillery
- Performance testing basics
- End-to-end API testing

Projects built:

- Unit test suite for services
- Integration tests for APIs
- Authentication flow tests
- Database operation tests
- Load testing scripts
- CI/CD with automated tests

Practice: Write comprehensive tests for all APIs

PHASE 3 COMPLETION - Advanced Backend (Week 27-39 (distributed))

Background Jobs & Message Queues

Topics:

- Background job processing need
- Bull/BullMQ for job queues
- Redis-based job queues
- Job scheduling and recurring jobs
- Job retries and error handling
- Job priorities and delays
- Message queue concepts
- RabbitMQ fundamentals
- Producer-consumer pattern
- Event-driven architecture
- Dead letter queues
- Queue monitoring and dashboards
- Scaling with workers
- Idempotency in job processing
- Distributed task processing

Projects built:

- Email queue processor
- Image processing queue
- Report generation jobs
- Scheduled data cleanup
- Order processing pipeline
- Notification queue system

Practice: Add background processing to applications

Logging, Monitoring & Observability

Topics:

- Logging best practices
- Winston logger configuration
- Pino for high-performance logging
- Log levels and formatting
- Structured logging (JSON)
- Request logging and tracing
- Application metrics with Prometheus
- Custom metrics collection
- Health check endpoints
- Error tracking with Sentry
- APM (Application Performance Monitoring)
- Distributed tracing basics
- OpenTelemetry introduction
- Monitoring dashboards
- Alerting strategies

Projects built:

- Centralized logging setup
- Metrics endpoint implementation

- Health check system
- Error tracking integration
- Performance monitoring
- Alerting rules configuration

Practice: Add observability to all backend services

Security & API Protection

Topics:

- OWASP Top 10 for Node.js
- SQL/NoSQL injection prevention
- XSS prevention strategies
- CSRF protection
- Security headers with Helmet
- Input sanitization
- Rate limiting implementation
- API key management
- Secrets management
- Environment security
- Dependency vulnerability scanning
- npm audit and fixes
- HTTPS and TLS
- Content Security Policy
- Secure file uploads

Projects built:

- Security-hardened API
- Rate limiting system
- API key authentication
- Vulnerability scanning setup
- Security audit checklist
- Penetration testing basics

Practice: Security audit all backend applications

Docker & Deployment

Topics:

- Docker fundamentals
- Dockerfile for Node.js apps
- Multi-stage builds
- Docker Compose for development
- Container orchestration basics
- Environment configuration
- Deploying to VPS (DigitalOcean, Linode)
- Process managers: PM2
- Nginx reverse proxy
- SSL certificates with Let's Encrypt
- CI/CD with GitHub Actions
- Deployment to Railway, Render
- Zero-downtime deployments
- Database backups
- Production checklist

Projects built:

- Dockerized Node.js app
- Docker Compose setup
- CI/CD pipeline creation
- VPS deployment
- Nginx configuration
- Automated deployments

Practice: Deploy all applications to production

Phase 3 Capstone Project

Topics:

- Full backend architecture
- Database design
- API development
- Authentication implementation
- File handling
- Real-time features
- Testing and documentation
- Deployment and monitoring

Projects built:

- MAJOR CAPSTONE: E-commerce Backend API
- Features: User auth (JWT), products, categories, cart, orders, payments (Stripe), email notifications, file uploads, admin panel API, comprehensive testing
- Alternative: Social Media API (users, posts, comments, likes, follows, notifications, real-time)
- Alternative: Project Management API (workspaces, projects, tasks, comments, file attachments)
- Alternative: Learning Platform Backend (courses, lessons, enrollments, progress tracking, certificates)

Assessment: Phase 3 Final Exam - Backend Engineering comprehensive test

Master Next.js, GraphQL, microservices architecture, cloud-native development, DevOps, system design, and career preparation.

Months 10-11: Next.js, GraphQL & Advanced Full-Stack (Week 40-48)

Next.js Fundamentals

Topics:

- Next.js introduction and benefits
- Next.js 14+ App Router architecture
- File-based routing system
- Layouts and nested layouts
- Loading and error states
- Server Components vs Client Components
- Data fetching in Server Components
- Static Site Generation (SSG)
- Server-Side Rendering (SSR)
- Incremental Static Regeneration (ISR)
- API Routes and Route Handlers
- Middleware for request handling
- Image optimization with next/image
- Font optimization with next/font
- Environment variables in Next.js

Projects built:

- Next.js blog with SSG
- E-commerce with SSR
- Dashboard with App Router
- Portfolio with ISR
- API routes implementation
- Middleware authentication

Practice: Build 6 Next.js applications

Advanced Next.js & Server Actions

Topics:

- Server Actions for mutations
- Form handling with Server Actions
- Revalidation strategies
- Caching in Next.js
- Parallel and sequential data fetching
- Streaming with Suspense
- Route groups for organization
- Intercepting and parallel routes
- Internationalization (i18n)
- Authentication with NextAuth.js
- Session management
- Protected routes and middleware
- SEO optimization in Next.js
- Metadata and OpenGraph
- Sitemaps and robots.txt

Projects built:

- Full CRUD with Server Actions
- Multi-language application
- Auth system with NextAuth
- SEO-optimized blog
- Admin dashboard with auth
- Real-time features in Next.js

Practice: Build 5 production-ready Next.js apps

GraphQL Fundamentals

Topics:

- GraphQL introduction and benefits
- GraphQL vs REST comparison
- Schema definition language (SDL)
- Types: Query, Mutation, Subscription
- Scalar and custom types
- Input types and enums
- Resolvers implementation
- Apollo Server setup
- TypeScript with GraphQL
- Code generation with GraphQL Codegen
- Apollo Client for React
- Queries and mutations in client
- Caching in Apollo Client
- Error handling in GraphQL
- DataLoader for optimized fetching

Projects built:

- GraphQL API for blog
- Full-stack app with Apollo
- Type-safe GraphQL with Codegen
- Real-time subscriptions
- GraphQL with Prisma
- Complex query patterns

Practice: Build 5 GraphQL applications

Advanced GraphQL & tRPC

Topics:

- GraphQL subscriptions setup
- Real-time updates with WebSocket
- Pagination: cursor and offset
- Federation for microservices
- Schema stitching
- GraphQL security best practices
- Query complexity and depth limiting
- tRPC introduction
- End-to-end type safety with tRPC
- tRPC with Next.js
- tRPC procedures and routers
- tRPC with Zod validation
- tRPC vs GraphQL comparison
- Choosing the right approach
- Full-stack TypeScript patterns

Projects built:

- Real-time chat with subscriptions
- Federated GraphQL setup
- tRPC full-stack app
- Type-safe API layer
- Combined approach example
- Production-ready API patterns

Practice: Implement both GraphQL and tRPC solutions

Microservices Architecture

Topics:

- Microservices vs monolith
- When to use microservices
- Service decomposition strategies
- Domain-Driven Design basics
- API Gateway pattern
- Service discovery
- Inter-service communication
- Synchronous vs asynchronous
- Event-driven architecture
- Message brokers: RabbitMQ, Kafka
- Saga pattern for transactions
- Circuit breaker pattern
- Service mesh basics
- Container orchestration
- Kubernetes introduction

Projects built:

- Decompose monolith exercise
- API Gateway implementation
- Event-driven microservices
- Service communication patterns
- Saga implementation
- Kubernetes deployment

Practice: Design 3 microservices architectures

Month 12: Cloud, DevOps & Career Excellence (Week 49-52)

Cloud Platforms & Serverless

Topics:

- Cloud computing fundamentals
- AWS core services overview
- Vercel for Next.js deployments
- Serverless architecture
- AWS Lambda with TypeScript
- API Gateway with Lambda
- Serverless Framework
- Database services: RDS, DynamoDB
- S3 for file storage
- CloudFront CDN
- Azure and GCP alternatives
- Infrastructure as Code
- Terraform basics
- Pulumi with TypeScript
- Cost optimization strategies

Projects built:

- Deploy to Vercel/Netlify
- Serverless API with Lambda
- Full-stack on AWS
- CDN configuration
- IaC with Terraform/Pulumi
- Multi-cloud strategy

Practice: Deploy applications to multiple platforms

CI/CD & DevOps Practices

Topics:

- DevOps culture and practices
- CI/CD pipeline design
- GitHub Actions deep dive
- Automated testing in CI
- Code quality gates
- Security scanning in pipeline
- Docker in production
- Container registries
- Kubernetes for orchestration
- Helm charts for deployment
- GitOps with ArgoCD
- Blue-green deployments
- Canary releases
- Feature flags
- Rollback strategies

Projects built:

- Complete CI/CD pipeline
- Automated testing workflow
- Docker-based deployment
- Kubernetes deployment
- Feature flag system
- Zero-downtime deployment

Practice: Set up CI/CD for all projects

System Design & Architecture

Topics:

- System design fundamentals
- Scalability: horizontal vs vertical
- Load balancing strategies
- Caching layers and patterns
- Database scaling: sharding, replication
- CAP theorem
- Eventual consistency
- Message queues for decoupling
- CDN for static content
- High availability design
- Disaster recovery planning
- Rate limiting at scale
- API design at scale
- Monitoring at scale
- Common system design interviews

Projects built:

- Design URL shortener
- Design Twitter clone
- Design e-commerce platform
- Design notification system
- Design chat application
- System design documentation

Practice: Solve 15 system design problems

Performance & Optimization

Topics:

- Frontend performance metrics
- Core Web Vitals optimization
- JavaScript bundle optimization
- Image and asset optimization
- Lazy loading strategies
- Server-side performance
- Database query optimization
- Caching strategies
- Memory management in Node.js
- Profiling Node.js applications
- Load testing with k6
- Performance monitoring
- Real User Monitoring (RUM)
- Synthetic monitoring
- Performance budgets

Projects built:

- Performance audit and fix
- Bundle optimization exercise
- Database optimization
- Load testing suite
- Performance monitoring setup
- Optimization documentation

Practice: Optimize all applications for performance

Advanced TypeScript Patterns

Topics:

- Advanced type manipulation
- Recursive types
- Variadic tuple types
- Template literal types advanced

- Type-level programming
- Branded types for type safety
- Builder pattern with types
- Dependency injection in TypeScript
- Decorators and metadata
- Reflection with TypeScript
- Monorepo with TypeScript
- Turborepo and Nx
- Shared packages and libraries
- Publishing npm packages
- TypeScript compiler API

Projects built:

- Type-safe library creation
- Monorepo setup
- Shared component library
- npm package publishing
- Advanced utility types
- Type-level validation

Practice: Create 5 advanced TypeScript projects

PHASE 4 COMPLETION - Career Preparation (Week 40-52 (distributed))

Code Quality & Best Practices

Topics:

- Clean code principles
- SOLID principles in TypeScript
- Design patterns in TypeScript
- Code refactoring techniques
- Code review best practices
- ESLint and Prettier configuration
- Static analysis tools
- Documentation with TSDoc
- README and wiki writing
- Architecture documentation
- ADRs (Architecture Decision Records)
- Technical debt management
- Code metrics and quality gates
- SonarQube integration
- Continuous improvement

Projects built:

- Code refactoring exercise
- Design patterns implementation
- Documentation overhaul
- Quality gates setup
- Code review guidelines
- Best practices checklist

Practice: Refactor all projects with best practices

Open Source & Community

Topics:

- Open source contribution guide
- Finding beginner-friendly projects
- Reading contribution guidelines
- Forking and pull requests
- Code review in open source
- Building open source portfolio
- Creating your own library
- npm package management
- Semantic versioning
- Changelog generation
- Community building
- Technical blogging
- Speaking at meetups
- Building personal brand
- Developer advocacy

Projects built:

- Contribute to 5 open source projects
- Create and publish npm package
- Technical blog posts
- Lightning talk preparation
- GitHub profile optimization
- Portfolio website creation

Practice: Active open source participation

Interview Preparation

Topics:

- TypeScript interview questions
- React and Next.js questions
- Node.js and Express questions
- System design interview prep
- Coding interview strategies
- LeetCode problem patterns

- Data structures review
- Algorithms review
- Behavioral interview questions
- STAR method for answers
- Salary negotiation tactics
- Job search strategies
- Resume optimization
- LinkedIn optimization
- Portfolio presentation

Projects built:

- Solve 150+ LeetCode problems
- System design case studies
- Mock interview practice
- Interview preparation guide
- Resume and portfolio polish
- LinkedIn profile optimization

Practice: Daily coding challenges and mock interviews

Final Capstone Planning

Topics:

- Project ideation and planning
- Requirement analysis
- System architecture design
- Technology stack selection
- Development workflow setup
- Sprint planning
- Agile methodology
- Project documentation
- Risk assessment
- Timeline estimation

Projects built:

- FINAL CAPSTONE: Enterprise Full-Stack Application
- Option 1: SaaS Platform (multi-tenant, subscriptions, dashboards)
- Option 2: E-commerce Marketplace (vendors, products, orders, payments)
- Option 3: Social Learning Platform (courses, progress, community, certificates)
- Option 4: Project Management Suite (workspaces, projects, collaboration, real-time)

Final Month - Capstone & Career Launch (Week 49-52)

Final Capstone Project - Part 1

Topics:

- Implementation phase
- Frontend development
- Backend API development
- Database design
- Authentication and authorization
- File handling and storage
- Real-time features
- Third-party integrations

Projects built:

- Full-stack implementation
- Frontend with Next.js
- Backend APIs
- Database setup with Prisma
- Authentication system
- Core features development

Final Capstone Project - Part 2

Topics:

- Implementation completion
- Comprehensive testing
- Performance optimization
- Security hardening
- Documentation writing
- Deployment to cloud
- CI/CD pipeline setup
- Monitoring and logging
- Load testing
- Project presentation preparation

Deliverables:

- Complete source code on GitHub
- Deployed application on Vercel/AWS
- API documentation (OpenAPI/GraphQL)
- Architecture diagrams
- Database schema documentation
- CI/CD pipeline configuration
- Test coverage report (80%+ coverage)
- Performance testing report
- Video demo and presentation
- Comprehensive README

Career Launch & Professional Development

Topics:

- Professional portfolio website
- Resume optimization for TypeScript roles
- LinkedIn profile enhancement
- GitHub profile showcase
- Technical blog writing
- Speaking at meetups/conferences
- Networking strategies
- Job application process
- Interview follow-up
- Continuous learning plan
- Staying updated with ecosystem
- Mentoring and teaching others
- Freelancing strategies
- Consulting career path
- Startup opportunities

Deliverables:

- Professional portfolio with 40+ projects
- Optimized resume for full-stack roles
- Enhanced LinkedIn profile
- GitHub profile with contributions
- Technical blog posts
- Personal brand establishment
- Job application materials ready
- Mock interview completion
- Continuous learning roadmap
- Professional network connections

Assessment: FINAL COMPREHENSIVE CERTIFICATION EXAM - Complete TypeScript mastery evaluation

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Months 1-3): 20+ foundational projects - calculators, games, console apps, TypeScript utilities
- Phase 2 (Months 4-6): 18+ frontend projects - React components, dashboards, forms, state management
- Phase 3 (Months 7-9): 20+ backend projects - REST APIs, databases, authentication, real-time apps
- Phase 4 (Months 10-12): 15+ advanced projects - Next.js apps, GraphQL, microservices, cloud deployments
- Final: 5 major capstone projects demonstrating complete full-stack expertise

Total projects: 70+ projects from beginner to enterprise-grade production systems

SKILLS THE STUDENT WILL HAVE AT THE END

- Core TypeScript: Types, Interfaces, Generics, Utility Types, Advanced Patterns, Type Guards
- Frontend: React, Hooks, State Management, Testing, Performance Optimization, Accessibility
- Backend: Node.js, Express, REST APIs, Authentication, Authorization, Security
- Databases: PostgreSQL, MongoDB, Prisma, Drizzle, Redis, Query Optimization
- Full-Stack: Next.js, Server Components, Server Actions, tRPC, GraphQL
- Testing: Jest, React Testing Library, Supertest, Cypress, Playwright
- DevOps: Docker, Kubernetes, CI/CD, GitHub Actions, Cloud Deployment
- Cloud: AWS, Vercel, Serverless, Lambda, S3, CloudFront
- Architecture: System Design, Microservices, Event-Driven, CQRS, SAGA
- Tools: VS Code, Git, npm, Webpack, Vite, ESLint, Prettier
- Soft Skills: Problem-solving, System Design Thinking, Code Review, Documentation, Agile

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 4-6 hours
 Hands on coding: 8-10 hours
 Projects: 3-5 hours
 Practice problems: 2-3 hours
 Total per week: 15-20 hours

SUPPORT INCLUDED

Live sessions: Weekly live coding sessions and doubt clearing
 Mentorship: 1-on-1 expert TypeScript developer mentorship
 Community: Active Discord/Slack community with 24/7 peer support
 Code review: Expert code reviews for all major projects
 Career support: Resume review, mock interviews, job referrals to product companies
 Lifetime access: All content, future updates, new TypeScript versions coverage
 Interview prep: Mock interviews with industry professionals
 Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Phase certificates: Certificate after each phase completion (4 certificates)

Final certificate: Professional Full-Stack TypeScript Developer Certification
Frontend certificate: React & Next.js Specialist Certificate
Backend certificate: Node.js Backend Developer Certificate
Linkedin badge: Digital badges for LinkedIn profile
Industry recognized: Recognized by Fortune 500 companies and startups
Portfolio projects: 40+ production-ready portfolio projects

WHERE THIS CAN LEAD

- TypeScript Developer (Junior to Senior levels)
- Frontend Developer / React Developer
- Full-Stack Developer / Full-Stack Engineer
- Next.js Developer
- Node.js Backend Developer
- Software Engineer
- Solutions Architect
- Technical Lead / Team Lead
- DevOps Engineer (TypeScript focus)
- Cloud Engineer (AWS/Vercel)
- Software Architect
- Principal Engineer
- Freelance Full-Stack Consultant
- Technical Trainer / Instructor
- Startup Founder / CTO

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Early career roles: Rs 3-5 LPA (Junior Developer / Fresher)

Growing roles: Rs 5-10 LPA (Frontend Developer / React Developer); Rs 8-15 LPA (Full-Stack Developer)

Experienced roles: Rs 12-25 LPA (Senior Full-Stack Developer / Tech Lead); Rs 20-45 LPA (Lead Developer / Architect)

Freelance: Rs 1,500-6,000/hour based on expertise

International usa: \$90k-200k USD (TypeScript Developer to Senior)

International europe: €55k-130k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need to know JavaScript before joining this TypeScript course?

A: TypeScript is a typed layer on top of JavaScript, so the course opens with a JavaScript foundation phase before moving into TypeScript itself. Complete beginners can start from zero, while students who already know JavaScript basics move through the early weeks faster and reach the React and Node.js phases sooner. Your mentor adjusts the pace to your starting point.

Q: What will I actually build during the 12 months?

A: You build in layers: typed utilities and small tools in the foundation phase, React applications with TypeScript in months 4 to 6, Node.js and Express REST APIs with real databases in months 7 to 9, and a full stack Next.js and GraphQL capstone deployed to the cloud in the final phase. Every project is pushed to GitHub so your portfolio grows with the course.

Q: Which TypeScript version and stack do you teach?

A: Classes run on current TypeScript 5.x with modern tooling, plus React, Node.js, Express, Next.js and GraphQL. When the language or ecosystem moves, the material moves with it, so you are never practicing on an outdated stack.

Q: Will this course help me get internships or clear placement interviews?

A: It prepares you honestly: TypeScript is one of the most requested skills in frontend and full stack job listings, and you finish with deployed projects, code review experience and practice on the JavaScript and TypeScript questions interviewers actually ask. We do not promise placements or salaries; we make sure your fundamentals and portfolio can stand up to scrutiny.

Q: Why join a live course when TypeScript tutorials are free on YouTube?

A: Tutorials show someone else coding; they cannot look at your code. In live classes an instructor watches you work, catches weak typing habits like overusing the any type, reviews your projects line by line and answers questions the moment you are stuck. That feedback loop is the difference between a watch history and a portfolio.

Q: Can I try a class before paying?

A: Yes. Book a free demo class at learn.modernagecoders.com/contact and attend a real live session with no commitment and no payment details required. You can also reach us on WhatsApp

at +91 91233 66161.

Q: How are the classes conducted?

A: Every class is a live 1-hour online session with an instructor, never a pre-recorded video. Sessions are interactive: you code along, ask questions and get corrected in real time. Evening and weekend slots are available, which works well around college timetables and jobs.

Q: Do I get a certificate at the end?

A: You receive a completion certificate when you finish the course. More importantly for hiring, you leave with a GitHub-ready portfolio of TypeScript projects, which is what recruiters and interviewers actually open.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/complete-typescript-programming-masterclass-college>

=====

COURSE 114 OF 118: VEDIC MATHS MASTERY: SPEED CALCULATION WITH ALL 16 SUTRAS

=====

Course link: <https://learn.modernagecoders.com/courses/vedic-maths-course-speed-calculation-mastery>
Subject area: Speed Mathematics
Designed for: Middle school
Who can join: Beginner friendly, ages 8 to adult
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 20-30 minutes of drill practice most days
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Vedic maths course online with live classes: all 16 sutras for fast multiplication, squares, division and mental calculation. Ages 8 to adult, free demo class.

In one line: The same sums, done in a third of the time, with a method you can check.

ABOUT THE PROGRAMME

Vedic maths is a system of calculation shortcuts organised around 16 sutras, short working rules like Nikhilam (all from 9 and the last from 10) and Urdhva-Tiryagbhyam (vertically and crosswise). Used well, they turn sums like 97×96 or 998 squared into a few seconds of mental work. This 6-month live course teaches the system the way a maths teacher would: every shortcut is shown, practised against the clock, and then explained, because a student who knows why a trick works can trust it in an exam and adapt it when the numbers change. The course runs in one straight line: multiplication shortcuts first, because they pay off fastest, then squares and cubes, then division including the flag method, then fractions, decimals and percentages at speed, and finally the algebra applications of the same sutras. Week 1 starts with a timed diagnostic; week 24 ends with the same style of paper, so every student can see exactly how much faster and more accurate they have become. We are honest about what this course is: it makes arithmetic faster and frees up exam time. It does not replace school maths, and it will not turn anyone into a prodigy. It rewards steady drill, and we structure that drill for you.

WHAT MAKES THIS COURSE DIFFERENT

- Every shortcut comes with its reason: students see the algebra behind Nikhilam and the duplex method, so the rules stick instead of jumbling together
- Timed drills from week 1, with a personal speed log so progress is measured against your own diagnostic, not against a claim
- We teach judgement, not just tricks: which sutra fits which sum, and when the ordinary school method is actually the better choice
- Digit-sum checking is built in from the start, so speed never comes at the cost of accuracy
- Mixed-age friendly: the same system is drilled at school-exam pace for children and at competitive-exam pace for older students
- Live small-group classes where the teacher watches you calculate, catches bad habits early, and adjusts the drill sheets to your errors

THE LEARNING PATH, STEP BY STEP

Phase 1: The multiplication engine: complements, Nikhilam base multiplication, special patterns, then Urdhva-Tiryagbhyam for any numbers, all against the clock
Phase 2: Squares, cubes and roots with Yavadunam and the duplex method, then division three ways: Nikhilam, Paravartya and the flag method
Phase 3: Fractions, decimals and percentages at speed, the algebra applications of the

sutras, and a timed capstone measured against the week 1 diagnostic

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable addition and subtraction, and multiplication tables up to 10. That is genuinely all

Age: 8 to adult. Batches are grouped by age so drills run at the right pace

Equipment: A notebook, a pen, and a device with a camera and stable internet for live classes

Prior vedic maths: None expected. The course starts from the first sutra

WHO THIS COURSE IS FOR

School students: Students in classes 3 to 10 who lose exam time to slow arithmetic and want it back

Competitive exam aspirants: Older students preparing for arithmetic-heavy MCQ exams where a saved minute per section matters

Parents learning alongside: Adults who want to sharpen mental arithmetic, sometimes in the same batch as their child

Students who fear calculation: Children who understand concepts but freeze on big multiplications; the drills rebuild confidence with evidence

Maths enjoyers: Students who simply like numbers and want to know why the shortcuts work, not just that they do

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: THE MULTIPLICATION ENGINE (MONTHS 1-2, WEEKS 1-8) -----

Multiplication shortcuts pay off fastest, so they come first: complements and base multiplication, the special patterns, then Urdhva-Tiryagbhyam for any two numbers.

Month 1: Complements and Base Multiplication (Weeks 1-4)

How the System Works: Complements

Topics:

- Timed diagnostic paper: 30 mixed sums, your before picture for the whole course
- What the 16 sutras are and how this course walks through them
- Nikhilam Navatashcaramam Dashatah: all from 9 and the last from 10
- Instant subtraction from 100, 1000 and 10000 using complements
- Left-to-right working: why Vedic methods read numbers the way we say them
- Digit sums (Beejank) as a ten-second answer check

Projects built:

- Personal speed log opened: diagnostic scores recorded by sum type, to be updated all course

Practice: 40 complement subtractions from 100, 1000 and 10000, target under 5 minutes with every answer digit-sum checked

Nikhilam Multiplication Below the Base

Topics:

- Multiplying numbers just under 100: 97×96 in one line
- The two-part answer: cross subtraction on the left, deficiency product on the right
- Base 1000 and base 10: 988×997 , 8×7 done the Nikhilam way
- Handling carries when the right-hand part overflows
- Why it works: the algebra of $(base - a)(base - b)$, shown honestly
- Spotting when a sum is close enough to a base for Nikhilam to be worth it

Projects built:

- One-page proof poster: a student's own explanation of why Nikhilam works, with one worked example per base

Practice: 30 Nikhilam multiplications near 100 and 1000, target under 8 minutes, misses redone with the algebra written out

Above the Base and Working Bases

Topics:

- Numbers just above the base: 104×103 , 1002×1015
- Mixed cases, one above and one below: 105×97 and the negative right-hand part
- Vinculum digits: writing 98 as 100 minus 2 inside a calculation
- Anurupyena, the working base idea: multiplying near 50, 200, 250 and 500
- Choosing the base that makes the sum smallest
- Speed drill technique: reading the sum, picking the method, then writing

Projects built:

- Worked set: 20 mixed base multiplications with the chosen base and method noted next to each

Practice: 35 mixed above/below/working-base multiplications, target under 10 minutes, with the base choice justified for any five

The Special Patterns

Topics:

- Ekadhikena Purvena: squares of numbers ending in 5, done in two seconds
- Antyayordashake'pi: same start, last digits adding to 10, such as 43×47
- Multiplying by 11 and by 12 with single-pass rules
- Multiplying by 5, 25 and 50 as fractions of powers of 10
- Pattern recognition drills: which of the five patterns fits, or none
- Consolidation of month 1: complements, Nikhilam, working bases, patterns

Projects built:

- Pattern wall chart: all month 1 shortcuts on one sheet in the student's own words and examples

Practice: 50-question pattern-spotting sheet mixing all month 1 techniques, target under 15 minutes

Assessment: Month 1 speed check: timed mixed paper, scores logged against the week 1 diagnostic

Month 2: Urdhva-Tiryagbhyam, Multiplication for Any Numbers (Weeks 5-8)

Vertically and Crosswise: Two Digits

Topics:

- Urdhva-Tiryagbhyam for 2-digit x 2-digit: the three-step cross pattern
- Where each part of the answer comes from, mapped to long multiplication
- Carry discipline: keeping the running answer clean
- One-line working: 34×72 without intermediate rows
- Checking with digit sums after every sum, as a habit not an afterthought
- When Nikhilam beats Urdhva-Tiryagbhyam and the reverse

Projects built:

- Cross-pattern diagram set: the student draws the crosswise structure for five sums before solving them

Practice: 40 two-digit by two-digit crosswise multiplications, target under 12 minutes at 90 percent accuracy

Three Digits and Mixed Sizes

Topics:

- The five-step cross pattern for 3-digit x 3-digit
- Mixed sizes: 3-digit x 2-digit by padding with a zero
- Managing bigger carries without losing the thread
- Writing less: moving from full working to compact working
- Common crosswise errors and how the digit-sum check catches them
- Speed strategy: which digit pairs to multiply first

Projects built:

- Worked set: 15 three-digit crosswise multiplications with compact working shown

Practice: 25 three-digit crosswise multiplications, target under 15 minutes, every answer digit-sum checked

Taking It Mental

Topics:

- Dropping the pen: holding the cross pattern in your head for 2-digit sums
- Chunking: multiplying 68×7 as 60×7 plus 8×7 , said aloud
- Doubling and halving: 16×35 as 8×70
- Multiplying by 125, 75 and 15 through fraction thinking
- Estimation first: knowing roughly what the answer must be before starting
- Building a personal mental-maths warm-up routine

Projects built:

- Mental drill routine card: a 5-minute daily warm-up the student designs and follows

Practice: Daily 5-minute no-paper drills: 20 two-digit multiplications spoken or typed, streaks recorded in the speed log

Speed Lab One

Topics:

- Full multiplication mix under exam timing: patterns, bases, crosswise, mental
- Error triage: was it method choice, a carry slip, or rushing
- Reading your speed log: what actually got faster since week 1
- Accuracy floors: why we never trade below 90 percent accuracy for speed
- Personal weak-spot drill sheets, built from your own error log
- Setting the phase 2 targets with the teacher

Projects built:

- Speed lab paper one: a timed 60-question multiplication paper, marked and logged the same day

Practice: Redo every speed lab miss untimed with full working, then a second

20-question timed set on just those types

Assessment: Phase 1 milestone: timed multiplication paper compared line by line with the week 1 diagnostic

PHASE 2: SQUARES, CUBES AND DIVISION (MONTHS 3-4, WEEKS 9-16)

Powers and roots with Yavadunam and the duplex method, then the part most courses skip: division done three ways, including Tirthaji's general flag method.

Month 3: Squares, Cubes and Roots (Weeks 9-12)

Squares Near a Base

Topics:

- Yavadunam: squaring by deficiency, 98 squared in one line
- Squaring just above the base: 104 squared, 1012 squared
- Working bases again: 48 squared using base 50
- The algebra behind it: $(base + d)^2$ squared, expanded and matched to the rule
- Squares of numbers ending in 5 revisited and extended to 3 digits
- Choosing between Yavadunam and the ending-in-5 rule

Projects built:

- Worked set: squares of every number from 91 to 109 by the fastest route, method noted for each

Practice: 30 near-base squares across bases 50, 100, 500 and 1000, target under 10 minutes

The Duplex Method: Any Square

Topics:

- The duplex (Dwandwa) of a number: definition and drill
- Squaring any 2-digit number with duplexes
- Squaring 3-digit numbers: the five duplex terms in order
- Keeping carries straight in duplex work
- Duplex squares against near-base squares: picking the faster tool
- Digit-sum checking squares, including the common square-check traps

Projects built:

- Duplex table: the student builds a reference table of duplexes for 1 to 3 digit forms with examples

Practice: 25 general squares of 2 and 3 digit numbers by duplex, target under 12 minutes at 90 percent accuracy

Square Roots and Cube Patterns

Topics:

- Square roots of perfect squares: pairing digits and reading the first digit
- Last-digit analysis: which digits can end a perfect square and what they tell you
- Deciding between the two candidate roots quickly
- Cubes near a base: 98 cubed and 102 cubed by the Nikhilam-style rule
- The proportional (Anurupyena) cube rule for 2-digit numbers
- Recognising perfect squares and cubes on sight

Projects built:

- Root-finder sheet: 20 perfect squares up to 5 digits, roots found and verified by squaring back

Practice: 20 square roots of perfect squares and 10 near-base cubes, target under 15 minutes

Cube Roots and the Powers Checkpoint

Topics:

- Cube roots of perfect cubes up to 6 digits from the two ends
- Why cube last digits are honest: the 0 to 9 cube-ending map
- Mixed powers drill: squares, cubes and roots on one sheet
- Where powers shortcuts show up in school exams and MCQ papers
- Month 3 consolidation: one method chart for all powers work
- Speed log review: powers section opened and baselined

Projects built:

- Powers method chart: every squares, cubes and roots technique on one page with a worked example each

Practice: Mixed 40-question powers paper covering all month 3 techniques, target under 18 minutes

Assessment: Month 3 milestone: timed powers paper, results logged and weak types tagged for drilling

Month 4: Division Three Ways (Weeks 13-16)

Nikhilam Division

Topics:

- Dividing by 9: the pattern that starts the whole method
- Nikhilam division with divisors just below a base: 89, 98, 888
- Quotient and remainder columns, and moving carries between them
- When the remainder column overflows and how to fix it
- Why it works: complements doing subtraction's job in division
- Speed comparison against long division on the same sums

Projects built:

- Side-by-side set: 10 divisions worked both by long division and Nikhilam with times recorded for each

Practice: 25 Nikhilam divisions with divisors near 10, 100 and 1000, target under 12 minutes

Paravartya: Transpose and Apply

Topics:

- Paravartya Yojayet for divisors just above a base: 12, 112, 1023
- Transposing digits: turning the divisor's excess into negative multipliers
- Working with negative intermediate digits without panic
- Nikhilam or Paravartya: reading the divisor and choosing in two seconds
- Remainder checks with digit sums
- The algebra behind Paravartya, shown for divisor 12

Projects built:

- Choice drill sheet: 20 divisions labelled Nikhilam or Paravartya before solving, choices reviewed in class

Practice: 25 Paravartya divisions with divisors near 10, 100 and 1000, target under 14 minutes

Straight Division: The Flag Method

Topics:

- Dhvajanka, the flag digit: general division for any divisor
- Setting up the flag: splitting the divisor into main digit and flag
- The subtract-and-divide rhythm, worked slowly then at pace
- Two-digit flags for harder divisors
- Decimal answers: continuing the division past the point
- Why the flag method matters: one method that always works

Projects built:

- Worked set: 12 flag-method divisions including two carried to three decimal places

Practice: 20 flag-method divisions with mixed divisors from 13 to 87, target under 20 minutes

Divisibility and Speed Lab Two

Topics:

- Divisibility tests for 2 through 12, consolidated and drilled
- Osculation: Ekadhikena-based tests for 7, 13, 19 and other awkward primes
- Remainder thinking: Navashesh checks for division answers
- Full division mix under time: Nikhilam, Paravartya, flag, divisibility
- Error triage on division work
- Phase 3 targets set with the teacher

Projects built:

- Speed lab paper two: a timed 50-question powers and division paper, marked and logged the same day

Practice: Redo all speed lab misses untimed, then 15 divisibility questions on primes 7, 13 and 19 by osculation

Assessment: Phase 2 milestone: timed powers and division paper, logged against months 3 and 4 baselines

PHASE 3: FRACTIONS, ALGEBRA AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24)

The system applied where exams actually spend marks: fractions, decimals and percentages at speed, the algebra behind the sutras, and a timed capstone measured against week 1.

Month 5: Fractions, Decimals and Percentages at Speed (Weeks 17-20)

Recurring Decimals the Short Way

Topics:

- Ekadhikena Purvena on fractions: $1/19$ written out in seconds
- The one-more-than-the-previous multiplier, forwards and backwards
- $1/7$ and the cyclic pattern family
- Auxiliary fractions: converting awkward denominators before dividing
- Terminating or recurring: reading the denominator to know in advance
- Where recurring-decimal questions appear in school papers

Projects built:

- Decimal expansion chart: $1/7$, $1/13$, $1/17$, $1/19$ expanded by sutra and verified by flag division

Practice: 15 recurring-decimal expansions by the Ekadhikena method, each verified against straight division

Fraction Arithmetic Without the Grind

Topics:

- Adding and subtracting fractions crosswise, no LCM hunt for two terms
- When the LCM is still worth finding, honestly
- Comparing fractions instantly by cross multiplication
- Multiplying and dividing mixed numbers with vinculum thinking
- Simplifying early: spotting common factors before multiplying
- Fraction chains: working three-term expressions left to right

Projects built:

- Worked set: 20 fraction additions and comparisons done crosswise with times against the textbook method

Practice: 30 mixed fraction operations, target under 15 minutes, five of them written both ways to compare

Percentages and Ratio Shortcuts

Topics:

- Percentages as fractions: the 12.5 percent equals one-eighth family
- Flipping the sum: 8 percent of 50 as 50 percent of 8
- Percentage change chains done mentally
- Ratio splitting at speed for word problems
- Profit, loss and simple interest set up as one-line calculations
- Estimation guards: catching a wrong percentage before it costs marks

Projects built:

- Percent-fraction equivalence card: the conversion table every student memorises, self-tested twice

Practice: 40 mental percentage and ratio questions, target under 12 minutes, no written working allowed

Decimals and Speed Lab Three

Topics:

- Multiplying decimals with base methods: 9.8×9.7 via 98×97
- Placing the decimal point by estimation, not rules memorised blind
- Dividing decimals with the flag method
- Mixed arithmetic under time: the full toolkit on one paper
- Error triage across fraction, decimal and percentage work
- Month 5 consolidation chart

Projects built:

- Speed lab paper three: a timed 50-question fractions, decimals and percentages paper, marked and logged

Practice: Redo all speed lab misses untimed, then a 20-question timed set on your two weakest question types

Assessment: Month 5 milestone: timed mixed arithmetic paper, logged against the phase baselines

Month 6: Algebra Applications and the Speed-Test Capstone (Weeks 21-24)

The Sutras Meet Algebra

Topics:

- Urdhva-Tiryagbhyam on binomials: $(x + 3)(x + 7)$ crosswise
- Multiplying larger polynomials by the same cross pattern
- Paravartya for dividing polynomials by $(x - a)$
- The connection made explicit: the arithmetic sutras were algebra all along
- Checking polynomial products with substitution
- Where this helps in school algebra chapters

Projects built:

- Bridge sheet: five arithmetic shortcuts and their algebraic twins worked side by side

Practice: 20 polynomial multiplications and 10 Paravartya polynomial divisions, checked by substitution at x equals 2

Equations at Speed

Topics:

- Sunyam Samyasamuccaye: spotting when a common factor makes the answer zero
- One-line solutions to equation types that usually take five lines
- Simultaneous equations by cross-multiplication patterns
- Anurupye Sunyamanyat: proportional coefficients and what they force
- Recognising equation types on sight before solving
- Honest limits: equations where the shortcuts do not apply

Projects built:

- Equation type-spotting sheet: 15 equations sorted by fastest method before any solving

Practice: 25 equations mixing shortcut types and standard types, with the method named before each solution

Choosing the Right Tool

Topics:

- Method triage: 40 mixed questions, decide the method in five seconds each
- Exam mapping: where each technique saves time in school and MCQ papers
- Where standard methods win and showing full working is required
- Building your personal shortcut handbook: what goes in, what gets dropped
- Mock speed test under full exam conditions
- Reviewing the mock: method choices, not just answers

Projects built:

- Personal shortcut handbook draft: every technique the student actually uses, in their own words and examples

Practice: Full 60-question mock speed test under timing, then a written half-page on your three best method choices

The Capstone Speed Test

Topics:

- Final timed paper: same structure as the week 1 diagnostic, full toolkit allowed
- Before and after: reading your speed log across all 24 weeks
- Accuracy audit: error rate then versus now
- Finishing the shortcut handbook: the take-away artifact of the course
- Keeping the speed: a 10-minute weekly maintenance routine
- Where to go next: exam-prep courses that use these skills daily

Projects built:

- Capstone speed test with a written before-and-after comparison against the week 1 diagnostic
- Finished personal shortcut handbook, reviewed by the teacher

Practice: Design your own maintenance drill sheet and run it once; bring the timing to the final class

Assessment: Course milestone: capstone speed test and handbook review, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal speed log kept for all 24 weeks, from diagnostic to capstone
- One-page proof poster explaining why Nikhilam multiplication works
- Pattern wall chart of every month 1 shortcut in the student's own words
- Duplex reference table built by the student for general squares
- Side-by-side division sets timing Vedic methods against long division
- Decimal expansion chart for the awkward sevenths and nineteenthths
- Percent-fraction equivalence card, memorised and self-tested
- Bridge sheet connecting five arithmetic sutras to their algebra twins
- Personal shortcut handbook: the student's complete method reference, finished in week 24
- Capstone speed test with a written before-and-after comparison

Total projects: 10+ artifacts, from a week 1 speed log to a complete self-written shortcut handbook

SKILLS THE STUDENT WILL HAVE AT THE END

- Base multiplication with Nikhilam, working bases and the special patterns
- General multiplication with Urdhva-Tiryagbhyam, on paper and mentally
- Squares, cubes and roots with Yavadunam, the duplex method and end-digit analysis
- Division by Nikhilam, Paravartya and the flag method, plus osculation divisibility tests
- Fractions, recurring decimals and percentages at exam speed

- Digit-sum checking as a permanent accuracy habit
- Method triage: reading a question and picking the fastest honest route

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, with timed drills done together in class
 Practice: 20-30 minutes of drill sheets most days; short daily practice beats long weekend sessions for speed work
 Review: Drill sheets reviewed with error triage: every miss classified as method, carry or rush, and drilled accordingly

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so a stuck technique never waits a week
 Progress updates: Regular progress notes to parents with the speed log attached: what got faster, what needs drilling

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside the student's own documented speed log

WHERE THIS CAN LEAD

- Faster arithmetic that pays off directly in school maths exams from class 4 upward
- A head start for competitive-exam arithmetic sections, where calculation speed buys thinking time
- Natural progression to our school maths and exam-prep courses, with calculation no longer the bottleneck
- A strong base for algebra, since Urdhva-Tiryagbhyam and Paravartya reappear there directly
- For adults: everyday numeracy, from splitting bills to sense-checking figures at work

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Vedic maths, honestly?

A: It is a system of 16 sutras, short calculation rules, compiled by Bharati Krishna Tirthaji and published in 1965. Historians debate how ancient the system really is, and we say so in class, because the techniques stand on their own: they are fast, they are checkable, and every one of them has ordinary algebra underneath. We teach the methods and the algebra, not the mythology.

Q: Will this replace my child's school maths?

A: No, and it should not. School maths teaches concepts and requires standard working in answers; this course makes the arithmetic inside those answers faster and more accurate. We explicitly teach when to use a shortcut, when to show the standard method, and how to use digit-sum checks on both. Think of it as removing the calculation bottleneck from maths a student is already learning.

Q: Does Vedic maths actually help in competitive exams?

A: It helps with one specific thing: calculation speed in arithmetic-heavy sections, which frees minutes for the questions that need real thought. It does not teach exam concepts, shortcuts for reasoning, or anything a syllabus tests directly. Students preparing for MCQ exams tend to feel the benefit most in percentages, fractions and multiplication-heavy questions, which is exactly where phase 3 of this course spends its time.

Q: Will this course make my child a genius or a human calculator?

A: No, and we would rather tell you that upfront. What steady practice on this course produces is documented in each student's speed log: the same kinds of sums done faster and with fewer errors than in week 1. That is a real, useful, measurable gain. Claims beyond that, about IQ or brain development, are not ones we make.

Q: What age is right for this course?

A: We take students from age 8, once multiplication tables are reasonably solid, and there is no upper limit; adults join to sharpen mental arithmetic, sometimes in the same house as an enrolled child. Batches are grouped by age so an 8-year-old drills at a different pace from a 16-year-old preparing for entrance exams.

Q: How is Vedic maths different from abacus classes?

A: Abacus training builds mental arithmetic through a physical tool and bead visualisation, and it works best when started young, roughly ages 5 to 10. Vedic maths is paper-and-head arithmetic built on algebraic patterns, so it suits any age that knows its tables and extends naturally into algebra. Younger children often do abacus first and Vedic maths later; we run both and will honestly tell you which fits your child.

Q: My child already struggles with maths. Is a speed course a bad idea?

A: Often it is the opposite, with one condition: the struggles must be with calculation, not

with concepts. A child who understands what multiplication means but dreads big numbers usually gains confidence quickly here, because the methods are short and the progress is visible in their own log. If the gaps are conceptual, we will say so after the demo class and point you to our school maths courses instead.

Q: Can we try a class before enrolling?

A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in, and the teacher will give you a straight opinion on whether this course or a school maths course is the better fit.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/vedic-maths-course-speed-calculation-mastery>

=====

COURSE 115 OF 118: VIBE CODING FOR COLLEGE: FULL STACK, AI & DSA (AGES 18+)

=====

Course link: <https://learn.modernagecoders.com/courses/vibe-coding-for-college-fullstack-ai-dsa-career-course>
Subject area: College-Level Software Engineering
Designed for: College students
Who can join: Intermediate to Advanced (Ages 18+ / College Students)
Course length: 3 months (12 weeks)
Time needed: 10-15 hours/week
Certificate: Vibe Coder, College Level Full Stack & AI Engineer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online vibe coding course for college students 18+: master the MERN stack, DSA, machine learning and system design, crack tech placements and internships.

In one line: From Student to Software Engineer, Ship Real Code, Ace Interviews, Land the Offer.

ABOUT THE PROGRAMME

Vibe Coding for College is the most comprehensive 3-month software engineering program for college students and fresh graduates aged 18+. 'Vibe Coding', coined by AI pioneer Andrej Karpathy in 2025, means building software by describing what you want in natural language and letting AI generate the code. In 2026, this has evolved into 'agentic engineering', orchestrating multiple AI agents to build production-grade software at startup speed. This course teaches you both: deep engineering fundamentals AND professional-grade AI development tools. You will master the MERN stack using Cursor IDE (the #1 AI code editor), accelerate development with Claude Code (Anthropic's terminal-based AI engineer), prototype at speed with Lovable and Bolt.new, and ship code with GitHub Copilot as your AI pair programmer. You'll also dive deep into DSA for placements, build ML projects, and learn System Design. But here's what separates great vibe coders from prompt-copy-pasters: understanding the code AI generates. That's why we teach you to be a developer FIRST and a vibe coder SECOND, so you can review, debug, secure, and scale what AI builds for you. Studies show 45% of AI-generated code contains security vulnerabilities (OWASP Top 10). We teach you to catch them. At Modern Age Coders, we've placed students at top tech companies. Our curriculum is designed by engineers from FAANG-adjacent companies who know exactly what interviews look for and what internships demand of you from day one.

WHAT MAKES THIS COURSE DIFFERENT

- Industry-grade curriculum, same tools used at Google, Amazon, and top startups
- Professional vibe coding tools: Cursor IDE, Claude Code, Windsurf, GitHub Copilot
- AI app builders for rapid prototyping: Lovable, Bolt.new, v0.dev, Replit Agent
- Learn to BUILD with AI AND REVIEW what AI builds, security-aware development
- MERN Stack mastery, the most in-demand full-stack for 2026 hiring
- DSA preparation integrated throughout, not an afterthought
- Machine Learning with Python, scikit-learn, TensorFlow fundamentals
- System Design basics, asked in every senior/product company interview
- Agentic engineering workflows, orchestrating AI agents for multi-step development tasks
- Real project deployment on cloud (AWS EC2 / Render / Vercel)
- Code reviews by industry-experienced mentors (including AI-generated code review)
- Placement preparation: mock interviews, resume reviews, LinkedIn optimization
- Access to Modern Age Coders alumni network of placed students
- Open-source contribution guidance, building a credible GitHub

THE LEARNING PATH, STEP BY STEP

Phase 1: MERN Vibes (Weeks 1-4): MongoDB, Express.js, React.js, Node.js, build full-stack

apps with Cursor IDE, GitHub Copilot & Claude Code

Phase 2: DSA & ML Vibes (Weeks 5-8): Data Structures, Algorithms, Python ML, model deployment
+ vibe coding rapid prototyping with Lovable & Bolt.new

Phase 3: Career Vibes (Weeks 9-12): System Design, agentic engineering, security for
AI-generated code, Capstone, placement prep, professional launch

WHAT A STUDENT NEEDS BEFORE STARTING

Education: College student (any year) or recent graduate, any branch welcome

Coding experience: Familiarity with at least one programming language (Python, Java, C++, or JavaScript)

Web basics: Basic understanding of HTML/CSS (completing Vibe Coding for Teens is ideal but not required)

Equipment: Computer/laptop (Windows, Mac, or Linux) with stable internet

Time commitment: 10-15 hours per week, this is an intensive program

Motivation: Serious about building a career in software engineering

WHO THIS COURSE IS FOR

Cs students: CS/IT/MCA students preparing for campus placements and off-campus hiring

Non cs engineers: Engineering students from any branch (Mech, Civil, EE) switching to tech careers

Career switchers: Recent graduates wanting to break into software development

Dsa preparation: Students who need structured DSA preparation for product company interviews

Startup founders: Aspiring founders who want to build their own MVPs at startup speed with AI tools

Freelancers: Students who want to earn through freelance full-stack development while studying

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: MERN VIBES, FULL STACK WEB DEVELOPMENT (WEEKS 1-4) -----

Phase 1 builds you into a full-stack MERN developer. You'll master both the frontend (React) and backend (Node.js + Express + MongoDB), learn professional API design, authentication, and deploy your first production-level web application.

Week 1: JavaScript Mastery, Node.js & Your AI Development Toolkit

Topics:

- What is Vibe Coding?, Andrej Karpathy's 2025 concept 2026's 'agentic engineering' evolution
- The professional vibe coding workflow: Intent Spec Prompt Generate Review Iterate Ship
- Setting up Cursor IDE, your primary AI-powered development environment
- Cursor Composer & Agent mode, multi-file AI development across your entire codebase
- Setting up GitHub Copilot (free for students), inline AI completions and chat
- Introduction to Claude Code, terminal-based AI that understands your full project
- Windsurf IDE overview, Cascade agent and SWE-1.5 model for lightning-fast completions
- Advanced JavaScript: ES6+ features, destructuring, spread/rest, optional chaining
- Asynchronous JavaScript: callbacks, promises, async/await deep dive
- Event loop and JavaScript's single-threaded model explained
- Node.js architecture, non-blocking I/O, the event loop in Node
- Express.js setup and first REST API, hand-coded, then rebuilt with Cursor Composer
- RESTful API design principles, GET, POST, PUT, DELETE semantics
- Comparing hand-coded vs AI-generated Express APIs, reviewing what AI produces
- Environment variables, nodemon, and development tooling

Projects built:

- Products REST API, CRUD operations for a product catalog (in-memory, no DB yet)
- URL Shortener API, shorten URLs and redirect, stored in memory
- News Aggregation Proxy, fetch and forward data from a public news API

Practice: Build a small Node.js CLI utility in Cursor, then run one Composer multi file refactor and review the diff line by line before committing.

Week 2: MongoDB & Database Design, Persistent, Scalable Data

Topics:

- SQL vs. NoSQL, when to use each, MongoDB's document model
- MongoDB Atlas setup, cloud database in 5 minutes
- CRUD operations with MongoDB shell and Compass
- Mongoose ODM, schemas, models, and data validation
- Schema design: embedded vs. referenced documents
- Population, joining documents across collections
- Indexes for performance, understanding query plans
- Aggregation pipeline, complex data transformations
- Pagination and sorting on large datasets
- Data validation with Joi / Zod
- Seeding databases with realistic test data
- Connecting MongoDB Atlas to your Express app securely

Projects built:

- Blogging Platform API, full CRUD for posts, with categories, tags, and pagination
- Student Records System, schema design with references between students, courses, grades
- E-Commerce Product Catalog, categories, variants, stock tracking in MongoDB

Practice: Design a Mongoose schema for a blog with users, posts and comments, seed it on Atlas and run every CRUD operation from a script.

Week 3: React.js, Modern Frontend Development

Topics:

- React fundamentals: JSX, components, props, and the virtual DOM
- State management with useState, reactive UI basics
- useEffect, side effects, data fetching, and cleanup
- Component composition, building UIs from small, reusable pieces
- Conditional rendering and list rendering with keys
- Event handling in React, forms, clicks, input controlled components
- React Router v6, client-side navigation and nested routes
- Custom hooks, extracting and reusing complex logic
- Context API, sharing state without prop drilling
- React Query / TanStack Query, server state management
- Tailwind CSS with React, rapid, utility-first styling
- v0.dev, generating polished React/Next.js components from text descriptions
- Vibe coding React components: using Cursor Composer to scaffold entire features
- Reading and reviewing AI-generated React code, catching anti-patterns
- Performance: React.memo, useMemo, useCallback, when and why

Projects built:

- Task Management Dashboard, Kanban-style board with drag and React state
- Movie Browser, fetches TMDb API, search, filter, detailed movie pages
- Expense Tracker, add/categorize/delete expenses, charts with Chart.js

Practice: Build a searchable product list in React with useState, useEffect data fetching and correct list keys, then deploy it to a free host.

Week 4: Full MERN, Authentication, Authorization & Deployment

Topics:

- User authentication, registration, login, logout flows
- Password hashing with bcrypt, never store plain text passwords
- JWT (JSON Web Tokens), stateless authentication explained
- Protected routes, both API endpoints and React pages
- Role-based access control (RBAC), admin vs. user permissions
- Connecting React frontend to Express backend across environments
- CORS configuration, handling cross-origin requests
- HTTP-only cookies for secure token storage
- File uploads with Multer and Cloudinary / AWS S3
- Rapid prototyping with Lovable, build a full-stack MVP by chatting with AI
- Bolt.new for speed: idea deployed app in under 30 minutes
- Understanding what AI app builders generate vs. what you need to customize
- Deploying backend on Render (with MongoDB Atlas)
- Deploying React frontend on Vercel
- CI/CD basics with GitHub Actions, auto-deploy on push

Projects built:

- Full-Stack Social Notes App, register/login, create/edit/delete notes, private per user
- Recipe Sharing Platform, MERN app with auth, image uploads, saved recipes

Practice: Add JWT based register and login plus a protected dashboard route to your MERN app, with bcrypt hashed passwords and one admin only endpoint.

PHASE 2: DSA & ML VIBES, THINK DEEPLY, BUILD INTELLIGENTLY (WEEKS 5-8)

Phase 2 is the intellectual heart of the program. We go deep into Data Structures & Algorithms for placement preparation, and simultaneously build practical Machine Learning projects that solve real problems. This phase makes you stand out in every technical interview.

Week 5: Data Structures, The Foundation of Every Interview

Topics:

- Why DSA matters, the real purpose of data structures in engineering
- Time Complexity & Big O Notation, measuring efficiency
- Space Complexity, memory analysis
- Arrays: two-pointer technique, sliding window, prefix sums
- String problems: anagrams, palindromes, pattern matching
- Hash Maps & Hash Sets, O(1) lookup and counting patterns
- Linked Lists: singly, doubly, circular, implementation and operations
- Linked list problems: reverse, detect cycle, find middle, merge sorted lists
- Stacks: implementation, use cases (balanced brackets, undo operations)
- Queues: implementation, BFS patterns
- Deque and monotonic stack patterns
- LeetCode/HackerRank strategy, how to approach problems systematically

Projects built:

- LeetCode Top 50 Arrays & Strings, solve 15 curated problems with explanations
- Custom Hash Map Implementation, build a hash map from scratch in Python
- Text Editor Undo/Redo, stack-based implementation with UI

Practice: Solve eight array and string problems using two pointers, sliding window and prefix sums, recording time and space complexity for each.

Week 6: Trees, Graphs & Advanced Algorithms

Topics:

- Binary Trees, traversals: inorder, preorder, postorder, level order
- Binary Search Trees (BSTs), operations and balancing concepts
- Tree problems: height, diameter, LCA, path sum, zigzag traversal
- Heaps & Priority Queues, min-heap, max-heap, heapify
- Graphs: representations (adjacency list vs. matrix), directed vs. undirected
- BFS, shortest path in unweighted graphs, level-order problems
- DFS, cycle detection, topological sort, connected components
- Dijkstra's Algorithm, weighted shortest path
- Union-Find (Disjoint Set Union), efficient connectivity queries
- Dynamic Programming: memoization vs. tabulation, top-down vs. bottom-up
- Classic DP patterns: Fibonacci, coin change, 0/1 knapsack, LCS, LIS
- Greedy Algorithms, interval scheduling, activity selection

Projects built:

- Graph Visualizer, interactive BFS/DFS visualization built with React
- Pathfinding Visualizer, Dijkstra's and A* in a grid, visually animated
- Course Prerequisite Checker, topological sort applied to a real problem

Practice: Solve six tree and graph problems including one BFS, one DFS and one heap problem, and explain one solution aloud as if in an interview.

Week 7: Machine Learning, Teach Python to Predict the Future

Topics:

- Machine Learning overview, supervised, unsupervised, reinforcement learning
- The ML workflow: data features model evaluate deploy
- Data exploration with pandas, statistical analysis and visualizations
- Feature engineering, creating meaningful inputs for ML models
- Train/test/validation split, avoiding overfitting and data leakage
- Linear Regression, predicting continuous values (housing prices, salaries)
- Logistic Regression, binary classification problems
- Decision Trees and Random Forests, ensemble methods
- Support Vector Machines (SVM), classification with hyperplanes
- K-Means Clustering, unsupervised customer segmentation
- Model evaluation: accuracy, precision, recall, F1, ROC-AUC
- scikit-learn pipelines, cleanly combining preprocessing and models

Projects built:

- House Price Predictor, regression model trained on real estate dataset
- Email Spam Classifier, text-based binary classification with logistic regression
- Customer Segmentation, K-means on e-commerce purchase data

Practice: Train a scikit-learn model on a public dataset with a proper train test split, reporting accuracy and one feature engineering decision.

Week 8: Deep Learning & AI Project Deployment

Topics:

- Neural Networks, neurons, layers, activation functions explained intuitively
- How deep learning trains, gradient descent, backpropagation
- TensorFlow and Keras, building and training neural networks
- Convolutional Neural Networks (CNNs), image recognition
- Transfer Learning, using pre-trained models (MobileNet, ResNet) for new tasks
- Natural Language Processing basics, tokenization, embeddings, TF-IDF
- Hugging Face Transformers, using BERT and GPT models via Python
- Model serialization, saving and loading trained models
- Deploying ML models as REST APIs with Flask/FastAPI
- Streamlit, building beautiful ML demo apps with pure Python
- Using Claude Code to scaffold ML pipelines: describe your data AI writes the training code
- Vibe coding ML dashboards: using Lovable or Bolt.new to build a frontend for your ML API
- Monitoring ML models in production, drift, versioning, logging
- AI responsible use, bias auditing and model transparency

Projects built:

- Image Classifier API, custom CNN trained on a dataset of your choice, deployed
- Sentiment Analysis Web App, Hugging Face transformer, Flask API, Streamlit UI
- Real-Time Object Detector, webcam feed with YOLO model via OpenCV

Practice: Fine tune a pre-trained CNN such as MobileNet on a small image dataset with Keras and record a short working demo.

PHASE 3: CAREER VIBES, SYSTEM DESIGN, CAPSTONE & PROFESSIONAL LAUNCH (WEEKS 9-12)

The final phase transforms a skilled developer into a career-ready software engineer. System Design, advanced cloud deployment, the grand capstone project, and an intensive placement preparation program, this phase exists to get you hired.

Week 9: System Design & Agentic Engineering, Think Like a Senior Engineer

Topics:

- What is System Design? Why it's asked in every senior tech interview
- Agentic Engineering, Karpathy's evolution of vibe coding: orchestrating AI agents for complex

tasks

- Using Claude Code for large codebase navigation, AI that understands your entire project
- Cursor Agent mode: giving AI a task, letting it plan and execute across dozens of files
- Multi-agent workflows: using Cursor (planning) + Claude Code (execution) + Copilot (inline) together
- Scalability: horizontal vs. vertical scaling
- Load Balancers, distributing traffic, round-robin, least connections
- Databases at scale: sharding, replication, read replicas
- SQL vs. NoSQL selection framework, when each is right
- Caching strategies: Redis basics, cache-aside, write-through, TTL
- Content Delivery Networks (CDN), how your assets go global
- Microservices vs. Monolith, honest trade-offs, not just hype
- Docker and containerization, packaging apps for consistent deployment
- Case studies: Design Twitter, Design YouTube, Design WhatsApp, methodically

Projects built:

- System Design Document, write a complete design document for a URL shortener at scale
- Redis Caching Layer, add Redis caching to your MERN capstone app
- Docker Compose Setup, containerize your full MERN stack with Docker Compose

Practice: Give Cursor Agent or Claude Code one scoped multi file task in your capstone repo, then audit the output and document what you kept, fixed or rejected.

Week 10: Cloud, DevOps & Security for AI-Generated Code

Topics:

- AWS core services: EC2, S3, RDS, CloudFront, Route 53
- Deploying a Node.js app on EC2 with Nginx reverse proxy
- AWS S3, storing and serving static assets and user uploads at scale
- SSL/TLS certificates, HTTPS with Let's Encrypt
- CI/CD pipelines with GitHub Actions, test, build, deploy automatically
- Environment management, dev, staging, production environments
- CRITICAL: Security risks of vibe-coded apps, 45% of AI-generated code contains OWASP vulnerabilities (Wiz Security study)
- Web Application Security: OWASP Top 10 for developers, with specific AI-generated code examples
- SQL injection, XSS, CSRF in AI-generated code, what AI commonly gets wrong
- Security audit workflow: using OWASP ZAP and Snyk to scan vibe-coded apps
- Rate limiting, IP blocking, and DDoS basics
- Secrets management, never commit credentials to GitHub (AI tools often leak .env files)
- Application monitoring with Datadog / Sentry (free tiers)
- The responsible vibe coder checklist: security review before every deployment

Projects built:

- Production Deployment: Deploy your MERN capstone with HTTPS on AWS EC2 via Nginx
- Automated CI/CD: Full GitHub Actions pipeline, lint, test, deploy on merge to main
- Security Audit: Run OWASP ZAP against your app and fix at least 3 vulnerabilities

Practice: Deploy your MERN app with HTTPS on EC2 or a comparable host, and add a GitHub Actions pipeline that runs tests before every deploy.

Week 11: Placement Preparation & Professional Identity

Topics:

- The placement roadmap, on-campus and off-campus strategy for 2026
- Resume that gets shortlisted, ATS optimization, STAR format, quantification
- GitHub portfolio for job seekers, README, contributions, and credibility
- LinkedIn optimization: headline, about section, skills endorsements
- Behavioural interview prep: STAR method for tech companies
- Technical interview process, online assessment, DSA round, system design, HR
- DSA rapid revision, top 20 most frequently asked interview questions
- Mock Interview Session 1: Full 45-minute technical mock with feedback
- Mock Interview Session 2: System design mock, 30 minutes
- Negotiating your offer, salary benchmarks for fresher SDE roles India
- Freelancing as a college developer, finding clients, pricing, contracts
- Open source strategy, how meaningful contributions open career doors

Projects built:

- Professional Portfolio Website v2, updated with all 12-week projects, live, SEO-optimized
- Optimized Resume, ATS-ready, reviewed by an industry mentor
- GitHub Portfolio Cleanup, all key repos have great READMEs, demo links, and screenshots

Practice: Finish an ATS friendly one page resume, refresh your GitHub profile README and rewrite your LinkedIn headline, then get feedback from one mentor or peer.

Week 12: Grand Capstone Launch, Showcase, Certify & Conquer

Topics:

- Grand Capstone final polish, code review, performance optimization
- Writing a technical blog post about your capstone project (Medium / Dev.to)
- SEO optimization for your portfolio, getting found by recruiters
- Open source contribution: submit a PR to a real open-source project
- The modern developer identity: traditional coding + vibe coding + agentic engineering
- Your vibe coding toolkit mastery recap: Cursor, Claude Code, Windsurf, Copilot, Lovable, Bolt.new, v0.dev
- The developer mindset, growth, continuous learning, imposter syndrome
- Building in public, Twitter/X, LinkedIn, community engagement
- Advanced paths beyond Vibe Coding: Competitive Programming, ML research, DevOps, AI

Engineering

- Modern Age Coders Alumni Network, joining and giving back
- Capstone presentations, live demo to a panel of industry judges
- Project feedback from industry professionals
- Receiving the Vibe Coder, College Level Certification
- Next steps: internships, placements, freelancing, and startups

Projects built:

- GRAND CAPSTONE, 'Production-Grade Vibe Project': A complete, full-stack (MERN) OR ML-powered application built using a professional vibe coding workflow (Cursor IDE + Claude Code + AI app builders for prototyping). Must include: user authentication, real data, API integrations, responsive UI, full cloud deployment, CI/CD pipeline, security audit (AI-generated code reviewed for OWASP vulnerabilities), documented on GitHub, and presented live to a panel of industry mentors with a walkthrough of both the product AND the AI-assisted development process

Assessment: Grand Capstone: Live Demo Code Review System Design Q&A Panel Feedback

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 8+ MERN stack projects, REST APIs, React apps, full-stack authenticated apps
- Phase 2 (Weeks 5-8): 6+ DSA visualizers + 3 ML models trained, evaluated & deployed
- Phase 3 (Weeks 9-12): System design documents, Docker/AWS deployments, Grand Capstone

Total projects: 15+ production-grade projects + 100+ DSA problems solved

SKILLS THE STUDENT WILL HAVE AT THE END

- Frontend: React 18, React Router, TanStack Query, Tailwind CSS, Vite, v0.dev
- Backend: Node.js, Express.js, REST API design, JWT auth, bcrypt, Multer
- Database: MongoDB, Mongoose, schema design, aggregation, indexing
- Vibe Coding: Cursor IDE (Composer + Agent), Claude Code, Windsurf, GitHub Copilot, Lovable, Bolt.new
- DSA: Arrays, Strings, Linked Lists, Trees, Graphs, DP, Greedy, 100+ LeetCode problems
- Machine Learning: scikit-learn, pandas, TensorFlow/Keras, Hugging Face, model deployment
- DevOps: Git, GitHub Actions, Docker, AWS EC2/S3, Nginx, CI/CD pipelines
- System Design: load balancing, caching, Redis, microservices, database scaling
- Security: OWASP Top 10, AI-generated code auditing, rate limiting, secrets management

HOW A TYPICAL WEEK IS SPLIT

Live sessions: 2 sessions (2 hours each)
Dsa practice: 3-4 hours (10+ problems/week)
Project work: 4-6 hours
Total per week: 10-15 hours

SUPPORT INCLUDED

Mentorship: Code reviews by industry-experienced mentors
Mock interviews: Technical mock interviews (DSA + System Design) with feedback
Placement support: Resume reviews, LinkedIn optimization, job referral channels
Community: Access to Modern Age Coders alumni network of placed students
Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Final certificate: Vibe Coder, College Level: Full Stack & AI Engineer (Modern Age Coders Certified)
Capstone required: Production-grade MERN or ML app with auth, cloud deployment, CI/CD, and security audit
Dsa milestone: 100+ LeetCode problems solved with documented approaches
Portfolio: Professional developer portfolio + optimized GitHub + LinkedIn

WHERE THIS CAN LEAD

- Full Stack Web Developer (Rs 6-18 LPA)
- Frontend Developer, React Specialist (Rs 7-20 LPA)
- Backend Developer, Node.js/Python (Rs 8-22 LPA)
- Junior Machine Learning Engineer (Rs 10-25 LPA)
- Software Development Engineer (SDE) at product companies
- Summer Internships at tech startups and MNCs
- Freelance Full Stack Developer (Rs 500-2,500/hour)
- AI/Vibe Coding Consultant, building MVPs for clients with AI tools

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.

Experienced roles: Rs 15-25 LPA (with ML specialization or senior full-stack role)
Other roles: Rs 6-12 LPA (SDE-1 at product companies and MNCs); Rs 10-18 LPA (with strong portfolio and MERN skills); \$40k, 100k USD (remote roles with international companies)
Freelance: Rs 500-2,500/hour based on project complexity and specialization

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Vibe Coding for College?

A: Vibe Coding for College is a 3-month, intensive software engineering program for college students and fresh graduates (18+). It uniquely combines deep engineering fundamentals with professional AI development tools: you master MERN stack using Cursor IDE and Claude Code, build rapid prototypes with Lovable and Bolt.new, learn DSA for placements, Machine Learning with Python, System Design, and security for AI-generated code, all taught live by industry-experienced mentors at Modern Age Coders.

Q: Who is this course for?

A: This course is for college students (any year) and recent graduates who know at least basic programming and want to break into software engineering. It's ideal for CS/IT/MCA students, but engineering students from any branch who are serious about tech can join. You should have basic knowledge of at least one programming language (Python, Java, C++, or JavaScript).

Q: What is the MERN stack and why learn it?

A: MERN stands for MongoDB (database), Express.js (backend framework), React.js (frontend library), and Node.js (JavaScript runtime). It is the most popular and in-demand full-stack technology combination for 2026, used by top startups and tech companies worldwide. MERN skills are specifically sought in Indian product companies, MNCs, and international remote roles.

Q: How does DSA preparation fit into this course?

A: DSA is integrated throughout all 12 weeks rather than crammed at the end. Weeks 5-6 are dedicated DSA weeks, but we recommend 10 problems per week from Week 1 onwards through our parallel preparation track. By Week 12, students have typically solved 100-150 problems across LeetCode, GeeksforGeeks, and Codeforces.

Q: Will studying 3 months really prepare me for placements?

A: Three months of focused, structured preparation, building real projects, solving 100+ DSA problems, and doing mock interviews, is absolutely transformative. Join as early in your college career as possible: students who take this course in their 2nd or 3rd year have maximally more time to practice and build. Our alumni have been placed at companies including Infosys, TCS, Wipro, Freshworks, Juspay, and international remote companies.

Q: What does 'Vibe Coding' mean at the college level?

A: Vibe Coding, coined by Andrej Karpathy in 2025, means building software by describing what you want and letting AI generate the code. At the college level, this evolves into 'agentic engineering': using professional AI tools (Cursor IDE, Claude Code, Windsurf, GitHub Copilot) alongside AI app builders (Lovable, Bolt.new, v0.dev) to build production-grade software at startup speed. But the key differentiator is that you also learn to REVIEW, SECURE, and SCALE what AI generates, because 45% of AI-generated code contains security vulnerabilities. You graduate as a developer who can build with AI AND without it.

Q: Do you help with interview and career preparation?

A: Week 11 is dedicated to career preparation: resume reviews, LinkedIn optimization, mock technical interviews, system design practice, and DSA rapid revision. You also get access to our alumni network and mentor panels. We prepare you thoroughly for interviews and applications, but we do not guarantee placement.

Q: What is the time commitment?

A: We recommend 10-15 hours per week: 2 live sessions (2 hours each), weekly DSA practice (3-4 hours), and project work (4-6 hours). This is an intensive program. The more you invest, the stronger your portfolio and placement readiness will be.

Q: Is Machine Learning part of this course?

A: Yes. Weeks 7 and 8 cover applied Machine Learning with Python, scikit-learn, regression, classification, clustering, neural networks with TensorFlow/Keras, and deploying ML models as APIs and web apps. This is applied/practical ML, not theory-heavy research ML.

Q: What certification will I receive?

A: This is a credential recognized by our industry partners and verifiable online.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/vibe-coding-for-college-fullstack-ai-dsa-career-course>

=====

Course link: <https://learn.modernagecoders.com/courses/vibe-coding-for-kids-beginners-ai-scratch-game-dev>
Subject area: Kids Coding & Creative Technology
Designed for: Kids
Who can join: Complete Beginner (Ages 8-12)
Course length: 3 months (12 weeks)
Time needed: 4-6 hours/week
Certificate: Vibe Coder, Kids Level Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online vibe coding course for kids ages 8-12: learn Scratch, AI tools and game development in a 3-month beginner program, no prior experience needed.

In one line: Where Imagination Meets Code, Build Games, Stories & AI Projects!

ABOUT THE PROGRAMME

Vibe Coding for Kids is the ultimate 3-month beginner coding journey designed specifically for children aged 8 to 12. 'Vibe Coding', a term coined by AI pioneer Andrej Karpathy in 2025, means telling a computer what you want to build in plain English and letting AI create the code for you. It's the future of how software gets made, and your child can start learning it now. Through playful, visual, and project-driven learning, kids discover the magic of creating their own games, animated stories, music with code, and real AI-powered apps, no complex typing required to get started! Kids begin with visual block coding (Scratch) to learn how computers think, then graduate to actual vibe coding using tools like Replit, Bolt.new, and Google Gemini, the same AI tools professionals use today. At Modern Age Coders, we believe every child is a natural creator. Our expert instructors blend visual block programming, creative exploration, and real AI app-building tools to make coding feel like play. By the end of 12 weeks, your child will have built 15+ original projects, including apps they built by talking to AI, and gained real digital superpowers.

WHAT MAKES THIS COURSE DIFFERENT

- 100% project-based, kids build something real every single week
- Age-appropriate content designed for 8-12 year olds
- No prior experience or technical knowledge needed
- Visual block coding (Scratch, Code.org), no complex syntax to start
- Real vibe coding: kids build apps by talking to AI (Replit, Bolt.new)
- Includes AI tools, music coding, and creative art projects
- Live classes with expert instructors + recorded replays
- Parent dashboard to track weekly progress
- Portfolio of 15+ completed projects at course end
- Fun, encouraging, and bully-free classroom environment

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation Vibes (Weeks 1-4): Scratch basics, animations, storytelling with code
Phase 2: Game Dev Vibes (Weeks 5-8): Game mechanics, score systems, interactive adventures
Phase 3: AI & Vibe Coding Vibes (Weeks 9-12): Real vibe coding with AI app builders (Replit, Bolt.new), AI tools, music coding, capstone project showcase

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 8-12 years old
Experience: Absolutely none, complete beginner friendly
Equipment: A computer or laptop with internet connection
Software: All tools are free and browser-based, nothing to install
Parental support: Parents encouraged to engage and review weekly projects

WHO THIS COURSE IS FOR

Curious kids: Children aged 8-12 who are curious about how games and apps are made
Creative kids: Kids who love drawing, storytelling, music, or building things
Screen time shift: Parents who want to turn screen time into productive learning time
Future ready: Families who want to give their child a head start in technology and AI
Struggling learners: Children with dyslexia, ADHD, or other learning differences, vibe coding removes syntax barriers

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION VIBES, SCRATCH & CREATIVE CODING (WEEKS 1-4)

Kids dive into the colourful world of Scratch and block-based programming. They create their first animations, interactive stories, and simple games while learning the core ideas of coding, loops, events, and conditionals, through hands-on fun.

Week 1: Welcome to Vibe Coding!, Your First Animations

Topics:

- What is Vibe Coding?, Telling computers what to build using your words (coined by AI scientist Andrej Karpathy in 2025)
- The two ways to code: Block Coding (Scratch) and Vibe Coding (talking to AI), we'll learn both!
- How computers think, understanding instructions step by step
- Introduction to Scratch 3.0, navigating the interface
- Sprites, stages, and backdrops, building your digital world
- Basic motion blocks: move, turn, glide
- Looks blocks: say, think, costumes, effects
- Sound blocks: play music, record your voice
- Your first animation: Make a character walk and talk
- Saving and sharing your Scratch project
- Online safety and being a kind digital citizen
- Setting up your Vibe Coder profile

Projects built:

- Animated Greeting Card, make a character say hello to the world
- Dancing Pet, a sprite that moves to your favourite music
- About Me Story, an interactive presentation about yourself

Practice: Build a mini dance party in Scratch where your sprite changes costume and moves each time you press an arrow key, then show someone at home.

Week 2: Storytelling with Code, Scenes, Dialogues & Adventures

Topics:

- Creating multi-scene stories with backdrop changes
- Broadcasting messages between sprites, making characters talk to each other
- Using the 'ask and answer' block for interactive dialogue
- Timing animations, wait blocks and synchronized stories
- Creating sprite costumes for character emotions
- Adding background music and sound effects
- Text-to-speech: making your characters speak in different voices
- Importing your own images and drawings as sprites
- Drawing custom sprites with the Scratch paint editor
- Story planning: beginning, middle, and end
- Sharing your story with family and friends

Projects built:

- A 3-Scene Adventure Story, hero, challenge, and victory
- Interactive Storybook, click to turn pages
- Talking Animal Show, two animals have a funny conversation

Practice: Create a two scene story where characters talk to each other using broadcast messages and ask the viewer one question with the ask block.

Week 3: Events & Interactivity, Making Things Happen!

Topics:

- Event blocks: when green flag clicked, when key pressed, when sprite clicked
- Keyboard controls, making characters move with arrow keys
- Mouse events, click-to-activate actions
- Creating clickable buttons and navigation menus
- Introduction to conditionals: If-Then logic blocks
- Touching and color detection, collision basics
- Creating interactive art, drawing with mouse movements
- Building a virtual musical keyboard
- Combining multiple events for complex interactions
- User input with 'ask' and storing answers
- Debugging basics: What to do when something goes wrong

Projects built:

- Virtual Piano, play notes by pressing keyboard keys
- Click-and-Colour Art Canvas, draw with code
- Interactive Quiz, 5 questions with right/wrong feedback

Practice: Make a clickable pet that reacts differently to a click, a key press and the green flag, using at least one if then block.

Week 4: Loops & Patterns, The Magic of Repetition

Topics:

- Forever loops, making things go on and on
- Repeat loops with a specific number of times
- Nested loops, loops inside loops for complex patterns
- Drawing geometric shapes with pen blocks
- Creating spirals, stars, and kaleidoscopes with loops
- The Stamp tool, making repeating patterns
- Color effects and graphic transformations
- Thinking like a computer: breaking big tasks into small loops

- Efficiency, doing more with less code
- Creating a looping background animation
- Phase 1 Review & Project Planning

Projects built:

- Geometric Art Machine, generates beautiful patterns automatically
- Spinning Galaxy, looping space animation
- Pattern Puzzle, create a design and challenge friends to recreate it

Practice: Draw a star, a spiral and one invented pattern with pen blocks inside repeat loops, and name your favorite creation.

----- PHASE 2: GAME DEV VIBES, BUILD YOUR OWN GAMES! (WEEKS 5-8) -----

Now the real fun begins! Kids learn the building blocks of game development: scoring, lives, enemies, levels, and game loops. By the end of Phase 2, every child will have built a complete, playable video game from scratch.

Week 5: Variables & Score Systems, 'You Win!' Moments

Topics:

- What are variables? Understanding data storage in simple terms
- Creating score, lives, and timer variables
- Changing variable values, adding points, losing lives
- Using random numbers, the element of surprise in games
- Countdown timers and time-based challenges
- Displaying variables on screen as a score HUD
- Win conditions: What happens when you reach a goal?
- Game Over conditions: Lives system and health bars
- High score saving basics
- Creating power-ups that boost your score
- Playtesting your games, the 'fun factor' check

Projects built:

- Catch the Falling Stars, catch sprites to earn points
- Number Guessing Game, with hints and score tracking
- Speed Challenge, collect 10 items before the timer runs out

Practice: Add a score, lives and a countdown timer to a simple catch the apple game and play test it three times.

Week 6: Collision Detection & Game Mechanics, Real Game Feels

Topics:

- Touching detection, sprites interacting with each other
- Color collision, detecting when you touch a wall or obstacle
- Gravity simulation using loops and variables
- Jumping mechanics, platformer physics basics
- Enemy movement, creating obstacles that move and follow
- Projectile mechanics, shooting and throwing
- Boundary detection, keeping sprites inside the screen
- Creating a scrolling background effect
- Power-up pickups and damage effects
- Sound effects for collisions, wins, and game over
- Making your game feel satisfying, game juice basics

Projects built:

- The Endless Runner, dodge obstacles as you run
- Space Invaders Jr., shoot incoming aliens
- Trampoline Jump, bounce as high as you can game

Practice: Give your game character a gravity powered jump and one moving enemy that ends the game on touch.

Week 7: Level Design & Multi-Screen Games, The Full Experience

Topics:

- What makes a great game level? Flow, challenge, and reward
- Designing level 1, level 2, and level 3 with increasing difficulty
- Using backdrops as levels, switching on game events
- Creating boss levels and final challenges
- Save and load game states using cloud variables
- Cutscenes and story sequences between levels
- Sound design, choosing music for different game moods
- UI design: Start screen, game over screen, victory screen
- Multiple character selection screens
- Accessibility in games, making your game easy to understand
- Beta testing with classmates and collecting feedback

Projects built:

- 3-Level Maze Adventure, escape each dungeon to advance
- Fruit Ninja Clone, 3 difficulty levels with progressive speed
- Boss Battle Game, survive the final boss to win

Practice: Design level 1 and a harder level 2 for your game using two backdrops that switch when the player wins.

Week 8: Complete Game Jam, Build Your Dream Game!

Topics:

- Game design document, planning your game on paper first
- Asset creation, custom sprites, backgrounds, and sounds
- Implementing full game loop: start, play, lose, win
- Bug hunting and debugging strategies
- Game polishing, making it look professional
- Writing game instructions for players
- Creating a game thumbnail and cover art
- Publishing your game to the Scratch community
- Presenting your game to classmates
- Getting featured on the Modern Age Coders showcase
- Reflecting on your journey from Week 1 to Week 8

Projects built:

- PHASE 2 CAPSTONE, My Dream Game: A complete, original game with: custom sprites, 2+ levels, score system, sound effects, start and end screens

Assessment: Phase 2 Game Development Assessment, completed game review with instructor feedback

PHASE 3: AI & REAL VIBE CODING, BUILD APPS BY TALKING TO AI! (WEEKS 9-12)

In the final phase, kids experience the real magic of vibe coding, they train their own machine learning models, build real apps by describing ideas to AI tools like Replit and Bolt.new, create music with code, design with AI, and build their capstone 'Vibe Showcase' project. This is where kids learn the skill that Andrej Karpathy says will define the future: telling AI what to build and watching it come to life.

Week 9: AI for Kids, Teaching Computers to See, Hear & Build!

Topics:

- What is Artificial Intelligence? A fun, simple explanation
- What is Vibe Coding?, The story of Andrej Karpathy and how AI writes code for you
- The vibe coding loop: Describe AI Builds You Test You Improve
- Machine Learning, teaching computers by showing examples
- Google Teachable Machine: train your first image model
- Training an AI to recognize your hand gestures
- Training an AI to recognize sounds
- Using your trained model in a Scratch project
- Introduction to Google Gemini, chatting with AI safely
- How to write good prompts, being clear and specific with AI
- What is bias in AI? Why AI needs diverse data
- Machine Learning for Kids (ML4Kids) platform
- AI ethics for kids, being responsible with technology
- The future of AI, jobs, creativity, and possibilities

Projects built:

- Rock Paper Scissors AI, a computer that plays against you using real ML
- Emotion Detector, AI that reads your facial expression
- AI Chat Experiment, ask Google Gemini to explain your favourite topic and fact-check its answers

Practice: Train a Teachable Machine model to tell two hand signs apart using ten photos of each, then test it and count how many it gets right.

Week 10: Vibe Coding Your First Apps, Build by Talking to AI!

Topics:

- Welcome to real vibe coding!, building apps by describing them in English
- Introduction to Replit, your browser-based coding playground
- Replit Agent, tell it what to build and watch it create your app live
- Writing great prompts: being specific about what you want (colours, buttons, features)
- Building your first vibe-coded app: a simple calculator
- Introduction to Bolt.new, type what you want, see it instantly
- Building a game with Bolt.new, describe the rules, AI codes it
- Testing your AI-built app, does it do what you asked?
- Fixing problems by telling AI what went wrong (iterating)
- Understanding the code AI wrote, reading what it made for you
- Comparing: Scratch (you drag blocks) vs. Vibe Coding (you describe in words)
- When AI makes mistakes, learning to spot and fix errors

Projects built:

- My Calculator App, tell Replit Agent to build a colourful calculator and customize it
- Quiz Game App, describe a quiz game to Bolt.new with topics you love
- Vibe-Coded Story App, describe an interactive story app and watch AI build it

Practice: Describe a tiny app to Replit Agent, like a compliment generator, then ask it for two improvements: a new color and an extra button.

Week 11: Music Coding, AI Art & Portfolio, Create Like a Pro

Topics:

- Introduction to Sonic Pi, coding music like a DJ
- Notes, beats, and rhythms in code, loops for musical patterns
- Live coding music, composing a 30-second original track
- Using AI to help compose: describe a mood, get a melody suggestion
- Introduction to AI image generation for kids (safe, educational tools)
- Design principles for kids: colour, space, balance, contrast

- Canva for Kids, templates, elements, and publishing
- Creating game covers and posters for your projects
- Vibe coding a portfolio: use Bolt.new to build your own showcase page
- Adding your Scratch games and vibe-coded apps to your portfolio
- Presentation design, making your showcase slides look amazing
- Building your Digital Portfolio, showcasing all 12 weeks of work

Projects built:

- My Vibe Track, compose an original beat in Sonic Pi
- Vibe-Coded Portfolio Website, tell Bolt.new to create a page showcasing your best projects
- Game Cover Design, a professional-looking cover for your dream game in Canva

Practice: Compose a 30 second Sonic Pi loop with two different beats and generate one AI artwork to be its album cover.

Week 12: Grand Vibe Showcase, Your Moment to Shine!

Topics:

- Finalizing your capstone Vibe Project, choosing your best idea
- Combining everything: block coding, vibe coding, AI, design, and music
- Project polishing and bug-free testing
- Writing a project description, SEO basics even for kids!
- Recording a demo video of your project
- Creating a 2-minute presentation pitch
- Peer review, giving kind and helpful feedback
- Receiving your Vibe Coder Kids Certification
- Sharing your work on the Modern Age Coders Hall of Fame
- What's next?, Preview of Vibe Coding for Teens
- Celebration and graduation ceremony

Projects built:

- GRAND CAPSTONE 'Vibe Project': An original multi-element creation combining Scratch game OR a vibe-coded app (built with Replit/Bolt.new), AI model, original music, and Canva design, presented live to parents and peers

Assessment: Final Capstone Assessment + Vibe Coder Kids Certification awarded

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 9+ Scratch animations, stories & interactive projects
- Phase 2 (Weeks 5-8): 9+ games including Dream Game capstone
- Phase 3 (Weeks 9-12): 6+ AI, vibe coding & creative projects + Grand Capstone

Total projects: 15+ creative projects + Grand Vibe Showcase Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Block Coding: Scratch 3.0, sprites, backdrops, motion, looks, sound blocks
- Programming Logic: loops, conditionals, events, variables, collision detection
- Game Development: score systems, lives, levels, enemy AI, game physics
- Vibe Coding: Replit Agent, Bolt.new, building apps by describing in English
- AI & Machine Learning: Google Teachable Machine, ML4Kids, Google Gemini
- Music Coding: Sonic Pi, beats, loops, live coding
- Digital Design: Canva, portfolio building, presentation skills
- Prompt Engineering: writing clear instructions for AI tools

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (1.5 hours each)

Independent projects: 1-3 hours

Vibe challenge: 1 fun weekly challenge

Total per week: 4-6 hours

SUPPORT INCLUDED

Live instruction: Expert-led live classes with recorded replays

Parent dashboard: Weekly progress tracking for parents

Age appropriate: Content designed specifically for ages 8-12

Safe environment: Fun, encouraging, and bully-free classroom

Community: Access to Modern Age Coders kids community

CERTIFICATION DETAIL

Final certificate: Vibe Coder, Kids Level (Modern Age Coders Certified)

Capstone required: Grand Vibe Showcase project combining coding, AI, music & design

Portfolio: 15+ completed projects showcased in digital portfolio

WHERE THIS CAN LEAD

- Digital Creator & Storyteller
- Junior Game Designer
- Creative Technologist
- AI Explorer
- Ready for Vibe Coding for Teens program

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Vibe Coding for Kids?

A: Vibe Coding for Kids is a 3-month, project-based program at Modern Age Coders designed for children aged 8-12. 'Vibe Coding' means building apps and software by describing what you want in plain English and letting AI create the code, a term coined by AI pioneer Andrej Karpathy in 2025 (Collins Dictionary Word of the Year!). Kids start with Scratch to learn coding fundamentals, then graduate to real vibe coding using AI tools like Replit, Bolt.new, and Google Gemini to build their own apps.

Q: What age is this course designed for?

A: This course is specifically designed for children aged 8 to 12. The content, pace, and teaching style are all optimized for this age group, with age-appropriate projects and language.

Q: Does my child need any prior coding experience?

A: Absolutely not! Vibe Coding for Kids is a complete beginner program. We start from zero, no prior coding or technical knowledge is required. All you need is curiosity and a computer!

Q: What tools and platforms will my child use?

A: Your child will use Scratch 3.0, Code.org, Replit (AI-powered browser coding), Bolt.new (AI app builder with instant preview), Google Gemini (AI chat), Google Teachable Machine, Sonic Pi (music coding), Canva, and ML4Kids. All of these are free, safe, and browser-based, nothing to install. These are the same AI tools professionals use for vibe coding in 2026.

Q: How many hours per week does the course require?

A: We recommend 4-6 hours per week, which includes 2 live classes (1.5 hours each) plus 1-3 hours of independent project time. All classes are recorded so your child never misses a session.

Q: What will my child have built by the end of the course?

A: By the end of 12 weeks, your child will have built 15+ projects including animated stories, original video games, a trained AI model, apps built using real vibe coding (AI app builders), original music tracks, their own portfolio website, and a full digital portfolio, all showcased at a Grand Vibe Showcase event.

Q: Is there a certificate at the end?

A: Yes! Every student who completes the course and their capstone project receives the official 'Vibe Coder, Kids Level' certification from Modern Age Coders.

Q: What comes after this course?

A: Upon completing Vibe Coding for Kids, students are ready for the Vibe Coding for Teens program, where they learn real programming languages (Python, JavaScript), build apps, and explore advanced AI, building directly on the foundation from this course.

To enrol or to book a free demo for this course, message Shivam Sir on +91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/vibe-coding-for-kids-beginners-ai-scratch-game-dev>

=====

COURSE 117 OF 118: VIBE CODING FOR TEENS: PYTHON, WEB DEV & AI (AGES 13-17)

=====

Course link: <https://learn.modernagecoders.com/courses/vibe-coding-for-teens-python-web-ai-projects-course>

Subject area: Teen Coding & Web Development

Designed for: Teens

Who can join: Beginner to Intermediate (Ages 13-17)

Course length: 3 months (12 weeks)

Time needed: 6-8 hours/week

Certificate: Vibe Coder, Teens Level Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online vibe coding course for teens 13-17: learn Python, HTML, CSS, JavaScript and AI projects, build real apps and a portfolio in a 3-month program.

In one line: Stop Scrolling. Start Building. Your Code. Your World.

ABOUT THE PROGRAMME

Vibe Coding for Teens is the definitive 3-month coding program for ages 13-17 who want to go

from beginner (or block-coding graduate) to building actual websites, Python apps, and AI-powered projects using the most cutting-edge tools in tech. 'Vibe Coding', coined by AI pioneer Andrej Karpathy in 2025 (Collins Dictionary Word of the Year!), means building software by describing what you want in natural language and letting AI generate the code. But here's the secret the pros know: the best vibe coders are also great traditional coders. That's why this course teaches you BOTH, you'll learn Python and JavaScript fundamentals AND master AI-powered development tools like Cursor IDE, GitHub Copilot, v0.dev, Bolt.new, and Lovable. At Modern Age Coders, we've designed this curriculum around what teens actually want to build: apps that solve real problems, games they can share with friends, websites that look professional, and AI tools that feel like the future. You'll use the same tools that professional developers use every day, not toy platforms. Graduate with a portfolio of 12+ real projects that proves you can code, with AND without AI assistance.

WHAT MAKES THIS COURSE DIFFERENT

- Real languages: Python and JavaScript, not toy platforms
- Real vibe coding tools: Cursor IDE, GitHub Copilot, v0.dev, Bolt.new, Lovable
- Learn BOTH ways: traditional coding fundamentals + AI-assisted vibe coding
- Project-first approach, you build something real every single week
- Portfolio focus, graduate with 12+ projects for college/job applications
- Industry-relevant tools: Cursor, VS Code, Git, GitHub, APIs
- AI-integrated projects, use and build AI tools
- Prompt engineering, learn to communicate with AI effectively
- Peer coding sessions, collaborate like real dev teams
- 1-on-1 code reviews with expert instructors
- Access to the Modern Age Coders teen developer community
- Hackathon preparation and coding competitions support

THE LEARNING PATH, STEP BY STEP

Phase 1: Python Vibes (Weeks 1-4): Python fundamentals with GitHub Copilot, functions, games, and automation scripts

Phase 2: Web Dev Vibes (Weeks 5-8): HTML, CSS, JavaScript + vibe coding with v0.dev and Bolt.new, build responsive websites and web apps

Phase 3: AI & Vibe Coding Mastery (Weeks 9-12): Cursor IDE, AI-powered development, Lovable for app building, final portfolio, deployment & showcase

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 13-17 years old

Experience: Complete beginner friendly, no prior coding needed (Vibe Coding for Kids graduates will move faster)

Equipment: Computer or laptop (Windows, Mac, or Linux) with stable internet

Software: All tools are free, Cursor IDE, VS Code, GitHub Copilot (free for students), Python, Git

Motivation: Curiosity and willingness to build real projects every week

WHO THIS COURSE IS FOR

Complete beginners: Teens aged 13-17 with zero coding experience who want to start building real projects

Scratch graduates: Kids who completed block coding and want to learn real programming languages

Aspiring developers: Teens who dream of building their own apps, games, or websites

College prep: Students who want a strong portfolio for CS/tech college applications

Hackathon enthusiasts: Teens interested in coding competitions, hackathons, and tech clubs

Creative builders: Anyone who wants to bring ideas to life using code and AI

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

----- PHASE 1: PYTHON VIBES, THINK LIKE A DEVELOPER (WEEKS 1-4) -----

Teens dive deep into Python, the world's most popular and versatile programming language. From variables and loops to building actual games and automation tools, Phase 1 transforms a complete beginner into a confident Python programmer.

Week 1: Python Unlocked, Your Developer Setup & First Programs

Topics:

- Why Python? The language of AI, data, and the web
- What is Vibe Coding?, Andrej Karpathy's 2025 concept that changed software forever
- The vibe coding workflow: Describe AI Generates You Review Iterate Ship
- Setting up VS Code & Cursor IDE, your professional AI-powered developer environment
- Setting up GitHub Copilot (free for students!), your AI pair programmer
- Python basics: print(), variables, strings, integers, floats
- String manipulation, slicing, formatting, f-strings
- Getting user input with input() function

- Basic arithmetic and math operations in Python
- Boolean logic: True, False, and comparisons
- Your first command-line programs
- Understanding error messages, the developer's best friend
- Running Python files from the terminal
- Introduction to Git, saving your code like a pro
- Creating your GitHub profile and first repository

Projects built:

- Personal Bio Generator, asks 5 questions and prints a formatted story about you
- Simple Calculator, performs all 4 operations with user input
- Mad Libs Creator, fill-in-the-blank story generator

Practice: Set up your editor, write a Python script that asks the user three questions and prints a profile card, then commit it to a new GitHub repo.

Week 2: Flow Control, Making Smart Programs

Topics:

- If, elif, else, decision-making in code
- Nested conditionals for complex logic
- While loops, repeating until a condition is met
- For loops, iterating over sequences
- Range() function, generating number sequences
- Loop control: break, continue, pass
- Lists, storing multiple values in one variable
- List methods: append, remove, sort, reverse, index
- Tuples and when to use them
- String loops, processing text character by character
- Nested loops for patterns and grids
- Debugging with print() statements and VS Code debugger

Projects built:

- Number Guessing Game, with hints, attempts counter, and high score
- Times Table Generator, print any multiplication table on demand
- Password Strength Checker, analyses a password and gives a strength rating

Practice: Build a number guessing game with a while loop and if elif else hints for higher or lower, counting the attempts taken.

Week 3: Functions & Modules, Writing Clean, Reusable Code

Topics:

- Defining functions with def keyword
- Parameters and arguments, passing data to functions
- Return values, getting data back from functions
- Default parameters and keyword arguments
- Scope: local vs. global variables
- Importing Python's built-in modules: math, random, datetime, os
- The random module, creating unpredictable programs
- Working with the datetime module for time-based features
- Writing your own modules and importing them
- Docstrings, documenting your functions properly
- Lambda functions, quick, anonymous functions
- Recursion basics, functions that call themselves

Projects built:

- Dice Rolling Simulator, roll any number of sides, track statistics
- Tip Calculator, restaurant bill splitter with formatted output
- Word Counter, analyses a paragraph for word count, unique words, and top 5 most used

Practice: Refactor your guessing game into functions with parameters and return values, then add a play again option.

Week 4: Data Structures & File Handling, Real Persistent Apps

Topics:

- Dictionaries, key-value pairs for structured data
- Dictionary methods: get, keys, values, items, update
- Nested dictionaries, representing complex real-world data
- Sets, unique collections and set operations
- List comprehensions, Pythonic one-line transformations
- Dictionary comprehensions
- Reading and writing text files with open()
- File modes: r, w, a, r+ explained
- Working with CSV files, reading datasets
- JSON files in Python, the language of the web
- Error handling with try, except, finally
- Building a simple contact book, CRUD operations

Projects built:

- Contact Manager, add, search, update, delete contacts saved to a file
- Student Grade Tracker, stores grades in JSON and calculates averages
- Vocabulary Flash Card App, add words, quiz yourself, track score

Practice: Create a contact book app that stores entries in a dictionary and saves them to a file so they survive a restart.

Now teens build things the whole world can see. Phase 2 covers HTML, CSS, and JavaScript, the core technologies of every website on the internet. By Week 8, students will have built fully responsive, beautifully designed websites deployed live on the internet.

Week 5: HTML, Structure is Everything

Topics:

- How the web works, browsers, servers, HTTP basics
- HTML document structure: DOCTYPE, html, head, body
- Semantic HTML: header, nav, main, section, article, footer
- Text elements: h1-h6, p, strong, em, blockquote
- Links and navigation: anchor tags, href, relative vs. absolute URLs
- Images: img tag, alt text for accessibility and SEO
- Lists: ordered, unordered, nested lists
- Tables, displaying structured data
- HTML Forms: input types, labels, buttons, textareas
- HTML attributes: id, class, style
- Accessibility basics: ARIA roles, alt text, heading hierarchy
- Vibe Coding HTML: using v0.dev to generate page layouts from descriptions
- Comparing hand-written HTML vs AI-generated HTML, understanding what AI creates
- Validating your HTML with W3C Validator

Projects built:

- Personal Portfolio Page v1, bio, skills, and contact section in pure HTML
- Recipe Website, multi-page site with ingredients and instructions
- Event Landing Page, HTML structure for a fictional school event

Practice: Hand code a semantic personal profile page with a header, nav, three sections and a contact link, without any AI assistance.

Week 6: CSS, Make It Look Amazing

Topics:

- CSS syntax: selectors, properties, values
- The Box Model: margin, padding, border, content
- CSS colours: hex, RGB, HSL, and CSS variables
- Typography: fonts, Google Fonts integration, font-size, line-height
- CSS Flexbox, modern, powerful layout system
- CSS Grid, two-dimensional layout mastery
- Responsive design with Media Queries
- Mobile-first design principles
- CSS animations and transitions
- Pseudo-classes: hover, focus, active, nth-child
- CSS custom properties (variables) for design systems
- Dark mode implementation with CSS
- Vibe Coding CSS: using v0.dev and Bolt.new to generate beautiful UI components from text prompts
- Reading AI-generated Tailwind CSS, understanding utility-first styling

Projects built:

- Fully Styled Portfolio, transform Week 5's HTML into a beautiful, responsive portfolio
- Product Card Component, an Instagram-worthy card with hover effects
- Responsive Navigation Bar, a professional nav that collapses on mobile

Practice: Style your profile page with Flexbox, a Google Font and CSS variables, and make sure it still looks right at phone width.

Week 7: JavaScript, Bring Your Website to Life

Topics:

- What is JavaScript and why it makes the web interactive
- JS basics: variables (let, const), data types, operators
- Functions in JavaScript, declarations, expressions, arrow functions
- Arrays and array methods: map, filter, reduce, forEach
- Objects in JavaScript, structuring real-world data
- The DOM, Document Object Model explained simply
- Selecting elements: getElementById, querySelector
- Changing content and styles with JavaScript
- Event listeners: click, mouseover, input, submit
- Form validation with JavaScript
- Fetch basics, getting data from the internet
- Local Storage, persisting data in the browser

Projects built:

- Interactive To-Do App, add, complete, and delete tasks (persists with LocalStorage)
- Quiz App, 10-question quiz with timer, score, and results screen
- Real-time Character Counter, live word/character counts as you type

Practice: Add a dark mode toggle and a live character counter to your page using DOM events, plus one array method such as map or filter.

Week 8: APIs & Full Web Projects, Connect to the Real World

Topics:

- What is an API? REST APIs explained with real examples
- Fetch API, making HTTP GET requests in JavaScript
- Async/await, handling asynchronous operations cleanly
- JSON parsing, working with API response data

- Error handling in fetch requests
- Public APIs: Weather, Quotes, Space, Pokemon, SpaceX
- Displaying API data dynamically in your webpage
- API keys, working with authenticated APIs safely
- Introduction to the OpenWeather API
- Deploying your website with Netlify, live in 60 seconds
- Custom domain basics
- Vibe Coding a full web app: using Bolt.new to build and deploy a project in minutes
- Comparing your hand-coded projects vs vibe-coded projects, understanding the trade-offs
- Phase 2 portfolio review, making your projects job-ready

Projects built:

- Live Weather App, fetch real weather data for any city using OpenWeather API
- Random Quote Machine, fetches and displays random motivational quotes
- Space Explorer, displays real NASA photo of the day with Astronomy API

Practice: Fetch data from a free public API, such as weather or jokes, and render it on your page with async await and visible error handling.

----- PHASE 3: AI & VIBE CODING MASTERY, BUILD YOUR FUTURE (WEEKS 9-12) -----

The most exciting phase, teens master real vibe coding tools (Cursor IDE, Lovable, Claude/Gemini APIs), combine Python + Web skills to build AI-powered projects, set up their professional developer portfolio, and prepare for real-world opportunities: internships, hackathons, and competitive coding. This is where you become a true vibe coder, someone who can build anything by combining traditional coding skills with AI-powered development.

Week 9: AI Tools & Vibe Coding Mastery, Build with AI Like a Pro

Topics:

- Introduction to AI APIs, using AI as a superpower
- Working with the Google Gemini API (or Claude API), setup and authentication
- Chat Completion API, building a simple AI chatbot
- System prompts, guiding AI behaviour for your use case
- Deep dive into Cursor IDE, the #1 AI-powered code editor for vibe coding
- Cursor Composer: describe a feature in English Cursor writes the code across multiple files
- Cursor Tab: AI auto-completes your code as you type
- GitHub Copilot Chat, asking your AI pair programmer questions about your code
- Prompt engineering for code: writing clear, specific prompts that get better results
- AI for text: summarization, sentiment analysis, translation
- Image generation API basics (DALL-E / Stable Diffusion)
- Comparing AI models: Claude Sonnet (best for code), GPT-4, Gemini, which to use when
- AI ethics for teen developers, bias, transparency, and responsibility

Projects built:

- AI Study Buddy, paste any topic, get an explanation in simple language (Python + Gemini API)
- Mood Journal Analyser, enter your journal entry, AI analyses the mood and gives advice
- Vibe-Coded AI App, use Cursor Composer to build an entire AI tool from a single description

Practice: Build a small chatbot on the Gemini or Claude API with a system prompt that gives it a personality, and note one prompt improvement you made.

Week 10: Vibe Coding Full Apps, Lovable, Bolt.new & Full Stack

Topics:

- Lovable, the AI app builder that generates clean React code from conversation
- Building a complete web app with Lovable: describe features AI codes you iterate
- Bolt.new deep dive, building and deploying apps entirely through AI prompts
- When to vibe code vs. hand code, understanding the trade-offs and limitations
- Introduction to Flask, Python for the web (understanding what AI builds for you)
- Setting up a Flask app, routes and views
- Connecting your AI Python code to a web frontend
- Building a simple AI web app, Python backend + HTML frontend
- Environment variables and .env files, keeping API keys safe
- Introduction to databases: SQLite with Python basics
- Deploying your apps: Replit, Vercel, Netlify, Render (free tier)
- Security basics: why vibe-coded apps need human review before going live
- Planning your capstone project, choosing between vibe-coded or hand-coded

Projects built:

- Lovable App, build a complete app by chatting with AI (e.g., habit tracker, expense tracker)
- AI Chatbot Web App, a proper web interface for your Week 9 AI chatbot using Flask
- Bolt.new Speed Build, go from idea to deployed app in under 30 minutes

Practice: Build the same to do app twice, once by hand and once with Lovable or Bolt.new, and write down three differences you noticed in the code.

Week 11: Developer Portfolio & Professional Presence

Topics:

- What is a developer portfolio and why every teen dev needs one
- Structuring your portfolio: projects, about, skills, contact
- Writing project descriptions that impress, STAR format
- README files on GitHub, making your repos presentable
- Vibe coding your portfolio website: use v0.dev or Lovable to build a stunning portfolio in minutes
- Picking your best 4 projects for the portfolio (mix of hand-coded and vibe-coded)

- GitHub profile optimization, pins, README profile, contributions graph
- LinkedIn basics for teen developers, setting up a professional profile
- Personal branding, your developer identity and niche
- Technical interview basics, what companies actually ask
- Writing a tech resume for internships and college applications
- Open source contribution basics, finding good first issues
- Hackathons: how to find, join, and win at hackathons

Projects built:

- Professional Developer Portfolio Website, multi-section, deployed, polished and live
- GitHub Profile README, a custom animated profile readme showcasing your skills
- Project README Collection, professional documentation for all 4 key portfolio projects

Practice: Draft your portfolio site with three project cards, STAR style descriptions and a proper README on your best repo.

Week 12: Grand Vibe Showcase, Launch Your Career!

Topics:

- Final capstone project polish, code review and bug fixes
- Code documentation best practices
- Recording a 3-minute project demo video
- The art of the technical demo, explaining code to non-coders
- Preparing for Q&A, answering questions about your code
- The vibe coding toolkit recap, Cursor, Copilot, v0.dev, Bolt.new, Lovable: when to use each
- Introduction to competitive coding: HackerRank, LeetCode, CodeChef
- What's next?, College applications with a portfolio, internships, freelancing
- Vibe Coding for College preview, MERN, DSA, Machine Learning, Claude Code, Windsurf, and more
- Networking with the Modern Age Coders alumni community
- Receiving the Vibe Coder Teens Certification
- Graduation ceremony and class showcase

Projects built:

- GRAND CAPSTONE, 'Vibe Project Launch': A complete, full-stack (or AI-powered) web application built using a combination of traditional coding and vibe coding tools (Cursor, Lovable, or Bolt.new). Must be deployed live, documented on GitHub, and presented at the showcase with a 3-minute live demo explaining both what you built AND how you used AI to build it

Assessment: Final Capstone Code Review + Live Demo + Panel Q&A

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 9+ Python CLI projects, games, calculators, automation tools
- Phase 2 (Weeks 5-8): 9+ web projects, portfolio, responsive sites, API-connected apps
- Phase 3 (Weeks 9-12): 6+ AI & vibe-coded projects + Grand Capstone deployed live

Total projects: 12+ real projects deployed live + professional portfolio

SKILLS THE STUDENT WILL HAVE AT THE END

- Python: variables, loops, functions, OOP basics, file handling, JSON, error handling
- Web Development: HTML5, CSS3, Flexbox, Grid, responsive design, JavaScript ES6+
- JavaScript: DOM manipulation, event listeners, Fetch API, LocalStorage, async/await
- Vibe Coding Tools: Cursor IDE, GitHub Copilot, v0.dev, Bolt.new, Lovable
- AI & APIs: Google Gemini API, Claude API, prompt engineering, chatbot development
- Backend Basics: Flask, Jinja2, SQLite, REST API concepts
- DevOps: Git, GitHub, deployment on Netlify, Vercel, Render
- Professional Skills: portfolio building, README writing, LinkedIn optimization

HOW A TYPICAL WEEK IS SPLIT

Live sessions: 2 sessions (1.5-2 hours each)
 Independent projects: 3-4 hours
 Vibe challenge: 1 weekly creative challenge
 Total per week: 6-8 hours

SUPPORT INCLUDED

Live instruction: Expert-led live classes with recorded replays
 Code reviews: 1-on-1 code reviews with instructors
 Peer coding: Collaborative coding sessions with classmates
 Community: Access to Modern Age Coders teen developer community
 Hackathon prep: Guidance for coding competitions and hackathons

CERTIFICATION DETAIL

Final certificate: Vibe Coder, Teens Level (Modern Age Coders Certified)
 Capstone required: Complete full-stack or AI-powered web app, deployed live, presented at showcase
 Portfolio: 12+ completed projects + professional developer portfolio website

WHERE THIS CAN LEAD

- Junior Web Developer (freelance or internship)

- Python Developer Intern
- Tech Entrepreneur / App Creator
- Competitive Programmer (ICPC, CodeChef, HackerRank)
- Freelance Web Developer (Rs 500-1,500/hour)
- Tech Club Leader / School Tech Projects
- Ready for Vibe Coding for College, MERN, DSA, ML & more

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Vibe Coding for Teens?

A: Vibe Coding for Teens is a 3-month, project-first coding program for ages 13-17. 'Vibe Coding', coined by AI pioneer Andrej Karpathy in 2025 (Collins Dictionary Word of the Year!), means building software by describing what you want in natural language and letting AI generate the code. Teens learn both traditional coding (Python, HTML/CSS/JavaScript) AND modern vibe coding tools (Cursor IDE, GitHub Copilot, v0.dev, Bolt.new, Lovable). You graduate with 12+ real projects and the ability to build anything, with or without AI assistance.

Q: Do I need to have done Vibe Coding for Kids first?

A: No! Vibe Coding for Teens is designed to welcome complete beginners aged 13-17. If you completed Vibe Coding for Kids, you'll find Phase 1 comes very naturally to you and you'll move faster. But anyone with curiosity and a laptop can join from zero.

Q: What programming languages and AI tools will I learn?

A: You will learn Python (Phases 1 and 3) and the core web trio, HTML, CSS, and JavaScript (Phase 2). But what makes this course special is the vibe coding tools: Cursor IDE (the #1 AI code editor), GitHub Copilot (free for students!), v0.dev (AI UI generator by Vercel), Bolt.new (AI app builder), and Lovable (builds full-stack React apps from conversation). You'll also learn AI APIs (Gemini / Claude), Git, and GitHub. By the end, you'll know which tool to use for any project.

Q: Will I be able to build real websites and apps by the end?

A: Absolutely. By Week 12, you will have built and deployed multiple live websites and web applications that anyone can visit on the internet. You'll also have Python CLI apps, an AI-powered project, and a professional portfolio on GitHub.

Q: How much time should I set aside each week?

A: We recommend 6-8 hours per week: 2 live sessions (1.5-2 hours each) plus 3-4 hours of independent project work. The more time you put in, the more impressive your portfolio will be.

Q: What tools and software will I need?

A: You need a computer (Windows, Mac, or Linux) and a stable internet connection. All primary tools are free: Cursor IDE (free tier), VS Code, GitHub Copilot (free for students via GitHub Education), Python, Git, v0.dev (free tier), Bolt.new (free tier), Lovable (free tier), and Replit (free tier). We walk through every setup in Week 1. No paid subscriptions are required.

Q: Can this help with college applications or internship applications?

A: Yes! A strong developer portfolio with real, deployed projects is an incredible differentiator in college applications (especially for CS/tech programs) and internship applications. Our Week 11 module specifically teaches you how to present your portfolio professionally.

Q: What comes after Vibe Coding for Teens?

A: Graduates are ready for Vibe Coding for College, which dives into Data Structures & Algorithms, full MERN stack development, Machine Learning with Python, cloud deployment, professional AI tools (Claude Code, Windsurf), and building production-grade applications with advanced vibe coding and agentic engineering workflows.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/vibe-coding-for-teens-python-web-ai-projects-course>

=====

COURSE 118 OF 118: WEB DEVELOPMENT FOR KIDS: BUILD WEBSITES WITH HTML, CSS & AI

=====

Course link: <https://learn.modernagecoders.com/courses/kids-ai-web-development-course>

Subject area: Kids Technology Education

Designed for: Kids

Who can join: Complete Beginner to Advanced Young Developer

Course length: 12 months (52 weeks)

Time needed: 5-10 hours/week recommended

Certificate: Young Web Developer Certificate with AI Specialization

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group \$100, one to one \$150

WHAT THIS COURSE IS

Live online web development course for kids: build real websites with HTML, CSS and AI tools in a 1-year program, from zero coding to professional sites.

In one line: From First Click to AI-Powered Websites in Just 1 Year!

ABOUT THE PROGRAMME

Welcome to the most exciting coding adventure designed specifically for young minds! This isn't just learning to code, it's a journey into the future of technology. Whether you're 8 or 18, this program will transform you from someone who uses websites to someone who creates them. You'll start with colorful, fun HTML pages and gradually master beautiful CSS designs. Then, you'll discover the magic of AI tools like ChatGPT, Claude, and GitHub Copilot to supercharge your coding. By the end, you'll have built 30+ amazing projects including games, interactive stories, and professional websites, all with the help of AI!

WHAT MAKES THIS COURSE DIFFERENT

- Designed specifically for young learners with fun, engaging content
- Uses AI tools from day one to make coding easier and more exciting
- Game-like progression with badges, achievements, and rewards
- Build real websites that friends and family can use
- Learn future-ready skills combining coding with AI
- Parent-friendly progress tracking and safe online environment
- Creative projects like games, animations, and interactive stories
- No boring theory, everything is hands-on and visual

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): HTML Basics, First Websites, Introduction to AI Helpers

Phase 2: Creative Design (Months 4-6): CSS Magic, Animations, AI-Powered Design

Phase 3: Interactive Websites (Months 7-9): JavaScript Basics, AI Code Generation, Dynamic Content

Phase 4: Professional Projects (Months 10-12): Complete Websites, AI Integration, Portfolio Building

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic reading and writing skills (Age 8+)

Coding experience: None required - complete beginner friendly

Equipment: Computer/laptop with internet, mouse recommended

Time commitment: 5-10 hours per week

English: Basic English understanding

Motivation: Curiosity and excitement to create

WHO THIS COURSE IS FOR

Kids 8-12: Young learners wanting to create their first websites

Teens 13-17: Teenagers interested in tech careers

Homeschoolers: Perfect for homeschool technology curriculum

Creative kids: Children who love art, design, and creativity

Future entrepreneurs: Kids who want to build their own projects

Parent child: Parents can learn alongside their children

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: HTML FOUNDATION & AI INTRODUCTION (MONTHS 1-3, WEEKS 1-13)

Start your coding adventure! Learn HTML basics while using AI assistants to help you code faster and smarter.

Months 1-2: HTML Basics & Your First Websites (Week 1-8)

Welcome to Web Development & AI Tools Setup

Topics:

- What is a website? How does the internet work (kid-friendly explanation)
- Meeting your AI coding friends: ChatGPT and Claude
- Setting up your coding playground: VS Code with fun themes
- Installing colorful extensions and emoji support
- Your first HTML file: Hello World with emojis!
- Understanding tags: like LEGO blocks for websites
- Creating a personal 'About Me' page

- Using AI to generate HTML code ideas
- Asking ChatGPT questions about HTML
- HTML structure: <html>, <head>, <body> explained with stories
- Saving and viewing your first webpage
- Celebrating your first website with screenshots!

Projects built:

- Create 'All About Me' webpage with AI suggestions
- Build a digital pet profile page
- Make a birthday invitation webpage

Practice: Daily: Create one fun HTML page with AI help

HTML Elements Zoo - Text, Images & Links

Topics:

- Heading tags (h1-h6): Creating titles like newspaper headlines
- Paragraph tag: Writing stories on your webpage
- Making lists: Favorite games, friends, hobbies (ul, ol, li)
- Bold and italic: Making text stand out (strong, em)
- Adding images: Your photos, drawings, and GIFs
- Image attributes: alt text for accessibility
- Creating links: Connect your pages like a treasure map
- Link to other websites: YouTube, games, educational sites
- Using AI to write alt text for images
- ChatGPT for content ideas and creative writing
- Making a photo album webpage
- Building a collection of favorite things

Projects built:

- Digital photo album with captions
- My favorite YouTube videos collection
- Interactive storybook with multiple pages
- School project presentation website

Practice: Build 5 themed pages (animals, sports, hobbies, etc.)

Fun with Forms & Tables

Topics:

- Creating forms: Like making a quiz for friends
- Input types: text, email, password, number
- Fun input types: color picker, date picker, range slider
- Checkboxes for multiple choices: Pizza toppings selector
- Radio buttons for single choice: Favorite superhero poll
- Dropdown menus: Choose your adventure options
- Textarea: Writing stories and comments
- Tables for organizing information: Sports scores, schedules
- Making tables pretty with borders and colors
- Using AI to generate form questions
- ChatGPT for creating quiz content
- Building interactive surveys

Projects built:

- Personality quiz generator with AI questions
- Friend information collector form
- Class schedule table
- Restaurant menu with order form

Practice: Create 5 different interactive forms

Multimedia Magic - Audio, Video & Canvas

Topics:

- Adding videos to your webpage: YouTube embeds
- HTML5 video tag: Playing your own videos
- Audio elements: Adding music and sound effects
- Creating playlists on your webpage
- Canvas basics: Drawing on webpages
- Embedding games and interactive content
- iframe: Windows to other websites
- Making a multimedia gallery
- Using AI to find copyright-free media
- ChatGPT for video script ideas
- Creating a personal YouTube-style page
- Building an online portfolio of creations

Projects built:

- Personal video channel page
- Music playlist website
- Digital art gallery with canvas
- Multimedia learning resource page

Practice: Add multimedia to all previous projects

Month 3: CSS Foundations & AI-Powered Styling (Week 9-13)

CSS Basics - Making Websites Beautiful

Topics:

- What is CSS? The fashion designer for HTML
- Three ways to add CSS: inline, internal, external stylesheets
- CSS syntax: selector { property: value; }
- Selecting elements: tags, classes, and IDs
- Colors everywhere: background-color, color

- Color codes: named colors, hex codes, RGB
- Using AI to generate color palettes
- ChatGPT for CSS styling suggestions
- Font families: Changing how text looks
- Google Fonts: Adding fun fonts to websites
- Text size, weight, and style
- Text alignment and decoration

Projects built:

- Rainbow text website
- Colorful mood board page
- Typography playground
- AI-generated color scheme portfolio

Practice: Style 10 HTML pages with different themes

CSS Box Model & Spacing Magic

Topics:

- The Box Model: Every element is a box!
- Padding: Space inside the box
- Margin: Space outside the box
- Border: The box's frame (styles, colors, radius)
- Width and height: Sizing your boxes
- Making round corners with border-radius
- Creating shapes with CSS: circles, triangles
- Box shadows for 3D effects
- Background images and patterns
- Using AI to generate CSS for complex shapes
- ChatGPT for spacing and layout advice
- Building card-based layouts

Projects built:

- Trading card collection webpage
- Social media profile page clone
- Product showcase with cards
- CSS art project with shapes

Practice: Create 10 different card designs

Phase 1 Celebration Project

Topics:

- Planning your first complete website
- Combining HTML and CSS skills
- Using AI to brainstorm website ideas
- Creating a sitemap with multiple pages
- Consistent styling across pages
- Adding personal creativity and flair
- Getting feedback from family and friends
- Publishing on GitHub Pages

Projects built:

- CAPSTONE: Complete 5-page personal website
- Options: Fan site, hobby showcase, family website, school project
- Must include: forms, multimedia, styled with CSS, AI-generated content

Assessment: Phase 1 Quiz and Project Presentation

----- PHASE 2: ADVANCED CSS & ANIMATION MAGIC (MONTHS 4-6, WEEKS 14-26) -----

Level up your design skills! Master advanced CSS, create stunning animations, and use AI to generate professional designs.

Months 4-5: CSS Layouts & Responsive Design (Week 14-21)

Flexbox - The Flexible Layout System

Topics:

- Introduction to Flexbox: Making layouts flexible
- Flex container and flex items
- Flex-direction: row, column, reverse
- Justify-content: Spacing items horizontally
- Align-items: Spacing items vertically
- Creating navigation bars with Flexbox
- Building photo galleries
- Flexbox games: Flexbox Froggy, Flexbox Defense
- Using AI to generate Flexbox layouts
- ChatGPT for solving layout challenges
- Making responsive card layouts
- Creating hero sections

Projects built:

- Responsive navigation menu
- Pinterest-style photo gallery
- Flexible dashboard layout
- Game character selection screen

Practice: Complete all Flexbox Froggy levels

CSS Grid - The Ultimate Layout Tool

Topics:

- CSS Grid basics: Rows and columns

- Grid-template-columns and grid-template-rows
- Grid gaps: Adding space between items
- Placing items in specific grid cells
- Grid areas: Naming your layout sections
- Creating magazine-style layouts
- Building game boards with Grid
- Grid Garden game for practice
- AI-assisted Grid layout generation
- Combining Flexbox and Grid
- Responsive grids with auto-fit
- Creating complex layouts easily

Projects built:

- Magazine homepage layout
- Tic-tac-toe game board
- Calendar layout with Grid
- Minecraft-style inventory grid

Practice: Build 10 different grid layouts

Responsive Design - Mobile First!

Topics:

- What is responsive design? One site for all devices
- Mobile-first approach: Start small, go big
- Media queries: Different styles for different screens
- Breakpoints: Phone, tablet, desktop
- Responsive units: %, vw, vh, rem, em
- Responsive images: srcset and picture element
- Responsive typography: Scaling text
- Testing on different devices
- Chrome DevTools device simulator
- Using AI to generate responsive CSS
- Mobile-friendly navigation patterns
- Touch-friendly button sizes

Projects built:

- Responsive portfolio website
- Mobile-first blog layout
- Adaptive gaming website
- Responsive school project site

Practice: Make all previous projects responsive

CSS Transforms & Transitions

Topics:

- Transform property: Move, rotate, scale, skew
- 2D transformations for fun effects
- 3D transformations: Perspective and depth
- Creating flip cards and rotating elements
- Transition property: Smooth changes
- Transition timing functions: ease, linear, cubic-bezier
- Hover effects and interactive elements
- Building interactive galleries
- CSS variables for dynamic styling
- Using AI to create complex transforms
- Creative hover animations
- Making websites feel alive

Projects built:

- 3D flip card memory game
- Interactive product showcase
- Animated navigation menu
- Hover effect gallery

Practice: Add transitions to 20 elements

CSS Animations & Keyframes

Topics:

- CSS animations vs transitions
- @keyframes: Creating animation sequences
- Animation properties: duration, delay, iteration
- Animation timing and easing
- Creating loading animations
- Bouncing, sliding, fading effects
- Animated backgrounds and patterns
- SVG animations with CSS
- Performance considerations
- Using AI to generate keyframe animations
- Animation libraries introduction
- Making animated stories

Projects built:

- Animated landing page
- Loading screen collection
- Animated logo design
- CSS animation showcase

Practice: Create 15 different animations

Advanced CSS Effects & Filters

Topics:

- CSS filters: blur, brightness, contrast, grayscale
- Creating Instagram-like photo filters
- Backdrop-filter for glass effects
- Mix-blend-mode for creative overlays
- Gradient backgrounds: Linear and radial
- Multiple backgrounds and patterns
- Clip-path for custom shapes
- CSS masks and masking
- Text effects: Shadows, outlines, gradients
- Using AI to generate CSS art
- ChatGPT for creative effect ideas
- Building artistic websites

Projects built:

- Instagram filter simulator
- Glassmorphism UI design
- Artistic portfolio with effects
- Creative text effects showcase

Practice: Apply 20 different CSS effects

CSS Frameworks & Libraries

Topics:

- Introduction to CSS frameworks
- Bootstrap basics: Grid, components, utilities
- Tailwind CSS: Utility-first approach
- Using pre-built components
- Customizing framework styles
- When to use frameworks vs custom CSS
- Component libraries for kids
- Icon libraries: Font Awesome, Material Icons
- Using AI to generate Tailwind classes
- ChatGPT for framework selection
- Building faster with frameworks
- Maintaining your style

Projects built:

- Bootstrap landing page
- Tailwind CSS blog design
- Component library showcase
- Framework comparison project

Practice: Build 5 projects with different frameworks

AI-Powered Design Tools

Topics:

- Introduction to AI design assistants
- Using ChatGPT for design ideas
- Claude for code generation
- Midjourney/DALL-E for image creation
- AI color palette generators
- AI-powered layout suggestions
- GitHub Copilot for CSS
- AI website builders overview
- Ethical use of AI in design
- Combining AI with personal creativity
- AI for accessibility testing
- Future of AI in web design

Projects built:

- AI-designed portfolio website
- AI-generated art gallery
- Website with AI-created images
- AI-assisted redesign project

Practice: Use AI tools in all projects

CSS Best Practices & Organization

Topics:

- Organizing CSS files
- Naming conventions: BEM methodology
- CSS custom properties (variables)
- Creating a design system
- Component-based CSS
- Performance optimization
- Minification and compression
- Cross-browser compatibility
- CSS validation tools
- Using AI for code review
- Documentation with comments
- Maintaining clean code

Projects built:

- Personal design system
- Refactor old projects
- Create CSS style guide
- Performance-optimized site

Practice: Optimize all previous CSS code

Phase 2 Showcase Project

Topics:

- Planning an impressive CSS project
- Incorporating all learned techniques
- Using AI for project planning
- Creating wireframes and mockups
- Building with best practices
- Testing across devices
- Getting peer feedback
- Presenting your work

Projects built:

- CAPSTONE: Advanced CSS Showcase Website
- Options: Animated story, game interface, business website, art portfolio
- Requirements: Responsive, animated, AI-assisted, framework usage

Assessment: Phase 2 Final Project Presentation

PHASE 3: JAVASCRIPT & AI INTEGRATION (MONTHS 7-9, WEEKS 27-39)

Add interactivity to your websites! Learn JavaScript basics and use AI to write complex code easily.

Months 7-8: JavaScript Fundamentals (Week 27-34)

JavaScript Basics - Making Websites Interactive

Topics:

- What is JavaScript? The brain of websites
- Adding JavaScript to HTML pages
- Console.log: Talking to the computer
- Variables: Storing information (let, const)
- Data types: Numbers, strings, booleans
- Basic math operations
- String concatenation: Joining words
- Template literals: Easy text formatting
- Popup boxes: alert, confirm, prompt
- Comments in JavaScript
- Using ChatGPT to explain JavaScript
- AI for debugging simple errors

Projects built:

- Interactive calculator
- Mad libs story generator
- Age calculator with messages
- Simple greeting program

Practice: Write 20 simple JavaScript programs

JavaScript Logic & Decisions

Topics:

- If statements: Making decisions
- Else and else if: Multiple choices
- Comparison operators: >, <, ==, ===
- Logical operators: AND (&&), OR (||), NOT (!)
- Switch statements for multiple cases
- Ternary operator: Quick decisions
- Random numbers with Math.random()
- Building simple games with logic
- Using AI to write conditional logic
- ChatGPT for algorithm help
- Creating quiz logic
- Score keeping systems

Projects built:

- Rock, Paper, Scissors game
- Number guessing game
- Quiz with scoring
- Adventure choice game

Practice: Build 10 decision-based programs

Loops & Arrays - Handling Multiple Items

Topics:

- Arrays: Lists of items
- Accessing array elements
- Array methods: push, pop, length
- For loops: Repeating actions
- While loops: Keep going until...
- Looping through arrays
- forEach: Modern array looping
- Creating patterns with loops
- Nested loops for complex patterns
- Using AI to generate loop logic
- Building lists dynamically
- Array-based games

Projects built:

- Todo list manager
- Student grade calculator
- Pattern generator
- Memory card game logic

Practice: Solve 20 loop and array challenges

Functions - Reusable Code Blocks

Topics:

- What are functions? Code recipes
- Creating simple functions
- Parameters: Giving functions information
- Return values: Getting answers back
- Function expressions vs declarations
- Arrow functions: Modern syntax
- Scope: Where variables live
- Built-in functions: Math, String methods
- Creating a function library
- Using AI to write functions
- ChatGPT for refactoring code
- Organizing code with functions

Projects built:

- Utility function library
- Calculator with functions
- Text analyzer tool
- Game with multiple functions

Practice: Create 30 useful functions

Objects - Organizing Related Data

Topics:

- Objects: Grouping related information
- Object properties and values
- Accessing object data
- Adding and changing properties
- Objects in arrays
- Arrays in objects
- JSON: Data format for the web
- Creating character objects for games
- Building data structures
- Using AI to design objects
- Working with complex data
- Object-based applications

Projects built:

- Character creator for games
- Product catalog system
- Student database
- Pet adoption website data

Practice: Build 10 object-based systems

Month 9: DOM & AI-Powered Interactivity (Week 35-39)

DOM Manipulation - Changing Websites with JavaScript

Topics:

- What is the DOM? The website's structure
- Selecting elements: getElementById, querySelector
- Changing text content dynamically
- Changing styles with JavaScript
- Adding and removing CSS classes
- Creating new elements
- Adding elements to the page
- Removing elements
- Show/hide elements
- Using AI to generate DOM code
- ChatGPT for DOM manipulation help
- Building dynamic interfaces

Projects built:

- Dynamic content generator
- Interactive story with choices
- Color theme switcher
- DOM-based game

Practice: Manipulate DOM in 20 different ways

Events - Responding to User Actions

Topics:

- Event listeners: Waiting for user actions
- Click events: Buttons and links
- Mouse events: hover, mouseenter, mouseleave
- Keyboard events: Typing and key presses
- Form events: submit, input, change
- Event objects: Getting event details
- Preventing default behaviors
- Creating interactive games
- Touch events for mobile
- Using AI to handle complex events

- Building event-driven apps
- User interaction patterns

Projects built:

- Interactive drawing app
- Keyboard-controlled game
- Form validation system
- Drag and drop interface

Practice: Create 15 event-driven features

Local Storage & Data Persistence

Topics:

- What is Local Storage? Saving data locally
- Storing data with `localStorage.setItem`
- Retrieving data with `localStorage.getItem`
- Removing data with `localStorage.removeItem`
- `JSON.stringify` and `JSON.parse`
- Creating save game functionality
- Remembering user preferences
- Building offline-capable apps
- Session storage vs local storage
- Using AI to manage storage logic
- Data persistence strategies
- Privacy considerations

Projects built:

- Note-taking app with save
- Game with save progress
- Settings preferences app
- Shopping cart with storage

Practice: Add storage to 10 projects

APIs & External Data

Topics:

- What are APIs? Getting data from the internet
- `fetch()`: Getting data from APIs
- Working with JSON data
- Displaying API data on websites
- Free APIs for kids: Pokemon, weather, jokes
- Handling API errors gracefully
- Loading states and spinners
- Using AI to work with APIs
- ChatGPT for API integration
- Building data-driven apps
- Real-time data updates
- API best practices

Projects built:

- Pokemon card collection
- Weather dashboard
- Joke generator app
- News reader for kids

Practice: Integrate 10 different APIs

Phase 3 Interactive Project

Topics:

- Planning a JavaScript-heavy project
- Combining DOM, events, and storage
- Using AI for complex features
- Creating smooth user experiences
- Testing interactive features
- Debugging with browser tools
- Code organization strategies
- Project documentation

Projects built:

- CAPSTONE: Full Interactive Web Application
- Options: Game, productivity app, educational tool, social platform
- Requirements: DOM manipulation, events, storage, API, AI-assisted

Assessment: Phase 3 JavaScript Mastery Test

----- PHASE 4: PROFESSIONAL PROJECTS & AI MASTERY (MONTHS 10-12, WEEKS 40-52) -----

Build professional websites using advanced AI tools, learn modern web development practices, and create your portfolio.

Months 10-11: Advanced AI Tools & Modern Development (Week 40-47)

Advanced AI Coding Assistants

Topics:

- GitHub Copilot: Your AI pair programmer
- Setting up Copilot in VS Code
- Writing code with Copilot suggestions
- Copilot for HTML/CSS/JavaScript
- Amazon Q Developer basics
- Tabnine AI assistant

- Cursor AI editor introduction
- AI code completion tips
- Writing better prompts for AI
- AI for code refactoring
- Debugging with AI help
- Code review with AI

Projects built:

- Build complete website with Copilot
- Refactor project with AI
- AI-assisted debugging challenge
- Speed coding with AI race

Practice: Use AI assistants in all coding

AI Website Builders & No-Code Tools

Topics:

- Introduction to AI website builders
- Webflow basics for visual development
- Framer for interactive designs
- Bubble.io for web applications
- WordPress with AI plugins
- Wix ADI (Artificial Design Intelligence)
- Combining no-code with custom code
- When to use builders vs coding
- AI design suggestions
- Rapid prototyping with AI
- Exporting code from builders
- Customizing AI-generated sites

Projects built:

- Business website with AI builder
- Portfolio site comparison
- No-code web app
- Hybrid builder-code project

Practice: Create 5 sites with different builders

Version Control with Git & GitHub

Topics:

- What is version control? Saving your work history
- Git basics: init, add, commit
- Understanding commits and history
- Creating branches for features
- Merging branches safely
- GitHub: Your code in the cloud
- Creating repositories
- Push and pull operations
- GitHub Pages for hosting
- Collaboration basics: forks and pull requests
- Using AI to write commit messages
- GitHub Copilot integration

Projects built:

- Version control for portfolio
- Collaborative class project
- Open source contribution
- GitHub Pages portfolio site

Practice: Use Git for all projects

Modern JavaScript Frameworks Introduction

Topics:

- What are frameworks? Making development easier
- React basics: Components and JSX
- Vue.js introduction for beginners
- Creating simple components
- Props: Passing data to components
- State: Managing component data
- Event handling in frameworks
- Using AI to learn frameworks
- ChatGPT for framework questions
- Component-based thinking
- When to use frameworks
- Framework ecosystem overview

Projects built:

- Simple React todo app
- Vue.js counter game
- Component library
- Framework comparison project

Practice: Build 5 small framework projects

Web Accessibility & Best Practices

Topics:

- What is accessibility? Websites for everyone
- Alt text for images
- Semantic HTML importance
- ARIA labels and roles
- Keyboard navigation

- Color contrast checking
- Screen reader testing
- Accessibility tools and extensions
- Using AI for accessibility audit
- Making forms accessible
- Video captions and transcripts
- Mobile accessibility

Projects built:

- Accessibility audit of old projects
- Fully accessible website
- Screen reader-friendly game
- Accessibility checker tool

Practice: Make all projects accessible

Month 12: Final Projects & Portfolio (Week 48-52)

Portfolio Development & Personal Branding

Topics:

- Planning your developer portfolio
- Showcasing your best projects
- Writing project descriptions
- Creating case studies
- About me page that stands out
- Skills section design
- Contact form implementation
- Resume/CV page creation
- Using AI for content writing
- SEO basics for portfolios
- Custom domain setup
- Professional email setup

Projects built:

- Professional portfolio website
- Project showcase pages
- Interactive resume
- Personal brand design

Practice: Polish and perfect portfolio

Real-World Client Projects

Topics:

- Understanding client requirements
- Creating project proposals
- Wireframing and mockups
- Client communication basics
- Project timelines and milestones
- Feedback and revisions
- Testing and quality assurance
- Launching client websites
- Using AI for client work
- Documentation and handover
- Maintenance and updates
- Building client relationships

Projects built:

- Mock client project
- Family business website
- School club website
- Community project site

Practice: Complete 2 real-world projects

Advanced Project: Full-Stack Application

Topics:

- Planning a complex application
- Database design basics
- Backend API introduction
- Connecting frontend to backend
- User authentication basics
- Real-time features with WebSockets
- Cloud deployment basics
- Using AI for full-stack development
- Testing your application
- Performance optimization
- Security best practices
- Launch preparation

Projects built:

- Social platform for kids
- Educational game platform
- Collaborative story writer
- School management tool

Practice: Build complete full-stack app

Performance & Optimization

Topics:

- Website speed importance
- Image optimization techniques

- Lazy loading for performance
- Minifying CSS and JavaScript
- Browser caching basics
- Content Delivery Networks (CDN)
- Google PageSpeed Insights
- Lighthouse performance audits
- Mobile performance optimization
- Using AI for optimization
- Performance monitoring tools
- Continuous improvement

Projects built:

- Optimize portfolio for speed
- Performance comparison study
- Speed optimization toolkit
- Before/after showcase

Practice: Optimize all major projects

Career Preparation & Future Learning

Topics:

- Creating your GitHub profile
- Building online presence
- LinkedIn for young developers
- Contributing to open source
- Participating in hackathons
- Coding competitions for kids
- Internship opportunities
- Freelancing basics for teens
- Continuous learning resources
- Advanced courses to consider
- Tech communities to join
- Setting career goals

Projects built:

- GitHub profile optimization
- Professional online presence
- Open source contribution
- Hackathon project

Practice: Prepare for next steps

Final Month: Capstone & Graduation (Week 50-52)

Final Capstone Project Planning

Topics:

- Choosing your capstone project
- Project scope and requirements
- Creating detailed wireframes
- Technical architecture planning
- Setting up development environment
- Project timeline creation
- Using AI for project planning
- Gathering resources and assets
- Setting success criteria
- Preparing for challenges

Projects built:

- Capstone project proposal
- Technical specification document
- Visual mockups and designs
- Development roadmap

Practice: Thorough project planning

Final Capstone Development

Topics:

- Agile development process
- Daily development sprints
- Feature implementation
- Regular testing and debugging
- AI-assisted development
- Code reviews and refactoring
- Progress tracking
- Problem-solving strategies
- Seeking help when stuck
- Version control best practices

Projects built:

- FINAL CAPSTONE PROJECT
- Options: E-learning platform, game portal, social network, business solution
- Requirements: Responsive, interactive, AI-integrated, accessible, optimized

Practice: Intensive development work

Testing, Polish & Deployment

Topics:

- Comprehensive testing strategies
- Cross-browser testing
- Mobile device testing
- User testing with friends/family

- Bug fixing and refinement
- Performance optimization
- Security checks
- Final polish and details
- Deployment preparation
- Domain and hosting setup
- Launch checklist
- Documentation completion

Projects built:

- Complete testing suite
- Bug fix tracker
- Performance report
- Deployment to production

Practice: Professional launch preparation

Presentation & Celebration

Topics:

- Creating project presentation
- Demo video recording
- Project documentation
- Success story writing
- Lessons learned reflection
- Peer project reviews
- Showcase event preparation
- Certificate ceremony
- Future plans discussion
- Alumni network joining
- Celebration activities
- Course feedback session

Projects built:

- Final presentation deck
- Demo video production
- Project case study
- Course portfolio compilation

Assessment: Final project presentation and graduation!

Bonus Content & Extended Learning (Ongoing)

Advanced AI Integration

Topics:

- ChatGPT API integration
- Building AI-powered features
- Image generation with AI
- Voice assistants in web apps
- AI chatbots for websites
- Machine learning basics
- TensorFlow.js introduction
- AI ethics for young developers
- Future of AI in web development

Projects built:

- AI chatbot integration
- AI-powered image gallery
- Voice-controlled website
- ML-powered recommendations

Mobile App Development Basics

Topics:

- Progressive Web Apps (PWA)
- Making websites installable
- React Native introduction
- Mobile-first development
- Touch gestures and interactions
- Mobile app deployment basics
- App store considerations
- Cross-platform development

Projects built:

- Convert website to PWA
- Simple React Native app
- Mobile-optimized game
- Installable web app

Lifetime Learning Resources

Topics:

- Advanced course recommendations
- University programs in CS
- Online learning platforms
- Tech bootcamps for teens
- Scholarship opportunities
- Mentorship programs
- Tech internships for students
- Building a learning routine
- Staying updated with tech
- Career pathways in tech

- Entrepreneurship basics
- Creating your own courses

Deliverables:

- Personal learning plan
- Resource library
- Network connections
- Future goals document

Assessment: Course completion certificate and badge!

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Personal website, Photo gallery, Quiz form, Multimedia showcase
- Phase 2: Animated portfolio, Responsive blog, CSS art gallery, Framework projects
- Phase 3: Interactive games, Todo apps, API integrations, Data visualizations
- Phase 4: AI-powered apps, Professional portfolio, Client projects, Full-stack application

Total projects: 30+ creative projects from simple to complex

SKILLS THE STUDENT WILL HAVE AT THE END

- Frontend: HTML5 mastery, CSS3 animations, Responsive design, JavaScript interactivity
- Design: Color theory, Typography, Layout systems, UI/UX basics, Accessibility
- Programming: Variables, Functions, Objects, Arrays, APIs, Local Storage, DOM manipulation
- AI Tools: ChatGPT, Claude, GitHub Copilot, Midjourney, AI builders, Prompt engineering
- Modern Tools: Git/GitHub, VS Code, Chrome DevTools, npm basics, Framework basics
- Soft Skills: Problem-solving, Creativity, Project planning, Debugging, Documentation

HOW A TYPICAL WEEK IS SPLIT

Theory videos: 2-3 hours
 Hands on coding: 3-4 hours
 Projects: 2-3 hours
 Practice problems: 1-2 hours
 Total per week: 5-10 hours

SUPPORT INCLUDED

Live sessions: Weekly fun coding sessions with instructors
 Mentorship: Dedicated mentor for guidance
 Community: Kid-friendly Discord community
 Code review: Instructor feedback on projects
 Parent updates: Regular progress reports for parents
 Certificates: Achievement badges and certificates

CERTIFICATION DETAIL

Phase certificates: Badge after each phase completion
 Final certificate: Young Web Developer Certificate with AI Specialization
 Linkedin badge: Shareable achievement badge
 Portfolio: 30+ projects to showcase
 Skills validation: Industry-recognized skills assessment

WHERE THIS CAN LEAD

- Junior Web Developer (part-time/internships)
- Freelance Website Creator for local businesses
- School Tech Team Member
- YouTube Content Creator (tech tutorials)
- Young Entrepreneur (build own products)
- Computer Science College Preparation
- Hackathon Participant
- Open Source Contributor
- Tech Blog Writer
- AI Tool Specialist

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my child build in this web development course?

A: Students create 30+ real projects over the year: colorful first HTML pages, styled CSS designs, responsive layouts, interactive JavaScript features, and finally complete professional websites for a graduation portfolio. Every project is a real website they can share with family and friends.

Q: How are AI tools used in the course?

A: Children learn to use tools like ChatGPT, Claude, and GitHub Copilot as coding assistants: to explain concepts, suggest improvements, and help debug. We teach AI as a helper for understanding, never a shortcut that replaces learning to write HTML, CSS, and JavaScript

themselves.

Q: What age is right for this course, and is experience needed?

A: The course welcomes complete beginners aged 8 to 18; no coding experience is needed, only basic reading and typing. Younger children begin with visual, playful HTML and CSS projects, while teens move faster into JavaScript and the AI-assisted development phases.

Q: What software or equipment is required?

A: A normal computer or laptop with internet access is enough. All the software used, including the VS Code editor and the AI tools introduced in class, is free, and we help students set everything up in the early lessons.

Q: What can my child learn after finishing this course?

A: The natural next step is our full stack development track, where students add JavaScript frameworks, backend programming, and databases to build complete applications. Others branch into Python or AI courses; the HTML, CSS, and JavaScript learned here is the foundation for all of it.

Q: How is the course priced?

A: Families outside India pay a price set for their own country, shown on this page in their local currency.

Q: Can we see a class before paying anything?

A: Yes. Book a free demo class at learn.modernagecoders.com/contact; there is no commitment, and parents are encouraged to sit in so they can judge the teaching quality firsthand.

Q: Are lessons live or video-based?

A: Every lesson is a live, interactive 1-hour online class with a real teacher, never a pre-recorded video. Because groups are small, the teacher sees each child's code and helps immediately when someone is stuck.

Q: Is there a certificate when the course ends?

A: Yes, graduates receive a Young Web Developer completion certificate. Even more useful is the portfolio of 30+ real websites they build along the way, which they can show to anyone with a link.

To enrol or to book a free demo for this course, message Shivam Sir on

+91 9123366161, or send the parent this link: <https://learn.modernagecoders.com/courses/kids-ai-web-development-course>

PART 7. HOW SOMEONE JOINS

THE STEPS, IN ORDER

1. First chat. Find out the age, the subject, the current level and the goal.
2. Free demo class. One live class with a real teacher. No card. Parent may sit in.
3. Pick a plan. Group, mini batch or one to one.
4. Shivam Sir matches the student to a batch that fits their timetable.
5. Pay the first month through a secure link.
6. The student gets dashboard access, the class link and the recordings.
7. Classes begin.
8. Progress updates go to the parent every month.

WHAT TO COLLECT BEFORE HANDING A LEAD OVER

- Student full name.
- Age, and school class or grade.
- School and board if it is a school subject (CBSE, ICSE, IGCSE, IB, state board).
- Parent name.
- Parent WhatsApp number, with the country code.
- City and country, so we know the time zone.
- Subject wanted: coding, maths, or both.
- The goal: school marks, board exam, hobby, Olympiad, college prep, a job.
- Roughly when they are free: weekdays or weekends, morning or evening.
- Preferred language: English or Hindi.

WHAT THE FREE DEMO ACTUALLY IS

- A full live class with a real teacher, not a sales call and not a recording.
- Free. No card details are ever asked for.
- The parent is welcome to watch.
- It doubles as a placement check, so the teacher recommends the right starting point.
- One free demo per student.
- If the child does not enjoy it, the family walks away owing nothing.

If a parent wants to judge us before booking, offer this: they can watch recordings of real, unedited classes first.

=====

PART 8. REFUND AND CANCELLATION. READ THIS OUT EXACTLY, DO NOT REWORD IT

=====

REFUND POLICY

Modern Age Coders offers a 7 day money back guarantee on all online courses, provided you have not completed more than 20 percent of the course content.

To be eligible:

- The request must come within 7 days of the original purchase date.
- Refunds apply to the first purchase only, not to renewals or repeat subscriptions.

Not refundable:

- Courses where more than 20 percent of the content has been viewed or downloaded.
- One to one mentorship sessions that have already been taught.
- Customised bootcamps or special arrangements.

Once a refund is approved it goes back to the original payment method. The credit appears within 7 to 10 business days, depending on the card issuer.

To request one, email contact@modernagecoders.com with the order details and the reason.

CANCELLATION POLICY

- Enrolment can be cancelled at any time. A cancellation after a live batch has started, or after materials have been accessed, falls under the refund policy above.
- A recurring subscription can be cancelled any time from the account dashboard. It takes effect at the end of the current billing cycle and there is no further charge.
- We may cancel or reschedule a course if too few students enrol or something unforeseen happens. Those students get a full refund or a transfer to another schedule.
- After cancelling, access continues until the end of the billing period already paid for.
- To cancel: use account settings, or email contact@modernagecoders.com.

IF SOMEONE ASKS "WHAT IF WE ARE NOT HAPPY?"

Say: "There is a 7 day money back guarantee. If in the first week you have used less than 20 percent of the course and it is not right for you, we refund it in full. And before any of that, the demo class is free, so you see the teaching before you pay anything at all."

=====

PART 9. OTHER PAGES YOU CAN SEND A PARENT

=====

THE PAGES PARENTS ASK FOR MOST

All courses	https://learn.modernagecoders.com/courses
Prices	https://learn.modernagecoders.com/pricing
Book a free demo	https://learn.modernagecoders.com/book-demo
Free trial class	https://learn.modernagecoders.com/free-trial
Our zero risk promise	https://learn.modernagecoders.com/guarantee
How we teach	https://learn.modernagecoders.com/how-we-teach
About us	https://learn.modernagecoders.com/about
Contact us	https://learn.modernagecoders.com/contact
Reviews from families	https://learn.modernagecoders.com/love
Student success stories	https://learn.modernagecoders.com/success-stories
Real student projects	https://learn.modernagecoders.com/student-labs
Course atlas, every course on one page	https://learn.modernagecoders.com/course-atlas
What to learn and in what order	https://learn.modernagecoders.com/coding-roadmap
Work out your monthly fee	https://learn.modernagecoders.com/coding-fee-calculator
Free resources	https://learn.modernagecoders.com/free-resources
Blog	https://learn.modernagecoders.com/blog
Schools programme	https://learn.modernagecoders.com/schools
For businesses	https://learn.modernagecoders.com/for-business
Refund policy	https://learn.modernagecoders.com/refund
Cancellation policy	https://learn.modernagecoders.com/cancellation
Which AI tools are safe at which age	https://learn.modernagecoders.com/ai-tools-age-guide

FREE THINGS TO PLAY WITH. Good for a parent who is unsure.

Beat the Babu, 90 second maths game	https://learn.modernagecoders.com/beat-the-babu
Aura Type, typing game	https://learn.modernagecoders.com/aura
Daily coding challenges	https://learn.modernagecoders.com/coding-challenges
Maths challenges	https://learn.modernagecoders.com/maths-challenges
Mistakes students make	https://learn.modernagecoders.com/mistakes-students-make
Misti AI, our study assistant	https://misti.modernagecoders.com
The Beautiful Mind, our blog	https://mind.modernagecoders.com
Student dashboard	https://dashboard.modernagecoders.com

Mark attendance	https://dashboard.modernagecoders.com/attendance
Class recordings	https://dashboard.modernagecoders.com/recording

CITY PAGES IN INDIA (131 pages)

<https://learn.modernagecoders.com/best-coding-class-in-agra>
<https://learn.modernagecoders.com/best-coding-class-in-ahmedabad>
<https://learn.modernagecoders.com/best-coding-class-in-ajmer>
<https://learn.modernagecoders.com/best-coding-class-in-akola>
<https://learn.modernagecoders.com/best-coding-class-in-aligarh>
<https://learn.modernagecoders.com/best-coding-class-in-amravati>
<https://learn.modernagecoders.com/best-coding-class-in-amritsar>
<https://learn.modernagecoders.com/best-coding-class-in-anand>
<https://learn.modernagecoders.com/best-coding-class-in-asansol>
<https://learn.modernagecoders.com/best-coding-class-in-aurangabad>
<https://learn.modernagecoders.com/best-coding-class-in-balasore>
<https://learn.modernagecoders.com/best-coding-class-in-bareilly>
<https://learn.modernagecoders.com/best-coding-class-in-belagavi>
<https://learn.modernagecoders.com/best-coding-class-in-bengaluru>
<https://learn.modernagecoders.com/best-coding-class-in-berhampur>
<https://learn.modernagecoders.com/best-coding-class-in-bhagalpur>
<https://learn.modernagecoders.com/best-coding-class-in-bhavnagar>
<https://learn.modernagecoders.com/best-coding-class-in-bhilai>
<https://learn.modernagecoders.com/best-coding-class-in-bhopal>
<https://learn.modernagecoders.com/best-coding-class-in-bhubaneswar>
<https://learn.modernagecoders.com/best-coding-class-in-bikaner>
<https://learn.modernagecoders.com/best-coding-class-in-bokaro>
<https://learn.modernagecoders.com/best-coding-class-in-chandigarh>
<https://learn.modernagecoders.com/best-coding-class-in-chennai>
<https://learn.modernagecoders.com/best-coding-class-in-coimbatore>
<https://learn.modernagecoders.com/best-coding-class-in-cuttack>
<https://learn.modernagecoders.com/best-coding-class-in-darbhanga>
<https://learn.modernagecoders.com/best-coding-class-in-davanagere>
<https://learn.modernagecoders.com/best-coding-class-in-dehradun>
<https://learn.modernagecoders.com/best-coding-class-in-delhi>
<https://learn.modernagecoders.com/best-coding-class-in-dhanbad>
<https://learn.modernagecoders.com/best-coding-class-in-durgapur>
<https://learn.modernagecoders.com/best-coding-class-in-erode>
<https://learn.modernagecoders.com/best-coding-class-in-faridabad>
<https://learn.modernagecoders.com/best-coding-class-in-gandhinagar>
<https://learn.modernagecoders.com/best-coding-class-in-gangtok>
<https://learn.modernagecoders.com/best-coding-class-in-gaya>
<https://learn.modernagecoders.com/best-coding-class-in-ghaziabad>
<https://learn.modernagecoders.com/best-coding-class-in-goa>
<https://learn.modernagecoders.com/best-coding-class-in-gorakhpur>
<https://learn.modernagecoders.com/best-coding-class-in-greater-noida>
<https://learn.modernagecoders.com/best-coding-class-in-guntur>
<https://learn.modernagecoders.com/best-coding-class-in-gurgaon>
<https://learn.modernagecoders.com/best-coding-class-in-guwahati>
<https://learn.modernagecoders.com/best-coding-class-in-gwalior>
<https://learn.modernagecoders.com/best-coding-class-in-haldwani>
<https://learn.modernagecoders.com/best-coding-class-in-haridwar>
<https://learn.modernagecoders.com/best-coding-class-in-hisar>
<https://learn.modernagecoders.com/best-coding-class-in-hubli>
<https://learn.modernagecoders.com/best-coding-class-in-hyderabad>
<https://learn.modernagecoders.com/best-coding-class-in-imphal>
<https://learn.modernagecoders.com/best-coding-class-in-indore>
<https://learn.modernagecoders.com/best-coding-class-in-jabalpur>
<https://learn.modernagecoders.com/best-coding-class-in-jaipur>
<https://learn.modernagecoders.com/best-coding-class-in-jalandhar>
<https://learn.modernagecoders.com/best-coding-class-in-jammu>
<https://learn.modernagecoders.com/best-coding-class-in-jamnagar>
<https://learn.modernagecoders.com/best-coding-class-in-jamshedpur>
<https://learn.modernagecoders.com/best-coding-class-in-jhansi>
<https://learn.modernagecoders.com/best-coding-class-in-jodhpur>
<https://learn.modernagecoders.com/best-coding-class-in-kakinada>
<https://learn.modernagecoders.com/best-coding-class-in-kannur>
<https://learn.modernagecoders.com/best-coding-class-in-kanpur>
<https://learn.modernagecoders.com/best-coding-class-in-karimnagar>
<https://learn.modernagecoders.com/best-coding-class-in-karnal>
<https://learn.modernagecoders.com/best-coding-class-in-kharagpur>
<https://learn.modernagecoders.com/best-coding-class-in-kochi>
<https://learn.modernagecoders.com/best-coding-class-in-kolhapur>
<https://learn.modernagecoders.com/best-coding-class-in-kolkata>
<https://learn.modernagecoders.com/best-coding-class-in-kollam>
<https://learn.modernagecoders.com/best-coding-class-in-kota>
<https://learn.modernagecoders.com/best-coding-class-in-kottayam>
<https://learn.modernagecoders.com/best-coding-class-in-kozhikode>
<https://learn.modernagecoders.com/best-coding-class-in-latur>

<https://learn.modernagecoders.com/best-coding-class-in-lucknow>
<https://learn.modernagecoders.com/best-coding-class-in-ludhiana>
<https://learn.modernagecoders.com/best-coding-class-in-madurai>
<https://learn.modernagecoders.com/best-coding-class-in-mangalore>
<https://learn.modernagecoders.com/best-coding-class-in-meerut>
<https://learn.modernagecoders.com/best-coding-class-in-mohali>
<https://learn.modernagecoders.com/best-coding-class-in-moradabad>
<https://learn.modernagecoders.com/best-coding-class-in-mumbai>
<https://learn.modernagecoders.com/best-coding-class-in-muzaffarpur>
<https://learn.modernagecoders.com/best-coding-class-in-mysore>
<https://learn.modernagecoders.com/best-coding-class-in-nagpur>
<https://learn.modernagecoders.com/best-coding-class-in-nashik>
<https://learn.modernagecoders.com/best-coding-class-in-navi-mumbai>
<https://learn.modernagecoders.com/best-coding-class-in-nellore>
<https://learn.modernagecoders.com/best-coding-class-in-nizamabad>
<https://learn.modernagecoders.com/best-coding-class-in-noida>
<https://learn.modernagecoders.com/best-coding-class-in-palakkad>
<https://learn.modernagecoders.com/best-coding-class-in-panipat>
<https://learn.modernagecoders.com/best-coding-class-in-patiala>
<https://learn.modernagecoders.com/best-coding-class-in-patna>
<https://learn.modernagecoders.com/best-coding-class-in-pimpri-chinchwad>
<https://learn.modernagecoders.com/best-coding-class-in-prayagraj>
<https://learn.modernagecoders.com/best-coding-class-in-pune>
<https://learn.modernagecoders.com/best-coding-class-in-raipur>
<https://learn.modernagecoders.com/best-coding-class-in-rajahmundry>
<https://learn.modernagecoders.com/best-coding-class-in-rajkot>
<https://learn.modernagecoders.com/best-coding-class-in-ranchi>
<https://learn.modernagecoders.com/best-coding-class-in-rishikesh>
<https://learn.modernagecoders.com/best-coding-class-in-rohtak>
<https://learn.modernagecoders.com/best-coding-class-in-roorkee>
<https://learn.modernagecoders.com/best-coding-class-in-rourkela>
<https://learn.modernagecoders.com/best-coding-class-in-salem>
<https://learn.modernagecoders.com/best-coding-class-in-sambalpur>
<https://learn.modernagecoders.com/best-coding-class-in-shimla>
<https://learn.modernagecoders.com/best-coding-class-in-shimoga>
<https://learn.modernagecoders.com/best-coding-class-in-sikar>
<https://learn.modernagecoders.com/best-coding-class-in-siliguri>
<https://learn.modernagecoders.com/best-coding-class-in-solapur>
<https://learn.modernagecoders.com/best-coding-class-in-sonapat>
<https://learn.modernagecoders.com/best-coding-class-in-srinagar>
<https://learn.modernagecoders.com/best-coding-class-in-surat>
<https://learn.modernagecoders.com/best-coding-class-in-thane>
<https://learn.modernagecoders.com/best-coding-class-in-thanjavur>
<https://learn.modernagecoders.com/best-coding-class-in-thiruvananthapuram>
<https://learn.modernagecoders.com/best-coding-class-in-thrissur>
<https://learn.modernagecoders.com/best-coding-class-in-tirunelveli>
<https://learn.modernagecoders.com/best-coding-class-in-tirupati>
<https://learn.modernagecoders.com/best-coding-class-in-trichy>
<https://learn.modernagecoders.com/best-coding-class-in-udaipur>
<https://learn.modernagecoders.com/best-coding-class-in-udupi>
<https://learn.modernagecoders.com/best-coding-class-in-ujjain>
<https://learn.modernagecoders.com/best-coding-class-in-vadodara>
<https://learn.modernagecoders.com/best-coding-class-in-varanasi>
<https://learn.modernagecoders.com/best-coding-class-in-vellore>
<https://learn.modernagecoders.com/best-coding-class-in-vijayawada>
<https://learn.modernagecoders.com/best-coding-class-in-visakhapatnam>
<https://learn.modernagecoders.com/best-coding-class-in-warangal>

AREAS IN AND AROUND KOLKATA (47 pages)

<https://learn.modernagecoders.com/coding-classes-near-ajc-bose-road-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-alipore-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-andul-howrah>
<https://learn.modernagecoders.com/coding-classes-near-baguiati-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-bally-howrah>
<https://learn.modernagecoders.com/coding-classes-near-ballygunge-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-baranagar-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-bhowanipore-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-camac-street-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-deshapriya-park-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-dhakuria-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-domjur-howrah>
<https://learn.modernagecoders.com/coding-classes-near-dum-dum-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-elgin-road-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-gariahat-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-golpark-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-hazra-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-howrah-city-howrah>
<https://learn.modernagecoders.com/coding-classes-near-jadavpur-kolkata>

<https://learn.modernagecoders.com/coding-classes-near-jodhpur-park-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-kadamtala-howrah>
<https://learn.modernagecoders.com/coding-classes-near-kalighat-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-kasba-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-kestopur-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-lake-gardens-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-lake-town-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-lansdowne-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-liluah-howrah>
<https://learn.modernagecoders.com/coding-classes-near-loudon-street-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-mandirtala-howrah>
<https://learn.modernagecoders.com/coding-classes-near-nager-bazar-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-new-alipore-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-new-town-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-park-circus-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-park-street-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-ramrajatala-howrah>
<https://learn.modernagecoders.com/coding-classes-near-rashbehari-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-salt-lake-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-santragachi-howrah>
<https://learn.modernagecoders.com/coding-classes-near-shakespeare-sarani-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-shibpur-howrah>
<https://learn.modernagecoders.com/coding-classes-near-shyambazar-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-southern-avenue-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-sreebhumi-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-tollygunge-kolkata>
<https://learn.modernagecoders.com/coding-classes-near-uluberia-howrah>
<https://learn.modernagecoders.com/coding-classes-near-vip-road-kolkata>

COUNTRY AND REGION PAGES (67 pages)

<https://learn.modernagecoders.com/coding-classes-in-abu-dhabi>
<https://learn.modernagecoders.com/coding-classes-in-andhra-pradesh>
<https://learn.modernagecoders.com/coding-classes-in-assam>
<https://learn.modernagecoders.com/coding-classes-in-australia>
<https://learn.modernagecoders.com/coding-classes-in-avani-riverside-howrah>
<https://learn.modernagecoders.com/coding-classes-in-avidipta-mukundapur>
<https://learn.modernagecoders.com/coding-classes-in-bihar>
<https://learn.modernagecoders.com/coding-classes-in-calcutta-riverside>
<https://learn.modernagecoders.com/coding-classes-in-canada>
<https://learn.modernagecoders.com/coding-classes-in-chhattisgarh>
<https://learn.modernagecoders.com/coding-classes-in-dammam>
<https://learn.modernagecoders.com/coding-classes-in-diamond-city-north>
<https://learn.modernagecoders.com/coding-classes-in-diamond-city-south>
<https://learn.modernagecoders.com/coding-classes-in-dlf-new-town-heights>
<https://learn.modernagecoders.com/coding-classes-in-doha>
<https://learn.modernagecoders.com/coding-classes-in-dubai>
<https://learn.modernagecoders.com/coding-classes-in-eden-city-maheshtala>
<https://learn.modernagecoders.com/coding-classes-in-elita-garden-vista>
<https://learn.modernagecoders.com/coding-classes-in-genexx-valley-joka>
<https://learn.modernagecoders.com/coding-classes-in-germany>
<https://learn.modernagecoders.com/coding-classes-in-goa>
<https://learn.modernagecoders.com/coding-classes-in-godrej-prakriti>
<https://learn.modernagecoders.com/coding-classes-in-greenfield-city-behala>
<https://learn.modernagecoders.com/coding-classes-in-gujarat>
<https://learn.modernagecoders.com/coding-classes-in-haryana>
<https://learn.modernagecoders.com/coding-classes-in-hiland-park>
<https://learn.modernagecoders.com/coding-classes-in-hiland-willows>
<https://learn.modernagecoders.com/coding-classes-in-himachal-pradesh>
<https://learn.modernagecoders.com/coding-classes-in-ideal-niketan-tangra>
<https://learn.modernagecoders.com/coding-classes-in-india>
<https://learn.modernagecoders.com/coding-classes-in-ireland>
<https://learn.modernagecoders.com/coding-classes-in-jeddah>
<https://learn.modernagecoders.com/coding-classes-in-jharkhand>
<https://learn.modernagecoders.com/coding-classes-in-karnataka>
<https://learn.modernagecoders.com/coding-classes-in-kerala>
<https://learn.modernagecoders.com/coding-classes-in-madhya-pradesh>
<https://learn.modernagecoders.com/coding-classes-in-maharashtra>
<https://learn.modernagecoders.com/coding-classes-in-natural-city-garia>
<https://learn.modernagecoders.com/coding-classes-in-new-zealand>
<https://learn.modernagecoders.com/coding-classes-in-odisha>
<https://learn.modernagecoders.com/coding-classes-in-punjab>
<https://learn.modernagecoders.com/coding-classes-in-qatar>
<https://learn.modernagecoders.com/coding-classes-in-rajasthan>
<https://learn.modernagecoders.com/coding-classes-in-riyadh>
<https://learn.modernagecoders.com/coding-classes-in-rosedale-garden>
<https://learn.modernagecoders.com/coding-classes-in-saudi-arabia>
<https://learn.modernagecoders.com/coding-classes-in-sharjah>
<https://learn.modernagecoders.com/coding-classes-in-sherwood-estate>

<https://learn.modernagecoders.com/coding-classes-in-siddha-town-madhyamgram>
<https://learn.modernagecoders.com/coding-classes-in-singapore>
<https://learn.modernagecoders.com/coding-classes-in-south-city>
<https://learn.modernagecoders.com/coding-classes-in-sugam-habitat>
<https://learn.modernagecoders.com/coding-classes-in-sukhobrishti>
<https://learn.modernagecoders.com/coding-classes-in-sweden>
<https://learn.modernagecoders.com/coding-classes-in-switzerland>
<https://learn.modernagecoders.com/coding-classes-in-tamil-nadu>
<https://learn.modernagecoders.com/coding-classes-in-telangana>
<https://learn.modernagecoders.com/coding-classes-in-uae>
<https://learn.modernagecoders.com/coding-classes-in-udayan-condoville-em-bypass>
<https://learn.modernagecoders.com/coding-classes-in-united-kingdom>
<https://learn.modernagecoders.com/coding-classes-in-united-states>
<https://learn.modernagecoders.com/coding-classes-in-uniworld-city>
<https://learn.modernagecoders.com/coding-classes-in-upohar>
<https://learn.modernagecoders.com/coding-classes-in-urbana>
<https://learn.modernagecoders.com/coding-classes-in-uttar-pradesh>
<https://learn.modernagecoders.com/coding-classes-in-uttarakhand>
<https://learn.modernagecoders.com/coding-classes-in-west-bengal>

SCHOOL PAGES (26 pages)

<https://learn.modernagecoders.com/coding-and-maths-for-birla-high-school-kolkata>
<https://learn.modernagecoders.com/coding-and-maths-for-birla-high-school-mukundapur-kolkata>
<https://learn.modernagecoders.com/coding-and-maths-for-don-bosco-park-circus-kolkata>
<https://learn.modernagecoders.com/coding-and-maths-for-heritage-school-kolkata>
<https://learn.modernagecoders.com/coding-and-maths-for-mahadevi-birla-world-academy-kolkata>
<https://learn.modernagecoders.com/coding-and-maths-for-sushila-birla-girls-school-kolkata>
<https://learn.modernagecoders.com/coding-classes-for-dps-bangalore>
<https://learn.modernagecoders.com/coding-classes-for-dps-dwarka-new-delhi>
<https://learn.modernagecoders.com/coding-classes-for-dps-greater-noida>
<https://learn.modernagecoders.com/coding-classes-for-dps-gurgaon>
<https://learn.modernagecoders.com/coding-classes-for-dps-hyderabad>
<https://learn.modernagecoders.com/coding-classes-for-dps-international-saket-delhi>
<https://learn.modernagecoders.com/coding-classes-for-dps-navi-mumbai>
<https://learn.modernagecoders.com/coding-classes-for-dps-new-town-kolkata>
<https://learn.modernagecoders.com/coding-classes-for-dps-noida>
<https://learn.modernagecoders.com/coding-classes-for-dps-rk-puram-new-delhi>
<https://learn.modernagecoders.com/coding-classes-for-dps-rohini-new-delhi>
<https://learn.modernagecoders.com/coding-classes-for-dps-ruby-park-kolkata>
<https://learn.modernagecoders.com/coding-classes-for-dps-vasant-kunj-new-delhi>
<https://learn.modernagecoders.com/coding-classes-for-girls>
<https://learn.modernagecoders.com/coding-classes-for-kids-cost>
<https://learn.modernagecoders.com/coding-classes-for-kids-in-doha>
<https://learn.modernagecoders.com/coding-classes-for-kids-in-dubai>
<https://learn.modernagecoders.com/coding-classes-for-kids-in-riyadh>
<https://learn.modernagecoders.com/coding-classes-for-la-martiniere-boys-kolkata>
<https://learn.modernagecoders.com/coding-classes-for-la-martiniere-girls-kolkata>

PAGES BY THE CHILD'S AGE (19 pages)

<https://learn.modernagecoders.com/coding-for-10-year-olds>
<https://learn.modernagecoders.com/coding-for-11-year-olds>
<https://learn.modernagecoders.com/coding-for-12-year-olds>
<https://learn.modernagecoders.com/coding-for-13-year-olds>
<https://learn.modernagecoders.com/coding-for-14-year-olds>
<https://learn.modernagecoders.com/coding-for-15-year-olds>
<https://learn.modernagecoders.com/coding-for-16-year-olds>
<https://learn.modernagecoders.com/coding-for-17-year-olds>
<https://learn.modernagecoders.com/coding-for-8-year-olds>
<https://learn.modernagecoders.com/coding-for-9-year-olds>
<https://learn.modernagecoders.com/python-for-10-year-olds>
<https://learn.modernagecoders.com/python-for-11-year-olds>
<https://learn.modernagecoders.com/python-for-12-year-olds>
<https://learn.modernagecoders.com/python-for-13-year-olds>
<https://learn.modernagecoders.com/python-for-14-year-olds>
<https://learn.modernagecoders.com/python-for-15-year-olds>
<https://learn.modernagecoders.com/python-for-16-year-olds>
<https://learn.modernagecoders.com/python-for-17-year-olds>
<https://learn.modernagecoders.com/python-for-9-year-olds>

PAGES BY SCHOOL CLASS (17 pages)

<https://learn.modernagecoders.com/coding-for-class-10>
<https://learn.modernagecoders.com/coding-for-class-11>
<https://learn.modernagecoders.com/coding-for-class-12>
<https://learn.modernagecoders.com/coding-for-class-3>
<https://learn.modernagecoders.com/coding-for-class-4>
<https://learn.modernagecoders.com/coding-for-class-5>

<https://learn.modernagecoders.com/coding-for-class-6>
<https://learn.modernagecoders.com/coding-for-class-7>
<https://learn.modernagecoders.com/coding-for-class-8>
<https://learn.modernagecoders.com/coding-for-class-9>
<https://learn.modernagecoders.com/python-for-class-10>
<https://learn.modernagecoders.com/python-for-class-11-cbse>
<https://learn.modernagecoders.com/python-for-class-12-cbse>
<https://learn.modernagecoders.com/python-for-class-6>
<https://learn.modernagecoders.com/python-for-class-7>
<https://learn.modernagecoders.com/python-for-class-8>
<https://learn.modernagecoders.com/python-for-class-9>

BLOG ARTICLES (116)

<https://learn.modernagecoders.com/blog/30-best-java-programs-for-class-8-students>
<https://learn.modernagecoders.com/blog/30-plus-scratch-project-ideas-kids-fun-coding-beginner-advanced>
<https://learn.modernagecoders.com/blog/50-java-programs-for-icse-class-10>
<https://learn.modernagecoders.com/blog/abacus-vs-vedic-vs-mental-maths>
<https://learn.modernagecoders.com/blog/advantages-functions-python-why-every-coder-should-use-them>
<https://learn.modernagecoders.com/blog/agentic-terminology-in-ai>
<https://learn.modernagecoders.com/blog/ai-automation-transforming-small-medium-businesses>
<https://learn.modernagecoders.com/blog/ap-computer-science-a-changes-2026>
<https://learn.modernagecoders.com/blog/app-idea-to-app-store-60-days-teen-developer-roadmap>
<https://learn.modernagecoders.com/blog/armstrong-number-python-complete-guide-examples-code>
<https://learn.modernagecoders.com/blog/best-ai-coding-tools-for-students-2026>
<https://learn.modernagecoders.com/blog/best-byjus-alternative>
<https://learn.modernagecoders.com/blog/best-coding-languages-kids-10-18-beginner-advanced>
<https://learn.modernagecoders.com/blog/best-free-paid-coding-courses-students>
<https://learn.modernagecoders.com/blog/best-free-platforms-for-vibe-coding>
<https://learn.modernagecoders.com/blog/best-java-ides-for-beginners>
<https://learn.modernagecoders.com/blog/best-programming-languages-college-students-2026>
<https://learn.modernagecoders.com/blog/career-transition-guide-moving-non-tech-to-tech-roles>
<https://learn.modernagecoders.com/blog/cbse-icse-gap-what-schools-not-teaching-about-tech>
<https://learn.modernagecoders.com/blog/cbse-maths-tuition-online>
<https://learn.modernagecoders.com/blog/cloud-computing-explained-business-owners-without-tech-jargon>
<https://learn.modernagecoders.com/blog/coding-class-fees-india-2026>
<https://learn.modernagecoders.com/blog/coding-for-5-year-olds-young-kids-start-learning-programming>
<https://learn.modernagecoders.com/blog/coding-for-6-year-olds>
<https://learn.modernagecoders.com/blog/coding-for-7-year-olds-what-how-kids-should-learn-programming>
<https://learn.modernagecoders.com/blog/coding-for-kids-guide>
<https://learn.modernagecoders.com/blog/coding-for-kids-parents-complete-guide-2026>
<https://learn.modernagecoders.com/blog/coding-is-the-new-english-why-2026-is-the-year-to-start>
<https://learn.modernagecoders.com/blog/coding-mathematics-engineering-science>
<https://learn.modernagecoders.com/blog/coding-vs-ai-what-should-my-child-learn>
<https://learn.modernagecoders.com/blog/common-myths-about-coding-for-school-students>
<https://learn.modernagecoders.com/blog/competitive-coding-vs-development-skills-what-students-focus>
<https://learn.modernagecoders.com/blog/custom-software-vs-ready-made-software>
<https://learn.modernagecoders.com/blog/data-analytics-math-help-businesses-better-decisions>
<https://learn.modernagecoders.com/blog/data-science-ai-math-skills-professionals>
<https://learn.modernagecoders.com/blog/data-science-course-india>
<https://learn.modernagecoders.com/blog/does-online-maths-tuition-work>
<https://learn.modernagecoders.com/blog/exception-handling-in-python>
<https://learn.modernagecoders.com/blog/fibonacci-series-in-python>
<https://learn.modernagecoders.com/blog/file-built-in-methods-python-guide>
<https://learn.modernagecoders.com/blog/file-organization-in-python>
<https://learn.modernagecoders.com/blog/file-types-in-python-complete-guide>
<https://learn.modernagecoders.com/blog/full-stack-vs-ai-engineer-salary-india>
<https://learn.modernagecoders.com/blog/group-vs-1-on-1-coding-classes>
<https://learn.modernagecoders.com/blog/help-child-with-maths-anxiety>
<https://learn.modernagecoders.com/blog/help-child-with-maths-homework>
<https://learn.modernagecoders.com/blog/how-coding-education-changing-future-india>
<https://learn.modernagecoders.com/blog/how-coding-improves-mathematical-thinking-children>
<https://learn.modernagecoders.com/blog/how-college-students-build-real-world-projects-while-studying>
<https://learn.modernagecoders.com/blog/how-schools-parents-prepare-children-future-tech-careers>
<https://learn.modernagecoders.com/blog/how-to-build-menu-driven-program-in-python>
<https://learn.modernagecoders.com/blog/how-to-choose-a-coding-class-for-your-child>
<https://learn.modernagecoders.com/blog/how-to-find-hcf-and-lcm-in-python>
<https://learn.modernagecoders.com/blog/how-to-get-better-at-maths>
<https://learn.modernagecoders.com/blog/how-to-make-maths-fun-for-kids>
<https://learn.modernagecoders.com/blog/how-to-read-and-write-files-in-python>
<https://learn.modernagecoders.com/blog/how-to-reverse-a-string-in-python>
<https://learn.modernagecoders.com/blog/how-to-teach-kids-ai-at-home>
<https://learn.modernagecoders.com/blog/how-to-write-a-leap-year-program-in-python>
<https://learn.modernagecoders.com/blog/html-css-javascript-project-ideas>
<https://learn.modernagecoders.com/blog/icse-class-10-computer-applications-revision>
<https://learn.modernagecoders.com/blog/igcse-computer-science-explained-for-parents>
<https://learn.modernagecoders.com/blog/is-coding-good-for-kids>
<https://learn.modernagecoders.com/blog/is-coding-worth-learning-2026>
<https://learn.modernagecoders.com/blog/is-vedic-maths-actually-useful>

<https://learn.modernagecoders.com/blog/java-array-programs-for-beginners>
<https://learn.modernagecoders.com/blog/java-constructors-explained>
<https://learn.modernagecoders.com/blog/java-for-kids-complete-guide-learning-programming-2026>
<https://learn.modernagecoders.com/blog/javascript-basics>
<https://learn.modernagecoders.com/blog/learn-python-step-by-step>
<https://learn.modernagecoders.com/blog/learn-to-code-career-change-2026>
<https://learn.modernagecoders.com/blog/mental-maths-tricks-for-kids>
<https://learn.modernagecoders.com/blog/minecraft-vs-roblox-which-teaches-kids-to-code>
<https://learn.modernagecoders.com/blog/mobile-apps-increase-customer-engagement-businesses>
<https://learn.modernagecoders.com/blog/modern-age-coders-certificate-earned-not-given>
<https://learn.modernagecoders.com/blog/outsourcing-software-development-vs-hiring-inhouse-developers>
<https://learn.modernagecoders.com/blog/parents-guide-starting-coding-education-home>
<https://learn.modernagecoders.com/blog/project-based-learning-why-theory-alone-not-enough-coding>
<https://learn.modernagecoders.com/blog/python-basic-programs-for-beginners>
<https://learn.modernagecoders.com/blog/python-dictionary-complete-guide>
<https://learn.modernagecoders.com/blog/python-for-beginners>
<https://learn.modernagecoders.com/blog/python-lambda-functions>
<https://learn.modernagecoders.com/blog/python-list-comprehension-explained>
<https://learn.modernagecoders.com/blog/python-oop-tutorial-for-beginners>
<https://learn.modernagecoders.com/blog/python-programs-for-cbse-class-12>
<https://learn.modernagecoders.com/blog/python-vs-javascript-which-to-learn-first>
<https://learn.modernagecoders.com/blog/robotics-coding-stem-kits-build-logical-thinking>
<https://learn.modernagecoders.com/blog/role-of-mathematics-programming-logical-problem-solving>
<https://learn.modernagecoders.com/blog/safe-ai-tools-for-kids-homework-2025>
<https://learn.modernagecoders.com/blog/scratch-vs-python-which-better-young-learners>
<https://learn.modernagecoders.com/blog/signs-your-child-is-ready-to-learn-coding>
<https://learn.modernagecoders.com/blog/star-pattern-programs-in-python>
<https://learn.modernagecoders.com/blog/stem-education-for-girls-india>
<https://learn.modernagecoders.com/blog/string-handling-in-java>
<https://learn.modernagecoders.com/blog/top-10-java-programs-every-college-student-must-know>
<https://learn.modernagecoders.com/blog/top-10-java-programs-for-school-students>
<https://learn.modernagecoders.com/blog/top-20-java-programs-for-icse-class-10>
<https://learn.modernagecoders.com/blog/top-coding-games-platforms-make-learning-fun-kids>
<https://learn.modernagecoders.com/blog/top-emerging-tech-skills-learn-2025-ai-low-code-cloud>
<https://learn.modernagecoders.com/blog/uae-ai-curriculum-schools-what-it-means>
<https://learn.modernagecoders.com/blog/vibe-coding-vs-learning-to-code-properly>
<https://learn.modernagecoders.com/blog/what-is-ai-complete-beginners-guide-how-to-start>
<https://learn.modernagecoders.com/blog/what-is-blockly-for-kids>
<https://learn.modernagecoders.com/blog/what-is-floor-division-in-python>
<https://learn.modernagecoders.com/blog/what-is-the-responsibility-of-developers-using-generative-AI>
<https://learn.modernagecoders.com/blog/what-is-vibe-coding-future-of-software-development>
<https://learn.modernagecoders.com/blog/what-type-of-data-is-generative-ai-most-suitable-for>
<https://learn.modernagecoders.com/blog/whitehat-jr-alternative>
<https://learn.modernagecoders.com/blog/who-are-modern-age-coders-why-choose-us>
<https://learn.modernagecoders.com/blog/why-coding-is-new-literacy-kids-2025>
<https://learn.modernagecoders.com/blog/why-early-exposure-coding-gives-students-competitive-advantage>
<https://learn.modernagecoders.com/blog/why-every-business-needs-website-mobile-app-2025>
<https://learn.modernagecoders.com/blog/why-every-teen-should-learn-coding>
<https://learn.modernagecoders.com/blog/why-is-my-child-struggling-with-maths>
<https://learn.modernagecoders.com/blog/why-python-most-powerful-skill-professionals-today>
<https://learn.modernagecoders.com/blog/working-professionals-upskill-coding-without-quitting-job>

=====

PART 10. GENERAL QUESTIONS AND THE ANSWERS TO GIVE

=====

Q: What is Modern Age Coders?

A: We teach live online coding and mathematics to learners aged 6 to 67. Kids, teens, college students, working professionals and corporate teams. We also build software for businesses, which is how our advanced students get real project experience.

Q: Does my child need any experience?

A: No. Our beginner courses start from the very beginning and most of our students start with zero experience.

Q: Are the classes live or recorded?

A: Every class is live with a real teacher. Recordings are provided as well, so a missed class can be caught up.

Q: What does my child need at home?

A: A computer, Windows, Mac or Chromebook, and steady internet. Most courses run in the browser so nothing needs installing. A webcam and microphone help, and for maths a notebook and pen.

Q: How many classes a week?

A: Two live classes a week, one hour each, which is eight classes a month.

Q: How big are the batches?

A: Group classes are capped at 10 students. A mini batch is just 3 to 4. One to one is a

single student with their own mentor.

Q: How long does a course take?

A: Beginner courses usually run 3 to 6 months. Advanced ones run 6 to 9 months. Custom programmes are flexible.

Q: What ages do you teach?

A: From 6 to 67. Kids are 6 to 12, teens are 13 to 18, and then college students and adults.

Q: Is there a free class?

A: Yes. One free demo class per student, with a real teacher, and we never ask for card details. Parents are welcome to sit in.

Q: What if we miss a class?

A: Every class is recorded, so it can be watched later. For one to one classes we can often reschedule if you tell us in advance.

Q: Can we pause?

A: Yes, for exams or holidays. Tell us about a week ahead and we hold the spot.

Q: Can we switch from group to one to one later?

A: Yes, any time. Tell us a few days before the next billing date and we adjust it.

Q: Do you give a certificate?

A: Yes, when the course is completed. It lists the skills learned and the projects built.

Q: Can parents see progress?

A: Yes. Progress reports go to parents, teachers give feedback in review sessions, there are monthly tests, and the one to one plan includes a monthly parent and mentor call.

Q: What if my child finds a topic hard?

A: We add extra doubt clearing sessions and one to one help. No student is left behind.

Q: Which boards do you cover?

A: CBSE, ICSE, IGCSE, IB, A Level and GCSE, American Common Core and AP, Singapore MOE, and most major Indian state boards.

Q: Do you prepare for Olympiads and competitive exams?

A: Yes. Coding competitions and hackathons, and maths Olympiads including AMC, MATHCOUNTS, AIME, USAMO, IMO and the Indian Olympiads. Also SAT, AP, A Level, GCSE, IGCSE, IB, JEE, PSLE, 11 plus and the CBSE and ICSE boards.

Q: Are there girls only batches?

A: Yes. Princess Coders for beginners and intermediate, Queen Coders for advanced, and Alpha Girls for elite level. There is also a comeback programme for women returning to tech.

Q: Do you teach AI?

A: Yes. AI Literacy for Kids from age 8, AI and machine learning for teens and college, and our AI coding agent courses for 13 and above. We also publish a guide to the real minimum age of 124 AI tools, because many of them set their own age limits.

Q: Can college students and working adults join?

A: Yes. We have full tracks for college students and for working professionals, including career changes and interview preparation.

Q: Do you do corporate training?

A: Yes, for company teams. The curriculum is built around what the team actually needs. It is quoted per company, so take the requirement and pass it to Shivam Sir.

Q: Where is your office? Can we visit?

A: We are an online school, so every class happens live over video and there is no campus to visit. Our registered office is 1B, JK Ambika Tower, Kolkata, West Bengal 700002.

Q: Are your teachers qualified?

A: Yes. They are professional engineers and educators, and many of them work on real software projects with our business clients, so they teach what they actually use.

Q: Is my child too young at 5?

A: Our programmes start at 6. For a 5 year old, ask Shivam Sir on +91 9123366161. Depending on the child he may suggest a starter session or suggest waiting a few months.

Q: I am 50 or 60. Am I too old?

A: Not at all. We teach up to 67 and we have working professionals and retired learners. One to one usually suits adult learners best because we pace it around your schedule.

Q: Do you help with internships or real projects?

A: Yes. Advanced students get the chance to work on real client projects.

Q: Can I gift a course?

A: Yes. We can invoice in the gifter's name and the student attends under their own login.
Ask Shivam Sir to set it up.

Q: Do you have a mobile app?

A: Classes are joined from a computer through the browser, so there is nothing to install.
The student dashboard is at dashboard.modernagecoders.com.

Q: Can we join in the middle of the month?

A: Yes. We either place the student in a batch already running or start a new one.

PART 11. READY MADE REPLIES

FIRST MESSAGE

"Hello, welcome to Modern Age Coders. I can help you with our live coding and maths classes.
Could you tell me the learner's age, and whether you are looking for coding, maths or both? I
will point you to the right starting place."

WHEN THEY ASK THE PRICE. Give only their own region, never both.

If they are in India:

"Our classes are Rs 1,499 a month for a group class, Rs 2,999 for a mini batch of just 3 to
4 students, and Rs 4,999 a month for fully one to one. That includes two live one hour
classes every week, recordings, doubt support and a certificate. The first class is free,
would you like me to book it?"

If they are outside India:

"Our classes are \$100 a month for a group class and \$150 a month for fully one to one. That
includes two live one hour classes every week, recordings, doubt support and a certificate.
The first class is free, would you like me to book it?"

For maths in India the one to one rate is Rs 4,999 a month. Everything else is the same.

WHEN THEY ASK ABOUT TIMINGS

"We run batches on weekday evenings and on weekend mornings and afternoons, and we have slots
for other time zones too. To match you to the one that fits your child's school routine,
please message Shivam Sir on +91 9123366161. He arranges every batch himself."

WHEN THEY ASK "IS IT WORTH IT?"

"The honest answer is to try the free class. No card, no commitment, one real lesson with an
actual teacher. You will know within the hour whether it suits your child, and the teacher
will tell you honestly where they should start. Shall I book it? I just need the student's
name, age and your WhatsApp number."

WHEN THEY ASK FOR A DISCOUNT

"Let me check the current offers with Shivam Sir. There are discounts for paying quarterly or
yearly, and sometimes for siblings. Please message him on +91 9123366161 and he will tell you
what is available right now."

WHEN THE COST WORRIES THEM

"I understand. Two things may help. The demo class is completely free and needs no card, so
you can see the teaching before spending anything. And there is no lock in, no registration
fee and no long package. You can stop any month. Would you like to try the free class first?"

CLOSING THE CONVERSATION

"Shall I book your free class? I just need three things: the student's name and age, whether
you want coding or maths, and your WhatsApp number with country code. Once I have those,
Shivam Sir will confirm the time."

WHEN YOU DO NOT KNOW THE ANSWER

"That is a good question and I want to give you the right answer rather than guess. Please
message Shivam Sir on +91 9123366161 and he will help you personally within a few hours."

END OF KNOWLEDGE BASE. WHEN IN DOUBT, SEND THEM TO SHIVAM SIR ON +91 9123366161.